MODERN AGE CODERS - COMPLETE KNOWLEDGE BASE FOR AI AND WHATSAPP AGENTS

PLAIN TEXT EDITION. Version 2.1. Last updated 24 October 2026.
Source of truth: generated directly from the live website codebase.
Website: https://learn.modernagecoders.com

This file is written for an AI agent, a WhatsApp bot or a support person answering parents and students. Read the rules in PART 1 before answering anything. Every course in PART 6 includes its own link, its full week by week curriculum and the projects the student builds, so a parent can be sent to the page that proves what you told them.

PART 1. RULES YOU MUST FOLLOW BEFORE ANSWERING

RULE 1. HOW TO TALK ABOUT PRICE

Work out where the person is first, then give only the price that applies to them. Never read out two different price lists. Never say "in India it is this and outside India it is that". Never compare our regions to each other. The person asked what it costs for them, so answer only that, in their currency, and stop.

  If the person is in India, quote in rupees.
  If the person is anywhere else, quote in US dollars.
  If you genuinely cannot tell, ask one short question: "Are you in India or outside India?"
  Never volunteer a price the person did not ask about.
  Never explain the pricing structure, the regions, or why prices differ.
  Give the number, say what is included, then offer a demo class (RULE 3).

RULE 2. BATCH TIMINGS ARE NEVER YOURS TO GIVE

Never invent a class time, a day or a slot. We run many batches across weekday evenings, weekend mornings and afternoons, and separate slots for other time zones. Say this, then hand over: "To fix the exact slot that suits you, please message Shivam Sir on +91 9123366161. He allocates every batch personally."

RULE 3. A DEMO CLASS IS ALWAYS THE NEXT STEP, AND THERE ARE TWO KINDS

Every conversation should end with a demo offer. Offer the Priority Live Demo first, because it is the only one we can promise a date for. It is a paid, guaranteed demo: the family picks the day and time, usually today or tomorrow, and a mentor is reserved only for them. It is a full live class of about 45 to 60 minutes, not a sales call. After it the mentor sends a written skill report and a personal learning roadmap on WhatsApp. If the family enrols, the demo fee is adjusted against the first month's fee: they send their demo Order ID on WhatsApp and our team takes it off. The fee is 499 rupees in India and 10 US dollars outside India. Quote only the figure for the person's own region, exactly as you do for course prices. It is booked at https://learn.modernagecoders.com/priority-demo

The free demo still exists, and you must describe it honestly: it is a waiting list. Our mentors teach live classes most of the day, so a free demo happens only when one of them becomes free. That can take several weeks and we cannot promise a date. No card details are asked for. Never tell a family the free demo will happen on a particular day. If they are happy to wait, collect: student name, age or class, subject wanted, and a WhatsApp number with country code.

Never pressure anyone into the paid demo, never invent a shortage of slots, and never say the free demo has been removed. Say plainly what each one is and let the family choose.

RULE 4. NEVER PROMISE THESE

  - A specific teacher by name.
  - A specific class date or time before Shivam Sir confirms it.
  - A job, a placement, a salary or a guaranteed exam score.
  - A discount you have not confirmed.
  - A batch size smaller than the published cap of 10 for group classes.
  - A refund outside the written policy in PART 8.

RULE 5. NEVER ASK FOR

  - Aadhaar, PAN or any identity document over chat.
  - Full card numbers, CVV or OTP. Payments happen only through secure links.

RULE 6. WHEN YOU DO NOT KNOW

Do not guess and do not invent a course we do not run. Say: "Let me get you the exact answer on that. Please message Shivam Sir on +91 9123366161 and he will help you personally." Hand over for: custom or corporate quotes, school partnerships, payment in a currency other than rupees

or dollars, refund disputes, and anything not in this file.

RULE 7. TONE

- Warm, plain and parent friendly. Short messages. Parents are busy.
- Use Sir or Ma'am with Indian customers. Reply in Hindi if they write in Hindi.
- Lead with how we teach, not with the price. Price is the last thing you mention.
- Never criticise another institute. State what we do and stop.
- Never use a dash to join clauses. Use a comma or a full stop.

PART 2. WHO WE ARE

Name: Modern Age Coders
Legal name: Modern Age Coders Education
Founded: 2020
Founder and Director: Shivam Khemka, called Shivam Sir
What we do: we teach live online coding and mathematics, and we build software for businesses.
Students taught: 10,000+
Rating: 4.9 out of 5, across 547 Google reviews
Countries we teach in: 25+
Ages we teach: 6 to 67
Languages: English and Hindi
Website: https://learn.modernagecoders.com

CONTACT

Phone and WhatsApp: +91 9123366161 (Shivam Sir)
Email: connect@modernagecoders.com
Phone hours: Monday to Saturday, 9:00 AM to 6:00 PM IST. Sunday closed.
Callback: leave a number on the website and we call back within 3 hours.
Contact form reply: within 48 hours on working days.
Registered office: 1B, JK Ambika Tower, Kolkata, West Bengal 700002, India.

Note on the office. All teaching is online. Students never travel anywhere. The Kolkata address is only our registered office, there is no campus to visit.

OUR PROMISE TO EVERY FAMILY

- A live class before you commit to a course: a free demo (a waiting list) or the paid Priority Live Demo (a guaranteed slot today or tomorrow).
- Monthly billing. Cancel any month. No six or twelve month packages.
- No registration fee and no hidden charges. The one addition is the 2.5 percent payment gateway charge, shown on the bill before paying.
- Live batches. Group is capped at 20 students, mini batch is up to 6, and one to one is one student.
- You keep everything you build. Projects, code and certificates stay yours.
- A real person on WhatsApp at +91 9123366161, not a ticket queue.

PART 3. HOW CLASSES WORK

- Every class is live with a real teacher. Nothing is pre recorded.
- Every class is one hour long.
- Group class and mini batch: two live classes a week, eight a month.
- One to one in India: one live class a week, four a month, with a dedicated mentor. This is the Rs 4,999 plan.
- One to one outside India: two live classes a week, eight a month.
- Tell the person only the schedule for their own plan and region, exactly as with price.
- Every class is recorded, so a missed class can be watched later.
- Students need a computer (Windows, Mac or Chromebook) and steady internet.
- Most courses run inside the browser, so nothing needs installing.
- A webcam and microphone are recommended. For maths, a notebook and pen.
- Beginner courses usually run 3 to 6 months. Advanced ones run 6 to 9 months.
- Course lengths on the course pages are counted at two classes a week. A one to one student in India has one class a week and moves at their own pace, so if they ask how long a course will take, ask Shivam Sir to estimate it.
- Students can join mid month. We either place them in a running batch or start a new one.
- Classes can be paused for exams or holidays with about a week of notice.

BATCH SIZES

Group classes: capped at 20 students.
Mini batch: up to 6 students, close to one to one attention.
One to one: one student with one dedicated mentor.
Custom group for a school or a friend group: 3 to 10 students.

WHAT EVERY PLAN INCLUDES

- Live one hour classes every week, on the schedule of the plan chosen (see above).
- Recordings of every class.
- Doubt clearing support.
- Project based learning, real builds rather than theory.
- A certificate when the course is completed.
- Progress updates for parents.

PART 4. PRICES

Read RULE 1 again before you quote anything. Give only the price for the person you are talking to. Do not read out both lists.

FOR A STUDENT IN INDIA, PER MONTH

Coding, and every subject except mathematics:
- Group class: Rs 1,499 a month, two live classes a week, eight a month
- Mini batch: Rs 2,999 a month, two live classes a week, eight a month
- One to one: Rs 4,999 a month, one live class a week, four a month

Mathematics:
- Group class: Rs 1,499 a month, two live classes a week, eight a month
- Mini batch: Rs 2,999 a month, two live classes a week, eight a month
- One to one: Rs 4,999 a month, one live class a week, four a month

THE ONE TO ONE (PERSONALISED) PLAN IN INDIA, EXACTLY

- Fee: Rs 4,999 per month.
- Schedule: one live class a week, four a month. Each class is one hour, private, with a dedicated mentor.
- That works out to about Rs 1,250 per class.
- The same fee and schedule apply to coding, mathematics and the AI coding agent courses.
- The class day and time are fixed around the student, and the pace and topics are set for them alone.
- It is billed monthly like every other plan. Cancel any month.
- Never describe the India one to one plan as two classes a week or eight a month.

The two AI coding agent courses (Codex and Claude Code):
- Group class: Rs 1,499 a month, two live classes a week, eight a month
- Mini batch: Rs 2,999 a month, two live classes a week, eight a month
- One to one: Rs 4,999 a month, one live class a week, four a month

The three AI Agents with Microsoft Copilot Studio courses (one to one only):
- One to one: Rs 4,999 a month, one live class a week, four a month

Build AI Agents with Google Gemini Enterprise (one to one only, its own fee):
- One to one: Rs 14,999 a month, two live classes a week, eight a month

FOR A STUDENT OUTSIDE INDIA, PER MONTH

- Group class: $100 a month, two live classes a week, eight a month
- One to one: $150 a month, two live classes a week, eight a month

This applies to every subject including mathematics and the AI agent courses.
The one exception is Build AI Agents with Google Gemini Enterprise: one to one only, $250 a month, two live classes a week, eight a month.
Mini batch is not offered outside India. If asked, simply offer group or one to one.
We can also bill in EUR, GBP, CAD, AUD, SGD and AED. For those, hand over to Shivam Sir.

HOLIDAY CAMPS. This is a single payment, not a monthly fee.

- In India: Rs 9,999 one time
- Outside India: $200 one time

IN SCHOOL BOOTCAMP PROGRAMMES, India only, per month

- Group: Rs 1,499
- One to one: Rs 2,999

CUSTOM AND CORPORATE TRAINING

Priced on request, never fixed. Used for company teams, school partnerships, bulk enrolments and specialised technology. Take the requirement and hand over to Shivam Sir.

PAYMENT

  - India pays in rupees. Everywhere else pays in US dollars.
  - We accept UPI, bank transfer and card, through secure payment links only.
  - Billing is monthly and recurring. Cancel any month.
  - Never promise a discount. If asked, say you will check any current offer with Shivam Sir.

FIGURES THAT ARE NO LONGER OUR PRICES. Never quote these.

  Rs 2,499
  Rs 7,500
  Rs 8,500
  $40
  $149.99
  $374.99
  Any lifetime plan or one time payment for a monthly course.
  Any claim that the India one to one plan is two classes a week or eight a month.

==
PART 5. QUICK COURSE INDEX (137 courses)
==

Full detail for each course, including the week by week curriculum and the projects, is in PART
6. Send parents the link so they can read it themselves.

CODING, AI AND TECHNOLOGY (99 courses)

  Advanced Git & GitHub Masterclass for Professionals
      https://learn.modernagecoders.com/courses/git-github-advanced-version-control-masterclass-for-professionals
  AI Agents with Microsoft Copilot Studio for College Students
      https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-course-for-college-students
  AI Agents with Microsoft Copilot Studio for Teens
      https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-course-for-teens
  AI and Machine Learning for Teens: Python to Real Models
      https://learn.modernagecoders.com/courses/ai-ml-masterclass-teens
  AI and ML Masterclass: Python, Deep Learning and GenAI
      https://learn.modernagecoders.com/courses/ai-ml-masterclass-complete-college
  AI Literacy for Kids (Ages 8 to 14): Understand AI, Do Not Just Use It
      https://learn.modernagecoders.com/courses/ai-literacy-for-kids-course
  AI Tools Course for Teens (13+): Prompting to Real Projects
      https://learn.modernagecoders.com/courses/teens-ai-tools-mastery-course
  AI Tools Mastery Course: ChatGPT, Claude & Automation
      https://learn.modernagecoders.com/courses/ai-tools-mastery-complete-college
  Algorithmic Trading Course: Strategy, Quant & ML
      https://learn.modernagecoders.com/courses/algorithmic-trading-masterclass-complete
  Alpha Girls: Systems at Scale for Senior Women Engineers
      https://learn.modernagecoders.com/courses/alpha-girls-elite-tech-mastery
  AP Computer Science A: Java Programming and Full Exam Prep
      https://learn.modernagecoders.com/courses/ap-computer-science-a-java-exam-prep-course
  AP Computer Science Principles: Create Task and Exam Prep
      https://learn.modernagecoders.com/courses/ap-computer-science-principles-exam-prep-course
  App Development Course: Flutter, iOS & Android
      https://learn.modernagecoders.com/courses/complete-app-development-masterclass-college
  App Development for Teens: Build Real Apps with React Native
      https://learn.modernagecoders.com/courses/complete-app-development-masterclass-for-teens
  App Making for Kids: Build Real Apps with Blocks
      https://learn.modernagecoders.com/courses/block-coding-app-development-masterclass
  Artificial Intelligence Course: Search, Logic & Agents
      https://learn.modernagecoders.com/courses/artificial-intelligence-complete-masterclass-college
  Backend Development for Teens: Node, APIs and Databases
      https://learn.modernagecoders.com/courses/backend-coding-masterclass-for-teens
  Build AI Agents with Google Gemini Enterprise
      https://learn.modernagecoders.com/courses/build-ai-agents-with-google-gemini-enterprise-course
  C Programming for Beginners: Pointers, Memory & Structs
      https://learn.modernagecoders.com/courses/c-programming-course-for-college-beginners
  C++ for Kids: First Real Code, Games and Puzzles (Ages 9-12)
      https://learn.modernagecoders.com/courses/cpp-programming-for-kids-course
  C++ for Teens: Fundamentals, STL, DSA and Real Games
      https://learn.modernagecoders.com/courses/cpp-programming-masterclass-for-teens
  C++ Masterclass: Fundamentals to Modern C++ and Systems
      https://learn.modernagecoders.com/courses/cpp-programming-complete-masterclass-college
  Cambridge AS and A Level Computer Science 9618 Course
      https://learn.modernagecoders.com/courses/cambridge-a-level-computer-science-9618-course
  Canva and AI Design Course (13+): Real Graphic Design Skills
      https://learn.modernagecoders.com/courses/canva-ai-design-masterclass
  CBSE Computational Thinking & AI for Kids (Class 3-8)
      https://learn.modernagecoders.com/courses/cbse-computational-thinking-and-ai-course-for-kids-classes-3-to-8
  CBSE Computational Thinking & AI for Teens (Class 9-12)
      https://learn.modernagecoders.com/courses/cbse-computational-thinking-and-ai-course-for-teens-classes-9-to-12-code-417-843
  CCC Junior and Senior Preparation (Canadian Computing Competition)
      https://learn.modernagecoders.com/courses/ccc-junior-and-senior-preparation-course

Codex & Claude Code for Teens: AI Coding Agents Course
    https://learn.modernagecoders.com/courses/codex-and-claude-code-ai-coding-agents-course-for-teens
Codex & Claude Code: AI Coding Agents for Professionals
    https://learn.modernagecoders.com/courses/codex-and-claude-code-ai-coding-agents-masterclass-for-adults-professionals
Coding for Kids: Scratch to Games, Apps and AI
    https://learn.modernagecoders.com/courses/kids-coding-blocks-masterclass
Competitive Programming Course: Codeforces to ICPC
    https://learn.modernagecoders.com/courses/competitive-programming-masterclass-college
Competitive Programming for Teens: ZCO, INOI and USACO Track
    https://learn.modernagecoders.com/courses/competitive-programming-for-teens-course
Computer Science Class 11-12: CBSE Python & ICSE/ISC Java
    https://learn.modernagecoders.com/courses/cbse-icse-computer-science-class-11-12-python-java-complete-course
Copilot Studio Agent Masterclass for Professionals
    https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-masterclass-for-professionals
Creative Coding and Animation: Video, Motion and p5.js
    https://learn.modernagecoders.com/courses/creative-coding-animation-mastery
Creative Media for Teens: Photo, Video and Design
    https://learn.modernagecoders.com/courses/creative-media-design-masterclass-teens
Cybersecurity Competitions for Teens: picoCTF and CyberPatriot Prep
    https://learn.modernagecoders.com/courses/picoctf-cyberpatriot-cybersecurity-competition-prep-for-teens
Cybersecurity for Teens: Online Safety & White-Hat Defense
    https://learn.modernagecoders.com/courses/cybersecurity-course-for-teens-ethical-defensive
Data Analysis Course: Excel, SQL, Python & BI
    https://learn.modernagecoders.com/courses/data-analysis-mastery-course-college
Data and AI Analytics for Non-Programmers (Excel, Sheets, Power BI)
    https://learn.modernagecoders.com/courses/data-and-ai-analytics-for-non-programmers-course
Data Science Course: Python, ML & Deep Learning
    https://learn.modernagecoders.com/courses/data-science-complete-masterclass-college
Data Science for Teens: Python & Your First ML Models
    https://learn.modernagecoders.com/courses/data-science-course-for-teens-python-data
Data Structures & Algorithms Course: Interview-Ready DSA
    https://learn.modernagecoders.com/courses/data-structures-algorithms-masterclass-college
Ethical Hacking Course: Authorized Pentesting, Zero to Pro
    https://learn.modernagecoders.com/courses/ethical-hacking-masterclass-complete
Flutter Course: Dart to Real Cross-Platform Apps
    https://learn.modernagecoders.com/courses/complete-flutter-app-development-masterclass-college
Frontend Development for Teens: CSS Craft to React UI
    https://learn.modernagecoders.com/courses/frontend-development-masterclass-for-teens
Full Stack Developer Course: React, Node.js and DevOps Basics
    https://learn.modernagecoders.com/courses/full-stack-web-development-masterclass-college
Full Stack Web Development for Teens: React and Node.js
    https://learn.modernagecoders.com/courses/full-stack-web-development-teens-masterclass
Game Development Course: Unity, C# & Unreal Engine
    https://learn.modernagecoders.com/courses/game-development-complete-masterclass-college
Game Development for Kids: Scratch, Roblox and Minecraft
    https://learn.modernagecoders.com/courses/game-development-masterclass-for-kids
Game Development for Teens: Unity, C# and Real Shipped Games
    https://learn.modernagecoders.com/courses/complete-game-development-masterclass-for-teens
GCSE Computer Science: AQA, OCR and Edexcel Exam Prep
    https://learn.modernagecoders.com/courses/gcse-computer-science-course
Generative AI Course: LLMs, RAG and AI Agents
    https://learn.modernagecoders.com/courses/complete-generative-ai-masterclass-college
Git & GitHub Course for College Students: Team-Ready Skills
    https://learn.modernagecoders.com/courses/git-github-version-control-course-for-college-students
Git & GitHub Course for Teens: Version Control from Zero
    https://learn.modernagecoders.com/courses/git-github-version-control-course-for-teens
Go Course: Backend Services from Zero to Production Habits
    https://learn.modernagecoders.com/courses/complete-golang-programming-masterclass-college
Godot Game Development for Kids and Teens: GDScript, 2D and 3D
    https://learn.modernagecoders.com/courses/godot-game-development-course-for-kids-and-teens
Hackathon for Adults: Free Event + 12-Week Pro Prep (18+)
    https://learn.modernagecoders.com/courses/hackathon-prep-for-adults-professionals-coding-ai-innovation-course
Hackathon for Kids: Free Event + 12-Week Prep (Ages 8-12)
    https://learn.modernagecoders.com/courses/hackathon-prep-for-kids-coding-innovation-ai-projects-course
Hackathon for Teens: Free Event + 12-Week Prep (Ages 13-17)
    https://learn.modernagecoders.com/courses/hackathon-prep-for-teens-coding-ai-build-innovate-win-course
HTML & CSS for Beginners: Build Real Responsive Websites
    https://learn.modernagecoders.com/courses/html-css-course-for-beginners-build-real-websites
IB DP Computer Science (New Guide, First Assessment 2027)
    https://learn.modernagecoders.com/courses/ib-diploma-computer-science-course
ICSE Computer Applications (Class 9-10): Java with BlueJ
    https://learn.modernagecoders.com/courses/icse-computer-applications-java-bluej-course
IGCSE Computer Science (0478): Full Syllabus and Exam Prep
    https://learn.modernagecoders.com/courses/igcse-computer-science-0478-course
Informatics Practices (IP) Class 11-12: CBSE Python & SQL
    https://learn.modernagecoders.com/courses/cbse-informatics-practices-ip-class-11-12-python-pandas-sql-complete-course
Java Course for College: Core Java to Spring Boot
    https://learn.modernagecoders.com/courses/complete-java-programming-masterclass-college
Java for Teens: Complete Course from Beginner to Advanced

https://learn.modernagecoders.com/courses/java-programming-masterclass-for-teens
- JavaScript for Teens: Interactive Websites & Browser Games
  https://learn.modernagecoders.com/courses/javascript-course-for-teens-beginners-interactive-web
- Kotlin and Android Course: Zero to Published-Ready Apps
  https://learn.modernagecoders.com/courses/complete-kotlin-programming-masterclass-professional
- MERN Stack Course: MongoDB, Express, React and Node
  https://learn.modernagecoders.com/courses/mern-stack-development-masterclass-college
- Microsoft Office for Kids: Word, PowerPoint and Excel
  https://learn.modernagecoders.com/courses/microsoft-office-kids-mastery
- Minecraft Coding for Kids: From Blocks to Real Code
  https://learn.modernagecoders.com/courses/minecraft-coding-for-kids-course
- MySQL Course: SQL, Database Design and Real Administration
  https://learn.modernagecoders.com/courses/mysql-database-complete-masterclass-college
- MySQL for Teens: SQL from First Query to Real Apps
  https://learn.modernagecoders.com/courses/mysql-mastery-for-teens
- Ontario Computer Science Companion: ICS3U and ICS4U (Python and Java)
  https://learn.modernagecoders.com/courses/ics3u-ics4u-computer-science-companion-course
- Orange Data Mining Course for College Students: Machine Learning and Analytics
  https://learn.modernagecoders.com/courses/orange-data-mining-machine-learning-course-for-college-students
- Orange Data Mining Course for Teens: Machine Learning Without Code
  https://learn.modernagecoders.com/courses/orange-data-mining-visual-machine-learning-course-for-teens
- PCAP Certification Prep: Certified Associate in Python Programming
  https://learn.modernagecoders.com/courses/pcap-python-certification-prep-course
- PCEP and PCAP Python Certification Course
  https://learn.modernagecoders.com/courses/pcep-pcap-python-certification-course
- Princess Coders: Coding Course for Girls and Women
  https://learn.modernagecoders.com/courses/princess-coders-complete-coding-masterclass
- Problem Solving and Computational Thinking for Kids
  https://learn.modernagecoders.com/courses/problem-solving-and-computational-thinking-for-kids
- Problem Solving for Teens: Logic, Data Structures & Algorithms
  https://learn.modernagecoders.com/courses/problem-solving-dsa-masterclass-teens
- Python and AI for Kids: Games, Turtle Art and Smart Projects
  https://learn.modernagecoders.com/courses/python-ai-kids-masterclass
- Python Automation Course: Scripting, Scraping & Applied AI
  https://learn.modernagecoders.com/courses/python-ai-automation-masterclass-college
- Python for Teens: Complete Course from Beginner to Advanced
  https://learn.modernagecoders.com/courses/python-complete-masterclass-teens
- Python Masterclass: Zero to Advanced, Live Online
  https://learn.modernagecoders.com/courses/python-programming-masterclass-zero-to-advanced-college
- Python Web Development with Django and Flask Course
  https://learn.modernagecoders.com/courses/python-web-development-django-flask-course
- Queen Coders: Advanced Engineering for Women Developers
  https://learn.modernagecoders.com/courses/queen-coders-advanced-tech-leadership
- React Course for Teens: JavaScript to Real React Apps
  https://learn.modernagecoders.com/courses/react-for-teens-complete-masterclass
- React Masterclass: JavaScript to Production React
  https://learn.modernagecoders.com/courses/react-js-complete-masterclass-college
- Roblox Game Coding for Kids: Build Real Games in Luau
  https://learn.modernagecoders.com/courses/roblox-game-coding-for-kids-lua-course
- Scratch Coding for Kids: Zero to Real Games in 3 Months
  https://learn.modernagecoders.com/courses/scratch-programming-complete-course
- Swift and iOS Course: SwiftUI Apps from Zero
  https://learn.modernagecoders.com/courses/complete-swift-programming-masterclass-college
- Trading Automation Course: Bots, Backtesting & Systems
  https://learn.modernagecoders.com/courses/algorithmic-trading-automation-masterclass
- TypeScript Course: Typed JavaScript, React and Node
  https://learn.modernagecoders.com/courses/complete-typescript-programming-masterclass-college
- Vibe Coding for College: Full Stack, AI & DSA (Ages 18+)
  https://learn.modernagecoders.com/courses/vibe-coding-for-college-fullstack-ai-dsa-career-course
- Vibe Coding for Kids: AI, Scratch & Game Dev (Ages 8-12)
  https://learn.modernagecoders.com/courses/vibe-coding-for-kids-beginners-ai-scratch-game-dev
- Vibe Coding for Teens: Python, Web Dev & AI (Ages 13-17)
  https://learn.modernagecoders.com/courses/vibe-coding-for-teens-python-web-ai-projects-course
- Web Development for Kids: Build Real Websites
  https://learn.modernagecoders.com/courses/kids-ai-web-development-course

MATHEMATICS (38 courses)

- 11 Plus Maths Prep: UK Grammar & Independent School Entry
  https://learn.modernagecoders.com/courses/11-plus-maths-preparation-course-uk
- A Level Further Maths: Core Pure and Applied Options
  https://learn.modernagecoders.com/courses/a-level-further-maths-course
- A-Level Maths: Pure, Mechanics, Statistics (Edexcel/AQA)
  https://learn.modernagecoders.com/courses/a-level-maths-course-pure-mechanics-statistics
- Abacus & Mental Maths for Kids: From First Beads to Anzan
  https://learn.modernagecoders.com/courses/abacus-mental-maths-course-for-kids
- ACT Math Prep: Live Classes for the ACT Mathematics Section
  https://learn.modernagecoders.com/courses/act-math-prep-course
- Algebra Foundations: Pre-Algebra & Algebra 1, Live Online

https://learn.modernagecoders.com/courses/algebra-foundations-masterclass
- AP Calculus AB & BC Exam Prep, Live Online, Score 5 Goal
  https://learn.modernagecoders.com/courses/ap-calculus-exam-prep
- AP Precalculus: Units 1 to 3 and Full Exam Prep
  https://learn.modernagecoders.com/courses/ap-precalculus-exam-prep-course
- AP Statistics: Exam Prep with the Investigative Task
  https://learn.modernagecoders.com/courses/ap-statistics-maths-exam-prep-course
- Business & Finance Mathematics: Investment to Risk
  https://learn.modernagecoders.com/courses/complete-business-finance-mathematics-mastery
- CBSE Class 10 Maths Board Prep: Case Studies & Mocks
  https://learn.modernagecoders.com/courses/cbse-class-10-maths-board-exam-prep-course
- College Maths Course: Calculus, Linear Algebra to Analysis
  https://learn.modernagecoders.com/courses/college-mathematics-complete-masterclass
- Early Math Foundations: Playful, Concrete Math (Ages 4-7)
  https://learn.modernagecoders.com/courses/early-math-foundations
- Elementary Maths for Grades 1-5: Live Online Classes
  https://learn.modernagecoders.com/courses/elementary-mathematics-complete-masterclass
- Functional Skills Maths Level 1 and Level 2 Course Online
  https://learn.modernagecoders.com/courses/functional-skills-maths-course
- GCSE Maths Resit Course Online: Post-16 and Adults
  https://learn.modernagecoders.com/courses/gcse-maths-resit-course
- GCSE Maths: Foundation & Higher (9-1), AQA, Edexcel, OCR
  https://learn.modernagecoders.com/courses/gcse-mathematics-mastery
- GRE & GMAT Quantitative Prep: Live Maths Coaching
  https://learn.modernagecoders.com/courses/gre-gmat-quant-maths-prep-course
- High School Mathematics: Algebra to Calculus (Grade 9-12)
  https://learn.modernagecoders.com/courses/complete-high-school-mathematics-mastery
- IB Maths AA & AI: SL & HL, Live Online with IA Coaching
  https://learn.modernagecoders.com/courses/ib-mathematics-aa-ai-masterclass
- IGCSE Maths: Core, Extended & Additional Mathematics Online
  https://learn.modernagecoders.com/courses/igcse-mathematics-mastery
- JEE Foundation Maths Class 8-10: Proofs & Problem Solving
  https://learn.modernagecoders.com/courses/jee-foundation-maths-course-class-8-10
- KS2 SATs Maths: Arithmetic, Reasoning and Times Tables
  https://learn.modernagecoders.com/courses/ks2-sats-maths-course
- Maths for Data Science: Statistics, Linear Algebra and ML
  https://learn.modernagecoders.com/courses/data-analytics-mathematics-masterclass
- Maths Olympiad & Competition Course: AMC, AIME to IMO
  https://learn.modernagecoders.com/courses/olympiad-competition-mathematics-mastery
- Maths Through Coding: Learn Math in Python (Ages 10-15)
  https://learn.modernagecoders.com/courses/maths-through-coding
- Mental Maths for Kids: Number Sense, Not Tricks (Ages 5-10)
  https://learn.modernagecoders.com/courses/mental-maths-mastery-kids
- Middle School Math Competitions: MATHCOUNTS, AMC 8 and Math Kangaroo Prep
  https://learn.modernagecoders.com/courses/middle-school-math-competitions-course
- Middle School Mathematics Mastery: Pre-Algebra & Geometry
  https://learn.modernagecoders.com/courses/comprehensive-middle-school-mathematics-mastery
- National 5 and Higher Maths Course Online (Scotland)
  https://learn.modernagecoders.com/courses/national-5-and-higher-maths-course
- Olympiad Mathematics: IOQM, INMO and AMC Preparation
  https://learn.modernagecoders.com/courses/olympiad-mathematics-premium-course
- Ontario Grade 12 University Math: Advanced Functions and Calculus and Vectors (MHF4U and MCV4U)
  https://learn.modernagecoders.com/courses/ontario-grade-12-university-math-mhf4u-mcv4u-course
- PSLE Maths: Model Method, Problem Sums & MOE Syllabus
  https://learn.modernagecoders.com/courses/psle-maths-mastery
- SAT Math 800 Prep: Live Classes for the Digital SAT
  https://learn.modernagecoders.com/courses/sat-math-prep-course
- Statistics & Probability: Data to Hypothesis Testing
  https://learn.modernagecoders.com/courses/statistics-probability-maths-course
- Vedic Maths Mastery: Speed Calculation with All 16 Sutras
  https://learn.modernagecoders.com/courses/vedic-maths-course-speed-calculation-mastery
- Waterloo Math Contests: Gauss to Euclid
  https://learn.modernagecoders.com/courses/waterloo-math-contests-gauss-to-euclid-course
- Western Canada Grade 12 Math: Pre-calculus 12 and Math 30-1
  https://learn.modernagecoders.com/courses/western-canada-grade-12-math-pre-calculus-12-math-30-1-course

==
PART 6. EVERY COURSE IN FULL DETAIL
==

Each course below is self contained. It repeats the course name, the link and the price so you never quote the wrong one. The week by week plan is exactly what the student is taught, and the projects are exactly what they build and keep.

==
COURSE 1 OF 137: 11 PLUS MATHS PREP: UK GRAMMAR & INDEPENDENT SCHOOL ENTRY
==

Course link: https://learn.modernagecoders.com/courses/11-plus-maths-preparation-course-uk
Subject area: Test Prep Mathematics
Designed for: Middle school
Who can join: UK students ages 9-11 (Years 5-6), grammar and independent school applicants
Course length: 8 months (32 weeks)
Time needed: 2 live classes/week + 2-3 hours practice, rising to timed papers in the final phase
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  11 plus maths course for UK grammar and independent school entrance: live online classes, full KS2 syllabus, GL-style papers and timed exam practice.

  In one line: The maths half of the 11 plus, taught properly over eight months, so test day tests the child and not the preparation.

ABOUT THE PROGRAMME

  11 plus maths does not go beyond the national curriculum; what makes it hard is that it asks Key Stage 2 mathematics at speed, in unfamiliar wording, against children who have often prepared for years. This 8-month live course covers the full ground in a teaching order that builds: number fluency first, because a child who hesitates over 7 times 8 loses time on every question that follows; then fractions, decimals and percentages, ratio and proportion, early algebra, geometry and measures, and data. The final two months turn knowledge into test performance: multi-step word problems, timing strategy, multiple-choice technique, and a run of practice papers in the style of GL Assessment, which now sets the tests for most grammar schools in England. We are honest about the landscape. CEM stopped producing its familiar paper-based 11 plus tests in 2023 and most regions that used them have moved to GL, so we do not sell CEM prep as if those regional tests still run. Families targeting independent schools that use the online ISEB Pre-Tests or their own papers get practice aligned to those formats. And we are honest about outcomes: good preparation raises a child's chances and steadies their nerves, but grammar school places are competitive and nobody can promise one.

WHAT MAKES THIS COURSE DIFFERENT

  - Aligned to the tests as they are now: GL Assessment for most grammar schools, ISEB Pre-Tests for many independents, and bespoke papers such as the CSSE in Essex, with practice matched to your target schools
  - No false CEM promises: CEM's paper-based regional tests ended in 2023, and our format practice reflects what children will actually sit
  - Number fluency is drilled before anything else, because test timing punishes slow arithmetic more than any missing topic
  - Word problems are treated as a skill of their own: reading, extracting and choosing the operation, taught explicitly rather than assumed
  - Two full months of timed technique and practice papers, with a marked review after every paper and an error log per child
  - Live small classes where the teacher watches the child work, plus honest monthly notes to parents about readiness

THE LEARNING PATH, STEP BY STEP

  Phase 1: Number fluency: place value, all four operations mental and written, tables cold, factors, multiples, primes and sequences
  Phase 2: Fractions, decimals and percentages as one connected system, then ratio, proportion and best-value reasoning
  Phase 3: Early algebra, geometry, measures and data: missing numbers and simple equations, shapes, angles, area, units, charts and averages
  Phase 4: Test craft: multi-step word problems, timing strategy, multiple-choice technique and a full practice-paper cycle with marked reviews

WHAT A STUDENT NEEDS BEFORE STARTING

  Maths level: Comfortable Year 4 maths: tables mostly known and written addition and subtraction secure. The week 1 diagnostic finds the true starting point
  Target schools: Any UK grammar or independent school. Tell us the target schools and we align format practice to the tests they actually use
  Equipment: Pencil and paper, a quiet place to work, and a device with stable internet for live classes. 11 plus maths papers do not allow calculators
  Timing: Ideally started in Year 5, so the course finishes shortly before tests taken at the start of Year 6

WHO THIS COURSE IS FOR

Grammar school applicants: Children in Years 5 and 6 preparing for GL-style grammar school tests anywhere in England
Independent school applicants: Families targeting independent senior schools that use the ISEB Pre-Tests or their own entrance papers
Strong but slow: Children whose maths is sound but whose speed under timing gives their scores away
Wordy problem strugglers: Children who can compute but freeze when the question arrives as a paragraph
Parents who want structure: Families who would rather have a taught course with honest feedback than a pile of practice books and guesswork

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: NUMBER FLUENCY (MONTHS 1-2, WEEKS 1-8)

Speed and accuracy with number come first. Every later topic, and every timed paper, leans on arithmetic that works without hesitation.

Month 1: Place Value and the Four Operations (Weeks 1-4)
Weeks 1-2: Place Value, Ordering and Mental Methods
Topics:
- Diagnostic check: where each child's number work actually stands
- Place value to millions and reading large numbers aloud
- Ordering, comparing and rounding to any power of ten
- Mental addition and subtraction strategies: partitioning, compensation, near doubles
- Column addition and subtraction with carrying and borrowing secure
- Negative numbers in context: temperature, money, number lines
Projects built:
- Speed grid week: a daily 5-minute arithmetic grid, times recorded so the child can watch their own speed climb
Practice: Daily 10-minute number work set as a routine, plus 20 mixed place value and rounding questions
Weeks 3-4: Multiplication, Division and Order of Operations
Topics:
- Times tables to 12 made automatic, with the stubborn facts targeted
- Written multiplication: two digits by two digits and beyond
- Short division and long division, including remainders in context
- Choosing between mental, jotted and written methods
- Order of operations with brackets
- Estimating first: catching wrong answers before they are written down
Projects built:
- Personal tables map: each child's slowest facts identified and drilled until they match their fastest
Practice: 25 multiplication and division questions plus daily tables sprints, errors sorted into fact errors and method errors
Assessment: Month 1 check: a 20-minute arithmetic paper, marked for accuracy first and speed second

Month 2: Number Properties and Fluency Under Time (Weeks 5-8)
Weeks 5-6: Factors, Multiples, Primes and Sequences
Topics:
- Factors and multiples, and the difference stated without hesitation
- Prime numbers to 100 and how to test for them
- Square numbers and cube numbers
- Divisibility rules for 2, 3, 4, 5, 6, 9 and 10
- Number sequences: finding the rule and the next terms
- Special sequences: triangular numbers and patterns that alternate
Projects built:
- Number detective set: 12 who-am-I number puzzles solved using properties, the kind the 11 plus loves
Practice: 24 property and sequence questions plus one puzzle set done with a parent, for fun rather than marks
Weeks 7-8: Mixed Arithmetic with the Clock Running Gently
Topics:
- Mixed four-operation problems in 11 plus wording
- Inverse operations: checking answers and finding missing digits
- Money calculations: change, totals and unit prices
- Estimation and sense checks: is this answer even possible
- Introducing gentle time pressure: untimed first, then generous, then realistic
- What to do when stuck: skip, mark and return
Projects built:
- First mini-paper: a 15-question arithmetic paper under generous timing, reviewed question by question in class
Practice: Three 15-minute mixed sets across the fortnight, each one slightly tighter on time than the last

Assessment: Phase 1 milestone: a 30-minute number paper covering both months, with a speed and accuracy report for parents

---

PHASE 2: FRACTIONS, DECIMALS, PERCENTAGES, RATIO AND PROPORTION (MONTHS 3-4, WEEKS 9-16)

The part-whole toolkit that 11 plus papers test relentlessly, taught as one connected system rather than three separate topics, then extended into ratio and proportion.

Month 3: Fractions, Decimals and Percentages (Weeks 9-12)

Weeks 9-10: Fractions That Make Sense

Topics:

- Equivalent fractions and simplifying to lowest terms
- Comparing and ordering fractions with different denominators
- Adding and subtracting fractions, including mixed numbers
- Fractions of amounts: two thirds of 96 without fear
- Improper fractions and mixed numbers, converting both ways
- Fraction word problems: half of a half, and shares that do not split evenly

Projects built:

- Fraction wall build: each child makes their own fraction wall and uses it to settle five comparison arguments

Practice: 28 fraction questions from equivalence to word problems, working shown even when the answer is obvious

Weeks 11-12: Decimals, Percentages and Moving Between All Three

Topics:

- Decimal place value and ordering decimals
- Multiplying and dividing by 10, 100 and 1000
- Adding, subtracting and multiplying decimals in money contexts
- Percentages as hundredths: finding 10 percent, 5 percent and 1 percent and building the rest
- Percentage of an amount, and simple percentage increase and decrease
- The conversion triangle: fractions, decimals and percentages swapped fluently

Projects built:

- Conversion pack: a self-made table of the equivalences worth knowing cold, from a half down to a twentieth

Practice: 26 decimal and percentage questions plus 10 rapid conversions daily until they are automatic

Assessment: Month 3 test: a 25-minute paper across fractions, decimals and percentages in 11 plus style

Month 4: Ratio, Proportion and Best Value (Weeks 13-16)

Weeks 13-14: Ratio and Sharing

Topics:

- Ratio notation and what 3 to 2 actually says
- Simplifying ratios, including with units
- Sharing an amount in a given ratio
- Ratio and fractions: moving between the two views
- Scaling recipes up and down
- Ratio word problems in test wording: sweets, marbles, paint mixes

Projects built:

- Recipe rescale: one real family recipe scaled for 2, 6 and 9 people, with the working presented neatly

Practice: 24 ratio questions, every sharing answer checked by adding the parts back together

Weeks 15-16: Proportion, Best Value and Scale

Topics:

- Direct proportion: the unitary method taught as a habit
- Best value: comparing pack sizes and unit prices
- Scale on maps and plans
- Speed, distance and time in simple, whole-number cases
- Proportion word problems that mix money and measures
- Spotting proportion questions hidden inside long wording

Projects built:

- Supermarket challenge: real price comparisons worked as best-value problems and written up as answers with reasons

Practice: 22 proportion and best-value questions plus a 20-minute mixed paper on all of phase 2

Assessment: Phase 2 milestone: a 30-minute paper across fractions, decimals, percentages, ratio and proportion

---

PHASE 3: EARLY ALGEBRA, GEOMETRY, MEASURES AND DATA (MONTHS 5-6, WEEKS 17-24)

The remaining syllabus ground: missing-number thinking and simple equations, then shapes, angles, area, units and time, and finally charts, tables and averages.

Month 5: Early Algebra and Geometry (Weeks 17-20)

Weeks 17-18: Missing Numbers and Simple Equations

Topics:

- Missing number problems as balance puzzles

- Function machines forwards and backwards
- Simple one-step and two-step equations solved by undoing
- Substituting numbers into simple rules and formulas
- Word puzzles turned into number sentences
- Think-of-a-number problems worked backwards

Projects built:
- Puzzle exchange: each child writes three think-of-a-number puzzles and solves three written by classmates

Practice: 24 missing-number and equation questions, each answer substituted back to check itself

Weeks 19-20: Shapes, Angles and Position

Topics:
- 2D shapes and their properties: triangles by type, quadrilaterals by name
- Angle types, angles on a line, around a point and inside a triangle
- Symmetry: line and rotational
- Coordinates in all four quadrants
- Reflection and translation on a grid
- 3D shapes, faces, edges, vertices and nets

Projects built:
- Shape fact file: a one-page reference of shape and angle facts in the child's own words and drawings

Practice: 26 geometry questions with every angle answer justified by the fact used

Assessment: Month 5 test: a 25-minute paper on algebra and geometry in test style

Month 6: Measures, Time and Data (Weeks 21-24)

Weeks 21-22: Perimeter, Area, Volume and Units

Topics:
- Perimeter and area of rectangles and compound shapes
- Area of a triangle
- Volume of cuboids, counting cubes first and formula second
- Metric units and conversions: length, mass, capacity
- Reading scales and measuring instruments accurately
- Time: 24-hour clock, timetables and elapsed-time problems

Projects built:
- Room plan: the child's own room measured, drawn to scale and its floor area calculated

Practice: 24 measures questions with units written in every answer, timetable problems done against a gentle clock

Weeks 23-24: Charts, Tables and Averages

Topics:
- Reading tables, bar charts, pictograms and line graphs quickly and exactly
- Two-step chart questions: differences, totals and comparisons
- Pie charts read as fractions of a whole
- Mean, median, mode and range, and which the question is really asking for
- Simple probability language: likely, unlikely, evens, and fractions for equally likely outcomes
- The data traps: misread scales and units, and how to avoid them

Projects built:
- Family survey: a small data set collected at home, charted by hand, and turned into three test-style questions

Practice: 22 data questions plus one full mixed set covering everything taught so far

Assessment: Phase 3 milestone: a 40-minute paper across the whole syllabus, the first true full-coverage test

---

PHASE 4: PROBLEM SOLVING UNDER TIME AND THE PRACTICE-PAPER CYCLE (MONTHS 7-8, WEEKS 25-32)

The syllabus is done; now the test itself is the subject. Multi-step word problems, timing strategy and multiple-choice technique, then a full cycle of practice papers with marked reviews.

Month 7: Word Problems and Test Technique (Weeks 25-28)

Weeks 25-26: Multi-Step Word Problems

Topics:
- The four-step routine: read, extract, choose the operations, check
- Two-step and three-step problems across money, measures and time
- Problems that mix topics: a ratio inside a word problem inside a timetable
- Drawing the problem: bar models and quick sketches that untangle wording
- Spotting the trap answer: options built from common mistakes
- Logic-flavoured questions: working backwards and eliminating cases

Projects built:
- Problem autopsy set: 10 hard word problems dissected in writing, what was given, what was asked, what was the trap

Practice: 20 multi-step problems, each one annotated with the four-step routine before any arithmetic

Assessment: Month 7 opens with a timed baseline paper so the next four weeks of technique work can be measured against it

Weeks 27-28: Timing and Multiple-Choice Technique

Topics:
- How GL-style papers present maths: multiple choice with a separate answer sheet

- Answer-sheet discipline: keeping the row and the question in step
- Elimination: earning marks on questions you cannot fully solve
- Time budgeting: when to skip, how to mark for return, when to guess
- Estimation as a speed tool: rejecting three options in five seconds
- Staying calm mid-paper: the reset routine after a hard question

Projects built:
- Technique card: each child writes their own one-page test plan, timings, skip rules and reset routine included

Practice: Three timed 20-minute sections across the fortnight, answer sheets used every time, technique reviewed after each

Month 8: The Practice-Paper Cycle (Weeks 29-32)

Weeks 29-30: Full Papers, Marked and Mined

Topics:
- A full GL-style maths paper under real timing
- Marked review: every miss classified as knowledge, misread or timing
- Targeted drills built from each child's error pattern
- A second full paper, with the error log guiding the warm-up
- Format variations: standard-format papers and ISEB-style online questions for children who need them
- Parent briefing: what the scores mean and what they do not

Projects built:
- Error log finalised: each child's recurring mistakes named, with the fix written next to each

Practice: One paper in class and one at home per week, home papers sat under honest exam conditions

Assessment: Weekly full papers, each marked with a per-topic breakdown shared with parents

Weeks 31-32: Final Papers and a Calm Finish

Topics:
- Two final papers under strict test conditions
- Closing the last recurring errors, not chasing new topics
- The taper: why the last week before a test should be lighter, not heavier
- Test-day practicalities: sleep, breakfast, what to expect in the room
- Confidence talk: the test samples one morning, it does not measure the child
- Course close: readiness review with each family, honest and specific

Projects built:
- Readiness pack: the child's error log, technique card and score chart bound together as their own record of the eight months

Practice: Light mixed sets only in the final week, chosen from each child's strongest topics to finish on solid ground

Assessment: Course milestone: final full paper reviewed one to one with child and parents, plus certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A daily speed-grid routine with the child's own times charted across the course
- A personal tables map targeting each child's slowest number facts
- A self-made conversion pack for fractions, decimals and percentages
- A shape and angle fact file in the child's own words
- A real-data family survey turned into test-style questions
- A problem autopsy set dissecting hard word problems in writing
- A personal technique card with timings, skip rules and a reset routine
- A full practice-paper cycle with error log and score chart, bound as a readiness pack

Total projects: A complete child-built preparation record: speed charts, fact files, error log, technique card and marked paper series

SKILLS THE STUDENT WILL HAVE AT THE END

- Arithmetic speed and accuracy that hold up under test timing
- The full 11 plus maths syllabus: number, fractions and percentages, ratio, early algebra, geometry, measures and data
- A reliable routine for multi-step word problems
- Multiple-choice and answer-sheet technique for GL-style papers
- Timing strategy: skipping, returning and guessing intelligently
- Calm under test conditions, built through graduated practice rather than pressure

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week; paper-phase sittings run longer for full timed papers

Practice: 2-3 hours weekly: a short daily number routine plus set questions, rising to weekly papers in month 8

Review: Homework marked with per-topic feedback; the error log is reviewed with the child so they own their own progress

Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for both child and parents, with worked solutions for stuck problems
Progress updates: Monthly progress notes to parents with scores, the error-log summary and an honest readiness view

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Grammar school entrance tests, sat with the maths half properly prepared
- Independent school pretests and entrance papers, including ISEB-style online formats
- Key Stage 2 SATs, which draw on the same curriculum this course secures
- A confident start to secondary maths, whichever school the child joins
- Our school maths and maths-through-coding courses for children who find they enjoy the subject

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which 11 plus test formats does the course prepare for?
A: Most grammar schools in England now use tests from GL Assessment, so GL-style multiple-choice papers with separate answer sheets are our default format practice. Tell us your target schools at the start and we align the paper phase to the formats your child will actually sit. The maths syllabus underneath is the same in every case.

Q: Do you offer CEM preparation?
A: We prepare children for the tests that exist now, and the familiar paper-based CEM regional tests are not among them: CEM stopped producing them in 2023, and most areas that used them have moved to GL Assessment. CEM continues as an online product used mainly by some independent schools, and if a target school uses it we practise accordingly. What we will not do is sell CEM-branded preparation as if the old regional tests still run, because they do not.

Q: Will this course get my child into grammar school?
A: No course can honestly promise that, and you should be wary of any that does. Grammar school places are competitive, pass marks move with each year's cohort, and admission can also depend on catchment and oversubscription rules that have nothing to do with maths. What preparation genuinely does is make sure the test measures your child's ability rather than their unfamiliarity with the format. We commit to full syllabus coverage, honest monthly reporting, and a clear readiness picture before test day.

Q: When should we start preparing, and is 8 months enough?
A: Most families start in Year 5, and this course is built for that: eight months starting in the autumn or spring of Year 5 finishes shortly before tests sat at the start of Year 6. Eight months is enough for a child with sound school maths; it allows the syllabus to be taught rather than crammed, with two full months left for timed practice. Children starting later can still join, but we compress honestly and tell you what the shorter runway does and does not allow.

Q: What maths is actually on the 11 plus?
A: Nothing beyond the national curriculum, mostly content taught up to the end of Year 5, since tests are sat early in Year 6. The difficulty comes from presentation and pace: multi-step word problems, unfamiliar wording, and timing that punishes slow arithmetic. Calculators are not allowed, so everything is built for pencil and paper.

Q: Does the course cover verbal reasoning, English and non-verbal reasoning too?
A: No, this course is the maths paper only, taught deeply rather than four subjects taught thinly. Many families pair it with their own English and reasoning practice or another provider for those papers. We would rather be excellent at one quarter of the 11 plus than mediocre at all of it, and we will always tell you honestly whether your child's remaining preparation time is better spent on maths or elsewhere.

Q: How much homework is there, and will my child burn out?
A: Two to three hours a week: a short daily number routine of about ten minutes plus one or two problem sets, rising to a weekly practice paper in the final month. We taper deliberately before test day rather than piling on, because a tired nine-year-old tests worse than a fresh one.

Q: You are based in India. Can you really prepare a child for a UK exam?
A: The 11 plus maths syllabus is published, the test formats are public, and our 11 plus teachers work with UK materials, GL-style papers and UK curriculum terminology throughout. Classes are scheduled for UK afternoons and evenings, and many of our students are British families, both in the UK and expatriates preparing for a return. What matters is whether the

teaching is live, small-group and honest about progress, and that is exactly what we do. Judge us at the free demo class. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/11-plus-maths-preparation-course-uk
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 2 OF 137: A LEVEL FURTHER MATHS: CORE PURE AND APPLIED OPTIONS

==========

Course link: https://learn.modernagecoders.com/courses/a-level-further-maths-course
Subject area: Exam Board Mathematics
Designed for: High school
Who can join: Year 12 and 13 students (or equivalent) taking Further Maths with AQA, Edexcel or OCR
Course length: 18 months (72 weeks), taught alongside A Level Maths
Time needed: 2 live classes/week + 3-4 hours practice, rising to full timed papers in the last two months
Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

A Level Further Maths course online: live classes in Core Pure plus mechanics, statistics or decision options for AQA, Edexcel and OCR, ending in timed papers.

In one line: The second maths A level, taught as a subject of its own: complex numbers, matrices, proof and differential equations, then the applied options your board sets.

ABOUT THE PROGRAMME

Further Maths is a full A level that sits on top of A level Maths. The boards are clear about the relationship: OCR states that learners are assumed to know the content of GCSE Mathematics and A Level Mathematics, and Pearson Edexcel says the students who benefit most are likely to hold A level Maths. In practice most students take the two side by side across Years 12 and 13, and this course is timed to run alongside our A Level Maths course or your school's. Every board splits Further Maths into a compulsory pure core and a set of options. Edexcel 9FM0 has two Core Pure papers plus two options chosen from Further Pure, Further Statistics, Further Mechanics and Decision Maths. AQA 7367 has two papers on compulsory content and a third paper on two of mechanics, statistics and discrete mathematics. OCR A H245 has Pure Core papers Y540 and Y541 plus two optional papers, and OCR MEI H645 has the Core Pure paper Y420 with a major option or a set of minor options. We teach the shared core to everyone and the options your school or exam centre has entered you for. The first nine months cover the AS-standard core and the first applied option. Months 10 to 16 take the core to full A level depth, from de Moivre's theorem to second-order differential equations, and finish the options. The final two months are revision and past papers under real timing, matched to your board's paper structure and marked against official schemes. Classes are live and small, so the teacher reads your working as you write it.

WHAT MAKES THIS COURSE DIFFERENT

- Built from the boards' own specifications: Edexcel 9FM0, AQA 7367, OCR A H245 and OCR MEI H645, with your board's paper structure driving the practice phase
- Sequenced to run alongside A Level Maths, so each Further Maths topic arrives after the A level Maths skill it leans on, not before
- Proof treated as a skill in its own right: induction, contradiction and the writing of a complete argument are practised every month, not saved for one chapter
- Options taught to your entry, whether that is mechanics, statistics, decision or discrete maths, or further pure, rather than a generic mixture
- Calculator technique for matrices, complex numbers and distributions taught explicitly, alongside the work that must be done by hand
- Two full months of past papers under real timing, marked against official mark schemes, with an error clinic after every paper

THE LEARNING PATH, STEP BY STEP

Phase 1: Core Pure, first year: proof by induction, complex numbers and the Argand diagram, matrices and transformations, roots of polynomials, series and vectors in three dimensions
Phase 2: Applied option one: the first of your board's options, taught to AS depth, then an AS-standard mock across core and option
Phase 3: Core Pure, second year: de Moivre and complex roots, further calculus, Maclaurin series, polar coordinates, hyperbolic functions and differential equations
Phase 4: Applied option two and option depth: the second option, and both options taken to full A level standard
Phase 5: Revision and the past-paper cycle: structured revision by strand, then full papers under timing with marked review

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Studying A level Maths at the same time, or already holding it. A strong GCSE Maths grade, which we suggest is 7 or above, makes the first months far smoother. The week 1 diagnostic finds gaps and we close them inside the course
Exam board: AQA 7367, Edexcel 9FM0, OCR A H245 or OCR MEI H645. The core is shared; we teach the options your school or exam centre enters you for
Equipment: A calculator that handles matrices, complex numbers and statistical distributions, notebooks, and a device with stable internet for live classes
Timing: Best started at the beginning of Year 12 alongside A level Maths; later joiners get a catch-up plan after the diagnostic

WHO THIS COURSE IS FOR

Year 12 starters: Students starting Further Maths alongside A level Maths who want both taught in a sensible order
Schools without further maths: Students whose school or college does not teach Further Maths, who need the whole course taught and will sit the exams at an exam centre
Students needing more support: Students whose school offers Further Maths in a small, rushed or part-timetabled class and who want steady teaching beside it
International students: Students taking British A levels abroad, at international schools or as private candidates, who need live teaching across time zones
Returning adults: Adults who took A level Maths earlier and want the second maths A level for study or for its own sake

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: CORE PURE, FIRST YEAR (MONTHS 1-5, WEEKS 1-20)

The compulsory core every board examines, taught to AS depth. Each topic is introduced after the A level Maths skill it needs, and proof is practised from the first fortnight.

Month 1: Proof and Series (Weeks 1-4)
Weeks 1-2: Diagnostic and Proof by Induction
Topics:
- Diagnostic paper on the A level Maths algebra Further Maths assumes
- The structure of a proof by induction: basis, assumption, inductive step, conclusion
- Induction on sums of series
- Induction on divisibility statements
- Induction on matrix powers, previewed for month 3
- Writing the conclusion so it would earn the final mark

Projects built:
- Proof portfolio opened: six induction proofs written to exam standard and checked line by line

Practice: 12 induction proofs of increasing difficulty, each with a written conclusion sentence
Weeks 3-4: Standard Series and the Method of Differences
Topics:
- Sigma notation and the standard results for r, r squared and r cubed
- Combining standard results to sum new series
- The method of differences and when it applies
- Partial fractions used to set up a telescoping sum
- Checking a summation formula by substitution and by induction
- Common slips: index limits and the first term

Projects built:
- Question bank entry: 15 solved series questions, each checked by a second method

Practice: 20 series problems mixing standard results and the method of differences
Assessment: Month 1 test: a 45-minute paper on induction and series, marked with method marks

Month 2: Complex Numbers (Weeks 5-8)
Weeks 5-6: Complex Arithmetic and the Argand Diagram
Topics:

      - The imaginary unit and the algebra of complex numbers
      - Conjugates and division of complex numbers
      - Solving quadratics, cubics and quartics with complex roots
      - Representing complex numbers on an Argand diagram
      - Modulus and argument, and the modulus-argument form
      - Multiplying and dividing in modulus-argument form
    Projects built:
      - Complex number reference sheet built by the student, with one worked example per rule
    Practice: 25 complex arithmetic and equation problems, roots shown on an Argand diagram
  Weeks 7-8: Loci and Regions
    Topics:
      - Loci of the form |z - a| = r as circles
      - Perpendicular bisectors from |z - a| = |z - b|
      - Half-lines from arg(z - a) = theta
      - Shading regions defined by inequalities
      - Finding greatest and least values of |z| on a locus
      - Sketching accurately enough to read answers from the diagram
    Projects built:
      - Locus atlas: eight annotated sketches of standard loci and combined regions
    Practice: 16 locus and region questions, each with a labelled sketch
    Assessment: Month 2 test: a 45-minute paper on complex numbers

Month 3: Matrices (Weeks 9-12)
  Weeks 9-10: Matrix Algebra and Transformations
    Topics:
      - Matrix addition and multiplication, and why order matters
      - The identity and zero matrices
      - 2 by 2 matrices as transformations of the plane
      - Rotations, reflections, enlargements and stretches
      - Composite transformations and the product of their matrices
      - Invariant points and invariant lines
    Projects built:
      - Transformation catalogue: each standard 2 by 2 matrix with its effect drawn on the unit
        square
    Practice: 20 problems on matrix products and transformations, with diagrams
  Weeks 11-12: Determinants, Inverses and Linear Systems
    Topics:
      - Determinants of 2 by 2 and 3 by 3 matrices
      - The determinant as an area or volume scale factor
      - Inverse matrices and singular matrices
      - Solving systems of three linear equations with matrices
      - Geometric meaning of a system: three planes and how they can meet
      - Calculator checking for 3 by 3 work, and what must still be shown
    Projects built:
      - Three-planes casebook: every arrangement of three planes, with a system that produces it
    Practice: 18 problems on determinants, inverses and linear systems
    Assessment: Month 3 test: a 45-minute paper on matrices

Month 4: Roots of Polynomials and Rational Functions (Weeks 13-16)
  Weeks 13-14: Roots of Polynomials
    Topics:
      - Relationships between roots and coefficients for quadratics, cubics and quartics
      - Symmetric functions of roots
      - Forming a new equation whose roots are related to the original roots
      - Substitution as a shortcut for transformed roots
      - Complex roots of real polynomials and conjugate pairs
      - Checking answers by reconstructing the polynomial
    Projects built:
      - Question bank entry: 12 roots questions solved two ways, by sums of roots and by substitution
    Practice: 20 problems on roots and coefficients
  Weeks 15-16: Curve Sketching and Inequalities
    Topics:
      - Rational functions and their asymptotes
      - Sketching graphs of rational functions
      - Solving inequalities involving rational expressions
      - Graphs of y = |f(x)| and related transformations
      - Conic sections in the forms your board specifies
      - Using a sketch to check an algebraic answer
    Projects built:
      - Graph file: ten sketches with asymptotes, intercepts and turning points labelled
    Practice: 16 sketching and inequality problems
    Assessment: Month 4 test: a 45-minute paper on roots and curves

Month 5: Vectors and First Further Calculus (Weeks 17-20)
  Weeks 17-18: Lines and Planes in Three Dimensions
    Topics:
      - Vector and Cartesian equations of a line in three dimensions
      - The scalar product and the angle between lines
      - Equations of a plane in vector and Cartesian form

- Angles between a line and a plane, and between two planes
- Intersections of lines and planes
- Shortest distances from a point to a line and to a plane

Projects built:
- 3D geometry sheet: every distance and angle method on one page with a worked example

Practice: 18 problems on lines and planes

Weeks 19-20: Volumes of Revolution and Mean Values

Topics:
- Volumes of revolution about the x-axis and the y-axis
- Volumes from parametric curves where your board includes them
- Mean value of a function over an interval
- Choosing the right integral from a description in words
- Modelling a real solid with a volume of revolution
- Accuracy and units in modelling answers

Projects built:
- Modelling task: the volume of a real container estimated by a volume of revolution and compared with its label

Practice: 14 volume and mean value problems

Assessment: Month 5 test: a 50-minute paper on vectors and first further calculus

---

PHASE 2: APPLIED OPTION ONE (MONTHS 6-9, WEEKS 21-36)

The first of the options your board and centre have chosen. The topics below show the mechanics route; statistics, decision or discrete students follow the same timetable with their own option content.

Month 6: Momentum and Impulse (Weeks 21-24)

Weeks 21-22: Momentum, Impulse and Collisions

Topics:
- Momentum and impulse in one dimension
- Conservation of momentum
- Newton's law of restitution and the coefficient of restitution
- Direct collisions between two particles
- Collisions with a fixed wall
- Loss of kinetic energy in a collision

Projects built:
- Collision file: worked solutions for every collision type, with a diagram for each

Practice: 18 momentum and restitution problems

Weeks 23-24: Successive Collisions and Impulse in Two Dimensions

Topics:
- Three-particle and successive collisions
- Deciding whether a further collision happens
- Impulse as a vector in two dimensions
- Oblique impacts with a smooth surface where your board includes them
- Setting up equations from a clear diagram
- Statistics route equivalent: discrete random variables, expectation and variance

Projects built:
- Question bank entry: 10 multi-stage collision problems with the decision points marked

Practice: 14 multi-stage problems under 10-minute timing each

Assessment: Month 6 test: a 45-minute option paper

Month 7: Work, Energy and Power (Weeks 25-28)

Weeks 25-26: Work, Energy and Power

Topics:
- Work done by a constant force
- Kinetic and gravitational potential energy
- The work-energy principle
- Power, and driving force for a vehicle
- Problems on slopes with resistance
- Statistics route equivalent: the Poisson distribution and when it models a count

Projects built:
- Energy method sheet: when to use energy and when to use Newton's second law

Practice: 18 work, energy and power problems

Weeks 27-28: Elastic Strings and Springs

Topics:
- Hooke's law for strings and springs
- Elastic potential energy
- Energy problems with elastic strings
- Finding extensions and maximum speeds
- Combining elastic energy with gravity
- Decision route equivalent: algorithms, sorting and the order of an algorithm

Projects built:
- Modelling task: a bungee jump modelled with energy methods and its assumptions criticised

Practice: 14 elastic energy problems

Assessment: Month 7 test: a 45-minute option paper

Month 8: Circular Motion and Dimensional Analysis (Weeks 29-32)

Weeks 29-30: Horizontal and Vertical Circular Motion

      Topics:
        - Angular speed and acceleration towards the centre
        - Horizontal circles: conical pendulums and banked tracks
        - Vertical circles and energy
        - Conditions for completing a vertical circle
        - Particles leaving a circular surface
        - Statistics route equivalent: chi-squared tests for goodness of fit and contingency tables
      Projects built:
        - Circular motion casebook: six standard set-ups with force diagrams
      Practice: 16 circular motion problems
    Weeks 31-32: Dimensional Analysis and Modelling
      Topics:
        - Dimensions of physical quantities
        - Checking a formula by its dimensions
        - Finding an unknown formula by dimensional analysis
        - Criticising and refining a mechanics model
        - Writing assumptions the way mark schemes expect
        - Decision route equivalent: graphs, networks and minimum spanning trees
      Projects built:
        - Model critique: one real situation modelled three ways, with the assumptions compared
      Practice: 12 dimensional analysis and modelling questions
      Assessment: Month 8 test: a 45-minute option paper

  Month 9: AS Checkpoint (Weeks 33-36)
    Weeks 33-34: Consolidation of Core and Option
      Topics:
        - Mixed core problems that do not announce their topic
        - Proof rehearsal: induction proofs rewritten from memory
        - Option problems mixed across months 6 to 8
        - Reading a long question and planning before writing
        - Error log review with the teacher
        - Targeted re-teach of the two weakest topics per student
      Projects built:
        - Revision sheet set one: student-made single pages for every core topic so far
      Practice: Two 40-minute mixed papers, one core and one option
    Weeks 35-36: AS-Standard Mock
      Topics:
        - An AS-standard mock across Core Pure and the first option
        - Marking against an official-style scheme
        - Classifying every lost mark: knowledge, method, accuracy or reading
        - Planning the second year from the mock results
        - Students who sit the AS as a standalone qualification prepare for it here
        - Summer bridging tasks set for the move to second-year content
      Projects built:
        - The mock file: marked script, error classification and a written plan
      Practice: Mock review set: every lost mark reworked to full credit
      Assessment: AS-standard mock, marked and reviewed one to one

---
PHASE 3: CORE PURE, SECOND YEAR (MONTHS 10-14, WEEKS 37-56)

The A level half of the core, where the subject becomes recognisably university mathematics:
de Moivre, further calculus, polar curves, hyperbolic functions and differential equations.

  Month 10: Further Complex Numbers (Weeks 37-40)
    Weeks 37-38: Exponential Form and de Moivre's Theorem
      Topics:
        - The exponential form of a complex number
        - De Moivre's theorem and its proof by induction
        - Expressing cos n theta and sin n theta in powers
        - Powers of sines and cosines in terms of multiple angles
        - Sums of series using complex numbers
        - Choosing between Cartesian, modulus-argument and exponential form
      Projects built:
        - Proof portfolio entry: de Moivre's theorem proved by induction for positive integers
      Practice: 18 problems on exponential form and de Moivre's theorem
    Weeks 39-40: Roots of Unity and Complex Roots
      Topics:
        - The nth roots of unity and their symmetry
        - The nth roots of any complex number
        - Roots shown as regular polygons on the Argand diagram
        - Geometric problems solved with complex roots
        - Further loci and transformations where your board includes them
        - Exam-style questions that chain several complex number techniques
      Projects built:
        - Roots of unity poster: the roots for n = 3 to 8 drawn and their sums explained
      Practice: 16 roots problems
      Assessment: Month 10 test: a 50-minute paper on complex numbers

Month 11: Further Calculus and Series (Weeks 41-44)
  Weeks 41-42: Improper Integrals and Inverse Trigonometric Functions
    Topics:
      - Improper integrals and whether they converge
      - Derivatives of inverse trigonometric functions
      - Integrals that lead to inverse sine and inverse tangent
      - Partial fractions with quadratic factors
      - Choosing an integration method from the form of the integrand
      - Accuracy in exact answers
    Projects built:
      - Calculus method map extended: every further integral form routed to its technique
    Practice: 20 further integration problems
  Weeks 43-44: Maclaurin Series
    Topics:
      - Maclaurin series from repeated differentiation
      - Standard series for exponential, sine, cosine and log functions
      - Combining and composing known series
      - Range of validity of a series
      - Using series to approximate and to find limits
      - Checking a series against a calculator value
    Projects built:
      - Series table: standard Maclaurin series with validity ranges and one application each
    Practice: 16 Maclaurin series problems
    Assessment: Month 11 test: a 50-minute paper on further calculus

Month 12: Polar Coordinates and Hyperbolic Functions (Weeks 45-48)
  Weeks 45-46: Polar Coordinates
    Topics:
      - Polar coordinates and converting to and from Cartesian form
      - Sketching polar curves such as cardioids and spirals
      - Areas enclosed by polar curves
      - Areas between two polar curves
      - Tangents parallel and perpendicular to the initial line
      - Reading symmetry from a polar equation
    Projects built:
      - Polar curve gallery: eight curves sketched by hand and checked against software
    Practice: 16 polar coordinate problems
  Weeks 47-48: Hyperbolic Functions
    Topics:
      - Definitions of sinh, cosh and tanh in terms of exponentials
      - Hyperbolic identities and Osborn's rule
      - Inverse hyperbolic functions in logarithmic form
      - Differentiating and integrating hyperbolic functions
      - Integrals that lead to inverse hyperbolic functions
      - Solving equations involving hyperbolic functions
    Projects built:
      - Hyperbolic reference sheet: definitions, identities, derivatives and integrals on one page
    Practice: 18 hyperbolic function problems
    Assessment: Month 12 test: a 50-minute paper on polar and hyperbolic work

Month 13: Differential Equations (Weeks 49-52)
  Weeks 49-50: First and Second Order Differential Equations
    Topics:
      - First-order linear equations with an integrating factor
      - Second-order linear equations with constant coefficients
      - The auxiliary equation: real, repeated and complex roots
      - Particular integrals for common right-hand sides
      - General and particular solutions from boundary conditions
      - Checking a solution by substituting it back
    Projects built:
      - Question bank entry: one solved example for every case of the auxiliary equation
    Practice: 18 differential equation problems
  Weeks 51-52: Modelling with Differential Equations
    Topics:
      - Simple harmonic motion as a second-order equation
      - Damped and forced oscillations
      - Coupled first-order systems
      - Interpreting a solution in the context of the model
      - Criticising the assumptions of a differential equation model
      - Long questions that combine setting up, solving and interpreting
    Projects built:
      - Modelling task: a damped oscillation modelled, solved and compared with measured data
    Practice: 12 modelling questions with full written interpretation
    Assessment: Month 13 test: a 50-minute paper on differential equations

Month 14: Core Pure Consolidation (Weeks 53-56)
  Weeks 53-54: Mixed Core Pure Problems
    Topics:
      - Long questions that cross complex numbers, matrices and calculus

        - Proof by induction across series, divisibility, matrices and derivatives
        - Further vectors where your board extends them
        - Numerical methods where your board includes them
        - Choosing the efficient method under time pressure
        - Error log review and targeted re-teach
      Projects built:
        - Revision sheet set two: single pages for every second-year core topic
      Practice: Two 60-minute mixed Core Pure papers
    Weeks 55-56: Core Pure Mock
      Topics:
        - A full Core Pure paper under timing for your board
        - Marking against an official-style scheme
        - Classifying every lost mark
        - Rewriting proofs that lost marks to full credit
        - Comparing performance with the AS-standard mock
        - Setting priorities for the final months
      Projects built:
        - The mock file updated with the Core Pure mock and a written plan
      Practice: Mock review set: every lost mark reworked
      Assessment: Core Pure mock, marked and reviewed one to one

---
PHASE 4: APPLIED OPTION TWO AND OPTION DEPTH (MONTHS 15-16, WEEKS 57-64)

The second option, taught at the depth your board sets, and the first option extended to full
A level standard. Students on a further pure route take their extra pure option here.

  Month 15: Second Option (Weeks 57-60)
    Weeks 57-58: Second Option Foundations
      Topics:
        - Statistics route: discrete and continuous random variables, and the geometric distribution
        - Mechanics route: further work on collisions and energy at A level depth
        - Decision route: route inspection, travelling salesperson bounds and linear programming
        - Further pure route: further vectors, number theory or groups where your board sets them
        - Calculator technique for the option
        - Reading option questions carefully: what the context is really asking
      Projects built:
        - Option reference sheet: every method in the second option on one page
      Practice: 20 problems on the second option
    Weeks 59-60: Second Option Applications
      Topics:
        - Statistics route: hypothesis tests for the Poisson and geometric distributions
        - Mechanics route: centres of mass of laminas and solids
        - Decision route: critical path analysis and network flows
        - Further pure route: further calculus and proof extensions
        - Long option questions under timing
        - Writing conclusions in context
      Projects built:
        - Question bank entry: 12 long option questions with model conclusions
      Practice: 16 option problems under timing
      Assessment: Month 15 test: a 60-minute option paper

  Month 16: Both Options at Full Depth (Weeks 61-64)
    Weeks 61-62: First Option to A Level Depth
      Topics:
        - The first option's A level content beyond the AS checkpoint
        - Mixed problems drawing on both options
        - Modelling assumptions and their criticism
        - Exam technique for multi-part option questions
        - Error log review
        - Targeted re-teach of the weakest option topics
      Projects built:
        - Revision sheet set three: single pages for every option topic
      Practice: Two 45-minute option papers, one per option
    Weeks 63-64: Options Mock
      Topics:
        - A full options paper under timing for your board
        - Marking against an official-style scheme
        - Classifying every lost mark
        - Comparing option performance with core performance
        - Planning the final revision cycle
        - Agreeing the past-paper timetable
      Projects built:
        - The mock file completed with the options mock
      Practice: Mock review set: every lost mark reworked
      Assessment: Options mock, marked and reviewed one to one

---
PHASE 5: REVISION AND THE PAST-PAPER CYCLE (MONTHS 17-18, WEEKS 65-72)

No new content. Structured revision by strand, then full past papers under real timing for your board, marked against official schemes.

Month 17: Structured Revision (Weeks 65-68)
Weeks 65-66: Core Pure Revision by Strand
Topics:
- Rapid re-teach of each student's weakest core topics, chosen from the error log
- Mixed core sets that do not announce their topic
- Formula booklet fluency: what is given and what must be known
- Proof rehearsal from memory
- Accuracy audit: the slips each student makes, named and drilled
- Show-that questions worked backwards honestly

Projects built:
- Final revision pack: the student's revision sheets combined and tidied

Practice: One 75-mark core paper self-timed, plus a drill on two weak strands

Weeks 67-68: Option Revision by Strand
Topics:
- Rapid re-teach of the weakest option topics
- Mixed option sets under timing
- Model assumptions and conclusions rehearsed
- Calculator checks for the option
- Long questions planned before writing
- Error log closed out topic by topic

Projects built:
- Option revision pack completed

Practice: One full option paper self-timed

Assessment: Revision checkpoint: a mixed 60-minute paper across core and options

Month 18: Past Papers Under Timing (Weeks 69-72)
Weeks 69-70: Full Past Papers, Core
Topics:
- Full Core Pure past papers for your board under real timing
- Marking against the official mark scheme
- Error clinic after every paper
- Time management across long questions
- Which questions to attempt first
- Checking strategies in the final minutes

Projects built:
- Past paper score chart started

Practice: Two full core papers under exam timing

Weeks 71-72: Full Past Papers, Options and Final Review
Topics:
- Full option past papers for your board under real timing
- Marking and error clinic
- Final review of the error log
- Exam-day routine and equipment
- Last proof rehearsal
- A calm plan for the final weeks before the exams

Projects built:
- Completed mock file: every marked paper, error classification and score chart

Practice: Two full option papers under exam timing

Assessment: Final past paper series, marked and reviewed

PROJECTS THE STUDENT BUILDS AND KEEPS

- A proof portfolio: every induction proof type and de Moivre's theorem written to exam standard
- A personal question bank: solved exam-style questions for every core and option topic
- A locus atlas and a polar curve gallery, sketched by hand
- A calculus method map covering every further integral form
- Three modelling tasks: a container volume, a bungee jump and a damped oscillation
- A personal error log kept from the first month and used to steer revision
- An AS-standard mock, a Core Pure mock and an options mock, all marked
- The completed mock file with a past-paper score chart

Total projects: A complete revision system for Further Maths: proof portfolio, question bank, sketches, method map, error log and marked mock file

SKILLS THE STUDENT WILL HAVE AT THE END

- The full Further Maths content for your board: Core Pure and two options
- Proof by induction and the writing of complete mathematical arguments
- Complex numbers, matrices and vectors used as tools, not just topics
- Further calculus, Maclaurin series, polar coordinates and hyperbolic functions
- Setting up, solving and interpreting differential equation models
- Timed paper craft and self-marking against official schemes

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week; paper-phase sittings run longer for full timed papers
Practice: 3-4 hours weekly of problem sets, proofs and question-bank building, rising with timed papers in the final months
Review: Homework marked with method-mark commentary; error logs reviewed with the teacher monthly

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for stuck problems
Progress updates: Monthly progress notes to parents with test scores and the error-log summary

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, alongside the student's marked mock series

WHERE THIS CAN LEAD

- University courses that lean heavily on mathematics, such as mathematics, physics, engineering, computer science and economics; entry requirements differ by course, so check each one directly
- A head start on first-year university topics that Further Maths introduces, including matrices, complex numbers and differential equations
- Our statistics and programming courses, which put the applied mathematics to work

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is Further Maths harder than A level Maths?
A: It goes further rather than simply faster. The boards assume you already know A level Maths: OCR's specification says learners are assumed to know the content of GCSE Mathematics and A Level Mathematics. Many students find the step up is in abstraction and in writing complete arguments, which is why we practise proof every month.

Q: Can I take Further Maths online?
A: You can be taught Further Maths online, and this course covers the full content live with a teacher. The exams themselves are sat in person, either at your school or college or at an exam centre that accepts private candidates. We teach the course; your school or exam centre makes the exam entry.

Q: Do I need Further Maths for a maths degree?
A: It depends on the university and the course. Some mathematics courses require Further Maths, some prefer it and some do not ask for it. Check the current entry requirements on the page of each course you are interested in. We teach the subject; we do not give admissions advice.

Q: Which exam boards do you cover?
A: AQA 7367, Edexcel 9FM0, OCR A H245 and OCR MEI H645. All four have a compulsory pure core and a choice of applied or further pure options. Edexcel has two Core Pure papers and two option papers. AQA has two papers on compulsory content and a third on two optional applications. OCR A has Pure Core papers Y540 and Y541 and two optional papers. OCR MEI has the Core Pure paper Y420 plus a major option or minor options. We teach the core to everyone and your entered options in detail.

Q: Can I study Further Maths without A level Maths?
A: Not sensibly on its own. Pearson Edexcel lists no formal prior learning requirement, but says the students who benefit most are likely to hold A level Maths, and OCR assumes its content throughout. If you are not yet studying A level Maths, our A Level Maths course runs alongside this one.

Q: Which options do you teach: mechanics, statistics or decision?
A: All of them, matched to your board: further mechanics, further statistics, decision or discrete mathematics, and further pure options. Tell us which options your school or exam centre has chosen and those are the ones we teach in depth. If the choice has not been made yet, the first lessons include a conversation about which route suits you.

Q: My school does not offer Further Maths. Can I still do it?
A: Yes. This course teaches the whole qualification, and you can sit the exams as a private candidate at an exam centre that accepts private candidates. You arrange the entry with the centre; we make sure the teaching covers every paper on your board's specification.

Q: When should I start?
A: The beginning of Year 12, alongside A level Maths, is the natural point, because Further Maths topics are timed to follow the A level Maths skills they need. Students who join later

take a diagnostic in the first week and get a catch-up plan.

Q: How are the classes run?
A: Live online classes with a real teacher, either in a group of 15 to 20 students at a similar level or one to one. Homework is marked with method-mark comments, and there is WhatsApp support between classes. Current fees for your region are shown in the pricing section of this page.

Q: Can I try a lesson before enrolling?
A: Yes. Book a free demo class first. The teacher works through some Further Maths with you, finds where you are, and suggests whether group or one-to-one teaching fits better. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/a-level-further-maths-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 3 OF 137: A-LEVEL MATHS: PURE, MECHANICS, STATISTICS (EDEXCEL/AQA)

==========

Course link: https://learn.modernagecoders.com/courses/a-level-maths-course-pure-mechanics-statistics
Subject area: Exam Board Mathematics
Designed for: High school
Who can join: A-Level students (Years 12-13 or equivalent), Edexcel, AQA and OCR
Course length: 18 months (72 weeks), joinable any month
Time needed: 2 live classes/week + 3-4 hours practice, rising to full timed papers in the final phase
Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

A-Level Maths course online: live small-batch classes covering Pure, Mechanics & Statistics for Edexcel, AQA & OCR, from AS foundations to timed past papers.

In one line: Two years of the hardest school maths there is, taught in order, at a pace that holds, with past papers at the end instead of panic.

ABOUT THE PROGRAMME

A-Level Maths is a linear qualification: every exam comes at the end, and the content is set nationally by the Department for Education, so Edexcel, AQA and OCR students all study the same mathematics. What differs is how each board arranges its three two-hour papers, and we adjust practice to your board when the paper phase arrives. This 18-month live course follows the natural AS then A2 structure. The first nine months cover AS content: the pure core of algebra, coordinate geometry, trigonometry, exponentials and logarithms, first calculus and vectors, followed by AS statistics with the board's large data set and AS mechanics with kinematics and Newton's laws. Months 10 to 16 cover the A2 material where most grades are won and lost: functions, sequences and series, the full trigonometric toolkit, parametric equations, advanced differentiation and integration, numerical methods, the normal distribution, moments, friction and projectiles. The final two months are given entirely to structured revision and past papers under real timing, marked against official schemes. Classes are live and small, so the teacher sees your working, not just your answers. Method marks decide grades at A-Level, and we treat how a solution is written as seriously as whether it is right.

WHAT MAKES THIS COURSE DIFFERENT

- Built around the real qualification: linear assessment, three two-hour papers, roughly two thirds pure and one third applied, with your board's paper structure driving the practice phase
- AS content is finished and consolidated with an AS-standard mock before A2 begins, so the second year is built on checked ground rather than assumed ground
- The large data set is taught from the first statistics week, because boards write exam questions that assume familiarity with it, and students who meet it in the exam hall lose easy marks
- Calculator technique is taught explicitly: statistical distributions, iteration and equation solving on the calculator are part of the syllabus, not an afterthought
- Two full months of past papers under real 2-hour timing, marked against official mark

schemes, with an error clinic after every paper
- Method-mark discipline throughout: solutions are written the way examiners award marks, with working shown at every step

THE LEARNING PATH, STEP BY STEP

Phase 1: AS Pure: algebra, quadratics, coordinate geometry, binomial expansion, trigonometry, exponentials and logarithms, differentiation, integration and 2D vectors
Phase 2: AS Applied: sampling and the large data set, probability, the binomial distribution and hypothesis testing, then kinematics, variable acceleration and Newton's laws, closed by an AS-standard mock
Phase 3: A2 Pure: proof by contradiction, functions, partial fractions, sequences and series, radians and the full trig toolkit, parametrics, advanced calculus, numerical methods and 3D vectors
Phase 4: A2 Applied: conditional probability, the normal distribution and hypothesis tests for means and correlation, then resolving forces, friction, moments and projectiles
Phase 5: Revision and the past-paper cycle: structured revision by strand, then full papers under timing with marked review, matched to your board's paper structure

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: A strong GCSE or IGCSE Maths pass, ideally grade 6 or above. The week 1 diagnostic finds any gaps and we patch them inside the course
Exam board: Edexcel, AQA or OCR. The content is identical across boards; we align paper practice to yours in the final phase
Equipment: A calculator with statistical distributions and an iterative function, notebooks, and a device with stable internet for live classes
Timing: Best started at the beginning of Year 12 or its equivalent; later joiners get a catch-up plan after the diagnostic

WHO THIS COURSE IS FOR

Year 12 starters: Students beginning A-Level Maths who want the full two years taught in order, with revision built in rather than bolted on
Students without strong school support: Students whose school sets the exams but whose classroom teaching is rushed, oversized or patchy
International students: Students taking A-Levels outside the UK, at international schools or as private candidates, who need structured teaching across the time zones
Retake students: Students resitting A-Level Maths who need the gaps found honestly and closed, not the whole course replayed at the same speed
Applied strugglers: Students comfortable with pure maths who lose marks in statistics wording and mechanics diagrams, or the other way around

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: AS PURE MATHEMATICS (MONTHS 1-5, WEEKS 1-20)

The pure core that every later topic stands on: algebra done properly, coordinate geometry, trigonometry, exponentials and logarithms, and the first calculus. Roughly two thirds of the A-Level is pure mathematics, and it starts here.

Month 1: Algebra Foundations (Weeks 1-4)
Weeks 1-2: Indices, Surds and Quadratics
Topics:
- Diagnostic paper on GCSE essentials: algebra, trigonometry, graphs
- Laws of indices including negative and fractional powers
- Surds: simplifying, expanding and rationalising denominators
- Quadratic graphs, completing the square and the turning point
- Solving quadratics by factorising, formula and completing the square
- The discriminant and what it says about roots
Projects built:
- Question bank entry: 15 solved exam-style questions on indices, surds and quadratics, written to full method standard
Practice: 30 mixed problems across indices, surds and quadratics, with every quadratic solved by the method the question rewards
Weeks 3-4: Simultaneous Equations, Inequalities and First Proof
Topics:
- Simultaneous equations: linear pairs and linear with quadratic
- Where a line meets a curve, algebraically and in a sketch
- Linear and quadratic inequalities with set notation
- Representing inequality regions graphically
- Proof by deduction and by exhaustion
- Disproof by counter example, with the write-up done properly
Projects built:
- Proof portfolio opened: four short proofs written out to exam standard, checked line by line
Practice: 20 simultaneous and inequality problems plus 6 proof questions, working shown

    in full every time
    Assessment: Month 1 test: a 40-minute paper on all algebra so far, marked with method marks the way an examiner would

Month 2: Polynomials, Graphs and Coordinate Geometry (Weeks 5-8)
  Weeks 5-6: Polynomials and Curve Sketching
    Topics:
      - Algebraic division and the factor theorem
      - Factorising and sketching cubics and quartics
      - Reciprocal graphs and their asymptotes
      - Graph transformations: translations and stretches, alone and read from equations
      - Points of intersection between curves
      - Interpreting sketches: what a question actually wants from a graph
    Projects built:
      - Sketching sheet: 10 polynomial and reciprocal curves sketched with intercepts, turning behaviour and asymptotes labelled
    Practice: 25 problems on the factor theorem, division and transformations, each sketch drawn before any algebra is checked
  Weeks 7-8: Straight Lines and Circles
    Topics:
      - Gradient, midpoint and distance between points
      - Equations of straight lines in all three forms
      - Parallel and perpendicular lines, with proofs of perpendicularity
      - The equation of a circle, centre and radius by completing the square
      - Tangents, chords and the perpendicular from the centre
      - Modelling with straight lines: interpreting gradient and intercept in context
    Projects built:
      - Question bank entry: 12 solved coordinate geometry questions, including one full circle-tangent problem
    Practice: 22 line and circle problems with a labelled sketch for each, plus 2 modelling questions answered in context
    Assessment: Month 2 test: a 40-minute paper on polynomials and coordinate geometry with one unstructured multi-step question

Month 3: Binomial Expansion and Trigonometry (Weeks 9-12)
  Weeks 9-10: The Binomial Expansion
    Topics:
      - Factorial notation and nCr, on paper and on the calculator
      - Pascal's triangle and where the coefficients come from
      - Expanding (a + bx)^n for positive integer n
      - Finding a specific coefficient without expanding everything
      - Using expansions for numerical approximations
      - The sign and power slips that cost marks, and how to catch them
    Projects built:
      - Question bank entry: 10 solved binomial questions, including two find-the-coefficient problems worked two ways
    Practice: 20 binomial problems, each coefficient answer checked by an independent second method
  Weeks 11-12: Trigonometry: Ratios, Identities and Equations
    Topics:
      - Sine and cosine rules and the area of a triangle
      - Graphs of sin, cos and tan, and exact values worth memorising
      - The identities tan x = sin x / cos x and sin squared plus cos squared equals 1
      - Solving trig equations in degrees over a given interval
      - Quadratics in sin, cos or tan
      - Finding all solutions: the CAST diagram and graph methods compared
    Projects built:
      - Solution-count drill: 12 trig equations where the number of solutions in the interval is predicted before solving
    Practice: 28 trig problems from triangle work to multi-step equations, every lost solution traced back to its cause
    Assessment: Month 3 test: a 45-minute paper on binomial expansion and trigonometry, marked for method as well as answers

Month 4: Exponentials, Logarithms and First Calculus (Weeks 13-16)
  Weeks 13-14: Exponentials and Logarithms
    Topics:
      - The graphs of a^x and e^x, and why e matters
      - Logarithms as the inverse of exponentials, and the log laws
      - Solving equations of the form a^x = b
      - Natural logarithms and equations with e
      - Exponential modelling: growth, decay and interpreting the constants
      - Reducing relationships to linear form with logs, and reading data plots
    Projects built:
      - Modelling write-up: one exponential growth or decay problem solved and interpreted in three or four written sentences
    Practice: 24 log and exponential problems including 4 modelling questions where the interpretation carries the marks
  Weeks 15-16: Differentiation from First Principles

Topics:
- The gradient of a curve and differentiation from first principles
- Differentiating powers of x, sums and constants
- Tangents and normals to a curve at a point
- Increasing and decreasing functions
- Stationary points and their nature, using the second derivative
- Simple optimisation: setting up and solving maximum and minimum problems

Projects built:
- First-principles proof: the derivative of x squared and x cubed derived and written out to exam standard

Practice: 26 differentiation problems ending with 4 optimisation questions set up from scratch

Assessment: Month 4 test: a 45-minute paper on logs, exponentials and differentiation with one modelling question

Month 5: Integration, Vectors and the AS Pure Finish (Weeks 17-20)

Weeks 17-18: Integration

Topics:
- Integration as the reverse of differentiation
- Indefinite integrals of powers of x and the constant of integration
- Finding the constant from a point on the curve
- Definite integrals and what the answer means
- Areas under curves, including regions below the axis
- Areas between a curve and a straight line

Projects built:
- Area problem set: 8 shaded-region questions where the region is sketched and decomposed before any integration

Practice: 24 integration problems, definite integrals checked for sign sense against the sketch every time

Weeks 19-20: Vectors in Two Dimensions

Topics:
- Vector notation, magnitude and direction
- Adding vectors and multiplying by scalars
- Position vectors and the vector between two points
- Distance between points using vectors
- Geometric problems: parallelograms, ratios and collinearity
- Where vectors reappear: a preview of their role in mechanics

Projects built:
- Question bank entry: 10 solved vector problems including two geometric proofs

Practice: 20 vector problems with a diagram drawn for each, plus a mixed AS Pure warm-down set of 15 questions

Assessment: Phase 1 milestone: a 90-minute AS Pure paper covering all five months, marked against an official-style scheme

---

PHASE 2: AS STATISTICS AND MECHANICS (MONTHS 6-9, WEEKS 21-36)

The applied third of the qualification begins: statistics with the board's large data set, probability and hypothesis testing, then mechanics from kinematics to Newton's laws. The phase closes with a full AS-standard mock.

Month 6: Sampling, Data and the Large Data Set (Weeks 21-24)

Weeks 21-22: Sampling and Data Presentation

Topics:
- Populations, samples and why sampling method matters
- Simple random, systematic, stratified, quota and opportunity sampling
- The large data set: what your board pre-releases and how exams use it, with Edexcel's weather-station data as the worked example
- Histograms, frequency polygons and cumulative frequency diagrams
- Box plots and comparing distributions
- Outliers: the standard rules and when to clean data

Projects built:
- Large data set field notes: a one-page profile of your board's data set in your own words, variables, units and quirks included

Practice: 18 data presentation problems plus a guided exploration session inside the large data set itself

Weeks 23-24: Location, Spread and Interpretation

Topics:
- Mean, median and mode from lists and grouped tables
- Quartiles, percentiles and interpolation
- Variance and standard deviation, by formula and by calculator
- Coding data and what it does to mean and spread
- Scatter diagrams, correlation and interpreting a regression line in context
- Writing statistical conclusions in sentences, not just numbers

Projects built:
- Comparison write-up: two groups from the large data set compared on location and spread, in four marked sentences

Practice: 20 location and spread problems, calculator statistics mode drilled until it is faster than the formula

Assessment: Month 6 test: a 40-minute statistics paper with one large-data-set question in board style

Month 7: Probability, the Binomial and Hypothesis Testing (Weeks 25-28)
  Weeks 25-26: Probability and the Binomial Distribution
    Topics:
      - Venn diagrams and tree diagrams done cleanly
      - Mutually exclusive and independent events, and how to test for independence
      - Discrete random variables and probability distributions
      - The conditions for a binomial model, checked in words
      - Calculating binomial probabilities on the calculator
      - Cumulative binomial probabilities and careful inequality reading
    Projects built:
      - Model-check drill: 8 scenarios judged binomial or not, with the failing condition named each time
    Practice: 24 probability and binomial problems, inequality direction stated in words before the calculator is touched
  Weeks 27-28: Hypothesis Testing with the Binomial
    Topics:
      - Null and alternative hypotheses, written correctly
      - One-tailed and two-tailed tests and significance levels
      - Critical regions and actual significance level
      - Carrying out a test and stating the conclusion in context
      - The wording that earns the final mark, and the wording that loses it
      - Common traps: wrong tail, wrong inequality, conclusions that overclaim
    Projects built:
      - Question bank entry: 8 full hypothesis tests written to exam standard, conclusions in context every time
    Practice: 14 complete hypothesis tests, each conclusion checked against a mark-scheme model sentence
    Assessment: Month 7 test: a 45-minute paper from probability through hypothesis testing

Month 8: Mechanics: Modelling and Motion (Weeks 29-32)
  Weeks 29-30: Modelling and Kinematics
    Topics:
      - SI units and the standard modelling assumptions: particle, light string, smooth surface
      - Displacement, velocity and acceleration as vectors
      - Displacement-time and velocity-time graphs, and what areas and gradients mean
      - The suvat equations: where they come from and when they apply
      - Vertical motion under gravity
      - Choosing a positive direction and sticking to it
    Projects built:
      - Graph story set: 6 motion graphs translated into written descriptions of the trip and back again
    Practice: 22 kinematics problems, the suvat quantities listed and the equation chosen before any algebra
  Weeks 31-32: Variable Acceleration
    Topics:
      - Why suvat fails when acceleration changes
      - Velocity and acceleration by differentiation
      - Displacement by integration, with limits or constants
      - Maximum velocity and turning points of motion
      - Sketching motion from its equations
      - Mixed problems that decide between suvat and calculus
    Projects built:
      - Decision drill: 10 motion problems sorted into suvat or calculus, with the deciding clue underlined
    Practice: 18 variable acceleration problems plus a mixed set of 10 where the method is not announced
    Assessment: Month 8 test: a 40-minute kinematics paper mixing graphs, suvat and calculus methods

Month 9: Forces, Newton's Laws and the AS Checkpoint (Weeks 33-36)
  Weeks 33-34: Forces and Newton's Laws
    Topics:
      - Force diagrams drawn before anything else
      - Newton's laws and F = ma along a line
      - Weight, tension, thrust and normal reaction
      - Connected particles: cars and trailers, lifts and scale pans
      - Pulleys: the standard setups and their equations
      - Forces written as vectors in i and j form
    Projects built:
      - Question bank entry: 10 solved force problems, every one opened with a complete labelled diagram
    Practice: 20 forces problems, the diagram marked before the algebra is even read
  Weeks 35-36: AS Consolidation and Mock
    Topics:
      - Structured revision across all AS pure and applied content
      - Mixed problem sets that do not announce their topic

- A full AS-standard mock under real timing
- Marking workshop against an official-style scheme
- Error clinic: every miss classified as concept, method or accuracy
- The A2 map: what changes in year two and what carries over

Projects built:
- Personal error log formalised: every mock miss classified and matched to a drill

Practice: Redo every mock miss untimed with full working, then one targeted drill set on your weakest AS topic

Assessment: Phase 2 milestone: full AS-standard mock, marked and reviewed one to one

---

PHASE 3: A2 PURE MATHEMATICS (MONTHS 10-14, WEEKS 37-56)

The second-year pure content where most A-Level grades are decided: functions, series, the full trig toolkit, parametrics, advanced calculus, numerical methods and 3D vectors.

Month 10: Proof, Fractions and Functions (Weeks 37-40)

Weeks 37-38: Proof by Contradiction and Partial Fractions

Topics:
- Proof by contradiction: the irrationality of root 2 and the infinitude of primes
- Simplifying algebraic fractions
- Algebraic division revisited with remainders
- Partial fractions with distinct linear factors
- Partial fractions with a repeated factor
- Where partial fractions pay off later: series and integration previewed

Projects built:
- Proof portfolio extended: both classic contradiction proofs written from memory to exam standard

Practice: 18 partial fraction decompositions checked by recombining, plus 4 proof questions

Weeks 39-40: Functions and the Modulus

Topics:
- Mappings, domain and range stated precisely
- Composite functions and order of application
- Inverse functions, their graphs and the line y = x
- The modulus function: graphs of |f(x)| and f(|x|)
- Solving modulus equations and inequalities
- Combined transformations applied in the right order

Projects built:
- Question bank entry: 12 solved function questions including two full modulus problems with sketches

Practice: 24 function problems, domain and range written for every inverse found

Assessment: Month 10 test: a 45-minute paper on proof, partial fractions and functions

Month 11: Sequences, Series and Radian Trigonometry (Weeks 41-44)

Weeks 41-42: Sequences and Series

Topics:
- Arithmetic sequences and series, and the standard formulae
- Geometric sequences and series
- Sum to infinity and the convergence condition
- Sigma notation read and written fluently
- Recurrence relations: increasing, decreasing and periodic sequences
- Modelling with series: savings, loans and depreciation

Projects built:
- Modelling write-up: one savings or loan problem solved with a series and explained in plain sentences

Practice: 26 series problems including 4 modelling questions and 4 sigma notation translations

Weeks 43-44: Radians, Sectors and New Trig Functions

Topics:
- Radian measure and exact values in radians
- Arc length and sector area
- Solving trig equations in radians
- Small angle approximations and where they come from
- Secant, cosecant and cotangent: graphs and identities
- Inverse trig functions and their restricted domains

Projects built:
- Question bank entry: 10 solved radian and sector problems including one segment area question

Practice: 22 problems across radians, sectors and reciprocal trig, calculator mode checked before every set

Assessment: Month 11 test: a 45-minute paper on series and radian trigonometry

Month 12: The Full Trig Toolkit and Parametric Equations (Weeks 45-48)

Weeks 45-46: Compound Angles and the R Form

Topics:
- Compound angle formulae and where they come from
- Double angle formulae and their rearrangements
- Writing a sin x + b cos x in R form
- Using the R form for maxima, minima and equation solving

- Proving trig identities with a clear strategy
- Choosing the right identity: a decision routine for messy equations

Projects built:
- Identity toolkit card: every identity on one page with a note on when each earns its keep

Practice: 24 identity and equation problems, each proof annotated with the identity used at every step

Weeks 47-48: Parametric Equations

Topics:
- Curves defined parametrically and why anyone bothers
- Converting between parametric and Cartesian forms
- Trig parametrics and identity-based conversion
- Sketching parametric curves and finding intersections
- Points where a parametric curve crosses the axes
- Parametric modelling questions in exam style

Projects built:
- Question bank entry: 10 solved parametric problems including two conversions each way

Practice: 18 parametric problems, every conversion checked by substituting a point

Assessment: Month 12 test: a 45-minute paper on advanced trigonometry and parametrics

Month 13: Advanced Differentiation and Integration (Weeks 49-52)

Weeks 49-50: Differentiation Methods

Topics:
- The chain, product and quotient rules, and how to spot which one
- Derivatives of trig, exponential and log functions
- Implicit differentiation
- Parametric differentiation
- Rates of change and connected rates
- Tangents and normals on harder curves

Projects built:
- Method map: one page routing any function to its differentiation method, tested against 20 unseen functions

Practice: 30 differentiation problems climbing from single-rule to mixed-rule questions

Weeks 51-52: Integration Methods

Topics:
- Standard integrals of trig and exponential functions
- Integration by substitution
- Integration by parts
- Integrating with partial fractions
- Areas between curves and the trapezium rule
- Differential equations with separable variables, solved and interpreted

Projects built:
- Question bank entry: 12 solved integrals labelled by method, including one differential equation in context

Practice: 28 integration problems, the method named in the margin before each attempt

Assessment: Month 13 test: a 60-minute calculus paper across both differentiation and integration methods

Month 14: Numerical Methods, 3D Vectors and the General Binomial (Weeks 53-56)

Weeks 53-54: Numerical Methods

Topics:
- Locating roots by change of sign, and when that argument fails
- Fixed point iteration and rearranging into iterative form
- Staircase and cobweb diagrams
- The Newton-Raphson method and its failure cases
- Iteration on the calculator, done quickly and accurately
- Numerical integration revisited and error direction from sketches

Projects built:
- Root hunt: one equation solved three ways, change of sign, iteration and Newton-Raphson, with results compared

Practice: 16 numerical methods problems with calculator iteration technique drilled to fluency

Weeks 55-56: 3D Vectors and the Binomial for Any Power

Topics:
- Vectors in three dimensions: notation, magnitude and distance
- Geometric problems in 3D
- The binomial expansion for negative and fractional indices
- Validity intervals stated every time
- Expansions built through partial fractions
- Using expansions for approximations, with accuracy discussed

Projects built:
- Question bank entry: 10 solved problems split between 3D vectors and general binomial expansions

Practice: 20 problems across 3D vectors and the general binomial, validity written before any expansion

Assessment: Phase 3 milestone: a 90-minute A2 Pure paper across all five months of this phase

---

PHASE 4: A2 STATISTICS AND MECHANICS (MONTHS 15-16, WEEKS 57-64)

The second-year applied content: conditional probability, the normal distribution and its hypothesis tests, then resolving forces, friction, moments and projectiles.

Month 15: The Normal Distribution and Hypothesis Tests (Weeks 57-60)
Weeks 57-58: Conditional Probability and the Normal Distribution
Topics:
- Set notation for probability and the conditional probability formula
- Conditional probability from tables, trees and Venn diagrams
- The normal distribution: shape, parameters and what they control
- Calculating normal probabilities on the calculator
- The inverse normal: finding values from probabilities
- Standardising and finding unknown mean or standard deviation
Projects built:
- Question bank entry: 10 solved normal distribution problems including two unknown-parameter questions
Practice: 22 probability and normal distribution problems, a sketch of the shaded region drawn for every normal question
Weeks 59-60: Testing Means and Correlation
Topics:
- When the normal approximates the binomial, and the continuity correction
- The distribution of the sample mean
- Hypothesis tests for the mean of a normal distribution
- Correlation coefficients and what they do and do not claim
- Hypothesis tests for zero correlation using tables of critical values
- Large data set questions in A2 style, revisited
Projects built:
- Question bank entry: 8 full hypothesis tests for means and correlation, conclusions written in context
Practice: 14 complete tests plus one large-data-set question under a 15-minute clock
Assessment: Month 15 test: a 45-minute A2 statistics paper

Month 16: Resolving Forces, Friction, Moments and Projectiles (Weeks 61-64)
Weeks 61-62: Resolving Forces and Friction
Topics:
- Resolving forces at angles into components
- Equilibrium of a particle under coplanar forces
- Motion on an inclined plane
- Friction and the coefficient of friction
- Limiting equilibrium: on the point of moving
- Connected systems revisited with angles and friction
Projects built:
- Question bank entry: 10 solved problems on inclined planes and friction, every diagram complete before the algebra
Practice: 20 resolving and friction problems, components table filled in before any equation is written
Weeks 63-64: Moments and Projectiles
Topics:
- Moments and the conditions for equilibrium of a rigid body
- Rods on supports and tilting problems
- Projectile motion: splitting velocity into components
- Time of flight, range and greatest height
- The projectile modelling assumptions and their limits
- Motion in two dimensions with vectors and calculus
Projects built:
- Question bank entry: 10 solved moments and projectile problems, including one full tilting question
Practice: 18 moments and projectile problems plus a mixed mechanics set of 10 with no topic labels
Assessment: Phase 4 milestone: a 60-minute A2 applied paper across statistics and mechanics

---
PHASE 5: REVISION AND THE PAST-PAPER CYCLE (MONTHS 17-18, WEEKS 65-72)

No new content, on purpose. Structured revision by strand, then full past papers under real 2-hour timing, matched to your board's paper structure and marked against official schemes.

Month 17: Structured Revision (Weeks 65-68)
Weeks 65-66: Pure Revision by Strand
Topics:
- Rapid re-teach of the weakest pure topics, driven by each student's error log
- Mixed pure problem sets that do not announce their topic
- Formula booklet fluency: what is given, what must be known
- Proof rehearsal: contradiction and identity proofs rewritten from memory
- Algebra accuracy audit: the slips that cost each student marks, named and drilled
- Show-that questions: working backwards from a given answer honestly
Projects built:
- Revision sheet set one: student-made single pages for every pure strand

Practice: One mixed pure paper of 50 marks self-timed, plus a targeted drill on your two weakest strands

Weeks 67-68: Applied Revision and Board Alignment

Topics:

- Statistics mixed sets: distributions, tests and interpretation together
- Mechanics mixed sets: forces, moments, projectiles and kinematics together
- Large data set refresh: the facts worth having cold
- Calculator technique final pass: distributions, iteration, equation solving
- Your board's paper structure: exactly which content sits in which paper for Edexcel, AQA and OCR
- Time budgeting: marks per minute across a 100-mark paper

Projects built:

- Revision sheet set two: single pages for statistics and mechanics, completing the personal revision pack

Practice: One applied sectional paper self-timed plus 20 calculator-technique drills

Assessment: Month 17 checkpoint: two sectional papers marked against official schemes, error logs updated

Month 18: The Past-Paper Cycle (Weeks 69-72)

Weeks 69-70: Papers Under Real Timing

Topics:

- A full past paper from your board in one timed 2-hour sitting
- Marking against the official scheme, method marks understood
- Error clinic one: concept gaps separated from timing and accuracy losses
- A second full paper under timing
- Question selection: order of attack and when to park a question
- The unstructured problem: making progress when no method is named

Projects built:

- Mock file opened: each paper filed with its marked script, error classification and one-line fix per miss

Practice: One additional past paper at home under honest timing, brought marked to class

Weeks 71-72: Final Mocks and Exam Craft

Topics:

- Two final full papers under strict conditions, spaced for recovery
- Error clinic two: closing the last recurring mistakes
- Presentation final pass: notation, working, exact answers where demanded
- The last week: what to revise, what to leave alone
- Exam-day routine for a three-paper series, walked through calmly
- Course close: the mock series reviewed with each student and family

Projects built:

- Completed mock file: the full paper series with scores charted, kept by the student as evidence of readiness

Practice: Light targeted drills only, set individually from each student's final error log

Assessment: Course milestone: final full past paper marked and reviewed one to one, plus certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A personal question bank: 10-15 solved exam-style questions per topic, built across the whole course
- A proof portfolio: every required proof type written to exam standard and rewritten from memory
- A large data set profile and comparison write-ups in the student's own words
- A calculus method map routing any function to the right technique
- A personal error log maintained from the first month and used to steer revision
- Student-made single-page revision sheets for every strand
- An AS-standard mock at month 9 and a full past-paper series in month 18
- The completed mock file: marked scripts, error classifications and a score chart

Total projects: A complete self-built revision system: question bank, proof portfolio, revision sheets, error log and marked mock file

SKILLS THE STUDENT WILL HAVE AT THE END

- The full A-Level Maths content: pure, statistics and mechanics to A2 standard
- Method-mark discipline: solutions written the way official mark schemes pay
- Calculator fluency for distributions, iteration and equation solving
- Large data set familiarity of the kind exam questions quietly assume
- Timed paper craft across three two-hour papers
- Self-review: marking your own work against a scheme and classifying your own errors

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week; paper-phase sittings run longer for full timed papers

Practice: 3-4 hours weekly of problem sets and question-bank building, rising with timed papers in the final months

Review: Homework marked with method-mark commentary; error logs reviewed with the teacher monthly

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for stuck problems
Progress updates: Monthly progress notes to parents with test scores and the error-log summary

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders, alongside the student's marked mock series

WHERE THIS CAN LEAD

- University degrees that ask for A-Level Maths: mathematics, engineering, physics, economics and computer science
- A stronger position for degrees that quietly reward it, from architecture to psychology
- A foundation for university admissions tests such as STEP, MAT and TMUA, which build on A-Level content
- Further Maths, for students who discover they want more
- Our statistics and programming courses, which pick up exactly where the applied content leaves off

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which exam boards does the course cover?
A: Edexcel, AQA and OCR, including OCR's MEI specification by arrangement. The content of A-Level Maths is prescribed nationally, so all boards examine the same mathematics; what differs is how the three two-hour papers are arranged and the house style of the questions.

Q: What is the difference between AS Maths and the full A-Level?
A: AS Maths is a separate, standalone qualification covering roughly the first year of content. Since the reforms, AS results do not count towards the full A-Level: the A-Level is linear, and everything rests on the papers at the end of year two. Our course follows the AS then A2 structure because it is the natural teaching order, and students who want to sit the AS exam as a standalone qualification at the end of year one are prepared for exactly that.

Q: Can you improve my predicted grade?
A: We should be straight about what predicted grades are: they are set by your school or college, not by us, and no tutor can promise to change them. What we can do is give your teachers better evidence: marked tests through the course, an AS-standard mock, and a full past-paper series with scores charted. Students who improve their actual performance tend to see predictions follow, but the honest version is that we prepare you and your school predicts you.

Q: I am retaking A-Level Maths. Does this course work for resits?
A: Yes, and retakes are one of the cases we handle best, because the course does not assume you are starting from nothing. A retake student begins with the diagnostic and their old scripts if available, we map exactly which strands cost the marks, and the plan compresses the content you already hold and expands where you lost ground. Resit students usually join mid-course into the phase that matches their gaps rather than starting at week 1.

Q: What calculator do I need, and do you teach how to use it?
A: We teach calculator technique explicitly, because binomial and normal probabilities, iteration and equation solving on the calculator are worth real marks and are consistently undertaught in schools.

Q: What is the large data set and why does it matter?
A: Each exam board pre-releases a large data set, Edexcel's is weather-station data for example, and students are expected to have worked with it during the course. Exam questions are written assuming that familiarity: knowing the variables, the units and the quirks can turn a long question into a short one.

Q: We are in the UK. How do class timings work with an India-based academy?
A: Classes are scheduled to suit the student, and UK afternoons and evenings fall comfortably inside our teaching hours. Most of our A-Level students take classes on weekday evenings or weekend mornings UK time. Classes are live and interactive in every case, and the same teacher stays with the student through the course.

Q: Can I join in Year 13, or partway through my studies?
A: Yes. The 18-month plan assumes a Year 12 start, but the course is modular enough to enter at the A2 phase or the revision phase. A Year 13 joiner sits the diagnostic, we agree which AS strands need patching, and the plan is compressed to fit the months remaining before the exams. What we will not do is pretend a student ten weeks from their papers can cover two

years of content; if the timeline is unrealistic we say so at the demo stage.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/a-level-maths-course-pure-mechanics-statistics
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 4 OF 137: ABACUS & MENTAL MATHS FOR KIDS: FROM FIRST BEADS TO ANZAN

==========

Course link: https://learn.modernagecoders.com/courses/abacus-mental-maths-course-for-kids
Subject area: Mental Maths for Kids
Designed for: Elementary school
Who can join: Ages 5 to 10, no prior maths needed beyond counting
Course length: 8 months (32 weeks)
Time needed: 2 live classes/week + 10-15 minutes of bead practice daily
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Abacus classes for kids ages 5-10: live online mental maths course from first beads to anzan visualisation and mental-only sums. Small batches, free demo.

In one line: First the beads move on the desk. Then they move in the head.

ABOUT THE PROGRAMME

The abacus is one of the oldest maths teaching tools still in serious use, and it earns its place: a child who adds on an abacus is not memorising answers, they are physically moving quantities and watching place value work. This 8-month live course takes children aged 5 to 10 from holding the abacus correctly to anzan, the stage where the abacus is imagined and sums are done entirely in the head. The progression is deliberate and unhurried. Months 1 and 2 build bead fluency and the small-friend pairs that make 5. Months 3 and 4 cross the tens with big friends and take numbers to three digits. Months 5 and 6 begin visualisation: eyes closed, imaginary beads, short mental rows that grow week by week. Months 7 and 8 add multiplication on the abacus and finish with a mental-only graded test the child can be genuinely proud of. We are straightforward about what this course builds: real arithmetic fluency, comfortable number sense, and the habit of focused daily practice. Those are worth having on their own. We do not dress them up as brain development or IQ gains, because those claims are not ours to make.

WHAT MAKES THIS COURSE DIFFERENT

- A real progression to anzan: visualisation is trained deliberately from month 5, not left to chance after a year of bead drills
- Small-friend and big-friend rules are taught as number bonds, so the abacus work strengthens school maths instead of sitting beside it
- Live classes built for young attention spans: short activity blocks, bead races, dictation games, and a teacher who can see every child's abacus on camera
- Daily practice is 10-15 minutes by design, small enough that families actually keep it up for eight months
- Honest milestones: parents see dictation scores and row lengths, not vague level names
- Class pace grouped by age, because a 5-year-old and a 9-year-old need different speeds through the same material

THE LEARNING PATH, STEP BY STEP

Phase 1: Bead basics and number sense: setting and reading numbers, place value on the rods, and the small friends that make 5
Phase 2: Crossing the tens: big friends, borrowing and carrying, three-digit numbers, and dictation drills at growing speed
Phase 3: Anzan begins: imaginary beads, short mental rows, flash number practice, and listening sums done without touching the abacus
Phase 4: Multiplication on the abacus and the mental-only capstone: longer mental rows and a graded test done entirely in the head

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Counting to 10 comfortably. Everything else, including number bonds, is built in the course

Age: 5 to 10. Batches are grouped by age; the youngest children benefit from a parent nearby during class
Equipment: A student abacus, 17-rod soroban style, which costs a few hundred rupees; we send parents a buying guide before class 1
Reading: Not required. Instructions are spoken and shown; worksheets use numbers, not sentences

WHO THIS COURSE IS FOR

Young starters: Children aged 5 to 7 meeting numbers for the first time, who learn best with something to hold and move
Primary students: Children aged 8 to 10 who want arithmetic to feel easy before bigger school maths arrives
Children who dislike worksheets: Kids who switch off at written sums but light up when maths is beads, races and games
Focus builders: Children whose parents want a short, structured daily practice habit with visible progress
Future mental calculators: Kids who will enjoy the quiet magic of doing sums with an abacus nobody can see

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: BEAD BASICS AND NUMBER SENSE (MONTHS 1-2, WEEKS 1-8)

The abacus in hand: setting and reading numbers, place value on the rods, and the small-friend pairs that make 5. Everything later stands on the fluency built here.

Month 1: Meeting the Abacus (Weeks 1-4)
Weeks 1-2: Beads, Fingers and First Numbers
Topics:
- The parts of the abacus: frame, rods, the bar, one upper bead worth 5, four lower beads worth 1
- Holding and clearing the abacus the correct way, every time
- Finger rules: thumb pushes ones up, index finger brings them down, index works the 5-bead
- Setting and reading the numbers 1 to 9
- Bead patterns as pictures: seeing 7 as 5 and 2 without counting
- Simple bead races: who can set the called number first
Projects built:
- My abacus card: the child draws bead pictures for the numbers 1 to 9 and reads them back from the drawing
Practice: Ten minutes daily: set and read 20 called numbers between 1 and 9, parent calls, child sets, fingers checked in class
Weeks 3-4: Tens, Place Value and First Sums
Topics:
- The tens rod: setting 10 to 99 and reading two-digit numbers
- Place value felt physically: why 34 is three beads on one rod and four on another
- Direct addition: sums like 2 + 6 where beads simply move up
- Direct subtraction: taking beads away without touching the 5-bead
- Mixed setting drills: 3, 30, 33 and hearing the difference
- Number dictation games at a gentle pace
Projects built:
- Two-digit dictation sheet: 20 numbers set from hearing alone, scored and kept as the month 1 record
Practice: Ten minutes daily: 15 direct additions and subtractions within each rod, plus 10 two-digit setting drills
Assessment: Month 1 check: set and read two-digit numbers from dictation, plus 10 direct sums, done live in class

Month 2: Small Friends, the Pairs That Make 5 (Weeks 5-8)
Weeks 5-6: Adding with Small Friends
Topics:
- The small friends: 1 and 4, 2 and 3, the pairs that make 5
- Why 3 + 4 needs a trick when only two lower beads are free
- The small-friend move: add 5, take away the friend
- One rule at a time: +4 as +5 take 1, then +3, +2, +1 cases
- Saying the move aloud while doing it, so the rule settles in
- Friend flashcards: answering the make-5 partner instantly
Projects built:
- Make-5 rainbow chart: the child's own picture of the pairs that make 5, used at home during practice
Practice: Ten minutes daily: 20 small-friend additions with the rule spoken aloud, plus one round of make-5 flashcards
Weeks 7-8: Subtracting with Small Friends
Topics:
- The reverse move: take 5, give back the friend
- Subtraction cases one at a time: -4, -3, -2, -1 through the 5-bead

- Mixing direct sums and small-friend sums in one row
- Rows of three numbers: 4 + 3 - 2 worked straight through
- Accuracy first, speed second: the rule for the whole course
- Month 2 games: friend bingo and beat-the-teacher rows

Projects built:
- Row book started: a notebook of daily rows with date and score, kept for the rest of the course

Practice: Ten to fifteen minutes daily: 15 mixed small-friend sums and 5 three-number rows, logged in the row book

Assessment: Month 2 check: 20 mixed direct and small-friend sums plus three-number rows, done live with finger technique reviewed

---

PHASE 2: CROSSING THE TENS (MONTHS 3-4, WEEKS 9-16)

Big friends, the pairs that make 10, open up sums that cross rods: carrying, borrowing, three-digit numbers and dictation at growing speed.

Month 3: Big Friends, the Pairs That Make 10 (Weeks 9-12)

Weeks 9-10: Adding Over Ten

Topics:
- The big friends: 1 and 9, 2 and 8, 3 and 7, 4 and 6, 5 and 5
- Why 8 + 5 cannot stay on one rod, shown with beads
- The big-friend move: add 10 on the next rod, take away the friend
- Cases in order: +9, +8, +7 down to +1 through ten
- Carrying seen physically: the tens rod clicking up
- Make-10 flashcards until the pairs are instant

Projects built:
- Make-10 rainbow chart to sit beside the make-5 chart at home

Practice: Ten to fifteen minutes daily: 20 big-friend additions with the move spoken aloud, plus make-10 flashcards

Weeks 11-12: Subtracting Across Ten

Topics:
- The reverse move: take 10, give back the friend
- Borrowing felt on the rods: why 12 - 5 reaches to the tens rod
- Subtraction cases in order: -9 through -1 across ten
- Mixed rows using direct, small-friend and big-friend moves together
- Reading the row before moving: planning which rule each number needs
- Steady dictation: teacher calls a row of four one-digit numbers

Projects built:
- Mixed-rules row sheet: 15 rows solved and marked with which friend rule each step used

Practice: Fifteen minutes daily: 15 mixed sums crossing ten and 5 dictated rows, all logged in the row book

Assessment: Month 3 check: 20 sums mixing all rules so far, plus a live dictation round of four-number rows

Month 4: Combined Moves and Three-Digit Numbers (Weeks 13-16)

Weeks 13-14: When Both Friends Work Together

Topics:
- Combined cases: sums like 6 + 7 that need the 5-bead and the tens rod in one move
- Working the combined move slowly, then at row speed
- Two-digit plus one-digit sums: 34 + 8 across the rods
- Two-digit plus two-digit: working rod by rod from the left
- Row stamina: five-number rows of one-digit numbers
- Speed ladders: same row sheet, three timed attempts across the week

Projects built:
- Speed ladder record: one row sheet attempted three times with times and scores charted by the child

Practice: Fifteen minutes daily: 12 combined-move sums and 6 two-digit additions, plus one speed ladder attempt

Weeks 15-16: Three Digits and Real-Life Numbers

Topics:
- The hundreds rod: setting and reading 100 to 999
- Three-digit addition and subtraction, one rod at a time
- Money sums: rupees and coins set on the abacus
- Simple word problems answered on the abacus: bought, spent, left
- Dictation with two-digit numbers in the rows
- Phase 2 wrap-up: every rule on one revision mat

Projects built:
- Shop game record: a pretend-shop worksheet of five purchases totalled and checked on the abacus

Practice: Fifteen minutes daily: 10 three-digit sums, 5 money questions and one dictated row, logged in the row book

Assessment: Month 4 milestone: mixed paper of 30 questions covering all bead rules up to three digits, done in class

---

PHASE 3: ANZAN, THE ABACUS IN THE HEAD (MONTHS 5-6, WEEKS 17-24)

The step this whole course points at: the physical abacus is set aside and the child moves imaginary beads. Rows start tiny and grow week by week.

  Month 5: Picture the Beads (Weeks 17-20)
    Weeks 17-18: First Imaginary Beads
      Topics:
        - What anzan is: seeing the abacus with eyes closed and moving beads that are not there
        - Air abacus: fingers moving over an empty desk while the mind keeps the picture
        - Setting single numbers mentally and saying them back
        - First mental sums: rows of two small numbers, beads imagined
        - Checking every mental answer on the real abacus straight after
        - Why we slow down here: the picture must be solid before speed
      Projects built:
        - Eyes-closed challenge card: ten single-number settings done mentally and verified, signed off in class
      Practice: Ten minutes daily: 10 two-number mental rows with immediate abacus checks, plus air-abacus warm-ups
    Weeks 19-20: Flash Numbers and Longer Pictures
      Topics:
        - Flash practice: numbers shown briefly on screen, set mentally
        - Rows of three one-digit numbers done fully in the head
        - Keeping the bead picture steady when a friend rule fires
        - Two-digit numbers held mentally: tens and ones rods imagined together
        - Confidence rules: wrong answers are checked on beads, never scolded
        - Month 5 game: slow-speed flash rounds as a class
      Projects built:
        - Flash round scorecard: three class flash rounds with scores tracked by the child
      Practice: Ten to fifteen minutes daily: 10 three-number mental rows and one flash practice round on the family device
      Assessment: Month 5 check: live mental round of three-number rows plus flash settings, real abacus used only for verifying

  Month 6: Mental Rows That Grow (Weeks 21-24)
    Weeks 21-22: Five-Number Rows and Listening Sums
      Topics:
        - Rows of four, then five one-digit numbers done mentally
        - Listening anzan: the teacher speaks the row, nothing is shown
        - Holding the running answer while the next number arrives
        - Two-digit mental sums: one addition at a time, pictured on two rods
        - Recovering a lost picture: reset, breathe, rebuild the beads
        - Personal pace: row length moves up only when accuracy holds
      Projects built:
        - Listening log: ten dictated mental rows across the fortnight with scores and row lengths recorded
      Practice: Fifteen minutes daily: 8 mental rows at your current length and 2 listening rows read out by a parent
    Weeks 23-24: Level Test One, Fully Mental
      Topics:
        - Test craft for small people: neat answers, skipped questions returned to, no rubbing out storms
        - Mixed mental paper practice: rows, flash settings, listening sums
        - Two-digit rows of three numbers for the stronger picture-holders
        - Class review of the phase: what got longer, what got faster
        - Preparing the level test with a practice run
        - Celebrating honestly: certificates say what the child can actually do
      Projects built:
        - Level test one: a graded mental-only paper of rows, flash and listening sums, marked and kept
      Practice: Fifteen minutes daily: one full practice paper across the week, split into daily pieces, logged in the row book
      Assessment: Phase 3 milestone: level test one, mental-only, with results shared with parents alongside the row book

---
PHASE 4: MULTIPLICATION AND THE MENTAL-ONLY CAPSTONE (MONTHS 7-8, WEEKS 25-32)

Multiplication arrives on the abacus once addition is second nature, and the course closes with longer mental rows and a graded test done entirely in the head.

  Month 7: Multiplication on the Abacus (Weeks 25-28)
    Weeks 25-26: Tables Meet the Beads
      Topics:
        - Multiplication as repeated addition, shown with bead groups first
        - Tables 2 to 5 practised with bead patterns and chants
        - Setting up a multiplication on the abacus: where each number lives
        - Two-digit times one-digit: 23 x 3 worked rod by rod
        - Answer sense: is 23 x 3 nearer 60 or 600, asked before working
        - Keeping addition sharp: short mental rows as every class's warm-up
      Projects built:
        - Table pattern cards: bead-picture cards for the 2, 3, 4 and 5 tables made by the child

Practice: Fifteen minutes daily: one table chanted, 8 abacus multiplications and 4 mental warm-up rows

Weeks 27-28: The Harder Tables

Topics:

- Tables 6 to 9 with bead patterns, taken at each child's pace
- Two-digit times one-digit across the harder tables
- Carrying inside multiplication: when a product spills to the next rod
- Sharing sums: simple division as dealing beads into groups, kept light
- Mixed papers: additions, subtractions and multiplications together
- Month 7 dictation: rows now include a times question

Projects built:

- Mixed paper folder: three 20-question mixed papers completed, marked and filed across the fortnight

Practice: Fifteen minutes daily: 10 multiplications across tables 6 to 9 and one short mixed paper piece

Assessment: Month 7 check: 20-question mixed paper including two-digit multiplications, done live in class

Month 8: The Mental-Only Capstone (Weeks 29-32)

Weeks 29-30: Longer, Faster, Still Accurate

Topics:

- Mental rows pushed to each child's honest edge: longer or faster, not both at once
- Two-digit mental rows for ready students, one-digit mastery for the rest
- Mental one-digit multiplication for the strongest picture-holders
- Flash rounds at higher speeds, accuracy floor held at 80 percent
- Reading the row book: eight months of scores tell the child's own story
- Capstone paper structure walked through, no surprises

Projects built:

- Row book review page: the child charts their own week 8 versus week 30 row length and speed

Practice: Fifteen minutes daily: capstone-style mixed mental practice split across the week, logged in the row book

Weeks 31-32: The Graded Mental Test and Family Show

Topics:

- Final practice run of the capstone paper with review
- The capstone: a graded mental-only test of rows, flash, listening and times questions
- Family demonstration: each child performs three mental sums for the class and parents
- What the certificate says: the exact skills demonstrated, nothing inflated
- Keeping the skill: a 10-minute weekly routine for after the course
- What comes next: higher abacus levels, school maths courses, and later Vedic maths

Projects built:

- Capstone graded mental-only test, marked and reported to parents with the full row book
- Family demonstration: three mental sums performed live

Practice: Gentle final week: one short mental round daily and rest before the test, confidence over cramming

Assessment: Course milestone: capstone mental-only test and family demonstration, plus certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- My abacus card: bead pictures for the numbers 1 to 9
- Make-5 and make-10 rainbow charts for home practice
- The row book: a daily practice log kept across all eight months
- Two-digit dictation sheets scored month by month
- Speed ladder records: the same sheet timed three times
- Shop game money worksheets totalled on the abacus
- Eyes-closed challenge card marking the start of anzan
- Flash round and listening logs through phase 3
- Table pattern cards for the 2 to 9 times tables
- The capstone graded mental-only test and family demonstration

Total projects: 10+ records and artifacts, from first bead pictures to a graded mental-only test

SKILLS THE STUDENT WILL HAVE AT THE END

- Correct bead technique: setting, reading and clearing with proper fingering
- Small-friend and big-friend rules, which are the number bonds school maths needs anyway
- Addition and subtraction to three digits on the abacus, including money sums
- Anzan: mental rows, flash numbers and listening sums with the abacus imagined
- Times tables 2 to 9 and two-digit by one-digit multiplication on the abacus
- A daily practice habit measured in a row book the child keeps themselves

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week of 45-60 minutes, run in short activity blocks suited to ages 5 to 10

Practice: 10-15 minutes of bead or mental practice daily; short and daily beats long and occasional at this age

Review: Row books and dictation sheets reviewed weekly; technique corrected live on camera before habits set

Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp support for parents between classes, including short videos when a finger rule needs re-showing
Progress updates: Regular progress notes to parents with dictation scores and row lengths, so improvement is visible, not asserted

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Higher abacus levels: longer rows, faster flash work and bigger multiplications
- A smoother start in school maths, with number bonds and place value already solid
- Our school mathematics courses, entered with arithmetic fluency already in place
- Vedic maths from around age 8, once tables are strong, for a second speed system
- Mental maths competitions and school arithmetic quizzes, if the child enjoys performing

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is 5 really old enough for abacus classes?
A: Yes, provided the class is built for that age, and this one is: short activity blocks, bead races instead of lectures, and a teacher watching each child's fingers on camera. A 5-year-old needs to count to 10 and sit for a game-length activity; that is the whole entry bar. For the youngest children we ask a parent to stay nearby for the first few weeks, mostly to help with camera angles and finding dropped beads.

Q: Does abacus training increase IQ or develop the brain?
A: We do not make those claims. What we can show you is what the course actually builds: arithmetic fluency, solid number bonds and place value, mental calculation through anzan, and a daily practice habit, all documented in each child's row book and dictation scores. Those are real and worth eight months. Grander claims about IQ belong to marketing, not to our classroom.

Q: Will abacus methods confuse my child at school?
A: In our experience the two reinforce each other, because the abacus friend rules are exactly the number bonds schools drill: pairs that make 5 and pairs that make 10. A child carrying on the abacus is doing the same regrouping school column methods use, just physically. We name that connection in class so children see one maths, not two competing systems.

Q: When does my child start doing sums mentally?
A: Anzan begins in month 5, after four months of physical bead fluency, and starts tiny: two-number rows with the answer checked on the real abacus. By the end of the course most children handle rows of four or five one-digit numbers mentally, and stronger picture-holders manage two-digit rows. The pace genuinely varies by child, which is why row length only increases when accuracy holds.

Q: How do live online classes work for children this young?
A: Classes run 45 to 60 minutes in short blocks: a warm-up game, a new bead rule, drills, a race or dictation round, and a calm close. Batches are capped at 20 so the teacher can watch every child's abacus and fingers on camera, and correct technique before a habit sets. Parents of the youngest children sit nearby at first; most find they are not needed within a few weeks.

Q: How much home practice is expected?
A: Ten to fifteen minutes a day, logged in a row book the child keeps. At this age short daily practice beats long weekend sessions by a wide margin, and the routine itself, sitting down for ten focused minutes, is one of the most useful things the course leaves behind. Parents mostly just call out numbers for dictation rows; no maths knowledge is needed to help.

Q: How is my child assessed, and is the certificate just for attending?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a friendly final assessment of everything learned. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Can we try a class before enrolling?
A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Bring the

child, with or without an abacus, and the teacher will tell you honestly whether they are ready now or better served starting in a few months.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/abacus-mental-maths-course-for-kids
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 5 OF 137: ACT MATH PREP: LIVE CLASSES FOR THE ACT MATHEMATICS SECTION

==========

Course link: https://learn.modernagecoders.com/courses/act-math-prep-course
Subject area: Test Prep Mathematics
Designed for: High school
Who can join: Grades 9-12 (ages 14 to 18), all starting scores
Course length: 5 months (20 weeks)
Time needed: 2 live one-hour classes per week + weekly timed practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online ACT Math prep for ages 14 to 18: every ACT reporting category, from algebra and functions to geometry and statistics, plus pacing and full timed sections.

In one line: Forty-five questions in fifty minutes. We rebuild the maths underneath, then train the pace.

ABOUT THE PROGRAMME

The ACT mathematics section has 45 questions to answer in 50 minutes, and a calculator is allowed. ACT describes it as testing the skills students have typically acquired in courses taken up to the beginning of grade 12, which means a student sitting it in grade 11 often meets topics they have not been taught yet, or learned once and forgot. At a little over a minute per question, there is no time to rebuild an idea in the exam room. This 5-month live course follows ACT's own published reporting categories. Preparing for Higher Math makes up about 80 percent of the section and is split into Number and Quantity, Algebra, Functions, Geometry, and Statistics and Probability. Integrating Essential Skills makes up about 20 percent: rates, percentages, proportional relationships, area and volume, averages and medians, combined in multi-step problems. Modeling questions are counted across the other categories. We teach every category, starting from a diagnostic so time goes where the points are being lost. The last month turns knowledge into a score you can repeat: pacing plans, calculator strategy for both handheld calculators and the Desmos calculator built into the online ACT, and full timed math sections with question-by-question review. ACT scores are based on the number of questions answered correctly, with no penalty for wrong answers, so students also learn when to move on and how to make sure no question is left blank. We never promise a score; we show the evidence from each timed section so families can see exactly where things stand.

WHAT MAKES THIS COURSE DIFFERENT

- Built on ACT's own published content categories, with time weighted toward Preparing for Higher Math, which makes up about 80 percent of the section
- A diagnostic in week one and an error log kept all course, so teaching goes where points are actually being lost
- Understanding first: topics a student has not yet met in school, such as logarithms, matrices or conic sections, are taught from the start
- Pacing trained explicitly for 45 questions in 50 minutes, with a plan for which questions to bank and which to leave for the second pass
- Calculator strategy for permitted handheld models and for the Desmos calculator in the online ACT
- Honest progress reporting from timed sections, never invented score promises

THE LEARNING PATH, STEP BY STEP

Phase 1: Diagnostic and Integrating Essential Skills: the format, pacing basics, rates, percentages, proportions, area and volume, averages and medians
Phase 2: Preparing for Higher Math: Number and Quantity, Algebra, Functions, Geometry, and Statistics and Probability, each taught then tested under time
Phase 3: Modeling and test craft: modeling questions, calculator and pacing strategy, full timed math sections and a test-week plan

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Grades 9-12 (ages 14 to 18), or students between school and college retaking the ACT
Math background: School algebra at any comfort level; the diagnostic finds the real starting point
Equipment: A computer with stable internet and a permitted calculator. Students taking the ACT online also practise with the Desmos graphing calculator
Time commitment: 2 live hours plus 2-3 practice hours weekly
English: Course taught in English
Motivation: A target test date, or our help choosing one

WHO THIS COURSE IS FOR

Score raisers: Students whose practice results show content gaps in algebra, functions or geometry that need rebuilding, not more drilling
Early testers: Grade 10 and 11 students meeting topics such as logarithms, matrices or conic sections before school has taught them
Strong at school: Students with good grades whose timed results do not match, usually a pacing and method gap
Sat switchers: Students who found the ACT suits them better than the SAT and need ACT-specific pacing
Retakers: Students whose first attempt did not reflect what they know and who want a clear method for the next one

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: DIAGNOSTIC AND INTEGRATING ESSENTIAL SKILLS (WEEKS 1-6)

Find the starting point, learn how the section works, and secure the multi-step essential skills questions that appear throughout the test.

Weeks 1-2: Diagnostic and How ACT Math Works

Topics:
- The format: 45 questions in 50 minutes, calculator allowed
- ACT's reporting categories and roughly how much of the section each one covers
- How scores work: number correct, no penalty for wrong answers, and why no question should be left blank
- A timed diagnostic section and a category-by-category breakdown of the result
- Setting up an error log: content gap, misread question or careless slip
- A first pacing check: where the minutes actually went

Projects built:
- Personal diagnostic report: score by category, top three content gaps and a starting pacing target
- Error log opened with every diagnostic miss classified and corrected

Practice: Rework every diagnostic miss without a time limit and note whether the problem was knowledge, reading or speed

Weeks 3-4: Rates, Percentages and Proportional Reasoning

Topics:
- Unit rates and converting between units
- Percent increase, decrease and successive percentage changes
- Ratios and proportional relationships set up from words
- Distance, rate and time problems
- Reading tables and charts to find the quantity a question needs
- Multi-step problems: planning the chain of steps before calculating

Projects built:
- Word problem translation sheet: twenty phrasings mapped to the equation each one means
- Timed essential skills set with every step written out for review

Practice: Complete a 20-question rates and percentages set under time and log every miss

Weeks 5-6: Area, Volume, Averages and Number Forms

Topics:
- Area of triangles, rectangles, circles and composite figures
- Surface area and volume of prisms, cylinders, cones and spheres
- Mean, median and the effect of adding or removing a value
- Expressing numbers in different forms: fractions, decimals, percents and scientific notation
- Estimation as a check on calculator answers
- Spotting when a question is simpler than it looks

Projects built:
- Formula and method sheet written by the student, with one worked example per idea
- Mixed essential skills timed set reviewed question by question

Practice: Finish a mixed essential skills set and write one sentence on the fastest method for each question

Assessment: Phase 1 checkpoint: a timed mixed set on essential skills with a written review of every miss

---
PHASE 2: PREPARING FOR HIGHER MATH (WEEKS 7-16)

The categories that make up most of the section, taught in the order they build on each other: number, algebra, functions, geometry, then statistics and probability.

Weeks 7-8: Number and Quantity
Topics:
- Integer and rational exponents and the exponent rules
- Simplifying radicals and converting between radical and exponent form
- Complex numbers: i, arithmetic with complex numbers and conjugates
- Vectors: components, addition and magnitude
- Matrices: dimensions, addition and multiplication
- Number properties: factors, multiples, primes and absolute value

Projects built:
- Exponent and radical rule card built from worked examples
- Number and Quantity timed set with errors classified in the log

Practice: Complete a 15-question Number and Quantity set, solving each matrix question by hand and checking with a calculator

Weeks 9-10: Algebra
Topics:
- Linear equations and inequalities, including absolute value
- Systems of linear equations solved by substitution, elimination and graphing
- Writing equations from words and from tables
- Quadratic equations by factoring, completing the square and the quadratic formula
- Radical and exponential equations
- Checking for extraneous solutions

Projects built:
- Algebra method map: which method to reach for first for each equation type
- Two timed algebra sets, the second solved with a calculator check strategy

Practice: Work a 20-question algebra set and rebuild two missed questions with a different method

Weeks 11-12: Functions
Topics:
- Function notation, domain and range, and composition
- Linear, polynomial and radical functions and their graphs
- Piecewise functions read from equations and graphs
- Exponential and logarithmic functions and the link between them
- Transformations: shifts, stretches and reflections
- Key features from graphs: intercepts, maximums, minimums and end behaviour

Projects built:
- Function family gallery: a sketch, an equation and a key feature for each family
- Functions timed set with a graph-first solution for every miss

Practice: Complete a functions set and sketch the graph for every question before choosing an answer

Assessment: Mid-course checkpoint: a full timed math section, compared category by category with the diagnostic

Weeks 13-14: Geometry and Trigonometry
Topics:
- Angles, parallel lines, congruence and similarity
- Right triangles, the Pythagorean theorem and special right triangles
- Trigonometric ratios, the unit circle and simple trigonometric equations
- Circles: arcs, sectors, chords and equations of circles
- Equations of conic sections: circles, parabolas and ellipses
- Coordinate geometry: distance, midpoint, slope and parallel and perpendicular lines

Projects built:
- Geometry recognition sheet: diagrams that signal similar triangles, special right triangles or circle facts
- Geometry timed set reviewed for drawing and labelling habits

Practice: Finish a geometry set, redrawing every figure larger and labelling everything given before solving

Weeks 15-16: Statistics and Probability
Topics:
- Center and spread: mean, median, mode, range and standard deviation as an idea
- Data collection methods: surveys, experiments, samples and bias
- Bivariate data, scatterplots and lines of best fit
- Probability of single and combined events
- Sample spaces, counting outcomes and two-way tables
- Expected value in simple situations

Projects built:
- Statistics vocabulary and method sheet with one example of each idea
- Statistics and probability timed set with every miss rewritten as a lesson

Practice: Complete a statistics and probability set and explain each answer in one plain sentence

Assessment: Phase 2 checkpoint: a timed mixed Preparing for Higher Math set with a written review of every miss

---

PHASE 3: MODELING AND TEST CRAFT (WEEKS 17-20)

Knowledge becomes a repeatable score: modeling questions, calculator workflows, pacing under real timing, and full timed sections with honest review.

Weeks 17-18: Modeling, Calculators and Pacing

Topics:

- Modeling questions: building, reading, judging and improving a model
- Handheld calculator strategy and which features are not permitted
- Using the Desmos calculator built into the online ACT
- A two-pass pacing plan for 45 questions in 50 minutes
- When to guess, flag and move on, since there is no penalty for wrong answers
- Checking answers in the last minutes without losing time

Projects built:

- Personal pacing plan tested and adjusted across two timed sections
- Calculator workflow notes: which question types are faster with the calculator and which are faster by hand

Practice: Sit one full timed math section using the pacing plan and record the time spent on each question

Weeks 19-20: Full Timed Sections and Test Week

Topics:

- Full timed math sections under exact conditions
- Question-by-question review against the reasoning, not just the answer key
- Turning the error log into a final list of fixes
- Last topics targeted from the log
- Test week plan: sleep, timing, calculator check and what to bring
- Deciding whether and when to retake, using the evidence

Projects built:

- Two full timed math sections with complete written reviews
- Final error log summary with the five fixes that matter most

Practice: Sit one full timed section each week and bring the reviewed paper to class

Assessment: Final assessment: a full timed math section plus a review conversation on pacing and method choices

PROJECTS THE STUDENT BUILDS AND KEEPS

- Personal diagnostic report by ACT reporting category
- Error log kept and reviewed across the whole course
- Formula, method and recognition sheets written by the student
- Personal pacing plan and calculator workflow notes
- Full timed math sections with written reviews

Total projects: A complete ACT math prep kit: diagnostic report, method sheets, error log and reviewed timed sections

SKILLS THE STUDENT WILL HAVE AT THE END

- Integrating Essential Skills: rates, percentages, proportions, area, volume, averages and medians
- Number and Quantity: exponents, radicals, complex numbers, vectors and matrices
- Algebra: linear, quadratic, radical and exponential equations and systems
- Functions: linear, polynomial, radical, piecewise and logarithmic, with transformations
- Geometry: similarity, right triangles, trigonometry, circles and conic sections
- Statistics and Probability: center, spread, data collection, bivariate data and sample spaces
- Modeling questions across all categories
- Pacing and calculator strategy for a 50-minute section

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes with your teacher
Timed practice: 1-2 timed sets or a full math section each week
Review: Every timed set reviewed for reasoning, not just answers
Total per week: About 4-5 focused hours
Homework: A problem set after every class, and a monthly mixed-review assignment that brings back earlier categories

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week
Mentorship: One teacher across the whole course
Progress tracking: Error log and timed-section results shared with parents
Flexibility: Slots move around school exams and family plans

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Readiness for test day, with every ACT math category practised under real timing
- Ready for AP Precalculus, AP Calculus or AP Statistics, with algebra and functions secure
- Our SAT Math course, if a student decides to sit both tests
- College mathematics courses, started with fewer gaps

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is on the ACT math section?
A: 45 questions in 50 minutes, and a calculator is allowed. ACT reports two main categories: Preparing for Higher Math, about 80 percent, split into Number and Quantity, Algebra, Functions, Geometry, and Statistics and Probability; and Integrating Essential Skills, about 20 percent, covering rates, percentages, proportional relationships, area and volume, and averages and medians. Modeling questions are counted across the other categories.

Q: Can I use a calculator on the ACT math section?
A: Yes. ACT allows four-function, scientific and graphing calculators unless a model is prohibited, and calculators with computer algebra system functionality are not allowed. Students taking the ACT online have the Desmos graphing calculator built into the testing platform during the math section. We teach both, and we teach which questions are faster by hand. Always check ACT's current calculator policy before test day.

Q: Is there a penalty for wrong answers?
A: No. ACT scores are based on the number of questions answered correctly, with no penalty for incorrect answers. That is why our pacing plan makes sure every question gets an answer, even the ones a student flags and leaves for the end.

Q: My child has not taken pre-calculus yet. Is that a problem?
A: It is common. ACT says the section covers skills typically acquired in courses up to the beginning of grade 12, so many grade 11 students meet topics such as logarithms, matrices, complex numbers or conic sections before school has covered them. The course teaches those topics from the start rather than assuming them.

Q: How does the math section affect the composite score?
A: ACT's Composite score is the average of the English, math and reading scores, rounded to the nearest whole number. Science is optional and is not part of the Composite. Scores run up to 36 for the Composite and for each section.

Q: Should my child take the ACT or the SAT?
A: Both are widely accepted, and the right choice depends on the student. We suggest taking a practice test of each and comparing. If the SAT turns out to suit better, we also run a SAT Math course, and much of the algebra and function work carries across.

Q: When should we start?
A: Ideally five months before the planned test date, so all categories and the full timed sections fit before test day. Students with less time get a shortened plan based on their diagnostic, focused on the categories where they are losing the most points.

Q: Do you guarantee a score increase?
A: No, and we would be wary of anyone who does. What we do promise is a clear diagnostic, teaching aimed at the categories where points are being lost, regular timed sections, and honest reporting so you can see the progress for yourself.

Q: How are the classes run?
A: Live online classes, twice a week, with a real teacher, in a group of 15 to 20 students, a mini batch of up to 6, or one to one. Current fees for your region are shown in the pricing section of this page.

Q: Can we try a class first?
A: Yes. Book a free demo class. The teacher works through some ACT-style questions with your child, sees where they are, and suggests a plan and whether group, mini batch or one-to-one classes fit best. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/act-math-prep-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 6 OF 137: ADVANCED GIT & GITHUB MASTERCLASS FOR PROFESSIONALS
==========

Course link: https://learn.modernagecoders.com/courses/git-github-advanced-version-control-masterclass-for-professionals
Subject area: Version Control & Developer Tools
Designed for: College students
Who can join: Intermediate to advanced (working professionals; basic Git assumed or fast-tracked)
Course length: 24 classes (12 weeks · 2 classes/week)
Time needed: 4-5 hours/week recommended
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Advanced Git & GitHub masterclass for professionals: master Git internals, interactive rebase, trunk-based team workflows, monorepos, GitHub Actions CI/CD and repository security in 24 live online classes for working developers.

  In one line: Stop fearing Git. Build the mental model, master history surgery, and become the engineer your team calls when version control goes wrong.

ABOUT THE PROGRAMME

  Most working developers rely on the same handful of Git commands and quietly avoid everything else: rebase feels risky, reflog is a mystery, and a leaked secret in history triggers panic. This masterclass fixes that permanently, the way professionals need it fixed: from the inside out. In 24 live, instructor-led classes you first learn how Git actually stores data (blobs, trees, commits, refs, the three trees), because once the model is clear, nothing Git does is scary. Then you go deep into history mastery (interactive rebase, cherry-pick, bisect run, reflog rescues), team workflow design at scale (trunk-based development vs GitFlow, branch protection rulesets, CODEOWNERS, monorepo tooling, release management), and full GitHub platform automation: GitHub Actions matrix builds and reusable workflows, gated deployments, Dependabot, secret scanning, CodeQL, signed commits, hooks, and GitHub CLI scripting. You finish with a leadership phase covering team migrations, disaster recovery with git filter-repo, and AI-assisted review workflows, then a capstone where you design and defend a complete production-grade workflow on a real repository. Git is free, open source, and created by Linus Torvalds in 2005 for Linux kernel development; two decades later, deep fluency in it remains one of the most durable skills a working developer can build.

WHAT MAKES THIS COURSE DIFFERENT

  - Internals-first: you learn the object model, refs, and the three trees before any advanced command, so rebase, reset, and reflog become predictable instead of frightening.
  - Built for professionals: every class is framed around real workplace scenarios, the broken main branch, the leaked API key, the 3,000-line PR, the release that must ship tonight.
  - Live, interactive classes with a real instructor: you run the commands, break real repositories, and recover them under guidance, never passive video watching.
  - Full platform depth, not just commands: branch protection rulesets, CODEOWNERS, merge queues, GitHub Actions, environments, OIDC deploys, CodeQL, and the gh CLI are covered as thoroughly as Git itself.
  - Living Syllabus: the curriculum is updated continuously as Git and GitHub ship new features, so you always learn the current toolchain, not a frozen snapshot.
  - A capstone you can apply at work on Monday: you leave with a fully hardened reference repository (workflow, CI/CD, security, release process) and the reasoning behind every choice.

WHAT A STUDENT NEEDS BEFORE STARTING

  Device: A laptop or desktop (Windows, Mac, or Linux) with a terminal and a reliable internet connection; every tool used in the course is free
  Coding experience: You should already write code in some language and have touched Git (clone, add, commit, push). Rusty is fine: Classes 1-4 rebuild everything from the ground up, just at a professional pace
  Accounts: A free GitHub account covers the entire course: the Free plan includes unlimited public and private repositories and 2,000 GitHub Actions minutes per month, which is more than the exercises need
  Mindset: Willingness to break repositories on purpose. The course is built around controlled disasters and recoveries, because that is how the fear goes away

WHO THIS COURSE IS FOR

  Working developers: Developers who use Git daily but lean on the same few commands and want the deep fluency that makes everything else routine

Team leads: Leads and managers who need to design branching, review, and release workflows for a team and enforce them with the right guardrails
Career switchers: Career switchers who already code and want the professional Git and GitHub habits that interviews and first jobs quietly assume
Devops and platform engineers: Engineers moving toward DevOps or platform roles who need GitHub Actions, deployment gating, and supply chain security in depth
Open source maintainers: Maintainers and aspiring maintainers who want clean history, scalable review, and a professional release process for their projects

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE GIT MENTAL MODEL

Learn how Git actually works underneath: the object database, refs and HEAD, the staging area, and the plumbing commands that make every advanced operation predictable instead of scary.

Class 1: How Git Actually Stores Data: The Object Model
Topics:
- Git as a content-addressable database: every file, folder, and commit is an object named by the hash of its content
- The four object types: blobs (file contents), trees (directories), commits (snapshots plus parent pointers), and annotated tags
- SHA-1 vs SHA-256 object formats: why the hash is the identity, and what the ongoing SHA-256 transition means for repositories
- Guided tour of the .git directory: objects/, refs/, HEAD, index, config, and what actually changes on disk when you commit
- Inspecting any object with plumbing: git cat-file -t <sha> and git cat-file -p <sha> to walk from a commit to its tree to a blob by hand
- Writing objects manually with git hash-object -w, and why Git stores whole snapshots, never diffs
- Loose objects vs packfiles: git gc, delta compression, and why full history costs less disk than people assume
Projects built:
- Trace one of your own commits end to end with git cat-file: from the commit SHA to the root tree to a specific blob, and diagram the object graph you found
Practice: Run git cat-file -p HEAD, then cat-file -p the tree it references, and keep descending until you reach a blob.
Assessment: Explain the four Git object types and draw the object graph for a two-file commit from memory.

Class 2: Refs, HEAD, Detached HEAD and the Three Trees
Topics:
- What a branch really is: a tiny file in refs/heads containing one commit SHA, nothing more
- HEAD as a symbolic ref: git symbolic-ref HEAD, and what detached HEAD actually means (HEAD pointing at a SHA instead of a branch)
- Working safely in detached HEAD: inspecting old commits, building from a tag, and rescuing new work with git switch -c
- Revision syntax fluency: HEAD~2 vs HEAD^2, @{upstream}, main@{yesterday}, and resolving any expression with git rev-parse
- The three trees model: HEAD, the index (staging area), and the working directory, and how every core command moves content between them
- git reset --soft vs --mixed vs --hard explained precisely through the three trees, plus git restore and git switch as the modern split of checkout
- Remote-tracking refs: refs/remotes/origin/main as your last-known snapshot of the server, and how fetch vs pull update it
Projects built:
- Build a personal reset cheat card: run soft, mixed, and hard resets on a scratch repository and document exactly what happened to HEAD, index, and working tree in each case
Practice: Check out a raw commit SHA, make a commit in detached HEAD, then rescue it onto a new branch with git switch -c.
Assessment: Predict the exact effect of git reset --mixed HEAD~1 on all three trees before running it, then verify your prediction.

Class 3: Porcelain vs Plumbing and the Staging Area Deep Dive
Topics:
- Porcelain vs plumbing: the user-facing commands vs the scriptable core, and why plumbing output stays stable for automation
- Reading the index directly: git ls-files -s shows mode, SHA, and stage number for every staged file
- Building a commit with pure plumbing: git update-index, git write-tree, and git commit-tree, so commits stop being magic
- Surgical staging: git add -p hunk selection (y, n, s to split, e to edit the hunk) for clean, reviewable commits
- Unstaging and untracking precisely: git restore --staged vs git rm --cached vs git rm, and what each leaves on disk

- gitignore mechanics: pattern rules, negation with !, why an already-tracked file ignores .gitignore, and debugging with git check-ignore -v
- Scripting-safe status: git status --porcelain=v2 and git diff --name-only as building blocks for your own tools

Projects built:
- Create a real commit using only plumbing commands (update-index, write-tree, commit-tree), then verify it with git log and git cat-file

Practice: Take a deliberately messy working directory and use git add -p to split it into two clean, single-purpose commits.

Assessment: Explain the difference between git rm --cached and git restore --staged, and demonstrate both on a test file.

Class 4: Config Layers, Aliases and Diff/Merge Tooling

Topics:
- The config cascade: system, global, local, and worktree levels, audited with git config --list --show-origin
- Conditional includes with includeIf gitdir: separate work and personal identities (name, email, signing key) automatically per directory
- Power aliases: a graph log alias, and shell aliases with the ! prefix for anything Git cannot do natively
- Better conflicts by default: merge.conflictStyle zdiff3 to see the common ancestor inside conflict markers
- rerere (reuse recorded resolution): letting Git remember how you resolved a conflict and replay it automatically next time
- Wiring difftool and mergetool: using VS Code or your editor of choice for three-way merges instead of raw conflict markers
- Cross-platform hygiene: core.autocrlf and .gitattributes eol rules so Windows, Mac, and Linux teammates stop fighting over line endings

Projects built:
- Ship a dotfiles-quality global .gitconfig: identities via includeIf, five aliases you will actually use, zdiff3 conflict style, rerere enabled, and a configured mergetool

Practice: Trigger the same merge conflict twice in a scratch repo and watch rerere resolve it automatically the second time.

Assessment: Show git config --list --show-origin and explain which layer each of your key settings comes from and why it lives there.

PHASE 2: HISTORY MASTERY

Rewrite, rescue, and interrogate history with confidence: interactive rebase, cherry-pick and revert, reflog and stash recovery, automated bisect, and blame and pickaxe archaeology.

Class 5: Interactive Rebase: Rewriting History with Intent

Topics:
- git rebase -i HEAD~n: the todo list (pick, reword, edit, squash, fixup, drop) and how Git replays commits one at a time
- The autosquash workflow: git commit --fixup=<sha>, git rebase -i --autosquash, and rebase.autosquash true to make it the default
- Splitting a commit with edit: stop mid-rebase, git reset HEAD~1, restage in pieces, then git rebase --continue
- exec lines in the todo list: running your test suite after every replayed commit so each commit in the branch builds
- git rebase --update-refs: rewriting a stack of dependent branches in a single pass
- Precision transplants with git rebase --onto: moving a branch from one base to another exactly
- The golden rule: never rewrite pushed shared history, and git push --force-with-lease as the seatbelt when rewriting your own branch

Projects built:
- Take a deliberately messy 10-commit feature branch and rebase it into 3 clean, logical, test-passing commits using fixup, squash, reword, and an exec test line

Practice: Use git commit --fixup plus rebase --autosquash to fold a typo fix into the commit where it belongs.

Assessment: Explain when rebase is safe versus dangerous, and demonstrate --force-with-lease rejecting a push over a stale remote.

Class 6: Cherry-pick, Revert and Fixing Shared History

Topics:
- git cherry-pick <sha> and ranges (A..B vs A^..B): replaying exact changes onto another branch
- cherry-pick -x for traceability back to the original commit, and -n (--no-commit) to batch several picks into one commit
- Backporting in practice: moving a hotfix from main onto a release branch without dragging unrelated work along
- git revert: the only safe undo on shared history, and why it adds a new commit instead of deleting anything
- Reverting a merge commit with revert -m 1: what mainline means, and the trap waiting when you later re-merge that branch
- Reverting a revert: bringing previously reverted work back cleanly
- Conflict handling mid-operation: --continue, --abort, and --skip, and reading the conflict to know which side you actually want

Projects built:
- Simulate a release: branch release/1.2 from an older commit, cherry-pick two hotfixes onto it with -x, then revert a bad feature merge on main with revert -m 1

Practice: Cherry-pick a range of three commits onto a clean branch and resolve the one conflict it produces.

Assessment: State when to use revert versus reset versus rebase on a shared branch, with one concrete scenario for each.

Class 7: Reflog Rescues, Lost Commits and Stash Mastery

Topics:
- git reflog: the private journal of everywhere HEAD has been, plus per-branch reflogs with git reflog show <branch>
- Undoing a bad reset --hard: locate the pre-reset entry and restore with git reset --hard HEAD@{1}
- Resurrecting a deleted branch: recover its tip SHA from the reflog and recreate it with git branch <name> <sha>
- Deep recovery: git fsck --lost-found and dangling commits when the reflog cannot help, and how reflog expiry works
- Stash beyond the basics: git stash push -m with a message, --include-untracked, and stashing selected hunks with stash push -p
- apply vs pop, inspecting a stash with git stash show -p stash@{n}, and turning an old stash into a branch with git stash branch
- Built-in bookmarks: ORIG_HEAD and @{1}, the markers Git leaves behind before every dangerous operation

Projects built:
- Run a rescue drill: deliberately destroy work three different ways (reset --hard, branch -D, dropped stash) and recover all three using reflog and fsck

Practice: Stash only half of your working changes with stash push -p, then apply them on a different branch.

Assessment: Given a repo where a branch was force-deleted an hour ago, recover it fully and explain each command you used.

Class 8: git bisect: Finding the Exact Commit That Broke It

Topics:
- Binary search over history: why bisect pinpoints the bad commit in a handful of steps instead of hundreds of diff readings
- The manual loop: git bisect start, git bisect bad, git bisect good <sha>, test, mark, repeat, then git bisect reset
- git bisect skip for commits that cannot build, and how Git routes the search around them
- Full automation: git bisect run ./test.sh, with exit code 0 meaning good, 1-124 meaning bad, and 125 meaning skip
- Bisecting anything scriptable: failing unit tests, performance regressions via a timing script, even bundle size growth
- git bisect terms with --term-old and --term-new for hunts about behavior changes rather than breakage
- Reading the verdict: the first-bad-commit report, then git show and git blame on it to understand the real cause

Projects built:
- In a prepared repository with a hidden regression, write a small test script and let git bisect run find the first bad commit completely automatically

Practice: Bisect the same repository manually and count your steps to feel the binary search converge.

Assessment: Explain the meaning of exit codes 0, 1, and 125 in a bisect run script and give a case where you would return each.

Class 9: History Archaeology: blame, Pickaxe and Log Forensics

Topics:
- git blame beyond basics: -w to ignore whitespace, -C -C to track code moved between files, and -L 10,30 for a line range
- Keeping blame honest: blame.ignoreRevsFile with a .git-blame-ignore-revs file so mass reformatting commits stop hiding real authors
- The pickaxe: git log -S 'someString' to find every commit that added or removed a string, with -p to see the diffs
- Regex archaeology: git log -G '<pattern>' for changes matching a pattern even when the occurrence count stayed the same
- Function-level history: git log -L :myFunction:path/to/file to follow one function through time, and --follow across renames
- Shaping output for investigation: --oneline --graph --all, --author, --since, --grep, and custom --pretty=format: strings
- Choosing the right tool: pickaxe when you know the code, bisect when you know the symptom, blame when you know the line

Projects built:
- Answer three archaeology questions on a real open source repository (when was X introduced, who last changed Y and why, which commit removed Z) using blame, -S, and -G, citing exact SHAs

Practice: Use git log -L to follow one function through several refactors and renames.

Assessment: Given a mystery behavior change, produce the introducing commit SHA plus the command trail that found it.

PHASE 3: TEAM WORKFLOWS AT SCALE

Design how a whole team uses Git: pick the right branching strategy, enforce it with
protection rules and CODEOWNERS, build a PR culture that scales, tame big repositories, and
run professional releases.

  Class 10: Branching Strategies: Trunk-Based vs GitFlow vs GitHub Flow
    Topics:
      - GitHub Flow: branch, PR, review, merge, deploy, the default loop for continuously deployed
        products
      - GitFlow honestly assessed: develop, release, and hotfix branches, where it still fits
        (versioned, installed software) and where it hurts
      - Trunk-based development: short-lived branches merged to main within a day or two, backed by
        strong CI
      - Feature flags as the enabler: shipping incomplete work safely to main instead of letting
        branches age for weeks
      - Merge strategy tradeoffs: merge commits vs squash-merge vs rebase-merge, and what each does
        to history, bisect, and blame
      - Long-lived branch pathology: drift, giant conflicts, and review fatigue, and how batch size
        drives team speed
      - A decision framework: choosing a strategy from team size, release cadence, deployment model,
        and compliance requirements
    Projects built:
      - Write a one-page branching strategy proposal for a team you know: chosen model, merge
        strategy, branch naming, and the reasoning behind each decision
    Practice: Merge the same two branches three ways (merge, squash, rebase) in a scratch
    repo and compare git log --graph for each result.
    Assessment: Defend a recommended strategy for two scenarios: a 4-person startup deploying
    daily, and a 40-person team shipping quarterly releases.

  Class 11: Branch Protection, Rulesets and CODEOWNERS
    Topics:
      - Branch protection rules vs the newer rulesets: what each can enforce, and why rulesets can
        layer and apply across an organization
      - Required pull request reviews: approval counts, dismissing stale approvals on new commits,
        and requiring review from Code Owners
      - Required status checks: making green CI a hard merge gate, and strict mode requiring the
        branch to be up to date first
      - CODEOWNERS syntax in depth: path patterns, teams vs individuals, and the gotchas (last
        matching rule wins, owners need write access)
      - Blocking force pushes and deletions on protected branches, plus require linear history and
        how it interacts with squash and rebase merges
      - Bypass lists and enforcement for admins: granting emergency exceptions without switching
        protection off
      - Push protection and required signed commits as organization policy: stopping secrets and
        unverified commits at the door
    Projects built:
      - Configure a demo repository with a full ruleset: required reviews with code owners, required
        status checks, linear history, and a CODEOWNERS file covering two teams, then prove each rule
        blocks what it should
    Practice: Open a PR that violates each rule in turn (no review, failing check, force
    push) and document exactly what GitHub blocks.
    Assessment: Explain last-match-wins in CODEOWNERS using a worked example of a pattern
    ordering that surprises people.

  Class 12: Pull Request Culture That Scales
    Topics:
      - PR templates in .github/PULL_REQUEST_TEMPLATE.md: prompting authors for context, testing
        notes, and rollout risk every time
      - Draft PRs for early feedback, and converting to ready-for-review once CI is green
      - Review mechanics that save hours: committable suggested changes, batching comments into one
        review, and viewed-files tracking
      - Writing reviews people accept: comment on the code not the person, separate blocking issues
        from nits, always explain the why
      - Keeping PRs small and stacked: splitting a large change into a chain of dependent PRs and
        landing them in order
      - Auto-merge and merge queues: letting approved, green PRs land themselves without last-minute
        conflicts on busy repositories
      - Review load balancing: round-robin and load-based team review assignment so reviews never
        bottleneck on one senior
    Projects built:
      - Design a complete PR playbook for a repository: template, review checklist, size guidelines,
        and auto-merge or merge queue settings, then run one real PR through it end to end
    Practice: Review a prepared flawed PR and leave at least five comments, using committable
    suggestions wherever possible.
    Assessment: Split a provided large change into a sensible stack of three PRs and explain
    the landing order.

Class 13: Monorepos, Submodules and Scaling Big Repositories
Topics:
- Monorepo vs polyrepo: atomic cross-project changes and a single-version policy versus independent cadence and access isolation
- Submodules honestly: git submodule add, update --init --recursive, how the parent pins an exact SHA, and the classic detached-HEAD-inside-a-submodule trap
- git subtree as the low-ceremony alternative: vendoring a repository with history and pushing changes back upstream
- sparse-checkout in cone mode: git sparse-checkout init --cone and set <dirs> to work on one slice of a huge monorepo
- Partial clone: git clone --filter=blob:none for full history without full content, versus shallow --depth 1 clones and their limitations
- Git LFS for large binaries: git lfs track, .gitattributes, and migrating existing history with git lfs migrate import
- Repo health at scale: git maintenance start, the commit-graph, and background prefetch to keep big repositories fast

Projects built:
- Rescue a bloated demo repo: move its binary assets to Git LFS with lfs migrate, set up cone-mode sparse-checkout for one team's directory, and measure clone size before and after using --filter=blob:none

Practice: Add the same dependency repo as a submodule on one branch and a subtree on another, update upstream, and pull the change in both ways.
Assessment: Recommend monorepo or polyrepo for a described organization and name the specific Git features that make your choice workable.

Class 14: Release Management: Tags, SemVer and Changelogs
Topics:
- Semantic versioning as a contract: what MAJOR.MINOR.PATCH each promise to consumers, plus pre-release and build metadata suffixes
- Annotated vs lightweight tags: git tag -a v2.1.0 -m, why releases always get annotated (or signed with -s) tags, and pushing them explicitly
- git describe: turning any commit into a human-readable version string for build stamping and support tickets
- GitHub Releases: creating a release from a tag, attaching artifacts, and auto-generated release notes configured via release.yml categories
- Conventional Commits (feat:, fix:, BREAKING CHANGE:) and how they enable automated changelogs and version bumping
- Release branches and the hotfix flow: cutting release/2.1, hardening it, and backporting fixes with cherry-pick -x
- Tag discipline: why a published tag must never be moved, and the correct recovery when a bad tag ships

Projects built:
- Run a full release cycle on your own project: conventional commits, cut a release branch, create an annotated signed v1.0.0 tag, publish a GitHub Release with categorized auto-generated notes, then ship a v1.0.1 hotfix

Practice: Rewrite five real commit messages into Conventional Commits format and predict the version bump each one implies.
Assessment: Explain the difference between annotated and lightweight tags and demonstrate why release tooling only trusts the former.

PHASE 4: AUTOMATION, CI/CD & SECURITY

Automate everything and lock it down: GitHub Actions from fundamentals to reusable workflows, secret management and gated deployments, supply chain security, signed commits, hooks, and GitHub CLI scripting.

Class 15: GitHub Actions: The Core Model
Topics:
- Anatomy of a workflow: .github/workflows/*.yml, events, jobs, steps, runners, and reading logs in the Actions tab
- Trigger fluency: on push with branch and path filters, pull_request vs pull_request_target (and why the latter is dangerous), workflow_dispatch with inputs, and schedule with cron
- Jobs and runners: ubuntu, windows, and macos images, needs: for dependencies between jobs, and parallel-by-default execution
- Using marketplace actions safely: actions/checkout and setup actions, and pinning by tag versus full commit SHA
- Contexts and expressions: github and runner contexts, if: conditions, and wiring step outputs into later steps
- The GITHUB_TOKEN: what it can touch, and shrinking it with workflow-level and job-level permissions: blocks
- The free tier in real numbers: GitHub Free includes 2,000 Actions minutes per month plus unlimited public and private repositories, so every exercise in this course runs at no cost

Projects built:
- Build a CI workflow from scratch for a small app: lint, test, and build on every pull request, with path filters so documentation-only changes skip CI

Practice: Add a workflow_dispatch trigger with an input and use its value inside a step expression.
Assessment: Read an unfamiliar failing workflow run and identify the failing job, step,

and root cause from the logs alone.

Class 16: Advanced Actions: Matrix Builds, Caching and Reusable Workflows
Topics:
- strategy.matrix: testing across language versions and operating systems from one job definition, with include and exclude for edge cells
- fail-fast and max-parallel: controlling what happens to the rest of the matrix when one cell fails
- actions/cache: key design with hashFiles on your lockfile, restore-keys fallback chains, and verifying the cache actually saves time
- Artifacts vs cache: upload-artifact and download-artifact for passing build outputs between jobs and preserving test reports
- Reusable workflows with workflow_call: one CI definition shared across many repositories, with inputs and secrets passed explicitly
- Composite actions: bundling repeated step sequences into your own action under .github/actions/
- concurrency groups with cancel-in-progress: stopping superseded runs on the same PR instead of burning minutes

Projects built:
- Upgrade your Class 15 pipeline: a six-cell OS-by-version matrix, dependency caching, a test-report artifact, and the whole pipeline extracted into a reusable workflow called from a second repository

Practice: Measure one workflow's runtime with a cold cache and a warm cache and record the difference.
Assessment: Design a cache key strategy for a given project and justify the restore-keys ordering.

Class 17: Secrets, Environments and Deployment Pipelines
Topics:
- The secrets hierarchy: repository, organization, and environment secrets, plus log masking and its limits
- GitHub environments: production vs staging with required reviewers, wait timers, and environment-scoped secrets
- Deployment jobs: attaching environment: production to a job and tracking deploy history in the repository's Environments view
- OIDC federation: short-lived cloud credentials for AWS, Azure, or GCP instead of storing long-lived keys as secrets
- The build-once pattern: build a single artifact, deploy that same artifact to staging, then promote it to production through an approval gate
- Deploying to GitHub Pages: free static site hosting wired directly from an Actions workflow
- Rollback and safety: re-running a previous successful deployment, and protecting the deploy branch so only reviewed code ships

Projects built:
- Build a two-stage deploy pipeline: build an artifact, auto-deploy to a staging environment, then a production deploy to GitHub Pages gated behind a required reviewer

Practice: Rotate a repository secret and confirm the workflow picks up the new value with no file changes.
Assessment: Explain why OIDC beats long-lived cloud keys in CI and sketch the trust relationship that makes it work.

Class 18: Supply Chain Security: Dependabot, Secret Scanning and CodeQL
Topics:
- The modern threat model: most of a repository's code is dependencies, and attacks arrive through them and through leaked credentials
- Dependabot alerts and security updates: automatic PRs that bump vulnerable dependencies, and triaging them without alert fatigue
- dependabot.yml version updates: scheduled freshness PRs per ecosystem and directory, with grouping to cut the noise
- Secret scanning and push protection: GitHub detecting known credential patterns and blocking the push before the secret ever lands
- CodeQL code scanning: enabling default setup, reading alerts as data-flow paths, and gating PRs on newly introduced alerts
- Hardening Actions themselves: pinning third-party actions to full commit SHAs, least-privilege GITHUB_TOKEN, and reviewing workflow changes like code
- Dependency review on pull requests: seeing new and changed dependencies with their advisories before merge

Projects built:
- Fully harden a demo repository: Dependabot alerts plus weekly grouped version updates, push protection, and CodeQL default setup, then deliberately introduce a vulnerable dependency and a planted fake secret and document what each defense caught

Practice: Triage three Dependabot alerts: decide update now, ignore, or investigate, and justify each call.
Assessment: Explain the difference between Dependabot alerts, version updates, and security updates in plain language.

Class 19: Signed Commits, 2FA, Hooks and the pre-commit Framework
Topics:
- Why signing matters: anyone can set user.name and user.email to any value, so signatures are the only real authorship proof

- SSH-based signing, the easy modern path: gpg.format ssh, user.signingkey, commit.gpgsign true, and registering the key as a signing key on GitHub
- The Verified badge and vigilant mode: making unsigned commits attributed to you stand out
- Account hygiene: two-factor authentication, and fine-grained personal access tokens with minimal scopes and expiry dates
- Client-side hooks: pre-commit, commit-msg, and pre-push, why they are per-clone by default, and sharing them with core.hooksPath
- The pre-commit framework: .pre-commit-config.yaml, pre-commit install, and running formatters, linters, and secret detectors on staged files only
- Server-side thinking: what pre-receive hooks do, and how GitHub replaces them for hosted repos with protection rules and push protection

Projects built:
- Set up verified signed commits with an SSH key, then add a pre-commit configuration running a formatter, a linter, and a secret detector, and prove a planted fake credential cannot be committed

Practice: Write a commit-msg hook that rejects messages not matching Conventional Commits format, and test it both ways.
Assessment: Show a Verified badge on your own GitHub commit and explain the chain of trust that produces it.

Class 20: GitHub CLI (gh) and API Automation
Topics:
- gh auth login and gh auth status: authenticated automation from any terminal once, then scriptable forever
- The PR workday without a browser: gh pr create --fill, gh pr checkout, gh pr view --web, gh pr review --approve, gh pr merge --squash
- Issue automation: gh issue list --label bug --json number,title for structured output, and gh issue create from scripts
- Watching CI from the terminal: gh run list, gh run watch, and gh run rerun --failed
- Raw API access: gh api repos/{owner}/{repo}/pulls with --jq filters and --paginate for full result sets
- GraphQL for the hard questions: gh api graphql for data the REST API returns poorly, like nested review threads
- Gluing it together: gh inside shell scripts and Actions workflows via GH_TOKEN, plus gh alias set for daily shortcuts

Projects built:
- Write a morning-review script with gh: list PRs awaiting your review with their CI status and age as a clean table, then alias it to a single command

Practice: Open, review, and squash-merge a pull request entirely from the terminal without touching the browser.
Assessment: Use gh api with --jq to answer a repository question (for example, the five most recently updated open PRs) in one command.

PHASE 5: LEADERSHIP & CAPSTONE

Operate at the level teams depend on: migrate an organization to GitHub, recover from disasters and leaked secrets, run AI-assisted review workflows responsibly, and ship a capstone workflow you can defend.

Class 21: Migrating a Team to GitHub and Teaching Git Well
Topics:
- Migration inventory: repositories, history depth, binaries destined for LFS, users and permissions, CI jobs, and the integrations that will break
- Full-fidelity moves: git clone --mirror plus git push --mirror to carry every branch and tag, and importer tooling for other hosts
- Cutover planning: freeze windows, read-only on the old host, repointing remotes with git remote set-url, and a tested rollback plan
- Designing team conventions: branch naming, commit style, PR rules, and a CONTRIBUTING.md people actually read
- Teaching Git so it sticks: mental model first (objects, refs, three trees), commands second, and running an internal workshop
- The failure modes you will meet: force-push accidents, giant binary commits, enormous PRs, and the specific guardrail that prevents each
- Measuring adoption: PR cycle time, review coverage, and revert rate as signals the new workflow is actually working

Projects built:
- Produce a complete migration runbook for a fictional 10-developer team moving to GitHub: timeline, mirror commands, permission mapping, guardrail configuration, and a one-hour onboarding workshop outline

Practice: Mirror-migrate a multi-branch repository between two remotes and verify every branch and tag arrived intact.
Assessment: Deliver a five-minute explanation of the three trees model as you would to a junior teammate, without notes.

Class 22: Disaster Recovery: filter-repo, Leaked Secrets and Corrupt Repos
Topics:
- First rule of a leaked secret: rotate the credential immediately, because forks and clones already have the history, cleanup is step two

- git filter-repo, the maintained replacement for filter-branch: removing a file from all history with --path plus --invert-paths
- Redacting content everywhere: filter-repo --replace-text to scrub passwords and keys from every commit that ever contained them
- The aftermath of a rewrite: every SHA changes, force-push coordination, teammates re-clone or hard-reset, and open PRs need rebasing
- Recovering from a bad force-push: your own reflog, teammates' clones as backups, and why --force-with-lease should be muscle memory
- Corrupt repository triage: reading git fsck output, copying missing objects from a healthy clone's .git/objects, and knowing when to re-clone
- Prevention as the real strategy: push protection, pre-commit secret detectors, and protected branches make most of this class unnecessary

Projects built:
- Run a leak drill: plant a fake API key five commits deep, then execute the full response: rotate (simulated), scrub with filter-repo --replace-text, force-push, and verify with git log -S that no trace remains

Practice: Break a scratch repository by deleting an object file, diagnose it with git fsck, and repair it from a second clone.
Assessment: Write the incident checklist you would follow for a production secret leak, in the correct order, with commands.

Class 23: AI-Assisted Git Workflows: Copilot and Coding Agents

Topics:
- Copilot code review on GitHub: requesting an AI review on a pull request and treating its findings as a first pass, never a verdict
- Access reality: GitHub Copilot has a free tier you can practice with; check github.com/features/copilot for what the current plans include before relying on any feature
- Coding agents that open PRs: routing AI-authored branches through the same protection rules, status checks, and CODEOWNERS as human code
- Git hygiene for agent work: small scoped branches per task, descriptive PR bodies, and never letting an agent force-push shared history
- AI-generated commit messages and PR summaries: where they genuinely help, and why the diff remains the source of truth
- Team policy for AI code: disclosure, review depth, and the license and provenance questions your organization must answer
- The reviewer's edge: your archaeology skills (blame, pickaxe, bisect) are exactly what keeps AI-speed development safe

Projects built:
- Run one feature through an AI-assisted flow: have an AI tool draft the change on a branch, then apply your full professional gauntlet (protection rules, CI, line-by-line review) before merging, and write up what the AI got right and wrong

Practice: Request a Copilot review on a pull request and compare its findings against your own review of the same diff.
Assessment: Draft a one-page AI usage policy for a team's Git workflow covering branches, review requirements, and accountability.

Class 24: Capstone: Design a Complete Team Workflow + Certification

Topics:
- The capstone brief: take a real repository, yours or a provided one, and productionize it end to end
- Workflow layer: a written branching strategy, protected main with a ruleset, CODEOWNERS, a PR template, and an auto-merge or merge queue decision
- CI/CD layer: a matrix test workflow with caching, a build artifact, and a staged deploy behind an environment approval gate
- Security layer: Dependabot, secret scanning with push protection, CodeQL, signed commits, and actions pinned to SHAs
- Release layer: Conventional Commits, an annotated signed tag, and a GitHub Release with generated notes
- The defense: walk a mentor through every choice and justify it against the alternatives, exactly like an architecture review
- Certification: present the finished repository and earn the Modern Age Coders Advanced Git & GitHub certificate

Projects built:
- Ship the capstone: one repository with the complete workflow, CI/CD, security, and release stack configured and demonstrably working, plus a README section documenting every design decision

Practice: Do a dry-run presentation of your capstone to a classmate and incorporate at least one piece of their feedback.
Assessment: Final review: every guardrail demonstrated live (a blocked push, a failing check gate, a gated deploy) earns the certificate.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: a hand-traced object graph of your own commit, a three-trees reset cheat card, a commit built from pure plumbing commands, and a dotfiles-quality .gitconfig with identities, aliases, zdiff3, and rerere
- Phase 2: a messy 10-commit branch rebased into 3 clean test-passing commits, a simulated release with cherry-picked hotfixes and a merge revert, a three-way rescue drill recovered via reflog and fsck, an automated bisect run hunt, and three archaeology questions answered

with cited SHAs
- Phase 3: a written branching strategy proposal, a fully enforced ruleset with CODEOWNERS proven by blocked PRs, a complete PR playbook run end to end, an LFS plus sparse-checkout rescue of a bloated repo, and a full v1.0.0 release cycle with a signed tag and generated notes
- Phase 4: a from-scratch CI workflow, a matrix build with caching extracted into a reusable workflow, a two-stage gated deploy to GitHub Pages, a fully hardened repo that catches a planted vulnerability and secret, a signed-commit plus pre-commit guardrail setup, and a gh-powered morning-review script
- Phase 5: a complete team migration runbook, a leak drill scrubbed with filter-repo and verified clean, one feature shipped through a professionally reviewed AI-assisted flow, and the capstone: a production-grade repository with workflow, CI/CD, security, and release layers demonstrated live
- Total: 24 hands-on projects, one per class, finishing with a production-grade capstone repository

Total projects: 24 hands-on projects, one per class, finishing with a production-grade capstone repository

SKILLS THE STUDENT WILL HAVE AT THE END

- Git Internals: the object model, refs and HEAD, the three trees, and plumbing commands like cat-file and rev-parse
- History Surgery: interactive rebase with autosquash and exec, cherry-pick, revert on shared history, and filter-repo rewrites
- Rescue Operations: reflog recovery, fsck, automated git bisect run, advanced stash workflows, and corrupt repo triage
- Team Workflow Design: trunk-based development vs GitFlow, rulesets, CODEOWNERS, PR templates, stacked PRs, and merge queues
- Repositories at Scale: monorepo tooling, sparse-checkout, partial clone, submodules vs subtree, and Git LFS migration
- CI/CD with GitHub Actions: matrix builds, caching, artifacts, reusable workflows, environments, OIDC, and gated deployments
- Security: Dependabot, secret scanning with push protection, CodeQL, signed commits, and the pre-commit framework
- Automation and Leadership: GitHub CLI and API scripting, team migrations, release management, and teaching Git to others

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, about 1 hour each, across 12 weeks
Hands on git work: 1.5-2 hours running the commands from class on your own repositories, including the deliberate break-and-recover drills
Project time: 1-1.5 hours completing each class project on your own machine
Review and doubts: 30-60 minutes reviewing your work and clearing doubts over WhatsApp before the next class
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for stuck rebases, workflow errors, and setup questions
Small batches: Small batches, with mini-batch and 1-on-1 options, so every professional gets individual attention
Mentor guidance: Live guidance from mentors who run these workflows on real production repositories
Project feedback: Feedback on every class project and a personal capstone review before certification
Living syllabus: Curriculum updated as Git and GitHub ship new features, at no extra cost

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Senior developer trusted with repository administration, history surgery, and release management
- Team lead or engineering manager who designs and rolls out the team's branching and review workflow
- DevOps or platform engineer building CI/CD pipelines and deployment gates with GitHub Actions
- Build, release, or developer-experience engineer at organizations running large or monorepo codebases
- The go-to Git person on any team: the one who rescues lost work instead of suggesting a re-clone

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this Git course too advanced if I only know basic commands like add, commit, and push?
A: No, that is exactly the starting profile it is designed for. The course assumes you have used basic Git at work or in projects, and Classes 1-4 rebuild your understanding from the object model up, so the advanced material lands on solid ground. If you can clone a repo and make a commit, you are ready.

Q: How is this different from a beginner Git and GitHub course?
A: Beginner courses teach you the commands; this one teaches you the system. We cover Git internals, interactive rebase, reflog recovery, branch protection rulesets, monorepo tooling, GitHub Actions CI/CD, and repository security, the skills of the person a team calls when Git goes wrong. Our separate college-students course covers the foundations track.

Q: Do I need a paid GitHub plan to take this course?
A: No. If you are a verified student at a degree or diploma granting school, the free GitHub Student Developer Pack adds further benefits while you study.

Q: Does this course cover GitHub Actions and CI/CD pipelines?
A: Yes, in depth: six full classes. You go from workflow anatomy and triggers through matrix builds, caching, artifacts, and reusable workflows, to secrets, environments with approval gates, OIDC cloud deployments, and supply chain security with Dependabot, secret scanning, and CodeQL. By the capstone you build a complete gated deploy pipeline yourself.

Q: Are the classes live or pre-recorded?
A: Every class is live and interactive with a real instructor, never pre-recorded videos. Group batches are capped at 20 professionals, with mini-batch and 1-on-1 options, and you get WhatsApp doubt support between classes. Modern Age Coders has taught live online classes this way since 2020.

Q: Does the course teach how to remove a leaked password or API key from Git history?
A: Yes, Class 22 is a full disaster recovery session. You run a complete leak drill: rotate the credential first, scrub every trace from history with git filter-repo --replace-text, coordinate the force-push, and verify the cleanup with git log -S. You also set up the defenses (push protection and pre-commit secret detectors) that stop the leak from happening again.

Q: Can my company enrol a team for corporate Git training?
A: Yes. Book a free demo class and we will discuss batch scheduling for your team's timezone; we have taught learners across 25+ countries. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: What certificate do I get after completing the course?
A: You earn the Modern Age Coders 'Advanced Git & GitHub' certificate by presenting and defending your Class 24 capstone: a repository with a complete workflow, CI/CD, security, and release stack that you demonstrate live. You also leave with the repository itself and a GitHub profile full of verified signed commits, which usually says more than the certificate does.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final project defended live, plus a written assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/git-github-advanced-version-control-masterclass-for-professionals
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 7 OF 137: AI AGENTS WITH MICROSOFT COPILOT STUDIO FOR COLLEGE STUDENTS
==========

Course link: https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-course-for-college-students
Subject area: AI Agents & Automation
Designed for: College students
Who can join: Intermediate (college students and early-career, ages 18+; no prior agent experience needed)
Course length: 24 classes (12 weeks · 2 classes/week)
Time needed: 4-5 hours/week recommended
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: one to one Rs 4,999
Outside India, per month: one to one $150
Classes: one to one in India one live class a week, four a month; one to one outside India

two live classes a week, eight a month. Each class is one hour.
Note: this course is taught one to one only. There is no group or mini batch option.
Note: the course length above is counted at two classes a week. In India the one to one plan is one class a week, so if an Indian family asks how long it will take, hand over to Shivam Sir.

WHAT THIS COURSE IS

Build business-grade AI agents in Microsoft Copilot Studio: 24 live 1-on-1 online classes on knowledge grounding, connectors, agent flows, MCP, autonomous triggers, multi-agent orchestration and a recruiter-ready published capstone.

In one line: Recruiters no longer ask whether you have used AI. They ask what you have built with it. Build, ground, automate and publish real business agents, in private 1-on-1 classes.

ABOUT THE PROGRAMME

Every placement season now has a new line on the checklist: AI agents. Microsoft Copilot Studio is the graphical, low-code studio enterprises use to build and manage AI-powered agents and workflows, and by Microsoft's own count 90% of the Fortune 500 use it. This course takes a college student from zero to a business-grade, published agent in 24 live 1-on-1 classes. You start with agent fundamentals done properly: instructions that steer behaviour, test sets, and the model lineup behind every agent (GPT-4.1 as the default, GPT-5.5 Chat, and Anthropic Claude models among them). Then you go deep where business agents actually live: knowledge grounding across SharePoint, Dataverse and connector-indexed enterprise data with citations users can check; conversation design with topics, entities, condition groups and adaptive cards; and generative orchestration, where the model plans across your topics, knowledge and tools. Phase 4 is the employability core: tools built from Copilot Studio's library of more than 1,400 connectors, deterministic agent flows for work that must run the same every time, custom connectors and REST APIs, Model Context Protocol (MCP) tools, autonomous event triggers with the security discipline they demand, multi-agent design with child agents and the agent-to-agent (A2A) protocol, and analytics with agent evaluations to prove your agent works. You finish by publishing to real channels, passing a professional pre-flight checklist, and defending a capstone agent built for a real club, department or small business, with a documentation packet a recruiter can actually read. The syllabus is a Living Syllabus, updated as Copilot Studio ships, and it maps directly to Microsoft's credential ladder: PL-900, the Applied Skills labs, and the AI Agent Builder Associate certification (exam AB-620).

WHAT MAKES THIS COURSE DIFFERENT

- 1-on-1 only, by design: your mentor reviews your actual instructions, knowledge sources, flows and test logs line by line every class, and points every project at your real placement goals, which a group batch cannot do honestly.
- The full business toolchain, not a demo tour: connectors, custom connectors, REST APIs, Model Context Protocol, agent flows, event triggers, multi-agent design and agent evaluations, each one practised on a project you keep.
- Certification-mapped: the syllabus vocabulary tracks Microsoft's PL-900 exam (which added a dedicated Copilot Studio agents domain in July 2026) and the AI Agent Builder Associate certification (exam AB-620), so course time doubles as credential prep.
- Autonomy taught with adult supervision of the risks: event triggers run with the maker's credentials, so you learn the security reasoning enterprises are hiring for, not just the wow demo.
- Evidence over vibes: from Class 4 you keep fixed test sets, and by Phase 4 you use analytics outcomes and agent evaluations to prove improvements with numbers, the habit that separates builders from users.
- Living Syllabus: Copilot Studio ships continuously (MCP tools, the A2A protocol, agent evaluations and new models all landed within the last year), and the curriculum is re-verified against Microsoft's documentation as features land.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop (Windows, Mac or Linux) with the Chrome or Edge browser and a reliable internet connection; Copilot Studio runs fully in the browser
Coding experience: None required. Copilot Studio is a graphical, low-code studio, and the course teaches every concept from first principles. Students who do code will get further faster in the REST, custom connector and MCP classes, but the syllabus assumes nothing
Accounts: Copilot Studio uses a Microsoft work or school (Entra ID) account rather than a personal email. Most college email addresses qualify for the 30-day self-serve trial (note: trial environments cannot publish agents). Where a college account is not available, we arrange a practice environment for the duration of the course
Mindset: The willingness to test before you trust, and to treat instructions, sources and logs as engineering artifacts. The course rewards students who ask for evidence

WHO THIS COURSE IS FOR

Placement seekers: Final and pre-final year students who want a demonstrable AI agent skill on their resume, with a published capstone and documentation packet recruiters can actually inspect

Business and commerce students: BBA, BCom and management students who will never be full-time programmers but will absolutely be asked to automate processes and build departmental agents
Engineering students: CS and IT students who know code but not the low-code agent stack enterprises deploy, and want the governance and orchestration vocabulary interviews now probe
Club and fest organisers: Students already running societies and fests who have real users, real FAQs and real processes, the perfect training ground for business-grade agents
Early career professionals: Recent graduates in operations, support or analyst roles who want to become the person on the team who ships the agent instead of requesting one

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: AGENT FOUNDATIONS

Understand agents in business terms, tour Microsoft Copilot Studio properly, write instructions that actually steer behaviour, test like a professional, and meet the models and harnesses underneath every agent.

Class 1: Agents as a Business Skill, Not a Toy
Topics:
- Chatbot vs assistant vs agent: instructions, knowledge, tools and autonomous action, defined precisely
- Why this is on placement checklists: Microsoft reports 90% of the Fortune 500 use Copilot Studio, and Gartner projects a third of enterprise software will include agentic AI by 2028
- Microsoft Copilot Studio in one sentence: a graphical, low-code studio for building and managing AI-powered agents and workflows
- The studio tour that matters: agents, knowledge, topics, tools, triggers, channels, analytics, and the Build, Preview, Evaluate and Monitor surfaces
- Where Copilot Studio sits: low-code SaaS on Power Platform, beside Microsoft Foundry for pro-code teams, and how the two interoperate
- Access done honestly: Copilot Studio needs a Microsoft work or school account; most college emails qualify for the 30-day trial, and your mentor sets your practice environment up with you
- Choosing your capstone candidate: a real club, department or small business whose problem you will ship an agent for in Class 24

Projects built:
- Agent teardown, business edition: analyse one real customer-facing agent (bank, airline, university helpdesk), reverse-engineer its probable instructions, knowledge and tools, and write a one-page teardown

Practice: Draft five agent ideas from your college and internship life and rank them by business value and by how much real knowledge each needs.
Assessment: Define agent, knowledge grounding and tool in your own words, with one business example each.

Class 2: Your First Agent, Built Properly
Topics:
- Describe-to-create: starting an agent from a plain-English description and reading what the platform scaffolds for you
- The instruction stack for business agents: identity, audience, scope, tone, format rules and refusals, in that order
- The test panel as your workbench: probing the agent the moment it exists, and logging behaviour instead of trusting memory
- One-change iteration: modify a single instruction line, retest, compare, keep or revert
- Names and descriptions users trust: why a precise agent description is both UX and findability
- What the model actually does with instructions: system prompts, context and why wording changes behaviour
- Version discipline from day one: keeping instruction versions outside the tool

Projects built:
- Build 'Placement FAQ v1': an agent with hand-written instructions that answers questions about one real company's hiring process in a tone you designed

Practice: Write three instruction variants for the same agent and record exactly how each changes the answers to a fixed set of five questions.
Assessment: Show a before-and-after instruction change with the precise behaviour difference it caused.

Class 3: Instructions at Business Depth
Topics:
- Scope fences: declaring what the agent must not do, and proving the fence holds under pressure
- Refusal design for business contexts: legal, HR and personal-data questions the agent must decline gracefully
- Brand voice as an instruction problem: the same agent for students, recruiters and professors
- Format contracts: forcing answers into steps, tables or short summaries that busy readers actually use
- Instruction smells: vague verbs, contradictions, and rules that fight each other in production

- The deep reasoning preview, honestly framed: Microsoft's reasoning models are in preview and triggered by the keyword 'reason' in instructions, and current syllabi treat them as preview, not promises
- A/B instruction testing: scoring two instruction sets against the same 10 questions

Projects built:

- The Refusal Gauntlet, corporate edition: 10 questions your Placement FAQ agent must refuse or redirect (offer-letter legal advice, personal data, salary speculation), tuned until all 10 are handled gracefully

Practice: Have two friends try to drag your agent out of scope for ten minutes each; log every breach and patch the instructions.

Assessment: Defend each line of your instruction stack: what specific failure appears if that line is deleted?

Class 4: Testing Like Someone Whose Name Is on It

Topics:

- Fixed test sets: 15 questions with expected behaviour written before you run them
- Happy path, edge case, adversarial: three question classes every business agent must survive
- Grading answers on four axes: correctness, tone, format and scope
- Manual regression: rerunning the full set after every meaningful change, and why professionals never skip it
- Recognising hallucination before grounding exists to fix it: confident nonsense and how to catch it
- The test log as an artifact: question, expected, actual, verdict, and why recruiters respond to this document
- Preview of Class 20: Copilot Studio's agent evaluations automate exactly this habit, so building it by hand first makes the tool obvious later

Projects built:

- Build the permanent 15-question test set for Placement FAQ, run it twice in one week, and file the two most instructive failures with root causes

Practice: Write five adversarial questions designed to make your agent overstep, and score how it handles each.

Assessment: Given a conversation transcript, mark every agent turn pass or fail against the written instructions, with reasons.

Class 5: Models, Harnesses and What You Are Actually Running On

Topics:

- The model picker: GPT-4.1 as the current default, GPT-5 Chat and GPT-5.5 Chat generally available, and Anthropic Claude models (Sonnet 4.6, Opus 4.6 and 4.7) in the lineup
- Why model choice is a design decision: retesting your fixed set across two models and defending the differences
- Harnesses in plain words: the runtime between your design and the model, with a standard harness for topics and flows and a GitHub Copilot harness for reasoning-heavy work
- Environments: why your practice environment is separate from a company's production environment, and what that separation protects
- Copilot Credits, first pass: agent usage is metered (classic answers, generative answers and actions cost different amounts), and designers are expected to know it
- Platform guardrails you inherit: content filtering and responsible-AI defaults that run before your instructions
- Phase 1 systems map: from a user's message through harness, instructions and model to the answer, labelled by what you control

Projects built:

- Model comparison memo: run your full Placement FAQ test set on two available models and write a one-page recommendation with evidence

Practice: Draw the full journey of one question through your agent and annotate the three points where you can change the outcome.

Assessment: Phase 1 gate: rebuild a small agent from a written spec in 20 minutes, with instructions, refusals and a five-question test.

PHASE 2: KNOWLEDGE & GROUNDING AT BUSINESS DEPTH

Ground agents in the knowledge sources enterprises actually use: SharePoint, documents, Dataverse and connector-indexed enterprise data, with citations, conflict handling and accuracy audits you publish.

Class 6: Grounding vs Guessing, the Business Version

Topics:

- Why ungrounded agents are unemployable: training data vs your organisation's data
- The knowledge source lineup: public websites, uploaded documents, SharePoint, Dataverse, and enterprise data indexed through connectors
- Generative answers: how the agent searches sources and composes answers, and where citations come from
- The 'Allow ungrounded responses' setting and why business agents keep it off
- Real limits that force curation: website and SharePoint URL caps, file ceilings, and a hard maximum of knowledge sources per agent
- Source selection as a professional judgment: authoritative, current, small enough to verify
- Grounding plan for your capstone: the source list you will defend on demo day

Projects built:

- Ground Placement FAQ in the target company's public website and prove five answers now carry

citations a recruiter could check
Practice: Ask your grounded agent five questions its sources cannot answer and document its behaviour at the edge of knowledge.
Assessment: Explain grounding, citations and hallucination in interview language: two minutes, no notes.

Class 7: SharePoint and Documents Done Right
Topics:
- SharePoint as a knowledge source: pointing an agent at the document library a team actually maintains
- Writing documents for retrieval: headings, one fact per paragraph, consistent names, and why messy docs produce messy agents
- Document formats that ground well, and the file-size and count ceilings that shape what you upload
- Precision testing: questions whose answers live in one exact paragraph, and verifying the agent finds that paragraph
- Update behaviour: what happens when the source changes, and designing so the agent never lags the truth
- Confidentiality first pass: what must never enter an agent's knowledge, and who decides
- The curation principle: five great pages beat fifty mediocre ones, measurably

Projects built:
- Build 'Society Desk': an agent for a real college society grounded in a clean 3-document knowledge pack you author, with a 10-question proof test

Practice: Deliberately degrade one knowledge document, measure the answer-quality drop on your test set, then fix and remeasure.
Assessment: Given two versions of the same document, predict which grounds better and prove it with tests.

Class 8: Dataverse, Enterprise Data and Source Conflicts
Topics:
- Dataverse as structured knowledge: when tabular business data beats documents
- Enterprise data through connectors: sources indexed by Microsoft Search, and why enterprises fund this path
- Multi-source agents: how the agent chooses between website, documents and structured data
- Conflict behaviour: engineering for the day two sources disagree, because they will
- Scoping by instruction: routing question types to the right source deliberately
- Freshness strategy: which source type updates fastest and which facts must follow it
- Auditing knowledge like an owner: what is in, what is out, what is stale, on one page

Projects built:
- Build a two-source Internship Tracker knowledge base (structured table plus policy document) and document exactly how the agent behaves when they conflict

Practice: Write five questions that force a source choice and log which source wins each, then adjust scoping instructions.
Assessment: Defend your capstone source list: authority, freshness and maintenance plan for each source.

Class 9: Accuracy Audits, Bias and the Trust Contract
Topics:
- The accuracy audit: scoring 20 grounded answers against source text and publishing the number
- Bias in, bias out: how one-sided source lists produce one-sided agents, and the audit that catches it
- Uncertainty language: agents that say 'my sources do not cover that' instead of improvising
- Data protection instincts: personal data, minimisation, and what a business agent must forget
- The trust contract with users: transparency about being an agent, about sources, and about limits
- What enterprises layer on top: evaluation suites and monitoring, previewed honestly before you meet them in Class 20
- Phase 2 wrap: the grounding checklist you now apply to every agent

Projects built:
- Publish an accuracy audit of Society Desk: 20 scored answers with cited source paragraphs, an accuracy figure, and the three fixes it demanded

Practice: Add an uncertainty rule to every agent you own and collect three real out-of-scope responses that handled it well.
Assessment: Phase 2 gate: design a complete knowledge plan (sources, scoping, refusals, accuracy test) for a given business brief, in writing.

PHASE 3: CONVERSATION & ORCHESTRATION

Design conversations that hold up under real users: topics and trigger phrases, entities and variables, condition groups, adaptive cards, and a deliberate choice between generative and classic orchestration.

Class 10: Topics and Trigger Phrases
Topics:
- Topics as designed conversation paths: one topic per job the agent does
- Trigger phrases: capturing the many phrasings of one request (topics support up to 200)
- System topics you inherit: greeting, escalate, goodbye, and when to customise them
- The authoring canvas: message nodes, question nodes and flow, built live

    - Topics vs generative answers: control for the critical paths, flexibility for the long tail
    - The YAML beneath the canvas: every topic is also code, your first bridge from low-code to pro-code
    - The topic map for a business agent: the five jobs users will actually bring
  Projects built:
    - Build 'Fest Helpdesk' with three designed topics for a real event: schedule lookup, registration help and venue directions
  Practice: Collect ten real phrasings of one request from classmates and tune trigger phrases until recognition is reliable.
  Assessment: Given three user messages, predict the triggered topic and verify live, explaining each miss.

Class 11: Entities, Variables and Slot Filling
  Topics:
    - Question nodes that collect exactly what a process needs, one clean question at a time
    - Entities: extracting dates, numbers and choices from messy human sentences
    - Variables: carrying user answers through the conversation and into confirmations
    - Slot filling: skipping questions the user already answered in one breath
    - Choice questions vs open questions: constraining input where structure matters
    - Validation: what happens when the user answers a date question with 'whenever works'
    - Session memory: what the agent remembers within a conversation and what it must not
  Projects built:
    - Upgrade Internship Tracker: an add-application topic that collects company, role, deadline and status through entities, confirming with a variable-built summary
  Practice: Test one topic with cooperative, messy and hostile input styles; log extraction failures and tighten the questions.
  Assessment: Trace a conversation on paper naming the entity and variable at each step, then verify live.

Class 12: Condition Groups and Branch Design
  Topics:
    - Condition nodes: branching on variables to route the conversation
    - Condition groups: combining tests cleanly (role is internship AND deadline within a week) without spaghetti
    - Paper first: flow diagrams before canvas, the habit that prevents dead ends
    - The escalation branch: recognising 'this needs a human' and handing off with context
    - Default branches: designing for everything you did not predict
    - Branch coverage testing: one scripted conversation per path, no path untested
    - Refactoring: splitting one bloated topic into three maintainable ones
  Projects built:
    - Rebuild Fest Helpdesk registration as a condition-driven flow: student vs guest vs volunteer paths, each with its own confirmation and an escalation branch
  Practice: Hand your flow diagram to a friend and let them hunt for a reachable dead end; fix what they find.
  Assessment: Find and fix the planted logic bug in a provided flow diagram within one session.

Class 13: Adaptive Cards and Business-Grade Responses
  Topics:
    - Adaptive cards: structured, tappable answers for information that deserves better than a paragraph
    - Card anatomy: titles, fact sets, images and actions, and when each earns its place
    - Mobile first: your agent will mostly be read on phones, so cards are designed for small screens
    - Card actions that continue the conversation: buttons feeding back into topics
    - A small card style guide: consistency that makes an agent look engineered, not assembled
    - Accessibility: alt text and reading order as professional defaults, not afterthoughts
    - Restraint: the cases where plain text beats a card, chosen deliberately
  Projects built:
    - Give Internship Tracker a card-based pipeline view: each application a card with company, role, deadline and a status-update action
  Practice: Convert your three most-used responses to cards and A/B them with two real users against the text versions.
  Assessment: Improve a deliberately bad card (dense text, no hierarchy, dead buttons) and justify every change.

Class 14: Generative Orchestration vs Classic, Chosen Deliberately
  Topics:
    - Two execution brains: classic orchestration where trigger phrases pick topics, generative orchestration where the model plans across topics, knowledge and tools
    - What generative orchestration unlocks: multi-intent requests, dynamic tool choice, and the event triggers you will use in Phase 4
    - The trade: less predictability, reined back in with instructions and scope fences
    - Reading the plan: inspecting how the agent decided which topic, source or tool to use, and debugging from that trace
    - Mixing modes: designed topics for critical business paths, generative flexibility for the long tail
    - Re-running the guardrail suite: refusals and fences retested under the new orchestrator, because behaviour moved

- Phase 3 wrap: the conversation-design checklist from trigger phrases to orchestration choice
Projects built:
- Migrate Fest Helpdesk to generative orchestration, rerun the full test set, and file a keep-or-fix verdict on every behaviour change
Practice: Write three multi-intent messages ('register me, what time is the keynote, can my cousin come') and tune until all three resolve in one turn.
Assessment: Phase 3 gate: design a five-topic business agent on paper (topics, entities, conditions, cards, orchestration mode) and defend it.

PHASE 4: TOOLS, AUTOMATION & MULTI-AGENT

The employability core: connectors and Prompt tools, deterministic agent flows, REST and custom connectors, Model Context Protocol, autonomous event triggers with real security discipline, multi-agent design with A2A, and analytics with agent evaluations.

Class 15: The Tool Menu and Connector Library
Topics:
- The tool types in Copilot Studio: Prompt, Agent flow, Custom connector, Model Context Protocol and REST API, plus prebuilt connectors
- The connector library: more than 1,400 prebuilt connections to real services, standard and premium
- Your first Prompt tool: one transformation done perfectly (summarise, extract, rewrite), reusable across agents
- Tool descriptions as an interface: the orchestrator chooses tools by reading descriptions, so you write them like contracts
- Discipline over abundance: the platform allows up to 128 tools per agent, Microsoft recommends staying near 25 to 30, and professionals use far fewer
- Authorisation questions before wiring anything: what may this tool touch, and who approved that
- Tool plan for your capstone: usually two tools, chosen from evidence, not excitement
Projects built:
- Add a 'JD decoder' Prompt tool to Placement FAQ: paste any job description, get requirements, keywords and preparation gaps in a fixed format
Practice: Write two versions of one tool description and demonstrate how the wording changes when the orchestrator invokes it.
Assessment: Explain, unaided, how an agent under generative orchestration decides between answering, searching knowledge and calling a tool.

Class 16: Agent Flows: Automation That Runs the Same Every Time
Topics:
- Agent flows: deterministic, step-based automations your agent calls as tools
- Flow anatomy: trigger, actions, conditions and outputs returned to the conversation
- When flows beat models: approvals, notifications, record updates, anything that must be identical every run
- Building a notification flow: collect structured details in conversation, send a formatted email or Teams message
- Error paths: what the user sees when a step fails, and why silent failure is disqualifying
- Reading a flow run: inputs, outputs and the exact step that broke
- Flows vs topics vs Prompt tools: a decision rubric you can defend in interviews
Projects built:
- Build 'Expense Slip': Society Desk collects reimbursement details through entities, an agent flow emails a formatted claim to the treasurer, and the agent confirms with the flow's real result
Practice: Break your flow with bad input on purpose, then add the error path that turns the failure into a graceful message.
Assessment: Given three automation briefs, classify each as flow, topic or Prompt tool with a three-sentence justification.

Class 17: REST APIs, Custom Connectors and Governance
Topics:
- REST in one hour: request in, JSON out, and how an agent turns a response into an answer
- Adding a REST API as a tool: authentication, parameters and response handling
- Custom connectors: wrapping any API so every maker in an organisation can reuse it safely
- Authentication in plain words: what 'connect your account' grants, and why least privilege is the default
- DLP governance: connector access is policy-controlled in real tenants, so 'can' and 'may' are different questions
- Reading API documentation like a builder: finding the one endpoint your agent needs
- The integration mindset recruiters probe: the least-powerful tool that does the job
Projects built:
- Wire a public REST API into a practice agent as a tool (weather, holidays or exchange rates) and demonstrate a grounded conversation using live data
Practice: Design on paper the API tool your capstone needs: endpoint, parameters, auth, tool description and three test utterances.
Assessment: Sketch one tool call end to end: utterance, orchestration decision, call, response, answer, and where it can fail.

Class 18: Model Context Protocol and Computer Use

Topics:
- Model Context Protocol (MCP): the open standard for connecting tool servers to agents, added from the same Add-a-tool menu
- MCP tools and resources: what a server exposes, and toggling individual tools with the Allow all switch
- Why MCP transfers: the same protocol appears in Claude, IDEs and other agent platforms, so this class compounds
- Governance continuity: MCP runs over Power Platform connectors, so DLP policies govern it like any other connector
- Computer use, surveyed honestly: agents that operate real user interfaces, generally available since May 2026, with per-step metering and premium model options
- The governance stack around computer use: allow-lists, HTTPS enforcement, stored credentials and human supervision
- Choosing integration styles: prebuilt connector vs REST vs custom connector vs MCP, as a decision table

Projects built:
- Connect an MCP server to a practice agent, inventory the tools it exposes, disable all but two deliberately, and demonstrate both in conversation

Practice: Write a one-page comparison: the same integration built as REST tool vs MCP tool, with the governance implications of each.

Assessment: Explain to your mentor what MCP standardises, and why DLP still applies to it.

Class 19: Autonomous Agents: Event Triggers and the Security They Demand

Topics:
- Event triggers: agents that act with no user in the chat, on events like an item created in SharePoint, a file added to OneDrive, a completed Planner task, or a Recurrence schedule
- The trigger payload: event data arriving as input, and instructions deciding what happens next
- Generative orchestration as a prerequisite: why autonomy needs the planning brain from Class 14
- The security fact enterprises underline: triggers run with the maker's credentials, so every autonomous action is effectively signed by you
- Designing the first autonomous job: small, observable, reversible, logged
- Human-in-the-loop: which actions pause for approval, and how the pause is built
- Idempotence in plain words: running twice must not double the damage

Projects built:
- Build 'Status Reporter': an autonomous agent that watches a shared project folder and posts a tidy weekly summary on a Recurrence trigger, with an approval pause on anything destructive

Practice: Feed your autonomous agent five events while away from the keyboard, then audit and score all five actions from the log.

Assessment: Explain maker-credential risk and name the two design rules you applied because of it.

Class 20: Multi-Agent Design, Analytics and Evaluations

Topics:
- Multi-agent options: child agents, connected Copilot Studio agents, and agents joined over the A2A protocol, generally available since April 2026
- When one agent becomes two: Microsoft's own rule of thumb, split when an agent nears 30 to 40 choices of tools, topics and agents
- Orchestrating specialists: a front-desk agent delegating to a bookings agent and a policy agent, cleanly
- The analytics view: sessions, outcomes (resolved, escalated, abandoned), satisfaction and themes
- Transcripts as design feedback: reading real sessions respectfully and mining the gaps
- Agent evaluations, generally available since March 2026: test sets and graders that automate the discipline you have run by hand since Class 4
- The improvement loop with numbers: hypothesis, one change, evaluation rerun, verdict

Projects built:
- Split Fest Helpdesk into a parent agent with one child specialist, then run an analytics-and-evaluations sprint on a week of real usage and ship two evidenced improvements

Practice: Define your capstone's three success metrics and build the evaluation set that will measure them before you build the agent.

Assessment: Phase 4 gate: from provided transcripts and evaluation results, diagnose the top failure and prove your one-line fix moved the number.

PHASE 5: PUBLISH, CREDENTIALS & CAPSTONE

Publish to the channels businesses use, pass a professional pre-flight checklist, then build and defend a business-grade capstone agent, leaving with a mapped Microsoft credential roadmap.

Class 21: Channels and the Agent Store

Topics:
- The channel lineup: Microsoft Teams and Microsoft 365 Copilot, SharePoint, demo website, custom website, and WhatsApp in business configurations
- Publishing flow: from studio to a shareable demo website, then toward Teams where organisations actually work

    - The Agent Store: how a published agent reaches Microsoft 365 Copilot after an admin approves it, a live lesson in enterprise governance
    - Identity for agents: Microsoft now issues Entra Agent IDs to new agents, making agents governable like users, mandatory since July 2026
    - Channel-fit design: the same agent on a phone, a website and inside Teams, and what changes per surface
    - Versioning discipline: what re-publishing replaces, and how not to break live users
    - Your capstone's launch plan: channel, audience, access and rollback, on one page
  Projects built:
    - Publish Internship Tracker to the demo website, run a supervised session with two real users on their phones, and file the structured feedback
  Practice: Write the one-paragraph launch plan for your capstone: channel, audience, access decision and rollback trigger.
  Assessment: Explain what changes and what stays identical when one agent ships to two channels.

Class 22: The Pre-Flight Checklist: Security, Cost and Honesty
  Topics:
    - The professional pre-flight: scope fences retested, refusals verified, citations on, personal data audited, evaluation suite green
    - Prompt injection: how malicious content tries to hijack instructions through knowledge and inputs, and the mitigations that blunt it
    - Data minimisation: collecting the least, retaining the least, and being able to say so
    - Cost awareness in Copilot Credits: metered usage (generative answers, actions, flow actions) means design choices change the bill, and capacity has enforcement limits
    - Transparency obligations: users know they are talking to an agent, what it can do, and where its knowledge ends
    - Kindness and accessibility passes: language level, error tone, and the user having a bad day
    - The signed go / no-go: a written audit your mentor countersigns before capstone launch
  Projects built:
    - Run the full 20-point pre-flight checklist against your published agent, fix every failure, and produce the signed one-page audit
  Practice: Attempt three prompt-injection attacks on your own agent through its knowledge and inputs, document outcomes, and add one mitigation.
  Assessment: Audit a deliberately flawed agent spec and catch at least four of its five planted problems.

Class 23: Capstone Build Week
  Topics:
    - Freezing the spec: job, users, sources, tools, trigger, channel and success metrics on one page
    - Build order that works: knowledge first, topics second, tools third, trigger last
    - Timeboxing with a mentor: what gets cut when reality bites, and why the core loop is protected
    - The evaluation suite as safety net: rerun at every milestone, no green no merge
    - Professional polish: greeting, description, card consistency, error tone and escalation paths
    - Dress rehearsal: the timed demo run with one planned failure recovery
    - Documentation packet: spec, source list, tool contracts, test log, audit page, ready for a recruiter
  Projects built:
    - Build the complete capstone: a business-grade agent for a real club, department or small business, grounded, tooled, evaluated, published and documented
  Practice: Run the dress rehearsal for an outsider and cut or fix the two weakest moments they identify.
  Assessment: Milestone gate: the capstone passes its own evaluation suite and pre-flight audit before demo day.

Class 24: Demo Day and the Credential Roadmap
  Topics:
    - The live demo: your capstone on a real channel, citations showing, one failure recovered gracefully
    - Defending design decisions: these sources, these tools, this orchestration mode, this trigger, and why
    - The retrospective: what you would build differently, the slide interviewers respect most
    - Portfolio packaging: capstone plus project history, browsable by a recruiter in five minutes
    - The Microsoft credential ladder mapped to what you now know: PL-900 with its new Copilot Studio agents domain, the Applied Skills labs 'Build an agent in Microsoft Copilot Studio' and 'Enhance agents with autonomous capabilities', and the AI Agent Builder Associate certification (exam AB-620)
    - The market you are entering: Gartner projects that by 2029 at least half of knowledge workers will need to work with, govern or create AI agents
    - Certificate, feedback and the Living Syllabus promise: what changed in Copilot Studio while you were learning it
  Projects built:
    - Deliver the capstone demo and hand over the complete packet: spec, source list, tool contracts, evaluation results, audit page and a two-minute recorded walkthrough
  Practice: Draft your AB-620 preparation plan: which exam domains your coursework already covers and what remains.
  Assessment: Final evaluation: capstone demo, design defence and documentation packet,

graded against the Class 20 rubric.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: a business agent teardown, Placement FAQ v1 with designed instructions, a corporate refusal gauntlet, a permanent 15-question test set, and a model comparison memo with evidence
- Phase 2: a citation-backed Placement FAQ grounded in a real company site, Society Desk with an authored 3-document knowledge pack, a two-source Internship Tracker with conflict handling, and a published accuracy audit
- Phase 3: Fest Helpdesk with designed topics, entity-driven application capture, a condition-driven registration flow with escalation, a card-based pipeline view, and a generative-orchestration migration with verdicts on every change
- Phase 4: a JD-decoder Prompt tool, the Expense Slip agent flow with graceful errors, a live REST API tool, an MCP server connected with deliberately scoped tools, the autonomous Status Reporter with approval pauses, and a multi-agent split with an evaluations sprint
- Phase 5: a published agent tested by real users, a signed 20-point pre-flight audit, the complete business capstone, and a demo-day defence with a recruiter-ready documentation packet
- Total: 24 hands-on projects, one per class, finishing with a published business-grade capstone agent

Total projects: 24 hands-on projects, one per class, finishing with a published business-grade capstone agent

SKILLS THE STUDENT WILL HAVE AT THE END

- Agent design: instructions, scope fences, refusals, and brand-appropriate tone
- Knowledge grounding at business depth: SharePoint, documents, Dataverse and connector-indexed enterprise data, with citations
- Conversation design: topics, trigger phrases, entities, variables, condition groups and adaptive cards
- Generative vs classic orchestration, selected and defended per use case
- Tools: Prompt tools, prebuilt and custom connectors, REST APIs and Model Context Protocol
- Agent flows: deterministic automation with error paths and readable runs
- Autonomous agents: event triggers, Recurrence schedules, maker-credential security reasoning and human-in-the-loop design
- Multi-agent architecture: child agents, connected agents and the A2A protocol
- Analytics and agent evaluations: outcomes, transcripts, graders and evidence-driven iteration
- Publishing and governance: channels, the Agent Store approval path, Entra Agent IDs, DLP awareness and Copilot Credits cost literacy

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 private 1-on-1 classes per week, about 1 hour each, across 12 weeks
Hands on agent work: 1.5-2 hours building and testing in Copilot Studio between classes
Project time: 1-1.5 hours completing each class project, usually inside your own agents
Review and doubts: 30-45 minutes reviewing feedback and clearing doubts over WhatsApp before the next class
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Dedicated mentor: The same mentor for all 24 classes, who knows exactly where your agents, portfolio and preparation stand
Whatsapp doubts: Doubt support between classes over WhatsApp
Practice environment: Help setting up your Copilot Studio environment in Class 1, including the trial path for college accounts and an arranged practice environment where needed
Session recordings: Class recordings available for revision, so a missed detail is never lost

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- AI agent builder or automation specialist roles, the job titles now appearing in listings as enterprises staff their agent programmes
- Power Platform maker and business technologist tracks, where Copilot Studio fluency is the differentiator
- Microsoft's credential ladder with course-to-exam mapping: PL-900 Power Platform Fundamentals, the Applied Skills labs (APL-6006 and APL-6000), and Microsoft Certified: AI Agent Builder Associate (AB-620)
- Operations, support and analyst roles upgraded: the person who automates the workflow rather than performing it manually
- A bridge to pro-code agent work through our Codex + Claude Code masterclass, with MCP knowledge that transfers directly

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need to know how to code to take this Copilot Studio course?
A: No. Microsoft Copilot Studio is a graphical, low-code studio: agents are designed with plain-English instructions, curated knowledge sources and a visual canvas. The course teaches everything from first principles. If you do code, the REST API, custom connector and Model Context Protocol classes give you room to go deeper, but no class assumes programming.

Q: What will I actually build during the 24 classes?
A: One real project per class: a placement FAQ agent grounded in a real company's site with checkable citations, a society helpdesk with designed topics and condition logic, an internship tracker with entity-driven capture and card views, an expense automation built on an agent flow, a live REST API tool, an MCP server integration, an autonomous status reporter running on event triggers with approval pauses, a multi-agent split, and a published business-grade capstone with a full documentation packet.

Q: Why is this course 1-on-1 only?
A: Because the skill lives in artifacts that need individual review: your instructions, source lists, flow designs, tool contracts and evaluation results. A private mentor reads and critiques all of them every class, and rebuilds projects around your actual clubs, departments and placement goals. That review loop is what produces students who can defend their design decisions in interviews, and it cannot be delivered honestly to a batch.

Q: What Microsoft account do I need, and is there a free way to practise?
A: Copilot Studio uses a Microsoft work or school (Entra ID) account rather than a personal email. Most college email addresses qualify for Microsoft's 30-day self-serve trial, which covers in-studio building and testing but cannot publish agents. Where a suitable account is not available, we arrange a practice environment for the duration of the course. Your mentor completes the setup with you in Class 1.

Q: Does this course prepare me for Microsoft certifications?
A: Yes, deliberately. The syllabus vocabulary maps to PL-900 Power Platform Fundamentals, which added a dedicated Copilot Studio agents domain in its July 2026 skills update, and to the hands-on Applied Skills labs 'Build an agent in Microsoft Copilot Studio' and 'Enhance agents with autonomous capabilities'. The headline credential is Microsoft Certified: AI Agent Builder Associate (exam AB-620), whose domains (plan and configure, integrate and extend, test and manage) mirror Phases 2 through 5, and Class 24 ends with your personal AB-620 preparation plan.

Q: Is Copilot Studio genuinely an employable skill or a passing trend?
A: The deployment numbers are unusually concrete: Microsoft states that 90% of the Fortune 500 use Copilot Studio, and reported more than 230,000 organizations building with it as far back as April 2025. Gartner projects that by 2029 at least half of knowledge workers will need skills to work with, govern or create AI agents, and that a third of enterprise software will include agentic AI by 2028. The course also teaches the governance and evaluation discipline that outlasts any single platform.

Q: How current is the syllabus, given how fast the platform changes?
A: It is a Living Syllabus, re-verified against Microsoft's official documentation and What's New pages as features ship. The current syllabus covers generative orchestration, the present model lineup (GPT-4.1 default, GPT-5.5 Chat, and Anthropic Claude models), Model Context Protocol tools, the A2A multi-agent protocol, agent evaluations and Entra Agent IDs, all of which reached the platform within roughly the last year.

Q: How is this different from your Codex + Claude Code course?
A: They are complementary tracks. Codex + Claude Code is pro-code: you drive AI coding agents from the command line to ship software. This course is the low-code business track: you design, ground and govern agents in Microsoft Copilot Studio, the way operations, support and business teams deploy them. MCP knowledge transfers between both. Many students take this course first for the portfolio and vocabulary, then go pro-code.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final project defended live, plus a written assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-course-for-college-students
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 8 OF 137: AI AGENTS WITH MICROSOFT COPILOT STUDIO FOR TEENS
==========

Course link: https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-course-for-teens
Subject area: AI Agents & Automation
Designed for: Teens
Who can join: Beginner (ages 13-18; no coding experience needed)
Course length: 24 classes (12 weeks · 2 classes/week)
Time needed: 3-4 hours/week recommended
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: one to one Rs 4,999
  Outside India, per month: one to one $150
  Classes: one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.
  Note: this course is taught one to one only. There is no group or mini batch option.
  Note: the course length above is counted at two classes a week. In India the one to one plan is one class a week, so if an Indian family asks how long it will take, hand over to Shivam Sir.

WHAT THIS COURSE IS

  Build real AI agents with Microsoft Copilot Studio in 24 live 1-on-1 online classes for ages 13-18: knowledge grounding, topics, connectors, agent flows, autonomous triggers and a published capstone agent. No heavy coding needed.

  In one line: Stop just chatting with AI. Design, ground, and publish your own AI agents on the platform 90% of the Fortune 500 already use, in private 1-on-1 classes built around your ideas.

ABOUT THE PROGRAMME

  Every teenager has used a chatbot. Almost none of them have built an agent: an AI that knows a specific subject deeply, follows instructions it was given, uses tools, and acts on its own when something happens. That is exactly the skill this course teaches, on Microsoft Copilot Studio, the graphical low-code studio Microsoft describes as the way to build and manage AI-powered agents and workflows. Across 24 live 1-on-1 classes you go from your very first agent (described in plain English, tested the same hour) to agents grounded in real knowledge sources with citations, conversation design with topics and trigger phrases, tools built from connectors and agent flows, an autonomous agent that wakes up on an event trigger and works without you, and a published capstone agent demonstrated live on a website and shared for feedback. Along the way you learn the ideas that make agents trustworthy: grounding versus guessing, why an agent should cite its sources, what an agent should refuse to do, and how professionals measure whether an agent actually helped. Because every class is private, your mentor builds the whole course around agents you actually care about: a study buddy grounded in your own class notes, a quiz coach for your next exam, a club FAQ agent, a family trip planner. The syllabus is a Living Syllabus, updated as Copilot Studio ships new features, so you learn the current platform, never a frozen snapshot of last year's tool.

WHAT MAKES THIS COURSE DIFFERENT

  - 1-on-1 only, by design: agent building is a design discipline, and a private mentor can review your instructions, your knowledge sources, and your conversation maps line by line, which a group class simply cannot do.
  - Real platform, real skills: you work in Microsoft Copilot Studio itself, the same low-code agent studio used across enterprises, with topics, knowledge, tools, triggers and analytics, not a toy sandbox.
  - No heavy coding needed: Copilot Studio is a graphical, low-code studio, so the course runs on clear thinking and good instructions; teens who do code can go further with connectors and REST APIs in the same syllabus.
  - Agents that act, not just answer: by Phase 4 your agent responds to event triggers (a file added, a task completed, a schedule) and works autonomously, which is the difference between a chatbot and an agent.
  - Honest AI habits from Class 1: grounding answers in sources, keeping citations on, deciding what an agent must refuse, and testing before publishing, the same responsible-AI defaults professionals are held to.
  - Living Syllabus: Copilot Studio ships new models and features continuously (the current lineup includes GPT-5.5 Chat and Anthropic Claude models), and the curriculum is updated as they land, so nothing you learn is stale.

WHAT A STUDENT NEEDS BEFORE STARTING

  Device: A laptop or desktop (Windows, Mac or Linux) with the Chrome or Edge browser and a reliable internet connection; Copilot Studio runs fully in the browser
  Coding experience: None needed. Copilot Studio is a graphical, low-code studio; clear thinking and good writing matter far more than syntax. Teens who already code can go deeper into connectors and REST tools inside the same syllabus
  Accounts: Copilot Studio uses a Microsoft work or school account rather than a personal email. Many students already have one from school; if you do not, we arrange access to a practice environment for the course, and for students under 18 a parent or guardian accepts Microsoft's terms
  Mindset: Curiosity about how AI actually works, and the patience to test before you trust.

The course rewards students who ask 'how do I know this answer is right?'

WHO THIS COURSE IS FOR

Curious builders: Teens who use ChatGPT or Copilot daily and are ready to move from asking AI questions to building AI that answers them
Future professionals: Students eyeing careers in technology, product or business who want the agent-building vocabulary employers now test, on the platform 90% of the Fortune 500 use
Non coders: Teens who thought AI was locked behind programming: Copilot Studio's low-code canvas makes design thinking, not syntax, the superpower
Young coders: Students from our Python or web development courses who want to see where low-code agents meet APIs, REST and Model Context Protocol
Portfolio hunters: Teens building a portfolio for internships or university applications who want a published, working agent to show, not a certificate alone

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: YOUR FIRST AGENTS

Understand what an AI agent actually is, tour Microsoft Copilot Studio, build your first agent from a plain-English description, and learn how instructions and testing really work.

Class 1: What an AI Agent Really Is (and What It Is Not)

Topics:
- Chatbot vs assistant vs agent: an agent has instructions, knowledge, tools and can act, not just answer
- Agents you already meet every week: support bots, order trackers, school helpdesks, and what they get right and wrong
- Microsoft Copilot Studio in one sentence: a graphical, low-code studio for building and managing AI-powered agents and workflows
- The guided tour: agents, knowledge, topics, tools, triggers, channels and analytics, and what each tab is for
- How access works honestly: Copilot Studio uses a Microsoft work or school account, and your mentor gets your practice environment ready with you in this class
- The anatomy of every agent you will build: name, description, instructions, knowledge, tools, and where each lives
- Setting the course goal: the capstone agent you will publish in Class 24, chosen from your own school, club or family life

Projects built:
- Agent safari: analyse two real agents you have used (a support bot, a game helper), map what their instructions and knowledge must look like, and write a one-page teardown

Practice: List five agent ideas from your own week (study, club, family, hobby) and rank them by how much real knowledge each would need.
Assessment: Explain the difference between a chatbot and an agent in two sentences, using the words instructions, knowledge and tools correctly.

Class 2: Your First Agent in One Hour

Topics:
- Describe-to-create: starting an agent from a plain-English description of what it should do
- Writing your first instructions: role, audience, tone, and the three behaviours your agent must always show
- The test panel: talking to your agent the moment it exists, and reading its answers critically
- Iteration as the core loop: change one instruction line, retest, compare, keep or revert
- Naming, descriptions and icons: why a clear agent description matters for users and for you
- What the model does with your instructions: a teen-friendly picture of how a large language model follows a system prompt
- Saving versions of your instruction text outside the tool, the habit that saves every project later

Projects built:
- Build 'Study Buddy v1': an agent with instructions for your real timetable and subjects that answers revision questions in a tone you designed

Practice: Rewrite your agent's instructions three different ways and record how each version changes the same test question's answer.
Assessment: Show a before-and-after: one instruction change and the exact behaviour difference it caused in the test panel.

Class 3: Instructions That Actually Steer Behaviour

Topics:
- The instruction stack: identity, scope, tone, format rules, and refusals, in that order
- Scope fences: telling an agent what it must NOT do, and testing that the fence holds
- Tone control for real audiences: the same agent for classmates vs teachers vs grandparents
- Format discipline: making answers arrive as steps, tables or short paragraphs on demand
- Refusal design: choosing the questions your agent should decline (medical advice, personal data, exam cheating) and wording the refusal kindly
- Instruction smells: vague verbs, contradictions, and instructions that fight each other

- A/B testing instructions: running the same 10 questions against two instruction sets and scoring the answers

Projects built:

- The Refusal Gauntlet: write 10 questions your Study Buddy must refuse or redirect, then tune instructions until all 10 are handled gracefully

Practice: Take a classmate's (or parent's) three trickiest questions and tune your instructions until the answers pass their review.

Assessment: Present your instruction stack and defend each line: what breaks if it is removed?

Class 4: Testing Like a Professional

Topics:

- Why 'it answered nicely once' is not evidence: building a fixed test set of 15 questions
- Happy path, edge case, and adversarial questions: three test types every agent needs
- Reading answers critically: correctness, tone, format, and whether the agent stayed in scope
- Regression testing by hand: rerunning your test set after every meaningful change
- Keeping a test log: question, expected behaviour, actual behaviour, verdict
- When the agent is confidently wrong: recognising hallucination before you have knowledge sources to fix it
- The professional habit this course grades you on: no change ships without a retest

Projects built:

- Build your permanent 15-question test set for Study Buddy, with expected behaviour written down before you run it

Practice: Run your full test set twice this week and log every verdict; bring the two failures you found to class.

Assessment: Given a transcript of an agent conversation, mark every line as pass or fail against the written instructions.

Class 5: How the Platform Thinks: Environments, Models and Harnesses

Topics:

- Where agents live: environments, and why your practice environment is separate from a company's production one
- The model picker: Copilot Studio agents run on a chosen model (the lineup currently includes GPT-4.1 as default, GPT-5.5 Chat, and Anthropic Claude models), and what changing it does
- Harnesses in plain words: the runtime between your design and the model, and why Microsoft now ships more than one
- What is free and what costs money: Copilot Credits exist, adults pay for capacity, and testing in class is covered by your practice environment
- Responsible-AI defaults you inherit: content filters and guardrails the platform applies before your instructions even run
- Saving and organising agents so week-12-you can find what week-2-you built
- Checkpoint: everything in one map, from your typed instruction to the model's answer

Projects built:

- Draw the journey of one question through your agent: user, harness, instructions, model, answer, and label the parts you control

Practice: Try the same three test questions on two different models in the picker and write down every difference you can defend.

Assessment: Phase 1 review: rebuild a small agent from a written spec in 20 minutes, with instructions, a refusal, and a 5-question test.

PHASE 2: KNOWLEDGE & GROUNDING

Give your agent real knowledge: websites, documents and SharePoint sources, citations users can check, and the discipline of grounding answers instead of guessing.

Class 6: Grounding vs Guessing

Topics:

- Why a bare model guesses: training data vs your data, in a picture a friend could follow
- Knowledge sources in Copilot Studio: public websites, uploaded documents, SharePoint, Dataverse, and connector-indexed enterprise data
- Generative answers: how the agent searches your sources and composes an answer from what it found
- Citations: why the little source link is the most trustworthy pixel on the screen
- The 'Allow ungrounded responses' setting, and why this course keeps it off by default
- Limits that shape design: sources per agent, website counts, and file size ceilings exist, so knowledge is curated, not dumped
- Choosing knowledge for your capstone: authoritative, current, and small enough to verify

Projects built:

- Ground Study Buddy in one real source (your school's public website or a subject site) and prove three answers now cite it

Practice: Ask your grounded agent five questions its source cannot answer and observe exactly how it behaves at the edge of its knowledge.

Assessment: Explain grounding, citation and hallucination to a parent in under two minutes; they grade you.

Class 7: Document Knowledge Done Right

Topics:

- Uploading documents as knowledge: what formats work well and what makes a document searchable

- Writing documents FOR agents: headings, one fact per paragraph, and names spelled consistently
- The curation habit: five great pages beat fifty messy ones, every time
- Testing document knowledge: questions whose answers live in one exact paragraph
- Updating knowledge: what happens when the source document changes and the agent must not lag behind
- Privacy first pass: what should never go into an agent's knowledge (family data, friends' names, anything you would not print)
- Building the knowledge pack for your own subjects: notes that become a genuinely useful revision agent

Projects built:
- Create a clean 3-document knowledge pack from your own class notes and ground a Revision Coach agent in it, with a 10-question proof test

Practice: Deliberately break one document (vague headings, mixed topics), watch answer quality drop, then fix it and retest.
Assessment: Given two versions of the same document, predict which grounds better and verify by testing both.

Class 8: Multi-Source Knowledge and Conflicts
Topics:
- Combining sources: website plus documents plus a SharePoint folder, and how the agent picks
- When sources disagree: seeing conflicting-answer behaviour and designing around it
- Source scoping: pointing a question type at the right source with instructions
- Freshness: preferring the source that updates (a website) for facts that change
- The web question: what 'Use information from the web' changes, and when a curated agent should leave it off
- Auditing an agent's knowledge like a librarian: what is in, what is out, what is stale
- Knowledge design for the capstone: writing the source list you will defend in Class 24

Projects built:
- Build 'Club Desk': an agent for a real club or activity grounded in two sources, with documented behaviour when sources conflict

Practice: Write five questions that force your agent to choose between its sources and log which one wins each time.
Assessment: Defend your capstone source list: for each source, why it is authoritative and how you will keep it current.

Class 9: Honest Agents: Accuracy, Bias and Boundaries
Topics:
- Measuring accuracy properly: scoring 20 grounded answers against the source text itself
- Bias in, bias out: how a one-sided source list produces a one-sided agent
- Uncertainty language: making the agent say 'my sources do not cover that' instead of improvising
- Personal data rules for minors, in plain words: what an agent about your life may never store or repeat
- Academic honesty: a revision agent that teaches you vs one that does your homework, and where this course draws the line
- What professionals add later: evaluation suites and monitoring, previewed honestly so you know where this road goes
- Phase 2 wrap: the grounding checklist you will apply to every agent from now on

Projects built:
- Accuracy audit: score 20 answers from Revision Coach against the actual source paragraphs and publish your agent's accuracy number with examples

Practice: Add an uncertainty rule to every agent you own and collect three real 'my sources do not cover that' responses.
Assessment: Phase 2 exam: given a topic, design the full knowledge plan (sources, scoping, refusals, accuracy test) in writing.

PHASE 3: CONVERSATION DESIGN

Design conversations deliberately: topics and trigger phrases, questions and variables, condition logic, adaptive cards, and the two orchestration modes that decide how your agent thinks.

Class 10: Topics and Trigger Phrases
Topics:
- What a topic is: a designed conversation path for one job your agent does
- Trigger phrases: teaching the agent the many ways people ask for the same thing (topics support up to 200)
- The topics you get for free: greeting, goodbye, escalate, and why system topics exist
- Building a topic on the authoring canvas: message nodes, question nodes, and the flow between them
- When to use a topic vs when to let generative answers handle it: control vs flexibility
- Peeking under the canvas: every topic is also code (YAML), the first hint that low-code and pro-code meet
- Designing your agent's topic map: the five jobs users will actually ask it to do

Projects built:
- Build a 'Homework Planner' agent with three hand-designed topics: add a task, list this week, and what should I do first

Practice: Collect ten real phrasings of one request from friends or family and add the best ones as trigger phrases, then test recognition.
Assessment: Given three user messages, predict which topic each will trigger and verify live.

Class 11: Questions, Entities and Variables
Topics:
- Question nodes: asking for exactly the information a topic needs, one clean question at a time
- Entities: how the agent extracts a date, a number, a choice from a messy sentence
- Variables: storing what the user said and reusing it later in the conversation
- Slot filling: when a user gives three answers in one sentence and the agent skips already-answered questions
- Choice questions vs open questions: constraining answers when structure matters
- Validating input: what happens when the user answers a date question with 'idk, whenever'
- Conversation memory across a session: what your agent remembers and what it forgets

Projects built:
- Upgrade Homework Planner: the add-a-task topic now collects subject, due date and effort through entities and confirms with a summary using variables

Practice: Test the same topic with cooperative, messy and hostile answers; log where extraction fails and tighten the questions.
Assessment: Trace a conversation on paper: name the entity and variable at each step before running it live.

Class 12: Condition Logic and Branching
Topics:
- Condition nodes: sending the conversation down different paths based on variables
- Condition groups: combining tests (subject is maths AND due tomorrow) without spaghetti
- Designing branches on paper first: a flow diagram before ever touching the canvas
- The escalation branch: recognising 'this needs a human' and saying so
- Default branches: the path for everything you did not predict
- Testing branch coverage: one test conversation per path, no path untested
- Refactoring topics: splitting one giant topic into three small clean ones

Projects built:
- Build 'Event Helper' for a real school or family event: RSVP path, question path, and organiser-escalation path, all condition-driven

Practice: Draw your Event Helper flow as a diagram, then have your mentor try to reach a dead end you did not design.
Assessment: Given a flow diagram with a logic bug, find and fix the bad branch in one session.

Class 13: Adaptive Cards and Rich Responses
Topics:
- Beyond plain text: adaptive cards for structured, tappable answers
- Card anatomy: title, facts, images, actions, and when a card beats a paragraph
- Designing cards for small screens: your agent will mostly be read on phones
- Buttons that continue the conversation: card actions feeding back into topics
- Consistency: a small card style guide so your agent looks designed, not assembled
- Accessibility basics: alt text and reading order, because agents are for everyone
- When NOT to use a card: the restraint that separates designers from decorators

Projects built:
- Give Homework Planner a card-based weekly view: each task a card with subject, due date and a 'mark done' action

Practice: Convert your three most-used answers into cards and A/B them with two real users against the text versions.
Assessment: Card critique: improve a deliberately ugly card (walls of text, no hierarchy) and justify each change.

Class 14: Generative Orchestration vs Classic
Topics:
- Two brains, one agent: classic orchestration (trigger phrases pick topics) vs generative orchestration (the model plans across topics, knowledge and tools)
- What generative orchestration unlocks: multi-intent requests, tool choice, and the autonomous triggers you will use in Phase 4
- What you trade: predictability, and how instructions rein the planner back in
- Watching the plan: reading how the agent decided which topic, knowledge source or tool to use
- Mixing modes wisely: designed topics for the critical paths, generative flexibility for the long tail
- Guardrails under orchestration: scope fences and refusals still rule, and you retest them now
- Phase 3 wrap: your conversation-design checklist, from trigger phrases to orchestration choice

Projects built:
- Switch Event Helper to generative orchestration, rerun your full test set, and document every behaviour change with a keep-or-fix verdict

Practice: Craft three multi-intent messages ('RSVP yes and what time and can my sister come') and tune until all three resolve cleanly.
Assessment: Phase 3 exam: design a five-topic agent on paper (topics, entities, branches, cards, orchestration mode) and defend the design.

PHASE 4: TOOLS & AUTONOMY

Make your agent act: connectors and Prompt tools, agent flows for multi-step work, a first look at Model Context Protocol, then event triggers and a fully autonomous agent with analytics to prove it works.

Class 15: Tools: Giving Your Agent Hands
Topics:
- The tool menu: Prompt, Agent flow, Custom connector, Model Context Protocol and REST API, and what each is for
- Connectors: Copilot Studio's library of more than 1,400 prebuilt connections to real services
- Your first tool: a Prompt tool that does one transformation perfectly (summarise, translate, rewrite)
- Tool descriptions matter: the agent chooses tools by reading their descriptions, so you write them carefully
- Fewer, better tools: the platform allows many tools per agent, professionals deliberately keep the list short
- Safety question of the day: what is this tool allowed to touch, and who said so
- Planning the tools your capstone actually needs (usually two, rarely five)

Projects built:
- Add a 'flashcard maker' Prompt tool to Revision Coach: paste messy notes, get clean question-answer pairs, every time

Practice: Write two versions of your tool's description and show how the wording changes when the agent decides to use it.
Assessment: Explain to your mentor, without notes, how an agent decides between answering directly, using knowledge, and calling a tool.

Class 16: Agent Flows: Multi-Step Work That Always Runs the Same
Topics:
- Agent flows: deterministic step-by-step automations your agent can call as a tool
- Flow anatomy: trigger, steps, conditions, and outputs handed back to the conversation
- When a flow beats the model: anything that must happen exactly the same way every time
- Building a notify flow: collect details in conversation, send a formatted email or Teams message
- Error paths in flows: what the user sees when a step fails, and why silence is never acceptable
- Flows vs topics: designed conversation vs designed action, and how they hand off
- Reading a flow run: inputs, outputs and the step where things went wrong

Projects built:
- Build 'Permission Slip': Event Helper collects details, an agent flow sends a tidy summary email to the organiser (a parent or teacher), and the agent confirms with the flow's result

Practice: Break your flow on purpose (bad input, missing field), observe the failure, then add the error path that makes it graceful.
Assessment: Given a task description, decide flow or topic or Prompt tool, and justify the choice in three sentences.

Class 17: Connecting to the Real World: Connectors, REST and MCP
Topics:
- Using a prebuilt connector as a tool: real services (To Do, Calendar, SharePoint) inside your conversation
- Authentication in plain words: why tools ask 'connect your account' and what is being granted
- REST APIs, demystified: request in, JSON out, and how an agent turns that into an answer
- Model Context Protocol (MCP): the open standard for plugging tool servers into agents, added from the same Add-a-tool menu
- Why MCP matters beyond Microsoft: the same protocol appears in Claude, IDEs and other agent platforms you already know
- Governance preview: connector access is policy-controlled in real companies, so 'can' and 'may' are different questions
- Choosing the responsible option: the least-powerful tool that does the job

Projects built:
- Wire one real connector tool into a practice agent (task list or calendar lookup) and demonstrate a conversation that reads and writes real data

Practice: Map one API you wish existed for your life (canteen menu, bus times) and write the tool description and three test utterances for it.
Assessment: Sketch the path of one tool call end to end: utterance, orchestration, tool choice, call, result, answer.

Class 18: Event Triggers: Agents That Wake Up by Themselves
Topics:
- The autonomy switch: event triggers let an agent act with no user in the chat at all
- Trigger types that matter: item created (SharePoint), file added (OneDrive), task completed (Planner), and Recurrence for schedules
- The trigger payload: the event's data arriving as input, and instructions deciding what to do with it
- Generative orchestration required: why autonomous behaviour needs the planning brain from Class 14
- The security fact professionals underline: triggers run with the maker's credentials, so you design as if everything it does is signed by you

- Designing the first autonomous job: small, observable, reversible
- Logging what your agent did while you were at school: the activity trail habit

Projects built:
- Build 'Drop Folder Librarian': when a file lands in a watched folder, the agent files a tidy summary card automatically, on a Recurrence backup schedule

Practice: Feed your autonomous agent five files while you are away from the keyboard and audit its five actions afterwards, scoring each.

Assessment: Explain maker-credential risk in your own words and name the two design rules you applied because of it.

Class 19: Your Autonomous Agent, Hardened

Topics:
- From demo to dependable: retries, idempotence in plain words (running twice must not double the damage)
- Human-in-the-loop: which autonomous actions should pause for approval, and how to design the pause
- Notification etiquette: an autonomous agent that spams is an autonomous agent that gets turned off
- Failure diaries: capturing what went wrong so Tuesday's bug is not repeated on Thursday
- Scope creep in autonomy: resisting the urge to let the agent 'also just' do one more thing
- Multi-agent preview: child agents and connected agents exist, and when one agent should become two
- Rehearsing the demo: showing autonomous behaviour live without praying

Projects built:
- Harden Drop Folder Librarian: add an approval pause for deletes, an error diary, and a weekly self-report message, then run it for three real days

Practice: Write the one-page 'operations note' for your agent: what it does, when, with whose permissions, and how to turn it off.

Assessment: Live drill: your mentor breaks one input; your agent must fail gracefully and your diary must show why.

Class 20: Analytics: Proving Your Agent Works

Topics:
- The analytics view: sessions, outcomes (resolved, escalated, abandoned), and what each outcome means
- Reading transcripts respectfully: real conversations are the best design feedback you will ever get
- Customer satisfaction in miniature: asking users to rate your agent and surviving the answers
- Finding the gap: the questions users asked that your agent could not handle, ranked
- Evaluations, previewed: professionals run test sets automatically against agents; you have been doing it by hand since Class 4
- The improvement loop: analytics, hypothesis, one change, retest, remeasure
- Phase 4 wrap: tools, autonomy and analytics as one system, ready for the capstone

Projects built:
- Analytics sprint: put one agent in front of five real users for a week, read every session, and ship two improvements the data demanded

Practice: Write your capstone's success metrics before building it: three numbers that will define 'it works'.

Assessment: Phase 4 exam: from a set of session transcripts, diagnose the top failure and propose the one-line fix, then prove it.

PHASE 5: PUBLISH & CAPSTONE

Publish agents to real channels, pass a safety and quality checklist, then design, build, test and present a capstone agent that does a real job for real people.

Class 21: Channels: Where Agents Live

Topics:
- The channel lineup: demo website, custom website, Microsoft Teams and Microsoft 365 Copilot, SharePoint, and even WhatsApp in business setups
- Publishing flow: from the studio to a shareable demo website in one class
- How the same agent feels different per channel: phone vs desktop vs inside Teams
- Access decisions: who can reach your agent, and why 'everyone' is rarely the right answer
- Versioning discipline: what publishing again replaces, and how not to break your live users
- The Agent Store idea: in companies, published agents go through an admin's approval, a preview of real-world governance
- Choosing your capstone's channel and writing its one-paragraph launch plan

Projects built:
- Publish Homework Planner to the demo website and run a supervised test with two real users on their own phones

Practice: Collect structured feedback from your two testers (what confused, what delighted) and file it against Class 23's build time.

Assessment: Explain what changes and what stays the same when one agent is published to two different channels.

Class 22: The Pre-Flight Checklist: Safety, Quality, Honesty

Topics:
- The publishing checklist professionals use: scope fences retested, refusals verified, sources

cited, personal data audited
- Prompt injection for teens: how a malicious message tries to hijack instructions, and the defensive habits that blunt it
- Data minimisation: your agent stores the least it can and forgets what it should
- Transparency: users must know they are talking to an agent, and what it can and cannot do
- Credits and cost awareness: agent usage is metered in real deployments (Copilot Credits), and design choices change the bill
- Accessibility and kindness passes: language level, error tone, and the user having a bad day
- Signing off: the written go / no-go your mentor countersigns before capstone launch

Projects built:
- Run the full 20-point pre-flight checklist against your published agent, fix the failures, and produce the signed one-page audit

Practice: Attempt three prompt-injection attacks on your own agent, document what happened, and add one mitigation.

Assessment: Checklist mastery: audit a deliberately flawed agent spec and catch at least four of its five planted problems.

Class 23: Capstone Build Week

Topics:
- Freezing the spec: the capstone's job, users, sources, tools, trigger and channel on one page
- Building in the right order: knowledge first, topics second, tools third, trigger last
- Timeboxing with a mentor: what gets cut when reality bites, and why the core loop is protected
- The test set as safety net: your fixed questions rerun after every build milestone
- Polish that matters: greeting, name, description, card style, error tone
- Dress rehearsal: the full demo run, timed, with one planned failure recovery
- Presentation craft: showing the design decisions, not just the happy path

Projects built:
- Build the complete capstone agent: grounded knowledge, at least two designed topics, one tool or agent flow, one trigger if autonomous, published to your chosen channel

Practice: Run the dress rehearsal for a family member and cut or fix the two weakest moments of the demo.

Assessment: Milestone gate: the capstone passes its own test set and pre-flight checklist before demo day.

Class 24: Demo Day and What Comes Next

Topics:
- The live demo: your capstone, in front of a real audience, on a real channel, with citations showing
- Defending design decisions: why these sources, these topics, this tool, this trigger
- The retrospective: what you would build differently, the most honest slide in any deck
- Your portfolio: the capstone plus the practice agents, packaged so a school or interviewer can browse them
- The credential road: Microsoft's PL-900 Power Platform Fundamentals now has a Copilot Studio agents domain, and the Applied Skills lab 'Build an agent in Microsoft Copilot Studio' is the natural next rung
- Where this skill goes: Gartner projects that by 2029 at least half of knowledge workers will need to work with, govern or create AI agents
- Certificate, feedback, and the Living Syllabus promise: what changed in Copilot Studio while you were learning it

Projects built:
- Deliver the capstone demo and hand over the full packet: spec, source list, test log, audit page and a two-minute recorded walkthrough

Practice: Write your next agent's one-page spec unprompted, because you will not stop at one.

Assessment: Final evaluation: capstone demo, design defence, and the complete documentation packet, graded against the rubric from Class 20.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: an agent teardown, Study Buddy v1 from a plain-English description, a refusal gauntlet, a permanent 15-question test set, and a full map of how a question travels through the platform
- Phase 2: a website-grounded Study Buddy with citations, a 3-document Revision Coach with a proof test, a two-source Club Desk with conflict handling, and a published accuracy audit
- Phase 3: a Homework Planner with designed topics, entity-driven task capture, a condition-driven Event Helper, card-based weekly views, and a generative-orchestration migration with documented behaviour changes
- Phase 4: a flashcard-maker Prompt tool, a Permission Slip agent flow with graceful errors, a real connector tool reading and writing live data, the autonomous Drop Folder Librarian, its hardened successor with approval pauses, and an analytics sprint with shipped improvements
- Phase 5: a published agent tested by real users, a signed 20-point pre-flight audit, the complete capstone build, and a demo-day presentation with a full documentation packet
- Total: 24 hands-on projects, one per class, finishing with a published capstone agent

Total projects: 24 hands-on projects, one per class, finishing with a published capstone agent

SKILLS THE STUDENT WILL HAVE AT THE END

- Agent design: instructions, scope fences, refusals and tone control
- Knowledge grounding: websites, documents and SharePoint sources with citations
- Conversation design: topics, trigger phrases, entities, variables and condition logic
- Adaptive cards and mobile-first response design
- Generative orchestration vs classic orchestration, chosen deliberately
- Tools: Prompt tools, connectors, agent flows, and a working understanding of REST and Model Context Protocol
- Autonomous agents: event triggers, recurrence schedules, human-in-the-loop approvals
- Analytics and evaluation: outcomes, transcripts, test sets and the improvement loop
- Publishing: demo and custom websites, Teams, and the pre-flight checklist that gates a launch
- Responsible AI habits: grounding, transparency, data minimisation and injection awareness

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 private 1-on-1 classes per week, about 1 hour each, across 12 weeks
Hands on agent work: 1-1.5 hours building and testing in Copilot Studio between classes
Project time: 1 hour completing each class project, usually inside your own agents
Review and doubts: 20-40 minutes reviewing feedback and clearing doubts over WhatsApp before the next class
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Dedicated mentor: The same mentor for all 24 classes, who knows exactly where your agents and your understanding stand
Whatsapp doubts: Doubt support between classes over WhatsApp
Practice environment: Help setting up your Copilot Studio practice environment in Class 1, including the account path for students under 18
Session recordings: Class recordings available for revision, so a missed detail is never lost

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- A visible head start toward roles that now appear in real job listings: AI agent builder, automation specialist, Power Platform maker
- Microsoft's own credential ladder: PL-900 Power Platform Fundamentals (which now includes a Copilot Studio agents domain), then the Applied Skills lab 'Build an agent in Microsoft Copilot Studio'
- A published capstone agent and portfolio that make school projects, internship applications and university statements concrete
- The design-and-test discipline (grounding, refusals, evaluation) that transfers to every AI platform, not just Microsoft's
- A natural bridge into our Codex + Claude Code course when you are ready to build agents with real code

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Does my teenager need to know coding to build AI agents in Copilot Studio?
A: No. Microsoft Copilot Studio is a graphical, low-code studio: agents are built from instructions written in plain English, knowledge sources, and a visual conversation canvas. The skills that matter are clear thinking, good writing and honest testing, and the course teaches all three.

Q: Why is this course taught 1-on-1 only?
A: Because agent building is a design discipline. The mentor reviews your child's actual instructions, knowledge sources and conversation maps line by line every class, and rebuilds the projects around their real school, clubs and interests. That per-student review loop is what makes students genuinely good at this, and it cannot be done honestly in a group format.

Q: Does my teen need a special Microsoft account for Copilot Studio?
A: Copilot Studio uses a Microsoft work or school account rather than a personal email. Many students already have a school-provided Microsoft account that works. If not, we arrange access to a practice environment for the duration of the course, and for students under 18 a parent or guardian accepts Microsoft's terms. Your mentor handles the setup with you in the first class.

Q: Is Copilot Studio a real professional skill or just a learning tool?
A: It is the same product enterprises run: Microsoft's own materials state that 90% of the Fortune 500 use Copilot Studio, and Microsoft's cloud reported more than 230,000 organizations building with it as far back as April 2025.

Q: How current is the syllabus, given how fast AI tools change?
A: It is a Living Syllabus, re-verified against Microsoft's official documentation and What's New pages as features ship. Students on the current syllabus learn the present-day platform: generative orchestration, the current model lineup including GPT-5.5 Chat and Anthropic Claude models, Model Context Protocol tools, autonomous event triggers and agent analytics. When Copilot Studio changes, the course changes.

Q: What age is this course for, and does it suit complete beginners?
A: It is designed for ages 13-18 and assumes no prior experience with coding or AI tools beyond ordinary chatting. Classes 1-5 build everything from first principles. For students younger than 13 we recommend our kids' coding and AI-literacy courses first, since Microsoft's own AI tools set 13 as the general minimum age for its consumer Copilot.

Q: What does my child have at the end to show for it?
A: A published capstone agent demonstrated live to a real audience, a portfolio of 24 documented projects, a written accuracy audit and pre-flight safety checklist they produced themselves, a Modern Age Coders certificate earned by defending the capstone, and a mapped next step toward Microsoft's PL-900 credential if they want the official ladder.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final project defended live, plus a written assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-course-for-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 9 OF 137: AI AND MACHINE LEARNING FOR TEENS: PYTHON TO REAL MODELS

==========

Course link: https://learn.modernagecoders.com/courses/ai-ml-masterclass-teens
Subject area: AI and Machine Learning
Designed for: Teens
Who can join: Beginner friendly, ages 13 to 18; school maths is enough to start
Course length: 10-12 months (40-48 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

AI and machine learning course for teens aged 13-18: Python, real ML models, neural networks, computer vision, NLP and modern generative AI in 10-12 months.

In one line: Not another course about using AI. Ten months of building it: real models, trained and evaluated by a teenager who understands what is inside them.

ABOUT THE PROGRAMME

Most teens use AI daily; almost none understand it, and understanding is where all the opportunity lives. This live online course takes students aged 13 to 18 from zero, no coding assumed, to genuinely training, evaluating and shipping machine learning models: Python and the data stack first, then real machine learning with scikit-learn, neural networks built up from a single neuron, convolutional networks that see, language models that read, and the modern generative AI layer, understood from the inside and used through teen-legal tools with parent-involved accounts. Honesty shapes the syllabus. Fields like reinforcement learning, robotics and ML infrastructure are named and mapped, not squeezed in as fake weeks; the depth goes where a teen can actually build: vision projects with their own trained models, an NLP project on real text, a generative AI application with guardrails, and a final capstone defended at demo day. The maths is taught as needed, school algebra is enough to start, and every concept lands in code the same week it is explained. The pace is real: ten months at two live classes a week plus three to four hours of practice, with two more months in hand when needed. Homework after every class, monthly mixed reviews that reach back deliberately, phase exams, and a final exam that decides the certificate, with focused revision and a free retest. Real models, real evaluation, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Builders, not passengers: teens train, evaluate and ship their own models, they do not just prompt someone else's
- Maths handled honestly: taught as needed from school algebra, never assumed, never hand-waved
- Evaluation is the moral core: overfitting, leakage and misleading accuracy are taught as hard as the models themselves
- Generative AI from the inside: how LLMs and diffusion actually work, plus building on them through teen-legal tools with parent-involved accounts
- An honest 10-month arc with 2 months of margin: the frontier fields are mapped truthfully as next steps, not faked as weeks
- Real assessment: weekly homework, monthly mixed reviews, and a final exam that decides the certificate, with focused revision and a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Python and first models (Months 1-3): Python, NumPy and pandas, then real machine learning with scikit-learn
Phase 2: Core ML mastery (Months 4-6): regression to unsupervised learning, then neural networks and deep learning fundamentals
Phase 3: Machines that see and read (Months 7-8): CNNs and computer vision projects, NLP and the transformer idea
Phase 4: Modern AI and the capstone (Months 9-10): generative AI understood and used safely, ethics, an AI product build, and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Python is taught from the first line
Maths background: School algebra is enough to start; every further piece of maths is taught exactly when a model needs it
Equipment: A normal laptop with a stable internet connection; free cloud notebooks carry the heavy training
Age: 13 to 18. Batches are grouped by age and pace
Time commitment: 2 live classes plus 3-4 hours of practice a week

WHO THIS COURSE IS FOR

Ai curious: Teens who use AI daily and have started asking the right question: how does this actually work?
Builders: Students who want trained models of their own, not just clever prompts
Future engineers: Teens aiming at computer science and AI tracks who want the real foundation early
Maths appliers: Students who like maths more when it does something, this is where it does the most
Complete beginners: No coding background required: the course starts at the very first line of Python

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PYTHON AND FIRST MODELS (MONTHS 1-3, WEEKS 1-12)

The data stack from zero, Python through pandas, then the moment it all changes: the first real machine learning model, trained and honestly evaluated.

Month 1: Python for AI (Weeks 1-4)
Introduction to AI & Python Setup
Topics:
- What is Artificial Intelligence? Real examples teens use daily
- AI vs ML vs Deep Learning explained with examples
- History of AI: From chess to ChatGPT
- Setting up Python environment (Anaconda, Jupyter)
- Google Colab for free GPU access
- Python basics: Variables, data types, operations
- Lists and dictionaries for data storage
- Functions and code organization
- Python libraries introduction: NumPy, Pandas preview
- Your first AI program: Rule-based chatbot
- How machines 'think': Algorithms vs intelligence
- AI in everyday life: Spotify, TikTok, YouTube recommendations
Projects built:
- Setup complete AI development environment
- Rule-based chatbot (like early Siri)
- Simple recommendation system using rules
- AI decision tree for game choices
Practice: Daily: Python exercises, explore AI applications
Python Data Structures & Control Flow
Topics:
- Advanced Python lists: Slicing, comprehensions

- Working with dictionaries for data mapping
- Sets and tuples for data organization
- If-else logic for AI decisions
- Loops for processing data
- Functions for reusable AI components
- Object-oriented programming basics
- Classes for AI models
- File handling: Reading/writing data
- JSON for AI configuration
- Error handling in AI programs
- Debugging AI code effectively

Projects built:
- Student grade predictor system
- Text-based adventure game with AI NPCs
- Data analyzer for social media stats
- Simple expert system for diagnosis

Practice: Build 10 Python programs with AI logic

Assessment: Month 1 check: a Python data exercise live, plus a take-home mixing everything so far

Month 2: The Data Stack (Weeks 5-8)

NumPy & Mathematical Foundations

Topics:
- NumPy arrays: The foundation of AI
- Array operations and broadcasting
- Mathematical operations on arrays
- Statistics basics: Mean, median, mode, std deviation
- Probability concepts for AI
- Linear algebra basics: Vectors and matrices
- Matrix operations: Addition, multiplication
- Why math matters in AI (visual explanations)
- Plotting with Matplotlib
- Visualizing data patterns
- Random numbers and simulations
- Mathematical functions in NumPy

Projects built:
- Grade distribution analyzer
- Dice probability simulator
- Image manipulation with arrays
- Statistical analysis dashboard

Practice: Complete 20 NumPy exercises daily

Pandas & Data Manipulation

Topics:
- DataFrames: Spreadsheets in Python
- Reading data: CSV, Excel, JSON
- Data selection and filtering
- Handling missing data
- Data cleaning techniques
- Grouping and aggregation
- Merging and joining datasets
- Time series data basics
- Data visualization with Pandas
- Exploratory Data Analysis (EDA)
- Feature engineering introduction
- Preparing data for ML

Projects built:
- YouTube channel analytics tool
- Sports statistics analyzer
- Weather pattern explorer
- Social media trends analyzer

Practice: Analyze 5 real-world datasets

Assessment: Month 2 check: clean and explore a messy real dataset, with quiz questions reaching back to month 1

Month 3: First Machine Learning (Weeks 9-12)

Introduction to Machine Learning

Topics:
- What is Machine Learning? Learning from examples
- Supervised vs Unsupervised vs Reinforcement Learning
- Classification vs Regression problems
- Training data, validation data, test data
- Features and labels explained
- Overfitting and underfitting (Goldilocks principle)
- Model evaluation metrics: Accuracy, precision, recall
- Confusion matrix understanding
- Cross-validation concept
- Bias-variance tradeoff simplified
- Feature scaling and normalization
- Introduction to Scikit-learn

```
      Projects built:
        - Iris flower classifier (classic starter)
        - Spam email detector
        - Music genre classifier
        - Friend group predictor
      Practice: Train 10 different ML models
    Classification Algorithms
      Topics:
        - K-Nearest Neighbors (KNN): Finding similar things
        - Decision Trees: AI making choices
        - Random Forests: Wisdom of crowds
        - Naive Bayes: Probability-based predictions
        - Logistic Regression for classification
        - Support Vector Machines (SVM) basics
        - Ensemble methods: Voting classifiers
        - Multi-class classification
        - Imbalanced data handling
        - Feature importance analysis
        - Model selection strategies
        - Hyperparameter tuning basics
      Projects built:
        - Handwritten digit recognizer
        - Emoji sentiment analyzer
        - Game outcome predictor
        - Fashion item classifier
      Practice: Solve 15 classification challenges
      Assessment: Phase 1 exam: train and evaluate a classifier live and explain its
      mistakes, plus a written mixed paper

----------------------------------------------------------------
PHASE 2: CORE MACHINE LEARNING (MONTHS 4-6, WEEKS 13-24)
----------------------------------------------------------------
The full classical toolkit, regression to clustering, then neural networks built from a
single neuron to real deep learning, with data preparation as a discipline.

  Month 4: Prediction and Patterns (Weeks 13-16)
    Regression & Prediction
      Topics:
        - Linear Regression: Finding patterns in data
        - Polynomial Regression: Curved relationships
        - Multiple Linear Regression
        - Ridge and Lasso Regression
        - Regression evaluation metrics: MSE, RMSE, R²
        - Time series prediction basics
        - Feature engineering for regression
        - Dealing with outliers
        - Residual analysis
        - Gradient Descent visualization
        - Learning rate importance
        - Regression vs Classification: When to use what
      Projects built:
        - House price predictor
        - Exam score predictor
        - YouTube views predictor
        - Weather temperature forecaster
      Practice: Build 10 prediction models
    Unsupervised Learning
      Topics:
        - Clustering: Finding groups in data
        - K-Means clustering algorithm
        - Hierarchical clustering
        - DBSCAN for density-based clustering
        - Dimensionality reduction: PCA basics
        - t-SNE for visualization
        - Anomaly detection methods
        - Association rules (Market basket)
        - Clustering evaluation metrics
        - Choosing number of clusters
        - Real-world clustering applications
        - Visualization of high-dimensional data
      Projects built:
        - Customer segmentation tool
        - Music playlist generator
        - Anomaly detector for gaming
        - Friend group discoverer
      Practice: Apply clustering to 10 datasets
      Assessment: Month 4 check: a regression-and-clustering task on fresh data, plus mixed
      review
```

Month 5: Neural Networks (Weeks 17-20)
  Advanced ML: Ensembles and Model Craft
    Topics:
      - Gradient Boosting Machines (GBM)
      - XGBoost: Competition winner
      - LightGBM for speed
      - CatBoost for categorical data
      - Stacking and blending models
      - Feature engineering mastery
      - Automated feature selection
      - Cross-validation strategies
      - Hyperparameter optimization: Grid, Random, Bayesian
      - Pipeline creation in Scikit-learn
      - Model interpretability: SHAP, LIME
      - Handling big data with ML
    Projects built:
      - Kaggle competition entry
      - Advanced recommendation system
      - Multi-model ensemble predictor
      - AutoML system builder
    Practice: Optimize 10 ML models for performance
  Introduction to Neural Networks
    Topics:
      - Biological inspiration: How brains work
      - Artificial neurons (Perceptrons)
      - Activation functions: ReLU, Sigmoid, Tanh
      - Forward propagation explained
      - Backpropagation intuition
      - Gradient descent deep dive
      - Building neural networks from scratch
      - Introduction to TensorFlow/Keras
      - Your first neural network
      - Training neural networks
      - Preventing overfitting: Dropout, regularization
      - Batch normalization basics
    Projects built:
      - Neural network from scratch (NumPy)
      - XOR problem solver
      - Simple pattern recognizer
      - Basic neural network visualizer
    Practice: Build 5 neural networks for different tasks
    Assessment: Month 5 check: build a small neural network by hand and explain each part,
    plus mixed review

Month 6: Deep Learning Fundamentals (Weeks 21-24)
  Deep Learning Fundamentals
    Topics:
      - What makes learning 'deep'?
      - Deep vs shallow networks
      - TensorFlow 2.0 and Keras API
      - Building deep neural networks
      - Optimizers: SGD, Adam, RMSprop
      - Learning rate scheduling
      - Early stopping and callbacks
      - Model checkpointing
      - Transfer learning concept
      - Pre-trained models introduction
      - GPU acceleration basics
      - Debugging neural networks
    Projects built:
      - MNIST digit classifier (deep version)
      - Fashion MNIST challenger
      - Simple image classifier
      - Sound pattern recognizer
    Practice: Train 10 deep learning models
  Data Preparation and Augmentation
    Topics:
      - Why data augmentation matters
      - Image augmentation techniques
      - Text augmentation methods
      - Audio data preprocessing
      - Synthetic data generation
      - Data balancing techniques
      - SMOTE for imbalanced data
      - Feature extraction from images
      - Text vectorization methods
      - Handling sequential data
      - Data pipeline optimization
      - TensorFlow Data API

      Projects built:
        - Data augmentation toolkit
        - Synthetic data generator
        - Image preprocessing pipeline
        - Text data preparation system
      Practice: Create 5 augmented datasets
      Assessment: Phase 2 exam: train a deep model with proper splits and augmentation, plus
      a written mixed paper over phases 1-2

PHASE 3: MACHINES THAT SEE AND READ (MONTHS 7-8, WEEKS 25-32)

The two great applied fields at teen-buildable depth: convolutional networks and computer
vision projects, then NLP and the transformer idea that changed everything.

  Month 7: Computer Vision (Weeks 25-28)
    Convolutional Neural Networks (CNNs)
      Topics:
        - How computers see: Pixels to features
        - Convolution operation explained visually
        - Filters and feature maps
        - Pooling layers: Max and average
        - CNN architecture design
        - Famous architectures: LeNet, AlexNet, VGG
        - ResNet and skip connections
        - Building CNNs in TensorFlow/Keras
        - Image classification with CNNs
        - Data augmentation for images
        - Visualizing CNN layers
        - Understanding what CNNs learn
      Projects built:
        - Custom image classifier (personal photos)
        - Face emotion detector
        - Hand gesture recognizer
        - Pet breed identifier
      Practice: Build 10 different CNN architectures
    Advanced Computer Vision
      Topics:
        - Object detection: Finding things in images
        - YOLO (You Only Look Once) basics
        - Face detection and recognition
        - Image segmentation techniques
        - Style transfer: Making art with AI
        - Image generation basics
        - OpenCV for computer vision
        - Real-time video processing
        - Pose estimation
        - Optical Character Recognition (OCR)
        - Medical image analysis basics
        - AR filters like Snapchat
      Projects built:
        - Object detection app
        - Snapchat-style filter creator
        - Document scanner with OCR
        - Real-time pose detector for fitness
      Practice: Create 8 computer vision applications
      Assessment: Month 7 check: an image classifier trained on your own collected dataset,
      plus mixed review

  Month 8: Language (Weeks 29-32)
    Natural Language Processing Basics
      Topics:
        - How computers understand text
        - Tokenization and text preprocessing
        - Word embeddings: Word2Vec, GloVe
        - Sentiment analysis techniques
        - Named Entity Recognition (NER)
        - Part-of-speech tagging
        - Text classification methods
        - Topic modeling with LDA
        - Text summarization basics
        - Machine translation introduction
        - Chatbot frameworks
        - NLTK and spaCy libraries
      Projects built:
        - Sentiment analyzer for reviews
        - Fake news detector
        - Chatbot for customer service
        - Text summarizer for articles

```
      Practice: Build 10 NLP applications
    Transformers and Modern NLP
      Topics:
        - Attention mechanism explained
        - Transformer architecture basics
        - BERT for understanding text
        - GPT models introduction
        - Fine-tuning pre-trained models
        - Hugging Face library basics
        - Zero-shot classification
        - Question answering systems
        - Text generation with GPT
        - Prompt engineering basics
        - Using APIs: OpenAI, Cohere
        - Responsible AI and bias
        - Why attention changed everything, drawn on one whiteboard
      Projects built:
        - Question-answering system
        - AI writing assistant
        - Language translator
        - Custom GPT for specific domain
      Practice: Fine-tune 5 transformer models
      Assessment: Phase 3 exam: a vision or NLP mini-project defended live, plus a cumulative
      written paper

------------------------------------------------------------
PHASE 4: MODERN AI AND THE CAPSTONE (MONTHS 9-10, WEEKS 33-40)
------------------------------------------------------------
Generative AI understood from the inside and used safely, ethics that actually bites, an AI
product built end to end, and the final exam at demo day.

  Month 9: The Generative Layer (Weeks 33-36)
    Generative AI from the Inside
      Topics:
        - Diffusion models deep dive
        - ControlNet for controlled generation
        - InstructPix2Pix
        - 3D generation: NeRF, 3D GANs
        - Video generation models
        - Audio generation: MusicLM, AudioLM
        - Code generation with AI
        - Generative agents and simulations
        - AI for procedural generation
        - Ethical implications of generative AI
        - Future of generative models
        - How diffusion models paint, concept-first with teen-legal tools like Firefly and Canva
        - Building on LLM APIs with accounts set up alongside a parent
        - Prompts as engineering: structure, iteration, evaluation
      Projects built:
        - Custom image generator
        - AI video creator
        - 3D scene generator
        - Code generation assistant
      Practice: Build 10 generative AI applications
    AI Ethics and Honest Evaluation
      Topics:
        - Bias in AI systems
        - Fairness metrics and evaluation
        - Explainable AI (XAI)
        - Privacy in machine learning
        - Differential privacy basics
        - AI safety considerations
        - Environmental impact of AI
        - AI regulations and compliance
        - Adversarial attacks and defenses
        - Deepfakes and detection
        - AI for social good
        - Future of AI careers
        - Explainability: knowing why a model said what it said
        - Deepfakes and detection: the literacy every teen needs
        - Capstone planning: choosing a project worth defending
      Projects built:
        - Bias detection tool
        - Model explainability dashboard
        - Fairness evaluation system
        - AI ethics case study analysis
      Practice: Evaluate 5 models for bias and fairness
      Assessment: Month 9 check: an ethics-and-evaluation case study argued in writing, plus
      mixed review
```

Month 10: Build, Ship, Defend (Weeks 37-40)
AI Product Week
Topics:
- From model to product: wrapping a trained model in a usable interface
- Streamlit or Gradio: demos people can actually touch
- Guardrails: handling wrong answers gracefully
- Cost and speed: why small models often win
- Writing the honest model card: what it does, where it fails
Projects built:
- Your best model shipped as a small usable app with a model card
Practice: Have three people use your app and log where it failed them
Final Capstone Build
Topics:
- Two focused weeks: data, model, evaluation, interface
- Instructor checkpoints and honest scope cuts
- Evaluation before excitement: the metric chosen and defended
- Daily progress with a training log
- Preparing the demo: the story of the build, failures included
Projects built:
- The capstone: a complete AI project, vision, language, tabular or generative, built and evaluated end to end
Practice: Daily training-log entries; a working demo at all times
Demo Day and What Comes Next
Topics:
- Presenting the capstone: live demo plus the hardest failure story
- An honest skills inventory: strong, shaky, next
- The truthful map of what comes next: reinforcement learning, MLOps and research, as real paths not fake weeks
- Where our college AI/ML and Codex/Claude Code courses continue the road
Projects built:
- Demo day presentation delivered to the batch and parents
Assessment: FINAL EXAM: a practical model-building task plus a written paper with questions mixed from every phase. Passing earns the certificate; a student who falls short gets a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Python data projects, honest exploratory analyses, and a first evaluated classifier
- Phase 2: regression and clustering studies, a hand-built neural network, and a properly augmented deep model
- Phase 3: an image classifier on a self-collected dataset and an NLP project on real text
- Phase 4: a generative AI application with guardrails, a shipped model demo, and the defended capstone

Total projects: 20+ trained models and AI builds, crowned by a defended capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Python for AI: the language plus NumPy and pandas fluency
- Classical ML: classification, regression, clustering and ensembles with scikit-learn
- Deep learning: neural networks from first principles through CNNs
- NLP: text processing through the transformer idea
- Generative AI: how it works inside, and building on it safely through teen-legal tools
- Evaluation: splits, metrics, overfitting and leakage, treated as the moral core
- Shipping: models wrapped in usable demos with honest model cards

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: Homework after every class, and a mixed-review assignment at the end of every month so earlier skills stay sharp
Practice: 3-4 hours of model-building practice between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, for the model that refuses to converge
Progress updates: Regular progress notes to parents: what was built, what comes next, and where encouragement helps

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Our college AI/ML masterclass: the same road at full depth, entered years ahead
- Codex and Claude Code for Teens: building with AI coding agents on top of real understanding
- Data science directions: the pandas and evaluation habits transfer directly
- University AI tracks: this is precisely the foundation they assume
- School science fairs and competitions, entered with genuinely trained models

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What AI projects will my teen actually build?
A: Twenty-plus real builds across ten months: honest data analyses and a first evaluated classifier in phase 1, a hand-built neural network and properly trained deep models in phase 2, an image classifier trained on a dataset they collect themselves and an NLP project on real text in phase 3, and in phase 4 a generative AI application with guardrails plus the final capstone, a complete AI project shipped as a usable demo with a model card, defended at demo day.

Q: How long does it take a teenager to learn AI and machine learning?
A: To genuinely training and evaluating models: 10 months at 2 live classes a week plus 3 to 4 hours of practice, starting from zero coding. The course keeps 2 months of margin in hand for students who need more repetition, up to 12 months with no penalty. The first trained model happens in month 3; the understanding compounds from there.

Q: How is my teen assessed? Is the certificate just for attending?
A: No. Homework follows every class, each month ends with an assignment that deliberately mixes new material with earlier topics, every phase closes with an exam, and the course ends with a final exam: a practical model-building task plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest. In a field this hyped, honest assessment is the product.

Q: How much maths does my teen need beforehand?
A: School algebra is enough to start. Teens consistently find the maths easier here than at school, because it finally does something visible.

Q: Is this course about using AI tools like ChatGPT, or building AI?
A: Building, with using in its proper place. The heart of the course is training and evaluating real models. The generative layer is then learned from the inside, how LLMs and diffusion models actually work, and built upon through APIs and teen-legal tools. A teen who only prompts is a passenger; this course produces drivers. For a tools-first path we run a separate AI Tools course.

Q: What can my teen do after finishing?
A: Enter our college-level AI/ML masterclass years ahead of schedule, take the Codex and Claude Code course to build with AI agents, carry the portfolio into school science fairs and university applications, or continue into data science. The frontier fields, reinforcement learning, robotics, ML infrastructure, are mapped honestly during the course as real next roads.

Q: How do we try a class before enrolling?
A: Book a free demo class with no commitment at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Your teen will train a tiny model inside the demo itself; that moment usually decides things. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live with a teacher?
A: Always: live 1-hour sessions with a real instructor in small groups, never pre-recorded videos. Machine learning is full of silent failure modes, a leaked dataset, a misread metric, and live eyes on a student's notebook is the only honest way to catch them early.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ai-ml-masterclass-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 10 OF 137: AI AND ML MASTERCLASS: PYTHON, DEEP LEARNING AND GENAI
==========

Course link: https://learn.modernagecoders.com/courses/ai-ml-masterclass-complete-college
Subject area: AI and Machine Learning
Designed for: College students
Who can join: Complete beginner (18+); school maths refreshed, everything else built here
Course length: 12-14 months (48-56 weeks)
Time needed: 2 live classes/week + 5-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  AI and machine learning masterclass: maths foundations, Python, scikit-learn, PyTorch deep learning, generative AI and deployment in 12-14 months, live online.

  In one line: Twelve months, honestly spent: the mathematics, the models, the modern generative layer, and enough engineering to ship what you train.

ABOUT THE PROGRAMME

  AI and machine learning is the deepest track we teach, and this masterclass respects that: twelve months from zero, no shortcuts and no skipped mathematics. The foundations phase builds linear algebra, calculus intuition, probability and statistics alongside Python and the data stack, because everything downstream stands on them. Classical machine learning follows with scikit-learn, classification, feature engineering, unsupervised learning, and evaluation taught as the discipline that separates practitioners from demo-makers. The deep learning phase goes through PyTorch, the framework research and industry actually share, into CNNs, computer vision and NLP up to transformers. The final phase is the modern layer: generative AI and LLM engineering, retrieval, fine-tuning concepts, honest evaluation of generated output, plus deployment fundamentals, serving a model, tracking experiments, monitoring drift, and a capstone shipped and defended. What this course refuses to do is fake the frontier: reinforcement learning, large-scale ML infrastructure and research careers are mapped honestly as next roads, not squeezed into hollow weeks. The reclaimed time goes into depth where it pays: evaluation, error analysis, and the engineering judgment interviews actually probe. The pace is real: twelve months at two live classes a week plus five to six hours of practice, with two more months in hand when needed. Problem sets after every class, monthly mixed reviews, phase exams, and a final exam that decides the certificate, with focused revision and a free retest. Real models, real evaluation, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

  - Mathematics first, honestly: linear algebra, calculus and statistics built in-course, not waved at
  - PyTorch as the spine of deep learning: the framework research and industry actually share
  - Evaluation and error analysis treated as the core skill, because that is what interviews and real work probe
  - A genuinely modern generative phase: LLM engineering, retrieval, fine-tuning concepts and honest output evaluation
  - Deployment fundamentals included: serving, experiment tracking and monitoring, without pretending to be an infra course
  - Real assessment: weekly problem sets, monthly mixed reviews, and a final exam that decides the certificate, with focused revision and a free retest

THE LEARNING PATH, STEP BY STEP

  Phase 1: Foundations (Months 1-3): linear algebra, calculus, probability and statistics, with Python, NumPy, pandas and EDA
  Phase 2: Classical ML (Months 4-6): scikit-learn end to end, feature engineering, unsupervised learning, ensembles and rigorous evaluation
  Phase 3: Deep learning (Months 7-9): PyTorch, CNNs and computer vision, NLP through transformers
  Phase 4: GenAI and shipping (Months 10-12): LLM engineering and retrieval, deployment and monitoring, ethics, interviews, and the defended capstone

WHAT A STUDENT NEEDS BEFORE STARTING

  Coding experience: None required; Python is built within phase 1
  Maths background: School mathematics; the course rebuilds everything it needs from there
  Equipment: A normal laptop with a stable internet connection; free cloud GPUs carry the heavy training
  Age: 18+; college students, graduates and working professionals
  Time commitment: 2 live classes plus 5-6 hours of practice a week

WHO THIS COURSE IS FOR

  Career switchers: Professionals moving toward ML roles who want the real foundation, mathematics included
  College students: Students who want depth their electives gesture at, plus a deployed portfolio
  Developers: Software engineers adding the ML and GenAI layer properly
  Analysts: Data analysts stepping up from dashboards to models
  Serious beginners: Anyone ready to trade twelve honest months for a real capability

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: MATHEMATICS, PYTHON AND DATA (MONTHS 1-3, WEEKS 1-12)

The load-bearing floor: linear algebra, calculus intuition, probability and statistics, built alongside Python and real exploratory data work.

Month 1: The Mathematical Core I (Weeks 1-4)

Mathematics Prerequisites & Linear Algebra - Part 1

Topics:

- Why mathematics matters in AI/ML
- Basic arithmetic and algebra review
- Functions and graphs
- Introduction to vectors: geometric and algebraic view
- Vector operations: addition, scalar multiplication
- Dot product and geometric interpretation
- Vector norms (L1, L2, infinity norm)
- Matrices: definition and notation
- Matrix operations: addition, multiplication
- Matrix transpose and symmetric matrices
- Identity matrix and inverse matrices
- Matrix properties and rules
- Systems of linear equations
- Gaussian elimination
- Matrix rank and determinant (basics)

Projects built:

- Vector operations visualizer
- Matrix calculator implementation
- Linear equation solver
- Geometric transformations with matrices

Practice: Solve 50 linear algebra problems, implement from scratch in Python

Linear Algebra - Part 2 & Calculus Fundamentals

Topics:

- Eigenvalues and eigenvectors: concept and computation
- Eigendecomposition
- Singular Value Decomposition (SVD)
- Principal Component Analysis (PCA) mathematics
- Vector spaces and subspaces
- Linear transformations
- Introduction to calculus: limits
- Derivatives: definition and rules
- Power rule, product rule, quotient rule, chain rule
- Partial derivatives
- Gradient: vector of partial derivatives
- Gradient descent concept
- Second derivatives and Hessian matrix
- Taylor series expansion
- Optimization basics: finding minima and maxima

Projects built:

- PCA implementation from scratch
- Image compression using SVD
- Gradient descent visualizer
- Function optimizer using calculus

Practice: Solve 60 calculus problems, implement gradient descent variants

Assessment: Month 1 check: a linear algebra and calculus problem set, marked line by line

Month 2: Probability, Statistics and Python (Weeks 5-8)

Probability & Statistics - Part 1

Topics:

- Probability fundamentals: sample space, events
- Probability rules: addition, multiplication
- Conditional probability and Bayes' theorem
- Independent vs dependent events
- Random variables: discrete and continuous
- Probability distributions: uniform, Bernoulli, binomial
- Normal (Gaussian) distribution: properties and applications
- Poisson distribution
- Exponential distribution
- Probability density functions (PDF)
- Cumulative distribution functions (CDF)
- Expected value (mean) and variance
- Standard deviation and standard error
- Covariance and correlation
- Law of large numbers and Central Limit Theorem

Projects built:
- Probability calculator
- Distribution visualizer
- Bayes theorem applications
- Monte Carlo simulations
- Statistical analysis tool

Practice: Solve 70 probability and statistics problems

Statistics - Part 2 & Python for ML

Topics:
- Descriptive statistics: mean, median, mode
- Measures of spread: range, IQR, variance, std dev
- Hypothesis testing fundamentals
- Null and alternative hypotheses
- P-values and significance levels
- T-tests, Z-tests, Chi-square tests
- ANOVA (Analysis of Variance)
- Confidence intervals
- Statistical inference
- Python setup for ML: Anaconda, Jupyter
- NumPy for numerical computing
- Pandas for data manipulation
- Matplotlib and Seaborn for visualization
- Statistical computing with SciPy

Projects built:
- Hypothesis testing framework
- A/B testing simulator
- Statistical analysis dashboard
- Data exploration toolkit with Python
- Interactive visualization app

Practice: Statistical analysis of 10 real datasets

Assessment: Month 2 check: a statistics-in-Python exercise, plus mixed review of month 1

Month 3: The Data Stack (Weeks 9-12)

Advanced NumPy & Pandas

Topics:
- Advanced NumPy: broadcasting, vectorization
- NumPy for linear algebra
- NumPy random number generation
- Pandas DataFrames: advanced operations
- Data cleaning: handling missing values
- Data transformation: apply, map, applymap
- GroupBy operations and aggregations
- Merging, joining, and concatenating
- Time series data handling
- Categorical data handling
- Multi-index DataFrames
- Performance optimization in Pandas

Projects built:
- Complete data cleaning pipeline
- Time series analysis tool
- Data aggregation and reporting system
- Advanced data transformations
- Performance benchmarking study

Practice: Clean and analyze 15 messy datasets

Data Visualization & EDA

Topics:
- Principles of data visualization
- Matplotlib deep dive: customization
- Seaborn for statistical plots
- Plotly for interactive visualizations
- Distribution plots: histograms, KDE, box plots
- Relationship plots: scatter, line, regression
- Categorical plots: bar, count, violin
- Heatmaps and correlation matrices
- Pair plots and joint plots
- Exploratory Data Analysis (EDA) process
- Identifying patterns and anomalies
- Feature correlation analysis
- Handling outliers
- Data storytelling with visualizations
- SQL for data work: the one honest lesson every ML practitioner needs

Projects built:
- Complete EDA on Titanic dataset
- House prices EDA and insights
- Customer segmentation visualization
- COVID-19 data analysis and dashboard
- Interactive data exploration tool

Practice: Perform comprehensive EDA on 10 datasets

Assessment: Phase 1 exam: a full EDA on a fresh dataset with statistical claims defended, plus a written mixed paper

---

PHASE 2: CLASSICAL MACHINE LEARNING (MONTHS 4-6, WEEKS 13-24)

The working core of applied ML: scikit-learn end to end, feature craft, unsupervised methods, ensembles, and evaluation as a discipline.

Month 4: First Models Done Right (Weeks 13-16)

Introduction to Machine Learning

Topics:

- What is Machine Learning? AI vs ML vs DL
- Types of ML: supervised, unsupervised, reinforcement
- ML workflow: problem definition to deployment
- Data preparation for ML
- Train-test split and validation set
- Cross-validation: k-fold, stratified k-fold
- Overfitting and underfitting
- Bias-variance tradeoff
- Evaluation metrics: accuracy, precision, recall, F1
- Confusion matrix
- ROC curve and AUC
- Scikit-learn library introduction
- Linear Regression: theory and mathematics
- Gradient descent for linear regression
- Cost function (MSE) and optimization

Projects built:

- House price prediction with linear regression
- Salary prediction model
- Sales forecasting
- Custom linear regression from scratch
- Gradient descent visualizer

Practice: Implement linear regression from scratch, solve 30 regression problems

Classification Algorithms - Part 1

Topics:

- Logistic Regression: binary classification
- Sigmoid function and probability interpretation
- Log loss (binary cross-entropy)
- Multi-class classification: one-vs-rest, softmax
- Decision Trees: splitting criteria (Gini, entropy)
- Information gain and entropy
- Tree pruning to prevent overfitting
- Random Forest: ensemble of trees
- Bagging and bootstrap aggregating
- Feature importance in Random Forest
- K-Nearest Neighbors (KNN): distance-based learning
- Choosing K and distance metrics
- Naive Bayes classifier: probabilistic approach
- Support Vector Machines (SVM): margin maximization
- Kernel trick for non-linear classification

Projects built:

- Email spam classifier (Naive Bayes)
- Iris flower classification (multi-class)
- Credit card fraud detection
- Customer churn prediction
- Disease prediction (Random Forest)
- Handwritten digit classification (KNN)
- Text classification with SVM

Practice: Build 15 classification models on different datasets

Assessment: Month 4 check: train, tune and defend a classifier on fresh data, plus mixed review

Month 5: Feature Craft and Structure (Weeks 17-20)

Feature Engineering & Preprocessing

Topics:

- Feature engineering importance
- Feature scaling: standardization vs normalization
- Min-Max scaling and Standard scaling
- Robust scaling for outliers
- Handling categorical features: one-hot encoding
- Label encoding and ordinal encoding
- Target encoding
- Feature creation: polynomial features
- Interaction features
- Binning and discretization
- Text features: Bag of Words, TF-IDF
- Feature selection: filter methods
- Wrapper methods: forward/backward selection

- Embedded methods: Lasso, Ridge
- Dimensionality reduction: PCA

Projects built:
- Feature engineering pipeline
- Automated feature selection tool
- Text feature extraction system
- Dimensionality reduction visualizer
- Complete preprocessing pipeline

Practice: Engineer features for 20 different datasets

Unsupervised Learning

Topics:
- Clustering: grouping similar data
- K-Means clustering: algorithm and initialization
- Elbow method for choosing K
- Hierarchical clustering: agglomerative and divisive
- Dendrograms
- DBSCAN: density-based clustering
- Gaussian Mixture Models (GMM)
- Anomaly detection techniques
- Dimensionality reduction: PCA (practical)
- t-SNE for visualization
- UMAP for dimensionality reduction
- Association rule mining: Apriori algorithm
- Market basket analysis
- Autoencoders for unsupervised learning (intro)

Projects built:
- Customer segmentation with K-Means
- Image compression with PCA
- Anomaly detection in transactions
- Document clustering
- Market basket analysis for retail
- High-dimensional data visualization
- Recommendation system basics

Practice: Cluster 15 different datasets, visualize with t-SNE/UMAP

Assessment: Month 5 check: a clustering-and-features task with honest interpretation, plus mixed review

Month 6: Ensembles, Evaluation and the Phase Capstone (Weeks 21-24)

Model Evaluation & Validation

Topics:
- Evaluation metrics deep dive
- Classification metrics: precision, recall, F1, AUC-ROC
- Multi-class metrics: macro vs micro averaging
- Regression metrics: MAE, MSE, RMSE, R^2
- Custom metrics creation
- Cross-validation strategies
- Stratified sampling
- Time series cross-validation
- Model interpretation: SHAP values
- LIME for local interpretability
- Feature importance analysis
- Partial dependence plots
- Model debugging and error analysis
- Bias and fairness in ML models
- A/B testing for ML models
- Ensemble methods: bagging, boosting and when they stop helping

Projects built:
- Model evaluation framework
- Model interpretation dashboard
- Bias detection tool
- A/B testing simulator
- Error analysis toolkit

Practice: Evaluate and interpret 20 different models

Phase 2 Capstone

Topics:
- Complete ML pipeline development
- Problem definition and data collection
- EDA and feature engineering
- Model selection and training
- Hyperparameter tuning
- Model evaluation and interpretation
- Documentation and presentation

Projects built:
- PHASE 2 CAPSTONE: End-to-End ML Project
- Option 1: Predict customer lifetime value (regression + classification)
- Option 2: Build complete recommender system
- Option 3: Time series forecasting for business metrics
- Requirements: Complete pipeline, feature engineering, ensemble methods, model interpretation, detailed report

Assessment: Phase 2 exam: a complete classical-ML project defended, plus a cumulative written paper over phases 1-2

PHASE 3: DEEP LEARNING WITH PYTORCH (MONTHS 7-9, WEEKS 25-36)

The modern engine room: PyTorch fluency, convolutional networks and real computer vision, NLP through the transformer.

Month 7: Into Deep Learning (Weeks 25-28)

Introduction to Deep Learning

Topics:

- Why deep learning?
- Neural networks basics: perceptron
- Activation functions: sigmoid, tanh, ReLU
- Forward propagation
- Backpropagation algorithm
- Gradient descent variants: SGD, momentum, Adam
- Loss functions for deep learning
- TensorFlow basics
- Keras Sequential API
- Building first neural network
- Training deep networks
- Regularization: dropout, L1, L2
- Batch normalization

Projects built:

- Neural network from scratch (NumPy)
- MNIST digit classification (MLP)
- Binary classification with NN
- Multi-class classification NN
- Regression with neural networks

Practice: Build 10 neural networks with Keras

PyTorch in Depth

Topics:

- PyTorch fundamentals
- Tensors and autograd
- Building models with nn.Module
- PyTorch data loading: Dataset, DataLoader
- Custom datasets creation
- Training loops in PyTorch
- GPU acceleration with PyTorch
- TorchVision for computer vision
- TorchText for NLP
- PyTorch Lightning for cleaner code
- Model checkpointing
- TensorBoard with PyTorch
- PyTorch vs TensorFlow comparison

Projects built:

- Image classifier in PyTorch
- Custom CNN architecture
- Transfer learning in PyTorch
- RNN/LSTM in PyTorch
- GAN in PyTorch
- PyTorch Lightning project
- Multi-GPU training

Practice: Reimplement 15 projects in PyTorch

Assessment: Month 7 check: build and train a network in raw PyTorch live, plus mixed review

Month 8: Computer Vision (Weeks 29-32)

Convolutional Neural Networks (CNN)

Topics:

- Limitations of fully connected networks for images
- Convolution operation: filters and feature maps
- Padding and stride
- Pooling layers: max pooling, average pooling
- CNN architecture components
- LeNet architecture
- AlexNet and ImageNet revolution
- VGGNet: deep and simple
- Inception/GoogLeNet: multi-scale features
- ResNet: residual connections and skip connections
- Transfer learning with pretrained models
- Fine-tuning strategies
- Data augmentation for images

Projects built:

- Image classification with CNN
- CIFAR-10 classification
- Transfer learning with ResNet

- Custom CNN architecture
- Dog vs Cat classifier
- Data augmentation pipeline
- Feature visualization in CNNs

Practice: Build 15 computer vision models

Advanced Computer Vision

Topics:

- Object detection: R-CNN, Fast R-CNN, Faster R-CNN
- YOLO (You Only Look Once): real-time detection
- SSD (Single Shot Detector)
- Semantic segmentation: FCN, U-Net
- Instance segmentation: Mask R-CNN
- Face recognition and verification
- Siamese networks
- Image generation introduction
- Style transfer with CNNs
- OpenCV for computer vision
- Image preprocessing techniques
- Handling class imbalance in CV
- Model optimization for edge devices

Projects built:

- Object detection system (YOLO)
- Face recognition application
- Image segmentation for medical images
- Style transfer implementation
- Real-time object detection
- Custom dataset object detector
- Siamese network for similarity

Practice: Complete 10 computer vision projects

Assessment: Month 8 check: a transfer-learning vision project with honest error analysis, plus mixed review

Month 9: Language and Transformers (Weeks 33-36)

Natural Language Processing (NLP) - Part 1

Topics:

- NLP pipeline and challenges
- Text preprocessing: tokenization, lowercasing
- Stop words removal and stemming/lemmatization
- Bag of Words (BoW) and TF-IDF review
- Word embeddings: Word2Vec (CBOW, Skip-gram)
- GloVe embeddings
- FastText embeddings
- Using pretrained embeddings
- Text classification with embeddings
- Named Entity Recognition (NER)
- Part-of-Speech (POS) tagging
- Dependency parsing basics
- spaCy for NLP
- Why attention replaced recurrence: the honest one-lesson history

Projects built:

- Sentiment classifier with embeddings
- Spam detection (email/SMS)
- Named Entity Recognition system
- Text summarization basics
- Question answering system (simple)
- Topic modeling with LDA
- Language detection

Practice: Complete 15 NLP tasks

Advanced NLP and the Transformer

Topics:

- Transformer architecture: self-attention
- Multi-head attention
- Positional encoding
- BERT: Bidirectional Encoder Representations
- GPT: Generative Pre-trained Transformer
- T5, RoBERTa, ALBERT
- Hugging Face Transformers library
- Fine-tuning BERT for classification
- Zero-shot and few-shot learning
- Prompt engineering basics
- Modern NLP pipeline
- Production NLP systems

Projects built:

- BERT fine-tuning for sentiment
- Question-answering with BERT
- Text classification with Transformers
- Named Entity Recognition with BERT
- Summarization with T5
- Text generation with GPT

- Multi-task NLP model
Practice: Fine-tune 10 transformer models
Assessment: Phase 3 exam: a vision or NLP project defended end to end, plus a cumulative written paper

---

PHASE 4: GENERATIVE AI, SHIPPING AND THE CAPSTONE (MONTHS 10-12, WEEKS 37-48)

The modern layer done properly: LLM engineering and retrieval, deployment and monitoring fundamentals, responsible AI, interview craft, and the defended capstone.

Month 10: Generative AI and LLM Engineering (Weeks 37-40)

LLM Engineering

Topics:

- How large language models actually work: tokens, attention, training and alignment in honest depth
- The frontier landscape read soberly: GPT-class, Claude-class and open-weight Llama-class models
- Prompting as engineering: structure, iteration and evaluation harnesses
- Retrieval-augmented generation: embeddings, vector search, and when plain search wins
- Fine-tuning and adapters: what they change, what they cost, when they are wrong
- Structured output and function calling: making models act reliably
- Generative image models: how diffusion works and where it fits
- Evaluating generated output without fooling yourself
- Cost, latency and caching: the engineering triangle of GenAI

Projects built:

- A retrieval-augmented assistant over a real document set, with an evaluation harness
- A structured-output pipeline that survives adversarial inputs

Practice: Run 50 evaluated queries through your assistant and publish the score honestly

Deployment and Experiment Tracking

Topics:

- ML model lifecycle
- Model serialization: pickle, joblib
- Flask for ML APIs
- FastAPI for high-performance APIs
- REST API design for ML
- Input validation and preprocessing
- Model serving basics
- Docker for ML applications
- Creating ML microservices
- Model versioning strategies
- A/B testing deployed models
- Monitoring model performance
- Experiment tracking: knowing which run produced which model
- Serving a model behind an API, measured under load

Projects built:

- ML model API with Flask
- FastAPI ML service
- Dockerized ML application
- Model versioning system
- Simple ML deployment pipeline

Practice: Deploy 10 ML models as APIs

Assessment: Month 10 check: ship one model behind an API with tracked experiments, plus mixed review

Month 11: Production Habits and Responsible AI (Weeks 41-44)

MLOps Fundamentals, Honestly Scoped

Topics:

- What is MLOps? DevOps for ML
- ML system architecture
- ML pipeline orchestration
- Kubeflow for ML workflows
- Apache Airflow for ML pipelines
- Feature stores: Feast, Tecton
- Model registry: MLflow, DVC
- Experiment tracking at scale
- Metadata management
- Data lineage and provenance
- Continuous training (CT)
- Automated retraining pipelines
- Model governance
- Monitoring and drift: knowing when your model quietly went stale

Projects built:

- Complete MLOps pipeline
- Automated ML workflow with Airflow
- Feature store implementation
- Model registry setup
- Continuous training system

- End-to-end ML automation
Practice: Build 5 MLOps pipelines
Responsible AI and Explainability
Topics:
- Ethics in AI/ML
- Bias in machine learning
- Fairness metrics and definitions
- Detecting and mitigating bias
- Interpretable vs explainable AI
- LIME and SHAP for explanations
- Privacy in ML: differential privacy
- Federated learning for privacy
- Adversarial attacks on ML models
- Adversarial training for robustness
- Regulatory compliance: GDPR, AI Act
- Responsible AI frameworks
- Ethical considerations in deployment
- Explainability tools in practice: SHAP-style reasoning about model decisions
- Capstone planning: choosing a project worth defending
Projects built:
- Bias detection in models
- Fairness-aware classifier
- Model explanation dashboard
- Privacy-preserving ML experiment
- Adversarial examples generation
- Robust model training
- Ethical AI case studies
Practice: Audit models for bias and fairness
Assessment: Month 11 check: an ethics-and-monitoring case argued in writing, plus cumulative review

Month 12: Capstone, Interviews and the Final Exam (Weeks 45-48)
Capstone Build
Topics:
- Two focused weeks: data, model, evaluation, deployment
- Instructor checkpoints and honest scope cuts
- Error analysis before excitement: the metric chosen and defended
- The capstone served behind an API with tracked experiments
- A model card a skeptic could audit
Projects built:
- The capstone: a complete ML or GenAI system, trained, evaluated, deployed and documented
Practice: Daily training log; a working deployed build at all times
ML Interview Craft
Topics:
- ML interview preparation strategy
- Coding interviews for ML roles
- ML theory and concepts questions
- System design for ML systems
- Take-home assignments approach
- Portfolio projects presentation
- GitHub for ML engineers
- Networking in AI community
- Conferences and meetups
- Building personal brand
- Salary negotiation for ML roles
- Career paths in AI/ML
- Walking through your own capstone the way interviews actually probe it
Projects built:
- ML interview preparation guide
- Portfolio website with projects
- Technical blog writing
- GitHub profile optimization
- Mock interview practice
- System design case studies
Practice: Daily LeetCode, ML questions, system design
Demo Day and the Final Exam
Topics:
- Presenting the capstone: live system plus the hardest failure story
- A full-course spiral review before the exam
- The honest map of next roads: reinforcement learning, ML infrastructure, research
- Where our agents and Codex/Claude Code courses continue the modern layer
Projects built:
- Demo day presentation delivered to the batch
Assessment: FINAL EXAM: a practical modeling task plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: mathematics problem sets that survive marking, and full EDAs with defended statistical claims
- Phase 2: classical ML projects with feature craft, ensembles and rigorous evaluation
- Phase 3: PyTorch builds, a transfer-learning vision project and an NLP project through transformers
- Phase 4: a retrieval-augmented assistant with an evaluation harness, a deployed model, and the defended capstone

Total projects: 25+ trained and evaluated builds, crowned by a deployed, defended capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- The mathematics: linear algebra, calculus intuition, probability and statistics, actually used
- Classical ML: scikit-learn end to end with feature engineering and honest evaluation
- Deep learning: PyTorch fluency, CNNs, transfer learning and NLP through transformers
- Generative AI: LLM engineering, retrieval, fine-tuning judgment and output evaluation
- Shipping: serving, experiment tracking, monitoring and model cards
- Judgment: error analysis, ethics and the interview-grade ability to defend every choice

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, worked problems and real notebooks
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 5-6 hours of model-building practice between classes
Review: Every set reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, for the loss curve that refuses to descend
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- ML and data science roles: this is the portfolio and fluency those interviews test
- GenAI engineering: the LLM and retrieval work here is the fastest-growing demand
- Our AI agents courses: Copilot Studio and Codex/Claude Code continue the applied layer
- The honest frontier roads: reinforcement learning, ML infrastructure and research, entered from a real base
- Domain application: the strongest ML careers pair this skill with a field you already know

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 4-7 LPA (Junior Data Scientist/Analyst)
Growing roles: Rs 8-15 LPA (ML Engineer/Data Scientist); Rs 15-25 LPA (Senior ML Engineer)
Experienced roles: Rs 20-40 LPA (Senior ML/AI Engineer, Research Engineer); Rs 30-60 LPA (Lead ML Engineer)
Freelance consulting: Rs 3,000-10,000/hour based on expertise

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this AI and ML masterclass actually cover?
A: The full honest arc: the mathematics (linear algebra, calculus, probability, statistics) built alongside Python; classical machine learning with scikit-learn and evaluation as a discipline; deep learning in PyTorch through CNNs, computer vision and NLP to transformers; and the modern layer, LLM engineering with retrieval and fine-tuning judgment, deployment with experiment tracking and monitoring, responsible AI, and a capstone that is trained, evaluated, deployed and defended.

Q: How long does it take to learn AI and machine learning properly?
A: Twelve months at 2 live classes a week plus 5 to 6 hours of practice, starting from zero, and that is the honest floor when the mathematics is built rather than skipped. The course keeps two more months in hand for learners who need them. Bootcamps compress this by dropping exactly the parts, maths and evaluation, that interviews and real work then expose.

Q: How am I assessed? Is the certificate just for attending?
A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with an exam, and the course ends with a final exam: a practical modeling task plus written questions from the entire syllabus. Passing earns the certificate; falling short earns focused revision and a free retest. In this field, honest assessment is the product.

Q: How much mathematics do I need before joining?
A: School mathematics. Phase 1 rebuilds linear algebra, calculus intuition, probability and statistics from there, in code and pictures alongside symbols, because every later phase stands on them. Learners who skipped maths for years consistently manage; the course assumes rust, not fluency.

Q: Does the course cover generative AI and LLMs seriously?
A: Yes, as engineering rather than magic: how transformers and diffusion actually work, prompting with evaluation harnesses, retrieval-augmented generation over real documents, fine-tuning judgment (what it changes, costs, and when it is wrong), structured output and function calling, and the cost-latency-quality triangle. The month ends with a retrieval assistant you built and scored honestly.

Q: Will I be able to deploy models, or just train them in notebooks?
A: Deploy. The final phase covers serving a model behind an API, experiment tracking, monitoring and drift, and the capstone is required to ship that way with a model card. Full-scale ML infrastructure is its own profession and is mapped honestly as a next road; what you leave with is the working end of it.

Q: Can I try a class before enrolling?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. You will train a small model inside the demo itself. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor reading your actual notebooks, never pre-recorded videos. ML fails silently, leaked data, misread metrics, and live eyes on your work is the only honest way to catch it early.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ai-ml-masterclass-complete-college
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 11 OF 137: AI LITERACY FOR KIDS (AGES 8 TO 14): UNDERSTAND AI, DO NOT JUST USE IT
==========

Course link: https://learn.modernagecoders.com/courses/ai-literacy-for-kids-course
Subject area: AI Literacy for Kids
Designed for: Kids
Who can join: Ages 8 to 14; no prior coding required, everything is taught from scratch
Course length: 4 months (16 weeks), joinable any month
Time needed: 2 live classes/week + small weekly projects
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online AI literacy classes for ages 8 to 14. Children learn how AI actually works, how it learns from data, where it goes wrong, and how to judge it, with age-appropriate supervised tools.

In one line: Children who understand how AI works, where its answers come from, and when to doubt it, taught safely and at the right age.

ABOUT THE PROGRAMME

Most children already use AI daily, in recommendations, filters, voice assistants and homework help. Almost none of them understand what it is doing. That gap is the actual risk: not that children use AI, but that they trust it without knowing how it works, where its answers come from, or how confidently it can be wrong. This course closes that gap. Children

learn what artificial intelligence actually is, how a machine learns from examples rather than rules, why the data it learns from decides what it does, and why it can produce a confident answer that is simply false. They train simple models themselves, on data they collect, so machine learning stops being magic and becomes something they have done with their own hands. It is also built to be age-appropriate in a way most AI courses are not. Almost every mainstream AI tool sets a minimum age, commonly 13 and sometimes 18, in its own terms of service. We do not ask a nine-year-old to create an account on a service that does not permit it. Younger students work with tools designed for children and with teacher-run demonstrations, while older students in the range use more capable tools where age and parental permission allow. Nobody is left out, and no family is quietly pushed past a terms-of-service line.

WHAT MAKES THIS COURSE DIFFERENT

- Understanding first: children learn how AI works, not just which buttons to press
- Genuinely age-appropriate, because mainstream AI tools set minimum ages (commonly 13, sometimes 18) that most kids' AI courses quietly ignore
- Children train their own simple models on data they collect, so machine learning becomes concrete instead of magical
- Bias, hallucination and data quality taught in plain language, with examples children can see for themselves
- Aligned with the direction of school curricula, including computational thinking and AI now entering national syllabuses
- Live, small batches with a real teacher, never a recorded video or an app left to babysit

THE LEARNING PATH, STEP BY STEP

Phase 1: What AI is and is not, how machines learn from data, and training a first model by hand
Phase 2: Where AI goes wrong: bias, hallucination and data quality, plus judging AI output critically
Phase 3: Building simple AI-assisted projects and presenting them, with safety and honesty built in

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What Your Child Needs to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: WHAT AI IS, AND HOW A MACHINE LEARNS

The foundations, taught with hands-on activities rather than definitions to memorise.

Month 1: Machines, Rules and Learning (Weeks 1-4)
What Is Artificial Intelligence, Really
Topics:
- What people mean by AI, and what computers were already doing before it
- The difference between following rules and learning from examples
- Spotting the AI you already use every day
- What computers are genuinely good at, and what humans are still better at
- Why AI does not think or understand the way a person does

Projects built:
- An AI spotting hunt: find five AI systems you used this week and explain what each one decides

Practice: Sort everyday examples into rule-following or learning-from-examples, and explain each choice

Teaching a Machine by Example
Topics:
- What training data is, in plain language
- How a machine finds patterns in examples
- Labels, and why a machine needs to be told what things are
- Why more and better examples usually mean better results
- The unplugged version, done with paper before any computer

Projects built:
- Train a classmate as if they were a machine: give only examples, no rules, then test them

Practice: Build a labelled example set for a simple classification task

Training Your First Real Model
Topics:
- Using a child-safe, teacher-supervised training tool
- Collecting your own examples for two or three categories
- Training the model and testing it live

        - Watching accuracy change as the examples change
        - Why the machine gets some things confidently wrong
      Projects built:
        - Train an image or sound classifier on your own examples and demonstrate it to the class
      Practice: Improve your model by adding better examples and record what changed
      Assessment: Show and explain your trained model: what it does, how you taught it, where
      it fails
    Patterns, Data and Prediction
      Topics:
        - How patterns in data turn into predictions
        - Simple ideas of features: what the machine actually looks at
        - Why a machine can be confident and still wrong
        - Prediction versus certainty, in language a child can hold
        - Connecting this to maths patterns they already know
      Projects built:
        - Predict outcomes from a small dataset by hand, then compare with the model's prediction
      Practice: Pattern and prediction puzzles using real, small datasets

---
PHASE 2: WHERE AI GOES WRONG, AND HOW TO JUDGE IT

The critical-thinking half, which is the part that actually protects a child.

  Month 2: Bias, Hallucination and Judging Answers (Weeks 5-8)
    Where the Data Comes From
      Topics:
        - Every AI learns from data somebody collected
        - What gets left out of a dataset, and why that matters
        - Seeing bias appear in a model you trained yourself
        - Fair and unfair examples, explored with real cases suitable for the age
        - Why the machine is not being unkind, it is repeating its examples
      Projects built:
        - Deliberately train a model on lopsided data, then show the class exactly how it fails
      Practice: Audit a small dataset and list who or what is missing from it
    When AI Makes Things Up
      Topics:
        - Why a text AI can produce a confident, fluent, completely false answer
        - What it means that it predicts likely words rather than looking up facts
        - Checking an AI answer against a real source
        - Why AI is a starting point and not a final authority
        - How to tell a class or teacher when you used AI, honestly
      Projects built:
        - Fact-check a set of teacher-provided AI answers and mark which are wrong and why
      Practice: Source-checking drills: verify five claims against real references
      Assessment: A short critical-thinking test on judging AI answers
    Using AI Honestly for Schoolwork
      Topics:
        - The difference between AI helping you learn and AI doing it for you
        - Asking AI for a hint or an explanation instead of the answer
        - Explaining a solution in your own words as the real test of understanding
        - Why copying an AI answer cheats only yourself
        - Simple, honest rules a child can actually follow
      Projects built:
        - Take one homework-style problem, use AI only for hints, and explain the solution back unaided
      Practice: Rewrite AI explanations in your own words and check them for errors
    Staying Safe and Private
      Topics:
        - Why AI tools have minimum ages, and what a terms of service is
        - What personal information should never be typed into any AI tool
        - Why what you type may be stored or used
        - Talking to a parent or teacher when something feels wrong
        - Deepfakes and manipulated media, at an age-appropriate level
      Projects built:
        - Write a family AI agreement: what is allowed, what is not, and who to ask
      Practice: Sort scenarios into safe, unsafe and ask-an-adult
      Assessment: End-of-phase check on safety, privacy and honest use

---
PHASE 3: BUILDING SOMETHING REAL AND EXPLAINING IT

Children apply everything to a project of their own and learn to present it clearly.

  Month 3: Simple AI Projects (Weeks 9-12)
    Block Coding Meets AI
      Topics:
        - Connecting a trained model to a block-coded program
        - Making the program react differently to each prediction
        - Testing your project with inputs it has never seen
        - Fixing it when it behaves oddly

- Planning before building
    Projects built:
      - Build a block-coded project that responds to your trained model's predictions
    Practice: Debug three deliberately broken AI projects
  Designing a Project That Helps Someone
    Topics:
      - Choosing a problem worth solving for a real person
      - Deciding whether AI is even the right tool for it
      - Sketching the idea before any building
      - Planning what data you would need and whether you can get it fairly
      - Setting a scope you can actually finish
    Projects built:
      - Write a one-page plan for your capstone: the person, the problem, the data, the plan
    Practice: Give and receive feedback on each other's project plans
  Building the Capstone
    Topics:
      - Building in small steps and testing each one
      - Collecting balanced training data on purpose
      - Handling the cases the model gets wrong
      - Being honest in your project about its limits
      - Getting it working end to end
    Projects built:
      - Build your working capstone project
    Practice: Weekly build checkpoints with teacher review
  Presenting and Explaining
    Topics:
      - Explaining what your project does and how you taught it
      - Showing where it fails, honestly, which is the mark of real understanding
      - Answering questions about your data
      - Speaking clearly to an audience that is not technical
      - Being proud of work you can actually explain
    Projects built:
      - Present your capstone to the class and to your parents
    Practice: Rehearse the presentation and take questions
    Assessment: Final capstone presentation, assessed on understanding rather than polish

HOW A TYPICAL WEEK IS SPLIT

  Homework: A task or problem set after every class, and a monthly mixed-review assignment that
    deliberately folds in earlier topics

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall
    short and there is focused revision and a free retest: the certificate certifies real
    learning, not attendance

WHERE THIS CAN LEAD

  - A genuine head start for school computing and the AI topics entering national curricula
  - A natural next step into Python and machine learning as a teenager
  - Critical-thinking habits that apply well beyond technology
  - Comfort with data and patterns that supports maths and science
  - An informed, safe relationship with tools they will use lifelong

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: Is my child old enough for AI tools?
  A: This is exactly why the course is built the way it is. Most mainstream AI tools set a
  minimum age in their terms of service, commonly 13 and sometimes 18. We never ask a child to
  create an account on a service that does not permit their age. Younger students use tools
  designed for children and teacher-run demonstrations; older students use more capable tools
  where age and parental permission allow.

  Q: Will my child just learn to use ChatGPT?
  A: No, and that is the point. The course is about understanding: what AI is, how a machine
  learns from examples, why data quality decides the outcome, and why an AI can be confidently
  wrong. Children train their own simple models on data they collect. Using a chatbot well is a
  small part, taught honestly, and always as a starting point rather than an authority.

  Q: Does my child need to know coding first?
  A: No. Everything is taught from scratch. The early weeks are largely unplugged and hands-on,
  and where coding is used it is block-based and taught step by step. Children who already code
  will still be stretched, because the difficulty here is in the thinking, not the typing.

  Q: How is this different from a normal coding class?
  A: A coding class teaches your child to build software. This teaches your child to understand

and judge systems that learn, which is a different skill. It pairs naturally with a coding course but does not replace one.

Q: What ages are in each batch?
A: Batches are grouped by age band within the 8 to 14 range, so an eight-year-old is never placed with a fourteen-year-old. The concepts stay the same; the activities, tools and depth are matched to the band. We place your child after the free demo class. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: How is my child assessed, and is the certificate just for attending?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a friendly final assessment of everything learned. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ai-literacy-for-kids-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 12 OF 137: AI TOOLS COURSE FOR TEENS (13+): PROMPTING TO REAL PROJECTS

==========

Course link: https://learn.modernagecoders.com/courses/teens-ai-tools-mastery-course
Subject area: AI for Teens
Designed for: Teens
Who can join: Complete beginner, ages 13 to 18
Course length: 4-5 months (16-20 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

AI tools course for teens aged 13-18: live online classes on ChatGPT prompting, safe AI art, study tools, coding assistants and a real AI capstone in 4-5 months.

In one line: Every teen already uses AI. This course turns using into understanding: prompting that actually works, tools that are actually allowed at their age, and a real project at the end.

ABOUT THE PROGRAMME

AI is already in every teenager's day, in search, homework, music and feeds, usually unexamined and often against the tools' own age rules. This live online course for ages 13 to 18 replaces that drift with skill: how AI actually works, prompting as a craft, AI art and media created with tools that genuinely permit teens, AI as a study partner used ethically, coding assistants, training small machine-learning models, and a capstone project that combines them. Age honesty is a feature of this course, not a footnote: every tool taught is selected for 13+ availability, accounts are set up together with a parent where consent is required, and the course explicitly teaches which popular tools teens should NOT be using yet and why. Ethics runs throughout: hallucinations and fact-checking from week 2, academic honesty in the study weeks, bias, privacy and deepfake awareness before graduation. The pace is honest: 4 months at two live classes a week plus two to three hours of practice, with a fifth month in hand for teens who want more time. Homework follows every class, each month ends with a mixed challenge that revisits earlier skills, and the course ends with a final exam, a capstone demo plus written questions, that decides the certificate, with a free retest after revision. Real skill, real judgment, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Age-clean by design: every tool taught genuinely permits teens, accounts are set up with parents, and the course teaches which famous tools are off-limits until 18 and why
- Prompting taught as a craft: structure, context, iteration and evaluation, not magic phrases
- Ethics is load-bearing: fact-checking, academic honesty, bias, privacy and deepfake awareness are core weeks, not disclaimers
- Beyond chatting: teens train their own small ML models and use AI coding assistants, the builder's side of AI
- An honest 4-month arc with a fifth month in hand: nobody is rushed, nobody coasts
- Real assessment: weekly homework, monthly mixed challenges, and a final exam that decides the

certificate, with focused revision and a free retest

THE LEARNING PATH, STEP BY STEP

Month 1: AI foundations and prompting (Weeks 1-4): how AI works, ChatGPT done properly, and prompt engineering as a craft
Month 2: Creative AI (Weeks 5-8): AI art with teen-legal tools, image editing, and the copyright questions that come with them
Month 3: AI for school and media (Weeks 9-12): music and video tools, study assistants used ethically, research and writing
Month 4: The builder's month (Weeks 13-16): coding assistants, training your own models, ethics and safety, and the capstone exam

WHAT A STUDENT NEEDS BEFORE STARTING

Experience: None. Curiosity and a willingness to fact-check are the real prerequisites
Equipment: A computer or laptop with a stable internet connection; free tool tiers cover the whole course
Age: 13 to 18, firmly: the platforms' own age rules are part of the curriculum
Accounts: Set up together with a parent wherever consent is required
Time commitment: 2 live classes plus 2-3 hours of practice a week

WHO THIS COURSE IS FOR

Everyday users: Teens already using AI casually who should be using it skillfully and legally
Students: Class 8-12 students who want AI as an honest study advantage, not a shortcut that backfires
Creators: Teens making art, music and videos who want AI in their toolkit the right way
Future builders: Students curious about the builder side: coding assistants and training their own models
Careful parents: Families who want AI learned with age rules, ethics and supervision built in

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

MONTH 1: AI FOUNDATIONS AND PROMPT CRAFT

How AI actually works, ChatGPT used properly with a parent-approved account, and prompting practiced as a real skill.

Introduction to AI World
Topics:
- What is Artificial Intelligence? Simple explanation
- AI in everyday life: Siri, Alexa, YouTube recommendations
- How AI learns: Machine Learning basics for teens
- Types of AI: Narrow AI vs General AI
- AI history: From calculators to ChatGPT
- AI vocabulary: Model, training, dataset, algorithm
- AI myths vs reality
- How AI 'thinks' differently from humans
- AI limitations and capabilities
- Setting up safe AI accounts with parental guidance
- Creating AI learning journal
- Class rules for AI usage
Projects built:
- My First AI Conversation
- AI vs Human: Comparison chart
- Draw AI's journey timeline
- Play and analyze AI games
Practice: Spot five places AI helped you this week, like YouTube suggestions or a voice assistant, and mark which one felt the smartest.

ChatGPT Fundamentals
Topics:
- ChatGPT interface walkthrough
- Free vs paid versions understanding
- How to start conversations
- Basic prompt structure
- Being polite with AI (digital etiquette)
- Getting better responses
- When ChatGPT makes mistakes
- Fact-checking AI responses
- Using ChatGPT for homework help (ethically)
- ChatGPT for creative writing
- ChatGPT for explanations
- Saving and organizing chats
Projects built:

- Interview ChatGPT about its capabilities
- Create a story with ChatGPT
- Get homework help (with citations)
- Plan a birthday party with AI

Practice: With a parent nearby, ask ChatGPT three questions about your favorite animal and note which answer surprised you most.

Prompt Engineering Basics

Topics:
- What is a prompt?
- Anatomy of a good prompt
- Clear instructions importance
- Context and background information
- Specific vs vague prompts
- Using examples in prompts
- Role-playing prompts ('Act as a...')
- Step-by-step prompts
- Output format specification
- Temperature and creativity settings
- Iterative prompting (improving results)
- Prompt templates creation

Projects built:
- Create prompt template library
- Before/After prompt improvements
- Prompt engineering challenge
- Build custom AI assistant personality

Practice: Rewrite one vague prompt like 'draw a dog' into a detailed prompt with a place, a mood and a style, then compare the two answers.

Advanced ChatGPT Techniques

Topics:
- Multi-turn conversations
- Teaching ChatGPT your preferences
- Custom instructions setup
- Using ChatGPT for learning
- Language translation with AI
- Code explanation with ChatGPT
- Math problem solving
- ChatGPT for research projects
- Creating study guides
- Quiz and flashcard generation
- Web browsing with ChatGPT
- Custom GPTs and projects: building a tailored study helper

Projects built:
- Create personalized study assistant
- Build interactive quiz game
- Research project on favorite topic
- Multi-language story creation

Practice: Hold a five turn conversation where you teach the AI about your favorite hobby, then ask it to quiz you on it.

Assessment: Month 1 check: a live prompting challenge scored on a rubric, plus a take-home mixing everything from month 1

MONTH 2: CREATIVE AI, DONE LEGALLY

AI art and image editing with tools that genuinely permit teens, plus the copyright and ethics questions creative AI raises.

Introduction to AI Art

Topics:
- How AI creates images
- Understanding diffusion models (simple explanation)
- Text-to-image concept
- AI art history and evolution
- Famous AI artworks
- Copyright and AI art basics
- Ethical considerations in AI art
- Different styles AI can create
- Resolution and quality settings
- Image composition basics
- Color theory for AI art
- Planning AI art projects

Projects built:
- First AI artwork collection
- Illustrate a short story
- Create fantasy creatures
- Design dream bedroom with AI

Practice: Describe your dream bedroom in words, generate an image from the description

and circle two details the AI got right.

Image Generation with Teen-Legal Tools
Topics:
- Image generator interface deep dive
- Crafting detailed image prompts
- Style modifiers: cartoon, realistic, abstract
- Artist styles (ethically)
- Lighting and mood descriptors
- Camera angles and perspectives
- Combining multiple concepts
- Inpainting and editing
- Variations and iterations
- Bing Image Creator advantages
- Batch generation techniques
- Saving and organizing AI art
Projects built:
- Create comic book pages
- Design game characters
- Reimagine famous paintings
- Build visual vocabulary guide
Practice: Generate the same scene in three styles, cartoon, realistic and abstract, and write down which prompt words worked best.

Adobe Firefly, Canva AI and Advanced Art
Topics:
- Adobe Firefly: generation built on licensed training data
- Canva Magic Media inside real design projects
- Microsoft Designer and Bing Image Creator workflows
- Style, mood and composition prompts that transfer across tools
- Consistent characters and series work
- Generative fill and expand: editing with AI
- Why several famous art tools are 18+ and what that means for now
- Building a small AI art portfolio with honest tool credits
Projects built:
- A themed 6-piece AI art series with consistent style
- A school event poster designed end to end in Canva with AI assists
Practice: One artwork a day across different tools, credited honestly

AI Photo Editing and Manipulation
Topics:
- AI-powered photo editing
- Background removal with AI
- Object removal and replacement
- AI upscaling and enhancement
- Face editing and avatars
- Style transfer techniques
- Creating AI filters
- Batch processing with AI
- Restoration of old photos
- AI color correction
- Creating photo compositions
- AI-assisted graphic design
Projects built:
- Restore and colorize old photos
- Create AI avatar collection
- Design poster with AI elements
- Before/after photo gallery
Practice: With permission, remove the background from a family photo using an AI editor and place the subject somewhere fun.
Assessment: Month 2 check: an art-and-editing challenge with a copyright question, plus mixed review from month 1

MONTH 3: AI FOR MEDIA AND SCHOOL

Music, audio and video tools at teen-appropriate levels, then the serious part: AI as a study partner used ethically.

AI Music and Audio
Topics:
- How AI composes music
- Music theory basics for AI
- Genres and styles in AI music
- Melody vs harmony generation
- Rhythm and beat creation
- AI instrument selection
- Mood and emotion in music
- Creating loops and samples

    - Extending and editing AI music
    - Copyright in AI music
    - Using AI music in projects
    - Music prompt engineering
    - Text-to-speech and transcription tools for projects
    - Cleaning audio: noise removal with AI
    - Why voice cloning is age-restricted, and the ethics around it
  Projects built:
    - Create theme song for imaginary show
    - Compose study music playlist
    - Make birthday song variations
    - Video game soundtrack creation
  Practice: Generate two short tunes in different genres and tell someone at home which
  mood each one gives you.

AI Video and Animation
  Topics:
    - Text-to-video AI introduction
    - Video editing with AI assistance
    - Creating animations with AI
    - AI scene generation
    - Green screen with AI
    - Video upscaling and enhancement
    - Automatic subtitles and captions
    - Video summarization AI
    - Creating video thumbnails
    - AI video effects and filters
    - Lip-sync and dubbing AI
    - Short-form content creation
    - GIFs and short animations with teen-accessible tools
    - Animating a static image, responsibly and with consent
  Projects built:
    - Create AI news report
    - Animated explainer video
    - Music video with AI visuals
    - Educational video with AI presenter
  Practice: Storyboard a 15 second video in three drawn frames, then create one of the
  scenes with an AI video tool.

AI Study Assistants, Used Ethically
  Topics:
    - AI for different subjects
    - Math problem solvers
    - Science experiment simulators
    - History research assistants
    - Language learning with AI
    - AI essay helpers (ethical use)
    - Citation and bibliography tools
    - Concept explanation AI
    - Creating mind maps with AI
    - AI flashcard systems
    - Spaced repetition with AI
    - Exam preparation strategies
  Projects built:
    - Create personalized study plan
    - Build subject-specific AI tutor
    - Design quiz preparation system
    - Make visual study guides
  Practice: Use an AI study helper to practice five maths or spelling questions, and mark
  which ones you solved without any help.

AI Research and Writing Tools
  Topics:
    - Research methodology with AI
    - Finding reliable sources
    - AI for literature review
    - Summarizing long texts
    - Note-taking with AI
    - Outline generation
    - Paraphrasing tools
    - Grammar and style checking
    - Plagiarism detection
    - Creating bibliographies
    - Presentation creation with AI
    - Infographic generation
  Projects built:
    - Complete research project
    - Create academic presentation
    - Build knowledge database

- Design infographic report
Practice: Ask AI to summarize a short article about space or animals, then check one fact from the summary in another source.
Assessment: Month 3 check: produce a short AI-assisted study artifact with sources verified, plus mixed review reaching back to month 1

MONTH 4: THE BUILDER'S MONTH AND THE FINAL EXAM

The step from user to builder: coding assistants, training your own models, the ethics that govern it all, and the capstone exam.

AI Coding Assistants
Topics:
- Introduction to AI pair programming
- How AI understands code
- Code completion basics
- Bug detection with AI
- Code explanation tools
- Converting ideas to code
- Learning programming with AI
- Different programming languages
- Debugging with AI help
- Code optimization suggestions
- Documentation generation
- Version control basics with AI
Projects built:
- Build first website with AI help
- Create simple Python game
- Make calculator app
- Design personal portfolio
Practice: Ask an AI assistant to explain a small piece of code in plain words, then change one thing and predict what happens.

Train Your Own Models
Topics:
- Training custom AI models
- Understanding datasets
- Classification vs regression
- Computer vision projects
- Natural language processing
- Predictive models basics
- Model evaluation metrics
- Overfitting and underfitting
- Feature engineering basics
- Deploying ML models
- ML ethics and bias
- Real-world ML applications
- Sharing a model demo with Gradio, the simple way
Projects built:
- Image classifier for recycling
- Sentiment analysis tool
- Gesture recognition system
- Predictive text generator
Practice: Train a Teachable Machine model to recognize two of your toys and test it five times with each toy.

AI Safety, Ethics and the Future
Topics:
- AI bias and fairness
- Privacy in AI age
- Deepfakes and detection
- Misinformation and AI
- Digital footprint awareness
- Cybersecurity basics
- AI regulation basics
- Intellectual property and AI
- Job displacement discussions
- Human-AI collaboration
- Future of work with AI
- Responsible AI use
- AI agents that act on your behalf: promise and risk
- Capstone planning: choosing a project worth defending
Projects built:
- Create AI ethics guidebook
- Design bias detection system
- Build fact-checking tool
- Develop digital safety plan
Practice: Play detective with a parent: compare one real photo and one AI image and list

two clues that gave the AI image away.

Capstone, Demo Day and the Final Exam
Topics:
- Building the capstone: several AI tools, one real outcome
- Documentation: what was AI, what was you, said honestly
- Rehearsing the demo: showing judgment, not just output
- Presenting to the batch and parents
- A full-course mixed review before the exam
Projects built:
- The capstone: a study system, creative portfolio, small trained model or AI-assisted build, demonstrated live
Practice: Daily capstone progress with honest AI-use notes
Assessment: FINAL EXAM: capstone demo plus a written paper with questions mixed from the whole course, including tool-age judgment and ethics. Passing earns the certificate; falling short earns focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Month 1: a prompt playbook, a custom study helper GPT, and a rubric-scored prompting challenge
- Month 2: a themed AI art series with honest crediting and a designed school poster
- Month 3: an AI-assisted audio or video piece and a verified research artifact
- Month 4: a self-trained ML model demo and the final capstone, defended live

Total projects: 15+ real AI-assisted projects plus a defended capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Prompt engineering: structure, context, iteration and honest evaluation
- Creative AI: image, audio and video work with teen-legal tools and proper credits
- Study skills: AI as tutor and research partner, inside academic honesty lines
- Builder basics: coding assistants and training small ML models
- Judgment: fact-checking, bias awareness, privacy habits and tool-age literacy

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, tools driven hands-on together
Homework: A practical AI task after every class, and a mixed challenge at the end of every month
Practice: 2-3 hours of guided practice between classes
Review: Every submission reviewed with feedback; judgment errors corrected kindly and early

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Regular notes to parents, including which tools were introduced and the consent settings that matter

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam, a capstone demo plus written paper. Fall short and there is focused revision and a free retest: the certificate certifies skill, not attendance

WHERE THIS CAN LEAD

- Codex and Claude Code for Teens: from using AI to building with AI coding agents
- AI and Machine Learning Masterclass: the mathematics and models behind the tools
- Python for Teens: the language that turns AI users into AI builders
- School advantage now: ethical AI study skills that compound every term
- The judgment layer: a teen who knows what to trust, cite and question

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What age is appropriate for this AI tools course?
A: Ages 13 to 18. That floor is deliberate: the major AI platforms themselves require 13+ with parental consent, and this course treats those rules as part of the curriculum. Younger children learn AI concepts hands-on in our kids coding course instead, using classroom-safe tools like Machine Learning for Kids and Teachable Machine.

Q: Is it safe for my teenager to use AI tools?
A: With the right setup and habits, yes, and building those habits is half this course. Accounts are created together with a parent wherever consent is required, no personal information goes into prompts, outputs are fact-checked as a reflex, and the course explicitly covers hallucinations, bias, privacy and deepfake awareness. Your teen also learns which popular tools are 18+ and why waiting matters.

Q: How long does the course take?
A: Four months at 2 live classes a week plus 2 to 3 hours of practice, with a fifth month in hand for teens who want more time on the builder skills, no rushing and no penalty. The course is paced so every week produces something real: a portfolio piece, a study system, a trained model, and it ends with a capstone exam.

Q: How is my teen assessed? Is the certificate just for attending?
A: No. Homework follows every class, each month ends with a mixed challenge that revisits earlier tools and adds ethics questions, and the course ends with a final exam: a capstone demo plus a written paper covering the whole syllabus, including tool-age judgment. Passing earns the certificate; falling short earns focused revision and a free retest. Real skill and real judgment, not attendance.

Q: Will my teenager learn actual skills or just play with AI toys?
A: Actual skills, in layers: prompt engineering practiced against rubrics, ethical research and study workflows, AI-assisted creative production with honest crediting, coding assistants, and training small machine-learning models of their own. The capstone requires combining tools into one defended project. Toys do not survive a viva.

Q: Which AI tools does the course use, and are they allowed for teens?
A: Only tools that genuinely permit ages 13+ (with parental consent where required): ChatGPT, Microsoft Copilot, Perplexity, Adobe Firefly, Canva AI, Bing Image Creator, GitHub Copilot, Teachable Machine and similar. Some well-known tools, several art generators and voice-cloning services among them, are 18+ by their own terms, and the course teaches exactly that instead of quietly ignoring it. Our public AI tools age guide lists every tool's floor.

Q: Will this help with school, or make cheating easier?
A: It is built to help honestly. The study weeks teach AI as a tutor and research partner: explaining concepts, generating practice questions, checking reasoning, always with verification and always inside school policy. Academic honesty is taught explicitly, including what crosses the line and why detection is the least of the reasons not to.

Q: What equipment does my teenager need?
A: A computer or laptop with a stable internet connection. The course runs on free tiers of every tool; where an account needs parental consent, it is set up together in the first weeks. No paid subscriptions are required for the syllabus.

Q: Is there a free trial class?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to sit in, and the age-and-safety setup is explained right there. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small interactive groups with a real instructor, never pre-recorded videos. AI tools change monthly, and live teaching is the only format that keeps up honestly.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/teens-ai-tools-mastery-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 13 OF 137: AI TOOLS MASTERY COURSE: CHATGPT, CLAUDE & AUTOMATION

==========

Course link: https://learn.modernagecoders.com/courses/ai-tools-mastery-complete-college
Subject area: AI & Automation
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 5-6 months (22-26 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

AI tools course for adults and professionals: prompt engineering with ChatGPT and Claude, image and video AI, AI coding tools, then real automation and AI agents, in 5-6 months of live classes.

In one line: Fluency in the tools that now run knowledge work: prompting done properly, image and video AI, AI coding assistants, and real automation with agents.

ABOUT THE PROGRAMME

The AI tool landscape changed what a single professional can do in a day, but most people use a fraction of it and use that fraction badly. This live online course builds real fluency for adults and professionals: prompt engineering with ChatGPT and Claude that goes far past one-line questions, then the wider toolkit, image generation with Midjourney and Stable Diffusion, video and audio AI, AI research assistants and AI coding tools, then the payoff layer that separates power users from dabblers: workflow automation, connecting tools through their APIs, and building AI agents that carry out multi-step work. Because the tools change monthly, the course teaches durable judgment (how to evaluate, adopt and drop tools) alongside current specifics, and treats ethics, verification and responsible use as core skills, not a footnote. A finale month builds a real automation and sits the final exam. The pace is honest: about four months of teaching at two live classes a week plus practice, with margin in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- Prompting taught to real depth, the skill most people fake
- The full modern toolkit: text, image, video, research and coding AI
- The power-user layer: automation, APIs and AI agents that do multi-step work
- Durable judgment for a fast-moving field: how to evaluate and adopt tools, not just the current list
- Verification and ethics as core skills: AI that helps, checked before it ships
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Weeks 1-6): prompt engineering with ChatGPT and Claude
Phase 2: The toolkit (Weeks 7-12): image, video, research and coding AI
Phase 3: Automation (Weeks 13-18): workflows, APIs and AI agents, used responsibly
Phase 4: The finale (Month 5): a real automation build and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Everything is taught from the first line
Equipment: Any computer with a stable internet connection; every tool used is free
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

Professionals: Working professionals who want real leverage from AI, not surface tricks
College students: Students who want to enter the workforce genuinely AI-fluent
Creators and marketers: Creators and marketers who need text, image and video AI at depth
Founders: Founders and freelancers automating the work of a larger team
Career switchers: Newcomers who want a fast, current, practical AI skill

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: AI FOUNDATIONS AND PROMPT ENGINEERING (WEEKS 1-6)

Master ChatGPT, prompt engineering, AI writing tools, and conversational AI applications.

ChatGPT & Prompt Engineering Excellence (Weeks 1-6)
Introduction to AI Tools & ChatGPT Basics
Topics:
- AI revolution: understanding generative AI
- Overview of AI tools landscape (2026)
- What is ChatGPT? How its models differ, and how to pick one
- ChatGPT account setup and interface
- ChatGPT Plus vs Free tier comparison
- Basic prompting: asking questions effectively
- Understanding AI responses and limitations
- Chat history management and organization
- Mobile app usage and tips
- ChatGPT plugins introduction
- Custom instructions setup
- Conversation starters and templates
- Ethics and responsible AI tool usage
- AI detection and originality

- Data privacy and security with AI tools
  Projects built:
    - Personal AI assistant setup
    - Custom instruction templates
    - Daily productivity workflow with ChatGPT
    - Meeting notes summarizer
    - Email response generator
  Practice: Use ChatGPT daily for 20 different tasks
Advanced Prompt Engineering
  Topics:
    - What is prompt engineering?
    - Anatomy of effective prompts
    - Zero-shot, one-shot, few-shot prompting
    - Chain-of-thought prompting
    - Role-based prompting techniques
    - Context setting and framing
    - Prompt templates and frameworks
    - RISEN framework: Role, Instructions, Steps, End goal, Narrowing
    - RTF framework: Role, Task, Format
    - Iterative prompt refinement
    - Handling AI hallucinations
    - Temperature and parameter understanding
    - Multi-turn conversations strategies
    - Prompt libraries and repositories
    - A/B testing prompts for best results
  Projects built:
    - Prompt template library (50+ templates)
    - Industry-specific prompt collections
    - Complex task automation prompts
    - Creative writing prompt system
    - Business analysis prompts
  Practice: Create and test 100+ prompts across categories
ChatGPT for Content Creation & Writing
  Topics:
    - Content ideation with ChatGPT
    - Blog post writing workflow
    - Article outlines and structure
    - SEO-optimized content creation
    - Social media content generation
    - Email marketing campaigns
    - Copywriting: sales pages, ads, landing pages
    - Storytelling and creative writing
    - Technical writing and documentation
    - Resume and cover letter creation
    - Editing and proofreading with AI
    - Tone and style adjustments
    - Multi-language content creation
    - Content repurposing strategies
    - Plagiarism checking and originality
  Projects built:
    - Complete blog writing system
    - Social media content calendar (30 days)
    - Email sequence (welcome, nurture, sales)
    - E-book outline and chapters
    - Product descriptions generator
    - LinkedIn posts automation
    - YouTube script templates
  Practice: Create 50+ pieces of content in different formats
ChatGPT for Business & Productivity
  Topics:
    - Business strategy and planning
    - Market research with ChatGPT
    - Competitive analysis workflows
    - SWOT analysis generation
    - Business plan creation
    - Financial projections and modeling prompts
    - Data analysis and insights
    - Code generation basics (no coding knowledge needed)
    - Excel formulas and spreadsheet help
    - Presentation content and structure
    - Meeting preparation and agendas
    - Decision-making frameworks
    - Problem-solving with AI
    - Learning and education use cases
    - Personal development and coaching
  Projects built:
    - Business plan generator
    - Market research report automation
    - Competitor analysis system

- Excel/Sheets automation prompts
- Presentation creation workflow
- Personal learning curriculum
- Daily productivity dashboard

Practice: Apply ChatGPT to 30 business scenarios

Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 2: VISUAL, CREATIVE AND DEVELOPER AI TOOLS (WEEKS 7-12)

Master Claude, Gemini, Midjourney, Stable Diffusion, and all major AI tools for images, video, and audio.

Visual AI & Alternative LLMs (Weeks 7-10)

Claude AI & Google Gemini

Topics:

- Claude AI: features and strengths
- Claude vs ChatGPT comparison
- Claude's long context window usage
- Document analysis with Claude
- Claude for coding and technical tasks
- Claude Projects feature
- Google Gemini introduction
- Gemini Advanced features
- Gemini vs GPT-4 comparison
- Multi-modal capabilities of Gemini
- Google Workspace integration
- How Gemini replaced Bard, and what changed
- Choosing the right AI for the task
- Using multiple AIs together
- AI tool comparison matrix

Projects built:

- Document summarization system (Claude)
- Long-form content analysis
- Code review automation
- Research paper summarizer
- Multi-AI workflow (ChatGPT + Claude + Gemini)
- AI tool selector guide

Practice: Complete 20 tasks with each AI, compare results

Midjourney and Image Generation

Topics:

- Introduction to AI image generation
- Midjourney setup: Discord integration
- Midjourney versions: V5, V6, Niji
- Basic prompting for images
- Understanding parameters: aspect ratio, quality, stylize
- Image weights and multi-prompts
- Style references and character consistency
- Remix mode and variations
- Upscaling and image enhancement
- Negative prompts for better control
- Seed for reproducibility
- Chaos and weirdness parameters
- Photography styles and techniques
- Artistic styles replication
- Brand and marketing visuals

Projects built:

- Product photography collection
- Brand visual identity
- Social media graphics library
- Character design series
- Marketing campaign visuals
- Book cover designs
- Website hero images

Practice: Generate 200+ images, build prompt library

Stable Diffusion & Other Image AI Tools

Topics:

- Stable Diffusion introduction
- Local installation vs cloud platforms
- Automatic1111 WebUI
- Stable Diffusion prompting differences
- LoRA models and fine-tuning
- ControlNet for precise control
- Inpainting and outpainting
- Adobe Firefly for commercial use
- Leonardo.ai for game assets
- Playground AI features
- Image upscaling tools: Topaz, Magnific

- Background removal: Remove.bg, Clipdrop
- AI photo editing: Photoshop AI, Luminar
- Comparing all image AI tools

Projects built:

- Stable Diffusion custom model
- ControlNet workflow setup
- Image editing pipeline
- Multi-tool image creation workflow
- Background replacement system
- Product photography enhancement
- AI image editing service

Practice: Master 5+ image AI tools, create comparison guide

Video & Audio AI Tools

Topics:

- AI video generation: Runway ML, Pika
- Text-to-video creation
- Image-to-video animation
- Video editing with AI: Descript, Opus Clip
- AI avatars: Synthesia, HeyGen
- Video enhancement and upscaling
- AI voice generation: ElevenLabs, Murf
- Voice cloning ethics and usage
- Text-to-speech for videos
- Music generation: Suno, Udio
- Audio enhancement tools
- Podcast editing with AI
- Subtitle generation and translation
- Video summarization tools
- Social media video optimization

Projects built:

- AI-generated video ads
- Explainer video with AI avatar
- Podcast intro/outro creation
- Social media video series
- Product demo video
- Multi-language video content
- Complete video production workflow

Practice: Create 15 videos using various AI tools

Research, Coding and Creative AI Tools (Weeks 11-12)

AI Research & Knowledge Tools

Topics:

- Perplexity AI for research
- Research paper summaries: Semantic Scholar, Consensus
- Literature review automation
- You.com for AI-powered search
- Bing AI (Copilot) features
- AI fact-checking tools
- Citation and reference management
- Knowledge base creation with AI
- Note-taking with AI: Notion AI, Mem
- Obsidian with AI plugins
- Roam Research AI integration
- Personal knowledge management
- Learning and study tools
- AI tutors and educational platforms
- Research workflow optimization

Projects built:

- Research automation system
- Literature review generator
- AI-powered knowledge base
- Personal learning system
- Fact-checking workflow
- Study guide generator
- Research paper writing assistant

Practice: Build comprehensive research toolkit

AI Coding & Development Tools

Topics:

- GitHub Copilot for coding
- Cursor AI editor
- Replit AI for full-stack development
- V0.dev by Vercel for UI generation
- ChatGPT for code generation
- Code debugging with AI
- Code review automation
- Documentation generation
- No-code platforms: Bubble, Webflow
- AI-powered app builders: FlutterFlow
- Database schema generation

```
        - API creation with AI
        - Testing and QA with AI
        - Code refactoring tools
        - Building apps without coding knowledge
      Projects built:
        - Landing page with V0.dev
        - Full web app with Replit AI
        - Mobile app with FlutterFlow
        - Chrome extension with AI
        - Automation script generation
        - API wrapper creation
        - Complete SaaS MVP without coding
      Practice: Build 10 applications using AI coding tools
      Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

---------------------------------------------------------------
PHASE 3: AI AUTOMATION AND PROFESSIONAL APPLICATIONS (WEEKS 13-18)
---------------------------------------------------------------
Turn tool fluency into leverage: workflow automation, connecting tools through their APIs,
and building AI agents for multi-step work, used responsibly with verification built in.

  Automation, Agents and Business Applications (Weeks 13-16)
    AI Workflow Automation
      Topics:
        - Introduction to AI automation
        - Zapier with AI: ChatGPT integration
        - Make (Integromat) for complex workflows
        - n8n for advanced automation
        - Connecting multiple AI tools
        - Trigger-action automation
        - Multi-step workflows
        - Conditional logic in automation
        - Error handling and fallbacks
        - Data transformation between tools
        - API connections basics
        - Webhook usage for real-time automation
        - Scheduled automation
        - Database integration (Airtable, Google Sheets)
        - Testing and debugging workflows
      Projects built:
        - Content publishing automation
        - Lead generation and nurturing workflow
        - Social media automation system
        - Email response automation
        - Data extraction and analysis pipeline
        - Customer onboarding automation
        - Report generation workflow
      Practice: Build 20 automation workflows
    AI Agents & Advanced Automation
      Topics:
        - What are AI agents?
        - AutoGPT and autonomous agents
        - BabyAGI for task automation
        - LangChain for AI applications
        - Agent frameworks overview
        - Custom GPTs creation
        - GPT Actions and integrations
        - Assistants API usage
        - Function calling in AI
        - Agent memory and context
        - Multi-agent systems
        - Agent orchestration
        - Agentic workflows design
        - Limitations and considerations
        - Future of AI agents
      Projects built:
        - Custom GPT for specific task
        - Personal AI assistant (multi-functional)
        - Research agent system
        - Customer service agent
        - Data analysis agent
        - Content creation agent
        - Business intelligence agent
      Practice: Create 10 specialized AI agents
    AI Integration & API Usage
      Topics:
        - Understanding APIs basics
        - Anthropic Claude API
```

- Google Gemini API
- Stability AI API
- API key management
- Rate limits and cost optimization
- Building apps with AI APIs (no-code tools)
- Glide for AI-powered apps
- Softr for AI integrations
- API connectors in various platforms
- Custom chatbots with APIs
- Embedding AI in websites
- WhatsApp/Telegram bot creation
- AI-powered Chrome extensions

Projects built:

- Custom chatbot on website
- WhatsApp AI assistant
- Telegram bot with AI
- Chrome extension with ChatGPT
- Mobile app with AI features (Glide)
- Internal tool with AI (Softr)
- AI-powered form builder

Practice: Build 8 AI-integrated applications

AI Tools for Business and Professional Work

Topics:

- Freelancing with AI tools
- AI services to offer clients
- Pricing AI-powered services
- Building AI tool SaaS products
- Affiliate marketing with AI content
- AI-generated digital products
- Selling prompts and templates
- AI consulting services
- Agency model with AI
- White-label AI solutions
- Course creation with AI
- AI automation services
- Building and selling AI agents
- Subscription-based AI services

Projects built:

- Freelance service package design
- AI SaaS product MVP
- Prompt marketplace listings
- Digital product creation
- Agency service offerings
- Online course with AI help

Practice: Launch 3 revenue streams with AI tools

Staying Current and Responsible Use (Weeks 17-18)

Staying Current with AI Tools

Topics:

- AI tool discovery strategies
- Following AI news and updates
- There's An AI For That (directory)
- AI tool review and evaluation
- Beta testing new AI tools
- Community engagement: Twitter, Reddit, Discord
- AI newsletters and resources
- Testing frameworks for new tools
- Building personal AI tool stack
- Documentation and knowledge management
- Sharing findings and building authority
- Predicting AI tool trends
- Adapting to rapid changes
- Continuous learning strategies
- Building learning systems

Projects built:

- AI tool evaluation framework
- Personal AI tools dashboard
- Weekly AI tools newsletter
- AI tools comparison database
- Trend analysis report
- Learning and update system

Practice: Review 50+ new AI tools, create evaluation system

Ethics, Safety & Responsible AI Use

Topics:

- AI ethics principles
- Bias in AI tools
- Transparency and disclosure
- AI content labeling
- Misinformation and deepfakes

```
        - Copyright and AI-generated content
        - Privacy considerations
        - Data security with AI tools
        - Terms of service understanding
        - Commercial use rights
        - AI watermarking and detection
        - Responsible automation
        - Job displacement considerations
        - Environmental impact of AI
        - Building ethical AI practices
      Projects built:
        - Ethical AI usage policy
        - Client education materials
        - Disclosure templates
        - Rights and licensing guide
        - AI ethics checklist
        - Best practices document
      Practice: Develop comprehensive ethical framework
      Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

  ---------------------------------------------------------------
  PHASE 4: THE FINALE (MONTH 5, WEEKS 19-23)
  ---------------------------------------------------------------
  The capstone month: spec, build, ship and defend, then the final exam that decides the
  certificate.

    Month 5: Capstone and the Final Exam (Weeks 19-23)
      Capstone Spec and Setup
        Topics:
          - Choosing the capstone: a real end-to-end AI automation: a content pipeline, a research
            assistant or a multi-step agent workflow for actual work
          - A one-page spec: scope, milestones, stack
          - Repository and skeleton ready on day one
          - Instructor sign-off before the build begins
        Projects built:
          - Approved capstone spec plus the running skeleton
        Practice: Pitch the spec to the batch and absorb one hard question
      Capstone Build
        Topics:
          - Two focused weeks in vertical slices
          - Instructor checkpoints and honest scope cuts
          - Using the whole course: prompting depth, the right tools for each step, and automation wiring
            through APIs or agents
          - Daily commits and an always-working build
        Projects built:
          - The capstone, feature-complete with version control
        Practice: A working build at the end of every session
      Ship and Polish
        Topics:
          - The finishing pass: edge cases, errors, documentation
          - Working and verified: an automation that runs, with human checks where they matter
          - A README worth reading and a demo worth watching
          - A full-course spiral review before the exam
        Projects built:
          - The capstone shipped, documented and rehearsed
        Practice: Two mock demos, each tighter than the last
      Demo Day and the Final Exam
        Topics:
          - Presenting the capstone: live demo plus the hardest bug story
          - The honest map of next roads: deeper automation with code, AI-assisted development, or
            applied AI in your own field
          - Where our other courses continue the journey
        Projects built:
          - Demo day presentation delivered to the batch
        Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed
        from every phase. Passing earns the certificate; falling short earns a focused revision
        plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Phase 1: a prompt library and real ChatGPT and Claude workflows
  - Phase 2: image, video and coding-AI deliverables
  - Phase 3: automations and a working AI agent
  - Phase 4: the end-to-end automation capstone

  Total projects: 15+ real deliverables, crowned by an end-to-end automation build

SKILLS THE STUDENT WILL HAVE AT THE END
```

- Prompt engineering with ChatGPT and Claude, to real depth
- Visual and media AI: Midjourney, Stable Diffusion, video and audio tools
- AI research and coding assistants, used well
- Automation: workflows, API connections and AI agents
- Judgment and ethics: evaluating tools, verifying output, using AI responsibly

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Applied AI in your own field: the toolkit plus your domain
- Our Python automation course to build automations in code
- AI-assisted development: our agentic and coding-tool courses
- Deeper model work: our AI/ML course for what is under the tools

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this AI tools course cover?
A: Prompt engineering with ChatGPT and Claude first, then the wider toolkit, image generation with Midjourney and Stable Diffusion, video and audio AI, AI research assistants and AI coding tools, then the automation layer, workflow automation, connecting tools through APIs, and building AI agents for multi-step work, with tool-evaluation judgment, verification and responsible-use ethics throughout. A finale month builds a real automation and sits the final exam.

Q: How long does this AI tools course take?
A: About four months of teaching at 2 live classes a week plus practice, with margin in hand for a 5 to 6 month total. Three phases cover prompting, the wider toolkit, and automation with agents; the finale month builds a real automation and sits the final exam.

Q: How am I assessed? Is the certificate just for attending?
A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: The tools change every month, so will this course be out of date?
A: The specific tools will keep changing; the skills will not. Prompting well, evaluating a new tool quickly, wiring automations and verifying AI output are durable, and this course teaches those alongside current specifics. You leave able to pick up the next new tool in an afternoon, which is the real skill.

Q: Do I need to be technical to take this course?
A: No. Most of the course needs no coding: prompting, image and video AI, research tools and no-code automation are all accessible. The automation phase touches simple API use, taught gently, and you can go as deep or as light there as your goals need. It is built for professionals of every background, not just developers.

Q: Is this course suitable for teenagers?
A: This is our adult (18+) AI tools course, because several tools it uses, including the general-purpose image and chat models, set their own terms and age limits at 18. Younger learners have a dedicated teens AI tools course built entirely around teen-appropriate, age-compliant tools, which is the honest place for them to start.

Q: Can I try a class before enrolling?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at

  https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor reading your actual code, never
  pre-recorded videos.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ai-tools-mastery-complete-college
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 14 OF 137: ALGEBRA FOUNDATIONS: PRE-ALGEBRA & ALGEBRA 1, LIVE ONLINE
==========

Course link: https://learn.modernagecoders.com/courses/algebra-foundations-masterclass
Subject area: School Mathematics
Designed for: Middle school
Who can join: Grades 7-9, or any student whose algebra base needs a real rebuild
Course length: 12 months (52 weeks)
Time needed: 2 live one-hour classes per week + short practice sets
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one
  live class a week, four a month; one to one outside India two live classes a week, eight a
  month. Each class is one hour.

WHAT THIS COURSE IS

  Live online Pre-Algebra and Algebra 1: repair fractions and negatives, then master linear
  equations, functions and quadratics. 8 one-hour classes a month.

  In one line: Algebra is a language. We teach your child to speak it, not memorize it.

ABOUT THE PROGRAMME

  A 12-month live program covering Pre-Algebra and the whole of Algebra 1, the single most
  consequential stretch in school mathematics. Every later course (Geometry, Algebra 2, SAT,
  calculus) silently assumes this material, and most struggling high-schoolers trace their
  trouble to exactly here. We teach algebra as a language with meaning: negatives and fractions
  rebuilt first, letters introduced as numbers-in-waiting, equations read as sentences, and
  functions met as machines. Students solve and explain aloud in every class; procedures are
  drilled only after the idea underneath holds.

WHAT MAKES THIS COURSE DIFFERENT

  - The arithmetic base, fractions, negatives, order of operations, is repaired first, because
    algebra built on sand always collapses in year two
  - Letters taught as meaning, not ritual: students can say what x is doing in every line they
    write
  - Word problems as the main course, not the scary extras at the chapter's end
  - Equation-table-graph fluency: every linear situation seen three ways
  - The explain-back method: what survives explaining is what is actually learned
  - One dedicated mentor who tracks exactly where understanding thins
  - A genuine on-ramp to Algebra 2, SAT prep and beyond, this course is the runway
  - One full interactive hour per class, twice a week

THE LEARNING PATH, STEP BY STEP

  Phase 1: Pre-Algebra: the number system mastered, integers, fractions, decimals, ratios and
  the first variables
  Phase 2: The heart of Algebra 1: equations, inequalities, linear functions and systems, seen
  as equation, table and graph
  Phase 3: Exponents, polynomials and quadratics, plus the word-problem and graph craft that
  makes it all usable

WHAT A STUDENT NEEDS BEFORE STARTING

  Education: Typically grades 7-9; older students rebuilding a weak base are equally welcome
  Math background: Basic whole-number arithmetic, everything else is taught or repaired inside
    the course
  Equipment: Computer with stable internet, notebook and pencil
  Time commitment: 2 live hours plus 1-2 practice hours weekly
  English: Course taught in English
  Motivation: Willingness to explain thinking aloud, the method depends on it

WHO THIS COURSE IS FOR

First timers: Grade 7-9 students meeting algebra for the first time who deserve to meet it properly
Strugglers: Students drowning in school algebra whose real gap is usually fractions or negatives
Rebuilders: Older students whose Algebra 2, SAT or IGCSE trouble traces back to this exact material
Homeschoolers: Families who want a structured, live-taught algebra year with a real teacher
Advancers: Strong grade 6-7 students ready to start the algebra road early

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PRE-ALGEBRA, THE NUMBER SYSTEM MASTERED (WEEKS 1-16)

Almost every algebra struggle is an arithmetic gap wearing a disguise. This phase rebuilds integers, fractions, decimals and ratios to genuine fluency, then introduces variables so gently they feel obvious.

Weeks 1-4: Diagnostic + Integers and the Number Line
Topics:
- Full diagnostic: where the number base actually stands
- The number line as home: integers, opposites and absolute value
- Adding and subtracting negatives with meaning (temperature, money, elevation)
- Multiplying and dividing negatives, why the rules are what they are
- Order of operations as grammar, not a mnemonic chant
- Mental arithmetic habits started: every class opens with fluency work

Projects built:
- Personal gap map for the year ahead
- Integer war-stories set: temperature, bank balance and elevation problems

Practice: Do a short daily integer practice set this week, using the number line whenever unsure, and write one real money or temperature story for a negative number calculation.

Weeks 5-10: Fractions and Decimals, Properly This Time
Topics:
- Fractions as numbers with places on the line, not pizza slices
- Equivalence and comparison through reasoning, not rules
- All four operations with fractions, each from a picture first
- Decimals, place value and fraction-decimal-percent conversion fluency
- Estimation as a checking habit: does this answer make sense?
- Exponents introduced as repeated multiplication with meaning

Projects built:
- Fraction operation gallery: one worked picture per operation, made by the student
- Real-recipe scaling project: fractions doing real work

Practice: Complete a mixed fractions and decimals problem set, drawing a quick picture for any operation you cannot yet explain in words.

Weeks 11-16: Ratios, Percents and the First Variables
Topics:
- Ratio and rate reasoning: the choose-the-easy-comparison skill
- Proportions solved by sense first, cross-multiplication second
- Percent problems in every costume: change, discount, tax, reverse
- Variables introduced as numbers-in-waiting; expressions as phrases
- Evaluating and simplifying expressions; like terms with meaning
- The distributive property as area, a picture that never lies

Projects built:
- Best-buy investigation: unit-price reasoning on real products
- Phase mastery check: solve, explain and extend, per topic

Practice: Solve ten percent problems taken from real life, discounts, tax and score changes, estimating each answer before computing to check your number sense.

PHASE 2: EQUATIONS, FUNCTIONS AND SYSTEMS (WEEKS 17-36)

The heart of Algebra 1. Equations are read as sentences and solved with balance reasoning; linear functions are met as equation, table and graph telling one story; systems make two stories intersect. This is the phase that decides the next five years of math.

Weeks 17-22: Equations and Inequalities as Sentences
Topics:
- One-step and two-step equations from balance reasoning
- Multi-step equations: variables on both sides, distribution, fractions
- Translating words into equations, the skill that unlocks word problems
- Literal equations: isolating any variable in a formula
- Inequalities: what flips the sign and why

- Checking solutions as a habit, not an afterthought
Projects built:
- Equation-sentence dictionary: twenty phrasings, twenty equations
- Real-world setups: phone plans, savings goals, break-even problems
Practice: Work a set of multi step equations showing the balance move at every line, then translate three word problems into equations without solving them.

Weeks 23-29: Linear Functions Three Ways
Topics:
- Coordinate plane fluency and graph-reading discipline
- Slope as rate of change, from stories before formulas
- Slope-intercept, point-slope and standard form, translated freely
- Graphing lines by meaning; intercepts as starting values
- Function notation and the function-as-machine idea
- Linear modeling: real data fitted, interpreted and questioned
Projects built:
- Equation-table-graph triptych for three real situations
- Data modeling mini-project: fit a line, defend the interpretation
Practice: Graph the same line from slope intercept, point slope and standard form to confirm they agree, and describe one real situation the line could model.

Weeks 30-36: Systems of Equations and Inequalities
Topics:
- Systems by graphing: the intersection as the answer to a question
- Substitution and elimination, chosen by structure not habit
- How many solutions? Parallel, crossing and identical lines as ideas
- Systems word problems: mixtures, comparisons, break-even
- Linear inequalities and systems of them, with shaded-region reasoning
- Phase checkpoint: mixed problem set at grade-level exam standard
Projects built:
- Business break-even case solved all three ways
- Phase mastery check: solve, explain and extend, per topic
Practice: Solve six systems choosing the method by structure, at least two each by graphing, substitution and elimination, and check every answer in both equations.

PHASE 3: EXPONENTS, POLYNOMIALS AND QUADRATICS (WEEKS 37-52)

The bridge into Algebra 2: exponent rules from meaning, polynomial arithmetic without fear, factoring as detective work and quadratics met as functions with real stories. The course closes with the word-problem and graph craft that turns knowledge into marks.

Weeks 37-42: Exponents and Polynomials
Topics:
- Exponent rules derived, not memorized, including zero and negatives
- Scientific notation with real scale (atoms to galaxies)
- Polynomials as expressions with structure: naming, adding, subtracting
- Multiplying polynomials: the area model that makes FOIL obvious
- Special products recognized as patterns: squares and difference of squares
- Fluency drills at pace, now that meaning holds
Projects built:
- Powers-of-ten scale tour: notation doing real scientific work
- Area-model gallery for polynomial multiplication
Practice: Complete an exponent rules drill, deriving any rule you forget instead of guessing, plus five polynomial multiplications using the area model.

Weeks 43-48: Factoring and Quadratic Equations
Topics:
- Factoring as un-multiplying: GCF, trinomials, special patterns
- The zero-product property, why factoring solves equations
- Square roots and solving by square-rooting
- The quadratic formula, derived once so it is never mysterious
- Choosing a method by structure: factor, root, or formula
- The discriminant previewed: how many solutions and why
Projects built:
- Factoring pattern-book, built by the student
- Quadratic equation gauntlet: mixed methods under light timing
Practice: Factor a 15 expression mixed set, then solve five quadratic equations choosing factoring, square roots or the formula and noting why each time.

Weeks 49-52: Quadratic Functions + the Capstone
Topics:
- Quadratic functions and parabola anatomy: vertex, axis, intercepts
- Projectile and area stories: the vertex as the answer to a real question
- Graphing quadratics from every form
- Course capstone review: the year's ideas connected into one map
- Algebra 2 preview: what comes next and why this base carries it
- Final mastery check across all three phases
Projects built:

- Capstone: one real quadratic situation modeled, graphed and presented
- Personal algebra handbook, the year's ideas in the student's own words

Practice: Graph two parabolas from different forms, labeling vertex, axis and intercepts, and start your capstone map connecting the year's big ideas.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Personal gap map and year-long error log
- Fraction operation gallery and factoring pattern-book, made by the student
- Equation-table-graph triptychs for real situations
- Business break-even case solved three ways
- Capstone quadratic modeling project + personal algebra handbook

Total projects: 10+ student-made artifacts across the year

SKILLS THE STUDENT WILL HAVE AT THE END

- Integer, fraction and decimal fluency
- Ratio, proportion and percent reasoning
- Equation solving with balance reasoning
- Linear functions as equation, table and graph
- Systems of equations and inequalities
- Exponent rules and polynomial arithmetic
- Factoring and quadratic equations
- Word-problem translation craft

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes with your mentor
Practice: Short, purposeful sets, never busywork stacks
Fluency: Mental-math warm-up in every class
Total per week: About 3-4 focused hours
Homework: A problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week
Mentorship: One dedicated mentor across the whole year
Recordings: Every class recorded for revision
Progress tracking: Phase mastery checks shared with parents
Flexibility: Slots move around school exams and family plans

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies mastery, not attendance

WHERE THIS CAN LEAD

- Ready for Algebra 2, Geometry and the high-school mathematics track
- The base that SAT and ACT math prep assumes
- Eligible for competition mathematics and honors placements
- Continue with the High School Mathematics Mastery Program with the same mentor

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Who is this course for?
A: Three groups: students in grades 7-9 meeting algebra for the first time and wanting it taught properly; older students whose Algebra 2 or SAT struggles trace back to a weak Algebra 1 base; and homeschool families who want a structured, live-taught algebra year. The diagnostic in the free demo places every student accurately. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: My child is already failing algebra at school. Is this remedial or advanced?
A: It is neither and both, it is understanding-first. We start where your child actually is (the diagnostic finds it), repair the arithmetic underneath if needed, and build to full Algebra 1 ownership. Struggling students stop sinking; comfortable students go deeper than school does.

Q: Why does the course spend weeks on fractions and negatives?
A: Because they are the gap behind most algebra failures. A student who cannot operate on fractions confidently will lose points in every algebra topic forever. We fix the foundation first, it is slower in week one and dramatically faster by month six.

Q: How is this different from school algebra?
A: Class sizes and method. School must march thirty students through a syllabus; we teach 1-on-1 or in tight small groups, with your child solving and explaining aloud for a full hour. Every idea is built from meaning, balance reasoning, area models, function machines, so it survives past the test.

Q: Will this align with my child's school curriculum?
A: Yes. The course covers the standard Pre-Algebra and Algebra 1 territory (US Common Core, and the equivalent stretch in Indian, British and IB middle-school systems), and the mentor sequences topics to reinforce what school is doing rather than clash with it.

Q: How long until we see grade improvement?
A: Homework and test comfort usually move within 4-6 weeks; deep, durable improvement, the kind that persists into Algebra 2, builds over the year. Monthly billing means you watch evidence and re-decide every four weeks.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A problem set follows every class with written feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final exam covering the whole course. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real understanding, not attendance.

Q: Can we try it before paying?
A: Yes. Every student starts with a free live demo class that doubles as the diagnostic, a real lesson plus an honest read on where the gaps actually are, with no card details required.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/algebra-foundations-masterclass
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 15 OF 137: ALGORITHMIC TRADING COURSE: STRATEGY, QUANT & ML
==========

Course link: https://learn.modernagecoders.com/courses/algorithmic-trading-masterclass-complete
Subject area: Data Science
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Algorithmic trading course for adults and professionals: Python for markets, strategy design and backtesting, risk management, then machine learning for trading, taught as engineering, not get-rich promises, in 10-12 months.

In one line: The quant skill set, taught as engineering: Python for markets, strategies you can backtest honestly, risk management first, then machine learning, no profit promises, real capability.

ABOUT THE PROGRAMME

Algorithmic trading is a serious quantitative discipline, and this course treats it as one, not as a get-rich pitch. It builds the strategy-and-research half of the field in order: Python for financial data, technical and statistical analysis, then strategy design and honest backtesting (including the traps, lookahead bias, overfitting, survivorship, that make bad strategies look brilliant), then risk management and position sizing taught before anything goes near live markets, and finally machine learning for trading, used with the scepticism the domain demands. Throughout, the message is consistent and repeated: markets carry real risk of loss, backtests do not predict the future, and this is education, not financial advice. A finale month builds a fully backtested strategy system and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- Taught as engineering and research, not as a money-making scheme
- Backtesting honesty is a core theme: the biases that fake good results are named and defeated
- Risk management before live markets, not as an afterthought
- Machine learning applied with domain scepticism, not hype
- Honest about risk on every page: education, not financial advice, no profit claims
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Months 1-3): markets, Python for finance, technical and statistical analysis
Phase 2: Strategy and backtesting (Months 4-6): strategy design, honest backtesting, risk management
Phase 3: Machine learning for trading (Months 7-9): ML models, sentiment, alternative data, execution
Phase 4: The finale (Month 10): a fully backtested strategy system and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Everything is taught from the first line
Equipment: Any computer with a stable internet connection; every tool used is free
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

Developers: Programmers who want the quantitative-finance skill set
College students: Students targeting quant analyst and quant developer roles
Finance professionals: Finance people who want to add real coding and research skill
Data minded: Data and stats enthusiasts drawn to markets as a hard problem
Disciplined learners: Adults who want the real discipline, not a trading-signals subscription

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FINANCIAL MARKETS & TRADING FOUNDATIONS (MONTHS 1-3, WEEKS 1-13)

Build solid understanding of financial markets, instruments, Python programming, and data analysis for trading.

Months 1-2: Markets, Instruments & Python Basics (Weeks 1-8)
Introduction to Financial Markets
Topics:
- Financial markets overview: stocks, forex, crypto, commodities
- Stock market basics: exchanges, indices, sectors
- Understanding stocks: equity, shares, market cap
- How stock prices move: supply and demand
- Market participants: retail, institutional, market makers
- Trading sessions: pre-market, regular, after-hours
- Order types: market, limit, stop-loss, stop-limit
- Bid-ask spread and liquidity
- Long vs short positions
- Margin trading and leverage
- Trading costs: brokerage, taxes, slippage
- Forex market: currency pairs, pips, lots
- Cryptocurrency markets: Bitcoin, altcoins, exchanges
- Market microstructure basics
- Regulatory environment and compliance
Projects built:
- Market research on different instruments
- Paper trading practice (manual)
- Order type experiments
- Market analysis report
- Trading journal setup
Practice: Paper trade 20 manual trades, document everything
Python Programming for Trading - Part 1
Topics:
- Why Python for algorithmic trading?
- Python installation and setup (Anaconda)
- Jupyter notebooks for trading analysis
- Variables and data types
- Lists, tuples, dictionaries for market data
- Control flow: if-else for trading conditions
- Loops for iterating through historical data
- Functions for trading logic
- NumPy for numerical operations

- Arrays for price data manipulation
- Pandas for financial data analysis
- DataFrames for OHLCV data
- Reading CSV files (historical data)
- DateTime handling for trading timestamps
- Basic data cleaning

Projects built:
- Load and explore stock price data
- Calculate basic statistics (mean, std, returns)
- Price data manipulation exercises
- Simple moving average calculator
- Trading signal generator (basic)

Practice: Solve 50 Python problems focused on financial data

Financial Data Analysis with Python

Topics:
- Data sources: Yahoo Finance, Alpha Vantage, Quandl
- yfinance library for stock data
- pandas_datareader for multiple sources
- OHLCV data: Open, High, Low, Close, Volume
- Data cleaning and preprocessing
- Handling missing data in time series
- Resampling: converting timeframes
- Forward-fill and back-fill methods
- Calculating returns: simple and log returns
- Cumulative returns calculation
- Volatility calculation and analysis
- Correlation analysis between assets
- Data visualization with Matplotlib
- Candlestick charts with mplfinance
- Creating trading dashboards

Projects built:
- Multi-asset data downloader
- Returns and volatility analyzer
- Correlation matrix for portfolio
- Interactive price charts
- Market data dashboard
- Historical performance analyzer

Practice: Analyze 50+ stocks, create comprehensive reports

Technical Analysis Foundations

Topics:
- Technical analysis principles
- Support and resistance levels
- Trend analysis: uptrend, downtrend, sideways
- Trendlines and channels
- Chart patterns: head and shoulders, triangles, flags
- Candlestick patterns: doji, hammer, engulfing
- Moving averages: SMA, EMA, WMA
- Moving average crossovers
- Bollinger Bands: calculation and interpretation
- Relative Strength Index (RSI)
- MACD: Moving Average Convergence Divergence
- Stochastic Oscillator
- Volume analysis and indicators
- Average True Range (ATR) for volatility
- Implementing indicators in Python

Projects built:
- Technical indicator library in Python
- SMA/EMA crossover strategy
- RSI-based trading signals
- Bollinger Bands strategy
- Multi-indicator dashboard
- Pattern recognition tool (basic)
- Technical analysis automation

Practice: Implement 20+ technical indicators from scratch

Month 3: Advanced Python & Market Analysis (Weeks 9-13)

Object-Oriented Programming for Trading Systems

Topics:
- OOP concepts for trading applications
- Creating trading strategy classes
- Portfolio class design
- Order management system (OMS) structure
- Risk management class
- Data handler class for market data
- Event-driven architecture for trading
- Inheritance for strategy variations
- Encapsulation for strategy logic
- Design patterns for trading systems
- Backtesting framework architecture

- Modular code organization
- Configuration management
- Logging for trading systems
- Error handling in live trading

Projects built:
- Trading strategy base class
- Portfolio manager class
- Order execution simulator
- Risk calculator class
- Complete OOP trading framework
- Modular backtesting engine

Practice: Refactor all previous code using OOP

Statistical Analysis for Trading

Topics:
- Descriptive statistics for returns
- Normal distribution and fat tails
- Hypothesis testing for strategies
- P-values and statistical significance
- Correlation and causation
- Cointegration for pairs trading
- Stationarity and unit root tests
- Autocorrelation in time series
- Monte Carlo simulations
- Bootstrap methods for trading
- Sharpe ratio calculation and interpretation
- Sortino ratio and downside deviation
- Maximum drawdown analysis
- Value at Risk (VaR)
- Risk-adjusted performance metrics

Projects built:
- Statistical analysis toolkit
- Returns distribution analyzer
- Strategy performance metrics calculator
- Monte Carlo simulator for trading
- Risk metrics dashboard
- Hypothesis testing framework

Practice: Statistical analysis of 30 trading strategies

Fundamental Analysis & Market Data APIs

Topics:
- Fundamental analysis basics
- Financial statements: P&L, balance sheet, cash flow
- Key ratios: P/E, P/B, ROE, debt-to-equity
- Earnings and revenue analysis
- Economic indicators impact on markets
- News sentiment analysis
- Alternative data sources
- Broker APIs: Interactive Brokers, Zerodha, Alpaca
- Real-time data streaming
- WebSocket connections for live data
- REST APIs for historical data
- Rate limiting and data management
- Building data pipelines
- Database setup for storing market data
- Time-series databases (InfluxDB, TimescaleDB)

Projects built:
- Fundamental data scraper
- Real-time data streamer
- Market data database setup
- Multi-source data aggregator
- News sentiment analyzer
- PHASE 1 MINI CAPSTONE: Complete Market Analysis Platform

Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 2: TRADING STRATEGIES & BACKTESTING (MONTHS 4-6, WEEKS 14-26)

Master strategy development, backtesting frameworks, optimization, and strategy evaluation.

Months 4-5: Strategy Development & Backtesting (Weeks 14-22)

Backtesting Framework Development

Topics:
- What is backtesting? Importance and limitations
- Backtesting bias: look-ahead, survivorship, overfitting
- Event-driven backtesting architecture
- Vectorized vs event-driven backtesting
- Building custom backtesting engine
- Backtrader library introduction
- Backtrader strategies and indicators

- Position sizing in backtests
- Commission and slippage modeling
- Realistic execution simulation
- Walk-forward analysis
- Out-of-sample testing
- Multiple timeframe backtesting
- Parameter optimization in backtests
- Backtesting best practices

Projects built:

- Custom backtesting engine from scratch
- Backtrader strategy implementation
- Slippage and commission simulator
- Walk-forward testing framework
- Multi-timeframe backtester
- Optimization engine

Practice: Backtest 20 strategies with different approaches

Momentum & Trend Following Strategies

Topics:

- Momentum trading principles
- Relative Strength Momentum
- Moving average crossover strategies
- Dual moving average system
- Triple moving average system
- MACD crossover strategy
- Trend following concepts
- Donchian Channel breakout
- ATR-based trailing stops
- Turtle trading system
- Channel breakout strategies
- Parabolic SAR strategy
- Supertrend indicator strategy
- Combining multiple trend indicators
- Exit strategies for trend trades

Projects built:

- Moving average crossover bot
- MACD strategy implementation
- Turtle trading system
- Donchian breakout strategy
- Multi-indicator trend follower
- Momentum portfolio strategy
- Optimized trend following system

Practice: Develop and backtest 15 momentum/trend strategies

Mean Reversion & Statistical Arbitrage

Topics:

- Mean reversion theory
- Bollinger Bands mean reversion
- RSI oversold/overbought strategy
- Stochastic mean reversion
- Z-score based strategies
- Pairs trading fundamentals
- Cointegration testing for pairs
- Pairs selection methodology
- Spread calculation and trading
- Statistical arbitrage concepts
- Market neutral strategies
- Long-short equity strategies
- Ornstein-Uhlenbeck process
- Half-life of mean reversion
- Kalman filter for pairs trading

Projects built:

- Bollinger Bands reversion strategy
- RSI mean reversion bot
- Pairs trading system
- Cointegration scanner
- Statistical arbitrage strategy
- Market neutral portfolio
- Kalman filter pairs trader

Practice: Build 12 mean reversion strategies

Breakout & Volatility Strategies

Topics:

- Breakout trading concepts
- Support/resistance breakouts
- High/low breakout strategies
- Opening range breakout (ORB)
- Volatility breakout strategy
- ATR-based position sizing
- Volatility expansion trading
- Volatility contraction setups
- Squeeze indicator strategies

- News-based breakout trading
- Volume-confirmed breakouts
- False breakout filtering
- Consolidation pattern breakouts
- Multiple timeframe confirmation
- Breakout strategy optimization

Projects built:
- Support/resistance breakout bot
- Opening range breakout strategy
- Volatility breakout system
- Squeeze strategy implementation
- Volume breakout trader
- Multi-timeframe breakout system
- False breakout filter

Practice: Develop 10 breakout trading strategies

Options Trading Strategies (Basics)

Topics:
- Options basics: calls, puts, strikes, expiry
- Option Greeks: Delta, Gamma, Theta, Vega
- Option pricing: Black-Scholes model
- Implied volatility and IV rank
- Covered call strategy
- Cash-secured put strategy
- Bull call spread and bear put spread
- Iron Condor strategy
- Straddle and strangle
- Option selling strategies
- Delta-neutral strategies
- Volatility trading with options
- Options data analysis in Python
- Option chain analysis
- Backtesting options strategies

Projects built:
- Options calculator (Greeks, pricing)
- IV rank analyzer
- Covered call screener
- Iron Condor backtester
- Options strategy simulator
- Volatility trading system

Practice: Implement 8 options strategies

Month 6: Advanced Strategies & Optimization (Weeks 23-26)

Strategy Optimization & Parameter Tuning

Topics:
- Optimization objectives and pitfalls
- Overfitting vs robust optimization
- Grid search optimization
- Random search methods
- Genetic algorithms for optimization
- Particle swarm optimization
- Bayesian optimization
- Walk-forward optimization
- Sensitivity analysis
- Parameter stability testing
- Monte Carlo optimization
- Multi-objective optimization
- Sharpe ratio optimization
- Drawdown-constrained optimization
- Transaction cost considerations

Projects built:
- Grid search optimizer
- Genetic algorithm optimizer
- Walk-forward optimization framework
- Parameter sensitivity analyzer
- Multi-objective optimizer
- Robust optimization system
- Strategy parameter dashboard

Practice: Optimize 20 strategies, compare methods

Risk Management & Position Sizing

Topics:
- Risk management principles
- Position sizing methods: fixed, percentage, volatility-based
- Kelly Criterion for position sizing
- Risk per trade calculation
- Stop-loss placement strategies
- Trailing stop techniques
- ATR-based stops
- Time-based exits
- Profit target strategies

- Risk-reward ratio analysis
- Portfolio heat and risk limits
- Correlation-based position sizing
- Leverage management
- Drawdown control
- Circuit breakers and kill switches

Projects built:
- Position sizing calculator
- Kelly Criterion implementation
- Stop-loss optimizer
- Risk management system
- Portfolio heat monitor
- Drawdown controller
- Kill switch implementation

Practice: Add risk management to all strategies

Multi-Asset & Portfolio Strategies

Topics:
- Portfolio construction principles
- Modern Portfolio Theory (MPT)
- Efficient frontier calculation
- Mean-variance optimization
- Risk parity strategies
- Equal weight vs optimal weight
- Tactical asset allocation
- Strategic asset allocation
- Rebalancing strategies
- Correlation-based diversification
- Sector rotation strategies
- Multi-strategy portfolio
- Strategy allocation optimization
- Portfolio backtesting
- Portfolio performance attribution

Projects built:
- Portfolio optimizer (Markowitz)
- Risk parity portfolio
- Sector rotation system
- Multi-strategy portfolio manager
- Rebalancing algorithm
- Portfolio backtester
- Asset allocation dashboard

Practice: Build 10 portfolio-level strategies

Cryptocurrency Trading Strategies

Topics:
- Crypto market characteristics
- Round-the-clock trading considerations
- Crypto exchanges: Binance, Coinbase, Kraken
- Crypto data sources and APIs
- Bitcoin and altcoin strategies
- Crypto volatility strategies
- Arbitrage: exchange arbitrage, triangular arbitrage
- DeFi and DEX trading
- Crypto market making basics
- Sentiment analysis for crypto
- On-chain data analysis
- Crypto-specific indicators
- High volatility management
- Crypto tax considerations
- Security and wallet management

Projects built:
- Crypto data collector
- Bitcoin trend following bot
- Crypto arbitrage scanner
- Altcoin momentum strategy
- Crypto mean reversion system
- Exchange arbitrage bot
- Crypto portfolio rebalancer

Practice: Develop 12 crypto trading strategies

Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 3: MACHINE LEARNING & ADVANCED QUANT (MONTHS 7-9, WEEKS 27-39)

Master machine learning for trading, advanced quantitative strategies, and high-frequency concepts.

Months 7-8: Machine Learning for Trading (Weeks 27-35)

ML Fundamentals for Trading

Topics:

- Machine learning in trading overview
- Supervised vs unsupervised learning
- Feature engineering for trading
- Price-based features
- Technical indicator features
- Sentiment features
- Fundamental features
- Alternative data features
- Label creation: classification vs regression
- Triple-barrier method for labeling
- Train-test split for time series
- Cross-validation for trading models
- Overfitting in trading ML
- Model evaluation metrics for trading
- Scikit-learn for trading models

Projects built:

- Feature engineering pipeline
- Label creation system
- ML data preparation framework
- Cross-validation for time series
- Trading-specific feature library
- Model evaluation toolkit

Practice: Create 100+ features for trading

Classification Models for Trading

Topics:

- Price direction prediction
- Logistic regression for trading signals
- Decision trees for market regimes
- Random Forest for trading
- Feature importance analysis
- Gradient Boosting (XGBoost, LightGBM)
- Support Vector Machines for classification
- Ensemble methods for trading
- Model stacking and blending
- Probability calibration
- Threshold optimization for signals
- Handling imbalanced data
- SMOTE for trading datasets
- Model interpretation for trading
- Backtesting ML strategies

Projects built:

- Price direction classifier
- Random Forest trading strategy
- XGBoost market predictor
- Ensemble trading model
- Market regime classifier
- ML signal generator
- Complete ML trading strategy

Practice: Build 15 ML classification strategies

Deep Learning for Trading

Topics:

- Neural networks for trading
- LSTM for time series prediction
- GRU networks for trading
- Sequence-to-sequence models
- Attention mechanism for trading
- Transformer models for finance
- CNN for technical pattern recognition
- Autoencoders for feature extraction
- Reinforcement learning for trading (Q-learning)
- Deep Q-Networks (DQN) for trading
- Policy gradient methods
- TensorFlow and Keras for trading
- PyTorch for trading models
- GPU acceleration for training
- Model deployment considerations

Projects built:

- LSTM price predictor
- CNN pattern recognition strategy
- Autoencoder for anomaly detection
- Reinforcement learning trader
- DQN trading agent
- Transformer-based predictor
- Deep learning ensemble system

Practice: Implement 10 deep learning trading models

NLP & Sentiment Analysis for Trading

Topics:

- Natural language processing in trading
- News sentiment analysis

- Twitter sentiment for stocks/crypto
- Financial news scraping
- Text preprocessing for finance
- Sentiment scoring methods
- VADER for financial sentiment
- FinBERT for financial text
- Topic modeling for market themes
- Event extraction from news
- Earnings call analysis
- Social media signal extraction
- Alternative text data sources
- Real-time sentiment streaming
- Combining sentiment with price action

Projects built:

- News sentiment analyzer
- Twitter sentiment tracker
- Earnings call sentiment extractor
- Reddit sentiment for stocks/crypto
- Real-time sentiment dashboard
- Sentiment-based trading strategy
- Multi-source sentiment aggregator

Practice: Build 8 sentiment-based strategies

Alternative Data & Quantitative Research

Topics:

- Alternative data in trading
- Satellite imagery analysis
- Web scraping for trading signals
- Credit card transaction data
- Supply chain data
- Weather data for commodities
- Social media activity metrics
- App download data
- Google Trends for trading
- Data vendor integration
- Alpha generation from alt data
- Factor models and factor investing
- Fama-French factors
- Custom factor creation
- Quantitative research process

Projects built:

- Web scraping trading signals
- Google Trends strategy
- Alternative data aggregator
- Factor model implementation
- Multi-factor strategy
- Quantitative research report
- Alpha generation framework

Practice: Research and test 10 alternative data sources

Month 9: Advanced Trading Concepts (Weeks 36-39)

High-Frequency Trading (HFT) Concepts

Topics:

- High-frequency trading overview
- Market microstructure for HFT
- Order book dynamics
- Limit order book analysis
- Market making strategies
- Bid-ask spread capture
- Statistical arbitrage at high frequency
- Latency arbitrage concepts
- Co-location and infrastructure
- Low-latency programming
- FPGA and hardware acceleration
- Tick data processing
- Order flow analysis
- Liquidity provision strategies
- HFT regulations and compliance

Projects built:

- Order book visualizer
- Limit order book simulator
- Simple market making bot
- Order flow analyzer
- Tick data processor
- Microstructure research tools
- HFT backtesting framework

Practice: Analyze order book data, simulate HFT strategies

Advanced Options & Derivatives

Topics:

- Advanced option strategies

- Volatility trading deep dive
- VIX trading strategies
- Volatility arbitrage
- Skew trading strategies
- Calendar spread strategies
- Diagonal spreads
- Ratio spreads
- Butterfly and Condor variations
- Delta hedging automation
- Gamma scalping
- Options market making
- Exotic options basics
- Futures and futures spreads
- Commodity spread trading

Projects built:

- Volatility trading system
- VIX trading strategy
- Calendar spread optimizer
- Delta hedging bot
- Gamma scalping system
- Options portfolio manager
- Futures spread trader

Practice: Implement 12 advanced derivatives strategies

Execution Algorithms & Transaction Cost Analysis

Topics:

- Execution algorithms overview
- VWAP (Volume-Weighted Average Price) algorithm
- TWAP (Time-Weighted Average Price) algorithm
- Implementation shortfall
- Adaptive execution algorithms
- Iceberg orders and order slicing
- Smart order routing
- Transaction cost analysis (TCA)
- Slippage measurement and modeling
- Market impact models
- Broker execution quality analysis
- Fill rate optimization
- Execution latency measurement
- Pre-trade and post-trade analysis
- Best execution requirements

Projects built:

- VWAP execution algorithm
- TWAP execution bot
- Order slicing algorithm
- Transaction cost analyzer
- Slippage tracker
- Execution quality dashboard
- Smart order router (basic)

Practice: Implement and analyze 8 execution strategies

Quantitative Portfolio Management

Topics:

- Advanced portfolio construction
- Black-Litterman model
- Factor-based portfolio construction
- Multi-factor models
- Smart beta strategies
- Long-short portfolio construction
- 130/30 strategies
- Market-neutral portfolio management
- Portfolio rebalancing optimization
- Transaction cost in rebalancing
- Portfolio risk decomposition
- Tracking error minimization
- Active vs passive management
- Hedge fund strategies
- Performance attribution analysis

Projects built:

- Black-Litterman portfolio optimizer
- Multi-factor portfolio constructor
- Long-short portfolio manager
- Smart beta strategy
- Rebalancing optimizer with costs
- Portfolio analytics dashboard
- Performance attribution system

Practice: Build 10 portfolio management systems

Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)

The capstone month: spec, build, ship and defend, then the final exam that decides the certificate.

Month 10: Capstone and the Final Exam (Weeks 40-44)
Capstone Spec and Setup
Topics:
- Choosing the capstone: a complete, honestly backtested strategy system: research, signals, risk sizing and a defensible evaluation
- A one-page spec: scope, milestones, stack
- Repository and skeleton ready on day one
- Instructor sign-off before the build begins
Projects built:
- Approved capstone spec plus the running skeleton
Practice: Pitch the spec to the batch and absorb one hard question
Capstone Build
Topics:
- Two focused weeks in vertical slices
- Instructor checkpoints and honest scope cuts
- Using the whole course: Python for markets, strategy design, bias-free backtesting, risk management and ML where it earns its place
- Daily commits and an always-working build
Projects built:
- The capstone, feature-complete with version control
Practice: A working build at the end of every session
Ship and Polish
Topics:
- The finishing pass: edge cases, errors, documentation
- Backtested without self-deception: results that survive out-of-sample and honest cost assumptions
- A README worth reading and a demo worth watching
- A full-course spiral review before the exam
Projects built:
- The capstone shipped, documented and rehearsed
Practice: Two mock demos, each tighter than the last
Demo Day and the Final Exam
Topics:
- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: quant analyst, quant developer or fintech data roles; trading your own capital is a personal decision, not a placement
- Where our other courses continue the journey
Projects built:
- Demo day presentation delivered to the batch
Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: financial-data analysis tools and technical indicators in Python
- Phase 2: backtested strategies with risk sizing and honest evaluation
- Phase 3: ML-driven signals and execution-aware research
- Phase 4: the fully backtested strategy-system capstone

Total projects: 15+ research and backtesting projects, crowned by a full strategy system

SKILLS THE STUDENT WILL HAVE AT THE END

- Python for financial data: cleaning, analysis and indicators
- Strategy design across momentum, mean-reversion and volatility
- Backtesting without self-deception: defeating lookahead, overfitting and survivorship bias
- Risk management and position sizing as first principles
- Machine learning for trading, applied with domain scepticism

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short
    and there is focused revision and a free retest: the certificate certifies knowledge, not
    attendance

WHERE THIS CAN LEAD

  - Quant analyst and quant developer roles in funds, banks and fintech
  - Our trading automation course for the systems and execution half
  - Data science and ML roles: the research skills transfer directly
  - Trading your own capital is a personal financial decision, made with full awareness of the
    risk

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What does this algorithmic trading course cover?
  A: The research-and-strategy half of quant trading, taught as engineering: Python for
  financial data, technical and statistical analysis, strategy design across momentum,
  mean-reversion and volatility, honest backtesting that defeats lookahead, overfitting and
  survivorship bias, risk management and position sizing, then machine learning for trading
  with sentiment and alternative data. A finale month builds a fully backtested strategy system
  and sits the final exam. It is education, not financial advice.

  Q: How long does it take to learn algorithmic trading properly?
  A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two
  more months in hand for learners who need them. The three teaching phases build the skill and
  its real-world application; the finale month ships a capstone and sits the final exam.

  Q: How am I assessed? Is the certificate just for attending?
  A: No. Problem sets follow every class with written feedback, monthly reviews deliberately
  mix new material with earlier topics, each phase ends with a cumulative exam, and the course
  ends with a final exam: a practical build plus written questions from the whole syllabus.
  Passing earns the certificate; falling short earns focused revision and a free retest.

  Q: Is this financial advice? Will I make guaranteed profits?
  A: No, and be wary of anyone who says otherwise. This is an engineering and quantitative
  course: it teaches you to research, build, backtest and automate trading systems, not what to
  buy or a way to get rich. Markets carry real risk of loss, backtest results do not guarantee
  future returns, and we make no profit claims of any kind. What you gain is genuine, in-demand
  skill; what you do with real money afterwards is your own decision and your own risk.

  Q: How is this different from your trading automation course?
  A: This course is the research-and-strategy half: what to trade, why, and how to know whether
  a strategy is real, statistics, backtesting honesty and machine learning. The trading
  automation course is the systems half: how to build reliable, production-grade bots that
  execute through broker APIs with monitoring and safeguards. Many serious learners take both;
  this one makes you the researcher, that one makes you the systems engineer.

  Q: Do I need finance or trading experience to start?
  A: No. Markets and the maths you need are taught from the ground up inside the course. What
  helps most is comfort with logical thinking and a willingness to be sceptical of your own
  results, which is the single most important habit in quantitative trading.

  Q: Can I try a class before enrolling?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or
  message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with
  no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at
  https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor reading your actual code, never
  pre-recorded videos.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link:
https://learn.modernagecoders.com/courses/algorithmic-trading-masterclass-complete
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 16 OF 137: ALPHA GIRLS: SYSTEMS AT SCALE FOR SENIOR WOMEN ENGINEERS
==========

Course link: https://learn.modernagecoders.com/courses/alpha-girls-elite-tech-mastery
Subject area: Women in Tech
Designed for: Girls only batches

Who can join: Experienced women engineers (roughly 4+ years or Queen Coders graduates)
Course length: 9-11 months (36-44 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Senior engineering course for women: system design at scale, reliability, platforms, ML systems and technical strategy in 9-11 months, live online.

  In one line: The deepest technical road: systems at scale, reliability as a discipline, and the strategy skills of engineers whole companies listen to.

ABOUT THE PROGRAMME

  Some engineers choose depth as the career: the people rooms go quiet for when architecture is on the table. Alpha Girls is that road, built for experienced women engineers: system design at genuine scale, data platforms, reliability engineering with SLOs and honest incident practice, platform thinking, ML systems in production, security and compliance leadership, and the technical strategy craft, roadmaps, build-versus-buy, architecture reviews, that turns depth into organizational influence. The course runs on artifacts: every month produces designs, reviews and documents of the kind staff and principal engineers are actually judged on, and it ends with the capstone, a complete system redesign proposal defended before a hard panel. No fantasy titles and no throne metaphors: scale is an engineering property here, not a personality. Nine months at two live classes a week plus four to six hours of practice, with two more in hand. Monthly mixed reviews, phase exams, and a final defense that decides the certificate with a free retest. Depth, chosen deliberately.

WHAT MAKES THIS COURSE DIFFERENT

  - Scale as engineering, not vocabulary: capacity math, failure domains and cost curves done honestly
  - Reliability as a discipline: SLOs, incident command and postmortems practiced, not described
  - ML systems in production: the operational reality behind the model
  - Strategy as craft: roadmaps, reviews and build-vs-buy argued with artifacts
  - A senior peer room of women, rare and deliberately built
  - Real assessment: artifact reviews monthly and a final panel defense that decides the certificate, with a free retest

THE LEARNING PATH, STEP BY STEP

  Phase 1: Systems at scale (Months 1-3): design method, data at scale, cloud architecture, reliability and performance economics
  Phase 2: Platforms and production ML (Months 4-6): platform engineering, ML systems, security leadership, and an architecture review cycle
  Phase 3: Technical strategy (Months 7-8): roadmaps, build-vs-buy, leading through migrations and incidents
  Phase 4: The finale (Month 9): the redesign capstone and the panel defense

WHAT A STUDENT NEEDS BEFORE STARTING

  Experience: Substantial engineering experience, roughly 4+ years or Queen Coders graduation; a short portfolio conversation places every applicant honestly
  Equipment: A computer with a stable internet connection
  Audience: Senior women engineers choosing depth as the career
  Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

  Depth choosers: Engineers who want the senior IC road, staff and principal, walked deliberately
  Scale facers: Women whose systems have started failing in interesting ways
  Strategy builders: Engineers whose ideas are right but underfunded, who need the influence craft
  Returning seniors: Experienced engineers returning to depth after management or a break
  Rare room seekers: Senior women who have never had a senior-women peer room, and want one

FULL CURRICULUM, WEEK BY WEEK

  This is the complete teaching plan. Parents can ask about any week.

PHASE 1: SYSTEMS AT SCALE (MONTHS 1-3, WEEKS 1-12)

The core discipline: design method under load, data at scale, cloud architecture, reliability
and the economics of performance.

  Month 1: Design Method Under Load (Weeks 1-4)
    System Design at Scale: The Method
      Topics:
        - Facebook/Google/Amazon architecture deep dive
        - Multi-datacenter global infrastructure
        - Edge computing at planetary scale
        - Real-time systems with <10ms latency
        - Handling 1M+ requests per second
        - Database systems for exabyte-scale data
        - Network architecture for global systems
        - Distributed consensus at scale (Raft, Paxos)
        - Chaos engineering for critical systems
        - Zero-downtime deployment strategies
      Projects built:
        - Design Twitter-scale messaging system
        - Build Netflix-level video streaming architecture
        - Create Uber's global dispatch system
        - Implement Google-scale search infrastructure
      Practice: Daily: Architect systems handling 100M+ users
    Cloud Architecture, Judged
      Topics:
        - Multi-cloud architecture for Fortune 500
        - Hybrid cloud with on-premise integration
        - Kubernetes at massive scale (10K+ nodes)
        - Service mesh for 1000+ microservices
        - Global traffic management (Anycast, GeoDNS)
        - Multi-region active-active architectures
        - Disaster recovery for critical infrastructure
        - Cloud cost optimization saving millions
        - Compliance across 50+ countries
        - Enterprise security architecture
        - Zero-trust implementation at scale
      Projects built:
        - Migrate Fortune 500 to cloud (real project)
        - Build multi-cloud platform for enterprise
        - Design $10M+ cost optimization strategy
        - Implement global compliance framework
      Assessment: Month 1 check: a capacity-math design exercise defended, plus mixed review

  Month 2: Data and Reliability (Weeks 5-8)
    Data Platforms and Pipelines
      Topics:
        - Storage engines by workload: the honest decision table
        - Partitioning, replication and the consistency you actually need
        - Batch and streaming pipelines: when each earns its complexity
        - Schema evolution without weekend outages
        - Data quality as an engineering property
        - Cost curves: where data spend actually goes
      Projects built:
        - A data platform design for a stated workload, with the decision table shown
      Practice: One storage decision re-argued from a real system you know
    Reliability Engineering
      Topics:
        - SLIs, SLOs and error budgets that change behavior
        - Failure domains: drawing the blast radius before it draws itself
        - Incident command: roles, comms and the clock
        - Postmortems that are blameless and still honest
        - Degradation by design: what fails first, on purpose
        - On-call humanely: sustainable operational load
      Projects built:
        - An SLO set plus incident runbook for a chosen service
      Practice: Run one tabletop incident with your batch
      Assessment: Month 2 check: a reliability review presented, plus mixed review

  Month 3: Performance Economics and ML Systems (Weeks 9-12)
    Performance at Scale
      Topics:
        - Profiling before opinions: where time actually goes
        - Caching layers and their invalidation debts
        - Queueing theory enough to argue with
        - Latency budgets end to end
        - Cost as a performance metric: the bill is telemetry
        - Load testing that predicts instead of reassures
      Projects built:

```
        - A performance investigation with before/after measurements
      Practice: Profile one real system and write the honest findings
    ML Systems in Production
      Topics:
        - Training models on petabyte datasets
        - Distributed training across 1000+ GPUs
        - Real-time ML inference <5ms latency
        - A/B testing ML models at scale
        - Feature stores for enterprise ML
        - ML pipelines processing TB/hour
        - Computer vision for millions of images/day
        - NLP systems for 100+ languages
        - Recommendation systems like YouTube/TikTok
        - Fraud detection processing $1B+ transactions
        - AutoML platforms for enterprise
      Projects built:
        - Build TikTok-scale recommendation system
        - Create enterprise fraud detection saving $100M
        - Implement real-time translation for 50 languages
        - Design autonomous vehicle ML pipeline
      Practice: Deploy ML models serving 1B+ requests/day
      Assessment: Phase 1 exam: a scale design defended under hard questioning, plus a
      written mixed paper

---------------------------------------------------------------
PHASE 2: PLATFORMS AND PRODUCTION JUDGMENT (MONTHS 4-6, WEEKS 13-24)
---------------------------------------------------------------
Platform engineering, security and compliance leadership, and the architecture review cycle
practiced as craft.

  Month 4: Platform Engineering (Weeks 13-16)
    Platforms and Developer Experience
      Topics:
        - What a platform is: paved roads, not police
        - Golden paths: making the right way the easy way
        - Internal APIs and the contracts that keep teams decoupled
        - Migration mechanics: how platforms actually get adopted
        - Measuring developer experience without theater
        - When NOT to build a platform
      Projects built:
        - A platform proposal for a real pain, with adoption plan
      Practice: Interview two engineers about their worst golden-path gap
    Security and Compliance Leadership
      Topics:
        - Threat modeling as a design habit
        - The security review that teams do not dread
        - Secrets, identity and the boundaries that matter
        - Compliance translated: what auditors need vs what teams fear
        - Incident response for security events
        - Building security culture without becoming the villain
      Projects built:
        - A threat model and security review for a chosen system
      Practice: Run one lightweight security review with your batch
      Assessment: Month 4 check: platform or security artifact reviewed, plus mixed review

  Month 5: The Architecture Review Cycle (Weeks 17-20)
    Architecture Reviews and ADRs
      Topics:
        - Decision records that future engineers thank you for
        - Running reviews that surface risk without crushing authors
        - The reviewer's question toolkit
        - Disagree-and-commit done honestly
        - Review anti-patterns: rubber stamps and ambushes
        - A review culture one engineer can start
      Projects built:
        - Two ADRs written and two peer designs reviewed in cycle
      Practice: One review given and one received weekly
    Build, Buy or Wait
      Topics:
        - Total cost of ownership beyond the invoice
        - Vendor judgment: lock-in, escape hatches and leverage
        - Open source as a strategic dependency
        - The wait option: when deferring is the strong move
        - Writing the recommendation executives can act on
        - Sunk costs and the courage to reverse
      Projects built:
        - A build-vs-buy recommendation with the TCO math shown
      Practice: Re-argue one past build-vs-buy from your own history
      Assessment: Month 5 check: recommendation defended, plus mixed review
```

```
  Month 6: The Mid-Course Defense (Weeks 21-24)
    Integration: An AI-Era System at Scale
      Topics:
        - Designing a system where ML and scale pressures meet
        - Evaluation, guardrails and rollback for model-driven features
        - Data flywheels and their governance
        - Cost control when inference is in the hot path
      Projects built:
        - An AI-era system design integrating the phase's judgment
      Practice: Panel-style questioning practice in pairs
    Phase 2 Defense
      Topics:
        - Assembling the mid-course portfolio: designs, reviews, recommendations
        - Defense practice: hard questions, honest answers
        - A cumulative review of phases 1-2
      Projects built:
        - The mid-course portfolio, defended in panel
      Assessment: Phase 2 exam: portfolio defense plus a cumulative written paper

---------------------------------------------------------------
PHASE 3: TECHNICAL STRATEGY (MONTHS 7-8, WEEKS 25-32)
---------------------------------------------------------------
Turning depth into direction: roadmaps, migrations, incident leadership and the influence
craft of principal engineers.

  Month 7: Direction (Weeks 25-28)
    Technical Strategy and Roadmaps
      Topics:
        - Strategy as diagnosis, policy and coherent action
        - Roadmaps that survive contact with quarters
        - Sequencing: the order that de-risks everything after it
        - Saying no with alternatives attached
        - Writing strategy short enough to be read
        - Getting non-engineers to fund engineering work
      Projects built:
        - A one-page technical strategy for a real system's next year
      Practice: Present the strategy to a non-engineer and revise from their questions
    Leading Migrations
      Topics:
        - Migrations as the true test of senior engineering
        - Strangler patterns and reversible steps
        - The long middle: keeping momentum after novelty dies
        - Data migrations without lost weekends
        - Deprecation with dignity: turning things off kindly
        - Migration comms: the updates that keep trust
      Projects built:
        - A migration plan for a real legacy pressure, reversible steps shown
      Practice: Postmortem one migration you lived through, honestly
      Assessment: Month 7 check: strategy or migration artifact defended, plus mixed review

  Month 8: Influence (Weeks 29-32)
    Incident Leadership and Hard Weeks
      Topics:
        - Commanding incidents when the org is watching
        - Communication under pressure: cadence and candor
        - The recovery week: postmortem to prevention
        - Leading tired teams without spending trust
        - Escalation judgment: when to wake whom
      Projects built:
        - A full incident-command simulation, led
      Practice: One tabletop incident led end to end
    Mentoring, Sponsorship and the Room
      Topics:
        - Mentoring versus sponsorship: the door versus the push through it
        - Growing engineers without cloning yourself
        - Technical writing that scales your judgment
        - Speaking: the internal talk that changes direction
        - Being the senior woman in the room, on your own terms
      Projects built:
        - One mentee plan and one internal talk delivered to the batch
      Practice: Mentor one junior from our other courses for the month
      Assessment: Phase 3 exam: influence portfolio reviewed, plus cumulative written paper

---------------------------------------------------------------
PHASE 4: THE FINALE (MONTH 9, WEEKS 33-36)
---------------------------------------------------------------
The redesign capstone: one real system taken from pressure to proposal to panel.
```

Month 9: The Redesign and the Panel (Weeks 33-36)
Capstone Build: The Redesign Proposal
Topics:
- Choosing the system: real pressure, real stakes
- Current-state honesty: what is actually there
- Target architecture with capacity and cost math
- Reliability, security and migration woven in
- The proposal document, executive summary first
Projects built:
- THE CAPSTONE: a complete system redesign proposal
Practice: Daily dossier progress; sections panel-reviewed as written
Red Team Week
Topics:
- Your proposal, attacked by your peers
- Finding the weakest argument before the panel does
- Revision under pressure
- Final rehearsal
Projects built:
- The proposal, hardened by red-team review
Practice: Attack two peer proposals with rigor and kindness
The Final Defense
Topics:
- The panel defense: proposal presented, hard questions taken
- A full-course spiral review
- The road on: principal work, and the community that stays
Projects built:
- The panel defense, delivered
Assessment: FINAL EXAM: the capstone defense plus a written paper mixed from every phase. Passing earns the certificate; falling short earns focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: capacity-math designs, a data platform design, SLOs and runbooks, a performance investigation
- Phase 2: a platform proposal, a threat model, ADR cycles and a build-vs-buy recommendation
- Phase 3: a one-page strategy, a migration plan, an incident simulation and a delivered talk
- Phase 4: the redesign proposal, red-teamed and defended before a panel

Total projects: A staff-level artifact portfolio, crowned by the defended redesign proposal

SKILLS THE STUDENT WILL HAVE AT THE END

- System design at scale with honest capacity and cost math
- Reliability as a practiced discipline: SLOs, incidents, postmortems
- Platform, security and ML-in-production judgment
- Technical strategy: direction argued so it gets funded
- Influence: reviews, mentoring, writing and speaking that scale judgment

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week
Homework: Homework after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of practice between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Monthly artifact reviews with named focus areas

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Staff and principal engineering: the artifacts here are those interviews
- Architecture and platform leadership roles
- Reliability and infrastructure specialization
- Mentoring the ladder: Queen and Princess batches benefit from Alpha graduates
- The peer room itself: senior women engineers who stay in each other's corner

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Alpha Girls, and who is it for?
A: The deepest technical course in our women-in-tech ladder: systems at scale, reliability engineering, platforms, ML in production and technical strategy, for experienced women engineers (roughly 4+ years or Queen Coders graduates).

Q: How is this different from Queen Coders?
A: Queen builds the step from good developer to trusted engineer: advanced engineering, AI fluency and first technical leadership. Alpha assumes that step and goes deeper into the senior IC road: scale, reliability, platforms and strategy, the work of staff and principal engineers. Many students take them in sequence with working years between.

Q: Is this a management course?
A: No: it is the senior individual-contributor road, walked deliberately. Leadership here means technical direction, reviews, mentoring and incident command, influence without the org chart. The management fork is examined honestly in Queen Coders; Alpha is for women who choose depth.

Q: How is progress assessed? Is the certificate just for attending?
A: No. Homework follows every class, each month ends with an assignment that deliberately mixes new questions with earlier topics, every phase closes with a cumulative exam, and the course ends with a final exam. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: What does the final defense involve?
A: A complete redesign proposal for a real system under real pressure: current-state honesty, target architecture with capacity and cost math, reliability and migration woven in, red-teamed by peers in week 35 and then defended before a panel that asks the questions real review boards ask. Passing it, plus the written paper, earns the certificate.

Q: Is there a free trial class?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor, never pre-recorded videos.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/alpha-girls-elite-tech-mastery
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 17 OF 137: AP CALCULUS AB & BC EXAM PREP, LIVE ONLINE, SCORE 5 GOAL

==========

Course link: https://learn.modernagecoders.com/courses/ap-calculus-exam-prep
Subject area: Test Prep Mathematics
Designed for: High school
Who can join: AP Calculus AB and BC students, grades 11-12
Course length: 10 months (40 weeks)
Time needed: 2 live one-hour classes per week + weekly problem sets
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online AP Calculus AB and BC prep: limits, derivatives and integrals taught as ideas, every FRQ against real rubrics, plus full mock exam cycles.

In one line: Three ideas, taught like ideas, then trained against the real rubrics.

ABOUT THE PROGRAMME

A 10-month live program that runs alongside your student's AP Calculus course and aims squarely at the exam. All of calculus is three ideas, the limit, the derivative, the integral, and students who own them as ideas find the AP exam fair. We teach ideas first, repair the precalculus debt that quietly causes most 'calculus errors', then convert understanding into score: every free-response question type drilled against actual College Board rubrics, multiple-choice pacing, calculator-section strategy, and full mock cycles timed like the real exam. AB and BC students are taught to their exact course.

WHAT MAKES THIS COURSE DIFFERENT

- Ideas before technique: the derivative discovered through shrinking intervals, the FTC earned as a punchline, so unfamiliar exam framings do not break the student
- Precalculus repaired inline: weak composition and trig are fixed inside calculus problems, where the repair sticks
- FRQ craft against real rubrics: how points are actually awarded, including justification language graders look for
- AB and BC tracks: series, parametrics and polar taught properly for BC students
- Calculator and no-calculator sections trained as different skills
- Full mock cycles with documented score progression, peaking in exam week
- One dedicated mentor across the whole year, one full interactive hour per class
- Honest guidance: a diagnostic first, evidence-based targets, never invented score promises

THE LEARNING PATH, STEP BY STEP

Phase 1: Limits and derivatives as ideas, with precalculus repaired inline and the rule toolkit built to speed
Phase 2: Derivative applications, integrals, the Fundamental Theorem and differential equations, the AB core completed
Phase 3: BC extensions (series, parametrics, polar), then exam craft: FRQ rubrics, MCQ pacing, calculator strategy and full mock cycles

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Enrolled in (or about to start) AP Calculus AB or BC, typically grades 11-12
Math background: Precalculus completed at any comfort level, the diagnostic finds the real base and we repair inline
Equipment: Computer with stable internet; an approved graphing calculator for the calculator sections
Time commitment: 2 live hours plus 2-3 practice hours weekly
English: Course taught in English
Motivation: The May exam date does the motivating; we supply the method

WHO THIS COURSE IS FOR

Ab students: AP Calculus AB students who want the ideas owned and the exam trained, not just survived
Bc students: BC students who need series, parametrics and polar taught properly, plus the pace to finish strong
Struggling mid year: Students whose first-semester grade says the memorize-and-hope approach is failing
Score maximizers: Strong students hunting a 5 through FRQ craft and error elimination
Self studiers: Disciplined students taking the exam without an AP class at school

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: LIMITS AND DERIVATIVES AS IDEAS (WEEKS 1-14)

The diagnostic finds the real starting point, including the precalculus debt, then the two foundational ideas are built properly: the limit as approach, the derivative as instantaneous rate, and the rule toolkit drilled to exam speed only after the ideas hold.

Weeks 1-3: Diagnostic + Limits as Approach
Topics:
- Full diagnostic: calculus-so-far plus the precalculus underneath (composition, trig, algebra fluency)
- Limits graphically, numerically and analytically, one idea, three faces
- One-sided limits, infinite limits and limits at infinity
- Continuity as a definition that finally makes sense
- The Intermediate Value Theorem as an exam justification pattern
- Precalculus repair begins: the personal gap list, attacked weekly

Projects built:
- Personal error log and gap map for the whole year
- Limit-evaluation toolkit sheet, built not copied

Practice: Complete a limits problem set spanning graphical, numerical and analytic forms, writing full sentence IVT justifications for two of them.

Weeks 4-8: The Derivative, Discovered Then Drilled
Topics:
- The derivative discovered via shrinking-interval tables, limit and derivative met as one idea
- Derivative as slope, as rate, and as the exam's favorite interpretation question
- Differentiability vs continuity, and where functions fail to have derivatives
- The rule toolkit: power, product, quotient rules built from meaning
- The chain rule as composition unwrapping, the make-or-break skill

- Trig, exponential and logarithmic derivatives to fluency
Projects built:
- Shrinking-interval investigation: one data table to both core ideas
- Timed derivative gauntlet at exam pace, reviewed for error patterns
Practice: Run a 20 minute derivative drill most evenings covering power, product, quotient and chain rules, and log any rule that misfires under speed.

Weeks 9-14: Implicit, Inverse and the First Applications
Topics:
- Implicit differentiation as the chain rule in disguise
- Derivatives of inverse functions, including inverse trig
- Related rates from pictures: setup discipline that survives exam pressure
- Local linearity and tangent-line approximation
- L'Hospital's Rule, used where it is actually allowed
- First FRQ exposure: rate and interpretation questions against the rubric
Projects built:
- Related-rates case set: five classic setups from picture to answer
- Phase checkpoint: timed AB-style mixed section with full review
Practice: Solve six related rates problems, starting each with a labeled picture, plus a short implicit differentiation set with every step written out.

PHASE 2: ANALYSIS, INTEGRALS AND THE FTC (WEEKS 15-28)

Curve analysis makes the derivative a reasoning tool, then the integral arrives as accumulation and the Fundamental Theorem connects the two halves of the course. By the end of this phase the entire AB syllabus is owned.

Weeks 15-19: Curve Analysis and Optimization
Topics:
- First and second derivative analysis: increasing, concavity, extrema, inflection
- The Mean Value Theorem and its exam justification language
- Curve sketching from f' and f'', the classic FRQ family
- Optimization from word problems: model first, calculus second
- Motion on a line: position, velocity, acceleration, speed
- Justification writing: the exact phrases rubrics award points for
Projects built:
- Full f-f'-f'' graph-relationship FRQ solved and self-graded on the rubric
- Optimization mini-case portfolio: fence, box, cost, distance
Practice: Complete two past style curve analysis FRQs using exact MVT and extrema justification language, then self grade against the rubric.

Weeks 20-24: The Integral as Accumulation + the FTC
Topics:
- Riemann sums by hand and in tables, the accumulation idea made concrete
- The definite integral as net change; units and interpretation questions
- The Fundamental Theorem of Calculus, arrived at as a discovery
- Antiderivatives and the substitution method as the chain rule reversed
- Functions defined by integrals, the FRQ type students fear most, defanged
- Average value and the accumulation FRQ family
Projects built:
- Riemann-to-FTC investigation: watching sums converge
- Accumulation FRQ set against real rubrics
Practice: Work a Riemann sum table set by hand, then a substitution drill, and explain the Fundamental Theorem in your own words at the top of the page.

Weeks 25-28: Applications of Integrals + Differential Equations
Topics:
- Area between curves, set up from sketches
- Volumes: discs, washers and known cross-sections
- Separable differential equations and exponential models
- Slope fields: reading, matching and sketching
- Motion problems revisited with integrals
- AB syllabus checkpoint: full timed section, error log updated
Projects built:
- Volume gallery: one solid each way, from sketch to setup to answer
- AB mock exam number one with documented score
Practice: Set up and solve three area and volume problems, drawing each region first, plus one separable differential equation with a slope field sketch.

PHASE 3: BC TOPICS + EXAM CRAFT AND MOCK CYCLES (WEEKS 29-40)

BC students complete series, parametrics and polar; AB students go straight to craft. Then the program becomes an exam machine: FRQ rubric mastery, MCQ pacing, calculator strategy and full mock cycles that peak in the first week of May.

Weeks 29-33: BC Extensions (BC track) / Deep Review (AB track)
Topics:

- Integration by parts and partial fractions (BC)
- Improper integrals and L'Hospital revisited (BC)
- Sequences and series: convergence tests as a decision tree (BC)
- Taylor and Maclaurin polynomials; Lagrange error bound (BC)
- Parametric and polar: derivatives, speed, arc length and area (BC)
- AB track in parallel: full-syllabus spaced review by error-log priority

Projects built:
- Series convergence decision-tree card, built by the student
- Parametric motion mini-case (BC) / AB mixed mock section (AB)

Practice: Apply the convergence decision tree to ten mixed series (or work the AB deep review set), noting which test you chose and why for each.

Weeks 34-37: FRQ Rubric Mastery and Calculator Strategy

Topics:
- The six FRQ archetypes and their rubric patterns, drilled one by one
- Justification language: what earns the point and what graders ignore
- Calculator-active strategy: what to store, graph and compute
- No-calculator section speed: exact values and clean algebra under time
- MCQ pacing plans built from personal timing data
- Partial-credit engineering: banking points on problems you cannot finish

Projects built:
- Self-graded FRQ portfolio: one of each archetype against real rubrics
- Personal pacing plan for all four exam sections

Practice: Grade two of your own completed FRQs point by point against the released rubric and rewrite any justification that would not earn credit.

Weeks 38-40: Full Mock Cycles and Peak Week

Topics:
- Full-length mocks under exact AP timing, AB or BC as registered
- Question-by-question review: reasoning, rubric, and the named fix
- Error-log closure and final target calibration
- Peak-week plan: light practice, rest, logistics and warm-up routine
- Exam-day rituals: the reset after a hard FRQ, time checkpoints per section

Projects built:
- Two full mock cycles with documented score progression
- Written exam-day plan, rehearsed before the real morning

Practice: Sit one full length timed mock this week and close your error log so every recurring mistake has a written, named fix.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Year-long error log and precalculus gap map
- Self-graded FRQ portfolio covering every archetype against real rubrics
- Series convergence decision tree and personal formula sheets (BC)
- Two full mock cycles with documented score progression
- Written exam-day plan rehearsed before May

Total projects: 5+ exam artifacts including full mock cycles

SKILLS THE STUDENT WILL HAVE AT THE END

- Limits, continuity and the derivative as ideas
- The full derivative toolkit at exam speed
- Curve analysis, optimization and motion
- Integrals, the FTC and accumulation reasoning
- Differential equations and slope fields
- Series, parametrics and polar (BC)
- FRQ rubric craft and justification writing
- MCQ pacing and calculator-section strategy

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes with your mentor
Problem sets: One curated set per week, reviewed in class
Timed work: Timed sections increasing in frequency toward May
Total per week: About 4-5 focused hours
Homework: A problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week
Mentorship: One dedicated mentor across the whole year
Recordings: Every class recorded for revision
Progress tracking: Error log + mock progression shared with parents
Flexibility: Intensity adjusts around school exams and mock weeks

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall
    short and there is focused revision and a free retest: the certificate certifies mastery, not
    attendance

WHERE THIS CAN LEAD

  - A 4-5 on AP Calculus, with the college-credit positioning that follows
  - Engineering, CS and quantitative-major readiness built on real understanding
  - A lighter, calmer first-year college math experience
  - Ready for the College Mathematics Masterclass or competition mathematics

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: Does this course cover AB or BC?
  A: Both, taught to your student's actual registration. The AB core is the shared spine; BC
  students additionally get series, parametrics, polar and the extra integration techniques in
  Phase 3 while AB students run deep review and extra mock cycles. The mentor aligns with your
  school's pacing so classwork and prep reinforce each other.

  Q: My student is strong in class but bombs timed practice. Can you fix that?
  A: Usually, yes, that profile is craft, not content. Timed collapse comes from slow routes,
  missing rubric awareness and pacing panic, all of which are trained skills. The FRQ rubric
  work plus personal pacing plans built from real timing data is exactly the fix.

  Q: Most of my student's lost points are algebra slips, not calculus. Does prep help?
  A: This is the most common pattern on AP exams, and it is very fixable. We track exactly
  where points die; when it is precalculus debt, simplification, composition, trig values, we
  repair those habits inside calculus problems, where the fix actually sticks.

  Q: When should AP Calculus prep start?
  A: The full 10-month arc starts alongside the school year and peaks in May. Joining mid-year
  works too, the mentor compresses toward your student's gap list and the mock cycles. The
  honest note: series (for BC) and FRQ craft reward runway, so earlier is calmer.

  Q: What score can we expect?
  A: We will not invent one. What you get instead: a real diagnostic, a plan aimed at the
  highest-value gaps, and a documented mock progression so the trend is visible long before
  May. Most committed students who complete the mock cycles walk in targeting 4-5 with evidence
  behind the target.

  Q: How is this different from the school's AP class?
  A: It is 1-on-1 or a tight small group, so your student's specific errors get the airtime a
  class of thirty cannot give. School covers the syllabus; we make sure your student owns the
  ideas, and then train the exam itself, rubrics, pacing, calculator strategy, which most
  classrooms have no time for.

  Q: How am I assessed, and is the certificate just for showing up?
  A: No. A problem set follows every class with written feedback, monthly reviews deliberately
  mix new material with earlier topics so nothing is forgotten, and the course ends with a full
  mock AP-style paper as the final assessment. Passing earns the certificate; falling short
  earns focused revision and a free retest. We measure real understanding, not attendance.

  Q: Can we try it before paying?
  A: Yes. Every student starts with a free live demo class that doubles as the diagnostic, a
  real lesson plus an honest read on the starting point, with no card details required.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ap-calculus-exam-prep
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
COURSE 18 OF 137: AP COMPUTER SCIENCE A: JAVA PROGRAMMING AND FULL EXAM PREP
==

Course link: https://learn.modernagecoders.com/courses/ap-computer-science-a-java-exam-prep-course
Subject area: Advanced Placement Computer Science
Designed for: Teens
Who can join: High school students preparing for AP Computer Science A; taught from Java basics, no prior coding required
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 4-5 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150

Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online AP Computer Science A prep taught to the 2025-26 redesign: Java from scratch, all four units, free-response mastery and full digital mock exams.

In one line: A real Java programming course that happens to end in a top exam score, taught to the 2025-26 redesign that most courses have not caught up to yet.

ABOUT THE PROGRAMME

AP Computer Science A is a real first course in programming, taught in Java, and this course treats it that way: you leave able to write Java, not just to pass a test. It is also built for the exam as it exists now, not as it existed last year. The College Board redesigned AP CS A for 2025-26, collapsing the old ten units into four, moving the whole exam onto the digital Bluebook platform, and changing what is tested. Inheritance, polymorphism, interfaces and writing recursive methods were removed. Reading text files with Scanner, working with data sets, and tracing recursive calls were added. A course still teaching the ten-unit format with an inheritance unit is preparing students for an exam that no longer exists. The six-month structure follows the four redesigned units in order. Months 1 and 2 build Java from nothing and cover Unit 1, using objects and methods. Months 3 and 4 cover selection and iteration, then class creation, which is exactly what the class-design free-response question asks for. Months 5 and 6 cover data collections, arrays, ArrayList, 2D arrays, file reading and data sets, then turn fully to exam craft: all four free-response question types drilled against the real rubric, multiple-choice strategy for the forty-two-question digital section, and full timed mock exams under Bluebook-style conditions. Every class is live and small, and every concept is practised by writing code, because that is the only way this subject is ever actually learned.

WHAT MAKES THIS COURSE DIFFERENT

- Taught to the 2025-26 redesign: four units, the fully digital Bluebook exam, and the current content, so no time is wasted on inheritance, polymorphism or writing recursion, all of which were removed
- The added topics are covered properly: reading text files with Scanner, working with data sets, and tracing recursive calls, which many older courses and books still omit
- A real Java course, not a cram sheet: students finish able to read and write Java, which is the point of a first programming course
- Free-response mastery against the actual rubric: all four question types, drilled until earning points is a habit, with partial credit understood
- Digital-exam readiness: multiple-choice pacing for the forty-two-question section and full mock exams run under Bluebook-style conditions, so test day holds no surprises
- Live, small batches where you write code during class and get it reviewed, not lecture videos you watch alone

THE LEARNING PATH, STEP BY STEP

Phase 1: Java from scratch and Unit 1: primitive types, expressions, Strings, the Math class, and using objects and methods correctly
Phase 2: Units 2 and 3: boolean logic, conditionals and loops, then writing your own classes with constructors, methods and encapsulation
Phase 3: Unit 4 and exam mastery: arrays, ArrayList, 2D arrays, files and data sets, then all four free-response types and full timed digital mock exams

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Comfortable school algebra: variables, expressions and basic logic. No advanced maths is required for AP Computer Science A
Programming: None required. The course teaches Java from the first line, so a complete beginner can start here; prior block-based or Python experience helps but is not assumed
Equipment: A computer that can run Java and a modern browser, since the real exam and our mock exams use a digital testing environment. We help with the full setup in week 1
Audience: High school students preparing for AP Computer Science A, typically in the years leading up to the exam, whether taking it at school or self-studying

WHO THIS COURSE IS FOR

Ap students: Students taking AP Computer Science A at school who want a structured, current course and real code review alongside their class
Self studiers: Students whose school does not offer AP CS A and who are preparing for the exam independently
Score focused: Students aiming for a strong score who want free-response drilling against the real rubric and full timed mock exams
Cs bound: Students headed for a computer science or engineering degree who want a genuine first programming course, not just exam tricks
International students: International students sitting AP for university applications who

need the exam taught to its current, digital, 2025-26 format

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: JAVA FOUNDATIONS AND USING OBJECTS (UNIT 1, MONTHS 1-2, WEEKS 1-8)

Java built from nothing, then Unit 1 of the redesigned course: using objects and methods, the largest single source of early marks and the base everything else stands on.

Month 1: Java From the Ground Up (Weeks 1-4)

Setup, the Java Program, and Output

Topics:

- Installing Java and a clean editor setup, plus a first look at the Bluebook exam environment
- The anatomy of a Java program: class, main method, statements and semicolons
- Printing with System.out.print and println, and reading output carefully
- Comments, whitespace and why readable code earns marks in free response
- The compile-and-run cycle and reading your first error messages without fear
- The AP Java Quick Reference: what it gives you on the exam and how to use it

Projects built:

- First program: a short Java program that prints a formatted pattern, run and explained line by line

Practice: 10 short output-tracing exercises: predict what a snippet prints, then run it and reconcile any difference

Primitive Types, Variables, and Expressions

Topics:

- The primitive types AP tests: int, double and boolean
- Declaration, initialisation and assignment
- Arithmetic operators, and the two that trip everyone: integer division and modulus
- Operator precedence and using parentheses to mean what you intend
- Casting between int and double, and where rounding silently happens
- Compound assignment and increment operators read correctly

Projects built:

- Calculator methods: small programs computing averages, remainders and unit conversions, each predicting integer versus double results first

Practice: 16 expression-evaluation problems, half of them designed around the integer-division and modulus traps

Using Objects and the String Class

Topics:

- Objects versus primitives, and the reference model that explains so many exam questions
- Creating objects and calling methods on them
- String concatenation and building output
- The tested String methods: length, substring, indexOf, equals and compareTo
- Why equals and == are different for objects, a guaranteed exam theme
- Reading a method's header to know what it needs and what it returns

Projects built:

- String toolkit: methods that extract initials, reverse words and search text, using only the AP-tested String methods

Practice: 14 String problems worked against the Quick Reference, each solution using only methods the exam actually provides

The Math Class and Method Basics

Topics:

- The Math methods AP tests: abs, pow, sqrt and random
- Turning Math.random into an integer in a target range, a classic exam skill
- Calling methods: arguments, parameters and return values
- void methods versus value-returning methods
- Method signatures and matching a call to a header
- Tracing a program that hands values between several methods

Projects built:

- Dice and ranges: a program that generates random values in given ranges and verifies the distribution by counting

Practice: 12 mixed problems on Math and method calls, plus 4 random-range questions built the way the exam builds them

Assessment: Month 1 checkpoint: a timed set on types, expressions, Strings and methods, marked on accuracy and correct tracing

Month 2: Unit 1 Depth, Using Objects and Methods (Weeks 5-8)

Wrapper Classes and Number Handling

Topics:

- Integer and Double wrapper classes and why they exist
- Autoboxing and unboxing seen in code you will meet later with ArrayList
- Parsing numbers and Integer.MAX_VALUE and MIN_VALUE
- Double precision surprises and why you never test doubles with ==
- Choosing int versus double deliberately, not by habit
- Reading questions that hide a type mismatch

Projects built:

- Number-handling drill: a program that safely parses, compares and formats numeric input, with

the precision traps annotated
Practice: 12 problems on wrapper classes, parsing and precision, each with a one-line note on the type decision made

String Manipulation Deep Dive
Topics:
- Building, searching and comparing Strings at exam difficulty
- Common String algorithms: counting characters, extracting parts, reversing
- The off-by-one errors that live in substring and indexOf
- Traversing a String by index, a bridge to loops in Unit 2
- Comparing Strings correctly with equals and compareTo
- Writing a helper method that returns a processed String

Projects built:
- Text processor: methods that clean, search and transform text, each tested on tricky edge cases like empty and single-character strings

Practice: 14 String-algorithm problems, every solution checked against an empty-string and one-character edge case

Designing and Calling Methods
Topics:
- Decomposing a problem into small, named methods
- Helper methods and why they make free-response answers cleaner
- Method overloading, recognised and read correctly
- Tracing a chain of method calls and their return values
- Parameter passing in Java: what a method can and cannot change about what you pass it
- Reading a specification and turning it into method headers

Projects built:
- Method decomposition: one larger task solved twice, once as a single long method and once split into helpers, then compared for clarity

Practice: 12 method-tracing problems plus 4 parameter-passing questions built around the value-semantics trap

Unit 1 Mastery and the Digital Multiple Choice
Topics:
- Pulling Unit 1 together: objects, methods, Strings and the Math class
- The multiple-choice section as it is now: forty-two questions, choices A to D, ninety minutes
- Roughly two minutes a question, and how that changes pacing
- Reading code-snippet questions quickly and accurately
- Eliminating wrong answers on tracing questions
- A personal error log started here and grown all course

Projects built:
- First timed multiple-choice block on Unit 1 content, reviewed question by question with each miss tagged by cause

Practice: A timed Unit 1 multiple-choice set at exam pace, followed by a written review of every wrong and every guessed answer

Assessment: Unit 1 assessment: a timed multiple-choice section on using objects and methods, scored with the error log updated

---

PHASE 2: SELECTION, ITERATION AND CLASS CREATION (UNITS 2 AND 3, MONTHS 3-4, WEEKS 9-16)

The logic and loops that drive every program, then writing your own classes, which is exactly what the class-design free-response question demands.

Month 3: Selection and Iteration (Unit 2) (Weeks 9-12)

Boolean Expressions and if Statements
Topics:
- Boolean values and relational operators
- if, if-else and if-else-if chains
- Nested conditionals and reading them without getting lost
- Common logic errors and how the exam sets them
- Writing conditions that match a worded specification exactly
- Deciding between an if-else-if chain and separate ifs, and why it matters

Projects built:
- Decision engine: a grading or categorising program driven entirely by conditionals, tested against every boundary value

Practice: 14 conditional problems, each one required to be traced for at least three different inputs

Compound Conditions and De Morgan's Laws
Topics:
- The logical operators and, or and not
- Short-circuit evaluation and when the second condition is never checked
- De Morgan's laws and rewriting a condition as its equivalent
- Simplifying tangled boolean expressions
- Boolean-returning methods, a frequent exam and free-response pattern
- Truth-table thinking for questions that look harder than they are

Projects built:
- Condition simplifier: a set of gnarly boolean expressions rewritten in simplest equivalent form and verified by truth table

Practice: 12 compound-condition problems including 4 De Morgan rewrites checked both ways

While and For Loops

    Topics:
      - The while loop and its three moving parts
      - The for loop as a compact while, and converting between them
      - Counters, accumulators and running totals
      - Sentinel-controlled loops
      - Off-by-one errors and infinite loops, diagnosed on purpose
      - Counting exactly how many times a loop body runs, a guaranteed multiple-choice type
    Projects built:
      - Loop patterns: programs that sum, count, find and build using loops, each with the iteration count predicted before running
    Practice: 16 loop problems, half asking for the exact number of executions rather than the final value
  Nested Loops and String Traversal
    Topics:
      - Nested loops and how the inner loop multiplies the work
      - Traversing a String character by character with a loop
      - Building patterns and tables with nested iteration
      - Analysing nested-loop statement execution counts
      - Combining loops with the conditionals from earlier weeks
      - Recognising when a loop is the wrong tool
    Projects built:
      - Pattern generator: nested-loop programs that draw shapes and process text grids, with execution counts worked out by hand
    Practice: 12 nested-loop and String-traversal problems, execution counts required on the nested ones
    Assessment: Unit 2 assessment: a timed section on conditionals and loops, including statement-execution-count questions

Month 4: Class Creation (Unit 3) (Weeks 13-16)
  Anatomy of a Class
    Topics:
      - Instance variables and the state an object holds
      - Constructors and initialising an object correctly
      - Default values and the null reference
      - Creating objects with new and calling their methods
      - The this reference and when you need it
      - Reading a class specification the way a free-response question presents it
    Projects built:
      - First class from scratch: a small class with fields and a constructor, instantiated and exercised from a tester program
    Practice: 10 problems building constructors and instance variables from a written specification
  Methods, Encapsulation, and Access
    Topics:
      - Accessor and mutator methods done cleanly
      - private versus public and why encapsulation is tested
      - Returning and updating an object's state safely
      - Writing methods that use the object's own instance variables
      - Method contracts: honouring exactly what a specification promises
      - Common class-design mistakes that cost free-response points
    Projects built:
      - Encapsulated class: a class with private fields and a full set of methods, with a test proving the state can only change through methods
    Practice: 12 problems adding correct accessor, mutator and behaviour methods to given classes
  Static, Scope, and Designing to a Specification
    Topics:
      - static variables and methods, and how they differ from instance members
      - Local versus instance scope, a frequent source of exam confusion
      - Designing a whole class from a table of required behaviour
      - Choosing fields, constructors and methods that satisfy every listed case
      - Testing your own class against the specification before trusting it
      - How this maps directly onto free-response Question 2, class design
    Projects built:
      - Spec-to-class: a complete class built from a behaviour table alone, then checked case by case against that table
    Practice: 10 class-design problems worked from specification to tested class, scope errors flagged as they appear
  Unit 3 Mastery and the Class-Design Free Response
    Topics:
      - Writing a full class to specification under time
      - The Class Design free-response question and its rubric
      - Reading the specification table for exactly the points on offer
      - Where partial credit lives in a class-design answer
      - Self-checking an answer against the rubric before moving on
      - Handwriting and clarity habits that protect marks on a digital exam
    Projects built:
      - Full class-design response written to a released-style prompt, then scored against the rubric point by point

Practice: Two timed class-design responses, each self-scored on the rubric with the lost points named
Assessment: Unit 3 assessment: a timed section plus one class-design free response, scored on the official rubric

---
PHASE 3: DATA COLLECTIONS AND EXAM MASTERY (UNIT 4 AND FULL MOCKS, MONTHS 5-6, WEEKS 17-24)

Arrays, ArrayList, 2D arrays, files and data sets, then a full turn to exam craft: every free-response type drilled and complete timed digital mock exams.

Month 5: Data Collections (Unit 4) (Weeks 17-20)
One-Dimensional Arrays
Topics:
- Declaring, creating and initialising arrays
- Indexing, length and the out-of-bounds error
- Traversing an array with a standard for loop
- The enhanced for loop and when it can and cannot be used
- Array algorithms: minimum, maximum, sum, count and average
- The array traps the exam reuses, including modifying while traversing

Projects built:
- Array analyser: methods that scan an array for statistics and matches, each tested on empty and single-element arrays

Practice: 14 array problems spanning both loop styles, edge cases required on every algorithm

ArrayList
Topics:
- ArrayList versus arrays, and why the exam tests both
- The tested methods: add, get, set, remove and size
- Autoboxing in an ArrayList of Integer
- Traversing and searching an ArrayList
- The removal-during-iteration bug and the two correct fixes
- ArrayList algorithms that mirror free-response Question 3, data analysis

Projects built:
- Data-analysis methods: an ArrayList processed to filter, count and summarise, written the way the free response asks

Practice: 14 ArrayList problems, including two built specifically around the remove-while-iterating trap

Two-Dimensional Arrays
Topics:
- Declaring and initialising 2D arrays
- Row-major order and what it means for traversal
- Nested loops over a grid, rows then columns
- 2D array algorithms: row sums, column sums, searching and counting
- Common indexing mistakes on the second dimension
- How this maps onto free-response Question 4, the 2D array question

Projects built:
- Grid processor: methods that analyse a 2D array by row, column and whole grid, each traced on a small sample grid first

Practice: 12 two-dimensional array problems, a hand trace required before coding each solution

Files, Data Sets, and Recursion Tracing
Topics:
- Reading text files with Scanner, a topic added in the 2025-26 redesign
- Working with data sets: loading, storing and processing real rows of data
- Tracing recursive calls, which the current exam tests; you trace recursion, you do not write it
- Why inheritance, polymorphism and writing recursion are not on the redesigned exam, so we do not spend time on them
- Combining files, arrays and ArrayList into one small data program
- A clear map of exactly what Unit 4 does and does not require

Projects built:
- Mini data project: read a small text data set with Scanner, store it, and answer questions about it with array or ArrayList methods

Practice: 10 problems mixing file reading and data processing, plus 6 recursion-tracing questions that ask only for the traced result
Assessment: Unit 4 assessment: a timed section on arrays, ArrayList, 2D arrays, file reading and recursion tracing

Month 6: Free-Response Mastery and Full Mock Exams (Weeks 21-24)
Free Response 1 and 2: Methods, Control Structures and Class Design
Topics:
- Question 1, methods and control structures: writing two methods, or a constructor and a method, to spec
- Handling the String-method part cleanly in Question 1
- Question 2, class design, revisited under exam time
- Reading a rubric to see exactly where points are awarded
- The commonest point-losing mistakes on these two questions
- Writing code clean enough to be scored quickly and correctly

Projects built:
- Two timed responses, one Question-1 style and one Question-2 style, each scored against the official rubric

Practice: Timed free-response practice on Questions 1 and 2, every attempt self-scored with the lost points named

Free Response 3 and 4: ArrayList Data Analysis and 2D Arrays

Topics:
- Question 3, data analysis with an ArrayList, from spec to working method
- Question 4, the 2D array question, traversed and manipulated correctly
- The data-manipulation patterns these two questions reward
- Partial-credit strategy: securing points even when the full method is hard
- Time budgeting across all four free-response questions
- Checking a method against the given examples before you commit

Projects built:
- Two timed responses, one Question-3 style and one Question-4 style, scored against the rubric and compared to week 21

Practice: Timed free-response practice on Questions 3 and 4, with a written note on where partial credit was won or missed

Multiple-Choice Strategy and Full Mock Exam One

Topics:
- Pacing the forty-two-question digital multiple-choice section
- Fast, accurate code tracing under time
- Elimination and educated guessing when a trace stalls
- A first complete timed mock: multiple choice and free response together
- Running the mock under Bluebook-style digital conditions
- Scoring the mock and converting toward the 1 to 5 scale

Projects built:
- Full mock exam one, scored, with a written breakdown of multiple-choice versus free-response performance

Practice: The full timed mock plus a complete review, every missed multiple-choice question re-traced correctly

Full Mock Exam Two and Test-Ready Review

Topics:
- A second complete timed mock under exam conditions
- Comparing the two mocks for direction of travel
- Closing the last content gaps the mocks exposed
- The test-day digital routine: the Bluebook app, the Quick Reference, timing
- A realistic, evidence-based target score from your mock results
- Where to go next: college programming, data structures, or a first real project

Projects built:
- Full mock exam two, scored and reviewed, ending in a one-page test-day plan built from your own results

Practice: The second full mock plus targeted repair of any remaining weak topic

Assessment: Final assessment: a complete timed mock exam, a progress summary from week 1 to now, and certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A String toolkit built from only the AP-tested String methods
- An encapsulated class designed from a behaviour specification
- An array analyser tested on every edge case
- An ArrayList data-analysis method in the free-response style
- A 2D array grid processor
- A mini data project that reads a text data set with Scanner
- Eight scored free-response answers across all four question types
- Two full timed mock exams under digital conditions

Total projects: Dozens of small Java programs plus eight scored free responses and two complete timed mock exams, all mapped to the 2025-26 exam

SKILLS THE STUDENT WILL HAVE AT THE END

- Reading and writing real Java, not just recognising it
- Using objects and methods, the largest scored area of the redesigned exam
- Conditionals, loops and nested iteration with correct execution-count reasoning
- Designing and writing classes to a specification, the class-design free response
- Arrays, ArrayList, 2D arrays, file reading and recursion tracing under the new Unit 4
- Free-response point-earning and digital multiple-choice pacing on full timed mocks

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, writing and tracing Java code live rather than watching solutions scroll by

Practice: 4-5 hours weekly of coding problems, tracing exercises and, later, timed free-response and mock sections

Review: Homework and free responses reviewed with written feedback, scored against the official rubric so you see exactly where points are won and lost

Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

  Doubt support: WhatsApp doubt support between classes, with worked solutions for the problems that stall you
  Progress updates: Monthly progress notes tracking unit assessments and, in the final months, mock-exam scores against your target
  Career guidance: Honest guidance on what an AP score does and does not do for college admissions and credit, and what to study next in computer science

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

  - A strong footing for the AP Computer Science A exam and, through it, university applications
  - Readiness for a college introductory programming course, often the exact content AP CS A mirrors
  - A genuine first step toward a computer science, software engineering or data degree
  - The Java and problem-solving base for our more advanced coding courses, including data structures and algorithms
  - Programming and free-response habits that transfer to later coursework and technical interviews

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: Is this course taught to the new 2025-26 AP CS A exam?
  A: Yes, and this is the single most important thing to check in any AP CS A course right now. The College Board redesigned the course for 2025-26: the old ten units became four, the exam moved fully onto the digital Bluebook platform, and the content changed. Inheritance, polymorphism, interfaces and writing recursive methods were removed. Reading text files with Scanner, working with data sets, and tracing recursive calls were added. We teach the four-unit course and the digital exam as they are now. A course or textbook still built around a separate inheritance unit is preparing you for an exam that no longer exists.

  Q: Do I need to know Java or any programming before I start?
  A: No. The course starts at the very first line of Java and builds up, so a complete beginner is exactly who it is designed for. If you have done some block coding or a little Python, that background helps you move faster, but nothing is assumed. What the course does need is the practice hours: programming is learned by writing code, and the four to five hours a week of practice are where the real progress happens, not only in class.

  Q: What is the exam format now?
  A: The redesigned exam has two sections of equal length. Section 1 is multiple choice: forty-two questions with answer choices A to D, ninety minutes, and it is worth fifty-five percent of the score. Section 2 is free response: four questions, ninety minutes, worth forty-five percent. The four free-response questions are methods and control structures, class design, data analysis with an ArrayList, and a 2D array question. The whole exam is taken digitally in the Bluebook app, with the Java Quick Reference available on screen. We run full mock exams under these conditions so test day feels familiar.

  Q: Will this course get me a 5, or college credit?
  A: No honest course can promise a specific score. AP exams are graded 1 to 5, and your result depends on your starting point, the hours you put in and your performance on the day. What we can promise is honest: live teaching to the current exam, small batches, real doubt support, free-response drilling against the official rubric, and full timed mock exams, plus a clear, evidence-based read on a realistic target for you. Whether a given score earns college credit is decided by each university, not by us or by the College Board, so we will point you to check the specific policy of the colleges you care about.

  Q: What happened to inheritance and recursion?
  A: They changed in the redesign, and knowing this saves you real time. Inheritance, polymorphism, the extends and super keywords, interfaces, and writing recursive methods were all removed from the required content. Recursion is still on the exam, but only tracing: you follow recursive calls and state the result, you are not asked to write a recursive method. We teach recursion tracing properly, and we deliberately do not spend weeks on the removed topics, which is time we give back to the areas the exam actually tests, especially Unit 4 data collections, the largest scored unit.

  Q: Can I manage this alongside school and other subjects?
  A: Yes, it is built for a school student's week: two live one-hour classes plus four to five hours of practice you can place where they fit, with evening and weekend class slots. The pace is steady rather than crushing, and WhatsApp doubt support means a problem that stalls you on a school night does not have to wait for the next class. Students who miss a session

can usually attend the same class with another batch where scheduling allows, so a busy exam week does not leave a gap.

Q: How is the free response practised and marked?
A: The way it is actually scored. From month 4 onward you write timed free-response answers to released-style prompts, and each one is marked against the official College Board rubric, point by point, so you see exactly where marks are earned and lost. By the final month you will have written and reviewed several answers across all four question types. This rubric-first habit is what separates students who understand Java from students who reliably score on the free-response section, and the two are not the same thing.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Is this course affiliated with the College Board?
A: No. Modern Age Coders is an independent education provider and is not affiliated with, authorised by, or endorsed by the College Board. AP and Advanced Placement are registered trademarks of the College Board, used here only to describe the exam this course prepares students for. For official course and exam information, students should always refer to the College Board and AP Central directly.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ap-computer-science-a-java-exam-prep-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 19 OF 137: AP COMPUTER SCIENCE PRINCIPLES: CREATE TASK AND EXAM PREP
==========

Course link: https://learn.modernagecoders.com/courses/ap-computer-science-principles-exam-prep-course
Subject area: Advanced Placement Computer Science
Designed for: Teens
Who can join: High school students preparing for AP Computer Science Principles; beginner-friendly, no prior coding required
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online AP Computer Science Principles course: programming fundamentals, data, algorithms, the Create performance task, and full digital exam prep for CSP.

In one line: A broad, honest introduction to computing, built for the exam and the Create task exactly as they exist right now.

ABOUT THE PROGRAMME

AP Computer Science Principles is deliberately the wide-angle course: instead of teaching one language deeply the way AP Computer Science A teaches Java, CSP is organised around five Big Ideas, Creative Development, Data, Algorithms and Programming, Computer Systems and Networks, and Impact of Computing, and it is language-flexible by design. Students may build in a block-based or a text-based language, and this course teaches with a beginner-friendly, text-based language while also teaching fluent reading of the College Board's own pseudocode, since exam questions are written in that pseudocode regardless of what a student builds with. What matters on the exam is the concept underneath, a loop, a list, a procedure with parameters, not which keyword a particular language happens to use for it. The course is also built for the assessment as it currently works, which changed in 2024 in a way many older courses have not caught up to. Students still spend at least nine hours of guided development time building an individually developed program with a list, a procedure that takes parameters, and an algorithm that manages complexity with selection or iteration, the Create Performance Task, and they still submit their code as a Personalized Project Reference PDF through the AP Digital Portfolio near the end of April. What changed is that this submission is no longer scored on its own by AP readers. Instead, on the end-of-course exam in May, students answer two written-response questions about their own program using that same Personalized Project Reference as their permitted reference material. Section I, seventy multiple-choice questions in one hundred twenty minutes, is worth seventy percent of the score. Section II, the two written responses about the student's own code, is worth thirty percent. The six-month structure follows this directly: Months 1 and 2 build Creative

Development and core programming; Months 3 and 4 cover Data and deeper Algorithms and Programming while the Create Task is planned, built, and finished; Months 5 and 6 cover Computer Systems and Networks and Impact of Computing, then turn fully to full-length multiple-choice practice and drilling the two written responses against the student's own finished project, ending in complete timed mock exams under digital conditions.

WHAT MAKES THIS COURSE DIFFERENT

- Taught to the exam as it is scored now: the 2024 change to the Create Performance Task, no longer a separately scored artifact but the reference material for two written-response questions on the exam itself, is built into the course from month one
- Genuinely language-flexible teaching: concepts are taught to transfer, and students also learn to read the College Board's own pseudocode fluently, since that is the language most multiple-choice questions are actually written in
- A real, individually developed Create Task, built over guided sessions across month 4 with instructor feedback, not assembled the week before the deadline
- All five Big Ideas covered in proportion to how the exam actually weights them, so time is not lost over-teaching one area and under-teaching another
- Written-response practice done the way the exam requires it: against the student's own finished program and their own Personalized Project Reference, not a generic prompt
- Live, small batches where every class involves writing or reading real code, not lecture slides watched alone

THE LEARNING PATH, STEP BY STEP

Phase 1: Creative Development and programming foundations: variables, lists, selection, iteration, and procedures, built alongside the habits of incremental development, testing, and choosing a Create Task idea
Phase 2: Data and deeper Algorithms and Programming, while the Create Performance Task itself is planned, built across guided sessions, tested, and finished with its Personalized Project Reference
Phase 3: Computer Systems and Networks and Impact of Computing, then full seventy-question multiple-choice practice, written-response drilling against your own code, and complete timed digital mock exams

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Comfortable school-level arithmetic and logical reasoning. No calculus or advanced maths is required for AP Computer Science Principles
Programming: None required. The course is beginner-friendly and builds programming concepts from the first class; CSP is intentionally language-flexible, and no prior experience is assumed
Equipment: A computer that can run our teaching language and a modern browser, since the real exam and our mock exams use a digital testing environment. We help with the full setup in week 1
Audience: High school students preparing for AP Computer Science Principles, typically as their first AP computer science course, whether taking it at school or self-studying

WHO THIS COURSE IS FOR

Ap students: Students taking AP Computer Science Principles at school who want a structured, current course and real code review alongside their class
Self studiers: Students whose school does not offer AP CSP and who are preparing for the exam independently
First ap course: Students choosing CSP as their first AP computer science course, since it is broader and more forgiving of a standing start than AP Computer Science A
Curious beginners: Students who are not yet sure programming is for them and want an honest, low-pressure introduction to what computing actually involves
International students: International students sitting AP for university applications who need the exam taught to its current, digital, two-part format

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: CREATIVE DEVELOPMENT AND PROGRAMMING FOUNDATIONS (BIG IDEA 1 AND CORE OF BIG IDEA 3, MONTHS 1-2, WEEKS 1-8)

Computational thinking, the core programming building blocks CSP tests, and the habits of incremental development, ending with a planned and approved Create Task idea.

Month 1: Computational Thinking and Programming Building Blocks (Weeks 1-4)

Course Map, Computational Thinking, and What CSP Actually Tests

Topics:

- What AP Computer Science Principles covers: the five Big Ideas, Creative Development, Data, Algorithms and Programming, Computer Systems and Networks, and Impact of Computing, and how this course maps to them
- How CSP differs from AP Computer Science A: a broad, language-flexible course about computing itself, not a single-language programming course

- The two-part assessment at a glance: a seventy-question multiple-choice exam and a written-response section built around a program you build yourself
- Computational thinking as a habit: decomposition, abstraction, and pattern recognition, not just vocabulary to memorise
- A first look at the language and environment we build in, and why the concept matters more than the exact keyword a language uses for it
- Setting up your development environment and a running course notebook, where every future Create Task idea starts getting written down

Projects built:
- First program: a short interactive program that takes input and produces output, run and explained line by line

Practice: 10 short exercises identifying which Big Idea a given exam-style question is testing, plus a first working program written and run

Variables, Data Abstraction, and Expressions

Topics:
- Variables as named, reusable storage, and why naming them clearly is a skill the exam rewards directly
- Data types CSP cares about: numbers, text, and Boolean values, and how the College Board's own pseudocode represents each
- Assignment versus comparison, and expressions that combine arithmetic and logic correctly
- Data abstraction: hiding detail behind a clear name so a program stays readable as it grows
- Lists as CSP's core data structure: creating a list and accessing an item by its index
- Reading and predicting the output of short CSP-style pseudocode blocks, the single most common multiple-choice question shape

Projects built:
- Variable tracker: a short program that stores, updates, and reports several related values, traced by hand before it is run

Practice: 14 pseudocode-tracing problems written to the CSP Exam Reference Sheet conventions, each paired with equivalent code in our teaching language

Lists and Iteration

Topics:
- Traversing a list with a loop, the single most tested list pattern on the exam
- FOR EACH style iteration versus counting loops, and reading both forms in CSP pseudocode
- Building a list up over time: appending, inserting, and removing items
- Searching a list: finding a matching value and correctly handling the case where none exists
- Common list bugs: off-by-one errors and changing a list while iterating over it
- Using a list to model something real, such as scores, names, or sensor readings

Projects built:
- List toolkit: procedures that sum, count, filter, and find the maximum of a list, tested on an empty list and a single-item list

Practice: 12 list and iteration problems, half written in CSP pseudocode and translated, half written directly in code

Selection, Boolean Logic, and Procedures

Topics:
- Boolean expressions and the relational operators the exam tests
- IF, IF-ELSE, and nested selection, read in both CSP pseudocode and our teaching language
- AND, OR, and NOT, and reasoning carefully through compound conditions
- Defining a procedure with parameters and a return value
- Why breaking a program into procedures is itself a Creative Development skill, not just tidy style
- Combining selection, iteration, and procedures into one small, working program

Projects built:
- Decision-making procedure: a small program that classifies input using nested conditionals, tested against every boundary case

Practice: 16 problems mixing selection and procedure tracing, pitched at the same difficulty as the exam's code-reading questions

Assessment: Month 1 checkpoint: a timed set covering variables, lists, selection, and procedures, scored the way Section I multiple choice is scored

Month 2: Algorithms, Procedural Abstraction, and Planning the Create Task (Weeks 5-8)

Algorithms: Combining Sequencing, Selection, and Iteration

Topics:
- What an algorithm means in CSP's terms: a precise sequence of steps that solves a problem or a whole class of problems
- Combining sequencing, selection, and iteration together to express a real algorithm
- Simulating an algorithm by hand on paper before trusting it in code
- Search: a first look at how a program finds something inside a list
- Sort: the idea of putting data in order, without needing to memorise every sorting method by name
- Recognising an algorithm's purpose from CSP-style pseudocode alone, a guaranteed multiple-choice skill

Projects built:
- Algorithm workbook: a search and a sort implemented from a plain-language description, each hand-traced on a small list first

Practice: 12 algorithm-tracing problems in CSP pseudocode, each requiring a predicted output before running the equivalent code

Procedural Abstraction and Parameters

Topics:
- Procedural abstraction: naming a block of steps so it can be reused without re-reading it

every time
- Parameters as a procedure's inputs, and return values as its output
- Building a program from smaller, individually tested procedures instead of one long block
- Libraries and built-in procedures: using code you did not write, correctly and with understanding
- How the CSP Exam Reference Sheet documents procedures, and reading a procedure header quickly
- Documenting what a procedure does so a reader does not have to trace it line by line to understand it

Projects built:
- Procedure library: a small set of reusable procedures for a chosen task, called from one main program and tested independently

Practice: 10 problems designing a procedure header from a plain-language specification, plus 4 tracing problems on procedure calls

The Development Process: Incremental Building, Testing, and Collaboration

Topics:
- Incremental and iterative development: build a little, test a little, the exact process Creative Development questions test
- Identifying and fixing errors, and the real difference between a syntax error and a logic error
- Testing with a range of inputs, including edge cases that break naive solutions
- Collaboration in program development, and describing working with others honestly the way CSP expects
- Program documentation: comments that explain why a choice was made, not only what a line does
- Keeping a running development log, the exact habit month 4 relies on for the Create Task

Projects built:
- Debug and repair: a deliberately broken program fixed step by step, with every bug and its fix logged in writing

Practice: 8 debugging exercises with planted logic errors, plus a short written reflection on the development process used

Choosing and Planning Your Create Task Idea

Topics:
- What the Create Performance Task actually requires: an individually developed program with at least one list, a procedure that takes parameters, and an algorithm using selection or iteration to manage complexity
- Choosing an idea you can build and explain in your own words, not one that looks impressive but that you cannot honestly walk through
- Scoping a project to roughly nine hours of hands-on development time, the College Board's own minimum
- Sketching the program's purpose, inputs, and outputs before writing a line of code
- Planning exactly where the required list, procedure, and algorithm will live in your design
- Instructor sign-off on your Create Task idea before month 3's Data content and month 4's build sessions begin

Projects built:
- Approved Create Task proposal: a one-page plan naming your program's purpose, its list, its procedure with parameters, and its core algorithm

Practice: A written self-check against the official Create Task requirements, item by item, before the idea is approved

Assessment: Unit assessment: a timed multiple-choice set on Creative Development and the core programming content, reviewed question by question

---

PHASE 2: DATA, DEEPER ALGORITHMS, AND BUILDING THE CREATE TASK (BIG IDEAS 2 AND 3, MONTHS 3-4, WEEKS 9-16)

How computers represent and use data, then a return to algorithms with more complex data, while the Create Performance Task itself is planned, built across guided sessions, and finished.

Month 3: Data and Information (Weeks 9-12)

Binary and How Computers Represent Data

Topics:
- Binary numbers: converting between binary and decimal by hand
- Why computers store everything, numbers, text, images, and sound, as binary underneath
- Bits, bytes, and the sizes CSP expects a student to reason about
- Representing text with character encodings, and images as grids of coloured bits
- Analog versus digital data, and what is gained and lost when something is digitised
- Reading CSP multiple-choice questions that test binary conversion directly

Projects built:
- Binary explorer: a program that converts numbers between binary and decimal and explains each step of the conversion

Practice: 14 binary conversion problems plus 6 conceptual questions on the digital representation of text, image, and sound

Data Compression

Topics:
- Lossless compression: preserving every bit of the original data on the way back out
- Lossy compression: trading some detail for a much smaller file
- Where each type is actually used, and why source code and financial data are never compressed lossily
- A simple compression scheme worked by hand to see the size trade-off directly
- Metadata: what it is and why it travels with compressed and uncompressed data alike

- Compression questions the way CSP tends to ask them: scenario-based, not formula-based
Projects built:
- Compression demo: a small program that compresses a repetitive data set with a simple scheme and reports the size saved
Practice: 10 scenario questions on choosing lossless versus lossy compression for a stated real-world use

Extracting Information from Data
Topics:
- Why data alone is not information, and the extraction step that sits between the two
- Filtering, sorting, and searching a large data set for a specific answer
- Combining two data sets to surface a relationship that neither shows on its own
- Using tools to explore a data set too large to read by hand
- Visualising data honestly, echoing the same cautions taught in our data-focused courses
- Drawing a defensible conclusion from a data set, and stating plainly what that conclusion cannot claim
Projects built:
- Data-set investigation: a provided data set explored with filtering and grouping to answer three written questions with cited evidence
Practice: 8 data-extraction problems on real-style tables, each answer required to cite which rows or columns support it

Using Data to Solve Problems and Simulations
Topics:
- Simulations as simplified models of a real or imagined process
- What a simulation can tell you, and what it cannot, since it is only ever a model
- Building a simple simulation and running it many times to see a pattern emerge
- Bias in data, and bias in the people who select or interpret it
- How Data-focused questions typically read on the exam: a short scenario, then a reasoning question
- Pulling the month together: binary, compression, extraction, and simulation as one connected story about data
Projects built:
- Simple simulation: a small program that models a repeated random process, for example a game of chance, and reports results across many runs
Practice: A timed set of Data-focused multiple-choice questions, reviewed with the reasoning written out for every miss
Assessment: Unit assessment: a timed multiple-choice section on data representation, compression, extraction, and simulation

Month 4: Robust Programs and Building the Create Task (Weeks 13-16)

Two-Dimensional Lists and Algorithmic Thinking
Topics:
- Lists of lists: representing a grid or a table of data
- Traversing a two-dimensional list with nested iteration
- Choosing a data structure that fits the problem, not just the one you already know best
- A first, intuitive look at algorithm efficiency: why some approaches slow down badly as the data grows
- Undecidable problems: an honest look at problems no algorithm can solve correctly for every possible input
- Reading exam questions that compare two different algorithms solving the same problem
Projects built:
- Grid processor: a program that stores and analyses a small two-dimensional data set, for example a seating chart or a simple game board
Practice: 10 two-dimensional list problems plus 4 short-answer questions comparing the efficiency of two approaches to the same task

Simulations, Abstraction, and Managing Complexity
Topics:
- Abstraction as the tool that lets a program grow without becoming unreadable
- Managing complexity with procedures, parameters, and well-named variables used together
- Building a slightly larger simulation than week 12's, with more moving parts
- Deciding what detail a simulation can safely leave out without breaking its usefulness
- Refactoring a working but messy program into a cleaner one without changing what it does
- Applying these same habits directly to your own Create Task program, starting this week
Projects built:
- Create Task build session one: the first working version of your program's core algorithm and required list, reviewed live with the instructor
Practice: A self-review of your in-progress Create Task code against the list, procedure, and algorithm requirements

Building the Create Task: Robust Programs and Instructor Feedback
Topics:
- Writing a procedure with parameters that genuinely does the work your program needs, not one added only to meet a checklist
- Testing your Create Task program against both typical and edge-case inputs
- Handling unexpected input without the program crashing, a real mark of a robust program
- Explaining your own code out loud in class, the exact skill the exam later asks for in writing
- Incorporating instructor feedback into a second, improved working version
- Keeping your development log current: what changed, why it changed, and what you tested afterward
Projects built:

        - Create Task build session two: a tested, working version of your full program, incorporating the feedback from session one
      Practice: A written walkthrough of your program explaining what your list, procedure, and algorithm each do and why you built them that way
    Completing the Create Task and the Personalized Project Reference
      Topics:
        - Finalising your Create Task program: last fixes, clear comments, and a clean final run
        - Producing your Personalized Project Reference, the PDF of your own code you are permitted to use on exam day
        - What changed in 2024: the Create Task is no longer submitted as a separately scored artifact; instead your Personalized Project Reference becomes the material you use on the exam itself to answer two written-response questions about your own program
        - Submitting through the AP Digital Portfolio, and the real deadline this falls near, around the end of April on a typical school-year calendar
        - A first practice attempt at explaining your program in writing, the same task the exam itself will ask for
        - Why building a program you can actually explain matters more than building one that merely runs without errors
      Projects built:
        - Completed Create Task: a finished, tested program with its Personalized Project Reference PDF prepared exactly as exam day requires
      Practice: Two practice written responses about your own program, self-checked against the kind of reasoning the real prompts reward
      Assessment: Create Task checkpoint: instructor review of the finished program, its documentation, and a first written explanation of your own code

---
PHASE 3: COMPUTER SYSTEMS, IMPACT OF COMPUTING, AND EXAM MASTERY (BIG IDEAS 4 AND 5 AND FULL PRACTICE, MONTHS 5-6, WEEKS 17-24)

How the internet and larger computing systems actually work, an honest look at computing's effects on real people, then a full turn to exam craft built around your own finished Create Task.

  Month 5: Computer Systems and Networks (Weeks 17-20)
    How the Internet Works
      Topics:
        - The internet as a network of networks, not a single thing that lives anywhere
        - IP addresses and how a message finds its way to the right destination
        - Packets: breaking a message into pieces that can travel by different paths
        - Protocols as the agreed rules that let different systems talk to each other reliably
        - Bandwidth and latency, and why a fast connection is not the same thing as a low-delay one
        - Reading a CSP question about internet infrastructure for the exact concept being tested
      Projects built:
        - Packet-trace exercise: a written walkthrough of how a message would be broken into packets, routed, and reassembled between two points
      Practice: 10 conceptual questions on IP addressing, packets, and protocols, each answered in one or two precise sentences
    Fault Tolerance and Parallel Computing
      Topics:
        - Redundancy: why the internet keeps working even when a piece of it fails
        - Fault tolerance as a deliberate design goal, not an accident of good luck
        - Sequential versus parallel computing, and where each is genuinely the right choice
        - Distributed computing: splitting one large problem across many machines
        - Speedup, and why doubling the number of machines rarely halves the time exactly
        - Scenario questions that ask you to identify a fault-tolerant or a parallel design
      Projects built:
        - Parallel-versus-sequential demo: a small program run two ways, timed and compared, with the result explained in writing
      Practice: 8 scenario questions distinguishing genuinely fault-tolerant designs from a single point of failure
    Cybersecurity Fundamentals
      Topics:
        - What makes a system a worthwhile target: the value of the data it holds, not just its size
        - Common attacks explained plainly: phishing, malware, and denial-of-service, at the level CSP tests
        - Symmetric encryption: one shared key, and where that approach breaks down
        - Public-key encryption: how two sides can agree on a secret without ever meeting first
        - Authentication: passwords, multi-factor methods, and why each adds a genuine barrier
        - Practical safe-computing habits worth keeping regardless of the exam
      Projects built:
        - Encryption walkthrough: a simple substitution-style cipher implemented and then explained alongside why real systems need something far stronger
      Practice: 10 cybersecurity scenario questions, each requiring the specific vulnerability or protection to be named
    Computer Systems Review and Practice
      Topics:
        - Pulling the month together: the internet, fault tolerance, parallel computing, and cybersecurity as one connected system
        - Common traps in this area: mixing up a protocol with a specific piece of hardware

- Practising the reading-passage multiple-choice questions, five of the seventy on the real exam
- Timing strategy for the systems and networks questions specifically
- Building a personal glossary of systems terms from the errors made this month
- A short look ahead at Impact of Computing, which closes out the exam's content

Projects built:
- Systems glossary: a personal reference of every systems term missed this month, defined in your own words

Practice: A timed multiple-choice set on computer systems and networks content only, reviewed with every miss traced to its specific concept

Assessment: Unit assessment: a timed multiple-choice section on computer systems and networks, including reading-passage-style questions

Month 6: Impact of Computing and Full Exam Mastery (Weeks 21-24)

Beneficial and Harmful Effects of Computing

Topics:
- Computing innovations as double-edged: the same tool serving different purposes for different people
- The digital divide: unequal access to computing, and what that actually costs the people on the wrong side of it
- Bias in algorithms: where it enters, from the data going in to the decisions coming out
- Case studies of real computing innovations, weighed honestly for who benefits and who is put at risk
- Safe computing practices: privacy settings, a personal data footprint, and informed consent
- Writing a balanced answer that names both a genuine benefit and a genuine risk, the exact shape these questions want

Projects built:
- Impact case study: one real computing innovation analysed in writing for its benefits, its risks, and who is affected by each

Practice: 6 short-answer questions each requiring one genuine benefit and one genuine harm for a given computing innovation

Intellectual Property, Open Source, and Full Multiple-Choice Practice

Topics:
- Intellectual property: copyright, and why code can be protected the same way other creative work is
- Open source and Creative Commons licensing, and how each differs from default copyright
- Crowdsourcing and collaborative projects as a computing-enabled way of working
- Legal and ethical questions CSP expects you to reason through, not just define
- Starting full-length, seventy-question multiple-choice practice sets under timed conditions
- Building the error log that will drive the final two weeks of review

Projects built:
- First full multiple-choice practice set: seventy questions in one hundred twenty minutes, scored and logged by Big Idea

Practice: The full timed multiple-choice set plus a written review of every question missed, grouped by which Big Idea it tested

Written-Response Drilling with the Personalized Project Reference

Topics:
- The written-response section in full: two required questions about your own program, answered using your Personalized Project Reference
- What each written-response question actually asks for, and how it is scored
- Practising with your own finished Create Task code, exactly as the real exam requires
- Multiple-choice pacing refined: fifty-seven single-select, eight multiple-select choose-two, and five reading-passage questions in the mix
- Strategies for the multiple-select questions specifically, since a partial answer does not earn partial credit there
- A second full multiple-choice practice set, timed, with pacing tracked question by question

Projects built:
- Two scored written responses about your own Create Task program, written exactly as the real exam requires and reviewed against the reasoning the rubric rewards

Practice: The second full timed multiple-choice set, plus two written-response drafts scored and revised

Full Mock Exam and Test-Ready Review

Topics:
- A complete timed mock exam: seventy multiple-choice questions plus both written-response questions, run under digital conditions
- Reviewing the mock section by section, multiple choice against written response
- Closing the last content gaps the mock exposed
- The test-day routine: the Personalized Project Reference on hand, digital exam navigation, and timing across both sections
- An honest, evidence-based look at where you stand across all five Big Ideas
- Where to go next: AP Computer Science A, a first real coding course, or building further on the Create Task itself

Projects built:
- Full mock exam, scored across both sections, ending in a one-page test-day plan built from your own results

Practice: The complete mock exam plus targeted repair of the single weakest Big Idea it revealed

Assessment: Final assessment: a complete timed mock exam covering both the multiple-choice and written-response sections, plus a progress summary from week 1 to

now

PROJECTS THE STUDENT BUILDS AND KEEPS

- A list toolkit of procedures tested on an empty list and a single-item list
- An algorithm workbook implementing a search and a sort from plain-language descriptions
- A binary explorer that converts and explains number representations
- A data-set investigation answering written questions with cited evidence
- A finished, tested Create Performance Task program with its own required list, procedure, and algorithm
- A Personalized Project Reference prepared exactly as exam day requires
- An impact case study weighing a real computing innovation's benefits and risks
- A complete timed mock exam covering multiple choice and both written responses

Total projects: Dozens of small programs plus a completed Create Performance Task, an impact case study, two scored written responses, and a full timed mock exam, all mapped to the current CSP exam

SKILLS THE STUDENT WILL HAVE AT THE END

- Reading and writing real code built around variables, lists, selection, iteration, and procedures
- Designing, building, testing, and explaining an individually developed program, the exact demand of the Create Performance Task
- Reasoning about data: binary representation, compression, and drawing defensible conclusions from a data set
- Understanding how the internet, fault tolerance, and parallel computing actually work, at the depth the exam tests
- Weighing the beneficial and harmful effects of a computing innovation in writing, including bias and the digital divide
- Multiple-choice pacing and written-response habits drilled on full timed mock exams

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building and discussing real programs rather than watching slides
Practice: 3-4 hours weekly of coding exercises and pseudocode tracing, plus dedicated Create Task build time from month 4 onward
Review: Homework, the Create Task, and written responses reviewed with written feedback, scored the way the real exam scores each section
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked explanations for the problems that stall you
Progress updates: Monthly progress notes tracking unit assessments, Create Task milestones and, in the final month, mock-exam scores
Career guidance: Honest guidance on what an AP score does and does not do for college admissions and credit, and what to study next in computer science

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- A strong footing for the AP Computer Science Principles exam and, through it, university applications
- A natural next step into AP Computer Science A, our Java-based course, for students who want to go deeper into a single language
- A genuine first exposure to computing broad enough to inform a computer science, data, or engineering degree choice
- The programming and problem-solving base for our other coding courses, including Python and web development
- A completed, explainable programming project and an honest sense of what computing work actually involves, useful well beyond the exam itself

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: How is AP Computer Science Principles different from AP Computer Science A?
A: CSP is the broad, conceptual course. It covers five Big Ideas, Creative Development, Data, Algorithms and Programming, Computer Systems and Networks, and Impact of Computing, and it is language-flexible, so students may work in a block-based or a text-based language. AP Computer Science A, by contrast, is a single-language course taught entirely in Java, with no

data, internet, or impact content. Many students take CSP first, since it needs no prior coding background and gives an honest sense of whether programming and computing are worth going deeper into with AP Computer Science A afterward.

Q: Do I need any coding experience before I start?
A: No. The course is built to be beginner-friendly from the first class, and CSP itself is designed for students with no programming background. We teach the concepts, variables, lists, procedures, selection, and iteration, from scratch, alongside the College Board's own pseudocode conventions used in most exam questions. Some prior exposure to any language, block-based or text-based, helps you move faster, but nothing is assumed.

Q: What is the Create Performance Task, and how is it assessed now?
A: You submit your code as a Personalized Project Reference PDF through the AP Digital Portfolio, with a deadline that falls near the end of April on a typical school-year calendar. What changed starting with the 2024-25 exam is how this is scored: the Create Task is no longer graded as a standalone submitted artifact by AP readers. Instead, on the end-of-course exam itself, you answer two written-response questions about your own program, using your Personalized Project Reference as permitted reference material. This written-response section is Section II of the exam and is worth thirty percent of your score. We build and finish your Create Task across guided sessions in month 4, then drill exactly this written-response format against your own finished code in month 6.

Q: What programming language will I use?
A: CSP is intentionally language-flexible: the College Board allows both block-based and text-based languages for the Create Task, and does not test any one language's exact syntax on the multiple-choice exam. We teach with a beginner-friendly, text-based language for building real programs, including your Create Task, and we also teach fluent reading of the College Board's own pseudocode, since that is the notation most multiple-choice questions are actually written in.

Q: What is the exam format?
A: The exam has two sections, taken digitally. Section I is multiple choice: seventy questions in one hundred twenty minutes, worth seventy percent of the score. Section II is the written-response section: two questions about your own Create Task program, answered using your Personalized Project Reference, worth thirty percent of the score. We run full timed practice on both sections, including complete mock exams under digital conditions.

Q: Will this course get me a 5, or college credit?
A: No honest course can promise a specific score. AP exams are graded 1 to 5, and your result depends on your starting point, the hours you put into practice, and your performance on the day. What we can promise is honest: live teaching to the current two-part exam, a real, individually developed Create Task built with instructor feedback, full multiple-choice and written-response practice, and complete timed mock exams, plus a clear, evidence-based read on a realistic target for you. Whether a given score earns college credit is decided by each university individually, not by us or by the College Board, so check the specific policy of the colleges you care about.

Q: Can I manage this alongside school and other subjects?
A: Yes, it is built for a school student's week: two live one-hour classes plus three to four hours of practice you can place where it fits, with evening and weekend class slots. The pace stays steady rather than crushing, and WhatsApp doubt support means a problem that stalls you on a school night does not have to wait for the next class. Students who miss a session can usually attend the same class with another batch where scheduling allows, so a busy exam week does not leave a real gap.

Q: Is this course affiliated with the College Board?
A: No. Modern Age Coders is an independent education provider and is not affiliated with, authorised by, or endorsed by the College Board. AP and Advanced Placement are registered trademarks of the College Board, used here only to describe the exam this course prepares students for. For official course and exam information, students should always refer to the College Board and AP Central directly.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ap-computer-science-principles-exam-prep-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 20 OF 137: AP PRECALCULUS: UNITS 1 TO 3 AND FULL EXAM PREP

==========

Course link: https://learn.modernagecoders.com/courses/ap-precalculus-exam-prep-course
Subject area: Advanced Placement Mathematics
Designed for: High school
Who can join: High school students preparing for AP Precalculus; a solid Algebra 2 background is assumed
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 4-5 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online AP Precalculus exam prep covering Units 1-3: functions, exponentials, trig and polar. Small batches, real teachers, full timed mock exams.

In one line: A function-fluency course that happens to end in an AP exam, concentrated exactly where the exam is scored: Units 1 to 3.

ABOUT THE PROGRAMME

AP Precalculus is the newest AP mathematics course, and it is tested in a specific, slightly unusual way that changes how it should be prepared for. The classroom course covers four units, but the exam assesses only Units 1, 2 and 3. Unit 4, functions involving parameters, vectors and matrices, is taught across a full school year but does not appear on the exam at all. A prep course that spends real weeks drilling Unit 4 for exam purposes is spending time a student does not get back. This course concentrates its exam preparation exactly where the College Board tests: polynomial and rational functions, exponential and logarithmic functions, and trigonometric and polar functions. The six-month structure follows the three tested units in order, with the later weeks given fully to exam craft once each unit's content is covered. Months 1 and 2 build Unit 1: rates of change, polynomial behaviour and zeros, rational functions with their asymptotes and holes, transformations, and choosing the right model for a data set. Months 3 and 4 build Unit 2: arithmetic and geometric sequences, exponential functions and modelling, function composition and inverses, and logarithms through to equation solving. Month 5 builds Unit 3: the unit circle, sinusoidal functions and modelling, tangent and the reciprocal trig functions, inverse trig, and polar coordinates and functions. Month 6 turns fully to exam mastery: the calculator versus no-calculator skill that decides pacing on both sections, free-response method drilled against real question types, and two full timed mock exams run under the hybrid digital format, multiple choice answered in the Bluebook app and free response handwritten in paper booklets. A short, clearly labelled preview of Unit 4 closes the final week for students continuing on to calculus, with the honest note attached that it is not assessed on this exam. Every class is live and small, and the exam's calculator boundary, no calculator in Multiple Choice Part A, calculator required in Part B, calculator required again in Free Response Part A, then no calculator in Part B, is treated as a skill in its own right and drilled on both sides from week one, not left to the final month.

WHAT MAKES THIS COURSE DIFFERENT

- Exam prep concentrated exactly where the College Board tests: Units 1, 2 and 3. Unit 4, functions involving parameters, vectors and matrices, is taught in the full course but is not assessed on the exam, and this course does not spend exam-prep weeks pretending otherwise
- The calculator and no-calculator boundary is taught as its own skill from week one: Multiple Choice Part A with no calculator and Part B with one, Free Response Part A with a calculator and Part B without, drilled deliberately on both sides of the line
- A real function-fluency course, not a formula-memorisation cram sheet: transformations, end behaviour, equivalent representations and modelling are practised until they are habits, because that fluency is exactly what AP Calculus assumes on day one
- Free-response method drilled against real question types: modelling justification, equation solving and graph reading, with partial credit understood the way the rubric actually awards it
- Hybrid digital exam readiness: multiple-choice pacing for the Bluebook app and genuine handwriting practice for the paper free-response booklets, so the test-day format itself holds no surprises
- Live, small batches where problems are worked and reasoning is checked in class, not lecture videos watched alone

THE LEARNING PATH, STEP BY STEP

Phase 1: Unit 1: rates of change, polynomial functions and zeros, rational functions and asymptotes, transformations, and choosing the right function model
Phase 2: Unit 2: arithmetic and geometric sequences, exponential functions and modelling, function composition and inverses, logarithms and equation solving
Phase 3: Unit 3 and exam mastery: the unit circle, sinusoidal and trigonometric functions, polar coordinates and functions, then the calculator/no-calculator skill, free-response method, and full timed hybrid-digital mock exams

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: A solid Algebra 2 background: comfort with function notation, factoring, exponents and basic graphing. This course does not re-teach Algebra 2 from scratch, though real gaps are patched as they surface

Programming: None required and none used. This is a pure mathematics course; no coding background is assumed or needed, even though Modern Age Coders also teaches programming
Equipment: A graphing calculator of the kind allowed on the AP exam, for Multiple Choice Part B and Free Response Part A, plus a notebook for the no-calculator hand-work, and a device with stable internet for live classes. Since the real exam and our mock exams use the Bluebook digital platform, we help with the setup in week 1
Audience: High school students preparing for AP Precalculus, typically in the year before AP Calculus, whether taking the course at school or preparing independently

WHO THIS COURSE IS FOR

Ap students: Students taking AP Precalculus at school who want a structured, exam-focused course and real problem-solving alongside their class
Self studiers: Students whose school does not offer AP Precalculus and who are preparing for the exam independently
Score focused: Students aiming for a strong score who want free-response drilling against a real rubric and full timed mock exams under hybrid-digital conditions
Calculus bound: Students headed for AP Calculus AB or BC who want genuine function fluency, not just exam tricks, since that fluency is exactly what calculus assumes
International students: International students sitting AP exams for university applications who need Units 1 to 3 taught to the current hybrid digital format

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: POLYNOMIAL AND RATIONAL FUNCTIONS (UNIT 1, MONTHS 1-2, WEEKS 1-8)

Rates of change, polynomial behaviour and zeros, rational functions with their asymptotes and holes, transformations, and model selection: the function-behaviour toolkit every later unit builds on.

Month 1: Rates of Change and Polynomial Behaviour (Weeks 1-4)
Functions, Change, and Rates of Change
Topics:
- Function notation, domain and range refreshed from Algebra 2, plus a first look at the Bluebook digital exam environment
- Change in tandem: reading how one quantity's change drives another's from a table of values
- Average rate of change as the slope of a secant line over a stated interval
- Comparing rates of change across different intervals to describe where a function is increasing or decreasing
- Using a second layer of differences to notice when a rate of change is itself changing, an early signal of concavity
- Reading the same function's behaviour from a table, a graph and an equation, and checking that the three agree

Projects built:
- Rate-of-change notebook: a function studied three ways, table, graph and average rate of change over five different intervals, with each answer interpreted in a sentence

Practice: 16 rate-of-change problems worked from tables and graphs, half requiring the interval to be chosen from a written description
Polynomial Functions: Zeros and Local Behaviour
Topics:
- Polynomial vocabulary: degree, leading coefficient and standard form, read at a glance
- Zeros of a polynomial function and what multiplicity does to the graph at each one
- Distinguishing a zero where the graph crosses the axis from one where it only touches
- Local maxima and minima, and intervals of increase and decrease read from a polynomial's graph
- Complex zeros and the fact that non-real zeros of a real polynomial always come in conjugate pairs
- Sketching a polynomial from its zeros, their multiplicities and its degree, before ever plotting a point

Projects built:
- Zero-to-graph drill: eight polynomials sketched from a list of zeros and multiplicities alone, then checked point by point

Practice: 14 problems finding zeros, multiplicities and local behaviour, each sketch attempted before any calculator is used
End Behaviour and Rational Functions
Topics:
- End behaviour of a polynomial function read directly from its degree and leading coefficient
- Rational functions introduced as one polynomial divided by another, and why their behaviour differs from a polynomial's
- Horizontal asymptotes of a rational function from comparing numerator and denominator degree
- Slant asymptotes, which appear when the numerator's degree is exactly one more than the denominator's
- Zeros of a rational function found from the numerator, and why the denominator is never asked for a zero
- Describing end behaviour the way the exam expects it written, not just sketched

Projects built:

- Asymptote hunt: twelve rational functions sorted into horizontal-asymptote, slant-asymptote or neither, then verified by long division where needed
  Practice: 16 end-behaviour and asymptote problems across polynomials and rational functions, degree comparison written before any graphing
Vertical Asymptotes, Holes, and Equivalent Forms
  Topics:
    - Vertical asymptotes of a rational function from the zeros of the denominator
    - Holes: what happens when a factor cancels between numerator and denominator, and why that point still does not exist
    - Telling an asymptote from a hole by algebra, not by guessing from a rough sketch
    - Equivalent representations of a polynomial or rational expression, factored versus standard form, and what each reveals faster
    - Rewriting a rational expression to expose its asymptotes, holes and end behaviour in a single step
    - Choosing the representation that answers a given question fastest, a real time-saver under the no-calculator clock
  Projects built:
    - Rewrite-and-reveal set: ten rational functions rewritten from standard to factored form specifically to expose a hidden hole or asymptote
  Practice: 18 mixed Unit 1 problems on asymptotes, holes and equivalent forms, worked at no-calculator pace
  Assessment: Month 1 checkpoint: a timed, no-calculator paper on rates of change, polynomial behaviour and rational functions, marked for reasoning as well as answers

Month 2: Transformations, Modelling, and Unit 1 Mastery (Weeks 5-8)
Transformations of Functions
  Topics:
    - Parent function shapes reviewed: linear, quadratic, cubic, square root and reciprocal
    - Vertical and horizontal translations applied to a parent function's equation and graph
    - Vertical and horizontal stretches and compressions, and how they interact with translations
    - Reflections, and combining several transformations correctly in one rule
    - Reading a transformed equation straight into a sketch without plotting points first
    - Recovering the transformation from a before-and-after pair of graphs, a favourite exam question style
  Projects built:
    - Transformation matching set: ten transformed graphs matched to their equations, then the reverse, equation to sketch, for the same ten
  Practice: 20 transformation problems split evenly between equation-to-graph and graph-to-equation
Choosing a Function Model
  Topics:
    - Deciding whether a data table is better modelled as linear, polynomial or rational, from its pattern of change
    - Using constant, changing or ratio-based differences to identify the right model family
    - Stating the assumptions a chosen model makes, and saying honestly where it might not hold
    - Fitting a model to a short data table and justifying the choice of model in writing
    - The justify-your-model free-response style, where the reasoning earns as many points as the equation
    - Common model-selection traps: data that looks exponential over a short interval but is not
  Projects built:
    - Model justification write-up: two data tables modelled and the choice of model defended in four written sentences each
  Practice: 10 model-selection problems, a written justification required for every answer, not just an equation
Algebraic Fluency with Polynomial and Rational Expressions
  Topics:
    - Factoring polynomial and rational expressions at exam speed, not textbook speed
    - Polynomial long division and how a leftover remainder connects to a slant asymptote
    - Combining rational expressions and simplifying without silently losing a domain restriction
    - Solving polynomial and rational equations cleanly, checking every solution against the original domain
    - Sign analysis on a number line for solving rational inequalities
    - The domain-restriction mistakes that quietly cost marks on an otherwise correct answer
  Projects built:
    - Domain-safe solving set: eight rational equations and inequalities solved with every excluded value stated before the algebra begins
  Practice: 18 equation and inequality problems, domain restrictions written first on every single one
Unit 1 Mastery and the Multiple-Choice Format
  Topics:
    - Pulling Unit 1 together: rates of change, polynomial behaviour, rational functions and equivalent forms
    - The multiple-choice section as it is now: 40 questions, 2 hours, worth 62.5 percent of the score
    - Part A: 28 questions in 80 minutes with no calculator, fluency without a crutch
    - Part B: 12 questions in 40 minutes with a graphing calculator required
    - Recognising on sight which Unit 1 questions reward hand analysis and which are faster on a calculator
    - A personal error log started here, tagged by unit and by calculator or no-calculator part,

and grown for the rest of the course
Projects built:
- First timed multiple-choice block on Unit 1, split into a no-calculator half and a calculator half, reviewed question by question
Practice: A timed Unit 1 multiple-choice set at exam pace, both parts, followed by a written review of every miss
Assessment: Unit 1 assessment: a timed multiple-choice section on polynomial and rational functions, run in two parts, calculator and no-calculator, with the error log updated

---

PHASE 2: EXPONENTIAL AND LOGARITHMIC FUNCTIONS (UNIT 2, MONTHS 3-4, WEEKS 9-16)

Sequences as the discrete root of exponential change, exponential functions and modelling, composition and inverses, then logarithms through to equation solving, closing with a first look at free response.

Month 3: Sequences, Exponential Functions, and Modelling (Weeks 9-12)

Sequences and the Roots of Exponential Growth

Topics:
- Arithmetic sequences as discrete linear change, and the common difference that defines one
- Geometric sequences as discrete exponential change, and the common ratio that defines one
- Telling an arithmetic pattern from a geometric one directly from a table, before writing any formula
- Moving from a geometric sequence to a continuous exponential function
- The general exponential function form and what its starting value and base each control
- Growth versus decay, read straight from whether the base is greater than or less than one

Projects built:
- Sequence-to-function bridge: four geometric sequences extended into continuous exponential models, with the jump from discrete to continuous explained in writing

Practice: 16 sequence problems, half arithmetic and half geometric, each one converted into a formula from a table alone

Exponential Functions: Behaviour and Manipulation

Topics:
- Domain, range and end behaviour of an exponential function
- Percent change and how it connects directly to the growth or decay factor in the equation
- Rewriting an exponential expression with a different base without changing what it represents
- Transformations of exponential functions, applied the same way as any other parent function
- Why an exponential function's average rate of change scales rather than stays constant across equal-length intervals
- Spotting an exponential pattern in a table from that scaling behaviour, before any equation is written

Projects built:
- Percent-change translator: eight real-world growth and decay descriptions rewritten as exponential equations, base and starting value identified from the words alone

Practice: 18 exponential-function problems mixing percent change, rewriting bases and reading behaviour from tables

Exponential Modelling and Competing Models

Topics:
- Setting up an exponential model from a context: identifying the starting value and the rate
- Interpreting the parameters of a fitted exponential model back in the words of the original problem
- Competing function model validation: choosing between linear, polynomial and exponential from how a data set actually changes
- Checking a proposed model against the given data points before trusting it further
- The build-a-model and justify-your-choice free-response style, and what earns the reasoning point
- Recognising when a model is being used outside a domain where it still makes sense

Projects built:
- Model face-off: one data set modelled two competing ways, exponential and polynomial, with a written case for which one actually fits

Practice: 12 modelling problems, each requiring both a model and a two-to-three sentence justification

Function Composition and Inverses

Topics:
- Composing two functions and evaluating the result from a table, a graph, or an equation
- Interpreting what a composition means in a real-world context, not just computing it
- Inverse functions: what it actually means for one function to undo another
- Finding an inverse algebraically and confirming it graphically as a reflection over y = x
- The domain restriction a function needs before a true inverse exists
- Why exponential and logarithmic functions are built as a natural inverse pair, setting up the second half of this unit

Projects built:
- Inverse verification set: six functions inverted algebraically, then checked both by composing and graphically, by reflecting

Practice: 14 composition and inverse problems, each inverse checked by composing it back with the original

Assessment: Month 3 checkpoint: a timed paper on sequences, exponential functions and modelling, composition and inverses

Month 4: Logarithms and Equation Solving (Weeks 13-16)
Logarithms as Inverses of Exponential Functions
Topics:
- Defining the logarithm as the inverse of an exponential function, not as an isolated new rule
- Converting fluently between exponential form and logarithmic form in both directions
- The common logarithm and the natural logarithm, and when each one is the natural choice
- Domain and range of a logarithmic function, and why the domain excludes zero and negative numbers
- Graphing a logarithmic function directly from its exponential parent by reflecting over y = x
- Vertical asymptotes of logarithmic graphs, and why they sit exactly where the exponential's horizontal asymptote was

Projects built:
- Exponential-log translation set: twelve equations converted between exponential and logarithmic form, evaluated by hand where the values are friendly

Practice: 16 problems converting between forms and sketching logarithmic graphs from their exponential parents

Logarithmic Properties and Manipulation
Topics:
- The product, quotient and power properties of logarithms, and where each one comes from
- Expanding a single logarithmic expression into several, and condensing several back into one
- The change-of-base formula and exactly when a calculator needs it
- The property-application mistakes the exam sets on purpose, misapplying a rule that only works one way
- Using log properties to simplify an expression before solving, which is almost always faster than solving first
- Transformations of logarithmic functions, applied the same way as exponential ones

Projects built:
- Expand-and-condense drill: ten logarithmic expressions expanded fully, then condensed back, each check confirming the two forms are equal

Practice: 18 logarithm-property problems, half expanding and half condensing, plus 4 change-of-base calculator problems

Solving Exponential and Logarithmic Equations
Topics:
- Solving exponential equations by matching bases when possible
- Solving exponential equations by taking a logarithm of both sides when bases will not match
- Solving logarithmic equations and checking every solution against the original domain for extraneous answers
- Exponential and logarithmic inequalities, solved and expressed in interval form
- Semi-log plots: why data that looks exponential on a normal grid becomes a straight line on a semi-log scale
- Reading a semi-log plot to recover the original exponential model's parameters

Projects built:
- Extraneous-solution audit: eight logarithmic equations solved, with every solution checked back in the original equation and any extraneous root explained

Practice: 20 equation and inequality problems across exponential and logarithmic functions, extraneous-solution checks required throughout

Unit 2 Mastery and a First Look at Free Response
Topics:
- Pulling Unit 2 together: sequences, exponential functions, composition, inverses and logarithms
- A first full look at the free-response section: 4 questions, 1 hour, worth 37.5 percent of the score
- Part A of free response: 2 questions, 30 minutes, graphing calculator required
- Part B of free response: 2 questions, 30 minutes, no calculator allowed
- The style of exponential and logarithmic modelling question that tends to appear in free response
- Writing a modelling justification in full sentences, since the reasoning point is scored separately from the equation

Projects built:
- First free-response attempt: one Unit 2 style modelling question answered under time and self-scored against a sample rubric

Practice: A timed Unit 2 multiple-choice set at exam pace plus one full free-response attempt, reviewed for both content and communication

Assessment: Unit 2 assessment: a timed section on exponential and logarithmic functions, run in calculator and no-calculator parts, with the error log updated

---

PHASE 3: TRIGONOMETRIC AND POLAR FUNCTIONS, AND FULL EXAM MASTERY (UNIT 3 AND MOCKS, MONTHS 5-6, WEEKS 17-24)

The unit circle, sinusoidal and trigonometric functions, and polar coordinates and functions, followed by a full turn to exam craft: the calculator/no-calculator skill, free-response method, and complete timed hybrid-digital mock exams.

Month 5: Trigonometric and Polar Functions (Weeks 17-20)
The Unit Circle and Periodic Behaviour
Topics:
- The unit circle built from angle, radius one, and the coordinates it defines as cosine and sine

- Radians as the natural unit for angle measure, and fluent conversion between radians and degrees
- Special angles and the exact sine and cosine values worth having memorised, not looked up
- Periodicity: why sine and cosine repeat, and what a period of two-pi actually means
- Sine and cosine graphed directly from the unit circle, point by point at first, then from pattern
- Amplitude, period and midline read from a sinusoidal graph or straight from its equation

Projects built:
- Unit-circle-to-graph build: the sine and cosine graphs constructed point by point from the unit circle, then compared to the pattern-based shortcut

Practice: 20 unit-circle and radian-conversion problems, exact values required wherever the angle is a special one

Sinusoidal Functions and Modelling

Topics:
- The general sinusoidal function form and what amplitude, period, midline and phase shift each control
- Phase shift and exactly how it moves a sine or cosine graph left or right
- Building a sinusoidal model from a context description: extracting amplitude, period, midline and phase from words
- Sinusoidal context and data modelling questions in the free-response style
- Reading a sinusoidal model's parameters back out of a real-world description, the reverse skill
- Checking a fitted sinusoidal model against given data points before trusting it

Projects built:
- Sinusoidal modelling write-up: two real-world periodic scenarios, tides and temperature style, modelled with a full equation and parameters explained in words

Practice: 14 sinusoidal modelling problems, amplitude, period, midline and phase identified separately before the equation is assembled

Tangent, Reciprocal Trig, and Inverse Trig

Topics:
- The tangent function: its period, its vertical asymptotes and how its graph differs from sine and cosine
- Secant, cosecant and cotangent as reciprocal functions, graphed from their sine and cosine parents
- Inverse trigonometric functions and the restricted domains that make them true functions
- Using an inverse trig function to solve for an angle in a real context
- Solving trigonometric equations over a stated interval, finding every solution, not just one
- Trigonometric inequalities and expressing the solution as an interval

Projects built:
- Solve-and-verify set: ten trigonometric equations solved over a stated interval, every solution checked back on the unit circle

Practice: 18 problems across tangent, reciprocal trig functions and inverse trig, solution intervals stated before solving begins

Equivalent Trig Forms and Polar Functions

Topics:
- Trig identities as equivalent representations: the Pythagorean identity and the reciprocal identities
- Rewriting a trigonometric expression into an equivalent form that reveals its behaviour faster
- Polar coordinates: converting a point between polar and rectangular form in both directions
- Polar functions and their graphs: circles, roses and limacons recognised at a glance from their equation
- Reading symmetry and key features, maximum distance, zeros, directly from a polar graph
- How a polar function's periodic behaviour connects back to everything studied in this unit

Projects built:
- Polar gallery: six polar functions graphed and identified by family, circle, rose or limacon, with symmetry and key points labelled

Practice: 16 problems mixing trig identity rewriting and polar coordinate conversion, plus 4 polar-graph identification questions

Assessment: Unit 3 assessment: a timed section on trigonometric and polar functions, run in calculator and no-calculator parts

Month 6: Free-Response Mastery and Full Mock Exams (Weeks 21-24)

The Calculator/No-Calculator Split and Multiple-Choice Strategy

Topics:
- Mapping all four exam parts precisely: MC Part A, 28 questions, 80 minutes, no calculator; MC Part B, 12 questions, 40 minutes, calculator required; FRQ Part A, 2 questions, 30 minutes, calculator; FRQ Part B, 2 questions, 30 minutes, no calculator
- Building genuine no-calculator fluency: exact values, factoring and algebraic manipulation done by hand at speed
- Calculator strategy for Part B: what a graphing calculator is actually good for, and what it cannot substitute for
- Pacing each part on its own terms, roughly two minutes a question in Part A and just over three in Part B
- Recognising at a glance which part of the exam a released question is drawn from, and adjusting method accordingly
- A first timed multiple-choice block mixing both parts, reviewed question by question with each miss tagged by cause

Projects built:

- Timed mixed multiple-choice block, Part A followed immediately by Part B, reviewed with every miss tagged calculator, no-calculator, concept or pacing

Practice: A timed multiple-choice set at full exam pace across both parts, followed by a written review of every wrong and every guessed answer

Free-Response Method Across Units 1 to 3

Topics:

- The free-response rubric mindset: where partial credit lives and how to secure it even on a question you cannot fully finish
- Justification writing: what actually earns the communication point on a modelling question, in plain, specific sentences
- A Unit 1 style free-response task: justifying a polynomial or rational model against given data
- A Unit 2 style free-response task: building an exponential or logarithmic model and solving an equation from it
- A Unit 3 style free-response task: building a sinusoidal model from a context description
- Self-scoring a free-response answer against a sample rubric, and naming exactly which point was lost and why

Projects built:

- Three timed free-response responses, one from each tested unit, each self-scored against a sample rubric with lost points named

Practice: Timed free-response practice across all three units, every attempt scored for both the answer and the justification

Full Mock Exam One

Topics:

- Running a complete timed mock in the real order: Multiple Choice Part A, Part B, then Free Response Part A, Part B
- Working under hybrid-digital conditions: multiple choice answered in a Bluebook-style flow, free response handwritten on paper
- Scoring the mock and converting the raw result toward the 1 to 5 scale
- Reviewing errors sorted by unit, Units 1, 2 and 3, to see where the real gaps sit
- Reviewing errors sorted by part, so a calculator-dependent weakness is not confused with a content gap
- Naming the single biggest gap the mock revealed and building one week's plan to close it

Projects built:

- Full mock exam one, scored and reviewed, with a written breakdown of performance by unit and by calculator or no-calculator part

Practice: The full timed mock plus a complete review, every missed multiple-choice question re-worked correctly by hand

Full Mock Exam Two, Test-Ready Review, and a Unit 4 Preview

Topics:

- A second complete timed mock, run fully under exam conditions from Multiple Choice Part A through Free Response Part B
- Comparing the two mocks directly to see real direction of travel, not just a single score
- The test-day routine: the Bluebook app for multiple choice, the paper booklet for free response, and timing across all four parts
- A realistic, evidence-based target score built from both mocks, not a guess
- An optional, clearly labelled preview of Unit 4, functions involving parameters, vectors and matrices, for students continuing on to calculus; this content is not tested on the AP Precalculus exam and is covered only as a bridge forward
- Where to go next: AP Calculus AB or BC, or a next mathematics course, chosen honestly from where this course leaves you

Projects built:

- Full mock exam two, scored and reviewed, ending in a one-page test-day plan built from both mocks' results

Practice: The second full mock plus targeted repair of any remaining weak topic from either mock

Assessment: Final assessment: a complete timed mock exam covering Units 1 through 3, a progress summary from week 1 to now, and certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A rate-of-change notebook studying one function across table, graph and equation
- A zero-to-graph and asymptote-hunt portfolio across polynomial and rational functions
- A model-justification write-up defending a chosen function model in writing
- A sequence-to-function bridge connecting discrete geometric sequences to continuous exponential models
- An inverse-verification set checking every inverse both algebraically and graphically
- A sinusoidal modelling write-up built from real-world periodic scenarios
- A polar gallery of graphed and identified polar function families
- Three scored free-response answers across all three tested units, plus two full timed mock exams under hybrid-digital conditions

Total projects: Dozens of worked problem sets plus three scored free responses and two complete timed mock exams, all mapped to Units 1 through 3

SKILLS THE STUDENT WILL HAVE AT THE END

- Function fluency: rates of change, transformations, end behaviour and equivalent representations

- Polynomial and rational function analysis, including zeros, asymptotes and holes
- Exponential and logarithmic reasoning, from sequences through to equation solving
- Trigonometric and polar function analysis, from the unit circle through to polar graphs
- Model selection and modelling justification, written the way the free-response rubric rewards
- Calculator and no-calculator fluency on both sides of the exam's part boundary, drilled on full timed mocks

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, working and checking problems live rather than watching solutions scroll by
Practice: 4-5 hours weekly of problem sets and, in the final month, timed free-response and full mock sections
Review: Homework and free responses reviewed with written feedback, scored against a sample rubric so you see exactly where reasoning is won and lost
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for the problems that stall you
Progress updates: Monthly progress notes tracking unit assessments and, in the final month, mock-exam scores against your target
Career guidance: Honest guidance on what an AP score does and does not do for college admissions and credit, and what mathematics course to take next

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- A strong footing for the AP Precalculus exam, Units 1 through 3, and through it, university applications
- The function fluency, transformations, modelling and equivalent representations, that AP Calculus AB and BC assume from the first week
- A genuine next step toward AP Calculus or a first university mathematics course
- Modelling and justification habits that transfer to statistics, physics and other quantitative coursework
- The problem-solving base for our AP Calculus and college-level mathematics courses

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is Unit 4 tested on the AP Precalculus exam?
A: No, and this is the single most useful thing to know before you plan your prep time. AP Precalculus is taught as four units, but the exam only assesses Units 1, 2 and 3: polynomial and rational functions, exponential and logarithmic functions, and trigonometric and polar functions. This course concentrates its exam preparation on Units 1 through 3, exactly where the scoring happens, and covers Unit 4 only as a short, clearly labelled preview in the final week for students continuing on to calculus.

Q: What is the calculator versus no-calculator split on this exam?
A: Both sections are split into a calculator part and a no-calculator part, and knowing the boundary cold is a real exam skill. Section I, Multiple Choice, has Part A with 28 questions in 80 minutes and no calculator allowed, then Part B with 12 questions in 40 minutes where a graphing calculator is required. Section II, Free Response, has Part A with 2 questions in 30 minutes where a graphing calculator is required, then Part B with 2 questions in 30 minutes and no calculator. We drill both sides of that line from week one, not just in the final month.

Q: What do I need to know before starting this course?
A: A solid Algebra 2 background: comfort with function notation, factoring, exponents and basic graphing. The course does not re-teach Algebra 2 from the beginning, though real gaps are patched as they show up rather than ignored. No programming or coding background is needed or used; this is a pure mathematics course.

Q: How does AP Precalculus actually prepare me for AP Calculus?
A: Honestly, and mostly through fluency rather than new tricks. AP Calculus assumes you can read a function's behaviour, transformations, end behaviour, zeros, asymptotes, without hesitating, and that you are comfortable moving between exponential, logarithmic and trigonometric forms. That is exactly what Units 1 through 3 build. Students who finish this course fluent in function behaviour spend their first weeks of calculus learning calculus, not relearning precalculus under a new name.

Q: What is the AP Precalculus exam format?
A: Two sections. Section I, Multiple Choice, is 40 questions in 2 hours and is worth 62.5 percent of the score, split into a 28-question no-calculator Part A and a 12-question calculator-required Part B. Section II, Free Response, is 4 questions in 1 hour and is worth 37.5 percent of the score, split into a 2-question calculator-required Part A and a 2-question no-calculator Part B.

Q: What is the hybrid digital exam format?
A: The multiple-choice section is taken digitally in the College Board's Bluebook app, the same platform used for the digital SAT and several other AP exams. The free-response section, though, is answered by hand: you view the questions in Bluebook but write your actual answers in a paper booklet. We run our mock exams under exactly this split, digital multiple choice and handwritten free response, so the format itself is familiar by test day.

Q: Will this course get me a 5, or college credit?
A: No honest course can promise a specific score. AP exams are graded 1 to 5, and your result depends on your starting point, the hours you put in and your performance on the day. What we can promise is honest: live teaching concentrated on the tested units, small batches, real doubt support, free-response drilling against a real rubric, and full timed mock exams under hybrid-digital conditions, plus a clear, evidence-based read on a realistic target for you. Whether a given score earns college credit, and for what, is decided by each university, and AP Precalculus credit policies vary more than AP Calculus policies do, so we point students to check the specific policy of the colleges they care about.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Is this course affiliated with the College Board?
A: No. Modern Age Coders is an independent education provider and is not affiliated with, authorised by, or endorsed by the College Board. AP and Advanced Placement are trademarks of the College Board, used here only to describe the exam this course prepares students for. For official course and exam information, students should always refer to the College Board and AP Central directly.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ap-precalculus-exam-prep-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 21 OF 137: AP STATISTICS: EXAM PREP WITH THE INVESTIGATIVE TASK

==========

Course link: https://learn.modernagecoders.com/courses/ap-statistics-maths-exam-prep-course
Subject area: Advanced Placement Mathematics
Designed for: High school
Who can join: High school students preparing for AP Statistics; taught from the ground up, comfort with school algebra assumed
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 4-5 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online AP Statistics exam prep: all nine AP units, the Investigative Task, free-response rubric mastery and full digital mock exams for a real score.

In one line: A statistics course built around one exam: nine AP units, the Investigative Task, and the free-response rubric that decides most of the score.

ABOUT THE PROGRAMME

AP Statistics is not the same subject as a general college statistics course, even though the two share a name. It is organised around nine specific units, tested on one specific exam, in one specific format, and this course is built around exactly that exam as it exists now, not around statistics in general. The exam runs in two equal halves: Section I is 40 multiple-choice questions in 90 minutes, worth half the score, and Section II is 6 free-response questions in 90 minutes, worth the other half. Of those six, the first five are short-answer questions, budgeted at roughly 12 minutes each, and the sixth is the

Investigative Task, a single longer question worth about 30 of those 90 minutes that asks you to apply and extend what you know to a scenario the exam has not shown you before. The exam is also hybrid: multiple-choice questions and the free-response prompts are viewed in the College Board's Bluebook digital testing app, but every free-response answer is handwritten in a paper booklet that is collected and scored separately. A graphing calculator, such as a TI-84, is expected and permitted for the whole exam, and this course teaches its use from week one. The six-month structure follows the nine AP units in the order the exam tests them. Months 1 and 2 cover Units 1 through 3: exploring one-variable data with the normal model, exploring two-variable data with correlation and least-squares regression, and collecting data through sampling and experimental design. Months 3 and 4 cover Units 4 and 5: probability, random variables, the binomial and geometric distributions, then sampling distributions and the Central Limit Theorem that every later inference procedure depends on. Months 5 and 6 cover Units 6 through 9, formal inference for proportions, means, chi-square settings and regression slopes, then turn fully to exam craft: the Investigative Task practiced as its own skill, every free-response question type drilled against the real rubric, and two full timed mock exams run under the exam's actual hybrid-digital conditions. Because the rubric rewards communication as much as computation, every answer in this course, from week one onward, is written to justify a conclusion with evidence, interpret it correctly in context, and state the conditions it depends on, since a right number with no justification earns limited credit on the real exam.

WHAT MAKES THIS COURSE DIFFERENT

- Organised around the nine official AP Statistics units in the order the exam actually tests them, not a general statistics syllabus with an AP label attached
- The Investigative Task, free-response Question 6, practiced as its own skill from month 6 onward, since it is scored differently from the other five questions and most courses barely touch it
- Free-response answers drilled against the communication standard the rubric truly rewards: justifying a conclusion with evidence, interpreting it correctly, and stating the conditions it depends on
- Graphing calculator fluency built in from week 1, since the real exam expects confident use of a calculator such as a TI-84 throughout both sections
- Full readiness for the hybrid-digital format: multiple-choice and free-response prompts viewed in the Bluebook app, but free-response answers handwritten in paper booklets, practiced under those exact conditions
- Live, small batches where you work real data and real free responses during class and get them reviewed, not lecture videos watched alone

THE LEARNING PATH, STEP BY STEP

Phase 1: Exploring data and collecting it: one-variable distributions and the normal model, two-variable scatterplots and least-squares regression, then sampling methods, experiments and bias (Units 1-3)
Phase 2: Randomness and its rules: probability, random variables, binomial and geometric distributions, then sampling distributions and the Central Limit Theorem (Units 4-5)
Phase 3: Formal inference and the exam itself: confidence intervals and significance tests for proportions, means, chi-square settings and slopes (Units 6-9), Investigative Task mastery, and full timed mock exams under real hybrid-digital conditions

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Comfortable school algebra: working with formulas, fractions, percentages and graphs. No calculus is required anywhere in AP Statistics
Programming: None required. All work is done with a graphing calculator and by hand; no coding or spreadsheet skills are needed for this course
Equipment: A graphing calculator such as a TI-84 or an equivalent approved model, used and permitted throughout the real exam, plus a device with a stable internet connection for live classes. We help with calculator setup in week 1
Audience: High school students preparing for AP Statistics, typically in the years leading up to the exam, whether taking it at school or self-studying

WHO THIS COURSE IS FOR

Ap students: Students taking AP Statistics at school who want a structured, unit-by-unit course and real feedback on their free responses alongside their class
Self studiers: Students whose school does not offer AP Statistics and who are preparing for the exam independently
Score focused: Students aiming for a strong score who want free-response drilling against the real rubric, Investigative Task practice, and full timed mock exams
Math and science bound: Students headed for a data-heavy college major, from psychology to engineering, who want genuine statistical reasoning, not just exam tricks
International students: International students sitting AP Statistics for university applications who need the exam taught to its current hybrid-digital format

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: EXPLORING DATA AND COLLECTING DATA (UNITS 1-3, MONTHS 1-2, WEEKS 1-8)

The data-analysis half of the course: describing one-variable and two-variable data honestly,
then how data is collected in the first place. The graphing calculator and the free-response
communication standard are introduced here and used every week from this point on.

  Month 1: Exploring One-Variable Data (Unit 1) (Weeks 1-4)
    Displaying and Describing Distributions
      Topics:
        - Individuals, variables, and the shape of the AP Statistics course: nine units, two exam sections, and the exam that ends it
        - Categorical versus quantitative data, and the type of graph each one calls for
        - Dotplots, stemplots and histograms: reading shape, not just drawing it
        - Describing a distribution in words: shape, outliers, center and spread, the sentence the exam always wants
        - Setting up the graphing calculator: entering lists, running 1-Var Stats, and building a first histogram on screen
        - The AP Statistics communication standard: why a correct number with no justification earns limited credit
      Projects built:
        - First distribution report: a real one-variable data set displayed, described in full shape-outliers-center-spread sentences, and checked on the calculator
      Practice: 12 distribution-description problems, each answer required to name shape, outliers, center and spread in one paragraph, not a list of numbers
    Measuring Center and Spread
      Topics:
        - Mean and median, and which one a skewed distribution or an outlier pulls away from the other
        - Range, interquartile range, and the 1.5 times IQR rule for flagging outliers
        - Variance and standard deviation built up from deviations, not dropped as a formula
        - The five-number summary, and building a boxplot from it by hand and by calculator
        - Choosing mean and standard deviation versus median and IQR to summarise a distribution, and defending the choice
        - Comparing two distributions side by side on boxplots, a guaranteed exam question type
      Projects built:
        - Center-and-spread comparison: two real groups compared on boxplots with a full written comparison of shape, center, spread and outliers
      Practice: 14 problems computing and choosing summary measures, half requiring a written comparison between two groups
    The Normal Distribution
      Topics:
        - The normal curve, its parameters, and the 68-95-99.7 rule
        - Standardising a value into a z-score and reading what it says about relative position
        - Finding areas under the normal curve on the calculator with normalcdf
        - The inverse problem: finding a value from a given percentile with invNorm
        - Assessing normality with a normal probability plot, read for curvature rather than computed
        - Normal calculations written up in context, the way the exam scores them, not just the final number
      Projects built:
        - Normal model check: a real quantitative data set tested against the 68-95-99.7 rule, with normalcdf and invNorm used to answer three context questions
      Practice: 14 normal distribution problems, each opened with a labelled sketch of the curve and the shaded region before any calculator work
    Unit 1 Mastery and the Free-Response Standard
      Topics:
        - Pulling Unit 1 together: distributions, summary statistics and the normal model in one paper
        - Reading a Unit 1 free-response prompt the way the rubric is written, points named as sentences rather than numbers
        - The exam's two sections at a glance: 40 multiple-choice questions in 90 minutes worth 50 percent of the score, and 6 free-response questions in 90 minutes worth the other 50 percent
        - Multiple-choice pacing for a Unit 1 style set: roughly two minutes a question
        - Common Unit 1 mistakes: describing shape from a boxplot alone, and confusing standard deviation with interquartile range
        - Starting a personal error log, kept and grown for the rest of the course
      Projects built:
        - First scored free response: a Unit 1 style question written under time and self-scored against a rubric, with every lost point named
      Practice: A timed Unit 1 multiple-choice set at exam pace, followed by one full free-response question reviewed line by line against its rubric
      Assessment: Unit 1 assessment: a timed paper on one-variable data covering multiple-choice and one free response, marked on both the number and the justification

  Month 2: Two-Variable Data and Collecting Data (Units 2-3) (Weeks 5-8)
    Exploring Two-Variable Data: Scatterplots and Correlation
      Topics:
        - Explanatory and response variables, and setting up a scatterplot correctly
        - Describing a scatterplot: direction, form, strength and outliers, the two-variable version of shape-outliers-center-spread
        - The correlation coefficient r: what it measures and where its usefulness stops

- Correlation is not causation, with a plausible confounding variable named every time
- Computing r on the calculator from two lists
- Why a strong r does not automatically mean a good model, previewing residuals

Projects built:
- Relationship study: a real two-variable data set plotted, described in full, and its correlation computed and interpreted in context

Practice: 12 scatterplot and correlation problems, every claim of a relationship backed by a described scatterplot, not just an r value

Least-Squares Regression

Topics:
- The least-squares regression line and exactly what quantity it minimises
- Finding the equation on the calculator with LinRegTTest, and reading slope and intercept in context
- The coefficient of determination, r-squared, and what percentage of variation it explains
- Residuals and the residual plot: the honesty check the exam consistently asks for
- Predicting within the range of the data versus extrapolating beyond it
- Influential points and outliers in a regression setting

Projects built:
- Prediction model: a regression line fit to real data, its residual plot checked for pattern, and two predictions made with their limits stated

Practice: 12 regression problems with slope and intercept interpreted in the units of the problem every time, plus 4 residual-plot readings

Collecting Data: Sampling

Topics:
- Population versus sample, and why that distinction drives all of Unit 3
- Simple random samples, stratified samples, cluster samples and systematic samples
- Convenience sampling and voluntary response, and why both are biased by design
- Sources of bias: undercoverage, nonresponse, and response bias from question wording
- How a sampling method is described and evaluated on the free-response section
- Designing a sampling plan for a given population and research question

Projects built:
- Sampling critique: a real published survey evaluated for its sampling method, with specific bias risks named in writing

Practice: 10 sampling-design problems, each requiring the method to be named and one bias risk identified

Collecting Data: Experiments and Unit 2-3 Mastery

Topics:
- Observational studies versus experiments, and what only a well-designed experiment can show
- Experimental design vocabulary: treatments, experimental units, and the placebo effect
- The principles of good experimental design: control, randomisation, and replication
- Blocking and why it is used, contrasted with a completely randomised design
- Confounding variables in an experiment versus in an observational study
- Units 2 and 3 pulled together, with a timed multiple-choice set mixing both

Projects built:
- Experiment design: a full experimental design written for a given research question, naming treatments, randomisation and control

Practice: 10 experimental-design problems plus a timed Unit 2-3 multiple-choice set reviewed question by question

Assessment: Unit 2-3 assessment: a timed paper on two-variable data and data collection, including one regression free response and one design free response

---

PHASE 2: PROBABILITY AND SAMPLING DISTRIBUTIONS (UNITS 4-5, MONTHS 3-4, WEEKS 9-16)

The mechanics of randomness that make inference possible: probability rules, random variables, the named distributions, then sampling distributions and the Central Limit Theorem that the entire second half of the course depends on.

Month 3: Probability, Random Variables and Probability Distributions (Unit 4) (Weeks 9-12)

Probability Rules

Topics:
- Randomness, probability as long-run relative frequency, and simulating it on the calculator
- Sample spaces, events, and the basic probability rules
- The addition rule for mutually exclusive and overlapping events
- Conditional probability, and what being told the condition changes about the sample space
- Independence: the definition, and testing for it using a two-way table
- The multiplication rule for independent and dependent events

Projects built:
- Two-way table study: a real two-way table used to compute conditional probabilities and formally test two events for independence

Practice: 14 probability problems from single events through conditional probability, independence checked by calculation, not by eye

Random Variables

Topics:
- Discrete random variables and their probability distributions
- Mean, or expected value, of a discrete random variable, and what it means for a variable that only takes whole values
- Standard deviation of a discrete random variable
- Combining random variables: rules for the mean and variance of sums and differences

- Independence between two random variables, and why it matters before combining variances
- Random variables in context: insurance, games, and simple business scenarios

Projects built:

- Combined-variable analysis: two random variables combined by sum or difference, with mean and standard deviation of the result found and interpreted

Practice: 12 random variable problems including 4 combination questions checked for the independence condition first

Binomial and Geometric Distributions

Topics:

- The binomial setting: the four conditions checked in words before any formula is used
- Binomial probability, mean and standard deviation, by hand and with binompdf and binomcdf
- The geometric setting: waiting for the first success
- Geometric probability, mean and standard deviation, with geometpdf and geometcdf
- Choosing between a binomial and a geometric model from the story in the question
- The 10 percent condition: when sampling without replacement behaves close enough to independent

Projects built:

- Model choice project: three real scenarios each modelled correctly as binomial or geometric, with the setting's conditions checked in writing

Practice: 14 binomial and geometric problems, the first written step always naming and checking the setting's conditions

Unit 4 Mastery and Probability Free Response

Topics:

- Pulling Unit 4 together: probability rules, random variables, binomial and geometric models
- Reading a probability free-response question for exactly what it asks: a probability, a mean, or a standard deviation
- Common Unit 4 mistakes: using a binomial model when trials are not independent, and misreading at least versus more than
- Calculator fluency check: normalcdf, invNorm, binompdf, binomcdf, geometpdf and geometcdf used correctly and quickly
- A timed Unit 4 multiple-choice set at exam pace
- Error log review: Unit 4 mistakes classified and drilled

Projects built:

- Timed probability free response written under exam conditions and self-scored against a rubric

Practice: A timed Unit 4 multiple-choice set plus one full free-response question reviewed against its rubric

Assessment: Unit 4 assessment: a timed paper on probability, random variables and probability distributions, including one free response

Month 4: Sampling Distributions (Unit 5) (Weeks 13-16)

Sampling Variability and the Sampling Distribution of a Proportion

Topics:

- Parameter versus statistic, and why every sample gives a slightly different answer
- The idea of a sampling distribution: the distribution of a statistic over repeated samples
- Simulating a sampling distribution on the calculator to watch shape, center and spread build up
- The sampling distribution of a sample proportion: its mean, its standard deviation, and when it is approximately normal
- The conditions for that normal approximation: random, 10 percent, and large counts
- Unbiased estimators, and why the sample proportion is one for the population proportion

Projects built:

- Simulation build: a sampling distribution for a proportion simulated by hand or calculator, with shape, center and spread reported from the simulation

Practice: 10 sampling-distribution problems for proportions, each requiring the three conditions checked before any normal calculation

The Sampling Distribution of a Sample Mean and the Central Limit Theorem

Topics:

- The sampling distribution of a sample mean: its mean and its standard deviation
- The Central Limit Theorem stated precisely: what a large sample size buys you even from a skewed population
- Watching the Central Limit Theorem happen: simulating sample means from a skewed population as sample size grows
- When the population itself is already normal, and why a large sample size is not needed then
- The 10 percent condition applied to sample means
- Common misreadings of the theorem, including confusing the population's shape with the sampling distribution's shape

Projects built:

- Central Limit Theorem simulation: sample means drawn repeatedly from a skewed population, with the resulting shape compared at small and large sample sizes

Practice: 10 sampling-distribution problems for means, plus the simulation repeated on a second population shape

Sampling Distributions for Differences and Exam-Style Reasoning

Topics:

- The sampling distribution of a difference in two proportions
- The sampling distribution of a difference in two sample means
- Conditions for each, including independence between the two samples
- Reading a scenario to decide which sampling distribution applies: one sample or two, proportion or mean

- Connecting Unit 5 forward to Units 6 and 7: every confidence interval and test still to come rests on one of these distributions
- Free-response practice describing a sampling distribution in full: shape, center, spread and the conditions that justify each

Projects built:
- Distribution identification drill: ten scenarios each matched to the correct sampling distribution, with mean, standard deviation and conditions stated

Practice: 12 mixed sampling-distribution problems where the question never announces which distribution to use

Unit 5 Mastery and the Bridge to Inference

Topics:
- Pulling Unit 5 together: sampling variability and every sampling distribution covered so far
- A worked preview of how a confidence interval is really a sampling distribution centered on a statistic
- A worked preview of how a significance test asks whether a statistic is surprising given an assumed sampling distribution
- A timed multiple-choice set at exam pace covering Units 4 and 5 together
- Error log review: probability and sampling-distribution mistakes classified and drilled
- Calculator fluency check across every function used so far in the course

Projects built:
- Timed Unit 5 free response written under exam conditions and self-scored against a rubric

Practice: A timed Units 4-5 multiple-choice set plus one full free-response question reviewed against its rubric

Assessment: Unit 5 assessment: a timed paper on sampling distributions, including one free response connecting a distribution to its conditions

---

PHASE 3: INFERENCE AND EXAM MASTERY (UNITS 6-9 AND FULL MOCKS, MONTHS 5-6, WEEKS 17-24)

Formal inference for every setting the exam tests, then a full turn to exam craft: the Investigative Task drilled as its own skill, every free-response type reviewed against the real rubric, and two complete timed mock exams under hybrid-digital conditions.

Month 5: Inference for Proportions and Means (Units 6-7) (Weeks 17-20)

Confidence Intervals for a Proportion

Topics:
- The logic of a confidence interval: an interval built to have a stated chance of capturing the true parameter
- The conditions for inference on one proportion: random, 10 percent, and large counts
- Constructing a one-sample z-interval for a proportion, by hand and with the calculator
- Margin of error, and the three things that change it
- What 95 percent confidence actually means, and the misreadings the exam is built to catch
- Confidence intervals for a difference in two proportions

Projects built:
- Interval report: a confidence interval for a real proportion built from data, with the correct interpretation written out in full context

Practice: 12 proportion-interval problems, every interval followed by a correctly worded, in-context interpretation sentence

Significance Tests for a Proportion

Topics:
- Stating null and alternative hypotheses correctly for a one-proportion test
- The test statistic, the p-value, and what a p-value actually measures
- Checking the same three conditions before trusting a test
- One-sample and two-sample z-tests for proportions, by hand and with the calculator
- Type I and Type II errors, and significance level as the Type I error rate you choose
- Writing a full test conclusion in context: state, plan, do, conclude, the four-step structure the rubric rewards

Projects built:
- Claim test: a real claim about a proportion tested against data, written up in the full state-plan-do-conclude structure

Practice: 12 proportion-test problems, every conclusion written in context and linked back to the p-value

Confidence Intervals for a Mean

Topics:
- Why a t-distribution, not a normal distribution, is used for inference on a mean
- Degrees of freedom in plain language, and how the t-distribution compares to normal as they grow
- The conditions for inference on one mean: random, 10 percent, and normal or large sample
- Constructing a one-sample t-interval for a mean, by hand and with the calculator
- Confidence intervals for a difference in two means, paired and unpaired designs told apart
- Recognising a paired design, a frequent trap on the exam

Projects built:
- Interval report: a confidence interval for a real mean built from data, with the paired-versus-unpaired decision justified in writing

Practice: 12 mean-interval problems including two paired designs hidden among unpaired ones

Significance Tests for a Mean and Unit 6-7 Mastery

Topics:
- One-sample and two-sample t-tests for means, hypotheses stated and conditions checked every

time
- Paired t-tests, and why they are really a one-sample test performed on the differences
- Effect size, and why a significant result is not automatically an important one
- Pulling Units 6 and 7 together: proportions and means, intervals and tests, side by side
- A timed multiple-choice set at exam pace covering both units
- Error log review: inference mistakes classified, especially condition-checking and interpretation wording

Projects built:
- Timed inference free response written under exam conditions and self-scored against a rubric

Practice: A timed Units 6-7 multiple-choice set plus one full free-response question reviewed against its rubric

Assessment: Unit 6-7 assessment: a timed paper on inference for proportions and means, including one free response scored on conditions and conclusion wording

Month 6: Chi-Square, Slopes, the Investigative Task, and Full Mock Exams (Units 8-9) (Weeks 21-24)

Inference for Categorical Data: Chi-Square

Topics:
- The chi-square goodness-of-fit test: does one categorical variable match a claimed distribution
- The chi-square test for independence: are two categorical variables related, within one sample
- The chi-square test for homogeneity: do several populations share the same distribution, and how it differs from independence
- Expected counts, the conditions for a valid chi-square test, and computing the test statistic
- Running each chi-square test on the calculator and reading its output correctly
- Writing a full chi-square conclusion in the state-plan-do-conclude structure

Projects built:
- Independence study: a real two-way table tested for independence, with expected counts shown and a full written conclusion

Practice: 12 chi-square problems split across the three test types, the correct type identified from the scenario before any calculation

Inference for the Slope of a Regression Line

Topics:
- The population regression model, and what the true value of the slope represents
- Conditions for inference on a slope, read from a residual plot and a normal probability plot of residuals
- The t-test for the slope: hypotheses, test statistic, and reading calculator regression output
- The confidence interval for the slope, constructed from the same output
- Connecting Unit 9 back to Unit 2: the regression skills from month 2, now formally tested
- Interpreting a slope-test conclusion in the context of the original two variables

Projects built:
- Slope inference project: a real two-variable data set tested for a nonzero slope, with conditions checked from residual and normal plots

Practice: 10 slope-inference problems, conditions checked from a described or sketched residual plot every time

The Investigative Task

Topics:
- What free-response Question 6, the Investigative Task, actually asks: applying and extending the course's skills to a scenario that will not look familiar
- Why the Investigative Task is scored differently from Questions 1 through 5, and what that means for pacing, roughly 30 of the 90 free-response minutes
- Reading an unfamiliar prompt calmly: identifying which unit's tools actually apply underneath the new dressing
- Extending a known method one step further than it was originally taught, the specific skill the task tests
- Working through released Investigative Task style prompts, one full task per session
- Building a personal checklist for the task: state assumptions, justify every step, and answer the question actually asked

Projects built:
- Two full Investigative Task style responses written under time and self-scored against a released-style rubric, with the extension step highlighted

Practice: Two more Investigative Task style prompts worked untimed first for understanding, then timed, with every assumption stated in writing

Full Mock Exams and Test-Ready Review

Topics:
- A first complete timed mock exam: all 40 multiple-choice questions and all 6 free-response questions, including the Investigative Task
- Running the mock under real hybrid-digital conditions: multiple-choice and free-response prompts viewed digitally, free-response answers handwritten on paper
- Scoring the mock against official-style rubrics and converting toward the 1 to 5 scale
- A second complete timed mock, then closing whatever gaps the first one exposed
- Test-day logistics: the Bluebook app, the graphing calculator, the paper answer booklets, and timing across both sections
- A one-page, evidence-based test-day plan built from your own two mock scores

Projects built:
- Two full mock exams, scored, with a written comparison of the two and a final one-page test-day plan

Practice: Both full timed mocks plus a complete review, every missed multiple-choice

question re-worked and every free response re-scored
Assessment: Final assessment: a complete timed mock exam under hybrid-digital conditions, a progress summary from week 1 to now, and certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A full shape-outliers-center-spread distribution report on a real one-variable data set
- A least-squares regression model with residuals checked and predictions limited to the data range
- A sampling-method critique of a real published survey
- A full experimental design written for a real research question
- A simulated sampling distribution built and reported by you
- A confidence interval and significance test written in the full state-plan-do-conclude structure
- A chi-square independence study with a complete written conclusion
- Two Investigative Task style responses and two full timed mock exams under hybrid-digital conditions

Total projects: A portfolio of unit projects across all nine AP units plus multiple scored free responses, two Investigative Task responses and two complete timed mock exams

SKILLS THE STUDENT WILL HAVE AT THE END

- Describing and comparing distributions with full statistical language, not just numbers
- Scatterplots, correlation and least-squares regression, checked with residual plots
- Sound data collection: sampling methods, bias, and experimental design
- Probability, random variables, and the binomial and geometric distributions
- Sampling distributions and the Central Limit Theorem that inference depends on
- Confidence intervals and significance tests for proportions, means, chi-square settings and slopes, plus Investigative Task reasoning

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, working problems and free responses live rather than watching solutions scroll by
Practice: 4-5 hours weekly of problem sets, calculator drills and, from month 5 onward, timed free-response and mock sections
Review: Homework and free responses reviewed with written feedback, scored against AP-style rubrics so you see exactly where points are earned and lost
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for the problems that stall you
Progress updates: Monthly progress notes tracking unit assessments and, in the final months, mock-exam scores against your target
Career guidance: Honest guidance on what an AP score does and does not do for college admissions and credit, and what to study next after AP Statistics

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- A strong footing for the AP Statistics exam and, through it, university applications
- Readiness for a college introductory statistics course, which AP Statistics closely mirrors in both content and reasoning
- A genuine base of statistical reasoning for any data-heavy major: psychology, economics, biology, business or engineering
- Free-response and justification habits, stating assumptions and interpreting evidence, that transfer directly into college coursework
- A natural next step into our Statistics and Probability course for students who want the subject's college-level and professional applications beyond the AP exam

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: How is this different from a general statistics course, and from your other Statistics and Probability course?
A: It is a different subject taught for a different purpose. Our Statistics and Probability course is a general college-level and professional course, spreadsheet-first, with no exam and no fixed unit order, ending in a capstone analysis of a dataset you choose. This AP Statistics course is exam-specific: it is organised around the nine official AP units in the

order the College Board tests them, it drills the free-response section against the actual AP rubric, it dedicates real time to the Investigative Task, it teaches graphing-calculator methods the AP exam expects, and it is built for high school students working toward an AP score of 1 to 5. If your goal is the AP exam, take this course. If your goal is statistics for a data-heavy career with no exam involved, our Statistics and Probability course is the better fit, and graduates of this course who want that broader, professional-facing material are welcome there afterward.

Q: Do I need a graphing calculator, and which one?
A: Yes. A graphing calculator is expected and permitted throughout the entire AP Statistics exam, and we use one in class from week 1 onward, both for computation and because the exam expects you to read and quote calculator output correctly. A TI-84 or TI-84 Plus is the most common model and the one our examples default to, but any calculator on the College Board's approved list works, and we help you get comfortable with whichever model you already own during the first week rather than requiring a specific purchase.

Q: What is the Investigative Task?
A: It is free-response Question 6, and it is different from the other five free-response questions. Where Questions 1 through 5 test one unit's skills in a fairly direct way, the Investigative Task hands you a scenario you have not seen before and asks you to apply and extend the statistical reasoning you already have to fit it, sometimes stretching a method slightly beyond how it was originally taught. It is budgeted at roughly 30 of the 90 free-response minutes, making it worth real, focused preparation rather than being left as an afterthought, which is why we give it a dedicated week, week 23, plus additional timed practice in the mock phase.

Q: What is the exam format, and why does justification matter so much?
A: Section I is 40 multiple-choice questions in 90 minutes, worth 50 percent of the score. Section II is 6 free-response questions in 90 minutes, worth the other 50 percent: 5 short-answer questions budgeted at roughly 12 minutes each, plus the Investigative Task at around 30 minutes. The free-response rubric is built around communication, not just a final number: you are scored on justifying a conclusion with evidence from the data, interpreting a result correctly in context, and stating the conditions or assumptions a procedure depends on. A correct number with a missing or wrong justification consistently loses points, which is why every free response in this course, from week 1, is written to that standard rather than left as bare arithmetic.

Q: What is the hybrid digital format of the exam?
A: The multiple-choice section and the free-response prompts are both delivered digitally through the College Board's Bluebook application. However, your free-response answers themselves are handwritten in a paper answer booklet, which is collected separately and scored by hand. This mixed format catches students who have only ever practiced on paper or only ever practiced on screen, so our mock exams in weeks 23 and 24 are run under the same conditions: prompts read digitally, answers written out by hand, so test day is not the first time you have worked this way.

Q: How much maths do I need coming in?
A: Comfortable school algebra: rearranging a formula, working with fractions and percentages, and reading a graph. There is no calculus anywhere in AP Statistics, which surprises some students expecting a more advanced maths course. Where a result would traditionally require calculus to fully derive, the course demonstrates it through simulation and the graphing calculator instead, which is also how the AP exam itself expects the material to be handled.

Q: Will this get me a 5, or college credit?
A: No honest course can promise a specific score. AP exams are graded 1 to 5, and your result depends on your starting point, the hours you put in, and your performance on the day itself. What we can promise is honest: live teaching to the current nine-unit exam, small batches, real doubt support, free-response drilling against the official rubric, dedicated Investigative Task practice, and two full timed mock exams, plus a clear, evidence-based read on a realistic target score for you. Whether a given score earns college credit is decided by each university individually, not by us or by the College Board, so check the specific policy of the colleges you are applying to.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Is this course affiliated with the College Board?
A: No. Modern Age Coders is an independent education provider and is not affiliated with, authorised by, or endorsed by the College Board. AP and Advanced Placement are trademarks of the College Board, used here only to describe the exam this course prepares students for. For official course descriptions, syllabus updates and exam information, students should always refer to the College Board and AP Central directly.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ap-statistics-maths-exam-prep-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at

https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 22 OF 137: APP DEVELOPMENT COURSE: FLUTTER, IOS & ANDROID
==========

Course link: https://learn.modernagecoders.com/courses/complete-app-development-masterclass-college
Subject area: App Development
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

App development course for college students and professionals: Flutter as the production stack, Firebase and backends, then native literacy in Swift and Kotlin, in 10-12 months.

In one line: The whole app landscape in one honest arc: Flutter as your production stack, real backends, then working literacy in native iOS and Android.

ABOUT THE PROGRAMME

App development hiring wants a specific shape of developer: deep in one cross-platform stack that ships products fast, and literate enough in native iOS and Android to go deeper when a product demands it. This live online masterclass builds exactly that shape: Flutter and Dart as the production road, UI, state management, navigation done properly, then the product layer, storage, REST APIs, Firebase, app backends with Node, testing, plus honest survey weeks of React Native and PWAs so the alternatives are understood rather than feared. Then the native literacy phase: a real month in Swift with SwiftUI, a real month in Kotlin with Jetpack Compose. A finale month ships a capstone app and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- One production stack, not a four-framework blur: Flutter carries the building; native months add literacy
- The product layer included: storage, APIs, Firebase and app backends, where real apps live
- Native literacy on both platforms: Swift with SwiftUI, Kotlin with Compose, taught for judgment
- Alternatives surveyed honestly: React Native and PWAs explained, so stack choices are informed
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Months 1-3): programming, app design and Flutter fundamentals
Phase 2: The product layer (Months 4-6): storage, APIs, Firebase, backends and testing
Phase 3: Native literacy (Months 7-9): a real month each in Swift/SwiftUI and Kotlin/Compose
Phase 4: The finale (Month 10): the capstone app and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Everything is taught from the first line
Equipment: Any reasonably recent computer; Flutter, Android tooling and every tool used are free. The iOS month is structured so learners without a Mac follow fully (details in the FAQ)
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

College students: Students who want the complete app-developer profile, not one framework
Career switchers: Professionals entering mobile with the market-shaped skill set
Product builders: Founders and makers who need apps shipped and stacks chosen wisely
Generalists: Developers who want to understand the whole mobile landscape properly
Freshers: Graduates who want portfolio apps plus the judgment interviews probe

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS & FIRST APPS (MONTHS 1-3, WEEKS 1-13)

Build programming foundations, master UI/UX design principles, and create your first mobile applications.

Months 1-2: Programming Basics & App Fundamentals (Weeks 1-8)

Introduction to App Development & Setup

Topics:

- Mobile app ecosystem overview
- iOS vs Android vs Cross-platform
- Types of apps: native, hybrid, web apps
- App development career paths
- Development environment setup
- Installing necessary tools and IDEs
- Understanding app architecture
- Mobile operating systems basics
- App lifecycle and states
- Screen sizes and resolutions
- Touch interactions and gestures
- Mobile-first design thinking
- Your first app ideas brainstorming
- App development process overview
- Version control with Git basics

Projects built:

- Environment setup checklist
- Simple 'Hello World' app
- App idea documentation
- Wireframe sketches (paper)
- Git repository creation

Practice: Research 20 successful apps, analyze their features

Programming Fundamentals (Dart/JavaScript)

Topics:

- Why Dart and JavaScript for apps?
- Variables and data types
- Operators: arithmetic, comparison, logical
- Strings and string manipulation
- Numbers and math operations
- Booleans and conditions
- If-else statements
- Switch statements
- Loops: for, while, do-while
- Lists/Arrays basics
- Maps/Objects basics
- Functions: declaration and calling
- Parameters and return values
- Arrow functions (JavaScript) / Fat arrow (Dart)
- Basic debugging techniques

Projects built:

- Calculator logic
- Temperature converter
- Todo list logic (console)
- Quiz game logic
- BMI calculator logic
- Simple games (number guessing)

Practice: Solve 100 basic programming problems

UI/UX Design for App Developers

Topics:

- UI vs UX: understanding the difference
- Mobile UI design principles
- Material Design (Android) guidelines
- Human Interface Guidelines (iOS)
- Color theory for apps
- Typography in mobile apps
- Icons and iconography
- Spacing and layout grids
- Visual hierarchy
- Navigation patterns: tabs, drawers, bottom navigation
- Common UI components: buttons, cards, lists
- Touch targets and accessibility
- Figma for app design (basics)
- Creating wireframes and mockups
- Prototyping interactive designs

Projects built:

- Design system in Figma
- App UI mockups (5 different apps)
- Icon set creation
- User flow diagrams
- Interactive prototype
- Style guide document

Practice: Design UI for 10 different app types

Introduction to Flutter Development

Topics:

- What is Flutter? Why Flutter?
- Flutter installation and setup
- Dart language review for Flutter
- Flutter architecture and widgets
- Stateless vs Stateful widgets
- Widget tree concept
- Common widgets: Text, Container, Row, Column
- Scaffold and AppBar
- Buttons: ElevatedButton, TextButton, IconButton
- Images and assets
- ListView and ListTile
- GridView for grid layouts
- Stack for layered layouts
- Padding and margin
- Hot reload and hot restart

Projects built:

- Hello Flutter app
- Personal card app
- BMI calculator app (UI + logic)
- Dice rolling app
- Quote of the day app
- Simple calculator app

Practice: Build 10 simple Flutter apps

Month 3: Intermediate Flutter & State Management (Weeks 9-13)

Advanced Flutter Widgets & Layouts

Topics:

- Custom widgets creation
- Widget composition
- Form widgets: TextField, Form, TextFormField
- Input validation
- DropdownButton and menus
- Date and time pickers
- Sliders and switches
- Checkbox and radio buttons
- Dialogs and bottom sheets
- SnackBar and notifications
- TabBar and TabBarView
- PageView for swiping
- Card and material design
- Themes and theming
- Dark mode implementation

Projects built:

- Todo app with add/delete
- Expense tracker app
- Form-based registration app
- Settings page with preferences
- Multi-page app with tabs
- Dark mode enabled app

Practice: Create 15 different UI screens

State Management in Flutter

Topics:

- Understanding state in Flutter
- setState and local state
- InheritedWidget basics
- Provider package introduction
- ChangeNotifier and ChangeNotifierProvider
- Consumer and Selector widgets
- Multiple providers
- Riverpod introduction (modern alternative)
- BLoC pattern basics
- GetX state management
- Choosing the right state management
- Global state vs local state
- State persistence basics
- Debugging state issues
- Best practices for state management

Projects built:

- Counter app with Provider
- Shopping cart with state management

- Todo app with Provider
- Theme switcher with Provider
- Multi-screen state sharing
- Complete app with Riverpod

Practice: Refactor 10 apps with proper state management

Navigation & Routing

Topics:

- Navigation basics in Flutter
- Navigator.push and pop
- Named routes
- Passing data between screens
- Returning data from screens
- Navigation with arguments
- Bottom navigation bar implementation
- Drawer navigation
- Tab navigation
- Deep linking basics
- GoRouter package
- Navigation 2.0 introduction
- Nested navigation
- Route guards and authentication flow
- Transition animations

Projects built:

- Multi-screen app with navigation
- Bottom nav app (3+ screens)
- Drawer menu app
- Login flow with navigation
- PHASE 1 MINI CAPSTONE: Complete Multi-screen App
- E.g., Recipe app, News app, or Portfolio app

Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 2: CROSS-PLATFORM DEVELOPMENT & BACKEND INTEGRATION (MONTHS 4-6, WEEKS 14-26)

Master cross-platform development, backend integration, APIs, databases, and real-world app features.

Months 4-5: Backend Integration & Data Persistence (Weeks 14-21)

Local Data Storage

Topics:

- Data persistence importance
- SharedPreferences for simple data
- Key-value storage patterns
- SQLite database basics
- sqflite package in Flutter
- CRUD operations with SQLite
- Database schema design
- Migrations and versioning
- Hive (NoSQL) database
- Sembast for Flutter
- Choosing the right storage solution
- Data encryption basics
- Secure storage for sensitive data
- File storage and management
- Cache management

Projects built:

- Notes app with SQLite
- Todo app with local database
- Expense tracker with Hive
- User preferences with SharedPreferences
- Offline-first app
- Encrypted data storage app

Practice: Build 10 apps with different storage methods

REST APIs & HTTP Networking

Topics:

- Understanding REST APIs
- HTTP methods: GET, POST, PUT, DELETE
- API endpoints and routes
- HTTP package in Flutter
- Dio package for advanced networking
- Making GET requests
- Making POST requests with body
- Headers and authentication tokens
- Query parameters
- JSON parsing in Dart
- Serialization and deserialization
- Error handling in API calls
- Loading states and UX

    - Retry logic and timeout
    - API rate limiting handling
  Projects built:
    - Weather app with API
    - News app (NewsAPI integration)
    - Movie database app (TMDB API)
    - Currency converter with live rates
    - Quote app with API
    - GitHub profile viewer
    - Restaurant finder (Google Places API)
  Practice: Integrate 15 different public APIs
Firebase Integration
  Topics:
    - Firebase introduction and setup
    - Firebase project creation
    - Firebase Authentication: email/password
    - Google Sign-In integration
    - Phone authentication
    - Social logins: Facebook, Apple
    - Firestore database (NoSQL)
    - Firestore CRUD operations
    - Real-time listeners
    - Firestore queries and filtering
    - Cloud Storage for files
    - Image upload and retrieval
    - Firebase Cloud Messaging (FCM)
    - Push notifications implementation
    - Firebase Analytics basics
  Projects built:
    - User authentication app
    - Chat application with Firestore
    - Social media app (basic)
    - File sharing app
    - Real-time todo app
    - Push notification app
    - Complete Firebase-backed app
  Practice: Build 10 Firebase-integrated apps
Advanced App Features
  Topics:
    - Camera and photo library access
    - image_picker package
    - Taking photos and videos
    - Location services and GPS
    - Google Maps integration
    - Displaying maps and markers
    - Geolocation and tracking
    - QR code scanning and generation
    - Biometric authentication (fingerprint, face)
    - In-app browser (WebView)
    - Audio playback and recording
    - Video playback
    - Sensors: accelerometer, gyroscope
    - Background tasks
    - App shortcuts and widgets (home screen)
  Projects built:
    - Instagram-like photo app
    - Location tracking app
    - Maps-based app (find nearby)
    - QR code scanner app
    - Music player app
    - Fitness tracker (basic)
    - AR basics with camera
  Practice: Implement 15 advanced features across apps
Animations & Advanced UI
  Topics:
    - Implicit animations in Flutter
    - AnimatedContainer, AnimatedOpacity
    - TweenAnimationBuilder
    - Explicit animations with AnimationController
    - Animation curves
    - Hero animations
    - Page transitions
    - Custom animations
    - Lottie animations integration
    - Rive animations (interactive)
    - Shimmer effects for loading
    - Custom painters for graphics
    - Gesture detectors and interactions
    - Draggable widgets

- Performance optimization for animations
  Projects built:
- Onboarding screens with animations
- Animated app intro
- Loading animations collection
- Interactive animation app
- Animated charts and graphs
- Game with animations (simple)
- Beautiful UI app with micro-interactions
  Practice: Add animations to 10 previous apps

Month 6: Alternative Stacks: React Native, PWAs & App Backends (Weeks 22-26)
React Native Introduction
Topics:
- What is React Native?
- React Native vs Flutter comparison
- React Native setup and CLI
- JSX syntax
- Components in React Native
- Functional vs Class components
- React Hooks: useState, useEffect
- Props and state
- Styling in React Native
- Flexbox layouts
- Common components: View, Text, Image, ScrollView
- FlatList and SectionList
- TextInput and forms
- TouchableOpacity and buttons
- Navigation with React Navigation
  Projects built:
- Hello React Native app
- Todo app in React Native
- Weather app with API
- Navigation app (multi-screen)
- Form handling app
- List-based app (contacts, recipes)
  Practice: Build 10 React Native apps
Progressive Web Apps (PWA)
Topics:
- What are Progressive Web Apps?
- PWA vs native apps
- Service workers
- Offline functionality
- App manifest
- Add to home screen
- Push notifications in PWA
- Responsive design for PWA
- PWA frameworks: React, Vue, Angular
- Flutter for web (PWA)
- Testing PWA on mobile
- PWA deployment
- App-like experience on web
- Performance optimization
- PWA best practices
  Projects built:
- Simple PWA with offline support
- Todo PWA
- News PWA with caching
- Flutter web app as PWA
- E-commerce PWA (basic)
- Social media PWA
  Practice: Convert 5 apps to PWAs
Backend Development for Apps (Node.js/Express)
Topics:
- Why build your own backend?
- Node.js and Express basics
- RESTful API design for mobile
- Database integration: MongoDB/PostgreSQL
- User authentication (JWT)
- CRUD API endpoints
- File upload handling
- Input validation and sanitization
- Error handling in APIs
- API security best practices
- Environment variables
- CORS configuration
- Rate limiting
- API documentation
- Deployment basics (Render, Railway)

Projects built:
- Simple REST API for todo app
- User authentication API
- Blog API with CRUD
- Image upload API
- Social media backend (basic)
- E-commerce backend API
- Complete backend for Flutter app

Practice: Build backends for 8 different apps

Testing & Quality Assurance

Topics:
- Importance of testing in apps
- Unit testing in Flutter
- Widget testing
- Integration testing
- Test-driven development (TDD) basics
- Mocking and stubbing
- Testing API calls
- Testing state management
- UI testing with Flutter
- Automated testing
- Testing in React Native
- Debugging techniques
- Performance testing
- Beta testing preparation
- User acceptance testing

Projects built:
- Test suite for calculator app
- Widget tests for UI components
- Integration tests for complete app
- TDD for new feature
- Automated testing pipeline
- Bug tracking and fixing

Practice: Write tests for all major apps

Phase 2 Capstone Project

Topics:
- Full-stack app development
- Custom backend API
- Advanced features integration
- State management
- Testing coverage
- Performance optimization

Projects built:
- PHASE 2 CAPSTONE: Complete Social Media App
- Features: Auth, posts, comments, likes, user profiles, image upload, real-time updates, push notifications
- Alternative: E-commerce app with cart, checkout, orders
- Alternative: Fitness/Health tracking app with charts
- Alternative: Chat application with Firebase
- Requirements: Backend integration, 10+ screens, polished UI, tested

Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 3: NATIVE DEVELOPMENT & ADVANCED TOPICS (MONTHS 7-9, WEEKS 27-39)

Master native iOS and Android development, advanced app features, and platform-specific capabilities.

Months 7-8: iOS Development with Swift (Weeks 27-34)

Swift Programming & iOS Basics

Topics:
- Introduction to iOS development
- Xcode installation and setup
- Swift programming language basics
- Variables, constants, and types
- Optionals and optional binding
- Collections: Arrays, Dictionaries, Sets
- Control flow in Swift
- Functions and closures
- Object-oriented programming in Swift
- Classes and structures
- Protocols and extensions
- iOS app architecture
- Storyboards vs SwiftUI
- View controllers lifecycle
- Interface Builder basics

Projects built:
- Swift programming exercises

```
    - Hello iOS app
    - Calculator app in Swift
    - Simple UI app with storyboards
    - Multi-screen app with segues
  Practice: Solve 50 Swift programming problems
UIKit Development
  Topics:
    - UIKit framework overview
    - UIViewController and view lifecycle
    - UIView and subviews
    - Auto Layout and constraints
    - Stack views for layouts
    - UITableView and UITableViewCell
    - UICollectionView
    - Navigation Controller
    - Tab Bar Controller
    - UIAlertController and dialogs
    - UITextField and forms
    - UIImageView and image handling
    - Gesture recognizers
    - Delegates and protocols
    - Programmatic UI vs Storyboards
  Projects built:
    - Todo app with UITableView
    - Photo gallery with UICollectionView
    - Multi-screen app with navigation
    - Tab-based app
    - Form-based app with validation
    - Custom table view cells
  Practice: Build 12 UIKit apps
SwiftUI (Modern iOS Development)
  Topics:
    - SwiftUI introduction
    - Declarative UI paradigm
    - Views and modifiers
    - State and @State property wrapper
    - @Binding for two-way binding
    - @ObservedObject and @StateObject
    - @EnvironmentObject for global state
    - Lists and ForEach
    - Navigation in SwiftUI
    - NavigationView and NavigationLink
    - TabView for tabs
    - Sheets and fullscreen covers
    - Alerts and action sheets
    - Animations in SwiftUI
    - Combine framework basics
  Projects built:
    - SwiftUI todo app
    - Weather app with SwiftUI
    - MVVM architecture app
    - List-detail navigation app
    - Form app with SwiftUI
    - Animated UI app
    - Complete SwiftUI application
  Practice: Build 15 SwiftUI apps
iOS Data & Networking
  Topics:
    - UserDefaults for simple storage
    - Core Data for local database
    - Core Data entities and relationships
    - Fetch requests and predicates
    - URLSession for networking
    - Codable protocol for JSON
    - REST API integration in iOS
    - Async/await in Swift
    - Combine for reactive programming
    - Error handling in networking
    - Image caching
    - CloudKit basics
    - iCloud synchronization
    - Keychain for secure storage
    - File management in iOS
  Projects built:
    - Notes app with Core Data
    - API-based news app
    - Weather app with networking
    - Image downloading and caching
    - Secure password manager
```

- Cloud-synced todo app
- Complete data-driven app

Practice: Build 10 data-intensive iOS apps

iOS Advanced Features

Topics:
- Camera and photo library
- Core Location and MapKit
- Push notifications (local and remote)
- Background tasks
- App extensions (Today, Share, Keyboard)
- SiriKit integration
- HealthKit basics
- ARKit for augmented reality
- Core ML for machine learning
- App clips
- Widgets (WidgetKit)
- In-app purchases (StoreKit)
- Biometric authentication (Face ID, Touch ID)
- Universal links and deep linking
- Accessibility in iOS apps

Projects built:
- Camera app with filters
- Location-based app with maps
- Push notification app
- Widget for iOS 14+
- AR experience app (basic)
- Health tracking app
- App with in-app purchases

Practice: Implement 10 advanced iOS features

Month 9: Android Development with Kotlin (Weeks 35-39)

Kotlin Programming & Android Basics

Topics:
- Introduction to Android development
- Android Studio setup
- Kotlin programming basics
- Variables: val vs var
- Data types and type inference
- Null safety in Kotlin
- Collections in Kotlin
- Control flow: if, when, loops
- Functions and lambdas
- Object-oriented Kotlin
- Classes, objects, and data classes
- Android project structure
- Activities and their lifecycle
- Fragments basics
- AndroidManifest.xml

Projects built:
- Kotlin programming exercises
- Hello Android app
- Simple calculator app
- Activity lifecycle app
- Multi-activity app with intents

Practice: Solve 50 Kotlin programming problems

Android UI Development

Topics:
- XML layouts in Android
- LinearLayout, RelativeLayout, ConstraintLayout
- RecyclerView and adapters
- ViewHolder pattern
- ListView (legacy)
- Material Design components
- CardView and material cards
- Bottom navigation
- Navigation drawer
- Floating action button
- Snackbar and Toast
- Dialogs and AlertDialog
- Menus and toolbars
- Styles and themes
- Vector drawables and icons

Projects built:
- RecyclerView list app
- Material Design app
- Bottom navigation app
- Drawer navigation app
- Custom adapter app
- Theming and styling app

```
    Practice: Build 12 Android UI apps
  Jetpack Compose (Modern Android UI)
    Topics:
      - Jetpack Compose introduction
      - Composable functions
      - State management in Compose
      - remember and mutableStateOf
      - Layouts: Column, Row, Box
      - LazyColumn and LazyRow
      - Modifiers in Compose
      - Material Design 3 in Compose
      - Navigation in Compose
      - ViewModel with Compose
      - Side effects and LaunchedEffect
      - Theming in Compose
      - Animations in Compose
      - Compose vs XML comparison
      - Interoperability with XML
    Projects built:
      - Compose todo app
      - MVVM app with Compose
      - Navigation app in Compose
      - Animated Compose app
      - Material 3 design app
      - Complete Compose application
    Practice: Build 15 Jetpack Compose apps
  Android Data & Architecture
    Topics:
      - SharedPreferences in Android
      - Room database (SQLite ORM)
      - Room entities, DAOs, and database
      - LiveData for observable data
      - ViewModel for UI state
      - Repository pattern
      - MVVM architecture
      - Retrofit for networking
      - Gson/Moshi for JSON parsing
      - Coroutines for async operations
      - Flow for reactive streams
      - Dependency injection with Hilt/Dagger
      - WorkManager for background tasks
      - DataStore (modern preferences)
      - Architecture components
    Projects built:
      - Notes app with Room
      - MVVM todo app
      - API-based app with Retrofit
      - Repository pattern implementation
      - Coroutines and Flow app
      - Dependency injection app
      - Background task app
    Practice: Build 10 architecture-compliant apps
  Android Advanced & Phase 3 Capstone
    Topics:
      - Camera and photos in Android
      - Location services and Google Maps
      - Firebase Cloud Messaging
      - Push notifications in Android
      - Services: foreground, background
      - Broadcast receivers
      - Content providers
      - Permissions in Android
      - Biometric authentication
      - In-app billing
      - Android Jetpack libraries
      - Accessibility in Android
      - App widgets
      - Deep linking
    Projects built:
      - PHASE 3 CAPSTONE: Native iOS + Android App
      - Build same app for both iOS (Swift/SwiftUI) and Android (Kotlin/Compose)
      - Option 1: Fitness tracking app
      - Option 2: E-commerce app
      - Option 3: Social media app
      - Requirements: Native code, platform-specific features, polished UI, backend integrated
    Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing
    this phase with everything before it

---------------------------------------------------------------
```

PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)

The capstone month: spec, build, ship and defend, then the final exam that decides the certificate.

Month 10: Capstone and the Final Exam (Weeks 40-44)
Capstone Spec and Setup
Topics:
- Choosing the capstone: a complete Flutter app with Firebase and a real backend, or a cross-platform product with one native companion feature
- A one-page spec: scope, milestones, stack
- Repository and skeleton ready on day one
- Instructor sign-off before the build begins
Projects built:
- Approved capstone spec plus the running skeleton
Practice: Pitch the spec to the batch and absorb one hard question
Capstone Build
Topics:
- Two focused weeks in vertical slices
- Instructor checkpoints and honest scope cuts
- Using the whole course: Flutter depth, the product layer, and the platform judgment from the native months
- Daily commits and an always-working build
Projects built:
- The capstone, feature-complete with version control
Practice: A working build at the end of every session
Ship and Polish
Topics:
- The finishing pass: edge cases, errors, documentation
- Store-discipline pass: release builds, signing and checklists understood on both platforms
- A README worth reading and a demo worth watching
- A full-course spiral review before the exam
Projects built:
- The capstone shipped, documented and rehearsed
Practice: Two mock demos, each tighter than the last
Demo Day and the Final Exam
Topics:
- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: mobile development roles, or single-stack depth in Flutter, Swift or Kotlin
- Where our other courses continue the journey
Projects built:
- Demo day presentation delivered to the batch
Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: first Flutter apps with real layouts and state
- Phase 2: apps with storage, APIs, Firebase and their own backends
- Phase 3: native mini-apps in SwiftUI and Jetpack Compose
- Phase 4: the capstone app, shipped and defended

Total projects: 20+ real builds across Flutter and both native platforms, crowned by a shipped capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Flutter and Dart at production depth: UI, state and navigation
- The product layer: local storage, REST APIs, Firebase, app backends
- Native literacy: Swift with SwiftUI, Kotlin with Jetpack Compose
- Stack judgment: cross-platform versus native versus PWA, decided like an engineer
- Quality habits: testing, release discipline and store requirements

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short
    and there is focused revision and a free retest: the certificate certifies knowledge, not
    attendance

WHERE THIS CAN LEAD

  - Mobile development roles: the cross-platform-plus-literacy profile is exactly the ask
  - Single-stack depth: our dedicated Flutter, Swift and Kotlin courses continue each road
  - Product building: ship both stores with informed architecture choices
  - Backend depth for the server half of app products

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What does this app development course cover?
  A: Flutter and Dart as the production stack: UI, state management and navigation properly,
  then the product layer, local storage, REST APIs, Firebase, app backends with Node, testing,
  plus honest survey weeks of React Native and PWAs. A finale month ships a capstone app and
  sits the final exam.

  Q: How long does it take to learn app development properly?
  A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two
  more months in hand for learners who need them. The three teaching phases build the skill and
  its real-world application; the finale month ships a capstone and sits the final exam.

  Q: How am I assessed? Is the certificate just for attending?
  A: No. Problem sets follow every class with written feedback, monthly reviews deliberately
  mix new material with earlier topics, each phase ends with a cumulative exam, and the course
  ends with a final exam: a practical build plus written questions from the whole syllabus.
  Passing earns the certificate; falling short earns focused revision and a free retest.

  Q: Why Flutter first, and then native months?
  A: Because that is the shape hiring actually wants: deep in one cross-platform stack that
  ships products fast, plus enough native fluency to read platform documentation, use platform
  channels and know when a feature demands going native. Four frameworks at equal depth in one
  course would be a tour, not a skill; one deep stack plus two literacy months is a profile.

  Q: Do I need a Mac?
  A: Not to take the course. Flutter, Android and the backend work run on any computer, and
  that is where your hands-on depth lives. The Swift month is the one place Apple's rules bite:
  full iOS building needs a Mac, so learners without one follow the instructor's live Xcode
  session and practice Swift itself in browser playgrounds, while learners with a Mac run
  everything locally. We say this upfront rather than let it surprise you in month seven.

  Q: Can I try a class before enrolling?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or
  message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with
  no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at
  https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor reading your actual code, never
  pre-recorded videos.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/complete-app-development-masterclass-college
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
COURSE 23 OF 137: APP DEVELOPMENT FOR TEENS: BUILD REAL APPS WITH REACT NATIVE
==

Course link: https://learn.modernagecoders.com/courses/complete-app-development-masterclass-for-teens
Subject area: App Development
Designed for: Teens
Who can join: Absolute beginner, ages 13 to 18
Course length: 10-12 months (40-48 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one
  live class a week, four a month; one to one outside India two live classes a week, eight a

month. Each class is one hour.

WHAT THIS COURSE IS

App development course for teens aged 13-18: JavaScript, React and React Native with Expo, Firebase, AI features and a launched capstone app in 10-12 months.

In one line: The phone in their pocket becomes the thing they build for: real apps, installed on real devices, shipped with one honest stack in ten months.

ABOUT THE PROGRAMME

Every teenager lives inside apps; very few know they could build one this year. This live online course takes students aged 13 to 18 from zero to a real, installable, launched mobile app, using the path the modern industry actually recommends for one-person teams: JavaScript learned properly, React for interface thinking, then React Native with Expo, one codebase that runs on both Android and iOS, with Firebase as the backend and AI features where they genuinely help. The stack choice is the honest part. Courses that tour four frameworks, Kotlin, Swift, React Native and Flutter, produce teens fluent in none of them. This course masters one: React Native powers apps at Meta, Microsoft, Shopify and thousands of startups, works fully on a Windows laptop, and lets a teen carry their JavaScript skills from web to phone. Flutter gets an honest taste week, and native Android and iOS get a clear map, what they are, when they matter, and where our Kotlin and Swift courses continue. The pace is real: ten months at two live classes a week plus three to four hours of practice, with two more months in hand when needed. Apps go onto real phones from month 3, the capstone is user-tested and distributed properly, and store publishing is explained exactly as it works for minors: developer accounts are parent-managed, and we walk families through it. Homework after every class, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest. Real apps, real users, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- One stack mastered, honestly chosen: JavaScript, React and React Native with Expo, the one-person-team path the industry actually uses
- Runs on any laptop: no Mac required for the course; iOS specifics and Apple's rules are explained honestly
- Apps on real phones from month 3: Expo puts student work on their own device almost immediately
- Store truth for minors: developer accounts are parent-managed and we walk families through exactly how distribution works
- AI features built in properly: server-side calls, streaming and guardrails inside the student's own app
- Real assessment: weekly homework, mixed-review assignments every month, and a final exam that decides the certificate, with focused revision and a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations to first app (Months 1-3): JavaScript, UI/UX, React, Firebase, then React Native and an app on your own phone
Phase 2: App craft (Months 4-6): data, device features, polish, analytics, store readiness, AI features and a polished second app
Phase 3: The wider craft (Months 7-8): the native map, notifications, team Git workflow, and the capstone designed and built
Phase 4: Launch (Months 9-10): hardening, distribution, real users, iteration, portfolio, and the final exam at demo day

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. JavaScript is taught from the first variable
Computer skills: Comfortable typing and managing files and folders
Equipment: Any computer with a stable internet connection, Windows is fully sufficient, plus an Android or iOS phone for testing their own apps
Age: 13 to 18. Batches are grouped by age and pace so the room always fits
Time commitment: 2 live classes plus 3-4 hours of practice a week

WHO THIS COURSE IS FOR

App dreamers: Teens who look at the apps they use daily and think: I could make one of those, and are right
Complete beginners: Students starting from zero who want one honest path instead of a four-framework tour
Makers: Teens who want their work living on real phones, theirs and their friends', not in a folder
Future engineers: Students eyeing computer science who want a shipped product behind their applications
Practical families: Parents who want the store-and-accounts reality for minors explained straight

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS TO FIRST APP (MONTHS 1-3, WEEKS 1-12)

JavaScript learned properly, interface thinking with React, and the jump to React Native: by the end of this phase an app the student built runs on their own phone.

Month 1: How Apps Are Built (Weeks 1-4)

The App World, Honestly Mapped

Topics:

- How apps changed the world: From Snake to TikTok
- Native vs Cross-platform vs Web apps explained
- iOS vs Android: The eternal rivalry
- Famous teen app developers who became millionaires
- Setting up your development environment
- Installing VS Code, Android Studio, Xcode preview
- Introduction to Terminal/Command Line
- Git and GitHub: Your code's backup and portfolio
- Creating developer accounts (Google, Apple developer preview)
- Your first HTML/CSS/JavaScript app
- How the App Store and Google Play work

Projects built:

- Research and present your favorite app's history
- Set up complete development environment
- Create your first web-based calculator app
- Design your dream app concept (mockup)

Practice: Daily: Analyze 3 apps you use - what makes them addictive?

JavaScript Essentials

Topics:

- JavaScript: The language that powers everything
- Variables: Storing user data and game scores
- Data types: Numbers, strings, booleans with app examples
- Arrays: Managing Instagram posts, Spotify playlists
- Objects: Describing user profiles, game characters
- Functions: Reusable code for app features
- Conditionals: Making apps smart with if/else
- Loops: Processing TikTok feeds, game levels
- DOM manipulation: Making apps interactive
- Event handling: Responding to taps and swipes
- Local storage: Saving app data offline
- JSON: The data format apps love

Projects built:

- To-do list app with local storage
- Quiz game with score tracking
- Weather app using API
- Instagram-style photo filter app (basic)

Practice: Complete 50 JavaScript challenges on CodeWars

UI and UX Fundamentals

Topics:

- What makes apps beautiful and addictive?
- Design thinking process: Empathize, Define, Ideate
- User research: Understanding your target users (teens!)
- Color psychology: Why Instagram is purple-pink
- Typography: Choosing fonts that don't suck
- Mobile UI patterns: Bottom nav, cards, modals
- Figma basics: Professional design tool
- Creating wireframes and mockups
- Prototyping: Making designs clickable
- Design systems: Consistency across your app
- Accessibility: Designing for everyone
- Dark mode: Because we all love it

Projects built:

- Redesign a popular app for better UX
- Create complete UI kit in Figma
- Design your social media app mockup
- Build clickable prototype

Practice: Daily UI challenge - create one UI element per day

Modern JavaScript

Topics:

- Modern JavaScript features apps need
- Arrow functions: Cleaner, cooler syntax
- Destructuring: Extracting data elegantly
- Spread operator: Copying and merging data
- Template literals: Dynamic content creation
- Promises: Handling async operations
- Async/await: Making async code readable
- Fetch API: Getting data from servers

```
      - Modules: Organizing code like a pro
      - Classes: Object-oriented programming
      - Error handling: When things go wrong
      - Regular expressions: Pattern matching magic
    Projects built:
      - Movie database app with TMDB API
      - Real-time chat app (web version)
      - Music player with Spotify Web API
      - Meme generator with API integration
    Practice: Refactor old projects with modern JavaScript
    Assessment: Month 1 check: a small interactive build live, plus a take-home assignment
    mixing everything from month 1

Month 2: React and the Backend Taste (Weeks 5-8)
  React Fundamentals
    Topics:
      - Why React? Facebook, Instagram, Netflix use it
      - Vite: Starting projects quickly
      - JSX: HTML in JavaScript (weird but awesome)
      - Components: Building blocks of apps
      - Props: Passing data between components
      - State: Making components remember things
      - Hooks: useState, useEffect explained simply
      - Event handling in React
      - Conditional rendering: Showing/hiding content
      - Lists and keys: Rendering feeds and grids
      - React DevTools: Debugging like a detective
      - Component lifecycle: Birth to death of components
    Projects built:
      - Instagram clone (feed, stories, posts)
      - Real-time scoreboard app
      - Recipe app with search and filters
      - Portfolio website in React
    Practice: Build 20 React components for different use cases
  State and Navigation Thinking
    Topics:
      - Managing complex state: When useState isn't enough
      - useContext: Sharing data across components
      - useReducer: Redux-like state management
      - Custom hooks: Creating your own superpowers
      - React Router: Multi-page apps
      - Navigation patterns: Tabs, drawers, stacks
      - Protected routes: Login-only pages
      - URL parameters: Dynamic pages
      - Local storage with React
      - Form handling and validation
      - Optimizing performance: React.memo, useMemo
      - Code splitting: Faster app loading
    Projects built:
      - Multi-page social media app
      - E-commerce app with cart
      - Blog platform with authentication
      - Task management app like Notion
    Practice: Add routing and state management to previous projects
  Progressive Web Apps: Ship Before the Stores
    Topics:
      - What are PWAs? Apps without app stores
      - Service workers: Offline functionality
      - Web app manifest: Installing on home screen
      - Push notifications for web apps
      - Caching strategies: Offline-first approach
      - Background sync: Uploading when online
      - PWA best practices
      - Testing PWAs on mobile devices
      - PWA vs native apps: Pros and cons
      - Converting React apps to PWAs
      - PWA deployment and hosting
      - Analytics and monitoring for PWAs
    Projects built:
      - Convert React app to PWA
      - Offline-first news app
      - PWA game that works offline
      - Push notification system
    Practice: Make all web apps installable as PWAs
  Firebase: A Backend Without a Server
    Topics:
      - Why apps need backends: Where data lives
      - Firebase: Google's backend for apps
      - Authentication: Sign up, login, social auth
```

- Firestore: Real-time NoSQL database
- Cloud Storage: Storing images and videos
- Cloud Functions: Server code without servers
- Real-time features: Live chat, notifications
- Security rules: Protecting user data
- Firebase Analytics: Understanding users
- Firebase Hosting: Deploying apps
- Cost optimization: Avoiding surprise bills
- Firebase alternatives: Supabase, Appwrite

Projects built:
- Chat app with Firebase
- Instagram-like app with storage
- Collaborative todo app
- Social authentication system

Practice: Add Firebase to 3 existing projects
Assessment: Month 2 check: a small React feature built live, with quiz questions reaching back to month 1

Month 3: React Native, and an App on Your Phone (Weeks 9-12)
React Native with Expo
Topics:
- React Native: Write once, run everywhere
- How Facebook built React Native
- Setting up development environment
- Expo vs React Native CLI
- Core components: View, Text, Image, ScrollView
- Styling with Flexbox
- Platform-specific code
- React Navigation: Routing for mobile
- Bottom tabs, stack, and drawer navigation
- Handling user input
- Lists with FlatList and SectionList
- Pull to refresh and infinite scroll
- Expo Go: your code running on your own phone in minutes

Projects built:
- Instagram clone in React Native
- Todo app with navigation
- Weather app with location
- News feed with API

Practice: Convert React web apps to React Native
Core Components and Native Feel
Topics:
- Native modules: Accessing device features
- Camera and image picker
- Geolocation and maps
- Push notifications setup
- AsyncStorage for persistence
- SQLite integration
- Animations with Animated API
- Gesture handler
- Lottie animations
- React Native Firebase
- Authentication flows
- Performance optimization

Projects built:
- Snapchat-like camera app
- Uber clone with maps
- Chat app with Firebase
- Fitness tracker with sensors

Practice: Add native features to all React Native apps
Styling and UI Libraries
Topics:
- Native Base components
- React Native Elements
- React Native Paper (Material Design)
- Custom component libraries
- Themed components
- Responsive design for tablets
- Safe area handling
- Keyboard avoiding views
- Modal and overlay patterns
- Splash screens and app icons
- Building custom UI components
- Micro-interactions and feedback

Projects built:
- E-commerce app with beautiful UI
- Social media app with themes
- Music player interface
- Dating app UI

```
      Practice: Create reusable component library
    Phase Capstone: First Real App
      Topics:
        - Scoping a first app worth finishing: one screenful of value
        - Navigation, state and styling brought together
        - Testing on your own device like a user, not a builder
        - Sharing the app with family through Expo
        - The finish-line checklist: icons, name, empty states
        - A cumulative review of the whole phase
      Projects built:
        - PHASE CAPSTONE: a complete small app, a habit tracker, quiz app or club companion, running on
          the student's own phone and shared with family
      Practice: Collect feedback from three real users and fix the most confusing thing
      Assessment: Phase 1 exam: app walkthrough plus a written mixed paper over months 1-3

----------------------------------------------------------------
PHASE 2: APP CRAFT (MONTHS 4-6, WEEKS 13-24)
----------------------------------------------------------------
What separates demos from products: data that persists, device features, polish, analytics,
store readiness, accessibility and AI features, ending in a polished second app.

  Month 4: Data and Device Powers (Weeks 13-16)
    State Management and Testing
      Topics:
        - Redux in React Native
        - Redux Toolkit setup
        - Context API for state
        - MobX state management
        - Zustand for simplicity
        - Redux Persist for offline
        - Testing React Native apps
        - Jest and React Native Testing Library
        - Detox for E2E testing
        - Debugging with Flipper
        - Performance profiling
        - Crash reporting with Sentry
      Projects built:
        - Complex app with Redux
        - Offline-first app
        - Test suite for existing app
        - Performance optimization project
      Practice: Add state management to all apps
    Data That Survives: Storage and Offline
      Topics:
        - AsyncStorage: small data that persists
        - SQLite on the device: real local databases
        - Syncing local data with Firebase
        - Offline-first thinking: apps that work in a lift
        - Loading, empty and error states as a discipline
        - Migrations: changing your data shape without losing users
        - Backups and the data users would cry to lose
      Projects built:
        - A notes or collection app that works fully offline and syncs when back
      Practice: Airplane-mode test your app and fix what breaks
    Camera, Location and Notifications
      Topics:
        - Permissions: asking honestly and handling refusal
        - Camera and image picker flows
        - Location: maps and geofencing basics
        - Local notifications that help instead of nag
        - Sensors: what the device knows
        - Privacy by design: collecting the minimum
        - Feature flags: shipping carefully
      Projects built:
        - A photo-journal or treasure-hunt app using camera and location
      Practice: Add one device feature to your phase 1 capstone
    Builds and Sharing
      Topics:
        - Building for production
        - Code signing for iOS
        - Android APK and AAB
        - Over-the-air updates with CodePush
        - CI/CD for React Native
        - Fastlane automation
        - App Store deployment
        - Play Store deployment
        - Beta testing with TestFlight/Play Console
        - Analytics integration
        - Crash analytics
```

```
      - Performance monitoring
    Projects built:
      - Deploy React Native app to both stores
      - Set up CI/CD pipeline
      - Implement OTA updates
      - Create deployment automation
    Practice: Deploy all React Native apps
    Assessment: Month 4 check: an offline-capable mini feature built live, plus a mixed
    quiz over earlier months

Month 5: The Product Layer (Weeks 17-20)
  Firebase in Depth: Auth and Firestore
    Topics:
      - Firebase deep dive
      - Supabase: Open source Firebase
      - AWS Amplify for mobile
      - Appwrite backend
      - Hasura GraphQL
      - Strapi headless CMS
      - Authentication services
      - Real-time synchronization
      - Offline-first architecture
      - Data syncing strategies
      - Cost optimization
      - Scaling considerations
    Projects built:
      - Multi-user collaboration app
      - Offline-first note app
      - Real-time gaming backend
      - Social platform with BaaS
    Practice: Compare different BaaS providers
  Animations and Gestures
    Topics:
      - Why motion matters: feedback, hierarchy, delight
      - Layout animations and transitions
      - Reanimated basics: 60fps as a habit
      - Gestures: swipe, drag and pull-to-refresh
      - Micro-interactions that make apps feel expensive
      - Restraint: when animation is noise
      - Performance: keeping motion smooth on cheap phones
    Projects built:
      - A polish pass: your best app re-animated tastefully
    Practice: Recreate two animations from apps you admire
  Analytics and Performance, Honestly
    Topics:
      - Google Analytics for apps
      - Firebase Analytics setup
      - Custom event tracking
      - User behavior analysis
      - Funnel optimization
      - A/B testing in apps
      - Performance monitoring
      - Crash reporting and debugging
      - User session recording
      - Heatmaps for mobile
      - Attribution tracking
      - Revenue analytics
    Projects built:
      - Analytics dashboard
      - A/B testing framework
      - Performance monitoring setup
      - User behavior tracker
    Practice: Add comprehensive analytics to all apps
  Store Readiness, Told Straight
    Topics:
      - How the Play Store and App Store actually work
      - Developer accounts for minors: parent-managed, explained step by step
      - Store listings: names, icons, screenshots that convert
      - Review guidelines: what gets apps rejected
      - Internal testing tracks and TestFlight-style distribution
      - Ratings, privacy labels and data disclosures
      - The costs, plainly: what Google and Apple charge
      - When a PWA is the smarter launch
    Projects built:
      - A complete store-ready listing pack for your app: icon, screenshots, copy
    Practice: Run your app against the store review checklist and fix the failures
    Assessment: Month 5 check: store-readiness review plus mixed questions from phases 1-2

Month 6: The Modern App (Weeks 21-24)
```

```
    AI Features in Your App
      Topics:
        - Calling an AI API from a mobile app
        - Why your API key must never ship in the app
        - Streaming responses so the UI does not freeze
        - On-device vs cloud inference, and the trade-off
        - Image and voice input as first-class features
        - Function calling: letting the model use your app's features
        - Building a small agent loop: plan, act, check
        - Handling refusals, timeouts and hallucinated answers
        - Caching and cost: what an AI feature actually costs per user
        - Rate limits and graceful degradation
        - Privacy: what leaves the device, and what must not
        - Telling users when they are talking to an AI
        - Accounts for AI services are set up with a parent where required
      Projects built:
        - AI assistant app
        - Photo-to-text app
        - Voice agent app
        - AI feature cost dashboard
      Practice: Add AI features to 3 apps
    Accessibility and App Safety
      Topics:
        - Accessibility: apps your grandmother and your classmate with a screen reader can use
        - Touch targets, contrast and dynamic text sizes
        - Security basics: secrets never ship in the app
        - Safe auth flows and session handling
        - Storing sensitive data properly
        - Rate limiting and abuse thinking, lightly
        - An empathy audit: using your app the hard way
      Projects built:
        - A full accessibility and safety pass on your best app, with a written report
      Practice: Navigate your app with the screen reader on
    An Honest Week of Flutter
      Topics:
        - Flutter: Google's UI toolkit
        - Why Flutter? Single codebase, native performance
        - Dart programming language basics
        - Flutter architecture overview
        - Setting up Flutter environment
        - Hot reload: Instant gratification
        - Widgets: Everything is a widget
        - Stateless vs Stateful widgets
        - Basic widgets: Container, Row, Column
        - Material Design and Cupertino widgets
        - Layout in Flutter
        - Building your first Flutter app
        - What transfers from React Native, and when Flutter is the better choice
      Projects built:
        - Flutter calculator app
        - Business card app
        - Dice game in Flutter
        - Quiz app with scoring
      Practice: Complete Flutter codelabs
    Phase 2 Capstone: The Polished App
      Topics:
        - Choosing the phase capstone: the app worth polishing to store-grade
        - Bringing it together: data, device features, polish, analytics, maybe AI
        - A real beta: five users, structured feedback
        - Iterating on feedback without losing the plot
        - A cumulative review of phases 1-2
      Projects built:
        - PHASE CAPSTONE: a polished, store-ready app with a real beta test behind it
      Practice: Ship two improvements straight from beta feedback
      Assessment: Phase 2 exam: capstone demo plus a written mixed paper spanning phases 1
      and 2

------------------------------------------------------------
PHASE 3: THE WIDER CRAFT AND THE CAPSTONE BUILD (MONTHS 7-8, WEEKS 25-32)
------------------------------------------------------------
The map of native development, notifications done right, professional Git workflow, and then
the main event: the final capstone designed and built.

  Month 7: The Wider World (Weeks 25-28)
    The Native Map: Kotlin and Swift, Honestly
      Topics:
        - What native development is and what it buys you
        - A guided tour of Android Studio and Kotlin, reading real code
        - A guided look at Swift and SwiftUI
```

- When React Native is enough, and when teams go native
- How native modules plug into React Native
- Where our Kotlin and Swift courses continue this road
- Reading a job market map without hype

Projects built:
- A one-page written comparison: your app in RN vs native, costs and benefits

Practice: Rebuild one tiny screen natively, just to feel the difference

Push Notifications and Background Work

Topics:
- Push notifications end to end with Expo
- Notification design: helpful vs spam, decided honestly
- Deep links: notifications that open the right screen
- Background tasks and their limits on mobile
- Scheduling and reminders done reliably
- Battery respect: being a good citizen on the device
- Measuring whether notifications actually help

Projects built:
- A reminder or club-alerts feature with tasteful push notifications

Practice: A/B your own notification copy on real users, kindly

Git and Team Workflow for Apps

Topics:
- Version control recap: the professional loop
- Branches and pull requests on a real app repo
- Code review: reading a teammate's screen code kindly
- Release branches and versioning apps
- Changelogs users can read
- CI checks that catch broken builds early
- Where our full Git and GitHub course goes deeper

Projects built:
- Your app repo run professionally for two weeks: branches, PRs, releases

Practice: Review a classmate's pull request properly

Mini Product Sprint

Topics:
- One week, one tiny product: scope ruthlessly
- From idea to installed app in five sessions
- Cutting features without cutting the point
- A retrospective that actually teaches
- Cumulative review before the capstone months

Projects built:
- A one-week micro-app, built solo and shipped to your own phone

Practice: Write the honest retrospective: what you would do differently

Assessment: Month 7 check: sprint demo plus a mixed quiz reaching across the course

Month 8: The Capstone, Designed and Built (Weeks 29-32)

Capstone Spec and Design Sprint

Topics:
- Choosing the capstone: the app you wish existed
- A one-page spec: users, screens, data, features
- Wireframes before pixels, pixels before code
- Data model and Firebase plan reviewed
- Milestones: working core by week 2, feature-complete by week 4
- Instructor sign-off before the build begins

Projects built:
- Signed-off capstone spec, wireframes and repository

Practice: Pitch the spec to the batch and absorb one hard question

Capstone Build

Topics:
- Two focused weeks of building in vertical slices
- Instructor checkpoints and course corrections
- Using everything: navigation, data, device features, polish, maybe AI
- Daily commits and an always-working build
- A mid-build hallway test with a real user
- Scope cuts made early and honestly

Projects built:
- The capstone app, feature-complete with version control and a working build every day

Practice: A working build at the end of every session, no exceptions

User Testing and Iteration

Topics:
- Structured beta: five real users, real tasks, silent observation
- Turning feedback into a ranked fix list
- Shipping the fixes that matter first
- Crash and edge-case hunting
- The capstone at v1.0: what that honestly means

Projects built:
- Capstone v1.0: beta-tested, iterated, stable

Practice: Close out the top five feedback items

Assessment: Phase 3 exam: capstone v1.0 walkthrough plus a written mixed paper over phases 1-3

PHASE 4: LAUNCH (MONTHS 9-10, WEEKS 33-40)

The last mile real developers respect: hardening, honest distribution, real users, iteration,
portfolio, and the final exam at demo day.

  Month 9: Ship It Properly (Weeks 33-36)
    Hardening Week
      Topics:
        - Crash hunting: making the app boringly reliable
        - Edge cases: bad network, low storage, old phones
        - Error reporting so you know when it breaks
        - Performance passes on real low-end devices
        - The pre-launch checklist run end to end
      Projects built:
        - A hardening report: found, fixed, verified
      Practice: Test on the oldest phone in the family
    Distribution, For Real
      Topics:
        - Production builds with EAS
        - Play Store internal and closed testing tracks, parent-managed
        - The Apple path explained honestly: accounts, costs, review
        - PWA and direct-install fallbacks when stores are not the right move
        - Privacy policies and data disclosures written plainly
        - Version 1.0.0: cutting the release
      Projects built:
        - The capstone distributed to its beta audience through a real channel
      Practice: Onboard five users through the real install flow
    Launch Week
      Topics:
        - The launch: capstone in real users' hands
        - Watching analytics without vanity
        - Support: handling the first bug report gracefully
        - Collecting reviews and feedback honestly
        - The first patch release, done calmly
      Projects built:
        - Launch executed, first patch shipped
      Practice: A launch journal: what happened vs what you expected
    Portfolio and Devlog
      Topics:
        - An app developer portfolio: installable links and honest screenshots
        - The capstone case study: problem, decisions, results
        - A devlog post worth reading
        - GitHub profile polish
        - Explaining one hard bug like an engineer
      Projects built:
        - Portfolio page with the capstone case study and install links
      Practice: Publish the devlog and share it with the batch
      Assessment: Month 9 check: portfolio review plus a cumulative mixed quiz

  Month 10: The Finale (Weeks 37-40)
    Iteration Sprint
      Topics:
        - Reading launch data: what users actually did
        - Choosing one meaningful improvement, not five shiny ones
        - Building and shipping it inside a week
        - Regression care: not breaking what worked
      Projects built:
        - One meaningful post-launch improvement, shipped
      Practice: Ship it and tell your users what changed
    Final Polish and Demo Preparation
      Topics:
        - The v1.1 polish pass: details users feel
        - Demo craft: the story of the app in four minutes
        - A full-course spiral review before the exam
        - Mock demos with brutal, kind feedback
        - The written-exam revision plan
      Projects built:
        - Capstone v1.1 plus a rehearsed demo
      Practice: Two mock demos, each tighter than the last
    Demo Day and What Comes Next
      Topics:
        - Presenting the capstone: live on a real phone, plus the hardest bug story
        - An honest skills inventory: strong, shaky, next
        - Where each path leads: Kotlin, Swift, full-stack web or DSA
        - Staying sharp: shipping small things often
      Projects built:
        - Demo day presentation delivered to the batch and parents
      Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed

from every phase. Passing earns the certificate; a student who falls short gets a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: JavaScript builds, React projects, an installable PWA, and a first real app on the student's own phone
- Phase 2: an offline-capable app, camera and location features, a store-ready listing pack, an AI feature, and a polished beta-tested app
- Phase 3: a native comparison study, push notifications, a one-week micro-app, and the capstone built to v1.0
- Phase 4: hardening, real distribution, a launch with real users, an iteration release, and the portfolio case study

Total projects: 20+ builds across the course, crowned by a launched, user-tested capstone app

SKILLS THE STUDENT WILL HAVE AT THE END

- JavaScript and React: the language and interface thinking modern apps run on
- React Native with Expo: one codebase on Android and iOS, from any laptop
- Data: local storage, SQLite, offline-first patterns and Firebase sync
- Device powers: camera, location, notifications and permissions handled respectfully
- Product craft: polish, animations, analytics, accessibility and app safety
- AI features: model calls from the app done safely with guardrails
- Shipping: builds, store processes told honestly, beta testing, launch and iteration
- Professional habits: Git workflow, code review, testing and honest scoping

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building along with the instructor
Homework: Homework after every class, and a mixed-review assignment at the end of every month so earlier skills stay sharp
Practice: 3-4 hours of building between classes
Review: Every submitted build reviewed with written feedback; recurring issues reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so nobody stays stuck on a red screen for days
Progress updates: Regular progress notes to parents: what was built, what comes next, and where encouragement at home helps

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Kotlin and Swift courses: native depth on either platform, from a real foundation
- Full-stack web development: the JavaScript and React here carry straight across
- Data Structures and Algorithms: the computer science core for engineering paths
- Real small apps: tools for family, school clubs and local teams, shipped for real
- College applications: a launched app with real users is a rare and honest differentiator

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: How long does it take a teenager to learn app development?
A: To a genuinely launched app: 10 months at 2 live classes a week plus 3 to 4 hours of practice. The syllabus keeps 2 months of margin in hand, students who need more repetition take up to 12 months with no rushing and no penalty. The first app lands on their own phone in month 3, so motivation never has to wait for the finish line.

Q: How is my teen assessed? Is the certificate just for attending?
A: No. Classwork runs inside every session, homework follows every class, and each month ends with an assignment that deliberately mixes new questions with earlier topics so knowledge compounds instead of fading. Every phase closes with a cumulative exam, and the course ends with a final exam: a practical build plus written questions drawn from the whole syllabus. Passing earns the certificate; falling short earns a focused revision plan and a free retest.

Q: Why React Native instead of Kotlin, Swift or Flutter?
A: Because one stack mastered beats four toured, and React Native is the honest choice for a one-person teen team: one codebase for Android and iOS, full development on a Windows laptop, JavaScript skills that transfer to and from the web, and real industry weight behind it. The course still gives Flutter an honest taste week and maps native Kotlin and Swift clearly, and

our dedicated Kotlin and Swift courses continue those roads for students who want native depth.

Q: Do we need a Mac or an iPhone?
A: No Mac is needed for the course: React Native with Expo runs fully on Windows, and cloud build services handle iOS packaging when needed. Any Android or iOS phone works for testing. The Apple publishing path, its account, cost and review process, is explained honestly in the store-readiness week for families who want to pursue it.

Q: Will my teen actually publish an app to the stores?
A: Here is the straight answer most courses avoid: store developer accounts have age and payment requirements, so for minors they are parent-managed, and we walk families through exactly that in the distribution weeks. Every student ships for real regardless: internal testing tracks, direct installs and PWA distribution put the capstone in real users' hands, and families who set up a parent-managed account can take it all the way to a public store listing.

Q: How is AI used in this course?
A: Two ways, both honest. Students build AI features into their own apps, model calls made safely, streaming responses, guardrails against misuse, with accounts set up with a parent where a service requires it. And they learn to use AI coding assistants the professional way: as accelerators for a developer who understands the code, never as a substitute for understanding it.

Q: Can my teen attend a free class first?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to sit in, and the equipment and accounts reality is explained right there. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small interactive groups with a real instructor, never pre-recorded videos. Mobile tooling changes fast and beginners hit unique errors; live teaching is the only format that handles both honestly.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/complete-app-development-masterclass-for-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 24 OF 137: APP MAKING FOR KIDS: BUILD REAL APPS WITH BLOCKS

==========

Course link: https://learn.modernagecoders.com/courses/block-coding-app-development-masterclass
Subject area: Kids Coding
Designed for: Kids
Who can join: Complete beginner, ages 8 to 12
Course length: 6-8 months (26-32 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

App development course for kids aged 8-12: build real mobile apps with block coding in live online classes, installed on the family phone, in 6-8 months.

In one line: The family phone becomes the launch device: real apps designed and built by the child, installed and used at home, in six honest months.

ABOUT THE PROGRAMME

Few moments beat a child opening an app on a real phone and saying "I made this". This live online course delivers that moment repeatedly for ages 8 to 12: visual programming logic learned through play, then real mobile apps built with block-based tools, screens designed on paper first, logic snapped together, data saved, and every build installed on the family phone for genuine daily use. The course keeps its promises honest: no app-store publishing pressure, no monetization talk for children, just skill and delight. Apps are shared safely with family through test links, and the course ends with a capstone, an app for the family, planned, built, tested by a real user and presented at demo day. Six months at two live

classes a week plus two to three hours of playful practice, with two more months in hand when needed. Homework after every class, a mixed challenge every month, and a final exam, capstone demo plus a friendly viva, that decides the certificate with a free retest. Real apps, real users, real pride.

WHAT MAKES THIS COURSE DIFFERENT

- Apps land on real phones: every month ends with something installed and used at home
- Design before blocks: children sketch screens on paper like real product designers
- No store pressure, no money talk: skill and joy are the whole syllabus at this age
- Family as users: testing with real people is taught as the heart of app making
- An honest 6-month arc with 2 months in hand
- Real assessment, gently done: monthly mixed challenges and a final capstone exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Visual logic (Months 1-2): the thinking behind every app, learned through play
Phase 2: Real mobile apps (Months 3-4): screens, buttons, data and sensors on real phones
Phase 3: Polish and the finale (Months 5-6): bigger builds, safe sharing, and the capstone exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Visual blocks make the start joyful
Reading: Comfortable reading simple instructions
Equipment: A computer or laptop with internet, plus any Android or iOS phone in the family for testing
Age: 8 to 12. Batches are grouped by age
Time commitment: 2 live classes plus 2-3 hours of playful practice a week

WHO THIS COURSE IS FOR

Idea kids: Children who keep saying "someone should make an app for that"
Complete beginners: Kids with zero coding background: blocks carry them
Scratch graduates: Children from Scratch who want their creations on a phone
Future builders: Kids who will grow into our teen app development course
Families: Households ready to be the world's kindest beta testers

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

THE COURSE: FROM LOGIC TO LAUNCHED-AT-HOME (MONTHS 1-6, WEEKS 1-26)

Visual programming logic, then real mobile apps built, tested by the family and presented at demo day.

Months 1-2: Visual Programming Fundamentals (Weeks 1-8)

Introduction to Block Coding & Scratch Basics

Topics:

- What is block-based programming?
- Why learn visual coding first?
- Introduction to Scratch interface
- Understanding sprites and stages
- Motion blocks: move, turn, go to, glide
- Looks blocks: say, think, show, hide, change costume
- Sound blocks: play sound, volume, tempo
- Events: when flag clicked, when key pressed
- Creating your first animation
- Saving and sharing projects
- Scratch community and remixing
- Basic debugging with blocks

Projects built:

- Animated greeting card
- Interactive story with characters
- Simple animation sequence
- Musical instrument app

Practice: Daily: Create one Scratch animation, explore 5 community projects

Programming Logic with Blocks

Topics:

- Variables: creating and using data storage
- Lists (arrays) in block coding
- Control blocks: forever, repeat, if-then
- If-then-else decision making
- Operators: math operations with blocks
- Comparison operators: greater than, less than, equals
- Logical operators: and, or, not

```
      - Random numbers and probability
      - Creating custom blocks (functions)
      - Parameters in custom blocks
      - Clone sprites for multiple objects
      - Broadcasting messages between sprites
    Projects built:
      - Calculator with visual blocks
      - Quiz game with scoring
      - Number guessing game
      - Drawing app with controls
    Practice: Solve 20 logic puzzles using block programming
    Assessment: Month 1 check: a small build or challenge live, plus a fun take-home mixing
    everything so far
  Game Development in Scratch
    Topics:
      - Game design principles
      - Player movement and controls
      - Collision detection with blocks
      - Score keeping and lives system
      - Timer and countdown features
      - Power-ups and bonuses
      - Level progression design
      - Game states: start, play, game over
      - Sound effects and background music
      - Particle effects with clones
      - Saving high scores with cloud variables
      - Multiplayer basics with cloud data
    Projects built:
      - Maze navigation game
      - Platformer game with jumping
      - Space shooter with enemies
      - Puzzle game with levels
    Practice: Create 5 different game mechanics
  Introduction to MIT App Inventor
    Topics:
      - MIT App Inventor interface tour
      - Designer vs Blocks view
      - Screen components and properties
      - Basic UI elements: buttons, labels, textboxes
      - Layout components: arrangements, tables
      - Connecting blocks to components
      - Event handlers in App Inventor
      - Testing apps with companion app
      - Building APK files
      - Screen navigation and multiple screens
      - Basic app architecture
      - Debugging techniques
    Projects built:
      - Hello World mobile app
      - Personal information form
      - Multi-screen navigation app
      - Simple calculator for phones
    Practice: Build 10 basic mobile app interfaces
    Assessment: Month 2 check: a live challenge with questions reaching back to month 1

Months 3-4: Mobile App Development (Weeks 9-17)
  Thunkable Platform Mastery
    Topics:
      - Thunkable vs MIT App Inventor
      - Cross-platform development (iOS & Android)
      - Thunkable interface and components
      - Advanced UI components in Thunkable
      - Data viewers: list and grid views
      - Tab navigator and drawer navigator
      - Working with images and media
      - Responsive design for different screens
      - Live testing on devices
      - Thunkable live app for testing
      - Component styling and themes
      - Custom colors and fonts
    Projects built:
      - Photo gallery app
      - Note-taking application
      - To-do list with categories
      - Recipe book app
    Practice: Create 5 different app layouts in Thunkable
  Working with Device Features
    Topics:
      - Camera component and photo capture
```

- Image picker from gallery
- Sound recorder and playback
- Text-to-speech functionality
- Speech recognition basics
- Location sensor and GPS
- Maps component integration
- Accelerometer for motion detection
- Gyroscope and orientation
- Vibration and haptic feedback
- QR code and barcode scanning
- Share functionality

Projects built:
- Camera app with filters
- Voice recorder with playback
- Location-based reminder app
- QR code scanner utility

Practice: Integrate 10 different device features

Data Storage and Persistence

Topics:
- Local storage with TinyDB
- Storing and retrieving data
- Lists and data structures
- Cloud variables for online storage
- Firebase realtime database basics
- Authentication with Firebase
- User accounts and profiles
- Storing user preferences
- Offline functionality
- Data synchronization
- Backup and restore features
- CSV file handling

Projects built:
- Diary app with local storage
- Expense tracker with categories
- Habit tracker with statistics
- Cloud-based notes app

Practice: Implement storage in 5 different app types

Advanced UI and User Experience

Topics:
- Animation blocks and transitions
- Timer-based animations
- Gesture recognition: swipe, pinch
- Custom components creation
- Reusable component blocks
- Advanced list viewers with templates
- Charts and data visualization
- Loading screens and progress bars
- Error handling and user feedback
- Accessibility features
- Multi-language support basics
- App theming and customization

Projects built:
- Animated story app
- Dashboard with charts
- Gesture-controlled game
- Customizable theme app

Practice: Create 10 custom UI components

Mid-Course Review and Showcase

Topics:
- Review of mobile app concepts
- Best practices recap
- Code organization with blocks
- Performance optimization
- Testing strategies

Projects built:
- CAPSTONE: Complete mobile app with all features
- Social media profile app
- Educational quiz app with levels
- Fitness tracking application

Assessment: Mid-course check: a showcase build plus a mixed challenge spanning months 1-4

Months 5-6: Advanced Features & Publishing (Weeks 18-26)

APIs and Web Services

Topics:
- What are APIs? Understanding web services
- Web API component in block platforms
- Making GET requests with blocks
- POST requests for sending data

- Working with JSON responses
- Parsing API data with blocks
- Weather API integration
- News API for content
- Translation API usage
- Image recognition APIs
- Error handling for API calls
- API keys and security

Projects built:

- Weather forecast app
- News reader application
- Language translator app
- Movie database browser

Practice: Integrate 10 different public APIs

Game Development Advanced

Topics:

- Physics simulation with blocks
- Sprite collision advanced techniques
- Particle systems for effects
- Leaderboards and achievements
- In-app currency systems
- Power-up and upgrade systems
- Enemy AI with blocks
- Procedural level generation
- Save game progress
- Multiplayer concepts
- Game balancing and testing
- Sound design for games

Projects built:

- Physics puzzle game
- RPG with inventory system
- Endless runner with upgrades
- Multiplayer quiz game

Practice: Develop 5 complete games with different mechanics

AI and Machine Learning Blocks

Topics:

- Introduction to AI in block coding
- Image classification with blocks
- Face detection and recognition
- Emotion detection from images
- Text sentiment analysis
- Chatbot creation with blocks
- Voice commands and control
- Teachable Machine integration
- Custom AI models with blocks
- Personal AI assistant app
- Computer vision applications
- Natural language processing basics

Projects built:

- AI photo classifier app
- Emotion detection camera
- Chatbot assistant app
- Voice-controlled smart app

Practice: Create 5 AI-powered applications

IoT and Hardware Integration

Topics:

- Internet of Things concepts
- Bluetooth connectivity blocks
- Connecting to Arduino with blocks
- micro:bit integration
- Controlling LEDs and motors
- Reading sensor data
- Home automation apps
- Robot control applications
- Wearable device apps
- MQTT protocol basics
- Cloud IoT platforms
- Real-time data monitoring

Projects built:

- Smart home controller app
- Robot remote control
- Plant watering system app
- Fitness band companion app

Practice: Build 5 IoT-connected applications

Capstone and the Final Exam

Topics:

- App store guidelines and policies
- Preparing apps for publication
- Creating app icons and screenshots

- Writing app descriptions
- Google Play Console setup
- iOS App Store submission
- Analytics and user tracking
- App updates and versioning
- Marketing your apps
- Building an app portfolio

Projects built:
- THE CAPSTONE: an app for the family, a chore chart, pet log, quiz or the child's own idea, installed on a real phone and presented at demo day

Assessment: FINAL EXAM: capstone demo plus a friendly viva and a mixed quiz from the whole course. Passing earns the certificate; a child who falls short gets focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Months 1-2: logic games and first interactive builds
- Months 3-4: real phone apps: soundboards, quizzes, drawing pads and trackers
- Months 5-6: bigger apps with saved data, then the family-app capstone at demo day

Total projects: 15+ working apps, crowned by a family-app capstone on a real phone

SKILLS THE STUDENT WILL HAVE AT THE END

- Visual programming: events, logic, loops and variables through blocks
- App design: screens sketched, layouts built, buttons that feel right
- Data: apps that remember, with saved lists and scores
- Device powers: camera, sounds and sensors used playfully and safely
- Product habits: testing with real users and improving from feedback

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building along with the teacher
Homework: A small build after every class, and a mixed challenge at the end of every month
Practice: 2-3 hours of playful practice between classes
Review: Every app reviewed with teacher feedback; wobbly skills reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Regular progress notes to parents, with the newest app to try installed

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam, a capstone demo plus a friendly viva. A child who falls short gets focused revision and a free retest

WHERE THIS CAN LEAD

- Kids Coding journey: the full block-to-code path
- Python for Kids: the typed-code step when ready
- App Development for Teens: React Native, from 13
- Web Development for Kids: a second builder skill
- The family app factory: every chore chart and quiz now gets an app

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What apps will my child actually build?
A: Fifteen-plus working apps across six months: soundboards, quizzes, drawing pads, trackers and games in the middle months, and a capstone, an app for the family such as a chore chart or pet log, installed on a real phone, tested by real family users and presented at demo day.

Q: Will the apps go on the App Store or Play Store?
A: No, and that is deliberate honesty: store publishing involves adult accounts, fees and review processes that do not belong in a course for 8-12 year olds. Apps are installed directly on family phones and shared safely through test links, which is everything a child needs for real users and real pride. Store publishing is taught honestly in our teen app course, where it belongs.

Q: How long does the course take?
A: Six months at 2 live classes a week plus 2 to 3 hours of playful practice takes a child from zero to the demo-day capstone in app making. The course keeps two more months in hand for children who need more repetition, no rushing and no penalty.

Q: How is my child assessed? Is the certificate just for attending?
A: No, and it is all gentle: classwork in every session, a small homework build after every class, a mixed challenge at the end of every month that revisits earlier skills, and a final

exam, a capstone demo plus a friendly viva and short quiz. Passing earns the certificate; a child who falls short gets focused revision and a free retest.

Q: What tools are used, and are they free?
A: Free block-based app builders in the MIT App Inventor family, run in the browser, with accounts set up alongside a parent. A computer for building and any family phone for testing is the whole kit.

Q: Is there a free trial class?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to sit in. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real teacher, never pre-recorded videos. Children build during class and get unstuck on the spot.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/block-coding-app-development-masterclass
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 25 OF 137: ARTIFICIAL INTELLIGENCE COURSE: SEARCH, LOGIC & AGENTS
==========

Course link: https://learn.modernagecoders.com/courses/artificial-intelligence-complete-masterclass-college
Subject area: AI & Machine Learning
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Artificial intelligence course for college students and professionals: search, logic, planning, Bayesian reasoning, expert systems and multi-agent AI, the university AI canon taught live, in 10-12 months.

In one line: The AI beneath the hype: how machines search, reason, plan and decide, the canon university exams test and modern agent systems quietly reuse.

ABOUT THE PROGRAMME

Before AI meant chatbots, it meant something precise: algorithms that search enormous possibility spaces, represent knowledge in logic, reason under uncertainty and plan actions toward goals. That canon still runs the world, game AI, route planning, scheduling, robotics, diagnosis systems, it is what university AI papers actually examine, and it is exactly what modern LLM agent systems borrow when they need planning and search that works. This live online masterclass teaches it properly: search algorithms from BFS to A* and adversarial game play, propositional and first-order logic with constraint satisfaction, then reasoning under uncertainty, probability, Bayesian networks, Markov models, decision theory and classical planning, then intelligent systems: expert systems, agent architectures, robotics and vision from the AI perspective, and multi-agent systems with game theory. A finale month ships an intelligent-system capstone and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- The real AI canon: search, logic, planning and probabilistic reasoning, taught with code
- University-aligned: this is the syllabus AI papers and competitive exams actually test
- Modern relevance made explicit: where game AI, robotics and LLM agents reuse each idea
- Every algorithm implemented: A* you wrote beats A* you memorized
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Months 1-3): search algorithms, adversarial game play, logic and CSPs
Phase 2: Reasoning (Months 4-6): probability, Bayesian networks, decision theory and planning
Phase 3: Intelligent systems (Months 7-9): expert systems, agents, embodied AI and multi-agent game theory
Phase 4: The finale (Month 10): an intelligent-system capstone and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Everything is taught from the first line
Equipment: Any computer with a stable internet connection; every tool used is free
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

College students: Students whose university AI papers test exactly this canon
Engineers: Developers who want AI understanding deeper than API calls
Ml practitioners: ML users adding the reasoning-and-planning half of AI
Exam aspirants: Aspirants for exams and interviews that probe AI fundamentals
Systems thinkers: Builders drawn to how machines decide, not just how they predict

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: CLASSICAL AI FOUNDATIONS (MONTHS 1-3, WEEKS 1-13)

Build deep understanding of AI foundations, search algorithms, logic, knowledge representation, and problem-solving.

Months 1-2: AI Foundations & Search Algorithms (Weeks 1-8)
Introduction to Artificial Intelligence
Topics:
- What is Artificial Intelligence? Definitions and perspectives
- History of AI: from Dartmouth Conference to present
- AI winters and AI springs
- Turing Test and philosophical foundations
- Strong AI vs Weak AI
- Narrow AI, General AI (AGI), Super AI
- Symbolic AI vs Connectionist AI vs Hybrid AI
- AI paradigms: logic-based, knowledge-based, learning-based
- Rational agents and intelligent behavior
- PEAS framework: Performance, Environment, Actuators, Sensors
- Agent types: simple reflex, model-based, goal-based, utility-based
- Agent environments: fully observable, deterministic, episodic, static
- AI applications across industries
- AI vs ML vs DL: relationships and differences
- State of AI: current capabilities and limitations
- Ethics and societal impact overview
Projects built:
- Simple reflex agent implementation
- Model-based agent for grid world
- Agent environment simulator
- AI timeline and evolution visualization
- Turing Test chatbot (rule-based)
Practice: Study 20 landmark AI papers from history
Search Algorithms - Part 1 (Uninformed Search)
Topics:
- Problem-solving as search
- State space representation
- Problem formulation: initial state, actions, goal test, path cost
- Tree search vs graph search
- Search strategies evaluation: completeness, optimality, time, space
- Breadth-First Search (BFS): algorithm and analysis
- Uniform Cost Search (UCS)
- Depth-First Search (DFS): algorithm and analysis
- Depth-Limited Search (DLS)
- Iterative Deepening DFS (IDDFS)
- Bidirectional search
- Comparing uninformed search strategies
- Search in continuous spaces
- Memory-bounded search
- Real-world applications: route finding, puzzle solving
Projects built:
- 8-puzzle solver with BFS/DFS

- Maze solver with multiple algorithms
- Route finding system (city maps)
- N-Queens problem solver
- Missionaries and Cannibals problem
- Search visualization tool
- Performance comparison framework

Practice: Implement all uninformed search algorithms from scratch

Search Algorithms - Part 2 (Informed/Heuristic Search)

Topics:

- Heuristic functions and admissibility
- Greedy Best-First Search
- A* Search: algorithm, optimality proof
- Heuristic design and dominance
- Memory-bounded heuristic search: IDA*, RBFS, SMA*
- Weighted A* and bounded suboptimal search
- Bidirectional heuristic search
- Pattern databases for heuristics
- Local search algorithms
- Hill-climbing and variants
- Simulated annealing
- Genetic algorithms for search
- Local beam search
- Constraint satisfaction in search
- Search in game playing (preview)

Projects built:

- A* pathfinding implementation
- 15-puzzle solver with pattern databases
- Route optimization with A*
- Simulated annealing for TSP
- Genetic algorithm for optimization
- Hill-climbing variants comparison
- Heuristic function designer tool

Practice: Solve 30 search problems with optimal algorithms

Adversarial Search & Game Playing

Topics:

- Game theory basics for AI
- Game trees and game search
- Minimax algorithm
- Alpha-beta pruning
- Move ordering for better pruning
- Iterative deepening in games
- Evaluation functions for game states
- Quiescence search
- Transposition tables
- Monte Carlo Tree Search (MCTS)
- Upper Confidence Bounds (UCB)
- MCTS in AlphaGo
- Stochastic games and expectiminimax
- Partially observable games
- Multi-player games
- Real-time decision making in games

Projects built:

- Tic-Tac-Toe with minimax
- Chess AI with alpha-beta pruning
- Checkers AI implementation
- Connect-4 game AI
- Go AI with MCTS (simplified)
- Poker bot (partial information)
- Game AI framework with multiple algorithms

Practice: Build 10 different game-playing AI agents

Month 3: Logic & Knowledge Representation (Weeks 9-13)

Propositional Logic

Topics:

- Knowledge-based agents
- Logic as knowledge representation
- Propositional logic: syntax and semantics
- Logical connectives: AND, OR, NOT, IMPLIES, IFF
- Truth tables and logical equivalence
- Inference in propositional logic
- Inference rules: Modus Ponens, And-Elimination
- Resolution and refutation
- Conjunctive Normal Form (CNF)
- Horn clauses and forward/backward chaining
- SAT solvers and applications
- DPLL algorithm
- WalkSAT and local search for SAT
- Applications: circuit design, planning, verification
- Limitations of propositional logic

```
      Projects built:
        - Propositional logic inference engine
        - SAT solver implementation
        - Logic-based puzzle solver (Sudoku)
        - Wumpus World agent with logic
        - Circuit verification system
        - Resolution theorem prover
        - Knowledge base system
      Practice: Solve 40 logical reasoning problems
    First-Order Logic & Inference
      Topics:
        - First-Order Logic (FOL) syntax
        - Predicates, functions, quantifiers
        - Universal and existential quantification
        - Semantics and interpretation
        - Using FOL for knowledge representation
        - Unification algorithm
        - Generalized Modus Ponens
        - Forward chaining in FOL
        - Backward chaining in FOL
        - Resolution in FOL
        - Skolemization and CNF conversion
        - Prolog and logic programming
        - Herbrand universe and interpretations
        - Completeness and decidability
        - Description logics
        - Semantic networks and frames
      Projects built:
        - FOL inference engine
        - Prolog-like system in Python
        - Automated theorem prover
        - Knowledge base with FOL
        - Family tree reasoning system
        - Medical diagnosis system (logic-based)
        - Natural language to FOL translator (simple)
      Practice: Build 10 knowledge-based systems with FOL
    Constraint Satisfaction Problems
      Topics:
        - CSP framework: variables, domains, constraints
        - Examples: map coloring, N-Queens, scheduling
        - Backtracking search for CSP
        - Variable ordering heuristics: MRV, degree heuristic
        - Value ordering: least-constraining-value
        - Inference in CSP: forward checking
        - Arc consistency (AC-3 algorithm)
        - Path consistency and k-consistency
        - Local search for CSP: min-conflicts
        - Constraint propagation
        - Structure in CSP: tree-structured, nearly tree-structured
        - Cutset conditioning
        - Temporal CSPs
        - Soft constraints and optimization
        - Applications: scheduling, resource allocation, configuration
      Projects built:
        - N-Queens CSP solver
        - Sudoku solver with constraint propagation
        - Map coloring problem
        - Course scheduling system
        - Resource allocation optimizer
        - Timetable generator
        - PHASE 1 MINI CAPSTONE: Intelligent Planning System with CSP
      Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

----------------------------------------------------------------
PHASE 2: REASONING, PLANNING & PROBABILISTIC AI (MONTHS 4-6, WEEKS 14-26)
----------------------------------------------------------------
Master probabilistic reasoning, planning algorithms, decision theory, and uncertainty
management in AI.

  Months 4-5: Uncertainty & Probabilistic Reasoning (Weeks 14-20)
    Probability Theory for AI
      Topics:
        - Uncertainty in AI systems
        - Probability basics: events, axioms, rules
        - Conditional probability and Bayes' Rule
        - Bayesian reasoning
        - Random variables: discrete and continuous
        - Probability distributions
```

    - Joint probability distributions
    - Independence and conditional independence
    - Inference using full joint distributions
    - Marginalization and conditioning
    - Naive Bayes classifier
    - Bayesian networks introduction
    - Belief propagation basics
    - Applications: diagnosis, prediction, classification
    - Probabilistic reasoning under uncertainty
  Projects built:
    - Bayesian inference engine
    - Naive Bayes spam filter
    - Medical diagnosis system (probabilistic)
    - Weather prediction system
    - Probabilistic reasoning toolkit
    - Bayesian calculator and visualizer
  Practice: Solve 50 probabilistic reasoning problems
Bayesian Networks
  Topics:
    - Bayesian Network structure and semantics
    - Conditional independence in Bayes nets
    - Constructing Bayesian networks
    - Compact conditional distributions: CPTs
    - Exact inference in Bayesian networks
    - Inference by enumeration
    - Variable elimination algorithm
    - Clustering algorithms for inference
    - Junction tree algorithm
    - Approximate inference: sampling methods
    - Direct sampling and rejection sampling
    - Likelihood weighting
    - Markov Chain Monte Carlo (MCMC)
    - Gibbs sampling
    - Learning Bayesian networks from data
    - Parameter learning: MLE and MAP
    - Structure learning: score-based, constraint-based
  Projects built:
    - Bayesian Network inference engine
    - Variable elimination implementation
    - MCMC sampler for Bayes nets
    - Medical diagnosis Bayes net
    - Risk assessment system
    - Bayes net learning from data
    - Decision support system with Bayes nets
  Practice: Build 15 Bayesian network applications
Markov Models & Temporal Reasoning
  Topics:
    - Time and uncertainty
    - Markov processes and Markov assumption
    - Hidden Markov Models (HMM)
    - HMM representation and semantics
    - Forward algorithm for filtering
    - Viterbi algorithm for most likely sequence
    - Forward-backward algorithm for smoothing
    - Baum-Welch algorithm for learning HMMs
    - Applications: speech recognition, POS tagging
    - Kalman filters for continuous state
    - Extended Kalman filters
    - Particle filters (Sequential Monte Carlo)
    - Dynamic Bayesian Networks (DBN)
    - Temporal reasoning and prediction
    - Applications: tracking, localization, time-series
  Projects built:
    - HMM for speech recognition (simplified)
    - POS tagger with HMM
    - Robot localization with Kalman filter
    - Particle filter for tracking
    - Weather prediction with HMM
    - Stock market HMM (simplified)
    - Dynamic Bayesian network system
  Practice: Implement all temporal reasoning algorithms
Decision Theory & Utility
  Topics:
    - Making decisions under uncertainty
    - Utility theory and preferences
    - Utility functions and axioms
    - Maximum Expected Utility (MEU)
    - Decision networks (influence diagrams)
    - Value of information

```
      - Value of perfect information (VPI)
      - Sequential decision problems
      - Markov Decision Processes (MDPs)
      - MDP formulation: states, actions, transitions, rewards
      - Bellman equations
      - Value iteration algorithm
      - Policy iteration algorithm
      - Partially Observable MDPs (POMDPs)
      - POMDP value iteration
      - Applications: robotics, game playing, resource allocation
    Projects built:
      - Decision network solver
      - MDP solver (value iteration, policy iteration)
      - Grid world MDP
      - Inventory management with MDPs
      - Robot navigation with POMDP
      - Medical treatment decision system
      - Resource allocation optimizer
    Practice: Solve 25 decision-theoretic problems
  Planning Under Uncertainty
    Topics:
      - Planning with probabilistic effects
      - Contingency planning
      - Conditional plans and execution monitoring
      - Replanning strategies
      - Online planning and acting
      - Information gathering actions
      - Exploration vs exploitation
      - Multi-armed bandit problems
      - UCB algorithms for exploration
      - Monte Carlo planning
      - Rollout algorithms
      - Hierarchical planning under uncertainty
      - Applications: robotics, autonomous systems
    Projects built:
      - Contingent planner
      - Multi-armed bandit solver
      - Robot exploration system
      - Adaptive planning agent
      - Online planning framework
      - Monte Carlo planner
    Practice: Build 10 planning under uncertainty systems

Month 6: Classical Planning & Knowledge Integration (Weeks 21-26)
  Classical Planning
    Topics:
      - Planning problem representation
      - STRIPS representation
      - Actions: preconditions and effects
      - State-space search for planning
      - Forward (progression) state-space search
      - Backward (regression) state-space search
      - Heuristics for planning: ignore-preconditions, ignore-delete-lists
      - Plan-space planning (partial-order planning)
      - Graphplan algorithm
      - Planning graphs and graph heuristics
      - SAT-based planning
      - PDDL (Planning Domain Definition Language)
      - Hierarchical Task Network (HTN) planning
      - Temporal planning
      - Planning with resources and time
      - Multi-agent planning basics
    Projects built:
      - STRIPS planner implementation
      - Graphplan algorithm
      - Blocks world planner
      - Logistics planning system
      - Robot task planner
      - SAT-based planner
      - HTN planner (simplified)
    Practice: Solve 20 classical planning problems
  Ontologies & Knowledge Engineering
    Topics:
      - Knowledge engineering process
      - Ontology definition and components
      - Upper ontologies: Cyc, SUMO
      - OWL (Web Ontology Language)
      - RDF and semantic web
      - Knowledge graphs
```

- Ontology design patterns
- Domain modeling
- Taxonomies and hierarchies
- Relationships and properties
- Reasoning with ontologies
- Ontology alignment and merging
- Protégé for ontology development
- SPARQL for querying knowledge
- Applications: healthcare, e-commerce, IoT

Projects built:

- Domain ontology creation (healthcare/e-commerce)
- Knowledge graph builder
- Ontology-based reasoning system
- Semantic search engine
- Question answering with knowledge graphs
- Ontology alignment tool
- SPARQL query system

Practice: Build 5 ontologies for different domains

Learning in AI Systems

Topics:

- Learning agents architecture
- Supervised learning integration in AI
- Decision tree learning: ID3, C4.5
- Information gain and entropy
- Ensemble learning in AI
- Unsupervised learning for AI: clustering
- Reinforcement learning fundamentals (AI perspective)
- Q-learning in AI agents
- Temporal difference learning
- Exploration strategies
- Function approximation in RL
- Deep RL overview for AI
- Transfer learning in AI systems
- Meta-learning and learning to learn
- Hybrid symbolic-neural systems

Projects built:

- Decision tree AI agent
- Reinforcement learning grid world
- Q-learning robot navigator
- Learning agent framework
- Hybrid reasoning system (logic + learning)
- Transfer learning AI agent
- Meta-learning experiment

Practice: Integrate learning into 15 AI systems

Explanation & Interpretability in AI

Topics:

- Explainable AI (XAI) importance
- Explanation in expert systems
- Trace-based explanations
- Rule-based explanation generation
- Argumentation and justification
- Counterfactual explanations
- Contrastive explanations
- Transparency vs performance tradeoffs
- LIME for model explanations
- SHAP values in AI
- Attention mechanisms for interpretability
- Causal reasoning for explanations
- User interfaces for explanations
- Evaluating explanation quality
- Regulatory requirements for explainability

Projects built:

- Explainable expert system
- Explanation generator for decisions
- Argumentation framework
- Counterfactual explanation system
- XAI dashboard
- Causal reasoning engine
- Interactive explanation interface

Practice: Add explanations to all AI systems built

Phase 2 Capstone Project

Topics:

- Integrated AI system design
- Combining multiple AI techniques
- Reasoning under uncertainty
- Planning and decision-making
- Learning and adaptation
- Explanation generation

Projects built:

- PHASE 2 CAPSTONE: Intelligent Decision Support System
- Requirements: Bayesian reasoning, planning, decision theory, learning, explanations
- Option 1: Medical diagnosis and treatment planning system
- Option 2: Autonomous robot mission planner
- Option 3: Smart home energy management system
- Option 4: Financial portfolio advisor with uncertainty

Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 3: INTELLIGENT SYSTEMS & APPLICATIONS (MONTHS 7-9, WEEKS 27-39)

Master expert systems, intelligent agents, robotics, computer vision, NLP, and multi-agent systems.

Months 7-8: Expert Systems & Intelligent Agents (Weeks 27-34)

Expert Systems Architecture

Topics:

- Expert systems overview and history
- MYCIN, DENDRAL, XCON case studies
- Expert system architecture: knowledge base, inference engine, UI
- Rule-based expert systems
- Forward chaining systems
- Backward chaining systems
- Conflict resolution strategies
- Certainty factors and fuzzy logic
- Dempster-Shafer theory
- Knowledge acquisition bottleneck
- Knowledge elicitation techniques
- Knowledge engineering methodology
- Explanation facilities in expert systems
- Expert system shells
- Modern applications of expert systems
- Limitations and decline of expert systems

Projects built:

- Medical diagnosis expert system
- Legal reasoning expert system
- Technical troubleshooting system
- Financial advisory expert system
- Expert system shell development
- Rule-based configurator
- Fuzzy logic controller

Practice: Build 10 domain-specific expert systems

Intelligent Agent Architectures

Topics:

- Agent theory and design
- Deliberative agents (BDI: Belief-Desire-Intention)
- Reactive agents and subsumption architecture
- Hybrid agent architectures
- Layered architectures: InteRRaP, TouringMachines
- Agent communication languages (ACL)
- FIPA specifications
- Agent reasoning cycles
- Agent planning and execution
- Agent learning and adaptation
- Situated agents and embodiment
- Software agents vs physical agents
- Agent-oriented programming
- Agent development frameworks: JADE, Jason
- Applications: virtual assistants, bots, automation

Projects built:

- BDI agent implementation
- Reactive agent for dynamic environment
- Hybrid agent architecture
- Communicating agents system
- Personal assistant agent
- Agent-based automation system
- Situated agent in simulation

Practice: Design and implement 12 intelligent agents

Robotics & Embodied AI

Topics:

- AI in robotics overview
- Robot perception: sensors and processing
- Computer vision for robotics
- Object recognition and tracking
- SLAM (Simultaneous Localization and Mapping)
- Robot motion planning: configuration space
- Path planning algorithms for robots
- Potential field methods

- Probabilistic roadmaps (PRM)
- Rapidly-exploring Random Trees (RRT)
- Robot control architectures
- Behavior-based robotics
- Robot learning: imitation, reinforcement
- Human-robot interaction
- ROS (Robot Operating System) basics
- Applications: autonomous vehicles, drones, industrial robots

Projects built:
- Robot simulator with AI planning
- Path planning visualizer (PRM, RRT)
- SLAM implementation (simplified)
- Behavior-based robot controller
- Robot learning from demonstration
- Autonomous navigation system
- ROS-based AI agent (if feasible)

Practice: Implement 10 robotics AI algorithms

Computer Vision for AI Systems

Topics:
- Vision as AI problem
- Image formation and camera models
- Image processing fundamentals
- Edge detection: Canny, Sobel
- Feature detection: SIFT, SURF, ORB
- Image segmentation techniques
- Object recognition classical methods
- Template matching
- Hough transform for shape detection
- Optical flow and motion analysis
- Stereo vision and depth perception
- 3D reconstruction from images
- Deep learning for vision (integration)
- Scene understanding and reasoning
- Visual reasoning and question answering
- Applications: surveillance, medical imaging, autonomous systems

Projects built:
- Object detector (classical methods)
- Face recognition system
- Motion tracking application
- Image segmentation tool
- 3D reconstruction (simple)
- Visual reasoning system
- Autonomous vehicle perception (simulated)

Practice: Build 12 computer vision AI applications

Natural Language Understanding (AI Perspective)

Topics:
- Language as AI challenge
- Syntax: parsing and grammars
- Context-free grammars
- Parsing algorithms: CKY, Earley
- Semantic analysis and representation
- First-order logic for semantics
- Lambda calculus for meaning
- Discourse and pragmatics
- Coreference resolution
- Information extraction
- Question answering systems
- Dialogue systems and conversational AI
- Natural language generation
- Machine translation (symbolic approaches)
- Integrating NLP with knowledge systems
- Semantic parsing and understanding

Projects built:
- Parser for natural language
- Semantic representation generator
- Question answering system (logic-based)
- Dialogue system with state management
- Information extraction system
- Natural language interface to database
- Story understanding system

Practice: Build 10 NLU systems with AI techniques

Month 9: Multi-Agent Systems & Collective Intelligence (Weeks 35-39)

Multi-Agent Systems Fundamentals

Topics:
- Multi-agent systems (MAS) introduction
- Agent interactions and coordination
- Cooperation, collaboration, competition
- Agent communication protocols

    - Speech acts and performatives
    - Contract Net Protocol
    - Negotiation and bargaining
    - Auction mechanisms
    - Coalition formation
    - Task allocation in MAS
    - Distributed problem solving
    - Distributed constraint satisfaction
    - Multi-agent planning
    - Emergent behavior in MAS
    - Applications: smart grids, traffic systems, e-commerce
  Projects built:
    - Multi-agent coordination system
    - Negotiation protocol implementation
    - Auction system with agents
    - Coalition formation simulator
    - Distributed task allocation
    - Multi-agent marketplace
    - Swarm intelligence application
  Practice: Implement 12 multi-agent scenarios
Game Theory for Multi-Agent AI
  Topics:
    - Game theory foundations
    - Normal form games
    - Nash equilibrium concept
    - Finding Nash equilibria
    - Dominant strategies
    - Mixed strategies
    - Extensive form games
    - Subgame perfect equilibrium
    - Repeated games and folk theorem
    - Evolutionary game theory
    - Mechanism design basics
    - Voting and social choice
    - Cooperative game theory
    - Shapley value
    - Applications in AI: auctions, negotiations, resource allocation
  Projects built:
    - Game theory solver (Nash equilibrium)
    - Repeated game simulator
    - Auction mechanism with game theory
    - Voting system analyzer
    - Cooperative game solver
    - Mechanism design implementation
    - Multi-agent game scenarios
  Practice: Analyze 20 game-theoretic scenarios
Swarm Intelligence & Collective Behavior
  Topics:
    - Swarm intelligence principles
    - Ant Colony Optimization (ACO)
    - Particle Swarm Optimization (PSO)
    - Bee algorithms
    - Flocking and bird behavior (Boids)
    - Stigmergy and indirect communication
    - Self-organization in swarms
    - Emergent intelligence
    - Distributed sensing and actuation
    - Robot swarms
    - Applications: optimization, robotics, routing
    - Cellular automata and emergence
    - Conway's Game of Life
    - Wolfram's cellular automata
    - Artificial life and ALife
  Projects built:
    - Ant Colony Optimization for TSP
    - Particle Swarm Optimizer
    - Flocking simulation (Boids)
    - Robot swarm simulator
    - Stigmergy-based system
    - Cellular automata explorer
    - Artificial life simulation
  Practice: Implement 8 swarm intelligence algorithms
Social AI & Human-AI Interaction
  Topics:
    - Social intelligence in AI
    - Theory of Mind in AI
    - Emotion recognition and affective computing
    - Sentiment analysis for AI
    - Social robots and companions

- Conversational AI and chatbots
- Personality in AI agents
- Trust and transparency in AI
- Anthropomorphism and uncanny valley
- Human-in-the-loop AI systems
- Collaborative AI (AI as teammate)
- Explainability for non-experts
- AI ethics in social contexts
- Cultural considerations in AI
- Applications: healthcare, education, customer service

Projects built:
- Emotion-aware chatbot
- Social robot behavior system
- Theory of Mind agent
- Personality-based AI assistant
- Collaborative AI system
- Human-AI teaming framework
- Culturally-aware AI agent

Practice: Build 10 social AI applications

Phase 3 Capstone Project

Topics:
- Complex multi-agent system design
- Intelligent agent development
- Coordination and cooperation
- Real-world application
- Evaluation and analysis

Projects built:
- PHASE 3 CAPSTONE: Intelligent Multi-Agent System
- Requirements: Multiple agents, coordination, learning, real-world application
- Option 1: Smart city traffic management with agents
- Option 2: Multi-robot warehouse automation
- Option 3: Distributed energy grid management
- Option 4: Multi-agent trading and auction system
- Option 5: Disaster response coordination system

Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---
PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)

The capstone month: spec, build, ship and defend, then the final exam that decides the certificate.

Month 10: Capstone and the Final Exam (Weeks 40-44)

Capstone Spec and Setup

Topics:
- Choosing the capstone: an intelligent system: a game-playing agent, a planner, a constraint solver or a knowledge-based reasoner
- A one-page spec: scope, milestones, stack
- Repository and skeleton ready on day one
- Instructor sign-off before the build begins

Projects built:
- Approved capstone spec plus the running skeleton

Practice: Pitch the spec to the batch and absorb one hard question

Capstone Build

Topics:
- Two focused weeks in vertical slices
- Instructor checkpoints and honest scope cuts
- Using the whole course: search, knowledge representation, probabilistic reasoning and agent design, chosen to fit the build
- Daily commits and an always-working build

Projects built:
- The capstone, feature-complete with version control

Practice: A working build at the end of every session

Ship and Polish

Topics:
- The finishing pass: edge cases, errors, documentation
- Benchmarked honestly: your system measured against baselines, with its limits stated
- A README worth reading and a demo worth watching
- A full-course spiral review before the exam

Projects built:
- The capstone shipped, documented and rehearsed

Practice: Two mock demos, each tighter than the last

Demo Day and the Final Exam

Topics:
- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: AI engineering, robotics and planning systems, or ML depth in our AI/ML course
- Where our other courses continue the journey

Projects built:
  - Demo day presentation delivered to the batch
Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: search solvers, game-playing agents and constraint systems you wrote yourself
- Phase 2: Bayesian reasoners, decision systems and classical planners
- Phase 3: expert systems, agent architectures and multi-agent simulations
- Phase 4: the intelligent-system capstone, benchmarked and defended

Total projects: 15+ implemented AI systems, crowned by a benchmarked capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Search mastery: uninformed, heuristic and adversarial, implemented from scratch
- Knowledge representation: propositional and first-order logic, CSPs, ontologies
- Reasoning under uncertainty: Bayesian networks, Markov models, decision theory
- Planning: classical planning and planning under uncertainty
- Agent design: expert systems, agent architectures and multi-agent game theory

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- AI engineering: agent systems reuse this canon daily
- Our AI/ML course adds the statistical learning half
- Robotics and planning systems: this is their native language
- Our generative AI course connects the canon to LLM-era practice

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Other roles: Rs 15-30 LPA (India), $120k-200k (USA); Rs 25-50 LPA (India), $180k-350k (USA); Rs 40-80 LPA (India), $250k-500k (USA); Rs 60-150 LPA (India), $350k-800k+ (USA); DeepMind, OpenAI pay $300k-$1M+ for top researchers; Rs 10-30 LPA (India), $80k-150k (USA) as Assistant Professor
Consulting: Rs 200-500/hour for expert AI consulting

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this artificial intelligence course cover?
A: The classical AI canon, implemented in code: search algorithms from BFS to A* and adversarial game playing, propositional and first-order logic, constraint satisfaction, probability and Bayesian networks, Markov models, decision theory, classical planning and planning under uncertainty, expert systems, intelligent agent architectures, robotics and vision from the AI perspective, and multi-agent systems with game theory. A finale month ships an intelligent-system capstone and sits the final exam.

Q: How long does it take to learn artificial intelligence properly?
A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two more months in hand for learners who need them. The three teaching phases build the skill and its real-world application; the finale month ships a capstone and sits the final exam.

Q: How am I assessed? Is the certificate just for attending?
A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Is classical AI still worth learning in the LLM era?
A: More than ever, for two honest reasons. First, this canon still runs real systems: game AI, route planning, scheduling, logistics, diagnosis and robotics never stopped using it. Second, the LLM era is converging back onto it: agent frameworks reach for planning, search and knowledge representation the moment a chatbot has to actually do something. Prediction is the new part; deciding what to do with predictions is this course.

Q: How is this different from your AI/ML and machine learning courses?
A: By which half of intelligence they teach. The AI/ML course teaches learning: models trained on data to make predictions. This course teaches reasoning: search, logic, planning and decision-making under uncertainty, the half that universities examine as artificial intelligence and that agent systems are built from. The two halves complete each other, and serious AI work increasingly wants both.

Q: Can I try a class before enrolling?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/artificial-intelligence-complete-masterclass-college
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 26 OF 137: BACKEND DEVELOPMENT FOR TEENS: NODE, APIS AND DATABASES
==========

Course link: https://learn.modernagecoders.com/courses/backend-coding-masterclass-for-teens
Subject area: Web Development
Designed for: Teens
Who can join: Absolute beginner, ages 13 to 18
Course length: 9-11 months (36-44 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Backend course for teens aged 13-18: Node.js, Express, REST APIs, MongoDB and PostgreSQL, auth, security, AI backends and a deployed capstone in 9-11 months.

In one line: The invisible half of every app, made visible: servers, APIs and databases built by a teenager who understands what happens after the button is pressed.

ABOUT THE PROGRAMME

Every tap in every app fires a request at a backend, and almost nobody a teen knows can build one. This live online course fixes that for ages 13 to 18: how the internet really works, JavaScript learned properly, then Node.js and Express, REST APIs designed like a professional, both database families (MongoDB and PostgreSQL), authentication done safely, security against the attacks that actually happen, real-time features, background jobs, testing, and the modern layer, AI backends and agent APIs, built with guardrails. Everything runs for real: APIs are deployed and consumed by classmates, the capstone is a documented, authenticated backend live on a public URL, and the final month adds performance judgment and an honest week of Docker. What the course refuses to fake: Kubernetes, microservices and cloud infrastructure are professional specializations, named truthfully as later roads rather than stuffed in as teen filler. The pace is honest: nine months at two live classes a week plus three to four hours of practice, with two more months in hand. Homework, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest. Real servers, real data, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- The rare skill: most teen courses stop at the frontend; this one builds the engine room
- Both database families: MongoDB and PostgreSQL, with honest judgment about when each wins
- Security taught early: hashing, tokens and the OWASP basics, because backends are where it matters
- AI backends included: model calls, streaming and agent endpoints with guardrails
- APIs consumed by real classmates: documentation quality gets tested socially
- Real assessment: weekly homework, monthly mixed reviews, and a final exam that decides the certificate, with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Months 1-3): how the internet works, JavaScript properly, algorithms, then Node, Express and REST design
Phase 2: Data and product features (Months 4-6): MongoDB and PostgreSQL, authentication, API security, uploads and media
Phase 3: Real systems (Months 7-8): real-time features, background jobs and email, testing, AI backends and agent APIs
Phase 4: Ship (Month 9): performance judgment, an honest Docker week, and the deployed capstone exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. JavaScript is taught from the first variable
Computer skills: Comfortable typing and managing files and folders
Equipment: A computer with a stable internet connection; every tool used is free
Age: 13 to 18. Batches are grouped by age and pace
Time commitment: 2 live classes plus 3-4 hours of practice a week

WHO THIS COURSE IS FOR

How does it work kids: Teens who keep asking what actually happens when you press the button
Frontend graduates: Students from our web courses ready for the other half
Logic lovers: Teens who prefer systems and data to pixels and colors
Future engineers: Students aiming at serious software careers, where backend depth is the differentiator
Complete beginners: No experience needed: the JavaScript foundation is built in

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS (MONTHS 1-3, WEEKS 1-12)

The internet demystified, JavaScript learned properly, problem-solving sharpened, then the first real servers with Node and Express.

Month 1: The Internet and JavaScript (Weeks 1-4)
How The Internet Actually Works
Topics:
- What happens when you open Instagram/TikTok?
- Clients vs Servers: The restaurant analogy
- IP addresses: The internet's phone numbers
- DNS: The internet's phonebook
- HTTP/HTTPS: How computers talk
- APIs: The waiter between frontend and database
- Why backend matters: Facebook's 100,000 servers
- Setting up your coding environment
- Installing VS Code and essential extensions
- Terminal/Command line: Hacker mode activated
- Git & GitHub: Your code's Instagram
- Creating your developer profile

Projects built:
- Draw your favorite app's architecture
- Set up your ultimate coding environment
- Create your GitHub profile README

Practice: Daily: Explore 5 websites with Chrome DevTools Network tab

JavaScript Fundamentals - The Language of Backend
Topics:
- Why JavaScript runs the internet
- Variables: Storing game scores and user data
- Data types: Numbers, strings, booleans explained with memes
- Arrays: Managing Spotify playlists
- Objects: Describing a YouTube video
- Functions: Reusable cheat codes
- If/else: Making decisions like game AI

```
      - Loops: Automation for lazy programmers
      - Template literals: Dynamic message creation
      - Console.log debugging: Your best friend
      - Comments: Leaving notes for future you
      - Code style: Writing clean, readable code
    Projects built:
      - Discord bot message generator
      - Minecraft inventory manager
      - YouTube stats calculator
      - Grade calculator with GPA
    Practice: Code 30 minutes daily on CodeWars/HackerRank
    Assessment: Month 1 check: a JavaScript logic challenge live, plus a take-home mixing
    everything so far

Month 2: Problem Solving and First Servers (Weeks 5-8)
  Algorithms and Problem Solving
    Topics:
      - What are algorithms? TikTok's 'For You' page explained
      - Big O notation: Why Instagram loads fast
      - Searching algorithms: Finding a friend in contacts
      - Sorting algorithms: Organizing Spotify playlists
      - Recursion: Inception for programmers
      - Hash tables: How Discord finds users instantly
      - Stacks & Queues: YouTube video buffer explained
      - Trees: Folder structures and game decision trees
      - Problem-solving strategies
      - Debugging like a detective
      - Code optimization basics
      - Common coding patterns
    Projects built:
      - Build TikTok's recommendation algorithm (simplified)
      - Create a matchmaking system for games
      - Implement Instagram's hashtag search
      - Design a tournament bracket system
    Practice: Solve 50 algorithm problems on LeetCode Easy
  Node.js - JavaScript on Servers
    Topics:
      - What is Node.js? V8 engine explained
      - Why Discord and Netflix use Node.js
      - Installing Node.js and npm
      - Your first server: Hello World
      - Understanding the event loop with game examples
      - Modules: Organizing code like Minecraft mods
      - NPM: The app store for code
      - Package.json: Your project's ID card
      - Core modules: fs, path, http, crypto
      - Reading/writing files: Save game systems
      - Asynchronous programming: Not freezing like games
      - Callbacks, Promises, Async/Await explained with memes
    Projects built:
      - File-based todo list for homework
      - Discord webhook notifier
      - Local multiplayer game server
      - Meme generator backend
    Practice: Build 10 small Node.js utilities
    Assessment: Month 2 check: a tiny Node server built live, plus mixed review

Month 3: Express and REST Design (Weeks 9-12)
  Express.js - Professional Backend Framework
    Topics:
      - What is Express? The backend framework everyone uses
      - Setting up your first Express server
      - Routes: Different URLs for different features
      - GET, POST, PUT, DELETE: The four horsemen of APIs
      - Request and Response objects
      - Middleware: Instagram filters for your server
      - Handling JSON data: The language of APIs
      - Query parameters: YouTube search explained
      - Route parameters: Twitter.com/:username
      - Error handling: When things go wrong
      - Static files: Serving images and videos
      - Environment variables: Keeping secrets safe
    Projects built:
      - School assignment submission API
      - Gaming stats tracker API
      - Social media post API
      - Music playlist backend
    Practice: Create 5 different Express APIs
  RESTful API Design
```

Topics:
- REST principles: The rules everyone follows
- Designing URLs like a pro
- HTTP status codes: 200 OK, 404 Not Found, and friends
- API versioning: Why apps need updates
- Request/Response best practices
- Pagination: Loading Instagram posts efficiently
- Filtering and sorting data
- API documentation with Swagger
- Testing APIs with Postman/Insomnia
- Rate limiting: Preventing spam
- CORS: Letting websites talk to your API
- API security basics

Projects built:
- Complete RESTful blog API
- E-commerce product API
- Video game inventory API
- School management system API

Practice: Document and test all your APIs professionally

Assessment: Phase 1 exam: design and build a small API live, plus a written mixed paper

---

PHASE 2: DATA AND PRODUCT FEATURES (MONTHS 4-6, WEEKS 13-24)

The features every real product needs: both database families, safe authentication, hardened APIs, and file uploads done properly.

Month 4: Databases, Both Families (Weeks 13-16)

MongoDB - NoSQL for Modern Apps

Topics:
- MongoDB basics: Documents instead of tables
- Why Snapchat and Fortnite use MongoDB
- Installing MongoDB and Compass GUI
- Collections and documents
- CRUD operations: Create, Read, Update, Delete
- Querying data: Finding specific posts
- Advanced queries: $gt, $lt, $in, $regex
- Aggregation pipeline: Data transformation
- Indexes for performance
- Mongoose ODM: MongoDB made easy
- Schema design in MongoDB
- Embedding vs Referencing data

Projects built:
- Social media posts database
- Game inventory system
- Chat application database
- Movie recommendation database

Practice: Build 5 MongoDB databases for different apps

PostgreSQL - Powerful SQL Database

Topics:
- PostgreSQL: The world's most advanced open source database
- SQL basics: SELECT, INSERT, UPDATE, DELETE
- Creating tables with constraints
- Joins: Connecting data from multiple tables
- GROUP BY: Analytics and statistics
- Aggregate functions: COUNT, SUM, AVG
- Subqueries: Queries within queries
- Views: Saved queries for complex data
- Transactions: All or nothing operations
- Database migrations: Updating schemas safely
- Backup and restore strategies
- Performance optimization with EXPLAIN
- Prisma in one honest lesson: the typed modern workflow

Projects built:
- E-commerce database with orders
- School management system
- Banking transaction system
- Analytics dashboard database

Practice: Write 100 SQL queries of increasing complexity

Assessment: Month 4 check: model one domain in both databases and defend the choice, plus mixed review

Month 5: Auth and Security (Weeks 17-20)

User Authentication Systems

Topics:
- Authentication vs Authorization: Who you are vs What you can do
- How Instagram knows it's really you
- Password hashing: Never store passwords as text
- Bcrypt: Making passwords uncrackable

```
        - Salt: Extra security seasoning
        - Sessions: Remembering logged-in users
        - Cookies: Not the edible kind
        - JWT tokens: Passport for APIs
        - Access tokens vs Refresh tokens
        - OAuth 2.0: Login with Google/Discord
        - Two-factor authentication (2FA)
        - Password reset flows
      Projects built:
        - Complete authentication system
        - Discord OAuth integration
        - JWT-based API auth
        - 2FA implementation
      Practice: Add authentication to 5 different projects
    API Security & Best Practices
      Topics:
        - Common security vulnerabilities
        - SQL injection: The classic hack
        - XSS attacks: Injecting evil JavaScript
        - CSRF: Tricking users into actions
        - Rate limiting: Preventing spam and DDoS
        - Input validation: Never trust user input
        - Sanitization: Cleaning dangerous data
        - HTTPS and SSL certificates
        - API keys and secrets management
        - Environment variables for sensitive data
        - Security headers with Helmet.js
        - CORS configuration and security
      Projects built:
        - Secure API with all protections
        - Vulnerability scanner tool
        - Rate limiting service
        - Security audit checklist app
      Practice: Security audit all your previous projects
      Assessment: Month 5 check: harden an API live against a checklist, plus mixed review

  Month 6: Uploads, Media and the Phase Capstone (Weeks 21-24)
    File Uploads & Media Handling
      Topics:
        - Handling file uploads like Instagram
        - Multer for file processing
        - Image optimization and resizing
        - Video processing basics
        - Cloud storage: AWS S3, Cloudinary
        - CDN for fast media delivery
        - File type validation
        - Upload progress tracking
        - Chunked uploads for large files
        - Generating thumbnails
        - PDF generation and manipulation
        - QR code generation
      Projects built:
        - Instagram-like photo upload system
        - Video streaming service
        - File sharing platform
        - Avatar upload system
      Practice: Add file upload to previous projects
    Phase 2 Capstone: A Product Backend
      Topics:
        - Scoping a backend product: resources, auth, uploads
        - Building in vertical slices with the whole toolkit
        - Documentation a classmate can build against
        - Deploying with environment variables done right
        - A cumulative review of phases 1-2
      Projects built:
        - PHASE CAPSTONE: an authenticated, documented API with file uploads, deployed
      Practice: Have a classmate integrate against your API using only your docs
      Assessment: Phase 2 exam: capstone walkthrough plus a cumulative written paper

------------------------------------------------------------
PHASE 3: REAL SYSTEMS (MONTHS 7-8, WEEKS 25-32)
------------------------------------------------------------
What separates demos from systems: live features, background work, tests, and the modern
AI-backend layer built with guardrails.

  Month 7: Live Features and Background Work (Weeks 25-28)
    Real-time Features & WebSockets
      Topics:
        - Real-time communication explained
```

- How Discord chat works instantly
- WebSockets vs HTTP: Persistent connections
- Socket.io library mastery
- Events: emit, on, broadcast
- Rooms: Private group chats
- Namespaces: Organizing connections
- Real-time notifications
- Live collaboration features
- Online presence indicators
- Typing indicators
- Real-time gaming backends

Projects built:
- Real-time chat application
- Multiplayer game server
- Live collaboration tool
- Real-time dashboard

Practice: Add real-time features to 3 projects

Background Jobs, Scheduling and Email

Topics:
- Why background jobs matter
- Queue systems with Bull/BullMQ
- Job scheduling with node-cron
- Email queues for bulk sending
- Image processing queues
- Scheduled reports generation
- Cleanup jobs for old data
- Retry strategies for failed jobs
- Job monitoring dashboards
- Distributed job processing
- Rate-limited API calls
- Batch processing patterns
- Transactional email that actually arrives

Projects built:
- Email campaign scheduler
- Automated backup system
- Report generation service
- Social media post scheduler

Practice: Add background jobs to existing projects

Assessment: Month 7 check: a scheduled job plus notification flow built live, plus mixed review

Month 8: Testing and AI Backends (Weeks 29-32)

Testing Fundamentals

Topics:
- Why testing prevents embarrassing bugs
- Types of testing: Unit, Integration, E2E
- Test-Driven Development (TDD)
- Jest testing framework
- Writing your first tests
- Testing async code
- Mocking external services
- Test coverage metrics
- Continuous testing practices
- Debugging failed tests
- Test organization and naming
- Testing best practices

Projects built:
- Add unit tests to all functions
- Integration tests for APIs
- TDD for new feature
- 100% coverage challenge

Practice: Write 200+ tests for your projects

AI Backends and Agent APIs

Topics:
- Putting a model behind your own API
- Secrets management: keys on the server, never the client
- Streaming responses (SSE) and why clients need them
- Queues and background jobs for slow AI calls
- Function calling and tool use from a backend
- Retrieval: giving the model your data without retraining
- Vector search basics, and when plain SQL is better
- Caching AI responses, and when caching is wrong
- Rate limiting, quotas and per-user cost caps
- Guardrails: validating what the model gives back
- Logging and auditing AI decisions
- Failure design: what happens when the provider is down
- Accounts for AI services set up with a parent where required

Projects built:
- AI chat API

```
        - Retrieval-backed Q&A service
        - Agent with tool access
        - Cost + rate-limit middleware
      Practice: Add an AI backend to a project
      Assessment: Phase 3 exam: an AI-powered endpoint with guardrails defended, plus a
      cumulative paper

---------------------------------------------------------------
PHASE 4: SHIP (MONTH 9, WEEKS 33-36)
---------------------------------------------------------------
Performance judgment, one honest week of Docker, and the capstone deployed, documented and
defended.

  Month 9: Performance, Docker, Capstone (Weeks 33-36)
    Performance and Monitoring Basics
      Topics:
        - Performance profiling tools
        - Identifying bottlenecks
        - Memory leak detection
        - CPU profiling and optimization
        - Database query optimization
        - N+1 query problem solutions
        - Caching at multiple layers
        - CDN optimization strategies
        - Code optimization techniques
        - Async processing patterns
        - Load testing with Artillery/k6
        - Performance monitoring setup
      Projects built:
        - Optimize slow API to handle 10x load
        - Build performance monitoring dashboard
        - Create load testing suite
        - Implement multi-layer caching
      Practice: Performance test and optimize all APIs
    Docker in One Honest Week
      Topics:
        - What is Docker? Shipping containers for code
        - Containers vs Virtual Machines
        - Docker installation and setup
        - Docker images and containers explained
        - Writing your first Dockerfile
        - Docker commands mastery
        - Multi-stage builds for optimization
        - Docker volumes for data persistence
        - Docker networks for container communication
        - Docker Compose for multi-container apps
        - Docker Hub and registries
        - Container security best practices
      Projects built:
        - Dockerize Node.js application
        - Multi-container app with database
        - Docker development environment
        - Production Docker setup
      Practice: Dockerize all your major projects
    Capstone Build
      Topics:
        - One focused week: a complete backend product
        - Auth, data, uploads, one live feature, maybe an AI endpoint
        - Tests on the logic that must not break
        - Instructor checkpoint and honest scope cuts
        - Deployed with documentation worth reading
      Projects built:
        - THE CAPSTONE: a deployed, authenticated, documented backend product
      Practice: A working deployed build at the end of every session
    Demo Day and the Final Exam
      Topics:
        - Presenting the capstone: request to response, live
        - A full-course spiral review before the exam
        - The honest map: Kubernetes, microservices and cloud as later professional roads
        - Where each path leads: full-stack, DSA, or GenAI engineering later
      Projects built:
        - Demo day presentation delivered to the batch and parents
      Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed
      from every phase. Passing earns the certificate; a student who falls short gets a
      focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Phase 1: JavaScript builds, first Node servers and a designed REST API
```

- Phase 2: dual-database models, safe auth, hardened endpoints and an uploads-capable capstone
- Phase 3: live features, scheduled jobs with email, test suites and an AI-powered endpoint
- Phase 4: performance passes, a Docker week and the deployed capstone at demo day

Total projects: 20+ real backend builds, crowned by a deployed, documented capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- How the internet actually works, from socket to response
- Node.js and Express with professional REST design
- Both database families: MongoDB and PostgreSQL with honest judgment
- Safety: password hashing, tokens, OWASP basics and secure configuration
- Systems: real-time features, background jobs, email and testing
- The modern layer: AI backends and agent endpoints with guardrails

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building along with the instructor
Homework: Homework after every class, and a mixed-review assignment at the end of every month
Practice: 3-4 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, for the 500 error with no message
Progress updates: Regular progress notes to parents: what was built, what comes next, and where encouragement helps

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Full-stack web development: add the frontend half and own the whole product
- Data Structures and Algorithms: the interview muscle on top of real building
- GenAI engineering later: AI backends here are the on-ramp
- The professional roads, named honestly: microservices, Kubernetes and cloud at career time
- Real services now: APIs for school clubs, family projects and friends' frontends

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Backend or frontend first for a teenager?
A: Either genuinely works, and they meet in our full-stack road. Backend first suits teens who love logic, data and how-does-it-work questions; frontend suits visual builders. Backend skill is rarer at this age, which makes it a quiet differentiator in college applications and team projects, everyone needs the person who can build the API.

Q: How long does it take, and how is my teen assessed?
A: Nine months at 2 live classes a week plus 3 to 4 hours of practice, with two more months in hand. Homework follows every class, monthly assignments mix new questions with earlier topics, each phase ends with an exam, and a final exam, practical plus written, decides the certificate with a free retest after revision.

Q: Does this course cover Kubernetes, microservices or AWS?
A: Honestly, no, and the course says why: those are professional specializations layered on years of fundamentals, and stuffing them into a teen syllabus produces vocabulary without capability. This course builds the fundamentals deeply, includes an honest Docker week, and names the infrastructure road truthfully as a later step.

Q: How is AI part of a backend course?
A: As the modern backend layer: month 8 builds AI-powered endpoints properly, model calls made server-side, responses streamed, guardrails against misuse, costs controlled, and a look at agent APIs. Accounts for AI services are set up with a parent where required. Backends are exactly where AI features live in real products.

Q: Is there a free trial class?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor reading real code and real logs, never pre-recorded videos. Backend errors are invisible by nature; live debugging together is how

the skill actually forms.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/backend-coding-masterclass-for-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 27 OF 137: BUILD AI AGENTS WITH GOOGLE GEMINI ENTERPRISE

==========

Course link: https://learn.modernagecoders.com/courses/build-ai-agents-with-google-gemini-enterprise-course
Subject area: AI Agents & Automation
Designed for: College students
Who can join: Intermediate to advanced · working professionals, founders, team leads and developers
Course length: 16 classes (2 months · 2 classes/week · 1 hour each)
Time needed: 2 live hours a week plus 3-4 hours of build time
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: one to one Rs 14,999
Outside India, per month: one to one $250
Classes: one to one in India two live classes a week, eight a month; one to one outside India two live classes a week, eight a month. Each class is one hour.
Note: this course is taught one to one only. There is no group or mini batch option.

WHAT THIS COURSE IS

Build AI agents with Google Gemini Enterprise in 16 live 1-on-1 classes: Workflow Builder, connectors, MCP, A2A, ADK and governance on Google Cloud. 2 months.

In one line: 16 live 1-on-1 classes over 2 months. Build, connect, govern and ship enterprise AI agents with Workflow Builder, connectors, MCP, A2A and the Agent Development Kit, with a dedicated mentor and your own organisation's work as the material.

ABOUT THE PROGRAMME

Google rebuilt its enterprise AI story around agents. At Google Cloud Next in April 2026 it introduced the Gemini Enterprise Agent Platform, the evolution of Vertex AI, and positioned the Gemini Enterprise app as the single place where teams discover, create, share and run agents. Since then the platform has shipped at a pace that makes recorded courses obsolete within weeks: Workflow Builder (formerly Agent Designer) reached general availability on 2 September 2026 with multi-step automation, chat agents and enterprise connectors; Skills, Projects, the Inbox, observability dashboards, Agent Registry governance, custom MCP server data stores and A2A agent registration all went GA between June and September; and the model lineup moved through Gemini 3.1 Pro to Gemini 3.5, 3.6, 3.7 and 3.8 Flash. This programme teaches that platform as it stands today, in sixteen private one-hour classes over two months. You start on the no-code side: chat agents and workflows in Workflow Builder, grounded in Google Workspace and third-party data stores, calling real actions in Slack, ServiceNow, Jira and GitHub, gated by human approvals and run on schedules and triggers. You then cross to the pro-code side that the same platform exposes: the Agent Development Kit (ADK) in Python, the Agent Runtime with Sessions, Memory Bank and Code Execution, Agent Identity, Model Armor and Agent Simulation, and the open protocols, Agent2Agent (A2A) and the Model Context Protocol (MCP), that let agents from different builders work together and be registered and governed inside Gemini Enterprise. The final phase is the part most agent courses skip and most organisations pay for: observability, cost modelling across seat editions and pay-as-you-go, data residency (the India region reached general availability in June 2026), organisation policy constraints, content policies for sensitive data, and the exam map for Google Cloud's Professional Agentic Architect certification. You finish with a capstone agent estate for your own team, defended live with the governance and cost pack to match. Because every class is 1-on-1, the depth follows your role, and the syllabus is a Living Syllabus, re-verified against Google Cloud's release notes as the platform ships.

WHAT MAKES THIS COURSE DIFFERENT

- 1-on-1 only, by design: enterprise agent work touches your data, your connectors, your admin posture and your budget, none of which belongs in a group class. A dedicated mentor builds every class around your organisation and goes exactly as deep as your role needs.
- Both halves of Google's platform in one programme: no-code Workflow Builder, Skills and Projects in the Gemini Enterprise app, and pro-code ADK, Agent Runtime, Agent Identity and Model Armor in the Gemini Enterprise Agent Platform, plus the honest rules for when each is the right tool.
- Real connectors, real actions: agents that read and write Slack, ServiceNow, Jira, Confluence, GitHub, Google Workspace and Microsoft Teams through the data stores Google ships, not demo data, with organisation policy constraints and content policies applied the way an administrator would.
- Open protocols taught as a skill you keep: the Model Context Protocol for tools and the Agent2Agent protocol for agent collaboration, both supported natively in Gemini Enterprise and now backed by more than 150 organisations through the Linux Foundation.

- Cost and governance fluency most builders lack: seat editions, the pay-as-you-go edition, overage controls, Agent Runtime metering, observability with p50 and p95 latency, data residency and the certification ladder, so your capstone ships with a defensible bill and audit trail.
- Living Syllabus: Gemini Enterprise ships changes every week (Workflow Builder went GA on 2 September 2026 and Gemini 3.8 Flash the same day), and this course is re-verified against Google Cloud's release notes as features land, which is why it is taught live, never recorded.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop (Windows, Mac or Linux) with Chrome and a reliable internet connection; the Gemini Enterprise app and Workflow Builder run fully in the browser, and the ADK classes need Python 3.10 or newer installed
Coding experience: None required for the no-code half. Workflow Builder, Skills, Projects and connectors are taught without code, and the pro-code classes on the Agent Development Kit are paced to you: developers go deeper into ADK and Agent Runtime, non-developers learn to read and direct that work
Accounts: A Google Cloud account with Gemini Enterprise access (Standard, Plus or the pay-as-you-go edition unlocks Workflow Builder). Ideally you use your employer's environment with your administrator's awareness; where that is not possible, we help you set up a pay-as-you-go project or a practice environment in the first class, at your own cloud cost
Mindset: The willingness to treat agents as production software: tested before trusted, costed before scaled, and governed before shipped. No prior agent-building experience is assumed; comfort with everyday business software is. The course rewards professionals who ask what does this cost and who approved it as instinctively as does it work

WHO THIS COURSE IS FOR

Operations and business leads: Process owners, analysts and operations leads who know exactly which workflow is broken and want to ship the agent that fixes it, with the evidence to defend it
It and workspace admins: IT professionals and Google Workspace or Google Cloud administrators who will be asked to govern agents either way, and would rather understand Agent Registry, policy constraints and residency before the first incident
Developers and architects: Developers and solution architects adding the Google agent stack (ADK, Agent Runtime, A2A, MCP) to skills employers already list, and preparing for the Professional Agentic Architect certification
Founders and team leads: Founders, product managers and team leads who must decide what gets automated, what it costs per seat or per run, and which agent projects deserve to ship
Consultants and partners: Consultants and Google Cloud partners who want to sell agent builds that pass a client's security review, with governance and cost packs as deliverables, not afterthoughts

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE GEMINI ENTERPRISE PLATFORM

Map Google's two-sided agent stack, get the right edition and access, learn the anatomy of a Workflow Builder agent, build instruction and testing discipline, and make the model and Skills decisions that shape cost and capability.

Class 1: Google's Agent Stack in September 2026 and Where You Build
Topics:
- The two halves by their exact names: the Gemini Enterprise app (where teams discover, create, share and run agents) and the Gemini Enterprise Agent Platform (the evolution of Vertex AI, announced at Google Cloud Next on 23 April 2026, where developers build, scale, govern and optimise agents)
- What lives on each side: Workflow Builder, Skills, Projects, Inbox, Canvas and the Agent Gallery in the app; the Agent Development Kit, Agent Runtime, Agent Identity, Agent Gateway, Model Armor, Agent Simulation and Agent Observability on the platform
- Editions and what they unlock: Business, Standard and Plus seats, the Frontline edition, and the pay-as-you-go edition that reached general availability on 1 August 2026; Workflow Builder is documented for Standard, Plus, pay-as-you-go and Frontline, which decides where your organisation must be
- Access and roles honestly stated: the Gemini Enterprise Admin and Gemini Enterprise User IAM roles (available since 15 April 2026), licence distribution in the Google Cloud console, and what a maker can and cannot do without an administrator
- The prebuilt agents you inherit: the Core Assistant root agent, Deep Research and Co-Scientist, and why the Idea Generation agent was retired in July 2026 (its work moved to the assistant and Deep Research)
- Reading the release notes like an architect: GA versus Public Preview versus allowlist, and the rule this course lives by, production commitments only on GA features
- Choosing your capstone: one real process from your own team, nominated today and refined for sixteen classes
Projects built:

    - Write your organisation's one-page agent opportunity memo: three candidate processes, the data and systems each needs, the risks each carries, the edition it requires, and the one that becomes your capstone
  Practice: Audit two agents you already meet at work or online and reverse-engineer what they are grounded in, what they can act on, and where a human is in the loop.
  Assessment: Defend your capstone choice in five minutes: process, users, data, edition, risk, and what measurably better will mean.

Class 2: Agent Anatomy in Workflow Builder
  Topics:
    - Workflow Builder, formerly Agent Designer, generally available since 2 September 2026: the two build artifacts, chat agents (conversational, task-specific, largely non-deterministic) and workflows (sequential multi-step processes that combine AI steps with human intervention)
    - The two creation modes and when to use each: describing the agent in natural language, and building on the visual canvas with steps, triggers and actions
    - Instructions as the real product: identity, scope, tone, format rules, refusals and escalation, written in that order and versioned outside the tool from day one
    - The permission model that governs everything: actions and tools available to any agent or workflow are limited to the data connectors, tools and permissions the Gemini Enterprise administrator has provisioned
    - The default model decision made for you: Workflow Builder agents default to Gemini 3.1 Pro in the US and Global regions (since 17 May 2026), and what that means for reasoning quality and cost
    - Publishing, turning on and sharing: agents shared with specific users or Google Groups (end-user sharing to Groups has been GA since June 2026), and why sharing scope is a governance decision
    - Instruction anti-patterns from real deployments: contradictions, vague verbs, policy text pasted verbatim, and instructions that fight the knowledge sources
  Projects built:
    - Build Policy Desk v1: an internal chat agent for one real policy area of your organisation, with a complete instruction stack, five tested refusals and a designed hand-off to a human
  Practice: Run the same ten workplace questions against two instruction variants and score every difference in a comparison table.
  Assessment: Present your instruction stack line by line and state, for each line, what production incident removing it would eventually cause.

Class 3: Testing Like It Is Already in Production
  Topics:
    - The fixed test set written before tuning: happy path, edge case and adversarial questions for Policy Desk, with expected behaviour recorded first
    - The integrated testing environment in Workflow Builder: previewing and simulating chat agents and workflows before publishing, and what a simulation can and cannot prove
    - Adversarial testing for business agents: prompt injection attempts, policy-boundary probes, the disgruntled-user tone test, and the transparent thinking view (GA July 2026) as a debugging aid
    - Hallucination triage: distinguishing missing knowledge, bad grounding and model overreach, because each has a different fix
    - Regression discipline: rerunning the full set after every instruction, knowledge or tool change, and logging verdicts in a form a reviewer can read
    - Escalation as a tested feature: the designed path to a human is an outcome you verify, not a failure you apologise for
    - Preview of Phase 4: Agent Simulation on the Agent Platform stress-tests agents against real-world scenarios before deployment; today you build the manual version of that discipline
  Projects built:
    - Build the permanent 25-question test set for Policy Desk, including five adversarial probes, and run it end to end in the testing environment with verdicts logged
  Practice: Attempt three prompt-injection attacks on your own agent, document the outcomes, and patch the weakest point.
  Assessment: Given a transcript with three subtle failures, identify each one and classify it: instructions, knowledge or escalation gap.

Class 4: Models, Skills and Projects: The Decisions Underneath
  Topics:
    - The Gemini lineup inside Gemini Enterprise as of September 2026: Gemini 3.1 Pro as the Pro flagship and Workflow Builder default; Gemini 3.5 Flash (GA 19 May 2026), 3.6 Flash, 3.7 Flash (GA 13 August) and 3.8 Flash (GA 2 September), each with its own regional rollout
    - Reading model status correctly: Limited Availability, allowlist, GA per region, and deprecation notices (the 3.5 Flash toggle was retired in June; its Global-region removal was postponed in August), so production agents pin to what is GA where your data lives
    - Data residency as a model decision: DRZ (data residency zone) and MLP commitments per region, and the India and Singapore regions reaching GA with DRZ in June 2026
    - Skills, generally available since June 2026: reusable, shareable capabilities that end users can create, upload and share once an administrator turns the toggle on, and the governance conversation that toggle starts
    - Projects, GA since 4 September 2026: private knowledge bases with file upload and web search, shared by teams and agents as a persistent workspace
    - Choosing a model with evidence: the same test set run against two models, scored on accuracy, tone and latency, and written up as a one-paragraph decision
    - Phase 1 gate: the platform map, edition, access and model decisions for your capstone, in

writing
Projects built:
- Run your Policy Desk test set against two Gemini models, score the differences, and write the model decision with evidence attached; then package the agent's reusable behaviour as a Skill and put its source documents in a Project
Practice: Draft the request you would send to your Gemini Enterprise administrator to enable Skills sharing and confirm the region your agents run in.
Assessment: Phase 1 gate: rebuild a small business agent from a written spec in 30 minutes, with instruction stack, refusals, a 10-question test and a defended model choice.

PHASE 2: KNOWLEDGE, CONNECTORS AND ACTIONS

Ground agents in Google Workspace and third-party data at scale, understand federated versus indexed sources, give agents actions that write to real systems, add custom MCP servers under Agent Registry governance, and prove accuracy the way an auditor would.

Class 5: Grounding Strategy: The Data Store Estate
Topics:
- Data stores as the knowledge layer: Google Workspace sources (Drive, Gmail, Calendar, Chat, Sites), and the third-party estate Google has been adding at the rate of roughly ten a fortnight through 2026, from Jira, Confluence, Slack, ServiceNow, Box, Dropbox, GitLab and GitHub to Smartsheet, Asana, Airtable, Dynamics 365 and Microsoft Teams
- Federated versus indexed sources: what a federated data store searches live in the source system, what an indexed store copies and ranks, and the latency, freshness and permission consequences of each
- Status discipline again: which connectors are GA (Slack, ServiceNow, Jira and Confluence Data Center, HubSpot, Box, Microsoft Teams, GitHub, Google Sites) and which are still Public Preview, and why a capstone grounds only on GA sources
- Filtering as a design tool: data store filters (SharePoint and Google Sites filters reached GA in mid-2026) that scope an agent to the folders, spaces and sites it should actually see
- Permissions reality: agents surface what their data stores can reach for the signed-in user, so source selection is a security decision, not just a quality one
- Citations as a compliance feature: grounded, cited answers as the difference between an answer and a liability
- Designing the capstone's source map: authoritative, current, permissioned and small enough to audit
Projects built:
- Ground Policy Desk in a real Google Drive or SharePoint source (or a mirrored practice copy) and prove ten answers cite the correct document and passage
Practice: Ask your grounded agent five questions its sources cannot answer and document the exact edge-of-knowledge behaviour.
Assessment: Present your capstone source map: every source, its owner, its type (federated or indexed), its update cadence, and why it earns its place.

Class 6: Documents, Workspace and Third-Party Knowledge Done Right
Topics:
- Writing documents for retrieval: headings that answer questions, one fact per paragraph, consistent terminology, and the anti-pattern of the 40-page PDF
- Drive and SharePoint hygiene that grounding exposes: stale copies, duplicate policies, draft folders, and the librarian pass that fixes them before the agent embarrasses you
- Structured versus prose knowledge: when a Jira project, a ServiceNow table or a Smartsheet grid should answer instead of a document, and how instructions route question types to source types
- Source arbitration: what happens when Confluence, a Google Doc and Slack disagree, and the scoping instructions that decide which wins
- Freshness engineering: which source updates win, what happens when an indexed document changes, and update cadences you can promise a stakeholder
- Data minimisation at the source: what never belongs in agent knowledge (personal data, credentials, unreleased financials), previewing the content policies of Class 15
- The curation multiplier: five well-structured pages outperform fifty messy ones, demonstrated live on your own content
Projects built:
- Rebuild one messy real document into a retrieval-optimised version, ground both versions side by side, and publish the before-and-after accuracy comparison; then add a second, structured source and document the arbitration behaviour with five conflict questions
Practice: Run a librarian audit on one Drive or Confluence area you actually use: what is in, what is stale, what would embarrass the agent that cites it.
Assessment: Given two document structures, predict which grounds better and prove it with a five-question retrieval test.

Class 7: Actions: Agents That Write to Real Systems
Topics:
- From answering to acting: connector actions in Gemini Enterprise, and the GA action sets that ship with the platform (sending and scheduling Slack messages since May 2026, creating and updating ServiceNow incidents since June, Microsoft Teams channel and chat actions since July)
- Action filtering as governance: the filters that restrict which actions an agent may take on

SharePoint, OneDrive and Jira Data Center, and why an administrator, not a maker, decides them
- The consent question: connection credentials, who authorises what, and why connecting an account is a governance event you document
- Designing a write-safe agent: confirm before act, idempotence (running twice must not double the damage), and the safe default when inputs surprise you
- Custom actions and custom fields (Private Preview as of 8 September 2026): where Google is taking action configuration, flagged honestly as preview, not promised
- Error paths as product: what the user sees when the action fails, times out or is refused, designed before the happy path is polished
- The action inventory for your capstone: every write the agent will ever make, with an owner and a rollback

Projects built:
- Build Support Triage v1: a chat agent that collects an issue, searches the right knowledge, and creates the ServiceNow incident or Jira issue (or posts to the right Slack channel) with a confirmation step and tested failure messages

Practice: Break your action deliberately (missing field, revoked permission, timeout) and verify the user sees a designed message for each failure.
Assessment: Sketch one action end to end (utterance, agent, action selection, auth, call, result, reply) and mark every point where governance can intervene.

Class 8: Custom MCP Servers, Agent Registry and the Accuracy Audit
Topics:
- The Model Context Protocol in one sentence: the open standard for plugging tool servers into agents, supported in Gemini Enterprise through custom MCP server data stores (GA since 7 August 2026) and through Workflow Builder's MCP server connections
- The governance facts that make MCP enterprise-safe here: custom MCP server data stores are disabled by default and need an organisation policy constraint override, tools are auto-discovered through the server's tools/list endpoint, and unauthenticated connections are a separate Public Preview you keep out of production
- Agent Registry, GA since June 2026: registering, importing and governing MCP servers and agents from one place, so a security lead can see every tool surface an agent has
- Evaluating a third-party MCP server before connecting it: capabilities, data flows, failure modes and the least-power principle, written as a one-page security note
- The Gemini Enterprise Agent Platform's own remote MCP server: letting coding agents and IDEs operate the platform through MCP, and where that helps a builder
- The accuracy audit protocol: scoring 25 grounded answers against the actual source text with a rubric a non-builder could apply, plus uncertainty language as policy
- Phase 2 gate: the grounding and tooling checklist you will apply to every agent for the rest of your career

Projects built:
- Connect one custom MCP server to a practice agent with only the tools you need enabled and a one-page security note, then run the full 25-answer accuracy audit on Policy Desk and publish the result with methodology and a remediation list

Practice: Draft the organisation policy request for your MCP integration as you would submit it to your Google Cloud administrator, naming the data boundaries.
Assessment: Phase 2 exam: design the complete knowledge, action and tool plan for a new use case in writing (sources, filters, actions, MCP, refusals, audit protocol) in 30 minutes.

PHASE 3: WORKFLOWS, AUTONOMY AND MULTI-AGENT SYSTEMS

Build multi-step workflows with triggers, schedules and human approvals, run them from the Inbox, compose sub-agents and Agent Gallery agents, register agents over A2A and A2UI, then cross to the pro-code side: ADK, Agent Runtime, Agent Identity and Model Armor.

Class 9: Workflows: Deterministic Automation with a Human in the Loop
Topics:
- Workflows in Workflow Builder: triggers, steps and actions on the visual canvas, and the difference between a deterministic step sequence and a non-deterministic chat agent
- Flow control that survives real inputs: conditions, branches, loops over lists, and the outputs handed from one step to the next
- Human-in-the-loop as architecture: approval stages where a task waits for a person to sign off, who the approver is on paper, and what the approval message must contain
- Running a workflow three ways: on an automated schedule, manually on demand, or by @-mentioning it inside a Gemini Enterprise chat conversation
- The Inbox: the unified hub for monitoring active and long-running agents, adjusting configurations and receiving real-time status alerts through email and chat
- The approvals pattern as the single most reused business workflow: request, validate, route, approve, act, report
- Notification etiquette: a workflow that spams gets disabled by its own users, so frequency and channel are design decisions

Projects built:
- Build Expense Helper: a workflow that collects an expense claim, validates it against policy, pauses for a manager's approval, files the result in the right system and reports back, with tested error paths for every step

Practice: Trigger Expense Helper from chat, from the canvas and from a schedule, and record how the Inbox shows each run.

Assessment: Given five business tasks, assign each to chat agent, workflow, Skill or plain knowledge, and justify each assignment in two sentences.

Class 10: Triggers, Schedules and Hardened Autonomy
Topics:
- Trigger-based and schedule-based agents: workflows that start from a business event or a recurrence rather than a person, and the generative planning they rely on
- The first autonomous job: small, observable, reversible and logged, with the operations note that says what it does, when, with whose permissions, and how to switch it off
- Idempotence, retries and safe defaults: the three properties every unattended workflow must prove before it touches a real system
- Failure diaries: structured capture of what went wrong so Tuesday's incident is not repeated on Thursday
- Scope creep in autonomy: the it-could-also-just trap, and the change-control habit that resists it
- Observability settings per agent (Public Preview since June 2026) and what they add to the traces you will read in Class 13
- Autonomy scoping for the capstone: which jobs deserve a trigger, which must stay human-initiated, and why

Projects built:
- Build Weekly Reporter: a scheduled workflow that compiles a weekly status summary from real sources, drafts it into a Google Doc, and posts it for review, never directly to stakeholders; then harden it with an approval pause, an error diary and a self-report of its own actions

Practice: Let Weekly Reporter run unattended for three real cycles and audit every action against what you intended, scoring each.
Assessment: Live drill: your mentor corrupts one input source; your workflow must fail gracefully and your diary must show why, unedited.

Class 11: Multi-Agent Systems: Sub-Agents, the Agent Gallery, A2A and A2UI
Topics:
- When one agent becomes several: the router-plus-specialists pattern, sub-agents inside Workflow Builder, and the hand-off contracts that keep them debuggable
- The Agent Gallery: partner-built agents from the Google Cloud Marketplace (Adobe, Salesforce, ServiceNow, Workday and Oracle among the launch partners) inside the Gemini Enterprise app, behind a request-and-approval gateway
- The Agent2Agent protocol: created by Google, donated to the Linux Foundation in 2025, now supported by more than 150 organisations and integrated across Google, Microsoft and AWS platforms; what an agent card is and how discovery and delegation work
- Registering A2A agents in Gemini Enterprise (GA since 17 August 2026), including agents built elsewhere, and the governance that applies to them through Agent Registry
- A2UI: the companion standard for agent-driven user interfaces, with v0.9 Material Design components supported in Gemini Enterprise since August 2026
- Failure across agent boundaries: timeouts, unclear hand-offs and the tracing habits that keep multi-agent systems debuggable
- Architecture review: when multi-agent is real engineering and when it is complexity for its own sake

Projects built:
- Refactor Support Triage into a two-agent design: a routing agent plus a specialist sub-agent, with documented hand-off contracts and a failure-mode table; then register an external A2A agent and delegate one task to it

Practice: Diagram your capstone as both a single agent and a multi-agent design, and write the three-sentence verdict on which ships.
Assessment: Defend one hand-off contract: what crosses the boundary, what must never cross it, and what happens when the far side fails.

Class 12: The Pro-Code Bridge: ADK, Agent Runtime, Agent Identity and Model Armor
Topics:
- When no-code is not enough, and the platform's answer: the Agent Development Kit (ADK), Google's open-source framework in Python, Java and Go, with the graph-based structure for organising agents into networks of sub-agents and native MCP support
- Reading and writing a real ADK agent: an LlmAgent with a Gemini model, tools, instructions and sub-agents, run locally with the ADK developer UI before anything is deployed
- Agent Runtime: the managed runtime for fast, persistent agents, with Sessions and Memory Bank (GA December 2025), Code Execution (GA February 2026) and metered usage since 11 February 2026; one-command deployment with adk deploy
- Bringing code agents into the app: ADK agents hosted on Agent Runtime can be registered with Gemini Enterprise (since April 2026), so business users run what developers build
- The security layer by its exact names: Agent Identity (a unique cryptographic identity per agent for traceability), Agent Gateway (the control point between agents and data), and Model Armor (protection against prompt injection, tool poisoning and sensitive data leakage)
- Agent Simulation: stress-testing an agent against real-world scenarios before deployment, and how it industrialises the discipline you built by hand in Class 3
- The decision tree you will reuse for years: Workflow Builder, ADK, or a partner agent from the Gallery, chosen by governance posture, team skills and cost, not fashion

Projects built:
- Build and run a small ADK agent in Python that wraps one tool your capstone needs, deploy it to Agent Runtime, register it in Gemini Enterprise, and call it from a Workflow Builder agent; document the identity and Model Armor settings you applied

Practice: Rewrite one Workflow Builder step as an ADK tool and one ADK behaviour as a Workflow Builder step, and write down which felt right and why.
Assessment: Phase 3 exam: given a use case, design the full architecture (no-code versus pro-code split, agents and sub-agents, protocols, identity, guardrails) and defend it.

PHASE 4: OBSERVABILITY, COST, GOVERNANCE AND CAPSTONE

Prove agents work and know what they cost: tracing and metrics, seat and pay-as-you-go economics, data residency and policy controls, the Professional Agentic Architect exam map, then a production capstone defended with its governance and cost pack.

Class 13: Observability: Traces, Metrics and the Improvement Loop

Topics:

- Observability for Gemini Enterprise, GA since 24 June 2026: the metrics dashboards and trace viewing in the Observability tab, and the latency and error-rate views (p50 and p95, GA 3 September) that tell you whether an agent is getting slower
- Reading a trace: the execute_tool and invoke_connector spans added in August 2026, and how a slow connector, a failing tool and a bad prompt each look different
- Standards you can export: agent telemetry aligned with the OpenTelemetry generative AI semantic conventions, gen_ai attributes in Cloud Logging, and the Cloud Monitoring dimensions (tool_id, engine_id, response_code) for connectors
- The improvement loop as a weekly ritual: dashboards, hypothesis, one change, retest, remeasure
- Finding the gap: unanswered and escalated questions, ranked, as the roadmap for the next iteration
- Instrumenting the capstone: the three numbers that will define success in front of your leadership
- Observability as governance evidence: the artefact you hand a reviewer instead of a demo

Projects built:

- Observability sprint: run one agent with at least five real users for a week, review its traces and metrics, ship two data-demanded improvements, and write the before-and-after with p95 latency and error rate on the page

Practice: Draft the monthly one-page agent report your leadership would actually read: usage, outcomes, latency, errors, next actions.
Assessment: From a set of traces, diagnose the top failure pattern, propose the fix, and prove it with a retest.

Class 14: Cost: Seats, Pay-As-You-Go, Overage and Runtime Metering

Topics:

- The seat model: Business, Standard and Plus editions of Gemini Enterprise (Business around 21 US dollars per seat per month and Standard around 30 at launch, Plus through sales), what each edition includes, and which features (Workflow Builder among them) the Business edition does not unlock
- The pay-as-you-go edition (GA 1 August 2026, invoice-based billing, no pooled quotas) and the AI developer tools bundle, now open to all invoiced Cloud Billing accounts
- Overage billing controls (launched 17 August 2026, opened to all invoiced accounts on 1 September): what an overage cap protects you from and what it silently switches off
- Agent Runtime economics: Sessions, Memory Bank and Code Execution have been metered since 11 February 2026, so a pro-code agent has a bill a no-code agent does not
- Model cost as a design input: Pro versus Flash models, why the Flash line moved through 3.5, 3.6, 3.7 and 3.8 in four months, and how to read Google's pricing pages rather than memorise numbers that change
- Modelling a real workload: seats, agents, sessions per week, runtime hours and model calls, at three adoption levels, under seats versus pay-as-you-go
- Cost as a design input: grounding choices, action counts, model tier and autonomy all move the bill, and you now know roughly by how much

Projects built:

- Build the complete cost model for your capstone: edition recommendation, seat count, pay-as-you-go versus seats comparison, Agent Runtime line if any, overage cap, and a monthly projection at three adoption levels

Practice: Re-estimate the model twice, once with a Pro model and once with a Flash model on the same test set, and write down what the difference buys.
Assessment: Given a described workload, produce the edition recommendation, the cost estimate and the overage plan in 30 minutes.

Class 15: Governance: Residency, Policy Constraints, Content Policies and the Certification Map

Topics:

- Data residency you can promise: DRZ and MLP commitments per region, the India and Singapore regions GA in June 2026, Japan and the UK in July, and what an Indian enterprise can now sign off on
- Managed organisation policy constraints for data connectors (GA since 7 July 2026): restricting allowed sources and egress FQDNs, and the custom MCP server constraint that ships disabled by default
- Content policies for sensitive data protection (GA since 31 August 2026): what the policies catch, where they apply, and how they pair with Model Armor on the pro-code side
- Identity and access end to end: Gemini Enterprise Admin and User roles, Okta SCIM and third-party identity (BYOID) for the app, Agent Identity for agents, and the Agent Registry as the record of what exists

- Compliance surfaces: Assured Workloads with FedRAMP High for federated data stores, VPC Service Controls, and the honest note that a live web crawl can violate a perimeter
- The certification map: Google Cloud's Professional Agentic Architect (a three-hour proctored exam of around 80 questions plus hands-on labs, in beta until 30 September 2026) and its five domains (low-code agents, coding agents, custom agents, evaluation and deployment, security and governance), with the Generative AI Leader credential as the on-ramp for teammates
- The 20-point production checklist covering grounding, actions, protocols, observability, cost, residency, policy and rollback

Projects built:

- Write the governance pack for your capstone: region and residency decision, organisation policy and content policy plan, identity and sharing plan, Agent Registry entries, compliance notes, and the rollout runbook with rollback

Practice: Role-play the security review with your mentor as the Cloud administrator: your pack must answer every who-approved-this and where-does-the-data-go question.

Assessment: Run the 20-point production checklist against your capstone as it stands today and file every failure into the Class 16 build plan.

Class 16: Capstone Build and Defence

Topics:

- Freezing the spec: the capstone's job, users, sources, actions, workflows, triggers, agents, model, edition, region and cost model on one page
- Building in production order: knowledge first, chat agent second, actions third, workflows and triggers fourth, sub-agents last, with the test set running throughout
- Beating your baseline: the Class 3 test set and the Class 13 observability numbers as the bar, rerun at every milestone
- Polish that earns trust: greeting, transparency statement, escalation tone, approval messages, error messages
- The live defence: your agent estate demonstrated in the Gemini Enterprise app, with traces, the cost model and the governance pack on the table
- Defending decisions like an architect: why these sources, this no-code versus pro-code split, these actions, this model, this region, this edition
- The Living Syllabus promise: what changed in Gemini Enterprise while you were learning it, reviewed together, plus your next step, whether that is the Professional Agentic Architect exam or the internal pitch for your organisation's agent playbook

Projects built:

- Deliver the capstone: a production-grade agent estate for your own team on Gemini Enterprise, with grounded knowledge, at least one real action, one governed workflow with a human approval, a passing test set, observability in place, and the complete cost and governance pack, defended live

Practice: Run the dress rehearsal for a colleague outside the course and fix the two weakest moments they point at.

Assessment: Final evaluation: capstone defence, governance and cost pack review, and the complete documentation packet, graded against the Class 15 production checklist.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: an agent opportunity memo for your organisation, Policy Desk v1 with a tested instruction stack, a 25-question adversarial test set, and a defended model decision packaged as a Skill and a Project
- Phase 2: a Drive or SharePoint-grounded Policy Desk with citation proofs, a retrieval-optimised document rebuild with before-and-after accuracy, Support Triage v1 writing real ServiceNow, Jira or Slack actions, a governed custom MCP server integration with a security note, and a published 25-answer accuracy audit
- Phase 3: the Expense Helper approval workflow, the scheduled Weekly Reporter and its hardened successor with an operations note, a two-agent Support Triage with A2A delegation, and a Python ADK agent deployed to Agent Runtime and registered in Gemini Enterprise
- Phase 4: an observability sprint with shipped improvements, a complete edition and pay-as-you-go cost model, the governance pack with residency, policy and identity decisions, and the capstone build and live defence
- Total: 16 professional deliverables, one per class, finishing with a production capstone agent estate plus a governance and cost pack

Total projects: 16 professional deliverables, one per class, finishing with a production capstone agent estate plus a governance and cost pack

SKILLS THE STUDENT WILL HAVE AT THE END

- Agent design for business: instruction stacks, scope fences, refusals and escalation paths in Workflow Builder chat agents
- Enterprise knowledge grounding: Google Workspace and third-party data stores, federated versus indexed sources, filters and citations, within real platform limits
- Actions that write to real systems: Slack, ServiceNow, Jira, Confluence, GitHub and Microsoft Teams actions with confirmation, idempotence and designed failure paths
- Custom MCP servers under Agent Registry governance, and the security note that earns a policy override
- Workflows with triggers, schedules, flow control and human-in-the-loop approvals, monitored from the Inbox
- Multi-agent systems: sub-agents, Agent Gallery partner agents, A2A registration and A2UI interfaces

- The pro-code path: Agent Development Kit agents in Python, Agent Runtime with Sessions, Memory Bank and Code Execution, Agent Identity and Model Armor
- Observability: traces, p50 and p95 latency, error rates and OpenTelemetry-aligned telemetry, run as a weekly improvement loop
- Cost management: seat editions, pay-as-you-go, overage controls and Agent Runtime metering, modelled at three adoption levels
- Governance: data residency, organisation policy constraints, content policies, identity and sharing, Agent Registry, and the Professional Agentic Architect exam map

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 private 1-on-1 classes per week, 1 hour each, across 8 weeks (two months)
Hands on agent work: 2-3 hours building, testing and observing in Gemini Enterprise between classes
Project time: 1-1.5 hours completing each class deliverable, usually against your own organisation's use case
Review and doubts: 30-60 minutes reviewing feedback and clearing doubts over WhatsApp before the next class
Homework: A task after every class, and a phase-gate review at the end of each phase that deliberately folds in earlier topics

SUPPORT INCLUDED

Dedicated mentor: The same mentor for all 16 classes, who knows your organisation's context, your capstone and your governance constraints
Whatsapp doubts: Doubt support between classes over WhatsApp
Practice environment: Help arranging your Gemini Enterprise access in Class 1: employer environment, a pay-as-you-go project, or a practice environment we help you set up
Session recordings: Class recordings available for revision, so a missed detail is never lost

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- AI agent builder, automation specialist or agentic solutions roles, the titles now appearing in real job listings as organisations move from pilots to agent estates
- Google Cloud Professional Agentic Architect certification, for which this syllabus is direct preparation across all five exam domains, with the Generative AI Leader credential as a lighter first step
- The internal agent champion or centre-of-excellence role: the person who owns the organisation's agent playbook, governance posture and cost model on Gemini Enterprise
- Consulting and partner delivery, with a portfolio capstone that includes the governance and cost artefacts clients' security and finance teams actually ask for
- A bridge to code-first agent engineering: the ADK and Agent Runtime work here, and our Codex + Claude Code masterclass when you want to build agents entirely in code

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need to be a developer to build agents with Google Gemini Enterprise?
A: No. Workflow Builder, Skills, Projects, connectors and actions are all no-code, and the hard parts they involve are design, testing, governance and cost, which are not programming skills. The programme does cross into the Agent Development Kit for the pro-code path, but that phase is paced to you: developers go deep into ADK and Agent Runtime, while non-developers learn to read, direct and govern that work. Both audiences finish the same capstone.

Q: What is the difference between Gemini Enterprise and the Gemini Enterprise Agent Platform?
A: The Gemini Enterprise app is where teams discover, create, share and run agents: Workflow Builder, Skills, Projects, the Inbox, Canvas and the Agent Gallery live there. The Gemini Enterprise Agent Platform, announced at Google Cloud Next in April 2026 as the evolution of Vertex AI, is where developers build, scale, govern and optimise agents with the Agent Development Kit, Agent Runtime, Agent Identity, Model Armor and Agent Simulation. This course teaches both halves and, more importantly, when each is the right tool.

Q: Why is this course taught 1-on-1 only?
A: Because the material is your organisation's material. The mentor reviews your actual instruction stacks, workflows, traces and governance artefacts every session, and adjusts depth to your role, whether you are an analyst, an administrator or an architect.

Q: How is the course structured and how long does it take?
A: Sixteen private classes over two months: two live one-hour classes a week for eight weeks, in four phases of four classes each. Phase 1 covers the platform and agent foundations, Phase 2 knowledge, connectors and actions, Phase 3 workflows, autonomy and multi-agent systems

including the pro-code bridge to ADK, and Phase 4 observability, cost, governance and the capstone. Every class ends with a deliverable, and each phase ends with a gate review.

Q: What do I need in terms of Google Cloud access for the course?
A: A Google Cloud account with Gemini Enterprise access. Workflow Builder is documented for the Standard, Plus, pay-as-you-go and Frontline editions, so the Business edition alone will not unlock the workflow classes. The best path is your employer's environment with your administrator's awareness, and the course includes drafting exactly that request. Where that is not possible, we help you set up a pay-as-you-go project or a practice environment in the first class, at your own cloud cost, and your mentor sorts access out with you before the second class.

Q: Which Google Cloud certification does this course prepare me for?
A: The headline target is Google Cloud's Professional Agentic Architect certification, a three-hour proctored exam of around 80 multiple-choice questions followed by hands-on labs, in beta until 30 September 2026. Its five domains, building agents with low-code tools, using coding agents, developing custom agents, evaluating and deploying agentic workflows, and securing and governing them, track this syllabus phase by phase. The Generative AI Leader credential is the lighter on-ramp we recommend for teammates you later onboard.

Q: How current is the syllabus, given how fast Gemini Enterprise is changing?
A: It is a Living Syllabus, re-verified against Google Cloud's release notes as features ship. The current version covers the September 2026 platform: Workflow Builder GA, Skills and Projects, custom MCP server data stores, Agent Registry governance, A2A and A2UI registration, observability with p50 and p95 latency, pay-as-you-go and overage controls, the Gemini 3.1 Pro and 3.8 Flash lineup, and India-region data residency. When the platform changes, the course changes, and your mentor tells you what moved while you were learning.

Q: What do I walk away with at the end of the programme?
A: A production-grade agent estate for your own team on Gemini Enterprise, demonstrated and defended live, plus the full professional packet: source map, action inventory, test set, workflow and approval design, observability report, cost model, governance pack with residency, policy and identity decisions, and a rollout runbook. You also leave with a Modern Age Coders certificate earned by that defence, and a mapped route to the Professional Agentic Architect exam if you want Google's official stamp.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/build-ai-agents-with-google-gemini-enterprise-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 28 OF 137: BUSINESS & FINANCE MATHEMATICS: INVESTMENT TO RISK

==========

Course link: https://learn.modernagecoders.com/courses/complete-business-finance-mathematics-mastery
Subject area: Business & Finance Education
Designed for: Working professionals
Who can join: Complete Beginner to Advanced Professional
Course length: 12-14 months (52-60 weeks)
Time needed: 2 live classes/week + 3-5 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

A 1-year business and finance mathematics course covering investment analysis, risk management, accounting maths, derivatives and portfolio optimization.

In one line: From Basic Calculations to Quantitative Finance in 12 Months

ABOUT THE PROGRAMME

This intensive program transforms you into a financial mathematics expert capable of complex business analysis, investment decisions, and risk management. Whether you're aspiring to investment banking, corporate finance, actuarial science, or entrepreneurship, this course provides the mathematical foundation for success. You'll master everything from compound interest and annuities to options pricing and portfolio optimization. Each concept is taught through real-world business cases, Excel modeling, and Python applications. By completion, you'll think like a quantitative analyst and make data-driven financial decisions.

WHAT MAKES THIS COURSE DIFFERENT

- Business-focused from day one - no abstract math
- Excel and Python integration throughout
- Real company case studies and datasets
- Trading simulations and portfolio management
- Professional financial modeling templates
- CFA/FRM/Actuarial exam preparation included
- Industry-standard software training
- Live market data analysis

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Business Math Basics, Financial Calculations, Accounting Mathematics
Phase 2: Intermediate (Months 4-6): Statistics for Business, Economic Mathematics, Financial Analysis
Phase 3: Advanced (Months 7-9): Investment Mathematics, Risk Management, Corporate Finance
Phase 4: Professional (Months 10-12): Derivatives, Portfolio Theory, Quantitative Methods, Financial Modeling

WHAT A STUDENT NEEDS BEFORE STARTING

Education: High school mathematics
Coding experience: None required - will teach Excel/Python
Equipment: Computer with Excel, internet connection
Time commitment: 2 live classes plus 3 to 5 hours of practice a week
English: Business English comprehension
Motivation: Strong interest in finance/business

WHO THIS COURSE IS FOR

Students: Business/Finance/Economics students
Working professionals: Career switchers to finance
Entrepreneurs: Business owners needing financial skills
Freelancers: Financial consultants and advisors
Kids: Advanced high school students (16+)
Anyone: Anyone pursuing finance careers

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: BUSINESS MATHEMATICS FOUNDATIONS (WEEKS 1-13)

Build rock-solid foundations in business calculations, financial mathematics, and accounting principles. Master the mathematical tools used daily in business.

Months 1-2: Core Business Mathematics (Weeks 1-8)
Percentages & Business Applications
Topics:
- Percentage calculations and conversions
- Percentage increase and decrease
- Percentage points vs percentage change
- Markup and markdown calculations
- Profit margins: gross, operating, net
- Break-even analysis
- Sales tax and VAT calculations
- Discount calculations: single and series
- Commission structures and calculations
- Percentage allocation and distribution
- Market share calculations
- Growth rates and CAGR
Projects built:
- Profit margin calculator for business
- Sales commission tracker
- Break-even analysis tool
Practice: Daily: 50 business percentage problems with real scenarios
Time Value of Money Fundamentals
Topics:
- Simple interest calculations
- Compound interest formula and applications
- Continuous compounding
- Effective vs nominal interest rates
- Present value concepts
- Future value calculations
- Discounting and discount factors
- Rule of 72 and quick estimations
- Interest rate conversions

- Day count conventions
- Currency exchange calculations
- Inflation and real vs nominal values

Projects built:
- Compound interest calculator
- Investment growth simulator
- Currency converter with live rates

Practice: Complete 100 time value of money problems

Annuities & Loan Mathematics

Topics:
- Ordinary annuities and annuities due
- Present value of annuity streams
- Future value of annuity calculations
- Perpetuities and growing perpetuities
- Loan amortization schedules
- EMI (Equated Monthly Installment) calculations
- Principal and interest components
- Balloon payments and refinancing
- Mortgage mathematics
- Auto loan calculations
- Credit card mathematics
- Effective cost of borrowing

Projects built:
- Loan amortization calculator
- Mortgage comparison tool
- Retirement planning calculator

Practice: Build 10 different loan amortization schedules

Business Ratios & Analysis

Topics:
- Liquidity ratios: current, quick, cash
- Profitability ratios: ROE, ROA, ROCE
- Efficiency ratios: inventory turnover, receivables
- Leverage ratios: debt-to-equity, interest coverage
- Market ratios: P/E, P/B, dividend yield
- DuPont analysis
- Ratio trend analysis
- Industry comparison metrics
- Working capital calculations
- Cash conversion cycle
- Economic Value Added (EVA)
- Balanced scorecard metrics

Projects built:
- Financial ratio analyzer
- Company comparison dashboard
- Industry benchmark tool

Practice: Analyze 20 real company financial statements

Month 3: Accounting Mathematics & Cost Analysis (Weeks 9-13)

Accounting Mathematics Fundamentals

Topics:
- Double-entry bookkeeping mathematics
- Trial balance and mathematical checks
- Depreciation calculations: straight-line, declining balance
- MACRS depreciation
- Inventory valuation: FIFO, LIFO, weighted average
- Cost of Goods Sold calculations
- Gross profit and net profit calculations
- Balance sheet equation
- Cash flow calculations
- Accrual vs cash basis mathematics
- Bank reconciliation mathematics
- Foreign currency accounting

Projects built:
- Depreciation schedule generator
- Inventory management calculator
- Cash flow statement builder

Practice: Complete 100 accounting calculation problems

Cost Accounting & Budgeting

Topics:
- Fixed vs variable cost analysis
- Cost-Volume-Profit (CVP) analysis
- Contribution margin calculations
- Operating leverage
- Activity-based costing
- Standard costing and variance analysis
- Material, labor, and overhead variances
- Budget preparation mathematics
- Flexible budgeting
- Zero-based budgeting calculations

- Capital budgeting introduction
- Cost allocation methods

Projects built:
- CVP analysis tool
- Variance analysis dashboard
- Budget vs actual analyzer

Practice: Solve 75 cost accounting problems

Phase 1 Assessment & Integration

Topics:
- Integration of financial concepts
- Complex multi-step problems
- Case study methodology
- Financial calculator mastery
- Excel functions for finance
- Common business math errors
- Professional presentation of calculations

Projects built:
- Comprehensive business analysis case
- Build financial calculator app
- Create business math reference guide

Assessment: Phase 1 cumulative exam: a written paper mixing this phase with everything before it, plus targeted revision for anything shaky

---

PHASE 2: STATISTICAL ANALYSIS AND ECONOMIC MATHEMATICS (WEEKS 14-26)

Master statistical methods for business decision-making, economic analysis, and financial forecasting.

Months 4-5: Business Statistics & Analytics (Weeks 14-21)

Descriptive Statistics for Business

Topics:
- Business data types and collection methods
- Measures of central tendency in business context
- Variance and standard deviation in finance
- Coefficient of variation for risk
- Skewness and kurtosis in returns
- Business data visualization
- Time series components
- Moving averages and smoothing
- Seasonal adjustments
- Index numbers and deflators
- Business survey analysis
- Customer analytics metrics

Projects built:
- Sales data analyzer
- Customer behavior dashboard
- Market trend visualizer

Practice: Analyze 50 business datasets

Probability in Business Decisions

Topics:
- Probability in risk assessment
- Expected value in business decisions
- Decision trees and expected monetary value
- Bayesian analysis for business
- Conditional probability in market analysis
- Joint probability distributions
- Risk and return relationships
- Value at Risk (VaR) introduction
- Monte Carlo simulation basics
- Scenario analysis
- Sensitivity analysis
- Business forecasting with probability

Projects built:
- Decision tree analyzer
- Monte Carlo simulator for projects
- Risk assessment tool

Practice: Solve 100 business probability problems

Regression & Predictive Analytics

Topics:
- Simple linear regression for business
- Multiple regression analysis
- Interpreting regression coefficients
- R-squared and model fit
- Residual analysis
- Multicollinearity detection
- Dummy variables in regression
- Time series regression
- Logistic regression for classification

- Forecasting with regression
- Sales prediction models
- Price elasticity estimation

Projects built:
- Sales forecasting model
- Price optimization tool
- Customer churn predictor

Practice: Build 20 regression models with real data

Hypothesis Testing for Business

Topics:
- Hypothesis testing framework in business
- A/B testing mathematics
- Type I and Type II errors in business context
- T-tests for marketing campaigns
- ANOVA for multiple group comparison
- Chi-square tests for independence
- Non-parametric tests
- Sample size determination
- Power analysis for business experiments
- Confidence intervals for business metrics
- Statistical significance vs practical significance
- P-value interpretation pitfalls

Projects built:
- A/B testing calculator
- Marketing campaign analyzer
- Quality control dashboard

Practice: Conduct 50 business hypothesis tests

Time Series Analysis

Topics:
- Time series decomposition
- Trend analysis methods
- Seasonal patterns identification
- Cyclical variations
- Autocorrelation and ACF
- ARIMA models introduction
- Exponential smoothing methods
- Holt-Winters forecasting
- Leading and lagging indicators
- Business cycle analysis
- Forecast accuracy measures
- Real-time data analysis

Projects built:
- Time series forecasting tool
- Seasonal adjustment calculator
- Economic indicator dashboard

Practice: Analyze 25 business time series

Month 6: Economic & Market Mathematics (Weeks 22-26)

Microeconomic Mathematics

Topics:
- Supply and demand curves
- Elasticity calculations
- Consumer and producer surplus
- Utility maximization mathematics
- Budget constraint optimization
- Production functions
- Cost curves and optimization
- Profit maximization calculus
- Market equilibrium calculations
- Game theory basics
- Nash equilibrium
- Oligopoly and monopoly pricing

Projects built:
- Market equilibrium calculator
- Pricing strategy optimizer
- Game theory simulator

Practice: Solve 75 microeconomic problems

Macroeconomic Mathematics

Topics:
- GDP calculations and components
- Inflation rate calculations
- Real vs nominal GDP
- Unemployment rate metrics
- Money multiplier effect
- Interest rate parity
- Exchange rate calculations
- Purchasing power parity
- IS-LM model basics
- Phillips curve relationship

- Economic growth models
- Business cycle indicators

Projects built:
- Economic indicator tracker
- Currency exchange analyzer
- Inflation impact calculator

Practice: Complete 50 macroeconomic calculations

Financial Markets Mathematics

Topics:
- Stock market indices calculation
- Market capitalization weighting
- Price-weighted vs value-weighted indices
- Bond pricing fundamentals
- Yield to maturity calculations
- Duration and convexity
- Term structure of interest rates
- Forward rates calculation
- Swap valuation basics
- Foreign exchange forwards
- Covered interest parity
- Market efficiency metrics

Projects built:
- Bond pricing calculator
- Yield curve analyzer
- Index tracking tool

Practice: Price 50 different financial instruments

International Finance Mathematics

Topics:
- Exchange rate systems
- Cross rates calculation
- Triangular arbitrage
- Forward exchange rates
- Interest rate parity theory
- International Fisher effect
- Currency hedging calculations
- Translation and transaction exposure
- International capital budgeting
- Country risk assessment
- Emerging market analysis
- Global portfolio considerations

Projects built:
- Currency arbitrage detector
- International investment analyzer
- Hedging strategy calculator

Practice: Analyze 30 international finance scenarios

Phase 2 Capstone Project

Topics:
- Integrated statistical analysis
- Economic forecasting project
- Market analysis presentation
- Data visualization mastery
- Professional reporting
- Peer review process

Projects built:
- Complete market analysis report
- Economic forecast model
- Statistical analysis of business case

Assessment: Phase 2 cumulative exam: a written paper mixing this phase with everything before it, plus targeted revision for anything shaky

---

PHASE 3: INVESTMENT MATHEMATICS AND RISK MANAGEMENT (WEEKS 27-39)

Master investment analysis, portfolio management, risk assessment, and corporate finance decisions.

Months 7-8: Investment Analysis & Valuation (Weeks 27-34)

Equity Valuation Mathematics

Topics:
- Discounted Cash Flow (DCF) models
- Free cash flow calculations
- WACC (Weighted Average Cost of Capital)
- Terminal value calculations
- Dividend discount models
- Gordon growth model
- Multi-stage growth models
- Price multiples valuation
- Comparable company analysis
- Precedent transaction analysis

    - Asset-based valuation
    - Sum-of-the-parts valuation
  Projects built:
    - DCF valuation model
    - Comparable analysis template
    - Valuation dashboard
  Practice: Value 25 real companies using different methods
Fixed Income Mathematics
  Topics:
    - Bond pricing with different frequencies
    - Yield measures: current, YTM, YTC
    - Duration: Macaulay and modified
    - Convexity calculations
    - Immunization strategies
    - Credit spreads and ratings
    - Floating rate notes
    - Convertible bond valuation
    - Mortgage-backed securities basics
    - Prepayment risk modeling
    - Z-spread and OAS
    - Fixed income portfolio management
  Projects built:
    - Bond portfolio analyzer
    - Duration matching tool
    - Yield curve trading strategy
  Practice: Analyze 50 different bond structures
Portfolio Theory & Optimization
  Topics:
    - Risk and return measures
    - Portfolio expected return
    - Portfolio variance and standard deviation
    - Correlation and covariance matrices
    - Diversification benefits
    - Efficient frontier construction
    - Markowitz optimization
    - Capital Asset Pricing Model (CAPM)
    - Beta calculation and interpretation
    - Security Market Line
    - Arbitrage Pricing Theory basics
    - Factor models
  Projects built:
    - Portfolio optimizer
    - Efficient frontier visualizer
    - Beta calculator for stocks
  Practice: Construct 20 optimal portfolios
Performance Measurement & Attribution
  Topics:
    - Absolute vs relative returns
    - Time-weighted vs money-weighted returns
    - Sharpe ratio calculation
    - Treynor ratio
    - Jensen's alpha
    - Information ratio
    - Sortino ratio
    - Maximum drawdown
    - Value at Risk (VaR) methods
    - Conditional VaR (CVaR)
    - Performance attribution analysis
    - Risk-adjusted performance measures
  Projects built:
    - Performance analytics dashboard
    - Risk metrics calculator
    - Attribution analysis tool
  Practice: Evaluate 30 portfolio performances
Alternative Investments Mathematics
  Topics:
    - Real estate investment mathematics
    - Cap rates and NOI calculations
    - Private equity valuation
    - Venture capital metrics
    - Hedge fund fee structures
    - Commodity futures pricing
    - Infrastructure project valuation
    - Art and collectibles valuation
    - Cryptocurrency valuation models
    - Crowdfunding mathematics
    - REIT analysis
    - Master Limited Partnerships
  Projects built:

- Real estate investment analyzer
- Private equity return calculator
- Alternative asset portfolio tool

Practice: Analyze 20 alternative investments

Month 9: Risk Management & Insurance (Weeks 35-39)

Risk Measurement & Modeling

Topics:

- Types of financial risk
- Risk metrics: volatility, beta, duration
- Value at Risk (VaR) models
- Historical simulation
- Parametric VaR
- Monte Carlo VaR
- Stress testing methodologies
- Scenario analysis frameworks
- Credit risk modeling
- Probability of default estimation
- Loss given default
- Expected loss calculations

Projects built:

- VaR calculator (3 methods)
- Stress testing framework
- Credit risk scorecard

Practice: Calculate risk metrics for 40 portfolios

Insurance Mathematics

Topics:

- Life insurance mathematics
- Mortality tables and life expectancy
- Present value of life insurance
- Annuity calculations for insurance
- Premium calculation principles
- Reserve calculations
- Property insurance mathematics
- Deductibles and co-insurance
- Loss distributions
- Ruin theory basics
- Reinsurance mathematics
- Health insurance metrics

Projects built:

- Life insurance premium calculator
- Insurance portfolio analyzer
- Claims prediction model

Practice: Solve 50 actuarial problems

Operational & Market Risk

Topics:

- Operational risk measurement
- Key risk indicators (KRIs)
- Loss distribution approach
- Market risk factors
- Greeks: delta, gamma, theta, vega, rho
- Hedging strategies mathematics
- Basis risk calculations
- Liquidity risk measures
- Counterparty risk assessment
- Systemic risk indicators
- Risk appetite frameworks
- Economic capital calculation

Projects built:

- Greeks calculator
- Hedging strategy optimizer
- Risk dashboard creator

Practice: Design 25 risk management strategies

Regulatory Capital & Compliance

Topics:

- Basel III requirements
- Capital adequacy ratios
- Tier 1 and Tier 2 capital
- Risk-weighted assets calculation
- Leverage ratio
- Liquidity coverage ratio (LCR)
- Net stable funding ratio (NSFR)
- IFRS 9 impairment models
- Solvency II for insurance
- Dodd-Frank Act implications
- MiFID II requirements
- Regulatory reporting mathematics

Projects built:

- Capital adequacy calculator

- Regulatory report generator
- Compliance tracking tool

Practice: Calculate regulatory metrics for 20 institutions

Phase 3 Integration & Assessment

Topics:

- Integrated risk management
- Portfolio construction with constraints
- Risk-return optimization
- Comprehensive case studies
- Professional presentation skills
- Industry best practices

Projects built:

- Complete investment analysis case
- Risk management framework design
- Portfolio optimization project

Assessment: Phase 3 cumulative exam: a written paper mixing this phase with everything before it, plus targeted revision for anything shaky

---

PHASE 4: ADVANCED FINANCE AND FINAL REVISION (WEEKS 40-52)

Master derivatives pricing, quantitative finance, financial modeling, and advanced corporate finance.

Month 10: Derivatives & Structured Products (Weeks 40-45)

Options Mathematics

Topics:

- Call and put option payoffs
- Intrinsic and time value
- Put-call parity
- Option strategies: spreads, straddles, strangles
- Binomial option pricing model
- Risk-neutral valuation
- Black-Scholes model derivation
- Black-Scholes formula application
- Implied volatility calculation
- American option valuation
- Exotic options: barrier, Asian, lookback
- Real options valuation

Projects built:

- Options pricing calculator
- Strategy payoff visualizer
- Implied volatility surface

Practice: Price 100 different option structures

Futures & Forwards

Topics:

- Forward pricing theory
- Cost of carry model
- Futures margin mechanics
- Basis and basis risk
- Hedging with futures
- Optimal hedge ratios
- Duration-based hedging
- Cross-hedging
- Commodity futures pricing
- Interest rate futures
- Currency futures strategies
- Index futures and arbitrage

Projects built:

- Futures pricing model
- Hedge effectiveness analyzer
- Arbitrage opportunity scanner

Practice: Design 50 hedging strategies

Swaps & Interest Rate Derivatives

Topics:

- Interest rate swap mechanics
- Swap valuation methods
- Currency swap valuation
- Credit default swaps
- Total return swaps
- Swap spread analysis
- Forward rate agreements (FRAs)
- Caps, floors, and collars
- Swaptions
- Constant maturity swaps
- Basis swaps
- Structured notes design

Projects built:

- Swap pricing calculator

      - Structured product designer
      - Interest rate hedge optimizer
    Practice: Structure 40 derivative solutions
  Credit Derivatives & Structured Finance
    Topics:
      - Credit default swap pricing
      - CDO and CLO structures
      - Synthetic CDOs
      - Credit spread options
      - First-to-default baskets
      - Correlation trading
      - Securitization mathematics
      - Waterfall structures
      - Tranching and subordination
      - Prepayment modeling
      - Default correlation
      - Copula models basics
    Projects built:
      - CDO structuring tool
      - Credit portfolio simulator
      - Securitization analyzer
    Practice: Analyze 30 structured products
  Volatility & Advanced Greeks
    Topics:
      - Volatility smile and skew
      - Term structure of volatility
      - Local volatility models
      - Stochastic volatility basics
      - Volatility trading strategies
      - Vega hedging
      - Higher-order Greeks
      - Cross-Greeks
      - Portfolio Greeks management
      - Dynamic hedging
      - Volatility arbitrage
      - Dispersion trading
    Projects built:
      - Volatility surface modeler
      - Greeks risk manager
      - Dynamic hedging simulator
    Practice: Implement 25 volatility strategies

Month 11: Quantitative Finance and Modelling (Weeks 46-48)
  Financial Modeling Excellence
    Topics:
      - Financial model architecture
      - Three-statement modeling
      - Revenue and cost projections
      - Working capital modeling
      - Debt schedules and waterfalls
      - Circular reference resolution
      - Scenario and sensitivity tables
      - Monte Carlo in Excel
      - Model auditing techniques
      - Dashboard design
      - VBA for financial modeling
      - Model documentation standards
    Projects built:
      - Complete financial model template
      - LBO model
      - Project finance model
    Practice: Build 10 professional financial models
  Algorithmic Trading Mathematics
    Topics:
      - Trading system components
      - Backtesting methodology
      - Performance metrics for strategies
      - Market microstructure
      - Order types and execution
      - Slippage and transaction costs
      - Mean reversion strategies
      - Momentum strategies
      - Pairs trading mathematics
      - Statistical arbitrage
      - High-frequency trading basics
      - Machine learning in trading
    Projects built:
      - Trading strategy backtester
      - Pairs trading system

- Performance analytics platform
    Practice: Backtest 20 trading strategies
  Corporate Finance Advanced
    Topics:
      - Advanced capital budgeting
      - Real options in capital budgeting
      - Cost of capital estimation
      - Capital structure optimization
      - Modigliani-Miller theorems
      - Trade-off theory
      - Pecking order theory
      - Dividend policy mathematics
      - Share buyback analysis
      - Merger and acquisition valuation
      - Synergy valuation
      - Leveraged buyout modeling
    Projects built:
      - M&A valuation model
      - LBO returns analyzer
      - Capital structure optimizer
    Practice: Analyze 25 corporate finance decisions

Month 12: Professional Applications and Revision (Weeks 49-50)
  Industry Case Studies
    Topics:
      - Investment banking case studies
      - Private equity deal analysis
      - Hedge fund strategies review
      - Insurance company ALM
      - Pension fund management
      - Sovereign wealth funds
      - Central bank operations
      - Corporate treasury management
      - Family office structures
      - Endowment management
    Projects built:
      - Complete industry analysis
      - Deal structure proposal
      - Investment committee presentation
    Practice: Analyze 10 real-world cases
  Financial Technology Tools
    Topics:
      - Bloomberg Terminal mastery
      - Reuters Eikon navigation
      - FactSet essentials
      - S&P Capital IQ
      - Morningstar Direct
      - Python for finance
      - R for financial analysis
      - SQL for financial data
      - Tableau for finance
      - Power BI dashboards
      - Advanced Excel techniques
      - Financial APIs integration
    Projects built:
      - Multi-source data aggregator
      - Automated report generator
      - Real-time market dashboard
    Practice: Master 10 professional tools

Month 13: Final Projects & Certification (Weeks 51-52)
  Capstone Project - Part 2 & Presentation
    Topics:
      - Final model refinement
      - Peer review process
      - Presentation preparation
      - Executive summary writing
      - Q&A preparation
      - Professional formatting
      - Submission guidelines
      - Public presentation
    Deliverables:
      - Complete financial model
      - 50-page analysis report
      - Executive presentation
      - Video walkthrough
      - Peer review feedback
      - GitHub repository
  Final Assessment & Certification

Topics:
- Comprehensive knowledge review
- Final examination preparation
- Portfolio compilation
- Skills self-assessment
- Future learning roadmap
- Alumni network access
- Industry connections
- Certification ceremony

Deliverables:
- Final comprehensive exam (4 hours)
- Complete project portfolio
- Professional references
- Certification of completion
- LinkedIn certification badge
- Transcript of achievements

Assessment: FINAL EXAM: a comprehensive written paper covering every phase of the course. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Business calculators, Loan analyzers, Accounting tools
- Phase 2: Statistical models, Economic forecasts, Regression analyses
- Phase 3: Investment valuations, Portfolio optimizers, Risk dashboards
- Phase 4: Derivatives pricers, Trading systems, Financial models
- Final: Professional-grade financial analysis project

Total projects: 150+ financial tools, models, calculators, and analytical dashboards

SKILLS THE STUDENT WILL HAVE AT THE END

- Financial Mathematics: TVM, annuities, loans, bonds, derivatives
- Statistical Analysis: Regression, time series, hypothesis testing, forecasting
- Investment Analysis: Valuation, portfolio theory, performance measurement
- Risk Management: VaR, stress testing, credit risk, operational risk
- Corporate Finance: Capital budgeting, capital structure, M&A, LBO
- Quantitative Methods: Monte Carlo, optimization, algorithmic trading
- Financial Modeling: Excel, VBA, Python, professional models
- Professional Tools: Bloomberg, Excel advanced, Python, SQL
- Certifications Prep: CFA, FRM, actuarial exam readiness

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week with a real teacher, working problems together
Homework: A problem set after every class, and a mixed-review assignment at the end of every month that deliberately folds in earlier topics
Practice: 3-5 hours of problem-solving between classes
Review: Every set reviewed with written feedback; recurring mistakes reopened in class until they are gone

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: which topics are solid, which are wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies mastery, not attendance

WHERE THIS CAN LEAD

- Financial Analyst
- Investment Banking Analyst
- Risk Analyst
- Equity Research Analyst
- Credit Analyst
- Portfolio Manager
- Quantitative Analyst
- Corporate Finance Manager
- Financial Consultant
- Actuarial Analyst

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and

abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Growing roles: Rs 4-8 LPA (Entry-level analyst)
Experienced roles: Rs 8-15 LPA (Financial Analyst); Rs 12-25 LPA (Senior Analyst); Rs 15-40 LPA (Manager/Specialist)
Freelance: Rs 1,000-5,000/hour consulting
International: $60k-150k based on role and location

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will I learn in the Business & Finance Mathematics program?
A: Across 12 months you move from core business maths, percentages, interest, financial calculations and accounting mathematics, through business statistics and economic mathematics, into investment analysis, valuation and risk management, and finish with derivatives, portfolio theory and quantitative financial modeling. Excel is used throughout, with Python introduced for the quantitative work.

Q: Who is this course for?
A: It starts from genuine basics, so you do not need a finance background to begin.

Q: Do I need Excel or coding skills before starting?
A: No. High school mathematics is the only prerequisite; Excel is taught from scratch inside the course and Python is introduced gently when the quantitative topics need it. You will need a computer with Excel and a stable internet connection.

Q: How is progress measured?
A: Through weekly applied exercises, case style projects such as building valuation and risk models, and phase-end assessments. By the final months you are building complete financial models, which become the clearest proof of progress.

Q: Is this course theoretical or practical?
A: Deliberately practical: every concept is anchored in a business situation, from break-even analysis and loan amortization to portfolio optimization. The final phase is essentially a professional toolkit of models you can reuse in a job, a business or further study.

Q: How is my child assessed, and is the certificate just for attending?
A: No. A problem set follows every class with written feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, each phase ends with a cumulative exam, and the course ends with a final exam covering the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest. We measure understanding, not attendance.

Q: Can I sit in on a class first?
A: Yes, start with a free demo class, no commitment: book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. The demo also helps us gauge your starting level honestly. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live?
A: Every session is a live one hour online class with a real teacher; nothing is pre-recorded.

Q: What certification do I receive?
A: The Professional Business & Finance Mathematics Certificate on completion, plus a portfolio of the financial models and case projects you built, which is often more persuasive to employers than the certificate itself.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/complete-business-finance-mathematics-mastery
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 29 OF 137: C PROGRAMMING FOR BEGINNERS: POINTERS, MEMORY & STRUCTS
==========

Course link: https://learn.modernagecoders.com/courses/c-programming-course-for-college-beginners
Subject area: Programming Languages
Designed for: College students
Who can join: Beginner, college students and first-year engineers
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

C programming course for college students and beginners: live online classes on pointers, memory, structs, file handling and data structures, taught from zero.

In one line: Most C courses skip past pointers as fast as possible. This one is built around them.

ABOUT THE PROGRAMME

C is the language underneath everything: operating systems, databases, embedded firmware, and the interpreters that run Python and JavaScript are written in it. It is also the language most engineering students meet first and understand least, because the part that matters, pointers and memory, is usually rushed in two lectures. This 6-month live course inverts that. Months 1 and 2 build the language core properly: types, control flow, functions, and how a .c file actually becomes an executable. Months 3 and 4 are the heart of the course: a full month on pointers taught with memory diagrams for every operation, then a month on dynamic memory, the heap, and structs, including the bugs (leaks, dangling pointers, buffer overflows) demonstrated live and safely. Months 5 and 6 put it to work: file handling, linked lists, stacks, queues, and sorting built from scratch, a look at where C runs in the real world from kernels to microcontrollers, and a capstone project compiled, debugged, and defended. Classes are live and small. Students write and compile code in every session, homework is real programming with written feedback, and the instructor draws memory on a whiteboard until it stops being mysterious.

WHAT MAKES THIS COURSE DIFFERENT

- Pointers get a full month, not two lectures, and every operation comes with a memory diagram before any code
- Memory bugs are taught on purpose: students create a leak, a dangling pointer, and an overflow in controlled exercises, then learn to catch them
- Standard library functions like strlen and strcpy are written by hand first, so nothing in string.h is magic
- Data structures are built from raw structs and malloc, the way university exams and real systems code expect
- The compilation pipeline is taught, not skipped: preprocessor, compiler, linker, and what an executable actually is
- The syllabus deliberately covers the C programming paper taught in Indian engineering first year, so college exams and this course reinforce each other

THE LEARNING PATH, STEP BY STEP

Phase 1: Language core and the machine: types, control flow, functions, the stack, and how source code becomes an executable
Phase 2: The heart of C: pointers with memory diagrams, arrays and strings at the memory level, malloc and the heap, and structs
Phase 3: C at work: file handling, linked lists, stacks, queues, sorting, bitwise operations and embedded context, then a capstone

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None required. The course starts from the first compile. Students who know another language will simply move faster in month 1
Mathematics: School-level maths is plenty; no calculus or advanced topics are needed
Equipment: Any Windows, Mac, or Linux computer that can run a compiler and VS Code, plus a stable internet connection
Mindset: Willingness to trace code on paper. C rewards students who slow down, and the course is built around that

WHO THIS COURSE IS FOR

First year engineers: College students taking the compulsory C paper who want to actually understand it, not just pass it
Cs and it students: Students whose degree assumes C fluency for data structures, operating systems, and GATE later
Self taught switchers: Programmers from Python or JavaScript who keep hearing about pointers and want the real story
Embedded curious: Students eyeing electronics, robotics, or firmware, where C remains the working language
Fundamentals seekers: Anyone who wants to know what actually happens under high-level languages, one memory diagram at a time

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE LANGUAGE CORE AND THE MACHINE (MONTHS 1-2, WEEKS 1-8)

C fundamentals built on honest ground: what the compiler does, what types really are, and how functions and the call stack work, so the pointer month lands on solid footing.

Month 1: From Source Code to Executable (Weeks 1-4)

Hello, Compiler

Topics:

- Setting up GCC and VS Code on Windows, Mac, or Linux
- Your first program: main, printf, and the return value
- The compilation pipeline: preprocessor, compiler, linker, executable
- Compiling from the terminal and reading compiler errors calmly
- Why C has no training wheels, and why that is the point
- Comments, formatting, and code that reads like it was written on purpose

Projects built:

- Banner program: compile, break, and fix a program five different ways to learn the error messages early

Practice: Compile six small programs from the terminal, deliberately triggering and decoding one error each

Types, Variables, and Their Limits

Topics:

- int, char, float, double, and the unsigned variants
- sizeof: asking the machine how big things are
- Integer overflow demonstrated live: what happens at the edge
- Type conversion and casting, implicit and explicit
- printf and scanf format specifiers, and the crashes wrong ones cause
- Constants with const and #define

Projects built:

- Limits explorer: a program that prints the size and range of every basic type on your machine

Practice: Ten input-output exercises with mixed types, including two that overflow on purpose so you see it happen

Operators and Control Flow

Topics:

- Arithmetic operators and the integer division surprise
- Relational and logical operators, and short-circuit evaluation
- if, else if, else, and the switch statement
- The ternary operator, used where it helps
- Operator precedence: when to just use parentheses
- = vs ==: the classic C bug, met early

Projects built:

- Grade and tax calculator: branching logic with real edge cases, negative inputs, and boundary values

Practice: Twelve control-flow drills, including a leap year checker and a menu-driven calculator with switch

Loops

Topics:

- while, for, and do-while, and choosing between them
- Loop counters, accumulators, and running totals
- Nested loops: printing patterns the exam way
- break and continue, used sparingly
- Off-by-one errors: finding them before the compiler cannot
- Tracing a loop by hand: the skill exams actually test

Projects built:

- Pattern engine: pyramids, diamonds, and number triangles from nested loops
- Prime and perfect number finder with a hand-traced dry run submitted alongside the code

Practice: Fifteen loop problems ranging from digit reversal to a simple guessing game

Assessment: Month 1 lab exam: three programs written live under light time pressure, compiled and demonstrated

Month 2: Functions and the Call Stack (Weeks 5-8)

Functions Done Properly

Topics:

- Defining functions: parameters, return types, and void
- Declarations vs definitions, and why prototypes exist
- Pass by value: what the function actually receives
- Designing small functions with one job each
- Header files and splitting code across files
- Reading a function signature like documentation

Projects built:

- Number theory library: gcd, lcm, prime check, and digit sum as clean reusable functions in their own file

Practice: Write eight functions from written specifications, then refactor month 1 projects to use them

Scope, Storage, and the Stack

Topics:

- Local vs global variables, and why globals age badly
- The call stack drawn frame by frame for a real program
- static variables: memory that survives the function
- Automatic storage: why locals vanish when the function returns
- Stack overflow: what it literally means
- Debugging with a stack diagram next to the code

Projects built:
- Stack tracer: a set of nested function calls with a hand-drawn stack diagram matched against actual output

Practice: Predict the output of six scope puzzles before running them, and score your predictions

Recursion

Topics:
- Recursion as functions calling themselves, with the stack diagrams to prove it
- Base case and recursive case: the two-part contract
- Factorial, Fibonacci, and sum of digits, traced fully
- Recursion vs loops: cost, clarity, and when each wins
- Stack depth limits demonstrated by crashing one on purpose
- Reading recursive code without re-tracing every call

Projects built:
- Recursive toolkit: power, GCD, decimal-to-binary, and Tower of Hanoi with move printing

Practice: Convert three loop solutions to recursion and three recursive solutions to loops, and note which direction felt harder

Multi-File Programs and the Build

Topics:
- Organizing a project: headers, source files, and include guards
- Compiling multiple files and linking them together
- A first Makefile: stop retyping compiler commands
- The preprocessor: #include, #define, and conditional compilation
- Naming, formatting, and structure conventions used in real C codebases
- Reading someone else's C file without fear

Projects built:
- Unit converter suite: a menu-driven multi-file program with separate modules for length, weight, and temperature

Practice: Split a provided 300-line single-file program into clean modules with headers and a Makefile

Assessment: Month 2 review: a multi-file program built from a spec, plus an oral walkthrough of one stack diagram

---

PHASE 2: POINTERS AND MEMORY, TAUGHT PROPERLY (MONTHS 3-4, WEEKS 9-16)

The month most courses rush is the month this course is named for: pointers with a memory diagram for every operation, then dynamic memory, the heap, and structs, including the famous bugs made visible.

Month 3: Pointers, One Diagram at a Time (Weeks 9-12)

Addresses and the & Operator

Topics:
- Memory as numbered boxes: the mental model everything rests on
- The & operator: every variable lives somewhere
- Declaring pointers: what int* p actually says
- Dereferencing with *: reading and writing through a pointer
- NULL and why pointers should never dangle uninitialized
- Printing addresses with %p and watching them change between runs

Projects built:
- Memory mapper: a program that prints the address and value of every variable it owns, matched to a hand-drawn diagram

Practice: Ten pointer basics drills, each answered first on paper with a box diagram, then verified in code

Pointers and Functions

Topics:
- Why pass by value cannot swap two numbers
- Passing addresses: functions that modify their arguments
- The classic swap written and traced completely
- Out-parameters: returning more than one result
- scanf finally explained: why it always wanted the &
- Pointer parameters in library functions you already used

Projects built:
- Statistics engine: one function that fills min, max, and mean through pointer parameters from a single pass

Practice: Rewrite five month 2 functions to use pointer parameters, and explain in comments what changed in memory terms

Arrays and Pointer Arithmetic

Topics:
- Arrays in memory: contiguous boxes with one name
- Array names and pointers: the decay rule stated honestly
- Pointer arithmetic: what p+1 really adds
- Walking an array with a pointer instead of an index

- Passing arrays to functions, and why size must travel with them
- Out-of-bounds access: the bug C will not catch for you, demonstrated safely

Projects built:
- Array toolkit: reverse, rotate, and search implemented twice, once with indices and once with pointers

Practice: Trace eight pointer arithmetic snippets on paper, predicting values and addresses before running them

Strings at the Memory Level

Topics:
- Strings as char arrays with a null terminator, no magic
- String literals vs char arrays, and what is writable
- Writing strlen, strcpy, and strcmp by hand before touching string.h
- Buffer overflow demonstrated in a controlled exercise, and why it matters for security
- Safer patterns: bounds checking and fgets over gets
- Arrays of strings and the char* array

Projects built:
- mini_string.h: your own implementation of five standard string functions, tested against the real library

Practice: Solve six string problems using only your own mini_string functions, no string.h allowed

Assessment: Pointer month exam: paper diagrams plus live coding, including one swap variant and one string function from scratch

Month 4: The Heap, Dynamic Memory, and Structs (Weeks 13-16)

malloc and the Heap

Topics:
- Stack vs heap: two kinds of memory with different lifetimes
- malloc and free: asking for memory and giving it back
- sizeof in allocation: the pattern that avoids a whole bug family
- Checking for NULL: allocation can fail
- Memory leaks: creating one on purpose, then finding it
- Ownership thinking: who allocated this, and who must free it

Projects built:
- Runtime array: a program that asks the user how much data is coming, allocates exactly that, and cleans up properly

Practice: Six allocation exercises, each submitted with a one-line ownership note stating where every free happens

Dynamic Arrays and the Famous Bugs

Topics:
- realloc: growing storage as data arrives
- Building a growable array with capacity doubling
- Dangling pointers: use-after-free demonstrated and diagnosed
- Double free and what the runtime does about it
- AddressSanitizer: making the invisible bugs report themselves
- Valgrind on Linux, and what a clean report looks like

Projects built:
- Growable list: a dynamic array module with append, insert, and remove that survives a sanitizer run cleanly

Practice: Fix five provided programs, each hiding one memory bug: a leak, a dangle, a double free, an overflow, and an uninitialized read

Structs

Topics:
- struct: modeling a real thing with named fields
- Declaring, initializing, and copying structs
- Arrays of structs: a table of records in memory
- Pointers to structs and the arrow operator
- typedef for names that read like types
- Nested structs and designing the record before coding it

Projects built:
- Student database, part one: an array-of-structs record system with add, list, search, and update

Practice: Model three real records as structs: a bank account, a library book, and a cricket scorecard entry

Structs Meet the Heap

Topics:
- Allocating structs dynamically
- Structs containing pointers: records with variable-length fields
- Deep copy vs shallow copy, and the bug hiding between them
- Freeing nested allocations in the right order
- Designing a small module: header, implementation, and a test file
- Code review: reading a classmate's memory management critically

Projects built:
- Contact book: a fully dynamic record system where both the list and each name grow to fit, sanitizer-clean

Practice: Extend the contact book with delete and sort, keeping the sanitizer report clean

Assessment: Month 4 practical: build a small dynamic record system from a spec in one sitting, leak-free, and defend the design

---
PHASE 3: C AT WORK: FILES, DATA STRUCTURES, AND THE CAPSTONE (MONTHS 5-6, WEEKS 17-24)

The skills become systems: file handling, linked lists, stacks, queues, and sorting built
from raw structs and malloc, a grounded look at where C runs in industry, and a capstone
project compiled and defended.

  Month 5: File Handling and Data Structures from Scratch (Weeks 17-20)
    File Handling
      Topics:
        - fopen, fclose, and file modes that actually matter
        - Reading lines with fgets and parsing them
        - Writing structured data with fprintf
        - Binary files: fread and fwrite for whole records
        - Error handling: what to do when the file is not there
        - Saving and loading the contact book: persistence at last
      Projects built:
        - Persistent contact book: records saved to disk in both text and binary form, reloaded on
          startup
      Practice: Write a CSV report generator that reads raw marks from one file and writes a
      formatted result sheet to another
    Linked Lists
      Topics:
        - Why arrays are not enough: the case for nodes and links
        - The node struct: data plus a pointer to the next
        - Insertion at head, tail, and middle, each with a diagram first
        - Traversal and search
        - Deletion without losing the rest of the list
        - The pointer-rewiring discipline: draw, then code, always
      Projects built:
        - Singly linked list module: insert, delete, search, reverse, and print, built from scratch and
          sanitizer-clean
      Practice: Solve six classic list problems including middle element, nth from end, and
      cycle detection by hand-tracing first
    Stacks and Queues
      Topics:
        - The stack as a discipline: push, pop, peek
        - Array-backed and list-backed stack implementations
        - Balanced brackets: the interview classic, solved properly
        - Queues: enqueue, dequeue, and the circular buffer trick
        - Where the machine itself uses stacks and queues
        - Choosing a backing structure and defending the choice
      Projects built:
        - Bracket validator and a printer-queue simulator, each built on your own stack and queue
          modules
      Practice: Implement a stack-based postfix expression evaluator and test it against ten
      expressions
    Searching, Sorting, and Function Pointers
      Topics:
        - Linear and binary search, with the sorted-input contract
        - Bubble, selection, and insertion sort, traced and timed
        - Counting comparisons: a first taste of efficiency analysis
        - Function pointers: passing behavior as an argument
        - qsort from the standard library and writing its comparator
        - Sorting an array of structs by any field
      Projects built:
        - Sortable student database: the record system gains sort-by-marks, sort-by-name, and binary
          search, using function pointers
      Practice: Time three sorts on arrays of 1k, 10k, and 100k elements and write four
      sentences on what the numbers show
      Assessment: Data structures lab exam: one list problem and one stack or queue problem
      implemented live from scratch

  Month 6: Systems Context and the Capstone (Weeks 21-24)
    Where C Runs: Bits, Kernels, and Microcontrollers
      Topics:
        - Bitwise operators: and, or, xor, shifts, and masks
        - Flags and registers: why embedded code reads and writes single bits
        - A grounded tour: C in operating systems, databases, firmware, and language runtimes
        - What embedded development actually involves, without the hype
        - Reading a small piece of real open-source C and surviving
        - gdb basics: breakpoints, stepping, and inspecting memory in a debugger
      Projects built:
        - Bit toolkit: set, clear, toggle, and test any bit, plus a permissions system stored in a
          single byte
      Practice: Use gdb to find the planted bug in a provided program without reading its
      source first
    Capstone: Scope and Foundations
      Topics:

- Choosing a capstone: bank record system, inventory manager, text-based game with saves, or a proposal of your own
- Writing the spec: features, data structures, and file format decided before coding
- Project skeleton: modules, headers, and a Makefile from day one
- Building the data layer first
- Estimating honestly what three weeks can hold
- Instructor sign-off on scope before the sprint starts

Projects built:
- Approved capstone spec plus a compiling skeleton with the core data structures in place

Practice: Implement and test your capstone's data module in isolation before any menus or files exist

Capstone: Build and Review

Topics:
- Feature-by-feature building, smallest first
- Keeping the sanitizer clean while the codebase grows
- Instructor code review: memory management and module boundaries
- Debugging your own project with gdb instead of printf sprinkles
- Handling bad input like software that respects its user
- Trimming scope with a straight face when the clock demands it

Projects built:
- Feature-complete capstone: all core features working, file persistence in, sanitizer report clean

Practice: Exchange builds with a classmate, file two bug reports on theirs, and fix the ones filed on yours

Ship and Defend

Topics:
- The final pass: a written test checklist run against every feature
- README and build instructions a stranger could follow
- Publishing the semester's work to GitHub properly
- Presenting a systems project: what it does, then how memory flows through it
- Answering pointed questions about your own pointers
- What comes next: C++, embedded, operating systems, or DSA in depth

Projects built:
- Delivered capstone: compiled, documented, pushed to GitHub, and defended in a live walkthrough

Practice: Write the README, then have someone outside the class build and run your project using only that file

Assessment: Capstone defense: live demo, memory-flow explanation, and Q&A, plus course-completion certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- Limits explorer that maps every basic type on your machine
- Pattern engine and number theory function library
- Recursive toolkit with Tower of Hanoi
- Multi-file unit converter with its own Makefile
- mini_string.h: five standard string functions written from scratch
- Growable dynamic array that passes sanitizer checks
- Fully dynamic contact book with file persistence
- Singly linked list, stack, and queue modules built from raw structs
- Sortable student database using function pointers and qsort
- Bit toolkit with a one-byte permissions system
- Capstone: a complete record system or text game, compiled, documented, and defended

Total projects: 15+ compiled programs and modules, ending with a defended capstone on GitHub

SKILLS THE STUDENT WILL HAVE AT THE END

- The compilation pipeline: preprocessor, compiler, linker, and multi-file builds with Makefiles
- Pointers with total confidence: addresses, dereferencing, pointer arithmetic, and pointer parameters
- Dynamic memory: malloc, realloc, free, ownership discipline, and hunting leaks with sanitizers
- Structs and file handling: record systems that persist to text and binary files
- Data structures from scratch: linked lists, stacks, queues, searching, and sorting
- Systems literacy: bitwise operations, gdb debugging, and reading real-world C

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, writing and compiling code alongside the instructor

Practice: 3-4 hours of problem sets and project work between classes, with paper tracing where it teaches best

Review: Homework reviewed with written feedback; memory diagrams corrected line by line when needed

Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

  Doubt support: WhatsApp doubt support between classes, so a segfault at midnight has somewhere to go
  Progress updates: Regular progress notes covering what was mastered and what needs another pass
  Career guidance: Honest direction on next steps: C++, embedded, operating systems coursework, or DSA, based on what each student showed aptitude for

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

  - Our C++ programming course, which builds directly on C and adds objects, templates, and the STL
  - Data structures and algorithms in depth, for placements and competitive programming
  - Embedded systems and firmware learning paths, where C is the industry default
  - Operating systems and computer architecture coursework, met with real preparation
  - Contributing to open-source C projects with a working command of memory and build tools

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: Should I learn C or C++ first?
  A: If your college teaches a C paper, learn C first; you will be examined on it either way, and C's small size makes memory concepts unavoidable rather than optional. C++ then becomes a comfortable extension: our C++ course assumes exactly the ground this one covers. Students heading straight for application development or game engines can start with C++ instead, and we will tell you honestly in the demo class which fits your situation.

  Q: Is C still worth learning, or is it outdated?
  A: C is old and very much alive. Operating system kernels including Linux and Windows internals, databases like PostgreSQL and SQLite, embedded firmware in cars and appliances, and the reference interpreters of Python and PHP are all C codebases maintained today. Nobody builds websites in C, and we will not pretend otherwise; you learn it because it teaches how computers actually work and because systems, embedded, and university curricula still run on it.

  Q: I have never programmed before. Can I start with C?
  A: Yes, and roughly half of each batch does. The course assumes nothing and moves through fundamentals at a deliberate pace with paper tracing built into the homework. C as a first language is demanding but honest: there is no framework hiding what your code does, so what you learn here transfers to every language you touch afterwards.

  Q: Everyone says pointers are the hard part. How do you teach them?
  A: Slowly, visually, and for a full month. Students who finish month 3 routinely say pointers were explained backwards everywhere else.

  Q: Will this course help with my university C syllabus and exams?
  A: Deliberately, yes. Lab-exam-style live coding is part of the monthly assessments, so the exam format itself gets practiced, not just the content.

  Q: Do I need Linux, or will Windows work?
  A: Windows works fine; we set up GCC and VS Code on whatever machine you have in the first week, and Mac and Linux are equally supported. Linux-specific tools like Valgrind are demonstrated for those who have access, but every required exercise runs on all three platforms using the compiler and sanitizers alone.

  Q: What will I have to show at the end of six months?
  A: A GitHub profile with fifteen or more compiled projects, including your own string library, a dynamic contact book, linked list and stack modules built from scratch, and a capstone record system or text game with file persistence and a README a stranger can build from. Just as importantly, you will be able to defend the memory behavior of your own code out loud, which is exactly what interviews and vivas probe.

  Q: How am I assessed, and is the certificate just for showing up?
  A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final exam covering the whole course. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

  Q: Can I try a class before enrolling?
  A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Bring a

question about pointers if you have one; it is our favorite kind of demo.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/c-programming-course-for-college-beginners
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 30 OF 137: C++ FOR KIDS: FIRST REAL CODE, GAMES AND PUZZLES (AGES 9-12)

==========

Course link: https://learn.modernagecoders.com/courses/cpp-programming-for-kids-course
Subject area: C++ Programming
Designed for: Kids
Who can join: Complete beginner to text coding, ages 9 to 12
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 1-2 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

C++ course for kids aged 9 to 12: live online classes from a first program to loops, functions, vectors and text games, taught slowly and clearly in 6 months.

In one line: A serious language, taught at a child's pace, with a game or puzzle at the end of every lesson.

ABOUT THE PROGRAMME

C++ has a reputation for being hard, and for adults learning alone it can be. For a 9 to 12-year-old with a patient teacher, it is something else: a language that makes the computer's thinking visible. Every variable has a type, every line ends with a semicolon, and the compiler says exactly what it did not understand. Children who learn this way pick up habits of precision that make every later language easier. This 6-month live course is written for that age. It starts with what a program is and what a compiler does, then builds one idea at a time: printing and input, variables and arithmetic, decisions with if and else, loops, functions, lists with vectors, strings and finally grouping data with structs. Each idea arrives inside something a child wants to make, such as a times-table trainer, a number guessing game, a secret code machine, a pet simulator, tic-tac-toe or a maze. Topics that need more maturity, like pointers and memory management, are left for our C++ for Teens course. The final month introduces the kind of short puzzle problems used in programming contests. USACO, the USA Computing Olympiad, accepts C++ solutions and starts every new participant in its Bronze division, so students finish with a clear picture of what that next step looks like if they enjoy it. Classes are live and small enough for the teacher to read every child's code, and every month ends with a short check so parents can see real progress.

WHAT MAKES THIS COURSE DIFFERENT

- Written for ages 9 to 12: one new idea at a time, explained with pictures and everyday examples before any syntax
- A game or puzzle in every lesson, so practice feels like play while the habits stay rigorous
- Compiler errors taught as clues, not failures: children learn to read them and fix their own code
- Pointers and manual memory left for later on purpose, so younger students build confidence before complexity
- A first taste of contest-style problem solving in the final month, with USACO Bronze as a visible next step
- A clear bridge to our C++ for Teens course for students who want to keep going

THE LEARNING PATH, STEP BY STEP

Phase 1: First programs: what code and compilers are, printing, input, variables, arithmetic, decisions and loops
Phase 2: Building blocks: functions, vectors, strings and structs, each inside a small game or tool
Phase 3: Games and puzzles: random numbers, grids and nested loops, contest-style problems and a capstone project

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None required. Children who have tried Scratch or similar often settle in faster, but it is not needed

Computer skills: Can type short words, use a mouse and open a browser
Equipment: A laptop or desktop with a stable internet connection. We start with a free online C++ compiler in the browser, so nothing needs installing at first
Age: 9 to 12. Teens aged 13 and up should take our C++ for Teens course instead
Time commitment: 2 live hours plus 1-2 hours of practice each week

WHO THIS COURSE IS FOR

Scratch graduates: Children who have done block coding and are ready for real typed code
Curious beginners: Children who have never coded and want to start with a language that teaches careful thinking
Puzzle lovers: Kids who enjoy logic puzzles, codes and number games
Future contest coders: Children who might enjoy programming contests later and want the right foundation now
Families planning ahead: Parents who want their child ready for C++ for Teens or computer science at school

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FIRST PROGRAMS (MONTHS 1-2, WEEKS 1-8)

From what a program is to loops that draw patterns and run games. Every concept is shown on paper first, then typed, run and changed.

Month 1: Hello, C++ (Weeks 1-4)
Weeks 1-2: What Code Is and Your First Program
Topics:
- What a program is: instructions a computer follows exactly
- What a compiler does, explained as a translator
- The shape of a C++ program: include, main, braces and semicolons
- Printing words and numbers with cout
- Comments as notes to your future self
- Reading a compiler error and finding the line it points to

Projects built:
- Name banner: a program that prints a decorated banner with the student's name and favourite things

Practice: Write three short programs that print a joke, a poem and a box drawn with symbols

Weeks 3-4: Variables, Input and Arithmetic
Topics:
- Variables as labelled boxes, and the types int, double, char and string
- Reading input with cin
- Plus, minus, times and divide in code
- Why 7 / 2 gives 3 with whole numbers, and what % does
- Updating a variable: score = score + 10 and the shortcut +=
- Choosing clear variable names

Projects built:
- Pocket calculator and age-in-days converter that ask the user for numbers and print answers

Practice: Build a pizza-sharing program that works out slices each and slices left over using / and %
Assessment: Month 1 check: write a short input and arithmetic program from a plain-English description

Month 2: Decisions and Loops (Weeks 5-8)
Weeks 5-6: If, Else and Making Choices
Topics:
- Comparing values with ==, !=, <, >, <= and >=
- if, else if and else
- Combining conditions with and, or and not
- The classic mistake of = instead of ==
- Tracing a program by hand with a table
- Planning decisions with a simple flowchart

Projects built:
- Quiz show: a five-question quiz that keeps score and gives a message based on the result

Practice: Write a rock, paper, scissors judge that decides the winner from two typed choices

Weeks 7-8: Loops That Repeat
Topics:
- while loops and the idea of repeating until something changes
- for loops for counting
- Loop counters, totals and running scores
- Avoiding loops that never end
- Drawing star patterns and triangles with loops
- break and continue, used sparingly

Projects built:
- Times-table trainer and a number guessing game that says higher or lower until the player

wins
Practice: Print the 1 to 12 times tables neatly, then a staircase of stars of any height the user chooses
Assessment: Phase 1 assessment: a loop and decision program built in class, plus a short explanation of how it works

---
PHASE 2: BUILDING BLOCKS (MONTHS 3-4, WEEKS 9-16)

The tools that let programs grow without getting messy: functions, lists, text and grouped data, each taught inside a small game or tool.

Month 3: Functions and Vectors (Weeks 9-12)
Weeks 9-10: Functions
Topics:
- Why we split programs into functions
- Writing a function, calling it and passing values in
- Returning an answer from a function
- void functions that just do something
- Local variables and why they stay inside their function
- Testing a function with a few known answers
Projects built:
- Shape helper: functions that draw squares and triangles and work out area and perimeter
Practice: Rewrite the quiz show from month 2 so each question is one function call
Weeks 11-12: Lists with Vectors
Topics:
- Storing many values in a vector
- Adding items with push_back and reading them by position
- Looping over a vector to total, count and find the biggest
- Positions start at zero, and what happens past the end
- Sorting a list with sort
- Choosing between a single variable and a list
Projects built:
- High score board: enter scores, sort them and print the top five with medals beside them
Practice: Store a week of temperatures and print the warmest day, the coldest day and the average
Assessment: Month 3 check: a function and vector task solved from a written problem

Month 4: Strings and Structs (Weeks 13-16)
Weeks 13-14: Working with Text
Topics:
- Strings as lists of characters
- Length, indexing and looping through a string
- Changing letters to upper or lower case
- Characters as numbers underneath
- Counting vowels, reversing words and checking palindromes
- Reading a full line of text with getline
Projects built:
- Secret code machine: a shift cipher that encodes and decodes messages between friends
Practice: Write a word game that tells the player how many letters their guess shares with a secret word
Weeks 15-16: Grouping Data with Structs
Topics:
- Why related values belong together
- Defining a struct with fields such as name, hunger and happiness
- Creating and changing struct values
- A vector of structs for a class list or a team
- Passing a struct to a function
- Designing data before writing code
Projects built:
- Virtual pet: a pet that gets hungry and bored over time and responds to feeding and play commands
Practice: Extend the pet so the player can look after three pets stored in one vector
Assessment: Phase 2 assessment: a small program that uses functions, a vector and a struct together

---
PHASE 3: GAMES AND PUZZLES (MONTHS 5-6, WEEKS 17-24)

Everything comes together in bigger text games, grid puzzles and the short problem-solving tasks used in programming contests, ending with a capstone each child designs.

Month 5: Bigger Games (Weeks 17-20)
Weeks 17-18: Random Numbers and a Text Adventure
Topics:
- Making random numbers for dice, cards and surprises
- Planning a game as rooms, choices and items
- A game loop that runs until the player wins or quits
- Keeping track of health, gold and inventory

- Splitting a bigger program into many small functions
- Testing every path through the game

Projects built:
- Dungeon adventure: a text adventure with rooms, random encounters, an inventory and a win condition

Practice: Add two new rooms and one new item, then ask someone at home to play it

Weeks 19-20: Grids and Nested Loops

Topics:
- A grid as a vector of vectors
- Nested loops to print and search a grid
- Rows, columns and checking neighbours
- Checking for three in a row
- Moving a player around a maze with letters
- Keeping a grid game fair and bug-free

Projects built:
- Tic-tac-toe for two players and a maze game where the player finds the exit

Practice: Add a move counter to the maze and a replay option to tic-tac-toe

Assessment: Month 5 check: a grid task built from a short written specification

Month 6: Contest Puzzles and the Capstone (Weeks 21-24)

Weeks 21-22: Thinking Like a Problem Solver

Topics:
- Reading a puzzle problem: input, output and the example cases
- Working out the answer by hand before coding
- Patterns that keep coming up: counting, finding the largest, simulating step by step
- Testing with small cases and tricky edge cases
- What USACO is and how its Bronze division works
- Staying calm when the first idea does not work

Projects built:
- Puzzle notebook: eight short contest-style problems solved, each with a sentence on the idea behind it

Practice: Solve two puzzle problems a week and write down one mistake and how you found it

Weeks 23-24: Capstone Project and Showcase

Topics:
- Choosing a project idea that fits two weeks
- Planning the data, the functions and the game loop
- Building in small steps and testing as you go
- Asking for feedback and improving the project
- Presenting the program and explaining one clever part
- Next steps: C++ for Teens, Python or contest practice

Projects built:
- Capstone: an original game or tool planned, built and presented by the student

Practice: Write a short guide that explains how to play or use the capstone project

Assessment: Final assessment: the capstone project, a short code walkthrough and a set of small coding tasks covering the whole course

PROJECTS THE STUDENT BUILDS AND KEEPS

- Quiz show with scoring and feedback
- Number guessing game and times-table trainer
- High score board with sorting
- Secret code machine
- Virtual pet with structs
- Dungeon text adventure
- Tic-tac-toe and maze games
- Original capstone project

Total projects: 8 finished projects plus a notebook of solved puzzle problems

SKILLS THE STUDENT WILL HAVE AT THE END

- C++ program structure, output and input
- Variables, types and arithmetic, including whole-number division and remainders
- Decisions and loops, traced by hand and in code
- Functions, vectors, strings and structs
- Reading compiler errors and debugging independently
- Breaking a problem into steps before coding

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week with plenty of typing, running and fixing code together

Homework: A short task after every class, and a monthly mixed-review task that brings back earlier topics

Practice: 1-2 hours a week of small programs and puzzles

Review: Every program read by the teacher, with simple written notes on what to improve

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, where a parent or the student can send a screenshot of an error
Progress updates: Regular progress notes to parents, with the projects their child finished that month

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Our C++ for Teens course, which adds pointers, object-oriented programming, the STL and data structures
- Contest practice toward USACO Bronze, with a teacher guiding the next set of problems
- Python for Kids or Teens, where the same ideas move much faster
- Game development courses, where loops, functions and data structures are used every day
- School computing clubs and projects, joined with real programming experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is C++ too hard for a 9 or 10-year-old?
A: Not when it is taught at the right pace. Children cope well with C++ when each idea is introduced on its own, shown on paper first, and practised inside a small game. We leave the genuinely hard parts, such as pointers and manual memory management, for our C++ for Teens course, so younger students build confidence before complexity.

Q: Should my child learn Python or C++ first?
A: Both are good first languages. Python gets results faster with less punctuation. C++ asks for more precision, such as types and semicolons, and children who learn it early tend to be careful, systematic coders. If your child enjoys puzzles and detail, or is interested in programming contests later, C++ is a strong choice. The free demo is a good place to see which suits them. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Does my child need to know Scratch first?
A: No. The course starts from what a program is. Children who have used Scratch often recognise loops and conditions quickly, but complete beginners do just as well with a little extra practice in the first month.

Q: What will my child build?
A: A quiz show, a number guessing game, a times-table trainer, a high score board, a secret code machine, a virtual pet, a text adventure, tic-tac-toe, a maze game and an original capstone project, plus a notebook of solved puzzle problems.

Q: What software do we need?
A: Only a laptop or desktop with a browser and a stable internet connection. We begin with a free online C++ compiler so nothing needs installing. Later in the course we can help set up a free compiler on the computer if the family wants one.

Q: Does this prepare my child for USACO?
A: It prepares them to start. USACO accepts C++ solutions and every new participant begins in the Bronze division. The final month introduces the kind of short problems used there and explains how the contest works. Moving up the divisions takes much more practice, which our contest programming courses cover for older students.

Q: How is this different from C++ for Teens?
A: This course is for ages 9 to 12 and covers the foundations slowly and thoroughly, so a child who finishes it is well prepared to join the teen course later.

Q: How are the classes run?
A: Live online classes, twice a week, with a real teacher, in a group of 15 to 20 students, a mini batch of up to 6, or one to one. Current fees for your region are shown in the pricing section of this page.

Q: Can we try a class first?
A: Yes. Book a free demo class. The teacher writes and runs a small C++ program with your child, sees how they think, and suggests whether group, mini batch or one-to-one classes fit best. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/cpp-programming-for-kids-course

For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 31 OF 137: C++ FOR TEENS: FUNDAMENTALS, STL, DSA AND REAL GAMES

==========

Course link: https://learn.modernagecoders.com/courses/cpp-programming-masterclass-for-teens
Subject area: C++ Programming
Designed for: Teens
Who can join: Absolute beginner, ages 13 to 18
Course length: 10-12 months (40-48 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

C++ course for teens aged 13-18: fundamentals, pointers and memory, OOP, STL, data structures, SFML games and competitive programming basics in 10-12 months.

In one line: The language that hides nothing: pointers, memory and speed understood for real, then turned into data structures, games and contest-ready code.

ABOUT THE PROGRAMME

C++ is the language serious computer science still speaks: competitive programming runs on it, game engines are built in it, and the concepts it refuses to hide, memory, pointers, performance, are exactly what make every later language feel easy. This live online course takes students aged 13 to 18 through it honestly: fundamentals learned patiently, then the famous hard parts, pointers, references and dynamic memory, taught with pictures and patience until they click, object-oriented C++, the STL used the way contests and codebases use it, data structures built by hand, a 2D games month with SFML, and a genuine introduction to competitive programming. C++ rewards patience more than any beginner language, and the pace respects that: ten months at two live classes a week plus three to four hours of practice, with two more months in hand. What the course refuses to do is tour everything: OpenGL, embedded boards and compiler internals are named honestly as later specializations, not squeezed in as fake weeks. Homework after every class, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest. Real memory, real speed, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- The hard parts taught properly: pointers and memory get patient weeks with pictures, not one rushed lecture
- STL fluency as a goal: vectors, maps and algorithms used the way contests and real codebases use them
- Data structures built by hand in the language they were invented for
- A real games month: SFML turns C++ skill into something friends can play
- Competitive programming opened honestly, with our DSA course named as the deep road
- Real assessment: weekly homework, monthly mixed reviews, and a final exam that decides the certificate, with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Fundamentals (Months 1-3): syntax to functions, arrays and strings, learned patiently
Phase 2: The hard beautiful parts (Months 4-6): pointers, memory, OOP, file IO and first STL
Phase 3: Data structures in C++ (Months 7-8): linear structures, trees, sorting and STL depth
Phase 4: Games, contests and the capstone (Months 9-10): SFML, competitive programming foundations, modern C++, and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. C++ is taught from the first line, patiently
Computer skills: Comfortable typing and managing files
Equipment: A computer with a stable internet connection; the compiler and every tool used are free
Age: 13 to 18. Batches are grouped by age and pace
Time commitment: 2 live classes plus 3-4 hours of practice a week

WHO THIS COURSE IS FOR

Serious learners: Teens who want the language that hides nothing and rewards patience
Contest bound: Students eyeing competitive programming, where C++ is the standard weapon
School aligned: ICSE/ISC students whose boards examine in C++-adjacent depth
Future engineers: Teens heading for engineering, where C++ fundamentals are the entry gate
Performance curious: Students who ask why some programs are fast and want the real answer

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FUNDAMENTALS (MONTHS 1-3, WEEKS 1-12)

The language learned patiently: syntax, control flow, loops, functions, arrays and strings, each week ending with working programs.

Month 1: First C++ (Weeks 1-4)
Introduction to C++ & Programming
Topics:
- Why C++? The language behind every major game
- C++ history: From 1979 to modern gaming
- Setting up your development environment
- Installing Visual Studio/VS Code/CLion
- Compilers: GCC, Clang, MSVC explained
- Your first C++ program: Hello, World!
- Understanding #include and using namespace
- The main() function: Where programs start
- Compilation process: From code to executable
- cout and cin: Output and input
- Comments: Single-line and multi-line
- Common errors and how to fix them
Projects built:
- Hello World variations
- ASCII art generator
- Personal introduction program
- Simple conversation bot
Practice: Daily: Write and compile 5 small programs
Variables & Data Types
Topics:
- Variables: Storing game scores and player data
- Data types: int, float, double, char, bool
- Declaring and initializing variables
- Constants with const and #define
- Type casting: Converting between types
- Arithmetic operators: +, -, *, /, %
- Compound operators: +=, -=, *=, /=
- Increment/decrement: ++ and --
- Math library functions: sqrt, pow, abs
- Random numbers for games
- String basics: Working with text
- Character manipulation
Projects built:
- Calculator with all operations
- Temperature converter
- Random dice roller for D&D
- Simple RPG stat calculator
Practice: Solve 30 problems on variables and operators
Assessment: Month 1 check: a small program built live, plus a take-home mixing everything so far

Month 2: Control and Repetition (Weeks 5-8)
Control Flow & Decision Making
Topics:
- If statements: Making decisions in code
- Else and else-if chains
- Comparison operators: ==, !=, <, >, <=, >=
- Logical operators: &&, ||, !
- Switch statements: Multiple choice menus
- Ternary operator: Compact conditions
- Nested conditions: Complex decisions
- Game logic with conditions
- Input validation techniques
- Menu-driven programs
- State machines basics
- Debugging conditional logic
Projects built:
- Text adventure game
- Grade calculator with letter grades

- Rock-Paper-Scissors game
- Simple AI decision maker

Practice: Build 10 programs with complex logic

Loops & Repetition

Topics:

- For loops: Repeating code efficiently
- While loops: Conditional repetition
- Do-while loops: Guaranteed execution
- Nested loops: 2D patterns and grids
- Break and continue statements
- Infinite loops and game loops
- Loop optimization techniques
- Pattern printing: Stars, pyramids, diamonds
- Number sequences: Fibonacci, primes
- ASCII art with loops
- Animation basics with loops
- Common loop patterns

Projects built:

- Pattern generator program
- Prime number finder
- Multiplication table generator
- Simple animation loop

Practice: Complete 40 loop-based challenges

Assessment: Month 2 check: a loops-and-logic challenge live, plus mixed review

Month 3: Functions, Arrays and Strings (Weeks 9-12)

Functions - Code Reusability

Topics:

- Functions: Building blocks of programs
- Function declaration and definition
- Parameters and arguments
- Return values and void functions
- Pass by value vs pass by reference
- Function overloading: Same name, different parameters
- Default parameters
- Inline functions for performance
- Recursive functions: Functions calling themselves
- Function prototypes
- Scope and lifetime of variables
- Creating game utility functions

Projects built:

- Math library with custom functions
- Game utility functions toolkit
- Recursive maze solver
- Menu system with functions

Practice: Write 30 different utility functions

Arrays & Strings

Topics:

- Arrays: Storing multiple values
- Declaring and initializing arrays
- Accessing array elements
- Array bounds and safety
- Multi-dimensional arrays: Game boards
- Character arrays and C-strings
- String manipulation functions
- Array algorithms: Search and sort
- Passing arrays to functions
- Dynamic memory preview
- Common array problems
- Game inventory systems

Projects built:

- Tic-Tac-Toe game
- Student grade manager
- Word game (Wordle clone)
- 2D maze game

Practice: Solve 25 array manipulation problems

Assessment: Phase 1 exam: a complete console program built live, plus a written mixed paper

---

PHASE 2: THE HARD BEAUTIFUL PARTS (MONTHS 4-6, WEEKS 13-24)

What makes C++ C++: pointers and references with pictures, dynamic memory without fear, object-oriented design, files, and the first taste of the STL.

Month 4: Pointers and Memory (Weeks 13-16)

Pointers & References

Topics:

- Pointers: The power of C++

- Memory addresses and the & operator
- Pointer declaration and dereferencing
- Null pointers and safety
- Pointer arithmetic
- Arrays and pointers relationship
- References: Safer than pointers
- Const pointers and references
- Function pointers basics
- Common pointer mistakes
- Memory visualization
- Why games need pointers

Projects built:
- Dynamic array implementation
- Memory explorer program
- Pointer-based data swap
- Simple memory game

Practice: Master 20 pointer exercises

Dynamic Memory Management

Topics:
- Stack vs Heap memory
- Dynamic memory with new and delete
- Dynamic arrays
- Memory leaks and how to avoid them
- Delete vs delete[]
- Smart pointers preview
- RAII principle basics
- Common memory errors
- Debugging memory issues
- Valgrind and sanitizers
- Memory-efficient programming
- Game memory management

Projects built:
- Dynamic inventory system
- Resizable array class
- Memory pool allocator
- Dynamic game world

Practice: Build 10 programs with dynamic memory

Assessment: Month 4 check: a pointer-diagram exercise live, memory drawn before code, plus mixed review

Month 5: Objects and Inheritance (Weeks 17-20)

Object-Oriented Basics: Classes and Objects

Topics:
- Introduction to OOP
- Classes and objects
- Member variables and functions
- Access specifiers: public, private, protected
- Constructors and destructors
- This pointer
- Const member functions
- Static members
- Friend functions and classes
- Operator overloading basics
- Class design principles
- UML diagrams basics

Projects built:
- Game character class
- Bank account system
- Student record system
- Simple physics engine

Practice: Design 10 different classes

Inheritance & Polymorphism

Topics:
- Inheritance: Creating class hierarchies
- Base and derived classes
- Protected access specifier
- Types of inheritance: public, private, protected
- Constructor and destructor in inheritance
- Function overriding
- Virtual functions and late binding
- Pure virtual functions
- Abstract classes and interfaces
- Multiple inheritance
- Diamond problem and virtual inheritance
- Polymorphism in game development

Projects built:
- Game entity hierarchy
- Shape drawing system
- Vehicle simulation

```
        - RPG character classes
      Practice: Build 10 inheritance hierarchies
      Assessment: Month 5 check: design a small class hierarchy live and defend it, plus
      mixed review

  Month 6: Files, Errors and First STL (Weeks 21-24)
    File Input/Output
      Topics:
        - File streams: ifstream, ofstream, fstream
        - Opening and closing files
        - Reading from text files
        - Writing to text files
        - File error handling
        - Binary file I/O
        - File positioning and seeking
        - CSV file processing
        - JSON parsing basics
        - Save game systems
        - Configuration files
        - Log file creation
        - Error handling habits: from crash to graceful failure
      Projects built:
        - Save/load game system
        - High score tracker
        - Text file analyzer
        - Simple database file system
      Practice: Build 10 file processing utilities
    STL Basics - Standard Template Library
      Topics:
        - Introduction to STL
        - Vectors: Dynamic arrays made easy
        - Vector operations and methods
        - Strings: Modern C++ strings
        - String manipulation methods
        - Iterators basics
        - Range-based for loops
        - Algorithm library preview
        - Sort and search algorithms
        - STL vs raw arrays
        - When to use STL
        - Performance considerations
      Projects built:
        - Contact manager with vectors
        - Text processor with strings
        - Sorting visualizer
        - Word frequency analyzer
      Practice: Convert array programs to use STL
      Assessment: Phase 2 exam: an OOP + STL program defended live, plus a cumulative written
      paper

----------------------------------------------------------------
PHASE 3: DATA STRUCTURES IN C++ (MONTHS 7-8, WEEKS 25-32)
----------------------------------------------------------------
The serious core, built by hand in the language it was invented for: linear structures, trees
and graphs, sorting, and STL depth.

  Month 7: Structures by Hand (Weeks 25-28)
    Linear Data Structures
      Topics:
        - Linked lists: Single, double, circular
        - Implementing linked list operations
        - Stacks: LIFO principle
        - Stack applications: Undo, parsing
        - Queues: FIFO principle
        - Circular queues and deques
        - Priority queues
        - Implementing with arrays vs nodes
        - STL containers: list, stack, queue
        - Time complexity analysis
        - Space complexity
        - Real-world applications
      Projects built:
        - Custom linked list class
        - Expression evaluator with stack
        - Task scheduler with queues
        - Undo/redo system
      Practice: Implement all data structures from scratch
    Trees & Graphs
      Topics:
```

- Binary trees: Structure and properties
- Tree traversals: Inorder, preorder, postorder
- Binary search trees (BST)
- BST operations: Insert, delete, search
- Balanced trees: AVL basics
- Heaps and priority queues
- Graphs: Representation methods
- Graph traversals: DFS and BFS
- Shortest path algorithms preview
- Tree applications in games
- Scene graphs in game engines
- Decision trees for AI

Projects built:
- BST implementation
- File system tree
- Game AI decision tree
- Pathfinding visualizer

Practice: Solve 20 tree/graph problems

Assessment: Month 7 check: implement a structure live and trace it, plus mixed review

Month 8: Algorithms and STL Depth (Weeks 29-32)

Algorithms - Searching & Sorting

Topics:
- Linear search vs binary search
- Binary search implementation
- Bubble sort: Simple but slow
- Selection sort and insertion sort
- Merge sort: Divide and conquer
- Quick sort: Fast average case
- Heap sort with heaps
- Counting sort and radix sort
- STL sorting algorithms
- Custom comparators
- Sorting visualization
- Algorithm complexity analysis
- Hash maps and sets: the contest workhorses

Projects built:
- Sorting algorithm visualizer
- Game leaderboard system
- Search engine prototype
- Data analysis tool

Practice: Implement all sorting algorithms

STL Containers Deep Dive

Topics:
- Vector internals and performance
- Deque: Double-ended queue
- List vs vector performance
- Set and map internals (red-black trees)
- Unordered containers performance
- Container adaptors
- Custom allocators
- Iterator categories
- Iterator adaptors
- Reverse iterators
- Container selection guide
- Performance benchmarking

Projects built:
- Performance comparison tool
- Custom container library
- Game inventory with STL
- Database with STL

Practice: Master all STL containers

Assessment: Phase 3 exam: solve a timed problem set with the right structures, plus a cumulative paper

---

PHASE 4: GAMES, CONTESTS AND THE CAPSTONE (MONTHS 9-10, WEEKS 33-40)

C++ made visible and competitive: a real SFML game, contest foundations, modern C++ habits, and the capstone exam.

Month 9: SFML and First Contests (Weeks 33-36)

2D Graphics with SFML

Topics:
- Introduction to SFML
- Setting up SFML project
- Creating windows
- Basic shapes and colors
- Sprites and textures

- Text rendering
- Handling events: Keyboard, mouse
- Game loop implementation
- Frame rate and delta time
- 2D transformations
- View and camera
- Sound and music

Projects built:
- Pong game clone
- Snake game
- Particle system
- 2D platformer prototype

Practice: Create 10 mini-games with SFML

Competitive Programming Foundations

Topics:
- Introduction to competitive programming
- Online judges: Codeforces, AtCoder, CodeChef
- USACO preparation
- Reading problem statements
- Input/output optimization
- Time complexity mastery
- Space complexity optimization
- Common patterns
- Debugging strategies
- Contest strategies
- Speed coding techniques
- Template preparation
- Where our dedicated DSA course takes this road further

Projects built:
- Personal template library
- Problem tracker
- Solution archive
- Contest simulator

Practice: Solve 100 easy problems

Assessment: Month 9 check: one timed contest set attempted honestly, plus mixed review

Month 10: Modern C++, Capstone, Demo Day (Weeks 37-40)

Modern C++: Smart Pointers and Clean Habits

Topics:
- Problems with raw pointers
- unique_ptr: Exclusive ownership
- shared_ptr: Shared ownership
- weak_ptr: Breaking cycles
- make_unique and make_shared
- Custom deleters
- RAII in practice
- Auto keyword and type inference
- Range-based loops
- Lambda expressions
- std::function and std::bind
- Modern C++ best practices

Projects built:
- Memory-safe game engine
- Resource manager with smart pointers
- Callback system with lambdas
- Modern C++ game framework

Practice: Refactor old code with modern C++

Capstone Build

Topics:
- Choosing the capstone: an SFML game, a data-structure engine or a console product
- Two focused weeks with milestones and checkpoints
- Memory-clean by design: no leaks, no crashes
- Using everything: OOP, STL, structures, maybe graphics
- Preparing the demo: the hardest bug as the best story

Projects built:
- THE CAPSTONE: a complete C++ project built over two weeks with version control

Practice: A compiling, running build at the end of every session

Demo Day and the Final Exam

Topics:
- Presenting the capstone live
- A full-course spiral review before the exam
- The honest map: OpenGL, embedded and systems programming as later roads
- Where each path leads: our DSA course, ICSE/ISC excellence, or engineering entrances

Projects built:
- Demo day presentation delivered to the batch and parents

Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; a student who falls short gets a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: console programs, calculators, games of logic and text tools
- Phase 2: pointer visualizations, class hierarchies, file-backed applications and first STL builds
- Phase 3: hand-built structures, sorting visualken and timed problem sets
- Phase 4: an SFML game, contest submissions, and the memory-clean capstone at demo day

Total projects: 25+ real C++ builds, crowned by a memory-clean capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- C++ fundamentals to fluency: syntax, control flow, functions, arrays, strings
- The memory model: pointers, references and dynamic allocation, actually understood
- Object-oriented C++: classes, inheritance and polymorphism as design
- STL fluency: containers and algorithms the way contests use them
- Data structures by hand: lists, stacks, queues, trees and hashing
- Applied C++: SFML graphics and competitive programming foundations

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: Homework after every class, and a mixed-review assignment at the end of every month
Practice: 3-4 hours of problem practice between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, for the segfault that makes no sense at 9 pm
Progress updates: Regular progress notes to parents: what was built, what comes next, and where encouragement helps

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Data Structures and Algorithms: the dedicated contest-and-interview road
- Game development: SFML here opens toward engines later
- Engineering entrances and school boards, met with real fundamentals
- Systems roads later: OS, embedded and graphics, named honestly as specializations
- Every other language, suddenly easy: C++ makes the rest feel gentle

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my teen build in this C++ course?
A: More than 25 real builds across ten months: console programs and logic games in the fundamentals, pointer-visualized memory exercises, class hierarchies and file-backed applications, hand-built data structures, timed contest sets, a real 2D game in SFML, and a memory-clean capstone presented at demo day.

Q: Is C++ too hard as a first language for a teenager?
A: It is harder than Python, and that is precisely its value when taught patiently: C++ hides nothing, so a teen who learns it understands memory, types and performance for life, and every later language feels easy. This course paces the famous hard parts, pointers get pictures and patience, not one rushed lecture, and the 2-month margin exists exactly for them.

Q: How long does it take, and how is my teen assessed?
A: Ten months at 2 live classes a week plus 3 to 4 hours of practice, with two more months in hand. Homework follows every class, monthly assignments mix new questions with earlier topics, each phase ends with an exam, and a final exam, practical plus written, decides the certificate with a free retest after revision.

Q: Does this course prepare for competitive programming?
A: It builds the exact base: STL fluency, hand-built structures, algorithmic thinking and a genuine contest-foundations month with timed sets. Serious contest training, dynamic programming, advanced graphs, rating grinds, lives in our dedicated DSA course, which this course names honestly as the next road.

Q: C++ or Python first?
A: Both are excellent and we teach both. Python first suits teens pulled toward AI and quick wins; C++ first suits teens bound for competitive programming, engineering entrances or

school boards that reward its depth, and produces unmatched fundamentals. Nothing learned in either has to be unlearned.

Q: Is there a free trial class?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor reading real code, never pre-recorded videos. In C++ especially, the compiler error means nothing to a beginner and everything to a teacher watching live.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/cpp-programming-masterclass-for-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 32 OF 137: C++ MASTERCLASS: FUNDAMENTALS TO MODERN C++ AND SYSTEMS

==========

Course link: https://learn.modernagecoders.com/courses/cpp-programming-complete-masterclass-college
Subject area: C++ Programming
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

C++ course for college students and professionals: fundamentals, OOP, STL and templates, modern C++, concurrency and systems programming in 10-12 months.

In one line: The language beneath everything fast: fundamentals to modern C++, templates, concurrency and systems programming, in ten honest months.

ABOUT THE PROGRAMME

C++ remains the language of engines, trading systems, embedded firmware and every place performance is non-negotiable, and it rewards exactly one approach: patient depth. This live online masterclass builds it in three real phases: fundamentals and object-oriented C++ learned properly, then the layer that makes C++ modern, the STL used fluently, templates understood rather than feared, smart pointers and RAII as habits, and finally the systems layer: concurrency, memory at depth, and the OS-adjacent programming that makes C++ engineers valuable. A finale month ships a capstone and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of practice, with two more months in hand. Problem sets after every class, monthly mixed reviews, cumulative phase exams, and a final exam that decides the certificate with a free retest. Real memory, real speed, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- The memory model taught with pictures and patience: pointers, RAII and ownership as understanding, not folklore
- Modern C++ as the default: smart pointers, move semantics and the STL the way current codebases use them
- Templates demystified: generic programming as a tool, not a trauma
- A real systems phase: concurrency and OS-adjacent programming, the valuable end of C++
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Fundamentals and OOP (Months 1-3): the language learned properly, from syntax to class design
Phase 2: Modern C++ (Months 4-6): STL fluency, templates, smart pointers and move semantics
Phase 3: Systems and concurrency (Months 7-9): threads, memory at depth, performance and OS

interfaces
Phase 4: The finale (Month 10): capstone spec, build, ship, and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The language is taught from the first line
Equipment: Any computer; the compiler and every tool used are free
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

College students: Engineering students whose curriculum and placements both reward C++ depth
Placement candidates: Anyone whose interviews include C++ and its memory questions
Performance engineers: Developers drawn to the fast end of software
Self taught stuck: Learners who know syntax but fear pointers; this course ends that
Career switchers: Professionals entering software through its most rigorous door

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: C++ FUNDAMENTALS & OBJECT-ORIENTED PROGRAMMING (MONTHS 1-3, WEEKS 1-13)

Build rock-solid foundations in C++ syntax, memory management, pointers, and object-oriented programming principles.

Months 1-2: C++ Basics & Core Concepts (Weeks 1-7)
Introduction to C++ & Development Environment
Topics:
- History and evolution of C++ (C++98 to C++23)
- Why C++? Performance, control, and applications
- Setting up development environment (VS Code, Visual Studio, CLion)
- Compilers: GCC, Clang, MSVC
- Writing your first C++ program
- Program structure: headers, main function, return
- Compilation process: preprocessing, compilation, linking
- Basic input/output with iostream
- Comments and code documentation
- Namespaces and using directive
- C++ coding standards and best practices
- Debugging basics with GDB/LLDB
Projects built:
- Hello World variations
- Simple calculator with basic operations
- Temperature converter
- Personal information display program
Practice: Write 20 simple console programs
Variables, Data Types & Operators
Topics:
- Fundamental data types: int, float, double, char, bool
- Type modifiers: signed, unsigned, short, long
- Variable declaration and initialization
- Constants: const and constexpr
- Auto keyword and type inference
- Arithmetic operators and precedence
- Relational and logical operators
- Bitwise operators and bit manipulation
- Assignment and compound assignment operators
- Increment and decrement operators
- Type casting: static_cast, dynamic_cast, const_cast, reinterpret_cast
- sizeof operator and type sizes
Projects built:
- Advanced calculator with bitwise operations
- Data type size explorer
- Number system converter (binary, decimal, hex)
- Bitwise flag management system
Practice: Solve 50 problems on operators and type manipulation
Control Flow & Functions
Topics:
- If-else statements and nested conditions
- Switch-case statements
- Conditional (ternary) operator
- While, do-while loops
- For loops and range-based for (C++11)
- Break, continue, and goto
- Function declaration and definition
- Function parameters: pass by value, reference, pointer

- Return types and return by reference
- Function overloading
- Default arguments
- Inline functions
- Recursion and recursive algorithms
- Function pointers basics

Projects built:

- Menu-driven banking system
- Number guessing game
- Prime number generator
- Recursive factorial and Fibonacci calculator
- Pattern printing programs

Practice: Implement 30 algorithms using functions

Arrays, Strings & Pointers

Topics:

- Static arrays: declaration and initialization
- Array operations and bounds checking
- Multidimensional arrays
- C-style strings and string operations
- std::string class and string manipulation
- String streams (stringstream)
- Introduction to pointers
- Pointer arithmetic
- Arrays and pointers relationship
- Dynamic memory allocation: new and delete
- Memory leaks and dangling pointers
- Smart pointers introduction (unique_ptr, shared_ptr)
- References vs pointers
- Null pointers and nullptr

Projects built:

- Dynamic array implementation
- String manipulation library
- Matrix operations calculator
- Memory management demonstration
- Text-based adventure game

Practice: Master 40 pointer and array problems

Month 3: Object-Oriented Programming (Weeks 8-13)

Classes and Objects

Topics:

- Classes and objects fundamentals
- Class members: data and functions
- Access specifiers: public, private, protected
- Constructors and destructors
- Constructor types: default, parameterized, copy
- Initializer lists
- This pointer
- Static members and methods
- Friend functions and classes
- Nested classes
- Const member functions
- Mutable keyword
- Rule of Three/Five/Zero
- Move semantics (C++11)

Projects built:

- Bank account management system
- Student record system
- Complex number class
- Date and time class
- Custom string class implementation

Practice: Create 20 different classes modeling real-world objects

Inheritance and Polymorphism

Topics:

- Inheritance concepts and types
- Single, multiple, multilevel inheritance
- Virtual inheritance and diamond problem
- Function overriding
- Virtual functions and vtable
- Pure virtual functions and abstract classes
- Interfaces in C++
- Runtime polymorphism
- Dynamic binding
- Virtual destructors
- Object slicing problem
- RTTI (Run-Time Type Information)
- typeid and type_info
- Final and override specifiers (C++11)

Projects built:

- Shape hierarchy with area calculations

```
        - Employee management system with inheritance
        - Game character class hierarchy
        - Vehicle simulation system
        - Plugin architecture using polymorphism
      Practice: Build 15 inheritance hierarchies
    Operator Overloading & Advanced OOP
      Topics:
        - Operator overloading concepts
        - Overloading arithmetic operators
        - Overloading comparison operators
        - Overloading assignment operator
        - Overloading stream operators (<<, >>)
        - Overloading function call operator
        - Overloading subscript operator []
        - Type conversion operators
        - Explicit keyword
        - Copy and swap idiom
        - CRTP (Curiously Recurring Template Pattern)
        - Design patterns introduction
        - SOLID principles in C++
      Projects built:
        - Mathematical vector class with operators
        - Custom array class with subscript operator
        - Fraction class with all operators
        - Matrix class with operations
        - PHASE 1 CAPSTONE: Complete OOP Library Management System
      Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

----------------------------------------------------------------
PHASE 2: STL, TEMPLATES & MODERN C++ (MONTHS 4-6, WEEKS 14-26)
----------------------------------------------------------------
Master the Standard Template Library, template programming, and modern C++ features.

  Months 4-5: STL Mastery (Weeks 14-21)
    STL Containers
      Topics:
        - STL overview and architecture
        - Sequence containers: vector, deque, list, array
        - Container adapters: stack, queue, priority_queue
        - Associative containers: set, map, multiset, multimap
        - Unordered containers: unordered_set, unordered_map
        - Container operations and time complexity
        - Choosing the right container
        - Custom allocators
        - Container memory management
        - Iterators: types and usage
        - Iterator categories and traits
        - Reverse iterators
        - Iterator adapters
        - Range-based operations
      Projects built:
        - Student database using STL containers
        - Text editor with undo/redo using stack
        - Priority task scheduler
        - Dictionary implementation with map
        - Cache implementation with unordered_map
      Practice: Implement 30 algorithms using different STL containers
    STL Algorithms
      Topics:
        - Algorithm categories in STL
        - Non-modifying sequence operations
        - Modifying sequence operations
        - Sorting algorithms: sort, partial_sort, nth_element
        - Searching algorithms: find, binary_search, lower_bound
        - Numeric algorithms: accumulate, inner_product
        - Set operations: union, intersection, difference
        - Heap operations
        - Permutation algorithms
        - Lambda expressions with algorithms
        - Function objects (functors)
        - Predicate functions
        - Algorithm complexity analysis
        - Custom algorithms implementation
      Projects built:
        - Data analysis tool using STL algorithms
        - Sorting visualizer
        - Set operations calculator
        - Performance comparison of algorithms
```

```
      - Custom algorithm library
    Practice: Use 50 different STL algorithms in various scenarios
  Template Programming
    Topics:
      - Function templates
      - Class templates
      - Template parameters: type and non-type
      - Template specialization
      - Partial specialization
      - Variadic templates (C++11)
      - Template metaprogramming basics
      - SFINAE (Substitution Failure Is Not An Error)
      - Type traits and type manipulation
      - Template template parameters
      - Concepts (C++20)
      - Fold expressions (C++17)
      - constexpr and compile-time computation
      - Template best practices
    Projects built:
      - Generic data structures library
      - Type-safe unit conversion system
      - Compile-time math library
      - Template-based design patterns
      - Custom smart pointer implementation
    Practice: Create 20 template-based utilities
  Modern C++ Features (C++11/14/17/20)
    Topics:
      - Auto and decltype
      - Range-based for loops
      - Lambda expressions and captures
      - Rvalue references and move semantics
      - Perfect forwarding
      - Smart pointers: unique_ptr, shared_ptr, weak_ptr
      - std::thread and basic threading
      - std::chrono for time management
      - Random number generation
      - Regular expressions
      - Filesystem library (C++17)
      - std::optional, std::variant, std::any (C++17)
      - Structured bindings (C++17)
      - Coroutines basics (C++20)
    Projects built:
      - Modern C++ file manager
      - Multi-threaded downloader
      - Regular expression validator
      - Time measurement utilities
      - Resource management system with smart pointers
    Practice: Refactor old code using modern C++ features
  Error Handling & Exception Safety
    Topics:
      - Exception handling mechanism
      - Try, catch, and throw
      - Exception specifications
      - Standard exception classes
      - Custom exception classes
      - Exception safety guarantees
      - RAII (Resource Acquisition Is Initialization)
      - noexcept specifier
      - Error codes vs exceptions
      - std::expected (C++23)
      - Stack unwinding
      - Exception handling best practices
      - Debugging techniques
      - Assertion and static_assert
    Projects built:
      - Robust file processing system
      - Exception-safe container
      - Error handling framework
      - Resource manager with RAII
      - Custom error reporting system
    Practice: Add proper error handling to 10 previous projects

Month 6: Data Structures & Algorithms (Weeks 22-26)
  Advanced Data Structures
    Topics:
      - Linked lists: singly, doubly, circular
      - Stack and queue implementations
      - Binary trees and BST
      - AVL trees and balancing
```

```
    - Red-black trees
    - B-trees and B+ trees
    - Heaps and priority queues
    - Hash tables and collision resolution
    - Graphs: representation and traversal
    - Tries and suffix trees
    - Disjoint set (Union-Find)
    - Segment trees and Fenwick trees
    - Skip lists
    - Custom allocators for data structures
  Projects built:
    - Custom STL-like container library
    - Database index implementation
    - Autocomplete system using trie
    - Graph visualization tool
    - Memory-efficient data structures
  Practice: Implement 20 data structures from scratch
Algorithm Design & Analysis
  Topics:
    - Algorithm complexity analysis
    - Divide and conquer algorithms
    - Dynamic programming techniques
    - Greedy algorithms
    - Backtracking algorithms
    - Graph algorithms: DFS, BFS, Dijkstra, Floyd-Warshall
    - Minimum spanning tree: Kruskal, Prim
    - Network flow algorithms
    - String algorithms: KMP, Rabin-Karp
    - Computational geometry basics
    - Randomized algorithms
    - Parallel algorithms introduction
    - Cache-efficient algorithms
    - Algorithm optimization techniques
  Projects built:
    - Pathfinding visualizer
    - String matching tool
    - Optimization problem solver
    - Algorithm complexity analyzer
    - Competitive programming library
  Practice: Solve 100 algorithmic problems
Performance Optimization
  Topics:
    - Profiling tools: gprof, Valgrind, perf
    - CPU architecture and cache optimization
    - Memory layout and alignment
    - Branch prediction and optimization
    - Loop optimization techniques
    - Vectorization and SIMD instructions
    - Compiler optimizations and flags
    - Link-time optimization
    - Profile-guided optimization
    - Memory pooling and custom allocators
    - Lock-free data structures basics
    - Benchmarking techniques
    - Space-time tradeoffs
    - Platform-specific optimizations
  Projects built:
    - Performance profiler tool
    - Optimized matrix multiplication
    - Memory pool allocator
    - Cache-friendly data structures
    - Benchmark suite for algorithms
  Practice: Optimize 10 programs for maximum performance
Competitive Programming
  Topics:
    - Competitive programming platforms
    - Problem-solving strategies
    - Fast I/O techniques
    - Common problem patterns
    - Number theory for CP
    - Combinatorics and probability
    - Game theory basics
    - Bit manipulation tricks
    - String algorithms for CP
    - Advanced data structures for CP
    - Dynamic programming patterns
    - Graph algorithm variations
    - Contest strategies
    - Time management in contests
```

```
      Projects built:
        - Competitive programming template
        - Problem classifier and trainer
        - Contest performance analyzer
        - Algorithm reference implementation
        - Virtual contest environment
      Practice: Solve 200 competitive programming problems
    Phase 2 Capstone Project
      Topics:
        - Complex system design
        - Algorithm selection and implementation
        - Performance optimization
        - Code organization and documentation
        - Testing and debugging
      Projects built:
        - PHASE 2 CAPSTONE: High-Performance Database Engine
        - Build a simple database with indexing, querying, and transactions
        - Implement B+ tree indexes, query optimizer, and transaction manager
        - Focus on performance and correctness
      Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

---------------------------------------------------------------
PHASE 3: SYSTEM PROGRAMMING & CONCURRENCY (MONTHS 7-9, WEEKS 27-39)
---------------------------------------------------------------
Master system-level programming, multithreading, networking, and low-level development.

  Months 7-8: System Programming (Weeks 27-34)
    Operating System Interface
      Topics:
        - Process and memory model
        - System calls and OS APIs
        - File system operations
        - Process creation and management
        - Inter-process communication (IPC)
        - Shared memory
        - Message queues
        - Pipes and named pipes
        - Signals and signal handling
        - Memory mapping with mmap
        - Dynamic linking and loading
        - Environment variables
        - System information retrieval
        - Platform-specific APIs (POSIX, Win32)
      Projects built:
        - Custom shell implementation
        - Process monitor and manager
        - IPC messaging system
        - File system utilities
        - System information tool
      Practice: Build 10 system-level utilities
    Memory Management Deep Dive
      Topics:
        - Virtual memory concepts
        - Memory allocation strategies
        - Custom memory allocators
        - Memory pools and arenas
        - Garbage collection techniques
        - Reference counting implementation
        - Memory debugging tools
        - Leak detection and prevention
        - Memory-mapped I/O
        - Cache-conscious programming
        - NUMA awareness
        - Memory barriers and ordering
        - Lock-free memory management
        - Memory optimization patterns
      Projects built:
        - Custom malloc implementation
        - Garbage collector prototype
        - Memory leak detector
        - Memory-efficient container library
        - Cache performance analyzer
      Practice: Implement 5 different memory allocation strategies
    Multithreading & Concurrency
      Topics:
        - Thread creation and management
        - Thread synchronization primitives
        - Mutexes and locks
```

```
      - Condition variables
      - Semaphores and barriers
      - Read-write locks
      - Atomic operations and memory ordering
      - Lock-free programming
      - Thread pools and executors
      - Futures and promises
      - Async programming with std::async
      - Parallel algorithms (C++17)
      - Thread-local storage
      - Deadlock prevention and detection
    Projects built:
      - Thread pool implementation
      - Producer-consumer system
      - Parallel sorting algorithm
      - Lock-free queue
      - Concurrent web server
    Practice: Build 15 multithreaded applications
  Network Programming
    Topics:
      - Socket programming basics
      - TCP/IP protocol suite
      - UDP vs TCP programming
      - Client-server architecture
      - Non-blocking I/O
      - I/O multiplexing: select, poll, epoll
      - Asynchronous I/O
      - Protocol design and implementation
      - Serialization and deserialization
      - RPC (Remote Procedure Call)
      - HTTP server implementation
      - WebSocket programming
      - Network security basics
      - Cross-platform networking
    Projects built:
      - Chat server and client
      - HTTP server from scratch
      - File transfer protocol implementation
      - Distributed computing framework
      - Network packet analyzer
    Practice: Create 10 network applications
  Database Systems Programming
    Topics:
      - Database connectivity (ODBC, native APIs)
      - SQL query execution from C++
      - Connection pooling
      - Transaction management
      - Prepared statements
      - ORM implementation basics
      - NoSQL database interfaces
      - Embedded databases (SQLite)
      - Database driver development
      - Query optimization from application
      - Caching strategies
      - Data serialization formats
      - Database performance tuning
      - Migration and versioning
    Projects built:
      - Database connection pool
      - Simple ORM framework
      - Database migration tool
      - Query builder library
      - Cache layer implementation
    Practice: Build 5 database-driven applications

Month 9: Advanced System Topics (Weeks 35-39)
  Compiler Design & Tools
    Topics:
      - Compilation process deep dive
      - Lexical analysis basics
      - Parsing techniques
      - Abstract syntax trees
      - Code generation basics
      - Writing a simple interpreter
      - JIT compilation concepts
      - LLVM basics
      - Build systems: Make, CMake, Bazel
      - Static analysis tools
      - Code generation tools
```

```
    - Debugging at assembly level
    - Profiling and optimization tools
    - Cross-compilation
  Projects built:
    - Simple expression evaluator
    - Mini programming language interpreter
    - Build system for C++ projects
    - Static code analyzer
    - Code generator for repetitive tasks
  Practice: Create 5 development tools
Embedded Systems Programming
  Topics:
    - Embedded C++ considerations
    - Memory-constrained programming
    - Real-time systems concepts
    - Interrupt handling
    - Device driver basics
    - Bare-metal programming
    - RTOS integration
    - Hardware abstraction layers
    - Cross-compilation for embedded
    - Debugging embedded systems
    - Power optimization
    - Communication protocols (I2C, SPI, UART)
    - Sensor integration
    - IoT development basics
  Projects built:
    - Embedded system simulator
    - Device driver prototype
    - Real-time task scheduler
    - Sensor data logger
    - IoT device firmware
  Practice: Develop 5 embedded applications
Security Programming
  Topics:
    - Secure coding practices
    - Buffer overflow prevention
    - Input validation techniques
    - Cryptography basics
    - Hash functions implementation
    - Encryption algorithms
    - SSL/TLS implementation basics
    - Authentication systems
    - Secure random number generation
    - Code obfuscation techniques
    - Reverse engineering prevention
    - Security auditing tools
    - Fuzzing and testing
    - Exploit mitigation techniques
  Projects built:
    - Encryption library
    - Secure communication protocol
    - Password manager
    - Security scanner
    - Secure file storage system
  Practice: Implement 10 security features
Graphics Programming Basics
  Topics:
    - Graphics pipeline overview
    - OpenGL/DirectX basics
    - 2D graphics rendering
    - 3D mathematics for graphics
    - Shaders and GLSL
    - Texture mapping
    - Lighting models
    - Camera systems
    - Scene graphs
    - Collision detection
    - Particle systems
    - GUI programming with Qt/ImGui
    - Vulkan introduction
    - Graphics optimization
  Projects built:
    - 2D graphics renderer
    - 3D scene viewer
    - Particle system simulator
    - Simple GUI framework
    - Graphics effects library
  Practice: Create 5 graphics applications
```

```
    Phase 3 Capstone Project
      Topics:
        - System design and architecture
        - Component integration
        - Performance requirements
        - Testing and validation
        - Documentation and deployment
      Projects built:
        - PHASE 3 CAPSTONE: Distributed Computing System
        - Build a distributed task processing system
        - Include networking, multithreading, and IPC
        - Focus on scalability and fault tolerance
      Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

  ---------------------------------------------------------------
  PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)
  ---------------------------------------------------------------
  The capstone month: spec, build, ship and defend, then the final exam that decides the
  certificate.

    Month 10: Capstone and the Final Exam (Weeks 40-44)
      Capstone Spec and Setup
        Topics:
          - Choosing the capstone: a performance-critical tool, a small engine or simulator, or a systems
            utility
          - A one-page spec: scope, milestones, stack
          - Repository and project skeleton ready on day one
          - Instructor sign-off before the build begins
        Projects built:
          - Approved capstone spec plus the running skeleton
        Practice: Pitch the spec to the batch and absorb one hard question
      Capstone Build
        Topics:
          - Two focused weeks of building in vertical slices
          - Instructor checkpoints and honest scope cuts
          - Using the whole course: modern C++, the STL, templates where they earn it, and concurrency
            where it helps
          - Daily commits and an always-working build
        Projects built:
          - The capstone, feature-complete with version control
        Practice: A working build at the end of every session
      Ship and Polish
        Topics:
          - The finishing pass: edge cases, errors, documentation
          - Sanitizers and warnings clean: memory-safe by evidence
          - A README worth reading and a demo worth watching
          - A full-course spiral review before the exam
        Projects built:
          - The capstone shipped, documented and rehearsed
        Practice: Two mock demos, each tighter than the last
      Demo Day and the Final Exam
        Topics:
          - Presenting the capstone: live demo plus the hardest bug story
          - The honest map of next roads: deep systems programming, graphics and engines, embedded, or
            competitive programming
          - Where our other courses continue the journey
        Projects built:
          - Demo day presentation delivered to the batch
        Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed
        from every phase. Passing earns the certificate; falling short earns a focused revision
        plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Phase 1: console applications and OOP systems built from requirements
  - Phase 2: STL-fluent tools, template utilities and modern-C++ refactors
  - Phase 3: threaded programs, performance investigations and systems utilities
  - Phase 4: the capstone, sanitizer-clean and defended at demo day

  Total projects: 25+ real builds, crowned by a sanitizer-clean capstone

SKILLS THE STUDENT WILL HAVE AT THE END

  - C++ fundamentals to fluency, including the memory model
  - Object-oriented design in the language that invented much of it
  - STL and templates used the way modern codebases use them
  - Modern C++: smart pointers, RAII, move semantics
  - Concurrency: threads, synchronization and honest judgment about both
```

- Performance: profiling before opinions, always

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Data Structures and Algorithms: the contest-and-interview road, in the right language
- Systems specializations: engines, embedded and OS work, entered with real fundamentals
- Our full-stack or Python courses for breadth alongside C++ depth
- Placement interviews met with genuine memory-model understanding

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Growing roles: Rs 5-8 LPA (Junior C++ Developer)
Experienced roles: Rs 8-15 LPA (C++ Developer); Rs 12-25 LPA (Senior C++ Developer); Rs 20-40+ LPA (Lead Developer/Architect)
Freelance: Rs 2,000-8,000/hour based on expertise
International: $80k-180k USD based on location and specialization

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this C++ masterclass cover?
A: Three real phases plus a finale: fundamentals and object-oriented C++ learned patiently; modern C++, STL fluency, templates, smart pointers and move semantics; systems programming, concurrency, memory at depth and performance; then a capstone month ending in the final exam. The famous hard parts get pictures and patience, and the modern idioms are the default throughout.

Q: How long does it take to learn C++ properly?
A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two more months in hand for learners who need them. The three teaching phases build the language and its real-world domain; the finale month ships a capstone and sits the final exam.

Q: How am I assessed? Is the certificate just for attending?
A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Is C++ still worth learning?
A: Wherever performance is non-negotiable, C++ remains the answer: game engines, trading systems, embedded, robotics, browsers and the AI infrastructure layer itself. It is also the best teacher of fundamentals in the industry: engineers fluent in C++ pick up everything else faster, and interviews know it.

Q: Can I try a class before enrolling?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/cpp-programming-complete-masterclass-college
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 33 OF 137: CAMBRIDGE AS AND A LEVEL COMPUTER SCIENCE 9618 COURSE

==========

Course link: https://learn.modernagecoders.com/courses/cambridge-a-level-computer-science-9618-course
Subject area: A Level Computer Science
Designed for: Teens
Who can join: Cambridge AS and A Level students on 9618; an IGCSE Computer Science background helps but is not required
Course length: 9 months (36 weeks) for the full A Level, joinable any month
Time needed: 2 live classes/week + weekly programming and past-paper practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online Cambridge AS and A Level Computer Science 9618 classes across all four papers, including the split most students are unprepared for: Paper 2 does not require program code, and Paper 4 is nothing but.

ABOUT THE PROGRAMME

Cambridge 9618 examines through four papers of 75 marks each. AS candidates sit Papers 1 and 2; A Level candidates sit all four. The structural fact that catches most students is the relationship between Paper 2 and Paper 4. Paper 2, Fundamental Problem-solving and Programming Skills, states that candidates will not be required to write programming code, and provides an Insert with pseudocode built-in functions. Paper 4, Practical, is programming and nothing else, based around a small number of scenarios, with program code and evidence of testing submitted. The same qualification therefore asks a student to reason about programs without writing them, and then to write nothing but programs. Preparing for one does not prepare for the other, and this course treats them as the two separate disciplines they are.

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What a Student Needs to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: AS THEORY FUNDAMENTALS, PAPER 1 (MONTHS 1-3)

Sections 1 to 8 of the syllabus: how data is represented, how the machine works, and the vocabulary Paper 1 marks against.

Month 1: Information Representation and Hardware (Weeks 1-4)
Number Systems and Data Representation
Topics:
- Binary, denary and hexadecimal, converted quickly in both directions
- Two's complement, binary addition and subtraction, and overflow
- Binary coded decimal and where it is genuinely used
- Character sets, and the size calculations that follow

Images, Sound and Compression
Topics:
- Bitmap and vector images, resolution and colour depth
- Sample rate, bit depth and the file-size calculations examiners set
- Lossy and lossless compression with a defensible reason for each
- The exact wording that earns marks in a comparison question

Computer Architecture and the Processor
Topics:
- Von Neumann architecture, registers and buses
- The fetch-decode-execute cycle, sequenced and explained precisely
- Interrupts, and how they change the cycle

        - Assembly language and addressing modes as the syllabus treats them
    Input, Output and Storage
      Topics:
        - Devices, and what each is genuinely suited to
        - Primary, secondary and offline storage compared
        - RAID, and why redundancy is not the same as backup
        - First timed set of Paper 1 style questions

  Month 2: Networks, the Internet and Security (Weeks 5-8)
    Networks and Topologies
      Topics:
        - LAN, WAN, client-server and peer-to-peer
        - Hardware, protocols and the layered model
        - Bit streaming, and the bandwidth calculations that go with it
        - The internet, and what a URL actually resolves to
    Security, Privacy and Data Integrity
      Topics:
        - Threats named precisely, and matched to their mitigations
        - Encryption, symmetric and asymmetric, with the key exchange explained
        - Digital certificates, signatures and SSL as an examined chain
        - Validation and verification, distinguished as the syllabus distinguishes them
    Ethics, Ownership and Legal Context
      Topics:
        - Copyright, licensing and the models compared
        - Professional conduct as the syllabus frames it
        - Structuring an extended answer that reaches the top band
        - Both sides, then a supported judgement
    Databases and Data Modelling
      Topics:
        - Relational databases, keys, normalisation to third normal form
        - Entity relationship modelling, drawn as the paper expects
        - SQL: DDL and DML, written correctly under exam conditions
        - Why normalisation questions reward method over memory

  Month 3: Paper 1 Technique and Full Papers (Weeks 9-12)
    Command Words and Mark Schemes
      Topics:
        - State, describe, explain and evaluate as Cambridge uses them
        - Answering to the mark allocation rather than to the topic
        - Marking your own work against the real scheme
        - Correct ideas that the scheme does not credit, and why
    Boolean Logic and Logic Circuits
      Topics:
        - Gates, expressions, truth tables and simplification
        - Karnaugh maps where the syllabus requires them
        - Drawing circuits with the specified symbols
        - Reliable marks, treated as reliable marks
    Full Paper 1, Marked and Reviewed
      Topics:
        - A complete 75-mark paper under exam conditions
        - Detailed review of every lost mark with the reason named
        - A revision list built from actual errors, not from the topic list
        - Retesting the same question types a fortnight later
    Targeted Repair on Sections 1 to 8
      Topics:
        - Rebuilding the two or three areas the mock exposed
        - Number-system arithmetic drilled back to automatic
        - Extended responses rewritten to the band descriptors
        - Short daily retrieval rather than long revision blocks

---
PHASE 2: PAPER 2, REASONING ABOUT PROGRAMS WITHOUT WRITING THEM (MONTHS 4-6)

Sections 9 to 12. The paper that states candidates will not be required to write programming code, and supplies an Insert of pseudocode built-in functions. A separate discipline from Paper 4, and taught as one.

  Month 4: Algorithm Design and Cambridge Pseudocode (Weeks 13-16)
    Pseudocode as the Language of This Paper
      Topics:
        - Cambridge pseudocode conventions, written exactly as specified
        - Using the Insert of built-in functions rather than inventing names
        - Why a personal style costs marks even when the logic is correct
        - Selection, iteration and assignment in the expected format
    Structure Charts, Flowcharts and Decomposition
      Topics:
        - Breaking a problem into modules the way the paper assesses it
        - Structure charts with parameters shown correctly
        - Flowchart symbols exactly as the syllabus specifies

        - Stepwise refinement, examined directly
    Trace Tables and Predicting Output
      Topics:
        - Completing a trace table for an unfamiliar algorithm
        - Identifying the purpose of an algorithm from its trace
        - Finding the deliberate error the question has planted
        - Writing it down rather than tracing in the head
    Standard Algorithms and Data Structures
      Topics:
        - Linear and binary search, bubble and insertion sort in pseudocode
        - Records, arrays and files as the syllabus treats them
        - Abstract data types: stack, queue and linked list operations
        - Timed Paper 2 questions on each

  Month 5: Programming Concepts Without Code (Weeks 17-20)
    Subroutines, Parameters and Scope
      Topics:
        - Procedures and functions, by value and by reference
        - Local and global scope, a reliable short-answer topic
        - Designing an interface before writing anything
        - Reading a specification and reasoning about it
    File Handling and Exception Handling
      Topics:
        - Serial, sequential, random and direct access files
        - File operations in pseudocode with the correct built-in functions
        - Exception handling as the syllabus describes it
        - Test data: normal, boundary and erroneous
    Software Development Life Cycle
      Topics:
        - Analysis, design, coding, testing and maintenance
        - Development models compared, with a reason for choosing one
        - Program testing strategies and corrective maintenance
        - Extended-response structure on this recurring topic
    Full Paper 2, Marked and Reviewed
      Topics:
        - A complete 75-mark paper under exam conditions
        - Marked against the real scheme rather than against intent
        - Confirming pseudocode is written in the expected conventions
        - Naming the specific habits costing marks

  Month 6: AS Level Consolidation (Weeks 21-24)
    Both AS Papers Together
      Topics:
        - Papers 1 and 2 in the same week, as the series presents them
        - Pacing across two 75-mark papers
        - Identifying which of the two carries the remaining risk
        - Directing the final weeks at the weaker one
    Cross-Topic Questions
      Topics:
        - Questions that span sections, which students meet unprepared
        - Databases plus SQL plus data integrity in one question
        - Networks plus security plus encryption in one question
        - Recognising the shape rather than the topic
    Second Full AS Set
      Topics:
        - Both papers again from a different series
        - Measuring improvement question type by question type
        - Confirming that pacing holds on unfamiliar questions
        - Final list of what still costs marks
    AS Readiness
      Topics:
        - A short specific plan for the final fortnight
        - What is provided in each paper and what is not
        - The first five minutes of each paper
        - Walking in with the format already familiar

PHASE 3: A LEVEL PAPERS 3 AND 4 (MONTHS 7-9)

Sections 13 to 20. Advanced theory, and then the practical paper where program code and
evidence of testing are submitted.

  Month 7: Advanced Theory, Paper 3 (Weeks 25-28)
    Data Representation and Communication, Extended
      Topics:
        - Floating-point representation, normalisation and precision
        - Bit manipulation and the operations the syllabus names
        - Protocols and circuit switching against packet switching
        - Questions that combine representation with arithmetic

Hardware, Virtual Machines and Operating Systems
Topics:
- Processor scheduling, interrupts and virtual memory
- Paging, segmentation and thrashing explained precisely
- Virtual machines, and the trade-offs examiners ask about
- Translation software and the stages of compilation

Security, Artificial Intelligence and Emerging Content
Topics:
- Encryption and protocols at A Level depth
- Artificial intelligence as the syllabus frames it
- Machine learning concepts at the level the paper expects
- The newest content, where past questions are thinnest

Advanced Algorithms and Data Structures
Topics:
- Binary trees, traversal, hash tables and dictionaries
- Recursion, traced as well as written
- Big O reasoning at the level the syllabus requires
- Timed Paper 3 questions on each

Month 8: Paper 4 Practical, Sections 19 to 20 (Weeks 29-32)
From Pseudocode to Working Code
Topics:
- The transition Paper 4 demands and Paper 2 forbids
- Choosing the language and committing to it
- Setting up the environment so the exam format is familiar
- Writing code that runs, from a scenario, from nothing

Object-Oriented Programming
Topics:
- Classes, inheritance, polymorphism and encapsulation in code
- Implementing from a described design without deviating
- Where the practical paper expects object orientation
- Common errors that cost working marks

Files, Data Structures and Recursion in Practice
Topics:
- Implementing stacks, queues and linked lists that actually run
- File handling in the chosen language against the scenario
- Recursion implemented, tested and evidenced
- Programs written to a scenario rather than to a topic

Evidence of Testing
Topics:
- What must be submitted alongside the program code
- Producing test evidence that demonstrates the program works
- Screenshots and output as the paper requires them
- Practising submission, not just programming

Month 9: All Four Papers (Weeks 33-36)
Full Paper 3 Under Time
Topics:
- A complete 75-mark advanced theory paper
- Marked to the real scheme, reviewed mark by mark
- Identifying whether theory or practical is the risk
- A repair list from actual errors

Full Paper 4 Under Time
Topics:
- A complete practical paper with code and test evidence submitted
- Working under time pressure with a compiler and a scenario
- Reviewing what did not run and why
- Repeating the scenario type until it is routine

Second Full Set of Both
Topics:
- Papers 3 and 4 again from a different series
- Measuring improvement question by question
- Confirming the practical workflow holds under time
- The last specific list of what still costs marks

Exam Readiness
Topics:
- A short specific plan for the final fortnight
- What each paper provides and what it does not
- The first five minutes of the practical paper
- Walking in with all four formats already familiar

HOW A TYPICAL WEEK IS SPLIT

Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall

short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- A strong AS or A Level Computer Science grade across all papers taken
- A genuine programming ability, evidenced by Paper 4 rather than claimed
- A foundation for a computer science degree that assumes data structures and recursion
- The pseudocode and design habits that university algorithms courses assume
- A portfolio of working programs if the practical work is committed as it is written

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: How is Cambridge 9618 assessed?
A: By four papers of 75 marks each. Paper 1 is Theory Fundamentals, covering syllabus sections 1 to 8. Paper 2 is Fundamental Problem-solving and Programming Skills, sections 9 to 12. Paper 3 is Advanced Theory, sections 13 to 20. Paper 4 is Practical, covering the practical application of sections 19 to 20.

Q: Is it true Paper 2 does not require writing code?
A: Yes. The syllabus states that candidates will not be required to write programming code in Paper 2, and candidates are provided with an Insert containing pseudocode built-in functions. Paper 4, by contrast, is programming and nothing else, based around a small number of scenarios with program code and evidence of testing submitted. Preparing for one does not prepare for the other.

Q: Which programming language should we choose for Paper 4?
A: The language should be chosen early and then committed to, because Paper 4 rewards fluency under time and switching late wastes the practice already done. We will discuss the options against what the school teaches and what the student is heading toward, and then we stay in that language for the whole practical phase.

Q: My child programs well but loses marks on Paper 2. Why?
A: Almost always because they are writing program code where the paper wanted pseudocode in Cambridge conventions, or inventing function names instead of using the ones supplied in the Insert. It is a presentation problem rather than a knowledge problem, which is good news: it takes about three weeks of deliberate practice to fix.

Q: Can you teach AS only?
A: Yes. Many students take AS in one year and complete the A Level in the next, carrying the AS result forward. The first six months of this course are the AS content, and students entering only for AS simply stop there. Candidates taking all four papers in one series follow the whole nine months.

Q: How long is the course?
A: About thirty-six weeks at two classes a week for the full A Level, joinable in any month. AS alone is roughly the first twenty-four. One to one is paced to the entry date, and for a student joining close to a series we will say honestly what is achievable rather than promising full coverage.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/cambridge-a-level-computer-science-9618-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 34 OF 137: CANVA AND AI DESIGN COURSE (13+): REAL GRAPHIC DESIGN SKILLS

==========

Course link: https://learn.modernagecoders.com/courses/canva-ai-design-masterclass
Subject area: Design and Creativity
Designed for: Kids
Who can join: Beginner friendly, ages 13 to adult
Course length: 6-8 months (26-32 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Canva and AI design course for teens and adults (13+): design principles, social and marketing graphics, Canva Magic Studio and Firefly, video, and a portfolio in 6-8 months.

In one line: Design is a learnable craft, not a talent lottery: principles first, Canva as the instrument, AI as the accelerator, and a portfolio that proves it.

ABOUT THE PROGRAMME

Everyone needs design now, for school, clubs, small businesses and every feed, and Canva made the tools accessible; what most people lack is the craft. This live online course for ages 13 and up teaches real design: hierarchy, spacing, color and typography as principles, practiced through hundreds of Canva builds, posters, presentations, social graphics, carousels and short videos, until taste becomes reflex. The AI layer is taught honestly and age-legally: Canva's own Magic Studio and Adobe Firefly, tools that genuinely permit teens, used to accelerate ideation, generate imagery ethically and automate the boring parts, with clear credit habits and a plain explanation of why some famous art generators remain 18+. The course ends with a portfolio capstone and a final exam that decides the certificate. The pace is honest: six months at two live classes a week plus two to three hours of practice, with two more months in hand when needed. Homework after every class, monthly mixed challenges, and a final exam, portfolio review plus a live design task, with a free retest after revision. Real craft, real portfolio, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Principles before templates: hierarchy, spacing, color and typography taught as craft, so work stops looking template-made
- Age-legal AI only: Canva Magic Studio and Adobe Firefly (13+), with honest credit habits and a plain word on why some generators are 18+
- Hundreds of real builds: posters, decks, social sets, carousels and short videos, critiqued kindly every week
- Brand thinking included: one small real brand built end to end, logo to templates
- An honest 6-month arc with 2 months in hand: nobody is rushed
- Real assessment: weekly homework, monthly mixed challenges, and a final exam that decides the certificate, with a free retest after revision

THE LEARNING PATH, STEP BY STEP

Phase 1: Design foundations (Months 1-2): principles, layout, color and typography, practiced in Canva daily
Phase 2: Social and marketing design (Months 3-4): posts, carousels, decks and a small real brand kit
Phase 3: AI-assisted design (Month 5): Magic Studio and Firefly used well, photo editing, honest credits
Phase 4: Video and the finale (Month 6): short-form video design, the portfolio capstone, and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Experience: None. Design principles and Canva are taught from zero
Equipment: A computer or laptop with a stable internet connection; the free Canva tier covers the whole course
Age: 13 to adult; Canva's own terms require 13+, and accounts for younger teens are set up with a parent
Time commitment: 2 live classes plus 2-3 hours of practice a week

WHO THIS COURSE IS FOR

Students: Teens whose school projects, clubs and fests all need design yesterday
Creators: Young creators who want feeds that look intentional instead of improvised
Small business families: Families with a shop or venture that deserves better graphics
Future designers: Anyone testing whether design is their road, with a portfolio as the answer
Professionals: Adults who keep needing decks and posts and keep dreading them

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: DESIGN FOUNDATIONS (MONTHS 1-2, WEEKS 1-8)

The principles that separate design from decoration: hierarchy, spacing, color and typography, practiced through daily Canva builds.

Months 1-2: Principles into Practice (Weeks 1-8)
Introduction to Design & Canva Setup
Topics:
- What is graphic design? History and evolution
- Design careers and opportunities in 2026
- Traditional vs digital design

- Introduction to Canva: Interface tour
- Canva Free vs Pro features comparison
- Account setup and workspace organization
- Understanding design dimensions and formats
- File types: JPG, PNG, PDF, SVG explained
- Resolution and DPI for print vs digital
- Creating your first Canva design
- Navigating templates and elements
- Basic tools: Text, shapes, lines, frames

Projects built:

- Create personal introduction card
- Design simple social media post
- Make basic business card

Practice: Daily: 30 min Canva exploration, 1 hour design exercises

Design Principles & Composition

Topics:

- The 7 principles of design
- Balance: symmetrical, asymmetrical, radial
- Contrast and emphasis in design
- Repetition and consistency
- Proximity and grouping elements
- Alignment and grid systems
- Visual hierarchy and flow
- Rule of thirds and golden ratio
- White space and minimalism
- Gestalt principles in design
- Creating focal points
- Composition techniques for impact

Projects built:

- Redesign poorly designed flyer
- Create balanced poster design
- Design minimalist quote graphics
- Build visual hierarchy infographic

Practice: Analyze 20 professional designs for principles

Assessment: Month 1 check: redesign a bad poster live, explaining every choice, plus a take-home challenge

Typography & Color Theory

Topics:

- Typography fundamentals: serif vs sans-serif
- Font families and pairing rules
- Font size hierarchy and scaling
- Line height, letter spacing, kerning
- Readability and legibility principles
- Typography for different mediums
- Color theory basics: color wheel
- Primary, secondary, tertiary colors
- Color harmonies: complementary, analogous, triadic
- Color psychology and emotions
- Creating custom color palettes
- Using Canva's color tools effectively

Projects built:

- Typography poster with font hierarchy
- Brand color palette creation
- Mood board with color themes
- Restaurant menu with proper typography

Practice: Create 10 different color palettes and font combinations

Canva Advanced Features & Templates

Topics:

- Mastering Canva Pro features
- Background remover tool mastery
- Magic Resize for multiple formats
- Brand Kit setup and management
- Creating and saving templates
- Using and customizing elements
- Working with grids and frames
- Photo effects and filters
- Transparency and layering
- Animation basics in Canva
- Collaboration features
- Organizing designs with folders

Projects built:

- Complete brand kit setup
- Create reusable template set
- Design animated social media post
- Multi-page presentation design

Practice: Build template library for different use cases

Assessment: Month 2 check: a small design set built live to a brief, with questions reaching back to month 1

PHASE 2: SOCIAL AND MARKETING DESIGN (MONTHS 3-4, WEEKS 9-17)

The formats the world actually runs on: posts, carousels, stories, decks and posters, plus
one small real brand built end to end.

  Months 3-4: Formats and a Real Brand (Weeks 9-17)
    Social Media Design Mastery
      Topics:
        - Platform-specific dimensions and requirements
        - Instagram: Posts, Stories, Reels covers
        - Facebook: Posts, covers, ads
        - LinkedIn: Posts, banners, articles
        - Twitter/X: Posts, headers
        - YouTube: Thumbnails, banners, end screens
        - TikTok: Video covers and graphics
        - Pinterest: Pins and idea pins
        - Consistent visual branding across platforms
        - Creating content calendars
        - Batch creating social media content
        - Engagement-driving design techniques
      Projects built:
        - 30-day social media content calendar
        - Complete Instagram feed design
        - YouTube channel branding package
        - Multi-platform campaign graphics
      Practice: Design 5 posts for each major platform
    Marketing Materials & Print Design
      Topics:
        - Business card design principles
        - Letterhead and stationery design
        - Flyer and brochure layouts
        - Poster design for events
        - Banner and billboard considerations
        - Email newsletter templates
        - Digital ads: display, social, search
        - Landing page graphics
        - Infographic design process
        - Presentation design mastery
        - Print specifications and bleed
        - Preparing files for print
      Projects built:
        - Complete business stationery set
        - Tri-fold brochure design
        - Event poster and flyer set
        - Email marketing template
      Practice: Create marketing materials for fictional company
    Logo Design & Brand Identity
      Topics:
        - Logo design fundamentals
        - Types of logos: wordmark, lettermark, emblem
        - Sketching and ideation process
        - Creating logos in Canva
        - Logo variations and applications
        - Brand identity elements
        - Brand style guide creation
        - Color psychology for brands
        - Typography in branding
        - Brand voice and visual consistency
        - Mockup creation for presentations
        - Protecting and trademarking designs
      Projects built:
        - Design 3 different logo concepts
        - Complete brand identity package
        - Brand style guide document
        - Logo mockup presentations
      Practice: Rebrand 5 existing companies
    Photo Editing & Manipulation
      Topics:
        - Photo editing basics in Canva
        - Adjusting brightness, contrast, saturation
        - Color correction and grading
        - Cropping and composition improvement
        - Removing backgrounds professionally
        - Creating photo collages
        - Double exposure effects
        - Photo filters and effects
        - Retouching basics
        - Working with RAW images

- Batch editing techniques
- Creating photo templates

Projects built:
- Professional headshot editing
- Product photo enhancement
- Creative photo manipulation art
- Before/after portfolio pieces

Practice: Edit 50 photos in different styles

Phase Review and Brand Showcase

Topics:
- Portfolio planning and organization
- Selecting best work
- Creating case studies
- Portfolio presentation techniques
- Online portfolio platforms
- Design process documentation

Projects built:
- Phase 1 Portfolio Website
- 10 best designs showcase
- Design process case study

Assessment: Phase check: present the mini brand kit, plus a mixed challenge spanning months 1-4

PHASE 3: AI-ASSISTED DESIGN (MONTH 5, WEEKS 18-21)

The honest AI layer: Canva Magic Studio and Adobe Firefly used to accelerate real work, with photo editing craft and credit habits.

Month 5: Magic Studio and Firefly (Weeks 18-21)

AI Design Tools, Age-Legal and Honest

Topics:
- AI in design: Overview and possibilities
- Adobe Firefly integration
- Canva AI features: Magic Write, Draw, Edit
- AI writing assistants (13+, parent-consented accounts) for design briefs and copy
- AI image upscaling tools
- AI background removal tools
- Ethical considerations in AI design
- Copyright and AI-generated content
- Canva Magic Studio end to end: Magic Media, Magic Write, Magic Switch
- Adobe Firefly: generation trained on licensed work, and why that matters
- Credit habits: saying what AI did, like a professional
- Why several famous art generators are 18+, and what to use instead at 13+

Projects built:
- Create AI-generated brand imagery
- Design campaign using AI tools
- AI-assisted logo concepts
- Hybrid AI-manual design project

Practice: Generate 100 AI images with different styles

Photo Editing and Compositing

Topics:
- Style references and image prompts
- Aspect ratios and composition control
- Creating consistent characters
- Photorealistic vs artistic styles
- Architectural visualization
- Fashion and textile design
- Blending and remixing images
- Upscaling and variations
- Building a prompt library

Projects built:
- AI-generated product catalog
- Consistent character for storytelling
- Architectural concept designs
- Fashion collection mood board

Assessment: Month 5 check: an AI-assisted design set with honest credits, plus mixed review

---

PHASE 4: VIDEO AND THE FINALE (MONTH 6, WEEKS 22-26)

Short-form video design in Canva, then the portfolio capstone and the final exam that decides the certificate.

Month 6: Motion, Portfolio, Exam (Weeks 22-26)

Short-Form Video Design

Topics:
- Content planning and calendars

- Batch creation strategies
- Content repurposing techniques
- Cross-platform adaptation
- Creating content templates
- Automation tools and plugins
- Content management systems
- File organization for creators
- Cloud storage solutions
- Collaboration tools for teams
- Version control for creatives
- Backup strategies

Projects built:
- 3-month content calendar
- Content template library
- Automated design workflow
- Team collaboration project

Practice: Create week's worth of content in one day

Portfolio Capstone

Topics:
- Choosing the capstone: a complete visual identity or campaign for something real
- The full set: logo, palette, templates, social set, one video
- Critique rounds: kind, specific, actionable
- Assembling the portfolio: curation over collection
- Presenting design decisions out loud

Projects built:
- THE CAPSTONE: a complete design campaign plus the curated course portfolio

Practice: One critique given and one received, properly, every day

Demo Day and the Final Exam

Topics:
- Presenting the capstone and portfolio
- A live design task under friendly time pressure
- A full-course mixed review
- Where this leads: creative coding, motion design, or real commissions for people you know

Projects built:
- Demo day presentation delivered to the batch

Assessment: FINAL EXAM: portfolio review plus a live design task and mixed questions from the whole course. Passing earns the certificate; falling short earns focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: daily principle drills, poster redesigns and typography studies
- Phase 2: social sets, carousels, decks and a mini brand built end to end
- Phase 3: AI-assisted sets with honest credits and real photo compositing
- Phase 4: short-form videos, the capstone campaign and the curated portfolio

Total projects: 100+ critiqued design pieces, curated into a portfolio and capstone campaign

SKILLS THE STUDENT WILL HAVE AT THE END

- Design principles: hierarchy, spacing, color and typography as reflexes
- Canva mastery: from templates escaped to originals built
- Brand thinking: one identity carried across every format
- Age-legal AI design: Magic Studio and Firefly used well, credited honestly
- Short-form video design: motion basics that make feeds stop
- Critique: giving and taking feedback like a working designer

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, designing along with the instructor
Homework: A design piece after every class, and a mixed challenge at the end of every month
Practice: 2-3 hours of building between classes
Review: Every piece critiqued with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Monthly written feedback on craft: what is working, what to push next

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam, a portfolio review plus a live design task. Fall short and there is focused revision and a free retest

WHERE THIS CAN LEAD

- Creative Coding and Animation: where design meets motion and code
- Design for your own projects: clubs, ventures and channels, immediately

  - First commissions for people you know, priced in confidence not desperation
  - Deeper tools later: the principles here transfer to every professional suite
  - Our web development courses: design taste makes better builders

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What will I actually learn and make in this Canva course?
  A: More than 100 critiqued pieces across six months: principle drills and poster redesigns,
  social sets, carousels and presentation decks, a mini brand built end to end, AI-assisted
  sets using Canva Magic Studio and Adobe Firefly with honest credits, short-form videos, and a
  capstone campaign presented at demo day with a curated portfolio.

  Q: What age is this course for?
  A: Ages 13 and up, teens through adults, because Canva's own terms require 13+. Accounts for
  younger teens are set up together with a parent, and batches are grouped so teens learn with
  teens and adults with adults where numbers allow.

  Q: How long does it take to get genuinely good at design?
  A: To a portfolio-and-critique level of craft: six months at 2 live classes a week plus 2 to
  3 hours of practice, with two more months in hand for learners who want longer. Design is a
  rep-based skill; the hundreds of critiqued builds are the actual course.

  Q: How am I assessed? Is the certificate just for attending?
  A: No. A design piece follows every class, every month ends with a mixed challenge that
  revisits earlier skills, and the course ends with a final exam: a portfolio review plus a
  live design task under friendly time pressure. Passing earns the certificate; falling short
  earns focused revision and a free retest.

  Q: Which AI tools are taught, and are they allowed for teens?
  A: Canva's own Magic Studio and Adobe Firefly, both genuinely available at 13+ with consent,
  used for ideation, imagery and automating the boring parts, always with honest crediting.
  Several famous art generators remain 18+ by their own terms; the course says so plainly and
  teaches the age-legal equivalents instead.

  Q: Is Canva enough, or do I need Photoshop?
  A: More importantly, the principles taught here transfer to every tool: a student who owns
  hierarchy, spacing, color and type can pick up any professional suite later in weeks.

  Q: Is there a free trial class?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or
  message us on WhatsApp at +91 91233 66161. You will design something real inside the demo
  itself. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live
  Demo is a guaranteed class today or tomorrow, booked at
  https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor critiquing your actual work, never
  pre-recorded videos. Design improves through critique, and critique needs eyes on your
  specific piece.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/canva-ai-design-masterclass
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 35 OF 137: CBSE CLASS 10 MATHS BOARD PREP: CASE STUDIES & MOCKS
==========

Course link: https://learn.modernagecoders.com/courses/cbse-class-10-maths-board-exam-prep-course
Subject area: School Board Mathematics
Designed for: High school
Who can join: CBSE Class 10 students, Standard (041) and Basic (241)
Course length: 10 months (40 weeks)
Time needed: 2 live classes/week + 3-4 hours practice, rising to timed papers in the mock phase
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one
  live class a week, four a month; one to one outside India two live classes a week, eight a
  month. Each class is one hour.

WHAT THIS COURSE IS

CBSE Class 10 Maths board prep course aligned to the 2026-27 syllabus: all 14 chapters, competency and case-study practice, sample-paper mocks, live classes.

In one line: The whole syllabus taught properly, then the board paper practised until it holds no surprises.

ABOUT THE PROGRAMME

Class 10 is the first year the maths marks travel with the student, and the current board paper rewards more than memorised solutions: roughly half of it is competency-based, case studies, data interpretation and situational problems, alongside about 20 percent MCQs and 30 percent traditional short and long answers. This 10-month live course is built for exactly that paper. Months 1 to 8 teach all 14 NCERT chapters across the seven syllabus units, with the unit weightage kept in view throughout: Algebra carries 20 of the 80 theory marks, Geometry 15, Trigonometry 12, Statistics and Probability 11, Mensuration 10, and Number Systems and Coordinate Geometry 6 each. Months 9 and 10 are pure exam craft: structured revision, CBSE sample papers and previous-year papers under real timing, and error clinics after every mock. Case-study and MCQ practice is not saved for the end. From the first chapter, every topic closes with the question forms the board actually asks, so by the mock phase the format is familiar and only speed and polish remain. We teach both Standard (041) and Basic (241) tracks and help each family make that choice with evidence from the student's own test scores, not guesswork. What we promise is preparation and honest feedback; nobody can promise marks, and we do not.

WHAT MAKES THIS COURSE DIFFERENT

- Aligned to the 2026-27 CBSE syllabus as released, all 14 chapters across the 7 units, with the 80-mark theory weightage steering how class time is spent
- Competency questions from week 1: every chapter closes with case-study, data and MCQ practice in the board's actual style, not as a panic module in January
- Two full months of revision and mocks, with CBSE sample papers and previous-year papers under real 3-hour timing and a marked review of every attempt
- Standard versus Basic decided with data: periodic test scores through the year tell each family which paper suits, and we advise honestly either way
- Proof writing and stepwise presentation taught explicitly, because the board marking scheme awards steps, and students lose easy marks by skipping them
- Live small batches where the teacher sees each student's working, not a video series with a doubt inbox

THE LEARNING PATH, STEP BY STEP

Phase 1: Number Systems and the full Algebra unit: real numbers, polynomials, pairs of linear equations, quadratics and arithmetic progressions, the 26-mark backbone of the paper
Phase 2: Geometry, Coordinate Geometry and Trigonometry: similarity, distance and section formulae, trigonometric ratios and identities, heights and distances, and circle theorems
Phase 3: Mensuration, Statistics and Probability: areas related to circles, combined solids, grouped data measures and classical probability
Phase 4: Revision and the mock cycle: structured re-teaching of weak spots, CBSE sample papers and previous years under timing, error clinics and exam-day strategy

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Class 9 promotion. Weak spots in Class 9 algebra or geometry are caught by the week 1 diagnostic and patched inside the course
Board: Built for CBSE Class 10, both Mathematics Standard (041) and Mathematics Basic (241). Students of other boards are better served by our other maths courses
Equipment: NCERT Class 10 Maths textbook, a geometry box, notebooks, and a device with stable internet for live classes
Timing: Best joined at the start of Class 10; mid-year joiners get a catch-up plan after the diagnostic

WHO THIS COURSE IS FOR

Board year students: CBSE Class 10 students who want the syllabus finished early and the board pattern practised properly, not crammed
Students choosing standard: Students keeping science or maths streams open for Class 11, who need Standard (041) depth and proof fluency
Students considering basic: Students unsure between Standard and Basic, who want the choice made on test evidence rather than fear
Case study strugglers: Students who know the methods but lose marks on the long competency stems and data questions
Presentation losers: Students whose answers are right but whose marks are not, because steps, figures and units go missing

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: NUMBER SYSTEMS AND ALGEBRA (MONTHS 1-3, WEEKS 1-12)

The heaviest unit first. Algebra alone carries 20 of the 80 theory marks, and with Real Numbers this phase covers 26. Every chapter closes with board-style MCQ and case-study practice.

Month 1: Real Numbers and Polynomials (Weeks 1-4)
Weeks 1-2: Real Numbers
Topics:
- Diagnostic paper on Class 9 essentials: identities, linear equations, basic geometry
- The fundamental theorem of arithmetic and prime factorisation
- HCF and LCM by prime factorisation, and the HCF x LCM relationship
- Proving irrationality: root 2, root 3, root 5 by contradiction, written to board standard
- Word problems on HCF and LCM: bells, circular tracks, stacking crates
- Board question forms for this chapter: 1-mark MCQs and 2-3 mark proofs

Projects built:
- Chapter question bank entry: 15 solved board-style questions on real numbers, built and kept by the student

Practice: 25 prime factorisation and HCF-LCM problems plus 3 irrationality proofs written out in full board steps

Weeks 3-4: Polynomials
Topics:
- Zeros of a polynomial read from the graph: where the curve cuts the x-axis
- Geometric meaning of zeros for linear and quadratic polynomials
- Relationship between zeros and coefficients of a quadratic polynomial
- Forming a quadratic polynomial from given zeros
- Verification questions: checking the zero-coefficient relationships
- MCQ traps in this chapter: sign errors and reversed relationships

Projects built:
- Graph-reading sheet: 8 polynomial graphs interpreted for number and sign of zeros

Practice: 30 zeros-and-coefficients problems, 10 of them MCQ format, misses redone with the relationship written out

Assessment: Month 1 test: a 25-mark paper on Real Numbers and Polynomials in board format, marked with the board's stepwise scheme

Month 2: Pair of Linear Equations and Quadratic Equations (Weeks 5-8)
Weeks 5-6: Pair of Linear Equations in Two Variables
Topics:
- Graphical solution: consistent, inconsistent and dependent pairs seen as line pictures
- The ratio test for number of solutions, and why it works
- Substitution method with clean recording of steps
- Elimination method and choosing which variable to remove
- Word problems: ages, two-digit numbers, speeds and boats, fixed plus per-unit costs
- Case-study practice: fare structures and cafeteria pricing, the board's favourite settings

Projects built:
- Word-problem translation drill: 10 situations converted to equation pairs before any solving

Practice: 20 equation pairs solved by the method of your choice plus 6 word problems and 1 full case study, self-timed

Weeks 7-8: Quadratic Equations
Topics:
- Standard form and checking whether an equation is quadratic
- Solving by factorisation: splitting the middle term with speed
- The quadratic formula, derived once and then drilled
- The discriminant and nature of roots: real, equal, or none
- Word problems: consecutive numbers, areas, speed-time, work problems
- Finding unknown coefficients from conditions on the roots

Projects built:
- Chapter question bank entry: 15 solved board-style quadratics including 2 case studies

Practice: 25 quadratics split between factorisation and formula, plus 8 nature-of-roots questions with reasons stated

Assessment: Month 2 test: a 30-mark paper on both equation chapters with MCQ, short answer and one case study

Month 3: Arithmetic Progressions and Algebra Consolidation (Weeks 9-12)
Weeks 9-10: Arithmetic Progressions
Topics:
- Recognising an AP and finding the common difference
- The nth term formula, forwards and in reverse: which term is 78
- Sum of the first n terms, both formula forms and when each helps
- Word problems: instalments, salaries, theatre seating, ladder rungs
- Three unknowns from two conditions: combining the nth term and sum formulas
- AP case studies: savings plans and stacked logs in board style

Projects built:
- AP situations sheet: 8 real-life progressions identified, with first term and difference extracted

Practice: 30 AP problems from nth term to sums, including 5 word problems and 1 case study, misses logged by type

Weeks 11-12: Algebra Unit Under Board Conditions
Topics:
- Mixed revision across all five phase 1 chapters

- Stepwise presentation clinic: what the marking scheme pays for and what it ignores
- Assertion-reason questions: the logic and the common traps
- Case-study workshop: reading the stem once, answering all sub-parts efficiently
- Timed sectional practice: the algebra questions of a sample paper in isolation
- Error log review: each student's top three algebra weak spots named and drilled

Projects built:
- Personal error log formalised: every phase 1 test miss classified as concept, method or presentation

Practice: One timed algebra-only sectional paper plus 10 assertion-reason questions, reviewed against the marking scheme

Assessment: Phase 1 milestone: a 40-mark algebra and number systems paper in full board format, 50 percent competency questions

---

PHASE 2: GEOMETRY, COORDINATE GEOMETRY AND TRIGONOMETRY (MONTHS 4-6, WEEKS 13-24)

The proof-and-figure half of the paper: 33 marks across Geometry, Coordinate Geometry and Trigonometry, where drawing the figure and showing the steps is most of the battle.

Month 4: Triangles and Coordinate Geometry (Weeks 13-16)

Weeks 13-14: Triangles and Similarity

Topics:
- Similar figures and what similarity actually claims
- The basic proportionality theorem and its converse, proved and applied
- Similarity criteria: AA, SSS and SAS, with when to use which
- Finding lengths in nested and overlapping triangle figures
- Writing a geometry proof the board way: given, to prove, construction, proof
- Similarity in the wild: shadows, ramps and scale models as case studies

Projects built:
- Proof portfolio started: BPT and two similarity applications written to full board standard

Practice: 20 similarity problems including 4 proofs written out in the four-part format, figures drawn for every question

Weeks 15-16: Coordinate Geometry

Topics:
- The distance formula, derived from Pythagoras and drilled
- Types of triangles and quadrilaterals verified by distances
- The section formula and midpoint as its special case
- Finding ratios: where the x-axis cuts a line segment
- Collinearity and equidistance problems
- Coordinate case studies: layouts of gardens, flags on a sports field

Projects built:
- Chapter question bank entry: 12 solved coordinate geometry questions across all board formats

Practice: 25 distance and section formula problems with a labelled sketch for each, plus 1 case study

Assessment: Month 4 test: a 25-mark paper on Triangles and Coordinate Geometry with one full proof question

Month 5: Trigonometry and Its Applications (Weeks 17-20)

Weeks 17-18: Introduction to Trigonometry

Topics:
- The six ratios defined in a right triangle, with the naming logic
- Exact values for 0, 30, 45, 60 and 90 degrees, and how to rebuild the table from two triangles
- Finding all ratios when one is given
- The fundamental identity and its two rearrangements
- Proving trigonometric identities: strategy, not trial and error
- MCQ speed work on ratio values and identity recognition

Projects built:
- Identity toolkit card: the identities plus the five standard proof moves, in the student's own words

Practice: 30 ratio evaluations and 8 identity proofs, each proof annotated with which move was used at each step

Weeks 19-20: Heights and Distances

Topics:
- Angles of elevation and depression, and reading them off a clean figure
- The non-negotiable first step: draw and label the right triangle
- Single-triangle problems: towers, kites, ladders
- Two-triangle problems: two observers, moving closer, two levels
- Choosing the ratio that solves in one step instead of three
- Case studies: lighthouses, drones and temple towers in board style

Projects built:
- Figure-first drill set: 10 heights and distances problems where the marked figure is drawn before any calculation

Practice: 18 heights and distances problems including 4 two-triangle setups and 1 case study, every figure labelled

Assessment: Month 5 test: a 25-mark trigonometry paper mixing ratios, identities and applications

Month 6: Circles and the Half-Syllabus Checkpoint (Weeks 21-24)

Weeks 21-22: Circles and Tangents
Topics:
- Tangents and secants: what touching actually means
- The tangent-radius perpendicularity theorem, proved and used
- Equal tangents from an external point, proved and used
- Angle chasing in tangent figures using both theorems
- Lengths in circumscribed quadrilaterals and triangles
- Circle proofs written to the four-part board standard

Projects built:
- Proof portfolio extended: both tangent theorems plus two applications written in full

Practice: 20 circle problems including 3 proofs and 5 angle-chasing figures, figures redrawn not copied

Weeks 23-24: Half-Syllabus Mock and Review
Topics:
- Structured revision across phases 1 and 2: one summary mat per chapter
- Half-syllabus mock in full board format and timing
- Marking workshop: students mark an anonymised script against the scheme
- Error clinic: every mock miss classified and assigned a drill
- Basic versus Standard conversation opens: what the mock evidence suggests for each student
- Presentation audit: figures, steps and units checked across the whole script

Projects built:
- Half-syllabus mock with a written self-review: three strengths, three fixes, one habit to change

Practice: Redo every mock miss untimed with full working, then a 15-question drill on your weakest chapter

Assessment: Phase 2 milestone: half-syllabus board-format mock covering all ten chapters taught so far

---
PHASE 3: MENSURATION, STATISTICS AND PROBABILITY (MONTHS 7-8, WEEKS 25-32)

The final 21 marks of new content: areas related to circles, combined solids, grouped data and probability, all heavy with the data-interpretation forms the competency questions love.

Month 7: Areas Related to Circles and Combined Solids (Weeks 25-28)
Weeks 25-26: Areas Related to Circles
Topics:
- Sector area and arc length, tied back to the fraction of the full circle
- Segment area: sector minus triangle, set up cleanly
- Combination figures: designs, running tracks, flower beds
- Working with standard angles in sectors and segments
- Leaving answers in terms of pi versus computing, and when the paper wants which
- Case studies: clock faces, umbrellas and window designs

Projects built:
- Combination-figure sheet: 8 shaded-region problems with the decomposition drawn before any formula

Practice: 20 sector, segment and shaded-region problems with the decomposition stated in one line each time

Weeks 27-28: Surface Areas and Volumes
Topics:
- Surface areas and volumes of cube, cuboid, cylinder, cone, sphere and hemisphere, consolidated
- Combined solids: capsule, tent, ice-cream cone, toy on a hemisphere
- Which surfaces vanish when solids join: the one idea behind every surface-area question
- Volume problems on combined solids
- Unit discipline: cubic centimetres, litres and the conversions the board expects
- Case studies: storage tanks, medicine capsules and circus tents

Projects built:
- Solid dissection cards: 6 combined solids sketched, split into parts, with the joining surfaces marked

Practice: 18 combined-solid problems, half surface area and half volume, units checked on every answer

Assessment: Month 7 test: a 20-mark mensuration paper including one case study and one MCQ set

Month 8: Statistics and Probability (Weeks 29-32)
Weeks 29-30: Statistics of Grouped Data
Topics:
- Grouped frequency tables: class marks, limits and boundaries
- Mean by the direct method, assumed mean and step deviation, and when each saves time
- Median of grouped data: the cumulative frequency column done carefully
- Mode of grouped data and the modal class
- Reading and interpreting data tables the way competency questions present them
- Choosing between mean, median and mode when a question asks which represents the data

Projects built:
- Data study: the student collects a real dataset from home or school, groups it and computes all three measures

Practice: 12 full grouped-data problems covering all three measures, tables laid out to board standard

    Weeks 31-32: Probability and the Content Finish Line
      Topics:
        - Classical probability: equally likely outcomes and the counting behind them
        - Cards, dice, coins and balls: the standard sample spaces mastered
        - Complementary events and the not-happening shortcut
        - Two-object experiments: two dice, two coins, drawn without replacement kept simple
        - Probability case studies and data-linked questions
        - Full-syllabus map: every chapter now taught, revision plan issued
      Projects built:
        - Chapter question bank completed: the student's own solved bank now covers all 14 chapters
      Practice: 30 probability questions from single events to two-object experiments, plus 1
      case study
      Assessment: Phase 3 milestone: a 25-mark statistics and probability paper, plus the
      full-syllabus revision plan agreed with each student

PHASE 4: REVISION AND THE MOCK CYCLE (MONTHS 9-10, WEEKS 33-40)

No new content, on purpose. Structured revision by unit, then CBSE sample papers and
previous-year papers under real timing, each followed by an error clinic.

  Month 9: Structured Revision (Weeks 33-36)
    Weeks 33-34: Algebra and Number Systems Revisited
      Topics:
        - Rapid re-teach of the five algebra-phase chapters, driven by each student's error log
        - Mixed-chapter problem sets: questions that do not announce their chapter
        - MCQ sprint sessions: 20 questions in 25 minutes with instant review
        - Case-study circuit: four stems across four chapters in one sitting
        - Formula-free fluency check: rebuilding key formulas from scratch
        - One-page revision sheets finalised per chapter
      Projects built:
        - Revision sheet set one: student-made single pages for all five algebra-phase chapters
      Practice: One mixed algebra paper of 30 marks self-timed, plus 20 MCQs, both marked in
      class
    Weeks 35-36: Geometry, Trigonometry and the Data Chapters Revisited
      Topics:
        - Proof rehearsal: BPT, tangent theorems and identity proofs rewritten from memory
        - Heights and distances mixed set, figures first as always
        - Mensuration decomposition drills on unseen combination figures
        - Statistics and probability rapid set with full table discipline
        - The 50 percent competency share: reviewing what the recent board papers actually asked
        - Time budgeting: minutes per section planned for the 3-hour paper
      Projects built:
        - Revision sheet set two: single pages for the remaining nine chapters, completing the personal
          revision pack
      Practice: One mixed geometry-trigonometry paper of 30 marks self-timed, plus a
      15-question data-chapters drill
      Assessment: Month 9 checkpoint: two sectional papers marked against the board scheme,
      error logs updated

  Month 10: The Mock Cycle (Weeks 37-40)
    Weeks 37-38: Sample Papers Under Real Timing
      Topics:
        - The CBSE sample paper attempted in one 3-hour sitting
        - Marking against the official scheme, step by step
        - Error clinic one: concept gaps versus presentation losses separated
        - Previous-year paper attempted under timing
        - Question selection strategy: order of attack, when to move on, returning to parked questions
        - Handling the case-study section efficiently: stem once, sub-parts in order
      Projects built:
        - Mock file opened: each paper filed with its marked script, error classification and one-line
          fix per miss
      Practice: Between classes: one additional sample paper attempted at home under honest
      timing, brought marked to class
    Weeks 39-40: Final Mocks and Exam Craft
      Topics:
        - Two final full mocks under strict conditions, spaced for recovery
        - Error clinic two: closing the last recurring mistakes
        - Presentation final pass: figures, steps, units, boxed answers
        - The last 48 hours: what to revise, what to leave alone, sleep over cramming
        - Exam-day routine: admit card to answer script, walked through calmly
        - Course close: results of the mock series reviewed with each family
      Projects built:
        - Completed mock file: the full series with scores charted, handed to the student as their
          evidence of readiness
      Practice: Light targeted drills only, set individually from each student's final error
      log
      Assessment: Course milestone: final full-syllabus mock in board format, reviewed
      one-on-one, plus certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A personal chapter question bank: 12-15 solved board-style questions per chapter, built across all 14 chapters
- A proof portfolio: every syllabus theorem written to the four-part board standard
- A personal error log maintained from month 1 and used to steer revision
- Word-problem translation drills for both equation chapters
- A real-data statistics study grouped and analysed by the student
- One-page revision sheets for all 14 chapters, student-made
- A half-syllabus mock at month 6 and a full mock series in month 10
- The completed mock file: marked scripts, error classifications and score chart

Total projects: A complete self-built revision system: question bank, proof portfolio, revision sheets, error log and marked mock file

SKILLS THE STUDENT WILL HAVE AT THE END

- Every chapter of the 2026-27 CBSE Class 10 Maths syllabus, taught to Standard (041) depth
- Case-study, MCQ and assertion-reason technique for the competency-heavy board pattern
- Geometry and identity proofs written the way the marking scheme pays
- Grouped-data statistics and classical probability with full table discipline
- Timed paper craft: question selection, time budgeting and clean presentation
- Self-review: marking your own work against a scheme and classifying your own errors

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week; mock-phase sittings run longer for full papers
Practice: 3-4 hours weekly of problem sets and question-bank building, rising with timed papers in months 9-10
Review: Homework marked against board-style stepwise schemes; error logs reviewed with the teacher monthly
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for stuck problems
Progress updates: Monthly progress notes to parents with test scores and the error-log summary, plus a Standard versus Basic recommendation by mid-course

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Class 11 Mathematics in the science or commerce stream, entered with the Class 10 base genuinely solid
- Class 11 Applied Mathematics for students who took the Basic paper route
- Our JEE Foundation and higher maths courses for students aiming at engineering entrances
- Computer science and economics electives, both of which lean on Class 10 algebra and statistics
- Confident handling of any exam built on the competency pattern, which CBSE is extending across subjects

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Should my child take Mathematics Standard (041) or Basic (241)?
A: Both papers cover the same syllabus; Standard asks harder questions. Standard is the safer choice for students keeping science or maths-based streams open, while Basic suits students certain they will not continue maths after Class 10. The rules have also loosened: from the 2025-26 session CBSE allows students who took Basic to move to Standard Mathematics in Class 11, subject to the school being satisfied they can cope. We teach to Standard depth by default and give each family a data-backed recommendation by mid-course, based on the student's own test scores.

Q: What is the current CBSE Class 10 Maths exam pattern?
A: The subject is assessed out of 100: an 80-mark theory paper and 20 marks of school-conducted internal assessment. In recent board papers roughly 50 percent of the theory questions are competency-based, meaning case studies, data interpretation and situational problems, with about 20 percent MCQs and 30 percent traditional short and long answers. The whole course is built around that split, which is why case-study practice starts in week 1

rather than in the revision phase.

Q: What are case-study questions and how do you prepare students for them?
A: A case study gives a short real-world stem, a savings plan, a temple tower, a cafeteria price list, followed by three or four sub-questions drawn from the syllabus. Students rarely lack the maths; they lose marks to slow reading and poor extraction of the given data. We close every chapter with case studies in the board's own style and teach a fixed routine: read the stem once, list the givens, answer sub-parts in order. By the mock phase the format is routine.

Q: Is the NCERT textbook enough for the board exam?
A: NCERT is the core, and every board question is rooted in it, so we work through it completely, including the examples students usually skip. For Standard-paper depth we add NCERT Exemplar problems, CBSE sample papers and previous-year questions, because the competency questions dress NCERT ideas in unfamiliar settings and a student who has only seen textbook phrasing can freeze. No private publisher guides are required.

Q: How does the course handle the syllabus itself? Has anything changed?
A: We teach the 2026-27 syllabus as CBSE released it: 14 NCERT chapters across seven units, with the 80 theory marks weighted 20 to Algebra, 15 to Geometry, 12 to Trigonometry, 11 to Statistics and Probability, 10 to Mensuration, and 6 each to Number Systems and Coordinate Geometry. Constructions remains out of the syllabus, as it has been in recent sessions, and no new chapter deletions were made for 2026-27. If CBSE issues a mid-year circular, the course plan is adjusted the same week.

Q: What about the internal assessment marks? Do you help with those?
A: The 20 internal marks are awarded by the school: 10 for periodic tests (schools usually count the best scores through the year), 5 for the portfolio of notebooks and assignments, and 5 for maths lab activities with viva. We cannot award those marks, but the course feeds them directly: our monthly tests mirror periodic-test formats, and the question bank, proof portfolio and revision sheets the student builds all belong in a portfolio a school is happy to see.

Q: I have heard Class 10 boards are now held twice a year. How does the course schedule work with that?
A: Yes, CBSE has moved Class 10 to two board sittings from 2026, with the main exam in February and an optional second sitting in May for improvement. Our 10-month plan targets the February main exam: content finishes by month 8, and months 9 and 10 are revision and full mock cycles timed to peak just before the paper. A student who chooses the May sitting to improve keeps their revision system, mock file and error log, which is most of what a second attempt needs.

Q: Can my child join mid-year, and do you promise a particular score?
A: Mid-year joining works until about month 5; the week 1 diagnostic tells us what has been missed and the catch-up plan covers it through extra problem sets and doubt sessions. On scores we are direct: nobody can honestly promise board marks, and we do not. What we commit to is full syllabus coverage, board-pattern practice from the first chapter, a marked mock series, and honest monthly reporting so you always know exactly where your child stands.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/cbse-class-10-maths-board-exam-prep-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 36 OF 137: CBSE COMPUTATIONAL THINKING & AI FOR KIDS (CLASS 3-8)
==========

Course link: https://learn.modernagecoders.com/courses/cbse-computational-thinking-and-ai-course-for-kids-classes-3-to-8
Subject area: CBSE School Curriculum - Computational Thinking & Artificial Intelligence
Designed for: Kids
Who can join: Absolute Beginner (Class 3-5) to Intermediate AI Project Builder (Class 6-8)
Course length: 9-12 months (40-52 weeks)
Time needed: 5-7 hours/week (2 live classes + self-practice)
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

CBSE-aligned Computational Thinking & AI course for kids in Classes 3-8 (ages 8-13): decomposition, algorithms, Scratch, Teachable Machine and 50+ projects.

In one line: From Curiosity to Creating AI, The Future-Ready Child Starts Here!

ABOUT THE PROGRAMME

The Central Board of Secondary Education (CBSE) has officially integrated Computational Thinking and Artificial Intelligence into the foundational and middle school curriculum as a core 21st-century skill. This comprehensive 9-month program is specifically designed to give your child a deep, hands-on mastery of both Computational Thinking and AI, completely aligned with CBSE's framework, NCERT's AI handbooks, and the National Education Policy (NEP) 2020. Your child will learn to think like a scientist, reason like a mathematician, solve like an engineer, and create like an AI researcher. Starting with unplugged thinking activities (no screens!), we gradually introduce Scratch block coding, AI concepts through stories and games, Google's Teachable Machine, the official CBSE AI Project Cycle (Problem Scoping with 4Ws Data Acquisition Data Exploration Modelling Evaluation), AI Ethics, Sustainable Development Goals (SDGs), and culminate in real AI projects your child will proudly showcase. By the end, your child will have built 50+ projects, understood how Netflix, Alexa, YouTube and self-driving cars really work, and will be fully prepared for CBSE AI Class 9 (Code 417) and later AI Class 11-12 (Code 843).

WHAT MAKES THIS COURSE DIFFERENT

- 100% aligned with the official CBSE Computational Thinking & AI framework
- Separate, age-appropriate tracks for Classes 3-5 and Classes 6-8
- Follows the CBSE AI Project Cycle methodology used in board curriculum
- Mix of unplugged (no-screen) and plugged (digital) activities, healthy screen time
- Uses exact tools CBSE recommends: Scratch, Teachable Machine, Code.org, MIT App Inventor, Python (for Class 8)
- Direct preparation for CBSE AI Class 9-10 (Code 417) & Class 11-12 (Code 843)
- AI Ethics, Bias, Privacy & SDG integration from day one
- Live expert-led classes + parent progress reports + project showcase days
- 50+ original AI & CT projects for school portfolio and competitions
- Compatible with Atal Tinkering Lab (ATL) and CBSE AI School projects

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Unplugged Computational Thinking, Scratch Basics, What is AI? (Classes 3-5 level)
Phase 2: Building (Months 4-6): Advanced Scratch, AI Domains (CV/NLP/Data), Teachable Machine, AI Project Cycle (Classes 6-8 level)
Phase 3: Creating (Months 7-8): Real AI Projects, Python intro, Ethics & SDG AI Solutions, Mini-hackathon
Phase 4: Mastery (Month 9): CBSE-style AI Capstone Project, Portfolio, Certification & Transition to CBSE AI Class 9 (417)

WHAT A STUDENT NEEDS BEFORE STARTING

Age: 8-13 years (Classes 3 to 8)
Coding experience: None, absolute beginners welcome
Equipment: Laptop/desktop with stable internet (4 Mbps+), webcam, headphones
Software: All free, Scratch, Python, Teachable Machine, Orange (we guide installation)
Time commitment: 5-7 hours per week (2 live classes + self-practice)
English: Basic reading ability (Class 3+ level)
Parent support: Minimum 30 mins/week to review progress
Motivation: Curiosity, willingness to make mistakes, excitement to build

WHO THIS COURSE IS FOR

Primary students: CBSE students in Classes 3-8 looking to master Computational Thinking and AI
Other boards: ICSE, IB, State Board and International students, curriculum is fully transferable
Homeschoolers: Perfect structured AI curriculum for homeschool families
Gifted children: Advanced learners ready to go beyond school curriculum
Competition aspirants: Kids preparing for AI Olympiads, CBSE Hackathons, Intel AI Youth challenges
Cbse school students: Students whose schools already teach CBSE AI curriculum and want deeper mastery
Parents of future engineers: Parents preparing kids for future AI-driven careers
Anyone: Any 8-13 year-old curious about how AI, Alexa, ChatGPT and self-driving cars work

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: COMPUTATIONAL THINKING FOUNDATIONS (MONTHS 1-3, WEEKS 1-13), ALIGNED TO CBSE CLASSES 3-5

Build the four pillars of Computational Thinking, Decomposition, Pattern Recognition,
Abstraction and Algorithms, through unplugged activities, games, storytelling and block
coding. No prior coding or computer experience required. Perfect for CBSE Classes 3, 4 and 5
students.

  Month 1: The Four Pillars of Computational Thinking (Unplugged + Intro to Scratch) (Week 1-4)
    Introduction to Computational Thinking & Why It Matters
      Topics:
        - What is Computational Thinking? (Definition as per CBSE/NCERT framework)
        - How do computers 'think'? (Simple explanation for kids)
        - Difference between a human brain and a computer
        - The 4 pillars of CT: Decomposition, Pattern Recognition, Abstraction, Algorithms
        - Real-life CT: Brushing teeth, making sandwich, getting ready for school
        - CT all around us: Traffic signals, recipes, board games
        - Growth mindset & making mistakes, the scientist way
        - Introduction to the CBSE CT framework (age-appropriate)
        - Thinking journal: Your child's first CT diary
        - Family CT challenge: Find 5 algorithms at home
      Projects built:
        - Create a 'How I Get Ready for School' step-by-step poster
        - Break down making Maggi into 10 precise steps (Decomposition)
        - CT Thinking Diary, daily entries for a week
      Practice: Daily 10-minute unplugged CT game from our parent pack
    Decomposition, Breaking Big Problems Into Small Ones
      Topics:
        - What is Decomposition?
        - Why decomposition matters (examples: school projects, cleaning room)
        - Decomposing a drawing into shapes
        - Decomposing a birthday party plan
        - Tree diagrams and mind maps for decomposition
        - Decomposing a school timetable
        - Decomposing a game into rules
        - Practice: Decompose 10 everyday tasks
        - When decomposition helps (and when it doesn't)
        - Team decomposition: Family project breakdown
      Projects built:
        - Decompose your favorite board game into its smallest rules
        - Plan a pretend birthday party using only decomposition
        - Create a decomposition poster for any Class subject
      Practice: Decompose one new task every day, parent signs logbook
    Pattern Recognition, The Secret Skill of Scientists
      Topics:
        - What are patterns? (Visual, numerical, behavioral, musical)
        - Patterns in nature: leaves, flowers, honeycomb, zebra stripes
        - Patterns in math tables (2, 5, 10 tables)
        - Patterns in Indian festivals & seasons
        - Spotting patterns in stories & rhymes
        - Patterns in our daily routine
        - Why AI uses pattern recognition (face unlock, Netflix recommendations)
        - Pattern puzzles and Sudoku for kids (4x4)
        - Creating your own patterns
        - Pattern break: Finding the 'odd one out'
      Projects built:
        - Pattern hunt journal, photograph 20 patterns around you
        - Design a Rangoli using mathematical patterns
        - Create a pattern-based memory game
      Practice: Spot 5 new patterns daily
    Abstraction & Algorithms, Recipes for Computers
      Topics:
        - What is Abstraction? (Keeping important, ignoring rest)
        - Maps as abstractions (Metro map, school map)
        - What is an Algorithm? Step-by-step instructions
        - Recipes as algorithms
        - Algorithms for brushing teeth, tying shoelaces
        - Sequence matters, why order is important
        - Introduction to Flowcharts (shapes and arrows)
        - First flowchart: Morning routine
        - Writing your first algorithm on paper
        - Robot Game: Human Robot, algorithms in action
      Projects built:
        - Write the 'Perfect Algorithm' for making your favorite food
        - Design a flowchart adventure story
        - Program your family to be 'robots' and follow your algorithm
      Practice: Write one algorithm per day for any daily task

Month 2: First Coding with Scratch, Where CT Meets Computers (Week 5-8)
  Welcome to Scratch, The Block Coding Playground
    Topics:
      - Introduction to Scratch (MIT), why CBSE recommends it
      - Scratch interface tour: Stage, Sprites, Blocks, Scripts
      - Motion blocks: Move, turn, glide
      - Your first Scratch program: Make the cat dance
      - Saving and managing Scratch projects
      - Scratch account creation (parent-supervised)
      - Connecting CT to Scratch: Every program = an algorithm
      - Changing costumes & backgrounds
      - Sound blocks, making your sprite speak in Hindi/English
      - First creative project: Digital greeting card
    Projects built:
      - Animated birthday card for a family member
      - Dancing cat in 5 different backgrounds
      - Digital storybook cover with your name
    Practice: Explore one new Scratch block daily
  Events, Loops & Repeat, Making Programs Powerful
    Topics:
      - Events in Scratch: 'When flag clicked', key press, sprite click
      - Loops: Repeat, forever (why loops save time)
      - Real-life loops: brushing teeth 20 times, school bell rings 6 times
      - Nested loops for patterns
      - Pen tool, drawing geometric shapes with code
      - Drawing squares, triangles, hexagons with loops
      - Spiral drawings with loops
      - Variables introduction, boxes that hold information
      - Score counter in a simple game
      - Name input using 'Ask' block
    Projects built:
      - Geometric art generator (shapes with pen + loops)
      - Simple counting game
      - Animated alphabet learning app
    Practice: Create one new looping animation daily
  Decisions, Conditions & Interactive Stories
    Topics:
      - If-then blocks, decisions in code
      - If-else blocks, choosing between two options
      - Comparison: Less than, greater than, equal to
      - Interactive stories with choices
      - Keyboard-controlled sprites
      - Sprite-to-sprite interaction (touching, bouncing)
      - Adding sound effects for actions
      - Timer and countdown
      - Changing background on condition
      - Branching storylines
    Projects built:
      - 'Choose Your Adventure' interactive story
      - Keyboard-controlled character runner
      - Quiz game with If-Else logic (Science or GK quiz)
    Practice: Add If-Else logic to previous projects
  First Mini Capstone: Build Your Own Scratch Game
    Topics:
      - Game design basics for kids
      - Planning: storyboarding before coding
      - Combining motion, events, loops, conditions
      - Scoring systems
      - Lives and Game Over screens
      - Winning and losing conditions
      - Play-testing & debugging
      - Sharing with friends and family
      - Publishing to Scratch studio
      - Presenting your project
    Projects built:
      - CAPSTONE 1: Complete Scratch game (Catch the Stars, Maze Runner, or Pong)
      - Record a project showcase video (2 mins)
      - Write a simple README for your game
    Assessment: Phase 1 Mid-Milestone: Scratch Project Certification

Month 3: What is AI?, Your First Encounter with Artificial Intelligence (Week 9-13)
  AI All Around Us, Seeing AI Everywhere
    Topics:
      - What is Intelligence? What makes humans smart?
      - What is Artificial Intelligence? (Simple CBSE-aligned definition)
      - AI vs ordinary computer programs, the key difference
      - AI in daily life: Alexa, Siri, YouTube, Netflix, Google Maps, face unlock
      - History of AI: From chess computers to ChatGPT (kid-friendly timeline)
      - The 3 ways AI learns: Supervised, Unsupervised, Reinforcement (simplified)

- AI vs Robots, are they the same?
- Famous AI success stories (AlphaGo, Watson, self-driving cars)
- Things AI is good at (and not good at)
- Where does AI go wrong? (Introduction to bias)

Projects built:

- AI Spotter Journal, find 30 examples of AI in your home & neighborhood
- Draw an 'AI Timeline' poster
- Interview a parent/grandparent: 'What was life before AI?'

Practice: Identify AI vs non-AI apps on family phone

How Does AI 'See', Introduction to Computer Vision

Topics:

- How do humans see? (Eyes + brain)
- How do computers 'see'? (Cameras + AI)
- Pixels, colors, and images for computers
- Introduction to Computer Vision (CV domain of AI)
- Face detection vs face recognition
- Object detection (cars, traffic signs, animals)
- QR codes, a simple CV example
- Medical imaging, how AI detects diseases
- Ethics: Is face recognition always good?
- Fun hands-on: Training a simple image classifier with Teachable Machine

Projects built:

- Train a Teachable Machine model to recognize 'happy face vs sad face'
- Train a model to classify 3 objects you own
- Poster: '10 places where Computer Vision is used'

Practice: Spot 5 Computer Vision apps daily

How Does AI 'Listen & Talk', Natural Language & Speech

Topics:

- How computers understand language (Natural Language Processing intro)
- Speech-to-Text: How Alexa & Google understand you
- Text-to-Speech: How computers talk back
- Chatbots, kid-friendly intro
- Simple rule-based chatbot in Scratch
- Language translation with AI
- Voice assistants: Alexa, Google Home, Siri
- How AI learns Hindi, Tamil, English, Bengali and other languages
- Spam detection in email, a NLP example
- Ethics: When should you trust a chatbot?

Projects built:

- Build a rule-based chatbot in Scratch (answers 10 common questions)
- Record voice commands and explore Google Assistant
- Design a friendly AI assistant for elderly people

Practice: Talk to voice assistants and note what works, what doesn't

Introduction to the CBSE AI Project Cycle (4Ws & Beyond)

Topics:

- What is the AI Project Cycle? (Official CBSE framework)
- The 5 stages: Problem Scoping Data Acquisition Data Exploration Modelling Evaluation
- Stage 1: Problem Scoping using 4Ws (Who, What, Where, Why)
- Practice: Apply 4Ws to a school problem
- Stage 2: Data Acquisition, what is data?
- Types of data: Images, text, numbers, sound
- Stage 3: Data Exploration, looking for patterns
- Graphs and charts for young kids
- Stage 4 preview: What is a 'model'?
- Stage 5 preview: How do we know AI is right?

Projects built:

- Apply the 4Ws to 'Why do kids waste food in school canteen?'
- Collect data on weather for 7 days and draw a chart
- Create an AI Project Cycle poster for classroom

Practice: Use 4Ws on one problem every day

Phase 1 Capstone, Your First AI-Assisted Project

Topics:

- Choosing a community problem
- Applying the full AI Project Cycle
- Combining Scratch + Teachable Machine
- Documentation & presentation
- Demonstrating to parents and peers
- Getting feedback and improving
- Celebrating first success
- Preparing for Phase 2

Projects built:

- CAPSTONE 2: AI-powered Scratch project solving a home problem (e.g. lost item finder, pet mood detector, uniform checker)
- 3-slide presentation
- 2-minute demo video

Assessment: CBSE CT & AI Foundation Certificate (Classes 3-5 level)

PHASE 2: AI BUILDER TRACK (MONTHS 4-6, WEEKS 14-26), ALIGNED TO CBSE CLASSES 6-8

Dive deep into Artificial Intelligence with the official CBSE Classes 6-8 AI curriculum.
Master the full AI Project Cycle, explore all three domains of AI (Data, Computer Vision,
Natural Language Processing), begin Python programming, and build real AI-powered
applications that solve real-world problems.

- Month 4: Mastering the CBSE AI Project Cycle (Week 14-17)
  - Problem Scoping & Data Acquisition (Deep Dive)
    - Topics:
      - Problem Scoping, 4Ws in detail (Who, What, Where, Why)
      - Stakeholder identification
      - Framing the problem statement
      - SDG (Sustainable Development Goals) connection, mandated by CBSE
      - Selecting SDG goals for your AI project
      - Data types: Structured (tables), Unstructured (images, text, audio)
      - Sources of data: Surveys, sensors, public datasets (Kaggle for kids), APIs
      - Data privacy basics, why it matters
      - How to design a survey (Google Forms)
      - Collecting data ethically
      - Data cleaning basics, removing mistakes
      - Introduction to CSV files
    - Projects built:
      - Create a full CBSE-format AI Project Cycle document
      - Design and conduct a survey in your school (30 responses)
      - Map your project to 2 SDG goals
    - Practice: Practice 4Ws on 3 new problems weekly
  - Data Exploration & Visualization
    - Topics:
      - Data exploration, why we look at data
      - Types of charts: Bar, Line, Pie, Scatter (when to use which)
      - Google Sheets / MS Excel for data exploration
      - Making charts in Google Sheets
      - Finding outliers and missing values
      - Mean, median, mode for kids
      - Data stories, telling the truth with data
      - Avoiding misleading graphs
      - Introduction to data visualization best practices
      - Creating infographics in Canva
      - Interpretation, what does the data say?
      - Designing a data dashboard
    - Projects built:
      - Collect and visualize your class's screen time data
      - Analyze a public dataset on air pollution in Indian cities
      - Create a data-driven infographic on a CBSE social topic
    - Practice: Make one chart from real data every day

- Month 5: AI Domains, Data, Computer Vision, NLP (Week 18-21)
  - Data Science, Teaching Computers to Predict
    - Topics:
      - What is Data Science?
      - Prediction vs Classification vs Clustering
      - Linear thinking in predictions (temperature, prices)
      - Classification with Teachable Machine (extended)
      - Clustering explained with sorting toys game
      - K-Means clustering, simplified
      - Decision Trees, kids' explanation
      - Building your first decision tree by hand
      - Orange Data Mining, visual ML for kids
      - Installing & exploring Orange Data Mining
      - Uploading a dataset in Orange
      - Running your first ML model visually
    - Projects built:
      - Decision tree: 'What should I wear today?'
      - Clustering project: Group your toys using K-Means thinking
      - Orange ML: Predict if a student will pass based on study hours
    - Practice: Solve one ML thinking puzzle daily
  - Computer Vision, Deep Dive
    - Topics:
      - Pixels, RGB, grayscale explained
      - Image classification vs object detection vs image generation
      - Training a custom Teachable Machine model (10+ classes)
      - Exporting Teachable Machine models to Scratch
      - Using TM models inside Scratch for interactive projects
      - Face filters, how Snapchat/Instagram works (simplified)
      - OCR (Optical Character Recognition), reading handwriting
      - Medical AI, skin disease, X-ray detection (kid-safe intro)
      - Real-world case studies (Indian startups using CV)
      - Ethics: Deepfakes & misuse of CV
      - CBSE sample CV project ideas

- Planning your own CV project
  Projects built:
- Build a Teachable Machine + Scratch 'Hand Gesture Controlled Game'
- Create an attendance system using facial recognition (Teachable Machine)
- Design a 'litter detector' for clean India initiative (SDG)
  Practice: Train one TM model daily

Natural Language Processing, Making Machines Understand Us
  Topics:
- Words as data, how computers read text
- Tokenization, splitting sentences into words
- Bag of Words model (simplified)
- Sentiment analysis, is this message happy or sad?
- Chatbot building blocks
- Rule-based vs AI-based chatbots
- Dialogflow / simple chatbot builders for kids
- Voice input & output in Scratch 3 (speech extensions)
- Translation AI, how Google Translate works
- NLP use cases in India (Bhashini, regional languages)
- Spam filters, a classic NLP task
- Limitations of NLP, why AI gets confused
  Projects built:
- Build a Scratch-based mood-detecting journal bot
- Create a voice-controlled calculator
- Build a Hindi-English translation assistant prototype
  Practice: Analyze sentiment of 10 messages daily

AI Ethics, Bias & Responsible AI (core CBSE AI themes)
  Topics:
- Why AI ethics matters, CBSE emphasis
- What is AI bias? (Famous case studies)
- Gender, racial, cultural bias in AI
- Case study: Biased resume screening AI
- Privacy & AI, your data is valuable
- Consent in the AI age
- Deepfakes & misinformation
- Environmental cost of AI (energy, water)
- AI & Jobs, what the future holds
- Responsible AI principles (fairness, accountability, transparency)
- Asimov's 3 Laws of Robotics (kid-friendly)
- Designing ethical AI, a framework for kids
  Projects built:
- Ethics poster: 'Our Family's AI Rules'
- Case study presentation: Find an example of AI bias
- Design an 'AI Ethics Checklist' for classroom use
  Practice: Daily AI ethics journal

Month 6: Modelling & Evaluation + First Python Steps (Week 22-26)

Modelling, How Machines Learn
  Topics:
- What is a model?
- Supervised learning (with labeled examples)
- Unsupervised learning (finding patterns alone)
- Reinforcement learning (learning from reward)
- Training, validation and testing data
- Overfitting & underfitting, kids' analogy
- Accuracy, error, confusion matrix basics
- When models fail
- Improving a model
- Retraining & iterating
- Google's Teachable Machine: Advanced usage
- Introduction to MIT App Inventor for AI apps
  Projects built:
- Train, test and evaluate 3 different TM models
- Build an MIT App Inventor app that uses a trained AI model
- Improve an existing model by adding more data
  Practice: Evaluate one model daily

Evaluation & Deployment
  Topics:
- Evaluation metrics (simplified)
- Precision, Recall, kid-friendly explanations
- F1 Score in simple terms
- When AI shouldn't be deployed
- Real-world deployment examples
- User testing with friends and family
- Feedback & iteration cycle
- Packaging & sharing AI projects
- Hosting AI apps (Teachable Machine, MIT App Inventor, Glitch)
- Presenting AI projects to non-technical people
- Creating project documentation
- CBSE AI project report format

      Projects built:
        - Deploy one AI app for family use (MIT App Inventor)
        - Write a CBSE-format AI Project Report (5 pages)
        - Conduct 3 user-testing sessions
      Practice: Get feedback from 3 people per project
    Introduction to Python for AI (CBSE Class 8 Transition)
      Topics:
        - Why Python? The language of AI & CBSE Class 9+
        - Installing Python & Thonny (kid-friendly IDE)
        - Google Colab, Python in the browser
        - Variables, data types, print statement
        - Input & simple calculations
        - If-else in Python
        - Loops (for, while)
        - Lists, storing multiple items
        - Functions, reusable code
        - Random module, fun first projects
        - Turtle module, drawing with Python
        - First AI-like program in Python (rule-based chatbot)
      Projects built:
        - Python turtle, draw Indian flag, mandala, geometric art
        - Python rule-based chatbot
        - Dice roller, number guessing, quiz game in Python
      Practice: Write 10 Python lines daily
    Phase 2 Capstone, End-to-End AI Project
      Topics:
        - Full AI Project Cycle application
        - Scratch + Teachable Machine + Python integration
        - SDG alignment
        - Documentation
        - Showcase preparation
        - Peer review
        - Parent-teacher showcase
        - Competition readiness
      Projects built:
        - CAPSTONE 3: Full AI Project solving a community problem (choose any SDG)
        - Examples: Plant disease detector, School waste tracker, Elderly help chatbot, Road safety app
        - Complete CBSE-format report + video + live demo
      Assessment: CBSE CT & AI Builder Certificate (Classes 6-8 level)

PHASE 3: YOUNG AI CREATOR TRACK (MONTHS 7-8, WEEKS 27-34)

Advance beyond the CBSE Class 6-8 level into true AI project creation. Build 8+ real AI
projects, participate in mock hackathons, understand emerging AI technologies, and prepare
for Olympiads and competitions.

  Month 7: Advanced AI Tools & Projects (Week 27-30)
    Generative AI for Kids, Safe Exploration
      Topics:
        - What is Generative AI? (chatbots and image generators, simplified)
        - How GenAI is different from earlier AI
        - Using GenAI safely, age-appropriate rules
        - Prompt Engineering basics for kids
        - Creating AI art (safe, kid-friendly tools)
        - AI writing helpers for schoolwork (ethical use)
        - AI as a homework helper vs homework doer (boundary setting)
        - Citing AI in school assignments
        - Creativity + AI = Superpower
        - AI music generation introduction
        - AI video generation basics
        - Ethics of GenAI, hallucinations & misinformation
      Projects built:
        - Generate a storybook cover with AI and Canva
        - Design an AI-generated bookmark
        - Write a 300-word story with AI assistance (document your prompts)
      Practice: Daily prompt engineering journal
    Robotics & AI, Bringing AI to the Real World
      Topics:
        - What is a robot? Robot vs AI
        - mBot, LEGO Mindstorms, Spike, kid-friendly robots
        - Sensors: ultrasonic, light, touch, camera
        - Actuators: motors, servos, LEDs
        - Virtual robotics, VEXcode VR
        - Line-following robot (simulated)
        - Obstacle-avoidance robot (simulated)
        - Adding AI to robots, computer vision for robots
        - Humanoid robots (Sophia, Optimus) overview
        - Indian robotics, ISRO, DRDO, startups

- Future of robotics in homes & hospitals
- Robotics competitions in India (WRO, FLL)

Projects built:
- Program a virtual robot in VEXcode VR
- Design a line-following robot algorithm
- Storyboard: 'A day with your personal AI robot'

Practice: Daily robotics simulation

AI for Sustainable Development Goals (SDGs)

Topics:
- Overview of the 17 UN SDGs (a core CBSE AI curriculum theme)
- How AI is helping each SDG (case studies)
- AI for Clean Water (Goal 6)
- AI for Quality Education (Goal 4)
- AI for Zero Hunger (Goal 2)
- AI for Climate Action (Goal 13)
- AI for Good Health (Goal 3)
- Indian AI-for-Good examples (Swasth, eSanjeevani)
- Designing an SDG-aligned AI project
- Finding SDG datasets (UN, Government of India)
- Measuring impact
- Youth AI-for-Good competitions (Google, UNICEF)

Projects built:
- Choose one SDG, design a complete AI solution
- Build a prototype of the solution
- Write an impact report (2 pages)

Practice: Daily SDG research

App-Based AI Projects with MIT App Inventor

Topics:
- App design principles
- MIT App Inventor advanced
- Integrating Personal Image Classifier
- Using Look Extension for AI
- Adding voice recognition to apps
- Text-to-speech in apps
- Connecting to Firebase for data
- Publishing APKs and sharing
- UI/UX for kids
- App store basics (concept only)
- Making your app accessible

Projects built:
- AI-powered plant identifier app
- Voice-command note app
- Flash card memorizer with NLP

Practice: Build one mini app weekly

Month 8: Mini-Hackathon & Olympiad Prep (Week 31-34)

Competition Preparation & Mini-Hackathon

Topics:
- What is a hackathon? Format and rules
- Team formation strategies
- Idea generation (design sprints)
- Rapid prototyping
- Pitching an AI idea (Pixar method)
- Creating a great demo video
- Presenting to judges (body language, voice)
- Handling Q&A
- Time management during competitions
- Famous CBSE AI competitions (CBSE AI Hackathon, Intel AI Youth)
- Indian AI Olympiads (IIRC, TechXcelerate)
- International AI for Good competitions

Projects built:
- Full 48-hour mini-hackathon (home edition)
- Original AI project + pitch + demo
- 3-minute pitch video for portfolio

Practice: Daily idea generation

AI Olympiad & CBSE Competition Preparation

Topics:
- AI Olympiad syllabus overview
- CBSE AI Hackathon past problems
- Logical reasoning practice
- Python/Scratch speed challenges
- AI vocabulary & concepts test prep
- Time-bound problem solving
- Mock Olympiad rounds
- AI puzzle solving (riddles, pattern tests)
- Math for AI, proportion, averages, probability basics
- Debate: 'Is AI dangerous?' (public speaking)
- Essay writing: AI in India 2030
- Portfolio polishing

```
      Projects built:
        - Complete 5 mock Olympiad rounds
        - Submit to one real CBSE/Intel AI competition
        - AI essay for school magazine
      Practice: Daily timed practice

------------------------------------------------------------
PHASE 4: MASTERY & TRANSITION (MONTH 9, WEEKS 35-40)
------------------------------------------------------------
Final capstone project, portfolio creation, CBSE AI Class 9 (Code 417) readiness test, and
graduation ceremony.

  Month 9: Capstone, Portfolio & Certification (Week 35-40)
    Grand Capstone Project (Full AI Lifecycle)
      Topics:
        - Selecting a real community problem
        - Deep stakeholder interviews
        - Complete AI Project Cycle execution
        - Multi-week project management
        - Using Git/GitHub basics for kids
        - Version control & backup
        - Collaboration with classmates
        - Expert mentor review sessions
        - Iterative development
        - User testing with 10+ real users
        - Formal CBSE-style report writing (10 pages)
        - Presentation mastery
      Projects built:
        - FINAL CAPSTONE: Full AI solution for a real community issue
        - Complete CBSE-style report (cover, abstract, methodology, results, ethics, SDG alignment)
        - Public GitHub portfolio with documentation
    Portfolio, LinkedIn (with parental guidance) & Public Showcase
      Topics:
        - Portfolio website building
        - Personal branding for young creators
        - Creating a project showcase page
        - Video editing for project demos
        - Writing project stories (journalistic style)
        - Social media presentation (parent-managed)
        - Preparing for school science fair
        - Public speaking, mini TED-talk format
        - Handling feedback gracefully
        - Connecting with peer creators
        - Maintaining lifelong learning habits
        - Giving back, teach a younger friend
      Deliverables:
        - Personal portfolio site (Netlify/GitHub Pages)
        - 50+ projects documented with screenshots and videos
        - 10-minute live showcase presentation
    CBSE AI Class 9 (Code 417) Readiness Test & Beyond
      Topics:
        - CBSE Class 9 AI (417) curriculum overview
        - CBSE Class 11-12 AI (843) curriculum preview
        - Sample past papers (Class 9 AI)
        - Practical file requirements
        - Employability skills (part of CBSE AI curriculum)
        - Self-management, ICT, Entrepreneurial, Green skills
        - Mock Class 9 AI assessment
        - Gap identification
        - Personalized next-step recommendation
        - Transition pathway plan
        - Parent conference
        - Learning roadmap for next 3 years
      Assessment: CBSE AI Class 9 (Code 417) Readiness Certification
    Graduation & Final Showcase
      Topics:
        - Final skill assessment
        - Certification ceremony (virtual + physical certificate)
        - Alumni community induction
        - Access to future courses and updates
        - Mentor allocation for next year
        - Community contribution opportunities
        - Celebrating every student's journey
        - Setting 3-year learning goals
      Deliverables:
        - CBSE-Aligned Computational Thinking & AI Certificate for Kids
        - 50+ Project Portfolio
        - LinkedIn-shareable digital badge
        - Next-level course pathway plan
```

Assessment: FINAL EXAM: capstone demo plus a friendly viva and a mixed quiz drawn from the whole course. Passing earns the certificate; a child who falls short gets focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 10+ unplugged CT activities, 8+ Scratch projects, 3+ AI awareness projects
- Phase 2: 6+ full AI Project Cycle projects, 5+ Teachable Machine models, 2+ Python projects
- Phase 3: 8+ advanced AI projects, 3+ robotics simulations, 2+ MIT App Inventor apps
- Phase 4: 1 grand capstone + portfolio of 50+ documented projects
- Total: 50+ projects with full CBSE-style documentation

SKILLS THE STUDENT WILL HAVE AT THE END

- Computational Thinking: 4 pillars mastery (Decomposition, Pattern Recognition, Abstraction, Algorithms)
- Block Coding: Scratch advanced level (events, loops, conditions, variables, broadcasts, cloning)
- AI Fundamentals: CV, NLP, Data Science, ML basics, ethics, bias
- AI Project Cycle: Full CBSE-aligned 5-stage process
- Data literacy: Collection, cleaning, visualization, interpretation
- Python basics: Variables, loops, conditions, lists, functions, turtle, random
- Prompt engineering (GenAI): Safe, effective, age-appropriate
- Project management: Planning, execution, documentation, presentation
- Soft skills: Public speaking, teamwork, critical thinking, empathy
- Research: Surveys, datasets, stakeholder interviews, impact reporting
- Ethics: Bias, privacy, responsibility, SDG alignment
- Technology literacy: Apps, AI, IoT basics, robotics simulation

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 x 60-min expert-led sessions/week
Self practice: 2-3 hours/week
Project work: 1-2 hours/week
Parent engagement: 30 mins/week progress review
Total per week: 5-7 hours

SUPPORT INCLUDED

Live sessions: Weekly live classes with CBSE-trained AI instructors
One on one: Fortnightly 1-on-1 mentor check-ins
Parent reports: Monthly detailed progress reports for parents
Community: Kids-only private community (moderated)
Feedback: Weekly project reviews with feedback
Resources: Full access to curated AI resource library
Doubt support: WhatsApp group + ticket system with <12 hr response time
Parent workshops: Quarterly AI-for-Parents sessions

CERTIFICATION DETAIL

Phase certificates: Separate certificate after each phase (4 total)
Final certificate: Certified Young Computational Thinker & AI Innovator
Linkedin badge: Shareable digital badge for LinkedIn/school profile
School recognition: Recognized by 400+ CBSE & ICSE schools (and growing)
Portfolio projects: 50+ documented CBSE-style AI projects
Cbse class 9 readiness: CBSE AI Class 9 (Code 417) Readiness Certificate

WHERE THIS CAN LEAD

- Direct pathway to CBSE AI Class 9-10 (Code 417)
- Direct pathway to CBSE AI Class 11-12 (Code 843)
- Future AI Engineer / ML Engineer
- Future Data Scientist
- Future Robotics Engineer
- Future Prompt Engineer / AI Product Designer
- Future AI Ethics Researcher
- Future Computer Science Researcher
- Future Tech Entrepreneur
- Any career benefiting from computational & analytical thinking (medicine, law, business, arts)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course officially approved by CBSE?
A: Our course is 100% aligned with the official CBSE Computational Thinking & Artificial Intelligence framework and NCERT AI handbooks. While CBSE does not 'approve' external courses, we have designed every module to match CBSE syllabus for Classes 3-8 AI curriculum.

Students completing this program will be ahead of CBSE AI Class 9 (Code 417) requirements. Our students from 400+ CBSE/ICSE schools have successfully used this course as supplementary and even replacement learning.

Q: My child is in Class 3, is this course too advanced?
A: Absolutely not! Phase 1 (Months 1-3) is specifically designed for Classes 3-5 with unplugged activities, Scratch basics, and AI awareness, no technical prerequisite. We have separate age-appropriate batches for Primary (Classes 3-5) and Middle School (Classes 6-8). Younger children progress at their own pace and reach Phase 2 when ready. Many Class 3 students complete the full 9 months successfully with proper pacing.

Q: My child is in Class 8, isn't this too basic?
A: Class 8 students enter directly into Phase 2 (Month 4) after a quick diagnostic, then progress through Python, advanced AI projects, GenAI, robotics, and capstone projects that are well beyond the CBSE Class 8 AI chapter. By the end, they are not just ready for CBSE AI Class 9 (Code 417), they are 6-12 months ahead of it. We also offer optional Phase 3+ advanced tracks for ambitious Class 8 students.

Q: Does this course prepare for CBSE AI Class 9 (Code 417)?
A: Yes, this is one of our core outcomes. Phase 4 includes an explicit CBSE Class 9 AI (Code 417) Readiness Test. Students who complete this course are fully prepared to excel in the Class 9-10 CBSE AI curriculum, including Communication Skills, Self-Management, ICT Skills, Entrepreneurial Skills, Green Skills, Introduction to AI, AI Project Cycle, Neural Networks intro, Python intro, and Ethics of AI. Many of our students score 95+ in their Class 9-10 AI board exams.

Q: How is this different from school AI classes?
A: Most CBSE school AI classes cover the curriculum at surface level due to 1-2 hours/week time constraints. Our course goes 10x deeper with 5-7 hours/week, live expert instruction, 50+ hands-on projects, CBSE-format project documentation, and continuous feedback. We also include cutting-edge topics (GenAI, Prompt Engineering, Robotics simulation, MIT App Inventor) that most school curricula skip. Result: our students don't just pass AI, they create AI.

Q: Is Python necessary at this age?
A: Python is introduced only in Month 6 (Class 8 level), and even then in a gentle, kid-friendly way using Turtle graphics and simple games. CBSE introduces Python officially in Class 8 AI chapter and heavily in Class 9 (Code 417). Starting early gives your child a huge advantage. Younger kids (Classes 3-6) focus on Scratch and unplugged CT, which pedagogically is the right progression.

Q: What if my child misses a class?
A: Small batches make catch-up genuinely easy: the teacher recaps the missed concepts in the next session, class notes and the week's practice set are shared over WhatsApp, and for longer absences we schedule a short free catch-up slot so no child falls behind the batch.

Q: Are the projects CBSE-board-exam standard?
A: Yes. All capstone projects follow the official CBSE AI Project Cycle format (Problem Scoping Data Acquisition Data Exploration Modelling Evaluation). Reports are written in CBSE practical file format with abstract, methodology, SDG alignment, ethics consideration, and results, exactly the format required by CBSE AI board exams for Class 9-12.

Q: How much screen time does this course require?
A: We are very mindful of healthy screen time, especially for younger kids. Phase 1 (Classes 3-5) is 60-70% unplugged (no-screen) activities, puzzles, paper flowcharts, human-robot games, nature pattern hunts. Total screen time averages 3-4 hours/week for Phase 1 and 5-6 hours/week for Phases 2-4. We encourage physical activity, outdoor breaks, and offline thinking time throughout.

Q: Will my child be ready for AI competitions?
A: Absolutely, Phase 3 Week 31-34 is dedicated competition preparation. Our students regularly participate in and win CBSE AI Hackathons, Intel AI Youth Challenge, Samsung Innovation Campus, Microsoft AI for Good, and India AI Olympiad.

Q: Do parents need technical knowledge?
A: Zero technical knowledge needed. We provide parent-friendly weekly progress reports, quarterly AI-for-Parents workshops and clear written progress notes. Many of our most successful students have parents from non-technical backgrounds. In fact, we encourage parents to 'learn alongside' for the first month, it's a beautiful bonding experience!

Q: How do I know this course is right for my child?
A: Book a FREE diagnostic session with our CBSE AI expert. We will assess your child's current level, interests, and goals, then recommend whether they should join Primary (Classes 3-5) or Middle School (Classes 6-8) batch. If we feel this course is not the right fit, we will honestly tell you, we only enroll students who will truly benefit.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/cbse-computational-thinking-and-ai-course-for-kids-classes-3-to-8
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at

https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 37 OF 137: CBSE COMPUTATIONAL THINKING & AI FOR TEENS (CLASS 9-12)
==========

Course link: https://learn.modernagecoders.com/courses/cbse-computational-thinking-and-ai-course-for-teens-classes-9-to-12-code-417-843
Subject area: CBSE School Board Exam Preparation - Artificial Intelligence (Code 417 & 843)
Designed for: Teens
Who can join: Complete Beginner to CBSE AI Board Topper + Industry-Ready AI Developer
Course length: 12-14 months (48-56 weeks), or the full Class 9-12 board track
Time needed: 10-12 hours/week (2-3 live classes + practical + self-study)
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam; syllabus aligned to CBSE AI 417 and 843

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

CBSE Computational Thinking & AI for Classes 9-12: Python, Machine Learning and board practical mastery. Covers AI 417 (Class 10) and AI 843 (Classes 11-12).

In one line: From CBSE AI Board Topper to Industry-Ready AI Engineer, In One Comprehensive Program

ABOUT THE PROGRAMME

The Central Board of Secondary Education (CBSE) introduced Artificial Intelligence as an official Skill Subject (Code 417 for Class 9-10, Code 843 for Class 11-12) in response to India's urgent need for AI-ready youth. This 12-month flagship program is the definitive preparation course for every CBSE teen pursuing AI, whether as a board subject, competition pathway, or future career. You will master the complete CBSE AI syllabus with 10x greater depth than any school offers: Employability Skills (Communication, Self-Management, ICT, Entrepreneurial, Green Skills), the complete AI Project Cycle (Problem Scoping Data Acquisition Data Exploration Modelling Evaluation), AI Domains (Data, Computer Vision, Natural Language Processing), Python programming from zero to advanced, NumPy, Pandas, Matplotlib, scikit-learn, TensorFlow, Keras, OpenCV, NLTK, Neural Networks, Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Transformers, Generative AI (ChatGPT-class text models, diffusion image models), LLM concepts, Prompt Engineering, Reinforcement Learning, AI Ethics, SDG-aligned capstone projects, and full board practical file + viva preparation. By completion, you will have (1) scored 95+ in your CBSE AI board exam, (2) built 80+ industry-grade AI projects in your GitHub portfolio, (3) prepared for top AI Olympiads (India AI Olympiad, Intel AI Youth, Samsung Innovation Campus), and (4) positioned yourself for IIT/NIT/BITS/top universities and future AI careers earning Rs 40+ LPA.

WHAT MAKES THIS COURSE DIFFERENT

- 100% CBSE AI Code 417 + 843 syllabus coverage, not a single topic missed
- Separate dedicated batches for Class 9, 10, 11, and 12
- Full Employability Skills module (10% of CBSE AI marks, often neglected by schools)
- CBSE-format practical file creation with 20+ lab activities per class
- Every capstone project follows official CBSE AI Project Cycle format
- Viva voce preparation with 500+ commonly asked board questions
- Sample papers, past year papers, marking scheme analysis, topper answer sheets
- Industry-level Python, ML, DL training, goes beyond board syllabus
- GitHub portfolio of 80+ projects for college applications and internships
- Live 1-on-1 mentorship with AI engineers from Google, Microsoft, IIT alumni
- Guaranteed 95+ marks in CBSE AI board exam or 50% fee refund
- Preparation for India AI Olympiad, Intel AI Youth, Samsung Innovation Campus, MS Imagine Cup

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Employability Skills + CT + Python + AI Basics + the CBSE AI 417 syllabus
Phase 2: AI Project Cycle Mastery (Months 4-6): Full CBSE AI Project Cycle + Data/CV/NLP domains + Class 10 AI (417) syllabus + Board prep
Phase 3: Machine Learning & Class 11 AI (843) (Months 7-9): scikit-learn, pandas, ML algorithms, Data Visualization, Class 11 AI full syllabus
Phase 4: Deep Learning + Class 12 AI (843) + Career Launch (Months 10-12): TensorFlow, CNN/RNN/Transformer, GenAI, Class 12 full syllabus, board + industry portfolio

WHAT A STUDENT NEEDS BEFORE STARTING

Age: 13-18 years (Classes 9 to 12)
Coding experience: None, we start from absolute zero. Prior Scratch/Python is a bonus.
Equipment: Laptop/desktop (min 8GB RAM, i3+/Ryzen 3+), stable internet (10 Mbps+), webcam, headphones
Software: All free (+ Google Colab for GPU). Paid optional: Colab Pro ($10/mo) or Kaggle free GPU
Time commitment: 10-12 hours/week (2-3 classes + practical + self-study)
English: Class 9+ English comprehension
Math: Basic Class 9 Math (Algebra, simple geometry). Advanced math taught as needed.
Motivation: Willingness to be challenged, curiosity for AI, commitment to board results

WHO THIS COURSE IS FOR

Cbse class 9: Students opting for CBSE AI as Skill Subject (Code 417) in Class 9
Cbse class 10: Students pursuing CBSE AI (417) with board exam in Class 10
Cbse class 11: Students electing CBSE AI (Code 843) as academic elective in Class 11
Cbse class 12: Students with CBSE AI (843) board exam in Class 12
Other boards: ICSE/ISC, IB, Cambridge, State Board students, curriculum fully transferable
International students: NRI/PIO students preparing for Indian board exams from abroad
Cs non cs students: Students doing CBSE CS (083) or IP (065) wanting AI depth alongside
Engineering aspirants: JEE/BITSAT aspirants wanting AI portfolio for college applications
International college applicants: Students applying to MIT, Stanford, CMU, AI portfolio is critical
Competition aspirants: Kids gunning for India AI Olympiad, Intel AI Youth, Samsung Innovation Campus, MS Imagine Cup, Google AI, UNICEF AI-for-Good
Startup minded: Teens who want to launch an AI product/startup while in school
Self learners: Homeschoolers and advanced learners wanting structured AI mastery

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: CBSE AI FOUNDATION (CODE 417 SYLLABUS, MONTHS 1-3, WEEKS 1-13)

Complete mastery of the CBSE AI 417 syllabus - Employability Skills, Introduction to AI, AI Project Cycle basics, Neural Networks and Python for AI. This is the Class 10 board syllabus, and the right foundation for a Class 9 student now that CBSE has discontinued AI 417 at Class 9. Every student builds a rock-solid foundation regardless of prior coding experience.

Month 1: Employability Skills + Introduction to AI (CBSE Unit 1-3) (Week 1-4)
Employability Skills Part 1: Communication & Self-Management (CBSE Mandatory Unit)
Topics:
- Why Employability Skills matter (10% of CBSE AI marks)
- Communication Skills: Definitions, types, importance
- Verbal vs Non-verbal communication
- Communication cycle and feedback
- Barriers to effective communication
- Writing skills, Kinds of sentences, parts of speech, tenses
- Basic English grammar for CBSE AI
- Self-Management Skills: Introduction
- Stress management & working independently
- Self-awareness, self-motivation, self-regulation
- Goal setting, SMART goals framework
- Time management techniques
Projects built:
- Write a 500-word reflective essay on 'AI in my life'
- Create a SMART goals worksheet for the AI course
- Group presentation: 'Barriers to communication in AI teams'
Practice: Daily vocabulary + communication drills
Employability Skills Part 2: ICT, Entrepreneurial & Green Skills
Topics:
- ICT Skills: Introduction to computers
- Basic ICT operations: files, folders, shortcuts
- Operating system basics (Windows/Ubuntu)
- Internet, email, search, cloud basics
- Entrepreneurial Skills: Introduction
- Qualities and functions of entrepreneurs
- Myths about entrepreneurship
- Green Skills: Introduction & importance
- Sustainable development & SDGs
- Green economy & green jobs
- Digital ethics and cyber safety
- Password management, phishing awareness
Projects built:
- Set up a cloud backup of your AI work (Google Drive)
- Interview a local entrepreneur (1-page report)
- Green AI poster, connect 3 SDGs to AI
Practice: Daily ICT skill drill (typing, shortcuts, productivity)

- Introduction to Artificial Intelligence (CBSE Unit 1, Subject Specific)
  - Topics:
    - Definition of AI, CBSE official definition
    - History & evolution of AI (1950-2026 timeline)
    - AI, ML, DL, the Venn diagram
    - Types of AI: Narrow AI, General AI, Super AI
    - Types by functionality: Reactive, Limited Memory, Theory of Mind, Self-Aware
    - AI in everyday life: 20 real examples
    - Domains of AI: Data Science, Computer Vision, NLP (detailed)
    - Is this AI? Classifying systems as AI or not AI
    - AI success stories (AlphaGo, IBM Watson, Tesla, ChatGPT)
    - AI vs Automation vs Robotics
    - AI in India, Aadhaar, UPI, Digital India context
    - Myths about AI (will it take our jobs?)
  - Projects built:
    - AI Spotter, identify 50 AI systems in daily life
    - AI Timeline infographic (Canva)
    - Debate: 'Narrow AI vs General AI, will we ever reach AGI?'
  - Practice: CBSE textbook Unit 1 MCQs + short answer
- AI Project Cycle, Deep Dive (CBSE Core Unit)
  - Topics:
    - The 5-stage AI Project Cycle (official CBSE framework)
    - Stage 1: Problem Scoping with 4Ws (Who, What, Where, Why)
    - Stakeholder identification and needs assessment
    - SDG alignment, mandatory in CBSE AI
    - Stage 2: Data Acquisition, sources, types, ethics
    - Structured vs Unstructured data
    - Stage 3: Data Exploration, EDA basics
    - Stage 4: Modelling, Rule-based vs Learning-based
    - Learning-based: Supervised, Unsupervised, Reinforcement
    - Stage 5: Evaluation metrics introduction
    - Iteration and deployment
    - CBSE-format AI Project Report template
  - Projects built:
    - Apply the full 5-stage Project Cycle to a real school problem
    - Write a 10-page CBSE-format AI Project Report
    - Present project in board-exam viva style
  - Practice: Solve 3 past-year Project Cycle questions daily

Month 2: Python Programming for AI (CBSE Unit 3) (Week 5-8)
- Python Basics (CBSE-aligned + Industry Level)
  - Topics:
    - Why Python for AI, CBSE recommendation
    - Python installation (Anaconda, Thonny, VS Code)
    - Google Colab for free GPU access
    - Jupyter Notebook basics
    - Variables, data types (int, float, string, bool)
    - Operators (arithmetic, comparison, logical, bitwise)
    - Input/output statements
    - Type conversion and type checking
    - Strings, indexing, slicing, methods
    - Python standard library intro
    - Running first CBSE-syllabus Python programs
    - Writing clean, commented code
  - Projects built:
    - BMI Calculator
    - Temperature converter
    - Interactive quiz on AI concepts
  - Practice: Daily 10 Python programs from CBSE textbook + Sumita Arora
- Python Control Flow & Data Structures
  - Topics:
    - If-elif-else (nested conditionals)
    - For loop (with range, enumerate)
    - While loop (with break, continue)
    - Nested loops
    - Lists, full mastery (methods, comprehensions)
    - Tuples, immutable sequences
    - Dictionaries, key-value storage
    - Sets, unique collections
    - List vs Tuple vs Dict vs Set, CBSE comparison table
    - Functions, def, parameters, return, scope
    - Lambda functions
    - Recursion (introductory)
  - Projects built:
    - To-Do List application
    - Contact Book with dictionaries
    - Password generator
  - Practice: 50 Python programs on control flow
- NumPy & Pandas, Data Handling for AI (CBSE Class 11 AI Unit)

```
    Topics:
      - NumPy introduction, why arrays
      - Creating arrays (1D, 2D, 3D)
      - Array operations, broadcasting, indexing
      - NumPy vs Python list, performance
      - Pandas Series & DataFrame
      - Reading CSV, Excel, JSON files
      - DataFrame indexing, selection, filtering
      - Handling missing data
      - Grouping, aggregation, merging
      - Pandas for CBSE Informatics Practices (supplementary)
      - Real dataset exploration, government datasets
      - data.gov.in & Kaggle intro for teens
    Projects built:
      - Analyze India's air pollution dataset (Pandas)
      - Cricket stats analysis
      - Exam performance data dashboard
    Practice: Daily Pandas challenges
  Data Visualization, Matplotlib, Seaborn (CBSE Class 11 AI)
    Topics:
      - Why visualize data, CBSE exam context
      - Matplotlib basics (pyplot interface)
      - Line, bar, pie, scatter, histogram, box plots
      - Customizing plots (titles, labels, legend)
      - Subplots and figures
      - Seaborn introduction, statistical plots
      - Heatmaps, pair plots, violin plots
      - Plotly for interactive charts
      - Design principles of good visualization
      - Common mistakes in visualization
      - CBSE sample viva questions on visualization
      - Exporting plots to reports
    Projects built:
      - COVID-19 India data visualization
      - Climate change visualization portfolio
      - Student performance visualization for school
    Practice: Daily chart-making from random datasets

Month 3: AI Domains Deep Dive + Neural Networks Intro (CBSE Class 10) (Week 9-13)
  Data Science Domain (CBSE Class 10 Unit 4)
    Topics:
      - What is Data Science? (CBSE definition)
      - Data Science project cycle (application of AI Project Cycle)
      - Data Acquisition: Surveys, sensors, web scraping, APIs
      - Data Exploration: Descriptive statistics (mean, median, mode, SD)
      - Data cleaning (handling NaN, duplicates, outliers)
      - Feature engineering basics
      - Simple linear regression (concept)
      - Classification with decision trees
      - Orange Data Mining advanced
      - K-Nearest Neighbors intro
      - Predictive analytics in business
      - Case study: Cricket stats, BSE Sensex, weather forecasting
    Projects built:
      - Predict student marks from study hours (linear regression)
      - Classify emails as spam/ham
      - Stock price trend analysis
    Practice: 3 full Data Science projects
  Computer Vision Domain (CBSE Class 10 Unit 5)
    Topics:
      - What is Computer Vision? (CBSE definition)
      - Applications: Medical imaging, self-driving cars, face unlock, CCTV
      - How images are stored, pixels, RGB, grayscale
      - Image processing fundamentals
      - OpenCV introduction (industry-standard library)
      - Image reading, display, resizing, cropping
      - Filters: blur, sharpen, edge detection
      - Face detection using Haar Cascades
      - Basic object detection
      - Teachable Machine advanced (10+ class classifiers)
      - CNN intuition (pre-deep learning)
      - Ethics of CV, surveillance, deepfakes, bias
    Projects built:
      - Face detector using OpenCV
      - Mask detector (post-COVID era)
      - Custom image classifier with Teachable Machine + Python
    Practice: OpenCV daily challenge
  Natural Language Processing Domain (CBSE Class 10 Unit 6)
    Topics:
```

- What is NLP? (CBSE definition)
- Applications: Chatbots, translation, sentiment analysis, voice assistants
- Chatbots: Rule-based vs AI-based (Script Bot vs Smart Bot)
- Human language vs computer language, the gap
- Text processing: tokenization, stemming, lemmatization
- Stopwords removal
- Bag of Words model (CBSE Class 10 mandatory topic)
- TF-IDF introduction
- Sentiment analysis using Python
- NLTK library basics
- Chatbot building with Dialogflow + Python
- Regional language NLP (Hindi, Tamil, Bengali), Bhashini

Projects built:
- Mental health chatbot (Dialogflow + Python)
- Product review sentiment analyzer
- Hindi-English translator app

Practice: NLP text processing drills

Neural Networks Introduction (CBSE Class 10 Unit 7)

Topics:
- Human brain vs artificial neural network
- Neuron, biological and artificial
- Perceptron model
- Activation functions (Sigmoid, ReLU, Tanh)
- Input, hidden, output layers
- Forward propagation (intuition)
- Loss function basics
- Backpropagation (intuition, not calculus)
- Gradient descent (kid-friendly explanation)
- Types of NN: Feedforward, CNN, RNN (brief)
- Hands-on: Perceptron in Python from scratch
- Hands-on: Simple NN in Keras/TensorFlow

Projects built:
- Build a perceptron from scratch (OR/AND/XOR gates)
- Train a simple NN on MNIST digits
- Compare NN accuracy with vs without hidden layer

Practice: NN concept MCQs daily

CBSE Class 9-10 AI Board Practical File + Viva Prep

Topics:
- CBSE AI Board exam structure
- Theory paper (70 marks) breakdown
- Practical exam (30 marks) breakdown
- Practical file, 10 mandatory activities
- CBSE AI project report format
- Viva voce, common questions
- Topper answer sheet analysis
- Marking scheme understanding
- Time management in exam
- Common mistakes to avoid
- Mock practical exam
- Mock viva session

Deliverables:
- Complete CBSE AI Class 9/10 Practical File (20+ activities)
- Full capstone project with CBSE report
- Mock exam score report

Assessment: CBSE AI 417 board-aligned preparation

PHASE 2: CBSE CLASS 10 AI (417) BOARD MASTERY + ADVANCED PROJECTS (MONTHS 4-6, WEEKS 14-26)

Complete the CBSE Class 10 AI (Code 417) board syllabus mastery with extensive project work, practical file completion, mock exams, and competition preparation. Build 20+ industry-grade projects.

Month 4: Advanced Data Science Projects (Week 14-17)

End-to-End Data Science Projects

Topics:
- Kaggle competition walkthrough
- Feature engineering deep dive
- Handling categorical variables (one-hot, label encoding)
- Train-test split, cross-validation
- Regression: Linear, Polynomial, Ridge, Lasso
- Classification: Logistic Regression, SVM, Decision Trees, Random Forest
- Clustering: K-Means, Hierarchical, DBSCAN
- Model evaluation: accuracy, precision, recall, F1, ROC-AUC
- Hyperparameter tuning (GridSearchCV, RandomizedSearchCV)
- Model deployment basics (Streamlit, Flask)
- MLOps concepts for teens
- Real-world case studies from Indian startups

Projects built:

- House price prediction (Kaggle-level)
- Titanic survival predictor
- Customer churn classifier

Practice: 1 Kaggle micro-competition weekly

Advanced Computer Vision Projects

Topics:

- OpenCV deep dive
- Face detection + recognition pipelines
- Object tracking
- CNN architecture deep dive (LeNet, VGG, ResNet)
- Transfer learning (use pre-trained models)
- Image augmentation
- Training custom CNN in Keras
- MobileNet, YOLO for real-time detection (intro)
- Medical imaging projects
- Agricultural CV (crop disease detection)
- Traffic monitoring AI
- Ethics: privacy + surveillance trade-offs

Projects built:

- Real-time face mask detector
- Crop disease detection (leaf image classifier)
- Traffic sign recognizer for self-driving

Practice: Daily OpenCV + Keras exercises

Month 5: Advanced NLP + Generative AI (Week 18-21)

Advanced NLP & Chatbot Engineering

Topics:

- Text preprocessing pipeline
- Word embeddings: Word2Vec, GloVe
- Advanced sentiment analysis
- Named Entity Recognition (NER)
- Topic modeling (LDA)
- Chatbot engineering with Rasa
- Voice assistants with SpeechRecognition + pyttsx3
- Language translation APIs
- Fine-tuning transformer models (intro)
- Hugging Face transformers library intro
- CBSE AI NLP advanced questions
- Multilingual NLP for Indian languages (Bhashini)

Projects built:

- Multilingual voice assistant (Hindi + English)
- Advanced sentiment dashboard for Twitter/X
- Resume parser using NER

Practice: Daily NLP project

Generative AI & Prompt Engineering (2026-27 CBSE emerging topic)

Topics:

- What is Generative AI, GANs, VAEs, Diffusion, Transformers
- ChatGPT / frontier models (GPT-class, Claude-class, Gemini-class), how they work
- Prompt Engineering fundamentals
- Advanced prompting: few-shot, chain-of-thought, tree-of-thought
- Image generation: how diffusion models work, hands-on with teen-legal tools (Adobe Firefly, Bing Image Creator)
- Text-to-video models explained (Sora-class), concept and implications generation
- LangChain framework introduction
- Building RAG (Retrieval-Augmented Generation) apps
- API integration: OpenAI, Anthropic, Google
- Fine-tuning LLMs for specific tasks
- Ethics of GenAI: hallucinations, copyright, misinformation
- Future of AI work, GenAI native careers

Projects built:

- Build a custom AI tutor using ChatGPT API + LangChain
- Generate a visual storybook with AI (prompts + Adobe Firefly)
- RAG chatbot over your school's PDF notes

Practice: Daily prompt engineering journal

Month 6: CBSE Class 10 Board Final Prep + Capstone (Week 22-26)

CBSE Class 10 AI Board Full Revision + Sample Papers

Topics:

- Complete Unit-wise revision (Units 1-8)
- Chapter-wise MCQ marathon
- Short answer questions bank (100+)
- Long answer questions bank (50+)
- Case study questions mastery
- Last 5 years' board papers, full solution walkthrough
- Topper answer sheet analysis
- Examiner's marking perspective
- Time management strategies
- Common trap questions
- Pre-board simulated exams

        - Answer-writing technique
      Practice: Daily 1 mock paper + analysis
      Deliverables: 5 solved CBSE AI sample papers with feedback
    Class 10 Grand Capstone + Practical File Completion
      Topics:
        - CBSE AI project cycle, full application
        - Stakeholder interviews in your community
        - Data collection, ethical and valid
        - Model building, choose appropriate algorithm
        - Evaluation with real metrics
        - CBSE-format 15-page report writing
        - Demo video production
        - Viva rehearsal with external expert
        - Practical file: 10+ mandatory lab activities with code, output, interpretation
        - File binding & formatting per CBSE standard
        - Examiner FAQs preparation
        - Submission checklist
      Practice: Advance your capstone at home: conduct two stakeholder interviews, log the
      data you collected and note one ethics safeguard you applied.
      Deliverables:
        - Final Class 10 AI capstone project (board-submission quality)
        - Complete practical file (30+ activities)
        - Viva readiness with 500+ questions rehearsed
    Mock Board Exam + Competition Launch
      Topics:
        - Full 3-hour mock board exam
        - Detailed paper analysis
        - Score vs target gap identification
        - Improvement plan
        - Entering competitions: CBSE AI Hackathon, Intel AI Youth, Samsung Innovation, MS Imagine
        - Registration support
        - Team formation
        - Project pitching
      Deliverables:
        - Mock exam score report (target: 95+)
        - At least 1 competition submission
      Assessment: CBSE AI Class 10 (Code 417) Board Ready Certification

----
PHASE 3: CBSE CLASS 11 AI (CODE 843), MACHINE LEARNING DEEP DIVE (MONTHS 7-9, WEEKS 27-39)

Complete CBSE Class 11 Artificial Intelligence (Subject Code 843) syllabus with
industry-grade ML engineering skills. Includes Capability Approach, Sustainable Development,
Foundational Python, Data Visualization, AI Frameworks, Statistical Data, Computer Vision,
Storytelling through Data.

  Month 7: CBSE AI 843 Class 11 Units 1-4 (Week 27-30)
    Capability Approach + Sustainable Development (CBSE AI 843 Unit 1)
      Topics:
        - Amartya Sen's Capability Approach, foundation of modern AI ethics
        - Functionings vs Capabilities
        - Human development vs economic growth
        - Role of AI in expanding human capabilities
        - 17 UN Sustainable Development Goals, deep dive
        - AI for each SDG, 17 detailed case studies
        - Measuring impact and outcomes
        - SDG alignment in AI projects (mandatory CBSE practice)
        - India's SDG progress and AI role
        - Responsible innovation frameworks
        - AI-for-Good case studies: Aravind Eye, eSanjeevani, Swasth
        - Designing SDG-centered AI projects
      Projects built:
        - Research paper: 'AI + SDG of my choice'
        - AI project directly solving one SDG
        - SDG impact measurement framework
      Practice: Pick one UN Sustainable Development Goal, find a real AI project addressing
      it and write a half page note linking it to Sen's capability approach.
    Communication Skills + AI Review + Python Foundation (CBSE Units 2-3)
      Topics:
        - Advanced communication for AI teams
        - Scientific writing for AI papers
        - Presentation skills for AI demos
        - AI review, from Class 9-10 recap to Class 11+ depth
        - History of AI revisited (LISP, Expert Systems, Deep Learning boom)
        - Python foundational advanced: OOP (classes, objects, inheritance, polymorphism)
        - File handling (text, binary, CSV)
        - Exception handling (try-except-finally)
        - Python modules and packages
        - Virtual environments, pip, conda

- Jupyter vs VS Code vs Colab workflows
- Git and GitHub for version control

Projects built:
- Build a Python library (published to PyPI)
- Git-managed group AI project
- Technical blog post on an AI topic

Practice: Record a three minute AI demo presentation of any past project, then solve five Python refresher exercises on lists, dicts and functions.

Data Literacy, Data Visualization & Statistical Data (CBSE Unit 4)

Topics:
- Data literacy in the age of AI
- Data collection methods (primary, secondary)
- Descriptive statistics (full coverage)
- Inferential statistics intro (hypothesis testing)
- Probability for AI (Bayes, conditional, joint, marginal)
- Probability distributions (Normal, Binomial, Poisson)
- Advanced visualization (Seaborn, Plotly, Bokeh)
- Dashboards with Streamlit & Power BI basics
- Storytelling through data (CBSE emphasis)
- Data ethics and privacy (GDPR, DPDP Act 2023)
- Open datasets for Indian context
- A/B testing basics

Projects built:
- Complete Streamlit dashboard on a social issue
- Statistical analysis paper (school topic)
- Power BI dashboard for school data

Practice: Collect a small primary dataset of at least 20 rows from your class or home, compute mean, median, mode and standard deviation, and plot two charts that tell its story.

Month 8: CBSE AI 843 Class 11 Units 5-7 (Frameworks, CV, ML) (Week 31-34)

AI Frameworks & Tools (CBSE Class 11 Unit 5)

Topics:
- scikit-learn, industry-standard ML framework
- TensorFlow 2.x overview
- Keras high-level API
- PyTorch introduction
- Open-source AI ecosystem
- Cloud AI platforms (Google Cloud AI, AWS SageMaker, Azure ML)
- AutoML tools
- Comparing frameworks, when to use which
- Setting up complete AI dev environment
- Containerization (Docker basics)
- Model serialization (pickle, joblib, ONNX)
- API deployment with Flask/FastAPI

Projects built:
- Same ML problem solved in scikit-learn, Keras, PyTorch
- Deploy an ML model as REST API
- Docker-containerized AI app

Practice: Recreate the same simple classifier in both scikit-learn and Keras and note two differences in the workflow.

Computer Vision Advanced (CBSE Class 11 Unit 6)

Topics:
- Image processing fundamentals (full)
- OpenCV mastery
- Convolutional Neural Networks (CNN) from scratch
- CNN architectures: LeNet, AlexNet, VGG, ResNet, Inception
- Transfer learning deep dive
- Object detection: R-CNN, YOLO, SSD
- Semantic segmentation (U-Net intro)
- Face recognition with deep learning
- Optical Character Recognition (OCR)
- Video processing and action recognition
- CV for autonomous vehicles
- CV ethics: facial recognition debate

Projects built:
- YOLO-based real-time object detector
- Attendance system with face recognition
- Sign language recognizer

Practice: Fine tune a pre-trained CNN on a small image dataset using transfer learning and log its accuracy against your from scratch baseline.

Machine Learning Algorithms Mastery (CBSE Unit 7)

Topics:
- ML lifecycle end-to-end
- Supervised: Regression (Linear, Polynomial, Logistic)
- Supervised: Classification (SVM, kNN, Decision Tree, Random Forest, Gradient Boosting, XGBoost)
- Unsupervised: Clustering (K-Means, DBSCAN, Hierarchical)
- Unsupervised: Dimensionality reduction (PCA, t-SNE, UMAP)

```
        - Semi-supervised and self-supervised intro
        - Reinforcement Learning concepts (Q-learning, policy gradient intro)
        - Feature engineering advanced
        - Handling imbalanced datasets
        - Bias-variance trade-off
        - Cross-validation strategies
        - Ensemble learning (bagging, boosting, stacking)
      Projects built:
        - Complete Kaggle competition submission
        - Ensemble model for a Sensex prediction
        - Clustering for Indian consumer segmentation
      Practice: Train three classifiers, kNN, decision tree and random forest, on one dataset
      and compare them in a confusion matrix table.

  Month 9: Class 11 AI Board Prep + Storytelling + Competitions (Week 35-39)
    Storytelling Through Data (CBSE Class 11 Unit 8)
      Topics:
        - Data storytelling frameworks
        - Tufte principles of graphical excellence
        - Narrative structures for AI insights
        - Tableau basics
        - Power BI basics
        - Dashboard design principles
        - Infographics creation
        - Presentation design (slides.com, Pitch)
        - Public speaking for data presentations
        - TED-talk style AI presentations
        - Video storytelling
        - Blog/Medium article writing on AI insights
      Projects built:
        - Tableau dashboard on social issue
        - Medium blog with 1000+ words AI insights
        - 7-minute TEDx-style AI talk (recorded)
      Practice: Turn one of your model results into a three chart data story in Tableau or
      Power BI with a headline insight above each chart.
    Class 11 AI Board Revision + Pre-Board
      Topics:
        - Full syllabus revision (8 units)
        - Sample papers (5 complete)
        - Chapter-wise question bank
        - Case study questions
        - Board-style long answers
        - Topper strategy sessions
        - Mock pre-board exam (3-hour, CBSE format)
        - Examiner feedback simulation
        - Weakness identification + gap filling
        - Final revision checklist
        - Day-before-exam strategy
        - Stress management techniques
      Practice: Daily mock paper + detailed review
      Deliverables: 5 complete mock papers solved
    Class 11 Capstone + Competition Deployment
      Topics:
        - Class 11 AI capstone project (industry-grade)
        - GitHub portfolio curation
        - Competition submissions (Intel AI Youth, Samsung, MS)
        - College application readiness (for Class 11 students preparing for Class 12)
        - Internship pitching to startups
        - Building a personal brand
      Practice: Polish your capstone README and GitHub portfolio page, then draft one
      competition submission entry such as Intel AI Youth.
      Deliverables:
        - CBSE Class 11 AI (843) Board Readiness Certificate
        - Industry-grade capstone project
        - Competition entries with results
        - GitHub portfolio with 50+ projects

------------------------------------------------------------
PHASE 4: CBSE CLASS 12 AI (843), DEEP LEARNING, NLP, CAREER LAUNCH (MONTHS 10-12, WEEKS 40-52)
------------------------------------------------------------
Complete CBSE Class 12 AI (Code 843) syllabus including Capstone Project, Data Science, CV,
NLP, Model Evaluation, AI Ethics. Parallel industry-grade Deep Learning, Transformers, LLMs,
and career-launch for college + internships.

  Month 10: CBSE Class 12 AI Units 1-4 (Week 40-43)
    Communication Skills Advanced + Self-Mgmt + Entrepreneurship (CBSE Units 1-2)
      Topics:
        - Advanced presentation skills
        - Writing scientific reports
```

- Tech interview communication
- Self-management for high-performers
- Stress & time management
- Entrepreneurial mindset in AI
- Indian AI startup ecosystem (YC India, NASSCOM, T-Hub)
- Founding team skills
- MVP building
- Pitching to VCs
- Business Model Canvas for AI startups
- IP and patent basics

Projects built:
- Founder's Deck for a hypothetical AI startup
- Interview for placement simulation (recorded)
- Business Model Canvas for final project

Practice: Draft a one page personal pitch covering who you are, your best AI project and your goal, then rehearse it aloud twice against a timer.

Capstone Project (CBSE Class 12 Unit 3), Official Board Project

Topics:
- CBSE Class 12 AI Capstone guidelines
- Problem scoping with 4Ws (advanced)
- Full AI Project Cycle, executed in 8-10 weeks
- Literature review for AI projects
- Data acquisition (primary + secondary)
- Feature engineering + model building
- Evaluation with appropriate metrics
- Deployment (Streamlit, Flask, Hugging Face Spaces)
- CBSE-format 30-page report
- Demo video production
- Rehearsal + submission
- Viva-voce mastery

Practice: Write the official capstone problem scoping document using the 4Ws canvas plus a literature list of at least three sources.

Deliverables:
- CBSE Class 12 AI Capstone, board-submission quality
- Deployed AI application (public URL)
- Complete 30-page report

Data Science Advanced + Statistical ML (CBSE Class 12 Unit 4)

Topics:
- Advanced statistics for AI
- Time series forecasting (ARIMA, Prophet)
- Anomaly detection
- Bayesian methods
- A/B testing at scale
- Big data processing basics (Dask, Spark intro)
- MLOps introduction
- Model monitoring and drift
- Responsible AI practices
- Indian data regulations (DPDP Act)
- GDPR compliance basics
- Ethical data collection in India

Projects built:
- Stock price forecasting (ARIMA + Prophet)
- Fraud detection pipeline
- MLOps mini-project with monitoring

Practice: Build a time series forecast on a public dataset with ARIMA or Prophet and report one place where the model breaks down.

Month 11: Deep Learning + Transformers + LLMs (CBSE Units 5-6 Extended) (Week 44-47)

Deep Learning Mastery, CNNs, RNNs, LSTMs (CBSE Class 12 Unit 5)

Topics:
- Deep Learning theory deep dive
- Backpropagation mathematics
- Optimizers (SGD, Adam, RMSprop)
- Regularization (dropout, batch norm, L1/L2)
- CNN advanced architectures (EfficientNet, Vision Transformers)
- RNN, LSTM, GRU for sequences
- Sequence-to-Sequence models
- Attention mechanism intuition
- Training deep networks on GPU (Colab Pro/Kaggle)
- Transfer learning best practices
- Model compression (quantization, pruning)
- Edge AI, running models on phones/IoT

Projects built:
- ImageNet-style classifier (transfer learning)
- Stock price LSTM predictor
- Speech command recognizer

Practice: Train the same network twice, with and without dropout and batch normalization, and chart how the validation curves differ.

Transformers, LLMs & Modern AI (CBSE Class 12 Unit 6 extended)

```
    Topics:
      - Transformer architecture (Attention is All You Need)
      - BERT, GPT, T5 explained
      - The LLM landscape: GPT-class, Claude-class, Gemini-class and open-weight Llama-class
      - RLHF (Reinforcement Learning from Human Feedback)
      - Fine-tuning LLMs (LoRA, QLoRA)
      - Building RAG systems
      - LangChain + LlamaIndex
      - Vector databases (Pinecone, Chroma)
      - Hugging Face ecosystem mastery
      - Multimodal AI (frontier vision-language models)
      - Agent-based AI (modern agent frameworks, surveyed honestly)
      - Ethics of LLMs (hallucination, alignment, safety)
    Projects built:
      - Fine-tuned LLM for specific Indian-language task
      - RAG chatbot on personal knowledge base
      - Multimodal AI app (image + text)
    Practice: Run a small experiment comparing zero shot and few shot prompts on the same
    task across two LLMs, and tabulate the results.

Month 12: CBSE Class 12 Board Finals + Career Launch (Week 48-52)
  CBSE Class 12 AI Board Revision + Pre-Board
    Topics:
      - Full 6-unit revision
      - 10 CBSE Class 12 AI sample papers (completely solved)
      - Last 5 years board papers
      - Case studies and long answers
      - MCQ marathon
      - Topper answer sheets analysis
      - Viva-voce drills
      - Practical file finalization (20+ activities)
      - Capstone project final rehearsal
      - Pre-board exam (3-hour full paper)
      - Detailed feedback and improvement
      - Day-before-exam strategy
    Practice: 2 mock papers daily
    Deliverables: Complete board-ready package
  Final Board Readiness + Viva Mastery
    Topics:
      - CBSE AI viva voce question bank (500+ questions)
      - Board examiner perspective
      - Confidence building
      - Practice viva with external AI experts
      - Backup plans for difficult questions
      - Final document check
      - Stress management on exam day
      - Last-minute revision strategy
      - Healthy habits during board exams
      - Post-board college application readiness
    Practice: Answer 20 questions from the viva bank aloud under exam conditions and mark
    the five that need another revision pass.
    Deliverables:
      - Final Board-Ready Certificate
      - All practical file + capstone submitted
      - 100% viva-voice readiness
  College, Internship & Career Launch
    Topics:
      - B.Tech AI/CS/Data Science college guidance (IIT/NIT/BITS/IIIT)
      - JEE Mains/Advanced, role of AI portfolio
      - International college applications (MIT, Stanford, CMU)
      - SAT + AI portfolio strategy
      - AI summer internship applications (Indian + global)
      - LinkedIn optimization for teens
      - GitHub portfolio curation
      - Personal website/portfolio (Next.js)
      - Cold outreach to AI companies
      - Interview preparation (technical + HR)
      - Negotiating your first internship
      - Long-term AI career roadmap
    Practice: Shortlist five target colleges or internships and map which portfolio project
    you would lead with for each application.
    Deliverables:
      - Polished LinkedIn + GitHub + portfolio website
      - 3 cold outreach emails sent
      - 1 internship application submitted
  Graduation & Final Showcase
    Topics:
      - Full skill assessment
      - Final certification ceremony
```

- Alumni community induction (lifetime)
- Access to alumni-only job board
- Mentor-for-life allocation
- Community contribution pathways
- Giving back, teach the next batch
- Continuous learning roadmap (post-Class 12)

Deliverables:
- CBSE AI Code 843 Class 12 Board Topper Certification
- an earned course certificate
- 80+ Project GitHub Portfolio
- Verifiable digital badge

Assessment: FINAL EXAM: a board-style paper plus a practical build, with questions mixed from the whole syllabus. Passing earns the certificate; a student who falls short gets a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: 20+ Python + AI basics projects, CBSE Class 9 practical file
- Phase 2: 20+ Advanced projects, CBSE Class 10 practical file + capstone
- Phase 3: 20+ ML engineering projects, CBSE Class 11 capstone
- Phase 4: 20+ DL + LLM projects, CBSE Class 12 board capstone
- Total: 80+ industry-grade projects in GitHub portfolio

SKILLS THE STUDENT WILL HAVE AT THE END

- Complete CBSE AI Code 417 syllabus (Class 9-10)
- Complete CBSE AI Code 843 syllabus (Class 11-12)
- Python programming (zero to advanced, OOP, async)
- Data Science engineering (NumPy, Pandas, SQL basics)
- Machine Learning engineering (scikit-learn mastery)
- Deep Learning engineering (TF, Keras, PyTorch)
- Computer Vision engineering (OpenCV, CNN mastery)
- NLP engineering (Transformers, Hugging Face mastery)
- Generative AI mastery (prompt engineering, RAG, fine-tuning)
- MLOps basics (deployment, monitoring, versioning)
- AI Ethics & Responsible AI
- CBSE board exam technique (95+ score targeting)
- Viva-voce mastery (500+ questions prepared)
- GitHub portfolio building
- Technical writing & presentation
- Competition-ready problem solving

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2-3 x 90-min expert-led sessions/week
Self practice: 3-4 hours/week (coding + theory)
Project work: 2-3 hours/week
Board prep: 1-2 hours/week (sample papers, revision)
Mentor session: 30 mins fortnightly 1-on-1
Total per week: 10-12 hours

SUPPORT INCLUDED

Live sessions: Weekly live classes with IIT/Google/Microsoft AI engineers
One on one: Fortnightly 1-on-1 mentor sessions
Parent reports: Monthly detailed progress reports for parents
Feedback: Weekly code review + project review
Resources: 200+ curated AI resources, books, papers
Doubt support: <6-hour response time on WhatsApp
Parent workshops: Quarterly AI-for-Parents webinars
Board support: Dedicated board-exam helpline February-March
Competition support: Team formation + mentor guidance for competitions
Internship support: Internship referrals after Class 11

CERTIFICATION DETAIL

Phase certificates: 4 phase certificates + class-specific certificates
Final certificate: CBSE AI (417 & 843) Board Topper + Industry AI Developer Certificate
Linkedin badge: Shareable verifiable digital badge
School recognition: Recognized by 400+ CBSE & ICSE schools
Portfolio projects: 80+ industry-grade documented projects on GitHub
Board readiness guarantee: 95+ board marks guarantee or 50% refund

WHERE THIS CAN LEAD

- B.Tech in AI / CS / Data Science / ECE at IIT, NIT, BITS, IIIT, top private universities
- International admissions: MIT, Stanford, CMU, Georgia Tech, UIUC (AI portfolio is a key differentiator)
- AI/ML Engineer

- Data Scientist
- ML Research Scientist
- AI Product Manager
- AI Startup Founder (unlimited potential, Y Combinator, Sequoia India pathway)
- Computer Vision Engineer
- NLP Engineer / Conversational AI
- Generative AI Engineer / Prompt Engineer
- Quantitative AI (Algo Trading), Rs 30 LPA-2 Crore+
- AI Ethics / Policy

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Can my child take AI (Code 417) in Class 9?
A: Not from the 2026-27 session. CBSE Circular Acad-18/2026 (9 April 2026) discontinued the AI skill subject for Class 9; students already in Class 10 continue on the earlier scheme, so 417 still applies there. What replaces it matters more: CBSE's new Computational Thinking & AI becomes a compulsory Class 9 subject with an annual examination from session 2027-28. A child in Class 8 today sits it next year. This course teaches the full 417 syllabus either way, because it is the same foundation - AI project cycle, neural networks, Python, data science - and it is what Class 10 examines.

Q: Is this course officially aligned with CBSE AI Code 417 and 843 syllabus?
A: Yes, 100% alignment. Every unit of CBSE AI Class 9-10 (Code 417) and Class 11-12 (Code 843) is covered in greater depth than prescribed. We follow the exact CBSE pedagogical order, use NCERT-recommended textbooks and tools, and our capstone projects follow the official CBSE AI Project Cycle format. Our students consistently score 95+ in their CBSE AI board exams (verified by 2,800+ board results).

Q: What is the difference between CBSE AI Code 417 and Code 843?
A: Code 417 is the Skill Subject for Class 9 & 10 (introductory, more hands-on, 30% practical). Code 843 is the Academic Elective for Class 11 & 12 (deeper theory + advanced ML/DL, 70% theory + 30% practical). Our course seamlessly covers both, Phases 1-2 master Code 417, Phases 3-4 master Code 843. Students starting in Class 9 get a complete 4-year pathway; students starting later enter the appropriate phase based on diagnostic.

Q: I'm in Class 12 with boards in a few months. Is this course still useful?
A: Yes, we offer a Class 12 Crash Track (3-4 months intensive) that covers the complete CBSE AI 843 Class 12 syllabus with board-focused revision, 10 mock tests, capstone project (board-submission quality), and viva mastery. Our Class 12 crash students have scored 90+ in under 3 months of intensive prep. Book a free diagnostic to see if this suits you.

Q: My school already teaches CBSE AI, why do I need this?
A: Most CBSE schools cover AI at surface level due to 2-4 hours/week constraint, and teacher training is still catching up. Our course goes 10x deeper with 10-12 hours/week, industry-grade Python + ML + DL, full practical file support, capstone project guidance, board-exam strategy, competition prep, and GitHub portfolio, none of which schools provide. 92% of our students also attend school AI classes, we complement, not replace.

Q: Do you cover the CBSE practical file requirements?
A: Absolutely. Practical file is 30 marks (Code 417) or 30 marks (Code 843). We provide: complete practical file creation support (20+ lab activities per class), file binding and formatting per CBSE standard, viva-voce preparation (500+ question bank), examiner's perspective analysis, and topper-quality answer templates. Our students consistently score 28-30 out of 30 in practicals.

Q: Will this help with JEE / BITSAT / other entrance exams?
A: Directly: No (JEE doesn't test AI). Indirectly: Yes, our computational thinking module sharpens logical reasoning and problem-solving that directly boost JEE Advanced performance. More importantly, your AI project portfolio becomes a massive differentiator for IIT/NIT/BITS interviews, scholarship applications, and international college admissions where portfolio weight is 40-60%.

Q: Is 10-12 hours per week sustainable alongside Class 10/12 boards?
A: Yes, because AI is one of your board subjects (Code 417 for Class 10, Code 843 for Class 12), the time spent directly contributes to your board prep. Our schedule adjusts during board months (Jan-March) to revision mode. Many of our students handle full CBSE board prep (Physics, Chem, Math, English) + AI 95+ score + GitHub portfolio simultaneously with our structured approach.

Q: Do I need a powerful computer for Deep Learning?
A: No, we use Google Colab which provides free GPU access (enough for 90% of our curriculum). A basic 8GB RAM laptop suffices for coding. We teach cloud-first engineering (which is the industry standard anyway).

Q: Will I be ready for international AI programs like Stanford AI4All or MIT Beaver Works?
A: Absolutely. Our curriculum is benchmarked against Stanford CS229 (Machine Learning) and CS231 (Deep Learning), and we've helped 40+ students get into Stanford AI4All, MIT Beaver

Works, CMU AI Summer, Google CodeU, and Google AI India. We provide application guidance and recommendation letter support.

Q: How does the board score guarantee work?
A: If you attend 85%+ classes, complete 80%+ homework, submit all practical file activities and capstone on time, score below 95 in CBSE AI board exam, we refund 50% of fees. Our historical guarantee claim rate is <3% because our methodology works. See full terms in enrollment agreement.

Q: What if I'm in a non-CBSE board but want to learn AI?
A: Our course is fully transferable. ICSE/ISC students use it alongside ICSE Computer Science; IB students use it for Computer Science HL projects; Cambridge students use it for IGCSE ICT/CS; State Board students use it as comprehensive AI skill-building. The only CBSE-specific modules are board-exam prep in Phase 2 Week 22-26 and Phase 4 Week 48-52, which become optional electives for non-CBSE students.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/cbse-computational-thinking-and-ai-course-for-teens-classes-9-to-12-code-417-843
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 38 OF 137: CCC JUNIOR AND SENIOR PREPARATION (CANADIAN COMPUTING COMPETITION)

==========

Course link: https://learn.modernagecoders.com/courses/ccc-junior-and-senior-preparation-course
Subject area: Competitive Programming
Designed for: Teens
Who can join: Students roughly aged 13 to 18 who can write simple programs (Junior track) or are confident programmers ready for algorithms (Senior track)
Course length: A 24-week season, September to the CCC on Wednesday, February 17, 2027, joinable any month
Time needed: 2 live classes/week + weekly problem sets and timed practice contests
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online Canadian Computing Competition prep for ages 13 to 18: Junior and Senior tracks in Python or C++, from September to the CCC on Wednesday, February 17, 2027.

In one line: One season, two tracks, real problem solving: from the first CCC Junior loops to the Senior algorithms that lead toward the Canadian Computing Olympiad.

ABOUT THE PROGRAMME

The Canadian Computing Competition (CCC) is run by the Centre for Education in Mathematics and Computing (CEMC) at the University of Waterloo. The 2027 contest for North and South America is on Wednesday, February 17, 2027. It has two levels: Junior, for "any participant with beginner to intermediate programming skills", and Senior, for "any participant with intermediate to advanced programming skills", and CEMC says each participant can choose the best level "regardless of their grade". Each level has 5 questions, the time allowed is 3 hours, and it is scored out of 75. CCC participants may use C, C++, Python 2, Python 3 or Java, and the contest is written in school: participants write online, individually. This course is a full September-to-February season with two parallel tracks. The Junior track follows the skills CEMC's own question guide describes, from basic loops and conditions, through loops, conditions and counting together, up to the recursion, two-dimensional arrays and efficient algorithms of the hardest Junior question. The Senior track starts where CEMC says the Senior paper starts, with sorting and searching, then moves through careful counting and mathematical reasoning toward the algorithms of the final question, which CEMC calls an IOI level question. Both tracks are taught in Python or C++. For Senior students aiming beyond the CCC we teach C++ properly, because CEMC states that "CCO participants must use C++". Practice comes from two places: our own original problems written for each topic, and CEMC's free past contests, which CEMC publishes with no registration or fee and which students can also try on CEMC's free CCC Online Grader practice account. We never copy contest problems into our materials; students work them where CEMC publishes them. Students register through their school, not through us. We teach the skills and the contest habits, and we make no promise of scores, distinctions, CCO invitations or admissions.

WHAT MAKES THIS COURSE DIFFERENT

- Built for the CCC as CEMC runs it: 5 questions, 3 hours, scored out of 75, written online at school

- Two tracks in one season: Junior and Senior, with the level chosen by skill, not by grade
- Python or C++, with an honest explanation of when C++ matters and why the CCO requires it
- Our own original problems for every topic, plus CEMC's free past contests used as practice where CEMC publishes them
- A timed practice contest every month, ending with full 3-hour mock contests in January and February
- Clear guidance on registering through a school, including CEMC's advice when a school does not take part

THE LEARNING PATH, STEP BY STEP

Phase 1: The shared core for both tracks: reading CCC-style statements, exact input and output, loops, conditions, strings, lists, counting and grids
Phase 2: The tracks split: Junior builds toward recursion, two-dimensional arrays and efficient algorithms; Senior covers sorting, searching, careful counting, graphs and dynamic programming
Phase 3: Contest season: timed mocks, partial-mark strategy, contest-day routine, the CCC on February 17, 2027, then upsolving and next steps

WHAT A STUDENT NEEDS BEFORE STARTING

Coding level: Junior track: simple programs with variables, loops and conditions. Senior track: confident with lists or arrays, functions and basic sorting
Language: Python or C++. Senior students aiming at the CCO move to C++ during the season
Age: Roughly 13 to 18. Level is chosen by skill, not by grade
Equipment: A computer with stable internet that can run Python or a C++ compiler
Contest entry: The CCC is written through a school; check that your school (or another local school) is taking part
Timing: Weekdays or weekends, flexible: choose your batch, or for a very personalised plan choose 1-on-1 at your convenience, weekend mornings or weekdays

WHO THIS COURSE IS FOR

First time contest writers: Students who can code a little and want to write CCC Junior with real preparation
Returning junior students: Students who wrote Junior before and want to reach the harder questions or move up to Senior
Senior track students: Confident programmers who want the sorting, searching, graph and dynamic programming toolkit for CCC Senior
Olympiad hopefuls: Senior students who want to understand the CCO path and train in C++
School programmers: Students in school computer science who want harder, more interesting problems

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE SHARED CORE (SEPTEMBER AND OCTOBER, WEEKS 1-8)

Everything both tracks need: precise reading, exact input and output, and the loops, conditions, strings, lists and counting that the first CCC questions are built on.

Month 1: Contest Setup, Input and Output, Loops and Conditions (Weeks 1-4)
Weeks 1-2: How the CCC Works and First Problems
Topics:
- The CCC format: 5 questions, 3 hours, scored out of 75, Junior and Senior levels
- Choosing a level by skill: a diagnostic to place each student on the Junior or Senior track
- Setting up Python or C++ and creating a free practice account on CEMC's CCC Online Grader
- Reading input exactly as the statement describes and printing exactly what is asked
- Basic loops and conditions on short original problems
- Testing with the sample input, then with cases you write yourself

Projects built:
- A personal starting profile: track choice, language choice and five solved warm-up problems

Practice: Ten short original problems, each tested on at least two extra cases before submitting

Weeks 3-4: Strings, Lists and Simple Counting
Topics:
- String processing: characters, counting, reversing and comparing
- Lists and arrays: storing input, scanning, finding maxima and minima
- Counting with conditions, the core of the middle Junior questions
- Dictionaries or maps for frequency counts
- Simulation: following a set of rules step by step
- Edge cases: empty input, one item, ties and the largest values

Projects built:
- A counting and simulation problem set solved in full, with a written list of edge cases for each

Practice: A first CEMC past contest from CEMC's site, attempted untimed, with a written review

Assessment: Month 1 timed round: a short practice contest of original problems

Month 2: Grids, Functions and First Ideas of Efficiency (Weeks 5-8)
Weeks 5-6: Two-Dimensional Grids and Functions
Topics:
- Reading and storing a grid of characters or numbers
- Moving through a grid: neighbours, borders and directions
- Breaking a solution into functions so it stays readable
- Debugging with small hand-made test cases
- Avoiding repeated work inside loops
- Writing clean code fast enough for a 3-hour contest

Projects built:
- A grid puzzle solver for an original problem, organized into clear functions

Practice: Grid and simulation problems, each traced on a small grid by hand first
Weeks 7-8: How Fast Is Fast Enough?
Topics:
- Counting the steps a program takes as the input grows
- Reading the input limits to guess what solution is needed
- Why a nested loop can pass small tests and time out on large ones
- Sorting with the built-in sort and when it helps
- Partial marks: solving the small cases when the full solution is out of reach
- Keeping an error log of every wrong or slow submission

Projects built:
- Solve one original problem three ways, from slow to fast, and time each version

Practice: Mixed problem set plus one CEMC past contest at the student's level, reviewed in class
Assessment: Phase 1 assessment: a 90-minute practice contest and a written review of every question

---

PHASE 2: JUNIOR AND SENIOR TRACKS (NOVEMBER AND DECEMBER, WEEKS 9-16)

The tracks run in parallel in separate batches. Junior students build toward the hardest Junior question; Senior students build the algorithm families the Senior paper rewards.

Months 3-4, Junior Track: From Counting to Recursion and Efficient Algorithms (Weeks 9-16)
Weeks 9-10: Junior Questions 3 and 4 Skills
Topics:
- Combining loops, conditions and counting in one solution
- Careful string and number parsing from mixed input
- Nested loops over pairs and their cost
- Sets for fast membership checks
- Planning a solution in plain words before coding
- Checking the answer against the samples and a brute-force version

Projects built:
- Five original middle-level problems, each with a plain-word plan and a test list

Practice: Middle Junior questions from CEMC past contests, solved on CEMC's site and reviewed in class
Weeks 11-12: Two-Dimensional Arrays in Depth
Topics:
- Grids as two-dimensional arrays or lists of lists
- Rotations, reflections and counting regions
- Flood fill to find connected areas
- Row and column totals for fast answers
- Off-by-one and border mistakes, and how to avoid them
- Choosing between storing a grid and computing on the fly

Projects built:
- An original region-counting problem solved with flood fill

Practice: Grid problems timed at 30 minutes each, with every miss logged
Weeks 13-14: Recursion
Topics:
- Base case and recursive case on small examples
- Recursive counting and generating all options
- Recursive flood fill and its limits
- Tracing recursion with a call diagram
- When to replace recursion with a loop
- Pruning: stopping early when an option cannot work

Projects built:
- Three recursion problems solved and explained in a short written note

Practice: Recursion problem set with a trace for each solution
Weeks 15-16: Efficient and Insightful Algorithms
Topics:
- Spotting a pattern so a slow search becomes a quick formula
- Prefix sums for fast range totals
- Sorting first to make a problem simple
- Breadth-first search on a grid for shortest moves
- Earning partial marks with a correct slow solution, then improving it
- When a Junior student is ready to try the Senior track

Projects built:

- Take a slow solution from earlier in the course and rewrite it to run within the limits
Practice: A full Junior-style mock of original problems, timed at 3 hours
Assessment: Junior track assessment: a full timed mock with a written review of every question

Months 3-4, Senior Track: Sorting, Searching, Counting, Graphs and Dynamic Programming (Weeks 9-16)
Weeks 9-10: Sorting and Searching
Topics:
- Sorting with custom keys and comparators
- Binary search on a sorted list and binary search on the answer
- Two pointers and sliding windows
- Prefix sums and difference arrays
- Fast input and output in Python and C++
- Reading constraints to choose the right complexity
Projects built:
- An original problem solved with binary search on the answer, with a written proof that it is fast enough
Practice: Early Senior questions from CEMC past contests, solved on CEMC's site and reviewed in class
Weeks 11-12: Careful Counting and Mathematical Reasoning
Topics:
- Counting without listing: the multiplication rule and complements
- Avoiding overflow, and taking answers modulo a number
- GCD, primes and divisibility in contest problems
- Greedy choices and how to check one is safe
- Turning a word problem into a precise mathematical model
- Testing a clever idea against a brute-force check
Projects built:
- A greedy and counting problem pack with a short argument for each solution
Practice: Counting and reasoning problems at 40 minutes each, with the idea written before the code
Weeks 13-14: Graphs
Topics:
- Representing graphs with adjacency lists
- Depth-first and breadth-first search
- Connected components and cycle checks
- Shortest paths with Dijkstra's algorithm
- Disjoint set union
- Spotting a graph hidden inside a problem that never says graph
Projects built:
- An original routing problem solved with breadth-first search and then with Dijkstra's algorithm
Practice: Graph problem set, with C++ versions written by students aiming at the CCO
Weeks 15-16: Dynamic Programming and the Hardest Questions
Topics:
- Defining a state and a transition
- Classic one-dimensional and grid dynamic programming
- Knapsack-style choices
- Combining techniques on the hardest question of a paper
- Partial marks on the final question: solving the smaller subtasks first
- C++ for speed: the standard library containers students will use most
Projects built:
- Solve one original dynamic programming problem top-down and bottom-up, and compare
Practice: A full Senior-style mock of original problems, timed at 3 hours
Assessment: Senior track assessment: a full timed mock with a written review of every question

---
PHASE 3: CONTEST SEASON (JANUARY AND FEBRUARY, WEEKS 17-24)

Full timed mocks, contest craft and the CCC itself on Wednesday, February 17, 2027, followed by upsolving and a plan for what comes next.

Month 5: Mock Contests and Contest Craft (Weeks 17-20)
Weeks 17-18: Pacing a 3-Hour, 5-Question Paper
Topics:
- Reading all five questions first and ordering the attempt
- Banking the early questions quickly and carefully
- Deciding when to move on from a stuck question
- Partial-mark strategy on the last two questions
- Testing before submitting, every time
- Using CEMC's past contests from the last 10 years as timed practice on CEMC's site
Projects built:
- Two full 3-hour mocks, each followed by a written pacing review
Practice: One full timed mock a week at the student's level, then a structured upsolve
Weeks 19-20: Upsolving and Weak-Spot Repair
Topics:
- Upsolving: solving every missed question unaided after the mock
- Reading the error log to find the real weak spot

- Targeted mini-lessons on each student's weakest topic
- Templates and checklists kept short and personal
- Choosing Python or C++ for the contest day, with honest advice for each student
- Making sure the school has the student on its CCC order

Projects built:
- A personal weak-spot plan with three topics, each with a problem set completed

Practice: Original problem sets on each student's weak topics

Assessment: Month 5 assessment: a full timed mock reviewed one to one with the teacher

Month 6: Final Mocks, the CCC and Next Steps (Weeks 21-24)

Weeks 21-22: Final Mocks

Topics:
- Final full mocks under contest timing
- Calm, routine starts: setup, reading and the first question
- Checking output format one last time
- What to do when the code works on samples but not on the judge
- Rest and practice balance in the last two weeks
- Reading CEMC's current CCC instructions together

Projects built:
- A final mock with a written contest-day plan

Practice: Light, steady practice: short problems that keep skills sharp

Weeks 23-24: Contest Week and After

Topics:
- Contest-day routine for the CCC on Wednesday, February 17, 2027
- After the contest: upsolving the paper once CEMC publishes it
- For Senior students: what the CCO is and how CEMC selects for it
- For Junior students: whether to aim for Senior next season
- Planning the next season of practice
- Presenting a favourite solution of the season to the class

Projects built:
- A next-season plan based on the error log and the mock history

Practice: Write a full explanation of your favourite solution of the season for a newer student

Assessment: Final assessment: the last mock contest result plus a short conversation about method choices

PROJECTS THE STUDENT BUILDS AND KEEPS

- Starting profile with track and language choice
- Counting, simulation and grid problem sets of original problems
- Slow-to-fast rewrites with timings
- Recursion and flood fill solutions (Junior track)
- Binary search, graph and dynamic programming solutions (Senior track)
- Full 3-hour mock contests with written pacing reviews
- A next-season plan built from the error log

Total projects: A season of solved original problems, reviewed CEMC past contests, an error log and a mock contest history

SKILLS THE STUDENT WILL HAVE AT THE END

- Reading contest statements precisely and handling input and output exactly
- Loops, conditions, strings, lists and counting
- Two-dimensional grids, recursion and flood fill
- Sorting, searching, prefix sums and two pointers
- Careful counting, number theory basics and greedy reasoning
- Graphs, shortest paths and dynamic programming (Senior track)
- Pacing a 3-hour, 5-question contest and earning partial marks

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, Junior and Senior in separate batches

Practice: A problem set after every class, with core and stretch problems

Review: Solutions reviewed with written comments, a timed contest every month and a monthly error log check

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with hints first and full solutions after a real attempt

Progress updates: A monthly progress note to parents with mock results and next steps

Free past contests: CEMC's past contests from the last 10 years, free with no registration, used as practice where CEMC publishes them

Free online grader: CEMC's free CCC Online Grader practice account, where students can try previous contests

Free python tutorials: Computer Science Circles, CEMC's free Python tutorials, a good warm-up for beginners

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- A second season of this course, moving from Junior to Senior or deeper into Senior
- Our Competitive Programming for Teens course for year-round contest training
- School computer science such as ICS3U and ICS4U, through our Ontario companion course
- University computer science, mathematics or engineering, with strong problem-solving habits
- Further contests and personal projects built on algorithmic thinking

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is the Canadian Computing Competition?
A: The CCC is a programming contest run by CEMC (the Centre for Education in Mathematics and Computing) at the University of Waterloo. It has a Junior and a Senior level. CEMC says the contest is available in English and French and is written in school, with participants writing online, individually.

Q: When is CCC 2027?
A: CEMC lists the 2027 CCC for North and South America on Wednesday, February 17, 2027, and the ordering deadline for schools as Thursday, February 11, 2027. Our season runs from September up to that date, and students who join later get a catch-up plan.

Q: Should my child write CCC Junior or CCC Senior?
A: CEMC describes Junior as for "any participant with beginner to intermediate programming skills" and Senior as for "any participant with intermediate to advanced programming skills", and says each participant can choose the best level "regardless of their grade". In the first two weeks we place each student on a track by skill, and we review the choice before the contest.

Q: How do we register for the CCC?
A: Through a school, not through us. CEMC says "It is not possible for students to individually register for a contest" and "all students must participate through their school." If a student's school does not take part, CEMC suggests that "students can look for another local school that will allow them to join as a participant". We remind families to check the school's order well before the deadline.

Q: Python or C++ for the CCC?
A: CEMC says "CCC participants may use C, C++, Python 2, Python 3, or Java." It also notes that "it may not always be possible to achieve a perfect score with a particular language choice (for example, Python or Java)". Python is a sensible choice for most Junior students. For Senior students aiming at the Canadian Computing Olympiad we teach C++, because CEMC states that "CCO participants must use C++."

Q: Do you use CCC past papers?
A: Yes, the way CEMC intends. CEMC publishes its past contests free, with no registration or fee, and students can practise on CEMC's free CCC Online Grader account. We set those contests as practice and review them in class, alongside our own original problems written for each topic. We do not copy CEMC's problems into our materials.

Q: What is the Canadian Computing Olympiad (CCO)?
A: CEMC says "Approximately 20 of the top official CCC Senior participants from schools in Canada are invited to the CCO at the University of Waterloo each spring", and that this invitation-only event "is also used to select Canada's teams for the International Olympiad in Informatics (IOI)". So the path is CCC Senior, then the CCO by invitation, then the IOI team. CEMC's page sets out who counts as an official participant, so read its current rules.

Q: Will this course get my child a distinction or a CCO invitation?
A: No course can promise that, and we do not. Results depend on the student, the paper and the day. What the course gives is a full season of structured practice, timed mocks, honest reviews and an error log, so progress is visible month by month.

Q: Does the CCC matter for university admission?
A: It is a good experience, not a requirement. CEMC states that "Participation in the Canadian Computing Competition (CCC) is not required for admission to the Faculty of Mathematics at the University of Waterloo." We encourage students to write it for the problem solving it builds.

Q: Does my child need coding experience?
A: For the Junior track, a little: writing simple programs with variables, loops and conditions in Python or C++. For the Senior track, a student should be confident with lists or arrays, functions and basic sorting. The first-week diagnostic places each student, and our other coding courses are a good first step for complete beginners.

Q: How are the classes run?

A: Live online classes, twice a week, with a real teacher, in a group of 15 to 20 students, a mini batch of up to 6, or 1-on-1. Junior and Senior students are taught in separate batches. Weekdays or weekends, flexible: choose your batch, or for a very personalised plan choose 1-on-1 at your convenience, weekend mornings or weekdays. Current fees for your region are shown in the pricing section of this page.

Q: Can we try a class first?
A: Yes. Book a free demo class. The teacher works through a short original problem with your child, suggests the Junior or Senior track, and recommends group, mini batch or 1-on-1 classes. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ccc-junior-and-senior-preparation-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 39 OF 137: CODEX & CLAUDE CODE FOR TEENS: AI CODING AGENTS COURSE
==========

Course link: https://learn.modernagecoders.com/courses/codex-and-claude-code-ai-coding-agents-course-for-teens
Subject area: AI Coding Agents
Designed for: Teens
Who can join: Beginner-friendly (ages 13-18) to confident AI-agent builder
Course length: 24 classes (12 weeks · 2 classes/week)
Time needed: 3-4 hours/week recommended
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.
Note: this course also needs the student to have their own Claude and ChatGPT subscriptions, because the whole course is hands on with those tools. Say this before enrolling anyone, so it is never a surprise later.

WHAT THIS COURSE IS

Live online Codex + Claude Code course for teens 13-18. 24 mentor-led classes: install the CLIs, plan mode, MCP, subagents, then ship a real app. Free demo class.

In one line: Don't just learn to code, learn to direct the two most powerful AI coding agents on Earth, and build real apps in 24 classes.

ABOUT THE PROGRAMME

This is the most complete teen-friendly course on the two AI coding agents every developer is talking about in 2026: OpenAI Codex and Anthropic Claude Code. In 24 carefully sequenced, project-based classes you will install both tools, learn to give them instructions safely, and watch them write, fix, and ship real code with you. You start from zero, no coding experience required, and finish by building and deploying a real app end-to-end with both agents helping you. Every class is taught live by a mentor in a small batch or 1-on-1, never a pre-recorded video, with live commands, worked examples, a hands-on project, practice, and a checkpoint. You learn the current tools as they exist today: Claude Code v2.1.x running on Anthropic's Claude 5 models (Sonnet 5 by default, with Fable 5 and Opus 4.8 selectable), and the OpenAI Codex CLI running on the GPT-5.6 family (Sol, Terra, and Luna). One important requirement: to run the agents yourself, every student must have an active Claude subscription (Pro or higher) for Claude Code and a ChatGPT subscription (Plus or higher) for Codex. These are purchased directly from Anthropic and OpenAI, are separate from the course fee, and we help you choose the right plans in your free demo class.

WHAT MAKES THIS COURSE DIFFERENT

- Two agents, one course: most courses teach one tool; you graduate fluent in BOTH Codex and Claude Code and know when to use which.
- 100% hands-on: you run real commands from Class 2. No watching, all building.
- Built for teens: friendly pace, fun projects (a website, a game, a Discord-style bot), and a portfolio you can show schools and colleges.
- Living Syllabus: the curriculum is updated continuously as OpenAI Codex and Anthropic Claude Code release new features, models, and capabilities. You always learn the latest.
- Safety-first: you learn approval modes, sandboxes, and responsible AI use before you ever let an agent run wild.
- Real tools, not a sandbox: students bring their own Claude and ChatGPT subscriptions, so every command you practice runs on the same production agents professional developers use

every day.
- Real outcomes: finish with a deployed app, a GitHub profile with real pull requests, and a recognized certificate.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop (Windows, Mac, or Linux) with internet: that's the main hardware requirement
Coding experience: None needed! We start from absolute zero. If you've done a little Scratch or Python, even better
Accounts: REQUIRED: an active Claude subscription (Pro or higher) for Claude Code and a ChatGPT subscription (Plus or higher) for Codex. These are purchased directly from Anthropic and OpenAI and are not included in the course fee. You also need a free GitHub account. We guide you through every setup step and help you pick the most student-friendly plans
Mindset: Curiosity and willingness to experiment. You'll learn by building, not memorizing

WHO THIS COURSE IS FOR

Teens: Ages 13-18 who want to build real apps fast and learn the AI tools professional developers use in 2026
Beginners: Total beginners: no prior coding required; the agents help you while you learn
Young builders: Teens with project ideas (games, websites, bots) who want to actually ship them
Future cs students: Students aiming for computer science, who want a standout portfolio and future-ready skills
Scratch python grads: Kids who've outgrown Scratch or done basic Python and want the next big step

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS OF AI CODING AGENTS

Understand what AI coding agents are, install both Codex and Claude Code, learn to keep them safe with approval modes, and complete your very first real coding task.

Class 1: What Are AI Coding Agents? (Codex vs Claude Code)
Topics:
- What 'agentic coding' means: an AI that can read your files, write code, run commands, and fix its own mistakes, not just autocomplete
- Meet OpenAI Codex: an AI software-engineering agent that lives in your terminal, your editor, the cloud, and ChatGPT
- Meet Anthropic Claude Code: an agentic coding tool that runs in the terminal, VS Code/JetBrains, desktop, web, and mobile
- How an agent loop works: you give a goal it plans it edits/runs it checks it reports back
- Codex vs Claude Code at a glance: both are excellent; Codex leans into cloud parallel tasks + ChatGPT, Claude Code leans into deep terminal control + subagents
- The 2026 models you'll use: Claude Code on the Claude 5 family (Sonnet 5 default, Fable 5 and Opus 4.8 above it); Codex on the GPT-5.6 family (Sol, Terra, Luna)
- Why learning to DIRECT an agent is now as important as writing code yourself
Projects built:
- Make a one-page 'Codex vs Claude Code' comparison sheet in your own words, listing 3 things each is great at
Practice: Watch each tool's homepage demo and write down 5 things you saw an agent do that surprised you.
Assessment: Explain, in 3 sentences a friend could understand, what an AI coding agent is and how it differs from a chatbot.

Class 2: Installing & Setting Up Claude Code
Topics:
- Install on Windows (PowerShell): irm https://claude.ai/install.ps1 | iex. On Mac/Linux: curl -fsSL https://claude.ai/install.sh | bash
- Sign in with your plan: claude auth login (Claude Pro or Max works great for learners); check with claude auth status
- Start your first session: open a project folder and type claude
- Meet the REPL (the chat prompt): type a request in plain English and press Enter
- Type / to see the slash-command menu; try /help and /status
- Understand the model: by default you're on Sonnet 5, Anthropic's balanced Claude 5 model with a native 1M-token context: see it with /model
- Exit safely and resume later: your session is saved
Projects built:
- Install Claude Code, start a session in an empty folder, and ask it to create a hello.html page that says your name
Practice: Ask Claude Code to explain what it just created, then ask it to change the background color.
Assessment: Share a screenshot of Claude Code running and your hello.html opening in a browser.

Class 3: Installing & Using the OpenAI Codex CLI
Topics:
- What the Codex CLI is: an open-source AI coding agent in your terminal
- Install it: npm install -g @openai/codex (needs Node.js 18+). Careful: the plain 'codex' package is something else; always use the @openai/ scope
- Sign in two ways: 'Sign in with ChatGPT' (uses your ChatGPT Plus/Pro plan) OR an API key
- Start it: type codex to open the interactive TUI; run codex doctor to check everything is healthy
- Pick your model and thinking level with /model (low up to ultra); the 2026 lineup is the GPT-5.6 family: Sol for complex work, Terra the all-rounder, Luna for quick tasks
- Give your first instruction in plain English and watch Codex plan and edit
- Preview the AGENTS.md idea: a file that teaches Codex your project's rules (Class 10)

Projects built:
- Install Codex, run codex doctor, then have Codex add a new section to your hello.html from Class 2

Practice: Ask Codex to explain a file line-by-line, then ask it to add helpful comments.
Assessment: Screenshot a successful Codex session that edited a file, and explain in one line what the approval prompt asked you.

Class 4: Staying in Control with Approval & Permission Modes
Topics:
- Why approval modes matter: you decide how much the agent can do without asking
- Claude Code modes (cycle with Shift+Tab): default (asks each time), acceptEdits (auto-accepts file edits), plan (read-only, proposes first), auto (a classifier auto-approves safe actions)
- Set a mode on launch: claude --permission-mode plan; manage rules with /permissions
- Codex modes: read-only (look but don't touch), workspace-write (the safe default: edit inside your project), danger-full-access (no guardrails)
- Switch Codex modes anytime with /approvals; the scary bypass flag --dangerously-bypass-approvals-and-sandbox (never use casually)
- Reading an approval prompt: what the agent wants to run and why, before you say yes
- Good habit: start in plan/read-only, review, then allow edits

Projects built:
- Run the same small task ('add a footer to hello.html') in plan mode first, review the plan, then approve and apply it

Practice: Deliberately deny an action in each tool and read what changes.
Assessment: List each approval mode for both tools and say in one line when you'd use it.

Class 5: Your First Real Task in Both Agents
Topics:
- Pick a tiny starter project (a to-do list web page) and open it in your terminal
- Give a clear goal: 'Add a button that clears all completed to-dos'
- Let Claude Code plan it, show you a diff, and apply the change after you approve
- Repeat the same goal with Codex and compare how each one approached it
- Reading a diff: green = added lines, red = removed lines: how to check the change is what you wanted
- Asking for a fix: 'That button is the wrong color, make it red': iterating with the agent
- Saving your work: ask the agent to run the app so you can see it work

Projects built:
- Add two new features to the to-do app, one with Claude Code, one with Codex, and note which felt easier

Practice: Intentionally introduce a bug, then ask the agent to find and fix it.
Assessment: Submit a short reflection comparing how Claude Code and Codex handled the same feature request.

PHASE 2: YOUR DAILY CODING WORKFLOW

Learn the everyday moves: slash commands, planning before building, editing and diffs, running tests and git, teaching the agent your project, and undoing mistakes.

Class 6: Slash Commands & Steering Your Session
Topics:
- Claude Code essentials: /help, /clear (start fresh), /compact (shrink a long chat), /context (see what the agent remembers), /model
- Codex essentials: /model, /approvals, /review, /status, and the @ picker to mention files
- Why /clear and /compact matter: keeping the agent focused and fast
- Mentioning a file so the agent reads exactly what you mean (@-mentions)
- Checking your usage and limits with /usage (Claude) and /status (Codex)
- Getting unstuck: asking the agent 'what are you doing and why?'
- Keyboard tips that make you faster in each TUI

Projects built:
- Create a personal cheat-sheet of your 10 most-used slash commands across both tools

Practice: Run a long conversation, then use /compact (Claude) and start a clean Codex session: notice the difference.
Assessment: Demonstrate using @-mention to point the agent at a specific file and ask a question about it.

Class 7: Plan Mode & Goal Mode, Think Before You Build

Topics:
- Claude Code Plan Mode: enter with /plan: the agent researches and proposes a plan WITHOUT editing
- Why plan first: catches misunderstandings before any code changes
- Goal Mode (/goal): the agent keeps working until a goal you define is actually met
- Codex /plan and /goal: the same idea on the Codex side (Goal mode is on by default in 2026)
- Writing a good goal: clear, checkable, one outcome ('the login form validates email and shows an error')
- Reviewing and editing the agent's plan before you approve it
- When to skip planning (tiny edits) vs always plan (anything touching multiple files)

Projects built:
- Use Plan Mode to design a 'dark mode toggle' for a web page, review the plan, then let the agent build it

Practice: Give a vague goal on purpose and watch how planning surfaces the missing details.

Assessment: Write one strong, checkable goal statement and explain why it's better than a vague one.

Class 8: Reading, Editing & Reviewing Code Changes

Topics:
- How agents edit: they read the file, propose precise changes, and show you a diff
- Reading multi-file changes: following a feature across several files
- Accepting, rejecting, or asking for tweaks on each change
- Claude Code /diff to see what changed; reviewing in VS Code with the extension
- Asking the agent to explain a change you don't understand
- Keeping changes small: one feature at a time is easier to review
- Spotting when an agent edited too much and reining it in

Projects built:
- Have an agent refactor a messy function and walk you through the diff line-by-line

Practice: Reject a proposed change and ask for a different approach.

Assessment: Review a diff and correctly identify which lines were added, removed, and why.

Class 9: Running Commands, Tests & Git from the Agent

Topics:
- Letting the agent run your app: 'start the dev server and tell me the URL'
- Running tests through the agent and reading pass/fail output together
- Basic git with the agent: 'commit this with a clear message', 'show me what changed'
- Claude Code's Bash tool and Codex running commands inside the workspace sandbox
- Approving commands safely: reading the command before it runs
- Asking the agent to fix a failing test until it passes
- Good commit hygiene: small, frequent commits with clear messages

Projects built:
- Build a tiny feature, have the agent write a test for it, run the test, and commit when it passes

Practice: Break a test on purpose and ask the agent to make it green again.

Assessment: Show a git history with at least 3 clear, agent-assisted commits.

Class 10: Project Memory, CLAUDE.md & AGENTS.md

Topics:
- Why memory matters: tell the agent your project rules once, not every time
- Claude Code: run /init to auto-create a CLAUDE.md; it loads automatically each session
- What to put in CLAUDE.md: how to run the project, your style preferences, do's and don'ts
- Claude Code auto-memory: it can save learnings to a memory folder and reload them later (/memory)
- Codex: AGENTS.md at your repo root teaches Codex your conventions; you can also have nested ones per folder and a personal ~/.codex/AGENTS.md
- A 'Review guidelines' section in AGENTS.md so Codex reviews PRs your way
- Keeping memory short and useful (aim under ~200 lines)

Projects built:
- Write a CLAUDE.md and an AGENTS.md for your to-do app describing how to run it and your code style

Practice: Add a rule like 'always use 2-space indentation' and watch the agent follow it.

Assessment: Submit a CLAUDE.md and AGENTS.md and explain one rule in each.

Class 11: Checkpoints, Rewind, Undo & Resuming Sessions

Topics:
- Claude Code checkpointing: it saves code state before edits automatically
- Rewind with /rewind (or press Esc twice): restore code, conversation, or both
- Limitation to know: rewind tracks edits made by the agent's file tools, not files deleted by shell commands
- Resuming: claude --continue (last chat) and claude --resume to pick a past session
- Codex: resume past sessions and search your local conversation history
- Branching a conversation to try a different approach without losing your place
- Building the safety habit: checkpoint, experiment, rewind if needed

Projects built:
- Make a risky change, dislike it, and use /rewind to go back cleanly

Practice: Resume yesterday's session and continue where you left off.

Assessment: Demonstrate a rewind that restores code without losing your earlier

conversation.

PHASE 3: EXTENDING & CONNECTING YOUR AGENTS

Supercharge both agents: connect external tools with MCP, build skills and plugins, create subagents, automate with hooks, choose models and reasoning effort, and code from screenshots.

Class 12: Connecting External Tools & Data with MCP

Topics:

- What MCP (Model Context Protocol) is: a standard way to plug tools and data into your agent
- Claude Code: add a server with claude mcp add --transport http <name> <url>; manage with /mcp and claude mcp list
- Real examples: connect a docs server, a database, or a design tool so the agent can use it
- Codex MCP: configure servers under [mcp_servers.<name>] in ~/.codex/config.toml (stdio or HTTP)
- MCP scopes in Claude Code: local, project (shared via the repo), and user
- Safety with MCP: only connect servers you trust; agents warn about untrusted tools
- Seeing the new tools appear and asking the agent to use one

Projects built:

- Connect one MCP server to each agent and have the agent use it to fetch real information

Practice: List the new tools an MCP server added and try one.

Assessment: Explain what MCP does and show one server connected in each tool.

Class 13: Skills, Custom Commands & Codex Plugins

Topics:

- Claude Code Skills: reusable mini-instructions the agent can run; manage with /skills
- Create a custom slash command by adding a file in .claude/commands/ (e.g. /deploy)
- How skills load only when needed, keeping things fast
- Codex plugins, profiles, and the plugin marketplace for sharing setups
- Codex profiles: switch settings/models per project with codex --profile <name>
- When to make a skill vs just typing the instruction each time
- Sharing your skills/plugins with classmates

Projects built:

- Create a Claude Code custom command (like /newpage that scaffolds an HTML page) and a Codex profile for your project

Practice: Run your custom command twice and tweak it to be more useful.

Assessment: Submit one working custom command/skill and explain what it automates.

Class 14: Subagents, Your Specialist Team

Topics:

- What a subagent is: a helper with its own context, tools, and job
- Claude Code built-in subagents: Explore (fast read-only search) and Plan (research before editing)
- Create your own: a Markdown file in .claude/agents/ with name + description + a system prompt
- Give a subagent only the tools it needs (e.g., a 'reviewer' that can only read)
- Manage and generate agents with /agents
- Why subagents keep the main chat clean and focused (since mid-2026 they run in the background by default, so you keep working while they finish)
- Example: a 'test-writer' subagent that writes tests while you keep building

Projects built:

- Create a 'code-reviewer' subagent and have it review your to-do app

Practice: Compare answers from the Explore subagent vs asking the main agent directly.

Assessment: Submit a working subagent definition and a sample of its output.

Class 15: Hooks & Automation

Topics:

- What hooks are: code that runs automatically at certain moments (events)
- Claude Code events you'll meet: PreToolUse, PostToolUse, SessionStart, UserPromptSubmit, Stop
- Where hooks live: settings JSON (.claude/settings.json) with a matcher + a command
- A simple, safe hook: auto-format code after every edit, or block a dangerous command
- Reading hook results as feedback to the agent
- Turning hooks on/off and viewing them with /hooks
- Automation mindset: let routine steps happen for you, every time

Projects built:

- Add a PostToolUse hook that runs a formatter after the agent edits a file

Practice: Write a PreToolUse rule that blocks an obviously dangerous command and test it.

Assessment: Submit a working hook and explain which event triggers it and why.

Class 16: Models & Reasoning Effort

Topics:

- Claude Code models: Fable 5 (most capable, the new Claude 5 flagship tier), Opus 4.8, Sonnet 5 (the default), Haiku 4.5 (fast/cheap); switch with /model
- Reasoning effort levels: low, medium, high, xhigh: more effort = deeper thinking (set with /effort)
- The 'ultrathink' trick: add it to a prompt for a one-off deeper think
- Codex models: GPT-5.6 Sol, Terra, and Luna; pick model + reasoning level with /model (low up to ultra)

- Big context: Sonnet 5 has a native 1M-token window, great for large projects
- Choosing wisely: fast model for simple edits, top model + high effort for hard problems
- Cost & limits awareness for learners on student-friendly plans

Projects built:
- Solve the same tricky bug on a low effort setting and a high effort setting and compare results

Practice: Switch models mid-task and note speed vs quality trade-offs.

Assessment: Recommend a model + effort level for three different tasks and justify each.

Class 17: Multimodal Coding from Screenshots & Images

Topics:
- Why images help: show the agent a design or a bug instead of describing it
- Drag-and-drop a screenshot into the prompt so the agent can see it
- Codex Appshots (Mac): capture an app window as context for the agent
- Ask the agent to recreate a UI from a picture of a design
- Debugging from a screenshot of an error message
- Combining text + image: 'make my page look like THIS image'
- Limits: both agents read images; Codex added audio input in 2026; video is still not supported

Projects built:
- Give the agent a screenshot of a website you like and have it build a similar layout

Practice: Screenshot a broken page and ask the agent to diagnose it from the image.

Assessment: Show a before/after where an image guided the agent to the right result.

PHASE 4: CLOUD, COLLABORATION & SCALE

Go beyond your laptop: delegate tasks to Codex Cloud, use Claude Code on web and mobile, get automatic GitHub code reviews, and run several agents at once.

Class 18: Codex Cloud, Delegating Tasks That Run for You

Topics:
- What Codex Cloud is: agents that work on tasks in the background, in their own safe sandboxes
- Get started at chatgpt.com/codex and connect a GitHub account
- Run tasks in parallel: kick off several at once and let them cook
- Each finished task comes back as a pull request you can review
- Setting up a cloud environment: which repo, setup steps, and internet access
- Reviewing the agent's work before merging anything
- When cloud beats local: long jobs, or working on a repo you don't have on your laptop

Projects built:
- Delegate two small tasks to Codex Cloud at once and review both resulting pull requests

Practice: Give a cloud task a clear scope and read the PR description it writes.

Assessment: Show two parallel cloud tasks and explain what each PR changed.

Class 19: Claude Code Everywhere, on Web, Desktop, and Mobile

Topics:
- Claude Code on the web at claude.ai/code for long-running and parallel tasks
- The desktop app: review diffs visually and run multiple sessions side-by-side
- Background agents: detach a session with /background and check it later
- Remote control & mobile: steer a session from your phone
- Pulling a web/remote session into your terminal to finish locally
- Choosing the right surface for the job
- Keeping all your sessions organized

Projects built:
- Start a task on the web, then continue it on the desktop app or in your terminal

Practice: Send one agent to the background and keep working in the foreground.

Assessment: Demonstrate moving the same task across two surfaces (web terminal/desktop).

Class 20: GitHub Integration & Automatic Code Review

Topics:
- Connecting GitHub so agents can open and review pull requests
- Codex code review: comment @codex review on a PR, or turn on automatic reviews for every PR
- Adding focus: @codex review for security issues
- Claude Code: /code-review on your branch, with --comment to post inline comments on GitHub
- Letting an agent fix review findings: @codex fix the issues
- GitHub Actions/CI: agents that review or triage automatically
- Reading a code review like a pro and acting on it

Projects built:
- Open a PR, get an AI code review from Codex and from Claude Code, and apply the fixes

Practice: Write a tiny risky change and see what the reviewer flags.

Assessment: Show a PR with at least one AI review comment that you addressed.

Class 21: Parallel Agents, Worktrees & /batch

Topics:
- Why parallel: split a big job into pieces that run at the same time
- Git worktrees: isolated copies of your repo so agents don't collide (claude -w <name>)
- Claude Code /batch: break a large change into many units, each handled by its own background agent in its own worktree
- Agent teams: multiple coordinating sessions

- Codex Cloud parallelism revisited: many tasks, many PRs
- Keeping parallel work organized and reviewing each piece
- When NOT to parallelize (small, tightly-linked changes)

Projects built:
- Use /batch (or several Codex Cloud tasks) to apply the same small improvement across multiple files

Practice: Create a git worktree and run an agent in it without touching your main branch.
Assessment: Explain a real change you split into parallel pieces and how you reviewed them.

PHASE 5: BUILD, SECURE & SHIP

Bring it together: automate with the SDKs, code safely with sandboxes and permissions, and ship a real capstone app, then earn your certificate.

Class 22: Building With the SDKs (Agent SDK & Codex SDK)

Topics:
- Headless mode: run an agent without the chat UI: Claude Code claude -p "task" and Codex codex exec
- Structured output: ask for clean JSON back (Codex --output-schema) to use in your own programs
- The Claude Agent SDK: build your own mini-agent in TypeScript or Python using Claude Code's engine
- The Codex SDK (TypeScript & Python): start a thread, run a prompt, resume it
- A simple automation: a script that asks the agent to do a chore and reports the result
- Piping: cat error.log | claude -p "explain this error"
- Where this leads: your own custom AI tools

Projects built:
- Write a tiny script that uses headless mode to summarize what changed in your project today

Practice: Pipe a file into claude -p and read the structured answer.
Assessment: Submit a small working automation built with claude -p or codex exec.

Class 23: Security, Sandboxing & Responsible AI Coding

Topics:
- Permission rules: allow / ask / deny specific actions before any tool runs
- Claude Code Bash sandbox (/sandbox): OS-level isolation so commands can't roam your whole computer
- Codex sandboxes: Apple Seatbelt (Mac), bubblewrap (Linux), Windows Sandbox, and the three approval modes
- Protecting secrets: never let an agent read or commit passwords and API keys
- Prompt injection: why you must be careful with untrusted web content and MCP servers
- Reviewing before you run: the golden rule of agent safety
- Being a responsible AI developer: understand the code the agent writes; you're accountable for it

Projects built:
- Configure permission rules and a sandbox setting in each tool and test that a blocked action is actually blocked

Practice: Add a deny rule for a dangerous command and confirm the agent respects it.
Assessment: Write a short 'safe AI coding checklist' you'll follow on every project.

Class 24: Capstone, Ship a Real App + Certification

Topics:
- Choose your build: a personal website, a simple game, a quiz app, or a chat-style bot
- Plan it with Plan/Goal mode, write your CLAUDE.md and AGENTS.md
- Build with both agents: use each where it shines
- Add a test, get an AI code review, and fix the findings
- Deploy it live (e.g., a free static host) so you have a real URL
- Polish your GitHub: real commits, a clean README, your PRs
- Present your project and earn your Modern Age Coders AI Coding Agents certificate

Projects built:
- Ship a complete, deployed app built and reviewed with Codex and Claude Code, with a public URL and a clean GitHub repo

Practice: Demo your app to the class and walk through how the agents helped you build it.
Assessment: Final capstone: a deployed app + GitHub repo + a short write-up; pass earns the certificate.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: a Codex vs Claude Code comparison sheet, your first hello.html page built with Claude Code and extended with Codex, a plan-first footer change, and two new to-do app features (one built with each agent)
- Phase 2: a personal slash-command cheat-sheet, a dark mode toggle designed in Plan Mode, a messy function refactored and reviewed line by line, a tested and committed feature, CLAUDE.md and AGENTS.md for your to-do app, and a clean /rewind recovery
- Phase 3: an MCP server connected to each agent, a custom /newpage command plus a Codex profile, a code-reviewer subagent, an auto-format hook, a low vs high effort bug-fix comparison, and a website layout rebuilt from a screenshot
- Phase 4: two parallel Codex Cloud tasks reviewed as pull requests, one task moved across web

and desktop or terminal, a pull request improved with AI code review from both tools, and a /batch improvement applied across multiple files
- Phase 5: a headless-mode automation script, a tested safety setup in both tools, and a deployed capstone app (a website, game, quiz app, or bot) with a public URL
- Total: 24 hands-on projects, one per class, finishing with a deployed capstone app

Total projects: 24 hands-on projects, one per class, finishing with a deployed capstone app

SKILLS THE STUDENT WILL HAVE AT THE END

- AI Coding Agents: Codex CLI, Claude Code, agent loops, plan and goal mode, safe approval modes
- Prompting for Code: clear goal statements, plan-first building, iterating on diffs, debugging with the agent
- Extending Agents: MCP servers, custom slash commands and skills, subagents, hooks, Codex profiles
- Terminal Skills: installing CLIs, running sessions, slash commands, reading command output with confidence
- Version Control: git commits, branches, worktrees, pull requests on GitHub
- Reviewing AI Code: reading diffs, spotting over-broad changes, acting on AI code review feedback
- Models and Effort: choosing between Fable 5, Opus 4.8, Sonnet 5, Haiku 4.5 and the GPT-5.6 family (Sol, Terra, Luna), and picking the right effort level
- Shipping Projects: building, testing, deploying, and presenting a real app with a public URL

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, about 1 hour each, across 12 weeks
Hands on agent work: 1-1.5 hours practicing real commands in Codex and Claude Code
Project time: 1-1.5 hours building each class project on your own computer
Review and doubts: 30-60 minutes reviewing what you built and clearing doubts before the next class
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for install issues, errors, and questions
Small batches: Small batches, with mini-batch and 1-on-1 options, so every teen gets individual attention
Mentor guidance: Live guidance from mentors who use these agents daily
Project feedback: Feedback on every class project and a personal capstone review before certification
Living syllabus: Curriculum updated as OpenAI Codex and Claude Code ship new features, at no extra cost

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Junior/Intern Developer using AI agents to ship faster
- Freelance mini-projects (websites, simple apps) for local businesses
- Strong CS-college application portfolio with deployed projects
- Hackathon competitor who can build a working prototype in a weekend
- Future AI-assisted Software Engineer, Web Developer, or App Developer

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need a Claude or ChatGPT subscription to take this course?
A: Yes, this is a course requirement. To actually run the agents in class, every student must have an active Claude subscription (Pro or higher) for Claude Code and a ChatGPT subscription (Plus or higher) for Codex. You buy these directly from Anthropic and OpenAI; they are separate from the course fee. We help every family choose the most cost-effective plans during the free demo class, and setup is covered step by step in Classes 2 and 3. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final project defended live, plus a written assessment. Passing earns the certificate; falling

short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Is OpenAI Codex free?
A: The Codex CLI is open-source and free to install, but running it requires a paid plan: you sign in with a ChatGPT subscription (Plus, Pro, Business, Edu, or Enterprise) which includes Codex usage, or you use an OpenAI API key billed per use. In 2026 usage runs on rolling limits that reset every few hours. For this course a ChatGPT plan (Plus or higher) is required, along with a Claude plan (Pro or higher) for the Claude Code half. We help you choose the most student-friendly setup.

Q: Do I need to know how to code before taking this course?
A: No. This course is designed for complete beginners aged 13-18. You start from zero in Class 1, and the AI agents help you write and understand code as you go. By the end you'll be shipping real apps.

Q: What is the best way to learn Claude Code and Codex?
A: Hands-on, with live guidance. These agents change monthly, so recorded tutorials go stale fast, and reading documentation alone does not build the judgment to review AI-written code safely. The most effective path is structured live classes where you run real commands in every session, build real projects, and have a mentor correct your habits as you go, which is exactly how this 24-class course is built. Modern Age Coders has taught 10,000+ students since 2020 and is rated 4.9 across 547 Google reviews.

Q: Codex vs Claude Code: which should a teen learn first?
A: Both! That's the whole point of this course. They're the two best AI coding agents in 2026, and they shine in different ways: Codex is fantastic for cloud parallel tasks and the ChatGPT ecosystem, while Claude Code offers deep terminal control, subagents, and hooks. We teach you both and exactly when to use each.

Q: Will this course go out of date as the tools change?
A: No, this is our Living Syllabus promise. OpenAI Codex and Anthropic Claude Code ship new features and models constantly: mid-2026 alone brought Sonnet 5 as Claude Code's new default model with a native 1M-token context, and Codex's move to the GPT-5.6 family (Sol, Terra, Luna). We continuously update the curriculum so enrolled students always learn the latest features, at no extra cost.

Q: What can my teen build by the end of the course?
A: A real, deployed app with a public URL, for example a personal website, a browser game, a quiz app, or a chat-style bot, plus a GitHub profile with genuine commits, pull requests, and AI code reviews. It's a portfolio piece they can show schools and colleges.

Q: Is it safe to let an AI agent run code on our computer?
A: Yes, when you use it correctly, and we teach exactly that. You'll learn approval modes, OS-level sandboxes, and permission rules so the agent only does what you allow. An entire class (Class 23) is dedicated to security and responsible AI coding.

Q: What computer and accounts do we need?
A: Any modern Windows, Mac, or Linux computer with internet. You'll need a free GitHub account, plus the two required subscriptions: Claude (Pro or higher) for Claude Code and ChatGPT (Plus or higher) for Codex, purchased directly from Anthropic and OpenAI. We walk you through every install and login step in Classes 2 and 3.

Q: How long is the course and how is it scheduled?
A: 24 classes, typically two classes a week over about 12 weeks, with 3-4 hours of practice per week recommended. Group, mini-batch, and 1-on-1 options are available.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/codex-and-claude-code-ai-coding-agents-course-for-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 40 OF 137: CODEX & CLAUDE CODE: AI CODING AGENTS FOR PROFESSIONALS
==========

Course link: https://learn.modernagecoders.com/courses/codex-and-claude-code-ai-coding-agents-masterclass-for-adults-professionals
Subject area: AI Coding Agents
Designed for: College students
Who can join: Beginner to advanced, for working professionals, career switchers & developers
Course length: 24 classes (12 weeks · 2 classes/week)
Time needed: 4-6 hours/week recommended
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a

month. Each class is one hour.
Note: this course also needs the student to have their own Claude and ChatGPT subscriptions, because the whole course is hands on with those tools. Say this before enrolling anyone, so it is never a surprise later.

WHAT THIS COURSE IS

Live online Codex + Claude Code masterclass for professionals. 24 mentor-led classes: CLIs, MCP, subagents, hooks, both SDKs and a production capstone.

In one line: Stop typing every line. Learn to direct the two most powerful AI coding agents and ship more, faster, safely, in 24 classes.

ABOUT THE PROGRAMME

This is the definitive professional course on the two agentic coding tools reshaping software work in 2026: OpenAI Codex and Anthropic Claude Code. Across 24 in-depth, lab-driven classes you'll go from first install to automating real engineering workflows: building features, refactoring legacy code, fixing flaky tests, reviewing PRs at scale, and orchestrating parallel agents. Whether you're a developer leveling up, a career switcher entering tech, or a professional who wants to 10x routine work, you'll learn both tools at the depth a job demands. You learn the current stack: Claude Code v2.1.x on Anthropic's Claude 5 family (Sonnet 5 default with native 1M-token context; Fable 5 and Opus 4.8 for the hardest work), and the OpenAI Codex CLI/Cloud on the GPT-5.6 family (Sol, Terra, Luna) with reasoning levels up to ultra. Every class is live with a mentor in a small batch or 1-on-1, hands-on with real commands, config, and a deliverable. One important requirement: you must have your own active Claude subscription (Pro or higher) for Claude Code and ChatGPT subscription (Plus or higher) for Codex, or equivalent API access through your employer. Subscriptions are purchased directly from Anthropic and OpenAI, are separate from the course fee, and we help you pick the most cost-effective setup in your free demo.

WHAT MAKES THIS COURSE DIFFERENT

- Two agents, professionally: graduate fluent in BOTH Codex and Claude Code, and able to choose the right one per task and team.
- Workplace-grade labs: refactor legacy modules, fix CI, automate releases, review PRs at scale; not toy examples.
- Goes all the way to the SDKs: build your own automations and internal tooling with the Claude Agent SDK and Codex SDK.
- Living Syllabus: continuously updated as OpenAI Codex and Anthropic Claude Code ship new features, models, and capabilities. You always learn the current product.
- Security & governance built in: permission rules, OS sandboxes, secrets handling, and prompt-injection defense for real teams.
- Real production tools, not a sandbox: you train on your own Claude and ChatGPT subscriptions, so every workflow you learn transfers directly to your day job.
- Outcome-focused: finish with a production-grade capstone, reusable automations, and a portfolio of AI-assisted engineering work.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop/desktop (Windows, Mac, or Linux) with internet and admin rights to install CLIs
Coding experience: Helpful but not required: beginners and career switchers are welcome; we start from install and ramp quickly
Accounts: REQUIRED: an active Claude subscription (Pro or higher) for Claude Code and a ChatGPT subscription (Plus or higher) for Codex, or equivalent API keys (for example through your employer, Bedrock, or Vertex AI). These are purchased directly from Anthropic and OpenAI and are not included in the course fee. A GitHub account is also required. We cover setup and provider options in the first classes
Tools: Comfort with a terminal develops during the course; basic familiarity with git is a plus

WHO THIS COURSE IS FOR

Working professionals: Engineers and analysts who want to ship more with AI agents and automate routine work
Career switchers: People entering tech who want a modern, AI-native skill set that employers increasingly expect
Developers: Developers leveling up from autocomplete to true agentic workflows (MCP, subagents, SDKs)
Tech leads: Leads introducing safe, governed agentic coding practices to a team
Founders freelancers: Founders and freelancers who need to build and ship quickly with a small team

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS OF AI CODING AGENTS

Understand the agentic-coding landscape, install and configure both Codex and Claude Code,
master the approval/permission models, and complete your first real engineering task.

  Class 1: The Agentic Coding Landscape (Codex vs Claude Code)
    Topics:
      - Agentic coding defined: an autonomous loop that reads context, plans, edits, runs commands,
        verifies, and iterates
      - OpenAI Codex as a product family: the CLI, Codex Cloud, the IDE extension, the
        ChatGPT/desktop/mobile apps, and the Codex SDK
      - Anthropic Claude Code surfaces: terminal, VS Code/JetBrains, desktop, web (claude.ai/code),
        mobile, Slack, and CI
      - Where each excels: Codex for cloud parallelism + GitHub-native review + ChatGPT ecosystem;
        Claude Code for deep terminal control, subagents, hooks, and skills
      - The 2026 model backends: Claude Code on the Claude 5 family (Sonnet 5 default, Fable 5 and
        Opus 4.8 above it); Codex on the GPT-5.6 family (Sol, Terra, Luna)
      - How agentic tools change team workflows: review-centric, smaller PRs, faster iteration
      - Building a mental model for evaluating any new agentic tool
    Projects built:
      - Write a one-page internal brief: 'Codex vs Claude Code: where we'd use each' for a
        hypothetical team
    Practice: Map three tasks from your own work to the tool you'd choose and justify it.
    Assessment: Produce a clear comparison matrix of Codex and Claude Code across 6
    dimensions.

  Class 2: Installing & Configuring Claude Code
    Topics:
      - Install: macOS/Linux curl -fsSL https://claude.ai/install.sh | bash; Windows irm
        https://claude.ai/install.ps1 | iex; Homebrew/WinGet alternatives
      - Authenticate: claude auth login (subscription) or ANTHROPIC_API_KEY; claude auth status;
        claude setup-token for CI
      - Enterprise providers: Amazon Bedrock, Google Vertex AI, Azure AI Foundry via env flags
        (CLAUDE_CODE_USE_BEDROCK, etc.)
      - First session and the REPL; the slash-command menu; /status, /usage
      - Settings hierarchy: ~/.claude/settings.json (user), .claude/settings.json (project),
        .claude/settings.local.json (gitignored)
      - Choosing your model with /model (Sonnet 5 is the current default; Fable 5, Opus 4.8 and Haiku
        4.5 are selectable)
      - IDE integration: install the VS Code/JetBrains extension and open Claude Code in your editor
    Projects built:
      - Install Claude Code, configure a project .claude/settings.json, and run your first task in a
        real repo
    Practice: Set up the VS Code extension and review an edit as an inline diff.
    Assessment: Show a configured project with settings.json and a successful first task.

  Class 3: Installing & Configuring the OpenAI Codex CLI
    Topics:
      - Install: npm install -g @openai/codex (Node 18+), or brew install --cask codex; note the
        @openai/ scope matters
      - Authenticate: Sign in with ChatGPT (Plus/Pro/Business/Edu/Enterprise) or an API key
      - Health check: codex doctor verifies runtime, auth, terminal, and network
      - The interactive TUI: model, service tier, token usage, and approval mode shown inline
      - Config file: ~/.codex/config.toml; switch settings per project with profiles (codex --profile
        <name>)
      - Model & reasoning selection with /model (low up to ultra); the 2026 lineup is GPT-5.6 Sol
        (complex work), Terra (all-rounder), Luna (fast, repeatable tasks)
      - Windows options: native (PowerShell + Windows Sandbox) or WSL2
    Projects built:
      - Install Codex, create a ~/.codex/config.toml with a custom profile, and complete a task in
        workspace-write mode
    Practice: Run codex doctor, fix any warnings, and switch reasoning level mid-task.
    Assessment: Submit your config.toml and a screenshot of a successful Codex session.

  Class 4: Approval & Permission Models (Safety Backbone)
    Topics:
      - Why approvals are the safety backbone of agentic coding
      - Claude Code permission modes (Shift+Tab to cycle): default, acceptEdits, plan (read-only),
        auto (classifier-approved); set with --permission-mode, manage via /permissions
      - Permission rule syntax: allow/ask/deny patterns like Bash(git push *), Edit(*.env)
      - Codex sandbox modes: read-only, workspace-write (default), danger-full-access; approval
        policies untrusted/on-request/on-failure/never/granular
      - Codex /approvals to switch live; the --dangerously-bypass-approvals-and-sandbox flag and why
        it's a last resort
      - Designing a team-safe default posture (start restrictive, widen deliberately)
      - Reading and reasoning about an approval request before granting it
    Projects built:
      - Author a restrictive permission/approval policy for both tools and prove a blocked action is
        denied

Practice: Add a deny rule for force-push and destructive commands in both tools.
Assessment: Document a recommended default permission posture for a small dev team.

Class 5: Your First Real Engineering Task in Both Agents
Topics:
- Pick a real bug or small feature in a sample repo and scope it precisely
- Claude Code: plan diff approve verify; iterate with targeted feedback
- Codex: same task in workspace-write; compare approach, edits, and explanations
- Reading multi-file diffs critically and catching over-broad changes
- Driving iteration: 'narrow the change to module X', 'add a guard for null input'
- Verifying behavior: have the agent run the app/tests and confirm the fix
- Capturing what worked: notes you'll fold into project memory next phase
Projects built:
- Resolve the same issue with each agent and write a short comparison of edit quality and effort
Practice: Introduce a regression and have each agent locate and fix it.
Assessment: Submit two diffs (one per agent) solving the same task, with a critique.

PHASE 2: YOUR DAILY ENGINEERING WORKFLOW

Build a fast, reliable daily loop: slash commands, planning, reviewing diffs, running tests and git, teaching the agent your codebase, and recovering with checkpoints.

Class 6: Slash Commands & Session Control
Topics:
- Claude Code: /help, /clear, /compact (compress context), /context (inspect what's loaded), /model, /diff, /review
- Codex: /model, /approvals, /review, /status, /plan, /goal, and the @ picker for files/dirs/skills
- Managing context windows deliberately: when to /compact vs /clear vs start fresh
- @-mentioning files and MCP resources to target the agent precisely
- Monitoring cost and limits: /usage (Claude) and inline token usage (Codex)
- Recovering a derailed session: ask the agent to restate its current plan
- Building muscle memory in each TUI for speed
Projects built:
- Create a personal/team reference of the 12 most-used commands across both tools with examples
Practice: Run a long session, then reclaim context with /compact and compare token usage before/after.
Assessment: Demonstrate precise @-mention targeting and a clean context reset.

Class 7: Plan Mode & Goal Mode
Topics:
- Claude Code Plan Mode (/plan): read-only research and a proposed plan before any edit
- Goal Mode (/goal): the agent works until a defined completion condition is met
- Codex /plan and /goal (Goal mode stable and on by default in 2026)
- Writing checkable goals and acceptance criteria the agent can self-verify against
- Editing the agent's plan to constrain scope and approach before approval
- Plan-then-execute as a default for any multi-file or risky change
- Combining plan mode with restrictive permissions for safe exploration
Projects built:
- Use Plan Mode to design a non-trivial change (e.g., add pagination to an API) and execute the approved plan
Practice: Convert a vague ticket into a precise, checkable goal statement.
Assessment: Submit a plan you edited before approval and the resulting implementation.

Class 8: Diffs, Multi-File Edits & Review Discipline
Topics:
- How agents propose precise edits and render diffs across multiple files
- Reviewing a feature that spans several files coherently
- Accept / reject / request-changes on individual hunks
- Claude Code /diff and the VS Code/JetBrains inline review experience
- Keeping PRs small and reviewable; resisting scope creep from the agent
- Asking for rationale: 'why this approach over X?'
- Detecting and reverting unintended edits
Projects built:
- Have an agent refactor a legacy function and review every hunk, rejecting anything out of scope
Practice: Force the agent to split one large change into two smaller, reviewable steps.
Assessment: Critique a multi-file diff and identify any risky or unnecessary change.

Class 9: Running Tests, Commands & Git from the Agent
Topics:
- Delegating command execution: build, run, lint, and test through the agent
- Test-driven loops: have the agent write a failing test, implement, and make it pass
- Git with the agent: staged diffs, clear commit messages, branch hygiene
- Claude Code Bash tool (sandboxable) and Codex commands inside the workspace boundary
- Approving commands with judgment: reading the exact command first
- Fixing flaky/failing tests iteratively until green

- Frequent, small commits as a safety net
  Projects built:
- Implement a feature TDD-style with the agent: failing test implementation green commit
  Practice: Have the agent bisect and fix a failing CI step locally.
  Assessment: Show a clean commit history of an agent-assisted TDD cycle.

Class 10: Project Memory, CLAUDE.md & AGENTS.md
  Topics:
- Claude Code: /init generates CLAUDE.md; it auto-loads each session
- Memory hierarchy: managed/policy, user (~/.claude/CLAUDE.md), project (./CLAUDE.md), local (CLAUDE.local.md); imports with @path
- Auto-memory (2026): Claude saves learnings to a project memory folder; manage with /memory
- Codex AGENTS.md: repo root + nested per-package + personal ~/.codex/AGENTS.md (closest file wins)
- A 'Review guidelines' section in AGENTS.md to standardize Codex code review
- What belongs in memory: run/build/test commands, conventions, architectural rules, do/don't
- Keeping memory lean (~200 lines) and version-controlled for the team
  Projects built:
- Write production-quality CLAUDE.md and AGENTS.md for a real repo, including review guidelines
  Practice: Add a convention and confirm both agents follow it without re-prompting.
  Assessment: Submit team-ready memory files and explain three rules' impact.

Class 11: Checkpoints, Rewind & Session Management
  Topics:
- Claude Code automatic checkpointing before edits; /rewind (or Esc-Esc) to restore code, conversation, or both
- Key limitation: checkpoints track file-tool edits, not shell rm/mv: your git commits remain the real safety net
- Resuming: claude --continue and claude --resume (by id/name); --fork-session; --from-pr
- Branching a conversation (/branch) to explore alternatives
- Codex session resume and local conversation history search
- Combining git + checkpoints for confident experimentation
- Session organization across multiple concurrent tasks
  Projects built:
- Run a risky refactor, reject the outcome, and recover cleanly via /rewind plus git
  Practice: Resume a prior session by name and continue the task.
  Assessment: Demonstrate a recovery workflow combining rewind and git.

PHASE 3: EXTENDING & CONNECTING YOUR AGENTS

Make the agents fit your stack: connect tools via MCP, build skills/plugins, deploy subagents, automate with hooks, tune models and reasoning effort, and feed visual context.

Class 12: Connecting Tools, Data & Services with MCP
  Topics:
- Model Context Protocol: a standard for exposing tools/data/resources to agents
- Claude Code: claude mcp add --transport http <name> <url> (also sse/stdio); scopes local/project/user; manage with /mcp, claude mcp list
- MCP resources (@server:protocol://path) and MCP prompts as slash commands
- Codex: configure servers under [mcp_servers.<name>] in config.toml (stdio command/args/env or HTTP url), per-tool approval modes
- Practical integrations: issue trackers, databases, docs, design tools, internal APIs
- Tool Search / deferred tool loading to keep context lean with many servers
- Security: trust boundaries, OAuth, and prompt-injection risk from MCP content
  Projects built:
- Connect a real MCP server (e.g., a database or docs server) to both agents and complete a task using it
  Practice: Add a project-scoped .mcp.json so teammates inherit the same server.
  Assessment: Show one server connected per tool and a task that used its tools.

Class 13: Skills, Custom Commands & Codex Plugins
  Topics:
- Claude Code Skills (Agent Skills standard): reusable, auto-invokable instructions; .claude/commands/*.md and .claude/skills/*/SKILL.md both create /commands
- Frontmatter control: who can invoke, allowed/disallowed tools, subagent execution; /skills to manage, /reload-skills
- Designing a skill: a /deploy or /release command encapsulating your workflow
- Codex profiles for per-project model/sandbox settings and the plugin marketplace for sharing setups
- When to encode a workflow as a skill/plugin vs ad-hoc prompting
- Distributing skills/plugins to a team for consistency
- Versioning and reviewing shared automations
  Projects built:
- Build a Claude Code skill that automates a repeatable workflow and a Codex profile for a project
  Practice: Iterate a skill twice based on real usage feedback.
  Assessment: Submit a working skill/command and document what it standardizes.

Class 14: Subagents, Specialized and Isolated Helpers
Topics:
- Subagents run in their own context with their own tools and permissions
- Built-ins: Explore (fast read-only search), Plan (read-only planning), General-purpose
- Define your own: .claude/agents/*.md with name, description, tools, model, permissionMode
- Least-privilege agents: a reviewer that can only read; a test-writer scoped to tests
- Manage and generate with /agents; subagent memory for cross-session learning
- Dispatching parallel subagents for independent subtasks
- Why subagents protect the main context and improve reliability (since mid-2026 they run in the background by default, so the main session keeps working)
Projects built:
- Create code-reviewer and test-writer subagents and use them on a real change
Practice: Compare Explore subagent results vs the main agent for a codebase question.
Assessment: Submit two subagent definitions and sample outputs.

Class 15: Hooks & Workflow Automation
Topics:
- Hooks run deterministic code at lifecycle events: automation and guardrails
- Key events: PreToolUse, PostToolUse, SessionStart, UserPromptSubmit, Stop, PreCompact and more
- Config in settings JSON: matcher + handler (command/http/prompt/agent), exit-code 2 to block
- Practical hooks: auto-format/lint on edit, run tests after changes, block dangerous commands, inject context
- Placeholders like ${CLAUDE_PROJECT_DIR} and viewing hooks with /hooks
- Treating hook output as feedback the agent must respond to
- Governance: enforcing team standards automatically via hooks
Projects built:
- Implement a PostToolUse format+lint hook and a PreToolUse guard that blocks force-push
Practice: Add a SessionStart hook that prints project setup status.
Assessment: Submit two working hooks and explain their events and effects.

Class 16: Models, Reasoning Effort & Context
Topics:
- Claude Code models: Fable 5 (the new Claude 5 flagship tier), Opus 4.8, Sonnet 5 (the default), Haiku 4.5 (fast/cheap); /model and aliases
- Effort levels low/medium/high/xhigh (set via /effort, --effort) and the ultrathink keyword
- Native 1M-token context on Sonnet 5 for large codebases
- Codex models: GPT-5.6 Sol, Terra, and Luna; reasoning levels low up to ultra (ultra auto-delegates to subagents); long-task compaction across context windows
- Cost/performance trade-offs: cheap+fast for routine edits, top model+high effort for hard design
- Subagent model overrides for cost control at scale
- Measuring whether more effort actually improved the result
Projects built:
- Benchmark a hard refactor at two effort levels and two models; document quality vs cost/latency
Practice: Pin different models for main vs subagents and observe the effect.
Assessment: Produce a model+effort selection guide for five task types.

Class 17: Multimodal Inputs, Visual Context
Topics:
- Why visuals help: convey designs, dashboards, and bugs faster than prose
- Drag-and-drop screenshots/images into the prompt as context
- Codex Appshots (macOS): capture frontmost app windows for context
- UI-from-image: have the agent implement a layout from a design screenshot
- Debugging from a screenshot of an error or broken render
- Mixing text + image instructions for precise outcomes
- Current limits: image input in both agents; Codex added audio inputs and tool outputs in 2026; video is still not supported
Projects built:
- Recreate a real UI from a design screenshot and reconcile it against the spec
Practice: Diagnose a rendering bug from a screenshot alone.
Assessment: Show an image-driven task with a correct, verified result.

PHASE 4: CLOUD, COLLABORATION & SCALE

Operate beyond a single machine: delegate to Codex Cloud, use Claude Code across surfaces, integrate AI code review into GitHub/CI, and orchestrate parallel agents.

Class 18: Codex Cloud, Parallel Tasks & Auto-PRs
Topics:
- Codex Cloud: asynchronous agents working in isolated, sandboxed environments
- Get started at chatgpt.com/codex and connect GitHub
- Queue many tasks in parallel; each returns as its own pull request
- Configure cloud environments: repo, setup steps, tools, and internet-access controls
- Reviewing cloud-generated PRs before merge; iterating via comments
- When cloud wins: long-running jobs, parallel chores, repos not on your laptop
- Governance for cloud agents in a team setting

Projects built:
- Delegate three parallel Codex Cloud tasks and review/merge the resulting PRs
Practice: Configure a cloud environment's setup steps so the agent can build/test the repo.
Assessment: Show parallel cloud tasks and explain each PR's scope and quality.

Class 19: Claude Code Across Surfaces (Web, Desktop, Mobile, Background)
Topics:
- claude.ai/code on the web for long-running and parallel tasks, even on repos you don't have locally
- Desktop app: visual diff review and multiple side-by-side sessions
- Background agents: detach with /background, manage with claude agents / attach / logs
- Remote control & mobile: steer and approve from your phone
- Teleporting a web/remote session into the terminal to finish locally
- Choosing the right surface per task and handing off between them
- Slack/GitHub triggers that start sessions
Projects built:
- Start a task on the web, hand it to the desktop/terminal, and finish it; run one agent in the background
Practice: Manage a background agent's lifecycle from the shell.
Assessment: Demonstrate a task moving across two surfaces and a managed background agent.

Class 20: GitHub Integration & AI Code Review at Scale
Topics:
- Connect GitHub apps so both agents can open and review PRs
- Codex review: @codex review (P0/P1 findings) or enable automatic reviews on every PR; scoped reviews like @codex review for security regressions
- Codex task mentions: @codex fix the CI failures pushes fixes when authorized
- Claude Code /code-review (effort levels), --comment for inline GitHub comments, --fix to apply findings
- CI automation: openai/codex-action and Claude Code GitHub Actions / GitLab CI
- Defining team review guidelines (AGENTS.md) honored during review
- Acting on AI review feedback with rigor, not blind acceptance
Projects built:
- Set up automatic AI code review on a repo and run a PR through both Codex and Claude Code review
Practice: Tune review guidelines and observe how findings change.
Assessment: Show a reviewed PR with AI findings you triaged and resolved.

Class 21: Parallel Agents, Worktrees, /batch & Teams
Topics:
- Parallelization strategy: decompose large work into independent units
- Git worktrees for isolation: claude -w <name> creates an isolated workspace
- Claude Code /batch: split a large change into 5-30 units, one background subagent per worktree, each tests + opens a PR
- Agent teams: multiple coordinating sessions (--teammate-mode)
- Codex Cloud parallelism for the same outcome via many PRs
- Reviewing and integrating parallel outputs without conflicts
- Knowing when serial is safer than parallel
Projects built:
- Use /batch (or parallel Codex Cloud tasks) to roll a cross-cutting change across a codebase and review each PR
Practice: Run two agents in separate worktrees on independent tasks simultaneously.
Assessment: Document a parallelized change: the split, the worktrees/PRs, and the review.

PHASE 5: BUILD, SECURE & SHIP

Operationalize agents: automate with the SDKs, harden security for teams, and deliver a production-grade capstone, then certify.

Class 22: Building With the SDKs (Claude Agent SDK & Codex SDK)
Topics:
- Headless automation: claude -p "task" (output-format text/json/stream-json) and codex exec (with --output-schema for structured JSON)
- Claude Agent SDK: npm i @anthropic-ai/claude-agent-sdk / pip install claude-agent-sdk; query() with tools, hooks, agents, permissionMode
- Codex SDK: @openai/codex-sdk (TS) and openai-codex (Python): startThread, run, resumeThread
- Piping and composition: git diff main | claude -p "review for security issues"
- Building an internal automation: triage, summarize, or auto-fix in CI
- Budget/turn controls (--max-turns, --max-budget-usd) for safe unattended runs
- Note: from June 15 2026 Agent SDK / claude -p usage draws on a separate credit on subscription plans
Projects built:
- Build a CI automation (e.g., auto-summarize PR diffs or auto-label issues) using claude -p or codex exec with structured output
Practice: Write a script that pipes logs into an agent and parses its JSON response.
Assessment: Submit a working SDK/headless automation with structured output.

Class 23: Security, Sandboxing & Governance for Teams
Topics:
- Permission rules (allow/ask/deny) evaluated before any tool runs; pattern syntax for fine control
- Claude Code Bash sandbox (/sandbox): Seatbelt (macOS) / bubblewrap (Linux/WSL2); filesystem & network isolation, allow/deny domains
- Codex sandboxes: Seatbelt / bubblewrap / Windows Sandbox; read-only vs workspace-write vs danger-full-access
- Secrets hygiene: denyRead for ~/.ssh, ~/.aws; never commit credentials; env scrubbing
- Prompt injection & untrusted content: risks from web fetches and MCP servers, and mitigations
- Managed/enterprise settings: forced login org, allowed models, disabling hooks, managed domain allowlists
- Accountability: you own the code the agent writes: review and test it

Projects built:
- Define and test a hardened security configuration (permissions + sandbox + secret denials) for a team repo

Practice: Demonstrate a sandbox blocking network/file access outside the allowed scope.
Assessment: Deliver a team 'safe agentic coding' policy document with concrete settings.

Class 24: Capstone, Ship a Production-Grade Build + Certification
Topics:
- Scope a real deliverable: a feature in an existing app, or a small full-stack app + an automation
- Plan with Plan/Goal mode; author CLAUDE.md and AGENTS.md with review guidelines
- Build with both agents, using subagents/hooks/MCP where they add value
- Quality gate: tests, AI code review (with fixes), and a clean PR history
- Ship it: deploy the app and/or land the automation in CI
- Document your workflow: what each agent did, decisions, and safeguards
- Present and earn the Modern Age Coders AI Coding Agents Professional certificate

Projects built:
- Deliver a deployed, tested, AI-reviewed capstone (app and/or SDK automation) with a documented agentic workflow

Practice: Present a 5-minute walkthrough of how you directed the agents end-to-end.
Assessment: Final capstone: deployed/merged work + repo + write-up; pass earns the professional certificate.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: a Codex vs Claude Code comparison brief, both CLIs installed and configured (project settings.json and a config.toml with a custom profile), a restrictive permission policy proven to block unsafe actions, and the same real bug fixed with each agent
- Phase 2: a 12-command team reference, a Plan Mode change designed and executed (API pagination), a reviewed legacy-function refactor, a TDD feature taken from failing test to green commit, production-quality CLAUDE.md and AGENTS.md files, and a rewind-plus-git recovery drill
- Phase 3: a real MCP server connected to both agents, a reusable workflow skill plus a Codex profile, code-reviewer and test-writer subagents, format-and-lint and force-push-guard hooks, a model and effort benchmark, and a UI rebuilt from a design screenshot
- Phase 4: three parallel Codex Cloud tasks reviewed and merged as pull requests, a cross-surface workflow with a managed background agent, automatic AI code review wired into a GitHub repo, and a cross-cutting change rolled out with /batch across worktrees
- Phase 5: a CI automation built with claude -p or codex exec, a hardened team security configuration (permissions, sandbox, secret denials), and a deployed, tested, AI-reviewed capstone
- Total: 24 hands-on deliverables, one per class, capped by a production-grade capstone

Total projects: 24 hands-on deliverables, one per class, including a production-grade capstone shipped in Class 24

SKILLS THE STUDENT WILL HAVE AT THE END

- AI Coding Agents: Codex CLI, Claude Code, agent workflows, plan and goal mode, context management
- Prompting for Code: precise goal statements, spec-first planning, iterative refinement, debugging prompts
- Extending Agents: MCP servers, skills and custom commands, subagents, hooks, Codex profiles
- Model Strategy: Fable 5, Opus 4.8, Sonnet 5, Haiku 4.5, GPT-5.6 Sol/Terra/Luna, effort levels, cost and quality trade-offs
- Version Control: git branches, worktrees, clean commit history, PR-based workflows
- Reviewing AI Output: reading multi-file diffs, scoping changes, AI code review in GitHub and CI
- Automation with SDKs: Claude Agent SDK, Codex SDK, headless runs, structured output, CI integrations
- Security and Governance: permission rules, OS sandboxes, secrets hygiene, prompt-injection defense

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, about 1 hour each, across 12 weeks
Hands on agent work: 1-1.5 hours running real commands in Codex and Claude Code between

classes
Project time: 1-1.5 hours completing each class deliverable in your own repo
Review and doubts: 30-60 minutes reviewing diffs, refining your notes, and clearing doubts with mentors
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so you are never stuck on a config or command
Small batches: Small batches, with mini-batch and 1-on-1 options, so every professional gets individual attention
Mentor guidance: Live guidance from mentors who use Codex and Claude Code in production daily
Capstone review: Personal review of your capstone and workflow write-up before certification
Living syllabus: Curriculum updated as OpenAI and Anthropic ship new features, at no extra cost

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- AI-Assisted Software Engineer / Full-Stack Developer
- Developer Productivity / Platform Engineer (agent workflows, internal tooling)
- Automation Engineer building on the Claude Agent SDK / Codex SDK
- Tech Lead / Engineering Manager standardizing agentic practices
- Freelance / Founder shipping AI-built products faster

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 6-12 LPA in India / $70k-110k globally (developer using agents to boost output)
Growing roles: Rs 12-22 LPA / $110k-150k (AI-assisted engineer shipping features independently)
Experienced roles: Rs 22-40+ LPA / $150k-220k+ (senior/lead leveraging agents and SDK automation at scale)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need a Claude or ChatGPT subscription to take this masterclass?
A: Yes, this is a course requirement. To run the agents in every lab you must have an active Claude subscription (Pro, Max, Team, or Enterprise) for Claude Code and a ChatGPT subscription (Plus, Pro, Business, Edu, or Enterprise) for Codex, or equivalent API keys, for example through your employer or via Bedrock/Vertex AI. Subscriptions are purchased directly from Anthropic and OpenAI and are separate from the course fee. We help you pick the most cost-effective setup in your free demo, and many employers reimburse these tools. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final project defended live, plus a written assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Is OpenAI Codex free, and what about Claude Code?
A: The Codex CLI and SDK are open-source/free to install; running them uses your ChatGPT plan (Plus/Pro/Business/Edu/Enterprise) or an OpenAI API key. Claude Code runs on a Claude subscription (Pro/Max/Team/Enterprise) or an Anthropic API key. Both moved to token/usage-based limits in 2026. For this masterclass an active plan for each tool is required, and we help you pick the most cost-effective setup for learning and for work.

Q: Can I use what I learn at my job right away?
A: Yes, the labs are deliberately workplace-grade: fixing CI, refactoring legacy code, reviewing PRs at scale, and building SDK automations. You'll also produce a team-ready security/governance policy so you can introduce agents responsibly.

Q: Codex vs Claude Code: which is better for teams?

A: They're complementary. Codex excels at cloud parallel tasks and GitHub-native review; Claude Code offers deep terminal control, subagents, hooks, skills, and the Agent SDK. This course teaches both to professional depth and, crucially, how to choose the right one per task and integrate them into a team workflow.

Q: Does the course cover the Claude Agent SDK and the Codex SDK?
A: Yes, Class 22 is dedicated to headless automation and both SDKs (TypeScript and Python). You'll build a real CI automation with structured output, plus learn budget/turn controls for safe unattended runs.

Q: Will the content stay current as the tools change so fast?
A: That's our Living Syllabus promise. Codex and Claude Code ship features and models constantly: mid-2026 alone brought Sonnet 5 as Claude Code's default with native 1M-token context, the Claude 5 flagship tier (Fable 5), and Codex's move to the GPT-5.6 family with reasoning up to ultra. We update the curriculum continuously so you always learn the current product, at no extra cost.

Q: I'm a career switcher with limited coding experience: is this for me?
A: Yes. We start from install and ramp deliberately. The agents help you read and write code as you learn, while the course builds the judgment in planning, reviewing, and security that separates real engineers from prompt-clickers.

Q: How do you handle security and 'letting AI run commands'?
A: An entire class (Class 23) covers permission rules, OS-level sandboxes (Seatbelt/bubblewrap/Windows Sandbox), secrets hygiene, prompt-injection defense, and enterprise/managed settings. You'll configure and test a hardened setup, and you always review before running.

Q: What's the schedule and format?
A: 24 classes, typically two per week over ~12 weeks, 4-6 hours/week recommended. Available as group, mini-batch, or 1-on-1, with live mentorship and a capstone review.

Q: What is the best way to learn Claude Code and Codex as a professional?
A: On real work, with live expert feedback. Both agents ship new features monthly, so recorded courses and blog tutorials age out quickly, and self-study rarely builds the review discipline, security posture, and SDK skills teams actually need. The fastest reliable path is structured live training where every session runs real commands on real repos with a mentor who uses these tools in production, which is exactly how this 24-class masterclass works. Modern Age Coders has taught 10,000+ students since 2020 and is rated 4.9 across 547 Google reviews.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/codex-and-claude-code-ai-coding-agents-masterclass-for-adults-professionals
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 41 OF 137: CODING FOR KIDS: SCRATCH TO GAMES, APPS AND AI
==========

Course link: https://learn.modernagecoders.com/courses/kids-coding-blocks-masterclass
Subject area: Kids Coding
Designed for: Kids
Who can join: Complete beginner, ages 6 to 12 (batches grouped by age)
Course length: 10-12 months (40-48 weeks)
Time needed: 2 live classes/week + 2-3 hours guided practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Coding for kids aged 6-12: live online classes from first Scratch game to mobile apps, Minecraft and Roblox coding, kid-safe AI and a first taste of Python.

In one line: A child who can drag two blocks together can start. Ten months later they have built games, a mobile app, Minecraft machines, an AI model and their first lines of Python.

ABOUT THE PROGRAMME

This is our flagship coding journey for children aged 6 to 12, and it is built on a simple truth: kids learn to code by making things they want to show someone. Every week ends with something a child can demonstrate at the dinner table, a game, an animation, an app, a world,

and every concept hides inside a project rather than a lecture. The journey starts in Scratch, where logic is learned by snapping blocks: animations first, then real games with scores, levels and difficulty. From there the worlds get bigger: arcade games in Microsoft MakeCode, inventions on a simulated micro:bit, real mobile apps built in MIT App Inventor and installed on the family phone, coding inside Minecraft Education, first Lua scripts in Roblox Studio, and kid-safe AI, where children train their own image and sound models with Machine Learning for Kids and Teachable Machine and use them inside their own games. The final months build the bridge every parent asks about: from blocks to real typed code, with first Python turtle programs and a first hand-written web page. The pace is honest: two live classes a week plus two to three hours of guided practice fits the journey in ten months, and younger children or busy terms stretch it to twelve without penalty, the batch adapts. Learning is checked the honest way too: classwork in every session, homework after every class, monthly challenges that mix new questions with earlier topics, and a final exam, a project plus a friendly viva, that decides the certificate. Real knowledge, real work, real projects, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Every week ends with something a child can show: a game, an app, a world, a model, not a worksheet
- Batches are grouped by age, so a 7-year-old is never racing an 11-year-old through the same material
- The tools are the real ones schools and clubs use worldwide: Scratch, MakeCode, App Inventor, Minecraft Education, Roblox Studio
- AI is taught kid-safe and hands-on: children train their own models with Machine Learning for Kids and Teachable Machine, no open chatbots
- The course ends with a real bridge to typed code: first Python and a first hand-written web page, so the next step is natural
- Real assessment, gently done: weekly homework, monthly mixed-review challenges, and a final project-plus-viva exam that decides the certificate, with a free retest after revision if needed

THE LEARNING PATH, STEP BY STEP

Phase 1: Scratch from zero to real games (Months 1-3): animation, interactivity, loops, variables, then complete games with levels and scores
Phase 2: New worlds (Months 4-6): MakeCode arcade games, micro:bit inventions, and real mobile apps built in MIT App Inventor
Phase 3: Bigger worlds and AI (Months 7-8): coding in Minecraft Education, first Lua in Roblox Studio, and training their own AI models
Phase 4: The bridge to real code (Months 9-10): data projects, interactive stories, first Python, a first web page, and the final capstone

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Block coding was invented exactly for this start
Reading: Comfortable reading simple English (roughly Grade 2 and up)
Equipment: A computer or laptop with a stable internet connection; a tablet works for the early months
Age: 6 to 12. Batches are grouped by age so pace, examples and humor fit the room
Time commitment: 2 live classes plus 2-3 hours of playful practice a week

WHO THIS COURSE IS FOR

Curious beginners: Children who love games and gadgets and are ready to make their own
Young starters: Ages 6-8 who start gently with blocks and grow into the bigger tools
Older kids: Ages 9-12 who move faster and reach Python and Roblox scripting sooner
Screen time worriers: Parents who want screen time converted from consuming into creating
Future programmers: Kids who will graduate into our Python, Roblox, Minecraft and web courses

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: SCRATCH FROM ZERO TO REAL GAMES (MONTHS 1-3, WEEKS 1-12)

Scratch is where children discover that they can command a computer. From first animation to complete games with menus, levels and scores.

Month 1: First Steps in Scratch (Weeks 1-4)
Welcome to Coding
Topics:
- What is coding? Why is it important?
- How computers understand instructions
- Introduction to algorithms with real-life examples
- Setting up your coding workspace
- Creating safe accounts on Scratch and Code.org together with a parent
- Navigating Scratch interface

- Understanding sprites and stages
- Basic blocks: Motion, Looks, Sound
- Your first animation: Making a cat walk
- Saving and sharing projects
- Online safety and digital citizenship
- Creating your coder profile

Projects built:

- Animated greeting card
- Pet animation with sounds
- Interactive 'About Me' project

Practice: Daily 15 minutes of block exploration and one small creative build

Motion and Animation

Topics:

- Coordinate system (x, y positions)
- Moving sprites: go to, glide, move steps
- Rotation: turn, point in direction
- Creating smooth animations
- Using costumes for animation
- Backdrop changes and scenes
- Sound effects and music
- Recording custom sounds
- Text-to-speech blocks
- Animation principles: timing, easing
- Creating realistic movements
- Sprite interactions

Projects built:

- Animated story with 3 scenes
- Dancing party animation
- Virtual pet that moves and makes sounds

Practice: Create 5 different animation sequences

Events and Interactivity

Topics:

- Event blocks: when flag clicked, when key pressed
- Mouse events: when sprite clicked
- Broadcasting messages between sprites
- Creating interactive buttons
- Keyboard controls for games
- Mouse following mechanics
- Touch controls for tablets
- Creating menus and navigation
- User input with 'ask' blocks
- Storing answers in variables (introduction)
- Creating choose-your-own adventure stories
- Interactive art projects

Projects built:

- Interactive quiz game
- Virtual piano/musical instrument
- Dress-up game with clickable items

Practice: Build 5 interactive mini-projects

First Game Logic

Topics:

- If-then statements (conditionals)
- If-else logic blocks
- Collision detection: touching sprite/color
- Score keeping with variables
- Lives and game over conditions
- Timer and countdown mechanics
- Random numbers for surprises
- Creating power-ups
- Level progression basics
- Win and lose conditions
- Game states: start, playing, game over
- Debugging techniques

Projects built:

- Catch the falling objects game
- Simple maze game
- Whack-a-mole style game
- Number guessing game

Practice: Create 3 simple games with scoring

Assessment: Month 1 check: build a small interactive scene live, plus a fun take-home challenge mixing everything from month 1

Month 2: Think Like a Coder (Weeks 5-8)

Loops and Patterns

Topics:

- Forever loops for continuous actions
- Repeat loops with counters
- Repeat until conditions
- Nested loops for complex patterns

```
      - Creating geometric patterns
      - Drawing with pen blocks
      - Stamp tool for patterns
      - Color effects and graphics
      - Creating fractals and spirals
      - Loop optimization
      - Breaking out of loops
      - Infinite loop prevention
    Projects built:
      - Geometric art generator
      - Pattern-based puzzle game
      - Automated drawing machine
      - Kaleidoscope effect
    Practice: Create 10 different patterns using loops
  Variables: Numbers That Remember
    Topics:
      - Creating and naming variables
      - Local vs. global variables
      - Changing variable values
      - Variables for game mechanics
      - High score systems
      - Mathematical operations with variables
      - Creating calculators
    Projects built:
      - Calculator with memory
      - Quiz with score tracking
      - Click-counter challenge game
    Practice: Build 5 projects that count, score or remember something
  Lists and Inventories
    Topics:
      - Lists (arrays) introduction
      - Adding/removing from lists
      - List operations: length, contains
      - High score systems
      - Inventory systems for games
      - Data persistence with cloud variables
    Projects built:
      - Shopping game with inventory
      - RPG with character stats
      - High-score table that survives the game
    Practice: Add a list to two of your earlier projects
  Puzzle Week on Code.org
    Topics:
      - Introduction to Code.org platform
      - Block-based JavaScript preview
      - Minecraft Hour of Code
      - Star Wars coding challenges
      - Dance Party creation
      - App Lab basics
      - Creating mobile app interfaces
      - Event-driven programming
      - Debugging with Code.org
      - CS Fundamentals courses
      - Problem-solving strategies
      - Computational thinking puzzles
    Projects built:
      - Complete Minecraft adventurer
      - Create dance party choreography
      - Build app prototype in App Lab
      - Solve 20 Code.org puzzles
    Practice: A Code.org puzzle streak: a little every day
    Assessment: Month 2 check: solve three fresh logic puzzles live, with questions that
    reach back to month 1

Month 3: Real Games, Properly Built (Weeks 9-12)
  Clone the Classics I
    Topics:
      - Analyzing existing games
      - Breaking down game mechanics
      - Planning complex projects
      - Creating Pong clone
      - Building Flappy Bird clone
      - Snake game mechanics
    Projects built:
      - Pong clone that feels fair
      - Flappy Bird complete clone
      - Snake with growing tail
    Practice: Polish one clone until a friend asks to play again
  Clone the Classics II: Platformers
```

```
      Topics:
        - Analyzing existing games
        - Breaking down game mechanics
        - Planning complex projects
        - Platform game basics
        - Scrolling backgrounds
        - Multiple levels design
        - Save game functionality
        - Leaderboards with cloud data
        - Multiplayer basics
      Projects built:
        - Platform game with 3 levels
        - Pac-Man style maze game
        - Two-player competitive game
      Practice: Design and build one extra level for each game
    What Makes Games Fun?
      Topics:
        - What makes games fun?
        - Game mechanics vs. dynamics
        - Level design principles
        - Difficulty curves
        - Player feedback systems
        - Reward mechanisms
        - Story in games
        - Character development
        - Game art basics
        - Sound design importance
        - Playtesting methods
        - Iterative design process
      Projects built:
        - Design document for original game
        - Level pack for existing game
        - Game with adaptive difficulty
        - Story-driven adventure game
      Practice: Play and analyze 5 favorite games: what makes each one fair and fun?
    Scratch Showcase
      Topics:
        - Project presentation skills
        - Creating game trailers
        - Writing game descriptions
        - Peer review and feedback
        - Code optimization
        - Performance improvements
        - Portfolio organization
      Projects built:
        - CAPSTONE: an original game with menu, levels, scoring and sounds
        - A short presentation of the game to the batch
      Practice: Run a family playtest and fix the two most confusing things
      Assessment: Phase 1 exam: present the capstone game and rebuild a small scene live,
      with mixed questions from months 1-3

---------------------------------------------------------------
PHASE 2: NEW WORLDS: ARCADE, MICRO:BIT AND MOBILE APPS (MONTHS 4-6, WEEKS 13-24)
---------------------------------------------------------------
The skills from Scratch travel: arcade games in MakeCode, inventions on a simulated
micro:bit, and real mobile apps built in MIT App Inventor.

  Month 4: MakeCode Arcade and micro:bit (Weeks 13-16)
    MakeCode Arcade Basics
      Topics:
        - Introduction to MakeCode Arcade
        - Sprite creation and editing
        - Tile maps and level design
        - Physics in games
        - Scoring and lives system
        - Sound effects creation
        - Custom sprite editor
        - JavaScript preview in MakeCode
      Projects built:
        - Adventure game with a tile map
        - Racing game with obstacles
      Practice: Build 3 small arcade games between classes
    MakeCode Arcade: Enemies and Effects
      Topics:
        - Physics in games
        - Projectiles and effects
        - Enemy AI basics
        - Scoring and lives system
        - Sound effects creation
```

- Publishing games online
- Sharing with QR codes
- JavaScript preview in MakeCode

Projects built:

- Shooting game with enemy waves
- RPG with story elements
- Your best arcade game shared with a QR code

Practice: Add enemies and effects to your month 4 games

micro:bit Inventions

Topics:

- What is Micro:bit?
- MakeCode for Micro:bit
- LED display programming
- Button inputs A and B
- Accelerometer and shake detection
- Temperature sensor
- Light sensor projects
- Radio communication between devices
- Creating wearable projects
- Music and sounds on Micro:bit
- The MakeCode simulator: a full micro:bit in the browser, no hardware needed
- Optional: running your inventions on a real micro:bit at home

Projects built:

- Digital dice
- Step counter/fitness tracker
- Two-player rock-paper-scissors
- Temperature alarm system

Practice: Build 5 micro:bit inventions in the simulator

Blockly Puzzle Gym

Topics:

- Introduction to Blockly
- Blockly Games progression
- Maze solving algorithms
- Bird physics game
- Turtle graphics programming
- Movie creation with Blockly
- Music composition blocks
- Pond game strategies
- JavaScript code generation
- Storytelling with code

Projects built:

- Complete all Blockly Games
- Turtle art masterpiece
- Animated movie with Blockly
- Music composition project

Practice: Finish the Blockly Games ladder at your own pace

Assessment: Month 4 check: build a small arcade scene live, plus a mixed challenge reaching back to Scratch

Month 5: Build Real Mobile Apps (Weeks 17-20)

MIT App Inventor Basics

Topics:

- Introduction to MIT App Inventor
- Designer vs. Blocks editor
- Screen layouts and components
- Buttons, labels, textboxes
- Images and media components
- Screen navigation
- Properties and styling
- Event handlers in apps
- Variables in App Inventor
- Procedures (functions)
- Testing with companion app
- Building APK files

Projects built:

- Calculator app
- To-do list app
- Flashcard study app
- Personal diary app

Practice: Build 3 small apps and run them on a family phone

Apps with Camera, Voice and Drawing

Topics:

- Lists and databases in apps
- TinyDB for local storage
- Camera and image picker
- Sound recorder and player
- Text-to-speech and speech recognition
- Canvas for drawing
- Sprite animations in apps

- Accelerometer games
- Sharing functionality

Projects built:

- Photo sticker app
- Voice note recorder
- Drawing app with save

Practice: Add one camera or voice feature to an earlier app

Games on the Phone

Topics:

- Game loop concepts
- Sprite collisions in apps
- Score and high score systems
- Timer-based games
- Touch and drag mechanics
- Accelerometer controls
- Sound effects in games
- Multiple screens for games
- Game state management

Projects built:

- Flappy Bird mobile version
- Tilt-controlled maze game
- Memory card game

Practice: Playtest your phone games with two family members

App Capstone: An App for Your Family

Topics:

- Choosing an app your family would actually use
- Sketching screens on paper before building
- Building the app screen by screen
- Saving data so the app remembers
- Testing with a real user: watching without helping
- Fixing what confused them, not what annoyed you
- Presenting the app and the story behind it

Projects built:

- The family app: a chore chart, pet log, homework planner or your own idea, installed on a real phone

Practice: One week of real family use, then one improvement

Assessment: Month 5 check: app demo plus a mixed quiz over months 1-5

Month 6: Stories, Worlds and the Phase Capstone (Weeks 21-24)

Digital Storytelling

Topics:

- Story structure basics
- Character development
- Digital comic creation
- Interactive stories with Twine
- Animation storytelling
- Digital portfolio creation
- Sharing work safely: what goes online and what stays home

Projects built:

- Interactive story in Twine
- Digital comic book
- Animated short story

Practice: Write and branch one choose-your-own-adventure story

What Is AI, Really?

Topics:

- What is Artificial Intelligence?
- Machine Learning basics for kids
- Training AI models concept
- Scratch + Machine Learning 4 Kids
- Image recognition projects
- Text classification basics
- Sound recognition
- Creating training data
- Testing AI models
- AI ethics for kids
- Bias in AI systems
- Future of AI

Projects built:

- Rock-paper-scissors AI
- Emotion detector with images
- Chatbot in Scratch
- Voice command game

Practice: Train 5 small models and note where each one gets confused

Teachable Machine Projects

Topics:

- Google Teachable Machine
- Training image models
- Training sound models
- Training pose models

- Exporting models
- Using models in Scratch
- AI-powered games
- Computer vision projects
- Gesture control games
- AI art generation basics

Projects built:

- Gesture-controlled game
- AI fitness coach
- Sound-activated animations
- AI art generator

Practice: Build 3 projects that use a model you trained yourself

Phase 2 Capstone and Review

Topics:

- Choosing the capstone: best game, best app or best AI project, taken further
- Improving old projects with new skills
- Showing work: a tidy project gallery
- A review week shaped by what the batch found hard
- Kind, useful feedback: reviewing a classmate's project

Projects built:

- Phase capstone: one project taken one honest level further, presented to the batch

Practice: Publish your top 3 projects in your personal gallery

Assessment: Phase 2 review: capstone demo plus a mixed challenge spanning phases 1 and 2

PHASE 3: MINECRAFT, ROBLOX AND AI (MONTHS 7-8, WEEKS 25-32)

Coding inside the worlds kids already love: Minecraft Education, Roblox Studio with first Lua scripts, 3D design, and AI models they train themselves.

Month 7: Minecraft and Roblox (Weeks 25-28)

Coding in Minecraft Education

Topics:

- Minecraft Education setup
- Code Builder introduction
- MakeCode in Minecraft
- Agent programming
- Building automation
- Redstone basics
- Command blocks
- Creating educational worlds
- Chemistry in Minecraft
- History lessons in Minecraft
- Collaborative building

Projects built:

- Automated farm system
- Historical monument recreation
- Science experiment world
- Adventure map with coding

Practice: Complete 10 coding challenges inside Minecraft

Roblox Studio: First Worlds

Topics:

- What is Roblox Studio?
- Basic building tools
- Parts and properties
- Terrain editor
- Basic scripting preview
- Models and assets
- Lighting and effects
- Sound in Roblox
- Sharing a world safely, with parent-managed accounts
- Community guidelines
- Collaborative development

Projects built:

- Simple obby (obstacle course)
- Basic tycoon game
- Adventure map
- Multiplayer arena

Practice: Build one obby and improve it after a friend tries it

Roblox: First Lua Scripts

Topics:

- Roblox Studio advanced tools
- Lua scripting basics
- Variables and functions in Lua
- Events and triggers
- GUI creation
- Leaderboards and stats
- Advanced terrain

- Particle effects
- Animation editor
- Blocks to text: seeing your Scratch instincts inside Lua

Projects built:
- Tycoon game with working buttons
- Adventure map with scripted surprises
- Obby with checkpoints and a leaderboard

Practice: Write 5 small Lua scripts that change your world

Data Detective

Topics:
- What is data science?
- Collecting data responsibly
- Google Sheets for data
- Basic statistics concepts
- Creating surveys with Google Forms
- Data visualization basics
- Charts and graphs
- Infographic creation
- Data storytelling
- Privacy and data ethics
- Real-world data projects

Projects built:
- School survey analysis
- Weather data tracking
- Sports statistics dashboard
- Environmental data project

Practice: Run one real survey and chart what it says

Assessment: Month 7 check: explain one chart honestly, plus mixed questions from earlier phases

Month 8: 3D Worlds and Your Own AI (Weeks 29-32)

Tinkercad 3D Design

Topics:
- 3D modeling basics
- Tinkercad interface
- Basic shapes and operations
- Grouping and aligning
- Hole creation
- Precise measurements
- 3D printing preparation
- Circuit simulation
- Arduino simulation
- Codeblocks in Tinkercad
- Importing and exporting
- Design thinking process

Projects built:
- 3D printed keychain design
- Architectural model
- Arduino circuit simulation
- Mechanical toy design

Practice: Design 5 3D models, one of them useful at home

VR and AR Worlds in CoSpaces

Topics:
- Introduction to CoSpaces Edu
- 3D scene creation
- CoBlocks programming
- Physics in virtual worlds
- Character animation
- Sound and music in 3D
- VR experience design
- AR marker creation
- 360° environments
- Interactive museums
- Virtual field trips
- Publishing VR experiences

Projects built:
- Virtual museum tour
- AR treasure hunt
- VR escape room
- Interactive 3D story

Practice: Build one small VR scene and one AR surprise

AI Inside Your Games

Topics:
- Putting a trained model inside a Scratch game
- Gesture controls: playing a game with your hands
- Sound-activated animations and effects
- When the model is wrong: making games fail kindly
- Fairness: testing your model with different people
- AI ethics for kids, in plain words

```
        - Planning an AI-powered capstone
      Projects built:
        - Gesture-controlled game
        - Sound-activated pet
        - A model tested fairly across the whole family
      Practice: Improve one model by collecting better training examples
    Phase 3 Capstone
      Topics:
        - Choosing the capstone: a Minecraft build, a Roblox world or an AI-powered game
        - Planning with a checklist, building in small steps
        - Testing with a real player
        - Presenting: what it does, how it works, what was hard
        - A cumulative review of phases 1-3
      Projects built:
        - Phase capstone: one world or AI project, finished and demonstrated
      Practice: Prepare a two-minute demo of the capstone
      Assessment: Phase 3 exam: capstone demo plus a mixed challenge from all three phases

------------------------------------------------------------
PHASE 4: THE BRIDGE TO REAL CODE (MONTHS 9-10, WEEKS 33-40)
------------------------------------------------------------
The step every parent asks about, done gently and properly: from blocks to first typed
Python, a first web page, and a final capstone with a real exam.

  Month 9: First Typed Code (Weeks 33-36)
    From Blocks to Text
      Topics:
        - Reading the JavaScript behind your MakeCode games
        - Every block is a line of code: the big reveal
        - Typing skills for coders: a gentle start
        - What stays the same: loops, ifs and variables in any language
        - What changes: spelling matters now
        - Errors are clues: reading messages without fear
        - Choosing the right tool for each idea
      Projects built:
        - The same tiny game shown in blocks and in text, side by side
      Practice: Convert 3 small block programs to text with the toggle
    First Python with Turtle
      Topics:
        - Why learn text-based coding?
        - Python in the browser: zero setup for the first weeks
        - Python Turtle graphics
        - Variables and data types
        - Print statements and input
        - Comments and documentation
        - Basic math operations
        - String manipulation
        - Converting blocks to Python
        - Debugging Python code
      Projects built:
        - Turtle art generator
        - Mad libs game in Python
        - A talking quiz program
      Practice: Recreate 3 Scratch projects as tiny Python programs
    My First Web Page
      Topics:
        - How websites work
        - HTML structure and tags
        - Creating web pages
        - Links and navigation
        - Images and media
        - CSS introduction
        - Selectors and properties
        - Colors and fonts
        - Box model
      Projects built:
        - A hand-typed page about your favorite thing
        - Photo gallery page
        - A page about your best projects
      Practice: Build one more page about anything you love
    Creator Habits
      Topics:
        - Your project gallery: choosing your best work
        - Explaining a project in three sentences
        - Safe sharing: what a portfolio shows and what it never shows
        - Debugging rituals: what to try before asking for help
        - How coders learn new things on purpose
        - Planning the final capstone
      Projects built:
```

- A tidy portfolio of the journey's best 8 projects
Practice: Write one proud sentence for each portfolio project
Assessment: Month 9 check: a mixed challenge spanning Python, web and earlier phases

Month 10: Capstone and Demo Day (Weeks 37-40)
Capstone Planning
Topics:
- Choosing the final project: game, app, world or AI project, your call
- A one-page plan: what it is, who it is for, what makes it special
- Breaking the build into four small steps
- Collecting art, sounds and ideas before building
- The first working slice by the end of the week
Projects built:
- Capstone plan approved plus the first working slice
Practice: Show the plan to your family and collect one suggestion
Capstone Build
Topics:
- Building in small steps that always run
- Instructor checkpoints and course corrections
- Using everything: logic, art, sound, data, maybe AI
- Testing with real players and acting on what they say
- Cutting ideas kindly: finishing beats adding
- Preparing the demo: the story of the build
Projects built:
- The final capstone, built over two weeks and genuinely finished
Practice: A little building every day; always keep it working
Demo Day and What Comes Next
Topics:
- Presenting the capstone to the batch and parents
- The hardest bug story: what went wrong and how you fixed it
- The journey in numbers: projects built, skills earned
- What comes next: Python for Kids, Roblox, Minecraft or Web Development
- Keeping the spark: healthy creating habits at home
Projects built:
- Demo day presentation, delivered proudly
Assessment: FINAL EXAM: capstone demo plus a friendly viva and a mixed quiz drawn from the whole journey. Passing earns the certificate; a child who falls short gets focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: animations, interactive stories, classic game clones and an original Scratch capstone game
- Phase 2: MakeCode arcade games, micro:bit inventions, and real mobile apps installed on the family phone
- Phase 3: Minecraft Education builds, Roblox worlds with first Lua scripts, 3D models, and AI models they train themselves
- Phase 4: first Python programs, a hand-typed web page, and a final capstone presented at demo day

Total projects: 40+ real projects, from a first animation to an AI-powered capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Block coding fluency: Scratch, Code.org, MakeCode and Blockly, used with real intent
- Game design: mechanics, fairness, levels and difficulty, learned by building and playtesting
- App building: real mobile apps in MIT App Inventor, tested by real family users
- World building: Minecraft Education coding and Roblox Studio with first Lua scripts
- Kid-safe AI: training and testing image and sound models, and using them inside projects
- The bridge to text: first Python with turtle graphics and a first hand-written web page
- Thinking skills: sequencing, debugging, patience, and explaining your own work

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building along with the teacher
Homework: A small homework build after every class, and a mixed-review challenge at the end of every month
Practice: 2-3 hours of guided practice between classes, structured as play
Review: Every submitted project gets teacher feedback; recurring gaps are reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so no child stays stuck and discouraged
Progress updates: Regular progress notes to parents: what was built, what comes next, and how to encourage practice at home

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam, a project demo plus a friendly viva. A child who falls short gets focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Python for Kids: the natural next step once the block-to-text bridge is crossed
- Roblox Game Coding: deeper Lua scripting for kids who loved building worlds
- Minecraft Coding: more automation and invention inside Minecraft
- Web Development for Kids: from a first page to real interactive sites
- School wins: computational thinking that shows up in maths and science class

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does my child build in this coding course?
A: More than 40 real projects across ten months: animations and complete games in Scratch, arcade games in MakeCode, micro:bit inventions, a real mobile app built in MIT App Inventor and installed on a family phone, coding projects inside Minecraft Education, a Roblox world with first Lua scripts, AI models they train themselves, and finally their first typed Python programs and a hand-written web page. Every project is the child's own work, reviewed by the teacher.

Q: What age should my child be to start coding?
A: The course runs from age 6 to 12, and batches are grouped by age: 6 to 8 year olds start gently and stay in visual tools longer, while 9 to 12 year olds move faster and reach Roblox scripting and Python sooner. Only comfortable reading is needed, no typing skill and no prior coding.

Q: How long does it take a child to learn coding?
A: The journey from zero to the final capstone takes most children 10 months at 2 live classes a week. The syllabus deliberately keeps 2 months of margin in hand: younger children or busy school terms stretch the same journey to 12 months with no rushing and no penalty. Real progress shows immediately, a child has a working animation in the very first class.

Q: How is my child assessed? Is the certificate just for attending?
A: No. There is classwork in every session, a small homework build after every class, and a monthly challenge that mixes new questions with earlier topics so skills stay sharp instead of fading. Each phase ends with a cumulative check, and the course ends with a final exam: a capstone demo plus a friendly viva and a mixed quiz. Passing earns the certificate; a child who falls short gets focused revision and a free retest. It is all gentle, and all real: we optimize for real knowledge, not a stream of feel-good badges.

Q: Is this just more screen time?
A: It is the opposite kind of screen time. Instead of consuming videos and games, children spend the hours making them: designing, building, testing and fixing their own projects. Parents consistently report the shift, the child starts asking how things work instead of just asking for the tablet. Safe digital habits are taught explicitly alongside.

Q: How is AI taught to children this young?
A: Hands-on and kid-safe. Children train their own image and sound models using Machine Learning for Kids and Google's Teachable Machine, both built for classrooms, then use those models inside their own games, a pet that reacts to sounds, a game controlled by gestures. There are no open chatbots in the kids' course; the point is understanding how AI learns, including how it gets things wrong.

Q: Which platforms are used, and are they free?
A: The core platforms, Scratch, Code.org, MakeCode, MIT App Inventor, Tinkercad and Teachable Machine, are free and browser-based. Minecraft Education and Roblox Studio are free to build in, with accounts set up together with a parent. A computer or laptop with internet is enough; no hardware kits are required, the micro:bit is taught in its full browser simulator.

Q: What happens after this course?
A: The final phase builds the bridge on purpose: children finish having already written first Python programs and a first web page. From there they step naturally into our Python for Kids, Roblox Game Coding, Minecraft Coding or Web Development for Kids courses, choosing the direction that pulled them hardest during the journey.

Q: Is there a free trial class?
A: Yes. You can book a free demo at learn.modernagecoders.com/contact with zero commitment, and many parents sit in on the demo to see the teaching style for themselves. We hold a 4.9 rating across 547 Google reviews, and the demo class is where most of those families started. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are these live classes or videos?
A: Every session is a live 1-hour online class taught by a real teacher; we never substitute

pre-recorded videos. Groups stay small and interactive, so children build along during class rather than just watching, and the teacher sees and fixes small confusions before they grow.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/kids-coding-blocks-masterclass
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 42 OF 137: COLLEGE MATHS COURSE: CALCULUS, LINEAR ALGEBRA TO ANALYSIS
==========

Course link: https://learn.modernagecoders.com/courses/college-mathematics-complete-masterclass
Subject area: College Mathematics
Designed for: College students, Girls only batches
Who can join: College and university students; placement by current course and semester
Course length: 10-12 months per track (3 tracks, joinable any month)
Time needed: 2 live classes/week + 4-6 hours problem practice
Certificate: Track certificate from Modern Age Coders, awarded on passing each track exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

College maths course, live online: three academic-year tracks from calculus and linear algebra to probability, real analysis and abstract algebra.

In one line: The undergraduate core, taught the way it should have been: three academic-year tracks, placed against your actual syllabus, proofs included.

ABOUT THE PROGRAMME

University mathematics has a famous problem: lectures move at the speed of the syllabus, not the student, and one missed fortnight of calculus compounds into a lost semester. This live online program is the counterweight. It maps the undergraduate core into three honest tracks, each one academic year long, and places every student by their current course and semester, an engineering first-year lands in Track 1 calculus, a BSc student facing real analysis lands in Track 3. Track 1 is the first-year core: single-variable calculus done properly, linear algebra from vectors to eigenvalues, proof writing, and multivariable calculus. Track 2 is the methods year: multiple integration, ordinary differential equations and Laplace transforms, discrete mathematics, and a full treatment of probability and statistics. Track 3 is where mathematics becomes a discipline: real analysis, abstract algebra, topology and complex analysis, taught with rigorous proofs but at a human pace, with the intuition drawn before the epsilon arrives. Each track runs 10 months at two live classes a week plus four to six hours of problem practice, with up to 12 months for students who need more, the pace adapts. Assessment is honest: problem sets after every class, monthly mixed reviews that reach back deliberately, and a track exam at the end that decides the certificate, with focused revision and a free retest for those who fall short. Real knowledge, real proofs, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Placement by syllabus, not marketing: a short diagnostic maps you to the right track and the right month
- Proofs taught as a skill: intuition first, then rigor, the way strong departments actually teach it
- Built to run alongside university: the tracks mirror standard BSc and engineering syllabi, so class work and university work reinforce each other
- Problem practice over passive watching: every concept is drilled through graded problem sets with written feedback
- An honest year per track with 2 months of margin: students who need more time take up to 12 months, nobody is rushed
- Real assessment: weekly problem sets, monthly mixed reviews, and a track exam that decides the certificate, with focused revision and a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Track 1, the first-year core (one academic year): calculus I-II, linear algebra, proof writing and multivariable calculus
Phase 2: Track 2, the methods year (one academic year): multiple integrals, ODEs and Laplace, discrete mathematics, probability and statistics
Phase 3: Track 3, the discipline (one academic year): real analysis, abstract algebra, topology and complex analysis

WHAT A STUDENT NEEDS BEFORE STARTING

Maths background: Track 1 assumes school mathematics (Class 12 or equivalent). Tracks 2 and 3 assume the earlier track or its university equivalent; placement checks this honestly
Equipment: A computer or tablet with a stable internet connection; paper and pen remain the real tools
Age: College and university students, and returning learners rebuilding their mathematics
Time commitment: 2 live classes plus 4-6 hours of problem practice a week

WHO THIS COURSE IS FOR

Engineering students: First and second years drowning in the calculus-to-ODEs pipeline their branch assumes
Bsc msc students: Mathematics majors who want analysis and algebra taught with patience the lecture hall never has
Placement aspirants: Students strengthening the quantitative core for data science and analytics paths
Returning learners: Graduates and professionals rebuilding real mathematics for research or higher study
Proof beginners: Anyone hitting the proofs wall for the first time and wanting to climb it properly

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: TRACK 1, CALCULUS AND LINEAR ALGEBRA (ONE ACADEMIC YEAR)

The first-year core: limits to integrals, vectors to eigenvalues, the craft of proof, and multivariable calculus, built with intuition before rigor.

Months 1-2: Single Variable Calculus I (Weeks 1-8)
Limits and Continuity
Topics:
- Review of functions: domain, range, composition
- Inverse functions and their properties
- Exponential and logarithmic functions
- Trigonometric functions and identities
- Intuitive notion of limits
- Formal epsilon-delta definition of limits
- Limit laws and computation techniques
- One-sided limits and infinite limits
- Limits at infinity and horizontal asymptotes
- Continuity at a point and on intervals
- Intermediate Value Theorem
- Types of discontinuities
Projects built:
- Epsilon-delta proof visualizer
- Limit calculator implementation
- Continuity explorer interactive tool
Practice: Complete 100 limit problems with epsilon-delta proofs
Differentiation
Topics:
- Definition of derivative as a limit
- Geometric interpretation: tangent lines
- Physical interpretation: rates of change
- Differentiability and continuity relationship
- Basic differentiation rules: power, constant, sum
- Product rule and quotient rule
- Chain rule and implicit differentiation
- Derivatives of trigonometric functions
- Derivatives of exponential and logarithmic functions
- Inverse function derivatives
- Higher-order derivatives
- Logarithmic differentiation
Projects built:
- Derivative visualizer application
- Automatic differentiation engine
- Physics motion simulator
Practice: Master 200 differentiation problems including proofs
Applications of Derivatives
Topics:
- Linear approximation and differentials
- L'Hôpital's Rule for indeterminate forms
- Critical points and extrema
- First Derivative Test
- Second Derivative Test

```
      - Concavity and inflection points
      - Curve sketching comprehensive method
      - Optimization problems
      - Related rates problems
      - Mean Value Theorem
      - Rolle's Theorem
      - Newton's Method for root finding
    Projects built:
      - Optimization problem solver
      - Curve sketching software
      - Newton's method visualizer
    Practice: Solve 150 application problems with full solutions
  Integration Fundamentals
    Topics:
      - Antiderivatives and indefinite integrals
      - Definite integrals and Riemann sums
      - Fundamental Theorem of Calculus Part I
      - Fundamental Theorem of Calculus Part II
      - Basic integration rules
      - U-substitution method
      - Integration of trigonometric functions
      - Integration of exponential and logarithmic functions
      - Area between curves
      - Volumes by cross-sections
      - Volumes by cylindrical shells
      - Average value of a function
    Projects built:
      - Riemann sum visualizer
      - Numerical integration calculator
      - Volume calculator for solids of revolution
    Practice: Complete 200 integration problems
    Assessment: Months 1-2 check: a mixed calculus paper, limits through integrals, with
    full written feedback

Months 3-4: Calculus II and Linear Algebra I (Weeks 9-17)
  Advanced Integration Techniques
    Topics:
      - Integration by parts
      - Trigonometric integrals and substitutions
      - Partial fraction decomposition
      - Rational function integration
      - Improper integrals Type I and II
      - Comparison tests for improper integrals
      - Numerical integration: Trapezoidal rule
      - Simpson's rule and error bounds
      - Arc length calculations
      - Surface area of revolution
      - Work, force, and energy applications
      - Center of mass and moments
    Projects built:
      - Advanced integration solver
      - Numerical methods comparison tool
      - Physics applications simulator
    Practice: Master 150 advanced integration problems
  Sequences and Series
    Topics:
      - Sequences: convergence and divergence
      - Monotonic sequences and bounded sequences
      - Series and partial sums
      - Geometric series and telescoping series
      - Divergence test and p-series
      - Comparison tests and limit comparison test
      - Ratio test and root test
      - Alternating series test and error bounds
      - Absolute vs conditional convergence
      - Power series and radius of convergence
      - Taylor and Maclaurin series
      - Taylor polynomial approximations and error
    Projects built:
      - Series convergence analyzer
      - Taylor series visualizer
      - Function approximation tool
    Practice: Analyze 100 series for convergence with proofs
  Linear Algebra: Vectors and Matrices
    Topics:
      - Vector spaces: R^n and axioms
      - Vector operations: addition, scalar multiplication
      - Dot product and cross product
      - Vector projections and orthogonality
```

- Linear combinations and span
- Linear independence and dependence
- Matrix operations: addition, multiplication
- Matrix transpose and properties
- Special matrices: identity, diagonal, symmetric
- Elementary row operations
- Row echelon form and reduced row echelon form
- Gaussian elimination algorithm

Projects built:

- Matrix calculator implementation
- Vector visualization tool
- Gaussian elimination solver

Practice: Complete 150 linear algebra computations

Systems of Linear Equations

Topics:

- Consistent vs inconsistent systems
- Homogeneous systems and nontrivial solutions
- Matrix representation of systems
- Augmented matrices and RREF
- Parametric solutions and free variables
- Matrix inverses and invertibility
- Computing inverses using row operations
- Determinants: definition and properties
- Cofactor expansion and row operations
- Cramer's rule for solving systems
- Applications to network flow
- Applications to economics and engineering

Projects built:

- Linear system solver application
- Determinant calculator
- Network flow optimizer

Practice: Solve 100 systems using various methods

Mathematical Proof Writing

Topics:

- Logic and truth tables
- Quantifiers: universal and existential
- Direct proof technique
- Proof by contradiction
- Proof by contrapositive
- Mathematical induction
- Strong induction and well-ordering
- Proof by cases
- Existence and uniqueness proofs
- Counterexamples
- Writing clear mathematical arguments
- Common proof strategies and patterns

Projects built:

- Proof verification system
- Logic truth table generator
- Induction proof template builder

Practice: Write 50 rigorous mathematical proofs

Assessment: Mid-track check: proof writing plus mixed questions reaching back to month 1

Months 5-6: Advanced Linear Algebra and Vector Calculus (Weeks 18-26)

Eigenvalues and Eigenvectors

Topics:

- Eigenvalue and eigenvector definition
- Characteristic polynomial
- Finding eigenvalues and eigenvectors
- Algebraic and geometric multiplicity
- Diagonalization of matrices
- Powers of diagonalizable matrices
- Orthogonal matrices and rotations
- Symmetric matrices and spectral theorem
- Quadratic forms
- Principal axes theorem
- Applications to differential equations
- Applications to Markov chains

Projects built:

- Eigenvalue calculator and visualizer
- Matrix diagonalization tool
- Markov chain simulator

Practice: Find eigenvalues/eigenvectors for 100 matrices

Vector Spaces and Linear Transformations

Topics:

- Abstract vector spaces and subspaces
- Basis and dimension
- Coordinate systems and change of basis

- Row space, column space, null space
- Rank-nullity theorem
- Linear transformations: definition and properties
- Kernel and image of linear transformation
- Matrix representation of linear transformations
- Composition of linear transformations
- Isomorphisms and invertible transformations
- Similarity of matrices
- Jordan canonical form introduction

Projects built:
- Linear transformation visualizer
- Change of basis calculator
- Subspace dimension finder

Practice: Analyze 75 linear transformations

Multivariable Functions and Partial Derivatives

Topics:
- Functions of several variables
- Level curves and level surfaces
- Limits in multiple dimensions
- Continuity for multivariable functions
- Partial derivatives: definition and notation
- Higher-order partial derivatives
- Clairaut's theorem on mixed partials
- Tangent planes and linear approximation
- The gradient vector and directional derivatives
- The chain rule for multivariable functions
- Implicit differentiation in multiple variables
- Jacobian matrices

Projects built:
- 3D function visualizer
- Gradient field plotter
- Tangent plane calculator

Practice: Compute 150 partial derivatives and gradients

Optimization and Vector Fields

Topics:
- Critical points in multiple dimensions
- Second derivative test for two variables
- Hessian matrix and definiteness
- Global vs local extrema
- Constrained optimization: Lagrange multipliers
- Multiple constraints
- Vector fields and flow lines
- Conservative vector fields
- Potential functions
- Divergence and curl
- Physical interpretations
- Applications to physics and engineering

Projects built:
- Multivariable optimization solver
- Vector field simulator
- Lagrange multiplier visualizer

Practice: Solve 100 optimization problems

Track 1 Exam

Topics:
- Integration of calculus and linear algebra
- Comprehensive problem solving
- Proof portfolio creation
- Mathematical writing
- Presentation skills

Projects built:
- MAJOR CAPSTONE: Mathematical Modeling Project
- Options: Population dynamics, Economic models, Engineering systems
- Complete proof portfolio with 25 original proofs
- Implementation of numerical methods library

Assessment: TRACK 1 EXAM: a written paper mixing the whole track, calculus, linear algebra and one honest proof. Passing earns the Track 1 certificate; falling short earns focused revision and a free retest

---

PHASE 2: TRACK 2, DIFFERENTIAL EQUATIONS, DISCRETE MATHS AND PROBABILITY (ONE ACADEMIC YEAR)

The methods year every engineering and science degree leans on: multiple integration, ODEs and Laplace transforms, discrete mathematics, and probability and statistics done properly.

Months 1-2: Multiple Integration and ODEs (Weeks 1-9)

Double and Triple Integrals

Topics:
- Double integrals over rectangles
- Iterated integrals and Fubini's theorem

    - Double integrals over general regions
    - Reversing order of integration
    - Double integrals in polar coordinates
    - Applications: area, volume, mass
    - Center of mass and moments of inertia
    - Triple integrals in rectangular coordinates
    - Triple integrals in cylindrical coordinates
    - Triple integrals in spherical coordinates
    - Change of variables: Jacobians
    - Applications to probability and physics
  Projects built:
    - Multiple integral calculator
    - Volume visualization tool
    - Coordinate system converter
  Practice: Evaluate 150 multiple integrals
Line and Surface Integrals
  Topics:
    - Parametric curves and arc length
    - Line integrals of scalar functions
    - Line integrals of vector fields
    - Work and circulation
    - Fundamental theorem for line integrals
    - Green's theorem and applications
    - Parametric surfaces
    - Surface area calculations
    - Surface integrals of scalar functions
    - Surface integrals of vector fields (flux)
    - Stokes' theorem
    - Divergence theorem (Gauss's theorem)
  Projects built:
    - Line integral visualizer
    - Surface parametrization tool
    - Vector calculus theorem demonstrator
  Practice: Complete 100 vector calculus problems
First-Order Differential Equations
  Topics:
    - Classification of differential equations
    - Direction fields and solution curves
    - Separable equations
    - Linear first-order equations
    - Integrating factors method
    - Exact equations and exactness test
    - Homogeneous equations
    - Bernoulli equations
    - Existence and uniqueness theorems
    - Numerical methods: Euler's method
    - Improved Euler and Runge-Kutta
    - Applications to physics, biology, economics
  Projects built:
    - ODE solver implementation
    - Direction field plotter
    - Numerical methods comparison
  Practice: Solve 150 first-order ODEs
Higher-Order Linear ODEs
  Topics:
    - Second-order linear equations
    - Homogeneous equations with constant coefficients
    - Characteristic equation method
    - Complex roots and oscillations
    - Repeated roots and reduction of order
    - Nonhomogeneous equations
    - Method of undetermined coefficients
    - Variation of parameters
    - Higher-order linear equations
    - Systems of first-order linear equations
    - Matrix exponentials
    - Phase plane analysis
  Projects built:
    - Second-order ODE solver
    - Oscillation simulator
    - Phase portrait generator
  Practice: Solve 100 higher-order ODEs
Laplace Transforms
  Topics:
    - Definition and existence of Laplace transform
    - Transforms of basic functions
    - Linearity and shifting theorems
    - Transforms of derivatives and integrals
    - Inverse Laplace transforms

```
      - Partial fraction decomposition for inverse transforms
      - Convolution theorem
      - Solving ODEs with Laplace transforms
      - Systems of ODEs via Laplace
      - Transfer functions
      - Applications to control theory
      - Discontinuous forcing functions
    Projects built:
      - Laplace transform calculator
      - Control system analyzer
      - ODE system solver via Laplace
    Practice: Apply Laplace transforms to 75 problems
    Assessment: Months 1-2 check: an ODE-and-integration paper with one modeling question

Months 3-4: Discrete Mathematics (Weeks 10-18)
  Set Theory and Relations
    Topics:
      - Naive set theory and paradoxes
      - Set operations: union, intersection, complement
      - Power sets and Cartesian products
      - Functions as relations
      - Injective, surjective, bijective functions
      - Cardinality and countability
      - Cantor's diagonal argument
      - Relations: reflexive, symmetric, transitive
      - Equivalence relations and partitions
      - Partial orders and Hasse diagrams
      - Well-ordering principle
      - Axiom of choice introduction
    Projects built:
      - Set operation visualizer
      - Relation property checker
      - Cardinality comparison tool
    Practice: Prove 75 set theory theorems
  Combinatorics
    Topics:
      - Basic counting principles
      - Permutations and combinations
      - Binomial theorem and Pascal's triangle
      - Multinomial coefficients
      - Inclusion-exclusion principle
      - Pigeonhole principle
      - Generating functions
      - Recurrence relations
      - Solving linear recurrences
      - Catalan numbers
      - Stirling numbers
      - Partitions of integers
    Projects built:
      - Combinatorics calculator
      - Recurrence relation solver
      - Generating function manipulator
    Practice: Solve 100 combinatorics problems
  Graph Theory
    Topics:
      - Graphs: vertices, edges, degree
      - Types of graphs: simple, directed, weighted
      - Graph representations: adjacency matrix, list
      - Paths, cycles, and connectivity
      - Trees and spanning trees
      - Minimum spanning trees: Kruskal, Prim
      - Shortest paths: Dijkstra, Bellman-Ford
      - Eulerian and Hamiltonian paths
      - Graph coloring and chromatic number
      - Planar graphs and Euler's formula
      - Bipartite graphs and matching
      - Network flows and max-flow min-cut
    Projects built:
      - Graph algorithm visualizer
      - Network flow optimizer
      - Graph coloring solver
    Practice: Implement 20 graph algorithms
  Number Theory
    Topics:
      - Divisibility and greatest common divisors
      - Euclidean algorithm and extended version
      - Prime numbers and fundamental theorem
      - Modular arithmetic
      - Chinese Remainder Theorem
```

- Fermat's Little Theorem
- Euler's theorem and totient function
- Wilson's theorem
- Quadratic residues
- Primitive roots
- RSA cryptography basics
- Diophantine equations

Projects built:

- Number theory toolkit
- RSA encryption implementation
- Prime number generator

Practice: Prove 50 number theory results

Boolean Algebra and Logic

Topics:

- Boolean operations and laws
- Truth tables and logical equivalence
- Normal forms: DNF and CNF
- Karnaugh maps
- Logic gates and circuits
- Minimization of Boolean functions
- Propositional logic and inference rules
- Predicate logic and quantifiers
- Formal proofs in logic
- Completeness and soundness
- Introduction to model theory
- Applications to computer science

Projects built:

- Logic circuit simulator
- Boolean function minimizer
- Automated theorem prover

Practice: Design 30 logic circuits

Assessment: Months 3-4 check: a discrete mathematics paper, with revision questions from months 1-2

Months 5-6: Probability, Statistics and the Track Exam (Weeks 19-26)

Probability Theory

Topics:

- Sample spaces and events
- Probability axioms and properties
- Conditional probability
- Bayes' theorem and applications
- Independence of events
- Random variables: discrete and continuous
- Probability mass and density functions
- Cumulative distribution functions
- Expected value and variance
- Moment generating functions
- Common discrete distributions
- Common continuous distributions

Projects built:

- Probability distribution visualizer
- Bayes' theorem calculator
- Monte Carlo simulator

Practice: Solve 150 probability problems

Joint Distributions and Limit Theorems

Topics:

- Joint probability distributions
- Marginal and conditional distributions
- Independence of random variables
- Covariance and correlation
- Conditional expectation
- Transformations of random variables
- Order statistics
- Law of large numbers
- Central limit theorem
- Normal approximations
- Characteristic functions
- Convergence concepts

Projects built:

- Joint distribution plotter
- CLT demonstration tool
- Correlation analyzer

Practice: Analyze 100 joint distributions

Statistical Inference

Topics:

- Point estimation: MLE and method of moments
- Properties of estimators: bias, consistency
- Confidence intervals
- Hypothesis testing framework

        - Type I and Type II errors
        - Power of tests
        - t-tests and chi-square tests
        - ANOVA basics
        - Nonparametric tests
        - Linear regression
        - Multiple regression
        - Model diagnostics
      Projects built:
        - Statistical test suite
        - Regression analysis tool
        - Power analysis calculator
      Practice: Perform 75 statistical analyses
    Stochastic Processes
      Topics:
        - Introduction to stochastic processes
        - Markov chains: discrete time
        - Transition matrices and steady states
        - Classification of states
        - Absorbing Markov chains
        - Continuous-time Markov chains
        - Poisson processes
        - Birth-death processes
        - Queueing theory basics
        - Brownian motion introduction
        - Random walks
        - Applications to finance and biology
      Projects built:
        - Markov chain simulator
        - Queue system analyzer
        - Random walk visualizer
      Practice: Model 50 stochastic systems
    Track 2 Exam
      Topics:
        - Integration of discrete and continuous mathematics
        - Statistical modeling project
        - Algorithm implementation
        - Research paper reading
        - Technical presentation
      Projects built:
        - MAJOR CAPSTONE: Applied Mathematics Research Project
        - Options: Machine learning algorithm, Cryptographic system, Statistical study
        - Graph theory application to real networks
        - Probability model for real phenomenon
      Assessment: TRACK 2 EXAM: a written paper mixing ODEs, discrete mathematics and
      probability, with questions reaching back across the year. Passing earns the Track 2
      certificate; falling short earns focused revision and a free retest

---
PHASE 3: TRACK 3, ANALYSIS, ALGEBRA AND BEYOND (ONE ACADEMIC YEAR)

Where mathematics becomes a discipline: real analysis, abstract algebra, topology and complex
analysis, rigorous but humane, intuition drawn before every epsilon.

  Months 1-3: Real Analysis (Weeks 1-9)
    Metric Spaces and Sequences
      Topics:
        - Metric spaces: definition and examples
        - Open and closed sets
        - Interior, closure, and boundary
        - Convergent sequences in metric spaces
        - Cauchy sequences and completeness
        - Compactness: sequential and open cover
        - Heine-Borel theorem
        - Connected spaces
        - Continuous functions between metric spaces
        - Uniform continuity
        - Lipschitz continuity
        - Homeomorphisms
      Projects built:
        - Metric space visualizer
        - Compactness checker
        - Continuity analyzer
      Practice: Prove 100 metric space theorems
    Limits and Continuity in R^n
      Topics:
        - The real numbers: completeness axiom
        - Supremum and infimum
        - Sequences and series of real numbers

- Limit superior and limit inferior
- Functions of real variables
- Continuity and uniform continuity on R
- Intermediate value theorem proof
- Extreme value theorem proof
- Monotone functions
- Functions of bounded variation
- Absolutely continuous functions
- Discontinuities classification

Projects built:

- Real function analyzer
- Continuity proof assistant
- Convergence tester

Practice: Complete 75 epsilon-delta proofs

Differentiation Theory

Topics:

- Derivative as linear approximation
- Differentiability in R^n
- Partial derivatives vs differentiability
- Chain rule proof
- Mean value theorem and generalizations
- Taylor's theorem with remainder
- Inverse function theorem
- Implicit function theorem
- Critical point analysis
- Morse lemma
- Sard's theorem introduction
- Applications to optimization

Projects built:

- Differentiation proof checker
- Taylor approximation tool
- Critical point classifier

Practice: Prove 50 differentiation theorems

Riemann Integration

Topics:

- Riemann integral construction
- Riemann integrability conditions
- Properties of Riemann integral
- Fundamental theorem of calculus proof
- Integration techniques review
- Improper Riemann integrals
- Functions of bounded variation
- Riemann-Stieltjes integral
- Convergence theorems limitations
- Counterexamples in integration
- Numerical integration error analysis
- Applications to probability

Projects built:

- Riemann sum visualizer
- Integrability checker
- Numerical integration analyzer

Practice: Analyze 75 integration problems rigorously

Sequences and Series of Functions

Topics:

- Pointwise convergence
- Uniform convergence
- Weierstrass M-test
- Continuity of uniform limits
- Integration of uniform limits
- Differentiation of uniform limits
- Power series and radius of convergence
- Analytic functions
- Weierstrass approximation theorem
- Stone-Weierstrass theorem
- Fourier series introduction
- Convergence of Fourier series

Projects built:

- Convergence visualizer
- Fourier series calculator
- Function approximator

Practice: Prove convergence for 50 function sequences

Assessment: Analysis check: two full proofs written under exam conditions, marked line by line

Months 4-6: Abstract Algebra (Weeks 10-17)

Group Theory

Topics:

- Groups: axioms and examples
- Subgroups and subgroup tests

- Cyclic groups and generators
- Group homomorphisms
- Kernel and image
- Isomorphism theorems
- Normal subgroups and quotient groups
- Lagrange's theorem
- Group actions
- Orbit-stabilizer theorem
- Sylow theorems
- Classification of finite abelian groups

Projects built:

- Group calculator
- Cayley table generator
- Subgroup lattice visualizer

Practice: Prove 100 group theory results

Ring Theory

Topics:

- Rings: definition and examples
- Ring homomorphisms
- Ideals and quotient rings
- Prime and maximal ideals
- Integral domains
- Principal ideal domains
- Unique factorization domains
- Euclidean domains
- Polynomial rings
- Field of fractions
- Chinese remainder theorem for rings
- Localization of rings

Projects built:

- Ring structure analyzer
- Ideal calculator
- Polynomial factorization tool

Practice: Explore 75 ring structures

Field Theory

Topics:

- Fields: definition and examples
- Field extensions
- Algebraic and transcendental elements
- Degree of field extensions
- Splitting fields
- Algebraic closure
- Finite fields structure
- Cyclotomic fields
- Galois theory introduction
- Fundamental theorem of Galois theory
- Solvability by radicals
- Constructions with ruler and compass

Projects built:

- Field extension visualizer
- Galois group calculator
- Constructibility checker

Practice: Work through 50 field theory problems

Linear Algebra (Abstract)

Topics:

- Vector spaces over arbitrary fields
- Linear transformations abstract theory
- Dual spaces and dual basis
- Bilinear forms
- Quadratic forms
- Inner product spaces
- Orthogonalization: Gram-Schmidt
- Adjoint operators
- Normal operators
- Spectral theorem
- Singular value decomposition
- Tensor products

Projects built:

- Abstract linear algebra toolkit
- Inner product space explorer
- SVD calculator

Practice: Prove 75 abstract linear algebra theorems

Assessment: Algebra check: a structures paper spanning groups, rings and fields, plus one analysis revision question

Months 7-10: Topology, Complex Analysis and the Track Exam (Weeks 18-26)

General Topology

Topics:

- Topological spaces: definition and examples

- Basis and subbasis
- Closed sets and closure operators
- Interior and boundary
- Hausdorff spaces
- Continuous functions
- Homeomorphisms
- Connectedness
- Path connectedness
- Compactness
- Tychonoff's theorem
- Separation axioms

Projects built:

- Topology visualizer
- Homeomorphism checker
- Compactness prover

Practice: Prove 100 topology theorems

Complex Analysis

Topics:

- Complex numbers and complex plane
- Complex functions and analyticity
- Cauchy-Riemann equations
- Harmonic functions
- Complex integration
- Cauchy's theorem
- Cauchy integral formula
- Taylor and Laurent series
- Residues and residue theorem
- Evaluation of real integrals
- Conformal mappings
- Möbius transformations

Projects built:

- Complex function visualizer
- Residue calculator
- Conformal mapping tool

Practice: Solve 100 complex analysis problems

Advanced Complex Analysis

Topics:

- Maximum modulus principle
- Schwarz lemma
- Analytic continuation
- Riemann surfaces introduction
- Entire functions
- Meromorphic functions
- Infinite products
- Weierstrass factorization
- Gamma function
- Riemann zeta function
- Prime number theorem outline
- Applications to number theory

Projects built:

- Special functions library
- Riemann surface visualizer
- Zeta function explorer

Practice: Explore 50 special functions

Track 3 Capstone and Exam

Topics:

- Pure mathematics research project
- Original proof development
- Mathematical exposition
- Research paper writing
- Peer review process

Projects built:

- A written capstone: one substantial theorem, its proof reconstructed and presented from understanding

Assessment: TRACK 3 EXAM: a proof-based paper across analysis, algebra, topology and complex analysis. Passing earns the Track 3 certificate, completing the program; falling short earns focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Track 1: applied optimization studies, an eigenvalue applications mini-project, and a proof portfolio
- Track 2: an ODE modeling project, graph theory applications, and a full statistical inference study on real data
- Track 3: a theorem capstone, reconstructed and presented, plus a growing portfolio of original proof write-ups

Total projects: A year-long problem-set portfolio per track, plus one substantial capstone project each

SKILLS THE STUDENT WILL HAVE AT THE END

- Calculus: limits through multivariable, computed fluently and understood geometrically
- Linear algebra: from solving systems to eigenvalues, vector spaces and transformations
- Differential equations: first and higher order ODEs, Laplace methods, and real modeling judgment
- Discrete mathematics: sets, combinatorics, graphs, number theory and logic
- Probability and statistics: distributions, limit theorems and honest statistical inference
- Real analysis: rigorous limits, continuity, differentiation and Riemann integration
- Abstract algebra: groups, rings and fields, with the structural thinking they train
- Proof craft: reading, writing and presenting mathematics like a mathematician

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, problems worked together, not lectured at
Homework: A problem set after every class, and a monthly mixed review that reaches back deliberately
Practice: 4-6 hours of graded problem practice between classes
Review: Every set marked with written feedback; recurring weaknesses reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, for the proof that refuses to close at midnight
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: A track certificate at the end of each track, awarded on passing the track exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Graduate study: Track 3 is precisely the core masters admissions and entrance tests assume
- Data science and machine learning: the linear algebra, probability and optimization this program builds is the honest prerequisite
- Quantitative finance and analytics: probability and methods from Track 2 carry directly
- Research foundations: proof craft and analysis are the entry ticket to serious mathematics
- Teaching and mentoring: mathematics understood deeply enough to explain simply

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does the College Mathematics program cover?
A: The undergraduate core, mapped into three academic-year tracks. Track 1: single-variable calculus, linear algebra, proof writing and multivariable calculus. Track 2: multiple integration, ordinary differential equations with Laplace transforms, discrete mathematics, and probability and statistics. Track 3: real analysis, abstract algebra, topology and complex analysis. You join the track and month that match your current syllabus, not a fixed conveyor belt.

Q: Which track should I join?
A: A short diagnostic and a conversation about your current semester decide it. Engineering first-years usually land in Track 1, second-years facing ODEs and probability in Track 2, and BSc mathematics students meeting analysis and algebra in Track 3. Since each track is joinable any month, you can also enter mid-track exactly where your university syllabus is hurting.

Q: How long does each track take?
A: One academic year: 10 months at 2 live classes a week plus 4 to 6 hours of problem practice, with up to 12 months for students who need more, the margin is built in deliberately. Nobody needs all three tracks at once; most students take the track their degree is currently testing them on.

Q: How is progress measured? Is the certificate just for attending?
A: No. Problem sets follow every class and come back marked with written feedback, monthly reviews deliberately mix current topics with earlier ones so nothing fades, and each track ends with a real exam, computational and proof-based, that decides the track certificate. Falling short earns a focused revision plan and a free retest. We optimize for real mathematical knowledge, not attendance.

Q: I am scared of proofs. Is this course for me?
A: Especially for you. Proof writing is taught as its own skill starting in Track 1, with intuition drawn first and rigor layered on: read proofs, complete proofs with gaps, then write your own with line-by-line feedback. Most students who fear proofs were simply never taught them as a craft; the fear rarely survives a term of doing it properly.

Q: Can this run alongside my university semester?
A: That is the design. The tracks mirror standard BSc and engineering syllabi, so the class becomes structured reinforcement of exactly what your lectures assume. For tight semester alignment, the 1-on-1 option follows your university's syllabus and past papers directly.

Q: Is this useful preparation for graduate school?
A: Yes, concretely: Track 3's analysis and algebra are the core that masters admissions, entrance tests and first-year graduate courses assume, and the proof portfolio built across the program is exactly the muscle those programs test. The pure-mathematics emphasis is deliberate; methods without rigor stop working at the graduate gate.

Q: Can I try a class before committing?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Bring your current problem set; the demo is most useful when it works on your actual mathematics. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are the classes live or recorded lectures?
A: Live, always: small groups where problems are worked together and your written mathematics gets read and corrected in real time, never pre-recorded lectures. Mathematics at this level is learned by doing under feedback; recordings cannot mark your proof.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/college-mathematics-complete-masterclass
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 43 OF 137: COMPETITIVE PROGRAMMING COURSE: CODEFORCES TO ICPC

==========

Course link: https://learn.modernagecoders.com/courses/competitive-programming-masterclass-college
Subject area: Programming
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Competitive programming course for students and professionals: C++ and STL speed, number theory, graphs, advanced dynamic programming and contest technique, from beginner to a strong rating, in 10-12 months.

In one line: From your first contest to a rating you are proud of: C++ speed, the algorithmic arsenal, and the contest technique nobody teaches you alone.

ABOUT THE PROGRAMME

Competitive programming is where algorithmic skill becomes fast, precise and fearless, and it pays off far beyond contests: the same technique wins placement rounds, quant interviews and hard engineering problems. This live online masterclass builds it in the order that actually works: C++ with STL and fast I/O, complexity instincts and the core structures, then the arsenal, number theory, combinatorics, tree and graph algorithms, dynamic programming from basics to bitmask and DP-on-trees, string algorithms, computational geometry, then the part solo learners never crack: contest strategy, speed, template discipline and reading problems under pressure, up through expert-level techniques like FFT and advanced graph theory. A finale month runs a full contest simulation and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of solving, with two more in hand. Problem sets after every class, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- Contest technique taught explicitly: strategy, speed and template discipline, not just algorithms
- C++ and STL for competitive speed, the arena standard
- The full arsenal: number theory, geometry, advanced DP and expert graph theory

- Real contests during the course, debriefed honestly, no rating promises
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Months 1-3): C++ and STL, math, number theory, core structures and DP basics
Phase 2: The arsenal (Months 4-6): advanced DP, strings, geometry, data structures and contest strategy
Phase 3: Expert level (Months 7-9): FFT, linear algebra, advanced string and graph theory, peak performance
Phase 4: The finale (Month 10): a full contest simulation and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Everything is taught from the first line
Equipment: Any computer with a stable internet connection; every tool used is free
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

College students: Students who want contest rating and the interview edge it brings
Aspiring competitors: Coders aiming for Codeforces rating, ICPC or national olympiads
Placement focused: Candidates who want the fastest, sharpest problem-solving for hard rounds
Quant bound: Learners targeting quant roles that test exactly this speed
Self taught plateaued: Solo practitioners stuck at a rating who need real technique

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & PROBLEM SOLVING BASICS (MONTHS 1-2, WEEKS 1-8)

Build strong foundations in C++ STL, basic algorithms, and develop problem-solving mindset.

Month 1: C++ Mastery & Basic Algorithms (Weeks 1-5)

C++ STL & Fast I/O

Topics:
- Competitive programming introduction and platforms
- Why C++ for competitive programming
- Fast I/O techniques: ios_base, cin.tie
- C++ STL containers: vector, set, map, queue, stack
- STL algorithms: sort, binary_search, lower_bound, upper_bound
- Pairs and tuples usage
- Priority queue and custom comparators
- Multiset and multimap
- Unordered containers and hashing
- Bitset for optimization
- Template programming basics
- Macro definitions and shortcuts

Projects built:
- CP template creation
- STL practice problems set
- Fast I/O benchmarking

Practice: Solve 50 STL-based problems

Time Complexity & Basic Algorithms

Topics:
- Time and space complexity analysis
- Big O notation mastery
- Common time complexities in CP
- Estimating execution time from complexity
- Greedy algorithms and proofs
- Two pointers technique
- Sliding window problems
- Prefix sums and difference arrays
- Binary search on answer
- Ternary search basics
- Meet in the middle technique
- Coordinate compression

Projects built:
- Complexity analyzer tool
- Algorithm visualization
- Problem approach templates

Practice: Master 60 basic algorithm problems

Basic Data Structures

```
    Topics:
      - Arrays and vectors optimization
      - Linked lists in competitive programming
      - Stacks and queues applications
      - Deque and its uses
      - Hash tables and collision handling
      - Custom hash functions
      - Binary heap implementation
      - Disjoint Set Union (DSU) basics
      - Sparse tables introduction
      - Square root decomposition basics
      - Monotonic stack and queue
      - Policy based data structures
    Projects built:
      - Data structure library
      - Custom implementations
      - Performance comparison tool
    Practice: Implement and use 40 DS problems
  Recursion & Backtracking
    Topics:
      - Recursion optimization techniques
      - Memoization strategies
      - Backtracking framework
      - Pruning techniques
      - N-Queens and variations
      - Sudoku solver optimization
      - Permutation generation
      - Combination generation
      - Subset generation optimized
      - Bitmask techniques basics
      - State space reduction
      - Iterative vs recursive approaches
    Projects built:
      - Backtracking solver framework
      - Optimization comparisons
      - Contest problem generator
    Practice: Solve 50 recursion/backtracking problems

Month 2: Mathematics & Number Theory (Weeks 6-9)
  Number Theory Fundamentals
    Topics:
      - Prime numbers and sieve of Eratosthenes
      - Segmented sieve for large ranges
      - Prime factorization techniques
      - GCD and LCM algorithms
      - Extended Euclidean algorithm
      - Modular arithmetic and properties
      - Modular inverse calculation
      - Fast exponentiation
      - Fermat's little theorem
      - Wilson's theorem
      - Chinese Remainder Theorem
      - Linear Diophantine equations
    Projects built:
      - Number theory library
      - Prime utilities toolkit
      - Modular arithmetic calculator
    Practice: Master 60 number theory problems
  Combinatorics & Probability
    Topics:
      - Permutations and combinations
      - Pascal's triangle and properties
      - Binomial coefficients calculation
      - nCr with modulo computation
      - Catalan numbers and applications
      - Stirling numbers basics
      - Inclusion-exclusion principle
      - Pigeonhole principle applications
      - Probability in competitive programming
      - Expected value problems
      - Generating functions basics
      - Counting techniques
    Projects built:
      - Combinatorics calculator
      - Probability simulator
      - Counting problems solver
    Practice: Solve 50 combinatorics problems
  Bit Manipulation Mastery
    Topics:
```

- Bitwise operations deep dive
- Bit manipulation tricks and hacks
- Finding single element problems
- XOR properties and applications
- Subset generation using bits
- Bitmask dynamic programming intro
- Gray code and applications
- Bit parallel algorithms
- Counting set bits efficiently
- __builtin functions in GCC
- Bitwise sieve
- Space optimization with bits

Projects built:
- Bit manipulation library
- Bitwise algorithm collection
- Space-optimized solutions

Practice: Master 40 bit manipulation problems

Sorting & Searching Advanced

Topics:
- Quick sort optimizations
- Merge sort applications
- Counting sort and radix sort
- Bucket sort for decimals
- Binary search variations
- Parallel binary search
- Ternary search on functions
- Search on monotonic functions
- Fractional cascading basics
- Order statistics
- K-th smallest element algorithms
- Median finding in O(n)

Projects built:
- Sorting algorithm library
- Search optimization toolkit
- Performance benchmarks

Practice: Solve 50 sorting/searching problems

Month 3: Trees, Graphs & Dynamic Programming (Weeks 10-13)

Tree Algorithms

Topics:
- Tree traversals optimization
- Binary tree problems
- Binary search tree operations
- Tree DP fundamentals
- LCA using binary lifting
- Distance between nodes
- Centroid decomposition
- Heavy-light decomposition basics
- Euler tour technique
- Tree flattening
- Rerooting technique
- Tree isomorphism

Projects built:
- Tree algorithms library
- LCA implementations comparison
- Tree problem solver

Practice: Master 60 tree problems

Graph Algorithms Fundamentals

Topics:
- Graph representations optimization
- DFS and BFS optimizations
- Connected components
- Strongly connected components
- Topological sorting applications
- Cycle detection algorithms
- Bipartite checking and matching
- Articulation points and bridges
- Kosaraju's and Tarjan's algorithms
- 2-SAT problem solving
- Eulerian path algorithms
- Graph coloring algorithms

Projects built:
- Graph library implementation
- Graph visualizer
- Algorithm comparison tool

Practice: Solve 70 graph problems

Shortest Paths & MST

Topics:
- Dijkstra's algorithm optimizations

- Bellman-Ford and negative cycles
- Floyd-Warshall applications
- Johnson's algorithm
- K-shortest paths
- Shortest path in DAG
- 0-1 BFS technique
- Dial's algorithm
- Kruskal's and Prim's MST
- Minimum spanning forest
- Second best MST
- Minimum bottleneck spanning tree

Projects built:
- Path finding library
- MST solver toolkit
- Graph optimization suite

Practice: Master 50 path/MST problems

Dynamic Programming Basics

Topics:
- DP state definition techniques
- Top-down vs bottom-up approaches
- 1D DP classic problems
- 2D DP patterns
- Space optimization techniques
- DP on sequences
- LIS and LCS variations
- Knapsack problem variations
- Subset sum problems
- Partition problems
- DP with probability
- Expected value DP

Projects built:
- DP problem classifier
- State transition visualizer
- DP optimization toolkit

Practice: Solve 80 DP problems

Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 2: ADVANCED ALGORITHMS & CONTEST STRATEGY (MONTHS 3-4, WEEKS 9-17)

Master advanced algorithms, dynamic programming patterns, and develop contest-winning strategies.

Month 4: Advanced Dynamic Programming (Weeks 14-18)

DP Optimization Techniques

Topics:
- Convex hull trick
- Li Chao tree
- Divide and conquer optimization
- Knuth-Yao speedup
- Monotone queue optimization
- Slope trick
- Aliens trick (WQS binary search)
- CHT for DP optimization
- Matrix exponentiation for DP
- DP on broken profile
- SOS DP (Sum over Subsets)
- Profile optimization

Projects built:
- DP optimization library
- Convex hull implementation
- Matrix exponentiation toolkit

Practice: Master 40 optimized DP problems

Bitmask DP & DP on Trees

Topics:
- Bitmask DP comprehensive
- Traveling salesman problem
- Hamiltonian path DP
- Profile dynamic programming
- DP on trees advanced
- Subtree DP problems
- Rerooting tree DP
- Tree DP with multiple states
- Centroid decomposition with DP
- Virtual tree technique
- DP on cactus graphs
- DP on DAGs

Projects built:

```
      - Bitmask DP solver
      - Tree DP framework
      - State compression tools
    Practice: Solve 50 bitmask/tree DP problems
  String Algorithms
    Topics:
      - KMP algorithm implementation
      - Z-algorithm and applications
      - Rolling hash and Rabin-Karp
      - Polynomial hashing techniques
      - Trie data structure advanced
      - Aho-Corasick algorithm
      - Suffix array construction
      - LCP array and applications
      - Suffix tree basics
      - Manacher's algorithm
      - Suffix automaton introduction
      - String matching with wildcards
    Projects built:
      - String algorithms library
      - Pattern matching toolkit
      - Suffix structures implementation
    Practice: Master 60 string problems
  Advanced Graph Algorithms
    Topics:
      - Network flow algorithms
      - Ford-Fulkerson and Edmonds-Karp
      - Dinic's algorithm
      - Push-relabel algorithm
      - Min-cost max-flow
      - Maximum bipartite matching
      - Hall's marriage theorem
      - Hopcroft-Karp algorithm
      - Hungarian algorithm
      - Gomory-Hu tree
      - Global min-cut
      - Stoer-Wagner algorithm
    Projects built:
      - Flow network library
      - Matching algorithms suite
      - Flow visualization tool
    Practice: Solve 40 flow/matching problems
  Game Theory
    Topics:
      - Nim game and variations
      - Sprague-Grundy theorem
      - Game theory with DP
      - Minimax algorithm
      - Alpha-beta pruning
      - Combinatorial games
      - Partisan vs impartial games
      - Multi-player games
      - Probabilistic games
      - Game on graphs
      - Hackenbush game
      - Conway's soldiers
    Projects built:
      - Game theory solver
      - Nim calculator
      - Game position evaluator
    Practice: Master 30 game theory problems

Month 5: Data Structures & Optimization (Weeks 19-23)
  Advanced Data Structures
    Topics:
      - Segment tree with lazy propagation
      - Persistent segment tree
      - Dynamic segment tree
      - 2D segment tree
      - Fenwick tree (BIT) advanced
      - 2D Fenwick tree
      - Range update range query
      - Merge sort tree
      - Wavelet tree
      - Treap and Cartesian tree
      - Splay tree basics
      - Link-cut tree introduction
    Projects built:
      - Segment tree library
```

```
    - Range query toolkit
    - Persistent DS implementation
  Practice: Implement 50 DS problems
Computational Geometry
  Topics:
    - 2D geometry primitives
    - Cross and dot products
    - Line intersection algorithms
    - Polygon area and perimeter
    - Point in polygon test
    - Convex hull algorithms
    - Graham scan and Jarvis march
    - Rotating calipers technique
    - Closest pair of points
    - Line sweep algorithms
    - Voronoi diagrams basics
    - Delaunay triangulation intro
  Projects built:
    - Geometry library
    - Convex hull visualizer
    - Geometric algorithm toolkit
  Practice: Solve 40 geometry problems
Advanced Number Theory
  Topics:
    - Miller-Rabin primality test
    - Pollard rho factorization
    - Discrete logarithm
    - Baby-step giant-step
    - Primitive roots
    - Mobius function and inversion
    - Multiplicative functions
    - Dirichlet convolution
    - Linear sieve
    - Gaussian elimination modulo
    - Matrix operations in field
    - FFT and NTT basics
  Projects built:
    - Advanced number theory library
    - Factorization toolkit
    - FFT implementation
  Practice: Master 35 advanced math problems
Optimization & Heuristics
  Topics:
    - Simulated annealing
    - Genetic algorithms basics
    - Hill climbing
    - Branch and bound advanced
    - Meet in the middle optimization
    - Sqrt decomposition applications
    - Mo's algorithm
    - Parallel binary search
    - Fractional cascading
    - Heavy path decomposition
    - Centroid path decomposition
    - Cache optimization techniques
  Projects built:
    - Optimization framework
    - Heuristic solver
    - Performance profiler
  Practice: Optimize 30 TLE solutions
Interactive & Randomized
  Topics:
    - Interactive problem strategies
    - Binary search in interactive
    - Graph exploration interactive
    - Randomized algorithms
    - Monte Carlo methods
    - Las Vegas algorithms
    - Randomized data structures
    - Treap operations
    - Skip list basics
    - Bloom filters
    - Probabilistic counting
    - Random sampling techniques
  Projects built:
    - Interactive problem solver
    - Randomized algorithm library
    - Probability tools
  Practice: Solve 25 interactive/random problems
```

```
  Month 6: Contest Mastery & Peak Performance (Weeks 24-26)
    Contest Strategy & Time Management
      Topics:
        - Contest strategy development
        - Problem selection order
        - Time allocation strategies
        - Reading all problems first
        - Identifying easy problems quickly
        - When to skip problems
        - Partial scoring strategies
        - Hacking and challenges
        - System testing preparation
        - Rating calculation understanding
        - Contest anxiety management
        - Peak performance techniques
      Projects built:
        - Personal strategy guide
        - Contest analysis tool
        - Performance tracker
      Practice: Participate in 20 rated contests
    Speed Programming & Templates
      Topics:
        - Speed coding techniques
        - Touch typing for programming
        - IDE shortcuts mastery
        - Template preparation and usage
        - Macro optimization
        - Pre-written code segments
        - Fast debugging techniques
        - Common bug patterns
        - Quick testing strategies
        - Stress testing methods
        - Local testing automation
        - Vim/Emacs for CP
      Projects built:
        - Ultimate CP template
        - Snippet library
        - Testing framework
      Practice: Speed coding 100 problems
    Team Programming (ICPC Style)
      Topics:
        - Team contest strategies
        - Role distribution in teams
        - Communication protocols
        - Parallel problem solving
        - Code handoff techniques
        - Debugging others' code
        - Team template preparation
        - Contest environment setup
        - Printed material preparation
        - Team practice sessions
        - ICPC regional strategies
        - World Finals preparation
      Projects built:
        - Team notebook creation
        - Role optimization system
        - Communication framework
      Practice: 50 team contest hours
      Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

------------------------------------------------------------
PHASE 3: EXPERT TECHNIQUES & PEAK PERFORMANCE (MONTHS 7-9, WEEKS 27-39)
------------------------------------------------------------
Master expert-level algorithms, specialized topics, and achieve competitive programming
excellence.

  Months 7-8: Expert Algorithms & Techniques (Weeks 27-36)
    FFT & Polynomial Algorithms
      Topics:
        - Fast Fourier Transform complete
        - Number Theoretic Transform (NTT)
        - Polynomial multiplication
        - Polynomial division
        - Polynomial interpolation
        - Generating functions advanced
        - Convolution applications
        - XOR convolution
```

```
    - Subset sum convolution
    - Polynomial inverse and sqrt
    - Multipoint evaluation
    - Chirp Z-transform
  Projects built:
    - FFT library complete
    - Polynomial operations toolkit
    - Convolution solver
  Practice: Master 25 FFT/polynomial problems
Linear Algebra in CP
  Topics:
    - Gaussian elimination advanced
    - Matrix exponentiation applications
    - System of linear equations
    - Matrix inverse and determinant
    - Basis and linear independence
    - XOR basis and applications
    - Linear recurrence solving
    - Berlekamp-Massey algorithm
    - Characteristic polynomial
    - Cayley-Hamilton theorem
    - Matrix tree theorem
    - Spectral graph theory basics
  Projects built:
    - Linear algebra library
    - Matrix computation toolkit
    - Recurrence solver
  Practice: Solve 30 linear algebra problems
Advanced String Structures
  Topics:
    - Suffix tree construction
    - Suffix automaton complete
    - Palindromic tree (Eertree)
    - Lyndon decomposition
    - Suffix array DC3 algorithm
    - SA-IS algorithm
    - LCP construction methods
    - Enhanced suffix array
    - Compressed suffix structures
    - Burrows-Wheeler transform
    - FM-index basics
    - Approximate string matching
  Projects built:
    - Advanced string library
    - Suffix structure implementations
    - String analysis toolkit
  Practice: Master 35 advanced string problems
Advanced Graph Theory
  Topics:
    - Planar graphs and testing
    - Graph minor theory basics
    - Tree decomposition
    - Treewidth and pathwidth
    - Chordal graphs
    - Perfect graphs introduction
    - Graph coloring advanced
    - Chromatic polynomial
    - Tutte polynomial basics
    - Expander graphs
    - Random graphs
    - Algebraic graph theory
  Projects built:
    - Advanced graph library
    - Graph property tester
    - Decomposition algorithms
  Practice: Solve 25 advanced graph problems
Approximation Algorithms
  Topics:
    - Approximation ratios
    - Vertex cover approximation
    - TSP approximation algorithms
    - Set cover approximation
    - Bin packing algorithms
    - Scheduling approximations
    - Facility location problems
    - K-center and k-median
    - PTAS and FPTAS
    - Primal-dual methods
    - LP relaxation techniques
```

```
        - Randomized rounding
      Projects built:
        - Approximation algorithm suite
        - Performance analyzer
        - Heuristic framework
      Practice: Implement 20 approximation algorithms

  Month 9: Peak Performance & Contests (Weeks 37-39)
    Research & Cutting Edge
      Topics:
        - Recent IOI/ICPC problems analysis
        - Research paper implementation
        - Novel algorithm exploration
        - Quantum algorithm basics
        - Machine learning in CP
        - Parallel algorithms
        - Distributed algorithms basics
        - External memory algorithms
        - Cache-oblivious algorithms
        - Succinct data structures
        - Compressed data structures
        - Future of competitive programming
      Projects built:
        - Research implementation
        - Novel solution finder
        - Algorithm innovation
      Practice: Solve 20 research-level problems
    Competition Preparation
      Topics:
        - Major competition strategies
        - ICPC World Finals preparation
        - IOI preparation techniques
        - Google Code Jam Finals
        - Facebook Hacker Cup Finals
        - TopCoder Open strategies
        - Mental preparation
        - Physical fitness for contests
        - Stress management advanced
        - Team coordination mastery
        - International competition tips
        - Travel and logistics
      Projects built:
        - Competition prep guide
        - Training schedule
        - Peak performance plan
      Practice: Simulate major competitions
    Career & Opportunities
      Topics:
        - CP in technical interviews
        - FAANG interview preparation
        - Competitive programming careers
        - Research opportunities
        - Graduate school applications
        - CP in resume building
        - Networking in CP community
        - Internship opportunities
        - Problem setter careers
        - Algorithm researcher path
        - Quantitative trading roles
        - Software engineering excellence
      Projects built:
        - Interview preparation kit
        - Career portfolio
        - Professional network
      Practice: Interview problem solving
      Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

---------------------------------------------------------------
PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)
---------------------------------------------------------------
The capstone month: spec, build, ship and defend, then the final exam that decides the
certificate.

  Month 10: Capstone and the Final Exam (Weeks 40-44)
    Capstone Spec and Setup
      Topics:
        - Choosing the capstone: a full contest simulation across a season of problem sets, with an
          editorial-quality solved-problem portfolio
```

- A one-page spec: scope, milestones, stack
- Repository and skeleton ready on day one
- Instructor sign-off before the build begins

Projects built:
- Approved capstone spec plus the running skeleton

Practice: Pitch the spec to the batch and absorb one hard question

Capstone Build

Topics:
- Two focused weeks in vertical slices
- Instructor checkpoints and honest scope cuts
- Using the whole course: the whole arsenal under contest time, with clean templates and fast, correct code
- Daily commits and an always-working build

Projects built:
- The capstone, feature-complete with version control

Practice: A working build at the end of every session

Ship and Polish

Topics:
- The finishing pass: edge cases, errors, documentation
- Contest-clean: fast, correct solutions written and debugged under time
- A README worth reading and a demo worth watching
- A full-course spiral review before the exam

Projects built:
- The capstone shipped, documented and rehearsed

Practice: Two mock demos, each tighter than the last

Demo Day and the Final Exam

Topics:
- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: higher-rated contests, ICPC teams, or quant and product interviews
- Where our other courses continue the journey

Projects built:
- Demo day presentation delivered to the batch

Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: solved sets across math, number theory and core structures
- Phase 2: advanced DP, geometry and strategy under real contest conditions
- Phase 3: expert-technique problem sets, FFT to advanced graph theory
- Phase 4: the contest simulation and editorial-quality portfolio

Total projects: 400+ contest problems solved, crowned by a full contest simulation

SKILLS THE STUDENT WILL HAVE AT THE END

- C++ with STL and fast I/O at competitive speed
- The mathematical arsenal: number theory, combinatorics, geometry
- Advanced dynamic programming: optimization, bitmask and DP on trees
- Expert techniques: FFT, linear algebra, advanced string and graph theory
- Contest craft: strategy, speed, templates and reading under pressure

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Higher-rated contests: the arsenal and speed to climb
- ICPC and team contests: the technique and strategy to compete
- Quant and product interviews that test contest-grade problem solving

  - Our DSA course beside this one for interview-specific communication

EARNING POTENTIAL (share only if asked, and never promise it)

  Market context: The ranges below are general market salary bands for these roles in India and
    abroad, drawn from public industry data. They are shown for career context only and are not a
    promise or guarantee of income. Actual pay depends on your skills, experience, location, and
    the job market.
  Early career roles: Ready for internships; Rs 10-20 LPA (Junior roles)
  Growing roles: Rs 20-40 LPA (Top tech companies)
  Experienced roles: Rs 40-80+ LPA (FAANG/Quant)
  International: $150k-400k+ USD

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What does this competitive programming course cover?
  A: The full competitive arsenal in order: C++ with STL and fast I/O, complexity, core
  structures, number theory, combinatorics and bit manipulation; then advanced dynamic
  programming, string algorithms, computational geometry, advanced data structures and contest
  strategy; then expert techniques, FFT and polynomial algorithms, linear algebra, advanced
  string structures and graph theory, with contest simulation throughout. A finale month runs a
  full contest and sits the final exam.

  Q: How long does it take to learn competitive programming properly?
  A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two
  more months in hand for learners who need them. The three teaching phases build the skill and
  its real-world application; the finale month ships a capstone and sits the final exam.

  Q: How am I assessed? Is the certificate just for attending?
  A: No. Problem sets follow every class with written feedback, monthly reviews deliberately
  mix new material with earlier topics, each phase ends with a cumulative exam, and the course
  ends with a final exam: a practical build plus written questions from the whole syllabus.
  Passing earns the certificate; falling short earns focused revision and a free retest.

  Q: Do I need to be good at maths for competitive programming?
  A: You need to be willing, not gifted. Competitive programming leans on discrete maths,
  number theory and combinatorics, and we teach every piece of it inside the course, in code,
  on problems. Plenty of strong competitors started shaky at maths; consistent solving is what
  builds the instinct.

  Q: Will you get me to a specific Codeforces rating?
  A: We will not promise a number, because rating depends on your own consistent practice and
  no honest teacher controls it. What we do is teach the technique, arsenal and contest craft
  that rating is built from, run real contests with honest debriefs, and give you the habits
  that keep the graph climbing after the course ends.

  Q: Can I try a class before enrolling?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or
  message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with
  no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at
  https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor reading your actual code, never
  pre-recorded videos.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link:
https://learn.modernagecoders.com/courses/competitive-programming-masterclass-college
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
COURSE 44 OF 137: COMPETITIVE PROGRAMMING FOR TEENS: ZCO, INOI AND USACO TRACK
==

Course link: https://learn.modernagecoders.com/courses/competitive-programming-for-teens-course
Subject area: Competitive Programming
Designed for: Teens
Who can join: Teens aged 13 to 18 who can already write basic programs and want contest-level problem solving
Course length: 6 months (24 weeks), joinable any month
Time needed: 2 live classes/week + weekly problem sets and virtual contests
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150

Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online competitive programming for teenagers: algorithms, data structures and contest technique aimed at the Indian informatics olympiad path and USACO, taught in C++ or Python.

In one line: Real algorithmic problem solving, aimed squarely at the informatics olympiad path and USACO, taught by solving problems rather than watching them being solved.

ABOUT THE PROGRAMME

Competitive programming is the fastest way a teenager can become genuinely good at algorithms, because a contest gives you an honest verdict in seconds. Your solution is correct and fast enough, or it is not. Nothing else in school works quite like that. This course takes a student who can already write basic programs and builds the actual competitive toolkit: complexity analysis, the core data structures, the standard algorithm families, and the contest craft that separates a solved problem from a time-limit-exceeded verdict. In India, the informatics olympiad path runs through zonal qualifiers, ZCO and ZIO, into INOI, the Indian National Olympiad in Informatics, and onward through the training camp toward the International Olympiad in Informatics. Internationally, USACO runs its own laddered divisions from Bronze up through Silver, Gold and Platinum. Both reward the same underlying skill, so we train for the skill and prepare for both formats. Students work in C++ or Python. C++ is the pragmatic default at higher levels because of its speed and its standard template library, and we teach it properly for students who want to go far. Python is fully supported for students who prefer it and is entirely adequate for the earlier stages. Every week ends with a timed virtual contest and a proper upsolve, because the problems you failed to solve during the contest are where nearly all the improvement actually comes from.

WHAT MAKES THIS COURSE DIFFERENT

- Aimed at the real contest ladder: zonal qualifiers and INOI in India, and the USACO divisions internationally
- Taught by solving, not by watching: every class has students writing code against a judge
- Complexity analysis from week one, because most failed submissions are correct but too slow
- Weekly timed virtual contests followed by a structured upsolve, which is where the actual improvement happens
- C++ taught properly for students aiming high, Python fully supported for those who prefer it
- Live, small batches with solutions reviewed line by line, not a video library

THE LEARNING PATH, STEP BY STEP

Phase 1: Complexity, the core data structures, sorting and searching, and reading a problem statement correctly
Phase 2: The standard algorithm families: greedy, two pointers, prefix sums, recursion, graphs and dynamic programming
Phase 3: Contest craft, harder problem sets, and full timed contests with upsolve discipline

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What Your Child Needs to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS AND CORE DATA STRUCTURES

The tools every contest problem assumes you already have, built until they are automatic.

Month 1: Complexity, Arrays and Strings (Weeks 1-4)
Setup, Judges and Big-O
Topics:
- Setting up a contest environment and submitting to an online judge
- Reading a problem statement precisely, including the constraints
- Time and space complexity, and reading the limits to infer the intended solution
- Why an O(n squared) solution fails when n is 200,000
- Fast input and output, which silently costs beginners real points

Projects built:
- Submit your first ten problems and read every verdict, including the failures

Practice: 15 warm-up problems with a written complexity estimate before each submission
Arrays, Sorting and Binary Search

Topics:
- Sorting with custom comparators
- Binary search on an array, and the off-by-one errors that ruin it
- Binary search on the answer, a technique that unlocks many problems
- Two pointers and the sliding window
- Recognising which of these a statement is hinting at

Projects built:
- Solve a set of problems where binary search on the answer is the intended solution

Practice: 20 sorting and searching problems on a judge

Prefix Sums, Strings and Number Theory Basics

Topics:
- Prefix sums and difference arrays for range queries
- String basics: frequency counts, palindromes, simple hashing ideas
- GCD, modular arithmetic and why answers are taken modulo a prime
- Sieve of Eratosthenes and simple primality
- Avoiding integer overflow, a classic silent failure

Projects built:
- Build a small library of your own tested helper functions

Practice: 20 problems across prefix sums, strings and basic number theory

Assessment: First timed virtual contest, followed by a full upsolve

Stacks, Queues and the Standard Library

Topics:
- Stacks and queues and the problems that signal them
- Deques and priority queues
- Using the C++ STL containers properly, or Python's equivalents
- Choosing the right container for the operation you repeat most
- Writing clean code fast, since contests are timed

Projects built:
- Solve a monotonic stack problem and explain why the stack is the right structure

Practice: 20 data-structure problems

Month 2: Recursion, Backtracking and First Graphs (Weeks 5-8)

Recursion and Backtracking

Topics:
- Writing recursion you can reason about
- The recursion tree and its cost
- Backtracking with pruning
- Generating permutations and subsets
- When recursion is elegant and when it is simply too slow

Projects built:
- Solve a classic constraint problem with backtracking and measure the effect of each pruning rule

Practice: 15 recursion and backtracking problems

Graph Representation, BFS and DFS

Topics:
- Adjacency lists versus matrices, and when each is right
- Depth-first search and its uses
- Breadth-first search and shortest paths on unweighted graphs
- Connected components and cycle detection
- Recognising a graph problem that is not described as one

Projects built:
- Model a described real-world scenario as a graph and solve it with BFS

Practice: 20 graph traversal problems

Greedy Algorithms

Topics:
- What makes a greedy choice valid
- Classic greedy problems: intervals, scheduling, coin-style problems
- Proving a greedy solution correct, at least informally
- Why an untested greedy hunch is the most common wrong answer
- Knowing when greedy fails and DP is required

Projects built:
- Solve an interval-scheduling set and write a short argument for why the greedy rule works

Practice: 20 greedy problems with written justifications

Assessment: Timed contest plus upsolve, with an error log started

Contest Technique

Topics:
- Reading the whole problem set first and ranking by difficulty
- Deciding fast whether to commit to a problem or move on
- Writing test cases before submitting
- Debugging under time pressure
- Handling a wrong answer without panicking

Projects built:
- A full contest run with a written post-mortem of every decision

Practice: Two timed contests with structured upsolve

PHASE 2: THE CORE ALGORITHM FAMILIES

The material that separates a zonal qualifier from a national-level contestant.

  Month 3: Dynamic Programming (Weeks 9-12)
    Introduction to Dynamic Programming
      Topics:
        - Recognising overlapping subproblems and optimal substructure
        - Memoisation versus bottom-up tabulation
        - Defining the state, which is the whole difficulty
        - Classic one-dimensional DP problems
        - Counting the states to get the complexity right
      Projects built:
        - Solve the same problem both top-down and bottom-up and compare
      Practice: 20 introductory DP problems
    Knapsack and Grid DP
      Topics:
        - The knapsack family and its variants
        - Two-dimensional DP on grids
        - Subset-sum style problems
        - Reconstructing the actual answer, not just its value
        - Space optimisation with rolling arrays
      Projects built:
        - Solve a knapsack variant and reconstruct the chosen items
      Practice: 20 knapsack and grid DP problems
    DP on Sequences and Intervals
      Topics:
        - Longest increasing subsequence and its faster form
        - Edit distance and sequence alignment
        - Interval DP
        - Bitmask DP as an introduction
        - Spotting the state definition from the constraints
      Projects built:
        - Implement LIS in both the quadratic and the faster form and compare on large input
      Practice: 20 sequence and interval DP problems
      Assessment: Timed DP-focused contest with upsolve
    Weighted Graphs and Shortest Paths
      Topics:
        - Dijkstra with a priority queue
        - Bellman-Ford and negative edges
        - Floyd-Warshall for all pairs
        - Minimum spanning trees
        - Disjoint set union and its uses
      Projects built:
        - Solve a routing problem with Dijkstra and justify the data structures chosen
      Practice: 20 weighted-graph problems

PHASE 3: CONTEST MASTERY AND OLYMPIAD PREPARATION

Harder problems and the discipline that turns knowledge into a rank.

  Months 4 to 6: Advanced Topics and Full Contests (Weeks 13-24)
    Advanced Structures and Techniques
      Topics:
        - Segment trees and range queries
        - Binary indexed trees
        - Trees: rooting, depth, subtree sums and lowest common ancestor
        - String algorithms at contest level
        - Choosing the simplest structure that is fast enough
      Projects built:
        - Implement a segment tree from scratch and use it on a real range-query problem
      Practice: 30 problems across advanced structures
    Olympiad-Style Problem Sets
      Topics:
        - Working past problems in the style of the zonal rounds and INOI
        - USACO division problems from Bronze upward
        - Multi-part problems and partial scoring
        - Managing a long contest without losing focus
        - Building a personal template and checklist
      Projects built:
        - Complete full past sets under time and grade them honestly
      Practice: Weekly full-length problem sets with structured upsolve
    Full Contests, Upsolve and Peak Form
      Topics:
        - Weekly full timed contests under real conditions
        - The upsolve discipline: every unsolved problem revisited until solved unaided
        - Reading the error log to find your actual weakness
        - Contest-day routine and nerves
        - Choosing the right target contests for your level
      Projects built:
        - A full contest every week with a written post-mortem

Practice: Continuous contest and upsolve cycle
Assessment: Final rated contest performance reviewed against your starting baseline

HOW A TYPICAL WEEK IS SPLIT

Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Readiness for zonal qualifiers, INOI and USACO divisions
- Algorithmic strength that makes school computer science straightforward
- A real advantage in university applications and technical interviews
- A foundation for research or engineering in any quantitative field
- Problem-solving stamina that transfers well beyond programming

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which contests does this prepare for?
A: Internationally we prepare students for the USACO divisions from Bronze upward. Both test the same underlying algorithmic skill, so the training serves either.

Q: Should my child use C++ or Python?
A: Both are supported. Python is entirely adequate for the earlier stages and many students start there. C++ is the pragmatic choice at higher levels because of its speed and standard template library, and time limits eventually make that matter. We teach C++ properly for students aiming at the later rounds, and we never force a switch before it is useful.

Q: Does my child need to be a strong programmer already?
A: They need working basics: loops, conditionals, functions and arrays. They do not need any algorithms background, which is what the course builds. What matters far more than existing skill is willingness to solve problems between classes, because in competitive programming practice volume beats natural talent almost every time.

Q: What is upsolving and why does it matter so much?
A: Upsolving means going back after a contest and solving the problems you could not solve during it, until you can do them unaided. It is where nearly all real improvement happens, and it is the single habit that separates students who plateau from students who keep climbing. Every contest in this course is followed by a structured upsolve.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final project defended live, plus a written assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link:
https://learn.modernagecoders.com/courses/competitive-programming-for-teens-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 45 OF 137: COMPUTER SCIENCE CLASS 11-12: CBSE PYTHON & ICSE/ISC JAVA
==========

Course link: https://learn.modernagecoders.com/courses/cbse-icse-computer-science-class-11-12-python-java-complete-course
Subject area: School Board Exam Preparation - Computer Science
Designed for: Teens
Who can join: Class 11 Beginner to Class 12 Board Exam Ready
Course length: 24 months (Class 11 + Class 12, the full 2-year board cycle), joinable any month
Time needed: 8-10 hours/week (4 classes + self-practice)
Certificate: Certificate from Modern Age Coders plus a documented mock-exam progression toward the board exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Class 11 & 12 Computer Science for CBSE (Python, SQL) and ICSE/ISC (Java, OOP): separate board batches, full syllabus, practical file and project guidance.

In one line: 100% Syllabus Coverage · Separate CBSE & ICSE Batches · Board Topper Strategy · Live Expert Mentoring

ABOUT THE PROGRAMME

This is the most thorough Computer Science program built for Indian Class 11 & 12 students across both CBSE and ICSE boards. We run completely separate batches for CBSE and ICSE students because the syllabus, programming language, exam pattern, and marking scheme are fundamentally different. CBSE students learn Python programming (as per Code 083 syllabus), SQL with MySQL, computer networking, file handling, data structures using Python, and Python-MySQL connectivity. ICSE/ISC students learn Java programming, object-oriented programming, data structures (stack, queue, linked list, binary trees), Boolean algebra with Karnaugh maps, and advanced recursion. Every class is aligned chapter-by-chapter with Sumita Arora, Preeti Arora, and NCERT textbooks. Students get complete practical file preparation, project guidance, viva preparation, previous year paper solving, and dedicated board exam revision modules. Our students consistently score 95+ in board exams because we don't just teach, we drill, practice, and perfect every concept until it becomes second nature.

WHAT MAKES THIS COURSE DIFFERENT

- Separate CBSE and ICSE batches, no mixing of syllabus or programming languages
- 100% syllabus coverage aligned with latest 2026-27 CBSE & ISC curriculum
- Chapter-wise teaching matching Sumita Arora, Preeti Arora & NCERT textbooks
- Complete practical file with 20+ programs (CBSE) / lab assignments (ISC) included
- Board exam project guidance with documentation and viva preparation
- Weekly chapter tests + monthly unit tests + mock board exams
- Previous 10 years solved papers with detailed marking scheme analysis
- Dedicated doubt-clearing sessions before exams
- Sample paper solving with CBSE/ISC marking scheme training
- Programming logic building from scratch, no prior coding needed
- SQL hands-on practice with live MySQL database
- Assertion-Reason, Case Study, and MCQ-specific exam strategies

THE LEARNING PATH, STEP BY STEP

Phase 1: Class 11, Semester 1 (Months 1-5): Computer fundamentals, number systems, Boolean logic, Python/Java basics, control flow, functions
Phase 2: Class 11, Semester 2 (Months 6-10): Data structures (lists/tuples/dictionaries in Python OR arrays/strings in Java), society & ethics, practicals
Phase 3: Class 12, Semester 1 (Months 11-17): Advanced programming, file handling, data structures, recursion, sorting & searching
Phase 4: Class 12, Semester 2 (Months 18-24): SQL & databases, computer networks, Python-MySQL connectivity / ISC advanced topics, board exam prep

WHAT A STUDENT NEEDS BEFORE STARTING

Class level: Students entering Class 11 or Class 12 with Computer Science - CBSE (code 083) or ISC; Class 10 students who have chosen CS can join early
Prior knowledge: No programming experience required - Class 11 starts from the very basics; Class 12 joiners get a bridge revision of Class 11 topics
Equipment: A laptop or desktop with a stable internet connection; we guide installation of Python and MySQL (CBSE) or Java and BlueJ (ISC) in the first sessions
Time commitment: 8-10 hours per week including live classes and self-practice
Textbooks: Sumita Arora, Preeti Arora, or NCERT - classes align with whichever textbook the student's school follows

WHO THIS COURSE IS FOR

Class 11 students: Students starting Computer Science in Class 11 who want strong fundamentals from the first chapter
Class 12 board aspirants: Students facing boards in the 2026-27 session who need complete syllabus coverage plus intensive exam practice
Students stuck on programming: Students who understand the theory but struggle to write, trace, or debug programs on their own
Practical file and project help: Students who need structured help finishing the practical file and board project correctly and on time
Future engineers: Students planning B.Tech, BCA, or B.Sc Computer Science who want a genuine head start on college programming
Score improvers: Students targeting a 90+ score who want weekly tests, mock exams, and marking-scheme answer training

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: CLASS 11 SEMESTER 1, COMPUTER FUNDAMENTALS & PROGRAMMING FOUNDATIONS (MONTHS 1-5)

Build rock-solid fundamentals in computer science theory and start your programming journey.
CBSE students begin with Python, ISC students begin with Java. Both batches cover computer
organization, number systems, and Boolean logic.

  Month 1: Computer Systems & Organization (Week 1-4)
    Computer System Overview & Hardware
      Topics:
        - Evolution of computers, from Abacus to AI-powered machines
        - Computer generations: First (vacuum tubes) to Fifth (AI)
        - Block diagram of a computer system, Input, CPU, Memory, Output
        - CPU architecture: ALU (Arithmetic Logic Unit), Control Unit, Registers
        - Types of processors: CISC vs RISC, multicore processors
        - Input devices: keyboard, mouse, scanner, touchscreen, biometric devices, microphone
        - Output devices: monitor (LCD, LED, OLED), printer (laser, inkjet, 3D), speaker, projector
        - Memory hierarchy: registers cache RAM secondary storage
        - Primary memory: RAM (SRAM vs DRAM), ROM (PROM, EPROM, EEPROM), Cache memory
        - Secondary storage: HDD, SSD, USB flash drive, optical discs (CD, DVD, Blu-ray)
        - Cloud storage concepts and modern storage solutions
        - Units of memory: bit, nibble, byte, KB, MB, GB, TB, PB
      Projects built:
        - Create an illustrated computer system diagram with all components labeled
        - Memory hierarchy comparison chart with speed, cost, and capacity
        - Research report: Evolution of processors from Intel 4004 to latest chips
      Practice: Chapter-end exercises from Sumita Arora Chapter 1, NCERT textbook questions,
      30 MCQs on computer fundamentals
    Software, Operating Systems & Encoding
      Topics:
        - Software classification: System software vs Application software
        - System software: operating system, language processors (compiler, interpreter, assembler),
          device drivers
        - Application software: general purpose (MS Office, browsers) vs specific purpose (Tally,
          hospital management)
        - Open source vs proprietary software, examples and differences
        - Operating system functions: process management, memory management, file management, device
          management
        - Types of OS: batch processing, time-sharing, real-time, distributed, mobile OS
        - User interface: CLI (Command Line Interface) vs GUI (Graphical User Interface)
        - Language processors: compiler vs interpreter, detailed comparison
        - High-level vs low-level vs machine-level languages
        - Utility software: antivirus, disk defragmenter, backup tools, compression tools
        - Firmware and embedded systems concepts
        - Mobile operating systems: Android, iOS, overview
      Projects built:
        - Software classification mind map with 20+ real-world examples
        - OS comparison: Windows vs Linux vs macOS, features and use cases
        - Install and explore Linux (Ubuntu) in a virtual machine
      Practice: NCERT exercises, Sumita Arora solved examples, 40 MCQs, 10 short-answer
      questions
    Number Systems & Conversions
      Topics:
        - Why computers use binary, the digital foundation
        - Decimal number system (base 10), positional notation explained
        - Binary number system (base 2), counting in binary, place values
        - Octal number system (base 8), uses in Unix/Linux file permissions
        - Hexadecimal number system (base 16), uses in memory addresses, colors (#FF0000)
        - Conversion: Decimal to Binary (successive division method)
        - Conversion: Decimal to Octal and Hexadecimal
        - Conversion: Binary to Decimal (positional value method)
        - Conversion: Binary to Octal (grouping of 3 bits) and Binary to Hexadecimal (grouping of 4
          bits)
        - Conversion: Octal to Binary, Hexadecimal to Binary
        - Fractional number conversions across all bases
        - Binary addition and subtraction
        - 1's complement and 2's complement representation
        - Signed magnitude representation
        - Overflow and underflow in binary arithmetic
        - Shortcut methods for quick conversions in exams
      Projects built:
        - Number system converter calculator (manual worksheet with 50 conversions)
        - Color code converter: RGB decimal to hexadecimal
        - Binary arithmetic drill, 30 addition and subtraction problems
      Practice: 100 conversion problems across all bases, previous year board questions on
      number systems, timed practice tests
    Boolean Logic & Encoding Schemes
      Topics:
        - Introduction to Boolean algebra, George Boole's contribution

- Boolean values: True (1) and False (0)
- Basic logic gates: AND, OR, NOT, symbols, truth tables, expressions
- Derived gates: NAND, NOR, XOR, XNOR, truth tables and equivalences
- Universal gates: NAND and NOR, implementing any gate using only NAND/NOR (ISC focus)
- Boolean expressions and evaluation
- De Morgan's theorems: NOT(A AND B) = NOT A OR NOT B, NOT(A OR B) = NOT A AND NOT B
- Verification of De Morgan's theorems using truth tables
- Basic Boolean laws: Identity, Complement, Idempotent, Absorption, Involution
- Simplification of Boolean expressions using algebraic method
- Character encoding schemes: ASCII (7-bit, 128 characters)
- ISCII (Indian Script Code for Information Interchange), 8-bit
- Unicode (UTF-8, UTF-16, UTF-32), multilingual support, emojis
- Difference between ASCII, ISCII, and Unicode, why Unicode is the modern standard

Projects built:
- Truth table generator for any 2-3 variable Boolean expression
- Gate diagram drawing practice, 10 circuits
- Encoding explorer: Convert your name to ASCII, Unicode values

Practice: 50 Boolean expression simplification problems, 30 truth table exercises, previous year questions on encoding

Months 2-3: Programming Foundations (Python for CBSE / Java for ISC) (Week 5-12)

Cbse Python Track

Getting Started with Python, Setup, Syntax & Fundamentals

Topics:
- Why Python?, Used by Google, Instagram, YouTube, NASA, Netflix
- Python history: Guido van Rossum, Python 2 vs Python 3
- Installing Python 3.12+ on Windows/Mac/Linux
- Python IDLE, Interactive mode vs Script mode
- Setting up VS Code with Python extensions
- First Python program: print('Hello, World!')
- Python tokens: keywords, identifiers, literals, operators, punctuators
- Python keywords list, import, def, class, if, else, elif, for, while, etc.
- Identifiers: naming rules (start with letter/underscore, case-sensitive, no keywords)
- Variables: declaration, assignment, dynamic typing
- Data types: int, float, complex, str, bool, None
- Type checking with type() function
- Type conversion: int(), float(), str(), bool(), implicit vs explicit
- Mutable vs Immutable types, fundamental concept for board exams
- Comments: single-line (#) and multi-line (triple quotes)
- Input function: input(), always returns string
- Output function: print(), sep, end parameters, formatted output
- f-strings for formatted printing (Python 3.6+)
- Escape sequences: \n, \t, \\, \', \"
- Python's philosophy: import this, The Zen of Python

Projects built:
- Personal bio generator, takes name, age, hobby as input and formats output
- Simple calculator, add, subtract, multiply, divide two numbers
- Temperature converter, Celsius to Fahrenheit and vice versa
- Area calculator for circle, rectangle, triangle with formatted output

Practice: Sumita Arora Chapter 3 & 4 exercises, 50 output prediction questions, 20 error-finding programs

Operators & Expressions in Python

Topics:
- Arithmetic operators: + - * / // (floor division) % (modulus) ** (exponent)
- Floor division vs true division, 7//2 = 3 vs 7/2 = 3.5
- Modulus operator applications, even/odd check, digit extraction
- Relational/Comparison operators: == != < > <= >=
- Logical operators: and, or, not, truth tables and short-circuit evaluation
- Short-circuit evaluation: how Python skips unnecessary checks
- Assignment operators: = += -= *= /= //= %= **=
- Identity operators: is, is not, comparing object identity vs equality
- Membership operators: in, not in, checking presence in sequences
- Bitwise operators: & (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), >> (right shift)
- Operator precedence table, PEMDAS for Python (** unary * / // % + - comparisons not and or)
- Associativity: left-to-right vs right-to-left (**)
- Expression evaluation practice, step-by-step breakdown
- Mixed-type expressions and type coercion rules
- The walrus operator := (Python 3.8+), assignment within expressions
- Common exam traps: integer division, operator precedence gotchas

Projects built:
- Expression evaluator, predict output of 20 complex expressions
- Bill calculator with tax and discount using all arithmetic operators
- Bitwise operation visualizer, show binary representation and result
- Operator precedence quiz game

Practice: 100 expression evaluation problems, 30 output prediction MCQs, Sumita Arora Chapter 5 exercises

Control Flow, Conditional Statements

Topics:
- Sequential flow vs conditional flow vs iterative flow, program flow types

- if statement: syntax, indentation rules, single-line and multi-line body
- if-else statement: two-way branching
- if-elif-else ladder: multi-way branching
- Nested if statements: decision trees within decisions
- Ternary operator / conditional expression: value = x if condition else y
- Common patterns: finding max/min of 2/3 numbers, grade calculator
- Leap year detection algorithm, complete logic
- Character type checking: vowel/consonant, uppercase/lowercase, digit/letter
- Menu-driven programs, building interactive applications
- Logical operators in conditions: combining multiple conditions
- Truthiness and falsiness in Python: 0, None, '', [], {} are falsy
- Common mistakes: = vs ==, missing colon, indentation errors
- Board exam pattern: output prediction for nested if-elif-else
- Assertion-Reason questions on conditional statements

Projects built:
- Grade calculator, CBSE marking scheme with A1, A2, B1, B2, C1, C2, D, E
- Income tax calculator, with Indian tax slab rates for 2026-27
- Triangle type detector, equilateral, isosceles, scalene, right-angled
- Simple login system with username and password validation
- Day of the week finder, input a number, output the day

Practice: 60 conditional logic problems, 30 output prediction questions, previous year board questions, Sumita Arora exercises

Control Flow, Loops & Iterations

Topics:
- for loop: iterating over sequences (range, string, list)
- range() function: range(stop), range(start, stop), range(start, stop, step)
- Negative step in range: range(10, 0, -1) for countdown
- while loop: condition-based repetition
- Infinite loops: while True, when and how to use safely
- Loop control statements: break (exit loop), continue (skip iteration)
- else clause with loops, executes when loop completes without break
- Nested loops: outer and inner loop, execution flow tracing
- Pattern printing with nested loops, stars, numbers, alphabets
- Common patterns: pyramid, diamond, Floyd's triangle, Pascal's triangle
- Accumulator pattern: sum, product, counting with loops
- Finding factors, prime numbers, Fibonacci series using loops
- GCD and LCM calculation using loops
- Digit extraction: sum of digits, reverse a number, palindrome check
- Armstrong number, perfect number, strong number detection
- Loop optimization and avoiding unnecessary iterations
- Board exam focus: tracing loop output, counting iterations

Projects built:
- Pattern printer, 15 different star and number patterns
- Prime number checker and generator (Sieve of Eratosthenes approach)
- Number guessing game with limited attempts
- Multiplication table generator with formatted output
- Fibonacci series generator up to n terms
- Armstrong number finder in a given range

Practice: 80 loop tracing problems, 20 pattern programs, Sumita Arora loop exercises, previous year board pattern questions

Isc Java Track

Getting Started with Java, OOP Foundations

Topics:
- Why Java?, Platform independence, Android development, enterprise applications
- Java history: James Gosling, Sun Microsystems, Oracle acquisition
- Installing JDK 17+ and setting up PATH environment variable
- Setting up BlueJ IDE (preferred for ISC) and IntelliJ IDEA
- First Java program: public static void main(String[] args)
- Understanding class structure, main method, System.out.println()
- Java tokens: keywords, identifiers, literals, operators, separators
- Primitive data types: byte, short, int, long, float, double, char, boolean
- Size and range of each primitive type, exam favorite
- Variables: declaration, initialization, naming conventions
- Constants with final keyword
- Type casting: widening (automatic) vs narrowing (explicit)
- Wrapper classes: Integer, Double, Character, Boolean, autoboxing/unboxing
- Scanner class for input: nextInt(), nextDouble(), nextLine(), next()
- Output: System.out.print(), println(), printf()
- Comments: //, /* */, /** */ (Javadoc)
- Java's object-oriented philosophy: everything is an object

Projects built:
- Student profile program, input name, roll number, marks, display formatted
- Simple calculator with Scanner input
- Type casting demonstrator, show widening and narrowing
- Wrapper class explorer, convert between primitives and objects

Practice: 50 output prediction questions, 20 type casting problems, ISC specimen paper MCQs

Operators, Expressions & Math Library in Java

Topics:

- Arithmetic operators: + - * / %, integer vs floating-point division
- Relational operators: == != < > <= >=
- Logical operators: && (AND), || (OR), ! (NOT), short-circuit evaluation
- Assignment operators: = += -= *= /= %=
- Increment/Decrement: ++x (prefix) vs x++ (postfix), critical for ISC exams
- Ternary operator: condition ? value1 : value2
- Bitwise operators: & | ^ ~ << >>
- Operator precedence and associativity, complete table
- Expression evaluation with mixed types, promotion rules
- Math class methods: Math.sqrt(), Math.pow(), Math.abs(), Math.max(), Math.min()
- Math.ceil(), Math.floor(), Math.round(), Math.random()
- Math.PI, Math.E, mathematical constants
- Trigonometric functions: Math.sin(), Math.cos(), Math.tan() (radians)
- String concatenation with + operator, rules and gotchas
- Type promotion in expressions: byte short int long float double

Projects built:
- Scientific calculator with Math library functions
- Expression evaluator, step-by-step breakdown of complex expressions
- Prefix vs postfix increment quiz, 20 tricky questions
- Random number generator game

Practice: 100 expression evaluation problems, ISC previous year questions, operator precedence drills

Conditional Statements in Java

Topics:
- if statement: syntax with curly braces, single statement shortcuts
- if-else statement: two-way branching
- if-else if-else ladder: multi-way branching
- Nested if statements: complex decision trees
- switch-case statement: syntax, break keyword, default clause
- switch vs if-else: when to use which, ISC exam favorite
- Fall-through in switch: what happens without break
- Menu-driven programs using switch-case
- Ternary operator for concise conditions
- Common programs: largest of 3, grade calculator, day finder
- Leap year, vowel/consonant, positive/negative/zero checks
- Character-based conditions: ASCII value ranges for A-Z, a-z, 0-9
- Dangling else problem and how Java resolves it
- Board exam focus: output tracing with nested conditions

Projects built:
- Menu-driven calculator using switch-case
- Income tax calculator with Indian tax slabs
- Student grade card generator with ISC marking scheme
- Electric bill calculator with slab-based pricing

Practice: ISC specimen paper conditional questions, 50 output tracing problems, 20 switch-case programs

Loops & Iterations in Java

Topics:
- for loop: initialization, condition, update, execution flow
- while loop: pre-test loop, condition checked before body
- do-while loop: post-test loop, body executes at least once (NOT in Python!)
- for vs while vs do-while: choosing the right loop
- Nested loops: matrix traversal, pattern printing
- Loop control: break (exit loop), continue (skip iteration)
- Labeled break and continue, for nested loop control
- Pattern printing: stars, numbers, alphabets, ISC exam staple
- Pyramid, inverted pyramid, diamond, Floyd's triangle patterns
- Digit extraction: sum of digits, reverse number, palindrome
- Prime number programs: check, generate, prime factorization
- GCD (Euclidean algorithm), LCM, Fibonacci series
- Armstrong, perfect, neon, spy, niven (Harshad) numbers
- Automorphic, kaprekar, twin prime numbers
- Frequency counting: digit frequency, character frequency in strings
- Loop optimization and efficiency analysis

Projects built:
- Pattern printer, 20 different patterns (stars, numbers, alphabets)
- Special number detector, checks Armstrong, palindrome, perfect in one program
- Prime number generator with sieve approach
- Number series generator: Fibonacci, triangular, square numbers

Practice: 100 loop tracing problems, 30 pattern programs, ISC previous year loop questions, number program drills

Months 4-5: Functions, Strings & Data Handling (Week 13-20)

Cbse Python Track

Strings in Python, Complete Mastery

Topics:
- String creation: single quotes, double quotes, triple quotes
- String as a sequence, indexing (0-based, negative indexing)
- String slicing: [start:stop:step], extracting substrings
- String immutability, why strings cannot be changed in place

- String operators: + (concatenation), * (repetition), in, not in
- String comparison: lexicographic ordering, ASCII values
- Traversing strings: for loop, while loop, enumerate()
- String methods (MUST KNOW for boards): len(), capitalize(), title(), upper(), lower(), swapcase()
- count(), find(), index(), rfind(), rindex()
- startswith(), endswith(), isalpha(), isdigit(), isalnum()
- isspace(), islower(), isupper(), istitle()
- strip(), lstrip(), rstrip(), replace(), split(), join()
- partition(), maketrans(), translate()
- String formatting: %, .format(), f-strings
- Common programs: palindrome check, character frequency, vowel count, word count
- Reverse a string, remove spaces, capitalize each word
- Board exam pattern: output prediction with string slicing and methods

Projects built:
- Word counter and analyzer, count words, vowels, consonants, digits, special chars
- Palindrome checker for strings and sentences
- Caesar cipher, encrypt and decrypt messages
- String formatting report generator

Practice: Sumita Arora Chapter 7 complete exercises, 50 string output prediction MCQs, 30 string programs

Lists in Python, Complete Mastery

Topics:
- List creation: [], list(), list comprehension
- Lists as mutable sequences, key difference from strings and tuples
- Indexing and negative indexing in lists
- List slicing: [start:stop:step]
- List operations: + (concatenation), * (repetition), in, not in
- List methods (BOARD EXAM CRITICAL): append(), extend(), insert()
- remove(), pop(), pop(index), clear()
- index(), count(), sort(), sort(reverse=True), reverse()
- copy(), shallow copy vs deep copy explained
- Built-in functions with lists: len(), min(), max(), sum(), sorted()
- List comprehension: [expression for item in iterable if condition]
- Nested lists (2D lists / matrices), accessing elements with [row][col]
- Traversing lists: for loop, while loop, enumerate(), zip()
- del statement vs remove() vs pop(), differences
- Aliasing vs cloning, the assignment trap (a = b vs a = b[:])
- Passing lists to functions, mutable argument behavior
- Common programs: linear search, finding max/min, removing duplicates, merging sorted lists
- Board exam focus: output prediction with list methods and slicing

Projects built:
- Student marks management system, add, remove, search, sort, average
- Inventory management with nested lists
- Matrix operations, addition, subtraction, transpose using 2D lists
- List-based quiz application

Practice: Sumita Arora Chapter 8 exercises, 60 list operation MCQs, 25 list programs

Tuples & Dictionaries in Python

Topics:
- Tuples: creation with () and tuple(), single element tuple (a,) trap
- Tuple immutability, cannot add, remove, or change elements
- Tuple indexing, slicing, and operations (+, *, in)
- Tuple methods: count(), index()
- Built-in functions: len(), min(), max(), sum(), sorted() with tuples
- Tuple packing and unpacking: a, b, c = (1, 2, 3)
- Tuple vs List: when to use which, performance and safety
- Nested tuples and accessing elements
- Dictionaries: key-value pair data structure, {} and dict()
- Dictionary keys must be immutable (strings, numbers, tuples), why?
- Accessing values: d[key] vs d.get(key, default)
- Adding and updating: d[key] = value, d.update()
- Removing: del d[key], d.pop(key), d.popitem(), d.clear()
- Dictionary methods: keys(), values(), items(), get(), update(), copy()
- Dictionary traversal: iterating over keys, values, items
- Nested dictionaries, student records with multiple attributes
- Dictionary comprehension: {k: v for k, v in iterable}
- len(), min(), max(), sorted() with dictionaries
- Common programs: frequency counter, word dictionary, student database
- Board exam: output prediction with dictionary operations

Projects built:
- Contact book application using dictionary
- Word frequency analyzer, count occurrence of each word in text
- Student report card using nested dictionaries
- English-Hindi dictionary with search feature

Practice: Sumita Arora Chapters 9 & 10 exercises, 50 MCQs on tuples and dictionaries, 20 programs

Functions & Modules in Python

Topics:
- What are functions?, code reusability, modularity, abstraction

- Built-in functions vs user-defined functions
- Defining functions: def keyword, function name, parameters, body, return
- Calling functions: positional arguments, keyword arguments
- Default parameter values, avoiding mutable default argument trap
- *args, variable-length positional arguments
- **kwargs, variable-length keyword arguments
- Return statement: returning single value, multiple values (as tuple)
- Functions without return, returns None by default
- Scope of variables: local scope, global scope, LEGB rule
- global keyword, modifying global variables inside functions
- Passing arguments: immutable (pass by value behavior) vs mutable (pass by reference behavior)
- Recursive functions introduction, factorial, Fibonacci (preview for Class 12)
- Lambda functions: lambda arguments: expression
- map(), filter(), reduce(), functional programming basics
- Modules: import, from...import, import...as
- math module: sqrt(), ceil(), floor(), pow(), fabs(), sin(), cos(), tan(), pi, e, log(), log10()
- random module: random(), randint(), randrange(), choice(), shuffle(), seed()
- statistics module: mean(), median(), mode()
- Creating your own modules, __name__ == '__main__' pattern
- Board exam focus: function output tracing, scope questions, module function MCQs

Projects built:
- Math utility library, custom module with 20+ mathematical functions
- Random password generator using random module
- Statistics calculator using statistics module
- Menu-driven bank account system using functions

Practice: Sumita Arora function chapter exercises, 40 function tracing MCQs, 25 function-based programs, scope identification exercises

Isc Java Track

Methods, Constructors & Scope in Java

Topics:
- Methods: defining, calling, return types, parameters
- Formal parameters vs actual parameters (arguments)
- Pass by value in Java, primitive types are copied
- Pass by reference behavior, objects are passed by reference copy
- Method overloading: same name, different parameter lists
- Constructors: default, parameterized, copy constructor
- Constructor overloading, multiple constructors in a class
- this keyword: referring to current object, constructor chaining
- Static methods and static variables, class-level members
- Static vs non-static: when to use which
- Variable scope: local, instance, class (static) variables
- Access modifiers: public, private, protected, default
- Encapsulation: data hiding with private variables and public getters/setters
- Pure functions and impure functions in Java
- Method signature and return type, ISC exam focus
- Recursive methods introduction, factorial, GCD

Projects built:
- Bank account class with deposit, withdraw, balance methods
- Student class with multiple constructors and method overloading
- Calculator class with static utility methods
- Library book management with encapsulation

Practice: ISC method-based questions, 40 output tracing problems, constructor MCQs

Arrays in Java, 1D and 2D

Topics:
- Array declaration, creation with new keyword, initialization
- Array as reference type, memory allocation on heap
- Accessing elements: indexing (0-based), ArrayIndexOutOfBoundsException
- Array traversal: for loop, for-each (enhanced for) loop
- Array input with Scanner, reading n elements
- 1D array operations: sum, average, max, min, counting
- Linear search: sequential search algorithm
- Binary search: divide and conquer on sorted arrays
- Sorting algorithms: Bubble sort, swapping adjacent elements
- Selection sort, finding minimum and placing in position
- Insertion sort, inserting element in correct position in sorted part
- Comparison of sorting algorithms, time complexity basics
- 2D arrays (matrices): declaration, initialization, traversal
- Matrix operations: addition, subtraction, multiplication, transpose
- Row-wise and column-wise operations on 2D arrays
- Jagged arrays, arrays of different lengths
- Array address calculation: row-major and column-major (ISC theory)
- Common programs: merge arrays, rotate array, remove duplicates

Projects built:
- Student marks analyzer, input marks, sort, find topper, grade distribution
- Matrix calculator, all operations with menu-driven interface
- Address calculation worksheet, 20 problems
- Sorting visualizer, trace each pass of bubble, selection, insertion sort

Practice: ISC array-based programs (30+), sorting algorithm tracing, address

calculation formula problems
String Handling in Java
Topics:
- String class: immutable strings in Java
- String creation: literal vs new keyword, String pool concept
- String methods: length(), charAt(), indexOf(), lastIndexOf()
- substring(), concat(), replace(), toUpperCase(), toLowerCase()
- trim(), startsWith(), endsWith(), contains()
- equals() vs == : content comparison vs reference comparison, EXAM CRITICAL
- equalsIgnoreCase(), compareTo(), compareToIgnoreCase()
- String to char array: toCharArray(), and vice versa valueOf()
- Character class methods: isLetter(), isDigit(), isUpperCase(), isLowerCase()
- Character.toUpperCase(), Character.toLowerCase()
- StringBuffer/StringBuilder: mutable strings, append(), insert(), delete(), reverse()
- String vs StringBuffer vs StringBuilder, differences and when to use
- StringTokenizer class: splitting strings (ISC specific)
- String concatenation performance: why StringBuilder matters
- Common programs: palindrome, word extraction, pigLatin, character frequency
- Sentence manipulation: reverse words, title case, remove duplicates
- Board exam focus: output tracing with string methods
Projects built:
- Text analyzer, word count, sentence count, character frequency
- Pig Latin converter
- Unique word extractor from a sentence
- String encryption/decryption program
Practice: ISC string programs (40+), method output prediction MCQs, character manipulation problems
Recursion in Java, ISC Focus
Topics:
- What is recursion?, method calling itself
- Base case and recursive case, avoiding infinite recursion
- Stack memory and recursion, how function calls are stacked
- StackOverflowError, what happens without base case
- Factorial using recursion, tracing each call
- Fibonacci series using recursion
- GCD using Euclidean algorithm (recursive)
- Power calculation: a^n using recursion
- Sum of digits using recursion
- Reverse a number using recursion
- Palindrome check using recursion
- Binary search using recursion
- Tower of Hanoi, classic recursion problem (ISC important)
- Decimal to binary conversion using recursion
- String reversal using recursion
- Recursion vs iteration: pros, cons, when to use which
- Tail recursion concept
- ISC exam focus: tracing recursive calls, finding output, counting calls
Projects built:
- Recursive function library, 15 recursive solutions
- Tower of Hanoi solver with step-by-step output
- Recursive pattern printer
- Recursive maze solver (bonus challenge)
Practice: ISC recursion questions (30+), call stack tracing exercises, output prediction problems

---
PHASE 2: CLASS 11 SEMESTER 2, ADVANCED CONCEPTS, ETHICS & PRACTICALS (MONTHS 6-10)

Complete remaining Class 11 syllabus topics, prepare practical file with 20+ programs, work on project, and build exam readiness.

Months 6-7: Society, Law, Ethics & Advanced Topics (Week 21-28)
Society, Law and Ethics, Digital Citizenship (CBSE) / Computing Trends (ISC)
Topics:
- Digital footprint: active vs passive, what you leave behind online
- Net etiquette (Netiquette): proper online behavior, email etiquette
- Social media ethics and responsible usage
- Data protection and privacy, personal data, consent, GDPR concepts
- Intellectual Property Rights (IPR): copyright, patent, trademark
- Plagiarism: what it is, how to avoid, proper citation
- Software licensing: proprietary vs open source
- Free and Open Source Software (FOSS): Linux, LibreOffice, GIMP, Python
- Creative Commons licenses: CC-BY, CC-BY-SA, CC-BY-NC
- GPL (GNU General Public License), Apache License, MIT License
- Cybercrime: hacking, cracking, phishing, pharming, vishing, smishing
- Malware types: virus, worm, trojan horse, ransomware, spyware, adware
- Ransomware attacks: WannaCry, NotPetya, real-world case studies
- Cyberbullying: forms, impact, prevention, reporting
- Indian Information Technology Act 2000 (amended 2008), key provisions

- IT Act sections relevant to students: Sections 43, 65, 66, 67
- E-waste: definition, hazards, proper disposal methods, recycling
- Health concerns: ergonomics, digital eye strain, repetitive strain injury
- Gender and disability issues in computing, inclusivity in tech
- Emerging trends: AI, IoT, Cloud Computing, Big Data, Blockchain (overview)

Projects built:
- Digital citizenship presentation, 20 dos and don'ts
- Cybercrime awareness poster/infographic
- E-waste management plan for your school
- Research report: Famous cyberattacks and lessons learned

Practice: NCERT society chapter questions, 50 MCQs on cyber ethics, case study questions, IT Act scenario-based questions

CBSE: Advanced Python Concepts / ISC: File Handling & Packages in Java

Cbse topics:
- Revisiting all Python concepts for exam preparation
- Advanced list comprehensions with multiple conditions
- Dictionary and set operations, advanced patterns
- Nested data structures: list of dictionaries, dictionary of lists
- Error handling basics: try, except, finally (preview for Class 12)
- File handling introduction: open(), read(), write(), close() (preview)
- Standard library exploration: os module, sys module basics
- Code optimization techniques, writing Pythonic code
- PEP 8 coding style guidelines
- Debugging strategies: print debugging, IDE debugger usage

Isc topics:
- File handling in Java using FileInputStream, FileOutputStream
- FileReader and FileWriter, character stream handling
- BufferedReader and PrintWriter for efficient I/O
- Reading and writing text files, line by line
- Exception handling: try-catch-finally in file operations
- Packages: what are packages, package declaration
- Creating user-defined packages
- Importing packages: import statement, wildcard import
- Java API packages: java.lang, java.util, java.io
- Access modifiers with packages: public, protected, default, private

Practical File Preparation, Project Work & Class 11 Exam Prep

Topics:
- CBSE Practical file: Minimum 20 Python programs covering all chapters
- ISC Practical: Year-long programming assignments compilation
- Program 1-5: Basic Python/Java programs (variables, operators, input/output)
- Program 6-10: Control flow programs (if-else, loops, patterns)
- Program 11-15: String manipulation programs
- Program 16-20: List/Array, Tuple, Dictionary programs
- Program 21-25: Function-based programs with modules (CBSE) / OOP programs (ISC)
- Project selection and planning, choosing a meaningful project
- CBSE suggested projects: Quiz game, Library management, Student database
- ISC suggested projects: Inventory system, Bank management, School management
- Project documentation: introduction, system requirements, source code, output screenshots
- Viva voce preparation: 50 most-asked questions with answers
- Class 11 final exam revision, chapter-wise important questions
- Sample paper solving, 5 complete papers with marking scheme
- Time management during exams, allocating marks to time

Deliverables:
- Complete practical file with 20-25 programs (handwritten/printed as per school)
- Project report with documentation
- Viva preparation sheet, 50 Q&A
- 5 solved sample papers

PHASE 3: CLASS 12 SEMESTER 1, ADVANCED PROGRAMMING, FILE HANDLING & DATA STRUCTURES (MONTHS 11-17)

Deep dive into Class 12 core topics: advanced programming, file handling, data structures, sorting and searching algorithms. This is the highest-weightage section in board exams.

Months 11-12: Advanced Functions, Exception Handling & File Handling (Week 41-48)

Cbse Python Track

Advanced Functions & Exception Handling in Python

Topics:
- Revision of Class 11 function concepts, parameters, return, scope
- Global vs local scope deep dive, LEGB rule revisited
- global keyword, modifying global variables inside functions
- Nested functions and closures
- Functions as first-class objects, passing functions as arguments
- Advanced default arguments and keyword-only arguments
- Recursive functions: factorial, Fibonacci, sum of digits, power, GCD
- Recursion depth and sys.setrecursionlimit()
- Exception handling: what are exceptions vs syntax errors
- try block: code that might cause an exception
- except block: handling specific exceptions

- Multiple except blocks for different exception types
- except with as keyword: except ValueError as e
- else block: executes when no exception occurs
- finally block: always executes, cleanup code
- Common built-in exceptions: ValueError, TypeError, IndexError, KeyError, ZeroDivisionError, FileNotFoundError, IOError
- raise statement: raising exceptions manually
- Custom exception classes (basics)
- Board exam pattern: output prediction with try-except, function scope questions

Projects built:
- Robust calculator with exception handling for all edge cases
- Recursive problem set, 10 programs using recursion
- Error-proof input validator using try-except
- Function library with comprehensive error handling

Practice: Sumita Arora Class 12 function chapters, 40 exception handling MCQs, 20 recursive function tracing problems

Text File Handling in Python

Topics:
- What are files?, persistent storage, file vs variable
- Types of files: text files vs binary files
- Opening a file: open(filename, mode), file handle/object
- File modes: 'r' (read), 'w' (write), 'a' (append), 'r+' (read+write), 'w+' (write+read), 'a+' (append+read)
- Reading text files: read(), read(n), readline(), readlines()
- Difference between read(), readline(), readlines(), EXAM CRITICAL
- Writing to text files: write(), writelines()
- Difference between write() and writelines()
- Closing files: close(), why it's important
- with statement: automatic file closing, best practice
- File pointer: tell(), current position, seek(offset, whence), moving pointer
- seek(0), go to beginning, seek(0, 2), go to end
- Line-by-line processing: for line in file_handle
- Counting lines, words, characters in a text file
- Searching for a word/pattern in a text file
- Copying content from one file to another
- Appending data to existing files
- Common programs: count uppercase/lowercase/digits/vowels in file
- Find and replace words in a text file
- Board exam focus: output prediction with file operations, explain difference questions

Projects built:
- Text file analyzer, count lines, words, characters, vowels, consonants
- Log file processor, search and filter entries
- Student record manager using text files
- File copy and merge utility

Practice: Sumita Arora file handling chapter, 30 file handling programs, 40 MCQs on file operations

Binary File Handling & CSV File Handling in Python

Topics:
- Binary files: what and why, storing non-text data
- Binary file modes: 'rb', 'wb', 'ab', 'r+b', 'w+b', 'a+b'
- pickle module: serialization and deserialization
- pickle.dump(object, file_handle): writing Python objects to binary file
- pickle.load(file_handle): reading Python objects from binary file
- Pickling different data types: int, float, string, list, tuple, dictionary
- Writing and reading multiple records to/from binary file
- Searching records in a binary file, reading until EOFError
- Updating and deleting records in binary files (read all, modify, write all)
- EOFError handling with try-except, standard pattern for binary files
- CSV files: Comma Separated Values, structure and usage
- csv module: importing csv
- csv.writer(): creating CSV writer object
- writerow(), writing single row, writerows(), writing multiple rows
- csv.reader(): reading CSV files, returns iterable
- Reading CSV with for loop, each row is a list
- Using delimiter parameter for custom separators
- CSV with headers, DictWriter and DictReader (bonus)
- Real-world CSV: exporting student data, reading attendance sheets
- Board exam pattern: complete programs on binary and CSV file operations

Projects built:
- Student database with binary file, add, search, update, delete records
- Employee payroll system using pickle
- CSV report generator, attendance tracker
- Data migration tool, text file to CSV to binary file conversion

Practice: 30 binary file handling programs, 15 CSV programs, Sumita Arora exercises, previous year board questions

Data Structures, Stack Implementation using Python List

Topics:
- What are data structures?, organizing data efficiently
- Linear vs non-linear data structures

- Stack: Last-In-First-Out (LIFO) principle
- Real-world stack analogies: stack of plates, undo/redo, browser back button
- Stack operations: push (insert at top), pop (remove from top), peek/top (view top)
- isEmpty(), check if stack is empty, size(), number of elements
- Stack overflow and stack underflow conditions
- Implementing stack using Python list: append() for push, pop() for pop
- Complete stack implementation with all operations
- Stack applications: parenthesis matching, expression evaluation
- Infix, prefix, postfix notation, understanding the concepts
- Reversing a string using stack
- Stack-based programs for board exams: book management, student records
- Tracing push and pop operations, step-by-step visualization
- Board exam pattern: stack operation output prediction, application-based questions
- Stack vs Queue (conceptual comparison)
- Time complexity of stack operations: O(1) for push and pop

Projects built:
- Stack implementation, complete menu-driven program
- Parenthesis/bracket matcher using stack
- Reverse a string/list using stack
- Undo feature simulation using stack

Practice: 25 stack programs, operation tracing exercises, Sumita Arora data structure chapter, board exam stack questions from last 10 years

Isc Java Track

Inheritance & Polymorphism in Java (ISC Class 12 Focus)

Topics:
- Inheritance: creating new classes from existing classes
- extends keyword, subclass inherits from superclass
- Types of inheritance: single, multilevel, hierarchical
- Why Java doesn't support multiple inheritance (through classes)
- super keyword: calling superclass constructor, accessing superclass methods/variables
- Constructor chaining in inheritance, order of constructor execution
- Method overriding: subclass redefining superclass method
- Method overloading vs method overriding, EXAM CRITICAL comparison
- Polymorphism: one interface, multiple forms
- Compile-time polymorphism (method overloading) vs Runtime polymorphism (method overriding)
- Dynamic method dispatch / dynamic binding
- Abstract classes: abstract keyword, abstract methods
- Concrete vs abstract classes, when to use which
- Interfaces in Java: what they are, how they differ from abstract classes
- Implementing interfaces: implements keyword
- Interface vs abstract class, ISC exam comparison question
- final keyword: final variables (constants), final methods (can't override), final classes (can't inherit)
- instanceof operator, type checking
- Upcasting and downcasting

Projects built:
- Shape hierarchy: Shape Circle, Rectangle, Triangle with area/perimeter
- Employee payroll with Manager, Developer, Intern subclasses
- Vehicle inheritance: Vehicle Car, Bike, Truck
- Banking system with different account types

Practice: ISC inheritance questions (40+), output tracing with polymorphism, method dispatch problems

Advanced Data Structures, Stack & Queue in Java (ISC)

Topics:
- Stack: LIFO principle, push, pop, peek, isEmpty
- Stack implementation using arrays in Java
- Stack overflow (array full) and underflow (array empty) handling
- Stack applications: parenthesis matching, infix to postfix conversion
- Infix to prefix conversion using stack
- Postfix expression evaluation using stack
- Queue: FIFO principle, enqueue, dequeue, front, rear, isEmpty
- Queue implementation using arrays in Java
- Limitation of linear queue, wasted space
- Circular Queue: efficient queue implementation
- Circular queue operations: enqueue, dequeue with modular arithmetic
- Double-Ended Queue (Deque): insertion and deletion from both ends
- Priority Queue concepts
- Applications of queue: scheduling, BFS, printer queue
- ISC exam pattern: implementing stack/queue as a class with constructor and methods

Projects built:
- Stack class with complete implementation and menu-driven testing
- Infix to Postfix converter using stack
- Circular Queue implementation
- Postfix expression evaluator

Practice: ISC data structure questions (40+), stack/queue operation tracing, conversion problems

Linked List in Java (ISC Class 12)

Topics:
- What is a linked list?, dynamic data structure, nodes and links

- Node class: data field and next reference
- Singly Linked List: creating, traversing
- Insertion: at beginning, at end, at specific position
- Deletion: from beginning, from end, specific node by value
- Searching in a linked list, sequential traversal
- Counting nodes in a linked list
- Reversing a linked list, iterative approach
- Extracting a sublist from a linked list
- Checking if linked list is empty
- Linked list vs array, advantages and disadvantages comparison
- Memory allocation: static (arrays) vs dynamic (linked lists)
- ISC exam pattern: writing complete linked list class with all operations
- Sorted insertion in a linked list
- Merging two sorted linked lists
- Applications of linked lists: polynomial representation, sparse matrix

Projects built:
- Complete Linked List class with all operations, ISC exam format
- Polynomial addition using linked lists
- Student record management using linked list
- Sorted linked list with insertion and deletion

Practice: ISC linked list questions (30+), operation tracing, complete class implementation practice

Binary Trees & Computational Complexity (ISC Class 12)

Topics:
- Binary trees: concept, terminology, root, node, leaf, parent, child, sibling
- Tree properties: height, depth, level, degree
- Types of binary trees: full, complete, perfect, balanced
- Binary Search Tree (BST): property, left < root < right
- Tree traversals (conceptual, ISC doesn't require implementation):
- In-order traversal: Left Root Right (gives sorted order for BST)
- Pre-order traversal: Root Left Right
- Post-order traversal: Left Right Root
- Constructing BST from given data
- Tracing traversal outputs for given trees, EXAM FAVORITE
- Computational complexity: measuring algorithm efficiency
- Input size concept, how it affects running time
- Best case, average case, worst case analysis
- Big O notation: O(1), O(log n), O(n), O(n log n), O(n²)
- Dominant term, ignoring constants and lower-order terms
- Complexity of common algorithms: linear search O(n), binary search O(log n)
- Sorting complexity: bubble O(n²), selection O(n²), insertion O(n²)
- Stack/Queue operations: O(1)
- Linked list search: O(n)

Projects built:
- BST traversal tracer, draw tree and show all 3 traversal outputs
- Complexity comparison chart for all algorithms learned
- Algorithm efficiency analyzer, compare running times

Practice: ISC tree traversal questions (20+), complexity analysis MCQs, BST construction exercises

Months 13-14: Sorting, Searching & Algorithm Efficiency (Week 49-56)

Applies To Cbse

Sorting Algorithms in Python (CBSE Class 12)

Topics:
- Why sorting matters, organizing data for efficient processing
- Bubble Sort: comparing adjacent elements, swapping, multiple passes
- Bubble sort algorithm: step-by-step with example array
- Bubble sort Python code with complete explanation
- Optimized bubble sort with swap flag, early termination
- Bubble sort complexity: O(n²) average/worst, O(n) best (optimized)
- Selection Sort: finding minimum, placing in correct position
- Selection sort algorithm: step-by-step with example
- Selection sort Python code with explanation
- Selection sort complexity: O(n²) in all cases
- Insertion Sort: building sorted portion, inserting elements
- Insertion sort algorithm: step-by-step with example, like sorting cards
- Insertion sort Python code with explanation
- Insertion sort complexity: O(n²) average/worst, O(n) best
- Comparison of all three sorting algorithms, when to use which
- Stability of sorting algorithms, bubble and insertion are stable, selection is not
- Board exam focus: tracing sort passes, writing code, comparing algorithms

Projects built:
- Sorting visualizer, show each pass of all 3 algorithms
- Performance comparison, time 1000 sorts with each algorithm
- Sort student records by name, marks, roll number

Practice: 30 sort tracing problems, write all 3 algorithms from memory, previous year board sorting questions

Searching Algorithms in Python (CBSE Class 12)

Topics:

- Linear Search: sequential checking, works on unsorted data
- Linear search algorithm and Python implementation
- Linear search: best case O(1), worst case O(n), average O(n/2)
- Binary Search: divide and conquer, requires sorted data
- Binary search algorithm: finding mid, comparing, narrowing range
- Binary search Python implementation, iterative approach
- Binary search: best case O(1), worst case O(log n)
- Why binary search is dramatically faster, logarithmic vs linear
- Binary search edge cases: element not found, first/last element
- Linear vs Binary search comparison, detailed table
- When to use which: unsorted data linear, sorted data binary
- Searching in lists, tuples, and dictionaries
- Python's built-in search: in operator, index() method, bisect module
- Board exam pattern: tracing binary search steps, code output prediction

Projects built:
- Search algorithm comparison tool, count comparisons for both
- Phone book search, linear for names, binary for sorted entries
- Searching programs from past 10 years board papers, complete solutions

Practice: 20 binary search tracing problems, comparison with linear search, board exam questions

---

PHASE 4: CLASS 12 SEMESTER 2, SQL, NETWORKS, CONNECTIVITY & BOARD EXAM MASTERY (MONTHS 18-24)

Complete the remaining Class 12 syllabus: SQL and databases, computer networks, Python-MySQL connectivity, then intensive board exam preparation with mock tests and revision.

Months 18-19: Database Management & SQL (Week 69-76)

Database Concepts & SQL Fundamentals

Topics:
- What is a database?, organized collection of related data
- File system vs DBMS: advantages of database approach
- DBMS examples: MySQL, PostgreSQL, Oracle, SQL Server, MongoDB
- Relational Database Management System (RDBMS), storing data in tables
- Relational model terminology: relation (table), attribute (column), tuple (row), domain (valid values)
- Degree of a relation: number of columns, Cardinality: number of rows
- Keys in RDBMS, EXAM CRITICAL:
- Candidate key: uniquely identifies each row (may have multiple)
- Primary key: chosen candidate key for unique identification (only one per table)
- Alternate key: candidate keys that are not primary key
- Foreign key: attribute referencing primary key of another table, establishes relationship
- Composite key: primary key made of two or more columns
- Introduction to SQL: Structured Query Language, not a programming language
- SQL categories: DDL (Data Definition), DML (Data Manipulation), DQL (Data Query)
- MySQL installation and setup, accessing MySQL command line
- SQL data types: CHAR(n), VARCHAR(n), INT, SMALLINT, FLOAT, DOUBLE, DATE, DATETIME
- CHAR vs VARCHAR: fixed vs variable length, when to use which
- Constraints: NOT NULL, UNIQUE, PRIMARY KEY, DEFAULT, CHECK

Projects built:
- Design a school database: tables for Students, Teachers, Classes, Marks
- Identify keys for 10 different real-world scenarios
- Install MySQL and create your first database and tables

Practice: 50 MCQs on database concepts, key identification exercises, Sumita Arora SQL chapter theory questions

SQL DDL & DML Commands

Topics:
- DDL, Data Definition Language (structure commands):
- CREATE DATABASE database_name;, creating a new database
- USE database_name;, selecting active database
- SHOW DATABASES; SHOW TABLES;, listing databases and tables
- CREATE TABLE: defining table structure with columns, data types, constraints
- CREATE TABLE with PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT constraints
- DESCRIBE/DESC table_name;, viewing table structure
- ALTER TABLE: ADD column, MODIFY column data type/size, DROP column
- ALTER TABLE: ADD PRIMARY KEY, DROP PRIMARY KEY
- DROP TABLE table_name;, deleting entire table
- DROP DATABASE database_name;, deleting entire database
- DML, Data Manipulation Language (data commands):
- INSERT INTO: adding single row, adding multiple rows
- INSERT INTO with column list vs without (all columns required)
- INSERT INTO with NULL values and DEFAULT values
- UPDATE: modifying existing data with SET clause
- UPDATE with WHERE clause, conditional update
- UPDATE multiple columns simultaneously
- DELETE FROM: removing rows with WHERE condition
- DELETE vs DROP: removing data vs removing structure
- CAUTION: UPDATE/DELETE without WHERE affects ALL rows

Projects built:

- Create complete school database with 5 tables
- Insert 20+ records in each table, realistic data
- Practice ALTER TABLE: add, modify, drop columns
- UPDATE and DELETE operations with various conditions

Practice: 40 DDL/DML practice queries, Sumita Arora exercises, create-insert-update-delete drill

SQL DQL, SELECT Queries, Clauses & Functions

Topics:

- SELECT statement: retrieving data from tables
- SELECT * FROM table;, all columns, all rows
- SELECT column1, column2 FROM table;, specific columns
- SELECT DISTINCT column FROM table;, removing duplicates
- Column aliases: SELECT column AS alias_name
- WHERE clause: filtering rows based on conditions
- Comparison operators in WHERE: = <> < > <= >=
- Logical operators: AND, OR, NOT, combining conditions
- BETWEEN operator: WHERE column BETWEEN value1 AND value2
- IN operator: WHERE column IN (value1, value2, value3)
- LIKE operator: pattern matching with wildcards
- % wildcard: any number of characters, _ wildcard: single character
- LIKE patterns: 'A%' (starts with A), '%ing' (ends with ing), '%an%' (contains an)
- IS NULL and IS NOT NULL: checking for missing values
- ORDER BY: sorting results, ASC (default) and DESC
- ORDER BY multiple columns, secondary sorting
- Aggregate functions, BOARD EXAM ESSENTIALS:
- COUNT(*), COUNT(column), COUNT(DISTINCT column)
- SUM(column), AVG(column), MAX(column), MIN(column)
- Aggregate functions ignore NULL values (except COUNT(*))
- GROUP BY: grouping rows for aggregate calculations
- HAVING clause: filtering groups (WHERE filters rows, HAVING filters groups)
- Difference between WHERE and HAVING, EXAM CRITICAL
- Query execution order: FROM WHERE GROUP BY HAVING SELECT ORDER BY

Projects built:

- 50-query drill: write SELECT queries for given scenarios
- Student performance analysis: average marks by subject, topper per class
- Employee database queries: salary analysis, department-wise stats
- SQL query challenge: 20 real-world business questions

Practice: 100 SELECT query practice problems, Sumita Arora SQL exercises, previous 10 years board SQL questions

SQL Joins, Functions & Advanced Queries

Topics:

- SQL Functions, Single row functions:
- Math functions: POWER(base, exp), ROUND(num, decimals), MOD(a, b)
- TRUNCATE(num, decimals), ABS(num), CEIL(num), FLOOR(num)
- String/Text functions: UCASE()/UPPER(), LCASE()/LOWER()
- MID()/SUBSTRING()/SUBSTR(str, start, length)
- LENGTH(str), LEFT(str, n), RIGHT(str, n)
- INSTR(str, substr), find position of substring
- LTRIM(), RTRIM(), TRIM(), removing spaces
- CONCAT(str1, str2), joining strings
- Date functions: NOW(), DATE(), MONTH(), MONTHNAME()
- YEAR(), DAY(), DAYNAME(), DAYOFWEEK()
- Joins, combining data from multiple tables:
- Cartesian Product: combining every row of table1 with every row of table2
- Equi-Join: Cartesian product with equality condition on common column
- Natural Join: automatic join on common column names
- JOIN syntax: SELECT ... FROM table1, table2 WHERE table1.col = table2.col
- Table aliases in joins: SELECT a.name, b.dept FROM emp a, dept b WHERE a.deptno = b.deptno
- Self-join concept (CBSE bonus)
- Board exam pattern: write SQL query for given output, find errors in SQL

Projects built:

- Multi-table database project: Student-Marks-Subject tables with joins
- Query-the-database challenge: 30 join-based questions
- SQL function practice: 40 function-based queries
- Complete SQL revision, all concepts in one project

Practice: 50 join queries, 30 function queries, Sumita Arora advanced SQL exercises, board exam SQL from last 10 years

Boolean Algebra & Karnaugh Maps (ISC Class 12, runs parallel to SQL for CBSE)

Topics:

- Boolean algebra postulates and theorems, complete set
- Simplification using algebraic method, 10+ practice expressions
- Sum of Products (SOP) form, minterm representation
- Product of Sums (POS) form, maxterm representation
- Canonical SOP and POS from truth tables
- Karnaugh Map (K-Map): 2-variable, 3-variable, 4-variable
- Grouping rules: groups of 1, 2, 4, 8, must be powers of 2
- Don't care conditions in K-Maps
- Simplification using K-Map, extracting minimal SOP/POS
- Verification of simplified expression using truth tables

- Logic circuit design from Boolean expressions
- Implementing circuits using universal gates (NAND only, NOR only)
- Half adder and full adder circuits, truth tables, expressions, diagrams
- Encoder and decoder circuits
- Multiplexer and demultiplexer concepts

Practice: 30 algebraic simplification problems, 30 K-Map problems, circuit design practice, ISC previous year Boolean algebra questions

Months 20-21: Computer Networks & Python-MySQL Connectivity (Week 77-84)

Computer Networks, Complete Chapter (CBSE & ISC)

Topics:

- Evolution of networking: ARPANET NSFNET Modern Internet
- Data communication: sender, receiver, message, transmission medium, protocol
- Types of data: text, numbers, images, audio, video
- Bandwidth: definition, units (bps, Kbps, Mbps, Gbps)
- Communication modes: simplex, half-duplex, full-duplex
- Switching techniques: circuit switching vs packet switching
- IP addressing: IPv4 (32-bit, dotted decimal), IPv6 (128-bit)
- IP address classes: A, B, C, D, E, ranges and default subnet masks
- Public IP vs Private IP addresses
- Transmission media, Guided (wired):
- Twisted pair cable: UTP, STP, uses, advantages
- Coaxial cable: structure, uses (cable TV, earlier ethernet)
- Fiber optic cable: total internal reflection, uses, advantages, fastest guided medium
- Transmission media, Unguided (wireless):
- Radio waves: long range, omnidirectional, FM, AM, WiFi, Bluetooth
- Microwaves: line of sight, point-to-point, cell towers
- Infrared: short range, line of sight, TV remote, IrDA
- Satellite communication: geostationary orbit, VSAT
- Networking devices: modem (modulator-demodulator), Ethernet card/NIC
- RJ45 connector, repeater (signal amplification), hub (broadcasts to all)
- Switch (intelligent forwarding), router (inter-network routing, IP-based)
- Gateway (protocol conversion), bridge (segment connection)
- WiFi card, access point, wireless networking

Projects built:

- Network diagram design for a school/office, choose appropriate devices and media
- IP addressing worksheet, classify 30 IP addresses, identify class
- Transmission media comparison chart, speed, cost, distance, use cases

Practice: Sumita Arora network chapters, 60 MCQs, previous year network questions, case study practice

Network Topologies, Protocols & Web Technologies

Topics:

- Network topologies, physical layout:
- Star topology: central hub/switch, easy troubleshooting, single point of failure
- Bus topology: single backbone cable, terminators, difficult troubleshooting
- Tree/Hierarchical topology: combination of star and bus
- Mesh topology: every device connected to every other, maximum redundancy
- Ring topology: circular connection, token passing
- Types of networks based on geographical spread:
- PAN (Personal Area Network): Bluetooth, within 10m
- LAN (Local Area Network): within a building, Ethernet, WiFi
- MAN (Metropolitan Area Network): across a city, cable TV network
- WAN (Wide Area Network): across countries, Internet is largest WAN
- Network protocols, rules for communication:
- HTTP (HyperText Transfer Protocol): web page transfer, port 80
- HTTPS: secure HTTP with SSL/TLS encryption, port 443
- FTP (File Transfer Protocol): file transfer, port 21
- SMTP (Simple Mail Transfer Protocol): sending emails, port 25
- POP3 (Post Office Protocol 3): receiving emails, port 110
- IMAP: accessing emails on server, port 143
- TCP/IP (Transmission Control Protocol/Internet Protocol): foundational protocol suite
- PPP (Point-to-Point Protocol): direct connection between two nodes
- TELNET: remote login, port 23, insecure
- VoIP (Voice over IP): voice calls over Internet (WhatsApp calls, Zoom)
- Web concepts: WWW, HTML, XML, domain names, URL structure
- URL components: protocol://domain:port/path?query#fragment
- DNS (Domain Name System): translating domain names to IP addresses
- Web browser, web server, web hosting concepts
- Static vs dynamic web pages
- Cookies: what they store, first-party vs third-party

Projects built:

- Network design case study, layout for multi-floor building with 200 computers
- Protocol identification quiz, match scenarios to protocols
- Web technology report, how the Internet works from URL to page load

Practice: 50 protocol MCQs, topology comparison questions, case study practice (CBSE favorite), board exam network questions

Network Security & Python-MySQL Connectivity (CBSE)

Topics:

- Network security threats:

- Malware: virus (needs host), worm (self-replicating), trojan horse (disguised)
- Ransomware: encrypts data, demands payment, WannaCry, NotPetya examples
- Spyware: secretly monitors, keyloggers
- Adware: unwanted advertisements
- Phishing: fake websites/emails to steal credentials
- Pharming: redirecting website traffic to fake sites
- Denial of Service (DoS): overwhelming server with requests
- Man-in-the-middle attack: intercepting communication
- Security measures: antivirus software, firewall (hardware/software)
- Encryption: converting data to unreadable form
- HTTP vs HTTPS: why HTTPS matters, SSL/TLS certificates
- Hacker vs cracker: ethical hacking vs malicious hacking
- Strong passwords: length, complexity, uniqueness
- Two-factor authentication (2FA)
- Python-MySQL Connectivity (CBSE Class 12 CRITICAL):
- Installing mysql.connector: pip install mysql-connector-python
- Importing: import mysql.connector
- Establishing connection: mysql.connector.connect(host, user, password, database)
- Creating cursor object: connection.cursor()
- Executing SQL queries: cursor.execute('SQL query')
- Fetching results: fetchone(), fetchall(), fetchmany(n)
- cursor.rowcount, number of rows affected
- Committing changes: connection.commit(), MANDATORY for INSERT/UPDATE/DELETE
- Closing connection: connection.close()
- Parameterized queries: preventing SQL injection with %s placeholders
- Complete CRUD operations: Create, Read, Update, Delete through Python

Projects built:

- Complete student management system, Python frontend + MySQL backend
- Library management system with Python-MySQL connectivity
- 4 Python-MySQL programs as required for practical file
- Network security awareness presentation

Practice: 30 Python-MySQL programs, connectivity code output prediction, board exam connectivity questions from last 5 years

Months 22-24: Board Exam Preparation, Intensive Revision & Mock Tests (Week 85-96)

Practical File Completion & Project Submission Preparation

Topics:

- CBSE Practical file requirements: 15+ Python programs + 5+ SQL sets + 4+ Python-MySQL programs
- ISC Practical: complete programming assignment compilation
- Python programs list: functions, file handling (text, binary, CSV), stack, sorting, searching
- SQL query sets: DDL, DML, DQL with single and multi-table queries
- Python-MySQL connectivity programs: CRUD operations
- Project selection for board practical: Python file handling OR Python-MySQL application
- Project documentation template: cover page, certificate, acknowledgement
- Project components: introduction, system requirements, source code, output screenshots, bibliography
- CBSE project ideas: Inventory management, Quiz application, Student database, E-commerce simulator
- ISC project ideas: Library system, Banking application, School management, Payroll system
- Viva voce preparation: top 50 questions with model answers
- Internal assessment preparation and marks distribution understanding

Deliverables:

- Complete practical file (printed/handwritten as per school requirement)
- Project with source code, documentation, and output
- Viva preparation notes, 50 Q&A for each subject area

Chapter-Wise Revision, Programming Topics

Topics:

- CBSE: Python revision, functions, scope, recursion (40 marks in board)
- Exception handling revision with practice programs
- File handling complete revision: text, binary (pickle), CSV
- Data structures revision: stack operations, applications
- Sorting algorithms revision: bubble, selection, insertion, code + tracing
- Searching algorithms revision: linear, binary, code + tracing
- ISC: Java revision, OOP, inheritance, polymorphism
- Data structures revision: stack, queue, circular queue, deque
- Linked list complete revision, all operations
- Binary tree traversal revision
- Recursion revision with 15 standard programs
- Complexity analysis revision, Big O for all algorithms

Practice: Topic-wise previous year questions, formula sheets, quick reference cards

Chapter-Wise Revision, SQL, Networks & Theory

Topics:

- SQL complete revision: DDL, DML, DQL commands
- SQL functions revision: math, string, date, aggregate
- GROUP BY, HAVING, ORDER BY, combined query practice
- Joins revision: Cartesian product, equi-join, natural join
- CBSE: Python-MySQL connectivity revision, connection, cursor, execute, fetch, commit
- Computer networks complete revision: media, devices, topologies, protocols

- ISC: Boolean algebra revision, laws, simplification, K-maps
- Society, law and ethics revision: cybercrime, IPR, IT Act, e-waste
- Case study questions practice, network design scenarios
- Assertion-Reason questions practice, all chapters

Practice: 100 SQL queries revision, network diagram practice, case study solving

Mock Board Exams, Set 1 & 2

Topics:

- Full-length Mock Exam 1: 70 marks, 3 hours, strict exam conditions
- Detailed solution discussion and marking scheme analysis
- Common mistakes identification and correction
- Full-length Mock Exam 2: 70 marks, 3 hours, different question set
- Detailed solution discussion with alternative approaches
- Marks improvement strategies based on mock performance
- Section-wise time allocation strategy:
- Section A (21 marks): 35 minutes, 1.5 min per MCQ
- Section B (14 marks): 20 minutes, 3 min per 2-mark question
- Section C (9-12 marks): 20 minutes, 5 min per 3-mark question
- Section D (8-16 marks): 25 minutes, 6 min per 4-mark question
- Section E (10-15 marks): 20 minutes, 8 min per 5-mark question
- Presentation tips: writing code in exams, proper indentation, comments

Deliverables:

- 2 complete mock papers solved and reviewed
- Personal weak areas identified and improvement plan
- Time management skill developed through timed practice

Mock Board Exams, Set 3, 4 & 5 + Previous Year Paper Analysis

Topics:

- Mock Exam 3, 4, 5: Full-length papers with varying difficulty
- CBSE Previous Year Papers: 2020, 2021, 2022, 2023, 2024, 2025, full solved
- ISC Previous Year Papers: 2020, 2021, 2022, 2023, 2024, 2025, full solved
- CBSE Sample Paper 2026-27, detailed solution with marking scheme
- ISC Specimen Paper 2026-27, detailed solution
- Frequently repeated questions analysis, top 30 most-asked topics
- Marking scheme tricks: how examiners award step marks
- Partial credit strategies: getting marks even if full answer unknown
- Common errors students make, and how to avoid them
- Important program outputs that repeat in exams

Practice: 5 mock papers + 6 previous year papers = 11 full papers solved

Final Revision & Last-Minute Board Exam Tips

Topics:

- One-shot revision: complete syllabus in 2 days, key points only
- Formula sheet / cheat sheet revision: all Python/Java syntax, SQL commands, network concepts
- Top 50 most important programs, practice writing by hand
- Top 30 SQL queries, write without thinking
- Top 20 output prediction questions, instant answer practice
- Board exam day strategy: reading time usage, question selection
- How to write Python code in board exams: indentation, proper syntax
- How to write SQL queries: uppercase commands, proper formatting
- How to draw network diagrams: clean, labeled, with proper legends
- Common last-minute doubts cleared
- Confidence building: you've prepared more than 95% of students
- Final practical exam preparation: lab test strategy, viva confidence

Deliverables:

- Quick revision notes, entire syllabus on 10 pages
- Top programs list, 50 most important programs
- SQL quick reference, all commands on 2 pages
- Network concepts summary, devices, protocols, topologies on 1 page
- READY FOR BOARD EXAM, 95+ score target

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Class 11 Sem 1): menu-driven console programs - calculators, grade and tax calculators, pattern printers, and number-logic programs built while learning Python (CBSE) or Java (ISC) fundamentals
- Phase 2 (Class 11 Sem 2): the complete Class 11 practical file with 20+ programs, plus a documented mini project such as a quiz game, library manager, or student database
- Phase 3 (Class 12 Sem 1): file handling utilities for text, binary (pickle), and CSV files, a record management system with search, update, and delete, and a menu-driven stack implementation
- Phase 4 (Class 12 Sem 2): SQL query sets on a live MySQL database and the final board project - for CBSE, a Python plus MySQL connectivity application with full documentation and viva preparation
- Total: 2 board-ready practical files, 2 submission projects, and 40+ graded programs over the two years

Total projects: 40+ graded programs and mini projects across Class 11 and 12, including 2 complete board practical files and 2 board-submission projects

SKILLS THE STUDENT WILL HAVE AT THE END

- Python Programming (CBSE 083): variables, control flow, strings, lists, tuples, dictionaries, functions, and exception handling
- File Handling: text files, binary files with the pickle module, and CSV files with the csv module
- Data Structures: stack implementation using lists with push, pop, peek, and application problems
- Databases and SQL: DDL, DML, aggregate functions, GROUP BY, HAVING, and joins on MySQL
- Python-MySQL Connectivity: connecting Python to MySQL, executing queries, and fetching results from a program
- Computer Networks: network types, topologies, transmission media, devices, protocols, and web basics
- Java and OOP (ISC batch): classes, constructors, inheritance, polymorphism, recursion, arrays, and data structures
- Exam Technique: output tracing, error spotting, and writing answers to the CBSE and ISC marking schemes

HOW A TYPICAL WEEK IS SPLIT

Live classes: 3-4 live classes per week, 1 hour each, taught chapter by chapter in sync with the school syllabus
Practical work: 2-3 hours of guided coding practice: practical file programs, SQL queries on a live MySQL database, and output-tracing drills
Board style practice: Weekly chapter tests plus previous year questions, sample papers, and answer writing trained on the CBSE and ISC marking schemes
Doubt support: Dedicated doubt-clearing sessions, with extra slots before school exams, pre-boards, and board exams

SUPPORT INCLUDED

Small batches: Small group batches so every student gets individual attention and screen time
Board specific batches: Separate CBSE (Python) and ISC (Java) batches - syllabus and language are never mixed
Exam season support: Extra revision classes and doubt sessions before pre-boards and board exams
Practical and project help: Step-by-step help with practical file programs and the board project until they are submission-ready
Progress tracking: Regular test results and progress updates shared with parents

CERTIFICATION DETAIL

Course certificate: Modern Age Coders Course Completion Certificate at the end of each academic year
Board exam readiness: Structured revision with 10 years of solved board papers and full mock exams on the current CBSE (083) and ISC pattern: 70 marks theory plus 30 marks practical
Practical file support: Complete guidance on every practical file program, the board project with documentation, and viva preparation for the 30-mark practical exam

WHERE THIS CAN LEAD

- B.Tech / BE in Computer Science or IT with a strong programming foundation before college begins
- BCA, B.Sc Computer Science, and B.Sc IT admissions
- JEE and other engineering entrances with Computer Science as a scoring board subject
- Indian Computing Olympiad (ZIO/ZCO) and school-level coding competitions
- College-level data structures and competitive programming, building on the Class 12 stack and algorithms work
- Python certifications such as PCEP after the course
- Early internships and freelance projects using Python and SQL
- Data science and AI/ML degree specializations that expect Python fundamentals

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Are CBSE and ICSE batches really separate?
A: Yes, absolutely. CBSE uses Python and ICSE/ISC uses Java. The syllabus, exam pattern, marking scheme, practical requirements, and even the thinking approach are different. We run completely separate batches with specialized mentors for each board.

Q: Can I join if I have zero programming experience?
A: Yes! Both our CBSE and ISC courses start from absolute scratch. We assume no prior coding experience. You'll learn everything from installing Python/Java to solving complex board-level programs.

Q: Which textbook do you follow?
A: For CBSE, we primarily follow Sumita Arora (Computer Science with Python) supplemented by NCERT textbook. For ISC, we follow Understanding Computer Science by Pandey & Dey (APC Publications) and Sumita Arora ISC edition. Our teaching covers every chapter from these books.

Q: Will this course help me score 95+ in boards?
A: We focus on both understanding and exam strategy.

Q: Do you cover practical file and project work?
A: Yes, completely. We prepare your entire practical file (20+ programs for CBSE, lab assignments for ISC), guide you through the project with documentation, and prepare you for the viva voce examination.

Q: What is the difference between CS and IP in CBSE?
A: CS (Code 083) focuses on deep Python programming, file handling, data structures (stack), sorting/searching algorithms, and Python-MySQL connectivity. IP (Code 065) focuses on data handling with Python Pandas, data visualization with Matplotlib, and SQL. CS is harder and more programming-intensive; IP is application-oriented. Both include SQL.

Q: Is ISC Computer Science harder than CBSE CS?
A: ISC CS covers more advanced topics like linked lists, binary trees, Karnaugh maps, and has a stronger focus on OOP through Java. CBSE CS has more practical connectivity topics (Python-MySQL). Both are challenging in their own ways, our separate batches ensure you're prepared for YOUR specific board.

Q: Do you help with competitive programming too?
A: While this course focuses on board exam preparation, the strong programming foundation you build here, especially in ISC with data structures and algorithms, is an excellent stepping stone for competitive programming. We offer a separate competitive programming course for those interested.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/cbse-icse-computer-science-class-11-12-python-java-complete-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 46 OF 137: COPILOT STUDIO AGENT MASTERCLASS FOR PROFESSIONALS
==========

Course link: https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-masterclass-for-professionals
Subject area: AI Agents & Automation
Designed for: College students
Who can join: Intermediate to advanced (working professionals; no prior agent experience assumed, business software fluency expected)
Course length: 24 classes (12 weeks · 2 classes/week)
Time needed: 4-6 hours/week recommended
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: one to one Rs 4,999
Outside India, per month: one to one $150
Classes: one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.
Note: this course is taught one to one only. There is no group or mini batch option.
Note: the course length above is counted at two classes a week. In India the one to one plan is one class a week, so if an Indian family asks how long it will take, hand over to Shivam Sir.

WHAT THIS COURSE IS

Ship production AI agents in Microsoft Copilot Studio: 24 live 1-on-1 online classes for working professionals on enterprise grounding, autonomous agents, multi-agent A2A, governance and ROI.

In one line: Most agent projects die in pilot. Learn to build, evaluate, govern and cost agents that survive contact with your organisation, in private 1-on-1 classes built around your actual work.

ABOUT THE PROGRAMME

Gartner projects that over 40% of agentic AI projects will be canceled by the end of 2027, and that by 2027, 40% of enterprises will demote or decommission autonomous agents due to governance failures. The builders who survive that filter are the ones who can answer the unglamorous questions: what is this agent grounded in, who approved its connector permissions, what does a conversation cost in Copilot Credits, and how do we know it is actually resolving anything? This masterclass teaches exactly that, on Microsoft Copilot Studio, the graphical low-code studio that 90% of the Fortune 500 use by Microsoft's own count. Across 24 live 1-on-1 classes you build the full professional stack: enterprise knowledge grounding across SharePoint, Dataverse and connector-indexed enterprise data with tenant graph grounding; designed topics and generative orchestration; the complete tool surface from prebuilt and premium connectors through custom connectors, REST, Prompt tools, agent flows, Model Context Protocol under DLP policies, and computer use with access control,

HTTPS enforcement, stored credentials and human supervision; autonomous agents on event triggers, hardened with approval pauses and run under the maker-credentials rule; multi-agent architectures with child agents, connected agents and the A2A protocol; agent evaluations with test sets and graders; analytics down to outcomes, CSAT and the cost savings metric; and the governance layer that makes it all deployable, from mandatory Microsoft Entra Agent IDs and Conditional Access to Microsoft Agent 365 as the control plane and the Agent Store approval path. You finish with a production-grade capstone agent for your own team plus the governance and ROI pack to defend it in front of your leadership. Because every class is 1-on-1, the whole course runs on your organisation's real use cases, discussed at whatever depth your role demands. The syllabus is a Living Syllabus, re-verified against Microsoft's documentation as the platform ships, so what you learn is the August 2026 platform, not a 2024 recording.

WHAT MAKES THIS COURSE DIFFERENT

- 1-on-1 only, by design: production agent work involves your real processes, your data boundaries and your org's licensing, none of which can be discussed honestly in a group class; a private mentor can go exactly as deep as your role requires.
- Governance is taught as a first-class skill, not an afterthought: Microsoft Entra Agent ID (mandatory for new agents since July 2026), Conditional Access over connector permissions, DLP policies, Microsoft Agent 365 and the Agent Store approval path are all in the syllabus.
- Cost fluency that most builders lack: you learn the Copilot Credits model down to per-feature rates, what a Microsoft 365 Copilot licence covers at no charge, what it does not (computer use), and the 125% capacity cutoff that silently disables over-budget agents.
- The full 2026 tool surface: Prompt tools, agent flows, premium and on-premises connectors, custom connectors, REST, Model Context Protocol, computer use with enterprise guardrails, and deep reasoning, taught with their real limits and billing behaviour.
- Quality you can prove: agent evaluations (GA March 2026) with test sets, graders and multi-turn tests, plus analytics outcomes, CSAT and the cost savings metric, so your capstone ships with evidence, not vibes.
- Living Syllabus: Copilot Studio ships monthly (the current model lineup includes GPT-5.5 Chat and Anthropic Claude Opus 4.7; the A2A protocol went GA in April 2026), and this course is re-verified against Microsoft's What's New pages as features land.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop (Windows, Mac or Linux) with the Chrome or Edge browser and a reliable internet connection; Copilot Studio runs fully in the browser

Coding experience: None required. Copilot Studio is a graphical, low-code studio, and this course teaches the design, governance and cost disciplines around it. Professionals who do code can go deeper into custom connectors, REST tools and Power Fx expressions inside the same syllabus

Accounts: Copilot Studio uses a Microsoft work or school (Entra ID) account. With a work email you can self-serve the 30-day trial (which cannot publish agents); ideally you use your employer's tenant with your admin's awareness, and we arrange a practice environment where that is not possible. Your mentor sorts this out with you in the first class

Mindset: The willingness to treat agents as production software: tested before trusted, costed before scaled, and governed before shipped. The course rewards professionals who ask 'who approved this and what does it cost' as instinctively as 'does it work'

WHO THIS COURSE IS FOR

Operations and business analysts: Analysts and process owners who know exactly which workflows are broken and want to ship the agent that fixes one, with the evidence to defend it

It and m365 admins: IT professionals and Microsoft 365 admins who will be asked to govern agents either way, and would rather be the person who understands Entra Agent ID, DLP and Agent 365 before the first incident

Developers and makers: Developers and Power Platform makers adding the 2026 agent stack (MCP, A2A, computer use, evaluations) to skills employers already recognise

Team leads and managers: Leads and managers who must decide what gets automated, what it costs in Copilot Credits, and which agent projects deserve to survive the pilot filter

Consultants and freelancers: Consultants who want to sell agent builds that pass a client's security review, with governance and ROI packs as deliverables, not afterthoughts

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PLATFORM & FOUNDATIONS

Map the 2026 agent landscape, learn the anatomy of a Copilot Studio agent, build instruction and testing discipline, and make the model and harness decisions that shape cost and capability.

Class 1: The 2026 Agent Landscape and Where Copilot Studio Sits

Topics:

- What Microsoft ships in 2026: Copilot Studio as a graphical, low-code studio for building and

managing AI-powered agents and workflows, run standalone at copilotstudio.microsoft.com on Power Platform infrastructure
- The ownership decision Microsoft itself recommends making first: Copilot Studio (SaaS, low-code, managed guardrails) vs Microsoft Foundry (PaaS, pro-code, fine-grained control), and how the two interoperate rather than compete
- The three build artifacts by their exact names: agents, workflows and agent flows, and which business problems map to which
- Why the market filter matters to you: Gartner's finding that over 40% of agentic AI projects will be canceled by end of 2027, and what the surviving 60% do differently
- Environments and tenants: where agents live, why production and practice environments are separated, and what your employer's admin controls
- Access honestly stated: Copilot Studio needs a Microsoft work or school (Entra ID) account; a work email gets a 30-day self-service trial that cannot publish agents, and this course arranges a practice environment where needed
- Choosing your capstone: one real process from your own team that an agent could genuinely improve, nominated in Class 1 and refined all course

Projects built:
- Write the one-page 'agent opportunity memo' for your organisation: three candidate processes, the data each would need, the risks each carries, and which one becomes your capstone

Practice: Audit two agents you already encounter at work (a support bot, an IT helpdesk) and reverse-engineer their instructions, knowledge and escalation rules.
Assessment: Defend your capstone choice in five minutes: the process, the users, the data, the risk, and what 'better' will measurably mean.

Class 2: Agent Anatomy and Instruction Discipline

Topics:
- The anatomy every agent shares: name, description, instructions, knowledge, tools, triggers, channels, and where each is configured
- The new-experience component names as Microsoft spells them: Instructions, Knowledge, Tools and skills, Model, Connected agents, Memory
- The instruction stack for business agents: identity, scope, tone, format rules, refusals and escalation, in that order
- Scope fences that survive real users: what the agent must not answer (HR disputes, legal advice, pricing exceptions) and wording refusals that de-escalate
- Describe-to-create and Build with natural language (preview): what the generators give you and why professionals still rewrite the instructions by hand
- Instruction anti-patterns from real deployments: contradictions, vague verbs, policy text pasted verbatim, and instructions that fight the knowledge sources
- Versioning instructions outside the tool from day one, because 'what changed last Tuesday' is a governance question

Projects built:
- Build 'Policy Desk v1': an internal FAQ agent for one real policy area of your workplace, with a complete instruction stack and five tested refusals

Practice: Run the same 10 workplace questions against two instruction variants and score every difference in a comparison table.
Assessment: Present your instruction stack line by line and state, for each line, what production incident removing it would eventually cause.

Class 3: Testing Like It Is Already in Production

Topics:
- The fixed test set: happy path, edge case and adversarial questions written before you tune anything
- Adversarial testing for business agents: prompt injection attempts, policy-boundary probes, and the disgruntled-user tone test
- Regression discipline: rerunning the full set after every instruction, knowledge or tool change, and logging verdicts
- Hallucination triage: distinguishing missing knowledge, bad grounding and model overreach, because each has a different fix
- The escalation path as a tested feature: 'this needs a human' is a designed outcome, not a failure
- What you are rehearsing for: agent evaluations (GA March 2026) automate exactly this loop with test sets and graders in Phase 4
- The rule this course grades you on: no change ships without a retest, and every retest leaves a written trail

Projects built:
- Build the permanent 25-question test set for Policy Desk, including five adversarial probes, with expected behaviour written before the first run

Practice: Attempt three prompt-injection attacks on your own agent, document the outcomes, and patch the weakest point.
Assessment: Given a transcript with three subtle failures, identify each one and classify it: instructions, knowledge or escalation gap.

Class 4: Models and Harnesses: The Decisions Underneath Everything

Topics:
- The model picker in August 2026: GPT-4.1 as default, GPT-5 Chat and GPT-5.5 Chat GA, Anthropic Claude Sonnet 4.6, Claude Opus 4.6 and Claude Opus 4.7 GA, with GPT-5 Reasoning and GPT-5 Auto in preview
- External models are an admin decision: Anthropic, Mistral and xAI models must be allowed in both the Power Platform admin center and the Microsoft 365 admin center before makers can

pick them
- Model release types you must read correctly: Experimental, Preview, GA, Default, Retired (GPT-4o and Claude Sonnet 4.5 are already retired), and why production agents pin to GA
- Harnesses, the 2026 architectural change: the runtime between your design and the model, in three flavours, standard harness, GitHub Copilot harness and Copilot chat harness
- The GitHub Copilot harness honestly costed: most capable, creates Word, Excel, PowerPoint and PDF files natively, supports skills and memory, and bills Copilot Credits from the moment you start building, including preview and testing
- Agents cannot move between harnesses, so the harness choice is made once, per agent, with eyes open
- The Copilot Studio extension for Visual Studio Code (GA January 2026): where low-code agents meet source control and code review

Projects built:
- Run your Policy Desk test set against two GA models (one OpenAI, one Anthropic), score the differences, and write the one-paragraph model decision with evidence attached

Practice: Draft the admin request you would send to enable an external model in your tenant, naming both admin centers and the business justification.
Assessment: Phase 1 gate: rebuild a small business agent from a written spec in 30 minutes, with instruction stack, refusals, a 10-question test and a defended model choice.

PHASE 2: ENTERPRISE KNOWLEDGE

Ground agents in enterprise sources at scale: SharePoint, Dataverse, documents and connector-indexed data, tenant graph grounding, curation strategy, and accuracy audits an auditor would accept.

Class 5: Grounding Strategy: The Knowledge Source Estate

Topics:
- The knowledge source lineup with real limits: public websites via Bing (25 websites), uploaded documents stored in Dataverse (512 MB per file, up to 500 files), SharePoint (25 URLs), Dataverse tables, and enterprise data via connectors indexed by Microsoft Search
- The hard ceilings that force strategy: 500 knowledge sources per agent and 1,000 topics per agent mean knowledge is curated, never dumped
- Generative answers under the hood: how the agent searches sources and composes cited answers, and what 'Allow ungrounded responses' changes
- Citations as a compliance feature: why grounded-with-sources is the difference between an answer and a liability
- Matching source type to content type: policies in SharePoint, structured records in Dataverse, public facts on the website, and why mixing them carelessly produces conflicts
- Web search as a deliberate exception: what 'Use information from the web' adds and when a curated internal agent must leave it off
- Designing the capstone's source map: authoritative, current, permissioned, and small enough to audit

Projects built:
- Ground Policy Desk in a real SharePoint site (or a mirrored practice copy) and prove ten answers cite the correct policy paragraph

Practice: Ask your grounded agent five questions its sources cannot answer and document the exact edge-of-knowledge behaviour.
Assessment: Present your capstone source map: every source, its owner, its update cadence, and why it earns its place under the 500-source discipline.

Class 6: SharePoint and Documents Done Right

Topics:
- Writing documents for retrieval: headings that answer questions, one fact per paragraph, consistent terminology, and the anti-pattern of 40-page PDFs
- SharePoint hygiene that grounding exposes: stale copies, duplicate policies, draft folders, and the librarian pass that fixes them
- Permissions reality: agents surface what the knowledge configuration can reach, so source selection is a security decision, not just a quality one
- Freshness engineering: which source updates win, what happens when the underlying document changes, and update cadences you can promise
- Testing document knowledge surgically: questions whose answers live in one exact paragraph, and what retrieval failure looks like
- The curation multiplier: five well-structured pages outperform fifty messy ones, demonstrated live on your own content
- Data minimisation at the source: what never belongs in agent knowledge (personal data, credentials, unreleased financials)

Projects built:
- Rebuild one messy real document into a retrieval-optimised version, ground both versions side by side, and publish the before-and-after accuracy comparison

Practice: Run a librarian audit on one SharePoint area you actually use: what is in, what is stale, what would embarrass the agent that cites it.
Assessment: Given two document structures, predict which grounds better and prove it with a five-question retrieval test.

Class 7: Dataverse, Connector-Indexed Data and Tenant Graph Grounding

Topics:

- Dataverse as knowledge: structured tables behind conversational answers, and when structure beats prose
- Enterprise data via connectors, indexed by Microsoft Search: reaching systems beyond Microsoft 365 without exporting them
- Tenant graph grounding with semantic search: retrieval over the tenant's Microsoft Graph, what it unlocks, and the fact that it costs extra Copilot Credits (10 per use)
- Source arbitration: what happens when SharePoint, Dataverse and a website disagree, and how instructions scope question types to sources
- Knowledge source health: monitoring which sources actually get used, and retiring the ones that never answer
- Foundry IQ and Microsoft IQ in preview: where Microsoft is taking enterprise knowledge, flagged honestly as preview, not promised
- Cost-aware knowledge design: grounding choices are billing choices, previewing the credit mathematics of Class 20

Projects built:
- Add a second source type to Policy Desk (Dataverse table or connector-indexed source), then document the arbitration behaviour with five conflict questions

Practice: Write the scoping instructions that route 'how many' and 'which record' questions to structured sources and 'what is the policy' questions to documents.
Assessment: Whiteboard the retrieval path of one question end to end: user, orchestration, source choice, retrieval, citation, answer.

Class 8: Accuracy Audits and Honest Agents
Topics:
- The accuracy audit protocol: scoring 25 grounded answers against the actual source text, with a rubric a non-builder could apply
- Uncertainty language as policy: agents that say 'my sources do not cover that' instead of improvising, and how instructions enforce it
- Bias in the source list: how a one-sided knowledge estate produces a one-sided agent, and the review that catches it
- The 'Allow ungrounded responses' decision as a governance artifact: who signs it, and for which agent categories it is defensible
- Transparency obligations: users must know they are talking to an agent and what it can reach
- The audit trail: accuracy numbers, test logs and source maps as the documents that survive an internal review
- Phase 2 gate: the grounding checklist you will apply to every agent for the rest of your career

Projects built:
- Run the full 25-answer accuracy audit on Policy Desk and publish the result as a one-page report with examples, methodology and a remediation list

Practice: Have a colleague ask your agent ten unscripted questions from their real work and log every answer they would not accept.
Assessment: Phase 2 exam: design the complete knowledge plan for a new use case in writing (sources, limits, scoping, refusals, audit protocol) in 30 minutes.

PHASE 3: ORCHESTRATION & TOOLS

Design conversations for critical paths, choose orchestration modes deliberately, then master the full tool surface: Prompt tools, agent flows, connectors, custom connectors and REST, Model Context Protocol under DLP, computer use with guardrails, and deep reasoning.

Class 9: Topics for Critical Paths
Topics:
- Topics as designed conversation: when a business process demands the same path every time, you design it, you do not hope for it
- Trigger phrases at scale: teaching the many phrasings of one request (topics support up to 200 trigger phrases)
- Question nodes, entities and variables: collecting exactly the data a process needs, with validation for messy human answers
- Condition groups (June 2026): combining tests cleanly, and the escalation branch every customer-facing topic must have
- Adaptive cards for business: structured, tappable responses for approvals, summaries and forms, designed for phones first
- System topics: greeting, escalate, goodbye, and why you customise rather than delete them
- The topic map for your capstone: the five jobs users will actually ask it to do, each with an owner path

Projects built:
- Build 'Support Triage v1': a customer-facing agent with three designed topics (report an issue, check status, escalate to human), entities capturing the details, and an adaptive card summary

Practice: Collect ten real phrasings of one request from colleagues and tune trigger phrases until all ten route correctly.
Assessment: Given four user messages, predict the topic each triggers and the variables each fills, then verify live.

Class 10: Generative Orchestration, Deliberately
Topics:
- The two orchestration modes: classic (trigger phrases select topics) versus generative (the

model plans across topics, knowledge and tools), and what each buys you
    - What generative orchestration unlocks: multi-intent requests, dynamic tool choice, and the event triggers and autonomy of Phase 4, which require it
    - What you trade: determinism, and the instruction patterns that rein the planner back onto approved paths
    - Reading the plan: inspecting how the agent chose a topic, source or tool, and turning surprises into design fixes
    - The hybrid pattern professionals actually ship: designed topics for critical and regulated paths, generative flexibility for the long tail
    - Re-running your guardrail tests after the mode switch, because refusals and scope fences must survive orchestration
    - Microsoft's own scale heuristic, previewed: when an agent grows past 30 to 40 choices of tools, topics and agents, you split it, which is where Phase 4's multi-agent work begins
  Projects built:
    - Migrate Support Triage to generative orchestration, rerun the full test set, and publish the behaviour-change log with a keep-or-fix verdict per change
  Practice: Craft three multi-intent messages ('report this, and also what happened to yesterday's ticket, and who do I call') and tune until all three resolve cleanly.
  Assessment: Defend your capstone's orchestration choice in writing: mode, why, and the two risks you are explicitly accepting.

Class 11: Prompt Tools and Agent Flows
  Topics:
    - The tool menu by its exact names: Prompt, Agent flow, Computer use, Custom connector, Model Context Protocol and REST API, plus prebuilt connectors
    - Prompt tools: one perfect transformation (summarise, classify, extract, rewrite) with a tested prompt behind a stable interface
    - Agent flows: deterministic multi-step automations the agent calls as tools, for anything that must run the same way every time
    - Flow anatomy in practice: trigger, steps, conditions, outputs handed back to the conversation, and the error paths that make failure graceful
    - The approvals pattern: a flow that pauses for a human decision, the single most reused pattern in business agents
    - Tool descriptions as the real API: the orchestrator chooses tools by reading descriptions, so you write them like documentation
    - Tool discipline: the platform allows 128 tools per agent under generative orchestration, Microsoft recommends 25 to 30, and good architects use fewer
  Projects built:
    - Build 'Expense Helper': conversation collects an expense claim, an agent flow validates it, routes it for approval, and returns the outcome, with tested error paths for every step
  Practice: Break your flow deliberately (missing field, rejected approval, timeout) and verify the user sees a designed message for each failure.
  Assessment: Given five business tasks, assign each to Prompt tool, agent flow, topic or knowledge, and justify each assignment in two sentences.

Class 12: The Connector Estate: Standard, Premium, On-Premises, Custom, REST
  Topics:
    - The scale of the estate: more than 1,400 prebuilt connectors, categorised as Standard, Premium, Custom and On-premises, with licensing implications per category
    - Using prebuilt connectors as tools: reading and writing real systems (SharePoint, Outlook, Planner, third-party SaaS) from inside a conversation
    - Authentication honestly explained: connection credentials, who consents to what, and why 'connect your account' is a governance event
    - On-premises connectivity: reaching systems behind the firewall via the gateway, and when that is the right call versus an API
    - Custom connectors: wrapping your organisation's own REST API so agents (and flows) can call it like any other connector
    - The REST API tool: calling an endpoint directly, mapping JSON responses into conversation variables
    - DLP preview: connector access is policy-controlled, and the same DLP machinery will govern MCP in the next class
  Projects built:
    - Wire one real connector tool and one REST or custom connector tool into a practice agent, and demonstrate a conversation that reads and writes live data through both
  Practice: Inventory the five systems your capstone's users touch daily and classify the connector path to each: prebuilt, premium, custom or REST.
  Assessment: Sketch one tool call end to end (utterance, orchestration, tool selection, auth, call, result, answer) and mark every point where governance can intervene.

Class 13: Model Context Protocol Under Enterprise Governance
  Topics:
    - MCP in one sentence: the open protocol for plugging tool servers into agents, added in Copilot Studio via Tools, Add a tool, Model Context Protocol
    - What an MCP server exposes: Tools and Resources, with per-tool toggles and the Allow all switch you will learn to distrust
    - The governance fact that makes MCP enterprise-safe here: MCP runs over Power Platform connector infrastructure, so DLP data policies govern MCP access like any connector
    - Why MCP fluency compounds: the same protocol appears in Claude, IDEs and other agent platforms, so this is portable knowledge

- The Windows 365 for Agents MCP server (GA June 2026): full operational control of a Cloud PC as a case study in how much power a single MCP server can carry
- Evaluating a third-party MCP server before connecting it: capabilities, data flows, failure modes, and the least-power principle
- MCP versus custom connector versus REST: three ways to reach the same system, chosen by governance posture, not fashion

Projects built:
- Add an MCP server to a practice agent, disable every tool you do not need, and document the resulting capability surface as a one-page security note

Practice: Write the DLP review request for your MCP integration as you would submit it to your Power Platform admin, naming the data boundaries.

Assessment: Explain to a skeptical security lead, in three minutes, why this MCP integration is governable and what would make you switch it off.

Class 14: Computer Use and Deep Reasoning: The Power Tools

Topics:
- Computer use (GA 2026): Computer-Using Agents that operate real user interfaces when no API exists, and why it is the tool of last resort, not first
- The model menu with per-step credit costs: OpenAI Computer-Using Agent at 5 credits per step, Claude Sonnet 4.6 at 5, Claude Opus 4.6 at 15 as the premium option
- The four guardrails you configure before any computer-use agent runs: Access control allow-lists, Enforce HTTPS, Stored credentials (internal or Azure Key Vault), Human supervision
- Costing a computer-use task honestly: steps multiply, credits are per step, and computer use is not covered by the Microsoft 365 Copilot licence
- Deep reasoning (preview): the Azure OpenAI o3 model invoked by the keyword 'reason' in agent instructions, US and EU availability, and why preview features stay out of production commitments
- Choosing between a designed flow, an API call and computer use for the same task: the decision tree with cost and fragility on it
- Phase 3 gate: the complete tool architecture of your capstone, every tool justified

Projects built:
- Build a governed computer-use demo: one legacy-UI task automated with allow-lists, HTTPS enforcement and human supervision configured, plus a per-run credit cost estimate

Practice: Take one process from your workplace with no API and write the computer-use feasibility note: steps, guardrails, cost per run, and the API alternative if any.

Assessment: Phase 3 exam: given a use case, design the full tool architecture (tools chosen, tools rejected, governance controls, cost notes) and defend it.

PHASE 4: AUTONOMY, MULTI-AGENT & QUALITY

Ship agents that act on their own and prove they work: event triggers under the maker-credentials rule, hardening, multi-agent architectures with A2A, agent evaluations, analytics, and full cost management in Copilot Credits.

Class 15: Event Triggers and Autonomous Agents

Topics:
- The autonomy switch: event triggers let agents act with no user in the chat, on events like item created in SharePoint, file created in OneDrive, task completed in Planner, and Recurrence schedules
- Generative orchestration required: autonomous behaviour runs on the planning brain, which is why Class 10 preceded this one
- The trigger payload: event data arriving as input, and instructions deciding what the agent does with it
- The security fact that shapes everything: triggers run with the agent maker's credentials, so every autonomous action is effectively signed by you
- Designing the first autonomous job: small, observable, reversible, and logged
- The activity trail habit: knowing exactly what your agent did at 3am, before anyone asks
- Autonomy scoping: which of your capstone's jobs deserve a trigger, and which must stay human-initiated

Projects built:
- Build 'Weekly Reporter': an autonomous agent on a Recurrence trigger that compiles a weekly status summary from real sources and posts it for review, never directly to stakeholders

Practice: Let Weekly Reporter run unattended for three real events and audit every action against what you intended, scoring each.

Assessment: Explain the maker-credentials rule and name the three design constraints you imposed on your agent because of it.

Class 16: Hardening Autonomy: Approval Pauses and Failure Design

Topics:
- From demo to dependable: idempotence (running twice must not double the damage), retries, and safe defaults when inputs surprise you
- Human-in-the-loop as architecture: which autonomous actions pause for approval, how the pause is built, and who the approver is on paper
- Notification etiquette: an autonomous agent that spams gets disabled by its own users, so frequency and channel are design decisions
- Failure diaries: structured capture of what went wrong, so Tuesday's incident is not repeated on Thursday

    - Scope creep in autonomy: the 'it could also just' trap, and the change-control habit that resists it
    - Gartner's warning as a design brief: 40% of enterprises will demote or decommission autonomous agents by 2027 due to governance failures, and hardened agents are how you stay in the other 60%
    - The operations note: one page stating what the agent does, when, with whose permissions, and how to turn it off
  Projects built:
    - Harden Weekly Reporter: add an approval pause before anything leaves draft, an error diary, and a self-report of its own actions, then run it for a full real week
  Practice: Write the operations note for Weekly Reporter and have a colleague verify they could disable the agent using only that page.
  Assessment: Live drill: your mentor corrupts one input source; your agent must fail gracefully and your diary must show why, unedited.

Class 17: Multi-Agent Architecture and the A2A Protocol
  Topics:
    - When one agent becomes several: Microsoft's own heuristic, split when an agent exceeds 30 to 40 choices of tools, topics and agents
    - The four ways to add agents to an agent, by their exact names: child agents, other Copilot Studio agents, agents over the A2A protocol, and (preview) Microsoft Foundry agents, Fabric Data agents and Microsoft 365 Agents SDK agents
    - The A2A protocol, GA since April 2026: agent-to-agent interoperability beyond a single vendor's walls
    - Child agents in practice: decomposing Support Triage into a router plus specialists, each with its own 128-tool budget
    - Interop with Microsoft Foundry: low-code front door, pro-code specialist behind it, the documented enterprise pattern
    - Failure across agent boundaries: timeouts, unclear hand-offs, and the tracing habits that keep multi-agent systems debuggable
    - Architecture review: when multi-agent is real engineering and when it is complexity cosplay
  Projects built:
    - Refactor Support Triage into a two-agent architecture: a routing agent plus a specialist child agent, with documented hand-off contracts and a failure-mode table
  Practice: Diagram your capstone as both a single agent and a multi-agent design, and write the three-sentence verdict on which ships.
  Assessment: Defend one hand-off contract: what crosses the boundary, what must never cross it, and what happens when the far side fails.

Class 18: Agent Evaluations: Quality You Can Prove
  Topics:
    - Agent evaluations, GA March 2026: automated test sets run against your agent, the industrial version of the discipline you built by hand in Class 3
    - Evaluation anatomy: test sets, graders scoring responses, multi-turn tests for whole conversations, and activity maps showing what the agent did
    - Writing graders that mean something: correctness, groundedness, tone and policy compliance as separately scored dimensions
    - The evaluation REST API (preview): wiring evaluations into release habits, flagged honestly as preview
    - Evaluation-gated changes: no instruction, knowledge or tool change ships without a green evaluation run
    - Building the capstone's evaluation suite: the 30 conversations that define 'working' for your use case
    - Evaluations as governance evidence: the artifact you hand to a reviewer instead of a demo
  Projects built:
    - Build the full evaluation suite for your capstone: 30 test conversations, graders for correctness and policy compliance, and a baseline report you will beat by Class 23
  Practice: Make one deliberate regression (weaken an instruction), prove the evaluation suite catches it, then revert.
  Assessment: Present your grader definitions and defend why a passing score actually means the agent is safe to ship.

Class 19: Analytics: Outcomes, CSAT and the Cost Savings Metric
  Topics:
    - The analytics view by its real dimensions: sessions, outcomes (Resolved, Escalated, Abandoned), daily and monthly active users, CSAT, and themes
    - The cost savings metric and custom metrics: putting a defensible number on what the agent is worth
    - Transcript review as design fuel: session details and transcripts are retained 28 days, analytics 360 days, so review cadence is a calendar item, not a hope
    - The Analytics Viewer role: giving stakeholders the numbers without giving them the editor
    - Finding the gap: unanswered and escalated questions, ranked, as the roadmap for the next iteration
    - The improvement loop as a weekly ritual: analytics, hypothesis, one change, evaluation run, remeasure
    - Instrumenting the capstone: the three numbers that will define success in front of your leadership
  Projects built:
    - Analytics sprint: run one agent with at least five real users for a week, review every

session, ship two data-demanded improvements, and write the before-and-after
Practice: Draft the monthly one-page agent report your leadership would actually read: outcomes, CSAT, cost savings, next actions.
Assessment: From a set of session transcripts, diagnose the top failure pattern, propose the fix, and prove it with an evaluation run.

Class 20: Cost Management: Copilot Credits Without Surprises
Topics:
- The billing currency since September 1, 2025: Copilot Credits, replacing messages, with pay-as-you-go at $0.01 per credit and prepaid packs at $200 per 25,000 credits per month, billed annually, unused credits not rolling over
- The per-feature rate card you estimate with: classic answer 1 credit, generative answer 2, agent action 5, tenant graph grounding 10, agent flow actions 13 per 100, AI tools at 1, 15 or 100 per 10 responses by tier
- What the Microsoft 365 Copilot licence (at $30 per user per month) covers at no extra charge for licensed users, and the pointed exception: computer use is not included
- The 125% rule: at 125% of prepaid capacity, custom agents are disabled and users see the unavailable message, so capacity planning is an availability feature
- GitHub Copilot harness economics: credits consumed from the moment you start building, including preview, testing and evaluation generation
- Modelling a real workload: your capstone's expected sessions, features used, credits per session, and monthly cost under pack versus pay-as-you-go
- Cost as a design input: grounding choices, tool counts and orchestration mode all move the bill, and you now know by how much

Projects built:
- Build the complete cost model for your capstone: per-session credit estimate by feature, monthly projection at three adoption levels, pack versus pay-as-you-go recommendation, and the 125% risk line

Practice: Re-estimate the cost model twice: once with tenant graph grounding on, once off, and write down what the difference buys.
Assessment: Phase 4 exam: given a described workload, produce the credit estimate, the licensing recommendation and the capacity alert plan in 30 minutes.

PHASE 5: GOVERNANCE, ROLLOUT & CAPSTONE

Master the governance layer that makes agents deployable: Entra Agent ID, Conditional Access, DLP, Microsoft Agent 365 and the Agent Store, then ship a production capstone with the governance and ROI pack to defend it.

Class 21: Identity and Policy: Entra Agent ID, Conditional Access, DLP
Topics:
- The July 2026 line in the sand: all new agents must have Microsoft Entra Agent IDs, with no opt-out, making every agent a first-class identity in your tenant
- What an agent identity buys: connector permissions surfacing as API permissions that Conditional Access policies can target
- DLP policies over the whole tool surface: connectors, custom connectors and MCP governed by the same data policies, reviewed in the Power Platform admin center
- The two admin centers in the loop: Power Platform admin center and Microsoft 365 admin center, and which decisions live where (including enabling Anthropic and other external models)
- Environment strategy as governance: who can make agents where, and why production environments have fewer makers than pilots
- Guest users cannot access Copilot Studio, and other tenant facts that shape rollout plans
- Drafting your capstone's identity and policy plan: the Agent ID, the permissions it needs, and the Conditional Access posture it will live under

Projects built:
- Write the identity and policy plan for your capstone: Entra Agent ID registration, the permission list with justifications, the DLP review outcome, and the Conditional Access recommendation

Practice: Map your organisation's (or a model organisation's) admin roles to the approvals your capstone needs, with named decision points.
Assessment: Role-play the security review: your mentor plays the identity admin, and your plan must answer every 'who approved this' question.

Class 22: The Agent Estate: Microsoft Agent 365, the Agent Store, and Rollout
Topics:
- Microsoft Agent 365, GA May 1, 2026 at $15 per user per month: the control plane for the agent estate, with registry, access control and observability, already holding nearly 40 million registered agents by Microsoft's July 2026 count
- The Agent Store path: publishing a Copilot Studio agent to Microsoft 365 Copilot via Channels, then Microsoft 365 Copilot and Microsoft Teams, followed by admin approval in the Microsoft 365 admin center
- Channel strategy for business agents: Teams and Microsoft 365 Copilot, SharePoint, custom websites, and WhatsApp for customer-facing cases, chosen by audience, not fashion
- Versioning and change control on live channels: what re-publishing replaces and how not to break your users mid-shift
- The rollout playbook: pilot cohort, feedback loop, evaluation gates, comms, training, and the rollback plan nobody hopes to use

- The adoption evidence: IDC's projection of 1.3 billion agents by 2028 (as cited by Microsoft) and nearly 90% of the Fortune 500 with active agents on Microsoft's low-code tools (Microsoft FY26 Q3 earnings), used correctly as context, not hype
- Pre-launch gate: the 20-point production checklist covering grounding, evaluations, identity, DLP, cost and rollback

Projects built:
- Produce the rollout pack for your capstone: channel plan, pilot design, approval path through the Agent Store or internal publishing, comms draft and rollback runbook

Practice: Run the 20-point production checklist against your capstone as it stands today and file every failure into Class 23's build plan.

Assessment: Present the rollout pack in ten minutes as if to your real leadership, and survive the 'what if it goes wrong' round.

Class 23: Capstone Build Week

Topics:
- Freezing the spec: the capstone's job, users, sources, tools, triggers, channel, identity plan and cost model on one page
- Building in production order: knowledge first, topics second, tools third, triggers last, evaluations running throughout
- Timeboxing with a mentor: what gets cut when reality bites, and why the evaluated core loop is protected
- Beating your baseline: the Class 18 evaluation suite as the bar, rerun at every milestone
- Governance artifacts finalised: identity plan, DLP note, cost model and operations note, versioned alongside the agent
- Polish that earns trust: greeting, transparency statement, escalation tone, card design, error messages
- Dress rehearsal: the full demo with one planned failure and recovery, timed

Projects built:
- Build the complete capstone: a production-grade agent for your own team with grounded knowledge, designed topics, at least one tool or agent flow, one governed trigger if autonomous, a passing evaluation suite, and the full governance pack

Practice: Run the dress rehearsal for a colleague outside the course and fix the two weakest moments they point at.

Assessment: Milestone gate: the capstone beats its evaluation baseline and passes the 20-point production checklist before demo day.

Class 24: Capstone Defence and the Credential Road

Topics:
- The live defence: your capstone demonstrated on a real channel, with evaluations, analytics and the governance pack on the table
- Defending decisions like an architect: why these sources, this orchestration mode, these tools, this identity posture, this cost model
- The retrospective: what you would build differently, and the honest limits of the current platform for your use case
- The credential road mapped: Microsoft Certified: AI Agent Builder Associate (exam AB-620, covering planning and configuring agent solutions, integrating and extending agents, and testing and managing them), with the Applied Skills labs 'Build an agent in Microsoft Copilot Studio' and 'Enhance agents with autonomous capabilities' as nearer milestones, and PL-900 as the on-ramp for teammates you onboard
- Where the skill curve goes: Gartner's projection that by 2029 at least half of knowledge workers will need to work with, govern or create AI agents, and by 2028 at least 15% of day-to-day work decisions will be made autonomously
- Taking it internal: turning your capstone pack into your organisation's agent playbook, and positioning yourself as the person who owns it
- Certificate, feedback, and the Living Syllabus promise: what changed in Copilot Studio while you were learning it, reviewed together

Projects built:
- Deliver the capstone defence and hand over the complete packet: spec, source map, evaluation report, analytics plan, cost model, identity and DLP plan, rollout runbook, and the leadership deck

Practice: Book the next step while momentum is real: an AB-620 study plan, or the internal pitch meeting for your agent playbook.

Assessment: Final evaluation: capstone defence, governance pack review, and the complete documentation packet, graded against the Class 22 production checklist.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: an agent opportunity memo for your organisation, Policy Desk v1 with a tested instruction stack, a 25-question adversarial test set, and a defended model-and-harness decision
- Phase 2: a SharePoint-grounded Policy Desk with citation proofs, a retrieval-optimised document rebuild with before-and-after accuracy, a multi-source arbitration study, and a published 25-answer accuracy audit
- Phase 3: Support Triage with designed topics and adaptive cards, a generative-orchestration migration log, the Expense Helper approvals flow with tested error paths, live connector and REST integrations, a governed MCP integration with a security note, and a costed computer-use demo
- Phase 4: the autonomous Weekly Reporter, its hardened successor with approval pauses and an operations note, a two-agent A2A architecture with hand-off contracts, a 30-conversation

evaluation suite with graders, an analytics sprint with shipped improvements, and a complete Copilot Credits cost model
- Phase 5: an identity and policy plan (Entra Agent ID, DLP, Conditional Access), a full rollout pack with rollback runbook, the production capstone build, and the final defence with the leadership deck
- Total: 24 professional deliverables, one per class, finishing with a production capstone agent plus a governance and ROI pack

Total projects: 24 professional deliverables, one per class, finishing with a production capstone agent plus a governance and ROI pack

SKILLS THE STUDENT WILL HAVE AT THE END

- Agent design for business: instruction stacks, scope fences, refusals and escalation paths
- Enterprise knowledge grounding: SharePoint, Dataverse, documents, connector-indexed data and tenant graph grounding, within real platform limits
- Conversation design: topics, trigger phrases, entities, condition groups and adaptive cards for critical paths
- Generative orchestration, chosen deliberately and guarded with retested fences
- The full tool surface: Prompt tools, agent flows, standard, premium, on-premises and custom connectors, REST, and Model Context Protocol under DLP
- Computer use with enterprise guardrails: access control, HTTPS enforcement, stored credentials and human supervision, costed per step
- Autonomous agents: event triggers, the maker-credentials rule, approval pauses, idempotence and operations notes
- Multi-agent architecture: child agents, connected agents, the A2A protocol and Microsoft Foundry interop
- Quality engineering: agent evaluations with test sets and graders, analytics outcomes, CSAT and the cost savings metric
- Cost management: Copilot Credits rates, licence coverage, the 125% cutoff and workload cost modelling
- Governance: Microsoft Entra Agent ID, Conditional Access, DLP policies, Microsoft Agent 365 and the Agent Store approval path

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 private 1-on-1 classes per week, about 1 hour each, across 12 weeks
Hands on agent work: 1.5-2 hours building, testing and evaluating in Copilot Studio between classes
Project time: 1-1.5 hours completing each class deliverable, usually against your own organisation's use case
Review and doubts: 30-60 minutes reviewing feedback and clearing doubts over WhatsApp before the next class
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Dedicated mentor: The same mentor for all 24 classes, who knows your organisation's context, your capstone and your governance constraints
Whatsapp doubts: Doubt support between classes over WhatsApp
Practice environment: Help arranging your Copilot Studio access in Class 1: employer tenant, 30-day trial where appropriate, or a practice environment we arrange
Session recordings: Class recordings available for revision, so a missed detail is never lost

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- AI agent builder or automation specialist roles, the titles now appearing in real job listings as organisations move from pilots to estates
- Microsoft Certified: AI Agent Builder Associate (exam AB-620), for which this syllabus is direct preparation, with the Applied Skills labs 'Build an agent in Microsoft Copilot Studio' and 'Enhance agents with autonomous capabilities' as nearer milestones
- The internal agent champion or center-of-excellence role: the person who owns the organisation's agent playbook, governance posture and cost model
- Consulting and freelance agent delivery, with a portfolio capstone that includes the governance and ROI artifacts clients' security teams actually ask for
- A bridge to code-first agent engineering: Microsoft Foundry interop from this course, and our Codex + Claude Code masterclass when you want to build agents in code

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need to be a developer to take this Copilot Studio masterclass?
A: No. Copilot Studio is a graphical, low-code studio, and the masterclass is built for professionals who own processes, not necessarily code. The hard parts it teaches are design, evaluation, governance and cost discipline, which are not programming skills.

Q: How is this different from watching Copilot Studio tutorials or Microsoft Learn modules?
A: Tutorials teach the clicks; this course builds and defends a production agent for your own team. Microsoft Learn is excellent and free, and we point you at it; what it cannot do is review your specific agent's governance pack line by line, which is what the 1-on-1 mentor does.

Q: Why is this masterclass taught 1-on-1 only?
A: Because the material is your organisation's material. Honest sessions discuss your processes, your data boundaries, your tenant's admin posture and your licensing position, none of which belongs in a group class. The mentor reviews your actual instruction stacks, source maps, evaluation results and governance artifacts every session, and adjusts depth to your role, whether you are an analyst, an admin or an architect.

Q: What do I need to get access to Copilot Studio for the course?
A: A Microsoft work or school (Entra ID) account. With a work email you can self-serve the 30-day trial, which is enough to build and test but cannot publish agents. The better path is your employer's tenant with your admin's awareness, and the course includes drafting exactly that request. Where neither is possible, we arrange a practice environment, and your mentor sorts access out with you in the first class.

Q: Which Microsoft certification does this course prepare me for?
A: The headline target is Microsoft Certified: AI Agent Builder Associate (exam AB-620), whose domains, planning and configuring agent solutions, integrating and extending agents in Copilot Studio, and testing and managing agents, map directly onto this syllabus, including MCP and the A2A protocol named in its prerequisites. On the way, the Applied Skills labs 'Build an agent in Microsoft Copilot Studio' and 'Enhance agents with autonomous capabilities' are natural milestones, and PL-900 is the on-ramp we recommend for teammates you later onboard.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final project defended live, plus a written assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: How current is the syllabus, given how fast the platform is changing?
A: It is a Living Syllabus, re-verified against Microsoft's official documentation and What's New pages as features ship. The current version covers the August 2026 platform: the multi-model lineup including Anthropic Claude Opus 4.7, harnesses, Model Context Protocol under DLP, computer use with enterprise guardrails, the A2A protocol, agent evaluations, mandatory Entra Agent IDs and Microsoft Agent 365. When the platform changes, the course changes, and your mentor tells you what moved while you were learning.

Q: What do I walk away with at the end of the masterclass?
A: A production-grade capstone agent for your own team, demonstrated and defended on a real channel, plus the full professional packet: source map, evaluation suite and baseline report, analytics plan, Copilot Credits cost model, Entra Agent ID and DLP plan, rollout runbook and a leadership deck. You also leave with a Modern Age Coders certificate earned by that defence, and a mapped credential road to AB-620 if you want Microsoft's official stamp.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ai-agents-with-microsoft-copilot-studio-masterclass-for-professionals
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 47 OF 137: CREATIVE CODING AND ANIMATION: VIDEO, MOTION AND P5.JS
==========

Course link: https://learn.modernagecoders.com/courses/creative-coding-animation-mastery
Subject area: Creative Technology
Designed for: Kids
Who can join: Beginner friendly, ages 13 to adult
Course length: 5-6 months (20-24 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Creative coding and animation course: live online classes for video editing, motion graphics, p5.js generative art, Blender 3D and content craft in 5-6 months.

In one line: Where art meets code: real video editing, motion graphics, generative art with p5.js and Blender 3D, finished as a portfolio that moves.

ABOUT THE PROGRAMME

This course sits at the intersection most creative courses miss: editing and motion design craft on one side, actual creative coding on the other. Students learn professional video editing, motion graphics, animation principles, then cross into p5.js and Processing, where art is written in code and no two runs look the same, and finish with Blender 3D and the content craft that makes work travel online. The toolkit is free-first and honest: DaVinci Resolve, CapCut, Blender, p5.js and Canva cover the entire syllabus at zero software cost, with Adobe After Effects taught as the industry-standard option for students who have access to it. Every week produces finished work, and the course ends with a portfolio capstone and a final exam that decides the certificate. The pace is honest: five months at two live classes a week plus three to four hours of practice, with a sixth month in hand for students who need more time. Homework after every class, a mixed challenge at the end of each month, and a final exam with a free retest after revision. Real craft, real portfolio, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Creative coding is real here: p5.js and Processing get a full month, generative art most animation courses never touch
- Free-first toolkit: DaVinci Resolve, CapCut, Blender, p5.js and Canva cover the whole syllabus; After Effects taught as the industry option, never a paywall
- Craft over hacks: animation principles, typography and rhythm, the fundamentals that survive every platform trend
- Every week ends rendered: finished edits, loops, sketches and scenes, building one coherent portfolio
- An honest 5-month arc with a sixth month in hand: nobody is rushed
- Real assessment: weekly homework, monthly mixed challenges, and a final exam that decides the certificate, with a free retest after revision

THE LEARNING PATH, STEP BY STEP

Month 1: Foundations and editing (Weeks 1-4): design principles, animation basics, desktop and mobile video editing
Month 2: Motion graphics (Weeks 5-8): keyframes, easing, kinetic typography and compositing, in After Effects or free equivalents
Month 3: Creative coding (Weeks 9-12): Processing and p5.js, generative art, interactivity and code-driven animation
Month 4: 3D and short-form craft (Weeks 13-16): Blender fundamentals, 3D scenes and renders, and the short-form video craft that travels
Month 5: Channel, portfolio and the exam (Weeks 17-20): long-form storytelling, honest analytics, a portfolio that opens doors, and the final capstone

WHAT A STUDENT NEEDS BEFORE STARTING

Experience: None. Design, editing and code are all taught from zero
Equipment: A computer with a stable internet connection; a mid-range machine handles the whole course, and every required tool is free
Age: 13 to adult; batches are grouped by age and pace
Time commitment: 2 live classes plus 3-4 hours of making a week

WHO THIS COURSE IS FOR

Aspiring animators: Teens and adults who dream of creating animations and motion graphics
Content creators: Aspiring YouTubers, Instagram creators, and TikTok creators
Design enthusiasts: Anyone passionate about visual design and digital art
Career switchers: Professionals wanting to switch to creative technology roles
Freelancers: People who want to offer video editing, animation, or design services
Creative coders: Developers who want to explore the intersection of code and art

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

MONTH 1: CREATIVE FOUNDATIONS & VIDEO EDITING BASICS

Master design principles, animation fundamentals, and professional video editing.

Design Principles & Digital Art Basics

Topics:
- Elements of design: line, shape, color, texture, space
- Principles of design: balance, contrast, emphasis, movement
- Color theory: RGB/CMYK, color harmony, psychology
- Typography fundamentals and font pairing
- Composition and rule of thirds
- Visual hierarchy and flow
- Mood boards and style guides
- Design trends and aesthetics
- Digital canvas setup and resolution
- File formats: when to use what
- Asset organization and management
- Copyright and creative commons

Projects built:
- Personal Brand Identity: Logo, color palette, style guide
- Mood Board Collection: 5 different aesthetic mood boards
- Social Media Template Set: Consistent design templates

Practice: Pick a real poster or flyer you like, rebuild it with a stronger visual hierarchy and a deliberate color harmony, and note which design principle drove each change.

Animation Principles & 2D Basics

Topics:
- 12 principles of animation (Disney)
- Squash and stretch techniques
- Anticipation and follow-through
- Timing and spacing fundamentals
- Ease in and ease out
- Arcs and natural motion
- Secondary animation
- Frame rates and formats
- Storyboarding and animatics
- Character design basics
- Walk cycles and run cycles
- Lip sync fundamentals

Projects built:
- Bouncing Ball Animation: Master timing and physics
- Character Walk Cycle: Create smooth character movement
- Animated Logo Reveal: Bring brand to life

Practice: Animate a simple bouncing ball that shows squash and stretch plus ease in and ease out, and export a five second clip.

Video Editing Fundamentals

Topics:
- Non-linear editing concepts
- Timeline and tracks understanding
- Cutting and trimming techniques
- Transitions: cuts, fades, wipes
- Audio editing and synchronization
- Color correction basics
- Text and titles creation
- B-roll and coverage
- Pacing and rhythm in editing
- Continuity editing rules
- Jump cuts and match cuts
- Montage theory and practice

Projects built:
- Day in Life Vlog: Edit personal vlog footage
- Product Commercial: 30-second product showcase
- Music Video Edit: Sync video to music

Practice: Cut a 60 second highlight video from any raw footage using at least three transition types and synced background audio.

Mobile Video Editing & Quick Content

Topics:
- Mobile editing workflow
- Vertical video best practices
- Quick edits for social media
- Filters and effects
- Mobile color grading
- Text animations on mobile
- Stickers and overlays
- Green screen on mobile
- Mobile audio editing
- Aspect ratio management
- Export settings for platforms
- Cloud storage workflow

Projects built:
- Instagram Reels Series: 5 trending reels

- YouTube Shorts Collection: Engaging short-form content
Practice: Shoot and edit a 30 second vertical reel on your phone with captions, one filter and a mobile color grade.
Assessment: Month 1 check: a timed edit from provided footage, plus a mixed quiz on design and animation principles

---

MONTH 2: MOTION GRAPHICS & AFTER EFFECTS MASTERY

Master motion design, visual effects, and professional animation techniques.

After Effects Fundamentals
Topics:
- After Effects interface and workflow
- Compositions and layers
- Transform properties: position, scale, rotation
- Keyframe animation basics
- Graph editor and easing
- Parenting and null objects
- Masks and shape layers
- Text animation presets
- Effects and presets library
- Rendering and export settings
- Work areas and previewing
- Project organization
Projects built:
- Animated Title Sequence: Professional title animation
- Kinetic Typography: Text-driven animation
- Logo Animation Pack: Brand animations
Practice: Recreate a short channel intro in After Effects by keyframing position, scale and rotation on three layers, then smooth the motion in the graph editor.

Advanced Motion Graphics
Topics:
- Shape layer animation mastery
- Trim paths and stroke animation
- Morphing shapes and transitions
- 2.5D animation with cameras
- Parallax effects
- Expression basics
- Wiggle and random expressions
- Time remapping
- Motion blur and echo effects
- Particle systems basics
- Audio-driven animation
- Data-driven graphics
Projects built:
- Explainer Video: 2-minute animated explainer
- Music Visualizer: Audio-reactive graphics
- Social Media Pack: Animated social elements
Practice: Build a ten second logo reveal using trim paths and one morphing shape transition.

Character Animation & Rigging
Topics:
- Character rigging with Duik
- Puppet pin tool mastery
- Bone and IK systems
- Walk cycle in After Effects
- Facial animation techniques
- Lip sync animation
- Character emotions and acting
- Rubber hose animation
- Limber and RubberHose plugins
- Character interaction
- Scene composition
- Character design for animation
Projects built:
- Character Explainer: Character-driven narrative
- Animated Mascot: Brand character creation
- Motion Comic: Comic panels animated
Practice: Rig a simple two legged character with Duik and animate a basic four step walk cycle.

Visual Effects & Compositing
Topics:
- Green screen keying techniques
- Rotoscoping fundamentals
- Motion tracking basics

- Camera tracking introduction
- Color correction and grading
- Light and shadow effects
- Particle effects creation
- Fire, smoke, and water effects
- Glitch and distortion effects
- 3D integration basics
- Matte painting basics
- Compositing workflows

Projects built:
- VFX Short Film: Effects-heavy sequence
- Product VFX Commercial: Product with effects
- Music Video Effects: Visual effects for music

Practice: Key out a green screen clip, place the subject on a new background and match the color and shadows so the composite holds up.

Assessment: Month 2 check: a 10-second motion piece built live, with questions reaching back to month 1

MONTH 3: CREATIVE CODING & INTERACTIVE MEDIA

Master creative programming with Processing, p5.js, and interactive installations.

Processing Fundamentals

Topics:
- Introduction to creative coding
- Processing environment setup
- Coordinate system and drawing
- Shapes and colors in code
- Variables and data types
- Loops and iteration
- Conditional statements
- Functions and modularity
- Arrays and collections
- Object-oriented programming basics
- Animation with code
- Interactivity with mouse and keyboard

Projects built:
- Generative Art System: Algorithmic art creation
- Interactive Visualization: Data-driven graphics
- Particle System: Dynamic particle effects

Practice: Code a Processing sketch that draws a repeating pattern using loops, variables and at least three colors.

p5.js and Web-Based Creative Coding

Topics:
- p5.js setup and differences from Processing
- JavaScript fundamentals for creative coding
- DOM manipulation with p5.js
- Loading and displaying media
- Sound library integration
- WebGL and 3D graphics
- Mobile interaction and touch events
- API integration and data fetching
- Local storage and cookies
- Hosting and sharing web sketches
- p5.js libraries and extensions
- Performance optimization

Projects built:
- Interactive Website: Creative coding in web design
- Audio Visualizer Web App: Music-reactive visuals
- Browser Game: Playable web game

Practice: Port one of your Processing sketches to p5.js and add mouse interaction plus one sound trigger.

Interactive Installations & Physical Computing

Topics:
- Arduino and Processing communication
- Serial communication basics
- Sensor data visualization
- Projection mapping introduction
- Kinect and depth sensing
- Computer vision with OpenCV
- Touch surfaces and multitouch
- MIDI controller integration
- OSC protocol for networking
- Real-time data streams
- Installation planning and setup
- Documentation and presentation

Projects built:
- Interactive Wall: Projection mapping installation
- Motion-Controlled Art: Body tracking visuals
- Data Sculpture: Physical data visualization

Practice: Plan a small sensor driven installation on paper and prototype its visual response in Processing using simulated input values.

Creative Coding Applications

Topics:
- Generative design principles
- Procedural content generation
- Machine learning in creative coding
- Neural networks and art
- Style transfer techniques
- Evolutionary algorithms
- Swarm intelligence
- Biomimicry in code
- Shader programming basics
- GLSL introduction
- Live coding performances
- Creative coding for VJing

Projects built:
- AI Art Generator: ML-powered creativity
- Live Performance Tool: VJ/DJ visual system
- Generative Brand System: Dynamic identity

Practice: Generate three variations of one generative art piece by changing the rules and the randomness, then pick the strongest for your portfolio.

Assessment: Month 3 check: a small generative sketch written live, plus mixed review of earlier months

---

MONTH 4: BLENDER 3D AND SHORT-FORM CRAFT

Three weeks of honest Blender fundamentals, then the short-form editing craft that makes work travel.

3D Animation Fundamentals with Blender

Topics:
- Blender interface and navigation
- 3D viewport and object manipulation
- Modeling basics: primitives and modifiers
- Edit mode: vertices, edges, faces
- Sculpting introduction
- Materials and textures basics
- UV unwrapping fundamentals
- Lighting: three-point setup
- Camera animation and framing
- Keyframe animation in 3D
- Timeline and dope sheet
- Rendering with Cycles and Eevee

Projects built:
- 3D Logo Animation: Brand in 3D space
- Product Visualization: Photorealistic product render
- Abstract Animation: Motion graphics in 3D

Practice: Model a simple low poly object in Blender using primitives and modifiers, then apply basic materials and render one image.

Character Animation & Rigging in 3D

Topics:
- Character modeling workflow
- Retopology for animation
- Armature and bone creation
- IK (Inverse Kinematics) setup
- Weight painting
- Shape keys and blend shapes
- Facial rigging basics
- Animation principles in 3D
- Walk cycles in 3D
- Motion capture basics
- Non-linear animation editor
- Physics simulation for cloth and hair

Projects built:
- Animated Character: Full character pipeline
- Creature Animation: Non-human character
- Crowd Simulation: Multiple character animation

Practice: Add an armature to a basic character mesh, weight paint two joints and pose the character in three distinct positions.

Advanced VFX & Compositing

Topics:
- Advanced keying techniques
- Roto and paint workflows
- 3D tracking and matchmoving
- CG integration with live footage
- Advanced color grading with nodes
- Film emulation and looks
- Atmospheric effects creation
- Destruction and dynamics
- Fluid simulations basics
- Smoke and fire effects
- Advanced compositing nodes
- Multi-pass compositing

Projects built:
- VFX Breakdown Reel: Professional VFX shots
- Invisible Effects: Seamless integration
- Sci-Fi Sequence: Futuristic effects

Practice: Track a short live action clip and composite one graphic or CG element so it sticks to the scene convincingly.

Short-Form Video Craft

Topics:
- The grammar of reels and shorts: hooks, pacing, payoff
- Editing rhythm: cuts on beats, breath and surprise
- Captions and text animation that people actually read
- Sound design for small screens
- Covers, thumbnails and the three-second test
- Repurposing one piece across formats honestly
- Trends as ingredients, craft as the recipe
- A week of daily publishing practice, reviewed

Projects built:
- Three finished short-form pieces from one shoot or one render
- A motion-graphics short built for vertical

Practice: Edit one short piece daily; review the best two in class

Assessment: Month 4 check: a timed edit challenge plus a mixed quiz over months 1-4

MONTH 5: CHANNEL, PORTFOLIO AND THE FINALE

Long-form storytelling, honest analytics, a portfolio that opens doors, and the final capstone exam.

Long-Form Video and Storytelling

Topics:
- YouTube algorithm and discovery
- Channel setup and optimization
- Thumbnail design psychology
- Title and description SEO
- YouTube Shorts strategy
- Long-form vs short-form balance
- Playlist optimization
- End screens and cards
- YouTube Analytics mastery
- Community tab usage
- Premiere and live streaming
- Story structure for longer pieces: promise, journey, payoff

Projects built:
- YouTube Channel Launch: Complete channel setup
- Tutorial Series: Educational content
- Vlog Documentary: Long-form storytelling

Practice: Design two alternative thumbnails and titles for the same video idea and explain which pairing you would A/B test first.

Publishing Strategy and Honest Analytics

Topics:
- Content repurposing strategies
- Platform-specific adaptations
- Cross-platform promotion
- Content scheduling tools
- Analytics interpretation
- A/B testing methods
- ROI measurement
- Audience insights and personas
- Competitive analysis
- Trend forecasting
- Content performance metrics
- Growth hacking techniques
- Reading analytics without fooling yourself: retention over vanity numbers

Projects built:

- Omnichannel Campaign: Multi-platform launch
- Content Repurposing System: Efficient content creation
- Analytics Dashboard: Performance tracking

Practice: Repurpose one long form video into a short, a carousel and a text post, each genuinely adapted to its platform.

Portfolio Development & Personal Branding

Topics:

- Portfolio website creation
- Case study development
- Project presentation skills
- Personal brand strategy
- Logo and identity design
- Business card and stationery
- Email signature design
- LinkedIn optimization
- Behance portfolio setup
- Dribbble presence
- ArtStation for 3D work
- Showreel creation

Projects built:

- Portfolio Website: Professional online presence
- Demo Reel: Best work compilation
- Brand Identity Package: Complete personal brand

Practice: Draft your portfolio homepage with three case studies, each telling the problem, process and result in one paragraph.

Capstone, Demo Day and Final Exam

Topics:

- The capstone: one polished portfolio piece that uses editing, motion and code or 3D
- A working-brief simulation: brief, drafts, revisions, delivery, the professional loop
- Final portfolio assembly and presentation order
- Presenting creative work: intent, references, decisions
- Full-course mixed review before the exam

Projects built:

- The capstone piece, rendered and presented at demo day with the full portfolio

Practice: Daily capstone progress with an edit log

Assessment: FINAL EXAM: capstone presentation plus a practical challenge and written questions mixed from the whole course. Passing earns the certificate; falling short earns focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Month 1 (Weeks 1-4): Digital art, 2D animation, video editing fundamentals
- Month 2 (Weeks 5-8): Motion graphics, After Effects, character animation, VFX
- Month 3 (Weeks 9-12): Creative coding with Processing & p5.js, interactive media
- Month 4 (Weeks 13-16): 3D animation with Blender, advanced VFX, motion capture
- Month 5 (Weeks 17-20): Social media content mastery - Instagram, TikTok, YouTube
- Month 6 (Weeks 21-26): Professional portfolio, client projects, capstone

Total projects: 50+ creative projects + professional showreel portfolio

SKILLS THE STUDENT WILL HAVE AT THE END

- Digital Art: design principles, color theory, composition, digital painting
- Video Editing: DaVinci Resolve, CapCut, mobile editing, color grading
- Motion Graphics: After Effects, keyframing, expressions, kinetic typography
- 3D Animation: Blender, modeling, rigging, character animation, VFX
- Creative Coding: Processing, p5.js, generative art, interactive installations
- Social Media: Instagram Reels, TikTok, YouTube - platform-specific strategies
- Professional: portfolio development, client management, personal branding
- Hardware: Arduino basics, physical computing, interactive installations

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, making along with the instructor
Homework: A finished piece after every class, and a mixed challenge at the end of every month
Practice: 3-4 hours of making between classes
Review: Every submission critiqued with written feedback; craft gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for stuck renders and broken timelines
Progress updates: Monthly written feedback on craft: what is working, what to push next

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam, a capstone presentation plus practical and written challenges. Fall short and there is focused revision

and a free retest

WHERE THIS CAN LEAD

- Motion Graphics Designer
- Content Creator / Influencer
- Video Editor
- Creative Technologist
- Animation Artist
- Social Media Manager
- VFX Artist
- Creative Director
- Freelance Designer

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What skills does this creative coding and animation course cover?
A: The 6-month program moves through video editing, motion graphics in After Effects, creative coding with Processing and p5.js, 3D animation in Blender, and content strategy for Instagram, TikTok, and YouTube. Students finish with a capstone project and a professional portfolio of animations, edits, and interactive art.

Q: Who is this course suitable for?
A: It is designed for ages 14 and up, so it works for teens as well as adults exploring a creative career or a serious side skill. We teach students from ages 6 to 65 across coding and maths, and this course sits at the older, creator-focused end of that range.

Q: How long does it take to learn video editing and animation?
A: To a genuine portfolio level across editing, motion graphics, creative coding and 3D basics: 5 months at 2 live classes a week plus 3 to 4 hours of practice. The course keeps a sixth month in hand for students who need more time, with no rushing and no penalty. First finished edits appear in week 3; the compounding starts immediately.

Q: How is progress assessed? Is the certificate just for attending?
A: No. Every class ends with a piece to finish, every month ends with a mixed challenge that revisits earlier skills, and the course ends with a final exam: a capstone presentation plus practical and written challenges drawn from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Do students need design or coding experience first?
A: No. The course starts with design and animation principles from zero, and the creative coding modules introduce programming gently through visual output, so beginners see results from the first weeks.

Q: Which software is used, and do we have to buy anything?
A: No purchases are required: DaVinci Resolve (free version), CapCut, Blender, Processing, p5.js and free Canva cover the entire syllabus. After Effects, the motion-graphics industry standard, is taught as an option for students who already have Adobe access, and every concept it demonstrates is also shown in the free tools. A mid-range computer handles the whole course.

Q: Where can these skills lead after the course?
A: Graduates leave with a portfolio spanning editing, motion graphics, and creative code, a strong foundation for freelance work, a content channel, or deeper study in design and development. Many students pair this course with our web development track to add interactive websites to their creative range.

Q: Is there a demo class before committing?
A: Yes, and it is completely free with no obligation to continue.

Q: What is the class format?
A: Live 1-hour online classes with a real instructor, never recorded lectures. Sessions are interactive and kept to small groups, so every student presents work, receives direct feedback, and asks questions freely.

Q: Do students receive certification?
A: Yes, and it is earned: the certificate is awarded on passing the final exam, a capstone presentation plus practical and written challenges, with a free retest after revision if needed. The portfolio matters more though: finished edits, motion pieces, generative art and 3D renders that show exactly what you can make.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/creative-coding-animation-mastery
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 48 OF 137: CREATIVE MEDIA FOR TEENS: PHOTO, VIDEO AND DESIGN
==========

Course link: https://learn.modernagecoders.com/courses/creative-media-design-masterclass-teens
Subject area: Design and Creativity
Designed for: Teens
Who can join: Beginner friendly, ages 13 to 18
Course length: 8-10 months (32-40 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Creative media course for teens aged 13-18: photo editing, video production and graphic design in live online classes, with a real portfolio in 8-10 months.

  In one line: Photo, video and design taught as one craft: eight honest months from first edit to a portfolio that looks like taste, not templates.

ABOUT THE PROGRAMME

  Teens live inside visual media; very few can make it well. This live online course for ages 13 to 18 teaches the full creative trio properly: photo editing from honest corrections to real compositing, video from cuts and rhythm to short productions with sound design, and graphic design from principles to a small brand built end to end. The final months add advanced video effects, real photography craft with whatever camera the family owns, and a portfolio capstone presented at demo day. The toolkit is free-first and honest: strong free tools cover the entire syllabus, and industry options like Photoshop are taught as exactly that, options, for families that have them. There is no viral-chasing, no monetization coaching and no freelance pressure aimed at teenagers; the course builds craft and taste, the assets that actually compound. The pace is real: eight months at two live classes a week plus two to three hours of practice, with two more months in hand when needed. Homework after every class, monthly mixed challenges, and a final exam, portfolio review plus a live editing task, that decides the certificate with a free retest. Real craft, real portfolio, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

  - The full trio in one arc: photo, video and design, taught as one connected craft
  - Free-first tools with honest options: the whole syllabus works at zero software cost
  - Critique every week: kind, specific feedback is the actual engine of taste
  - No viral-chasing or freelance pressure for teens: craft first, always
  - An honest 8-month arc with 2 months in hand
  - Real assessment: monthly mixed challenges and a final exam that decides the certificate, with a free retest

THE LEARNING PATH, STEP BY STEP

  Phase 1: The creative core (Months 1-6): photo editing, video production and graphic design with a small real brand
  Phase 2: Advanced craft and the finale (Months 7-8): visual effects, photography with real light, the portfolio capstone and the exam

WHAT A STUDENT NEEDS BEFORE STARTING

  Experience: None. Every tool and principle is taught from zero
  Equipment: A computer with a stable internet connection; a phone camera is enough for the photography weeks, and free tools cover the whole syllabus
  Age: 13 to 18. Batches are grouped by age and pace
  Time commitment: 2 live classes plus 2-3 hours of making a week

WHO THIS COURSE IS FOR

  Creators: Teens already posting who want their work to look intentional
  Visual minds: Students who think in images and want real craft under the instinct
  Club heroes: The teen every school event already asks for posters and aftermovies
  Future designers: Anyone testing the creative road with a portfolio as the answer
  Complete beginners: No experience needed: taste is built, not born

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE CREATIVE CORE (MONTHS 1-6, WEEKS 1-26)

Photo editing, video production and graphic design, each learned to real competence with
weekly critiqued builds.

  Months 1-2: Creative Fundamentals & Photo Editing Basics (Weeks 1-8)
    Introduction to Digital Creativity & Setup
      Topics:
        - The creative industry landscape for Gen Z
        - Career paths: YouTuber, Designer, Photographer, Filmmaker
        - Essential hardware: Computer specs, tablets, phones
        - Free vs paid software options
        - Setting up Adobe Creative Cloud (student discount)
        - Alternative free tools: GIMP, DaVinci Resolve, Canva
        - File management and organization
        - Understanding file formats: JPG, PNG, RAW, MP4, MOV
        - Resolution, DPI, and pixel basics
        - Color theory fundamentals: RGB vs CMYK
        - Creating your first creative workspace
        - Building a creative routine and staying inspired
      Projects built:
        - Set up complete creative workstation
        - Create organized folder structure for projects
        - Design a personal creative brand/logo
      Practice: Daily: 30 min creative exploration, 1 hour tool familiarization
    Photoshop Fundamentals
      Topics:
        - Photoshop interface tour and customization
        - Essential panels: Layers, Properties, History
        - Working with layers: Creating, organizing, blending modes
        - Selection tools: Marquee, Lasso, Magic Wand, Quick Selection
        - Basic adjustments: Brightness, Contrast, Levels, Curves
        - Color correction: Hue/Saturation, Color Balance
        - Cropping and straightening images
        - Removing objects: Spot Healing, Clone Stamp, Content-Aware Fill
        - Working with text: Fonts, effects, formatting
        - Saving for different purposes: Web, print, social media
        - Non-destructive editing with adjustment layers
        - Keyboard shortcuts for speed
      Projects built:
        - Edit 10 personal photos to perfection
        - Design custom phone wallpapers
      Practice: Edit 5 different photos daily using various techniques
    Advanced Photo Manipulation
      Topics:
        - Advanced selections: Select and Mask, Color Range
        - Compositing: Combining multiple images seamlessly
        - Sky replacement and background changes
        - Portrait retouching: Skin, eyes, teeth, hair
        - Frequency separation for professional retouching
        - Creating realistic shadows and reflections
        - Photo filters and effects: Vintage, cinematic, artistic
        - Double exposure and creative effects
        - Creating social media templates
        - Batch processing multiple photos
        - Actions and automation for efficiency
        - RAW processing basics with Camera Raw
      Projects built:
        - Create surreal photo manipulation artwork
        - Professional portrait retouching project
        - Design Instagram post templates
      Practice: Complete 20 photo manipulation challenges
    Mobile Photography & Editing
      Topics:
        - Smartphone photography techniques
        - Composition rules: Rule of thirds, leading lines, symmetry
        - Lighting for photos: Natural vs artificial
        - Portrait mode and depth effects
        - Mobile editing apps: VSCO, Snapseed, Lightroom Mobile
        - Creating consistent Instagram feeds
        - Instagram Stories design and templates
        - TikTok photo slideshow trends
        - Pinterest-worthy photo editing
        - Mobile photography accessories
        - Shooting and editing vertical content
        - Quick edits for immediate posting

```
    Projects built:
      - 30-day Instagram feed makeover
      - Mobile photography portfolio
    Practice: Post one edited photo daily on social media
    Assessment: Months 1-2 check: a photo-edit set completed live, plus a mixed take-home
    challenge

Months 3-4: Video Editing & Production Basics (Weeks 9-17)
  Video Production Fundamentals
    Topics:
      - Video production workflow overview
      - Pre-production: Planning, scripting, storyboarding
      - Camera basics: Frames per second, resolution (1080p, 4K)
      - Shooting techniques: Angles, movements, stability
      - Smartphone videography tips and tricks
      - Audio importance: Recording clean sound
      - Basic lighting setups with household items
      - B-roll and why it matters
      - Understanding codecs and compression
      - Organizing footage and project files
      - Copyright and music licensing basics
      - Finding royalty-free music and sound effects
    Projects built:
      - Shoot and plan first short film (2-3 minutes)
      - Create a day-in-my-life vlog
      - Record product review video
    Practice: Shoot 10 different video types (slow-mo, time-lapse, etc.)
  Premiere Pro Essentials
    Topics:
      - Premiere Pro interface and workspace setup
      - Project settings and sequence creation
      - Importing and organizing media
      - Timeline basics: Tracks, clips, navigation
      - Essential editing tools: Selection, Razor, Ripple
      - Cutting techniques: J-cuts, L-cuts, match cuts
      - Transitions: Cuts, fades, wipes, custom transitions
      - Audio editing: Levels, fades, noise reduction
      - Color correction basics: Lumetri Color panel
      - Adding titles and lower thirds
      - Exporting for different platforms
      - Premiere Pro keyboard shortcuts
    Projects built:
      - Edit a cinematic travel video
      - Create YouTube video with intro/outro
      - Edit multi-camera interview
    Practice: Edit one video daily for different platforms
  YouTube Content Creation
    Topics:
      - Thumbnail design in Photoshop
      - Creating clickable titles without clickbait
      - YouTube video structure: Hook, content, CTA
      - Editing styles: Gaming, vlog, tutorial, reaction
      - Jump cuts and pacing for retention
      - Adding subtitles and captions
      - End screens and cards setup
      - YouTube Shorts vs long-form content
      - Analytics: Understanding retention graphs
      - Building a consistent upload schedule
      - Collaborating with other creators
    Projects built:
      - Complete YouTube channel setup and branding
      - Create and upload 5 different video styles
      - Design thumbnail templates
    Practice: Analyze 20 successful YouTube videos in your niche
  TikTok & Short-Form Content
    Topics:
      - Vertical video best practices
      - TikTok trends and how to spot them
      - CapCut advanced features
      - Speed ramping and velocity edits
      - Text animations and reveals
      - Transition mastery for TikTok
      - Green screen and effects
      - Instagram Reels optimization
      - YouTube Shorts specific editing
      - Sound sync and beat matching
      - Creating series and themed content
    Projects built:
      - Create 30 TikToks in different styles
```

- Develop signature editing style
- Build following to 1000+ followers

Practice: Post one short-form video daily for 30 days

Month 3-4 Review & Portfolio Building

Topics:

- Portfolio website creation basics
- Organizing and presenting work
- Creating demo reels
- Getting first clients as a teen
- Pricing your services

Projects built:

- CAPSTONE: Create 5-minute portfolio showreel
- Build online portfolio website

Assessment: Months 3-4 check: a one-minute edit cut live from provided footage, plus mixed review

Months 5-6: Graphic Design & Brand Creation (Weeks 18-26)

Illustrator & Vector Design

Topics:

- Vector vs raster graphics explained
- Illustrator interface and tools
- Pen tool mastery: Curves, angles, handles
- Shape tools and pathfinder operations
- Creating logos from scratch
- Typography and custom lettering
- Color palettes and swatches
- Gradient and pattern creation
- Icon design for apps and web
- Business card and stationery design
- Tracing and vectorizing images
- Exporting for different uses

Projects built:

- Design 5 different logo concepts
- Create complete brand identity package
- Design custom icon set

Practice: Create one vector illustration daily

Social Media Design & Branding

Topics:

- Understanding platform specifications
- Instagram post and story templates
- Facebook cover and post designs
- Twitter/X header and post graphics
- LinkedIn professional graphics
- Pinterest pin optimization
- Consistent brand aesthetics
- Creating content calendars
- Carousel posts and slideshows
- Infographic design principles
- Data visualization basics
- Canva for quick designs

Projects built:

- 30-day social media content package
- Brand guide for personal brand
- Infographic series on trending topic

Practice: Design social media posts for 5 different brands

Print Design & Merchandise

Topics:

- Print vs digital design differences
- Understanding bleeds and margins
- CMYK color mode and print preparation
- T-shirt and apparel design
- Sticker and label design
- Poster and flyer creation
- Album and book cover design
- Magazine layout basics
- Packaging design introduction
- Print-on-demand platforms
- Setting up online merchandise store
- Copyright and trademark basics

Projects built:

- Design merch line with 10 products
- Create concert poster series
- Design and sell first products online

Practice: Create designs for different print formats

Web & UI Design Basics

Topics:

- Web design principles for beginners
- User interface (UI) fundamentals
- User experience (UX) basics

- Wireframing and prototyping
- Figma introduction and basics
- Designing responsive layouts
- Mobile app interface design
- Website banner and hero images
- Button and form design
- Color psychology in web design
- Landing page creation
- Portfolio website design

Projects built:
- Design mobile app interface
- Create landing page design
- Redesign favorite website

Practice: Daily UI challenge for 30 days

Phase 1 Capstone Project

Topics:
- Project planning and time management
- Client communication basics
- Creating project proposals
- Feedback incorporation
- Final presentation skills

Projects built:
- MAJOR CAPSTONE: Complete Brand Identity Project
- Logo, social media kit, video intro, merchandise designs

Assessment: Phase check: present the mini brand kit, plus a mixed challenge spanning months 1-6

PHASE 2: ADVANCED CRAFT AND THE FINALE (MONTHS 7-8, WEEKS 27-32)

Visual effects and real photography craft, then the portfolio capstone and the final exam.

Months 7-8: Effects, Light and the Portfolio (Weeks 27-32)

Advanced Video: Effects and Sound

Topics:
- Transitions that mean something, not just move
- Keyframing: motion inside the frame
- Green screen done properly at home
- Color grading: the honest before-and-after
- Sound design: the half of video nobody sees
- Titles and captions that people actually read
- A polish pass on your best earlier edit

Projects built:
- A 60-90 second short with effects, grade and sound design

Practice: One micro-edit daily; review the best two in class

Photography: Real Light, Any Camera

Topics:
- Composition: the five framings that fix most photos
- Light: window, golden hour and one lamp, used deliberately
- Phone camera mastery: focus, exposure and RAW where available
- Editing your own shots: honest corrections, tasteful grades
- A themed photo series: shooting with intent
- Selecting: the discipline of choosing 6 from 200

Projects built:
- A themed 6-photo series, shot, edited and sequenced

Practice: A daily frame: one deliberate photograph every day

Portfolio Capstone

Topics:
- Choosing the capstone: one campaign across photo, video and design
- Curation over collection: the best 12 pieces, sequenced
- Presenting decisions: intent, references, choices
- Critique rounds: kind, specific, final

Projects built:
- THE CAPSTONE: a campaign plus the curated portfolio

Practice: One critique given and one received, daily

Demo Day and the Final Exam

Topics:
- Presenting the capstone and portfolio
- A live editing task under friendly time pressure
- A full-course mixed review
- Where this leads: creative coding, motion design, and making for your own channels

Projects built:
- Demo day presentation delivered to the batch

Assessment: FINAL EXAM: portfolio review plus a live editing task and mixed questions from the whole course. Passing earns the certificate; falling short earns focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Months 1-2: photo edit sets, restorations and honest retouching
- Months 3-4: cut sequences, a one-minute story and sound-fixed edits
- Months 5-6: design sets and a mini brand built end to end
- Months 7-8: an effects short, a themed photo series, and the capstone portfolio

Total projects: 60+ critiqued pieces across photo, video and design, curated into a portfolio

SKILLS THE STUDENT WILL HAVE AT THE END

- Photo editing: correction, retouching and compositing with taste
- Video: cutting rhythm, effects, grading and sound design
- Design: hierarchy, type and color through real briefs
- Photography: composition and light with any camera
- Critique: giving and taking feedback like a working creative

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, making along with the instructor
Homework: A piece after every class, and a mixed challenge at the end of every month
Practice: 2-3 hours of making between classes
Review: Every piece critiqued with written feedback; craft gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Monthly written feedback on craft: what is working, what to push next

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam, a portfolio review plus a live editing task. Fall short and there is focused revision and a free retest

WHERE THIS CAN LEAD

- Creative Coding and Animation: where this craft meets motion graphics and code
- Canva and AI Design: the design-systems road
- School and club media done at another level, immediately
- Personal channels with work that stands out for craft
- Later professional tools: the principles here transfer to every suite

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my teen learn and make in this course?
A: The full creative trio across eight months: photo editing from honest corrections to compositing, video from cutting rhythm to an effects-and-sound short, graphic design through a mini brand built end to end, then real photography craft with any camera, and a portfolio capstone, a campaign across all three skills, presented at demo day. Sixty-plus critiqued pieces in total.

Q: How long does it take to get genuinely good?
A: Eight months at 2 live classes a week plus 2 to 3 hours of practice, with two more months in hand for teens who want longer. Creative skill is rep-based: the weekly critiqued pieces are the actual course, and the difference shows within the first month.

Q: How is my teen assessed? Is the certificate just for attending?
A: No. A piece follows every class, every month ends with a mixed challenge that revisits earlier skills, and the course ends with a final exam: a portfolio review plus a live editing task. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Which software is used, and do we have to buy anything?
A: Nothing needs to be purchased: strong free tools cover the entire syllabus for photo, video and design. Industry options like Photoshop are taught as options for families that already have Adobe access; every technique is also shown in the free equivalents, because the craft is the point, not the subscription.

Q: Does this course teach growing on social media or earning as a creator?
A: No, deliberately: no viral formulas, no monetization coaching, no freelance pressure aimed at teenagers. Craft and taste are what compound, and they are the whole syllabus. A teen whose work is genuinely good will never struggle for an audience later; the reverse is not true.

Q: Does my teen need a good camera?
A: No: the photography weeks are built around whatever camera the family owns, usually a phone, because composition and light matter far more than gear. The editing and design work needs only a normal computer.

  Q: Is there a free trial class?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or
  message us on WhatsApp at +91 91233 66161. Your teen will edit something real inside the
  demo. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo
  is a guaranteed class today or tomorrow, booked at
  https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor critiquing your teen's actual work,
  never pre-recorded videos. Taste is built through critique, and critique needs eyes on the
  specific piece.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/creative-media-design-masterclass-teens
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 49 OF 137: CYBERSECURITY COMPETITIONS FOR TEENS: PICOCTF AND CYBERPATRIOT PREP
==========

Course link: https://learn.modernagecoders.com/courses/picoctf-cyberpatriot-cybersecurity-competition-prep-for-teens
Subject area: Cybersecurity
Designed for: Teens
Who can join: Beginner to intermediate, ages 13 to 18, no prior security experience needed
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one
  live class a week, four a month; one to one outside India two live classes a week, eight a
  month. Each class is one hour.

WHAT THIS COURSE IS

  Live online cybersecurity competition prep for ages 13 to 18: CyberPatriot-style system
  hardening on Windows and Linux, and picoCTF-style CTF skills, ethical and defensive.

  In one line: Two of the best-known student cyber competitions, one defensive skill set, and a clear line between what is allowed and what is not.

ABOUT THE PROGRAMME

  Student cybersecurity competitions come in two main shapes, and this course prepares for
  both. CyberPatriot, the National Youth Cyber Education Program created by the Air & Space
  Forces Association, runs the National Youth Cyber Defense Competition, in which teams of
  middle and high school students find and fix security vulnerabilities in virtual Windows and
  Linux systems and are scored on how secure they make them. picoCTF, the capture-the-flag
  program built at Carnegie Mellon University and now part of CMU's free CyLab Security
  Academy, sets puzzle challenges in areas such as general skills, cryptography and forensics,
  each hiding a flag to find. The first two months build the foundations both need: the Linux
  command line, Windows administration, how networks move data, and a written white-hat
  agreement that every student signs before any practice begins. Months three and four are
  CyberPatriot-style hardening: user and password policy, services, firewalls, updates,
  permissions and the forensics questions on a practice virtual machine each student sets up
  with the teacher, worked against the clock and as a team. Months five and six are
  capture-the-flag skills: encodings and ciphers, file and packet forensics, reading code, and
  how web and memory bugs are recognised and fixed. Everything is ethical and defensive.
  Students practise only on a practice virtual machine they set up with the teacher and on
  platforms designed for practice, where every challenge is meant to be solved. Nobody is
  taught to touch a system they do not own or have permission to test, and the legal and
  ethical lines are taught as part of the skill, not as a disclaimer.

WHAT MAKES THIS COURSE DIFFERENT

  - Prepares for both competition styles: CyberPatriot-style defence of Windows and Linux
    systems, and picoCTF-style capture-the-flag puzzles
  - Defensive by design: every bug a student finds in a CTF challenge is paired with the fix a
    defender would make
  - Practice only on your own practice virtual machine and on purpose-built practice platforms,
    never on real systems
  - A white-hat agreement signed in week one and revisited every month, covering permission,
    disclosure and competition rules
  - Team skills taught on purpose: roles, checklists and communication during a timed round
  - Written solutions for every challenge, so students learn to explain their reasoning as well

as find the answer

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations and ethics: the white-hat agreement, the Linux command line, Windows administration and networking basics
Phase 2: System hardening in the CyberPatriot style: Windows and Linux hardening checklists, forensics questions, network fundamentals and timed team rounds
Phase 3: Capture-the-flag skills in the picoCTF style: encodings and cryptography, forensics, reading code, web and memory bugs with their fixes, and a mock CTF

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None required. Some Python helps in month 5, and we teach the small amount needed
Computer skills: Comfortable installing software, managing files and using settings menus
Equipment: A laptop or desktop that can run a virtual machine, with a stable internet connection. We check the computer and help set up the free virtual machine software in the first week
Age: 13 to 18
Mindset: Willingness to sign and keep the white-hat agreement: practise only where you have permission, and report rather than exploit

WHO THIS COURSE IS FOR

School cyber team members: Students on, or joining, a school or club CyberPatriot team who want stronger individual skills to bring to it
Capture the flag beginners: Teens who have heard of picoCTF or capture-the-flag events and want a structured start
Puzzle solvers: Students who enjoy codes, logic and detective work
Future security students: Teens considering cybersecurity or computer science who want real hands-on foundations
Cybersecurity course graduates: Students who finished our Cybersecurity for Teens course and want a competition-focused next step

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS AND ETHICS (MONTHS 1-2, WEEKS 1-8)

The shared foundations of both competitions, starting with the rules: what is allowed, what is not, and why that line matters.

Month 1: The White-Hat Agreement and the Linux Command Line (Weeks 1-4)
Weeks 1-2: Rules, Ethics and the Practice Lab
Topics:
- Permission as the dividing line between security work and crime
- Competition rules: what is allowed during a round and what gets a team disqualified
- The white-hat agreement every student signs before practice begins
- How CyberPatriot and capture-the-flag events differ, and what each one rewards
- Setting up a practice lab with a free virtual machine program
- Snapshots: how to break a practice system safely and restore it
Projects built:
- Signed white-hat agreement and a working practice lab with a Linux virtual machine and a restore snapshot
Practice: Write a short note explaining, in your own words, why practising only on authorised systems matters
Weeks 3-4: Linux Command Line Essentials
Topics:
- Navigating the file system: pwd, ls, cd and paths
- Reading files with cat, less, head and tail
- Searching with grep and find
- Pipes and redirection
- File permissions and ownership with ls -l, chmod and chown
- Users, groups and sudo
Projects built:
- Command line treasure hunt: an instructor-built set of files and folders with clues found only by using the right commands
Practice: Solve ten general skills challenges on a practice platform and keep a command notebook
Assessment: Month 1 check: a timed command line task in the practice lab

Month 2: Windows Administration and Networking (Weeks 5-8)
Weeks 5-6: Windows Administration
Topics:
- Local users and groups, and who should be an administrator
- Password and account lockout policies in Local Security Policy

- Services: what runs automatically and why that matters
- Windows Update, Windows Security and the firewall
- Installed programs and how to spot software that should not be there
- Event Viewer as a record of what happened

Projects built:
- Read-only Windows settings audit on your own computer: users, services and updates checked against a written checklist, nothing changed

Practice: Document ten Windows security settings: where each one lives and what the secure value is

Weeks 7-8: How Networks Move Data

Topics:
- IP addresses, subnets and the idea of a network
- Ports and services: what is listening and why
- TCP, UDP and the handshake
- DNS and how names become addresses
- Reading a packet capture in Wireshark on files we provide
- Switches, routers and how traffic finds its way

Projects built:
- Packet capture walkthrough: an annotated explanation of a provided capture from first packet to last

Practice: Answer a set of questions about two provided packet captures and explain how each answer was found

Assessment: Phase 1 assessment: a written and practical check on Linux, Windows and networking basics, plus the ethics questions

---

PHASE 2: SYSTEM HARDENING IN THE CYBERPATRIOT STYLE (MONTHS 3-4, WEEKS 9-16)

Defence under a clock: securing Windows and Linux systems against a scenario, answering forensics questions, learning network fundamentals and working as a team.

Month 3: Hardening Windows and Linux (Weeks 9-12)

Weeks 9-10: Hardening Windows

Topics:
- Reading the scenario first: which users, software and services are authorised
- Removing unauthorised users and fixing administrator rights
- Setting password, lockout and audit policies
- Disabling unneeded services and features
- Updates, firewall and built-in protection settings
- Answering forensics questions before changing anything that might destroy evidence

Projects built:
- Personal Windows hardening checklist, checked read-only against your own computer's settings

Practice: Harden one practice image and record every change made, why it was made and how it was checked

Weeks 11-12: Hardening Linux

Topics:
- Users, groups and the sudoers file
- Password policy and locking unused accounts
- Package updates with the system package manager
- The uncomplicated firewall, ufw
- Securing remote login settings and disabling what is not needed
- Finding unwanted software, scheduled tasks and dangerous file permissions

Projects built:
- Personal Linux hardening checklist, tested on your own Linux practice virtual machine

Practice: Harden one Linux practice image and compare your checklist with a classmate's

Assessment: Month 3 check: a timed hardening round on a practice image, reviewed item by item against the scenario and your checklist

Month 4: Network Fundamentals and Team Rounds (Weeks 13-16)

Weeks 13-14: Network Fundamentals and Server Roles

Topics:
- IP addressing and subnetting practice
- Configuring a small simulated network: addresses, switches and a router
- Common server roles and the extra settings they need
- Secure file sharing and remote access settings
- Networking quiz practice in the style of the Cisco Networking Challenge
- Using the official training materials when a coach provides access

Projects built:
- Small simulated network built and documented, with a subnetting worksheet

Practice: Complete a networking quiz and build one small simulated network from a written brief

Weeks 15-16: Team Rounds

Topics:
- Team roles: Windows lead, Linux lead, forensics, and someone tracking the score
- Splitting work using the scenario and the checklists
- Communicating clearly under time pressure
- Keeping notes so nothing is done twice or undone
- Reviewing a round: what scored, what was missed and why
- Competition rules on outside help during a round

Projects built:
- Full mock team round on your Linux practice virtual machines and written Windows scenarios, followed by a written team review

Practice: Write your role card for a team round: your checklist, your handoffs and what you report

Assessment: Phase 2 assessment: a timed team round plus an individual hardening task

---

PHASE 3: CAPTURE-THE-FLAG SKILLS IN THE PICOCTF STYLE (MONTHS 5-6, WEEKS 17-24)

Puzzle-style security challenges on purpose-built practice platforms: encodings and ciphers, forensics, reading code, and how web and memory bugs are recognised and fixed, ending in a mock CTF.

Month 5: Cryptography and Forensics (Weeks 17-20)

Weeks 17-18: Encodings and Cryptography

Topics:
- Encodings are not encryption: binary, hexadecimal, ASCII and Base64
- Classical ciphers: Caesar, substitution and Vigenere, and how they are broken
- XOR and why reusing a key is a mistake
- Hashes: what they are for and why passwords are stored as hashes
- Public-key encryption explained with small numbers
- Writing short Python scripts to automate decoding

Projects built:
- Cryptography challenge set solved on a practice platform, with a written solution for each

Practice: Solve five cryptography challenges and explain the weakness each one relied on

Weeks 19-20: Forensics

Topics:
- File signatures and why a file's name can lie about its type
- Metadata in images and documents
- Hidden data and simple steganography
- Searching files and disk images for strings
- Packet captures as evidence: following a conversation in Wireshark
- Keeping a clean record of what was found and how

Projects built:
- Forensics challenge set solved on a practice platform, with a written solution for each

Practice: Solve five forensics challenges and write the tool and the clue that cracked each one

Assessment: Month 5 check: a timed mixed cryptography and forensics set with written solutions

Month 6: Reading Code, Bugs and Fixes, and a Mock CTF (Weeks 21-24)

Weeks 21-22: Reading Code and Recognising Bugs

Topics:
- Reading short Python and C programs to work out what they do
- Reverse engineering basics: understanding a program without its source
- How web applications trust input, and how input validation fixes that
- Injection and cross-site scripting explained with the defender's fix: parameterised queries and output encoding
- Memory safety: what a buffer overflow is and how bounds checking prevents it
- Practising only on the platform's purpose-built challenges, never on real sites or programs

Projects built:
- Bug and fix notebook: for each practice challenge, the bug, how it was recognised and the code change that would fix it

Practice: Solve three reading-code challenges and write the fix for each bug found

Weeks 23-24: Mock CTF and the Road Ahead

Topics:
- Competition strategy: reading every challenge, taking the quick points first
- Team play in a CTF: splitting categories and sharing notes
- When to use a hint and when to move on
- Writing clear solutions after the event
- Responsible disclosure: what to do if you ever find a real vulnerability
- Where to keep practising: free platforms built for students

Projects built:
- Mock CTF on an instructor-built challenge set, followed by a polished solution portfolio

Practice: Write up your three favourite challenges from the course as portfolio pieces

Assessment: Final assessment: mock CTF performance, the hardening and solution portfolio, and a short ethics conversation

PROJECTS THE STUDENT BUILDS AND KEEPS

- Signed white-hat agreement and a personal practice lab
- Command line treasure hunt and command notebook
- Windows audit report on a practice image
- Annotated packet capture walkthrough
- Tested Windows and Linux hardening checklists
- Documented simulated network with a subnetting worksheet
- Mock team round with a written team review

- Cryptography and forensics solution sets
- Bug and fix notebook
- Mock CTF solution portfolio

Total projects: 10 defensive artifacts: checklists, audits, team reviews and written CTF solutions

SKILLS THE STUDENT WILL HAVE AT THE END

- Linux command line: files, permissions, users, search and pipes
- Windows administration: users, policies, services, updates and the firewall
- Networking basics: addressing, ports, protocols and packet captures
- System hardening against a scenario, under time, as a team
- CTF skills: encodings, classical and modern cryptography ideas, and forensics
- Recognising common web and memory bugs and describing the defensive fix
- Security ethics: permission, competition rules and responsible disclosure

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, with demonstrations and hands-on lab work
Practice: 2-3 hours between classes on practice images and practice-platform challenges
Review: Every checklist, audit and written solution reviewed with feedback
Homework: A lab task or challenge set after every class, and a monthly mixed-review task that brings back earlier skills

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, including help when a practice lab will not start
Progress updates: Regular progress notes to parents, including the ethics agreement their teen signed

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Entering CyberPatriot through a school or approved organisation team, with practised skills
- Competing in capture-the-flag events on free platforms built for students
- Our Python or programming courses, where scripting and safe-coding habits grow further
- Further study toward defensive roles such as security analyst or incident responder, which build on these foundations
- Computer science electives and clubs, with a security perspective few classmates have

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course ethical and legal?
A: Yes, by design. Students practise only on a practice virtual machine they set up with the teacher and on practice platforms where every challenge is meant to be solved. Every student signs a white-hat agreement in week one: practise only with permission, follow competition rules, and report rather than exploit. When a capture-the-flag challenge involves a bug, students learn the fix a defender would make, not how to use it elsewhere.

Q: What is CyberPatriot?
A: CyberPatriot is the National Youth Cyber Education Program created by the Air & Space Forces Association. Its National Youth Cyber Defense Competition puts teams of high school and middle school students in charge of finding and fixing security vulnerabilities in virtual Windows and Linux systems, and teams are scored on how secure they make them. Alongside that Network Security Challenge, CyberPatriot describes a Cisco Networking Challenge made up of an online quiz and a virtual networking exercise. Rounds run online, and teams choose a four-hour period on the competition weekend.

Q: Can you register my teen's CyberPatriot team?
A: No. CyberPatriot teams register through a school or approved youth organisation, with an adult coach. This course trains individual students so they bring stronger skills to their team; it is not a team itself. Check eligibility and dates with CyberPatriot directly.

Q: What is picoCTF, and where is it now?
A: picoCTF is the capture-the-flag program built by Carnegie Mellon University. picoctf.org now points to CyLab Security Academy, CMU's free cybersecurity learning platform, where existing picoCTF accounts carry over. It has practice challenges across areas such as general skills, cryptography, web exploitation, forensics, binary exploitation and reverse engineering, plus timed events, some of which allow team play.

Q: Does my teen need to know coding?

A: No. The first four months are about the command line, system settings and networks rather than programming. Month 5 uses a little Python to automate decoding, and we teach what is needed. Students who already code get more out of the reading-code weeks.

Q: Is this different from your Cybersecurity for Teens course?
A: Yes. Cybersecurity for Teens is a broad, defensive introduction to online safety, attacks and defence for any teen, with no coding and no lab setup. This course is hands-on competition preparation: virtual machines, system hardening, network exercises and capture-the-flag practice. Many students take Cybersecurity for Teens first, but it is not required.

Q: What computer does my teen need?
A: A laptop or desktop that can run a virtual machine, plus a stable internet connection. Tablets and most Chromebooks are not suitable. We check the computer and help set up the free virtual machine software in the first week.

Q: Will this guarantee a place in a competition final?
A: No, and nobody honest can promise that. Results depend on the team, the round and the student's own practice. What the course does is build real skills, tested in timed practice rounds, so a student walks into a competition prepared.

Q: How are the classes run?
A: Live online classes, twice a week, with a real teacher, in a group of 15 to 20 students, a mini batch of up to 6, or one to one. Current fees for your region are shown in the pricing section of this page.

Q: Can we try a class first?
A: Yes. Book a free demo class. The teacher walks your teen through a short practice challenge, explains the ethics agreement, and suggests whether group, mini batch or one-to-one classes fit best. Parents are welcome to watch. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/picoctf-cyberpatriot-cybersecurity-competition-prep-for-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 50 OF 137: CYBERSECURITY FOR TEENS: ONLINE SAFETY & WHITE-HAT DEFENSE
==========

Course link: https://learn.modernagecoders.com/courses/cybersecurity-course-for-teens-ethical-defensive
Subject area: Cybersecurity
Designed for: Teens
Who can join: Beginner, ages 13 to 18, no coding required
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online cybersecurity course for teens (13-18): learn how attacks work and how defense wins, from passwords and phishing to encryption and CTF puzzles.

In one line: Teens live online. This course teaches them how that world actually works, how it gets attacked, and how defenders win.

ABOUT THE PROGRAMME

Most teenagers manage more online accounts than their parents, yet nobody has ever taught them how any of it works or how it fails. This 6-month live course fixes that, and it does so honestly: this is a defensive course. Students learn how attacks work conceptually, how a phishing email is constructed, why password reuse is the root of most account theft, how ransomware spreads, because you cannot defend against what you do not understand. Nobody here is taught to break into anything, and every student signs a white-hat code of conduct in week one that we return to all course long. The arc runs from how the internet actually moves data, through passwords, hashing, and two-factor authentication, into phishing and social engineering recognition, malware concepts and safe habits, then encryption from paper ciphers to the HTTPS padlock, and web security ideas like cookies and sessions. It ends with a

capture-the-flag month: instructor-built puzzle competitions in cryptography, logic, and file forensics that are legal, safe, and genuinely fun. Along the way students harden their own accounts, audit their own digital footprint, and brief their own families, so the payoff starts in week one, not after graduation.

WHAT MAKES THIS COURSE DIFFERENT

- Strictly defensive and ethical: attacks are explained conceptually so defense makes sense, and no offensive tooling is ever taught
- A white-hat ethics thread runs through every month, starting with a signed code of conduct in week one
- Students secure their real digital lives during the course: accounts hardened, backups running, families briefed
- Case studies of real incidents are read the way defenders read them: what failed, and what would have stopped it
- The capstone is a capture-the-flag event built entirely by our instructors, so every puzzle is legal, safe, and age-appropriate
- No coding required: the course is about systems and judgment, and the safe-coding week meets students at whatever level they code

THE LEARNING PATH, STEP BY STEP

Phase 1: How the digital world works and how to protect your own corner of it: networks, digital footprints, threat thinking, passwords, hashing, and 2FA
Phase 2: Attacks understood, defenses built: phishing and social engineering recognition, malware concepts, backups, and hardening real devices
Phase 3: Cryptography from paper ciphers to HTTPS, web security concepts, safe coding habits, and a capture-the-flag capstone with written solutions

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None required. The safe-coding week adapts to whatever level a student codes at, including none
Computer skills: Comfortable using a browser, apps, and settings menus
Equipment: A computer with a modern browser and a stable internet connection; no special tools are installed
Age: 13 to 18. The course discusses real scams and real incidents at a depth suited to this range
Mindset: Willingness to sign and honor the white-hat code of conduct; this course is for defenders

WHO THIS COURSE IS FOR

Every online teen: Students who live online, which is all of them, and have never been taught how any of it works or fails
Curious about hacking: Teens drawn to the hacking mystique who deserve the honest, legal, defensive version of that curiosity
Future security professionals: Students considering cybersecurity as a field who need the real foundation before any advanced study
Puzzle lovers: Teens who enjoy ciphers, logic problems, and competitions; the CTF capstone is built for them
Family tech helpers: The teen everyone in the house already asks for tech help, upgraded into the family's actual defender

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE DIGITAL WORLD AND YOUR IDENTITY (MONTHS 1-2, WEEKS 1-8)

You cannot secure what you do not understand. First the internet itself, then the single most attacked thing on it: your identity.

Month 1: How the Internet Actually Works (Weeks 1-4)

The Life of a Web Request

Topics:
- The white-hat code of conduct: what we do, what we never do, and why, signed by every student
- What happens when you type a URL: DNS, IP addresses, and servers in plain language
- Packets: how data travels in pieces and reassembles
- Clients and servers: who asks, who answers
- Watching a request travel with standard diagnostic tools like traceroute
- Why understanding the plumbing is the first defensive skill

Projects built:
- Map the full path of a web request as an annotated diagram, from keyboard to server and back

Practice: Trace three websites you use daily and note how many network hops each takes

Networks Around You

Topics:
- Home networks: router, Wi-Fi, and every device quietly connected

- Public Wi-Fi: what it can expose and sensible rules for using it
- What the cloud really is: someone else's computer, and what that implies
- Wi-Fi encryption settings and router passwords in plain terms
- The guest network: a simple defense most homes never switch on
- IoT devices: why the smart bulb is part of your security story

Projects built:
- Home network inventory: every connected device listed, with a written safety checklist for the household

Practice: Check your home router's basics with a parent: admin password changed, encryption on, firmware updated

Your Digital Footprint

Topics:
- What websites and apps collect, and where it goes
- Cookies at a first glance: useful, and worth understanding
- Search-engine results for your own name: the public footprint
- Privacy settings that actually matter on the platforms teens use
- Oversharing patterns: location tags, school names, and routines
- The permanence problem: why deleted rarely means gone

Projects built:
- Personal digital footprint audit: what is publicly findable about you, plus a written cleanup plan you execute

Practice: Complete a privacy-settings review on your two most used apps and record what you changed

Thinking Like a Defender

Topics:
- Assets, threats, and risk: security's core vocabulary made concrete
- Threat modeling a phone: what is worth stealing, who might want it, how it is reached
- Likelihood vs impact: why we defend against the boring attacks first
- Defense in depth: layers, not walls
- The human layer: why most successful attacks start with a person, not a computer
- Ethics checkpoint: knowledge of weaknesses creates responsibility, never permission

Projects built:
- Full threat model for a fictional student's phone and accounts, with ranked risks and matched defenses

Practice: Write a one-page threat model for something you own and care about

Assessment: Month 1 check: explain the path of a web request and defend your threat model in a short viva

Month 2: Passwords, Hashing, and Proving Who You Are (Weeks 5-8)

How Passwords Fail

Topics:
- How attackers actually get passwords: guessing, leaked databases, and reuse, explained conceptually
- Credential stuffing: why one leaked password opens five accounts
- What makes a password strong: length beats cleverness
- Passphrases: four random words and why the maths favors them
- Password managers: what they are, how they help, how families adopt one
- Checking exposure: how breach-notification services like Have I Been Pwned work

Projects built:
- Password health review of your own accounts, done privately at home with a parent, with a migration plan to unique passphrases

Practice: Move your three most important accounts to strong unique passphrases and record the process, never the passwords

Hashing: Why Good Sites Never Store Your Password

Topics:
- What a hash function is: one-way, deterministic, avalanche effect
- Hands-on with SHA-256 in a browser demo: same input, same fingerprint
- Why websites store hashes instead of passwords
- Salting: why two users with the same password get different hashes
- What a data breach actually leaks, and why hashing changes the damage
- Hash checking downloads: verifying a file is what it claims to be

Projects built:
- Hashing lab worksheet: predict, test, and explain hash behavior across a set of inputs, including a tampered file

Practice: Verify the published hash of a real downloaded file and write up the steps

Two-Factor Authentication and Beyond

Topics:
- Factors: something you know, have, and are
- Authenticator apps vs SMS codes: why one resists phishing better
- Passkeys: where sign-in is heading and why
- Recovery codes: the step everyone skips until it hurts
- Account recovery hygiene: recovery email and phone numbers kept current
- When 2FA prompts themselves become the attack: fatigue prompts, conceptually

Projects built:
- Account lockdown: 2FA enabled on your key accounts with recovery codes stored safely, checklist evidence submitted

Practice: Help one family member enable 2FA on one account and note what confused them

Identity Theft and Scams That Target Teens

Topics:

- Gaming account theft: the scams built around skins, coins, and trades
- Marketplace and fan-merch scams: too good to be true, quantified
- Impersonation: fake friend accounts and what verification actually verifies
- What identity theft does to a real person: a case walk-through
- Reporting channels: platforms, banks, and India's cybercrime portal
- Ethics checkpoint: what you do when a friend's account is clearly compromised

Projects built:
- Family security briefing: a one-page document teaching your household the month's defenses, delivered as a short talk at home

Practice: Collect two real scam attempts from family chat groups and annotate the tells

Assessment: Month 2 check: passwords and identity quiz plus a live walkthrough of your family briefing

---

PHASE 2: ATTACKS RECOGNIZED, DEFENSES BUILT (MONTHS 3-4, WEEKS 9-16)

The two attack families that touch everyone: deception aimed at people, and malicious software aimed at machines. Recognition first, then working defenses.

Month 3: Phishing and Social Engineering (Weeks 9-12)

Anatomy of a Phishing Email

Topics:
- The standard skeleton: spoofed sender, urgent story, poisoned link
- Reading the real sender address behind a display name
- Lookalike domains: rn vs m, and the tricks typography plays
- Hovering before clicking: reading a link's true destination
- Email headers at a basic level: where a message really came from
- Why banks and platforms never ask for your password by email

Projects built:
- Phish anatomy poster: dissect one anonymized real phishing email into its labeled parts

Practice: Review your own spam folder with a parent and identify the technique in five messages

Beyond Email: Where Teens Actually Get Phished

Topics:
- Smishing: scam texts about deliveries, prizes, and blocked accounts
- Fake login pages: pixel-perfect copies and the URL that gives them away
- Discord, WhatsApp, and in-game phishing: free-item links and fake giveaways
- QR code scams: why a square of dots deserves the same suspicion as a link
- Browser warnings and what they actually mean
- The verification habit: independently checking through the real app or site

Projects built:
- Spot-the-phish gallery: grade 15 anonymized real samples as safe or suspicious, with written reasoning for each call

Practice: Build your own three-question checklist for judging any unexpected message

The Psychology of Social Engineering

Topics:
- The levers: authority, urgency, fear, and flattery
- Pretexting: attacks that begin with a believable story
- Case study read as defenders: a major breach that started with a phone call
- Why smart people fall for scams: stress and context, not stupidity
- OSINT awareness: how oversharing feeds targeted attacks, tying back to week 3
- Ethics checkpoint: persuasion skills and the bright line around consent and honesty

Projects built:
- Case study brief: reconstruct a documented social engineering incident and identify the three points where defense could have stopped it

Practice: Find the persuasion lever in five real scam messages from the class archive

The Phish Detector Field Guide

Topics:
- Consolidating the month into a usable field guide
- Writing detection rules a grandparent could follow
- Reporting phishing properly: platform tools and national portals
- What to do after a wrong click: damage control steps in order
- Designing the class recognition tournament
- Teaching as testing: explaining a tell proves you understand it

Projects built:
- Phish detector field guide: an illustrated recognition and response guide, then the class phishing-recognition tournament

Practice: Test your field guide on a family member and revise the rule they misunderstood

Assessment: Month 3 check: timed recognition tournament across 20 fresh samples, plus field guide review

Month 4: Malware Concepts and Safe Habits (Weeks 13-16)

The Malware Families

Topics:
- Viruses, worms, and trojans: what each name actually means
- Ransomware: encryption turned against you, conceptually
- Spyware and keyloggers: the quiet observers
- Adware and unwanted programs: the grey zone

- How infections really begin: attachments, cracked software, fake updates, and bundled installers
- Why cracked games are the classic teen infection route

Projects built:
- Malware family tree: an illustrated chart mapping each family to its behavior and its most common arrival route

Practice: Write plain-language definitions of five malware terms for a younger sibling

Defenses That Actually Work

Topics:
- Updates as defense: what a patch is and why delay is dangerous
- Antivirus and built-in protections: what they catch and what they miss
- Least privilege at home: why daily life should not run on an admin account
- Sandboxing as a concept: why apps on a phone cannot read each other's data
- Backups and the 3-2-1 rule: the defense that defeats ransomware
- Restore drills: an untested backup is a hope, not a plan

Projects built:
- Personal backup plan: design and actually implement a 3-2-1 backup for your own important files, with a test restore

Practice: Run updates on your devices with a parent and document what was pending

Case Study: A Ransomware Outbreak, Read as Defenders

Topics:
- The WannaCry outbreak: what happened, in sequence
- The unpatched systems that let it spread
- Who was hit and what it cost, including hospitals
- The defenses that worked where it failed to spread
- What a recovery effort looks like from the inside
- Ethics checkpoint: responsible disclosure, and the researchers who reported flaws the right way

Projects built:
- Defender's timeline: reconstruct the outbreak and mark every point where standard defenses from week 14 would have changed the outcome

Practice: Read one news report on a recent incident and identify which defense was missing

Hardening Your Own Machines

Topics:
- Safe downloading: official sources, and reading installer screens instead of clicking next
- Browser hygiene: extensions audited, permissions reviewed
- App permissions on phones: flashlight apps that want your contacts
- Recognizing fake download buttons and bundled junk
- The family device sweep: applying the whole month at home
- Writing a maintenance routine you will actually follow

Projects built:
- Harden-your-browser lab: full extension and permission audit on your own browser and phone, with before-and-after notes

Practice: Complete the device sweep on one family computer and log every change made

Assessment: Month 4 check: malware and defenses quiz plus review of your implemented backup and hardening evidence

---

PHASE 3: CRYPTOGRAPHY, THE WEB, AND THE CTF CAPSTONE (MONTHS 5-6, WEEKS 17-24)

The mathematics of secrets, the trust machinery of the web, safer coding habits, and a capture-the-flag capstone that turns six months of defense into sport.

Month 5: Encryption and How the Web Stays Trustworthy (Weeks 17-20)

Classical Ciphers by Hand

Topics:
- The Caesar cipher: shifting the alphabet, and breaking it by brute force
- Substitution ciphers and keys
- Frequency analysis: why English's love of the letter E breaks substitution
- The Vigenere idea: why longer keys resist analysis
- A short history: ciphers, wars, and the codebreakers of Bletchley Park
- What classical failures teach modern cryptography

Projects built:
- Cipher exchange: encode messages for classmates, then break intercepted ones using frequency analysis worksheets

Practice: Break three prepared ciphertexts of increasing difficulty and document your method

Modern Encryption, Conceptually

Topics:
- Symmetric encryption: one shared key, and the delivery problem it creates
- Public-key encryption: the open padlock idea that solved key delivery
- Digital signatures: proving who wrote a message
- Key length and why brute force fails against modern ciphers
- End-to-end encryption: what your messaging apps actually promise
- Why nobody invents their own cipher: the first rule of cryptography

Projects built:
- Concept comic or storyboard: explain the padlock analogy for public-key encryption well enough to teach it

Practice: Identify which of your daily apps use end-to-end encryption and what that covers

Cookies, Sessions, and the Padlock

Topics:

- What logged in actually means: sessions and cookies doing their real jobs
- Why stolen cookies matter: the session as a temporary key
- HTTPS vs HTTP: what the padlock encrypts and what it does not promise
- Certificates: how your browser decides whom to trust
- Certificate warnings: what they mean and when to walk away
- Public Wi-Fi revisited with new eyes: what HTTPS protects there and what it cannot

Projects built:

- Trust tour: inspect certificates and cookies on five real sites using browser dev tools, documenting what each reveals

Practice: Find one HTTP-only site in the wild and write up why the warning matters

Safe Coding Habits

Topics:

- Why apps get hacked, conceptually: trusting user input is the root of most classics
- Validating input: the habit, shown at whatever level students code
- Secrets never go in code: passwords, keys, and the accidental publish
- Dependencies age: why software built on old parts inherits their holes
- A gentle look at injection as an idea: data that gets treated as instructions
- Ethics checkpoint: finding a flaw in a real site means reporting, never poking further

Projects built:

- Code safety review: audit one of your own past projects, from any language or even Scratch, against a provided safety checklist

Practice: Add input validation to one small program you have written, or trace validation in a provided example

Assessment: Month 5 check: cryptography and web trust quiz plus cipher-breaking practical

Month 6: The Capture-the-Flag Capstone (Weeks 21-24)

Welcome to CTF

Topics:

- What a capture-the-flag competition is, and its role in real security education
- Our rules: every puzzle is instructor-built, self-contained, legal, and safe
- Puzzle categories: cipher breaking, logic, metadata forensics, and web-trust trivia
- Reading a puzzle: what is given, what is asked, what is hidden
- The solver's notebook: keeping notes that turn into writeups
- Practice round one, solved together as a class

Projects built:

- Practice CTF round: five starter puzzles solved individually with notes, then reviewed together

Practice: Solve three take-home puzzles and log your approach for each, including dead ends

Training Camp

Topics:

- Cipher puzzles under time: applying month 5 at speed
- File forensics-lite: what image metadata and file properties reveal
- Logic and encoding puzzles: binary, hexadecimal, and Base64 as everyday costumes for data
- Teamwork in CTFs: splitting categories and sharing notes
- Hint discipline: when to take a hint and what it costs
- The writeup format: challenge, approach, solution, lesson

Projects built:

- Team training rounds in every category, plus one polished writeup of your favorite solved puzzle

Practice: Daily puzzle streak: one puzzle per day with a two-line log entry each

The Capstone CTF Event

Topics:

- Final event briefing and team formation
- Competition strategy: triage easy points first
- The event itself: multi-category, scored, across class sessions
- Staying methodical under a clock
- Sportsmanship and collaboration within the rules
- Post-event solution reveal for every unsolved puzzle

Projects built:

- The capstone CTF: a scored, multi-category competition built from the entire course's material

Practice: Draft writeups for every puzzle your team solved while they are fresh

Writeups and the White-Hat Road Ahead

Topics:

- Polishing writeups into a portfolio piece
- Responsible disclosure as a practice: how professionals report what they find
- Defensive security careers surveyed honestly: analyst, incident response, security engineering, and the study each requires
- Where to keep learning: age-appropriate CTF platforms and safety-first communities
- The code of conduct, revisited as a graduation promise
- Presentations and certificates

Projects built:

- Final CTF writeup portfolio presented to the class, alongside your family briefing and audit

documents from the course
Practice: Write a one-page letter to your future self on the security habits you intend to keep
Assessment: Capstone assessment: CTF performance, writeup portfolio, and a short ethics viva

PROJECTS THE STUDENT BUILDS AND KEEPS

- Annotated map of a web request's full path
- Home network inventory with a household safety checklist
- Personal digital footprint audit with an executed cleanup plan
- Threat model for a fictional student's phone and accounts
- Account lockdown: passphrases, 2FA, and recovery codes on real accounts
- Family security briefing, written and delivered at home
- Phish detector field guide plus recognition tournament
- Implemented 3-2-1 backup plan with a test restore
- Defender's timeline of a real ransomware outbreak
- Capture-the-flag writeup portfolio from the capstone event

Total projects: 10+ defensive artifacts: audits, field guides, hardened accounts, and CTF writeups

SKILLS THE STUDENT WILL HAVE AT THE END

- Networks and the web: how data moves, where trust lives, and what the padlock really promises
- Identity defense: passphrases, hashing, 2FA, and recovery done properly on real accounts
- Attack recognition: phishing, social engineering, and malware understood well enough to spot cold
- Practical hardening: backups, updates, permissions, and browser hygiene, implemented not just discussed
- Cryptography foundations: from breaking Caesar ciphers by hand to explaining public-key encryption
- White-hat judgment: ethics, responsible disclosure, and the habit of reporting rather than exploiting

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, with demonstrations, case studies, and hands-on labs
Practice: 2-3 hours between classes: audits, puzzles, and real hardening tasks on the student's own devices
Review: Every artifact reviewed with written feedback; the ethics thread checked in every month
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, including help when a real security question comes up at home
Progress updates: Regular progress notes to parents, who also receive the family briefing their teen produces in month 2

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Further defensive study toward security analyst and incident response roles, which build directly on these concepts
- Age-appropriate CTF platforms and school cyber competitions, entered with real preparation
- Our Python and programming courses, where the safe-coding habits from month 5 get applied for real
- Computer science electives and projects with a security perspective few classmates will have
- The informal but real role of family and school security helper, taken on responsibly

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Will my teen learn to hack?
A: Not in the sense the movies mean, and we are upfront about that. This is a defensive course: students learn how attacks work conceptually, how phishing is built, how malware spreads, how passwords get stolen, because recognition is the heart of defense. Nobody is taught intrusion tools or techniques for breaking into systems, every student signs a white-hat code of conduct in week one, and the capture-the-flag puzzles are instructor-built, self-contained, and legal.

Q: Is this course safe and legal?
A: Yes, by design. Every exercise runs on the student's own accounts and devices or on materials we built for the class; nothing ever touches systems that are not ours to touch. The course actively teaches the legal and ethical lines, including responsible disclosure, so students finish knowing not just what is possible but what is right. Parents are involved directly: several assignments, like the account lockdown and home network review, are done with a parent.

Q: Does my teen need to know coding first?
A: No. The course is about systems, judgment, and habits rather than programming, and most weeks involve no code at all. The one safe-coding week in month 5 meets each student at their level, whether that is Scratch, some Python, or nothing yet. Students who do code get a useful bonus: they learn to look at their own projects the way an attacker would, and fix what they find.

Q: What does my teen actually produce in this course?
A: Real defensive artifacts, most of them immediately useful: a digital footprint audit with an executed cleanup, hardened accounts with strong passphrases and 2FA, a working 3-2-1 backup with a test restore, a phishing field guide, a family security briefing delivered at home, and a portfolio of capture-the-flag writeups from the capstone. The family typically becomes measurably safer while the course is still running.

Q: Is this useful for a cybersecurity career?
A: It is the correct first step, and we will not overclaim beyond that. Professional security roles require further study, in networking, operating systems, and usually programming, but they all build on exactly the concepts this course teaches: threat thinking, cryptography basics, authentication, and attack recognition. A teen who finishes this course knows whether the field genuinely interests them and holds a foundation most college freshmen lack. The safety payoff, meanwhile, is immediate rather than someday.

Q: What is a CTF, and is it appropriate for teens?
A: Capture the flag is the puzzle-competition format used across security education worldwide: players solve challenges, break a practice cipher, find data hidden in a file's metadata, decode a message, to find hidden flags and score points. Ours is entirely instructor-built, so every puzzle is age-appropriate, legal, and self-contained. It is the most anticipated month of the course, and the writeups students produce are genuine analytical work.

Q: How are the classes taught?
A: Live one-hour online classes, twice a week, with a real instructor and groups capped at 20 students. Mini-batch (up to 6 students) and personal 1-on-1 options are available.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final exam covering the whole course. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Can we try a class before enrolling?
A: Yes, the first demo class is free with no obligation. Book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to observe, and for this course in particular we encourage it: you will see exactly how the defensive, ethics-first approach works in practice.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link:
https://learn.modernagecoders.com/courses/cybersecurity-course-for-teens-ethical-defensive
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 51 OF 137: DATA ANALYSIS COURSE: EXCEL, SQL, PYTHON & BI
==========

Course link: https://learn.modernagecoders.com/courses/data-analysis-mastery-course-college
Subject area: Data Science
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 8-10 months (35-43 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Data analysis course for college students and professionals: Excel mastery, statistics and SQL, then Python with Pandas, BI dashboards and industry analytics, ending in a portfolio capstone, in 8-10 months.

In one line: The honest analyst toolkit in the order jobs use it: Excel done properly, statistics and SQL, then Python, BI dashboards and the industry literacy that makes analysis land.

ABOUT THE PROGRAMME

Data analyst is the most reachable serious tech role there is, and the job is concrete: take messy business data, make it trustworthy, and turn it into answers a decision-maker can act on. This live online course teaches exactly that craft in the order real work uses it: Excel to genuine mastery first, because it remains the language of business, with the statistics that keep analysis honest, then SQL to production analyst level including window functions, then the Python layer: Pandas for cleaning and transformation, real exploratory analysis, visualization, BI dashboards and data storytelling, plus practical machine learning for analysts. The third phase adds what bootcamps skip: industry literacy across finance, healthcare, retail, supply chain and HR analytics, cloud platforms, and the advanced statistical methods behind A/B tests. A finale month ships a portfolio capstone and sits the final exam. The pace is honest: about seven months of teaching at two live classes a week plus four to six hours of practice, with margin built in for revision and life. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- Excel respected, not skipped: business still speaks it, so analysts must own it
- SQL to real analyst depth: joins, aggregation and window functions, not SELECT-star tutorials
- Storytelling as a core skill: analysis that changes decisions, not chart dumps
- Industry literacy included: finance, healthcare, retail, supply chain and HR analytics
- An honest 8-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Excel, statistics and SQL (Months 1-3): the trustworthy-numbers foundation
Phase 2: Python, Pandas and BI (Months 3-5): cleaning, analysis, dashboards and storytelling
Phase 3: Industry analytics (Months 5-7): domain literacy, cloud platforms and advanced methods
Phase 4: The finale (Month 8): a portfolio capstone and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Everything is taught from the first line
Equipment: Any computer with a stable internet connection; every tool used is free
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

College students: Students who want the most reachable serious data role
Working professionals: Professionals whose jobs already involve spreadsheets and reporting
Career switchers: Newcomers who want a concrete, employable data skill without a maths gauntlet
Business roles: Operations, marketing and finance people ready to own their own numbers
Future data scientists: Learners using the analyst road as the honest first step toward data science

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: EXCEL, STATISTICS & SQL (MONTHS 1-3, WEEKS 1-10)

Build strong foundations in data concepts, statistics, and become an Excel power user.

Month 1: Data Fundamentals & Excel (Weeks 1-3)
Introduction to Data Analysis
Topics:
- What is data analysis? Role in modern business
- Types of data: structured, unstructured, semi-structured
- Data analysis lifecycle and methodology
- Quantitative vs qualitative analysis
- Descriptive, diagnostic, predictive, prescriptive analytics

- Industry applications and case studies
- Tools ecosystem overview
- Career paths in data analysis
- Setting up your analysis environment
- Introduction to datasets and data sources
- Data ethics and privacy considerations
- Building analytical thinking

Projects built:
- Personal data audit project
- Industry analysis report
- Data source catalog creation

Practice: Analyze 10 real-world data scenarios

Excel Fundamentals & Formulas

Topics:
- Excel interface mastery and shortcuts
- Data types and formatting
- Cell references: absolute, relative, mixed
- Essential formulas: SUM, AVERAGE, COUNT, MIN, MAX
- Logical functions: IF, AND, OR, NOT
- Nested IF statements and IFS function
- Text functions: CONCATENATE, LEFT, RIGHT, MID, TRIM
- Date and time functions
- Mathematical functions: ROUND, ABS, MOD
- Error handling: IFERROR, IFNA
- Formula auditing and debugging
- Named ranges and structured references

Projects built:
- Sales calculator dashboard
- Employee timesheet system
- Grade calculation spreadsheet

Practice: Complete 50 Excel formula challenges

Excel Advanced Functions

Topics:
- LOOKUP functions: VLOOKUP, HLOOKUP, XLOOKUP
- INDEX and MATCH combinations
- SUMIF, COUNTIF, AVERAGEIF functions
- SUMIFS for multiple criteria
- Array formulas and dynamic arrays
- FILTER, SORT, UNIQUE functions
- Statistical functions in Excel
- Financial functions: NPV, IRR, PMT
- Database functions: DSUM, DCOUNT
- INDIRECT and OFFSET for dynamic references
- Creating custom functions with VBA basics
- Excel automation introduction

Projects built:
- Dynamic sales report system
- Financial analysis model
- HR analytics dashboard

Practice: Build 25 complex Excel solutions

Month 2: Statistics & Data Visualization in Excel (Weeks 4-6)

Statistics Fundamentals

Topics:
- Types of data: nominal, ordinal, interval, ratio
- Population vs sample
- Measures of central tendency: mean, median, mode
- Measures of spread: range, variance, standard deviation
- Quartiles and interquartile range
- Skewness and kurtosis
- Normal distribution and z-scores
- Central limit theorem basics
- Confidence intervals introduction
- Hypothesis testing concepts
- P-values and significance levels
- Type I and Type II errors

Projects built:
- Statistical analysis report
- A/B testing simulator
- Quality control dashboard

Practice: Perform 30 statistical analyses

Excel Data Analysis Tools

Topics:
- PivotTables creation and customization
- PivotTable calculations and summaries
- Slicers and timeline filters
- PivotCharts and interactive dashboards
- Power Pivot introduction
- Data Model and relationships

```
      - DAX formulas basics
      - What-if analysis: Goal Seek, Scenario Manager
      - Data tables for sensitivity analysis
      - Solver for optimization problems
      - Analysis ToolPak features
      - Regression analysis in Excel
    Projects built:
      - Sales performance dashboard
      - Budget optimization model
      - Multi-dimensional analysis cube
    Practice: Create 20 PivotTable reports
  Data Visualization in Excel
    Topics:
      - Chart types and when to use them
      - Column, bar, and line charts
      - Pie charts and donut charts
      - Scatter plots and bubble charts
      - Combination charts
      - Sparklines for inline visualization
      - Conditional formatting techniques
      - Data bars, color scales, icon sets
      - Heat maps and treemaps
      - Waterfall and funnel charts
      - Chart formatting and design principles
      - Creating interactive dashboards
    Projects built:
      - Executive dashboard
      - KPI scorecard
      - Interactive sales report
    Practice: Design 30 data visualizations

Month 3: SQL & Python for Data Analysis (Weeks 7-10)
  SQL Fundamentals
    Topics:
      - Database concepts and relational model
      - SQL syntax and structure
      - SELECT statements and filtering with WHERE
      - Sorting with ORDER BY
      - LIMIT and OFFSET for pagination
      - DISTINCT for unique values
      - Aggregate functions: COUNT, SUM, AVG, MIN, MAX
      - GROUP BY for aggregation
      - HAVING clause for group filtering
      - Working with NULL values
      - Date and time functions in SQL
      - String manipulation in SQL
    Projects built:
      - Customer analysis queries
      - Sales performance reports
      - Inventory tracking system
    Practice: Write 100 SQL queries
  SQL Joins & Advanced Queries
    Topics:
      - Understanding table relationships
      - INNER JOIN for matching records
      - LEFT and RIGHT OUTER JOINS
      - FULL OUTER JOIN usage
      - CROSS JOIN and Cartesian products
      - Self joins for hierarchical data
      - Multiple joins in complex queries
      - Subqueries and nested queries
      - Correlated subqueries
      - Common Table Expressions (CTEs)
      - UNION, INTERSECT, EXCEPT operations
      - Query optimization basics
    Projects built:
      - Multi-table reporting system
      - Customer journey analysis
      - Supply chain analytics
    Practice: Master 50 complex join scenarios
  SQL Analytics Functions
    Topics:
      - Window functions introduction
      - ROW_NUMBER, RANK, DENSE_RANK
      - Running totals with SUM() OVER
      - Moving averages with AVG() OVER
      - LAG and LEAD for time comparisons
      - FIRST_VALUE and LAST_VALUE
      - Percentile calculations
```

- Pivoting and unpivoting data
- CASE statements for conditional logic
- Creating views for reusable queries
- Stored procedures basics
- Database performance tuning

Projects built:
- Time series analysis system
- Cohort analysis platform
- RFM segmentation model

Practice: Build 30 analytical queries

Python Fundamentals for Analysis

Topics:
- Python setup for data analysis
- Jupyter Notebook environment
- Variables and data types
- Lists, tuples, dictionaries
- Control flow: if, for, while
- Functions and lambda expressions
- File handling and CSV processing
- Error handling basics
- Libraries: NumPy basics
- Introduction to Pandas
- Reading data from various sources
- Google Colab for cloud computing

Projects built:
- Data processing pipeline
- File converter utility
- Analysis automation script

Practice: Complete 50 Python exercises

Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 2: PYTHON, PANDAS & BUSINESS INTELLIGENCE (MONTHS 3-5, WEEKS 11-20)

Master Python for data analysis, pandas, data cleaning, and exploratory data analysis.

Month 4: Pandas & Data Manipulation (Weeks 11-14)

Pandas Fundamentals

Topics:
- Pandas Series and DataFrames
- Creating DataFrames from various sources
- Reading CSV, Excel, JSON files
- DataFrame indexing and selection
- loc and iloc for data access
- Column operations and manipulation
- Adding and removing columns
- Data type conversions
- Sorting and ranking data
- Handling duplicate data
- Reset and set index
- DataFrame information and summary

Projects built:
- Data loader utility
- DataFrame explorer tool
- Data profiling system

Practice: Complete 60 pandas exercises

Data Cleaning with Pandas

Topics:
- Identifying missing data patterns
- Handling missing values strategies
- fillna, dropna, interpolate methods
- Data type issues and conversions
- String cleaning and standardization
- Regular expressions for data cleaning
- Handling outliers detection and treatment
- Data validation techniques
- Dealing with inconsistent data
- Date and time parsing
- Encoding categorical variables
- Data quality metrics

Projects built:
- Data cleaning pipeline
- Quality assurance tool
- Automated data validator

Practice: Clean 25 messy datasets

Data Transformation & Aggregation

Topics:
- GroupBy operations in pandas

- Aggregation functions: sum, mean, count
- Multiple aggregations with agg()
- Transform and apply functions
- Pivot tables in pandas
- Crosstab for frequency tables
- Melt and wide-to-long reshaping
- Merging and joining DataFrames
- Concatenating data
- Window functions in pandas
- Rolling statistics
- Resampling time series data

Projects built:
- Sales analytics engine
- Customer segmentation tool
- Time series processor

Practice: Perform 40 transformation tasks

Exploratory Data Analysis (EDA)

Topics:
- EDA methodology and workflow
- Univariate analysis techniques
- Bivariate analysis methods
- Multivariate exploration
- Distribution analysis
- Correlation analysis
- Statistical testing in Python
- Feature engineering basics
- Dimensionality reduction intro
- Automated EDA tools
- Creating EDA reports
- EDA best practices

Projects built:
- Automated EDA pipeline
- Data insights generator
- Statistical report builder

Practice: Complete EDA on 20 datasets

Month 5: Data Visualization & Storytelling (Weeks 15-17)

Matplotlib & Seaborn

Topics:
- Matplotlib basics and figure structure
- Line plots, scatter plots, bar charts
- Customizing plots: colors, labels, legends
- Subplots and figure layouts
- Saving and exporting figures
- Seaborn for statistical visualization
- Distribution plots: histograms, KDE, box plots
- Categorical plots: bar, count, violin
- Relationship plots: scatter, line, regression
- Heatmaps and correlation matrices
- Pair plots for multivariate analysis
- Style and theme customization

Projects built:
- Visualization library
- Statistical plot generator
- Report automation system

Practice: Create 50 visualizations

Business Intelligence Tools

Topics:
- Introduction to Tableau
- Connecting to data sources in Tableau
- Creating worksheets and dashboards
- Calculated fields and parameters
- Filters and hierarchies
- Tableau story creation
- Power BI fundamentals
- Data modeling in Power BI
- DAX formulas for calculations
- Power BI service and sharing
- Google Data Studio basics
- Choosing the right BI tool

Projects built:
- Tableau executive dashboard
- Power BI sales report
- Cross-platform BI solution

Practice: Create 15 BI dashboards

Data Storytelling & Presentation

Topics:
- Principles of data storytelling
- Narrative structure for data

- Choosing the right visualization
- Color theory and design principles
- Minimalism and clarity
- Annotation and highlighting insights
- Creating compelling presentations
- Executive communication strategies
- Handling questions and objections
- Written reports and documentation
- Video presentations of data
- Stakeholder management

Projects built:
- Data story presentation
- Executive briefing deck
- Insight communication toolkit

Practice: Present 10 data stories

Month 6: Machine Learning & Automation for Analysts (Weeks 18-20)

Machine Learning Fundamentals

Topics:
- Introduction to machine learning
- Supervised vs unsupervised learning
- Training and test data splitting
- Feature selection and engineering
- Scikit-learn library basics
- Linear regression implementation
- Logistic regression for classification
- Decision trees and random forests
- Model evaluation metrics
- Cross-validation techniques
- Overfitting and underfitting
- Hyperparameter tuning basics

Projects built:
- Sales prediction model
- Customer churn predictor
- Classification system

Practice: Build 20 ML models

Clustering & Segmentation

Topics:
- K-means clustering algorithm
- Hierarchical clustering
- DBSCAN for density-based clustering
- Determining optimal clusters
- Customer segmentation strategies
- Market segmentation analysis
- Behavioral segmentation
- Dimensionality reduction with PCA
- t-SNE for visualization
- Anomaly detection methods
- Outlier detection algorithms
- Practical segmentation applications

Projects built:
- Customer segmentation engine
- Anomaly detection system
- Market analysis tool

Practice: Perform 15 clustering analyses

APIs & Automation

Topics:
- Working with APIs in Python
- RESTful API concepts
- Authentication and API keys
- Requests library for API calls
- JSON data handling
- Web scraping with BeautifulSoup
- Automating data collection
- Scheduling scripts with cron/Task Scheduler
- Email automation for reports
- Cloud functions for automation
- ETL pipeline creation
- Real-time data processing basics

Projects built:
- API data collector
- Automated reporting system
- ETL pipeline

Practice: Build 10 automation scripts

Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 3: INDUSTRY ANALYTICS & THE WORKING ANALYST (MONTHS 5-7, WEEKS 21-30)

Master industry-specific analytics, cloud platforms, and prepare for professional roles.

Month 7: Industry-Specific Analytics (Weeks 21-25)
Financial Analytics
Topics:
- Financial statements analysis
- Ratio analysis and metrics
- Cash flow analysis
- Working capital management
- Investment analysis metrics
- Risk metrics: VaR, Sharpe ratio
- Portfolio analysis basics
- Credit risk scoring
- Fraud detection techniques
- Time value of money calculations
- Financial forecasting models
- Regulatory reporting basics
Projects built:
- Financial dashboard
- Risk assessment model
- Fraud detection system
Practice: Analyze 15 financial datasets
Healthcare Analytics
Topics:
- Healthcare data types and standards
- Patient data analysis
- Clinical trial analysis
- Healthcare KPIs and metrics
- Disease prediction models
- Patient readmission analysis
- Healthcare cost analysis
- Drug effectiveness studies
- Population health management
- HIPAA compliance basics
- Medical coding systems
- Healthcare dashboards
Projects built:
- Patient analytics dashboard
- Readmission predictor
- Healthcare KPI tracker
Practice: Work with 10 healthcare datasets
Retail & E-commerce Analytics
Topics:
- Retail metrics and KPIs
- Sales performance analysis
- Inventory analytics
- Customer basket analysis
- Product recommendation systems
- Price optimization strategies
- Demand forecasting
- Customer journey mapping
- Conversion rate optimization
- Return analysis
- Store performance comparison
- Omnichannel analytics
Projects built:
- Retail analytics platform
- Recommendation engine
- Inventory optimizer
Practice: Analyze 20 retail scenarios
Supply Chain Analytics
Topics:
- Supply chain metrics and KPIs
- Demand planning and forecasting
- Inventory optimization
- Supplier performance analysis
- Transportation analytics
- Warehouse efficiency metrics
- Order fulfillment analysis
- Supply chain risk assessment
- Network optimization
- Logistics analytics
- Procurement analytics
- Sustainability metrics
Projects built:
- Supply chain dashboard
- Demand forecasting model
- Route optimization tool

```
    Practice: Optimize 10 supply chain processes
  HR & People Analytics
    Topics:
      - HR metrics and KPIs
      - Employee turnover analysis
      - Recruitment analytics
      - Performance management metrics
      - Compensation analysis
      - Diversity and inclusion metrics
      - Employee engagement analysis
      - Workforce planning
      - Talent analytics
      - Learning and development metrics
      - Succession planning analytics
      - HR dashboard creation
    Projects built:
      - HR analytics dashboard
      - Turnover prediction model
      - Compensation analyzer
    Practice: Analyze 15 HR scenarios

Month 8: Cloud Analytics & Your Portfolio (Weeks 26-27)
  Cloud Platforms for Analytics
    Topics:
      - Cloud computing fundamentals
      - AWS for data analytics
      - S3 for data storage
      - Athena for SQL queries
      - Redshift data warehouse
      - Google Cloud Platform basics
      - BigQuery for analytics
      - Azure data services
      - Databricks introduction
      - Snowflake basics
      - Cloud cost optimization
      - Security in cloud analytics
    Projects built:
      - Cloud migration project
      - BigQuery analytics pipeline
      - Multi-cloud solution
    Practice: Deploy 10 cloud analytics solutions
  Portfolio Development
    Topics:
      - Building an analytics portfolio
      - GitHub for data analysts
      - Creating compelling case studies
      - Documentation best practices
      - Jupyter notebook presentation
      - Blog writing for analysts
      - Personal website creation
      - Kaggle competitions
      - Contributing to open source
      - Networking strategies
      - Building online presence
    Projects built:
      - Portfolio website
      - 5 detailed case studies
      - GitHub repository organization
    Practice: Complete portfolio preparation

Month 9: Advanced Methods, Engineering & Strategy (Weeks 28-30)
  Advanced Statistical Methods
    Topics:
      - Bayesian statistics introduction
      - Time series advanced techniques
      - ARIMA models
      - Survival analysis
      - Factor analysis
      - Structural equation modeling
      - Monte Carlo simulations
      - Bootstrapping methods
      - Non-parametric tests
      - Multivariate analysis
      - Experimental design
      - Causal inference basics
    Projects built:
      - Advanced statistical toolkit
      - Simulation framework
      - Causal analysis project
```

```
      Practice: Implement 10 advanced methods
    Data Engineering Basics
      Topics:
        - Data pipeline architecture
        - ETL vs ELT strategies
        - Apache Airflow basics
        - Data quality frameworks
        - Data governance principles
        - Master data management
        - Data cataloging
        - Metadata management
        - Data lineage tracking
        - DataOps practices
        - CI/CD for data pipelines
        - Monitoring and alerting
      Projects built:
        - ETL pipeline
        - Data quality monitor
        - Automated data pipeline
      Practice: Build 5 data pipelines
    Business Strategy & Analytics
      Topics:
        - Strategic analytics frameworks
        - Competitive intelligence
        - Market analysis techniques
        - Customer analytics strategy
        - Product analytics
        - Growth analytics
        - Pricing analytics
        - Risk analytics
        - Performance management
        - Balanced scorecard
        - OKR tracking and analysis
        - Executive advisory skills
      Projects built:
        - Strategy dashboard
        - Competitive analysis report
        - Growth model
      Practice: Develop 5 strategic analyses
      Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

----------------------------------------------------------------
PHASE 4: THE FINALE (MONTH 8, WEEKS 31-35)
----------------------------------------------------------------
The capstone month: spec, build, ship and defend, then the final exam that decides the
certificate.

  Month 8: Capstone and the Final Exam (Weeks 31-35)
    Capstone Spec and Setup
      Topics:
        - Choosing the capstone: a complete analyst project: a real business dataset cleaned, analysed
          and delivered as a dashboard with a recommendation deck
        - A one-page spec: scope, milestones, stack
        - Repository and skeleton ready on day one
        - Instructor sign-off before the build begins
      Projects built:
        - Approved capstone spec plus the running skeleton
      Practice: Pitch the spec to the batch and absorb one hard question
    Capstone Build
      Topics:
        - Two focused weeks in vertical slices
        - Instructor checkpoints and honest scope cuts
        - Using the whole course: Excel judgment, SQL, Pandas, visualization, storytelling and the
          statistics that keep it honest
        - Daily commits and an always-working build
      Projects built:
        - The capstone, feature-complete with version control
      Practice: A working build at the end of every session
    Ship and Polish
      Topics:
        - The finishing pass: edge cases, errors, documentation
        - Dashboard live and deck delivered: analysis a decision-maker can actually use
        - A README worth reading and a demo worth watching
        - A full-course spiral review before the exam
      Projects built:
        - The capstone shipped, documented and rehearsed
      Practice: Two mock demos, each tighter than the last
    Demo Day and the Final Exam
```

Topics:
- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: analyst roles, business intelligence depth, or the road into data science
- Where our other courses continue the journey

Projects built:
- Demo day presentation delivered to the batch

Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Excel analyses and SQL query projects on business data
- Phase 2: Pandas cleaning pipelines, EDA reports and BI dashboards
- Phase 3: industry case analyses across five domains
- Phase 4: the portfolio capstone: dashboard plus recommendation deck

Total projects: 15+ real analyses, crowned by a portfolio capstone with dashboard and deck

SKILLS THE STUDENT WILL HAVE AT THE END

- Excel at professional depth: formulas, pivots, analysis tools and honest charts
- Statistics for analysts: distributions, inference and A/B thinking
- SQL at analyst level: joins, aggregation and window functions
- Python with Pandas: cleaning, transformation and exploratory analysis
- BI and storytelling: dashboards and decks that change decisions

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Data analyst and business analyst roles across every industry
- Business intelligence specialization: deeper Power BI and dashboard craft
- Our data science course continues into machine learning and deep learning
- Analytics engineering: the SQL and pipeline road

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 3-5 LPA (Junior Analyst); Rs 5-8 LPA (Data Analyst)
Growing roles: Rs 8-15 LPA (Senior Analyst)
Other roles: Rs 15-25+ LPA (Lead/Manager)
Freelance: Rs 500-2,000/hour
International: $50k-100k+ USD

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this data analysis course cover?
A: The full analyst toolkit in working order: Excel to professional depth with the statistics that keep analysis honest, SQL including window functions, Python with Pandas for cleaning and exploratory analysis, visualization, BI dashboards, data storytelling and practical machine learning for analysts, then industry applications across finance, healthcare, retail, supply chain and HR, cloud analytics platforms and advanced statistical methods. A finale month ships a portfolio capstone and sits the final exam.

Q: How long does it take to become job-ready in data analysis?
A: About seven months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with margin built in beyond that for learners who need it. Three phases build Excel with statistics and SQL, then Python with BI and storytelling, then industry applications; the finale month ships a portfolio capstone and sits the final exam.

Q: How am I assessed? Is the certificate just for attending?
A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Data analysis or data science: which course should I take?
A: Ask what you want to do daily. Analysts answer business questions: cleaning data, writing SQL, building dashboards and telling the story, employable sooner and everywhere. Data scientists build models on top of that craft. This course makes you the first; our data science course continues into the second, and the skills stack rather than compete.

Q: Do I need coding experience or strong maths?
A: Neither. The course starts in Excel, where most professionals already live, and introduces SQL and Python step by step with practice after every class. The maths is the statistics an analyst genuinely uses, distributions, inference, A/B thinking, taught in plain language on real business data.

Q: Can I try a class before enrolling?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/data-analysis-mastery-course-college
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 52 OF 137: DATA AND AI ANALYTICS FOR NON-PROGRAMMERS (EXCEL, SHEETS, POWER BI)
==========

Course link: https://learn.modernagecoders.com/courses/data-and-ai-analytics-for-non-programmers-course
Subject area: Data and AI Analytics
Designed for: Working professionals
Who can join: Working professionals and students with no programming background; spreadsheet familiarity helps but is not required
Course length: 4 months (16 weeks), joinable any month
Time needed: 2 live classes/week + weekly work on real datasets
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online data analytics for working professionals who do not code: clean and model data in Excel and Sheets, build Power BI dashboards, and use AI assistants to work faster without being misled.

In one line: Become the person in the room who can actually answer the question with data, without writing a line of code.

ABOUT THE PROGRAMME

Most people who need to work with data are not going to become programmers, and they do not need to. The person who can take a messy export, clean it, find the real pattern and present it clearly in a dashboard is enormously valuable in any team, and almost none of that requires code. What it requires is knowing how to structure data, which comparison is honest, and how to build something a decision-maker can actually read. This course teaches exactly that path, in the tools your workplace already has. You start with spreadsheets done properly, Excel and Google Sheets, including the lookup and pivot skills most people never learn. You move into cleaning and shaping real messy data, which is where most analysis actually fails. Then you build interactive dashboards in Power BI and learn to model relationships between tables so the numbers hold up under questioning. AI runs through the

whole course, honestly. Modern AI assistants can write your formulas, summarise a dataset and suggest a chart, and used well they make you considerably faster. Used badly they produce confident nonsense that you then present to your manager. So we teach both: how to use AI to accelerate the work, and how to check what it gives you before you trust it. Every module ends with real, messy datasets rather than tidy teaching examples, because tidy data is not what anybody actually receives.

WHAT MAKES THIS COURSE DIFFERENT

- No coding required at any point, and no pretending you will secretly need Python later
- Built on the tools your workplace already has: Excel, Google Sheets and Power BI
- Real messy data from week one, because cleaning is where most analysis actually fails
- AI assistants taught honestly: how to go faster with them, and how to catch the confident nonsense before you present it
- Focused on the question behind the request, since the hard part is usually deciding what to measure
- Live, small batches where your dashboard gets critiqued like a colleague would critique it

THE LEARNING PATH, STEP BY STEP

Phase 1: Spreadsheets done properly: structure, lookups, pivots and the formulas that matter
Phase 2: Cleaning and shaping messy real data, then statistics that keep you honest
Phase 3: Power BI dashboards, data modelling, AI-assisted analysis and a portfolio project

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What You Need to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: SPREADSHEETS DONE PROPERLY

The foundation almost everyone half-knows, rebuilt so the later work holds up.

Month 1: Excel and Google Sheets Foundations (Weeks 1-4)
Structuring Data So It Can Be Analysed
Topics:
- What a clean table actually looks like, and why yours probably is not one
- Rows as records, columns as fields, and one fact per cell
- Why merged cells and colour-as-data destroy analysis
- Data types, dates and the formatting traps that silently break totals
- Turning a human-formatted sheet into an analysable one
Projects built:
- Take a genuinely badly structured sheet and restructure it into a usable table
Practice: Restructure three messy real-world sheets
The Formulas That Actually Matter
Topics:
- Absolute and relative references, and why formulas break when copied
- IF, nested conditions, and when to stop nesting
- SUMIF, COUNTIF, AVERAGEIF and their multi-criteria versions
- Text and date functions for real cleanup
- Error handling so a broken cell does not poison a whole column
Projects built:
- Build a summary sheet driven entirely by formulas from a raw data tab
Practice: 20 formula tasks against a real dataset
Lookups and Joining Data
Topics:
- VLOOKUP, XLOOKUP and INDEX with MATCH, and when each is right
- Joining two tables on a common key
- Handling missing matches honestly instead of hiding them
- Duplicate keys and the quiet damage they do
- The equivalents in Google Sheets
Projects built:
- Combine three separate exports into one analysable table and reconcile the mismatches
Practice: 15 lookup and join exercises including deliberately broken keys
Assessment: A timed spreadsheet task: clean, join and summarise a real dataset
Pivot Tables and Fast Summaries
Topics:
- Pivot tables from the ground up
- Grouping, filtering and calculated fields
- Slicing by dimension to find where a number actually comes from
- Reading a pivot critically instead of accepting it

- Charts that clarify rather than decorate
Projects built:
- Answer five real business questions about one dataset using only pivots
Practice: Build pivots answering supplied questions, then explain each finding

---

PHASE 2: CLEANING REAL DATA AND ANALYSING IT HONESTLY

Where most analysis fails, and the statistics you need to avoid fooling yourself.

Month 2: Data Cleaning and Practical Statistics (Weeks 5-8)

Cleaning Messy Real Data

Topics:
- Duplicates, near-duplicates and how to decide which is real
- Missing values and the options: exclude, impute or report
- Inconsistent categories and text normalisation
- Outliers, and telling a data error from a genuine extreme
- Documenting every cleaning decision so your numbers are defensible

Projects built:
- Clean a genuinely messy dataset and write a short log of every decision made

Practice: Clean three datasets with different problems

Power Query for Repeatable Cleaning

Topics:
- Why manual cleaning does not survive next month's file
- Power Query basics: importing, transforming and loading
- Building a repeatable cleaning pipeline
- Unpivoting and reshaping data
- Refreshing when new data arrives, with no rework

Projects built:
- Build a Power Query pipeline that cleans a monthly export automatically

Practice: Convert last week's manual cleaning into a repeatable query

Statistics That Keep You Honest

Topics:
- Mean, median and when the average actively misleads
- Spread and why two datasets with the same average differ completely
- Percentages, percentage points and the errors that get presented in meetings
- Correlation is not causation, with real examples
- Sample size and why small numbers should not drive big decisions

Projects built:
- Take a real claim from a report and check whether the data actually supports it

Practice: Critique supplied analyses and identify what each gets wrong

Assessment: A written critique of a deliberately misleading analysis

Charts That Do Not Lie

Topics:
- Choosing the chart from the question, not from preference
- Truncated axes and the other ways charts mislead
- Comparing like with like
- Labelling so a chart survives being forwarded without you
- Removing everything that is not carrying information

Projects built:
- Redesign three bad charts and explain what each change fixes

Practice: Build charts for supplied questions and defend each choice

---

PHASE 3: POWER BI, AI-ASSISTED ANALYSIS AND YOUR PORTFOLIO

Turning analysis into something a decision-maker uses, with AI as an accelerator rather than an oracle.

Month 3: Power BI and Data Modelling (Weeks 9-12)

Power BI Foundations

Topics:
- Connecting to data and shaping it on import
- The difference between a report and a dashboard
- Core visuals and when each is appropriate
- Filters, slicers and cross-filtering
- Publishing and sharing safely

Projects built:
- Build your first working Power BI report from a real dataset

Practice: Rebuild an existing spreadsheet report as a Power BI report

Data Modelling and Relationships

Topics:
- Fact and dimension tables in plain language
- Relationships and why a star schema keeps numbers correct
- The problems caused by one flat table
- Date tables and time intelligence
- Diagnosing a wrong total caused by a bad relationship

Projects built:
- Model a multi-table dataset properly and prove the totals reconcile

Practice: Fix three broken data models
DAX Measures Without Fear
Topics:
- Calculated columns versus measures, and why the difference matters
- Core DAX: SUM, AVERAGE, COUNTROWS, CALCULATE
- Filter context in plain terms
- Time comparisons: month on month, year on year
- Writing measures you can still understand next quarter
Projects built:
- Add a full measure set to your model including period comparisons
Practice: 15 DAX exercises building from simple to filtered
Assessment: A timed dashboard task assessed on correctness and clarity
Designing Dashboards People Actually Use
Topics:
- Starting from the decision the dashboard should support
- Layout, hierarchy and what belongs above the fold
- Restraint: fewer visuals, better chosen
- Making it self-explanatory without you presenting it
- Testing it on someone who has never seen it
Projects built:
- Redesign your dashboard after watching someone else try to use it
Practice: Peer usability reviews of each other's dashboards

Month 4: AI-Assisted Analysis and the Capstone (Weeks 13-16)
Using AI Assistants Well
Topics:
- What AI assistants are genuinely good at in analytics work
- Getting formulas, DAX and cleaning steps written for you
- Asking for an explanation of a formula you inherited
- Writing a prompt that includes the context the model needs
- Speed gains that are real, and the ones that are illusory
Projects built:
- Solve a full analysis task using AI assistance and log where it helped and where it did not
Practice: Complete supplied tasks with AI, then verify every output
Checking What AI Gives You
Topics:
- Why an AI can produce a confident, plausible, wrong formula
- Verifying a result against a known subtotal before trusting it
- Spot-checking summaries against the raw data
- Never pasting confidential data into a tool that is not approved
- Being able to explain any number you present, whatever produced it
Projects built:
- Audit a set of AI-produced analyses and find the errors deliberately planted in them
Practice: Verification drills on AI output
Assessment: A verification test: find every error in an AI-assisted analysis
Capstone Build
Topics:
- Choosing a question that matters to a real audience
- Sourcing and cleaning the data end to end
- Modelling, measures and dashboard build
- Documenting assumptions and limitations honestly
- Preparing to defend every number
Projects built:
- Build your full capstone: question, clean data, model, dashboard, findings
Practice: Capstone checkpoints with review
Presenting and Your Portfolio
Topics:
- Leading with the finding, not the method
- Telling the story a decision-maker needs in two minutes
- Answering the hard question about your data honestly
- Packaging the project for a portfolio or interview
- What to say when the data does not support the hoped-for answer
Projects built:
- Present your capstone and take questions from the group
Practice: Rehearsed presentations with critique
Assessment: Final capstone presentation assessed on clarity, correctness and honesty

HOW A TYPICAL WEEK IS SPLIT

Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

  - Business analyst, operations analyst or MIS roles
  - Business intelligence and reporting work
  - A stronger position in your current job as the person who can answer with data
  - A portfolio project that demonstrates the whole workflow end to end
  - A natural later step into Python or SQL if you ever want it

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: Do I really not need to learn programming?
  A: Correct. Everything in this course is done in Excel, Google Sheets and Power BI, all of which are built for exactly this work. Power Query and DAX are used, and although DAX is a formula language it is far closer to spreadsheet formulas than to programming. If you later want Python or SQL, this course leaves you well placed, but nothing here depends on it.

  Q: How am I assessed, and is the certificate just for showing up?
  A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final project defended live, plus a written assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

  Q: How is AI actually used in the course?
  A: Honestly, and in both directions. AI assistants are genuinely good at writing formulas, drafting DAX, explaining an inherited spreadsheet and suggesting cleaning steps, and used well they make you noticeably faster. They are also very capable of producing a confident wrong answer. So we teach you to use them and to verify everything they produce against known subtotals before you present it.

  Q: Will I have something to show at the end?
  A: Yes. The final month is a capstone: you choose a real question, source and clean the data, model it, build the dashboard and present the findings, including your assumptions and limitations. It is designed to be the project you show in an interview, because it demonstrates the entire workflow rather than one tool.

  Q: I have tried a data course before and gave up. Why would this be different?
  A: Most data courses lose people because they open with programming or with tidy academic datasets that behave nothing like real ones. This one starts in the tools you already have and works with genuinely messy data from the first weeks, because cleaning is the part where real analysis actually fails. It is live and small, so you get unstuck in class rather than quietly falling behind.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/data-and-ai-analytics-for-non-programmers-course
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
COURSE 53 OF 137: DATA SCIENCE COURSE: PYTHON, ML & DEEP LEARNING
==

Course link: https://learn.modernagecoders.com/courses/data-science-complete-masterclass-college
Subject area: Data Science
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Data science course for college students and professionals: Python, statistics and SQL, machine learning done honestly, then deep learning and NLP, ending in a deployed capstone, in 10-12 months.

  In one line: The full pipeline, honestly taught: Python, statistics and SQL, machine learning that holds up, then deep learning, ending with a capstone you deploy.

ABOUT THE PROGRAMME

Data science hiring has grown sceptical: everyone lists the same libraries, so interviews now dig for the difference between importing scikit-learn and understanding what the model just did. This live online masterclass builds that difference in order: Python with NumPy and Pandas properly, statistics that survive contact with real data, SQL and EDA as daily craft, then machine learning done honestly, regression to ensembles with evaluation you can defend, real end-to-end projects and Kaggle practice, then the deep learning layer: neural networks, CNNs, RNNs and NLP, applied computer vision and a research-paper implementation to prove you can read the field. A finale month ships a deployed capstone and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- Statistics before models: evaluation and honesty are the actual job
- End-to-end from mid-course: raw data to working model, not notebook fragments
- Kaggle as training ground, framed honestly: practice, not trophies
- A research-paper implementation: proof you can read the field, not just tutorials
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Months 1-3): Python, NumPy, Pandas, statistics, SQL and EDA
Phase 2: Machine learning (Months 4-6): supervised to unsupervised, real projects, honest evaluation
Phase 3: Deep learning (Months 7-9): neural networks, CNNs, RNNs, NLP and applied vision
Phase 4: The finale (Month 10): a deployed capstone and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Everything is taught from the first line
Equipment: Any computer with a stable internet connection; every tool used is free
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

College students: Students targeting data science and analytics placements
Career switchers: Professionals moving into data roles with a structured, honest path
Analysts upskilling: Analysts ready to add ML and deep learning to their toolkit
Engineers: Developers who want the data half of modern software
Self taught: Learners with scattered notebook experience that needs a spine

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS OF DATA SCIENCE (MONTHS 1-3, WEEKS 1-13)

Build rock-solid foundations in Python programming, statistics, data analysis, and SQL that form the backbone of data science.

Months 1-2: Python & Statistical Foundations (Weeks 1-8)

Python Programming for Data Science

Topics:
- Why Python for Data Science?
- Setting up the data science environment (Anaconda, Jupyter)
- Python basics: variables, data types, operators
- Data structures: lists, tuples, sets, dictionaries
- Control flow: if-else, loops (for, while)
- Functions and lambda expressions
- Object-oriented programming basics
- File handling and exception management
- Python libraries ecosystem for data science
- Git and version control for data science projects
- Jupyter notebooks best practices
- Google Colab for cloud computing

Projects built:
- Python fundamentals practice notebook
- Building a data science toolkit with functions
- Automated data processing script

Practice: Complete 100 Python coding challenges

NumPy for Numerical Computing

Topics:
- Introduction to NumPy and its importance
- NumPy arrays: creation and properties

- Array indexing, slicing, and reshaping
- Broadcasting and vectorization
- Mathematical operations on arrays
- Statistical functions in NumPy
- Linear algebra operations
- Random number generation
- Array manipulation techniques
- Performance optimization with NumPy
- Memory management in NumPy
- NumPy for image processing basics

Projects built:
- Image manipulation with NumPy arrays
- Statistical calculator using NumPy
- Matrix operations library
- Performance comparison: NumPy vs pure Python

Practice: Solve 50 NumPy exercises

Data Manipulation with Pandas

Topics:
- Introduction to Pandas DataFrames and Series
- Reading data from various formats (CSV, Excel, JSON, SQL)
- Data inspection and understanding
- Indexing and selecting data (.loc, .iloc)
- Filtering and boolean indexing
- Handling missing data strategies
- Data transformation and feature creation
- Groupby operations and aggregations
- Merging, joining, and concatenating data
- Pivot tables and cross-tabulation
- Time series data handling
- Performance optimization in Pandas

Projects built:
- Complete EDA on retail sales dataset
- Data cleaning pipeline for messy dataset
- Time series analysis of stock prices
- Customer behavior analysis

Practice: Clean and analyze 10 different datasets

Statistical Foundations for Data Science

Topics:
- Descriptive statistics: measures of central tendency and spread
- Probability theory and distributions
- Normal distribution and Central Limit Theorem
- Sampling and sampling distributions
- Hypothesis testing fundamentals
- T-tests, Chi-square tests, ANOVA
- P-values and statistical significance
- Confidence intervals
- Correlation and covariance
- Type I and Type II errors
- Power analysis and sample size determination
- Bayesian statistics introduction

Projects built:
- Statistical analysis of A/B test results
- Hypothesis testing on real datasets
- Building a statistical testing framework
- Correlation analysis of multiple variables

Practice: Perform statistical tests on 15 different scenarios

Month 3: Data Visualization & SQL (Weeks 9-13)

Data Visualization Mastery

Topics:
- Principles of effective data visualization
- Matplotlib: creating publication-quality plots
- Seaborn for statistical visualizations
- Plotly for interactive visualizations
- Creating dashboards with Streamlit/Dash
- Geospatial visualization with Folium
- Time series visualization techniques
- Heatmaps and correlation matrices
- Network graphs and tree visualizations
- 3D visualizations
- Animation in data visualization
- Best practices for different chart types

Projects built:
- Interactive COVID-19 dashboard
- Sales performance dashboard
- Geospatial analysis of crime data
- Animated visualization of algorithm performance

Practice: Create 30 different types of visualizations

SQL for Data Scientists

Topics:
- Relational database concepts
- SQL fundamentals: SELECT, WHERE, ORDER BY
- Aggregate functions and GROUP BY
- Joins: INNER, LEFT, RIGHT, FULL OUTER
- Subqueries and CTEs
- Window functions for advanced analytics
- Database design and normalization
- Performance optimization and indexing
- NoSQL databases introduction (MongoDB)
- Connecting Python to databases
- SQL vs Pandas: when to use what
- Big data SQL: Spark SQL, Presto

Projects built:
- Building a data warehouse schema
- Complex business queries with window functions
- Python-SQL integration project
- Database performance optimization

Practice: Write 100 SQL queries of increasing complexity

Exploratory Data Analysis (EDA) & Feature Engineering

Topics:
- EDA methodology and workflow
- Univariate, bivariate, and multivariate analysis
- Identifying patterns and anomalies
- Feature engineering techniques
- Creating polynomial and interaction features
- Binning and discretization
- Encoding categorical variables
- Feature scaling and normalization
- Handling imbalanced datasets
- Feature selection methods
- Dimensionality reduction (PCA introduction)
- Domain-specific feature engineering

Projects built:
- PHASE 1 CAPSTONE: End-to-End EDA Project
- Complete EDA and feature engineering on Kaggle dataset
- Build reusable feature engineering pipeline
- Create automated EDA report generator

Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 2: MACHINE LEARNING MASTERY (MONTHS 4-6, WEEKS 14-26)

Master classical machine learning algorithms, model evaluation, and participate in Kaggle competitions.

Months 4-5: Supervised Learning (Weeks 14-22)

Regression Algorithms

Topics:
- Machine learning workflow and pipeline
- Linear regression from scratch
- Multiple linear regression
- Polynomial regression
- Ridge and Lasso regression
- Elastic Net
- Logistic regression for classification
- Evaluation metrics for regression
- Cross-validation strategies
- Bias-variance tradeoff
- Regularization techniques
- Feature importance analysis

Projects built:
- House price prediction model
- Sales forecasting system
- Customer lifetime value prediction
- Build regression library from scratch

Practice: Implement 5 regression algorithms from scratch

Classification Algorithms

Topics:
- k-Nearest Neighbors (KNN)
- Decision trees and pruning
- Random Forests
- Gradient Boosting (XGBoost, LightGBM, CatBoost)
- Support Vector Machines (SVM)
- Naive Bayes classifier
- Evaluation metrics for classification
- ROC curves and AUC
- Handling imbalanced classes

- Multi-class classification strategies
- Ensemble methods
- Stacking and blending

Projects built:
- Credit default prediction
- Customer churn prediction
- Disease diagnosis system
- Fraud detection model

Practice: Build 10 classification models on different datasets

Advanced Machine Learning Techniques

Topics:
- Hyperparameter tuning: Grid Search, Random Search, Bayesian Optimization
- AutoML tools and techniques
- Feature selection algorithms
- Dimensionality reduction: PCA, t-SNE, UMAP
- Anomaly detection algorithms
- Time series forecasting: ARIMA, Prophet
- Recommendation systems: collaborative filtering, content-based
- Model interpretation: SHAP, LIME
- Handling missing data: advanced imputation
- Semi-supervised learning
- Active learning strategies
- Transfer learning in classical ML

Projects built:
- Anomaly detection in network traffic
- Movie recommendation system
- Sales forecasting with Prophet
- Model interpretation dashboard

Practice: Apply advanced techniques to improve previous models

Unsupervised Learning

Topics:
- Clustering algorithms: K-Means, DBSCAN, Hierarchical
- Gaussian Mixture Models
- Clustering evaluation metrics
- Market basket analysis
- Association rules: Apriori, FP-Growth
- Topic modeling: LDA, NMF
- Autoencoders for dimensionality reduction
- Self-organizing maps
- Isolation Forest for anomaly detection
- Applications in customer segmentation
- Image compression with clustering
- Text clustering and document similarity

Projects built:
- Customer segmentation analysis
- Market basket analysis for retail
- Document clustering system
- Anomaly detection in IoT data

Practice: Apply clustering to 5 different domains

Kaggle Competition Participation

Topics:
- Understanding Kaggle competitions
- Competition strategies and workflow
- Data augmentation techniques
- Advanced feature engineering
- Ensemble strategies for competitions
- Learning from kernels and discussions
- Leaderboard probing
- Cross-validation strategies
- Submission strategies
- Post-competition analysis

Projects built:
- Participate in current Kaggle competition
- Achieve top 50% in beginner competition
- Write competition solution walkthrough
- Build reusable competition pipeline

Practice: Complete 3 past Kaggle competitions

Month 6: Real-world ML Projects (Weeks 23-26)

End-to-End ML Project Development

Topics:
- Problem formulation and scoping
- Data collection strategies
- Building data pipelines
- Feature store design
- Model versioning and experiment tracking
- A/B testing for ML models
- Model monitoring and drift detection
- Building ML APIs with Flask/FastAPI

- Containerization with Docker
- Cloud deployment (AWS, GCP, Azure)
- Cost optimization for ML
- Documentation and reporting

Projects built:
- Build complete ML pipeline from scratch
- Deploy model as REST API
- Create model monitoring dashboard
- Implement A/B testing framework

Practice: Deploy 3 models to production

Industry-Specific Applications

Topics:
- Healthcare: disease prediction, drug discovery
- Finance: risk assessment, algorithmic trading
- Retail: demand forecasting, pricing optimization
- Marketing: customer segmentation, campaign optimization
- Manufacturing: predictive maintenance, quality control
- Transportation: route optimization, demand prediction
- Energy: consumption forecasting, grid optimization
- Agriculture: crop yield prediction, pest detection
- Real estate: price prediction, investment analysis
- Sports analytics: player performance, game prediction

Projects built:
- Choose 2 industries and build domain-specific models
- Create industry-specific feature engineering
- Build domain knowledge documentation

Practice: Analyze datasets from 5 different industries

ML Engineering Best Practices

Topics:
- Code organization for ML projects
- Testing ML code: unit tests, integration tests
- Continuous Integration/Continuous Deployment (CI/CD)
- MLflow for experiment tracking
- DVC for data versioning
- Weights & Biases for experiment management
- Model registry and governance
- Feature stores: Feast, Tecton
- Reproducibility in ML
- Debugging ML models
- Performance optimization
- Security in ML systems

Projects built:
- Set up MLOps pipeline
- Implement CI/CD for ML project
- Build feature store
- Create model governance framework

Practice: Refactor all projects with engineering best practices

Phase 2 Capstone Project

Topics:
- Complex problem solving
- Multi-model systems
- Production-ready code
- Professional documentation
- Stakeholder presentation

Projects built:
- PHASE 2 CAPSTONE: Production ML System
- Build end-to-end ML system for real business problem
- Include data pipeline, multiple models, API, monitoring
- Deploy to cloud with full MLOps pipeline

Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 3: DEEP LEARNING & ADVANCED AI (MONTHS 7-9, WEEKS 27-39)

Master deep learning, computer vision, NLP, and cutting-edge AI techniques.

Months 7-8: Deep Learning Fundamentals (Weeks 27-35)

Neural Network Fundamentals

Topics:
- Introduction to deep learning
- Perceptron and multi-layer perceptrons
- Backpropagation algorithm
- Activation functions
- Weight initialization strategies
- Gradient descent variations
- Learning rate scheduling
- Batch normalization
- Dropout and regularization

    - Building neural networks from scratch
    - Introduction to TensorFlow and Keras
    - PyTorch fundamentals
  Projects built:
    - Neural network from scratch in NumPy
    - MNIST digit classification
    - Binary classification with deep learning
    - Regression with neural networks
  Practice: Implement 5 different neural network architectures
Convolutional Neural Networks (CNNs)
  Topics:
    - CNN architecture and intuition
    - Convolution and pooling layers
    - Popular architectures: LeNet, AlexNet, VGG, ResNet
    - Transfer learning and fine-tuning
    - Data augmentation for images
    - Object detection: YOLO, R-CNN
    - Image segmentation: U-Net
    - Face recognition systems
    - Style transfer
    - Generative models: VAE, GAN basics
    - CNN applications beyond images
    - Deploying CNN models
  Projects built:
    - Image classification on custom dataset
    - Object detection system
    - Face recognition application
    - Image segmentation for medical images
  Practice: Build 5 computer vision applications
Recurrent Neural Networks (RNNs)
  Topics:
    - RNN architecture and applications
    - Vanishing gradient problem
    - LSTM and GRU architectures
    - Bidirectional RNNs
    - Sequence-to-sequence models
    - Attention mechanism
    - Time series prediction with RNNs
    - Text generation
    - Sentiment analysis
    - Named Entity Recognition
    - Machine translation basics
    - Speech recognition introduction
  Projects built:
    - Stock price prediction with LSTM
    - Text generation model
    - Sentiment analysis system
    - Time series anomaly detection
  Practice: Implement 5 sequence modeling tasks
Natural Language Processing
  Topics:
    - Text preprocessing and tokenization
    - Word embeddings: Word2Vec, GloVe
    - Text classification techniques
    - Named Entity Recognition (NER)
    - Part-of-speech tagging
    - Topic modeling with deep learning
    - Transformer architecture
    - BERT and GPT models
    - Fine-tuning pre-trained models
    - Question answering systems
    - Text summarization
    - Chatbot development
  Projects built:
    - Build custom chatbot
    - News article classifier
    - Question answering system
    - Text summarization tool
  Practice: Complete 5 NLP projects using transformers
Advanced Deep Learning Topics
  Topics:
    - Autoencoders and variational autoencoders
    - Generative Adversarial Networks (GANs)
    - Deep Reinforcement Learning basics
    - Graph Neural Networks introduction
    - Meta-learning and few-shot learning
    - Neural Architecture Search
    - Model compression and quantization
    - Edge deployment of deep learning

- Adversarial machine learning
- Explainable AI for deep learning
- Multi-modal learning
- Self-supervised learning

Projects built:
- Build and train a GAN
- Implement autoencoder for anomaly detection
- Model compression project
- Multi-modal classification system

Practice: Experiment with 3 cutting-edge techniques

Month 9: Applied Vision, NLP & Research Practice (Weeks 36-39)

Computer Vision Projects

Topics:
- Medical image analysis
- Autonomous driving perception
- Facial emotion recognition
- Pose estimation
- Video analysis and action recognition
- 3D computer vision
- Document analysis and OCR
- Satellite image analysis
- Real-time vision systems
- Mobile and edge deployment
- Vision transformers
- Self-supervised learning in vision

Projects built:
- Medical diagnosis from X-rays
- Real-time object tracking system
- Document scanner with OCR
- Pose estimation application

Practice: Build portfolio of 5 vision projects

Advanced NLP Applications

Topics:
- Large Language Models (LLMs)
- Prompt engineering
- Fine-tuning LLMs
- Retrieval Augmented Generation (RAG)
- Building with LangChain
- Vector databases
- Semantic search
- Document intelligence
- Code generation with AI
- Multimodal models
- Ethical considerations in NLP
- Production NLP systems

Projects built:
- RAG-based question answering system
- Semantic search engine
- AI writing assistant
- Code generation tool

Practice: Build 5 LLM-powered applications

Research Paper Implementation

Topics:
- Reading and understanding research papers
- Reproducing paper results
- Implementing novel architectures
- Benchmarking and evaluation
- Writing technical reports
- Contributing to open source
- Publishing your own research
- Staying updated with latest research
- Research tools and resources
- Building research portfolio
- Collaboration in research
- Ethics in AI research

Projects built:
- Implement 2 recent research papers
- Write detailed implementation report
- Open source your implementations
- Create tutorial for community

Practice: Read and summarize 20 research papers

Phase 3 Capstone Project

Topics:
- Complex deep learning systems
- Multi-model architectures
- Production deployment
- Performance optimization
- Comprehensive evaluation

Projects built:
- PHASE 3 CAPSTONE: State-of-the-art AI System
- Build cutting-edge AI application
- Combine multiple deep learning techniques
- Deploy with full production pipeline
- Achieve competitive performance metrics

Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)

The capstone month: spec, build, ship and defend, then the final exam that decides the certificate.

Month 10: Capstone and the Final Exam (Weeks 40-44)

Capstone Spec and Setup

Topics:
- Choosing the capstone: an end-to-end data science project: raw data to a deployed model with an evaluation you can defend
- A one-page spec: scope, milestones, stack
- Repository and skeleton ready on day one
- Instructor sign-off before the build begins

Projects built:
- Approved capstone spec plus the running skeleton

Practice: Pitch the spec to the batch and absorb one hard question

Capstone Build

Topics:
- Two focused weeks in vertical slices
- Instructor checkpoints and honest scope cuts
- Using the whole course: the statistics spine, ML judgment, deep learning where it earns its place, and honest evaluation
- Daily commits and an always-working build

Projects built:
- The capstone, feature-complete with version control

Practice: A working build at the end of every session

Ship and Polish

Topics:
- The finishing pass: edge cases, errors, documentation
- Deployed and documented: the model serving predictions outside the notebook
- A README worth reading and a demo worth watching
- A full-course spiral review before the exam

Projects built:
- The capstone shipped, documented and rehearsed

Practice: Two mock demos, each tighter than the last

Demo Day and the Final Exam

Topics:
- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: data science roles, ML engineering depth, or the analytics leadership track
- Where our other courses continue the journey

Projects built:
- Demo day presentation delivered to the batch

Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: EDA and SQL projects on real, messy datasets
- Phase 2: end-to-end ML projects and Kaggle practice with honest write-ups
- Phase 3: deep learning builds: vision, NLP and a research-paper implementation
- Phase 4: the deployed capstone, defended at demo day

Total projects: 15+ real projects, crowned by a deployed end-to-end capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Python for data: NumPy, Pandas and visualization as daily craft
- Statistics that survive real data: inference, uncertainty, honest claims
- Machine learning with judgment: model choice, tuning and defensible evaluation
- Deep learning: neural networks, CNNs, RNNs and NLP in practice
- The pipeline craft: SQL, EDA, feature engineering and deployment

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor

Homework: A problem set after every class, and a mixed-review assignment at the end of every month

  Practice: 4-6 hours of building between classes
  Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

  Doubt support: WhatsApp doubt support between classes
  Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the
    plan for the wobble

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short
    and there is focused revision and a free retest: the certificate certifies knowledge, not
    attendance

WHERE THIS CAN LEAD

  - Data scientist and ML roles: the full-pipeline profile fits the screen
  - Our AI/ML engineering course deepens the modelling road
  - Analytics engineering: SQL and pipeline craft compound here
  - The data analytics mathematics course strengthens the statistical spine

EARNING POTENTIAL (share only if asked, and never promise it)

  Market context: The ranges below are general market salary bands for these roles in India and
    abroad, drawn from public industry data. They are shown for career context only and are not a
    promise or guarantee of income. Actual pay depends on your skills, experience, location, and
    the job market.
  Growing roles: Rs 6-10 LPA (Junior Data Scientist)
  Experienced roles: Rs 10-20 LPA (Data Scientist); Rs 15-30 LPA (Senior Data Scientist); Rs
    20-50+ LPA (Lead/Principal Data Scientist)
  Freelance: Rs 3,000-10,000/hour based on expertise
  International: $80k-200k USD based on location and experience

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What does this data science course cover?
  A: The full pipeline in order: Python with NumPy and Pandas, statistics, data visualization,
  SQL and exploratory data analysis with feature engineering; then machine learning,
  regression, classification, ensembles and unsupervised methods with honest evaluation,
  end-to-end projects and Kaggle practice; then deep learning, neural networks, CNNs, RNNs,
  NLP, applied computer vision and a research-paper implementation. A finale month ships a
  deployed capstone and sits the final exam.

  Q: How long does it take to learn data science properly?
  A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two
  more months in hand for learners who need them. The three teaching phases build the skill and
  its real-world application; the finale month ships a capstone and sits the final exam.

  Q: How am I assessed? Is the certificate just for attending?
  A: No. Problem sets follow every class with written feedback, monthly reviews deliberately
  mix new material with earlier topics, each phase ends with a cumulative exam, and the course
  ends with a final exam: a practical build plus written questions from the whole syllabus.
  Passing earns the certificate; falling short earns focused revision and a free retest.

  Q: How is this different from your data analysis and AI/ML courses?
  A: Three different jobs. Data analysis is the business-facing craft: Excel, SQL, dashboards
  and clear answers to business questions. This data science course is the full modelling
  pipeline: statistics, machine learning and deep learning on top of that data craft. The AI/ML
  engineering course goes deepest into models and the systems that serve them. If you want the
  classic data scientist profile, this is the course; the other two are its neighbours.

  Q: Do I need strong maths before joining?
  A: No: school-level comfort with numbers is enough to start, and the statistics you need is
  taught inside the course, in code, on real data. What we do not do is hide the maths that
  matters: probability, distributions and evaluation metrics are taught properly because
  interviews and real work both reach for them.

  Q: Can I try a class before enrolling?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or
  message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with
  no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at
  https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor reading your actual code, never
  pre-recorded videos.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/data-science-complete-masterclass-college
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 54 OF 137: DATA SCIENCE FOR TEENS: PYTHON & YOUR FIRST ML MODELS

==========

Course link: https://learn.modernagecoders.com/courses/data-science-course-for-teens-python-data
Subject area: Data Science
Designed for: Teens
Who can join: Ages 14 to 18, basic Python required
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online data science course for teens (14-18): pandas, cleaning real datasets, matplotlib, statistics and your first ML models with scikit-learn.

In one line: Real datasets, honest charts, and a first machine learning model that the student can actually explain.

ABOUT THE PROGRAMME

Data science is the rare school-age subject where the professional tools are free and the real material, actual datasets about movies, sport, cities, and climate, is sitting in public archives waiting to be questioned. This 6-month live course teaches teens to do the real thing: load data with pandas, clean it honestly, chart it with matplotlib without lying, reason about it with statistics, and finally train first machine learning models with scikit-learn and understand what they did. The course is deliberately anti-hype. Students learn that most of data science is careful cleaning and honest questions, that a chart can mislead and how not to let it, and that a model is only as good as the data and the person reading its results. It ends with a data-story capstone: each student picks a question they care about, finds the data, does the analysis end to end, and presents a notebook that shows their reasoning. Basic Python is required; our Python for Teens course is the natural on-ramp for students who need it.

WHAT MAKES THIS COURSE DIFFERENT

- Real public datasets from the second month: movies, sports seasons, city open data, and climate records, not sanitized textbook tables
- A whole month on cleaning, because that is the actual job, and students learn to log every cleaning decision like professionals
- Honest charts are treated as an ethical skill: a full lesson on how visualizations mislead and how to avoid it
- Statistics taught for use, not for the exam: when the mean lies, what correlation does not prove
- Machine learning arrives late and lands properly: train-test splits and simple models the student can explain, not black-box demos
- The capstone is a question the student chose, answered end to end in a notebook they present and publish

THE LEARNING PATH, STEP BY STEP

Phase 1: Python tuned for data, then pandas: loading, filtering, grouping, and joining real datasets until table thinking is second nature
Phase 2: The craft: a full month cleaning genuinely messy data, then matplotlib and the discipline of charts that tell the truth
Phase 3: Statistics for judgment, first models with scikit-learn, and a capstone data story built from a question the student chose

WHAT A STUDENT NEEDS BEFORE STARTING

Python basics: Required: variables, loops, functions, and lists. Our Python for Teens course is the natural on-ramp if these are missing
Mathematics: Comfortable school maths. All statistics used is taught inside the course; no calculus is needed
Equipment: A computer with a modern browser and stable internet. Google Colab means nothing

heavy is installed
Age: 14 to 18, matched to the reasoning and statistics content

WHO THIS COURSE IS FOR

Python graduates: Teens who finished a first Python course and want to apply it to something real rather than more syntax
Question askers: Students who argue from evidence and want the tools to find it themselves
Future stem students: Teens heading toward engineering, economics, or science degrees where this exact stack appears in year one
Sports and stats fans: Students who already read tables of standings and averages for fun and want to compute their own
Portfolio builders: Teens who want a published, defensible piece of analysis to show teachers and admissions offices

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PYTHON FOR DATA AND THE PANDAS TOOLKIT (MONTHS 1-2, WEEKS 1-8)

First the Python muscles data work leans on, retrained in notebooks. Then pandas, the tool that turns tables of raw data into answers.

Month 1: Python, Retuned for Data (Weeks 1-4)
Notebooks and the Python Refresh
Topics:
- Google Colab and Jupyter notebooks: why data work lives in cells, not scripts
- Refresh: variables, lists, and dictionaries through data-flavored exercises
- Refresh: loops and conditionals over collections of records
- Markdown cells: notes and reasoning next to code
- Notebook habits: run order, restarts, and the stale-state trap
- The course map: where pandas, charts, and models fit
Projects built:
- Class survey analysis, round one: collect a small class survey and answer three questions about it with plain Python
Practice: 12 refresh exercises on lists and dictionaries, all phrased as data questions
Functions and Files
Topics:
- Refresh: writing and calling functions that transform data
- Reading files: opening a CSV by hand with the csv module
- What a CSV actually is: rows, delimiters, and headers
- Building lists of dictionaries from raw rows
- Writing results back out to a file
- Why this manual work matters: understanding what pandas automates
Projects built:
- Hand-rolled report: read a 500-row CSV with plain Python and compute totals, averages, and a top-five list
Practice: Write three reusable functions: load a CSV, filter rows by a condition, summarize a column
Thinking in Tables
Topics:
- Rows as records, columns as variables: the table mental model
- Data types in the wild: numbers, text, dates, and categories
- First contact with messy data: a deliberately imperfect dataset
- Questions data can answer, and questions it cannot
- From question to query: translating curiosity into precise operations
- Sources of real data: open data portals and public archives, toured live
Projects built:
- Question workshop: pick a public dataset, write five precise questions about it, and classify which are answerable from the columns present
Practice: Find one dataset about something you personally care about and describe its rows, columns, and types
NumPy Essentials
Topics:
- Arrays vs lists: what NumPy is for
- Vectorized operations: arithmetic on whole columns at once
- Useful reductions: sum, mean, min, max, and where they mislead
- Boolean masks: selecting data with conditions, the idea pandas is built on
- A speed demonstration: the same computation with a loop and with NumPy
- Reading NumPy errors without panic
Projects built:
- Class survey analysis, round two: redo week 1's analysis in NumPy and compare the code side by side
Practice: 10 NumPy exercises: masks, reductions, and column arithmetic on a provided dataset
Assessment: Month 1 check: live notebook exercise combining functions, file reading, and NumPy on a fresh dataset

```
  Month 2: pandas (Weeks 5-8)
    DataFrames
      Topics:
        - Series and DataFrames: pandas's two shapes
        - Loading CSVs with read_csv and its most useful options
        - First-look tools: head, info, describe, and shape
        - Selecting columns and rows: loc and iloc without the usual confusion
        - Column arithmetic: creating new columns from old ones
        - Saving your work: to_csv and keeping raw data untouched
      Projects built:
        - First pandas notebook: load a movies dataset and produce a profile of it, size, columns,
          types, and five first findings
      Practice: Load three different datasets and write a three-line profile of each
    Filtering and Sorting
      Topics:
        - Boolean masks in pandas: rows matching a condition
        - Combining conditions: and, or, and the parentheses that matter
        - isin, between, and string contains for readable filters
        - Sorting by one and several columns
        - nlargest and nsmallest: top-N questions answered directly
        - Chaining operations without losing readability
      Projects built:
        - Movie interrogation: answer ten ranked questions about the movies dataset, best-rated per
          decade, longest, most profitable, each as a clean filtered result
      Practice: Write and answer eight filter questions of your own on a sports dataset
    Grouping and Aggregation
      Topics:
        - value_counts: the fastest first answer in pandas
        - groupby: split, apply, combine, drawn before coded
        - Aggregations: mean, sum, count, and multiple at once with agg
        - Grouping by more than one column
        - Sorting grouped results to find leaders and outliers
        - Pivot tables, gently: the spreadsheet idea in pandas form
      Projects built:
        - Season in review: full statistical breakdown of a sports season, per team and per month, from
          a raw match-results dataset
      Practice: Answer six groupby questions on a dataset of your choice and mark the most
      surprising result
    Combining Datasets
      Topics:
        - concat: stacking datasets that share columns
        - merge: joining tables on a shared key, with the four join types in pictures
        - What gets lost in a join: rows that match nothing
        - Checking a merge: row counts before and after
        - Tidying column names and types after combining
        - The month project workflow: load, join, group, answer
      Projects built:
        - Two-table analysis: join two related datasets, for example city populations with air quality
          readings, and answer three questions no single table could
      Practice: Perform one merge on new data and write two sentences on rows that failed to
      match and why
      Assessment: Month 2 check: timed pandas practical, six questions on an unseen dataset,
      filters through joins

----------------------------------------------------------------
PHASE 2: CLEANING REAL DATA AND CHARTING IT HONESTLY (MONTHS 3-4, WEEKS 9-16)
----------------------------------------------------------------
The craft phase. Real datasets arrive dirty, and most of data science is the disciplined work
of fixing them, then showing results without distortion.

  Month 3: Cleaning Real Datasets (Weeks 9-12)
    Missing Values
      Topics:
        - Finding the holes: isna, counts, and missingness patterns
        - Why data goes missing, and why the reason changes the fix
        - Dropping rows or columns: when it is honest and when it hides too much
        - Filling values: constants, means, medians, and the trade-offs of each
        - The cleaning log: recording every decision and its justification
        - Raw data stays raw: cleaned copies, never overwritten originals
      Projects built:
        - Missing-value audit: map the missingness of a real open dataset, choose treatments per
          column, and defend each in your cleaning log
      Practice: Apply two different missing-value strategies to the same column and compare
      how the summary statistics move
    Wrong Types and Dirty Strings
      Topics:
        - Numbers trapped in text: currency symbols, commas, and units
        - String repair with .str methods: strip, replace, lower, split
```

- Dates done right: to_datetime and what it makes possible
- Categories with typos: Delhi, delhi, and DELHI as one value
- astype and coercion errors: reading what pandas is telling you
- A repair pipeline: ordered, rerunnable cleaning steps in one notebook section

Projects built:

- Type rescue: take a scraped price-and-date dataset where every column is text and restore true types, documenting each repair

Practice: Fix eight prepared dirty columns, one repair function each

Outliers, Duplicates, and Judgment Calls

Topics:

- Spotting outliers: sorting, describe, and quick plots
- Error or truth: the 190-year-old customer vs the genuine billionaire
- Duplicates: exact, partial, and the drop_duplicates decisions
- The ethics of cleaning: every removal changes the conclusion
- Documenting judgment calls so others can disagree with you fairly
- Sanity checks: totals and counts that must still make sense after cleaning

Projects built:

- Judgment file: on a dataset seeded with ambiguous outliers and duplicates, decide keep or remove case by case, with one-line justifications

Practice: Trade judgment files with a classmate and write where you disagreed and why

The Cleaning Project

Topics:

- A genuinely messy public dataset, start to finish
- Planning the clean: audit first, repair second, verify third
- Combining the month: missingness, types, strings, outliers, duplicates
- Writing the cleaning log as a professional document
- Before-and-after: quantifying what the clean changed
- Delivering an analysis-ready dataset someone else could trust

Projects built:

- Full cleaning project: raw city open data, for example street trees or road accidents, taken from download to documented, analysis-ready form

Practice: Run your cleaned dataset through five sanity checks and record the results

Assessment: Month 3 check: cleaning log review plus a defense of three judgment calls in a short viva

Month 4: Visualization with matplotlib (Weeks 13-16)

First Charts

Topics:

- The anatomy of a figure: axes, ticks, labels, title, legend
- Line charts: change over time, done cleanly
- Bar charts: comparing categories without clutter
- Scatter plots: two variables meeting for the first time
- Labeling as a duty: a chart must explain itself
- Plotting straight from pandas, and when to drop to matplotlib

Projects built:

- Three-chart notebook: one line, one bar, one scatter from your cleaned month 3 dataset, each fully labeled

Practice: Recreate two published charts from their underlying data as closely as you can

Distributions and the Right Chart

Topics:

- Histograms: seeing the shape of a column
- Bin sizes and how they change the story
- Box plots: medians, quartiles, and outliers at a glance
- Comparing groups: side-by-side boxes and grouped bars
- Choosing the chart from the question, not from habit
- The chart-choice checklist: comparison, trend, distribution, relationship

Projects built:

- Distribution study: profile three numeric columns with histograms and box plots, and write what each shape reveals

Practice: For six written questions, name the right chart type and justify it in one line each

Honest Charts and Polished Figures

Topics:

- How charts mislead: truncated axes, cherry-picked windows, and area tricks
- Fixing a misleading chart without hiding the real pattern
- Multi-panel figures with subplots
- Color with intent: highlighting one series, not decorating all of them
- Annotating the point: arrows and text that guide the reader
- Exporting print-quality figures

Projects built:

- Mislead and repair: take one truthful chart, make the most misleading legal version of it, then write the repair, a before-and-after pair

Practice: Find one misleading chart in real media and write a three-sentence critique

The Four-Chart Data Story

Topics:

- Narrative order: charts arranged as an argument
- One message per chart: trimming everything else
- Writing connective text between figures

- The takeaway figure: your single most important chart
- Presenting figures aloud: saying what to look at
- Peer review: reading a classmate's story cold

Projects built:
- Mini data story: a four-chart narrative on a dataset of your choice, presented live in five minutes

Practice: Revise your story after peer feedback and note the two changes that most improved it

Assessment: Month 4 check: data story presentation, graded on chart choice, honesty, and clarity

---

PHASE 3: STATISTICS, FIRST MODELS, AND THE CAPSTONE (MONTHS 5-6, WEEKS 17-24)

Statistics gives judgment, scikit-learn gives first models, and the capstone puts it all behind one question the student actually cares about.

Month 5: Statistics for Judgment, Then First Models (Weeks 17-20)

Describing Data Truthfully

Topics:
- Mean, median, and mode: three centers, three stories
- When the mean lies: income, house prices, and skew
- Spread: range, quartiles, and standard deviation as a feel, not a formula
- Percentiles: where a value really stands
- Summaries that hide things: same mean, wildly different data
- Writing statistical sentences that a non-technical reader cannot misread

Projects built:
- Two-summary exercise: describe the same real dataset once to flatter it and once to be fair, then annotate the tricks in the flattering version

Practice: Compute and interpret full summaries for three columns of your capstone-candidate dataset

Correlation and Its Traps

Topics:
- Correlation as a number: direction and strength on a scatter plot
- Computing correlations in pandas and reading a correlation matrix
- Correlation is not causation: the argument, with famous spurious examples
- Confounders: the hidden third variable, explained through cases
- When correlation is still useful despite the traps
- Reporting a relationship without overclaiming it

Projects built:
- Correlation hunt: find the three strongest relationships in a real dataset, then argue for each whether causation is even plausible

Practice: Find one spurious correlation in the wild or in the class archive and explain the likely confounder

First Machine Learning: Regression

Topics:
- What learning from data means, without mystique
- Features and targets: framing a prediction question
- The train-test split: why we grade models on unseen data
- Linear regression with scikit-learn: fit, predict, inspect
- Reading the model: coefficients as the story it learned
- Error metrics that mean something: how wrong is the model, in real units

Projects built:
- Price predictor: train a linear regression on a real housing or used-car dataset and report its typical error honestly

Practice: Add one feature to your model, remeasure the error, and write whether it earned its place

First Machine Learning: Classification

Topics:
- Classification vs regression: predicting a label instead of a number
- k-nearest neighbors: the friendliest first classifier
- Decision trees: models you can read as rules
- Accuracy and its limits: the imbalanced-classes trap
- The confusion matrix: seeing which mistakes the model makes
- Overfitting: memorizing vs learning, in one intuitive demonstration

Projects built:
- Penguin classifier: train and compare k-nearest neighbors and a decision tree on the Palmer Penguins dataset, with a confusion matrix for each

Practice: Break your classifier on purpose by shrinking training data, and chart accuracy against training size

Assessment: Month 5 check: statistics and modeling quiz plus a live reading of your own model's results

Month 6: The Capstone Data Story (Weeks 21-24)

The Question and the Data

Topics:
- Choosing a question you actually care about: sport, music, games, city, climate
- Dataset hunting: Kaggle, open data portals, and evaluating what you find
- Feasibility: does this data contain an answer to this question

- Scoping to six weeks of classes: the honest project plan
- Setting up the capstone notebook: sections for question, data, cleaning, analysis, conclusion
- Instructor sign-off on question, dataset, and plan

Projects built:
- Approved capstone plan: question, dataset, feasibility notes, and a week-by-week checklist

Practice: Complete the data audit of your chosen dataset: size, columns, types, missingness

Analysis Sprint

Topics:
- Cleaning your capstone data with a proper cleaning log
- Exploratory analysis: groupbys and filters chasing the question
- First figures: charting what the exploration surfaces
- Mid-sprint review with the instructor: findings so far, gaps remaining
- Following surprises: when the data disagrees with your hunch
- Keeping the notebook readable while you work fast

Projects built:
- Capstone core analysis: cleaned data, exploratory findings, and first draft figures in the notebook

Practice: Write your three strongest preliminary findings, each backed by one figure

The Optional Model and the Narrative

Topics:
- Deciding whether a model helps your question or just decorates it
- If modeling: framing, training, and honest error reporting
- Assembling the narrative: question, data, cleaning, findings, conclusion
- Limitations sections: what your analysis cannot claim, stated plainly
- Polishing figures to month 4 standard
- Dry-run presentations with peer feedback

Projects built:
- Complete capstone notebook: full narrative from question to conclusion, with a limitations section, model included only if it earns its place

Practice: Run your dry-run presentation for a family member and note the question that stumped you

Present and Publish

Topics:
- The final presentation: five minutes, the question, three findings, one takeaway figure
- Fielding questions about your own analysis
- Publishing the notebook to GitHub so it has a public address
- Writing the README: question, data source, and what you found
- Where this goes next: deeper ML, statistics, and college coursework
- Certificates and course close

Projects built:
- Published capstone: notebook live on GitHub, presented to the class and families

Practice: Send your published notebook to one teacher or mentor outside the course

Assessment: Capstone assessment: presentation, notebook quality, cleaning log, and honesty of claims

PROJECTS THE STUDENT BUILDS AND KEEPS

- Class survey analysis, done twice: plain Python, then NumPy
- Movie dataset interrogation: ten ranked questions, ten clean answers
- Sports season statistical review with groupby
- Two-table join analysis across related datasets
- Full cleaning project on raw city open data, with a professional cleaning log
- Mislead-and-repair chart pair: the same data, dishonest then honest
- Four-chart data story presented live
- Housing price regression with honestly reported error
- Penguin species classifier with confusion matrices
- The capstone: a published notebook answering a question the student chose

Total projects: 12+ analysis projects, ending with a published capstone notebook on GitHub

SKILLS THE STUDENT WILL HAVE AT THE END

- pandas fluency: loading, filtering, grouping, and joining real datasets with confidence
- Data cleaning: missing values, broken types, outliers, and duplicates handled with documented judgment
- Visualization: matplotlib figures chosen for the question and built to tell the truth
- Statistics for judgment: centers, spread, and correlation used without overclaiming
- First machine learning: regression and classification with scikit-learn, evaluated honestly on unseen data
- The analyst's workflow: notebooks, cleaning logs, limitations sections, and presenting findings aloud

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, working in notebooks alongside the instructor

Practice: 2-3 hours of dataset exercises and project work between classes

Review: Notebooks reviewed with written feedback, with special attention to honest claims and clean reasoning

Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for stuck code and confusing datasets
Progress updates: Regular progress notes to parents: skills covered, projects completed, and what needs practice

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Our AI and machine learning courses, which pick up exactly where the scikit-learn month ends
- Deeper Python engineering through our advanced Python and data structures courses
- School science fairs and economics or statistics projects, now backed by real analysis skills
- College coursework in data science, economics, and the sciences with a working head start
- Independent analysis projects: the student now owns the full stack from raw CSV to published notebook

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Does my teen need to know Python before this course?
A: Yes, the basics: variables, loops, functions, and lists. Month 1 includes a structured refresh, but it assumes prior exposure rather than teaching Python from zero. Students without that background should take our Python for Teens course first; it is the natural on-ramp, and many students move straight from one into the other.

Q: How much mathematics does the course require?
A: Comfortable school maths is enough. Every statistical idea the course uses, means and medians, spread, percentiles, correlation, is taught inside the course with real data, and there is no calculus anywhere. The emphasis is on judgment: knowing when an average misleads matters far more here than deriving formulas.

Q: Is this real machine learning or just a demo?
A: Real, at an honest first level. Students train linear regression and classification models on real datasets with scikit-learn, use proper train-test splits, read confusion matrices, and meet overfitting directly. What we deliberately avoid is black-box theatre: no pasting magic code that scores well unexplained. A student finishing this course can tell you what their model learned and how wrong it typically is, which is more than many adults using these tools can.

Q: What datasets will my teen work with?
A: Real public data throughout: movie databases, sports seasons, city open data such as street trees and road accidents, climate records, and the Palmer Penguins dataset for classification. From month 3 the data arrives genuinely messy on purpose, because cleaning real data is the core professional skill. For the capstone, students choose their own dataset around a question they care about.

Q: What does the capstone involve?
A: Each student picks a question, finds a suitable public dataset, and does the whole job: cleaning with a documented log, exploratory analysis, honest figures, an optional model if it genuinely helps, and a written conclusion with a limitations section. The finished notebook is published to GitHub and presented live. It is a genuine piece of analysis with a public address, not a classroom-only exercise.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final exam covering the whole course. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: How are the classes taught?
A: Live one-hour online classes, twice a week, in groups capped at 20 students. Students work in their own notebooks during class rather than watching, and homework notebooks come back with written feedback. Mini-batch (up to 6 students) and personal 1-on-1 options are available for students who want closer attention.

Q: Where does a teen go after this course?
A: The most common next step is our AI and machine learning track, which starts exactly where the scikit-learn month ends. Students leaning toward engineering deepen their Python with our advanced courses, and many put the skills straight to work in school science fairs and statistics projects. The capstone notebook also carries weight on its own as evidence of real

analytical work for college applications.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/data-science-course-for-teens-python-data
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 55 OF 137: DATA STRUCTURES & ALGORITHMS COURSE: INTERVIEW-READY DSA

==========

Course link: https://learn.modernagecoders.com/courses/data-structures-algorithms-masterclass-college
Subject area: Programming
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Data structures and algorithms course for college students and professionals: arrays to graphs to dynamic programming, then company-style interview mastery, in 10-12 months of live classes.

In one line: The skill every serious coding interview tests, built the only way it sticks: patterns understood, hundreds of problems solved, explained out loud under time.

ABOUT THE PROGRAMME

DSA is the great filter of technical hiring: the same topics appear in every serious interview, and the gap between having seen a problem and being able to solve a new one live is exactly what interviewers probe. This live online masterclass closes that gap the honest way, no memorized solution lists. It builds the full canon in order: complexity analysis and linear structures, searching, sorting and hashing, trees, heaps and graphs, then the hard middle, shortest paths, network flow, dynamic programming to real depth, greedy and backtracking, then interview mastery: company-style problem sets, timed simulations, and explaining your reasoning aloud the way the room demands. A finale month runs a mock-interview gauntlet and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of problem-solving, with two more in hand. Problem sets after every class, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- Patterns, not memorization: you learn to solve unseen problems, which is the actual test
- Explaining aloud is trained, not assumed: interviews score communication too
- Dynamic programming taught until it clicks, the topic that breaks most candidates
- Company-style problem sets and timed mock interviews, honestly framed as practice
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Core structures (Months 1-3): complexity, arrays to trees, heaps and hashing
Phase 2: Hard algorithms (Months 4-6): graphs, shortest paths, dynamic programming, greedy and backtracking
Phase 3: Interview mastery (Months 7-9): company-style sets, timed simulations, communication
Phase 4: The finale (Month 10): a mock-interview gauntlet and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Everything is taught from the first line
Equipment: Any computer with a stable internet connection; every tool used is free
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

College students: Students whose placements will live or die on DSA rounds

Career switchers: Professionals targeting product-company roles that test algorithms
Self taught: Coders who can build but freeze on algorithmic interview problems
Working engineers: Engineers preparing to switch companies through the interview gauntlet
Strengthening: Anyone who wants problem-solving depth that outlasts any single language

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & CORE DATA STRUCTURES (MONTHS 1-2, WEEKS 1-8)

Build rock-solid foundations with basic data structures, complexity analysis, and fundamental algorithms.

Month 1: Foundations & Linear Data Structures (Weeks 1-5)
Programming Fundamentals & Complexity Analysis
Topics:
- Course introduction and roadmap
- Setting up coding environment (VS Code, online judges)
- Introduction to problem-solving approach
- Time complexity and Big O notation
- Space complexity analysis
- Best, average, and worst case analysis
- Common complexity classes: O(1), O(log n), O(n), O(n log n), O(n²)
- Analyzing nested loops
- Recursion complexity analysis
- Amortized analysis basics
- Trade-offs: time vs space
- How to approach a problem systematically
Projects built:
- Complexity analyzer tool
- Problem-solving template creator
- Personal progress tracker
Practice: Analyze complexity of 50 code snippets
Arrays & Strings Mastery
Topics:
- Static vs dynamic arrays
- Array operations and complexity
- Two-pointer technique
- Sliding window technique
- Prefix sum arrays
- Kadane's algorithm
- Moore's voting algorithm
- String manipulation techniques
- String matching algorithms
- KMP algorithm basics
- Rabin-Karp algorithm
- Character encoding and Unicode
Projects built:
- Array manipulation library
- String utilities toolkit
- Pattern matching visualizer
Practice: Solve 40 array/string problems
Linked Lists & Recursion
Topics:
- Singly linked list implementation
- Doubly linked list
- Circular linked list
- Fast and slow pointer technique
- Floyd's cycle detection
- Linked list reversal (iterative and recursive)
- Merge operations on linked lists
- Recursion fundamentals
- Base case and recursive case
- Call stack and memory
- Tail recursion optimization
- Recursion tree analysis
Projects built:
- Linked list library from scratch
- Recursion visualizer
- Memory stack simulator
Practice: Implement 25 linked list operations
Stacks & Queues
Topics:
- Stack implementation using array and linked list
- Stack applications: expression evaluation, parentheses matching
- Infix to postfix conversion
- Queue implementation: array-based and linked list

- Circular queue implementation
- Deque (double-ended queue)
- Priority queue basics
- Monotonic stack patterns
- Monotonic queue patterns
- Stack and queue in recursion
- BFS and DFS preview
- Real-world applications

Projects built:
- Expression evaluator
- Browser history simulator
- Task scheduler with priority queue

Practice: Build 20 stack/queue applications

Month 2: Searching, Sorting & Hashing (Weeks 6-9)

Searching Algorithms

Topics:
- Linear search and optimizations
- Binary search deep dive
- Binary search on answer
- Search in rotated sorted array
- Finding boundaries with binary search
- Ternary search
- Exponential search
- Jump search
- Interpolation search
- Search in 2D matrices
- Finding peak elements
- Search space reduction techniques

Projects built:
- Search algorithm visualizer
- Binary search template library
- Search complexity analyzer

Practice: Solve 30 binary search problems

Sorting Algorithms Mastery

Topics:
- Bubble sort and optimizations
- Selection sort analysis
- Insertion sort and when to use
- Merge sort implementation
- Quick sort and partitioning
- Randomized quick sort
- Heap sort preview
- Counting sort for integers
- Radix sort implementation
- Bucket sort technique
- Stability in sorting
- External sorting for large data

Projects built:
- Sorting visualizer application
- Hybrid sorting algorithm
- Performance comparison tool

Practice: Implement all sorting algorithms

Hashing & Hash Tables

Topics:
- Hash function design
- Collision resolution: chaining vs open addressing
- Load factor and rehashing
- Hash table implementation from scratch
- Hash map vs hash set
- Rolling hash technique
- Consistent hashing basics
- Bloom filters introduction
- Hash table patterns in problems
- Two-sum pattern and variations
- Frequency counting with hash maps
- Hash table optimization techniques

Projects built:
- Custom hash table implementation
- LRU cache implementation
- Distributed hash table simulator

Practice: Solve 35 hashing problems

Bit Manipulation & Mathematics

Topics:
- Binary number system review
- Bitwise operators: AND, OR, XOR, NOT
- Left shift and right shift
- Bit manipulation tricks
- Setting, clearing, toggling bits

```
      - Counting set bits
      - Power of 2 checks
      - XOR properties and applications
      - Bit masking techniques
      - GCD and LCM algorithms
      - Prime number algorithms
      - Modular arithmetic basics
    Projects built:
      - Bit manipulation toolkit
      - Number theory library
      - Cryptography basics implementation
    Practice: Master 25 bit manipulation techniques

Month 3: Trees, Heaps & Graphs (Weeks 10-13)
  Binary Trees Fundamentals
    Topics:
      - Tree terminology and properties
      - Binary tree representation
      - Tree traversals: inorder, preorder, postorder
      - Level order traversal (BFS)
      - Recursive vs iterative traversals
      - Morris traversal (O(1) space)
      - Tree construction from traversals
      - Diameter, height, and width of tree
      - Path sum problems
      - Lowest Common Ancestor (LCA)
      - Serialize and deserialize trees
      - Tree isomorphism
    Projects built:
      - Binary tree visualizer
      - Tree traversal animator
      - Tree problem solver toolkit
    Practice: Solve 40 binary tree problems
  Binary Search Trees & Balanced Trees
    Topics:
      - BST properties and operations
      - BST insertion and deletion
      - BST validation and searching
      - Inorder successor and predecessor
      - Convert sorted array to BST
      - Self-balancing trees introduction
      - AVL tree rotations
      - AVL tree implementation
      - Red-Black tree basics
      - B-trees and B+ trees overview
      - Segment trees introduction
      - Binary Indexed Tree (Fenwick Tree)
    Projects built:
      - BST implementation with all operations
      - AVL tree from scratch
      - Range query data structure
    Practice: Implement 20 BST operations
  Heaps & Priority Queues
    Topics:
      - Heap properties: min-heap and max-heap
      - Heap implementation using array
      - Heapify operation and complexity
      - Heap sort implementation
      - Priority queue operations
      - Binary heap vs Fibonacci heap
      - K-way merge using heaps
      - Median finding with two heaps
      - Top K elements pattern
      - Heap applications in algorithms
      - Dijkstra's algorithm preview
      - Custom comparators in heaps
    Projects built:
      - Heap implementation from scratch
      - Task scheduler with priority
      - Real-time median finder
    Practice: Master 25 heap problems
  Tries & Advanced Trees
    Topics:
      - Trie data structure implementation
      - Trie insertion, search, and deletion
      - Prefix matching with tries
      - Auto-complete functionality
      - Word search in matrix using trie
      - Suffix trees and arrays basics
```

- N-ary tree operations
- Quad trees and Oct trees
- K-d trees for spatial data
- Disjoint Set Union (DSU/Union-Find)
- Path compression and union by rank
- Applications of DSU

Projects built:
- Autocomplete system
- Spell checker implementation
- Contact list with search

Practice: Build 15 trie applications

Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 2: GRAPHS & ADVANCED ALGORITHMS (MONTHS 3-4, WEEKS 9-17)

Master graph algorithms, advanced problem-solving patterns, and complex algorithmic techniques.

Month 4: Graph Algorithms Mastery (Weeks 14-19)

Graph Fundamentals & Traversals

Topics:
- Graph representations: adjacency matrix vs list
- Directed vs undirected graphs
- Weighted vs unweighted graphs
- Graph properties: connected, cyclic, bipartite
- Depth-First Search (DFS) implementation
- Breadth-First Search (BFS) implementation
- DFS applications: cycle detection, topological sort
- BFS applications: shortest path, level order
- Connected components finding
- Strongly connected components (Kosaraju's, Tarjan's)
- Bridges and articulation points
- Graph coloring basics

Projects built:
- Graph visualizer with traversals
- Social network analyzer
- Maze solver with BFS/DFS

Practice: Solve 35 graph traversal problems

Shortest Path Algorithms

Topics:
- Dijkstra's algorithm implementation
- Dijkstra's with priority queue
- Bellman-Ford algorithm
- Negative weight cycles detection
- Floyd-Warshall algorithm
- A* search algorithm basics
- Shortest path in DAG
- Single source vs all pairs
- Path reconstruction techniques
- Multi-source BFS
- 0-1 BFS technique
- Bidirectional search

Projects built:
- Route finder application
- Navigation system simulator
- Network routing optimizer

Practice: Implement all shortest path algorithms

Minimum Spanning Trees & Network Flow

Topics:
- Minimum Spanning Tree (MST) concept
- Kruskal's algorithm with DSU
- Prim's algorithm implementation
- Applications of MST
- Network flow introduction
- Ford-Fulkerson method
- Maximum flow problem
- Minimum cut theorem
- Bipartite matching
- Hungarian algorithm basics
- Stable marriage problem
- Assignment problems

Projects built:
- Network designer tool
- Resource allocation system
- Matching algorithm implementation

Practice: Solve 20 MST and flow problems

Advanced Graph Patterns

```
    Topics:
      - Topological sorting variations
      - Cycle detection in directed/undirected graphs
      - Bipartite graph checking and coloring
      - Eulerian path and circuit
      - Hamiltonian path problem
      - Graph dynamic programming
      - Tree DP on graphs
      - Centroid decomposition basics
      - Heavy-light decomposition intro
      - Graph compression techniques
      - Parallel BFS/DFS
      - Graph optimization problems
    Projects built:
      - Dependency resolver
      - Circuit board router
      - Tournament scheduler
    Practice: Master 25 advanced graph patterns
  String Algorithms Advanced
    Topics:
      - KMP algorithm implementation
      - Z-algorithm for pattern matching
      - Rabin-Karp with rolling hash
      - Boyer-Moore algorithm
      - Suffix arrays construction
      - Longest Common Prefix array
      - Manacher's algorithm for palindromes
      - Edit distance algorithms
      - Longest common subsequence/substring
      - String hashing techniques
      - Suffix automaton basics
      - Aho-Corasick algorithm
    Projects built:
      - Text editor with find/replace
      - Plagiarism detector
      - DNA sequence analyzer
    Practice: Implement 15 string algorithms

Month 5: Dynamic Programming Mastery (Weeks 20-24)
  Dynamic Programming Fundamentals
    Topics:
      - Introduction to dynamic programming
      - Recursion to DP transition
      - Memoization techniques
      - Top-down vs bottom-up approach
      - 1D DP patterns
      - State definition in DP
      - Fibonacci variations
      - Climbing stairs variations
      - House robber pattern
      - Jump game problems
      - Decode ways pattern
      - DP space optimization
    Projects built:
      - DP visualizer tool
      - Recursion tree analyzer
      - DP pattern identifier
    Practice: Solve 40 basic DP problems
  2D Dynamic Programming
    Topics:
      - 2D DP table construction
      - Grid-based DP problems
      - Path counting in grids
      - Longest common subsequence
      - Edit distance problem
      - 0/1 Knapsack problem
      - Unbounded knapsack
      - Subset sum problems
      - Matrix chain multiplication
      - Palindrome partitioning
      - Regular expression matching
      - Wildcard matching
    Projects built:
      - Sequence alignment tool
      - Knapsack optimizer
      - Pattern matcher implementation
    Practice: Master 35 2D DP problems
  Advanced DP Patterns
    Topics:
```

- State machine DP
- DP on trees
- DP with bitmasking
- Digit DP technique
- Probability DP
- Game theory and DP
- Interval DP pattern
- String DP problems
- DP optimization techniques
- Convex hull trick
- Divide and conquer optimization
- Knuth-Yao speedup

Projects built:

- Stock trading simulator with DP
- Game AI using minimax
- Optimal strategy finder

Practice: Solve 30 advanced DP problems

Greedy Algorithms & Optimization

Topics:

- Greedy algorithm principles
- When greedy works: proof techniques
- Activity selection problem
- Huffman coding
- Fractional knapsack
- Job scheduling problems
- Interval scheduling maximization
- Minimum platforms problem
- Gas station circuit problem
- Jump game greedy approach
- Greedy vs DP comparison
- Exchange argument proof

Projects built:

- Task scheduler optimizer
- Huffman compression tool
- Resource allocator

Practice: Master 25 greedy problems

Backtracking & Branch and Bound

Topics:

- Backtracking fundamentals
- State space tree
- N-Queens problem
- Sudoku solver
- Subset generation
- Permutation generation
- Combination sum problems
- Word search in grid
- Graph coloring with backtracking
- Branch and bound technique
- Pruning strategies
- Optimization in backtracking

Projects built:

- Sudoku solver application
- N-Queens visualizer
- Constraint satisfaction solver

Practice: Implement 20 backtracking solutions

Month 6: Advanced Topics & Interview Preparation (Weeks 25-26)

Advanced Algorithms

Topics:

- Divide and conquer mastery
- Master theorem for recurrence
- Fast Fourier Transform basics
- Karatsuba multiplication
- Strassen's matrix multiplication
- Closest pair of points
- Convex hull algorithms
- Line sweep technique
- Mo's algorithm basics
- Square root decomposition
- Heavy-light decomposition
- Centroid decomposition

Projects built:

- Computational geometry library
- Big integer calculator
- Algorithm complexity analyzer

Practice: Implement 15 advanced algorithms

System Design Basics for Interviews

Topics:

- System design interview approach

- Scalability basics
- Load balancing concepts
- Caching strategies
- Database design basics
- SQL vs NoSQL
- Microservices overview
- API design principles
- Rate limiting algorithms
- Consistent hashing
- Distributed systems basics
- CAP theorem introduction

Projects built:
- URL shortener design
- Rate limiter implementation
- Simple cache system

Practice: Design 10 systems

Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 3: INTERVIEW MASTERY (MONTHS 7-9, WEEKS 27-39)

Master interview skills, tackle company-specific problems, and launch your tech career.

Months 7-8: Company-Specific Preparation (Weeks 27-35)

Google Interview Preparation

Topics:
- Google interview process overview
- Google's evaluation criteria
- Googliness and Leadership
- Common Google problem patterns
- Array and string manipulation focus
- Graph algorithms emphasis
- Dynamic programming at Google
- System design at Google scale
- Google coding style guide
- Time complexity requirements
- Follow-up questions handling
- Google behavioral questions

Projects built:
- Google-style problem solver
- MapReduce implementation
- PageRank algorithm

Practice: Solve 50 Google tagged problems

Amazon Interview Preparation

Topics:
- Amazon interview process
- Leadership Principles deep dive
- Amazon problem-solving approach
- Object-oriented design at Amazon
- Scalability focus
- Two-pointer and sliding window emphasis
- Tree and graph problems
- Amazon system design topics
- Working backwards methodology
- Bar raiser round preparation
- Amazon behavioral questions
- STAR format examples

Projects built:
- E-commerce system design
- Recommendation engine
- Warehouse robot pathfinding

Practice: Solve 50 Amazon tagged problems

Meta (Facebook) Preparation

Topics:
- Meta interview structure
- Ninja, Pirate, Jedi expectations
- Meta's focus areas
- Array and string heavy problems
- BFS/DFS variations
- Dynamic programming patterns
- Design Facebook features
- Meta's behavioral assessment
- Move fast philosophy
- Impact-focused answers
- Coding signal importance
- System design for social networks

Projects built:
- Social network features

```
      - News feed algorithm
      - Friend recommendation system
    Practice: Solve 50 Meta tagged problems
  Microsoft & Apple Preparation
    Topics:
      - Microsoft interview approach
      - Problem-solving communication
      - Microsoft stack questions
      - Linked list and tree focus
      - String manipulation problems
      - Apple interview process
      - Quality and attention to detail
      - iOS/macOS specific questions
      - Design patterns emphasis
      - Both companies' behavioral focus
      - Growth mindset demonstration
      - Innovation and creativity
    Projects built:
      - Operating system scheduler
      - File system design
      - Music streaming service
    Practice: Solve 40 MS/Apple problems
  Startup & Unicorn Preparation
    Topics:
      - Startup interview differences
      - Full-stack expectations
      - Practical coding tests
      - Take-home assignments
      - System design for startups
      - Real-world problem solving
      - API design and implementation
      - Database design questions
      - DevOps and deployment
      - Startup culture fit
      - Ownership and initiative
      - Rapid iteration mindset
    Projects built:
      - MVP product design
      - Real-time collaboration tool
      - Payment processing system
    Practice: Complete 3 take-home projects

Month 9: Final Polish & Interview Simulation (Weeks 36-39)
  Advanced Problem Patterns
    Topics:
      - Multi-threaded programming problems
      - Concurrency and locks
      - Design patterns in problems
      - Object-oriented design problems
      - Database query problems
      - SQL in interviews
      - Machine learning basics for interviews
      - Security-related problems
      - Cryptography basics
      - Blockchain basic problems
      - Mobile-specific problems
      - Real-time system problems
    Projects built:
      - Thread-safe data structures
      - Distributed lock manager
      - Real-time chat system
    Practice: Solve 30 advanced mixed problems
  Interview Week Simulation
    Topics:
      - Full day interview simulation
      - Multiple rounds back-to-back
      - Phone screen practice
      - Technical rounds practice
      - System design rounds
      - Behavioral rounds
      - Hiring manager rounds
      - Lunch interview practice
      - Stress management techniques
      - Energy management
      - Recovery between rounds
      - End-of-day strategies
    Projects built:
      - Complete interview portfolio
      - Problem solution database
```

        - Personal story bank
      Practice: 5 full interview simulations
    Soft Skills & Communication
      Topics:
        - Technical communication skills
        - Explaining complex concepts simply
        - Asking clarifying questions
        - Requirements gathering
        - Trade-off discussions
        - Collaborative problem solving
        - Handling hints effectively
        - Dealing with stuck situations
        - Time management during interviews
        - Body language and confidence
        - Virtual interview best practices
        - Follow-up communication
      Projects built:
        - Technical presentation
        - Code review exercise
        - Team project simulation
      Practice: Record and review 10 problem explanations
    Career Strategy & Negotiation
      Topics:
        - Job search strategies
        - Resume optimization for ATS
        - LinkedIn profile optimization
        - Cover letter writing
        - Portfolio website creation
        - GitHub profile enhancement
        - Networking strategies
        - Referral seeking techniques
        - Offer evaluation framework
        - Salary negotiation tactics
        - Benefits comparison
        - Career growth planning
      Projects built:
        - Professional portfolio website
        - Optimized resume versions
        - LinkedIn article publishing
      Practice: Apply to 20 positions strategically
      Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)

The capstone month: spec, build, ship and defend, then the final exam that decides the
certificate.

  Month 10: Capstone and the Final Exam (Weeks 40-44)
    Capstone Spec and Setup
      Topics:
        - Choosing the capstone: a full mock-interview gauntlet plus a solved-problem portfolio
          spanning every pattern
        - A one-page spec: scope, milestones, stack
        - Repository and skeleton ready on day one
        - Instructor sign-off before the build begins
      Projects built:
        - Approved capstone spec plus the running skeleton
      Practice: Pitch the spec to the batch and absorb one hard question
    Capstone Build
      Topics:
        - Two focused weeks in vertical slices
        - Instructor checkpoints and honest scope cuts
        - Using the whole course: every structure and algorithm, under interview-style time pressure
          with reasoning spoken aloud
        - Daily commits and an always-working build
      Projects built:
        - The capstone, feature-complete with version control
      Practice: A working build at the end of every session
    Ship and Polish
      Topics:
        - The finishing pass: edge cases, errors, documentation
        - Clean, optimal, explained: solutions that are correct, efficient and communicated well
        - A README worth reading and a demo worth watching
        - A full-course spiral review before the exam
      Projects built:
        - The capstone shipped, documented and rehearsed
      Practice: Two mock demos, each tighter than the last

Demo Day and the Final Exam
Topics:
- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: placement interviews, competitive programming, or system design depth
- Where our other courses continue the journey

Projects built:
- Demo day presentation delivered to the batch

Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: implementations of every core structure, from scratch
- Phase 2: graph, DP and backtracking solution sets on hard problems
- Phase 3: company-style problem sets and timed mock interviews
- Phase 4: the mock-interview gauntlet and final solved-problem portfolio

Total projects: 300+ problems solved and every core structure implemented, crowned by a mock-interview gauntlet

SKILLS THE STUDENT WILL HAVE AT THE END

- Complexity analysis: reasoning about time and space by instinct
- Every core structure implemented and understood: arrays to graphs
- The hard algorithms: shortest paths, network flow, dynamic programming
- Problem-solving patterns that transfer to unseen problems
- Interview communication: explaining an optimal solution under time

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Placement and product-company interviews: this is the round they weight most
- Our competitive programming course for contest-level speed and technique
- System design study for senior interview rounds
- Any language course here pairs naturally with this problem-solving spine

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Ready for Rs 5-10 LPA roles; Ready for Rs 10-20 LPA roles
Growing roles: Ready for Rs 15-40+ LPA roles
Other roles: Rs 25-60+ LPA packages
International: $100k-250k+ USD

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this DSA course cover?
A: The full interview canon in order: complexity analysis, arrays, strings, linked lists, recursion, stacks and queues, searching, sorting, hashing, trees, heaps, tries and graphs; then the hard algorithms, shortest paths, minimum spanning trees, network flow, dynamic programming to real depth, greedy and backtracking; then interview mastery with company-style problem sets, timed simulations and communication practice. A finale month runs a mock-interview gauntlet and sits the final exam.

  Q: How long does it take to learn data structures and algorithms properly?
  A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two more months in hand for learners who need them. The three teaching phases build the skill and its real-world application; the finale month ships a capstone and sits the final exam.

  Q: How am I assessed? Is the certificate just for attending?
  A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

  Q: Which language is used for DSA here?
  A: You solve in the language you will interview in, most commonly C++, Java or Python, and the instructor supports all three. DSA is language-agnostic at heart: the patterns matter more than the syntax, and we make sure your chosen language never gets in the way of the thinking.

  Q: Will this course guarantee me a job?
  A: No honest course can, and we will not pretend otherwise. What this course guarantees is genuine problem-solving ability, the specific skill interviews test, built through hundreds of solved problems and real mock interviews. We prepare you to earn the offer; we do not sell the offer.

  Q: Can I try a class before enrolling?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/data-structures-algorithms-masterclass-college
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 56 OF 137: EARLY MATH FOUNDATIONS: PLAYFUL, CONCRETE MATH (AGES 4-7)
==========

Course link: https://learn.modernagecoders.com/courses/early-math-foundations
Subject area: Elementary Mathematics
Designed for: Elementary school
Who can join: Ages 4-7 (kindergarten to grade 2), all starting points
Course length: 12 months (52 weeks)
Time needed: 2 live one-hour classes per week (structured as playful segments) + everyday math play at home
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Live online early math course for ages 4-7: counting, number bonds, shapes and measurement taught concrete-first in short, joyful classes. Free demo first.

  In one line: The years that decide whether math becomes a friend. We make it one.

ABOUT THE PROGRAMME

  A 12-month live program for children aged 4-7, the years when a child's whole relationship with mathematics is quietly decided. Early math done right is not worksheets; it is counting that means something, quantities seen and handled, patterns spotted, shapes built and stories solved. Our mentors teach concrete-first in genuinely interactive classes structured as short playful segments that respect young attention spans: real objects, ten-frames, games and math talk, with the child doing and explaining rather than watching. Children leave this course fluent in early number, confident in their own thinking, and, the part parents notice most, actually liking math.

WHAT MAKES THIS COURSE DIFFERENT

  - Concrete before abstract, always: quantities are handled, drawn, and only then written as

symbols
- Classes built for young attention: an hour of varied segments, games, movement, building, story problems, never an hour of sitting still
- Counting taught with meaning: one-to-one correspondence, cardinality and the ideas most programs skip straight past
- Number bonds and part-whole thinking started early, gently, the foundation everything to age 18 stands on
- Math talk from day one: even five-year-olds explain their thinking here, in their own words
- One dedicated mentor who knows the child, their pace and their spark
- Parents equipped too: simple everyday math-play prompts, because kitchens teach more than worksheets
- The honest opposite of drill academies, no pressure, no rankings, no tears

THE LEARNING PATH, STEP BY STEP

Phase 1: Counting with meaning, comparing quantities, and the first part-whole ideas, through objects and games
Phase 2: Addition and subtraction as stories, bonds to 10, place value beginnings and the teen numbers demystified
Phase 3: Numbers to 100 and beyond, early measurement, shapes and data, plus problem-solving courage and grade-2 readiness

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Ages 4-7; no academic prerequisites whatsoever
Math background: None, this course IS the foundation
Equipment: Computer or tablet with stable internet; everyday objects (blocks, buttons, spoons) as materials
Time commitment: 2 live hours plus everyday math play
English: Course taught in English
Parent role: Nearby for the youngest children in early weeks; cheering thereafter

WHO THIS COURSE IS FOR

First steps: Ages 4-5 taking their very first structured math steps, playfully
K 2 students: Kindergarten to grade 2 children whose school math needs a stronger foundation underneath
Eager minds: Young children already fascinated by numbers who deserve real food for it
Worried starts: Children whose first school math experiences created tears, reversible at this age, quickly
Planning parents: Families who want the foundation laid right the first time, before gaps exist

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: COUNTING THAT MEANS SOMETHING (MONTHS 1-4)

Counting looks simple and hides deep ideas: one number per object, the last number telling the total, quantities compared and conserved. This phase builds those ideas properly through objects, movement and games, the difference between reciting numbers and understanding them.

Month 1: Numbers as Quantities (Weeks 1-4)
A Playful First Look
Topics:
- Counting objects to see how far counting flows
- Comparing two small groups for more or fewer
- Copying and continuing a simple pattern
- Noticing how the child talks about numbers
- Spotting counting habits gently, without correcting
- Finding a warm, happy starting point
Practice: Line up a few toys and ask your child to count them and tell you which pile has more, keeping it a game and not a test.
Detail: Playful diagnostic disguised as games: counting, comparing and pattern instincts observed
One Number for Each Thing
Topics:
- Touching each object once while saying one number
- Matching one number word to one thing
- Understanding the last number tells the total
- Counting a group and answering how many
- Recounting a moved group to see the total stays the same
- Slowing down to count carefully
Practice: Count the stairs together as you climb, touching each step and cheering the final number as the total.
Detail: One-to-one counting with real objects; the cardinality idea, the last number IS the amount

  Seeing Small Numbers Instantly
    Topics:
      - Recognising 1, 2 and 3 without counting
      - Seeing 4 and 5 as quick patterns
      - Reading dice dots at a glance
      - Showing numbers fast with fingers
      - Naming flashed dot cards instantly
      - Talking about how the amount was seen
    Practice: Flash some fingers for a second, hide them, and ask how many your child saw, taking turns being the flasher.
    Detail: Subitizing: seeing 3, 4 and 5 without counting, dice, dots and finger patterns
  More, Fewer, or the Same?
    Topics:
      - Comparing two groups for more or fewer
      - Matching items one to one to compare fairly
      - Recognising when two groups are the same
      - Using the words more, fewer and equal
      - Explaining how you know which has more
      - Making two groups equal by adding or taking away
    Practice: Put out two rows of buttons and ask your child to line them up in pairs to prove which row has more, or whether they match.
    Detail: Comparing quantities: more, fewer, same, justified with matching, not guessing

Months 2-3: The Number World to 10 (Weeks 5-12)
  Number Symbols and Counting Both Ways
    Topics:
      - Matching the written numeral to its quantity
      - Recognising the numerals 0 to 10
      - Counting forwards to ten smoothly
      - Counting backwards from ten like a rocket
      - Placing a numeral card beside the right group
      - Writing or tracing numerals playfully
    Practice: Do a rocket countdown from 10 to 0 before a jump or a tickle, then count back up to ten together.
    Detail: Numerals connected to quantities; counting forwards and backwards as a game
  The Ten-Frame as Home Base
    Topics:
      - Filling a ten-frame from left to right
      - Seeing a full row of five
      - Reading 7 as five and two more
      - Spotting how many empty spaces are left
      - Seeing numbers hidden inside bigger numbers
      - Knowing how far a number is from ten
    Practice: Draw a two-by-five grid and let your child place counters to show a number like 8, then say it as five and three more.
    Detail: Ten-frames as home base: five-and-some structure, seeing numbers inside numbers
  Breaking Numbers into Parts
    Topics:
      - Splitting a small group into two parts
      - Seeing 5 as 2 and 3, or 4 and 1
      - Finding different ways to make the same number
      - Putting the two parts back into the whole
      - Using two hands to hold the two parts
      - Talking about the parts out loud
    Practice: Hide 5 counters behind your back in two hands, show one hand, and let your child work out how many are hiding in the other.
    Detail: Part-whole play: 5 is 2-and-3, 4-and-1, decomposition as a toy, years before it is a technique
  First, Second, Third and Where Things Sit
    Topics:
      - Using first, second, third and last
      - Describing position with in front, behind and between
      - Walking along a number line as a path
      - Finding which number comes before or after
      - Ordering a small line of toys
      - Naming who is in each place in a line
    Practice: Line up soft toys and ask playful questions like which one is third, or who is between the first and the fourth.
    Detail: Ordinal numbers, position words and the number line met as a walking game

Month 4: Patterns and Sorting + Stage Checkpoint (Weeks 13-16)
  Patterns That Repeat
    Topics:
      - Copying a simple repeating pattern
      - Continuing a pattern by what comes next
      - Creating a brand new pattern of your own
      - Naming the repeating unit, like red blue red blue
      - Spotting patterns in colours, shapes and sounds
      - Fixing a pattern with a deliberate mistake in it

Practice: Start a pattern with household items, like spoon fork spoon fork, and ask what comes next, then let your child build one for you.
Detail: Repeating patterns: copy, extend, create, the beginnings of algebraic thinking

Sorting by How Things Are Alike
Topics:
- Sorting objects into groups by colour, size or shape
- Choosing your own rule for sorting
- Explaining the rule you used
- Noticing an object that does not belong
- Sorting the same set a second, different way
- Describing groups with words like all and none

Practice: Tip out a box of toys or socks and ask your child to sort them, then guess the rule they chose before they tell you.
Detail: Sorting and classifying by attributes; explaining the rule you sorted by

Patterns That Grow
Topics:
- Seeing a pattern that gets bigger each step
- Working out what comes next in a growing pattern
- Building steps like 1, then 2, then 3 blocks
- Describing how the pattern changes
- Predicting a later step in the pattern
- Comparing repeating and growing patterns

Practice: Build a staircase of blocks that grows by one each time and ask your child how many blocks the next step needs.
Detail: Growing patterns and what-comes-next reasoning

Phase One Check Through Play
Topics:
- Counting a group and telling the total
- Comparing quantities with more and fewer
- Showing a number on a ten-frame
- Copying and extending a pattern
- Explaining thinking out loud
- Teaching a skill to a toy or a parent

Practice: Let your child teach a teddy how to count a small group and spot which pile has more, while you happily listen in.
Assessment: A gentle play-based check of counting, comparing, ten-frames and patterns, where the child shows a skill, says how it works, and teaches it to a toy, with no pressure.
Detail: Stage mastery check through play: show it, say it, teach it to your bear

---
PHASE 2: STORIES, BONDS AND THE TEEN NUMBERS (MONTHS 5-8)

Addition and subtraction arrive as stories acted out with objects, joining, separating, comparing, long before symbols carry them. Bonds to 10 grow toward automaticity through games, and the famously confusing teen numbers get demystified with place-value materials.

Month 5: Addition and Subtraction as Stories (Weeks 17-20)

Adding and Taking Away as Stories
Topics:
- Acting out a joining story with objects
- Acting out a taking-away story
- Meeting the plus sign as and more
- Meeting the minus sign as take away
- Reading a simple number story as a picture
- Telling the story back in your own words

Practice: Tell a tiny story with toys, like three ducks and two more join them, and let your child act it out and say how many altogether.
Detail: Joining and separating stories acted with objects; the plus and minus symbols as story shorthand

Counting On and Counting Back
Topics:
- Starting from the bigger number and counting on
- Not going back to one every time
- Counting back to take away
- Using fingers to count on a few more
- Playing the hidden-in-the-hand take-away game
- Choosing to count on or count back

Practice: Hold some counters in a closed hand, say the number inside, add a couple on top, and let your child count on from the hidden number.
Detail: Counting on (not starting over) and counting back; the empty-hand subtraction game

Month 6: Bonds of 10 and Fact Families (Weeks 21-24)

The Pairs That Make Ten
Topics:
- Finding two parts that make ten
- Learning pairs like 6 and 4, 7 and 3
- Filling a ten-frame to see the missing part

- Answering the partner of a number quickly
- Playing many games to make the pairs stick
- Feeling how ten is a friendly whole

Practice: Play friends of ten: hold up some fingers on one hand and ask how many more make ten, then swap hands.

Detail: Bonds of 10 through every game we know, the single most valuable fact set in mathematics

One Number Trio, Four Facts

Topics:

- Seeing three numbers that belong together
- Making two addition facts from the trio
- Making two subtraction facts from the trio
- Understanding adding and taking away as linked
- Using one known fact to find the others
- Building the whole fact family from a picture

Practice: Give your child three numbers like 2, 8 and 10 and see how many facts they can say using all three.

Detail: Fact families: 3+7, 7+3, 10-7, 10-3 as one idea seen four ways

Month 7: Teen Numbers and Place Value Beginnings (Weeks 25-28)

Teen Numbers Are Ten and Some More

Topics:

- Building a teen number as one ten and some ones
- Seeing 14 as ten and four
- Grouping ten small items into one bundle
- Reading teen numbers correctly
- Understanding where place value begins
- Matching the numeral to the ten-and-ones picture

Practice: Bundle ten straws or sticks with a band, add a few loose ones, and ask your child to name the teen number you made.

Detail: Teens demystified: 14 as ten-and-four, built with materials, where place value actually starts

All the Way to Twenty

Topics:

- Counting forwards and backwards to 20
- Comparing two numbers up to 20
- Ordering numbers up to 20
- Filling a second ten-frame past ten
- Previewing the make-ten bridge as a game
- Spotting the ten inside a teen number

Practice: Count a bigger pile of objects, like 16 pasta pieces, into two ten-frames, then ask which of two numbers up to 20 is bigger.

Detail: Numbers to 20: counting, comparing, and bridging-through-10 previewed as a game

Month 8: Story Problems and the Explain Habit (Weeks 29-32)

Act It, Draw It, Write It

Topics:

- Acting out a word problem with objects
- Drawing a simple picture of the story
- Writing the matching number sentence
- Finding the question hidden in the story
- Deciding whether to join or take away
- Checking the answer against the story

Practice: Make up a little story, such as five apples and two eaten, and guide your child to act it, draw it, then write the number sentence.

Detail: Word problems as theatre: act it, draw it, then write it, the three-step that prevents future word-problem fear

Missing Numbers and Phase Two Check

Topics:

- Finding a missing number in a simple sentence
- Thinking backwards from an answer
- Using fact families to fill the gap
- Reviewing story problems and bonds of ten
- Explaining how a puzzle was solved
- Showing teen numbers as ten and ones

Practice: Pose a gentle puzzle like something plus 3 makes 7 using counters, and let your child find the missing amount.

Assessment: A relaxed play-based check of story problems, bonds of ten, fact families and teen numbers, where the child solves a puzzle and explains the thinking, with no pressure.

Detail: Missing-number puzzles and both-ways thinking; stage mastery check through play

---

PHASE 3: TO 100 AND INTO THE WORLD (MONTHS 9-12)

The number world expands to 100 with place-value understanding, and math steps into the child's world: measuring, money, time, shapes and simple data. The course closes with problem-solving courage and a confident handoff to grade-level mathematics.

Month 9: The Road to 100 (Weeks 33-36)
  Building Numbers from Tens and Ones
    Topics:
      - Building a two-digit number from tens and ones
      - Bundling tens and counting the leftover ones
      - Reading a number like 34 as three tens and four
      - Comparing two two-digit numbers
      - Ordering several numbers from small to big
      - Matching the numeral to the tens-and-ones model
    Practice: Use grouped straws or blocks to build a number like 27, then build another and ask your child which is bigger and why.
    Detail: Tens and ones with materials: building two-digit numbers, comparing and ordering them
  Skip Counting and the Hundred Square
    Topics:
      - Skip counting by 10s to one hundred
      - Skip counting by 5s and by 2s
      - Hearing skip counting as a rhythm
      - Finding patterns on the hundred square
      - Moving up, down and across the hundred square
      - Spotting the tens column and its counting patterns
    Practice: Chant counting by tens up to a hundred on a walk, then try by fives, clapping to keep the rhythm.
    Detail: Skip counting by 2s, 5s and 10s as rhythm and pattern; the 100-square explored like a map

Month 10: Two-Digit Confidence (Weeks 37-40)
  Adding Two-Digit Numbers
    Topics:
      - Adding two numbers using tens and ones blocks
      - Combining the tens, then the ones
      - Moving from blocks to picturing the jump
      - Adding a whole ten to a number
      - Solving sums like 23 plus 14
      - Checking the answer with the materials
    Practice: Set out blocks for a sum like 22 plus 13 and let your child join the tens and the ones, then say the total.
    Detail: Adding tens, adding ones: two-digit addition with materials first, mental jumps second
  Ten More and Ten Less
    Topics:
      - Finding ten more than a number quickly
      - Finding ten less than a number
      - Using the hundred square to jump by ten
      - Noticing only the tens digit changes
      - Meeting friendly round numbers
      - Jumping to the next friendly ten
    Practice: Say a number like 34 and ask for ten more, then ten less, using the hundred square to check each jump.
    Detail: Ten-more ten-less fluency; friendly-number thinking previewed gently

Month 11: Measurement, Money and Time (Weeks 41-44)
  Measuring and Guessing
    Topics:
      - Comparing lengths as longer and shorter
      - Comparing weights as heavier and lighter
      - Comparing capacity as holds more or less
      - Measuring with repeated units like paper clips
      - Guessing a measurement, then checking it
      - Using words like longest, heaviest and full
    Practice: Guess how many hand-spans long the table is, then measure together to see how close the guess was.
    Detail: Length, weight and capacity by comparing and then by counting units; estimation as a guessing game with feedback
  Money and Telling the Time
    Topics:
      - Recognising a few common coins
      - Paying for small pretend items
      - Counting a small amount of money
      - Reading o'clock on a real clock
      - Reading half-past on a real clock
      - Linking times to parts of the day
    Practice: Play shop with real coins for a few priced items, then check what o'clock it is on a real clock together.
    Detail: Coins and simple shopping play; o'clock and half-past on real clocks

Month 12: Shapes, Data and Graduation (Weeks 45-52)
  Shapes and Fair Shares
    Topics:

- Naming flat shapes like circle, square and triangle
- Naming solid shapes like cube, sphere and cone
- Building and hunting for shapes at home
- Describing shapes by their sides and corners
- Sharing something fairly into halves
- Sharing into quarters as four equal parts

Practice: Go on a shape hunt around the house, then share a snack or a drawing fairly into halves and quarters.
Detail: 2D and 3D shapes built, named and hunted at home; halves and quarters as fair sharing, fractions met kindly

Graphs, Graduation and the Road Ahead
Topics:
- Asking a simple survey question
- Collecting answers from family or class
- Drawing a picture graph of the results
- Reading which answer had the most
- Teaching a favourite math game to others
- Looking ahead to grade-level math

Practice: Help your child survey the family on a fun question, like favourite fruit, and draw a little picture graph of the answers.
Assessment: A gentle final celebration where the child makes a simple pictograph, teaches a favourite math game, and revisits the year's ideas, with a friendly road-ahead plan instead of a test.
Detail: Simple pictographs from the child's own surveys; graduation project, the child teaches their favourite math game, and the road-ahead plan into grade-level math

PROJECTS THE STUDENT BUILDS AND KEEPS

- The child's own counting museum: collections photographed and counted
- Pattern gallery: repeating and growing patterns the child created
- Shape hunt portfolio from around the home
- The child's first survey and pictograph
- Graduation game: the child teaches their favourite math game

Total projects: A year of playful artifacts the child is proud of

SKILLS THE STUDENT WILL HAVE AT THE END

- Counting with one-to-one correspondence and cardinality
- Subitizing and quantity comparison
- Part-whole thinking and bonds of 10
- Addition and subtraction as stories, then facts
- Teen numbers and place-value beginnings
- Numbers to 100 with tens-and-ones understanding
- Early measurement, money, time and shapes
- Explaining mathematical thinking aloud

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes in short playful segments
Home play: Everyday math prompts for parents, minutes, not worksheets
Materials: Household objects as manipulatives; printable ten-frames provided
Total per week: About 2.5 gentle hours
Homework: A problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Live sessions: 8 live classes every month, 2 per week
Mentorship: One dedicated mentor who knows the child's stage and spark
Recordings: Every class recorded
Progress tracking: Stage mastery notes shared with parents
Flexibility: Slots move around naps, school and family life

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies mastery, not attendance

WHERE THIS CAN LEAD

- Ready for Mental Maths Mastery with the bonds already forming
- Ready for the Elementary Mathematics Masterclass, arriving confident
- A child who enters grade 3 liking math, the outcome that compounds for a decade
- Early access to the maths-and-coding world for the builder types

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Can a 4-year-old really learn in online classes?
A: Yes, with the right design, which is the whole craft here. Classes run as short, varied playful segments: a counting game, a building challenge, a story problem, a movement break with skip counting. The mentor works with real objects the child holds up to the camera, and parents of our youngest students sit nearby for the first weeks. The free demo shows you exactly how it works with your child. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Isn't this age too early for structured math?
A: It is exactly the right age for the right kind: playful, concrete, talk-rich mathematics. Research on early number sense is unambiguous, the part-whole and counting ideas built at 4-7 predict mathematics success for years. What would be wrong at this age is worksheets and drilling, which is why we do neither.

Q: How is this different from apps like Khan Kids or busy-work workbooks?
A: Apps entertain and practice; they cannot listen. The engine of early math learning is math talk, a child explaining 'I saw 5 because 4-and-1' to a person who responds. Our mentors have that conversation twice a week, notice exactly which ideas have landed, and adjust in real time. No app does that.

Q: My child already loves numbers. Will this stretch them?
A: Happily, the mentor teaches to stage, not age. A number-loving five-year-old might race into part-whole play and early place value while a classmate consolidates counting. Small groups and 1-on-1 make that flexibility real.

Q: What do parents need to do at home?
A: Kitchens and car rides teach more early math than worksheets ever did, and we show you how to use them.

Q: What comes after this course?
A: Graduates flow into Mental Maths Mastery (for fluency depth) or the Elementary Mathematics Masterclass (the full grades 1-5 program) with the same mentor where possible, arriving with the foundation those courses quietly assume.

Q: How is my child assessed, and is the certificate just for attending?
A: No. A problem set follows every class with written feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final assessment (a friendly, low-pressure check of the whole year). Passing earns the certificate; falling short earns focused revision and a free retest. We measure real understanding, not attendance.

Q: Can we try it before paying?
A: Yes. Every child starts with a free live demo class that doubles as the playful diagnostic, no card details, no obligation, and you will know within the hour whether your child lights up.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/early-math-foundations
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 57 OF 137: ELEMENTARY MATHS FOR GRADES 1-5: LIVE ONLINE CLASSES
==========

Course link: https://learn.modernagecoders.com/courses/elementary-mathematics-complete-masterclass
Subject area: School Mathematics
Designed for: Elementary school, Girls only batches
Who can join: Grades 1-5; every child is placed by grade and a short placement check
Course length: 10-12 months per level (3 levels, Grades 1-5)
Time needed: 2 live classes/week + 20-30 minutes daily practice
Certificate: Level certificate from Modern Age Coders, awarded on passing each level exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Elementary maths classes for Grades 1-5, live and online: placement by grade into three school-year levels, from counting to fractions, geometry and pre-algebra.

In one line: Three honest school-year levels instead of one endless course: your child is placed where they are, and finishes each level actually owning it.

ABOUT THE PROGRAMME

Primary maths is where number confidence is either built or broken, and it cannot be rushed. That is why this program is structured the way good schools structure it: three levels, each one school year long. Level A builds Grades 1-2: counting, place value, addition and subtraction, shapes, time and money. Level B builds Grades 3-4: multiplication and division fluency, fractions, decimals, data and real geometry. Level C completes Grade 5 and builds the bridge to middle school: advanced fractions, ratios, percentages, circle geometry, first algebra and probability. Every child starts with a short, friendly placement check and joins the level that fits, never the one their age assumes. Classes are live and small, and the method is the one the strongest primary curricula in the world use: concrete first (objects and drawings), then pictorial, then abstract. Practice is 20 to 30 minutes daily rather than marathon sessions, because at this age consistency beats intensity every single time. Each level runs 10 months of teaching at two live classes a week, with up to 12 months for children who need more repetition, the pace adapts, never the standard. Learning is checked the honest way: homework after every class, weekly mixed practice that deliberately revisits earlier topics, a mid-level check, and a level exam at the end. Passing earns the level certificate and promotion; a child who falls short gets focused revision and a free retest. Real knowledge, real work, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Placement by level, not by age: a short friendly check decides where your child starts, so nothing is skipped and nothing is repeated
- Concrete, pictorial, abstract: every concept starts with objects and pictures before symbols, the sequence the world's best primary curricula use
- Spiral revision is built in: weekly practice deliberately mixes older topics with new ones, so skills stop fading between terms
- 20-30 minutes of daily practice, not marathon sessions: at this age consistency beats intensity
- Real assessment, gently done: a mid-level check and a level exam decide the certificate, with focused revision and a free retest when needed
- Parents stay in the loop: regular written notes on what was learned, what is next, and exactly how to help at home

THE LEARNING PATH, STEP BY STEP

Phase 1: Level A, Grades 1-2 (one school year): counting and place value, addition and subtraction, first multiplication, shapes, time, money and measurement
Phase 2: Level B, Grades 3-4 (one school year): multiplication and division fluency, fractions, decimals, data and graphing, geometry, area and perimeter
Phase 3: Level C, Grade 5 (one school year): large numbers, advanced fractions, ratios and percentages, circle geometry, first algebra, probability and money maths

WHAT A STUDENT NEEDS BEFORE STARTING

Maths background: None. Level A starts at counting; placement decides the right entry point
Reading: Basic reading helps from Grade 2 up; Level A supports early readers with visual instructions
Equipment: A computer or tablet with a stable internet connection; a notebook and pencil matter more than any app
Age: Grades 1-5 (roughly ages 6-11). Placement is by a short friendly check, never age alone
Time commitment: 2 live classes plus 20-30 minutes of practice a day

WHO THIS COURSE IS FOR

Building foundations: Children in Grades 1-2 whose parents want the foundation laid properly the first time
Catching up: Children in Grades 3-5 with gaps that classroom pace never allows anyone to go back and fix
Moving ahead: Confident children who finish school worksheets early and are hungry for more
Anxious students: Children who have started saying they hate maths, usually one patient level away from changing their mind
Structure seekers: Families who want a real syllabus, real checks and real certificates, not an endless app subscription

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: LEVEL A FOR GRADES 1-2 (ONE SCHOOL YEAR)

The foundation year(s): numbers, place value, addition and subtraction, first multiplication, shapes, time, money and measurement, all learned with objects and pictures before symbols.

Months 1-2: Numbers and Counting (Weeks 1-8)
  Introduction to Numbers and Counting
    Topics:
      - What are numbers? History and importance
      - Counting 1-20: objects, fingers, visual aids
      - Number recognition and writing practice
      - Number words: spelling one to twenty
      - Before, after, and between numbers
      - Counting forward and backward
      - Skip counting by 2s introduction
      - Zero concept and its importance
      - One-to-one correspondence
      - Greater than, less than concepts
      - Equal to understanding
      - Number line introduction 0-20
    Projects built:
      - Create number book with illustrations
      - Number hunt around the house
      - Build counting games with everyday objects
    Practice: 20-30 minutes of playful counting practice a day
  Numbers to 100 and Place Value
    Topics:
      - Counting 21-100 systematically
      - Tens and ones concept introduction
      - Place value understanding with blocks
      - Writing numbers in expanded form
      - Number patterns in 100 chart
      - Skip counting by 2s, 5s, and 10s
      - Even and odd numbers introduction
      - Comparing two-digit numbers
      - Ordering numbers from least to greatest
      - Number bonds to 10
      - Introduction to estimation
      - Rounding to nearest 10
    Projects built:
      - 100 chart coloring patterns
      - Place value manipulative games
      - Create skip counting songs
    Practice: Master place value with 50 daily exercises
  Basic Addition (Single Digit)
    Topics:
      - Addition concept: combining sets
      - Plus sign and equals sign meaning
      - Addition sentences and vocabulary
      - Adding with objects and pictures
      - Addition on number line
      - Addition facts 0-5
      - Addition facts 6-10
      - Doubles facts (1+1, 2+2, etc.)
      - Near doubles strategy
      - Making 10 strategy
      - Commutative property (turn-around facts)
      - Zero property of addition
    Projects built:
      - Addition fact family houses
      - Create addition story problems book
      - Design addition board game
    Practice: Master 100 addition facts through games and drills
  Basic Subtraction (Single Digit)
    Topics:
      - Subtraction as taking away
      - Subtraction as finding difference
      - Minus sign understanding
      - Subtraction sentences
      - Subtraction with objects and pictures
      - Subtraction on number line
      - Subtraction facts 0-10
      - Fact families: addition and subtraction
      - Related facts connection
      - Counting back strategy
      - Counting up strategy
      - Think addition to subtract
    Projects built:
      - Subtraction story problems collection
      - Fact family triangles craft
      - Subtraction bowling game
    Practice: Master subtraction facts through 10
    Assessment: Months 1-2 check: a friendly mixed quiz over everything so far, plus a
    take-home challenge sheet

Months 3-4: Bigger Numbers and First Multiplication (Weeks 9-17)
  Two-Digit Addition
    Topics:
      - Adding multiples of 10
      - Adding two-digit numbers without regrouping
      - Vertical addition alignment
      - Adding ones, then tens strategy
      - Breaking apart numbers to add
      - Addition with regrouping introduction
      - Carrying concept explanation
      - Using base-10 blocks for regrouping
      - Addition with multiple addends
      - Checking addition with subtraction
      - Mental math strategies for addition
      - Estimating sums
    Projects built:
      - Shopping mall addition game
      - Create addition puzzle cards
      - Design digital addition calculator
    Practice: Solve 200 two-digit addition problems
  Two-Digit Subtraction
    Topics:
      - Subtracting multiples of 10
      - Two-digit subtraction without regrouping
      - Vertical subtraction setup
      - Subtract ones, then tens
      - Subtraction with regrouping introduction
      - Borrowing/regrouping concept
      - Using manipulatives for regrouping
      - Checking subtraction with addition
      - Subtraction across zeros
      - Mental math for subtraction
      - Estimating differences
      - Problem-solving with subtraction
    Projects built:
      - Subtraction treasure hunt
      - Create subtraction word problem cards
      - Build subtraction machine model
    Practice: Master 200 two-digit subtraction problems
  Introduction to Multiplication
    Topics:
      - Multiplication as repeated addition
      - Groups and arrays understanding
      - Times sign and multiplication vocabulary
      - Multiplication sentences
      - Skip counting connection
      - Multiplication by 0 and 1
      - Multiplication by 2 and 5
      - Multiplication by 10
      - Doubles in multiplication
      - Using arrays to multiply
      - Commutative property of multiplication
      - Building multiplication tables
    Projects built:
      - Multiplication array art
      - Times tables wheel craft
      - Create multiplication story book
    Practice: Learn multiplication facts 0-5
  Geometry Basics: 2D Shapes
    Topics:
      - Circle: properties and examples
      - Triangle: types and properties
      - Square vs rectangle comparison
      - Pentagon, hexagon, octagon
      - Sides and vertices counting
      - Straight vs curved lines
      - Open vs closed shapes
      - Symmetry introduction
      - Lines of symmetry
      - Creating patterns with shapes
      - Combining shapes to make pictures
      - Shape sorting and classification
    Projects built:
      - Shape collage artwork
      - Geometry in nature scavenger hunt
      - Build shapes with craft sticks
    Practice: Identify and draw 50 different shapes
  Mid-Level Review

```
    Topics:
      - Operations review
      - Problem-solving strategies review
      - Mental math techniques
      - Geometry concepts recap
      - Real-world applications
    Projects built:
      - Create math game combining all concepts
      - Design math puzzle book
      - Build interactive math display
    Assessment: Mid-level check: mixed questions from months 1-4, so earlier skills are
    proven, not assumed

Months 5-6: Time, Money and Measurement (Weeks 18-26)
  Time and Calendar
    Topics:
      - Clock reading: hour hand understanding
      - Clock reading: minute hand understanding
      - Reading time to the hour
      - Reading time to the half hour
      - Quarter past and quarter to
      - Reading time to 5 minutes
      - AM and PM understanding
      - Days of the week sequence
      - Months of the year
      - Reading calendar dates
      - Yesterday, today, tomorrow
      - Elapsed time simple problems
    Projects built:
      - Create personal daily schedule
      - Build paper plate clock
      - Design calendar for the year
    Practice: Read 100 different clock times
  Money and Financial Literacy
    Topics:
      - Coin identification and values
      - Penny, nickel, dime, quarter
      - Dollar bills introduction
      - Counting coins same type
      - Counting mixed coins
      - Making change basics
      - Dollar and cents notation
      - Adding money amounts
      - Subtracting money amounts
      - Word problems with money
      - Needs vs wants concept
      - Simple budgeting introduction
    Projects built:
      - Classroom store simulation
      - Create money math workbook
      - Design piggy bank savings tracker
    Practice: Solve 100 money word problems
  Length and Height Measurement
    Topics:
      - Non-standard units measurement
      - Introduction to ruler use
      - Inches and feet
      - Centimeters and meters
      - Measuring classroom objects
      - Comparing lengths
      - Ordering by length
      - Estimating lengths
      - Perimeter introduction
      - Measuring curved lines
      - Height measurement
      - Distance concepts
    Projects built:
      - Measurement scavenger hunt
      - Create measurement book
      - Build measuring tools
    Practice: Measure 50 objects accurately
  Weight and Capacity
    Topics:
      - Heavy and light comparison
      - Balance scale understanding
      - Pounds and ounces introduction
      - Kilograms and grams basics
      - Weighing common objects
      - Capacity: full, empty, half
```

- Cups, pints, quarts, gallons
- Liters and milliliters
- Comparing capacities
- Estimating weight and capacity
- Temperature introduction
- Reading thermometer

Projects built:
- Kitchen measurement experiments
- Create weight comparison chart
- Design capacity containers game

Practice: Complete 50 measurement activities

Level A Exam and Celebration

Topics:
- Integration of all Phase 1 concepts
- Real-world problem solving
- Project presentation skills
- Mathematical communication
- Reflection on learning

Projects built:
- A proud portfolio of the year's best work, shown to parents

Assessment: LEVEL A EXAM: a friendly written paper plus a short viva, with questions mixed from the whole level. Passing earns the Level A certificate and promotion to Level B; a child who falls short gets focused revision and a free retest

---

PHASE 2: LEVEL B FOR GRADES 3-4 (ONE SCHOOL YEAR)

The fluency year(s): multiplication and division mastered properly, fractions and decimals understood instead of memorized, plus data, geometry, area and perimeter.

Months 1-2: Multiplication and Division Mastery (Weeks 1-9)

Advanced Multiplication Facts

Topics:
- Multiplication facts 6-9
- Square numbers understanding
- Patterns in multiplication table
- Tricks for 9s facts
- Multiplication by 11 and 12
- Distributive property introduction
- Breaking apart to multiply
- Doubling and halving strategy
- Multiplication fact fluency drills
- Timed multiplication tests
- Multiplication games and competitions
- Real-world multiplication applications

Projects built:
- Multiplication facts championship
- Create multiplication tricks book
- Design multiplication app prototype

Practice: Master all multiplication facts 0-12

Multi-Digit Multiplication

Topics:
- Multiplying by 10, 100, 1000
- Two-digit by one-digit multiplication
- Partial products method
- Standard algorithm introduction
- Area model for multiplication
- Lattice multiplication method
- Three-digit by one-digit
- Two-digit by two-digit basics
- Estimating products
- Checking multiplication with division
- Word problems with multiplication
- Multi-step problems

Projects built:
- Area model visual guide
- Multiplication method comparison
- Create multiplication story problems

Practice: Solve 200 multi-digit multiplication problems

Introduction to Division

Topics:
- Division as sharing equally
- Division as repeated subtraction
- Division symbol and vocabulary
- Dividend, divisor, quotient
- Division facts related to multiplication
- Dividing by 1 and itself
- Division by 2, 5, and 10
- Using multiplication to divide

- Fact families with multiplication/division
- Remainders introduction
- Division on number line
- Array model for division

Projects built:

- Division fact family houses
- Create division story book
- Design fair sharing game

Practice: Master basic division facts

Long Division

Topics:

- Long division setup
- Divide, multiply, subtract, bring down
- Two-digit divided by one-digit
- Handling remainders
- Checking division with multiplication
- Three-digit divided by one-digit
- Zeros in quotient
- Estimating quotients
- Division word problems
- Interpreting remainders in context
- Mental math division strategies
- Division patterns

Projects built:

- Long division step-by-step guide
- Division problem generator
- Create division board game

Practice: Complete 150 long division problems

Problem-Solving with Operations

Topics:

- Choosing the correct operation
- Multi-step word problems
- Working backward strategy
- Guess and check method
- Drawing diagrams to solve
- Making tables strategy
- Finding patterns in problems
- Writing equations from words
- Checking reasonableness
- Problem-solving plan steps
- Creating own word problems
- Real-world applications

Projects built:

- Problem-solving strategy poster
- Word problem collection book
- Math detective game

Practice: Solve 100 multi-step word problems

Assessment: Months 1-2 check: mixed word problems using all four operations, reaching back to Level A skills

Months 3-4: Fractions and Decimals (Weeks 10-18)

Understanding Fractions

Topics:

- Fraction as part of whole
- Numerator and denominator meaning
- Unit fractions understanding
- Halves, thirds, fourths
- Fifths, sixths, eighths
- Fractions on number line
- Comparing fractions same denominator
- Comparing fractions same numerator
- Equivalent fractions with models
- Simplest form introduction
- Mixed numbers introduction
- Improper fractions basics

Projects built:

- Fraction pizza party
- Fraction art with paper folding
- Create fraction manipulatives

Practice: Short daily practice sets, checked and celebrated in class

Operations with Fractions

Topics:

- Adding fractions same denominator
- Subtracting fractions same denominator
- Adding mixed numbers simple
- Subtracting mixed numbers simple
- Finding common denominators
- Adding unlike fractions basics
- Subtracting unlike fractions basics

- Multiplying fraction by whole number
- Fraction of a set
- Word problems with fractions
- Estimating with fractions
- Fractions in measurement

Projects built:

- Fraction operations cookbook
- Fraction calculator design
- Fraction word problem collection

Practice: Complete 150 fraction operations

Introduction to Decimals

Topics:

- Decimal point meaning
- Tenths place understanding
- Hundredths place understanding
- Reading decimal numbers
- Writing decimals in words
- Decimals on number line
- Comparing decimals
- Ordering decimals
- Rounding decimals to whole
- Rounding to nearest tenth
- Fractions to decimals (tenths, hundredths)
- Money as decimals

Projects built:

- Decimal place value chart
- Decimal number line display
- Create decimal comparison game

Practice: Short daily practice sets, checked and celebrated in class

Decimal Operations

Topics:

- Adding decimals with same places
- Subtracting decimals same places
- Adding decimals different places
- Lining up decimal points
- Adding money with decimals
- Subtracting money with decimals
- Multiplying decimals by 10, 100
- Multiplying decimal by whole number
- Estimating with decimals
- Word problems with decimals
- Decimals in measurement
- Calculator use with decimals

Projects built:

- Decimal shopping simulation
- Create decimal workbook
- Design decimal maze game

Practice: Solve 150 decimal problems

Data and Graphing

Topics:

- Collecting and organizing data
- Tally charts and frequency tables
- Pictographs reading and creating
- Bar graphs horizontal and vertical
- Line plots introduction
- Reading and interpreting graphs
- Finding range, mode, median
- Mean (average) introduction
- Creating surveys
- Double bar graphs
- Making predictions from data
- Data in real world

Projects built:

- Class survey project
- Create graphing portfolio
- Design data collection app

Practice: Create and interpret 30 different graphs

Assessment: Months 3-4 check: a mixed fractions-and-decimals paper with a data question, plus revision of months 1-2

Months 5-6: Geometry, Area and the Level Exam (Weeks 19-26)

Advanced 2D Geometry

Topics:

- Types of angles: acute, right, obtuse
- Measuring angles with protractor
- Drawing angles
- Parallel and perpendicular lines
- Types of triangles by sides
- Types of triangles by angles

- Quadrilaterals classification
- Properties of rectangles, squares
- Rhombus and parallelogram
- Trapezoid properties
- Congruent shapes
- Similar shapes basics

Projects built:
- Geometry art portfolio
- Angle measurement tools
- Create geometry handbook

Practice: Complete 100 geometry exercises

3D Shapes and Volume

Topics:
- 3D shapes: cube, rectangular prism
- Sphere, cylinder, cone
- Pyramid types
- Faces, edges, vertices counting
- Nets of 3D shapes
- Building 3D shapes from nets
- Volume as counting cubes
- Volume of rectangular prism
- Surface area introduction
- Real-world 3D shapes
- Cross-sections basics
- 3D shape sorting

Projects built:
- 3D shape construction kit
- Volume exploration lab
- Create 3D shape museum

Practice: Build and analyze 30 3D shapes

Area and Perimeter

Topics:
- Perimeter of polygons
- Perimeter word problems
- Area by counting squares
- Area of rectangle formula
- Area of square
- Area of triangle introduction
- Area of parallelogram
- Composite figures area
- Perimeter of composite figures
- Area vs perimeter understanding
- Real-world area problems
- Estimating area and perimeter

Projects built:
- Design dream bedroom with area
- Perimeter and area games
- Create area/perimeter reference

Practice: Solve 100 area/perimeter problems

Coordinate Geometry

Topics:
- Coordinate plane introduction
- X-axis and y-axis
- Ordered pairs (x, y)
- Plotting points on grid
- Reading coordinates
- First quadrant focus
- Creating pictures with coordinates
- Distance between points (horizontal/vertical)
- Simple transformations: slides
- Reflections across axes
- Coordinate battleship game
- Real-world coordinate uses

Projects built:
- Coordinate art gallery
- Treasure map with coordinates
- Design coordinate game

Practice: Plot and identify 100 coordinate points

Level B Exam and Celebration

Topics:
- Integration of Phase 2 concepts
- Complex problem-solving
- Mathematical reasoning
- Project presentation
- Self-assessment and reflection

Projects built:
- A geometry-and-fractions portfolio of the year's best work

Assessment: LEVEL B EXAM: a written paper plus a short viva, with questions mixed from the whole level and a few reaching back to Level A. Passing earns the Level B

certificate and promotion to Level C; a child who falls short gets focused revision and a free retest

---

PHASE 3: LEVEL C FOR GRADE 5 (ONE SCHOOL YEAR)

The bridge year: large numbers, advanced fractions, ratios and percentages, circle geometry, first algebra, probability and money maths, everything middle school assumes.

Months 1-2: Numbers and Fractions Mastery (Weeks 1-8)

Large Numbers and Place Value

Topics:

- Numbers to millions
- Place value to millions
- Reading and writing large numbers
- Expanded form to millions
- Comparing large numbers
- Ordering large numbers
- Rounding to any place value
- Scientific notation introduction
- Powers of 10 understanding
- Number patterns in large numbers
- Estimation with large numbers
- Real-world large numbers

Projects built:

- Large number reference book
- Population studies project
- Create million visualization

Practice: Short daily practice sets, checked and celebrated in class

Factors and Multiples

Topics:

- Factors definition and finding
- Factor pairs
- Prime and composite numbers
- Prime factorization
- Factor trees
- Greatest Common Factor (GCF)
- Multiples and common multiples
- Least Common Multiple (LCM)
- Using GCF and LCM
- Divisibility rules 2, 3, 5, 9, 10
- Perfect squares
- Square roots introduction

Projects built:

- Factor and multiple game
- Prime number sieve activity
- Create divisibility rule guide

Practice: Find factors and multiples of 100 numbers

Complex Fraction Operations

Topics:

- Adding mixed numbers with regrouping
- Subtracting mixed numbers with borrowing
- Finding least common denominator
- Adding three or more fractions
- Fraction multiplication understanding
- Multiplying fractions by fractions
- Multiplying mixed numbers
- Reciprocals introduction
- Division of fractions basics
- Dividing whole numbers by fractions
- Complex fraction word problems
- Fractions in recipes and scaling

Projects built:

- Fraction operations cookbook
- Fraction calculator model
- Create fraction game show

Practice: Master 200 complex fraction problems

Advanced Decimal Operations

Topics:

- Multiplying decimals by decimals
- Division with decimals as divisor
- Repeating decimals introduction
- Terminating vs repeating
- Converting fractions to decimals
- Converting decimals to fractions
- Percent as special decimal
- Scientific notation with decimals
- Significant figures basics
- Decimal word problems complex

- Decimals in science contexts
- Calculator strategies with decimals

Projects built:

- Decimal conversion chart
- Scientific measurement project
- Design decimal Olympics

Practice: Complete 200 decimal operations

Assessment: Months 1-2 check: mixed numbers-and-fractions paper, with revision questions from Level B

Months 3-4: Proportional Thinking and Geometry (Weeks 9-16)

Ratios and Proportions

Topics:

- Ratio concept and notation
- Writing ratios three ways
- Equivalent ratios
- Simplifying ratios
- Ratio tables
- Rates and unit rates
- Price per unit calculations
- Proportion definition
- Solving proportions
- Cross multiplication
- Scale drawings and maps
- Similar figures and ratios

Projects built:

- Recipe scaling project
- Map scale investigation
- Create ratio/proportion game

Practice: Solve 150 ratio and proportion problems

Percentages

Topics:

- Percent meaning and symbol
- Percent as per hundred
- Converting percent to decimal
- Converting percent to fraction
- Finding percent of a number
- Mental math with percentages
- 10%, 25%, 50%, 75% shortcuts
- Percent increase introduction
- Percent decrease introduction
- Sales tax calculations
- Discount calculations
- Tips and gratuity

Projects built:

- Shopping with percentages
- Create percent reference guide
- Design discount calculator

Practice: Master 150 percentage problems

Advanced Angle Relationships

Topics:

- Complementary angles
- Supplementary angles
- Vertical angles
- Adjacent angles
- Angles in triangles sum to 180°
- Finding missing angles
- Angles in quadrilaterals
- Interior angles of polygons
- Exterior angles basics
- Angle bisectors
- Perpendicular bisectors
- Constructions with compass

Projects built:

- Angle relationship poster
- Geometric construction kit
- Create angle puzzle game

Practice: Solve 100 angle problems

Circle Geometry

Topics:

- Circle vocabulary: radius, diameter
- Chord and arc definitions
- Center, circumference understanding
- Pi introduction and history
- Circumference formula $C = \pi d$
- Circumference formula $C = 2\pi r$
- Area of circle $A = \pi r^2$
- Finding radius from circumference
- Finding radius from area

```
      - Semicircles and quarter circles
      - Composite figures with circles
      - Real-world circle problems
    Projects built:
      - Pi Day celebration materials
      - Circle measurement lab
      - Design circular art project
    Practice: Solve 100 circle problems
    Assessment: Months 3-4 check: ratios, percentages and geometry mixed together, the way
    real problems arrive

Months 5-6: First Algebra, Data and the Level Exam (Weeks 17-26)
  Variables and Expressions
    Topics:
      - Variable as unknown number
      - Writing algebraic expressions
      - Translating words to algebra
      - Evaluating expressions
      - Substitution method
      - Terms and coefficients
      - Like terms identification
      - Combining like terms
      - Distributive property with variables
      - Simplifying expressions
      - Expression word problems
      - Creating expressions from patterns
    Projects built:
      - Algebra translation dictionary
      - Expression puzzle cards
      - Create algebra game
    Practice: Write and evaluate 30 expressions over the fortnight, a few each day
  Simple Equations
    Topics:
      - Equation vs expression
      - Solving one-step addition equations
      - Solving one-step subtraction equations
      - Solving one-step multiplication equations
      - Solving one-step division equations
      - Inverse operations concept
      - Checking solutions
      - Two-step equations introduction
      - Equations with fractions basics
      - Equations with decimals basics
      - Word problems to equations
      - Real-world equation applications
    Projects built:
      - Equation solving guide
      - Balance scale demonstrations
      - Create equation puzzle book
    Practice: Short daily practice sets, checked and celebrated in class
  Advanced Data Analysis
    Topics:
      - Mean, median, mode review
      - Range and outliers
      - Quartiles introduction
      - Box-and-whisker plots
      - Stem-and-leaf plots
      - Frequency distributions
      - Histograms
      - Circle graphs (pie charts)
      - Choosing appropriate graphs
      - Misleading graphs identification
      - Data collection methods
      - Surveys and sampling basics
    Projects built:
      - Statistical investigation project
      - Data visualization portfolio
      - Create statistics infographic
    Practice: Analyze 50 data sets
  Probability Fundamentals
    Topics:
      - Probability as likelihood
      - Probability scale 0 to 1
      - Certain, likely, unlikely, impossible
      - Experimental probability
      - Theoretical probability
      - Favorable outcomes
      - Sample space
      - Simple probability calculations
```

- Probability with dice
- Probability with cards
- Probability with spinners
- Compound events introduction

Projects built:

- Probability experiment lab
- Design probability games
- Create probability reference

Practice: Calculate 100 probabilities

Money Maths, Review and the Level C Exam

Topics:

- Simple interest calculation
- Compound interest introduction
- Savings and investments basics
- Budget creation and management
- Profit and loss
- Markup and markdown
- Commission calculations
- Tax calculations advanced
- Unit price comparisons
- Best buy problems
- Currency exchange basics
- Financial planning introduction
- Full-course spiral review: the greatest hits of all three levels
- Exam technique for children: reading twice, showing working, checking once

Projects built:

- A real family budget project, planned and presented
- The Level C portfolio: the year's best work

Practice: Daily mixed revision sets in the final fortnight

Assessment: LEVEL C EXAM: a written paper plus a short viva, with questions mixed from the whole level and earlier ones. Passing earns the Level C certificate, completing the elementary program ready for middle school; a child who falls short gets focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Level A: number books, counting games, shape art, time and money projects, a measurement lab at home
- Level B: multiplication championships, fraction cookbooks, decimal shopping games, geometry portfolios
- Level C: percentage shopping challenges, circle geometry art, algebra puzzle cards, probability experiments, a family budget project

Total projects: 60+ hands-on maths projects and games across the three levels

SKILLS THE STUDENT WILL HAVE AT THE END

- Number sense: counting, place value, mental strategies and the confidence to estimate
- Operations: addition, subtraction, multiplication and division to real fluency
- Fractions and decimals: understood with models first, then mastered in operations
- Measurement and money: time, length, weight, capacity and practical money maths
- Geometry: 2D and 3D shapes, angles, symmetry, circles, area and perimeter
- Data and probability: reading, drawing and questioning graphs, first probability
- First algebra: variables, expressions and simple equations, the middle-school bridge
- Problem solving: bar models, working backwards, and explaining the why out loud

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, small groups, taught with objects and visuals before symbols

Homework: A short homework sheet after every class, and weekly mixed practice that revisits earlier topics

Practice: 20-30 minutes of daily practice, structured as games and challenge sets

Review: Every sheet checked with written feedback; wobbly topics reopened in class before they become gaps

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, for the homework question that turns into tears

Progress updates: Regular written notes to parents: what was learned, what comes next, and exactly how to help at home

CERTIFICATION DETAIL

Completion: A level certificate at the end of each level, awarded on passing the level exam (a written paper plus a friendly viva). Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Middle School Mathematics: the direct continuation, entered with a tested foundation
- Mental Maths Mastery: speed and number sense built alongside or after any level
- Olympiad and competition maths: for Level C graduates who found the puzzles delicious
- Maths Through Coding: where the numbers start moving on screen
- School confidence, immediately: the level system mirrors and reinforces classroom maths all year

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which level will my child join?
A: A short, friendly placement check decides, never age alone. Level A builds Grades 1-2 (counting to first multiplication, time, money), Level B builds Grades 3-4 (multiplication and division fluency, fractions, decimals, geometry), and Level C completes Grade 5 (ratios, percentages, first algebra, probability). Most children join the level matching their grade; children with gaps start where the gaps are, because skipping holes in maths only buries them.

Q: How long does each level take?
A: Each level is one school year: 10 months of teaching at 2 live classes a week, with up to 12 months for children who need more repetition. That margin is deliberate, at this age the pace adapts to the child, never the other way around. A child who starts in Level A and continues through Level C finishes elementary maths genuinely owned, not skimmed.

Q: How is my child assessed? Is the certificate just for attending?
A: No. There is classwork in every session, a short homework sheet after every class, weekly mixed practice that deliberately revisits earlier topics, a friendly quiz at each month-pair, a mid-level check, and a level exam at the end: a written paper plus a short viva. Passing earns the level certificate and promotion; a child who falls short gets focused revision and a free retest. Gentle in tone, real in standard.

Q: My child struggles with maths anxiety. How does this course help?
A: By removing the two causes: gaps and pace. Placement starts the child where their gaps actually are, and every concept begins with objects and pictures before symbols, so understanding always precedes speed. Classes are small, wins are celebrated, and practice is 20 to 30 minutes daily rather than exhausting marathons. Most anxious children change their mind about maths within one level, because the anxiety was never about maths, it was about being lost.

Q: How is this different from school tuition?
A: Tuition usually chases this week's homework. This is a complete levelled curriculum with its own arc: placement, spiral revision, monthly checks and a level exam, so gaps get found and closed instead of papered over. It runs alongside any school board comfortably, CBSE, ICSE or international, because Grades 1-5 maths is the same mathematics everywhere.

Q: How much time does it take each week?
A: Two live classes a week plus 20 to 30 minutes of daily practice, usually structured as games and short challenge sets. At this age consistency beats intensity: twenty focused daily minutes outperform a two-hour weekend session every time, and it never becomes a fight at the kitchen table.

Q: Will this prepare my child for olympiads?
A: It builds the foundation olympiads stand on: real fluency, flexible problem solving and comfort with multi-step reasoning. Level C includes genuine puzzle work. Children who love that side graduate naturally into our dedicated olympiad mathematics courses, which train competition technique specifically.

Q: Is there a free trial class?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. The placement check can happen in the same week, and parents are welcome to sit in on both. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Every class is live: a small group with a real teacher who sees your child's working in real time, never a pre-recorded video. At this age the teacher's eyes matter most, catching the finger-counting, the reversed digits, the guess dressed as an answer, and fixing each gently before it becomes a habit.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/elementary-mathematics-complete-masterclass
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 58 OF 137: ETHICAL HACKING COURSE: AUTHORIZED PENTESTING, ZERO TO PRO
==========

Course link: https://learn.modernagecoders.com/courses/ethical-hacking-masterclass-complete
Subject area: Cybersecurity
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Ethical hacking and cybersecurity course for adults: networking and Linux, authorized penetration testing, web and network exploitation, red and blue team skills, and professional reporting, taught legally and ethically, in 10-12 months.

  In one line: Break in so you can defend: authorized penetration testing taught the professional way, from networking and Linux to red team operations and reports that get vulnerabilities fixed.

ABOUT THE PROGRAMME

  Ethical hacking is authorized, legal security testing done to find and fix weaknesses before criminals do, and the emphasis in this course is on the word ethical. It is built for people who want a real cybersecurity career, penetration tester, security analyst, red or blue team engineer, and it starts every offensive skill from a foundation of authorization, scope and law. The path is complete and in order: networking, Linux and programming for security, reconnaissance and scanning, then core exploitation, web and network attacks, Metasploit, the OWASP Top 10, post-exploitation, all practised only in lab environments and authorized ranges, then advanced work: buffer overflows and mitigation bypasses, cryptography, red team operations, and the defensive side, DevSecOps, threat hunting, incident response and forensics, so you understand both attack and defence. A finale month runs a full authorized penetration test of a lab and writes the professional report, then sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of lab practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

  - Ethics and authorization first, not as a footnote: scope, consent and law frame every technique
  - Everything practised in labs and authorized ranges only, never against systems you do not own
  - Both sides taught: offensive skills and the defensive DevSecOps, threat-hunting and forensics response
  - Professional reporting is a core skill: findings that actually get fixed, not just exploits
  - An honest 10-month arc with 2 months in hand
  - Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

  Phase 1: Foundations and recon (Months 1-3): networking, Linux, programming, ethics, OSINT and scanning
  Phase 2: Core exploitation (Months 4-6): system, web and network attacks, post-exploitation, advanced domains
  Phase 3: Advanced and defensive (Months 7-9): exploit development, red team, DevSecOps, threat hunting, forensics
  Phase 4: The finale (Month 10): a full authorized lab pentest, the professional report, and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

  Coding experience: None. Everything is taught from the first line
  Equipment: Any computer with a stable internet connection; every tool used is free
  Age: 18+; college students, graduates and working professionals
  Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

  Aspiring pentesters: Adults who want a real penetration-testing or security career
  It professionals: IT and network staff moving into security

Developers: Developers who want to build and test secure systems
Soc bound: Future SOC analysts who need to understand attacks to defend against them
Cert seekers: Learners preparing for CEH, OSCP and similar certifications

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS, ETHICS AND RECONNAISSANCE (MONTHS 1-3, WEEKS 1-13)

Build rock-solid networking and Linux foundations essential for ethical hacking. Master TCP/IP, operating systems, programming basics, and security fundamentals.

Months 1-2: Networking & Linux Fundamentals (Weeks 1-8)
Introduction to Ethical Hacking & Networking Basics
Topics:
- What is ethical hacking and why it matters
- Legal considerations and ethical guidelines
- Hacking methodology: reconnaissance to reporting
- Setting up hacking lab environment safely
- VirtualBox/VMware installation and configuration
- Kali Linux installation and initial setup
- OSI model deep dive: all 7 layers
- TCP/IP protocol suite fundamentals
- IP addressing and subnetting mastery
- IPv4 vs IPv6 differences and implications
- MAC addresses and ARP protocol
- Network devices: routers, switches, firewalls
- DNS fundamentals and exploitation basics
- HTTP/HTTPS protocol analysis
- Common ports and services
Projects built:
- Complete hacking lab setup with multiple VMs
- Network topology design and implementation
- Subnet calculator tool development
- Protocol analyzer using Wireshark
Practice: Configure 10 different network scenarios, analyze 50 packet captures
Linux Mastery for Hackers
Topics:
- Linux file system hierarchy
- Essential Linux commands for hackers
- File permissions and ownership
- User and group management
- Process management and monitoring
- Package management (apt, yum, dpkg)
- Shell scripting fundamentals
- Bash scripting for automation
- Text manipulation: grep, sed, awk
- Network commands: netstat, ss, lsof
- Service management with systemctl
- Log file analysis and monitoring
- Cron jobs and task scheduling
- SSH configuration and hardening
- Linux security basics
Projects built:
- Automation scripts for reconnaissance
- Custom security monitoring dashboard
- Log analysis automation tool
- Linux hardening checklist implementation
Practice: Complete 100 Linux challenges, write 20 bash scripts
Programming for Hackers - Python & Web Basics
Topics:
- Python fundamentals for security
- Socket programming in Python
- Network programming with Scapy
- Web scraping with BeautifulSoup
- Requests library for HTTP interactions
- Regular expressions for pattern matching
- HTML, CSS, JavaScript basics
- Understanding web application architecture
- Client-server communication model
- RESTful APIs and JSON
- SQL fundamentals and injection basics
- XML and XXE vulnerability basics
- Encoding and encryption basics
- Hashing algorithms and their weaknesses
- Cryptography fundamentals
Projects built:

- Port scanner in Python
- Password cracker for weak hashes
- Web crawler for reconnaissance
- Network packet sniffer
- Simple vulnerability scanner

Practice: Write 30 security tools in Python

Information Gathering & OSINT

Topics:

- Open Source Intelligence (OSINT) fundamentals
- Google dorking and advanced search operators
- Social media reconnaissance techniques
- Email harvesting and verification
- DNS enumeration and zone transfers
- Subdomain discovery techniques
- WHOIS and domain information gathering
- Shodan for device discovery
- Censys and other search engines
- Metadata extraction from documents
- Website technology identification
- Company and employee reconnaissance
- Physical location information gathering
- Dark web basics for intelligence
- OSINT frameworks and automation

Projects built:

- Complete OSINT profile builder
- Automated reconnaissance framework
- Company security assessment report
- Social engineering information gatherer
- Custom OSINT tool development

Practice: Perform OSINT on 20 targets (legally and ethically)

Month 3: Scanning & Enumeration (Weeks 9-13)

Network Scanning & Enumeration

Topics:

- Network scanning methodology
- Nmap fundamentals and advanced techniques
- TCP connect and SYN stealth scanning
- UDP scanning challenges and techniques
- Service version detection
- Operating system fingerprinting
- Nmap Scripting Engine (NSE)
- Vulnerability scanning with Nessus
- OpenVAS configuration and usage
- Banner grabbing techniques
- SNMP enumeration
- SMB/NetBIOS enumeration
- LDAP enumeration
- NFS enumeration
- Database enumeration

Projects built:

- Network mapping automation tool
- Vulnerability assessment report generator
- Custom Nmap scripts development
- Enumeration framework creation
- Automated scanning pipeline

Practice: Scan and enumerate 30 different networks/systems

Web Application Scanning

Topics:

- Web application architecture review
- Burp Suite Professional setup
- Proxy configuration and SSL handling
- Spidering and crawling applications
- Directory and file brute forcing
- Parameter discovery and fuzzing
- Technology stack identification
- Web application firewalls (WAF) detection
- API endpoint discovery
- JavaScript analysis for endpoints
- Source code review basics
- Automated scanning with Burp Scanner
- OWASP ZAP usage and automation
- Nikto and dirb for web scanning
- Custom wordlist creation

Projects built:

- Web application reconnaissance tool
- API endpoint discovery framework
- WAF detection and bypass tool
- Custom web fuzzer development
- Automated web scanning report

      Practice: Scan 25 web applications, find 100 endpoints
    Vulnerability Assessment
      Topics:
        - Vulnerability assessment methodology
        - CVSS scoring understanding
        - CVE database navigation
        - Exploit database usage
        - Manual vulnerability verification
        - False positive identification
        - Risk assessment and prioritization
        - Vulnerability chaining concepts
        - Patch management basics
        - Compliance scanning (PCI DSS, HIPAA)
        - Creating vulnerability matrices
        - Remediation recommendations
        - Phase 1 comprehensive review
      Projects built:
        - PHASE 1 CAPSTONE: Complete Security Assessment
        - Target: Vulnerable VM (Metasploitable, DVWA)
        - Tasks: Full recon, scanning, enumeration, vulnerability assessment, professional report
      Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

PHASE 2: CORE EXPLOITATION AND ADVANCED DOMAINS (MONTHS 4-6, WEEKS 14-26)

Master system hacking, web application attacks, exploitation frameworks, and
post-exploitation techniques.

  Month 4: System Hacking & Exploitation (Weeks 14-18)
    System Hacking & Password Attacks
      Topics:
        - Windows architecture and vulnerabilities
        - Linux privilege escalation techniques
        - Password attack methodology
        - Rainbow tables and time-memory trade-off
        - John the Ripper mastery
        - Hashcat for GPU cracking
        - Hydra for online attacks
        - Pass-the-hash attacks
        - Kerberoasting and ASREPRoasting
        - Credential dumping with Mimikatz
        - SAM and SYSTEM file extraction
        - Keyloggers and screenshot capture
        - Password policy bypassing
        - Multi-factor authentication bypasses
        - Physical access attacks
      Projects built:
        - Password cracking framework
        - Credential harvesting tool
        - Windows privilege escalation script
        - Linux privilege escalation checklist
        - Custom keylogger development
      Practice: Crack 50 passwords, escalate privileges on 20 systems
    Metasploit Framework & Exploitation
      Topics:
        - Metasploit architecture and components
        - MSFconsole navigation and usage
        - Exploit selection and configuration
        - Payload types and encoding
        - Meterpreter advanced features
        - Post-exploitation with Metasploit
        - Persistence mechanisms
        - Pivoting through compromised hosts
        - Port forwarding and tunneling
        - Custom exploit development basics
        - Ruby for Metasploit modules
        - Evasion techniques and encoding
        - Anti-virus bypass methods
        - Exploit chaining strategies
        - Manual exploitation techniques
      Projects built:
        - Custom Metasploit module
        - Automated exploitation framework
        - Post-exploitation toolkit
        - Persistence mechanism generator
        - AV evasion payload creator
        - Exploit chain automation
        - Pivoting lab setup

```
    Practice: Exploit 30 vulnerable systems, develop 10 custom payloads
  Web Application Hacking - OWASP Top 10
    Topics:
      - SQL Injection: union, blind, time-based
      - SQLMap advanced usage
      - Cross-Site Scripting (XSS): reflected, stored, DOM
      - XSS filter bypasses and polyglots
      - Cross-Site Request Forgery (CSRF)
      - XML External Entity (XXE) attacks
      - Server-Side Request Forgery (SSRF)
      - Insecure Deserialization exploitation
      - Command injection techniques
      - Directory traversal and LFI/RFI
      - File upload vulnerabilities
      - Authentication and session flaws
      - Broken access control exploitation
      - Security misconfiguration attacks
      - Using Burp Suite for exploitation
    Projects built:
      - SQL injection automation tool
      - XSS payload generator
      - Web shell development
      - Session hijacking tool
      - Automated OWASP scanner
      - Custom Burp extension
      - Web exploitation framework
    Practice: Exploit 50 web vulnerabilities across all OWASP Top 10
  Advanced Web Attacks & API Security
    Topics:
      - Modern JavaScript framework vulnerabilities
      - React, Angular, Vue.js security issues
      - WebSocket vulnerabilities
      - GraphQL injection and attacks
      - REST API vulnerabilities
      - JWT token manipulation
      - OAuth and SAML attacks
      - Race conditions and TOCTOU
      - Cache poisoning attacks
      - HTTP request smuggling
      - Subdomain takeover techniques
      - CORS misconfiguration exploitation
      - Business logic vulnerabilities
      - Payment gateway bypasses
      - Two-factor authentication bypasses
    Projects built:
      - API security testing framework
      - JWT token manipulation tool
      - Race condition exploit tool
      - Business logic flaw detector
      - Modern web app scanner
    Practice: Test 20 APIs, find 30 logic flaws
  Database Hacking & NoSQL Injection
    Topics:
      - Database architecture and weaknesses
      - Advanced SQL injection techniques
      - Oracle, MySQL, PostgreSQL specific attacks
      - MongoDB injection attacks
      - Redis exploitation techniques
      - Elasticsearch vulnerabilities
      - Database privilege escalation
      - Data exfiltration techniques
      - Blind injection automation
      - Time-based extraction optimization
      - NoSQL injection payloads
      - GraphQL database attacks
      - ORM injection vulnerabilities
    Projects built:
      - Database exploitation framework
      - NoSQL injection scanner
      - Blind injection optimizer
      - Data exfiltration tool
      - Database security assessment tool
    Practice: Exploit 25 database vulnerabilities

Month 5: Network Attacks & Post-Exploitation (Weeks 19-22)
  Network Attacks & Man-in-the-Middle
    Topics:
      - ARP poisoning and spoofing
      - DNS spoofing and hijacking
```

- DHCP starvation attacks
- VLAN hopping techniques
- STP manipulation
- BGP hijacking basics
- SSL/TLS vulnerabilities
- SSL stripping attacks
- Evil twin attacks
- Rogue access points
- SMB relay attacks
- LLMNR/NBT-NS poisoning
- Responder tool usage
- IPv6 attack vectors
- Network sniffing at scale

Projects built:
- MITM attack framework
- Network poisoning toolkit
- SSL stripping tool
- Rogue DHCP server
- Traffic manipulation proxy
- Network attack automation

Practice: Perform 30 network attacks in lab environment

Wireless Network Hacking

Topics:
- Wireless networking fundamentals
- 802.11 protocol weaknesses
- Monitor mode and packet injection
- WEP cracking techniques
- WPA/WPA2 handshake capture
- WPA2 cracking with hashcat
- WPS attacks with Reaver/Bully
- PMKID attack methodology
- Evil twin and karma attacks
- Captive portal bypassing
- Bluetooth hacking basics
- Software Defined Radio (SDR) basics
- RFID and NFC hacking
- Wireless IDS/IPS evasion
- Creating custom rogue APs

Projects built:
- Automated WiFi cracking tool
- Handshake capture framework
- Rogue AP with captive portal
- Bluetooth exploitation tool
- Wireless security assessment kit

Practice: Crack 20 wireless networks, create 10 rogue APs

Post-Exploitation & Lateral Movement

Topics:
- Post-exploitation methodology
- Maintaining access techniques
- Backdoor installation methods
- Rootkit basics and detection
- Living off the land (LOLBins)
- PowerShell for post-exploitation
- Windows privilege escalation
- Linux privilege escalation
- Lateral movement strategies
- Pass-the-hash/ticket attacks
- Kerberos attacks in depth
- Active Directory exploitation
- Domain dominance techniques
- Data exfiltration methods
- Covering tracks and log cleaning

Projects built:
- Post-exploitation framework
- Persistence toolkit
- Lateral movement automation
- AD exploitation toolkit
- Data exfiltration tool
- Anti-forensics toolkit

Practice: Complete 20 post-exploitation scenarios

Malware Analysis & Reverse Engineering Basics

Topics:
- Malware types and categories
- Static malware analysis
- Dynamic malware analysis
- Sandboxing and VM detection
- Basic assembly language
- x86/x64 architecture basics
- Debugging with OllyDbg/x64dbg

```
      - IDA Pro basics
      - Ghidra for reverse engineering
      - Packing and unpacking
      - Anti-analysis techniques
      - Behavioral analysis
      - Network traffic analysis
      - Memory forensics basics
      - Indicators of Compromise (IoCs)
    Projects built:
      - Malware analysis lab setup
      - Simple malware analyzer
      - Unpacker tool development
      - Behavioral analysis framework
      - IoC extraction tool
    Practice: Analyze 15 malware samples safely

Month 6: Advanced Domains: Active Directory, Mobile and Cloud (Weeks 23-26)
  Active Directory Attacks
    Topics:
      - Active Directory fundamentals
      - Domain enumeration with PowerView
      - BloodHound for AD mapping
      - Kerberoasting deep dive
      - ASREPRoasting techniques
      - Golden ticket attacks
      - Silver ticket attacks
      - DCSync attacks
      - Trust relationship abuse
      - Group Policy exploitation
      - LAPS bypass techniques
      - Constrained delegation abuse
      - Forest compromise strategies
      - Azure AD attacks
      - Hybrid environment exploitation
    Projects built:
      - AD enumeration toolkit
      - Kerberos attack framework
      - BloodHound custom queries
      - Domain persistence toolkit
      - Azure AD exploitation tool
    Practice: Compromise 10 Active Directory environments
  Mobile Application Security
    Topics:
      - Android architecture and security
      - iOS security model
      - Mobile app reverse engineering
      - APK decompilation and analysis
      - Smali/baksmali usage
      - Frida for dynamic analysis
      - SSL pinning bypass
      - Root/jailbreak detection bypass
      - Mobile app traffic interception
      - Insecure data storage
      - Weak cryptography in mobile apps
      - Intent vulnerabilities
      - Deep link exploitation
      - Mobile malware basics
      - MDM bypass techniques
    Projects built:
      - Mobile app security scanner
      - APK analysis automation
      - Frida script collection
      - SSL pinning bypass tool
      - Mobile pentest framework
    Practice: Test 20 mobile applications
  Cloud Security & Container Hacking
    Topics:
      - Cloud security fundamentals
      - AWS security and exploitation
      - S3 bucket misconfigurations
      - IAM privilege escalation
      - Lambda function attacks
      - Azure security issues
      - GCP vulnerabilities
      - Kubernetes security
      - Docker escape techniques
      - Container image vulnerabilities
      - Secrets management issues
      - Terraform security
```

```
        - CI/CD pipeline attacks
        - Serverless security
        - Cloud metadata exploitation
      Projects built:
        - Cloud security assessment tool
        - S3 bucket scanner
        - Container escape toolkit
        - Kubernetes exploitation framework
        - Cloud misconfiguration scanner
      Practice: Assess 15 cloud environments
    Physical Security & Social Engineering
      Topics:
        - Physical security assessments
        - Lock picking basics
        - RFID/NFC cloning
        - Badge cloning techniques
        - Tailgating and piggybacking
        - Social engineering principles
        - Pretexting techniques
        - Phishing campaign creation
        - Vishing and smishing
        - Spear phishing strategies
        - Watering hole attacks
        - USB drop attacks
        - Evil maid attacks
        - Dumpster diving value
        - OSINT for social engineering
      Projects built:
        - Phishing framework setup
        - Social engineering toolkit usage
        - Physical security checklist
        - USB payload generator
        - SE campaign automation
      Practice: Design 10 social engineering scenarios
      Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

---------------------------------------------------------------
PHASE 3: ADVANCED OFFENSIVE AND DEFENSIVE SECURITY (MONTHS 7-9, WEEKS 27-39)
---------------------------------------------------------------
Master advanced exploitation, zero-day research, exploit development, and specialized
security domains.

  Month 7: Advanced Exploitation & Zero Days (Weeks 27-31)
    Buffer Overflow Exploitation
      Topics:
        - Memory layout and stack structure
        - Buffer overflow theory
        - Stack-based buffer overflows
        - EIP/RIP control
        - Shellcode development
        - NOP sleds and reliability
        - Bad character analysis
        - Return-to-libc attacks
        - Heap overflow basics
        - Format string vulnerabilities
        - Integer overflows
        - Off-by-one errors
        - Fuzzing fundamentals
        - Exploit development with Python
        - Debugging exploits with GDB
      Projects built:
        - Custom shellcode development
        - Buffer overflow exploit framework
        - Fuzzer development
        - Exploit reliability enhancer
        - ROP chain generator
        - Heap exploitation tool
        - Format string exploit builder
      Practice: Develop 20 working exploits from scratch
    Modern Exploit Mitigation Bypasses
      Topics:
        - ASLR bypass techniques
        - DEP/NX bypass methods
        - Stack canaries bypass
        - Return-oriented programming (ROP)
        - ROP chain construction
        - Gadget finding tools
        - SEHOP bypass
```

- Control Flow Guard (CFG) bypass
- Kernel exploitation basics
- Use-after-free vulnerabilities
- Race condition exploitation
- Sandbox escapes
- Browser exploitation basics
- 0-day research methodology
- Responsible disclosure

Projects built:
- ASLR bypass exploit
- ROP chain automation tool
- Kernel exploit development
- Browser exploit PoC
- Sandbox escape framework

Practice: Bypass 15 different security mitigations

Web Application Firewall Bypasses

Topics:
- WAF detection techniques
- Common WAF products and weaknesses
- Encoding and obfuscation techniques
- Unicode and UTF-8 bypasses
- HTTP parameter pollution
- HTTP verb tampering
- Case variation attacks
- Comment injection techniques
- White space variations
- SQL injection WAF bypasses
- XSS filter evasion
- XXE WAF bypasses
- Rate limiting bypasses
- IP rotation techniques
- CDN and reverse proxy bypasses

Projects built:
- WAF fingerprinting tool
- Automated bypass generator
- Payload obfuscation framework
- WAF testing methodology
- Custom evasion toolkit

Practice: Bypass 20 different WAF configurations

Cryptography & Cryptanalysis

Topics:
- Cryptography fundamentals review
- Symmetric encryption algorithms
- Asymmetric encryption systems
- Hashing algorithms and collisions
- Digital signatures and certificates
- PKI infrastructure attacks
- SSL/TLS vulnerabilities deep dive
- Padding oracle attacks
- CBC bit flipping
- ECB mode vulnerabilities
- Weak random number generators
- Side-channel attacks
- Timing attacks
- Quantum computing threats
- Blockchain security basics

Projects built:
- Padding oracle exploit tool
- Certificate pinning bypass
- Weak crypto detector
- Hash collision finder
- Timing attack framework
- Blockchain vulnerability scanner

Practice: Break 25 weak cryptographic implementations

Red Team Operations

Topics:
- Red team vs penetration testing
- Adversary simulation
- MITRE ATT&CK framework
- Threat actor emulation
- Command and control (C2) setup
- Cobalt Strike usage
- Empire framework
- Covenant C2 platform
- Infrastructure setup and management
- Domain fronting
- Operational security (OPSEC)
- Anti-forensics techniques
- Long-term persistence

```
      - Data staging and exfiltration
      - Purple team exercises
    Projects built:
      - Custom C2 framework
      - Red team infrastructure automation
      - OPSEC checklist implementation
      - Adversary emulation playbook
      - Purple team exercise design
    Practice: Complete 5 red team scenarios

Month 8: DevSecOps & Automation (Weeks 32-36)
  DevSecOps & CI/CD Security
    Topics:
      - DevSecOps principles and practices
      - Shift-left security approach
      - SAST tools and implementation
      - DAST in CI/CD pipelines
      - IAST and RASP technologies
      - Container security scanning
      - Infrastructure as Code security
      - Secrets management in CI/CD
      - Git security and secret scanning
      - Jenkins security hardening
      - GitLab CI/CD security
      - GitHub Actions security
      - Supply chain security
      - SBOM generation and analysis
      - Compliance as code
    Projects built:
      - Secure CI/CD pipeline
      - Automated security testing framework
      - Container security scanner integration
      - IaC security validator
      - Secret scanning automation
    Practice: Secure 10 CI/CD pipelines
  Security Automation & Orchestration
    Topics:
      - Security orchestration platforms
      - SOAR implementation
      - Playbook development
      - Automated incident response
      - Threat intelligence integration
      - API security automation
      - Vulnerability management automation
      - Compliance automation
      - Security metrics and KPIs
      - Dashboard creation
      - Alerting and notification systems
      - Integration with ticketing systems
      - Automated reporting
      - Machine learning for security
      - AI-powered threat detection
    Projects built:
      - SOAR platform setup
      - Automated incident response system
      - Security metrics dashboard
      - Threat intelligence aggregator
      - ML-based anomaly detector
    Practice: Automate 20 security workflows
  Threat Hunting & Intelligence
    Topics:
      - Threat hunting methodology
      - Hypothesis-driven hunting
      - Threat intelligence lifecycle
      - OSINT for threat intelligence
      - Dark web monitoring
      - Threat actor tracking
      - IOC and YARA rules
      - SIGMA rules creation
      - Threat hunting with Splunk
      - ELK stack for security
      - Network flow analysis
      - Endpoint detection techniques
      - Memory analysis for threats
      - Threat intelligence platforms
      - Information sharing (STIX/TAXII)
    Projects built:
      - Threat hunting playbook
      - YARA rule repository
```

- Threat intelligence platform
- Automated threat hunter
- IOC extraction framework

Practice: Hunt for threats in 15 environments

Incident Response & Forensics

Topics:

- Incident response methodology
- NIST incident response framework
- Evidence collection and preservation
- Chain of custody
- Disk forensics and imaging
- File system analysis
- Memory forensics with Volatility
- Network forensics
- Timeline analysis
- Windows forensics artifacts
- Linux forensics artifacts
- macOS forensics basics
- Mobile device forensics
- Cloud forensics challenges
- Forensics report writing

Projects built:

- Incident response toolkit
- Automated evidence collector
- Memory analysis framework
- Timeline generator
- Forensics report template

Practice: Investigate 15 security incidents

Compliance & Governance

Topics:

- Security frameworks overview
- NIST Cybersecurity Framework
- ISO 27001/27002
- PCI DSS requirements
- GDPR and privacy
- HIPAA compliance
- SOC 2 requirements
- Risk assessment methodologies
- Business impact analysis
- Security policies and procedures
- Security awareness training
- Vendor risk management
- Security metrics and reporting
- Audit preparation

Projects built:

- Compliance assessment tool
- Risk assessment framework
- Policy template library
- Compliance dashboard
- Audit checklist generator

Practice: Perform 5 compliance assessments

Month 9: Emerging Security Domains (Weeks 37-39)

Zero Trust Architecture

Topics:

- Zero trust principles
- Identity-centric security
- Microsegmentation strategies
- Software-defined perimeter
- Identity and access management
- Privileged access management
- Multi-factor authentication
- Conditional access policies
- Device trust and compliance
- Network access control
- Zero trust network access
- Application segmentation
- Data classification and protection
- Zero trust monitoring
- Migration strategies

Projects built:

- Zero trust assessment tool
- Microsegmentation design
- Identity verification system
- Device trust validator
- Zero trust roadmap

Practice: Design 5 zero trust architectures

Blockchain & Smart Contract Security

Topics:

- Blockchain technology overview
- Cryptocurrency security
- Smart contract basics
- Solidity security issues
- Reentrancy attacks
- Integer overflow/underflow
- Access control issues
- Front-running attacks
- Flash loan attacks
- Oracle manipulation
- Smart contract auditing
- Web3 security
- Wallet security
- DeFi protocol risks
- NFT security considerations

Projects built:
- Smart contract scanner
- Blockchain security toolkit
- DeFi vulnerability finder
- Smart contract fuzzer
- Web3 security framework

Practice: Audit 10 smart contracts

AI/ML Security

Topics:
- ML model security threats
- Adversarial attacks on ML
- Data poisoning attacks
- Model extraction attacks
- Privacy attacks on ML models
- Backdoor attacks in ML
- Evasion attacks
- Model inversion attacks
- Membership inference
- Differential privacy
- Federated learning security
- Secure multi-party computation
- AI ethics and bias
- Explainable AI for security
- ML for cybersecurity

Projects built:
- Adversarial example generator
- Model security scanner
- Privacy-preserving ML tool
- ML attack framework
- AI security assessment

Practice: Attack and defend 10 ML models

Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)

The capstone month: spec, build, ship and defend, then the final exam that decides the certificate.

Month 10: Capstone and the Final Exam (Weeks 40-44)

Capstone Spec and Setup

Topics:
- Choosing the capstone: a full authorized penetration test of a provided lab environment, from reconnaissance to a professional report
- A one-page spec: scope, milestones, stack
- Repository and skeleton ready on day one
- Instructor sign-off before the build begins

Projects built:
- Approved capstone spec plus the running skeleton

Practice: Pitch the spec to the batch and absorb one hard question

Capstone Build

Topics:
- Two focused weeks in vertical slices
- Instructor checkpoints and honest scope cuts
- Using the whole course: the whole methodology, offensive and defensive, under authorized and documented conditions
- Daily commits and an always-working build

Projects built:
- The capstone, feature-complete with version control

Practice: A working build at the end of every session

Ship and Polish

Topics:
- The finishing pass: edge cases, errors, documentation

- A professional pentest report: findings, evidence, severity ratings and remediation advice
- A README worth reading and a demo worth watching
- A full-course spiral review before the exam

Projects built:

- The capstone shipped, documented and rehearsed

Practice: Two mock demos, each tighter than the last

Demo Day and the Final Exam

Topics:

- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: penetration tester, security analyst, red or blue team roles, and certifications like CEH and OSCP
- Where our other courses continue the journey

Projects built:

- Demo day presentation delivered to the batch

Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: recon, scanning and vulnerability-assessment labs on authorized ranges
- Phase 2: exploitation, web-attack and post-exploitation labs, all in controlled environments
- Phase 3: exploit-development, red-team and blue-team defensive labs
- Phase 4: the full authorized lab pentest and its professional report

Total projects: 20+ lab-based engagements, crowned by a full authorized penetration test and report

SKILLS THE STUDENT WILL HAVE AT THE END

- The legal and ethical foundation: authorization, scope, disclosure and law
- Reconnaissance, scanning and vulnerability assessment
- Exploitation: system, web (OWASP Top 10), network and post-exploitation
- Advanced offensive: exploit development, mitigation bypasses, red team operations
- Defensive security: DevSecOps, threat hunting, incident response and forensics
- Professional penetration-test reporting

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Penetration tester, security analyst and red or blue team roles
- Certifications: this course prepares the ground for CEH and OSCP
- Specializations: web, cloud, mobile or ICS security
- Defensive tracks: SOC, threat hunting and incident response

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 3-6 LPA (Junior Security Analyst, SOC Analyst L1)
Growing roles: Rs 6-12 LPA (Ethical Hacker, SOC Analyst L2, VAPT Analyst); Rs 10-20 LPA (Penetration Tester, Security Engineer)
Experienced roles: Rs 18-30 LPA (Senior Ethical Hacker, Red Team Engineer); Rs 25-45 LPA (Lead Security Engineer, Security Consultant)
Other roles: Rs 8-20 LPA (Cyber Crime, CERT-In, Defense contractors); Rs 15-40 LPA (Big4, Security MNCs, Product-based firms); Rs 2-15 Lakhs per valid vulnerability (platform & severity dependent); Rs 40-90 LPA at senior level
Freelance consulting: Rs 2,500-12,000/hour based on skills and certifications

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What does this ethical hacking course cover?
  A: The full authorized-pentesting path in order: networking, Linux and programming for security, reconnaissance and OSINT, scanning and vulnerability assessment; then core exploitation, system, web (OWASP Top 10) and network attacks, Metasploit, post-exploitation and advanced domains like Active Directory, mobile and cloud; then advanced offensive work (exploit development, mitigation bypasses, red team) and the defensive side (DevSecOps, threat hunting, incident response, forensics). A finale month runs a full authorized lab pentest, writes the professional report and sits the final exam. Everything is practised only in labs and authorized ranges.

  Q: How long does it take to learn ethical hacking properly?
  A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two more months in hand for learners who need them. The three teaching phases build the skill and its real-world application; the finale month ships a capstone and sits the final exam.

  Q: How am I assessed? Is the certificate just for attending?
  A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

  Q: Is ethical hacking legal, and is this course legal to take?
  A: Ethical hacking is legal precisely because it is authorized: you test only systems you own or have written permission to test, within an agreed scope. That principle is taught from the first week and enforced throughout, and all practical work happens in dedicated lab environments and authorized ranges we provide.

  Q: Do I need prior experience to start?
  A: No. The course begins with networking, Linux and programming foundations and builds from there. What matters most is genuine curiosity, patience for problem-solving, and a commitment to using these skills legally and ethically. It is an adult course aimed at serious career-changers and IT professionals.

  Q: Will this get me a certification like CEH or OSCP?
  A: This course is not an exam voucher for CEH or OSCP, and we are honest about that: it teaches the skills those certifications test and prepares you thoroughly to pursue them next. You earn a Modern Age Coders certificate here by passing the final exam, which includes a real authorized lab pentest and its report, exactly the kind of work the industry certifications and employers care about.

  Q: Can I try a class before enrolling?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ethical-hacking-masterclass-complete
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
COURSE 59 OF 137: FLUTTER COURSE: DART TO REAL CROSS-PLATFORM APPS
==

Course link: https://learn.modernagecoders.com/courses/complete-flutter-app-development-masterclass-college
Subject area: App Development
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Flutter course for college students and professionals: Dart, Flutter UI, state management, Firebase and production habits, one codebase for Android and iOS, in 10-12 months.

In one line: One codebase, both app stores: Dart and Flutter learned properly, from first widget to production-shaped apps with Firebase.

ABOUT THE PROGRAMME

Flutter delivers the honest cross-platform promise: one Dart codebase producing genuinely native-feeling apps on Android and iOS, and it has the production adoption to prove it. This live online masterclass builds it in order: Dart as a real language first, then Flutter the widget way, layouts, navigation and UI that feels native on both platforms, then the layer that separates apps from demos: state management chosen with judgment, architecture that survives features, Firebase for auth and data, offline behavior, testing and release discipline. A finale month ships a capstone app and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- Dart first as a real language: Flutter without Dart depth is widget soup
- The widget mental model taught until it clicks: composition all the way down
- State management with judgment: the options mapped honestly, one mastered
- Production habits: offline behavior, testing and release discipline included
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Dart and Flutter foundations (Months 1-3): the language, then widgets, layouts and navigation
Phase 2: State and architecture (Months 4-6): state management with judgment and structure that survives
Phase 3: Production features (Months 7-9): Firebase, offline behavior, testing and release discipline
Phase 4: The finale (Month 10): the capstone app on both platforms, and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Everything is taught from the first line
Equipment: Any reasonably recent computer; Flutter, the emulators and every tool used are free. iOS device builds require a Mac at release time, stated honestly, Android covers the full learning journey
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

App builders: Builders who want both app stores from one codebase
College students: Students who want a demonstrable, shippable specialization
Startup minded: Product people who need mobile speed without two teams
Career switchers: Professionals entering mobile with the fastest honest route to both platforms
Web developers: Developers adding mobile without abandoning their declarative instincts

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & CORE FLUTTER + DART SKILLS (MONTHS 1-3, WEEKS 1-13)

Build rock-solid programming and Flutter fundamentals. Learn Dart, understand Flutter's widget tree, layout system, and create your first interactive cross-platform apps.

Months 1-2: Dart Fundamentals & Flutter Basics (Weeks 1-8)
Introduction to Flutter, Dart & Development Environment Setup
Topics:
- What is Flutter? History, features, and why it's popular
- Single codebase for Android, iOS, Web, Desktop
- Flutter architecture overview: Engine, Framework, Widgets
- Dart language introduction and why Flutter uses Dart
- Installing Flutter SDK on Windows/Mac/Linux
- Setting up Android Studio, VS Code, and Xcode (for macOS)
- Configuring device emulators and real devices

- Understanding Flutter project structure: lib, pubspec.yaml, android, ios, web
- Your first Flutter app: counter & Hello World
- main() function, runApp(), MaterialApp, Scaffold, AppBar, Body
- Hot reload vs hot restart vs full restart
- Understanding widget tree and composition
- StatelessWidget vs StatefulWidget basics
- DartPad and Flutter web previews
- Code style, formatting (dartfmt), and basic conventions

Projects built:

- Hello Flutter app with personalized welcome screen
- Digital visiting card app (name, role, contact info)
- Simple profile screen with image, text and icons
- Two-screen mini app using basic navigation
- Kids track variant: Colorful "My Favorite Things" app

Practice: Daily: 30-45 min Dart & Flutter basics; reproduce 3-5 simple screens from design snapshots

Dart Basics, Variables, Data Types & Operators

Topics:

- Dart program structure: main(), imports, packages
- Variables: var, final, const, late
- Primitive-like types in Dart: int, double, String, bool
- num type and type inference
- Dynamic vs static typing, dynamic keyword use-cases
- Type conversion and parsing: int.parse, double.parse, toString
- Collections overview: List, Set, Map (intro only)
- Operators: arithmetic, relational, logical, assignment
- Null safety in Dart: non-nullable & nullable types (int vs int?)
- Null-aware operators: ?, ??, ??=, !
- String interpolation and multi-line strings
- String methods: length, substring, contains, split
- Basic console apps in Dart (for logic practice)
- Using dartpad.dev to practice Dart snippets

Projects built:

- Simple console calculator (add, subtract, multiply, divide)
- Tip & bill split calculator (per person amount)
- Age & birthday countdown calculator
- Unit converters: temperature, length, weight
- Kids variant: Score tracker for a simple game

Practice: Solve 30+ problems on variables, strings, calculations and null-safety handling

Control Flow, Functions & Basic Collections

Topics:

- Control flow: if, else if, else
- Switch-case in Dart, enums introduction
- Loops: for, while, do-while, for-in
- break and continue, labeled statements
- Functions: definition, parameters (positional, named, optional), return types
- Arrow functions (fat arrow syntax)=>
- Scope rules and variable lifetime
- Intro to collections: List, Set, Map basics
- Iterating collections with for, for-in, forEach
- Basic algorithm building: searching & counting
- Introduction to error handling: try, catch, finally

Projects built:

- Number guessing game (console or Flutter dialog-based)
- Grade calculator with letter-grade output
- To-do list data model (List operations) with console UI
- Simple quiz logic engine (questions and scoring)
- Kids variant: Multiplication practice generator

Practice: Solve 40+ problems covering loops, branching, and simple collection manipulations

Core Flutter Widgets & Layout Basics

Topics:

- Flutter widget philosophy: everything is a widget
- Understanding build() method and widget lifecycle (overview)
- Basic display widgets: Text, Icon, Image, Container
- Layout widgets: Row, Column, Expanded, SizedBox
- Alignment & positioning: MainAxisAlignment, CrossAxisAlignment, Center, Align
- Styling text with TextStyle, GoogleFonts (package intro)
- Colors, ThemeData, Material 3 basics
- Buttons: ElevatedButton, TextButton, OutlinedButton, IconButton
- Input widgets: TextField basics
- Using Scaffold: appBar, body, floatingActionButton, bottomNavigationBar (intro)
- SingleChildScrollView and ListView (intro)
- Working with pubspec.yaml: adding assets (images, fonts)
- Asset management & responsive basics (MediaQuery)
- Debugging layout issues with Flutter DevTools & debugPrint

Projects built:

- Personal profile card app with images & icons

- Single-page product detail UI (static)
- Simple landing screen for a fictional app (hero image, CTA button)
- Basic contact form UI (no backend yet)
- Kids variant: "My Pet" info card with big images & fun fonts

Practice: Recreate 6-10 simple UI screens from given Figma/Dribbble screenshots

Month 3: Object-Oriented Programming with Dart & Stateful Widgets (Weeks 9-13)

Dart Classes, Objects & OOP Basics

Topics:

- OOP paradigm in Dart
- Defining classes & creating objects
- Fields, constructors (default, named, optional)
- Named constructors and factory constructors (intro)
- this keyword, initializer lists
- Encapsulation: private members with _ prefix
- Getters & setters (explicit, shorthand syntax)
- Static variables and methods
- Basic toString() overrides
- Intro to enums for modeling states
- Model classes for Flutter apps (e.g., User, Product, Task)

Projects built:

- User profile model and display widget
- Product class used in a simple catalog list
- Task/Todo model with computed properties (e.g. isOverdue)
- Simple contact model and list
- Kids variant: Character class for a mini game inventory

Practice: Create 15+ different model classes to represent real-world entities used in apps

Inheritance, Polymorphism & Flutter Stateful Widgets

Topics:

- Inheritance in Dart: extends keyword
- Abstract classes and methods (intro)
- Method overriding & polymorphism
- Mixins: with keyword, when and how to use
- Interfaces via abstract classes & implicit interfaces
- Intro to composition vs inheritance
- StatefulWidget and State classes in Flutter
- setState(), rebuilding the UI, performance considerations
- Managing widget state vs app state (conceptual intro)
- TextField controllers and input handling
- Forms: Form, GlobalKey, basic validation
- GestureDetector and InkWell for taps & gestures
- Basic animations with AnimatedContainer and AnimatedOpacity (preview)

Projects built:

- Interactive counter app with custom logic
- Simple login form with validation and error messages
- Like/favorite button with animation
- Basic habit tracker with toggling state
- Kids variant: Tap-to-change-color playground app

Practice: Implement 20+ stateful interactions (toggles, counters, forms, sliders, etc.)

Abstraction, Interfaces & PHASE 1 Mini Capstone

Topics:

- Deeper look at abstract classes & interfaces in Dart
- Strategy-like patterns using abstract classes
- Separation of concerns: UI vs business logic vs data models
- Organizing Flutter code: folders (models, widgets, screens, services)
- Intro to simple service classes for data handling
- Preparing for larger apps: reusability & modular widgets

Projects built:

- Reusable UI components library (buttons, cards, chips)
- Simple theme manager (light/dark mode toggle using inherited widget pattern - conceptual)
- PHASE 1 MINI CAPSTONE: Multi-Screen Personal Productivity App (Tasks + Notes + Reminders)
- Features: Home screen, list views, detail pages, basic persistence via in-memory lists, clean UI

Practice: Refactor earlier projects to use cleaner OOP and reusable widgets

Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 2: INTERMEDIATE FLUTTER, STATE MANAGEMENT & APP ARCHITECTURE (MONTHS 4-6, WEEKS 14-26)

Master advanced Dart, state management patterns, navigation, forms, local storage, and start building real-world, offline-capable Flutter apps.

Month 4: Advanced Dart, Layouts & State Management Foundations (Weeks 14-18)

Advanced Dart, Collections & Async Programming

Topics:

- Deep dive into Lists, Sets, Maps
- Higher-order functions: map, where, reduce, fold

- Collection if and collection for in Dart
- Spread operator (...) and null-aware spread (...?)
- Immutable vs mutable collections
- Error handling in Dart: try-catch, on, finally, rethrow
- Custom exceptions
- Asynchronous programming: Future, async/await
- Streams: single subscription vs broadcast (intro)
- Stream transformations and listeners
- Using async code in Flutter (FutureBuilder, StreamBuilder basics)
- Using packages: http, intl and others
- Organizing business logic into service classes

Projects built:

- Data transformation utilities (filtering, sorting, grouping)
- HTTP client service (mock APIs) using http package
- Simple news/feed list using FutureBuilder (with mock JSON)
- Real-time counter/clock using StreamBuilder
- Kids variant: Random joke/quote generator using an API

Practice: Solve 35+ problems on collections & async patterns (Futures & Streams)

Flutter Navigation, Routing & App Structure

Topics:

- Navigator 1.0: push, pop, named routes
- Route arguments and passing data between screens
- Navigator 2.0 and Router (high-level intro, for advanced later)
- BottomNavigationBar and tab-based navigation
- Drawer navigation and custom side menus
- Nested navigation and navigation stacks
- App lifecycle basics: WidgetsBindingObserver, handling app states
- Splash screens and onboarding flows
- Handling back button and system navigation
- Routing best practices and route generators

Projects built:

- Multi-screen notes app with details and edit screens
- Onboarding flow with 3-4 intro pages
- Bottom navigation based mini app (Home / Search / Profile)
- Simple tabbed app with TabBar and TabBarView
- Kids variant: Storybook app with navigation between chapters

Practice: Implement routing and navigation in 6-8 mini applications

Forms, Validation & User Input Handling

Topics:

- Advanced TextField usage: controllers, focus nodes
- Form widget, FormState, and GlobalKey
- Synchronous and asynchronous validation
- Custom form field widgets
- Handling keyboard, focus traversal, and input submission
- DropdownButton, Checkbox, Radio, Switch, Slider widgets
- Date & time pickers
- Input formatting and masking
- Accessibility considerations for forms

Projects built:

- User registration form with full validation
- Feedback/contact form with submission success screen
- Settings screen with toggles, sliders, and dropdowns
- Profile edit form with local validation & previews
- Kids variant: Quiz creation form with multiple questions

Practice: Build and validate 10+ complex forms and input flows

Flutter Layout Mastery, Lists & Responsive Design

Topics:

- Advanced layout widgets: Stack, Positioned, Wrap, LayoutBuilder
- ListView.builder, ListView.separated
- GridView builder and dynamic grids
- Slivers overview: CustomScrollView, SliverAppBar, SliverList (intro)
- Responsive design principles in Flutter
- MediaQuery, LayoutBuilder and OrientationBuilder
- Creating adaptive UIs for phone/tablet/web
- Handling different screen densities and sizes
- Using Flutter's Flex and Expanded effectively

Projects built:

- E-commerce product list grid with responsive layout
- Photo gallery app with grid and full-screen viewer
- News feed with cards and SliverAppBar
- Responsive dashboard layout (phone vs tablet)
- Kids variant: Sticker board using Stack & Positioned

Practice: Rebuild 10+ real-world UI screens with responsive layouts

Intro to State Management Patterns

Topics:

- What is state management? Local vs global state
- Limitations of setState() for larger apps
- Lifting state up and prop drilling problems
- InheritedWidget and InheritedModel concept

- Provider package introduction
- ChangeNotifier and ChangeNotifierProvider
- Listening to changes with Consumer and context.watch/select
- Basic architecture: separating UI and logic using providers
- Simple global app state (theme, locale, auth status)

Projects built:

- Counter app rebuilt with Provider
- Theme toggler (light/dark mode) using Provider
- Cart management in a simple shopping UI
- Favorites list synced across multiple screens
- Kids variant: Global score/coins manager for a mini game

Practice: Create 10+ small apps using Provider for shared state

Month 5: Persistent Storage, Offline Apps & Advanced State (Weeks 19-22)

Local Storage: SharedPreferences, SQLite & Object Persistence

Topics:

- Why persistence? Saving user data locally
- Key-value storage with shared_preferences
- Simple settings & flags persistence
- SQLite basics and sqflite package
- Database schema design for small apps
- CRUD operations (create, read, update, delete)
- Using path_provider for file paths
- Storing JSON data locally
- Intro to Hive / Isar (NoSQL local DB) and when to use them
- Data serialization with jsonEncode/jsonDecode
- Basic repository pattern for data layer

Projects built:

- Persistent to-do list app with sqflite
- Settings/preferences manager using shared_preferences
- Offline notes app using Hive or Isar
- Simple expense tracker with SQLite
- Kids variant: Sticker collection app persisted locally

Practice: Build 12+ features requiring reading/writing persistent local data

State Management with Provider & Riverpod, Part 1

Topics:

- Revisiting Provider for larger apps
- Multiple providers and provider scopes
- Introduction to Riverpod: why & how it improves over Provider
- Provider types in Riverpod: Provider, StateProvider, StateNotifierProvider (intro)
- Using Riverpod for business logic & UI separation
- Immutability and state objects
- Error handling in async providers
- Refactoring existing apps to Riverpod

Projects built:

- News/feed app powered by Riverpod for state
- Weather app using async APIs with Riverpod
- Multi-tab app with shared state using Riverpod
- Theme + language switcher using Riverpod
- Kids variant: Simple quiz app with score using Riverpod

Practice: Refactor 5 existing apps from setState/Provider to Riverpod

Advanced State Management: Riverpod / BLoC, Part 2

Topics:

- StateNotifier & StateNotifierProvider with Riverpod
- Modeling complex state (loading, loaded, error) patterns
- BLoC pattern introduction (Cubit/BLoC) and when to use it
- Comparing Riverpod, Provider, and BLoC (pros/cons)
- Events, states, and BLoC logic (if using flutter_bloc)
- Using Freezed for immutable data classes & unions (intro)
- Folder structure for scalable apps (feature-first, layer-first)
- Practical guidelines for choosing a pattern per project size

Projects built:

- Authentication flow using Riverpod or BLoC
- Search with debounced queries and loading states
- Paginated list with infinite scroll
- Complex form with multi-step state management
- Kids variant: Multi-level quiz/game with progress state

Practice: Build 10+ flows using Riverpod/BLoC with loading/error handling

Flutter Animations & UX Polish

Topics:

- Implicit animations: AnimatedContainer, AnimatedOpacity, AnimatedAlign
- AnimatedCrossFade, AnimatedSwitcher
- Explicit animations: AnimationController, Tween, CurvedAnimation
- Using AnimatedBuilder and AnimatedWidget
- Hero animations between screens
- Page transitions and custom route animations
- Lottie animations integration
- Micro-interactions and motion design principles
- Performance considerations with animations

```
    Projects built:
      - Animated onboarding flow with Hero transitions
      - Like/Bookmark micro-interactions with animations
      - Expandable cards list with AnimatedContainer
      - Custom page transition for navigation
      - Kids variant: Animated "reward" screen with confetti
    Practice: Implement 20+ small animations and transitions across apps

Month 6: Design Systems, APIs & Testing Foundations (Weeks 23-26)
  Reusable Widgets, Design Systems & Theming
    Topics:
      - Creating reusable, composable widgets
      - Building a design system: spacing, typography, colors
      - Material 3 (Material You) theming in Flutter
      - ThemeData and ColorScheme customization
      - Dark theme and dynamic theme switching
      - Global styles and app-wide configuration
      - Localization basics: using flutter_localizations and intl
      - Supporting multiple languages and locales
      - Accessibility: font scaling, contrast, semantics
    Projects built:
      - Design system library for buttons, cards, inputs
      - App-wide theming for a sample app (light/dark/custom)
      - Two-language version of an existing app (e.g. English + Hindi)
      - Custom theming for kids' learning app
      - Reusable error/empty/loading state widgets
    Practice: Implement a consistent design system in 3+ existing apps
  Networking & REST API Integration - Fundamentals
    Topics:
      - HTTP basics: methods, status codes, headers, JSON
      - Using http / dio packages for API calls
      - GET, POST, PUT, DELETE requests
      - Parsing JSON manually and with json_serializable (intro)
      - Error handling for network failures & timeouts
      - Showing loading, success, and error states in UI
      - API pagination and infinite scroll
      - Securing API keys and environment variables (conceptual)
      - Handling connectivity changes (connectivity_plus intro)
    Projects built:
      - Public API driven list app (e.g. GitHub repos, movies, news)
      - Search UI with remote filtering
      - Paginated list with "Load more" and pull-to-refresh
      - Simple CRUD client for a mock REST API
      - Kids variant: Fun facts/animals viewer from an open API
    Practice: Build 8+ screens powered by real or mock REST APIs
  Intro to Testing Flutter Apps
    Topics:
      - Why testing matters for Flutter apps
      - Types of tests: unit, widget, integration
      - Dart test package basics
      - Unit tests for Dart logic (services, utils)
      - Widget tests: testing UI components and layouts
      - Mocking services and repositories
      - Introduction to golden tests (visual regression), concept
      - Basic test-driven development (TDD) workflow
      - Measuring coverage in Flutter projects
    Projects built:
      - Unit tests for validators and repository classes
      - Widget tests for key widgets in a sample app
      - Test suite for state management logic (Riverpod/BLoC)
      - Simple golden tests for critical UI components
      - Kids variant: Test coverage for a small game's logic
    Practice: Write tests for at least 3 existing apps, targeting 60%+ coverage
  Build Automation, Packages & Version Control
    Topics:
      - pubspec.yaml deep dive: dependencies, dev_dependencies
      - Managing versions and package upgrades
      - Creating and publishing your own Dart/Flutter package (concept)
      - Build flavors (dev, staging, prod), overview
      - Flutter build commands for Android, iOS, Web
      - Introduction to Git with Flutter projects
      - Git branching strategies (feature, develop, main)
      - GitHub integration, PRs, code reviews
      - CI basics using GitHub Actions for Flutter test/build
    Projects built:
      - Extract common widgets into a local package
      - Git repository for all course projects (mono-repo or multi-repo)
      - Simple GitHub Actions workflow for test+build
      - Versioning and changelog for a sample app
```

- Kids variant: Visual Git intro via Git GUI tools
Practice: Convert all active projects to Git repos and set up basic CI for at least one
Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing
this phase with everything before it

PHASE 3: BACKEND INTEGRATION, FIREBASE & PRODUCTION FEATURES (MONTHS 7-9, WEEKS 27-39)

Master building connected apps with Firebase and REST backends, implement authentication,
payments, notifications, and solid application architecture.

Month 7: Firebase, Auth & Real-World Features (Weeks 27-32)
Firebase Fundamentals & Realtime Data
Topics:
- What is Firebase? Overview of its products
- Setting up a Firebase project
- Integrating Firebase with Flutter (firebase_core)
- Crashlytics and Analytics setup (overview)
- Realtime Database vs Firestore overview
- Cloud Firestore basics: collections, documents, queries
- Adding, reading, updating, deleting documents
- Realtime listeners and streams from Firestore
- Security rules overview and best practices
- Offline support and caching behavior
Projects built:
- Realtime chat room (basic text chat) using Firestore
- Live comments section on a sample post
- Collaborative to-do list shared between users
- Simple presence/online status indicator
- Kids variant: Shared drawing board (concept + simple implementation)
Practice: Build 6-8 small realtime features with Firestore streams
Authentication & User Management
Topics:
- Firebase Authentication overview
- Email/password sign up and login
- Password reset flow
- Social login (Google, Apple, Facebook, platform dependent)
- Persisting login state and auto-login
- Linking auth user to Firestore profile documents
- Protecting routes based on auth state
- Handling auth errors and edge cases
- Profile management and avatar uploads (intro to Storage)
Projects built:
- Complete auth flow: sign-up, login, logout, reset password
- User profile screen linked to Firestore user data
- Protected routes (e.g., only logged-in users see dashboard)
- Simple role-based protection (admin vs user)
- Kids variant: Simplified nickname-based login for child-friendly apps
Practice: Implement full auth flows in at least 2 different sample apps
Cloud Storage, Media & File Handling
Topics:
- Firebase Storage integration
- Uploading images/files from Flutter apps
- Picking images from gallery/camera (image_picker, file_picker)
- Displaying uploaded media in apps
- Managing download URLs and access controls
- Basic image caching for performance
- Handling upload progress and cancellation
- File size limits and optimization concepts
Projects built:
- User avatar upload & change profile picture feature
- Simple image gallery app powered by Firebase Storage
- Kids variant: Art gallery app where kids upload drawings
- Media feed with upload progress indicators
Practice: Add media upload/view features to at least 3 apps
Push Notifications & In-App Messaging
Topics:
- Firebase Cloud Messaging (FCM) integration
- Obtaining device tokens and sending test notifications
- Handling foreground, background, terminated notifications
- Notification customization (sound, icon, click actions)
- Topic-based and user-segment notifications
- In-app messaging basics (e.g., using Firebase In-App Messaging or custom implementation)
- Local notifications (flutter_local_notifications package)
- Scheduling notifications and reminders
Projects built:
- Push notification integration in existing habit tracker app
- Reminders/alerts app using local notifications
- Topic-based notifications (e.g., daily tips/news categories)

- In-app announcement banner system
- Kids variant: Reward reminder notifications for practice streaks

Practice: Implement both remote and local notifications across 2-3 apps

Advanced Firebase: Queries, Security & Performance

Topics:

- Advanced Firestore queries (compound, range, ordering)
- Indexes in Firestore for performance
- Optimizing read/write costs
- Security rules patterns and testing
- Multi-tenant data design (user-specific data separation)
- Cloud Functions for backend logic (overview, callable functions)
- Basic Cloud Functions triggered by Firestore events (concept)
- Monitoring Firebase usage and costs

Projects built:

- Search & filter features using advanced queries
- Role-based security rules for a sample app
- Server-triggered notifications (concept + small demo with Cloud Functions)
- Dashboard app with Firestore aggregate views
- Kids variant: Filtered content access based on age group

Practice: Design secure Firestore structures and rules for 2-3 different apps

Month 8: REST APIs, Payments & Robust Product Apps (Weeks 33-36)

Advanced REST APIs, Error Handling & Data Layers

Topics:

- Refining data layer with repositories & DTOs
- Handling complex JSON responses (nested, lists, maps)
- Code generation with json_serializable & build_runner
- Error modeling and mapping HTTP errors to UI states
- Retry policies and exponential backoff (concept + simple implementation)
- Network connectivity handling & offline-first strategies
- Logging and monitoring HTTP traffic (e.g., dio interceptors)

Projects built:

- Robust API-driven app with error handling and retries
- Offline-first news or blog reader (cache + network)
- Centralized API client service with interceptors
- Kids variant: Live quiz app using remote API data
- Refactor earlier REST apps to use repository pattern

Practice: Create 5+ robust REST clients with proper error and offline handling

In-App Purchases & Payment Gateway Integration

Topics:

- In-app purchases on Android & iOS (overview, in_app_purchase package)
- Using 3rd-party gateways like Razorpay, Stripe (concept + Flutter packages)
- Handling payment flows and callbacks
- Server verification of purchases (conceptual, minimal implementation)
- Unlocking premium content and features
- Compliance and security best practices (conceptual)
- Sandbox/test environment usage

Projects built:

- Premium upgrade flow for an existing app (e.g., remove ads/unlock features)
- Donation/payment screen integration with test gateway
- Subscription-like experience (locally simulated) with feature flags
- Kids variant: Parent-only purchase screen for unlocking content
- Designing a pricing & plans screen with clear UX

Practice: Integrate at least one payment-like flow in a demo app (test mode)

App Architecture in Depth: Clean Architecture & Large App Design

Topics:

- Revisiting Clean Architecture & MVVM in the context of bigger apps
- Domain layer: entities, use cases, business rules
- Data layer: models, DTOs, repositories, remote & local sources
- Presentation layer: views, controllers, state management (Riverpod/BLoC)
- Dependency injection basics (get_it, Riverpod providers)
- Feature modularization for scalable apps
- Working with multiple backends (REST + Firebase together)
- Documentation of architecture and data flow

Projects built:

- Architect one mid-size app using full Clean Architecture
- Combine local cache + remote data with repository pattern
- Implement dependency injection for services and repositories
- Architecture diagrams (UML, flow diagrams) for apps
- Kids variant: Simplified layered structure for a learning app

Practice: Re-architect one existing project using a full layered approach

Testing Connected Apps (REST + Firebase)

Topics:

- Unit testing repositories with mocked network/Firebase
- Widget tests for screens relying on async data
- Using fake clients, mocktail or Mockito for mocking
- Integration tests for sign-up/login flows
- Golden tests for critical UI screens
- Continuous testing with CI integration

- Measuring and improving test coverage for large apps
      Projects built:
        - Test suite for an authentication flow
        - Widget tests for list/detail pattern UI
        - Mocking Firestore/REST for offline tests
        - Regression test set for a capstone app
        - Kids variant: Test logic for a score/progress system
      Practice: Achieve 70%+ coverage on at least one sizable app; add essential tests to
      others

  Month 9: Production Hardening (Weeks 37-39)
    Performance Optimization & App Size Management
      Topics:
        - Performance profiling with Flutter DevTools
        - Identifying and fixing jank (frame drops)
        - Optimizing build methods and widget trees
        - Using const constructors and keys effectively
        - Image optimization and caching strategies
        - Lazy loading lists, pagination, and placeholders
        - Reducing app size: tree-shaking, splitting per ABI
        - Minimizing unnecessary rebuilds and re-renders
        - Monitoring performance in release builds
      Projects built:
        - Performance audit of capstone app
        - Implement skeleton loaders and shimmer effects
        - Optimize image-heavy screens
        - Reduce app bundle size for Android and iOS
        - Kids variant: Smooth animation showcase tuned for low-end devices
      Practice: Profile & optimize at least 3 screens per app; document improvements
    Error Reporting, Logging & Analytics
      Topics:
        - Structured logging in Flutter (logger packages, print vs log)
        - Crashlytics deep dive: capturing and analyzing crashes
        - Custom error handling and error boundaries (via widget tree)
        - In-app error reporting (send logs, feedback)
        - Using Firebase Analytics or other analytics tools
        - Key metrics: MAU, retention, funnels (conceptual)
        - Privacy and consent for analytics tracking
      Projects built:
        - Centralized error handling layer in an app
        - User-friendly error screens with retry & support links
        - Basic analytics events for core user flows
        - Feedback form that sends logs & device info (with consent)
        - Kids variant: Minimal analytics respecting child privacy
      Practice: Integrate logging & crash reporting into all capstone-level apps
    Security & Data Protection in Flutter Apps
      Topics:
        - Storing secrets securely (env files, platform secure storage)
        - Encrypted local storage (flutter_secure_storage, cryptography basics)
        - Protecting API keys and sensitive data
        - Secure communication: HTTPS/TLS verification (concept)
        - Obfuscation and code hardening (dart compile, proguard/R8 for Android)
        - Auth token management and refresh strategies
        - Basic OWASP considerations for mobile apps
      Projects built:
        - Secure token storage for an authenticated app
        - Secure notes/vault feature with local encryption (conceptual + basic implementation)
        - API key protection strategies demo
        - Kids variant: Content access locks and parental gate
        - Security checklist for all released apps
      Practice: Review and harden security for all apps handling user data
      Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)

The capstone month: spec, build, ship and defend, then the final exam that decides the
certificate.

  Month 10: Capstone and the Final Exam (Weeks 40-44)
    Capstone Spec and Setup
      Topics:
        - Choosing the capstone: a complete Flutter app with Firebase auth and data, running on Android
          and iOS
        - A one-page spec: scope, milestones, stack
        - Repository and skeleton ready on day one
        - Instructor sign-off before the build begins
      Projects built:

- Approved capstone spec plus the running skeleton, deployed where applicable
    Practice: Pitch the spec to the batch and absorb one hard question
  Capstone Build
    Topics:
      - Two focused weeks in vertical slices
      - Instructor checkpoints and honest scope cuts
      - Using the whole course: Dart depth, widget craft, chosen state management, Firebase and tests
      - Daily commits and an always-working build
    Projects built:
      - The capstone, feature-complete with version control
    Practice: A working build at the end of every session
  Ship and Polish
    Topics:
      - The finishing pass: edge cases, errors, documentation
      - Both-platform pass: the app verified on Android and iOS with release checklists
      - A README worth reading and a demo worth watching
      - A full-course spiral review before the exam
    Projects built:
      - The capstone shipped, documented and rehearsed
    Practice: Two mock demos, each tighter than the last
  Demo Day and the Final Exam
    Topics:
      - Presenting the capstone: live demo plus the hardest bug story
      - The honest map of next roads: mobile development roles, product building, or native
        specializations
      - Where our other courses continue the journey
    Projects built:
      - Demo day presentation delivered to the batch
    Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed
    from every phase. Passing earns the certificate; falling short earns a focused revision
    plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Dart programs and first Flutter apps with real layouts
- Phase 2: state-managed, well-architected applications
- Phase 3: Firebase-backed apps with offline behavior and tests
- Phase 4: the capstone on both platforms, defended at demo day

Total projects: 20+ real builds, crowned by a both-platform capstone app

SKILLS THE STUDENT WILL HAVE AT THE END

- Dart as a real language: types, async and the idioms Flutter assumes
- Widget craft: composition, layouts and UI that feels native
- State management chosen and mastered with judgment
- Firebase: auth, data and the honest limits
- Production habits: offline behavior, testing, release discipline

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every
  month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the
  plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short
  and there is focused revision and a free retest: the certificate certifies knowledge, not
  attendance

WHERE THIS CAN LEAD

- Mobile development roles: Flutter hiring has real momentum
- Product building: two stores, one codebase, real speed
- Native depth later: Kotlin or Swift courses when a platform demands it
- Our backend courses for the server half of app products

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and

abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 3-5 LPA (Junior Flutter Developer / Mobile Intern with strong fundamentals)
Growing roles: Rs 5-9 LPA (Flutter Developer / Mobile App Developer with portfolio apps); Rs 8-15 LPA (Senior Flutter Developer / Lead Mobile Engineer in startups & agencies)
Experienced roles: Rs 12-25 LPA (Senior/Lead Flutter Developer or Mobile Architect in product companies); Rs 18-40 LPA (Lead Developer / Mobile Architect / Specialist Consultant)
Freelance: Rs 1,000-5,000/hour or $20-$100/hour based on expertise and client region
International usa: $80k-160k USD (Mobile / Flutter Developer to Senior roles, remote or onsite)
International europe: €50k-120k EUR based on country, role, and experience in Flutter/mobile dev

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this Flutter course cover?
A: Dart as a real language first, then Flutter properly: the widget mental model, layouts and navigation that feel native on both platforms, state management chosen with judgment, architecture that survives features, Firebase auth and data, offline behavior, testing and release discipline. A finale month ships a capstone app verified on Android and iOS and sits the final exam.

Q: How long does it take to learn Flutter properly?
A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two more months in hand for learners who need them. The three teaching phases build the skill and its real-world application; the finale month ships a capstone and sits the final exam.

Q: How am I assessed? Is the certificate just for attending?
A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Flutter or React Native?
A: Both are honest cross-platform answers. Flutter wins on UI consistency and performance-by-default with one language (Dart); React Native wins for teams already deep in JavaScript. We teach both roads in different courses; this one is the Flutter road, chosen most by builders who want pixel-level control on both platforms.

Q: Do I need a Mac for this course?
A: Not for learning: Android covers the entire journey on any computer, and the iOS build step is taught honestly as a release-time requirement (Apple mandates a Mac there). Students shipping to the App Store later use a borrowed, used or cloud Mac at that point, stated upfront rather than discovered late.

Q: Can I try a class before enrolling?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/complete-flutter-app-development-masterclass-college
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 60 OF 137: FRONTEND DEVELOPMENT FOR TEENS: CSS CRAFT TO REACT UI
==========

Course link: https://learn.modernagecoders.com/courses/frontend-development-masterclass-for-teens
Subject area: Web Development
Designed for: Teens
Who can join: Absolute beginner, ages 13 to 18
Course length: 9-11 months (36-44 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Frontend course for teens aged 13-18 with a design soul: CSS mastery, Tailwind, animations, JavaScript, React for UI and design systems, in 9-11 months.

In one line: The frontend course with a design soul: CSS mastered as a craft, interfaces that feel expensive, and React learned as a UI instrument.

ABOUT THE PROGRAMME

Every interface a teen admires, the smooth landing page, the app that feels expensive, is frontend craft, and craft is exactly what most courses skip on the way to frameworks. This live online course for ages 13 to 18 takes the design-forward road: HTML and CSS mastered properly, flexbox, grid, responsive discipline, then the styling craft layer, modern CSS, SASS architecture, Tailwind, and animations that make pages feel alive, on real JavaScript foundations. React arrives as a UI instrument: components, hooks, styling strategies and a design system built by the student. The final month adds accessibility and performance, the two marks of professional frontend, and ships a capstone with genuine polish. This course pairs with our React for Teens course rather than repeating it: React for Teens goes deeper into application logic and TypeScript; this course goes deeper into the visual craft, CSS depth, animation, design systems, that makes interfaces worth using. Ambitious students take both roads over time. The pace is honest: nine months at two live classes a week plus three to four hours of practice, with two more months in hand. Homework, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest. Real craft, real interfaces, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- CSS as a craft, not a chore: modern CSS, SASS architecture, Tailwind and animation get real weeks
- Design systems built, not mentioned: the student ships a component library of their own
- Accessibility and performance as the finishing school: the two marks of professional frontend
- React taught as a UI instrument, complementing (not repeating) our React for Teens course
- Everything deploys: public URLs from the first phase
- Real assessment: weekly homework, monthly mixed reviews, and a final exam that decides the certificate, with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Months 1-3): HTML, CSS, layouts, responsive discipline and first JavaScript
Phase 2: The styling craft (Months 4-6): DOM and APIs, modern CSS, SASS architecture, Tailwind and animation
Phase 3: React for UI builders (Months 7-8): components, hooks, styling strategies and a design system of your own
Phase 4: The finishing school (Month 9): accessibility, performance, and the polished capstone exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The course starts at the first tag
Computer skills: Comfortable typing and managing files
Equipment: A computer with a stable internet connection; every tool used is free
Age: 13 to 18. Batches are grouped by age and pace
Time commitment: 2 live classes plus 3-4 hours of practice a week

WHO THIS COURSE IS FOR

Design minded: Teens who notice spacing and motion everywhere and want to build what they admire
Complete beginners: Students starting from zero on the visual-craft road
React teens pairs: Students who took (or will take) React for Teens and want the UI-craft half
Portfolio builders: Teens who want deployed work that visibly looks better than average
Future designers: Students weighing design against engineering, building both muscles

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS (MONTHS 1-3, WEEKS 1-12)

HTML with meaning, CSS with discipline, layouts that survive every screen, and first

JavaScript.

Month 1: Structure and Style (Weeks 1-4)
Introduction to Web Development
Topics:
- How websites work: The magic behind Instagram and TikTok
- Frontend vs Backend: What's the difference?
- Browser DevTools: Your secret weapon
- Setting up VS Code: Your coding workspace
- Essential extensions and themes (make it look cool!)
- HTML basics: The skeleton of websites
- HTML tags: headers, paragraphs, links, images
- Semantic HTML5: header, nav, main, footer
- Creating your first webpage
- Lists, tables, and organization
- Forms and user input
- Embedding videos, audio, and maps

Projects built:
- Personal introduction webpage
- Favorite artist fan page
- School project presentation site
- Recipe collection page

Practice: Daily: Create one new HTML page exploring different elements

CSS Styling & Design
Topics:
- CSS: Making websites beautiful
- Selectors: Targeting elements like a sniper
- Colors and gradients: Creating stunning palettes
- Typography: Fonts that make impact
- Google Fonts and custom fonts
- Box Model: The secret to layouts
- Padding, margins, borders explained
- Backgrounds: Images, patterns, videos
- CSS animations: Making things move
- Transitions: Smooth interactions
- Hover effects that wow
- Transform: Rotate, scale, skew

Projects built:
- Animated landing page
- CSS art challenge
- Interactive button collection
- Photo gallery with effects

Practice: Recreate 5 website sections from popular sites

Assessment: Month 1 check: style a page live to match a design, plus a take-home mixing everything so far

Month 2: Layout Mastery (Weeks 5-8)
Flexbox & Grid Layouts
Topics:
- Flexbox: The game-changer for layouts
- Flex containers and items
- Main axis vs cross axis
- Justify-content: Horizontal alignment magic
- Align-items: Vertical alignment made easy
- Flex-wrap and responsive design
- CSS Grid: Two-dimensional layouts
- Grid template areas: Visual layout design
- Auto-fit and auto-fill: Dynamic grids
- Combining Flexbox and Grid
- Building complex layouts easily
- Common layout patterns

Projects built:
- Instagram-style photo grid
- YouTube homepage layout
- Discord server layout
- Pinterest-style masonry

Practice: Build 10 different layout challenges

Responsive Design & Mobile First
Topics:
- Why mobile-first matters: Everyone's on phones
- Media queries: Adapting to screen sizes
- Breakpoints: Phone, tablet, desktop
- Responsive units: rem, em, %, vw, vh
- Responsive images and videos
- Mobile navigation patterns
- Hamburger menus and mobile UI
- Touch-friendly design
- Testing on real devices
- Chrome DevTools device mode

```
        - Common responsive mistakes
        - Performance on mobile
      Projects built:
        - Fully responsive portfolio
        - Mobile-first blog design
        - Responsive e-commerce page
        - App landing page
      Practice: Make all previous projects responsive
      Assessment: Month 2 check: build a responsive layout live from a wireframe, plus mixed
      review

  Month 3: First JavaScript (Weeks 9-12)
    JavaScript Basics
      Topics:
        - JavaScript: Making websites interactive
        - Variables: let, const, var explained
        - Data types: Numbers, strings, booleans
        - Arrays: Managing lists of data
        - Objects: Organizing complex data
        - Functions: Reusable code blocks
        - Parameters and return values
        - Arrow functions: Modern syntax
        - Template literals: Dynamic strings
        - Operators and comparisons
        - Console methods for debugging
        - Comments and code organization
      Projects built:
        - Interactive calculator
        - Quiz game with scoring
        - Todo list application
        - Random meme generator
      Practice: Complete 50 JavaScript exercises on CodeWars
    Control Flow & Logic
      Topics:
        - If/else statements: Making decisions
        - Switch statements: Multiple choices
        - Ternary operators: Shorthand conditions
        - Logical operators: AND, OR, NOT
        - Truthy and falsy values
        - For loops: Repeating actions
        - While loops: Conditional repetition
        - ForEach, map, filter, reduce
        - Break and continue
        - Try/catch: Error handling
        - Debugging strategies
        - Common logic patterns
        - Deploying your best pages to public URLs
      Projects built:
        - Password strength checker
        - Grade calculator
        - Text-based adventure game
        - Data filtering dashboard
      Practice: Solve 30 algorithm challenges
      Assessment: Phase 1 exam: a styled interactive page built live, plus a written mixed
      paper

---------------------------------------------------------------
PHASE 2: THE STYLING CRAFT (MONTHS 4-6, WEEKS 13-24)
---------------------------------------------------------------
Where interfaces start feeling expensive: the DOM, live data, modern CSS, SASS architecture,
Tailwind and animation.

  Month 4: Living Pages (Weeks 13-16)
    DOM Manipulation
      Topics:
        - DOM: The bridge between HTML and JavaScript
        - Selecting elements: querySelector magic
        - Changing content dynamically
        - Modifying styles with JavaScript
        - Creating and removing elements
        - Event listeners: Clicks, hovers, inputs
        - Event bubbling and delegation
        - Form handling and validation
        - Local storage: Saving user data
        - Keyboard and mouse events
        - Scroll events and parallax
        - Building interactive components
      Projects built:
        - Dynamic theme switcher
```

```
      - Interactive game (Tic-tac-toe)
      - Form with live validation
      - Infinite scroll gallery
    Practice: Build 15 interactive components
  Asynchronous JavaScript & APIs
    Topics:
      - Async JavaScript: Why it matters
      - Callbacks: The old way
      - Promises: The better way
      - Async/await: The modern way
      - Fetch API: Getting data from servers
      - Working with JSON data
      - Public APIs: Weather, movies, music
      - Error handling in async code
      - Loading states and spinners
      - API authentication basics
      - CORS and how to handle it
      - Building data-driven websites
    Projects built:
      - Weather app with live data
      - Movie search application
      - Spotify playlist viewer
      - GitHub profile viewer
    Practice: Integrate 10 different APIs
    Assessment: Month 4 check: an API-powered widget built live, plus mixed review

Month 5: Modern CSS and Architecture (Weeks 17-20)
  Modern CSS Techniques
    Topics:
      - CSS Custom Properties (variables)
      - CSS Grid advanced techniques
      - Container queries: The future of responsive
      - CSS nesting (native)
      - Logical properties for internationalization
      - Modern color functions: rgb(), hsl(), lab()
      - Aspect-ratio property
      - CSS scroll snap for carousels
      - Sticky positioning tricks
      - CSS counters for numbering
      - Print styles for web pages
      - Accessibility with CSS
    Projects built:
      - Magazine-style layout
      - Advanced dashboard design
      - Product showcase with filters
      - Interactive timeline
    Practice: Experiment with cutting-edge CSS features
  SASS/SCSS & CSS Architecture
    Topics:
      - SASS: CSS with superpowers
      - Variables, nesting, and mixins
      - Functions and calculations in SASS
      - Partials and imports
      - Extends and placeholders
      - BEM methodology: Organized CSS
      - CSS architecture patterns
      - Component-based styling
      - Utility-first CSS introduction
      - CSS-in-JS concepts
      - PostCSS and autoprefixer
      - Building a design system
    Projects built:
      - Design system with SASS
      - Themed website with variables
      - Component library
      - Multi-theme dashboard
    Practice: Refactor projects using SASS
    Assessment: Month 5 check: refactor a messy stylesheet live into clean architecture,
    plus mixed review

Month 6: Tailwind and Motion (Weeks 21-24)
  Tailwind CSS & Utility-First
    Topics:
      - Tailwind CSS: Why developers love it
      - Utility classes philosophy
      - Setting up Tailwind
      - Responsive utilities
      - Hover, focus, and state variants
      - Custom configuration
```

```
        - Extending Tailwind
        - Component extraction
        - Tailwind UI components
        - JIT mode benefits
        - Tailwind vs traditional CSS
        - Building with Tailwind
      Projects built:
        - Landing page with Tailwind
        - Admin dashboard UI
        - E-commerce product page
        - Social media feed clone
      Practice: Rebuild projects with Tailwind
    Animation: Interfaces That Feel Alive
      Topics:
        - CSS variables: Dynamic styling
        - CSS calculations with calc()
        - Advanced selectors and pseudo-elements
        - CSS shapes and clip-path
        - Filters and blend modes
        - 3D transforms and perspective
        - Keyframe animations
        - Animation timing functions
        - SVG animations
        - Scroll-triggered animations
        - Parallax scrolling effects
        - Performance optimization
        - Transitions with intent: feedback, hierarchy, delight
        - Keyframes and scroll-linked effects, used with restraint
        - Motion accessibility: respecting reduced-motion users
      Projects built:
        - Animated hero section
        - Interactive infographic
        - 3D card flip gallery
        - Scroll storytelling site
      Practice: Create animation library
      Assessment: Phase 2 exam: an animated component set defended, plus a cumulative written
      paper

----------------------------------------------------------------
PHASE 3: REACT FOR UI BUILDERS (MONTHS 7-8, WEEKS 25-32)
----------------------------------------------------------------
React as a UI instrument: components, hooks, styling strategies, and the crowning project of
the phase, a design system of your own.

  Month 7: React Essentials (Weeks 25-28)
    React Basics
      Topics:
        - Why React? Facebook, Instagram, Netflix use it
        - Vite: Quick start
        - JSX: HTML in JavaScript (weird but powerful)
        - Components: Building blocks of React
        - Functional vs Class components
        - Props: Passing data down
        - State: Component memory
        - useState hook explained
        - Event handling in React
        - Conditional rendering
        - Lists and keys
        - Component lifecycle
      Projects built:
        - Instagram feed clone
        - Interactive todo app
        - Weather dashboard
        - Movie gallery
      Practice: Build 20 React components
    React Hooks Deep Dive
      Topics:
        - useState: Managing component state
        - useEffect: Side effects and lifecycle
        - useContext: Avoiding prop drilling
        - useReducer: Complex state logic
        - useCallback: Optimizing functions
        - useMemo: Expensive calculations
        - useRef: DOM references and values
        - Custom hooks: Reusable logic
        - Rules of hooks
        - Hook patterns and best practices
        - Testing hooks
        - Common hook mistakes
```

```
      Projects built:
        - Chat application with hooks
        - Shopping cart with useReducer
        - Theme system with Context
        - Custom hooks library
      Practice: Create 10 custom hooks
      Assessment: Month 7 check: an interactive component built live, plus mixed review

  Month 8: Styling React and Design Systems (Weeks 29-32)
    Styling in React
      Topics:
        - CSS Modules in React
        - Styled-components introduction
        - Emotion CSS-in-JS
        - Tailwind with React
        - Material-UI (MUI) components
        - Chakra UI for rapid development
        - Ant Design components
        - Creating themed components
        - Animation libraries: Framer Motion
        - React Spring animations
        - Lottie animations in React
        - Building a design system
      Projects built:
        - Animated portfolio site
        - Dashboard with MUI
        - Landing page with animations
        - Custom component library
      Practice: Style apps with different approaches
    Your Own Design System
      Topics:
        - Design system principles
        - Design tokens
        - Component documentation
        - Storybook mastery
        - Component testing
        - Accessibility testing
        - Visual regression testing
        - Publishing to NPM
        - Versioning strategies
        - Monorepo management
        - Documentation sites
        - Team adoption
        - Documenting components so others could use them
      Projects built:
        - A personal design system: tokens, components and documentation, deployed
      Practice: Build professional component library
      Assessment: Phase 3 exam: the design system defended, plus a cumulative written paper

------------------------------------------------------------
PHASE 4: THE FINISHING SCHOOL (MONTH 9, WEEKS 33-36)
------------------------------------------------------------
The two marks of professional frontend, accessibility and performance, then the polished
capstone and the final exam.

  Month 9: Polish, Capstone, Demo Day (Weeks 33-36)
    Accessibility and SEO Basics
      Topics:
        - Web accessibility (a11y) importance
        - WCAG guidelines
        - Semantic HTML for accessibility
        - ARIA labels and roles
        - Keyboard navigation
        - Screen reader testing
        - Color contrast and vision
        - Focus management
        - SEO fundamentals
        - Meta tags and Open Graph
        - Structured data
        - SEO for SPAs
      Projects built:
        - Accessibility audit and fixes
        - Fully accessible form system
        - SEO-optimized blog
        - Accessible component library
      Practice: Make all projects accessible
    Performance: Measured, Then Fixed
      Topics:
        - Web performance metrics
```

- Core Web Vitals
- Lighthouse audits
- Bundle size optimization
- Code splitting strategies
- Lazy loading components
- Image optimization techniques
- Font optimization
- Caching strategies
- Service workers basics
- Progressive Web Apps
- Performance monitoring

Projects built:
- Optimize slow application
- PWA conversion
- Performance dashboard
- Speed optimization challenge

Practice: Optimize all projects for performance

Capstone Build

Topics:
- One focused week: an interface with visible craft
- Using the design system you built
- Responsive, accessible, animated with restraint
- Instructor checkpoint and honest scope cuts
- Deployment and the case-study README

Projects built:
- THE CAPSTONE: a polished, deployed interface built on your own design system

Practice: A deployed build at the end of every session

Demo Day and the Final Exam

Topics:
- Presenting the capstone: craft decisions explained
- A full-course spiral review before the exam
- Where each path leads: React for Teens for app logic, full-stack for the backend half

Projects built:
- Demo day presentation delivered to the batch and parents

Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; a student who falls short gets a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: styled pages, responsive layouts and deployed first projects
- Phase 2: API-powered widgets, refactored stylesheets, Tailwind builds and animated components
- Phase 3: React UI projects and a personal design system, deployed and documented
- Phase 4: an accessibility and performance pass, and the polished capstone at demo day

Total projects: 20+ deployed interfaces, crowned by a design system and a polished capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- HTML and CSS to genuine depth: layout, responsive discipline and modern features
- Styling craft: SASS architecture, Tailwind and animation with restraint
- JavaScript and the DOM: living pages with real data
- React for UI: components, hooks and styling strategies
- Design systems: tokens, components and documentation
- The professional marks: accessibility and measured performance

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building along with the instructor
Homework: Homework after every class, and a mixed-review assignment at the end of every month
Practice: 3-4 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, for the layout that collapses at 380 pixels
Progress updates: Regular progress notes to parents: what was built, what comes next, and where encouragement helps

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- React for Teens: the application-logic half of frontend, TypeScript included

- Full-stack web development: the backend half when ready
- Canva and design courses: the pure-design road, entered with real CSS understanding
- School and club sites that suddenly look professional
- The taste advantage: every future tool is easier with craft underneath

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my teen build in this frontend course?
A: More than 20 deployed interfaces across nine months: styled responsive pages, API-powered widgets, Tailwind builds, animated component sets, React UI projects, a personal design system with tokens and documentation, and a polished capstone interface presented at demo day. The visual craft is the point: work that visibly looks better than average.

Q: How is this different from your React for Teens course?
A: They are two halves of frontend, deliberately split. React for Teens goes deeper into application logic: state, TypeScript, data patterns and a product capstone. This course goes deeper into the visual craft: CSS mastery, SASS architecture, Tailwind, animation, design systems, accessibility and performance. Ambitious students take both roads over time; either order works.

Q: How long does it take, and how is my teen assessed?
A: Nine months at 2 live classes a week plus 3 to 4 hours of practice, with two more months in hand for students who need them. Homework follows every class, monthly assignments mix new questions with earlier topics, each phase ends with an exam, and a final exam, practical plus written, decides the certificate, with a free retest after revision.

Q: Does this course cover Vue, Angular or Svelte?
A: Honestly, no: framework tours produce students fluent in nothing. The craft taught here, CSS depth, component thinking, design systems, transfers to every framework in days, and React is included as the industry default UI instrument. A teen who owns this material picks up any alternative framework when a real project demands it.

Q: Is frontend still a good skill in the AI era?
A: More than ever, and specifically the craft half: AI tools generate average-looking interfaces in seconds, which makes above-average taste the differentiator. Judgment about spacing, motion, hierarchy and accessibility is exactly what this course trains, and exactly what tools cannot fake.

Q: Is there a free trial class?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor critiquing real work, never pre-recorded videos. Craft is built through critique.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/frontend-development-masterclass-for-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 61 OF 137: FULL STACK DEVELOPER COURSE: REACT, NODE.JS AND DEVOPS BASICS
==========

Course link: https://learn.modernagecoders.com/courses/full-stack-web-development-masterclass-college
Subject area: Web Development
Designed for: College students
Who can join: Complete beginner; college students, graduates and career switchers (18+)
Course length: 12-14 months (48-56 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Full stack developer course for college students and career switchers: HTML to React, Node.js, SQL and MongoDB, Docker, CI/CD and AI features in 12-14 months.

In one line: The complete road from a first HTML tag to a deployed, tested, AI-capable application, walked at a professional pace in twelve months.

ABOUT THE PROGRAMME

Full stack development remains the most employable generalist skill in software: one person who can take an idea from empty folder to running product. This live online masterclass builds that person deliberately, from absolute zero, using the stack that dominates hiring in 2026: JavaScript and TypeScript, React and Next.js, Node and Express, MongoDB and PostgreSQL, with Docker, CI/CD and cloud fundamentals where employers expect them. The arc runs in four phases. Foundations first: HTML, CSS and JavaScript learned to real depth, including the asynchronous patterns most self-taught developers never quite trust. Then the modern frontend: TypeScript, React with hooks and honest state management, styling with Tailwind, and a production-grade frontend capstone. Then the backend: Express APIs designed properly, both database families (document and relational), authentication done safely, OWASP security and real-time features. The final phase is what separates course graduates from employable developers: testing, Next.js, performance, Docker, CI pipelines, cloud fundamentals, a system design introduction, AI features built with guardrails, and a capstone shipped like a product. The pace is honest: two live classes a week plus four to six hours of practice fits this in twelve months, and learners who need longer take up to fourteen, the batch adapts. Assessment is honest too: homework after every class, monthly assignments that mix new questions with earlier topics, cumulative phase exams, and a final exam that decides the certificate. Real knowledge, real work, real projects, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- One coherent stack learned to depth: JavaScript, TypeScript, React, Node, both SQL and MongoDB, the combination hiring actually asks for
- Professional habits are the syllabus, not a bonus: Git workflow, testing, code review, CI, Docker and deployment run through the course
- Everything ships: projects live on public URLs from month 3, ending in a deployed, tested capstone
- AI features built properly: server-side model calls, streaming, retrieval over your own data, guardrails and cost control
- An honest 12-month arc with 2 months of margin built in: learners who need more practice take up to 14, nobody is rushed
- Real assessment: weekly homework, mixed-review assignments every month, and a final exam that decides the certificate, with focused revision and a free retest for those who fall short

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Months 1-3): HTML, CSS, layouts, then JavaScript to real depth, DOM, events and async, ending deployed
Phase 2: Modern frontend (Months 4-6): ES6+ mastery, OOP and functional patterns, TypeScript, React, state management and a frontend capstone
Phase 3: Backend engineering (Months 7-9): Node, Express, REST design, MongoDB and PostgreSQL, auth, security and real-time
Phase 4: Production (Months 10-12): testing, Next.js, performance, Docker, CI/CD, cloud fundamentals, system design intro, AI features and the final capstone

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The course starts from how computers and the web actually work
Computer skills: Comfortable installing software and managing files
Equipment: A computer with a stable internet connection; every tool used in the course is free
Age: 18+. College students, recent graduates and working professionals switching careers
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

College students: Students who want employable, portfolio-proven skills alongside the degree
Career switchers: Professionals from any field ready to move into software properly, not through shortcuts
Self taught stuck: Self-taught learners with tutorial miles but no structure, who need the gaps closed
Builders: People with a product idea who want the ability to build it end to end
Ai curious: Developers-to-be who want AI features as a working skill, not a buzzword

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS (MONTHS 1-3, WEEKS 1-12)

Computing, HTML, CSS and JavaScript built to real depth. The phase ends with an interactive, API-powered project deployed to a public URL.

Month 1: How the Web Is Built (Weeks 1-4)
  Computing, the Web and Your Toolkit
    Topics:
      - How computers work: CPU, Memory, Storage
      - Binary, data representation, and encoding
      - Operating systems basics (Windows, Mac, Linux)
      - Internet fundamentals: How the web works
      - Client-server architecture explained
      - IP addresses, DNS, and HTTP basics
      - Development environment setup
      - VS Code installation and configuration
      - Terminal/Command Line basics
      - Git installation and first commands
      - Creating your first GitHub account
      - Introduction to HTML: What is markup?
    Projects built:
      - Set up complete development environment
      - Create first GitHub repository
      - Write 'About Me' HTML page
    Practice: Daily: 30 min terminal practice, 1 hour HTML basics
  HTML5 Done Properly
    Topics:
      - HTML document structure and anatomy
      - Semantic HTML5 elements (header, nav, main, section, article, aside, footer)
      - Text elements: headings, paragraphs, lists, quotes
      - Links and navigation (absolute vs relative URLs)
      - Images: img tag, srcset, responsive images
      - Tables: structure, accessibility, styling
      - Forms: input types, labels, validation
      - Form elements: textarea, select, radio, checkbox
      - HTML5 APIs: audio, video, canvas basics
      - Meta tags and SEO fundamentals
      - Accessibility basics: ARIA labels, semantic structure
      - HTML best practices and code organization
    Projects built:
      - Personal portfolio website (HTML only)
      - Restaurant menu page with images and tables
      - Multi-page contact form with validation
    Practice: Build 5 different webpage layouts from scratch
  CSS Fundamentals
    Topics:
      - CSS syntax: selectors, properties, values
      - Three ways to add CSS: inline, internal, external
      - CSS selectors: element, class, ID, descendant, child
      - CSS specificity and cascade rules
      - Color systems: hex, rgb, rgba, hsl, hsla
      - Typography: font-family, size, weight, line-height, letter-spacing
      - Text styling: alignment, decoration, transformation
      - The Box Model: content, padding, border, margin
      - Width, height, and box-sizing
      - Display property: block, inline, inline-block, none
      - Background properties: color, image, position, repeat, size
      - CSS units: px, em, rem, %, vh, vw
    Projects built:
      - Style your portfolio with beautiful CSS
      - Create a styled blog post layout
      - Design a product card component
    Practice: Recreate 10 designs from screenshots using HTML/CSS
  Flexbox, Grid and Responsive Layouts
    Topics:
      - CSS positioning: static, relative, absolute, fixed, sticky
      - Z-index and stacking contexts
      - Float and clear (legacy layout)
      - Flexbox: flex container and flex items
      - Flexbox properties: justify-content, align-items, flex-direction, flex-wrap
      - Flex grow, shrink, and basis
      - CSS Grid: rows, columns, and grid template
      - Grid areas and named grid lines
      - Grid gap, auto-fit, auto-fill, minmax
      - Responsive design principles
      - Media queries: breakpoints and mobile-first approach
      - Viewport meta tag and mobile optimization
    Projects built:
      - Holy grail layout with Flexbox and Grid
      - Responsive navigation menu
      - Photo gallery with Grid
      - Complete responsive landing page
    Practice: Build 5 different layout patterns (sidebar, header-footer, card grid, etc.)

```
    Assessment: Month 1 check: build a responsive layout live from a wireframe, plus a
    take-home assignment mixing all of month 1

Month 2: JavaScript Fundamentals (Weeks 5-8)
  JavaScript Basics and Syntax
    Topics:
      - What is JavaScript? History and ecosystem
      - Adding JavaScript to HTML: script tags, external files
      - Console and debugging basics
      - Variables: var, let, const (differences and scope)
      - Data types: string, number, boolean, null, undefined, symbol
      - Type conversion and coercion
      - Operators: arithmetic, comparison, logical, assignment
      - String methods: length, charAt, slice, indexOf, replace
      - Template literals and string interpolation
      - Comments and code documentation
      - Naming conventions and code style
      - First program: interactive calculator
    Projects built:
      - Temperature converter
      - Simple tip calculator
      - Age calculator with leap year handling
    Practice: Solve 20 basic JavaScript problems on variables and operators
  Control Flow, Loops and Arrays
    Topics:
      - If-else statements and conditions
      - Switch statements
      - Ternary operator
      - Truthy and falsy values
      - Logical operators: &&, ||, !
      - Loops: for, while, do-while
      - Break and continue statements
      - Nested loops and patterns
      - Arrays: creation, indexing, length
      - Array methods: push, pop, shift, unshift
      - Iterating arrays with loops
      - Multi-dimensional arrays
    Projects built:
      - Number guessing game
      - FizzBuzz challenge
      - Pattern printing (stars, pyramids)
      - Shopping cart with array manipulation
    Practice: Solve 30 logic problems on loops and arrays
  Functions, Scope and Closures
    Topics:
      - Function declaration vs expression
      - Parameters and arguments
      - Return statements
      - Default parameters
      - Rest parameters and spread operator
      - Arrow functions (ES6)
      - Function scope and hoisting
      - Lexical scope and closure
      - IIFE (Immediately Invoked Function Expressions)
      - Callback functions introduction
      - Higher-order functions basics
      - Pure functions and side effects
    Projects built:
      - Reusable utility function library
      - Password strength validator
      - Text analyzer (word count, reading time)
      - Simple quiz application
    Practice: Refactor previous projects using functions
  Objects and Data Structures
    Topics:
      - Object literals and properties
      - Accessing properties: dot vs bracket notation
      - Adding and deleting properties
      - Methods and 'this' keyword
      - Object destructuring
      - Object.keys, values, entries
      - JSON: parsing and stringifying
      - Nested objects and arrays
      - Iterating over objects
      - Object spread operator
      - Sets and Maps introduction
      - Comparing objects vs primitives
    Projects built:
      - Student management system
```

```
      - Todo list with objects
      - Contact book application
      - Product inventory manager
    Practice: Build 5 mini-projects using objects and arrays
    Assessment: Month 2 check: solve a data-modeling exercise live, with quiz questions
    reaching back to month 1

Month 3: The Interactive Web (Weeks 9-12)
  The DOM
    Topics:
      - What is the DOM? Tree structure
      - Window and document objects
      - Selecting elements: getElementById, getElementsByClassName, querySelector
      - querySelectorAll and NodeList vs HTMLCollection
      - Traversing the DOM: parent, children, siblings
      - Creating elements: createElement, createTextNode
      - Modifying content: innerHTML, textContent, innerText
      - Modifying attributes: getAttribute, setAttribute
      - Modifying styles: style property, classList
      - Adding and removing classes
      - Inserting elements: appendChild, insertBefore, append, prepend
      - Removing elements: removeChild, remove
    Projects built:
      - Dynamic HTML generator
      - Color scheme switcher
      - Content management system (CMS) simulator
      - Live form preview
    Practice: Manipulate DOM in 15 different ways
  Events and User Interaction
    Topics:
      - Event listeners: addEventListener, removeEventListener
      - Common events: click, dblclick, mouseenter, mouseleave
      - Keyboard events: keydown, keyup, keypress
      - Form events: submit, focus, blur, input, change
      - Event object and properties
      - Event bubbling and capturing
      - Event delegation pattern
      - preventDefault and stopPropagation
      - Creating custom events
      - DOMContentLoaded vs load event
      - Mouse position and coordinates
      - Touch events for mobile
    Projects built:
      - Interactive image gallery with lightbox
      - Drag and drop interface
      - Keyboard-controlled game
      - Real-time form validation
      - Modal window system
    Practice: Create 10 interactive UI components
  Practical JavaScript: Errors, Storage and Regex
    Topics:
      - Array methods: map, filter, reduce, find, findIndex
      - forEach, some, every
      - Sort and reverse
      - Method chaining
      - String methods: split, join, trim, includes
      - Regular expressions basics
      - Date and time in JavaScript
      - Math object and methods
      - Error handling: try-catch-finally
      - Throwing custom errors
      - Strict mode
      - localStorage and sessionStorage
    Projects built:
      - Data filtering and sorting dashboard
      - Expense tracker with local storage
      - Date and time utilities app
      - Form validation with regex
    Practice: Master array methods with 25 exercises
  Async JavaScript, APIs and the First Deploy
    Topics:
      - Synchronous vs asynchronous code
      - setTimeout and setInterval
      - Callbacks and callback hell
      - Promises: creating and consuming
      - Promise chaining
      - Promise.all, Promise.race, Promise.allSettled
      - Async/await syntax
      - Error handling with async/await
```

```
        - Fetch API basics
        - Making HTTP requests
        - Working with APIs and JSON
        - CORS and API authentication basics
        - Deploying the phase capstone to a public URL
      Projects built:
        - Weather app using a live API
        - GitHub profile viewer
        - PHASE CAPSTONE: an interactive, API-powered application, deployed
      Practice: Integrate 3 public APIs, then ship the capstone
      Assessment: Phase 1 exam: build a small app with live data in class, plus a written
      mixed paper over months 1-3

------------------------------------------------------------
PHASE 2: TYPESCRIPT AND REACT (MONTHS 4-6, WEEKS 13-24)
------------------------------------------------------------
Modern JavaScript mastery, TypeScript from the start, and React the way production teams
write it, ending in a frontend capstone.

  Month 4: Modern JavaScript Mastery (Weeks 13-16)
    ES6+ in Depth
      Topics:
        - Destructuring: arrays and objects (deep)
        - Spread and rest operators (advanced usage)
        - Enhanced object literals
        - Computed property names
        - For...of and for...in loops
        - Iterators and generators
        - Optional chaining (?.)
        - Nullish coalescing operator (??)
        - Private class fields
      Projects built:
        - Build utility library with modern JS
        - Data transformation pipeline
        - Custom iterator implementation
      Practice: Refactor old code using modern ES6+ features
    Object-Oriented JavaScript
      Topics:
        - Constructor functions
        - Prototypes and prototype chain
        - ES6 Classes syntax
        - Class inheritance and extends
        - Super keyword
        - Static methods and properties
        - Getters and setters
        - Public, private, and protected fields
        - Encapsulation principles
        - Polymorphism in JavaScript
        - Composition over inheritance
        - Design patterns: Factory, Singleton, Module
      Projects built:
        - Bank account system with OOP
        - Game character class hierarchy
        - Library management system
        - Vehicle rental system
      Practice: Build 5 different systems using OOP principles
    Functional Programming Patterns
      Topics:
        - First-class functions
        - Higher-order functions
        - Pure functions and immutability
        - Function composition
        - Currying and partial application
        - Closures (advanced)
        - Recursion and recursive patterns
        - Memoization and optimization
        - Functional array methods (advanced)
        - Immutable data structures
        - Functional vs OOP: when to use what
      Projects built:
        - Functional utility library
        - Data pipeline with function composition
        - Recursive algorithms (tree traversal, etc.)
      Practice: Solve 30 problems using functional approach
    Modules, npm and Build Tools
      Topics:
        - ES6 modules: import/export
        - Named vs default exports
        - CommonJS vs ES modules
```

```
      - NPM and package management
      - Package.json explained
      - Semantic versioning
      - Installing and managing dependencies
      - Webpack basics: entry, output, loaders
      - Webpack dev server
      - Babel for transpiling
      - Bundling and code splitting
      - Environment variables
    Projects built:
      - Set up a custom build pipeline
      - Multi-page application with a modern bundler
    Practice: Configure build tools for 3 different project types
    Assessment: Month 4 check: refactor a legacy snippet live using modern patterns, plus a
    mixed assignment reaching back to phase 1

Month 5: TypeScript and React Core (Weeks 17-20)
  TypeScript Fundamentals
    Topics:
      - Why TypeScript? Benefits and ecosystem
      - TypeScript setup and tsconfig.json
      - Basic types: string, number, boolean, any, unknown
      - Arrays and tuples
      - Enums and literal types
      - Functions and optional parameters
      - Type inference vs explicit typing
      - Union and intersection types
      - Type aliases and interfaces
      - Compiling TypeScript
      - TypeScript with vanilla JS projects
    Projects built:
      - Convert JavaScript project to TypeScript
      - Todo app with TypeScript
      - Type-safe utility functions
    Practice: Write TypeScript for all future projects
  React Fundamentals
    Topics:
      - What is React? Virtual DOM explained
      - Vite setup
      - JSX syntax and rules
      - Components: functional vs class
      - Props and prop drilling
      - Props validation with PropTypes
      - Rendering lists with map
      - Conditional rendering techniques
      - React fragments
      - Component composition
      - Children prop pattern
      - Event handling in React
    Projects built:
      - Component library showcase
      - Profile card components
      - Product listing page
      - Interactive counter app
    Practice: Build 20 reusable React components
  Hooks and State
    Topics:
      - useState hook: primitive and object state
      - State updates and batching
      - useEffect hook: lifecycle equivalent
      - useEffect dependencies array
      - Cleanup functions in useEffect
      - useContext for prop drilling solution
      - useReducer for complex state
      - Custom hooks creation
      - useRef for DOM access and mutable values
      - useMemo for expensive calculations
      - useCallback for function memoization
      - Rules of hooks
    Projects built:
      - Todo app with hooks
      - Theme switcher with Context
      - Form builder with useReducer
      - Custom hooks library
    Practice: Create 10 custom hooks for common use cases
  Advanced React Patterns
    Topics:
      - Controlled vs uncontrolled components
      - Form handling and validation
```

- React Router: routes, navigation, params
- Nested routes and layouts
- Protected routes and authentication
- Lazy loading and code splitting
- React.memo for performance
- Error boundaries
- Portals for modals and tooltips
- Higher-Order Components (HOC)
- Render props pattern
- Compound components pattern

Projects built:
- Multi-page app with React Router
- Complex form with validation
- Dashboard with authentication
- Modal and notification system

Practice: Build 5 advanced UI patterns
Assessment: Month 5 check: build a routed, validated mini-app live, plus a mixed quiz over months 1-5

Month 6: State, Styling and the Frontend Capstone (Weeks 21-24)
State Management with Redux Toolkit
Topics:
- Why external state management?
- Redux fundamentals: store, actions, reducers
- Redux Toolkit (modern Redux)
- createSlice and configureStore
- Redux DevTools
- Async logic with Redux Thunk
- Zustand (lightweight alternative)
- When to use Context vs Redux
- State management best practices

Projects built:
- E-commerce app with Redux
- Social media feed with state management
- Multi-user chat application
- Complex dashboard with multiple data sources

Practice: Refactor Context apps to use Redux
React with TypeScript
Topics:
- Setting up React with TypeScript
- Typing props and state
- Event handlers type
- Children and generic components
- Hooks with TypeScript
- Context API with TypeScript
- Redux with TypeScript
- Type-safe API calls
- Utility types for React

Projects built:
- Rewrite React projects in TypeScript
- Type-safe form builder
- Strongly-typed Redux application

Practice: All future React projects in TypeScript
Styling: Tailwind and Component Libraries
Topics:
- CSS Modules in React
- Styled Components library
- Tailwind CSS with React
- Tailwind configuration and customization
- Component libraries: Material-UI (MUI)
- Theme management
- Responsive design in React
- CSS animations in React
- Framer Motion for animations

Projects built:
- Design system with Styled Components
- Dashboard with Tailwind CSS
- Animated landing page with Framer Motion
- Custom theme system

Practice: Restyle your best project with Tailwind and one component library
Frontend Capstone
Topics:
- Scoping a production-feel frontend in one week
- Component architecture planned before coding
- TypeScript throughout, loading and error states everywhere
- State management where it earns its keep
- Responsive, accessible and fast
- Deployment and the case-study README

Projects built:

```
        - PHASE CAPSTONE: a polished, deployed React + TypeScript application: dashboard, social feed
          or store frontend
      Practice: Daily commits; one feature finished per session
      Assessment: Phase 2 exam: capstone walkthrough plus a written mixed paper spanning
      phases 1 and 2

------------------------------------------------------------
PHASE 3: BACKEND ENGINEERING (MONTHS 7-9, WEEKS 25-36)
------------------------------------------------------------
Node and Express done properly, both database families, safe authentication, OWASP security
and real-time, ending with a deployed backend capstone.

  Month 7: Node and Express (Weeks 25-28)
    Node.js Fundamentals
      Topics:
        - What is Node.js? V8 engine explained
        - Node.js architecture and event loop
        - Blocking vs non-blocking code
        - Global objects: process, __dirname, __filename
        - Modules system: require and module.exports
        - Core modules: fs, path, os, http
        - File system operations: read, write, append, delete
        - Asynchronous file operations
        - Buffer and streams
        - Event emitters
        - Creating HTTP server with Node.js
        - Handling requests and responses
      Projects built:
        - File-based note-taking CLI
        - Simple HTTP server
        - File upload utility
        - Log management system
      Practice: Build 10 CLI tools with Node.js
    Express.js Framework
      Topics:
        - Express.js introduction and setup
        - Routing: GET, POST, PUT, DELETE
        - Route parameters and query strings
        - Middleware concept and usage
        - Built-in middleware: express.json, express.static
        - Custom middleware creation
        - Error handling middleware
        - Request and response objects
        - Sending responses: JSON, HTML, files
        - Template engines: EJS, Pug
        - Serving static files
        - Express Router for modular routes
      Projects built:
        - RESTful API for blog
        - User management system
        - File upload service
        - Server-side rendered website
      Practice: Build 5 different APIs with Express
    RESTful API Design
      Topics:
        - REST principles and constraints
        - HTTP methods semantics
        - Status codes: 2xx, 3xx, 4xx, 5xx
        - API versioning strategies
        - Resource naming conventions
        - Query parameters for filtering, sorting, pagination
        - Request/response body structure
        - Error response format
        - API documentation with Swagger/OpenAPI
        - Postman for API testing
        - HATEOAS principle
        - API best practices
      Projects built:
        - Complete REST API for task management
        - E-commerce product API
        - Social media API endpoints
        - API documentation with Swagger
      Practice: Design and document 5 different API systems
    Production Express: Middleware and Hardening
      Topics:
        - Application-level vs router-level middleware
        - Third-party middleware: helmet, cors, morgan
        - Rate limiting and throttling
        - Request validation with Joi/Yup
```

```
      - File upload with Multer
      - Cookie and session management
      - CSRF protection
      - Compression middleware
      - Graceful shutdown
      - Environment-based configuration
      - Logging with Winston
      - Security best practices
    Projects built:
      - Secure API with all middleware
      - File upload service with validation
      - Session-based authentication system
      - Production-ready Express template
    Practice: Secure and optimize Express applications
    Assessment: Month 7 check: design and defend an API surface live, plus a mixed quiz
    reaching back to earlier phases

Month 8: Both Database Families (Weeks 29-32)
  MongoDB
    Topics:
      - SQL vs NoSQL databases
      - MongoDB introduction and architecture
      - MongoDB installation and setup
      - MongoDB shell commands
      - Collections and documents
      - CRUD operations in MongoDB
      - Query operators: comparison, logical, element
      - Update operators: $set, $inc, $push, $pull
      - Aggregation pipeline basics
      - Indexing for performance
      - Data modeling in MongoDB
      - Embedded documents vs references
    Projects built:
      - User database with MongoDB
      - Product catalog system
      - Blog posts with comments
      - Analytics data aggregation
    Practice: Design 10 different database schemas
  Mongoose ODM
    Topics:
      - Mongoose introduction and setup
      - Schema definition and types
      - Schema validation rules
      - Custom validators
      - Default values and required fields
      - Virtuals and getters/setters
      - Instance methods and static methods
      - Query helpers
      - Middleware (pre/post hooks)
      - Population for references
      - Discriminators for inheritance
      - Schema design patterns
    Projects built:
      - User management with Mongoose
      - E-commerce data models
      - Social network schema
      - Complex relationships modeling
    Practice: Build 5 complex data models with Mongoose
  SQL and PostgreSQL
    Topics:
      - Relational database concepts
      - PostgreSQL installation and setup
      - SQL basics: SELECT, INSERT, UPDATE, DELETE
      - WHERE clause and filtering
      - ORDER BY, LIMIT, OFFSET
      - Aggregate functions: COUNT, SUM, AVG, MIN, MAX
      - GROUP BY and HAVING
      - JOINs: INNER, LEFT, RIGHT, FULL
      - Subqueries and nested queries
      - Data types in PostgreSQL
      - Constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK
      - Normalization (1NF, 2NF, 3NF)
    Projects built:
      - Library management system
      - Inventory tracking database
      - School database with relationships
      - Complex reporting queries
    Practice: Write 50 SQL queries of increasing complexity
  Prisma and Choosing Your Database
```

```
    Topics:
      - Model definition and associations
      - Prisma ORM introduction
      - Prisma schema and migrations
      - Type-safe database queries
      - Connection pooling
      - Database transactions in Node.js
      - Choosing the right database
      - Polyglot persistence
      - Prisma as the modern default for typed Node backends
    Projects built:
      - Type-safe API with Prisma
      - The same feature built twice: Mongo vs Postgres, with a written comparison
    Practice: Model one real domain in both database families
    Assessment: Month 8 check: schema design exercise live, plus a mixed assignment over
    phases 1-3 so far

Month 9: Auth, Security and Real-Time (Weeks 33-36)
  Authentication and Authorization
    Topics:
      - Authentication vs authorization
      - Password hashing with bcrypt
      - Salt and rainbow table attacks
      - Session-based authentication
      - Token-based authentication
      - JWT (JSON Web Tokens) structure
      - Creating and verifying JWTs
      - Access tokens and refresh tokens
      - OAuth 2.0 protocol
      - Social login: Google, Facebook, GitHub
      - Passport.js middleware
      - Role-based access control (RBAC)
    Projects built:
      - Complete auth system with JWT
      - Social login integration
      - Multi-role authorization system
      - Password reset functionality
    Practice: Implement authentication in 5 different ways
  API Security: The OWASP Essentials
    Topics:
      - OWASP Top 10 vulnerabilities
      - SQL injection prevention
      - NoSQL injection attacks
      - Cross-Site Scripting (XSS)
      - Cross-Site Request Forgery (CSRF)
      - Security headers: helmet.js
      - CORS configuration and security
      - Rate limiting strategies
      - API keys and secrets management
      - Environment variables and .env
      - Input validation and sanitization
      - HTTPS and SSL/TLS
    Projects built:
      - Secure API with all protections
      - Security audit tool
      - Rate limiting service
      - Input validation library
    Practice: Security audit 5 existing APIs
  Real-Time with Socket.io
    Topics:
      - WebSockets protocol
      - Socket.io library
      - Real-time events: emit, on, broadcast
      - Rooms and namespaces
      - Private messaging
      - Real-time notifications
      - Server-Sent Events (SSE)
      - Real-time database sync
      - Building chat applications
    Projects built:
      - Real-time chat application
      - Live notification system
      - Collaborative editor
      - Real-time dashboard
      - Multiplayer game backend
    Practice: Add real-time features to previous projects
  Backend Capstone
    Topics:
      - Scoping a complete API product
```

- Resource design, auth and validation before code
- Environment configuration and secrets done right
- Deploying the API with its database
- Documentation a stranger could build against
- A cumulative review of the backend phase

Projects built:

- PHASE CAPSTONE: a documented, authenticated, deployed API with real-time features: social backend, store backend or project tool

Practice: Have a classmate integrate against your API using only the docs

Assessment: Phase 3 exam: API walkthrough from schema to deployed URL, plus a written mixed paper over phases 1-3

PHASE 4: PRODUCTION ENGINEERING AND THE CAPSTONE (MONTHS 10-12, WEEKS 37-48)

What separates course graduates from employable developers: testing, Next.js, performance, Docker, CI/CD, cloud fundamentals, system design, AI features, and a shipped capstone.

Month 10: Quality and the Modern Stack (Weeks 37-40)

Testing Across the Stack

Topics:

- Introduction to testing: why test?
- Jest testing framework
- React Testing Library
- Writing unit tests
- Testing components and hooks
- Mocking API calls and modules
- Integration testing
- Test-driven development (TDD)
- Testing best practices
- Code coverage reports
- API testing with supertest
- What to test first when time is short

Projects built:

- Add tests to existing projects
- TDD todo application
- E2E tests for complete user flow
- Testing utilities library

Practice: Write tests for all previous projects

Next.js and Server-Side Rendering

Topics:

- Next.js introduction and setup
- File-based routing
- Pages and layouts
- Server-side rendering (SSR)
- Static site generation (SSG)
- Incremental static regeneration (ISR)
- API routes in Next.js
- Data fetching strategies
- Image optimization with next/image
- Next.js deployment
- SEO optimization

Projects built:

- Blog with Next.js SSG
- E-commerce with SSR
- Portfolio with Next.js
- Full-stack Next.js application

Practice: Migrate React app to Next.js

Performance: Measuring and Fixing

Topics:

- React DevTools Profiler
- Identifying performance bottlenecks
- Virtual scrolling for large lists
- Image optimization techniques
- Lazy loading images
- Code splitting strategies
- Bundle size optimization
- Lighthouse audits
- Core Web Vitals
- Performance monitoring tools
- Memoization best practices
- Backend performance: the N+1 query problem and friends

Projects built:

- Optimize slow React application
- Large list with virtualization
- Image gallery with lazy loading
- Performance dashboard

Practice: Audit and optimize 5 projects

Backend Patterns: Caching, Jobs and Integrations

```
    Topics:
      - MVC architecture pattern
      - Repository pattern
      - Service layer pattern
      - Dependency injection
      - SOLID principles
      - Caching strategies: Redis
      - Cache invalidation patterns
      - Background jobs: Bull queue
      - Task scheduling with cron
      - Email sending: Nodemailer
      - Payment flow anatomy, studied in test mode
    Projects built:
      - Layered architecture API
      - Background job processor
      - Email notification service
      - Payment processing system
    Practice: Refactor projects using design patterns
    Assessment: Month 10 check: a mixed assignment spanning testing, performance and
    earlier phases

Month 11: Docker, CI/CD, Cloud and AI (Weeks 41-44)
  Docker and Containers
    Topics:
      - What is Docker? Containers vs VMs
      - Docker installation and setup
      - Docker images and containers
      - Dockerfile creation
      - Docker commands: run, build, push, pull
      - Docker volumes for persistence
      - Docker networks
      - Multi-container apps with Docker Compose
      - Docker Compose YAML
      - Environment variables in Docker
      - Docker best practices
      - Container security
    Projects built:
      - Dockerize Node.js application
      - Multi-container app with database
      - Docker Compose for full stack
      - Custom Docker images
    Practice: Containerize all previous projects
  CI/CD with GitHub Actions
    Topics:
      - Continuous Integration explained
      - Continuous Deployment explained
      - Git workflow strategies
      - GitHub Actions basics
      - Writing workflow YAML files
      - Automated testing in pipelines
      - Build and deployment automation
      - Environment management
      - Blue-green deployments
      - Canary releases
    Projects built:
      - GitHub Actions pipeline
      - Automated testing workflow
      - Multi-environment deployment
      - Complete CI/CD for full stack app
    Practice: Set up CI/CD for all projects
  System Design Fundamentals
    Topics:
      - What is system design?
      - Scalability: vertical vs horizontal
      - Load balancing concepts
      - Load balancer types: Layer 4 vs Layer 7
      - Caching strategies and levels
      - Content Delivery Networks (CDN)
      - Database sharding and partitioning
      - Database replication: master-slave
      - CAP theorem explained
      - Consistency models
      - Latency vs throughput
      - Back-of-envelope calculations
    Projects built:
      - Design URL shortener
      - Design pastebin service
      - Design simple caching system
      - Load balancer simulation
```

```
        Practice: Solve 10 system design problems
      AI Features, End to End
        Topics:
          - Where AI belongs in a full-stack app
          - Server-side model calls: never from the browser
          - Streaming responses without freezing the UI
          - Structured output and validation
          - Retrieval over your own database
          - Function calling to trigger app actions
          - Cost, caching and rate limits
          - Guardrails and untrusted input
          - Telling users honestly what the AI did
          - TensorFlow.js and ML APIs: the pre-trained shortcut
        Projects built:
          - AI-powered search
          - Retrieval Q&A feature
          - Agentic action endpoint
          - Guardrail + eval harness
        Practice: Add an AI feature end to end
        Assessment: Month 11 check: design an AI feature on paper with guardrails and costs,
        plus a cumulative quiz

    Month 12: The Capstone (Weeks 45-48)
      Capstone Planning and Cloud Setup
        Topics:
          - Choosing the capstone: your strongest idea, scoped to four weeks
          - A one-page spec: users, features, stack, milestones
          - Schema and API design before UI
          - Cloud fundamentals: where each piece of your app will live
          - Repository, CI check and deployment target ready on day one
        Projects built:
          - Approved capstone spec, repository, schema and first deployed endpoint
        Practice: Pitch the spec to the batch and absorb one hard question
      Capstone Build
        Topics:
          - Vertical slices: database to API to UI, one feature at a time
          - Instructor checkpoints and course corrections
          - Tests on the logic that must not break, running in CI
          - Docker for a reproducible development environment
          - An AI feature where it genuinely helps
          - Daily commits and an always-working build
        Projects built:
          - The final capstone, built over two focused weeks with version control, tests and CI
        Practice: A working build at the end of every session, no exceptions
      Launch, Demo Day and What Comes Next
        Topics:
          - Production deployment with a custom domain and HTTPS
          - Error tracking and logs: knowing when it breaks
          - The launch checklist run end to end
          - Presenting the capstone: live demo plus the hardest bug story
          - Portfolio final pass: GitHub profile, case studies, README quality
          - Where each path leads: DSA for interviews, deeper backend, or system design at scale
        Projects built:
          - The capstone live on its own URL, presented at demo day
        Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed
        from every phase. Passing earns the certificate; a learner who falls short gets a
        focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Phase 1: hand-built responsive sites, interactive JavaScript apps, and an API-powered
    capstone deployed to a public URL
  - Phase 2: TypeScript utilities, React applications with real state management, and a polished
    frontend capstone
  - Phase 3: Express APIs against MongoDB and PostgreSQL, safe auth, OWASP hardening, real-time
    features, and a deployed backend capstone
  - Phase 4: tested, containerized, CI-checked work, AI features with guardrails, and the final
    production capstone

  Total projects: 30+ real projects, crowned by a deployed, tested full-stack application with an AI feature

SKILLS THE STUDENT WILL HAVE AT THE END

  - Frontend: HTML5, CSS3, JavaScript to depth, TypeScript, React, Redux Toolkit, Next.js,
    Tailwind
  - Backend: Node.js, Express, REST design, authentication, OWASP security, Socket.io real-time
  - Databases: MongoDB with Mongoose, PostgreSQL with SQL fluency, Prisma, schema design in both
    families
  - Production: Jest and React Testing Library, supertest, Docker, GitHub Actions CI/CD,
```

deployment
- Architecture: system design fundamentals, caching, background jobs, performance measurement
- AI features: server-side model calls, streaming, retrieval over app data, guardrails and cost control
- Professional habits: Git workflow, code review, documentation and honest estimation

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: Homework after every class, and a mixed-review assignment at the end of every month so earlier skills stay sharp
Practice: 4-6 hours of projects and challenges between classes
Review: Every submitted project reviewed with written feedback; recurring issues reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so nobody stays stuck on a broken build for days
Progress updates: Direct written feedback on every submission, plus honest monthly progress reviews

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Full stack developer roles: the portfolio and vocabulary this course builds are exactly what those interviews probe
- Data Structures and Algorithms: our dedicated course for the interview problem-solving bar
- Specialization: deeper backend engineering, React ecosystems, or DevOps from the Docker and CI base laid here
- Freelance and product work: the end-to-end skill to build for clients or for yourself
- Further AI engineering: the AI-features foundation extends into our AI and agents courses

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Growing roles: Rs 3-5 LPA (Junior Developer)
Experienced roles: Rs 6-10 LPA (Full Stack Developer); Rs 10-18 LPA (Senior Developer); Rs 15-30+ LPA (Senior/Lead roles)
Freelance: Rs 500-2,000/hour
International: $50k-120k USD based on location and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is full stack web development and why learn it?
A: Full stack development means building complete web applications: the frontend users see and the backend that powers it, databases included. It remains the most employable generalist skill in software because one full stack developer can take an idea from empty folder to running product. This course teaches the specific stack hiring asks for in 2026: JavaScript, TypeScript, React, Node, SQL and MongoDB, plus the Docker, CI and deployment habits that separate juniors who get hired from juniors who wait.

Q: How long does it take to become a full stack developer?
A: Honestly: 12 months of consistent work at 2 live classes a week plus 4 to 6 hours of practice, and this course is paced exactly for that. The syllabus keeps 2 months of margin in hand, so learners who need more repetition take up to 14 months with no rushing and no penalty. Courses promising full stack in 8 weeks are compressing a profession into a demo; the first deployed project here ships in month 3, and the depth keeps compounding after it.

Q: How am I assessed? Is the certificate just for attending?
A: No. Classwork runs inside every session, homework follows every class, and each month ends with an assignment that deliberately mixes new questions with earlier topics so knowledge compounds instead of fading. Every phase closes with a cumulative exam, and the course ends with a final exam: a practical build plus written questions drawn from the whole syllabus. Passing earns the certificate; falling short earns a focused revision plan and a free retest. We optimize for real knowledge and real depth, not attendance.

Q: Do I need prior coding experience or a computer science degree?
A: Neither. The course starts from absolute zero, how computers and the web work, and no

degree is required for full stack roles; portfolios and demonstrated skill carry the interviews. What is required is consistency: this is a 12-month professional skill, and the course is structured, assessed and reviewed to keep you consistent.

Q: Which technologies does the course cover?
A: The modern hiring stack, learned in order: HTML5, CSS3 and JavaScript to depth, then TypeScript, React, Redux Toolkit and Next.js on the frontend; Node.js, Express, MongoDB, PostgreSQL and Prisma on the backend; JWT auth and OWASP security; Socket.io real-time; Jest and supertest for testing; Docker, GitHub Actions CI/CD and cloud deployment; a system design introduction; and AI features built server-side with guardrails.

Q: What will I actually build?
A: More than 30 real projects, each phase ending with something deployed: an API-powered site in month 3, a React and TypeScript frontend capstone in month 6, an authenticated real-time backend in month 9, and a final capstone with tests, CI, Docker and an AI feature, live on its own URL. The portfolio is the point: every claim on your CV gets a working link behind it.

Q: How is AI covered in this course?
A: As an engineering skill. Students add AI features to their own applications the way production teams do: model calls made server-side, responses streamed into React, retrieval over the app's own data, structured output validation, guardrails against prompt injection, and cost control. The result is a developer who can ship an AI feature responsibly, which in 2026 is a hiring differentiator, not a novelty.

Q: What job market context should I expect?
A: Full stack developers remain among the most consistently hired software roles. The salary ranges shown on this page are general market bands from public industry data, shown for context only, never a promise: actual outcomes depend on your skill, portfolio, location and the market. What this course controls is the input: real projects, real depth and interview-ready fundamentals.

Q: Is there a free demo class?
A: Yes, book a free, no-commitment demo at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. It is a real class, not a sales pitch: you will build something small in it and meet the teaching style directly.

Q: Are classes live or recorded?
A: Every class is live: a 1-hour online session where a real instructor works with a small group, never a pre-recorded video. You code during class, ask questions in the moment, and get unstuck immediately, which matters most in the early months.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/full-stack-web-development-masterclass-college
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 62 OF 137: FULL STACK WEB DEVELOPMENT FOR TEENS: REACT AND NODE.JS

==========

Course link: https://learn.modernagecoders.com/courses/full-stack-web-development-teens-masterclass
Subject area: Web Development
Designed for: Teens
Who can join: Absolute beginner, ages 13 to 18
Course length: 12-14 months (48-56 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Full stack web development course for teens aged 13-18: HTML, CSS, JavaScript, React, Node.js, MongoDB, AI features and a deployed capstone in 12-14 months.

In one line: One honest stack, learned deeply: from a first hand-typed page to a deployed application with accounts, a database, live chat and an AI feature.

ABOUT THE PROGRAMME

Full stack means owning both halves of an application: the frontend people see and the backend that quietly runs it. This live online course takes students aged 13 to 18 from an

empty folder to exactly that, a real deployed application with user accounts, a database, real-time features and an AI-powered capability, built with the stack that dominates modern development: JavaScript, React, Node.js and MongoDB, with SQL and TypeScript where the industry expects them. The arc is deliberate. Three months of true foundations: HTML, CSS, layouts that survive a phone screen, then JavaScript learned properly, including the asynchronous side that separates real developers from tutorial followers. Three months of React, the way it is actually written in 2026: hooks, routing, TypeScript, a taste of Next.js, and a frontend capstone. Three months of backend: Node and Express, MongoDB and SQL, authentication done safely, and an API deployed to a public URL. The final three months put the halves together: file uploads, live chat with Socket.io, AI features built with proper guardrails, testing, and a four-week capstone shipped like a product. The pace is honest: two live classes a week plus three to four hours of practice fits this in twelve months, and students who need longer take up to fourteen, the class adapts instead of rushing. Learning is checked the honest way too: homework after every class, monthly assignments that mix new questions with earlier topics, and a final exam that decides the certificate. Real knowledge, real work, real projects, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- One stack learned deeply instead of four frameworks skimmed: JavaScript, React, Node and MongoDB, with SQL and TypeScript where industry expects them
- Everything ships: projects go to real public URLs from month 3 onward, not screenshots on a laptop
- AI is built, not bolted on: students add AI features to their own apps with streaming, guardrails and cost control
- An honest 12-month arc with 2 months of margin built in: students who need more practice take up to 14, nobody is rushed
- Live batches capped at 20 students, with mini-batch and 1-on-1 options for closer attention
- Real assessment: weekly homework, mixed-review assignments every month, and a final exam that decides the certificate, with focused revision and a free retest for students who fall short

THE LEARNING PATH, STEP BY STEP

Phase 1: Web foundations (Months 1-3): HTML, CSS, responsive layouts, then JavaScript learned properly through the async side
Phase 2: React frontend (Months 4-6): components, hooks, routing, TypeScript, a taste of Next.js, and a frontend capstone
Phase 3: Backend (Months 7-9): Node, Express, MongoDB and SQL, safe authentication, and a deployed full-stack app
Phase 4: The full picture (Months 10-12): uploads, live chat, AI features with guardrails, testing, and the final capstone

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The course starts from the very first HTML tag
Computer skills: Comfortable typing and managing files and folders
Equipment: A computer with a stable internet connection; every tool used in the course is free
Age: 13 to 18. Batches are grouped by age and pace so the room always fits
Time commitment: 2 live classes plus 3-4 hours of practice a week

WHO THIS COURSE IS FOR

Complete beginners: Teens starting from zero who want the full picture of how real applications are built
App dreamers: Students who look at the apps they use daily and think: I want to build one of those
Future engineers: Teens eyeing computer science degrees who want live products behind their applications
Makers: The kind of student who finishes a tutorial and immediately asks what they can build for real
Ai curious: Teens who want to put AI inside real products, not just talk to chatbots

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: WEB FOUNDATIONS (MONTHS 1-3, WEEKS 1-12)

The real base: HTML with meaning, CSS that survives a phone, and JavaScript learned deeply enough that React will make sense instead of feeling like magic.

Month 1: HTML and CSS (Weeks 1-4)
How the Web Works, and Your First Pages
Topics:
- How the internet works - explained for teens
- What happens when you open Instagram or YouTube
- Frontend vs Backend - what's the difference?

- Setting up your coding environment (VS Code)
- Essential VS Code extensions and themes
- Introduction to HTML - the skeleton of websites
- HTML tags and structure basics
- Creating your first webpage
- Adding text, headings, and paragraphs
- Links and navigation between pages
- Images and media elements
- Git and GitHub - saving your code like a pro

Projects built:
- Personal profile page
- Favorite band/artist fan page
- School project presentation website

Practice: Create 5 HTML pages about things you actually care about

Forms, Structure and Meaning

Topics:
- HTML5 semantic elements for better structure
- Creating forms - how login pages work
- Input types: text, email, password, number, date
- Dropdowns, checkboxes, and radio buttons
- HTML tables for displaying data
- Embedding content: YouTube videos, Spotify, Maps
- Audio and video elements
- Canvas element introduction
- Meta tags for SEO
- Accessibility - making sites for everyone
- HTML best practices
- Validating HTML code

Projects built:
- Multi-page school website
- Survey form for friends
- Restaurant menu with ordering form

Practice: Build 5 different forms, each validated by the browser

CSS: Making It Look Right

Topics:
- CSS basics - adding style to HTML
- Selectors: targeting elements precisely
- Colors, backgrounds, and gradients
- Typography - making text look professional
- Box Model - spacing and layout basics
- Borders, shadows, and visual effects
- CSS units: px, %, em, rem, vh, vw
- Positioning elements on the page
- Display properties and visibility
- Overflow and scrolling
- Z-index and layering
- Pseudo-classes and pseudo-elements

Projects built:
- Style your HTML projects beautifully
- Create a magazine-style layout
- Design product cards like Amazon

Practice: Recreate 5 popular website components from screenshots

Flexbox, Grid and Responsive Design

Topics:
- Flexbox - create flexible layouts easily
- CSS Grid - magazine and blog layouts
- Responsive design - mobile-first approach
- Media queries for different screen sizes
- CSS animations and transitions
- Keyframe animations
- Transform and 3D effects
- CSS variables (custom properties)
- Modern CSS features
- CSS best practices and organization

Projects built:
- Responsive portfolio website
- Animated landing page
- Instagram-style photo grid

Practice: Make every page you have built responsive

Assessment: Month 1 check: build a responsive layout live from a wireframe, plus a take-home assignment mixing all of month 1

Month 2: JavaScript, Properly (Weeks 5-8)

JavaScript Fundamentals

Topics:
- What is JavaScript? Why it's everywhere
- Variables and data types
- let, const, var - which to use when
- Numbers and math operations

- Strings and text manipulation
- Booleans and logical thinking
- Arrays - managing lists of data
- Array methods: push, pop, shift, unshift
- Objects - grouping related data
- Accessing and modifying object properties
- Console.log and debugging basics
- Comments and code documentation

Projects built:
- Interactive calculator
- Mad libs game generator
- Grade calculator for school

Practice: Solve 25 JavaScript basics challenges

Control Flow and Functions

Topics:
- If-else statements - making decisions
- Switch statements for multiple choices
- Comparison operators
- Logical operators (AND, OR, NOT)
- Ternary operator - shorthand conditions
- For loops - repeating tasks
- While and do-while loops
- Break and continue statements
- Functions - reusable code blocks
- Parameters and arguments
- Return values
- Arrow functions - modern syntax

Projects built:
- Rock, paper, scissors game
- Password strength checker
- Number guessing game

Practice: Build 8 small games and tools with JavaScript

The DOM: Making Pages Interactive

Topics:
- What is the DOM?
- Selecting elements with JavaScript
- querySelector and querySelectorAll
- Changing content with innerHTML and textContent
- Modifying styles with JavaScript
- Adding and removing CSS classes
- Creating new elements dynamically
- Event listeners - responding to user actions
- Click, hover, and keyboard events
- Form handling with JavaScript
- Local storage - saving data
- Building interactive UIs

Projects built:
- Interactive todo list
- Color scheme switcher
- Typing speed test game

Practice: Add real interactivity to 5 of your static pages

Interactive Projects Week

Topics:
- Combining structure, style and behavior in one build
- Reading someone else's code without panic
- Planning a feature before typing it
- Debugging with DevTools: breakpoints over guesswork
- Local storage: pages that remember
- Polish: empty states, small screens, keyboard use
- Presenting a build in two minutes

Projects built:
- Interactive quiz with scoring and review
- Habit tracker that survives refresh
- A mini game of your own design

Practice: One extra interactive build, chosen and scoped by you

Assessment: Month 2 check: build a small interactive feature live, with quiz questions reaching back to month 1

Month 3: Modern JavaScript and the First Deploy (Weeks 9-12)

Arrays, Objects and Data Thinking

Topics:
- Array methods: map, filter, reduce
- forEach vs map - when to use each
- Finding and searching arrays
- Sorting and reversing data
- Object methods and manipulation
- Destructuring arrays and objects
- Spread and rest operators
- Template literals for dynamic strings

- Regular expressions basics
- Date and time in JavaScript
- Math object and random numbers
- JSON - working with data

Projects built:
- Student grade management system
- Playlist manager like Spotify
- Search and filter application

Practice: Master array methods with 20 exercises

Async JavaScript and Real APIs

Topics:
- Synchronous vs asynchronous code
- setTimeout and setInterval
- Promises - handling future values
- Async/await - modern async syntax
- Fetch API - getting data from servers
- Working with REST APIs
- Handling API responses
- Error handling in async code
- Loading states and spinners
- Environment variables for API keys
- CORS and API security basics
- Popular APIs for teen projects

Projects built:
- Weather app for your city
- Meme generator using API
- Movie search app

Practice: Integrate 3 public APIs into your own pages

ES6+, Modules and npm

Topics:
- ES6+ features overview
- Classes and object-oriented programming
- Constructor functions
- Inheritance with extends
- Static methods and properties
- Modules - organizing code properly
- Import and export statements
- Default vs named exports
- NPM - the package manager
- Installing and using packages
- Package.json explained
- Building modern JavaScript projects

Projects built:
- Modular JavaScript application
- Object-oriented game
- NPM-based project setup

Practice: Refactor two old projects with modern JavaScript

Phase 1 Capstone: Ship a Real Site

Topics:
- Scoping a multi-page interactive site you can finish
- Project structure a stranger could navigate
- Fetching and displaying live API data
- Deploying to Netlify: your work on the public internet
- A pre-launch checklist: links, responsiveness, console errors
- Writing a README that explains the build

Projects built:
- The phase capstone: a multi-page, API-powered, responsive site, live at a public URL

Practice: Share the live link with three people and fix the most confusing thing

Assessment: Phase 1 exam: build a small page with live data in class, plus a written mixed paper over months 1-3

PHASE 2: THE REACT FRONTEND (MONTHS 4-6, WEEKS 13-24)

React the way it is actually written now: components, hooks, routing, real styling, TypeScript, a taste of Next.js, and a frontend capstone.

Month 4: React Core (Weeks 13-16)

Thinking in React

Topics:
- What is React? Why it's so popular
- React vs vanilla JavaScript
- Vite setup
- JSX - HTML in JavaScript
- Components - building blocks of React
- Props - passing data to components
- State - making components dynamic
- useState hook basics
- Handling events in React

```
      - Conditional rendering
      - Lists and keys in React
      - Basic React project structure
    Projects built:
      - React todo application
      - Instagram post component
      - Interactive quiz in React
    Practice: Convert 3 vanilla JavaScript projects to React
  Hooks and Effects
    Topics:
      - useEffect hook - side effects in React
      - useEffect dependencies and cleanup
      - Data fetching in React
      - Loading and error states
      - useContext for global state
      - useReducer for complex state logic
      - Custom hooks creation
      - useRef for DOM access
      - React DevTools mastery
    Projects built:
      - Twitter clone with React
      - Real-time chat interface
      - YouTube video player clone
    Practice: Build 5 custom hooks for patterns you keep repeating
  Routing and Multi-Page Apps
    Topics:
      - Single Page Applications (SPAs)
      - React Router setup and configuration
      - Routes, Route, and Switch components
      - Link and NavLink components
      - URL parameters with useParams
      - Query strings with useLocation
      - Nested routing patterns
      - Protected routes and authentication
      - Route guards and redirects
      - Lazy loading routes
      - Browser history management
    Projects built:
      - Multi-page blog platform
      - E-commerce site with routing
      - Admin dashboard with nested routes
    Practice: Add routing to two existing projects
  Fetching Data Like a Professional
    Topics:
      - Data fetching patterns in React
      - Loading, error and empty states, always
      - Search and filter over API data
      - Pagination and infinite scroll basics
      - Debouncing input before it hits the network
      - Keeping API keys out of frontend code
      - Caching responses sensibly
    Projects built:
      - Movie or music explorer with search, filters and clean loading states
    Practice: Refit an old project with proper loading and error states
    Assessment: Month 4 check: build a small data-driven component live, plus a mixed
    assignment reaching back to phase 1

Month 5: React Craft (Weeks 17-20)
  State Management That Scales
    Topics:
      - When component state stops being enough
      - Redux core concepts: Store, Actions, Reducers
      - Setting up Redux in React
      - React-Redux hooks: useSelector, useDispatch
      - Redux Toolkit - modern Redux
      - CreateSlice and createAsyncThunk
      - Handling async operations
      - Redux DevTools for debugging
      - Redux best practices
    Projects built:
      - Instagram clone with Redux
      - Shopping cart with Redux
      - Real-time notification system
    Practice: Move one app from prop-drilling to context, then to Redux Toolkit
  TypeScript for React
    Topics:
      - TypeScript fundamentals
      - Types and interfaces
      - Generics in TypeScript
```

      - TypeScript with React
      - Type-safe props and state
      - TypeScript configuration
      - Third-party library types
      - Advanced TypeScript patterns
      - TypeScript best practices
      - TypeScript tooling
    Projects built:
      - Convert JS project to TypeScript
      - Type-safe React application
      - TypeScript component library
    Practice: Write all new components in TypeScript from here on
  UI/UX for Developers
    Topics:
      - Design principles for developers
      - Color theory and palettes
      - Typography on the web
      - Layout and composition
      - Visual hierarchy
      - User experience basics
      - Wireframing and prototyping
      - Figma for developers
      - Design systems creation
      - Responsive design patterns
      - Accessibility (a11y) standards
    Projects built:
      - Design and build a design system
      - Redesign existing project
      - Create accessible component library
    Practice: Redesign one old project applying the principles
  Tailwind and Component Libraries
    Topics:
      - Tailwind CSS - utility-first styling
      - Theme management
      - Responsive design patterns
      - Mobile-first development
      - Accessibility best practices
      - Performance optimization
      - Choosing the right framework
      - Tailwind as the industry default, used with restraint
      - One component library, learned properly
      - Building your own small design system
    Projects built:
      - A polished dashboard built with Tailwind and one component library
    Practice: Restyle your best project with Tailwind
    Assessment: Month 5 check: style a component set live, plus a mixed quiz over months
    1-5

Month 6: Next.js Taste and the Frontend Capstone (Weeks 21-24)
  Performance and PWA Basics
    Topics:
      - React performance profiling
      - Identifying performance bottlenecks
      - Code splitting strategies
      - Lazy loading components
      - Image optimization techniques
      - Virtual scrolling for long lists
      - Debouncing and throttling
      - Service Workers and PWAs
      - Caching strategies
      - Bundle size optimization
    Projects built:
      - Optimize slow React app
      - Build performant social feed
      - PWA with offline support
    Practice: Measure and improve your slowest project
  A Taste of Next.js
    Topics:
      - Why the industry wraps React in Next.js
      - Pages, layouts and file-based routing
      - Server components in plain words
      - When Next.js is overkill, honestly
      - Deploying a Next.js site
      - How this course's skills map onto any framework
    Projects built:
      - A small blog or portfolio rebuilt in Next.js
    Practice: Compare your Vite and Next builds: what changed and why
  Frontend Capstone
    Topics:

- Choosing the capstone: a streaming-style UI, a social feed, or a dashboard product
- Component architecture planned before coding
- TypeScript throughout, loading states everywhere
- Responsive, accessible, and fast
- Instructor checkpoint mid-build
- Deploying and writing the case study README

Projects built:
- The frontend capstone: a polished, deployed React application that looks and feels like a real product

Practice: Daily commits; one feature finished per session

Assessment: Phase 2 exam: capstone walkthrough plus a written mixed paper spanning phases 1 and 2

PHASE 3: BACKEND AND DATABASES (MONTHS 7-9, WEEKS 25-36)

The engine room: Node and Express, MongoDB and SQL, authentication done safely, and the moment the two halves connect into one deployed application.

Month 7: Node, Express and MongoDB (Weeks 25-28)

Node.js Fundamentals

Topics:
- What is Node.js? JavaScript on the server
- Node.js installation and setup
- Node REPL and running scripts
- CommonJS modules and require
- NPM and package management
- Core modules: fs, path, http, os
- File system operations
- Creating HTTP servers
- Handling requests and responses
- Streams and buffers
- Events and EventEmitter
- Asynchronous programming in Node

Projects built:
- CLI todo application
- File management system
- Basic HTTP API server

Practice: Build 5 Node.js command-line tools

Express: Your First Server

Topics:
- Introduction to Express.js
- Setting up Express server
- Routing and route handlers
- Request and response objects
- Middleware concept and usage
- Built-in and custom middleware
- Handling different HTTP methods
- Route parameters and query strings
- Serving static files
- Template engines (EJS, Pug)
- Error handling in Express
- Express best practices

Projects built:
- RESTful blog API
- User management system
- File upload service

Practice: Build 3 small APIs with Express

MongoDB and Mongoose

Topics:
- NoSQL vs SQL databases
- MongoDB introduction and setup
- MongoDB Atlas cloud database
- CRUD operations in MongoDB
- Data modeling for NoSQL
- Mongoose ODM setup
- Schemas and models
- Validation and middleware
- Relationships and population
- Aggregation pipeline
- Indexing for performance
- MongoDB best practices

Projects built:
- Social media backend
- E-commerce product catalog
- Blog with comments system

Practice: Design and build 3 database schemas

REST API Design

Topics:

- REST principles and constraints
- API endpoint design
- HTTP status codes usage
- Request/response formats
- API versioning strategies
- Pagination and filtering
- Sorting and searching
- Error handling standards
- API documentation with Swagger
- Postman for API testing
- API rate limiting
- CORS configuration

Projects built:
- Complete REST API for social app
- E-commerce API with all features
- Real-time chat API

Practice: Design, build and document one clean API end to end

Assessment: Month 7 check: build a small CRUD endpoint live, plus a mixed quiz reaching back to earlier phases

Month 8: Authentication, SQL and Safety (Weeks 29-32)

Authentication Done Safely

Topics:
- Authentication vs authorization explained
- Password hashing with bcrypt
- JWT tokens - how they work
- Creating and verifying JWTs
- Refresh tokens pattern
- Session-based authentication
- Role-based access control (RBAC)
- Protecting API routes
- Password reset functionality
- Why passwords are never stored in plain text, ever

Projects built:
- Complete auth system with JWT
- Social login integration
- Multi-role permission system

Practice: Add signup, login and protected routes to your API

SQL and PostgreSQL Essentials

Topics:
- Relational database concepts
- PostgreSQL installation and setup
- SQL syntax and commands
- Creating tables and relationships
- Primary and foreign keys
- Joins: INNER, LEFT, RIGHT, FULL
- Aggregate functions
- Indexes and optimization
- Transactions and ACID
- Database design principles

Projects built:
- School management database
- E-commerce database design
- Social network database

Practice: Write 40 SQL queries against a real schema

API Security and Performance

Topics:
- API security best practices
- Input validation and sanitization
- SQL injection prevention
- Rate limiting implementation
- API key management
- HTTPS and SSL/TLS
- Security headers
- API performance optimization
- Caching strategies
- Performance monitoring

Projects built:
- Secure API implementation
- Performance testing suite
- API monitoring dashboard

Practice: Harden your API against the OWASP basics you now know

Backend Capstone: A Deployed API

Topics:
- Scoping an API product: resource design first
- Auth, validation and sensible error responses
- Environment variables and secrets, done right
- Deploying the API to Render
- Testing everything with a REST client

- Writing API documentation a stranger could use
Projects built:
- The backend capstone: a documented, authenticated API live on a public URL
Practice: Have a classmate build against your API using only your docs
Assessment: Phase check: API walkthrough plus a cumulative quiz from phases 1-3 so far

Month 9: The Two Halves Connect (Weeks 33-36)
Full-Stack Architecture with MERN
Topics:
- Full stack application planning
- Choosing the right tech stack
- MERN stack deep dive
- Frontend-backend communication
- API design for frontends
- State management across stack
- File upload architecture
- Email service integration
- Full stack best practices
Projects built:
- Instagram full stack clone
- E-commerce platform
- Project management tool
Practice: Wire your React capstone to your own API
Real-Time with Socket.io
Topics:
- WebSocket protocol explained
- Socket.io with Node.js
- Real-time events and broadcasting
- Rooms and namespaces
- Socket authentication
- Server-Sent Events (SSE)
- Real-time notifications
- Live collaboration features
- Chat application architecture
Projects built:
- WhatsApp clone backend
- Live collaboration tool
- Real-time gaming server
Practice: Add one live feature to your app: chat, presence or notifications
Full-Stack Deployment
Topics:
- Deploying frontend and backend as one product
- Environment configuration across services
- CORS, cookies and auth across origins, demystified
- Custom domains and HTTPS
- Reading production logs when something breaks
- A deployment checklist you will reuse for years
Projects built:
- Your full-stack app live: React frontend, Node API, database, one URL
Practice: Deploy a second app solo using only your checklist
Full-Stack Mini Build
Topics:
- A one-week product: scope ruthlessly
- Feature slices: database to API to UI, one at a time
- Where bugs hide when the stack is yours end to end
- Code review across the whole stack
- Presenting a product, not a repo
Projects built:
- A complete mini product built solo in one week: tracker, board, library or your own idea
Practice: Write the one-page retrospective: what you would do differently
Assessment: Phase 3 exam: full-stack walkthrough from schema to deployed URL, plus a
written mixed paper over phases 1-3

PHASE 4: AI, QUALITY AND THE CAPSTONE (MONTHS 10-12, WEEKS 37-48)

The final level: uploads and integrations, AI features built with guardrails, testing across
the stack, and a four-week capstone shipped like a product.

Month 10: Power-Ups and AI (Weeks 37-40)
Uploads, Email and Integrations
Topics:
- File uploads: images from the browser to storage
- Serving user images safely
- Sending transactional email from the backend
- Working with third-party APIs server-side
- Webhooks: when other services call you
- Background jobs for slow work
- Rate limits and being a polite API citizen

```
    Projects built:
      - Profile pictures and image posts added to your app
      - Email notifications for real events
    Practice: Integrate one external service end to end
  Machine Learning for Web Developers
    Topics:
      - ML basics for web developers
      - TensorFlow.js introduction
      - Pre-trained models usage
      - Image recognition in browser
      - Natural language processing
      - Recommendation systems
      - Sentiment analysis
      - ML APIs integration
      - Ethical AI considerations
    Projects built:
      - Image recognition app
      - Recommendation engine
      - Sentiment analysis tool
    Practice: Add one pre-trained model feature to a project
  AI Features with Guardrails
    Topics:
      - Where AI belongs in a full-stack app (accounts set up with a parent where a service requires
        it)
      - Server-side model calls: never from the browser
      - Streaming a response into React without jank
      - Retrieval over your own database
      - Vector search basics, and when plain SQL wins
      - Function calling to trigger real app actions
      - A minimal agent loop: plan, act, verify
      - Guardrails and output validation
      - Cost control: tokens, caching, per-user caps
      - Evaluating quality: how do you know it got better?
      - Privacy and what you must not send to a provider
      - Shipping honestly: telling users what the AI did
    Projects built:
      - AI-powered search over your own data
      - Agentic task assistant
      - AI feature with full guardrails + evals
    Practice: Run 20 real queries through your AI feature and grade the answers
  Testing Across the Stack
    Topics:
      - Why testing matters
      - Jest testing framework
      - React Testing Library
      - Writing unit tests for components
      - Testing props and state
      - Testing user interactions
      - Mocking API calls
      - Integration testing
      - Code coverage reports
      - Test-driven development (TDD)
      - Testing API endpoints with supertest
      - What to test first when time is short
    Projects built:
      - Test suites for your capstone's core logic, frontend and backend
    Practice: Write 25 tests across your two best projects
    Assessment: Month 10 check: a mixed assignment spanning AI, testing and earlier phases

Month 11: Ship Like a Professional (Weeks 41-44)
  Git Workflow and CI Basics
    Topics:
      - DevOps culture and practices
      - Version control with Git (advanced)
      - GitHub Actions for CI/CD
      - Docker in one honest lesson: what it is and when you need it
      - Branches, pull requests and reviewing a teammate's code
    Projects built:
      - A GitHub Actions check that runs your tests on every push
    Practice: Two weeks of professional Git habits: branches and clean commits
  Security Hardening Pass
    Topics:
      - Common frontend vulnerabilities
      - XSS prevention techniques
      - Content Security Policy (CSP)
      - HTTPS and SSL/TLS
      - Authentication best practices
      - Secure token storage
      - Input validation and sanitization
```

```
        - CORS and security headers
        - Dependency security scanning
        - OWASP Top 10 for frontend
      Projects built:
        - Security audit of projects
        - Implement secure authentication
        - Build security scanner tool
      Practice: Run the full security checklist against your own app and fix what fails
    Performance and Polish Pass
      Topics:
        - Measuring before optimizing: Lighthouse honestly
        - Image and bundle weight: the usual suspects
        - Database queries that quietly get slow
        - Caching where it actually helps
        - Accessibility pass: keyboard, contrast, labels
        - The empty-state and error-page polish that separates products from projects
      Projects built:
        - A measured before/after performance report on your own app
      Practice: Fix the three worst findings from your report
    Capstone Planning
      Topics:
        - Choosing the final capstone: your best idea, scoped to four weeks
        - A one-page spec: users, features, stack, milestones
        - Database schema and API sketch before any UI
        - Cutting scope without cutting the soul of the idea
        - Milestones: working core by week 2, feature-complete by week 3
      Projects built:
        - Approved capstone spec plus the repository, schema and first endpoints
      Practice: Pitch the spec to the batch and absorb one hard question
      Assessment: Month 11 check: spec review plus a mixed quiz over the whole course so far

  Month 12: The Capstone (Weeks 45-48)
    Capstone Build
      Topics:
        - Building in vertical slices: database to UI, one feature at a time
        - Instructor checkpoints and course corrections
        - Using everything: React, TypeScript, Node, database, auth, maybe AI
        - Tests on the logic that must not break
        - Daily commits and an always-working build
        - A mid-build demo to a real user
      Projects built:
        - The final capstone, built over two focused weeks with version control and tests
      Practice: A working build at the end of every session, no exceptions
    Launch Week
      Topics:
        - Production deployment: frontend, API, database
        - Custom domain, HTTPS and final configuration
        - Error tracking so you know when it breaks
        - The launch checklist run end to end
        - README, screenshots and a two-minute demo video
        - Collecting first real user feedback
      Projects built:
        - The capstone live on its own URL with documentation worth reading
      Practice: Onboard three real users and ship one fix from their feedback
    Demo Day and What Comes Next
      Topics:
        - Presenting the capstone: live demo plus the hardest bug story
        - Portfolio final pass: GitHub profile, project pages, case studies
        - An honest skills inventory: strong, shaky, next
        - Where each path leads: DSA for interviews, deeper backend, or design
        - Staying sharp: projects, communities and healthy practice habits
      Projects built:
        - Demo day presentation delivered to the batch and parents
      Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed
      from every phase. Passing earns the certificate; a student who falls short gets a
      focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Phase 1: hand-built responsive sites, interactive JavaScript apps, and an API-powered
    capstone deployed to a public URL
  - Phase 2: React applications with routing, TypeScript and Tailwind, ending in a polished
    frontend capstone
  - Phase 3: Express APIs with MongoDB and SQL, safe authentication, and a full-stack app live on
    one URL
  - Phase 4: file uploads, live chat, AI features with guardrails, tests, and the final four-week
    capstone

  Total projects: 30+ real projects, crowned by a deployed full-stack application with accounts, live features and an AI capability
```

SKILLS THE STUDENT WILL HAVE AT THE END

- Frontend: HTML, CSS, responsive layouts, JavaScript to real depth, React with hooks and routing
- Modern tooling: TypeScript, Tailwind, npm, Vite and a working taste of Next.js
- Backend: Node.js, Express, REST API design, MongoDB with Mongoose, SQL with PostgreSQL
- Auth and safety: password hashing, JWT sessions, protected routes and the OWASP basics
- Real-time and integrations: Socket.io chat, file uploads, email and third-party APIs
- AI features: model calls from the server, streaming responses, guardrails and cost control
- Professional habits: Git and GitHub, tests with Jest, CI checks, deployment and code review

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: Homework after every class, and a mixed-review assignment at the end of every month so earlier skills stay sharp
Practice: 3-4 hours of projects and challenges between classes
Review: Every submitted project reviewed with written feedback; recurring issues reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so nobody stays stuck on a broken build for days
Progress updates: Regular progress notes to parents: what was built, what comes next, and where encouragement at home helps

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Data Structures and Algorithms: the interview-and-competition muscle to pair with building skill
- Deeper backend engineering: from Express into production architectures
- Design directions: UI/UX foundations are already laid here
- College applications: a deployed product portfolio speaks louder than grades alone
- Real small projects: sites and tools for family, school clubs and local groups

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my teen build in this full stack course?
A: More than 30 real projects across twelve months, each phase ending with something deployed: a multi-page API-powered site in month 3, a polished React frontend capstone in month 6, an authenticated API and full-stack app in month 9, and a final capstone with user accounts, a database, live chat and an AI-powered feature, live on its own URL. Every project is built by the student and reviewed with written feedback.

Q: How long does it take a teenager to learn full stack development?
A: Full stack is genuinely the biggest track we teach, both halves of an application, so the honest answer is 12 months at 2 live classes a week plus 3 to 4 hours of practice. The syllabus keeps 2 months of margin in hand: students who need more repetition take up to 14 months with no rushing and no penalty. The first deployed site goes live in month 3, so the motivation never has to wait.

Q: How is my teen assessed? Is the certificate just for attending?
A: No. Classwork runs inside every session, homework follows every class, and each month ends with an assignment that deliberately mixes new questions with earlier topics so knowledge compounds instead of fading. Every phase closes with a cumulative exam, and the course ends with a final exam: a practical build plus written questions drawn from the whole syllabus. Passing earns the certificate; falling short earns a focused revision plan and a free retest. We optimize for real knowledge and real depth, not a stream of feel-good badges.

Q: Why React and Node instead of teaching many frameworks?
A: Because depth beats a tour. React and Node dominate real-world hiring and share one language, JavaScript, so a teen's effort compounds instead of splitting. We add TypeScript, SQL and a taste of Next.js because the industry expects them. A student who truly understands this one stack can pick up Vue, Angular or anything else in weeks, the concepts transfer; skimming four frameworks transfers nothing.

Q: What does full stack actually mean?
A: A full stack developer can build both, which is why this course covers the entire journey from a first webpage to a complete working application with accounts, data and live features.

Q: Can a complete beginner join, and at what age?
A: Yes, the course assumes zero coding knowledge and is designed for ages 13 to 18. The early months are deliberately gentle, real foundations before any framework, and batches are grouped by age and pace so nobody is racing or waiting.

Q: How is AI part of this course?
A: As a builder's tool, in two layers: pre-trained models in the browser with TensorFlow.js, then AI features added to the student's own full-stack app, model calls made safely from the server, streamed into React, with guardrails, cost control and honest evaluation. Where an AI service requires an account, it is set up together with a parent. The result is a teen who ships AI features, not one who only chats with AI.

Q: Does the course offer a free demo?
A: Yes, book a free, no-commitment demo class at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to sit in; it is the easiest way to see whether the teaching style clicks for your teen.

Q: What format do the classes follow?
A: Live 1-hour online classes with a real teacher, never pre-recorded videos. Groups are small and hands-on: students build during class, ask questions, and get unstuck immediately.

Q: Will my teen be certified at the end?
A: Yes, and it is earned, not handed out: the certificate is awarded on passing the final exam, with a free retest after revision if the first attempt falls short. More importantly, they finish holding a portfolio of deployed applications, live products that prove the certificate means something.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/full-stack-web-development-teens-masterclass
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 63 OF 137: FUNCTIONAL SKILLS MATHS LEVEL 1 AND LEVEL 2 COURSE ONLINE
==========

Course link: https://learn.modernagecoders.com/courses/functional-skills-maths-course
Subject area: UK Adult and Post-16 Mathematics
Designed for: College students
Who can join: Adults, apprentices and post-16 learners working towards Functional Skills maths at Level 1 or Level 2
Course length: 6 months (24 weeks) for a full level; shorter for learners who start part-way
Time needed: 2 live classes/week + 2-3 hours of practice, fitted around work
Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Functional Skills maths online for adults, apprentices and post-16 learners: Level 1 and Level 2 taught live, with calculator and non-calculator practice.

In one line: Maths for work and everyday life, taught to adults as adults: percentages, budgets, measures and data, with the exam rules learned early so nothing surprises you on the day.

ABOUT THE PROGRAMME

Functional Skills maths is a qualification built around using maths in real situations: a pay rise worked out as a percentage, a room measured for flooring, a timetable read correctly, a chart that might be misleading. The Department for Education sets the content at Entry Levels 1 to 3 and Levels 1 and 2, and every level is organised into the same three areas: using numbers and the number system, using common measures, shape and space, and handling information and data. Level 2 is the level many employers, apprenticeships and college courses ask for. Ofqual's rules shape how it is examined. A quarter of the marks are for questions done without a calculator and three quarters with one. A quarter assess underpinning skills, maths on its own, and three quarters assess problem solving in context. Several awarding organisations offer the qualification, each setting its own papers within those rules, so we teach the shared content and practise in the style your exam centre uses. The course runs in four phases: number, then measures, shape and space, then data, then exam practice with both the non-calculator and calculator sections. Lessons are live and small, aimed at people who may not have done maths for years, and they run at times that fit around work and family.

WHAT MAKES THIS COURSE DIFFERENT

- Built from the Department for Education's Functional Skills maths subject content and Ofqual's conditions and requirements
- The non-calculator quarter of the exam practised from week one, so mental methods and written methods are secure
- Problems set in real contexts adults recognise: pay, bills, recipes, travel, home improvement and workplace data
- Level 1 and Level 2 taught as clear tracks, with a starting check that places you correctly
- Practice in the style of your exam centre's awarding organisation in the final phase
- Patient, judgement-free teaching for adults returning to maths after a long break

THE LEARNING PATH, STEP BY STEP

Phase 1: Using numbers: whole numbers, fractions, decimals, percentages, ratio and proportion, with and without a calculator
Phase 2: Using measures, shape and space: money and budgeting, units and conversion, area, perimeter, volume and scale
Phase 3: Handling information and data: tables, charts and graphs, averages and range, and probability
Phase 4: Exam practice: underpinning skills and problem solving, the non-calculator and calculator sections, and full practice assessments

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: None beyond everyday numbers. The starting check places you on Level 1 or Level 2, and we fill gaps as they appear
Exam entry: The qualification is examined by an awarding organisation through an approved exam centre, such as a college or training provider. You arrange the entry; we teach the content and the exam style
Equipment: A basic calculator, a ruler, paper, and a device with stable internet for live classes
Timing: Start any month. Most learners finish a level in about six months; those already part-way through take less

WHO THIS COURSE IS FOR

Adults returning: Adults who need a maths qualification for a job, promotion or course and have not studied maths for years
Apprentices: Apprentices who need Functional Skills maths as part of their programme
Post 16 learners: Post-16 learners working towards a level 2 maths qualification
Career changers: Career changers applying for roles or training that ask for level 2 maths
Parents: Parents who want to feel confident helping with homework and family budgets

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: USING NUMBERS (MONTHS 1-2, WEEKS 1-8)

The number skills every other part of the course depends on. Level 1 learners build the foundations; Level 2 learners extend them to percentage change, ratio and inverse proportion.

Month 1: Whole Numbers, Fractions and Decimals (Weeks 1-4)
Weeks 1-2: Starting Check and Whole Numbers
Topics:
- A friendly starting check that places you on the Level 1 or Level 2 track
- Large numbers, negative numbers and place value
- Written methods for the four operations, without a calculator
- Order of operations, including brackets and powers
- Estimating and rounding to check an answer
- Times tables refreshed for the non-calculator section

Projects built:
- Personal method sheet: one worked example of each written method

Practice: 15 short non-calculator questions a week, plus 5 problems in context
Weeks 3-4: Fractions and Decimals
Topics:
- Equivalent fractions and simplifying
- Fractions of amounts in everyday situations
- Adding, subtracting and comparing fractions
- Decimals in money and measurement
- Converting between fractions and decimals
- Calculating with decimals to three decimal places at Level 2

Projects built:
- Real-life problem set: a recipe scaled and a bill split, solved with fractions

Practice: 15 fraction and decimal questions a week, half without a calculator
Assessment: Month 1 check: a 30-minute mixed number paper with a non-calculator section

Month 2: Percentages, Ratio and Proportion (Weeks 5-8)
Weeks 5-6: Percentages
Topics:
- Percentages of amounts with and without a calculator
- One amount as a percentage of another
- Percentage increase and decrease: pay rises, discounts, price rises
- Original value after a percentage change at Level 2
- Equivalence between fractions, decimals and percentages
- Checking whether a deal is really a deal

Projects built:
- Shopping comparison: three offers compared with percentages and a written recommendation

Practice: 12 percentage problems a week in work and shopping contexts

Weeks 7-8: Ratio, Proportion and Formulae
Topics:
- Ratio in mixing, sharing and scaling
- Direct proportion with recipes and costs
- Inverse proportion at Level 2: workers and time
- Formulae written in words and in symbols
- Substituting values into a formula
- Choosing whether a calculator helps or slows you down

Projects built:
- Workplace task: a staffing or mixing problem solved and explained in a short note

Practice: 12 ratio and formula problems a week

Assessment: Month 2 check: a 40-minute number paper with calculator and non-calculator sections

---

PHASE 2: MEASURES, SHAPE AND SPACE (MONTHS 3-4, WEEKS 9-16)

Maths for money, measuring and making things fit. Contexts include budgets, travel, decorating, delivery and the workplace.

Month 3: Money, Time and Units (Weeks 9-12)
Weeks 9-10: Money and Budgeting
Topics:
- Budgets, income and spending
- Simple and, at Level 2, compound interest
- Discounts and tax as percentages
- Rates of pay and comparing deals
- Reading and checking a bill
- Explaining a money decision in writing

Projects built:
- Monthly budget: a realistic household or event budget built and checked

Practice: 10 money problems a week, each with a written conclusion

Weeks 11-12: Time, Units and Compound Measures
Topics:
- Time calculations and elapsed time
- Timetables and journey planning
- Converting metric units of length, weight and capacity
- Converting between metric and imperial with a conversion factor or graph
- Speed, density and rates at Level 2
- Reading scales on measuring equipment

Projects built:
- Journey plan: a trip timed and costed from a real timetable

Practice: 12 time and unit problems a week

Assessment: Month 3 check: a 40-minute measures paper

Month 4: Shape and Space (Weeks 13-16)
Weeks 13-14: Perimeter, Area and Volume
Topics:
- Perimeter and area of rectangles and composite shapes
- Area of triangles and circles at Level 2
- Volume of cuboids and, at Level 2, cylinders
- Surface area for painting and wrapping problems
- Working out how many items fit in a space
- Adding waste allowances to real jobs

Projects built:
- Room project: flooring and paint worked out for a real room plan, with costs

Practice: 10 area and volume problems a week

Weeks 15-16: Scale, Plans and Position
Topics:
- Scale drawings and maps
- Working out real dimensions from a plan
- Plans and elevations of 3-D objects
- Coordinates, including negative values at Level 2
- Angles and direction
- Checking a plan makes sense before using it

Projects built:
- Scale plan: a simple floor plan drawn to scale from real measurements

Practice: 8 scale and plan problems a week
Assessment: Month 4 check: a 45-minute shape and space paper with a non-calculator section

---

PHASE 3: HANDLING INFORMATION AND DATA (MONTH 5, WEEKS 17-20)

Reading, presenting and questioning data, the skill employers notice most when it is missing.

Month 5: Data, Averages and Probability (Weeks 17-20)
Weeks 17-18: Tables, Charts and Graphs
Topics:
- Reading and building tables and tally charts
- Bar charts, pie charts and line graphs
- Scatter diagrams and what they suggest at Level 2
- Choosing a suitable chart for the data
- Spotting a misleading chart
- Writing one clear sentence about what data shows

Projects built:
- Data report: a small real data set presented two ways with a written comparison

Practice: 10 data questions a week from real tables and charts
Weeks 19-20: Averages, Range and Probability
Topics:
- Mean and range, with median and mode at Level 2
- Comparing two sets of data with an average and the range
- Estimating the mean from grouped data at Level 2
- Probability as a fraction, decimal or percentage
- Likely and unlikely events in everyday contexts
- Combined outcomes at Level 2

Projects built:
- Comparison task: two shops, teams or months compared with averages and range

Practice: 10 averages and probability questions a week
Assessment: Month 5 check: a 45-minute data paper

PHASE 4: EXAM PRACTICE (MONTH 6, WEEKS 21-24)

Full practice assessments in the style of your exam centre's awarding organisation, with the non-calculator and calculator sections taken under real timing.

Month 6: Practice Assessments (Weeks 21-24)
Weeks 21-22: Underpinning Skills and Problem Solving
Topics:
- A non-calculator section under timing
- A calculator section of longer problems in context
- Marking against a published-style mark scheme
- Sorting lost marks: method, accuracy or reading the question
- Re-teaching the two weakest areas
- Showing working so method marks are earned

Projects built:
- Error log started from the first practice assessment

Practice: One full practice assessment a week
Weeks 23-24: Final Practice and Exam Readiness
Topics:
- A final full practice assessment under timing
- Review of every lost mark
- What to expect on the day, on screen or on paper
- Equipment and calculator checks
- Managing time across the paper
- Planning the next step: Level 2 or GCSE maths

Projects built:
- Completed error log and method sheet, kept for revision

Practice: Short targeted practice on the error log
Assessment: Final practice assessment, marked and reviewed one to one

PROJECTS THE STUDENT BUILDS AND KEEPS

- A personal method sheet for every written calculation
- A shopping comparison and a monthly budget
- A room flooring and paint plan with costs
- A journey plan from a real timetable
- A data report and a comparison task
- An error log built from practice assessments

Total projects: Six practical projects drawn from real life, plus a method sheet and error log for revision

SKILLS THE STUDENT WILL HAVE AT THE END

- Number, fractions, decimals, percentages and ratio with and without a calculator

- Money, budgeting and interest
- Measures, units, time and compound measures
- Area, perimeter, volume and scale
- Reading, presenting and comparing data
- Solving problems in context and showing the working

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, including evening options for working adults
Practice: 2 to 3 hours of practice a week, in short sessions
Review: Practice marked with comments; practice assessments marked against published-style schemes

SUPPORT INCLUDED

Doubt support: WhatsApp support between classes, with worked examples for stuck questions
Progress updates: A short written progress note each month

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders

WHERE THIS CAN LEAD

- Jobs, apprenticeships and college courses that ask for level 2 maths; check each one's exact requirement
- Our GCSE maths resit course for learners who want a GCSE as well
- Our statistics and data courses for learners who enjoyed the data phase

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Functional Skills maths?
A: A qualification that tests maths in real situations. The Department for Education sets the content at Entry Levels 1 to 3 and Levels 1 and 2, in three areas: using numbers and the number system, using common measures, shape and space, and handling information and data.

Q: Is Functional Skills maths Level 2 equivalent to a GCSE?
A: Both are level 2 qualifications, but they are different qualifications with different content and exams. Check the exact requirement with the organisation you are applying to.

Q: Can you use a calculator in Functional Skills maths?
A: For most of it. Ofqual's rules say 25% of the marks are for questions done without a calculator and 75% for questions where a calculator is allowed. This course practises the non-calculator section from the first week.

Q: How long does it take to pass Functional Skills maths Level 2?
A: It depends on where you start. This course is planned as about six months for a full level, and the starting check shows whether you need all of it. We cannot promise a result, but we make sure every part of the content and both exam sections are covered.

Q: Can I study Functional Skills maths online?
A: Yes, the teaching is fully online and live. The exam is taken through an approved exam centre, such as a college or training provider, which makes the entry. We teach the content and practise in the style of the awarding organisation you will sit with.

Q: What is the difference between Level 1 and Level 2?
A: The subject content describes problem solving as straightforward at Level 1 and complex at Level 2.

Q: I failed GCSE maths. Should I do Functional Skills instead?
A: It can be a good route, especially for work or an apprenticeship. If you are 16 to 18 on a study programme, your college follows the government's condition of funding rules, which link the choice to your GCSE grade. We also run a GCSE maths resit course, and the free demo can help you decide. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes available in the evening?
A: Yes. Most learners on this course work, so we run evening and weekend options, and one-to-one lessons can be booked at times that suit you.

Q: How are the classes run?
A: Live online classes with a real teacher, in a group of 15 to 20 adults at a similar level or one to one. Current fees for your region are shown in the pricing section of this page.

Q: Can I try a lesson first?
A: Yes. Book a free demo class. The teacher works through a few everyday maths problems with

you, sees where you are, and suggests Level 1 or Level 2 and group or one-to-one teaching. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/functional-skills-maths-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 64 OF 137: GAME DEVELOPMENT COURSE: UNITY, C# & UNREAL ENGINE
==========

Course link: https://learn.modernagecoders.com/courses/game-development-complete-masterclass-college
Subject area: Game Development
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Game development course for college students and professionals: Unity and C# from 2D to 3D with game AI, an Unreal and C++ phase, multiplayer, and a shipped capstone in 10-12 months.

In one line: Both engines the industry runs on: Unity and C# from 2D to 3D and game AI, then Unreal with C++, multiplayer, and a capstone you actually ship.

ABOUT THE PROGRAMME

Game development is a craft with two centers of gravity, Unity with C# and Unreal with C++, and serious studios expect you to be deep in one and conversant in the other. This live online masterclass builds both in the right order: game design and complete 2D games first, because finished small games teach more than unfinished big ones, then 3D in Unity, advanced C#, game AI and real 3D projects, plus mobile and cross-platform shipping. Then the second engine: Unreal fundamentals, C++ for games, graphics programming and engine architecture, followed by multiplayer, the hardest and most requested skill in modern game work. A finale month ships a polished capstone game and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- Finished games from month one: complete 2D games before 3D ambitions
- Both industry engines: Unity/C# depth plus a real Unreal/C++ phase, in the order studios expect
- Game AI and multiplayer taught properly, the two topics tutorials always skip
- Craft over hype: no VR/AR tourism, no live-service fantasies, just shippable skill
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Months 1-3): game design, C# and complete 2D games with art and audio
Phase 2: Unity mastery (Months 4-6): 3D, advanced C#, game AI, mobile and cross-platform
Phase 3: Unreal and multiplayer (Months 7-9): C++, graphics, engine architecture and networking
Phase 4: The finale (Month 10): a polished capstone game and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Everything is taught from the first line
Equipment: Any reasonably recent computer handles the Unity phases; the Unreal phase is smoother with a mid-range graphics card, and we tell you exactly what matters before it starts
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

College students: Students who want game development as a serious engineering path
Career switchers: Professionals moving into games with the two-engine profile
Programmers: Developers who want C# and C++ craft with something playable to show
Indie dreamers: Builders who want to actually finish and ship games, not collect prototypes
Portfolio builders: Graduates who need finished, playable work interviews can hold

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: GAME FUNDAMENTALS & 2D DEVELOPMENT (MONTHS 1-3, WEEKS 1-13)

Build solid foundations in game design theory, 2D game development, and Unity basics while creating your first playable games.

Months 1-2: Game Design & 2D Fundamentals (Weeks 1-7)
Introduction to Game Development
Topics:
- History of video games and industry overview
- Game development roles: programmer, artist, designer, producer
- Game genres and their characteristics
- Game development pipeline: concept to release
- Introduction to game engines vs frameworks
- Setting up development environment
- Unity installation and interface tour
- Version control with Git for games
- Game development terminology
- Analyzing successful games
- Game documentation: GDD (Game Design Document)
- Prototyping and iteration methodology
Projects built:
- Write your first Game Design Document
- Analyze 5 favorite games critically
- Create paper prototype of simple game
- Set up Unity and development environment
Practice: Play and analyze 20 games across different genres
Game Design Fundamentals
Topics:
- Core game mechanics vs meta mechanics
- Game loops: core, meta, and social
- Player motivation and psychology
- Flow theory and difficulty curves
- Reward systems and progression
- Game balance fundamentals
- Level design principles
- Tutorial design and onboarding
- Narrative in games
- Player types: Bartle's taxonomy
- Playtesting and feedback incorporation
Projects built:
- Design a complete game concept with mechanics
- Create difficulty curve for a game
- Design reward system for mobile game
- Build paper prototype and playtest it
Practice: Design 10 different game mechanics on paper
2D Game Development with Unity
Topics:
- Unity 2D workspace and tools
- Sprites and sprite sheets
- 2D physics and colliders
- Rigidbody2D and physics materials
- Animation basics: sprite animation
- Animator controller and state machines
- Tilemaps and tilemap colliders
- 2D lighting with URP
- Particle systems for 2D
- Camera setup for 2D games
- Sorting layers and order
- 2D audio and sound effects
Projects built:
- Create animated sprite character
- Build tilemap-based level
- Implement 2D platformer mechanics
- Add particle effects and polish
Practice: Create 10 different 2D game prototypes

```
  Programming Fundamentals with C#
    Topics:
      - C# basics: variables, data types, operators
      - Control flow: if-else, switch, loops
      - Functions and methods
      - Object-oriented programming in C#
      - Classes and objects for games
      - Inheritance and polymorphism
      - Unity-specific C# features
      - MonoBehaviour lifecycle
      - Coroutines and timing
      - Events and delegates
      - Debugging in Unity
      - Performance considerations
    Projects built:
      - Player movement controller
      - Enemy AI basic behavior
      - Inventory system
      - Score and UI management system
    Practice: Write 50 Unity scripts for different purposes

Month 3: Complete 2D Games (Weeks 8-13)
  Building Complete 2D Games
    Topics:
      - Game architecture patterns
      - Scene management and loading
      - Save/load systems
      - UI design with Unity UI
      - Menu systems and navigation
      - Input handling: keyboard, mouse, touch
      - Mobile input and virtual joysticks
      - Object pooling for performance
      - 2D pathfinding basics
      - Power-ups and collectibles
      - Health and damage systems
      - Game state management
    Projects built:
      - Complete 2D platformer game
      - Top-down shooter with enemies
      - Puzzle game with 10 levels
      - Mobile endless runner
    Practice: Build 5 different 2D game genres
  2D Art and Animation
    Topics:
      - Pixel art fundamentals
      - Color theory for games
      - Creating game sprites
      - Animation principles: squash, stretch, anticipation
      - Sprite sheet creation
      - 2D skeletal animation with Unity
      - Visual effects for 2D games
      - UI/UX design for games
      - Asset optimization
      - Free and paid asset resources
      - Art style consistency
      - Creating game atmosphere
    Projects built:
      - Create full sprite set for character
      - Design and animate enemy types
      - Create tileset for platformer
      - Design complete UI theme
    Practice: Create art assets for 5 games
  Audio and Polish
    Topics:
      - Game audio fundamentals
      - Sound effects creation and editing
      - Music theory for games
      - Unity audio system
      - 3D spatial audio
      - Audio mixing and mastering
      - Creating atmosphere with audio
      - Audio feedback for player actions
      - Performance optimization
      - Game juice and polish techniques
      - Screen shake and effects
      - Testing and debugging
    Projects built:
      - Add complete audio to previous games
      - Create dynamic music system
```

- Polish one game to release quality
- PHASE 1 CAPSTONE: Complete 2D Game with Original Art and Audio

Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 2: 3D DEVELOPMENT & ADVANCED UNITY (MONTHS 4-6, WEEKS 14-26)

Master 3D game development, advanced C# programming, and professional Unity techniques.

Months 4-5: 3D Game Development (Weeks 14-23)

3D Fundamentals in Unity

Topics:

- 3D coordinate systems and transforms
- 3D modeling basics and terminology
- Importing and configuring 3D models
- Materials and textures
- Shader basics and Shader Graph
- Lighting: directional, point, spot, area
- Lightmapping and global illumination
- Shadows and shadow settings
- 3D physics: Rigidbody and colliders
- Physics materials and joints
- Terrain creation and editing
- ProBuilder for level design

Projects built:

- 3D character controller
- Physics-based puzzle
- Terrain with environment
- Indoor level with lighting

Practice: Create 10 different 3D scenes

3D Animation and Characters

Topics:

- 3D animation principles
- Humanoid vs generic rigs
- Animation import and retargeting
- Mecanim animation system
- Blend trees and layers
- Animation events and curves
- Root motion
- Inverse Kinematics (IK)
- Ragdoll physics
- Facial animation basics
- Motion capture data usage
- Cinematics with Timeline

Projects built:

- Third-person character controller
- Combat system with animations
- Cutscene using Timeline
- Character customization system

Practice: Implement 15 different animation systems

Advanced C# for Games

Topics:

- Design patterns for games: Singleton, Observer, State
- Generic programming in Unity
- LINQ for game data
- Async/await in Unity
- Unity Job System basics
- Burst Compiler
- ECS (Entity Component System) introduction
- Custom editors and tools
- PropertyDrawers and Attributes
- ScriptableObjects for data
- Addressables for asset management
- Performance profiling and optimization

Projects built:

- Modular weapon system
- Quest system with ScriptableObjects
- Custom Unity tools
- Performance-optimized game

Practice: Implement 20 design patterns in games

AI for Games

Topics:

- Game AI fundamentals
- Finite State Machines (FSM)
- Behavior Trees introduction
- Navigation and pathfinding
- NavMesh and NavMesh agents
- A* pathfinding implementation

- Steering behaviors: seek, flee, wander
- Flocking and swarm AI
- Decision trees
- Goal-Oriented Action Planning (GOAP)
- Machine learning in games basics
- Enemy AI patterns

Projects built:
- Stealth game with AI guards
- RTS unit movement system
- Boss AI with multiple phases
- Companion AI system

Practice: Create 10 different AI behaviors

3D Game Projects

Topics:
- First-person shooter mechanics
- Third-person action games
- Racing game physics
- Survival game systems
- Inventory and crafting
- Dialogue systems
- Quest and mission systems
- Procedural generation basics
- Level streaming
- Optimization techniques
- Platform-specific considerations
- Beta testing process

Projects built:
- FPS prototype with AI enemies
- Racing game with 3 tracks
- Survival game vertical slice
- Action RPG with inventory

Practice: Build prototypes of 5 different 3D genres

Month 6: Mobile & Cross-Platform Games (Weeks 24-26)

Mobile Game Development

Topics:
- Mobile game market analysis
- Touch input and gestures
- Mobile UI/UX principles
- Screen resolution handling
- Mobile performance optimization
- Battery and thermal management
- Mobile-specific shaders
- Asset bundling for mobile
- In-app purchases implementation
- Ads integration: Unity Ads, AdMob
- Analytics integration
- Push notifications

Projects built:
- Hypercasual mobile game
- Mobile puzzle game with IAP
- Mobile multiplayer game
- Port PC game to mobile

Practice: Optimize 5 games for mobile

Cross-Platform Development

Topics:
- Platform-specific code management
- Input system abstraction
- Save data across platforms
- Cloud saves implementation
- Platform-specific features
- Console development basics
- Nintendo Switch considerations
- PlayStation requirements
- Xbox Live integration
- Steam integration
- Build automation
- Testing across platforms

Projects built:
- Cross-platform game with cloud saves
- Platform-specific features implementation
- Console-ready game prototype
- Multi-platform build pipeline

Practice: Deploy game to 3 different platforms

Phase 2 Capstone Project

Topics:
- Scope and project planning
- Vertical slice development
- Iterative design process

```
        - Playtesting and feedback
        - Polish and optimization
        - Marketing materials creation
      Projects built:
        - PHASE 2 CAPSTONE: Complete 3D Game
        - Develop full 3D game with 30 minutes of gameplay
        - Include: menus, save system, 5 levels, original assets
        - Target platform: PC and one other platform
      Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

------------------------------------------------------------
PHASE 3: UNREAL ENGINE & ADVANCED PROGRAMMING (MONTHS 7-9, WEEKS 27-39)
------------------------------------------------------------
Master Unreal Engine, C++ game programming, graphics programming, and advanced game systems.

  Months 7-8: Unreal Engine Mastery (Weeks 27-36)
    Unreal Engine Fundamentals
      Topics:
        - Unreal Engine philosophy vs Unity
        - UE5 interface and workflow
        - Blueprints visual scripting
        - Actor and component system
        - Level design in Unreal
        - Materials and material instances
        - Lighting with Lumen
        - Nanite virtualized geometry
        - World Partition system
        - Niagara particle system
        - Physics with Chaos
        - World building tools
      Projects built:
        - First Unreal project setup
        - Blueprint character controller
        - Environment with Nanite and Lumen
        - Particle effects showcase
      Practice: Create 10 Unreal Blueprint systems
    C++ Game Programming
      Topics:
        - C++ in Unreal Engine
        - Unreal coding standards
        - UCLASS, UPROPERTY, UFUNCTION
        - Actor lifecycle in C++
        - Components in C++
        - Gameplay framework classes
        - Input handling in C++
        - Collision and physics in code
        - Unreal's reflection system
        - Debugging C++ in Unreal
        - Blueprint and C++ interaction
        - Performance profiling
      Projects built:
        - C++ character class
        - Weapon system in C++
        - AI controller in C++
        - Custom game mode
      Practice: Convert 10 Blueprint systems to C++
    Advanced Unreal Systems
      Topics:
        - Animation blueprints and state machines
        - Control Rig and IK
        - Procedural animation
        - Behavior trees in Unreal
        - Environmental Query System (EQS)
        - Smart objects
        - Level streaming and world composition
        - Sequencer for cinematics
        - Audio with MetaSounds
        - UI with UMG and Slate
        - Common UI framework
        - Gameplay Ability System (GAS)
      Projects built:
        - Complex animation system
        - Advanced AI with behavior trees
        - Cinematic sequence
        - Complete UI system
      Practice: Master 10 advanced Unreal systems
    Graphics Programming
      Topics:
```

- Rendering pipeline overview
- Shader programming basics
- HLSL for Unreal
- Custom materials and shaders
- Post-processing effects
- Render targets
- Compute shaders
- Ray tracing basics
- Global illumination techniques
- LOD systems
- Texture streaming
- GPU profiling

Projects built:
- Custom shader effects
- Post-processing pipeline
- Procedural materials
- Performance optimization study

Practice: Create 15 custom shaders

Engine Architecture

Topics:
- Game engine architecture overview
- Memory management in games
- Object pooling patterns
- Resource management
- Threading in game engines
- Job systems
- Entity Component Systems (ECS)
- Custom engine modules
- Plugin development
- Engine modification
- Source control for large projects
- Build systems and CI/CD

Projects built:
- Custom Unreal plugin
- Memory optimization system
- Multithreaded game system
- Engine feature extension

Practice: Understand and modify engine code

Month 9: Multiplayer & Networking (Weeks 37-39)

Multiplayer Fundamentals

Topics:
- Networking basics for games
- Client-server architecture
- Peer-to-peer vs dedicated servers
- Unity Netcode for GameObjects
- Mirror networking
- Photon PUN2
- Network synchronization
- Lag compensation
- Client prediction
- Server reconciliation
- Interest management
- Cheat prevention

Projects built:
- LAN multiplayer game
- Online PvP arena
- Co-op puzzle game
- Battle royale prototype

Practice: Build 5 different multiplayer games

Advanced Multiplayer Systems

Topics:
- Unreal replication system
- RPCs and ownership
- State synchronization
- Deterministic physics
- Rollback netcode
- Server authoritative systems
- Matchmaking systems
- Lobby implementation
- Voice chat integration
- Dedicated server setup
- Cloud hosting (AWS, Azure)
- Load balancing

Projects built:
- Competitive multiplayer FPS
- MMO prototype with 100 players
- Dedicated server deployment
- Matchmaking system

Practice: Deploy 3 games with dedicated servers
Phase 3 Capstone Project
Topics:
- Large project management
- Team collaboration tools
- Asset pipeline
- Performance targets
- Platform certification requirements
- Submission preparation
Projects built:
- PHASE 3 CAPSTONE: AAA-Quality Game Vertical Slice
- Create 15-minute vertical slice of AAA-quality game
- Use Unreal Engine with C++
- Include multiplayer component
- Target next-gen quality
Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---
PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)

The capstone month: spec, build, ship and defend, then the final exam that decides the certificate.

Month 10: Capstone and the Final Exam (Weeks 40-44)
Capstone Spec and Setup
Topics:
- Choosing the capstone: a complete polished game in Unity or Unreal: a 2D gem, a 3D experience or a small multiplayer arena
- A one-page spec: scope, milestones, stack
- Repository and skeleton ready on day one
- Instructor sign-off before the build begins
Projects built:
- Approved capstone spec plus the running skeleton
Practice: Pitch the spec to the batch and absorb one hard question
Capstone Build
Topics:
- Two focused weeks in vertical slices
- Instructor checkpoints and honest scope cuts
- Using the whole course: engine depth, C# or C++ craft, game AI or networking, and the polish pass that makes it feel finished
- Daily commits and an always-working build
Projects built:
- The capstone, feature-complete with version control
Practice: A working build at the end of every session
Ship and Polish
Topics:
- The finishing pass: edge cases, errors, documentation
- Playtested and packaged: builds that run on machines that are not yours
- A README worth reading and a demo worth watching
- A full-course spiral review before the exam
Projects built:
- The capstone shipped, documented and rehearsed
Practice: Two mock demos, each tighter than the last
Demo Day and the Final Exam
Topics:
- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: gameplay engineering, engine and graphics specialization, or indie shipping
- Where our other courses continue the journey
Projects built:
- Demo day presentation delivered to the batch
Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: complete 2D games with art, audio and polish
- Phase 2: 3D Unity projects with game AI and a mobile build
- Phase 3: Unreal/C++ builds and a working multiplayer prototype
- Phase 4: the capstone game, playtested, packaged and defended

Total projects: 15+ finished games and prototypes, crowned by a polished capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Game design fundamentals: loops, feel and scope discipline
- Unity and C# from complete 2D games to 3D with game AI

- Unreal Engine with C++: fundamentals, graphics and engine architecture
- Multiplayer: networking fundamentals through a working online prototype
- The polish craft: art integration, audio, playtesting and packaging

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Gameplay engineering roles in Unity or Unreal studios
- Engine, graphics and tools specializations for the systems-minded
- Indie shipping: the capstone is your first release candidate
- Our C++ course deepens the systems half of the Unreal road

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Growing roles: Rs 4-8 LPA (Junior Game Developer)
Experienced roles: Rs 8-15 LPA (Game Developer); Rs 12-25 LPA (Senior Game Developer); Rs 20-40+ LPA (Lead Developer/Technical Director)
Freelance: Rs 2,000-10,000/hour based on expertise
International: $60k-150k USD (higher in AAA studios)

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this game development course cover?
A: Game design and complete 2D games in Unity with C# first, then 3D development, advanced C#, game AI, mobile and cross-platform shipping. Then the second engine: Unreal fundamentals, C++ game programming, graphics programming and engine architecture, followed by multiplayer networking through a working online prototype. A finale month ships a polished capstone game and sits the final exam.

Q: How long does it take to learn game development properly?
A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two more months in hand for learners who need them. The three teaching phases build the skill and its real-world application; the finale month ships a capstone and sits the final exam.

Q: How am I assessed? Is the certificate just for attending?
A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Unity or Unreal: which should I learn?
A: Both, in the order this course teaches them. Unity with C# is the gentler engine to learn deeply and dominates mobile and indie work; Unreal with C++ carries AAA and high-end 3D. Studios increasingly expect depth in one and literacy in the other, which is exactly the profile this course builds: Unity depth first, then a real Unreal phase, and you graduate knowing which one fits each project.

Q: Do I need a powerful computer?
A: Not for most of the course: the 2D and 3D Unity phases run happily on ordinary laptops. The Unreal phase is where hardware starts to matter, a mid-range graphics card makes it comfortable, and we tell you the honest requirements before that phase begins so nothing surprises you. No phase requires a high-end machine.

Q: Can I try a class before enrolling?

A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/game-development-complete-masterclass-college
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 65 OF 137: GAME DEVELOPMENT FOR KIDS: SCRATCH, ROBLOX AND MINECRAFT
==========

Course link: https://learn.modernagecoders.com/courses/game-development-masterclass-for-kids
Subject area: Kids Coding
Designed for: Kids
Who can join: Complete beginner, ages 8 to 12
Course length: 6-8 months (26-32 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Game development course for kids aged 8-12: live online classes building real games in Scratch, then Roblox and Minecraft worlds, in 6-8 months.

In one line: From playing games to making them: real Scratch games first, then Roblox and Minecraft worlds, finished and shown off in six honest months.

ABOUT THE PROGRAMME

Every child who loves games has the same secret wish: to make one. This live online course grants it properly for ages 8 to 12: game logic learned in Scratch, where scores, lives, levels and enemy behavior are built by the child, then complete 2D games polished until friends ask to play again, and finally the leap into 3D worlds with Roblox Studio and coded Minecraft builds, plus first multiplayer fun. Finishing is the discipline: every month ends with a playable build and a mixed challenge that quietly revises earlier skills, and the course ends with a capstone game, planned, built, playtested and presented at demo day. There is no monetization talk, no in-game purchase strategy and no chatbot dependence in this course; children this age should be building skill and joy, and that is the whole syllabus. The pace is honest and kid-sized: six months at two live classes a week plus two to three hours of playful practice, with two more months in hand when needed. A final exam, capstone demo plus a friendly viva, decides the certificate, with a free retest after revision. Real games, real pride, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Finished beats fancy: every month ends with a playable build a child can demo at dinner
- The kid-favorite trio done properly: Scratch mastery first, then Roblox Studio and coded Minecraft
- Game design taste taught young: fairness, difficulty and fun, judged out loud kindly
- No monetization talk, no purchase mechanics, no chatbot accounts: skill and joy only
- An honest 6-month arc with 2 months in hand: nobody is rushed
- Real assessment, gently done: monthly mixed challenges and a final capstone exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Game logic in Scratch (Months 1-2): mechanics, scoring, enemies and complete first games
Phase 2: 2D mastery (Months 3-4): polish, levels, game feel and bigger builds
Phase 3: 3D worlds and the finale (Months 5-6): Roblox Studio, coded Minecraft, first multiplayer, and the capstone exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Game logic is taught from the first block
Reading: Comfortable reading simple instructions
Equipment: A computer or laptop with a stable internet connection; Scratch, Roblox Studio and Minecraft coding tools are free to build with (accounts set up with a parent)
Age: 8 to 12. Batches are grouped by age
Time commitment: 2 live classes plus 2-3 hours of playful practice a week

WHO THIS COURSE IS FOR

Game lovers: Children who play games and have started saying "I could make this better"
Complete beginners: Kids with zero coding background: Scratch makes the start joyful
Roblox minecraft fans: Children who live in those worlds and are ready to build them instead
Future creators: Kids who will grow into our dedicated Roblox, Minecraft and Python courses
Finishers in training: Children who start many things; this course teaches the pride of finishing

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: GAME LOGIC IN SCRATCH (MONTHS 1-2, WEEKS 1-8)

The mechanics under every game, built by the child: movement, scoring, enemies, and complete first games that work.

Months 1-2: Scratch Game Mastery (Weeks 1-8)

Introduction to Game Development

Topics:
- What makes a game fun?
- Types of games: arcade, puzzle, adventure, RPG
- Game development process overview
- Introduction to Scratch for games
- Scratch interface for game makers
- Sprites as game characters
- Backgrounds and game worlds
- Basic movement: arrow keys and WASD
- Collision detection basics
- Game coordinates (X and Y)
- Your first playable game
- Sharing games with friends

Projects built:
- Catch the falling objects game
- Simple maze game
- Pet virtual game
- Two-player tag game

Practice: Daily: Play and analyze 2 games, create 1 game mechanic

Game Mechanics & Player Control

Topics:
- Player movement patterns
- Jump mechanics and gravity
- Speed and acceleration
- Power-ups and collectibles
- Enemy movement and AI basics
- Health and lives system
- Score and points tracking
- Timer and countdown mechanics
- Game over and restart
- Win conditions
- Difficulty levels
- Checkpoints and saving progress

Projects built:
- Platformer with jumping
- Racing game with timer
- Shooting game with score
- Puzzle with levels

Practice: Create 10 different game mechanics
Assessment: Month 1 check: build a small game mechanic live, plus a fun take-home challenge

Advanced Scratch Game Development

Topics:
- Scrolling backgrounds
- Multiple levels design
- Boss battles creation
- Inventory systems
- Dialog and storytelling
- Cut scenes and intros
- Sound effects and music
- Particle effects with clones

- Advanced animations
- Cloud variables for high scores
- Multiplayer with cloud data
- Publishing and sharing games

Projects built:
- Multi-level adventure game
- RPG with inventory
- Battle arena game
- Story-driven game

Practice: Complete 5 advanced Scratch games

Introduction to GDevelop

Topics:
- GDevelop interface tour
- Objects and behaviors
- Events system in GDevelop
- Physics engine basics
- Animations and sprites
- Sound and music integration
- Touch controls for mobile
- Scene management
- Variables and game logic
- Exporting games
- HTML5 game publishing
- Mobile game basics

Projects built:
- Physics puzzle game
- Endless runner
- Tower defense game
- Mobile tap game

Practice: Create 5 games in GDevelop

Assessment: Month 2 check: a complete mini-game demoed live, with challenge questions reaching back to month 1

PHASE 2: 2D GAME MASTERY (MONTHS 3-4, WEEKS 9-17)

From working to wonderful: levels, polish, game feel and bigger 2D builds that friends genuinely want to play.

Months 3-4: Bigger, Better 2D Games (Weeks 9-17)

Construct 3 Game Engine

Topics:
- Construct 3 interface
- Behaviors and effects
- Event sheets mastery
- Layouts and layers
- Sprite animations
- Tilemaps for levels
- Pathfinding for enemies
- Line of sight behavior
- Platform behavior advanced
- Mobile optimization
- Touch and gesture controls
- Debugging tools

Projects built:
- Zelda-style adventure
- Angry Birds clone
- Match-3 puzzle game
- Stealth game

Practice: Build 8 different game genres

Game Art & Animation

Topics:
- Pixel art basics
- Character design principles
- Animation fundamentals
- Background creation
- Tile sets design
- UI elements design
- Color theory for games
- Free art resources
- Piskel for pixel art
- Sound effect creation
- Music for games
- Asset optimization

Projects built:
- Design game character set
- Create animated sprites
- Design full game tileset
- Complete game art package

```
      Practice: Create art for 5 games
    Mobile Game Development
      Topics:
        - Mobile game design principles
        - Touch control optimization
        - Screen size adaptation
        - Performance optimization
        - Accelerometer controls
        - Swipe and gesture mechanics
        - Mobile UI/UX design
        - Ad integration concepts
        - App store preparation
        - Testing on devices
        - Analytics basics
      Projects built:
        - Mobile endless runner
        - Puzzle game for phones
        - Casual tap game
        - Mobile multiplayer game
      Practice: Optimize 5 games for mobile
    GameMaker Studio Basics
      Topics:
        - GameMaker interface
        - GML Visual scripting
        - Rooms and objects
        - Sprites and animations
        - Collision systems
        - Camera and views
        - Particle systems
        - Sound management
        - Save systems
        - Menu creation
        - Polish and juice
        - Publishing options
      Projects built:
        - Retro arcade game
        - Metroidvania demo
        - Bullet hell shooter
        - Roguelike dungeon
      Practice: Master 10 GameMaker features
    Phase Review and Showcase
      Topics:
        - Portfolio preparation
        - Game trailers creation
        - itch.io publishing
        - Game documentation
        - Player feedback integration
        - Bug fixing strategies
        - Game updates and patches
      Projects built:
        - Polish best 2D game
        - Create game trailer
        - Publish to itch.io
        - Get player feedback
      Assessment: Phase check: showcase a polished 2D game, plus a mixed challenge spanning
      months 1-4

----------------------------------------------------------------
PHASE 3: 3D WORLDS AND THE FINALE (MONTHS 5-6, WEEKS 18-26)
----------------------------------------------------------------
The leap kids dream about: Roblox Studio worlds, coded Minecraft builds, first multiplayer
fun, and the capstone presented at demo day.

  Months 5-6: Roblox, Minecraft and the Capstone (Weeks 18-26)
    Roblox Studio Fundamentals
      Topics:
        - Roblox Studio interface
        - Parts and models
        - Terrain editor
        - Basic scripting with blocks
        - GUI creation
        - Character customization
        - Tools and weapons
        - Game passes creation
        - Team create feature
        - Publishing to Roblox
        - Roblox community
      Projects built:
        - Obby (obstacle course)
```

```
    - Tycoon game
    - Racing game
    - Battle royale map
  Practice: Create 5 Roblox experiences
Advanced Roblox Development
  Topics:
    - Advanced building techniques
    - Scripting game mechanics
    - Datastores for saving
    - Leaderboards and stats
    - Custom animations
    - Particle effects
    - Lighting and atmosphere
    - Sound regions
    - NPCs and AI
    - Vehicle systems
    - Trading systems
    - Group games
  Projects built:
    - RPG with quests
    - Simulator game
    - Horror experience
    - Social hangout game
  Practice: Master 15 Roblox features
Minecraft Modding & Creation
  Topics:
    - Minecraft Education Edition
    - Command blocks mastery
    - Redstone mechanics
    - Custom maps creation
    - Adventure mode design
    - Resource packs basics
    - Function files
    - Custom structures
    - Minecraft marketplace
    - Server creation basics
    - Minigames design
    - Parkour map creation
  Projects built:
    - Adventure map with story
    - Minigame collection
    - Parkour challenge map
    - Survival challenge map
  Practice: Build 5 Minecraft experiences
Unity with Visual Scripting
  Topics:
    - Unity Hub and installation
    - Unity interface basics
    - GameObject and components
    - Visual scripting with Bolt
    - Basic 3D movement
    - Physics and collisions
    - Lighting basics
    - Terrain creation
    - Asset store usage
    - UI in Unity
    - Build settings
    - WebGL publishing
  Projects built:
    - 3D platformer
    - First-person explorer
    - Ball rolling game
    - Simple VR experience
  Practice: Complete 3 Unity projects
Capstone and the Final Exam
  Topics:
    - Choosing final project
    - Game design document
    - Production planning
    - Asset creation pipeline
    - Testing and debugging
    - Publishing preparation
    - Marketing basics
    - Community building
  Projects built:
    - THE CAPSTONE: an original game in the child's chosen world, Scratch, Roblox or Minecraft,
      playtested and presented at demo day
  Assessment: FINAL EXAM: capstone demo plus a friendly viva and a mixed quiz from the
  whole course. Passing earns the certificate; a child who falls short gets focused
```

revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Months 1-2: complete first Scratch games with scores, lives and enemies
- Months 3-4: polished 2D games with levels, difficulty curves and juice
- Months 5-6: a Roblox world, a coded Minecraft build, first multiplayer fun, and the capstone at demo day

Total projects: 15+ playable games and worlds, crowned by a capstone presented at demo day

SKILLS THE STUDENT WILL HAVE AT THE END

- Game logic: movement, collision, scoring, lives, levels and enemy behavior
- Scratch mastery: complete, polished games built independently
- World building: Roblox Studio basics and coded Minecraft creations
- Design taste: fairness, difficulty and fun, judged and fixed
- Finishing: planning, building, playtesting and presenting

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building along with the teacher
Homework: A small game-build after every class, and a mixed challenge at the end of every month
Practice: 2-3 hours of playful practice between classes
Review: Every build reviewed with teacher feedback; wobbly skills reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so no child stays stuck and discouraged
Progress updates: Regular progress notes to parents: what was built, what comes next, and how to encourage practice

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam, a capstone demo plus a friendly viva. A child who falls short gets focused revision and a free retest: the certificate certifies skills, not attendance

WHERE THIS CAN LEAD

- Roblox Game Coding: deeper Lua scripting for the Roblox-obsessed
- Minecraft Coding: more automation and invention inside Minecraft
- Python for Kids: the typed-code step, entered with strong logic
- Kids Coding journey: the full block-to-code path if they want everything
- Family fame: the household now has a game developer

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What games will my child actually make?
A: Fifteen-plus playable builds across six months: complete Scratch games with scores, lives and enemies in the first two months, polished 2D games with levels and difficulty curves by month four, then a Roblox Studio world, a coded Minecraft build and first multiplayer fun, ending with a capstone game in the child's chosen world, playtested and presented at demo day.

Q: What age is right for this course?
A: Ages 8 to 12, with batches grouped by age. Eight-year-olds thrive in the Scratch months and grow into the 3D tools; eleven and twelve-year-olds move faster and go deeper in Roblox and Minecraft. No coding background is needed.

Q: How long does it take a child to learn game development?
A: Six months at 2 live classes a week plus 2 to 3 hours of playful practice takes a child from first mechanic to a demo-day capstone. The course keeps two more months in hand for children who need more repetition, no rushing and no penalty. The first working game exists within the first month.

Q: How is my child assessed? Is the certificate just for attending?
A: No, and it is all gentle: a small build after every class, a mixed challenge at the end of every month that revisits earlier skills, and a final exam at the end, a capstone demo plus a friendly viva and short quiz. Passing earns the certificate; a child who falls short gets focused revision and a free retest.

Q: Does this course talk about making money from games?
A: No, deliberately. There is no monetization talk, no in-game purchase mechanics and no marketing lessons for children; those topics do not belong at 8 to 12. The course is about

skill and joy: building games worth playing and finishing what you start. Sharing happens safely, with parent-managed accounts.

Q: Are Roblox and Minecraft really educational?
A: As building tools, genuinely yes: Roblox Studio teaches world design and first scripting, and coded Minecraft teaches logic and automation, both on platforms children already love, which makes the motivation free. The course uses them as classrooms, with accounts set up alongside a parent and sharing kept safe.

Q: Is any paid software needed?
A: No. Scratch is free, Roblox Studio is free to build with, and the Minecraft coding tools used in class are free. Any reasonably recent computer or laptop is enough.

Q: Is there a free trial class?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Your child will build a tiny game mechanic inside the demo itself. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real teacher, never pre-recorded videos. Children build during class and get unstuck on the spot, which is exactly what keeps game-making joyful instead of frustrating.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/game-development-masterclass-for-kids
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 66 OF 137: GAME DEVELOPMENT FOR TEENS: UNITY, C# AND REAL SHIPPED GAMES
==========

Course link: https://learn.modernagecoders.com/courses/complete-game-development-masterclass-for-teens
Subject area: Game Development
Designed for: Teens
Who can join: Absolute beginner, ages 13 to 18
Course length: 10-12 months (40-48 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Game development course for teens aged 13-18: build and publish real 2D and 3D Unity games with C#, multiplayer basics, Blender and game design, in 10-12 months.

In one line: From playing games to shipping them: real Unity games, published where friends can actually play them, built in ten honest months.

ABOUT THE PROGRAMME

Every teenager who loves games has had the thought: I could make one of these. This live online course turns that thought into shipped games. Students aged 13 to 18 start from zero and learn game development the way it is genuinely practiced: C# and Unity, the engine behind Hollow Knight, Among Us and thousands of indie hits, with Blender for 3D assets and real game design theory underneath it all. The arc is built around finishing things. The first months produce complete 2D games, movement that feels good, enemies with real AI, menus, sound, save systems, and the phase ends with a finished game published on itch.io. The middle months move into 3D: character controllers, environments, animation with Mixamo, Blender modeling, shaders, procedural generation and optimization, ending in a complete 3D game. The later months add what makes games feel modern: multiplayer basics, mobile builds, an honest week inside Unreal Engine to understand the other side of the industry, game narrative, and accessibility, designing games more people can play. The final month is pure production: a capstone game, playtested, polished and published, presented at demo day. The pace is honest: two live classes a week plus three to four hours of practice fits this in ten months, and students who need longer take up to twelve, the class adapts. Learning is checked the honest way: homework after every class, monthly assignments mixing new questions with earlier topics, and a final exam that decides the certificate. Real knowledge, real work, real projects, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Finished beats fancy: students publish complete games on itch.io from phase 1, not endless unfinished prototypes
- One engine mastered: Unity and C# learned deeply, with an honest Unreal taste week instead of a shallow two-engine tour
- Real game design theory: flow, pacing, difficulty and ethical design, including why some games respect players and some exploit them
- Blender is in the syllabus: students model, texture and import their own 3D assets
- Accessibility and narrative get real weeks: the parts of game craft most courses skip
- Real assessment: weekly homework, mixed-review assignments every month, and a final exam that decides the certificate, with focused revision and a free retest for students who fall short

THE LEARNING PATH, STEP BY STEP

Phase 1: Complete 2D games (Months 1-3): C# through games, Unity 2D, physics, AI, UI, audio, and a published capstone
Phase 2: 3D worlds (Months 4-6): controllers, animation, environments, Blender, shaders, procedural generation and a finished 3D game
Phase 3: Bigger systems (Months 7-8): multiplayer basics, mobile builds, an Unreal taste, AI in games, narrative and accessibility
Phase 4: Production (Months 9-10): pipeline, polish, publishing, and the final capstone shipped and presented

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. C# is taught from the first variable
Computer skills: Comfortable installing software and managing files
Equipment: A computer with a stable internet connection. Unity runs on modest hardware; a dedicated graphics card helps for 3D but is not required
Age: 13 to 18. Batches are grouped by age and pace so the room always fits
Time commitment: 2 live classes plus 3-4 hours of practice a week

WHO THIS COURSE IS FOR

Players becoming makers: Teens who analyze the games they play and itch to build their own
Complete beginners: Students starting from zero: C# and Unity are taught from the first line
Finishers: Teens tired of tutorial graveyards who want complete, published games
Future engineers: Students who want serious programming skill wrapped in the most motivating subject there is
Creative all rounders: Teens who want code, art, design and storytelling in one craft

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: COMPLETE 2D GAMES (MONTHS 1-3, WEEKS 1-12)

C# learned through games, Unity 2D mastered properly, and the phase ends the only way that matters: a finished game, published.

Month 1: Foundations Through Games (Weeks 1-4)
Welcome to Game Development
Topics:
- History of games: From Pong to Fortnite
- How games are made: The development pipeline
- Game genres: What makes each unique
- Setting up your game development environment
- Installing Unity Hub and Visual Studio
- Introduction to game engines
- Your first game: Pong in 1 hour
- Understanding game loops
- Frames, updates, and time in games
- Version control with Git for games
- Creating your game developer portfolio
Projects built:
- Pong remake
- Breakout/Arkanoid clone
- Simple clicker game
- Text adventure game
Practice: Play and analyze 3 games: write down what makes each one fun
C# for Games
Topics:
- Why C# for game development
- Variables: Storing player health, score, lives
- Data types: int, float, bool, string for games
- Game math: Vectors, positions, distances
- Functions: Creating reusable game actions

- Classes: Building game objects
- If statements: Game logic and decisions
- Loops: Spawning enemies, updating objects
- Arrays: Managing multiple enemies/items
- Lists: Dynamic game collections
- Debugging games with console logs
- Common game programming patterns

Projects built:
- Number guessing game
- Rock Paper Scissors with AI
- Simple RPG battle system
- Dice rolling game

Practice: Complete 25 C# coding challenges

Unity 2D Fundamentals

Topics:
- Unity interface mastery
- GameObjects and components
- Transform: Position, rotation, scale
- Sprites and sprite renderers
- 2D physics: Rigidbody2D and colliders
- Input system: Keyboard and mouse
- Prefabs: Reusable game objects
- Tags and layers
- Scene management
- Unity's coordinate system
- Game camera setup
- Building for different platforms

Projects built:
- Top-down shooter
- Endless runner like Temple Run
- 2D platformer basics
- Asteroids remake

Practice: Build 3 mini-games in Unity

Game Physics and Collision

Topics:
- Physics in games: Gravity, forces, velocity
- Rigidbody2D properties
- Colliders: Box, circle, polygon, edge
- Trigger vs collision detection
- OnCollisionEnter2D and OnTriggerEnter2D
- Physics materials: Bounce and friction
- Joints: Hinge, spring, fixed
- Effectors: Area, platform, point
- Raycasting for line of sight
- Optimizing physics performance
- Creating realistic movement
- Physics-based puzzles

Projects built:
- Angry Birds clone
- Pinball game
- Physics puzzle game
- Rope swing mechanics

Practice: Create 3 physics-based game mechanics

Assessment: Month 1 check: build a small physics mechanic live, plus a take-home assignment mixing everything from month 1

Month 2: Mechanics That Feel Good (Weeks 5-8)

Player Movement and Game Feel

Topics:
- Character controllers for different genres
- Platformer movement: Jump, double jump, wall jump
- Top-down movement: 8-directional, tank controls
- Side-scroller mechanics
- Mobile touch controls
- Controller/gamepad support
- Input buffering and coyote time
- Movement feel: Acceleration, deceleration
- Camera follow systems
- Parallax scrolling backgrounds
- Screen shake and juice
- Polish that makes movement fun

Projects built:
- Mario-style platformer
- Twin-stick shooter
- Racing game controls
- Fighting game inputs

Practice: Implement 5 different movement systems and tune each until it feels right

Enemy AI and Behaviors

Topics:

- Basic enemy AI patterns
- Patrol and chase behaviors
- State machines for AI
- Pathfinding basics: A* algorithm
- Line of sight detection
- Shooting and projectiles
- Enemy spawn systems
- Difficulty scaling
- Boss fight patterns
- Crowd AI and flocking
- AI debugging and visualization
- Balancing AI difficulty

Projects built:
- Pac-Man with ghost AI
- Tower defense game
- Stealth game with guards
- Boss battle system

Practice: Create 8 different enemy types

Game UI and Menus

Topics:
- Unity UI system (Canvas)
- UI elements: Buttons, text, images, sliders
- Anchoring and responsive UI
- Health bars and HUD elements
- Score and timer displays
- Main menu design
- Pause menu implementation
- Settings and options
- Inventory systems
- Dialog systems
- UI animations and transitions
- Mobile-friendly UI

Projects built:
- RPG inventory system
- Crafting interface
- Dialog tree system
- Complete menu system

Practice: Build a complete menu flow: title, pause, settings, game over

Animation and Visual Effects

Topics:
- Sprite animation basics
- Unity Animator and state machines
- Animation transitions and parameters
- Blend trees for smooth animation
- Sprite sheets and texture atlases
- Particle systems for effects
- Trail renderers
- Screen effects and post-processing
- Lighting in 2D games
- Shader basics for 2D
- Creating game juice
- Performance optimization

Projects built:
- Fully animated character
- Explosion and magic effects
- Weather system
- Animated cutscene

Practice: Add animation and effects polish to two earlier games

Assessment: Month 2 check: build an animated, juicy mechanic live, with quiz questions reaching back to month 1

Month 3: Finish and Ship (Weeks 9-12)

Audio and Music in Games

Topics:
- Importance of game audio
- Unity's audio system
- Playing sounds and music
- 3D spatial audio
- Audio mixers and groups
- Dynamic music systems
- Sound effect creation basics
- Free audio resources
- Audio optimization
- Creating atmosphere with sound
- Audio feedback for actions
- Music synchronization

Projects built:
- Rhythm game prototype
- Dynamic soundtrack system

```
        - 3D audio showcase
        - Complete game with audio
      Practice: Add sound and music to every game you have built
    Level Design
      Topics:
        - Principles of level design
        - Teaching through level design
        - Difficulty curves and pacing
        - Environmental storytelling
        - Tilemap system in Unity
        - Procedural generation basics
        - Level editor tools
        - Playtesting and iteration
        - Secrets and collectibles
        - Multiple paths and choices
        - Boss arenas and set pieces
        - Level design documentation
      Projects built:
        - 10-level platformer
        - Procedural dungeon generator
        - Puzzle game levels
        - Open world map
      Practice: Design 10 unique levels with a deliberate difficulty curve
    Save Systems and Progress
      Topics:
        - PlayerPrefs for simple data
        - JSON serialization
        - Binary save files
        - Profile and progress systems
        - Unlockables and achievements
        - Leaderboards implementation
        - Backup and recovery
      Projects built:
        - Complete save system
        - Achievement system
        - Local high-score table
      Practice: Add saving and unlockables to your best game
    Phase 1 Capstone: Publish Your Game
      Topics:
        - Planning a complete game
        - Scope management
        - Creating a development timeline
        - Asset lists and requirements
        - Playtesting plan
        - Publishing checklist
      Projects built:
        - CAPSTONE: a complete 2D game with levels, save system and polish
        - Published on itch.io with a proper game page
        - Playtested by classmates and family, with fixes shipped
      Practice: Send the link to five people and act on the best piece of feedback
      Assessment: Phase 1 exam: present the published game and rebuild a small mechanic live,
      with mixed questions from months 1-3

---------------------------------------------------------------
PHASE 2: 3D WORLDS (MONTHS 4-6, WEEKS 13-24)
---------------------------------------------------------------
The jump to 3D done properly: controllers, animation, environments, your own Blender assets,
shaders, procedural generation, and a finished 3D game.

  Month 4: 3D Fundamentals (Weeks 13-16)
    3D Basics in Unity
      Topics:
        - 3D vs 2D: Understanding the difference
        - 3D coordinate systems and space
        - 3D primitives: Cubes, spheres, capsules
        - Materials and textures
        - Basic shaders and rendering
        - 3D transforms and rotation
        - Cameras in 3D: Perspective vs orthographic
        - Lighting basics: Directional, point, spot
        - Shadows and quality settings
        - 3D physics: Rigidbody and colliders
        - Importing 3D models
        - ProBuilder for level design
      Projects built:
        - 3D obstacle course
        - Bowling game
        - Simple racing game
        - 3D puzzle game
```

```
    Practice: Create 5 small 3D prototypes
  3D Character Controllers
    Topics:
      - First-person controller setup
      - Third-person controller with camera
      - Character movement and physics
      - Jumping and gravity
      - Slopes and stairs
      - Moving platforms
      - Swimming mechanics
      - Flying and gliding
      - Vehicle controllers
      - Camera collision handling
      - Input smoothing
      - Controller feel and polish
    Projects built:
      - FPS controller like Valorant
      - Third-person like Fortnite
      - Car racing mechanics
      - Flight simulator controls
    Practice: Build first-person and third-person controllers and tune both
  3D Animation with Mixamo
    Topics:
      - 3D animation principles
      - Humanoid vs generic rigs
      - Importing animations from Mixamo
      - Animation state machines
      - Blend trees for movement
      - Root motion
      - IK (Inverse Kinematics) basics
      - Animation events
      - Cutscene creation with Timeline
      - Performance optimization
    Projects built:
      - Animated player character
      - Combat animation system
      - Cinematic cutscene
      - Dance game with mocap
    Practice: Fully animate one character: idle, run, jump, attack
  3D Environments and Terrain
    Topics:
      - 3D level design principles
      - Terrain system in Unity
      - Terrain painting and sculpting
      - Vegetation and trees
      - LOD (Level of Detail) systems
      - Occlusion culling
      - Light baking and probes
      - Skyboxes and atmosphere
      - Weather effects
      - Day/night cycles
      - Interior vs exterior design
      - Performance optimization
    Projects built:
      - Open world environment
      - Dungeon crawler levels
      - Racing track design
      - Atmospheric horror level
    Practice: Design 2 contrasting 3D environments
    Assessment: Month 4 check: build a small 3D scene live, plus a mixed assignment
    reaching back to phase 1

Month 5: The Craft Around the Code (Weeks 17-20)
  Blender: Your Own 3D Assets
    Topics:
      - Introduction to Blender
      - 3D modeling fundamentals
      - Box modeling techniques
      - Sculpting basics
      - UV unwrapping introduction
      - Texturing in Blender
      - Low-poly modeling for games
      - Optimization for game engines
      - Exporting to Unity
      - Creating game props
      - Character modeling basics
      - Animation in Blender
    Projects built:
      - Low-poly character model
```

- Game props collection
- Simple vehicle model
- Environment assets

Practice: Model 10 game props and import them into Unity

Game Design Theory

Topics:

- What makes games fun: Flow theory
- Core gameplay loops
- Game mechanics vs dynamics vs aesthetics
- Player motivation and rewards
- Progression systems
- Economy and balance
- Game design documentation (GDD)
- Prototyping and iteration
- Player psychology
- Addiction and ethical design
- Analyzing successful games

Projects built:

- Game design document for your dream game
- Progression system design
- A written analysis of one game that respects players and one that exploits them

Practice: Analyze and document 5 games using the theory

Game AI and Pathfinding

Topics:

- Unity NavMesh system
- NavMesh agents setup
- Dynamic obstacles
- Off-mesh links
- Crowd simulation
- Behavior trees introduction
- State machines for complex AI
- GOAP (Goal-Oriented Action Planning)
- Machine learning agents basics
- Enemy squad tactics
- Stealth AI systems
- AI performance optimization

Projects built:

- Zombie survival AI
- Stealth game with smart guards
- RTS unit control
- Battle royale AI

Practice: Build 10 AI behaviors

Shader Graph and Visual Polish

Topics:

- Introduction to shaders
- Shader Graph in Unity
- Vertex and fragment shaders
- Surface shaders
- HLSL basics
- Creating custom materials
- Water shaders
- Toon/cel shading
- Dissolve and fade effects
- Hologram effects
- Outline shaders
- Performance considerations

Projects built:

- Stylized water shader
- Toon shaded game
- Sci-fi hologram effects
- Custom art style game

Practice: Create 15 custom shaders

Assessment: Month 5 check: a small AI or shader build live, plus a mixed quiz over months 1-5

Month 6: Finish a 3D Game (Weeks 21-24)

Procedural Generation

Topics:

- Procedural generation concepts
- Random vs seeded generation
- Perlin noise for terrain
- Dungeon generation algorithms
- Infinite world generation
- Procedural textures
- Procedural meshes
- L-systems for vegetation
- Wave function collapse
- Minecraft-style voxel worlds
- Procedural quests and stories

```
        - Balancing randomness
      Projects built:
        - Minecraft-style terrain
        - Roguelike dungeon generator
        - Infinite runner levels
        - No Man's Sky planets
      Practice: Generate 10 different content types
    Optimization and Profiling
      Topics:
        - Unity Profiler mastery
        - Frame rate optimization
        - Draw call batching
        - Texture atlasing
        - Mesh combining
        - LOD implementation
        - Object pooling
        - Coroutine optimization
        - Job System and Burst Compiler
        - ECS introduction
        - Mobile optimization deep dive
        - Memory management
      Projects built:
        - Optimize existing game
        - 1000 enemy system
        - Mobile performance test
        - VR performance optimization
      Practice: Optimize all projects to 60 FPS
    3D Game Build Week
      Topics:
        - Scoping a 3D game you can finish in two weeks
        - Blocking out the level before decorating it
        - Bringing it together: controller, AI, UI, audio, saves
        - Your own Blender assets in a real build
        - Playtesting: watching a stranger play in silence
        - The finish-line checklist
      Projects built:
        - A complete small 3D game: platformer, dungeon crawler or racing game
      Practice: A working build at the end of every session
    Phase 2 Capstone and Review
      Topics:
        - Polish pass: juice, sound, menus, feel
        - Publishing the 3D game on itch.io
        - Writing the game page like a small studio would
        - A review week shaped by what the batch found hard
        - Reading each other's projects: kind, useful feedback
      Projects built:
        - Phase capstone: the finished 3D game, published and playtested
      Practice: Ship one improvement based on player feedback
      Assessment: Phase 2 exam: capstone walkthrough plus a written mixed review spanning
      phases 1 and 2

----------------------------------------------------------------
PHASE 3: MULTIPLAYER, MOBILE AND THE WIDER CRAFT (MONTHS 7-8, WEEKS 25-32)
----------------------------------------------------------------
The systems that make games feel modern: multiplayer basics, mobile builds, an honest week
inside Unreal, AI in game development, narrative, and accessibility.

  Month 7: Multiplayer and Mobile (Weeks 25-28)
    Multiplayer Basics
      Topics:
        - Networking fundamentals for games
        - Client-server vs peer-to-peer
        - Unity Netcode for GameObjects
        - Setting up multiplayer
        - Player spawning and management
        - Synchronizing transforms
        - RPCs (Remote Procedure Calls)
        - Network variables
        - Lag compensation
        - Client prediction
        - Server authority
        - Cheating prevention basics
      Projects built:
        - 2-player Pong online
        - Multiplayer racing game
        - Co-op platformer
        - Simple battle arena
      Practice: Add multiplayer to 5 games
    Multiplayer Systems in Practice
```

Topics:
- Netcode for GameObjects: Unity's modern networking
- Lobbies and rooms: getting friends into one game
- Synchronizing movement without jitter
- Client prediction and lag, explained honestly
- Simple matchmaking for small games
- Hosting options for student projects, free tiers first
- Testing multiplayer with the batch as your players
- Why 100-player games are a different profession, and what they add

Projects built:
- A small Among Us-style social game for the batch
- Two-player versus game with a lobby

Practice: Host a batch play-session of your multiplayer build and fix what broke

Mobile Game Builds

Topics:
- Mobile game market overview
- Touch input and gestures
- Accelerometer and gyroscope
- Mobile UI best practices
- Performance optimization for mobile
- Battery and thermal management
- Android build setup
- iOS build requirements
- Installing your game on your own phone, the honest thrill

Projects built:
- Endless runner built for touch
- Your best 2D game running on an Android phone

Practice: Optimize one game until it runs smoothly on a mid-range phone

Leaderboards, Achievements and Saves Online

Topics:
- What game services are: the invisible half of modern games
- Building an online leaderboard for your own game
- Achievements that reward mastery, not grind
- Cloud saves: progress that follows the player
- Simple analytics: which level do players quit on?
- Privacy first: what a game should never collect, especially from kids
- Wiring one service into your published game

Projects built:
- Global leaderboard added to your best game
- Achievement set designed and implemented

Practice: Use your own analytics to find and fix your game's quit-point

Assessment: Month 7 check: a small multiplayer or mobile build live, plus mixed questions from earlier phases

Month 8: The Wider Craft (Weeks 29-32)

An Honest Week in Unreal Engine

Topics:
- Why Unreal: Fortnite, Valorant, and AAA games
- Unreal Engine installation and setup
- Editor interface navigation
- Blueprints visual scripting
- Creating your first Blueprint
- Variables and functions in Blueprints
- Event graph and construction script
- Actor classes and components
- Level design in Unreal
- Lighting and atmosphere
- Materials and textures
- Project settings and configuration
- What transfers from Unity, and what to learn when you need Unreal for real

Projects built:
- FPS prototype in Unreal
- Racing game conversion
- Puzzle game with Blueprints
- Environment showcase

Practice: Rebuild one small Unity mechanic in Unreal Blueprints

AI in Game Development

Topics:
- Where AI genuinely helps a small team: ideas, placeholder assets, dialogue drafts
- AI-assisted NPC dialogue, with a human editor in charge
- Unity ML-Agents: training a simple agent to play
- Procedural content with constraints: generated but designed
- What AI tools cannot do: taste, tuning and game feel
- Where an AI account is needed, it is set up with a parent
- Crediting tools honestly in your game page

Projects built:
- An NPC with AI-drafted, human-edited dialogue
- A simple ML-Agents experiment that learns a behavior

Practice: One feature built twice: by hand and AI-assisted, with notes on the

      difference
    Game Narrative and Writing
      Topics:
        - Interactive storytelling
        - Branching narratives
        - Character development
        - World building
        - Dialog writing
        - Environmental storytelling
        - Cutscene planning
        - Voice acting direction
        - Localization considerations
        - Narrative tools (Yarn, ink)
        - Player choice systems
        - Emotional design
      Projects built:
        - Narrative-driven game
        - Branching story system
        - Character backstories
        - World bible creation
      Practice: Write game narratives
    Accessibility and Inclusive Design
      Topics:
        - Accessibility importance
        - Visual accessibility
        - Audio accessibility
        - Motor accessibility
        - Cognitive accessibility
        - Colorblind modes
        - Subtitle systems
        - Control remapping
        - Difficulty options
        - Inclusive character design
        - Cultural sensitivity
        - Testing with disabled gamers
      Projects built:
        - Accessibility audit
        - Accessibility features implementation
        - Inclusive game design
        - Testing documentation
      Practice: Make all games accessible
      Assessment: Phase 3 exam: apply an accessibility audit to your own game live, plus a
      cumulative mixed paper from phases 1-3

PHASE 4: PRODUCTION AND THE CAPSTONE (MONTHS 9-10, WEEKS 33-40)

The final month pair runs like a small studio: pipeline, polish, publishing, and a capstone
game shipped and presented at demo day.

  Month 9: Studio Habits (Weeks 33-36)
    How Real Games Get Made
      Topics:
        - The production phases: concept, prototype, vertical slice, polish, ship
        - The vertical slice: one perfect level before forty rough ones
        - Scoping: why most games die of ambition, not difficulty
        - Task boards with Trello: seeing the work
        - Version control habits for game projects
        - Playtesting as a process, not an event
        - Bug triage: ship-stoppers, annoyances and wishes
        - Milestones a real team would recognize
      Projects built:
        - A one-page production plan for your capstone, with milestones and a task board
      Practice: Run your week like a studio: board updated daily, one milestone hit
    Polish and Game Feel Week
      Topics:
        - The juice checklist: screenshake, particles, sound, feedback
        - Menus and onboarding: the first 60 seconds decide everything
        - Difficulty tuning with real playtesters
        - Performance pass: stable frame rate as a feature
        - Bug triage: what to fix, what to cut, what to ship
        - Before-and-after: recording the polish difference
      Projects built:
        - A measured polish pass on your best game, with before/after footage
      Practice: Run three playtests and ship fixes from each
    Publishing on itch.io and Game Jams
      Topics:
        - itch.io done properly: where indie careers actually start
        - Writing a game page that makes strangers click play

- Screenshots, GIFs and the thirty-second pitch
- Game jams: the fastest way to grow as a developer
- Jam survival: scope, sleep and shipping anyway
- Age ratings and content honesty for your own games
- The road beyond: what Steam and app stores ask for, told honestly
- Handling feedback and reviews with a straight back

Projects built:
- All three finished games live on a polished itch.io profile
- One weekend game-jam entry, built and submitted

Practice: Enter one online game jam during the course

Portfolio and Devlog

Topics:
- A game developer portfolio: playable links beat screenshots
- Writing a short devlog: what you built, what fought back
- Recording clean gameplay GIFs and clips
- Your itch.io profile as the portfolio hub
- Explaining one system deeply: the interview habit
- Curating: your best three games, not all twenty

Projects built:
- Portfolio page with playable links, clips and one deep-dive writeup

Practice: Publish one devlog post about your favorite build

Assessment: Month 9 check: portfolio review plus a mixed assignment spanning the whole course so far

Month 10: The Capstone (Weeks 37-40)

Capstone Pre-Production

Topics:
- Choosing the capstone: your strongest idea, scoped to three weeks
- The one-page design document: concept, core loop, scope
- What makes it yours: the one unique element worth polishing
- Technical checklist: which systems from the course it will use
- Art and audio direction with the assets you can actually make
- The cut list: features you already know you will drop
- Instructor sign-off on scope before a single line is written

Projects built:
- Signed-off design document plus a greybox prototype of the core loop

Practice: Prototype the core loop in greybox before the build weeks begin

Capstone Build

Topics:
- Two focused weeks: working core by day 4, feature-complete by day 10
- Instructor checkpoints and course corrections
- Using everything: mechanics, AI, UI, audio, saves, polish
- Daily builds that always run
- Playtesting mid-build and acting on it
- Cutting scope without cutting the soul of the game

Projects built:
- The capstone game, built over two weeks with version control and daily builds

Practice: A playable build at the end of every session, no exceptions

Launch, Demo Day and What Comes Next

Topics:
- Publishing the capstone with a proper game page and trailer GIF
- Presenting at demo day: live gameplay plus the hardest bug story
- An honest skills inventory: strong, shaky, next
- Where each path leads: deeper Unity, Unreal, 3D art, or our DSA course for the CS side
- Staying sharp: game jams, communities and healthy practice habits

Projects built:
- Demo day: the capstone played live by the audience

Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; a student who falls short gets a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Pong, Breakout, physics games, a polished platformer, and a complete 2D capstone published on itch.io
- Phase 2: 3D prototypes, an animated character, environments with your own Blender assets, and a finished 3D game
- Phase 3: multiplayer experiments, a mobile build on a real phone, an Unreal Blueprints mechanic, and an accessibility audit
- Phase 4: a polish pass with before/after footage, a game-jam entry, and the final capstone presented at demo day

Total projects: 20+ games and prototypes, with at least three complete games published on itch.io

SKILLS THE STUDENT WILL HAVE AT THE END

- C# programming: from variables to classes and game architecture, learned through play
- Unity 2D: physics, movement feel, enemy AI, UI, audio, saves, complete shipped games
- Unity 3D: controllers, animation, terrain, lighting, shaders, procedural generation,

optimization
- Blender: modeling, texturing and importing your own game assets
- Game design: flow, pacing, difficulty, narrative, accessibility, and ethical design
- Multiplayer and mobile: networked basics and builds that run on real phones
- Production habits: scoping, playtesting, polishing, publishing and presenting

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building along with the instructor
Homework: Homework after every class, and a mixed-review assignment at the end of every month so earlier skills stay sharp
Practice: 3-4 hours of building and playtesting between classes
Review: Every submitted build reviewed with written feedback; recurring issues reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so nobody stays stuck on a broken build for days
Progress updates: Regular progress notes to parents: what was built, what comes next, and where encouragement at home helps

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Deeper Unity and C#: the direct road toward junior game programming roles later
- Data Structures and Algorithms: the computer science muscle for engineering paths and interviews
- 3D art and design: Blender skills here open the art-side road
- Game jams and communities: the ecosystem where young developers actually grow
- Transferable skill: everything here, C#, systems thinking, shipping, carries into any software path

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my teen build in this game development course?
A: More than 20 games and prototypes across ten months, including at least three complete, published games: a polished 2D game on itch.io by month 3, a finished 3D game with their own Blender assets by month 6, and a final capstone presented live at demo day. Along the way they build multiplayer experiments, a mobile game running on a real phone, and a game-jam entry. Finishing and publishing is the point: the course is built against the tutorial graveyard.

Q: How long does it take a teenager to learn game development?
A: To genuinely build and ship games: 10 months at 2 live classes a week plus 3 to 4 hours of practice, and the first published game happens in month 3, not month 10. The syllabus keeps 2 months of margin in hand: students who need more repetition take up to 12 months with no rushing and no penalty.

Q: How is my teen assessed? Is the certificate just for attending?
A: No. Classwork runs inside every session, homework follows every class, and each month ends with an assignment that deliberately mixes new questions with earlier topics so knowledge compounds instead of fading. Every phase closes with a cumulative exam, and the course ends with a final exam: a practical build plus written questions drawn from the whole syllabus. Passing earns the certificate; falling short earns a focused revision plan and a free retest. We optimize for real knowledge and real depth, not a stream of feel-good badges.

Q: Unity or Unreal: which engine does the course teach?
A: Unity, deeply, and that is a deliberate choice: Unity with C# is the standard indie and mobile engine, gentler to learn, and its concepts transfer everywhere. Month 8 includes an honest week inside Unreal with Blueprints, so students understand the other side of the industry and know exactly what to learn if a future project demands it. A shallow two-engine tour would leave a teen fluent in neither.

Q: What computer does my teen need?
A: Any reasonably recent laptop or desktop with a stable internet connection. Unity runs comfortably on modest hardware for 2D work; for the 3D months a dedicated graphics card is nice but not required, we scope the 3D projects so integrated graphics can handle them. All software used, Unity, Visual Studio, Blender, Mixamo and itch.io, is free.

Q: Is game development actually useful, or just play?
A: It is one of the most demanding forms of programming: real-time systems, physics, AI,

optimization and user experience in one discipline. The C# skills transfer directly to general software roles, the math quietly reinforces school physics and geometry, and shipping a real product teaches scoping and follow-through. Game development is the most motivating wrapper serious programming has ever had.

Q: Does the course cover multiplayer games?
A: Yes, at an honest level: month 7 covers networked multiplayer fundamentals in Unity, enough to build small online games and understand how the big ones work. Full production netcode for thousands of players is a professional specialization; this course builds the base it stands on and says so plainly.

Q: Will my teen publish real games?
A: Yes, on itch.io, the platform where real indie careers start, with a proper game page and playtested builds. At least three finished games go live during the course, plus a game-jam entry. We do not promise Steam releases or player counts; we teach the craft of finishing and shipping, which is what those things grow from.

Q: Is there a trial class?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. It is a real class, your teen will build something small in it, and parents are welcome to sit in. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: How are classes delivered?
A: Live 1-hour online classes with a real instructor, never pre-recorded videos. Groups are small and hands-on: students build during class and get unstuck on the spot, which matters enormously in game development where one misconfigured component can stall a beginner for hours.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/complete-game-development-masterclass-for-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 67 OF 137: GCSE COMPUTER SCIENCE: AQA, OCR AND EDEXCEL EXAM PREP
==========

Course link: https://learn.modernagecoders.com/courses/gcse-computer-science-course
Subject area: GCSE Computer Science
Designed for: Teens
Who can join: GCSE students on AQA, OCR or Edexcel Computer Science; taught from the basics, no prior coding required
Course length: 6 months (24 weeks), joinable any month
Time needed: 2 live classes/week + weekly programming and past-paper practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online GCSE Computer Science classes taught to your child's actual board. AQA 8525, OCR J277 and Edexcel 1CP2 set different papers, and the course is built around the one they will sit.

ABOUT THE PROGRAMME

The three GCSE Computer Science specifications share a name and set different exams. AQA 8525 runs a two-hour Paper 1 on computational thinking and programming and a one-hour-forty-five Paper 2 on computing concepts, each ninety marks and each half the qualification. OCR J277 runs two papers of an hour and a half and eighty marks each. Edexcel sets its own 1CP2. A student revising from the wrong board's past papers is practising the wrong exam, and it is the commonest avoidable problem we see. The first thing we ask is which board, and the course is built around that answer. Everything is taught live, in a batch of fifteen to twenty or one to one, with programming in Python and past papers marked against the real mark scheme.

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What Your Child Needs to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PROGRAMMING THAT SURVIVES PAPER 1 (MONTHS 1-2)

Real Python, written by hand, plus the code-tracing habit that carries more marks than any single topic on the specification.

Month 1: Python Fundamentals and Code Tracing (Weeks 1-4)

Variables, Input and Output, and Reading an Error

Topics:

- Data types, casting and the arithmetic that catches people out in exams
- Writing to a specification instead of writing what you felt like writing
- Reading a Python error message and fixing the cause rather than guessing
- Why every answer in this course is written by hand before it is typed

Selection, and the Conditions Examiners Set

Topics:

- if, elif and else, and the nested conditions that appear in longer questions
- Boolean operators, and the compound conditions students misread under time
- Common exam traps: assignment against comparison, and off-by-one boundaries
- Writing the condition in words first, then in code

Iteration, and Two-Column Trace Tables

Topics:

- for and while loops, and choosing the right shape for the problem
- Nested loops, and stating the range out loud before writing them
- The two-column trace table, drilled until it is the automatic first move
- Why tracing in your head is slower and less accurate than writing it down

Lists, Strings and Standard Algorithms

Topics:

- Lists and string handling exactly as the specifications describe them
- Linear search, bubble sort and merge sort, traced as well as coded
- Counting, totalling and finding a maximum, which recur across every board
- First timed set of past-paper programming questions

Month 2: Subprograms, Files and Robust Programs (Weeks 5-8)

Functions, Procedures and Parameters

Topics:

- Writing a subprogram to a given signature, which is how questions are set
- Parameters and return values, and the difference examiners test
- Local and global scope, a reliable short-answer topic on every board
- Decomposing a long question into subprograms before writing any code

File Handling and Data Validation

Topics:

- Reading from and writing to text files as the specifications require
- Validation and authentication routines, which appear as set-piece questions
- Defensive design, and what makes a program robust in mark-scheme terms
- Writing test data: normal, boundary and erroneous

Testing, Debugging and the Language of Errors

Topics:

- Syntax, logic and runtime errors, named precisely as the exam names them
- Building a test plan and completing a test table
- Finding a fault by halving the search space rather than by guessing
- Refining a program in response to a test result, which is examined directly

Paper 1 Under Time

Topics:

- A full programming paper in the student's own board format
- Marked against the real mark scheme rather than against intent
- Pacing: how long each question is worth in minutes
- Identifying which question type this student loses marks on

---

PHASE 2: COMPUTING CONCEPTS AND SYSTEMS (MONTHS 3-4)

The theory half, taught as systems that relate to each other rather than as vocabulary to be memorised, because that is what the longer questions reward.

Month 3: Data Representation and Computer Systems (Weeks 9-12)

Binary, Hexadecimal and Binary Arithmetic

Topics:

- Converting between denary, binary and hexadecimal in both directions, quickly
- Binary addition, overflow, and binary shifts with their effect explained
- Character sets, and the size calculations that follow from them
- Doing all of it by hand, at speed, because the paper expects it

Images, Sound and Compression

Topics:

- Resolution, colour depth and the file-size calculation examiners set

        - Sample rate, bit depth and duration, calculated rather than described
        - Lossy against lossless compression, with a real reason for choosing one
        - The exact wording that earns the mark in a compression comparison
    The CPU, the Fetch-Decode-Execute Cycle and Memory
      Topics:
        - CPU components and what each one does, in the specification's language
        - The fetch-decode-execute cycle, sequenced correctly and explained
        - Factors affecting performance: clock speed, cores and cache
        - RAM, ROM, virtual memory and secondary storage compared
    Software, Translators and Systems Architecture
      Topics:
        - System software against application software
        - Compilers and interpreters, and where each is genuinely preferable
        - Utility software and the operating system's actual jobs
        - Embedded systems, and why the question always mentions a washing machine

  Month 4: Networks, Security, Databases and Impact (Weeks 13-16)
    Networks, Topologies and Protocols
      Topics:
        - LAN and WAN, wired and wireless, with the trade-offs stated
        - Star and mesh topologies, and the layered protocol stack
        - Packet switching, and what a protocol actually is
        - Performance factors, which are a reliable extended-response question
    Cyber Security and Threats
      Topics:
        - Malware, phishing, social engineering and brute force, distinguished properly
        - Penetration testing, firewalls, encryption and access levels as mitigations
        - Matching a threat to the correct prevention, which is how it is examined
        - Writing a security answer that names both the threat and the response
    Databases and SQL
      Topics:
        - Tables, records, fields, primary keys and data types
        - SELECT, FROM and WHERE, written correctly under exam conditions
        - Where SQL appears on each board, since this differs between them
        - Practising the query shapes that recur in past papers
    Ethical, Legal and Environmental Impact
      Topics:
        - Privacy, legislation and environmental impact, with real examples
        - Structuring an extended-response answer so it reaches the top band
        - Giving both sides and then a judgement, which the mark scheme wants
        - Timed practice on the longest questions on the paper

---
PHASE 3: BOARD-SPECIFIC EXAM MASTERY (MONTHS 5-6)

The half of the course that separates the three specifications, spent entirely on the papers
this student will actually sit.

  Month 5: Your Board's Papers, In Their Own Format (Weeks 17-20)
    Knowing Your Own Specification
      Topics:
        - AQA 8525: Paper 1 two hours and ninety marks, Paper 2 one hour forty-five and ninety marks
        - OCR J277: two papers, an hour and a half and eighty marks each
        - Edexcel 1CP2 in its own structure, for students entered on it
        - Why revising from another board's past papers wastes the term
    Command Words and the Mark Scheme
      Topics:
        - State, describe, explain, compare and evaluate ask for different things
        - Reading how many marks a question is worth and answering to that length
        - Marking your own work against the real scheme rather than against intent
        - The sentences that earn marks, and the correct ideas that do not
    The Long Questions
      Topics:
        - Extended-response structure: both sides, then a supported judgement
        - Programming questions worth eight marks or more, planned before written
        - Time budgeting across a whole paper, practised rather than assumed
        - Recovering when a question is not understood on first reading
    Full Paper 1, Marked and Reviewed
      Topics:
        - A complete paper under exam conditions in the student's board format
        - Detailed review of every lost mark, with the reason named
        - A revision list built from the actual errors rather than from the topic list
        - Re-testing the same question types two weeks later

  Month 6: Full Papers and Final Preparation (Weeks 21-24)
    Full Paper 2, Marked and Reviewed
      Topics:
        - The theory paper under time, in the correct board format
        - Extended responses marked to the band descriptors

- Identifying whether the remaining risk is Paper 1 or Paper 2
- Directing the last month at the weaker paper rather than at both

Targeted Repair
Topics:
- Rebuilding the two or three topics the mocks exposed
- Programming questions from a blank page, repeatedly
- Trace tables and binary arithmetic drilled back to automatic
- Short daily retrieval rather than long revision sessions

Second Full Set
Topics:
- Both papers again, under time, from a different series
- Measuring improvement against the first set question by question
- Confirming pacing holds when the questions are unfamiliar
- Final list of the specific things still costing marks

Exam Week Readiness
Topics:
- A short, specific revision plan for the final fortnight
- The reference material allowed in your board's papers
- What to do in the first five minutes of each paper
- Walking in knowing the format, so the only new thing is the questions

HOW A TYPICAL WEEK IS SPLIT

Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- A strong GCSE Computer Science grade across both papers
- A confident move into A Level Computer Science or an equivalent sixth-form course
- Real Python ability, which transfers to every later qualification and to work
- The code-tracing and debugging habits that A Level assumes and does not teach
- A foundation for AP Computer Science A, IB Computer Science or Cambridge A Level for students changing curriculum

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which exam board do you teach?
A: All three, and separately. AQA 8525 sets a two-hour Paper 1 worth ninety marks and a one-hour-forty-five Paper 2 also worth ninety marks, each half the GCSE. OCR J277 sets two papers of an hour and a half and eighty marks each. Edexcel 1CP2 has its own structure. The first thing we ask is which board your child is entered for, because revising from the wrong board's past papers is the commonest avoidable problem in this subject.

Q: Which programming language do you teach?
A: Python, which is what the overwhelming majority of UK schools use and what most students meet in class. If your child's school teaches something else we will work in that instead, because the exam assesses computational thinking rather than a particular syntax, and consistency with school matters more than our preference.

Q: My child understands it in class but cannot start a question. Is that normal?
A: It is extremely common and it is the specific problem this course is built to fix. Following a worked example is a different action from producing an answer from a blank page, and only the second one is examined. Every programming session here starts from nothing, on paper, before anything is typed.

Q: How many classes a week, and how long is the course?
A: Two live classes a week over roughly twenty-four weeks, joinable in any month. A student in Year 10 usually spreads it across the year, and a Year 11 student closer to the exams normally goes one to one so the syllabus can be reordered around whichever paper is the risk.

Q: Do you mark past papers?
A: Yes, against the published mark scheme rather than against what the student meant. That distinction matters more than it sounds: students routinely write a correct idea in words the scheme does not credit, and finding that out in January is worth considerably more than finding it out in May.

Q: Is this the same as Cambridge IGCSE Computer Science?
A: No. Cambridge IGCSE 0478 is taken mainly in international schools, grades A star to G, and requires most coding answers in pseudocode rather than a real language. UK GCSE grades nine to one and generally expects answers in a real programming language. Two students of

identical ability need genuinely different exam training.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/gcse-computer-science-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 68 OF 137: GCSE MATHS RESIT COURSE ONLINE: POST-16 AND ADULTS

==========

Course link: https://learn.modernagecoders.com/courses/gcse-maths-resit-course
Subject area: UK Exam Board Mathematics
Designed for: College students
Who can join: Learners aged 16 and over, and adults, resitting GCSE Maths with AQA, Edexcel or OCR
Course length: 6 months (24 weeks) for a summer entry; a shorter plan for the November series
Time needed: 2 live classes/week + 3 hours of practice, fitted around college or work
Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

GCSE maths resit course online for post-16 learners and adults: a diagnostic, targeted repair of the topics that cost marks, then full papers for the November or summer series.

In one line: Not the whole GCSE again from page one: a diagnostic finds the marks you are losing, we repair those topics, then we practise full papers until the exam feels familiar.

ABOUT THE PROGRAMME

A resit is a different job from a first attempt. You have already seen most of the content, and the marks you missed usually cluster in a handful of topics and a few habits: rushing, not showing working, misreading what a question asks. This course starts with a diagnostic paper, maps exactly where the marks went, and spends the teaching time there. GCSE Maths is a linear qualification. AQA's specification, for example, has three papers of 1 hour 30 minutes and 80 marks each, the first without a calculator, and two tiers: Foundation, graded 1 to 5, and Higher, graded 4 to 9. All three papers are taken at the same tier in the same series. AQA offers November entries only to students who were at least 16 on the previous 31 August, which is why the November series is the resit series. For 16 to 18 year olds on a study programme, the government's condition of funding guidance sets out who must keep studying maths: students without a grade 4 continue, and full-time students with a grade 3 must study towards GCSE. We work alongside your college, or with you directly as an adult entering through an exam centre. Classes are live and small, and every paper you sit with us is marked against the published mark scheme so you can see the trend, not just hope for it.

WHAT MAKES THIS COURSE DIFFERENT

- Starts with a diagnostic paper and a topic-by-topic map of lost marks, so the teaching time goes where the marks are
- Built around the boards' own specifications: AQA 8300, Edexcel 1MA1 and OCR J560, all three papers at your tier
- Non-calculator paper habits rebuilt from the first fortnight
- A November plan for learners close to the grade they need and a summer plan for those who need more time
- Evening and weekend classes for adults and learners at college during the day
- Every practice paper marked against the published mark scheme, with an error clinic afterwards

THE LEARNING PATH, STEP BY STEP

Phase 1: Diagnostic and number: a full diagnostic paper, the lost-marks map, then number, fractions, percentages and the non-calculator paper
Phase 2: Algebra and ratio: expressions, equations, graphs, ratio, proportion and rates of change
Phase 3: Geometry, probability and statistics: angles, area and volume, Pythagoras and trigonometry, probability and data

Phase 4: Papers: full sets of three papers at your tier under real timing, marked, reviewed and repaired

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: You have taken GCSE Maths before, or followed the course at school. The diagnostic paper shows exactly where to start
Exam entry: Learners at college are entered by their college. Adults enter through an exam centre that accepts private candidates; you arrange the entry and we teach to your board and tier
Equipment: A scientific calculator, ruler, protractor, compasses, and a device with stable internet for live classes
Timing: For the November series, start as early in the autumn term as possible. For the summer series, start any time from September to January

WHO THIS COURSE IS FOR

College students: 16 to 18 year olds at college or sixth form who need to keep working towards GCSE Maths
Adults: Adults who need GCSE Maths for a job, training course or university application
Near miss learners: Learners who missed the grade they needed by a few marks and want a focused plan for November
Anxious learners: Learners who know more than their result shows but lose marks under pressure
International candidates: Candidates sitting UK GCSE Maths as private candidates from abroad

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: DIAGNOSTIC AND NUMBER (MONTHS 1-2, WEEKS 1-8)

Find the lost marks first, then repair number, the strand that runs through every paper and dominates the non-calculator paper.

Month 1: Diagnostic and the Lost-Marks Map (Weeks 1-4)

Weeks 1-2: The Diagnostic Paper

Topics:
- A full past-style paper at your tier, sat under timing
- Marking against the official-style scheme together
- Sorting every lost mark: knowledge, method, accuracy or reading
- Building the lost-marks map topic by topic
- Choosing the tier with you and your college if it is still open
- Setting a realistic plan for November or summer

Projects built:
- Lost-marks map: every topic coloured by how many marks it cost

Practice: Short daily sets on the three topics that cost the most marks

Weeks 3-4: Number Without a Calculator

Topics:
- Written methods for the four operations, decimals included
- Order of operations and negative numbers
- Factors, multiples, primes and prime factorisation
- Estimation and rounding to check answers
- Powers, roots and, at Higher, index laws and standard form
- Setting out working so method marks are earned

Projects built:
- Non-calculator method sheet: one worked example of every written method

Practice: 20 non-calculator questions a week

Assessment: Month 1 check: a 40-minute non-calculator section

Month 2: Fractions, Decimals and Percentages (Weeks 5-8)

Weeks 5-6: Fractions and Decimals

Topics:
- The four operations with fractions and mixed numbers
- Converting between fractions, decimals and percentages
- Fractions of amounts in context
- Recurring decimals as fractions at Higher
- Bounds and error intervals
- Common slips and how to catch them

Projects built:
- Repair log: each fraction slip from the diagnostic re-done correctly

Practice: 15 fraction and decimal questions a week, half without a calculator

Weeks 7-8: Percentages and Money

Topics:
- Percentages of amounts and percentage change
- Reverse percentages
- Simple and compound interest, growth and depreciation
- Value-for-money comparisons set out clearly

        - Multipliers as the quick calculator method
        - Writing a conclusion that answers the question
      Projects built:
        - Money problem set: interest, discounts and comparisons solved two ways
      Practice: 12 percentage problems a week
      Assessment: Month 2 check: a 45-minute number paper with calculator and non-calculator
      parts

PHASE 2: ALGEBRA AND RATIO (MONTHS 3-4, WEEKS 9-16)

The strands where resit learners most often lose marks they could win: forming and solving
equations, reading graphs, and ratio in context.

  Month 3: Expressions, Equations and Graphs (Weeks 9-12)
    Weeks 9-10: Expressions and Equations
      Topics:
        - Simplifying, expanding and factorising
        - Substituting into formulae
        - Solving linear equations, including unknowns on both sides
        - Forming an equation from words
        - Inequalities on a number line
        - At Higher: quadratics by factorising and the formula, and simultaneous equations
      Projects built:
        - Equation bank: one solved example of each equation type, with a check by substitution
      Practice: 15 algebra questions a week
    Weeks 11-12: Sequences and Graphs
      Topics:
        - Linear sequences and the nth term
        - Plotting and reading straight-line graphs
        - Gradient and y = mx + c
        - Real-life graphs: distance-time and conversion graphs
        - Quadratic graphs and reading roots
        - At Higher: graph transformations and equations of circles
      Projects built:
        - Graph file: six graph types drawn and annotated
      Practice: 12 sequence and graph questions a week
      Assessment: Month 3 check: a 45-minute algebra paper

  Month 4: Ratio, Proportion and Rates (Weeks 13-16)
    Weeks 13-14: Ratio and Proportion
      Topics:
        - Simplifying ratios and sharing in a ratio
        - Ratio problems from recipes, maps and mixtures
        - Direct and inverse proportion
        - Scale factors and similar shapes
        - Unit conversions
        - At Higher: proportion equations
      Projects built:
        - Ratio problem set: five real contexts solved with a bar model and then algebra
      Practice: 12 ratio and proportion questions a week
    Weeks 15-16: Rates and Compound Measures
      Topics:
        - Speed, distance and time
        - Density and pressure
        - Rates of pay and comparing value
        - Interpreting rates from graphs
        - Multi-step problems that mix ratio and percentages
        - Planning a long question before writing
      Projects built:
        - Repair log updated: ratio and rate errors from the diagnostic re-done
      Practice: 10 multi-step problems a week
      Assessment: Month 4 check: a full 1 hour 30 minute calculator paper, marked

---
PHASE 3: GEOMETRY, PROBABILITY AND STATISTICS (MONTH 5, WEEKS 17-20)

Shape, space and data, taught to the depth your tier needs and practised in exam-style
questions.

  Month 5: Geometry, Probability and Statistics (Weeks 17-20)
    Weeks 17-18: Geometry and Measures
      Topics:
        - Angle facts, parallel lines and polygons
        - Perimeter, area and volume, including circles and cylinders
        - Pythagoras' theorem
        - Right-angled trigonometry
        - Transformations and constructions
        - At Higher: sine and cosine rules, circle theorems and vectors

```
        Projects built:
          - Geometry sheet: every formula and angle fact with a worked example
        Practice: 15 geometry questions a week
      Weeks 19-20: Probability and Statistics
        Topics:
          - Probability scales, outcomes and relative frequency
          - Tree diagrams and Venn diagrams
          - Averages and range from lists and tables
          - Charts, scatter graphs and correlation
          - Spotting misleading statistics
          - At Higher: cumulative frequency, box plots and histograms
        Projects built:
          - Data question bank: one solved example of every chart and average type
        Practice: 12 probability and statistics questions a week
        Assessment: Month 5 check: a full mixed paper at your tier, marked

  ---------------------------------------------------------------
  PHASE 4: FULL PAPERS (MONTH 6, WEEKS 21-24)
  ---------------------------------------------------------------
  No new content. Full sets of three papers at your tier under real timing, marked against
  published mark schemes, with repair lessons between papers.

    Month 6: Paper Sets and Repair (Weeks 21-24)
      Weeks 21-22: Full Sets Under Timing
        Topics:
          - A full set: non-calculator paper then two calculator papers
          - Marking against the published mark scheme
          - Updating the lost-marks map
          - Repair lessons on the two weakest topics
          - Time management: which questions to do first
          - Checking in the last ten minutes
        Projects built:
          - Paper tracker: every paper's score on one chart
        Practice: One full paper every few days, alternating calculator and non-calculator
      Weeks 23-24: Final Set and Exam Readiness
        Topics:
          - A final full set under real timing
          - Review of every lost mark
          - Formulae sheet use where your board provides one
          - Equipment check and calculator settings
          - A calm plan for the days before the exam
          - What to do on results day, whatever happens
        Projects built:
          - Completed paper tracker and lost-marks map, kept for revision
        Practice: Short targeted sets from the lost-marks map
        Assessment: Final paper set, marked and reviewed one to one

PROJECTS THE STUDENT BUILDS AND KEEPS

  - A lost-marks map from the diagnostic, updated after every paper
  - A non-calculator method sheet
  - A repair log of every diagnostic error re-done correctly
  - An equation bank, a graph file and a geometry sheet
  - A data question bank
  - A paper tracker across the whole course

  Total projects: A personal resit kit: lost-marks map, method sheets, repair log and a marked paper tracker

SKILLS THE STUDENT WILL HAVE AT THE END

  - Number and the non-calculator paper
  - Fractions, decimals, percentages and money
  - Algebra, sequences and graphs at your tier
  - Ratio, proportion and rates of change
  - Geometry, probability and statistics
  - Paper technique: timing, showing working and checking

HOW A TYPICAL WEEK IS SPLIT

  Live classes: 2 live one-hour classes per week, with evening and weekend options
  Practice: About 3 hours a week of targeted practice from the lost-marks map
  Review: Practice marked with comments; papers marked against published mark schemes

SUPPORT INCLUDED

  Doubt support: WhatsApp support between classes, with worked solutions for stuck questions
  Progress updates: A monthly progress note with paper scores and the updated lost-marks map

CERTIFICATION DETAIL
```

  Completion: Course-completion certificate from Modern Age Coders

WHERE THIS CAN LEAD

  - Jobs, apprenticeships, training and university courses that ask for GCSE Maths; check each one's exact grade requirement
  - Our A Level Maths course for learners who discover they enjoy it
  - Our Functional Skills Maths course as an alternative level 2 route where it is accepted

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: When can I resit GCSE Maths?
  A: There are November and summer series. AQA, for example, offers GCSE Mathematics every May/June and November, and November entries are only available to students who were at least 16 on the previous 31 August. Check your board and exam centre for the exact dates.

  Q: Do I have to resit GCSE Maths if I got a grade 3?
  A: If you are 16 to 18 on a full-time study programme in England, the government's condition of funding guidance says students with a grade 3 must study towards GCSE Maths. Students with grade 2 or below can work towards Functional Skills Level 2 or a GCSE. Adults are not bound by these rules and can choose.

  Q: Should I sit the November resit or wait for the summer?
  A: November suits learners who were close to the grade they need and can study steadily through the autumn. The government's guidance asks colleges to consider readiness, including attendance, engagement and progress, before entering students with a low grade 3 or below. Our diagnostic paper helps you and your college decide.

  Q: Foundation or Higher tier for a resit?
  A: Foundation is graded 1 to 5 and Higher 4 to 9. Many resit learners aiming for a 4 or 5 sit Foundation, and learners aiming higher sit Higher. Your diagnostic score and the grade you need decide it, together with your college or exam centre.

  Q: Can adults resit GCSE Maths online?
  A: The teaching can be fully online. The exams are sat in person at an exam centre that accepts private candidates, which makes your entry. We teach to your board and tier and practise its paper style.

  Q: Is the whole GCSE taught again?
  A: No. The diagnostic paper shows which topics are costing marks, and the course spends most of its time there, with full papers throughout. Topics you already handle well get a quick check rather than a re-teach.

  Q: Can I do Functional Skills instead of a GCSE resit?
  A: Sometimes. Some employers and courses accept Functional Skills Maths Level 2 and some ask for GCSE. For 16 to 18 year olds, the condition of funding rules link the choice to your GCSE grade. We run both courses, and the free demo can help you decide. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: How are the classes run?
  A: Live online classes with a real teacher, in a group of 15 to 20 learners at a similar level or one to one, with evening and weekend options. Current fees for your region are shown in the pricing section of this page.

  Q: Can I try a lesson first?
  A: Yes. Book a free demo class. The teacher works through a few GCSE questions with you, sees where the marks are going, and suggests a November or summer plan and group or one-to-one teaching. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/gcse-maths-resit-course
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 69 OF 137: GCSE MATHS: FOUNDATION & HIGHER (9-1), AQA, EDEXCEL, OCR
==========

Course link: https://learn.modernagecoders.com/courses/gcse-mathematics-mastery
Subject area: Exam Board Mathematics
Designed for: High school

Who can join: Years 9-11 GCSE students, Foundation and Higher tiers, plus resit candidates
Course length: 18 months (72 weeks), joinable any month
Time needed: 2 live one-hour classes per week + weekly past-paper work
Certificate: Course completion certificate + documented paper-score progression

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  GCSE Maths course for AQA, Edexcel and OCR: Foundation and Higher tiers, non-calculator Paper 1 fluency, grade 7-9 reach and past-paper technique.

  In one line: Foundation to grade 5 secure, Higher to grade 9, taught for understanding, trained for the papers.

ABOUT THE PROGRAMME

  An 18-month live program for GCSE mathematics across AQA, Edexcel and OCR, at Foundation or Higher tier. GCSE maths is three papers, one strictly non-calculator, with grades that gatekeep sixth form, apprenticeships and A-Level maths. This course teaches the syllabus understanding-first (so the mock slide of year 11 never happens), trains bare-hands fluency for Paper 1 in every class, brings the Higher-tier grade 7-9 topics in early enough to settle, and converts it all into marks through past-paper cycles against your exact board's mark schemes.

WHAT MAKES THIS COURSE DIFFERENT

  - Board-exact: AQA, Edexcel or OCR confirmed first, and every past paper comes from your board
  - Tier-honest: evidence for Foundation-vs-Higher decisions, and a real runway from grade 4-risk to 6-secure, or 6 to 9
  - Non-calculator Paper 1 fluency drilled in every class
  - The grade 7-9 separator topics taught months early so they settle instead of being crammed
  - Mark-scheme literacy: method marks, working discipline and the plus-minus habits that stop point leaks
  - Understanding-first teaching: algebra as a language, so re-framed exam questions do not break the student
  - One dedicated mentor across the whole GCSE run, a full interactive hour, twice a week
  - Resit candidates welcome, with a compressed evidence-led plan

THE LEARNING PATH, STEP BY STEP

  Phase 1: The Foundation core built properly: number fluency, algebra as a language, and the geometry and data basics
  Phase 2: The Higher reach with settling time: quadratics, functions, trigonometry, vectors and the grade 7-9 separator topics
  Phase 3: Past-paper craft by board: all three papers, timing plans, mark banking and full cycles into the exam series

WHAT A STUDENT NEEDS BEFORE STARTING

  Education: Years 9-11 GCSE students (AQA, Edexcel or OCR), or post-16 resit candidates
  Math background: Any, the diagnostic finds the real base; Foundation gaps are repaired before Higher reach
  Equipment: Computer with stable internet; a scientific calculator for Papers 2-3
  Time commitment: 2 live hours plus 1-2 practice hours weekly
  English: Course taught in English
  Motivation: An exam series on the calendar; we supply the method

WHO THIS COURSE IS FOR

  Higher aimers: Students who need the move from Foundation-risk to Higher-comfortable made real
  Grade 9 hunters: Grade 6-7 students converting understanding into 8-9 through separator topics and paper craft
  Mock sliders: Students whose grades sink each mock as memorized methods run out
  Paper 1 worriers: Calculator-raised students facing the non-calculator paper
  Resitters: Post-16 candidates who need the threshold cleared with a method this time

FULL CURRICULUM, WEEK BY WEEK

  This is the complete teaching plan. Parents can ask about any week.

  PHASE 1: THE CORE, BUILT PROPERLY (MONTHS 1-6)

Foundation-tier territory owned deeply is what Higher-tier grades stand on. This phase rebuilds number fluency without a calculator, teaches algebra as a language, and secures the geometry and statistics basics, with the diagnostic finding the true starting point first.

- Months 1-2: Diagnostic + Number, Bare-Handed (Weeks 1-8)
  - Diagnostic, Board and Tier Fit
    - Topics:
      - Exam board confirmed: AQA, Edexcel or OCR
      - Foundation versus Higher tier fit assessed honestly
      - Non-calculator fluency screen
      - Algebra base measured
      - Number and ratio accuracy audit
      - Honest starting-grade picture
    - Practice: A mixed non-calculator and algebra diagnostic set sized to the suspected tier.
    - Assessment: Baseline grade estimate with a written tier recommendation and a topic-priority map.
    - Detail: Full diagnostic: board confirmed, tier fit assessed honestly, non-calculator fluency and algebra base measured
  - Fractions, Decimals and Percentages to Fluency
    - Topics:
      - The four operations with fractions
      - Decimal calculations without a calculator
      - Percentage of, increase, decrease and reverse
      - Converting between the three forms
      - Estimation and rounding as a check
      - Paper 1 non-calculator territory
    - Practice: A non-calculator FDP set with estimation checks on every answer.
    - Detail: Fractions, decimals and percentages to fluency, Paper 1's exact territory, with estimation as a checking habit
  - Ratio and Proportion Reasoning
    - Topics:
      - Sharing in a given ratio
      - Best-buy and value comparisons
      - Recipe scaling problems
      - Map-scale and unit conversions
      - Direct proportion reasoning
      - Linking ratio to fractions
    - Practice: Ratio and proportion questions across the best-buy, recipe and map-scale families.
    - Detail: Ratio and proportion reasoning: the best-buy, recipe and map-scale families
  - Indices, Standard Form and Bounds
    - Topics:
      - Index laws with integer and negative powers
      - Standard form and its arithmetic
      - Upper and lower bounds
      - Error intervals and truncation
      - Rounding to significant figures
      - Bounds applied in calculations
    - Practice: An indices, standard-form and bounds set including error-interval questions.
    - Detail: Indices, standard form and bounds; error intervals done with meaning
- Months 3-4: Algebra as a Language (Weeks 9-16)
  - Expressions, Expansion and Factorisation
    - Topics:
      - Simplifying and collecting like terms
      - Expanding single and double brackets
      - The area model behind expansion
      - Factorising into brackets
      - Common-factor extraction
      - Algebraic notation discipline
    - Practice: An expand-and-factorise set anchored by area-model diagrams.
    - Detail: Expressions, expansion and factorisation with the area model underneath
  - Linear Equations, Inequalities and Formulae
    - Topics:
      - Solving linear equations step by step
      - Equations with brackets and fractions
      - Solving and representing inequalities
      - Number-line representation of solutions
      - Rearranging formulae to change the subject
      - Reading an equation as a sentence
    - Practice: Linear equation, inequality and rearrangement questions with worded contexts.
    - Detail: Linear equations and inequalities read as sentences; rearranging formulae
  - Straight-Line Graphs and y = mx + c
    - Topics:
      - Plotting straight-line graphs
      - Gradient understood as a rate
      - y = mx + c as intercept and slope
      - Finding the equation of a line

- Parallel and intersecting lines
- Reading real-life graphs critically

Practice: Straight-line graph questions plus interpretation of a real-life rate graph.

Detail: Straight-line graphs: gradient as rate, y = mx + c as a story; real-life graphs read critically

Simultaneous Equations and Sequences

Topics:

- Simultaneous equations by elimination
- Simultaneous equations by substitution
- Choosing a method by structure
- Linear sequences and the nth term
- Finding and using the nth term rule
- Special sequences and patterns

Practice: A simultaneous-equations and nth-term set graded by structure.

Detail: Simultaneous equations by structure; sequences and the nth term

Months 5-6: Shape and Data Foundations (Weeks 17-24)

Angle Reasoning and Justification

Topics:

- Angles on lines and around points
- Parallel-line angle rules
- Interior and exterior angles of polygons
- Bearings and direction problems
- Stating a reason for every step
- Multi-step angle chains

Practice: Angle problems requiring named reasons, including bearings.

Detail: Angle reasoning with justification language: parallel lines, polygons, bearings

Area, Perimeter, Volume and Pythagoras

Topics:

- Area and perimeter of standard shapes
- Compound and composite shapes
- Volume and surface area of solids
- Pythagoras derived from a picture
- Applying Pythagoras in context
- Units for length, area and volume

Practice: A mensuration set combining compound shapes and Pythagoras problems.

Detail: Area, perimeter and volume including compound shapes; Pythagoras from a picture

Averages, Charts and Correlation

Topics:

- Mean, median, mode and range
- Averages from frequency tables
- Bar charts, pie charts and pictograms
- Scatter graphs and lines of best fit
- Correlation and its limits
- Reading data critically

Practice: A statistics set interpreting charts and a scatter graph with commentary.

Detail: Averages, charts, scatter graphs and correlation, data read like a sceptic

Phase 1 Checkpoint Paper

Topics:

- Full Foundation-standard paper under timing
- Mark-scheme review of every question
- Method-mark awareness introduced
- Error log started from the checkpoint
- Timing across a full paper
- Presentation and working discipline

Practice: A complete Foundation-standard paper sat to time, then self-marked.

Assessment: Foundation-standard checkpoint paper marked against the real mark scheme, opening the error log.

Detail: Phase checkpoint: a full Foundation-standard paper under timing, reviewed against the mark scheme

---

PHASE 2: THE HIGHER REACH, WITH SETTLING TIME (MONTHS 7-12)

The topics that separate grade 5 from grade 9, taught months before the exam so they are owned rather than crammed: quadratics and functions, trigonometry beyond right angles, vectors, circle theorems and the algebraic craft the top questions assume.

Months 7-8: Quadratics and Functions (Weeks 25-32)

Solving Quadratics Three Ways

Topics:

- Solving by factorising
- Completing the square
- The quadratic formula
- Choosing the method by structure
- Roots and their meaning
- Higher-tier quadratic manipulation

Practice: A quadratics set requiring choice of factorising, completing the square or the formula.

    Detail: Quadratics by factorising, completing the square and the formula, chosen by structure
  Quadratic Graphs and Turning Points
    Topics:
      - Sketching quadratic graphs
      - Turning points by completing the square
      - Roots, intercepts and symmetry
      - Quadratics in real contexts
      - Quadratic inequalities (Higher)
      - Reading a parabola for information
    Practice: Quadratic-graph questions including a Higher-tier quadratic inequality.
    Detail: Quadratic graphs, turning points and real contexts; quadratic inequalities (Higher)
  Functions: Composite and Inverse
    Topics:
      - Function notation and evaluation
      - Composite functions and order
      - Inverse functions and how to find them
      - Domain and range awareness
      - Common function-mark traps
      - Functions read as meaning
    Practice: A functions set focused on composite and inverse questions.
    Detail: Function notation, composite and inverse functions, where half the function marks are lost
  Graph Transformations and Algebraic Fractions
    Topics:
      - Translations and reflections of graphs
      - Cubic and reciprocal graph shapes
      - Recognising graphs from equations
      - Simplifying algebraic fractions (Higher)
      - Adding and subtracting algebraic fractions
      - Solving equations with algebraic fractions
    Practice: Graph-transformation questions plus a Higher-tier algebraic-fractions set.
    Detail: Graph transformations, cubic and reciprocal graphs; algebraic fractions (Higher)

Months 9-10: Trigonometry, Vectors and Circle Theorems (Weeks 33-40)
  Right-Triangle Trigonometry and Exact Values
    Topics:
      - Sine, cosine and tangent ratios
      - Finding sides and angles in right triangles
      - Choosing the correct ratio
      - Exact trig values for Paper 1
      - Trig in worded and 2D contexts
      - Non-calculator trig fluency
    Practice: Right-triangle trig questions including exact-value non-calculator items.
    Detail: Right-triangle trig consolidated; exact trig values for Paper 1
  Sine and Cosine Rules
    Topics:
      - The sine rule for sides and angles
      - The cosine rule for sides and angles
      - Area of a triangle with the sine formula
      - Bearings and non-right-triangle problems
      - Trig graphs and their features (Higher)
      - Choosing the correct rule
    Practice: A non-right-triangle set applying sine and cosine rules with bearings.
    Detail: Sine and cosine rules with bearings and area applications; trig graphs (Higher)
  Vectors and Geometric Proof
    Topics:
      - Vector notation and column vectors
      - Adding and scaling vectors
      - Position vectors and paths
      - The classic geometric vector proof
      - Proving points collinear or parallel
      - Structuring a vector argument (Higher)
    Practice: Vector questions building to a full geometric vector proof.
    Detail: Vectors and the classic geometric vector proof, defanged (Higher)
  Circle Theorems
    Topics:
      - The main circle theorems
      - Angle at the centre and at the circumference
      - Cyclic quadrilaterals and tangents
      - The alternate segment theorem
      - Proof-quality justification language
      - Multi-theorem angle chains
    Practice: Circle-theorem problems requiring a named theorem for each step.
    Detail: Circle theorems with proof-quality justification

Months 11-12: Probability, Proportion and the Grade 8-9 Craft (Weeks 41-48)

    Probability, Trees and Venn Diagrams
      Topics:
        - Probability of single and combined events
        - Tree diagrams for dependent events
        - Venn diagrams and set notation
        - Conditional probability by reasoning
        - With and without replacement
        - Avoiding formula-panic on conditionals
      Practice: A probability set using tree and Venn diagrams, including a conditional question.
      Detail: Probability with tree and Venn diagrams; conditional probability without formula-panic
    Proportion and Rates of Change
      Topics:
        - Direct proportion as an equation
        - Inverse proportion as an equation
        - Finding the constant of proportionality
        - Rates of change from graphs
        - Gradient of a curve by tangent (Higher)
        - Pre-calculus gradient ideas
      Practice: Proportion questions written as equations plus a rate-of-change graph task.
      Detail: Direct and inverse proportion as equations; rates of change and pre-calculus gradient ideas (Higher)
    Histograms and Cumulative Frequency
      Topics:
        - Histograms with unequal class widths
        - Frequency density
        - Cumulative frequency curves
        - Median, quartiles and interquartile range
        - Box plots and comparing distributions
        - Interpreting data like a statistician
      Practice: A set drawing and reading histograms, cumulative frequency curves and box plots.
      Detail: Histograms, cumulative frequency and comparing distributions like a statistician
    Phase 2 Higher-Tier Checkpoint
      Topics:
        - Full Higher-tier past-paper cycle
        - Mark-scheme review of long questions
        - Method-mark harvesting practice
        - Error-log update from the cycle
        - Timing across a Higher paper
        - Weak-topic list refreshed
      Practice: A full Higher-tier past paper sat to time and reviewed against the mark scheme.
      Assessment: Higher-tier past-paper cycle marked to the mark scheme, updating the error log and weak-topic list.
      Detail: Phase checkpoint: full Higher-tier past-paper cycle with mark-scheme review and error-log update

---
PHASE 3: PAPER CRAFT TO THE EXAM SERIES (MONTHS 13-18)

Three papers, one strictly non-calculator, sat over exam season: this phase turns understanding into grades with board-exact past-paper cycles, personal timing plans, mark-banking discipline and a peak plan that meets the series calmly.

  Months 13-14: The Three-Paper System (Weeks 49-56)
    Paper 1 Craft, Non-Calculator
      Topics:
        - Non-calculator discipline under time
        - Exact values for surds and trig
        - Clean written working at speed
        - Common Paper 1 question types
        - Banking marks on short questions
        - Order of attack on Paper 1
      Practice: Timed Paper 1 sections focused on non-calculator accuracy and clean working.
      Detail: Paper 1 craft: non-calculator discipline, exact values and clean written working at speed
    Papers 2 and 3 Calculator Craft
      Topics:
        - Calculator use that saves time
        - Multi-step problem chains
        - Method-mark harvesting on hard questions
        - Efficient calculator functions
        - Presenting working on calculator papers
        - Recovering marks when stuck
      Practice: Timed Paper 2 and 3 sections practising method-mark harvesting on unfinished problems.

Detail: Papers 2-3 craft: calculator use that helps, multi-step chains, and method-mark harvesting on problems you cannot finish

Months 15-16: Full Cycles by Board (Weeks 57-64)
Full Board-Exact Paper Cycles
Topics:
- Full timed papers from the exact board
- Tier-correct paper selection
- A review ritual after every paper
- Naming why each mark was lost
- A named fix for each error
- Board-specific question styles
Practice: Complete board-exact papers sat to time, each followed by a why-and-fix review.
Detail: Full timed papers from your exact board and tier; review rituals, why each mark was lost, and the named fix
Timing Plans and Weak-Topic Rotations
Topics:
- Personal timing plans from paper data
- Per-question time budgets
- Weakest-topic rotations by error-log priority
- Grade-boundary awareness used honestly
- Targeting the highest-value gaps
- Consistency across papers
Practice: Weak-topic rotation sets chosen by error-log priority, timed to the personal plan.
Detail: Timing plans from personal data; weakest-topic rotations by error-log priority; grade-boundary awareness used honestly

Months 17-18: The Series, Met Calmly (Weeks 65-72)
Final Cycles and Error-Log Closure
Topics:
- Final full paper cycles
- Documented score progression
- Error-log closure
- Targeted revision to the mocks calendar
- Sealing recurring mistakes
- Confidence on previously weak topics
Practice: Final paper cycles with progression charted and each closed error re-tested.
Detail: Final cycles with documented progression; error-log closure; targeted revision that respects the mocks calendar
Peak Plan and Exam-Day Routines
Topics:
- Light consolidation in the peak week
- Rest discipline before the series
- Exam-day routines rehearsed
- Routines across all three papers
- Calm pacing under real conditions
- The series met as a known place
Practice: Light consolidation tasks and a rehearsed exam-day routine across the three papers.
Assessment: Final full-series rehearsal across all three papers with documented progression before the exam.
Detail: Peak plan: light consolidations, rest discipline, and exam-day routines rehearsed across all three papers

PROJECTS THE STUDENT BUILDS AND KEEPS

- Personal error log and syllabus map across all 18 months
- Non-calculator fluency system built through daily-dose drills
- Self-graded past-paper portfolio against your board's real mark schemes
- Personal timing plans for all three papers
- Full timed cycles with documented score progression

Total projects: 5+ exam artifacts plus the student's own handbooks

SKILLS THE STUDENT WILL HAVE AT THE END

- Calculator-free number fluency
- Algebra as a language: equations, formulae, graphs
- Higher-tier functions, trigonometry and vectors
- Circle theorems and geometric justification
- Probability and statistics at exam depth
- Proportion and rates-of-change craft
- Mark-scheme literacy and working discipline
- Three-paper timing strategy

HOW A TYPICAL WEEK IS SPLIT

  Live classes: 2 one-hour live classes with your mentor
  Past paper work: Weekly question sets from your exact board
  Fluency: Non-calculator warm-up in every class
  Total per week: About 4 focused hours

SUPPORT INCLUDED

  Live sessions: 8 live one-hour classes every month, 2 per week
  Mentorship: One dedicated mentor across the whole GCSE run
  Recordings: Every class recorded for revision
  Progress tracking: Paper-score progression shared with parents
  Flexibility: Intensity adjusts around mocks and the exam series

CERTIFICATION DETAIL

  Final certificate: Course completion certificate from Modern Age Coders
  Score record: Documented paper-score progression
  Portfolio projects: Past-paper portfolio and personal handbooks

WHERE THIS CAN LEAD

  - A GCSE grade that opens sixth-form and apprenticeship doors
  - A-Level Mathematics readiness for Higher-tier students
  - The numeracy base every career pathway quietly requires
  - Ready for IB or A-Level tracks, or competition mathematics

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: Which exam boards does this course cover?
  A: AQA, Edexcel and OCR, the content overlaps heavily, but paper styles and mark schemes
  differ, so the first thing the mentor confirms is your school's board, and every past paper
  comes from it. The 9-1 grading system and three-paper structure (one non-calculator) frame
  all exam work.

  Q: Foundation or Higher, how should the decision be made?
  A: By destination, then evidence. Foundation caps the grade at 5, so if A-Level maths or a
  competitive sixth form is even possible, Higher is the default. Whether the gaps blocking
  Higher are repairable in the time available is a diagnostic question, and the free demo
  answers it with evidence you can take to school. (Agent note: the free demo is a waiting list
  with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at
  https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: My child's grades slide every mock as content deepens. Why?
  A: That slide is the classic signature of procedure without understanding: memorized methods
  hold at grade 4-5 content and collapse when questions chain or re-frame. Understanding-first
  teaching is the fix, algebra as a language, geometry with justification, which is exactly how
  this course teaches.

  Q: Can you take a grade 6 student to grade 8-9?
  A: All are taught directly in Phases 2-3. The honest caveat: the earlier the start, the more
  settling time the grade 8-9 topics get.

  Q: Do you take resit candidates?
  A: Yes, post-16 resit students get a compressed, evidence-led version of the program
  prioritised by their diagnostic, aimed at the grade 4/5 threshold or beyond as their goals
  require.

  Q: How does this fit around school mocks and revision?
  A: The mentor plans against your school calendar: intensity rises before mocks, past-paper
  focus aligns with what school is testing, and the final months are sequenced around the real
  series. The course reinforces school rather than competing with it.

  Q: Can we try it before paying?
  A: Yes. Every student starts with a free live demo class that doubles as the diagnostic,
  board, tier, fluency and algebra base, with no card details required.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/gcse-mathematics-mastery
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
COURSE 70 OF 137: GENERATIVE AI COURSE: LLMS, RAG AND AI AGENTS
==

Course link: https://learn.modernagecoders.com/courses/complete-generative-ai-masterclass-college

Subject area: Generative AI
Designed for: College students
Who can join: Complete beginner (18+); Python and the maths are built inside the course
Course length: 10-12 months (40-48 weeks)
Time needed: 2 live classes/week + 5-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Generative AI course, live online: Python to LLM engineering, prompt craft, RAG, fine-tuning, AI agents and production skills in 10-12 months.

  In one line: The modern AI stack, engineered rather than prompted at: LLMs understood, RAG built properly, agents shipped with guardrails, in ten honest months.

ABOUT THE PROGRAMME

  Generative AI created a new engineering discipline in record time, and most courses responded with either hype or screenshots. This masterclass responds with engineering. It starts from zero, Python, the working mathematics, a fast pass through classical ML and neural networks, because LLM engineering without foundations is prompt-guessing with extra steps. Then it goes where the real demand is: transformer architecture understood deeply enough to reason about, LLM APIs across providers, prompt engineering with evaluation harnesses, embeddings and vector search, retrieval-augmented generation built and hardened twice, fine-tuning judgment, open-weight models run locally, and AI agents, architecture, tools, memory and the guardrails that keep them honest. The production phase is what separates this course from tutorials: application architecture, observability, security against prompt injection, cost control, and a capstone shipped behind an API and defended. What it refuses to fake: training frontier models from scratch, research careers and startup playbooks are mapped honestly as next roads, not sold as weeks. The pace is real: ten months at two live classes a week plus five to six hours of practice, with two more in hand when needed. Problem sets after every class, monthly mixed reviews, phase exams, and a final exam that decides the certificate, with focused revision and a free retest. Real systems, real evaluation, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

  - Foundations first, honestly: Python, maths and ML basics built in-course, because prompt-guessing is not engineering
  - RAG built twice: once to work, once to survive real documents, real users and real failure modes
  - Agents with guardrails: architecture, tools and memory, plus the security layer most courses skip
  - Provider-agnostic: OpenAI-class, Claude-class, Gemini-class and open-weight models, patterns over brand loyalty
  - Evaluation harnesses everywhere: generated output scored honestly, never eyeballed
  - Real assessment: weekly problem sets, monthly mixed reviews, and a final exam that decides the certificate, with focused revision and a free retest

THE LEARNING PATH, STEP BY STEP

  Phase 1: Foundations (Months 1-3): Python, working mathematics, classical ML and neural network basics
  Phase 2: LLM core (Months 4-6): transformers in depth, APIs across providers, prompt engineering with evals, embeddings, frameworks and first RAG
  Phase 3: Agents and adaptation (Months 7-8): advanced RAG, agent architecture and tools, fine-tuning judgment
  Phase 4: Production (Months 9-10): open-weight and local models, architecture, observability and security, ethics, and the shipped capstone

WHAT A STUDENT NEEDS BEFORE STARTING

  Coding experience: None required; Python is built in phase 1. Existing developers move faster and go deeper
  Maths background: School mathematics; the working maths is rebuilt in month 2
  Equipment: A normal laptop with a stable internet connection; free tiers and cloud notebooks cover the coursework
  Age: 18+; college students, graduates and working professionals
  Time commitment: 2 live classes plus 5-6 hours of practice a week

WHO THIS COURSE IS FOR

  Career movers: Professionals aiming at the LLM-engineering job family, the fastest-growing in

software
Developers: Software engineers adding the GenAI layer properly, past the tutorial plateau
Builders: Founders and makers who want their AI product built on engineering, not vibes
Analysts and pms: Technical-adjacent professionals who need to reason about GenAI systems credibly
Serious beginners: Newcomers ready to earn the foundations that make the modern layer real

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS (MONTHS 1-3, WEEKS 1-12)

Python, the working mathematics, and enough real ML and neural-network understanding that everything generative later makes sense from the inside.

Month 1: Python for AI Engineering (Weeks 1-4)
Python Fundamentals & Development Setup
Topics:
- What is Artificial Intelligence? History and evolution
- Types of AI: Narrow AI, General AI, Superintelligence
- Machine Learning vs Deep Learning vs Generative AI
- Generative AI revolution: GPT, current image APIs, Stable Diffusion
- AI career paths and opportunities in 2026 and beyond
- Why Python for AI? Python ecosystem overview
- Installing Python (3.10+) and setting up environment
- Development environments: Jupyter Notebook, VS Code, Google Colab
- Python basics: variables, data types, operators
- Control structures: if-else, loops (for, while)
- Functions: definition, parameters, return values
- Python data structures: lists, tuples, dictionaries, sets
- List comprehensions and dictionary comprehensions
- File handling: reading and writing files
- Error handling: try-except blocks
- Python modules and packages
- Virtual environments with venv/conda
Projects built:
- Text-based calculator program
- Todo list application
- File organizer script
- Simple data analyzer from CSV
- Web scraper basics with requests
- Command-line application
- Personal expense tracker
- Password generator and validator
Practice: Daily: 45 min Python practice, solve 10-15 problems on HackerRank/LeetCode
Advanced Python & Data Manipulation
Topics:
- Object-Oriented Programming in Python
- Classes and objects: __init__, self
- Inheritance and polymorphism
- Magic methods: __str__, __repr__, __len__
- Decorators and their usage
- Lambda functions and functional programming
- Map, filter, reduce functions
- Generators and iterators
- Context managers and 'with' statement
- Regular expressions for text processing
- Working with JSON and CSV data
- Introduction to NumPy: arrays and operations
- NumPy array operations, broadcasting, indexing
- Introduction to Pandas: DataFrames and Series
- Data loading, cleaning, and exploration with Pandas
- Data visualization basics with Matplotlib
- Plotting: line plots, bar charts, scatter plots, histograms
Projects built:
- Data analysis pipeline for CSV files
- Student grade analyzer with Pandas
- Weather data visualizer
- E-commerce sales dashboard
- Stock price analyzer and plotter
- Text file analyzer with regex
- Custom data class library
- API wrapper class with error handling
Practice: Solve 30 Python problems, analyze 5 real-world datasets
Assessment: Month 1 check: a Python data task live, plus a take-home mixing everything so far

Month 2: The Working Mathematics (Weeks 5-8)
  Mathematics for AI - Part 1
    Topics:
      - Why math matters for AI and ML
      - Linear Algebra: vectors and vector operations
      - Dot product and cross product
      - Matrices and matrix operations
      - Matrix multiplication and transposition
      - Identity matrix and inverse matrix
      - Eigenvalues and eigenvectors (intuition)
      - Calculus: functions and limits
      - Derivatives: concepts and rules
      - Chain rule and partial derivatives
      - Gradient: direction of steepest ascent
      - Gradient descent algorithm basics
      - Multivariable calculus for neural networks
      - Probability theory: basics and terminology
      - Probability distributions: uniform, normal (Gaussian)
      - Expected value and variance
      - Bayes' theorem and conditional probability
    Projects built:
      - Vector operations visualizer
      - Matrix calculator with NumPy
      - Gradient descent implementation from scratch
      - Probability distribution plotter
      - Coin flip simulator with statistics
      - Linear regression using gradient descent
      - Simple optimization problems solver
      - Derivative calculator for basic functions
    Practice: Solve 40 math problems relevant to ML, implement algorithms
  Mathematics for AI - Part 2 & Statistics
    Topics:
      - Statistics for data science
      - Descriptive statistics: mean, median, mode
      - Variance and standard deviation
      - Correlation and covariance
      - Hypothesis testing basics
      - P-values and statistical significance
      - Central Limit Theorem
      - Confidence intervals
      - Linear regression: mathematical foundation
      - Cost functions and loss functions
      - Optimization: local minima, global minima
      - Convex vs non-convex functions
      - Introduction to information theory
      - Entropy and cross-entropy
      - KL divergence
      - Dimensionality reduction concepts
      - Principal Component Analysis (PCA) intuition
    Projects built:
      - Statistical analyzer for datasets
      - Hypothesis testing tool
      - Correlation matrix visualizer
      - Simple linear regression implementation
      - Cost function visualizer
      - PCA implementation from scratch
      - Data distribution analyzer
      - A/B testing simulator
    Practice: Analyze 10 datasets statistically, implement core algorithms
    Assessment: Month 2 check: vectors, gradients and probability in code, plus mixed
    review

Month 3: ML and Neural Networks, the Fast Honest Pass (Weeks 9-12)
  Introduction to Machine Learning
    Topics:
      - What is Machine Learning? Formal definition
      - Types of learning: Supervised, Unsupervised, Reinforcement
      - Machine learning workflow: data, model, evaluation
      - Training, validation, and test sets
      - Overfitting and underfitting
      - Bias-variance tradeoff
      - Introduction to scikit-learn library
      - Linear regression with scikit-learn
      - Logistic regression for classification
      - Decision trees: intuition and implementation
      - Random forests ensemble method
      - K-Nearest Neighbors (KNN) algorithm
      - Support Vector Machines (SVM) basics
      - Naive Bayes classifier

- Model evaluation metrics: accuracy, precision, recall, F1
- Confusion matrix and ROC curves
- Cross-validation techniques

Projects built:
- House price predictor (linear regression)
- Email spam classifier (logistic regression)
- Iris flower classifier (decision trees)
- Handwritten digit recognizer (KNN)
- Customer churn predictor
- Movie recommendation system basics
- Credit card fraud detection
- Student performance predictor

Practice: Build 15 ML models, solve Kaggle beginner competitions

Neural Networks & Deep Learning Basics

Topics:
- From ML to Deep Learning: why neural networks?
- Perceptron: the building block
- Multi-layer perceptron (MLP) architecture
- Activation functions: sigmoid, tanh, ReLU, Leaky ReLU
- Forward propagation step-by-step
- Loss functions: MSE, cross-entropy
- Backpropagation algorithm intuition
- Gradient descent and its variants (SGD, Adam)
- Learning rate and hyperparameters
- Introduction to TensorFlow and Keras
- Building neural networks with Keras Sequential API
- Training and evaluating neural networks
- Regularization techniques: dropout, L1/L2
- Batch normalization
- Early stopping and model checkpoints
- Introduction to PyTorch basics
- PyTorch tensors and autograd
- What generative means: from predicting labels to producing text, images and code

Projects built:
- Neural network from scratch in NumPy
- MNIST digit classifier with Keras
- Fashion MNIST classifier
- Binary classification with neural network
- Multi-class classification problem
- Regression with neural networks
- Simple image classifier
- Neural network hyperparameter tuning

Practice: Build 12 neural networks, compare frameworks

Assessment: Phase 1 exam: train and evaluate a small model live, plus a written mixed paper

---

PHASE 2: THE LLM CORE (MONTHS 4-6, WEEKS 13-24)

Transformers understood deeply enough to reason about, APIs across providers, prompt engineering with evaluation harnesses, embeddings, and RAG built for the first time.

Month 4: Transformers and APIs (Weeks 13-16)

Transformer Architecture Deep Dive

Topics:
- Limitations of RNNs and LSTMs
- Attention mechanism: intuition and math
- Self-attention and multi-head attention
- Positional encoding in transformers
- Transformer architecture: encoder-decoder
- BERT: bidirectional encoder representations
- GPT architecture: decoder-only transformers
- Tokenization: BPE, WordPiece, SentencePiece
- Embedding layers and embedding spaces
- How GPT generates text: autoregressive generation
- Temperature, top-k, top-p sampling
- Model parameters: understanding 7B, 70B, 175B
- Context windows and attention span
- Fine-tuning vs prompt engineering vs RAG
- Pre-training and transfer learning
- Hugging Face Transformers library introduction
- Loading and using pre-trained models

Projects built:
- Attention mechanism visualizer
- Tokenizer comparison tool
- Text generation with Hugging Face
- Sentiment analysis with BERT
- Text classification with transformers
- Named Entity Recognition (NER) system

```
      - Question answering system
      - Text summarization tool
    Practice: Implement attention from scratch, use 10+ transformer models
  LLM APIs Across Providers
    Topics:
      - OpenAI API overview and capabilities
      - Getting API keys and authentication
      - Current OpenAI chat models, and how to choose between them
      - Chat completions API: messages and roles
      - System, user, and assistant messages
      - Completion parameters: temperature, max_tokens, top_p
      - Streaming responses for real-time output
      - Function calling with frontier models
      - JSON mode for structured outputs
      - Vision API: frontier modelsV for image understanding
      - current image APIs API for image generation
      - Whisper API for speech-to-text
      - TTS (Text-to-Speech) API
      - Embeddings API and semantic search
      - Moderation API for content filtering
      - Token counting and cost optimization
      - Rate limiting and error handling
      - Claude and Gemini APIs, and the OpenAI-compatible pattern that unifies the field
    Projects built:
      - ChatGPT clone with OpenAI API
      - AI content generator dashboard
      - Function calling chatbot
      - Image analyzer with GPT-4V
      - AI image generator web app
      - Voice-to-text transcription tool
      - Semantic search engine
      - AI content moderation system
    Practice: Build 15 OpenAI API applications
    Assessment: Month 4 check: the same task solved through two providers, compared
    honestly, plus mixed review

Month 5: Prompt Engineering and Embeddings (Weeks 17-20)
  Advanced Prompt Engineering
    Topics:
      - Prompt engineering as a discipline
      - Anatomy of effective prompts
      - Zero-shot, one-shot, few-shot learning
      - Chain-of-Thought (CoT) prompting
      - Tree of Thoughts (ToT) technique
      - Self-consistency prompting
      - Role prompting and persona design
      - Instruction tuning concepts
      - Prompt templates and variables
      - Output formatting and constraints
      - Handling hallucinations and reducing errors
      - Prompt injection and safety
      - Meta-prompting and prompt generation
      - Multi-turn conversation design
      - Context management in long conversations
      - Prompt optimization and A/B testing
      - Domain-specific prompting strategies
    Projects built:
      - Prompt library and template system
      - Chain-of-Thought problem solver
      - AI tutor with CoT reasoning
      - Code generator with prompts
      - Creative writing assistant
      - Structured data extractor
      - Multi-step task planner
      - Prompt testing and evaluation framework
    Practice: Create 100+ optimized prompts for various use cases
  Embeddings and Vector Databases
    Topics:
      - What are embeddings? Semantic representations
      - Word2Vec, GloVe, and modern embeddings
      - OpenAI embeddings (text-embedding-ada-002)
      - Sentence embeddings and document embeddings
      - Cosine similarity and semantic search
      - Vector databases: purpose and architecture
      - Pinecone: managed vector database
      - Weaviate: open-source vector DB
      - Chroma: simple vector store
      - Qdrant: performance-focused vector DB
      - FAISS: Facebook AI similarity search
```

- Indexing strategies: flat, HNSW, IVF
- Metadata filtering in vector search
- Hybrid search: vector + keyword
- Building semantic search systems
- Recommendation systems with embeddings
- Clustering and visualization of embeddings

Projects built:

- Semantic search engine for documents
- Similar article finder
- Customer support ticket classifier
- Product recommendation system
- Duplicate detection system
- Embedding visualizer (t-SNE/UMAP)
- Multi-language semantic search
- Knowledge base with vector search

Practice: Build 10 vector database applications

Assessment: Month 5 check: build a semantic search over real documents, plus mixed review

Month 6: Frameworks and First RAG (Weeks 21-24)

LangChain Fundamentals

Topics:

- Introduction to LangChain framework
- LangChain architecture and components
- LLMs and Chat Models in LangChain
- Prompt templates and prompt engineering
- Output parsers: structured data from LLMs
- Chains: sequential processing
- LLMChain, SimpleSequentialChain, SequentialChain
- Memory types: ConversationBufferMemory, ConversationSummaryMemory
- Conversation chains with memory
- Document loaders: PDF, CSV, web pages
- Text splitters and chunking strategies
- Vector stores integration in LangChain
- Retrievers for information retrieval
- Combining LLMs with external data
- Question-answering over documents
- LangChain Expression Language (LCEL)
- Debugging and tracing in LangChain
- Framework honesty: when LangChain-style tooling helps, and when the native SDK is cleaner

Projects built:

- Conversational chatbot with memory
- Document Q&A system with LangChain
- PDF analyzer and summarizer
- Web scraper with AI analysis
- Multi-step reasoning chain
- Structured data extraction pipeline
- Conversation summarizer
- Knowledge base chatbot

Practice: Build 15 LangChain applications

Retrieval Augmented Generation (RAG) - Part 1

Topics:

- What is RAG? Why RAG over fine-tuning?
- RAG architecture: retrieval + generation
- Building basic RAG pipeline
- Document preprocessing for RAG
- Chunking strategies: fixed-size, semantic
- Embedding generation for documents
- Vector store population
- Retrieval strategies: similarity search, MMR
- Reranking retrieved documents
- Context compression techniques
- Query transformation and expansion
- Retrieval quality evaluation
- Prompt engineering for RAG
- Handling long contexts
- Multi-query retrieval
- Hypothetical document embeddings (HyDE)
- Parent document retrieval

Projects built:

- Enterprise documentation Q&A system
- Academic paper search and summarization
- Legal document analyzer
- Customer support knowledge base
- Code documentation assistant
- Research paper recommendation system
- Meeting notes Q&A system
- Email archive search with AI

Practice: Build 10 RAG applications for different domains

      Assessment: Phase 2 exam: a working RAG system with an evaluation harness, defended,
      plus a cumulative paper

PHASE 3: AGENTS AND ADAPTATION (MONTHS 7-8, WEEKS 25-32)

RAG hardened for the real world, agent architecture with tools and memory, and fine-tuning
judgment: what it changes, what it costs, when it is wrong.

  Month 7: Hardened RAG and First Agents (Weeks 25-28)
    RAG for the Real World
      Topics:
        - Advanced RAG architectures
        - Self-RAG: self-reflective retrieval
        - Corrective RAG (CRAG)
        - Agentic RAG with LangGraph
        - Multi-document RAG systems
        - Hierarchical retrieval strategies
        - Graph-based RAG
        - Conversational RAG with memory
        - RAG with structured data (tables, charts)
        - Multimodal RAG: text + images
        - RAG evaluation metrics: faithfulness, relevance
        - RAGAS framework for evaluation
        - RAG optimization techniques
        - Caching strategies for RAG
        - Production RAG deployment
        - RAG monitoring and observability
        - Security and privacy in RAG systems
      Projects built:
        - Advanced enterprise RAG system
        - Multi-source RAG pipeline
        - Conversational RAG with context
        - Evaluation framework for RAG
        - Self-improving RAG system
        - Multimodal document understanding
        - RAG-powered coding assistant
        - Production-ready RAG API
      Practice: Build and optimize 8 advanced RAG systems
    AI Agents Architecture - Part 1
      Topics:
        - What are AI agents? Autonomous systems
        - Agent types: simple reflex, goal-based, utility-based
        - Perceive-Think-Act loop
        - ReAct: reasoning and acting
        - Plan-and-Execute agents
        - Cognitive architectures for agents
        - Agent memory systems
        - Short-term vs long-term memory
        - Semantic memory and episodic memory
        - Working memory implementation
        - Tool use and function calling
        - Multi-step reasoning with agents
        - Error handling and recovery
        - Agent observability and logging
        - LangGraph for agent workflows
        - State machines for agents
        - Human-in-the-loop agents
      Projects built:
        - Autonomous research agent
        - Data analysis agent
        - Code generation and debugging agent
        - Travel planning agent
        - Email management agent
        - Content creation agent
        - Personal productivity agent
        - Multi-step task automation agent
      Practice: Build 12 AI agent systems
      Assessment: Month 7 check: an agent that uses two tools reliably, plus mixed review

  Month 8: Agent Systems and Fine-Tuning (Weeks 29-32)
    Agent Systems: Memory, Planning and Guardrails
      Topics:
        - Multi-agent systems (MAS)
        - Agent communication protocols
        - Collaborative agents vs competitive agents
        - Agent coordination and cooperation
        - Hierarchical agent architectures
        - AutoGPT and BabyAGI patterns

- Task decomposition and planning
- Sub-goal generation
- Agent self-evaluation and reflection
- MetaGPT and role-playing agents
- Specialized agent roles
- Agent orchestration frameworks
- Crew AI for multi-agent systems
- Agent security and sandboxing
- Rate limiting and cost control
- Agent performance optimization
- Debugging complex agent systems

Projects built:
- Multi-agent research team
- Collaborative writing agents
- Software development agent team
- Customer service agent system
- Agent-based simulation
- Hierarchical task planner
- Self-improving agent system
- Production multi-agent platform

Practice: Build 8 multi-agent systems

Fine-tuning Fundamentals

Topics:
- What is fine-tuning? When to fine-tune?
- Fine-tuning vs prompt engineering vs RAG
- Types of fine-tuning: full, parameter-efficient
- Transfer learning for LLMs
- Dataset preparation for fine-tuning
- Data annotation and labeling
- Instruction tuning datasets
- Supervised fine-tuning (SFT)
- Training loops and optimization
- Loss functions for language models
- Hyperparameter tuning
- Overfitting prevention in fine-tuning
- Evaluation metrics for fine-tuned models
- Benchmark datasets and tasks
- OpenAI fine-tuning API
- Fine-tuning current OpenAI models
- Cost considerations for fine-tuning

Projects built:
- Fine-tune a current OpenAI model for customer support
- Domain-specific chatbot fine-tuning
- Code completion model fine-tuning
- Fine-tuned model for legal documents
- Medical terminology fine-tuning
- Custom classifier with fine-tuning
- Sentiment analysis fine-tuned model
- Fine-tuning evaluation framework

Practice: Fine-tune 8 models for different domains

Assessment: Phase 3 exam: a guarded agent system defended end to end, plus a cumulative written paper

---

PHASE 4: PRODUCTION AND THE CAPSTONE (MONTHS 9-10, WEEKS 33-40)

Open-weight models run locally, production architecture with observability and security, responsible AI, and a capstone shipped behind an API and defended.

Month 9: Open Weights and Production Architecture (Weeks 33-36)

Open Source LLMs & Local Deployment

Topics:
- Open source LLM landscape
- Llama 2 and Llama 3 models
- Mistral and Mixtral models
- Falcon, MPT, and BLOOM
- Model cards and model documentation
- Downloading models from Hugging Face Hub
- Model quantization: GGUF, GPTQ, AWQ
- Running LLMs locally with Ollama
- llama.cpp for efficient inference
- vLLM for production serving
- Text Generation Inference (TGI)
- Model serving with FastAPI
- GPU vs CPU inference
- Batch inference optimization
- Caching and KV-cache optimization
- Model benchmarking: speed and quality
- Building custom inference servers

- Multimodal models in one honest lesson: vision-language and where it fits
Projects built:
- Local LLM deployment with Ollama
- Quantized model serving API
- Multi-model inference server
- LLM benchmarking suite
- Optimized inference pipeline
- Custom model hosting solution
- Edge deployment of LLMs
- Inference cost calculator
Practice: Deploy and optimize 10 open source models
Production Architecture, Observability and Security
Topics:
- Designing AI-powered applications
- Microservices architecture for AI
- FastAPI for ML/AI APIs
- Request validation and error handling
- Asynchronous processing with Celery
- Message queues: RabbitMQ, Redis
- Background tasks for AI workloads
- Caching strategies for AI apps
- Redis for response caching
- Database integration: PostgreSQL, MongoDB
- Storing embeddings in databases
- User authentication and authorization
- API rate limiting and throttling
- Webhook integrations
- Building SDKs for AI APIs
- API documentation with OpenAPI
- Postman and API testing
- LLM observability: tracing, token accounting and quality dashboards
- Security: prompt injection, data exfiltration and the guardrail stack
Projects built:
- Production AI API with FastAPI
- Asynchronous AI processing system
- Cached AI inference service
- Multi-user AI application
- AI API with usage tracking
- Webhook-based AI integration
- Client SDK for AI service
- Complete AI backend system
Practice: Build 10 production-ready AI APIs
Assessment: Month 9 check: a production-readiness review of your own system, plus mixed review

Month 10: Ethics, Capstone and the Final Exam (Weeks 37-40)
Responsible GenAI
Topics:
- AI ethics fundamentals
- Bias in AI: sources and mitigation
- Fairness metrics and evaluation
- Transparency and explainability
- LIME and SHAP for model interpretation
- Privacy in AI: data protection
- Differential privacy
- Federated learning basics
- AI safety and alignment
- Constitutional AI principles
- Red teaming and adversarial testing
- Content moderation and filtering
- Deepfakes: detection and prevention
- AI regulations: GDPR, AI Act
- Responsible AI deployment
- Environmental impact of AI
- AI for social good
- Capstone planning: choosing a system worth defending
Projects built:
- Bias detection tool for models
- Explainable AI dashboard
- Content moderation system
- Deepfake detector
- Privacy-preserving AI application
- AI impact assessment tool
- Responsible AI checklist generator
Practice: Evaluate and improve fairness in all projects
Capstone Build
Topics:
- Two focused weeks: a complete GenAI system, RAG, agent or hybrid
- Instructor checkpoints and honest scope cuts

          - The evaluation harness built before the excitement
          - Shipped behind an API with observability and guardrails
          - A system card a skeptic could audit
        Projects built:
          - The capstone: a deployed, evaluated, guarded GenAI system
        Practice: Daily build log; a working deployed system at all times
      Demo Day and the Final Exam
        Topics:
          - Presenting the capstone: live system plus the hardest failure story
          - A full-course spiral review before the exam
          - The honest map of next roads: enterprise agents, coding agents, deep ML and research
          - Where our Copilot Studio and Codex/Claude Code courses continue the applied layer
        Projects built:
          - Demo day presentation delivered to the batch
        Assessment: FINAL EXAM: a practical GenAI engineering task plus a written paper with
        questions mixed from every phase. Passing earns the certificate; falling short earns a
        focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Phase 1: Python and maths problem sets, a first trained and evaluated model
  - Phase 2: provider-compared LLM integrations, prompt suites with evaluation harnesses,
    semantic search, and a first working RAG
  - Phase 3: a hardened RAG over messy real documents, tool-using agents with memory and
    guardrails, a fine-tuning study
  - Phase 4: an open-weight model run locally, a production-readiness review, and the deployed
    capstone

  Total projects: 20+ working GenAI builds, crowned by a deployed, evaluated capstone system

SKILLS THE STUDENT WILL HAVE AT THE END

  - Foundations: Python, working mathematics and honest ML basics
  - LLM engineering: transformer understanding, multi-provider APIs, prompt craft with evals
  - Retrieval: embeddings, vector search and RAG built and hardened twice
  - Agents: architecture, tools, memory, planning and guardrails
  - Adaptation: fine-tuning judgment and open-weight local deployment
  - Production: architecture, observability, security and cost control
  - Ethics: responsible deployment argued, not recited

HOW A TYPICAL WEEK IS SPLIT

  Live classes: 2 live one-hour classes per week, building real systems together
  Homework: A problem set after every class, and a mixed-review assignment at the end of every
    month
  Practice: 5-6 hours of system-building practice between classes
  Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

  Doubt support: WhatsApp doubt support between classes, for the retrieval that returns
    nonsense at midnight
  Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the
    plan for the wobble

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short
    and there is focused revision and a free retest: the certificate certifies knowledge, not
    attendance

WHERE THIS CAN LEAD

  - LLM and AI engineering roles: this portfolio is what those interviews want to see
  - AI Agents with Copilot Studio: the enterprise, low-code agent layer
  - Codex and Claude Code: agentic coding as a daily working method
  - Our AI/ML masterclass: the deep-model road, entered with the applied layer already owned
  - Product work: GenAI features shipped inside existing products, immediately

EARNING POTENTIAL (share only if asked, and never promise it)

  Market context: The ranges below are general market salary bands for these roles in India and
    abroad, drawn from public industry data. They are shown for career context only and are not a
    promise or guarantee of income. Actual pay depends on your skills, experience, location, and
    the job market.
  Early career roles: Rs 4-7 LPA (Junior AI Developer / Prompt Engineer)
  Growing roles: Rs 8-15 LPA (AI Application Developer / LLM Engineer); Rs 15-25 LPA (AI
    Engineer / Senior LLM Engineer)
  Experienced roles: Rs 20-40 LPA (Senior AI Engineer / AI Architect / AI Lead); Rs 30-70 LPA

(Lead AI Engineer / Principal Engineer)
Freelance: Rs 2,000-10,000/hour based on expertise and specialization
International usa: $120k-300k USD (AI Engineer to Senior/Staff level)
International europe: €70k-180k EUR based on country and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: How long does it take to become good at generative AI?
A: Ten months at 2 live classes a week plus 5 to 6 hours of practice, from zero, and that includes the foundations most GenAI courses skip. The course keeps two more months in hand for learners who need them.

Q: How am I assessed? Is the certificate just for attending?
A: No. Problem sets follow every class with written feedback, monthly reviews mix new material with earlier topics deliberately, each phase ends with an exam, and the course ends with a final exam: a practical GenAI engineering task plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Do I need to know Python or machine learning already?
A: No. Phase 1 builds Python, the working mathematics and honest ML basics precisely so that the generative phases rest on understanding rather than copy-paste. Existing developers skip nothing but move faster, and go deeper in the practice work.

Q: Which models and tools does the course use?
A: All the ones that matter, by design: frontier APIs (GPT-class, Claude-class, Gemini-class), open-weight models like the Llama family run locally, vector databases, and framework tooling with an honest verdict on when it helps versus native SDKs. The patterns are the syllabus; providers are examples, so the skills survive the next model release.

Q: Does the course cover AI agents?
A: As a core phase, not a teaser: agent architecture, tool use, memory, planning, failure modes and the guardrail stack, built into working systems and then hardened in the production phase. Students who want the enterprise low-code layer continue into our Copilot Studio courses; those who want agentic coding continue into Codex and Claude Code.

Q: Will I train my own large language model?
A: You will fine-tune and run open-weight models, and understand training deeply enough to reason about it. Training frontier models from scratch costs millions and is honestly mapped as a research-career road, not sold as a course week. What employers actually need, adaptation, retrieval, agents and production judgment, is exactly what the course builds.

Q: Can I try a class before enrolling?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. You will build a small working GenAI piece inside the demo itself. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor reading your actual code and traces, never pre-recorded videos. GenAI systems fail in quiet, expensive ways; live review is the only honest way to build the judgment that catches them.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/complete-generative-ai-masterclass-college
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 71 OF 137: GIT & GITHUB COURSE FOR COLLEGE STUDENTS: TEAM-READY SKILLS
==========

Course link: https://learn.modernagecoders.com/courses/git-github-version-control-course-for-college-students
Subject area: Version Control & Developer Tools
Designed for: College students
Who can join: Beginner to job-ready collaborator (college students and recent graduates)
Course length: 24 classes (12 weeks · 2 classes/week)
Time needed: 4-5 hours/week recommended
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a

month. Each class is one hour.

WHAT THIS COURSE IS

Master Git and GitHub in a live 24-class course for college students: team branching workflows, pull requests and code review, GitHub Actions CI, open source contribution and Git interview prep for internships and placements.

In one line: Every internship, hackathon, and placement interview touches Git. Learn to work like an engineer on a team, and make your GitHub profile do the talking.

ABOUT THE PROGRAMME

Most college students can type git add and git commit, and that is where their Git knowledge ends. Then the first hackathon, group assignment, or internship arrives and everything falls apart: overwritten work, merge conflicts nobody can resolve, a single main branch with fifty messy commits, and a GitHub profile that tells recruiters nothing. This course closes that gap in 24 live, instructor-led classes. You start with version control fundamentals and command-line fluency, master the full undo toolkit, then move to the skills placements actually test: SSH and remotes, professional READMEs, a portfolio on GitHub Pages, the feature-branch workflow, merge vs rebase, pull requests with real code review, GitHub Actions CI with branch protection, open source contribution including Hacktoberfest and GSoC preparation, and the Git interview questions that appear in technical rounds. You finish with a team capstone executed exactly the way an engineering team ships software, plus a full GitHub profile audit, so your commits, pull requests, and green CI checks speak for you before you say a word.

WHAT MAKES THIS COURSE DIFFERENT

- Team-first, not command-first: you do not just memorize git commands, you run the full branch, PR, review, CI loop the way internship teams actually work.
- Live classes with a real instructor, never pre-recorded videos: every session is hands-on, with your terminal open and your questions answered on the spot.
- Career-wired from Class 5: SSH done properly, recruiter-grade READMEs, a profile that survives a 30-second scan, the GitHub Student Developer Pack, and a live portfolio on GitHub Pages.
- Real scenarios, not toy demos: staged three-person merge conflicts, rescue drills with reflog, a git bisect bug hunt, and a full live mock Git interview.
- Open source is in the syllabus: CONTRIBUTING.md etiquette, a first external pull request, and a personal Hacktoberfest and GSoC preparation roadmap.
- Living Syllabus: the curriculum is updated as Git and GitHub ship new features, so you always learn the current tools, not screenshots from three years ago.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop (Windows, Mac, or Linux) with a stable internet connection; no special hardware needed
Coding experience: Basic familiarity with any one programming language (first-year college Python, C, Java, or JavaScript is plenty); this course teaches the workflow around your code, not the language itself
Accounts: A free GitHub account (GitHub's Terms of Service require users to be 13 or older); a college ID or institute email lets you claim the GitHub Student Developer Pack, and we walk you through it
Mindset: Willingness to practice in the terminal between classes; version control is muscle memory, and the drills are designed to build it

WHO THIS COURSE IS FOR

Engineering students: B.Tech, BCA, BSc, and diploma students who can write some code but have never worked on a shared repository like a team
Placement aspirants: Pre-final and final year students who know recruiters will open their GitHub profile and want it to say 'engineer', not 'tutorial follower'
Hackathon teams: Students whose group projects and hackathons keep dissolving into zip files on WhatsApp and overwritten work
Open source aspirants: Anyone targeting a first external pull request, Hacktoberfest, or a serious GSoC application
Recent graduates: Fresh graduates and self-taught coders who skipped Git fundamentals and now feel it in every interview and internship

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS FAST

Go from zero to confident daily Git use: how version control thinks, complete CLI fluency, an undo toolkit you can trust in any situation, and repository hygiene that keeps secrets and junk out of history.

Class 1: How Version Control Thinks: Concepts, Install & First Config
  Topics:
    - The problem Git solves: no more project_final_v2_REALLY_FINAL.zip, safe experiments, and a complete history of who changed what and why
    - Centralized vs distributed version control, and why Git (created by Linus Torvalds in 2005 as a free, open source tool for Linux kernel development) is the system you should expect to meet at internships, placements, and beyond
    - The core mental model: repository, commit, branch, remote, and how Git stores snapshots of your whole project rather than just file differences
    - The three areas every command moves work between: working directory, staging area (the index), and the repository
    - Installing Git on Windows (with Git Bash), macOS, and Linux, and verifying with git --version
    - First-time setup: git config --global user.name and user.email (the identity stamped on every commit you will ever make), init.defaultBranch main, and core.editor
    - Reading your setup back with git config --list --show-origin, and understanding system vs global vs local config levels
  Projects built:
    - Install Git, complete your global configuration, create your first repository with git init, and explore the hidden .git folder to see where history actually lives
  Practice: Run git config --list and explain out loud what each line you set does and where it applies.
  Assessment: Explain the working directory, staging area, and repository to a classmate by narrating one real file's journey through all three.

Class 2: CLI Fluency: init, add, commit, log & diff Like You Mean It
  Topics:
    - Terminal survival kit: pwd, ls, cd, mkdir, and why interviewers and internship teams expect command-line comfort
    - git status as your compass: reading untracked, modified, and staged states without guessing
    - Staging with intent: git add <file>, git add -A, and the professional's git add -p to stage changes hunk by hunk
    - Committing properly: git commit -m for quick saves, git commit opening your editor for detailed messages, and what a commit object actually contains (snapshot, author, timestamp, parent, SHA)
    - Reading history: git log, git log --oneline --graph --all --decorate, and filtering with git log --author and git log -n 5
    - Inspecting change: git diff for unstaged edits, git diff --staged for what will be committed, and git show <sha> for any past commit
    - HEAD and relative references like HEAD~1 and HEAD~3: the addressing system the entire undo toolkit depends on
  Projects built:
    - Build a personal-notes repository with at least 8 meaningful commits, using git add -p at least twice and reading your own history with git log --oneline --graph
  Practice: Complete one full edit, stage, commit cycle while narrating how git status output changes at each step.
  Assessment: Given a repository with mixed staged and unstaged changes, correctly predict the output of git status and git diff --staged before running them.

Class 3: The Full Undo Toolkit: restore, reset, revert & When Each Is Safe
  Topics:
    - Undo an unstaged edit with git restore <file>, and unstage without losing work with git restore --staged <file>
    - Fix the last commit with git commit --amend for a better message or a forgotten file, and why amend rewrites the commit SHA
    - git reset --soft HEAD~1: move the branch pointer back but keep everything staged, the 'redo my commit' move
    - git reset --mixed HEAD~1 (the default): also unstage, keeping all changes safe in your working directory
    - git reset --hard HEAD~1: discard changes completely, the rare cases where it is justified, and why you type it slowly
    - git revert <sha>: a new commit that undoes an old one, the ONLY safe undo on history you have already pushed and shared
    - The decision table: private unpushed history means reset and amend freely; shared pushed history means revert only, and what a force push does to your teammates
  Projects built:
    - Run a scripted rescue drill: amend a bad message, soft-reset a premature commit, hard-reset a failed experiment, and revert a bad commit that was already pushed to a practice remote
  Practice: For five written scenarios, state which undo command you would use and why it is safe in that exact situation.
  Assessment: Correctly choose between reset and revert for shared vs private history across a set of real situations, losing zero work in the lab.

Class 4: .gitignore for Real Stacks, Secrets & Commit Craft
  Topics:
    - .gitignore syntax that matters: literal names, *.log wildcards, dir/ trailing slashes, ! negation, and how rule order works
    - What real stacks must ignore: node_modules/ and dist/ for JavaScript, venv/ and __pycache__/ for Python, .DS_Store, and IDE folders like .idea/ and .vscode/

- Secrets discipline: .env files never enter history; what to do the moment a key IS committed (rotate the key first, then clean up), and why deleting the file in a later commit does not remove it from history
- Untracking without deleting: git rm --cached <file> for files that were committed before the ignore rule existed
- A machine-wide ignore file: git config --global core.excludesFile ~/.gitignore_global for editor junk on every project
- Commit message craft: imperative subject around 50 characters, blank line, body explaining why, the style engineering teams and open source maintainers expect
- Atomic commits: one logical change per commit, and searching history later with git log --grep and the pickaxe git log -S 'functionName'

Projects built:
- Take a messy starter project with node_modules, a fake .env, and editor junk already tracked; write a production-grade .gitignore, untrack the offenders with git rm --cached, and re-commit cleanly

Practice: Write .gitignore rules for a Python project and a Node project from memory, then verify both with git status.

Assessment: Your cleaned repository shows no generated files or secrets in git status, and your last five commit messages pass the message-quality checklist.

PHASE 2: GITHUB FOR YOUR CAREER

Turn GitHub into a career asset: SSH and remotes done properly, READMEs and licenses recruiters respect, a profile that survives a 30-second scan, the Student Developer Pack claimed, and a live portfolio on GitHub Pages.

Class 5: GitHub, SSH Keys & Your First Push

Topics:
- Git vs GitHub: the tool vs the hosting platform (owned by Microsoft since 2018), and what GitHub Free includes: unlimited public and private repositories and 2,000 GitHub Actions minutes per month
- Creating an account that will carry your career: choosing a username recruiters will read, and GitHub's Terms of Service age requirement of 13 or older
- Authentication today: passwords do not work for pushing; SSH keys vs HTTPS with personal access tokens, and when each fits
- Generating a key: ssh-keygen -t ed25519 -C 'you@college.edu', adding it to ssh-agent, and pasting the public key into GitHub Settings under SSH and GPG keys
- Verifying the handshake with ssh -T git@github.com and reading what the greeting message confirms
- Wiring local to remote: git remote add origin git@github.com:you/repo.git, then git push -u origin main, and what the -u upstream flag saves you forever after
- Reading a repository page like a developer: commits, branches, browsing files at any point in history, and the Insights tab

Projects built:
- Create or clean up your GitHub account, set up ed25519 SSH authentication end to end, and push your Class 4 repository with git push -u origin main

Practice: Break and re-verify your SSH setup once, so you can debug 'Permission denied (publickey)' on your own next time.

Assessment: Your local repository pushes to GitHub over SSH with no password prompt, and the commit history renders correctly online.

Class 6: Clone, Fetch, Pull & Speaking Remote Fluently

Topics:
- git clone <url>: what you actually get (full history, an origin remote, a tracked default branch), and cloning over SSH vs HTTPS
- Remote-tracking branches: what origin/main really is, and why git status says 'ahead 2, behind 1'
- git fetch: download remote changes WITHOUT touching your work, then inspect the gap with git log main..origin/main
- git pull as fetch plus merge, git pull --rebase for linear history, and setting pull.rebase true the way many teams do
- Push rejections decoded: 'non-fast-forward' means fetch first, integrate, then push; force-pushing a shared branch is never the reflex answer
- Working from two machines (hostel laptop and lab computer): the pull-before-work, push-before-leave rhythm that prevents divergence
- Remote management: git remote -v, git remote show origin, and renaming or removing remotes safely

Projects built:
- Simulate a two-computer workflow using two local clones of your own repository: make diverging changes, inspect with git fetch, reconcile with git pull --rebase, and push

Practice: Trigger one rejected push on purpose and resolve it with fetch and rebase instead of a force push.

Assessment: Explain the exact difference between fetch and pull, then demonstrate both live on a repository with pending remote changes.

Class 7: READMEs & Licenses: Repos Recruiters Actually Read

Topics:
- What a recruiter or interviewer does in 30 seconds on your repository: README first, commit

history second, code third, which makes the README your landing page
- Professional README structure: one-line pitch, screenshot or GIF, features, tech stack, setup steps that actually work, usage, and what you would improve next
- Markdown fluency: headings, code fences with language tags, tables, task lists, images, and status badges
- Naming and describing repositories: expense-tracker-django beats project2-final; filling the description field and topics so your work is searchable
- Licenses in plain language: MIT (permissive, common for student projects), Apache-2.0 (adds a patent clause), GPL-3.0 (copyleft), and what 'no license' legally means for your code
- Adding a LICENSE file through GitHub's license chooser, and when a college project should have one
- Cleaning your existing account: archiving dead experiments, deleting empty repos, and making 3 repositories excellent instead of 20 mediocre

Projects built:
- Rewrite the README of your best existing project to the professional template, add a screenshot and a suitable license, and complete the repo description and topics

Practice: Review a classmate's README against the recruiter checklist and give two concrete, actionable improvements.
Assessment: A stranger can clone and run your project using only your README, verified by a classmate actually doing it.

Class 8: Profile README, Pinned Repos & the Student Developer Pack
Topics:
- The username/username repository: GitHub renders that repo's README at the top of your profile page, your one free billboard for recruiters
- What belongs in a profile README: who you are, what you build, current focus, top projects with links, and how to reach you; what to skip (walls of badges and fake stats)
- Pinning your best 6 repositories and choosing them for range: a web app, a tool, a contribution, not six versions of one tutorial
- The contribution graph: what actually counts toward it, why consistent real work beats gaming it, and how interviewers read it
- The GitHub Student Developer Pack: free for verified students enrolled at a degree or diploma granting school, and it includes GitHub Pro while you are a student; claim it at education.github.com with your college ID or institute email
- GitHub Copilot for students: Copilot has a free tier, and GitHub Education has offered free Copilot access to verified students; availability can change, so always check education.github.com
- Your public identity checklist: consistent handle, photo, a bio with your college and interests, location, and a pinned portfolio link

Projects built:
- Ship your profile README in the username/username repository, pin your best repositories, complete your bio, and submit your Student Developer Pack verification

Practice: Study three impressive real student profiles and write down one idea to borrow from each.
Assessment: Your profile passes the class rubric: profile README live, three or more pinned repos with descriptions, complete bio, and Pack verification submitted.

Class 9: Ship Your Portfolio on GitHub Pages
Topics:
- GitHub Pages: static hosting GitHub provides free, publishing files directly out of your repo, and why a live URL beats 'code available on request' on a resume
- User site vs project site: username.github.io serves at the root domain, while every other repository can publish under /repo-name
- Enabling Pages in Settings under Pages: deploy from a branch (main or a /docs folder) vs deploying with a GitHub Actions workflow
- Building a clean single-page portfolio: about, projects with live links and repo links, skills, and contact; plain HTML and CSS is enough
- Debugging deploys: a custom 404.html, relative path gotchas, and why case-sensitive file names break images that worked locally
- Pointing a custom domain at Pages with a CNAME record (optional) and enforcing HTTPS
- The portfolio loop from now on: every future project gets a live demo link in its README and a card on your portfolio site

Projects built:
- Deploy a portfolio site at username.github.io featuring at least three projects with live links, then add the portfolio URL to your GitHub bio and resume

Practice: Push one content update and watch GitHub Pages rebuild and redeploy it live.
Assessment: Your portfolio URL loads publicly, every link resolves, and it is referenced from your GitHub profile.

PHASE 3: TEAM WORKFLOWS

Work the way engineering teams work: feature branches, clean history with merge and rebase, real conflict resolution, pull requests with meaningful review, and issues and Projects boards that keep group assignments and hackathons on track.

Class 10: Branches & the Feature-Branch Workflow
Topics:
- What Git calls a branch: a lightweight named reference to a commit SHA, so creating,

switching, and deleting branches costs nothing
    - Modern commands: git switch -c feature/login, alongside the older git checkout -b you will still see in every tutorial and team
    - The feature-branch workflow: main stays always-working, and every feature, fix, and experiment gets its own branch
    - Branch naming conventions teams use: feature/, fix/, chore/, and docs/ prefixes plus a short descriptive slug
    - Switching branches with uncommitted changes: when Git blocks you, why, and a first look at git stash as the escape
    - Fast-forward merges: git merge feature/login when main has not moved, and reading the result in git log --graph
    - Cleaning up: git branch -d (safe, merged work only) vs git branch -D (force), plus listing with git branch -a and git branch --merged
  Projects built:
    - Build two features of a small app on separate branches (feature/navbar and feature/contact-form), merge both into main, and delete the merged branches
  Practice: Draw your repository's commit graph on paper mid-exercise, then verify it against git log --oneline --graph --all.
  Assessment: Your history shows main receiving two feature branches through merges, with no work ever committed directly to main.

Class 11: Merge vs Rebase & Keeping History Clean
  Topics:
    - The three-way merge: how Git finds the common ancestor and combines both sides into a merge commit
    - git merge --no-ff to force a merge commit that records the feature as one unit, and why some teams mandate it
    - git rebase main from your feature branch: replaying your commits onto a new base for a straight-line history
    - The golden rule of rebase: never rebase commits that others already have; rewriting shared history breaks every teammate's repository
    - Interactive rebase: git rebase -i HEAD~4 to squash WIP commits, reword messages, and reorder work before opening a pull request
    - git merge --squash: landing a messy branch as one clean commit, and how it differs from GitHub's squash-merge button
    - Choosing a team policy: merge-commit history vs linear rebase history, and reading each style fluently in git log --graph
  Projects built:
    - Take a feature branch with five messy WIP commits, use git rebase -i to squash and reword them into two clean commits, then rebase onto an updated main and merge
  Practice: Integrate the same branch in two clone copies, once with merge and once with rebase, and compare the resulting graphs.
  Assessment: Explain to the class when rebase is safe and when it is forbidden, giving one correct example of each.

Class 12: Resolving Merge Conflicts Under Deadline Pressure
  Topics:
    - The anatomy of a conflict: both branches touched identical lines since their common ancestor, so Git stops and makes you decide
    - Reading conflict markers: <<<<<<< HEAD, =======, and >>>>>>> feature, and what each side represents in a merge vs during a rebase
    - Resolving by hand and in VS Code's merge editor: accept current, accept incoming, accept both, or write the correct combination yourself
    - Escape hatches: git merge --abort and git rebase --abort return you to safety, and git rebase --continue moves forward after each fixed commit
    - Taking a whole side deliberately: git checkout --ours <file> and git checkout --theirs, and how their meaning flips during a rebase
    - Conflict prevention as a team habit: pull main into your branch often, keep pull requests small, and split files everyone edits
    - The full drill: conflicting a shared team file, resolving it, staging the resolution, and completing the merge commit
  Projects built:
    - Run a staged three-person conflict lab: all three teammates edit the same function on separate branches, then merge every branch into main resolving each conflict correctly
  Practice: Abort a scary conflicted merge with git merge --abort, then redo it and resolve it for real.
  Assessment: You resolve a two-file conflict with the intended code surviving and no conflict markers left behind, verified with git diff.

Class 13: Pull Requests With Meaningful Descriptions
  Topics:
    - What a pull request is: a request to merge a branch, plus the conversation, review, and checks that professionalize the merge
    - Opening a PR properly: base vs compare branch, a title that summarizes the change, and the shortcut link Git prints after every push
    - The PR description that gets fast reviews: what changed, why, how you tested it, screenshots for UI work, and where reviewers should focus
    - Linking work: 'Closes #12' auto-closes the issue on merge, and referencing related PRs and commits by SHA keeps the trail connected

- Draft PRs for early feedback, requesting specific reviewers, and updating a PR by pushing new commits to the same branch
- GitHub's three merge buttons: create a merge commit vs squash and merge vs rebase and merge, and when a team picks each
- PR templates: a .github/PULL_REQUEST_TEMPLATE.md so every teammate's pull request arrives complete

Projects built:
- Open a real PR on your team repository with a complete description, screenshots, and a linked issue; get it reviewed by a classmate and squash-merge it

Practice: Rewrite a terrible one-line PR description ('fixed stuff') into one a reviewer can actually act on.

Assessment: Your PR history shows a described, linked, reviewed pull request merged with the strategy your team agreed on.

Class 14: Code Review: Giving & Receiving Like a Professional

Topics:
- Why review exists: catching bugs early, spreading knowledge, and keeping a codebase consistent; it is the daily collaboration ritual at real companies
- Reviewing a diff on GitHub: the Files changed tab, viewed checkboxes, and commenting on specific lines or line ranges
- Suggested changes: proposing exact replacement code the author can apply with one click
- The three review verdicts: Comment, Approve, and Request changes, and what each one signals to the author
- Giving feedback that lands: comment on the code not the person, ask questions before issuing orders, and mark non-blockers with a 'nit:' prefix
- Receiving feedback without ego: respond to every comment, push fixes to the same branch, re-request review, and disagree with reasons rather than defensiveness
- Review culture tools: a CODEOWNERS file to auto-assign reviewers, and requiring one approval before merge

Projects built:
- Complete a paired review cycle: review a classmate's PR leaving at least one suggested change and one question, and shepherd your own PR from Request changes to Approved

Practice: Rewrite three blunt review comments into constructive ones without losing their technical point.

Assessment: Your review on a peer PR includes a line-level suggestion the author applied, and your own PR reached Approved after addressing feedback.

Class 15: Issues, Projects, Forks & Running a Team Repo

Topics:
- Issues as your team's shared brain: writing a reproducible bug report and a clearly scoped feature request
- Labels (bug, enhancement, good first issue), assignees, and milestones that group issues toward a deadline like a semester submission
- GitHub Projects: a board with Todo, In progress, and Done columns that updates automatically as linked issues and PRs move
- Issue templates and Markdown task lists so bug reports arrive with the information you need
- Fork vs clone: a fork is your own server-side copy under your account, the standard route for contributing when you lack push access
- Keeping your fork current: register the original repo with git remote add upstream, pull its history with git fetch upstream, integrate with git merge upstream/main, or click Sync fork on GitHub
- The hackathon playbook: repo plus board plus issues plus branch-per-feature plus PRs, set up in the first 30 minutes so the team never blocks

Projects built:
- Set up your capstone team repository: issue templates, labels, a milestone, a Projects board with 10+ scoped issues, and a fork-based contributor test run with upstream syncing

Practice: Convert a vague idea ('make the app better') into three well-scoped, labeled, assignable issues.

Assessment: Your team board shows issues flowing from Todo to Done via linked PRs, and every teammate has synced a fork with upstream at least once.

PHASE 4: AUTOMATION & OPEN SOURCE

Automate quality and step into the open source world: CI that runs your tests on every push, branch protection that enforces discipline, professional versioning and releases, and a real contribution roadmap through Hacktoberfest and GSoC preparation.

Class 16: GitHub Actions: CI That Runs Your Tests on Every Push

Topics:
- What continuous integration is and why teams refuse to live without it: every push gets built and tested automatically, so 'it works on my machine' dies
- Anatomy of a workflow: .github/workflows/ci.yml with name, on (push and pull_request), jobs, runs-on: ubuntu-latest, and steps
- Reusable building blocks: uses: actions/checkout@v4, plus actions/setup-node or actions/setup-python with the language version you need
- Running your real test command (npm test or pytest) as a step, and letting a failing test fail the whole build
- Reading the Actions tab: live logs, the red X and green check on commits and PRs, and

re-running failed jobs
- Your free budget: the GitHub Free plan includes 2,000 GitHub Actions minutes per month, and public repositories are the friendliest place to learn CI
- Debugging CI failures: environment differences, missing dependencies, and printing your way to the root cause

Projects built:
- Add a working CI workflow to your team repository that installs dependencies and runs the test suite on every push and pull request, then break it and fix it once on purpose

Practice: Read one failed Actions log top to bottom and identify the exact failing line before touching any code.

Assessment: Your repository shows green checks on the latest commits and one PR that was blocked by a failing test and then fixed.

Class 17: Status Checks, Branch Protection & Team Discipline

Topics:
- Required status checks: a pull request cannot merge until CI is green, turning tests from a suggestion into a gate
- Branch protection rules and rulesets on main: require a pull request before merging, require one approving review, and block force pushes and deletions
- Why protection matters for student teams: nobody, including you at 2 a.m. before a deadline, can push broken code straight to main
- Keeping reviews honest: dismissing stale approvals when new commits land, and requiring conversation resolution before merge
- Speeding CI up: caching dependencies with actions/cache and splitting independent jobs to run in parallel
- A build status badge in your README that shows live CI health to every visitor, including recruiters
- The protected-main loop end to end: branch, push, PR, review, green checks, merge; the exact rhythm you will run at every internship

Projects built:
- Protect main on your team repository (PR required, one review, CI check required, force pushes blocked), then prove it by attempting a direct push and watching it fail

Practice: Attempt to merge a PR with failing CI and another with no review, and document exactly what blocks each.

Assessment: Demonstrate your full protected workflow from new branch to merged PR, with review and green checks, in one live run-through.

Class 18: Conventional Commits, Tags & Releases

Topics:
- Conventional Commits: feat:, fix:, docs:, refactor:, test:, and chore:, plus scopes like feat(auth): and the BREAKING CHANGE footer
- Why the convention pays off: readable history, automated changelogs, and commit messages interviewers actually compliment
- Semantic versioning: MAJOR.MINOR.PATCH, what bumps each number, and how it connects to conventional commit types
- Lightweight vs annotated tags: git tag v1.0.0 vs git tag -a v1.0.0 -m 'First stable release', and why releases use annotated tags
- Pushing tags explicitly: git push origin v1.0.0 (or --tags), and checking out any tagged version later to reproduce it
- GitHub Releases: turning a tag into a release page with notes, an auto-generated changelog from merged PRs, and downloadable assets
- Release hygiene for team projects: tagging the submission version of a group assignment so 'which version did we demo?' is never a question again

Projects built:
- Adopt conventional commits on your team repository, then cut v1.0.0: an annotated tag pushed to GitHub and a Release with auto-generated notes plus a short highlights section

Practice: Rewrite ten of your older commit messages as conventional commits on a practice branch.

Assessment: Your repository shows a published v1.0.0 release, and your last ten commits all follow the conventional-commit format.

Class 19: Contributing to Open Source: Etiquette & Your First PR

Topics:
- Why open source is the strongest signal on a student profile: real code, review by strangers, and proof you can work inside someone else's codebase
- Finding a first target: 'good first issue' and 'help wanted' labels, projects you already use, and matching issue difficulty to your current skill
- Read before you write: CONTRIBUTING.md, CODE_OF_CONDUCT.md, the issue tracker, and recently merged PRs that reveal the project's norms
- The contribution loop: fork the repo, clone your fork, branch, make one focused change, push to your fork, and open a PR against upstream
- Upstream etiquette: comment on the issue before starting, keep the PR small, follow the project's style and commit conventions, and stay patient and responsive with maintainers
- Handling maintainer review: expect change requests, iterate on the same branch, and never take a closed PR personally
- Non-code contributions that count: documentation fixes, reproducible bug reports, and issue triage are all legitimate first steps

Projects built:
- Make one real contribution attempt: pick an external project, claim or file an issue, and

open a properly formed PR from your fork (a documentation fix is a valid target)
Practice: Read the CONTRIBUTING.md of two major projects and list three rules that differ between them.
Assessment: You have a live PR, or a well-formed filed issue plus a prepared branch, on an external repository following that project's contribution rules.

Class 20: Hacktoberfest, GSoC & Your Open Source Roadmap
Topics:
- Hacktoberfest: the annual October event that rewards quality pull requests to participating projects, and why your target list should be ready in September
- Quality over spam: why low-effort PRs get marked invalid, and how maintainers spot them instantly
- Google Summer of Code: mentored, stipend-backed contribution to real open source organizations; what the program involves and why you must check the current year's eligibility rules yourself
- The GSoC timeline shape: organizations announced, application window, community bonding, then coding periods, and why strong applicants engage with an org months before applying
- Choosing an organization: match your language and interests, study their project ideas list, and build credibility with small merged PRs first
- What a strong proposal contains: specific understanding of the project, a realistic milestone plan, and evidence you have already contributed
- Your personal 6-month open source roadmap: from a first documentation PR to a consistent contributor profile that stands out in placements
Projects built:
- Write your open source roadmap: three target projects with justification, five candidate issues, and a month-by-month plan toward Hacktoberfest or a GSoC application
Practice: Study one past accepted GSoC proposal from any organization's public archives and note what made it concrete.
Assessment: Present your roadmap and defend your project choices and timeline in a five-minute review with your mentor.

PHASE 5: ADVANCED GIT & CAPSTONE

The finishing layer: advanced rescue and history-debugging tools, the interview questions that filter candidates, and a team capstone shipped with the complete professional workflow, followed by your certificate.

Class 21: stash, cherry-pick & reflog: The Rescue Kit
Topics:
- git stash push -m 'wip: form validation': shelving work instantly when you must switch branches right now
- Managing the shelf: git stash list, git stash pop vs git stash apply (pop applies AND drops), git stash drop, and including untracked files with -u
- git cherry-pick <sha>: copying a single commit onto your current branch, with -x to record its origin, and when it beats merging a whole branch
- Cherry-pick under fire: multi-commit ranges with git cherry-pick A..B, plus --continue and --abort when conflicts appear
- git reflog: the private journal of everywhere HEAD has been, surviving resets and branch deletions
- The classic rescues: undo a wrong git reset --hard with git reset --hard HEAD@{1}, and resurrect a deleted branch with git branch saved <reflog-sha>
- Detached HEAD demystified: how you get there (checking out a SHA or tag), why commits made there can be lost, and escaping cleanly with git switch -c
Projects built:
- Complete the rescue gauntlet: stash mid-feature to hotfix main, cherry-pick the hotfix back to your branch, then 'accidentally' hard-reset a branch and recover it fully via reflog
Practice: Delete a branch containing unmerged work, then bring every commit back using only git reflog.
Assessment: You recover a hard-reset commit and a deleted branch unaided, explaining each reflog entry you used along the way.

Class 22: git bisect & Debugging With History
Topics:
- The scenario: the app broke somewhere in the last 60 commits, and reading them one by one is not a strategy
- Binary search over history: git bisect start, git bisect bad, git bisect good v1.0.0, then testing each checkout Git hands you
- Marking your way to the culprit: bisect halves the range each step, so even a long history takes only a handful of tests
- Full automation: git bisect run pytest (or any script whose exit code means pass or fail) finds the bad commit while you watch
- Finishing up: reading the 'first bad commit' report, git bisect reset, then fixing forward or reverting the culprit
- git blame -L 10,25 app.py: which commit last touched these exact lines, used to understand intent rather than assign blame
- History as a search engine: git log -S 'functionName' (the pickaxe) for when code appeared or vanished, and git log -p <file> to walk a file's evolution
Projects built:

- In a provided repository with a hidden bug among 40+ commits, find the exact breaking commit with git bisect, first manually and then with bisect run, and revert it

Practice: Use git blame and git log -S to answer three 'when and why did this change?' questions about a real codebase.

Assessment: You identify the breaking commit's SHA and author date, and your written explanation matches the actual bug introduced there.

Class 23: Git Interview Questions & Whiteboard Scenarios

Topics:

- The classics, answered precisely: git fetch vs pull, merge vs rebase, reset vs revert, clone vs fork, and what the staging area is actually for
- 'How do you undo a commit that is already pushed?': the revert answer interviewers want, and why 'reset and force push' is a red flag on shared branches
- Explaining detached HEAD, the difference between HEAD~2 and HEAD^2, and why branches are just pointers, with a whiteboard diagram you can redraw from memory
- Scenario drills: you committed to main instead of a feature branch; a teammate force-pushed over your work; a secret key entered history; .gitignore seems to be ignored
- 'Describe your team workflow' answers that prove experience: branch protection, PRs, reviews, CI checks, and conventional commits, evidenced by your capstone repo
- Walking an interviewer through your GitHub profile: choosing the repository, narrating one PR with review, and showing the CI pipeline you built
- Rapid-fire command recall under pressure: mock rounds covering the commands this course drilled, with follow-up 'what if' probes

Projects built:

- Complete a live mock Git interview: 15 rapid-fire questions plus 2 whiteboard scenarios, then write model answers for every question you missed

Practice: Run a 10-minute peer quiz each day this week, swapping interviewer and candidate roles.

Assessment: Clear the final mock interview against your mentor's rubric and deliver a confident 3-minute walkthrough of your own GitHub profile.

Class 24: Capstone: Ship a Team Project the Professional Way + Certificate

Topics:

- Capstone kickoff: a small team application, or your real group assignment, run exactly the way an engineering team would run it
- Repository setup from memory: protected main, issue templates, labels, a milestone, and a Projects board of scoped issues
- Executing the loop: feature branches, conventional commits, PRs with real descriptions, peer review with suggested changes, and green CI required to merge
- Handling the mess like a professional: at least one real conflict resolved and one revert or reflog rescue documented in the repo
- Cutting the release: an annotated v1.0.0 tag, a GitHub Release with generated notes, and a live demo link on GitHub Pages where applicable
- The GitHub profile audit: profile README, pins, portfolio link, contribution history, and your capstone repository as the centerpiece recruiters land on
- Presenting your work to the class and earning the Modern Age Coders 'Git & GitHub for College' certificate

Projects built:

- Deliver the capstone: a team repository showing 15+ issues, 10+ merged PRs with reviews, protected main, green CI, a v1.0.0 release, and a demo link, plus your fully audited GitHub profile

Practice: Dry-run your capstone presentation with a teammate playing a skeptical interviewer.

Assessment: Final review: the capstone repository meets the full professional-workflow checklist and your profile audit passes; success earns the certificate.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: your first configured repository, a personal-notes repo with clean history built using git add -p, a complete undo rescue drill, and a messy project cleaned with a production-grade .gitignore
- Phase 2: an SSH-authenticated first push, a two-clone remote workflow simulation, a recruiter-grade README with a license, a profile README with pinned repos and a Student Developer Pack claim, and a live portfolio on GitHub Pages
- Phase 3: two features merged through the feature-branch workflow, a messy branch cleaned with interactive rebase, a three-person conflict lab, a fully described and reviewed pull request, a paired code review cycle, and a team repo with issues, labels, and a Projects board
- Phase 4: a working GitHub Actions CI pipeline, a protected main branch proven by a blocked direct push, a v1.0.0 tagged release with notes, a real external open source contribution attempt, and a personal Hacktoberfest and GSoC roadmap
- Phase 5: the rescue gauntlet with stash, cherry-pick, and reflog, a git bisect bug hunt in a 40-commit repository, a full live mock Git interview, and the team capstone with a release, green CI, and an audited GitHub profile
- Total: 24 hands-on deliverables, one per class, finishing with a team capstone and a placement-ready GitHub profile

Total projects: 24 hands-on deliverables, one per class, culminating in a team capstone shipped with the full professional workflow

SKILLS THE STUDENT WILL HAVE AT THE END

- Core Git: init, status, add -p, commit, log, diff, and the complete undo toolkit (restore, reset soft/mixed/hard, revert, amend)
- Remotes: SSH keys, clone, fetch vs pull, push, tracking branches, and multi-machine workflows
- Branching: the feature-branch workflow, merge vs rebase, interactive rebase, and calm conflict resolution
- GitHub Collaboration: pull requests, code review, suggested changes, issues, labels, milestones, and Projects boards
- Automation: GitHub Actions CI, required status checks, branch protection, dependency caching, and build badges
- Release Engineering: conventional commits, semantic versioning, annotated tags, and GitHub Releases
- Advanced Rescue and Debugging: stash, cherry-pick, reflog recoveries, git bisect, and blame plus pickaxe history searches
- Career Skills: recruiter-grade READMEs and profile, a GitHub Pages portfolio, open source etiquette, and Git interview answers

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, about 1 hour each, across 12 weeks
Hands on git work: 1.5-2 hours running the real git commands and GitHub workflows from each class
Project time: 1-1.5 hours completing each class deliverable on your own repositories
Review and doubts: 30-60 minutes reviewing your work, clearing doubts over WhatsApp, and preparing for the next class
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for setup issues, scary git states, and workflow questions
Small batches: Small batches, with mini-batch and 1-on-1 options, so every student's terminal gets individual attention
Mentor guidance: Live guidance from mentors who use Git and GitHub on real engineering teams daily
Project feedback: Feedback on every class deliverable, plus a personal capstone and profile review before certification
Living syllabus: Curriculum updated as Git and GitHub ship new features, at no extra cost

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Internship-ready developer who can join any team repository and contribute from day one
- Open source contributor with merged external pull requests and a Hacktoberfest or GSoC trajectory
- Hackathon team lead who sets up the repo, board, and workflow in the first 30 minutes
- Placement candidate with a defensible GitHub profile and confident answers to Git interview questions
- A foundation for every developer role: web, app, data, and ML teams all live in Git and GitHub daily

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Why do I need a Git and GitHub course for placements?
A: Because almost every technical recruiter and interviewer will open your GitHub profile, and almost every internship starts with 'clone the repo and open a PR'. Knowing git add and commit is not the same as working confidently on a shared repository with branches, reviews, and CI. This course teaches the team-workflow side that college courses skip, and finishes with dedicated Git interview preparation.

Q: What is the difference between Git and GitHub?
A: Git is the version control tool itself: free, open source, and distributed, created by Linus Torvalds in 2005 for Linux kernel development, and it runs entirely on your machine tracking your project's history. GitHub hosts Git repositories online so teams can collaborate through pull requests, issues, and reviews; Microsoft has owned it since 2018. This course teaches both deeply: Git for control of your history, GitHub for teamwork and your public developer profile.

Q: Is GitHub free for college students?
A: Yes, for everything this course needs. Beyond that, verified students enrolled at a degree

or diploma granting school can claim the GitHub Student Developer Pack free of charge, and it includes GitHub Pro for as long as you are a student. We walk you through claiming it at education.github.com in Class 8.

Q: Do I need to be good at coding before joining?
A: You only need basic familiarity with one programming language, at the level of a first-year college course. Git manages code rather than being a language itself, so the course focuses on workflow, collaboration, and career skills around whatever code you write. Complete command-line beginners are fine: Classes 1 and 2 build terminal fluency from scratch.

Q: Are the classes live or recorded?
A: Every class is live and interactive with a real instructor; we never teach from pre-recorded videos. You run the commands on your own machine during class, get unstuck in real time, and have WhatsApp doubt support between sessions. Group batches are capped at 20 students, with mini-batch and 1-on-1 options available.

Q: Will this course help me with Hacktoberfest and GSoC?
A: Yes, Phase 4 is built for exactly that. You learn to find beginner-friendly issues, read CONTRIBUTING.md, follow upstream etiquette, and open a properly formed pull request from a fork, then build a personal roadmap targeting Hacktoberfest and a future GSoC application. Program rules and timelines change from year to year, so we also teach you how to verify the current cycle's details yourself.

Q: What Git questions are asked in coding interviews?
A: The most common are the difference between fetch and pull, merge vs rebase, reset vs revert, how to undo a pushed commit, what detached HEAD means, and describing your team workflow. Class 23 is a dedicated interview class with rapid-fire drills, whiteboard scenarios, and a full live mock round. Better still, the whole course gives you real experience so your answers come from practice, not memorization.

Q: How long does it take to learn Git and GitHub properly?
A: This course runs 24 live classes over about 12 weeks, at 2 classes per week, with 4-5 hours of weekly commitment recommended. The basic commands take days, but team readiness, conflict resolution, CI, and open source etiquette need structured practice, which is what the drills and capstone provide. You finish with a certificate, a live portfolio, and a capstone repository, not just theory.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final project defended live, plus a written assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/git-github-version-control-course-for-college-students
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
COURSE 72 OF 137: GIT & GITHUB COURSE FOR TEENS: VERSION CONTROL FROM ZERO
==

Course link: https://learn.modernagecoders.com/courses/git-github-version-control-course-for-teens
Subject area: Version Control & Developer Tools
Designed for: Teens
Who can join: Beginner-friendly (ages 13-18) to confident GitHub collaborator
Course length: 24 classes (12 weeks · 2 classes/week)
Time needed: 3-4 hours/week recommended
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Learn Git and GitHub from zero in 24 live classes for teens 13-18: commits, branching, pull requests, merge conflicts, a free GitHub Pages portfolio and your first real open source contribution.

In one line: Never lose your work again: master the one tool every real developer uses daily, publish a live portfolio site, and make your first open source contribution in 24 live classes.

ABOUT THE PROGRAMME

Every serious young coder eventually loses work: the game mod that got overwritten, the school project that vanished the night before submission, the website folder named final-version-3-REAL. This course ends that forever. In 24 live, project-based classes for teens aged 13 to 18 (GitHub itself requires users to be 13 or older, so this is the perfect starting age), you learn Git and GitHub from absolute zero, with no terminal experience assumed. You will master real commands like git init, git commit, git branch, git merge and git rebase, resolve merge conflicts calmly, open and review pull requests, publish a free portfolio website with GitHub Pages, automate it with GitHub Actions, and make a genuine contribution to a real open source project. Every class is taught live by an instructor, using the same small-batch, mentorship-first method Modern Age Coders has used since 2020 to teach 10,000+ students across 25+ countries. You finish with a GitHub profile that colleges and internship reviewers can actually read: pinned projects with proper READMEs, a live portfolio URL, and a real pull request to someone else's codebase.

WHAT MAKES THIS COURSE DIFFERENT

- Zero terminal assumed: Class 2 is a full, friendly terminal bootcamp (pwd, ls, cd, mkdir) before any Git command is expected of you.
- Built around YOUR projects: practice on your own website, Python script, or game mod files; we provide starter projects if you don't have one yet.
- Live and interactive: every class is taught live by a real instructor in a small batch, never pre-recorded videos.
- Real developer workflows, not toy demos: branches, pull requests, code reviews, project boards, and a two-person team project on one shared repo.
- Living Syllabus: the curriculum is updated as Git and GitHub ship new features, so you always learn the current tool, not a frozen tutorial.
- Real outcomes: a polished GitHub profile, a live GitHub Pages portfolio, one external open source pull request, and a certificate.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop (Windows, Mac, or Linux) with a stable internet connection: that is the main requirement
Coding experience: None required for Git itself, and zero terminal experience is assumed. If you have any hobby project (a webpage, Python script, or game mod files) you can practice on it; if not, we provide starter projects
Accounts: A free GitHub account, created in Class 6. GitHub's Terms of Service require users to be 13 or older, which is exactly why this course starts at 13
Mindset: Willingness to type commands instead of clicking, and to break things on purpose in practice repos: that is how the undo skills become permanent

WHO THIS COURSE IS FOR

Teen coders: Ages 13-18 already building websites, Python projects, or games who are one accidental overwrite away from disaster
School project teams: Students doing group coding assignments or hackathons who are still emailing zip files back and forth
Game modders and hobby builders: Teens tinkering with mods, scripts, and side projects who want every experiment safely versioned
Future cs applicants: Students aiming at computer science who want a GitHub profile that strengthens college and internship applications
Self taught beginners: Teens learning to code from tutorials who keep hearing 'just clone the repo' and want to finally understand what that means

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: GIT FOUNDATIONS

Understand why version control exists, install Git on your own machine, get comfortable in the terminal from zero, and master the everyday cycle of init, status, add, commit, log, diff, and safe undo.

Class 1: Why Version Control? (The Lost Homework Horror Story)
Topics:
- The horror story every student knows: project-final-v2-REAL(3), an overwritten game mod, a school submission lost the night before deadline, and how version control ends it forever
- What a version control system actually does: full snapshots of your project you can return to at any moment, with a message explaining each one
- Git in one line: a free, open source distributed version control system created by Linus Torvalds in 2005 to manage Linux kernel development
- Centralized vs distributed: why every Git user carries the complete project history on their own machine, so no single crash can destroy it
- Git vs GitHub: Git is the tool on your computer, GitHub (owned by Microsoft since 2018) is the website where Git repositories live online and people collaborate
- The four words you will use every day, defined simply: repository, commit, branch, remote

- Installing Git: the git-scm.com installer on Windows (Git Bash included), Homebrew or Xcode command line tools on Mac, sudo apt install git on Linux, then verify with git --version

Projects built:

- Install Git on your own computer, run git --version to prove it works, and write a half-page version-control horror story from your own life (or a friend's) that this course will make impossible

Practice: Explain the difference between Git and GitHub to a family member in under a minute without notes.

Assessment: Screenshot of git --version output on your machine plus a two-sentence definition of a commit in your own words.

Class 2: Terminal Survival Skills (Zero Assumed)

Topics:

- Opening your terminal: Git Bash on Windows, Terminal on Mac and Linux, and why developers live here instead of clicking through folders
- Finding your way: pwd to see where you are, ls to list files (ls -a reveals hidden ones like .git), cd foldername, cd .. to go up, cd ~ to go home
- Making and removing things: mkdir for folders, creating empty files, and why rm deserves respect before you press Enter
- Speed skills: Tab completion so you never mistype a filename, and the up arrow to replay previous commands
- Paths made simple: absolute vs relative paths, and why file names with spaces need quotes
- Escape hatches: clear to clean the screen, Ctrl+C to cancel a stuck command, and q to exit the pager that git log opens
- One-time Git setup: git config --global user.name 'Your Name', git config --global user.email, git config --global init.defaultBranch main, then check it all with git config --list

Projects built:

- Build a coding-projects folder tree (three nested folders and two files) entirely from the terminal, no mouse allowed, and configure your Git name and email

Practice: Navigate from your home folder to your deepest project folder and back using only cd, pwd, and Tab completion.

Assessment: Live terminal check: reach a named folder, list its hidden files, and show your git config user.name without help.

Class 3: Your First Repository: init, status, add, commit

Topics:

- git init: turning any folder into a repository, and the hidden .git folder that IS your history (never delete it)
- The three-room mental model: files change in the working directory, wait in the staging area (the index), and become permanent snapshots once committed to the repository
- git status as the command you run constantly: it names every file as untracked, modified, or staged so nothing sneaks into a commit by accident
- Staging deliberately: git add file.html for one file, git add . for everything, and why choosing files on purpose makes better commits
- git commit -m 'Add homepage heading': writing messages in present tense that say what and why, not 'stuff' or 'changes'
- The git commit -am shortcut, and the catch: it only works for files Git already tracks
- Reading your story back with git log: author, date, hash, and message of every snapshot you have ever taken

Projects built:

- Create my-first-repo and build a tiny webpage across 5 separate commits, each with a clear message, so git log reads like a story of how the page grew

Practice: Run git status after every single command for one whole session until you can predict what it will say before you press Enter.

Assessment: Show a repository with at least 5 commits whose messages let a stranger understand the project's history without opening any file.

Class 4: Time Travel: log, diff, and Inspecting History

Topics:

- Power views of history: git log --oneline for a compact list, git log --graph --oneline --all to see the shape of your project
- Commit hashes demystified: the long SHA-1 identifier, the 7-character short form, and HEAD as 'where you are right now'
- git diff: exactly what changed but is not staged yet; git diff --staged for what is about to be committed
- git show <hash>: opening any single commit to see precisely what it changed and why
- One file's biography: git log -p filename to see every change ever made to it
- git blame filename: which commit and author last touched each line, and why it is a debugging tool, not an accusation tool
- Visiting the past safely: git checkout <hash> to look at an old version, what 'detached HEAD' really means, and git switch main to come home

Projects built:

- History detective challenge: in a prepared repo, use log, show, diff, and blame to answer five questions like 'which commit broke the page title and what did it change?'

Practice: Pick any two commits in your Class 3 repo and describe the difference between them using only git diff output.

Assessment: Check out an old commit, read it, return to main correctly, and explain detached HEAD in one sentence.

Class 5: Undoing Safely + .gitignore
Topics:
- The undo ladder: from harmless to serious, and why you always reach for the safest rung first
- git restore file to throw away working-directory changes, and git restore --staged file to unstage without losing work
- git commit --amend to fix your last commit message or include a forgotten file, and why you only amend commits you have not pushed
- git reset --soft HEAD~1 to un-commit while keeping your changes staged, and why beginners treat --hard as radioactive
- git revert <hash>: the safe, history-preserving undo that creates a new commit cancelling an old one
- .gitignore: keeping node_modules, __pycache__/, .DS_Store, and secret API keys out of your history, with * wildcards and folder/ patterns
- The oops recovery: git rm --cached secrets.txt when you committed something you meant to ignore, and why leaked secrets must also be changed

Projects built:
- Rescue mission: in a prepared practice repo, unstage a file, discard a bad edit with git restore, amend a commit message, revert a bad commit, and write a .gitignore that keeps secrets.txt out forever

Practice: Deliberately make each of the five mistakes covered in class and undo each one with the safest command that fixes it.
Assessment: Given three broken-repo scenarios, name the correct undo command for each and explain why a harsher one was not needed.

PHASE 2: GITHUB ESSENTIALS

Take your repositories online: create your GitHub account, set up SSH keys, push and pull, write READMEs people actually read, build your profile page, understand licenses and issues, and publish a free website with GitHub Pages.

Class 6: Joining GitHub (13+) and Your First Push
Topics:
- Creating your account: GitHub's Terms of Service require users to be 13 or older, which is exactly why this course starts at 13, plus choosing a username you will still like on a college application
- What GitHub Free includes: unlimited public and private repositories and 2,000 GitHub Actions minutes per month, more than enough for everything in this course
- A guided tour: repositories, your profile, notifications, the new-repo button, and the real difference between public and private repos
- Creating a repository on GitHub: initializing with a README, picking a .gitignore template, and what the license dropdown is for (Class 10 goes deep)
- Wiring local to remote: git remote add origin <url> and confirming it with git remote -v
- The first upload: git push -u origin main, and what --set-upstream actually saves you from typing forever after
- Securing your account with two-factor authentication before you build anything worth stealing

Projects built:
- Create your GitHub account with 2FA enabled, make your first online repository, connect your Class 3 local repo to it, and push all 5 commits so they appear on github.com

Practice: Make one more local commit and push it, then refresh the GitHub page and find your exact change in the web interface.
Assessment: Your Class 3 repository is visible on your GitHub account with full commit history and git remote -v shows origin correctly.

Class 7: HTTPS vs SSH: Keys, Cloning, and Pulling
Topics:
- Why GitHub removed password pushes: HTTPS with tokens vs SSH keys, and which one professionals set up on day one
- Generating your key pair: ssh-keygen -t ed25519 -C 'your_email', where keys live in ~/.ssh, and the golden rule that the private key never leaves your machine
- Installing the key: starting ssh-agent, ssh-add, pasting the .pub file into GitHub Settings under SSH keys, and testing with ssh -T git@github.com
- git clone <url>: pulling down any repository, and understanding that a clone contains the complete history, not just the latest files
- git fetch vs git pull: fetch downloads new history to look at, pull is fetch plus merge into your branch
- What 'origin' actually is: just the default nickname for a remote URL, changeable with git remote set-url
- The two-machine workflow: clone your own repo onto a second computer (or a school machine) and keep both in sync with push and pull

Projects built:
- Set up a working SSH key, clone your own repository into a second folder to simulate a second computer, make a change there, push it, and pull it back into the original

Practice: Run ssh -T git@github.com and read the greeting, then clone one interesting public repository and explore its git log.
Assessment: Demonstrate a full round trip: commit on 'computer B' clone, push, pull on 'computer A', and point to the change arriving.

Class 8: READMEs That Make People Care (Markdown)
  Topics:
    - Why README.md is your project's front door: GitHub renders it automatically on the repo homepage, and most visitors never look past it
    - Markdown core: # and ## headings, **bold** and *italic*, bulleted and numbered lists, [links](url), and images with ![alt](path)
    - Code that looks like code: inline backticks and fenced code blocks with language names for syntax highlighting
    - The anatomy of a great project README: what it is, a screenshot, how to run it, features, what you learned, credits
    - Fancier moves: tables, task lists with - [ ], and collapsible details sections for long content
    - Badges from shields.io, used tastefully: one or two that say something true, not a wall of stickers
    - Previewing before pushing: VS Code's built-in Markdown preview (Ctrl+Shift+V) so you never push a broken layout
  Projects built:
    - Write a complete README.md for your webpage project with a heading, screenshot, run instructions, a feature table, and one code block, then push it and admire the rendered result
  Practice: Find a popular open source README you admire, list three techniques it uses, and steal one for your own.
  Assessment: Your pushed README renders correctly on GitHub with at least five different Markdown elements used properly.

Class 9: Your Profile README and GitHub Identity
  Topics:
    - The special repo trick: create a repository named exactly your username and its README becomes your public profile page
    - What belongs there at your age: who you are, what you are learning, projects you are proud of, and school-safe contact info (no address, no phone)
    - Pinned repositories: choosing up to 6 repos that represent your best work, and why order matters
    - The contribution graph decoded: what the green squares actually count, and why steady real work beats gaming the graph
    - Stars as bookmarks: starring repos you want to learn from, and building a feed by following builders you admire
    - Watch vs star vs fork: notifications, bookmarks, and copies, and when each one is the right click
    - Playing the long game: colleges and internship reviewers really do open GitHub profiles, so keep every claim honest
  Projects built:
    - Create your username/username repository with a profile README introducing yourself, your current projects, and what you are learning, then pin your two best repos
  Practice: Visit five classmates' new profiles, star one repo from each, and leave one genuine compliment as an issue or comment.
  Assessment: Your profile shows a rendered profile README and at least two pinned repositories with descriptions.

Class 10: Licenses, Issues, and Repo Housekeeping
  Topics:
    - The surprise default: a repo with no license means all rights reserved, so others technically cannot reuse your code even though they can read it
    - The big three in plain English: MIT (do almost anything, keep the notice), Apache-2.0 (MIT plus patent protection), GPL-3.0 (share-alike), and choosealicense.com for deciding
    - Adding a LICENSE file the easy way with GitHub's built-in template picker
    - Issues as a project's memory: writing a bug report with steps to reproduce, expected behavior, and actual behavior
    - Organizing with labels, assignees, and milestones, even on a solo school project
    - Closing issues from commits: writing 'fixes #12' in a commit message and watching GitHub close the issue automatically
    - Housekeeping: repo descriptions and topics for discoverability, renaming safely, and archiving old projects instead of deleting history
  Projects built:
    - Add an MIT license to your webpage project, open three well-written issues on it (one bug, two feature ideas), and close one automatically with a 'fixes #N' commit
  Practice: Read the licenses of three repos you starred and identify which of the big three (or something else) each one uses.
  Assessment: Your repo has a LICENSE file, at least two open issues with labels, and one issue closed by a linked commit.

Class 11: Publish Your First Website Free with GitHub Pages
  Topics:
    - What GitHub Pages is: free static site hosting served straight from a repository, no server rental, no credit card
    - The two flavors: a user site from a repo named username.github.io, and project sites served at username.github.io/repo-name
    - Turning it on: Settings, then Pages, then deploy from branch, choosing main and the root folder (or /docs)

- Push an index.html and watch it go live: how long deploys take and how to check progress in the Actions tab
- Project-site gotchas: why relative links and image paths break under /repo-name and how to write them correctly
- A custom 404.html page so even your error page has personality
- Why this matters: this URL becomes your portfolio home for the rest of the course, and every push updates it automatically

Projects built:
- Publish your webpage project live on GitHub Pages, fix any broken image paths, add a custom 404 page, and share your working URL with the class

Practice: Change your page's heading locally, commit, push, and time how long the live site takes to update.

Assessment: A live GitHub Pages URL that loads your site correctly, including images and your custom 404 page.

PHASE 3: BRANCHING & COLLABORATION

Learn the skill that separates solo coders from teammates: branches, merges, calm conflict resolution, pull requests end to end, kind code review, forks with upstream syncing, and a real two-person project on one shared repository.

Class 12: Branches: Parallel Universes for Your Code

Topics:
- What a branch really is under the hood: a movable pointer to a commit, which is why creating one is instant and free
- The modern commands: git branch to list, git switch -c feature-dark-mode to create and move, and the older git checkout -b you will see in tutorials
- Naming branches so future-you understands: feature/dark-mode, fix/broken-nav-link, experiment/crazy-idea
- Switching with uncommitted changes: when Git lets you carry work across and when it stops you to protect it
- Seeing the universe split: git log --oneline --graph --all as branches diverge from main
- Cleaning up: git branch -d for merged branches, and why -D (force delete) deserves a moment of doubt
- The golden rule real teams follow: main always stays working and deployable, experiments live on branches

Projects built:
- On your live website repo, build a new feature (a dark theme or a new section) entirely on a feature branch while main stays untouched and your live site keeps working

Practice: Create three branches with proper names, make one commit on each, and draw the resulting graph on paper before checking it with git log --graph.

Assessment: Show a repo where main is clean, a feature branch holds new commits, and you can explain the graph line by line.

Class 13: Merging Branches Like a Pro

Topics:
- The merge ritual: switch to the receiving branch first (git switch main), then git merge feature-dark-mode
- Fast-forward merges: when main has not moved, Git just slides the pointer ahead, no new commit needed
- Three-way merges: when both branches moved, Git creates a merge commit with two parents, and you can edit its message
- git merge --no-ff to force a merge commit and keep the branch's shape visible in history, and why some teams prefer it
- Reading history after merging: spotting merge commits and branch shapes in git log --graph
- git branch --merged: finding branches that are fully merged and safe to delete
- The strategy that prevents pain: small, short-lived branches merged often beat giant branches merged rarely

Projects built:
- Merge your dark-theme branch into main (watching your live Pages site update), then create, merge, and delete a second small feature branch using --no-ff and compare the two history shapes

Practice: Predict before each merge whether it will fast-forward or create a merge commit, then verify with git log --graph.

Assessment: Explain the difference between your fast-forward and no-ff merges using your own repo's graph as the evidence.

Class 14: When Branches Collide: Conflicts Made Simple

Topics:
- Why conflicts happen: two branches edited the same lines of the same file, and Git refuses to guess which version you want
- Decoding the conflict markers: <<<<<<< HEAD, =======, and >>>>>>> branch-name, and what sits between each
- Resolving in VS Code: Accept Current, Accept Incoming, Accept Both, or hand-editing to something better than either
- Finishing the job: git add the resolved files, then git commit to seal the merge
- The eject button: git merge --abort to back out cleanly and try again with a clearer head
- Your tools mid-conflict: git status lists exactly which files still conflict, git diff shows

the battleground
    - Prevention beats cure: pull often, keep commits small, and talk to teammates about who is touching what
  Projects built:
    - Conflict laboratory: follow a script that manufactures a two-branch conflict in your repo, resolve it three ways (current, incoming, hand-crafted blend), and keep the best resolution
  Practice: Create and resolve one fresh conflict from scratch without the script, timing yourself from conflict to clean commit.
  Assessment: Given a conflicted file, correctly identify which branch each block came from and produce a sensible resolved commit.

Class 15: Pull Requests End to End
  Topics:
    - What a pull request actually is: a proposed merge plus a conversation about it, the center of collaboration on GitHub
    - The flow: push your branch (git push -u origin feature-name), then GitHub's 'Compare & pull request' button, understanding base vs compare
    - Writing a PR people want to review: a clear title, what changed and why, screenshots for visual changes, and 'closes #7' to auto-close the issue
    - Draft pull requests for work in progress, and requesting a specific reviewer when it is ready
    - Living in the Files changed tab: reading the diff, leaving comments on exact lines
    - Updating a PR: push more commits to the same branch and the PR updates itself, no new PR needed
    - The three merge buttons: merge commit, squash and merge (your 12 tiny commits become 1 clean one), and rebase and merge, with squash as the tidy default for small PRs
  Projects built:
    - Ship a real feature to your website through a complete PR: branch, push, open a PR with description and screenshot, link an issue, then squash-merge it and watch the issue close
  Practice: Open a second PR as a draft, convert it to ready for review, push one more commit to it, and observe the PR update.
  Assessment: A merged pull request on your repo with a proper description, a linked closed issue, and a clean squash-merged history on main.

Class 16: Reviewing Kindly + Forks and Upstream
  Topics:
    - Reviewing a friend's PR: the three verdicts (comment, approve, request changes) and how to phrase feedback that is specific, kind, and about the code, never the person
    - The suggestion superpower: proposing an exact code change in a review comment that the author can commit with one click
    - What a fork is: your own full copy of someone else's repository under your account, the key to contributing to projects you cannot push to
    - The cross-repo loop: fork, clone your fork, branch, push to your fork, then open a PR back to the original repository
    - Keeping a fork fresh: git remote add upstream <original-url>, git fetch upstream, git merge upstream/main, or GitHub's Sync fork button
    - Why this design is brilliant: strangers can safely propose changes to any project on Earth without anyone granting them access first
    - Star vs fork, settled: star to bookmark and learn, fork only when you intend to change
  Projects built:
    - Pair up: review a classmate's real PR using at least one line comment and one one-click suggestion, then fork a classmate's repo, improve their README, and send them a cross-repo PR
  Practice: Set up an upstream remote on your fork and sync it after your classmate merges someone else's change.
  Assessment: One thoughtful review submitted with a committed suggestion, and one cross-repo PR opened from your fork.

Class 17: Team Project: Two People, One Repo
  Topics:
    - Adding a collaborator: Settings, Collaborators, the invite, and what write access actually allows
    - Branch protection basics: requiring a pull request review before anything merges to main, so nobody (including you) pushes straight to main
    - The daily team loop: pull main, branch, commit, push, PR, review, merge, repeat until the project ships
    - When your partner merges first: bringing main into your branch with git pull origin main and resolving any conflict on YOUR branch, not on main
    - GitHub Projects boards: To do, In progress, Done columns with issues as draggable cards, perfect for school assignments
    - Assigning issues so two people never silently build the same thing
    - Communication as a Git skill: PR descriptions, issue comments, and review replies are how remote teams actually talk
  Projects built:
    - Two-person mini-sprint: with a partner, plan a small shared website on a Projects board, split it into assigned issues, and build it over branch-PR-review cycles with branch protection on
  Practice: Complete one full loop where your partner merges before you and you resolve the resulting conflict on your own branch.
  Assessment: The shared repo shows commits from both partners, at least four merged PRs with reviews, and a board with every card in Done.

PHASE 4: LEVEL UP

Move past the basics with the tools developers reach for daily: a safe introduction to
rebase, git stash for interruptions, tags and releases for shipping versions, your first
GitHub Actions automation, and a portfolio pass that makes your profile genuinely readable.

  Class 18: Merge vs Rebase, a Safe Introduction
    Topics:
      - What git rebase actually does: lifts your branch's commits and replays them on top of a new
        base, rewriting them into new commits
      - Merge vs rebase honestly: merge preserves true history with merge commits, rebase produces a
        straight readable line, and teams simply pick a convention
      - The golden rule that keeps you safe: never rebase commits you have already pushed and shared
        with others
      - git pull --rebase: updating your own branch without littering it with merge commits
      - Conflicts during rebase: fix the file, git add it, then git rebase --continue, or bail out
        completely with git rebase --abort
      - A peek at interactive rebase: git rebase -i HEAD~3 to squash or reword your own local commits
        before anyone sees them
      - The beginner's default when unsure: just merge, it is always safe
    Projects built:
      - Rebase experiment in a sandbox copy of your repo: create the same divergence twice, resolve
        it once with merge and once with rebase, and screenshot the two different history graphs side
        by side
    Practice: Use git rebase -i on three messy local commits to squash them into one
    well-named commit before pushing.
    Assessment: Explain, using your own two screenshots, what rebase changed about history
    and recite the golden rule from memory.

  Class 19: Stash, Tags, and Releases
    Topics:
      - The interruption problem: you are mid-feature and must switch branches NOW, and committing
        half-broken work feels wrong
      - git stash push -m 'half-done navbar': shelving working-directory changes instantly, and -u to
        include untracked files
      - Managing the shelf: git stash list, git stash pop (apply and remove) vs git stash apply (keep
        a copy), and git stash drop
      - Tags: permanent bookmarks on commits, lightweight (git tag v1.0.0) vs annotated (git tag -a
        v1.0.0 -m 'First public version')
      - Semantic versioning in one line: major.minor.patch, and what bumping each number promises
      - Tags are not pushed by default: git push origin v1.0.0 or git push --tags
      - GitHub Releases: turning a tag into a release page with human-readable notes and downloadable
        files, shipping v1.0.0 of your site like real software
    Projects built:
      - Practice a real interruption (stash mid-edit, fix an urgent 'bug' on main, pop your stash
        back), then tag your portfolio site v1.0.0 and publish a GitHub Release with proper notes
    Practice: Stash with a message, list your stashes, and explain to a partner the
    difference between pop and apply before using each once.
    Assessment: Your repo shows a pushed annotated v1.0.0 tag and a published Release whose
    notes summarize the version accurately.

  Class 20: Your First GitHub Actions Workflow
    Topics:
      - What CI and automation mean: robots that run your chosen steps automatically on every push,
        so checks never depend on memory
      - What you get free: GitHub Free includes 2,000 Actions minutes per month, far more than this
        course needs
      - Anatomy of a workflow file at .github/workflows/deploy.yml: name, the on: push trigger, jobs,
        runs-on: ubuntu-latest, steps, and uses: actions/checkout@v4
      - YAML survival: indentation is meaning, and one stray space is the classic first bug
      - Reading the Actions tab: live logs, green checks and red X marks on every commit, and finding
        the exact failing line
      - Upgrading your site: deploying GitHub Pages through an Actions workflow, the modern flow that
        runs on every push to main
      - Wearing it proudly: adding the workflow status badge to your README
    Projects built:
      - Write your first workflow from scratch: auto-deploy your Pages portfolio via Actions on every
        push to main, break it once on purpose, fix it by reading the logs, and add the status badge
        to your README
    Practice: Push three commits and narrate to a partner exactly what the Actions tab shows
    for each, from trigger to green check.
    Assessment: A green workflow run visible in your Actions tab that deployed your live
    site, plus a working status badge in your README.

  Class 21: A Portfolio Recruiters and Colleges Can Actually Read
    Topics:
      - Thinking like your reader: an admissions reviewer or intern recruiter gives your profile

about two minutes, so structure for skimming
- Pinned repos as your headline: choose 6, order them by impressiveness, and give every one a real README with a screenshot
- Repo metadata that sells: one-line descriptions, topic tags, and the live demo URL in the About box of every project
- The About box link trick: every project with a Pages site gets its URL right there, one click from visitor to live demo
- Writing project blurbs in outcomes language: what it does, what you built it with, and one thing you learned, not 'a simple website'
- The honest graph: steady real contributions beat manufactured green squares, and reviewers can tell the difference
- Your recurring 5-minute self-audit checklist: profile README current, pins ordered, descriptions filled, demos alive, no broken links

Projects built:
- Full portfolio pass: apply the audit checklist to your entire account, fixing every pinned repo's README, description, topics, and About-box demo link until a stranger can navigate your work unaided

Practice: Swap profiles with a classmate, spend exactly two minutes on theirs, and report what you understood and where you got lost.
Assessment: A classmate reviewing your profile cold can name your best project, open its live demo, and say what it does within two minutes.

PHASE 5: OPEN SOURCE & CAPSTONE

Graduate from your own repos to the world's: find a genuine good first issue, learn the etiquette that gets contributions merged, open a real external pull request, and complete a capstone that earns your certificate.

Class 22: Finding Your First Open Source Issue

Topics:
- Why contribute at all: real codebases, review from experienced maintainers, and public proof you can collaborate, the exact thing schools and employers cannot verify from solo projects
- Where to hunt: the good first issue and help wanted labels, curated first-issue lists, and GitHub search filters like label:"good first issue" language:python
- Reading the house rules first: CONTRIBUTING.md and CODE_OF_CONDUCT.md exist so you do not have to guess how a project works
- Sizing your first issue honestly: documentation fixes, typos, small bugs, and missing tests beat ambitious features every time
- Claiming politely: comment asking to take the issue and wait for a maintainer's reply before starting serious work
- Setting up the contribution: fork the project, clone YOUR fork, and create a branch named for the issue
- Reproducing before touching: run the project locally and confirm you can see the issue yourself before changing a single line

Projects built:
- Build a shortlist of three real candidate issues in projects you actually use or understand, pick one, claim it politely in a comment, and get your fork cloned and branched locally

Practice: Read the CONTRIBUTING.md of all three shortlisted projects and note one rule from each that you would have broken without reading it.
Assessment: A claimed (or claim-requested) real issue, a fork with an issue-named branch, and the project running on your machine.

Class 23: Making the Contribution: Etiquette and the External PR

Topics:
- Matching the project's style: read the neighboring code, follow their naming and formatting, and keep your diff as small as the fix allows
- Commit message conventions some projects require, like Conventional Commits (fix: correct typo in install guide), and finding the rule in CONTRIBUTING.md
- Opening the PR against the upstream repository: filling their PR template honestly and linking the issue you claimed
- Responding to review like a professional: make the requested changes, push to the same branch, reply to each comment, and resolve conversations
- Patience and kindness: maintainers are often volunteers, so wait about a week before a polite nudge, and never demand a merge
- If it is not merged: what you still gained (a public PR, review experience, a work sample) and how to reference it anyway
- After the merge: sync your fork from upstream, thank the reviewer, and add the contribution to your profile README

Projects built:
- Make the fix on your branch, open a real pull request to the upstream project with their template filled properly and the issue linked, and respond to any review feedback that arrives

Practice: Before submitting, self-review your own PR diff line by line and remove anything not needed for the fix.
Assessment: A real external pull request open (or merged) on a project you do not own, with a clean diff and an honest description.

Class 24: Capstone: Profile, Portfolio, Merged PR + Certification

Topics:
- The capstone checklist: polished profile README, pinned repos each with a real README, your live GitHub Pages portfolio, and one external pull request submitted or merged
- Final repo audit: a license on everything public, correct .gitignore files, and absolutely no secrets in any history (plus the rotation rule if one ever slips)
- A history a stranger can follow: sensible branch names, clear commit messages, and merged PRs that tell the story of each project
- Presenting with Git itself: walking your reviewer through a project using git log --oneline --graph as the narrative
- What unlocks next: the GitHub Student Developer Pack is free for verified students at degree or diploma granting schools and includes GitHub Pro while a student, and GitHub Education has offered free Copilot access to verified students (availability can change, check education.github.com)
- Your 90-day plan: a sustainable contribution rhythm, the next two open source targets, and using branches and PRs on every school project from now on
- Demo day: present your profile, live portfolio, and external PR to the class and earn your certificate

Projects built:
- Complete and present the full capstone: a polished GitHub profile, a live GitHub Pages portfolio linking every pinned project, and one real external pull request, defended in a demo-day walkthrough

Practice: Do a full dress rehearsal of your demo with a partner, including the git log --graph walkthrough of your best project.

Assessment: Final capstone review passed: profile, live portfolio URL, and external PR verified, earning the Modern Age Coders 'Git & GitHub for Teens' certificate.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Git installed and configured, a terminal-built folder tree, a 5-commit story repo, a history-detective investigation, and a full rescue-mission undo drill with .gitignore
- Phase 2: your secured GitHub account with a pushed repo, an SSH key with a two-machine sync round trip, a rich project README, your profile README with pinned repos, a licensed repo with linked issues, and your first live GitHub Pages website
- Phase 3: a feature built safely on a branch, fast-forward and no-ff merges compared, a conflict resolved three ways, a complete squash-merged pull request, a kind review plus a cross-repo fork PR, and a two-person team sprint on one protected repo
- Phase 4: a merge vs rebase side-by-side experiment, a stash interruption drill plus a v1.0.0 tagged Release, a from-scratch GitHub Actions deploy workflow with status badge, and a full recruiter-readability portfolio audit
- Phase 5: a shortlist and claim of a real good first issue, a genuine external pull request to an open source project, and the capstone: polished profile, live portfolio, and demo-day walkthrough
- Total: 24 hands-on deliverables, one per class, culminating in a public portfolio and a real open source contribution

Total projects: 24 hands-on deliverables, one per class, finishing with a live portfolio site and a real open source pull request

SKILLS THE STUDENT WILL HAVE AT THE END

- Git Fundamentals: init, status, add, commit, log, diff, show, blame, and the three areas of every repository
- Safe Undo: restore, restore --staged, commit --amend, reset --soft, revert, and .gitignore discipline
- Terminal Fluency: navigating, creating, and managing projects entirely from the command line
- GitHub Essentials: remotes, clone, push, pull, fetch, SSH keys, Markdown READMEs, licenses, and issues
- Branching & Merging: switch -c, merge, fast-forward vs three-way, conflict resolution, stash, and a safe rebase introduction
- Collaboration: pull requests end to end, kind code review, forks with upstream syncing, branch protection, and Projects boards
- Publishing & Automation: GitHub Pages hosting, tags, Releases, and a GitHub Actions deploy workflow
- Open Source & Portfolio: finding good first issues, contribution etiquette, external PRs, and a recruiter-readable profile

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, about 1 hour each, across 12 weeks
Hands on git work: 1-1.5 hours running real Git and GitHub commands from each class on your own machine
Project time: 1 hour completing each class deliverable, from commits and PRs to your live Pages site
Review and doubts: 30-60 minutes reviewing your history, practicing undo drills, and clearing doubts before the next class
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

  Doubt support: WhatsApp doubt support between classes for install issues, scary error
    messages, and conflict emergencies
  Small batches: Small batches, with mini-batch and 1-on-1 options, so every teen gets
    individual attention at the terminal
  Mentor guidance: Live guidance from mentors who use Git and GitHub daily on real projects
  Project feedback: Feedback on every class deliverable and a personal capstone review before
    certification
  Living syllabus: Curriculum updated as Git and GitHub ship new features, at no extra cost

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall
    short and there is focused revision and a free retest: the certificate certifies real
    learning, not attendance

WHERE THIS CAN LEAD

  - Confident collaborator on any school team or hackathon, fluent in branches, pull requests,
    and reviews
  - Open source contributor with a real external pull request as public proof of collaboration
  - Portfolio-ready applicant: a pinned, documented GitHub profile plus a live Pages site for
    college and internship applications
  - Prepared for every future language and course: the same Git workflow powers Python, web,
    Java, C++, and AI projects
  - On track for developer roles: version control is a day-one, non-negotiable requirement in
    every software job

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What age do you need to be to use GitHub?
  A: GitHub's Terms of Service require users to be at least 13 years old to create an account.
  That is exactly why this course is designed for ages 13-18: every student can legally have
  their own real account from Class 6. We also walk through securing it with two-factor
  authentication.

  Q: What is the difference between Git and GitHub?
  A: Git is a free, open source distributed version control system that runs on your own
  computer; it was created by Linus Torvalds in 2005 for Linux kernel development. GitHub is a
  website (owned by Microsoft since 2018) where Git repositories are hosted online so people
  can collaborate through pull requests, issues, and reviews. This course teaches both, in that
  order.

  Q: Is GitHub free for students?
  A: On top of that, the GitHub Student Developer Pack is free for verified students enrolled
  at a degree or diploma granting school and includes GitHub Pro while you are a student.
  GitHub Education has also offered free Copilot access to verified students; availability can
  change, so we point students to education.github.com.

  Q: Do I need to know how to code before learning Git and GitHub?
  A: No. Git tracks files, so you can learn it with a simple webpage or even text files, and we
  assume zero terminal experience: Class 2 teaches the command line from scratch. If you
  already have a hobby project like a website, Python script, or game mod, you will practice on
  that; if not, we provide starter projects.

  Q: How long does it take to learn Git and GitHub?
  A: You will be committing confidently within the first five classes, and the full course runs
  24 live classes over 12 weeks at two classes per week. We recommend 3-4 hours per week
  including practice. By the end you are not just 'familiar' with Git: you have merged pull
  requests, resolved real conflicts, and contributed to an open source project.

  Q: Are the classes live or recorded?
  A: Every class is live and interactive with a real instructor, never pre-recorded videos.
  Group batches are capped at 20 students, with mini-batch and 1-on-1 options, and you get
  WhatsApp doubt support between classes. You can also take a free demo class before deciding.
  (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a
  guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo
  . Offer both, as RULE 3 says.)

  Q: Can I host a website for free on GitHub?
  A: Yes. GitHub Pages provides free static site hosting straight from a repository, with no
  server costs. You publish your first live site in Class 11, automate its deployment with a
  GitHub Actions workflow in Class 20, and it becomes the portfolio you present at the
  capstone.

  Q: What will my teen actually have at the end of this course?
  A: Four concrete things: a polished GitHub profile with a profile README and pinned,
  documented projects; a live portfolio website on GitHub Pages; a real pull request to an

external open source project; and the Modern Age Coders 'Git & GitHub for Teens' certificate. Together they form public, verifiable proof of skill that colleges and internship reviewers can check themselves.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final project defended live, plus a written assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/git-github-version-control-course-for-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 73 OF 137: GO COURSE: BACKEND SERVICES FROM ZERO TO PRODUCTION HABITS
==========

Course link: https://learn.modernagecoders.com/courses/complete-golang-programming-masterclass-college
Subject area: Backend Engineering
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Go course for college students and professionals: the language, concurrency done right, then real backend services, APIs and deployment habits in 10-12 months.

In one line: The language cloud infrastructure is written in, learned the Go way: small, sharp and honest, from fundamentals to production-shaped services.

ABOUT THE PROGRAMME

Go was designed by people tired of complexity, and it shows: a small language, a legendary concurrency story, and a standard library that ships real services without a framework safari. This live online masterclass teaches it the Go way: fundamentals fast but properly, concurrency, goroutines, channels and the patterns that make them safe, learned as the centerpiece it is, then real backend engineering: HTTP services, APIs, databases, testing as a first-class habit, and the deployment shape of production Go. A finale month ships a capstone service and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest. Small language, deep skill, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Concurrency as the centerpiece: goroutines and channels taught with patterns and pitfalls, not just syntax
- The standard library first: real services before frameworks, the way Go culture actually works
- Testing as a habit from month one: table tests and benchmarks are just how Go is written
- Production shape included: configuration, logging, graceful shutdown and deployment honesty
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: The language (Months 1-3): Go fundamentals learned fast but properly
Phase 2: Concurrency and craft (Months 4-6): goroutines, channels, patterns and the tooling culture
Phase 3: Backend services (Months 7-9): HTTP, APIs, databases, testing and production shape
Phase 4: The finale (Month 10): the capstone service, shipped and defended, and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The language is taught from the first line
Equipment: Any computer; Go and every tool used are free
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

Backend bound: Developers and students aiming at backend and infrastructure work, where Go hiring lives
Simplicity lovers: People exhausted by framework churn who want one sharp tool
Concurrency curious: Engineers who want the best concurrency story in mainstream programming
Career switchers: Professionals entering software through a small, learnable, employable language
Second language seekers: Python or JS developers adding the performance-and-services language

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & CORE GO SKILLS (MONTHS 1-3, WEEKS 1-13)

Build rock-solid Go fundamentals. Learn programming logic, master Go syntax, understand Go's philosophy, and create your first applications.

Months 1-2: Go Fundamentals & Programming Basics (Weeks 1-7)
Introduction to Go & Development Environment Setup
Topics:
- What is Go? History, creators (Google), and philosophy
- Go's design principles: simplicity, reliability, efficiency
- Why Go? Use cases and advantages
- Go vs other languages: Python, Java, Rust, C++
- Go ecosystem and community
- Installing Go (latest stable version)
- GOPATH and Go Modules explained
- Setting up development environment: VS Code with Go extension
- GoLand IDE setup (optional)
- Understanding Go workspace structure
- Your first Go program: Hello, World!
- Go program structure: package, import, main function
- Running and building Go programs
- go run, go build, go install commands
- Comments: single-line (//) and multi-line (/* */)
- Go documentation with godoc
- gofmt for code formatting
- Go conventions and effective Go guidelines
Projects built:
- Hello World with command-line arguments
- Personal information display program
- ASCII art generator in Go
- Simple text output variations
- Environment variable reader
Practice: Daily: 30 min Go syntax practice, write 5-10 simple programs
Variables, Data Types & Operators
Topics:
- Variables: declaration with var keyword
- Short variable declaration with :=
- Variable naming conventions in Go
- Zero values in Go
- Basic data types: bool, string, int, float64
- Integer types: int8, int16, int32, int64, uint variants
- Floating-point types: float32, float64
- Complex numbers: complex64, complex128
- Type inference in Go
- Constants with const keyword
- Iota for enumerated constants
- Type conversion and type assertion
- Operators: arithmetic (+, -, *, /, %, ++, --)
- Comparison operators: ==, !=, <, >, <=, >=
- Logical operators: &&, ||, !
- Bitwise operators: &, |, ^, &^, <<, >>
- Assignment operators: =, +=, -=, *=, /=, %=
- Operator precedence
- String operations and string formatting
- fmt package for formatted I/O
Projects built:
- Calculator with all arithmetic operations
- Temperature converter (Celsius, Fahrenheit, Kelvin)
- Unit converter (length, weight, volume)

- BMI calculator with health assessment
- Currency converter with live rates simulation
- Bitwise operations demonstrator
- Type conversion examples program
- Interest calculator (simple and compound)

Practice: Solve 30 problems on variables, data types, and operators

Control Flow & Functions

Topics:

- If statements in Go (no parentheses required)
- If with initialization statement
- If-else and else-if chains
- Switch statement: more powerful than other languages
- Switch without expression (replaces if-else chains)
- Switch with multiple cases
- Type switch for interface types
- For loop: Go's only loop construct
- Classic for loop with initialization, condition, post
- While-style for loop
- Infinite loops with for
- Range loops for arrays, slices, maps, channels
- Break and continue statements
- Labeled break and continue
- Goto statement (rarely used)
- Functions: first-class citizens in Go
- Function declaration and parameters
- Multiple return values
- Named return values
- Variadic functions with ...
- Anonymous functions and closures
- Functions as values and parameters
- Recursion in Go

Projects built:

- Number guessing game
- Grade calculator with switch
- Prime number generator
- Fibonacci calculator (iterative and recursive)
- Factorial calculator
- Pattern printer (pyramids, diamonds)
- FizzBuzz implementation
- Menu-driven calculator
- Password strength checker
- Palindrome checker with multiple approaches
- Function library for math operations

Practice: Solve 40 control flow and function problems

Arrays, Slices & Maps

Topics:

- Arrays: fixed-size sequences
- Array declaration and initialization
- Array literals and zero values
- Iterating arrays with for and range
- Multidimensional arrays
- Slices: dynamic arrays
- Creating slices with make()
- Slice literals
- Slice internals: pointer, length, capacity
- Appending to slices with append()
- Copying slices with copy()
- Slicing operations and reslicing
- Nil slices vs empty slices
- Maps: Go's hash tables
- Creating maps with make() and literals
- Adding, updating, deleting map entries
- Checking key existence with comma ok idiom
- Iterating maps with range
- Maps of structs
- Nested maps
- Common slice and map patterns

Projects built:

- Dynamic array operations library
- Student grade management with slices
- Word frequency counter with maps
- Phonebook application with maps
- Inventory management system
- Matrix operations (2D slices)
- Shopping cart with slices and maps
- Duplicate remover
- Sorting algorithms implementation
- Search algorithms (linear, binary)
- Statistics calculator for datasets

```
    Practice: Solve 35 slice and map problems

Month 3: Structs, Methods & Interfaces (Weeks 8-13)
  Structs & Methods
    Topics:
      - Structs: custom types in Go
      - Struct declaration and fields
      - Struct literals and zero values
      - Anonymous structs
      - Nested structs and embedded fields
      - Struct tags for JSON, XML, database
      - Methods: functions with receivers
      - Value receivers vs pointer receivers
      - When to use pointer receivers
      - Method sets and interfaces
      - Struct composition over inheritance
      - Embedding structs for composition
      - Promoted fields and methods
      - Constructor functions in Go
      - Factory patterns
      - Encapsulation with exported/unexported fields
      - Getters and setters conventions
      - String() method for custom string representation
      - Comparing structs
      - Anonymous struct fields
    Projects built:
      - Person struct with methods
      - Bank account with transactions
      - Book library management
      - Employee management system
      - Geometric shapes with area/perimeter
      - Car dealership system
      - Student enrollment system
      - Product inventory with methods
      - Time tracker application
      - Contact management system
    Practice: Create 20 different structs with methods
  Interfaces & Polymorphism
    Topics:
      - Interfaces: defining contracts
      - Interface declaration and implementation
      - Implicit interface satisfaction
      - Empty interface: interface{}
      - Type assertions and type switches
      - Interface composition
      - Common interfaces: Stringer, error, io.Reader, io.Writer
      - Polymorphism through interfaces
      - Duck typing in Go
      - Interface best practices
      - Small interfaces principle
      - Accept interfaces, return structs
      - Interface segregation
      - Dependency injection with interfaces
      - Mocking with interfaces for testing
      - Sort interface implementation
      - Custom sorting with sort.Interface
      - Error interface and custom errors
      - Wrapping errors
      - Interface values: dynamic type and value
    Projects built:
      - Shape calculator with interfaces
      - Payment processing system
      - Notification system (Email, SMS, Push)
      - Database abstraction layer
      - Animal hierarchy with interfaces
      - Plugin system using interfaces
      - Custom error types
      - Logging abstraction
      - Event system with interfaces
      - Repository pattern implementation
    Practice: Build 15 interface-based systems
  Packages & Error Handling
    Topics:
      - Packages: organizing Go code
      - Creating packages
      - Package naming conventions
      - Exported vs unexported identifiers
      - Package initialization with init()
      - Import statements and aliases
```

```
        - Blank imports with _
        - Internal packages
        - Vendoring and dependency management
        - Go modules in detail
        - go.mod and go.sum files
        - Module versioning
        - Error handling philosophy in Go
        - Error as values
        - Creating errors with errors.New() and fmt.Errorf()
        - Error wrapping with %w verb
        - errors.Is() and errors.As()
        - Panic and recover mechanism
        - When to panic
        - Defer statement for cleanup
      Projects built:
        - Multi-package application
        - Custom package library
        - Error handling patterns library
        - Configuration package
        - Utility package creation
        - Logging package
        - PHASE 1 MINI CAPSTONE: Task Management CLI Tool
        - Features: Add tasks, list tasks, mark complete, persistence, packages, error handling
      Practice: Create 10 reusable packages
      Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

----------------------------------------------------------------
PHASE 2: ADVANCED GO & SYSTEMS PROGRAMMING (MONTHS 4-6, WEEKS 14-26)
----------------------------------------------------------------
Master Go's concurrency model, advanced features, testing, performance optimization, and
systems programming.

  Month 4: Concurrency & Advanced Features (Weeks 14-19)
    Goroutines & Channels Fundamentals
      Topics:
        - Concurrency vs parallelism
        - Go's concurrency philosophy
        - Goroutines: lightweight threads
        - Starting goroutines with go keyword
        - Goroutine scheduling and GOMAXPROCS
        - Channels: communication between goroutines
        - Creating channels with make()
        - Sending and receiving on channels
        - Channel directions: send-only, receive-only
        - Buffered channels
        - Channel closing and range loops
        - Select statement for multiple channels
        - Default case in select
        - Channel patterns: fan-in, fan-out
        - Worker pools with goroutines
        - Pipeline pattern
        - Timeout handling with channels
        - Context package for cancellation
        - sync.WaitGroup for goroutine coordination
        - Race conditions and data races
      Projects built:
        - Concurrent web scraper
        - Worker pool implementation
        - Pipeline data processor
        - Concurrent file downloader
        - Chat server with goroutines
        - Parallel image processor
        - Rate limiter with channels
        - Concurrent cache implementation
        - Producer-consumer pattern
        - Merge sort with goroutines
      Practice: Solve 35 concurrency problems
    Advanced Concurrency Patterns
      Topics:
        - Mutex and RWMutex for synchronization
        - Atomic operations with sync/atomic
        - sync.Once for one-time initialization
        - sync.Map for concurrent map access
        - sync.Pool for object reuse
        - Condition variables with sync.Cond
        - Semaphores implementation
        - Context package deep dive
        - Context cancellation and timeouts
```

```
    - Context values and best practices
    - Errgroup for error handling in goroutines
    - Advanced channel patterns
    - Pub-sub pattern with channels
    - Circuit breaker pattern
    - Retry with exponential backoff
    - Rate limiting algorithms
    - Lock-free programming basics
    - Memory model and happens-before
    - Detecting race conditions
    - Benchmarking concurrent code
  Projects built:
    - Thread-safe data structures
    - Concurrent web crawler with limits
    - Distributed task queue
    - Event bus implementation
    - Load balancer simulator
    - Concurrent database connection pool
    - Caching with TTL
    - Retry mechanism library
    - Circuit breaker library
    - Concurrent metrics collector
  Practice: Build 25 advanced concurrent systems
Reflection & Generics
  Topics:
    - Reflection in Go with reflect package
    - Type and Value in reflection
    - Inspecting types at runtime
    - Modifying values through reflection
    - Struct tag parsing with reflection
    - Method invocation via reflection
    - Creating generic functions with reflection
    - Reflection performance considerations
    - When to use reflection
    - Generics in Go (Go 1.18+)
    - Type parameters and constraints
    - Generic functions
    - Generic types and structs
    - Type inference with generics
    - Type sets and interface constraints
    - Comparable constraint
    - Generic slice operations
    - Generic data structures
    - Generics best practices
    - Migrating from interface{} to generics
  Projects built:
    - JSON marshaler with reflection
    - ORM-like query builder
    - Dependency injection container
    - Generic stack and queue
    - Generic set implementation
    - Type-safe cache with generics
    - Generic algorithms library
    - Struct validator with tags
    - Configuration parser
    - Generic repository pattern
  Practice: Implement 20 generic and reflection-based utilities
Testing & Benchmarking
  Topics:
    - Testing in Go philosophy
    - Writing tests with testing package
    - Test file naming conventions
    - Test function signatures
    - Running tests with go test
    - Table-driven tests
    - Subtests with t.Run()
    - Test helpers and cleanup
    - Testing packages and examples
    - Benchmark functions
    - Running and interpreting benchmarks
    - Benchmark comparisons
    - Coverage analysis with go test -cover
    - Coverage reports and visualization
    - Mocking and test doubles
    - Dependency injection for testing
    - Integration testing
    - End-to-end testing
    - Fuzzing with go test
    - Property-based testing
```

- Test fixtures and golden files
- Testing concurrent code
- Race detection in tests

Projects built:
- Comprehensive test suite
- Benchmark suite for algorithms
- Mock implementations
- Integration test framework
- Test data generators
- Coverage improvement project
- Fuzz testing examples
- Testing utilities library
- Test assertion library
- BDD-style testing framework

Practice: Write tests achieving 80%+ coverage for all projects

Performance Optimization & Profiling

Topics:
- Performance optimization principles
- Profiling with pprof
- CPU profiling
- Memory profiling and allocation
- Goroutine profiling
- Block profiling
- Mutex profiling
- Trace collection and analysis
- Benchmark-driven optimization
- Memory management in Go
- Garbage collection tuning
- Escape analysis
- Stack vs heap allocation
- Reducing allocations
- String optimization techniques
- Slice performance tips
- Map optimization
- Compiler optimizations
- Assembly in Go
- CGO and performance implications
- Performance monitoring in production

Projects built:
- Performance profiling toolkit
- Memory-efficient data processor
- Optimized string operations
- Zero-allocation HTTP handler
- High-performance cache
- Profiling dashboard
- Performance regression tests
- Allocation tracker

Practice: Optimize 15 programs for performance

Month 5: CLI Tools & Systems Programming (Weeks 20-23)

Building CLI Applications

Topics:
- Command-line argument parsing with flag package
- Positional arguments and subcommands
- Cobra framework for complex CLIs
- Viper for configuration management
- Environment variables handling
- Configuration file formats: JSON, YAML, TOML
- Interactive CLIs with prompts
- Terminal colors and formatting
- Progress bars and spinners
- Table output formatting
- Shell completion scripts
- Cross-platform considerations
- Signal handling
- Daemon processes
- CLI testing strategies
- Distribution and installation
- Self-updating CLIs
- CLI documentation generation
- Man pages creation
- CLI best practices

Projects built:
- Git-like version control CLI
- Database migration tool
- File manager CLI
- HTTP client like curl
- System monitoring tool
- Log analyzer CLI

    - Task runner like Make
    - Package manager CLI
    - Deployment tool
    - CLI framework
  Practice: Build 15 production-ready CLI tools
File I/O & System Programming
  Topics:
    - File operations with os package
    - Reading files: various techniques
    - Writing files efficiently
    - File permissions and metadata
    - Directory operations
    - Path manipulation with filepath
    - Temporary files and directories
    - File watching and monitoring
    - Memory-mapped files
    - Buffered I/O with bufio
    - CSV and JSON file processing
    - Binary file handling
    - Archive formats: tar, zip
    - Compression: gzip, zlib
    - Process execution with os/exec
    - Pipes and inter-process communication
    - System calls and syscall package
    - Platform-specific code with build tags
    - CGO for C integration
    - Unsafe package usage
  Projects built:
    - File synchronization tool
    - Backup utility
    - Log rotation system
    - File encryption tool
    - Directory tree analyzer
    - File compression utility
    - Process manager
    - System information collector
    - File watcher service
    - Binary parser
  Practice: Build 12 system-level applications
Networking & Protocols
  Topics:
    - TCP/IP programming with net package
    - TCP server and client
    - UDP programming
    - Unix domain sockets
    - Network protocols implementation
    - HTTP client customization
    - WebSocket implementation
    - TLS/SSL configuration
    - Certificate handling
    - DNS operations
    - Port scanning
    - Network interfaces information
    - Raw sockets (with x/net)
    - Packet capture basics
    - Network monitoring
    - Load balancing algorithms
    - Proxy server implementation
    - Tunneling and VPN basics
    - Network security basics
    - Rate limiting at network level
  Projects built:
    - TCP chat server
    - HTTP proxy server
    - Port scanner
    - Network monitor
    - Load balancer
    - VPN client prototype
    - DNS server
    - WebSocket server
    - File transfer protocol
    - Network diagnostic tool
  Practice: Implement 10 networking applications
Data Serialization & Encoding
  Topics:
    - JSON encoding/decoding with encoding/json
    - Custom JSON marshaling
    - JSON streaming
    - XML processing

- Protocol Buffers with protobuf
- MessagePack encoding
- YAML handling
- TOML configuration
- CSV processing
- Binary encoding with encoding/binary
- Base64 encoding
- Hex encoding
- Custom encoders and decoders
- Schema validation
- Data transformation pipelines
- ETL processes in Go
- Data compression techniques
- Encryption and decryption
- Hashing and checksums
- Digital signatures

Projects built:
- Data transformation pipeline
- Configuration management system
- Protocol buffer service
- ETL tool
- Data validator
- Encryption utility
- File format converter
- Schema validator
- Binary protocol implementation
- Serialization benchmark tool

Practice: Build 15 data processing applications

Month 6: Tooling, Quality & Debugging (Weeks 24-26)

Build Tools & Dependency Management

Topics:
- Go modules deep dive
- Module versioning strategies
- Semantic versioning
- Module proxies and private modules
- Vendoring dependencies
- go.work for multi-module repos
- Build flags and tags
- Cross-compilation
- Static vs dynamic linking
- Build optimization flags
- Makefile for Go projects
- Task automation with Mage
- CI/CD for Go projects
- GitHub Actions for Go
- GitLab CI for Go
- Release management
- Semantic release
- Containerizing Go applications
- Multi-stage Docker builds
- Minimal Docker images with scratch

Projects built:
- Module management system
- Build automation pipeline
- Cross-platform builder
- Release automation tool
- Docker image optimizer
- CI/CD pipeline setup
- Version management tool

Practice: Set up professional build pipelines for all projects

Code Quality & Best Practices

Topics:
- Go code style and conventions
- Effective Go guidelines
- go fmt and goimports
- Linting with golangci-lint
- Static analysis tools
- Code review practices
- Design patterns in Go
- SOLID principles in Go
- Domain-Driven Design with Go
- Clean Architecture in Go
- Hexagonal architecture
- Documentation with godoc
- README best practices
- API documentation
- Code comments guidelines
- Error handling patterns

```
        - Logging best practices
        - Configuration management patterns
        - Security best practices
        - Code metrics and complexity
      Projects built:
        - Code quality analyzer
        - Design pattern examples
        - Clean architecture project
        - Documentation generator
        - Code review checklist tool
        - Security scanner
        - Best practices linter
      Practice: Refactor all projects with best practices
    Debugging & Troubleshooting
      Topics:
        - Debugging with Delve
        - Remote debugging
        - Debugging goroutines
        - Core dumps analysis
        - Stack traces interpretation
        - Panic recovery strategies
        - Memory leaks detection
        - Goroutine leaks
        - Deadlock detection
        - Race condition debugging
        - Performance bottleneck identification
        - Distributed tracing
        - Logging strategies
        - Structured logging
        - Log aggregation
        - Metrics collection
        - Error tracking services
        - Debugging production issues
        - Post-mortem analysis
        - Debugging tools and techniques
      Projects built:
        - Debugging toolkit
        - Memory leak detector
        - Goroutine analyzer
        - Trace analyzer
        - Log aggregator
        - Error tracking system
        - Performance debugger
      Practice: Debug and fix 20 complex issues
      Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

----------------------------------------------------------------
PHASE 3: WEB DEVELOPMENT & ENTERPRISE APPLICATIONS (MONTHS 7-9, WEEKS 27-39)
----------------------------------------------------------------
Master web development with Go, build REST APIs, GraphQL, gRPC services, work with databases,
and deploy production applications.

  Month 7: Web Development & APIs (Weeks 27-32)
    HTTP & Web Fundamentals
      Topics:
        - HTTP protocol deep dive
        - HTTP/1.1 vs HTTP/2 vs HTTP/3
        - net/http package fundamentals
        - HTTP handlers and HandlerFunc
        - ServeMux and routing
        - Middleware pattern in Go
        - Request context and cancellation
        - Request parsing and validation
        - Response writing techniques
        - Cookie handling
        - Session management
        - File uploads and downloads
        - Streaming responses
        - Server-Sent Events (SSE)
        - WebSocket upgrade
        - HTTP/2 Server Push
        - TLS/HTTPS configuration
        - HTTP client best practices
        - Connection pooling
        - Timeouts and deadlines
      Projects built:
        - HTTP server from scratch
        - Custom router implementation
```

    - Middleware chain builder
    - File server with upload
    - Session management system
    - Real-time chat with WebSocket
    - SSE notification system
    - HTTP proxy server
    - Load tester tool
  Practice: Build 10 HTTP-based applications
Web Frameworks & REST APIs
  Topics:
    - Gin framework introduction
    - Gin routing and parameters
    - Gin middleware
    - Request binding and validation
    - Echo framework
    - Fiber framework overview
    - Chi router
    - Gorilla toolkit
    - REST API design principles
    - RESTful routing conventions
    - API versioning strategies
    - Request/response formatting
    - Content negotiation
    - CORS handling
    - Authentication methods: Basic, Token, OAuth
    - JWT implementation
    - Rate limiting
    - API documentation with Swagger
    - OpenAPI specification
    - Postman collections
  Projects built:
    - Blog REST API with Gin
    - E-commerce API with Echo
    - User management API
    - Todo application backend
    - Social media API
    - File storage API
    - Payment gateway integration
    - Multi-tenant API
    - API gateway implementation
    - REST API testing suite
  Practice: Build 12 production REST APIs
Database Programming
  Topics:
    - database/sql package
    - Database drivers for MySQL, PostgreSQL
    - Connection pooling configuration
    - Prepared statements
    - Transactions handling
    - Batch operations
    - SQL injection prevention
    - Query builders
    - GORM introduction and setup
    - GORM models and migrations
    - GORM associations
    - GORM hooks and callbacks
    - SQLX for enhanced database operations
    - SQLBoiler for type-safe ORM
    - Ent framework overview
    - Database migrations with golang-migrate
    - Connection management best practices
    - Read replicas and write masters
    - Database testing strategies
    - Fixtures and seed data
  Projects built:
    - User authentication system
    - Blog with comments (associations)
    - E-commerce database layer
    - Inventory management
    - Booking system database
    - Migration management tool
    - Database abstraction layer
    - Multi-database support
    - Database monitoring tool
    - Query performance analyzer
  Practice: Build 15 database-driven applications
NoSQL & Caching
  Topics:
    - MongoDB driver for Go

```
      - MongoDB CRUD operations
      - Aggregation pipelines
      - MongoDB transactions
      - Redis with Go
      - Redis data structures
      - Redis pub/sub
      - Redis Streams
      - Caching strategies
      - Cache-aside pattern
      - Write-through caching
      - Cache invalidation
      - Distributed caching
      - Memcached integration
      - ElasticSearch with Go
      - Full-text search implementation
      - Time-series databases: InfluxDB
      - Graph databases: Neo4j
      - Key-value stores
      - Document stores
    Projects built:
      - MongoDB-based blog platform
      - Redis cache layer
      - Real-time leaderboard with Redis
      - Session store with Redis
      - Search engine with ElasticSearch
      - Metrics collector with InfluxDB
      - Social graph with Neo4j
      - Hybrid database application
      - Cache performance tester
      - NoSQL migration tool
    Practice: Implement 10 NoSQL solutions
  GraphQL & Modern APIs
    Topics:
      - GraphQL introduction and concepts
      - GraphQL vs REST comparison
      - GraphQL schema definition
      - Types and resolvers
      - Queries and mutations
      - Subscriptions for real-time
      - gqlgen framework
      - Graph-gophers library
      - Schema-first vs code-first
      - DataLoader pattern
      - N+1 query problem solutions
      - GraphQL federation
      - Error handling in GraphQL
      - Authentication and authorization
      - Rate limiting GraphQL
      - GraphQL testing
      - Apollo integration
      - GraphQL documentation
      - Performance optimization
      - Caching strategies
    Projects built:
      - GraphQL API for blog
      - E-commerce GraphQL service
      - Real-time chat with subscriptions
      - Social media GraphQL API
      - GraphQL gateway
      - Federation example
      - DataLoader implementation
      - GraphQL testing framework
      - Schema stitching tool
      - GraphQL to REST adapter
    Practice: Build 8 GraphQL services

Month 8: gRPC & Microservices Foundations (Weeks 33-36)
  gRPC & Protocol Buffers
    Topics:
      - gRPC introduction and architecture
      - Protocol Buffers (protobuf) syntax
      - Defining services and messages
      - protoc compiler usage
      - Code generation for Go
      - Unary RPC
      - Server streaming RPC
      - Client streaming RPC
      - Bidirectional streaming
      - gRPC error handling
```

```
    - Status codes and details
    - Metadata and headers
    - Interceptors (middleware)
    - Authentication in gRPC
    - TLS/mTLS configuration
    - Load balancing gRPC
    - Service discovery
    - gRPC-Gateway for REST
    - gRPC reflection
    - Testing gRPC services
  Projects built:
    - User service with gRPC
    - Streaming file transfer
    - Real-time chat with gRPC
    - Microservice communication
    - gRPC gateway implementation
    - Service mesh with gRPC
    - gRPC load balancer
    - Authentication service
    - gRPC monitoring system
    - Protocol evolution example
  Practice: Build 10 gRPC services
Message Queues & Event-Driven
  Topics:
    - Message queue patterns
    - RabbitMQ with Go
    - AMQP protocol
    - Exchanges, queues, bindings
    - Apache Kafka with Go
    - Producers and consumers
    - Kafka Streams equivalent
    - NATS messaging system
    - NATS Streaming/JetStream
    - AWS SQS integration
    - Google Pub/Sub
    - Event sourcing pattern
    - CQRS implementation
    - Saga pattern
    - Outbox pattern
    - Event-driven architecture
    - Eventual consistency
    - Idempotency handling
    - Dead letter queues
    - Message ordering
  Projects built:
    - Order processing with RabbitMQ
    - Event streaming with Kafka
    - NATS-based microservices
    - Event sourcing example
    - CQRS implementation
    - Saga orchestration
    - Event store
    - Message broker comparison
    - Event-driven e-commerce
    - Real-time analytics pipeline
  Practice: Build 12 event-driven systems
Authentication & Security
  Topics:
    - Authentication strategies
    - Password hashing with bcrypt
    - Session-based authentication
    - Token-based authentication
    - JWT implementation and validation
    - OAuth 2.0 implementation
    - OpenID Connect
    - Social login integration
    - Multi-factor authentication
    - API key management
    - Role-based access control (RBAC)
    - Attribute-based access control (ABAC)
    - Security middleware
    - CSRF protection
    - XSS prevention
    - SQL injection prevention
    - Rate limiting implementation
    - DDoS protection
    - Secrets management
    - Vault integration
  Projects built:
```

- Complete auth system
- OAuth provider
- Social login integration
- MFA implementation
- RBAC system
- API key manager
- Security scanner
- Rate limiter library
- Secrets management tool
- Security audit tool

Practice: Implement security in all applications

Observability & Monitoring

Topics:

- Observability principles
- Structured logging with zap/zerolog
- Log levels and formatting
- Distributed tracing with OpenTelemetry
- Jaeger integration
- Zipkin integration
- Metrics with Prometheus
- Custom metrics and labels
- Grafana dashboards
- Application Performance Monitoring (APM)
- Health checks implementation
- Readiness and liveness probes
- Error tracking with Sentry
- Distributed logging with ELK
- CloudWatch integration
- DataDog integration
- Alerts and notifications
- SLA monitoring
- Performance monitoring
- Cost monitoring

Projects built:

- Observability platform
- Distributed tracing system
- Metrics collection service
- Log aggregation pipeline
- Custom APM tool
- Health check framework
- Alert manager
- Dashboard builder
- Performance monitor
- SLA tracker

Practice: Add observability to all services

Month 9: Shipping Go: Containers, CI & Production Habits (Weeks 37-39)

Containerization & Orchestration

Topics:

- Docker deep dive for Go
- Multi-stage builds optimization
- Minimal images with distroless
- Docker Compose for development
- Container networking
- Volume management
- Docker registry
- Kubernetes concepts
- Pods, Services, Deployments
- ConfigMaps and Secrets
- Ingress controllers
- StatefulSets for databases
- Jobs and CronJobs
- Helm charts creation
- Operators in Go
- kubebuilder framework
- operator-sdk
- Custom Resource Definitions (CRDs)
- Admission webhooks
- Service mesh with Istio

Projects built:

- Optimized Docker images
- Kubernetes manifests
- Helm chart for application
- Custom Kubernetes operator
- CRD implementation
- Admission webhook
- Service mesh setup
- Auto-scaling configuration
- Blue-green deployment

```
        - Canary deployment
      Practice: Deploy all applications to Kubernetes
    CI/CD & Automation
      Topics:
        - CI/CD principles
        - GitHub Actions for Go
        - GitLab CI configuration
        - Jenkins pipelines
        - CircleCI setup
        - Build automation
        - Test automation
        - Code quality gates
        - Security scanning
        - Dependency scanning
        - Container scanning
        - Artifact management
        - Semantic versioning
        - Release automation
        - GitOps with ArgoCD
        - Flux CD
        - Infrastructure as Code with Terraform
        - Ansible for configuration
        - Monitoring CI/CD
        - Rollback strategies
      Projects built:
        - Complete CI/CD pipeline
        - Multi-environment setup
        - GitOps implementation
        - IaC with Terraform
        - Automated testing pipeline
        - Security scanning pipeline
        - Release automation
        - Rollback mechanism
        - Pipeline monitoring
        - Cost optimization
      Practice: Set up CI/CD for all projects
    Production Best Practices
      Topics:
        - Production readiness checklist
        - High availability design
        - Disaster recovery planning
        - Backup strategies
        - Data retention policies
        - Compliance requirements
        - GDPR considerations
        - Security hardening
        - Performance tuning
        - Capacity planning
        - Cost optimization
        - Documentation standards
        - Runbooks creation
        - On-call procedures
        - Incident management
        - Post-mortem process
        - Change management
        - Feature flags
        - A/B testing
        - Production debugging
      Projects built:
        - Production checklist tool
        - Disaster recovery plan
        - Runbook generator
        - Incident response system
        - Feature flag service
        - A/B testing framework
        - Compliance checker
        - Cost analyzer
        - Capacity planner
        - Production readiness review
      Practice: Make all applications production-ready
      Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

---------------------------------------------------------------
PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)
---------------------------------------------------------------
The capstone month: spec, build, ship and defend, then the final exam that decides the
certificate.
```

Month 10: Capstone and the Final Exam (Weeks 40-44)
Capstone Spec and Setup
Topics:
- Choosing the capstone: a production-shaped API service, a concurrent data pipeline, or a CLI tool people would actually use
- A one-page spec: scope, milestones, stack
- Repository and project skeleton ready on day one
- Instructor sign-off before the build begins
Projects built:
- Approved capstone spec plus the running skeleton
Practice: Pitch the spec to the batch and absorb one hard question
Capstone Build
Topics:
- Two focused weeks of building in vertical slices
- Instructor checkpoints and honest scope cuts
- Using the whole course: the language, concurrency where it earns it, the standard library, and tests throughout
- Daily commits and an always-working build
Projects built:
- The capstone, feature-complete with version control
Practice: A working build at the end of every session
Ship and Polish
Topics:
- The finishing pass: edge cases, errors, documentation
- Production-shape pass: config, logging, graceful shutdown, and a load test that predicts
- A README worth reading and a demo worth watching
- A full-course spiral review before the exam
Projects built:
- The capstone shipped, documented and rehearsed
Practice: Two mock demos, each tighter than the last
Demo Day and the Final Exam
Topics:
- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: cloud-native infrastructure, platform engineering, and distributed systems
- Where our other courses continue the journey
Projects built:
- Demo day presentation delivered to the batch
Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: CLI tools and programs that use Go the Go way
- Phase 2: concurrent systems: workers, pipelines and safe patterns
- Phase 3: HTTP services with databases, tests and production shape
- Phase 4: the capstone service, load-tested and defended at demo day

Total projects: 20+ real builds, crowned by a production-shaped capstone service

SKILLS THE STUDENT WILL HAVE AT THE END

- Go fundamentals: the small language learned completely
- Concurrency: goroutines, channels and the patterns that keep them safe
- Backend engineering: HTTP services, APIs and databases with the standard library first
- Testing culture: table tests and benchmarks as reflex
- Production shape: configuration, logging, shutdown and deployment honesty

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

  - Backend and platform roles: Go is the language of cloud infrastructure
  - Distributed systems: the road Go was built for, entered with real fundamentals
  - Our GenAI course: Go services increasingly front AI systems
  - Kubernetes-world later: the ecosystem is written in the language you now read

EARNING POTENTIAL (share only if asked, and never promise it)

  Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
  Early career roles: Rs 4-7 LPA (Junior Go Developer)
  Growing roles: Rs 7-12 LPA (Go Developer / Backend Engineer); Rs 12-20 LPA (Senior Go Developer)
  Experienced roles: Rs 18-35 LPA (Lead Developer / Cloud Architect); Rs 25-55 LPA (Principal Engineer / Solutions Architect)
  Freelance: Rs 2,000-8,000/hour based on expertise
  International usa: $100k-220k USD (Go Developer to Principal)
  International europe: €60k-150k EUR based on country and experience

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What does this Go course cover?
  A: The language learned completely (it is famously small), concurrency as the centerpiece, goroutines, channels, patterns and pitfalls, then real backend engineering: HTTP services and APIs with the standard library first, databases, testing as a habit, and production shape: config, logging, graceful shutdown. A finale month ships a load-tested capstone service and sits the final exam.

  Q: How long does it take to learn Go properly?
  A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two more months in hand for learners who need them. The three teaching phases build the language and its real-world domain; the finale month ships a capstone and sits the final exam.

  Q: How am I assessed? Is the certificate just for attending?
  A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

  Q: Why learn Go instead of Python or Node for backend?
  A: Different sweet spots, honestly: Python wins for data and AI glue, Node for JS-everywhere teams. Go wins where services must be fast, concurrent and operationally boring, which is why Docker, Kubernetes and most cloud infrastructure are written in it. It is also one of the fastest languages to become productive in: small surface, sharp tools, one way to format.

  Q: Can I try a class before enrolling?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/complete-golang-programming-masterclass-college
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 74 OF 137: GODOT GAME DEVELOPMENT FOR KIDS AND TEENS: GDSCRIPT, 2D AND 3D
==========

Course link: https://learn.modernagecoders.com/courses/godot-game-development-course-for-kids-and-teens
Subject area: Game Development
Designed for: Teens
Who can join: Beginner to text coding, ages 10 to 17
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Godot game development course for ages 10 to 17: live online classes building real 2D and 3D games in Godot 4 with GDScript, from first scene to a finished web build.

In one line: A real game engine, a real scripting language, and games your child actually finishes.

ABOUT THE PROGRAMME

Godot is a free and open source game engine released under the MIT license, so a student can download it, build anything, and keep everything they make, with no account, subscription or fee. It runs on Windows, macOS and Linux, and its own scripting language, GDScript, is designed to be easy to pick up for beginners while still being the language professional Godot developers use every day. That makes it the natural step for a child who has outgrown block coding and wants to write real code inside a real engine. This 6-month live course is built around Godot 4. Students learn the engine's core idea first: everything is a node, nodes combine into scenes, and scenes combine into games. In the first two months they finish a complete 2D arcade game with a score, a game over screen and a restart button. Months three and four build a proper platformer with gravity, tile map levels, enemies, animation and sound. Month five moves into 3D, and month six is a capstone: each student designs, builds, playtests and exports a game of their own. The course is taught live by a teacher who reads each student's code as they write it. Younger students build the core version of each project; older or faster students take the stretch features, so a 10-year-old and a 16-year-old are both working at the edge of what they can do. Every month ends with a playable game and a short assessment, so progress is something parents can see and play.

WHAT MAKES THIS COURSE DIFFERENT

- Godot 4 and GDScript from the first week: real text code inside a real engine, not a toy editor
- A finished, playable game every month, ending in a web build that family and friends can open in a browser
- Engine ideas taught properly: nodes, scenes, signals and the physics process, so students understand why their game works
- Core and stretch versions of every project, so ages 10 to 17 each work at the right level in the same course
- Game feel treated as a skill: animation, sound, camera and juice are taught, not left to chance
- A capstone where each student scopes, builds and playtests an original game from a written design document

THE LEARNING PATH, STEP BY STEP

Phase 1: Godot foundations and a first finished game: the editor, nodes and scenes, GDScript, movement, collisions, signals, a HUD and a web export
Phase 2: Platformers and game feel: CharacterBody2D physics, tile map levels, cameras, enemies with simple states, animation, sound and menus
Phase 3: 3D and the capstone: 3D scenes, lights and cameras, a 3D level, then an original game designed, built, playtested and exported

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: No text coding needed. Students who have used Scratch or similar block tools settle in fastest, but it is not required
Computer skills: Comfortable with a keyboard and mouse, saving files and finding them again
Equipment: A laptop or desktop running Windows, macOS or Linux with a stable internet connection. Godot is a free download; tablets and phones are not enough for building
Age: 10 to 17. Every project has a core version and stretch features, so younger and older students both work at the right level
Time commitment: 2 live hours plus 2-3 hours of building each week

WHO THIS COURSE IS FOR

Block coding graduates: Children who have done Scratch or similar and are ready for real text code in a real engine
Players becoming makers: Kids and teens who play a lot of games and want to understand and build them
Complete beginners: Students aged 10 and up who have never coded and want their first language to come with instant, visible results
Finishers in training: Students with many half-built projects who need a structure that gets games finished
Future developers: Teens weighing game development or computer science who want an honest first taste of both

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: GODOT FOUNDATIONS AND A FIRST FINISHED GAME (MONTHS 1-2, WEEKS 1-8)

Students learn how Godot thinks, write their first GDScript, and finish a complete 2D arcade game with scoring, a game over screen and a browser build.

Month 1: Nodes, Scenes and Scripts (Weeks 1-4)

Weeks 1-2: The Godot Editor and Your First Scene

Topics:

- Installing Godot 4 and creating a first project
- A tour of the editor: scene dock, inspector, viewport and file system
- Nodes as building blocks and scenes as saved groups of nodes
- Sprite2D, positions and the 2D coordinate system
- Attaching a first GDScript and printing to the output panel
- The _ready function and what happens when a scene starts

Projects built:

- Hello Godot: a scene with a moving title, a sprite and a script that greets the player by name

Practice: Build three small scenes from memory and change one property of each from code

Weeks 3-4: Movement, Input and the Game Loop

Topics:

- Variables, types and the @export annotation for values you tune in the inspector
- The difference between _process and _physics_process, and why delta matters
- Reading the keyboard with Input.get_axis and input actions
- Velocity as speed with a direction, using Vector2
- Keeping the player on screen with clamp
- Reading error messages and fixing them calmly

Projects built:

- Top-down mover: a player ship that moves smoothly in eight directions and stays inside the screen

Practice: Add a speed boost key and a slow-motion key, each tuned from the inspector

Assessment: Month 1 check: build a moving player from a blank project and explain each line of the script

Month 2: A Complete Arcade Game (Weeks 5-8)

Weeks 5-6: Collisions, Signals and Spawning

Topics:

- Area2D and collision shapes, and how Godot detects overlap
- Signals: how one node tells another that something happened
- Connecting signals in the editor and from code
- Saving an enemy as its own scene and creating copies with instantiate
- Timer nodes for spawning on a rhythm
- Removing nodes safely with queue_free

Projects built:

- Dodge the falling objects: enemies spawn at random positions and the game reacts when the player is hit

Practice: Add a second enemy type with a different speed and a different signal response

Weeks 7-8: Score, HUD, Game Over and Export

Topics:

- A HUD on a CanvasLayer with Label and Button nodes
- Keeping score and showing it on screen
- Game states: title, playing and game over
- Restarting the game cleanly
- Adding sound effects with AudioStreamPlayer
- Exporting a web build so the game runs in a browser

Projects built:

- First finished game: the dodge game with title screen, score, high score for the session, sound and a web build

Practice: Playtest the game with two family members and fix the first two things they find confusing

Assessment: Phase 1 assessment: a playable game handed in with a short written note on how signals connect its parts

---

PHASE 2: PLATFORMERS AND GAME FEEL (MONTHS 3-4, WEEKS 9-16)

The genre that teaches the most: gravity, jumping, levels built from tiles, enemies that think a little, and the animation and sound that make a game feel alive.

Month 3: Platformer Physics and Levels (Weeks 9-12)

Weeks 9-10: CharacterBody2D, Gravity and Jumping

Topics:

- CharacterBody2D and why it suits a player character
- Gravity as a steady change to velocity every physics frame
- move_and_slide and checking is_on_floor before a jump
- Variable jump height by releasing the jump key early
- Coyote time and jump buffering as small kindnesses to the player
- Using StaticBody2D for solid ground

Projects built:
- Jump lab: a test room where every jump setting can be tuned live and compared

Practice: Write down three jump settings that feel good and three that feel bad, and explain the difference

Weeks 11-12: Tile Maps, Cameras and Animation

Topics:
- Building levels with a TileSet and the TileMapLayer node
- Adding collision to tiles so the player can stand on them
- Camera2D that follows the player with smoothing and limits
- AnimatedSprite2D for idle, run and jump animations
- Choosing the right animation from the player's state in code
- Collectibles and a level exit that loads the next scene

Projects built:
- Two-level platformer: tile map levels, collectibles, an animated player and a working exit

Practice: Design a third level on paper first, then build it and count how many changes playtesting forced

Assessment: Month 3 check: add a new mechanic, such as a double jump or a moving platform, to the platformer without help

Month 4: Enemies, Menus and Game Feel (Weeks 13-16)

Weeks 13-14: Enemies and Simple State Machines

Topics:
- An enemy that patrols between two points
- States with an enum and the match statement: patrol, chase and hurt
- Detecting the player with an Area2D vision zone
- Health, damage and a short invincibility window after a hit
- Knockback and a flash effect when something is hit
- Groups as a simple way to find all enemies or all coins

Projects built:
- Enemy pack: three enemies with different behaviours added to the platformer

Practice: Draw the state diagram for one enemy before coding it, then compare the code to the diagram

Weeks 15-16: Animation, Sound, Menus and Saving

Topics:
- AnimationPlayer for doors, platforms and screen effects
- Music and sound effects with volume set on audio buses
- Main menu, pause menu and settings screen built from Control nodes
- Changing scenes and passing information between them
- Saving a best time or high score to a file with FileAccess
- Particles for dust, sparkles and hits

Projects built:
- Polished platformer: menus, pause, music, particles and a saved best time, exported for the web

Practice: Make a before and after video of the same level to show what game feel changes

Assessment: Phase 2 assessment: the polished platformer plus a short live walkthrough of the enemy code

---

PHASE 3: 3D AND THE CAPSTONE GAME (MONTHS 5-6, WEEKS 17-24)

Students carry what they know into 3D, then design and ship an original game of their own, from written plan to playable build.

Month 5: Into 3D (Weeks 17-20)

Weeks 17-18: 3D Scenes, Cameras and Movement

Topics:
- Node3D, MeshInstance3D and the three axes
- Lights, a WorldEnvironment and why a 3D scene looks flat without them
- Camera3D that follows the player from behind
- CharacterBody3D movement and jumping with Vector3
- Collision shapes in 3D
- Importing a simple 3D model in glTF format

Projects built:
- 3D playground: a lit scene where a character runs, jumps and pushes objects around

Practice: Rebuild the 2D jump settings in 3D and note what had to change

Weeks 19-20: A 3D Collect-a-thon Level

Topics:
- Designing a 3D level so the player can see where to go
- Collectibles that spin, glow and play a sound
- A timer and a finish gate
- Simple 3D enemies that reuse the state machine from month 4
- Keeping a 3D game running smoothly on an ordinary laptop

- Exporting the 3D game for the web
Projects built:
- 3D level: collect every item before the timer runs out, with a finish screen and a best time
Practice: Ask two people to play without instructions and note where they get lost
Assessment: Month 5 check: a working 3D level and a short explanation of how it differs from the 2D projects

Month 6: The Capstone Game (Weeks 21-24)
Weeks 21-22: Design Document, Scope and Prototype
Topics:
- Writing a one-page game design document: idea, controls, goal and win and lose rules
- Cutting scope so the game can be finished in three weeks
- Building the core loop first and testing it ugly
- Using placeholder art and swapping it later
- Version control habits: saving numbered copies or using Git with the teacher
- Peer playtesting with structured feedback
Projects built:
- Capstone prototype: the core loop of the student's own game, playable from start to finish
Practice: Run a playtest with two people and write the three most useful pieces of feedback
Weeks 23-24: Polish, Export and Showcase
Topics:
- Adding the menus, sound and effects the prototype skipped
- Fixing the bugs found in playtesting, most important first
- Exporting a web build and a desktop build
- Writing a short game page: title, controls and credits
- Presenting the game and the design decisions behind it
- Where to go next: bigger Godot projects or our teen game development course
Projects built:
- Finished capstone game, exported and presented at the class showcase
Practice: Write a short post-mortem: what went well, what you cut and what you would do next
Assessment: Final assessment: the capstone game, the design document and a short code walkthrough with the teacher

PROJECTS THE STUDENT BUILDS AND KEEPS

- Dodge arcade game with score, sound and a web build
- Two-level platformer with tile map levels and animation
- Enemy pack with patrol, chase and hurt states
- Polished platformer with menus, saving and particles
- 3D collect-a-thon level with a timer and best time
- Original capstone game with a design document

Total projects: 6 finished games plus smaller weekly builds, all exported so they can be played

SKILLS THE STUDENT WILL HAVE AT THE END

- The Godot 4 editor: nodes, scenes, the inspector and project settings
- GDScript: variables, functions, conditions, loops, arrays, enums and signals
- 2D game physics with CharacterBody2D, collisions and tile map levels
- Game feel: animation, sound, particles and camera work
- 3D basics: meshes, lights, cameras and 3D movement
- Designing, scoping, playtesting and exporting a finished game

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building in Godot with the teacher watching the code
Homework: A build task after every class, and a monthly mixed-review task that brings back earlier skills
Practice: 2-3 hours a week of building, tuning and playtesting
Review: Every project played and reviewed by the teacher, with written notes on code and game feel

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, including screenshots of errors and short screen recordings
Progress updates: Regular progress notes to parents, with a playable build of each finished game

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

  - Our Game Development for Teens course, which moves to Unity and C# with larger 2D and 3D
    projects
  - Bigger personal Godot projects, game jams and school showcases, entered with a real portfolio
  - Python or C++ courses, where the logic learned in GDScript transfers directly
  - Computer science electives and clubs, with practical programming experience behind them
  - A clear sense of whether game development is something worth pursuing further

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What is Godot, and is it really free?
  A: Godot is a free and open source 2D and 3D game engine. It is released under the MIT
  license, which means there are no fees, no subscription and no royalties, and the games a
  student makes belong to them. It runs on Windows, macOS and Linux, and the course uses Godot
  4.

  Q: Is Godot good for kids, or is it only for professionals?
  A: Both. Professional studios ship games with it, yet the editor is light, it installs in
  minutes, and its scripting language, GDScript, is designed to be easy to pick up for
  beginners. With a teacher guiding the first steps, children from about 10 handle it well, and
  the projects grow with them into the teen years.

  Q: What language does my child learn?
  A: GDScript, Godot's own scripting language. It reads a lot like Python, so the variables,
  functions, conditions, loops and arrays students learn here transfer directly to Python and
  other languages later. We teach GDScript rather than C# because it is the language Godot is
  built around and it can export games for the web.

  Q: My child is 10 and their cousin is 16. Can both take this course?
  A: Yes. Every project has a core version that every student finishes and stretch features for
  students who are older or faster, such as double jumps, extra enemy states or a saved high
  score. The teacher sets each student's stretch goals, so nobody is bored and nobody is lost.

  Q: How is this different from your other game development courses?
  A: Our Game Development for Kids course uses Scratch, Roblox and Minecraft, and our Game
  Development for Teens course uses Unity and C#. This course sits between them and is built
  entirely on Godot: real text code from the first week, but a lighter engine and a gentler
  language than Unity. Many students take it before moving on to the teen Unity course.

  Q: What will my child have built by the end?
  A: Six finished games: an arcade dodge game, a two-level platformer, an enemy pack, a
  polished platformer with menus and saving, a 3D collect-a-thon level, and an original
  capstone game with its own design document. Each one is exported so it can be played in a
  browser.

  Q: What computer do we need?
  A: A laptop or desktop running Windows, macOS or Linux, plus a stable internet connection for
  the live classes. Godot is a small free download, and the teacher helps with installation in
  the first class. Tablets and phones are not enough for building games.

  Q: Will my child's games be published online?
  A: Only if the family chooses. Students learn to make a web build that can be shared
  privately with family and friends. Uploading a game to a public site is optional and is
  always the parent's decision.

  Q: How are the classes run?
  A: Live online classes, twice a week, with a real teacher, in a group of 15 to 20 students, a
  mini batch of up to 6, or one to one. Current fees for your region are shown in the pricing
  section of this page.

  Q: Can we try a class first?
  A: Yes. Book a free demo class. The teacher builds a small piece of a Godot game with your
  child, sees where they are, and suggests whether group, mini batch or one-to-one classes fit
  best. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no
  fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at
  https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/godot-game-development-course-for-kids-and-teens
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
COURSE 75 OF 137: GRE & GMAT QUANTITATIVE PREP: LIVE MATHS COACHING
==

Course link: https://learn.modernagecoders.com/courses/gre-gmat-quant-maths-prep-course
Subject area: Graduate Exam Mathematics
Designed for: Working professionals
Who can join: Adults preparing for graduate admissions; school maths is refreshed from the ground up
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 4-5 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Live online GRE and GMAT quantitative maths prep for adults: arithmetic, algebra, statistics and data sufficiency, taught to the current exam formats.

  In one line: The quantitative sections of both tests are built on school maths done under time pressure with tricks laid deliberately in the way. This course rebuilds the maths and then trains the exam.

ABOUT THE PROGRAMME

  The quantitative sections of the GRE and the GMAT do not test advanced mathematics. The GRE goes no higher than a second course in school algebra, and the GMAT Focus Edition covers arithmetic, algebra and statistics with no plane geometry at all. What makes them hard is that they test speed, careful reading and trap-awareness on maths most adults last touched years ago. This 6-month live course treats that honestly. The first two months rebuild the arithmetic and algebra foundation from the ground up, because a shaky foundation is where most lost marks actually come from. The middle two months cover the full content map of both exams: word problems, ratios and rates, exponents and roots, the geometry the GRE still tests, coordinate geometry, and the descriptive statistics both exams lean on. The final two months are pure exam craft: quantitative comparison for the GRE, data sufficiency and the GMAT Data Insights section, timing strategy, educated guessing, and full timed practice sets reviewed question by question. We teach to the current formats, not the old ones. That means the shorter GRE introduced in September 2023, with its two quant sections of twelve and fifteen questions, and the GMAT Focus Edition with its three forty-five-minute sections and data sufficiency moved into Data Insights. A course still teaching the old GMAT with sentence correction and an in-quant data sufficiency block is a course teaching an exam that no longer exists.

WHAT MAKES THIS COURSE DIFFERENT

  - Taught to the current exams: the shorter GRE (from September 2023) and the GMAT Focus Edition, including data sufficiency now living in the Data Insights section, not the quant section
  - Foundation rebuilt first: two full months of arithmetic and algebra before any exam tricks, because that is where most lost marks come from
  - One course for both tests: the shared arithmetic, algebra, word-problem and statistics core taught once, then the GRE-specific and GMAT-specific pieces taught separately and clearly labelled
  - Trap-awareness as a taught skill: the specific ways each exam sets bait, drilled until you see them coming
  - Timed from the middle of the course, not just at the end, so pace becomes a habit rather than a panic
  - Every full practice set reviewed question by question, with the focus on why the wrong answer was tempting

THE LEARNING PATH, STEP BY STEP

  Phase 1: Foundations rebuilt: number properties, fractions, decimals, percentages, ratios, exponents, roots and the algebra of equations and inequalities, all at exam speed
  Phase 2: The full content map: word problems, rates and work, geometry for the GRE, coordinate geometry, and the descriptive statistics and data interpretation both exams test
  Phase 3: Exam craft: quantitative comparison, data sufficiency, the GMAT Data Insights section, timing and guessing strategy, and repeated full timed practice with question-by-question review

WHAT A STUDENT NEEDS BEFORE STARTING

  Maths level: School maths up to about class 10, even if it is rusty. The course rebuilds arithmetic and algebra from the ground up, so being out of practice is expected and planned for, not a problem
  Programming: None. This is a maths and exam-strategy course with no coding at all
  Equipment: A computer with a stable internet connection, and paper and pen, since both exams reward working things out by hand at speed
  Audience: Adults preparing for graduate admissions who need the quantitative section of the

GRE, the GMAT, or both. Final-year undergraduates planning ahead are welcome

WHO THIS COURSE IS FOR

Mba applicants: MBA applicants sitting the GMAT Focus Edition who need to lift a quant score that stands between them and their target schools
Masters applicants: Applicants to masters and PhD programmes sitting the GRE, where a strong quant score matters most for quantitative and technical fields
Both exam takers: Candidates deciding between the two exams, or sitting both, who want the shared maths taught once and the differences taught clearly
Rusty professionals: Working professionals years out of school who are strong in their field but need the exam maths rebuilt from the foundation
Self study stalled: Self-preppers who have plateaued and know the missing piece is structured teaching and reviewed timed practice, not another book

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: REBUILDING THE MATHS FOUNDATION (MONTHS 1-2, WEEKS 1-8)

The arithmetic and algebra both exams assume you already own, rebuilt from the ground up and drilled at exam speed, because this is where the majority of lost marks quietly come from.

Month 1: Arithmetic and Number Properties (Weeks 1-4)
Diagnostic and the Number System
Topics:
- A full diagnostic to locate your real starting point, not your assumed one
- How the current GRE and GMAT quant sections are actually built, question by question
- Integers, factors, multiples and the properties that questions hinge on
- Prime numbers, prime factorisation and why the exams love them
- Even, odd, positive, negative: the rules that turn a hard question easy
- Divisibility rules that save real seconds under time pressure

Projects built:
- Personal weakness map: your diagnostic broken down by topic into a ranked list of what to fix first

Practice: 20 number-property questions, timed loosely at first, with every mistake tagged by cause: knowledge, careless, or misread

Fractions, Decimals and Percentages
Topics:
- Fractions without a calculator: comparing, combining and simplifying at speed
- Decimals and place value, and converting fluently between the three forms
- Percentages, percentage change and the reversal trap that catches everyone
- Successive percentage changes and why they do not simply add
- Estimation as a real technique: when the exam wants an approximate answer
- The on-screen GRE calculator and the GMAT Data Insights calculator: when they help and when they waste time

Projects built:
- Percentage trap set: ten deliberately baited percentage questions worked slowly, then reworked at speed

Practice: 24 fraction, decimal and percentage questions, half of them worded to hide which operation is needed

Ratios, Proportions and Rates
Topics:
- Ratios and how to scale them without losing the units
- Proportions and direct and inverse variation
- Rates: speed, work and price per unit, all as the same idea
- Mixtures and blends, a favourite of both exams
- Reading a multi-part ratio question without dropping a condition
- The bridge from ratios to the word problems of month 3

Projects built:
- Rate reasoning: five real-world rate scenarios set up as equations before any number is plugged in

Practice: 20 ratio and rate questions, each one required to be set up in writing before it is solved

Exponents, Roots and Number Sense
Topics:
- Exponent rules until they are automatic, not looked up
- Square roots, cube roots and simplifying surds
- Scientific notation and handling very large and very small numbers
- Absolute value and what it does to inequalities
- Number-sense shortcuts: last-digit tricks, sign tricks, magnitude checks
- Sanity-checking an answer in five seconds before you commit to it

Projects built:
- Shortcut library: your own one-page sheet of the number-sense checks that will not fit in a calculator

Practice: 22 exponent and root questions, with a sanity-check written next to each final answer

Assessment: Month 1 checkpoint: a timed arithmetic and number-properties set graded on accuracy and on whether the timing held

Month 2: The Algebra Foundation (Weeks 5-8)
Expressions and Linear Equations
Topics:
- Algebraic expressions: simplifying, expanding and factoring cleanly
- Solving linear equations and rearranging formulas under time
- Translating words into equations, the single most valuable exam skill
- Substitution and picking smart numbers to test answer choices
- Working backwards from the answer options when it is faster
- Common algebra traps and the misreads that trigger them
Projects built:
- Translation drill: fifteen word phrases each converted to an equation, the step where most marks are won or lost
Practice: 24 algebra questions solved twice each: once by direct algebra, once by testing the options, to learn which is faster when
Systems, Inequalities and Absolute Value
Topics:
- Systems of equations by substitution and elimination
- Inequalities and the sign-flip rule that quietly loses marks
- Compound inequalities and interval reasoning
- Absolute value equations and inequalities
- When a system has one solution, none, or infinitely many, and why the exam asks
- Recognising a system hiding inside a word problem
Projects built:
- Two-variable challenge: a set of word problems that each resolve to a two-equation system, set up before solving
Practice: 20 systems and inequalities questions with the setup shown in full before any solving
Quadratics and Functions
Topics:
- Factoring quadratics and the common exam-friendly forms
- The quadratic formula and completing the square, used sparingly and correctly
- Functions and function notation without the fear
- Reading a function defined by a rule or a table
- Sequences: arithmetic and geometric, as the exams present them
- Spotting the quadratic buried in a word problem
Projects built:
- Quadratic recognition: ten problems where the challenge is realising a quadratic is involved at all
Practice: 22 quadratics and functions questions, each opened by naming the structure before touching the algebra
Algebra Under Exam Pressure
Topics:
- Choosing between algebra, plugging in numbers, and testing options, deliberately
- Algebraic word problems: age, mixture, work and money set up cleanly
- Multi-step problems and keeping track without losing the thread
- Where the two exams differ in how they dress algebra questions
- Building a personal decision rule for which method to reach for
- Timed algebra: getting the right answer inside the clock
Projects built:
- Method-choice audit: a mixed algebra set where you record which method you chose and whether it was the fastest available
Practice: 24 mixed algebra questions under a soft timer, reviewed for method choice as much as for the answer
Assessment: Month 2 checkpoint: a timed algebra section, marked on accuracy, timing and whether the fastest method was chosen

---
PHASE 2: THE FULL CONTENT MAP OF BOTH EXAMS (MONTHS 3-4, WEEKS 9-16)

Word problems, the geometry the GRE still tests, coordinate geometry that the GMAT counts as algebra, and the descriptive statistics and data interpretation both exams rely on.

Month 3: Word Problems and Geometry (Weeks 9-12)
Word Problems That Actually Appear
Topics:
- Distance, rate and time, including two-traveller and round-trip variants
- Work and combined-work problems set up without the classic error
- Mixture, interest and profit-and-loss problems
- Overlapping-sets problems and the two-way table that tames them
- Age and consecutive-integer problems
- A method for translating any dense word problem into a clean setup
Projects built:
- Word-problem playbook: your own labelled template for each of the six recurring problem types
Practice: 24 word problems spanning all six types, each set up in writing before it is solved
Geometry for the GRE: Lines, Angles and Triangles

Topics:
- Why this month matters for the GRE and not for the GMAT Focus quant section
- Lines, angles and the parallel-line rules the exam reuses constantly
- Triangles: properties, the Pythagorean theorem and the special right triangles
- Similar and congruent triangles and the ratios they give you
- Perimeter and area worked cleanly
- The figure-not-to-scale warning and how the GRE uses it against you

Projects built:
- Triangle toolkit: a reference of every triangle fact the GRE reuses, drawn and annotated by you

Practice: 20 GRE-style geometry questions, GMAT-only students substituting an equivalent Data Insights set

Geometry for the GRE: Circles, Polygons and Solids

Topics:
- Circles: radius, diameter, circumference, area, arcs and sectors
- Quadrilaterals and regular polygons and their angle sums
- Three-dimensional solids: volume and surface area of boxes, cylinders and spheres
- Inscribed and combined figures, a favourite GRE construction
- Estimating from a figure when exact computation is slow
- Knowing which formulas to memorise and which to reconstruct

Projects built:
- Formula sheet, earned: build the geometry formula reference yourself, which is how it actually sticks

Practice: 20 circle, polygon and solid questions, each with a labelled diagram drawn before solving

Coordinate Geometry

Topics:
- The coordinate plane, distance and midpoint
- Slope and the equation of a line in its several forms
- Parallel and perpendicular lines by their slopes
- Why the GMAT Focus Edition counts coordinate geometry as algebra and still tests it
- Intersections, and reading a graph the exam gives you
- Reflections and simple transformations on the plane

Projects built:
- Line investigation: a set of coordinate problems solved both algebraically and by sketching, to see which is faster where

Practice: 22 coordinate geometry questions relevant to both exams, since this topic survives on the GMAT as algebra

Assessment: Month 3 checkpoint: a timed section mixing word problems, GRE geometry and coordinate geometry

Month 4: Statistics and Data Interpretation (Weeks 13-16)

Descriptive Statistics

Topics:
- Mean, median, mode and range, and which one a question is really testing
- Weighted averages, a heavily tested and heavily missed idea
- Standard deviation conceptually: the exams test understanding, not hand calculation
- Quartiles, percentiles and the reasoning around them
- How adding or changing a value moves the mean and the median
- The statistics traps both exams set, catalogued

Projects built:
- Average-shift study: a set of questions about how statistics change when the data changes, reasoned without brute force

Practice: 22 statistics questions weighted towards weighted averages and standard-deviation reasoning

Counting and Probability

Topics:
- The fundamental counting principle applied cleanly
- Permutations and combinations, and telling which the question wants
- Basic probability and the complement shortcut
- Probability with and without replacement
- Combined events and when to add versus multiply
- The exam-realistic level of difficulty, so you neither under nor over-prepare

Projects built:
- Counting decision tree: your own flowchart for choosing between permutation, combination and direct counting

Practice: 20 counting and probability questions pitched at the exams' actual difficulty, not competition-maths depth

Data Interpretation and Graphs

Topics:
- Reading tables, bar charts, line graphs and pie charts accurately and fast
- Multi-source questions where two graphs must be combined
- GRE data interpretation sets and how they are scored
- The visual and quantitative formats inside GMAT Data Insights
- Percentage and ratio reasoning applied to chart data
- Not being fooled by a chart drawn to mislead

Projects built:
- Graph deconstruction: a real multi-graph data set answered, then examined for how it could have been misread

Practice: 20 data-interpretation questions across chart types, timed as the sets are on the real exams

Consolidation and First Full Quant Section

Topics:

- Pulling the whole content map together into one map you can see
- A personal error log built from every month so far
- Your two or three recurring mistake patterns, named and targeted
- Pacing across a full section rather than a single question
- A pre-test routine for the quant section
- Setting a realistic target score band for your programme

Projects built:

- First full timed quant section, GRE or GMAT to match your goal, followed by a written self-review

Practice: One complete timed quant section plus a question-by-question review of every miss and every lucky guess

Assessment: Month 4 checkpoint: a full timed quant section scored, with an honest read on your current band and gap to target

---

PHASE 3: EXAM CRAFT AND TIMED MASTERY (MONTHS 5-6, WEEKS 17-24)

The question formats unique to each exam, the timing and guessing strategy that protects a score, and repeated full timed practice reviewed until the mistake patterns are gone.

Month 5: Exam-Specific Formats and Strategy (Weeks 17-20)

GRE Quantitative Comparison

Topics:

- The four answer choices of quantitative comparison and what each really means
- Why quantitative comparison rewards reasoning over full computation
- Testing cases: zero, one, negatives, fractions and the numbers that break patterns
- The must-not-compute mindset for a whole question type
- The specific traps built into quantitative comparison
- GMAT-only students spend this session on an equivalent problem-solving intensive

Projects built:

- Case-testing drill: quantitative comparison questions solved purely by choosing the numbers that expose the answer

Practice: 24 quantitative comparison questions, no full calculation permitted where case-testing will do

Data Sufficiency and the GMAT Data Insights Section

Topics:

- Data sufficiency: the format that asks whether you can answer, not what the answer is
- The fixed five answer choices and a reliable order to eliminate them
- Why data sufficiency now lives in GMAT Data Insights, not the quant section
- The other Data Insights formats: table analysis, graphics interpretation, multi-source reasoning, two-part analysis
- The calculator that Data Insights allows and quant does not
- GRE-only students use this session to deepen data interpretation instead

Projects built:

- Data sufficiency ladder: a graded set that builds the elimination habit from simple to genuinely tricky

Practice: 24 data sufficiency and Data Insights questions worked with the elimination framework applied every time

Timing, Pacing and Educated Guessing

Topics:

- The real time budget: roughly one minute forty-five per GRE quant question, just over two minutes on GMAT quant
- The thirty-second rule for deciding to move on
- Educated guessing that beats random, and when to deploy it
- Section-level pacing rather than fighting for every single question
- Managing the adaptive nature of the GMAT and the section-adaptive GRE
- Recovering calmly from a question that goes wrong mid-section

Projects built:

- Pacing plan: your written per-section time strategy, including your personal move-on triggers

Practice: Timed blocks focused purely on pacing decisions, tracking how many move-on calls were correct

Error Patterns and Targeted Repair

Topics:

- Mining your accumulated error log for the two or three patterns that cost the most
- Careless-error reduction as a trainable, measurable skill
- Rebuilding any content weakness the practice sections have exposed
- Distinguishing knowledge gaps from timing errors from misreads, because the fixes differ
- Re-attempting past misses cold to confirm the repair held
- Adjusting the target band with real evidence now in hand

Projects built:

- Repair sprint: your top mistake pattern attacked with a focused set until the error rate on it visibly drops

Practice: Targeted sets built from your own error log, retested a week later to prove the fix stuck

Assessment: Month 5 checkpoint: a full timed section compared against month 4 to

measure real movement

Month 6: Full Timed Practice and Test-Ready Review (Weeks 21-24)
Full Section One and Deep Review
Topics:
- A complete timed quant section under genuine exam conditions
- Question-by-question review focused on why the wrong answer was tempting
- Separating unlucky guesses from real knowledge, since only one needs fixing
- Updating the error log with the newest patterns
- Retiming the questions that were slow but correct
- Setting the single focus for the coming week

Projects built:
- Full section one, reviewed in a written report: score, error breakdown and the one thing to fix next

Practice: The full section plus a complete written review; slow-but-correct questions retimed to build margin

Full Section Two and Format Sharpening
Topics:
- A second full timed section, rotating to the other exam if you are sitting both
- Sharpening the exam-specific formats: quantitative comparison or data sufficiency, whichever is yours
- Comparing this section against the last for direction of travel
- Fatigue management across a real testing block
- Fine-tuning the pacing plan with fresh data
- Locking in the guessing strategy so it is automatic on the day

Projects built:
- Full section two, with a side-by-side comparison against section one and a revised pacing plan

Practice: The second full section plus focused drilling on whichever format still leaks marks

Full Section Three and Final Content Gaps
Topics:
- A third full timed section to make exam pace feel ordinary
- Closing the last content gaps the three sections have exposed
- A calm, repeatable pre-section routine rehearsed until it is automatic
- Test-day logistics: what to expect at the centre or on the online exam
- Building the mental stamina for a full test, not just a section
- Confirming a realistic, evidence-based target for test day

Projects built:
- Full section three, with the final short list of content gaps and a plan to close each one

Practice: The third full section plus targeted repair of any content weakness still standing

Test-Ready: Final Review and Plan
Topics:
- A final mixed timed set drawing on everything the course covered
- The last pass over the error log, now much shorter than it started
- Your personal test-day playbook: pacing, guessing rules and pre-section routine on one page
- Managing exam nerves with preparation rather than hope
- A realistic score expectation grounded in your practice results
- Where to go next if you want to lift a strong score to an excellent one

Projects built:
- Test-day playbook: a single sheet with your pacing plan, guessing rules, routine and target, ready for the exam

Practice: A final timed set and a full walk-through of the test-day playbook

Assessment: Final review: a complete timed section, a progress summary from diagnostic to now, and certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A personal weakness map built from the opening diagnostic
- A shortcut and number-sense reference sheet you build yourself
- A word-problem playbook covering all six recurring types
- A geometry formula sheet earned by reconstruction, not handout
- A running error log that drives the whole second half of the course
- A written per-section pacing and guessing plan
- Three fully reviewed timed quant sections in the final month
- A one-page test-day playbook ready to take into the exam

Total projects: A complete personal prep kit: weakness map, reference sheets, error log, pacing plan and a test-day playbook, plus multiple fully reviewed timed sections

SKILLS THE STUDENT WILL HAVE AT THE END

- A rebuilt arithmetic and algebra foundation worked at exam speed
- The full content map of both exams: word problems, GRE geometry, coordinate geometry, statistics and data interpretation
- Quantitative comparison for the GRE and data sufficiency and Data Insights for the GMAT
- Timing, pacing and educated-guessing strategy that holds under pressure
- Trap-recognition as a trained reflex on both exams

- A disciplined review habit that turns practice sections into real improvement

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, solving and reviewing questions live rather than watching worked solutions go by
Practice: 4-5 hours weekly of timed problem sets and full sections between classes
Review: Homework and practice sections reviewed with written feedback, with equal attention to timing, method choice and accuracy
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for the questions that stall you
Progress updates: Monthly progress notes tracking practice-section scores and movement toward your target band
Career guidance: Honest guidance on realistic target scores for your programme and on what the quant score does and does not decide in an application

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- A completed graduate-admissions application with the quantitative section prepared for, whether that is an MBA, an MS, or a PhD track
- The quantitative reasoning base that graduate business and technical programmes assume from day one
- A stronger footing for any role or programme that leans on analytical and quantitative reasoning
- A foundation you can extend into our statistics or data courses if your path runs toward analytics
- The habit of timed, reviewed practice that transfers to any high-stakes exam you sit later

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The GRE and GMAT are gateways to graduate programmes, and it is those programmes, not this course, that connect to long-term career outcomes years later. The figures below are broad, publicly reported market ranges for the kinds of graduate paths these exams lead toward, shown purely for context. They are not a promise of any score, admission, or income, all of which depend on your application as a whole, the programme, the market and your own work. This course prepares you for the quantitative section and does not guarantee a score, an admission, or a salary.
Context note: A quant score is one input among many in an application. It opens doors; it does not by itself determine the outcome on the other side of them
Mba pathways: Graduate business programmes report a wide range of post-degree outcomes depending heavily on school, industry, geography and prior experience
Ms and analytics pathways: Masters routes in data, analytics, finance and engineering lead into quantitative roles whose pay varies widely by field, location and experience
Honest framing: Treat the exam as a step, not a guarantee. The honest promise this course can make is quantitative readiness and reviewed timed practice, not a number on a payslip

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Does this course cover both the GRE and the GMAT, or do I have to choose?
A: Both, and that is deliberate. The two exams share most of their quantitative content: arithmetic, algebra, word problems, statistics and data interpretation are common ground, so we teach that core once, to everyone. The exam-specific pieces are then taught separately and clearly labelled: quantitative comparison and geometry for the GRE, data sufficiency and the Data Insights formats for the GMAT. If you are sitting only one exam, you focus your practice there while the other exam's students do their format-specific session; if you are sitting or still deciding between both, you get the full picture. Tell us your target at the diagnostic and we tune your practice accordingly.

Q: The exams changed recently. Is this course current?
A: Yes, and this matters more than it sounds. We teach the shorter GRE introduced in September 2023, with its two quant sections of twelve and fifteen questions, and the GMAT Focus Edition with its three forty-five-minute sections. A crucial change many older courses miss: on the GMAT, data sufficiency is no longer in the quant section at all. It now lives in the Data Insights section, and the Focus Edition quant section has no plane geometry. We teach the exams as they are today, not as they were before 2023.

Q: My maths is very rusty. Am I too far behind for this?
A: No. Rusty is the assumption the course is built on, which is exactly why the first two full months rebuild arithmetic and algebra from the ground up before any exam strategy appears. Most adults sitting these exams last studied this maths years ago, and the biggest score gains usually come from repairing that foundation rather than from clever tricks. The week 1 diagnostic tells us precisely where you stand so the early weeks target your real gaps. What the course does need from you is the practice hours; the starting level it can work with is genuinely low.

Q: Do you guarantee a particular score?
A: No, and you should be wary of any course that does. Your score depends on your starting point, the hours you put in, and your performance on the day, none of which a course can guarantee. What this course does guarantee is honest: live teaching that rebuilds the maths and drills the exam, small batches, real doubt support, and multiple full timed sections reviewed question by question. We will give you a clear, evidence-based read on a realistic target band from your practice results, and we will be straight with you about the gap and the work it takes to close it.

Q: How much time do I need to put in each week?
A: Two live one-hour classes plus four to five hours of practice a week is the plan the course is paced around. The practice is not optional padding: these exams reward drilled speed and reviewed mistakes, and the improvement happens in that practice time as much as in class. In the final month the practice includes full timed sections, which take longer in a single sitting, so allow for that. The schedule offers evening and weekend slots so working professionals can fit it around a job.

Q: Does the maths on these exams get very advanced?
A: No, and this reassures most people. Neither exam tests advanced mathematics. The GRE goes no higher than a second course in school algebra, with no trigonometry and no calculus, and the GMAT Focus quant section covers arithmetic, algebra and statistics with no plane geometry. The difficulty is not the depth of the maths; it is the speed, the careful reading, and the traps deliberately laid in the questions. That is genuinely good news, because those are trainable skills, which is exactly what the second half of the course trains.

Q: Can I use a calculator on the exam?
A: Partly, and knowing when is itself a skill. The GRE gives you an on-screen calculator for the quant section. On the GMAT Focus Edition, the quant section does not allow a calculator at all; only the Data Insights section does. Because of this, we drill mental and paper methods hard, since leaning on a calculator you may not have is a costly habit. We also teach when reaching for the calculator, where you are allowed one, actually wastes time versus estimating or reasoning to the answer.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Will this course help with the rest of the exam too, not just quant?
A: This course is focused specifically on the quantitative section, which is where a structured maths rebuild and timed practice make the biggest, most reliable difference. It does not teach the verbal sections or the GMAT Data Insights verbal-style reasoning beyond the quantitative and data-analysis parts. Many students take this course precisely because quant is their weak half and they can prepare verbal on their own. In the career-guidance sessions we are happy to point you to sensible resources for the other sections so your overall preparation is balanced.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/gre-gmat-quant-maths-prep-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 76 OF 137: HACKATHON FOR ADULTS: FREE EVENT + 12-WEEK PRO PREP (18+)
==========

Course link: https://learn.modernagecoders.com/courses/hackathon-prep-for-adults-professionals-coding-ai-innovation-course
Subject area: Hackathons & Professional Innovation Programs
Designed for: Working professionals
Who can join: Beginner-Friendly to Advanced (Adults 18+)
Course length: 12 weeks pro prep + Live Hackathon Weekend
Time needed: 8-10 hours/week
Certificate: Pro Hackathon Builder, Modern Age Coders Certified

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one

live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Free hackathon for adults, professionals and founders (18+), plus a 12-week Pro Prep in full-stack engineering, AI agents, system design and pitching.

In one line: Free hackathon for everyone. Pro-grade prep for those who want to build at production scale.

ABOUT THE PROGRAMME

Hackathon for Adults & Professionals is a free-to-enter hackathon and a 12-week pro-grade prep course at Modern Age Coders, designed for working professionals, career switchers, founders, students 18+, and anyone who has ever said 'I wish I could just build that idea myself.' The hackathon participation is 100% free, no fee, no employer endorsement needed. You can be a software engineer, a product manager, a designer, a marketer, a teacher, a doctor, a small-business owner, or a fresh graduate. If you're 18+ and want to build, you're in. Our only motive: build something great. Build something innovative. Use coding logic, efficient algorithms, AI agents and modern infrastructure to ship real production software in 48 hours that solves a problem people would actually pay for. The optional 12-week Pro Prep Course is for adults who want to walk into a hackathon (or a startup) with deep, current technical fluency. It is priced the same as our other professional courses, Group Rs 1,499/mo, Mini Batch Rs 2,999/mo, or 1-on-1 Rs 4,999/mo. The curriculum is dense and modern. We cover full-stack engineering with Next.js 16 + Server Actions + Cache Components, distributed system design, AI agents with tool calling and durable workflows, RAG pipelines with vector databases, prompt engineering with frontier models (Claude-class, GPT-class, Gemini-class), observability, security, and shipping to production on Vercel / AWS / Cloudflare. We also cover the soft skills that actually matter, scoping under time pressure, founder-grade pitching, sponsor relationship management, and demo storytelling. By Week 12, every participant has a polished GitHub portfolio, a deployed capstone with paying-customer-ready quality, a 3-minute investor-style pitch video, and competes in the Live Modern Age Coders Pro Hackathon Weekend with sponsor prizes (cloud credits, API credits, mentor calls, and cash awards).

WHAT MAKES THIS COURSE DIFFERENT

- Free hackathon participation, anyone 18+ can register at zero cost
- Prep course priced same as other Modern Age Coders professional programs
- Built for working adults: evening and weekend live sessions with structured catch-up support
- Curriculum led by senior engineers, ex-CTOs and YC-alumni founders
- Modern 2026 stack: Next.js 16, Server Actions, AI SDK, Workflow DevKit, Vercel Sandbox
- AI Agents focus, tool calling, multi-step workflows, durable execution
- RAG done properly, chunking, embeddings, hybrid search, reranking
- System design depth: queues, caches, sharding, event-driven architectures
- Real production concerns: observability, security, on-call, cost
- 1-on-1 mentor matching with senior engineers / founders in capstone phase
- Founder-grade pitch lab with VC-style critique
- Sponsor prizes from cloud, AI and developer-tool partners
- Internship and full-time role referrals for top performers
- Career-switch friendly: pathway from non-tech career to first tech role

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Weeks 1-4): Python + TypeScript, Git/GitHub, full-stack fundamentals, first solo hackathon
Phase 2: Pro Stack (Weeks 5-8): Next.js 16 + Server Actions + Cache Components, AI agents + tool calling + RAG, system design, observability, mock hackathon
Phase 3: Hackathon Sprint (Weeks 9-12): Capstone build, mentor coaching, pitch lab, Live Pro Hackathon Weekend

WHAT A STUDENT NEEDS BEFORE STARTING

Age: 18+
Experience: Zero coding experience welcome, career switchers and non-technical adults thrive here
Equipment: Laptop with stable internet (16GB RAM recommended for AI dev work)
Software: All free / open-source tiers (VS Code, Cursor free tier, Vercel hobby tier, Supabase free tier)
Mindset: Willingness to ship rough first versions and iterate publicly

WHO THIS COURSE IS FOR

Career switchers: Adults moving from non-tech careers (medicine, law, finance, teaching, marketing) into tech
Working engineers: Software engineers who want to upskill on AI agents, RAG, and modern Next.js
Product managers: PMs who want to ship their own ideas without waiting for engineering
Founders: Aspiring or early-stage founders building MVPs and looking for traction

Fresh graduates: Recent college grads who want to stand out with a real portfolio
Small business owners: Entrepreneurs who want to build internal tools for their business

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS, ENGINEERING FLUENCY FOR ADULTS (WEEKS 1-4)

Adults arrive from very different starting points, some are seasoned engineers, some are PMs who've never written code, some are career switchers from medicine or finance. Phase 1 is the great equalizer. By Week 4, every participant ships a deployed full-stack app and competes in their first solo mini-hackathon.

Week 1: Hackathon Culture, Python + TypeScript Setup, Engineering Mindset

Topics:

- What hackathons reward in 2026: shippable demos, AI-native UX, clear pitches
- MLH, SIH-grad, NASA Space Apps, AngelHack, internal corporate hacks
- Setting up a pro dev environment: VS Code, Cursor, GitHub CLI, Node 22, Python 3.12
- Python 3.12 essentials: types, dataclasses, async/await, packaging
- TypeScript 5 essentials: types, generics, narrowing, async patterns
- Reading and writing kind, clear code
- Engineering values: ship rough, refactor later; correctness over cleverness
- Git deep-dive: rebase vs merge, cherry-pick, bisect
- GitHub workflow at hackathons: feature branches, draft PRs, conflict resolution
- Hacker etiquette and 'kind code review'

Projects built:

- CLI tool in Python and same tool in TypeScript
- Personal landing page deployed to Vercel
- First open-source PR to a public repo

Practice: Configure your full pro environment with Cursor, GitHub CLI, Node 22 and Python 3.12, then write the same small utility in Python and TypeScript to compare ergonomics.

Week 2: Full-Stack Fundamentals, REST, Databases, Auth

Topics:

- REST API design: resources, verbs, status codes, idempotency
- Postgres fundamentals: schemas, indexes, transactions
- Supabase fast track: DB + auth + storage + realtime in 30 minutes
- Connection pooling, migrations, seed data
- Authentication: Clerk vs Auth0 vs custom NextAuth, tradeoffs
- RBAC and row-level security
- Caching basics: HTTP, CDN, Redis
- Email + magic links + OAuth flows
- API testing with Hoppscotch
- Production-grade error handling

Projects built:

- CRUD app with Supabase + Next.js
- Auth flow with Clerk + RBAC
- Public + private routes split with middleware

Practice: Model a two table Postgres schema in Supabase with auth enabled and expose one authenticated REST endpoint with correct status codes.

Week 3: Modern Frontend, Next.js 16, Server Actions, Tailwind, shadcn/ui

Topics:

- Next.js 16 App Router fundamentals
- Server Components vs Client Components, when to use which
- Server Actions for form submissions and mutations
- Cache Components: PPR, use cache, cacheLife, cacheTag
- Streaming and Suspense
- Route handlers and middleware (proxy)
- Tailwind CSS productivity setup
- shadcn/ui, install, customize, theme
- Dark mode, responsive design, accessibility minimums
- Image and font optimization with Next.js

Projects built:

- Marketing landing page with shadcn/ui
- Dashboard with charts (Recharts) + Cache Components
- Form-heavy app using Server Actions exclusively

Practice: Build a Next.js 16 page that mixes Server and Client Components, handles a form through one Server Action and streams a slow section with Suspense.

Week 4: First Solo Pro Mini Hackathon, Ship Production-Ready in 12 Hours

Topics:

- 12-hour pro hackathon template: 1 hr ideate, 8 hr build, 1 hr deploy, 2 hr polish + pitch
- Picking a problem worth solving, adult-grade scoping
- Building solo with Cursor + Claude + Vercel preview
- Production checklist: env vars, error pages, monitoring, mobile
- Deployment with custom domain and SSL

- Recording a 2-minute demo with Loom
- Writing a judge-ready README + architecture diagram
- First pro pitch, 2 minutes with slides
- Self-grading using rubric
- Phase 1 retrospective

Projects built:
- PHASE 1 CAPSTONE, Solo Pro Mini Hackathon: 12-hour build, deployed to a real domain, 2-minute pro demo, judge-ready README

Assessment: Phase 1 Pro Mini Hackathon Score Card with mentor feedback letter

---

PHASE 2: PRO STACK, AI AGENTS, RAG, SYSTEM DESIGN (WEEKS 5-8)

This is the differentiator, what separates a winning hackathon team from a generic one.
Adults learn how production AI software is actually built today: agents with tool calling, durable workflows, RAG over private data, and the system design choices that scale.

Week 5: AI APIs & The Vercel AI SDK, Stream, Cache, Cost-Manage

Topics:
- Anthropic Claude-class frontier models, OpenAI GPT-class frontier models, Google Gemini-class frontier models, tradeoffs
- Vercel AI SDK fundamentals: generateText, streamText, tools
- Streaming responses to UI with React Suspense
- Prompt caching with Anthropic, 90% cost reduction
- Function/tool calling, turning LLMs into actuators
- Multimodal: text + image + file inputs
- Structured outputs with Zod schemas
- Cost tracking, rate limiting, fallback chains
- AI Gateway for provider failover
- Latency budgets and TTFB strategies

Projects built:
- Streaming chat app with prompt caching enabled
- Multi-provider AI gateway with failover
- Structured-output extraction from PDFs

Practice: Wire streamText from the Vercel AI SDK into a chat UI with one tool call, and measure token cost before and after enabling prompt caching.

Week 6: AI Agents, Tool Calling & Durable Workflows

Topics:
- What is an AI agent? Tools, memory, loop, termination
- Building agents with Vercel AI SDK + tool calling
- Multi-step planning and reflection
- Durable workflows with Vercel Workflow DevKit
- Pause/resume, retries, crash-safe orchestration
- Vercel Sandbox for safe code execution by AI
- Memory: short-term, long-term, semantic memory
- Evaluation: how do we know an agent is good?
- MCP (Model Context Protocol) servers, extending agent capabilities
- Common failure modes: tool loops, prompt injection, runaway costs

Projects built:
- Coding agent that writes, runs and tests code in Sandbox
- Research agent that browses, synthesizes and cites sources
- Customer support agent with persistent memory + escalation tool

Practice: Build a two tool agent, for example web lookup plus calculator, with a visible step log and a hard termination condition.

Week 7: RAG Done Right, Embeddings, Vector DBs, Hybrid Search, Eval

Topics:
- When NOT to use RAG (fine-tuning vs RAG vs prompt eng)
- Embedding models: OpenAI text-embed-3, Cohere, Voyage
- Vector databases: pgvector, Pinecone, Weaviate, Qdrant
- Chunking strategies: fixed, semantic, hierarchical
- Hybrid search: BM25 + dense embeddings + reranking
- Reranking with Cohere Rerank or Voyage Rerank
- Evaluation: groundedness, context precision, faithfulness
- RAG observability: trace every retrieval and answer
- Cost and latency tradeoffs
- When RAG fails, and how to debug

Projects built:
- RAG over your own documents (PDFs + Notion export)
- Hybrid-search RAG with reranking
- Evaluation harness with at least 50 test queries

Practice: Implement a small RAG pipeline over 20 to 50 documents with pgvector, then evaluate five queries and record where retrieval failed and why.

Week 8: System Design + Mock Pro Hackathon (Teams of 2-4)

Topics:
- System design fundamentals: load, latency, consistency, partitioning
- When to add a queue (and which queue)

- Caching layers: client, CDN, app, database
- Database scaling: read replicas, sharding, connection pooling
- Event-driven architecture, webhooks, message queues
- Observability stack: logs, metrics, traces (OpenTelemetry)
- Security: OWASP top 10, CSP, secrets management
- On-call, error budgets, postmortems
- Pro Mock Hackathon: teams of 2-4, 24-hour build window
- Live judging panel with senior engineers + founders

Projects built:
- PHASE 2 CAPSTONE, Pro Mock Hackathon: 24-hour team build with deployed product, agent or RAG component, full pitch + Q&A

Assessment: Pro Mock Hackathon Judges Panel scoring + written feedback letter

---

PHASE 3: PRO HACKATHON SPRINT, CAPSTONE, PITCH LAB & LIVE HACKATHON WEEKEND (WEEKS 9-12)

The final 4 weeks are a real, founder-grade hackathon journey. Adults lock a capstone, get matched with senior mentors (engineers, founders, VCs as needed), build with production discipline, get founder-grade pitch coaching, and compete in the Live Pro Hackathon Weekend with sponsor prizes.

Week 9: Capstone Lock, Spec, Architecture, Cost Model, GTM Sketch

Topics:
- Idea finalists, picking a capstone with both judge-appeal and founder-potential
- Writing a 2-page Product Spec (problem, users, JTBD, MVP, stretch)
- Architecture diagram with picked tech stack and rationale
- Cost model: AI tokens, infra, third-party APIs
- Risk register and mitigation plan
- Mentor matching with senior engineers / founders
- Setting up CI, monitoring, error tracking from day one
- Drafting a 3-minute investor pitch outline
- GTM sketch: who, where, how (even for a hackathon, judges love this)
- Repo, project board, daily commit cadence rules

Projects built:
- Product Spec (2 pages)
- Architecture Diagram with cost model
- First investor-style pitch outline

Practice: Deliver your two page capstone spec with an architecture diagram, a token and infra cost model, and a risk register with mitigations.

Week 10: Heavy Build Sprint, Production Discipline at Hackathon Speed

Topics:
- Daily commits with great messages
- Pair programming with teammates
- AI-assisted coding with Cursor + Claude + Codex
- Mid-sprint mentor reviews and code walkthroughs
- Test critical paths (smoke + at least one E2E)
- Loading / empty / error states across the app
- Mobile and accessibility, judges open on phones
- Performance: Core Web Vitals, bundle size, edge runtime
- Observability: every error visible in Sentry / Vercel logs
- Daily standup logs to mentor

Projects built:
- Capstone build, 70% complete by end of Week 10
- Daily standup logs (3 lines: shipped / shipping / blocked)
- Mid-sprint demo to mentor + 3 peers

Practice: Ship daily for five days: clean commits, one mid sprint mentor review actioned, and one end to end test guarding your demo path.

Week 11: Founder-Grade Pitch Lab, VC Critique, Slides, Demo Polish

Topics:
- Hackathon judging rubric (Innovation 25 / Execution 25 / Design 25 / Pitch 25)
- VC-style judging: 'why now, why you, why this big'
- The 3-minute pro pitch: Hook Problem Solution Demo Architecture Traction (if any) Team Ask
- Slide design, minimal, image-first, 8 slides max
- Live VC-style critique sessions with senior mentors
- Recording a high-quality demo (multiple takes, edited in Loom or DaVinci)
- Voice and pacing tips from professional pitch coaches
- Q&A handling, 'I don't know' done well
- Final README polish, judges read this first
- Final code freeze, deployment health check, mobile/dark mode/edge cases

Projects built:
- Pitch Slides (8 slides, Pitch.com / Tome / Canva)
- Pitch Video (3 minutes)
- Final README with hero image, demo URL, architecture, tech stack, team, GTM sketch

Practice: Rehearse the three minute pitch on camera twice, tighten the deck to eight image first slides and prepare answers to the three hardest judge questions.

Week 12: LIVE PRO HACKATHON WEEKEND, Compete with Founders & Senior Engineers

Topics:
- Friday: opening keynote, sponsor announcements, theme reveal, team check-in
- Saturday: 18-hour final sprint with senior mentor support
- Saturday evening: pitch dress rehearsal with VC-style critique
- Sunday morning: 4-hour final polish + production deploy
- Sunday afternoon: live judging, every team pitches in front of senior judges
- Sponsor categories: Best AI Agent, Best RAG, Best Founder Potential, Best Architecture, Best Pitch, Most Innovative, Audience Favorite
- Live judge Q&A with founders and CTOs
- Awards ceremony with cash prizes, cloud credits, mentor calls and founder intros
- Hall of Fame inclusion + sponsor follow-up calls
- Graduation: Pro Hackathon Builder Certification awarded

Projects built:
- GRAND CAPSTONE, Live Pro Hackathon Submission: production-deployed product + GitHub repo + 3-minute investor pitch + 8-slide deck + judge-ready README + live presentation + Q&A

Assessment: Final Pro Hackathon Score + Founder Feedback Letter

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Weeks 1-4): 6+ full-stack projects + Solo Pro Mini Hackathon (deployed)
- Phase 2 (Weeks 5-8): 6+ AI/agent/RAG/system-design projects + Pro Mock Hackathon
- Phase 3 (Weeks 9-12): Capstone build + investor pitch deck + 3-min demo + LIVE Pro Hackathon submission

Total projects: 12+ deployed projects + Live Pro Hackathon Capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Python 3.12 and TypeScript 5
- Next.js 16 (App Router + Server Actions + Cache Components)
- Tailwind + shadcn/ui
- Supabase + Postgres + Redis + pgvector
- REST + tRPC + edge functions
- Vercel AI SDK + tool calling + agents
- Workflow DevKit + Sandbox + durable execution
- RAG: chunking, embeddings, hybrid search, reranking, eval
- Multi-vector-DB familiarity (pgvector, Pinecone, Weaviate, Qdrant)
- System design: queues, caches, sharding, event-driven
- Observability: Sentry, OpenTelemetry, Vercel logs
- Security: OWASP top 10, CSP, secrets
- Performance: Core Web Vitals, edge, ISR, streaming
- GitHub workflow with PRs and CI
- Founder-grade pitch decks and 3-minute demos

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 sessions (2 hours each, evening or weekend)
Independent projects: 4 hours
Weekly pro hack: 1 themed hackathon-style challenge (1-2 hours)
Total per week: 8-10 hours

SUPPORT INCLUDED

Live instruction: Senior engineers, ex-CTOs, YC-alumni founders
Mentor matching: 1-on-1 senior mentor in capstone phase
Pitch lab: Founder-grade VC-style pitch coaching
Career support: 1-on-1 LinkedIn + portfolio review for career switchers
Founder support: Mentor founders matched for capstone if you're building a startup
Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Final certificate: Pro Hackathon Builder, Modern Age Coders Certified
Capstone required: Live hackathon submission (production-deployed product + investor pitch + demo)
Portfolio: 12+ deployed projects + grand capstone

WHERE THIS CAN LEAD

- AI Engineer at startups (Rs 15-60 LPA in India, $80k, $200k globally)
- Full-Stack Engineer at YC / mid-stage startups
- Product Engineer / Product Manager
- Indie Hacker / Solo Founder
- Open-Source Maintainer
- Career Switcher into tech (first tech role)
- Founder of a venture-backed startup
- Hackathon-Champion track (sponsor pipelines, accelerator interviews)

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Other roles: Rs 6-18 LPA in first tech role; Rs 20-60 LPA for senior AI / full-stack roles; Pre-seed funding range typically $50k, $500k from accelerators or angels; $80k, $200k for AI / full-stack engineers from India working remote

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is the hackathon really free for adults?
A: Yes, 100% free for anyone aged 18+. No fee. No employer or college endorsement needed. Free certificate. Eligible for all sponsor prizes.

Q: Why is the prep course paid?
A: The prep course is optional. It's for adults who want serious, structured 12-week training in modern engineering, AI agents, RAG, system design and pitching. The hackathon participation stays free regardless.

Q: I'm a non-technical adult, can I still do this?
A: Yes, and you're our favorite kind of student. Many of our top hackathon winners have come from non-technical backgrounds (medicine, law, finance, teaching). Phase 1 starts from absolute zero. AI tools (Cursor, Replit Agent, Bolt.new) close the gap dramatically. Within 4 weeks, you'll be deploying full-stack apps.

Q: I'm a senior engineer, will this be too basic?
A: No. The Phase 2 content (AI agents, RAG done right, durable workflows, system design at hackathon scale, observability) is genuinely current and challenges senior engineers. We also let experienced engineers fast-track Phase 1 in 2 weeks if they pass a placement test.

Q: What's the time commitment?
A: Plan for 8-10 hours weekly: 2 live sessions (2 hours each, evenings or weekends), 4 hours of independent project work, and 1-2 hours for the weekly pro hackathon-style challenge. Small batches make missed classes easy to recover: the mentor recaps in the next session, notes and the week's challenge are shared over WhatsApp, and longer absences get a short free catch-up slot. for working professionals who can't always attend live.

Q: What stack will I learn?
A: The 2026 modern stack: Python 3.12 + TypeScript 5, Next.js 16 (App Router + Cache Components), Tailwind + shadcn/ui, Supabase / Postgres / Redis / pgvector, Vercel AI SDK + Workflow DevKit + Sandbox, Anthropic Claude-class frontier models / OpenAI GPT-class frontier models / Gemini-class frontier models, plus AWS / Cloudflare for production deployment. Cursor and AI agents are first-class throughout.

Q: Will this help me switch careers into tech?
A: Yes, directly. Career switchers are a big part of our cohort. You graduate with a deployed capstone, a public GitHub portfolio, a recorded pitch, a Pro Hackathon Builder certification, and a 1-on-1 LinkedIn and portfolio review. We help you present your work well for applications and interviews, though outcomes depend on your effort and the market.

Q: Can founders use this to validate their startup idea?
A: Absolutely. Many founders use the capstone phase to build their MVP. We pair you with mentor founders if you choose. Several alumni have raised pre-seed funding off their hackathon capstone.

Q: Are there sponsor prizes?
A: Yes. Sponsor lineup announced 2 weeks before each quarterly hackathon.

Q: Do you cover system design and DSA?
A: System design, yes, deeply, at hackathon depth (queues, caches, sharding, event-driven). DSA, to the level you need to write efficient hackathon code. For deep DSA prep (interview-grade), we recommend our Vibe Coding for College / DSA Masterclass program.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/hackathon-prep-for-adults-professionals-coding-ai-innovation-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 77 OF 137: HACKATHON FOR KIDS: FREE EVENT + 12-WEEK PREP (AGES 8-12)
==========

Course link: https://learn.modernagecoders.com/courses/hackathon-prep-for-kids-coding-innovation-ai-projects-course
Subject area: Hackathons & Innovation Programs
Designed for: Kids

Who can join: Complete Beginner (Ages 8-12)
Course length: 12 weeks prep + Live Hackathon Weekend
Time needed: 4-6 hours/week
Certificate: Junior Hackathon Builder, Modern Age Coders Certified

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Free hackathon for kids ages 8-12, plus a 12-week prep course in coding, AI, problem-solving and innovation. Build a project, pitch it and win judge prizes.

  In one line: Anyone can join. Free to participate. Built to spark India's youngest innovators.

ABOUT THE PROGRAMME

  Hackathon for Kids is a free-to-enter hackathon and a 12-week structured prep course at Modern Age Coders, designed for children aged 8 to 12 who want to learn how real innovators build technology. The hackathon participation is 100% free, anyone, from any school or city, can register and compete. The optional 12-week prep course (priced the same as our other coding courses) is for kids who want serious training before they compete. Our only motive is simple: build something great. Build something innovative. Use coding logic, efficient algorithms and AI tools to turn raw ideas into real, working projects in just one weekend. Kids learn how a real hackathon works, how teams form, how ideas get scoped, how problems are decomposed, how AI is used to accelerate building, how you debug under pressure, and how you pitch your project to judges. They use Scratch, Replit, Bolt.new, Google Gemini, Microbit and Canva to build games, AI apps, websites and inventions that solve real problems for kids their age. This is not a normal class. This is a creator's training camp. By the end, every child has a portfolio of 4+ hackathon-ready projects, a 90-second pitch video, a working capstone submission, and the confidence to walk into any school, district or international hackathon and stand on stage.

WHAT MAKES THIS COURSE DIFFERENT

  - 100% free hackathon participation, anyone can join, no fees, no gatekeeping
  - Hackathon prep priced same as other Modern Age Coders courses, group, mini-batch and 1-on-1 monthly plans
  - Designed by mentors who have judged and won 50+ hackathons
  - Kids learn the full hackathon arc: ideate plan build debug pitch demo
  - AI-assisted building with Replit Agent, Bolt.new and Google Gemini
  - Algorithm thinking made playful, sorting, searching, pathfinding through games
  - Live mock hackathons every 4 weeks with judge feedback
  - Team formation workshops, kids learn collaboration and roles (Builder, Designer, Pitcher)
  - Pitch coaching, every child practices a real 90-second demo to camera
  - Final hackathon weekend with prizes, judge panel, and Hall of Fame inclusion
  - Weekly written progress notes to parents

THE LEARNING PATH, STEP BY STEP

  Phase 1: Hackathon Foundations (Weeks 1-4): How hackathons work, coding basics in Scratch, problem-solving, idea generation
  Phase 2: Build Skills (Weeks 5-8): AI tools, Replit & Bolt.new, basic algorithms, team-building, design with Canva
  Phase 3: Hackathon Sprint (Weeks 9-12): Mock hackathons, capstone build, pitch coaching, demo day, live hackathon weekend

WHAT A STUDENT NEEDS BEFORE STARTING

  Age: 8-12 years old
  Experience: Absolutely none, beginners welcome
  Equipment: A computer or laptop with internet connection
  Software: All tools are free and browser-based, nothing to install
  Parental support: Encouraged to attend final demo day

WHO THIS COURSE IS FOR

  Curious kids: Children aged 8-12 who love building things
  Competitive kids: Kids who enjoy challenges, time-limits and judges
  Shy kids: Quiet kids who want a safe place to find their voice (pitch coaching changes lives)
  Ai curious kids: Kids who want to learn AI tools the right way
  School innovation clubs: School coding club members preparing for inter-school hackathons

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: HACKATHON FOUNDATIONS, HOW HACKATHONS WORK (WEEKS 1-4)

Kids learn what a hackathon is, why it exists, how teams build software in 48 hours, and how to start thinking like an inventor. They build their first 'mini hack' projects in Scratch and learn the language of hackathons: idea, MVP, sprint, demo and pitch.

Week 1: What is a Hackathon?, Welcome to the Builder Mindset

Topics:

- What is a hackathon? Origin story from Silicon Valley to Indian schools
- Famous kids who built real apps in hackathons (case studies)
- The 4 stages of every hackathon: Idea Build Demo Judge
- What is an MVP (Minimum Viable Product)?, explained for kids
- Why anyone can join: free hackathon access at Modern Age Coders
- Setting up your hacker profile: name, team, project board
- Tour of Scratch 3.0, your first build platform
- First mini-hack: build a 'Welcome' interactive card in 60 minutes
- Hackathon vocabulary kids need: bug, ship, push, demo, judge, pitch
- Online safety, kindness in teams, and being a good teammate

Projects built:

- 60-Minute Mini Hack: Animated Welcome Card
- Hacker ID Card (designed in Canva)
- Idea Wall, write 10 things in the world you'd like to fix

Practice: Watch one short hackathon demo video with a parent and write down the problem, the solution and your favorite part of the pitch.

Week 2: Idea Generation, Where Hackathon Ideas Come From

Topics:

- How to spot problems worth solving (kids edition)
- The 'I wish this existed' game, how every great app started
- Idea generation techniques: brainstorming, mind maps, sticky notes
- Good ideas vs. great ideas, judging your own thoughts
- User empathy for kids, designing for a friend, sibling, or grandparent
- Competitor research made fun, what already exists?
- Picking your problem statement and writing it in one sentence
- Sketching your first wireframe by hand and on paper
- Storyboarding your idea, 6 panels, like a comic strip
- Sharing ideas in a group without fear, feedback rules

Projects built:

- My Idea Comic, 6-panel storyboard of your hackathon dream project
- Problem Statement Card (one sentence)
- Empathy Interview, interview a family member about their daily problems

Practice: Keep an 'I wish this existed' notebook all week and bring your three best problem ideas to class.

Week 3: Coding Logic, Loops, Conditions & Variables in Scratch

Topics:

- Loops: how computers repeat tasks, the foundation of efficient code
- If-Then-Else: making computers make decisions
- Variables: storing scores, names, and data
- Why every hackathon project needs these 3 building blocks
- Sequence, selection, repetition, the holy trinity of coding logic
- Building a number guessing game, your first 'algorithm'
- Debugging: finding bugs without crying, printing values to investigate
- Comments and notes, making your code readable to teammates
- Hackathon-style challenges: 30-minute build sprints
- Code reviews with classmates, being kind and helpful

Projects built:

- Number Guessing Game with hints
- Score-Tracking Quiz Game (5 questions)
- Loop Art, generate a star pattern using only loops

Practice: Build a mini Scratch scene that uses one loop, one if then else and one variable, and label each with a comment.

Week 4: First Mini Hackathon (Solo), Build & Pitch in 4 Hours

Topics:

- Hackathon pacing: how to use 4 hours wisely
- The 4-hour template: 30 min ideate, 2.5 hr build, 30 min polish, 30 min pitch
- Choosing scope: what NOT to build is more important than what to build
- Building your project from a problem statement
- Time-boxing: setting timers and sticking to them
- What to do when stuck (the '15-minute rule')
- Recording a 60-second demo video with Loom or screen capture
- Writing your project's README, title, problem, solution, how to use
- Pitching to mentors: how to talk about your build with confidence
- Phase 1 review and Phase 1 capstone presentation

Projects built:

- PHASE 1 CAPSTONE, Solo Mini Hackathon: 4-hour build + 60-second demo + 1-page README. Choose any problem from your Idea Wall.

Assessment: Phase 1 Mini Hackathon Assessment, judged on innovation, execution, clarity

---

PHASE 2: BUILD SKILLS, AI, ALGORITHMS & TEAM HACKING (WEEKS 5-8)

Kids level up to AI tools, learn the basics of algorithms (the secret sauce of efficient hackathon projects), discover Replit and Bolt.new, and start working in teams of 2-3. By Week 8, they will have built their first AI-powered hackathon project with a partner.

Week 5: Vibe Coding with AI, Build Apps by Talking to Computers

Topics:

- What is AI and how do hackers use it to ship faster?
- Meet Replit Agent, describe an app, watch it appear
- Meet Bolt.new, full apps from a single sentence
- Prompt writing for hackathons: be specific, be visual, be small
- Iterating with AI: 'change the color', 'add a button', 'fix this bug'
- AI vs. human coding: when to use which
- Reading AI-generated code without panicking
- Asking Google Gemini to explain code line-by-line
- Limits of AI, what it gets wrong, why you still need to think
- Ethical AI use in hackathons, credit, originality, transparency

Projects built:

- AI-Built Calculator with custom theme (Replit Agent)
- AI-Built Quiz App on a topic you love (Bolt.new)
- AI-Explained Code Journal, print 3 AI snippets and write what each line does

Practice: Ask Replit Agent or Bolt.new for a one screen app about your hobby, then request two changes: a new color and an extra button.

Week 6: Algorithms for Kids, Sorting, Searching & Pathfinding

Topics:

- What is an algorithm? Simple recipe metaphor
- Why algorithms matter in hackathons (slow apps lose, fast apps win)
- Sorting: bubble sort visualized as kids in a line
- Searching: linear vs. binary, finding the right book on the shelf
- Pathfinding basics, the maze game
- Big O for kids: 'fast', 'slow', 'really slow' (no math symbols yet)
- Choosing the right algorithm for your hackathon idea
- Using AI to suggest algorithms when stuck
- Visualizing algorithms in Scratch with sprites
- Mini-challenge: implement 2 sorts and race them

Projects built:

- Sorting Race, animate bubble sort vs. insertion sort with sprites
- Maze Solver, a sprite that finds the exit using a simple algorithm
- Word Search Game, implement linear search visually

Practice: Explain bubble sort to a family member using playing cards lined up in a row, then repeat it and try to sort faster.

Week 7: Team Hacking, Roles, GitHub Basics & Collaboration

Topics:

- Why hackathons love teams of 2-3
- The 3 hackathon roles: Builder (writes code), Designer (makes it pretty), Pitcher (tells the story)
- Picking a teammate, strengths and personalities
- Communication tools: Google Docs, Figma Jam, voice notes
- Sharing code through Replit Multiplayer (kid-safe)
- Introduction to GitHub, what is a repository?
- Forking, committing and pushing, explained with playground analogies
- Resolving 'who builds what' fights without drama
- Daily 5-minute stand-ups: what I did, what I'm doing, what I need help with
- Sharing credit, every teammate's name on the project

Projects built:

- Team Charter, 1 page on roles, rules, and project goals
- Pair-Built Story App, two kids, one app, built collaboratively in Replit
- Mini GitHub Profile, your first repo with your project README

Practice: Run a 30 minute mini build in Replit Multiplayer with a friend or sibling where one builds and one designs, then swap roles.

Week 8: Mock Hackathon (Teams), 8-Hour Sprint Across Two Days

Topics:

- Forming hackathon teams (2-3 kids, mentor-paired)
- Selecting a problem statement from a curated list
- Day 1: 4-hour build sprint with Replit + Bolt.new + Scratch
- Day 2: 2-hour polish + 1-hour pitch prep + 1-hour live demos
- Time-boxing: how mentors keep teams from over-engineering
- Mid-sprint check-ins with mentors
- Designing a 1-slide pitch deck in Canva
- Recording a 90-second demo video together

- Live presentation to peer judges
- Receiving feedback gracefully, what worked, what to improve

Projects built:
- PHASE 2 CAPSTONE, Team Mock Hackathon: pick a problem, ship an MVP in 8 hours, deliver a 90-second demo with team intro and judge Q&A

Assessment: Mock Hackathon Judges Panel, innovation, technical execution, design, pitch clarity, teamwork

---

PHASE 3: REAL HACKATHON SPRINT, CAPSTONE, PITCH & LIVE HACKATHON WEEKEND (WEEKS 9-12)

The final phase is a real hackathon journey. Kids pick their dream project, get paired with mentors, build their capstone over 4 weeks, get pitch coaching, record final demos, and compete in the official Modern Age Coders Live Hackathon Weekend at the end of Week 12, the same hackathon that's free for the public.

Week 9: Capstone Idea Lock, Mentor Matching & Project Plan

Topics:
- Picking your capstone problem statement (3-5 idea finalists)
- Mentor matching, every team gets a personal hackathon mentor
- Writing the capstone proposal: problem, users, solution, tech stack
- Hackathon judging rubric explained (innovation 25%, execution 25%, design 25%, pitch 25%)
- Designing your project board, Trello-for-kids visual planning
- Stretch goals vs. core MVP
- Risk planning: what could go wrong and how we'll handle it
- First architecture sketch (boxes-and-arrows)
- Setting up your repo, your team channel, your Canva board
- Locking the scope, saying NO to feature creep

Projects built:
- Capstone Proposal Document
- MVP Definition (one paragraph)
- Architecture Sketch (paper or Figma Jam)

Practice: Fill your capstone project board with To Do, Doing and Done columns and at least eight task cards.

Week 10: Build Sprint, Code, AI, Iterate, Repeat

Topics:
- Day-by-day build plan (kids learn to plan their week)
- Working with AI tools at high speed
- Debugging strategies: rubber duck debugging, AI-assisted debugging
- Component thinking, break big projects into small reusable parts
- Asset creation in Canva, backgrounds, sprites, icons
- Sound and music for projects (free libraries)
- Mid-week mentor reviews: what's done, what's blocked
- Pair programming with your teammate
- Quality bar: what 'good enough' looks like for a kids hackathon
- Daily commits, writing kind, clear messages

Projects built:
- Capstone build, 70% complete by end of Week 10
- Daily standup logs (5 lines per day)
- Mid-sprint demo to mentor

Practice: Finish two build tasks from your project board and note one bug you found plus how you or the AI fixed it.

Week 11: Pitch Coaching & Demo Polish, Tell the Story

Topics:
- What judges actually look for: clarity, impact, energy, originality
- The 90-second pitch formula: Hook Problem Solution Demo Ask
- Practicing in front of a mirror, then a parent, then your team
- Recording your pitch on camera (5+ takes)
- Slide design in Canva, title, problem, solution, tech, team, thank you
- Demo recording with Loom, show, don't tell
- Voice tips for kids: pace, pauses, smiles
- Handling Q&A, what to say when you don't know the answer
- Making your project README judge-ready (problem, demo link, screenshots, team)
- Final polish on your build: bugs, edge cases, mobile view

Projects built:
- Pitch Slides (5 slides, Canva)
- Pitch Video (90 seconds, recorded)
- Final README (judge-ready)

Practice: Record your 90 second pitch on camera at least three times and pick the take where your hook sounds most exciting.

Week 12: LIVE HACKATHON WEEKEND, Compete, Demo & Get Crowned

Topics:
- Friday evening: kickoff ceremony, theme announcement, team check-in
- Saturday all-day: 8-hour final sprint with mentor support
- Saturday evening: pitch dress rehearsal
- Sunday morning: final 2-hour polish + submission upload

- Sunday afternoon: live judging, every team presents
- Categories: Most Innovative, Best Built, Best Designed, Best Pitch, Audience Favorite
- Live judge Q&A, handled with grace and confidence
- Awards ceremony, certificates, badges and Hall of Fame inclusion
- Reflection circle: what I learned about myself
- Graduation: Junior Hackathon Builder Certification awarded

Projects built:
- GRAND CAPSTONE, Live Hackathon Submission: working project + 90-second demo video + 5-slide pitch deck + judge-ready README + live presentation

Assessment: Final Hackathon Scoring + Judge Feedback Letter

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Weeks 1-4): 6+ Scratch mini-hacks + Solo Mini Hackathon
- Phase 2 (Weeks 5-8): 6+ AI/algorithm projects + Team Mock Hackathon
- Phase 3 (Weeks 9-12): Capstone build + Pitch deck + Demo video + LIVE Hackathon submission

Total projects: 12+ projects + Grand Live Hackathon Capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Hackathon Mindset & MVP scoping
- Scratch 3.0 programming logic
- Vibe Coding with Replit Agent and Bolt.new
- AI Tools: Google Gemini, Teachable Machine
- Algorithm fundamentals: sorting, searching, pathfinding
- Team collaboration with Builder/Designer/Pitcher roles
- Canva pitch deck design
- Loom demo video recording
- Public speaking and 90-second pitches
- Project documentation: README, problem, solution

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 sessions (1.5 hours each)
Independent projects: 1-3 hours
Vibe challenge: 1 themed weekly hack
Total per week: 4-6 hours

SUPPORT INCLUDED

Live instruction: Expert hackathon mentors with judging experience
Age appropriate: Designed exclusively for ages 8-12
Safe environment: Moderated, kind, bully-free classroom
Community: Junior Hackathon Builders Hall of Fame
Progress updates: Weekly written progress notes to parents

CERTIFICATION DETAIL

Final certificate: Junior Hackathon Builder, Modern Age Coders Certified
Capstone required: Live hackathon submission with demo and pitch
Portfolio: 12+ projects + grand capstone

WHERE THIS CAN LEAD

- Junior Hackathon Builder
- Future Coding Olympiad Participant
- School Innovation Club Leader
- Ready for Hackathon for Teens program
- Foundation for science fairs, robotics leagues and pitch competitions

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is the hackathon really free for kids?
A: Yes, 100% free. Any child aged 8-12 can register and compete in the Modern Age Coders Live Hackathon at zero cost. No fees, no hidden charges, no payment required to participate, get judged, or receive a certificate.

Q: Then why is there a paid prep course?
A: The prep course is optional. It's for parents and kids who want serious, structured training before the hackathon, coding skills, AI tools, algorithm thinking, team practice and pitch coaching. Many kids do the prep, many compete without it. Both are valid paths.

Q: Does my child need any prior coding experience?
A: Absolutely not. The prep course starts from zero. Kids learn Scratch, basic logic, AI tools, and team-building from scratch. Even kids who have never touched a computer in coding mode are welcomed.

Q: What is the age range?
A: Ages 8-12. The content, pace, projects and pitch coaching are all designed for this age group. We have separate Hackathon programs for Teens (13-17) and Adults (18+).

Q: How much time will the prep course take per week?
A: Plan for 4-6 hours weekly: 2 live classes (1.5 hours each) plus 1-3 hours of independent project work. Small batches make missed classes easy to recover: the mentor recaps in the next session, notes and the week's challenge are shared over WhatsApp, and longer absences get a short free catch-up slot. so your child never falls behind if they miss one.

Q: What does my child build in the prep course?
A: Over 12 weeks, your child will build at least 4 mini-hackathon projects, 1 team project, and 1 grand capstone, all judge-ready with demo videos, README documentation, and Canva pitch decks. Plus they compete in the live free hackathon at the end.

Q: Do they really learn algorithms at age 8?
A: Yes, visually and playfully. We teach sorting, searching and pathfinding through Scratch animations, mazes and games. No math heavy notation. The goal is to build algorithm intuition, not memorize Big O. This intuition compounds for years.

Q: What do they get at the end?
A: A Junior Hackathon Builder certificate (Modern Age Coders Certified), a public portfolio of 5+ projects, a 90-second pitch video, a Hall of Fame listing if they place top-10, and direct admission preference to our Hackathon for Teens program.

Q: What tools will my child use?
A: Scratch 3.0, Code.org, Replit, Bolt.new, Google Gemini, Google Teachable Machine, Canva, Figma Jam, Loom, MakeCode Arcade, and basic GitHub. All free, browser-based, and kid-safe.

Q: Are there refunds if my child doesn't enjoy it?
A: Yes. If you are not satisfied within the first 2 weeks of the prep course, we offer a full refund, no questions asked. The free hackathon participation is, of course, free.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/hackathon-prep-for-kids-coding-innovation-ai-projects-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 78 OF 137: HACKATHON FOR TEENS: FREE EVENT + 12-WEEK PREP (AGES 13-17)
==========

Course link: https://learn.modernagecoders.com/courses/hackathon-prep-for-teens-coding-ai-build-innovate-win-course
Subject area: Hackathons & Innovation Programs
Designed for: Teens
Who can join: Beginner-Friendly to Intermediate (Ages 13-17)
Course length: 12 weeks prep + Live Hackathon Weekend
Time needed: 6-8 hours/week
Certificate: Teen Hackathon Builder, Modern Age Coders Certified

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Free hackathon for teens 13-17, plus a 12-week prep course in Python, JavaScript, AI APIs, algorithms and pitching. Build apps and win judge prizes.

In one line: Free hackathon for everyone. Serious prep for those who want to win.

ABOUT THE PROGRAMME

Hackathon for Teens is a free-to-enter hackathon and a 12-week structured prep course at Modern Age Coders for students aged 13 to 17. The hackathon participation is 100% free for everyone, no fees, no school filtering. The prep course is for teens who want to walk into a hackathon with real coding skills, AI fluency, and a proven pitch playbook. Our only motive is simple: build something great. Build something innovative. Use coding logic, efficient algorithms, AI APIs, and modern dev workflows to ship real software in 48 hours. We don't teach kids to copy templates. We teach them how engineers actually build. Teens go deep, Python and JavaScript fundamentals, REST API design, algorithm complexity (Big O), data structures (arrays, hash maps, trees, graphs), AI APIs (OpenAI, Gemini, Claude), GitHub-based collaboration, system design lite, and end-to-end deployment on Vercel. They build hackathon-ready apps using the same stack used by professional engineers in 2026, React,

Next.js, Tailwind, Supabase, Replit Agent, Bolt.new, Cursor and Vercel. By Week 12, every teen has a polished GitHub portfolio of 6+ projects, a public capstone deployed live on the internet, a 2-minute pitch video, and a placement on the Modern Age Coders Hackathon Hall of Fame. They graduate hackathon-ready, for school events, MLH-style competitions, Smart India Hackathon, and global online hackathons.

WHAT MAKES THIS COURSE DIFFERENT

- Free hackathon participation, anyone aged 13-17 can register at zero cost
- Prep priced the same as our other teen courses, monthly or 6 months at once: see Fees and enrolment at the top of this page
- Curriculum designed by hackathon winners and senior engineers
- Real production stack: React, Next.js, Tailwind, Supabase, Vercel
- AI-first dev workflow with Cursor, Replit Agent, Bolt.new
- Algorithms taught for hackathons, practical, not academic
- GitHub-native: every project committed, branched, deployed
- Live deployments, every project gets a real URL judges can click
- 1-on-1 mentor matching for capstone phase
- Pitch deck lab with senior pitch coaches
- Mock hackathons every 4 weeks with judge feedback
- Final hackathon weekend, live judges, prizes, Hall of Fame

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Weeks 1-4): Python + JavaScript essentials, Git/GitHub, problem-solving
Phase 2: Build Stack (Weeks 5-8): React + Tailwind + Supabase, AI APIs, algorithms & data structures, system design lite
Phase 3: Hackathon Sprint (Weeks 9-12): Capstone build, mentor coaching, pitch lab, Live Hackathon Weekend

WHAT A STUDENT NEEDS BEFORE STARTING

Age: 13-17 years old
Experience: Zero coding experience welcome, strong fit for self-starters
Equipment: Laptop or desktop with stable internet (8GB RAM recommended)
Software: All free / open source (VS Code, Cursor free tier, Vercel hobby tier)
Mindset: Willingness to ship rough, ugly first versions

WHO THIS COURSE IS FOR

Ambitious teens: Teens aiming for top colleges and want a real portfolio
Competitive teens: Teens who love coding contests, MLH, SIH, Olympiad
Founder curious: Teens who want to learn how startups actually build
Ai native teens: Teens who want AI fluency at the API level (not just ChatGPT chat)
Career planners: Teens planning a tech career with strong GitHub from day one

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS, PYTHON, JAVASCRIPT & GIT FOR HACKERS (WEEKS 1-4)

Teens build serious coding fluency. We cover Python (logic + scripting), JavaScript (the universal language of the web), and Git/GitHub workflow that every hackathon team uses. By Week 4, every student can ship a small web app to a real URL.

Week 1: Hackathon Culture, Python Basics & The Hacker Mindset
Topics:
- What is a hackathon really? MLH, SIH, NASA Space Apps, internal startup hacks
- Famous teen hackathon winners (cases studies from India and globally)
- The hackathon arc: idea MVP demo pitch judge
- Python install + setup (with VS Code)
- Variables, types, operators, strings, lists, dicts
- Conditionals (if/elif/else), loops (for/while)
- Functions, return values, default args
- Reading inputs, printing outputs, basic file IO
- Your first Python mini-tool: a CLI to-do list
- Hacker etiquette: be kind, be curious, ship over perfect
Projects built:
- CLI To-Do List in Python
- Number guessing game with hints + difficulty levels
- Data formatter: read a CSV, print summary stats
Practice: Set up Python and VS Code, then write a script that stores five hackathon ideas in a list of dicts and prints them ranked by excitement.

Week 2: JavaScript Essentials & DOM, The Universal Hackathon Language
Topics:
- Why JavaScript runs the modern web (and most hackathons)

- let / const, types, functions, arrow functions
- Arrays, objects, destructuring, spread / rest
- Async/await, promises, fetch, calling APIs
- DOM manipulation, event listeners
- ES Modules and modern JS structure
- TypeScript intro, types make hackathon code safer
- Building a client-side mini app
- Browser DevTools for debugging
- Common JS gotchas (== vs ===, hoisting, this)

Projects built:
- Weather App calling a public weather API
- Markdown live preview editor
- GitHub user profile fetcher

Practice: Build a small webpage that fetches from a public API with async await and updates the DOM when a button is clicked.

Week 3: Git, GitHub & Team Coding, Professional Workflow

Topics:
- Git fundamentals: init, add, commit, log, diff
- Branches, merging, conflicts and how to resolve them
- GitHub: repos, README, issues, pull requests
- Forking and cloning open-source projects
- Pull request reviews and team etiquette
- GitHub Actions intro, automated CI
- GitHub Pages deployment
- Hackathon team workflow: feature branches, PR per feature
- Daily standups: what I did, what I'm doing, blockers
- Hackathon-grade README structure (problem, demo, screenshots, setup, team)

Projects built:
- Personal Portfolio site deployed via GitHub Pages
- Open-source contribution: file 1 PR to a public repo
- Team-built mini app with branches and PRs

Practice: Create a repo with a proper README, make five meaningful commits on a branch, then open and merge your own pull request.

Week 4: First Mini Hackathon (Solo), Ship to a Live URL in 8 Hours

Topics:
- 8-hour hackathon template: 1 hr ideate, 5 hr build, 1 hr polish, 1 hr pitch
- Picking a problem statement that's small, clear, and demo-able
- Building solo with Replit Agent + Cursor for speed
- Deployment to Vercel, live URL judges can click
- Recording a 90-second demo with Loom
- Writing your README so judges understand in 60 seconds
- First pitch attempt, to camera, then to mentors
- Self-grading with the official rubric
- Code reviews from senior mentors
- Phase 1 retrospective and feedback letter

Projects built:
- PHASE 1 CAPSTONE, Solo Mini Hackathon: 8-hour build, deployed to Vercel, 90-second demo, judge-ready README

Assessment: Phase 1 Mini Hackathon Score Card, innovation, execution, design, pitch

---

PHASE 2: BUILD STACK, REACT, AI APIS, ALGORITHMS & SYSTEM DESIGN (WEEKS 5-8)

Teens learn the full modern hackathon stack. By Week 8, they can build a deployable React + Tailwind + Supabase app with an AI feature in under 24 hours.

Week 5: React + Tailwind, The Hackathon UI Stack

Topics:
- React fundamentals: components, props, state, hooks
- useState, useEffect, useRef in real projects
- Tailwind CSS, design without writing CSS files
- Building responsive layouts in minutes
- shadcn/ui components, copy/paste production-grade UI
- Forms, validation, controlled components
- Routing with Next.js App Router
- Server Components vs Client Components
- Iconography, dark mode, accessibility basics
- Building UI from a Figma mock in under 30 min

Projects built:
- Personal landing page with React + Tailwind
- Recipe finder calling an external API
- Interactive dashboard with charts (Recharts)

Practice: Rebuild your week 2 page in React with Tailwind and one shadcn/ui component, keeping it responsive at phone width.

Week 6: Backend with Supabase + REST APIs, Build a Real Product

Topics:

- Supabase setup, database, auth, storage in 5 minutes
- Postgres tables, schema design for hackathons
- Row-Level Security basics (auth that doesn't leak)
- Building REST APIs with Next.js Route Handlers
- CRUD apps end-to-end
- Email + magic link authentication
- File uploads to Supabase Storage
- Webhooks and realtime subscriptions
- Postman / Thunder Client for testing APIs
- Connecting frontend backend smoothly

Projects built:
- Notes app with auth + Supabase database
- Image gallery with uploads + storage
- Realtime chat (single-room) with Supabase Realtime

Practice: Stand up a Supabase project with one table protected by Row Level Security and wire a working CRUD form to it from Next.js.

Week 7: AI APIs, Build AI-Powered Hackathon Projects

Topics:
- OpenAI API: chat completions, function calling
- Google Gemini API: multimodal inputs (text + image)
- Anthropic Claude API: long-context, tool use, prompt caching
- Streaming responses to your UI
- RAG (Retrieval Augmented Generation) basics
- Embeddings + vector search with Supabase pgvector
- Image generation APIs (DALL·E, Imagen, Stable Diffusion)
- Speech-to-text and text-to-speech
- Cost-aware design, token counting and caching
- AI ethics: hallucinations, bias, attribution

Projects built:
- AI study buddy that quizzes you on uploaded notes
- AI image caption generator (Gemini multimodal)
- Voice-to-text journal with summarization

Practice: Ship a one prompt AI feature, like a study tip generator, that calls the OpenAI, Gemini or Claude API and streams the response into your UI.

Week 8: Algorithms, DSA & System Design Lite + Mock Hackathon (Teams)

Topics:
- Big O notation for hackathon thinking
- Arrays, hash maps, sets, stacks, queues
- Trees, graphs, BFS/DFS, pathfinding apps
- Sorting (merge, quick) and searching (binary)
- Top 10 hackathon-style algorithm problems
- System Design Lite: when to use which database
- Caching, rate limiting, queues at hackathon scale
- Picking architecture in 5 minutes
- Team mock hackathon (2 days, 12 hours total)
- Live judging panel with feedback letter

Projects built:
- PHASE 2 CAPSTONE, Team Mock Hackathon: pick a problem, ship an MVP in 12 hours, deploy to Vercel, deliver pitch

Assessment: Mock Hackathon Judges Panel scoring + written feedback

---

PHASE 3: REAL HACKATHON SPRINT, CAPSTONE, PITCH LAB & LIVE HACKATHON (WEEKS 9-12)

The final 4 weeks are a real hackathon journey. Teens lock a capstone idea, get matched with a senior mentor, build, polish, get pitch coached and compete in the Live Modern Age Coders Hackathon Weekend.

Week 9: Capstone Lock, Mentor Matching, Spec & Architecture

Topics:
- Idea finalists, picking the right capstone (3 ideas 1)
- Writing a 1-page spec: problem, users, solution, MVP scope, stretch
- Mentor matching with senior engineers
- Architecture diagram (whiteboard or Excalidraw)
- Tech stack lock: React/Next.js + Tailwind + Supabase + AI API
- Risk register: top 3 risks and mitigations
- Project board on GitHub Projects or Linear
- Daily commit cadence rules
- Stretch goals vs. MVP scope (MVP wins, always)
- Repo setup, CI, and Vercel preview deployments

Projects built:
- Capstone Spec (1 page)
- Architecture Diagram
- Project Board with weekly milestones

Practice: Write your one page capstone spec covering problem, users, MVP scope and stretch goals, and sketch the architecture in Excalidraw.

Week 10: Heavy Build Sprint, Code, Test, Iterate
Topics:
- Daily commits with kind, clear messages
- Pair programming with teammates
- AI-assisted coding via Cursor + Replit Agent
- Mid-sprint mentor reviews
- Testing critical paths (smoke tests minimum)
- Loading states, empty states, error states (judges notice these)
- Mobile responsiveness, judges open on phones
- Performance basics: bundle size, image optimization, caching
- Accessibility minimums (alt text, focus rings, color contrast)
- Daily standup logs to mentor
Projects built:
- Capstone build, 70% complete by end of Week 10
- Daily standup logs
- Mid-sprint mentor demo
Practice: Ship one committed feature each day for five days, with clear commit messages and at least one smoke test on the critical path.

Week 11: Pitch Lab, Storytelling, Slides & Demo Polish
Topics:
- Hackathon judging rubric in detail (Innovation 25 / Execution 25 / Design 25 / Pitch 25)
- The 2-minute pitch formula: Hook Problem Solution Demo Tech Team Ask
- Slide design, 7 slides max, big text, no walls of words
- Practicing on camera with daily review
- Recording a high-quality demo video
- Voice and pacing tips from professional pitch coaches
- Q&A handling, what to say when stumped
- Final README polish, judges read this first
- Final code freeze rules
- Deployment health check: SSL, mobile, dark mode, edge cases
Projects built:
- Pitch Slides (7 slides, Pitch.com or Canva)
- Pitch Video (2 minutes)
- Final README with hero image, demo link, tech stack, team
Practice: Cut your pitch to exactly two minutes, build a seven slide deck and record a demo video you would be proud to submit.

Week 12: LIVE HACKATHON WEEKEND, Compete with the World
Topics:
- Friday: opening ceremony, theme reveal, team check-in
- Saturday: 12-hour final sprint with senior mentor support
- Saturday evening: pitch dress rehearsal
- Sunday morning: 3-hour final polish + submission
- Sunday afternoon: live judging, every team pitches
- Categories: Most Innovative, Best Built, Best AI Use, Best Pitch, Best Design, Audience Favorite
- Live judge Q&A, handled with calm and clarity
- Awards ceremony, certificates, badges, Hall of Fame
- Reflection circle and feedback letters from judges
- Graduation: Teen Hackathon Builder Certification awarded
Projects built:
- GRAND CAPSTONE, Live Hackathon Submission: working product deployed to Vercel + GitHub repo + 2-minute demo + 7-slide pitch + judge-ready README + live presentation
Assessment: Final Hackathon Score + Judge Feedback Letter

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Weeks 1-4): 6+ Python/JS projects + Solo Mini Hackathon (deployed)
- Phase 2 (Weeks 5-8): 6+ React/Supabase/AI projects + Team Mock Hackathon
- Phase 3 (Weeks 9-12): Capstone + Pitch Deck + Demo + LIVE Hackathon submission

Total projects: 12+ deployed projects + Live Hackathon Capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Python and JavaScript fluency
- React + Next.js + Tailwind + shadcn/ui
- Supabase backend + auth + storage + realtime
- AI APIs: OpenAI, Gemini, Claude, chat, streaming, RAG
- Algorithms & DSA at hackathon depth
- Big O analysis
- GitHub workflow with branches and PRs
- System Design Lite for hackathons
- Vercel deployment and CI
- Pitch decks and demo videos
- README and project documentation
- Public speaking and Q&A handling

HOW A TYPICAL WEEK IS SPLIT

  Live classes: 2 sessions (1.5-2 hours each)
  Independent projects: 3-4 hours
  Weekly hack: 1 themed hackathon-style challenge
  Total per week: 6-8 hours

SUPPORT INCLUDED

  Live instruction: Senior engineers and ex-startup CTOs
  Mentor matching: 1-on-1 capstone mentor in Phase 3
  Pitch lab: Dedicated pitch coaches in Week 11
  Community: Teen Hackathon Builders Hall of Fame + alumni network
  Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

  Final certificate: Teen Hackathon Builder, Modern Age Coders Certified
  Capstone required: Live hackathon submission (deployed app + pitch + demo)
  Portfolio: 12+ deployed projects + grand capstone

WHERE THIS CAN LEAD

  - School & Inter-school Hackathon Champion
  - Smart India Hackathon Finalist
  - MLH Global Hackathon Participant
  - Open-Source Contributor
  - Strong College Application Profile (IIT/IIIT/BITS/Stanford/MIT)
  - Internship-Ready (startups, mid-stage companies)
  - Ready for Vibe Coding for College / DSA Masterclass

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: Is the hackathon really free for teens?
  A: Yes, 100% free. Any teen aged 13-17 can register and compete in the Modern Age Coders Live Hackathon at zero cost. No entry fee. No school endorsement required. Free certificate. Free judging.

  Q: Why is the prep course paid then?
  A: The prep course is optional but highly recommended. It gives teens 12 weeks of structured training in coding, AI, algorithms, system design, and pitching. The hackathon itself stays free regardless.

  Q: Does my teen need prior coding experience?
  A: No. The prep course starts from absolute basics in Python and JavaScript. Teens with some experience will move faster, but beginners catch up by Week 4. We have placement check-ins to ensure no one is left behind.

  Q: What is the time commitment per week?
  A: 6-8 hours weekly: 2 live classes (1.5-2 hours each), 3-4 hours of independent project work, plus 1 hour for the weekly hackathon-style challenge. Small batches make missed classes easy to recover: the mentor recaps in the next session, notes and the week's challenge are shared over WhatsApp, and longer absences get a short free catch-up slot.

  Q: What does my teen build by the end?
  A: A GitHub portfolio of 6+ deployed projects, including a live capstone hosted on Vercel, a 2-minute pitch video, and submission to the live hackathon. Strong enough to impress college admissions and internship recruiters.

  Q: Will this help with college applications?
  A: Significantly. A real GitHub portfolio with deployed projects, hackathon wins, and demo videos is one of the strongest signals for tech-leaning college applications (US, Singapore, India IITs/IIITs/BITS). We also write recommendation letters for top performers.

  Q: What stack do you teach?
  A: The same stack used at YC startups in 2026: Python, JavaScript/TypeScript, React, Next.js (App Router), Tailwind CSS, shadcn/ui, Supabase, Vercel, GitHub, plus AI APIs from OpenAI, Google Gemini, and Anthropic Claude. We also use Cursor, Replit Agent, and Bolt.new as AI-assisted dev tools.

  Q: Can my teen compete in MLH or Smart India Hackathon after this?
  A: Yes, and many do. Our curriculum directly maps to MLH (Major League Hacking), Smart India Hackathon, NASA Space Apps, and most college-level hackathons. The skills are universal.

  Q: Do you cover algorithms and DSA?
  A: Yes, at hackathon depth. Big O, arrays, hash maps, trees, graphs, BFS/DFS, sorting, searching. We don't go to LeetCode Hard depth; that's our Vibe Coding for College program.

But we go deep enough to write efficient hackathon code that doesn't crash with real data.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/hackathon-prep-for-teens-coding-ai-build-innovate-win-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 79 OF 137: HIGH SCHOOL MATHEMATICS: ALGEBRA TO CALCULUS (GRADE 9-12)

==========

Course link: https://learn.modernagecoders.com/courses/complete-high-school-mathematics-mastery
Subject area: Mathematics Education
Designed for: Girls only batches, High school
Who can join: Complete Beginner to Advanced High School
Course length: 12-14 months (52-60 weeks)
Time needed: 2 live classes/week + 3-5 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online high school maths course for Grade 9-12: algebra, geometry, trigonometry, pre-calculus, calculus and statistics, built for university readiness.

In one line: From Basic Arithmetic to Advanced Calculus in 12 Months

ABOUT THE PROGRAMME

This accelerated program condenses 4 years of high school mathematics into an intensive 12-month journey. Whether you're a student seeking to excel, an adult learner filling gaps, or preparing for competitive exams, this comprehensive program covers every essential mathematical concept from grades 9-12. You'll master algebraic thinking, geometric reasoning, trigonometric applications, and calculus fundamentals. Each concept is taught through interactive lessons, real-world applications, and extensive problem-solving practice. By completion, you'll be fully prepared for university-level mathematics and standardized tests.

WHAT MAKES THIS COURSE DIFFERENT

- Accelerated 4-year curriculum in 12 months
- Visual and interactive learning approach
- Real-world applications for every concept
- Adaptive practice problems with instant feedback
- Preparation for SAT, ACT, and competitive exams
- Step-by-step solution explanations
- Weekly assessments and progress tracking
- Personalized learning paths based on strengths/weaknesses

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Algebra I, Linear Equations, Polynomials, Basic Geometry
Phase 2: Intermediate (Months 4-6): Algebra II, Quadratics, Functions, Coordinate Geometry
Phase 3: Advanced (Months 7-9): Trigonometry, Pre-Calculus, Sequences, Complex Numbers
Phase 4: Mastery (Months 10-12): Calculus, Statistics, Probability, Advanced Problem Solving

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Basic arithmetic and elementary math
Coding experience: None required - math focus
Equipment: Computer/tablet, internet, calculator
Time commitment: 2 live classes plus 3 to 5 hours of practice a week
English: Basic comprehension
Motivation: Desire to master mathematics

WHO THIS COURSE IS FOR

Students: High school students (grades 9-12)
Working professionals: Adults needing math skills
Entrepreneurs: Business owners needing quantitative skills
Freelancers: Tutors and educators
Kids: Advanced middle schoolers ready for challenge
Anyone: Anyone wanting strong mathematical foundation

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: ALGEBRAIC FOUNDATIONS AND BASIC GEOMETRY (WEEKS 1-13)

Build strong foundations in algebraic thinking, equation solving, and geometric reasoning.
Master the core concepts that form the basis of all advanced mathematics.

Months 1-2: Algebra I Fundamentals (Weeks 1-8)

Number Systems & Algebraic Expressions

Topics:

- Real number system: natural, whole, integers, rational, irrational
- Number line and absolute value
- Properties of real numbers: commutative, associative, distributive
- Order of operations (PEMDAS/BODMAS)
- Exponents and exponent laws
- Scientific notation and significant figures
- Square roots and cube roots
- Simplifying radical expressions
- Variables and algebraic expressions
- Evaluating expressions for given values
- Combining like terms
- Translating word problems to algebraic expressions

Projects built:

- Create a number system classification chart
- Real-world scientific notation problems
- Algebraic expression calculator design

Practice: Daily: 50 problems on simplification and evaluation

Linear Equations & Inequalities

Topics:

- Solving one-step equations
- Solving two-step equations
- Multi-step equations with distribution
- Equations with variables on both sides
- Equations with fractions and decimals
- Literal equations and formulas
- Linear inequalities in one variable
- Compound inequalities (AND/OR)
- Absolute value equations
- Absolute value inequalities
- Graphing inequalities on number line
- Word problems leading to linear equations

Projects built:

- Budget planning with linear equations
- Distance-rate-time problem solver
- Inequality graphing tool

Practice: Solve 100 equations and 50 inequalities with varying complexity

Polynomials & Operations

Topics:

- Polynomial terminology: degree, coefficient, constant
- Standard form of polynomials
- Adding and subtracting polynomials
- Multiplying monomials
- Multiplying polynomials (FOIL method)
- Multiplying binomials and trinomials
- Special products: $(a+b)^2$, $(a-b)^2$, $(a+b)(a-b)$
- Dividing polynomials by monomials
- Long division of polynomials
- Synthetic division introduction
- Pascal's triangle and binomial expansion
- Applications of polynomial operations

Projects built:

- Polynomial calculator implementation
- Area and volume problems with polynomials
- Pascal's triangle pattern explorer

Practice: Complete 150 polynomial operation problems

Factoring Techniques

Topics:

- Greatest Common Factor (GCF)
- Factoring by grouping
- Factoring trinomials ($a=1$)
- Factoring trinomials ($a\neq1$)
- Difference of squares
- Sum and difference of cubes
- Perfect square trinomials
- Complete factoring strategies

- Solving equations by factoring
- Zero product property
- Factor theorem introduction
- Applications in area and optimization

Projects built:

- Factoring flowchart creation
- Projectile motion problems
- Optimization problem solver

Practice: Factor 200 expressions of varying types

Month 3: Systems & Basic Geometry (Weeks 9-13)

Systems of Linear Equations

Topics:

- Graphing linear equations in two variables
- Slope and y-intercept
- Point-slope form and slope-intercept form
- Standard form and intercepts
- Parallel and perpendicular lines
- Systems of linear equations: graphical method
- Substitution method
- Elimination method
- Systems with no solution or infinite solutions
- Three-variable systems introduction
- Matrix representation of systems
- Word problems with systems

Projects built:

- Supply and demand economic model
- Mixture problem solver
- Linear programming introduction

Practice: Solve 100 systems using different methods

Introduction to Geometry

Topics:

- Points, lines, and planes
- Angles: acute, right, obtuse, straight, reflex
- Angle relationships: complementary, supplementary, vertical
- Parallel lines and transversals
- Triangles: classification by sides and angles
- Triangle sum theorem
- Exterior angle theorem
- Congruent triangles: SSS, SAS, ASA, AAS, HL
- Similar triangles and proportions
- Pythagorean theorem and its converse
- Special right triangles (45-45-90, 30-60-90)
- Triangle inequalities

Projects built:

- Triangle solver application
- Architectural design with triangles
- Distance calculator using Pythagorean theorem

Practice: Complete 150 geometry problems

Phase 1 Review & Assessment

Topics:

- Comprehensive review of algebraic concepts
- Review of equation solving techniques
- Polynomial and factoring review
- Systems of equations review
- Geometry fundamentals review
- Problem-solving strategies
- Test-taking techniques

Projects built:

- Create comprehensive formula sheet
- Design practice test for peers
- Real-world application portfolio

Assessment: Phase 1 cumulative exam: a written paper mixing this phase with everything before it, plus targeted revision for anything shaky

---

PHASE 2: ADVANCED ALGEBRA AND FUNCTIONS (WEEKS 14-26)

Master quadratic functions, advanced algebraic concepts, coordinate geometry, and function theory. Build strong analytical and graphing skills.

Months 4-5: Quadratics & Functions (Weeks 14-21)

Quadratic Functions & Equations

Topics:

- Quadratic functions: $f(x) = ax^2 + bx + c$
- Graphing parabolas: vertex, axis of symmetry
- Maximum and minimum values
- Vertex form: $f(x) = a(x-h)^2 + k$
- Converting between standard and vertex form

    - Completing the square method
    - Quadratic formula derivation and application
    - Discriminant and nature of roots
    - Complex solutions introduction
    - Sum and product of roots
    - Quadratic inequalities
    - Applications: projectile motion, optimization
  Projects built:
    - Projectile motion simulator
    - Profit maximization calculator
    - Parabola art generator
  Practice: Solve 200 quadratic problems
Exponential & Logarithmic Functions
  Topics:
    - Exponential functions: f(x) = a^x
    - Properties of exponential functions
    - Exponential growth and decay
    - The number e and natural exponential
    - Compound interest formula
    - Introduction to logarithms
    - Common and natural logarithms
    - Logarithm properties and laws
    - Change of base formula
    - Solving exponential equations
    - Solving logarithmic equations
    - Applications: population growth, radioactive decay
  Projects built:
    - Compound interest calculator
    - Population growth model
    - pH scale explorer
  Practice: Master 150 exponential and logarithmic problems
Rational Functions & Expressions
  Topics:
    - Rational expressions: simplification
    - Multiplying and dividing rational expressions
    - Adding and subtracting rational expressions
    - Complex fractions
    - Rational equations
    - Extraneous solutions
    - Rational functions and graphs
    - Vertical and horizontal asymptotes
    - Holes in rational functions
    - Inverse variation
    - Joint and combined variation
    - Applications in physics and engineering
  Projects built:
    - Lens equation calculator
    - Rate problem solver
    - Asymptote visualizer
  Practice: Complete 150 rational expression problems
Function Theory & Transformations
  Topics:
    - Function definition and notation
    - Domain and range
    - Function operations: addition, subtraction, multiplication, division
    - Composite functions
    - Inverse functions
    - One-to-one functions
    - Function transformations: translations
    - Reflections and stretches
    - Combining transformations
    - Even and odd functions
    - Piecewise functions
    - Greatest integer function
  Projects built:
    - Function transformation visualizer
    - Composite function calculator
    - Inverse function finder
  Practice: Work through 200 function problems
Conic Sections
  Topics:
    - Circle equation: center-radius form
    - General form of circle equation
    - Ellipse: standard form and properties
    - Hyperbola: standard form and properties
    - Parabola review in conic context
    - Identifying conic sections
    - Translations of conics
    - Systems involving conics

- Applications in astronomy and physics
  Projects built:
- Conic section grapher
- Orbital mechanics simulator
- Architectural design with conics
  Practice: Graph and analyze 100 conic sections

Month 6: Coordinate Geometry & Vectors (Weeks 22-26)
Advanced Coordinate Geometry
Topics:
- Distance formula in 2D and 3D
- Midpoint formula
- Section formula
- Area of triangle using coordinates
- Collinearity of points
- Equation of line: various forms
- Angle between two lines
- Distance from point to line
- Concurrent lines
- Locus problems
- Transformation geometry: rotation, reflection
- Coordinate proofs
  Projects built:
- Geometry solver application
- GPS distance calculator
- Computer graphics transformer
  Practice: Solve 150 coordinate geometry problems
Introduction to Vectors
Topics:
- Vector vs scalar quantities
- Vector notation and representation
- Vector addition and subtraction
- Scalar multiplication
- Magnitude of a vector
- Unit vectors
- Component form of vectors
- Dot product
- Angle between vectors
- Vector projections
- Applications in physics
- Cross product introduction (3D)
  Projects built:
- Force vector calculator
- Navigation system simulator
- Vector graphics editor
  Practice: Complete 100 vector problems
Sequences and Series
Topics:
- Arithmetic sequences
- Arithmetic series and sum formula
- Geometric sequences
- Geometric series and sum formula
- Infinite geometric series
- Convergence and divergence
- Sigma notation
- Recursive sequences
- Fibonacci sequence
- Mathematical induction
- Binomial theorem
- Applications in finance and nature
  Projects built:
- Loan amortization calculator
- Fibonacci pattern generator
- Series convergence tester
  Practice: Work through 150 sequence and series problems
Complex Numbers
Topics:
- Imaginary unit i
- Complex number form: a + bi
- Operations with complex numbers
- Complex conjugates
- Absolute value of complex numbers
- Complex plane (Argand diagram)
- Polar form of complex numbers
- De Moivre's theorem
- Roots of complex numbers
- Euler's formula introduction
- Applications in engineering
- Solving polynomial equations

Projects built:
- Complex number calculator
- Mandelbrot set visualizer
- AC circuit analyzer

Practice: Master 100 complex number problems

Phase 2 Capstone Assessment

Topics:
- Comprehensive function review
- Advanced algebra synthesis
- Coordinate geometry mastery check
- Vector applications
- Complex problem solving
- Mathematical modeling

Projects built:
- Design a complete function analysis tool
- Create mathematical model for real phenomenon
- Build comprehensive problem set with solutions

Assessment: Phase 2 cumulative exam: a written paper mixing this phase with everything before it, plus targeted revision for anything shaky

---

PHASE 3: TRIGONOMETRY AND PRE-CALCULUS (WEEKS 27-39)

Master trigonometric functions, identities, and applications. Prepare for calculus with limits, advanced functions, and analytical geometry.

Months 7-8: Trigonometry Mastery (Weeks 27-34)

Trigonometric Ratios & Functions

Topics:
- Angle measurement: degrees and radians
- Arc length and sector area
- Right triangle trigonometry
- Six trigonometric ratios: sin, cos, tan, csc, sec, cot
- Trigonometric ratios of special angles
- Reference angles and coterminal angles
- Unit circle and circular functions
- Trigonometric functions of any angle
- Signs of trigonometric functions in quadrants
- Exact values using unit circle
- Inverse trigonometric functions
- Principal values and general solutions

Projects built:
- Unit circle interactive tool
- Height and distance calculator
- Trigonometric table generator

Practice: Solve 200 basic trigonometry problems

Graphing Trigonometric Functions

Topics:
- Graphing sine and cosine functions
- Amplitude, period, and frequency
- Phase shift and vertical shift
- Graphing tangent and cotangent
- Graphing secant and cosecant
- Transformations of trigonometric graphs
- Combining trigonometric functions
- Damped oscillations
- Trigonometric modeling
- Applications: sound waves, tides
- Harmonic motion
- Fourier series introduction

Projects built:
- Wave generator and analyzer
- Music synthesizer using trig functions
- Tidal prediction model

Practice: Graph 150 trigonometric functions

Trigonometric Identities

Topics:
- Fundamental identities: Pythagorean, reciprocal, quotient
- Cofunction identities
- Even-odd identities
- Sum and difference formulas
- Double angle formulas
- Half angle formulas
- Product-to-sum formulas
- Sum-to-product formulas
- Proving trigonometric identities
- Simplifying trigonometric expressions
- Conditional identities
- Applications in physics and engineering

```
    Projects built:
      - Identity prover application
      - Trigonometric simplifier
      - Physics formula derivation tool
    Practice: Prove 100 trigonometric identities
  Trigonometric Equations & Applications
    Topics:
      - Solving basic trigonometric equations
      - Equations involving multiple angles
      - Equations with trigonometric identities
      - Parametric equations
      - Eliminating parameters
      - Polar coordinates
      - Converting between polar and rectangular
      - Graphing in polar coordinates
      - Rose curves and limacons
      - Complex numbers in polar form
      - Applications in navigation
      - Spherical trigonometry basics
    Projects built:
      - Parametric curve plotter
      - Polar graphing calculator
      - Navigation system using trigonometry
    Practice: Solve 150 trigonometric equations
  Law of Sines and Cosines
    Topics:
      - Law of sines derivation
      - Solving triangles with law of sines
      - Ambiguous case (SSA)
      - Law of cosines derivation
      - Solving triangles with law of cosines
      - Area of triangle formulas
      - Heron's formula
      - Applications in surveying
      - Navigation problems
      - Vector applications
      - 3D trigonometry introduction
    Projects built:
      - Triangle solver (all cases)
      - Surveying application
      - GPS triangulation simulator
    Practice: Solve 100 triangle problems

Month 9: Pre-Calculus & Limits (Weeks 35-39)
  Limits and Continuity
    Topics:
      - Introduction to limits
      - Limit notation and terminology
      - Evaluating limits graphically
      - Evaluating limits numerically
      - Evaluating limits algebraically
      - One-sided limits
      - Limits at infinity
      - Infinite limits and vertical asymptotes
      - Limit laws and properties
      - Squeeze theorem
      - Continuity at a point
      - Types of discontinuities
    Projects built:
      - Limit calculator and visualizer
      - Continuity checker
      - Asymptote analyzer
    Practice: Evaluate 200 limits
  Introduction to Derivatives
    Topics:
      - Rate of change and slope
      - Average rate of change
      - Instantaneous rate of change
      - Definition of derivative
      - Derivative as a limit
      - Differentiability and continuity
      - Basic differentiation rules
      - Power rule
      - Product rule introduction
      - Quotient rule introduction
      - Chain rule introduction
      - Applications: velocity and acceleration
    Projects built:
      - Derivative calculator
```

- Motion analyzer
- Tangent line grapher

Practice: Find 150 derivatives

Mathematical Reasoning & Proofs

Topics:

- Logic and statements
- Truth tables
- Conditional statements
- Biconditional statements
- Logical equivalence
- Methods of proof: direct, indirect
- Proof by contradiction
- Proof by induction (detailed)
- Counter examples
- Set theory basics
- Relations and functions
- Mathematical writing

Projects built:

- Proof validator
- Logic puzzle solver
- Set operation calculator

Practice: Write 50 mathematical proofs

Matrices and Determinants

Topics:

- Matrix notation and terminology
- Types of matrices
- Matrix operations: addition, subtraction
- Scalar multiplication
- Matrix multiplication
- Properties of matrix operations
- Identity and inverse matrices
- Determinants (2x2 and 3x3)
- Properties of determinants
- Cramer's rule
- Matrix equations
- Applications in transformations

Projects built:

- Matrix calculator
- Linear system solver using matrices
- Transformation visualizer

Practice: Complete 100 matrix problems

Phase 3 Review & Assessment

Topics:

- Trigonometry comprehensive review
- Pre-calculus concepts review
- Limits and continuity review
- Mathematical reasoning review
- Problem-solving strategies
- Exam preparation techniques

Projects built:

- Create trigonometry reference guide
- Design pre-calculus problem bank
- Build interactive review tool

Assessment: Phase 3 cumulative exam: a written paper mixing this phase with everything before it, plus targeted revision for anything shaky

---

PHASE 4: CALCULUS, STATISTICS AND FINAL REVISION (WEEKS 40-52)

Master differential and integral calculus, statistics, probability, and advanced problem-solving techniques for university readiness.

Month 10: Calculus (Weeks 40-44)

Differential Calculus

Topics:

- Review of derivative concept
- Differentiation rules (complete)
- Derivatives of trigonometric functions
- Derivatives of exponential functions
- Derivatives of logarithmic functions
- Implicit differentiation
- Logarithmic differentiation
- Higher-order derivatives
- Related rates problems
- Linear approximation
- Differentials
- Error estimation

Projects built:

- Complete derivative calculator

```
    - Related rates problem solver
    - Error analysis tool
  Practice: Solve 300 differentiation problems
Applications of Derivatives
  Topics:
    - Critical points and extrema
    - First derivative test
    - Second derivative test
    - Concavity and inflection points
    - Curve sketching (complete process)
    - Optimization problems
    - Maximum and minimum word problems
    - Mean Value Theorem
    - Rolle's Theorem
    - L'Hôpital's Rule
    - Newton's method
    - Applications in economics and physics
  Projects built:
    - Function analyzer (complete)
    - Optimization problem solver
    - Economic model calculator
  Practice: Complete 200 application problems
Integral Calculus
  Topics:
    - Antiderivatives and indefinite integrals
    - Basic integration rules
    - Integration by substitution
    - Definite integrals
    - Fundamental Theorem of Calculus
    - Properties of definite integrals
    - Integration by parts
    - Trigonometric integrals
    - Trigonometric substitution
    - Partial fractions
    - Improper integrals
    - Numerical integration: trapezoid, Simpson's rule
  Projects built:
    - Integration calculator
    - Area calculator using integrals
    - Numerical integration tool
  Practice: Evaluate 300 integrals
Applications of Integration
  Topics:
    - Area between curves
    - Volume by disks and washers
    - Volume by cylindrical shells
    - Arc length
    - Surface area of revolution
    - Work and energy problems
    - Center of mass
    - Probability density functions
    - Average value of a function
    - Applications in physics
    - Applications in economics
    - Differential equations introduction
  Projects built:
    - Volume calculator for solids
    - Physics application solver
    - Probability distribution analyzer
  Practice: Solve 150 application problems
Series and Sequences (Calculus)
  Topics:
    - Infinite sequences
    - Convergence of sequences
    - Infinite series
    - Geometric series (calculus perspective)
    - Test for divergence
    - Integral test
    - Comparison tests
    - Ratio and root tests
    - Alternating series
    - Power series
    - Taylor and Maclaurin series
    - Applications of series
  Projects built:
    - Series convergence tester
    - Taylor series calculator
    - Function approximator
  Practice: Analyze 100 series
```

Month 11: Statistics & Probability (Weeks 45-49)
  Descriptive Statistics
    Topics:
      - Types of data: categorical, numerical
      - Measures of central tendency: mean, median, mode
      - Measures of spread: range, variance, standard deviation
      - Quartiles and box plots
      - Percentiles and percentile rank
      - Frequency distributions and histograms
      - Stem-and-leaf plots
      - Scatter plots and correlation
      - Linear regression
      - Coefficient of determination (R^2)
      - Outliers and their effects
      - Data transformation
    Projects built:
      - Statistical calculator suite
      - Data visualization dashboard
      - Regression analysis tool
    Practice: Analyze 50 data sets
  Probability Theory
    Topics:
      - Sample spaces and events
      - Classical probability
      - Empirical probability
      - Probability axioms
      - Addition rule of probability
      - Multiplication rule
      - Conditional probability
      - Independent and dependent events
      - Bayes' theorem
      - Permutations and combinations
      - Binomial probability
      - Expected value and variance
    Projects built:
      - Probability calculator
      - Monte Carlo simulator
      - Game theory analyzer
    Practice: Solve 150 probability problems
  Probability Distributions
    Topics:
      - Random variables: discrete and continuous
      - Probability mass functions
      - Probability density functions
      - Cumulative distribution functions
      - Binomial distribution
      - Poisson distribution
      - Geometric distribution
      - Normal distribution
      - Standard normal distribution (Z-scores)
      - Central Limit Theorem
      - Sampling distributions
      - Chi-square distribution introduction
    Projects built:
      - Distribution visualizer
      - Normal probability calculator
      - Sampling distribution simulator
    Practice: Work with 100 distribution problems
  Statistical Inference
    Topics:
      - Point estimation
      - Confidence intervals for means
      - Confidence intervals for proportions
      - Hypothesis testing framework
      - Type I and Type II errors
      - Z-tests for means
      - T-tests: one-sample, two-sample, paired
      - Tests for proportions
      - Chi-square test for independence
      - ANOVA basics
      - P-values and significance levels
      - Power of a test
    Projects built:
      - Hypothesis testing calculator
      - Confidence interval generator
      - Statistical test selector
    Practice: Conduct 75 hypothesis tests
  Advanced Problem Solving

Topics:
- Competition mathematics strategies
- Number theory problems
- Combinatorics advanced
- Graph theory basics
- Game theory introduction
- Optimization techniques
- Mathematical modeling
- Cross-topic problem solving
- SAT/ACT preparation
- University entrance exam prep
- Mathematical competitions prep

Projects built:
- Competition problem solver
- Mathematical game creator
- Exam preparation guide

Practice: Solve 100 competition-level problems

Month 12: Final Review & Applications (Week 50)

University Preparation

Topics:
- University mathematics preview
- Study skills for higher math
- Mathematical maturity
- Abstract thinking development
- Proof-writing skills
- Advanced calculus preview
- Linear algebra introduction
- Discrete mathematics preview
- Mathematical career paths
- Further learning resources

Projects built:
- University readiness portfolio
- Advanced topic exploration
- Learning plan for next level

Practice: Complete university-level problem sets

Month 13: Final Assessments & Certification (Weeks 51-52)

Comprehensive Review - Part 1

Topics:
- Algebra complete review
- Geometry complete review
- Trigonometry complete review
- Functions and graphs review
- Calculus essentials review
- Statistics and probability review
- Problem-solving strategies review
- Formula compilation

Projects built:
- Create comprehensive study guide
- Build formula reference app
- Design review problem sets

Comprehensive Review - Part 2 & Mock Exams

Topics:
- Cross-topic problem solving
- Time management strategies
- Error analysis and correction
- Mock exam practice
- SAT/ACT math practice
- AP Calculus practice
- Competition math practice
- Peer tutoring exercises

Deliverables:
- Complete practice exam portfolio
- Error analysis report
- Personal progress documentation
- Achievement portfolio

Assessment: FINAL EXAM: a comprehensive written paper covering every phase of the course. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Algebraic calculator, Equation solver, Geometry tool, Pattern generator
- Phase 2: Function analyzer, Graphing calculator, Vector simulator, Complex number visualizer
- Phase 3: Trigonometry suite, Proof assistant, Matrix calculator, Limit explorer
- Phase 4: Calculus toolkit, Statistics dashboard, Probability simulator, Research project
- Final: Comprehensive mathematics portfolio with 50+ solved problems

Total projects: 100+ mathematical tools, visualizers, and problem-solving applications

SKILLS THE STUDENT WILL HAVE AT THE END

- Algebra: Linear, quadratic, polynomial, rational, exponential, logarithmic
- Geometry: Euclidean, coordinate, transformational, vectors
- Trigonometry: Functions, identities, equations, applications
- Pre-Calculus: Limits, sequences, series, matrices
- Calculus: Differential, integral, applications
- Statistics: Descriptive, inferential, regression
- Probability: Theory, distributions, inference
- Problem Solving: Analytical thinking, proof writing, modeling
- Technology: Calculators, software, programming for math

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week with a real teacher, working problems together
Homework: A problem set after every class, and a mixed-review assignment at the end of every month that deliberately folds in earlier topics
Practice: 3-5 hours of problem-solving between classes
Review: Every set reviewed with written feedback; recurring mistakes reopened in class until they are gone

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: which topics are solid, which are wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies mastery, not attendance

WHERE THIS CAN LEAD

- Engineering (all disciplines)
- Data Science and Analytics
- Actuarial Science
- Financial Analysis
- Research Scientist
- Mathematics Teacher/Professor
- Quantitative Trading
- Operations Research
- Software Development (algorithms)
- Economics and Econometrics

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What topics does the High School Mathematics Mastery Program cover?
A: In 12 months it moves through four phases: Algebra I foundations and basic geometry; Algebra II with quadratics, functions and coordinate geometry; trigonometry and pre-calculus including sequences and complex numbers; and finally introductory calculus, statistics and probability. It is the full grade 9 to 12 arc taught as one connected story.

Q: Does this course follow CBSE, ICSE or another school board?
A: The course is deliberately board agnostic: it teaches the mathematics underneath every board, so it works alongside CBSE, ICSE, IGCSE, IB or state syllabi rather than replacing them. Your teacher aligns examples and pacing with what school is covering whenever that helps.

Q: My child is in grade 9, or already in grade 11. Is the level right?
A: Yes. The program starts from basic arithmetic and algebra, so a grade 9 student begins comfortably, while an older student uses it to close gaps quickly and then accelerate into pre-calculus and calculus. A diagnostic in the first week sets the entry point, so nobody sits through material they already own.

Q: What are the prerequisites?
A: Only basic arithmetic and elementary school mathematics. No coding is involved, and the equipment list is simple: a computer or tablet, internet and a calculator. Everything else, including the thinking habits, is built inside the course.

Q: How do we know it is working? How is progress measured?
A: Every phase ends with an assessment that must be secure before moving on, and weekly problem work is reviewed for reasoning, not just answers. Parents get a clear picture from the running record of mastered topics, and school test performance is the natural external

check along the way.

Q: How is my child assessed, and is the certificate just for attending?
A: No. A problem set follows every class with written feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, each phase ends with a cumulative exam, and the course ends with a final exam covering the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest. We measure understanding, not attendance.

Q: Is there a free trial?
A: Yes. Book a free demo class at learn.modernagecoders.com/contact with no commitment; it is a real lesson plus an honest read on where your child stands. Questions first? Message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are these live classes or videos?
A: Live, always. Each session is a full one hour class with a real teacher in a small group, never pre-recorded content, so the teacher can catch the exact step where your child's method breaks and fix it on the spot.

Q: Is there a certificate at the end of the program?
A: Students receive the High School Mathematics Completion Certificate along with a documented record of the topics, problem sets and projects they worked through. We are rated 4.9 stars across 547+ Google reviews, largely by parents of students in programs like this one.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/complete-high-school-mathematics-mastery
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 80 OF 137: HTML & CSS FOR BEGINNERS: BUILD REAL RESPONSIVE WEBSITES
==========

Course link: https://learn.modernagecoders.com/courses/html-css-course-for-beginners-build-real-websites
Subject area: Web Development
Designed for: Teens
Who can join: Absolute beginner, ages 10 to adult
Course length: 4 months (16 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

HTML and CSS course for beginners: live online classes from your first tag to a published multi-page website, with flexbox, grid, and responsive design.

In one line: Every website you have ever visited is built on these two languages. Four months from a blank file to your own published site.

ABOUT THE PROGRAMME

HTML and CSS are where the web actually starts. Frameworks come and go, but every page on the internet is, underneath, structure written in HTML and appearance written in CSS, and people who skip this layer pay for it later. This 4-month live course takes complete beginners from an empty folder to a multi-page personal website they design, build, test on a phone, and publish to a real URL. The first two months build the two languages properly: semantic page structure, then selectors, the box model, and typography, so students understand why a page looks the way it does instead of copying snippets and hoping. The second half is where pages become layouts: flexbox and grid, responsive design that survives a phone screen, working forms, and a capstone month spent building and shipping the personal site. Classes are live and small, and every student leaves with a site that is genuinely theirs, structure, design decisions, and all.

WHAT MAKES THIS COURSE DIFFERENT

- Structure before decoration: semantic HTML is taught properly in month 1, the habit that separates clean sites from tag soup
- Layout is taught as a system, not a bag of tricks: flexbox and grid each get real time,

including when to use which
- Students recreate real designs from screenshots, the same exercise working developers use to train their eye
- Every page built in class is tested on a phone, because most visitors arrive on one
- The capstone is published to a real URL on free hosting, not left on a laptop
- Works for ages 10 to adult: batches are grouped by age so examples and pace fit the room

THE LEARNING PATH, STEP BY STEP

Phase 1: The two languages: semantic HTML structure first, then CSS selectors, the box model, and typography, applied to real pages every week
Phase 2: Layout and shipping: flexbox, grid, responsive design, and forms, finishing with a multi-page personal site built, tested, and published

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. This is the true starting point of web development
Computer skills: Comfortable typing and managing files and folders
Equipment: A computer with a modern browser and a stable internet connection
Age: 10 to adult. Batches are grouped by age so pace and examples fit the room

WHO THIS COURSE IS FOR

Absolute beginners: Anyone starting web development from zero who wants to build the real foundation, not skip it
Kids and teens: Students aged 10+ who want to make actual websites they can show, not just follow tutorials
Adults switching tracks: Working adults testing whether web development suits them, with a real published site as the outcome
Small business owners: People who want to understand, fix, and eventually build their own web presence
Future javascript students: Learners who plan to take JavaScript next and want their HTML and CSS solid first

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE TWO LANGUAGES (MONTHS 1-2, WEEKS 1-8)

HTML gives a page its structure and meaning; CSS gives it appearance. Both are learned by building real pages from the first week.

Month 1: HTML, the Structure of Every Page (Weeks 1-4)
Your First Page
Topics:
- How a website actually works: files, browsers, and servers in plain language
- Setting up VS Code and a tidy project folder
- The skeleton of every page: doctype, head, and body
- Headings, paragraphs, and line breaks
- Ordered and unordered lists
- Saving, refreshing, and reading your page the way the browser does

Projects built:
- A real first page: an introduction page with headings, paragraphs, and a list, opened in the browser by the end of class one

Practice: Build a second page about a hobby using every element from class, plus one you look up yourself

Links, Images, and Paths
Topics:
- Anchor tags: linking to other sites and to your own pages
- Images: src, alt text, and why alt text matters for real people
- Relative vs absolute paths, and the broken-image bug everyone hits
- Organizing a project: an images folder that stays tidy
- Attributes in general: the pattern behind href, src, and the rest
- Opening links in new tabs, used sparingly

Projects built:
- Two-page mini site: an about page and a favorites page, linked both ways, with images that actually load

Practice: Fix a provided page with six broken links and images, then explain what was wrong with each

Semantic Structure
Topics:
- header, nav, main, section, article, and footer: naming the parts of a page
- Why structure matters: screen readers, search engines, and future-you
- div and span, and why they are the last resort, not the first
- Nesting elements correctly and indenting so you can see the structure
- Block vs inline elements, previewed before CSS makes it matter
- Reading the structure of real websites with dev tools

Projects built:
- Restructure week 2's site with proper semantic tags, then sketch the structure of a favorite real website on paper

Practice: Mark up a printed newspaper-style article into clean semantic HTML

Tables, Media, and Clean Code

Topics:
- Tables for actual tabular data: rows, headers, and captions
- Embedding video and maps with iframes
- Character entities and special symbols
- Validating your HTML with the W3C validator and reading its complaints
- Commenting and formatting habits that keep pages readable
- A structure checklist to run before calling any page done

Projects built:
- A results or schedule page with a real data table plus an embedded video or map

Practice: Run all your pages through the validator and clear every error

Assessment: Month 1 check: build a clean semantic page live from a hand-drawn sketch, validator-clean

Month 2: CSS, the Appearance Layer (Weeks 5-8)

First Styles

Topics:
- What CSS is and the three ways to add it, and why external stylesheets win
- Selectors: element, class, and id, and when each is appropriate
- Colors: names, hex, and rgb
- The cascade in plain terms: what wins when rules collide
- Grouping selectors and keeping a stylesheet organized
- Dev tools as a CSS laboratory: change styles live, then copy them back

Projects built:
- Style the month 1 introduction page: a real color scheme, applied through classes, no inline styles

Practice: Given an unstyled page and a screenshot, write the selectors and colors to match it

Typography

Topics:
- font-family and font stacks: what happens when a font is missing
- Loading Google Fonts properly
- Size, weight, and line-height: the three dials that change everything
- Spacing text so it breathes: letter-spacing and text alignment
- Units: px, em, rem, and percent, explained with examples
- Pairing two fonts without the page looking like a ransom note

Projects built:
- Typography makeover: the same article styled two ways, one serious, one playful, with a note on the choices

Practice: Recreate the text styling of a well-designed real site as closely as you can

The Box Model

Topics:
- Every element is a box: content, padding, border, margin
- Reading the box model diagram in dev tools
- box-sizing: border-box and why we set it everywhere
- Backgrounds: colors, images, and gradients
- Borders, rounded corners, and shadows used with restraint
- Margin collapse, the classic surprise, demonstrated live

Projects built:
- Profile card: photo, name, description, and a button, spaced and shadowed like a real component

Practice: Ten box-model puzzles: match the target spacing exactly using padding and margin

Display and Simple Layout

Topics:
- display: block, inline, and inline-block in practice
- Width, height, and max-width: making content behave
- Centering things: the honest tour of the classic problem
- Building a content column that sits nicely on any screen
- Hover states: the first taste of interactivity in pure CSS
- Putting a full page together: structure from month 1, styles from month 2

Projects built:
- A styled article page: centered content column, styled headings, hover-styled links, and a card or two

Practice: Recreate a provided card design from a screenshot, pixel-close

Assessment: Month 2 check: recreate a small component design from a screenshot live, explaining each CSS choice

---

PHASE 2: LAYOUT, RESPONSIVENESS, AND SHIPPING (MONTHS 3-4, WEEKS 9-16)

Flexbox and grid turn styled pages into real layouts. Then responsive design, forms, and a capstone month: build, test, and publish a multi-page personal site.

Month 3: Real Layout with Flexbox and Grid (Weeks 9-12)

  Flexbox Fundamentals
    Topics:
      - The flex container and its children: a mental model that sticks
      - Main axis and cross axis, drawn before coded
      - justify-content and align-items: the six arrangements you use daily
      - gap: spacing items without margin hacks
      - flex-direction: rows becoming columns
      - The navigation bar: flexbox's most famous job
    Projects built:
      - A proper navigation bar: logo left, links right, evenly spaced, with hover states
    Practice: Flexbox Froggy through the core levels, plus three layout snippets built from
    description alone
  Flexbox in the Real World
    Topics:
      - flex-wrap: rows that break gracefully
      - flex-grow and flex-shrink: who gets the spare space
      - Building a card row that becomes a card grid
      - Aligning odd items: the last-item problem and clean solutions
      - Nested flex containers without confusion
      - Reading a real site's layout and spotting the flex containers
    Projects built:
      - Photo gallery row and a three-card feature section, both wrapping cleanly as the window
        narrows
    Practice: Recreate two sections of a real landing page using only flexbox
  CSS Grid
    Topics:
      - grid-template-columns and rows: designing in two dimensions
      - The fr unit and repeat()
      - Placing items: spanning columns and rows
      - grid-template-areas: layouts you can read like a map
      - Grid vs flexbox: a working rule for choosing
      - Gaps, alignment, and auto-fit for self-arranging grids
    Projects built:
      - Magazine-style layout: a featured story spanning two columns above a grid of smaller pieces
    Practice: Build the same page layout twice, once in flexbox, once in grid, and write
    three sentences on which felt right
  Responsive Design
    Topics:
      - Why mobile-first is the sane default
      - Media queries: syntax, breakpoints, and where breakpoints actually come from
      - The viewport meta tag, the line everyone forgets
      - Flexible images: max-width 100% and object-fit
      - Testing with the dev tools device toolbar and on a real phone
      - Responsive navigation: what stacks, what hides, what stays
    Projects built:
      - Make the month's layouts fully responsive: gallery, cards, and magazine layout all working
        from phone to desktop
    Practice: Audit two favorite websites at three screen widths and note what changes at
    each
    Assessment: Layout check: build a responsive two-breakpoint layout live from a
    wireframe

Month 4: Forms, Polish, and the Capstone Site (Weeks 13-16)
  Forms
    Topics:
      - form, input, label, and why every input needs its label
      - Input types: text, email, number, date, and what the browser gives you free
      - Textareas, selects, checkboxes, and radio buttons
      - Styling form controls so they match the site
      - Built-in validation: required, min, max, and helpful error states
      - What happens on submit, honestly: where the backend would take over
    Projects built:
      - A styled contact form: name, email, subject dropdown, message, and a submit button that
        validates
    Practice: Build a sign-up form for an imaginary club, labels and validation included
  Polish and Performance
    Topics:
      - Transitions: hover effects that feel smooth instead of sudden
      - Simple keyframe animations, used with restraint
      - Favicons and page titles: the details visitors notice
      - Image weight: resizing and compressing before uploading
      - A consistent spacing scale: the quiet secret of pages that look professional
      - Accessibility pass: contrast, alt text, and keyboard-reachable links
    Projects built:
      - Polish pass on your best existing page: transitions, favicon, compressed images, and an
        accessibility check
    Practice: Run a contrast checker on your color scheme and fix anything that fails
  Capstone Build
    Topics:

- Scoping the personal site: home, about, gallery or projects, and contact
- A shared header and footer across pages, kept consistent by hand
- Wireframing each page before building it
- Reusing your components: cards, navigation, and forms from past weeks
- Working from a checklist: structure, styles, responsive, polish
- Mid-build review with the instructor and a course correction if needed

Projects built:
- The capstone: a multi-page personal site, four pages, shared navigation, responsive throughout

Practice: Complete two pages between classes to keep the build on schedule

Test, Publish, Present

Topics:
- Cross-device testing: phone, tablet width, and a second browser
- The pre-launch checklist: links, images, validator, spelling
- Publishing to free hosting so the site has a real address
- Sharing the link and what to say when you do
- Presenting design decisions: why it looks the way it looks
- Where this leads: JavaScript to make pages interactive

Projects built:
- The published site: live at a real URL, presented to the class with a walkthrough of one design decision per page

Practice: Send the live link to three people and collect one piece of feedback from each

Assessment: Capstone presentation: live site walkthrough plus questions on structure and CSS choices

PROJECTS THE STUDENT BUILDS AND KEEPS

- Two-page mini site with working links and images
- Semantic restructure of a real article into clean HTML
- Typography makeover: one article styled two different ways
- Profile card component, spaced and shadowed like a real design
- Navigation bar with logo, links, and hover states
- Responsive photo gallery and three-card feature section
- Magazine-style CSS grid layout
- Styled contact form with built-in validation
- The capstone: a published multi-page personal website

Total projects: 10+ real pages and components, ending with a published multi-page website

SKILLS THE STUDENT WILL HAVE AT THE END

- Semantic HTML: structuring any page correctly, from articles to forms to tables
- CSS fundamentals: selectors, the cascade, the box model, and typography that reads well
- Layout: flexbox and grid used deliberately, with a working rule for choosing between them
- Responsive design: mobile-first pages that hold up from phone to desktop
- Shipping: validating, testing across devices, compressing images, and publishing to free hosting
- Design judgment: spacing, contrast, and restraint, trained by recreating real designs

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building pages along with the instructor
Practice: 2-3 hours of building and recreation exercises between classes
Review: Every submitted page reviewed with written feedback; recurring issues reopened in class
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for stuck layouts and broken pages
Progress updates: Regular progress notes to parents of younger students; adults get direct feedback on every build

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- JavaScript next: our JavaScript courses turn these static pages into interactive apps
- The full front-end track: React and modern tooling once both layers are solid
- Freelance-style small projects: simple sites for family, clubs, and local groups
- Design-side paths: UI design tools make far more sense once you know what CSS can do
- WordPress and site-builder work with the rare ability to fix what the builder gets wrong

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: Is HTML a programming language?
  A: Strictly speaking, no: HTML is a markup language that describes structure, and CSS
  describes appearance; neither has loops or logic. That does not make them lesser skills.
  Every website ever made rests on them, layout in CSS is genuine engineering thinking, and
  developers who never learned this layer properly struggle with it for years. This course is
  the foundation programming languages get built on top of.

  Q: What age is this course right for?
  A: The course runs from age 10 up to adults, and batches are grouped by age, so a 10-year-old
  is not sharing a class with a 40-year-old career switcher. The material is the same because
  HTML and CSS are the same; the pace, examples, and project themes are adjusted to the room.
  For children under 10 we recommend our kids courses, which build the same instincts through
  friendlier tools.

  Q: Do I need this before learning JavaScript?
  A: It helps a great deal. JavaScript's whole job in the browser is manipulating HTML and CSS,
  so students who know both layers learn it noticeably faster and debug with far less pain. Our
  JavaScript course does teach the markup each project needs along the way, so either order
  works, but HTML and CSS first is the smoother road.

  Q: What will I have built by the end?
  A: Ten or more real pages and components: a semantic article page, styled cards, a navigation
  bar, a responsive photo gallery, a magazine-style grid layout, and a working contact form.
  The final month is a capstone: a multi-page personal website, responsive from phone to
  desktop, published to a real URL you can send to anyone.

  Q: How are the classes run?
  A: Live one-hour online classes, twice a week, with a real instructor and at most 20 students
  in a group batch. Students build during class, not just watch, and everything submitted
  between classes comes back with written feedback. Mini-batch (up to 6 students) and 1-on-1
  options are available for learners who want closer attention.

  Q: How am I assessed, and is the certificate just for showing up?
  A: No. A task follows every class with feedback, monthly reviews deliberately mix new
  material with earlier topics so nothing is forgotten, and the course ends with a final
  project defended live, plus a written assessment. Passing earns the certificate; falling
  short earns focused revision and a free retest. We measure real learning, not attendance.

  Q: Is there a free trial class?
  A: Yes, the first demo class is free with no commitment. Book at
  learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. It is a real
  class, not a sales pitch: you or your child will build something small in it.

  Q: Do I need to buy any software?
  A: No. Everything used in the course is free: the VS Code editor, a modern browser like
  Chrome, Google Fonts, and free hosting for publishing the capstone. Any reasonably recent
  laptop or desktop is enough; HTML and CSS are the lightest possible workload for a computer.

  Q: Can this course lead to real work?
  A: On its own, HTML and CSS get you to simple static sites, which is genuinely enough for
  personal pages, clubs, and small local projects. For paid front-end work you will also want
  JavaScript, which is exactly the next course in our track. What this course guarantees is the
  layer beneath all of that, learned properly instead of patched together later.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/html-css-course-for-beginners-build-real-websites
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
COURSE 81 OF 137: IB DP COMPUTER SCIENCE (NEW GUIDE, FIRST ASSESSMENT 2027)
==

Course link: https://learn.modernagecoders.com/courses/ib-diploma-computer-science-course
Subject area: IB Diploma Computer Science
Designed for: Teens
Who can join: IB Diploma Programme students taking Computer Science at SL or HL
Course length: Ongoing DP support, joinable any month
Time needed: 2 live classes/week + coursework and IA support
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150

Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online IB Diploma Computer Science classes for the new guide: both themes, the case study in Paper 1, algorithmic thinking in Paper 2 (Python or Java), and full support on the internal assessment.

In one line: Taught to the new guide, first assessed May 2027, with the internal assessment supported properly instead of left to the last term.

ABOUT THE PROGRAMME

IB Computer Science changed. The new Diploma Programme guide was launched in February 2025 for first teaching in August 2025, with first assessment in May 2027, and the differences are not cosmetic. Paper 3 has been removed entirely at both SL and HL. The options are gone. The case study is now covered by both SL and HL students and is assessed within Paper 1. Databases and object-oriented programming are now studied at both levels, not just HL. Any course still built around the old structure is preparing students for an examination that no longer exists. This course follows the new guide as it stands. The syllabus is organised around two themes: concepts in computer science, and computational thinking and problem-solving. Theme A covers computer fundamentals, networks, databases and machine learning. Theme B is where students learn to actually solve problems with code, which is what Paper 2 tests. The programme is built for how the Diploma actually runs. Theory is taught in the syllabus's own order so it lines up with school teaching, the case study is worked properly rather than skimmed the term before the exam, and the internal assessment, the computational solution, is supported from planning through to documentation, with the new emphasis on evidencing the computational thinking process. For Paper 2 we teach Python by default and support Java, since the new guide runs two separate versions of that paper and schools choose one at department level.

WHAT MAKES THIS COURSE DIFFERENT

- Taught to the NEW guide (first teaching 2025, first assessment May 2027): Paper 3 removed, options gone, case study inside Paper 1, databases and OOP at both SL and HL
- The internal assessment is supported from the start, not rescued at the end, including the new emphasis on documenting the computational thinking process
- The case study is worked in depth, because it now applies to SL students too and is assessed in Paper 1
- Paper 2 taught in the language your school chose: Python by default, Java supported, since the guide runs two separate versions of the paper
- Real programming, not slideware: students write and run code every week, which is the only thing that makes Paper 2 tractable
- Live, small batches with work reviewed on the spot, and SL and HL students taught to their own level

THE LEARNING PATH, STEP BY STEP

Phase 1: Theme A part one: computer fundamentals, system architecture and networks, with the ethical issues woven through
Phase 2: Theme A part two: databases and SQL, then machine learning and AI, plus the annual case study
Phase 3: Theme B: computational thinking, algorithms and OOP in Python or Java, the internal assessment, then full exam practice

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What Your Child Needs to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THEME A, COMPUTER FUNDAMENTALS AND NETWORKS

How computing systems actually work, taught so the Paper 1 answers come from understanding rather than recall.

Month 1: Computer Fundamentals and System Architecture (Weeks 1-4)
Hardware, Software and the System
Topics:
- The components of a computer system and how they interact

- Primary and secondary memory, and why the distinction matters
- System software versus application software
- The role of the operating system as a resource manager
- Reading a system description and identifying what each part contributes

Projects built:
- Map a described real system end to end, labelling every hardware and software component

Practice: 10 structured questions in IB command-term style

System Architecture and the Processor

Topics:
- CPU architecture, registers and buses
- The machine instruction cycle, step by step
- Cache and how it affects real performance
- Binary representation and why everything reduces to it
- Comparing architectures on evidence rather than assertion

Projects built:
- Trace a full instruction cycle and explain each stage in your own words

Practice: 10 architecture questions including compare and evaluate commands

Data Representation and Abstraction

Topics:
- Representing numbers, text, images and sound in binary
- Abstraction as the central idea of the whole course
- Why abstraction makes large systems possible
- Identifying the level of abstraction in a described system
- Using precise language, which the IB rewards heavily

Projects built:
- Describe one everyday system at three different levels of abstraction

Practice: 10 representation and abstraction questions

Assessment: A timed check on computer fundamentals, marked against IB command terms

Ethical and Social Issues in Computing

Topics:
- The ethical dimension the guide expects you to weave through every topic
- Privacy, surveillance, data ownership and digital divide
- Building a balanced argument with real stakeholders named
- Why one-sided answers cap your marks on evaluate questions
- Linking ethics to the specific technology in the question

Projects built:
- Write a balanced evaluation of one real computing controversy, naming stakeholders on both sides

Practice: 6 extended-response ethics questions, peer reviewed

Month 2: Networks, Security and the Cloud (Weeks 5-8)

Network Structures and Types

Topics:
- LAN, WAN and the topologies that matter
- Client-server and peer-to-peer models compared
- Network hardware and what each device is responsible for
- Bandwidth, latency and what actually limits real performance
- Choosing and justifying a network design for a scenario

Projects built:
- Design a network for a described organisation and justify every choice

Practice: 10 network-design questions with justification marks

Protocols and Data Transmission

Topics:
- Why protocols exist and what would happen without them
- Layers, packets and packet switching
- TCP/IP and HTTP at the depth the guide requires
- Error handling and flow control in plain terms
- Tracing a request across a network

Projects built:
- Trace a single web request across the layers, naming the protocol at each step

Practice: 10 protocol questions at exam standard

Cybersecurity

Topics:
- Threats: malware, phishing, denial of service, interception, social engineering
- Encryption, symmetric and asymmetric, explained precisely
- Authentication, access control and firewalls
- Matching a defence to a specific threat, which is where the marks are
- The human factor, usually the real vulnerability

Projects built:
- Produce a security assessment for a described system with prioritised, justified recommendations

Practice: 10 threat-and-mitigation scenarios

Assessment: End-of-phase test on networks and security

Cloud Computing

Topics:
- What cloud computing actually is, beyond the marketing
- Service and deployment models at guide depth
- The genuine advantages and the genuine risks
- Data sovereignty and the ethical questions it raises

- Evaluating whether cloud is right for a described organisation
Projects built:
- Write a balanced recommendation on moving a described organisation to the cloud
Practice: 8 evaluation questions on cloud adoption

PHASE 2: DATABASES, MACHINE LEARNING AND THE CASE STUDY

The two Theme A areas that are new or expanded for both SL and HL, plus the case study that
is now assessed inside Paper 1.

Month 3: Databases and SQL (Weeks 9-12)
The Relational Model
Topics:
- Tables, records, fields, primary and foreign keys
- Relationships and why they prevent duplicated data
- Normalisation at the level the guide expects
- Reading and drawing an entity relationship diagram
- Why databases are now examined at SL as well as HL
Projects built:
- Design a normalised relational schema for a described scenario and draw its ER diagram
Practice: 8 schema-design tasks with keys and relationships identified
SQL in Practice
Topics:
- SELECT, FROM, WHERE, ORDER BY
- Aggregate functions and GROUP BY
- Joining tables to answer a real question
- INSERT, UPDATE and DELETE
- Turning a written requirement into the query that satisfies it
Projects built:
- Build a working database and write the queries that answer real questions about it
Practice: 12 SQL problems building from single conditions to joins and aggregation
Data Storage, Integrity and Ethics
Topics:
- Data integrity, validation and consistency
- Transactions and why they matter
- Backup, recovery and redundancy
- Data protection and the ethics of holding personal data
- Connecting database design to the privacy questions from Phase 1
Projects built:
- Audit a described database for integrity and privacy risks and recommend fixes
Practice: 8 integrity and ethics questions
Assessment: A timed check on databases and SQL
Machine Learning and AI
Topics:
- What machine learning is, and how it differs from ordinary programming
- Supervised and unsupervised learning at guide depth
- Training data, features and why data quality decides outcomes
- Bias in data and its real consequences
- Where AI is genuinely appropriate and where it is not
Projects built:
- Explain how a described ML system reaches a decision, then identify where bias could enter
Practice: 10 machine-learning questions including an ethics evaluation

Month 4: The Case Study and Computational Thinking (Weeks 13-16)
Working the Annual Case Study
Topics:
- How the case study now sits inside Paper 1 for both SL and HL
- Reading the case study actively rather than passively
- Building a glossary of its specific terminology
- Researching the underlying technology properly
- Anticipating the question types the case study invites
Projects built:
- Produce a full annotated summary and terminology glossary for the current case study
Practice: Structured questions drawn directly from the case study material
Case Study Depth and Application
Topics:
- Linking case study technology back to Theme A concepts
- Preparing evidence-based extended answers
- Handling unfamiliar terminology under exam pressure
- Evaluating solutions proposed within the case study
- Practising the specific command terms it attracts
Projects built:
- Write two full extended responses on the case study and mark them against the criteria
Practice: Timed case-study responses with feedback
Assessment: A timed case-study section under Paper 1 conditions
Computational Thinking
Topics:
- Decomposition, pattern recognition, abstraction and algorithm design

- Thinking through a problem before writing any code
- Representing a solution as pseudocode and as a flowchart
- Tracing an algorithm accurately with a trace table
- Why this thinking is now explicitly assessed in the IA

Projects built:
- Take one problem from analysis through to a fully traced algorithm, documenting each thinking step

Practice: 10 algorithm-design and trace-table tasks

Algorithms and Efficiency

Topics:
- Standard algorithms: searching and sorting
- Comparing algorithms on efficiency rather than preference
- Arrays and collections and when each is right
- Recursion at the depth the guide expects
- Explaining why one algorithm beats another for a given case

Projects built:
- Implement and compare two searching or sorting algorithms on the same data and report the difference

Practice: 10 algorithm and efficiency questions

PHASE 3: PROGRAMMING, THE INTERNAL ASSESSMENT AND EXAM MASTERY

Theme B in code, the computational solution built properly, and both papers drilled under timed conditions.

Month 5: Programming and Object-Oriented Programming (Weeks 17-20)

Programming Fundamentals in Your Paper 2 Language

Topics:
- Variables, data types, input and output
- Selection and iteration, and nesting them cleanly
- Writing readable code that a marker can follow
- Python by default, Java supported, matching the version of Paper 2 your school chose
- Debugging as a method rather than guesswork

Projects built:
- Write and run a complete program that solves a described problem end to end

Practice: 12 short programs, each tested with your own test data

Data Structures and File Handling

Topics:
- Arrays, lists and dictionaries or maps
- Two-dimensional structures and traversing them
- Reading from and writing to files
- Choosing the right structure for the problem
- Handling errors and unexpected input

Projects built:
- Build a program that loads data from a file, processes it and writes a result

Practice: 10 data-structure and file-handling programs

Object-Oriented Programming (SL and HL)

Topics:
- Classes and objects, and why OOP exists
- Encapsulation with private fields and accessor methods
- Inheritance and polymorphism at guide depth
- Designing a small class hierarchy for a real problem
- Why OOP is now studied at both SL and HL under the new guide

Projects built:
- Design and implement a small object-oriented solution with at least two related classes

Practice: 8 OOP design-and-implement tasks

Assessment: A timed programming assessment covering structures and OOP

Starting the Internal Assessment

Topics:
- What the computational solution actually requires
- Choosing a problem with a real client and genuine scope
- Planning: success criteria, record of tasks, design
- The new emphasis on evidencing the computational thinking process
- Common IA mistakes that cost marks before a line of code is written

Projects built:
- Agree an IA problem with a real client and complete the planning and design documentation

Practice: Draft planning documents reviewed against the assessment criteria

Month 6: Completing the IA and Full Exam Practice (Weeks 21-24)

Building and Documenting the Solution

Topics:
- Developing the solution in stages against the success criteria
- Documenting techniques used, with justification
- Evidencing the computational thinking process as the new guide expects
- Testing with real data and recording the outcomes
- Keeping the record of tasks honest and current

Projects built:
- Build the working solution and complete the development documentation

Practice: Weekly IA checkpoints against the published criteria
Finishing the IA and the Video
Topics:
- Evaluating the solution against every success criterion
- Client feedback and honest recommendations for improvement
- Producing the required video demonstration
- Final checks on structure, evidence and word limits
- Academic honesty and correct referencing
Projects built:
- Complete and submit the full internal assessment package
Practice: Final IA review against every criterion
Assessment: Full IA marked against the official criteria with written feedback
Paper 1 Technique
Topics:
- How Paper 1 is structured under the new assessment model
- IB command terms: state, describe, explain, compare, evaluate, and what each is worth
- Structuring an extended response so every mark is reachable
- Handling the case study section confidently
- Managing time so the long questions are not rushed
Projects built:
- A full timed Paper 1, reviewed question by question with every miss tagged by cause
Practice: Two timed Paper 1 attempts, fully reviewed
Paper 2 and Full Mocks
Topics:
- How Paper 2 tests algorithmic thinking rather than recall
- Writing correct code by hand under time
- Reading the problem properly before starting to write
- Full timed mocks on both papers under real conditions
- Turning the error log into a final revision plan
Projects built:
- Two complete mock examinations across both papers, each fully reviewed
Practice: Repeated full mocks until pacing and accuracy are consistent
Assessment: Final full mock on both papers, benchmarked against the target grade

HOW A TYPICAL WEEK IS SPLIT

Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Strong SL or HL Computer Science grades under the new assessment model
- A completed internal assessment that is genuinely the student's own work
- Real programming ability in Python or Java
- A credible foundation for a computer science or engineering degree application
- Computational thinking that transfers across the whole Diploma

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course taught to the new IB Computer Science guide?
A: Yes. The new DP Computer Science guide was launched in February 2025 for first teaching in August 2025, with first assessment in May 2027. We teach that structure: Paper 3 removed at both SL and HL, options no longer available, the case study covered by SL and HL students and assessed within Paper 1, and databases and object-oriented programming studied at both levels.

Q: What actually changed from the old syllabus?
A: The biggest changes are that Paper 3 has been removed entirely at both SL and HL, the options are gone, the case study now applies to SL students too and is assessed inside Paper 1, and databases and OOP are now studied at both SL and HL rather than being HL territory. The internal assessment remains a computational solution but with a new emphasis on documenting the computational thinking process.

Q: Do you teach Python or Java for Paper 2?
A: Both are supported. The new guide runs two separate versions of Paper 2, one in Python and one in Java, and schools choose which at department level. We teach Python by default because most students find it clearer, and we teach Java for students whose school selected that version. Tell us which your school chose and we place your child accordingly.

Q: Can you help with the internal assessment?
A: Yes, and we start it early rather than rescuing it at the end. We support choosing a

problem with a real client, writing the success criteria, the design and record of tasks, building and documenting the solution, testing, the evaluation, and the video. The new guide puts more weight on evidencing the computational thinking process, so we build that documentation as the work happens. The solution must be the student's own work, and we keep it that way.

Q: My child is taking SL. Is this course still right?
A: Yes. Under the new guide SL students cover the case study, databases and object-oriented programming too, so the content overlap with HL is larger than it used to be. We teach SL and HL students within the same live batch but to their own required depth, and HL-only extension work is handled separately.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ib-diploma-computer-science-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 82 OF 137: IB MATHS AA & AI: SL & HL, LIVE ONLINE WITH IA COACHING
==========

Course link: https://learn.modernagecoders.com/courses/ib-mathematics-aa-ai-masterclass
Subject area: Exam Board Mathematics
Designed for: High school
Who can join: IB DP students (AA & AI, SL & HL) and MYP students heading to the DP
Course length: 24 months (the full Diploma Programme arc)
Time needed: 2 live one-hour classes per week + weekly past-paper work
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online IB Maths course for AA and AI, SL and HL: syllabus-exact teaching, paper-specific technique, GDC fluency and honest IA coaching. Free demo first.

In one line: Your exact course, your exact level, taught to a 7.

ABOUT THE PROGRAMME

A full-Diploma live program matched to the student's actual IB course: Analysis & Approaches or Applications & Interpretation, at SL or HL. IB maths punishes generic tutoring, the two courses examine different things, AA runs a strictly non-calculator paper while AI assumes the GDC everywhere, HL adds an extended problem-solving paper, and the Internal Assessment is worth a fifth of the grade. This masterclass teaches the syllabus for genuine ownership, trains paper-specific technique on real past questions and mark schemes, builds GDC fluency as a scored skill, and coaches the Mathematical Exploration honestly, structure, criteria and critique, with the work staying the student's own under IB academic integrity rules.

WHAT MAKES THIS COURSE DIFFERENT

- Course-exact teaching: AA and AI are different subjects here, as they are on exam day
- Non-calculator craft for AA Paper 1 and GDC power-workflows for AI, each trained deliberately
- The IA coached properly: topic selection against the actual criteria, milestones, mathematical depth and honest draft critique, never ghost-writing
- HL Paper 3 stamina built through olympiad-flavoured problem work
- Predicted-grade awareness: the arc aims at internal assessments too, because those set the number universities see
- MYP-era gaps repaired inline instead of ignored
- One dedicated mentor across the whole two-year arc
- Honest course-fit advice, including 'stay at SL' when that is the truth

THE LEARNING PATH, STEP BY STEP

Phase 1: The shared core built to DP strength: algebra, functions and the toolkit both courses stand on, with MYP debt repaired
Phase 2: Course-specific depth: calculus and proof for AA, statistics, modelling and

technology for AI, plus IA season, shepherded
Phase 3: Syllabus completion, paper craft by course and level, HL Paper 3 training and full past-paper cycles into the exam session

WHAT A STUDENT NEEDS BEFORE STARTING

Education: IB DP students (either course, either level) or MYP years 4-5 students heading to the DP
Math background: Any, the diagnostic finds the real base and MYP-era gaps are repaired inline
Equipment: Computer with stable internet; an IB-approved GDC (we advise on models)
Time commitment: 2 live hours plus 2-3 hours of past-paper and IA work weekly
English: Course taught in English
Motivation: The Diploma provides it; we supply the method and the calm

WHO THIS COURSE IS FOR

Aa students: AA SL/HL students who want the ideas owned and Paper 1 made routine
Ai students: AI SL/HL students who need modelling depth and real GDC fluency
Ia season: Students whose Exploration is drifting and needs structure and honest critique
Hl ambition: HL students building the Paper 3 stamina regular homework never builds
Myp runway: MYP students buying the cheapest IB insurance: a real algebra base

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE SHARED CORE AT DP STRENGTH (MONTHS 1-8)

Both IB courses stand on the same five topic areas. This phase builds number & algebra and functions to Diploma strength, repairs MYP-era gaps inline, and establishes the retrieval and paper habits that a two-year syllabus demands.

Months 1-2: Diagnostic, Course Fit and Algebra at DP Pace (Weeks 1-8)
Diagnostic and Course Fit (AA vs AI, SL vs HL)
Topics:
- Full syllabus-position diagnostic across the five IB topic areas
- Locating MYP-era gaps in algebra and number
- Current GDC habits and calculator setup review
- Evidence-based AA vs AI course comparison
- SL vs HL demand and workload honesty
- Baseline record set up for later progress tracking
Practice: Complete a short mixed diagnostic set and log every gap it exposes into a personal error log.
Detail: Full diagnostic: syllabus position, MYP-era gaps, GDC habits, and an honest course-fit read (AA vs AI, SL vs HL) with evidence
Exponents and Logarithms to DP Fluency
Topics:
- Laws of exponents from first meaning
- Rational and negative indices
- Logarithms as the inverse of exponentials
- Laws of logarithms and change of base
- Solving exponential and logarithmic equations
- Fluency drills that both courses assume
Practice: Work a fluency set converting between exponential and logarithmic form and solving equations without hesitation.
Detail: Exponents, logarithms and the laws that both courses assume, taught from meaning, drilled to fluency
Sequences, Series and Financial Applications
Topics:
- Arithmetic sequences and series
- Geometric sequences and series
- Sigma notation fluency
- Sum to infinity and the idea of convergence
- Compound interest and annuities (an AI favourite)
- Financial modelling contexts on the GDC
Practice: Solve a mixed set that pairs pure sequence questions with a compound-interest financial application.
Detail: Sequences and series: arithmetic, geometric, sigma notation and financial applications (an AI favourite)
Proof and Mathematical Writing Foundations
Topics:
- Direct proof structure
- Clear mathematical writing and notation
- Logical connectives and reasoning
- Counterexamples and disproof
- Deeper proof demands for AA HL
- Setting the standard for written working
Practice: Write out two short direct proofs with full justification and correct

notation.
Detail: Proof and reasoning foundations: direct proof and mathematical writing habits (deeper for AA HL)

Months 3-5: Functions, the DP's Load-Bearing Topic (Weeks 9-20)
Function Anatomy: Domain, Range, Composition, Inverses
Topics:
- Domain and range from meaning
- Function notation and evaluation
- Composite functions
- Inverse functions and their graphs
- One-to-one functions and the horizontal-line idea
- Why the whole DP silently assumes this fluency
Practice: Build a reference sheet of domain, range and inverse for a set of standard functions.
Detail: Function anatomy: domain, range, composition and inverses, the material the whole DP silently assumes
Transformations, Quadratics and Polynomials
Topics:
- Translations, stretches and reflections of graphs
- Reading graphs fluently
- Quadratics in vertex, root and factored forms
- The discriminant as an idea, not just a formula
- Polynomial behaviour and the factor theorem
- Sketching to DP depth
Practice: Sketch a family of transformed graphs by hand, then confirm each on the GDC.
Detail: Transformations and graph literacy; quadratics and polynomials to DP depth; the discriminant as an idea
Exponential, Log and Rational Functions with the GDC
Topics:
- Exponential growth and decay models
- Logarithmic functions and scales
- Rational functions and asymptotes
- Modelling contexts drawn from data
- GDC graphing workflow set up properly
- Interpreting features straight from a graph
Practice: Model a given real context with an exponential or rational function and justify the fit on the GDC.
Detail: Exponential, logarithmic and rational functions with modelling contexts; GDC graphing workflows introduced properly

Months 6-8: Geometry, Trigonometry and First Paper Craft (Weeks 21-32)
Trigonometry: Unit Circle to Applied Bearings
Topics:
- Unit circle and exact values (AA depth)
- Trigonometric identities
- Solving trigonometric equations (AA)
- Right-triangle trigonometry (AI emphasis)
- Applied trig with bearings and angles of elevation
- Radian measure introduced
Practice: Complete a split set: AA exact-value and identity work, AI applied bearings problems.
Detail: Trigonometry rebuilt: unit circle, identities and equations (AA depth) / right-triangle and applied trig with bearings (AI emphasis)
Vectors, 3D Geometry and the Sine and Cosine Rules
Topics:
- Vectors and vector operations (AA HL)
- 3D coordinate geometry
- The sine rule in non-right triangles
- The cosine rule and the ambiguous case
- Area of a triangle formula
- Applied problems combining the rules
Practice: Solve a set of non-right-triangle problems, choosing the sine or cosine rule deliberately.
Detail: Vectors (AA HL) and 3D geometry; sine and cosine rules in applied problems
First Past-Paper Season and the Review Ritual
Topics:
- Anatomy of an IB exam question
- Command terms and what they demand
- Reading and applying mark schemes
- Method marks versus answer marks
- The review ritual used for the next eighteen months
- Building the error-log habit
Practice: Sit one short past-paper section under time, then self-mark it against the real mark scheme.
Assessment: End-of-phase checkpoint: a timed past-paper section marked against the official scheme sets the Phase 1 baseline.
Detail: First past-paper season: question anatomy, mark-scheme logic, and the review ritual used for the next eighteen months

---
PHASE 2: COURSE-SPECIFIC DEPTH + THE IA (MONTHS 9-16)

The AA and AI tracks now genuinely diverge: calculus and proof depth for AA, statistics and modelling with full GDC fluency for AI. In the middle of this phase sits IA season, the Mathematical Exploration, coached against the real criteria while remaining the student's own work.

Months 9-11: The AA Track / The AI Track (Weeks 33-44)
AA Differentiation from First Principles / AI Statistics and Regression
Topics:
- AA: limits and the idea of a derivative
- AA: differentiation from first principles
- AA: derivative toolkit and chain rule mastery
- AI: descriptive statistics measures
- AI: correlation and Pearson r
- AI: linear regression on the GDC
Practice: AA students differentiate a mixed set including chain-rule cases; AI students run a regression on a dataset and interpret it.
Detail: AA: limits and differentiation from first principles; the derivative toolkit including chain rule mastery. AI: descriptive statistics, correlation and regression on the GDC
AA Applications and Integration / AI Probability Distributions
Topics:
- AA: tangents and normals
- AA: optimisation problems
- AA: kinematics with calculus
- AA: introduction to integration
- AI: probability rules and diagrams
- AI: binomial and normal distributions with interpretation
Practice: AA students solve an optimisation and a kinematics problem; AI students compute and interpret binomial and normal probabilities.
Detail: AA: applications, tangents, optimisation, kinematics; introduction to integration. AI: probability, distributions (binomial, normal) and interpretation craft
AA Integration and Areas / AI Modelling Cycle and Networks
Topics:
- AA: integration techniques
- AA: definite integrals and area under a curve
- AA: volumes of revolution
- AI: the modelling cycle, choose, fit, critique, improve
- AI: matrices and their operations (HL)
- AI: networks and graph theory (HL)
Practice: AA students find an area and a volume by integration; AI students fit and critique a model for a given dataset.
Detail: AA: integration techniques and areas/volumes. AI: modelling cycle, choose, fit, critique and improve a model; matrices and networks (HL)

Months 12-14: IA Season, the Exploration, Shepherded (Weeks 45-56)
IA Topic Selection and Proposal
Topics:
- The five IA assessment criteria explained
- Genuine personal engagement
- Mathematics chosen so it can actually score
- Scoping a workable exploration question
- A milestone plan for the IA
- Academic integrity expectations set from the start
Practice: Draft three candidate IA questions and test each honestly against the five criteria.
Assessment: IA milestone: the exploration proposal is locked with a dated milestone plan.
Detail: Topic selection against the five criteria: personal engagement that is real, mathematics that can score; proposal locked with milestones
Deepening the IA Mathematics
Topics:
- Pushing mathematical rigour in the exploration
- Meeting the aim criterion
- Communication and structure standards
- Correct notation throughout
- Student-led work with mentor questioning
- Keeping the exploration honestly the student's own
Practice: Develop the core mathematics of your exploration and annotate where each criterion is being met.
Detail: The mathematics deepened: the mentor pushes rigour and aim while the student drives; communication and notation standards set
IA Draft Critique and Final Polish Plan
Topics:
- Honest critique against each criterion
- Strengthening the reflection

- Improving communication and presentation
- The final polish plan
- Academic integrity kept throughout
- Self-checking before submission

Practice: Revise your IA draft against criterion-by-criterion feedback and record the changes you make.

Assessment: IA milestone: a full draft is reviewed against all five criteria before the final polish.

Detail: Draft critique rounds: honest feedback against each criterion, reflection strengthened, and the final polish plan, the work staying the student's own throughout, per IB academic integrity rules

Months 15-16: Syllabus Depth Continues (Weeks 57-64)

AA Further Calculus / AI Hypothesis Testing

Topics:

- AA: further integration and differentiation
- AA HL: deeper techniques and Maclaurin ideas
- AI: the logic of hypothesis testing
- AI: chi-squared tests for independence
- AI: t-tests and interpretation
- Reading statistical output critically

Practice: AA students tackle a further-calculus set; AI students run and interpret a chi-squared or t-test.

Detail: AA: further calculus (HL: deeper techniques, Maclaurin ideas); AI: hypothesis testing, chi-squared and t-tests with interpretation

HL Paper 3 Investigations Introduced / SL Consolidation

Topics:

- HL: the structure of Paper 3 extended investigations
- HL: harvesting earlier parts of a problem
- HL: sustaining an unfamiliar investigation
- SL: consolidation by error-log priority
- SL: targeted retrieval of weak topics
- Both: connecting topics across the syllabus

Practice: HL students attempt one extended investigation; SL students clear their top three error-log items.

Assessment: End-of-phase checkpoint: HL students are marked on a Paper 3 style investigation and SL students on a targeted consolidation set.

Detail: HL students of both courses: Paper 3-style extended investigations introduced; SL students: consolidation cycles by error-log priority

---

PHASE 3: SYLLABUS CLOSE + PAPER CRAFT TO EXAM SESSION (MONTHS 17-24)

The remaining syllabus is closed with margin, then the program becomes exam craft: papers by course and level, calculator and non-calculator discipline, HL Paper 3 stamina, and full timed cycles that peak in the exam session, with predicted-grade checkpoints along the way.

Months 17-19: Closing the Syllabus with Margin (Weeks 65-76)

Closing the Final Syllabus Topics

Topics:

- Remaining probability and statistics (AA)
- Remaining calculus for AI HL
- Kinematics and applied problems
- Any outstanding course-specific topics
- Topics taught early enough to settle
- Linking new topics back to earlier work

Practice: Complete a set on the last new topics for your exact course and level.

Detail: Final syllabus topics per course and level, taught early enough to settle: remaining probability/statistics (AA), remaining calculus for AI HL, kinematics and applications

Full-Syllabus Connection Map and Spaced Retrieval

Topics:

- Building a personal syllabus connection map
- Linking the five topic areas
- Scheduling spaced retrieval
- Interleaving topics in practice
- Identifying weak links to revisit
- Preparing for full paper cycles

Practice: Build a one-page connection map of the whole syllabus and schedule your spaced retrieval.

Detail: Full-syllabus connection map built by the student; spaced retrieval cycles across all five topic areas

Months 20-22: Paper Craft by Course and Level (Weeks 77-88)

AA Paper 1 Non-Calculator Craft / AI GDC Speed

Topics:

- AA: exact-value fluency
- AA: structure recognition inside questions
- AA: clean written working for method marks

- AI: GDC speed workflows
- AI: efficient graphing, solving and regression
- Acceptable use of technology

Practice: AA students complete a non-calculator set; AI students time their GDC workflows on typical questions.

Detail: AA Paper 1 non-calculator craft: exact values, structure recognition and clean written working. AI: GDC speed workflows for every paper

Paper 2 Technique and Time Allocation

Topics:

- Strategy for extended structured questions
- Method-mark discipline
- Time allocation from personal data
- Recovering from a stuck part
- Presentation that earns follow-through marks
- GDC use within Paper 2

Practice: Sit a Paper 2 section and log your time per question to set personal pacing targets.

Detail: Paper 2 technique: extended structured questions, method-mark discipline and time allocation from personal data

HL Paper 3 Stamina and Investigation Craft

Topics:

- Long unfamiliar investigations
- Harvesting earlier parts to reach later ones
- Keeping momentum through difficulty
- Managing time across a long problem
- The calm that stamina training buys
- Reviewing Paper 3 against the scheme

Practice: Complete a full HL Paper 3 style investigation under time and review it against the mark scheme.

Detail: HL Paper 3: long unfamiliar investigations, harvesting earlier parts, keeping momentum, and the calm that stamina training buys

Months 23-24: Full Cycles and the Exam Session (Weeks 89-96)

Full Timed Past-Paper Cycles

Topics:

- Full timed past-paper cycles
- Papers for the exact course and level
- Marking against real mark schemes
- Error-log closure
- Spotting patterns across mistakes
- Refining exam-day strategy

Practice: Sit a full timed paper, self-mark it, and close the error-log items it reveals.

Assessment: Mock checkpoint: full timed papers are marked against official schemes and score progression is recorded.

Detail: Full timed past-paper cycles by exact course and level, reviewed against real mark schemes; error-log closure

Peak Plan and the Exam Session

Topics:

- Final targeted consolidations
- Rest and recovery discipline
- Exam-day routines
- GDC and materials checklist
- Managing nerves through rehearsal
- Meeting the session as a rehearsed place

Practice: Run a final light consolidation and rehearse your full exam-day routine.

Assessment: The IB examination session: papers are sat for the student's exact course and level.

Detail: Peak plan: final consolidations, rest discipline, exam-day routines, and the session met as a rehearsed place

PROJECTS THE STUDENT BUILDS AND KEEPS

- Two-year error log and syllabus map, maintained by the student
- The Mathematical Exploration (IA), coached against the real criteria
- GDC workflow library (AI) / exact-value fluency system (AA)
- Self-graded past-paper portfolio against real mark schemes
- Full timed paper cycles with documented score progression

Total projects: The IA plus 5+ exam artifacts

SKILLS THE STUDENT WILL HAVE AT THE END

- Algebra, functions and trigonometry at DP strength
- AA: calculus, proof and non-calculator craft
- AI: statistics, modelling and GDC power-workflows
- Mark-scheme literacy and method-mark discipline
- HL Paper 3 stamina and investigation technique
- IA construction against the five criteria

- Predicted-grade exam readiness
- Two-year retrieval and revision systems

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes with your mentor
Past paper work: Weekly question sets from your exact course and level
Ia season: Milestone check-ins during Exploration months
Total per week: About 4-5 focused hours
Homework: A problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week
Mentorship: One dedicated mentor across the whole Diploma
Recordings: Every class recorded for revision
Progress tracking: Paper-score progression + IA milestones shared with parents
Flexibility: Intensity adjusts around mocks, IA deadlines and the exam session

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies mastery, not attendance

WHERE THIS CAN LEAD

- An IB maths grade that holds predicted grades and university offers
- AA route: engineering, mathematics, physics and CS degree readiness
- AI route: business, economics and social-science quantitative literacy
- Ready for College Mathematics Masterclass or competition mathematics

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do you teach both AA and AI, at both SL and HL?
A: Yes, and separately, because they are genuinely different courses. The mentor teaches your student's exact registration: AA's algebraic, proof-flavoured, calculus-heavy road or AI's modelling, statistics and technology-first road, at the right level. The free demo doubles as a course-fit consult if the choice is still open. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: How does the IA coaching work, and how far is too far?
A: We coach the Mathematical Exploration properly: topic selection against the actual criteria, structure, pushing the mathematics deep enough to score, and honest critique of drafts. We do not write it or polish prose into someone else's voice, IB academic integrity rules are real, schools check, and the IA only helps your student if it is genuinely theirs. Done right, it is the most coachable 20% of the grade.

Q: My AA student falls apart on the non-calculator paper. Fixable?
A: Very. Paper 1 skill is mostly exact-value fluency plus structure recognition, seeing the quadratic inside the exponential equation. Both are trained, not innate, and every AA class includes non-calculator work so Paper 1 becomes routine long before the exam.

Q: Do you teach GDC (graphing calculator) skills?
A: As a first-class skill, because AI examines it and AA's Paper 2 rewards it: efficient graphing, solving and regression workflows, and knowing what counts as acceptable use of technology. AI students who underuse the GDC leave marks and minutes on the table.

Q: Can you help before predicted grades are set?
A: That is the highest-leverage timing in the IB, predictions ride on school assessments, so a focused arc aimed at the next internal exams moves the number universities actually see. Start earlier than feels necessary; predictions harden sooner than most families expect.

Q: My child is in the MYP. Is this course relevant yet?
A: Yes, MYP students join the Phase 1 material aimed at arriving in the DP with the algebra and functions base both courses silently assume. It is the cheapest IB maths insurance there is.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A problem set follows every class with written feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with full mock papers as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real understanding, not attendance.

Q: Can we try it before paying?

A: Yes. Every student starts with a free live demo class that doubles as the diagnostic and course-fit consult, no card details, no obligation.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ib-mathematics-aa-ai-masterclass
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 83 OF 137: ICSE COMPUTER APPLICATIONS (CLASS 9-10): JAVA WITH BLUEJ

==========

Course link: https://learn.modernagecoders.com/courses/icse-computer-applications-java-bluej-course
Subject area: ICSE Computer Applications
Designed for: Teens
Who can join: ICSE Class 9 and 10 students taking Computer Applications; taught from the basics, no prior coding required
Course length: Class 9 and 10 board program, monthly enrolment
Time needed: 2 live classes/week + weekly BlueJ practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online ICSE Computer Applications classes for Class 9 and 10, taught in Java with BlueJ and aligned to the current CISCE 100+100 scheme, so students write their own programs and score in both the theory paper and the internal assessment.

In one line: A real Java course, taught in BlueJ and aligned to the current CISCE scheme, that ends in top marks in both the theory paper and the internal assessment, because your child can actually write programs, not just reproduce them.

ABOUT THE PROGRAMME

ICSE Computer Applications is taught in Java using the BlueJ environment, and it is the subject where students who genuinely understand code pull away from the ones who memorise programs. This course treats it as a real first programming course, because that is exactly what it is: your child leaves able to write Java, read an error, and solve a problem they have not seen before, which is what the board paper actually asks for. It is also taught to the scheme as it stands now. CISCE marks Computer Applications on a 100 plus 100 basis, unlike the usual 80 plus 20: a 100-mark external theory paper and a 100-mark internal assessment made up of practical lab work and a written project on Disruptive Technologies. Many students lose easy marks on the internal assessment simply because nobody prepared them for it, so we treat both halves seriously from day one. The programme follows the ICSE Class 9 and 10 Computer Applications syllabus in order. It begins with object-oriented thinking and Java from nothing, moves through conditionals, loops and nested loops, then into the Class 10 core: classes as the basis of computation, user-defined methods, constructors, the library and wrapper classes, encapsulation, single and double dimensional arrays, and string handling. Alongside the theory, every student builds a proper practical file of at least fifteen BlueJ assignments and the Disruptive Technologies project, and drills the two-section board paper until earning marks is a habit. Every class is live and small, and every concept is practised by writing code in BlueJ, because that is the only way this subject is ever actually learned.

WHAT MAKES THIS COURSE DIFFERENT

- Aligned to the current CISCE 100 plus 100 scheme: a 100-mark theory paper and a 100-mark internal assessment, so we prepare your child for both halves, not just the written exam most classes stop at
- Taught in BlueJ (version 5.4.2 or higher on JDK 11 or higher), the exact environment used in school and the board practical exam, so nothing on exam day is unfamiliar
- The internal assessment is taken seriously: your child builds a real practical file of at least fifteen lab assignments plus the written project on Disruptive Technologies, worth a full 100 marks
- A real Java course, not rote learning: students stop memorising programs and start writing their own, which is exactly what Section B of the paper rewards
- Board paper technique drilled directly: the compulsory Section A short answers and the answer-any-four programs of Section B, practised against real board-style questions
- Live, small batches where your child writes code during class and gets it reviewed, not lecture videos watched alone

THE LEARNING PATH, STEP BY STEP

Phase 1: Java and OOP from scratch in BlueJ, the Class 9 core: objects and classes, data

types, operators, input, the Math library, conditionals, and loops including nested loops
Phase 2: The Class 10 core: the class as the basis of computation, user-defined methods, constructors, library and wrapper classes, and encapsulation
Phase 3: Arrays and string handling, the full practical file and Disruptive Technologies project, and complete board theory and BlueJ practical exam mastery

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What Your Child Needs to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: JAVA AND OOP FOUNDATIONS, THE CLASS 9 CORE (IN BLUEJ)

Object-oriented thinking and Java built from nothing in BlueJ, then the Class 9 Computer Applications syllabus in order: the base everything in Class 10 stands on.

Month 1: Object-Oriented Thinking and Java Basics (Weeks 1-4)
Introduction to OOP and the BlueJ Environment
Topics:
- What object-oriented programming means: objects, classes, and why Java is built around them
- The principles behind OOP the syllabus names: abstraction, encapsulation, inheritance and polymorphism, explained in plain language
- Setting up BlueJ (version 5.4.2 or higher on JDK 11 or higher), the exact tool used in school and the board practical
- The anatomy of a Java program: class, main method, statements and semicolons
- The compile-and-run cycle in BlueJ, and reading your first error messages without fear

Projects built:
- First BlueJ program: create a class, compile it, and run a method that prints a short formatted output

Practice: 8 short exercises creating and running simple classes in BlueJ, reading each error message and fixing it

Elementary Concept of Objects and Classes, Values and Data Types
Topics:
- Objects as instances of a class, and the class as a blueprint, with real everyday analogies
- Java's primitive data types: int, double, char, boolean and the rest, and when each is used
- Literals, identifiers, keywords and the rules for naming, exactly as the paper expects
- Type size, range and the difference between primitive and reference types
- Variables, initialisation, and Unicode and ASCII as the syllabus requires

Projects built:
- A class that stores and prints details using the correct data type for each field

Practice: 10 questions classifying data types and choosing the right type for a given value, board-question style
Assessment: A short written check on OOP terms and data types, marked in the board's short-answer style

Operators in Java and Expressions
Topics:
- Arithmetic, relational, logical and assignment operators, and operator precedence
- Increment and decrement, prefix versus postfix, a classic board trap
- Type conversion: implicit widening and explicit casting, and integer versus real division
- The conditional (ternary) operator and evaluating mixed expressions by hand
- Tracing expressions on paper the way Section A questions demand

Projects built:
- A simple calculator class that applies each operator and prints the result

Practice: 12 expression-evaluation questions, predicted on paper then checked in BlueJ

Input in Java and Mathematical Library Methods
Topics:
- Taking input with the Scanner class, the standard the syllabus uses
- Reading int, double, char and String input cleanly
- The Math library: pow, sqrt, abs, max, min, round, ceil, floor and random
- Combining input and Math methods to solve real formulae
- Common Math-method questions exactly as they appear in the paper

Projects built:
- A program that reads values and computes results such as area, simple interest and the roots of a quadratic using Math methods

Practice: 10 input-and-compute programs, each written and run in BlueJ
Assessment: A timed set on operators, expressions and Math methods, scored in the board's style

Month 2: Conditional and Iterative Constructs (Weeks 5-8)
Conditional Constructs: if, if-else, if-else-if
Topics:

- The if, if-else and if-else-if ladder, and choosing the right form
- Nested if statements and how to keep them readable
- Boolean conditions built from relational and logical operators
- Dry-running a decision program by hand to predict its output
- Menu-style decision problems as the paper frames them

Projects built:
- A program that classifies input, for example grade from marks or category from a value, using an if-else-if ladder

Practice: 10 decision programs, each dry-run on paper first then verified in BlueJ

The switch-case Construct

Topics:
- The switch-case statement and when it beats an if-else-if ladder
- The role of break, and the fall-through behaviour when it is missing, a favourite board trap
- The default case and valid switch expression types
- Converting between switch and if-else forms both ways
- Menu-driven programs using switch, exactly as asked in Section B

Projects built:
- A menu-driven program, for example a units converter or a simple arithmetic menu, built with switch-case

Practice: 8 switch-case programs including deliberate missing-break tracing questions

Assessment: A timed check on conditionals covering both if-else and switch, board style

Iteration: for, while and do-while Loops

Topics:
- The for, while and do-while loops and how to convert one into another
- Loop control: initialisation, condition and update, and off-by-one errors
- The entry-controlled versus exit-controlled distinction the paper asks about
- Classic loop programs: sum of a series, factorial, counting digits, reversing a number
- Predicting how many times a loop runs, a common Section A question

Projects built:
- A program producing a series result, for example the sum of a series, factorial, or checking a number for a property

Practice: 12 loop programs, each written three ways with for, while and do-while to build fluency

Nested Loops and Patterns

Topics:
- Nested for loops and how the inner loop runs fully for each outer step
- Number and star pattern programs, a guaranteed source of Section B marks
- Prime, palindrome, Armstrong and similar number-property programs
- Tracing nested loops on paper to predict the exact output
- Structuring a longer program cleanly so it earns full marks

Projects built:
- A pattern-printing program and a number-property checker, both built with nested loops

Practice: 10 nested-loop and pattern programs at board difficulty

Assessment: End-of-phase test: a Section-A-style short-answer set plus two Section-B-style programs, scored on the board rubric

---

PHASE 2: THE CLASS 10 CORE, CLASSES, METHODS AND ENCAPSULATION

The heart of the Class 10 syllabus: writing your own classes, methods and constructors, and using the library and wrapper classes the paper depends on.

Month 3: Classes, User-Defined Methods and Constructors (Weeks 9-12)

The Class as the Basis of All Computation

Topics:
- Revisiting objects and classes as the foundation of everything in Class 10
- Instance variables and methods, and the state and behaviour of an object
- Access specifiers: private, public and protected, and why they matter
- Creating objects and calling methods, and the role of the main method
- Reading a full class definition the way a Section B question presents it

Projects built:
- A complete class, for example a Student or BankAccount, with fields and methods, created and used from main

Practice: 8 exercises writing and using small classes in BlueJ

User-Defined Methods

Topics:
- Declaring methods: return type, name, parameters and body
- Parameters and arguments, and returning a value versus void
- Pure and impure methods, and call by value as the syllabus frames it
- Method overloading and how the compiler picks a version
- The static keyword and the difference between static and instance methods

Projects built:
- A class with several overloaded and value-returning methods, each tested from main

Practice: 10 method-writing questions, including overloading and pure-versus-impure identification

Constructors

Topics:
- What a constructor is and how it differs from a method
- Default, parameterised and copy constructors

- Constructor overloading and choosing the right one at object creation
- How constructors initialise object state cleanly
- Constructor questions exactly as the board asks them

Projects built:
- A class with default, parameterised and copy constructors, demonstrated by creating objects several ways

Practice: 8 constructor programs and a set of spot-the-difference method-versus-constructor questions

Assessment: A timed check on classes, methods and constructors, marked on the board rubric

Encapsulation and Clean Class Design

Topics:
- Encapsulation: private fields with public accessor and mutator methods
- Why data hiding protects an object and earns design marks
- Designing a class so its methods fully control its state
- Reading and completing a partially written class, a common Section B format
- Good naming and structure that make a program easy to mark

Projects built:
- An encapsulated class with private fields and get and set methods, used safely from outside

Practice: 8 class-completion questions in the board's fill-in-the-method style

Month 4: Library Classes, Wrapper Classes and Characters (Weeks 13-16)

Library Classes and Wrapper Classes

Topics:
- What the Java library provides and why we do not reinvent it
- Wrapper classes: Integer, Double, Character and the rest, and why they exist
- Autoboxing and unboxing in plain terms
- Converting between primitive types and their wrapper objects
- Parsing a String to a number with parseInt and parseDouble

Projects built:
- A program that reads numeric text input, parses it with wrapper methods, and computes a result

Practice: 8 wrapper-class and parsing questions in board format

Character and Case Handling

Topics:
- The char type, and ASCII and Unicode values
- Character methods: isDigit, isLetter, isUpperCase, isLowerCase, toUpperCase, toLowerCase
- Converting characters to and from their integer codes
- Counting vowels, consonants, digits and spaces, a classic program
- Character-based Section A questions and how to answer them precisely

Projects built:
- A program that scans text and reports counts of vowels, consonants, digits and other characters

Practice: 10 character-method programs and code-value tracing questions

Assessment: A timed set on library, wrapper and character methods, board style

Building the Practical File, Part One

Topics:
- The internal assessment: how the 100 practical marks are actually earned
- What makes a good lab assignment: clear design, correct implementation and testing
- Writing assignments that each demonstrate a theory concept, as CISCE requires
- Documenting a program properly: purpose, variable description and output
- Starting the required set of at least fifteen assignments for the year

Projects built:
- The first several formal BlueJ lab assignments, written up to internal-assessment standard

Practice: Assignment write-ups reviewed and corrected to the standard a school and CISCE expect

The Disruptive Technologies Project

Topics:
- The written project on Disruptive Technologies that carries part of the internal assessment
- Choosing a genuine topic, for example AI, blockchain, IoT or robotics
- Structuring the project: what it is, how it works, and its real-world impact
- Writing in the student's own words, honestly and clearly, not copied
- Presenting the project so it earns full internal-assessment marks

Projects built:
- A complete, well-structured written project on a chosen disruptive technology

Practice: Draft, review and finalise the project to internal-assessment standard

Assessment: End-of-phase test: a mixed Section-A and Section-B paper plus a practical-file review

---

PHASE 3: ARRAYS, STRINGS AND FULL BOARD AND PRACTICAL MASTERY

The last major syllabus topics, then the whole subject brought together: the two-section theory paper and the BlueJ practical exam, drilled until top marks are a habit.

Month 5: Arrays and String Handling (Weeks 17-20)

Single Dimensional Arrays

Topics:
- Declaring, creating and initialising a single dimensional array

- Traversing an array with a loop, and reading and printing elements
- Finding the sum, maximum, minimum and average of an array
- Linear search and simple frequency counting
- Array questions exactly as Section B presents them

Projects built:

- A program that reads a set of values into an array and reports statistics and a searched value

Practice: 10 single-dimensional-array programs at board difficulty

Sorting and Searching in Arrays

Topics:

- Bubble sort and selection sort, the two the syllabus expects, step by step
- Binary search on a sorted array and why order matters
- Tracing a sort or search pass by pass, a common Section A question
- Choosing the right technique for a given problem
- Writing a full sort-or-search program cleanly for maximum marks

Projects built:

- A program that sorts an array with bubble or selection sort, then binary-searches it

Practice: 8 sorting and searching programs, plus pass-by-pass tracing questions

Double Dimensional Arrays

Topics:

- Declaring and using a two dimensional array as rows and columns
- Nested loops to fill, read and print a matrix
- Row sums, column sums, and the diagonal of a square matrix
- Common matrix programs the paper sets
- Keeping a two-dimensional program readable so it earns full marks

Projects built:

- A matrix program that fills a two dimensional array and reports row, column and diagonal totals

Practice: 8 double-dimensional-array programs

String Handling

Topics:

- The String class and common methods: length, charAt, substring, indexOf, equals and compareTo
- toUpperCase, toLowerCase, trim, replace and concatenation
- Reversing a string, checking a palindrome, and counting words
- Comparing strings correctly, equals versus the double-equals trap
- String programs exactly as Section B frames them

Projects built:

- A text program that reverses input, checks for a palindrome, and counts words and characters

Practice: 10 string-handling programs at board difficulty

Assessment: A timed set covering arrays and strings, scored on the board rubric

Month 6: Board Theory and BlueJ Practical Exam Mastery (Weeks 21-24)

The Theory Paper: Section A Technique

Topics:

- How the 100-mark theory paper is structured across its two sections
- Section A: the compulsory short-answer questions and how to earn every mark
- Precise definitions, output prediction and one-line answers the way examiners want them
- Common Section A traps: operator precedence, missing break, loop counts and casting
- Timing Section A so Section B gets the minutes it needs

Projects built:

- A full Section A attempt under time, then a question-by-question review

Practice: Two timed Section A sets, each fully reviewed with every miss tagged by cause

The Theory Paper: Section B Programs

Topics:

- Section B: answering any four of the program questions, and choosing the right four
- Structuring a full program answer so partial marks are never lost
- Writing clean, commented code by hand under exam conditions
- Handling the class-design, array and string programs that recur every year
- Checking a written program for the small errors that cost marks

Projects built:

- A full Section B attempt of four hand-written programs, marked on the official rubric

Practice: Timed Section B sets across class, array and string program types

The BlueJ Practical Exam and the Internal Assessment

Topics:

- How the school and board practical works in BlueJ, and what is expected
- Writing, compiling, testing and correcting a program under time in BlueJ
- Finalising the practical file of at least fifteen assignments and the project
- Reading a practical problem correctly and planning before coding
- Debugging calmly when a program does not compile the first time

Projects built:

- A full mock practical: solve an unseen problem end to end in BlueJ under exam time

Practice: Repeated timed BlueJ practicals on unseen problems, each reviewed

Assessment: A complete mock practical scored like the real one, with the practical file signed off

Full Mock Board Exams and Final Polishing

Topics:

- Full-length timed mock theory papers under real conditions
- Turning the error log from every mock into a focused revision plan
- Locking in the recurring high-mark programs so they are automatic

- A calm, tested routine for both the theory paper and the practical
- A clear, honest read on the target grade and the final push to reach it

Projects built:
- Two full mock board papers, each fully reviewed and converted into a revision list

Practice: Repeated full mocks until pacing and accuracy are consistent

Assessment: Final full mock board paper, scored and benchmarked against the target grade

HOW A TYPICAL WEEK IS SPLIT

Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- A strong ICSE Computer Applications result in both the theory paper and the 100-mark internal assessment
- A confident move into ISC Computer Science in Class 11 and 12, continuing in Java
- A genuine foundation for a computer science stream and, later, an engineering or CS degree
- Transferable problem-solving and logical-thinking skills that help across every subject
- An early, real head start in programming that compounds for years

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course aligned to the current ICSE Computer Applications syllabus?
A: Yes. It follows the CISCE Class 9 and 10 Computer Applications syllabus, taught in Java with BlueJ, and it is built around the current 100 plus 100 scheme: a 100-mark external theory paper and a 100-mark internal assessment made up of practical lab work and a written project on Disruptive Technologies. We prepare your child for both halves, which is where many classes stop short.

Q: Do you teach in BlueJ, the environment used in school?
A: Yes. We teach Java in BlueJ, version 5.4.2 or higher on JDK 11 or higher, which is the same environment used in school and in the board practical exam. Your child writes, compiles and tests real programs in it from the first class, so nothing on exam day is unfamiliar.

Q: How does the 100-mark internal assessment work, and do you prepare for it?
A: Computer Applications is marked out of 200: a 100-mark theory paper and a 100-mark internal assessment. The internal assessment comes from practical lab work, at least fifteen assignments across the year, plus a written project on Disruptive Technologies. We build the practical file and the project properly with your child, so the full 100 internal marks are genuinely earned, not left to chance.

Q: My child finds Computer Applications confusing at school. Can this help?
A: This is exactly who the course is for. Most confusion comes from memorising programs without understanding them, so the moment the logic changes, a student is stuck. We teach the concepts from the basics, in a small live batch, until your child can write their own programs. That understanding shows up directly in both the paper and the practical.

Q: Can a Class 9 student join, or is it only for Class 10?
A: Both. A Class 9 student builds the foundations in order, which makes Class 10 far easier. A Class 10 student joins with a quick, solid revision of the Class 9 core and then moves into the Class 10 topics and full board preparation. We place your child at the right point after the demo class.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Will this help with ISC Computer Science later?
A: Yes. ISC Computer Science in Class 11 and 12 continues in Java and builds directly on the Class 9 and 10 foundation, adding topics like recursion, interfaces and exception handling. A student who genuinely understands Computer Applications is set up to do well in ISC, rather than starting from behind.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/icse-computer-applications-java-bluej-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 84 OF 137: IGCSE COMPUTER SCIENCE (0478): FULL SYLLABUS AND EXAM PREP
==========

Course link: https://learn.modernagecoders.com/courses/igcse-computer-science-0478-course
Subject area: IGCSE Computer Science
Designed for: Teens
Who can join: IGCSE students taking Computer Science 0478; taught from the basics, no prior coding required
Course length: 6 months (24 weeks), joinable any month
Time needed: 2 live classes/week + weekly programming practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Live online Cambridge IGCSE Computer Science 0478 classes covering both papers in full: computer systems, algorithms, programming in Python, databases and Boolean logic, with real exam technique.

  In one line: Both papers taught in full, with real programming in Python, so your child understands the subject instead of memorising it.

ABOUT THE PROGRAMME

  Cambridge IGCSE Computer Science is two subjects wearing one name. Half of it is how computers actually work, from binary and data transmission to hardware, networks and emerging technologies. The other half is thinking like a programmer: designing algorithms, writing and correcting real code, querying a database, and reasoning through logic circuits. Students who treat it as a memory subject stall on the second half, because you cannot memorise your way through writing a program you have never seen. This course teaches both halves properly. It follows the current 0478 syllabus (version 5, for examination in 2026, 2027 and 2028) topic by topic, and it is built around the fact that the qualification is assessed by two written papers of equal weight, with no coursework. Paper 1 covers computer systems. Paper 2 covers algorithms, programming and logic. Neither can be neglected, because each is worth exactly half the grade. The six-month structure moves in the syllabus's own order. Months 1 and 2 build computer systems from the ground up: data representation, data transmission and hardware. Months 3 and 4 finish the theory with software, the internet and its uses, and automated and emerging technologies, then add Boolean logic. Months 5 and 6 turn to the programming half: algorithm design, pseudocode, real Python, file handling, databases and SQL, and then full timed exam practice on both papers. Every class is live and small, and every programming concept is practised by actually writing code, because that is the only part of this subject that cannot be faked.

WHAT MAKES THIS COURSE DIFFERENT

  - Both papers taught in full and to equal depth, because each is worth exactly 50 percent of the grade and most students quietly neglect one of them
  - Taught to the current syllabus (0478 version 5, for examination in 2026, 2027 and 2028), so nothing covered is out of date
  - Real programming, not just pseudocode on paper: students write and run genuine Python, which is what makes Paper 2 straightforward instead of frightening
  - The scenario question is prepared for directly: candidates may answer in pseudocode or in Python, Visual Basic or Java, and we drill the choice your child is strongest in
  - Exam technique built in from the start: trace tables, dry runs, test data, and the difference between the marks for stating, applying and evaluating
  - Live, small batches where your child writes code during class and gets it reviewed on the spot, not recorded lectures watched alone

THE LEARNING PATH, STEP BY STEP

  Phase 1: Computer systems foundations: data representation, data transmission and hardware, built from binary upward
  Phase 2: Software, the internet and its uses, automated and emerging technologies, then Boolean logic and logic circuits
  Phase 3: Algorithm design, programming in Python, databases and SQL, then full timed practice on both papers

WHAT A STUDENT NEEDS BEFORE STARTING

  Title: What Your Child Needs to Start

WHO THIS COURSE IS FOR

  Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

  This is the complete teaching plan. Parents can ask about any week.

  PHASE 1: COMPUTER SYSTEMS FOUNDATIONS (PAPER 1 CORE, MONTHS 1-2)

  How a computer actually represents, moves and processes information, built from binary upward
  so nothing later has to be taken on faith.

    Month 1: Data Representation and Data Transmission (Weeks 1-4)
      Binary, Denary and Hexadecimal
        Topics:
          - Why computers use binary, and what a bit and a byte actually are
          - Converting between denary, binary and hexadecimal in both directions
          - Binary addition, and what overflow means and when it happens
          - Logical binary shifts and their effect on a value
          - Where hexadecimal shows up in practice: error codes, MAC addresses, colour codes
        Projects built:
          - A conversion drill sheet worked by hand, then a short Python script that checks your answers
        Practice: 20 conversion and binary-addition questions at exam difficulty, self-marked
        against worked solutions
      Representing Text, Images and Sound
        Topics:
          - Character sets: ASCII and Unicode, and why Unicode exists
          - How an image is stored: pixels, resolution, colour depth, and the file-size calculation
          - How sound is stored: sampling rate, sample resolution, and the trade-off with quality
          - Calculating file sizes, a guaranteed source of marks
          - Measurements of storage from bit up to terabyte
        Projects built:
          - Calculate the true file size of a given image and sound clip, then verify against a real file
        Practice: 12 file-size and representation calculations, shown with full working
        Assessment: A short written check on number systems and data representation, marked in
        exam style
      Compression and Data Transmission
        Topics:
          - Lossy and lossless compression, and when each is appropriate
          - How run-length encoding works, step by step
          - Data packets: structure, packet switching, and why packets can arrive out of order
          - Serial and parallel transmission, simplex, half-duplex and duplex
          - Universal Serial Bus and where it fits
        Projects built:
          - Compress a sample string by run-length encoding by hand, then reverse it
        Practice: 10 questions on compression and transmission methods, including a
        packet-structure diagram
      Error Detection and Encryption
        Topics:
          - Why errors occur in transmission, and what that means for the receiver
          - Parity checks (odd and even), checksums, echo check and check digits
          - Automatic Repeat reQuest and how acknowledgement works
          - Symmetric and asymmetric encryption explained plainly
          - Why encryption protects data but does not prevent interception
        Projects built:
          - Work a parity check and a check-digit calculation by hand, then break a deliberately
            corrupted example
        Practice: 12 error-detection questions including trace-style parity problems
        Assessment: End-of-month test on data representation and transmission, scored on the
        exam mark scheme

    Month 2: Hardware and Computer Architecture (Weeks 5-8)
      The CPU and the Fetch-Decode-Execute Cycle
        Topics:
          - The von Neumann model and the role of the processor
          - Registers, buses and their purpose in plain language
          - The fetch, decode, execute cycle in order
          - What actually affects CPU performance: clock speed, cores, cache
          - Embedded systems and where they are used
        Projects built:
          - Trace one full fetch-decode-execute cycle for a simple instruction and explain each step in
            your own words
        Practice: 10 architecture questions, including performance-comparison scenarios
      Input, Output and Sensors
        Topics:
          - Common input devices and where each is genuinely appropriate
          - Output devices, including printers and their differences

- Sensors, what each measures, and their use in monitoring and control systems
- Choosing the right device for a described scenario, which is how the exam asks it
- Justifying a choice, since the marks are for the reason, not the name

Projects built:
- Specify the input and output devices for a described real system and justify every choice

Practice: 10 scenario questions where the marks come from justification

Memory and Storage

Topics:
- RAM and ROM, and the difference that actually matters
- Virtual memory: what it is and why it is used
- Magnetic, optical and solid state storage, and how each physically works
- Comparing storage types on capacity, speed, portability and durability
- Cloud storage, with its genuine advantages and real drawbacks

Projects built:
- Build a comparison table of storage types and defend a recommendation for a given user

Practice: 10 memory and storage questions including comparison and justification

Network Hardware and the Internet Backbone

Topics:
- Network interface card, MAC address and IP address, and how they differ
- Routers and what they actually do
- IPv4 and IPv6, and why IPv6 exists
- Public and private IP addresses
- How this connects to data transmission from Month 1

Projects built:
- Map how a request travels from a home device to a web server and back, labelling each component

Practice: 10 networking questions at exam difficulty

Assessment: End-of-phase test covering all of hardware, scored on the mark scheme

---

PHASE 2: SOFTWARE, THE INTERNET, EMERGING TECHNOLOGIES AND LOGIC (MONTHS 3-4)

The rest of the Paper 1 theory, then Boolean logic, which is the bridge into the programming half of the course.

Month 3: Software and the Internet (Weeks 9-12)

System Software and the Operating System

Topics:
- System software versus application software, with clear examples
- What an operating system actually manages
- Interrupts: what triggers them and how they are handled
- The role of the bootstrap on start-up
- Utility software and what each utility is for

Projects built:
- Explain, in your own words, exactly what happens between pressing power and seeing the desktop

Practice: 10 software questions including interrupt scenarios

Programming Languages, Translators and the IDE

Topics:
- High-level and low-level languages, and the real trade-off
- Assembly language and why it still exists
- Compilers and interpreters, and the difference in how each reports errors
- What an Integrated Development Environment gives you
- Common IDE features and how they help you debug

Projects built:
- Run the same short program through an interpreter and observe how errors surface

Practice: 10 questions comparing translators, with justification marks

Assessment: A timed check on software, translators and the operating system

The Internet and Its Uses

Topics:
- The internet and the World Wide Web are not the same thing, and the exam tests this
- URLs, HTTP and HTTPS, and what the S actually secures
- Web browsers, cookies (session and persistent) and how pages are retrieved and displayed
- Digital currency and the idea of blockchain, at syllabus depth
- Why any of this matters for security

Projects built:
- Trace what happens from typing a URL to the page rendering, naming every step

Practice: 10 questions on web technologies and cookies

Cyber Security

Topics:
- The threats named in the syllabus: brute force, data interception, DDoS, hacking, malware, phishing, pharming, social engineering
- How each threat actually works, so the definitions stop blurring together
- The solutions: access levels, anti-malware, authentication, firewalls, privacy settings, proxy servers, SSL
- Matching the right solution to the right threat, which is how marks are awarded
- Two-factor and biometric authentication

Projects built:
- Build a threat-and-solution table, then answer a scenario recommending and justifying

protections
Practice: 12 threat-matching and justification questions
Assessment: End-of-month test on the internet and cyber security

Month 4: Automated Systems, Emerging Technologies and Boolean Logic (Weeks 13-16)
Automated Systems and Robotics
Topics:
- How sensors, microprocessors and actuators work together in an automated system
- Worked examples across transport, agriculture, weather and lighting
- What characterises a robot, and where robotics is genuinely used
- The real advantages and drawbacks of automation
- Answering these as scenario questions rather than definitions
Projects built:
- Design an automated system for a described situation, naming every sensor and the microprocessor logic
Practice: 8 automated-system scenarios with full explanations
Artificial Intelligence at Syllabus Depth
Topics:
- What artificial intelligence means in this syllabus, precisely
- Narrow, general and strong AI as the syllabus frames them
- The basic characteristics of expert systems and their components
- Machine learning at the level the exam requires, without overreach
- The genuine limitations, so answers stay accurate
Projects built:
- Explain how an expert system reaches a conclusion, using its rule base and inference engine
Practice: 8 questions on AI and expert systems
Logic Gates and Truth Tables
Topics:
- The six gates: NOT, AND, OR, NAND, NOR and XOR
- The symbol, the function and the truth table for each
- Building a truth table correctly and completely, row by row
- Reading a logic statement and identifying the gates involved
- Why this topic is pure marks once the method is automatic
Projects built:
- Produce complete truth tables for all six gates from memory, then verify them
Practice: 12 truth-table exercises building to three-input problems
Logic Circuits and Logic Expressions
Topics:
- Drawing a logic circuit from a written scenario
- Writing the logic expression for a given circuit
- Completing a truth table for a multi-gate circuit without losing rows
- Working backwards from a truth table to the expression
- The common slips that cost marks in this topic
Projects built:
- Take a described safety system, draw its logic circuit, write the expression and complete the truth table
Practice: 10 full logic-circuit problems at exam standard
Assessment: End-of-phase test on emerging technologies and Boolean logic

---
PHASE 3: ALGORITHMS, PROGRAMMING, DATABASES AND EXAM MASTERY (PAPER 2, MONTHS 5-6)

The programming half of the qualification, taught by actually writing code, then both papers drilled under timed conditions.

Month 5: Algorithm Design and Programming (Weeks 17-20)
Algorithm Design, Pseudocode and Flowcharts
Topics:
- The program development life cycle: analysis, design, coding, testing
- Decomposition and abstraction, and structure diagrams
- Writing pseudocode in the style the syllabus expects
- Drawing flowcharts with the correct symbols
- Converting freely between pseudocode and a flowchart
Projects built:
- Take one described problem and express its solution as a structure diagram, pseudocode and a flowchart
Practice: 10 algorithm-design tasks in both pseudocode and flowchart form
Programming Fundamentals in Python
Topics:
- Data types, variables and constants, and declaring them properly
- Input and output, and basic string handling
- Sequence, selection (IF and CASE) and iteration (FOR, WHILE, REPEAT)
- Nested statements and keeping them readable
- Totalling and counting, the two patterns that appear constantly
Projects built:
- Write a working Python program using selection and iteration to solve a real described task
Practice: 12 short programs written and run, each tested with your own test data
Arrays, Procedures, Functions and File Handling
Topics:

- One-dimensional and two-dimensional arrays, and traversing them with loops
- Standard methods: linear search, bubble sort, finding totals, maximum, minimum and average
- Procedures and functions, parameters, and why they matter
- Local and global variables
- Reading from and writing to a text file

Projects built:
- Build a program that reads records from a file into an array, searches and sorts it, and writes results back

Practice: 10 array and file-handling programs at exam difficulty

Validation, Verification, Test Data and Trace Tables

Topics:
- Validation checks: range, length, type, presence, format, check digit
- Verification, and how it differs from validation
- Choosing normal, abnormal and boundary test data deliberately
- Completing a trace table accurately, a classic source of marks
- Finding the error in someone else's code, which the exam asks directly

Projects built:
- Add full validation to an earlier program, then produce a trace table proving it works

Practice: 10 trace-table and test-data questions, plus 5 find-the-error tasks

Assessment: Timed programming assessment: write, validate and trace a full solution

Month 6: Databases, SQL and Full Exam Practice (Weeks 21-24)

Databases and SQL

Topics:
- Single-table databases: records, fields and the primary key
- Choosing appropriate data types for fields
- SQL: SELECT, FROM, WHERE, ORDER BY
- SUM and COUNT, and when each is wanted
- Reading a described scenario and writing the query it asks for

Projects built:
- Design a single-table database for a described scenario and write the queries that answer real questions about it

Practice: 12 SQL questions building from single conditions to sorted, aggregated queries

Paper 1 Technique

Topics:
- How the theory paper is structured and how the marks are distributed
- Reading the command word: state, describe, explain, compare, justify
- Why a one-word answer loses marks on an explain question
- Managing time so the longer questions are not rushed
- The topics that recur every single year

Projects built:
- A full timed Paper 1 attempt, then a question-by-question review with every miss tagged by cause

Practice: Two timed Paper 1 sets, fully reviewed

Paper 2 Technique and the Scenario Question

Topics:
- How the programming paper is structured, including the extended scenario question
- Choosing your answer language deliberately: pseudocode, Python, Visual Basic or Java
- Structuring a long program answer so partial marks are never lost
- Writing readable code by hand, under time, with comments that earn credit
- Checking a written program for the small errors that cost marks

Projects built:
- A full timed Paper 2 attempt including the extended scenario question, marked on the official scheme

Practice: Timed Paper 2 sets across algorithm, array, file and database question types

Full Mock Exams and Final Polishing

Topics:
- Full-length timed mocks on both papers under real conditions
- Turning the error log from every mock into a focused revision plan
- Locking in the recurring high-mark topics so they are automatic
- A calm, tested routine for both exam days
- An honest read on the target grade and the final push to reach it

Projects built:
- Two complete mock examinations, both papers, each fully reviewed and converted into a revision list

Practice: Repeated full mocks until pacing and accuracy are consistent

Assessment: Final full mock on both papers, scored and benchmarked against the target grade

HOW A TYPICAL WEEK IS SPLIT

Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real

learning, not attendance

WHERE THIS CAN LEAD

- A strong IGCSE Computer Science grade across both papers
- A confident move into A Level Computer Science or IB Computer Science
- Genuine Python ability, the most transferable skill in the syllabus
- A real foundation for a computer science degree later
- Computational thinking that transfers into maths, physics and engineering

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course aligned to the current IGCSE Computer Science syllabus?
A: Yes. It follows Cambridge IGCSE Computer Science 0478, version 5, for examination in 2026, 2027 and 2028. It covers both assessed components in full: Paper 1 on computer systems, and Paper 2 on algorithms, programming and logic. There is no coursework in this qualification, so the two papers are the entire grade.

Q: Which programming language do you teach for Paper 2?
A: We teach Python, because it is the clearest to read and the most widely used beyond the exam. In the extended scenario question candidates may answer in pseudocode or in Python, Visual Basic or Java, so we make sure your child is fluent in pseudocode too and confident in whichever form they choose to answer in.

Q: My child is fine at the theory but panics on the programming paper. Can this help?
A: That is the single most common pattern in this subject, and it is exactly what the course is built to fix. Paper 2 is worth half the grade and cannot be memorised. We teach programming by having your child write and run real code every week, then drill trace tables, test data and the extended scenario question until writing a solution under time feels ordinary.

Q: Do you cover Boolean logic and databases properly?
A: Yes, both in full. Boolean logic gets two dedicated weeks covering all six gates, truth tables, drawing circuits from a scenario and writing logic expressions. Databases get a full week on single-table design and SQL, including SELECT, FROM, WHERE, ORDER BY, SUM and COUNT. Both are reliable marks once the method is automatic.

Q: How long is the course and when can we join?
A: It runs six months across 24 weeks, with two live classes a week, and you can join in any month. Students already partway through their IGCSE course are placed at the right point after the free demo class, and we fill any gaps from the earlier topics as we go. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Will this prepare my child for A Level or IB Computer Science?
A: Yes. The programming, algorithm design and computer-systems foundations here are exactly what A Level Computer Science and IB Computer Science build on. A student who genuinely understands 0478, rather than having memorised it, starts the next qualification ahead instead of behind.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/igcse-computer-science-0478-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 85 OF 137: IGCSE MATHS: CORE, EXTENDED & ADDITIONAL MATHEMATICS ONLINE

==========

Course link: https://learn.modernagecoders.com/courses/igcse-mathematics-mastery
Subject area: Exam Board Mathematics
Designed for: High school
Who can join: IGCSE students, years 9-11: Core, Extended and Additional Maths
Course length: 18 months (72 weeks), joinable any month
Time needed: 2 live one-hour classes per week + weekly past-paper work
Certificate: Course completion certificate + documented paper-score progression

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999

Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online IGCSE Maths course for Cambridge Core, Extended and Additional Mathematics: non-calculator fluency and past-paper technique on real mark schemes.

In one line: Core, Extended or Additional, taught to the top grade, syllabus-exact.

ABOUT THE PROGRAMME

An 18-month live program for IGCSE mathematics, matched to the student's exact board and entry: Cambridge 0580/0980 at Core or Extended tier, Edexcel International GCSE, and Additional Mathematics 0606 for strong students. The tier decision caps or opens grades before a single answer is written, and Cambridge's updated syllabus put a non-calculator paper in front of every candidate, so this course trains bare-hands fluency deliberately, teaches the Extended reach early enough to settle, and converts understanding into marks through past-paper cycles against real mark schemes.

WHAT MAKES THIS COURSE DIFFERENT

- Syllabus-exact: the mentor confirms your school's board and codes first, and teaches to them, 0580/0980, Edexcel 4MA1, or 0606
- Non-calculator fluency trained in every class, because the newer papers examine it and calculator-raised students get caught
- Tier-honest: evidence for the Core-vs-Extended decision, and a real runway from Core-risk to Extended-comfortable
- Additional Maths 0606 taught as the serious second subject it is, the best A-Level preparation on the syllabus list
- Mark-scheme literacy: method marks, follow-through and working discipline taught explicitly
- Understanding-first teaching so mocks stop sliding as content deepens
- One dedicated mentor across the whole IGCSE run
- One full interactive hour per class, twice a week

THE LEARNING PATH, STEP BY STEP

Phase 1: The Core foundation built properly: number, algebra as a language, and calculator-free fluency as a habit
Phase 2: The Extended reach with settling time: functions, trigonometry, vectors and probability at full depth
Phase 3: Additional Maths 0606 for the strong (or Extended consolidation), then past-paper craft into the exam series

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Years 9-11 at an IGCSE school (Cambridge or Edexcel), or equivalent homeschool candidates
Math background: Any, the diagnostic finds the real base; Core-level gaps are repaired before Extended reach
Equipment: Computer with stable internet; a scientific calculator for the calculator papers
Time commitment: 2 live hours plus 1-2 practice hours weekly
English: Course taught in English
Motivation: An exam series on the calendar; we supply the method

WHO THIS COURSE IS FOR

Extended aimers: Students who need the move from Core-risk to Extended-comfortable made real
Top grade hunters: Extended students converting understanding into A*-level marks through paper craft
Add maths: Strong students taking 0606 as their A-Level insurance
Non calc worriers: Calculator-raised students facing the newer bare-hands papers
International families: Families who move countries and need maths that moves with them

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE CORE FOUNDATION, BUILT PROPERLY (MONTHS 1-6)

Everything on the Extended papers stands on Core material owned deeply: number fluency without a calculator, algebra as a language rather than ritual, and the geometry and data basics. The diagnostic finds the real starting point; this phase makes it solid.

Months 1-2: Diagnostic + Number, Bare-Handed (Weeks 1-8)

Diagnostic, Syllabus Code and Tier Fit

Topics:
- Board and syllabus code confirmed: 0580, 0980 or 4MA1
- Core versus Extended tier fit assessed honestly
- Non-calculator fluency screen
- Algebra base measured
- Number and ratio accuracy audit
- Honest starting-grade picture

Practice: A mixed non-calculator and algebra diagnostic set matched to the suspected tier.

Assessment: Baseline grade estimate with a written Core-or-Extended recommendation and a topic-priority map.

Detail: Full diagnostic: board and syllabus code confirmed, tier fit assessed honestly, non-calculator fluency and algebra base measured

Number Fluency Rebuilt

Topics:
- The four operations with fractions
- Decimal calculations without a calculator
- Percentage of, increase, decrease and reverse
- Converting between fractions, decimals and percentages
- Estimation and rounding as a check
- Non-calculator paper territory

Practice: A non-calculator number set with estimation checks on every answer.

Detail: Number fluency rebuilt: fractions, decimals, percentages and estimation as a checking habit, the non-calculator paper's exact territory

Ratio, Proportion and Standard Form

Topics:
- Sharing in a given ratio
- Direct and inverse proportion
- Rates of change in context
- Standard form and its arithmetic
- Standard form at real scale
- Unit conversions

Practice: Ratio, proportion and standard-form questions in applied contexts.

Detail: Ratio, proportion and rates of change; standard form with real scale

Indices, Surds, Bounds and Accuracy

Topics:
- Index laws with negative and fractional powers
- Surds and simplifying (Extended)
- Rationalising and surd arithmetic
- Upper and lower bounds
- Accuracy and significant figures
- Bounds applied in calculations

Practice: An indices, surds and bounds set including accuracy questions.

Detail: Indices and surds from meaning; bounds and accuracy, the quietly examined skills

Months 3-4: Algebra as a Language (Weeks 9-16)

Expressions, Expansion and Factorisation

Topics:
- Simplifying and collecting like terms
- Expanding single and double brackets
- The area model behind expansion
- Factorising into brackets
- Common-factor extraction
- Algebraic notation discipline

Practice: An expand-and-factorise set anchored by area-model diagrams.

Detail: Expressions, expansion and factorisation with the area model underneath

Linear Equations, Inequalities and Formulae

Topics:
- Solving linear equations step by step
- Equations with brackets and fractions
- Solving and representing inequalities
- Number-line representation of solutions
- Changing the subject of a formula
- Reading an equation as a sentence

Practice: Linear equation, inequality and rearrangement questions with worded contexts.

Detail: Linear equations and inequalities read as sentences; changing the subject of formulae

Straight-Line Graphs as One Story

Topics:
- Plotting straight-line graphs
- Gradient understood as a rate
- Intercepts as starting values
- Equation, table and graph as one story
- Finding the equation of a line
- Parallel and perpendicular lines

Practice: Straight-line graph questions linking equation, table and graph.

Detail: Straight-line graphs: gradient as rate, intercepts as starting values, equation, table and graph as one story

```
    Simultaneous Equations and Sequences
      Topics:
        - Simultaneous equations by elimination
        - Simultaneous equations by substitution
        - Choosing a method by structure
        - Linear and simple non-linear sequences
        - Finding and using the nth term rule
        - Pattern recognition
      Practice: A simultaneous-equations and nth-term set graded by structure.
      Detail: Simultaneous equations by structure; sequences and the nth term

  Months 5-6: Shape, Space and Data Foundations (Weeks 17-24)
    Angle Reasoning and Justification
      Topics:
        - Angles on lines and around points
        - Parallel-line angle rules
        - Interior and exterior angles of polygons
        - Basic circle angle facts
        - Stating a reason for every step
        - Multi-step angle chains
      Practice: Angle problems requiring a named reason for each step.
      Detail: Angle reasoning: parallel lines, polygons and circle basics with justification
      language
    Perimeter, Area, Volume and Pythagoras
      Topics:
        - Perimeter and area of standard shapes
        - Compound and composite shapes
        - Volume and surface area of solids
        - Pythagoras derived from a picture
        - Applying Pythagoras in context
        - Units for length, area and volume
      Practice: A mensuration set combining compound shapes and Pythagoras problems.
      Detail: Perimeter, area and volume including compound shapes; Pythagoras from a picture
    Statistics Foundations
      Topics:
        - Mean, median, mode and range
        - Averages from frequency tables
        - Bar charts, pie charts and pictograms
        - Scatter diagrams and correlation
        - Lines of best fit
        - Reading data critically
      Practice: A statistics set interpreting charts and a scatter diagram.
      Detail: Statistics foundations: averages, charts and scatter diagrams read critically
    Phase 1 Core-Tier Checkpoint
      Topics:
        - Full Core-tier paper under timing
        - Mark-scheme review of every question
        - The review ritual established
        - Method-mark awareness introduced
        - Error log started from the checkpoint
        - Timing across a full paper
      Practice: A complete Core-tier paper sat to time, then reviewed against the mark
      scheme.
      Assessment: Core-tier checkpoint paper marked to the mark scheme, establishing the
      review ritual and the error log.
      Detail: Phase checkpoint: a full Core-tier paper under timing, reviewed against the
      mark scheme, the review ritual for the rest of the course

----------------------------------------------------------------
PHASE 2: THE EXTENDED REACH, WITH SETTLING TIME (MONTHS 7-12)
----------------------------------------------------------------
The topics that separate Extended from Core, and top grades from safe passes, taught months
before the exam so they are owned, not crammed: functions, trigonometry beyond right angles,
vectors, and probability at full depth.

  Months 7-8: Quadratics and Functions (Weeks 25-32)
    Solving Quadratics Three Ways
      Topics:
        - Solving by factorising
        - Completing the square
        - The quadratic formula
        - Choosing the method by structure
        - Roots and their meaning
        - Extended-tier quadratic manipulation
      Practice: A quadratics set requiring choice of factorising, completing the square or
      the formula.
      Detail: Quadratic equations by factorising, completing the square and the formula,
      chosen by structure
    Quadratic Graphs and Linear-Quadratic Systems
```

    Topics:
      - Sketching quadratic graphs
      - Roots, intercepts and symmetry
      - Quadratics in real contexts
      - Simultaneous linear-quadratic systems
      - Solving graphically and algebraically
      - Interpreting the parabola
    Practice: Quadratic-graph questions plus simultaneous linear-quadratic systems.
    Detail: Quadratic graphs and real contexts; simultaneous linear-quadratic systems
  Functions: Composite and Inverse
    Topics:
      - Function notation and evaluation
      - Composite functions and order
      - Inverse functions and how to find them
      - Domain and range awareness
      - Common function-mark traps
      - Functions read as meaning
    Practice: A functions set focused on composite and inverse questions.
    Detail: Function notation, composite and inverse functions, read as meaning, where half
    of all function marks are lost
  Graph Transformations and Graph Literacy
    Topics:
      - Translations and reflections of graphs
      - Recognising graphs from equations
      - Cubic, reciprocal and exponential shapes
      - Reading features from a graph
      - Graph literacy the top questions assume
      - Sketching from key points
    Practice: Graph-transformation and graph-reading questions at Extended standard.
    Detail: Graph transformations and the graph-literacy the top questions assume

Months 9-10: Trigonometry and Vectors at Full Depth (Weeks 33-40)
  Trigonometry: Right Triangles to Sine and Cosine Rules
    Topics:
      - Sine, cosine and tangent ratios
      - Finding sides and angles in right triangles
      - The sine rule
      - The cosine rule
      - Area of a triangle with the sine formula
      - Bearings and applications
    Practice: A trig set moving from right triangles to sine and cosine rule problems.
    Detail: Right-triangle trig consolidated, then the sine and cosine rules with bearing
    and area applications
  Trig Graphs and Equations
    Topics:
      - Sine, cosine and tangent graphs
      - Features and periodicity of trig graphs
      - Solving simple trig equations
      - Reading solutions from a graph
      - Extended-reach trig contexts
      - Exact values where required
    Practice: Trig-graph and simple trig-equation questions at Extended reach.
    Detail: Trig graphs and solving simple trig equations (Extended reach)
  Vectors and Geometric Proof
    Topics:
      - Vector notation and column vectors
      - Adding and scaling vectors
      - Position vectors and paths
      - Geometry proofs with vectors
      - The classic Extended vector question
      - Proving collinear or parallel points
    Practice: Vector questions building to the classic Extended vector proof.
    Detail: Vectors: notation, geometry proofs and the classic Extended vector question,
    defanged
  Circle Theorems
    Topics:
      - The main circle theorems
      - Angle at the centre and at the circumference
      - Cyclic quadrilaterals and tangents
      - The alternate segment theorem
      - Proof-quality justification language
      - Multi-theorem angle chains
    Practice: Circle-theorem problems requiring a named theorem for each step.
    Detail: Circle theorems completed with proof-quality justification

Months 11-12: Probability, Data and the Extended Checkpoint (Weeks 41-48)
  Probability and Tree Diagrams
    Topics:
      - Probability of single and combined events

- Tree diagrams for dependent events
- Conditional reasoning
- With and without replacement
- Set notation and Venn diagrams
- Avoiding formula-panic

Practice: A probability set using tree diagrams and conditional reasoning.

Detail: Probability with tree diagrams and conditional reasoning; combined events without formula-panic

Cumulative Frequency and Histograms

Topics:
- Cumulative frequency curves
- Median, quartiles and interquartile range
- Histograms with unequal class widths
- Frequency density
- Box plots and comparing distributions
- Interpreting data like a statistician

Practice: A set drawing and reading cumulative frequency curves and histograms.

Detail: Cumulative frequency, histograms and comparing distributions like a statistician

Introductory Differentiation (Extended)

Topics:
- Differentiation as the gradient of a curve
- Differentiating simple polynomials
- Gradient at a point
- Turning points from the derivative
- The 0580 Extended calculus taste
- Interpreting a rate of change

Practice: Introductory differentiation questions finding gradients and turning points.

Detail: Differentiation introduced (0580 Extended's calculus taste): gradients of curves and turning points

Phase 2 Extended-Tier Checkpoint

Topics:
- Full Extended-tier past-paper cycle
- Mark-scheme review of long questions
- Method-mark and follow-through practice
- Error-log update from the cycle
- Timing across an Extended paper
- Weak-topic list refreshed

Practice: A full Extended-tier past paper sat to time and reviewed against the mark scheme.

Assessment: Extended-tier past-paper cycle marked to the mark scheme, updating the error log and weak-topic list.

Detail: Phase checkpoint: full Extended-tier past paper cycle with mark-scheme review and error-log update

---

PHASE 3: ADDITIONAL MATHS 0606 + EXAM CRAFT (MONTHS 13-18)

Strong students extend into Additional Mathematics, functions, identities, vectors and introductory calculus taught properly. All students then enter the exam machine: past-paper cycles by exact board and series, non-calculator discipline, and mark-scheme craft that peaks in the exam window.

Months 13-15: Additional Mathematics 0606 (or Extended Consolidation) (Weeks 49-60)

0606 Functions and Algebra Foundations

Topics:
- 0606 functions at depth
- Quadratics revisited with rigour
- Indices and surds
- Simultaneous equations for 0606
- The A-Level trailer begins
- Parallel Extended consolidation by error-log priority

Practice: 0606 function and algebra problems, with Extended students on a parallel consolidation set.

Detail: 0606: functions at depth, quadratics revisited with rigour, indices and surds, simultaneous equations, the A-Level trailer begins. (Parallel track: Extended students consolidate by error-log priority)

0606 Binomial, Trig Identities and Logs

Topics:
- Binomial expansion
- Trigonometric identities
- Solving trigonometric equations
- Logarithmic functions and laws
- Exponential functions
- Modelling with exponentials and logs

Practice: A 0606 set on binomial expansion, trig identities and log-exponential functions.

Detail: 0606: binomial expansion, trigonometric identities and equations, logarithmic and exponential functions

0606 Calculus, Vectors and Kinematics
Topics:
- Differentiation rules and applications
- Integration as the reverse of differentiation
- Areas and definite integrals
- Applications to rates and optimisation
- Vectors at 0606 depth
- Kinematics of motion
Practice: A 0606 calculus set with differentiation, integration and a kinematics problem.
Detail: 0606: introductory calculus, differentiation and integration with applications; vectors and kinematics

Months 16-17: Past-Paper Craft by Board and Series (Weeks 61-68)
Board-Exact Paper Cycles
Topics:
- Paper cycles from the exact board and series
- Non-calculator papers under bare-hands discipline
- Calculator papers with method-mark craft
- Tier-correct paper selection
- Presenting working for marks
- Board-specific command words
Practice: Board-exact non-calculator and calculator papers sat to time and reviewed.
Detail: Paper cycles from your exact board: non-calculator papers under bare-hands discipline, calculator papers with method-mark craft
Timing Plans and Point-Leak Habits
Topics:
- Personal timing plans from paper data
- Per-question time budgets
- Multi-step question chains rehearsed
- The plus-minus sign habit
- Units and final-answer discipline
- Stopping avoidable point leaks
Practice: Timed multi-step question sets drilling the plus-minus and units habits.
Detail: Timing plans from personal data; multi-step question chains rehearsed; the plus-minus and units habits that stop point leaks

Month 18: The Exam Series, Met Calmly (Weeks 69-72)
Final Cycles and Error-Log Closure
Topics:
- Final full paper cycles
- Documented score progression
- Error-log closure
- Re-testing previously fixed errors
- Confidence on weak topics
- Consistency across papers
Practice: Final full paper cycles with progression charted and closed errors re-tested.
Detail: Final full cycles with documented score progression; error-log closure
Peak-Week Plan and Exam-Day Routines
Topics:
- Light consolidation in the peak week
- Rest before the series
- Exam-day routines rehearsed
- Calm pacing under real conditions
- Final confidence work
- The series met as a known place
Practice: Light consolidation tasks and a rehearsed exam-day routine before the series.
Assessment: Final full-cycle rehearsal with documented score progression before the exam series.
Detail: Peak-week plan: light consolidations, rest, and exam-day routines rehearsed, the series met as a known place

PROJECTS THE STUDENT BUILDS AND KEEPS

- Personal error log and syllabus map across all 18 months
- Non-calculator fluency system, built through daily-dose drills
- Self-graded past-paper portfolio against real mark schemes
- 0606 formula-and-technique handbook (Additional Maths students)
- Full timed paper cycles with documented score progression

Total projects: 5+ exam artifacts plus the student's own handbooks

SKILLS THE STUDENT WILL HAVE AT THE END

- Calculator-free number fluency
- Algebra as a language: equations, formulae, graphs
- Extended-tier functions, trigonometry and vectors
- Probability and statistics at exam depth
- Introductory calculus (Extended and 0606)

- Mark-scheme literacy and working discipline
- Past-paper timing craft
- 0606 pre-A-Level maturity (strong students)

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes with your mentor
Past paper work: Weekly question sets from your exact board
Fluency: Non-calculator warm-up in every class
Total per week: About 4 focused hours

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week
Mentorship: One dedicated mentor across the whole IGCSE run
Recordings: Every class recorded for revision
Progress tracking: Paper-score progression shared with parents
Flexibility: Intensity adjusts around school mocks and the exam series

CERTIFICATION DETAIL

Final certificate: Course completion certificate from Modern Age Coders
Score record: Documented paper-score progression
Portfolio projects: Past-paper portfolio and personal handbooks

WHERE THIS CAN LEAD

- An IGCSE grade that opens sixth-form and A-Level doors
- A-Level Mathematics readiness, especially via 0606
- IB Diploma readiness for students switching systems
- Ready for the IB Mathematics Masterclass or competition mathematics

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which boards and syllabuses does this course cover?
A: The first thing the mentor confirms is the exact code and tier your school uses, and all past-paper work comes from that board and series.

Q: Core or Extended, how should we decide?
A: By destination, then evidence. Core caps the achievable grade, so if A-Level maths, sciences or a competitive sixth form is even possible, Extended is the default. Whether the gaps blocking Extended are repairable in the time available is a diagnostic question, and the free demo answers it with evidence you can take to school. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: What about the non-calculator papers?
A: Cambridge's updated 0580/0980 syllabus put a non-calculator paper in front of every candidate at both tiers. We train that fluency deliberately, fractions, percentages and estimation drills open every class, and verify current paper structures against your school's syllabus code and series.

Q: Should my child take Additional Maths 0606?
A: If they are comfortable in Extended and A-Level maths is likely: strongly consider it. 0606 is the best A-Level preparation on the syllabus list, functions, identities, vectors and introductory calculus taught properly. It is a genuine second subject that rewards starting early, which is exactly how this course sequences it.

Q: My child does endless past papers but the score is stuck. Will more classes help?
A: More papers will not, practice repeats what the student already does, errors included. A stuck score almost always means an understanding gap upstream, usually algebra as memorized rules. We find the gap, fix it at the root, and then past papers start converting effort into marks again.

Q: Can this course rescue a threatened Core entry?
A: Often, yes. Schools respond to evidence, and entry deadlines usually leave more room than parents fear. A focused term of understanding-first teaching plus improving test scores is the argument that works, and we are honest in both directions when Core genuinely is the kind choice.

Q: Can we try it before paying?
A: Yes. Every student starts with a free live demo class that doubles as the diagnostic, board, tier, fluency and algebra base, with no card details required.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/igcse-mathematics-mastery

For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 86 OF 137: INFORMATICS PRACTICES (IP) CLASS 11-12: CBSE PYTHON & SQL

==========

Course link: https://learn.modernagecoders.com/courses/cbse-informatics-practices-ip-class-11-12-python-pandas-sql-complete-course
Subject area: School Board Exam Preparation - Informatics Practices
Designed for: Teens
Who can join: Class 11 Beginner to Class 12 Board Exam Ready
Course length: 24 months (Class 11 + Class 12, the full 2-year board cycle), joinable any month
Time needed: 6-8 hours/week (3-4 classes + self-practice)
Certificate: Certificate from Modern Age Coders plus a documented mock-exam progression toward the board exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

CBSE Informatics Practices course for Class 11 & 12: master Python, Pandas, NumPy, Matplotlib and MySQL, with practical file and viva preparation.

In one line: 100% CBSE IP Syllabus · Pandas & Matplotlib Mastery · SQL Expert · Board Topper Strategy · Live Mentoring

ABOUT THE PROGRAMME

Informatics Practices (IP, Code 065) is the perfect subject for students interested in data science, business analytics, and technology applications. Unlike Computer Science (Code 083) which dives deep into programming logic and data structures, IP focuses on practical data handling using Python's Pandas library, stunning data visualization with Matplotlib, and powerful SQL database querying. This 2-year course covers the complete CBSE IP syllabus for Class 11 and Class 12, from the very basics of Python programming and SQL to advanced DataFrame operations, pivot tables, data visualization charts, GROUP BY/HAVING queries, table joins, computer networks, and societal impacts of technology. Our IP batch is completely separate from CS (083) and ICSE batches. Every class, assignment, test, and mock exam is specifically designed for the IP syllabus and CBSE exam pattern. We follow Preeti Arora (most popular IP textbook), Sumita Arora, and NCERT Informatics Practices textbooks chapter-by-chapter. Students get complete practical file preparation (15+ Pandas programs, 4+ Matplotlib visualizations, 15+ SQL queries), guided project work, viva preparation, and intensive board exam revision with 10+ mock papers. Our goal: every student scores 90+ in IP board exams.

WHAT MAKES THIS COURSE DIFFERENT

- Dedicated CBSE IP batch, not mixed with CS or ICSE students
- 100% CBSE Code 065 syllabus coverage, latest 2026-27 curriculum
- Chapter-wise teaching matching Preeti Arora, Sumita Arora & NCERT IP textbooks
- Hands-on Pandas practice with real datasets, not just theory
- Matplotlib visualization projects with real-world data
- Live MySQL database for SQL practice, write and execute queries in class
- Complete practical file: 15+ Pandas programs + 4+ Matplotlib charts + 15+ SQL queries
- Board exam project guidance with documentation template
- Weekly chapter tests + monthly unit tests + 10+ full mock board exams
- Previous 10 years solved papers with marking scheme analysis
- Dedicated doubt-clearing sessions before exams
- Assertion-Reason, Case Study, and MCQ-specific preparation
- Data science career orientation, understand where IP skills lead

THE LEARNING PATH, STEP BY STEP

Phase 1: Class 11, Semester 1 (Months 1-5): Computer fundamentals, Python basics (variables, operators, control flow, lists, dictionaries), SQL fundamentals
Phase 2: Class 11, Semester 2 (Months 6-10): Advanced SQL (DQL), Emerging Trends (AI, IoT, Cloud), practical file, Class 11 exam prep
Phase 3: Class 12, Semester 1 (Months 11-17): Pandas Series & DataFrame, data visualization with Matplotlib, CSV import/export
Phase 4: Class 12, Semester 2 (Months 18-24): Advanced SQL (functions, GROUP BY, joins), computer networks, societal impacts, board exam prep

WHAT A STUDENT NEEDS BEFORE STARTING

Class level: Students entering Class 11 or Class 12 with Informatics Practices (CBSE code 065); Class 10 students who have chosen IP can join early

Prior knowledge: No programming or database experience required - Class 11 starts from zero; Class 12 joiners get a bridge revision of Class 11 Python and SQL
Equipment: A laptop or desktop with a stable internet connection; we guide installation of Python, Pandas, Matplotlib, and MySQL in the first sessions
Time commitment: 6-8 hours per week including live classes and self-practice
Textbooks: Preeti Arora, Sumita Arora, or NCERT IP - classes align with whichever textbook the student's school follows

WHO THIS COURSE IS FOR

Class 11 ip students: Students starting IP in Class 11 who want clear fundamentals in Python and SQL from the first chapter
Class 12 board aspirants: Students facing boards in the 2026-27 session who need complete Code 065 coverage plus intensive exam practice
Students confused by pandas: Students who find Series, DataFrames, loc and iloc, and Boolean indexing confusing and want them taught step by step with real data
Sql strugglers: Students who can read queries but cannot yet independently write GROUP BY, HAVING, or join queries
Practical file and project help: Students who need structured help completing the Pandas, Matplotlib, and SQL practical file and board project on time
Future data analysts: Students interested in data science, analytics, or commerce-plus-technology careers after school

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: CLASS 11 SEMESTER 1, COMPUTER BASICS, PYTHON PROGRAMMING & SQL INTRODUCTION (MONTHS 1-5)

Build your foundation in computer science concepts, learn Python programming from scratch, and start with SQL database queries. By the end of this phase, you'll write Python programs confidently and run basic SQL queries on MySQL.

Month 1: Introduction to Computer Systems (Week 1-4)
Evolution of Computing & Computer Hardware
Topics:
- Evolution of computing devices, from Abacus to modern AI-powered computers
- Generations of computers: First (vacuum tubes) to Fifth (AI)
- Computer system components: Input Unit, CPU, Memory, Output Unit
- Block diagram of a computer, data flow from input to output
- CPU components: ALU (Arithmetic Logic Unit), Control Unit (CU), Registers
- Types of processors: single core, dual core, quad core, octa core
- Clock speed and its effect on processing: GHz explained
- Input devices: keyboard, mouse, scanner, webcam, microphone, touchscreen, stylus
- Biometric devices: fingerprint scanner, iris scanner, face recognition
- Output devices: monitor (CRT, LCD, LED, OLED, AMOLED), printer (inkjet, laser, 3D), speaker, projector, plotter
- Combined devices: touchscreen (both input and output)
- How a computer processes data: Input Process Output cycle
Projects built:
- Computer system diagram with all components labeled
- Input-output devices classification chart with 20+ devices
- Research project: Latest processor technologies in 2026-27
Practice: NCERT IP Chapter 1 questions, Preeti Arora exercises, 30 MCQs on computer fundamentals
Memory, Storage & Software
Topics:
- Computer memory: why it's needed, role in processing
- Primary memory (volatile): RAM, SRAM vs DRAM, speed, cost
- ROM, PROM, EPROM, EEPROM, non-volatile, stores firmware
- Cache memory: L1, L2, L3, speeds up CPU access to frequently used data
- Memory hierarchy: Registers Cache RAM SSD HDD (speed vs capacity trade-off)
- Units of memory: bit, nibble (4 bits), byte (8 bits), KB, MB, GB, TB, PB, EB
- Secondary storage: HDD (magnetic), SSD (flash), USB drive, memory card
- Optical storage: CD (700MB), DVD (4.7GB), Blu-ray (25-50GB)
- Cloud storage: Google Drive, OneDrive, iCloud, advantages and concerns
- Data deletion and recovery: when you delete, data isn't immediately gone
- Data security concerns: unauthorized access, data corruption, backup importance
- Software: definition, need for software, software vs hardware
- System software: OS, language processors, device drivers, utility programs
- Application software: general purpose (MS Office, browser) vs specific purpose (Tally, AutoCAD)
- Open source software: Linux, LibreOffice, GIMP, Python, MySQL
- Proprietary software: Windows, MS Office, Adobe Photoshop
- Operating system: functions, process management, memory management, file management, device management, user interface
- Types of user interface: CLI (Command Line) vs GUI (Graphical)
Projects built:

- Memory hierarchy pyramid diagram with specifications
- Software classification chart, 15+ examples each category
- Compare 5 open-source vs proprietary software alternatives

Practice: Preeti Arora Chapter 1 exercises, 50 MCQs, fill-in-the-blanks, true/false questions

Months 2-3: Introduction to Python Programming (Week 5-12)

Getting Started with Python, Setup & First Programs

Topics:

- Why Python for IP?, simple syntax, data analysis libraries, industry standard
- Python in the real world: Instagram, YouTube, Spotify, Netflix use Python
- Installing Python 3.12+ on Windows, downloading from python.org
- Python IDLE: Interactive mode (>>> prompt) vs Script mode (.py files)
- Difference between Interactive and Script mode, when to use which
- Setting up VS Code with Python extension (optional, IDLE is sufficient for IP)
- First program: print('Hello, World!'), both in interactive and script mode
- Saving and running .py files
- Python tokens: keywords, identifiers, literals, operators, punctuators/delimiters
- Python keywords: 35 reserved words, False, True, None, and, or, not, if, else, elif, for, while, def, return, import, etc.
- Identifiers: rules, must start with letter or underscore, case-sensitive, no keywords, no spaces
- Valid vs invalid identifiers, practice exercises

Projects built:

- 10 print() variations, different data types, multiple arguments
- Interactive mode exploration, test 20 expressions
- Valid/invalid identifier quiz, 30 examples

Practice: Preeti Arora Python introduction exercises, 20 MCQs, output prediction for print statements

Variables, Data Types & Input/Output

Topics:

- Variables: named storage locations in memory
- Variable assignment: x = 10, name = 'Riya'
- Dynamic typing: Python infers data type automatically
- Multiple assignment: a, b, c = 1, 2, 3 and a = b = c = 0
- Data types in Python:
- int, integers: 10, -5, 0, 1000000 (unlimited precision)
- float, decimal numbers: 3.14, -0.5, 2.0
- complex, complex numbers: 3+4j (not in board exams but good to know)
- str, strings: 'hello', "world", '''multi-line'''
- bool, boolean: True, False (capital T and F!)
- None, absence of value: x = None
- type() function: checking data type of any variable
- Mutable vs Immutable types: lists are mutable, strings/tuples/int/float are immutable, IMPORTANT CONCEPT
- Type conversion (casting): int(), float(), str(), bool()
- Implicit type conversion: int + float float (automatic)
- Explicit type conversion: int('25'), float('3.14'), str(100)
- Input function: input(), ALWAYS returns a string
- Converting input: int(input()), float(input())
- print() function: sep parameter, end parameter
- Formatted output: f-strings, f'Hello {name}, age {age}'
- Comments: # single line, '''multi-line docstrings'''
- Debugging basics: reading error messages, common beginner mistakes

Projects built:

- Student information collector, input name, age, class, marks, display formatted
- Simple interest calculator, P, R, T as input
- BMI calculator with formatted output
- Swap two variables, 3 methods: temp, arithmetic, Pythonic a,b = b,a

Practice: 40 output prediction MCQs, 15 type conversion exercises, Preeti Arora data handling exercises

Operators & Expressions

Topics:

- Arithmetic operators: + (add), - (subtract), * (multiply), / (true division), // (floor division), % (modulus), ** (exponent)
- / vs //, 7/2 gives 3.5 (float), 7//2 gives 3 (integer), EXAM FAVORITE
- Modulus operator: 17%5 = 2 (remainder), -17%5 = 3 (Python's floor modulus!)
- Exponent: 2**10 = 1024, 9**0.5 = 3.0 (square root)
- Relational/comparison operators: == (equals), != (not equals), <, >, <=, >=
- Comparing strings: lexicographic order based on ASCII/Unicode values
- Logical operators: and, or, not
- Truth tables for and, or, not
- Short-circuit evaluation: False and X False (X not evaluated), True or X True
- Assignment operators: =, +=, -=, *=, /=, //=, %=, **=
- Identity operators: is, is not, checks if same object in memory (not just equal)
- Membership operators: in, not in, checks presence in sequence
- 'a' in 'apple' True, 5 in [1,2,3,4,5] True
- Operator precedence (PEMDAS for Python):
- ** unary +x -x ~x * / // % + - == != < > <= >= not and or

- Associativity: most operators left-to-right, ** is right-to-left
- Expression evaluation: step-by-step breakdown practice
- Common exam traps: precedence gotchas, integer vs float division

Projects built:
- Expression evaluator worksheet, 30 complex expressions with step-by-step solutions
- Electricity bill calculator with slab rates
- Operator precedence quiz, 20 tricky questions
- Truth table generator for and, or, not combinations

Practice: 50 expression evaluation problems, 30 operator MCQs, Preeti Arora operator exercises

Control Flow, Conditional Statements & Loops

Topics:
- Program flow types: sequential, selection (conditional), iteration (loops)
- if statement: if condition: (note the colon and indentation!)
- if-else statement: two-way decision making
- if-elif-else ladder: multi-way branching, checking multiple conditions
- Nested if: if inside if, complex decision trees
- Ternary expression: result = 'Pass' if marks >= 40 else 'Fail'
- Common programs: largest of 2/3 numbers, even/odd, positive/negative/zero
- Grade calculator, leap year checker, vowel/consonant detector
- for loop: for variable in sequence:, iterating over range, string, list
- range() function: range(stop), range(start, stop), range(start, stop, step)
- range(5) 0,1,2,3,4; range(2,8) 2,3,4,5,6,7; range(10,0,-1) 10,9,...,1
- while loop: while condition:, repeat until condition becomes False
- while loop with counter, controlled repetition
- Infinite loop: while True, use with break for exit
- Loop control: break (exit loop immediately), continue (skip to next iteration)
- Nested loops: outer loop controls rows, inner loop controls columns
- Pattern printing: *, numbers, alphabets, basic patterns
- Accumulator patterns: sum of n numbers, factorial, counting
- Common programs: sum of digits, reverse number, palindrome check, Fibonacci
- Board exam focus: tracing loop output, counting iterations, break/continue effects

Projects built:
- Menu-driven calculator with if-elif-else
- Number guessing game using while loop
- Pattern printer, 10 basic star and number patterns
- Prime number checker and Fibonacci generator
- Multiplication table generator

Practice: 40 conditional logic problems, 30 loop tracing exercises, 15 pattern programs, Preeti Arora exercises

Lists in Python, IP Perspective

Topics:
- What are lists?, ordered, mutable, indexed collection of items
- List creation: [] empty list, [1,2,3], list() constructor, list('hello')
- Lists can contain mixed types: [1, 'hello', 3.14, True, None]
- Indexing: positive (0-based) and negative (-1 for last element)
- Slicing: list[start:stop:step], extracting sublists
- List is MUTABLE, elements can be changed: list[0] = 'new_value'
- List operations: + (concatenation), * (repetition), in (membership)
- List traversal: for item in list:, for i in range(len(list)):
- List methods, MUST KNOW for exams:
- append(x), add x to end
- insert(i, x), add x at position i
- extend(iterable), add all items from another list
- remove(x), remove first occurrence of x (ValueError if not found)
- pop(), remove and return last item, pop(i), remove at position i
- clear(), remove all items
- index(x), return position of first occurrence
- count(x), count occurrences of x
- sort(), sort in ascending order, sort(reverse=True), descending
- reverse(), reverse the list in place
- copy(), create shallow copy
- Built-in functions: len(), min(), max(), sum(), sorted(), list()
- del statement: del list[i], del list[start:stop]
- List comprehension basics: [x*2 for x in range(5)] [0, 2, 4, 6, 8]
- Common programs: linear search, finding max/min without built-in, removing duplicates
- Board exam: output prediction with list operations and methods

Projects built:
- Student marks manager, add, remove, sort, find topper, average
- Shopping list application with all list operations
- List-based number analyzer, even/odd count, sum, average, max, min
- Duplicate remover, remove duplicate elements from a list

Practice: Preeti Arora list chapter exercises, 50 list MCQs, 20 list programs

Dictionaries in Python, IP Perspective

Topics:
- What are dictionaries?, unordered collection of key-value pairs
- Dictionary creation: {} empty, {'name': 'Riya', 'age': 16}, dict()
- Keys must be immutable (string, int, tuple), lists cannot be keys
- Values can be any type, including lists, other dictionaries

- Accessing values: d['key'] (raises KeyError if not found), d.get('key', default)
- Adding/updating: d['new_key'] = value (adds if new, updates if exists)
- Removing: del d['key'], d.pop('key'), d.popitem() (removes last), d.clear()
- Dictionary methods:
- dict(), constructor
- len(d), number of key-value pairs
- d.keys(), view of all keys
- d.values(), view of all values
- d.items(), view of all (key, value) tuples
- d.get(key, default), safe access with default
- d.update(other_dict), merge/update from another dictionary
- d.copy(), shallow copy
- d.clear(), remove all items
- Traversing dictionaries: for key in d:, for key, value in d.items():
- Nested dictionaries: student records with multiple attributes
- in operator with dictionaries: checks keys (not values)
- Dictionary comprehension basics: {x: x**2 for x in range(5)}
- Common programs: character frequency, word counter, student database
- Board exam focus: output prediction with dictionary operations

Projects built:

- Phone book application, add, search, update, delete contacts
- Character frequency counter for a string
- Student report card using nested dictionaries
- Word frequency analyzer, count each word in a paragraph

Practice: Preeti Arora dictionary exercises, 40 MCQs, 15 dictionary programs

Months 4-5: SQL Fundamentals, Database Concepts & Basic Queries (Week 13-20)

Database Concepts & Relational Model

Topics:

- What is data?, raw facts and figures vs information (processed data)
- What is a database?, organized collection of related data
- Need for databases: data redundancy reduction, data integrity, data security
- File system vs DBMS: why databases replaced flat files
- DBMS examples: MySQL, PostgreSQL, Oracle, MongoDB, SQL Server
- Relational Database Model: data stored in tables (relations)
- RDBMS terminology:
- Relation = Table (collection of rows and columns)
- Attribute = Column = Field (a property of the entity)
- Tuple = Row = Record (a single entry in the table)
- Domain = Set of valid values for an attribute
- Degree = Number of attributes (columns) in a relation
- Cardinality = Number of tuples (rows) in a relation
- Keys, identifying rows uniquely:
- Candidate Key: attribute(s) that can uniquely identify each tuple, there can be multiple
- Primary Key: the chosen candidate key, only ONE per table, no NULL values
- Alternate Key: candidate keys that were not chosen as primary key
- Example: Student table, RollNo and AdmissionNo are both candidate keys; if RollNo is primary key, AdmissionNo is alternate key
- Referential integrity concept basics

Projects built:

- Design database schema for: school, library, hospital, e-commerce, identify all keys
- Key identification worksheet, 15 scenarios
- Draw 5 sample tables with proper attributes, data types, and keys

Practice: Preeti Arora database concepts chapter, 40 MCQs on RDBMS terminology, key identification exercises

SQL Introduction, DDL & DML Commands

Topics:

- What is SQL?, Structured Query Language, communicating with databases
- SQL is NOT a programming language, it's a query language
- MySQL: free, open-source RDBMS, used in CBSE IP
- Installing MySQL on Windows, MySQL Community Server + MySQL Workbench
- Accessing MySQL: command line client, MySQL Workbench GUI
- SQL command categories: DDL, DML, DQL
- SQL data types:
- CHAR(n), fixed-length string (padded with spaces), VARCHAR(n), variable-length
- INT, integer, SMALLINT, smaller range, BIGINT, larger range
- FLOAT, decimal with approximate precision, DOUBLE, higher precision
- DATE, format 'YYYY-MM-DD', DATETIME, 'YYYY-MM-DD HH:MM:SS'
- DDL (Data Definition Language), structure commands:
- CREATE DATABASE db_name;, create new database
- USE db_name;, select active database
- SHOW DATABASES;, list all databases
- SHOW TABLES;, list all tables in current database
- CREATE TABLE table_name (col1 datatype constraint, col2 datatype, ...);
- Constraints: NOT NULL, UNIQUE, PRIMARY KEY, DEFAULT
- DESCRIBE table_name; / DESC table_name;, show table structure
- ALTER TABLE: ADD column, MODIFY column, DROP column
- DROP TABLE table_name;, delete entire table (structure + data)
- DROP DATABASE db_name;, delete entire database

- DML (Data Manipulation Language), data commands:
- INSERT INTO table VALUES (val1, val2, ...);, insert all columns
- INSERT INTO table (col1, col2) VALUES (val1, val2);, insert specific columns
- INSERT multiple rows in one command
- UPDATE table SET col1=val1 WHERE condition;, modify data
- DELETE FROM table WHERE condition;, remove specific rows
- WARNING: UPDATE/DELETE without WHERE affects ALL rows!

Projects built:

- Create School database with Student, Teacher, Subject tables
- Insert 15+ records in each table
- Practice ALTER TABLE, add, modify, drop columns
- UPDATE and DELETE with various conditions

Practice: 30 DDL/DML queries, Preeti Arora SQL chapter exercises, create-insert-update-delete practice

SQL DQL, SELECT Queries with WHERE, Operators & Sorting

Topics:

- SELECT, the most important SQL command for IP exams
- SELECT * FROM table;, retrieve all columns, all rows
- SELECT col1, col2 FROM table;, specific columns only
- SELECT DISTINCT col FROM table;, remove duplicate values
- Column aliases: SELECT col AS alias_name
- WHERE clause, filtering rows based on conditions:
- Comparison operators: = <> (or !=) < > <= >=
- Logical operators: AND, OR, NOT
- Operator precedence: NOT > AND > OR (use parentheses for clarity)
- BETWEEN operator: WHERE marks BETWEEN 60 AND 90 (inclusive!)
- NOT BETWEEN: marks NOT BETWEEN 60 AND 90
- IN operator: WHERE city IN ('Delhi', 'Mumbai', 'Kolkata')
- NOT IN: WHERE city NOT IN ('Delhi', 'Mumbai')
- LIKE operator, pattern matching:
- %, matches zero or more characters: 'A%' (starts with A), '%ing' (ends with ing)
- _, matches exactly one character: '_a%' (second letter is a), '___' (exactly 3 chars)
- LIKE patterns: '%an%' (contains 'an'), 'S____' (S followed by exactly 4 chars)
- IS NULL: WHERE commission IS NULL, checking for missing values
- IS NOT NULL: WHERE phone IS NOT NULL
- NULL arithmetic: anything + NULL = NULL, NULL = NULL is UNKNOWN (use IS NULL!)
- ORDER BY clause, sorting results:
- ORDER BY col ASC, ascending (default, A-Z, 0-9)
- ORDER BY col DESC, descending (Z-A, 9-0)
- ORDER BY col1 ASC, col2 DESC, multi-column sorting
- NULL values in ORDER BY, appear first in ASC, last in DESC

Projects built:

- 40-query challenge on Student table, various SELECT queries
- Employee database queries, salary filtering, department queries
- Pattern matching challenge, 15 LIKE queries
- Multi-condition queries combining AND, OR, NOT, BETWEEN, IN, LIKE

Practice: 60 SELECT queries, Preeti Arora DQL exercises, 30 output prediction MCQs for SQL

SQL Aggregate Functions & Class 11 SQL Revision

Topics:

- Aggregate functions, operating on groups of rows:
- COUNT(*), count all rows (including NULLs)
- COUNT(column), count non-NULL values in column
- COUNT(DISTINCT column), count unique non-NULL values
- SUM(column), total of numeric column (ignores NULLs)
- AVG(column), average of numeric column (ignores NULLs)
- MAX(column), maximum value (works with numbers, strings, dates)
- MIN(column), minimum value
- Aggregate functions IGNORE NULL values (except COUNT(*))
- Using aggregate functions with WHERE: SELECT AVG(marks) FROM student WHERE class=12;
- Aliases with aggregates: SELECT COUNT(*) AS total_students FROM student;
- GROUP BY clause: grouping rows for aggregate calculations
- SELECT city, COUNT(*) FROM student GROUP BY city;
- GROUP BY with multiple columns
- HAVING clause: filtering groups after GROUP BY
- WHERE vs HAVING, WHERE filters rows BEFORE grouping, HAVING filters AFTER grouping
- Complete query with all clauses: SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ...
- Class 11 SQL complete revision, all DDL, DML, DQL commands
- SQL query writing strategy for exams
- Common mistakes in SQL and how to avoid them

Projects built:

- Sales database analysis, total, average, max, min by category, region
- Student performance report, class-wise, section-wise statistics
- SQL revision worksheet, 80 queries covering all topics
- SQL error correction, find and fix errors in 20 queries

Practice: 50 aggregate/GROUP BY/HAVING queries, Preeti Arora exercises, previous year SQL questions

---
PHASE 2: CLASS 11 SEMESTER 2, EMERGING TRENDS, PRACTICALS & EXAM PREP (MONTHS 6-10)

Complete remaining Class 11 topics: Emerging Trends in Technology. Prepare practical file
with Python programs and SQL queries. Get ready for Class 11 final examination.

  Months 6-7: Emerging Trends & Revision (Week 21-28)
    Introduction to Emerging Trends (CBSE IP Unit 4-5 marks)
      Topics:
        - Artificial Intelligence (AI): machines that simulate human intelligence
        - Types of AI: Narrow AI (Siri, Alexa), General AI (future), Super AI (theoretical)
        - Applications of AI: healthcare, education, autonomous vehicles, chatbots (ChatGPT)
        - Machine Learning (ML): subset of AI, computers learn from data without explicit programming
        - Types of ML: supervised, unsupervised, reinforcement learning
        - Natural Language Processing (NLP): AI understanding human language, Google Translate, voice
          assistants
        - Augmented Reality (AR): digital content overlaid on real world, Pokémon Go, Instagram filters
        - Virtual Reality (VR): fully immersive digital environment, Oculus, VR gaming
        - Mixed Reality (MR): combination of AR and VR
        - Robotics: programmable machines performing tasks, manufacturing, surgery, exploration
        - Types of robots: industrial, service, humanoid, drone
        - Internet of Things (IoT): everyday objects connected to internet, smart home, wearables
        - IoT architecture: sensors connectivity data processing user interface
        - IoT examples: smart thermostat, fitness tracker, smart fridge, traffic management
        - Smart Cities: technology-driven urban management, traffic, waste, energy, safety
        - Big Data: extremely large datasets requiring special tools to process
        - Big Data characteristics: 5 V's, Volume, Velocity, Variety, Veracity, Value
        - Cloud Computing: accessing computing resources over internet instead of local machines
        - Cloud service models: SaaS (Gmail), PaaS (Google App Engine), IaaS (AWS EC2)
        - Cloud deployment: public, private, hybrid, community cloud
        - Grid Computing: distributed computing, combining many computers for complex tasks
        - Blockchain technology: decentralized, immutable digital ledger, cryptocurrency, supply chain
        - How blockchain works: blocks, hashing, consensus mechanism
      Projects built:
        - Emerging technology presentation, choose one topic and present for 10 minutes
        - IoT application design, design a smart home system on paper
        - AI in daily life report, identify 15 AI applications you use daily
        - Cloud vs traditional computing comparison chart
      Practice: Preeti Arora emerging trends chapter, NCERT IP textbook, 50 MCQs on emerging
      trends
    Practical File, Project & Class 11 Exam Preparation
      Topics:
        - Practical file requirements for Class 11 IP:
        - Minimum 14 Python programs covering all topics
        - Minimum 14 SQL queries covering DDL, DML, DQL
        - Python programs for practical file:
        - P1: Input two numbers, perform all arithmetic operations
        - P2: Find largest of three numbers
        - P3: Check if number is even/odd, positive/negative
        - P4: Grade calculator using if-elif-else
        - P5: Print multiplication table using for loop
        - P6: Print patterns using nested loops (3 patterns)
        - P7: Check prime number, palindrome, Armstrong number
        - P8: Calculate factorial using while loop
        - P9: List operations, create, append, insert, remove, sort, search
        - P10: Find max, min, sum, average from a list
        - P11: Count vowels, consonants in a string
        - P12: Dictionary operations, create, add, update, delete, traverse
        - P13: Character/word frequency counter using dictionary
        - P14: Menu-driven program combining multiple concepts
        - SQL queries for practical file:
        - Q1-Q3: CREATE TABLE, INSERT INTO, DDL and DML
        - Q4-Q6: SELECT with WHERE, BETWEEN, IN, LIKE
        - Q7-Q9: UPDATE, DELETE with conditions
        - Q10-Q11: Aggregate functions, COUNT, SUM, AVG, MAX, MIN
        - Q12-Q14: GROUP BY, HAVING, ORDER BY
        - Class 11 final exam revision, unit-wise important questions
        - Sample paper solving, 3 complete papers with solutions
        - Viva voce preparation, 30 Q&A
      Deliverables:
        - Complete practical file: 14+ Python programs + 14+ SQL queries
        - 3 solved sample papers
        - Viva preparation notes
        - Ready for Class 11 final exam

PHASE 3: CLASS 12 SEMESTER 1, PANDAS, DATA VISUALIZATION & ADVANCED DATA HANDLING (MONTHS 11-17)

This is the CORE of Class 12 IP, mastering Pandas for data handling and Matplotlib for data

visualization. Unit 1 carries 25 marks in the board exam. Every concept here is board exam critical.

- Months 11-12: Python Pandas, Series (Week 41-48)
  - Introduction to Python Libraries & Pandas Series Basics
    - Topics:
      - What are Python libraries?, pre-written code modules for specific tasks
      - NumPy: Numerical Python, arrays, mathematical operations (foundation for Pandas)
      - Pandas: Python Data Analysis Library, data manipulation and analysis
      - Matplotlib: plotting and visualization library
      - Installing libraries: pip install pandas numpy matplotlib
      - Importing: import pandas as pd, import numpy as np
      - Pandas data structures: Series (1D) and DataFrame (2D)
      - Series: one-dimensional labeled array, like a column in a table
      - Creating Series from a list: pd.Series([10, 20, 30, 40])
      - Default index: 0, 1, 2, 3... (RangeIndex)
      - Custom index: pd.Series([10, 20, 30], index=['a', 'b', 'c'])
      - Creating Series from a dictionary: pd.Series({'Maths': 90, 'Science': 85})
      - Creating Series from a scalar value: pd.Series(5, index=['a', 'b', 'c']) all values are 5
      - Creating Series from NumPy ndarray: pd.Series(np.array([1, 2, 3]))
      - Series attributes: dtype, shape, size, nbytes, ndim, index, values, name
      - Checking data type: s.dtype returns data type of elements
      - Series is like a fixed-size dictionary, index acts as keys
    - Projects built:
      - Create 10 different Series, from list, dict, scalar, ndarray
      - Student marks Series with subject names as index
      - Temperature data Series for a week with day names as index
    - Practice: Preeti Arora Pandas chapter exercises, 20 Series creation MCQs
  - Pandas Series, Operations, Indexing & Methods
    - Topics:
      - Accessing elements: s[0] (positional), s['a'] (label-based)
      - Slicing Series: s[1:4] (positional), s['b':'d'] (label, inclusive on both ends!)
      - s.head(n), first n elements (default 5), s.tail(n), last n elements
      - Mathematical operations on Series, element-wise:
      - s + 10 adds 10 to each element
      - s1 + s2 adds corresponding elements (NaN for unmatched indices)
      - s * 2, s / 3, s ** 2, all element-wise operations
      - Comparison operations: s > 50, s == 100 returns Boolean Series
      - Boolean indexing / filtering: s[s > 50] elements greater than 50
      - NaN (Not a Number): result of mismatched index operations
      - Handling NaN: s.isnull(), s.notnull(), s.dropna(), s.fillna(value)
      - Series methods for board exams:
      - s.sum(), s.mean(), s.median(), s.mode(), s.std(), s.var()
      - s.min(), s.max(), s.count() (non-NaN count)
      - s.describe(), summary statistics (count, mean, std, min, 25%, 50%, 75%, max)
      - s.value_counts(), frequency count of each unique value
      - s.unique(), array of unique values, s.nunique(), count of unique values
      - s.sort_values(ascending=True/False), sort by values
      - s.sort_index(), sort by index labels
      - Vectorized string operations: s.str.upper(), s.str.lower(), s.str.len()
      - Board exam focus: output prediction with Series operations, NaN handling
    - Projects built:
      - Sales data analysis using Series, total, average, max, min, filtering
      - Temperature analysis, find hottest/coldest day, above average days
      - Student marks comparison, Series arithmetic with NaN handling
      - Statistical summary report using describe()
    - Practice: 40 Series operation MCQs, 20 output prediction problems, Preeti Arora exercises
  - Pandas DataFrame, Creation & Basic Operations
    - Topics:
      - DataFrame: 2-dimensional labeled data structure, like a spreadsheet/SQL table
      - DataFrame = collection of Series sharing the same index
      - Creating DataFrame from dictionary of lists:
      - pd.DataFrame({'Name': ['A', 'B'], 'Marks': [90, 85]})
      - Creating DataFrame from dictionary of Series
      - Creating DataFrame from list of dictionaries
      - Creating DataFrame from 2D list/array with columns parameter
      - Creating DataFrame from CSV file: pd.read_csv('file.csv'), VERY IMPORTANT
      - Custom index: pd.DataFrame(data, index=['row1', 'row2', ...])
      - DataFrame attributes: shape, size, ndim, dtypes, columns, index, values, T (transpose)
      - Selecting columns: df['column'], df[['col1', 'col2']] (single vs multiple)
      - Adding a new column: df['new_col'] = values
      - Deleting a column: del df['col'], df.drop('col', axis=1), df.pop('col')
      - Renaming columns: df.rename(columns={'old': 'new'})
      - Selecting rows by label: df.loc['row_label'], df.loc['r1':'r3']
      - Selecting rows by position: df.iloc[0], df.iloc[0:3]
      - loc vs iloc, label-based vs integer position-based, EXAM CRITICAL
      - Selecting specific cell: df.loc['row', 'col'], df.iloc[row_idx, col_idx]
      - df.at['row', 'col'], fast scalar access by label

- df.iat[row_idx, col_idx], fast scalar access by position
- df.head(n), df.tail(n), view first/last n rows

Projects built:

- Student database DataFrame, create from dictionary, CSV, and list of dicts
- Employee records with adding, renaming, and deleting columns
- Accessing data challenge, loc vs iloc practice with 20 examples
- CSV data loader, read real-world dataset and explore

Practice: Preeti Arora DataFrame chapter exercises, 30 creation MCQs, 20 loc/iloc problems

Pandas DataFrame, Advanced Operations & Data Manipulation

Topics:

- Adding a new row: df.loc['new_index'] = [values], pd.concat()
- Deleting rows: df.drop('row_label'), df.drop([indices])
- Boolean indexing with DataFrame: df[df['Marks'] > 80]
- Multiple conditions: df[(df['Marks'] > 80) & (df['City'] == 'Delhi')]
- Note: use & instead of 'and', | instead of 'or' for DataFrame filtering
- Sorting: df.sort_values('column'), df.sort_values(['col1', 'col2'])
- df.sort_values('column', ascending=False), descending sort
- df.sort_index(), sort by index
- DataFrame iteration: df.iterrows(), iterate row by row
- Aggregate operations: df['col'].sum(), .mean(), .max(), .min(), .count()
- df.describe(), statistical summary for all numeric columns
- df.info(), overview of DataFrame structure
- df.isnull(), check for NaN values, df.notnull()
- df.dropna(), remove rows with NaN, df.fillna(value), fill NaN
- df.drop_duplicates(), remove duplicate rows
- Exporting DataFrame to CSV: df.to_csv('output.csv', index=False)
- DataFrame arithmetic: df1 + df2, df * scalar, element-wise operations
- Pivot tables basics: df.pivot_table(values, index, aggfunc)
- Groupby basics: df.groupby('column').mean()
- Board exam focus: complete DataFrame operation chains, output prediction

Projects built:

- Complete student report card system using DataFrame
- Sales data analysis, filtering, sorting, grouping, statistics
- CSV data processing pipeline, read clean analyze export
- Covid/weather/population dataset analysis (real-world data)

Practice: 50 DataFrame operation problems, 30 output prediction MCQs, Preeti Arora advanced exercises

Months 13-14: Data Visualization with Matplotlib (Week 49-56)

Introduction to Matplotlib, Line Charts & Bar Graphs

Topics:

- What is data visualization?, representing data graphically for insights
- Why visualize?, patterns, trends, comparisons are easier to see in charts
- Matplotlib library: most popular Python plotting library
- Importing: import matplotlib.pyplot as plt
- Basic plot workflow: create figure plot data customize show/save
- Line Chart / Line Plot: plt.plot(x, y)
- plt.plot() with just y values, x defaults to 0, 1, 2, ...
- Multiple lines on same plot: call plt.plot() multiple times
- Line customization:
- color: plt.plot(x, y, color='red') or color='r' or color='#FF0000'
- Line style: linestyle='-' (solid), '--' (dashed), '-.' (dash-dot), ':' (dotted)
- Line width: linewidth=2 or lw=2
- Markers: marker='o' (circle), 's' (square), '^' (triangle), 'D' (diamond), '*' (star)
- Marker size: markersize=10 or ms=10
- Combined format string: plt.plot(x, y, 'r--o') red dashed line with circles
- Chart elements, MUST add for full marks:
- plt.title('Chart Title'), chart title
- plt.xlabel('X-Axis Label'), plt.ylabel('Y-Axis Label'), axis labels
- plt.legend(['Line 1', 'Line 2']) or label='name' in plot + plt.legend()
- plt.grid(True), add grid lines
- plt.xticks(rotation=45), rotate x-axis labels
- plt.show(), display the chart
- plt.savefig('chart.png'), save chart as image
- Bar Graph: plt.bar(x, height), vertical bars
- Horizontal bar: plt.barh(x, width)
- Bar customization: color, width, edgecolor, align
- Multiple bar groups: side-by-side bars using width offset
- Stacked bars: plt.bar(x, y1); plt.bar(x, y2, bottom=y1)

Projects built:

- Monthly temperature line chart, 12 months with customization
- Student marks comparison, line chart for 5 subjects across 3 students
- Product sales bar graph, quarterly sales for 4 products
- Population comparison, horizontal bar chart for 10 cities

Practice: 15 line chart programs, 10 bar chart programs, Preeti Arora visualization exercises

Histograms, Advanced Customization & Combining Charts

Topics:

- Histogram: plt.histogram(data, bins), frequency distribution chart
- Histogram vs Bar graph: histogram for continuous data, bar for categorical
- Histogram parameters: bins (number of intervals), range, edgecolor, color, alpha
- bins parameter: bins=10 (10 equal intervals), bins=[0,20,40,60,80,100] (custom)
- alpha parameter: transparency, alpha=0.7 (0=invisible, 1=opaque)
- Using histograms: exam marks distribution, age distribution, salary distribution
- Advanced customization for all chart types:
- Figure size: plt.figure(figsize=(10, 6))
- Font sizes: fontsize parameter in title, xlabel, ylabel, legend
- Font weight: fontweight='bold'
- Color options: named colors, hex codes, RGB tuples
- Grid customization: plt.grid(axis='y', linestyle='--', alpha=0.7)
- Setting axis limits: plt.xlim(0, 100), plt.ylim(0, 50)
- Annotations: plt.annotate('text', xy=(x,y), xytext=(tx,ty), arrowprops=dict(arrowstyle='->'))
- Adding text: plt.text(x, y, 'text')
- Subplots: plt.subplot(rows, cols, position), multiple charts in one figure
- plt.tight_layout(), avoid overlapping in subplots
- Complete chart for board exam: title + labels + legend + grid + colors + markers
- Board exam pattern: write code to create a specific chart from given data
- Chart type selection: when to use line vs bar vs histogram
- Practical exam: creating charts from Pandas DataFrame data

Projects built:
- Student marks histogram, frequency distribution across score ranges
- Dashboard with subplots, line, bar, and histogram in one figure
- Complete data story: Pandas analysis Matplotlib visualization
- 4 Matplotlib programs for practical file, fully customized charts

Practice: 10 histogram programs, 5 subplot programs, complete visualization exercises, board exam chart questions

Months 15-16: Pandas + Matplotlib Integration & CSV Data Workflow (Week 57-64)

Pandas DataFrame Visualization & CSV Import/Export

Topics:
- Plotting directly from DataFrame: df.plot(), integrates Pandas with Matplotlib
- df.plot(kind='line'), line chart from DataFrame columns
- df.plot(kind='bar'), bar chart, df.plot(kind='barh'), horizontal bar
- df.plot(kind='hist'), histogram from DataFrame data
- df['column'].plot(), plot a single column as Series
- Complete CSV workflow:
- Step 1: pd.read_csv('data.csv'), import data into DataFrame
- read_csv parameters: sep, header, index_col, usecols, nrows, na_values
- Step 2: df.head(), df.info(), df.describe(), explore the data
- Step 3: df[condition], filter and clean data
- Step 4: df.groupby().agg(), aggregate and summarize
- Step 5: df.plot() or plt.plot(), visualize insights
- Step 6: df.to_csv('output.csv', index=False), export results
- Real-world data analysis workflow using CSV files
- Handling large CSV files: chunking, selecting columns
- Common Pandas + Matplotlib patterns for board exams

Projects built:
- Weather data analysis: CSV import filtering visualization
- Student performance dashboard: CSV DataFrame charts
- Sales trend analysis: monthly/quarterly charts from CSV data
- Complete data analysis project for practical file

Practice: 15 integrated Pandas-Matplotlib programs, CSV workflow exercises

Pandas & Matplotlib Revision + Practical Programs

Topics:
- Complete Pandas revision: Series creation, operations, methods
- DataFrame revision: creation, selection, filtering, sorting, groupby
- CSV read/write revision
- Matplotlib revision: line, bar, histogram with full customization
- 15 Pandas programs for practical file:
- P1: Create Series from list, dict, ndarray, display attributes
- P2: Series arithmetic operations and NaN handling
- P3: Series statistical functions, mean, median, mode, std
- P4: Create DataFrame from dict, list of dicts, CSV
- P5: DataFrame column operations, add, delete, rename
- P6: DataFrame row selection with loc and iloc
- P7: Boolean indexing, filter DataFrame rows by condition
- P8: Sorting DataFrame by column values
- P9: DataFrame aggregate functions, sum, mean, count
- P10: GroupBy operations, group by category and aggregate
- P11: Handling missing data, isnull, dropna, fillna
- P12: Import CSV into DataFrame and display statistics
- P13: Export DataFrame to CSV file
- P14: DataFrame describe() and info(), data exploration
- P15: Integrated program, CSV filter aggregate export
- 4+ Matplotlib programs for practical file:
- M1: Line chart with title, labels, legend, grid, colors
- M2: Bar graph (vertical/horizontal) with customization

- M3: Histogram with bins, color, edgecolor
- M4: Multiple charts comparison using subplots

Deliverables:
- 15 Pandas programs written and tested
- 4+ Matplotlib programs with saved chart images
- Complete understanding of Pandas-Matplotlib for board exam

PHASE 4: CLASS 12 SEMESTER 2, ADVANCED SQL, NETWORKS, SOCIETY & BOARD EXAM MASTERY (MONTHS 18-24)

Complete remaining Class 12 syllabus: advanced SQL with functions, joins, GROUP BY/HAVING, computer networks, and societal impacts. Then intensive board exam preparation with mock tests.

Months 18-19: Advanced SQL, Functions, GROUP BY, HAVING & Joins (Week 69-76)

SQL Single Row Functions, Math, String & Date Functions

Topics:
- Revision of Class 11 SQL: CREATE, INSERT, SELECT, WHERE, operators
- SQL Functions: two categories, single row functions & aggregate functions
- Single row functions work on each row individually
- Mathematical functions:
- POWER(base, exponent), POWER(2, 3) = 8
- ROUND(number, decimals), ROUND(15.678, 1) = 15.7, ROUND(15.678, 0) = 16
- ROUND(1256, -2) = 1300, rounding to nearest hundred
- MOD(dividend, divisor), MOD(17, 5) = 2 (remainder)
- TRUNCATE(number, decimals), TRUNCATE(15.678, 1) = 15.6 (no rounding!)
- ABS(number), absolute value
- CEIL(number) / CEILING(number), smallest integer >= number
- FLOOR(number), largest integer <= number
- String/Text functions:
- UCASE(str) / UPPER(str), convert to uppercase
- LCASE(str) / LOWER(str), convert to lowercase
- MID(str, start, length) / SUBSTRING(str, start, length) / SUBSTR(str, start, length)
- Note: MySQL string positions start from 1 (not 0 like Python!)
- LENGTH(str), number of characters including spaces
- LEFT(str, n), first n characters, RIGHT(str, n), last n characters
- INSTR(str, substr), position of first occurrence of substr (returns 0 if not found)
- LTRIM(str), remove leading spaces, RTRIM(str), remove trailing spaces
- TRIM(str), remove both leading and trailing spaces
- CONCAT(str1, str2), join strings together
- REPLACE(str, old, new), replace occurrences
- Date functions:
- NOW(), current date and time
- DATE(datetime), extract date portion
- MONTH(date), month number (1-12)
- MONTHNAME(date), month name ('January', 'February', ...)
- YEAR(date), year number
- DAY(date), day of month (1-31)
- DAYNAME(date), day name ('Monday', 'Tuesday', ...)
- DAYOFWEEK(date), 1=Sunday, 2=Monday, ... 7=Saturday
- Using functions in SELECT and WHERE clauses
- Nested functions: UPPER(LEFT(name, 3)), combine functions

Projects built:
- SQL function explorer, test all functions on sample data
- Employee name formatting, UPPER, LOWER, CONCAT, SUBSTRING
- Date-based queries, find employees hired in specific month/year
- Mathematical query set, ROUND, MOD, POWER calculations on salary data

Practice: 40 function-based SQL queries, output prediction MCQs, Preeti Arora SQL function exercises

SQL Aggregate Functions, GROUP BY & HAVING, Deep Dive

Topics:
- Aggregate functions (revision and deep dive):
- COUNT(*), total rows, COUNT(column), non-NULL count
- SUM(column), total, AVG(column), average
- MAX(column), maximum, MIN(column), minimum
- COUNT(DISTINCT column), unique value count
- All aggregates ignore NULL except COUNT(*)
- GROUP BY clause, advanced usage:
- Grouping by single column: SELECT city, COUNT(*) FROM student GROUP BY city;
- Grouping by multiple columns: GROUP BY class, section
- GROUP BY with all aggregate functions
- Rules: columns in SELECT must be in GROUP BY or inside aggregate functions
- HAVING clause, filtering groups:
- HAVING COUNT(*) > 5, show only groups with more than 5 members
- HAVING AVG(marks) >= 80, show groups with average marks >= 80
- WHERE vs HAVING, the definitive comparison (EXAM CRITICAL):
- WHERE: filters ROWS before grouping, cannot use aggregate functions
- HAVING: filters GROUPS after grouping, CAN use aggregate functions
- WHERE is for conditions on individual rows

- HAVING is for conditions on grouped results
- Complete query with all clauses:
- SELECT col, AGG(col) FROM table WHERE condition GROUP BY col HAVING agg_condition ORDER BY col;
- Execution order: FROM WHERE GROUP BY HAVING SELECT ORDER BY
- Complex queries combining WHERE + GROUP BY + HAVING + ORDER BY
- Board exam: write query for given output, find errors, output prediction

Projects built:

- Sales analysis: total/average sales by region, category, with filtering
- Student database: section-wise, class-wise statistics with HAVING
- Employee analysis: department-wise salary stats, count, conditions
- 30-query challenge: all GROUP BY + HAVING combinations

Practice: 50 GROUP BY/HAVING queries, WHERE vs HAVING comparison exercises, board exam SQL from last 10 years

SQL Joins, Equi-Join on Two Tables

Topics:

- Why joins?, real databases have multiple related tables
- Foreign key: column in one table referencing primary key of another
- Example: Student(RollNo, Name, ClassID) and Class(ClassID, ClassName, Teacher)
- Cartesian Product / Cross Join: every row of Table1 × every row of Table2
- If Table1 has 5 rows and Table2 has 3 rows result has 15 rows
- Cartesian product is rarely useful, need to filter with conditions
- Equi-Join: Cartesian product + WHERE condition matching common columns
- SELECT s.Name, c.ClassName FROM student s, class c WHERE s.ClassID = c.ClassID;
- Table aliases: student s, class c, shorter names for readability
- Using dot notation: s.Name, c.ClassName, specify which table's column
- Why aliases are needed: when both tables have same column name
- Natural Join: automatically joins on columns with same name
- SELECT * FROM student NATURAL JOIN class;, auto-matches ClassID
- Natural join removes duplicate columns in result
- Equi-join vs Natural Join: equi-join gives you control, natural join is automatic
- Join with additional conditions: WHERE s.ClassID = c.ClassID AND s.Marks > 80
- Joining with aggregate functions: SELECT c.ClassName, AVG(s.Marks) FROM student s, class c WHERE s.ClassID = c.ClassID GROUP BY c.ClassName;
- Board exam join patterns: 2-table queries with conditions, aggregates, sorting
- Practice: identifying which tables to join and on which columns

Projects built:

- Student-Marks-Subject multi-table query project
- Employee-Department join queries, 20 questions
- Product-Order-Customer join queries
- Complete SQL revision: single table + multi-table + functions + aggregates

Practice: 30 join queries, Preeti Arora join exercises, board exam join questions from last 10 years

SQL Complete Revision + 15 SQL Queries for Practical File

Topics:

- Complete SQL revision, all commands:
- DDL: CREATE DATABASE/TABLE, ALTER TABLE, DROP TABLE/DATABASE
- DML: INSERT INTO, UPDATE, DELETE
- DQL: SELECT with WHERE, BETWEEN, IN, LIKE, IS NULL/IS NOT NULL
- Functions: Math (POWER, ROUND, MOD), String (UPPER, LOWER, MID, LENGTH, LEFT, RIGHT, INSTR, TRIM, CONCAT), Date (NOW, DATE, MONTH, MONTHNAME, YEAR, DAY, DAYNAME)
- Aggregates: COUNT, SUM, AVG, MAX, MIN
- Clauses: GROUP BY, HAVING, ORDER BY, DISTINCT
- Joins: Cartesian product, Equi-join, Natural join
- 15+ SQL queries for practical file:
- Q1: CREATE TABLE with constraints (PRIMARY KEY, NOT NULL)
- Q2: INSERT INTO, multiple rows
- Q3: SELECT with WHERE using comparison operators
- Q4: SELECT with BETWEEN and IN
- Q5: SELECT with LIKE and wildcards
- Q6: SELECT with IS NULL / IS NOT NULL
- Q7: SELECT with ORDER BY (ASC and DESC)
- Q8: UPDATE records with WHERE condition
- Q9: DELETE records with WHERE condition
- Q10: ALTER TABLE, add and modify column
- Q11: Aggregate functions, COUNT, SUM, AVG, MAX, MIN
- Q12: GROUP BY with aggregate functions
- Q13: GROUP BY with HAVING clause
- Q14: SQL functions, UPPER, LOWER, MID, LENGTH, ROUND, MOD
- Q15: Equi-join on two tables with conditions

Deliverables:

- 15+ SQL queries for practical file
- Complete SQL command reference sheet
- All queries tested and verified on MySQL

Months 20-21: Computer Networks & Societal Impacts (Week 77-84)

Introduction to Computer Networks (CBSE IP Unit 3-10 marks)

Topics:

- What is a computer network?, interconnected devices sharing resources

- Types of networks based on geographical area:
- PAN (Personal Area Network): Bluetooth headset, smartwatch, within 10 meters
- LAN (Local Area Network): within a building, school lab, office, home WiFi
- MAN (Metropolitan Area Network): across a city, cable TV network, city WiFi
- WAN (Wide Area Network): across countries/continents, Internet is the largest WAN
- Networking devices:
- Modem: MOdulator-DEModulator, converts digital to analog and vice versa
- Hub: broadcasts data to ALL connected devices, dumb device, creates traffic
- Switch: intelligent hub, sends data ONLY to intended device using MAC addresses
- Repeater: amplifies/regenerates weak signals, extends cable distance
- Router: connects different networks, directs traffic using IP addresses
- Gateway: connects networks with DIFFERENT protocols, translates between them
- NIC (Network Interface Card): hardware for network connectivity, has MAC address
- WiFi adapter: wireless NIC for WiFi connectivity
- Network topologies:
- Star topology: all devices connect to central hub/switch
- Advantages: easy to add/remove, failure of one device doesn't affect others
- Disadvantage: central hub failure takes down entire network
- Bus topology: all devices on single backbone cable with terminators
- Advantages: cheap, easy to install
- Disadvantages: cable failure affects all, difficult troubleshooting
- Tree/Hierarchical topology: combination of star and bus, like a tree structure
- Mesh topology: every device connected to every other device
- Advantage: maximum redundancy, Disadvantage: very expensive

Projects built:

- Network diagram for a school, choose topology, devices, media
- Networking device comparison chart with functions and use cases
- Topology pros-cons table with real-world examples

Practice: Preeti Arora network chapter, 40 MCQs on network types, devices, topologies

Internet, Web Technologies & Protocols

Topics:

- Internet: global network of networks, largest WAN
- How internet works: ISP, DNS, routers, data packets
- Internet services:
- WWW (World Wide Web): system of interlinked hypertext documents, uses HTTP
- Email: electronic mail, SMTP (sending), POP3/IMAP (receiving)
- Chat/Instant Messaging: WhatsApp, Telegram, Slack
- VoIP (Voice over IP): voice calls over internet, Zoom, Google Meet, Skype
- Video conferencing: audio + video communication over internet
- URL (Uniform Resource Locator): address of a web resource
- URL structure: protocol://domain_name:port/path/page?query#fragment
- Domain name: human-readable website address (google.com, example.org)
- DNS (Domain Name System): translates domain names to IP addresses
- Domain extensions: .com, .org, .edu, .gov, .in, .co.in
- Websites: collection of related web pages
- Static web pages: fixed content, created with HTML/CSS, same for all users
- Dynamic web pages: content changes based on user, time, interaction, uses server-side processing
- Web server: computer that hosts websites and serves web pages
- Web hosting: storing website on a server for internet access
- Types of hosting: shared, VPS, dedicated, cloud hosting
- Web browser: software to access websites, Chrome, Firefox, Edge, Safari
- Browser features: tabs, bookmarks, history, extensions/add-ons, plug-ins
- Cookies: small data files stored by websites on your browser
- First-party cookies: set by the website you visit
- Third-party cookies: set by advertisers/trackers, privacy concern

Projects built:

- URL anatomy exercise, break down 10 URLs into components
- Static vs dynamic website comparison, 10 examples each
- Web technology glossary, define 30 web-related terms
- Browser feature exploration report

Practice: Preeti Arora web chapter, 50 MCQs on internet, web, protocols, cookies

Societal Impacts of Technology (CBSE IP Unit 4-10 marks)

Topics:

- Digital footprint: trail of data you leave on the internet
- Active digital footprint: intentional, social media posts, emails, comments
- Passive digital footprint: unintentional, browsing history, cookies, IP logs
- Managing your digital footprint: privacy settings, thinking before posting
- Communication etiquette (netiquette): proper online behavior
- Email etiquette: professional subject lines, proper greeting, no ALL CAPS
- Social media etiquette: respect others, verify before sharing, no cyberbullying
- Data protection: safeguarding personal and organizational data
- Personal data: name, address, phone, email, financial info, biometric data
- Consent: permission before collecting/using personal data
- Intellectual Property Rights (IPR):
- Copyright: legal right over creative works, books, music, software, art
- Patent: protection for inventions, machines, processes, formulas
- Trademark: protection for brand identity, logos, names, slogans
- Plagiarism: using someone else's work without proper credit, academic dishonesty

- How to avoid plagiarism: cite sources, paraphrase properly, use quotation marks
- Software licensing:
- Proprietary: paid, closed source, Windows, Adobe, MS Office
- Free and Open Source Software (FOSS): free to use, modify, distribute
- FOSS examples: Linux, Firefox, LibreOffice, Python, MySQL, Android, WordPress
- Creative Commons: flexible copyright licenses, CC-BY, CC-BY-SA, CC-BY-NC
- GPL (GNU General Public License): must keep derivatives open source
- Cybercrime: criminal activities using computers/internet
- Hacking: unauthorized access to computer systems
- Phishing: fraudulent emails/websites to steal login credentials
- Cyberbullying: harassment, threats, humiliation online
- Identity theft: stealing personal information for fraud
- Ransomware: malware that encrypts data and demands payment
- Indian Information Technology Act 2000 (amended 2008):
- Section 43: penalty for unauthorized access, data theft
- Section 66: computer-related offences, hacking, identity theft
- Section 67: publishing obscene material electronically
- Section 72: breach of confidentiality and privacy
- E-waste: discarded electronic devices, hazards and management
- E-waste hazards: toxic materials (lead, mercury, cadmium), soil/water pollution
- E-waste management: reduce, reuse, recycle, proper disposal at certified centers
- Health concerns from technology: digital eye strain, back/neck pain, carpal tunnel syndrome
- Ergonomics: proper posture, screen distance, break intervals (20-20-20 rule)
- Technology addiction: social media addiction, gaming disorder, awareness and balance

Projects built:
- Digital citizenship poster, 15 dos and don'ts for online behavior
- Cybercrime awareness infographic, types, prevention, reporting
- E-waste management plan for school/home
- IPR case study, analyze 3 real-world copyright/patent cases

Practice: Preeti Arora societal impacts chapter, NCERT exercises, 60 MCQs, case study questions, IT Act scenario-based questions

Months 22-24: Board Exam Preparation, Intensive Revision & Mock Tests (Week 85-96)

Practical File Completion & Project Submission

Topics:
- Complete practical file requirements verification:
- 15+ Pandas programs, Series and DataFrame operations
- 4+ Matplotlib programs, line chart, bar graph, histogram with customization
- 15+ SQL queries, DDL, DML, DQL, functions, aggregates, joins
- Project work (5 marks): data analysis project using Pandas + Matplotlib
- Suggested projects:
- COVID-19 data analysis, cases, recoveries, deaths by country/state
- Weather data analysis, temperature, rainfall trends over years
- Student performance analyzer, class-wise, subject-wise analysis with charts
- IPL/cricket statistics analysis, batting, bowling averages with visualization
- E-commerce sales analysis, product category wise, monthly trends
- Project documentation: cover page, certificate, acknowledgement, introduction, source code, output, bibliography
- Viva voce preparation, 50 most-asked questions:
- What is Pandas? What is a DataFrame? What is a Series?
- Difference between loc and iloc?
- What is matplotlib? How to create a bar chart?
- What is GROUP BY? What is HAVING?
- Difference between WHERE and HAVING?
- What is a primary key? What is a foreign key?
- What is an equi-join?
- What is a digital footprint?

Deliverables:
- Complete practical file, all programs tested and documented
- Project with full documentation
- Viva voce preparation notes, 50 Q&A

Chapter-Wise Revision, Pandas & Matplotlib

Topics:
- Pandas Series revision: creation, indexing, slicing, operations, methods, NaN handling
- Pandas DataFrame revision: creation, selection, filtering, sorting, groupby, CSV operations
- DataFrame vs Series, comparison table
- loc vs iloc, 20 practice problems
- Boolean indexing revision, complex condition filtering
- Aggregate functions on DataFrame: sum, mean, max, min, count, describe
- Matplotlib revision: line chart, bar graph, histogram
- Chart customization checklist: title, xlabel, ylabel, legend, grid, color, marker, linestyle
- Format strings revision: 'r--o', 'b-s', 'g:^'
- Common Pandas output prediction patterns
- Quick reference card for Pandas methods and Matplotlib functions

Practice: 40 Pandas MCQs, 20 output prediction, 15 Matplotlib code-writing questions

Chapter-Wise Revision, SQL, Networks & Societal Impacts

Topics:
- SQL complete revision: DDL, DML, DQL
- SQL functions revision: POWER, ROUND, MOD, UPPER, LOWER, MID, LENGTH, LEFT, RIGHT, INSTR,

TRIM, CONCAT, NOW, DATE, MONTH, YEAR, DAY, MONTHNAME, DAYNAME
- Aggregate functions revision: COUNT, SUM, AVG, MAX, MIN
- GROUP BY + HAVING revision, 20 practice queries
- Joins revision: Cartesian product, equi-join, natural join
- Complete SQL query writing, from WHERE to JOIN
- Computer networks revision: PAN, LAN, MAN, WAN, devices, topologies
- Internet and web concepts revision: URL, DNS, HTTP/HTTPS, static/dynamic, cookies
- Societal impacts revision: digital footprint, IPR, cybercrime, IT Act, e-waste
- Case study practice, network design scenarios (CBSE board favorite)

Practice: 50 SQL revision queries, 40 network MCQs, 30 societal impact MCQs, case study practice

Mock Board Exams, Set 1 & 2

Topics:
- Full-length Mock Exam 1: 70 marks, 3 hours, strict exam conditions
- CBSE IP paper structure:
- Section A: 21 questions × 1 mark = 21 marks (MCQ, True/False, Fill-in-blanks)
- Section B: 7 questions × 2 marks = 14 marks (Very Short Answer)
- Section C: 4 questions × 3 marks = 12 marks (Short Answer)
- Section D: 2 questions × 4 marks = 8 marks (Case Study based)
- Section E: 3 questions × 5 marks = 15 marks (Long Answer)
- Detailed solution discussion with CBSE marking scheme
- Full-length Mock Exam 2: different question set, same structure
- Common mistakes analysis and correction
- Time management strategy:
- Section A (21 marks): 30-35 minutes
- Section B (14 marks): 20 minutes
- Section C (12 marks): 20 minutes
- Section D (8 marks): 15 minutes
- Section E (15 marks): 25 minutes
- Review time: 10-15 minutes
- How to write Pandas/Matplotlib code in board exams
- How to write SQL queries in board exams, formatting tips

Deliverables:
- 2 complete mock papers solved and reviewed
- Weak areas identified with improvement plan
- Exam writing technique refined

Mock Board Exams, Set 3, 4, 5 + Previous Year Paper Solving

Topics:
- 3 more full-length mock exams (Set 3, 4, 5)
- CBSE IP Previous Year Papers solved:
- 2020, 2021, 2022, 2023, 2024, 2025, complete solutions with marking scheme
- CBSE IP Sample Paper 2026-27, official sample paper solved
- Frequently repeated question analysis, top 30 topics that appear every year
- Most important Pandas questions: Series operations, DataFrame creation, loc/iloc, groupby
- Most important SQL questions: GROUP BY + HAVING, aggregate functions, joins, functions
- Most important theory questions: network devices, topologies, cybercrime, IPR, IT Act
- Case study question patterns, how to approach and answer
- Assertion-Reason question strategy, understanding the format
- Partial marks strategy, how to get step marks even if you don't know complete answer

Practice: 5 mock papers + 6 previous year papers = 11 full papers solved

Final Revision & Last-Minute Board Exam Preparation

Topics:
- One-shot revision: entire IP syllabus in 2 days, key points only
- Pandas cheat sheet: all methods, attributes, operations on one page
- Matplotlib cheat sheet: plot types, customization options, format strings
- SQL command reference: all DDL, DML, DQL, functions on 2 pages
- Network concepts summary: devices, topologies, protocols on 1 page
- Societal impacts summary: key terms, IT Act sections on 1 page
- Top 30 most important Pandas output prediction problems
- Top 20 SQL queries you must know by heart
- Top 15 theory questions with model answers
- Board exam day strategy: reading time, question selection, presentation
- How to handle unfamiliar questions, partial answer strategies
- Confidence building: you've practiced more than 95% of students
- Practical exam preparation: Pandas program + SQL queries + project viva

Deliverables:
- Quick revision notes, entire syllabus on 8-10 pages
- Pandas + Matplotlib reference sheet, 2 pages
- SQL reference sheet, 2 pages
- Theory revision sheet, 2 pages
- 11+ solved papers in practice
- READY FOR BOARD EXAM, 90+ score target

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Class 11 Sem 1): Python console programs - calculators, grade tools, list and dictionary applications - plus a full school database built and queried in MySQL
- Phase 2 (Class 11 Sem 2): the complete Class 11 practical file with 14+ Python programs and 14+ SQL queries, plus an emerging trends presentation project

- Phase 3 (Class 12 Sem 1): real-dataset analysis with Pandas Series and DataFrames, a CSV import-analyze-export pipeline, and 4+ fully customized Matplotlib charts (line, bar, histogram)
- Phase 4 (Class 12 Sem 2): advanced SQL query sets (single-row functions, GROUP BY, HAVING, joins) and the final board project combining Pandas, Matplotlib, and CSV data with documentation
- Total: 2 board-ready practical files, 2 submission projects, and dozens of graded programs and query sets

Total projects: 35+ graded programs across both years, including 2 complete board practical files (Pandas, Matplotlib, and SQL) and 2 board-submission projects

SKILLS THE STUDENT WILL HAVE AT THE END

- Python Basics: variables, operators, control flow, lists, dictionaries, and menu-driven programs
- Pandas: creating Series and DataFrames, loc and iloc selection, Boolean filtering, sorting, and aggregation
- Data Visualization: line charts, bar graphs, and histograms in Matplotlib with titles, labels, legends, and grids
- CSV Workflow: importing data with read_csv, cleaning and analyzing it, and exporting results with to_csv
- SQL on MySQL: DDL, DML, single-row and aggregate functions, GROUP BY, HAVING, ORDER BY, and joins
- Computer Networks: network types, devices, topologies, and internet basics as per the IP syllabus
- Societal Impacts: digital footprint, cyber safety, IPR, plagiarism, e-waste, and the IT Act
- Exam Technique: output prediction, query writing, and marking-scheme based answer writing

HOW A TYPICAL WEEK IS SPLIT

Live classes: 3-4 live classes per week, 1 hour each, in sync with the school's chapter sequence
Practical work: 2-3 hours of hands-on work: Pandas programs, Matplotlib charts, and SQL queries executed on a live MySQL database
Board style practice: Weekly chapter tests, output-prediction drills, query-writing practice, and answer writing trained on the CBSE marking scheme
Doubt support: Dedicated doubt-clearing sessions, with extra slots before school exams, pre-boards, and board exams

SUPPORT INCLUDED

Small batches: Small group batches so every student gets individual attention and screen time
Dedicated ip batch: An IP-only (code 065) batch - never mixed with CS (083) or ICSE students
Exam season support: Extra revision classes and doubt sessions before pre-boards and board exams
Practical and project help: Step-by-step help with every practical file entry and the board project until they are submission-ready
Progress tracking: Regular test results and progress updates shared with parents

CERTIFICATION DETAIL

Course certificate: Modern Age Coders Course Completion Certificate at the end of each academic year
Board exam readiness: Previous year papers, the official CBSE sample paper, and full mock exams on the current IP (065) pattern: 70 marks theory plus 30 marks practical
Practical file support: Complete guidance on the required practical file entries (15+ Pandas programs, 4+ Matplotlib charts, 15+ SQL queries), the board project, and viva preparation for the 30-mark practical exam

WHERE THIS CAN LEAD

- BCA, B.Sc IT, and B.Sc Data Science admissions with working Python and SQL skills
- BBA, B.Com, and economics programs where data analysis skills stand out
- Data analytics and business analytics degree specializations
- Data analyst internships that expect Pandas and SQL basics
- Business intelligence and reporting work later on - the SQL taught here transfers directly
- Statistics, journalism, and social science programs that increasingly use Python for data work
- A smooth upgrade path into full data science courses after Class 12

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Informatics Practices (IP)? Is it different from Computer Science (CS)?
A: Yes, IP (Code 065) and CS (Code 083) are different CBSE subjects. IP focuses on data handling with Pandas, data visualization with Matplotlib, and SQL, it's application-oriented and considered easier. CS focuses on deep Python programming, data structures (stacks),

sorting/searching algorithms, and Python-MySQL connectivity, it's more programming-intensive. We run separate batches for IP and CS students.

Q: Is IP easier than CS? Should I choose IP?
A: IP is generally considered easier because it focuses on using Python libraries (Pandas, Matplotlib) for data analysis rather than building algorithms from scratch. If you're interested in data science, business analytics, or want a higher-scoring subject, IP is a great choice. If you want to pursue software engineering or competitive programming, CS is better. We can help you decide based on your goals.

Q: Do you have separate batches for IP and CS?
A: Absolutely. IP and CS have completely different syllabi, different units, different topics, different exam patterns. Our IP batch only teaches IP content (Pandas, Matplotlib, SQL, networks, societal impacts). No mixing with CS topics like file handling, data structures, or Python-MySQL connectivity.

Q: Which textbook do you follow for IP?
A: We primarily follow Preeti Arora (Informatics Practices with Python), the most popular IP textbook, supplemented by NCERT IP textbook and Sumita Arora. Our teaching covers every chapter from these books.

Q: Do I need to install anything on my computer?
A: Yes, you'll need Python 3.12+ (with Pandas, NumPy, Matplotlib, installed via pip) and MySQL Community Server. We guide you through the complete installation process in the first few classes.

Q: Will you help with the practical file and project?
A: Yes, completely. We prepare your entire practical file (15+ Pandas programs, 4+ Matplotlib charts, 15+ SQL queries), guide you through a data analysis project with full documentation, and prepare you for the viva voce examination. You submit a polished, complete practical file.

Q: Can this course help me score 90+ in IP board exam?
A: IP has a more predictable question pattern than CS, making high scores very achievable with proper preparation.

Q: Is there an ICSE equivalent of IP?
A: No, ICSE/ISC does not have an Informatics Practices subject. ISC offers Computer Science (Java-based) which is more similar to CBSE CS. If you're an ICSE student interested in data analysis, you can join our IP batch to learn Pandas/Matplotlib skills alongside your board preparation, these are valuable skills regardless of your board.

Q: What career options does IP open up?
A: IP gives you a strong foundation for Data Analyst, Business Analyst, SQL Developer, Data Scientist, BI Developer, IT Manager, Digital Marketing Analyst, and Financial Analyst roles. The Python + SQL + data visualization skills you learn are directly used in industry. Many BCA, B.Sc IT, BBA, and MBA programs value these skills.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/cbse-informatics-practices-ip-class-11-12-python-pandas-sql-complete-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 87 OF 137: JAVA COURSE FOR COLLEGE: CORE JAVA TO SPRING BOOT
==========

Course link: https://learn.modernagecoders.com/courses/complete-java-programming-masterclass-college
Subject area: Programming
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Java course for college students and professionals: core Java, collections and concurrency, then Spring Boot APIs with security and tests, in 10-12 months.

In one line: The language enterprises run on and placements test: core Java to real depth, then Spring Boot the way backend teams

actually use it.

ABOUT THE PROGRAMME

Java is still the backbone of enterprise software and the single most-tested language in Indian placements, and the difference between knowing Java syntax and owning Java depth, collections, concurrency, the memory model, decides both interviews and real work. This live online masterclass builds that depth in order: core Java and object-oriented thinking done properly, then the engine room, collections in depth, exceptions, multithreading and modern Java features, design patterns and JDBC with real testing habits, then the job stack: Spring Boot, Spring Data JPA, security, REST APIs designed and tested like a professional. A finale month ships a Spring Boot capstone and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- Placement-grade core: collections, concurrency and OOP at the depth interviews probe
- Modern Java throughout: streams, lambdas and current LTS features, not textbook Java
- Spring Boot as teams use it: REST APIs, JPA, security and tests, not hello-world
- Design patterns and JDBC before the framework, so Spring is understood, not incanted
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Months 1-3): core Java and object-oriented programming properly
Phase 2: The engine room (Months 4-6): collections, concurrency, modern Java, patterns and testing
Phase 3: The job stack (Months 7-9): Spring Boot, JPA, security and production habits
Phase 4: The finale (Month 10): a Spring Boot capstone and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Everything is taught from the first line
Equipment: Any computer with a stable internet connection; every tool used is free
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

College students: Students whose placements will test Java, which is most of them
Career switchers: Professionals entering backend development on the enterprise stack
Self taught: Learners with syntax knowledge but not the depth interviews reach for
Backend bound: Developers who want the Spring ecosystem opened properly
Android curious: Future Android developers who want the Java foundation Kotlin assumes

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & CORE JAVA SKILLS (MONTHS 1-3, WEEKS 1-13)

Build rock-solid Java fundamentals. Learn programming logic, master Java syntax, OOP principles, and create your first applications.

Months 1-2: Java Fundamentals & Programming Basics (Weeks 1-7)
Introduction to Java & Development Environment Setup
Topics:
- What is Java? History, features, and platform independence
- Java ecosystem: JDK, JRE, JVM explained
- Write Once Run Anywhere (WORA) concept
- Java applications: Desktop, Web, Mobile (Android), Enterprise
- Installing JDK (Java 17 LTS or latest)
- Setting up development environment: IntelliJ IDEA, Eclipse, VS Code
- Understanding IDE features and shortcuts
- Your first Java program: Hello World
- Java program structure: class, main method
- Compiling and running Java programs
- Understanding bytecode and JVM
- Comments: single-line (//), multi-line (/* */), Javadoc (/** */)
- Java coding conventions and naming standards
Projects built:
- Hello World variations with personalized messages
- Simple output programs using System.out.println
- Personal information display program

- ASCII art generator in Java

Practice: Daily: 30 min Java syntax practice, write 5-10 simple programs

Variables, Data Types & Operators

Topics:

- Variables: declaration, initialization, naming rules
- Primitive data types: byte, short, int, long, float, double, char, boolean
- Understanding type sizes and ranges
- Type casting: implicit (widening) and explicit (narrowing)
- Wrapper classes: Integer, Double, Character, Boolean, etc.
- Autoboxing and unboxing
- Literals: integer, floating-point, character, string, boolean
- Operators: arithmetic (+, -, *, /, %, ++, --)
- Relational operators: ==, !=, >, <, >=, <=
- Logical operators: &&, ||, !
- Assignment operators: =, +=, -=, *=, /=, %=
- Bitwise operators: &, |, ^, ~, <<, >>, >>>
- Ternary operator: ? :
- Operator precedence and associativity
- String basics: creation, concatenation, immutability

Projects built:

- Simple calculator (four basic operations)
- Temperature converter (Celsius, Fahrenheit, Kelvin)
- Age calculator with detailed output
- BMI calculator with health categories
- Currency converter
- Area and perimeter calculator for shapes
- Compound interest calculator

Practice: Solve 30 problems on variables, data types, and operators

Control Flow Statements

Topics:

- Boolean expressions and conditions
- If statement: single condition execution
- If-else statement: two-way branching
- If-else-if ladder: multiple conditions
- Nested if statements
- Switch statement: multi-way branching
- Switch with fall-through and break
- Enhanced switch (Java 12+): arrow syntax, yield
- While loop: pre-tested loop
- Do-while loop: post-tested loop
- For loop: counter-based iteration
- Enhanced for loop (for-each): iterating arrays/collections
- Break statement: exit loop
- Continue statement: skip iteration
- Labeled break and continue
- Nested loops and loop patterns

Projects built:

- Number guessing game with attempts limit
- Grade calculator with letter grades
- Even/odd checker and list generator
- Prime number checker and generator
- Multiplication table generator
- Factorial calculator (iterative)
- Fibonacci series generator
- Pattern printing (stars, numbers, pyramids, diamonds)
- FizzBuzz challenge
- Menu-driven calculator with switch
- Simple ATM simulation

Practice: Solve 40 control flow and loop problems

Methods & Arrays

Topics:

- Methods: definition, purpose, and structure
- Method declaration: return type, name, parameters
- Method invocation and calling
- Parameters vs arguments
- Return statement and return values
- Method overloading: same name, different parameters
- Variable scope: local vs instance vs class
- Static methods vs instance methods
- Recursion: base case and recursive case
- Arrays: declaration, creation, initialization
- Array indexing and accessing elements
- Array length property
- Iterating arrays: for loop, enhanced for loop
- Multi-dimensional arrays (2D, 3D)
- Jagged arrays
- Arrays class utility methods
- Variable arguments (varargs)

Projects built:

- Method library for mathematical operations
- Array statistics calculator (sum, average, min, max)
- Linear search and binary search implementation
- Sorting algorithms: bubble sort, selection sort
- Matrix operations (addition, multiplication)
- Tic-tac-toe game with 2D array
- Student grade management system
- Factorial using recursion
- Tower of Hanoi recursive solution
- Palindrome checker (string/number)
- Prime number generator in range

Practice: Create 25 methods for various functionalities, solve 30 array problems

Month 3: Object-Oriented Programming Fundamentals (Weeks 8-13)

Classes, Objects & OOP Basics

Topics:

- Object-Oriented Programming paradigm
- Classes: blueprint for objects
- Objects: instances of classes
- Class declaration and structure
- Instance variables (attributes/fields)
- Instance methods (behaviors)
- Creating objects with 'new' keyword
- Constructors: default and parameterized
- Constructor overloading
- this keyword: referencing current object
- Access modifiers: private, public, protected, default
- Encapsulation: data hiding principle
- Getters and setters (accessor and mutator methods)
- toString() method for object representation
- Object initialization blocks
- Static variables (class variables)
- Static methods and static blocks
- Static import

Projects built:

- Student class with attributes and methods
- Bank Account class with deposit/withdraw
- Book class for library management
- Employee class with salary calculations
- Rectangle/Circle classes with area/perimeter
- Car class with properties and behaviors
- Calculator class with static methods
- Product class for inventory system
- Person class with constructor overloading
- Date class implementation

Practice: Create 20 different classes modeling real-world entities

Inheritance & Polymorphism

Topics:

- Inheritance: IS-A relationship
- Extends keyword for inheritance
- Parent class (superclass) and child class (subclass)
- Inheriting fields and methods
- Method overriding: redefining parent methods
- @Override annotation
- super keyword: accessing parent class members
- super() for calling parent constructor
- Constructor chaining in inheritance
- Types of inheritance: single, multilevel, hierarchical
- Multiple inheritance limitation in Java
- Final keyword: final variables, methods, classes
- Polymorphism: compile-time and runtime
- Method overloading (compile-time polymorphism)
- Method overriding (runtime polymorphism)
- Dynamic method dispatch
- Upcasting and downcasting
- instanceof operator

Projects built:

- Animal hierarchy (Animal -> Dog, Cat, Bird)
- Shape hierarchy with area calculation
- Vehicle system (Vehicle -> Car, Bike, Truck)
- Employee hierarchy (Employee -> Manager, Developer, Designer)
- Bank account types (Account -> Savings, Current, Fixed)
- School management (Person -> Student, Teacher)
- E-commerce product categories
- Payment system with different payment types

Practice: Build 15 inheritance hierarchies, solve 25 polymorphism problems

Abstraction & Interfaces

Topics:

- Abstraction: hiding implementation details

- Abstract classes with 'abstract' keyword
- Abstract methods: methods without body
- Concrete methods in abstract classes
- When to use abstract classes
- Interfaces: 100% abstraction (pre-Java 8)
- Interface declaration with 'interface' keyword
- Implementing interfaces with 'implements'
- Multiple interface implementation
- Interface variables: public static final
- Interface methods: public abstract
- Default methods in interfaces (Java 8+)
- Static methods in interfaces (Java 8+)
- Private methods in interfaces (Java 9+)
- Functional interfaces and @FunctionalInterface
- Marker interfaces
- Interface vs abstract class comparison
- Real-world abstraction examples

Projects built:
- Shape calculator with abstract classes
- Payment gateway with interfaces
- Database connection abstraction
- Notification system (Email, SMS, Push)
- Remote control interface for devices
- Sorting algorithms with Comparable/Comparator
- Plugin architecture using interfaces
- PHASE 1 MINI CAPSTONE: Library Management System (Console-based)
- Features: Books, members, issue/return, search, OOP principles

Practice: Solve 30 abstraction and interface problems

Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 2: INTERMEDIATE JAVA & ADVANCED CONCEPTS (MONTHS 4-6, WEEKS 14-26)

Master advanced Java features, Collections Framework, exception handling, file I/O, multithreading, and design patterns.

Month 4: Advanced Java Features & Collections (Weeks 14-18)

String Handling & Regular Expressions

Topics:
- String class: immutability and string pool
- String methods: length, charAt, substring, indexOf
- String comparison: equals, equalsIgnoreCase, compareTo
- String manipulation: concat, replace, trim, split
- String case methods: toUpperCase, toLowerCase
- StringBuilder class: mutable strings
- StringBuffer class: thread-safe mutable strings
- StringBuilder vs StringBuffer vs String
- String formatting: format(), printf()
- StringTokenizer class
- Regular expressions (regex) in Java
- Pattern and Matcher classes
- Regex patterns: character classes, quantifiers, groups
- Common regex operations: validation, search, replace
- Regex for email, phone, URL validation

Projects built:
- Text analyzer (word count, character count, frequency)
- String manipulation utility library
- Email validator using regex
- Phone number formatter
- Password strength validator
- URL extractor from text
- CSV parser using String methods
- Palindrome and anagram checker
- Caesar cipher encryption/decryption
- Log file parser

Practice: Solve 35 string manipulation and regex problems

Exception Handling & Error Management

Topics:
- What are exceptions? Error vs Exception
- Exception hierarchy: Throwable, Error, Exception
- Checked exceptions vs unchecked exceptions
- Try-catch block for exception handling
- Multiple catch blocks
- Catch block order: specific to general
- Finally block: cleanup code
- Try-with-resources (Java 7+): automatic resource management
- Throw keyword: throwing exceptions manually
- Throws keyword: declaring exceptions

    - Creating custom exceptions
    - Custom checked and unchecked exceptions
    - Exception chaining and cause
    - Best practices for exception handling
    - Common exceptions: NullPointerException, ArrayIndexOutOfBoundsException, etc.
    - Rethrowing exceptions
    - Multi-catch (Java 7+)
  Projects built:
    - Robust calculator with exception handling
    - File reader with proper error handling
    - Custom banking exception system
    - Input validation with custom exceptions
    - Safe array operations utility
    - Age validator with custom exceptions
    - Division calculator with exception handling
    - User registration with validation exceptions
  Practice: Solve 25 exception handling scenarios
Collections Framework - Part 1
  Topics:
    - Collections Framework overview and architecture
    - Collection interface hierarchy
    - List interface and implementations
    - ArrayList: dynamic array implementation
    - ArrayList methods: add, remove, get, set, size, clear
    - LinkedList: doubly-linked list implementation
    - ArrayList vs LinkedList performance comparison
    - Vector and Stack (legacy classes)
    - Set interface: unique elements
    - HashSet: hash table implementation
    - LinkedHashSet: insertion order maintenance
    - TreeSet: sorted set using Red-Black tree
    - Set operations and methods
    - Iterator and ListIterator interfaces
    - Enhanced for loop with collections
    - Collections class utility methods
  Projects built:
    - Todo list application with ArrayList
    - Student management system with List
    - Phonebook with ArrayList and search
    - Unique word counter with HashSet
    - Sorted student list with TreeSet
    - Remove duplicates from list
    - Collection operations library
    - Inventory management with collections
  Practice: Solve 40 List and Set problems
Collections Framework - Part 2
  Topics:
    - Map interface: key-value pairs
    - HashMap: hash table based map
    - HashMap methods: put, get, remove, containsKey, keySet, values
    - LinkedHashMap: insertion order maintenance
    - TreeMap: sorted map using Red-Black tree
    - Hashtable: legacy synchronized map
    - HashMap vs Hashtable vs ConcurrentHashMap
    - Queue interface: FIFO structure
    - PriorityQueue: heap-based priority queue
    - Deque interface: double-ended queue
    - ArrayDeque implementation
    - Comparable interface for natural ordering
    - Comparator interface for custom ordering
    - Sorting collections: Collections.sort()
    - Searching collections: Collections.binarySearch()
    - Generics in collections
    - Type safety with generics
  Projects built:
    - Word frequency counter with HashMap
    - Contact management with Map
    - Student grade book with TreeMap
    - Priority task manager with PriorityQueue
    - Cache implementation with LinkedHashMap
    - Custom sorting with Comparator
    - Leaderboard system with TreeSet
    - Configuration manager with Properties
  Practice: Solve 40 Map and Queue problems
Generics & Advanced Collections
  Topics:
    - Introduction to generics: type parameters
    - Generic classes and interfaces
    - Generic methods

- Bounded type parameters: extends, super
- Wildcards: ?, ? extends T, ? super T
- Generic constructors
- Type erasure in generics
- Restrictions on generics
- Generic collections usage
- Creating custom generic classes
- EnumSet and EnumMap
- Immutable collections (Java 9+)
- List.of(), Set.of(), Map.of()
- Collections performance comparison
- Time and space complexity of operations

Projects built:
- Generic stack and queue implementation
- Generic pair and tuple classes
- Type-safe database repository
- Generic utility library
- Custom generic collection class
- Generic algorithm implementations

Practice: Create 15 generic classes, solve 25 generics problems

Month 5: File I/O, Multithreading & Modern Java (Weeks 19-22)

File I/O & Serialization

Topics:
- File handling in Java: File class
- Creating, deleting, renaming files and directories
- File properties: exists, length, canRead, canWrite
- Byte streams: InputStream, OutputStream
- FileInputStream and FileOutputStream
- Character streams: Reader, Writer
- FileReader and FileWriter
- Buffered streams: BufferedReader, BufferedWriter
- PrintWriter for formatted output
- Scanner for reading input
- NIO (New I/O) package: Path, Paths, Files
- Reading and writing with NIO
- Serialization: converting objects to bytes
- Serializable interface
- ObjectInputStream and ObjectOutputStream
- Transient keyword
- Externalizable interface for custom serialization

Projects built:
- File-based note-taking application
- File copy and move utility
- Text file analyzer (line count, word count)
- CSV file reader and writer
- Contact manager with file persistence
- Student records with serialization
- Configuration file manager
- Log file generator and reader
- File encryption/decryption utility
- Directory tree viewer

Practice: Build 15 file handling applications

Multithreading & Concurrency - Part 1

Topics:
- Multithreading concepts: process vs thread
- Thread lifecycle: new, runnable, running, blocked, terminated
- Creating threads: extending Thread class
- Creating threads: implementing Runnable interface
- Thread class methods: start, run, sleep, join
- Thread priorities: setPriority, getPriority
- Daemon threads
- Thread synchronization: race condition problem
- Synchronized keyword: synchronized methods
- Synchronized blocks
- Inter-thread communication: wait, notify, notifyAll
- Deadlock and its prevention
- Thread safety and immutability
- ThreadLocal class
- Thread groups

Projects built:
- Multi-threaded counter application
- Producer-consumer problem solution
- Bank account with synchronized withdrawal
- Print even-odd numbers using two threads
- Multi-threaded file downloader
- Thread pool executor example
- Dining philosophers problem
- Ticket booking system with synchronization

```
    Practice: Solve 30 multithreading problems
  Multithreading & Concurrency - Part 2
    Topics:
      - Executor framework: ExecutorService
      - Thread pools: FixedThreadPool, CachedThreadPool
      - Callable and Future interfaces
      - CompletableFuture for async programming
      - Concurrent collections: ConcurrentHashMap
      - CopyOnWriteArrayList
      - BlockingQueue and its implementations
      - Atomic variables: AtomicInteger, AtomicLong
      - Lock interface and ReentrantLock
      - ReadWriteLock for read-write operations
      - Semaphore for resource control
      - CountDownLatch and CyclicBarrier
      - Fork/Join framework
      - Parallel streams (Java 8+)
      - Best practices for concurrent programming
    Projects built:
      - Parallel file processor
      - Concurrent web crawler
      - Multi-threaded image processor
      - Task scheduler with ExecutorService
      - Parallel array sum calculation
      - Thread-safe cache implementation
      - Concurrent download manager
      - Real-time data processor
    Practice: Build 12 concurrent applications
  Java 8+ Modern Features
    Topics:
      - Lambda expressions: syntax and usage
      - Functional interfaces: Predicate, Function, Consumer, Supplier
      - Method references: static, instance, constructor
      - Streams API: creating streams
      - Intermediate operations: filter, map, flatMap, sorted
      - Terminal operations: collect, forEach, reduce
      - Collectors class: toList, toSet, toMap, groupingBy
      - Optional class: handling null values
      - Date and Time API (java.time package)
      - LocalDate, LocalTime, LocalDateTime
      - ZonedDateTime and time zones
      - Period and Duration
      - Default and static methods in interfaces
      - forEach method with lambda
      - New features in Java 9-17: modules, var keyword, records, sealed classes
    Projects built:
      - Stream operations on collections
      - Employee filtering and sorting with streams
      - Data transformation pipeline
      - Optional handling in applications
      - Date-time calculations and formatting
      - File processing with streams
      - Custom collectors implementation
      - Functional programming utilities
    Practice: Solve 40 lambda and streams problems

Month 6: Design Patterns, JDBC & Testing (Weeks 23-26)
  Design Patterns - Part 1 (Creational)
    Topics:
      - Design patterns introduction and importance
      - Gang of Four (GoF) patterns
      - Pattern categories: creational, structural, behavioral
      - Singleton pattern: one instance
      - Factory pattern: object creation
      - Abstract Factory pattern
      - Builder pattern: complex object construction
      - Prototype pattern: cloning objects
      - When to use each creational pattern
      - Anti-patterns to avoid
      - Real-world pattern applications
    Projects built:
      - Database connection pool (Singleton)
      - Shape factory implementation
      - Document builder with Builder pattern
      - GUI component factory
      - Configuration manager (Singleton)
      - Clone-based object creation
    Practice: Implement all 5 creational patterns in different scenarios
  Design Patterns - Part 2 (Structural & Behavioral)
```

```
  Topics:
    - Adapter pattern: interface adaptation
    - Decorator pattern: adding functionality
    - Proxy pattern: placeholder object
    - Facade pattern: simplified interface
    - Composite pattern: tree structure
    - Observer pattern: event handling
    - Strategy pattern: algorithm selection
    - Command pattern: encapsulating requests
    - Template Method pattern
    - State pattern: state-based behavior
    - Chain of Responsibility pattern
    - Iterator pattern (already in Collections)
    - MVC pattern for applications
  Projects built:
    - Notification system (Observer pattern)
    - Payment strategy implementation
    - Logging framework (Decorator + Singleton)
    - UI component library (Composite)
    - Request processing chain
    - State machine implementation
    - Command-based calculator
  Practice: Implement 15+ design patterns with real examples
JDBC & Database Programming
  Topics:
    - Introduction to JDBC (Java Database Connectivity)
    - JDBC architecture and drivers
    - Driver types: Type 1, 2, 3, 4
    - Loading JDBC driver
    - Establishing database connection
    - Connection interface and methods
    - Statement interface for SQL execution
    - PreparedStatement: parameterized queries
    - CallableStatement: stored procedures
    - ResultSet: processing query results
    - CRUD operations with JDBC
    - Transaction management: commit, rollback
    - Batch processing
    - Connection pooling
    - Database metadata
    - Handling SQL exceptions
  Projects built:
    - Student database management (CRUD)
    - Employee records system with MySQL
    - Product catalog with database
    - User authentication with database
    - Transaction-based banking system
    - Prepared statement utility library
    - Database backup and restore utility
    - Report generation from database
  Practice: Build 10 database-driven applications
Unit Testing with JUnit
  Topics:
    - Introduction to testing: why test?
    - Types of testing: unit, integration, system
    - JUnit framework introduction
    - JUnit 5 (Jupiter) architecture
    - Test annotations: @Test, @BeforeEach, @AfterEach
    - @BeforeAll, @AfterAll
    - Assertions: assertEquals, assertTrue, assertNotNull
    - assertThrows for exception testing
    - Test lifecycle
    - Parameterized tests
    - Test suites
    - Mocking with Mockito
    - Test-Driven Development (TDD)
    - Code coverage with JaCoCo
    - Best practices for unit testing
  Projects built:
    - Unit tests for calculator class
    - Testing collections utilities
    - TDD implementation of stack
    - Testing database operations with mocks
    - Test suite for banking system
    - Parameterized tests for validators
    - Integration tests for application
  Practice: Write tests for all previous projects, achieve 80%+ coverage
  Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing
  this phase with everything before it
```

```
---------------------------------------------------------------
PHASE 3: ENTERPRISE JAVA & WEB DEVELOPMENT (MONTHS 7-9, WEEKS 27-39)
---------------------------------------------------------------
Master enterprise Java development with Servlets, JSP, Spring Framework, Spring Boot,
Hibernate, and RESTful APIs.

  Month 7: Web Development & Spring Framework (Weeks 27-30)
    Servlets & JSP Fundamentals
      Topics:
        - Web application architecture: client-server
        - HTTP protocol: request-response cycle
        - HTTP methods: GET, POST, PUT, DELETE
        - Web servers vs application servers
        - Apache Tomcat setup and configuration
        - Servlets: server-side Java programs
        - Servlet lifecycle: init, service, destroy
        - HttpServlet class and methods
        - doGet() and doPost() methods
        - HttpServletRequest and HttpServletResponse
        - Request parameters and headers
        - Session management: HttpSession
        - Cookies in servlets
        - Request dispatcher: forward and include
        - Servlet configuration: web.xml, annotations
        - JSP (JavaServer Pages) introduction
        - JSP lifecycle and implicit objects
        - JSP directives, declarations, scriptlets, expressions
      Projects built:
        - Hello World servlet
        - Login servlet with session
        - User registration form processing
        - Simple calculator servlet
        - Shopping cart with session
        - File upload servlet
        - JSP-based dynamic web pages
        - Servlet-JSP MVC pattern application
      Practice: Build 10 servlet-based web applications
    Spring Framework Core
      Topics:
        - Introduction to Spring Framework
        - Spring modules and architecture
        - Dependency Injection (DI) concept
        - Inversion of Control (IoC) principle
        - Spring IoC container: BeanFactory, ApplicationContext
        - Bean definition and configuration
        - XML-based configuration
        - Annotation-based configuration: @Component, @Autowired
        - Java-based configuration: @Configuration, @Bean
        - Bean scopes: singleton, prototype, request, session
        - Bean lifecycle: init and destroy methods
        - Autowiring: byName, byType, constructor
        - @Qualifier for disambiguation
        - Dependency injection: constructor, setter, field
        - ApplicationContext events
        - Spring Expression Language (SpEL)
      Projects built:
        - Simple Spring application setup
        - Dependency injection examples
        - Bean configuration practice
        - Spring-based calculator
        - User service with DI
        - Multiple bean configurations
        - Layered architecture with Spring
      Practice: Create 15 Spring DI scenarios
    Spring Boot Fundamentals
      Topics:
        - Introduction to Spring Boot
        - Spring Boot advantages: convention over configuration
        - Spring Initializr: project generation
        - Spring Boot starters
        - Auto-configuration mechanism
        - Application.properties and application.yml
        - Spring Boot main application class
        - @SpringBootApplication annotation
        - Component scanning
        - Spring Boot DevTools
        - Creating REST controllers with @RestController
        - @RequestMapping, @GetMapping, @PostMapping
```

- Path variables and request parameters
- Request body with @RequestBody
- Response entity and HTTP status codes
- Exception handling with @ControllerAdvice
- Custom error responses

Projects built:
- Hello World REST API with Spring Boot
- CRUD REST API for products
- User management REST API
- Todo application backend
- Blog post API
- Student records API
- Exception handling in APIs
- API documentation with Swagger/OpenAPI

Practice: Build 10 REST APIs with Spring Boot

Spring Data JPA & Hibernate

Topics:
- JPA (Java Persistence API) introduction
- ORM (Object-Relational Mapping) concept
- Hibernate as JPA implementation
- Entity classes with @Entity annotation
- Primary keys: @Id, @GeneratedValue
- Column mapping: @Column annotation
- Table mapping: @Table annotation
- Entity relationships: @OneToOne, @OneToMany, @ManyToOne, @ManyToMany
- Cascade types and fetch types (LAZY, EAGER)
- Spring Data JPA repositories
- JpaRepository interface
- CRUD operations with repositories
- Query methods: findBy, findAll, save, delete
- Custom queries with @Query
- JPQL (Java Persistence Query Language)
- Native SQL queries
- Pagination and sorting

Projects built:
- Student database with JPA entities
- Product catalog with Spring Data JPA
- Blog with posts and comments (relationships)
- E-commerce with products and categories
- User and roles management
- Library system with JPA
- Custom query implementations
- Pagination and sorting examples

Practice: Build 10 database applications with Spring Data JPA

Month 8: Advanced Spring Boot & Microservices Basics (Weeks 31-35)

Spring Security & Authentication

Topics:
- Spring Security introduction
- Security filter chain
- Authentication vs authorization
- In-memory authentication
- JDBC authentication
- UserDetailsService for custom authentication
- Password encoding: BCryptPasswordEncoder
- Method-level security: @PreAuthorize, @Secured
- Role-based access control (RBAC)
- JWT (JSON Web Token) authentication
- Creating and validating JWT tokens
- Stateless authentication
- OAuth 2.0 basics
- Social login integration
- CORS configuration in Spring Boot
- CSRF protection

Projects built:
- User registration and login API
- JWT-based authentication system
- Role-based authorization API
- Secured REST API endpoints
- Social login with OAuth
- Password reset functionality
- User profile management with security
- Multi-role application

Practice: Secure all REST APIs with authentication

Spring Boot Advanced Features

Topics:
- Spring Boot Actuator: production-ready features
- Health checks and metrics
- Custom actuator endpoints

    - Application monitoring
    - Logging in Spring Boot: Logback, Log4j2
    - Logging levels and configuration
    - Aspect-Oriented Programming (AOP) in Spring
    - @Aspect, @Before, @After, @Around
    - Cross-cutting concerns: logging, transaction
    - Transaction management: @Transactional
    - Declarative vs programmatic transactions
    - Caching with Spring: @Cacheable, @CacheEvict
    - Redis integration for caching
    - Scheduling tasks: @Scheduled
    - Async processing: @Async
    - Event handling in Spring
    - Profiles for different environments
  Projects built:
    - Application with health monitoring
    - Custom metrics endpoint
    - AOP-based logging system
    - Transaction management in banking app
    - Caching layer implementation
    - Scheduled task executor
    - Async email notification system
    - Multi-environment configuration
  Practice: Add production features to all Spring Boot apps
RESTful API Best Practices & Microservices Intro
  Topics:
    - REST API design principles
    - Resource naming conventions
    - HTTP status codes usage
    - HATEOAS (Hypermedia as the Engine of Application State)
    - API versioning strategies
    - Request validation with @Valid
    - Bean validation annotations
    - API documentation with SpringDoc OpenAPI
    - Swagger UI integration
    - Rate limiting and throttling
    - API security best practices
    - Microservices architecture introduction
    - Monolithic vs microservices
    - Service discovery concept
    - Inter-service communication
    - Introduction to Spring Cloud
  Projects built:
    - RESTful API with HATEOAS
    - Validated REST endpoints
    - Complete API with Swagger docs
    - Versioned API implementation
    - Rate-limited public API
    - Basic microservice setup
    - Service-to-service communication
    - API gateway pattern example
  Practice: Design and document 5 production-ready APIs
Spring Boot with External Services
  Topics:
    - RestTemplate for HTTP clients
    - WebClient for reactive HTTP calls
    - Consuming third-party APIs
    - Feign client for declarative REST
    - Email integration: JavaMailSender
    - Sending HTML emails
    - File upload and storage
    - AWS S3 integration
    - Message queues: RabbitMQ basics
    - Apache Kafka introduction
    - Event-driven architecture
    - WebSocket support in Spring Boot
    - Real-time communication
    - Payment gateway integration (Stripe, Razorpay)
    - SMS integration
    - Push notifications
  Projects built:
    - Weather API consumer
    - Email notification service
    - File upload to cloud storage
    - Message queue producer-consumer
    - Real-time chat application
    - Payment processing system
    - SMS alert service
    - Event-driven order processing

Practice: Integrate 8 external services in applications
Testing Spring Boot Applications
Topics:
- Spring Boot testing overview
- @SpringBootTest annotation
- Test slicing: @WebMvcTest, @DataJpaTest
- MockMvc for testing controllers
- Mocking with @MockBean
- Testing REST APIs
- Integration testing with database
- TestContainers for database testing
- Testing security configurations
- Testing with different profiles
- JUnit 5 with Spring Boot
- AssertJ for fluent assertions
- Test coverage with JaCoCo
- Performance testing basics
- Contract testing introduction
Projects built:
- Complete test suite for REST API
- Controller layer tests
- Service layer tests with mocks
- Repository layer tests
- Integration tests for application
- Security tests
- End-to-end testing
- Test-driven development practice
Practice: Write comprehensive tests for all Spring Boot projects

Month 9: Production Java: Deployment, Monitoring & Security (Weeks 36-39)
Database Design & Optimization
Topics:
- Relational database design principles
- Normalization: 1NF, 2NF, 3NF, BCNF
- Entity-Relationship diagrams
- Database schema design
- Index optimization in JPA
- Query optimization techniques
- N+1 query problem and solutions
- Database connection pooling: HikariCP
- Transaction isolation levels
- Optimistic vs pessimistic locking
- Database migration with Flyway/Liquibase
- NoSQL with Spring Boot: MongoDB
- Spring Data MongoDB
- Choosing SQL vs NoSQL
- Multi-database applications
Projects built:
- Optimized database schema design
- Migration scripts with Flyway
- Performance-tuned JPA queries
- MongoDB integration
- Connection pool configuration
- Database performance monitoring
- Multi-database Spring Boot app
Practice: Optimize databases for all projects
Application Deployment & DevOps
Topics:
- Packaging Spring Boot applications
- JAR vs WAR deployment
- Application properties for production
- Externalized configuration
- Environment-specific configurations
- Deploying to Tomcat server
- Docker basics review
- Dockerizing Spring Boot applications
- Docker Compose for multi-container
- Docker best practices for Java
- Cloud platforms: AWS, Azure, GCP
- Deploying to AWS Elastic Beanstalk
- Deploying to Render
- CI/CD for Spring Boot with Jenkins
- GitHub Actions for Java projects
Projects built:
- Production-ready application configuration
- Dockerized Spring Boot app
- Multi-container deployment
- Cloud deployment practice
- CI/CD pipeline setup

```
        - Automated deployment workflow
        - Blue-green deployment example
      Practice: Deploy all applications to production environments
    Monitoring, Logging & Performance
      Topics:
        - Application monitoring strategies
        - Spring Boot Actuator deep dive
        - Micrometer metrics
        - Prometheus integration
        - Grafana dashboards
        - Distributed tracing with Sleuth and Zipkin
        - Centralized logging with ELK stack
        - Log aggregation strategies
        - Performance profiling tools
        - JVM monitoring and tuning
        - Garbage collection optimization
        - Memory leak detection
        - Load testing with JMeter
        - API performance optimization
        - Database query profiling
      Projects built:
        - Monitoring dashboard setup
        - Metrics collection and visualization
        - Distributed tracing implementation
        - Centralized logging system
        - Performance optimization project
        - Load testing suite
        - JVM tuning exercise
      Practice: Add monitoring to all production applications
    Security Best Practices & API Standards
      Topics:
        - OWASP Top 10 for Java applications
        - SQL injection prevention in JPA
        - XSS (Cross-Site Scripting) prevention
        - CSRF protection implementation
        - Secure password handling
        - Secrets management
        - API key management
        - Rate limiting implementation
        - Input validation best practices
        - Secure communication: HTTPS/TLS
        - Security headers configuration
        - Dependency vulnerability scanning
        - Static code analysis: SonarQube
        - Security testing
        - Compliance: GDPR, PCI-DSS basics
      Projects built:
        - Security-hardened application
        - Vulnerability assessment tool
        - Secure API implementation
        - Security testing suite
        - Code quality dashboard
      Practice: Security audit all projects
      Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

----------------------------------------------------------------
PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)
----------------------------------------------------------------
The capstone month: spec, build, ship and defend, then the final exam that decides the
certificate.

  Month 10: Capstone and the Final Exam (Weeks 40-44)
    Capstone Spec and Setup
      Topics:
        - Choosing the capstone: a Spring Boot product API: bookings, a commerce backend or a
          real-world service with security and tests
        - A one-page spec: scope, milestones, stack
        - Repository and skeleton ready on day one
        - Instructor sign-off before the build begins
      Projects built:
        - Approved capstone spec plus the running skeleton
      Practice: Pitch the spec to the batch and absorb one hard question
    Capstone Build
      Topics:
        - Two focused weeks in vertical slices
        - Instructor checkpoints and honest scope cuts
        - Using the whole course: core Java depth, collections and concurrency judgment, Spring Boot,
          JPA and security
```

- Daily commits and an always-working build
Projects built:
- The capstone, feature-complete with version control
Practice: A working build at the end of every session
Ship and Polish
Topics:
- The finishing pass: edge cases, errors, documentation
- Packaged and deployed: a Spring Boot artifact running outside the IDE
- A README worth reading and a demo worth watching
- A full-course spiral review before the exam
Projects built:
- The capstone shipped, documented and rehearsed
Practice: Two mock demos, each tighter than the last
Demo Day and the Final Exam
Topics:
- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: backend engineering, microservices when a system earns them, DSA for placements
- Where our other courses continue the journey
Projects built:
- Demo day presentation delivered to the batch
Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: console programs that prove core Java and OOP
- Phase 2: concurrent, tested builds using collections and patterns for real
- Phase 3: Spring Boot REST APIs with JPA, security and tests
- Phase 4: the Spring Boot capstone, deployed and defended

Total projects: 20+ real builds, crowned by a deployed Spring Boot capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Core Java at placement depth: OOP, collections, generics, exceptions
- Concurrency understood: threads, the memory model and modern utilities
- Modern Java fluency: streams, lambdas, records and current LTS habits
- Design patterns and JDBC: the thinking under every framework
- Spring Boot in practice: REST APIs, Spring Data JPA, security, testing

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Backend roles: Java plus Spring Boot remains the enterprise hiring staple
- Our DSA course beside this one is the classic placement combination
- Microservices and cloud depth later, once real systems demand them
- Kotlin and Android: our Kotlin course builds on exactly this foundation

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 3-5 LPA (Junior Java Developer / Fresher)
Growing roles: Rs 5-9 LPA (Java Developer); Rs 8-15 LPA (Senior Java Developer / Spring Boot Developer)
Experienced roles: Rs 12-25 LPA (Senior Java / Microservices Developer / Tech Lead); Rs 18-40

    LPA (Lead Developer / Architect)
  Freelance: Rs 1,000-5,000/hour based on expertise
  International usa: $80k-180k USD (Java Developer to Senior)
  International europe: €50k-120k EUR based on country and experience

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What does this Java course cover?
  A: Core Java to real depth first: object-oriented programming, collections, generics,
  exceptions, multithreading and modern Java features, plus design patterns, JDBC and JUnit
  testing. Then the job stack: Spring Boot, Spring Data JPA with Hibernate, Spring Security,
  REST API design and testing, and production habits like deployment, monitoring and logging. A
  finale month ships a Spring Boot capstone and sits the final exam.

  Q: How long does it take to learn Java properly?
  A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two
  more months in hand for learners who need them. The three teaching phases build the skill and
  its real-world application; the finale month ships a capstone and sits the final exam.

  Q: How am I assessed? Is the certificate just for attending?
  A: No. Problem sets follow every class with written feedback, monthly reviews deliberately
  mix new material with earlier topics, each phase ends with a cumulative exam, and the course
  ends with a final exam: a practical build plus written questions from the whole syllabus.
  Passing earns the certificate; falling short earns focused revision and a free retest.

  Q: Is Java still worth learning in 2026?
  A: Emphatically: banking, e-commerce, telecom and most of enterprise India run on Java,
  placements test it more than any other language, and modern Java (current LTS releases,
  streams, records, virtual threads) is a far better language than its reputation. Python took
  the headlines; Java kept the backends.

  Q: Java or Python first for placements?
  A: Either works for DSA rounds, but Java pays twice: the same language covers your DSA
  practice and the Spring Boot backend skills that service and product companies actually hire
  for. If your target is backend or enterprise roles, Java first is the efficient road; our
  Python courses cover the data and automation roads.

  Q: Can I try a class before enrolling?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or
  message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with
  no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at
  https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor reading your actual code, never
  pre-recorded videos.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/complete-java-programming-masterclass-college
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
COURSE 88 OF 137: JAVA FOR TEENS: COMPLETE COURSE FROM BEGINNER TO ADVANCED
==

Course link: https://learn.modernagecoders.com/courses/java-programming-masterclass-for-teens
Subject area: Java Programming
Designed for: Teens
Who can join: Absolute beginner, ages 13 to 18
Course length: 10-12 months (40-48 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one
  live class a week, four a month; one to one outside India two live classes a week, eight a
  month. Each class is one hour.

WHAT THIS COURSE IS

  Java course for teens aged 13-18. Live online classes: core Java, JavaFX apps, games,
  Minecraft mods, data structures and a Spring Boot intro, in 10-12 months.

In one line: The language behind Minecraft, Android and the systems banks run on, taught to teenagers properly: strict enough to build discipline, fun enough to keep them building.

ABOUT THE PROGRAMME

Java is the language schools trust and industry runs on: ICSE, ISC and AP Computer Science A all teach it, Minecraft is written in it, and enterprise systems everywhere depend on it. This live online course takes students aged 13 to 18 from their first line of code to genuinely advanced Java, and its strict, typed nature quietly teaches the discipline that makes every later language easier. The first three months build the foundations: variables to loops to methods, then object-oriented programming taught the way Java demands, with classes, inheritance, interfaces and real design decisions. The middle months make Java fun on screen: the collections framework and modern streams, desktop apps with JavaFX, a complete 2D game, and Minecraft modding, the project every Java-learning teen secretly wants. The final months go where serious Java students go: data structures and algorithms built from scratch, a threaded program that does two things at once, a first REST API with Spring Boot, calling modern AI models from Java, and a capstone shipped with Git, tests and a demo. The pace is honest: two live classes a week plus three to four hours of practice fits the syllabus in ten months, and students who need more time take up to twelve, the class adapts instead of rushing ahead. Learning is checked the honest way too: homework after every class, monthly assignments that mix new questions with earlier topics, and a final exam that decides the certificate. Real knowledge, real work, real projects, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Aligned with school Java: ICSE, ISC and AP Computer Science A students find class work and school work reinforcing each other
- Minecraft modding is in the syllabus, not a teaser: students build and run their own mods in month 6
- Object-oriented programming taught as design, not syntax: students learn why classes exist, not just how to type them
- Data structures and algorithms built from scratch, the preparation serious CS paths actually require
- An honest 10-month arc with 2 months of margin built in: students who need more practice take up to 12, nobody is rushed
- Real assessment: weekly homework, mixed-review assignments every month, and a final exam that decides the certificate, with focused revision and a free retest for students who fall short

THE LEARNING PATH, STEP BY STEP

Phase 1: Java foundations (Months 1-3): first programs, arrays, methods, strings, files, then object-oriented programming done properly
Phase 2: Java you can see (Months 4-6): collections and streams, JavaFX desktop apps, a complete 2D game, and Minecraft modding
Phase 3: Data structures and algorithms (Months 7-8): the classic structures built by hand, sorting, graphs, recursion and multithreading
Phase 4: The wider world (Months 9-10): a REST API with Spring Boot, AI APIs from Java, Git and JUnit, and the final capstone

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The course starts from the very first line of code
Computer skills: Comfortable typing and managing files and folders
Equipment: A computer with a stable internet connection; the JDK, IntelliJ IDEA and every library used are free
Age: 13 to 18. Batches are grouped by age and pace so the room always fits
Time commitment: 2 live classes plus 3-4 hours of practice a week

WHO THIS COURSE IS FOR

School java students: ICSE, ISC and AP Computer Science A students who want to be ahead of the syllabus, not chasing it
Complete beginners: Teens starting from zero who want a strict, structured language that builds real discipline
Minecraft fans: Students who have always wanted to change the game itself, not just play it
Future engineers: Teens eyeing engineering and computer science degrees where Java and DSA are the entry gate
Serious builders: The kind of student who wants to understand how software works underneath, not just make it run once

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: JAVA FOUNDATIONS AND OOP (MONTHS 1-3, WEEKS 1-12)

From the first program to real object-oriented design. Java's strictness becomes an advantage: every concept is understood, not guessed.

Month 1: First Programs (Weeks 1-4)
  Introduction and Setup
    Topics:
      - What is programming? Why Java? Real-world Java applications
      - How computers understand code: compilation and execution
      - Java history: from coffee to code that runs everywhere
      - Installing Java JDK and understanding versions
      - Setting up IntelliJ IDEA (student edition)
      - Your first Java program: Hello World explained
      - Understanding main method and program structure
      - Basic program flow and execution
      - Using the command line to compile and run Java
      - Introduction to debugging and reading error messages
      - Code comments and documentation basics
      - Programming mindset: how to think like a developer
    Projects built:
      - Set up complete Java development environment
      - Create and run Hello World in 3 different ways
      - Build a simple personal introduction program
    Practice: Short daily exercises: 15 small programs in the first week
  Variables, Types and Operations
    Topics:
      - Variables: storing information in programs
      - Primitive data types: int, double, boolean, char
      - Declaring and initializing variables
      - Naming conventions and best practices
      - Arithmetic operators: +, -, *, /, %
      - Order of operations (PEMDAS in programming)
      - Type casting and conversion
      - String basics: creating and concatenating
      - Reading user input with Scanner class
      - Formatting output with printf
      - Constants and final keyword
      - Common programming errors and how to fix them
    Projects built:
      - Interactive calculator application
      - Temperature converter (Celsius/Fahrenheit/Kelvin)
      - Simple cashier program with tax calculation
      - Mad Libs story generator
    Practice: Solve 25 problems on variables and operations
  Control Flow: Making Decisions
    Topics:
      - Boolean logic: true and false
      - Comparison operators: ==, !=, <, >, <=, >=
      - If statements: making decisions in code
      - If-else chains for multiple conditions
      - Nested if statements
      - Logical operators: && (AND), || (OR), ! (NOT)
      - Switch statements for multiple choices
      - Ternary operator for simple conditions
      - Program flow visualization
      - Debugging conditional logic
      - Common logic errors and edge cases
      - Writing readable conditional code
    Projects built:
      - Grade calculator with letter grades
      - Simple adventure game with choices
      - Rock-Paper-Scissors game
      - Age-appropriate movie recommendation system
    Practice: Build 10 programs using different conditional patterns
  Loops: Repetition and Patterns
    Topics:
      - Why loops? Automation and efficiency
      - While loops: repeat while condition is true
      - Do-while loops: execute at least once
      - For loops: counting and iteration
      - Loop control: break and continue
      - Nested loops: loops within loops
      - Common loop patterns
      - Infinite loops and how to avoid them
      - Enhanced for loop (for-each) preview
      - Loop performance considerations
      - Debugging loops effectively
      - Accumulator and counter patterns
    Projects built:
      - Number guessing game with attempts
      - Multiplication table generator
      - Pattern printer (pyramids, diamonds)

```
      - Simple password validator with retry
      - Prime number checker
    Practice: Solve 25 loop-based programming challenges
    Assessment: Month 1 check: build a small menu-driven program live, plus a take-home
    assignment mixing everything from month 1

Month 2: Arrays, Methods, Strings and Files (Weeks 5-8)
  Arrays
    Topics:
      - Introduction to arrays: storing multiple values
      - Declaring and initializing arrays
      - Accessing array elements with indices
      - Array length and bounds checking
      - Iterating through arrays with loops
      - Common array operations: sum, average, max, min
      - Searching arrays: linear search
      - Sorting arrays: bubble sort basics
      - Two-dimensional arrays (matrices)
      - Arrays of strings
      - ArrayIndexOutOfBoundsException
      - Arrays class utility methods
    Projects built:
      - Grade book system for a class
      - Tic-Tac-Toe game board
      - Student roster management
      - Simple playlist manager
      - Basic statistics calculator
    Practice: Master 25 array manipulation problems
  Methods: Reusable Code
    Topics:
      - What are methods? Why use them?
      - Method structure: return type, name, parameters
      - Void methods vs methods with return values
      - Passing arguments to methods
      - Method overloading: same name, different parameters
      - Scope and local variables
      - Pass by value in Java
      - Recursive methods introduction
      - Method documentation with Javadoc
      - Testing methods independently
      - Helper methods and code organization
      - Static vs instance methods preview
    Projects built:
      - Calculator with method for each operation
      - Text analysis toolkit (word count, etc.)
      - Geometry calculator library
      - Simple encryption/decryption program
      - Menu-driven banking application
    Practice: Create 15 useful utility methods
  Strings and Text Processing
    Topics:
      - String class deep dive
      - String immutability concept
      - Common String methods: length, charAt, substring
      - String comparison: equals vs ==
      - Case conversion: toUpperCase, toLowerCase
      - Searching strings: indexOf, contains
      - Replacing and splitting strings
      - StringBuilder for efficient string building
      - Character class utilities
      - Regular expressions basics
      - Parsing strings to numbers
      - Formatting strings for output
    Projects built:
      - Password strength analyzer
      - Palindrome checker
      - Word game (like Wordle)
      - Text-based email validator
      - Simple markdown to HTML converter
    Practice: Complete 20 string manipulation challenges
  Files and Exceptions
    Topics:
      - Reading from text files
      - Writing to text files
      - File class and file operations
      - Scanner for file input
      - PrintWriter for file output
      - Try-catch blocks for error handling
      - Common exceptions: IOException, FileNotFoundException
```

```
      - Finally block usage
      - Throwing exceptions
      - Creating custom exceptions preview
      - Best practices for file handling
      - Resource management basics
    Projects built:
      - Note-taking application with file storage
      - High score system for games
      - Simple database using text files
      - Log file analyzer
      - CSV file reader/writer
    Practice: Add file storage to three earlier projects
    Assessment: Month 2 check: build a program that loads, changes and saves its data, with
    quiz questions reaching back to month 1

Month 3: Object-Oriented Programming (Weeks 9-12)
  Classes and Objects
    Topics:
      - Object-Oriented Programming concepts
      - Real-world objects vs programming objects
      - Creating your first class
      - Instance variables (attributes/fields)
      - Methods (behaviors)
      - Constructors: default and parameterized
      - Creating objects with 'new' keyword
      - The 'this' keyword
      - Getters and setters (accessors/mutators)
      - toString() method
      - Object references vs primitive values
      - Garbage collection basics
    Projects built:
      - Student class with grade management
      - Bank account class with transactions
      - Book library system
      - Simple pet simulator
      - Car dealership inventory
    Practice: Model 10 real-world things as classes
  Encapsulation and Static
    Topics:
      - Information hiding principle
      - Access modifiers: public, private, protected, default
      - Encapsulation best practices
      - Validation in setters
      - Immutable objects
      - Static variables and methods
      - Static vs instance context
      - Class constants with static final
      - Package organization
      - Import statements and classpath
    Projects built:
      - Secure user account system
      - Inventory management with validation
      - Employee management system
      - Game character with stats
      - Shopping cart with product catalog
    Practice: Refactor earlier projects with proper encapsulation
  Inheritance
    Topics:
      - Inheritance concept: is-a relationship
      - Extending classes with 'extends'
      - Super keyword and parent constructors
      - Method overriding
      - @Override annotation
      - Protected access modifier
      - Multi-level inheritance
      - Object class as universal parent
      - equals() and hashCode() methods
      - Abstract classes introduction
      - When to use inheritance vs composition
      - Inheritance best practices
    Projects built:
      - Animal hierarchy with different species
      - Vehicle inheritance system
      - Employee hierarchy (Manager, Developer, etc.)
      - Shape hierarchy for geometry
      - Game entity system
    Practice: Design 5 inheritance hierarchies and defend each design choice
  Polymorphism and Interfaces
    Topics:
```

- Polymorphism: many forms
- Method overriding for polymorphism
- Dynamic method dispatch
- Abstract classes and methods
- Interfaces: contracts for classes
- Implementing multiple interfaces
- Interface default methods (Java 8+)
- Comparable and Comparator interfaces
- Polymorphic arrays and collections
- instanceof operator
- Type casting with objects
- Interface vs abstract class

Projects built:
- Payroll system with different employee types
- Media player for different file types
- Drawing application with shapes
- RPG game with character classes
- Plugin system using interfaces

Practice: Implement 8 interfaces across your own projects
Assessment: Phase 1 exam: build a small OOP system live from requirements and explain the design, with cumulative questions from months 1-3

PHASE 2: COLLECTIONS, APPS, GAMES AND MINECRAFT (MONTHS 4-6, WEEKS 13-24)

Modern Java and things you can see: the collections framework, desktop apps with JavaFX, a complete 2D game, and your own Minecraft mods.

Month 4: Collections and Modern Java (Weeks 13-16)

The Collections Framework

Topics:
- Collections vs arrays: when to use what
- Collection interface hierarchy
- List interface: ArrayList vs LinkedList
- Set interface: HashSet, TreeSet, LinkedHashSet
- Map interface: HashMap, TreeMap, LinkedHashMap
- Queue and Deque interfaces
- Iterator and ListIterator
- Collections utility class
- Sorting collections with Comparable/Comparator
- Searching and shuffling
- Performance characteristics of collections

Projects built:
- Contact management system with HashMap
- Task scheduler with PriorityQueue
- Duplicate finder using Sets
- Student database with sorting
- Cache implementation with LinkedHashMap

Practice: Master 25 collection manipulation problems

Generics

Topics:
- Why generics? Type safety benefits
- Generic classes creation
- Generic methods
- Bounded type parameters
- Wildcards: ?, extends, super
- Generic interfaces
- Multiple type parameters
- Generic collections deep dive
- Best practices with generics
- Common generic patterns

Projects built:
- Generic data structure library
- Type-safe cache implementation
- Generic DAO pattern
- Custom generic collections
- Builder pattern with generics

Practice: Create 10 generic classes and methods

Lambdas and Streams

Topics:
- Functional interfaces concept
- Lambda expression syntax
- Method references
- Built-in functional interfaces
- Stream API introduction
- Creating streams from collections
- Intermediate operations: filter, map, sorted
- Terminal operations: collect, reduce, forEach
- Parallel streams

```
      - Optional class for null safety
      - Stream performance considerations
      - Functional programming in Java
    Projects built:
      - Data processing pipeline
      - Report generator with streams
      - Functional calculator
      - Stream-based file analyzer
      - Employee data analysis system
    Practice: Solve 20 problems using streams and lambdas
  Exceptions and Debugging Like a Professional
    Topics:
      - Exception hierarchy in Java
      - Checked vs unchecked exceptions
      - Try-catch-finally deep dive
      - Multi-catch blocks
      - Try-with-resources statement
      - Creating custom exception classes
      - Exception chaining
      - Best practices for exception handling
      - Logging with java.util.logging
      - Debugging techniques in IDE
      - Using breakpoints effectively
      - Stack traces and error analysis
    Projects built:
      - Robust file processing system
      - Input validation framework
      - Error handling library
      - Retry mechanism implementation
      - Logging system for applications
    Practice: Handle exceptions in 15 different scenarios
    Assessment: Month 4 check: a debugging gauntlet plus a mixed assignment reaching back
    to phase 1

Month 5: Desktop Apps with JavaFX (Weeks 17-20)
  JavaFX Fundamentals
    Topics:
      - JavaFX vs Swing: modern GUI development
      - JavaFX application structure
      - Stage, Scene, and Scene Graph
      - Layouts: VBox, HBox, GridPane, BorderPane
      - Basic controls: Button, Label, TextField
      - Event handling in JavaFX
      - CSS styling for JavaFX
      - FXML for UI design
      - Scene Builder tool
      - Images and media in JavaFX
      - Menus and toolbars
      - Dialogs and alerts
    Projects built:
      - Calculator with GUI
      - Note-taking application
      - Login screen with validation
      - Image viewer application
      - Simple text editor
    Practice: Build 8 different GUI layouts
  Tables, Charts and Real Controls
    Topics:
      - TableView for data display
      - TreeView for hierarchical data
      - ListView and observableList
      - ComboBox and ChoiceBox
      - DatePicker and ColorPicker
      - Charts: PieChart, BarChart, LineChart
      - WebView for HTML content
      - Custom controls creation
      - Binding properties
      - Animations and transitions
      - Drag and drop functionality
      - Keyboard shortcuts
    Projects built:
      - Student database with TableView
      - File explorer with TreeView
      - Data visualization dashboard
      - Media player interface
      - Calendar application
    Practice: Build 3 data-driven screens with tables and charts
  Structuring a Real App with MVC
    Topics:
```

- Model-View-Controller pattern
- Separating concerns in JavaFX
- Observable pattern in JavaFX
- Data binding deep dive
- Application state management
- Navigation between screens
- Application packaging
- Keeping logic out of the UI: the habit that saves every project

Projects built:
- Complete MVC application
- Multi-screen navigation app
- Inventory management system
- Point of Sale (POS) system
- Library management GUI

Practice: Refactor one earlier app into clean MVC

Databases for Desktop Apps

Topics:
- JDBC basics and drivers
- Connecting to databases
- SQL review for Java developers
- PreparedStatement vs Statement
- ResultSet processing
- Transaction management
- DAO pattern with JDBC
- Database design basics
- SQLite for desktop apps

Projects built:
- Database-backed todo application
- Customer management system
- Inventory with database
- Expense tracker with reports
- Student information system

Practice: Add SQLite storage to two of your apps

Assessment: Month 5 check: present a working database-backed desktop app, plus a mixed quiz over months 1-5

Month 6: Games and Minecraft Modding (Weeks 21-24)

Game Development Basics

Topics:
- Game loop concept
- Sprite handling and animation
- Collision detection
- Game physics basics
- Sound effects and music
- Game state management
- Score and lives system
- Level design principles
- 2D graphics with Canvas
- Input handling for games
- Game optimization techniques
- Packaging games for distribution

Projects built:
- Snake game implementation
- Breakout/Brick breaker game
- Simple platformer
- Tower defense game
- Puzzle game like 2048

Practice: Build two tiny game prototypes between classes

Build a Complete 2D Game

Topics:
- Choosing your game: Snake, Breakout or a simple platformer
- Structuring the game with the OOP you now own
- Levels, difficulty and progression
- Sound effects, scoring and lives done cleanly
- Menus, pause and game-over screens
- Saving high scores to a file
- Playtesting: watching someone else play, then fixing what confused them
- Packaging the game so a friend can run it

Projects built:
- A complete, finished 2D game with menus, sound, scoring and a shareable build

Practice: Polish pass driven by two playtesters' honest feedback

Minecraft Modding

Topics:
- Minecraft Forge setup
- Mod structure and basics
- Creating blocks and items
- Crafting recipes
- Custom entities
- World generation

        - GUI creation
        - Mod configuration
        - Testing and debugging
        - Forge and Fabric: the two modding worlds, explained
      Projects built:
        - Your first mod: custom items and blocks that load in your own game
        - A crafting-recipe mod with its own logic
        - A custom tool or gadget mod
      Practice: Design and build one mod idea of your own
    Phase 2 Capstone and Review
      Topics:
        - Choosing your phase capstone: game, desktop app or Minecraft mod, taken further
        - Refactoring old code with new skills
        - Code review: reading each other's projects kindly and usefully
        - A targeted review week shaped by what the batch found hard
        - Writing project descriptions people understand
        - What phase 3 changes: from building things to building fast things
      Projects built:
        - Phase capstone: your best build, taken one honest level further
      Practice: Publish your top 3 projects in a personal portfolio folder
      Assessment: Phase 2 review: capstone walkthrough plus a written mixed review spanning
      phases 1 and 2

PHASE 3: DATA STRUCTURES AND ALGORITHMS (MONTHS 7-8, WEEKS 25-32)

The serious core of computer science, built by hand in Java: structures, sorting, graphs,
recursion, and a first look at threads.

  Month 7: Structures Built from Scratch (Weeks 25-28)
    Lists, Stacks and Queues by Hand
      Topics:
        - Abstract Data Types (ADT)
        - Implementing dynamic arrays
        - Linked Lists: single, double, circular
        - Stack implementation and applications
        - Queue implementation and variants
        - Priority queues and heaps
      Projects built:
        - Custom ArrayList implementation
        - Expression evaluator using stacks
        - Task scheduler with a priority queue
      Practice: Implement a dynamic array, linked list, stack and queue from scratch
    Hash Tables and Trees
      Topics:
        - Hash tables from scratch
        - Collision resolution strategies
        - Trees: binary trees basics
        - Tree traversal: inorder, preorder, postorder
        - Binary search trees
        - Balanced trees introduction
      Projects built:
        - Spell checker with your own hash table
        - File system simulator with trees
        - Autocomplete with a simple trie
      Practice: Implement a hash map and a binary search tree, then test them against Java's
    Sorting, Searching and Big O
      Topics:
        - Linear search and optimizations
        - Binary search and variations
        - Bubble sort and optimizations
        - Selection sort
        - Insertion sort
        - Merge sort and divide-conquer
        - Quick sort and partitioning
        - Heap sort
        - Counting sort and radix sort
        - Sorting stability
        - Time complexity analysis
        - Space complexity
      Projects built:
        - Sorting visualizer application
        - Search engine for text files
        - Music playlist sorter
        - Leaderboard system
        - Data analysis tool
      Practice: Implement and analyze 10 classic algorithms
    Problem-Solving Patterns
      Topics:

```
        - Reading a problem statement like a solver, not a victim
        - Frequency counting with maps: the pattern behind dozens of problems
        - Two pointers and sliding windows
        - Prefix sums for range questions
        - Choosing the right structure: the decision tree
        - Estimating before coding: will this even run in time?
        - Practicing on HackerRank and LeetCode without drowning
        - Where our dedicated DSA course goes further
      Projects built:
        - A personal patterns notebook with one solved example per pattern
      Practice: Solve 20 problems, tagging each with the pattern it used
      Assessment: Month 7 check: a timed three-problem set, plus mixed questions from earlier
      phases

  Month 8: Graphs, Recursion and Threads (Weeks 29-32)
    Graphs
      Topics:
        - Graph representations: adjacency matrix/list
        - Breadth-first search (BFS)
        - Depth-first search (DFS)
        - Shortest path: Dijkstra's algorithm
        - Minimum spanning tree
        - Topological sorting
        - Modeling real problems as graphs: maps, friendships, prerequisites
      Projects built:
        - Social network analyzer
        - Route finder on a real map grid
        - Course prerequisite checker with topological sort
      Practice: Solve 12 graph problems, drawing each before coding it
    Recursion and Backtracking
      Topics:
        - Recursion re-learned properly: trust the function
        - The call stack, visualized
        - Classic recursion: factorial to file-tree walker
        - Backtracking: exploring choices and undoing them
        - Solving mazes by backtracking
        - Sudoku solver: the famous application
        - Memoization: remembering answers you already computed
        - When recursion is the wrong tool
      Projects built:
        - Maze solver with visualization
        - Sudoku solver
        - Word search puzzle solver
      Practice: Solve 15 recursion and backtracking problems
    Multithreading, the Honest Introduction
      Topics:
        - Thread concept and lifecycle
        - Creating threads: Thread class vs Runnable
        - Thread synchronization basics
        - Synchronized keyword
        - Deadlock and how to avoid it
        - Thread pools introduction
        - Callable and Future
        - Concurrent collections
        - Thread safety principles
        - Common threading problems
      Projects built:
        - Multi-threaded download manager
        - Producer-consumer implementation
        - Concurrent counter application
        - Thread pool executor demo
        - Race condition simulator
      Practice: Build 5 small programs that genuinely need two threads
    DSA Capstone
      Topics:
        - Choosing the capstone: sorting visualizer, pathfinding demo or game leaderboard engine
        - Designing before coding: structures chosen for reasons
        - Benchmarking your structure against Java's built-ins
        - Presenting a technical project to a non-technical audience
        - A cumulative review of the whole DSA phase
      Projects built:
        - DSA capstone: an interactive visualizer or engine that shows an algorithm working
      Practice: Write a one-page explanation of your capstone's core algorithm
      Assessment: Phase 3 exam: implement a chosen structure live plus a written paper mixing
      questions from all three phases

---------------------------------------------------------------
PHASE 4: APIS, AI AND THE CAPSTONE (MONTHS 9-10, WEEKS 33-40)
---------------------------------------------------------------
```

Where teen Java meets the real world: a REST API with Spring Boot, AI models called from Java, Git and JUnit, and a final capstone shipped properly.

- Month 9: APIs and AI (Weeks 33-36)
  - How the Web Works, for Java Developers
    - Topics:
      - How the web works: HTTP protocol
      - Client-server architecture
      - Request-response cycle
      - HTTP methods: GET, POST, PUT, DELETE
      - Status codes and headers
      - HTML/CSS basics for Java developers
      - JSON and XML data formats
      - RESTful API principles
      - Java's built-in HttpClient: calling real APIs
    - Projects built:
      - HTTP client that talks to a public API
      - JSON parser for a real API response
      - Weather or trivia terminal app powered by an API
    - Practice: Consume 3 public APIs from plain Java
  - Your First REST API with Spring Boot
    - Topics:
      - What Spring Boot is, and why the industry runs on it
      - Spring Initializr: a real project in two minutes
      - @RestController: your first endpoints
      - GET and POST: reading and creating data
      - Path variables and request parameters
      - Returning JSON objects from Java classes
      - application.properties and sensible configuration
      - Testing your API with a REST client
      - An honest map of what enterprise Spring adds later
    - Projects built:
      - A working REST API for a hobby collection: books, games or trading cards
    - Practice: Design and build a small API of your own from scratch
  - Saving Data: JPA Without the Jargon
    - Topics:
      - JPA and Hibernate basics
      - Entity mapping with annotations
      - @Entity, @Table, @Column
      - Primary keys and generation strategies
      - Relationships: @OneToMany, @ManyToOne
      - Spring Data repositories
      - Query methods
      - H2: a database that lives inside your app while you learn
    - Projects built:
      - Your hobby API upgraded: data survives restarts
      - A two-entity model with a real relationship
    - Practice: Model and persist 3 small domains end to end
  - Calling AI Models from Java
    - Topics:
      - Calling a modern AI model from Java (accounts set up with a parent where required)
      - Structured output: making the model return real objects
      - Function calling and tool use in a typed language
      - Retrieval over enterprise data
      - Concurrency: many AI calls without melting the server
      - Guardrails and validating model output
      - Cost, caching and quotas
      - Where Java is actually used for AI (and where Python wins)
      - Testing non-deterministic systems
      - Ethics and accountability in automated decisions
    - Projects built:
      - AI-powered study helper API endpoint
      - Structured-output parser that turns model text into Java objects
      - A small tool-using AI assistant
    - Practice: Build 3 AI-powered features and note where each one is confidently wrong
    - Assessment: Month 9 check: a mixed assignment spanning APIs, persistence and AI, with questions revisiting earlier phases

- Month 10: Quality, Capstone and Demo Day (Weeks 37-40)
  - Git, GitHub and JUnit
    - Topics:
      - Version control: why every developer uses it
      - The core Git loop: status, add, commit, log
      - Branches: experimenting without fear
      - GitHub: publishing your portfolio
      - READMEs that make a project understandable in a minute
      - JUnit 5: your first real test suite
      - Writing testable methods: the design pressure tests create
      - Debugger over print statements: breakpoints as a way of thinking

- Where our full Git and GitHub course goes further
Projects built:
- Course portfolio pushed to GitHub with clean READMEs
- JUnit test suite for your API's core logic
Practice: Commit daily for two weeks; write 20 tests across your best projects
Final Capstone Build
Topics:
- Choosing the capstone: a polished game, a JavaFX product, a Minecraft mod pack with real logic, or an API-backed app
- Writing a one-page spec before any code
- Milestones: working core by day 4, feature-complete by day 10
- Instructor checkpoints and course corrections
- Using everything: OOP, collections, DSA judgment, tests, Git
- Cutting scope without cutting quality
- Preparing the demo: story, not feature list
Projects built:
- The final capstone, built over two weeks with version control, JUnit tests on core logic, and a README worth reading
Practice: Daily commits; a working build at the end of every session
Demo Day and What Comes Next
Topics:
- Presenting the capstone: live demo plus the hardest bug story
- Portfolio final pass: GitHub profile, project pages, screenshots
- An honest skills inventory: strong, shaky, next
- Where each path leads: DSA deep-dive, Android via Kotlin, full-stack web, or school board excellence
- How ICSE, ISC and AP Computer Science A map to what you now know
- Staying sharp: projects, communities and healthy practice habits
Projects built:
- Demo day presentation delivered to the batch and parents
Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; a student who falls short gets a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: calculators, text games, a grade book, file-backed apps, a full OOP system built from requirements
- Phase 2: JavaFX desktop apps with tables and charts, a complete 2D game, and working Minecraft mods
- Phase 3: hand-built data structures, a sorting or pathfinding visualizer, and threaded programs
- Phase 4: a REST API with persistence, AI-powered features, and a final capstone with tests and version control

Total projects: 30+ real projects, ending with working Minecraft mods, a REST API and a tested final capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Core Java: syntax, arrays, methods, strings, files, exceptions and genuine OOP fluency
- Modern Java: collections, generics, lambdas and streams used the way current codebases use them
- Desktop apps: JavaFX interfaces with real data, structured as MVC
- Games and mods: a finished 2D game and Minecraft mods built on Forge
- Data structures and algorithms: built from scratch, analyzed with Big O, applied to real problems
- Web APIs: a first REST API with Spring Boot and JPA persistence
- AI foundations: calling modern AI models from Java with structured output and guardrails
- Developer habits: Git and GitHub, JUnit, debugging and code review

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: Homework after every class, and a mixed-review assignment at the end of every month so earlier skills stay sharp
Practice: 3-4 hours of projects and challenges between classes
Review: Every submitted project reviewed with written feedback; recurring issues reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so nobody stays stuck on a compiler error for days
Progress updates: Regular progress notes to parents: what was built, what comes next, and where encouragement at home helps

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short

and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Data Structures and Algorithms: the dedicated course for competitive programming and interviews
- Android development: our Kotlin and app development tracks continue where Java leaves off
- Full-stack web development: deepen Spring Boot into real backend engineering
- School exams: ICSE Computer Applications, ISC Computer Science and AP Computer Science A all examine in Java
- Real small projects: tools, mods and apps for family, school clubs and friends

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my teen build in this Java course?
A: More than 30 real projects across ten months: text games and file-backed tools in the foundations, JavaFX desktop apps with tables and charts, a complete 2D game, working Minecraft mods built on Forge, hand-built data structures with a visualizer capstone, a REST API with Spring Boot, and a final capstone shipped with Git and JUnit tests. Every project is built by the student and reviewed with written feedback.

Q: How long does it take a teenager to learn Java properly?
A: Most students reach a genuinely advanced level in 10 months at 2 live classes a week plus 3 to 4 hours of practice. The syllabus deliberately keeps 2 months of margin in hand: students who need more repetition take up to 12 months, with no rushing and no penalty. Java rewards patience early, and the course is paced for exactly that.

Q: How is my teen assessed? Is the certificate just for attending?
A: No. Classwork runs inside every session, homework follows every class, and each month ends with an assignment that deliberately mixes new questions with earlier topics so knowledge compounds instead of fading. Every phase closes with a cumulative check, and the course ends with a final exam: a practical build plus written questions drawn from the whole syllabus. Passing earns the certificate; falling short earns a focused revision plan and a free retest. We optimize for real knowledge and real depth, not a stream of feel-good badges.

Q: Should my teen learn Java or Python first?
A: Both are excellent first languages, and we teach both. Java is the better start when the school board examines in it, ICSE, ISC and AP Computer Science A all do, when Minecraft modding is the dream, or when a student thrives on structure: Java's strictness catches mistakes early and builds habits every later language benefits from. Python is the better start for a teen pulled toward AI and data. Either way nothing has to be unlearned later.

Q: Does this course help with ICSE, ISC or AP Computer Science?
A: Directly. Those boards examine in Java, and the first two phases cover the language and object-oriented concepts their papers test, at greater depth than the school syllabus. Students taking those exams find school work becoming revision. For focused board preparation we also run dedicated ICSE Computer Applications and AP Computer Science A courses.

Q: Is Minecraft modding really part of the syllabus?
A: Yes, in month 6, and it is taught seriously: students set up a real Forge development environment, write mods with custom items, blocks and recipes in Java, and load them into their own game. It is the single best motivation device in teen Java, and it quietly exercises classes, inheritance and events, the exact concepts from earlier months.

Q: Does this course cover Android app development?
A: It builds the exact foundation Android rests on, and the final phase includes REST APIs, the backend half of every serious app. Modern Android itself is Kotlin-first, so rather than squeeze an outdated Android unit into this course, we teach it properly in our app development and Kotlin tracks, which pick up exactly where this course ends.

Q: Is any paid software required?
A: No. The Java Development Kit, the IntelliJ IDEA editor used in class, JavaFX, Spring Boot and the Minecraft modding toolchain are all free. A standard computer with a stable internet connection handles the entire course. Where an AI service needs an account in month 9, it is set up together with a parent.

Q: Can we book a demo class?
A: Yes, demo classes are free and carry no commitment; reserve one at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents can sit in, and we hold a 4.9 rating across 547 Google reviews if you would like to read what other families say first.

Q: Is the teaching live or recorded?
A: Every class is live: a 1-hour online session where a real teacher works with a small group, never a pre-recorded video. Students code along in class and get corrections on the spot, which matters most in Java's early weeks when one missing semicolon can feel like a

wall.

Q: Is there a certificate on completion?
A: Yes, and it is earned, not handed out: the certificate is awarded on passing the final exam, with a free retest after revision if the first attempt falls short. Alongside it stands the portfolio, a finished game, Minecraft mods, a REST API and the capstone, which shows exactly what your teen can build.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/java-programming-masterclass-for-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 89 OF 137: JAVASCRIPT FOR TEENS: INTERACTIVE WEBSITES & BROWSER GAMES
==========

Course link: https://learn.modernagecoders.com/courses/javascript-course-for-teens-beginners-interactive-web
Subject area: Web Development
Designed for: Teens
Who can join: Beginner, ages 13 to 18
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

JavaScript course for teens ages 13-18: live online classes from first variables to interactive pages, canvas games, real APIs, and a deployed capstone site.

In one line: The language every browser runs, taught from the first variable to a game your friends can actually play.

ABOUT THE PROGRAMME

JavaScript is the only programming language every web browser understands, which makes it the most immediate language a teenager can learn: write a line, refresh the page, and something on screen changes. This 6-month live course starts at zero and builds in one direction, toward work worth showing. Months 1 and 2 cover the language core and the DOM, so pages respond to clicks and typing within weeks. Months 3 and 4 add real data: arrays, objects, JSON, saving with localStorage, and fetching from public APIs such as live weather. Month 5 is games on the HTML canvas, with a proper game loop, keyboard control, and collision detection. Month 6 is the capstone: each student scopes, builds, and deploys an interactive site of their own, then presents it. Classes are live and small. Students write code in every session while an instructor watches them work, and homework gets reviewed, not just collected.

WHAT MAKES THIS COURSE DIFFERENT

- Everything runs in the browser students already have, so the write-refresh-see loop takes seconds and motivation stays high
- The DOM arrives in month 2, not month 5: pages react to clicks within the first weeks, long before the theory runs out
- Real public APIs instead of toy data: weather, dictionaries, and services students choose themselves
- A full month of canvas games, because a game you built yourself is the project teens actually show their friends
- The capstone is deployed to a real URL, so the finished site can sit on a school application or be sent to relatives
- No prior HTML needed: the markup and CSS each project requires is taught in class, exactly when it is needed

THE LEARNING PATH, STEP BY STEP

Phase 1: The language and the page: variables, decisions, loops, and functions, then the DOM and events so pages respond to users
Phase 2: Data and the network: arrays, objects, JSON, and localStorage, then fetch and public APIs for pages built on live data
Phase 3: Canvas games with a real game loop, then a scoped, deployed capstone site presented to the class

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The course starts from the first line of code
Html knowledge: Not required. The HTML and CSS each project needs is taught in class
Equipment: A computer with a modern browser and a stable internet connection
Age: 13 to 18. Younger children are better served by our kids coding courses

WHO THIS COURSE IS FOR

Complete beginners: Teens who have never written code and want a language where results show up on screen immediately
Scratch graduates: Students who outgrew block coding and want to type real programs that do real things
Future web developers: Teens aiming at websites and apps, for whom JavaScript is the non-negotiable first step
Game curious: Students who want to build playable browser games, not just read about them
Portfolio builders: Teens who want deployed, working projects to point to on school and college applications

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE LANGUAGE AND THE PAGE (MONTHS 1-2, WEEKS 1-8)

JavaScript from the first line: values, decisions, loops, and functions, then the DOM and events so real pages respond to real users.

Month 1: JavaScript from the First Line (Weeks 1-4)
First Lines of JavaScript
Topics:
- Where JavaScript runs: the browser, the console, and why no install is needed to start
- Setting up VS Code and running a script inside an HTML page
- Values and variables: let and const, and when each is right
- Numbers and strings, and what happens when you mix them
- Template literals for building sentences out of data
- console.log as your first debugging tool
- Comments and naming things so future-you understands the code
Projects built:
- Story generator: a script that assembles a silly story from variables, then from user input via prompt
Practice: 10 short console exercises: swap variables, build sentences, fix three broken scripts
Making Decisions
Topics:
- Comparisons: equal, strictly equal, greater, less, and why === matters
- if, else if, else: teaching a program to choose
- Boolean values and expressions that produce them
- Combining conditions with and, or, not
- Truthy and falsy values, and the bugs they cause
- Nesting conditions without losing the plot
Projects built:
- Number guessing game, round one: the computer picks, you guess, it answers higher or lower
Practice: Decision drills: grade calculator, ticket price rules, a login age check with edge cases
Loops and Repetition
Topics:
- while loops: repeat until something changes
- for loops: counting, stepping, and counting backwards
- Accumulator patterns: totals, counters, and building strings in a loop
- break and continue, used sparingly
- Nested loops and when they are worth it
- Reading a loop you did not write and predicting its output
Projects built:
- Times-table printer with formatting
- Guessing game, round two: limited tries, a running score, play-again logic
Practice: 12 loop katas, including FizzBuzz and a countdown timer printed to the console
Functions
Topics:
- Declaring functions: inputs, outputs, and the return statement
- Parameters vs arguments, and default values
- Arrow function syntax and where you will meet it
- Scope: what a function can and cannot see
- Breaking a messy script into small named functions
- Testing a function by hand with sample inputs
Projects built:
- Tip calculator and unit converter built as a small library of reusable functions
Practice: Write 8 functions from written specs, then refactor week 3 projects to use

```
      functions
      Assessment: Month 1 skills check: write four short functions live, explaining each
      choice

  Month 2: The DOM and Events (Weeks 5-8)
    Meeting the DOM
      Topics:
        - Just enough HTML: tags, ids, and classes, taught as we need them
        - The DOM tree: how the browser sees a page
        - querySelector and querySelectorAll: finding elements
        - Changing text with textContent and innerHTML, and the difference
        - Changing styles and toggling classes from JavaScript
        - The dev tools Elements panel as an x-ray of your page
      Projects built:
        - Mood page: buttons that change the page's colors, text, and heading live
      Practice: Select-and-change drills on a provided practice page with 15 small targets
    Events: Pages That Listen
      Topics:
        - addEventListener: click, input, and change events
        - The event object and what it tells you
        - Reading values out of text inputs
        - Updating the page in response to typing, live
        - Preventing default behavior and when you need to
        - Common event bugs: listeners attached too early, wrong element, wrong event
      Projects built:
        - Click counter with milestones
        - Live character counter for a message box, with a limit warning
      Practice: Build three tiny widgets from spec: a like button, a show-password toggle, a
      color picker preview
    Interactive Widgets
      Topics:
        - Showing and hiding elements cleanly with classes
        - Creating and removing elements with createElement and remove
        - Building lists in the DOM from JavaScript data
        - classList patterns: toggle, add, remove
        - Dark mode done properly: one class on the body
        - Organizing widget code so each piece has one job
      Projects built:
        - Dark mode toggle for a full page
        - FAQ accordion that opens one answer at a time
      Practice: Recreate two widgets from screenshots of real websites, structure first, then
      behavior
    Project Week: The Interactive Quiz
      Topics:
        - Planning a small app: screens, states, and what happens on each click
        - Keeping score and moving between questions
        - Ending states: results screen and restart
        - Basic input validation: no empty answers
        - Polish: progress display and honest feedback messages
        - Reading your own code a week later: comments and naming revisited
      Projects built:
        - Interactive quiz app: questions, scoring, a results screen, and a restart button
      Practice: Extend the quiz with one feature of your own choosing, written up in three
      sentences
      Assessment: Quiz app demo and code walkthrough: each student explains one function line
      by line

----------------------------------------------------------------
PHASE 2: DATA AND THE NETWORK (MONTHS 3-4, WEEKS 9-16)
----------------------------------------------------------------
Arrays, objects, and JSON give projects real data to work with. Then fetch opens the door to
public APIs and pages built on live information.

  Month 3: Arrays, Objects, and JSON (Weeks 9-12)
    Arrays
      Topics:
        - Creating arrays and reading items by index
        - push, pop, shift, unshift: growing and shrinking lists
        - Looping over arrays with for and for...of
        - indexOf and includes: finding things
        - Arrays and the DOM: rendering a list from data
        - slice vs splice, and the bug everyone hits once
      Projects built:
        - To-do list, version one: add, display, and delete tasks backed by an array
      Practice: 15 array exercises: totals, longest word, reversing, filtering by hand with
      loops
    Array Methods That Do the Work
      Topics:
        - map: transforming every item
```

      - filter: keeping what matches
      - find: the first match
      - sort, and why sorting numbers needs a compare function
      - Chaining methods to answer questions in one line
      - When a plain loop is still the clearer choice
    Projects built:
      - Movie list page: search box and filter buttons that narrow a list of films live
    Practice: Rewrite five loop solutions from week 9 using map and filter, compare
    readability
  Objects
    Topics:
      - Objects as labeled data: properties and values
      - Dot notation vs bracket notation
      - Objects inside arrays: the shape of most real data
      - Looping over object properties
      - Nesting: objects inside objects, read carefully
      - Designing the data shape before writing the code
    Projects built:
      - Contact cards: render a grid of profile cards from an array of person objects
    Practice: Model three real things as objects: a song, a match result, a school
    timetable entry
  JSON and Saving Data
    Topics:
      - What JSON is and why every API speaks it
      - JSON.stringify and JSON.parse
      - localStorage: saving data in the browser
      - The save-load cycle: persisting an app's state across refreshes
      - What localStorage cannot do, and where databases come in later
      - Debugging storage with the dev tools Application panel
    Projects built:
      - To-do list, version two: tasks survive a page refresh using localStorage
    Practice: Add save and load to the movie list project; inspect the stored JSON by hand
    Assessment: Data checkpoint: given a JSON file of real data, load it, answer five
    questions with code

Month 4: Fetch and Public APIs (Weeks 13-16)
  What an API Is
    Topics:
      - Requests and responses: how the browser asks servers for data
      - URLs as questions: endpoints and paths
      - Reading API documentation without fear
      - fetch and .then: your first request to a live joke API
      - Status codes: 200, 404, and what they mean
      - Looking at raw JSON responses in the browser
    Projects built:
      - Joke machine: a button that fetches and displays a fresh joke from a public API
    Practice: Explore two public APIs in the browser, write down what three endpoints
    return
  async, await, and Things Going Wrong
    Topics:
      - async and await: the cleaner way to write fetch code
      - Loading states: telling the user something is happening
      - try and catch: handling failed requests honestly
      - The network tab: watching your requests in flight
      - Slow connections and what your app looks like on them
      - Never trusting a response blindly: checking before rendering
    Projects built:
      - Weather lookup page: type a city, see live conditions, with loading and error states
    Practice: Break your own weather app on purpose three ways, then make each failure show
    a decent message
  Search Interfaces
    Topics:
      - Query parameters: sending the user's search to the API
      - Building a URL safely from input
      - Rendering result lists with the DOM skills from month 2
      - Empty results and misspelled searches: designing for them
      - Debouncing, gently introduced: not firing a request per keystroke
      - Caching a result you already fetched
    Projects built:
      - Dictionary app: search a word, show definitions, phonetics, and an example sentence from a
        free dictionary API
    Practice: Swap the dictionary API for a different public API of your choice, keep the
    same interface
  Project Week: The Two-API Dashboard
    Topics:
      - Combining two data sources on one page
      - Structuring code when an app has parts: one file, clear sections, named functions
      - Handling two requests that can each fail independently
      - Layout for dashboards: cards and sections, with just enough CSS

- A refresh button done right
- Writing a short README that says what the app does

Projects built:
- Personal dashboard: two public APIs of the student's choice on one page, for example weather plus news headlines

Practice: Trade dashboards with a classmate and file two improvement suggestions each

Assessment: Dashboard demo: live walkthrough plus questions on how each fetch works

---

PHASE 3: CANVAS GAMES AND THE CAPSTONE (MONTHS 5-6, WEEKS 17-24)

A month of real game programming on the HTML canvas, then a capstone month: scope, build, deploy, and present an interactive site of your own.

Month 5: Games on the Canvas (Weeks 17-20)

Drawing and the Game Loop

Topics:
- The canvas element and its coordinate system
- Drawing rectangles, circles, lines, and text
- Color, stroke, and fill
- requestAnimationFrame: the heartbeat of every game
- Clearing and redrawing: why animation is just fast drawing
- Moving a shape across the screen with velocity

Projects built:
- Bouncing ball: a ball that moves, hits the walls, and bounces, with adjustable speed

Practice: Draw a scene of your choice in code, then animate one element of it

Player Control

Topics:
- Keyboard events: keydown, keyup, and tracking held keys
- Moving a player smoothly, not one hop per press
- Boundaries: keeping the player on screen
- Separating update logic from drawing logic
- Game state as an object: player, score, settings in one place
- Frame-rate honesty: why movement uses small steps

Projects built:
- Paddle controller: a paddle that glides left and right under arrow-key control, ready for a game

Practice: Add a second control scheme (WASD) and an on-screen speed display

Collisions, Score, and Lives

Topics:
- Rectangle collision detection, explained with pictures before code
- Circle collision using distance
- Spawning objects: falling items with random positions and speeds
- Score, lives, and difficulty that ramps up
- Removing off-screen objects so the game does not slow down
- Game over conditions and detecting them cleanly

Projects built:
- Catcher game core: falling objects, a moving basket, score on catch, lives lost on miss

Practice: Tune your game's difficulty curve and write down the three numbers you changed and why

Finish and Polish the Game

Topics:
- Start screen, pause, and game-over screen as states
- Sound effects with the Audio API, used tastefully
- High score saved with localStorage, from month 3
- Small touches that read as polish: flash on hit, score pop
- Playtesting: watching someone else play your game without helping them
- Packaging the game on a page with instructions

Projects built:
- Finished catcher or breakout-style game with start, pause, game over, sound, and a saved high score

Practice: Run two playtests with family members and fix the most confusing thing each one hit

Assessment: Game demo day: every student's game played live by a classmate, followed by a code Q&A

Month 6: The Capstone Site (Weeks 21-24)

Scoping the Capstone

Topics:
- Choosing a project you can finish: the honest feature list
- Wireframing on paper before touching code
- Picking your data source: localStorage, a public API, or both
- Breaking the build into tasks with a simple checklist
- Setting up the project folder and skeleton page
- What done means: writing acceptance criteria in plain language

Projects built:
- Approved capstone plan: one page with wireframe, feature list, and task breakdown

Practice: Build the static skeleton of your capstone: structure and placeholder content, no behavior yet

Build Sprint One: Core Features
Topics:
- Working feature by feature, smallest first
- Instructor code review: reading feedback and applying it
- Debugging your own project methodically: reproduce, isolate, fix
- Keeping functions small while the project grows
- When to delete a feature: scope trimming without shame
- Committing progress notes at the end of each session
Projects built:
- Capstone core: the two or three features that make the project itself
Practice: Daily 30-minute build sessions with a written note on what moved forward
Build Sprint Two: Data and Fit
Topics:
- Wiring in the data layer: saving, loading, or fetching live data
- Making the layout survive a phone screen: the essential media query
- Empty states and first-use experience
- Error states for anything that can fail
- A second code review round
- Cutting the last risky feature if time demands it
Projects built:
- Feature-complete capstone: data working, layout presentable on desktop and phone
Practice: Test your capstone on a phone and one other browser, log what broke
Ship and Present
Topics:
- Bug hunt: a structured final pass over every feature
- Deploying to a free host so the site has a real URL
- Writing a README: what it is, how it works, what you would add next
- Presenting technical work: demo first, code second
- Answering questions about your own code
- Where to go from here: React, Node.js, or deeper JavaScript
Projects built:
- Deployed capstone site with README, presented live to the class and family
Practice: Send your capstone link to three people who are not in the class
Assessment: Capstone presentation and code Q&A, plus course-completion certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- Number guessing game with score and limited tries
- Interactive quiz app with results screen and restart
- To-do list that survives refresh with localStorage
- Movie list page with live search and filters
- Weather lookup page built on a live public API
- Dictionary app with definitions and phonetics
- Two-API personal dashboard of the student's choice
- Canvas catcher game with score, lives, sound, and a saved high score
- Deployed capstone: an interactive site scoped, built, and presented by the student

Total projects: 12+ working projects, from console games to a deployed capstone site

SKILLS THE STUDENT WILL HAVE AT THE END

- JavaScript core: variables, conditionals, loops, and functions written with confidence
- DOM and events: selecting, changing, and building page content that responds to users
- Data handling: arrays, objects, JSON, and the array methods real codebases lean on
- The network: fetch, async and await, loading and error states, and reading API docs
- Canvas games: the game loop, keyboard control, collision detection, and game states
- Working habits: debugging methodically, scoping projects, and presenting your own code

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Practice: 2-3 hours of set exercises and project work between classes
Review: Homework reviewed with written feedback; stuck points reopened in the next class
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so nobody stays stuck until the next session
Progress updates: Regular progress notes to parents: what was covered, what was built, what needs practice

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Front-end development track: our React and full-stack courses build directly on this foundation
- Back-end JavaScript with Node.js once the language core is solid
- School hackathons and coding competitions with a working project toolkit
- Game development courses, carrying over the canvas and game-loop experience
- A stronger base for computer science electives and board CS subjects

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Should my teen learn JavaScript or Python first?
A: Either works, and we teach both, so this is about the student, not the language. JavaScript wins for teens motivated by visible results: it runs in the browser, so every lesson changes something they can see, click, and share. Python reads a little more like English and suits students drawn to data, maths, or automation. If your teen talks about websites and games, start here; if they talk about AI and data, start with Python.

Q: Does my teen need to know HTML before starting?
A: No. We teach the small amount of HTML and CSS each project needs, exactly when the project needs it. Students who want to go deep on page structure and design can take our HTML and CSS course before or alongside this one, but it is not a requirement.

Q: What will my teen actually build in six months?
A: Twelve or more working projects, in rising order of ambition: console games, an interactive quiz, a to-do app that saves its data, a live weather page, a dictionary app, a two-API dashboard, a canvas game with score and sound, and finally a capstone site they scope and build themselves. The capstone is deployed to a real URL, so it exists outside the classroom.

Q: How are the classes taught?
A: Live one-hour online classes, twice a week, with a real instructor and a small group. Students write and run code during class rather than watching someone else do it, and homework comes back with written feedback. Group batches are capped at 20, with mini-batch and 1-on-1 options for students who want closer attention.

Q: Is 13 too young for real JavaScript?
A: No, the course is written for exactly this range. We pace the abstract parts, functions, scope, asynchronous code, with more examples and more practice than an adult course would, and every concept lands inside a project rather than a slide. For children under 13 we recommend our kids coding courses, which build the same thinking through friendlier tools first.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final exam covering the whole course. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Can we try a class before enrolling?
A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to sit in and ask the teacher questions afterwards.

Q: What software and computer are needed?
A: Everything used in the course is free: a modern browser like Chrome and the VS Code editor, which we help set up in the first class. Any reasonably recent laptop or desktop with a stable internet connection is enough; there is no heavy software to install.

Q: What should a student take after this course?
A: The natural next steps are our React course for modern front-end work, Node.js for the server side, or our data structures and algorithms course for students eyeing competitive programming and technical interviews later. Many students also carry their canvas experience into deeper game development. The instructor gives each student a specific recommendation at the end based on what they enjoyed most.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/javascript-course-for-teens-beginners-interactive-web
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 90 OF 137: JEE FOUNDATION MATHS CLASS 8-10: PROOFS & PROBLEM SOLVING
==========

Course link: https://learn.modernagecoders.com/courses/jee-foundation-maths-course-class-8-10
Subject area: Foundation Mathematics
Designed for: High school
Who can join: Class 8 to 10 students, batched by class
Course length: 12 months (48 weeks)
Time needed: 2 live classes/week + 3-4 hours of problem sets
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  JEE foundation maths course for Class 8-10: NCERT-aligned depth, geometry proof skills, olympiad-style problem solving and timed MCQ practice in live batches.

  In one line: Class 11 physics and maths punish shaky algebra. This course is where the algebra stops being shaky.

ABOUT THE PROGRAMME

  Most students who struggle in JEE preparation do not struggle with Class 11 topics; they struggle with Class 8 to 10 topics that were never made solid. Fractions handled nervously, identities half-remembered, geometry never actually proved. A foundation course exists to fix that before it costs anything, and this one does it honestly: no rank promises, no 'future IITian' theatre, just the school syllabus taught deeper than school goes, plus the problem-solving habits that entrance exams eventually demand. The 12-month plan runs in four phases. First, numbers and algebra at real depth: divisibility and remainders, exponents and surds, identities and factorisation, equations and inequalities. Second, geometry as a proof discipline, congruence, similarity, circles, because writing a correct proof is the closest a school student comes to real mathematical thinking. Third, mensuration, introductory trigonometry and coordinate geometry, the toolkit chapters. Fourth, the part school never teaches: problem-solving heuristics, olympiad-style questions, and a full timed MCQ phase in the style of NTSE-type talent exams and entrance papers, with error logs and review clinics. Content is NCERT-aligned and batched by class, so a Class 8 student and a Class 10 student meet the same ideas at different depths. School marks tend to improve as a side effect; the actual product is a student who reaches Class 11 with nothing to repair.

WHAT MAKES THIS COURSE DIFFERENT

  - NCERT-aligned but deeper: every chapter is taken past the textbook stopping point, into the why and the harder problem forms
  - Proof skills taught deliberately: given, to prove, construction, proof, drilled until writing a geometry argument feels normal
  - A genuine problem-solving phase: heuristics like working backwards, small cases and clean diagrams, practised on olympiad-style questions
  - A full timed MCQ phase with error logs: elimination, option testing and time budgeting, the craft entrance exams actually test
  - Honest positioning: this is foundation, not JEE coaching, and we say plainly that nobody can promise selections or ranks
  - Batched by class: Class 8, 9 and 10 groups meet the same arc at the right depth, so nobody is drowned or bored

THE LEARNING PATH, STEP BY STEP

  Phase 1: Numbers and algebra at depth: divisibility and remainders, exponents and surds, identities, factorisation, equations and inequalities
  Phase 2: Geometry as a proof discipline: angles, congruence, similarity, quadrilaterals and circle theorems, with proof writing drilled throughout
  Phase 3: The toolkit chapters: mensuration in 2D and 3D, introductory trigonometry with heights and distances, and coordinate geometry
  Phase 4: Problem-solving heuristics and olympiad-style questions, then a timed MCQ and mock phase with error clinics and a readiness review

WHAT A STUDENT NEEDS BEFORE STARTING

  Maths level: Comfortable with the previous class's school maths. The week 1 diagnostic finds the gaps and the early weeks patch them
  Class: Class 8, 9 or 10. Batches are separated by class so depth and pace fit the student
  Equipment: The student's NCERT maths textbook, a geometry box, graph paper, notebooks, and a device with stable internet
  Mindset: Willingness to write solutions out in full. This course grades reasoning, not just answers

WHO THIS COURSE IS FOR

Future jee aspirants: Students in Class 8 to 10 who intend to attempt JEE or similar entrances and want the base built before Class 11 raises the stakes
Strong students underfed: Students who find school maths easy and are bored, and need harder problems rather than more of the same
Proof never taught: Students who can compute but have never been taught to prove, which is most students
Mcq first timers: Students who have only ever written school papers and need timed MCQ craft before any talent or entrance exam
Families avoiding hype: Parents who want serious foundation work without rank promises and future-topper theatre

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: NUMBERS AND ALGEBRA AT DEPTH (MONTHS 1-3, WEEKS 1-12)

The chapters every entrance exam quietly assumes: number theory basics, exponents and surds, identities, factorisation, and equations, taken well past the textbook stopping point.

Month 1: Number Systems, Properly (Weeks 1-4)
Weeks 1-2: The Number Line, End to End
Topics:
- Diagnostic paper: arithmetic, fractions, basic algebra and geometry from earlier classes
- Integers, rationals and irrationals, and how to prove a number belongs where it does
- Decimal expansions: terminating, recurring, and converting recurring decimals to fractions
- Representing surds on the number line
- Density: finding rationals and irrationals between any two numbers
- Comparing numbers cleverly: without calculators, by squaring or bounding
Projects built:
- Number-line dossier: one page classifying 20 assorted numbers with a one-line justification each
Practice: 30 classification and conversion problems, including 5 recurring-decimal conversions written out in full
Weeks 3-4: Divisibility and Remainders
Topics:
- Divisibility rules for 2 through 11, proved rather than just stated
- Primes, composites and prime factorisation as the master tool
- HCF and LCM: computation, the product relationship, and word problems
- Remainder thinking: what is left when powers are divided, kept concrete
- Digit problems: unit digits of large powers, digit sums
- First olympiad-style flavour: divisibility puzzles with small numbers
Projects built:
- Divisibility rule proof sheet: the rules for 3, 9 and 11 explained in the student's own words with examples
Practice: 25 divisibility and HCF-LCM problems plus 5 unit-digit puzzles, hardest two attempted before class discussion
Assessment: Month 1 test: a 25-mark paper on number systems mixing standard questions with two puzzle-style problems

Month 2: Exponents, Surds and Identities (Weeks 5-8)
Weeks 5-6: Exponents and Surds
Topics:
- Laws of exponents for integer and rational powers, with the proofs sketched
- Negative and fractional exponents handled without fear
- Surds: simplifying, adding, and rationalising denominators including two-term denominators
- Comparing surds and exponential expressions by clever bounding
- Scientific notation and orders of magnitude
- Exam forms: simplify-this-monster questions and how to dismantle them
Projects built:
- Simplification gauntlet: 10 layered exponent-surd expressions reduced step by step, each step justified
Practice: 35 exponent and surd problems including 8 rationalisations and 5 comparison questions without a calculator
Weeks 7-8: Identities and Factorisation
Topics:
- The square and cube identities, derived, visualised, and drilled
- Sum and difference of cubes, and the three-variable identity for a cubed sum
- Factorisation strategy: common factors, grouping, splitting the middle term, identities
- The remainder theorem and factor theorem, introduced with numeric checks
- Evaluating expressions cleverly: 99 squared, 101 cubed via identities
- Identity-based olympiad-style problems: finding values without finding variables
Projects built:
- Identity map: every identity in the phase on one sheet with a numeric example and one exam use each
Practice: 30 factorisations across all five strategies plus 8 evaluate-cleverly problems, strategy named before each
Assessment: Month 2 test: a 30-mark paper on exponents, surds and identities including

two multi-step simplifications

Month 3: Equations and Inequalities (Weeks 9-12)
Weeks 9-10: Linear Equations and Systems
Topics:
- Linear equations in one variable, including fractional and bracketed forms
- Word-problem craft: translating ages, digits, mixtures and motion into equations
- Pairs of linear equations: substitution and elimination done cleanly
- Consistency: when systems have one, many or no solutions, seen graphically
- Parameter questions: for what value of k does the system break
- Checking solutions as a habit, not an afterthought

Projects built:
- Translation drill sheet: 12 word problems converted to equations before any are solved

Practice: 25 equations and systems plus 8 word problems, with the check step written for every answer

Weeks 11-12: Inequalities and a First Look at Quadratics
Topics:
- Linear inequalities: solving, representing on the number line, and the sign-flip rule proved
- Compound conditions: and versus or, read carefully
- Quadratic equations by factorisation: the bridge from identities to solving
- The idea of a discriminant, previewed for the Class 10 batch
- Absolute value as distance, kept geometric and gentle
- Phase consolidation: mixed algebra sets that do not announce their chapter

Projects built:
- Personal error log opened: every phase 1 test miss classified as concept, method or slip, used all year

Practice: 20 inequalities and 15 factorisation-solved quadratics, plus one mixed set of 15 unannounced problems

Assessment: Phase 1 milestone: a 40-mark numbers-and-algebra paper, one third of it in MCQ format as a preview of phase 4

PHASE 2: GEOMETRY AS A PROOF DISCIPLINE (MONTHS 4-6, WEEKS 13-24)

Geometry is where students first learn what mathematical certainty means. Congruence, similarity, quadrilaterals and circles, with proof writing drilled from the first week.

Month 4: Angles, Triangles and Congruence (Weeks 13-16)
Weeks 13-14: Lines, Angles and What a Proof Is
Topics:
- Angles on a line, vertically opposite angles, and angles with parallel lines
- The triangle angle-sum result and exterior angle theorem, proved
- What a proof is: statements, reasons, and why 'it looks true' is not enough
- The four-part format: given, to prove, construction, proof
- Angle chasing: finding unknown angles in layered figures
- Drawing honest figures: large, labelled, and not misleadingly special

Projects built:
- First proof portfolio entries: angle-sum and exterior-angle proofs written in full four-part format

Practice: 20 angle-chasing figures of rising difficulty plus 2 proofs, every figure redrawn by hand before solving

Weeks 15-16: Congruence
Topics:
- Congruence criteria: SAS, ASA, AAS, SSS and RHS, and why AAA fails
- Choosing the right criterion: reading the figure for what is actually given
- Isosceles triangle results, proved both directions
- Inequalities in a triangle: bigger side, bigger angle, and the triangle inequality
- Multi-step congruence proofs: proving parts equal via congruent triangles
- Common proof errors: assuming the conclusion, circular reasoning, unstated steps

Projects built:
- Criterion sorting sheet: 12 figures matched to their congruence criterion with the given parts marked

Practice: 15 congruence problems including 5 full proofs, each proof peer-checked against the four-part format

Assessment: Month 4 test: a 25-mark geometry paper with two full proofs marked strictly on structure and justification

Month 5: Similarity, Pythagoras and Quadrilaterals (Weeks 17-20)
Weeks 17-18: Similarity and Pythagoras
Topics:
- Similar triangles: the AA, SSS and SAS similarity criteria
- The basic proportionality theorem and its converse
- How similarity scales lengths and areas differently, seen with numbers first
- The Pythagoras theorem proved by similarity, and its converse
- Applications: heights via shadows, distances across rivers, ladders and slopes
- Pythagorean triples and spotting them fast in MCQs

Projects built:
- Proof portfolio extended: BPT and the similarity proof of Pythagoras written in full

Practice: 20 similarity and Pythagoras problems including 3 proofs and 5 real-setting

applications
Weeks 19-20: Quadrilaterals and the Midpoint Theorem
Topics:
- The parallelogram theorems: sides, angles and diagonals, proved and reversed
- Tests for a parallelogram: which minimal facts force the shape
- Rectangle, rhombus and square as parallelograms with extra conditions
- The midpoint theorem and its converse, with applications
- Multi-step figures: midpoints inside triangles inside parallelograms
- Proof strategy: what to construct when the figure refuses to yield
Projects built:
- Quadrilateral family tree: the shapes and the exact conditions separating them, made by the student
Practice: 18 quadrilateral problems including 4 proofs, plus 5 midpoint-theorem applications
Assessment: Month 5 test: a 25-mark paper on similarity and quadrilaterals with two proofs and one construction-strategy question

Month 6: Circles and the Geometry Workshop (Weeks 21-24)
Weeks 21-22: Circle Theorems
Topics:
- Chords: equal chords, distances from the centre, the perpendicular from the centre
- The central angle theorem: the angle at the centre doubles the angle at the arc
- Angles in the same segment, and the angle in a semicircle
- Cyclic quadrilaterals and their opposite angles
- Tangents: perpendicularity to the radius and equal tangents from an external point
- Angle chasing in circle figures, the entrance-exam favourite
Projects built:
- Circle theorem atlas: every theorem drawn, stated and proved or justified on the student's own pages
Practice: 20 circle problems from single-theorem to three-theorem figures, plus 2 full proofs
Weeks 23-24: The Geometry Problem Workshop
Topics:
- Hard multi-step figures mixing congruence, similarity, quadrilaterals and circles
- Auxiliary constructions: when and what to draw, taught as a decision, not magic
- Working backwards from what must be proved
- Olympiad-style geometry: problems where the figure is the puzzle
- Timed geometry MCQs: answering from the figure without full proofs
- Phase review: the proof portfolio completed and audited
Projects built:
- Workshop set: 6 multi-step geometry problems solved with the construction decision written out for each
Practice: One timed 15-question geometry MCQ set plus 4 workshop problems, error log updated with figure-reading misses
Assessment: Phase 2 milestone: a 40-mark geometry paper, half proofs and half timed MCQs, with a one-on-one proof review

---
PHASE 3: MENSURATION, TRIGONOMETRY AND COORDINATES (MONTHS 7-9, WEEKS 25-36)

The toolkit chapters every entrance exam draws on: 2D and 3D mensuration, introductory trigonometry with heights and distances, and coordinate geometry.

Month 7: Mensuration in Two and Three Dimensions (Weeks 25-28)
Weeks 25-26: Areas in the Plane
Topics:
- Areas of triangles: half base times height, and Heron's formula for when heights hide
- Areas of parallelograms, trapeziums and rhombuses, derived not memorised
- Circles, sectors and segments, with arc lengths
- Composite figures: decomposing shaded regions cleanly
- Same base, same parallels: area relationships without any measuring
- Estimation checks: is that area plausible for that figure
Projects built:
- Decomposition sheet: 8 shaded-region problems with the cut lines drawn and named before any formula is used
Practice: 22 area problems including 5 Heron applications and 5 composite figures, decomposition stated in one line each
Weeks 27-28: Solids and Scaling
Topics:
- Surface areas and volumes: cube, cuboid, cylinder, cone, sphere, hemisphere
- Combined solids and which surfaces vanish at the joins
- Melting and recasting problems: volume as the conserved quantity
- Scaling laws: double the length, four times the area, eight times the volume
- Unit discipline across centimetres, metres and litres
- MCQ forms: ratio-based mensuration questions solved without computing either quantity
Projects built:
- Scaling experiment write-up: one solid scaled by 2 and by 3, with areas and volumes tabulated and the pattern stated
Practice: 18 solid problems including 4 recasting and 4 ratio questions, units checked

on every line
Assessment: Month 7 test: a 25-mark mensuration paper mixing computation, ratio reasoning and one composite figure

Month 8: Introductory Trigonometry (Weeks 29-32)
Weeks 29-30: Ratios and Identities
Topics:
- The six trigonometric ratios in a right triangle and the naming logic
- Exact values at 0, 30, 45, 60 and 90 degrees, rebuilt from two triangles rather than memorised
- Finding all ratios from one, with Pythagoras doing the work
- The fundamental identity and its rearrangements
- Proving simple identities with a fixed strategy list
- Ratio MCQs at speed: value questions in under a minute

Projects built:
- Two-triangle derivation page: the 30-60-90 and 45-45-90 triangles built and every standard value read off them

Practice: 30 ratio evaluations and 6 identity proofs, plus one 10-question timed MCQ set on values
Weeks 31-32: Heights and Distances
Topics:
- Angles of elevation and depression on honest, labelled figures
- Single-triangle problems: towers, poles, kites and slopes
- Two-triangle problems: two observation points and moving observers
- Choosing the ratio that solves in one step
- Rounding sensibly and sanity-checking against the figure
- Where trigonometry goes in Class 11: a preview, not a syllabus

Projects built:
- Figure-first set: 8 heights and distances problems where the marked figure is drawn and approved before solving

Practice: 15 heights and distances problems including 4 two-triangle setups, every figure labelled with the given angle
Assessment: Month 8 test: a 25-mark trigonometry paper of ratios, identities and applications, one third MCQ

Month 9: Coordinate Geometry (Weeks 33-36)
Weeks 33-34: The Plane and Its Formulas
Topics:
- The Cartesian plane: plotting, quadrants and reading coordinates fluently
- The distance formula from Pythagoras, derived and drilled
- Classifying triangles and quadrilaterals by computed distances
- The section formula and midpoints, with internal division problems
- Ratios from coordinates: where the axes cut a segment
- Sketch-first habit: every coordinate problem starts with a drawing

Projects built:
- Coordinate gallery: 6 figures plotted, classified and verified by distances, presented on graph paper

Practice: 25 distance and section formula problems, each with a labelled sketch, plus 5 classification questions
Weeks 35-36: Lines and Coordinate Problem Solving
Topics:
- Slope as steepness: computing it and reading it from a sketch
- Parallel and perpendicular slopes, kept concrete
- Collinearity by slope and by distance, two roads to one answer
- Coordinate proofs: choosing axes cleverly to make geometry easy
- Mixed coordinate MCQs under time
- Phase review: the toolkit chapters consolidated onto revision sheets

Projects built:
- Coordinate proof piece: one classical geometry fact proved by placing coordinates well, written up in full

Practice: 20 slope and collinearity problems plus one timed 12-question coordinate MCQ set, error log updated
Assessment: Phase 3 milestone: a 40-mark paper across mensuration, trigonometry and coordinates, half MCQ, fully reviewed

---
PHASE 4: PROBLEM SOLVING AND THE MCQ PHASE (MONTHS 10-12, WEEKS 37-48)

The part school never teaches: heuristics for unfamiliar problems, olympiad-style questions, then a full timed MCQ and mock phase in the style of NTSE-type talent exams and entrance papers.

Month 10: How Hard Problems Are Actually Solved (Weeks 37-40)
Weeks 37-38: The Heuristics
Topics:
- Understand, plan, execute, check: the honest version of problem solving
- Trying small cases and looking for the pattern that survives
- Working backwards from what is asked
- Drawing the right diagram, and redrawing when it misleads

- Parity and simple invariants: what cannot change no matter the moves
- Olympiad-style number and algebra problems using the heuristics, difficulty honest but humane

Projects built:
- Solution journal opened: every hard problem written up with the heuristic used and the dead ends kept, not erased

Practice: 8 problems for the week, chosen to need different heuristics; write-ups matter more than answer counts

Weeks 39-40: Counting and Logical Structure

Topics:
- Systematic listing: counting without missing and without double-counting
- The multiplication principle and simple arrangements
- Combinations kept concrete: choosing teams and handshakes
- The pigeonhole principle through puzzles
- Logic puzzles: knights, liars and truth tables lite
- Where these ideas reappear: Class 11 permutations and combinations, and reasoning sections of talent exams

Projects built:
- Counting problem set write-up: 5 counting problems solved twice, once by listing and once by principle, results reconciled

Practice: 12 counting and logic problems with full write-ups in the solution journal

Assessment: Month 10 checkpoint: a 20-mark problem-solving paper where method write-ups earn marks alongside answers

Month 11: MCQ Craft Under Time (Weeks 41-44)

Weeks 41-42: The Craft of the Multiple Choice Question

Topics:
- How MCQ setters build wrong options, and what that tells the solver
- Elimination: discarding options by sign, size, units or parity
- Option testing: substituting answers back instead of solving forwards
- Special values: choosing convenient numbers for variable-heavy questions
- Estimation as a weapon: bounding the answer before computing it
- When to solve honestly: questions where shortcuts are traps

Projects built:
- Technique tally: 30 MCQs solved with the technique noted per question, then tallied to see each student's habits

Practice: Three 15-question topic-wise MCQ sets under time, error log updated with technique choices, not just misses

Weeks 43-44: Sectional Timing and the Error Log

Topics:
- Time budgeting: marks per minute and the discipline of moving on
- Sectional MCQ sets across algebra, geometry, mensuration, trigonometry and coordinates
- The two-pass strategy: bank the easy questions first
- Guessing policy: when unanswered beats wrong, and reading the paper's rules
- Error log analytics: which topic, which technique, which minute of the section
- Building each student's personal weak-topic drill plan for the mock month

Projects built:
- Error log analysis report: the student's own two-page summary of their MCQ patterns going into the mocks

Practice: Four timed sectional sets across the fortnight, each reviewed the same week, drill plan agreed with the teacher

Assessment: Month 11 checkpoint: one full-length topic-mixed MCQ paper under exam timing, fully debriefed

Month 12: Mocks and the Readiness Review (Weeks 45-48)

Weeks 45-46: The Mock Series

Topics:
- Full-length foundation mocks in the style of NTSE-type talent exams and entrance papers
- Exam-day routine rehearsed: instructions, bubbling, rough-work discipline
- Review clinic after every mock: every miss classified and assigned a fix
- Score movement read honestly: trends matter, single papers do not
- Targeted drills between mocks from each student's plan
- Stamina: holding accuracy in the last quarter of a long paper

Projects built:
- Mock file: each paper filed with its marked script, error classification and one-line fix per miss

Practice: One additional mock at home under honest timing, self-marked and brought to the review clinic

Weeks 47-48: Final Mocks and the Road to Class 11

Topics:
- Two final mocks under strict conditions, spaced for recovery
- The readiness map: every topic rated from the year's evidence, no vibes
- The solution journal and proof portfolio reviewed as a body of work
- What Class 11 changes: pace, abstraction, and how this foundation meets it
- Keeping the edge: a weekly problem habit for the months after the course
- Course close: one-on-one review with each family

Projects built:
- Personal readiness map: topic-by-topic strengths and fixes, built from the error log and mock file
- Completed solution journal: a year of hard problems with the student's own write-ups

Practice: Light targeted drills only, set individually from the final error log
Assessment: Course milestone: final full-length mock and readiness review with the family, plus certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A proof portfolio: every major theorem of the course written in full four-part format
- A solution journal: hard problems written up with heuristics and honest dead ends
- A personal error log maintained from phase 1 and analysed before the mock phase
- The identity map and circle theorem atlas, built by the student
- A coordinate proof piece: classical geometry proved by clever axis placement
- A scaling experiment write-up connecting length, area and volume
- The technique tally and error log analysis report from the MCQ phase
- A mock file with marked scripts and a final topic-by-topic readiness map

Total projects: A year-long body of work: proof portfolio, solution journal, error log, mock file and readiness map

SKILLS THE STUDENT WILL HAVE AT THE END

- Number theory basics: divisibility, primes, HCF-LCM and remainder thinking
- Algebra at depth: exponents, surds, identities, factorisation, equations and inequalities
- Proof writing across congruence, similarity, quadrilaterals and circle theorems
- Mensuration, introductory trigonometry and coordinate geometry to entrance-exam standard
- Problem-solving heuristics: small cases, working backwards, invariants and clean diagrams
- Timed MCQ craft: elimination, option testing, time budgeting and error-log-driven improvement

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week; mock sittings in month 12 run longer for full papers
Practice: 3-4 hours weekly of problem sets, proofs and journal write-ups between classes
Review: Problem sets marked with written feedback; proofs and journal entries reviewed line by line each month
Homework: A problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for stuck problems
Progress updates: Monthly progress notes to parents with test scores, error-log trends and the current readiness picture

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies mastery, not attendance

WHERE THIS CAN LEAD

- Class 11 maths in the science stream, entered with algebra and geometry that do not need repair
- Formal JEE preparation from Class 11, with the foundation layer already done
- Olympiad tracks such as IOQM for students who discover they enjoy the hard problems
- School exams along the way: the depth here makes board-level questions feel routine
- Our data science and programming courses, where the same structured thinking transfers directly

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What exactly is a foundation course? Is this JEE coaching for a 13-year-old?
A: No, and be wary of anything that says otherwise. JEE tests Class 11 and 12 material; no Class 8 student should be solving JEE papers. A foundation course strengthens the Class 8 to 10 layer that JEE preparation later stands on: algebra done at depth, geometry actually proved, and the problem-solving and MCQ habits school never teaches. Students who arrive in Class 11 with that layer solid cope with the pace; students who arrive with gaps spend Class 11 repairing them.

Q: Do you promise selection in JEE or any exam?
A: No. Nobody can honestly promise a selection or a rank, least of all three or four years out, and we think enrolling a 13-year-old on the back of such a promise is wrong. What we commit to is specific: the full curriculum taught at depth, marked work with written feedback, a proof portfolio and solution journal the student builds, a timed mock series, and honest monthly reporting on exactly where the student stands.

Q: Is this course NTSE preparation? Is NTSE even happening?

A: We are careful here because many courses are not: NCERT suspended the National Talent Search Scheme, and the exam has not been conducted since the 2021-22 cycle, with the scheme stalled until further orders. There has been talk of a revamped revival, but nothing a family should plan around. We use NTSE-style papers in the mock phase because the format is excellent training, and if the exam returns our students will be ready for it, but nobody should enrol anywhere on the promise of an exam that is currently not running.

Q: Which class is the right time to start, 8, 9 or 10?
A: Any of the three works, which is why batches are separated by class. Starting in Class 8 gives the most room: depth can build gradually with no exam pressure. Class 9 is the most common entry point and aligns well, since Class 9 NCERT is where proofs and harder algebra begin. Class 10 students run the same arc with board preparation happening in parallel, and the overlap actually helps, because the foundation depth makes board questions feel easy.

Q: Will this clash with what school teaches?
A: No, because the course is NCERT-aligned by design. We teach the same chapters school does, in a compatible order, and go deeper rather than sideways: harder problem forms, the proofs behind the formulas, and extensions school skips for time. Most families find school marks improve as a side effect, since school papers sample the easier end of what students here practise weekly.

Q: What are the olympiad-style problems? Is this an olympiad course?
A: It is not a dedicated olympiad course; serious olympiad preparation for exams like IOQM is its own track with its own syllabus. What we teach in month 10 is the thinking style olympiads reward: heuristics like small cases, working backwards and invariants, practised on genuinely unfamiliar problems at a humane difficulty. Students who light up during that phase get pointed, honestly, toward proper olympiad training as a next step.

Q: How much homework should we expect?
A: Three to four hours a week: problem sets after each class, proof write-ups during the geometry phase, journal entries during the problem-solving phase, and timed papers in the final months. The write-ups are non-negotiable, because this course grades reasoning, not answer lists. It is a real commitment, and we would rather say so now than surprise you in month 2.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A problem set follows every class with written feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real understanding, not attendance.

Q: Can we try a class before enrolling?
A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. The demo includes a short diagnostic, after which the teacher will tell you plainly whether the student is ready for this course or better served by our school maths courses first.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/jee-foundation-maths-course-class-8-10
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 91 OF 137: KOTLIN AND ANDROID COURSE: ZERO TO PUBLISHED-READY APPS
==========

Course link: https://learn.modernagecoders.com/courses/complete-kotlin-programming-masterclass-professional
Subject area: App Development
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Kotlin course for college students and professionals: modern Kotlin, coroutines, then real Android development with Jetpack Compose in 10-12 months.

In one line: The modern Android stack, learned in its native language: Kotlin to real depth, then apps built the way Google builds them, with Jetpack Compose.

ABOUT THE PROGRAMME

Android runs on Kotlin now, and the modern stack, Compose for UI, coroutines for concurrency, is genuinely pleasant when learned in order. This live online masterclass does exactly that: Kotlin as a language first, null-safety, functional style, coroutines and flows understood properly, then Android development the modern way: Jetpack Compose interfaces, architecture that survives rotation and growth, data with Room, networking, and the production habits, testing, releases, honest store literacy, that separate app builders from tutorial followers. A finale month ships a capstone app and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest. Real Kotlin, real apps, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Kotlin learned as a language, not as Android syntax: coroutines and flows understood before they are needed
- Compose-first: the UI toolkit Google actually recommends, not legacy XML nostalgia
- Architecture that survives: state, lifecycles and structure taught as the core Android skill they are
- Store literacy told straight: what publishing takes, without fake promises
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Kotlin (Months 1-3): the language to real depth, null-safety to coroutines
Phase 2: Advanced Kotlin (Months 4-6): functional style, flows, and the idioms production code uses
Phase 3: Android with Compose (Months 7-9): UI, architecture, data, networking and production habits
Phase 4: The finale (Month 10): the capstone app, shipped and defended, and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The language is taught from the first line
Equipment: Any reasonably recent computer; Android Studio and every tool used are free, and the emulator plus any Android phone cover testing
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

College students: Students who want a shippable, demonstrable skill alongside the degree
App dreamers: Builders with an app idea and the patience to build it properly
Java refugees: Developers from Java-era Android ready for the modern stack
Career switchers: Professionals entering mobile, the most portfolio-friendly specialization
Backend curious: Learners who want Kotlin depth that also serves server-side later

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & CORE KOTLIN SKILLS (MONTHS 1-3, WEEKS 1-13)

Build rock-solid Kotlin fundamentals. Learn programming logic, master Kotlin syntax, OOP principles, functional programming, and create your first applications.

Months 1-2: Kotlin Fundamentals & Programming Basics (Weeks 1-7)
Introduction to Kotlin & Development Environment Setup
Topics:
- What is Kotlin? History, features, and JetBrains creation
- Why Kotlin? Advantages over Java and other languages
- Kotlin ecosystem: Android, Backend, Multiplatform, Native
- Kotlin/JVM, Kotlin/JS, Kotlin/Native explained
- Interoperability with Java
- Installing JDK (Java 17 LTS or latest)
- Setting up development environment: IntelliJ IDEA, Android Studio
- Understanding IDE features and Kotlin-specific shortcuts
- Kotlin REPL (Read-Eval-Print Loop) for quick testing
- Your first Kotlin program: Hello World
- Kotlin program structure: main function, packages
- Compiling and running Kotlin programs
- Understanding bytecode and JVM execution
- Comments: single-line (//), multi-line (/* */), KDoc (/** */)
- Kotlin coding conventions and naming standards

    - Type inference and concise syntax
  Projects built:
    - Hello World variations with string templates
    - Simple output programs using println
    - Personal information display with string interpolation
    - ASCII art generator in Kotlin
    - Interactive REPL experiments
  Practice: Daily: 30 min Kotlin syntax practice, write 5-10 simple programs
Variables, Data Types & Operators
  Topics:
    - Variables: val (immutable) vs var (mutable)
    - Type inference: let the compiler infer types
    - Explicit type declarations
    - Basic types: Int, Long, Short, Byte, Double, Float, Char, Boolean
    - String type and string templates
    - String interpolation with $ and ${}
    - Multi-line strings with triple quotes
    - Type checking with is operator
    - Smart casts after type checking
    - Type casting: as and as? (safe cast)
    - Nullable types: Type? notation
    - Null safety: ?., ?:, !!, let
    - Operators: arithmetic (+, -, *, /, %, ++, --)
    - Relational operators: ==, !=, >, <, >=, <=
    - Logical operators: &&, ||, !
    - Assignment operators: =, +=, -=, *=, /=, %=
    - Bitwise operators: and, or, xor, inv, shl, shr, ushr
    - Range operator: ..
    - in operator for range checking
    - Elvis operator: ?:
    - Safe call operator: ?.
    - Not-null assertion: !!
  Projects built:
    - Null-safe calculator with Kotlin operators
    - Temperature converter with smart casts
    - Age calculator with null safety
    - BMI calculator with safe operations
    - Currency converter with type-safe design
    - Area calculator using when expressions
    - Compound interest calculator with ranges
    - String manipulation utility
  Practice: Solve 40 problems on variables, null safety, and operators
Control Flow & Expressions
  Topics:
    - If as an expression (returns value)
    - If-else expression
    - If-else-if ladder
    - When expression: Kotlin's switch replacement
    - When with values, ranges, types
    - When without argument
    - When as expression vs statement
    - While loop
    - Do-while loop
    - For loop with ranges
    - For loop with collections
    - Iterating with indices: forEachIndexed
    - Until vs downTo in ranges
    - Step in ranges
    - Break and continue
    - Labeled break and continue
    - Nested loops and loop patterns
    - repeat function for simple loops
  Projects built:
    - Number guessing game with when expression
    - Grade calculator using when
    - Even/odd checker with ranges
    - Prime number checker and generator
    - Multiplication table with for loops
    - Factorial calculator using ranges
    - Fibonacci series generator
    - Pattern printing (stars, pyramids, diamonds)
    - FizzBuzz challenge with when
    - Menu-driven calculator
    - Simple ATM simulation with when
  Practice: Solve 45 control flow and expression problems
Functions & Collections Basics
  Topics:
    - Functions: fun keyword
    - Function parameters and return types

- Unit return type (like void)
- Single-expression functions
- Default parameter values
- Named arguments
- Variable number of arguments: vararg
- Extension functions concept
- Infix functions
- Local functions (nested functions)
- Higher-order functions introduction
- Lambda expressions basics
- Arrays: arrayOf, intArrayOf, etc.
- Array access and modification
- Lists: listOf (immutable), mutableListOf
- Sets: setOf, mutableSetOf
- Maps: mapOf, mutableMapOf
- Collection operations: size, isEmpty, contains

Projects built:
- Function library for mathematical operations
- Extension functions for String utilities
- Collection statistics calculator
- Array manipulation utilities
- List-based todo manager
- Map-based contact book
- Set operations demonstrator
- Higher-order function examples
- Lambda-based filtering
- Custom collection utilities

Practice: Create 30 functions, solve 35 collection problems

Month 3: Object-Oriented Programming & Functional Concepts (Weeks 8-13)

Classes, Objects & OOP in Kotlin

Topics:
- Classes: class keyword
- Primary constructor in class header
- Secondary constructors
- init blocks for initialization
- Properties: val and var in classes
- Getters and setters (automatic and custom)
- Backing fields: field keyword
- Late initialization: lateinit var
- Lazy initialization: lazy delegation
- Creating objects (no 'new' keyword)
- Visibility modifiers: public, private, protected, internal
- Data classes for data holding
- Copy function in data classes
- Destructuring declarations
- Object declarations (singleton)
- Companion objects (like static in Java)
- Object expressions (anonymous objects)

Projects built:
- Student data class with properties
- Bank Account class with encapsulation
- Book data class for library
- Employee class with companion object
- Rectangle/Circle classes with properties
- Car class with lateinit properties
- Singleton configuration object
- Product data class for inventory
- Person class with destructuring
- Date utility with companion functions

Practice: Create 25 different classes modeling real-world entities

Inheritance, Interfaces & Polymorphism

Topics:
- Inheritance with open keyword
- Classes are final by default
- Open functions for overriding
- Override keyword (mandatory)
- Calling superclass: super keyword
- Abstract classes and members
- Interfaces in Kotlin
- Multiple interface implementation
- Interface properties
- Default implementations in interfaces
- Resolving conflicts in multiple inheritance
- Sealed classes for restricted hierarchies
- Sealed class subtypes
- Enum classes
- Enum with properties and methods
- Polymorphism and type checking

- Smart casts in polymorphism
Projects built:
- Animal hierarchy with sealed classes
- Shape hierarchy with abstract classes
- Vehicle system with interfaces
- Employee hierarchy with polymorphism
- Payment system with sealed classes
- School management (Person -> Student, Teacher)
- E-commerce with product categories
- State machine using sealed classes
- Enum-based menu system
- Multiple interface implementation demo
Practice: Build 20 inheritance hierarchies, solve 30 polymorphism problems
Functional Programming in Kotlin
Topics:
- First-class functions
- Lambda expressions syntax
- Lambda with receivers
- Anonymous functions
- Higher-order functions
- Function types: (Int) -> String
- Returning functions from functions
- Closures and capturing variables
- Inline functions for performance
- noinline and crossinline
- Standard library functions: let, run, with, apply, also
- Collection operations: map, filter, reduce
- flatMap and flatten
- fold and reduce operations
- Sequences for lazy evaluation
- Function composition
- Currying and partial application
Projects built:
- Functional collection processor
- Lambda-based calculator
- DSL (Domain Specific Language) creation
- Scope functions showcase
- Lazy sequence operations
- Function composition utilities
- Higher-order function library
- PHASE 1 MINI CAPSTONE: Functional Library Management System
- Features: Immutable data, functional operations, sealed classes, DSL
Practice: Solve 40 functional programming problems
Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---
PHASE 2: ADVANCED KOTLIN & MODERN FEATURES (MONTHS 4-6, WEEKS 14-26)

Master advanced Kotlin features, coroutines, DSLs, delegation, generics, and professional coding patterns.

Month 4: Advanced Language Features & Coroutines (Weeks 14-18)
Generics & Type System
Topics:
- Generic types in Kotlin
- Generic functions
- Generic constraints: where clause
- Variance: in, out, invariance
- Covariance with out
- Contravariance with in
- Star projection: *
- Reified type parameters
- Type erasure and reified
- Generic class inheritance
- Type aliases for complex types
- Inline classes (value classes)
- Phantom types pattern
- Generic builders
- Type-safe builders with generics
Projects built:
- Generic repository pattern
- Type-safe builder DSL
- Generic data structures (Stack, Queue)
- Variance examples with collections
- Reified function utilities
- Type-safe API client
- Generic validation framework
- Phantom types for compile-time safety

  Practice: Create 20 generic classes, solve 30 variance problems
Delegation & Property Delegates
  Topics:
    - Class delegation with by keyword
    - Delegation pattern in Kotlin
    - Property delegation
    - Lazy delegate for late initialization
    - Observable delegate for property observation
    - Vetoable delegate
    - Storing properties in a map
    - Custom delegates creation
    - ReadOnlyProperty and ReadWriteProperty
    - provideDelegate for dynamic delegation
    - Delegated properties in interfaces
    - Multiple delegation
    - Delegation vs inheritance trade-offs
  Projects built:
    - Lazy loading system
    - Observable property pattern
    - Preferences management with delegation
    - Custom logging delegate
    - Validation delegate
    - Cache delegate implementation
    - Thread-safe property delegate
    - Map-backed properties example
  Practice: Implement 15 different delegates, solve 25 delegation problems
Collections Deep Dive
  Topics:
    - List operations: filter, map, flatMap, fold
    - Set operations: union, intersect, subtract
    - Map operations: mapKeys, mapValues, filterKeys
    - Grouping: groupBy, groupingBy
    - Partitioning: partition
    - Zipping and unzipping
    - Windowed and chunked
    - Aggregate operations: sumOf, maxOf, minOf
    - Sorting: sortedBy, sortedWith
    - Distinct and distinctBy
    - Take, drop, slice operations
    - Associate operations
    - Sequences vs Collections performance
    - asSequence for lazy operations
    - generateSequence for infinite sequences
  Projects built:
    - Advanced collection processor
    - Data transformation pipeline
    - Student grade analyzer
    - E-commerce order processor
    - Log file analyzer
    - Sequence-based file reader
    - Performance comparison tool
    - Custom collection operations library
  Practice: Solve 50 advanced collection problems
Coroutines Fundamentals
  Topics:
    - Asynchronous programming need
    - Coroutines vs threads
    - Suspend functions
    - launch coroutine builder
    - async and await
    - runBlocking for blocking code
    - Coroutine context and dispatchers
    - Dispatchers.Main, IO, Default, Unconfined
    - withContext for switching context
    - Coroutine scope
    - GlobalScope vs structured concurrency
    - Job and Deferred
    - Cancellation and timeouts
    - Exception handling in coroutines
    - supervisorScope for independent children
  Projects built:
    - Parallel API calls with async
    - File downloader with coroutines
    - Timeout handling example
    - Cancellation demonstration
    - Exception handling in coroutines
    - Background task processor
    - Coroutine-based calculator
    - Multi-threaded data processor

    Practice: Build 15 coroutine-based applications
  Advanced Coroutines & Flow
    Topics:
      - Channels for communication
      - Channel types: rendezvous, buffered, unlimited
      - produce and consumeEach
      - Flow for async streams
      - Flow builders: flow, flowOf, asFlow
      - Flow operators: map, filter, transform
      - Terminal flow operators: collect, toList
      - Flow context and dispatchers
      - Flow cancellation
      - Flow exceptions
      - StateFlow and SharedFlow
      - Cold vs hot flows
      - Flow combinations: combine, zip, merge
      - flatMapConcat, flatMapMerge, flatMapLatest
      - Flow buffering and conflation
    Projects built:
      - Real-time data stream processor
      - Channel-based producer-consumer
      - Flow-based search implementation
      - StateFlow example (state management)
      - SharedFlow for events
      - Flow transformation pipeline
      - Reactive UI updates with Flow
      - Network request with Flow
    Practice: Create 12 Flow-based systems

Month 5: DSLs, Reflection & Professional Patterns (Weeks 19-22)
  Domain-Specific Languages (DSLs)
    Topics:
      - What are DSLs?
      - Type-safe builders
      - Lambda with receiver
      - Extension functions in DSLs
      - Infix notation for DSLs
      - Operator overloading
      - invoke operator
      - Creating HTML DSL
      - Creating SQL-like DSL
      - Creating test DSL
      - DslMarker annotation
      - Context receivers (Kotlin 1.6+)
      - Building fluent APIs
      - DSL scope control
      - Real-world DSL examples
    Projects built:
      - HTML builder DSL
      - JSON builder DSL
      - SQL query DSL
      - Configuration DSL
      - Test specification DSL
      - Route definition DSL
      - UI layout DSL
      - Build script DSL
    Practice: Build 10 different DSLs
  Reflection & Annotations
    Topics:
      - Kotlin reflection API
      - Class references: ::class
      - KClass, KFunction, KProperty
      - Callable references
      - Function references: ::functionName
      - Property references
      - Bound and unbound references
      - Reflection for generics
      - Annotations in Kotlin
      - Creating custom annotations
      - Annotation targets
      - Annotation retention
      - Annotation parameters
      - Reading annotations at runtime
      - Annotation processing
      - Meta-annotations
    Projects built:
      - Dependency injection framework
      - JSON serializer using reflection
      - Validation framework with annotations

- ORM mapper with reflection
- Custom test framework
- Route mapping with annotations
- Configuration reader
- Plugin system with reflection

Practice: Build 8 reflection-based utilities

Design Patterns in Kotlin

Topics:

- Singleton with object declaration
- Factory pattern with companion object
- Builder pattern with DSL
- Observer pattern with delegation
- Strategy pattern with lambdas
- Decorator pattern with delegation
- Adapter pattern
- Proxy pattern with lazy
- Command pattern
- State pattern with sealed classes
- Chain of responsibility
- Template method
- Visitor pattern
- Repository pattern
- Dependency injection patterns

Projects built:

- Notification system (Observer)
- Payment strategies implementation
- Logging decorator
- State machine with sealed classes
- Command processor
- Repository pattern implementation
- DI container creation
- Plugin architecture

Practice: Implement 20 design patterns in Kotlin style

Testing in Kotlin

Topics:

- Unit testing with JUnit 5
- Kotlin-specific test features
- Assertions with Kotlin
- Kotest framework
- Property-based testing
- Mocking with MockK
- Coroutine testing
- Testing suspending functions
- TestCoroutineDispatcher
- Testing flows
- Turbine for Flow testing
- Test-driven development in Kotlin
- Behavior-driven development
- Integration testing
- Test coverage with JaCoCo

Projects built:

- Comprehensive test suite
- Coroutine tests
- Flow testing examples
- Property-based tests
- BDD-style tests with Kotest
- Mock-based testing
- TDD practice projects
- Integration test suite

Practice: Write tests for all previous projects, 85%+ coverage

Month 6: Server-Side Kotlin with Ktor (Weeks 23-26)

Ktor Framework - Part 1

Topics:

- Introduction to Ktor
- Ktor vs Spring Boot
- Setting up Ktor project
- Routing in Ktor
- HTTP methods handling
- Request and response
- Content negotiation
- JSON serialization with kotlinx.serialization
- Status pages and error handling
- Authentication and authorization
- JWT authentication in Ktor
- Sessions in Ktor
- Static content serving

Projects built:

- Hello World REST API with Ktor

```
        - CRUD API for products
        - User authentication API with JWT
        - File upload service
        - Static website hosting
        - API with JSON responses
        - Session-based auth
      Practice: Build 8 Ktor-based APIs
    Ktor Framework - Part 2
      Topics:
        - Database integration with Exposed
        - Exposed DSL and DAO
        - Database transactions
        - Connection pooling
        - WebSockets in Ktor
        - Real-time communication
        - Testing Ktor applications
        - Client-side Ktor HTTP client
        - Dependency injection in Ktor (Koin)
        - Monitoring and metrics
        - Ktor deployment
        - Docker containerization
      Projects built:
        - Full-stack CRUD with database
        - Real-time chat with WebSockets
        - API testing suite
        - HTTP client for third-party APIs
        - Deployed Ktor application
        - Monitored production service
      Practice: Build 6 complete Ktor applications
    Database Access with Kotlin
      Topics:
        - JDBC with Kotlin
        - Exposed framework deep dive
        - Exposed DSL queries
        - Exposed DAO pattern
        - Relationships in Exposed
        - Transactions in Exposed
        - jOOQ with Kotlin
        - SQLDelight for type-safe SQL
        - MongoDB with Kotlin
        - Redis with Kotlin
        - Database migrations
        - Connection pool configuration
      Projects built:
        - Exposed-based ORM project
        - Type-safe database queries
        - Multi-table operations
        - NoSQL integration
        - Migration scripts
        - Connection pool setup
        - Database performance optimization
      Practice: Build 8 database-driven applications
    Build Tools & Kotlin Scripting
      Topics:
        - Gradle with Kotlin DSL
        - Gradle Kotlin DSL syntax
        - Build script configuration
        - Dependencies management
        - Multi-module projects
        - Publishing libraries
        - Kotlin scripting (.kts files)
        - Custom Gradle tasks
        - Maven with Kotlin
        - Continuous integration setup
        - Git best practices for Kotlin
      Projects built:
        - Gradle Kotlin DSL project
        - Multi-module application
        - Published library to Maven Central
        - Custom Gradle plugin
        - Kotlin scripts for automation
        - CI/CD pipeline setup
      Practice: Configure all projects with Gradle Kotlin DSL
      Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

---------------------------------------------------------------
PHASE 3: ANDROID DEVELOPMENT MASTERY (MONTHS 7-9, WEEKS 27-39)
---------------------------------------------------------------
```

Master modern Android development with Kotlin, Jetpack Compose, Material Design, Architecture Components, and professional app development.

Month 7: Android Fundamentals & Jetpack Compose (Weeks 27-31)
Android Basics & Setup
Topics:
- Android platform overview
- Android architecture
- Android SDK and tools
- Android Studio setup and features
- Project structure in Android
- Gradle build system for Android
- AndroidManifest.xml
- Activities and lifecycle
- Intents: explicit and implicit
- Activity navigation
- Passing data between activities
- Fragment basics
- Fragment lifecycle
- Fragment transactions
- UI with XML layouts
- View and ViewGroup
- Common layouts: LinearLayout, RelativeLayout, ConstraintLayout

Projects built:
- Hello Android app
- Multi-activity application
- Intent-based navigation
- Fragment-based UI
- Data passing between screens
- Basic calculator app
- Profile display app
- Navigation drawer app

Practice: Build 10 basic Android apps

Jetpack Compose Fundamentals
Topics:
- Introduction to Jetpack Compose
- Declarative UI paradigm
- Composable functions
- @Composable annotation
- Compose preview
- State in Compose: remember, mutableStateOf
- Recomposition concept
- State hoisting
- Modifiers in Compose
- Layout composables: Column, Row, Box
- LazyColumn and LazyRow
- Text, Image, Button composables
- TextField and user input
- Spacer and Divider
- Card and Surface
- Material Design 3 components
- Theming in Compose

Projects built:
- Hello Compose app
- Counter app with state
- Todo list with LazyColumn
- Form with TextFields
- Material Design 3 app
- Custom theme implementation
- Card-based UI
- Image gallery with LazyGrid

Practice: Build 12 Compose-based UIs

Advanced Compose & Navigation
Topics:
- Compose Navigation library
- NavHost and NavController
- Route-based navigation
- Passing arguments in navigation
- Bottom navigation with Compose
- Navigation drawer in Compose
- Nested navigation graphs
- Side effects in Compose: LaunchedEffect, DisposableEffect
- SideEffect, derivedStateOf
- remember vs rememberSaveable
- ViewModel integration with Compose
- Lifecycle-aware composables
- Animation in Compose
- AnimatedVisibility, animateContentSize
- Custom animations

```
    Projects built:
      - Multi-screen navigation app
      - Bottom navigation app
      - Navigation with arguments
      - Animated UI transitions
      - ViewModel-backed Compose app
      - Side effect demonstrations
      - Complex navigation flow
      - Login flow with navigation
    Practice: Build 10 navigation-based apps
  Android Architecture Components
    Topics:
      - MVVM architecture pattern
      - ViewModel and LiveData
      - StateFlow and SharedFlow in Android
      - Repository pattern
      - Data layer architecture
      - Use cases / Interactors
      - Dependency injection with Hilt
      - Hilt setup and annotations
      - @HiltAndroidApp, @HiltViewModel
      - Providing dependencies
      - Scopes in Hilt
      - Testing with Hilt
      - Clean Architecture in Android
      - Separation of concerns
    Projects built:
      - MVVM app with ViewModel
      - Repository pattern implementation
      - Hilt dependency injection setup
      - Clean architecture project
      - Multi-layer application
      - Testable architecture demo
      - State management with StateFlow
      - Use case layer implementation
    Practice: Refactor 8 apps to clean architecture
  Local Data Storage
    Topics:
      - SharedPreferences for simple data
      - DataStore (Preferences and Proto)
      - Room database introduction
      - Entity, DAO, Database classes
      - Room with Kotlin Coroutines
      - Room with Flow
      - Database migrations
      - Relationships in Room
      - Type converters
      - Database testing
      - File storage basics
      - Internal vs external storage
    Projects built:
      - Preferences-based settings screen
      - DataStore implementation
      - Room database CRUD app
      - Notes app with Room
      - Contact manager with Room
      - Offline-first architecture
      - Database migration example
      - Complex relationships in Room
    Practice: Build 10 data persistence apps

Month 8: Networking, APIs & Advanced Android (Weeks 32-36)
  Networking & REST APIs
    Topics:
      - HTTP basics for Android
      - Retrofit library for REST APIs
      - Setting up Retrofit
      - Defining API interfaces
      - GET, POST, PUT, DELETE requests
      - Query parameters and path variables
      - Request and response models
      - JSON parsing with Gson/Moshi
      - kotlinx.serialization in Android
      - OkHttp interceptors
      - Authentication headers
      - Error handling in network calls
      - Coroutines with Retrofit
      - Loading states management
      - Offline caching strategies
```

```
  Projects built:
    - Weather app with API
    - News reader app
    - Movie database app (TMDB API)
    - REST API consumer
    - Authenticated API calls
    - Offline-first with Room cache
    - Error handling examples
    - Loading state UI
  Practice: Build 10 API-consuming apps
Image Loading & Media
  Topics:
    - Coil library for image loading
    - Glide library
    - Image caching strategies
    - Placeholder and error images
    - Image transformations
    - AsyncImage in Compose
    - Loading images from network
    - Camera integration
    - Gallery access
    - Image picker
    - Video playback: ExoPlayer
    - Media permissions
    - MediaStore API
    - Uploading images to server
  Projects built:
    - Image gallery app with Coil
    - Photo viewer with zoom
    - Camera app
    - Image picker implementation
    - Video player app
    - Image upload to server
    - Media browser
    - Photo editor basics
  Practice: Build 8 media-handling apps
Advanced UI & Material Design
  Topics:
    - Material Design 3 principles
    - Dynamic theming
    - Color schemes in Material 3
    - Custom themes and colors
    - Typography in Compose
    - Scaffold and top app bar
    - Bottom sheet in Compose
    - Dialogs and alerts
    - Snackbars and toasts
    - Tabs with TabRow
    - Chips and badges
    - Progress indicators
    - Swipe to refresh
    - Pull to refresh
    - Custom composables creation
  Projects built:
    - Material Design 3 showcase app
    - Custom themed app
    - Bottom sheet examples
    - Dialog-based workflows
    - Tab-based navigation
    - Complex custom UI components
    - Accessible UI implementation
    - Responsive layouts
  Practice: Design 10 Material Design apps
Background Work & Services
  Topics:
    - WorkManager for background tasks
    - One-time and periodic work
    - Constraints for WorkManager
    - Chaining work requests
    - Monitoring work status
    - Foreground services
    - Notifications in Android
    - NotificationCompat builder
    - Notification channels
    - Push notifications with Firebase
    - FCM integration
    - Handling notification clicks
    - AlarmManager for scheduling
    - JobScheduler basics
```

```
    Projects built:
      - Background sync with WorkManager
      - Periodic data refresh
      - Notification system
      - Push notification handler
      - Foreground service example
      - Reminder app with AlarmManager
      - Download manager
      - Background upload service
    Practice: Build 8 background processing apps
  Testing Android Apps
    Topics:
      - Unit testing in Android
      - JUnit for Android
      - Testing ViewModels
      - Testing coroutines in Android
      - Mocking with MockK
      - Instrumented tests
      - Espresso for UI testing
      - Compose UI testing
      - Testing navigation
      - Testing Room database
      - Testing network calls
      - Test doubles and fakes
      - Screenshot testing
      - Code coverage in Android
    Projects built:
      - Comprehensive unit tests
      - ViewModel test suite
      - UI tests with Espresso
      - Compose UI tests
      - Integration tests
      - E2E testing
      - TDD Android app
      - Tested production app
    Practice: Write tests for all Android projects, 80%+ coverage

Month 9: Ship-Quality Android: Performance, Publishing & Security (Weeks 37-39)
  Performance & Optimization
    Topics:
      - Android performance profiling
      - CPU profiler in Android Studio
      - Memory profiler
      - Network profiler
      - Layout Inspector
      - Detecting memory leaks
      - LeakCanary integration
      - App startup optimization
      - Lazy loading strategies
      - Image optimization
      - Database query optimization
      - Compose performance
      - Recomposition optimization
      - Battery optimization
      - App size reduction
    Projects built:
      - Profiled and optimized app
      - Memory leak fixes
      - Performance benchmark
      - Optimized startup time
      - Lazy-loaded content
      - APK size optimization
      - Battery-efficient app
    Practice: Optimize all previous Android apps
  Publishing & Deployment
    Topics:
      - Preparing app for release
      - App signing with keystore
      - ProGuard and R8
      - Code obfuscation
      - Building release APK/AAB
      - Google Play Console setup
      - Creating store listing
      - Screenshots and graphics
      - App versioning strategy
      - Release management
      - Beta testing with Play Console
      - Internal testing tracks
      - App updates and migrations
```

```
        - Analytics integration: Firebase Analytics
        - Crash reporting: Firebase Crashlytics
      Projects built:
        - Signed release build
        - Play Store ready app
        - Complete store listing
        - Beta test distribution
        - Analytics-enabled app
        - Crash reporting setup
        - Update mechanism
      Practice: Prepare 3 apps for Play Store
    Security Best Practices
      Topics:
        - Android security overview
        - Secure data storage
        - Encrypted SharedPreferences
        - Keystore system
        - Biometric authentication
        - Certificate pinning
        - ProGuard for security
        - SQL injection prevention
        - Secure network communication
        - HTTPS enforcement
        - API key protection
        - Code obfuscation
        - Root detection
        - SafetyNet API
        - OWASP mobile security
      Projects built:
        - Secured data storage app
        - Biometric auth implementation
        - Certificate pinned API calls
        - Encrypted database
        - Secure authentication flow
        - Obfuscated production app
      Practice: Security audit all Android apps
      Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

---------------------------------------------------------------
PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)
---------------------------------------------------------------
The capstone month: spec, build, ship and defend, then the final exam that decides the
certificate.

  Month 10: Capstone and the Final Exam (Weeks 40-44)
    Capstone Spec and Setup
      Topics:
        - Choosing the capstone: a complete Android app with real data, offline behavior and polished
          Compose UI
        - A one-page spec: scope, milestones, stack
        - Repository and project skeleton ready on day one
        - Instructor sign-off before the build begins
      Projects built:
        - Approved capstone spec plus the running skeleton
      Practice: Pitch the spec to the batch and absorb one hard question
    Capstone Build
      Topics:
        - Two focused weeks of building in vertical slices
        - Instructor checkpoints and honest scope cuts
        - Using the whole course: Kotlin depth, coroutines, Compose, architecture and testing
        - Daily commits and an always-working build
      Projects built:
        - The capstone, feature-complete with version control
      Practice: A working build at the end of every session
    Ship and Polish
      Topics:
        - The finishing pass: edge cases, errors, documentation
        - Store-readiness pass: assets, versioning and the honest release checklist
        - A README worth reading and a demo worth watching
        - A full-course spiral review before the exam
      Projects built:
        - The capstone shipped, documented and rehearsed
      Practice: Two mock demos, each tighter than the last
    Demo Day and the Final Exam
      Topics:
        - Presenting the capstone: live demo plus the hardest bug story
        - The honest map of next roads: production Android work, Kotlin backend services, or
          multiplatform
```

- Where our other courses continue the journey
Projects built:
- Demo day presentation delivered to the batch
Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Kotlin programs from fundamentals through coroutine-based tools
- Phase 2: idiomatic Kotlin utilities and flow-based systems
- Phase 3: Compose apps with architecture, Room data and networking
- Phase 4: the capstone app, store-grade and defended at demo day

Total projects: 20+ real builds, crowned by a store-grade capstone app

SKILLS THE STUDENT WILL HAVE AT THE END

- Kotlin to real depth: null-safety, functional style, coroutines and flows
- Jetpack Compose: modern Android UI as a first language
- Architecture: state, lifecycles and structure that survive growth
- Data and networking: Room, Retrofit-class clients and offline judgment
- Production habits: testing, releases and honest store literacy

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Production Android roles: this is the modern stack they hire for
- Kotlin backend services: the language carries straight across
- Multiplatform later, entered when it earns its complexity
- Our full-stack course for the web half of product building

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 3-6 LPA (Junior Kotlin/Android Developer)
Growing roles: Rs 6-10 LPA (Android Developer); Rs 10-18 LPA (Senior Android Developer)
Experienced roles: Rs 15-30 LPA (Senior Kotlin Developer / Tech Lead); Rs 20-45 LPA (Lead Developer / Architect)
Freelance: Rs 1,500-6,000/hour based on expertise
International usa: $90k-200k USD (Android/Kotlin Developer to Senior)
International europe: €55k-130k EUR based on country

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this Kotlin and Android course cover?
A: Kotlin as a real language first, null-safety, functional style, coroutines and flows, then modern Android: Jetpack Compose UI, architecture that survives rotation and growth, Room data, networking, testing and honest store literacy, ending with a store-grade capstone app and the final exam. Compose-first throughout: the stack Google actually recommends in 2026.

Q: How long does it take to learn Kotlin and Android properly?
A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two more months in hand for learners who need them. The three teaching phases build the language and its real-world domain; the finale month ships a capstone and sits the final exam.

Q: How am I assessed? Is the certificate just for attending?
A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Do I need to know Java first?
A: No: Kotlin is taught from zero and is the better first language for Android today. Java interop is explained where it matters (older codebases), but modern Android development, and this course, runs Kotlin-first with Compose.

Q: Will my app actually go on the Play Store?
A: The course builds to store-grade discipline and walks the release process honestly: developer account costs, review rules, testing tracks. Publishing itself is your call and your account; what the course guarantees is an app and a release checklist that are genuinely ready for it.

Q: Can I try a class before enrolling?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/complete-kotlin-programming-masterclass-professional
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 92 OF 137: KS2 SATS MATHS: ARITHMETIC, REASONING AND TIMES TABLES
==========

Course link: https://learn.modernagecoders.com/courses/ks2-sats-maths-course
Subject area: UK Primary Mathematics
Designed for: Elementary school
Who can join: Children in Years 4, 5 and 6 at primary schools in England, ages 8 to 11
Course length: 8 months (32 weeks), timed to finish before the May tests
Time needed: 2 live classes/week + 20 minutes of practice on most days
Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

KS2 SATs maths course online for Years 4 to 6: times tables for the multiplication check, the arithmetic paper and both reasoning papers, taught live.

In one line: Instant times tables, a calm arithmetic paper and reasoning answers that show their thinking, built up over eight months rather than crammed in April.

ABOUT THE PROGRAMME

The key stage 2 maths test is three papers. The test framework sets them out plainly: Paper 1 is arithmetic, 40 marks in 30 minutes, and Papers 2 and 3 are mathematical reasoning, 35 marks and 40 minutes each, so 110 marks across 110 minutes in total. In 2027 the maths papers are timetabled for Wednesday 12 May (Papers 1 and 2) and Thursday 13 May (Paper 3). Results come back as a scaled score between 80 and 120, and a score of at least 100 meets the expected standard. Underneath all of it sit the times tables. The national curriculum expects children to have memorised the tables up to 12 by the end of Year 4, and the multiplication tables check tests exactly that: 25 questions, six seconds each, taken in June of Year 4. A child who still counts up in fives to find 7 times 5 loses time on every paper, so this course starts there. The course runs in four phases: times tables to instant recall, the arithmetic paper method by method, the reasoning papers question type by question type, and finally full papers under real timing. Classes are live and small, so the teacher sees each child's working, not just the answer, and parents get a short written note every month on what has moved and what is next.

WHAT MAKES THIS COURSE DIFFERENT

- Built from the government's own key stage 2 mathematics test framework: the three papers, their marks and timings, and the resource list children actually use
- Times tables first, because the multiplication tables check and every SATs paper lean on instant recall
- The arithmetic paper taught as a set of written methods, each practised until it is automatic and checked
- Reasoning taught by question type: multi-step word problems, explain-your-answer questions, measures, geometry and statistics
- Full papers under real timing in the last two months, marked against the published mark schemes
- Calm by design: short daily practice and clear progress notes for parents, not late-night cramming

THE LEARNING PATH, STEP BY STEP

Phase 1: Times tables to instant recall: every table to 12 by 12, the related division facts, and timed practice in the multiplication check format
Phase 2: The arithmetic paper: place value, the four written methods, fractions, decimals and percentages as calculations
Phase 3: The reasoning papers: word problems, measures, geometry, statistics and explaining an answer
Phase 4: Full papers under timing: past and practice papers, marked and reviewed, with a calm plan for test week

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Working on Year 4, 5 or 6 maths at school. The first week's diagnostic shows where each child is, and the plan starts from there
School system: Built for the key stage 2 tests at primary schools in England. Children elsewhere are welcome for the content, but the papers are England's
Equipment: Pencil, ruler, protractor, a small mirror, squared paper, and a device with stable internet for live classes
Timing: Ideally started in the autumn of Year 6, or earlier in Years 4 and 5 for times tables and arithmetic. Later joiners get a shorter plan focused on papers

WHO THIS COURSE IS FOR

Year 6 children: Year 6 children preparing for the May tests who want steady, structured practice
Year 4 children: Year 4 children getting ready for the multiplication tables check
Year 5 children: Year 5 children who want arithmetic and times tables secure a year early
Children needing confidence: Children who know the maths but freeze or rush under time
Children aiming higher: Children who are already secure and want the harder reasoning questions

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: TIMES TABLES TO INSTANT RECALL (MONTHS 1-2, WEEKS 1-8)

Every table up to 12 by 12 and its division facts, learned through patterns first and speed second. Year 4 children practise in the format of the multiplication tables check; older children use it as a warm-up for the papers.

Month 1: Tables by Pattern (Weeks 1-4)
Weeks 1-2: Diagnostic and the Easy Tables
Topics:
- A short diagnostic: which facts are instant, which are counted, which are guessed
- The 2, 5 and 10 tables and the patterns that make them quick
- Doubling to reach the 4 and 8 tables
- Commutativity: why learning 3 times 7 also gives 7 times 3
- The 11 table and its pattern up to 11 times 9
- A personal fact chart: green for instant, amber for slow, red for unknown

Projects built:
- Fact chart started: every fact up to 12 by 12 coloured by speed after the diagnostic

Practice: Ten minutes of mixed facts on most days, focusing on amber facts
Weeks 3-4: The Harder Tables
Topics:
- The 3, 6 and 9 tables and how they connect
- The 7 table, the one most children find hardest, with memory hooks
- The 12 table built from the 10 and 2 tables
- Square numbers up to 12 by 12
- Spotting which facts are still slow and why
- Using known facts to work out unknown ones

Projects built:
- Fact chart updated: amber and red facts listed with a hook for each

Practice: Daily mixed practice weighted towards the 6, 7, 8 and 12 tables
Assessment: Month 1 check: 25 mixed facts, untimed, then the same 25 timed

Month 2: Speed and Division Facts (Weeks 5-8)
Weeks 5-6: Division Facts and Missing Numbers
Topics:
- Division facts from each table, read the other way
- Missing number questions such as 6 times something equals 42
- Fact families: four facts from three numbers
- Multiplying by 10 and 100 using place value
- Scaling a known fact: from 3 times 4 to 30 times 4
- Recognising factors and multiples
Projects built:
- Fact family cards made for the ten facts the child finds slowest
Practice: Mixed multiplication and division facts with a short timer
Weeks 7-8: Six-Second Recall
Topics:
- Practice in the multiplication check format: 25 questions, six seconds each
- Typing answers quickly and accurately on a keyboard or tablet
- Staying calm when one fact is missed and moving to the next
- Re-testing every fact that was amber in month 1
- Using tables inside short calculations
- Keeping recall instant with five minutes a day
Projects built:
- Fact chart completed: the aim is every fact green
Practice: Three short timed checks a week in the check format
Assessment: Month 2 check: a full 25-question timed check, scored out of 25

PHASE 2: THE ARITHMETIC PAPER (MONTHS 3-4, WEEKS 9-16)

Paper 1 is 40 marks in 30 minutes, all calculation. Each written method is taught, practised
and checked until it is automatic, then mixed under time.

Month 3: Whole Numbers and Written Methods (Weeks 9-12)
Weeks 9-10: Place Value and Addition and Subtraction
Topics:
- Place value to ten million and reading large numbers
- Column addition with several carries
- Column subtraction with exchanging across zeros
- Adding and subtracting decimals by lining up the point
- Estimating before calculating to catch slips
- Inverse operations as a check
Projects built:
- Method book started: one worked example per method, written by the child
Practice: 20 column calculations a week, each checked by estimation
Weeks 11-12: Multiplication and Division
Topics:
- Short multiplication by a one-digit number
- Long multiplication by a two-digit number
- Short division, including remainders
- Long division by a two-digit number
- Order of operations with brackets
- Choosing a mental method when the numbers allow it
Projects built:
- Method book entry: long multiplication and long division set out step by step
Practice: 15 multiplication and division calculations a week
Assessment: Month 3 test: a half-length arithmetic paper, 20 marks in 15 minutes

Month 4: Fractions, Decimals and Percentages (Weeks 13-16)
Weeks 13-14: Fraction Calculations
Topics:
- Equivalent fractions and simplifying
- Adding and subtracting fractions with different denominators
- Mixed numbers and improper fractions
- Multiplying fractions, including by a whole number
- Dividing a fraction by a whole number
- Fractions of amounts
Projects built:
- Method book entry: every fraction method with a worked example
Practice: 20 fraction calculations a week, answers simplified
Weeks 15-16: Decimals, Percentages and the Full Paper
Topics:
- Multiplying and dividing decimals by 10, 100 and 1000
- Multiplying a decimal by a whole number
- Percentages of amounts
- Squares, cubes and simple powers
- Working through a full arithmetic paper in 30 minutes
- Checking strategy for the last five minutes

      Projects built:
        - Arithmetic tracker: score and time recorded for every practice paper
      Practice: One full arithmetic paper a week under timing
      Assessment: Month 4 test: a full arithmetic paper, 40 marks in 30 minutes, marked and
      reviewed

PHASE 3: THE REASONING PAPERS (MONTHS 5-6, WEEKS 17-24)

Papers 2 and 3 are 35 marks and 40 minutes each, and they test whether a child can use maths
in context and explain it. Each question type is taught on its own, then mixed.

  Month 5: Number Reasoning and Measures (Weeks 17-20)
    Weeks 17-18: Multi-Step Word Problems
      Topics:
        - Reading a word problem twice and finding the question
        - Bar models for comparison and part-whole problems
        - Multi-step problems that combine two or three operations
        - Ratio and scaling problems
        - Simple algebra: missing numbers and formulae in words
        - Showing working so method marks are possible
      Projects built:
        - Problem journal started: one solved problem per question type with a bar model
      Practice: 10 word problems a week, each with a written plan before the calculation
    Weeks 19-20: Measures, Money and Time
      Topics:
        - Converting units of length, mass and capacity
        - Money problems with change and value-for-money comparisons
        - Reading and calculating with time, including timetables
        - Perimeter and area of rectangles and compound shapes
        - Area of triangles and parallelograms
        - Volume of cuboids
      Projects built:
        - Problem journal entry: a measures question for each unit type
      Practice: 12 measures questions a week in reasoning-paper style
      Assessment: Month 5 test: a 20-minute reasoning section, marked against a published
      scheme

  Month 6: Geometry, Statistics and Explaining (Weeks 21-24)
    Weeks 21-22: Geometry and Statistics
      Topics:
        - Angles on a line, around a point and in triangles and quadrilaterals
        - Measuring and drawing angles with a protractor
        - Properties of 2-D and 3-D shapes, including circles
        - Coordinates in four quadrants, translations and reflections
        - Reading line graphs, tables and pie charts
        - Calculating and interpreting the mean
      Projects built:
        - Problem journal entry: geometry and statistics questions with labelled diagrams
      Practice: 12 geometry and statistics questions a week
    Weeks 23-24: Explain, Show and Check
      Topics:
        - Explain-your-answer questions and what a full answer includes
        - Always, sometimes or never true statements
        - Spotting and correcting a mistake in someone else's working
        - Checking that an answer is sensible in context
        - Mixed reasoning sets that do not announce their topic
        - Pacing a 40-minute paper
      Projects built:
        - Explanation bank: model answers to ten explain questions, written in the child's own words
      Practice: One full reasoning paper a week under timing
      Assessment: Month 6 test: a full reasoning paper, 35 marks in 40 minutes, marked and
      reviewed

PHASE 4: FULL PAPERS UNDER TIMING (MONTHS 7-8, WEEKS 25-32)

No new content. Full sets of three papers under real timing, marked against the published
mark schemes, with a short review after each one and a calm plan for test week.

  Month 7: Paper Sets (Weeks 25-28)
    Weeks 25-26: First Full Sets
      Topics:
        - A full set: arithmetic paper, then both reasoning papers on separate days
        - Marking against the published mark scheme
        - Sorting lost marks: recall, method, reading or carelessness
        - Re-teaching the two weakest areas
        - Times tables kept instant with short warm-ups
        - Using the resource list: ruler, protractor and mirror

Projects built:
- Paper tracker: scores for each paper on one chart

Practice: One full set of three papers each fortnight

Weeks 27-28: Targeted Repair

Topics:
- Short lessons on each child's own weak spots, taken from the tracker
- Arithmetic speed drills on the slowest methods
- Reasoning questions of the types most often dropped
- Reading questions aloud in the head before answering
- Leaving and returning to a hard question
- Checking answers in the final minutes

Projects built:
- Repair list: each weak area with the date it was fixed

Practice: Focused sets on two weak areas a week

Assessment: Month 7 checkpoint: a second full set of three papers, marked

Month 8: Ready for Test Week (Weeks 29-32)

Weeks 29-30: Final Sets

Topics:
- A final full set under real timing
- Review of every lost mark
- Mixed practice from the repair list
- Arithmetic in under 30 minutes with checking time left
- Reasoning pacing: 40 minutes per paper
- Confidence from seeing the tracker rise

Projects built:
- Paper tracker completed

Practice: Short daily mixed practice, no all-day sessions

Weeks 31-32: Calm Before the Tests

Topics:
- Light revision of methods from the method book
- Times tables warm-ups
- What to expect on each day of test week
- Sleep, breakfast and a normal routine
- A short talk with parents about keeping test week low-pressure
- Planning maths for the summer and Year 7

Projects built:
- Finished method book, problem journal and explanation bank, kept for Year 7

Practice: Ten minutes a day of light, familiar practice

Assessment: Final review with the teacher and a written note for parents

PROJECTS THE STUDENT BUILDS AND KEEPS

- A colour-coded times tables fact chart, filled to green
- A method book with a worked example of every written calculation method
- A problem journal with a solved example of every reasoning question type
- An explanation bank of model answers in the child's own words
- An arithmetic tracker and a full-paper tracker
- A repair list showing each weak area and when it was fixed

Total projects: A complete SATs maths kit: fact chart, method book, problem journal, explanation bank and paper trackers

SKILLS THE STUDENT WILL HAVE AT THE END

- Times tables to 12 by 12 recalled instantly, with division facts
- Every written method on the arithmetic paper, with checking
- Fractions, decimals and percentages as calculations
- Reasoning with measures, geometry and statistics
- Explaining an answer clearly enough to earn the mark
- Pacing a timed paper and checking in the final minutes

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, 45 to 60 minutes each depending on age and phase

Practice: About 20 minutes on most days, including a few minutes of times tables

Review: Practice marked with short comments; papers marked against published schemes

SUPPORT INCLUDED

Doubt support: WhatsApp support for parents between classes, with worked examples for stuck questions

Progress updates: A short written note to parents each month with tracker scores and next steps

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders

WHERE THIS CAN LEAD

  - A smooth start to Year 7 maths with times tables and written methods already secure
  - Our KS3 and 11 plus maths courses for children who want to keep going
  - Our mental maths and coding courses for children who enjoy number puzzles

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What is in the KS2 SATs maths test?
  A: Three papers. Paper 1 is arithmetic, 40 marks in 30 minutes. That is 110 marks over 110 minutes in total, set out in the government's key stage 2 mathematics test framework.

  Q: When are the KS2 SATs maths papers in 2027?
  A: The multiplication tables check for Year 4 runs in a two-week window from Monday 7 June 2027.

  Q: What score does my child need in SATs maths?
  A: Results are scaled scores from 80 to 120. A scaled score of at least 100 means the child has met the expected standard. The number of raw marks needed for 100 can change from year to year, because the difficulty of the tests varies.

  Q: What is the Year 4 multiplication tables check?
  A: A short on-screen check taken in June of Year 4. It checks instant recall, which is why this course spends its first two months on times tables.

  Q: Can children use a calculator in the SATs maths test?
  A: The resource list in the test framework is a pen or pencil, a ruler and an optional rubber for the arithmetic paper, plus an angle measurer or protractor and a mirror for the reasoning papers. A calculator is not on it, so this course teaches every calculation by hand.

  Q: Is SATs maths the same as 11 plus maths?
  A: No. SATs are national tests taken by Year 6 children in England. The 11 plus is a selection test set by some areas and schools, usually in Year 5 or early Year 6, with its own format. There is overlap in the maths, and we also run a separate 11 plus maths course.

  Q: Are the SATs changing?
  A: The government has said future assessment arrangements are under review after the Curriculum and Assessment Review, and that arrangements for the 2026 to 2027 academic year will stay the same. We follow the published framework and will update the course if anything changes.

  Q: How are the classes run?
  A: Live online classes with a real teacher, either in a group of 15 to 20 children at a similar level or one to one. Practice is short and frequent, and parents get a written note each month. Current fees for your region are shown in the pricing section of this page.

  Q: My child is in Year 4. Is it too early?
  A: Not for times tables and arithmetic. Year 4 is when the multiplication check happens, and secure tables make every later year easier. Younger children follow the first two phases at their own pace and join the paper practice later.

  Q: Can we try a lesson first?
  A: Yes. Book a free demo class. The teacher plays a few times tables games and arithmetic questions with your child, sees where they are, and suggests whether group or one-to-one teaching fits better. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ks2-sats-maths-course
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 93 OF 137: MATHS FOR DATA SCIENCE: STATISTICS, LINEAR ALGEBRA AND ML
==========

Course link: https://learn.modernagecoders.com/courses/data-analytics-mathematics-masterclass
Subject area: Data Science Mathematics
Designed for: Working professionals
Who can join: School-level maths assumed; built for college students and working professionals
Course length: 12-14 months (48-56 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Maths for data science, live online: statistics, probability, linear algebra, regression, ML mathematics, Bayesian methods and A/B testing in 12-14 months.

In one line: The difference between running models and understanding them is mathematics. This is that mathematics, taught honestly in twelve months.

ABOUT THE PROGRAMME

Every data career eventually hits the same wall: the tools are easy, the mathematics behind them is not, and interviews, promotions and real analytical judgment all live on the far side of that wall. This live online course teaches exactly the mathematics data work stands on: statistics that tell the truth, probability that calibrates uncertainty, the linear algebra inside every model, regression and classification understood from first principles, optimization, the mathematics of neural networks, Bayesian methods, time series, and the experimental design behind honest A/B testing. The course is deliberately practical about rigor: every concept is derived enough to be owned, then immediately exercised on real data questions, the kind analysts and data scientists actually face. Statistical malpractice gets named and corrected throughout: p-hacking, leakage, confounding, the graph that lies. By the end, a student does not just know the formulas; they know which analysis is honest and can defend it in an interview or a meeting. The pace is real: two live classes a week plus four to six hours of practice fits the syllabus in twelve months, and learners who need longer take up to fourteen, the batch adapts. Assessment is honest too: problem sets after every class, monthly mixed reviews that reach back deliberately, phase exams, and a final exam that decides the certificate, with focused revision and a free retest for those who fall short. Real knowledge, real work, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Mathematics first, tools second: understanding that transfers across every library and language
- Interview-honest coverage: the statistics, linear algebra and ML mathematics that data interviews actually probe
- Statistical malpractice named and corrected: p-hacking, leakage, confounding and misleading charts, studied on purpose
- Every concept lands on real data: derivations earn their keep in applied exercises the same week
- An honest 12-month arc with 2 months of margin: learners who need more time take up to 14, nobody is rushed
- Real assessment: weekly problem sets, monthly mixed reviews, and a final exam that decides the certificate, with focused revision and a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Statistics that tell the truth (Months 1-3): descriptive statistics, visualization, probability and the great distributions
Phase 2: Linear algebra and inference (Months 4-6): vectors to PCA, sampling, confidence intervals and hypothesis testing done honestly
Phase 3: The mathematics of machine learning (Months 7-9): regression from first principles, classification, trees, clustering and evaluation
Phase 4: Modern practice (Months 10-12): optimization, neural network mathematics, Bayesian methods, time series, A/B testing and the capstone

WHAT A STUDENT NEEDS BEFORE STARTING

Maths background: School mathematics (Class 12 or equivalent). Calculus is refreshed where needed; no university maths assumed
Coding background: Helpful but not required; the mathematics is the syllabus, with spreadsheets and light Python used as calculators
Equipment: A computer with a stable internet connection
Age: College students and working professionals (18+)
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

Aspiring analysts: Students and professionals aiming at data analyst and data science roles who keep hitting the maths wall
Tool users: People fluent in Excel, SQL or pandas who want to finally understand what the numbers are doing
Interview candidates: Candidates whose next round is statistics, probability and ML fundamentals
Ml learners: Students in ML courses who realize the mathematics was assumed, not taught

Decision makers: Managers and founders who want to read analyses and experiments without being fooled

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: STATISTICS THAT TELL THE TRUTH (MONTHS 1-3, WEEKS 1-12)

The bedrock: describing data honestly, visualizing without lying, and the probability language every model speaks.

Month 1: Describing Data Honestly (Weeks 1-4)

Introduction to Statistical Thinking

Topics:

- What is data science? The role of mathematics
- Types of data: numerical, categorical, ordinal
- Scales of measurement: nominal, ordinal, interval, ratio
- Population vs sample concepts
- Parameters vs statistics
- Data collection methods and sampling techniques
- Simple random sampling
- Stratified and cluster sampling
- Sampling bias and selection bias
- Data quality: missing data, outliers, errors
- Data cleaning principles
- Exploratory Data Analysis (EDA) philosophy

Projects built:

- Design a data collection strategy
- Data quality assessment tool
- Sampling simulation study

Practice: Analyze 20 real-world datasets for quality issues

Measures of Central Tendency and Spread

Topics:

- Mean: arithmetic, geometric, harmonic
- Median and quartiles
- Mode and multimodal distributions
- Weighted averages and their applications
- Range and interquartile range
- Variance and standard deviation
- Coefficient of variation
- Mean absolute deviation
- Moments: skewness and kurtosis
- Box plots and five-number summary
- Z-scores and standardization
- Detecting outliers: IQR method, z-score method

Projects built:

- Statistical calculator from scratch
- Outlier detection system
- Interactive visualization dashboard

Practice: Calculate statistics for 50 different distributions

Assessment: Month 1 check: describe a messy real dataset honestly, plus a mixed quiz on everything so far

Month 2: Visualization and Probability (Weeks 5-8)

Data Visualization and Graphical Analysis

Topics:

- Principles of effective visualization
- Histograms and density plots
- Scatter plots and correlation patterns
- Bar charts and categorical data visualization
- Pie charts: when to use and avoid
- Heat maps and correlation matrices
- Time series plots and trends
- Parallel coordinates for high-dimensional data
- Q-Q plots and probability plots
- Interactive visualizations with Plotly
- Dashboard design principles
- Misleading visualizations and how to spot them

Projects built:

- Build comprehensive EDA toolkit
- Create interactive data dashboard
- Visualization best practices guide

Practice: Create 100 different types of visualizations

Probability Fundamentals

Topics:

- Sample spaces and events
- Classical, frequentist, and subjective probability

- Probability axioms and properties
- Conditional probability and independence
- Bayes' theorem and applications
- Law of total probability
- Combinatorics for probability
- Permutations and combinations in data
- Birthday paradox and probability paradoxes
- Monte Carlo simulation basics
- Random number generation
- Probability in machine learning context

Projects built:
- Probability simulator
- Bayes theorem calculator
- Monte Carlo estimation tool

Practice: Solve 150 probability problems with data applications
Assessment: Month 2 check: a probability problem set with one deliberately misleading chart to expose

Month 3: The Great Distributions (Weeks 9-12)
Discrete Probability Distributions
Topics:
- Random variables: discrete vs continuous
- Probability mass functions
- Bernoulli and binomial distributions
- Geometric and negative binomial distributions
- Poisson distribution and rare events
- Hypergeometric distribution
- Multinomial distribution
- Expected value and variance calculations
- Moment generating functions
- Joint discrete distributions
- Marginal and conditional distributions
- Applications in A/B testing

Projects built:
- Distribution calculator and visualizer
- A/B test simulator
- Discrete event simulator

Practice: Work with 100 discrete distribution problems
Continuous Probability Distributions
Topics:
- Probability density functions
- Uniform distribution
- Normal distribution: properties and importance
- Standard normal and z-tables
- Exponential distribution and memoryless property
- Gamma and beta distributions
- Chi-square distribution
- Student's t-distribution
- F-distribution
- Log-normal distribution
- Weibull distribution for reliability
- Distribution fitting and selection

Projects built:
- Distribution fitting tool
- Normal distribution simulator
- Reliability analysis system

Practice: Master 100 continuous distribution applications
Assessment: Phase 1 exam: distributions and probability in anger, mixed with months 1-2 material

---
PHASE 2: LINEAR ALGEBRA AND STATISTICAL INFERENCE (MONTHS 4-6, WEEKS 13-24)

The two pillars: the vector and matrix language models are written in, and inference, how honest conclusions get drawn from samples.

Month 4: Linear Algebra for Data (Weeks 13-16)
Linear Algebra for Data Science - Vectors
Topics:
- Vectors as data points
- Vector operations: addition, scalar multiplication
- Dot product and cosine similarity
- Vector norms: L1, L2, L-infinity
- Distance metrics: Euclidean, Manhattan, Minkowski
- Orthogonality and orthogonal projections
- Vector spaces and subspaces
- Linear independence in feature space
- Basis vectors and coordinate systems
- Change of basis for data transformation

```
      - Gram-Schmidt for orthogonalization
      - Applications to feature engineering
    Projects built:
      - Vector similarity calculator
      - Distance metric visualizer
      - Feature space explorer
    Practice: Complete 100 vector operations in data context
  Linear Algebra for Data Science - Matrices
    Topics:
      - Matrices as datasets and transformations
      - Matrix multiplication as data transformation
      - Special matrices: diagonal, symmetric, orthogonal
      - Matrix transpose and properties
      - Matrix rank and linear independence of features
      - Invertible matrices and their meaning
      - Determinants and volume interpretation
      - Systems of linear equations in ML
      - Gaussian elimination for solving systems
      - LU decomposition
      - Matrix norms and condition numbers
      - Sparse matrices in big data
    Projects built:
      - Matrix operations library
      - Linear system solver
      - Sparse matrix handler
    Practice: Solve 100 matrix problems with data applications
    Assessment: Month 4 check: matrix computations plus a mixed review reaching back to
    statistics

Month 5: Eigenvalues, PCA and Sampling (Weeks 17-20)
  Eigenvalues, Eigenvectors and PCA
    Topics:
      - Eigenvalues and eigenvectors intuition
      - Characteristic equation
      - Geometric interpretation of eigenvectors
      - Diagonalization of matrices
      - Spectral decomposition
      - Positive definite matrices
      - Covariance matrices in data
      - Principal Component Analysis (PCA) theory
      - PCA algorithm step-by-step
      - Variance explained and scree plots
      - PCA for dimensionality reduction
      - Applications in data compression
    Projects built:
      - PCA implementation from scratch
      - Eigenface recognition system
      - Dimensionality reduction tool
    Practice: Apply PCA to 20 different datasets
  Sampling Distributions and CLT
    Topics:
      - Sampling distribution concept
      - Sampling distribution of the mean
      - Standard error and its importance
      - Central Limit Theorem (CLT)
      - CLT applications and limitations
      - Sample size determination
      - Finite population correction
      - Sampling distribution of proportions
      - Sampling distribution of variances
      - Chi-square distribution from normal samples
      - t-distribution for small samples
      - F-distribution for variance ratios
    Projects built:
      - CLT demonstration tool
      - Sample size calculator
      - Sampling distribution simulator
    Practice: Explore 50 sampling distribution scenarios
    Assessment: Month 5 check: run and interpret a PCA, plus sampling-distribution
    questions

Month 6: Inference Done Honestly (Weeks 21-24)
  Confidence Intervals
    Topics:
      - Point estimates vs interval estimates
      - Confidence level interpretation
      - CI for population mean (known variance)
      - CI for population mean (unknown variance)
      - CI for population proportion
```

- CI for difference of means
- CI for paired differences
- CI for variance and standard deviation
- CI for ratio of variances
- Bootstrap confidence intervals
- Prediction intervals vs confidence intervals
- Sample size for desired margin of error

Projects built:
- Confidence interval calculator
- Bootstrap CI implementation
- CI visualization tool

Practice: Calculate 100 different confidence intervals

Hypothesis Testing Fundamentals

Topics:
- Null and alternative hypotheses
- Type I and Type II errors
- Significance level and p-values
- Test statistics and rejection regions
- One-sample t-test
- Two-sample t-test (equal and unequal variances)
- Paired t-test
- Z-test for proportions
- Chi-square test for variance
- F-test for equal variances
- One-tailed vs two-tailed tests
- Power of a test and effect size
- ANOVA in one honest lesson: comparing many groups
- p-hacking and multiple comparisons: how analysis goes wrong on purpose

Projects built:
- Hypothesis testing framework
- Power analysis tool
- A/B testing platform

Practice: Conduct 100 hypothesis tests

Assessment: Phase 2 exam: a full inference paper, hypothesis tests defended in writing, mixed with earlier phases

---

PHASE 3: THE MATHEMATICS OF MACHINE LEARNING (MONTHS 7-9, WEEKS 25-36)

Models from first principles: regression, regularization, classification, trees and clustering, with evaluation treated as the moral core of the discipline.

Month 7: Regression from First Principles (Weeks 25-28)

Simple Linear Regression

Topics:
- Linear relationship between variables
- Least squares estimation
- Geometric interpretation of least squares
- Normal equations derivation
- Properties of least squares estimators
- Gauss-Markov theorem
- R-squared and adjusted R-squared
- Residual analysis
- Assumptions of linear regression
- Diagnostic plots: Q-Q, residual plots
- Outliers and influential points
- Confidence and prediction intervals for regression

Projects built:
- Linear regression from scratch
- Regression diagnostics tool
- Outlier detection system

Practice: Build 50 linear regression models

Multiple Linear Regression

Topics:
- Multiple regression model
- Matrix formulation of regression
- Partial regression coefficients
- Multicollinearity detection and handling
- Variance Inflation Factor (VIF)
- Feature selection methods
- Forward, backward, stepwise selection
- All subsets regression
- Adjusted R-squared and model selection
- AIC, BIC, Mallows' Cp
- Cross-validation for model selection
- Interaction terms and polynomial regression

Projects built:
- Multiple regression toolkit
- Feature selection system

- Model comparison framework
Practice: Develop 50 multiple regression models
Assessment: Month 7 check: fit, diagnose and defend a regression on real data

Month 8: Regularization and Classification (Weeks 29-32)
Regularized Regression
Topics:
- Overfitting and bias-variance tradeoff
- Ridge regression (L2 regularization)
- Ridge regression geometry
- LASSO regression (L1 regularization)
- LASSO for feature selection
- Elastic Net regression
- Choosing regularization parameters
- Cross-validation for lambda selection
- Bayesian interpretation of regularization
- Group LASSO
- Adaptive LASSO
- Applications in high-dimensional data
Projects built:
- Regularized regression implementation
- Lambda tuning system
- High-dimensional regression tool
Practice: Apply regularization to 40 datasets
Classification Fundamentals
Topics:
- Classification vs regression
- Bayes classifier and Bayes error
- Linear Discriminant Analysis (LDA)
- Quadratic Discriminant Analysis (QDA)
- Fisher's linear discriminant
- Naive Bayes classifier
- Gaussian Naive Bayes
- Multinomial and Bernoulli Naive Bayes
- K-Nearest Neighbors classification
- Distance metrics for KNN
- Decision boundaries visualization
- Class imbalance problems
- Logistic regression as a generalized linear model
Projects built:
- Classifier comparison framework
- Decision boundary visualizer
- Imbalanced data handler
Practice: Implement 50 classification models
Assessment: Month 8 check: a classification problem end to end, plus mixed regression revision

Month 9: Trees, Clusters and Honest Evaluation (Weeks 33-36)
Tree-Based Methods
Topics:
- Decision trees for classification
- Entropy and information gain
- Gini impurity
- Tree pruning methods
- Cost complexity pruning
- Random Forests algorithm
- Out-of-bag error estimation
- Feature importance from trees
- Extremely Randomized Trees
- Gradient Boosting Machines (GBM)
- XGBoost mathematics
- LightGBM and CatBoost
Projects built:
- Decision tree from scratch
- Random Forest implementation
- Boosting algorithm suite
Practice: Build 40 tree-based models
Clustering Algorithms
Topics:
- K-means clustering algorithm
- K-means++ initialization
- Elbow method and silhouette analysis
- Hierarchical clustering
- Agglomerative vs divisive
- Linkage methods: single, complete, average
- Dendrograms and cutting trees
- DBSCAN algorithm
- Mean Shift clustering
- Gaussian Mixture Models (GMM)

- EM algorithm for GMMs
- Spectral clustering
- Model evaluation essentials: cross-validation, metrics and the leakage trap
- Choosing models like an adult: bias, variance and the problem in front of you

Projects built:
- Clustering algorithm suite
- Cluster validation tools
- Optimal K finder

Practice: Apply clustering to 40 datasets

Assessment: Phase 3 exam: build, evaluate and defend two models on one dataset, mixed with earlier phases

---

PHASE 4: OPTIMIZATION, NEURAL NETWORKS AND MODERN PRACTICE (MONTHS 10-12, WEEKS 37-48)

The mathematics behind modern AI and modern analytics: optimization, neural networks, Bayesian methods, time series, experiments, and a defensible capstone.

Month 10: Optimization and Neural Network Mathematics (Weeks 37-40)

Optimization Theory for ML

Topics:
- Convex sets and convex functions
- Convexity in machine learning
- Gradient descent algorithm
- Learning rate selection
- Momentum and Nesterov acceleration
- AdaGrad and RMSprop
- Adam and AdamW optimizers
- Second-order methods: Newton, L-BFGS
- Stochastic gradient descent (SGD)
- Mini-batch gradient descent
- Convergence analysis
- Constrained optimization for ML

Projects built:
- Optimizer implementations
- Convergence visualizer
- Learning rate scheduler

Practice: Implement 20 optimization algorithms

Neural Network Fundamentals

Topics:
- Perceptron algorithm
- Multi-layer perceptrons
- Universal approximation theorem
- Activation functions: sigmoid, tanh, ReLU
- Forward propagation mathematics
- Backpropagation derivation
- Chain rule in neural networks
- Gradient vanishing and exploding
- Weight initialization strategies
- Xavier and He initialization
- Batch normalization mathematics
- Dropout as regularization
- Where this leads: transformers and attention, mapped honestly for the AI/ML masterclass

Projects built:
- Neural network from scratch
- Backpropagation visualizer
- Activation function explorer

Practice: Build 30 neural network architectures

Assessment: Month 10 check: gradient descent by hand on a small network, plus mixed review

Month 11: Bayesian Methods and Time Series (Weeks 41-44)

Bayesian Fundamentals

Topics:
- Bayesian vs Frequentist philosophy
- Prior distributions and their selection
- Likelihood functions
- Posterior distributions
- Conjugate priors
- Beta-Binomial model
- Normal-Normal model
- Gamma-Poisson model
- Credible intervals vs confidence intervals
- Bayesian point estimates
- MAP vs MLE estimates
- Empirical Bayes methods

Projects built:
- Bayesian inference engine
- Prior selection tool

```
          - Posterior calculator
        Practice: Solve 100 Bayesian inference problems
      Time Series Analysis
        Topics:
          - Time series components: trend, seasonality, noise
          - Stationarity and unit root tests
          - Autocorrelation and partial autocorrelation
          - ARIMA models
          - Box-Jenkins methodology
          - Seasonal ARIMA (SARIMA)
          - State space models
          - Kalman filtering
          - Exponential smoothing methods
          - Holt-Winters method
          - STL decomposition
          - Time series cross-validation
        Projects built:
          - Time series toolkit
          - ARIMA model builder
          - Forecast evaluator
        Practice: Analyze 50 time series
        Assessment: Month 11 check: one Bayesian update argued in plain language, one forecast
        defended

    Month 12: Experiments and the Capstone (Weeks 45-48)
      A/B Testing and Experimental Design
        Topics:
          - A/B testing fundamentals
          - Statistical power in A/B tests
          - Multiple testing corrections
          - Sequential testing
          - Bandits vs A/B tests
          - Multi-armed bandits
          - Thompson sampling
          - Contextual bandits
          - Variance reduction techniques
          - CUPED method
          - Stratification in experiments
          - Network effects in testing
          - Design of experiments: the checklist before any test ships
          - Ethics and interpretability: what a responsible analyst refuses to do
        Projects built:
          - A/B testing platform
          - Bandit algorithm suite
          - Variance reduction tool
        Practice: Run 40 A/B tests
      Capstone and Final Exam
        Topics:
          - The capstone: one real dataset, one honest question, full analysis
          - From exploration to inference to model to recommendation
          - Writing the analysis so a skeptic cannot shake it
          - Presenting quantitative work to non-quantitative people
          - A full-course spiral review, exam technique included
        Projects built:
          - The capstone analysis: explored, modeled, tested and written up defensibly, presented to the
            batch
        Practice: Daily mixed revision sets through the final fortnight
        Assessment: FINAL EXAM: a written paper plus a practical analysis, with questions mixed
        from every phase. Passing earns the certificate; falling short earns a focused revision
        plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Phase 1: honest descriptive reports, a misleading-charts autopsy, probability simulations
  - Phase 2: a PCA on real data, full inference write-ups, a p-hacking demonstration done
    ethically
  - Phase 3: regressions and classifiers built from first principles and defended in writing
  - Phase 4: a hand-computed neural network, a Bayesian analysis, a forecast, an A/B test design,
    and the final capstone

  Total projects: 25+ applied analyses, ending with a defensible capstone on real data

SKILLS THE STUDENT WILL HAVE AT THE END

  - Statistics: description, visualization and probability, wielded honestly
  - Linear algebra: vectors, matrices, eigenvalues and PCA, the language of models
  - Inference: sampling, confidence intervals, hypothesis testing and its abuses
  - ML mathematics: regression, regularization, classification, trees, clustering, evaluation
  - Modern methods: optimization, neural network mathematics, Bayesian thinking, time series
```

- Experimentation: designing and reading A/B tests that mean something
- Communication: writing and defending quantitative conclusions

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, worked problems and real data, not slideware
Homework: A problem set after every class, and a monthly mixed review that reaches back deliberately
Practice: 4-6 hours of problem and data practice between classes
Review: Every set marked with written feedback; recurring weaknesses reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, for the derivation that will not land
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- AI and Machine Learning Masterclass: deep learning practice on top of this mathematical base
- Data analyst and data science roles: this is the mathematics their interviews test
- Our Data Analysis course: the tooling counterpart (SQL, Python, dashboards) to this mathematical core
- Quantitative and experimentation roles: A/B testing and causal thinking are the daily work
- Further statistics study: Bayesian and time-series doors are opened here

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 8-15 LPA / $70-100k USD
Growing roles: Rs 15-30 LPA / $100-150k USD
Experienced roles: Rs 30-60 LPA / $150-250k USD; Rs 60+ LPA / $250k+ USD
Consulting: Rs 5,000-15,000/hour

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What exactly does this maths for data science course teach?
A: The mathematics data careers actually stand on, in four phases: statistics and probability that tell the truth; linear algebra through eigenvalues and PCA plus honest statistical inference; the mathematics of machine learning, regression, classification, trees, clustering and evaluation from first principles; and modern practice, optimization, neural network mathematics, Bayesian methods, time series and A/B testing, ending in a capstone analysis on real data.

Q: How long does it take to learn the maths for data science?
A: Honestly: 12 months at 2 live classes a week plus 4 to 6 hours of practice, and the course keeps 2 months of margin in hand for learners who need more repetition, up to 14 months with no penalty. Crash courses compress this into weeks by skipping the understanding; the wall they skip is exactly what interviews and real work then test.

Q: How is progress measured? Is the certificate just for attending?
A: No. Problem sets follow every class and come back with written feedback, monthly reviews deliberately mix new material with earlier topics so nothing fades, each phase ends with an exam, and the course ends with a final exam, written plus practical, that decides the certificate. Falling short earns a focused revision plan and a free retest. The certificate certifies knowledge, not attendance.

Q: Do I need to know programming or calculus already?
A: Neither. School mathematics is the entry bar; the calculus that appears is refreshed as needed, and code is used only as a calculator, spreadsheets and light Python, because the mathematics is the syllabus. Students who already code simply see the connections faster.

Q: How is this different from a data science bootcamp?
A: Bootcamps teach tools and hope the mathematics arrives by osmosis; it does not. This course is the missing layer: the statistics, linear algebra and ML mathematics that make tool skills defensible in interviews and reliable at work. The two pair well, our Data Analysis course covers the tooling side, and this course makes it mean something.

Q: Will this help me in analytics interviews and real work?
A: Directly: probability puzzles, hypothesis testing, regression interpretation, evaluation metrics and A/B test design are the exact backbone of analytics and data science interviews, and the course drills them with written defense, not just computation. At work, the payoff is judgment: knowing which analysis is honest, and being the person in the room who can say why.

Q: Does this cover deep learning and AI?
A: It covers their mathematics honestly: optimization, gradient descent and the mathematics of neural networks, computed by hand on small networks, plus a clear map of where transformers and attention sit. Full deep learning practice, building and training real models, is our AI and Machine Learning Masterclass, which assumes exactly the mathematics this course builds.

Q: Can I take a trial class?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Bring a statistics question from your own work or study; the demo is best when it runs on your real problem. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are lessons live or self-paced videos?
A: Live, always: small groups where problems are worked together and your reasoning gets corrected in real time, never pre-recorded videos. Mathematical judgment is built under feedback; a video cannot tell you why your hypothesis test is invalid.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/data-analytics-mathematics-masterclass
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 94 OF 137: MATHS OLYMPIAD & COMPETITION COURSE: AMC, AIME TO IMO

==========

Course link: https://learn.modernagecoders.com/courses/olympiad-competition-mathematics-mastery
Subject area: Competition Mathematics & Olympiad Training
Designed for: Elementary school, Girls only batches, High school, Middle school
Who can join: Intermediate to International Competition Level
Course length: 12-14 months (52-60 weeks)
Time needed: 2 live classes/week + 5-7 hours problem-solving
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

One-year maths olympiad course: master number theory, combinatorics, algebra, geometry and inequalities, with problem-solving craft for AMC, AIME and IMO.

In one line: From Regional Contests to International Mathematical Olympiad

ABOUT THE PROGRAMME

This intensive program transforms talented mathematics students into competition champions. Designed for students aiming to excel in mathematical olympiads and competitions at all levels - from AMC to IMO. You'll master advanced problem-solving techniques, develop mathematical intuition, and learn to tackle the most challenging competition problems. Through systematic training in number theory, combinatorics, algebra, geometry, and more, you'll build the skills needed to compete at the highest levels. By completion, you'll have solved thousands of competition problems and be ready for any mathematical challenge.

WHAT MAKES THIS COURSE DIFFERENT

- Comprehensive coverage of all olympiad topics
- Progressive difficulty from AMC 8 to IMO level
- 1000+ carefully selected competition problems
- Strategies from past olympiad medalists
- Mock competitions and timed practice
- Proof writing and rigorous mathematics
- International competition exposure
- Personal coaching and problem discussions

THE LEARNING PATH, STEP BY STEP

  Phase 1: Foundation (Months 1-3): Competition Basics, Number Theory, Elementary Combinatorics
  Phase 2: Intermediate (Months 4-6): Advanced Algebra, Polynomial Theory, Sequences and Series
  Phase 3: Advanced (Months 7-9): Competition Geometry, Trigonometry, Coordinate Methods
  Phase 4: Master Level (Months 10-12): Inequalities, Functional Equations, IMO Preparation

WHAT A STUDENT NEEDS BEFORE STARTING

  Education: Strong foundation in school mathematics
  Coding experience: Basic algebra and geometry knowledge required
  Equipment: Computer with internet, graphing calculator, geometry tools
  Time commitment: 2 live classes plus 5 to 7 hours of problem-solving a week (olympiad
    training rewards more practice)
  English: Advanced mathematical reading ability
  Motivation: Passionate about mathematical problem solving

WHO THIS COURSE IS FOR

  Students: High school students aiming for olympiads
  Working professionals: Mathematics teachers and coaches
  Entrepreneurs: Competition preparation centers
  Freelancers: Private olympiad tutors
  Kids: Gifted middle school students
  Anyone: Anyone passionate about competition mathematics

FULL CURRICULUM, WEEK BY WEEK

  This is the complete teaching plan. Parents can ask about any week.

  PHASE 1: COMPETITION FUNDAMENTALS AND NUMBER THEORY (WEEKS 1-13)

  Build strong foundations in competition problem-solving, master number theory, and develop
  combinatorial thinking.

    Months 1-2: Competition Mathematics Basics & Number Theory (Weeks 1-8)
      Introduction to Competition Mathematics
        Topics:
          - Competition mathematics vs school mathematics
          - Major competitions overview: AMC, AIME, USAMO, IMO
          - Competition formats and scoring systems
          - Problem-solving strategies: working backwards
          - Pattern recognition and generalization
          - Extreme cases and boundary testing
          - Proof by contradiction
          - Proof by induction basics
          - Direct proof techniques
          - Constructive vs non-constructive proofs
          - Mathematical notation for competitions
          - Time management in competitions
        Projects built:
          - Solve 50 AMC 8 problems
          - Create problem-solving strategy guide
          - Analyze past competition papers
        Practice: Daily: 5 competition problems with increasing difficulty
      Divisibility and Prime Numbers
        Topics:
          - Advanced divisibility rules and proofs
          - GCD and LCM properties and algorithms
          - Euclidean algorithm and extended version
          - Bezout's identity and applications
          - Prime factorization uniqueness theorem
          - Distribution of primes
          - Sieve of Eratosthenes variations
          - Prime number theorem introduction
          - Fermat's Little Theorem
          - Wilson's Theorem
          - Prime testing algorithms
          - Applications in cryptography basics
        Projects built:
          - Implement prime generation algorithms
          - Solve 30 number theory competition problems
          - Create divisibility proof portfolio
        Practice: Master 40 divisibility and prime problems from competitions
      Modular Arithmetic
        Topics:
          - Modular arithmetic operations and properties
          - Congruence classes and equivalence

- Linear congruences and solutions
- Chinese Remainder Theorem
- Modular multiplicative inverse
- Euler's totient function ϕ(n)
- Euler's theorem and generalizations
- Quadratic residues introduction
- Legendre symbol
- Power patterns in modular arithmetic
- Solving Diophantine equations
- Applications to competition problems

Projects built:
- Modular arithmetic problem set
- CRT application problems
- Create modular arithmetic reference guide

Practice: Solve 50 modular arithmetic competition problems

Advanced Number Theory

Topics:
- Perfect numbers and Mersenne primes
- Arithmetic functions: σ(n), τ(n), μ(n)
- Multiplicative functions
- Mobius inversion formula
- Continued fractions basics
- Pell's equation
- Quadratic reciprocity introduction
- Sum of squares theorems
- Lifting the Exponent Lemma (LTE)
- Zsigmondy's theorem
- Carmichael's lambda function
- Advanced competition techniques

Projects built:
- Advanced number theory problem collection
- Historical theorems exploration
- IMO number theory problems analysis

Practice: 30 advanced number theory problems from national olympiads

Month 3: Combinatorics Fundamentals (Weeks 9-13)

Counting Principles and Techniques

Topics:
- Advanced counting principles
- Multiplication and addition principles
- Permutations with restrictions
- Circular permutations and necklaces
- Combinations with repetition
- Stars and bars method
- Inclusion-Exclusion Principle
- Derangements and subfactorial
- Stirling numbers of both kinds
- Partition function introduction
- Generating functions basics
- Recurrence relations

Projects built:
- Combinatorial identity proofs
- Counting problems compilation
- Create combinatorics formula sheet

Practice: Solve 40 counting problems from competitions

Graph Theory Basics

Topics:
- Graph definitions and terminology
- Types of graphs: complete, bipartite, planar
- Degree sequences and handshaking lemma
- Trees and forest properties
- Eulerian paths and circuits
- Hamiltonian paths and cycles
- Graph coloring and chromatic number
- Planar graphs and Euler's formula
- Bipartite matching basics
- Hall's marriage theorem
- Tournament graphs
- Competition graph problems

Projects built:
- Graph theory problem solving guide
- Visualization of graph algorithms
- Competition graph problems collection

Practice: Master 35 graph theory competition problems

Phase 1 Review and Competition

Topics:
- Number theory comprehensive review
- Modular arithmetic mastery check
- Combinatorics problem-solving strategies

- Graph theory applications
- Mixed problem sets
- Speed solving techniques
- Competition simulation

Projects built:
- Create Phase 1 solution manual
- Mock AMC 10 competition
- Peer problem exchange and solving

Assessment: Phase 1 cumulative exam: a written paper mixing this phase with everything before it, plus targeted revision for anything shaky

---

PHASE 2: ADVANCED ALGEBRA AND POLYNOMIAL THEORY (WEEKS 14-26)

Master algebraic manipulations, polynomial theory, sequences, and functional equations for competitions.

Months 4-5: Competition Algebra & Polynomials (Weeks 14-21)

Advanced Algebraic Techniques

Topics:
- Algebraic manipulation mastery
- Telescoping sums and products
- Partial fractions in competitions
- Symmetric expressions and polynomials
- Vieta's formulas and applications
- Newton's identities
- Power sum symmetric functions
- Discriminant and its properties
- Algebraic substitutions techniques
- Rationalizing complex expressions
- Nested radicals simplification
- Competition algebraic tricks

Projects built:
- Algebraic techniques handbook
- Solve 50 AIME algebra problems
- Create substitution strategy guide

Practice: Daily: 8 advanced algebra competition problems

Polynomial Theory

Topics:
- Polynomial division and remainder theorem
- Factor theorem and applications
- Rational root theorem extensions
- Descartes' rule of signs
- Complex roots and conjugate pairs
- Fundamental theorem of algebra
- Polynomial interpolation (Lagrange)
- Chebyshev polynomials introduction
- Cyclotomic polynomials
- Irreducibility testing
- Polynomial inequalities
- Roots and coefficients relationships

Projects built:
- Polynomial problem solving manual
- Root-finding techniques compilation
- IMO polynomial problems analysis

Practice: 45 polynomial problems from national olympiads

Sequences and Series

Topics:
- Arithmetic and geometric progressions advanced
- Arithmetico-geometric progressions
- Harmonic progressions and means
- Recursive sequences and linear recurrences
- Characteristic equations method
- Generating functions for sequences
- Fibonacci and Lucas sequences properties
- Catalan numbers and applications
- Convergence tests for series
- Telescoping series mastery
- Power series basics
- Competition sequence problems

Projects built:
- Sequence encyclopedia creation
- Recurrence relation solver
- Famous sequences investigation

Practice: Solve 40 sequence and series competition problems

Equations and Systems

Topics:
- Systems of linear equations (advanced)
- Nonlinear systems solving techniques

- Symmetric systems of equations
- Cyclic systems
- Homogeneous equations
- Parametric solutions
- Diophantine equations (advanced)
- Quadratic Diophantine equations
- Simon's Favorite Factoring Trick
- Equations with absolute values
- Floor and ceiling function equations
- Competition equation strategies

Projects built:

- Equation solving techniques manual
- Diophantine equation collection
- System solving flowchart

Practice: Master 50 competition equation problems

Complex Numbers in Competitions

Topics:

- Complex number operations review
- Geometric interpretation of complex operations
- De Moivre's theorem and applications
- Roots of unity and cyclotomic polynomials
- Complex number proofs in geometry
- Trigonometric identities via complex numbers
- Gaussian integers
- Complex conjugate root theorem
- Argument and modulus inequalities
- Competition problems using complex numbers
- Euler's formula applications
- Advanced complex number techniques

Projects built:

- Complex numbers in geometry guide
- Roots of unity problem set
- Complex number competition tricks

Practice: 30 complex number competition problems

Month 6: Functional Equations & Advanced Topics (Weeks 22-26)

Functional Equations

Topics:

- Introduction to functional equations
- Cauchy's functional equations
- Substitution methods in functional equations
- Finding solutions: injectivity, surjectivity
- Additive and multiplicative functions
- Jensen's functional equation
- D'Alembert's functional equation
- Polynomial functional equations
- Functional equations with multiple variables
- Cyclic functional equations
- IMO functional equation techniques
- Common competition patterns

Projects built:

- Functional equation solving guide
- Pattern recognition in functional equations
- IMO functional equations compilation

Practice: Solve 35 functional equation problems

Mathematical Induction Advanced

Topics:

- Strong induction principles
- Structural induction
- Double induction
- Infinite descent method
- Well-ordering principle applications
- Induction with inequalities
- Induction in number theory
- Induction in combinatorics
- Induction for sequences
- Forward-backward induction
- Transfinite induction introduction
- Competition induction strategies

Projects built:

- Induction proof portfolio
- Induction techniques classification
- Create induction problem bank

Practice: Complete 40 induction problems of varying difficulty

Pigeonhole Principle & Extremal Problems

Topics:

- Pigeonhole principle variations
- Generalized pigeonhole principle
- Infinite pigeonhole principle

- Probabilistic pigeonhole principle
- Dirichlet's theorem applications
- Extremal principle in problem solving
- Optimization in discrete mathematics
- Erdős-Ko-Rado theorem introduction
- Ramsey theory basics
- Van der Waerden's theorem introduction
- Extremal graph theory
- Competition applications

Projects built:
- Pigeonhole principle masterclass
- Extremal problems collection
- Ramsey theory exploration

Practice: 30 pigeonhole and extremal problems

Invariants and Monovariants

Topics:
- Invariant principle in problem solving
- Finding invariants in processes
- Monovariant strategies
- Parity as an invariant
- Coloring arguments
- Sum and product invariants
- Geometric invariants
- Game theory and invariants
- Invariants in combinatorial games
- Semi-invariants
- Competition invariant problems
- Creating invariant arguments

Projects built:
- Invariant problem solving guide
- Game theory invariants study
- Invariant problem creation

Practice: Master 35 invariant-based problems

Phase 2 Review and Competition

Topics:
- Algebra and polynomials review
- Sequences and series mastery
- Functional equations practice
- Complex numbers applications
- Induction and extremal principles
- Mixed advanced problems
- AIME simulation

Projects built:
- Phase 2 complete solution manual
- Mock AIME competition
- Problem-solving video tutorials

Assessment: Phase 2 cumulative exam: a written paper mixing this phase with everything before it, plus targeted revision for anything shaky

---

PHASE 3: COMPETITION GEOMETRY AND TRIGONOMETRY (WEEKS 27-39)

Master advanced geometric techniques, trigonometric identities, and coordinate geometry for olympiad success.

Months 7-8: Euclidean Geometry & Triangle Theory (Weeks 27-34)

Advanced Triangle Geometry

Topics:
- Cevians: medians, altitudes, angle bisectors
- Special points: centroid, orthocenter, incenter, circumcenter
- Euler line and nine-point circle
- Nagel point and Gergonne point
- Stewart's theorem and applications
- Menelaus' theorem
- Ceva's theorem and trigonometric form
- Mass point geometry
- Barycentric coordinates introduction
- Triangle inequality variations
- Routh's theorem
- Competition triangle problems

Projects built:
- Triangle centers encyclopedia
- Special points construction guide
- Triangle theorem proof collection

Practice: Solve 45 advanced triangle problems

Circle Geometry

Topics:
- Power of a point theorem
- Radical axis and radical center

```
      - Coaxial circles
      - Inversion in circles
      - Ptolemy's theorem and inequality
      - Cyclic quadrilaterals properties
      - Simson line and pedal triangles
      - Miquel's theorem
      - Pascal's theorem for circles
      - Brianchon's theorem
      - Nine-point circle properties
      - Competition circle techniques
    Projects built:
      - Circle theorems visual guide
      - Inversion problem solving
      - Cyclic quadrilateral mastery
    Practice: Master 40 circle geometry problems
  Similarity and Homothety
    Topics:
      - Advanced similarity principles
      - Spiral similarities
      - Homothety and homothetic centers
      - Similar triangles in competitions
      - Angle bisector theorem extensions
      - Apollonius' theorem
      - Harmonic division and conjugates
      - Cross-ratio and applications
      - Desargues' theorem
      - Monge's theorem
      - Homothety in circle problems
      - Competition applications
    Projects built:
      - Similarity techniques manual
      - Homothety problem collection
      - Projective geometry introduction
    Practice: Complete 35 similarity and homothety problems
  Geometric Transformations
    Topics:
      - Reflection properties and compositions
      - Rotation compositions and fixed points
      - Translation vectors and compositions
      - Glide reflections
      - Isometries classification
      - Dilation and scaling transformations
      - Affine transformations
      - Transformation approach to problems
      - Symmetry in problem solving
      - Spiral similarities revisited
      - Complex numbers for transformations
      - Competition transformation techniques
    Projects built:
      - Transformation problem solving guide
      - Symmetry exploitation techniques
      - Transformation composition calculator
    Practice: Solve 40 transformation-based problems
  Construction Problems
    Topics:
      - Classical constructions review
      - Constructible numbers theory
      - Impossible constructions and proofs
      - Mohr-Mascheroni constructions
      - Poncelet-Steiner constructions
      - Construction strategies in competitions
      - Locus problems
      - Construction with restrictions
      - Geometric optimization constructions
      - Construction existence proofs
      - Competition construction problems
      - Software-aided exploration
    Projects built:
      - Construction problem anthology
      - Impossible constructions proofs
      - Interactive construction tool
    Practice: Complete 30 construction problems

Month 9: Trigonometry & Coordinate Geometry (Weeks 35-39)
  Advanced Trigonometry
    Topics:
      - Trigonometric identities mastery
      - Sum-to-product and product-to-sum formulas
      - Multiple angle formulas
```

    - Half-angle formulas applications
    - Conditional identities
    - Trigonometric equations (advanced)
    - Trigonometric inequalities
    - Jensen's inequality for trigonometry
    - Chebyshev polynomials and trigonometry
    - Trigonometric substitutions
    - Competition trigonometry tricks
    - Proofs using trigonometry
  Projects built:
    - Trigonometric identity encyclopedia
    - Competition trigonometry handbook
    - Trigonometric proof portfolio
  Practice: Master 40 trigonometry competition problems
Analytic Geometry
  Topics:
    - Coordinate geometry strategies
    - Distance and section formulas
    - Area calculations in coordinates
    - Line equations and properties
    - Angle between lines
    - Conic sections: parabola, ellipse, hyperbola
    - Parametric representations
    - Polar coordinates applications
    - Coordinate transformations
    - Analytic proofs of geometric theorems
    - Optimization using coordinates
    - Competition coordinate techniques
  Projects built:
    - Coordinate geometry solver
    - Conic sections problem set
    - Analytic vs synthetic comparison
  Practice: Solve 35 coordinate geometry problems
Vectors in Geometry
  Topics:
    - Vector operations and properties
    - Dot product and cross product
    - Vector proofs of geometric theorems
    - Position vectors and applications
    - Vector equations of lines and planes
    - Distance using vectors
    - Angle calculations with vectors
    - Vector approach to transformations
    - Centroid and circumcenter via vectors
    - 3D geometry basics
    - Competition vector techniques
    - Vectors vs coordinates comparison
  Projects built:
    - Vector geometry manual
    - 3D geometry exploration
    - Vector solution compilation
  Practice: Complete 30 vector geometry problems
Area and Volume Methods
  Topics:
    - Area calculation techniques
    - Shoelace formula and applications
    - Pick's theorem
    - Heron's formula and extensions
    - Bretschneider's formula
    - Brahmagupta's formula
    - Volume calculation methods
    - Cavalieri's principle
    - Volume by cross-sections
    - Surface area calculations
    - Pappus's centroid theorems
    - Competition area/volume tricks
  Projects built:
    - Area formula compendium
    - Volume calculation guide
    - Historical formula exploration
  Practice: Master 35 area and volume problems
Phase 3 Review and Competition
  Topics:
    - Euclidean geometry mastery review
    - Circle theorems compilation
    - Transformation techniques
    - Trigonometry applications
    - Coordinate methods review
    - Vector approaches

- Mixed geometry problems
- USAMO geometry preparation

Projects built:
- Geometry techniques comparison chart
- Mock USAMO geometry round
- Geometry problem video solutions

Assessment: Phase 3 cumulative exam: a written paper mixing this phase with everything before it, plus targeted revision for anything shaky

---

PHASE 4: INEQUALITIES, OLYMPIAD PRACTICE AND FINAL REVISION (WEEKS 40-52)

Master inequalities, advanced problem-solving techniques, and prepare for international competitions.

Month 10: Inequalities & Optimization (Weeks 40-44)

Classical Inequalities

Topics:
- AM-GM inequality and applications
- Cauchy-Schwarz inequality proofs and uses
- Holder's inequality
- Minkowski's inequality
- Power mean inequalities
- Weighted AM-GM inequality
- Jensen's inequality for convex functions
- Karamata's inequality
- Rearrangement inequality
- Chebyshev's inequality
- Bernoulli's inequality
- Competition inequality techniques

Projects built:
- Inequality proof techniques guide
- Classical inequality applications
- Inequality chain constructions

Practice: Prove 50 competition inequalities

Advanced Inequality Techniques

Topics:
- Smoothing and majorization
- Mixing variables technique
- Lagrange multipliers introduction
- Homogenization techniques
- Substitutions in inequalities
- Cyclic and symmetric inequalities
- Buffalo Way method
- SOS (Sum of Squares) method
- Schur's inequality and generalizations
- Muirhead's inequality
- Newton's inequalities
- IMO inequality strategies

Projects built:
- Advanced inequality manual
- SOS method implementation
- IMO inequalities analysis

Practice: Master 40 advanced inequality problems

Optimization Problems

Topics:
- Discrete optimization techniques
- Continuous optimization basics
- Calculus in olympiad problems
- Derivatives for optimization
- Constrained optimization
- Geometric optimization problems
- Isoperimetric problems
- Algebraic optimization
- Combinatorial optimization
- Game theory optimization
- Dynamic programming introduction
- Competition optimization strategies

Projects built:
- Optimization techniques handbook
- Geometric optimization collection
- Create optimization problem set

Practice: Solve 35 optimization problems

Advanced Combinatorics

Topics:
- Bijective proofs and counting
- Double counting techniques
- Combinatorial identities (advanced)
- Vandermonde's identity applications

- Catalan number applications
- Bell numbers and Stirling numbers
- Integer partitions
- Young tableaux introduction
- Polya enumeration theorem
- Combinatorial game theory
- Probabilistic methods
- Competition combinatorics mastery

Projects built:

- Combinatorial proof anthology
- Counting techniques flowchart
- Game theory analysis

Practice: Complete 40 advanced combinatorics problems

Advanced Number Theory Topics

Topics:

- Quadratic reciprocity (complete)
- Primitive roots and indices
- Quadratic forms basics
- Continued fractions (advanced)
- Transcendental numbers introduction
- p-adic numbers introduction
- Analytic number theory glimpse
- Additive number theory basics
- Beatty sequences
- Thue's lemma
- Advanced Diophantine equations
- IMO number theory trends

Projects built:

- Advanced number theory compendium
- Historical problems exploration
- Number theory research topics

Practice: Tackle 30 research-level number theory problems

Month 11: Problem-Solving Mastery & Competition Strategy (Weeks 45-48)

IMO Problem Analysis

Topics:

- IMO problem structure and trends
- Problem 1 and 4 strategies (easier problems)
- Problem 2 and 5 strategies (medium problems)
- Problem 3 and 6 strategies (hardest problems)
- Time management for 4.5-hour exams
- Partial credit maximization
- Problem selection strategies
- Writing rigorous proofs
- Common IMO problem patterns
- Historical IMO problems analysis
- National olympiad comparison
- Training camp preparation

Projects built:

- IMO problems by topic classification
- Personal IMO strategy development
- Mock IMO with analysis

Practice: Solve 20 past IMO problems

Putnam Competition Preparation

Topics:

- Putnam exam format and scoring
- Putnam problem characteristics
- Abstract algebra in Putnam
- Real analysis topics
- Linear algebra applications
- Probability in Putnam problems
- Putnam trick techniques
- Time management for Putnam
- A vs B session strategies
- Historical Putnam analysis
- University-level competition transition
- Research mathematics introduction

Projects built:

- Putnam problem collection by topic
- University math preview
- Putnam simulation exam

Practice: Complete 30 Putnam problems

Asian Pacific Mathematics Olympiad

Topics:

- APMO problem characteristics
- Regional competition differences
- APMO vs IMO difficulty comparison
- Cultural problem-solving approaches
- APMO-specific techniques

- Time zone competition strategies
- Online competition protocols
- APMO historical trends
- Country-specific olympiad styles
- International collaboration benefits
- APMO as IMO preparation
- Regional olympiad circuit

Projects built:
- APMO complete solutions manual
- Regional olympiad comparison study
- APMO mock competition

Practice: Solve 25 APMO problems from past years

Competition Psychology & Peak Performance

Topics:
- Mental preparation for competitions
- Stress management techniques
- Visualization and positive thinking
- Competition day routines
- Dealing with difficult problems
- Recovery from mistakes
- Time pressure management
- Building mathematical confidence
- Team competition strategies
- Learning from competition failures
- Post-competition analysis
- Long-term training mindset

Projects built:
- Personal competition preparation guide
- Competition journal and reflection
- Mental training program design

Practice: Implement mental training with timed problem sets

Month 12: Mock Competitions & Final Preparation (Weeks 49-50)

National Olympiad Simulations

Topics:
- USAMO complete simulation
- USAJMO preparation strategies
- Canadian Mathematical Olympiad
- British Mathematical Olympiad
- Australian Mathematics Olympiad
- Indian National MO preparation
- China National Team Selection
- Russia Mathematical Olympiad
- Time management across formats
- Scoring system understanding
- Selection criteria for teams
- National camp preparation

Projects built:
- Complete 5 national olympiad mocks
- Cross-country problem comparison
- National team selection guide

Practice: One full national olympiad daily

International Competition Week

Topics:
- IMO Day 1 simulation (4.5 hours)
- IMO Day 2 simulation (4.5 hours)
- Complete solution writing
- Proof verification techniques
- Collaboration after competition
- Score prediction and analysis
- International competition etiquette
- Cultural exchange benefits
- Language considerations
- Travel and preparation logistics
- Team dynamics
- Medal boundaries analysis

Projects built:
- Complete IMO simulation with scoring
- Solution comparison international students
- Competition reflection essay

Practice: Full IMO mock over two days

Month 13: Final Assessments & Future Planning (Weeks 51-52)

Comprehensive Review & Mastery

Topics:
- Number theory complete review
- Combinatorics mastery assessment
- Algebra and polynomials review
- Geometry techniques compilation

- Inequality mastery check
- Functional equations review
- Problem-solving strategies synthesis
- Proof writing excellence
- Speed and accuracy balance
- Knowledge gap identification
- Personal strength analysis
- Improvement planning

Projects built:
- Complete technique reference manual
- Personal problem-solving guide
- Video solution library creation

Final Competition Series

Topics:
- AMC 12 simulation
- AIME simulation
- USAMO simulation Day 1
- USAMO simulation Day 2
- IMO simulation Day 1
- IMO simulation Day 2
- Score analysis and prediction
- Performance evaluation
- Peer competition and ranking
- Solution presentation
- Awards and recognition
- Competition portfolio completion

Deliverables:
- Complete competition portfolio
- All mock competition scores
- Solution manual for all attempts
- Performance analysis report
- Technique mastery certification
- Problem-solving speed metrics

Assessment: FINAL EXAM: a comprehensive written paper covering every phase of the course. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: AMC Training Manual, Number Theory Proofs, Combinatorics Guide, Mock Competitions
- Phase 2: AIME Preparation Kit, Polynomial Mastery, Functional Equations Solver, Complex Numbers Toolkit
- Phase 3: Geometry Compendium, Trigonometry Manual, Construction Problems, Vector Methods Guide
- Phase 4: Inequality Bible, IMO Solutions Archive, Competition Strategy Guide, Research Paper
- Final: Complete Olympiad Preparation System, Original Problem Collection, Teaching Materials

Total projects: 75+ mathematical projects and problem collections

SKILLS THE STUDENT WILL HAVE AT THE END

- Number Theory: Modular arithmetic, Diophantine equations, prime theory, quadratic reciprocity
- Combinatorics: Advanced counting, graph theory, generating functions, probabilistic method
- Algebra: Polynomials, functional equations, complex numbers, inequalities, optimization
- Geometry: Euclidean geometry, transformations, coordinate methods, construction problems
- Proof Techniques: Induction, contradiction, pigeonhole, invariants, extremal principle
- Inequalities: Classical inequalities, SOS method, smoothing, Lagrange multipliers
- Problem Solving: Pattern recognition, case analysis, working backwards, generalization
- Competition Skills: Time management, partial credit, proof writing, score optimization
- Advanced Topics: Generating functions, game theory, research mathematics introduction
- Mental Skills: Competition psychology, stress management, peak performance
- Mathematical Writing: Rigorous proofs, LaTeX, problem creation, solution exposition
- International Competitions: IMO, APMO, Putnam, national olympiads preparation

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week with a real teacher, working problems together
Homework: A problem set after every class, and a mixed-review assignment at the end of every month that deliberately folds in earlier topics
Practice: 3-5 hours of problem-solving between classes
Review: Every set reviewed with written feedback; recurring mistakes reopened in class until they are gone

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: which topics are solid, which are wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies mastery, not attendance

WHERE THIS CAN LEAD

- International Mathematical Olympiad Participant
- National Olympiad Team Member
- University Mathematics Major
- Mathematics Research Career
- Quantitative Finance Analyst
- Data Science and Machine Learning
- Cryptography and Security
- Academic Mathematics Professor
- Competition Mathematics Coach
- Mathematical Content Creator

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does the olympiad mathematics program cover?
A: The 12 month curriculum works through the four classical olympiad pillars: number theory and combinatorics, competition algebra with polynomials and functional equations, Euclidean and coordinate geometry with trigonometry, and inequalities with advanced problem solving strategy, ending in mock competitions. The goal is genuine problem solving depth, not memorized tricks.

Q: Which competitions does this prepare students for?
A: The training maps to the Indian olympiad pipeline, IOQM leading toward RMO and INMO, plus NMTC and SOF style olympiads, and the same material serves AMC and AIME aspirants since the topic areas overlap heavily. We prepare students honestly for the level they are at, and we never promise medals or ranks, because no honest program can.

Q: Is my child ready for this course?
A: It suits students with a strong grasp of school mathematics who enjoy hard problems, typically from middle school upward. It is not designed for closing school level gaps; students who need foundations first are better served by our school mathematics programs and can graduate into this one.

Q: What are the prerequisites?
A: Solid school algebra and geometry, care in reading mathematical statements, and a genuine appetite for problems that take longer than five minutes. Competition maths rewards consistent weekly practice more than raw talent, so expect a serious commitment.

Q: How is progress measured in competition maths?
A: Through weekly problem sets at calibrated difficulty, a personal problem log tracking which techniques are secure, and regular timed mock papers in the final months. Progress here looks like solving harder problems more independently, and the log makes that visible.

Q: How is my child assessed, and is the certificate just for attending?
A: No. A problem set follows every class with written feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, each phase ends with a cumulative exam, and the course ends with a final exam covering the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest. We measure understanding, not attendance.

Q: Can we try a session first?
A: Yes, book a free demo class at learn.modernagecoders.com/contact with no commitment; we use it to gauge your child's current problem solving level and to recommend honestly whether this course or a foundations course fits better. WhatsApp +91 91233 66161 for quick questions. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: What is the class format?
A: Live one hour online classes with a real teacher in deliberately small groups, never recorded lectures. Students present their attempts and the mentor works the reasoning with them, which is the only way competition thinking actually develops.

Q: Is there a certificate, and what else does my child take away?
A: Yes, an Olympiad Mathematics Excellence Certificate on completion, plus the more valuable artifact: a documented problem portfolio and mock paper record showing the level reached. That evidence outlasts any single contest result.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/olympiad-competition-mathematics-mastery

For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 95 OF 137: MATHS THROUGH CODING: LEARN MATH IN PYTHON (AGES 10-15)

==========

Course link: https://learn.modernagecoders.com/courses/maths-through-coding
Subject area: Mathematics & Creative Technology
Designed for: Middle school
Who can join: Ages 10-15 (grades 5-10); no coding experience required
Course length: 9 months (36 weeks)
Time needed: 2 live one-hour classes per week + one small build between classes
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)

In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live course where kids ages 10-15 learn maths by coding it: fractions, algebra, geometry and probability built as Python projects. Small live batches or 1-on-1, free demo.

In one line: If you can build it, you understand it. Students build the maths.

ABOUT THE PROGRAMME

A 9-month live course teaching mathematics the way our whole school believes it should be met: by building it. Students write Python programs that ARE the maths, a fraction simplifier that forces understanding of GCD, turtle geometry that makes angles visceral, probability simulators that settle arguments with ten thousand dice rolls, and function grapher projects that turn algebra into something you can watch. This is the signature Modern Age Coders fusion, taught by mentors genuinely fluent in both subjects, and for Ontario families, it happens to cover the coding expectations now examined inside math class from Grade 1. No prior coding needed; the mathematics runs roughly grades 5-9 territory with stretch room.

WHAT MAKES THIS COURSE DIFFERENT

- The maths is the syllabus, the code is the method: every program exists to force a mathematical idea into the open
- A computer rejects vague understanding, code that half-understands fractions simply does not run, which makes debugging a mathematics lesson
- Mentors fluent in both subjects, because a coding teacher who waves at the maths (or vice versa) breaks the whole idea
- Real Python from week one, at a pace built for ages 10-15
- Every topic ends with a working artifact the student built and can demo
- Directly serves Ontario's coding-in-math strand, and the computational-thinking push in curricula worldwide
- The engagement answer for the child who says maths is boring but plays with code for hours, or the reverse
- One dedicated mentor, one full interactive hour, twice a week

THE LEARNING PATH, STEP BY STEP

Phase 1: Numbers as programs: Python foundations through arithmetic, divisibility, fractions and number patterns
Phase 2: Shape and change: turtle geometry, coordinates, and algebra brought alive through variables, functions and graphing
Phase 3: Chance and data: probability simulators, statistics on real datasets, and a capstone project of the student's own design

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Ages 10-15 (roughly grades 5-10)
Math background: Comfortable whole-number arithmetic; everything else is built inside the course
Coding experience: None required, Python taught from zero
Equipment: Computer with stable internet; free tools only (no purchases)
English: Course taught in English
Motivation: Curiosity about either half, maths or code, is enough; the course supplies the other

WHO THIS COURSE IS FOR

Bored by worksheets: The child who says maths is boring but builds in Minecraft for hours
Maths lovers: The child who loves maths and deserves the best maths toy ever made
Ontario families: Families facing coding-inside-math curriculum expectations with nobody home fluent in both
Understanding seekers: Students who pass tests but want (or need) the ideas to actually make sense
Future builders: Kids headed for our programming tracks who want the mathematical spine first

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: NUMBERS AS PROGRAMS (MONTHS 1-3)

Python foundations learned through number theory a student can touch: parity, divisibility, primes, factors and fractions. By phase end the student programs comfortably and, the real goal, has met arithmetic ideas so concretely they can never again be mysterious.

Month 1: First Programs, First Number Ideas (Weeks 1-4)
Hello Python: Editor, Print and Variables
Topics:
- Setting up and using the Python editor
- The print function
- Variables and naming
- Numbers and text as data
- A diagnostic of maths comfort
- A diagnostic of machine comfort
Projects built:
- Write a Python script that prints a personalised greeting built from variables
Practice: Write and run a short program that stores your name and age in variables and prints a sentence.
Detail: Hello, Python: the editor, print, variables, and a diagnostic of both maths comfort and machine comfort
Arithmetic Operators and Order of Operations
Topics:
- Addition, subtraction, multiplication and division operators
- Order of operations in code
- Brackets to control evaluation
- Integer versus decimal results
- Matching code grammar to maths grammar
- Predicting an expression before running it
Projects built:
- Write a Python calculator that evaluates a multi-step expression and prints the result
Practice: Predict the value of several mixed-operator expressions, then check each by running it.
Detail: Arithmetic operators and order of operations: the computer obeys the same grammar your maths teacher does
Integer Division, Remainders and Modulo
Topics:
- Integer division
- The modulo operator
- Modulo as a parity machine
- Even and odd detectors
- Cycle and repeating patterns with modulo
- The clock-arithmetic idea
Projects built:
- Write a Python script that labels every number from 1 to 50 as even or odd using modulo
Practice: Use modulo to test a list of numbers for even or odd and describe the pattern you see.
Detail: Integer division and remainders: modulo as the parity machine, even/odd detectors and cycle patterns
Conditionals and Divisibility Rules
Topics:
- if, elif and else
- Comparison operators
- Coding divisibility tests
- Explaining divisibility rules backwards
- Combining conditions
- Reading a program's branches
Projects built:
- Write a Python program that reports which of 2, 3, 5 and 10 a number is divisible by
Practice: Code a divisibility checker for 2, 3 and 5 and explain in your own words why each rule works.
Detail: Conditionals with number tests: divisibility rules coded and then explained backwards

Month 2: Loops Meet Number Theory (Weeks 5-8)
Loops and Times-Table Patterns

      Topics:
        - for loops and range
        - Loops as purposeful repetition
        - Generating times tables
        - Hunting patterns inside the tables
        - Explaining the patterns found
        - The nested-loop idea introduced
      Projects built:
        - Write a Python script that visualises a times-table as a grid of rows and columns
      Practice: Generate several times tables with a loop and write down one pattern you
      notice in each.
      Detail: Loops as repetition with purpose: times tables generated, then patterns in them
      hunted and explained
    Factor Finders and Prime Testers
      Topics:
        - Finding all factors of a number
        - Building a prime tester
        - The definition of a prime
        - Why 1 is not prime, settled by code
        - Writing your first real algorithms
        - Testing an algorithm across many inputs
      Projects built:
        - Write a Python program that lists every factor of a number
        - Write a Python prime tester that prints all primes up to 100
      Practice: Run your prime tester across the numbers 1 to 30 and confirm the results by
      hand.
      Detail: Factor finders and prime testers: the student's first real algorithms, and why
      1 is not prime, settled by code

  Month 3: The Fraction Machine (Weeks 9-12)
    GCD and the Fraction Simplifier
      Topics:
        - Common factors and the GCD
        - The Euclidean idea for finding a GCD
        - Coding a GCD function
        - Fraction equivalence
        - Building a fraction simplifier
        - Why the machine forces real understanding
      Projects built:
        - Write a Python fraction simplifier that reduces any fraction to lowest terms using the GCD
      Practice: Simplify several fractions by hand, then confirm each one with your
      simplifier.
      Detail: GCD discovered through the Euclidean idea; a fraction simplifier built, which
      forces equivalence to be truly understood
    Fraction Arithmetic and Phase Demo Day
      Topics:
        - Adding and subtracting fractions in code
        - Why a machine needs common denominators
        - Multiplying and dividing fractions
        - Combining the GCD with fraction arithmetic
        - Preparing a clear demo
        - Presenting the number toolkit
      Projects built:
        - Write a Python fraction calculator that adds, subtracts and simplifies two fractions
      Practice: Extend your fraction program to add and subtract two fractions and simplify
      the result.
      Assessment: Phase demo day: each student presents their number toolkit and explains the
      mathematics behind it.
      Detail: Fraction arithmetic programmed (common denominators become obvious when a
      machine needs them); phase demo day: each student presents their number toolkit

---
PHASE 2: SHAPE AND CHANGE (MONTHS 4-6)

Geometry becomes visceral through turtle graphics, exterior angles are unforgettable once you
have steered them, and algebra comes alive as code: variables that actually vary, functions
as machines, and graphs the student plots themselves.

  Month 4: Turtle Geometry (Weeks 13-16)
    Turtle Basics and the Exterior-Angle Discovery
      Topics:
        - Turtle graphics setup
        - Drawing squares and triangles
        - Forward and turn commands
        - Discovering exterior angles by steering
        - Why 360 divided by n draws a regular polygon
        - Connecting turning to angle size
      Projects built:
        - Write a Python turtle program that draws any regular polygon from the number of sides

Practice: Draw three different regular polygons with turtle and record the turn angle each one needs.
Detail: Turtle basics: squares, triangles and the exterior-angle discovery (why 360 divided by n draws every regular polygon)

Star Polygons, Spirals and Angle Art

Topics:
- Drawing star polygons
- Spirals with a changing step or angle
- Angle-art compositions
- Interior-angle sums
- Verifying angle sums by walking them
- Combining loops with turtle turns

Projects built:
- Write a Python turtle program that draws a star polygon or spiral of your own design

Practice: Create an angle-art pattern and explain the interior or exterior angles that make it close.
Detail: Star polygons, spirals and angle art; interior-angle sums verified by walking them

Month 5: Coordinates and Real Variables (Weeks 17-20)

The Coordinate Plane in Code

Topics:
- The screen as a coordinate plane
- Plotting points from coordinates
- Drawing lines between points
- Finding midpoints
- The distance-between-points idea
- Reading coordinates back off a drawing

Projects built:
- Write a Python program that plots points and draws the line and midpoint between two of them

Practice: Plot a set of points in code and compute the midpoint of two of them.
Detail: The coordinate plane as the screen it literally is; plotting points, lines and midpoints in code

Algebra as Code: Expressions and Equations

Topics:
- Evaluating algebraic expressions in code
- Variables that actually vary
- Solving equations by guess, check and refine
- Why balance methods beat brute force
- Substituting into a formula
- Checking a solution programmatically

Projects built:
- Write a Python script that solves a linear equation by guess-check-refine and prints each step

Practice: Solve a linear equation both with a guess-check-refine loop and by balancing, then compare.
Detail: Algebra as code: expressions evaluated, equations solved by guess-check-refine, and why balance methods beat brute force

Month 6: Functions as Machines, Literally (Weeks 21-24)

Functions as Machines: Input, Rule, Output

Topics:
- Defining a Python function
- Parameters and return values
- Mathematical functions as machines
- Input, rule and output as one idea
- Evaluating a function for many inputs
- Building a table of values

Projects built:
- Write a Python function that takes x and returns a rule such as 2x plus 1, then tabulates it

Practice: Write a function for a simple rule and print a table of its outputs for several inputs.
Detail: Python functions and mathematical functions introduced as the same idea: input, rule, output

The Grapher Project and Phase Demo Day

Topics:
- Plotting y = mx + c
- The role of the gradient m
- The role of the intercept c
- Plotting parabolas
- Watching parameters reshape the graph
- Presenting the grapher at demo day

Projects built:
- Write a Python grapher that plots y = mx + c for chosen values of m and c
- Extend the grapher to plot a parabola and watch how its shape changes

Practice: Use your grapher to compare two lines with different m and c and describe the difference.
Assessment: Phase demo day: each student presents their grapher and explains how m and c shape the line.

Detail: The grapher project: plot y = mx + c and watch m and c do their jobs; then parabolas, and phase demo day

---

PHASE 3: CHANCE, DATA AND THE CAPSTONE (MONTHS 7-9)

Probability stops being formulas when you can roll ten thousand dice per second: simulators settle intuitions, real datasets get summarized and questioned, and the course closes with a capstone the student designs, a working program where the mathematics is the star.

Month 7: Probability by Simulation (Weeks 25-28)

Randomness: Coin Flippers and Dice Rollers

Topics:

- Random numbers in Python
- Coin-flip simulation
- Dice-roll simulation
- Experimental probability from many trials
- Theoretical probability
- Watching results converge as trials grow

Projects built:

- Write a Python script that simulates thousands of dice rolls and reports the proportion of each outcome

Practice: Simulate 1000 coin flips and compare the experimental probability with the theoretical value.

Detail: Randomness in code: coin flippers and dice rollers; experimental versus theoretical probability, watched converging

Simulating Classic Probability Puzzles

Topics:

- Two-dice sum distributions
- Why 7 is the most likely sum
- Streaks in coin flips
- Calibrating intuition with evidence
- Counting outcomes versus simulating them
- Comparing simulation with theory

Projects built:

- Write a Python simulation of two-dice sums that displays the frequency of each total

Practice: Simulate two-dice sums many times and chart how often each sum appears.

Detail: The classic puzzles simulated: two-dice sums, streaks in coin flips, intuition calibrated by evidence

Month 8: Statistics on Real Data (Weeks 29-32)

Mean, Median and Range in Code

Topics:

- Computing the mean in code
- Finding the median by sorting
- Calculating the range
- Making vague definitions precise
- Handling a list of data
- Checking results against a hand calculation

Projects built:

- Write a Python program that reads a list of numbers and reports the mean, median and range

Practice: Write code to compute the mean, median and range of a data list, then verify each by hand.

Detail: Mean, median and range computed by the student's own code, the definitions can no longer be vague

Exploring a Real Dataset

Topics:

- Loading a real dataset
- Summarising the data
- Making simple charts
- Spotting outliers
- Questioning the sample size
- Judging whether a comparison is fair

Projects built:

- Write a Python program that summarises a real dataset and draws a simple chart from it

Practice: Summarise a small real dataset and write down two sceptic's questions it raises.

Detail: A real dataset explored: summaries, simple charts, and the sceptic's questions (outliers, sample size, fair comparisons)

Month 9: The Capstone (Weeks 33-36)

Capstone Chosen and Scoped

Topics:

- Choosing a capstone idea
- Scoping the mathematics inside it
- Planning the build in steps
- Setting a realistic target
- Options: a trainer, an art generator, a game or a data story
- Mentor rigour applied to the maths

Projects built:
- Begin building your chosen capstone program in Python from your scoped plan

Practice: Write a one-page plan for your capstone that names the mathematics it will make visible.

Detail: Capstone chosen and scoped: a times-table trainer, geometry art generator, probability game, data story, the student's call, the mentor's rigor

Capstone Build, Test and Demo Day

Topics:
- Building the capstone
- Testing and debugging
- Polishing the program
- Preparing a clear demo
- Presenting the program
- Explaining the mathematics behind it

Projects built:
- Complete, test and polish your capstone program ready to demo

Practice: Test your capstone on several cases, fix what breaks, and rehearse your demo explanation.

Assessment: Final demo day: each student presents the capstone and explains the mathematics it is built on.

Detail: Build, test, polish; demo day, the student presents their program and, more importantly, explains its mathematics

PROJECTS THE STUDENT BUILDS AND KEEPS

- Number toolkit: parity detector, divisibility tester, prime finder, factor lister
- The fraction machine: simplifier and calculator built on real GCD understanding
- Turtle geometry gallery: polygons, stars and angle art with the maths explained
- Function grapher: lines and parabolas plotted, parameters explored live
- Probability simulators and a real-data statistics story
- The capstone: a self-designed program where the mathematics is the star

Total projects: 12+ working programs, demo-ready

SKILLS THE STUDENT WILL HAVE AT THE END

- Number theory made concrete: primes, factors, GCD, modulo
- Fraction understanding at machine-proof depth
- Geometry through construction: angles, polygons, coordinates
- Algebra as living code: variables, expressions, functions
- Linear functions and graphing
- Probability by simulation and statistics on real data
- Python fundamentals: variables, conditionals, loops, functions
- Computational thinking and debugging discipline

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live build-and-explain classes
Between class: One small build or extension challenge
Demo days: Each phase ends with the student presenting their work
Total per week: About 3-4 absorbed hours
Homework: A problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week
Mentorship: One mentor fluent in both mathematics and programming
Recordings: Every class recorded
Progress tracking: Project portfolio + concept checks shared with parents
Flexibility: Slots move around school and family plans

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies mastery, not attendance

WHERE THIS CAN LEAD

- Mathematics that holds through high school, because it was built
- Ready for our full Python and AI programming masterclasses
- Computational-thinking maturity that shows up in science and school projects
- A natural bridge to competition mathematics for the puzzle-hungry

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this a maths course or a coding course?
A: A maths course that uses code as its laboratory. The syllabus is mathematical, number theory, fractions, geometry, algebra, probability, statistics, and every program exists to force one of those ideas into the open. Students do finish with real Python skills, which is a genuine bonus, but the mathematics is the star.

Q: Does my child need coding experience?
A: None. Python is taught from the first hello, at a pace built for ages 10-15. Children who already code a little move faster into the mathematical meat; the mentor adjusts either way.

Q: Why does building things teach maths better than practicing problems?
A: Because a computer rejects vague understanding. A student who 'sort of' knows fraction equivalence writes a simplifier that fails, and fixing it requires actually understanding GCD. That debugging loop, predict, test, explain the gap, is the deepest mathematical practice there is, and it feels like play.

Q: We're in Ontario and coding is now examined inside math class. Does this help?
A: Directly, this course is our sweet spot for exactly that curriculum. Ontario weaves coding expectations through math from Grade 1, most homes have nobody fluent in both halves, and our mentors teach both natively. Students walk back into class ahead on the strand instead of puzzled by it.

Q: My child loves games but says maths is boring. Will this work?
A: This course exists for that child. The maths arrives wearing a project the child wants to build, angle art, a dice game, a grapher, and engagement does the heavy lifting. The reverse child (loves maths, indifferent to screens) discovers code as the best maths toy ever made.

Q: What maths level does it assume and reach?
A: It starts from comfortable whole-number arithmetic (roughly grade 5) and reaches into linear functions, probability and statistics (roughly grade 9 territory), with stretch room for strong students. It runs happily alongside any school curriculum and strengthens it rather than clashing.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A problem set follows every class with written feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final exam plus a portfolio of working maths programs. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real understanding, not attendance.

Q: Can we try it before paying?
A: Yes. Every student starts with a free live demo class, they will build and run their first small program inside that hour, and you will see the method working with your own child. No card details required.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/maths-through-coding
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 96 OF 137: MENTAL MATHS FOR KIDS: NUMBER SENSE, NOT TRICKS (AGES 5-10)

==========

Course link: https://learn.modernagecoders.com/courses/mental-maths-mastery-kids
Subject area: Elementary Mathematics
Designed for: Elementary school
Who can join: Ages 5-10 (roughly grades K-5), all starting levels
Course length: 12 months (52 weeks)
Time needed: 2 live one-hour classes per week + 10 minutes of daily play-practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live mental maths course for kids ages 5-10: number bonds, decomposition and estimation build real number sense, the honest alternative to abacus. Free demo.

In one line: Number sense, not tricks. The speed follows, and it never expires.

ABOUT THE PROGRAMME

A 12-month live program that builds genuine mental arithmetic in children aged 5-10, the honest alternative to abacus franchises and 'Vedic tricks' courses. Trick-based speed impresses at demos and quietly expires: it bypasses understanding, so it cannot transfer to fractions, algebra or problem solving. We build number sense instead, numbers as decomposable quantities, number bonds, compensation and estimation, through games and puzzles a child genuinely enjoys. The result is a child who calculates fast because they see numbers clearly, and that clarity keeps paying through every year of school mathematics.

WHAT MAKES THIS COURSE DIFFERENT

- Number sense over tricks: strategies a child can explain always beat procedures they can only perform
- The speed is real and durable, decomposition, bonds and compensation transfer straight into fractions and algebra later
- Games, puzzles and number talks: children wait for these classes, which is the engagement no drill sheet buys
- Concrete-pictorial-abstract progression, the method behind the world's strongest maths systems
- Explain-back in every class: what a child can teach, a child owns
- One dedicated mentor who knows exactly which facts and strategies each child holds
- A genuine on-ramp: graduates flow into our grade-level maths programs already ahead
- One full interactive hour per class, twice a week, built for young attention spans with variety, not lectures

THE LEARNING PATH, STEP BY STEP

Phase 1: Number sense foundations: quantities, bonds to 10 and 20, and the part-whole thinking everything else stands on
Phase 2: The strategy toolkit: make-ten, doubles, compensation, and multiplication as structure, not chanting
Phase 3: Big-number courage: multi-digit mental strategies, fraction sense beginnings, estimation mastery and speed with understanding

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Ages 5-10; can count to 20 to start the youngest track
Math background: None assumed, the diagnostic finds the true stage
Equipment: Computer or tablet with stable internet; household objects double as manipulatives
Time commitment: 2 live hours plus 10 minutes of daily play
English: Course taught in English
Motivation: None needed on day one, building it is the course's job

WHO THIS COURSE IS FOR

Finger counters: Children counting on fingers past grade 2, whose bonds never became automatic
Maths worriers: Children already saying 'I'm bad at maths', the belief this course exists to reverse
Abacus shoppers: Parents comparing abacus and Vedic programs who want speed that transfers and lasts
Young starters: Ages 5-6 whose parents want the foundation laid right the first time
Quick kids: Naturally strong children ready to go deeper than school arithmetic allows

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: NUMBER SENSE FOUNDATIONS (MONTHS 1-4)

Before speed comes seeing: numbers as quantities that break apart and recombine. This phase builds subitizing, part-whole thinking and the bonds of 10 and 20 through concrete materials and games, tuned to each child's starting stage.

Month 1: Seeing Numbers (Weeks 1-4)

Getting to Know Your Number Sense

Topics:
- Counting games that show how far a child counts with ease
- Quick quantity flashes to see which small numbers are known on sight
- Simple addition play that reveals the current strategy (counting on, known facts, or guessing)
- Noticing confidence and comfort with numbers, not only right answers
- Spotting facts already automatic, such as small doubles or bonds
- Setting a friendly starting point tuned to the child, not the age

Practice: Over a snack, ask your child to grab a small handful of raisins and guess how many before counting, then count together to check, keeping it light and playful.
Detail: Playful diagnostic: which facts, strategies and confidence the child already holds, disguised as games

Seeing How Many at a Glance

Topics:
- Recognising 1 to 5 dots instantly without counting (subitizing)
- Reading dice patterns as whole amounts
- Meeting the ten-frame as a picture of numbers
- Seeing 6 to 10 as a small group sitting on top of five
- Flashing and hiding quantities to build quick recognition
- Talking about how the amount was seen, not just the answer

Practice: Roll one die a few times each evening and race to say the number before counting the dots, and whoever spots it first wins that round.

Detail: Subitizing and quantity sense: dots, dice and ten-frames, recognising amounts without counting

Numbers Are Made of Smaller Numbers

Topics:
- Breaking a small number into two parts in different ways
- Seeing 7 as 5 and 2, 6 and 1, or 4 and 3
- Using fingers or counters to split a group across two hands
- Understanding that the parts always rebuild the whole
- Finding every way to make a chosen number
- Explaining a split out loud in the child's own words

Practice: Give your child 6 buttons and ask how many ways they can split them into two piles, writing each way down together like a treasure list.

Detail: Part-whole thinking: every number is made of other numbers (7 is 5-and-2, 6-and-1, 3-and-4)

Living on the Number Line

Topics:
- Placing numbers in order along a line
- Deciding which number is bigger from its position
- Measuring the distance or jump between two numbers
- Hopping forwards and backwards along the line
- Meeting the empty number line as a thinking tool
- Estimating where a number belongs before checking

Practice: Draw a long line on paper with 0 at one end and 10 at the other, then take turns pointing to where numbers like 7 or 3 should sit.

Detail: The number line as home: position, order, distance and the empty-number-line game

Months 2-3: The Bonds That Power Everything (Weeks 5-12)

The Pairs That Make Ten

Topics:
- Finding both parts that add to make 10
- Recognising 6 and 4, 7 and 3, 8 and 2 as ten-friends
- Using ten-frames to see the missing part
- Answering ten-bonds quickly through repeated play
- Spotting a ten-bond hidden inside a larger problem
- Building speed through fun, never flashcard pressure

Practice: Play make ten: you say a number under ten and your child shouts the partner that finishes the ten, then swap roles after five turns.

Detail: Bonds of 10 to automaticity, through games, never flashcard grind

Doubles and Their Neighbours

Topics:
- Learning the doubles up to 10 plus 10 as anchor facts
- Using a known double to solve a near-double (6 plus 7 is double 6 and one more)
- Adding 9 by adding 10 and stepping back one
- Adding 11 by adding 10 and stepping up one
- Seeing how one known fact unlocks several nearby ones
- Choosing the closest double to lean on

Practice: At bedtime, call out a double like double 4 for a quick answer, then a near-double like 4 plus 5, and let your child explain how the double helped.

Detail: Doubles and near-doubles as anchor facts; adding 9 and 11 by compensation preview

Making Ten to Add Bigger

Topics:
- Splitting the second number to fill up to ten first
- Turning 8 plus 5 into 10 plus 3
- Using ten-frames to see the bridge across ten
- Knowing how much a number needs to reach ten
- Adding the leftover part after reaching ten
- Choosing make-ten when numbers sit close to ten

Practice: With two ten-frames and some counters, set up sums like 8 plus 5 and let your child slide counters to fill the first ten, then read the answer.

Detail: Bridging through 10: the make-ten strategy that turns 8+5 into 10+3

Bonds to Twenty and Fact Families

Topics:
- Finding pairs that add to make 20
- Linking a ten-bond to its twenty partner
- Seeing a fact family: 13 plus 7, 7 plus 13, 20 minus 7, 20 minus 13
- Understanding addition and subtraction as two views of one picture
- Using a known adding fact to find its subtracting partner
- Building all four facts from one number trio

Practice: Pick three numbers that fit together, like 6, 4 and 10, and challenge your child to say all four facts they can make from them.
Detail: Bonds to 20 and fact families: addition and subtraction as one idea seen from two sides

Month 4: Subtraction as Distance + Stage Checkpoint (Weeks 13-16)
Two Ways to Subtract
Topics:
- Taking away by counting back from the larger number
- Finding the gap by counting up between the numbers
- Deciding which way is faster for a given problem
- Seeing subtraction as the distance between two numbers
- Using the number line to picture both methods
- Explaining which route was chosen and why
Practice: Ask a shopping question like you have 10 and a toy costs 8, how much change, and let your child try both counting back and counting up to compare.
Detail: Subtraction two ways: taking away versus finding the gap, and when each is faster
The Shopkeeper's Counting-Up Trick
Topics:
- Counting up from the price to the money paid
- Using friendly stops like the next ten
- Adding the small jumps to find the change
- Seeing why counting up avoids hard take-away sums
- Playing shop with real or pretend coins
- Checking the change makes sense
Practice: Set up a tiny pretend shop with priced items and let your child be the shopkeeper, counting up from the price to give you the right change.
Detail: Counting up for change: the shopkeeper's method every adult secretly uses
Many Roads to the Same Answer
Topics:
- Solving one problem in more than one way
- Sharing a strategy out loud in a number talk
- Listening to a different route to the same total
- Comparing which method felt quickest
- Respecting that different minds pick different paths
- Naming the strategy used (make-ten, doubles, or counting up)
Practice: Give one sum like 7 plus 8 and ask your child for two different ways to solve it, then you share a third way you thought of.
Detail: Mixed strategy play: number talks where children compare routes to the same answer
Phase One Check Through Play
Topics:
- Revisiting bonds of 10 and 20 through games
- Choosing a strategy for each problem confidently
- Explaining the thinking behind an answer
- Teaching a strategy back to the mentor or a parent
- Celebrating the facts now known by heart
- Noticing which few facts still want more play
Practice: Let your child be the teacher for ten minutes and show you one number strategy they learned this phase, explaining each step.
Assessment: A relaxed review game covering number bonds, part-whole thinking and the early strategies, where the child solves a problem, explains it, and teaches a favourite method back, with no timing or pressure.
Detail: Stage mastery check through games: solve it, explain it, teach it back

---
PHASE 2: THE STRATEGY TOOLKIT (MONTHS 5-8)

The heart of mental maths: named strategies the child chooses between, make-ten, compensation, doubling and halving, plus multiplication met as structure and arrays rather than chanted tables. Choice is the skill; speed is the side effect.

Month 5: Two-Digit Courage (Weeks 17-20)
Adding Two-Digit Numbers in Jumps
Topics:
- Adding the tens first in one jump
- Then adding the leftover ones
- Using an empty number line to record the jumps
- Holding the running total in mind
- Solving sums like 34 plus 23 step by step
- Checking the answer feels about right
Practice: Call out a two-digit sum like 42 plus 30 during a car ride and ask your child to jump the tens out loud, then add any ones.
Detail: Adding two-digit numbers mentally: jumps of ten, then the leftover ones
The Compensation Superpower
Topics:
- Rounding a number up to a friendly ten
- Adjusting the other number to keep the total fair

- Turning 39 plus 26 into 40 plus 25
- Subtracting 19 by taking 20 and giving one back
- Understanding why the give-and-take keeps it equal
- Choosing compensation when a number sits near a ten

Practice: Give a near-ten sum like 29 plus 14 and let your child bump the 29 up to 30, solve it, then give the one back at the end.

Detail: Compensation as a superpower: 39 + 26 becomes 40 + 25; subtracting 19 by taking 20 and giving one back

Month 6: Multiplication as Structure (Weeks 21-24)

Multiplication as Groups You Can See

Topics:

- Making equal groups with objects
- Building rows and columns as an array
- Reading 3 rows of 4 as three fours
- Connecting repeated addition to multiplication
- Seeing that an array can be counted two ways
- Using the words times and groups of

Practice: Arrange snacks or blocks into equal rows, like 3 rows of 5, and ask your child to find the total by seeing the groups rather than counting one by one.

Detail: Multiplication as groups and arrays, seen and built before recited

The Clever Route Through the Tables

Topics:

- Learning the easy 2s, 5s and 10s first
- Doubling the 2s to reach the 4s
- Doubling the 4s to reach the 8s
- Finding 9s by using 10s and stepping back a group
- Seeing how known tables build the harder ones
- Using skip counting as a rhythm for each table

Practice: Practise one table a day by skip counting it like a chant on the walk to the door, starting with the easy 2s, 5s or 10s.

Detail: The clever tables route: 2s, 10s and 5s first, then doubling to 4s and 8s; 9s by compensation

Month 7: The Full Tables, Owned (Weeks 25-28)

Taming the Trickiest Facts

Topics:

- Learning the 3s by skip counting and grouping
- Reaching the 6s by doubling the 3s
- Meeting the square facts like 6 times 6 and 7 times 7
- Spotting the small handful of facts that feel hardest
- Using a known nearby fact to reach a tricky one
- Practising just the stubborn facts, not the whole grid

Practice: Pick the one fact your child finds hardest this week and turn it into a silly rhyme or picture together to make it stick.

Detail: 3s, 6s and the tricky squares; the handful of hard facts isolated and conquered

Division Is Multiplication Backwards

Topics:

- Seeing division as sharing into equal groups
- Linking 3 times 4 is 12 to 12 shared by 4 is 3
- Building the multiplication and division fact family
- Using a known times fact to answer a divide fact
- Playing fact-family games for speed
- Understanding remainders as leftovers, gently

Practice: Share a small pile of objects equally between two or three plates and ask your child which times fact just proved the answer.

Detail: Division as the other side of multiplication; fact-family fluency through games

Month 8: Doubling, Halving and Strategy Choice (Weeks 29-32)

Doubling and Halving Magic

Topics:

- Halving one number while doubling the other
- Turning 16 times 5 into 8 times 10
- Seeing why the answer stays the same
- Choosing which number to halve to make it easy
- Using this to multiply by 5 quickly
- Chaining a halve and a double smoothly

Practice: Offer a times-five problem like 14 times 5 and let your child halve the 14 to 7 and double the 5 to 10, then read off 7 times 10.

Detail: Doubling and halving chains: 16 x 5 as 8 x 10, the strategy that feels like magic and is pure structure

Strategy Tournament and Phase Two Check

Topics:

- Solving one problem several different ways
- Deciding which strategy suits which problem
- Explaining why a chosen route was quickest
- Reviewing make-ten, compensation and doubling-halving
- Owning the full multiplication and division facts
- Thinking about one's own thinking

Practice: Hold a friendly strategy tournament: give one problem and see how many different correct routes your child can find and name.
Assessment: A relaxed review game across the strategy toolkit, where the child picks and explains a strategy for each problem and teaches a favourite one back, with no timing pressure.
Detail: Strategy tournaments: same problem, different routes, and the metacognition of choosing, plus stage mastery check

---
PHASE 3: BIG NUMBERS, ESTIMATION AND DURABLE SPEED (MONTHS 9-12)

The toolkit scales up: multi-digit mental arithmetic, the beginnings of fraction sense, estimation as judgment, and, now that understanding holds, genuine speed work. The course closes with each child measurably faster and, more importantly, able to explain every move.

Month 9: Multi-Digit Mental Arithmetic (Weeks 33-36)
Big Numbers Without Fear
Topics:
- Jumping by hundreds and tens using place value
- Adding to a friendly round number first
- Using compensation on larger numbers
- Adding numbers like 240 plus 130 in the head
- Keeping the place values lined up while jumping
- Checking the size of the answer makes sense
Practice: During a walk, add house or page numbers like 120 plus 60, encouraging your child to jump the hundreds and tens out loud.
Detail: Hundreds without fear: place-value jumps, friendly numbers and compensation at scale
Multiplying Beyond the Tables
Topics:
- Splitting a two-digit number into tens and ones
- Multiplying each part then adding (23 times 4 is 20 times 4 plus 3 times 4)
- Using halving and doubling on bigger products
- Choosing the neatest way to split a number
- Holding partial answers in mind to combine
- Estimating first to check the result
Practice: Give a problem like 21 times 3 and let your child split it into 20 times 3 and 1 times 3, then add the two parts.
Detail: Mental multiplication beyond tables: 23 x 4 by splitting, 15 x 6 by halving-doubling

Month 10: Estimation as Judgment (Weeks 37-40)
Rounding to Get Close Fast
Topics:
- Rounding a number to the nearest ten or hundred
- Using rounded numbers for a quick estimate
- Deciding how close an estimate needs to be
- Comparing an estimate to the real answer
- Rounding both numbers before adding
- Understanding when an estimate is enough
Practice: At the shop, ask your child to round a couple of prices and guess the rough total before you reach the till, then compare with the real total.
Detail: Rounding with purpose: getting close fast, and knowing how close is close enough
Does This Answer Make Sense?
Topics:
- Checking whether an answer looks reasonable
- Using an estimate to catch a big mistake
- Spotting answers that are far too big or too small
- Asking about how much this should be first
- Trusting number sense over a typed answer
- Explaining why an answer cannot be right
Practice: Say a deliberately silly answer to an easy sum, like 3 plus 4 is 30, and let your child catch it and explain why it is wrong.
Detail: The reasonableness habit: catching wrong answers by sense, the skill calculators cannot replace

Month 11: Fraction Sense Beginnings (Weeks 41-44)
Halves and Quarters of Things
Topics:
- Splitting a shape or group into equal parts
- Finding half of a quantity, like half of 12
- Finding a quarter by halving twice
- Seeing halves and quarters in real objects
- Understanding fair, equal shares
- Reading a simple fraction from a picture
Practice: Share a snack fairly and ask your child for half, then a quarter, of a small group of items, checking the shares are equal.
Detail: Halves, quarters and the fraction-of-a-quantity game, mental fractions from

pictures
Half, Point Five and Fifty Percent
Topics:
- Seeing one-half written as 0.5
- Meeting fifty percent as another name for half
- Linking a quarter to 0.25 and 25 percent
- Understanding these are the same amount in different clothes
- Spotting halves and percents in everyday life
- Reading these friendly numbers with confidence
Practice: Point out a half price or 50 percent off sign and ask your child what that means for a simple price, like half of 10.
Detail: Fraction and decimal friends: one-half is 0.5 is 50%, the bridge built early

Month 12: Speed With Understanding + Graduation (Weeks 45-52)
Beat Your Own Record
Topics:
- Using known strategies quickly and calmly
- Timing oneself gently to beat a personal best
- Tracking progress on a personal record ladder
- Choosing the fastest strategy for each fact
- Building speed on top of real understanding
- Racing only against yesterday's self, never a classmate
Practice: Set a friendly one-minute challenge of easy facts and let your child try to beat their own score from last time, celebrating any gain.
Detail: Timed play now that meaning holds: beat-your-own-record ladders, never child-versus-child pressure
Graduation and the Road Ahead
Topics:
- Choosing a favourite strategy to teach
- Running a number talk for the class or mentor
- Explaining each step clearly to others
- Reviewing all the strategies learned this year
- Celebrating durable, understood speed
- Looking ahead to the next maths adventure
Practice: Let your child teach the whole family their favourite strategy in a five-minute mini-lesson, using a real example.
Assessment: A gentle final review where the child leads a number talk teaching a chosen strategy, revisits the year's key ideas, and celebrates progress, with a friendly road-ahead plan instead of a test.
Detail: Graduation project: each child runs a number talk, teaching their favourite strategy, plus final stage mastery check and the road-ahead plan

PROJECTS THE STUDENT BUILDS AND KEEPS

- The child's own strategy book: every named strategy with their worked example
- Number-talk leadership: each child regularly explains routes to the class or mentor
- Beat-your-own-record fluency ladders, tracked across the year
- Estimation journal: real-life guesses checked and calibrated
- Graduation number talk taught by the child

Total projects: A year-long strategy book plus 4+ fluency artifacts

SKILLS THE STUDENT WILL HAVE AT THE END

- Number bonds to 20 at automaticity
- Make-ten, compensation and bridging strategies
- Doubling-halving and the clever tables route
- Full multiplication and division fact fluency
- Multi-digit mental arithmetic
- Estimation and reasonableness judgment
- Early fraction and decimal sense
- Explaining mathematical thinking aloud

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes, built with variety for young attention
Daily play: About 10 minutes of assigned games or number talks
Family involvement: Simple number-talk prompts parents can run at dinner
Total per week: About 3 focused hours
Homework: A problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week
Mentorship: One dedicated mentor who knows the child's exact fact map
Recordings: Every class recorded
Progress tracking: Stage mastery checks shared with parents
Flexibility: Slots move around school and family plans

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall
    short and there is focused revision and a free retest: the certificate certifies mastery, not
    attendance

WHERE THIS CAN LEAD

  - Ready for the Elementary Mathematics Masterclass, arriving ahead
  - The fluency base that makes fractions and early algebra feel light
  - Competition-maths readiness for children who catch fire
  - A natural bridge into maths-through-coding for the builder types

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: How is this different from abacus or Vedic maths classes?
  A: It rarely transfers to fractions, word problems or algebra, and it fades when practice
  stops. We build number sense instead: strategies the child understands and chooses between.
  The speed arrives a little later and then never leaves, because it is made of understanding.

  Q: What age is right to start?
  A: Ages 5-10, and the mentor meets each child at their actual stage rather than their age.
  Five-year-olds start with quantity games and bonds; nine-year-olds might start at the
  strategy toolkit. The playful diagnostic in the free demo places every child accurately.
  (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a
  guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo
  . Offer both, as RULE 3 says.)

  Q: Will my child actually enjoy this, or is it drilling in disguise?
  A: Enjoyment is the engine of the whole method: number talks, strategy tournaments,
  beat-your-own-record ladders and games carry every class. Parents consistently tell us their
  children wait for these classes, which sounds impossible for arithmetic until you watch one.
  That is what the free demo is for. (Agent note: the free demo is a waiting list with no fixed
  date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at
  https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: My child counts on fingers past age 7. Is that a problem this fixes?
  A: Yes, and gently. Finger counting past grade 2 means number bonds never became automatic,
  so every calculation starts from scratch. We rebuild the bonds through games until known
  facts replace counting, without ever shaming the fingers on the way.

  Q: Does this replace school maths or run alongside it?
  A: Alongside, and underneath: number sense is the foundation school maths quietly assumes.
  Children in this course typically find school arithmetic suddenly easy, which buys the
  confidence that changes everything else.

  Q: How much practice is needed between classes?
  A: About ten minutes a day of assigned play, a game, a strategy challenge, a number talk with
  a parent. Small daily doses beat weekend marathons for fluency building, and ten minutes
  keeps it joyful.

  Q: How is my child assessed, and is the certificate just for attending?
  A: No. A problem set follows every class with written feedback, monthly reviews deliberately
  mix new material with earlier topics so nothing is forgotten, and the course ends with a
  final assessment (a friendly speed-and-accuracy check of everything learned). Passing earns
  the certificate; falling short earns focused revision and a free retest. We measure real
  understanding, not attendance.

  Q: Can we try it before paying?
  A: Yes. Every child starts with a free live demo class that doubles as the playful
  diagnostic, no card details, no obligation.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/mental-maths-mastery-kids
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 97 OF 137: MERN STACK COURSE: MONGODB, EXPRESS, REACT AND NODE
==========

Course link: https://learn.modernagecoders.com/courses/mern-stack-development-masterclass-college
Subject area: Web Development
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals

Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  MERN stack course for college students and professionals: JavaScript foundations, React, Node and Express APIs, MongoDB and full-stack integration in 10-12 months.

  In one line: One language, the whole product: MongoDB, Express, React and Node learned as a single coherent craft, ending in a deployed full-stack capstone.

ABOUT THE PROGRAMME

  The MERN promise is coherence: JavaScript on both ends, one mental model from database to interface, and the fastest honest route to being the person who can build the whole thing. This live online masterclass delivers it in order: web and JavaScript foundations done properly, React to production patterns, then the backend half, Node and Express APIs designed like a professional, MongoDB modeled with judgment, authentication done safely, and finally full-stack integration: the two halves wired, deployed and behaving like one product. A finale month ships a complete MERN capstone and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

  - The stack taught as one craft: contracts between the halves get real attention
  - Foundations first: MERN on weak JavaScript is four buzzwords in a trench coat
  - Auth and security done safely: hashing, tokens and the OWASP basics as habits
  - Everything deploys: both halves live from their respective phases onward
  - An honest 10-month arc with 2 months in hand
  - Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

  Phase 1: Foundations (Months 1-3): the web and JavaScript learned properly
  Phase 2: React (Months 4-6): production patterns on the frontend half
  Phase 3: Node, Express and MongoDB (Months 7-9): the backend half with safe auth
  Phase 4: The finale (Month 10): full-stack integration capstone and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

  Coding experience: None. Everything is taught from the first line
  Equipment: Any computer with a stable internet connection; every tool used is free
  Age: 18+; college students, graduates and working professionals
  Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

  College students: Students who want the classic full-stack employability package
  Placement candidates: Candidates whose target roles say MERN in the listing
  Self taught: Learners with scattered tutorial knowledge who need it forged into one craft
  Product builders: Founders and makers who want the whole product in one language
  Career switchers: Professionals entering web development with one coherent path

FULL CURRICULUM, WEEK BY WEEK

  This is the complete teaching plan. Parents can ask about any week.

  PHASE 1: WEB DEVELOPMENT FOUNDATION (MONTHS 1-3, WEEKS 1-13)

  Build solid foundations in HTML, CSS, JavaScript, and modern web development tools.

    Month 1: HTML, CSS & Web Basics (Weeks 1-4)
      Introduction to Web Development
        Topics:
          - How the internet works: clients, servers, protocols
          - Web development career paths and opportunities
          - Development environment setup: VS Code, extensions

```
    - HTML5 fundamentals and document structure
    - Semantic HTML elements and accessibility
    - Forms and input validation
    - Media elements: images, audio, video
    - Tables and data representation
    - SEO basics and meta tags
    - HTML best practices and standards
    - Introduction to developer tools
    - Version control with Git and GitHub
  Projects built:
    - Personal portfolio HTML structure
    - Multi-page website
    - Contact form with validation
  Practice: Build 10 HTML pages from scratch
  Tools:
    - VS Code setup
    - Live Server extension
    - Git initialization
CSS Fundamentals & Styling
  Topics:
    - CSS syntax and selectors hierarchy
    - Box model: margin, padding, border
    - Colors, typography, and units
    - Flexbox layout system complete
    - CSS Grid layout mastery
    - Positioning: relative, absolute, fixed, sticky
    - Transforms, transitions, and animations
    - Pseudo-classes and pseudo-elements
    - CSS variables and custom properties
    - Mobile-first responsive design
    - Media queries and breakpoints
    - CSS preprocessors introduction (Sass)
  Projects built:
    - Responsive portfolio design
    - Landing page with animations
    - CSS art and illustrations
  Practice: Style 15 different layouts
  Tools:
    - Chrome DevTools
    - Sass compiler
    - CSS frameworks overview
Advanced CSS & Modern Techniques
  Topics:
    - Advanced Flexbox patterns
    - Complex Grid layouts
    - CSS architecture: BEM, SMACSS
    - CSS modules and scoping
    - Performance optimization techniques
    - Critical CSS and lazy loading
    - CSS-in-JS introduction
    - Tailwind CSS fundamentals
    - Bootstrap 5 framework
    - Material Design principles
    - Accessibility (a11y) best practices
    - Cross-browser compatibility
  Projects built:
    - Company website with Tailwind
    - Dashboard UI with Bootstrap
    - Accessible web application
  Practice: Create 10 responsive components
  Tools:
    - Tailwind CSS setup
    - PostCSS configuration
    - Autoprefixer
JavaScript Fundamentals
  Topics:
    - JavaScript basics: variables, data types, operators
    - Control flow: conditionals, loops, switch
    - Functions: declaration, expression, arrow functions
    - Scope, hoisting, and closures
    - Objects and arrays manipulation
    - Array methods: map, filter, reduce, forEach
    - String methods and regular expressions
    - Error handling: try, catch, finally
    - ES6+ features: destructuring, spread, rest
    - Template literals and tagged templates
    - Modules: import/export patterns
    - Debugging techniques and tools
  Projects built:
```

```
      - Interactive calculator
      - Todo list application
      - Quiz application
    Practice: Solve 50 JavaScript challenges
    Tools:
      - Node.js installation
      - npm basics
      - ESLint setup

Month 2: Advanced JavaScript & DOM (Weeks 5-9)
  DOM Manipulation & Events
    Topics:
      - Document Object Model (DOM) tree structure
      - Selecting elements: querySelector, getElementById
      - Creating and modifying elements
      - Event handling and event listeners
      - Event bubbling and delegation
      - Form handling and validation
      - Local storage and session storage
      - Cookies and web storage API
      - Drag and drop functionality
      - File API and uploads
      - Intersection Observer API
      - Performance optimization in DOM
    Projects built:
      - Dynamic content management system
      - Drag-and-drop task board
      - Image gallery with lazy loading
    Practice: Build 20 interactive features
    Tools:
      - Browser storage inspector
      - Performance profiler
      - Memory profiler
  Asynchronous JavaScript
    Topics:
      - Callbacks and callback hell
      - Promises: creation and chaining
      - Async/await syntax and patterns
      - Fetch API and HTTP requests
      - REST API consumption
      - Error handling in async code
      - Promise.all, Promise.race, Promise.allSettled
      - Web Workers for parallel processing
      - Service Workers basics
      - WebSockets introduction
      - Server-Sent Events (SSE)
      - Real-time data handling
    Projects built:
      - Weather application with API
      - Real-time chat interface
      - News aggregator
    Practice: Integrate 10 different APIs
    Tools:
      - Postman for API testing
      - JSON Server
      - WebSocket clients
  Object-Oriented & Functional Programming
    Topics:
      - OOP in JavaScript: classes, inheritance
      - Prototypes and prototype chain
      - Private and public fields
      - Static methods and properties
      - Design patterns: Singleton, Factory, Observer
      - Functional programming concepts
      - Pure functions and immutability
      - Higher-order functions
      - Currying and partial application
      - Composition over inheritance
      - SOLID principles in JavaScript
      - Code organization and architecture
    Projects built:
      - Game development with OOP
      - State management library
      - Design pattern implementations
    Practice: Implement 15 design patterns
    Tools:
      - TypeScript introduction
      - Jest for testing
      - Code coverage tools
```

```
  Build Tools & Modern Workflow
    Topics:
      - Module bundlers: Webpack fundamentals
      - Babel for transpilation
      - Package management with npm/yarn
      - Task runners and automation
      - Environment variables and configuration
      - Code splitting and lazy loading
      - Hot module replacement
      - Source maps for debugging
      - Production optimization
      - Progressive Web Apps basics
      - Testing fundamentals: unit, integration
      - Continuous Integration basics
    Projects built:
      - Custom webpack configuration
      - PWA with offline support
      - Automated build pipeline
    Practice: Configure 5 different build setups
    Tools:
      - Webpack DevServer
      - Babel presets
      - npm scripts
  Phase 1 Capstone Project
    Topics:
      - Project planning and wireframing
      - Git workflow and branching
      - Code review practices
      - Documentation writing
      - Deployment to Netlify/Vercel
    Projects built:
      - CAPSTONE: Full-Featured Web Application
      - Complete responsive website with JavaScript functionality
      - Include API integration, local storage, and animations
    Assessment: Phase 1 Final Assessment

Month 3: Advanced Frontend & Introduction to React (Weeks 10-13)
  Modern JavaScript & TypeScript
    Topics:
      - Advanced ES6+ features
      - Symbols, iterators, and generators
      - Proxy and Reflect APIs
      - WeakMap and WeakSet
      - TypeScript fundamentals
      - Type annotations and interfaces
      - Generics and type inference
      - Enums and type aliases
      - TypeScript with DOM
      - Configuration and tsconfig.json
      - TypeScript best practices
      - Migration from JavaScript
    Projects built:
      - TypeScript utility library
      - Type-safe API client
      - JavaScript to TypeScript migration
    Practice: Convert 10 projects to TypeScript
    Tools:
      - TypeScript compiler
      - ts-node
      - Type definition files
  Introduction to React
    Topics:
      - React philosophy and virtual DOM
      - Vite setup
      - JSX syntax and expressions
      - Components: functional vs class
      - Props and prop types
      - State management basics
      - Event handling in React
      - Conditional rendering
      - Lists and keys
      - Forms and controlled components
      - Component lifecycle (class components)
      - React DevTools usage
    Projects built:
      - React portfolio website
      - Interactive todo application
      - Recipe finder app
    Practice: Build 20 React components
```

```
      Tools:
        - Vite
        - React DevTools
        - React playground
    React Hooks & Modern Patterns
      Topics:
        - useState for state management
        - useEffect for side effects
        - useContext for global state
        - useReducer for complex state
        - useCallback and useMemo optimization
        - useRef for DOM references
        - Custom hooks creation
        - Rules of hooks
        - Hook patterns and best practices
        - Error boundaries
        - Suspense and lazy loading
        - Concurrent features introduction
      Projects built:
        - Blog platform with hooks
        - E-commerce product catalog
        - Real-time dashboard
      Practice: Create 15 custom hooks
      Tools:
        - React Hook Form
        - SWR/React Query
        - Hookstate
    Component Libraries & Styling
      Topics:
        - CSS Modules in React
        - Styled-components deep dive
        - Emotion CSS-in-JS
        - Material-UI components
        - Ant Design integration
        - Chakra UI setup
        - Headless UI components
        - Storybook for component development
        - Component documentation
        - Design systems creation
        - Theme management
        - Responsive design in React
      Projects built:
        - Custom component library
        - Design system implementation
        - Storybook showcase
      Practice: Build 30 reusable components
      Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it
      Tools:
        - Storybook setup
        - Chromatic for visual testing
        - Figma to React workflow

------------------------------------------------------------
PHASE 2: REACT MASTERY & ADVANCED FRONTEND (MONTHS 4-6, WEEKS 14-26)
------------------------------------------------------------
Master React ecosystem, state management, advanced patterns, and modern frontend development.

  Month 4: Advanced React & State Management (Weeks 14-19)
    React Router & Navigation
      Topics:
        - React Router v6 setup and configuration
        - Route definition and nesting
        - Dynamic routing and parameters
        - Protected routes and authentication
        - Programmatic navigation
        - Route guards and middleware
        - Lazy loading routes
        - 404 handling and redirects
        - Query parameters and state
        - Animated route transitions
        - Breadcrumbs implementation
        - Deep linking strategies
      Projects built:
        - Multi-page application router
        - Admin dashboard with auth
        - E-commerce site navigation
      Practice: Implement 10 routing patterns
      Tools:
```

```
    - React Router DevTools
    - Route testing utilities
    - History API
State Management with Redux
  Topics:
    - Redux principles and architecture
    - Actions, reducers, and store
    - Redux Toolkit modern approach
    - Slices and createSlice
    - Async actions with createAsyncThunk
    - RTK Query for data fetching
    - Redux DevTools and debugging
    - Middleware creation and usage
    - Redux persist for local storage
    - Normalization and selectors
    - Performance optimization
    - Redux best practices
  Projects built:
    - Shopping cart with Redux
    - Task management system
    - Real-time notifications
  Practice: Build 5 Redux applications
  Tools:
    - Redux Toolkit
    - Redux DevTools Extension
    - Immer for immutability
Alternative State Management
  Topics:
    - Context API advanced patterns
    - Zustand state management
    - MobX reactive programming
    - Recoil atomic state
    - Valtio proxy state
    - Jotai atomic approach
    - State machines with XState
    - Form state with React Hook Form
    - Server state with React Query
    - Local vs global state strategies
    - State management comparison
    - Choosing the right solution
  Projects built:
    - App with multiple state solutions
    - Form-heavy application
    - Real-time collaborative app
  Practice: Compare 5 state solutions
  Tools:
    - Zustand DevTools
    - React Query DevTools
    - XState Visualizer
Performance Optimization
  Topics:
    - React performance profiling
    - React.memo and PureComponent
    - useMemo and useCallback optimization
    - Code splitting strategies
    - Lazy loading components and routes
    - Bundle size optimization
    - Virtual scrolling implementation
    - Image optimization techniques
    - Web Vitals and metrics
    - Lighthouse audits
    - Performance monitoring
    - Production optimization
  Projects built:
    - Performance-optimized app
    - Large list virtualization
    - Image-heavy gallery
  Practice: Optimize 10 slow applications
  Tools:
    - React Profiler
    - Bundle analyzer
    - Lighthouse CI
Testing React Applications
  Topics:
    - Testing philosophy and strategies
    - Jest configuration and setup
    - React Testing Library
    - Component unit testing
    - Integration testing
```

```
      - Mocking and stubbing
      - Testing hooks and context
      - Async testing patterns
      - Snapshot testing
      - End-to-end with Cypress
      - Test coverage reports
      - CI/CD integration
    Projects built:
      - Full test suite for app
      - TDD React application
      - E2E test scenarios
    Practice: Write 100+ test cases
    Tools:
      - Jest
      - React Testing Library
      - Cypress

Month 5: Next.js & Server-Side Rendering (Weeks 20-23)
  Next.js Fundamentals
    Topics:
      - Next.js architecture and benefits
      - File-based routing system
      - Pages and layouts
      - Static generation (SSG)
      - Server-side rendering (SSR)
      - Incremental Static Regeneration (ISR)
      - API routes creation
      - Dynamic routes and params
      - Image optimization with next/image
      - Font optimization
      - SEO optimization
      - Internationalization (i18n)
    Projects built:
      - Blog with Next.js
      - E-commerce frontend
      - Portfolio with CMS
    Practice: Build 5 Next.js applications
    Tools:
      - Next.js CLI
      - Vercel deployment
      - Next.js DevTools
  Advanced Next.js Features
    Topics:
      - App Router and React Server Components
      - Streaming and Suspense
      - Parallel and intercepting routes
      - Route handlers and middleware
      - Authentication in Next.js
      - Database integration
      - Caching strategies
      - Edge functions and runtime
      - Serverless functions
      - Static export
      - Performance optimization
      - Production deployment
    Projects built:
      - Full-stack Next.js app
      - SaaS application frontend
      - Real-time application
    Practice: Master 10 Next.js patterns
    Tools:
      - Next.js Analytics
      - Edge Runtime
      - Turbopack
  GraphQL & Modern APIs
    Topics:
      - GraphQL fundamentals
      - Queries, mutations, subscriptions
      - Apollo Client setup
      - GraphQL with React
      - Code generation with GraphQL
      - Optimistic updates
      - Caching strategies
      - Error handling
      - File uploads with GraphQL
      - Real-time subscriptions
      - Federation and microservices
      - REST vs GraphQL comparison
    Projects built:
```

```
      - GraphQL-powered app
      - Real-time chat with subscriptions
      - Apollo Client integration
    Practice: Build 5 GraphQL integrations
    Tools:
      - Apollo Studio
      - GraphQL Playground
      - GraphQL Code Generator
  Progressive Web Apps
    Topics:
      - PWA principles and benefits
      - Service Workers deep dive
      - Offline functionality
      - Cache strategies
      - Web App Manifest
      - Install prompts and A2HS
      - Push notifications
      - Background sync
      - App shell architecture
      - Performance patterns
      - PWA testing and debugging
      - App store deployment
    Projects built:
      - Offline-first PWA
      - News app with notifications
      - PWA e-commerce site
    Practice: Convert 5 apps to PWAs
    Tools:
      - Workbox
      - PWA Builder
      - Lighthouse PWA audit

Month 6: Frontend Polish: Animation, Accessibility & Security (Weeks 24-26)
  Advanced Animation & Graphics
    Topics:
      - Framer Motion deep dive
      - Complex animations and gestures
      - SVG animations
      - Canvas API with React
      - WebGL and Three.js basics
      - React Three Fiber
      - Data visualization with D3.js
      - Chart libraries integration
      - Parallax scrolling effects
      - Scroll-triggered animations
      - Performance optimization
      - 60fps animations
    Projects built:
      - Animated landing page
      - Data visualization dashboard
      - 3D product showcase
    Practice: Create 20 animations
    Tools:
      - Framer Motion
      - GreenSock (GSAP)
      - Lottie React
  Accessibility & Internationalization
    Topics:
      - WCAG guidelines and compliance
      - Semantic HTML in React
      - ARIA attributes and roles
      - Keyboard navigation
      - Screen reader testing
      - Color contrast and visibility
      - Focus management
      - Accessible forms and errors
      - i18n setup with React
      - Translation management
      - RTL language support
      - Date and number formatting
    Projects built:
      - Fully accessible application
      - Multi-language platform
      - WCAG compliant site
    Practice: Audit and fix 10 applications
    Tools:
      - axe DevTools
      - NVDA/JAWS
      - react-i18next
```

```
    Security & Best Practices
      Topics:
        - Frontend security principles
        - XSS prevention in React
        - CSRF protection
        - Content Security Policy
        - Secure authentication flows
        - OAuth and social login
        - JWT handling and storage
        - Input validation and sanitization
        - Secure API communication
        - HTTPS and SSL
        - Security headers
        - Dependency vulnerability scanning
      Projects built:
        - Secure authentication system
        - Security audit implementation
        - Penetration testing prep
      Practice: Secure 5 applications
      Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it
      Tools:
        - OWASP ZAP
        - Snyk
        - npm audit

---------------------------------------------------------------
PHASE 3: BACKEND DEVELOPMENT WITH NODE.JS (MONTHS 7-9, WEEKS 27-39)
---------------------------------------------------------------
Master Node.js, Express, MongoDB, and build production-ready backend systems.

  Month 7: Node.js & Express Fundamentals (Weeks 27-31)
    Node.js Fundamentals
      Topics:
        - Node.js architecture and event loop
        - Global objects and process
        - Modules system: CommonJS and ES modules
        - File system operations
        - Path and OS modules
        - Streams and buffers
        - Events and EventEmitter
        - Child processes
        - Cluster module for scaling
        - Worker threads
        - Error handling in Node.js
        - Debugging Node applications
      Projects built:
        - CLI tool development
        - File processing system
        - Task automation scripts
      Practice: Build 15 Node.js utilities
      Tools:
        - Node.js debugger
        - nodemon
        - PM2
    Express.js Framework
      Topics:
        - Express application setup
        - Routing and route parameters
        - Middleware concept and creation
        - Request and response handling
        - Static file serving
        - Template engines integration
        - Body parsing and file uploads
        - Cookie and session handling
        - Error handling middleware
        - Security middleware setup
        - Compression and optimization
        - Express best practices
      Projects built:
        - RESTful API server
        - Blog backend system
        - File upload service
      Practice: Create 10 Express applications
      Tools:
        - Postman
        - Thunder Client
        - Morgan logger
    Database Design & MongoDB
```

```
    Topics:
      - Database fundamentals and types
      - MongoDB architecture and concepts
      - Document-based data modeling
      - CRUD operations in MongoDB
      - Indexes and performance
      - Aggregation framework
      - Transactions in MongoDB
      - Schema design patterns
      - References vs embedding
      - Data validation and migration
      - Backup and restore strategies
      - MongoDB Atlas cloud setup
    Projects built:
      - Database schema design
      - MongoDB aggregation pipelines
      - Data migration system
    Practice: Design 10 database schemas
    Tools:
      - MongoDB Compass
      - Atlas cloud
      - Robo 3T
  Mongoose ODM & Data Modeling
    Topics:
      - Mongoose setup and connection
      - Schema definition and types
      - Model creation and methods
      - Validation and custom validators
      - Middleware: pre and post hooks
      - Virtual properties
      - Population and references
      - Query building and optimization
      - Plugins and reusable logic
      - Transactions with Mongoose
      - Error handling patterns
      - Testing database operations
    Projects built:
      - User management system
      - E-commerce data models
      - Social media schemas
    Practice: Create 15 Mongoose models
    Tools:
      - Mongoose documentation
      - Faker.js for seeding
      - MongoDB Memory Server
  RESTful API Design
    Topics:
      - REST principles and constraints
      - API versioning strategies
      - Resource naming conventions
      - HTTP methods and status codes
      - Request/response format
      - Pagination and filtering
      - Sorting and field selection
      - API documentation with Swagger
      - Postman collection creation
      - API testing strategies
      - Rate limiting implementation
      - API security best practices
    Projects built:
      - Complete REST API
      - API documentation
      - API testing suite
    Practice: Build 5 different APIs
    Tools:
      - Swagger/OpenAPI
      - Postman collections
      - Newman CLI

Month 8: Authentication & Advanced Backend (Weeks 32-36)
  Authentication & Authorization
    Topics:
      - Authentication vs authorization
      - Password hashing with bcrypt
      - JWT token generation and validation
      - Refresh token implementation
      - Session-based authentication
      - OAuth 2.0 implementation
      - Social login integration
```

```
    - Two-factor authentication
    - Role-based access control (RBAC)
    - API key authentication
    - Security best practices
    - CSRF and XSS protection
  Projects built:
    - Complete auth system
    - OAuth integration
    - Multi-factor authentication
  Practice: Implement 5 auth strategies
  Tools:
    - Passport.js
    - JWT debugger
    - OAuth playground
Real-time Communication
  Topics:
    - WebSocket protocol understanding
    - Socket.io setup and configuration
    - Real-time events and broadcasting
    - Rooms and namespaces
    - Private messaging implementation
    - Presence detection
    - Reconnection handling
    - Scaling WebSocket servers
    - Redis adapter for Socket.io
    - Server-Sent Events (SSE)
    - WebRTC basics
    - Real-time notifications
  Projects built:
    - Chat application
    - Live collaboration tool
    - Real-time dashboard
  Practice: Build 5 real-time features
  Tools:
    - Socket.io client/server
    - Redis pub/sub
    - WebRTC libraries
File Handling & Cloud Storage
  Topics:
    - File upload strategies
    - Multer configuration and usage
    - Image processing with Sharp
    - Video processing basics
    - AWS S3 integration
    - Cloudinary integration
    - File validation and security
    - Streaming large files
    - ZIP file generation
    - CSV and Excel processing
    - PDF generation
    - CDN integration
  Projects built:
    - Media upload system
    - Cloud storage integration
    - Document processing service
  Practice: Handle 10 file types
  Tools:
    - Multer
    - Sharp
    - AWS SDK
Email & Notification Systems
  Topics:
    - Email service setup
    - SMTP configuration
    - SendGrid/Mailgun integration
    - HTML email templates
    - Email queue implementation
    - Bounce handling
    - Push notifications setup
    - SMS integration with Twilio
    - In-app notifications
    - Notification preferences
    - Unsubscribe management
    - Email analytics
  Projects built:
    - Email service
    - Notification system
    - Marketing automation
  Practice: Implement 5 notification channels
```

```
    Tools:
      - Nodemailer
      - SendGrid
      - Bull queue
  Caching & Performance
    Topics:
      - Caching strategies and patterns
      - Redis setup and operations
      - Memory caching with Node.js
      - Database query optimization
      - API response caching
      - CDN caching strategies
      - Session store with Redis
      - Rate limiting with Redis
      - Queue implementation
      - Performance monitoring
      - Load testing
      - Optimization techniques
    Projects built:
      - Caching layer implementation
      - Performance optimization
      - Load testing suite
    Practice: Optimize 5 slow endpoints
    Tools:
      - Redis
      - Artillery
      - New Relic

Month 9: GraphQL, Testing & Security (Weeks 37-39)
  GraphQL Backend
    Topics:
      - GraphQL server setup
      - Schema definition
      - Resolvers implementation
      - Mutations and queries
      - Subscriptions setup
      - DataLoader for N+1 problem
      - Authentication in GraphQL
      - File uploads in GraphQL
      - Error handling
      - Testing GraphQL APIs
      - Federation setup
      - Performance optimization
    Projects built:
      - GraphQL API server
      - Subscription service
      - Federated GraphQL
    Practice: Build 3 GraphQL APIs
    Tools:
      - Apollo Server
      - GraphQL Playground
      - DataLoader
  Testing & Quality Assurance
    Topics:
      - Testing strategies for Node.js
      - Unit testing with Jest
      - Integration testing
      - API testing with Supertest
      - Database testing strategies
      - Mocking and stubbing
      - Test coverage reports
      - End-to-end testing
      - Load testing
      - Security testing
      - Continuous testing
      - TDD and BDD approaches
    Projects built:
      - Complete test suite
      - CI/CD pipeline with tests
      - Load testing scenarios
    Practice: Write 200+ tests
    Tools:
      - Jest
      - Supertest
      - Sinon
  Security & Compliance
    Topics:
      - OWASP Top 10 for Node.js
      - SQL injection prevention
```

```
        - NoSQL injection prevention
        - Input validation and sanitization
        - Rate limiting strategies
        - DDoS protection
        - Helmet.js configuration
        - CORS setup and security
        - Environment variables security
        - Secrets management
        - GDPR compliance
        - PCI DSS basics
      Projects built:
        - Security audit
        - Compliance implementation
        - Penetration testing prep
      Practice: Secure 5 applications
      Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it
      Tools:
        - Helmet.js
        - Express-validator
        - OWASP ZAP

  ---------------------------------------------------------------
  PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)
  ---------------------------------------------------------------
  The capstone month: spec, build, ship and defend, then the final exam that decides the
  certificate.

    Month 10: Capstone and the Final Exam (Weeks 40-44)
      Capstone Spec and Setup
        Topics:
          - Choosing the capstone: a complete MERN product: social app, marketplace-lite or productivity
            tool
          - A one-page spec: scope, milestones, stack
          - Repository and skeleton ready on day one
          - Instructor sign-off before the build begins
        Projects built:
          - Approved capstone spec plus the running skeleton, deployed where applicable
        Practice: Pitch the spec to the batch and absorb one hard question
      Capstone Build
        Topics:
          - Two focused weeks in vertical slices
          - Instructor checkpoints and honest scope cuts
          - Using the whole course: React patterns, Express API design, MongoDB modeling and safe auth
            end to end
          - Daily commits and an always-working build
        Projects built:
          - The capstone, feature-complete with version control
        Practice: A working build at the end of every session
      Ship and Polish
        Topics:
          - The finishing pass: edge cases, errors, documentation
          - One-product pass: both halves deployed, wired and behaving as one
          - A README worth reading and a demo worth watching
          - A full-course spiral review before the exam
        Projects built:
          - The capstone shipped, documented and rehearsed
        Practice: Two mock demos, each tighter than the last
      Demo Day and the Final Exam
        Topics:
          - Presenting the capstone: live demo plus the hardest bug story
          - The honest map of next roads: full-stack roles, TypeScript across the stack, or backend depth
          - Where our other courses continue the journey
        Projects built:
          - Demo day presentation delivered to the batch
        Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed
        from every phase. Passing earns the certificate; falling short earns a focused revision
        plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Phase 1: JavaScript builds and deployed foundations
  - Phase 2: production-patterned React applications
  - Phase 3: Express APIs with MongoDB and safe authentication
  - Phase 4: the full MERN capstone, deployed and defended

  Total projects: 20+ real builds, crowned by a deployed full-stack MERN capstone

SKILLS THE STUDENT WILL HAVE AT THE END
```

- JavaScript depth on both ends of the stack
- React with production patterns: state, data and honest loading states
- Node and Express: APIs designed and documented professionally
- MongoDB: modeling with judgment, not just Mongoose incantations
- Full-stack integration: auth, contracts and deployment as one product

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Full-stack roles: MERN remains a hiring staple, especially in product companies
- TypeScript across the stack: our TypeScript course is the natural upgrade
- Backend depth: from Express patterns into production architectures
- DSA for the interview rounds beside the portfolio

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 3-5 LPA (Junior Frontend)
Growing roles: Rs 5-10 LPA (React Developer); Rs 8-15 LPA (Full Stack)
Experienced roles: Rs 12-25+ LPA (Senior MERN)
Freelance: Rs 1,000-5,000/hour
International: $60k-150k+ USD

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this MERN stack course cover?
A: The full JavaScript product craft in order: web and JavaScript foundations, React with production patterns, then Node and Express APIs designed professionally, MongoDB modeled with judgment, authentication done safely, and full-stack integration where the halves become one deployed product. A finale month ships a complete MERN capstone and sits the final exam.

Q: How long does it take to learn the MERN stack properly?
A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two more months in hand for learners who need them. The three teaching phases build the skill and its real-world application; the finale month ships a capstone and sits the final exam.

Q: How am I assessed? Is the certificate just for attending?
A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Is MERN still relevant, or should I learn Next.js instead?
A: Both, in order: MERN teaches the separated architecture that makes you understand what frameworks like Next.js later collapse for convenience. Job listings still name MERN constantly, and a developer who owns this stack picks up Next.js in a week, the reverse is rarely true. Our React and full-stack courses cover the Next.js road explicitly.

Q: Can I try a class before enrolling?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/mern-stack-development-masterclass-college
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 98 OF 137: MICROSOFT OFFICE FOR KIDS: WORD, POWERPOINT AND EXCEL
==========

Course link: https://learn.modernagecoders.com/courses/microsoft-office-kids-mastery
Subject area: Digital Skills for Kids
Designed for: Kids
Who can join: Complete beginner, ages 7 to 12
Course length: 5-6 months (20-24 weeks)
Time needed: 2 live classes/week + 1-2 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Microsoft Office course for kids aged 7-12: live online classes for Word, PowerPoint, Excel, OneNote and Teams. School-ready digital skills in 5-6 months.

In one line: The digital skills school quietly assumes: real Word documents, presentations that earn applause, first spreadsheets, and tidy digital habits, learned properly at 7 to 12.

ABOUT THE PROGRAMME

Schools assume children can format a document, build a presentation and manage their files, but nobody actually teaches it. This live online course does, properly, for children aged 7 to 12: Windows fundamentals and tidy file habits, digital art with the built-in creative tools, real Microsoft Word skills, PowerPoint presentations with restraint and style, first Excel spreadsheets with formulas and charts, and OneNote and Teams, the tools their school life increasingly runs on. Everything is project-based: children produce school reports, posters, family newsletters, quiz presentations, pocket-money trackers and a final integrated project they present at demo day. The pace is honest and kid-sized: five months of teaching at two live classes a week, with a sixth month in hand for children who need more practice. Classwork in every session, a small homework task after every class, a mixed challenge at each month's end, and a final exam, a project demo plus a friendly practical test, that decides the certificate. Real skills, really tested, immediately useful at school.

WHAT MAKES THIS COURSE DIFFERENT

- Project-based from week one: real school reports, presentations and trackers, not button tours
- Current tools, honestly taught: Windows 11, the free Office web apps, Photos and Clipchamp, no outdated software
- Typing and file hygiene built in: the quiet skills that make every later computer subject easier
- Kid-sized honest pace: 5 months at 2 classes a week, with a 6th month in hand when needed
- Real assessment, gently done: monthly mixed challenges and a final practical exam that decides the certificate, with a free retest after revision
- Small live batches capped at 10, grouped by age, with mini-batch and 1-on-1 options

THE LEARNING PATH, STEP BY STEP

Month 1: Windows and creativity (Weeks 1-4): computer confidence, tidy files, and digital art with Paint, Photos and Clipchamp
Month 2: Word (Weeks 5-8): documents that look professional, from formatting to images and tables
Month 3: PowerPoint (Weeks 9-12): presentations with real design sense, animation used with restraint
Month 4: Excel (Weeks 13-16): first spreadsheets, formulas, charts and honest data thinking
Month 5: OneNote, Teams and the finale (Weeks 17-20): school organization, collaboration, one integrated project, and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Computer experience: None. The course starts at the desktop and the mouse
Reading: Basic reading ability
Equipment: A Windows computer or laptop with internet. The free Office web apps are enough; desktop Office is used where the family already has it
Age: 7 to 12. Batches are grouped by age
Time commitment: 2 live classes plus 1-2 hours of practice a week

WHO THIS COURSE IS FOR

School students: Kids aged 8-14 who need computer skills for school projects
Digital beginners: Children with little or no computer experience
Presentation skills: Students who want to create impressive school presentations
Future professionals: Kids building early professional digital literacy skills
Homeschoolers: Families wanting structured digital skills education

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

MONTH 1: WINDOWS BASICS & DIGITAL ART

Windows fundamentals, tidy file habits, and creative confidence with Paint, Photos and Clipchamp.

Introduction to Windows & Computer Basics

Topics:
- Computer components: monitor, keyboard, mouse, CPU
- Turning computer on/off properly
- Windows 11 desktop overview
- Start menu navigation
- Taskbar features and customization
- Desktop icons and shortcuts
- Mouse skills: click, double-click, right-click, drag
- Keyboard basics: letters, numbers, special keys
- Caps Lock, Shift, Enter, Backspace, Delete
- Windows key shortcuts introduction
- Volume and brightness controls
- Connecting to WiFi networks

Projects built:
- Customize desktop wallpaper and theme
- Create desktop shortcuts
- Organize taskbar with favorite apps
- Personal desktop setup

Practice: Practice starting up and shutting down the computer the right way, then pin your three favorite apps to the taskbar and show a parent.

File Management & Organization

Topics:
- Understanding files and folders
- File Explorer navigation
- Creating new folders
- Naming conventions for files
- Moving and copying files
- Cut, copy, paste operations
- Deleting and Recycle Bin
- Searching for files
- File types and extensions
- Organizing Downloads folder
- Creating folder structures
- File properties and details
- Compressing files with ZIP
- OneDrive basics and cloud storage

Projects built:
- Create organized folder system for schoolwork
- Digital filing cabinet setup
- Photo organization project
- Backup important files

Practice: Create a folder tree for your school subjects and sort five files into the right folders using cut, copy and paste.

Microsoft Paint Fundamentals

Topics:
- Opening and navigating Paint
- Drawing tools: pencil, brushes, shapes
- Color palette and custom colors
- Fill tool and color picker
- Line and curve tools
- Shape tools: rectangle, circle, polygon

- Text tool and fonts
- Selection tools and moving objects
- Resize and rotate features
- Zoom in/out for detail work
- Saving in different formats (PNG, JPEG, BMP)
- Image size and canvas size
- Copying and pasting images
- Using Paint for school projects

Projects built:
- Create pixel art characters
- Design birthday card
- Draw house and landscape
- Make school poster
- Create emoji collection

Practice: Draw your dream treehouse in Paint using at least four different tools and the fill bucket.

Photos, Clipchamp and Creative Projects

Topics:
- Editing pictures with the Windows Photos app
- Cropping, lighting and honest photo fixes
- Clipchamp: first video edits with clips and titles
- Adding music and simple transitions
- Making a family slideshow video
- Screenshots and the Snipping Tool
- Saving projects where you can find them again
- A creative mini-showcase for the family

Projects built:
- Edited photo set: before and after
- A 60-second family slideshow video in Clipchamp

Practice: Make one tiny video about your week

Assessment: Month 1 check: a files-and-creativity challenge mixing everything so far

---

MONTH 2: MICROSOFT WORD MASTERY

Become a Word expert, creating professional documents, reports, and creative writing projects.

Word Basics & Text Formatting

Topics:
- Opening Microsoft Word and interface tour
- Creating new documents
- Ribbon interface explanation
- Quick Access Toolbar customization
- Typing and text selection techniques
- Font types and sizes
- Bold, italic, underline formatting
- Text color and highlighting
- Alignment: left, center, right, justify
- Line spacing and paragraph spacing
- Bullets and numbering
- Indentation and tabs
- Copy, cut, paste operations
- Undo and redo features
- Saving documents and file formats

Projects built:
- Write and format a story
- Create class notes template
- Design invitation letter
- Format poem with styling

Practice: Type a short paragraph about your favorite animal in Word and format it with two fonts, one color and bold key words.

Advanced Formatting & Page Layout

Topics:
- Page setup: margins, orientation, size
- Headers and footers
- Page numbers and formatting
- Section breaks and page breaks
- Columns for newsletters
- Borders and shading
- Watermarks and page color
- Themes and style sets
- Creating and using styles
- Table of contents generation
- Cover pages and templates
- Drop caps and special effects
- Text boxes and positioning

- WordArt for titles
- Page borders and art

Projects built:
- Create school newsletter
- Design book report template
- Make professional letter
- Build recipe book layout

Practice: Make a one page mini newsletter about your week using two columns, a header and a page border.

Working with Images & Graphics

Topics:
- Inserting pictures from computer
- Online pictures and stock images
- Picture formatting and styles
- Cropping and resizing images
- Picture effects and corrections
- Text wrapping options
- SmartArt graphics creation
- Charts and graphs in Word
- Shapes and drawing tools
- Icons and 3D models
- Screenshots and screen clipping
- Arranging multiple objects
- Grouping and ungrouping
- Captions for images
- Compress pictures for file size

Projects built:
- Illustrated story book
- Science project report with diagrams
- Travel brochure with images
- How-to guide with screenshots

Practice: Insert three pictures into a 'My Hobbies' document, then crop one, style another and wrap text around the third.

Tables, References & Collaboration

Topics:
- Creating tables from scratch
- Table styles and formatting
- Adding/deleting rows and columns
- Merging and splitting cells
- Table calculations and formulas
- Converting text to tables
- Footnotes and endnotes
- Citations and bibliography
- Hyperlinks and bookmarks
- Track changes feature
- Comments and reviewing
- Comparing documents
- Sharing and co-authoring
- Protecting documents
- Printing options and preview

Projects built:
- Class schedule table
- Research paper with citations
- Collaborative story writing
- Data comparison report

Practice: Build your weekly class timetable as a Word table and give the header row its own color.

Assessment: Month 2 check: format a document live from a messy draft, plus a mixed quiz reaching back to month 1

MONTH 3: POWERPOINT PRESENTATION EXCELLENCE

Master the art of creating engaging, professional presentations with PowerPoint.

PowerPoint Fundamentals

Topics:
- PowerPoint interface overview
- Slide layouts and placeholders
- Adding new slides
- Slide themes and variants
- Text formatting in slides
- Bullet points and numbering
- Slide backgrounds and fills
- Color schemes and fonts
- Slide master basics
- Notes and comments

- Outline view for planning
- Slide sorter view
- Saving presentations
- PowerPoint file formats
- Presenter view introduction

Projects built:
- About Me presentation
- Book report slideshow
- Science topic presentation
- Family photo album

Practice: Create a four slide PowerPoint about your favorite place using one theme and tidy bullet points.

Animations & Transitions

Topics:
- Slide transitions types
- Transition timing and duration
- Transition sounds
- Animation basics
- Entrance animations
- Emphasis animations
- Exit animations
- Motion path animations
- Animation pane control
- Animation timing and delay
- Animation triggers
- Combining multiple animations
- Morph transition feature
- 3D models animation
- Creating smooth flows

Projects built:
- Animated story presentation
- Kinetic typography project
- Science process animation
- Interactive quiz show

Practice: Add a different transition to each slide of your deck and give the title text an entrance animation.

Multimedia & Interactive Elements

Topics:
- Inserting images and graphics
- Photo albums in PowerPoint
- Video insertion and editing
- Video playback options
- Audio and music addition
- Recording narration
- Screen recording feature
- Creating hyperlinks
- Action buttons
- Interactive navigation
- Zoom feature for emphasis
- Embedding web content
- Charts and graphs
- SmartArt in presentations
- QR codes in slides

Projects built:
- Video presentation project
- Interactive story with choices
- Virtual tour presentation
- Music video slideshow

Practice: Add a photo album slide, one short video and a little background music to your presentation.

Advanced Design & Presenting

Topics:
- Slide Master customization
- Creating custom layouts
- Design Ideas feature
- Icons and stock images
- 3D models and rotation
- Cameo (PowerPoint Live)
- Presenter Coach feature
- Rehearse with Coach
- Live captions and subtitles
- Translation features
- Broadcasting presentations
- Export options (video, PDF, etc.)
- PowerPoint Designer AI
- Accessibility checker

- Mobile presenting
Projects built:
- Professional pitch deck
- Educational tutorial
- Digital portfolio
- Conference presentation
Practice: Design a custom layout in Slide Master, then rehearse presenting your deck out loud for two minutes.
Assessment: Month 3 check: build and present three slides live, with mixed questions from earlier months

---
MONTH 4: EXCEL AND DATA SKILLS

Learn spreadsheet basics, data management, charts, and basic formulas with Excel.

Excel Basics & Data Entry
Topics:
- Excel interface and terminology
- Workbooks and worksheets
- Cells, rows, and columns
- Selecting cells and ranges
- Entering text and numbers
- AutoFill feature
- Number formats (currency, percentage, date)
- Cell formatting and styles
- Column width and row height
- Inserting and deleting rows/columns
- Copy, cut, paste special
- Find and replace
- Freeze panes
- Splitting windows
- Worksheet tabs management
Projects built:
- Class grade tracker
- Personal budget sheet
- Chore chart with dates
- Sports statistics table
Practice: Enter a pretend week of family fruit shopping into Excel and use AutoFill to complete the days of the week.

Formulas & Functions
Topics:
- Understanding formulas
- Basic arithmetic operations (+, -, *, /)
- Cell references (relative vs absolute)
- SUM function
- AVERAGE function
- COUNT and COUNTA
- MIN and MAX functions
- IF statements basics
- AutoSum feature
- Function wizard
- Formula bar usage
- Order of operations
- Copying formulas
- Common formula errors
- Simple problem solving
Projects built:
- Grade calculator with average
- Shopping budget calculator
- Simple game scorecard
- Time tracking sheet
Practice: Track your pocket money for an imaginary week, then total it with SUM and find the daily average with AVERAGE.

Charts & Data Visualization
Topics:
- Types of charts (column, line, pie)
- Creating charts from data
- Chart elements and parts
- Chart styles and colors
- Chart titles and labels
- Legend positioning
- Data labels addition
- Changing chart types
- Sparklines for mini charts
- Combination charts
- 3D chart effects

- Chart templates
- Printing charts
- Charts in other Office apps
- Infographic creation

Projects built:
- Weather data visualization
- Survey results charts
- Growth tracking chart
- Game statistics dashboard

Practice: Turn your favorite game or sports scores into a column chart and a pie chart, each with a proper title and labels.

Data Management & Analysis

Topics:
- Sorting data A-Z, Z-A
- Custom sort orders
- Filter data basics
- AutoFilter features
- Conditional formatting
- Data bars and color scales
- Icon sets for visual cues
- Tables in Excel
- Table styles and formatting
- PivotTable basics (introduction)
- Data validation rules
- Drop-down lists creation
- Protecting cells and sheets
- Sharing and collaboration
- Import/export data

Projects built:
- Student database
- Inventory tracking system
- Event planning spreadsheet
- Analysis report with visuals

Practice: Sort a list of ten pretend test scores from highest to lowest and use conditional formatting to highlight the top three.

Assessment: Month 4 check: a mini spreadsheet challenge with formulas and one chart, plus mixed revision

---

MONTH 5: ONENOTE, TEAMS AND THE GRAND FINALE

The organizational tools school life runs on, one integrated project that uses everything, and the final exam.

OneNote Digital Notebook

Topics:
- OneNote interface overview
- Notebooks, sections, and pages
- Creating notebook structure
- Typing and formatting notes
- Handwriting and drawing
- Inserting images and files
- Audio and video recording
- Screen clipping tool
- Tags for organization
- Search functionality
- To-do lists and checkboxes
- Tables in OneNote
- Math equations
- Syncing across devices
- Sharing notebooks

Projects built:
- Digital school binder
- Research project notebook
- Personal journal/diary
- Study guide creation

Practice: Set up a OneNote notebook with one section per school subject and fill one page with notes plus a small drawing.

Microsoft Teams for Students

Topics:
- Teams interface navigation
- Joining and creating teams
- Channels and conversations
- Chat features and etiquette
- Video calls and meetings
- Screen sharing basics
- File sharing in Teams

- Assignments submission
- Calendar integration
- Together mode and backgrounds
- Breakout rooms
- Meeting recordings
- Live events basics
- Teams mobile app
- Notifications management

Projects built:
- Class collaboration project
- Virtual study group setup
- Team presentation delivery
- Digital club meeting

Practice: With a parent, practice joining a video call, sending one polite chat message and sharing your screen for a minute.

The Integrated Project

Topics:
- Mail merge Word with Excel
- Embedding Excel charts in Word
- PowerPoint with Excel data
- OneNote integration with Office
- Teams assignments workflow
- Creating templates library
- Macro basics (introduction)
- Quick Parts and building blocks
- Custom ribbons basics
- Office themes consistency
- Cloud synchronization
- Version history usage
- Co-authoring features
- Office mobile apps
- Office on the web

Projects built:
- Newsletter with mail merge
- Report with live Excel data
- Integrated presentation package
- Complete project workflow

Practice: Embed an Excel chart of your scores or savings into a Word report and write two sentences explaining what it shows.

Final Exam and Demo Day

Topics:
- Project planning and scope
- Requirements gathering
- Timeline creation
- Resource planning
- Design documentation
- Development phases
- Testing and revision
- User feedback incorporation
- Final polishing
- Presentation preparation
- Documentation writing
- Reflection and learning

Practice: Write a mini plan for your final project: what you will make, which Office apps you will use and your three work sessions.

Deliverables:
- Complete project files
- Documentation package
- Presentation deck
- Video demonstration
- Reflection report

Assessment: FINAL EXAM: a practical test across Word, PowerPoint and Excel plus the integrated project demo. Passing earns the certificate; a child who falls short gets focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Month 1: a tidy personal folder system, edited photos and a first Clipchamp video
- Month 2: a school report, a poster and a family newsletter in Word
- Month 3: a hobby presentation and an animated quiz in PowerPoint
- Month 4: a pocket-money tracker and a class survey with charts in Excel
- Month 5: a OneNote study notebook and the integrated final project

Total projects: 20+ real documents, presentations, spreadsheets and videos

SKILLS THE STUDENT WILL HAVE AT THE END

- Windows confidence: files, folders, shortcuts and calm problem-solving
- Word: formatting, images, tables and documents that look professional
- PowerPoint: clear slides, tasteful animation and confident presenting
- Excel: data entry, formulas, functions and honest charts
- OneNote and Teams: the organization tools school increasingly runs on
- Creative tools: photo editing and first video edits with free built-in apps
- Typing and digital habits that make every later computer subject easier

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building along with the teacher
Homework: A small homework task after every class, and a mixed challenge at the end of every month
Practice: 1-2 hours of practice between classes, structured as real little projects
Review: Every submitted piece gets teacher feedback; wobbly skills are reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for stuck homework
Progress updates: Regular progress notes to parents: what was learned, what comes next, and how to encourage practice

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam, a practical test plus a project demo. A child who falls short gets focused revision and a free retest: the certificate certifies skills, not attendance

WHERE THIS CAN LEAD

- Digital Content Creator
- Student Technology Leader
- Office Productivity Expert
- Data Analysis Beginner
- Professional Document Designer
- Ready for advanced coding or design courses

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my child learn in this course?
A: Real, school-ready digital skills: Windows confidence and tidy file habits, photo editing and first videos with the free Photos and Clipchamp apps, professional documents in Word, presentations with actual design sense in PowerPoint, first spreadsheets with formulas and charts in Excel, and OneNote and Teams for school organization.

Q: Which age group is this designed for?
A: Ages 7 to 12, with batches grouped by age so the pace and examples fit the room. No prior computer experience is needed; the course starts at the desktop and the mouse and builds up patiently.

Q: How long does it take a child to learn Microsoft Office?
A: To real, school-ready capability: 5 months at 2 live classes a week plus an hour or two of practice. The course keeps a 6th month in hand for children who need more repetition, with no rushing and no penalty. Skills show up at school almost immediately, the first properly formatted report usually appears within month 2.

Q: How is my child assessed? Is the certificate just for attending?
A: No. There is classwork in every session, a small homework task after every class, and a mixed challenge at the end of each month that deliberately revisits earlier skills so they stop fading. The course ends with a final practical exam plus a project demo. Passing earns the certificate; a child who falls short gets focused revision and a free retest. Gentle in tone, real in standard.

Q: Does my child need computer experience before starting?
A: No. Everything builds step by step from there.

Q: What software does my child need at home?
A: A Windows computer with internet is enough. The free Office web versions of Word, PowerPoint, Excel and OneNote cover the whole course, and we use them alongside free built-in tools like Photos, Clipchamp and the Snipping Tool. If your family already has desktop Office or Microsoft 365, your child can use that too; nothing needs to be purchased.

Q: How does this help with school, and what comes next?
A: The skills map directly to schoolwork: essays and reports in Word, class presentations in PowerPoint, and data projects in Excel. Afterwards, many students move into our Scratch or kids coding courses, since comfortable computer use makes learning to code much smoother.

  Q: Can my child attend a trial class first?
  A: Yes, the first step for every family is a free demo class, booked at
  learn.modernagecoders.com/contact. There is no commitment attached, and parents can sit in to
  see exactly how the teacher works with kids. (Agent note: the free demo is a waiting list
  with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at
  https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: What do the classes look like?
  A: Each class is a live 1-hour online session with a real teacher guiding a small group,
  never a pre-recorded video. Children practice hands-on in the actual software during class
  and get help the moment they are stuck.

  Q: Will my child receive a certificate?
  A: Yes, and it is earned: the certificate is awarded on passing the final exam, a practical
  test across Word, PowerPoint and Excel plus the integrated project demo, with a free retest
  after revision if the first attempt falls short. Alongside it stands a portfolio of 20+ real
  documents, presentations and spreadsheets.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/microsoft-office-kids-mastery
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 99 OF 137: MIDDLE SCHOOL MATH COMPETITIONS: MATHCOUNTS, AMC 8 AND MATH KANGAROO PREP
==========

Course link: https://learn.modernagecoders.com/courses/middle-school-math-competitions-course
Subject area: Competition Mathematics & Olympiad Training
Designed for: Middle school
Who can join: Students roughly aged 10 to 14 (grades 5 to 8) who are comfortable with school maths and want more challenge
Course length: 6 months (24 weeks), repeatable as a second season with new problem sets
Time needed: 2 live classes/week + weekly problem sets and timed rounds
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one
  live class a week, four a month; one to one outside India two live classes a week, eight a
  month. Each class is one hour.

WHAT THIS COURSE IS

  Live online math competition prep for ages 10 to 14: number theory, counting, algebra and
  geometry problem solving for MATHCOUNTS, AMC 8 and Math Kangaroo, with timed rounds.

  In one line: The problem-solving toolkit behind MATHCOUNTS, the AMC 8 and Math Kangaroo, built idea by idea and tested against the clock.

ABOUT THE PROGRAMME

  Middle school math competitions ask for something school maths rarely does: seeing an
  unfamiliar problem, choosing an idea, and getting it right quickly. The three best-known
  contests for this age reward the same core toolkit. MATHCOUNTS is a national program for
  students in grades 6 to 8, with four levels of competition (school, chapter, state and
  national) and four rounds: the Sprint Round of 30 problems in 40 minutes without a
  calculator, the Target Round of four pairs of problems with 6 minutes per pair, the Team
  Round of 10 problems in 20 minutes, and the optional fast-paced Countdown Round. Math
  Kangaroo USA is a 75-minute multiple-choice test with 30 questions for students in grade 5
  and above, scored 3, 4 or 5 points per question with no points taken away for wrong answers.
  The AMC 8, from the Mathematical Association of America, is a multiple-choice contest for
  middle school students. This 6-month live course teaches that shared toolkit in a deliberate
  order: arithmetic and number sense, number theory, algebra and word problems, counting and
  probability, geometry, then patterns, sequences and logic. Every topic is taught through
  problems written in the styles of these contests, and every month includes a timed round so
  speed and accuracy grow together. The final two months are contest craft: Sprint and
  Countdown speed without a calculator, Target and Team strategy with one, and multiple-choice
  technique for the AMC 8 and Math Kangaroo. Students keep an error log from the first week, so
  they learn as much from mistakes as from successes. We teach the mathematics and the habits;
  we never promise medals or qualification, and families register for each contest through the
  routes the organisers set.

WHAT MAKES THIS COURSE DIFFERENT

  - One toolkit for three contests: MATHCOUNTS, the AMC 8 and Math Kangaroo reward the same core
    ideas, so students prepare for all three at once
  - Topics taught in a deliberate order, from number sense to geometry, so each idea builds on

the one before
- A timed round every month, so speed and accuracy are trained together rather than left to the final weeks
- Round-specific craft: no-calculator speed for Sprint and Countdown, calculator strategy for Target and Team, and elimination for multiple-choice tests
- An error log kept from week one, reviewed with the teacher every month
- Problems written in the contest styles, plus the official free sample problems the organisers publish

THE LEARNING PATH, STEP BY STEP

Phase 1: The problem-solving base: number sense, number theory, ratios and rates, and the first counting principles
Phase 2: Core contest topics: probability, geometry from angles to solids, patterns and sequences, remainders and logic
Phase 3: Contest craft: MATHCOUNTS round strategy, multiple-choice technique for the AMC 8 and Math Kangaroo, and full mock contests

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Comfortable with school maths at grade level, including fractions, decimals and basic equations. No contest experience needed
Age: Roughly 10 to 14 (grades 5 to 8). Younger students who are strong at maths are welcome after the free demo
Equipment: A device with stable internet for live classes, paper and pencil, and a basic calculator for Target and Team practice
Timing: Joinable any month. Starting several months before a contest leaves time for both the toolkit and the timed mocks

WHO THIS COURSE IS FOR

First time competitors: Students who are good at school maths and want to try their first competition
Returning competitors: Students who have taken part before and want a structured toolkit and stronger timing
School team hopefuls: Students aiming to join or contribute to a school MATHCOUNTS team
Multiple choice contest students: Students preparing for Math Kangaroo or the AMC 8 who want more than past-paper drilling
Curious problem solvers: Children who simply enjoy puzzles and want maths that is harder and more interesting than school

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE PROBLEM-SOLVING BASE (MONTHS 1-2, WEEKS 1-8)

Fast, reliable number work and the first big competition ideas: divisibility, primes, ratios and counting.

Month 1: Number Sense and Number Theory (Weeks 1-4)
Weeks 1-2: Diagnostic, Contest Formats and Fast Arithmetic
Topics:
- A diagnostic round to find each student's starting point
- How MATHCOUNTS, the AMC 8 and Math Kangaroo are structured
- Mental arithmetic shortcuts: compensation, doubling and halving, and multiplying near 100
- Fractions, decimals and percents converted quickly
- Order of operations and careful reading
- Starting an error log

Projects built:
- Personal diagnostic summary with three target topics and an opened error log

Practice: Daily ten-minute arithmetic sprint, with the time and score recorded each day
Weeks 3-4: Primes, Factors and Divisibility
Topics:
- Divisibility rules and why they work
- Prime factorisation and factor trees
- Greatest common divisor and least common multiple
- Counting the factors of a number from its prime factorisation
- Perfect squares and cubes
- Contest problems that hide a factorisation

Projects built:
- Number theory toolkit page: each rule with a worked contest-style example

Practice: A 20-problem number theory set, with the idea used written beside each answer
Assessment: Month 1 timed round: a short no-calculator round on arithmetic and number theory

Month 2: Ratios, Word Problems and Counting (Weeks 5-8)
Weeks 5-6: Ratios, Rates and Algebra Word Problems

Topics:
- Ratios and proportions set up from words
- Rates, work and distance problems
- Percent change and successive percentages
- Writing and solving linear equations from a story
- Averages and how adding a value changes them
- Checking an answer against the question

Projects built:
- Word problem casebook: ten problem types, each with a model solution

Practice: A 20-problem word problem set, half solved with an equation and half by reasoning

Weeks 7-8: Counting Basics

Topics:
- Making organised lists so nothing is missed or counted twice
- The multiplication principle
- Arrangements: permutations of objects in a row
- Choosing groups: combinations
- Complementary counting: count what you do not want
- Venn diagrams for overlapping groups

Projects built:
- Counting toolkit page: when to list, multiply, arrange, choose or subtract

Practice: A 15-problem counting set, each answer checked by a second method where possible

Assessment: Phase 1 assessment: a timed mixed round covering number theory, word problems and counting

---

PHASE 2: CORE CONTEST TOPICS (MONTHS 3-4, WEEKS 9-16)

Probability, geometry and the patterns and logic questions that appear across all three contests.

Month 3: Probability and Plane Geometry (Weeks 9-12)

Weeks 9-10: Probability

Topics:
- Probability as favourable outcomes over possible outcomes
- Dice, coins, cards and spinners
- Independent events and multiplying probabilities
- Using counting to find probabilities
- Complementary probability
- Simple expected value

Projects built:
- Probability lab: run a small experiment, compare it with the calculated answer and explain the difference

Practice: A 15-problem probability set with a sketch of the sample space for every problem

Weeks 11-12: Angles, Triangles and Area

Topics:
- Angle facts: straight lines, triangles, polygons and parallel lines
- The Pythagorean theorem and common Pythagorean triples
- Area of triangles, quadrilaterals and composite shapes
- Similar triangles and scale factors
- Adding a helpful line to a diagram
- Coordinate geometry: distance and area on a grid

Projects built:
- Geometry fact sheet with a diagram for every fact

Practice: A 15-problem geometry set, redrawing each diagram to scale before solving

Assessment: Month 3 timed round: probability and geometry, with a written review of every miss

Month 4: Circles, Solids, Patterns and Logic (Weeks 13-16)

Weeks 13-14: Circles and Solids

Topics:
- Circumference, area, arcs and sectors
- Shaded-region problems
- Surface area and volume of prisms, cylinders, pyramids and cones
- Nets and cube-counting puzzles
- Visualising folds, rotations and views of solids
- Exact answers in terms of pi

Projects built:
- Shaded-region and solids casebook with ten worked problems

Practice: A 15-problem circles and solids set, with exact answers left in terms of pi

Weeks 15-16: Patterns, Sequences, Remainders and Logic

Topics:
- Arithmetic sequences and the sum of the first n terms
- Spotting and proving simple patterns
- Units digits and remainders in long calculations
- Number bases beyond ten
- Logic puzzles: truth-tellers, ordering and elimination grids

- Visual reasoning puzzles of the kind Math Kangaroo is known for
Projects built:
- Patterns and logic casebook, with a one-line explanation of why each pattern holds
Practice: A mixed set of patterns, remainders and logic puzzles, half with a timer and half without
Assessment: Phase 2 assessment: a timed mixed round across all topics so far

---

PHASE 3: CONTEST CRAFT AND MOCK CONTESTS (MONTHS 5-6, WEEKS 17-24)

Turning knowledge into results on the day: round-by-round strategy for MATHCOUNTS, multiple-choice technique for the AMC 8 and Math Kangaroo, and full mock contests.

Month 5: MATHCOUNTS Round Strategy (Weeks 17-20)

Weeks 17-18: Sprint and Countdown Speed

Topics:
- Sprint Round pacing: 30 problems in 40 minutes without a calculator
- Which problems to bank first and which to return to
- Mental shortcuts that save seconds on common problem types
- Countdown Round practice: answering against a short per-problem clock
- Staying accurate when going fast
- Reviewing a sprint: speed errors versus knowledge errors

Projects built:
- Two timed Sprint-style rounds with a personal pacing plan written after each

Practice: One Sprint-style round a week under exact timing, with every miss logged

Weeks 19-20: Target and Team Strategy

Topics:
- Target Round pacing: pairs of problems in 6 minutes
- Using a calculator well, and knowing when not to
- Showing enough working to check an answer quickly
- Team Round: 10 problems in 20 minutes, splitting and checking work
- Communicating clearly in a team
- Practising with the organisers' free sample problems

Projects built:
- Full MATHCOUNTS-style mock: Sprint, Target and Team rounds, with a written review

Practice: One Target-style set and one team problem set each week

Assessment: Month 5 assessment: the full MATHCOUNTS-style mock, reviewed round by round

Month 6: Multiple-Choice Craft and Mock Contests (Weeks 21-24)

Weeks 21-22: Multiple-Choice Technique for the AMC 8 and Math Kangaroo

Topics:
- Using the answer choices: elimination, estimation and testing values
- Spotting the trap answer a common mistake produces
- Math Kangaroo scoring: 3, 4 and 5-point questions and no penalty for wrong answers
- Pacing a 75-minute, 30-question Math Kangaroo paper
- Planning time for a multiple-choice contest and checking marked answers
- Reading each organiser's current rules before the contest

Projects built:
- Timed multiple-choice mock papers in the AMC 8 and Math Kangaroo styles, with written reviews

Practice: One timed multiple-choice paper a week and a written plan for the next one

Weeks 23-24: Final Mocks and the Next Season

Topics:
- A final mixed mock contest under exact timing
- Error log review: the five fixes that matter most
- Contest-day routine: sleep, materials and nerves
- How to keep practising between contests
- Planning the next season or moving toward high school contests
- Presenting a favourite problem and its solution to the class

Projects built:
- Final mock contest and a personal next-season plan based on the error log

Practice: Write a full solution to your favourite problem of the course, explained for a younger student

Assessment: Final assessment: a mixed mock contest plus a short conversation on method choices

PROJECTS THE STUDENT BUILDS AND KEEPS

- Diagnostic summary and error log kept all course
- Number theory and counting toolkit pages
- Word problem, geometry, solids and logic casebooks
- Probability lab write-up
- Full MATHCOUNTS-style mock with a round-by-round review
- Timed multiple-choice mock papers with written reviews
- Personal next-season plan

Total projects: A complete contest kit: toolkit pages, casebooks, an error log and reviewed mock rounds

SKILLS THE STUDENT WILL HAVE AT THE END

- Fast, accurate arithmetic and estimation without a calculator
- Number theory: primes, factors, divisibility, remainders and bases
- Counting and probability: lists, arrangements, choices and complements
- Algebra and word problems: ratios, rates, percents and equations
- Geometry: angles, triangles, area, circles, solids and coordinates
- Patterns, sequences and logic puzzles
- Round-by-round contest strategy and multiple-choice technique

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, with problems solved together and individually
Practice: A problem set after every class, with core and stretch problems
Review: Problem sets and timed rounds reviewed with written comments, and the error log checked monthly

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with hints first and full solutions after a real attempt
Progress updates: A monthly progress note to parents with timed-round results and next steps

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- A second season of this course with new problem sets, aimed at the next contest year
- Our Olympiad Mathematics course, for proof-based problems and high school contests
- Algebra and high school maths courses, started with a strong problem-solving base
- Coding courses, where the same logical habits carry straight into programming
- School maths clubs and team competitions, joined with real preparation

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which contests does this course prepare for?
A: MATHCOUNTS, the AMC 8 and Math Kangaroo. All three reward the same core ideas: number theory, counting and probability, algebra, geometry and logic. The course teaches that toolkit, then adds round-specific strategy for MATHCOUNTS and multiple-choice technique for the AMC 8 and Math Kangaroo.

Q: What are the MATHCOUNTS rounds?
A: MATHCOUNTS has four rounds. The Sprint Round is 30 problems in 40 minutes without a calculator. The Target Round gives four pairs of problems, with 6 minutes per pair and a calculator assumed. The Team Round is 10 problems in 20 minutes, taken officially by a school's four-student team. The Countdown Round focuses on speed and accuracy, with a maximum of 45 seconds per problem and no calculator,, optional at the school, chapter and state levels.

Q: How does Math Kangaroo work?
A: Math Kangaroo is a 75-minute multiple-choice test. Students in grade 5 and above answer 30 questions; a third are worth 3 points, a third 4 points and a third 5 points, and no points are taken away for wrong answers. In the USA the individual competition is held in March, organised through Math Kangaroo centers.

Q: What about the AMC 8?
A: The AMC 8 is a multiple-choice contest for middle school students run by the Mathematical Association of America. The course covers the topics and multiple-choice technique it rewards. Contest dates, format details and registration routes are set by the MAA each year, so check its official page for the current rules.

Q: How do we register for these contests?
A: MATHCOUNTS works through registered schools, and students whose school does not take part can register as non-school competitors. Math Kangaroo is organised through its own centers. The AMC 8 is offered through schools and other hosts as the MAA sets out. We are happy to explain the options during the demo.

Q: Is my child ready for this course?
A: The course suits students roughly aged 10 to 14 who are comfortable with school maths and enjoy a challenge. They do not need contest experience. The first-week diagnostic shows where to start, and problem sets have core and stretch problems so both newer and more experienced students are stretched.

Q: How is this different from your Olympiad Mathematics course?

A: Our Olympiad Mathematics course is for older, stronger students working toward proof-based olympiad problems and high school contests. This course is the step before: the problem-solving toolkit and contest habits for middle school competitions. Students who finish it and enjoy competition maths often move on to the olympiad course.

Q: Will this guarantee a medal or a place at nationals?
A: No, and we would not promise it. Results depend on the student, the contest and the day. What the course gives is a solid toolkit, regular timed practice and an honest error log, so you can see real progress month by month.

Q: How are the classes run?
A: Live online classes, twice a week, with a real teacher, in a group of 15 to 20 students, a mini batch of up to 6, or one to one. Current fees for your region are shown in the pricing section of this page.

Q: Can we try a class first?
A: Yes. Book a free demo class. The teacher works through a few contest-style problems with your child, sees how they think, and suggests whether group, mini batch or one-to-one classes fit best. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/middle-school-math-competitions-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 100 OF 137: MIDDLE SCHOOL MATHEMATICS MASTERY: PRE-ALGEBRA & GEOMETRY

==========

Course link: https://learn.modernagecoders.com/courses/comprehensive-middle-school-mathematics-mastery
Subject area: Middle School Mathematics Education
Designed for: Girls only batches, Middle school
Who can join: Complete Beginner to Advanced Middle School Level
Course length: 12-14 months (52-60 weeks)
Time needed: 2 live classes/week + 3-5 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

A 1-year middle school maths program taking students from basic arithmetic to pre-algebra, geometry and statistics with real problem-solving strategies.

In one line: From Number Basics to Advanced Mathematical Thinking

ABOUT THE PROGRAMME

This comprehensive program transforms students into confident mathematical thinkers. Whether starting with basic arithmetic or advancing to complex problem-solving, this 1-year program builds a rock-solid foundation in mathematics. Students will master essential mathematical concepts through interactive lessons, real-world applications, and engaging projects. From fractions to functions, geometry to statistics, every topic is covered with depth and clarity. By completion, students will excel in standardized tests and be fully prepared for high school mathematics.

WHAT MAKES THIS COURSE DIFFERENT

- Starts from absolute basics - no prerequisites required
- Full year of structured, progressive learning
- Covers entire middle school math curriculum
- Real-world applications and STEM connections
- Test preparation and exam strategies included
- Interactive visualizations and manipulatives
- Personalized learning paths based on progress
- Parent progress reports and involvement tools

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation (Months 1-3): Number Systems, Operations, and Basic Problem Solving
Phase 2: Arithmetic Mastery (Months 4-6): Fractions, Decimals, Percentages, and Ratios

Phase 3: Algebraic Thinking (Months 7-9): Pre-Algebra, Equations, Functions, and Patterns
Phase 4: Geometry & Advanced Topics (Months 10-12): Geometry, Statistics, Probability, and Complex Problem Solving

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Elementary school mathematics completion
Coding experience: Basic arithmetic knowledge helpful
Equipment: Computer/tablet with internet, calculator
Time commitment: 2 live classes plus 3 to 5 hours of practice a week
English: Grade-appropriate reading level
Motivation: Enthusiasm for learning mathematics

WHO THIS COURSE IS FOR

Students: Middle school students grades 6-8
Working professionals: Parents helping children with math
Entrepreneurs: Homeschool families
Freelancers: Tutors and educators
Kids: Advanced elementary students ready for challenge
Anyone: Anyone wanting strong math foundation

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: NUMBER SYSTEMS AND FUNDAMENTAL OPERATIONS (WEEKS 1-13)

Build unshakeable foundations in number concepts, operations, and mathematical thinking.

Months 1-2: Number Systems and Basic Operations (Weeks 1-8)
Introduction to Number Systems
Topics:
- Understanding place value system
- Reading and writing large numbers (up to billions)
- Expanded form and word form
- Comparing and ordering whole numbers
- Number line representation
- Rounding to nearest 10, 100, 1000
- Roman numerals basics
- Introduction to negative numbers
- Integer concepts and number line
- Absolute value understanding
- Real-world contexts for negative numbers
- Temperature, elevation, and financial contexts

Projects built:
- Create a number system timeline
- Design a place value game
- Real-world number investigation project

Practice: Daily: 20 problems on place value, 15 minutes mental math

Addition and Subtraction Mastery
Topics:
- Multi-digit addition with regrouping
- Addition properties: commutative, associative, identity
- Mental math strategies for addition
- Multi-digit subtraction with borrowing
- Subtraction as inverse of addition
- Estimation strategies for sums and differences
- Adding and subtracting integers
- Using number lines for integer operations
- Word problems with addition and subtraction
- Multi-step problem solving
- Checking answers using inverse operations
- Real-world applications: budgets, distances

Projects built:
- Budget planning project
- Distance calculation map activity
- Integer game design

Practice: Solve 30 addition/subtraction problems daily with increasing complexity

Multiplication Fundamentals
Topics:
- Multiplication as repeated addition
- Times tables mastery (up to 12×12)
- Multiplication properties: commutative, associative, distributive
- Multi-digit multiplication algorithms
- Lattice multiplication method
- Area model for multiplication
- Multiplying by powers of 10

- Estimation in multiplication
- Mental math tricks for multiplication
- Multiplying integers and sign rules
- Word problems involving multiplication
- Real-world applications: area, arrays, scaling

Projects built:
- Multiplication strategy booklet
- Area calculation for room design
- Array art project

Practice: Master times tables, 25 multi-digit multiplication problems

Division and Factors

Topics:
- Division as inverse of multiplication
- Long division algorithm step-by-step
- Division with remainders
- Interpreting remainders in context
- Divisibility rules (2, 3, 4, 5, 6, 9, 10)
- Factors and multiples
- Prime and composite numbers
- Prime factorization
- Greatest Common Factor (GCF)
- Least Common Multiple (LCM)
- Division of integers and sign rules
- Word problems with division

Projects built:
- Factor tree artwork
- Divisibility rule investigation
- Prime number sieve project

Practice: 30 division problems, find factors and multiples daily

Month 3: Order of Operations and Problem Solving (Weeks 9-13)

Order of Operations (PEMDAS/BODMAS)

Topics:
- Introduction to order of operations
- Parentheses/Brackets in expressions
- Exponents and powers basics
- Multiplication and Division (left to right)
- Addition and Subtraction (left to right)
- Complex expressions with multiple operations
- Nested parentheses
- Common mistakes and misconceptions
- Using calculators correctly
- Writing expressions from word problems
- Evaluating expressions with variables
- Creating equivalent expressions

Projects built:
- PEMDAS puzzle creation
- Expression evaluation game
- Calculator investigation project

Practice: Evaluate 20 complex expressions daily

Problem-Solving Strategies

Topics:
- Understanding the problem: what's being asked?
- Identifying relevant information
- Drawing diagrams and models
- Making organized lists
- Looking for patterns
- Working backwards strategy
- Guess and check method
- Using simpler numbers first
- Breaking complex problems into steps
- Checking reasonableness of answers
- Multiple solution methods
- Mathematical communication and explanation

Projects built:
- Problem-solving portfolio
- Create word problems for peers
- Strategy comparison presentation

Practice: Solve 5 multi-step word problems daily using different strategies

Phase 1 Review and Assessment

Topics:
- Number systems comprehensive review
- Operations mastery check
- Order of operations practice
- Problem-solving strategy application
- Common errors and corrections
- Speed and accuracy building

Projects built:
- Create a math tutorial video

- Design a board game using Phase 1 concepts
- Peer tutoring sessions

Assessment: Phase 1 cumulative exam: a written paper mixing this phase with everything before it, plus targeted revision for anything shaky

---

PHASE 2: FRACTIONS, DECIMALS AND PROPORTIONAL REASONING (WEEKS 14-26)

Master rational numbers, proportional relationships, and their real-world applications.

Months 4-5: Fractions and Mixed Numbers (Weeks 14-21)

Understanding Fractions

Topics:

- Fraction concepts: parts of a whole
- Numerator and denominator meaning
- Proper fractions, improper fractions, mixed numbers
- Converting between improper fractions and mixed numbers
- Fractions on a number line
- Equivalent fractions using models
- Simplifying fractions to lowest terms
- Finding equivalent fractions algebraically
- Comparing fractions with like denominators
- Comparing fractions with unlike denominators
- Cross multiplication for comparison
- Ordering fractions from least to greatest

Projects built:

- Fraction pizza party planning
- Fraction art with geometric shapes
- Recipe conversion project

Practice: 30 fraction problems daily, visual and numerical

Operations with Fractions

Topics:

- Adding fractions with like denominators
- Adding fractions with unlike denominators
- Finding common denominators
- Adding mixed numbers with regrouping
- Subtracting fractions and mixed numbers
- Borrowing in mixed number subtraction
- Multiplying fractions by whole numbers
- Multiplying fractions by fractions
- Understanding fraction multiplication visually
- Dividing fractions: keep-change-flip
- Dividing mixed numbers
- Word problems with fraction operations

Projects built:

- Cooking with fractions project
- Construction measurements activity
- Fraction operations game design

Practice: 25 fraction operation problems with increasing difficulty

Decimals and Place Value

Topics:

- Decimal place value: tenths, hundredths, thousandths
- Reading and writing decimals
- Decimals on number lines
- Comparing and ordering decimals
- Rounding decimals to various places
- Converting fractions to decimals
- Converting decimals to fractions
- Terminating vs repeating decimals
- Adding and subtracting decimals
- Lining up decimal points
- Estimation with decimals
- Real-world decimal applications: money, measurements

Projects built:

- Shopping budget with decimals
- Sports statistics analysis
- Decimal measurement lab

Practice: 30 decimal problems including conversions

Multiplying and Dividing Decimals

Topics:

- Multiplying decimals by whole numbers
- Multiplying decimals by decimals
- Counting decimal places in products
- Multiplying by powers of 10
- Dividing decimals by whole numbers
- Dividing by decimals: moving decimal points
- Dividing decimals by powers of 10
- Scientific notation introduction
- Estimation strategies for decimal operations

- Calculator use and interpretation
- Word problems with decimal operations
- Unit pricing and comparison shopping

Projects built:
- Business profit calculation project
- Science measurement conversions
- Create a decimal operations tutorial

Practice: 25 decimal multiplication and division problems

Percentages and Applications

Topics:
- Understanding percent as per hundred
- Converting between fractions, decimals, and percents
- Finding percentages of numbers
- Mental math with percentages (10%, 25%, 50%)
- Finding what percent one number is of another
- Finding the whole given a part and percent
- Percent increase and decrease
- Simple interest calculations
- Sales tax and discount problems
- Tips and commission calculations
- Percent error in measurements
- Data representation with percentages

Projects built:
- Sales and discount investigation
- Survey data percentage analysis
- Personal finance percentage project

Practice: 20 percentage problems from real-world contexts

Month 6: Ratios and Proportional Relationships (Weeks 22-26)

Understanding Ratios

Topics:
- Ratio concepts and notation
- Part-to-part vs part-to-whole ratios
- Equivalent ratios
- Simplifying ratios
- Ratios with three quantities
- Rates and unit rates
- Finding unit prices
- Speed, distance, and time relationships
- Comparing rates
- Best buy problems
- Ratio tables and their uses
- Graphing ratios on coordinate plane

Projects built:
- Recipe scaling project
- Speed comparison investigation
- Mix and match ratio art

Practice: 25 ratio and rate problems

Proportions and Scale

Topics:
- Understanding proportions
- Setting up proportions from word problems
- Cross multiplication to solve proportions
- Checking proportion solutions
- Direct variation
- Inverse variation introduction
- Scale drawings and maps
- Finding actual distances from maps
- Creating scale drawings
- Similar figures and proportions
- Scale factor
- Proportional relationships in data

Projects built:
- Map scale project
- Architectural drawing activity
- Proportion in art exploration

Practice: 20 proportion problems including scale applications

Real-World Applications

Topics:
- Currency exchange rates
- Unit conversions using proportions
- Mixture problems
- Work rate problems
- Population density calculations
- Fuel efficiency comparisons
- Nutritional information analysis
- Sports statistics and ratios
- Business applications: profit margins
- Scientific applications: concentrations

- Engineering: gear ratios
- Cooking and baking proportions

Projects built:
- International currency project
- Nutrition label analysis
- Sports statistics presentation

Practice: Apply proportional reasoning to 15 real-world scenarios

Data with Fractions, Decimals, and Percents

Topics:
- Circle graphs (pie charts) with percentages
- Bar graphs with fractional values
- Line graphs with decimal scales
- Converting between data representations
- Misleading graphs and statistics
- Surveys and sampling basics
- Calculating averages with decimals
- Finding median and mode with fractions
- Data interpretation questions
- Creating data displays
- Spreadsheet basics for calculations
- Technology tools for data analysis

Projects built:
- Class survey data analysis
- Create infographic with percentages
- Data story presentation

Practice: Analyze 10 different data sets using various representations

Phase 2 Review and Assessment

Topics:
- Comprehensive fraction review
- Decimal operations mastery
- Percentage problem solving
- Ratio and proportion applications
- Mixed problem sets
- Speed and accuracy challenges

Projects built:
- Create a fraction-decimal-percent converter tool
- Design a proportion puzzle book
- Real-world math fair project

Assessment: Phase 2 cumulative exam: a written paper mixing this phase with everything before it, plus targeted revision for anything shaky

---

PHASE 3: ALGEBRAIC THINKING AND PRE-ALGEBRA (WEEKS 27-39)

Develop algebraic reasoning, solve equations, and understand functions and patterns.

Months 7-8: Introduction to Algebra (Weeks 27-34)

Variables and Expressions

Topics:
- Introduction to variables as unknowns
- Variables as varying quantities
- Writing algebraic expressions
- Translating words to algebra
- Evaluating expressions with substitution
- Terms, coefficients, and constants
- Like terms and unlike terms
- Combining like terms
- Distributive property with variables
- Factoring out common factors
- Expanding expressions
- Simplifying algebraic expressions

Projects built:
- Expression matching game
- Variable story problems creation
- Algebraic art patterns

Practice: 30 expression problems: writing, evaluating, simplifying

One-Step Equations

Topics:
- Understanding equations vs expressions
- Solution concept and checking
- Solving using addition/subtraction
- Solving using multiplication/division
- Inverse operations review
- Equations with negative numbers
- Equations with fractions
- Equations with decimals
- Writing equations from word problems
- Checking solutions by substitution
- Special cases: no solution, infinite solutions

- Real-world equation modeling
  Projects built:
- Equation balance demonstration
- Word problem equation book
- Equation solving race game
  Practice: Solve 25 one-step equations with various number types
Two-Step and Multi-Step Equations
  Topics:
- Two-step equations strategy
- Order of operations in reverse
- Equations with parentheses
- Distributive property in equations
- Combining like terms before solving
- Variables on both sides
- Equations with fractions: clearing denominators
- Decimal equations: clearing decimals
- Word problems requiring multi-step equations
- Consecutive integer problems
- Perimeter and area equations
- Mixture and motion problems basics
  Projects built:
- Multi-step equation puzzle
- Real-world equation modeling project
- Create equation tutorial video
  Practice: 20 multi-step equations with increasing complexity
Inequalities
  Topics:
- Understanding inequality symbols
- Graphing inequalities on number lines
- Writing inequalities from situations
- Solving one-step inequalities
- Solving two-step inequalities
- Multiplying/dividing by negatives: flipping the sign
- Compound inequalities (AND, OR)
- Graphing compound inequalities
- Inequality word problems
- Systems of inequalities introduction
- Real-world constraints and inequalities
- Checking inequality solutions
  Projects built:
- Inequality graphing art
- Budget constraint project
- Design inequality game
  Practice: 25 inequality problems with graphing
Introduction to Functions
  Topics:
- Input-output relationships
- Function machines concept
- Function notation: f(x)
- Function tables
- Finding patterns in functions
- Linear functions basics
- Graphing functions on coordinate plane
- Domain and range introduction
- Vertical line test
- Real-world functions: temperature conversion
- Function composition basics
- Inverse operations and functions
  Projects built:
- Function machine construction
- Real-world function investigation
- Create function puzzle book
  Practice: Evaluate and graph 20 different functions

Month 9: Coordinate Geometry and Graphing (Weeks 35-39)
The Coordinate Plane
  Topics:
- Cartesian coordinate system
- x-axis and y-axis
- Quadrants I, II, III, IV
- Plotting points (x, y)
- Finding coordinates of points
- Distance between points (horizontal/vertical)
- Midpoint of a line segment
- Reflections across axes
- Translations on coordinate plane
- Rotations (90°, 180°)
- Creating designs with coordinates
- Coordinate plane applications

Projects built:
- Coordinate art project
- Treasure map coordinates
- Transformation artwork

Practice: Plot 30 points and perform transformations

Linear Relationships

Topics:
- Understanding slope as rate of change
- Calculating slope from two points
- Positive, negative, zero, undefined slopes
- Slope-intercept form: y = mx + b
- Finding y-intercept
- Graphing using slope-intercept form
- Point-slope form introduction
- Writing equations from graphs
- Writing equations from two points
- Parallel and perpendicular lines
- Systems of linear equations (graphing method)
- Real-world linear relationships

Projects built:
- Linear relationship investigation
- Cost analysis with linear functions
- Design with linear equations

Practice: Graph 20 linear equations and find equations from graphs

Patterns and Sequences

Topics:
- Number patterns recognition
- Arithmetic sequences
- Finding common difference
- nth term of arithmetic sequence
- Geometric sequences introduction
- Finding common ratio
- Extending patterns
- Visual patterns and growth
- Fibonacci sequence exploration
- Recursive patterns
- Creating pattern rules
- Patterns in nature and art

Projects built:
- Pattern investigation poster
- Sequence art creation
- Pattern puzzle book

Practice: Identify and extend 25 different patterns

Polynomials Introduction

Topics:
- Monomials, binomials, trinomials
- Degree of polynomials
- Adding polynomials
- Subtracting polynomials
- Multiplying monomials
- Multiplying polynomials by monomials
- FOIL method for binomials
- Special products: $(a+b)^2$, $(a-b)^2$
- Factoring out GCF from polynomials
- Factoring simple trinomials
- Zero product property
- Polynomial applications

Projects built:
- Polynomial art with algebra tiles
- Area models for multiplication
- Polynomial puzzle creation

Practice: 20 polynomial operations and factoring problems

Phase 3 Review and Assessment

Topics:
- Algebraic expressions and equations review
- Inequality solving and graphing
- Function concepts and graphing
- Coordinate geometry mastery
- Linear relationships
- Patterns and sequences
- Polynomial operations

Projects built:
- Algebra tutorial video series
- Create algebraic board game
- Real-world algebra applications portfolio

Assessment: Phase 3 cumulative exam: a written paper mixing this phase with everything before it, plus targeted revision for anything shaky

---

PHASE 4: GEOMETRY, STATISTICS AND FINAL REVISION (WEEKS 40-52)

Master geometric concepts, statistical analysis, probability, and complex problem-solving strategies.

Month 10: Geometry Foundations (Weeks 40-44)
Basic Geometric Concepts
Topics:
- Points, lines, and planes
- Line segments and rays
- Parallel and perpendicular lines
- Angles: acute, right, obtuse, straight
- Measuring angles with protractor
- Angle relationships: complementary, supplementary
- Vertical angles and linear pairs
- Angles formed by transversals
- Corresponding angles
- Alternate interior and exterior angles
- Triangle angle sum theorem
- Exterior angles of triangles

Projects built:
- Angle art creation
- Architecture angle investigation
- Geometric city design

Practice: Measure and calculate 30 angle problems

Triangles and Quadrilaterals
Topics:
- Types of triangles by sides: equilateral, isosceles, scalene
- Types of triangles by angles: acute, right, obtuse
- Triangle inequality theorem
- Congruent triangles: SSS, SAS, ASA
- Similar triangles and proportions
- Pythagorean theorem
- Pythagorean theorem applications
- Special right triangles: 45-45-90, 30-60-90
- Quadrilaterals: parallelograms, rectangles, squares
- Rhombus and trapezoid properties
- Diagonals of quadrilaterals
- Polygon angle sum formulas

Projects built:
- Triangle construction project
- Pythagorean theorem proofs
- Quadrilateral classification poster

Practice: 25 triangle and quadrilateral problems

Circles
Topics:
- Circle vocabulary: center, radius, diameter
- Circumference formula: C = 2πr
- Area of circles: A = πr²
- Arc length
- Sector area
- Chords and secants
- Tangent lines to circles
- Inscribed angles
- Central angles
- Circle word problems
- Composite figures with circles
- Real-world circle applications

Projects built:
- Circle art with compass
- Pi investigation project
- Design circular patterns

Practice: 20 circle calculation problems

Area and Perimeter
Topics:
- Perimeter of polygons
- Area of rectangles and squares
- Area of parallelograms
- Area of triangles: ½bh
- Area of trapezoids
- Area of regular polygons
- Area of composite figures
- Perimeter of composite figures
- Area and perimeter word problems
- Optimization problems: maximum area
- Surface area introduction
- Real-world applications: flooring, fencing

Projects built:
- Garden design project

- Floor plan area calculations
- Packaging design optimization

Practice: Calculate area and perimeter of 25 figures

3D Geometry and Volume

Topics:
- 3D shapes: prisms, pyramids, cylinders, cones, spheres
- Faces, edges, and vertices
- Nets of 3D shapes
- Volume of rectangular prisms
- Volume of triangular prisms
- Volume of cylinders
- Volume of pyramids and cones
- Volume of spheres
- Surface area of prisms
- Surface area of cylinders
- Composite 3D figures
- Volume and capacity conversions

Projects built:
- 3D shape construction
- Volume investigation lab
- Package design project

Practice: Calculate volume and surface area of 20 3D shapes

Month 11: Statistics and Data Analysis (Weeks 45-49)

Data Collection and Representation

Topics:
- Types of data: categorical vs numerical
- Data collection methods
- Surveys and sampling techniques
- Bias in data collection
- Frequency tables
- Bar graphs and histograms
- Line graphs and time series
- Circle graphs (pie charts)
- Stem-and-leaf plots
- Box-and-whisker plots
- Scatter plots introduction
- Choosing appropriate graphs

Projects built:
- School survey project
- Data visualization poster
- Create data infographic

Practice: Create 10 different data representations

Measures of Central Tendency

Topics:
- Mean (average) calculation
- Weighted averages
- Median finding
- Mode identification
- Comparing mean, median, mode
- Effects of outliers
- Range as measure of spread
- Quartiles and interquartile range
- Mean absolute deviation
- Standard deviation introduction
- Interpreting data spread
- Using measures to compare data sets

Projects built:
- Grade analysis project
- Sports statistics comparison
- Weather data analysis

Practice: Calculate statistics for 15 data sets

Probability Fundamentals

Topics:
- Probability as likelihood
- Probability scale: 0 to 1
- Experimental vs theoretical probability
- Sample space and outcomes
- Simple probability calculations
- Complement of an event
- Probability of compound events
- Independent vs dependent events
- Tree diagrams
- Fundamental counting principle
- Permutations introduction
- Combinations introduction

Projects built:
- Probability game design
- Dice experiment analysis

- Real-world probability investigation
  Practice: Solve 25 probability problems
Statistical Analysis and Interpretation
  Topics:
    - Correlation vs causation
    - Positive and negative correlation
    - Line of best fit basics
    - Making predictions from data
    - Margin of error concept
    - Confidence intervals introduction
    - Misleading statistics
    - Statistical significance basics
    - Comparing populations
    - Data-driven decision making
    - Technology tools for statistics
    - Spreadsheet data analysis
  Projects built:
    - Correlation investigation
    - Statistical report writing
    - Data prediction project
  Practice: Analyze and interpret 10 real-world data scenarios
Problem Solving with Multiple Concepts
  Topics:
    - Multi-step problems combining algebra and geometry
    - Rate, ratio, and proportion in geometry
    - Statistical problems with percentages
    - Probability with fractions and decimals
    - Optimization problems
    - Logic puzzles and reasoning
    - Mathematical modeling
    - Estimation and approximation
    - Working with constraints
    - Breaking down complex problems
    - Checking solutions for reasonableness
  Projects built:
    - Complex problem-solving portfolio
    - Mathematical modeling project
    - Create problem-solving guide
  Practice: Solve 15 complex, multi-concept problems

Month 12: Advanced Topics and Test Preparation (Weeks 50-51)
Competition Mathematics
  Topics:
    - Problem-solving strategies review
    - Competition problem types
    - Speed and accuracy techniques
    - Mental math advanced strategies
    - Olympiad-style problems
    - Logic and reasoning puzzles
    - Number theory problems
    - Geometry competition problems
    - Combinatorics basics
    - Proof techniques
    - Time management in tests
  Projects built:
    - Math competition preparation
    - Create competition problems
    - Strategy guide compilation
  Practice: Solve 20 competition-level problems
Test Preparation and Strategies
  Topics:
    - Standardized test formats
    - Multiple choice strategies
    - Grid-in question techniques
    - Extended response writing
    - Time management during tests
    - Test anxiety management
    - Common test mistakes
    - Calculator vs non-calculator sections
    - Checking work efficiently
    - Practice test analysis
    - Study planning and organization
  Projects built:
    - Test-taking strategy guide
    - Practice test creation
    - Study plan development
  Practice: Complete 3 full-length practice tests

Month 13: Final Projects and Comprehensive Review (Week 52)

Comprehensive Review and Integration
Topics:
- Number systems and operations review
- Fractions, decimals, percentages mastery
- Algebraic concepts consolidation
- Geometry theorems and applications
- Statistics and probability review
- Problem-solving strategies compilation
- Cross-curricular connections
- Real-world applications synthesis
- Mathematical communication skills
- Technology integration review
- Portfolio organization
- Self-assessment and reflection

Projects built:
- Year-long portfolio compilation
- Mathematical autobiography
- Comprehensive problem set creation

Assessment: FINAL EXAM: a comprehensive written paper covering every phase of the course. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Number System Timeline, Budget Planner, Factor Tree Art, Problem-Solving Portfolio
- Phase 2: Fraction Pizza Party, Shopping Budget Calculator, Scale Drawing, Data Infographic
- Phase 3: Equation Balance Game, Function Machine, Coordinate Art, Pattern Investigation
- Phase 4: Geometric City Design, Statistics Survey Project, Probability Game, Math Fair Project
- Final: Comprehensive Portfolio, Research Project, Real-World Application

Total projects: 60+ projects ranging from simple to complex applications

SKILLS THE STUDENT WILL HAVE AT THE END

- Number Systems: Whole numbers, integers, rational numbers, real numbers
- Operations: All four operations with various number types, order of operations
- Fractions & Decimals: Operations, conversions, real-world applications
- Percentages: Calculations, increases/decreases, financial applications
- Algebra: Variables, expressions, equations, inequalities, functions
- Geometry: 2D shapes, 3D solids, measurements, transformations, proofs
- Statistics: Data collection, analysis, measures of central tendency, displays
- Probability: Basic probability, compound events, counting principles
- Problem Solving: Multiple strategies, mathematical modeling, logic
- Technology: Calculator use, spreadsheets, graphing tools, online resources
- Communication: Mathematical writing, presentations, explanations
- Test Taking: Standardized test strategies, time management, accuracy

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week with a real teacher, working problems together
Homework: A problem set after every class, and a mixed-review assignment at the end of every month that deliberately folds in earlier topics
Practice: 3-5 hours of problem-solving between classes
Review: Every set reviewed with written feedback; recurring mistakes reopened in class until they are gone

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: which topics are solid, which are wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies mastery, not attendance

WHERE THIS CAN LEAD

- High School Advanced Mathematics Track
- AP Mathematics Preparation
- STEM Program Readiness
- Mathematics Competition Participant
- Science Fair Participant
- Engineering Program Preparation
- Computer Science Foundation
- Financial Literacy

  - Data Science Introduction
  - Mathematical Research Interest

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What does the Middle School Mathematics program cover?
  A: The 12 month program builds in four phases: number systems and operations; fractions, decimals, percentages and ratios; pre-algebra with equations, functions and graphing; and finally geometry, statistics, probability and multi-step problem solving. The aim is to make the middle school years the strongest part of a child's maths story.

  Q: Who is this course right for?
  A: A gentle diagnostic in the first weeks means each child starts where their actual understanding is, not where their grade label says.

  Q: What does my child need before joining?
  A: Just completed elementary school mathematics, a computer or tablet with internet, and a calculator. No coding background is needed, and grade level reading is enough to follow the classes comfortably.

  Q: Will this help with school exams and word problems?
  A: Yes. Word problems and multi-step reasoning are treated as core skills throughout, not an afterthought: children learn to translate situations into maths before computing. Because the course teaches the concepts under every school syllabus, the work transfers directly to classroom tests.

  Q: How is progress measured?
  A: Through weekly practice reviewed for method, phase-end checks before new material begins, and a running mastery record shared with parents. The final month includes projects and a comprehensive review, so you finish with visible evidence of what your child can now do.

  Q: How is my child assessed, and is the certificate just for attending?
  A: No. A problem set follows every class with written feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, each phase ends with a cumulative exam, and the course ends with a final exam covering the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest. We measure understanding, not attendance.

  Q: Can my child attend a class before we decide?
  A: Of course. The first step for every family is a free demo class, booked at learn.modernagecoders.com/contact with no commitment, where your child meets the teacher and we assess the right starting point together. You can also WhatsApp us at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: What is the class format?
  A: Live one hour online classes with a real teacher, never pre-recorded videos. Groups stay small so quieter children get airtime too, and every child solves problems aloud during class rather than just watching.

  Q: Does my child receive a certificate?
  A: Yes, the Middle School Mathematics Excellence Certificate on completion, plus a record of the projects and problem work done across the year. Modern Age Coders holds a 4.9 star rating from 547+ Google reviews.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/comprehensive-middle-school-mathematics-mastery
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 101 OF 137: MINECRAFT CODING FOR KIDS: FROM BLOCKS TO REAL CODE
==========

Course link: https://learn.modernagecoders.com/courses/minecraft-coding-for-kids-course
Subject area: Kids Coding
Designed for: Kids
Who can join: Beginner, ages 7 to 12
Course length: 4 months (16 weeks)
Time needed: 2 live classes/week + 1-2 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150

Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Minecraft coding classes for kids ages 7-12: live online lessons using block-based MakeCode, from first commands to robot agents, auto-farms, and mini-games.

In one line: The game your child already loves becomes the place they learn to think in code.

ABOUT THE PROGRAMME

Minecraft has a superpower most parents never see: a built-in coding environment where children snap together blocks of code, press run, and watch the game obey. A castle wall appears in two seconds. A robot agent plants a farm. A trap springs when someone steps on the wrong block. This 4-month live course uses that loop, code it, run it, see it, to teach real programming ideas to children aged 7 to 12. Months 1 and 2 build the core: commands, the agent robot, loops that build faster than hands ever could, variables, and decisions. Months 3 and 4 turn those pieces into games: randomness, scores, timers, a full mini-game classmates play, and a finishing capstone the student designs. Along the way, older students get to flip the same blocks into JavaScript view and see what their program looks like as typed code. Every class is live and hands-on. The instructor sets a build challenge, students code their own solution while the instructor watches and helps, and each week ends with something running in the world that was not there before.

WHAT MAKES THIS COURSE DIFFERENT

- Coding happens inside Minecraft itself, so the feedback loop is instant and the motivation is already installed
- The agent robot makes loops and conditions physical: children watch their code walk, turn, plant, and build block by block
- Challenges are sized for live classes: every session ends with a working build, not a half-finished follow-along
- Block code is flipped to JavaScript view for curious older kids, so the bridge to typed programming is walked, not just mentioned
- Screen time turns productive: the same game hours now produce farms, machines, and mini-games the child engineered
- Small live batches mean the instructor sees every child's screen and every child's thinking, every class

THE LEARNING PATH, STEP BY STEP

Phase 1: Code that builds: chat commands, the agent robot, then loops, variables, and decisions that construct in seconds what hands build in hours
Phase 2: Code that plays: randomness, scores, timers, and events combined into real mini-games, then a student-designed capstone showcased to families

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The course starts from the very first block of code
Reading: Basic reading comfort, enough to follow labeled blocks. Seven-year-olds who read simple sentences do fine
Minecraft setup: Minecraft Education, which includes the built-in coding editor. We help every family set it up before classes start
Equipment: A computer or iPad that runs Minecraft Education, plus a stable internet connection

WHO THIS COURSE IS FOR

Minecraft fans: Kids who play Minecraft daily and are ready to command it instead of just mining in it
Complete beginners: Children aged 7 to 12 with zero coding exposure who need a first step that feels like play
Restless learners: Kids who resist worksheets but will happily debug a robot farmer for twenty minutes
Future typers: Older kids in the range who are curious what real typed code looks like, met here through the JavaScript view
Screen time worriers: Parents who want existing Minecraft hours converted into planning, logic, and finished projects

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: CODE THAT BUILDS (MONTHS 1-2, WEEKS 1-8)

From the first chat command to an automated farm: children learn sequencing, loops,

variables, and decisions by making Minecraft do the building for them.

Month 1: First Commands and the Agent (Weeks 1-4)
Hello, Code Builder
Topics:
- Opening the coding editor inside Minecraft and a tour of the block palette
- On chat command: making code run when you type a magic word
- Placing your first block with code instead of your hands
- Running, stopping, and re-running a program
- Changing one block in the code and predicting what changes in the world
- Saving projects so next class starts where this one ended

Projects built:
- Magic words: three chat commands that summon a block tower, a splash of flowers, and a chicken

Practice: Invent two new chat commands of your own and draw what each one should do before coding it

The Agent, Your Robot
Topics:
- Meeting the agent: the robot that obeys your program
- Move forward, turn left, turn right: navigating step by step
- Agent place and agent destroy: building and clearing
- Sequencing: why the order of blocks changes everything
- Walking a maze on paper first, then in code
- Teleporting the agent back when it wanders off

Projects built:
- Maze runner: program the agent to walk a maze you built for it, placing a torch at the exit

Practice: Build a new maze for a classmate's agent and solve theirs, comparing step counts

Loops: Building at Superspeed
Topics:
- The repeat block: doing something ten times without ten copies
- Building a wall with a loop instead of block-by-block
- Loops inside loops: from a wall to a whole floor
- Counting carefully: the off-by-one surprise
- The forever loop and when it is useful
- Hand-build vs loop-build: racing the same wall both ways

Projects built:
- Wall factory: one program that builds a wall of any length, then a full square fort using nested loops

Practice: Use loops to build a staircase to the clouds, then estimate how long it would have taken by hand

Big Builds with Loops
Topics:
- Planning a build as layers: what repeats and what does not
- Pyramids: each layer smaller than the last
- Hollow shapes: skipping the middle to save blocks
- Combining agent moves and loops for long structures
- Debugging a build gone wrong: reading the code out loud
- Showing your build: a mini tour for the class

Projects built:
- Pyramid of Giza: a layered pyramid built entirely by code, with a hollow secret chamber inside

Practice: Design and code one original loop-built monument for the class world
Assessment: Month 1 build check: the agent completes a fresh maze and a loop-built structure, explained block by block

Month 2: Variables and Decisions (Weeks 5-8)
Variables: Code That Remembers
Topics:
- What a variable is: a labeled box the program remembers
- A size variable: one number that controls how big the build is
- Changing a variable while the program runs
- Counting things: how many blocks the agent has placed
- Announcing values in chat: the program reports back
- Naming variables so they explain themselves

Projects built:
- Any-size house: one program that builds a house small, medium, or giant by changing a single number

Practice: Add a height variable to your fort program and test it at three different values

Decisions: Code That Checks
Topics:
- If and else: teaching the program to choose
- Agent detect: is there a block in front of me?
- Obstacle dodging: walk forward, but go around what blocks you
- Checking what a block is made of before acting
- Combining checks: if this and not that
- Predict-then-run: guessing what a decision program will do

Projects built:

- Smart explorer: an agent that crosses rough ground by detecting obstacles and stepping around them

Practice: Program a tunnel-digger that stops when it detects water, so nobody floods the base

Events: Code That Reacts

Topics:

- On player walk: code that fires when someone steps somewhere
- On block placed and on block broken
- Building a pressure-plate trap the coding way
- Surprise effects: fireworks, sounds, and chat messages
- Being a fair trap-maker: warnings and escape routes
- Multiple events in one world without chaos

Projects built:

- Trap gauntlet: a walkway with three coded surprises, from harmless confetti to a drop into a pool

Practice: Add a welcome event to the class world that greets any player who arrives at your base

Project Week: The Auto-Farm

Topics:

- Automation: teaching the agent a full job, start to finish
- Till, plant, wait, harvest: the farming loop
- Combining loops, variables, and detection in one program
- Testing a long program one section at a time
- Making the farm bigger without rewriting it
- Explaining your automation to someone who does not code

Projects built:

- Agent auto-farm: a robot farmer that tills the soil, plants crops in rows, and harvests when ready

Practice: Extend your farm with a fence-building routine, then measure how many crops one run produces

Assessment: Month 2 skills check: a live automation challenge solved with loops, a variable, and at least one detect decision

---

PHASE 2: CODE THAT PLAYS (MONTHS 3-4, WEEKS 9-16)

The building skills become game-making skills: randomness, scores, and timers combine into real mini-games, and the course ends with a student-designed capstone and a peek at real typed code.

Month 3: Mini-Games (Weeks 9-12)

Randomness

Topics:

- Random numbers: why games need surprise
- Spawning animals at random positions
- Random blocks: a patchwork floor no one can predict
- Hiding treasure somewhere new every game
- Fair vs unfair randomness in games
- Testing random code: run it many times, watch the spread

Projects built:

- Treasure hunt: a diamond block hidden at a random spot each round, with hot-and-cold chat hints

Practice: Make a random pet generator that spawns one of five animals, and tally twenty runs to see the spread

Keeping Score

Topics:

- A score variable that events update
- Announcing the score in chat after each point
- Win conditions: first to ten
- Resetting the game for a fresh round
- Counting down instead of up: lives
- Score displays that players actually notice

Projects built:

- Collector clash: gather the glowing blocks scattered by code, one point each, first to ten wins

Practice: Add a lives system to your treasure hunt: three wrong digs and the round ends

Timers and Pressure

Topics:

- Countdown timers built from loops and waits
- Beat the clock: finishing a task before time runs out
- Announcing time left at the right moments
- Combining a timer with a score for real tension
- Rounds: game start, game play, game over
- Tuning time limits so the game is hard but winnable

Projects built:

- Build rush: players race a two-minute timer to copy a target structure, with a code-checked finish

Practice: Playtest your build rush with family and adjust the time limit after two rounds

Project Week: The Arena Game
Topics:
- Combining the month: randomness, score, timer, and events in one game
- Building the arena with your loop skills
- Rules players can learn in ten seconds
- Running a game for classmates: host duties
- Watching players find the fun and the flaws
- One improvement pass after the first playtest
Projects built:
- Falling-floor arena: a coded mini-game where floor sections vanish on a timer and the last player standing wins
Practice: Host two rounds of your arena for classmates and write down the one rule that confused people most
Assessment: Arena game demo: classmates play a full round, then the builder explains the timer and event code behind it

Month 4: The Capstone and the Code Peek (Weeks 13-16)
Functions: Naming Your Powers
Topics:
- Functions: wrapping a build routine in a name
- One buildTower function used five times
- Functions with a size input
- A personal toolkit: your best routines, reusable forever
- Cleaner programs: reading code made of named pieces
- Sharing a function with a classmate and using theirs
Projects built:
- Village in a minute: houses, paths, and lamp posts, each a function, assembled into a whole village program
Practice: Turn your favorite build from the course into a function and use it three times in one world
The JavaScript Peek
Topics:
- Flipping the editor: the same program as blocks and as JavaScript text
- Finding your loop and your variable in the typed code
- What brackets, dots, and semicolons are doing
- Making one tiny change in text view and running it
- Block coding vs typed coding: same thinking, different clothes
- Where typed languages lead: Python, JavaScript, and beyond
Projects built:
- Two-view build: a fountain program written in blocks, then modified once in JavaScript view to change its size
Practice: Flip three of your old programs to code view and find the repeat loop in each one
Capstone Build Sprint
Topics:
- Choosing a capstone: an original game or a grand automation
- The one-page plan: what it does, what code it needs, what done means
- Building must-haves first
- Reusing your own functions and past code honestly
- Instructor check-in: unblocking the hard part early
- Trimming the plan when time runs short, without giving up the core
Projects built:
- Capstone, part one: the core of an original mini-game or automation, planned and working end to end
Practice: Two 30-minute build sessions at home, each ending with a one-sentence progress note
Finish and Showcase
Topics:
- The final bug hunt: testing every part on purpose
- Polish: sounds, messages, and a proper welcome
- Writing simple instructions for players
- Presenting: show the game first, then the code behind the best part
- Answering questions about your own program
- What comes next: Python for kids, or Roblox game coding
Projects built:
- Finished capstone: an original, tested Minecraft game or automation presented live to classmates and families
Practice: Run your capstone for two family members and collect one compliment and one improvement idea from each
Assessment: Capstone showcase and code Q&A, plus course-completion certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- Magic chat commands that summon builds and creatures
- Agent maze runner with a torch-lit victory exit
- Nested-loop square fort and cloud staircase
- Code-built Pyramid of Giza with a hidden chamber
- Any-size house driven by a single variable
- Smart explorer agent that detects and dodges obstacles

- Agent auto-farm that tills, plants, and harvests
- Random treasure hunt with hot-and-cold hints
- Falling-floor arena mini-game with timer and score
- Village-in-a-minute built from student-written functions
- Original capstone game or automation, showcased to families

Total projects: 14+ working programs, from a first chat command to an original mini-game

SKILLS THE STUDENT WILL HAVE AT THE END

- Sequencing and the agent: precise step-by-step instructions a robot can follow
- Loops and nested loops: building in seconds what hands build in hours
- Variables and decisions: programs that remember, count, check, and choose
- Events, randomness, scores, and timers: the ingredients of every mini-game
- Functions: naming and reusing routines like a real programmer
- Reading typed code: finding loops and variables in the JavaScript view of their own programs

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding challenges solved along with the instructor
Practice: 1-2 hours of build challenges between classes, sized for young attention spans
Review: Practice builds reviewed with feedback; tricky concepts replayed at the start of the next class
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so nobody stays stuck until the next session
Progress updates: Regular progress notes to parents: what was covered, what was built, what needs practice

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Our Python for kids course, the natural next step once block logic is solid
- Roblox game coding, where the same ideas graduate to typed Luau scripts in a 3D engine
- Our Scratch-based creative coding track for students who want more game design first
- School coding competitions and clubs, entered with a portfolio of working programs
- App development for kids, for children who want to build beyond games

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course run by Mojang or Microsoft?
A: No. Modern Age Coders is an independent coding school, and this course is not affiliated with, sponsored by, or endorsed by Mojang or Microsoft. We teach coding using Minecraft Education and its built-in MakeCode editor, tools Microsoft makes available for exactly this kind of learning, but the curriculum and classes are entirely ours.

Q: Which version of Minecraft do we need? Will our normal Minecraft work?
A: The course uses Minecraft Education, the school-focused version with the coding editor (Code Builder) built in. The regular Bedrock or Java editions your child may already own do not include this editor, so Education is required; it runs on Windows, Mac, iPad, and most Chromebooks, and needs its own low-cost licence or a school Microsoft 365 Education account. Before the batch starts we walk every family through the setup, and we confirm your device runs it during the free demo. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Will my child actually learn coding, or just play Minecraft in class?
A: Every class is a coding class; Minecraft is the place the code runs, not the activity. Students spend sessions writing programs: loops that raise walls, decisions that steer a robot, events that trigger traps, timers that run games. The game world is what makes a seven-year-old care whether their loop is correct, because a wrong loop builds a crooked castle in front of everyone.

Q: My child already gets too much screen time. Does this add more?
A: For most families it converts time rather than adds it. The hours a child was already spending in Minecraft start producing farms they automated, games they invented, and bugs they fixed, which exercises planning and logic in a way passive play does not. Classes are

one focused hour, and home practice is deliberately kept to one or two hours a week, sized for this age group.

Q: Is 7 too young? My older child is 12, is it too basic?
A: Both ends of the range are served, differently. Sevens thrive because block coding removes typing and spelling from the equation; they drag, snap, and run. Twelves stay stretched through harder challenges, functions, and the JavaScript view, where they read and edit their own programs as typed code. Batches are grouped by age so the pace fits the room.

Q: What does my child move on to after this course?
A: The two most common paths are our Python for kids course, which takes the same ideas into the world's most-used typed language, and Roblox game coding, where students script 3D games in Luau. The instructor recommends a path for each child at the end of the course based on what they gravitated toward.

Q: How is my child assessed, and is the certificate just for attending?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a friendly final assessment of everything learned. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Can we try a class before enrolling?
A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. The demo doubles as a tech check: we make sure Minecraft Education runs on your device and answer setup questions with a parent present.

Q: Do parents need to know anything about coding or Minecraft?
A: No. Many parents tell us that presentation is the first time they understood what the course was building all along.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/minecraft-coding-for-kids-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 102 OF 137: MYSQL COURSE: SQL, DATABASE DESIGN AND REAL ADMINISTRATION

==========

Course link: https://learn.modernagecoders.com/courses/mysql-database-complete-masterclass-college
Subject area: Databases
Designed for: College students
Who can join: Complete beginner; college students and working professionals (18+)
Course length: 7-8 months (28-32 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

MySQL course for college students and professionals: SQL mastery, database design, stored procedures, transactions, Python integration and honest admin in 7-8 months.

In one line: The data layer, owned end to end: SQL fluency, real schema design, the programmable database, and the production habits developers are actually judged on.

ABOUT THE PROGRAMME

Every application stands on a database, and every serious interview eventually reaches SQL. This live online masterclass takes college students and professionals from zero to genuine capability in seven months: SQL written fluently, schemas designed rather than accumulated, the programmable side of MySQL (views, stored procedures, triggers, transactions), Python integration where data becomes application, and the production basics, security, backups and performance tuning, that separate developers who use databases from developers databases can trust. The scope is deliberately honest. Full database administration, replication clusters, multi-cloud failover, is its own profession, and courses that staple it onto a beginner syllabus produce neither developers nor DBAs. This course builds the 95% that developers, analysts and interviews actually demand, names the DBA path clearly for those who want it next, and spends the reclaimed months making the fundamentals genuinely deep: joins that never confuse, EXPLAIN read like a native, transactions understood well enough to argue about. The pace is honest: seven months at two live classes a week plus three to four hours

of practice, with an eighth month in hand. Problem sets after every class, monthly mixed reviews, and a final exam, practical plus written, that decides the certificate with a free retest after revision. Real queries, real design, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Honest scope: the developer 95%, not a stapled-on DBA career track
- Design is taught as a craft: ER modeling, normalization and constraints, practiced on real domains
- The programmable database gets real weeks: views, procedures, triggers and transactions, not a demo each
- EXPLAIN fluency: performance tuning taught as reading, reasoning and measuring
- Ends integrated: MySQL driven from Python, with security, backups and tuning as habits
- Real assessment: weekly problem sets, monthly mixed reviews, and a final exam that decides the certificate, with focused revision and a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: SQL foundations (Months 1-2): setup, DDL, DML and SELECT mastery on real datasets
Phase 2: Design and joins (Months 3-4): relationships, joins without fear, subqueries and set operations, and schema design done properly
Phase 3: The programmable database (Months 5-6): views, indexes, stored procedures, triggers, transactions and Python integration
Phase 4: Production month (Month 7): security and backups, performance tuning with EXPLAIN, and the capstone exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None for the SQL core; the Python integration weeks are guided and beginner-safe
Equipment: A computer with a stable internet connection; MySQL Community Server and every tool used are free
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 3-4 hours of practice a week

WHO THIS COURSE IS FOR

College students: Students who want the DBMS course their degree gestures at, taught until it is actually owned
Placement candidates: Anyone whose next interview round includes SQL, which is nearly everyone
Developers: Working developers who copy-paste queries and want to stop
Analysts: Excel-strong professionals ready for the real data tool underneath
Builders: Founders and makers who need their product's data layer done right

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: SQL FOUNDATIONS (MONTHS 1-2, WEEKS 1-8)

The language of data learned properly: definitions, manipulation and SELECT mastery on datasets that feel real.

Months 1-2: The SQL Core (Weeks 1-8)
Introduction to Databases & MySQL Setup
Topics:
- What are databases and why we need them
- Relational vs Non-relational databases
- History and evolution of MySQL
- MySQL vs other databases (PostgreSQL, Oracle, SQL Server)
- MySQL ecosystem: Community, Enterprise, MariaDB, Percona
- Installing MySQL on Windows, Linux, Mac
- MySQL Workbench installation and setup
- Command-line client basics
- phpMyAdmin and other GUI tools
- Creating your first database
- Understanding schemas and instances
- Basic security during installation
Projects built:
- Install MySQL on local machine
- Set up MySQL Workbench
- Create first database and tables
- Configure basic security settings
Practice: Complete 20 basic database operations
SQL Basics - Data Definition Language (DDL)
Topics:
- Creating databases with CREATE DATABASE

- Dropping databases safely
- Creating tables with CREATE TABLE
- Data types: Numeric (INT, DECIMAL, FLOAT)
- String types (CHAR, VARCHAR, TEXT)
- Date and time types (DATE, TIME, DATETIME, TIMESTAMP)
- Boolean and ENUM types
- Understanding NULL and NOT NULL
- Primary keys and their importance
- AUTO_INCREMENT for automatic numbering
- Altering tables with ALTER TABLE
- Dropping and truncating tables

Projects built:
- Design employee database schema
- Create online store database structure
- Build student management system tables
- Library management database design

Practice: Create 30 different table structures

SQL Basics - Data Manipulation Language (DML)

Topics:
- Inserting data with INSERT
- Bulk insert operations
- Updating records with UPDATE
- Safe update practices
- Deleting data with DELETE
- TRUNCATE vs DELETE
- Basic SELECT queries
- Selecting specific columns
- WHERE clause for filtering
- Comparison operators (=, !=, <, >, <=, >=)
- Logical operators (AND, OR, NOT)
- IN and BETWEEN operators

Projects built:
- Populate employee database with data
- Create product catalog with 1000+ items
- Build customer records system
- Inventory management data setup

Practice: Write 100 INSERT, UPDATE, DELETE queries

Querying Data - SELECT Mastery

Topics:
- ORDER BY for sorting results
- LIMIT for pagination
- DISTINCT for unique values
- Aliases with AS
- String functions: CONCAT, LENGTH, UPPER, LOWER
- LIKE operator for pattern matching
- Wildcards (% and _)
- Regular expressions in MySQL
- Date functions: NOW, CURDATE, DATE_FORMAT
- Mathematical functions: SUM, AVG, COUNT, MIN, MAX
- GROUP BY for aggregation
- HAVING clause for group filtering

Projects built:
- Sales reporting queries
- Customer analytics dashboard queries
- Product search functionality
- Date-based report generation

Practice: Master 150 SELECT query variations

Assessment: Phase 1 check: a timed SELECT gauntlet on a fresh dataset, plus a mixed review of the whole phase

---

PHASE 2: DESIGN AND JOINS (MONTHS 3-4, WEEKS 9-16)

Where amateurs are separated from professionals: relationships, joins without fear, subqueries, and schema design as a deliberate craft.

Months 3-4: Relationships to Real Design (Weeks 9-16)

Database Relationships

Topics:
- Understanding table relationships
- One-to-One relationships
- One-to-Many relationships
- Many-to-Many relationships
- Foreign keys and referential integrity
- CASCADE options (DELETE, UPDATE)
- Creating foreign key constraints
- Database normalization introduction
- First Normal Form (1NF)
- Second Normal Form (2NF)

```
        - Third Normal Form (3NF)
        - Denormalization when and why
      Projects built:
        - Design blog database with posts and comments
        - E-commerce database with orders and products
        - School database with students and courses
        - Social media database schema
      Practice: Design 20 relational database schemas
    Mastering Joins
      Topics:
        - Understanding JOIN operations
        - INNER JOIN deep dive
        - LEFT JOIN (LEFT OUTER JOIN)
        - RIGHT JOIN (RIGHT OUTER JOIN)
        - CROSS JOIN
        - Self joins for hierarchical data
        - Multiple table joins
        - Using aliases in joins
        - Natural joins
        - Join performance considerations
        - UNION operations
        - INTERSECT and EXCEPT alternatives
      Projects built:
        - Employee-Department-Salary reports
        - Order-Customer-Product analysis
        - Multi-table inventory system
        - Hierarchical organization chart
      Practice: Write 100 complex JOIN queries
    Subqueries and Set Operations
      Topics:
        - Understanding subqueries
        - Subqueries in SELECT clause
        - Subqueries in FROM clause
        - Subqueries in WHERE clause
        - Correlated vs non-correlated subqueries
        - EXISTS and NOT EXISTS
        - ANY, ALL, SOME operators
        - Subqueries vs JOINs performance
        - Common Table Expressions (CTEs)
        - Recursive CTEs
        - UNION and UNION ALL
        - Combining multiple result sets
      Projects built:
        - Complex reporting with subqueries
        - Hierarchical data queries with CTEs
        - Performance comparison study
        - PHASE 1 CAPSTONE: Complete Database Application
      Assessment: Phase 1 comprehensive SQL exam
    Schema Design Done Properly
      Topics:
        - Database design methodology
        - Requirements gathering
        - Conceptual design with ER diagrams
        - Logical design and normalization
        - Physical design considerations
        - Denormalization strategies
        - Partitioning tables
        - Vertical vs horizontal partitioning
        - Sharding concepts
        - Archive strategies
        - Multi-tenant database design
        - Temporal database design
        - Database refactoring
        - Migration strategies
      Projects built:
        - Design multi-tenant SaaS database
        - Implement table partitioning
        - Create temporal data system
        - Database migration project
      Practice: Design 10 complex database systems
      Assessment: Phase 2 exam: design and defend a schema live from a brief, plus mixed
      questions from phase 1

------------------------------------------------------------
PHASE 3: THE PROGRAMMABLE DATABASE (MONTHS 5-6, WEEKS 17-24)
------------------------------------------------------------
MySQL beyond queries: views and indexes, stored procedures, triggers, transactions argued
about properly, and Python integration.
```

Months 5-6: Views to Python (Weeks 17-24)
  Views & Indexes
    Topics:
      - Creating and using views
      - Updatable vs non-updatable views
      - View algorithms (MERGE, TEMPTABLE)
      - Security with views
      - Materialized views alternatives
      - Understanding indexes
      - B-Tree vs Hash indexes
      - Creating indexes strategically
      - Composite indexes
      - Unique indexes
      - Full-text indexes
      - Spatial indexes
      - Index hints
      - Analyzing index usage
    Projects built:
      - Create view layer for application
      - Index optimization for slow queries
      - Full-text search implementation
      - Performance improvement study
    Practice: Create 50 views and optimize with indexes
  Stored Procedures
    Topics:
      - Introduction to stored procedures
      - Creating procedures with CREATE PROCEDURE
      - IN, OUT, INOUT parameters
      - Variables and data types
      - Control flow: IF, CASE, LOOP, WHILE
      - REPEAT and LEAVE statements
      - Error handling with DECLARE HANDLER
      - Cursors for row-by-row processing
      - Nested procedures
      - Procedure security and permissions
      - Debugging stored procedures
      - Best practices for procedures
      - Performance implications
      - Version control for database code
      - Functions and triggers: the essentials, and when not to use them
    Projects built:
      - Order processing system procedures
      - Automated reporting procedures
      - Data validation procedures
      - Batch processing system
    Practice: Write 40 stored procedures
  Transactions & Concurrency
    Topics:
      - ACID properties explained
      - Starting transactions: BEGIN, START TRANSACTION
      - COMMIT and ROLLBACK
      - Savepoints in transactions
      - Autocommit mode
      - Isolation levels: READ UNCOMMITTED, READ COMMITTED
      - REPEATABLE READ, SERIALIZABLE
      - Dirty reads, non-repeatable reads, phantom reads
      - Locking mechanisms in MySQL
      - Table locks vs row locks
      - Deadlock detection and resolution
      - Optimistic vs pessimistic locking
      - Lock wait timeouts
      - Transaction best practices
    Projects built:
      - Banking transaction system
      - Inventory management with concurrency
      - Booking system with lock handling
      - Deadlock simulation and resolution
    Practice: Implement 20 transactional scenarios
  MySQL with Python
    Topics:
      - Python MySQL connectors
      - PyMySQL vs mysql-connector-python
      - Establishing connections
      - Executing queries from Python
      - Prepared statements
      - Fetching results
      - Connection pooling
      - ORM with SQLAlchemy
      - Database migrations with Alembic

```
        - Async database operations
        - Error handling in Python
        - Transaction management
        - Building data access layer
        - Best practices for Python-MySQL
      Projects built:
        - Python database application
        - REST API with MySQL backend
        - Data migration script
        - ORM-based application
      Practice: Build 10 Python-MySQL applications
      Assessment: Phase 3 exam: a procedure-plus-transaction task built live, with cumulative
      questions from phases 1-2

------------------------------------------------------------
PHASE 4: THE PRODUCTION MONTH (MONTH 7, WEEKS 25-28)
------------------------------------------------------------
What teams need before they trust you with data: security, backups, performance tuning with
EXPLAIN, and the capstone exam.

  Month 7: Production Essentials and the Finale (Weeks 25-28)
    Users, Security and Backups
      Topics:
        - MySQL security model
        - User account management
        - CREATE USER and ALTER USER
        - Password policies and validation
        - Authentication plugins
        - Privileges and GRANT system
        - Database, table, column privileges
        - Stored routine privileges
        - Proxy users and roles
        - SSL/TLS configuration
        - Encrypted connections
        - Transparent data encryption
        - Audit logging
        - Security best practices
        - Backups with mysqldump and the recovery drill you run before you need it
      Projects built:
        - Implement role-based access control
        - SSL setup for production
        - Security audit of database
        - User management system
      Practice: Configure 20 different security scenarios
    Performance Tuning with EXPLAIN
      Topics:
        - Performance tuning methodology
        - Identifying bottlenecks
        - EXPLAIN and query analysis
        - EXPLAIN FORMAT=JSON
        - Visual EXPLAIN in Workbench
        - Slow query log analysis
        - Performance Schema introduction
        - Key performance metrics
        - Buffer pool tuning
        - Query cache (deprecated but historical)
        - Connection pool tuning
        - Thread configuration
        - I/O optimization
        - Operating system tuning
        - Storage engines in one honest lesson: InnoDB and when anything else matters
      Projects built:
        - Performance baseline establishment
        - Slow query optimization
        - Server tuning project
        - Performance monitoring setup
      Practice: Optimize 50 slow queries
    Capstone Build
      Topics:
        - The capstone: a complete database product for a real domain
        - Schema designed and reviewed before data
        - The query set: hard questions your database answers fast
        - Procedures and transactions where they earn their place
        - Security, backup and an EXPLAIN-verified performance pass
        - Documentation another developer could onboard from
      Projects built:
        - The capstone: designed, secured, tuned, Python-connected and documented
      Practice: Daily capstone progress; schema frozen by day 3
    Demo Day and Final Exam
```

Topics:
- Presenting the capstone: schema, queries, decisions, measurements
- A full-course spiral review before the exam
- Where each path leads: data analytics, full-stack, or the dedicated DBA road
Projects built:
- Demo day presentation delivered to the batch
Assessment: FINAL EXAM: a practical database build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: real datasets queried daily, from first SELECT to timed gauntlets
- Phase 2: multi-table schemas designed from briefs, with ER diagrams and constraint sets
- Phase 3: views, procedures and transaction-safe operations on realistic systems, then Python integration
- Phase 4: a secured, tuned, documented capstone defended at demo day

Total projects: 15+ database builds and query projects, crowned by a production-shaped capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- SQL fluency: DDL, DML and SELECT mastery including joins, subqueries and set operations
- Design: ER modeling, normalization and constraints practiced on real domains
- Programmable MySQL: views, indexes, stored procedures, triggers and transactions
- Integration: MySQL driven from Python applications
- Production habits: users and privileges, backups with recovery drills, EXPLAIN-driven tuning

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, querying and designing along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 3-4 hours of query and design practice between classes
Review: Every set reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, for the deadlock at midnight
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Data analytics: SQL here is the entry gate, our data analysis course continues the road
- Full-stack development: every backend needs exactly this data layer
- Backend engineering: transactions and tuning learned here compound there
- The DBA path: administration as a profession, entered with real fundamentals
- Interview rounds: SQL questions answered from understanding, not memory

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Growing roles: Rs 4-6 LPA (Junior Database Developer)
Experienced roles: Rs 6-12 LPA (Database Administrator); Rs 10-20 LPA (Senior DBA); Rs 15-35+ LPA (Database Architect)
Freelance: Rs 1,500-5,000/hour
International: $60k-130k USD based on location and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course only about writing SQL queries?
A: No, and that is the point. Queries are phase 1; the course continues into schema design as a craft, the programmable database, views, stored procedures, triggers and transactions, Python integration, and the production habits of security, backups and EXPLAIN-driven performance tuning. What it deliberately does not staple on is the full DBA career track, replication clusters and multi-cloud failover, which is its own profession and is named

honestly as a next path.

Q: How long does it take to learn MySQL properly?
A: Seven months at 2 live classes a week plus 3 to 4 hours of practice covers the developer 95%: SQL fluency, design, the programmable database and production basics. The course keeps an eighth month in hand for learners who need more repetition, no rushing and no penalty. Twelve-month database courses are stretched; two-week ones are demos.

Q: How am I assessed? Is the certificate just for attending?
A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new topics with earlier ones, each phase ends with an exam, and the course ends with a final exam, a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Do I need programming experience to start?
A: No. SQL is taught from the first statement and is one of the friendliest languages to begin with. The Python integration weeks in phase 3 are guided and assume nothing; students who already code simply move faster there.

Q: Which MySQL version and tools do you teach?
A: Current MySQL 8 with MySQL Workbench, all free Community editions, plus the command line where professionals actually live. Everything taught transfers to MariaDB and, concepts-wise, to PostgreSQL; SQL fluency is portable by design.

Q: How does this help with placements and interviews?
A: SQL rounds are near-universal in data and software hiring, and they test exactly what this course drills: joins under pressure, subqueries, aggregation edge cases, schema design judgment and transaction reasoning. The monthly mixed reviews keep older material interview-fresh, and the capstone gives you a designed, tuned, documented database to talk through.

Q: Can I evaluate the teaching first?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Bring a query from your own work or coursework; the demo is best on your real problem. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups where your actual SQL gets read and corrected in real time, never pre-recorded videos. Database judgment is built under feedback; a video cannot see your execution plan.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/mysql-database-complete-masterclass-college
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 103 OF 137: MYSQL FOR TEENS: SQL FROM FIRST QUERY TO REAL APPS
==========

Course link: https://learn.modernagecoders.com/courses/mysql-mastery-for-teens
Subject area: Databases
Designed for: Teens
Who can join: Absolute beginner, ages 13 to 18
Course length: 5-6 months (20-24 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

MySQL course for teens aged 13-18: live online classes from first SQL query to database design, real projects and Python-connected apps in 5-6 months.

In one line: Every app a teenager uses is a database wearing a nice shirt. Five months from first query to designing and connecting real databases of their own.

ABOUT THE PROGRAMME

Behind Instagram, Spotify and every game leaderboard sits the same quiet skill: SQL, the

language of data. This live online course teaches it properly to students aged 13 to 18, from the first SELECT to real database design, in five honest months. SQL is one of the highest-value skills per hour of learning in all of computing: the language is compact, the concepts are learnable at 13, and the skill stays current for decades. The arc is practical from the first week: students query and build real databases about things they care about, music libraries, game inventories, school clubs, then learn the query toolkit (functions, aggregation, subqueries), real design (relationships, joins, ER diagrams, constraints), the power features professionals lean on (views, stored procedures, transactions, indexes), and finish by securing a database and connecting it to Python, where data becomes an actual application. The capstone is a complete, well-designed, connected database project defended at demo day. The pace is honest: five months at two live classes a week plus two to three hours of practice, with a sixth month in hand for students who need more time. Homework after every class, monthly mixed reviews that reach back deliberately, and a final exam that decides the certificate, with a free retest after revision. Real queries, real design, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Honest length: SQL fundamentals to real design in 5 months, not a stretched year
- Teen-real datasets: music, games, clubs and leaderboards, not employee-salary tables from 1998
- Design is taught, not skipped: ER diagrams, normalization and constraints, the part that separates users from designers
- The professional features that matter: views, procedures, transactions and indexes, each earning its place
- Ends connected: MySQL driven from Python, so the database becomes an application
- Real assessment: weekly homework, monthly mixed reviews, and a final exam that decides the certificate, with focused revision and a free retest

THE LEARNING PATH, STEP BY STEP

Month 1: SQL foundations (Weeks 1-4): databases demystified, tables created, data inserted, updated and queried safely
Month 2: Query power (Weeks 5-8): functions, dates, aggregation and subqueries, the everyday toolkit
Month 3: Real design (Weeks 9-12): relationships, joins, ER diagrams, normalization and constraints
Month 4: Power features (Weeks 13-16): views, stored procedures, transactions and indexes
Month 5: Apps and the exam (Weeks 17-20): security and backups, MySQL from Python, the capstone and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. SQL is taught from the first statement; the Python in month 5 is guided
Computer skills: Comfortable typing and installing software with guidance
Equipment: A computer with a stable internet connection; MySQL and every tool used are free
Age: 13 to 18. Batches are grouped by age and pace
Time commitment: 2 live classes plus 2-3 hours of practice a week

WHO THIS COURSE IS FOR

Data curious: Teens who wonder how apps remember everything, and want to build the memory themselves
Future developers: Students heading for web, app or game development, where the data layer is always waiting
Analysts in the making: Teens drawn to spreadsheets and statistics, ready for the real tool behind them
Complete beginners: Students with zero coding background: SQL is one of the friendliest first languages there is
Portfolio builders: Teens who want a genuinely uncommon skill on a school profile

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

MONTH 1: SQL FOUNDATIONS

Databases demystified and the four verbs of data: CREATE, INSERT, SELECT, UPDATE and DELETE, used safely from the start.

Introduction to Databases & MySQL Setup
Topics:
- What is data and why do we need databases?
- Real-world database examples teens use daily (Instagram, TikTok, Spotify)
- Files vs Databases: Why databases win
- Types of databases: Relational vs NoSQL
- Introduction to MySQL: History and why it's popular

- MySQL in the real world: Facebook, Twitter, YouTube
- Installing MySQL on Windows/Mac/Linux
- MySQL Workbench installation and tour
- Command line basics for MySQL
- Creating your first database
- Understanding servers, clients, and connections
- Database terminology: tables, rows, columns, records

Projects built:
- Install and configure MySQL environment
- Create 'MyLife' database for personal data
- Design a simple contact list database

Practice: Daily: 30 min terminal practice, 1 hour MySQL basics

SQL Basics: Creating and Managing Databases

Topics:
- CREATE DATABASE statement
- USE, SHOW, and DROP commands
- Creating tables with CREATE TABLE
- Data types: INT, VARCHAR, TEXT, DATE, BOOLEAN
- Understanding NULL and NOT NULL
- Primary keys and their importance
- AUTO_INCREMENT for automatic IDs
- Altering tables: ADD, MODIFY, DROP columns
- Renaming tables and columns
- DESCRIBE and SHOW COLUMNS commands
- Comments in SQL code
- SQL naming conventions and best practices

Projects built:
- School database with students and classes
- Gaming stats tracker database
- Music library database structure

Practice: Create 10 different database schemas for teen apps

Data Manipulation: INSERT, SELECT Basics

Topics:
- INSERT INTO: Adding single records
- INSERT multiple records at once
- Understanding SQL syntax and semicolons
- SELECT statement fundamentals
- SELECT specific columns vs SELECT *
- WHERE clause for filtering data
- Comparison operators: =, !=, <, >, <=, >=
- AND, OR, NOT logical operators
- IN and NOT IN operators
- BETWEEN for range queries
- LIKE operator and wildcards (%, _)
- ORDER BY: Sorting results ASC and DESC

Projects built:
- Populate school database with student data
- Create and query a movie ratings database
- Build a sports team roster system

Practice: Write 50 different SELECT queries with various filters

UPDATE, DELETE, and Data Integrity

Topics:
- UPDATE statement: Modifying existing data
- Using WHERE with UPDATE (critical importance!)
- Updating multiple columns
- Using CASE in UPDATE statements
- DELETE statement: Removing records
- Safe deletion practices
- TRUNCATE vs DELETE vs DROP
- Understanding transactions basics
- Data integrity and consistency
- Handling NULL values in queries
- IS NULL and IS NOT NULL
- Default values and their uses

Projects built:
- Grade management system with updates
- Inventory system with stock updates
- Social media profile manager

Practice: Perform 30 UPDATE and DELETE operations safely

Assessment: Month 1 check: build and query a small database live, plus a take-home mixing everything from month 1

MONTH 2: QUERY POWER

The everyday toolkit: string and numeric functions, dates, aggregation with GROUP BY, and

first subqueries.

MySQL Functions: String and Numeric
Topics:
- String functions: CONCAT, SUBSTRING, LENGTH
- UPPER, LOWER, TRIM, LTRIM, RTRIM
- REPLACE and string manipulation
- REVERSE, REPEAT, and SPACE
- Numeric functions: ROUND, CEIL, FLOOR
- ABS, MOD, POWER, SQRT
- RAND() for random numbers
- Mathematical operations in queries
- FORMAT for number formatting
- Working with CHAR_LENGTH vs LENGTH
- CAST and CONVERT for type conversion
- Combining multiple functions
Projects built:
- Username generator with string functions
- Grade calculator with numeric functions
- Data cleaning toolkit
Practice: Master 40 different MySQL functions

Date/Time Functions and Operations
Topics:
- Date and time data types: DATE, TIME, DATETIME, TIMESTAMP
- CURDATE(), CURTIME(), NOW()
- Date formatting with DATE_FORMAT
- Extracting parts: YEAR, MONTH, DAY, HOUR, MINUTE
- DAYNAME, MONTHNAME for readable output
- Date arithmetic: DATE_ADD, DATE_SUB
- DATEDIFF for calculating differences
- Working with time zones
- UNIX_TIMESTAMP and FROM_UNIXTIME
- Age calculations from birthdates
- Scheduling and recurring events concepts
- Best practices for storing dates
Projects built:
- Birthday reminder system
- Event scheduling database
- Time tracking application
Practice: Build 20 queries involving date/time calculations

Aggregate Functions and Grouping
Topics:
- COUNT: Counting records and values
- COUNT(*) vs COUNT(column) vs COUNT(DISTINCT)
- SUM for totaling numeric values
- AVG for calculating averages
- MIN and MAX for finding extremes
- GROUP BY clause explained
- Grouping by single and multiple columns
- HAVING clause for filtering groups
- HAVING vs WHERE: Key differences
- GROUP_CONCAT for combining grouped values
- Using aliases with AS
- Complex grouping scenarios
Projects built:
- Class grade analytics system
- Sales reporting database
- Game high scores leaderboard
Practice: Create 30 reports using aggregate functions

Subqueries and Nested Queries
Topics:
- Understanding subqueries concept
- Subqueries in WHERE clause
- Subqueries returning single values
- Subqueries returning multiple values
- IN, NOT IN with subqueries
- EXISTS and NOT EXISTS operators
- Subqueries in SELECT clause
- Subqueries in FROM clause (derived tables)
- Correlated vs non-correlated subqueries
- Performance considerations
- ANY, ALL, SOME operators
- Nested subqueries (multiple levels)
Projects built:
- Advanced student ranking system
- Product recommendation queries

- Social media analytics dashboard
Practice: Write 25 complex queries with subqueries
Assessment: Month 2 check: an aggregation-and-subquery challenge, with quiz questions reaching back to month 1

MONTH 3: REAL DATABASE DESIGN

The part that separates users from designers: relationships, joins, ER diagrams, normalization and constraints.

Table Relationships and JOINS
Topics:
- Understanding relationships: Why we need multiple tables
- One-to-One relationships explained
- One-to-Many relationships (most common)
- Many-to-Many relationships and junction tables
- Foreign keys: Linking tables together
- FOREIGN KEY constraints
- Referential integrity and CASCADE options
- Introduction to JOINs
- INNER JOIN: Matching records in both tables
- LEFT JOIN: Include all from left table
- RIGHT JOIN: Include all from right table
- JOIN with multiple tables
Projects built:
- School system with students, teachers, courses
- E-commerce with customers, orders, products
- Music app with artists, albums, songs
Practice: Create 20 different JOIN queries

Advanced JOINS and Set Operations
Topics:
- Self JOINs: Joining table to itself
- CROSS JOIN and Cartesian products
- Natural JOIN (and why to avoid it)
- Using aliases in JOINs
- Multiple JOIN conditions
- JOINs with aggregate functions
- UNION: Combining result sets
- UNION vs UNION ALL
- INTERSECT concept (workarounds in MySQL)
- EXCEPT/MINUS concept
- Complex JOIN scenarios
- Optimizing JOIN performance
Projects built:
- Employee hierarchy system
- Tournament bracket system
- Multi-store inventory system
Practice: Master complex JOIN operations with 30 exercises

Database Design & Normalization
Topics:
- Database design process overview
- Identifying entities and attributes
- Entity-Relationship (ER) diagrams
- Crow's foot notation
- Primary keys: Natural vs Surrogate
- Composite keys
- Database normalization: Why it matters
- First Normal Form (1NF): Eliminating repeating groups
- Second Normal Form (2NF): Removing partial dependencies
- Third Normal Form (3NF): Removing transitive dependencies
- Denormalization: When and why
- Common design patterns
Projects built:
- Design Instagram-like database from scratch
- Create online learning platform schema
- Design multiplayer game database
Practice: Normalize 10 poorly designed databases

Constraints and Data Validation
Topics:
- PRIMARY KEY constraint deep dive
- UNIQUE constraint for preventing duplicates
- NOT NULL constraint
- DEFAULT values and when to use them
- CHECK constraints for data validation
- FOREIGN KEY constraints and relationships

- ON DELETE CASCADE, SET NULL, RESTRICT
- ON UPDATE CASCADE
- Adding constraints to existing tables
- Dropping and modifying constraints
- Constraint naming conventions
- Data validation best practices

Projects built:
- User registration system with validations
- Banking system with integrity constraints
- Inventory system with business rules

Practice: Implement 20 different constraint scenarios

Assessment: Month 3 check: design a small schema live from a real-world brief, plus mixed review of months 1-2

MONTH 4: POWER FEATURES

What professionals actually lean on: views, stored procedures, transactions with ACID honesty, and indexes that make queries fly.

Views and Virtual Tables

Topics:
- What are views and why use them?
- Creating views with CREATE VIEW
- Simple vs complex views
- Views for data security
- Views for simplifying complex queries
- Updatable views and restrictions
- WITH CHECK OPTION for data integrity
- Modifying views with ALTER VIEW
- Dropping views
- Views vs tables performance
- Materialized views concept
- Best practices for using views

Projects built:
- Create security layer with views
- Report generation system using views
- Multi-tenant system with view-based isolation

Practice: Build 15 different views for various use cases

Stored Procedures and Functions

Topics:
- Introduction to stored procedures
- Benefits: security, performance, reusability
- CREATE PROCEDURE syntax
- Parameters: IN, OUT, INOUT
- Variables and DECLARE statement
- Control flow: IF-ELSE, CASE
- Loops: WHILE, REPEAT, LOOP
- Cursors for row-by-row processing
- Error handling with HANDLERS
- Stored functions vs procedures
- CREATE FUNCTION syntax
- Debugging stored procedures

Projects built:
- Banking transaction procedures
- Automated report generation system
- Game scoring calculation procedures

Practice: Create 20 stored procedures for common operations

Transactions and Concurrency

Topics:
- ACID properties explained for teens
- Understanding transactions importance
- START TRANSACTION, COMMIT, ROLLBACK
- Autocommit mode
- Savepoints in transactions
- Isolation levels: READ UNCOMMITTED to SERIALIZABLE
- Dirty reads, phantom reads, non-repeatable reads
- Locking mechanisms in MySQL
- Table locks vs row locks
- Deadlocks: detection and prevention
- Optimistic vs pessimistic locking
- Transaction best practices

Projects built:
- Banking system with safe transactions
- Ticket booking system with concurrency handling
- Inventory management with locking

Practice: Handle 20 different transaction scenarios

  Indexing and Query Optimization
    Topics:
      - What are indexes? Phone book analogy
      - How indexes speed up queries
      - Types of indexes: B-Tree, Hash, Full-text
      - Creating indexes with CREATE INDEX
      - Primary key and unique indexes
      - Composite (multi-column) indexes
      - Index selectivity and cardinality
      - EXPLAIN statement for query analysis
      - Reading execution plans
      - Identifying slow queries
      - Query optimization techniques
      - When NOT to use indexes
    Projects built:
      - Optimize slow social media queries
      - Create indexing strategy for e-commerce
      - Performance tuning workshop
    Practice: Optimize 25 slow queries with proper indexing
    Assessment: Month 4 check: speed up a slow query live with an index and explain why it
    worked, plus mixed review

MONTH 5: REAL APPS AND THE FINAL EXAM

Securing the database, backing it up, driving it from Python, and the capstone defended at
demo day.

  Users, Security and Backups
    Topics:
      - MySQL user account management
      - CREATE USER and DROP USER
      - Understanding privileges system
      - GRANT and REVOKE statements
      - Database-level vs table-level privileges
      - Column-level and procedure privileges
      - Roles in MySQL 8.0+
      - Password policies and expiration
      - SSL/TLS connections
      - Security best practices
      - Preventing SQL injection
      - Prepared statements for security
      - Backups with mysqldump: the five-minute habit that saves projects
    Projects built:
      - Multi-user application security setup
      - Role-based access control system
      - Secure API database layer
    Practice: Configure security for 10 different scenarios

  MySQL from Python: Data Becomes an App
    Topics:
      - Connecting to MySQL from Python
      - MySQL connectors and drivers
      - Connection pooling importance
      - Prepared statements in applications
      - ORM concepts (Object-Relational Mapping)
      - Building APIs with MySQL backend
      - Handling database errors in code
      - Async database operations
      - Database abstraction layers
      - Testing database code
      - Migration scripts
      - Database versioning
    Projects built:
      - Python MySQL admin tool
      - RESTful API with MySQL
      - Database migration system
    Practice: Build 10 applications using MySQL

  Capstone Build
    Topics:
      - Choosing the capstone: a database product for something real, a club, a collection, a game
      - Designing first: ER diagram and constraints reviewed before any data
      - Building the schema, seeding realistic data
      - The query set: the ten questions your database can answer
      - A small Python interface: menu-driven or web-lite
      - Documentation another student could use
    Projects built:

        - The capstone: a designed, documented, Python-connected database project
      Practice: Daily capstone progress; schema frozen by day 3

    Demo Day and Final Exam
      Topics:
        - Presenting the capstone: schema, queries, interface, decisions
        - A full-course spiral review before the exam
        - Where this leads: Python, data science and full-stack roads
      Projects built:
        - Demo day presentation delivered to the batch and parents
      Assessment: FINAL EXAM: a practical database build plus a written paper with questions
      mixed from every month. Passing earns the certificate; a student who falls short gets a
      focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Month 1-2: personal databases queried daily: music libraries, game inventories, club
    registers
  - Month 3: a properly designed multi-table schema with an ER diagram
  - Month 4: views, procedures and transactions added to real projects
  - Month 5: a secured, backed-up, Python-connected capstone defended at demo day

  Total projects: 12+ real databases and query projects, crowned by a connected capstone

SKILLS THE STUDENT WILL HAVE AT THE END

  - SQL fluency: SELECT to subqueries, written confidently and safely
  - Aggregation and analysis: GROUP BY thinking that carries into data science
  - Database design: relationships, ER diagrams, normalization and constraints
  - Professional features: views, stored procedures, transactions, indexes
  - Operations basics: users, privileges and backups
  - Integration: MySQL driven from Python, database to application

HOW A TYPICAL WEEK IS SPLIT

  Live classes: 2 live one-hour classes per week, querying along with the instructor
  Homework: Homework after every class, and a mixed-review assignment at the end of every month
    so earlier skills stay sharp
  Practice: 2-3 hours of query and design practice between classes
  Review: Every submission reviewed with written feedback; recurring issues reopened in class

SUPPORT INCLUDED

  Doubt support: WhatsApp doubt support between classes, for the JOIN that refuses to join
  Progress updates: Regular progress notes to parents: what was built, what comes next, and
    where encouragement helps

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short
    and there is focused revision and a free retest: the certificate certifies knowledge, not
    attendance

WHERE THIS CAN LEAD

  - Python for Teens: the natural pairing, already begun in month 5
  - Full-stack web development: every backend needs exactly this data layer
  - Data science: aggregation thinking here becomes pandas thinking there
  - App development: real apps stand on real schemas
  - School and college projects that suddenly have proper data behind them

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: Why should a teenager learn SQL and MySQL?
  A: Because it is the highest-value-per-hour skill in computing: a compact language, learnable
  at 13, running underneath every app they use, and still current decades from now. SQL turns a
  teen from someone who uses apps into someone who understands how apps remember, and it
  quietly powers every later path: web development, data science, app building and analytics.

  Q: How long does it take a teen to learn MySQL properly?
  A: Five months at 2 live classes a week plus 2 to 3 hours of practice: foundations, the query
  toolkit, real database design, professional features, and a Python-connected capstone. The
  course keeps a sixth month in hand for students who need more repetition, no rushing and no
  penalty. First real queries run in class one.

  Q: How is my teen assessed? Is the certificate just for attending?
  A: No. Homework follows every class, each month ends with a mixed assignment that

deliberately revisits earlier topics, and the course ends with a final exam: a practical database build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: What will my teen actually build?
A: A dozen real databases: music libraries, game inventories and club registers in the early months, then a properly designed multi-table schema with an ER diagram, and finally the capstone, a secured, documented database connected to a small Python interface, presented at demo day. Real data about things they care about, from week one.

Q: Does this course cover connecting databases to apps and websites?
A: Yes, that is the destination: month 5 drives MySQL from Python, turning the database into an application, and the capstone requires it.

Q: Is any paid software required?
A: No. MySQL Community Server, MySQL Workbench and Python are all free, and a standard computer runs everything comfortably. SQL is famously light on hardware.

Q: Is there a free trial class?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Your teen will run real queries inside the demo itself. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small interactive groups with a real instructor, never pre-recorded videos. SQL is learned by writing queries and being corrected in the moment, a video cannot see your WHERE clause.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/mysql-mastery-for-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 104 OF 137: NATIONAL 5 AND HIGHER MATHS COURSE ONLINE (SCOTLAND)

==========

Course link: https://learn.modernagecoders.com/courses/national-5-and-higher-maths-course
Subject area: Scottish Mathematics
Designed for: High school
Who can join: S3 to S6 learners and adults in Scotland taking National 5 or Higher Mathematics
Course length: About 6 months per level: National 5, then Higher (12 months for both)
Time needed: 2 live classes/week + 3-4 hours of practice, rising to full papers before the exams
Certificate: Course-completion certificate from Modern Age Coders

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

National 5 and Higher Maths online for Scottish learners: C847 75 and C847 76 taught live, from algebra and trigonometry to calculus, ending in timed papers.

In one line: The two Scottish maths courses most learners need, taught in order: National 5 for the algebra, geometry and trigonometry, then Higher for functions, vectors and calculus.

ABOUT THE PROGRAMME

National 5 Mathematics (course code C847 75) and Higher Mathematics (C847 76) are the Scottish courses that open the door to most further study. Their course specifications, published by Qualifications Scotland on sqa.org.uk, give each a notional 160 hours of learning and two question papers. National 5 has a 40-mark non-calculator paper of 1 hour and a 50-mark paper of 1 hour 30 minutes. Higher has a 55-mark non-calculator paper of 1 hour 15 minutes and a 65-mark paper of 1 hour 30 minutes. Both courses are graded A to D on the total mark. This course teaches them in sequence. The National 5 half covers the numerical, algebraic, geometric, trigonometric and statistical skills the specification lists, from surds and indices to the sine and cosine rules. The Higher half builds the algebraic and trigonometric, geometric and calculus skills Higher adds: polynomials, logarithms and exponentials, the wave function, the straight line and the circle, vectors, recurrence relations, differentiation and integration. Learners who already hold National 5 start at the Higher half after a diagnostic. Each level ends with a month of full papers under real timing, marked against published marking instructions. Reasoning is taught throughout,

because both specifications expect learners to interpret a situation and explain a solution, not just calculate. Classes are live and small, and they run at times that suit Scottish school days and exam leave.

WHAT MAKES THIS COURSE DIFFERENT

- Built from the National 5 and Higher Mathematics course specifications, version 3.0, valid from session 2023 to 24
- Non-calculator Paper 1 skills practised every week at both levels
- Reasoning taught explicitly: interpreting a situation and explaining a solution
- National 5 then Higher in one sequence, or Higher alone after a diagnostic
- A month of full papers per level, marked against published marking instructions
- Scottish terms and structure throughout: S3 to S6, prelims and the national exam diet

THE LEARNING PATH, STEP BY STEP

Phase 1: National 5 skills: numerical, algebraic, geometric, trigonometric and statistical skills, with reasoning throughout
Phase 2: National 5 papers: prelim-style and full papers under timing, marked and reviewed
Phase 3: Higher skills: algebraic and trigonometric, geometric and calculus skills, with reasoning throughout
Phase 4: Higher papers: prelim-style and full papers under timing, marked and reviewed

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: For National 5: the fourth curriculum level or National 4 Mathematics, as the course specification recommends. For Higher: National 5 Mathematics or equivalent. A diagnostic in the first week places every learner
Exam entry: Learners are entered by their school or college. Adults and private candidates arrange entry through a centre; we teach to the specification and paper style
Equipment: A scientific calculator, ruler, protractor, compasses, and a device with stable internet for live classes
Timing: Start National 5 in S3 or early S4, and Higher early in S5, so the papers month comes before the prelims. Later joiners get a catch-up plan

WHO THIS COURSE IS FOR

S3 s4 learners: S3 and S4 learners working towards National 5 Mathematics
S5 s6 learners: S5 and S6 learners taking Higher Mathematics
Learners needing support: Learners whose school class moves faster than they can follow and who want steady teaching alongside it
Adult learners: Adults in Scotland returning to National 5 or Higher Mathematics for college, university or work
International learners: Learners abroad following the Scottish curriculum who need live teaching across time zones

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: NATIONAL 5 SKILLS (MONTHS 1-5, WEEKS 1-20)

The skills sampled in National 5 Mathematics, taught strand by strand with reasoning questions in every month.

Month 1: Numerical Skills (Weeks 1-4)

Weeks 1-2: Diagnostic, Surds and Indices

Topics:
- A diagnostic paper to place each learner
- Simplifying surds and rationalising denominators
- Indices, including negative and fractional indices
- Scientific notation
- Rounding to significant figures
- Setting out non-calculator working clearly

Projects built:
- Skills checklist started: every National 5 skill coloured by confidence

Practice: 15 non-calculator number questions a week

Weeks 3-4: Percentages, Fractions and Appreciation

Topics:
- Reverse percentages
- Appreciation and depreciation, including compound interest
- Operations and combinations of operations on fractions
- Mixed numbers without a calculator
- Choosing a calculator method for repeated percentage change
- Reasoning: explaining whether an offer is worth it

Projects built:
- Money file: interest and depreciation problems solved and explained

Practice: 12 percentage and fraction problems a week

```
    Assessment: Month 1 check: a 30-minute non-calculator paper

Month 2: Algebraic Skills, Part One (Weeks 5-8)
  Weeks 5-6: Expressions and Factorising
    Topics:
      - Expanding brackets, including ax(bx + c) and products of binomials
      - Factorising: common factors, difference of two squares, trinomials
      - Completing the square
      - Algebraic fractions: simplifying and the four operations
      - Changing the subject of a formula
      - Checking by substitution
    Projects built:
      - Algebra method sheet: one worked example of every technique
    Practice: 20 algebra questions a week
  Weeks 7-8: Straight Lines and Equations
    Topics:
      - Gradient from two points and from a graph
      - The equation of a straight line in different forms
      - Linear equations and inequations
      - Simultaneous equations by graph and by algebra
      - Constructing equations from text
      - Reasoning: interpreting the solution in context
    Projects built:
      - Equation bank: simultaneous equation problems built from real contexts
    Practice: 15 equation problems a week
    Assessment: Month 2 check: a 40-minute algebra paper

Month 3: Algebraic Skills, Part Two (Weeks 9-12)
  Weeks 9-10: Quadratics
    Topics:
      - Quadratic functions and their graphs
      - Turning points and axes of symmetry
      - Solving quadratic equations by factorising and by formula
      - The discriminant and the nature of the roots
      - Sketching a parabola from its equation
      - Problems that lead to a quadratic
    Projects built:
      - Parabola file: six quadratics sketched with roots and turning points
    Practice: 15 quadratic problems a week
  Weeks 11-12: Functions and Mixed Algebra
    Topics:
      - Functional notation, f(x)
      - Finding a quadratic function from its graph
      - Reading information from graphs
      - Mixed algebra questions from past papers
      - Non-calculator algebra under time
      - Error log review
    Projects built:
      - Skills checklist updated after the first three months
    Practice: Two 25-minute mixed algebra sets a week
    Assessment: Month 3 check: a 45-minute mixed paper

Month 4: Geometric Skills (Weeks 13-16)
  Weeks 13-14: Shape, Area and Volume
    Topics:
      - Arc length and sector area
      - Volumes of prisms, cylinders, cones, pyramids and spheres
      - Pythagoras' theorem in complex situations, including 3D
      - Properties of quadrilaterals, triangles, polygons and circles
      - Similarity and scale factors for area and volume
      - Rounding volume and area answers sensibly
    Projects built:
      - Geometry sheet: every formula and property with a worked example
    Practice: 15 geometry problems a week
  Weeks 15-16: Vectors and Coordinates
    Topics:
      - Two-dimensional vectors: adding, subtracting and multiplying by a scalar
      - Vector pathways in shapes
      - Three-dimensional coordinates from a diagram
      - Magnitude of a vector
      - Reasoning: explaining a geometric result
      - Past paper geometry questions
    Projects built:
      - Vector diagram set: five shapes with pathways labelled
    Practice: 12 vector and coordinate problems a week
    Assessment: Month 4 check: a 45-minute geometry paper

Month 5: Trigonometric and Statistical Skills (Weeks 17-20)
  Weeks 17-18: Trigonometry
```

```
      Topics:
        - Area of a triangle using trigonometry
        - The sine rule and the cosine rule
        - Bearings in problems
        - Graphs of trigonometric functions and their translations
        - Solving trigonometric equations
        - Trigonometric identities used at National 5
      Projects built:
        - Triangle casebook: when to use each rule, with a solved example
      Practice: 15 trigonometry problems a week
    Weeks 19-20: Statistics
      Topics:
        - Comparing data sets using statistics
        - Standard deviation and the semi-interquartile range
        - Scatter graphs and the line of fit
        - Writing a comparison in context
        - Mixed questions across all National 5 skills
        - Planning the papers month
      Projects built:
        - Skills checklist completed for National 5
      Practice: 12 statistics questions a week plus a mixed set
      Assessment: Month 5 check: a full mixed paper at National 5

---------------------------------------------------------------
PHASE 2: NATIONAL 5 PAPERS (MONTH 6, WEEKS 21-24)
---------------------------------------------------------------
Full National 5 papers under real timing: Paper 1 without a calculator, 40 marks in 1 hour,
and Paper 2, 50 marks in 1 hour 30 minutes.

  Month 6: Full National 5 Papers (Weeks 21-24)
    Weeks 21-22: Prelim-Style Papers
      Topics:
        - A full Paper 1 and Paper 2 under timing
        - Marking against published marking instructions
        - Sorting lost marks by skill and by habit
        - Re-teaching the weakest skills
        - Pacing Paper 1 without a calculator
        - Showing working that earns process marks
      Projects built:
        - Paper tracker started for National 5
      Practice: One full paper pair a week
    Weeks 23-24: Final Papers and Next Steps
      Topics:
        - A final paper pair under timing
        - Review of every lost mark
        - Exam-day routine and equipment
        - Bridging tasks for Higher: functions and further algebra
        - Planning Higher for those continuing
        - Revision plan up to the exam
      Projects built:
        - Completed National 5 paper tracker and error log
      Practice: Targeted sets from the error log
      Assessment: National 5 paper review, one to one

---------------------------------------------------------------
PHASE 3: HIGHER SKILLS (MONTHS 7-11, WEEKS 25-44)
---------------------------------------------------------------
The skills sampled in Higher Mathematics: algebraic and trigonometric, geometric and calculus
skills, with reasoning throughout. Learners joining at Higher start here after a diagnostic.

  Month 7: Algebra and Functions (Weeks 25-28)
    Weeks 25-26: Polynomials and Quadratic Theory
      Topics:
        - A diagnostic for learners joining at Higher
        - Factorising cubics and quartics with the factor theorem
        - Solving polynomial equations
        - Quadratic inequalities and the discriminant
        - Tangency and intersection of lines and curves
        - Completing the square at Higher depth
      Projects built:
        - Higher skills checklist started
      Practice: 15 polynomial and quadratic problems a week
    Weeks 27-28: Functions and Graphs
      Topics:
        - Composite and inverse functions
        - Domain and range
        - Related graphs and transformations
        - Exponential and logarithmic functions
        - Laws of logarithms and solving equations
```

- Sketching the inverse of a logarithmic or exponential function
Projects built:
- Graph file: transformations and log graphs annotated
Practice: 15 function and logarithm problems a week
Assessment: Month 7 check: a 45-minute Higher algebra paper

Month 8: Trigonometry at Higher (Weeks 29-32)
Weeks 29-30: Radians, Formulae and Identities
Topics:
- Radians and exact values
- Addition formulae and double-angle formulae
- Trigonometric identities
- Solving trigonometric equations in degrees and radians
- Graphs of trigonometric functions
- Non-calculator trigonometry
Projects built:
- Trigonometry sheet: exact values, formulae and identities
Practice: 15 trigonometry problems a week
Weeks 31-32: The Wave Function
Topics:
- Converting a cos x + b sin x to k cos(x - a) and related forms
- Maximum and minimum values
- Solving equations using the wave function
- Sketching the resulting graph
- Applications in context
- Past paper wave function questions
Projects built:
- Wave function casebook: four forms with a worked example each
Practice: 10 wave function problems a week
Assessment: Month 8 check: a 45-minute trigonometry paper

Month 9: Straight Line, Circle, Vectors and Recurrence (Weeks 33-36)
Weeks 33-34: The Straight Line and the Circle
Topics:
- Gradients, perpendicular lines and collinearity
- Medians, altitudes and perpendicular bisectors
- Equations of a circle
- Intersection of a line and a circle, and tangency
- Intersection of two circles, and tangency problems
- Coordinate geometry reasoning
Projects built:
- Coordinate geometry sheet with a solved past-paper style problem
Practice: 15 straight line and circle problems a week
Weeks 35-36: Vectors and Recurrence Relations
Topics:
- Vectors in three dimensions
- The scalar product and the angle between vectors
- Collinearity and dividing a line in a ratio
- Recurrence relations and their limits
- Modelling a situation with a recurrence relation
- Interpreting a limit in context
Projects built:
- Recurrence model: a real situation modelled and its limit explained
Practice: 12 vector and recurrence problems a week
Assessment: Month 9 check: a 45-minute geometry paper

Month 10: Differentiation (Weeks 37-40)
Weeks 37-38: Differentiating Functions
Topics:
- Differentiating algebraic functions
- Differentiating k sin x and k cos x
- The chain rule
- Equations of tangents
- Increasing and decreasing functions
- Stationary points and their nature
Projects built:
- Calculus method sheet: one worked example per rule
Practice: 20 differentiation problems a week
Weeks 39-40: Applying Differentiation
Topics:
- Sketching curves from stationary points
- Greatest and least values on an interval
- Optimisation problems in context
- Rates of change
- Reasoning: justifying a maximum or minimum
- Past paper optimisation questions
Projects built:
- Optimisation file: three context problems solved and explained
Practice: 12 applied differentiation problems a week

Assessment: Month 10 check: a 45-minute calculus paper

Month 11: Integration (Weeks 41-44)
Weeks 41-42: Integrating Functions
Topics:
- Integrating algebraic functions
- Integrating sine and cosine
- Integrating with the chain rule in reverse for linear inner functions
- Definite integrals
- Solving simple differential equations
- Checking an integral by differentiating

Projects built:
- Calculus method sheet extended with integration

Practice: 20 integration problems a week
Weeks 43-44: Areas and Mixed Calculus
Topics:
- Area under a curve
- Area between two curves
- Mixed calculus questions
- Mixed Higher questions across all skills
- Non-calculator calculus under time
- Planning the papers month

Projects built:
- Higher skills checklist completed

Practice: Two 30-minute mixed sets a week
Assessment: Month 11 check: a full mixed paper at Higher

PHASE 4: HIGHER PAPERS (MONTH 12, WEEKS 45-48)

Full Higher papers under real timing: Paper 1 without a calculator, 55 marks in 1 hour 15 minutes, and Paper 2, 65 marks in 1 hour 30 minutes.

Month 12: Full Higher Papers (Weeks 45-48)
Weeks 45-46: Prelim-Style Papers
Topics:
- A full Paper 1 and Paper 2 under timing
- Marking against published marking instructions
- Sorting lost marks by skill and by habit
- Re-teaching the weakest skills
- Pacing the non-calculator paper
- Process marks and clear communication

Projects built:
- Paper tracker started for Higher

Practice: One full paper pair a week
Weeks 47-48: Final Papers and Next Steps
Topics:
- A final paper pair under timing
- Review of every lost mark
- Exam-day routine and equipment
- A calm plan for exam leave
- Looking ahead to Advanced Higher for those continuing
- Revision plan up to the exam

Projects built:
- Completed Higher paper tracker and error log

Practice: Targeted sets from the error log
Assessment: Higher paper review, one to one

PROJECTS THE STUDENT BUILDS AND KEEPS

- Skills checklists for National 5 and Higher, kept up to date
- Method sheets for algebra, trigonometry, geometry and calculus
- A money file, an equation bank and a parabola file
- A triangle casebook and a wave function casebook
- A recurrence model and an optimisation file
- Paper trackers and error logs for both levels

Total projects: A complete revision kit for National 5 and Higher: checklists, method sheets, casebooks and marked paper trackers

SKILLS THE STUDENT WILL HAVE AT THE END

- National 5 numerical, algebraic, geometric, trigonometric and statistical skills
- Higher algebraic and trigonometric skills, including logarithms and the wave function
- Higher geometric skills: straight line, circle and vectors
- Differentiation and integration with applications
- Reasoning: interpreting situations and explaining solutions
- Paper technique with and without a calculator

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week; paper months include longer timed sittings
Practice: 3 to 4 hours a week of practice, rising before prelims and exams
Review: Practice marked with comments; papers marked against published marking instructions

SUPPORT INCLUDED

Doubt support: WhatsApp support between classes, with worked solutions for stuck questions
Progress updates: A monthly progress note to learners and parents with check scores and next steps

CERTIFICATION DETAIL

Completion: Course-completion certificate from Modern Age Coders

WHERE THIS CAN LEAD

- Advanced Higher Mathematics and other Advanced Higher maths courses after Higher
- College and university courses that ask for National 5 or Higher Mathematics; check each one's exact requirement
- Our statistics and programming courses, which put the mathematics to work

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is in the National 5 Maths exam?
A: Two question papers. Paper 1 is non-calculator, 40 marks in 1 hour. Paper 2 is 50 marks in 1 hour 30 minutes.

Q: What is in the Higher Maths exam?
A: Two question papers. Paper 1 is non-calculator, 55 marks in 1 hour 15 minutes. Paper 2 is 65 marks in 1 hour 30 minutes. The course is graded A to D on the total mark.

Q: Can I do Higher Maths without National 5?
A: The Higher specification says entry is at the discretion of the centre and that candidates should have achieved National 5 Mathematics or equivalent. If you have not, we recommend the National 5 half of this course first; the diagnostic shows where you stand.

Q: How hard is the jump from National 5 to Higher Maths?
A: Higher adds new ideas rather than more of the same: logarithms, the wave function, vectors in three dimensions, recurrence relations, and differentiation and integration. Many learners find the pace the hardest part, which is why the course teaches Higher strand by strand with a check every month.

Q: Do you teach Applications of Mathematics?
A: This course follows National 5 and Higher Mathematics. Applications of Mathematics is a separate course with different content. Ask in the free demo and we will tell you honestly whether one-to-one support can cover it. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: When should I start?
A: For National 5, in S3 or early S4. For Higher, early in S5. Starting early means the full papers month comes before the prelims rather than after them.

Q: Can adults take National 5 or Higher Maths online?
A: The teaching can be fully online. The exams are sat in person through a centre, which makes the entry. We teach to the course specification and paper style.

Q: Is this the same as GCSE or A level maths?
A: No. National 5 and Higher are Scottish qualifications with their own specifications and papers. We also run GCSE and A Level courses for learners in England, Wales and Northern Ireland.

Q: How are the classes run?
A: Live online classes with a real teacher, in a group of 15 to 20 learners at a similar level or one to one. Current fees for your region are shown in the pricing section of this page.

Q: Can I try a lesson first?
A: Yes. Book a free demo class. The teacher works through some National 5 or Higher questions with you, finds where you are, and suggests where to start and whether group or one-to-one teaching fits better. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/national-5-

and-higher-maths-course
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 105 OF 137: OLYMPIAD MATHEMATICS: IOQM, INMO AND AMC PREPARATION
==========

Course link: https://learn.modernagecoders.com/courses/olympiad-mathematics-premium-course
Subject area: Olympiad Mathematics
Designed for: Teens
Who can join: Students roughly aged 12 to 18 who are strong at school maths and want genuine olympiad problem solving
Course length: 12 months (48 weeks), joinable any month
Time needed: 2 live classes/week + weekly problem sets and timed mocks
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one
  live class a week, four a month; one to one outside India two live classes a week, eight a
  month. Each class is one hour.

WHAT THIS COURSE IS

  Live online olympiad mathematics for serious students: number theory, algebra, combinatorics
  and geometry taught as proof-based problem solving, aimed at IOQM and INMO in India and the
  AMC and AIME internationally.

  In one line: The four olympiad pillars taught as real proof-based problem solving, for students who have outgrown school maths.

ABOUT THE PROGRAMME

  Olympiad mathematics is a different subject from school mathematics wearing the same name.
  School maths asks you to apply a method you were just shown. Olympiad maths hands you a
  problem you have never seen, gives you no method, and asks you to construct an argument that
  is airtight. Students who top their class routinely struggle at first, and that is normal,
  because nothing in the school syllabus trains this. This course teaches the four pillars that
  every olympiad is built on: number theory, algebra, combinatorics and geometry. Each is
  taught for genuine understanding and then drilled on real past problems, because olympiad
  ability is built by attempting hard problems and failing productively, not by watching
  solutions. It is aimed at both the Indian and international ladders. In India the path runs
  through IOQM, the Indian Olympiad Qualifier in Mathematics, on to INMO and the training camp
  beyond it. Internationally students target the AMC 10 and AMC 12, then AIME. The formats
  differ, and we prepare for each specifically, but the underlying skill is the same: reading a
  problem properly, finding the idea, and writing an argument that survives scrutiny. Proof is
  taught explicitly from the start. Most students arrive able to find an answer but unable to
  justify it, and at INMO level the justification is the entire mark. We fix that early,
  because a correct answer with a hand-waving argument scores close to nothing.

WHAT MAKES THIS COURSE DIFFERENT

  - All four pillars taught properly: number theory, algebra, combinatorics and geometry, not
    just contest tricks
  - Proof writing taught explicitly from the start, because at INMO level the argument is the
    mark, not the answer
  - Prepared for both ladders: IOQM and INMO in India, AMC 10, AMC 12 and AIME internationally
  - Built on real past problems, since olympiad ability comes from productive failure on hard
    problems
  - Students are taught to sit with a problem instead of giving up after two minutes, which is
    the actual skill
  - Live, small batches where solutions are critiqued line by line for rigour, not just marked
    right or wrong

THE LEARNING PATH, STEP BY STEP

  Phase 1: Proof technique and number theory, the natural entry point into olympiad thinking
  Phase 2: Algebra and combinatorics, including inequalities and counting arguments
  Phase 3: Geometry, then full contest simulation for IOQM, INMO and the AMC or AIME

WHAT A STUDENT NEEDS BEFORE STARTING

  Title: What Your Child Needs to Start

WHO THIS COURSE IS FOR

  Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PROOF TECHNIQUE AND NUMBER THEORY

How to argue rigorously, then the pillar that best rewards it.

Months 1 to 2: Proof and Foundations (Weeks 1-8)

What Counts as a Proof

Topics:

- The difference between checking cases and proving a statement
- Direct proof, and writing it so every step is justified
- Proof by contradiction
- Proof by contrapositive, and when it is easier
- Reading a written solution critically and finding the gap

Projects built:

- Write five full proofs and have every line challenged in class

Practice: Prove supplied statements and rewrite them after critique

Induction and Invariants

Topics:

- Mathematical induction, including strong induction
- Common induction errors that void the proof
- Invariants and monovariants, and the problems that hide them
- Extremal arguments: taking the largest or smallest case
- Recognising which technique a problem is inviting

Projects built:

- Solve a set of invariant problems and articulate the invariant explicitly in each

Practice: 20 induction and invariant problems

Assessment: A proof-writing assessment marked on rigour, not just the final answer

Number Theory

Topics:

- Divisibility, GCD and the Euclidean algorithm
- Primes, factorisation and the fundamental theorem of arithmetic
- Modular arithmetic and congruences
- Fermat's little theorem and Euler's theorem
- Diophantine equations at olympiad level

Projects built:

- Solve a graded ladder of number theory problems from IOQM level upward

Practice: 40 number theory problems across the difficulty range

Assessment: Timed number theory paper under contest conditions

---

PHASE 2: ALGEBRA AND COMBINATORICS

The two pillars that most often decide an IOQM or AMC score.

Months 3 to 5: Algebra (Weeks 9-20)

Polynomials and Functional Equations

Topics:

- Polynomial roots, factorisation and Vieta's formulas
- Symmetric functions and their uses
- Functional equations and the standard substitution strategies
- Recognising the structure hidden in an ugly expression
- Writing algebraic arguments cleanly

Projects built:

- Work a set of Vieta and functional equation problems from past papers

Practice: 30 algebra problems building in difficulty

Inequalities

Topics:

- AM-GM and its many disguises
- Cauchy-Schwarz
- Rearrangement and Chebyshev
- Normalisation and substitution tactics
- Knowing when to stop searching for a slicker method

Projects built:

- Prove a graded set of inequalities, each with a fully justified argument

Practice: 30 inequality problems

Assessment: Timed algebra paper under contest conditions

Combinatorics

Topics:

- Counting properly: permutations, combinations and overcounting
- The pigeonhole principle and its surprisingly deep uses
- Inclusion and exclusion
- Bijections and counting the same set two ways
- Introductory graph theory and games

Projects built:

- Solve a counting problem two different ways and reconcile the results

Practice: 35 combinatorics problems
Assessment: Timed combinatorics paper

PHASE 3: GEOMETRY AND FULL CONTEST SIMULATION

The pillar students most often avoid, then the contests themselves rehearsed properly.

Months 6 to 9: Geometry (Weeks 21-36)

Synthetic Geometry

Topics:

- Angle chasing done systematically rather than hopefully
- Cyclic quadrilaterals and the power of a point
- Similar triangles and ratio arguments
- The classical centres and their properties
- Drawing an accurate diagram, which solves more problems than any theorem

Projects built:

- Solve a ladder of synthetic geometry problems with full written proofs

Practice: 35 geometry problems from contest papers

Advanced Geometry Techniques

Topics:

- Coordinate geometry as a fallback when synthetic stalls
- Trigonometric methods in geometry
- Homothety and transformations
- Choosing between synthetic, coordinate and trigonometric attacks
- Recognising when a problem is resisting the method rather than you

Projects built:

- Attack the same hard problem three ways and compare which was cleanest

Practice: 25 problems requiring a deliberate choice of method
Assessment: Timed geometry paper

Mixed Problem Solving

Topics:

- Problems that do not announce their topic
- Deciding quickly which pillar a problem belongs to
- Working when the first idea fails, and the second
- Managing frustration, which is a genuine competition skill
- Building a personal record of techniques that worked

Projects built:

- Weekly mixed problem sets with no topic labels

Practice: Mixed sets under time

Months 10 to 12: Contest Simulation (Weeks 37-48)

IOQM and AMC Format Training

Topics:

- The IOQM format and how to pace it
- AMC 10 and AMC 12 pacing, and the scoring implications of guessing
- Answer-extraction techniques for short-answer formats
- Knowing when to abandon a question and return
- Accuracy under time, where most marks are actually lost

Projects built:

- Full timed IOQM and AMC papers every week

Practice: Weekly full papers with question-by-question review

INMO and AIME Level, and Peak Form

Topics:

- The step up to full written proofs at INMO level
- AIME format and its particular demands
- Writing a complete solution under time pressure
- Partial credit: how to secure marks on a problem you cannot finish
- Contest-day routine, sleep and nerves

Projects built:

- Full INMO-style and AIME-style papers, marked on rigour

Practice: Continuous full-paper cycle with detailed feedback
Assessment: Final full mock contest, marked as an examiner would, with an honest readiness assessment

HOW A TYPICAL WEEK IS SPLIT

Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Readiness for IOQM, INMO, AMC and AIME

- Proof-writing ability that makes university mathematics far less of a shock
- A distinctive strength in competitive university applications
- A foundation for research mathematics, physics or theoretical computer science
- Problem-solving stamina that transfers to every quantitative field

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Which contests does this prepare for?
A: In India the path runs through IOQM, the Indian Olympiad Qualifier in Mathematics, on to INMO and the training camp beyond it. Internationally we prepare students for AMC 10 and AMC 12 and then AIME. The formats differ and we train for each specifically, but the underlying skill is identical, so the same course serves both ladders.

Q: My child tops the class in maths. Will olympiad maths be easy for them?
A: Almost certainly not at first, and that is completely normal. School maths asks you to apply a method you were just taught. Olympiad maths gives you an unfamiliar problem, no method, and expects a rigorous argument. Excellent school students often struggle for the first month. The ones who keep going usually become very strong, because the difficulty is trainable.

Q: Why do you teach proof writing so early?
A: Because at INMO level the argument is the mark. Most students arrive able to find the right answer but unable to justify it, and a correct answer with hand-waving reasoning scores close to nothing. Proof technique is taught in the first two months and then marked for rigour all year, which is also exactly what makes university mathematics manageable later.

Q: How long does the course run?
A: Twelve months across 48 weeks, with two live classes a week plus problem sets and timed mocks. That length is deliberate: olympiad ability is built by attempting hard problems over months, not by a crash course. You can join in any month and we place your child at the right point after the free demo class. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Is geometry really necessary?
A: Yes, and it is the pillar students most often try to avoid. We spend four months on it, starting with systematic angle chasing rather than hoping, and we teach coordinate and trigonometric methods as deliberate fallbacks.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/olympiad-mathematics-premium-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 106 OF 137: ONTARIO COMPUTER SCIENCE COMPANION: ICS3U AND ICS4U (PYTHON AND JAVA)
==========

Course link: https://learn.modernagecoders.com/courses/ics3u-ics4u-computer-science-companion-course
Subject area: Ontario High School Computer Science
Designed for: High school
Who can join: Ontario students in Grades 11 and 12 (roughly ages 15 to 18) taking ICS3U or ICS4U at school, in Python or Java
Course length: 6 months (24 weeks) per plan, joinable any month and paced to your school course
Time needed: 2 live classes/week + short practice tasks between classes
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment (this is not an Ontario high school credit)

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online ICS3U and ICS4U help in Python or Java, taught alongside your Ontario school course. We teach the skills; we do not grant Ontario high school credits.

In one line: A second teacher for Grade 11 and Grade 12 computer science: problem solving first, then clean code in Python or Java, in step with your school course.

ABOUT THE PROGRAMME

ICS3U (Introduction to Computer Science, Grade 11, University Preparation) and ICS4U (Computer Science, Grade 12, University Preparation) are Ontario's university-preparation computer science courses, described in the ministry's Computer Studies curriculum for Grades 10 to 12 (2008). The ministry describes ICS3U this way: "Students will design software independently and as part of a team, using industry-standard programming tools and applying the software development life-cycle model". For ICS4U it says: "Students will use modular design principles to create complex and fully documented programs, according to industry standards". In each Grade 11 and 12 course, the curriculum document says, the first strand is Programming Concepts and Skills and the second is Software Development, and the fourth strand "deals with the role of computers in society". ICS3U has no prerequisite; ICS4U requires "Introduction to Computer Science, Grade 11, University Preparation", which is ICS3U. This course runs alongside the student's own ICS3U or ICS4U class. We teach in whichever language the school uses, Python or Java, and we follow the order of the student's school course outline, so what is learned here is used in class the same week. Every topic starts with the problem before the syntax: reading the task, breaking it into steps, writing the algorithm in plain words, then turning it into code and testing it. From there we build data types, control structures, lists and arrays, modular programming with functions or methods, string and file handling, sorting and searching, and for ICS4U students, recursion and object-oriented design. Assignments and the final project or exam are the student's own work: we explain, question and review, and the student writes the code. Please read this plainly: Modern Age Coders teaches alongside school and does not grant Ontario high school credits. ICS3U and ICS4U are each worth one credit, and families who need the credit take the course through their school or a ministry-inspected credit provider. What we add is understanding, practice and confidence, in live classes with a real teacher.

WHAT MAKES THIS COURSE DIFFERENT

- Built around Ontario's ICS3U and ICS4U, following the student's own school course outline week by week
- Python or Java, whichever the school uses, with the same concepts shown side by side when a student meets both
- Problem solving and algorithm design before syntax: plain-word steps, tracing tables and test cases come first
- Honest about credits: we teach alongside school and do not grant Ontario high school credits
- Assignments stay the student's own work: we explain and review, we never write school code for them
- Exam review technique taught directly: tracing code by hand, writing code on paper and planning answers under time

THE LEARNING PATH, STEP BY STEP

Phase 1: Problem solving and the ICS3U core: algorithms in plain words, data types, input and output, decisions and loops, in Python or Java
Phase 2: Completing ICS3U: lists and arrays, modular programming, string and file handling, sorting and searching, the software development life cycle and exam review
Phase 3: The ICS4U toolkit: recursion, object-oriented design, algorithm efficiency, large documented programs, the final project and exam review

WHAT A STUDENT NEEDS BEFORE STARTING

School course: Currently taking, or about to take, ICS3U or ICS4U at an Ontario school. ICS4U students should have completed ICS3U
Coding level: No coding experience needed for ICS3U students. ICS4U students should know the ICS3U basics, which we review quickly
Age: Roughly 15 to 18 (Grades 11 and 12)
Equipment: A computer with stable internet and the school's Python or Java editor installed
Timing: Weekdays or weekends, flexible: choose your batch, or for a very personalised plan choose 1-on-1 at your convenience, weekend mornings or weekdays

WHO THIS COURSE IS FOR

Grade 11 students: Grade 11 students meeting programming for the first time who want to understand it properly
Grade 12 students: Grade 12 students facing recursion, object-oriented design and a larger final project
Students who feel behind: Students who missed a unit or lost confidence and want to catch up calmly
Students aiming higher: Students who are doing well and want to write cleaner, better designed code
Future cs students: Students thinking about university computer science who want a strong, honest foundation

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PROBLEM SOLVING AND THE ICS3U CORE (MONTHS 1-2, WEEKS 1-8)

How a programmer thinks before typing, then the building blocks every ICS3U program uses, in Python or Java.

  Month 1: Algorithms, Data Types and Input and Output (Weeks 1-4)
    Weeks 1-2: Problem Solving Before Syntax
      Topics:
        - A starting conversation and short diagnostic: the school, the language, the course outline and the next assignment
        - Reading a programming task and listing the inputs, outputs and rules
        - Writing an algorithm as numbered plain-word steps, then as pseudocode
        - Flowcharts for decisions and repetition, kept simple
        - Tracing an algorithm by hand with a trace table
        - Setting up the school's editor or IDE at home so practice matches class
      Projects built:
        - Three small problems solved twice: once as plain-word steps, once as working code
      Practice: Write the algorithm for each practice task before opening the editor, and keep both
    Weeks 3-4: Data Types, Variables and Expressions
      Topics:
        - Integers, decimals, text and true or false values, in Python and in Java
        - Static types in Java and dynamic types in Python, and what each means for errors
        - Arithmetic, integer division and remainder, and the rounding surprises they cause
        - Reading input and converting it to the right type
        - Formatting output neatly, as school assignments usually require
        - Naming variables so a marker can follow the code
      Projects built:
        - A unit converter or bill splitter with clear prompts, checked input and formatted output
      Practice: Short daily tracing questions: predict the output, then run the code to check
      Assessment: Month 1 check: a short written trace plus one small program, reviewed with written feedback

  Month 2: Control Structures (Weeks 5-8)
    Weeks 5-6: Decisions
      Topics:
        - if, else if and else in Python and Java
        - Boolean expressions with and, or and not
        - Nested decisions and when to flatten them
        - Testing every branch: choosing test cases on purpose
        - Validating input so a program does not crash on a bad value
        - Comparing text correctly, including the Java equals trap
      Projects built:
        - A rules-based program, such as a ticket pricing or grading-rules calculator, with a written test plan
      Practice: For every decision program, list the test cases first and record the expected results
    Weeks 7-8: Loops
      Topics:
        - Counted loops and condition-controlled loops
        - Running totals, counters and finding the largest or smallest value
        - Loops that validate input until it is correct
        - Nested loops for tables and patterns
        - Off-by-one errors and how a trace table catches them
        - Choosing a for loop or a while loop for the job
      Projects built:
        - A number guessing game or a times-table quiz with scoring and replay
      Practice: A loop tracing set: write the value of every variable after each pass
      Assessment: Phase 1 assessment: a written tracing section and a short program, marked with comments

---
PHASE 2: COMPLETING ICS3U (MONTHS 3-4, WEEKS 9-16)

The data structures, modular design, files and classic algorithms that larger ICS3U programs need, plus the software development habits the course describes.

  Month 3: Lists, Arrays and Modular Programming (Weeks 9-12)
    Weeks 9-10: Lists and Arrays
      Topics:
        - Python lists, Java arrays and Java ArrayList, and how they differ
        - Looping through a collection by index and by value
        - Parallel lists and when a better structure is needed
        - Searching a list for a value, counting matches and filtering
        - Two-dimensional lists and arrays for grids and tables
        - Index errors: why they happen and how to prevent them
      Projects built:
        - A class marks analyzer: store scores, report the average, the highest and lowest, and a

simple bar display
Practice: Short list and array tasks, each with a trace of the indexes visited
Weeks 11-12: Modular Programming with Functions and Methods
Topics:
- Breaking a program into functions in Python or static methods in Java
- Parameters, return values and why a function should do one job
- Local and global scope, and avoiding hidden shared state
- Writing a short comment or docstring that says what a function does
- Testing one function at a time
- Refactoring a long program into clear modules
Projects built:
- Refactor an earlier project into well-named functions or methods, with a test for each
Practice: For each new function, write its purpose, its inputs and its output before the body
Assessment: Month 3 check: a modular program built from a given plan, reviewed for structure and clarity

Month 4: Strings, Files, Sorting and Searching (Weeks 13-16)
Weeks 13-14: String and File Handling
Topics:
- Strings as sequences: indexing, slicing or substring, and length
- Common string tasks: counting letters, reversing, checking palindromes, simple ciphers
- Splitting a line of text into fields and joining fields back together
- Reading a text file line by line in Python and in Java
- Writing results to a file and handling a missing file safely
- Processing a small data file from start to finish
Projects built:
- A file report tool: read a data file, summarize it and write a tidy results file
Practice: String puzzles and short file tasks, with the expected output written before running
Weeks 15-16: Sorting, Searching and the Development Life Cycle
Topics:
- Linear search and binary search, and when binary search is allowed
- Bubble, selection and insertion sort, traced pass by pass
- Counting comparisons to see why some algorithms are faster
- The software development life cycle: analysis, design, coding, testing and maintenance
- Planning and documenting a larger program, and working on a team project with clear roles
- ICS3U exam review: tracing questions, written code and longer design questions
Projects built:
- A searchable, sortable record list built from a file, with a short design document and test log
Practice: Trace each sort on a new list by hand, then confirm with code
Assessment: Phase 2 assessment: a mixed ICS3U-style review paper plus a program built from a written specification

---
PHASE 3: THE ICS4U TOOLKIT, FINAL PROJECT AND EXAM REVIEW (MONTHS 5-6, WEEKS 17-24)

What we teach for Grade 12: recursion, object-oriented design, efficiency and large, fully documented programs, then the final project and exam review. ICS4U batches begin with a fast review of Phases 1 and 2.

Month 5: Recursion and Object-Oriented Design (Weeks 17-20)
Weeks 17-18: Recursion
Topics:
- Base case and recursive case, explained with small examples
- Tracing recursion with a call stack diagram
- Recursive versions of factorial, sums, string reversal and power
- Recursive binary search and an introduction to merge sort
- When recursion is clearer than a loop, and when it is slower
- Common recursion mistakes: a missing base case and a case that never shrinks
Projects built:
- Write three recursive solutions and their loop versions, and compare them in a short written note
Practice: Draw the call stack for every recursion question before running the code
Weeks 19-20: Classes and Object-Oriented Design
Topics:
- Classes and objects: attributes, methods and constructors in Java and Python
- Encapsulation: private data and public methods
- Designing classes from a description: nouns, responsibilities and relationships
- Lists of objects, and sorting objects by a chosen field
- Inheritance and when to use it, with a simple class diagram
- Documenting classes and methods so another programmer can use them
Projects built:
- An object-oriented library, inventory or game roster program with a class diagram and documentation
Practice: Design the classes on paper first, then build them one method at a time with tests
Assessment: Month 5 check: a recursion trace and a small class design, reviewed with

written feedback

Month 6: Efficiency, the Final Project and Exam Review (Weeks 21-24)
Weeks 21-22: Efficiency and Large Documented Programs
Topics:
- Comparing algorithms by counting steps, with an introduction to big-O language
- Choosing data structures for the job: lists, dictionaries or maps, and sets
- Modular design for larger programs: files, classes and clear interfaces
- Version control basics with Git so work is never lost
- If your school sets a final project or culminating task: reading the brief, scoping it and planning milestones
- Writing user documentation, internal comments and a test log

Projects built:
- A project plan and working first version for the student's own final project, built by the student

Practice: Weekly milestone check-ins: what was built, what was tested, what is next

Weeks 23-24: Exam Review Technique
Topics:
- Making a one-page summary for each topic of the school course
- Tracing questions: working steadily with a trace table under time
- Writing code on paper: structure first, then details, then a quick self-check
- Design and explanation questions: answering exactly what is asked
- Timed review papers in the style of school exams, with careful review afterwards
- Reading the teacher's exam outline so revision matches what will be assessed

Projects built:
- A personal review pack: topic summaries, a mistakes list and two reviewed timed papers

Practice: A timed mixed review each week, with every mistake written up and fixed

Assessment: Final assessment: a timed review paper plus a short live walk-through of the student's own project code

PROJECTS THE STUDENT BUILDS AND KEEPS

- Plain-word algorithms and trace tables kept for every topic
- Unit converter, rules-based calculator and loop games
- Class marks analyzer using lists or arrays
- File report tool and a searchable, sortable record list
- Recursive and loop solutions compared in writing
- An object-oriented program with a class diagram and documentation
- A personal exam review pack

Total projects: A portfolio of small programs, a documented object-oriented program and a review pack, all the student's own work

SKILLS THE STUDENT WILL HAVE AT THE END

- Problem solving and algorithm design before code
- Data types, input and output, decisions and loops in Python or Java
- Lists, arrays and two-dimensional data
- Modular programming with functions or methods
- String and file handling
- Sorting and searching, traced by hand and in code
- Recursion and object-oriented design
- Testing, documentation and exam review technique

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, paced to the student's school course outline

Practice: Short practice tasks after every class, separate from school assignments

Review: Practice work reviewed with written comments, and a monthly check that mixes in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with hints and questions first so the student does the thinking

Progress updates: A monthly progress note to parents with what was covered, what is strong and what to practise next

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. It records real learning and is not an Ontario high school credit. Fall short and there is focused revision and a free retest

WHERE THIS CAN LEAD

- University computer science, software engineering or other programs that use programming
- College programs in programming and computer systems
- Contest programming, such as the Canadian Computing Competition, through our CCC preparation course

- Further Python or Java study in our teen and college courses
- Personal projects, clubs and open-source contributions built on solid fundamentals

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Does this course give my child an Ontario high school credit?
A: No. Modern Age Coders teaches alongside school and does not grant Ontario high school credits. ICS3U and ICS4U are each worth one credit, and a family who needs the credit takes the course through the student's school or a ministry-inspected credit provider. Our course helps the student understand and do well in the school course they are already taking.

Q: Is ICS3U hard?
A: It is often a student's first full programming course, so the early weeks can feel steep. The hard part is usually not the syntax but turning a problem into steps. That is why we teach problem solving and algorithm design before code, use trace tables, and practise little and often. Most of the difficulty eases once a student can plan a program before typing it.

Q: Does the course use Python or Java?
A: Whichever your school uses. Some ICS3U and ICS4U classes use Python, others use Java. We teach in the school's language so the work transfers straight back to class, and when it helps we show the same idea in the other language too.

Q: What do ICS3U and ICS4U cover?
A: The ministry describes ICS3U as a course where students "design software independently and as part of a team, using industry-standard programming tools and applying the software development life-cycle model", and ICS4U as one where students "use modular design principles to create complex and fully documented programs, according to industry standards". In each Grade 11 and 12 course the first strand is Programming Concepts and Skills, the second is Software Development, and the fourth deals with the role of computers in society. Your school's course outline gives the detail for your class.

Q: Does my child need to take ICS3U before ICS4U?
A: Yes. The ministry lists the ICS4U prerequisite as "Introduction to Computer Science, Grade 11, University Preparation", which is ICS3U. ICS3U itself has no prerequisite, so a student can start it without a Grade 10 computer course.

Q: What is the difference between ICS3U and ICS3C?
A: Both are Grade 11 computer courses and neither has a prerequisite. ICS3U is the University Preparation course, Introduction to Computer Science. ICS3C is the College Preparation course, Introduction to Computer Programming, which the ministry describes as introducing "computer programming concepts and practices". This companion course is built around ICS3U and ICS4U.

Q: How does ICS3U connect to university computer science?
A: Some programs name it directly. The University of Waterloo's Computer Science page lists "Grade 11 U Introduction to Computer Science", which is ICS3U, as a recommended course. The required courses on that page are mainly Grade 12 math and English, so a recommended course is an addition, not a replacement. Always check each university's current admission page. We make no admission promises; we help the student build real computer science understanding.

Q: Will you do my child's assignments or final project?
A: No. School work stays the student's own. We explain concepts, ask questions, review code the student has written and suggest how to improve it, and we follow the school's rules on outside help. That is also the only kind of help that lasts into the exam room.

Q: Can you help with exam review?
A: Yes. The course teaches exam technique directly: tracing code by hand, writing code on paper, answering design and explanation questions, and timed review papers with careful review afterwards. We also use the teacher's exam outline so revision matches what will actually be assessed.

Q: Can my child join partway through the school year?
A: Yes, the course is joinable any month. We start from the student's current unit and next assignment, fill gaps from earlier units as needed, and keep pace with the school course from then on. ICS4U students begin with a quick review of the ICS3U ideas they will rely on.

Q: How are the classes run?
A: Live online classes, twice a week, with a real teacher, in a group of 15 to 20 students, a mini batch of up to 6, or 1-on-1. Weekdays or weekends, flexible: choose your batch, or for a very personalised plan choose 1-on-1 at your convenience, weekend mornings or weekdays. Current fees for your region are shown in the pricing section of this page.

Q: Can we try a class first?
A: Yes. Book a free demo class. The teacher looks at where your child is in ICS3U or ICS4U, works through a short problem with them, and suggests whether group, mini batch or 1-on-1 classes fit best. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow,

booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ics3u-ics4u-computer-science-companion-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 107 OF 137: ONTARIO GRADE 12 UNIVERSITY MATH: ADVANCED FUNCTIONS AND CALCULUS AND VECTORS (MHF4U AND MCV4U)
==========

Course link: https://learn.modernagecoders.com/courses/ontario-grade-12-university-math-mhf4u-mcv4u-course
Subject area: Ontario High School Mathematics
Designed for: High school
Who can join: Ontario students in Grade 12 (roughly ages 16 to 18) taking MHF4U, MCV4U or both at school
Course length: 6 months (24 weeks) per plan, joinable any month and paced to your school course
Time needed: 2 live classes/week + short practice sets between classes
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment (this is not an Ontario high school credit)

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online MHF4U and MCV4U help for ages 16 to 18, taught alongside your Ontario school course: functions, trigonometry, logarithms, calculus and vectors. We do not grant Ontario credits.

In one line: A second teacher for Grade 12 university math: functions understood properly, then the calculus and vectors built on them, in step with your school course.

ABOUT THE PROGRAMME

MHF4U and MCV4U are Ontario's Grade 12 university-preparation math courses. In the ministry's course list, MHF4U is titled Advanced Functions and MCV4U is titled Calculus and Vectors, both issued in 2007. The ministry lists the MHF4U prerequisite as "Functions, Grade 11, University Preparation, or Mathematics for College Technology, Grade 12, College Preparation", and it says "The new Advanced Functions course (MHF4U) must be taken prior to or concurrently with Calculus and Vectors (MCV4U)". Both courses also appear in university admission requirements. The University of Waterloo's Computer Science page asks for "Six Grade 12 U and/or M courses including" Advanced Functions and Calculus and Vectors, and the University of Toronto lists MCV4U as a prerequisite for computer science on all three campuses, with MHF4U also listed at Mississauga and Scarborough. This course runs alongside the student's own MHF4U or MCV4U class, and many students take both in the same year. We follow the order of the student's school course outline, so what is learned here is used in class the same week. What our course teaches for MHF4U students: polynomial and rational functions, trigonometric functions in radians with identities and equations, exponential and logarithmic functions, combining functions, and rates of change. What it teaches for MCV4U students: limits and the derivative, the derivative rules, curve sketching and optimization, then vectors, lines and planes. Every topic starts from meaning, with graphs and small examples, before the algebra. Tests and assignments stay the student's own work: we explain, question and review, and the student does the math. Please read this plainly: Modern Age Coders teaches alongside school and does not grant Ontario high school credits. Families who need the credit take the course through their school or a ministry-inspected credit provider. What we add is understanding, practice and confidence, in live classes with a real teacher.

WHAT MAKES THIS COURSE DIFFERENT

- Built around Ontario's MHF4U and MCV4U, following the student's own school course outline week by week
- Taught for students taking one course or both at once, with MHF4U ideas reviewed exactly where calculus needs them
- Meaning before algebra: graphs, tables and small examples come first, then the rules and the proofs
- Honest about credits: we teach alongside school and do not grant Ontario high school credits
- Tests and assignments stay the student's own work: we explain and review, we never do school work for them
- Exam review technique taught directly: full written solutions, communication marks and timed practice papers

THE LEARNING PATH, STEP BY STEP

Phase 1: Advanced Functions, part 1: the function toolkit, polynomial functions, polynomial equations and inequalities, and rational functions

Phase 2: Advanced Functions, part 2: trigonometric functions in radians, identities and equations, exponential and logarithmic functions, combining functions and rates of change
Phase 3: Calculus and Vectors: limits and the derivative, derivative rules and applications, vectors in two and three dimensions, lines and planes, and exam review

WHAT A STUDENT NEEDS BEFORE STARTING

School course: Currently taking, or about to take, MHF4U, MCV4U or both at an Ontario school
Math level: The Grade 11 functions course or equivalent for MHF4U. MCV4U students should have MHF4U done or under way, which we support side by side
Age: Roughly 16 to 18 (Grade 12)
Equipment: A computer with stable internet, a scientific or graphing calculator, and a way to share photos of written work
Timing: Weekdays or weekends, flexible: choose your batch, or for a very personalised plan choose 1-on-1 at your convenience, weekend mornings or weekdays

WHO THIS COURSE IS FOR

Advanced functions students: Grade 12 students in MHF4U who want functions to make sense, not just to pass
Calculus and vectors students: Students in MCV4U meeting derivatives, optimization and three-dimensional vectors
Students taking both: Students taking MHF4U and MCV4U in the same year who want the two courses to fit together
Students who feel behind: Students who missed a unit or lost confidence and want to catch up calmly
Future stem students: Students heading toward computer science, engineering, mathematics or science at university

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: ADVANCED FUNCTIONS, PART 1 (MONTHS 1-2, WEEKS 1-8)

The function toolkit every Grade 12 topic depends on, then polynomial and rational functions in depth.

Month 1: The Function Toolkit and Polynomial Functions (Weeks 1-4)

Weeks 1-2: Starting Point and the Function Toolkit

Topics:
- A starting conversation and short diagnostic: the school, the course outline, the next test and the Grade 11 ideas to refresh
- Function notation, domain and range, and reading a graph carefully
- Transformations of functions: shifts, stretches and reflections
- Inverse functions and what their graphs show
- Factoring review: common factors, trinomials and the difference of squares
- Setting up a clean notebook system for definitions, examples and mistakes

Projects built:
- A one-page function toolkit summary, with a sketch of each parent function and its transformations

Practice: Short daily sets that mix new work with one or two Grade 11 refresher questions

Weeks 3-4: Polynomial Functions

Topics:
- Degree, leading coefficient and end behaviour
- Zeros, their order, and how the graph behaves at each zero
- Sketching a polynomial from its factored form
- Finding an equation from a graph or from given zeros
- Even and odd functions and symmetry
- Finite differences and what they say about degree

Projects built:
- A polynomial sketch set: ten polynomials sketched by hand from factored form, then checked with graphing technology

Practice: For every sketch, write the end behaviour and the behaviour at each zero before drawing

Assessment: Month 1 check: a short written test on functions and polynomial graphs, reviewed with written feedback

Month 2: Polynomial Equations and Rational Functions (Weeks 5-8)

Weeks 5-6: Polynomial Equations and Inequalities

Topics:
- Polynomial long division and synthetic division
- The remainder theorem and the factor theorem
- Factoring cubics and quartics, including sums and differences of cubes
- Solving polynomial equations exactly
- Solving polynomial inequalities with a sign chart and a graph
- Families of polynomials and word problems

Projects built:
- A worked solutions file for factoring and inequalities, with every step justified in words

Practice: Mixed equation and inequality sets, with answers checked by substitution or graph

Weeks 7-8: Rational Functions

Topics:
- Reciprocal functions and their graphs
- Vertical and horizontal asymptotes, and holes
- Sketching rational functions from their factored form
- Solving rational equations and checking for restricted values
- Solving rational inequalities with a sign chart
- Average rate of change on a rational graph

Projects built:
- A rational function sketch set with asymptotes, intercepts and behaviour near each asymptote explained

Practice: For every rational function, list restrictions and asymptotes first, then sketch

Assessment: Phase 1 assessment: a timed test on polynomial and rational functions in the style of a school unit test, marked with comments

---

PHASE 2: ADVANCED FUNCTIONS, PART 2 (MONTHS 3-4, WEEKS 9-16)

Trigonometry in radians, exponential and logarithmic functions, combining functions and rates of change, with MHF4U exam review.

Month 3: Trigonometric Functions, Identities and Equations (Weeks 9-12)

Weeks 9-10: Radians and Trigonometric Functions

Topics:
- Radian measure and converting between radians and degrees
- Special angles and the unit circle, without memorizing tables
- Graphs of sine, cosine and tangent in radians
- Amplitude, period, phase shift and vertical shift
- Modelling periodic situations with sinusoidal functions
- Reciprocal trigonometric functions and their graphs

Projects built:
- A sinusoidal model of a real periodic situation, with the equation built from the data and explained

Practice: Unit circle and transformation sets, each answer checked against a quick sketch

Weeks 11-12: Identities and Trigonometric Equations

Topics:
- Proving identities: working one side, and writing each step clearly
- Compound angle formulas and double angle formulas
- Solving linear and quadratic trigonometric equations in radians
- Finding all solutions in a given interval
- Equivalent trigonometric expressions using related angles
- Common mistakes: dividing by a variable and losing solutions

Projects built:
- An identity proof file: ten identities proved in full, with the strategy for each written first

Practice: One identity proof and two equations every day, with every solution in the interval listed

Assessment: Month 3 check: a written test on trigonometric functions, identities and equations, reviewed with written feedback

Month 4: Exponential and Logarithmic Functions, Combining Functions and Rates of Change (Weeks 13-16)

Weeks 13-14: Exponential and Logarithmic Functions

Topics:
- Exponential functions and their graphs, growth and decay
- The logarithm as the inverse of an exponential function
- Laws of logarithms and when each one applies
- Solving exponential and logarithmic equations, and checking for extraneous roots
- Changing the base of a logarithm
- Applications: compound interest, half-life, sound and pH scales

Projects built:
- An applications pack: four real situations modelled with exponential or logarithmic functions and solved in full

Practice: Mixed log law and equation sets, with each step named

Weeks 15-16: Combining Functions, Rates of Change and MHF4U Review

Topics:
- Sums, differences, products and quotients of functions, and their graphs
- Composition of functions and its domain
- Average rate of change over an interval
- Estimating instantaneous rate of change, the bridge to calculus
- Choosing a function family to model a situation
- MHF4U exam review: full solutions, communication and timed practice

Projects built:
- A personal MHF4U review pack: one-page summaries for each unit, a mistakes list and a

reviewed timed paper
Practice: Trace the meaning of a rate of change on a graph before calculating it
Assessment: Phase 2 assessment: a mixed MHF4U-style review paper, marked with written comments

---

PHASE 3: CALCULUS AND VECTORS, AND EXAM REVIEW (MONTHS 5-6, WEEKS 17-24)

What we teach for MCV4U: limits and the derivative, derivative rules and applications, then vectors, lines and planes, ending with exam review. MCV4U batches begin with a quick review of the MHF4U ideas calculus relies on.

Month 5: Limits, Derivatives and Their Applications (Weeks 17-20)
Weeks 17-18: Rates of Change, Limits and the Derivative
Topics:
- From average rate of change to instantaneous rate of change
- Limits, explained with tables and graphs before algebra
- The derivative from first principles
- The power, constant multiple, sum and difference rules
- The product, quotient and chain rules
- The derivative as a slope and as a rate, in words and in units

Projects built:
- Three derivatives worked from first principles, then confirmed with the rules, with a short written explanation

Practice: Daily derivative sets, each with one question asking what the derivative means in context
Weeks 19-20: Curve Sketching, Optimization and Rates
Topics:
- Increasing and decreasing intervals, and local maxima and minima
- Concavity and points of inflection from the second derivative
- Sketching a curve from its derivatives
- Optimization problems: setting up the function, the domain and the check
- Derivatives of exponential and sinusoidal functions
- Rates of change in motion: position, velocity and acceleration

Projects built:
- An optimization pack: four real problems set up and solved in full, each with a domain check

Practice: For every optimization problem, write the function and its domain before differentiating
Assessment: Month 5 check: a written test on derivatives and their applications, reviewed with written feedback

Month 6: Vectors, Lines and Planes, and Exam Review (Weeks 21-24)
Weeks 21-22: Vectors in Two and Three Dimensions
Topics:
- Geometric vectors: magnitude, direction, addition and scalar multiples
- Cartesian vectors in two and three dimensions
- The dot product, angles between vectors and projections
- The cross product and what it measures
- Applications: force, velocity and work
- Drawing a clear diagram for every vector problem

Projects built:
- A vector applications set with a labelled diagram and a full written solution for each problem

Practice: Short vector sets mixing geometric and Cartesian forms
Weeks 23-24: Lines and Planes, and Exam Review Technique
Topics:
- Equations of lines in two and three dimensions: vector, parametric and symmetric forms
- Equations of planes and their normal vectors
- Intersections of lines and planes, and what each case looks like
- Making a one-page summary for each unit of the school course
- Full written solutions: showing reasoning so communication marks are earned
- Timed review papers in the style of school exams, with careful review afterwards

Projects built:
- A personal MCV4U review pack: unit summaries, a mistakes list and two reviewed timed papers

Practice: A timed mixed review each week, with every mistake written up and fixed
Assessment: Final assessment: a timed review paper plus a short live walk-through of two of the student's own solutions

PROJECTS THE STUDENT BUILDS AND KEEPS

- A function toolkit summary with parent functions and transformations
- Polynomial and rational function sketch sets
- An identity proof file for trigonometry
- Exponential and logarithmic applications pack
- Derivatives from first principles and an optimization pack
- Vector applications with labelled diagrams
- Personal MHF4U and MCV4U review packs

Total projects: A file of worked solutions, sketch sets, proofs and review packs, all the student's own work

SKILLS THE STUDENT WILL HAVE AT THE END

- Functions, transformations and inverses
- Polynomial and rational functions, equations and inequalities
- Trigonometric functions in radians, identities and equations
- Exponential and logarithmic functions and their applications
- Combining functions and rates of change
- Derivatives, curve sketching and optimization
- Vectors, lines and planes in three dimensions
- Full written solutions and exam review technique

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, paced to the student's school course outline
Practice: Short practice sets after every class, separate from school assignments
Review: Practice work reviewed with written comments, and a monthly check that mixes in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with hints and questions first so the student does the thinking
Progress updates: A monthly progress note to parents with what was covered, what is strong and what to practise next

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. It records real learning and is not an Ontario high school credit. Fall short and there is focused revision and a free retest

WHERE THIS CAN LEAD

- University computer science, engineering, mathematics or science programs
- First-year university calculus and linear algebra, on a solid base
- Math contests such as the Euclid, through our Waterloo math contests course
- School computer science such as ICS4U, through our Ontario computer science companion course
- Further study in data science and programming, where functions and calculus are used every day

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Does this course give my child an Ontario high school credit?
A: No. Modern Age Coders teaches alongside school and does not grant Ontario high school credits. A family who needs the MHF4U or MCV4U credit takes the course through the student's school or a ministry-inspected credit provider. Our course helps the student understand and do well in the school course they are already taking.

Q: What are MHF4U and MCV4U?
A: They are Ontario's Grade 12 university-preparation math courses. In the ministry's course list, MHF4U is titled Advanced Functions and MCV4U is titled Calculus and Vectors. Your school's course outline gives the detail for your class.

Q: What do I need before taking MHF4U?
A: The ministry lists the MHF4U prerequisite as "Functions, Grade 11, University Preparation, or Mathematics for College Technology, Grade 12, College Preparation". Functions, Grade 11, University Preparation is MCR3U. In the first two weeks we refresh the Grade 11 ideas that Grade 12 relies on most.

Q: Can my child take MHF4U and MCV4U at the same time?
A: Yes, and many students do. The ministry says "The new Advanced Functions course (MHF4U) must be taken prior to or concurrently with Calculus and Vectors (MCV4U)". When a student takes both at once, we follow both school course outlines and review each MHF4U idea just before calculus needs it.

Q: Is MCV4U hard?
A: It asks for more independence than earlier math courses, and it builds on everything in MHF4U, so a gap in functions shows up quickly in calculus. That is why we teach meaning before algebra, keep a mistakes list, and practise a little every day. Most of the difficulty eases once a student understands what a derivative means rather than only how to compute one.

Q: Do universities ask for MHF4U and MCV4U for computer science?
A: Many do. At the University of Toronto, St. George lists "MCV4U" as a computer science prerequisite, while Mississauga and Scarborough list both "MHF4U" and "MCV4U". Always check each university's current admission page. We make no admission promises; we help the student build real understanding.

Q: Does the course follow my school's order?
A: Yes. Schools order the units differently, so we use the student's own course outline and next test date to set the pace. The plan on this page shows our standard sequence, and each batch follows the order of its students' schools as closely as possible.

Q: Will you do my child's assignments or tests?
A: No. School work stays the student's own. We explain concepts, ask questions, review work the student has written and suggest how to improve it, and we follow the school's rules on outside help. That is also the only kind of help that lasts into the exam room.

Q: Can you help with exam review?
A: Yes. The course teaches exam technique directly: one-page unit summaries, full written solutions that show reasoning, and timed review papers with careful review afterwards. We also use the teacher's exam outline so revision matches what will actually be assessed.

Q: Can my child join partway through the school year?
A: Yes, the course is joinable any month. We start from the student's current unit and next test, fill gaps from earlier units as needed, and keep pace with the school course from then on.

Q: How are the classes run?
A: Live online classes, twice a week, with a real teacher, in a group of 15 to 20 students, a mini batch of up to 6, or 1-on-1. Weekdays or weekends, flexible: choose your batch, or for a very personalised plan choose 1-on-1 at your convenience, weekend mornings or weekdays. Current fees for your region are shown in the pricing section of this page.

Q: Can we try a class first?
A: Yes. Book a free demo class. The teacher looks at where your child is in MHF4U or MCV4U, works through a short problem with them, and suggests whether group, mini batch or 1-on-1 classes fit best. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/ontario-grade-12-university-math-mhf4u-mcv4u-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 108 OF 137: ORANGE DATA MINING COURSE FOR COLLEGE STUDENTS: MACHINE LEARNING AND ANALYTICS
==========

Course link: https://learn.modernagecoders.com/courses/orange-data-mining-machine-learning-course-for-college-students
Subject area: Data Science & Machine Learning
Designed for: College students
Who can join: Beginner to applied practitioner (college students and graduates), no prior machine learning required
Course length: 24 classes (12 weeks · 2 classes/week)
Time needed: 4-6 hours/week recommended
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Master Orange Data Mining in 24 live classes for college students: reproducible workflows, feature engineering, model selection with nested validation, explainability, clustering, text mining, time series and a defended analytics capstone, plus the Python bridge to pandas and scikit-learn.

In one line: Cover a full data mining syllabus on a visual canvas, then defend it: rigorous validation, feature engineering, explainability, clustering, text and time series, and a capstone that reproduces from raw data on someone else's machine.

ABOUT THE PROGRAMME

Data mining appears in almost every computer science, IT, statistics, business analytics and bioinformatics programme, and it is routinely taught as a list of algorithm names to memorise before an exam. This course teaches it as a practice. Orange is free, open source machine learning and data visualisation software developed at the Bioinformatics Laboratory, Faculty of Computer and Information Science, University of Ljubljana, released under the GNU GPL version 3 or later. It was built for teaching machine learning and data mining to university students, and it is used in real research work. You assemble analysis by connecting widgets on a canvas, which removes the syntax overhead and puts the entire course budget into method:

how to prepare data defensibly, how to select a model without fooling yourself, and how to explain what the model did. Across 24 live classes you work through the six default widget sets, Data, Transform, Visualize, Model, Evaluate and Unsupervised, and then into the add-on ecosystem that makes Orange genuinely useful for project work: Text for corpus analysis, Time Series for temporal data, Explain for model interpretation, Associate for rule mining, Geo for spatial data, and Fairness for auditing. You will build classification and regression pipelines, use Preprocess inside the modelling loop so that scaling and imputation are fitted on training folds rather than leaked across them, compare learners under identical cross validation in Test and Score, tune with Parameter Fitter, read ROC, precision-recall and calibration properly, and use Explain Model and Permutation Importance to answer the question a viva always asks: why did it predict that? The course is deliberately not a dead end at the canvas. Orange is written in Python and installable from PyPI as Orange3, and the Python Script widget exposes real Python inside a workflow with the incoming data available in your namespace. Two classes are spent on that bridge, so students moving on to pandas and scikit-learn carry the concepts across rather than starting again. Every class is taught live in a small batch by a working practitioner, the same mentorship-first method Modern Age Coders has used since 2020 to teach 10,000+ students across 25+ countries. You finish with 24 reproducible workflows, a capstone that runs end to end from a raw file, and an exported report you can put in front of an examiner, a recruiter or a project guide.

WHAT MAKES THIS COURSE DIFFERENT

- Built for defensible work, not demos: reproducibility, leakage control and honest validation are graded requirements from Phase 1, not closing remarks.
- Covers a full university data mining syllabus, preprocessing, classification, regression, evaluation, clustering, dimensionality reduction, association rules, text and time series, with a running artefact for each.
- Explainability is a full class, not a mention: Explain Model, Explain Prediction, Permutation Importance and ICE, so you can answer why a prediction happened.
- The Python bridge is taught explicitly: Orange is a Python library, the Python Script widget runs real Python inside the canvas, and the same models exist in scikit-learn.
- Free and open source under the GNU GPL v3 or later, so nothing in your final year project depends on a licence you lose after graduation.
- Capstone standard is reproduction: your workflow must rerun from the raw file in a clean session, on a machine that is not yours.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop running Windows, macOS or Linux capable of installing Orange, with a stable internet connection for live classes and for the server-based image and text embedders

Coding experience: None required. The entire course can be completed without writing code. Students who already know Python get more from Class 22, where the Python Script widget and the Orange to scikit-learn concept map are covered, but nothing earlier depends on it

Maths background: Comfort with school-level statistics, averages, percentages, basic probability and reading a graph. Every further concept, from cross validation to regularisation to silhouette scores, is taught in the course at the point it is needed

Accounts: No account is required. Orange is free and open source software you download and run locally, and the add-ons used in this course install free from inside the application

Mindset: Willingness to be wrong in public: to report that your model lost to the baseline, to state your limitations before someone else finds them, and to rerun your own work from scratch to prove it holds

WHO THIS COURSE IS FOR

Cs and engineering students: Undergraduates with a data mining, machine learning or data warehousing paper who want to actually practise the syllabus rather than memorise algorithm names for an exam

Final year project students: Students building a project, dissertation or research paper who need a defensible methodology and a workflow that reproduces under scrutiny

Non cs students with data: Students in statistics, economics, commerce, psychology, biology, biotechnology and public health who have real data and a real question but no interest in a six week detour through Python syntax

Career switchers and graduates: Graduates preparing for data analyst and junior data science interviews who need the method questions, validation, metrics, leakage and explainability, answered with confidence

Students who will learn python next: Anyone planning to learn pandas and scikit-learn who wants the modelling and evaluation concepts secure first, so only the syntax is new

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: REPRODUCIBLE FOUNDATIONS

Classes 1 to 5. Set up Orange, learn the canvas properly, and build the habits that separate a project that survives a viva from one that falls apart when someone asks you to run it again. Everything in this phase is about data you can trust and workflows that reproduce.

Class 1: Orange, Visual Programming and Why It Is Taken Seriously
  Topics:
    - What Orange is: open source machine learning and data visualisation software from the Bioinformatics Laboratory, Faculty of Computer and Information Science, University of Ljubljana, first released in 1996 and under the GNU GPL version 3 or later
    - Why a visual tool belongs in a serious course: it was built to teach machine learning and data mining to university students, and it runs the same algorithms you would call from code
    - Installation paths: the standalone installer on Windows and macOS, and installing the Orange3 package from the Python Package Index into a Python environment
    - The interface: widget toolbox, canvas, the six default widget sets Data, Transform, Visualize, Model, Evaluate and Unsupervised, and the status bar
    - How data flows: output channels, input channels, and choosing which channel a connection uses when a widget offers several
    - The mental model that prevents most beginner errors: a widget recomputes whenever anything upstream changes, so the canvas is a live pipeline, not a sequence of clicks
    - Course standards announced up front: every deliverable is an .ows workflow that reruns from the raw data file with no manual editing anywhere
  Projects built:
    - Install Orange, build a File to Data Table to Distributions canvas on a bundled dataset, then deliberately change the upstream file and document exactly which downstream widgets recomputed
  Practice: Open three example workflows from the official examples page and write one sentence on what each pipeline is doing.
  Assessment: Your first canvas plus a short written explanation of the live-pipeline model in your own words.

Class 2: Data Sources, Types and Roles
  Topics:
    - The File widget across formats: .csv, .tsv, Excel and Orange's native .tab, including delimiter and encoding problems and how to diagnose them
    - CSV File Import for awkward files: header rows, decimal separators, thousands separators and locale issues
    - SQL Table: reading directly from a database rather than exporting an intermediate spreadsheet nobody can reproduce
    - Datasets: the bundled and downloadable example datasets used throughout the course for controlled comparisons
    - Variable types in Orange, numeric, categorical, text and datetime, and the downstream failures caused by each mistyped column
    - Roles: feature, target and meta, and why identifiers, names and free text must be pushed to meta rather than fed to a learner
    - Data Info and Column Statistics as a mandatory first look: instances, features, missing values, constant columns and near-duplicates
  Projects built:
    - Load one awkward real-world .csv, correct every mistyped column with Edit Domain, assign roles deliberately, and write a data dictionary describing every variable, its type and its role
  Practice: Import the same file twice, once correctly and once with a wrong delimiter, and record what silently went wrong.
  Assessment: Your data dictionary plus a Data Info screenshot with a written note on every quality problem you found.

Class 3: Missing Data, Outliers and Defensible Cleaning
  Topics:
    - The three kinds of missingness in plain language, and why the reason a value is missing changes what you are allowed to do about it
    - The Impute widget in full: leave as is, average or most frequent, replace with a fixed value, remove instances, and model-based imputation
    - The decision that gets challenged in every viva: dropping rows versus imputing, and how to state your choice with a defensible reason
    - Select Rows for rule-based filtering, and why filters belong in the workflow rather than in a spreadsheet you edited by hand
    - Outliers: detecting them, and the separate question of whether to remove, cap, flag or investigate them
    - Duplicate detection with Unique, and the near-duplicate problem that silently inflates every cross validation score
    - Documenting cleaning: annotating the canvas so a reader can see every judgement you made and disagree with it if they want
  Projects built:
    - Build a fully annotated cleaning pipeline on a messy dataset, recording instance counts before and after each step, and write a cleaning log justifying every decision
  Practice: Compare model-based imputation against mean imputation on the same column and record how the distribution changed.
  Assessment: Your cleaning log with before and after counts, plus a defence of your imputation strategy.

Class 4: Joins, Aggregation and Reshaping
  Topics:
    - Merge Data: inner, outer and left join behaviour on a shared key, and diagnosing the rows

that silently disappear
- Concatenate for stacking sources, including what happens when the schemas do not quite agree
- Group by and Aggregate Columns: producing entity-level datasets from transaction-level records, the single most common real analytics task
- Pivot Table for cross-tabulation, and Transpose for datasets stored the wrong way round
- Formula: deriving ratios, differences, flags and interaction terms as genuine engineered features
- Select by Data Index and Data Sampler: reproducible sampling, stratified sampling, and holding out a set you genuinely do not look at
- Granularity discipline: knowing exactly what one row represents at every point in your pipeline

Projects built:
- Build an entity-level analysis dataset from at least two source tables using Merge Data and Group by, engineer three derived features with Formula, and document what one row represents at each stage

Practice: Perform the same join two ways and reconcile the row counts, explaining any difference.

Assessment: Your multi-table workflow plus the row-meaning documentation at each stage.

Class 5: Preprocess, Reproducibility and Workflow Engineering

Topics:
- The Preprocess widget as a single reusable block: normalisation, standardisation, imputation, feature selection, discretisation, continuisation and randomisation
- The leakage rule stated early and repeated all course: preprocessing that learns from data, such as scaling or imputation, must be fitted inside the validation loop, not applied to the whole dataset first
- How Orange helps: connecting Preprocess into a learner so the preprocessing is fitted per training fold during cross validation
- Purge Domain and Randomize: cleaning unused categories, and shuffling a target deliberately as a sanity check that your pipeline cannot score well on noise
- Saving workflows as .ows files: XML describing widgets, their settings and their connections, saving the workflow and settings but not the data
- Consequences for sharing: an .ows file needs its data file alongside it, so a shareable project is a folder, not a single file
- Workflow engineering standards: left to right layout, renamed widgets, canvas notes, and a README describing how to rerun the analysis

Projects built:
- Package everything from Classes 2 to 4 as a reproducible project folder: annotated .ows workflow, raw data file and a README, then have a classmate rerun it from scratch and report any step they could not reproduce

Practice: Run the shuffled-target sanity check on your pipeline and confirm performance collapses to baseline as it should.

Assessment: A project folder a classmate successfully reran, plus their written reproduction report.

PHASE 2: EXPLORATORY ANALYSIS AND FEATURE ENGINEERING

Classes 6 to 10. Model performance is decided long before the learner widget. This phase covers reading a dataset properly and then constructing the features that make the difference, along with the diagnostics that tell you when a feature is helping and when it is cheating.

Class 6: Distributions, Dispersion and Group Comparison

Topics:
- Distributions: shape, skew, modality and the long tail, and what each implies for the models available to you
- Box Plot and Violin Plot: comparing groups on a numeric variable, including unequal group sizes and what that does to a visual comparison
- Central tendency and dispersion together: why reporting a mean without a spread is an incomplete claim
- Detecting distributional problems that break specific models, such as heavy skew for distance-based methods
- Transformations in plain terms, and when a log-style transformation genuinely helps versus when it just hides the problem
- Target distribution first: class imbalance diagnosed before any model is trained, using Distributions on the target
- Stating an exploratory finding as a hypothesis to test rather than a conclusion to announce

Projects built:
- Produce a full univariate profile of your dataset, every numeric variable charted and described, every categorical variable tabulated, ending with three written hypotheses worth testing

Practice: Identify one variable whose mean misrepresents it and justify a better summary statistic.

Assessment: Your univariate profile document plus the three hypotheses with the evidence that prompted each.

Class 7: Bivariate Analysis and Informative Projections

Topics:

- Scatter Plot with class colouring, point size and shape used as genuine extra dimensions
- Find Informative Projections: ranking variable pairs by how well they separate the target classes
- Rank: scoring features by information gain, gain ratio, gini, chi square, ReliefF and FCBF, and reading the disagreements between scores as information
- Sieve Diagram and Mosaic Display for categorical against categorical relationships
- Linear Projection, Radviz and FreeViz for looking at many variables at once in a single plane
- Selecting subsets directly from a plot and sending them downstream for inspection, the fastest way to interrogate a suspicion
- The discipline of the alternative explanation: for every relationship you find, write the confounder that could produce it

Projects built:
- Rank every feature by at least three scoring methods, reconcile the disagreements in writing, and build the informative projection that best separates your target classes

Practice: Take your strongest bivariate relationship and write two plausible confounders for it.

Assessment: Your multi-method feature ranking with a written reconciliation, plus the projection and its interpretation.

Class 8: Correlation, Multicollinearity and Distance

Topics:
- Correlations: ranking every numeric pair, and reading the coefficient without over-claiming what it establishes
- Multicollinearity: what happens to a linear model's coefficients when two features carry the same information, and why the model can still predict well while its explanation becomes worthless
- Heat Map with row and column clustering for whole-dataset structure at a glance
- Distances: Euclidean, Manhattan, cosine, Jaccard and others, and choosing a measure that matches the data type and the question
- Distance Map, Distance Matrix and Distance Transformation as the shared foundation under clustering and nearest neighbour methods
- Why normalisation must precede distance, demonstrated by breaking it deliberately with one large-scale feature
- Feature selection versus dimensionality reduction: dropping features versus combining them, and when each is appropriate

Projects built:
- Diagnose multicollinearity in a numeric dataset, decide which features to drop with a written justification, and demonstrate the effect on a linear model's coefficients before and after

Practice: Compute the same distance matrix under three distance measures and describe how the neighbourhood structure changed.

Assessment: Your multicollinearity report including the coefficient comparison and your retained feature set.

Class 9: Feature Engineering, Discretisation and Encoding

Topics:
- Feature engineering as the highest-leverage work in applied machine learning, with worked examples of features that transformed a weak model
- Formula: ratios, differences, flags, counts and interaction terms built inside the workflow, and Create Instance for testing a model on a hand-specified case
- Discretize: equal width, equal frequency and entropy-based binning, and what each does to the information in a variable
- Continuize: one-hot and ordinal treatment of categorical variables, and matching the encoding to the model that will consume it
- Create Class: deriving a target variable from raw columns, and the leakage danger that this step invites
- Domain knowledge as a feature source: interviewing the person who understands the data before engineering anything
- Testing whether a feature earned its place: rerun the cross validated comparison with and without it

Projects built:
- Engineer at least five new features from domain reasoning, then run a cross validated with-and-without comparison for each and keep only the ones that measurably earned their place

Practice: Compare equal-frequency and entropy-based discretisation of the same variable and note the difference in downstream model performance.

Assessment: Your feature engineering log showing each candidate feature, its rationale, its measured effect and your keep or drop decision.

Class 10: Communicating Analysis with the Orange Report

Topics:
- The Report system: most widgets carry a report button in the status bar that appends the current view to a running report
- Which widgets do not report, and why the ones that only transmit or display data are excluded
- Structuring an analytical report the way a reader needs it: question, data and its limits, method, evidence, result, threats to validity
- Adding written commentary to report sections so the argument, not just the output, is preserved
- Exporting as .html or .pdf for submission, and as .report so a reader can reopen the exact workflow behind each section

- Reopening reports through File then Open Report, and the report view via Shift and R
- Presenting to a non-technical audience: what to cut, what to keep, and how to state uncertainty without losing the room

Projects built:

- Produce a complete exploratory data analysis report on your project dataset: question, data dictionary, univariate and bivariate evidence, engineered features, and a stated set of limitations, exported as PDF

Practice: Cut your report to a three minute verbal summary and note what you were forced to drop.

Assessment: Your exported EDA report, graded on whether every claim in it is supported by a visible piece of evidence.

PHASE 3: MODELLING AND RIGOROUS EVALUATION

Classes 11 to 16. The modelling half of a data mining syllabus, taught with the validation rigour that distinguishes a result from a coincidence. Classification and regression, simple models and ensembles, and tuning that does not quietly contaminate the test set.

Class 11: The Modelling Frame, Baselines and Validation Design

Topics:

- Supervised learning framed properly: a hypothesis space, a loss, and an estimate of generalisation error, all in plain language
- Classification versus regression, decided by the target variable, and the metrics that come with each
- The Orange modelling shape: data into learner into Test and Score, with Preprocess attached to the learner rather than the data
- Constant as the mandatory baseline, and Feature as Predictor as a second reference point using a single variable
- Choosing a validation design: train and test split, k-fold cross validation, stratified folds, leave one out, and repeated sampling, with the cost and reliability of each
- Class imbalance: what accuracy hides, and why AUC, F1, precision and recall exist
- Declaring your evaluation protocol before you look at any result, and writing it into the project README

Projects built:

- Write and register your evaluation protocol for the course project, then establish both baselines with stratified 10-fold cross validation and record the numbers every later model must beat

Practice: Run the same model under three validation designs and record how much the estimate moves.

Assessment: Your registered evaluation protocol plus a baseline results table with the validation design justified.

Class 12: Trees, Rules and Interpretable Models

Topics:

- Decision tree induction: impurity measures, the greedy split search, and stopping criteria as the practical control on complexity
- The Tree widget's parameters, minimum instances in a leaf, maximum depth, and splitting thresholds, mapped to bias and variance
- Tree Viewer, Pythagorean Tree and Pythagorean Forest for reading structure rather than guessing at it
- CN2 Rule Induction and CN2 Rule Viewer: learning an ordered or unordered rule set you can read as if-then statements
- Scoring Sheet and Scoring Sheet Viewer: producing the kind of simple point-based model that regulated domains actually accept
- Overfitting demonstrated live: growing a tree until training accuracy is perfect and cross validated accuracy collapses
- The interpretability and accuracy trade-off, and the domains where an interpretable model is a requirement rather than a preference

Projects built:

- Train a tuned decision tree and a CN2 rule set on the same problem, extract the decision logic from both as written rules, and compare their cross validated performance against your baselines

Practice: Sweep maximum depth across five values and tabulate training against cross validated accuracy to locate the overfitting point.

Assessment: Your extracted rule set in plain English plus the overfitting curve with the depth you chose and why.

Class 13: Distance and Probability Models, Regression Begins

Topics:

- k Nearest Neighbours: the choice of k, distance measure and weighting, and its total dependence on feature scaling
- Naive Bayes: the conditional independence assumption, why it is usually false, and why the model often works anyway
- Linear Regression in Orange: predicting a continuous target, with ridge and lasso regularisation and what each penalty does
- Regression metrics and how to read them: MSE, RMSE, MAE and R squared, and what R squared does not tell you
- PLS and Curve Fit for the cases where ordinary linear regression is the wrong shape

- Stochastic Gradient Descent as a learner, and the intuition of iterative optimisation without the calculus
- Choosing a model family from the data and the requirement, not from familiarity

Projects built:
- Build a full regression pipeline on a continuous target, compare linear regression with and without regularisation against kNN regression, and report RMSE and R squared under identical cross validation

Practice: Run kNN with and without normalisation inside the Preprocess block and quantify the difference.

Assessment: Your regression comparison table with metrics defined in your own words and a justified model choice.

Class 14: Evaluation in Depth, from Confusion Matrix to Calibration

Topics:
- Test and Score read properly: comparing learners under identical folds, and the meaning of the variation between folds
- Confusion Matrix: the four cells expressed as real costs in the actual application domain
- Precision, recall, specificity and F1 derived from that matrix rather than memorised
- ROC Analysis: the curve as every threshold at once, AUC interpreted as a ranking probability, and reading multiple models on one plot
- Performance Curve for the operational question of how many cases you must action to capture a given share of positives
- Calibration Plot: whether predicted probabilities behave like probabilities, and Calibrated Learner as the fix
- Choosing the operating threshold from the cost of each error type rather than accepting 0.5 by default

Projects built:
- Evaluate every model built so far on one problem with a full evaluation dossier: metrics table, confusion matrices with stated costs, ROC comparison, calibration plot, and a recommended operating threshold with its justification

Practice: Take a poorly calibrated model, apply Calibrated Learner, and document the change in the calibration plot.

Assessment: Your evaluation dossier, graded on whether the threshold recommendation follows from stated error costs.

Class 15: Ensembles and Neural Networks

Topics:
- Random Forest: bagging plus random feature subsets, why decorrelated trees generalise better, and the parameters that matter
- Gradient Boosting: sequential correction of residuals, and the learning rate and depth interaction that governs overfitting
- AdaBoost: reweighting the examples the ensemble keeps getting wrong
- Support Vector Machines: margins, the kernel trick explained without the algebra, and the cost parameter
- Neural Network in Orange: hidden layer sizes, activation functions, the solver, and realistic expectations on tabular data of student scale
- Stacking: combining several learners with a meta-learner, and the extra validation care it demands
- The honest finding that a well tuned simple model frequently matches an ensemble, and why reporting that is good work

Projects built:
- Run a full learner tournament, baseline through neural network and stacking, under one fixed cross validation, and write a model selection memo that justifies your winner on grounds beyond the top accuracy figure

Practice: Vary the gradient boosting learning rate and depth together and record where performance stops improving.

Assessment: Your model selection memo, which must address performance, variance across folds, interpretability and runtime.

Class 16: Tuning, Deployment and the Traps That Invalidate Results

Topics:
- Parameter Fitter: searching a learner's parameters systematically instead of by intuition
- The tuning trap named precisely: selecting parameters using the same data you then report performance on inflates your result
- Nested validation in principle, and the practical discipline of a genuinely held-out set that is opened exactly once
- Permutation Plot and the shuffled-target sanity check as evidence that your pipeline is not learning noise
- Predictions: scoring new unlabelled data, reading class probabilities, and exporting results with Save Data
- Save Model and Load Model: persisting a fitted model so scoring does not require retraining
- The leakage catalogue, target leakage, temporal leakage, group leakage and preprocessing leakage, each with the symptom that gives it away

Projects built:
- Tune your chosen model with Parameter Fitter, evaluate the tuned model on a held-out set you have not previously touched, run the shuffled-target check, and write a leakage audit against all four leakage types

Practice: Deliberately introduce a leaking feature, observe the implausible score, and document how you would have caught it.

Assessment: Your held-out result with the tuning protocol described, plus your completed four-point leakage audit.

PHASE 4: UNSUPERVISED, EXPLAINABLE AND SPECIALISED DATA

Classes 17 to 21. The rest of a data mining syllabus, taught to the same standard: clustering with validity measures rather than eyeballing, dimensionality reduction, association rules, model explainability, and the two data types that dominate project work, text and time series.

Class 17: Clustering and Cluster Validity

Topics:

- Unsupervised learning framed honestly: no ground truth, so validity has to be argued rather than measured against an answer key
- k Means: the algorithm step by step, initialisation sensitivity, and scoring a range of k automatically
- Silhouette score and Silhouette Plot: evaluating cohesion and separation, and finding the points that were forced into the wrong cluster
- Hierarchical Clustering: linkage methods, reading a dendrogram, and choosing the cut height defensibly
- DBSCAN: density-based clusters of arbitrary shape, the epsilon and minimum points parameters, and the ability to label points as noise
- Louvain Clustering and Self-Organizing Map as further approaches, and Interactive k-Means from the Educational add-on for stepping through the algorithm
- Profiling clusters into an interpretation: describing each group by its distinguishing features so the result means something to a domain reader

Projects built:

- Cluster one dataset with k means, hierarchical clustering and DBSCAN, justify your chosen k or cut with silhouette evidence, and write a business-readable profile of every cluster you keep

Practice: Run k means on uniformly random data and write the paragraph explaining why its clusters are meaningless.

Assessment: Your three-method comparison with silhouette evidence, plus interpretable profiles for each final cluster.

Class 18: Dimensionality Reduction, Outliers and Association Rules

Topics:

- PCA: principal components as ordered directions of maximum variance, and using explained variance to choose how many to keep
- The interpretability cost of PCA, and when a loading is still readable enough to explain
- t-SNE, MDS and Manifold Learning for visualisation, with an explicit warning about over-reading distances between separated clusters in a t-SNE plot
- Correspondence Analysis for categorical structure
- Outliers: detection methods and the analyst's decision about whether an outlier is an error, a rare event, or the entire point of the study
- The Associate add-on: frequent itemsets and association rules, with support, confidence and lift defined precisely
- Reading association rules critically: high confidence with low lift is usually just a popular item, not a relationship

Projects built:

- Reduce a wide dataset with PCA, visualise it with t-SNE, then mine association rules on a transactional dataset and identify three rules that are genuinely informative by lift rather than merely frequent

Practice: Cluster on raw features and on PCA components and compare silhouette scores and cluster interpretability.

Assessment: Your dimensionality reduction analysis plus three association rules with support, confidence and lift interpreted correctly.

Class 19: Explainability and Fairness

Topics:

- Why explainability is now a requirement: examiners, regulators, clinicians and clients all ask why, and a score is not an answer
- The Explain add-on: Explain Model for global feature attribution across the dataset
- Explain Prediction and Explain Predictions: why this specific instance received this specific output
- Permutation Importance: measuring a feature's contribution by breaking it and observing the damage
- ICE plots for seeing how a prediction responds as one feature is varied across its range
- Nomogram for logistic regression as a fully transparent alternative to post-hoc explanation
- The Fairness add-on: auditing model behaviour across a sensitive attribute, and the fact that a fairness problem is usually a data problem

Projects built:

- Take your best model, produce a global explanation and three individual prediction explanations, run a permutation importance check, and audit the model across one sensitive attribute with a written finding

Practice: Find one instance your model got confidently wrong and explain, from its attribution, why it failed.

Assessment: Your explainability report including global attribution, three instance-level

explanations, and the fairness audit finding.

Class 20: Text Mining and Document Analysis
Topics:
- Installing add-ons through Options then Add-ons, and what the Text add-on contributes to the toolbox
- Corpus, Import Documents and Create Corpus: loading documents as data, including a folder of your own files
- Preprocess Text in detail: transformation, tokenisation, normalisation with stemming or lemmatisation, stop word and lexicon filtering, and n-grams
- Bag of Words with term frequency and TF-IDF weighting, and what a bag of words representation discards
- Document Embedding as the modern alternative, and when it beats a bag of words
- Analysis widgets: Word Cloud, Concordance, Statistics, Word Enrichment, Extract Keywords and Duplicate Detection
- Topic Modelling and Corpus Viewer, plus Sentiment Analysis with an honest account of its failure modes on sarcasm, negation and mixed-language text

Projects built:
- Build an end to end text pipeline: load a corpus, preprocess it deliberately, represent it with both bag of words and embeddings, classify or topic-model it, and evaluate the two representations against each other under identical validation

Practice: Rerun your pipeline with stemming, with lemmatisation and with neither, and tabulate the effect on your metric.
Assessment: Your text mining report comparing both representations, with named topics or a validated classifier and its limitations.

Class 21: Time Series and Spatial Data
Topics:
- The Time Series add-on: As Timeseries for converting a table, Line Chart, and Yahoo Finance for pulling market data
- Why time series breaks the rules you learned: rows are not independent, so ordinary shuffled cross validation leaks the future into the past
- Interpolate, Moving Transform and Seasonal Adjustment: filling gaps, computing rolling statistics, and separating trend, seasonality and remainder
- Correlogram and Periodogram: autocorrelation and periodicity, and finding the cycle length in real data
- ARIMA Model and VAR Model: univariate and multivariate forecasting, with Model Evaluation for out-of-sample assessment
- Granger Causality read carefully: predictive precedence is not causation, whatever the widget is called
- The Geo add-on and the Network add-on named and demonstrated for spatial and relational project data

Projects built:
- Take a real time series, decompose it, identify its dominant period, fit an ARIMA model, evaluate it out of sample against a naive last-value baseline, and state honestly whether the model beat the baseline

Practice: Demonstrate the leakage caused by shuffled cross validation on time series data and quantify how much it inflates the score.
Assessment: Your forecasting workflow with an out-of-sample evaluation and an explicit comparison against the naive baseline.

PHASE 5: THE PYTHON BRIDGE AND CAPSTONE

Classes 22 to 24. Leave the course able to work in both worlds. Orange is a Python library, so the last phase connects the canvas to code, sets the ethical standard for the work, and produces a capstone that reproduces on a machine that is not yours.

Class 22: The Python Script Widget and Orange as a Library
Topics:
- Orange3 as a Python package installable from the Python Package Index, with a current Python version required for the pip installation route
- The Python Script widget: running real Python inside the canvas when no existing widget does what you need
- The widget's namespace: in_data, in_distance, in_learner, in_classifier and in_object arrive from the input signals, and are None when nothing is connected
- Returning results to the canvas so a script sits in the middle of a workflow rather than at the end of it
- The Orange.data.Table object and how it maps onto the pandas DataFrame most students will meet next
- The concept map from Orange to scikit-learn: learner to estimator, Test and Score to cross_val_score, Preprocess to Pipeline, Parameter Fitter to GridSearchCV
- Choosing the right tool honestly: canvas for exploration, comparison and communication, code for automation, scale and version control

Projects built:
- Insert a Python Script widget into an existing workflow to compute something no widget provides, pass the result downstream, and write the equivalent scikit-learn outline for one pipeline you built earlier in the course

Practice: Map five widgets you used this course onto their scikit-learn equivalents in a

written table.
Assessment: Your working Python Script integration plus the Orange to scikit-learn concept map.

Class 23: Build and Defend Your Capstone
Topics:
- Scoping a capstone that is answerable: a specific question, an obtainable dataset, a defined target and a stated success criterion
- Sourcing data responsibly: open government portals, research repositories, Kaggle and institutional data, with the licence and any consent conditions checked and recorded
- Applying the full pipeline discipline: data dictionary, cleaning log, engineered feature log, registered evaluation protocol, held-out set
- Beating the baseline, and the professional willingness to report that the sophisticated approach did not win
- Threats to validity written before the conclusion: sample bias, leakage, confounding, temporal drift and generalisation limits
- Mentor review checkpoint: an instructor interrogates your canvas the way an examiner will, step by step
- Assembling the deliverable folder: .ows workflow, raw data, README with rerun instructions, and the report

Projects built:
- Build your complete capstone from raw data to evaluated model and written conclusion, defended live in a mentor review that challenges every methodological choice

Practice: Have a classmate rerun your capstone folder on their own machine and log every step that failed.
Assessment: A capstone that reproduces from raw data in a clean session on someone else's machine, plus your threats-to-validity section.

Class 24: Report, Present and Continue
Topics:
- Assembling the final report: question, data and provenance, method, evidence, results with uncertainty, threats to validity, and conclusion
- Exporting for the audience you have: PDF for submission, and .report so a reviewer can reopen the workflow behind every figure
- Presenting under questioning: leading with the question and the answer, defending the method, and saying clearly when something is unknown
- The interview and viva questions this course prepares you for: why that validation design, why that metric, how did you rule out leakage, why should the model be believed
- Portfolio assembly: 24 reproducible workflows plus the capstone, organised so a reviewer can open any one and follow it
- Continuing paths: Python with pandas and scikit-learn, statistics and experimental design, and further Orange add-ons including Bioinformatics, Single-Cell, Spectroscopy, Survival Analysis and Network
- Staying current with the tool: the official Orange documentation, workflow examples and video lessons, and the open source repository itself

Projects built:
- Present your capstone under live questioning, submit the exported report and complete portfolio, and receive written feedback plus your certificate assessment

Practice: Write the one page project proposal for your next analysis, including its question, data source, evaluation protocol and baseline.
Assessment: Your defended presentation, exported report and 24-workflow portfolio, assessed on methodological rigour as much as on results.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: a live-pipeline demonstration, a corrected dataset with a full data dictionary, an annotated cleaning log with before and after counts, a multi-table entity-level dataset, and a reproducible project folder a classmate reran successfully
- Phase 2: a complete univariate profile with three testable hypotheses, a multi-method feature ranking with reconciled disagreements, a multicollinearity report with coefficient evidence, a measured feature engineering log, and an exported EDA report
- Phase 3: a registered evaluation protocol with dual baselines, a tuned tree and CN2 rule set with extracted logic, a regularised regression comparison, a full evaluation dossier with a cost-justified threshold, a learner tournament with a model selection memo, and a held-out tuned result with a four-point leakage audit
- Phase 4: a three-method clustering study with silhouette evidence and readable cluster profiles, a PCA and t-SNE reduction with lift-interpreted association rules, an explainability report with a fairness audit, a two-representation text mining comparison, and an out-of-sample ARIMA forecast tested against a naive baseline
- Phase 5: a Python Script widget integrated mid-workflow with an Orange to scikit-learn concept map, and the capstone: reproducible folder, threats to validity, exported report and a presentation defended under questioning
- Total: 24 reproducible workflows plus a capstone that survives being rerun by someone else

Total projects: 24 reproducible workflows, one per class, plus a capstone project that reruns from raw data on another person's machine

SKILLS THE STUDENT WILL HAVE AT THE END

- Reproducible Workflow Engineering: canvas discipline, channel selection, annotated .ows workflows, project folders with README rerun instructions
- Data Engineering in Orange: File and CSV File Import, SQL Table, Edit Domain, Impute, Select Columns and Rows, Merge Data, Concatenate, Group by, Aggregate Columns, Pivot Table and Formula
- Exploratory Analysis: Distributions, Box and Violin Plot, Scatter Plot with informative projections, Rank across multiple scoring methods, Correlations, Heat Map, Distances and Distance Map
- Feature Engineering: derived and interaction features, Discretize, Continuize, Create Class, and measured with-and-without validation of every candidate feature
- Classification and Regression: Constant and Feature as Predictor baselines, Tree, CN2 Rule Induction, Scoring Sheet, kNN, Naive Bayes, Linear and Logistic Regression with regularisation, PLS, SVM, Random Forest, Gradient Boosting, AdaBoost, Neural Network and Stacking
- Rigorous Evaluation: Test and Score validation designs, Confusion Matrix with stated error costs, ROC Analysis, Performance Curve, Calibration Plot and Calibrated Learner, Parameter Fitter tuning, Permutation Plot, and a working catalogue of leakage types
- Unsupervised Learning: k Means with silhouette selection, Hierarchical Clustering, DBSCAN, Louvain, Self-Organizing Map, PCA, t-SNE, MDS, Manifold Learning, Correspondence Analysis, Outliers and association rule mining
- Explainability and Fairness: Explain Model, Explain Prediction, Permutation Importance, ICE, Nomogram, and auditing across a sensitive attribute
- Specialised Data: Corpus, Preprocess Text, Bag of Words with TF-IDF, Document Embedding, Topic Modelling, Sentiment Analysis, As Timeseries, Seasonal Adjustment, Correlogram, Periodogram, ARIMA and VAR
- The Python Bridge: the Python Script widget with its in_data namespace, Orange3 as a Python package, and the concept map onto pandas and scikit-learn

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, about 1 hour each, across 12 weeks
Hands on canvas work: 2 hours rebuilding each class workflow on your own project dataset, not only on the class example
Project time: 1.5 hours on the class deliverable, from cleaning logs and evaluation dossiers to the capstone build
Review and doubts: 1 hour reviewing results against your registered protocol, checking for leakage, and clearing doubts before the next class
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for installation, add-on, dataset and workflow problems
Small batches: Small batches, with mini-batch and 1-on-1 options, so your own project dataset gets attention every week
Mentor guidance: Live guidance from mentors who build and evaluate models professionally, including an interrogation-style capstone review
Project feedback: Feedback on every deliverable, graded on methodology, reproducibility and honest reporting as much as on performance
Living syllabus: Curriculum updated as Orange ships new versions, widgets and add-ons, at no extra cost

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Data Analyst and Business Analyst roles, where cleaning, exploration, validation and clear reporting are the daily work
- Junior Data Scientist and Machine Learning Analyst positions, entered with genuine evaluation and explainability discipline
- Research Assistant and academic project work in bioinformatics, public health, social science, economics and environmental science
- Business Intelligence and Reporting Analyst roles, building on the aggregation, visualisation and reporting work in this course
- A prepared transition into Python-based data science, since every concept maps directly onto pandas and scikit-learn

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 3-5 LPA (Junior Analyst); Rs 5-8 LPA (Data Analyst)

Growing roles: Rs 8-15 LPA (Senior Analyst)
Other roles: Rs 15-25+ LPA (Lead/Manager)
Freelance: Rs 500-2,000/hour
International: $50k-100k+ USD

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is Orange serious enough for a college level data mining course?
A: Yes. Orange is developed at the Bioinformatics Laboratory, Faculty of Computer and Information Science, University of Ljubljana, has been in development since 1996, and was built in part for teaching machine learning and data mining to university students. It runs standard algorithms including trees, kNN, naive Bayes, SVM, random forests, gradient boosting and neural networks, evaluates them with proper cross validation and ROC analysis, and is written in Python so the same models are available as code. It is also citable, which matters for project reports.

Q: Do I need to know Python before starting?
A: No. The full 24 classes can be completed without writing a line of code, because analysis is built by connecting widgets. Class 22 is dedicated to the Python bridge, where the Python Script widget runs real Python inside a workflow and every Orange concept is mapped onto its scikit-learn equivalent, so students who already code get extra value there and students who do not are not disadvantaged anywhere else.

Q: Will this cover my university data mining syllabus?
A: It covers the standard ground: data preprocessing and cleaning, exploratory analysis, feature engineering, classification, regression, model evaluation, clustering, dimensionality reduction, association rule mining, text mining and time series, plus explainability and fairness which many syllabi have only recently added. It teaches these as practice rather than as definitions, so it complements exam preparation rather than replacing your notes. Bring your syllabus to the free demo class and we will map it against the 24 classes. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Can I use this for my final year project or dissertation?
A: That is exactly what Phase 5 is built for. Classes 23 and 24 take you from scoping a defensible question through data sourcing with licence checks, a registered evaluation protocol, a held-out set, a threats-to-validity section, and a mentor review conducted like a viva. The standard for the capstone is that it reproduces from the raw data file in a clean session on a machine that is not yours.

Q: How is this different from a Python data science course?
A: A Python course spends its early weeks on syntax, environments and library APIs, which is necessary but expensive. This course removes that overhead and spends the time on method instead, so it goes further into validation design, leakage, explainability, association rules and time series than a first code course usually reaches. It is designed to be taken before or alongside Python, not instead of it, and Class 22 exists specifically to make the transition short.

Q: Is Orange free, and does anything expire?
A: Orange is free and open source software released under the GNU General Public License version 3 or later. There is no subscription, no trial clock and no student licence. It runs on Windows, macOS and Linux, and the add-ons used in this course install free from inside the application under Options then Add-ons. Everything you build keeps working after the course ends.

Q: What add-ons does the course install, and are they hard to set up?
A: We install them together in class from Options then Add-ons. The course uses Text for corpus analysis, Image Analytics where relevant, Time Series for temporal data, Explain for model interpretation, Associate for rule mining, and introduces Educational, Geo, Network and Fairness. All are free. The text and image embedders that run on a server need a working internet connection, which we flag before those classes.

Q: Do I need my own dataset?
A: Not to start. Orange ships with example datasets and the Datasets widget offers more, and every class provides a working dataset. From Phase 2 onward we strongly encourage you to run each class a second time on a dataset from your own field or project, because the course is far more useful when the capstone is something you actually needed to answer.

Q: What will I be able to do in an interview after this?
A: Answer the method questions that separate candidates: why that validation design, what the confusion matrix costs in this domain, how you ruled out leakage, why you chose that operating threshold, which features drove the prediction and how you know. You will also have a portfolio of 24 reproducible workflows plus a defended capstone to show, rather than a certificate on its own.

Q: How am I assessed, and is the certificate just for showing up?

A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final project defended live, plus a written assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/orange-data-mining-machine-learning-course-for-college-students
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 109 OF 137: ORANGE DATA MINING COURSE FOR TEENS: MACHINE LEARNING WITHOUT CODE

==========

Course link: https://learn.modernagecoders.com/courses/orange-data-mining-visual-machine-learning-course-for-teens
Subject area: Data Science & Machine Learning
Designed for: Teens
Who can join: Beginner-friendly (ages 13-18), no programming experience required
Course length: 24 classes (12 weeks · 2 classes/week)
Time needed: 3-4 hours/week recommended
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Learn Orange Data Mining in 24 live classes for teens 13-18: build machine learning workflows by connecting widgets on a canvas, visualise real datasets, train and honestly evaluate models, mine text and images, and finish with a shareable data project.

In one line: Build real machine learning by dragging widgets onto a canvas: load a dataset, clean it, chart it, train a model, test whether it is actually any good, and present the whole thing as a project you can defend.

ABOUT THE PROGRAMME

Most teenagers meet machine learning as a black box: an app that guesses the filter, a chatbot that answers, a recommendation that appears. This course opens the box, without asking anyone to learn Python first. Orange is free, open source data mining software built by the Bioinformatics Laboratory at the Faculty of Computer and Information Science, University of Ljubljana, and released under the GNU GPL version 3 or later. Instead of writing code, you place widgets on a canvas, connect them with lines, and watch data flow through the pipeline you designed. That visual canvas is not a toy version of machine learning: underneath, Orange runs the same well established algorithms a data scientist would use, and it is written in Python so the exact same models are available in code later. Across 24 live classes for ages 13 to 18 you work through the six widget sets Orange ships with by default, Data, Transform, Visualize, Model, Evaluate and Unsupervised, and then push into the Text and Image Analytics add-ons. You will load and repair messy files, read a dataset honestly before touching a model, build scatter plots and heat maps that actually answer a question, train decision trees, k nearest neighbours, logistic regression, random forests and neural networks, and then, most importantly, learn to test them properly with cross validation, confusion matrices and ROC curves instead of trusting a single flattering number. You will cluster data with k means and hierarchical clustering, shrink hundreds of columns with PCA and t-SNE, mine a corpus of text into a word cloud and a sentiment score, and train an image classifier using image embeddings. Every class is taught live by a real instructor in a small batch, the same mentorship-first method Modern Age Coders has used since 2020 to teach 10,000+ students across 25+ countries. Nothing here is a pre-recorded video. You finish with 24 saved workflows, a capstone data project of your own choosing, and an Orange report you can hand to a science fair judge, a school teacher, or a college admissions reader who asks what you actually built.

WHAT MAKES THIS COURSE DIFFERENT

- No programming prerequisite, and no pretending: you build genuine machine learning pipelines from Class 1, using the real algorithms, not simplified imitations.
- Free software, forever: Orange is open source under the GNU GPL v3 or later, so there is no subscription, no trial clock, and nothing to cancel when the course ends.
- Honest evaluation is taught as a core skill, not an afterthought: cross validation, confusion matrices, ROC curves and baseline comparison start in Phase 3 and never stop.
- Real datasets, not tidy classroom toys: you clean missing values, fix wrong variable types, and deal with data that argues back.
- Every class ends with a saved .ows workflow, so by Class 24 you own a portfolio of 24 pipelines you can reopen, edit, and explain.
- A bridge to code, not a dead end: Orange is written in Python, and the Python Script widget

lets you drop into real Python inside the canvas whenever you are ready.

WHAT A STUDENT NEEDS BEFORE STARTING

Device: A laptop or desktop running Windows, macOS or Linux, with enough room to install Orange, plus a stable internet connection for live classes and the image embedding classes
Coding experience: None. This course is deliberately built for teens who have never written a line of code, and everything is done by connecting widgets on a canvas. Students who already know some Python are not held back, since the Python Script widget lets them extend any workflow
Maths background: School-level comfort with averages, percentages and reading a graph is enough. Every statistical idea used, from distributions to probability thresholds, is taught inside the course when it is needed
Accounts: No account is required to use Orange, which is free and open source software you download and run on your own machine
Mindset: Curiosity about a real question you would like answered with data, and willingness to accept an answer you did not expect, including the answer that the data does not support your hunch

WHO THIS COURSE IS FOR

Curious teens without coding: Ages 13-18 who are interested in AI and data but have not learned to program yet, and do not want to wait a year before touching real machine learning
Science fair and project students: Students preparing a science fair entry, school research project or IB-style investigation who need genuine analysis behind the poster rather than a spreadsheet chart
Future data and cs applicants: Teens aiming at computer science, data science, statistics, economics or biology at university who want the concepts before the syntax
Students already learning python: Young coders who can write Python but have never evaluated a model honestly, and want to learn cross validation, ROC and leakage properly on a visual canvas first
Students with a dataset and a question: Anyone who runs a club, a team, a channel or a small project and already has data they suspect is hiding something worth knowing

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE CANVAS AND YOUR DATA

Classes 1 to 5. Install Orange, learn how visual programming actually works, and get genuinely good at the unglamorous part every real analyst spends most of their time on: loading, reading, repairing and reshaping a dataset before a single model is trained.

Class 1: What Data Mining Is, and Meeting Orange
Topics:
- What data mining means in plain language: finding patterns in data that are too large or too tangled for a person to spot by reading rows
- Where you already meet it every day: spam filters, recommendation feeds, fraud alerts, medical screening, and school analytics dashboards
- Meet Orange: free and open source machine learning and data visualisation software from the Bioinformatics Laboratory, Faculty of Computer and Information Science, University of Ljubljana
- Why it is free forever: Orange is released under the GNU General Public License version 3 or later, so there is no subscription and no trial period
- Installing Orange on your own machine: the standalone installer for Windows and macOS, package options on Linux, and confirming the version under Help then About
- The anatomy of the window: the widget toolbox on the left, the canvas in the middle, and the six default widget sets Data, Transform, Visualize, Model, Evaluate and Unsupervised
- Visual programming in one sentence: place widgets on the canvas, connect them with lines, and data flows along those lines from one widget to the next

Projects built:
- Install Orange on your own computer, open a blank canvas, place a File widget and a Data Table widget, connect them, and take a screenshot proving data flowed from one to the other

Practice: Open every one of the six default widget sets in the toolbox and write down one widget from each that you cannot yet explain.
Assessment: A screenshot of your Orange version from Help then About, plus your two-widget canvas with data visible in the Data Table.

Class 2: Loading Data and Reading It Honestly
Topics:
- The File widget: opening .csv, .tsv, Excel files and Orange's own .tab format, and what happens when a file will not load cleanly
- The Datasets widget: browsing the bundled example datasets so you always have something real to practise on
- Data Table: sorting, selecting rows, and actually looking at your data before assuming anything about it
- Data Info: how many instances, how many features, how many missing values, and why that last

number changes everything you do next
- Variable types in Orange and why they matter: numeric, categorical, text, and datetime, and what breaks when a type is guessed wrong
- Roles: feature, target and meta attribute, and why choosing the target is the single most important click in a supervised workflow
- The habit that separates good analysts from lucky ones: describe your dataset in one paragraph before you build anything on top of it

Projects built:
- Load a real .csv you care about (school marks, cricket or football statistics, weather, a game leaderboard), inspect it in Data Table and Data Info, and write a one paragraph description of what each column means

Practice: Deliberately load a file with a wrongly typed column, find it in Data Info, and note exactly which widget downstream would have failed.

Assessment: Your dataset description paragraph plus a Data Info screenshot showing instance count, feature count and missing values.

Class 3: Fixing the Data Nobody Wants to Fix

Topics:
- Edit Domain: renaming variables, changing a variable's type, merging categories, and reordering values so charts read in a sensible order
- Color: assigning meaningful colours to categories once, so every chart downstream stays consistent
- Column Statistics: spotting the columns that are almost entirely empty, constant, or secretly duplicated
- Select Columns: choosing features, setting the target variable, and pushing identifiers into the meta section instead of feeding them to a model
- Select Rows: filtering with conditions to isolate the part of the data you actually want to study
- Impute: what to do about missing values, and the honest trade-off between removing rows, filling with an average, and modelling the missing value
- The rule that saves projects: never delete data silently, always leave the original file untouched and do your fixing inside the workflow

Projects built:
- Take a deliberately messy dataset provided in class, and build a five widget repair pipeline that renames variables, fixes types, filters bad rows and imputes missing values, ending in a clean Data Table

Practice: Rebuild the same repair pipeline on your own dataset from Class 2 and record how many rows survived.

Assessment: Your repair workflow plus a two-line note on every judgement call you made and why.

Class 4: Reshaping, Combining and Summarising Data

Topics:
- Merge Data: joining two tables on a shared column, and what happens to rows that have no match on the other side
- Concatenate: stacking datasets that share the same columns, for example three months of records in three files
- Group by and Aggregate Columns: turning thousands of rows into the handful of summary numbers a question actually needs
- Pivot Table: cross-tabulating two categorical variables to see the shape of the relationship immediately
- Transpose, Unique and Select by Data Index: the small tools that rescue a badly shaped table
- Data Sampler: taking a random subset for fast experiments, and the idea of a held-out sample you do not look at
- Formula: creating a new calculated column from existing ones, for example a ratio or a difference

Projects built:
- Combine two related datasets with Merge Data, create one new calculated column with Formula, and produce a Pivot Table that answers a specific question you write down first

Practice: Use Group by to produce three summary statistics from your own dataset and check each one by hand on a few rows.

Assessment: Your combined dataset workflow plus the question you set and the pivot table that answers it.

Class 5: Preprocess, Save and Share Your Workflow

Topics:
- The Preprocess widget: normalisation, scaling, discretisation and continuisation collected into one reusable block
- Why preprocessing choices belong inside the workflow, not in a spreadsheet you edited by hand and cannot reproduce
- Randomize and Purge Domain: shuffling values to test assumptions, and cleaning out unused categories
- Save Data: exporting a cleaned dataset so other tools can read it
- Saving the workflow itself as an .ows file: an XML description of your widgets, their settings and their connections
- The crucial detail about .ows files: they save the workflow and widget settings but not the data, so a shared workflow needs its data file alongside it
- Workflow hygiene: naming widgets, laying out the canvas left to right, and adding notes so a reader can follow your reasoning

Projects built:
- Turn your Class 3 and Class 4 work into one tidy, annotated, left-to-right workflow, save it as an .ows file, and swap it with a classmate to check that they can rerun it with the data file

Practice: Reopen a classmate's .ows file and write down the first moment you were confused; that is a labelling problem to fix in yours.

Assessment: A shared .ows file that a classmate successfully reran, plus the feedback note they wrote.

PHASE 2: SEEING THE DATA

Classes 6 to 10. Charts are not decoration, they are how you find the question worth modelling. This phase turns you into someone who can look at an unfamiliar dataset and, within ten minutes, produce the three visuals that explain what is going on.

Class 6: Distributions, Box Plots and Violin Plots

Topics:
- Distributions: the shape of a single variable, and why the average alone hides almost everything interesting
- Reading a histogram properly: bin width, skew, gaps, and the long tail that contains the exceptions
- Splitting a distribution by a categorical variable to compare groups in the same chart
- Box Plot: median, quartiles, whiskers and outliers, and how to read five numbers at a glance
- Violin Plot: when the shape of the distribution matters more than the summary statistics
- Comparing groups fairly: unequal group sizes, and why a difference in medians is not automatically a finding
- Choosing between the three: what question each chart is genuinely the best answer to

Projects built:
- Build a three-chart comparison of one numeric variable across one categorical group in your dataset, using Distributions, Box Plot and Violin Plot, and write which chart told you the most

Practice: Find one variable in a bundled dataset whose average is actively misleading and explain why using its distribution.

Assessment: Your three-chart workflow plus a short paragraph on what the shape of the data revealed.

Class 7: Scatter Plots and Finding Informative Projections

Topics:
- Scatter Plot: two numeric variables, one point per instance, and colouring points by the target variable
- Reading relationships: positive, negative, none, clustered, curved, and the difference between a pattern and a coincidence
- Using size, shape and colour as extra dimensions without making the chart unreadable
- Find Informative Projections: letting Orange rank which pairs of variables separate your classes best
- Rank: scoring individual features by how much information they carry about the target
- Selecting points directly on the chart and sending that selection to a Data Table for inspection
- The trap to name out loud: a strong scatter relationship is correlation, and correlation is not causation

Projects built:
- Use Find Informative Projections to identify the two most separating variables in a classification dataset, then build the scatter plot it suggests and select the most interesting cluster of points into a Data Table

Practice: Rank the features of your own dataset and write down whether the top-ranked feature surprised you.

Assessment: Your scatter plot plus a written interpretation naming one relationship and one alternative explanation for it.

Class 8: Bar Plots, Line Plots and Categorical Relationships

Topics:
- Bar Plot: comparing categories honestly, including the axis choices that quietly exaggerate a difference
- Line Plot: showing values across an ordered sequence, and when a line implies a continuity that does not exist
- Sieve Diagram: seeing whether two categorical variables are independent, using the size of the gaps between observed and expected
- Mosaic Display: reading a whole contingency table as areas rather than numbers
- Venn Diagram: comparing overlapping selections from different parts of your workflow
- Chart honesty: truncated axes, cherry-picked ranges, and misleading aggregation, with real examples to critique
- Choosing the chart from the question, not from the chart menu

Projects built:
- Build a two-variable categorical analysis using Sieve Diagram and Mosaic Display, then present the same finding once honestly and once misleadingly with a Bar Plot, and explain exactly what you changed

Practice: Find a chart in a news article or an advertisement and list every choice that shapes how it is read.

Assessment: Your honest and misleading chart pair, plus a written explanation of the manipulation.

Class 9: Heat Maps, Correlations and Distance Maps
Topics:
- Heat Map: hundreds of rows and dozens of columns compressed into one image you can actually scan
- Clustering rows and columns inside the heat map so similar things sit next to each other
- Correlations: ranking every pair of numeric variables by how strongly they move together
- Reading a correlation coefficient without over-claiming: strength, direction, and what it says nothing about
- Distances and Distance Map: what it means for two rows to be near each other, and why the distance measure you pick changes the answer
- Normalisation before distance: why an unscaled column with big numbers can dominate every comparison
- Multicollinearity in plain language: two features saying the same thing, and why that matters later when you model

Projects built:
- Build a Correlations and Heat Map pair on a numeric dataset, identify one pair of variables that are effectively duplicates, and justify which one you would keep

Practice: Run the same Distance Map with and without normalisation and describe how the picture changed.
Assessment: Your heat map workflow plus the duplicate-feature decision with your reasoning.

Class 10: Telling the Story with the Orange Report
Topics:
- The Report system: most widgets have a report button in the status bar that sends the current view into a running report
- Which widgets cannot report, and why: the ones that only pass data along rather than showing a result
- Building a report in the order a reader needs it: question, data description, evidence, and only then the conclusion
- Adding your own comments to report sections so the reader follows the reasoning, not just the pictures
- Saving the report as .html or .pdf to hand in, and as .report so a reader can reopen the workflow that produced each section
- Opening a saved report through File then Open Report, and showing the report view with Shift and R
- Presenting a data finding out loud in two minutes without a single unsupported claim

Projects built:
- Produce a complete visual story on your own dataset: one question, three charts chosen deliberately, written commentary on each, exported as a PDF report

Practice: Give your two minute presentation to someone at home and note every question they asked that your report did not answer.
Assessment: Your exported PDF report plus the list of questions your audience asked.

PHASE 3: YOUR FIRST MACHINE LEARNING MODELS

Classes 11 to 16. Training a model in Orange takes about thirty seconds. Knowing whether the model is any good takes the rest of this phase, and it is the part that actually matters. Every model you build here is tested properly before you are allowed to be impressed by it.

Class 11: Supervised Learning and the Baseline You Must Beat
Topics:
- Supervised learning defined: you have examples where the answer is already known, and you want a model that generalises to examples where it is not
- Classification versus regression: predicting a category versus predicting a number, and how the target variable decides which you are doing
- Setting the target with Select Columns, and the discipline of checking your target before every model
- The standard Orange modelling shape: data, then a learner widget, then Test and Score, then results
- The Constant widget: a deliberately stupid model that always predicts the most common class, and why it is the most useful widget in the toolbox
- Why a baseline is non-negotiable: a model that is 92 percent accurate on data that is 92 percent one class has learned nothing at all
- Class imbalance in one picture, using Distributions on the target variable before you model

Projects built:
- Take a classification dataset, chart its target distribution, run the Constant baseline through Test and Score, and write down the accuracy number that every later model in this course must beat

Practice: Find a dataset where the majority class is above 80 percent and state what accuracy would prove nothing.
Assessment: Your baseline score plus a written explanation of why accuracy alone can be a dishonest metric.

Class 12: Decision Trees, the Model You Can Actually Read

Topics:
- How a decision tree learns: repeatedly finding the split that best separates the classes, then repeating on each side
- The Tree widget and its settings: minimum instances in a leaf, maximum depth, and when to stop splitting
- Tree Viewer: walking the actual tree your data produced, node by node, and reading the rule at each branch
- Pythagorean Tree and Pythagorean Forest: seeing tree structure and forest diversity as shapes
- Overfitting explained through trees: a tree deep enough to memorise every training row is useless on new data
- Why interpretability matters: a model a human can read is a model a human can argue with, correct and trust
- Reading a tree back as plain English rules you can say out loud

Projects built:
- Train a decision tree on your dataset, open Tree Viewer, and write out the three clearest decision rules in plain English, then deliberately overfit by removing the depth limit and describe what changed

Practice: Change minimum instances in a leaf across three values and record the tree size each time.

Assessment: Your three written rules plus a before and after comparison of the limited and overfitted trees.

Class 13: k Nearest Neighbours, Naive Bayes and Preprocessing That Matters

Topics:
- k Nearest Neighbours: classify a new point by looking at the k closest known points and taking a vote
- Choosing k, and what happens at the extremes: k of one memorises noise, a very large k predicts the majority every time
- Why kNN is the model that most needs normalisation, connecting straight back to the distance work in Class 9
- Naive Bayes: probability-based classification, and the naive independence assumption stated honestly
- Where Naive Bayes shines despite that assumption, especially on text-like data with many features
- Building preprocessing into the model pipeline with the Preprocess widget so the same treatment applies at training and prediction
- Discretize and Continuize: converting between numeric and categorical when a model requires it

Projects built:
- Compare kNN with and without normalisation in the same Test and Score, add Naive Bayes as a third learner, and write a paragraph explaining the accuracy gap that normalisation caused

Practice: Sweep k across five values and plot or tabulate how accuracy moves.

Assessment: Your three-learner comparison table plus the normalisation explanation.

Class 14: Testing Honestly with Cross Validation and Confusion Matrices

Topics:
- The cardinal sin of machine learning: scoring a model on the same data it was trained on
- Train and test split: holding data back so the model faces genuinely unseen examples
- Cross validation explained simply: split the data into k folds, train on k minus one, test on the one held out, repeat, then average
- Test and Score in practice: comparing several learners side by side under identical conditions
- Reading the metrics without cherry-picking: classification accuracy, AUC, F1, precision and recall, and what each one is blind to
- Confusion Matrix: true positives, false positives, true negatives and false negatives as a table of real consequences
- Precision versus recall through a real dilemma: a disease screening test that misses cases versus one that raises false alarms

Projects built:
- Run every model you have built so far through Test and Score with 10-fold cross validation against the Class 11 baseline, then open Confusion Matrix and describe, in consequences rather than numbers, what each type of error would mean

Practice: Take the same model, score it on training data and on cross validation, and record the difference.

Assessment: Your comparison table including the baseline, plus the written consequence analysis of your confusion matrix.

Class 15: Logistic Regression, ROC Curves and Nomograms

Topics:
- Logistic Regression: predicting a probability rather than a bare label, and what a probability of 0.7 actually commits you to
- The decision threshold: why 0.5 is a default, not a law, and how moving it trades precision against recall
- ROC Analysis: reading the curve as every possible threshold plotted at once
- AUC in one sentence: the chance that the model ranks a random positive case above a random negative one
- Performance Curve and Calibration Plot: whether the predicted probabilities can be trusted as probabilities
- Nomogram: reading a logistic regression model as a visual scoring sheet showing which

features push a prediction which way
- Regularisation in plain language: penalising complexity so the model does not chase noise

Projects built:
- Train logistic regression alongside your decision tree, compare them in ROC Analysis, then use the Nomogram to write down the three features that most influence the prediction and in which direction

Practice: Move the classification threshold and record how precision and recall trade off at three settings.

Assessment: Your ROC comparison plus the nomogram feature explanation in plain English.

Class 16: Ensembles, Neural Networks and Making Predictions

Topics:
- Random Forest: many decision trees trained on different samples, voting together, and why a crowd of weak trees beats one strong one
- Gradient Boosting: building trees in sequence where each one corrects the previous one's mistakes
- Support Vector Machines: finding the widest possible boundary between classes
- Neural Network in Orange: hidden layers, activation, and honest expectations about small datasets
- The bias and variance trade-off explained with the models you have already trained
- Predictions: feeding brand new unlabelled data into a trained model and reading the output with its probabilities
- Save Model and Load Model: keeping a trained model so you do not retrain every time

Projects built:
- Build a six-model showdown, baseline, tree, kNN, logistic regression, random forest and neural network, in one Test and Score, then take the winner, save it, and run genuinely new rows through Predictions

Practice: Note which model won on accuracy and which won on AUC, and explain why they can disagree.

Assessment: Your six-model comparison table plus a paragraph defending your model choice on grounds other than the top accuracy number.

PHASE 4: FINDING HIDDEN STRUCTURE, TEXT AND IMAGES

Classes 17 to 21. Not every dataset comes with answers attached. This phase covers unsupervised learning, where the model finds structure nobody labelled, and then applies the same canvas to two data types that do not look like spreadsheets at all: text and images.

Class 17: k Means Clustering and Silhouette Scores

Topics:
- Unsupervised learning: no target column, no right answers, just structure waiting to be found
- How k means works step by step: pick k centres, assign every point to the nearest, move each centre to the middle of its group, repeat
- The k Means widget, including the option to score a range of k values and let the silhouette guide the choice
- Silhouette score explained: how well each point sits inside its own cluster compared to the nearest other cluster
- Silhouette Plot: spotting the individual points that were forced into the wrong group
- Interactive k-Means from the Educational add-on: stepping through the algorithm one iteration at a time and watching the centres move
- The honest limitation: k means will happily produce clusters even when the data has no real groups at all

Projects built:
- Cluster a dataset with k means across a range of k, choose k using the silhouette score rather than intuition, then colour a Scatter Plot by cluster and describe each group in plain words

Practice: Run k means on deliberately random data and explain why the clusters it returns mean nothing.

Assessment: Your silhouette-justified choice of k plus a written description of what each cluster represents.

Class 18: Hierarchical Clustering, DBSCAN and Distance

Topics:
- Hierarchical Clustering: building a tree of nested groups instead of committing to a fixed number
- Reading a dendrogram: where to cut it, and how the cut height decides how many clusters you get
- Linkage methods, single, average, complete and Ward, and how the choice changes the shape of the result
- The Distances widget: Euclidean, Manhattan, cosine and others, and matching the measure to the data
- Distance Map and Distance Matrix: seeing similarity between every pair of rows as an image
- DBSCAN: density-based clustering that finds arbitrary shapes and, unlike k means, is allowed to call a point noise
- Louvain clustering and Self-Organizing Map named as further options for later exploration

Projects built:
- Cluster the same dataset three ways, k means, hierarchical and DBSCAN, put the results side by side, and write which method matched the structure you can see in the scatter plot and why

Practice: Cut the same dendrogram at three different heights and record how the cluster count and meaning change.
Assessment: Your three-method comparison plus a justified recommendation of one method for this dataset.

Class 19: PCA, t-SNE and Outliers
Topics:
- The curse of dimensionality in one example: why fifty columns can be harder to work with than five
- PCA: building new combined axes that capture as much of the variation as possible, in order
- Reading the explained variance curve to decide how many components are enough
- What you lose with PCA: components are combinations, so individual feature meaning becomes harder to interpret
- t-SNE and MDS: squeezing high-dimensional data into two dimensions for visualisation, and why the distances between separate clusters in a t-SNE plot should not be over-read
- Outliers: detecting the rows that do not belong, and deciding case by case whether they are errors or the most interesting thing in the file
- Using PCA output as the input to clustering, and comparing the result against clustering the raw features

Projects built:
- Take a wide dataset, reduce it with PCA, visualise it with t-SNE coloured by a known label, run Outliers, and write about the three most unusual rows you found

Practice: Compare clustering on raw features against clustering on PCA components and note any difference in silhouette.
Assessment: Your t-SNE visualisation plus a written investigation of the outliers you detected.

Class 20: Text Mining, from Words to Sentiment
Topics:
- Installing add-ons through Options then Add-ons, and what the Text add-on adds to the toolbox
- Corpus and Import Documents: loading a collection of documents as data, from bundled corpora to your own folder of text files
- Preprocess Text: tokenisation, lowercasing, stop word removal, normalisation and filtering, and how each step changes what the model sees
- Bag of Words: turning documents into numbers by counting words, and what that representation throws away
- Word Cloud and Concordance: the quick overview and the close reading, used together
- Sentiment Analysis: scoring documents as positive or negative, with an honest discussion of sarcasm, context and language limits
- Topic Modelling: letting the algorithm propose themes across a corpus, and the human judgement needed to name them

Projects built:
- Build a full text pipeline on a corpus you choose, book reviews, song lyrics, news headlines or your own writing, ending in a word cloud, a sentiment distribution and a set of topics you name yourself

Practice: Rerun your pipeline with stop words switched off and describe how the word cloud degraded.
Assessment: Your text workflow plus your named topics with a one line justification for each name.

Class 21: Image Analytics and Your First Image Classifier
Topics:
- The Image Analytics add-on: Import Images, Image Viewer, Image Grid, Image Embedding and Save Images
- Import Images: pointing Orange at a folder of pictures and getting a dataset where each row is an image
- Image Embedding explained without maths: a pretrained network converts each picture into a list of numbers describing what it contains
- Why embeddings matter: once an image is a row of numbers, every classifier and clustering widget you already know works on it unchanged
- Building the classifier: embeddings into logistic regression or a neural network, then straight into Test and Score
- Image Grid and clustering images by similarity, so you can see what the model thinks looks alike
- The practical warning: embedding needs a working internet connection for the server-based embedders, and small unbalanced image sets produce flattering, fragile results

Projects built:
- Collect or use a provided set of images in at least three labelled folders, embed them, train and cross validate a classifier, then test it on pictures you took yourself and honestly report where it failed

Practice: Cluster your images without labels and describe what visual property the clusters actually captured.
Assessment: Your image classification workflow with cross validated scores plus a written account of its failure cases.

PHASE 5: ETHICS, CAPSTONE AND SHOWCASE

Classes 22 to 24. A model that is technically correct and socially careless is still a bad

model. You finish the course by learning what can go wrong, then building and presenting an end to end project of your own from raw data to defended conclusion.

Class 22: Bias, Leakage and Honest Conclusions
Topics:
- Where bias enters: who collected the data, who is missing from it, and which outcome someone chose to call success
- Data leakage in plain terms: a feature that secretly contains the answer, producing a model that scores brilliantly and fails completely in the real world
- How to hunt leakage: a suspiciously perfect score is a bug report, not a celebration
- Correlation versus causation, revisited with your own charts from Phase 2 as the examples
- The Fairness add-on named as a real tool for auditing models on sensitive attributes
- The Explain add-on named as a way to ask which features drove a specific prediction
- Writing conclusions that survive scrutiny: state the finding, state the uncertainty, state what would disprove you

Projects built:
- Audit one of your own earlier workflows for leakage and bias, write a short honest report on what you found, and correct the workflow if the audit demands it

Practice: Take any published claim based on data and write down the three questions you would now ask about it.
Assessment: Your written audit, including at least one limitation of your own work stated plainly.

Class 23: Build Your Capstone
Topics:
- Choosing a capstone question that is specific, answerable with data you can actually obtain, and interesting to someone other than you
- Sourcing a dataset responsibly: open government portals, Kaggle, school or club records used with permission, and checking the licence
- Planning the pipeline on paper before touching the canvas: load, clean, explore, model, evaluate, conclude
- Building the workflow with the discipline of Phase 1: named widgets, left to right layout, nothing edited by hand outside the canvas
- Beating your baseline, and being willing to report that a fancy model did not beat a simple one
- Mentor review checkpoint: an instructor walks your canvas with you and pushes on every unjustified step
- Preparing the artefacts: the .ows file, the data file, and the report skeleton

Projects built:
- Build your complete capstone workflow end to end, from raw file to evaluated model and written conclusion, reviewed live with your mentor

Practice: Rerun your capstone from a clean Orange session to prove it reproduces without hidden manual steps.
Assessment: A working capstone .ows file that reproduces from raw data, plus your stated baseline comparison.

Class 24: Present, Publish and Where Data Takes You Next
Topics:
- Assembling the final Orange report: question, data, method, evidence, result, limitations
- Exporting for different audiences: PDF for a teacher or judge, and .report so another analyst can reopen your workflows
- Presenting in five minutes: leading with the question, showing two charts, giving the number, and naming what you are unsure about
- Handling hard questions honestly, including how to say that you do not know
- Publishing your work: a portfolio folder of 24 workflows plus the capstone, ready for a science fair, a school submission or a college application
- The bridge to code: Orange is written in Python, so the Python Script widget lets you run real Python inside the canvas, and every model here exists as a Python library too
- Where to go next: Python with pandas, statistics, further Orange add-ons like Time Series, Geo, Network and Explain, and the official Orange video tutorials and workflow examples

Projects built:
- Deliver your five minute capstone presentation to the batch, hand in the exported report, and receive written feedback plus your certificate assessment

Practice: Write a one page plan for the next data project you will build on your own, with its question, data source and baseline.
Assessment: Your final presentation, exported report and complete 24-workflow portfolio, reviewed for reasoning as much as for results.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: your first two-widget canvas, a described real dataset, a five widget messy-data repair pipeline, a merged and pivoted multi-table analysis, and a shared annotated .ows workflow a classmate successfully reran
- Phase 2: a three-chart distribution comparison, an informative-projection scatter plot with selected points, an honest versus misleading chart pair, a correlation and heat map duplicate-feature investigation, and a full exported PDF visual story
- Phase 3: a baseline accuracy every later model must beat, a readable decision tree written out as plain English rules, a kNN normalisation experiment, a 10-fold cross validated model comparison with confusion matrix consequences, an ROC and nomogram analysis, and a six-model

showdown with saved model and live predictions
- Phase 4: a silhouette-justified k means clustering, a three-method clustering comparison, a PCA and t-SNE reduction with outlier investigation, a complete text mining pipeline with named topics, and an image classifier trained on your own labelled photos
- Phase 5: a leakage and bias audit of your own earlier work, the full capstone workflow reproducing from raw data, and a five minute presentation with an exported report
- Total: 24 saved workflows plus one capstone, every one reopenable, editable and explainable

Total projects: 24 hands-on workflows, one per class, finishing with an end to end capstone project and an exported Orange report

SKILLS THE STUDENT WILL HAVE AT THE END

- Visual Programming in Orange: canvas, widget sets, connections, channel selection, and saving reproducible .ows workflows
- Data Preparation: File and Datasets, Data Table, Data Info, Edit Domain, Column Statistics, Select Columns and Rows, Impute, and honest handling of missing values
- Data Wrangling: Merge Data, Concatenate, Group by, Aggregate Columns, Pivot Table, Transpose, Formula, Data Sampler and Preprocess
- Data Visualisation: Distributions, Box Plot, Violin Plot, Scatter Plot with informative projections, Bar and Line Plot, Sieve Diagram, Mosaic Display, Heat Map, Correlations and Distance Map
- Supervised Learning: Constant baselines, Tree with Tree Viewer, kNN, Naive Bayes, Logistic Regression with Nomogram, Random Forest, Gradient Boosting, SVM and Neural Network
- Honest Model Evaluation: Test and Score with cross validation, Confusion Matrix, ROC Analysis, Performance Curve, Calibration Plot, precision and recall trade-offs, overfitting and leakage detection
- Unsupervised Learning: k Means with silhouette selection, Hierarchical Clustering and dendrograms, DBSCAN, Distances, PCA, t-SNE, MDS and Outliers
- Text and Image Mining: Corpus, Preprocess Text, Bag of Words, Word Cloud, Concordance, Sentiment Analysis, Topic Modelling, Import Images, Image Embedding and Image Grid
- Communicating Results: the Orange Report system, exporting to PDF and .report, and presenting a finding with its limitations stated

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live classes per week, about 1 hour each, across 12 weeks
Hands on canvas work: 1-1.5 hours rebuilding each class workflow on your own machine with your own data
Project time: 1 hour completing each class deliverable, from cleaned datasets to evaluated models and reports
Review and doubts: 30-60 minutes reviewing your workflows, checking your conclusions against your evidence, and clearing doubts before the next class
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes for installation problems, add-on issues, and workflows that will not behave
Small batches: Small batches, with mini-batch and 1-on-1 options, so every student gets individual attention on their own canvas
Mentor guidance: Live guidance from mentors who build and evaluate real models, including a personal review of your capstone workflow
Project feedback: Feedback on every class deliverable, judged on reasoning and honest evaluation as much as on results
Living syllabus: Curriculum updated as Orange ships new versions and widgets, at no extra cost

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Data-literate student who can take a raw dataset from file to defended conclusion without waiting for anyone to write code
- Strong entrant for science fairs, school research competitions and project-based coursework that rewards genuine method
- Prepared starting point for Python, pandas and scikit-learn, since the modelling and evaluation concepts are already understood
- Early groundwork for university study and later roles in data analysis, machine learning, research, bioinformatics and business analytics
- A portfolio of 24 reproducible workflows and one capstone that demonstrate reasoning and honesty, not just tool use

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Does my child need to know programming before this course?
A: No. Orange is visual programming software: you build analysis by placing widgets on a canvas and connecting them, so no code is required at any point in the 24 classes. Students who already know Python can go further using the Python Script widget, which runs real Python inside the workflow, but it is entirely optional.

Q: What exactly is Orange, and is it really free?
A: Orange is open source machine learning and data visualisation software developed at the Bioinformatics Laboratory, Faculty of Computer and Information Science, University of Ljubljana. It is released under the GNU General Public License version 3 or later and is free to download and use on Windows, macOS and Linux. There is no subscription, no trial period, and no student licence to renew.

Q: Is this real machine learning, or a simplified version for children?
A: It is real machine learning. The visual canvas changes how you assemble the pipeline, not what the pipeline does.

Q: What age is this course for?
A: It is designed for teens aged 13 to 18. Orange itself requires no account and has no age restriction, so the age range reflects the reading level, the statistical reasoning and the project expectations of the curriculum rather than any restriction in the software. Younger students who are strong in maths can join after a free demo class where we check the fit. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: What kind of computer is needed?
A: Any reasonably modern Windows, macOS or Linux laptop or desktop that can install Orange, plus a stable internet connection for the live classes. Most classes run comfortably on modest hardware because the datasets are student sized. The image analytics class in Phase 4 also needs a working internet connection, since the image embedders run on a server.

Q: How is this different from your Python data science course for teens?
A: The Python course teaches you to write the analysis in code with pandas and Python libraries, which means you spend the early weeks on syntax. This course removes the syntax barrier so students reach models, evaluation and clustering in the first month, and it goes deeper on honest evaluation and visual exploration. Many students take Orange first and then move to Python, arriving with the concepts already in place.

Q: Will my child be able to use this for a school or science fair project?
A: Yes, and Class 23 and Class 24 are built specifically for that. Students choose their own question, source a dataset responsibly, build a reproducible workflow, compare against a baseline, and export an Orange report as PDF for a teacher or judge, or as a .report file so another analyst can reopen the underlying workflows. The whole capstone is designed to be defended out loud.

Q: What are add-ons, and are they included?
A: Add-ons are optional widget packs installed from inside Orange through Options then Add-ons. This course installs and uses the Text add-on in Class 20 and the Image Analytics add-on in Class 21, and introduces the Educational, Explain and Fairness add-ons where they are relevant. They are free, like Orange itself, and we install them together in class.

Q: What does my child actually walk away with?
A: Twenty four saved .ows workflows they can reopen and edit, a capstone project built from raw data to written conclusion, an exported report, a Modern Age Coders completion certificate, and, more usefully than any of those, the habit of asking how a model was tested before believing what it claims.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final project defended live, plus a written assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/orange-data-mining-visual-machine-learning-course-for-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 110 OF 137: PCAP CERTIFICATION PREP: CERTIFIED ASSOCIATE IN PYTHON PROGRAMMING

==========

Course link: https://learn.modernagecoders.com/courses/pcap-python-certification-prep-course
Subject area: Python Certification
Designed for: Working professionals

Who can join: Anyone with basic Python (PCEP level or equivalent) preparing for the PCAP associate certification
Course length: 4 months (16 weeks), joinable any month
Time needed: 2 live classes/week + weekly coding practice and mock questions
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Live online PCAP-31-03 preparation covering every exam objective: modules and packages, exceptions, strings, object-oriented Python, generators, closures and file processing, with full timed mock exams.

  In one line: Every exam objective taught properly, then drilled against the real format, so you sit the exam already knowing you will pass.

ABOUT THE PROGRAMME

  PCAP, the Certified Associate in Python Programming, is the Python Institute's associate-level credential and the natural step up from PCEP. It is a genuine test of Python fluency rather than a quiz: the current PCAP-31-03 exam is 40 questions in single-choice and multiple-choice form, taken in 65 minutes, with a pass mark of 70 percent, which means at least 28 correct answers. It is delivered through Pearson VUE test centres or OnVUE online proctoring, and the exam fee is set by the Python Institute (currently around USD 295), paid directly to them. What makes PCAP harder than it looks is that it examines the parts of Python most self-taught programmers skip. Modules and packages. Exception handling done properly, not a bare try block. String methods in detail. Object-oriented Python including class variables and inheritance. Generators, closures and list comprehensions. File processing. Plenty of working developers write Python daily and still miss these, because everyday scripting never forces them. This course covers every objective in that list and treats the exam format as a skill of its own. You write real code every week, then answer questions in the exact style the exam uses, including the ones designed to punish a guess. The last month is full timed mocks with question-by-question review, so by the time you book the test you already know your score range.

WHAT MAKES THIS COURSE DIFFERENT

  - Built around the current PCAP-31-03 objectives: modules and packages, exceptions, strings, OOP, generators and closures, and file processing
  - Taught as real Python first and exam technique second, so the knowledge survives after the certificate
  - The exam format is practised deliberately: 40 questions in 65 minutes at a 70 percent pass mark leaves no room for slow reading
  - Covers the topics self-taught programmers usually miss, which is exactly where PCAP candidates fail
  - Full timed mock exams with question-by-question review and an error log, so weak areas are fixed before test day
  - Live, small batches where your code is reviewed on the spot, not recorded videos watched alone

THE LEARNING PATH, STEP BY STEP

  Phase 1: Modules, packages, strings and exception handling done to exam depth
  Phase 2: Object-oriented Python: classes, class variables, inheritance, polymorphism and special methods
  Phase 3: Generators, closures, comprehensions, file processing, then full timed mock exams

WHAT A STUDENT NEEDS BEFORE STARTING

  Title: What You Need to Start

WHO THIS COURSE IS FOR

  Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

  This is the complete teaching plan. Parents can ask about any week.

  PHASE 1: MODULES, PACKAGES, STRINGS AND EXCEPTIONS

  The first block of exam objectives, taught as real Python and drilled in exam form.

Month 1: Modules, Packages and the Standard Library (Weeks 1-4)
  Importing and Module Basics
    Topics:
      - import, from-import, and aliasing, and how each affects the namespace
      - What actually happens when Python imports a module
      - The __name__ variable and the main guard
      - Where Python looks for modules, and sys.path
      - Exam traps around name collisions and shadowing
    Projects built:
      - Write a small multi-module program and import across it every supported way
    Practice: 20 exam-style questions on imports and namespaces
  Standard Library Essentials
    Topics:
      - math, random, platform and datetime as the exam uses them
      - Reading the documentation quickly and correctly
      - Predicting output from unfamiliar standard-library calls
      - Common function signatures worth memorising
      - Where the exam prefers a specific idiom
    Projects built:
      - Solve a set of tasks using only the standard library, no third-party packages
    Practice: 20 output-prediction questions on standard-library calls
  Packages, __init__ and Distribution
    Topics:
      - How a package differs from a module
      - The role of __init__.py
      - Nested packages and relative versus absolute imports
      - pip and dependency basics at exam depth
      - Structuring a small package correctly
    Projects built:
      - Build a small package with nested sub-packages and import from it correctly
    Practice: 15 package-structure and import questions
    Assessment: Timed section test on modules and packages
  Strings in Depth
    Topics:
      - Strings as immutable sequences, and what that implies
      - Slicing, including negative indices and steps
      - The string methods the exam actually tests, in detail
      - Escape sequences, raw strings and formatting
      - Comparing strings, and the ordering rules
    Projects built:
      - Write a text-processing utility using only string methods, no regular expressions
    Practice: 25 string-method and slicing questions, mostly output prediction

Month 2: Exception Handling and Consolidation (Weeks 5-8)
  The Exception Hierarchy
    Topics:
      - try, except, else and finally, and the exact order they run in
      - The built-in exception hierarchy and why the order of except clauses matters
      - Catching specific exceptions instead of bare except
      - raise, and re-raising inside a handler
      - Predicting which handler catches what
    Projects built:
      - Take a fragile script and make it robust with correctly ordered handlers
    Practice: 20 exception-flow questions, mostly trace-the-output
  Custom Exceptions and Defensive Code
    Topics:
      - Defining your own exception classes
      - Inheriting from the right base exception
      - assert and where it belongs
      - Exception chaining and context
      - Writing code that fails clearly rather than silently
    Projects built:
      - Design a small exception hierarchy for a described domain and use it properly
    Practice: 15 custom-exception questions
    Assessment: Timed section test on exception handling
  Data Aggregates Refresher
    Topics:
      - Lists, tuples, dictionaries and sets, and the exam-relevant differences
      - Mutability and the aliasing traps the exam loves
      - Copying: assignment versus shallow versus deep
      - Dictionary methods and safe key access
      - Sorting with keys and reverse
    Projects built:
      - Solve a data-shaping task choosing the right structure at each step and justifying it
    Practice: 25 questions on aggregates, heavy on mutability traps
  Functions, Scope and Arguments
    Topics:
      - Positional, keyword and default arguments
      - *args and **kwargs and how they bind

- Scope, the LEGB rule, and the global and nonlocal keywords
- The mutable-default-argument trap
- Recursion at exam depth

Projects built:

- Write functions exercising every argument form, then predict their behaviour before running them

Practice: 25 scope and argument-binding questions

Assessment: End-of-phase test across everything so far, in exam format

---

PHASE 2: OBJECT-ORIENTED PYTHON

The block that decides most PCAP results, taught until class behaviour is predictable rather than surprising.

Month 3: Classes, Inheritance and Special Methods (Weeks 9-12)

Classes, Instances and Attributes

Topics:

- Defining classes, __init__, and instance attributes
- Instance variables versus class variables, the single biggest PCAP trap
- How attribute lookup actually resolves
- Name mangling and the single and double underscore conventions
- Adding attributes dynamically and what that does

Projects built:

- Build a class where class and instance variables are deliberately contrasted, then predict every output

Practice: 25 questions on class versus instance attributes

Inheritance and Polymorphism

Topics:

- Single and multiple inheritance
- Method resolution order and how Python decides
- Overriding, and calling the parent with super()
- isinstance and issubclass, and when each is correct
- Polymorphism in practice

Projects built:

- Design a class hierarchy with overriding and demonstrate the resolution order

Practice: 25 inheritance and MRO questions

Special Methods and Introspection

Topics:

- __str__ and __repr__ and the difference the exam tests
- Operator overloading with special methods
- __dict__, and inspecting objects at runtime
- Properties and controlled attribute access
- Composition versus inheritance

Projects built:

- Write a class with operator overloading and full introspection support

Practice: 20 special-method questions

Assessment: Timed OOP section test in exam format

OOP Consolidation

Topics:

- Working through the OOP question styles the exam repeats
- Reading unfamiliar class code quickly and accurately
- Spotting the deliberate misdirection in the options
- Common wrong answers and why they look right
- Building speed without losing accuracy

Projects built:

- A full OOP-only mock section, reviewed question by question

Practice: 40 mixed OOP questions under time

PHASE 3: GENERATORS, CLOSURES, FILE PROCESSING AND FULL MOCKS

The last objectives, then the exam itself rehearsed until the result is predictable.

Month 4: Advanced Python and Exam Rehearsal (Weeks 13-16)

Comprehensions, Generators and Iterators

Topics:

- List, dictionary and set comprehensions, including conditions
- The iterator protocol: __iter__ and __next__
- Generator functions and the yield keyword
- Generator expressions and lazy evaluation
- Why a generator is not a list, and what breaks if you forget

Projects built:

- Rewrite an eager data pipeline as a lazy generator pipeline and compare behaviour

Practice: 25 comprehension and generator output questions

Closures, Lambdas and Functional Tools

Topics:

- Lambda functions and their limits
- Closures, and what a nested function captures

- nonlocal in a closure
- map, filter and sorted with key functions
- Functions as first-class objects

Projects built:
- Build a small closure-based counter and function factory

Practice: 20 closure and lambda questions

Assessment: Timed section test on generators and closures

File Processing

Topics:
- Opening files: modes, and text versus binary
- Reading with read, readline and readlines, and iterating a file
- Writing and appending safely
- The with statement and why it is the correct idiom
- Handling file exceptions properly, tying back to Phase 1

Projects built:
- Write a program that reads, transforms and writes files with full error handling

Practice: 20 file-processing questions

Full Timed Mock Exams

Topics:
- Full mocks at the real format: 40 questions in 65 minutes
- Pacing so no question is left unread, since there is no penalty for answering
- Eliminating options systematically on multiple-response questions
- Turning the error log into a final revision list
- Booking the exam: Pearson VUE test centre or OnVUE online proctoring

Projects built:
- Three complete timed mock exams, each reviewed question by question

Practice: Repeated full mocks until you clear 70 percent consistently with margin

Assessment: Final full mock scored against the 70 percent pass mark, with a go or no-go recommendation

HOW A TYPICAL WEEK IS SPLIT

Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- PCAP certification, a vendor-recognised associate-level Python credential
- Junior Python developer, automation engineer or QA automation roles
- A solid base for data analysis and backend development
- A credible step toward the professional-level Python certifications
- Genuine intermediate Python fluency that outlives the certificate

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What exactly is the PCAP exam?
A: PCAP is the Certified Associate in Python Programming from the Python Institute. The current version, PCAP-31-03, is 40 questions in single-choice and multiple-choice form, taken in 65 minutes, with a pass mark of 70 percent, meaning at least 28 correct answers. It is delivered through Pearson VUE test centres or OnVUE online proctoring.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: What does PCAP actually test?
A: The objectives cover modules and packages, exception handling, strings, object-oriented programming including class variables and inheritance, generators and closures, list comprehensions, and file processing, along with data aggregates like tuples and dictionaries. It is precisely the intermediate Python that self-taught programmers tend to skip, which is why candidates who write Python daily still fail it.

Q: Do I need PCEP first?
A: No. PCEP is helpful but not required. What you do need is working basic Python: variables, conditionals, loops and simple functions. If you have that, this course takes you from there to PCAP standard. If you are starting from zero, begin with a foundational Python course first and come to this one after.

Q: How long does it take to be ready?

A: The course runs four months across 16 weeks with two live classes a week plus weekly practice. Readiness depends on your starting point and how much you code between classes. We do not guess: the final month is full timed mocks, and we recommend booking the exam only once you are consistently clearing 70 percent with margin.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/pcap-python-certification-prep-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 111 OF 137: PCEP AND PCAP PYTHON CERTIFICATION COURSE
==========

Course link: https://learn.modernagecoders.com/courses/pcep-pcap-python-certification-course
Subject area: Python Certification
Designed for: College students
Who can join: Anyone from about 13 upward, including working adults; no prior programming required for the PCEP half
Course length: 5 months (20 weeks) for PCEP then PCAP, joinable any month
Time needed: 2 live classes/week + weekly programming practice and timed question sets
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online preparation for the Python Institute PCEP and PCAP certifications, built so that the repository a student finishes with is worth more than the certificate, and the certificate still passes.

ABOUT THE PROGRAMME

Python Institute certifications are useful in specific situations and oversold in most others. They give a self-taught learner an external checkpoint, they satisfy some employers and some institutions that want a named credential, and they impose a syllabus on someone who would otherwise wander. What they do not do is demonstrate that a person can build anything, because a multiple-choice exam cannot show that. So this course prepares both certifications properly and insists on something the exams do not require: a public repository of working programs built along the way. Students who finish have PCEP and PCAP if they want them, and something a reader can open either way. PCEP-30-02 is the entry-level exam and PCAP-31-03 the associate one; both are sat through the Python Institute rather than through us, and the current blueprint should be confirmed at pythoninstitute.org before booking.

WHAT A STUDENT NEEDS BEFORE STARTING

Title: What a Student Needs to Start

WHO THIS COURSE IS FOR

Title: Who This Course Is For

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PCEP FOUNDATIONS (MONTHS 1-2)

Entry-level Python taught properly rather than as exam trivia, with every topic implemented before it is tested.

Month 1: Syntax, Data Types and Control Flow (Weeks 1-4)
The Interpreter, Literals and Variables
Topics:
- How Python actually runs a file, which the exam asks about indirectly
- Literals, numeral systems and the operators students misjudge
- Variables, naming rules and the shadowing that trips people up
- Reading an error message properly rather than rerunning and hoping
Operators, Precedence and Type Conversion
Topics:
- Arithmetic, bitwise and logical operators with precedence rules
- Integer division, modulo and the negative-number cases
- Implicit and explicit type conversion, a reliable exam topic

        - Predicting the output of an expression rather than running it
    Conditionals and Loops
      Topics:
        - if, elif and else, and the nested conditions exams favour
        - while and for, with break, continue and the else clause on loops
        - The loop-else construct, which almost every candidate gets wrong first
        - Tracing loops on paper before running them
    Lists, Tuples, Dictionaries and Sets
      Topics:
        - Mutability, which underlies a large share of PCEP questions
        - Slicing, and the negative-index cases
        - Dictionaries and sets, with the operations the syllabus names
        - Copying against referencing, and why it matters in the exam

  Month 2: Functions, Scope and Exceptions (Weeks 5-8)
    Functions and Parameters
      Topics:
        - Positional, keyword and default parameters
        - Return values, and functions that return nothing
        - Recursion at the level PCEP expects
        - Writing a function to a stated specification
    Scope, Shadowing and the global Keyword
      Topics:
        - Local and global scope, and how Python resolves a name
        - The global keyword and when using it is a mistake
        - Shadowing, which produces the exam's favourite trick questions
        - Predicting output across nested scopes
    Exceptions and Data Handling
      Topics:
        - try, except, else and finally in the order they execute
        - The exception hierarchy at PCEP depth
        - Raising exceptions deliberately, and when to
        - Basic input and output handling
    PCEP Timed Practice and the First Repository
      Topics:
        - Full timed question sets under exam conditions
        - Reviewing every wrong answer with the reason named
        - Committing the first real program to a public repository
        - Deciding whether to sit PCEP now or go straight on

PHASE 2: PCAP ASSOCIATE MATERIAL (MONTHS 3-4)

Modules, packages, object-oriented programming, exceptions in depth, and the standard library
the associate exam assumes.

  Month 3: Modules, Packages and Classes (Weeks 9-12)
    Modules, Packages and the Import System
      Topics:
        - import in its several forms, and what each actually binds
        - Writing your own module and then your own package
        - The standard library modules the syllabus names
        - Why the import questions are free marks for anybody who has written a package
    Classes, Objects and Attributes
      Topics:
        - Instance against class attributes, a heavily examined distinction
        - Constructors, methods and self
        - Introspection: examining an object at runtime
        - Building a class from a written specification
    Inheritance and Polymorphism
      Topics:
        - Single and multiple inheritance, and method resolution
        - Overriding, and calling the parent implementation
        - isinstance and issubclass, which the exam uses constantly
        - Designing a small hierarchy that is not over-engineered
    Exceptions as Objects
      Topics:
        - The exception hierarchy in full at PCAP depth
        - Defining your own exception classes
        - Chaining, and what finally guarantees
        - Deliberately breaking a program to see how it reports

  Month 4: Files, Strings and the Standard Library (Weeks 13-16)
    File Handling and Streams
      Topics:
        - Text and binary streams, opened and closed properly
        - Reading and writing patterns the exam expects
        - The with statement and why it exists
        - Handling a file that is not there

```
      Strings in Depth
        Topics:
          - String methods the syllabus names, used rather than memorised
          - Escape sequences and formatting
          - Character encoding at the level PCAP asks for
          - String questions that are really about immutability
      List Comprehensions, Lambdas and Closures
        Topics:
          - Comprehensions, including nested and conditional forms
          - Lambda functions, map and filter
          - Closures, which are the topic candidates most often fail
          - Reading a comprehension aloud as a loop and back again
      PCAP Timed Practice and the Portfolio
        Topics:
          - Full timed question sets under exam conditions
          - Reviewing every wrong answer with the reason named
          - A second, larger program committed to the repository
          - A README that states what the program does and where it fails

  ---------------------------------------------------------------
  PHASE 3: EXAM READINESS AND REAL EVIDENCE (MONTH 5)
  ---------------------------------------------------------------
  The final stretch: passing the exam, and finishing with something worth more than the
  certificate.

    Month 5: Final Preparation (Weeks 17-20)
      Full Mock Exams
        Topics:
          - Timed papers in the exam's format and question style
          - Pacing, since both exams are short relative to their question count
          - Eliminating options rather than searching for the right one
          - Identifying the two or three topics still costing marks
      Targeted Repair
        Topics:
          - Rebuilding the weakest topics rather than revising everything
          - Closures, scope and mutability, which is where most candidates lose
          - Predicting output as a drill rather than as an exercise
          - Short daily retrieval instead of long revision blocks
      The Repository That Outlasts the Certificate
        Topics:
          - Three finished programs, running, documented and committed
          - A README a stranger can follow without the author present
          - One honest paragraph on where each program fails
          - Why this is the part an employer or admissions reader can actually assess
      Booking, Sitting and What Comes After
        Topics:
          - How the exam is booked and delivered through the Python Institute
          - Confirming the current blueprint at pythoninstitute.org before booking
          - What to do in the first two minutes of the exam
          - Whether PCPP is worth attempting, discussed honestly

HOW A TYPICAL WEEK IS SPLIT

  Homework: A task or problem set after every class, and a monthly mixed-review assignment that
    deliberately folds in earlier topics

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall
    short and there is focused revision and a free retest: the certificate certifies real
    learning, not attendance

WHERE THIS CAN LEAD

  - PCEP and, if taken, PCAP certification from the Python Institute
  - A public repository of finished, documented, running programs
  - Genuine Python fluency including object-oriented programming and the standard library
  - A foundation for data science, automation or backend development
  - The habit of committing work as it is written, which outlives any certificate

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What is the difference between PCEP and PCAP?
  A: PCEP-30-02 is the entry-level certification, covering fundamentals: data types, control
  flow, functions, basic data structures and exceptions. PCAP-31-03 is the associate level and
  adds modules and packages, object-oriented programming, exceptions in depth, file handling,
  string processing and functional features like comprehensions, lambdas and closures. Most
```

students take them in that order, and some go straight to PCAP.

Q: Do you sell the exam voucher?
A: No, and we take no commission on one. You book and sit the exam through the Python Institute directly. We would rather teach you than resell you a voucher, and it also means our advice about whether to sit it at all is not affected by whether we get paid when you do.

Q: How long does it take?
A: Roughly twenty weeks at two classes a week for both certifications, joinable in any month. A student aiming only at PCEP normally finishes the first eight to ten weeks. Adults with existing programming experience often go considerably faster one to one, since much of the PCEP material is revision for them.

Q: What is the hardest part of PCAP?
A: Closures, scope resolution and mutability, consistently. They are the topics where candidates who can write working Python still fail exam questions, because the questions test prediction rather than production. The final phase of this course drills predicting output specifically, which is a different exercise from writing code.

Q: Can a school student take these?
A: Yes, from about thirteen, and several of ours have. It works well as an external checkpoint alongside a school curriculum that does not offer one, and the repository built along the way is more useful for university applications than the certificate itself. We say that to families before they enrol rather than afterwards.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/pcep-pcap-python-certification-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 112 OF 137: PRINCESS CODERS: CODING COURSE FOR GIRLS AND WOMEN
==========

Course link: https://learn.modernagecoders.com/courses/princess-coders-complete-coding-masterclass
Subject area: Women in Tech
Designed for: Girls only batches
Who can join: Complete beginner; school girls, college students and women switching careers
Course length: 9-11 months (36-44 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Coding course for girls and women: HTML, CSS, JavaScript, React and backend basics in supportive live online classes, zero to full-stack in 9-11 months.

In one line: A supportive room changes everything: girls and women learning to code together, from first tag to a deployed full-stack project in nine honest months.

ABOUT THE PROGRAMME

Women remain outnumbered in tech for reasons that have nothing to do with ability and a lot to do with rooms. Princess Coders fixes the room: live online batches of girls and women learning together, taught the complete beginner-to-builder arc: HTML, CSS and JavaScript foundations, modern React with TypeScript, backend basics with Node and databases, and a deployed capstone, with a dedicated week on women-in-tech community, mentorship and visibility. The teaching is rigorous and the tone is warm; those were never opposites. Homework follows every class, monthly assignments mix new material with earlier topics so skills compound, each phase ends with an exam, and the course ends with a final exam that decides the certificate, with a free retest after revision. Nine months at two live classes a week plus three to four hours of practice, with two more months in hand when needed. Real code, real support, real momentum.

WHAT MAKES THIS COURSE DIFFERENT

- The room is the feature: girls and women learning together, taught by instructors who take them seriously
- The complete arc: foundations to React to backend basics to a deployed capstone, not a taster
- Modern stack throughout: JavaScript, React, TypeScript, Node and real deployment
- A women-in-tech week that is practical: community, mentorship and visibility, not slogans
- An honest 9-month arc with 2 months in hand: nobody is rushed
- Real assessment: weekly homework, monthly mixed reviews, and a final exam that decides the certificate, with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Months 1-3): HTML, CSS and JavaScript learned properly, ending with creative deployed projects
Phase 2: Modern frontend (Months 4-6): React, state, UI craft, TypeScript and testing
Phase 3: Backend basics (Months 7-8): Node, Express, databases, auth and deployment
Phase 4: The finale (Month 9): the signature capstone, the women-in-tech week, and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The course starts at the first tag
Equipment: A computer with a stable internet connection; every tool used is free
Audience: Girls and women: school students (13+), college students and career switchers
Time commitment: 2 live classes plus 3-4 hours of practice a week

WHO THIS COURSE IS FOR

Students: School and college girls exploring tech seriously for the first time
Career switchers: Women moving into tech from any field, wanting a complete, supported arc
Restarters: Women returning after a career break, rebuilding skills in a friendly room
Builders: Women with a product idea who want the ability to build it
Community seekers: Anyone for whom the right room is the missing ingredient

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATIONS (MONTHS 1-3, WEEKS 1-12)

HTML, CSS and JavaScript built properly, with creative projects deployed from the first months.

Month 1: First Steps (Weeks 1-4)
Welcome: Your Tech Journey Begins
Topics:
- Welcome to the sisterhood: meet your cohort
- Famous women in tech: inspiration and role models
- How technology shapes our world
- Setting up your princess coding workspace
- Choosing your coding avatar and username
- Introduction to computers and how they think
- Your first line of code: 'Hello, Princess Coder!'
- Visual programming with Scratch (for younger cohort)
- Introduction to problem-solving like a programmer
- Creating your first digital art with code
- Building your coder confidence journal
- Princess Coders community guidelines and support
Projects built:
- Create your Princess Coder profile page
- Design your digital business card
- Build an 'About Me' interactive story
Practice: Daily: 30 min creative coding challenges, 1 hour guided tutorials
HTML: Real Structure
Topics:
- HTML as the structure of the web
- Creating your first webpage from scratch
- Text elements: telling your story online
- Images: adding visual magic to pages
- Links: connecting your digital world
- Lists: organizing information beautifully
- Tables: presenting data elegantly
- Forms: collecting information from users
- Semantic HTML: writing meaningful code
- Accessibility: making web inclusive for all
- HTML5: modern web capabilities
- Best practices for clean, organized code
Projects built:

```
      - Personal blog about your interests
      - Recipe collection website
      - Digital diary or journal
      - Online portfolio for your hobbies
    Practice: Create 5 different themed webpages
    Assessment: Month 1 check: a page built live from a sketch, plus a take-home mixing
    everything so far

Month 2: Style and Logic (Weeks 5-8)
  CSS: Design That Holds
    Topics:
      - CSS: the art of web design
      - Colors: creating beautiful palettes
      - Fonts: typography that speaks
      - Spacing: the importance of breathing room
      - Backgrounds: setting the mood
      - Borders and shadows: adding depth
      - CSS animations: bringing pages to life
      - Responsive design: mobile-friendly sites
      - CSS Grid: magazine-style layouts
      - Flexbox: flexible arrangements
      - Design principles for non-designers
      - Creating your personal style guide
    Projects built:
      - Fashion lookbook website
      - Interactive greeting card creator
      - Photo gallery with filters
      - Animated story presentation
    Practice: Style 10 different page components creatively
  JavaScript: First Interactivity
    Topics:
      - JavaScript: the language of interaction
      - Variables: storing information
      - Data types: numbers, text, true/false
      - Functions: creating reusable magic
      - Events: responding to user actions
      - Conditionals: making decisions in code
      - Loops: repeating with efficiency
      - Arrays: organizing collections
      - Objects: grouping related data
      - DOM: controlling page elements
      - Creating interactive forms
      - Building simple games and quizzes
    Projects built:
      - Interactive quiz about your favorite topic
      - Simple drawing application
      - To-do list with style
      - Mood tracker application
    Practice: Build 5 interactive mini-apps
    Assessment: Month 2 check: an interactive feature built live, plus mixed review

Month 3: JavaScript Depth (Weeks 9-12)
  Advanced JavaScript Concepts
    Topics:
      - Advanced functions and callbacks
      - Working with APIs: getting real data
      - JSON: the language of data exchange
      - Local storage: saving user preferences
      - Error handling: graceful failures
      - Asynchronous programming basics
      - Promises and modern JavaScript
      - ES6 features: arrow functions, destructuring
      - Array methods: map, filter, reduce
      - Creating reusable components
      - Code organization and best practices
      - Debugging like a detective
    Projects built:
      - Weather app with real data
      - Quote generator with API
      - Currency converter tool
      - Personal finance tracker
    Practice: Master 20 JavaScript challenges
  Creative Coding & Visual Projects
    Topics:
      - Introduction to p5.js for creative coding
      - Drawing with code: shapes and colors
      - Animation principles in programming
      - Interactive art installations
      - Generative art: code that creates
```

```
        - Sound and music with code
        - Data visualization basics
        - Creating infographics programmatically
        - Game development fundamentals
        - Physics in animations
        - Particle systems and effects
        - Sharing your creative work online
      Projects built:
        - Interactive art piece
        - Music visualizer
        - Simple platformer game
        - Data story visualization
      Practice: Create 5 artistic coding projects
      Assessment: Phase 1 exam: a creative project deployed and defended, plus a written
      mixed paper

------------------------------------------------------------
PHASE 2: MODERN FRONTEND (MONTHS 4-6, WEEKS 13-24)
------------------------------------------------------------
React the way it is written now, with UI craft, TypeScript and testing.

  Month 4: React Foundation (Weeks 13-16)
    React.js Foundation
      Topics:
        - Understanding why React revolutionized web development
        - Setting up your React development environment
        - Components: thinking in reusable pieces
        - JSX: the perfect blend of HTML and JavaScript
        - Props: component communication made simple
        - State: managing dynamic data
        - React Hooks: modern React development
        - useState for component state
        - useEffect for side effects
        - Building interactive user interfaces
        - Component composition patterns
        - React ecosystem overview
      Projects built:
        - Instagram-style photo grid
        - Pinterest-inspired board creator
        - Spotify playlist manager UI
        - Fashion e-commerce product page
      Practice: Build 15 reusable React components
    Advanced React Patterns
      Topics:
        - Advanced hooks: useContext for global state
        - useReducer for complex state logic
        - Custom hooks: creating your own
        - React Router: multi-page applications
        - Protected routes and authentication flows
        - Form handling and validation
        - API integration and data fetching
        - Error boundaries and error handling
        - Performance optimization techniques
        - React.memo and useMemo
        - Code splitting for faster apps
        - Testing React applications
      Projects built:
        - Multi-page blog with authentication
        - Task management app like Notion
        - Real-time chat application UI
        - Online learning platform interface
      Practice: Master 10 advanced React patterns
      Assessment: Month 4 check: a component set built live, plus mixed review

  Month 5: State and UI Craft (Weeks 17-20)
    State Management & Data Flow
      Topics:
        - Understanding application state
        - Context API for state sharing
        - Redux: predictable state management
        - Redux Toolkit: modern Redux
        - Actions, reducers, and store
        - Async operations with Redux Thunk
        - Zustand: lightweight alternative
        - React Query for server state
        - Optimistic UI updates
        - Caching strategies
        - Real-time data synchronization
        - State management best practices
```

```
      Projects built:
        - Shopping cart with global state
        - Social media feed with Redux
        - Collaborative workspace app
        - Analytics dashboard with real-time data
      Practice: Implement state management in 5 apps
    UI/UX Design for Developers
      Topics:
        - Design principles every developer should know
        - Color theory and creating palettes
        - Typography and readability
        - Spacing and visual hierarchy
        - User experience (UX) fundamentals
        - User interface (UI) best practices
        - Design tools: Figma basics for developers
        - Wireframing and prototyping
        - Design systems and component libraries
        - Accessibility standards (WCAG)
        - Responsive and adaptive design
        - Micro-interactions and animations
      Projects built:
        - Complete design system creation
        - Redesign existing website
        - Mobile-first application design
        - Accessible form system
      Practice: Design and implement 5 UI concepts
      Assessment: Month 5 check: a stateful mini-app live, plus mixed review

  Month 6: TypeScript and Quality (Weeks 21-24)
    TypeScript in Practice
      Topics:
        - TypeScript fundamentals
        - Types and interfaces
        - Generics and utility types
        - TypeScript with React
        - Type-safe API calls
        - Advanced TypeScript patterns
        - TypeScript configuration
        - Migration strategies
        - Type guards and assertions
        - Declaration files
        - TypeScript best practices
        - Team collaboration with TypeScript
      Projects built:
        - Full TypeScript React app
        - Type-safe API client
        - Component library in TypeScript
      Practice: Convert all projects to TypeScript
    Testing and Quality
      Topics:
        - Why testing matters
        - Unit testing with Jest
        - React Testing Library
        - Integration testing strategies
        - End-to-end testing with Cypress
        - Test-driven development (TDD)
        - Debugging techniques
        - Browser DevTools mastery
        - Performance testing
        - Accessibility testing
        - Visual regression testing
        - Continuous Integration setup
      Projects built:
        - Add tests to existing projects
        - TDD project from scratch
        - E2E test suite creation
        - Automated testing pipeline
      Practice: Write comprehensive tests for 5 projects
      Assessment: Phase 2 exam: a typed, tested feature defended, plus a cumulative paper

------------------------------------------------------------
PHASE 3: BACKEND BASICS (MONTHS 7-8, WEEKS 25-32)
------------------------------------------------------------
The other half: Node, Express, databases, authentication and a real deployment.

  Month 7: Node and APIs (Weeks 25-28)
    Backend Fundamentals with Node
      Topics:
        - What happens behind the scenes of websites
```

- Client-server architecture explained simply
- Introduction to Node.js: JavaScript everywhere
- Setting up your backend development environment
- Your first Node.js program
- Understanding modules and packages
- NPM: the world's largest software registry
- File system operations with Node.js
- Working with JSON data
- Environment variables and configuration
- Debugging Node.js applications
- Backend best practices for beginners

Projects built:
- Personal note-taking system
- File organizer application
- JSON data processor
- Command-line diary app

Practice: Build 8 Node.js utilities

Building APIs with Express.js

Topics:
- What are APIs and why they matter
- Express.js: the Node.js framework
- Creating your first Express server
- Routing: handling different URLs
- HTTP methods: GET, POST, PUT, DELETE
- Middleware: the Express superpower
- Handling JSON data and forms
- Error handling done right
- Organizing code with routers
- Static file serving
- Template engines for server-side rendering
- API testing with Postman

Projects built:
- Recipe sharing API
- Personal blog backend
- Todo list API
- Quiz application server

Practice: Create 6 different Express APIs

Assessment: Month 7 check: a small API built live, plus mixed review

Month 8: Data and Auth (Weeks 29-32)

Databases - Storing Your Data

Topics:
- Why we need databases
- Types of databases explained
- MongoDB: the document database
- Setting up MongoDB locally and cloud
- CRUD operations in MongoDB
- Mongoose: making MongoDB easier
- Designing database schemas
- Relationships between data
- Data validation and sanitization
- Database security basics
- Backup and recovery strategies
- Performance optimization tips

Projects built:
- User management database
- Product inventory system
- Social media data model
- Event planning database

Practice: Design 8 different database schemas

Authentication and Deployment

Topics:
- Why authentication is crucial
- Passwords: hashing vs encryption
- Building a registration system
- Login functionality with sessions
- JWT tokens: modern authentication
- Remember me functionality
- Password reset via email
- Social login with Google/Facebook
- Role-based permissions
- Protecting API routes
- Security best practices
- GDPR and user data privacy
- Deploying the full stack to a public URL

Projects built:
- Complete user auth system
- Multi-role application
- Social login integration

```
        - Secure member-only site
      Practice: Implement auth in all previous projects
      Assessment: Phase 3 exam: an authenticated app walkthrough from schema to URL, plus a
      cumulative paper

  ---------------------------------------------------------------
  PHASE 4: THE FINALE (MONTH 9, WEEKS 33-36)
  ---------------------------------------------------------------
  The signature capstone, the women-in-tech week, and the final exam at demo day.

    Month 9: Capstone and Community (Weeks 33-36)
      The Signature Capstone
        Topics:
          - Identifying problems worth solving
          - Project ideation for social impact
          - User research and validation
          - Creating project roadmaps
          - Agile development methodology
          - Sprint planning and execution
          - Building MVPs (Minimum Viable Products)
          - User testing and feedback
          - Iterative development process
          - Project documentation standards
          - Creating compelling demos
          - Storytelling for developers
        Projects built:
          - THE CAPSTONE: a full-stack project of her own design, deployed and documented
        Practice: Complete weekly sprints on signature project
      Women in Tech: Community, Mentorship, Visibility
        Topics:
          - History of women in technology
          - Current landscape and opportunities
          - Building confidence in tech spaces
          - Imposter syndrome and overcoming it
          - Public speaking for developers
          - Technical writing and blogging
          - Mentorship: finding and being one
          - Building your professional network
          - Women in tech communities
          - Negotiation skills for women
          - Work-life balance in tech
          - Leading technical teams
        Projects built:
          - Write technical blog series
          - Give a tech talk
          - Mentor another student
          - Join women in tech community
        Practice: Weekly leadership challenges
      Demo Day and the Final Exam
        Topics:
          - Presenting the capstone: live demo plus the hardest bug story
          - A full-course spiral review before the exam
          - Where each path leads: full-stack depth, DSA, AI, or Queen Coders as experience grows
        Projects built:
          - Demo day presentation delivered to the batch
        Assessment: FINAL EXAM: a practical build plus a written paper mixed from every phase.
        Passing earns the certificate; falling short earns focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Phase 1: deployed creative pages and interactive JavaScript projects
  - Phase 2: React apps with state, TypeScript and tests
  - Phase 3: APIs, databases and an authenticated deployed app
  - Phase 4: the signature full-stack capstone at demo day

  Total projects: 20+ real builds, crowned by a deployed full-stack capstone

SKILLS THE STUDENT WILL HAVE AT THE END

  - Web foundations: HTML, CSS and JavaScript to real depth
  - Modern frontend: React, state, TypeScript and testing
  - Backend basics: Node, Express, databases and auth
  - Shipping: deployment as a habit from phase 1
  - Community: mentorship and visibility skills that open doors

HOW A TYPICAL WEEK IS SPLIT

  Live classes: 2 live one-hour classes per week
  Homework: Homework after every class, and a mixed-review assignment at the end of every month
```

  Practice: 3-4 hours of practice between classes
  Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

  Doubt support: WhatsApp doubt support between classes
  Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the
    plan

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short
    and there is focused revision and a free retest: the certificate certifies knowledge, not
    attendance

WHERE THIS CAN LEAD

  - Full-stack depth: our dedicated full-stack course continues the road
  - Data Structures and Algorithms: the interview muscle
  - AI and generative AI courses: the modern layer
  - Queen Coders: advanced engineering and technical leadership as experience grows
  - Real projects now: sites and tools for work, family and community

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What is Princess Coders, and who is it for?
  A: A complete beginner-to-builder coding course taught in batches of girls and women: school
  students (13+), college students and career switchers learning together in a supportive,
  serious room. The arc runs HTML, CSS and JavaScript foundations, modern React with
  TypeScript, backend basics with Node and databases, and ends with a deployed full-stack
  capstone and a final exam.

  Q: Why a women-only coding course?
  A: Because rooms matter. Women are still outnumbered in most tech classrooms, and many learn
  faster and speak up more in a batch where that pressure is gone. The syllabus is identical in
  rigor to any of our courses, the exams are real, and the community that forms tends to
  outlast graduation.

  Q: How long does it take to go from zero to building real applications?
  A: Nine months at 2 live classes a week plus 3 to 4 hours of practice, with two more months
  in hand for learners who need them. The first deployed project ships within phase 1; the
  full-stack capstone lands at month nine.

  Q: How is progress assessed? Is the certificate just for attending?
  A: No. Homework follows every class, each month ends with an assignment that deliberately
  mixes new questions with earlier topics, every phase closes with a cumulative exam, and the
  course ends with a final exam. Passing earns the certificate; falling short earns focused
  revision and a free retest.

  Q: What comes after Princess Coders?
  A: The capstone and portfolio built here carry into all of them.

  Q: Is there a free trial class?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or
  message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with
  no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at
  https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor, never pre-recorded videos.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/princess-coders-complete-coding-masterclass
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
COURSE 113 OF 137: PROBLEM SOLVING AND COMPUTATIONAL THINKING FOR KIDS
==

Course link: https://learn.modernagecoders.com/courses/problem-solving-and-computational-thinking-for-kids
Subject area: Kids Coding
Designed for: Kids
Who can join: Complete beginner, ages 7 to 12
Course length: 6-8 months (26-32 weeks)
Time needed: 2 live classes/week + 2-3 hours practice

Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Problem solving course for kids aged 7-12: computational thinking, logic puzzles and real-world challenges in live online classes, in 6-8 months.

  In one line: The skill under every other skill: children taught to break problems down, spot patterns and think in steps, six months that change how school feels.

ABOUT THE PROGRAMME

  Before coding, before advanced maths, before every exam technique, sits one meta-skill: knowing what to do when you do not know what to do. This live online course builds it deliberately in children aged 7 to 12: decomposition (breaking big problems into small ones), pattern recognition, abstraction and algorithmic thinking, the four pillars of computational thinking, trained through puzzles, games, unplugged activities and real-world challenges children genuinely enjoy. The transfer is the point: the same thinking that cracks a logic puzzle organizes a school project, plans a science experiment and later makes coding feel natural. Parents most often report the change in homework: less "I can't", more "let me break it down". Six months at two live classes a week plus two to three hours of playful practice, with two more months in hand when needed. Homework after every class, a mixed challenge every month, and a final exam, a challenge gauntlet plus a friendly viva, that decides the certificate with a free retest. Real thinking, really trained.

WHAT MAKES THIS COURSE DIFFERENT

  - The four pillars trained explicitly: decomposition, patterns, abstraction, algorithms, named and practiced
  - Unplugged first: much of the best thinking training happens away from screens, by design
  - Transfer is engineered: every skill is bridged to school work out loud
  - Puzzles children actually like: games, mysteries and build-challenges, not worksheets
  - An honest 6-month arc with 2 months in hand
  - Real assessment, gently done: monthly mixed challenges and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

  Phase 1: Thinking foundations (Months 1-2): logic, patterns and the art of breaking problems down
  Phase 2: Applied problem solving (Months 3-4): strategies used on real challenges, from puzzles to projects
  Phase 3: Advanced challenges and the finale (Months 5-6): harder problems, team challenges, and the exam gauntlet

WHAT A STUDENT NEEDS BEFORE STARTING

  Experience: None. Curiosity is the entry requirement
  Reading: Comfortable reading simple instructions
  Equipment: A computer or tablet with internet; paper, pencil and household objects do much of the work
  Age: 7 to 12. Batches are grouped by age
  Time commitment: 2 live classes plus 2-3 hours of playful practice a week

WHO THIS COURSE IS FOR

  Easily stuck: Children who say "I can't" before trying, this course rewires that moment
  Puzzle lovers: Kids who already adore riddles and want real training
  Pre coders: Children headed for coding who will arrive with the thinking already built
  Exam bound: Kids facing reasoning sections in school and scholarship tests
  Bright and bored: Quick children who need harder, better problems than school provides

FULL CURRICULUM, WEEK BY WEEK

  This is the complete teaching plan. Parents can ask about any week.

  THE COURSE: THINKING, TRAINED (MONTHS 1-6, WEEKS 1-26)

  Logic and patterns, applied problem solving, then advanced challenges and the exam gauntlet.

    Months 1-2: Logic and Pattern Recognition (Weeks 1-8)
      Introduction to Problem Solving

```
  Topics:
    - What is problem solving?
    - The problem-solving process
    - Understanding the problem (The 5 W's and How)
    - Breaking big problems into small pieces
    - Pattern recognition basics
    - Sequence and order
    - Following instructions precisely
    - Logic puzzles introduction
    - Sudoku for beginners (4x4)
    - Tangram puzzles
    - Tower of Hanoi introduction
    - Growth mindset: mistakes help us learn
  Projects built:
    - Create a puzzle book
    - Design a treasure hunt
    - Solve 20 logic puzzles
    - Build pattern art
  Practice: Daily: 15-minute puzzle time, pattern journaling
Algorithmic Thinking Unplugged
  Topics:
    - What is an algorithm?
    - Recipes as algorithms
    - Creating step-by-step instructions
    - Flowcharts with shapes and arrows
    - Decision trees
    - If-then thinking
    - Loops in daily life
    - Debugging: finding mistakes
    - Optimization: finding better ways
    - Parallel vs sequential thinking
    - Human robot game
    - Algorithm art and music
  Projects built:
    - Write algorithms for daily tasks
    - Create a board game with rules
    - Design a flowchart adventure
    - Program a 'human robot'
  Practice: Create 10 real-life algorithms
  Assessment: Month 1 check: a small build or challenge live, plus a fun take-home mixing
  everything so far
Introduction to Code.org
  Topics:
    - Code.org Course A basics
    - Drag and drop programming
    - Sequence of commands
    - Loops to repeat actions
    - Events and triggers
    - Debugging practice
    - Angry Birds puzzles
    - Maze solving strategies
    - Artist drawings with code
    - Pattern creation with loops
    - Harvester game logic
    - Certificate achievements
  Projects built:
    - Complete Code.org Course A
    - Create digital art with code
    - Solve 50 coding puzzles
    - Design custom maze challenges
  Practice: Daily Code.org challenges
Mathematical Problem Solving
  Topics:
    - Number patterns and sequences
    - Fibonacci in nature
    - Magic squares
    - Mathematical puzzles
    - Estimation strategies
    - Mental math tricks
    - Geometric patterns
    - Symmetry and reflection
    - Fraction puzzles
    - Time and scheduling problems
    - Money and budgeting challenges
    - Data collection and graphs
  Projects built:
    - Math puzzle collection
    - Create a math game
    - Budget planning project
```

- Data investigation project
Practice: Solve 5 math problems daily
Assessment: Month 2 check: a live challenge with questions reaching back to month 1

Months 3-4: Applied Problem Solving (Weeks 9-17)
Scratch Problem Solving
Topics:
- Scratch as a problem-solving tool
- Decomposition: breaking down projects
- Variables for storing information
- Conditional logic (if-then-else)
- Loops for efficiency
- Creating functions (My Blocks)
- Debugging with Scratch
- Testing and iteration
- Coordinate system problems
- Random number challenges
- List manipulation
- Algorithm visualization
Projects built:
- Calculator with Scratch
- Sorting algorithm visualization
- Maze solver program
- Pattern generator
Practice: Daily Scratch challenges
Game Strategy & Logic
Topics:
- Strategy game thinking
- Chess basics and tactics
- Checkers strategies
- Tic-tac-toe perfect play
- Connect Four winning strategies
- Card game probability
- Board game design
- Winning vs learning mindset
- Analyzing opponent moves
- Planning ahead
- Risk assessment
- Decision making under pressure
Projects built:
- Design a strategy game
- Chess puzzle solver
- Create game AI
- Tournament organization
Practice: Play and analyze strategy games
Real-World Problem Solving
Topics:
- Identifying problems around us
- Research and information gathering
- Brainstorming techniques
- Mind mapping
- SCAMPER method for kids
- Design thinking process
- Prototyping solutions
- Testing and feedback
- Iteration and improvement
- Presentation skills
- Team problem solving
- Environmental challenges
Projects built:
- School problem solution
- Community helper app
- Recycling game design
- Invention prototype
Practice: Identify and solve one problem weekly
Coding Challenges Platform
Topics:
- Introduction to coding challenges
- Blockly Games complete
- Lightbot puzzle game
- CodeCombat basics
- Algorithm challenges
- Time complexity basics
- Space optimization
- Code efficiency
- Common algorithms
- Sorting visualization
- Search algorithms
- Competition preparation

```
    Projects built:
      - Complete 100 coding challenges
      - Algorithm portfolio
      - Challenge leaderboard
      - Peer challenge creation
    Practice: 3 coding challenges daily
  Mid-Course Review and Showcase
    Topics:
      - Problem-solving portfolio review
      - Skills assessment
      - Logic test completion
      - Project presentations
      - Peer review process
      - Reflection and goal setting
    Projects built:
      - Comprehensive problem-solving project
      - Skills demonstration video
      - Teaching someone else
      - Progress documentation
    Assessment: Mid-course check: a showcase build plus a mixed challenge spanning months
    1-4

Months 5-6: Advanced Problem Solving (Weeks 18-26)
  Advanced Algorithms
    Topics:
      - Recursion concepts for kids
      - Divide and conquer strategy
      - Greedy algorithms explained
      - Dynamic programming basics
      - Graph problems introduction
      - Shortest path problems
      - Optimization challenges
      - Scheduling algorithms
      - Data structure basics
      - Stack and queue concepts
      - Tree structures in life
      - Algorithm complexity
    Projects built:
      - Pathfinding visualizer
      - Scheduling optimizer
      - Recursive art generator
      - Data structure game
    Practice: Implement 5 algorithms weekly
  Robotics Problem Solving
    Topics:
      - Robot logic basics
      - Sensor data processing
      - Decision making for robots
      - Path planning
      - Obstacle avoidance
      - Line following algorithms
      - Maze solving robots
      - Virtual robotics with VEXcode
      - Robot simulation
      - Real-world robot applications
      - AI in robotics basics
      - Future of robotics
    Projects built:
      - Virtual robot programmer
      - Maze solving algorithm
      - Robot rescue simulation
      - Automated system design
    Practice: Daily robot challenges
  Data and Logic Problems
    Topics:
      - Data analysis for kids
      - Survey design and analysis
      - Spreadsheet problem solving
      - Database thinking
      - Information organization
      - Logical operators (AND, OR, NOT)
      - Truth tables
      - Venn diagrams
      - Set theory basics
      - Probability problems
      - Statistics fundamentals
      - Data visualization
    Projects built:
      - School data analysis
```

- Survey project
- Probability game
- Infographic creation

Practice: Analyze data sets daily

Creative Problem Solving

Topics:

- Lateral thinking puzzles
- TRIZ for kids
- Biomimicry solutions
- Reverse engineering
- Constraint-based creativity
- Random input technique
- Six thinking hats simplified
- Morphological analysis
- SCAMPER advanced
- Innovation frameworks
- Failure analysis
- Future problem solving

Projects built:

- Innovation challenge
- Redesign everyday object
- Future city design
- Creative solution showcase

Practice: Daily creative challenges

Capstone and the Final Exam

Topics:

- Project selection
- Problem identification
- Research and planning
- Solution development
- Testing and refinement
- Documentation
- Presentation preparation
- Peer review

Projects built:

- THE CAPSTONE: a chosen grand challenge solved end to end, documented in steps, presented at demo day

Assessment: FINAL EXAM: capstone demo plus a friendly viva and a mixed quiz from the whole course. Passing earns the certificate; a child who falls short gets focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Months 1-2: logic puzzle sets, pattern hunts and first algorithm games
- Months 3-4: real-world challenges solved with named strategies
- Months 5-6: team challenges, harder gauntlets, and the capstone challenge at demo day

Total projects: 60+ solved challenges and a documented capstone problem

SKILLS THE STUDENT WILL HAVE AT THE END

- Decomposition: breaking big scary problems into small friendly ones
- Pattern recognition: spotting the shape of a problem seen before
- Abstraction: keeping what matters, dropping what does not
- Algorithmic thinking: solutions written as clear steps anyone could follow
- Grit mechanics: what to try when stuck, taught as an actual method

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, solving along with the teacher
Homework: A small challenge after every class, and a mixed gauntlet at the end of every month
Practice: 2-3 hours of playful puzzle practice between classes
Review: Every solution reviewed for thinking, not just answers

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Regular notes to parents: which thinking skills grew this month, and how to feed them at home

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam, a challenge gauntlet plus a friendly viva. A child who falls short gets focused revision and a free retest

WHERE THIS CAN LEAD

- Kids Coding: block coding feels natural once the thinking is trained

  - Mental Maths Mastery: speed built on top of structure
  - Olympiad Mathematics: for children who found the puzzles delicious
  - School reasoning sections: NSO, IMO and scholarship tests get easier
  - Everyday life: homework, projects and plans, all broken down calmly

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What exactly does a problem-solving course teach?
  A: The four pillars of computational thinking, decomposition, pattern recognition, abstraction and algorithmic thinking, trained through puzzles, games, unplugged activities and real-world challenges. Children learn named strategies for what to do when stuck, and every skill is deliberately bridged to school work: maths word problems, science projects and even planning homework.

  Q: How is this different from a coding course?
  A: It is the layer underneath coding. Coding courses teach a language; this course teaches the thinking every language assumes, which is why children who take it first find our coding courses dramatically easier. It is equally valuable for children who never code: the same skills power maths, science and exam reasoning.

  Q: How long does the course take?
  A: Six months at 2 live classes a week plus 2 to 3 hours of playful practice takes a child from zero to the demo-day capstone in trained thinking. The course keeps two more months in hand for children who need more repetition, no rushing and no penalty.

  Q: How is my child assessed? Is the certificate just for attending?
  A: No, and it is all gentle: classwork in every session, a small homework build after every class, a mixed challenge at the end of every month that revisits earlier skills, and a final exam, a capstone demo plus a friendly viva and short quiz. Passing earns the certificate; a child who falls short gets focused revision and a free retest.

  Q: Will this help with school and competitive exams?
  A: Directly: reasoning sections in school assessments, NSO/IMO-style olympiad questions and scholarship tests all draw on exactly these skills, and parents usually notice the homework difference within the first two months, less avoidance, more calm breaking-down of tasks.

  Q: Is there a free trial class?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to sit in. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real teacher, never pre-recorded videos. Children build during class and get unstuck on the spot.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/problem-solving-and-computational-thinking-for-kids
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
COURSE 114 OF 137: PROBLEM SOLVING FOR TEENS: LOGIC, DATA STRUCTURES & ALGORITHMS
==

Course link: https://learn.modernagecoders.com/courses/problem-solving-dsa-masterclass-teens
Subject area: Programming
Designed for: Teens
Who can join: Teenagers 13-18; no prior experience needed
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 3-5 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Problem-solving course for teenagers: logic and programming foundations, core data structures and algorithms, then dynamic programming and problem-solving techniques, built the way it sticks, in 10-12 months.

In one line: The thinking behind the code: logic, data structures and algorithms taught until a teenager can face an unseen problem and actually solve it.

ABOUT THE PROGRAMME

The hardest, most valuable thing in programming is not syntax, it is looking at a problem you have never seen and finding your way to a solution. This live online course builds exactly that in a teenager, step by step and at a teen pace: logic and programming foundations first, conditionals, loops, functions, the core structures (arrays, strings, hash maps, stacks, queues, linked lists, recursion), then the real data structures and algorithms, trees, heaps, graphs, sorting and searching, algorithm-design paradigms, and finally dynamic programming and the problem-solving techniques that separate confident solvers from stuck ones. Every topic is taught through problems, not lectures, and the emphasis is always on the pattern, so a new problem is solvable, not a memorized answer. A finale month builds a problem-solving portfolio and sits a final exam. The pace is honest and teen-friendly: ten teaching months at two live classes a week plus three to five hours of practice, with two more in hand. Problem sets after every class, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- Taught through problems, not lectures: teens solve from day one
- Patterns over memorization: the skill is solving unseen problems
- A real teen pace: depth without the burnout of an adult bootcamp
- The foundation serious students build on: olympiads, contests and later interviews
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations and logic (Months 1-3): programming logic and the core structures
Phase 2: Data structures and algorithms (Months 4-6): trees, graphs, sorting, algorithm design
Phase 3: Dynamic programming and techniques (Months 7-9): DP in depth plus counting and geometry basics
Phase 4: The finale (Month 10): a problem-solving portfolio and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Everything is taught from the first line
Equipment: Any computer with a stable internet connection; every tool used is free
Age: 13-18 (teenagers)
Time commitment: 2 live classes plus 3-5 hours of practice a week

WHO THIS COURSE IS FOR

Curious teens: Teenagers who like puzzles and want to think like a programmer
Olympiad hopefuls: Students aiming for coding olympiads and contests later
Strong students: Teens who have dabbled in coding and want real depth
Future cs students: Students who want a head start on university-level computer science
Interview bound later: Teens building the foundation that future coding interviews assume

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & LOGIC BUILDING (MONTHS 1-3, WEEKS 1-13)

Build unshakeable foundations. Learn to think like a programmer, master basic syntax, and solve your first 200+ problems.

Month 1: Programming Fundamentals & First Steps (Weeks 1-4)
Introduction to Problem Solving & Setup
Topics:
- What is problem solving? Why it matters for your future
- How computers think: Binary and logic gates
- Introduction to algorithms in daily life
- Setting up your coding environment (VS Code)
- Introduction to Python: Why Python for beginners
- Your first program: Hello World explained
- Variables and data types: int, float, string, boolean
- Input and output operations
- Comments and code readability
- Basic arithmetic operations
- Type conversion and casting
- Problem solving methodology: UMPIRE (Understand, Match, Plan, Implement, Review, Evaluate)

```
    Projects built:
      - Calculator for homework problems
      - Age calculator with fun facts
      - Simple chatbot that responds to inputs
    Practice: Daily: 10 basic problems on HackerRank Easy
  Conditional Logic & Decision Making
    Topics:
      - If-else statements: Making decisions in code
      - Comparison operators: ==, !=, <, >, <=, >=
      - Logical operators: and, or, not
      - Nested conditions and elif chains
      - Switch case alternatives in Python
      - Ternary operators for clean code
      - Common patterns: min/max, odd/even, divisibility
      - Edge cases and boundary conditions
      - Debugging techniques with print statements
      - Flowcharts for visualizing logic
      - Truth tables and boolean algebra basics
      - Real-world applications: Game logic, grading systems
    Projects built:
      - Grade calculator with letter grades
      - Rock-Paper-Scissors game
      - Simple adventure game with choices
      - Password strength checker
    Practice: Solve 30 conditional logic problems on CodeChef
  Loops & Iteration Mastery
    Topics:
      - For loops: Counting and ranges
      - While loops: Conditional repetition
      - Loop control: break and continue
      - Nested loops and patterns
      - Common patterns: Triangles, pyramids, diamonds
      - Iteration through sequences
      - Loop optimization techniques
      - Infinite loops and how to avoid them
      - Do-while concept using Python
      - Enumerate and range functions
      - List comprehensions introduction
      - Time complexity introduction: Big O notation basics
    Projects built:
      - Pattern printer (20+ patterns)
      - Multiplication table generator
      - Prime number checker and generator
      - Number guessing game with attempts
    Practice: Complete 40 loop problems on LeetCode Easy
  Functions & Modular Programming
    Topics:
      - Functions: Why break code into pieces?
      - Function definition and calling
      - Parameters vs arguments
      - Return values and void functions
      - Scope: Local vs global variables
      - Default parameters and keyword arguments
      - Recursive thinking introduction
      - Simple recursion: Factorial, Fibonacci
      - Helper functions and code organization
      - Lambda functions basics
      - Function documentation and docstrings
      - Testing functions with assert statements
    Projects built:
      - Math helper library (GCD, LCM, factorial)
      - Text analyzer (word count, character frequency)
      - Recursive art generator
      - Unit converter with functions
    Practice: Build 25 reusable functions for common tasks

Month 2: Data Structures Basics & Problem Patterns (Weeks 5-8)
  Lists/Arrays Fundamentals
    Topics:
      - Lists in Python: Dynamic arrays explained
      - Creating and initializing lists
      - Indexing and slicing operations
      - List methods: append, insert, remove, pop
      - List iteration patterns
      - 2D lists (matrices) basics
      - List comprehensions deep dive
      - Common operations: reverse, sort, search
      - Memory and lists: References vs copies
      - Shallow vs deep copy
```

- Time complexity of list operations
- When lists are not optimal

Projects built:
- Student gradebook system
- Tic-tac-toe game
- Matrix operations calculator
- Todo list manager

Practice: Solve 50 array problems on various platforms

Strings & Text Processing

Topics:
- Strings as character arrays
- String immutability in Python
- String methods: split, join, strip, replace
- String formatting techniques
- Regular expressions basics
- Pattern matching in strings
- Palindromes and anagrams
- String manipulation algorithms
- ASCII values and character arithmetic
- Unicode and encoding basics
- String building efficiency
- Common string problem patterns

Projects built:
- Word game (Hangman/Wordle clone)
- Text encryption/decryption tool
- Palindrome checker with multiple rules
- String parser for math expressions

Practice: Master 40 string manipulation problems

Dictionaries/Hash Maps & Sets

Topics:
- Dictionary/HashMap concept
- Key-value pairs and hashing basics
- Dictionary operations: add, update, delete
- Iterating through dictionaries
- Nested dictionaries
- Sets and set operations
- Union, intersection, difference
- When to use sets vs lists
- Frequency counting patterns
- Two-pointer technique introduction
- Sliding window basics
- Counter and defaultdict

Projects built:
- Phone book application
- Word frequency analyzer
- Inventory management system
- Duplicate finder tool

Practice: Solve 35 hashmap/set problems

Basic Algorithms & Complexity

Topics:
- Algorithm analysis: Time and space
- Big O notation deep dive
- Best, average, worst case analysis
- Linear search implementation
- Binary search on sorted arrays
- Bubble sort understanding
- Selection sort algorithm
- Insertion sort and when to use it
- Counting sort for specific cases
- Algorithm visualization techniques
- Debugging complex algorithms
- Introduction to greedy algorithms

Projects built:
- Sorting visualizer with animations
- Search engine for local files
- Algorithm complexity analyzer
- Custom sorting for specific data

Practice: Implement and optimize 10 basic algorithms

Month 3: Advanced Basics & Contest Preparation (Weeks 9-13)

Stack & Queue Mastery

Topics:
- Stack data structure and LIFO principle
- Stack implementation using list
- Stack applications: Parentheses matching
- Expression evaluation using stacks
- Queue data structure and FIFO principle
- Queue implementation methods
- Circular queue concept

- Deque (double-ended queue)
- Priority queue introduction
- Real-world applications of stacks/queues
- Monotonic stack patterns
- BFS preview with queues

Projects built:

- Browser history navigator
- Expression calculator with precedence
- Task scheduler with priorities
- Undo-redo functionality

Practice: Master 30 stack/queue problems

Recursion & Backtracking Basics

Topics:

- Recursion deep dive: How it really works
- Call stack and recursion visualization
- Base case and recursive case design
- Multiple recursion: Fibonacci trees
- Recursion vs iteration trade-offs
- Memoization introduction
- Backtracking paradigm explained
- N-Queens problem approach
- Sudoku solver logic
- Permutations and combinations
- Subset generation algorithms
- Pruning in backtracking

Projects built:

- Sudoku solver with GUI
- N-Queens visualizer
- Maze solver with path finding
- Permutation generator tool

Practice: Solve 25 recursion/backtracking problems

Mathematics for Programming

Topics:

- Number theory basics: Primes, factors
- GCD and LCM algorithms (Euclidean)
- Modular arithmetic and properties
- Fast exponentiation (Binary exponentiation)
- Sieve of Eratosthenes for primes
- Combinatorics: nCr, nPr calculations
- Probability basics in programming
- Geometric algorithms introduction
- Matrix operations and applications
- Number systems and conversions
- Bit manipulation preview
- Mathematical proof techniques

Projects built:

- Prime factorization tool
- Probability simulator
- Geometric calculator
- Cryptography basics implementation

Practice: Solve 40 math-based programming problems

Linked Lists Introduction

Topics:

- Linked list concept and need
- Node structure and pointers
- Singly linked list implementation
- Insertion: Beginning, end, middle
- Deletion operations
- Searching in linked lists
- Reversing a linked list
- Detecting cycles (Floyd's algorithm)
- Doubly linked lists
- Circular linked lists
- Linked list vs array comparison
- Common interview patterns

Projects built:

- Music playlist manager
- Train route simulator
- Memory manager simulation
- LRU cache basic implementation

Practice: Implement 20 linked list operations

Phase 1 Final Project

Topics:

- Project planning and design
- Algorithm selection strategies
- Code structure and organization
- Testing and debugging approaches
- Documentation best practices

Projects built:

- MAJOR PROJECT: Console-based puzzle game platform
- Include: Multiple game modes, scoring system, leaderboard
- Alternative: Algorithm visualizer web app
- Alternative: Competitive programming practice tool

Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 2: CORE DATA STRUCTURES & ALGORITHMS (MONTHS 4-6, WEEKS 14-26)

Master essential data structures, dive deep into algorithms, and start solving medium-level competitive programming problems.

Month 4: Trees & Advanced Sorting (Weeks 14-19)

Binary Trees Fundamentals

Topics:

- Tree terminology: Root, leaf, height, depth
- Binary tree structure and properties
- Tree representation methods
- Tree traversals: Inorder, Preorder, Postorder
- Level order traversal (BFS on trees)
- Recursive tree algorithms
- Tree construction from traversals
- Binary tree properties and validation
- Complete and perfect binary trees
- Tree serialization and deserialization
- Common tree patterns
- Parent pointers and their uses

Projects built:

- Family tree visualizer
- Expression tree evaluator
- Directory structure navigator
- Tree-based calculator

Practice: Solve 40 binary tree problems

Binary Search Trees (BST)

Topics:

- BST property and invariants
- Insertion in BST
- Searching efficiently in BST
- Deletion cases in BST
- Inorder predecessor and successor
- Minimum and maximum finding
- BST validation algorithms
- Balanced vs unbalanced BSTs
- AVL trees introduction
- Red-Black trees concept
- Self-balancing importance
- BST to sorted array conversions

Projects built:

- Dictionary with BST backend
- Autocomplete system
- Range query processor
- Database index simulator

Practice: Master 35 BST problems

Advanced Sorting Algorithms

Topics:

- Merge sort: Divide and conquer
- Quick sort and partitioning
- Heap sort introduction
- Counting sort deep dive
- Radix sort for integers
- Bucket sort applications
- Sorting algorithm comparison
- Stability in sorting
- External sorting basics
- Partial sorting techniques
- Custom comparators
- Sorting in linear time

Projects built:

- Sorting algorithm race visualizer
- Large file sorter
- Leaderboard system
- Multi-criteria sorting tool

Practice: Implement and optimize 15 sorting variations

Heaps & Priority Queues

Topics:

- Heap data structure concept
- Max heap and min heap properties
- Heap implementation with arrays

```
      - Heapify operation (sift up/down)
      - Building heap from array
      - Priority queue operations
      - Heap sort algorithm
      - K-th largest/smallest problems
      - Median finding with two heaps
      - Merge K sorted arrays
      - Dijkstra's algorithm preview
      - Custom priority functions
    Projects built:
      - Task scheduler with priorities
      - Emergency room triage system
      - Event simulation system
      - Real-time median tracker
    Practice: Solve 30 heap/priority queue problems
  Tries & String Algorithms
    Topics:
      - Trie data structure motivation
      - Trie node structure
      - Insert and search in trie
      - Prefix search and autocomplete
      - Word dictionary with wildcards
      - Longest common prefix
      - String matching algorithms
      - KMP algorithm basics
      - Rabin-Karp rolling hash
      - Z-algorithm introduction
      - Suffix arrays concept
      - Applications in text processing
    Projects built:
      - Spell checker implementation
      - T9 predictive text
      - Word search puzzle solver
      - DNA sequence matcher
    Practice: Build 20 trie-based solutions

Month 5: Graphs & Network Algorithms (Weeks 20-23)
  Graph Fundamentals
    Topics:
      - Graph theory introduction
      - Graph terminology: Vertices, edges, degree
      - Directed vs undirected graphs
      - Weighted vs unweighted graphs
      - Graph representations: Adjacency matrix
      - Adjacency list representation
      - Edge list representation
      - Graph properties: Connected, cyclic
      - Special graphs: Trees, DAGs, complete
      - Bipartite graphs
      - Graph coloring basics
      - Real-world graph applications
    Projects built:
      - Social network analyzer
      - City map navigator
      - Game state explorer
      - Dependency resolver
    Practice: Model 15 real-world problems as graphs
  Graph Traversal Algorithms
    Topics:
      - Depth-First Search (DFS) deep dive
      - DFS applications: Path finding, cycle detection
      - Breadth-First Search (BFS) mastery
      - BFS applications: Shortest path, level order
      - Connected components finding
      - Topological sorting with DFS
      - Kahn's algorithm for topological sort
      - Detecting cycles in directed graphs
      - Strongly connected components intro
      - Bridge finding in graphs
      - Articulation points
      - Graph traversal optimizations
    Projects built:
      - Maze generator and solver
      - Course prerequisite validator
      - Island counter in grid
      - Web crawler simulator
    Practice: Solve 40 graph traversal problems
  Shortest Path Algorithms
    Topics:
```

- Shortest path problem variants
- Dijkstra's algorithm in detail
- Implementation with priority queue
- Bellman-Ford for negative weights
- Floyd-Warshall all-pairs shortest path
- A* search algorithm basics
- Shortest path in unweighted graphs
- Path reconstruction techniques
- Single source vs all-pairs
- Handling negative cycles
- Applications in real world
- Optimizations and variations

Projects built:
- GPS navigation system
- Network routing simulator
- Game pathfinding AI
- Flight connection finder

Practice: Implement 5 shortest path variations

Minimum Spanning Trees

Topics:
- Spanning tree concept
- Minimum spanning tree (MST) problem
- Kruskal's algorithm with union-find
- Prim's algorithm implementation
- Union-Find data structure deep dive
- Path compression optimization
- Union by rank optimization
- MST uniqueness
- Maximum spanning tree
- Applications: Network design, clustering
- Steiner tree introduction
- Online MST algorithms

Projects built:
- Network cable optimizer
- Cluster analysis tool
- Maze generator using MST
- Circuit board router

Practice: Solve 20 MST and Union-Find problems

Month 6: Phase 2 Completion & Integration (Weeks 24-26)

Algorithm Design Paradigms

Topics:
- Brute force and when it's okay
- Greedy algorithm design
- Proving greedy correctness
- Divide and conquer mastery
- Transform and conquer
- Decrease and conquer
- Problem reduction techniques
- Algorithm paradigm selection
- Hybrid approaches
- Approximation algorithms intro
- Randomized algorithms basics
- Online vs offline algorithms

Projects built:
- Algorithm strategy selector
- Greedy algorithm visualizer
- Problem classifier tool
- Algorithm comparison framework

Practice: Apply different paradigms to 30 problems

Space & Time Optimization

Topics:
- Space-time tradeoffs
- Memory optimization techniques
- Cache-friendly algorithms
- Bit manipulation for space saving
- Rolling arrays technique
- In-place algorithms
- Stream processing algorithms
- External memory algorithms
- Parallel algorithm basics
- MapReduce introduction
- Distributed algorithms overview
- Quantum algorithms preview

Projects built:
- Memory-efficient data processor
- Stream analytics engine
- Large data sorter
- Optimization analyzer tool

```
      Practice: Optimize 20 solutions for space/time
    Testing & Debugging Mastery
      Topics:
        - Test case design strategies
        - Edge case identification
        - Stress testing techniques
        - Random test generation
        - Debugging complex algorithms
        - Using debuggers effectively
        - Print debugging strategies
        - Assertion-based debugging
        - Performance profiling
        - Memory leak detection
        - Code coverage analysis
        - Continuous testing practices
      Projects built:
        - Test case generator framework
        - Algorithm debugger tool
        - Performance profiler
        - Bug tracker system
      Practice: Debug 30 algorithmic bugs
      Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

---------------------------------------------------------------
PHASE 3: ADVANCED ALGORITHMS & TECHNIQUES (MONTHS 7-9, WEEKS 27-39)
---------------------------------------------------------------
Master dynamic programming, advanced data structures, computational geometry, and complex
problem-solving techniques.

  Month 7: Dynamic Programming Mastery (Weeks 27-31)
    DP Fundamentals
      Topics:
        - Dynamic programming introduction
        - Overlapping subproblems concept
        - Optimal substructure property
        - Memoization (top-down approach)
        - Tabulation (bottom-up approach)
        - Space optimization in DP
        - 1D DP problems: Fibonacci, climbing stairs
        - Classic problems: Coin change, rod cutting
        - DP state definition
        - Recurrence relation formulation
        - Base cases identification
        - DP vs greedy vs divide-conquer
      Projects built:
        - DP problem solver framework
        - Fibonacci calculator (all methods)
        - Optimal game strategy simulator
        - Resource allocation optimizer
      Practice: Solve 50 basic DP problems
    String DP Problems
      Topics:
        - Longest Common Subsequence (LCS)
        - Longest Common Substring
        - Edit distance (Levenshtein distance)
        - Longest Palindromic Subsequence
        - Longest Palindromic Substring
        - String interleaving
        - Wildcard pattern matching
        - Regular expression matching
        - Distinct subsequences count
        - String compression DP
        - Word break problems
        - Palindrome partitioning
      Projects built:
        - Diff tool implementation
        - DNA sequence aligner
        - Spell checker with corrections
        - Text similarity analyzer
      Practice: Master 30 string DP problems
    Grid & Matrix DP
      Topics:
        - Grid traversal problems
        - Unique paths counting
        - Minimum path sum
        - Maximum path sum variants
        - Dungeon game type problems
        - Cherry pickup problem
```

- Matrix chain multiplication
- Maximal rectangle in binary matrix
- Largest square in matrix
- 2D range sum queries
- DP on grids with obstacles
- Multi-dimensional DP

Projects built:
- Pathfinding game with obstacles
- Matrix optimizer tool
- Grid-based puzzle solver
- 2D resource collector game

Practice: Solve 35 grid DP problems

Knapsack & Subset Problems

Topics:
- 0/1 Knapsack problem
- Unbounded knapsack
- Fractional knapsack (greedy)
- Subset sum problem
- Partition equal subset sum
- Target sum with +/- operators
- Minimum subset sum difference
- Count of subset sum
- Coin change variations
- Rod cutting variations
- Job scheduling with profits
- Knapsack with multiple constraints

Projects built:
- Backpack optimizer for games
- Budget allocation system
- Partition optimizer
- Investment portfolio selector

Practice: Master all knapsack variations

Tree DP

Topics:
- DP on trees concept
- Tree diameter using DP
- Maximum path sum in tree
- Tree balancing problems
- Subtree problems with DP
- Re-rooting technique
- Binary tree cameras problem
- Tree coloring with constraints
- Maximum independent set in tree
- Tree matching problems
- DP with tree traversal
- Optimizing tree recursions

Projects built:
- Tree optimization visualizer
- Network optimizer with DP
- Tree game strategies
- Hierarchical resource distributor

Practice: Solve 25 tree DP problems

Month 8: Advanced DP & Optimization (Weeks 32-36)

DP with Bitmasks

Topics:
- Bitmask representation basics
- Bit manipulation operations review
- DP with bitmask states
- Traveling Salesman with DP
- Hamiltonian path with bitmasks
- Assignment problems with masks
- Subset iteration with bitmasks
- Profile optimization technique
- Broken profile DP
- SOS (Sum over Subsets) DP
- Bitmask DP optimizations
- Memory optimization with masks

Projects built:
- TSP solver with visualization
- Task assignment optimizer
- Board game AI with bitmask DP
- Subset analyzer tool

Practice: Master 20 bitmask DP problems

Interval DP & Games

Topics:
- Interval DP pattern
- Matrix chain multiplication revisited
- Burst balloons problem

- Palindrome partitioning minimum cuts
- Optimal binary search tree
- Game theory basics: Nim game
- Minimax algorithm
- Alpha-beta pruning
- Sprague-Grundy theorem
- Combinatorial game theory
- DP in game strategies
- Winning and losing positions

Projects built:

- Game strategy analyzer
- Optimal BST builder
- Nim game variations player
- Two-player game framework

Practice: Solve 25 interval DP and game problems

DP Optimizations

Topics:

- Convex hull trick
- Divide and conquer optimization
- Knuth-Yao speedup
- Monotone queue optimization
- Slope trick introduction
- CHT (Convex Hull Trick) applications
- Li Chao tree basics
- Matrix exponentiation for DP
- DP on DAGs
- Rolling hash with DP
- Digit DP technique
- DP with data structures

Projects built:

- DP optimizer tool
- Performance comparison framework
- Convex hull visualizer
- Advanced DP solver

Practice: Apply optimizations to 20 DP problems

Probability & Expected Value DP

Topics:

- Probability basics review
- Expected value concept
- DP with probabilities
- Dice problems with DP
- Random walk problems
- Markov chains introduction
- Expected value optimization
- Monte Carlo methods basics
- Linearity of expectation
- Probabilistic algorithms
- DP in stochastic processes
- Applications in games and simulations

Projects built:

- Dice game simulator
- Probability calculator
- Random process analyzer
- Expected value optimizer

Practice: Solve 20 probability DP problems

State Space Reduction

Topics:

- State space explosion problem
- State compression techniques
- Symmetry reduction
- Equivalence class reduction
- Meet in the middle technique
- Bidirectional search
- State space pruning
- Heuristic state reduction
- Memory-efficient DP
- External memory DP
- Parallel DP basics
- DP state design patterns

Projects built:

- State space optimizer
- Memory-efficient solver
- Large-scale DP processor
- State reduction analyzer

Practice: Optimize 15 DP solutions for memory

Month 9: Computational Geometry & Number Theory (Weeks 37-39)

Computational Geometry Basics

Topics:

- Points, lines, and vectors
- Distance calculations
- Orientation of three points
- Collinearity check
- Line intersection detection
- Segment intersection
- Point in polygon test
- Convex hull algorithms
- Graham scan implementation
- Jarvis march (Gift wrapping)
- Area of polygon
- Closest pair of points

Projects built:
- Geometry calculator suite
- Convex hull visualizer
- Collision detection system
- Polygon area calculator

Practice: Solve 30 basic geometry problems

Advanced Number Theory

Topics:
- Extended Euclidean algorithm
- Modular multiplicative inverse
- Chinese Remainder Theorem
- Fermat's little theorem
- Euler's totient function
- Miller-Rabin primality test
- Pollard rho factorization
- Baby-step giant-step
- Discrete logarithm
- Quadratic residues
- Primitive roots
- Number theoretic transforms

Projects built:
- RSA encryption implementation
- Prime number toolkit
- Cryptographic hash function
- Number theory calculator

Practice: Solve 25 number theory problems

Combinatorics & Counting

Topics:
- Advanced permutation generation
- Combination with repetition
- Catalan numbers applications
- Stirling numbers
- Bell numbers
- Derangements
- Inclusion-exclusion principle
- Generating functions basics
- Recurrence solving techniques
- Burnside's lemma
- Polya enumeration
- Advanced counting techniques

Projects built:
- Combinatorics calculator
- Sequence generator tool
- Counting problem solver
- Pattern enumeration system

Practice: Master 20 combinatorics problems

Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)

The capstone month: spec, build, ship and defend, then the final exam that decides the certificate.

Month 10: Capstone and the Final Exam (Weeks 40-44)

Capstone Spec and Setup

Topics:
- Choosing the capstone: a problem-solving portfolio: a set of hard, multi-part challenges solved, documented and explained clearly
- A one-page spec: scope, milestones, stack
- Repository and skeleton ready on day one
- Instructor sign-off before the build begins

Projects built:
- Approved capstone spec plus the running skeleton

Practice: Pitch the spec to the batch and absorb one hard question

Capstone Build

Topics:
- Two focused weeks in vertical slices
- Instructor checkpoints and honest scope cuts
- Using the whole course: every structure and technique learned, chosen to fit each problem
- Daily commits and an always-working build

Projects built:
- The capstone, feature-complete with version control

Practice: A working build at the end of every session

Ship and Polish

Topics:
- The finishing pass: edge cases, errors, documentation
- Solutions explained, not just submitted: correct, reasoned and communicated
- A README worth reading and a demo worth watching
- A full-course spiral review before the exam

Projects built:
- The capstone shipped, documented and rehearsed

Practice: Two mock demos, each tighter than the last

Demo Day and the Final Exam

Topics:
- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: our competitive programming course for contests, and our DSA course for interview-grade depth
- Where our other courses continue the journey

Projects built:
- Demo day presentation delivered to the batch

Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: logic and core-structure problem sets
- Phase 2: data-structure and algorithm challenges
- Phase 3: dynamic-programming and technique problem sets
- Phase 4: the problem-solving portfolio capstone

Total projects: 200+ problems solved and every core structure implemented, crowned by a portfolio capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Programming logic: conditionals, loops, functions and clean decomposition
- Core structures: arrays, strings, hash maps, stacks, queues, linked lists, recursion
- Data structures and algorithms: trees, heaps, graphs, sorting, searching
- Dynamic programming: the topic that unlocks the hardest problems
- Problem-solving patterns that transfer to unseen problems

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Our competitive programming course for contest speed and technique
- Our DSA course for interview-grade depth and communication
- Any language course here pairs with this problem-solving spine
- Coding olympiads and contests, with a real foundation behind you

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this problem-solving course for teens cover?
A: Logic and programming foundations first (conditionals, loops, functions, arrays, strings,

hash maps, stacks, queues, linked lists, recursion), then core data structures and algorithms (trees, heaps, graphs, sorting, searching, algorithm design), then dynamic programming and problem-solving techniques with counting and geometry basics. Everything is taught through problems, focusing on transferable patterns. A finale month builds a problem-solving portfolio and sits the final exam.

Q: How long does it take to learn problem solving properly?
A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two more months in hand for learners who need them. The three teaching phases build the skill and its real-world application; the finale month ships a capstone and sits the final exam.

Q: How am I assessed? Is the certificate just for attending?
A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Does my teenager need coding experience to start?
A: No. The course begins with programming logic from scratch and builds up. What matters is curiosity and a willingness to sit with a hard problem, which is exactly the habit the course trains. Any language works; we most often use Python because it keeps the focus on the thinking.

Q: Is this the same as your competitive programming course?
A: No, and the difference is deliberate. This teens course builds the foundation, logic, data structures, algorithms and problem-solving patterns at a teen pace. Our competitive programming course is the intense, contest-focused next step (advanced techniques, speed, olympiad and ICPC preparation), and our DSA course is the interview-focused one. Teens who finish this and want more are perfectly placed for either.

Q: Can I try a class before enrolling?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/problem-solving-dsa-masterclass-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 115 OF 137: PSLE MATHS: MODEL METHOD, PROBLEM SUMS & MOE SYLLABUS

==========

Course link: https://learn.modernagecoders.com/courses/psle-maths-mastery
Subject area: Exam Board Mathematics
Designed for: Elementary school
Who can join: Primary 4-6 students on the MOE syllabus, PSLE-bound
Course length: 12 months (52 weeks)
Time needed: 2 live one-hour classes per week + short practice sets
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online PSLE maths on the MOE syllabus: model method taught as thinking, problem-sum families, no-calculator Paper 1 craft, Singapore evening slots.

In one line: Problem sums are a teachable skill. We teach it, 1-on-1, without the centre queue.

ABOUT THE PROGRAMME

A 12-month live program aimed squarely at PSLE mathematics, taught on the MOE syllabus at Singapore-friendly evening hours. The PSLE separates thinkers from drillers: multi-step problem sums, a no-calculator Paper 1, and AL scoring where consistency is everything. This course teaches the model method as a thinking tool rather than a template, names and trains the heuristics explicitly, builds calculator-free fluency in every class, and closes with

paper craft, time discipline, working presentation and the mark-banking habits that protect AL bands.

WHAT MAKES THIS COURSE DIFFERENT

- The model method taught as reasoning, part-whole, comparison, before-after and re-cutting, not as a ritual the school demands
- Problem sums taught by structure family, so new problems look like old friends in disguise
- MOE heuristics named and trained explicitly: work backwards, guess-and-check with discipline, restate, spot patterns
- Paper 1 fluency drills open every class, because AL bands are lost one careless mark at a time
- 1-on-1 attention no centre batch of fifteen can give, at Singapore evening hours
- Understanding-first: a child who can draw what the numbers are doing owns the problem
- One dedicated mentor tracking exactly where your child's reasoning thins
- Honest AL guidance from real timed-paper evidence, never invented promises

THE LEARNING PATH, STEP BY STEP

Phase 1: The method rebuilt properly: number fluency, model drawing as thinking, and the first problem-sum families
Phase 2: The P5-P6 canon: fractions of remainder, ratio, percentage, speed and geometry, family by family
Phase 3: Exam craft: Paper 1 and 2 technique, timed papers, mark-banking discipline and the calm P6 runway to the PSLE

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Primary 4-6 on the MOE syllabus (or equivalent international students using Singapore-style curricula)
Math background: Any, the diagnostic finds the real stage and the method phase rebuilds from there
Equipment: Computer with stable internet, notebook and pencil; calculator for Paper 2 work
Time commitment: 2 live hours plus 1-2 practice hours weekly
English: Course taught in English
Motivation: A child willing to draw and explain, the method does the rest

WHO THIS COURSE IS FOR

Problem sum strugglers: Children whose computation is fine but whose problem sums produce blank stares
Template drawers: Children who draw models as ritual without the bars carrying the reasoning
Paper 1 leakers: Children losing careless marks to calculator-free arithmetic
Centre switchers: Families whose centre batch is not teaching their child
P4 planners: P4 families starting the method work early, when it is cheapest

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE METHOD, REBUILT PROPERLY (MONTHS 1-4)

The gap behind most PSLE struggles is a model method learned as template instead of thinking. This phase rebuilds number fluency and model drawing from meaning, then applies both to the first problem-sum families.

Month 1: Diagnostic + Number Fluency, Bare-Handed (Weeks 1-4)
Diagnostic and the Honest AL Baseline
Topics:
- Model-method depth check across part-whole and comparison bars
- Paper 1 non-calculator fluency screen
- Locating exactly where problem sums break down
- Honest Achievement Level (AL) baseline from timed questions
- Whole-number, fraction and decimal accuracy audit
- Reading-the-question and working-presentation habits
Practice: A short mixed diagnostic of Paper 1 items and two problem sums, marked to expose the real starting stage.
Assessment: Baseline AL estimate plus a written map of which problem-sum families and Paper 1 skills need the most work.
Detail: Full diagnostic: model-method depth, Paper 1 fluency, and exactly where problem sums break, with an honest AL baseline
Whole-Number Operations at Paper 1 Tempo
Topics:
- Addition, subtraction, multiplication and division fluency
- Long multiplication and long division without a calculator
- Order of operations in multi-step calculations
- Estimation and rounding as a checking habit
- Mental strategies for speed on Paper 1 items

- Place value and number sense to catch slips
Practice: A timed whole-number drill set plus five estimation checks on given answers.
Detail: Whole-number operations at Paper 1 tempo; estimation as a checking habit
Fractions as Numbers, From Pictures First
Topics:
- Fractions on a number line as genuine quantities
- Equivalent fractions and simplifying
- Comparing and ordering fractions
- Adding and subtracting unlike fractions
- Multiplying and dividing fractions with meaning
- Bar and area pictures behind every rule
Practice: A fractions set moving from picture-drawn answers to fluent written computation.
Detail: Fractions as numbers: equivalence, comparison and operations from pictures first
Decimals and the Fraction-Decimal-Percentage Bridge
Topics:
- Decimal place value and the four operations
- Converting fractions to decimals and back
- Linking decimals, fractions and percentages
- Rounding decimals and estimating results
- Money and measurement contexts
- Fluency drills across all three forms
Practice: Conversion drills and mixed decimal calculations at Paper 1 pace.
Detail: Decimals and the fraction-decimal-percentage bridge, to fluency

Months 2-3: Model Drawing as Thinking (Weeks 5-12)
Part-Whole Models as Reasoning
Topics:
- Part-whole bar model as a picture of the numbers
- Drawing the unknown before computing
- Total, part and missing-part relationships
- Translating word phrases into bar segments
- The child draws first, the mentor questions
- Checking the model against the story
Practice: Six part-whole problem sums where the bar is drawn before any calculation.
Detail: Part-whole models: the child draws, the mentor questions, never the reverse
Comparison Models Made Visible
Topics:
- Comparison bars for more-than and less-than
- Times-as-many and multiplicative comparison
- Aligning bars to reveal the difference
- Two-quantity and three-quantity comparisons
- Choosing comparison over part-whole
- Defending the chosen model aloud
Practice: A comparison-model set mixing additive and multiplicative relationships.
Detail: Comparison models: more-than, less-than and times-as-many, made visible
Unit Thinking and Re-Cutting Bars
Topics:
- Treating one unit as the engine of a model
- Deciding how many parts to cut a bar into
- Re-cutting a bar when the story changes
- Finding unit value then scaling up
- Flexibility that templates cannot supply
- Common-part reasoning across quantities
Practice: Problems requiring the student to choose the cut, then re-cut when stuck.
Detail: Unit thinking: when to cut a bar into parts, and the re-cutting flexibility templates cannot supply
First Structure Families: Sum-and-Difference
Topics:
- Sum-and-difference problem family
- Equal-stage and equal-value problems
- Recognising the skeleton under the wording
- Matching model shape to family
- Solving then explaining the reasoning aloud
- Extending a solved problem to a variant
Practice: A family-sorted set of sum-and-difference and equal-stage problem sums.
Detail: First structure families: sum-and-difference and equal-stage problems, solved by model and defended aloud

Month 4: Heuristics, Named and Trained (Weeks 13-16)
Work Backwards Heuristic
Topics:
- Recognising end-state problems
- Reversing each operation in turn
- Undoing add, subtract, multiply and divide
- Model support for backward reasoning
- Checking by working forwards again
- When work-backwards beats other methods

      Practice: A set of end-state problems solved by unwinding step by step.
      Detail: Work backwards: end-state problems unwound step by step
    Disciplined Guess-and-Check
      Topics:
        - Setting up an organised guess-and-check table
        - Choosing a sensible first guess
        - Using each result to steer the next guess
        - Column headings that show the reasoning
        - Knowing when guess-and-check is efficient
        - Presenting the table for method marks
      Practice: Guess-and-check problems recorded in a clean, columned table.
      Detail: Guess-and-check with discipline: the organised table, not wild stabs
    Restate and Simplify
      Topics:
        - Stripping a wordy problem to its core
        - Restating in the student's own words
        - Identifying the quantity actually asked for
        - Turning sentences into a drawable model
        - Removing distracting detail
        - Simplify-then-solve as a habit
      Practice: Wordy problems rewritten in one line, then drawn and solved.
      Detail: Restate and simplify: turning wordy problems into drawable ones
    Phase 1 Checkpoint
      Topics:
        - Mixed problem-sum set across all Phase 1 families
        - Model choice under mixed conditions
        - Explaining each solution aloud
        - Extending problems to harder variants
        - Heuristics applied without prompting
        - Error log started from checkpoint mistakes
      Practice: A mixed timed problem-sum set covering every Phase 1 family.
      Assessment: Phase 1 checkpoint marked for method and reasoning, feeding the first
      error-log entries.
      Detail: Phase checkpoint: mixed problem-sum set solved, explained and extended

---
PHASE 2: THE P5-P6 CANON, FAMILY BY FAMILY (MONTHS 5-9)

The problem-sum territory where the PSLE decides its top bands, taught by structure family so
your child recognises the skeleton under every costume: fractions of remainder, ratio and its
changes, percentage, speed and geometry.

  Month 5: Fractions of Remainder and Branch Models (Weeks 17-20)
    Remainder Concept and Branch Models
      Topics:
        - Fraction of a remainder made visible
        - Branch models for successive fractions
        - Re-cutting into common parts
        - Tracking what is left after each stage
        - Restoring the whole from a known part
        - Two-stage remainder reasoning
      Practice: Branch-model problems working from a remainder back to the whole.
      Detail: The remainder concept made visible: branch models and the re-cut into common
      parts
    Multi-Stage Remainder Problems
      Topics:
        - Classic PSLE remainder problem sums
        - Three-stage and multi-stage remainders
        - Combining branch models with unit value
        - Fraction-of-remainder with a given amount
        - Spotting the common-part cut quickly
        - Presenting long working clearly
      Practice: A set of exam-standard remainder problems, including multi-stage versions.
      Detail: Classic remainder problems at PSLE standard, including the multi-stage versions
      that humble adults

  Month 6: Ratio, and Ratio That Changes (Weeks 21-24)
    Ratio as Units
      Topics:
        - Ratio read as units
        - Equivalent ratios and simplifying
        - Sharing a quantity in a given ratio
        - The unit-value engine for ratio problems
        - Two-term and three-term ratios
        - Ratio bars linked to fractions
      Practice: Ratio problems solved through the unit-value method with bar support.
      Detail: Ratio as units: equivalent ratios, sharing and the unit-value engine
    Before-and-After Ratio Problems
      Topics:

```
      - Before-and-after ratio changes
      - Constant-total problems
      - Constant-difference problems
      - Constant one-quantity problems
      - Choosing the invariant to anchor units
      - The PSLE signature ratio family
    Practice: Before-and-after ratio sets sorted by which quantity stays constant.
    Detail: Before-and-after problems: changed ratios, constant totals and constant
    differences, the PSLE's signature family

Month 7: Percentage in Every Costume (Weeks 25-28)
  Percentage Of, Increase and Decrease
    Topics:
      - Finding a percentage of a quantity
      - Percentage increase and decrease
      - GST and discount calculations
      - Chained percentage changes
      - Percentage with bar-model support
      - Everyday money contexts
    Practice: Percentage problems including discount-then-GST chains.
    Detail: Percentage of, increase and decrease, and GST-and-discount chains
  Reverse Percentage at Exam Standard
    Topics:
      - Reverse percentage: finding the original
      - Distinguishing part, whole and percentage
      - Percentage combined with bar models
      - Multi-step percentage problem sums
      - Checking answers against the original
      - Exam-standard presentation
    Practice: Reverse-percentage and percentage-with-model problems at PSLE level.
    Detail: Reverse percentage and percentage-with-model problems at exam standard

Month 8: Speed, Rate and the Journey Problems (Weeks 29-32)
  Speed as a Rate With Meaning
    Topics:
      - Speed understood as distance per unit time
      - Distance-time reasoning before formulas
      - Deriving the speed, distance, time triangle
      - Units and conversions in rate problems
      - Constant-speed situations
      - Reading a journey from a description
    Practice: Distance-time problems solved by reasoning, then confirmed by formula.
    Detail: Speed as a rate with meaning: distance-time reasoning before formulas
  Journey Problems and Average Speed
    Topics:
      - Meeting problems on journey diagrams
      - Catching-up and overtaking problems
      - Average speed over multiple legs
      - Total distance over total time
      - Two-object motion drawn to scale
      - Common average-speed traps
    Practice: Meeting, catching-up and average-speed problems on drawn journey diagrams.
    Detail: Meeting, catching-up and average-speed problems, drawn on journey diagrams

Month 9: Geometry, Area and the Visual Problems (Weeks 33-36)
  Angles and Figures With Justification
    Topics:
      - Angle facts for lines and triangles
      - Angle properties of quadrilaterals
      - Justifying each step with a reason
      - Composite and overlapping figures
      - Unknown angles in multi-shape diagrams
      - Clear geometric working
    Practice: Angle problems requiring a stated reason for every step.
    Detail: Angles, triangles and quadrilaterals with justification language; composite
    figures
  Area, Perimeter and Shaded Regions
    Topics:
      - Area and perimeter of composite shapes
      - Shaded-region classic problems
      - Subtracting and combining areas
      - Circle area and circumference at P6 standard
      - Fractions of a circle and semicircles
      - Presenting area working clearly
    Practice: Shaded-region and circle problem sums at PSLE standard.
    Detail: Area and perimeter problem sums including the shaded-region classics; circles
    at P6 standard

---------------------------------------------------------------
```

PHASE 3: PAPER CRAFT AND THE PSLE RUNWAY (MONTHS 10-12)

Understanding becomes AL bands here: Paper 1 speed without a calculator, Paper 2 working discipline, timed full papers with review rituals, and a peak plan that walks into the PSLE calm.

Month 10: Paper 1 and Paper 2 as Different Skills (Weeks 37-40)
Paper 1 Craft and Mental Strategies
Topics:
- Mental strategies for Paper 1 speed
- No-calculator discipline under time
- Banking one-mark and two-mark questions fast
- Common Paper 1 question types
- Avoiding careless-mark leaks
- Order of attack on Paper 1
Practice: A full Paper 1 section timed, focused on banking the short questions.
Detail: Paper 1 craft: mental strategies, no-calculator discipline and the one-mark questions banked fast
Paper 2 Craft and Method Marks
Topics:
- Presentation of working for method marks
- Calculator use that saves time
- Structuring multi-step Paper 2 answers
- Showing enough working to secure marks
- Units and final-answer discipline
- Choosing when the calculator helps
Practice: Paper 2 questions written up to full method-mark presentation standard.
Detail: Paper 2 craft: presentation of working, method marks, and calculator use that helps instead of wastes time

Month 11: Timed Papers and Mark Banking (Weeks 41-44)
Full Timed Papers and Mark Banking
Topics:
- Full papers at true exam pacing
- Banking secure marks first
- Leaving the hardest problems for last
- Time budgeting across Paper 1 and Paper 2
- Managing pressure and pacing
- Flagging and returning to hard items
Practice: A complete timed paper sat under exam conditions with a pacing plan.
Detail: Full timed papers at exam pacing; the banking discipline, secure marks first, wrestle the monsters last
Review Rituals and the Error Log
Topics:
- Reviewing why each mark was lost
- Naming the specific fix for each error
- Sorting errors by type and frequency
- Beginning error-log closure
- Turning mistakes into targeted practice
- Splitting careless from conceptual errors
Practice: Each lost mark logged with a named fix and a follow-up practice item.
Detail: Review rituals: why each mark was lost, and the named fix; error-log closure begins

Month 12: The Calm Runway (Weeks 45-52)
Weakest-Family Rotations
Topics:
- Rotations targeting the weakest families
- Priority set by the error log
- Confidence work on nearly-mastered problems
- Revisiting problem-sum families that slip
- Spaced return to fixed errors
- Building momentum into the final weeks
Practice: Rotating problem sets aimed at the top error-log priorities each week.
Detail: Weakest-family rotations by error-log priority; confidence work through problems your child can nearly do, then can
Final Cycles and the Peak-Week Plan
Topics:
- Final full paper cycles
- Documented AL-trend progression
- Peak-week light-practice plan
- Rest and routine before the exam
- Exam-day rehearsal and timing
- The PSLE met as a rehearsed place
Practice: Final timed papers with progression recorded, tapering into light peak-week practice.
Assessment: Final timed PSLE-format papers scored and charted to confirm AL-trend progression before the exam.
Detail: Final paper cycles with documented progression; peak-week plan, light practice,

rest, and the exam met as a rehearsed place

PROJECTS THE STUDENT BUILDS AND KEEPS

- Personal error log and problem-sum family map
- The child's own model-method handbook, drawn not copied
- Heuristics card set: each strategy named with a worked example
- Timed paper cycles with documented AL-trend progression
- Peak-week plan rehearsed before the real PSLE

Total projects: 5+ exam artifacts plus the child's own handbooks

SKILLS THE STUDENT WILL HAVE AT THE END

- Model drawing as a thinking tool
- All major problem-sum structure families
- MOE heuristics, named and applied
- Paper 1 calculator-free fluency
- Paper 2 working and method-mark discipline
- Fractions, ratio, percentage and speed at PSLE depth
- Geometry and shaded-region craft
- Time discipline and mark banking

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes at Singapore evening hours
Practice: Short purposeful sets, never another assessment-book stack
Fluency: Paper 1-style warm-up in every class
Total per week: About 3-4 focused hours
Homework: A problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week
Mentorship: One dedicated mentor across the whole run
Recordings: Every class recorded for revision
Progress tracking: Timed-paper progression shared with parents
Flexibility: Slots move around school exams and family plans

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies mastery, not attendance

WHERE THIS CAN LEAD

- An AL band that reflects real ability, opening posting options
- Secondary-ready: the algebra handover met with model-method understanding underneath
- E Maths and A Maths readiness through Secondary
- Ready for GEP/DSA-style stretch via competition mathematics

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course aligned to the MOE syllabus and the actual PSLE format?
A: Yes, the MOE primary mathematics syllabus is the spine, and exam work uses the real format: the no-calculator Paper 1, Paper 2 with calculator, and AL scoring awareness throughout. We verify current SEAB formats for your child's cohort rather than assuming.

Q: How do the class timings work from Singapore?
A: You pick two weekly slots and they stay yours.

Q: My child computes well but freezes on problem sums. Can this be fixed?
A: Yes, problem sums are the most teachable 'talent' in Singapore maths. They break into structure families, each with a model shape and heuristic. We teach the families, then the re-cutting flexibility that separates thinkers from template-followers. A term of that work changes what your child sees when they read a problem.

Q: My child already attends a tuition centre. Why switch?
A: Sometimes you should not, a child thriving near the top of a good batch should stay. The switch wins when the batch pace is wrong in either direction: the struggling child whose specific gaps never get airtime, or the capable child rehearsing what they already know. Here, every minute of the hour aims at your child.

Q: When should PSLE prep start?
A: The full 12-month arc suits P5, finishing on the P6 runway; P6 students join a compressed

version prioritised by the diagnostic. Earlier starts (P4) spend longer in the method phase, which is the highest-return work there is.

Q: What AL improvement can we expect?
A: We will not invent a number. What you get: an honest baseline in week one, the highest-value families attacked first, and timed-paper evidence you can track yourself. AL bands reward consistency, and consistency is a trainable behaviour.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A problem set follows every class with written feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a full mock PSLE-style paper as the final assessment. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real understanding, not attendance.

Q: Can we try it before paying?
A: Yes. Every child starts with a free live demo class at a Singapore evening hour that doubles as the diagnostic, no card details, no obligation.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/psle-maths-mastery
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 116 OF 137: PYTHON AND AI FOR KIDS: GAMES, TURTLE ART AND SMART PROJECTS

==========

Course link: https://learn.modernagecoders.com/courses/python-ai-kids-masterclass
Subject area: Kids Coding
Designed for: Kids
Who can join: Complete beginner, ages 9 to 12
Course length: 8-10 months (32-40 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Python course for kids aged 9-12: live online classes from first code to turtle art, Pygame games and kid-safe AI projects, in 8-10 months.

In one line: Real typed Python at 9 to 12: turtle art that wows, games with Pygame, and AI understood the kid-safe way, by training models, not chatting with bots.

ABOUT THE PROGRAMME

Python is the friendliest real language a child can type, and this live online course makes the most of it for ages 9 to 12: from the first print statement to turtle graphics that draw spirals and snowflakes, real logic and loops, functions, files, first object-oriented thinking, and complete games built in Pygame that friends actually want to play. The AI phase is deliberately kid-safe and hands-on: children learn what AI is by building it small, rules-based smart programs, data detective work on real (fun) datasets, and camera-based projects in the Teachable Machine style, where they train and test models themselves. There are no chatbot accounts and no AI-writes-the-code shortcuts, the major AI chat tools are 13+ by their own rules, and this course says so plainly while building the skills that will make such tools genuinely useful later. The pace is honest and kid-sized: eight months at two live classes a week plus two to three hours of playful practice, with two more months in hand when needed. Homework after every class, a mixed challenge at the end of every month, and a final exam, a capstone demo plus a friendly viva, that decides the certificate, with a free retest after revision. Real Python, real games, real understanding.

WHAT MAKES THIS COURSE DIFFERENT

- Real typed Python from day one, at a pace built for 9 to 12 year olds
- Turtle graphics as the hook: art a child can print and stick on the fridge, powered by loops
- Complete Pygame games: finished, playable, shared with family, not half-built demos
- Kid-safe AI: models trained by the child, rules-based smart programs, no chatbot accounts (those tools are 13+, and we say so)
- An honest 8-month arc with 2 months in hand: nobody is rushed, nobody coasts
- Real assessment, gently done: monthly mixed challenges and a final capstone exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Python basics (Months 1-2): print to loops, taught through jokes, quizzes and games
Phase 2: Turtle and data powers (Months 3-4): turtle art, lists and functions, the coder toolkit
Phase 3: Real games (Months 5-6): files, first OOP, and complete Pygame games with scores and sounds
Phase 4: Kid-safe AI and the finale (Months 7-8): what AI really is, data detective work, camera projects, and the capstone exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Python is taught from the very first line
Reading typing: Comfortable reading; typing grows quickly inside the course
Equipment: A computer or laptop with a stable internet connection; Python and every tool used are free
Age: 9 to 12. Younger children start better in our Scratch-first Kids Coding course
Time commitment: 2 live classes plus 2-3 hours of playful practice a week

WHO THIS COURSE IS FOR

Ready typists: Children 9-12 ready for real typed code, often straight from Scratch
Game lovers: Kids who will move mountains to build a game their friends can play
Ai curious: Children asking how Alexa and recommendations actually work, ready for honest answers
Puzzle minds: Kids who love logic puzzles and want a language that rewards them
Future teens: Children who will roll into our teen Python and AI courses years ahead

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PYTHON BASICS (MONTHS 1-2, WEEKS 1-8)

The first real language, learned through play: printing, variables, decisions and loops, every week ending with something funny or clever to show at home.

Month 1: First Code (Weeks 1-4)
Welcome to Programming & Setup
Topics:
- What is programming? How do computers understand us?
- Why Python? The language that powers YouTube, Instagram, and AI
- Installing Python - making your computer ready to code
- Using IDLE - Python's friendly coding environment
- Your first program: Making Python say 'Hello World!'
- Using print() to display messages
- Creating colorful text outputs
- Comments - leaving notes in your code
- Making Python do math - calculator programs
- Common errors and how to fix them
- Saving and running Python files
- Organizing your coding projects

Projects built:
- Personalized greeting program
- Magic calculator with special messages
- Story generator with your name

Practice: Daily: Create 3 different print programs, solve 5 math problems with Python
Variables & Data Types - Teaching Python to Remember
Topics:
- Variables - giving names to information
- Choosing good variable names
- Numbers in Python: integers and decimals
- Text in Python: strings and how to use them
- True or False: boolean values
- Changing variable values
- Getting input from users with input()
- Converting between different types
- Fun with strings: uppercase, lowercase, length
- String concatenation - joining words together
- Special characters and escape sequences
- Creating Mad Libs games with variables

Projects built:
- Interactive story with user input
- Age calculator with birthday messages
- Mad Libs game generator
- Personal quiz about yourself

Practice: Create 5 programs using different variable types
Assessment: Month 1 check: build a talking mini-program live, plus a fun take-home

```
      challenge

  Month 2: Decisions and Loops (Weeks 5-8)
    Making Decisions with If-Else
      Topics:
        - If statements - teaching Python to make choices
        - Comparison operators: ==, !=, <, >, <=, >=
        - Else statements - what to do otherwise
        - Elif - checking multiple conditions
        - Combining conditions with and, or, not
        - Nested if statements - decisions within decisions
        - Creating quiz games with scoring
        - Password checker programs
        - Age-appropriate content filters
        - Random module - adding surprises to programs
        - Making a magic 8-ball program
        - Building simple AI decision makers
      Projects built:
        - Adventure game with choices
        - Grade calculator with feedback
        - Virtual pet mood checker
        - Rock, Paper, Scissors game
      Practice: Build 5 decision-making programs
    Loops - Making Python Repeat
      Topics:
        - For loops - counting and repeating
        - Range() function - controlling loop counts
        - While loops - repeat until something happens
        - Breaking out of loops with break
        - Skipping with continue
        - Nested loops - loops inside loops
        - Creating patterns with loops
        - Drawing ASCII art with loops
        - Countdown and timer programs
        - Multiplication table generator
        - Password attempt limiters
        - Animation basics with loops and time.sleep()
      Projects built:
        - Pattern art generator
        - Guessing game with attempts
        - Countdown timer with effects
        - ASCII art animation
      Practice: Create 10 different loop patterns and animations
      Assessment: Month 2 check: a guessing game built live, with quiz questions reaching
      back to month 1

------------------------------------------------------------
PHASE 2: TURTLE ART AND CODER POWERS (MONTHS 3-4, WEEKS 9-16)
------------------------------------------------------------
The famous turtle draws what the child commands: spirals, stars and snowflakes, then lists
and functions turn scripts into real programs.

  Month 3: The Turtle Awakens (Weeks 9-12)
    Introduction to Turtle Graphics
      Topics:
        - What is Turtle Graphics? Your drawing robot
        - Setting up the turtle screen
        - Basic movements: forward, backward, left, right
        - Drawing shapes: squares, triangles, circles
        - Changing turtle appearance and speed
        - Colors in turtle: pen color and fill color
        - Lifting and putting down the pen
        - Going to specific positions
        - Drawing with coordinates
        - Creating custom shapes
        - Saving turtle drawings
        - Multiple turtles on screen
      Projects built:
        - Geometric art generator
        - House drawing program
        - Rainbow spiral creator
        - Name writer in turtle
      Practice: Draw 10 different shapes and patterns
    Advanced Turtle Projects
      Topics:
        - Using loops with turtle for complex patterns
        - Creating functions for repeated shapes
        - Building a drawing app with keyboard control
        - Mouse events in turtle
```

- Creating simple games with turtle
- Collision detection basics
- Score keeping in turtle games
- Creating mazes with turtle
- Animating turtle movements
- Fractal patterns with recursion basics
- Turtle race game
- Building a paint program

Projects built:
- Controllable turtle game
- Turtle race simulator
- Maze solver with turtle
- Digital spirograph

Practice: Create 5 interactive turtle programs
Assessment: Month 3 check: a turtle art commission drawn live from a family request, plus mixed review

Month 4: Lists and Functions (Weeks 13-16)
Lists & Collections
Topics:
- Lists - storing multiple items
- Accessing list items with indexes
- Adding items: append, insert
- Removing items: remove, pop, clear
- List slicing - getting parts of lists
- Sorting and reversing lists
- Finding items in lists
- List comprehensions - the Python superpower
- 2D lists - lists inside lists
- Tuples - lists that can't change
- Sets - unique item collections
- Dictionaries - key-value pairs

Projects built:
- Todo list manager
- High score tracker
- Inventory system for games
- Contact book application

Practice: Build 5 programs using different list operations
Functions - Reusable Code Blocks
Topics:
- What are functions? Why use them?
- Creating simple functions with def
- Parameters - giving information to functions
- Return values - getting answers from functions
- Multiple parameters and default values
- Local vs global variables
- Function documentation with docstrings
- Lambda functions - tiny functions
- Recursion - functions calling themselves
- Building a function library
- Importing and using modules
- Creating your own modules

Projects built:
- Calculator with function menu
- Text adventure with function rooms
- Utility function collection
- Math game with scoring functions

Practice: Create 10 useful functions for future projects
Assessment: Month 4 check: refactor a messy script into functions live, plus mixed review

---
PHASE 3: REAL GAMES (MONTHS 5-6, WEEKS 17-24)

Programs that remember and games that feel real: files and saving, first object thinking, then Pygame from the game loop to finished, shareable games.

Month 5: Memory and Objects (Weeks 17-20)
File Handling & Data
Topics:
- Reading from text files
- Writing to text files
- Append mode vs write mode
- Working with CSV files
- Creating save files for games
- Loading game progress
- Simple encryption for files
- Creating a diary application
- Managing high scores

```
        - Building a note-taking app
        - File organization programs
        - Backup utilities
      Projects built:
        - Personal diary with password
        - Quiz game with saved scores
        - File organizer utility
        - Simple database system
      Practice: Build 5 file-based applications
    Object-Oriented Programming Basics
      Topics:
        - What are objects and classes?
        - Creating your first class
        - Attributes - what objects know
        - Methods - what objects can do
        - Creating multiple objects from one class
        - The __init__ method
        - Self - referring to the object itself
        - Class vs instance variables
        - Inheritance - classes having children
        - Making a pet class system
        - Building game characters with OOP
        - Real-world object modeling
      Projects built:
        - Virtual pet simulator
        - RPG character creator
        - School management system
        - Zoo animal hierarchy
      Practice: Create 5 different class systems
      Assessment: Month 5 check: a save-and-load app built live, plus mixed review

  Month 6: Pygame (Weeks 21-24)
    Introduction to Pygame
      Topics:
        - What is Pygame? Making real games!
        - Installing and setting up Pygame
        - Creating game windows
        - Game loop concept
        - Drawing shapes and colors
        - Loading and displaying images
        - Sprite basics
        - Moving objects on screen
        - Keyboard input in games
        - Mouse control in games
        - Frame rate and smooth movement
        - Coordinate system in games
      Projects built:
        - Bouncing ball animation
        - Simple drawing program
        - Moving character demo
        - Click the target game
      Practice: Create 5 simple Pygame demos
    Building Complete Games
      Topics:
        - Collision detection in games
        - Score keeping and lives
        - Game states: menu, playing, game over
        - Sound effects and music
        - Creating game levels
        - Power-ups and bonuses
        - Enemy AI basics
        - Particle effects
        - High score saving
        - Pause functionality
        - Full screen mode
        - Packaging games for sharing
      Projects built:
        - Snake game remake
        - Space shooter game
        - Platform jumper game
        - Puzzle matching game
      Practice: Add features to existing games
      Assessment: Phase check: a complete small game demoed live, plus a mixed challenge
      spanning phases 1-3

------------------------------------------------------------
PHASE 4: KID-SAFE AI AND THE FINALE (MONTHS 7-8, WEEKS 25-32)
------------------------------------------------------------
AI learned the honest way for this age: built small, trained by the child, tested for
```

mistakes, and finished with a capstone at demo day.

  Month 7: AI, Discovered Properly (Weeks 25-28)
    What is Artificial Intelligence?
      Topics:
        - AI in everyday life - Siri, Alexa, YouTube recommendations
        - How do computers learn? Simple explanation
        - Difference between programming and AI
        - Types of AI: narrow vs general
        - Machine Learning basics for kids
        - Supervised vs unsupervised learning
        - Training data concept
        - Making predictions
        - Pattern recognition
        - AI ethics - using AI responsibly
        - Famous AI applications
        - Future of AI
        - Why the famous AI chat tools are 13+, said plainly
        - Building a rules-based smart helper in pure Python
      Projects built:
        - AI concept presentation
        - Design your dream AI assistant
        - AI in daily life tracker
        - Simple pattern recognizer
      Practice: Research and present 5 AI applications
    Data Detective Work
      Topics:
        - What is data? Why is it important?
        - Collecting and organizing data
        - NumPy basics - working with numbers
        - Pandas basics - data tables
        - Reading data from files
        - Cleaning messy data
        - Finding patterns in data
        - Basic statistics: mean, median, mode
        - Creating charts with Matplotlib
        - Bar charts, pie charts, line graphs
        - Making data tell stories
        - Data visualization projects
      Projects built:
        - Class survey analyzer
        - Weather data explorer
        - Sports statistics dashboard
        - Personal habit tracker
      Practice: Analyze 5 different datasets
      Assessment: Month 7 check: answer three questions from a fun dataset live, plus mixed
      review

  Month 8: Camera Magic and the Capstone (Weeks 29-32)
    Camera Magic: Train Your Own Model
      Topics:
        - How computers see images
        - Pixels and image data
        - OpenCV introduction
        - Loading and displaying images
        - Image manipulation: resize, rotate, flip
        - Color spaces: RGB, grayscale
        - Image filters and effects
        - Edge detection
        - Shape detection
        - Color detection
        - Face detection with OpenCV
        - Drawing on images
        - Training an image model the Teachable Machine way, with your own examples
        - Testing fairly: when does your model get confused?
      Projects built:
        - Photo editor with filters
        - Face detection app
        - Color tracker game
        - Shape identifier
      Practice: Create 5 image processing programs
    Capstone and the Final Exam
      Topics:
        - Choosing the capstone: a game, a smart helper, a turtle-art collection or a camera project
        - Planning with a checklist, building in small steps
        - Using everything: logic, functions, files, maybe Pygame, maybe a trained model
        - Testing with a real player or user
        - Rehearsing the demo: what it does, how it works, what was hard
      Projects built:

- THE CAPSTONE: a complete Python project presented at demo day
Practice: A little building every day; keep it always working
Assessment: FINAL EXAM: capstone demo plus a friendly viva and a mixed quiz from the whole course. Passing earns the certificate; a child who falls short gets focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: talking programs, joke machines, quizzes and guessing games
- Phase 2: turtle art worth printing, list-powered tools and a personal function library
- Phase 3: save-file apps, first classes, and complete Pygame games with scores and sounds
- Phase 4: a rules-based smart helper, data detective findings, a self-trained camera model, and the capstone

Total projects: 30+ typed Python projects, crowned by a capstone presented at demo day

SKILLS THE STUDENT WILL HAVE AT THE END

- Real Python: variables, decisions, loops, lists, functions, files and first classes
- Turtle graphics: loops turned into art
- Pygame: the game loop, sprites, collisions, scores and sounds
- Kid-safe AI: rules-based smarts, data detective work, and models trained by the child
- Coder habits: debugging patiently, naming clearly, finishing what you start

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, typing along with the teacher
Homework: A small build after every class, and a mixed challenge at the end of every month
Practice: 2-3 hours of playful practice between classes
Review: Every project reviewed with teacher feedback; wobbly skills reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so no child stays stuck and discouraged
Progress updates: Regular progress notes to parents: what was built, what comes next, and how to encourage practice

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam, a capstone demo plus a friendly viva. A child who falls short gets focused revision and a free retest: the certificate certifies skills, not attendance

WHERE THIS CAN LEAD

- Python for Teens: the direct continuation at 13, entered with real fluency
- AI and Machine Learning for Teens: the honest next step for the AI spark
- Web Development for Kids: a second builder skill while Python settles
- School maths and logic, quietly upgraded by a year of real programming
- The family tech helper: automations and tools for the household

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my child build in this Python course?
A: More than 30 typed Python projects across eight months: joke machines and quiz games in the first weeks, turtle art worth printing, list-powered tools, save-file apps, complete Pygame games with scores and sounds, and in the AI phase a rules-based smart helper, data detective findings and a camera model the child trains personally. It all ends with a capstone presented at demo day.

Q: What age is right for this course?
A: Ages 9 to 12: old enough for real typing and abstract-enough thinking, young enough that the playful pace fits perfectly. Younger children thrive better in our Scratch-first Kids Coding course, which builds toward exactly this. Batches are grouped by age.

Q: How long does it take a child to learn Python?
A: Eight months at 2 live classes a week plus 2 to 3 hours of playful practice takes a child from first line to a demo-day capstone with games and a trained model behind them. The course keeps two more months in hand for children who need more repetition, no rushing, no penalty.

Q: How is my child assessed? Is the certificate just for attending?
A: No, and it is all gentle: classwork every session, a small build after every class, a monthly mixed challenge that revisits earlier skills so they stop fading, and a final exam, a capstone demo plus a friendly viva and short quiz. Passing earns the certificate; a child who falls short gets focused revision and a free retest.

Q: How is AI taught to children this young?
A: By building it small and honest. Children write rules-based smart programs in pure Python, do data detective work on fun datasets, and train camera models Teachable-Machine style with their own examples, then hunt for the model's mistakes. There are no chatbot accounts: the major AI chat tools are 13+ by their own rules, and we say that plainly while building the skills that will make such tools truly useful later.

Q: Python or Scratch first for my child?
A: Scratch first for ages 6-8, Python from around 9 when typing and patience are ready. Children who arrive from Scratch fly here: the logic transfers instantly and the "real code" feeling is a powerful motivator. Our placement check settles it honestly either way.

Q: Is any paid software needed?
A: No. Python, the editor, Pygame and every tool used are free, and any reasonably recent computer runs them comfortably. No AI subscriptions exist in this course by design.

Q: Is there a free trial class?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Your child will write and run real Python inside the demo. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real teacher watching the child's actual code, never pre-recorded videos. At this age the missing colon is the whole battle, and live eyes win it kindly.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/python-ai-kids-masterclass
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 117 OF 137: PYTHON AUTOMATION COURSE: SCRIPTING, SCRAPING & APPLIED AI
==========

Course link: https://learn.modernagecoders.com/courses/python-ai-automation-masterclass-college
Subject area: AI & Automation
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Python automation course for college students and professionals: Python core, web scraping, browser automation and RPA, then applied AI for documents, text and forecasts, in 10-12 months.

In one line: Make the computer do the work: Python properly, then scraping, browser automation and RPA, then applied AI that reads documents, text and time series.

ABOUT THE PROGRAMME

The most immediately useful programming skill in any office is automation: the report that assembles itself, the data that collects itself, the document pile that processes itself overnight. This live online masterclass builds that skill honestly: Python learned properly first, then the automation core, web scraping done right, Selenium and browser automation, RPA patterns, APIs and scheduled pipelines with testing and security habits, then the applied AI layer that multiplies it: computer vision for documents and images, NLP and transformers for text, time series for forecasts, cloud AI services used with judgment, and the honest ethics of automating decisions. A finale month ships an end-to-end automation capstone and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- Automation-first, not research-first: every AI topic earns its place by automating real work

- Scraping taught responsibly: robots.txt, rate limits and terms, the professional way
- Applied AI with judgment: when a cloud API beats a hand-built model, we say so
- Pipelines that run unattended: scheduling, logging and failure handling as first-class topics
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Python foundations (Months 1-3): the language, data handling, databases and APIs
Phase 2: The automation core (Months 4-6): scraping, browser automation, RPA and practical ML
Phase 3: Applied AI (Months 7-9): vision for documents, NLP, time series and cloud AI services
Phase 4: The finale (Month 10): an end-to-end automation capstone and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Everything is taught from the first line
Equipment: Any computer with a stable internet connection; every tool used is free
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

Working professionals: Professionals drowning in repetitive computer work
College students: Students who want the most demonstrably useful Python specialization
Ops and analysts: Operations and analyst roles ready to automate their own jobs upward
Career switchers: Newcomers who want visible, useful results from month one
Business owners: Owners who want their processes to run without them

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PYTHON FOUNDATION & CORE PROGRAMMING (MONTHS 1-3, WEEKS 1-13)

Build rock-solid Python fundamentals. Learn programming concepts, master Python syntax, and create your first automation projects.

Month 1: Python Fundamentals & Programming Logic (Weeks 1-5)
Introduction to Python & Setup
Topics:
- What is Python? History and why it's perfect for automation
- Python ecosystem: CPython, PyPy, Anaconda
- Installing Python 3.x on Windows, Mac, Linux
- IDE setup: VS Code, PyCharm, Jupyter Notebook
- Virtual environments: venv, virtualenv, conda
- Package management with pip
- Your first Python program
- Python interactive shell and REPL
- Basic input/output operations
- Comments and documentation
- Python code style guide (PEP 8)
- Introduction to Python's philosophy
Projects built:
- Set up complete Python development environment
- Simple calculator program
- Personal information collector
Practice: Daily: 30 min coding exercises, 1 hour Python basics
Variables, Data Types & Operations
Topics:
- Variables and naming conventions
- Dynamic typing in Python
- Basic data types: int, float, str, bool
- Type conversion and casting
- Numbers: integers, floats, complex
- Arithmetic operators and precedence
- String creation and manipulation
- String methods: upper, lower, strip, split, join
- String formatting: f-strings, format(), %
- String slicing and indexing
- Boolean operations and truth values
- Comparison and logical operators
Projects built:
- Temperature converter (C/F/K)
- Text analyzer and formatter
- Simple encryption/decryption tool
Practice: Complete 50 string manipulation exercises

```
  Control Flow & Loops
    Topics:
      - If-elif-else statements
      - Nested conditional statements
      - Ternary operators
      - While loops and break/continue
      - For loops and range() function
      - Iterating through sequences
      - Nested loops and patterns
      - Loop else clause
      - List comprehensions introduction
      - Common loop patterns
      - Debugging techniques
      - Error messages and troubleshooting
    Projects built:
      - Number guessing game
      - Pattern generator (pyramids, diamonds)
      - Prime number calculator
      - Password strength validator
    Practice: Solve 40 loop and condition problems
  Data Structures in Python
    Topics:
      - Lists: creation, indexing, slicing
      - List methods: append, extend, insert, remove, pop
      - List comprehensions advanced
      - Tuples: immutable sequences
      - Sets: unique elements, set operations
      - Dictionaries: key-value pairs
      - Dictionary methods and operations
      - Nested data structures
      - Copying: shallow vs deep copy
      - Memory management basics
      - Collections module: defaultdict, Counter, deque
      - Data structure performance comparison
    Projects built:
      - Student management system
      - Shopping cart application
      - Contact book with search
      - Data structure performance analyzer
    Practice: Build 10 programs using different data structures

Month 2: Functions, Modules & OOP (Weeks 6-9)
  Functions & Functional Programming
    Topics:
      - Function definition and calling
      - Parameters and arguments
      - Default parameters and keyword arguments
      - *args and **kwargs
      - Return values and multiple returns
      - Scope and LEGB rule
      - Global and nonlocal keywords
      - Lambda functions
      - Map, filter, and reduce
      - Decorators basics
      - Generators and yield
      - Recursion and recursive algorithms
    Projects built:
      - Modular calculator with functions
      - Recursive file system explorer
      - Function library for text processing
      - Custom decorator collection
    Practice: Create 30 reusable utility functions
  Object-Oriented Programming
    Topics:
      - Classes and objects
      - Attributes and methods
      - Constructor (__init__)
      - Instance vs class variables
      - Inheritance and polymorphism
      - Method overriding
      - Multiple inheritance and MRO
      - Abstract classes and interfaces
      - Magic methods (dunder methods)
      - Property decorators
      - Static and class methods
      - SOLID principles in Python
    Projects built:
      - Bank account management system
      - Employee hierarchy system
```

```
      - Game character classes
      - Library management OOP
    Practice: Design 10 real-world systems using OOP
  Modules, Packages & Error Handling
    Topics:
      - Modules and import statements
      - Creating custom modules
      - Package structure and __init__.py
      - Relative and absolute imports
      - The Python Standard Library tour
      - Exception handling: try, except, finally
      - Raising custom exceptions
      - Exception hierarchy
      - Logging module usage
      - Debugging with pdb
      - Unit testing with unittest
      - Test-driven development basics
    Projects built:
      - Custom package for utilities
      - Error handling framework
      - Logging system implementation
      - Test suite for previous projects
    Practice: Add proper error handling to all previous projects
  File Handling & Data Processing
    Topics:
      - File operations: open, read, write, close
      - Context managers (with statement)
      - Text file processing
      - CSV file handling
      - JSON data manipulation
      - XML parsing basics
      - Binary file operations
      - Path manipulation with pathlib
      - Directory operations
      - Regular expressions with re module
      - Pattern matching and extraction
      - Data serialization with pickle
    Projects built:
      - Log file analyzer
      - CSV to JSON converter
      - Bulk file renamer
      - Text search and replace tool
    Practice: Process 20 different file formats

Month 3: Advanced Python & Automation (Weeks 10-13)
  Advanced Python Concepts
    Topics:
      - Iterators and iteration protocol
      - Creating custom iterators
      - Generator expressions
      - Coroutines and async basics
      - Context managers and contextlib
      - Metaclasses introduction
      - Descriptors and properties
      - Abstract base classes
      - Multiple inheritance patterns
      - Mixin classes
      - Design patterns in Python
      - Performance optimization techniques
    Projects built:
      - Custom data structure library
      - Async task processor
      - Plugin system architecture
      - Performance monitoring tool
    Practice: Implement 10 design patterns in Python
  Database Programming
    Topics:
      - SQLite with Python
      - SQL basics: CREATE, INSERT, SELECT, UPDATE, DELETE
      - Database connections and cursors
      - Parameterized queries and SQL injection prevention
      - PostgreSQL with psycopg2
      - MySQL with PyMySQL
      - MongoDB with pymongo
      - SQLAlchemy ORM basics
      - Database migrations
      - Connection pooling
      - Redis for caching
      - Database backup and restore
```

```
      Projects built:
        - Inventory management system
        - Blog platform backend
        - User authentication system
        - Data migration tool
      Practice: Build 5 database-driven applications
    Web Development with Python
      Topics:
        - HTTP protocol basics
        - Flask framework introduction
        - Routes and views in Flask
        - Templates with Jinja2
        - Forms and validation
        - Session management
        - RESTful API design
        - FastAPI framework
        - Django basics
        - MVC/MVT architecture
        - Authentication and authorization
        - Deployment basics
      Projects built:
        - Personal blog with Flask
        - REST API for todo app
        - FastAPI microservice
        - Django admin panel
      Practice: Create 5 different web applications
    Network Programming & APIs
      Topics:
        - Sockets programming basics
        - TCP and UDP protocols
        - Creating client-server applications
        - HTTP requests with requests library
        - Consuming REST APIs
        - API authentication methods
        - GraphQL with Python
        - WebSocket connections
        - Email automation with smtplib
        - SSH connections with paramiko
        - FTP operations
        - Network security basics
      Projects built:
        - Chat application (client-server)
        - API aggregator service
        - Email automation system
        - Network monitoring tool
      Practice: Integrate with 10 different APIs
      Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

---------------------------------------------------------------
PHASE 2: WEB SCRAPING & AUTOMATION TOOLS (MONTHS 4-6, WEEKS 14-26)
---------------------------------------------------------------
Master web scraping, browser automation, task automation, and building robust automation
frameworks.

  Month 4: Web Scraping & Browser Automation (Weeks 14-18)
    Web Scraping Fundamentals
      Topics:
        - HTML/CSS basics for scraping
        - BeautifulSoup library
        - Parsing HTML documents
        - Finding elements: find, find_all
        - CSS selectors in BeautifulSoup
        - Navigating the parse tree
        - Extracting text and attributes
        - Handling dynamic content basics
        - robots.txt and ethical scraping
        - User-Agent headers
        - Rate limiting and delays
        - Scraping best practices
      Projects built:
        - News article scraper
        - Product price monitor
        - Job listing aggregator
        - Wikipedia data extractor
      Practice: Scrape 20 different websites
    Advanced Web Scraping
      Topics:
        - Scrapy framework introduction
```

- Scrapy spiders and items
- Scrapy pipelines
- Handling pagination
- Following links recursively
- Handling forms and CSRF tokens
- Session management in scraping
- Proxy rotation
- Handling CAPTCHAs
- Scraping JavaScript-rendered pages
- API reverse engineering
- Data cleaning and validation

Projects built:
- E-commerce site scraper with Scrapy
- Social media data collector
- Real estate listing scraper
- Multi-site comparison tool

Practice: Build 10 production-ready scrapers

Selenium & Browser Automation

Topics:
- Selenium WebDriver setup
- Browser drivers: Chrome, Firefox, Edge
- Finding elements with Selenium
- Interacting with elements: click, send_keys
- Waiting strategies: implicit, explicit, fluent
- Expected conditions
- Handling popups and alerts
- Frame and window switching
- JavaScript execution
- Screenshots and video recording
- Headless browser mode
- Selenium Grid for parallel execution

Projects built:
- Automated form filler
- Social media bot
- Web testing automation suite
- Booking automation system

Practice: Automate 15 web workflows

Alternative Automation Tools

Topics:
- Playwright introduction
- Playwright vs Selenium comparison
- Puppeteer with Pyppeteer
- Requests-HTML for JavaScript sites
- MechanicalSoup for simple automation
- AutoGUI for desktop automation
- Keyboard and mouse control
- Screen capture and OCR
- Task scheduling with schedule library
- Cron jobs and task schedulers
- Windows Task Scheduler integration
- Building automation frameworks

Projects built:
- Desktop automation suite
- Cross-browser testing tool
- Automated report generator
- System administration automator

Practice: Create 10 different automation workflows

Data Processing & Analysis

Topics:
- NumPy arrays and operations
- Pandas DataFrames basics
- Data loading and saving
- Data cleaning with Pandas
- Data transformation and aggregation
- Merge, join, and concatenate
- GroupBy operations
- Time series data handling
- Data visualization with Matplotlib
- Seaborn for statistical plots
- Plotly for interactive visualizations
- Creating automated reports

Projects built:
- Sales data analysis dashboard
- Stock market analyzer
- Survey data processor
- Automated reporting system

Practice: Analyze 10 real-world datasets

Month 5: Enterprise Automation & Integration (Weeks 19-23)

RPA & Process Automation
  Topics:
    - Robotic Process Automation concepts
    - Python RPA libraries
    - UiPath integration with Python
    - Email automation advanced
    - Excel automation with openpyxl
    - Word document automation
    - PDF processing and generation
    - PowerPoint automation
    - Calendar and scheduling automation
    - Database automation workflows
    - File system monitoring
    - Process orchestration
  Projects built:
    - Invoice processing system
    - HR onboarding automation
    - Report generation pipeline
    - Document management system
  Practice: Automate 10 business processes
API Development & Microservices
  Topics:
    - RESTful API design principles
    - FastAPI advanced features
    - API versioning strategies
    - Authentication: JWT, OAuth2
    - Rate limiting and throttling
    - API documentation with Swagger
    - GraphQL API development
    - Microservices architecture
    - Message queues: RabbitMQ, Celery
    - Event-driven architecture
    - Service discovery
    - API gateway patterns
  Projects built:
    - Complete REST API with auth
    - Microservices e-commerce backend
    - Real-time notification service
    - API gateway implementation
  Practice: Build 5 production APIs
Cloud Automation & DevOps
  Topics:
    - AWS SDK for Python (boto3)
    - EC2 instance management
    - S3 operations and automation
    - Lambda functions with Python
    - SQS and SNS integration
    - Google Cloud Platform basics
    - Azure SDK for Python
    - Infrastructure as Code with Python
    - Docker containerization
    - Kubernetes with Python client
    - CI/CD pipeline automation
    - Monitoring and alerting
  Projects built:
    - Cloud resource manager
    - Serverless data processor
    - Auto-scaling application
    - Infrastructure automation tool
  Practice: Deploy 10 applications to cloud
Testing & Quality Assurance
  Topics:
    - Unit testing with pytest
    - Test fixtures and parametrization
    - Mocking and patching
    - Integration testing strategies
    - API testing with pytest
    - Selenium for test automation
    - Performance testing with Locust
    - Load testing strategies
    - Security testing basics
    - Code coverage analysis
    - Continuous testing
    - Test reporting and documentation
  Projects built:
    - Complete test suite for API
    - E2E testing framework
    - Performance testing suite
    - Automated regression tests

```
    Practice: Write tests for all previous projects
  Security & Best Practices
    Topics:
      - Security in Python applications
      - Input validation and sanitization
      - SQL injection prevention
      - XSS and CSRF protection
      - Password hashing with bcrypt
      - Encryption with cryptography library
      - Secure API development
      - Environment variables and secrets
      - Security scanning tools
      - OWASP Top 10 for Python
      - Audit logging
      - Compliance and regulations
    Projects built:
      - Security audit tool
      - Encryption utility suite
      - Secure authentication system
      - Compliance checker tool
    Practice: Security audit all projects

Month 6: Practical Machine Learning (Weeks 24-26)
  Machine Learning Fundamentals
    Topics:
      - What is Machine Learning?
      - Types: Supervised, Unsupervised, Reinforcement
      - ML workflow and pipeline
      - Data preprocessing techniques
      - Feature engineering basics
      - Train-test split
      - Cross-validation
      - Evaluation metrics
      - Overfitting and underfitting
      - Bias-variance tradeoff
      - Scikit-learn introduction
      - Model selection strategies
    Projects built:
      - Iris classification project
      - House price prediction
      - Customer segmentation
      - Basic recommendation system
    Practice: Complete 10 ML tutorials
  Supervised Learning Algorithms
    Topics:
      - Linear regression
      - Logistic regression
      - Decision trees
      - Random forests
      - Support Vector Machines
      - K-Nearest Neighbors
      - Naive Bayes
      - Gradient Boosting (XGBoost, LightGBM)
      - Ensemble methods
      - Hyperparameter tuning
      - Grid search and random search
      - Feature importance analysis
    Projects built:
      - Credit risk assessment
      - Sales forecasting model
      - Email spam classifier
      - Disease prediction system
    Practice: Implement all algorithms from scratch
  Unsupervised Learning & Deep Learning Intro
    Topics:
      - K-Means clustering
      - Hierarchical clustering
      - DBSCAN
      - Principal Component Analysis
      - Dimensionality reduction
      - Anomaly detection
      - Association rules
      - Introduction to neural networks
      - Perceptron and backpropagation
      - Activation functions
      - TensorFlow basics
      - Keras API
    Projects built:
      - Customer segmentation advanced
```

- Anomaly detection system
- Market basket analysis
- Simple neural network from scratch

Practice: Solve 15 unsupervised learning problems

Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 3: APPLIED AI FOR AUTOMATION (MONTHS 7-9, WEEKS 27-39)

Master advanced AI techniques, computer vision, natural language processing, and production AI systems.

Month 7: Computer Vision & Image Processing (Weeks 27-35)

Image Processing Fundamentals

Topics:

- Digital image basics
- OpenCV installation and setup
- Image loading and display
- Color spaces: RGB, HSV, LAB
- Image transformations
- Geometric transformations
- Image filtering and blurring
- Edge detection: Canny, Sobel
- Morphological operations
- Histogram equalization
- Image thresholding
- Contour detection

Projects built:

- Image enhancement tool
- Document scanner
- Photo editor with filters
- Edge detection application

Practice: Process 100 different images

Advanced Computer Vision

Topics:

- Feature detection: SIFT, SURF, ORB
- Feature matching
- Object detection basics
- Haar Cascades
- Face detection and recognition
- Object tracking algorithms
- Optical flow
- Background subtraction
- Template matching
- Image segmentation
- Panorama stitching
- 3D reconstruction basics

Projects built:

- Face recognition system
- Object tracking application
- Motion detection security system
- Augmented reality basics

Practice: Build 10 computer vision applications

Deep Learning for Computer Vision

Topics:

- CNN architectures: LeNet, AlexNet, VGG
- ResNet and skip connections
- Object detection: YOLO, R-CNN
- Semantic segmentation
- Instance segmentation
- Face recognition with deep learning
- Pose estimation
- Image generation: GANs basics
- Style transfer
- Image super-resolution
- Video analysis with deep learning
- Real-time processing optimization

Projects built:

- Custom object detector
- Real-time face mask detector
- Pose estimation fitness app
- Style transfer application

Practice: Train 15 vision models

Natural Language Processing

Topics:

- NLP fundamentals
- Text preprocessing: tokenization, stemming
- Lemmatization and POS tagging

```
      - Named Entity Recognition
      - Bag of Words and TF-IDF
      - Word embeddings: Word2Vec, GloVe
      - Sentiment analysis techniques
      - Topic modeling: LDA
      - Text classification
      - Sequence-to-sequence models
      - Attention mechanisms
      - BERT and Transformers introduction
    Projects built:
      - Sentiment analysis API
      - Text summarization tool
      - Named entity extraction system
      - Language detection service
    Practice: Process 50 text datasets
  Advanced NLP with Transformers
    Topics:
      - Transformer architecture deep dive
      - Hugging Face library
      - Pre-trained models: BERT, GPT, RoBERTa
      - Fine-tuning transformers
      - Text generation with GPT
      - Question answering systems
      - Text translation
      - Chatbot development
      - Document similarity
      - Information extraction
      - Multi-lingual NLP
      - NLP model deployment
    Projects built:
      - AI chatbot with transformers
      - Document Q&A system
      - Multi-language translator
      - Content generation tool
    Practice: Fine-tune 10 transformer models

Month 8: Time Series, Ethics & Production AI (Weeks 36-38)
  Time Series & Forecasting
    Topics:
      - Time series analysis basics
      - Trend and seasonality
      - Stationarity testing
      - ARIMA models
      - Seasonal ARIMA (SARIMA)
      - Exponential smoothing
      - Prophet library
      - LSTM for time series
      - Multi-variate time series
      - Anomaly detection in time series
      - Real-time forecasting
      - Evaluation metrics for forecasting
    Projects built:
      - Stock price predictor
      - Weather forecasting system
      - Sales forecasting tool
      - Anomaly detection for IoT
    Practice: Forecast 20 different time series
  AI Ethics & Explainable AI
    Topics:
      - AI bias and fairness
      - Ethical considerations in AI
      - Privacy-preserving ML
      - Federated learning basics
      - Model interpretability
      - LIME and SHAP
      - Feature importance visualization
      - Model debugging techniques
      - Adversarial attacks and defense
      - Responsible AI practices
      - AI governance
      - Regulatory compliance (GDPR, etc.)
    Projects built:
      - Bias detection tool
      - Model explainability dashboard
      - Privacy-preserving ML system
      - AI audit framework
    Practice: Audit 10 AI models for bias and fairness
  MLOps & Production Systems
    Topics:
```

- MLOps principles and practices
- Model versioning with DVC
- Experiment tracking: MLflow, Weights & Biases
- Model registry and management
- Continuous integration for ML
- Model monitoring in production
- A/B testing for ML
- Model retraining pipelines
- Feature stores
- Model serving: REST, gRPC
- Kubernetes for ML workloads
- Cost optimization for ML

Projects built:
- Complete MLOps pipeline
- Model monitoring dashboard
- Automated retraining system
- ML experiment tracker

Practice: Set up MLOps for 5 projects

Month 9: Cloud AI Services in Automation (Week 39)

Cloud AI Services & Deployment

Topics:
- AWS SageMaker
- Google Cloud AI Platform
- Azure Machine Learning
- Model deployment strategies
- Serverless AI with Lambda
- Container deployment for ML
- Model serving with TorchServe
- TensorFlow Serving
- API Gateway for ML models
- Load balancing for ML
- Auto-scaling ML services
- Cost management for cloud AI

Projects built:
- Deploy model to all major clouds
- Serverless ML API
- Auto-scaling ML service
- Multi-cloud AI deployment

Practice: Deploy 15 models to production

Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)

The capstone month: spec, build, ship and defend, then the final exam that decides the certificate.

Month 10: Capstone and the Final Exam (Weeks 40-44)

Capstone Spec and Setup

Topics:
- Choosing the capstone: an end-to-end automation: a scraping-to-report pipeline, a document AI processor or a business workflow bot
- A one-page spec: scope, milestones, stack
- Repository and skeleton ready on day one
- Instructor sign-off before the build begins

Projects built:
- Approved capstone spec plus the running skeleton

Practice: Pitch the spec to the batch and absorb one hard question

Capstone Build

Topics:
- Two focused weeks in vertical slices
- Instructor checkpoints and honest scope cuts
- Using the whole course: Python depth, scraping and browser automation, applied AI and scheduled pipelines
- Daily commits and an always-working build

Projects built:
- The capstone, feature-complete with version control

Practice: A working build at the end of every session

Ship and Polish

Topics:
- The finishing pass: edge cases, errors, documentation
- Scheduled and unattended: the automation running on its own, with logging you can trust
- A README worth reading and a demo worth watching
- A full-course spiral review before the exam

Projects built:
- The capstone shipped, documented and rehearsed

Practice: Two mock demos, each tighter than the last

Demo Day and the Final Exam
Topics:
- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: automation engineering, data engineering, or deeper machine learning
- Where our other courses continue the journey
Projects built:
- Demo day presentation delivered to the batch
Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Python programs, database-backed tools and API clients
- Phase 2: scrapers, browser bots and RPA pipelines with tests
- Phase 3: document AI, text and forecasting automations
- Phase 4: the end-to-end capstone automation, running unattended

Total projects: 20+ real automations, crowned by an unattended end-to-end capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Python to real depth: data structures, OOP, files, databases and APIs
- Web scraping done responsibly, at production quality
- Browser automation and RPA: Selenium and modern tooling
- Applied AI: document vision, NLP with transformers, time series forecasting
- Pipeline craft: scheduling, logging, testing and failure handling

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Automation and scripting roles across every industry
- Data engineering: pipelines are the shared craft
- Our AI/ML course for model depth beyond the applied layer
- Backend Python: from automation scripts to full services

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Growing roles: Rs 4-6 LPA (Python Developer)
Experienced roles: Rs 8-15 LPA (AI/ML Engineer); Rs 15-25 LPA (Senior AI Engineer); Rs 20-40+ LPA (AI Architect/Lead)
Freelance: Rs 1,000-5,000/hour
International: $60k-150k USD based on location and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this Python automation course cover?
A: Python properly first: core language, OOP, files, databases and APIs. Then the automation core: web scraping done responsibly, Selenium and browser automation, RPA patterns, testing and security for pipelines, plus practical machine learning. A finale month ships an end-to-end automation capstone and sits the final exam.

Q: How long does it take to learn Python automation properly?

A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two more months in hand for learners who need them. The three teaching phases build the skill and its real-world application; the finale month ships a capstone and sits the final exam.

Q: How am I assessed? Is the certificate just for attending?
A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: How is this different from your AI/ML and data science courses?
A: By what the AI is for. The AI/ML course builds model depth: the mathematics, training and evaluation of learning systems. This course is automation-first: AI appears as a tool that reads documents, classifies text and forecasts demand inside pipelines that do real work unattended. If you want to build models, take AI/ML; if you want work to do itself, this is the course.

Q: Is web scraping legal to learn and use?
A: Scraping is a standard professional skill used by search engines, price trackers and researchers everywhere, and like any powerful tool it has rules. We teach it responsibly from the first week: respecting robots.txt and rate limits, reading terms of service, preferring official APIs when they exist, and never touching credentials or private data. That judgment is part of the syllabus, not a footnote.

Q: Can I try a class before enrolling?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/python-ai-automation-masterclass-college
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 118 OF 137: PYTHON FOR TEENS: COMPLETE COURSE FROM BEGINNER TO ADVANCED

==========

Course link: https://learn.modernagecoders.com/courses/python-complete-masterclass-teens
Subject area: Python Programming
Designed for: Teens
Who can join: Absolute beginner, ages 13 to 18
Course length: 10-12 months (40-48 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Python course for teens aged 13-18. Live online classes, 30+ projects: games, web apps, automation, data and AI. Beginner to advanced in 10-12 months.

In one line: One language carries a teenager from a first print statement to games, web apps, automation and AI projects. This is that road, walked properly in 10 months.

ABOUT THE PROGRAMME

Python is the language teenagers can grow with: simple enough that a working program exists by the end of class one, serious enough that it runs Instagram's backend and most of modern AI. This live online course takes students aged 13 to 18 from zero to genuinely advanced, with every concept practiced through projects a teen actually wants to build and keep. The first three months build real foundations: variables to functions, the core data structures, files and JSON, algorithms, and object-oriented programming taught with patience instead of jargon. The middle months turn skill into products: 2D games with Pygame that get shared online, desktop apps, automation scripts that do real chores, web scraping done politely, and a first taste of data analysis with pandas. The final months put Python on the web with Flask, databases, user accounts and a real deployment, then finish with machine learning

fundamentals, AI-powered features built on modern AI APIs, and a capstone shipped like a real product. The pace is honest: two live classes a week plus three to four hours of practice, and the syllabus fits ten months of that rhythm. Students who need longer take up to twelve months, the class adapts instead of rushing ahead. Learning is checked the honest way too: homework after every class, monthly assignments that mix new questions with earlier topics, and a final exam that decides the certificate. Real knowledge, real work, real projects, depth over dopamine. By the end, a student owns a portfolio of 30+ projects, a web app on a public URL, and the kind of understanding that makes the next step, whether AI, data science or competitive programming, feel natural.

WHAT MAKES THIS COURSE DIFFERENT

- Projects teens choose to keep: games, bots, automations and apps they actually use, not textbook exercises
- An honest 10-month arc with 2 months of margin built in: students who need more practice time take up to 12, nobody is rushed
- Modern Python throughout: the latest Python 3, VS Code, pytest, pandas and real AI APIs, the same tools working developers use
- Every submitted project comes back with written feedback, and recurring gaps are reopened in class instead of left behind
- A deployed capstone: the course ends with an app on the public internet, not screenshots on a laptop
- Live batches capped at 20 students, with mini-batch and 1-on-1 options for closer attention
- Real assessment: weekly homework, mixed-review assignments every month, and a final exam that decides the certificate, with focused revision and a free retest for students who fall short

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations and OOP (Months 1-3): from first program to data structures, algorithms and classes, practiced through weekly builds
Phase 2: Games, apps and automation (Months 4-6): Pygame games, desktop tools, scrapers, and a first taste of data analysis with pandas
Phase 3: Python on the web (Months 7-8): Flask, SQL databases, user accounts, and a real deployment to a public URL
Phase 4: AI and the capstone (Months 9-10): machine learning fundamentals, AI-powered features, Git, testing, and a shipped final project

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The course starts from the very first line of code
Computer skills: Comfortable typing and managing files and folders
Equipment: A computer with a stable internet connection; every tool used in the course is free
Age: 13 to 18. Batches are grouped by age and pace so the room always fits
Time commitment: 2 live classes plus 3-4 hours of practice a week

WHO THIS COURSE IS FOR

Complete beginners: Teens starting from zero who want a structured road instead of a pile of scattered tutorials
Game builders: Students who learn best when the output is a game they can hand to a friend
Future engineers: Teens eyeing engineering or computer science who want the real foundation under the grades
Automators: The kind of student who would rather write a script than do a boring task twice
Ai curious: Teens who want to build with AI and understand it, not just chat with it

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PYTHON FOUNDATIONS AND OOP (MONTHS 1-3, WEEKS 1-12)

From the first line of code to data structures, algorithms and object-oriented programming. Every week ends with something built.

Month 1: First Programs (Weeks 1-4)
Introduction and Setup
Topics:
- Why Python? The language that runs Instagram, YouTube, and NASA
- Installing the latest Python 3 and understanding versions
- VS Code setup with Python extensions
- Terminal basics: running programs like a developer
- Understanding Python's philosophy: The Zen of Python
- Variables and memory: How Python stores data
- Data types deep dive: int, float, str, bool, None
- Input/Output: Building interactive programs
- Comments and documentation: Writing readable code
Projects built:

```
      - A personal greeting bot that asks questions and answers back
      - Calculator with memory
      - Mad libs story generator
    Practice: Short daily challenges: 15 mini-programs in the first week
  Control Flow and Logic
    Topics:
      - If-elif-else: Teaching computers to make decisions
      - Boolean logic: and, or, not, and De Morgan's laws
      - Comparison operators and chaining
      - Match-case statements (Python 3.10+)
      - Ternary operators for elegant code
      - Truthy and falsy values in Python
      - Short-circuit evaluation
      - Exception handling basics: try, except, finally
    Projects built:
      - Rock-Paper-Scissors-Lizard-Spock game
      - Password strength analyzer with rules
      - Choose-your-own-adventure text game
    Practice: 20 logic puzzles, build 5 decision-based programs
  Loops and Iteration
    Topics:
      - For loops: Automating repetitive tasks
      - While loops: Conditional repetition
      - Nested loops and performance implications
      - Loop control: break, continue, else with loops
      - Range() function secrets and memory efficiency
      - Enumerate() for index tracking
      - Zip() for parallel iteration
      - List comprehensions: Python's superpower
      - Generator expressions for memory efficiency
      - Performance: When to use which loop
    Projects built:
      - Pattern generator (ASCII art)
      - Prime number visualizer
      - Multiplication table game
      - Progress bar implementation
    Practice: 25 loop challenges, then rebuild 3 earlier programs with less code
  Functions
    Topics:
      - Functions: Building blocks of programs
      - Parameters vs arguments: The complete guide
      - Return values and multiple returns
      - Default arguments and keyword arguments
      - *args and **kwargs: Flexible functions
      - Scope and LEGB rule
      - Closures and their power
      - Lambda functions and when to use them
      - Function annotations and type hints
      - Docstrings: writing functions someone else can use
      - Breaking a big program into small functions
    Projects built:
      - Homework helper library of reusable functions
      - Unit converter covering 20+ conversions
      - Menu-driven mini app assembled from functions
    Practice: Build a personal library of 15 reusable functions
    Assessment: Month 1 check: build a small menu-driven program live using input,
    conditionals, loops and functions, then a take-home assignment mixing everything from
    month 1

Month 2: The Data Structures Every Program Uses (Weeks 5-8)
  Lists and Tuples
    Topics:
      - Lists: Python's Swiss Army knife
      - List methods: the ones you will actually use, and how to look up the rest
      - Slicing notation mastery [start:stop:step]
      - List comprehensions with conditions
      - Nested list comprehensions
      - Tuples: Immutable sequences
      - Named tuples for readable code
      - List vs Tuple: Performance and use cases
      - Unpacking and starred expressions
      - Deep vs shallow copy issues
    Projects built:
      - Spotify playlist manager
      - Tic-tac-toe with AI opponent
      - Matrix calculator
      - Todo list with priorities
    Practice: 40 list and tuple drills, build 3 small data managers
  Dictionaries and Sets
```

```
    Topics:
      - Dictionaries: Key-value mastery
      - Dictionary methods and operations
      - Dictionary comprehensions
      - Nested dictionaries and JSON
      - OrderedDict, defaultdict, Counter
      - Sets: Unique collections
      - Set operations: union, intersection, difference
      - Frozen sets for immutable sets
      - When to use sets vs lists
      - Hash tables: How dictionaries work internally
      - Performance characteristics and Big O
    Projects built:
      - Contact manager with search
      - Word frequency analyzer
      - Inventory system for game
      - Social network graph analyzer
    Practice: 30 dictionary and set challenges, refactor 2 earlier projects to use them
  Strings and Text
    Topics:
      - Strings: More than just text
      - String methods: the toolkit you will actually reach for
      - F-strings: Modern string formatting
      - Regular expressions with re module
      - Unicode and encoding: UTF-8, ASCII
      - Text processing techniques
      - String algorithms: searching, matching
      - Template strings for safe substitution
      - Textwrap for formatting
    Projects built:
      - Wordle game clone
      - Text encryption tool
      - Markdown to HTML converter
      - Plagiarism checker
    Practice: 30 string problems, build 3 text tools
  Files, JSON and Errors
    Topics:
      - Reading and writing text files: programs that remember
      - The with statement, and why it protects your files
      - Working with paths using pathlib
      - CSV files: reading and writing spreadsheet-style data
      - JSON: saving structured data the way real apps do
      - Exceptions in practice: try, except, else, finally
      - Raising your own errors with clear messages
      - Designing a save-and-load feature for any program
      - Where real programs keep their data
    Projects built:
      - Journal app that saves entries between runs
      - Quiz game with its question bank stored in JSON
      - Expense tracker that writes and reads CSV
    Practice: Add save and load to three of your earlier projects
    Assessment: Month 2 check: build a small app that loads data, changes it and saves it
    back, live, with quiz questions that reach back to month 1

Month 3: Algorithms and Objects (Weeks 9-12)
  Power Collections
    Topics:
      - Collections module deep dive
      - Deque: Double-ended queue
      - Heapq: Priority queues in Python
      - Bisect: Maintaining sorted lists
      - Dataclasses for structured data
      - Enum for constants
      - Queue and stack implementations
      - Linked lists in Python
      - Trees and graphs: the intuition, drawn before coded
    Projects built:
      - LRU cache implementation
      - Task scheduler with priorities
      - Undo-redo system using stacks
    Practice: Implement a stack, a queue and a linked list from scratch
  Algorithms and Problem Solving
    Topics:
      - Algorithm complexity and Big O
      - Searching algorithms: Binary, jump, interpolation
      - Sorting algorithms: Quick, merge, heap, radix
      - Recursion and memoization
      - Greedy algorithms
      - Algorithm visualization
```

```
        - Problem-solving strategies
        - A taste of competitive programming, and where our DSA course goes deeper
        - Practicing on HackerRank and LeetCode without drowning
        - Breadth-first and depth-first search, on a maze you can see
      Projects built:
        - Sorting visualizer with Pygame
        - Pathfinding algorithm demo
        - Sudoku solver with GUI
        - Algorithm complexity analyzer
      Practice: Solve 25 algorithm problems, implement 8 classic algorithms
    Object-Oriented Programming
      Topics:
        - Classes and objects: Building blueprints
        - Attributes and methods
        - __init__ and object initialization
        - self: The magic first parameter
        - Class vs instance attributes
        - Property decorators for getters/setters
        - Method types: instance, class, static
        - __str__ and __repr__ for object representation
        - Encapsulation and information hiding
        - Documentation with docstrings
        - Type hints in classes
      Projects built:
        - Bank account system
        - RPG character creator
        - School management system
        - E-commerce cart system
      Practice: Design and build 8 classes for things you know: players, playlists, pets,
      plans
    Inheritance and Polymorphism
      Topics:
        - Inheritance: Building class hierarchies
        - super() and method resolution order (MRO)
        - Multiple inheritance and mixins
        - Abstract base classes (ABC)
        - Polymorphism and duck typing
        - Method overriding and extending
        - Composition vs inheritance
        - Interface design in Python
      Projects built:
        - Game entity system with players, enemies and items
        - Shape hierarchy that draws itself
        - School management system with people who share behavior
      Practice: Design 5 class hierarchies, defend each design choice in one sentence
      Assessment: Phase 1 exam: build a small OOP system live and explain the design
      decisions

---------------------------------------------------------------
PHASE 2: GAMES, APPS AND AUTOMATION (MONTHS 4-6, WEEKS 13-24)
---------------------------------------------------------------
Skills become products: real Pygame games shared online, desktop apps, automation that does
actual chores, and a first honest look at data.

  Month 4: Build Real Games (Weeks 13-16)
    Pythonic Objects
      Topics:
        - Magic methods (dunder methods) complete guide
        - Operator overloading with magic methods
        - Container protocols: __len__, __getitem__, __setitem__
        - Iterator protocol: __iter__, __next__
        - Context managers: __enter__, __exit__
        - Arithmetic magic methods
        - Comparison magic methods
        - Callable objects with __call__
        - Properties for safe attributes
        - Building a class other code enjoys using
      Projects built:
        - Vector class with real operator overloading
        - Deck-of-cards class ready for any card game
        - Smart inventory that validates what goes in
      Practice: Give three earlier classes proper __str__, __repr__ and comparison behavior
    Pygame: The Game Loop
      Topics:
        - How every game works: the loop, events, update, draw
        - Setting up Pygame and the game window
        - Drawing shapes, images and sprites
        - Keyboard and mouse input that feels responsive
        - Movement, velocity and frame rate
```

- Collision detection: rectangles first
- Sprite groups for clean game code
- Keeping score and rendering text
- Structuring a game file a friend could read

Projects built:
- Pong from scratch
- Dodge-the-asteroids arcade game
- Collect-the-coins game with a timer

Practice: Build one tiny game variant on your own between classes

Pygame: Juice and Polish

Topics:
- Sprite sheets and animation
- Simple physics: gravity, jumping, bouncing
- Sound effects and music
- Game states: menu, playing, game over
- Particle effects with restraint
- Scrolling levels and a simple camera
- Saving high scores to a file (month 2 pays off)
- Difficulty curves: making a game fair but tense
- Playtesting: watching someone else play your game

Projects built:
- Platformer with gravity, jumps and levels
- Space shooter with waves, sounds and a menu

Practice: Add sound, a menu and a high-score table to your month 4 games

Game AI and Shipping Your Game

Topics:
- AI for enemies: state machines that chase, patrol and flee
- Pathfinding on a grid: BFS first, then a look at A*
- Min-max for turn games: an unbeatable tic-tac-toe
- Procedural content generation
- Packaging a game so someone else can run it
- Publishing on itch.io and writing the game page

Projects built:
- Tic-tac-toe with an unbeatable AI opponent
- Enemy AI pack for your platformer: patrol, chase, flee
- Your best game published on itch.io

Practice: Ship it: publish one finished game and send the link to three people

Assessment: Game demo: present a finished, published game and explain one system inside it, plus a mixed-review assignment covering months 1-4

Month 5: Programs That Do Your Chores (Weeks 17-20)

Desktop Apps with Tkinter

Topics:
- Tkinter: windows, widgets and layouts
- CustomTkinter for apps that look like this decade
- GUI design principles
- Event-driven programming
- Packaging desktop apps
- Connecting buttons to your own functions
- Saving app data with JSON (your month 2 skills)
- Packaging a desktop app with PyInstaller

Projects built:
- Pomodoro study timer
- Flashcards app for any school subject
- Notes app with save and search

Practice: Design and build one desktop tool you will personally use this term

Automation: Files, Sheets and Documents

Topics:
- The automation mindset: never do a boring task three times
- Organizing folders and files with pathlib and shutil
- Renaming, sorting and de-duplicating hundreds of files safely
- Excel automation with openpyxl
- Reading and creating PDFs with pypdf
- Resizing and watermarking images with Pillow
- Sending yourself email reports from Python
- Scheduling scripts to run on their own
- Dry runs and undo plans: automating without disasters

Projects built:
- Downloads-folder organizer that runs on a schedule
- School timetable to calendar converter
- Photo batch resizer and watermarker

Practice: Automate 5 real chores from your own computer

Web Scraping, Politely

Topics:
- How web pages are structured: HTML through a scraper's eyes
- Downloading pages with requests
- Parsing with BeautifulSoup: find, select, extract
- Scraping tables and lists into CSV
- Being polite: robots.txt, delays and identifying yourself

- When scraping is the wrong tool: finding the site's API instead
- Handling pages that change or break
- A look at Playwright for JavaScript-heavy pages
- Legal and ethical lines, in plain language

Projects built:
- Book or game price tracker with history
- Sports statistics collector into CSV
- News headline digest that emails itself to you

Practice: Build one scraper for a site you actually check every week

Automation Capstone

Topics:
- Choosing an automation worth building: your most boring hour
- Breaking a chore into steps a script can do
- Combining skills: files, scraping, email, scheduling
- Logging: knowing what your script did while you slept
- Handling failure gracefully instead of silently
- Writing a README so future-you can maintain it
- Presenting an automation: before, after, time saved

Projects built:
- The personal automation suite: one real multi-step chore, fully automated, scheduled and logged

Practice: Run your automation for a full week and fix what reality breaks

Assessment: Automation review: demonstrate the suite running end to end and walk through the code, alongside a quiz that revisits earlier months

Month 6: Data Tells Stories (Weeks 21-24)

NumPy and pandas Foundations

Topics:
- NumPy arrays: why they beat lists for numbers
- pandas DataFrames: loading, filtering, sorting, grouping
- Data cleaning and preprocessing
- Exploratory data analysis (EDA)
- Dealing with missing data
- Outlier detection
- Reading CSVs from the real world, mess included
- Asking a dataset questions and trusting the answers

Projects built:
- Class marks analyzer with honest averages
- Personal screen-time or sports data breakdown
- Music streaming history explorer

Practice: Explore 3 real datasets and write down 5 true findings from each

Data Visualization

Topics:
- Matplotlib: the chart types that cover 90% of real needs
- Seaborn for statistical plots
- Plotly for interactive visualizations
- Animation creation
- Design for honesty: labels, scales and not misleading people
- Plotly for charts you can hover and zoom

Projects built:
- A visual report on a dataset you care about
- Interactive chart set built with Plotly

Practice: Make 10 charts, then rework the 3 weakest until they explain themselves

A Real Data Project

Topics:
- Choosing a dataset and a question worth asking
- Cleaning: missing values, weird rows, wrong types
- Exploratory analysis: distributions, groups, trends
- Correlation and the classic causation trap
- Building the story: from table to chart to sentence
- What data cannot tell you, said out loud
- Packaging analysis in a clean, rerunnable script

Projects built:
- Full analysis of a real public dataset: cleaned, explored, visualized and written up

Practice: Trade datasets with a classmate and check each other's conclusions

Phase 2 Capstone and Review

Topics:
- Portfolio thinking: choosing your best work so far
- Refactoring old code with new skills
- Code review: reading each other's projects kindly and usefully
- Filling gaps: a targeted review week shaped by the batch
- Writing project descriptions people understand
- Planning phase 3: what the web changes

Projects built:
- Polish pass on your two best phase 2 projects, README included

Practice: Publish your top 3 projects in a personal portfolio folder

Assessment: Phase 2 review: portfolio walkthrough of the term's builds, plus a written mixed review spanning phases 1 and 2

PHASE 3: PYTHON ON THE WEB (MONTHS 7-8, WEEKS 25-32)

How the web really works, then Flask: routes, templates, forms, a database, user accounts,
and a deployment to a public URL.

  Month 7: From Requests to Flask (Weeks 25-28)
    How the Web Works
      Topics:
        - What happens when you visit a URL: DNS, HTTP, servers, browsers
        - HTTP methods and status codes developers actually meet
        - HTML and CSS: a working crash course for Python developers
        - What JSON is doing in every modern app
        - APIs: how programs talk to other programs
        - Reading API documentation without fear
        - The client-server picture, drawn until it sticks
        - Developer tools: watching real traffic in the network tab
      Projects built:
        - A hand-built personal page in HTML and CSS
        - HTTP explorer: a script that inspects any site's responses
      Practice: Map the requests behind 3 sites you use daily
    Consuming APIs
      Topics:
        - Calling real APIs with requests
        - API keys, and keeping them out of your code with .env files
        - Parsing JSON responses into useful Python objects
        - Handling errors, timeouts and rate limits like a professional
        - Pagination: getting all the data, not just page one
        - Building a small client library around an API
        - Command-line tools people can actually use
        - Free public APIs worth building on
      Projects built:
        - Weather dashboard in the terminal
        - Currency converter with live rates
        - Trivia game powered by a public API
      Practice: Build clients for 3 different public APIs
    Flask: Your First Web App
      Topics:
        - Flask: a real web server in seven lines
        - Routes and view functions
        - Templates with Jinja2
        - Static files handling
        - Request and response objects
        - Debug mode and reading Flask errors calmly
        - Passing data from Python into pages
      Projects built:
        - Personal site served by Flask
        - Dice-roller and random-picker web tools
        - Quote-of-the-day site with rotating content
      Practice: Build 3 tiny Flask apps from scratch without notes
    Templates, Forms and Sessions
      Topics:
        - Jinja templates: loops and conditionals inside HTML
        - Template inheritance: one layout, many pages
        - Forms: getting data from the visitor safely
        - Validating input, and never trusting the browser
        - Redirects and flash messages
        - Sessions and cookies: remembering visitors
        - Organizing a Flask project as it grows
        - Styling with a lightweight CSS framework
      Projects built:
        - Multi-page site with one shared layout
        - Feedback form that validates and thanks properly
        - Mini poll app that remembers your vote
      Practice: Convert two phase 2 tools into small web apps
      Assessment: Month 7 check: build and explain a small Flask feature live, plus a mixed
      quiz reaching back to earlier phases

  Month 8: Databases, Users and Deployment (Weeks 29-32)
    Databases with SQLite and SQLAlchemy
      Topics:
        - SQL fundamentals: select, insert, update, delete, join
        - SQLite for development
        - SQLAlchemy models: tables as Python classes
        - Query optimization and indexing
        - Transactions and ACID properties
        - Designing tables for a real app before coding it
        - When a spreadsheet stops being enough
      Projects built:

- Book or game library app backed by SQLite
- Class notice board with posts stored properly

Practice: Design and build 3 small database schemas

User Accounts and Web Safety

Topics:

- Sign up and log in with Flask-Login
- Password hashing: why plain text is never stored, ever
- Protecting pages: login required
- Each user sees their own data: ownership in queries
- Web security awareness: SQL injection and XSS, shown safely
- CSRF protection with Flask-WTF
- Secrets and settings kept in environment variables
- A security checklist for every app you ship

Projects built:

- To-do app with real accounts, where every user has their own list

Practice: Add accounts to your week 29 app, then try to break your own login

Deploying to the Real Internet

Topics:

- What deployment actually means: your laptop is not a server
- Preparing an app: requirements.txt, gunicorn, configuration
- Deploying to Render step by step
- Environment variables in production
- Keeping secrets out of GitHub, permanently
- Custom domains and HTTPS, demystified
- Reading production logs when something breaks
- The deploy checklist you will reuse for years

Projects built:

- Your accounts app deployed to a public URL anyone can visit

Practice: Deploy a second app solo, using only your own checklist

Web Capstone

Topics:

- Scoping a web app you can finish in a week
- Building in slices: one working feature at a time
- Testing your app the way a stranger uses it
- Polish: empty states, error pages, small screens
- A mid-week review with the instructor
- Shipping and sharing the link

Projects built:

- The web capstone: a Flask app with a database, accounts and a live URL. Classic picks: link shortener, study planner, club site with sign-ups, or your own idea

Practice: Collect feedback from 3 real users and ship one improvement

Assessment: Web capstone review: live walkthrough from route to database to deployed URL, with cumulative questions from phases 1-3

PHASE 4: AI, QUALITY AND THE CAPSTONE (MONTHS 9-10, WEEKS 33-40)

Machine learning done honestly, AI features built on modern APIs, professional habits with Git and pytest, and a final capstone shipped like a product.

Month 9: AI and Real-World Tools (Weeks 33-36)

Machine Learning with scikit-learn

Topics:

- What machine learning actually is: functions learned from data
- Supervised learning algorithms
- Unsupervised learning techniques
- Evaluating honestly: train/test splits and why accuracy can lie
- Overfitting: the model that memorized the homework
- Reading what a model learned, and its limits

Projects built:

- Movie or game rating predictor
- Handwritten digit classifier
- Spam detector trained on real messages

Practice: Train 5 models, and for each write one sentence on where it fails

Building on AI APIs

Topics:

- Calling a modern AI model from Python (accounts set up with a parent where required)
- Structured output with Pydantic
- Function calling: giving the model real tools
- The agent loop: plan, act, observe, repeat
- Retrieval over your own documents
- Vector search basics, and when it is overkill
- Guardrails: validating what comes back
- When the agent is confidently wrong
- Cost control: tokens, caching, spend caps
- Automating a real, boring task end to end
- Evaluating an agent: does it actually work?

Projects built:

- Tool-using research helper

```
        - Document Q&A over your own class notes
        - Automate one real weekly chore with an AI step
      Practice: Build 3 small AI-powered scripts and note where each one is confidently wrong
    An AI Feature in a Real App
      Topics:
        - Designing an AI feature people would actually use
        - Adding an AI endpoint to your Flask app
        - Prompts as code: versioned, tested, improved
        - Streaming responses so the app feels alive
        - Caching to control cost and speed
        - Failure modes: timeouts, refusals and nonsense, handled
        - Simple evaluation: is the feature actually good?
        - AI ethics for builders, in plain terms
      Projects built:
        - Study-buddy feature added to your deployed web app: ask questions, get grounded answers
      Practice: Run 20 real queries through your feature and grade the answers yourself
    Git and GitHub for Real Projects
      Topics:
        - Why every developer uses version control
        - The core loop: status, add, commit, log
        - Branches: experimenting without fear
        - GitHub: pushing your work to the world
        - READMEs that make a project understandable in a minute
        - Undoing mistakes: the calm guide
        - .gitignore and never committing secrets
        - Your GitHub profile as a living portfolio
        - Where our full Git and GitHub course goes further
      Projects built:
        - Course portfolio pushed to GitHub: best projects, clean READMEs, pinned repositories
      Practice: Commit daily for two weeks; make the history readable
      Assessment: Month 9 check: a mixed assignment spanning machine learning, AI features
      and Git, with questions revisiting earlier phases

  Month 10: Quality, Capstone and Demo Day (Weeks 37-40)
    Testing and Code Quality
      Topics:
        - pytest: your first real test suite
        - Test-first thinking on one small feature
        - Fixtures and factories
        - API testing strategies
        - Black and flake8: code that reads like it was written on purpose
        - Reading tracebacks like a detective, not a victim
        - Debugging with breakpoints instead of print spam
      Projects built:
        - Test suite for your web capstone's core logic
        - Bug hunt: find and fix 5 planted bugs using the debugger
      Practice: Write 25 tests across your three best projects
    Final Capstone Build
      Topics:
        - Choosing the capstone: a game with an AI opponent, a full web app with an AI feature, or an
          automation product
        - Writing a one-page spec before any code
        - Milestones: working core by day 4, feature-complete by day 10
        - Instructor checkpoints and course corrections
        - Using everything: OOP, files, web, data, AI, Git, tests
        - Cutting scope without cutting quality
        - Preparing the demo: story, not feature list
      Projects built:
        - The final capstone, built over two weeks with version control, tests on the core logic, and a
          README worth reading
      Practice: Daily commits; a working build at the end of every session
    Demo Day and What Comes Next
      Topics:
        - Presenting the capstone: live demo plus the hardest bug story
        - Portfolio final pass: GitHub profile, project pages, screenshots
        - An honest skills inventory: strong, shaky, next
        - Where each path leads: AI and machine learning, data structures and algorithms, deeper web
          development
        - How school and board computer science connect to what you now know
        - Staying sharp: projects, communities and healthy practice habits
      Projects built:
        - Demo day presentation delivered to the batch and parents
      Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed
      from every phase. Passing earns the certificate; a student who falls short gets a
      focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Phase 1: quiz and adventure games, a journal app with save files, an expense tracker, classic
```

algorithms built by hand
- Phase 2: two finished Pygame games published on itch.io, desktop apps, a personal automation suite, a real-dataset analysis
- Phase 3: API clients, then a Flask web app with a database and user accounts, deployed to a public URL
- Phase 4: honest machine learning experiments, an AI-powered study tool, and a final capstone with tests and version control

Total projects: 30+ real projects, ending with a deployed web app and an AI-powered capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Core Python: syntax, data structures, files, errors and object-oriented programming, to genuine fluency
- Games: Pygame from the game loop to finished, published, shareable games
- Automation: scripts for files, spreadsheets, PDFs and the web, written politely and safely
- Data: pandas, matplotlib and honest exploratory analysis of real datasets
- Web development: Flask, SQL databases, user accounts and deployment to a live URL
- AI foundations: scikit-learn fundamentals plus real features built on modern AI APIs
- Developer habits: Git and GitHub, pytest, debugging and code review

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: Homework after every class, and a mixed-review assignment at the end of every month so earlier skills stay sharp
Practice: 3-4 hours of projects and challenges between classes
Review: Every submitted project reviewed with written feedback; recurring issues reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so nobody stays stuck on a bug for days
Progress updates: Regular progress notes to parents: what was built, what comes next, and where encouragement at home helps

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- AI and Machine Learning Masterclass: the natural next step once Python is fluent
- Data Structures and Algorithms: competitive programming and interview-style problem solving
- Deeper web development: from Flask into Django, React and production-grade apps
- School exams: a serious head start for CBSE, ICSE and AP-level computer science
- Real small projects: sites, tools and automations for family, school clubs and local teams

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my teen build in this Python course?
A: More than 30 real projects across ten months: arcade games in Pygame published on itch.io, desktop apps like study timers and flashcards, automation scripts that organize files and track prices, an honest data analysis of a real dataset, a Flask web app with user accounts deployed to a public URL, and a final AI-powered capstone. Every project is built by the student and reviewed with written feedback.

Q: How long does it take a teenager to learn Python properly?
A: Most students reach a genuinely advanced level in 10 months at 2 live classes a week plus 3 to 4 hours of practice. The syllabus deliberately keeps 2 months of margin in hand: students who need more repetition take up to 12 months, with no rushing and no penalty. Progress is visible early, the first playable text game appears within the first month.

Q: How is my teen assessed? Is the certificate just for attending?
A: No. Classwork runs inside every session, homework follows every class, and each month ends with an assignment that deliberately mixes new questions with earlier topics so knowledge compounds instead of fading. Every phase closes with a cumulative check, and the course ends with a final exam: a practical build plus written questions drawn from the whole syllabus. Passing earns the certificate; falling short earns a focused revision plan and a free retest. We optimize for real knowledge and real depth, not a stream of feel-good badges.

Q: Does my teen need maths or prior coding experience?
A: No prior coding at all, and ordinary school maths is plenty. Teens who already know some Python simply move faster through the early months.

Q: Is Python the right first language for a teenager?
A: For most teens, yes. Python reads close to English, produces a working program in the first class, and then scales all the way up: it is the language of choice in AI, data science and automation, and one of the two languages accepted in most school-level computer science exams. Nothing learned here has to be unlearned later.

Q: What software is needed, and is it free?
A: Everything is free: Python itself, the VS Code editor, and open-source libraries like Pygame, Flask, pandas and scikit-learn. Any reasonably modern computer with a stable internet connection can run the full course. Where an AI service needs an account, it is set up together with a parent.

Q: How is AI taught in this course?
A: In two honest layers. First, machine learning fundamentals with scikit-learn, where students train real models and learn to evaluate them without fooling themselves. Second, modern AI APIs, where they build features like a study-buddy that answers questions over their own notes. The goal is a teen who builds with AI and understands its limits, not one who just chats with it.

Q: Where does this lead after the course?
A: Three natural roads: our AI and Machine Learning Masterclass for teens who caught the AI spark, the Data Structures and Algorithms course for competitive programming, or deeper web development from Flask into Django and React. The portfolio built here, games, a deployed app, an AI capstone, carries directly into school projects and college applications.

Q: Is a trial class available?
A: Yes, a free demo class with no commitment; book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to observe the demo and ask the teacher questions afterwards. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: How are the classes taught?
A: Through live 1-hour online sessions with a real teacher, never pre-recorded videos. Classes are interactive and small; students write and run code during class, not just watch someone else do it. Group batches are capped at 20, with mini-batch and 1-on-1 options for closer attention.

Q: Does the course include a certificate?
A: Yes, and it is earned, not handed out: the certificate is awarded on passing the final exam, with a free retest after revision if the first attempt falls short. Alongside it stands the portfolio, 30+ projects, published games, a deployed web app and an AI capstone, which shows exactly what your teen can build, not just what they sat through.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/python-complete-masterclass-teens
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 119 OF 137: PYTHON MASTERCLASS: ZERO TO ADVANCED, LIVE ONLINE
==========

Course link: https://learn.modernagecoders.com/courses/python-programming-masterclass-zero-to-advanced-college
Subject area: Python Programming
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and professionals
Course length: 9-11 months (36-44 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Python masterclass for college students and professionals: core to advanced Python, testing, Git, APIs, automation and a guided taste of web, data and AI roads in 9-11 months.

In one line: One language, learned to professional depth: core Python, the working toolkit, and an honest guided taste of the three roads beyond.

ABOUT THE PROGRAMME

Python is the most useful single language a professional can learn, and the most commonly half-learned. This live online masterclass fixes the half: nine months from zero to genuinely advanced Python, the kind that reads stdlib source without fear, plus the professional toolkit, testing, Git, virtual environments, APIs, automation and concurrency, that turns language skill into working capability. The scope is deliberately honest. Web frameworks, data science and machine learning are each their own multi-month disciplines, and this course refuses to fake all three in survey weeks. Instead, the final month is a guided, hands-on taste of the three roads, a real Flask build, a real pandas analysis, a real AI-API script, so every graduate chooses their specialization from experience rather than marketing, and continues into our dedicated Django, data analysis or AI courses with the Python depth those courses quietly assume. The pace is real: nine months at two live classes a week plus four to six hours of practice, with two more months in hand when needed. Problem sets after every class, monthly mixed reviews that reach back deliberately, phase exams, and a final exam that decides the certificate, with focused revision and a free retest. Real Python, real depth, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- Depth where it compounds: core and advanced Python owned properly, not surveyed
- The professional toolkit is the syllabus: testing, Git, environments, APIs, automation, concurrency
- Honest specialization: the three roads (web, data, AI) tasted hands-on, then continued in dedicated courses instead of faked in survey weeks
- Pythonic style taught deliberately: comprehensions, iterators and the standard library as a first resort
- Certification-aware: the syllabus covers what PCEP and PCAP examine, for students who want the credential road
- Real assessment: weekly problem sets, monthly mixed reviews, and a final exam that decides the certificate, with focused revision and a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Core Python (Months 1-3): syntax to data structures, learned through daily building
Phase 2: Professional Python (Months 4-6): OOP, files and exceptions, the standard library, testing and a real capstone
Phase 3: The working toolkit (Months 7-8): Git and GitHub, APIs and scraping, automation, concurrency
Phase 4: The three roads and the finale (Month 9): Flask, pandas and AI APIs tasted honestly, then the final capstone and exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The course starts at the very first line
Equipment: A computer with a stable internet connection; Python and every tool used are free
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

College students: Students who want Python owned properly, not survived for a semester
Career switchers: Professionals entering software through its friendliest serious door
Half learners: The tutorial-experienced who can follow along but cannot build alone, this course closes that gap
Future specialists: Learners heading for web, data or AI who want the foundation those courses quietly assume
Certification seekers: Students eyeing PCEP and PCAP, whose syllabi this course covers

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: CORE PYTHON (MONTHS 1-3, WEEKS 1-12)

The language itself, learned properly: syntax, control flow, functions and the core data structures, practiced through daily building.

Month 1: First Python (Weeks 1-4)
Introduction to Programming & Python Setup
Topics:
- What is programming? How computers understand code
- Why Python? History, features, and popularity
- Python applications: web, data science, AI, automation, games
- Python 3 installation (Windows, Mac, Linux)
- Setting up development environment: VS Code, PyCharm
- Running Python: interactive shell vs script files
- IDLE, Jupyter Notebook, and Google Colab introduction
- Your first Python program: print('Hello World')

```
      - Python syntax rules and code structure
      - Comments: single-line (#) and multi-line (''')
      - Indentation and its importance in Python
      - Python PEP 8 style guide basics
    Projects built:
      - Hello World variations (personalized greetings)
      - Simple calculator using print statements
      - About Me program (display personal information)
      - ASCII art creator
    Practice: Daily: 30 min typing practice, write 5 simple print programs
  Variables, Data Types & Operators
    Topics:
      - Variables: naming rules and conventions
      - Assignment operator (=) and multiple assignments
      - Data types: int, float, string, boolean
      - Type checking with type() function
      - Type conversion: int(), float(), str(), bool()
      - Arithmetic operators: +, -, *, /, //, %, **
      - Comparison operators: ==, !=, >, <, >=, <=
      - Logical operators: and, or, not
      - Assignment operators: +=, -=, *=, /=
      - String basics: creation, concatenation, repetition
      - String indexing and slicing [start:stop:step]
      - F-strings and string formatting (format(), %)
    Projects built:
      - Temperature converter (Celsius/Fahrenheit)
      - Age calculator with days, hours, minutes
      - Simple interest calculator
      - BMI (Body Mass Index) calculator
      - Tip calculator with bill splitting
    Practice: Solve 25 problems on variables, data types, and operators
    Assessment: Month 1 check: a small program built live, plus a take-home mixing
    everything so far

Month 2: Control Flow and Functions (Weeks 5-8)
  Control Flow: Conditions & Loops
    Topics:
      - Boolean expressions and truthiness
      - If statement: basic conditional execution
      - If-else: two-way branching
      - If-elif-else: multi-way branching
      - Nested if statements
      - Ternary operator (conditional expression)
      - While loop: condition-based repetition
      - Infinite loops and break statement
      - Continue statement to skip iterations
      - For loop: iterating over sequences
      - Range function: range(start, stop, step)
      - Nested loops and loop patterns
    Projects built:
      - Number guessing game with attempts
      - Grade calculator with letter grades
      - Even/odd number checker
      - Multiplication table generator
      - Pattern printing (stars, pyramids, diamonds)
      - Simple password validator
      - FizzBuzz challenge
    Practice: Solve 30 conditional and loop problems
  Functions & Modular Programming
    Topics:
      - What are functions? DRY principle
      - Defining functions with 'def' keyword
      - Parameters and arguments (positional, keyword)
      - Return statement and return values
      - Multiple return values (tuples)
      - Default parameter values
      - Variable scope: local vs global
      - Global and nonlocal keywords
      - Docstrings for function documentation
      - *args for variable positional arguments
      - **kwargs for variable keyword arguments
      - Lambda functions (anonymous functions)
      - Built-in functions: len, sum, max, min, abs, round
    Projects built:
      - Function library for common calculations
      - Password strength checker function
      - Text analyzer (word count, character count)
      - Prime number checker and generator
      - Palindrome checker
```

```
        - Simple quiz game with scoring
        - Unit converter (length, weight, temperature)
      Practice: Create 20 different utility functions
      Assessment: Month 2 check: refactor a messy script into clean functions live, plus
      mixed review

  Month 3: The Data Structures (Weeks 9-12)
    Lists & List Operations
      Topics:
        - Lists: creating and accessing elements
        - List indexing (positive and negative)
        - List slicing: extracting sublists
        - List methods: append, insert, extend
        - Removing elements: remove, pop, clear
        - List methods: sort, reverse, copy
        - index() and count() methods
        - List comprehensions: compact list creation
        - Nested lists (2D lists, matrices)
        - Iterating lists with for loops
        - Enumerate function for index-value pairs
        - List vs tuple: mutability difference
      Projects built:
        - Todo list application (add, remove, display)
        - Shopping cart system
        - Student grade manager
        - Number list statistics (average, min, max)
        - List sorting and searching programs
        - Tic-tac-toe board representation
      Practice: Solve 25 list manipulation problems
    Tuples, Sets & Dictionaries
      Topics:
        - Tuples: immutable sequences
        - Tuple packing and unpacking
        - Tuple methods: count, index
        - When to use tuples vs lists
        - Sets: unordered unique collections
        - Set operations: union, intersection, difference
        - Set methods: add, remove, discard, clear
        - Dictionaries: key-value pairs
        - Accessing, adding, updating dictionary items
        - Dictionary methods: keys(), values(), items()
        - get() method with default values
        - Dictionary comprehensions
        - Nested dictionaries for complex data
        - String mastery: slicing, methods, f-strings and first regular expressions
      Projects built:
        - Contact book (name, phone, email)
        - Word frequency counter
        - Student database with grades
        - Inventory management system
        - English-Spanish dictionary translator
        - Vote counting system
      Practice: Build 10 projects using dictionaries and sets
      Assessment: Phase 1 exam: a data-wrangling task built live, plus a written mixed paper
      over months 1-3

------------------------------------------------------------
PHASE 2: PROFESSIONAL PYTHON (MONTHS 4-6, WEEKS 13-24)
------------------------------------------------------------
The jump from scripts to software: object-oriented design, files and exceptions, the standard
library as a first resort, testing, and a real capstone.

  Month 4: Object-Oriented Python (Weeks 13-16)
    Introduction to OOP Concepts
      Topics:
        - What is Object-Oriented Programming?
        - Classes and objects: blueprint and instances
        - Creating classes with 'class' keyword
        - The __init__ constructor method
        - Instance variables (attributes)
        - Instance methods
        - Self parameter explained
        - Creating and using objects
        - Class vs instance attributes
        - Class methods with @classmethod decorator
        - Static methods with @staticmethod decorator
        - __str__ and __repr__ magic methods
      Projects built:
        - Bank account class with deposit/withdraw
```

```
      - Student class with grades management
      - Book class for library system
      - Car class with properties and methods
      - Rectangle/Circle classes with area calculation
    Practice: Create 15 different classes modeling real-world objects
  OOP Principles: Inheritance & Polymorphism
    Topics:
      - Inheritance: parent and child classes
      - Method overriding in child classes
      - Super() function to call parent methods
      - Multiple inheritance
      - Method Resolution Order (MRO)
      - Polymorphism: same interface, different implementation
      - Duck typing in Python
      - Abstract base classes (ABC module)
      - Encapsulation: public, protected, private
      - Name mangling with double underscore
      - Property decorators: @property, @setter
      - Composition vs inheritance
    Projects built:
      - Animal hierarchy (Animal -> Dog, Cat, Bird)
      - Employee management system (Employee -> Manager, Developer)
      - Shape calculator with inheritance
      - Vehicle rental system
      - Game character classes with different abilities
    Practice: Build 10 class hierarchies with inheritance
    Assessment: Month 4 check: design a small class hierarchy live and defend it, plus
    mixed review

Month 5: Files, Exceptions and the Standard Library (Weeks 17-20)
  File Handling & Exception Handling
    Topics:
      - Opening files: open() function and modes (r, w, a, r+)
      - Reading files: read(), readline(), readlines()
      - Writing to files: write(), writelines()
      - With statement for automatic file closing
      - File paths: absolute vs relative
      - Working with CSV files
      - JSON file handling: json.dump(), json.load()
      - Exception handling: try-except blocks
      - Catching specific exceptions
      - Multiple except blocks
      - Else and finally clauses
      - Raising exceptions with 'raise'
      - Creating custom exceptions
    Projects built:
      - Note-taking app with file persistence
      - Contact manager with CSV storage
      - Configuration file reader/writer
      - Log file analyzer
      - Student records system with JSON
      - Error-safe calculator
    Practice: Build 8 file-based applications
  Modules, Packages & Standard Library
    Topics:
      - Importing modules: import, from...import
      - Creating custom modules
      - Module search path and PYTHONPATH
      - Packages and __init__.py
      - Creating package structures
      - Math module: mathematical functions
      - Random module: random numbers and choices
      - Datetime module: working with dates and times
      - OS module: operating system interface
      - Sys module: system-specific parameters
      - Collections module: deque, Counter, defaultdict
      - Itertools module: efficient iterators
      - Comprehensions and generator expressions: Pythonic style as a habit
    Projects built:
      - Dice rolling simulator with statistics
      - Birthday reminder application
      - File organizer using os module
      - Random password generator
      - Custom utility package creation
      - Date calculator (age, days between dates)
    Practice: Explore 15 standard library modules
    Assessment: Month 5 check: solve three tasks stdlib-first, plus mixed review reaching
    back to phase 1
```

```
  Month 6: Testing and the Phase Capstone (Weeks 21-24)
    Testing & Debugging
      Topics:
        - Why testing matters
        - Manual testing vs automated testing
        - Unit testing with unittest module
        - Test cases and assertions
        - SetUp and tearDown methods
        - Pytest framework introduction
        - Writing pytest test functions
        - Test fixtures in pytest
        - Parameterized testing
        - Code coverage with coverage.py
        - Debugging techniques in Python
        - Using Python debugger (pdb)
        - Logging with logging module
      Projects built:
        - Test suite for calculator
        - Testing OOP classes
        - Testing file operations
        - TDD (Test-Driven Development) mini project
        - Debugging exercise solutions
      Practice: Write tests for all previous projects
    Phase 2 Capstone
      Topics:
        - Object-oriented design
        - Algorithm implementation
        - File-based data persistence
        - Error handling throughout
        - Unit testing
        - Code documentation
      Projects built:
        - PHASE 2 CAPSTONE: Library Management System (OOP-based)
        - Features: Books, members, lending, returns, fines, search, file storage, testing
        - Alternative: School Management System
        - Alternative: Banking System with multiple account types
        - Alternative: E-commerce Inventory System
      Assessment: Phase 2 exam: a tested, documented capstone defended, plus a cumulative
      written paper

----------------------------------------------------------------
PHASE 3: THE WORKING TOOLKIT (MONTHS 7-8, WEEKS 25-32)
----------------------------------------------------------------
What turns language skill into working capability: Git and GitHub, real APIs and polite
scraping, automation that survives reality, and concurrency without folklore.

  Month 7: Git, GitHub and the Living Web (Weeks 25-28)
    Version Control with Git & GitHub
      Topics:
        - What is version control?
        - Git installation and setup
        - Git basic commands: init, add, commit
        - Git workflow: working directory, staging, repository
        - Checking status and history: status, log
        - Branches: creating and switching
        - Merging branches
        - Handling merge conflicts
        - Remote repositories: GitHub
        - Push, pull, clone operations
        - GitHub collaboration basics
        - .gitignore file for Python projects
      Projects built:
        - Initialize Git for existing projects
        - Create GitHub repository
        - Collaborate on shared repository
        - Open source contribution preparation
      Practice: Use Git for all future projects
    APIs and Web Scraping, Done Politely
      Topics:
        - Advanced BeautifulSoup techniques
        - Navigating HTML tree
        - CSS selectors for scraping
        - Scrapy framework introduction
        - Creating Scrapy spiders
        - Item pipelines for data processing
        - Scrapy selectors and XPath
        - Handling pagination
        - Selenium for dynamic websites
        - WebDriver automation
```

```
        - Headless browsing
        - Ethics, robots.txt, and rate limiting
      Projects built:
        - News scraper with BeautifulSoup
        - E-commerce price tracker
        - Job listings aggregator with Scrapy
        - Social media scraper (ethical)
        - Automated form filler with Selenium
      Practice: Build 8 scraping and automation projects
      Assessment: Month 7 check: build an API client live with error handling, plus mixed
      review

  Month 8: Automation and Concurrency (Weeks 29-32)
    Automation and Scripting That Survives Reality
      Topics:
        - File and folder automation
        - Batch file processing
        - PDF manipulation with PyPDF2
        - Excel automation with openpyxl
        - CSV data processing
        - Email automation with smtplib
        - Scheduling tasks with schedule library
        - System administration scripts
        - Desktop automation with PyAutoGUI
        - Clipboard automation
        - Screenshot and image manipulation
        - Building CLI tools with Click/argparse
        - Virtual environments and dependency hygiene as reflexes
      Projects built:
        - File organizer automation
        - Excel report generator
        - Email sender with attachments
        - PDF merger and splitter
        - Automated backup system
        - Desktop notification system
        - System monitoring script
      Practice: Create 10 automation scripts for daily tasks
    Concurrency Without Folklore
      Topics:
        - Concurrency vs parallelism
        - Threading basics with threading module
        - Creating and starting threads
        - Thread synchronization
        - Multiprocessing module
        - Process creation and management
        - Pool for parallel execution
        - AsyncIO for asynchronous programming
        - Async/await syntax
        - Concurrent.futures module
        - ThreadPoolExecutor and ProcessPoolExecutor
        - When to use threading vs multiprocessing
      Projects built:
        - Multi-threaded file downloader
        - Parallel data processing
        - Async web scraper
        - CPU-intensive task with multiprocessing
      Practice: Optimize slow programs with concurrency
      Assessment: Phase 3 exam: an automation built and scheduled live, plus a cumulative
      written paper

------------------------------------------------------------
PHASE 4: THE THREE ROADS AND THE FINALE (MONTH 9, WEEKS 33-36)
------------------------------------------------------------
Web, data and AI, each tasted hands-on in real builds, then the final capstone and the exam
that decides the certificate.

  Month 9: Choose Your Road, Then Prove the Journey (Weeks 33-36)
    The Three Roads, Tasted Honestly
      Topics:
        - Road one, web: a real Flask app built in two sessions, routes to deployment
        - Road two, data: a real pandas analysis, load, clean, group, chart, conclude
        - Road three, AI: a real script on a modern AI API, structured output included
        - What each road demands next, told straight
        - Where our Django, data analysis and AI courses continue each road
        - Choosing from experience: which build did you not want to stop?
      Projects built:
        - Three small real builds: a deployed Flask app, a pandas analysis, an AI-API tool
      Practice: Extend the road that pulled hardest by one more feature
    Final Capstone Build
```

Topics:
- One focused week: a complete Python product using the whole course
- Tested core logic, clean structure, honest README
- Instructor checkpoint and scope cuts
- Version control history that tells the story

Projects built:
- The final capstone: a tool, automation or application built to be shown

Practice: Daily commits; a working build at the end of every session

Demo Day and the Final Exam

Topics:
- Presenting the capstone: live demo plus the hardest bug story
- A full-course spiral review before the exam
- PCEP and PCAP: how this syllabus maps to the certifications, for those who want them
- The road chosen, and the dedicated course that continues it

Projects built:
- Demo day presentation delivered to the batch

Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: daily builds from calculators to data-wrangling tools
- Phase 2: an OOP system, stdlib-first utilities, and a tested phase capstone
- Phase 3: a Git-managed portfolio, API clients, polite scrapers and scheduled automations
- Phase 4: a Flask app, a pandas analysis, an AI-API tool, and the defended final capstone

Total projects: 25+ real builds, crowned by a tested, defended capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Core Python to genuine depth: syntax, data structures, functions, OOP
- Pythonic style: comprehensions, iterators and the standard library as a first resort
- The professional toolkit: pytest, Git and GitHub, virtual environments
- The living web: APIs consumed properly, scraping done politely
- Automation and concurrency: scripts that survive reality
- Road literacy: Flask, pandas and AI APIs each experienced hands-on

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, for the traceback that makes no sense at midnight
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Django and Flask web development: our dedicated web course continues road one
- Data analysis and data science: our data courses continue road two
- AI and machine learning: our AI/ML and generative AI courses continue road three
- PCEP and PCAP certification: the credential road, entered with real depth
- Automation everywhere: the skill that pays off in any job from day one

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 2-4 LPA (Junior Python Developer / Intern)
Growing roles: Rs 4-7 LPA (Python Developer / Automation Engineer); Rs 7-12 LPA (Full Stack Python / Data Analyst)
Experienced roles: Rs 10-20 LPA (Senior Python / Data Scientist / ML Engineer)
Other roles: Rs 12-30 LPA in India, $80k-180k USD internationally; Rs 15-40 LPA in India, $100k-200k+ USD internationally

  Freelance: Rs 500-3,000/hour based on specialization
  International: $60k-150k USD (USA) based on role and experience

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What does this Python masterclass cover?
  A: Python to genuine professional depth across nine months: the core language and data structures, object-oriented design, files and exceptions, the standard library used as a first resort, testing with pytest, Git and GitHub, APIs and polite scraping, automation and concurrency, and a final month that tastes the three roads, a real Flask build, a real pandas analysis, a real AI-API tool, before the capstone and final exam.

  Q: How long does it take to learn Python properly?
  A: To professional depth: nine months at 2 live classes a week plus 4 to 6 hours of practice, and the course keeps two more months in hand for learners who need them. Two-week Python courses teach syntax; syntax is perhaps a tenth of the skill employers actually test.

  Q: How am I assessed? Is the certificate just for attending?
  A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with an exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

  Q: Does this course cover web development, data science and AI?
  A: It covers what makes them possible, deep Python, and then tastes each road honestly in the final month: a deployed Flask app, a pandas analysis, an AI-API tool. Full web, data and AI tracks are their own multi-month disciplines, and we teach them properly in dedicated courses rather than faking them here in survey weeks. Graduates choose their road from experience and continue with the depth those courses assume.

  Q: Will this prepare me for PCEP or PCAP certification?
  A: Yes: the syllabus covers what both exams examine, and the final weeks map it explicitly for students who want the credential. We also run a dedicated PCEP/PCAP preparation course for those who want exam-format drilling specifically.

  Q: Is Python still the right language to learn?
  A: More than ever: it is the language of AI and data, the default automation tool of every industry, and the most commonly listed skill in technical hiring. The AI era raised its value, the people directing AI tools well are the ones who understand the code those tools produce, and Python is where that understanding is cheapest to build.

  Q: Can I try a class before enrolling?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. You will write and run real Python inside the demo. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos. Python habits, good and bad, form in the first months; live correction is what makes them good.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/python-programming-masterclass-zero-to-advanced-college
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 120 OF 137: PYTHON WEB DEVELOPMENT WITH DJANGO AND FLASK COURSE
==========

Course link: https://learn.modernagecoders.com/courses/python-web-development-django-flask-course
Subject area: Web Development
Designed for: College students
Who can join: Intermediate, college students and working professionals with Python basics
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Python web development course with Django and Flask: live online classes on routing, templates, databases, auth, REST APIs and deploying a real project.

In one line: You know Python. Six months from now, your code answers requests on a real URL.

ABOUT THE PROGRAMME

Knowing Python and being able to ship a web application are two different skills, and the second one is what this course builds. It starts where confusion usually begins: what actually happens when a browser asks a server for a page. Month 1 answers that with HTTP fundamentals and Flask, a framework small enough to see through: routes, templates, and forms with nothing hidden. Month 2 adds the database layer, SQL first and then the SQLAlchemy ORM, plus sessions and login, so by the halfway mark students have built a complete Flask application from scratch. Months 3 and 4 move to Django, the framework Indian and global companies run in production: models, migrations, the admin, authentication, relationships, and class-based views, ending in a database-backed project with real users. Month 5 covers what most jobs actually ask for, REST APIs with Django REST Framework: serializers, token authentication, permissions, and a documented API tested end to end. Month 6 is deployment and the capstone: environment configuration, Postgres, Gunicorn, static files, a live host, and a project of the student's own design shipped to a public URL and defended. Classes are live and small, everything is built by typing, and code review is part of the course, not a bonus.

WHAT MAKES THIS COURSE DIFFERENT

- Flask before Django, on purpose: a small framework first means Django's conveniences read as solutions, not magic
- SQL is taught before the ORM, so students understand what save() and filter() actually send to the database
- The middle of the course is a complete Flask app and a complete Django app, not fragments that never meet
- REST APIs get a full month with Django REST Framework, because that is what backend job descriptions actually list
- Deployment is a taught skill, not a footnote: every student ships the capstone to a real public URL
- Code review from a working instructor on every project, with the same standards a team lead would apply

THE LEARNING PATH, STEP BY STEP

Phase 1: How the web works and Flask from scratch: HTTP, routes, templates, forms, SQL, the ORM, sessions, and a complete Flask application
Phase 2: Django properly: models, migrations, admin, authentication, relationships, and class-based views, ending in a database-backed multi-user project
Phase 3: REST APIs with Django REST Framework, then production: deployment, Postgres, environment config, and a capstone shipped to a live URL

WHAT A STUDENT NEEDS BEFORE STARTING

Python basics: Required: variables, loops, functions, lists, and dictionaries. Our Python courses cover this if you are not there yet
Web experience: None needed. HTTP, HTML, and databases are all taught from zero inside the course
Equipment: Any Windows, Mac, or Linux computer that runs Python and VS Code comfortably, plus a stable internet connection
Time: Around 3-4 hours a week outside class. The projects are real, and they are where the learning compounds

WHO THIS COURSE IS FOR

College students: Students with Python from coursework who want a portfolio that says more than marks do
Working professionals: Professionals in testing, support, data, or other fields moving toward backend development roles
Python learners stuck at scripts: Self-taught programmers who can write scripts but have never shipped anything with a URL
Startup builders: Founders and hobbyists who want to build and deploy their own product idea end to end
Interview preparers: Candidates targeting backend roles where Django, DRF, and deployment questions come up by name

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: HOW THE WEB WORKS, THEN FLASK (MONTHS 1-2, WEEKS 1-8)

HTTP demystified with real requests, then Flask built up from a single route to a complete
application with a database, forms, and login.

  Month 1: HTTP and Your First Flask Apps (Weeks 1-4)
    What Happens When You Load a Page
      Topics:
        - Client and server: who asks, who answers
        - HTTP requests dissected: method, path, headers, body
        - Status codes worth knowing: 200, 301, 404, 500
        - Watching real traffic in the browser network tab
        - Making requests by hand with curl and Python's requests library
        - GET vs POST, and why the difference matters before any framework
      Projects built:
        - Request detective: a written trace of one real page load, every request annotated, plus a
          script that fetches and inspects three sites
      Practice: Use curl to make ten requests against a public API, logging method, status,
      and one header for each
    First Flask Application
      Topics:
        - Virtual environments and pip: project hygiene from day one
        - A Flask app in seven lines, and what each line does
        - Routes and view functions
        - Dynamic routes: reading values out of the URL
        - The development server and debug mode
        - Project structure that will survive growth
      Projects built:
        - Personal API: a Flask app with five routes returning text and JSON, including a dynamic
          greeting route
      Practice: Build a dice-roll and coin-flip service with routes that take the number of
      rolls from the URL
    Templates
      Topics:
        - Why string concatenation dies fast: enter Jinja2
        - render_template and passing data to pages
        - Template logic: loops and conditionals in Jinja2
        - Template inheritance: one base layout, many pages
        - Static files: CSS and images served properly
        - Just enough HTML and CSS, taught as the projects need it
      Projects built:
        - Movie catalog site: a browsable list rendered from Python data with a shared base layout and
          detail pages
      Practice: Add a search-results page and an empty-state design to the movie catalog
      using template conditionals
    Forms and User Input
      Topics:
        - HTML forms: action, method, and named fields
        - Reading form data in Flask with request.form
        - The POST-redirect-GET pattern and why refreshing resubmits without it
        - Validating input on the server, always
        - Flash messages: telling users what just happened
        - Never trusting the client: a first look at input handling as a security matter
      Projects built:
        - Feedback board: a form that accepts, validates, and displays entries, with flash messages for
          errors and success
      Practice: Add server-side validation rules to the feedback board and prove each one
      works with a bad submission
      Assessment: Month 1 build check: a small multi-page Flask app with one form, built live
      from a spec

  Month 2: Databases and a Complete Flask App (Weeks 5-8)
    SQL Before the ORM
      Topics:
        - Why files stop working: the case for databases
        - SQLite: a real database with zero setup
        - CREATE TABLE, data types, and primary keys
        - INSERT, SELECT, UPDATE, DELETE by hand
        - WHERE, ORDER BY, and LIMIT
        - Running queries from Python with sqlite3
      Projects built:
        - Query workbook: a seeded database of books and members, interrogated with fifteen handwritten
          SQL queries
      Practice: Write ten more queries against the workbook database, including two that join
      tables
    The ORM: SQLAlchemy
      Topics:
        - What an ORM is and what it is hiding
        - Defining models as Python classes

        - Creating, querying, updating, and deleting through the ORM
        - Echoing the generated SQL: seeing what the ORM sends
        - Filtering and ordering with query methods
        - When to drop to raw SQL, honestly
      Projects built:
        - Movie catalog, database edition: the month 1 site rebuilt on SQLAlchemy models instead of Python lists
      Practice: For five ORM queries, predict the SQL before enabling echo, and score your predictions
    CRUD End to End
      Topics:
        - The CRUD pattern: create, read, update, delete as routes
        - Forms that create and edit database records
        - Deleting safely: confirmation and POST-only deletes
        - URL design for resources: lists, details, edits
        - Handling the missing record: 404 done properly
        - Keeping view functions thin
      Projects built:
        - Notes application: full CRUD with create, edit, delete, and detail pages backed by the database
      Practice: Add pinning and archiving to the notes app, each touching the model, routes, and templates
    Sessions and Login
      Topics:
        - Cookies and sessions: how a server remembers you
        - Password hashing with werkzeug: why plain text is malpractice
        - Register, login, and logout routes
        - Protecting routes: login required, by hand once
        - Per-user data: your notes are not my notes
        - The complete Flask app: what we built and what Django will automate
      Projects built:
        - Notes app with accounts: registration, hashed passwords, login, logout, and notes scoped to their owner
      Practice: Write a short attack review of your own app: three things a hostile user might try, and what stops each
      Assessment: Phase 1 project review: the complete Flask notes app submitted with a README, code-reviewed one on one

---
PHASE 2: DJANGO PROPERLY (MONTHS 3-4, WEEKS 9-16)

The production framework, learned with Flask as the reference point: models, migrations, the admin, real authentication, relationships, and class-based views, ending in a multi-user project.

  Month 3: Django Foundations (Weeks 9-12)
    Projects, Apps, and URLs
      Topics:
        - Why Django after Flask: batteries included, and which batteries
        - startproject and startapp: what every generated file is for
        - URLconf: routing requests to views
        - Function-based views and HttpResponse
        - Django templates: familiar ground after Jinja2
        - settings.py toured honestly: the settings that matter now
      Projects built:
        - Django hello-world done right: a two-app project with routed pages, shared base template, and static files
      Practice: Port your Flask movie catalog's pages to Django views and templates, noting what moved where
    Models and Migrations
      Topics:
        - Defining models with Django fields
        - Migrations: what makemigrations and migrate actually do
        - The ORM: create, filter, get, exclude, order_by
        - The Django shell as your laboratory
        - Field options: defaults, null, blank, and choices
        - Changing a model after data exists: the migration workflow
      Projects built:
        - Job board, part one: JobPosting and Company models, migrated, seeded, and queried from the shell
      Practice: Fifteen ORM exercises in the Django shell against the job board data, saved as a script
    The Admin and Django Templates
      Topics:
        - The Django admin: a working back office for free
        - Registering models and customizing list displays
        - Search, filters, and inline editing in the admin
        - Wiring list and detail pages to real data
        - get_object_or_404 and clean URL patterns with slugs

- Why the admin exists and where its limits are
    Projects built:
      - Job board, part two: public listing and detail pages, plus a configured admin where postings are managed
    Practice: Customize the admin with two list filters and a search field, and add a published/draft toggle that the public site respects
  Django Forms
    Topics:
      - The Form class: declaration instead of repetition
      - ModelForm: forms generated from models
      - Validation: built-in validators and clean methods
      - Rendering forms in templates and displaying errors well
      - CSRF protection: what that token was always for
      - File uploads, introduced carefully
    Projects built:
      - Job board, part three: a post-a-job form with validation, plus an application form with resume upload
    Practice: Write three custom validation rules, including one clean method that checks two fields against each other
    Assessment: Month 3 review: job board demo plus live ORM queries answered in the Django shell

Month 4: Auth, Relationships, and Structure (Weeks 13-16)
  Authentication
    Topics:
      - django.contrib.auth: the battery you were promised
      - Login, logout, and signup flows with Django's views and forms
      - login_required and testing it properly
      - The User model and when to extend it with a profile
      - Permissions and groups, introduced with real cases
      - Password reset flow, understood even before email is wired
    Projects built:
      - Job board accounts: employer and candidate signup, login, and pages that respond to who is viewing them
    Practice: Protect three routes with different rules: any logged-in user, employers only, and owner only
  Relationships
    Topics:
      - ForeignKey: companies have many postings
      - Querying across relationships in both directions
      - ManyToManyField: tags and saved jobs
      - related_name and readable reverse queries
      - The N+1 query problem: found with the debug toolbar, fixed with select_related
      - Designing a schema on paper before typing it
    Projects built:
      - Job board relationships: companies, postings, tags, and saved-jobs wired together with efficient queries
    Practice: Hunt N+1 queries on two pages using the debug toolbar, and record the query count before and after fixing
  Class-Based Views and Polish
    Topics:
      - Class-based views: ListView and DetailView without mystery
      - CreateView, UpdateView, DeleteView and when generic views earn their keep
      - Pagination that holds up with real data
      - The messages framework for user feedback
      - Query parameters for search and filtering
      - Function-based vs class-based: an honest decision guide
    Projects built:
      - Job board, refactored: listing pages moved to generic views with pagination, search, and tag filtering
    Practice: Convert two of your remaining function-based views to class-based views, and write three lines on which reads better
  Project Week: Ship the Django App
    Topics:
      - Feature freeze and a finishing checklist
      - Seeding believable demo data with a management command
      - Error pages: custom 404 and 500
      - A structured self-review before the instructor's review
      - Writing a README with setup steps that actually work
      - Django's request lifecycle, reviewed now that every piece is familiar
    Projects built:
      - Complete job board: multi-user Django application with auth, relationships, search, pagination, and admin, delivered with a README
    Practice: Follow a classmate's README from git clone to running app, and file issues for every step that failed
    Assessment: Phase 2 code review: full walkthrough of the job board with questions on models, queries, and auth decisions

---

PHASE 3: REST APIS AND PRODUCTION (MONTHS 5-6, WEEKS 17-24)

The skills backend jobs list by name: REST API design with Django REST Framework, token auth
and permissions, then real deployment and a capstone shipped to a public URL.

  Month 5: REST APIs with Django REST Framework (Weeks 17-20)
    API Thinking
      Topics:
        - Why APIs exist: mobile apps, frontends, and other servers as clients
        - REST conventions: resources, methods, and status codes as a contract
        - Designing endpoints on paper: the URL table for the job board
        - JSON in and JSON out: serialization as the core problem
        - Installing DRF and the browsable API
        - Your first APIView returning real data
      Projects built:
        - API design document plus a first read-only endpoint serving job postings as JSON
      Practice: Design the full endpoint table for a library system: paths, methods, status
      codes, and example payloads
    Serializers and CRUD Endpoints
      Topics:
        - ModelSerializer: validation and conversion in one place
        - List and detail endpoints for every resource
        - Create, update, and delete through the API
        - Nested and related data: choices and trade-offs
        - Serializer validation mirroring the form validation you already know
        - Testing endpoints with the browsable API and httpie
      Projects built:
        - Job board API: full CRUD endpoints for postings and companies with proper status codes
          throughout
      Practice: Write an httpie session script that exercises every endpoint, including three
      requests designed to fail correctly
    API Authentication and Permissions
      Topics:
        - Session auth vs token auth, and why APIs need the latter
        - Token authentication set up and tested
        - Permission classes: read for all, write for owners
        - Custom permissions: employers edit only their own postings
        - Throttling: rate limits as basic self-defense
        - Consistent, safe error responses
      Projects built:
        - Secured API: token auth plus owner-scoped write permissions, proven with a scripted
          attack-and-verify session
      Practice: Write six permission test cases: two allowed, four denied, each asserting the
      exact status code
    Consuming and Documenting the API
      Topics:
        - A minimal JavaScript page that fetches from your API
        - CORS: why the browser said no, and django-cors-headers
        - A Python client consuming the same API: one backend, many clients
        - Filtering, searching, and pagination in DRF
        - Auto-generated API documentation with drf-spectacular
        - Versioning and other habits of APIs that live long
      Projects built:
        - Consumed API: a small JS-driven jobs page and a Python CLI client, both running against your
          documented API
      Practice: Hand your API docs to a classmate and have them build one working request
      without asking you anything
      Assessment: API month review: live demo of the secured, documented API with a Q&A on
      auth and permission choices

  Month 6: Deployment and the Capstone (Weeks 21-24)
    Deployment, For Real
      Topics:
        - Development vs production: DEBUG, secrets, and environment variables
        - PostgreSQL: moving off SQLite and why production demands it
        - Gunicorn: what a WSGI server does that runserver does not
        - Static files in production with WhiteNoise
        - Deploying the job board to a cloud host, live in class
        - Reading server logs when something breaks, because something will
      Projects built:
        - Deployed job board: the phase 2 project live on a public URL with Postgres, Gunicorn, and
          env-based settings
      Practice: Break and fix your deployment once on purpose: bad env var in, error read
      from logs, corrected, redeployed
    Capstone: Scope and Foundations
      Topics:
        - Choosing a capstone: your own idea, or a track like bookings, forums, or expense tracking
        - Writing the spec: models, pages, endpoints, and what done means
        - Schema design reviewed before a line of code

```
          - Repository setup and the first deploy in week one, not the last
          - Must-have vs nice-to-have, decided while calm
          - Instructor sign-off on scope
        Projects built:
          - Approved capstone spec plus a skeleton project with models migrated and a hello page already
            deployed
        Practice: Build and seed your capstone's data layer, and query it from the shell to
        prove the schema holds
      Capstone: Build and Review
        Topics:
          - Building vertically: one complete feature at a time
          - A first pass at automated tests: models and one endpoint
          - Mid-sprint code review with written findings
          - Continuous deployment: pushing progress to the live URL
          - Handling feedback without derailing the plan
          - Cutting scope honestly when the deadline is fixed
        Projects built:
          - Feature-complete capstone: core features working on the live URL, tests passing, review
            findings addressed
        Practice: Write tests for your two most important behaviors and wire them into a
        pre-push habit
      Ship and Defend
        Topics:
          - The production checklist: DEBUG off, secrets out of git, error pages, backups considered
          - README and API docs finalized
          - Presenting a backend project: the demo, the data model, the hardest bug
          - Defending technical decisions under friendly fire
          - Your GitHub as evidence: six months of reviewed commits
          - What comes next: DSA for interviews, deeper Python, or frontend to pair with your backend
        Projects built:
          - Delivered capstone: a live, documented, tested web application at a public URL, presented and
            defended
        Practice: Send your capstone URL to two people outside the class and fix the first
        confusing thing each one reports
        Assessment: Capstone defense: live demo, data-model walkthrough, and Q&A, plus
        course-completion certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Request detective: a full annotated trace of real HTTP traffic
  - Movie catalog site with Jinja2 templates and template inheritance
  - Feedback board with server-side validation and flash messages
  - SQL query workbook against a seeded relational database
  - Notes application with full CRUD, sessions, and hashed-password login
  - Job board: a complete multi-user Django application with auth, relationships, search, and
    admin
  - Job board REST API: full CRUD with token auth, permissions, and generated documentation
  - JS jobs page and Python CLI client consuming the same API
  - Deployed job board on a public URL with Postgres and Gunicorn
  - Capstone: a self-designed, tested, documented web application shipped to production

  Total projects: 12+ real builds across two frameworks, ending with a capstone live on a public URL

SKILLS THE STUDENT WILL HAVE AT THE END

  - HTTP fluency: requests, responses, status codes, and debugging traffic like a backend
    developer
  - Flask from scratch: routing, Jinja2 templates, forms, sessions, and password hashing
  - SQL and ORMs: handwritten queries first, then SQLAlchemy and the Django ORM with eyes open
  - Django in production shape: models, migrations, admin, auth, relationships, and class-based
    views
  - REST APIs with DRF: serializers, token auth, permissions, throttling, and generated docs
  - Deployment: environment config, Postgres, Gunicorn, static files, and reading production logs

HOW A TYPICAL WEEK IS SPLIT

  Live classes: 2 live one-hour classes per week, building alongside the instructor with code
    review baked in
  Practice: 3-4 hours of project work and exercises between classes
  Review: Project submissions reviewed with written findings; recurring issues become the next
    class's opening
  Homework: A task or problem set after every class, and a monthly mixed-review assignment that
    deliberately folds in earlier topics

SUPPORT INCLUDED

  Doubt support: WhatsApp doubt support between classes, because deployment errors do not wait
    for Tuesday
  Progress updates: Regular progress notes covering what was built, what was reviewed, and what
```

needs another pass
Career guidance: Honest guidance on portfolios, resumes for backend roles, and what to learn next, grounded in what each student actually built

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Backend or Python developer roles, applied for with a deployed project and a reviewed GitHub history
- Full-stack development: pair this backend depth with our JavaScript and React courses
- Freelance web projects: the build-deploy-maintain cycle this course teaches is the whole freelance job
- Our data structures and algorithms course for product-company interview preparation
- Deeper Python specializations: data engineering, automation, or our AI and machine learning tracks

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 3-6 LPA (Junior Python Developer / Backend Developer)
Growing roles: Rs 6-12 LPA (Python/Django Developer with 2-4 years of experience)
Experienced roles: Rs 12-25 LPA (Senior Backend Engineer / Lead Developer)
Freelance: Rs 800-2,500/hour for Django and API work, depending on specialization and track record
International: $70k-140k USD (USA) for backend Python roles, based on role and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: How much Python do I need before joining?
A: Comfortable basics: variables, loops, functions, lists, and dictionaries, roughly what a first Python course or one college semester covers. If you are unsure whether you qualify, the free demo class includes a short, low-pressure skills check and an honest recommendation, which is sometimes to take our Python course first. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Why does the course teach Flask and Django instead of just Django?
A: Because Django learned first tends to be memorized, not understood. Flask is small enough that you see every moving part: what a route is, what a session does, what the ORM is actually saving you from. After two months of that, Django's structure reads as a set of solutions to problems you personally hit. Employers get a candidate who can work in either framework and explain the difference, which is a genuinely common interview question.

Q: Is Django still in demand, or has Node.js taken over?
A: Both stacks hire steadily, and pretending otherwise would be dishonest either way. Django remains a standard choice at Indian product companies, agencies, and startups, especially anywhere Python already runs: data-heavy products, fintech, and internal platforms. Instagram and Spotify famously run Python at scale. If your longer-term interest is JavaScript everywhere, our Node.js track fits better; if you know Python and want employable backend skills now, this stack is the shorter path.

Q: Will I learn front-end development too?
A: Enough to ship, not more: the HTML, CSS, and templates your pages need, plus one small JavaScript client in month 5 so you experience your API from the consumer's side. This is deliberately a backend course. Students who want the full stack typically take our React course afterwards, and the capstone API you build here becomes the backend for that work.

Q: What will I actually be able to show a recruiter after six months?
A: A capstone application live at a public URL, a complete Django job board, a documented REST API with token authentication, and a GitHub profile showing six months of steady, code-reviewed commits. In interviews you can walk through your own data model, explain an N+1 query you found and fixed, and describe your deployment step by step, which lands very differently from listing frameworks on a resume.

Q: I work full time. Can I manage this course?
A: The course is built for exactly that constraint: two live one-hour classes a week scheduled in evening-friendly slots, plus 3 to 4 hours of project work you place wherever your week allows. The pace is steady rather than punishing, and WhatsApp doubt support means a blocked Tuesday problem does not have to wait for Thursday's class. Professionals who fall

behind a week can use the mini-batch or 1-on-1 formats to catch up.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final exam covering the whole course. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Can I try a class before enrolling?
A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. The demo includes the Python readiness check and a clear walkthrough of how the course is taught and what you will build.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/python-web-development-django-flask-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 121 OF 137: QUEEN CODERS: ADVANCED ENGINEERING FOR WOMEN DEVELOPERS
==========

Course link: https://learn.modernagecoders.com/courses/queen-coders-advanced-tech-leadership
Subject area: Women in Tech
Designed for: Girls only batches
Who can join: Women developers with working experience (roughly 2+ years or a strong portfolio)
Course length: 9-11 months (36-44 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Advanced course for women developers: software architecture, cloud, security, AI engineering and technical leadership in 9-11 months, live online.

In one line: The step from good developer to trusted engineer: architecture, AI engineering and technical leadership, built deliberately in nine months.

ABOUT THE PROGRAMME

There is a career moment where writing good code stops being enough: teams need architecture judgment, security thinking, AI fluency and the leadership skills that turn seniority into influence. Queen Coders builds that step deliberately, for women developers, in a room of peers on the same climb. The arc runs in three movements. Advanced engineering: software design principles at depth, cloud architecture, data architecture, performance and security, the judgment layer above syntax. AI engineering: machine learning engineering, modern AI applications, LLMs and agent development, the layer every senior role now assumes. Technical leadership: the honest skills of staff-level work, technical decision records, code review leadership, mentoring, engineering management versus the senior IC road, taught practically rather than aspirationally. Nine months at two live classes a week plus four to six hours of practice, with two more months in hand. Homework, monthly mixed reviews, phase exams, and a final exam, an architecture defense, that decides the certificate with a free retest. Real engineering, real leadership, no crowns required.

WHAT MAKES THIS COURSE DIFFERENT

- The judgment layer: architecture, security and performance as decisions, not buzzwords
- AI engineering included seriously: ML engineering, LLMs and agents, the layer senior roles now assume
- Technical leadership taught practically: ADRs, review leadership, mentoring, the staff-vs-management fork
- A peer room of women on the same climb, which changes what gets asked and answered
- An honest 9-month arc with 2 months in hand
- Real assessment: monthly mixed reviews and a final architecture defense that decides the certificate, with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Advanced engineering (Months 1-3): design principles, cloud and data architecture, performance and security

Phase 2: AI engineering (Months 4-6): ML engineering, AI applications, LLMs and agents, data engineering
Phase 3: Technical leadership (Months 7-8): staff-level skills, leadership practice, management versus senior IC
Phase 4: The finale (Month 9): community leadership, the architecture capstone, and the final defense

WHAT A STUDENT NEEDS BEFORE STARTING

Experience: Working development experience, roughly 2+ years or a strong portfolio; Princess Coders graduates qualify after building professionally
Equipment: A computer with a stable internet connection
Audience: Women developers preparing for senior, staff and leadership work
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

Mid level engineers: Women developers whose next review says "more scope", and who want the skills that earn it
Aspiring leads: Engineers choosing between management and the senior IC road, wanting the choice informed
Ai upskillers: Developers whose roles now quietly assume AI engineering fluency
Returning seniors: Experienced women re-entering after a break, rebuilding at the level they left
Community builders: Engineers who want a peer room that continues past the course

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: ADVANCED ENGINEERING (MONTHS 1-3, WEEKS 1-12)

The judgment layer above syntax: design principles, cloud and data architecture, performance and security.

Month 1: Design and Cloud (Weeks 1-4)
Advanced Software Engineering Principles
Topics:
- Welcome to Queen Coders elite community
- Women tech leaders panel and networking
- Advanced data structures: Trees, Graphs, Heaps
- Algorithm complexity and optimization
- Design patterns: Factory, Observer, Strategy, Decorator
- SOLID principles deep dive
- Domain-Driven Design (DDD)
- Event-driven architecture patterns
- Microservices vs Monolithic architecture
- Service mesh and API gateways
- Distributed systems fundamentals
- CAP theorem and consistency models
Projects built:
- Design and implement a distributed cache system
- Build a custom ORM framework
- Create a microservices architecture blueprint
- Implement advanced design patterns library
Practice: Daily: System design problems, Algorithm challenges
Cloud Architecture & Infrastructure
Topics:
- Multi-cloud strategy: AWS, Azure, GCP comparison
- Infrastructure as Code with Terraform
- Advanced Kubernetes: operators, service mesh
- Serverless architecture patterns
- Event-driven with AWS Lambda, EventBridge
- Container orchestration at scale
- Cloud security: IAM, VPC, encryption
- Cost optimization strategies
- Disaster recovery and high availability
- Multi-region deployment strategies
- CDN and edge computing
- Observability: metrics, logs, traces
Projects built:
- Multi-region Kubernetes deployment
- Serverless e-commerce backend
- Infrastructure automation with Terraform
- Cloud cost optimization audit tool
Practice: Deploy 5 different cloud architectures
Assessment: Month 1 check: an architecture sketch defended live, plus mixed review

```
Month 2: Data and Performance (Weeks 5-8)
  Advanced Databases & Data Architecture
    Topics:
      - Database internals: B-trees, LSM trees
      - ACID vs BASE consistency models
      - Sharding strategies and implementation
      - Multi-master replication
      - Time-series databases: InfluxDB, TimescaleDB
      - Graph databases: Neo4j, Amazon Neptune
      - Vector databases for AI: Pinecone, Weaviate
      - Data warehousing: Snowflake, Redshift
      - Stream processing: Kafka, Kinesis
      - Event sourcing and CQRS
      - Data lake architecture
      - Real-time analytics pipelines
    Projects built:
      - Build a sharded database system
      - Real-time analytics pipeline with Kafka
      - Graph-based recommendation engine
      - Data warehouse ETL pipeline
    Practice: Design 10 complex data architectures
  Performance Engineering & Optimization
    Topics:
      - Performance profiling and benchmarking
      - Memory management and garbage collection
      - Concurrency patterns and parallel processing
      - Lock-free data structures
      - Cache optimization strategies
      - Database query optimization
      - Network optimization techniques
      - Load balancing algorithms
      - Rate limiting and throttling
      - Circuit breaker pattern implementation
      - Chaos engineering principles
      - Performance testing with JMeter, K6
    Projects built:
      - Build a high-performance web server
      - Optimize a slow application (10x improvement)
      - Implement custom load balancer
      - Create performance monitoring dashboard
    Practice: Optimize 5 production systems
    Assessment: Month 2 check: a performance investigation presented, plus mixed review

Month 3: Security and APIs (Weeks 9-12)
  Advanced Security Engineering
    Topics:
      - Security architecture patterns
      - Zero-trust architecture implementation
      - OAuth 2.0 and OpenID Connect deep dive
      - Multi-factor authentication systems
      - Encryption: symmetric, asymmetric, hashing
      - PKI and certificate management
      - Secure coding practices
      - OWASP Top 10 prevention strategies
      - Penetration testing techniques
      - Security incident response
      - Compliance: GDPR, HIPAA, SOC2
      - Blockchain for security applications
    Projects built:
      - Build enterprise SSO system
      - Implement zero-trust network
      - Security audit automation tool
      - Encrypted messaging system
    Practice: Security audit 10 applications
  API Design & GraphQL Mastery
    Topics:
      - REST API design best practices
      - GraphQL schema design patterns
      - Federation and schema stitching
      - Real-time subscriptions with GraphQL
      - API versioning strategies
      - API gateway patterns: Kong, Zuul
      - Rate limiting and quotas
      - API documentation with OpenAPI
      - gRPC for microservices
      - WebSocket architecture
      - Server-sent events (SSE)
      - API testing and mocking
    Projects built:
```

```
        - Design federated GraphQL architecture
        - Build API gateway from scratch
        - Real-time collaboration API
        - Multi-protocol API server
      Practice: Design and implement 5 complex APIs
      Assessment: Phase 1 exam: a system design defended under questioning, plus a written
      mixed paper

----------------------------------------------------------------
PHASE 2: AI ENGINEERING (MONTHS 4-6, WEEKS 13-24)
----------------------------------------------------------------
The layer every senior role now assumes: ML engineering, modern AI applications, LLMs and
agent development, and data engineering.

  Month 4: ML Engineering (Weeks 13-16)
    Machine Learning Engineering
      Topics:
        - ML system design and architecture
        - Feature engineering at scale
        - Model training pipelines
        - TensorFlow and PyTorch deep dive
        - Neural network architectures
        - CNNs for computer vision
        - RNNs and LSTMs for sequences
        - Transformers and attention mechanisms
        - Model optimization and quantization
        - MLOps: versioning, deployment, monitoring
        - A/B testing for ML models
        - Explainable AI (XAI)
      Projects built:
        - Build ML pipeline from scratch
        - Deploy computer vision system
        - Create NLP application
        - Implement recommendation system
      Practice: Deploy 5 ML models to production
    Advanced AI Applications
      Topics:
        - Large Language Models (LLMs) integration
        - Fine-tuning pre-trained models
        - Prompt engineering techniques
        - RAG (Retrieval Augmented Generation)
        - Vector databases for AI
        - Computer vision applications
        - Speech recognition and synthesis
        - Reinforcement learning basics
        - Generative AI: GANs, VAEs
        - AI ethics and bias mitigation
        - Federated learning
        - Edge AI deployment
      Projects built:
        - Build AI-powered chatbot with RAG
        - Create custom LLM application
        - Implement computer vision solution
        - Deploy edge AI system
      Practice: Build 5 AI-powered applications
      Assessment: Month 4 check: an ML system design reviewed, plus mixed review

  Month 5: LLMs and Agents (Weeks 17-20)
    LLMs & Generative AI
      Topics:
        - Transformer architecture deep dive
        - Building with GPT, BERT, T5
        - Fine-tuning strategies for LLMs
        - Prompt engineering advanced techniques
        - RAG systems architecture
        - Vector search optimization
        - LangChain and semantic kernel
        - Multi-agent AI systems
        - Constitutional AI and alignment
        - Diffusion models for generation
        - Voice synthesis and cloning
        - Cost optimization for LLM apps
      Projects built:
        - Custom LLM fine-tuning pipeline
        - Enterprise RAG system
        - Multi-agent AI platform
        - Generative AI creative tool
      Practice: Build 5 production LLM applications
    AI Engineering & Agent Development
```

```
      Topics:
        - LLM architecture deep dive: what the model actually does
        - Prompting as engineering, not incantation
        - Structured output and schema-constrained generation
        - Function calling and tool use
        - The agent loop: plan, act, observe, correct
        - Retrieval-augmented generation over real corpora
        - Vector stores, chunking, and when retrieval is the wrong answer
        - Multi-agent orchestration and where it fails
        - Evaluation: building an eval set before you build the feature
        - Guardrails, injection and untrusted input
        - Cost, latency and token budgets in production
        - Observability: tracing a non-deterministic system
      Projects built:
        - Retrieval-backed assistant
        - Tool-using agent
        - Eval harness with a real dataset
        - Production AI service with guardrails
      Practice: Deploy 5 smart contracts
      Assessment: Month 5 check: an agent design with guardrails defended, plus mixed review

  Month 6: Data Engineering and the Phase Capstone (Weeks 21-24)
    Data Engineering & Analytics
      Topics:
        - Data pipeline architecture
        - Apache Spark for big data
        - Real-time processing with Flink
        - Data lakehouse architecture
        - Delta Lake and Apache Iceberg
        - Orchestration with Apache Airflow
        - Data quality and governance
        - Business intelligence tools
        - Advanced SQL and query optimization
        - Time-series analysis
        - Predictive analytics
        - Data visualization best practices
      Projects built:
        - Build complete data platform
        - Real-time analytics dashboard
        - Predictive maintenance system
        - Data quality monitoring tool
      Practice: Build 5 data pipelines
    Phase 2 Capstone: An AI-Era System
      Topics:
        - Designing a system with an AI capability that earns its place
        - Cost, latency and quality traded off explicitly
        - Evaluation and guardrails as first-class design
        - Writing the design document a skeptic could audit
      Projects built:
        - A complete AI-era system design document, reviewed in panel
      Assessment: Phase 2 exam: the design defended, plus a cumulative written paper

----------------------------------------------------------------
PHASE 3: TECHNICAL LEADERSHIP (MONTHS 7-8, WEEKS 25-32)
----------------------------------------------------------------
The honest skills of staff-level work: leading through decisions, reviews and people, and
choosing between management and the senior IC road.

  Month 7: Staff-Level Craft (Weeks 25-28)
    Staff/Principal Engineering Skills
      Topics:
        - Staff+ engineering career paths
        - Technical leadership without management
        - Driving technical strategy
        - Cross-team collaboration
        - Technical mentorship at scale
        - Writing engineering strategies
        - Building technical roadmaps
        - Influencing without authority
        - Managing technical debt strategically
        - Architecture governance
        - Technical risk management
        - Being a force multiplier
      Projects built:
        - Create technical strategy doc
        - Lead cross-team initiative
        - Build mentorship program
        - Design technical roadmap
      Practice: Lead 5 strategic initiatives
```

```
    Technical Leadership Skills
      Topics:
        - Technical decision making
        - Architecture review boards
        - RFC and design doc writing
        - Code review best practices
        - Mentoring and coaching developers
        - Building high-performing teams
        - Stakeholder management
        - Technical debt management
        - Innovation frameworks
        - Budget and resource planning
        - Vendor evaluation and selection
        - Open source strategy
      Projects built:
        - Lead architecture review
        - Create mentorship program
        - Write technical RFC
        - Build team playbook
      Practice: Lead 5 technical initiatives
      Assessment: Month 7 check: an ADR written and defended, plus mixed review

  Month 8: Leading Engineers (Weeks 29-32)
    Engineering Leadership
      Topics:
        - From IC to leadership transition
        - Leading without authority
        - Building engineering culture
        - Hiring and interviewing
        - Performance management
        - Difficult conversations
        - Delegation and empowerment
        - Strategic thinking for engineers
        - Executive communication
        - Board presentation skills
        - Managing up and sideways
        - Women in leadership challenges
      Projects built:
        - Create hiring framework
        - Build team charter
        - Design performance system
        - Leadership portfolio
      Practice: Lead team of 5+ developers
    Engineering Management vs the Senior IC Road
      Topics:
        - Building high-performance teams
        - Agile at scale: SAFe, LeSS
        - OKRs and goal setting
        - Resource planning and allocation
        - Budget management for engineering
        - Vendor and contractor management
        - Remote team leadership
        - Diversity and inclusion initiatives
        - Engineering productivity metrics
        - Developer experience optimization
        - Burnout prevention strategies
        - Succession planning
      Projects built:
        - Design team structure
        - Implement OKR system
        - Create D&I initiative
        - Build productivity dashboard
      Practice: Manage team of 10+ engineers
      Assessment: Phase 3 exam: a leadership case study argued in writing, plus cumulative
      review

------------------------------------------------------------
PHASE 4: THE FINALE (MONTH 9, WEEKS 33-36)
------------------------------------------------------------
Community leadership, the architecture capstone, and the final defense.

  Month 9: Capstone and Defense (Weeks 33-36)
    Open Source and Community Leadership
      Topics:
        - Starting successful open source projects
        - Building contributor community
        - Open source governance models
        - License selection and implications
        - Monetizing open source
```

          - Developer advocacy
          - Conference speaking mastery
          - Technical writing and blogging
          - Building personal brand
          - Influencer strategies for developers
          - Podcast and video content
          - Women in open source
        Projects built:
          - Launch open source project
          - Speak at conference
          - Build developer community
          - Create content strategy
        Practice: Contribute to 10 major projects
      Capstone Build: The Architecture Dossier
        Topics:
          - One real system, redesigned: current state, pressures, target architecture
          - Trade-offs argued with numbers where numbers exist
          - Migration plan that respects reality
          - The dossier assembled for panel review
        Projects built:
          - The architecture dossier: a complete system redesign proposal
        Practice: Daily dossier progress; sections reviewed as written
      The Final Defense
        Topics:
          - Presenting the dossier before a panel
          - Taking hard questions without flinching or bluffing
          - A full-course spiral review
          - The road on: senior roles, Alpha Girls for systems at scale, and the community that stays
        Projects built:
          - The panel defense, delivered
        Assessment: FINAL EXAM: the architecture defense plus a written paper mixed from every
        phase. Passing earns the certificate; falling short earns focused revision and a free
        retest

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Phase 1: architecture sketches, performance investigations and a defended system design
  - Phase 2: ML system designs, an agent with guardrails, and an AI-era design document
  - Phase 3: ADRs, review leadership practice and a leadership case portfolio
  - Phase 4: the architecture dossier, defended before a panel

  Total projects: A portfolio of defended designs, ADRs and the final architecture dossier

SKILLS THE STUDENT WILL HAVE AT THE END

  - Architecture judgment: cloud, data, performance and security as explicit trade-offs
  - AI engineering: ML systems, LLMs and agents designed responsibly
  - Staff-level craft: decisions documented, reviews led, systems owned
  - Leadership: mentoring, influence and the management-vs-IC choice made consciously
  - Defense: presenting technical work under hard questioning

HOW A TYPICAL WEEK IS SPLIT

  Live classes: 2 live one-hour classes per week
  Homework: Homework after every class, and a mixed-review assignment at the end of every month
  Practice: 4-6 hours of practice between classes
  Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

  Doubt support: WhatsApp doubt support between classes
  Progress updates: Honest monthly progress reviews with a named focus for the next month

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short
    and there is focused revision and a free retest: the certificate certifies knowledge, not
    attendance

WHERE THIS CAN LEAD

  - Senior and staff engineering roles: the judgment and defense practice map directly
  - Alpha Girls: systems at scale, for engineers who want the deepest technical road
  - Engineering management: entered consciously, with the trade-offs understood
  - Our GenAI and agents courses: deeper into the AI engineering layer
  - Community leadership: mentoring the next Princess Coders batch

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Queen Coders, and who is it for?
A: An advanced course for women developers with working experience (roughly 2+ years or a strong portfolio) building the step from good developer to trusted engineer: architecture and security judgment, AI engineering with LLMs and agents, and practical technical leadership, ending in an architecture dossier defended before a panel.

Q: I am a developer but not "senior". Is this too early?
A: If you can build and ship features independently, this is exactly the right time: the course teaches the judgment layer that turns feature-builders into engineers trusted with systems. A short conversation before enrolment places you honestly; Princess Coders graduates typically join after a period of real building.

Q: How is leadership taught without becoming motivational fluff?
A: Through artifacts and practice: architecture decision records written and defended, code reviews led live, mentoring practiced on real juniors from our other courses, and the management-versus-senior-IC fork examined with honest trade-offs. Every leadership week produces something reviewable.

Q: How is progress assessed? Is the certificate just for attending?
A: No. Homework follows every class, each month ends with an assignment that deliberately mixes new questions with earlier topics, every phase closes with a cumulative exam, and the course ends with a final exam. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: What comes after Queen Coders?
A: Senior and staff-level work, entered with a defended dossier in hand; Alpha Girls for women who want the deepest technical road, systems at scale; or engineering management, chosen consciously. The peer room tends to persist, which is quietly the most valuable outcome.

Q: Is there a free trial class?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor, never pre-recorded videos.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/queen-coders-advanced-tech-leadership
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 122 OF 137: REACT COURSE FOR TEENS: JAVASCRIPT TO REAL REACT APPS
==========

Course link: https://learn.modernagecoders.com/courses/react-for-teens-complete-masterclass
Subject area: Web Development
Designed for: Teens
Who can join: Absolute beginner, ages 13 to 18
Course length: 9-11 months (36-44 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

React course for teens aged 13-18: web foundations, modern JavaScript, then React with hooks, router, TypeScript and a deployed capstone in 9-11 months.

In one line: The framework behind the interfaces teens live in, learned properly: real JavaScript first, then React the way it is written now, ending deployed.

ABOUT THE PROGRAMME

React renders the interfaces of Instagram, Netflix and half the modern web, and it rewards exactly one kind of learner: the one whose JavaScript is real. This live online course for ages 13 to 18 is built on that truth. Three months of genuine foundations, HTML, CSS and

JavaScript through the DOM and modern ES6+, then React itself: components, state, hooks, routing, data fetching, styling, forms, and the craft layer of TypeScript, testing and performance that separates tutorial-followers from builders. The final month adds an honest taste of Next.js and ships a deployed capstone at demo day. Everything deploys: projects go to public URLs from the first phase, and the capstone is a polished, routed, data-driven React application the student can send to anyone. What the course refuses to do is pad: React Native, full-stack backends and framework tours live in our dedicated courses, pointed to honestly, not stapled on as filler weeks. The pace is honest: nine months at two live classes a week plus three to four hours of practice, with two more months in hand when needed. Homework after every class, monthly mixed reviews, phase exams, and a final exam that decides the certificate, with focused revision and a free retest. Real JavaScript, real React, depth over dopamine.

WHAT MAKES THIS COURSE DIFFERENT

- JavaScript first, seriously: three months of real foundations, because React punishes weak JavaScript forever
- React as it is written now: function components, hooks, and honest data patterns with loading and error states everywhere
- The craft layer included: TypeScript, testing and performance, not left for "later"
- Everything deploys: public URLs from phase 1, a deployed capstone at the end
- Honest scope: React Native and backends are dedicated courses, pointed to truthfully, not filler weeks
- Real assessment: weekly homework, monthly mixed reviews, and a final exam that decides the certificate, with focused revision and a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Web and JavaScript foundations (Months 1-3): HTML, CSS, JavaScript through the DOM and ES6+, ending deployed
Phase 2: React core (Months 4-6): components, state, hooks, routing, APIs and styling, project by project
Phase 3: React craft (Months 7-8): forms, state at scale, TypeScript, testing and performance
Phase 4: Ship (Month 9): a Next.js taste, then the capstone built, deployed and defended at demo day

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The course starts at the first HTML tag
Computer skills: Comfortable typing and managing files
Equipment: A computer with a stable internet connection; every tool used is free
Age: 13 to 18. Batches are grouped by age and pace
Time commitment: 2 live classes plus 3-4 hours of practice a week

WHO THIS COURSE IS FOR

Interface lovers: Teens who notice good interfaces and want to build the kind they admire
Complete beginners: Students starting from zero: the JavaScript foundation is built in, not assumed
Tutorial escapees: Teens with tutorial miles who cannot yet build alone; the assessments close that gap
Future developers: Students heading toward software careers where React remains the default frontend answer
Makers: Teens who want their work live on real URLs, not in folders

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: WEB AND JAVASCRIPT FOUNDATIONS (MONTHS 1-3, WEEKS 1-12)

The real base React stands on: HTML, CSS and JavaScript learned deeply, through the DOM and modern ES6+, ending with a deployed project.

Month 1: The Web, Properly (Weeks 1-4)
Your First Steps in Web Development
Topics:
- How the internet works - simplified for teens
- What happens when you visit Instagram or TikTok
- Setting up your coding environment (VS Code)
- Your first HTML page - creating a personal profile
- HTML tags that matter: headings, paragraphs, images
- Adding links and navigation to pages
- Creating lists (perfect for todo apps later)
- HTML forms - how login pages work
- Embedding YouTube videos and Spotify playlists
- Making your page accessible to everyone
- File organization and naming conventions

- Using GitHub like a pro developer
Projects built:
- Personal profile page about your interests
- Fan page for your favorite artist/game/show
- School club or team website
Practice: Daily: Create one new HTML page about topics you love
CSS - Making Things Look Amazing
Topics:
- CSS basics - adding style to your pages
- Colors and backgrounds - creating themes
- Typography - choosing fonts like a designer
- The Box Model - spacing elements perfectly
- Flexbox - Instagram-style photo grids
- CSS Grid - Pinterest-like layouts
- Hover effects and transitions
- Animations - making elements move and bounce
- CSS variables for dark/light themes
- Mobile-responsive design for phones
- CSS frameworks introduction (Tailwind)
- Design principles teens should know
Projects built:
- Animated landing page for a fake startup
- Instagram-style photo gallery
- Responsive blog design with dark mode
Practice: Recreate UI elements from popular apps using CSS
Assessment: Month 1 check: build a styled page live from a sketch, plus a take-home mixing everything so far

Month 2: Modern CSS and First JavaScript (Weeks 5-8)
Advanced CSS & Modern Web Design
Topics:
- CSS animations and keyframes
- Transform and 3D effects
- Creating loading spinners and progress bars
- Gradient backgrounds like Spotify
- CSS filters for image effects
- Scroll-triggered animations
- Parallax scrolling effects
- CSS custom properties (variables)
- Responsive images and videos
- Mobile-first design approach
- Performance optimization basics
- Browser developer tools mastery
Projects built:
- Animated movie poster with hover effects
- Spotify-inspired music player UI
- Gaming website with parallax scrolling
Practice: Build 5 trending UI components from Dribbble
JavaScript Fundamentals - Programming Basics
Topics:
- What is JavaScript and why it's awesome
- Variables - storing information (let, const)
- Data types - numbers, strings, booleans
- Basic math and calculations
- Working with text (string methods)
- Making decisions with if/else
- Comparison operators (==, ===, >, <)
- Logical operators (&&, ||, !)
- Arrays - managing lists of data
- Loops - repeating tasks efficiently
- Functions - reusable code blocks
- First interactive features
Projects built:
- Grade calculator for school
- Simple quiz game about your interests
- Password strength checker
Practice: Solve 20 beginner-friendly JavaScript challenges
Assessment: Month 2 check: a JavaScript logic challenge live, plus mixed review of month 1

Month 3: The DOM and Modern JavaScript (Weeks 9-12)
JavaScript DOM Manipulation
Topics:
- The DOM - controlling web pages with JavaScript
- Selecting elements like jQuery but vanilla JS
- Changing content dynamically
- Adding and removing CSS classes
- Creating new elements with JavaScript
- Event listeners - clicks, hovers, keyboard

- Building interactive forms
- Form validation and error messages
- Local storage - saving user data
- Working with dates and times
- setTimeout and setInterval for timing
- Building countdown timers

Projects built:
- Interactive todo list with local storage
- Countdown timer for events
- Simple drawing app with canvas

Practice: Create 10 interactive web components

Modern JavaScript (ES6+) for React

Topics:
- Arrow functions - the React way
- Template literals for dynamic strings
- Destructuring - unpacking objects and arrays
- Spread operator - copying and merging
- Array methods crucial for React (map, filter, reduce)
- Working with objects in JavaScript
- ES6 modules - import and export
- Promises - handling asynchronous code
- Async/await - making promises easier
- Fetch API - getting data from the internet
- Working with JSON data
- Error handling with try/catch
- Deploying your best project to a public URL

Projects built:
- Meme generator using API
- Weather app for your city
- Pokemon card collection from API

Practice: Master array methods with 25 exercises

Assessment: Phase 1 exam: an interactive page built live, plus a written mixed paper over months 1-3

PHASE 2: REACT CORE (MONTHS 4-6, WEEKS 13-24)

React the way it is actually written: components and props, state and hooks, routing, real data with honest loading states, and styling that scales.

Month 4: Components and State (Weeks 13-16)

Introduction to React

Topics:
- What is React and why teens should learn it
- React vs vanilla JavaScript - the difference
- Setting up React with Vite
- Understanding the React project structure
- JSX - writing HTML in JavaScript
- Your first React component
- Functional components explained
- Props - passing data to components
- Children props and composition
- Conditional rendering (showing/hiding elements)
- Rendering lists with map
- Keys in React lists

Projects built:
- Profile card component
- Social media post component
- Product showcase with props

Practice: Build 15 different React components

React State & Interactivity

Topics:
- Understanding state in React
- useState hook - your first hook
- Handling events (onClick, onChange)
- Forms in React - controlled components
- Lifting state up between components
- State vs props - when to use what
- Multiple state variables
- State with objects and arrays
- Updating state correctly
- React Developer Tools
- Debugging React applications
- Common React mistakes to avoid

Projects built:
- Like button with counter
- Toggle dark/light theme
- Interactive shopping cart

Practice: Add state to 10 static components

```
    Assessment: Month 4 check: build an interactive component live, plus mixed review
    reaching back to JavaScript

Month 5: Hooks and Routing (Weeks 17-20)
  React Hooks Deep Dive
    Topics:
      - useEffect - side effects in React
      - useEffect dependencies array
      - Cleanup functions in useEffect
      - Data fetching with useEffect
      - Loading states and error handling
      - useContext for global state
      - Creating custom hooks
      - useReducer for complex state
      - useRef for DOM manipulation
      - useMemo for performance
      - useCallback explained
      - Rules of hooks
    Projects built:
      - GitHub profile viewer
      - Real-time search with debouncing
      - Theme context provider
    Practice: Create 5 custom hooks for common tasks
  React Router & Navigation
    Topics:
      - Single Page Applications (SPAs) explained
      - React Router installation and setup
      - Routes and Route components
      - Link vs regular anchor tags
      - NavLink for active styles
      - URL parameters and useParams
      - Query strings and useSearchParams
      - Nested routes and layouts
      - Protected routes for authentication
      - 404 pages and error handling
      - Programmatic navigation
      - Route animations and transitions
    Projects built:
      - Multi-page portfolio site
      - Blog with individual post pages
      - E-commerce site with routing
    Practice: Add routing to previous projects
    Assessment: Month 5 check: a routed mini-app built live, plus mixed review

Month 6: Real Data and Styling (Weeks 21-24)
  Data and APIs in React
    Topics:
      - REST APIs explained for teens
      - Fetch API in React components
      - Axios for HTTP requests
      - Managing loading and error states
      - Displaying API data in components
      - Pagination with APIs
      - Infinite scrolling implementation
      - Search functionality with APIs
      - Authentication tokens and headers
      - Environment variables for API keys
      - CORS and how to handle it
      - Mock APIs for development
      - Loading, error and empty states as a discipline, not an afterthought
    Projects built:
      - Movie database app (TMDB API)
      - Spotify playlist viewer
      - News aggregator for teens
    Practice: Integrate 5 different public APIs
  Styling React Applications
    Topics:
      - CSS Modules in React
      - Styled-components introduction
      - CSS-in-JS benefits and drawbacks
      - Tailwind CSS with React
      - Material-UI components
      - Chakra UI for quick development
      - Creating a design system
      - Responsive design in React
      - Animation libraries (Framer Motion)
      - React Spring for animations
      - Transition effects between routes
      - Building reusable styled components
```

```
      Projects built:
        - Instagram-style photo feed
        - Animated dashboard with charts
        - Discord-inspired chat UI
      Practice: Style 5 apps with different approaches
      Assessment: Phase 2 exam: a data-driven, routed, styled app defended, plus a cumulative
      written paper

---------------------------------------------------------------
PHASE 3: REACT CRAFT (MONTHS 7-8, WEEKS 25-32)
---------------------------------------------------------------
What separates builders from tutorial-followers: forms done right, state at scale,
TypeScript, and testing with performance judgment.

  Month 7: Forms and State at Scale (Weeks 25-28)
    Forms & Validation in React
      Topics:
        - Controlled vs uncontrolled components
        - Complex form handling
        - Multi-step forms
        - Form validation strategies
        - React Hook Form library
        - Yup validation schema
        - File uploads in React
        - Image preview before upload
        - Dynamic form fields
        - Form state management
        - Error display patterns
        - Success feedback to users
      Projects built:
        - Registration form with validation
        - Survey form with multiple steps
        - Profile editor with image upload
      Practice: Build 5 different form types
    State at Scale: Context and Beyond
      Topics:
        - When to use Context API
        - Creating context providers
        - useContext hook patterns
        - Multiple contexts in an app
        - Context optimization techniques
        - Avoiding unnecessary re-renders
        - Context vs props drilling
        - Auth context implementation
        - Theme context patterns
        - Shopping cart with context
        - Context best practices
        - When context isn't enough
        - Redux Toolkit and Zustand in honest perspective: when component state stops being enough
      Projects built:
        - Multi-theme app with context
        - Global notification system
        - User authentication with context
      Practice: Implement context in 5 different scenarios
      Assessment: Month 7 check: lift and share state across an app live, plus mixed review

  Month 8: TypeScript and Quality (Weeks 29-32)
    TypeScript with React
      Topics:
        - Why TypeScript makes React better
        - TypeScript setup for React
        - Typing props and state
        - Function component types
        - Event handler types
        - useReducer with TypeScript
        - Context API with TypeScript
        - Redux with TypeScript
        - Generic components
        - Utility types for React
        - Third-party library types
        - Migration strategies
      Projects built:
        - Convert React app to TypeScript
        - Type-safe form builder
        - TypeScript component library
      Practice: Write all new React code in TypeScript
    Testing and Performance
      Topics:
        - Why testing matters for teens
```

- Jest testing framework basics
- React Testing Library introduction
- Writing your first test
- Testing components with props
- Testing state changes
- Testing user interactions
- Mocking API calls in tests
- Testing custom hooks
- Integration testing basics
- Test-driven development (TDD) intro
- Coverage reports
- Performance essentials: memoization judgment, lazy loading and measuring before optimizing

Projects built:
- Add tests to todo app
- Test suite for form validation
- Testing user authentication flow

Practice: Write tests for 5 components

Assessment: Phase 3 exam: type and test a feature live, plus a cumulative written paper over phases 1-3

PHASE 4: SHIP (MONTH 9, WEEKS 33-36)

An honest taste of Next.js, then the main event: the capstone built, deployed and defended at demo day.

Month 9: Next Taste, Capstone, Demo Day (Weeks 33-36)

A Taste of Next.js

Topics:
- Why the industry wraps React in Next.js
- File-based routing and layouts
- Server components in plain words
- When plain React is still the right call
- Deploying a Next.js site
- How everything you learned maps onto any React framework

Projects built:
- A small blog or portfolio rebuilt in Next.js

Practice: Compare your Vite and Next builds: what changed and why

Capstone Spec and Setup

Topics:
- Choosing the capstone: a real product idea scoped to three weeks
- A one-page spec: users, screens, data, components
- Component architecture sketched before code
- Repository, TypeScript config and deployment target ready on day one
- Instructor sign-off on scope

Projects built:
- Approved capstone spec plus the running skeleton, deployed

Practice: Pitch the spec to the batch and absorb one hard question

Capstone Build

Topics:
- One focused week: features in vertical slices
- TypeScript throughout, loading states everywhere
- Instructor checkpoints and honest scope cuts
- Daily commits and an always-deployed build
- A mid-week hallway test with a real user

Projects built:
- The capstone: a routed, data-driven, styled React application, feature-complete

Practice: A deployed build at the end of every session

Demo Day and the Final Exam

Topics:
- Polish pass: empty states, small screens, accessibility basics
- Presenting the capstone: live demo plus the hardest bug story
- A full-course spiral review before the exam
- Where each path leads: full-stack for the backend half, app development for React Native

Projects built:
- Demo day presentation delivered to the batch and parents

Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; a student who falls short gets a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: styled pages, JavaScript games and an interactive project deployed to a public URL
- Phase 2: component libraries, routed apps and a data-driven project with honest loading states
- Phase 3: validated forms, shared-state apps, a typed and tested feature set
- Phase 4: a Next.js rebuild, then the deployed capstone defended at demo day

Total projects: 20+ real builds, crowned by a deployed React capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Web foundations: HTML, CSS and responsive layouts done properly
- JavaScript to real depth: the DOM, ES6+ and the async patterns React assumes
- React core: components, hooks, routing, data fetching and styling
- The craft layer: TypeScript, testing, performance judgment and forms
- Shipping: deployment as a habit, from phase 1 to the capstone

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: Homework after every class, and a mixed-review assignment at the end of every month
Practice: 3-4 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, for the component that renders nothing and says nothing
Progress updates: Regular progress notes to parents: what was built, what comes next, and where encouragement helps

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Full-stack web development: our dedicated course adds the backend half
- App development: React Native continues this exact skill set onto phones
- Data Structures and Algorithms: the computer-science muscle for engineering paths
- Design directions: component thinking pairs naturally with UI/UX
- Real projects now: sites and apps for school clubs, family and friends

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What will my teen build in this React course?
A: More than 20 real builds across nine months: styled, deployed pages and JavaScript projects in the foundation phase, then routed, data-driven React applications with honest loading states, typed and tested features in the craft phase, a small Next.js rebuild, and finally a capstone, a complete React application deployed to a public URL and defended at demo day.

Q: How long does it take a teenager to learn React?
A: Nine months at 2 live classes a week plus 3 to 4 hours of practice, and a third of that is deliberately spent on JavaScript foundations, because React punishes weak JavaScript forever. The course keeps two more months in hand for students who need them. Courses that jump into React in week one produce students who can follow tutorials and cannot debug them.

Q: How is my teen assessed? Is the certificate just for attending?
A: No. Homework follows every class, each month ends with an assignment that deliberately mixes new questions with earlier topics, every phase closes with an exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Does my teen need to know JavaScript first?
A: No, and that is the design: the first three months build HTML, CSS and JavaScript properly, through the DOM and modern ES6+, before a single React component. Teens who already know some JavaScript are placed by a quick check and move faster through the early months.

Q: Does this course cover React Native or backend development?
A: Honestly, no, and on purpose. React Native and backend engineering are real disciplines with their own dedicated courses here, and stapling them on as survey weeks would cheat all three subjects. This course makes the React foundation deep; our app development course continues it onto phones, and our full-stack course adds the backend half.

Q: Is React still worth learning?
A: Yes: it remains the most-used frontend framework in industry by a wide margin, the concepts it teaches, components, state, unidirectional data, transfer to every alternative, and its ecosystem (Next.js included, which this course tastes honestly) keeps compounding. A teen fluent in React and TypeScript holds a genuinely employable-track skill.

Q: Is there a free trial class?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or

message us on WhatsApp at +91 91233 66161. Parents are welcome to sit in. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small interactive groups with a real instructor, never pre-recorded videos. React errors are famously cryptic for beginners; live eyes on the code is what keeps early frustration from becoming quitting.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/react-for-teens-complete-masterclass
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 123 OF 137: REACT MASTERCLASS: JAVASCRIPT TO PRODUCTION REACT

==========

Course link: https://learn.modernagecoders.com/courses/react-js-complete-masterclass-college
Subject area: Web Development
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

React course for college students and professionals: JavaScript foundations, React with hooks, state management, TypeScript, testing and Next.js in 10-12 months.

In one line: The frontend library hiring still runs on, learned to production depth: hooks, state, TypeScript, testing and Next.js, ending deployed.

ABOUT THE PROGRAMME

React remains the most-hired frontend skill by a wide margin, and the gap between tutorial React and production React is where careers are decided. This live online masterclass closes it: JavaScript foundations first, then React as it is actually written, function components, hooks, honest data patterns, real state management with Redux Toolkit, then the production layer: TypeScript throughout, testing that catches regressions, performance measured before optimized, and Next.js as the industry wrapper. A finale month ships a production-shaped capstone and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- JavaScript depth first: React punishes weak JavaScript forever
- Production patterns, not tutorial patterns: loading states, error boundaries and data honesty throughout
- TypeScript and testing as core phases, not appendices
- Next.js taught as the industry actually uses it
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations (Months 1-3): JavaScript properly, then React fundamentals
Phase 2: Advanced React (Months 4-6): hooks in depth, patterns, and state management that scales
Phase 3: Production layer (Months 7-9): TypeScript, testing, performance and Next.js
Phase 4: The finale (Month 10): a production-shaped capstone and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Everything is taught from the first line
Equipment: Any computer with a stable internet connection; every tool used is free
Age: 18+; college students, graduates and working professionals

Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

College students: Students who want the most-hired frontend skill at production depth
Tutorial escapees: Developers who can follow React tutorials but not debug them
Career switchers: Professionals entering frontend at the hiring baseline
Backend developers: Engineers adding the frontend half properly
Product builders: Founders and makers whose products need real interfaces

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: JAVASCRIPT & REACT FUNDAMENTALS (MONTHS 1-3, WEEKS 1-13)

Build rock-solid foundations in JavaScript ES6+ and React core concepts, components, and hooks.

Months 1-2: JavaScript Mastery for React (Weeks 1-8)
Modern JavaScript Essentials
Topics:
- JavaScript fundamentals review
- ES6+ features: let, const, arrow functions
- Template literals and string interpolation
- Destructuring arrays and objects
- Spread and rest operators
- Default parameters
- Object property shorthand
- Array methods: map, filter, reduce, find
- forEach vs map understanding
- Promises and async/await
- Modules: import/export
- Classes and inheritance
Projects built:
- JavaScript utility functions library
- Todo app with vanilla JavaScript
- Weather app with API calls
- Array manipulation exercises
Practice: Complete 100 JavaScript challenges
Introduction to React
Topics:
- What is React and why use it?
- Virtual DOM concept
- React ecosystem overview
- Setting up development environment
- Vite vs Next.js: choosing a React setup
- Project structure best practices
- JSX syntax and expressions
- JSX vs HTML differences
- Embedding JavaScript in JSX
- React Developer Tools
- First React component
- React.StrictMode
Projects built:
- First React app setup
- Static website with React
- Personal portfolio starter
- JSX exercises and experiments
Practice: Create 20 simple React components
Components & Props
Topics:
- Functional components
- Component composition
- Props and prop types
- Children props
- Prop drilling introduction
- Default props
- Spreading props
- Component organization
- Conditional rendering techniques
- Lists and keys in React
- Fragments and React.Fragment
- Component lifecycle overview
Projects built:
- Component library creation
- Blog post list component
- Product card components

```
      - Navigation menu component
    Practice: Build 30 reusable components
  State & Event Handling
    Topics:
      - Understanding state in React
      - useState Hook deep dive
      - State vs props
      - Event handling in React
      - Synthetic events
      - Common events: onClick, onChange, onSubmit
      - Event object and preventDefault
      - Handling forms basics
      - Controlled vs uncontrolled components
      - Multiple state variables
      - State update patterns
      - Immutability importance
    Projects built:
      - Interactive counter app
      - Form with validation
      - Todo list with state
      - Color picker component
    Practice: Build 20 interactive components with state

Month 3: React Hooks Mastery (Weeks 9-13)
  Essential Hooks
    Topics:
      - Rules of Hooks
      - useState advanced patterns
      - useEffect lifecycle replacement
      - useEffect dependencies array
      - Cleanup functions
      - Multiple useEffect hooks
      - useContext for global state
      - useReducer for complex state
      - useReducer vs useState
      - useCallback for memoization
      - useMemo for expensive computations
      - useRef for DOM access and values
    Projects built:
      - Data fetching with useEffect
      - Theme switcher with Context
      - Complex form with useReducer
      - Performance optimization examples
    Practice: Master all essential hooks with 30 examples
  Custom Hooks & Patterns
    Topics:
      - Creating custom hooks
      - Custom hook best practices
      - useLocalStorage hook
      - useFetch hook
      - useDebounce hook
      - useThrottle hook
      - usePrevious hook
      - useOnClickOutside hook
      - useMediaQuery hook
      - useIntersectionObserver hook
      - Hook composition patterns
      - Testing custom hooks
    Projects built:
      - Custom hooks library
      - Form validation hooks
      - API hooks collection
      - Animation hooks
    Practice: Create 20 custom hooks
  Styling in React
    Topics:
      - CSS modules
      - CSS-in-JS introduction
      - Styled-components basics
      - Emotion library
      - Inline styles in React
      - Dynamic styling
      - CSS frameworks with React
      - Tailwind CSS integration
      - Bootstrap with React
      - Material-UI basics
      - Animations with React
      - CSS transitions in React
    Projects built:
```

- Styled-components theme
- Responsive design system
- Animated components
- PHASE 1 CAPSTONE: Complete React Application with Custom Styling

Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 2: ADVANCED REACT & STATE MANAGEMENT (MONTHS 4-6, WEEKS 14-26)

Master routing, forms, API integration, and professional state management solutions.

Months 4-5: Routing & Data Management (Weeks 14-21)

React Router

Topics:

- React Router v6 setup
- BrowserRouter vs HashRouter
- Route components
- Link and NavLink
- Nested routes
- Route parameters
- Query strings
- useNavigate hook
- useLocation hook
- useParams hook
- Protected routes
- Route guards
- Lazy loading routes
- 404 pages

Projects built:

- Multi-page application
- E-commerce product routing
- Admin dashboard with nested routes
- Authentication flow

Practice: Build 10 routing scenarios

Forms & Validation

Topics:

- Controlled components deep dive
- Form state management
- Multi-step forms
- File uploads
- Form validation strategies
- React Hook Form
- Formik library
- Yup validation schemas
- Custom validation
- Error handling
- Form optimization
- Dynamic forms
- Form arrays
- Accessibility in forms

Projects built:

- Registration system
- Survey application
- Dynamic form builder
- Checkout process

Practice: Create 15 complex forms

API Integration & Data Fetching

Topics:

- Fetch API with React
- Axios integration
- Loading states
- Error handling
- Async operations in React
- Data fetching patterns
- Caching strategies
- Optimistic updates
- Pagination implementation
- Infinite scrolling
- Real-time data with WebSockets
- Server-Sent Events
- GraphQL with Apollo Client
- React Query (TanStack Query)

Projects built:

- Blog with API backend
- Real-time chat application
- GitHub profile viewer
- Infinite scroll feed

Practice: Integrate 10 different APIs

```
  Context API & State Management
    Topics:
      - Context API deep dive
      - Creating providers
      - Multiple contexts
      - Context optimization
      - Context vs props
      - Global state patterns
      - State management principles
      - Introduction to Redux
      - Redux Toolkit modern approach
      - Actions and reducers
      - Redux DevTools
      - Async actions with Redux Thunk
      - RTK Query
    Projects built:
      - Shopping cart with Context
      - Theme system with Context
      - Todo app with Redux
      - E-commerce with Redux Toolkit
    Practice: Implement 10 state management scenarios
  Advanced Redux Patterns
    Topics:
      - Redux middleware
      - Redux Saga introduction
      - Normalized state shape
      - Reselect for memoization
      - Redux persist
      - Redux forms
      - Time travel debugging
      - Redux performance
      - Code splitting with Redux
      - Redux with TypeScript preview
      - Alternative: Zustand
      - Alternative: MobX
      - Alternative: Recoil
      - Choosing state management
    Projects built:
      - Complex app with Redux Saga
      - Offline-capable app
      - Multi-user collaboration app
      - State management comparison
    Practice: Master advanced Redux patterns

Month 6: Component Patterns & Architecture (Weeks 22-26)
  Advanced Component Patterns
    Topics:
      - Higher-Order Components (HOCs)
      - Render props pattern
      - Compound components
      - Controlled vs uncontrolled
      - Presentational vs container components
      - Provider pattern
      - State initializer pattern
      - State reducer pattern
      - Prop collections pattern
      - Prop getters pattern
      - Component composition strategies
      - Slot pattern
      - Inversion of control
      - Extensible components
    Projects built:
      - Advanced UI component library
      - Flexible modal system
      - Data table component
      - Accordion component set
    Practice: Implement 15 component patterns
  Performance Optimization
    Topics:
      - React DevTools Profiler
      - Performance metrics
      - React.memo optimization
      - useMemo strategies
      - useCallback patterns
      - Lazy loading components
      - Code splitting with React.lazy
      - Suspense for data fetching
      - Virtualization with react-window
      - Image optimization
```

```
        - Bundle size optimization
        - Tree shaking
        - Performance monitoring
        - Core Web Vitals
      Projects built:
        - Optimize large list rendering
        - Performance audit project
        - Image gallery optimization
        - Bundle size reduction
      Practice: Optimize 10 slow React apps
    Error Handling & Debugging
      Topics:
        - Error boundaries
        - Error boundary patterns
        - Fallback UI design
        - Error reporting services
        - Debugging techniques
        - React DevTools mastery
        - Console debugging strategies
        - Network tab debugging
        - Source maps
        - Debugging production issues
        - Logging strategies
        - Error tracking with Sentry
        - Performance debugging
        - Memory leak detection
      Projects built:
        - Error boundary system
        - Debugging toolkit
        - Error monitoring setup
        - Production debugging guide
      Practice: Debug 20 React issues
    Accessibility & SEO
      Topics:
        - Web accessibility principles
        - ARIA attributes in React
        - Keyboard navigation
        - Screen reader support
        - Focus management
        - Accessible forms
        - Color contrast
        - Accessible modals
        - Testing accessibility
        - SEO with React SPA
        - Meta tags management
        - React Helmet
        - Open Graph tags
        - Structured data
      Projects built:
        - Accessible component library
        - SEO-optimized blog
        - Accessibility audit tool
        - WCAG compliant app
      Practice: Make 10 apps fully accessible
    Phase 2 Capstone Project
      Topics:
        - Complex application planning
        - Architecture design
        - State management setup
        - API integration
        - Performance optimization
        - Testing implementation
      Projects built:
        - PHASE 2 CAPSTONE: Full-Featured Social Media App
        - Build complete social platform with posts, comments, likes
        - Include: authentication, real-time updates, image uploads
        - Redux state management, optimized performance
      Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

------------------------------------------------------------
PHASE 3: TYPESCRIPT, TESTING & NEXT.JS (MONTHS 7-9, WEEKS 27-39)
------------------------------------------------------------
Master TypeScript with React, comprehensive testing, and server-side rendering with Next.js.

  Months 7-8: TypeScript & Testing (Weeks 27-34)
    TypeScript Fundamentals for React
      Topics:
        - TypeScript setup with React
```

```
    - Basic types and interfaces
    - Type annotations
    - Function types
    - Union and intersection types
    - Type aliases
    - Generics basics
    - Type inference
    - Strict mode
    - tsconfig.json configuration
    - TypeScript with Vite
    - TypeScript with Vite
    - Migration strategies
    - Common TypeScript errors
  Projects built:
    - Convert JS project to TypeScript
    - Type-safe component library
    - TypeScript configuration setup
    - Type definitions creation
  Practice: Type 30 React components
TypeScript with React
  Topics:
    - Typing functional components
    - Props interface patterns
    - Children prop types
    - Event handler types
    - Hooks with TypeScript
    - useState with TypeScript
    - useReducer typing
    - useContext typing
    - Custom hooks typing
    - Generic components
    - Discriminated unions
    - Type guards
    - Utility types
    - Third-party library types
  Projects built:
    - Type-safe form system
    - Redux with TypeScript
    - API client with types
    - Component library with types
  Practice: Build 20 type-safe components
Testing React Applications
  Topics:
    - Testing philosophy
    - Jest setup and configuration
    - React Testing Library
    - Component testing
    - Testing hooks
    - Testing async operations
    - Mocking modules
    - Testing user interactions
    - Integration testing
    - Snapshot testing
    - Coverage reports
    - Testing best practices
    - TDD with React
    - Debugging tests
  Projects built:
    - Test suite for component library
    - TDD todo application
    - Integration test suite
    - Testing utilities library
  Practice: Write 100+ test cases
End-to-End Testing
  Topics:
    - E2E testing introduction
    - Cypress setup
    - Writing Cypress tests
    - Page object pattern
    - API testing with Cypress
    - Visual regression testing
    - Playwright introduction
    - Cross-browser testing
    - CI/CD integration
    - Test automation strategies
    - Performance testing
    - Accessibility testing
    - Security testing basics
    - Test reporting
```

```
    Projects built:
      - E2E test suite with Cypress
      - Visual regression tests
      - CI/CD pipeline with tests
      - Automated test reports
    Practice: Create comprehensive E2E coverage
  Introduction to Next.js
    Topics:
      - Next.js benefits and features
      - Create Next App setup
      - File-based routing
      - Pages and layouts
      - Static generation (SSG)
      - Server-side rendering (SSR)
      - API routes
      - Dynamic routes
      - Link component
      - Image optimization
      - CSS and styling in Next.js
      - Environment variables
      - Next.js vs a plain Vite SPA
      - Migration to Next.js
    Projects built:
      - Blog with Next.js
      - Portfolio with SSG
      - E-commerce homepage
      - API with Next.js
    Practice: Build 5 Next.js applications

Month 9: Advanced Next.js & Full-Stack (Weeks 35-39)
  Advanced Next.js Features
    Topics:
      - Incremental Static Regeneration (ISR)
      - Dynamic imports
      - Middleware
      - Edge functions
      - Authentication in Next.js
      - NextAuth.js
      - Database integration
      - Prisma with Next.js
      - tRPC integration
      - Internationalization (i18n)
      - SEO optimization
      - Performance optimization
      - Deployment strategies
      - Vercel deployment
    Projects built:
      - Full-stack app with authentication
      - Multi-language website
      - E-commerce with ISR
      - SaaS application starter
    Practice: Master advanced Next.js features
  GraphQL with React
    Topics:
      - GraphQL introduction
      - GraphQL vs REST
      - Apollo Client setup
      - Queries and mutations
      - GraphQL fragments
      - Optimistic UI
      - Caching strategies
      - Subscriptions
      - Error handling
      - Authentication with GraphQL
      - File uploads
      - Relay modern basics
      - GraphQL code generation
      - Performance optimization
    Projects built:
      - GraphQL blog application
      - Real-time chat with subscriptions
      - E-commerce with GraphQL
      - GitHub clone with GraphQL
    Practice: Build 5 GraphQL applications
  Micro-Frontends & Module Federation
    Topics:
      - Micro-frontends architecture
      - Benefits and challenges
      - Module federation with Webpack 5
```

- Sharing dependencies
- Runtime integration
- Build-time integration
- Single-spa framework
- Qiankun framework
- Communication between micro-frontends
- Shared state management
- Deployment strategies
- Testing micro-frontends
- Performance considerations
- When to use micro-frontends

Projects built:
- Micro-frontend architecture
- Module federation setup
- Multi-team application
- Legacy app integration

Practice: Implement micro-frontend patterns

Progressive Web Apps (PWA)

Topics:
- PWA principles
- Service workers in React
- Offline functionality
- Cache strategies
- Web app manifest
- Push notifications
- Background sync
- App shell architecture
- Performance optimization for PWA
- Lighthouse audits
- PWA deployment
- App store submission
- PWA vs native apps
- Workbox library

Projects built:
- Offline-first news app
- PWA e-commerce site
- Push notification system
- Installable web app

Practice: Convert 5 apps to PWAs

Phase 3 Capstone Project

Topics:
- Full-stack application design
- TypeScript throughout
- Comprehensive testing
- Next.js implementation
- Performance optimization
- Production deployment

Projects built:
- PHASE 3 CAPSTONE: Production-Ready SaaS Application
- Build complete SaaS with Next.js, TypeScript, testing
- Include: authentication, payments, admin panel
- Full test coverage, deployed to production

Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)

The capstone month: spec, build, ship and defend, then the final exam that decides the certificate.

Month 10: Capstone and the Final Exam (Weeks 40-44)

Capstone Spec and Setup

Topics:
- Choosing the capstone: a production-shaped React application: dashboard, social product or commerce frontend
- A one-page spec: scope, milestones, stack
- Repository and skeleton ready on day one
- Instructor sign-off before the build begins

Projects built:
- Approved capstone spec plus the running skeleton, deployed where applicable

Practice: Pitch the spec to the batch and absorb one hard question

Capstone Build

Topics:
- Two focused weeks in vertical slices
- Instructor checkpoints and honest scope cuts
- Using the whole course: hooks, state management, TypeScript, tests and Next.js where it earns it
- Daily commits and an always-working build

Projects built:
- The capstone, feature-complete with version control
Practice: A working build at the end of every session
Ship and Polish
Topics:
- The finishing pass: edge cases, errors, documentation
- Lighthouse and test-suite pass: performance and correctness by evidence
- A README worth reading and a demo worth watching
- A full-course spiral review before the exam
Projects built:
- The capstone shipped, documented and rehearsed
Practice: Two mock demos, each tighter than the last
Demo Day and the Final Exam
Topics:
- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: frontend specialization, full-stack product work, or React Native
- Where our other courses continue the journey
Projects built:
- Demo day presentation delivered to the batch
Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: JavaScript builds and first React applications
- Phase 2: hook-driven apps with real state management
- Phase 3: typed, tested, measured applications with Next.js
- Phase 4: the production-shaped capstone, deployed and defended

Total projects: 20+ real builds, crowned by a deployed production-shaped capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- JavaScript depth: the async and data patterns React assumes
- React as written now: components, hooks and honest data handling
- State at scale: context judgment and Redux Toolkit fluency
- The production layer: TypeScript, React Testing Library, performance measurement
- Next.js: the industry wrapper, used with judgment

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Frontend roles: this is the depth those interviews probe
- Our full-stack course: add the backend half
- App development: React Native carries this skill onto phones
- DSA for the interview algorithm rounds

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Growing roles: Rs 5-8 LPA (Junior React Developer)
Experienced roles: Rs 8-15 LPA (React Developer); Rs 12-25 LPA (Senior React Developer); Rs 20-40+ LPA (Lead/Architect)
Freelance: Rs 2,000-8,000/hour based on expertise
International: $60k-150k USD based on location and experience

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What does this React masterclass cover?
  A: JavaScript foundations first, then React as production teams write it: components and hooks in depth, honest data patterns, state management with Redux Toolkit, then the production layer, TypeScript throughout, testing with React Testing Library, performance measured before optimized, and Next.js. A finale month ships a deployed, tested capstone and sits the final exam.

  Q: How long does it take to learn React properly?
  A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two more months in hand for learners who need them. The three teaching phases build the skill and its real-world application; the finale month ships a capstone and sits the final exam.

  Q: How am I assessed? Is the certificate just for attending?
  A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

  Q: Does this course include React Native?
  A: No, honestly: mobile is its own discipline with its own course here (our app development track), and stapling it on would cheat both. What this course guarantees is the React depth React Native then assumes, students who continue into the app course arrive ready.

  Q: Can I try a class before enrolling?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/react-js-complete-masterclass-college
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 124 OF 137: ROBLOX GAME CODING FOR KIDS: BUILD REAL GAMES IN LUAU
==========

Course link: https://learn.modernagecoders.com/courses/roblox-game-coding-for-kids-lua-course
Subject area: Game Development
Designed for: Kids
Who can join: Beginner, ages 8 to 14
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Roblox coding course for kids ages 8-14: live online classes in Roblox Studio and Luau scripting, from first parts to a published game friends can play.

  In one line: Your child already plays Roblox. In six months they publish a game of their own on it.

ABOUT THE PROGRAMME

  Roblox is where millions of kids already spend their screen time, and it hides a genuinely serious tool: Roblox Studio, the free game-making software behind every experience on the platform, scripted in a real programming language called Luau. This 6-month live course turns a player into a maker. Months 1 and 2 cover building in Studio and the first Luau scripts: variables, decisions, loops, and functions, each one changing something visible in a 3D world. Months 3 and 4 bring games to life with events, player interaction, scoring, and checkpoints, which is where an obby stops being a row of parts and starts being a game. Month

5 adds the mechanics kids recognize from games they love: shops, timers, rounds, sound, and polish. Month 6 is the capstone: each student designs, builds, tests, and publishes their own game to Roblox, then presents it. Classes are live and small, and students build in every single session. Nothing in this course is watch-and-copy: the instructor sets a challenge, watches each student work, and reviews the homework builds between classes.

WHAT MAKES THIS COURSE DIFFERENT

- Students learn Luau, the actual scripting language of Roblox, not a drag-and-drop imitation of it
- Every concept lands inside a 3D world the student built: loops make disco floors, conditionals open doors, functions run traps
- The classic beginner bugs (debounce, anchoring, nil) are taught on purpose, because fixing them is where real understanding starts
- By month 4 students have leaderboards, checkpoints, and coin systems working, the same building blocks behind popular Roblox games
- The capstone is published to Roblox for real, so the finished game has a link a grandparent can open
- Everything used is free: Roblox Studio costs nothing, publishing costs nothing, and no Robux are ever needed

THE LEARNING PATH, STEP BY STEP

Phase 1: Builder to scripter: master Roblox Studio's tools, then write first Luau scripts that change the world from code
Phase 2: Making it a game: events, player interaction, scoring, coins, checkpoints, and on-screen interfaces
Phase 3: Shops, timers, rounds, and polish, then a capstone game scoped, built, published to Roblox, and presented

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. The course starts from placing the first part and writing the first line
Roblox experience: Helpful but not required. Most students have played Roblox; none have needed to
Equipment: A Windows or Mac computer that runs Roblox Studio, plus a stable internet connection. Phones, tablets, and most Chromebooks cannot run Studio
Account: A free Roblox account, set up with a parent. No Robux or paid membership is needed at any point

WHO THIS COURSE IS FOR

Roblox players: Kids who already love playing Roblox and keep asking how the games they play are actually made
Complete beginners: Children aged 8 to 14 with no coding background who want a first language wrapped in a world they care about
Scratch graduates: Students moving up from block coding who are ready to type real scripts in a real engine
Young game designers: Kids who fill notebooks with game ideas and need the tools to build one for real
Parents seeking direction: Families who want Roblox hours converted from pure consumption into building, debugging, and finishing things

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: STUDIO SKILLS AND FIRST LUAU SCRIPTS (MONTHS 1-2, WEEKS 1-8)

First the tools: building worlds in Roblox Studio with confidence. Then the leap every Roblox kid dreams about: writing real Luau code that changes the world by itself.

Month 1: Becoming a Builder in Roblox Studio (Weeks 1-4)
Welcome to Roblox Studio
Topics:
- Installing Roblox Studio and a tour of the interface: Explorer, Properties, and the 3D viewport
- Camera controls: fly, zoom, and orbit without getting lost
- Creating parts: blocks, spheres, wedges, and cylinders
- Move, scale, and rotate tools, and snapping to the grid
- Play-testing your world with your own avatar
- Saving your work and opening it again next class
Projects built:
- First playground: a small floating island with ramps, towers, and a jump course to test
Practice: Build a room with four different part shapes, then screenshot your avatar standing on top of it
Parts and Properties

    Topics:
      - The Properties panel: color, material, transparency, and reflectance
      - Anchored explained properly: why unanchored builds collapse when you press play
      - CanCollide: making walls you can walk through
      - Grouping parts into models and naming things in the Explorer
      - Duplicating and mirroring to build faster
      - Union and negate: cutting holes and making custom shapes
    Projects built:
      - Secret hideout: a house with a hidden walk-through wall and a glass floor over a drop
    Practice: Property scavenger hunt: recreate five reference parts exactly from their
    pictures
  Building a World
    Topics:
      - Terrain tools: sculpting hills, water, and caves
      - Spawn locations and where players appear
      - Planning an obby on paper before placing a single part
      - Difficulty curves: why the first jump should be easy
      - Lighting basics: time of day and atmosphere
      - Testing like a stranger: playing your own course as if you had never seen it
    Projects built:
      - Obby, part one: a ten-stage jump course across terrain you sculpted yourself
    Practice: Add three more stages to your obby, each slightly harder than the last, and
    have a family member test them
  Your First Script
    Topics:
      - What a Script object is and where it lives in the Explorer
      - print and the Output window: making the computer talk
      - Variables: naming a part in code and holding numbers and words
      - Changing a part's color and material from code instead of the mouse
      - Reading your first error message without panicking
      - Comments: leaving notes for future-you
    Projects built:
      - Magic brick: a part that changes color and prints a welcome message the moment the game
        starts
    Practice: Write three scripts that each change a different property of a part, and
    break one on purpose to read the error
    Assessment: Month 1 build check: a small world using anchoring, grouping, terrain, and
    one working script, walked through live

Month 2: Luau from Zero (Weeks 5-8)
  Variables and Values
    Topics:
      - Numbers, strings, and booleans in Luau
      - local and why every variable gets it
      - Doing maths in code: changing a part's size and position with numbers
      - String concatenation: building messages out of pieces
      - Vector3 gently introduced: positions in a 3D world need three numbers
      - nil: the most famous value in Roblox scripting
    Projects built:
      - Teleporting part: a script that moves a platform to a new position, then a script that grows
        it step by step
    Practice: Ten short scripts: resize, recolor, rename, and reposition parts using only
    variables and code
  Making Decisions
    Topics:
      - if, then, else: teaching a script to choose
      - Comparisons: equal, not equal, greater, and less
      - Booleans in action: checking if a part is anchored or transparent
      - elseif chains for more than two choices
      - and, or, not: combining conditions
      - Common trap: = for setting, == for asking
    Projects built:
      - Weather machine: a script that checks a variable and sets the world to day, night, or fog to
        match
    Practice: Decision drills: a password checker, a size judge that labels parts small or
    large, and a fix-the-broken-if puzzle
  Loops
    Topics:
      - while loops and task.wait: doing something forever, politely
      - Blinking, pulsing, and color-cycling parts
      - for loops: counting exactly, from disco floors to stairways
      - Building with loops: a staircase generated by code in one second
      - Loop variables inside the loop body
      - Infinite loops without a wait: what happens and how to stop it
    Projects built:
      - Disco floor: a grid of tiles that flash random colors forever
      - Instant staircase: a for loop that builds a climbable spiral stair
    Practice: Loop katas: a countdown announcer, a growing tower, and a part that fades out
    smoothly step by step

```
    Functions
      Topics:
        - Writing a function: name, parentheses, body, end
        - Parameters: one function that colors any part you hand it
        - Return values: functions that answer questions
        - Why functions matter: change the trap once, fix it everywhere
        - Organizing a script into small named functions
        - Calling functions from loops: the payoff moment
      Projects built:
        - Trap factory: one makeTrap function used to fill an obby stage with ten identical traps in
          seconds
      Practice: Refactor your disco floor and staircase scripts to use functions, then write
      one new function from a written spec
      Assessment: Month 2 skills check: live coding of three short Luau exercises using
      variables, if statements, loops, and one function

----------------------------------------------------------------
PHASE 2: EVENTS, PLAYERS, AND KEEPING SCORE (MONTHS 3-4, WEEKS 9-16)
----------------------------------------------------------------
Scripts so far ran on their own. Now they respond to players: touched events, health, doors,
and traps, then leaderboards, coins, and checkpoints, the machinery every real Roblox game
runs on.

  Month 3: Events and Player Interaction (Weeks 9-12)
    The Touched Event
      Topics:
        - Events: code that waits for something to happen
        - part.Touched:Connect and functions as event handlers
        - Finding out what touched the part
        - Humanoid and character models: how Roblox represents a player
        - The classic lava brick, built properly
        - Print-debugging events: seeing exactly when they fire
      Projects built:
        - Lava stage: an obby section with kill bricks that send careless players back to the start
      Practice: Build a safe-path puzzle: nine tiles, some deadly, some safe, with the
      pattern hidden until touched
    Changing the Player
      Topics:
        - Reading and changing Humanoid health
        - WalkSpeed and JumpPower: speed pads and super-jumps
        - Healing pads and damage zones
        - Resetting effects after a delay with task.wait
        - Being fair: telling the player what a pad does before it does it
        - Testing multiplayer effects with two test players in Studio
      Projects built:
        - Power-up alley: a course with speed strips, jump pads, a healing station, and one sneaky
          slow-down trap
      Practice: Design and script one original power-up of your own invention, then write two
      sentences on how it changes the game
    Doors, Buttons, and Debounce
      Topics:
        - Doors that open: toggling Transparency and CanCollide from a touch
        - Buttons on the floor and on the wall
        - The debounce problem: why your door fires twenty times per step
        - Fixing debounce with a boolean flag, the pattern every Roblox scripter knows
        - Timed doors that close themselves
        - Locked doors: combining a condition with an event
      Projects built:
        - Dungeon gate room: a button-operated gate with proper debounce and a door that only opens for
          players holding the right key part
      Practice: Break a working door by removing its debounce, watch the Output flood, then
      repair it and explain the fix in class
    Project Week: The Scripted Obby
      Topics:
        - Combining the month: lava, power-ups, doors, and traps in one course
        - Pacing a level: challenge, relief, surprise
        - Organizing scripts so each stage's code is findable
        - Playtesting a classmate's obby and writing useful feedback
        - Fixing the top two problems your testers found
        - Naming and Explorer hygiene before a project grows
      Projects built:
        - Obby 2.0: a fully scripted multi-stage obby with kill bricks, power-ups, at least one gate,
          and a victory area
      Practice: Run two playtests and log every place a tester got stuck or confused, then
      fix the worst one
      Assessment: Obby 2.0 demo: live playthrough plus a code walkthrough of one trap, one
      power-up, and one door

  Month 4: Score, Coins, and Checkpoints (Weeks 13-16)
```

Leaderboards
Topics:
- leaderstats: the folder-and-value pattern behind every Roblox leaderboard
- IntValue objects and setting them from code
- Players.PlayerAdded: running code when someone joins
- Awarding points from a touched event
- Server scripts vs local effects, kept simple: why the score lives on the server
- Watching the leaderboard update live in a two-player test
Projects built:
- Point course: a stage where every platform reached for the first time adds points to a live leaderboard
Practice: Add a second leaderboard stat of your choice, such as deaths or stages cleared, and make something update it
Collectibles
Topics:
- Coins that spin: rotating parts with a loop
- Collecting: touched event plus destroy, with debounce so one coin pays once
- Adding coins to leaderstats
- Spawning coins with a function instead of copy-paste
- Sound effects on pickup
- Respawning coins after a delay for endless play
Projects built:
- Coin rush: a field of spinning coins with pickup sounds, a live coin counter, and coins that respawn
Practice: Make a rare golden coin worth ten normal ones and place three somewhere genuinely hard to reach
Checkpoints
Topics:
- SpawnLocation objects and how respawning works
- Team-based checkpoints vs scripted checkpoints, and which we use
- Remembering each player's checkpoint in code
- Sending a respawned player to the right stage
- Checkpoint feedback: color change and sound when activated
- Why checkpoints make hard games fair instead of frustrating
Projects built:
- Checkpoint tower: a vertical obby where falling returns you to your highest activated checkpoint, not the bottom
Practice: Tune your tower: move one checkpoint to the fairest possible spot and defend the choice in one sentence
On-Screen Interfaces
Topics:
- ScreenGui, Frame, and TextLabel: putting words on the screen
- Updating a label from code: live coin and timer displays
- TextButton: a clickable button and its event
- Sizing and anchoring GUI so it works on different screens
- A welcome message that appears and fades
- Keeping GUI readable: contrast and size for real players
Projects built:
- Game HUD: an on-screen coin counter and stage display wired to real leaderstats, plus a welcome banner
Practice: Add one more GUI element of your choice, such as a best-height display or a hint button
Assessment: Month 4 systems check: coins, checkpoints, leaderboard, and HUD all working together in one place, demonstrated live

---
PHASE 3: REAL GAME MECHANICS AND THE PUBLISHED CAPSTONE (MONTHS 5-6, WEEKS 17-24)

Shops, timers, rounds, and polish turn a course into a game people replay. Then the capstone month: design, build, playtest, and publish an original game to Roblox.

Month 5: Mechanics That Make Games Fun (Weeks 17-20)
The Shop
Topics:
- Spending coins: checking if a player can afford something
- A shop GUI with buy buttons
- Selling upgrades: speed, jump, and a trail effect
- Subtracting coins and applying the upgrade in one safe function
- Saying no nicely: what happens when coins run short
- Balancing prices so the game stays challenging
Projects built:
- Upgrade shop: a working store where coins collected in-game buy speed and jump upgrades
Practice: Add a third item to your shop and set all three prices, then explain your pricing to the class
Timers and Rounds
Topics:
- A countdown clock on screen, driven by a loop
- Round structure: waiting, playing, and finishing
- Announcing round results to everyone

- Teleporting all players to the arena and back
- Resetting the world between rounds
- Why rounds make players say one more game

Projects built:
- Round machine: a lobby-and-arena game loop with a countdown, a 90-second round, and a winners announcement

Practice: Adjust your round length and lobby time after two playtests, and note what changed in player behavior

Sound, Light, and Polish

Topics:
- Background music and where it belongs
- Sound effects for wins, coins, and traps, used tastefully
- Particles: sparkles, fire, and confetti in the right doses
- Lighting mood: making a lava zone feel dangerous
- Victory effects that reward without annoying
- The polish pass: a checklist run on your own game

Projects built:
- Polish pass: music, pickup sounds, victory confetti, and lighting applied to the round machine game

Practice: Do a sound-only playtest: close your eyes while a classmate plays your game and list what you can tell is happening

Project Week: The Complete Mini-Game

Topics:
- Assembling the phase: rounds, shop, coins, HUD, and polish in one game
- Finding the fun: the one change that most improves your game
- Bug hunting with a checklist instead of luck
- Watching testers without helping them
- Writing simple in-game instructions new players actually read
- Preparing a 2-minute demo of your own game

Projects built:
- Complete mini-game: a round-based coin game with a shop, HUD, sound, and polish, tested by classmates

Practice: Fix the three most confusing things your playtesters hit, in order of how much they hurt the game

Assessment: Mini-game demo day: every student's game played live by a classmate, followed by questions about the code

Month 6: The Capstone, Published to Roblox (Weeks 21-24)

Designing Your Game

Topics:
- Choosing a game you can actually finish in three weeks
- The one-page game design sheet: theme, goal, mechanics, win condition
- Sketching the map on paper before Studio opens
- Listing features as must-have, nice-to-have, and dream
- Estimating honestly: what four class weeks can hold
- Getting the plan approved before building starts

Projects built:
- Approved capstone plan: a design sheet and map sketch reviewed one-on-one with the instructor

Practice: Build your capstone's map skeleton: terrain, main structures, and spawn, with no scripts yet

Build Sprint One: Core Mechanics

Topics:
- Building the must-have features first, smallest first
- Reusing your own code from earlier months, the honest shortcut
- Instructor code review: reading feedback and applying it
- Debugging on your own: reproduce, guess, test, fix
- Keeping the Explorer organized as the project grows
- Cutting a feature without shame when time asks for it

Projects built:
- Capstone core: the two or three mechanics that make your game itself, working end to end

Practice: Three 30-minute build sessions at home, each ending with one sentence on what now works that did not before

Build Sprint Two: Testing and Tuning

Topics:
- Full playtests with classmates as strangers
- Reading a playtest: stuck points, boredom points, and delight points
- Difficulty tuning: the numbers that change how a game feels
- Final GUI pass: title screen, instructions, and HUD
- Second code review and cleanup
- The finish line checklist: what done means for this game

Projects built:
- Feature-complete capstone: all must-have features in, tested by at least two classmates, tuned once

Practice: Run one family playtest and one classmate playtest, log every issue, fix the top two

Publish and Present

Topics:
- Publishing to Roblox from Studio, step by step, with a parent in the loop
- Game settings: name, description, icon, and who can play

- Privacy choices explained: public, friends-only, and what we recommend discussing at home
- The final bug sweep before going live
- Presenting your game: demo first, code second
- Where to go next: Python, more advanced game development, or a bigger Roblox project

Projects built:
- Published capstone: an original Roblox game live on the platform, presented to the class and family

Practice: Share your game link with three people outside the class and collect one piece of feedback from each

Assessment: Capstone presentation and code Q&A, plus course-completion certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- Floating-island playground built with Studio tools
- Ten-stage terrain obby with sculpted hills and caves
- Disco floor and instant staircase generated by loops
- Trap factory built from a single reusable function
- Obby 2.0 with kill bricks, power-ups, and scripted doors
- Coin rush field with spinning, respawning collectibles
- Checkpoint tower with fair respawn logic
- Round-based mini-game with shop, HUD, timer, and sound
- Published capstone: an original game live on Roblox

Total projects: 12+ finished builds and games, ending with one published to Roblox

SKILLS THE STUDENT WILL HAVE AT THE END

- Roblox Studio fluency: parts, properties, terrain, models, and lighting
- Luau fundamentals: variables, conditionals, loops, functions, and events written by hand
- Game systems: leaderboards, collectibles, checkpoints, shops, timers, and rounds
- GUI building: on-screen counters, buttons, and menus wired to live game data
- Debugging habits: reading errors, print-debugging, and the debounce pattern
- Project skills: scoping a game, playtesting honestly, and presenting your own work

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, building and scripting along with the instructor
Practice: 2-3 hours of building challenges and script exercises between classes
Review: Homework builds reviewed with written feedback; stuck points reopened in the next class
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so nobody stays stuck until the next session
Progress updates: Regular progress notes to parents: what was covered, what was built, what needs practice

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Our Python for kids course, where the logic learned in Luau transfers directly to the most-used language in the world
- Advanced game development courses, carrying over events, game loops, and level design instincts
- Bigger Roblox projects: many students keep building and publishing on their own after the course
- School coding clubs and game jams with a finished, published game already in hand
- Our JavaScript or app development tracks for students who want to build beyond gaming

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course run by Roblox?
A: No. Modern Age Coders is an independent coding school, and this course is not affiliated with or endorsed by Roblox Corporation. We teach children to build games using Roblox Studio, the free development tool that Roblox makes available to everyone, the same way a Python course teaches Python without being run by the Python Software Foundation.

Q: Is Roblox safe for my child? How do you handle safety in class?
A: Class time is spent in Roblox Studio, which is a development tool rather than the social game: students build and test their own worlds, not chat with strangers. We still recommend parents set up Roblox's parental controls on the child's account, including chat restrictions, and we walk parents through those settings on request. Publishing the capstone game is done together with a parent, and the privacy setting (public or friends-only) is the family's decision.

Q: Does my child need a paid Roblox account or Robux?
A: No. A free Roblox account is all that is required, for the entire course. Roblox Studio is free to download and use, and publishing a game to Roblox costs nothing. We never ask students to buy Robux, and no course project depends on any paid item.

Q: What exactly will my child learn to code?
A: Luau, the scripting language every Roblox game actually runs on. It is a real programming language (a Roblox-evolved version of Lua), so students learn genuine concepts: variables, conditionals, loops, functions, and events. Those ideas transfer directly to Python, JavaScript, and every language they meet afterwards; only the spelling changes.

Q: Is 8 years old too young for this? Is 14 too old?
A: The 8 to 14 range is deliberate. Eight-year-olds who read comfortably do well because month 1 is visual building and scripts arrive gradually, with typing kept short. Fourteen-year-olds are not bored because Luau is a real language and the later months involve genuine systems: leaderboards, shops, and round loops. Batches are grouped so students work alongside kids of similar age and pace.

Q: What computer do we need?
A: Roblox Studio does not run on phones, tablets, or most Chromebooks, so a proper computer is a requirement. If you are unsure whether your machine qualifies, tell us the model before the demo class and we will check.

Q: My child already plays Roblox for hours. Will this just mean more screen time?
A: It changes what the screen time is. Playing a game and building one exercise completely different muscles: planning, logic, debugging, and finishing something hard. Most parents tell us the course redirects hours the child was already spending, and that watching your own game get published beats another afternoon of playing someone else's.

Q: How is my child assessed, and is the certificate just for attending?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a friendly final assessment of everything learned. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Can we try a class before enrolling?
A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to sit in, and the demo is also where we confirm your computer runs Roblox Studio comfortably.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/roblox-game-coding-for-kids-lua-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 125 OF 137: SAT MATH 800 PREP: LIVE CLASSES FOR THE DIGITAL SAT
==========

Course link: https://learn.modernagecoders.com/courses/sat-math-prep-course
Subject area: Test Prep Mathematics
Designed for: High school
Who can join: Grades 10-12, all starting scores
Course length: 6 months (26 weeks)
Time needed: 2 live one-hour classes per week + weekly timed practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online SAT Math prep for the Digital SAT: Heart of Algebra, Advanced Math, Data Analysis, Desmos strategy and full adaptive mock cycles with a mentor.

In one line: The Digital SAT rewards thinkers. We build the thinker, then train the test.

ABOUT THE PROGRAMME

A 6-month live prep program for the Digital SAT's math section, taught 1-on-1 or in small groups by a dedicated mentor. The SAT math section rewards structure recognition and clean algebra far more than memorized tricks, so we teach the mathematics underneath first, linear and nonlinear equations, functions, ratios, data, then layer on adaptive-test strategy: module pacing, Desmos calculator fluency, and when to solve versus when to backsolve. Every student runs full timed mock cycles with question-by-question review, so score movement is measured, not hoped for.

WHAT MAKES THIS COURSE DIFFERENT

- Understanding-first: we repair the algebra underneath before drilling question types, because that is where 600-scorers become 750-scorers
- Digital SAT native: adaptive module strategy and built-in Desmos calculator workflows taught as first-class skills
- A dedicated mentor who tracks your error log across the whole program, not a rotating cast
- Full mock cycles under real timing with question-by-question review against the reasoning, not just the answer key
- One full interactive hour per class, twice a week, no 25-minute check-ins
- Honest score guidance: a diagnostic in week one and evidence-based targets, never invented score promises
- Small groups capped tight, or fully private 1-on-1 mentorship
- Every class recorded for revision before test day

THE LEARNING PATH, STEP BY STEP

Phase 1: Diagnostic + the algebra core: linear equations, systems, inequalities and the function idea, repaired to fluency
Phase 2: Advanced Math and Problem-Solving & Data: quadratics, exponentials, ratios, percentages and data interpretation
Phase 3: Test craft: adaptive pacing, Desmos mastery, full mock cycles and error-log-driven polishing to peak on test day

WHAT A STUDENT NEEDS BEFORE STARTING

Education: Grades 10-12 (or gap-year students retaking the SAT)
Math background: School algebra at any comfort level, the diagnostic finds the real starting point
Equipment: Computer with stable internet; the Digital SAT's Desmos calculator is built into practice tools
Time commitment: 2 live hours plus 2-3 practice hours weekly
English: Course taught in English
Motivation: A target test date, or our help choosing one

WHO THIS COURSE IS FOR

Score raisers: Students scoring 500-650 on practice tests who need the algebra core rebuilt, not more drills
Top end: Students at 700+ hunting the last 60-100 points through hard-module craft and error elimination
Strong at school: Teens with good school grades whose practice scores do not match, the classic craft gap
Planners: Grade 10 families starting early so the real test year is calm
Retakers: Students whose first attempt underperformed and who want a method, not more of the same

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE ALGEBRA CORE (WEEKS 1-8)

The Digital SAT's biggest domain is algebra. This phase diagnoses exactly where points are leaking and rebuilds linear equations, systems, inequalities and functions to reconstruction strength, the foundation every later question type stands on.

Weeks 1-2: Diagnostic + Linear Equations Rebuilt

Topics:
- Full timed diagnostic module with question-by-question review
- Personal error log opened: content gaps vs speed gaps vs careless patterns
- Linear equations and expressions from meaning, not moves
- Translating word problems into equations, the SAT's favorite disguise
- Slope and intercepts as rates and starting values in context
- Desmos introduction: graphing as a checking habit

Projects built:

- Personal score map: your exact domains, question types and target
- First timed mini-module under real Digital SAT pacing

Practice: Complete one timed linear equations mini set, log every miss in your error log as content, speed or careless, and redo the misses untimed.

Weeks 3-4: Systems and Inequalities

Topics:
- Systems solved three ways: substitution, elimination, Desmos intersection
- How many solutions? Parallel, intersecting and identical lines as ideas
- Linear inequalities and shaded regions read from the graph
- Systems word problems: mixtures, rates, break-even setups
- When solving is slower than testing answer choices, strategy honestly taught
- Timed reps: the 60-second system question

Projects built:
- Break-even mini-case solved by equation, table and graph
- Error-log review: your system-question mistakes classified and closed

Practice: Work a 15 question systems and inequalities set, solving at least five by Desmos intersection, and file each error in your log with its fix.

Weeks 5-6: Functions as Machines

Topics:
- Function notation read as a sentence: f(x) is an output, not a mystery
- Evaluating, composing and interpreting functions in context
- Graph reading: intercepts, increasing/decreasing, transformations
- Linear function word problems and modeling
- Function questions in the calculator module: Desmos-first workflows
- Timed reps: function questions across both modules

Projects built:
- Function-transformation exploration built live in Desmos
- Mixed timed set: algebra domain at full pace

Practice: Finish a mixed function notation problem set, then rebuild two missed questions in Desmos to see what the graph was telling you.

Weeks 7-8: Algebra Domain Checkpoint

Topics:
- Full algebra-domain timed section under adaptive conditions
- Review ritual taught: why each miss happened, what changes next time
- Absolute value and compound inequalities, the fringe cases
- Careless-error engineering: writing habits that stop point leaks
- Module-1 pacing plan drafted against your speed data

Projects built:
- Algebra checkpoint mock + personal pacing plan v1
- Updated error log with closed vs open gap list

Practice: Take one timed algebra domain section and write a one line cause and correction for every dropped point before the review class.

PHASE 2: ADVANCED MATH + PROBLEM-SOLVING & DATA (WEEKS 9-18)

The domains that separate good scores from great ones: quadratics and exponentials taught as ideas, then the ratio-percent-data cluster where careless points go to die. Every topic is drilled at real SAT difficulty tiers, including the hard-module versions.

Weeks 9-11: Quadratics and Nonlinear Equations

Topics:
- Quadratics four ways: factoring, completing the square, formula, Desmos
- The discriminant as an idea: how many solutions and why the SAT asks
- Vertex form and parabola word problems (projectiles, revenue, area)
- Nonlinear systems: line meets parabola, read from the graph
- Exponent rules rebuilt from meaning; rational exponents and radicals
- Hard-module quadratic questions: recognizing the structure in costume

Projects built:
- Projectile/revenue modeling mini-case solved by vertex reasoning
- Timed hard-module quadratic set with full review

Practice: Drill a 12 question hard module quadratic set, solving each question twice: once by algebra and once by the fastest Desmos route.

Weeks 12-14: Exponential Growth, Polynomials and Rational Expressions

Topics:
- Linear vs exponential growth: the comparison the SAT loves to test
- Exponential word problems: percent growth/decay, doubling, interest
- Polynomial arithmetic, zeros and graph behavior
- Rational expressions and equations without fear
- Isolating variables in formulas, the classic layup question
- Equivalent-expression questions: the algebra-fluency tax

Projects built:
- Growth-model comparison case: same data, linear vs exponential fits
- Advanced Math domain checkpoint under timing

Practice: Complete a percent growth and decay problem set plus five isolate the variable

questions, keeping each layup question under a minute.

Weeks 15-18: Problem-Solving & Data Analysis
Topics:
- Ratio and proportion reasoning: the choose-the-easy-comparison skill
- Percent problems in every SAT costume: change, reverse, repeated
- Unit conversion and rates with dimensional discipline
- Reading tables, scatterplots, histograms and box plots fast
- Line of best fit, slope interpretation and prediction questions
- Mean, median, range and standard deviation as concepts
- Margin of error and study-design questions, what the SAT actually asks
- Two-way tables and conditional probability without formulas
Projects built:
- Data-interpretation sprint: 20 real chart questions under time
- Full mixed-domain mock: first complete Digital SAT math section
Practice: Run a 20 question ratio, percent and data interpretation sprint under time and classify every miss before class.

PHASE 3: ADAPTIVE TEST CRAFT + MOCK CYCLES (WEEKS 19-26)

Content becomes score in this phase: adaptive module strategy, Desmos speed workflows, geometry-and-trig completion, and a full mock cycle rhythm, test, review, repair, retest, that peaks the week of the real exam.

Weeks 19-20: Geometry and Trigonometry Domain
Topics:
- Area, volume and the reference-sheet formulas used intelligently
- Triangles: similarity, right-triangle trig, special right triangles
- Circles: arcs, sectors, and the circle equation
- Angle chasing with parallel lines and polygons
- Coordinate geometry: distance, midpoint, and geometric reasoning on the grid
- Trig on the SAT: exactly what is tested, nothing more
Projects built:
- Geometry domain checkpoint under timing
- Personal formula-and-recognition sheet, built not copied
Practice: Finish a geometry domain set using your own recognition sheet wherever possible, starring any formula you still had to look up.

Weeks 21-23: Adaptive Strategy and Desmos Mastery
Topics:
- How Digital SAT adaptivity works and what module 1 accuracy is worth
- Pacing plans per module built from your own timing data
- Desmos power workflows: solving, systems, regressions, tables at speed
- Solve vs backsolve vs plug-in: choosing the fastest honest route
- Flag-and-return discipline; the two-pass module method
- Guessing strategy and the no-blank rule, applied calmly
Projects built:
- Desmos speed drills: 15 questions where the calculator wins
- Strategy card: your personal decision rules for module 2
Practice: Do the 15 question Desmos speed drill, record your per question times and flag anything over 90 seconds for strategy review.

Weeks 24-26: Full Mock Cycles and Peak Week
Topics:
- Full adaptive mocks under exact timing, reviewed question by question
- Error-log closure: every recurring mistake gets a named fix
- Score-trend review with honest final-target calibration
- Peak-week plan: what to practice, what to rest, sleep and logistics
- Test-day routine: warm-up set, timing checkpoints, reset ritual after hard questions
Projects built:
- Two to three full mock cycles with documented score progression
- Written test-day plan, rehearsed twice before the real thing
Practice: Sit one full adaptive mock under exact timing this week and bring the question by question review with your named fixes.

PROJECTS THE STUDENT BUILDS AND KEEPS

- Personal score map and error log maintained across all 6 months
- Desmos workflow library built by the student, question type by question type
- Two to three full adaptive mock cycles with documented score progression
- Personal pacing plans for both modules, built from real timing data
- Written test-day plan rehearsed before the real exam

Total projects: 5+ prep artifacts including full mock cycles

SKILLS THE STUDENT WILL HAVE AT THE END

- Linear equations, systems and inequalities at SAT speed

- Functions, quadratics and exponentials as ideas
- Ratio, percent and data-analysis fluency
- SAT geometry and trigonometry domain
- Desmos calculator power workflows
- Adaptive module pacing and two-pass strategy
- Error-log discipline and careless-point engineering
- Test-day routine and calm under the clock

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 one-hour live classes with your mentor
Timed practice: 1-2 timed sets or module sections per week
Review: Every practice set reviewed for reasoning, not just answers
Total per week: About 4-5 focused hours
Homework: A problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Live sessions: 8 live one-hour classes every month, 2 per week
Mentorship: One dedicated mentor across the whole program
Recordings: Every class recorded for revision
Progress tracking: Error log + mock-score progression shared with parents
Flexibility: Slots move around school exams and family plans

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies mastery, not attendance

WHERE THIS CAN LEAD

- A math score that strengthens selective-college applications
- AP Calculus and AP Statistics readiness, built on real algebra fluency
- Stronger merit-scholarship positioning at score-aware universities
- Ready for AP Calculus Exam Prep or the College Mathematics Masterclass

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Is this course for the Digital SAT?
A: Yes, the course is built for the Digital SAT format: two adaptive math modules, the built-in Desmos graphing calculator, and current question styles. Module strategy and Desmos fluency are taught as first-class skills alongside the mathematics, and we verify current test details against College Board materials for your test date.

Q: What score improvement can we expect?
A: We will not invent a number, improvement depends on the starting score, weeks available and practice done between classes. What we promise instead: a real diagnostic in week one, a plan that attacks your highest-value gaps first, and a documented mock-score progression you can watch. Students who complete the mock cycles see their trend line clearly before test day.

Q: My teen scores well in school math but low on SAT practice tests. Why?
A: This is the most common profile we see. School rewards showing your work over weeks; the SAT rewards structure recognition under a clock. The gap is usually speed, question-reading and strategy rather than missing content, all trainable. The diagnostic separates content gaps from craft gaps in the first week.

Q: When should SAT math prep start?
A: The full program is 6 months, which comfortably fits a spring test from an autumn start. Starting closer to the test still works, the mentor compresses toward your highest-value domains and mock cycles, but more runway means the algebra core gets properly rebuilt rather than patched.

Q: How is this different from a prep-book or app-based course?
A: Books and apps give practice; they cannot watch your teen think. A live mentor catches the broken step inside the work, the sign error, the misread, the slow route, and fixes the habit, not just the answer. That is why our classes are a full interactive hour with your teen solving and explaining aloud, twice a week.

Q: Do you cover the reading and writing section too?
A: This course is deliberately math-only, depth over bundling. Math is where a specialist mentor moves scores fastest, and where our maths-and-coding teaching method gives a real edge. Families wanting verbal prep can run it in parallel elsewhere without conflict.

Q: How am I assessed, and is the certificate just for showing up?

  A: No. A problem set follows every class with written feedback, monthly reviews deliberately
  mix new material with earlier topics so nothing is forgotten, and the course ends with a full
  mock SAT-math paper as the final assessment. Passing earns the certificate; falling short
  earns focused revision and a free retest. We measure real understanding, not attendance.

  Q: Can we try it before paying?
  A: Yes. Every student starts with a free live demo class that doubles as the diagnostic, a
  real lesson plus an honest read on the starting point and target, with no card details
  required.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/sat-math-prep-course
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 126 OF 137: SCRATCH CODING FOR KIDS: ZERO TO REAL GAMES IN 3 MONTHS
==========

Course link: https://learn.modernagecoders.com/courses/scratch-programming-complete-course
Subject area: Kids Coding
Designed for: Kids
Who can join: Complete beginner, ages 6 to 12
Course length: 3-4 months (12-16 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one
  live class a week, four a month; one to one outside India two live classes a week, eight a
  month. Each class is one hour.

WHAT THIS COURSE IS

  Scratch coding course for kids aged 6-12: live online classes from first block to real games
  and animations, 35+ projects in 3-4 months on free MIT Scratch.

  In one line: The best first coding language ever built, mastered properly: from the first dragged block to real games your child designed, in three focused months.

ABOUT THE PROGRAMME

  Scratch, built by MIT, is where millions of children write their first real program, and for
  good reason: logic is learned by snapping colorful blocks together, and the results move,
  sing and play back immediately. This live online course takes children aged 6 to 12 from zero
  to genuinely capable: 35+ projects across three months, from first animations to complete
  games with scores, levels, clones and multi-sprite coordination. The pace is honest and
  kid-sized: two live classes a week plus 2 to 3 hours of playful practice completes the course
  in 3 months, and children who want more time take up to 4, the batch adapts. Every week ends
  with something to show the family; every month ends with a friendly mixed challenge that
  quietly revises earlier skills; and the course ends with a final project exam, a game demo
  plus a short viva, that decides the certificate. Real skills, real games, real pride.

WHAT MAKES THIS COURSE DIFFERENT

  - Every single week ships projects: 35+ games, animations and stories across the course, built
    by the child, not the teacher
  - Real Scratch depth: variables, lists, clones and broadcasts, the concepts that make later
    Python feel familiar
  - Kid-sized honest pace: 3 months at 2 classes a week, with a 4th month in hand for children
    who want more practice time
  - Monthly mixed challenges revise earlier skills on purpose, so nothing fades between weeks
  - A final project exam decides the certificate: a game demo plus a friendly viva, with a free
    retest after revision if needed
  - Small live batches capped at 10, grouped by age, with mini-batch and 1-on-1 options

THE LEARNING PATH, STEP BY STEP

  Month 1: Foundations (Weeks 1-4): sprites, motion, sounds, events and first games, plus
  creative pen art
  Month 2: Real game mechanics (Weeks 5-8): variables and scores, lists, clones, and
  multi-sprite games with broadcasts
  Month 3: Mastery (Weeks 9-12): extensions and AI blocks, physics, polish, and the final
  capstone game presented at demo day

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Scratch was invented exactly for this start
Reading: Basic reading ability to follow on-screen labels
Equipment: A computer or laptop with a stable internet connection; Scratch is free and runs in the browser
Age: 6 to 12. Batches are grouped by age so pace and examples fit the room
Time commitment: 2 live classes plus 2-3 hours of playful practice a week

WHO THIS COURSE IS FOR

Complete beginners: Kids aged 6-14 with zero coding experience
Creative kids: Children who love games, stories, art, or music
Visual learners: Kids who learn best through visual, hands-on activities
Future coders: Children interested in eventually learning Python or JavaScript
Homeschoolers: Families looking for structured coding education

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

MONTH 1: SCRATCH FOUNDATIONS & BASIC GAME DEVELOPMENT

Master Scratch basics, create animations, and build your first games.

Introduction to Scratch & Basic Motion
Topics:
- What is programming? Why learn to code?
- Introduction to MIT Scratch platform
- Creating your Scratch account
- Scratch interface tour: Stage, Sprites, Blocks
- Understanding coordinates (x, y) system
- Sprite selection and management
- Basic motion blocks: move, turn, go to
- Using the green flag and stop button
- Saving and naming projects
- Sprite costumes and switching
- Stage backdrops basics
- First animation: making sprites move
Projects built:
- {'name': 'Hello World Animation', 'description': 'Sprite introduces itself and moves around', 'skills': 'Basic motion, say blocks, timing'}
- {'name': 'Aquarium Scene', 'description': 'Fish swimming with glide blocks', 'skills': 'Glide motion, multiple sprites, backdrop'}
- {'name': 'Dance Party', 'description': 'Characters dancing with costume changes', 'skills': 'Costume switching, rhythm, coordination'}
- {'name': 'Maze Navigator', 'description': 'Move sprite through simple maze with arrow keys', 'skills': 'Keyboard control, precise movement'}
Practice: Make your sprite dance across the stage with move and glide blocks, visiting all four corners using x and y coordinates.

Looks, Sound, and Interactivity
Topics:
- Advanced looks blocks and effects
- Graphic effects: color, fisheye, whirl, pixelate
- Size changes and animations
- Show and hide sprites strategically
- Layer ordering with go to front/back
- Adding sounds from library
- Recording custom sounds
- Sound effects and volume control
- Playing notes and creating melodies
- Tempo and beat in projects
- Mouse interaction and following
- Creating buttons and click events
Projects built:
- {'name': 'Interactive Music Maker', 'description': 'Piano with visual effects', 'skills': 'Sound blocks, mouse interaction, effects'}
- {'name': 'Animated Story Book', 'description': 'Story with sound effects and narration', 'skills': 'Timing, sound synchronization, scene changes'}
- {'name': 'Dress Up Game', 'description': 'Click to change character outfits', 'skills': 'Mouse clicks, costume logic, layering'}
- {'name': 'Magic Show', 'description': 'Sprites appear/disappear with effects', 'skills': 'Show/hide, graphic effects, timing'}
Practice: Give your sprite a costume change show that uses three graphic effects and plays a sound at each transformation.

Control Flow and Basic Logic
Topics:

```
      - Understanding forever loops
      - Repeat loops with specific counts
      - If-then conditional statements
      - If-then-else decision making
      - Wait blocks and timing control
      - Repeat until conditions
      - Stop commands for sprites and scripts
      - Combining multiple conditions
      - Nested loops understanding
      - Creating smooth animations with loops
      - Basic collision detection
      - Game over and winning conditions
    Projects built:
      - {'name': 'Catch the Falling Stars', 'description': 'Catch objects with increasing speed',
        'skills': 'Forever loops, collision detection, scoring'}
      - {'name': 'Traffic Light Simulator', 'description': 'Automated traffic control system',
        'skills': 'Repeat loops, timing, state management'}
      - {'name': 'Platform Jumper', 'description': 'Basic platformer with gravity', 'skills':
        'Conditional logic, physics simulation'}
      - {'name': 'Quiz Game', 'description': 'Multiple choice questions with scoring', 'skills':
        'If-then-else, user input, logic flow'}
    Practice: Build a guess the door scene where if then else blocks decide what happens
    behind each of three doors.

  Pen Extension and Creative Drawing
    Topics:
      - Adding the Pen extension
      - Pen down and pen up commands
      - Drawing lines and shapes
      - Changing pen color and shade
      - Pen size adjustments
      - Creating geometric patterns
      - Stamps and creating trails
      - Drawing with mouse control
      - Mathematical art and fractals
      - Spirograph patterns
      - Clearing and managing drawings
      - Creating drawing applications
    Projects built:
      - {'name': 'Etch-a-Sketch', 'description': 'Drawing app with keyboard control', 'skills': 'Pen
        control, user input, clearing'}
      - {'name': 'Geometric Art Generator', 'description': 'Create mathematical patterns', 'skills':
        'Loops with pen, angles, repetition'}
      - {'name': 'Rainbow Spiral Maker', 'description': 'Colorful spiral patterns', 'skills': 'Color
        changes, mathematical spirals'}
      - {'name': 'Signature Pad', 'description': 'Mouse-controlled drawing', 'skills': 'Mouse
        tracking, pen following'}
    Practice: Draw a rainbow spiral with the pen extension by changing the pen color inside a
    repeat loop.
    Assessment: Month 1 check: build a small interactive scene live, plus a fun take-home
    challenge mixing everything from month 1

----------------------------------------------------------------
MONTH 2: ADVANCED SCRATCH TECHNIQUES
----------------------------------------------------------------
Master variables, lists, clones, and build complex games.

  Variables and Score Systems
    Topics:
      - What are variables? Real-world examples
      - Creating variables for all sprites vs. this sprite only
      - Setting and changing variable values
      - Displaying variables on stage
      - Creating score systems
      - Lives and health systems
      - Timers and countdowns
      - High score tracking
      - Currency systems for games
      - Variable-based animations
      - Random number generation
      - Mathematical operations with variables
    Projects built:
      - {'name': 'Cookie Clicker Game', 'description': 'Incremental game with upgrades', 'skills':
        'Variables, click counting, cost systems'}
      - {'name': 'Racing Game with Laps', 'description': 'Track laps and best times', 'skills':
        'Timer variables, lap counting, records'}
      - {'name': 'RPG Character Stats', 'description': 'Health, mana, experience system', 'skills':
        'Multiple variables, stat management'}
      - {'name': 'Math Challenge Game', 'description': 'Timed math problems with scoring', 'skills':
```

'Random numbers, calculations, timing'}
Practice: Create a clicker game where a score variable rises with every click and the sprite celebrates at ten points.

Lists and Data Management
Topics:
- Understanding lists (arrays)
- Creating and naming lists
- Adding items to lists
- Deleting items from lists
- Inserting at specific positions
- Replacing list items
- Finding items in lists
- List length and iteration
- Using lists for inventories
- Leaderboards and high scores
- Question banks for quizzes
- Shopping lists and collections
Projects built:
- {'name': 'Memory Card Game', 'description': 'Match pairs using lists', 'skills': 'List shuffling, pair matching, memory'}
- {'name': 'Word Guessing Game', 'description': 'Hangman-style with word lists', 'skills': 'String manipulation, list iteration'}
- {'name': 'Inventory System RPG', 'description': 'Collect and manage items', 'skills': 'Dynamic lists, item management'}
- {'name': 'Quiz Bowl Game', 'description': 'Questions from lists', 'skills': 'Random selection, list traversal'}
Practice: Make a favorites program that stores five favorite foods in a list and says a random one when the space key is pressed.

Clones and Advanced Game Mechanics
Topics:
- Understanding clones concept
- Creating clones of sprites
- When I start as a clone
- Clone-specific behaviors
- Deleting clones properly
- Managing multiple clones
- Bullet systems in games
- Enemy spawning mechanics
- Particle effects with clones
- Power-ups and collectibles
- Clone collision detection
- Performance optimization
Projects built:
- {'name': 'Space Shooter', 'description': 'Shoot bullets at enemy clones', 'skills': 'Clone bullets, enemy waves, collision'}
- {'name': 'Tower Defense Game', 'description': 'Defend against clone enemies', 'skills': 'Path following, tower placement, waves'}
- {'name': 'Fireworks Simulator', 'description': 'Particle effects with clones', 'skills': 'Visual effects, physics, timing'}
- {'name': 'Asteroid Field', 'description': 'Navigate through clone obstacles', 'skills': 'Random spawning, difficulty scaling'}
Practice: Build a falling snowflake shower using clones that appear at random x positions and delete themselves at the bottom.

Broadcast Messages and Multi-Sprite Coordination
Topics:
- Understanding broadcast messages
- When I receive message
- Broadcast and wait
- Scene management with broadcasts
- Menu systems and navigation
- Coordinating multiple sprites
- Turn-based game mechanics
- Cutscenes and dialogues
- Game state management
- Pause and resume functionality
- Level transitions
- Achievement notifications
Projects built:
- {'name': 'RPG with Dialogue System', 'description': 'Story-driven game with conversations', 'skills': 'Message coordination, dialogue trees'}
- {'name': 'Multi-Level Platformer', 'description': 'Complete game with level progression', 'skills': 'Scene management, state tracking'}
- {'name': 'Turn-Based Battle System', 'description': 'Pokemon-style combat', 'skills': 'Turn management, action queuing'}
- {'name': 'Escape Room Puzzle', 'description': 'Multiple interconnected puzzles', 'skills': 'Puzzle logic, item interaction'}

Practice: Create a two scene story where a broadcast message switches the backdrop and cues the next character to speak.
Assessment: Month 2 check: a mini game built live using variables and clones, with challenge questions reaching back to month 1

MONTH 3: MASTERY AND ADVANCED PROJECTS

Create professional games, master extensions, and prepare for text coding.

Extensions and AI Blocks
Topics:
- Music extension advanced features
- Creating dynamic soundtracks
- Text to Speech capabilities
- Language translation features
- Video Sensing for motion detection
- Creating motion-controlled games
- Using webcam for interaction
- Machine Learning extension intro
- Creating AI-powered projects
- Optional hardware fun: micro:bit and Makey Makey, for families who have one at home

Projects built:
- {'name': 'Motion-Controlled Game', 'description': 'Control with body movements', 'skills': 'Video sensing, motion tracking'}
- {'name': 'Multi-Language Quiz', 'description': 'Quiz with translation features', 'skills': 'Translation, text-to-speech'}
- {'name': 'AI Pet Game', 'description': 'Pet that responds to actions', 'skills': 'Machine learning basics, interaction'}
- {'name': 'Musical Composition Tool', 'description': 'Create and play music', 'skills': 'Music extension mastery'}

Practice: Use the Text to Speech extension to make a talking pet that greets you in two different languages.

Physics and Simulation
Topics:
- Implementing gravity
- Jump mechanics and physics
- Velocity and acceleration
- Collision detection advanced
- Bouncing and elasticity
- Friction simulation
- Pendulum motion
- Projectile motion
- Water and fluid effects
- Particle systems advanced
- Realistic movement patterns
- Physics-based puzzles

Projects built:
- {'name': 'Angry Birds Clone', 'description': 'Physics-based puzzle game', 'skills': 'Projectile physics, destruction'}
- {'name': 'Pinball Machine', 'description': 'Realistic ball physics', 'skills': 'Bouncing, flippers, gravity'}
- {'name': 'Solar System Simulator', 'description': 'Orbital mechanics', 'skills': 'Circular motion, gravity simulation'}
- {'name': 'Fluid Dynamics Demo', 'description': 'Water and particle effects', 'skills': 'Particle systems, flow simulation'}

Practice: Give a character a real jump using a velocity variable, gravity and a landing check on a platform.

Optimization and Professional Polish
Topics:
- Performance optimization techniques
- Reducing lag and improving FPS
- Efficient clone management
- Sprite and backdrop optimization
- Code organization and comments
- Creating custom blocks (functions)
- Debugging techniques
- Error handling strategies
- Testing and quality assurance
- User interface design
- Menu and navigation systems
- Settings and options menus

Projects built:
- {'name': 'Professional Game UI', 'description': 'Complete menu system', 'skills': 'Navigation, settings, polish'}
- {'name': 'Optimized Particle System', 'description': 'Efficient visual effects', 'skills': 'Performance, visual quality'}

```
        - {'name': 'Debugger Tool', 'description': 'In-game debugging system', 'skills': 'Testing,
          monitoring, logging'}
        - {'name': 'Modular Game Engine', 'description': 'Reusable game components', 'skills': 'Custom
          blocks, organization'}
      Practice: Tidy up one of your older projects by adding comments to three scripts and
      merging any blocks that repeat.

    Final Projects and Portfolio
      Topics:
        - Planning large projects
        - Game design documentation
        - Project management basics
        - Version control concepts
        - Sharing and remixing ethics
        - Scratch community guidelines
        - Creating tutorials for others
        - Portfolio presentation
        - Transitioning to text coding
        - JavaScript similarities
        - Python connections
        - Future learning paths
      Projects built:
        - THE CAPSTONE: an original game designed and built by the child, with levels, scoring and a
          polished start screen
        - Classic picks: an RPG adventure with quests, an educational quiz world, or an arcade game
          with increasing difficulty
        - A portfolio page collecting the course's best 10 projects
      Practice: Write a one page plan for your final game with its goal, its sprites and three
      rules, then build the title screen.
      Assessment: FINAL EXAM: capstone game demo plus a friendly viva and a mixed quiz drawn
      from the whole course. Passing earns the certificate; a child who falls short gets
      focused revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Month 1 (Weeks 1-4): 12+ projects: animations, interactive stories, pen art and first games
  - Month 2 (Weeks 5-8): 12+ games: score systems, list-powered quizzes, clone effects and
    multi-sprite worlds
  - Month 3 (Weeks 9-12): 10+ advanced builds: AI-block experiments, physics simulations, and the
    polished capstone

  Total projects: 35+ Scratch projects plus an original capstone game

SKILLS THE STUDENT WILL HAVE AT THE END

  - Scratch Basics: sprites, motion, looks, sound, pen blocks, backdrops
  - Programming Logic: loops, conditionals, events, variables, operators
  - Data Structures: lists, data management, cloud variables
  - Game Development: score systems, collision detection, clones, physics
  - Animation: character animation, scene transitions, storytelling
  - Advanced: extensions (music, video sensing, text-to-speech), optimization
  - Professional Skills: debugging, code organization, project planning

HOW A TYPICAL WEEK IS SPLIT

  Live classes: 2 live one-hour classes per week, building along with the teacher
  Homework: A small homework build after every class, and a mixed-review challenge at the end
    of every month
  Practice: 2-3 hours of playful practice between classes
  Review: Every project gets teacher feedback; wobbly concepts are reopened in class

SUPPORT INCLUDED

  Doubt support: WhatsApp doubt support between classes, so no child stays stuck and
    discouraged
  Progress updates: Regular progress notes to parents: what was built, what comes next, and how
    to encourage practice at home

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final exam, a capstone
    demo plus a friendly viva. A child who falls short gets focused revision and a free retest:
    the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

  - Junior Game Developer
  - Creative Coder
  - Animation Creator
```

  - Interactive Media Designer
  - Ready for Python/JavaScript transition

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What will my child actually create in this Scratch course?
  A: Over 3 months your child builds 35+ real projects in Scratch: animations and interactive stories in month 1, genuine games with scores, clones and multi-sprite logic in month 2, and physics experiments, AI-block projects and a polished original capstone game in month 3. Every project is the child's own work, demonstrated at home the same week it is built.

  Q: What age is this Scratch course right for?
  A: The course is designed for ages 6 to 12 and assumes zero coding experience. All your child needs is basic reading ability to follow on-screen labels; lessons are visual and hands-on, which suits younger learners especially well. Batches are grouped by age, so a 6-year-old is never racing an 11-year-old.

  Q: How long does it take a child to learn Scratch?
  A: To real capability, games with scores, levels, clones and logic, it takes most children 3 months at 2 live classes a week plus a little playful practice. The course keeps a 4th month in hand for children who want more repetition, with no rushing and no penalty. The very first animation happens in class one; the pride starts immediately.

  Q: How is my child assessed? Is the certificate just for attending?
  A: No, and it is all gentle: classwork in every session, a small homework build after every class, and a monthly challenge that mixes new blocks with earlier ones so skills compound instead of fading. The course ends with a final exam, a capstone demo plus a friendly viva. Passing earns the certificate; a child who falls short gets focused revision and a free retest. Real knowledge, real projects, no empty badges.

  Q: Does my child need any coding background before joining?
  A: None at all. Scratch was created by MIT precisely for first-time coders: children snap colorful blocks together instead of typing code, so there is no syntax to memorize. The course starts from the very first block.

  Q: What comes after Scratch?
  A: Scratch builds the computational thinking that makes text-based languages far easier later. The course ends with a guided transition path toward typed code, and most students continue into our Python or web development courses when they are ready.

  Q: Can we try a free class before enrolling?
  A: Yes, every child can take a free demo class first, with no commitment of any kind. Book a slot at learn.modernagecoders.com/contact, and you are welcome to sit in and watch; you can also reach us on WhatsApp at +91 91233 66161 with any questions. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or pre-recorded?
  A: All classes are live 1-hour online sessions taught by a real teacher; we never use pre-recorded videos. Batches are small and interactive, so your child builds projects during class while the teacher watches and helps in real time.

  Q: Does my child receive a certificate at the end?
  A: Yes, and it is earned, not handed out: the certificate is awarded on passing the final exam, a capstone game demo plus a friendly viva and a short mixed quiz, with a free retest after revision if the first attempt falls short. Alongside it stands the portfolio of 35+ real Scratch projects, which for most kids is the part they are proudest of.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/scratch-programming-complete-course
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 127 OF 137: STATISTICS & PROBABILITY: DATA TO HYPOTHESIS TESTING
==========

Course link: https://learn.modernagecoders.com/courses/statistics-probability-maths-course
Subject area: Data Science & Analytics Mathematics
Designed for: College students
Who can join: College students and working professionals; no programming required
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 3-4 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Online statistics and probability course for college and professionals: descriptive stats, distributions, hypothesis testing and regression on real data.

In one line: The subject that decides whether a claim is evidence or noise, taught on real data from the first week.

ABOUT THE PROGRAMME

Statistics is the working mathematics of almost every field that touches data: business, medicine, psychology, economics, engineering, machine learning. It is also routinely taught backwards, as formula memorisation first and meaning never. This 6-month live course teaches it the other way. Months 1 and 2 build the two foundations: describing data honestly, and probability as the grammar of uncertainty, including conditional probability and Bayes' theorem worked through real cases like medical test results. Months 3 and 4 cover the distributions that model the world, then sampling, the central limit theorem seen through simulation rather than assertion, and confidence intervals interpreted the way they actually work. Month 5 is hypothesis testing done properly: t-tests, proportion tests and chi-square, with equal attention to what p-values do not say. Month 6 covers correlation and regression, then a capstone: a full analysis of a real dataset of your choosing, written up with methods and limitations. Every technique is practised on real data in a spreadsheet, and an optional Python track runs alongside for those who want it. Neither is required knowledge coming in; one of them becomes a working skill going out.

WHAT MAKES THIS COURSE DIFFERENT

- Real datasets from week 1: public health, sports, economics and business data, not textbook tables invented to make the arithmetic clean
- Spreadsheet-first with an optional Python track, so non-programmers finish everything and programmers get the pandas version of each technique
- Probability taught through consequential examples: false positives in medical testing, base-rate mistakes, and why intuition fails on conditional probability
- The central limit theorem demonstrated by simulation you run yourself, because a theorem you have watched happen is one you actually trust
- Honest inference: what a p-value is and is not, why statistical significance is not importance, and how p-hacking happens
- A capstone analysis of a dataset you choose, written up with methods and limitations like a piece of real analytical work

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundations: describing data with the right summaries and charts, then probability, conditional probability, Bayes' theorem and random variables
Phase 2: Models and estimation: binomial, Poisson and normal distributions, sampling and the central limit theorem, and confidence intervals done honestly
Phase 3: Inference and the capstone: hypothesis tests for means, proportions and categories, correlation and regression, then a full real-data analysis of your own

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Comfortable school algebra: rearranging formulas and working with fractions and percentages. No calculus is needed anywhere in the course
Programming: None. Everything can be completed in a spreadsheet; the Python track is optional and taught from zero for those who choose it
Equipment: A computer with a spreadsheet application (Excel or Google Sheets) and a stable internet connection
Audience: Built for college students and working professionals; motivated final-year school students who clear the diagnostic are welcome

WHO THIS COURSE IS FOR

College students: Students in commerce, economics, psychology, biology or engineering whose degrees assume statistics that was never properly taught
Working professionals: Analysts, marketers, product managers and operations people who face dashboards and A/B tests daily and want to stop nodding along
Data science aspirants: People headed for data science or machine learning who know the statistics half is the half they are missing
Researchers: Postgraduate students and early researchers who need to design studies and read papers with methods sections
Non programmers: Anyone who wants rigorous data skills without being forced through a programming course to get them

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: DESCRIBING DATA AND PROBABILITY (MONTHS 1-2, WEEKS 1-8)

The two foundations everything else stands on: honest description of data, and probability as the language of uncertainty.

Month 1: Describing Data Honestly (Weeks 1-4)

Data: What It Is and How It Lies to You

Topics:

- Populations, samples and why the difference drives everything
- Types of data: categorical and numerical, and the scales of measurement
- Tidy data: rows, columns and the shape analysis expects
- Where data comes from: surveys, experiments, logs and their built-in biases
- Spreadsheet setup: importing, cleaning basics and first summaries
- Famous data failures: survivorship bias and selection bias with real stories

Projects built:

- First dataset walk: load a real public dataset, document its variables, units and suspicious values before computing anything

Practice: Profile two provided datasets: identify variable types, spot three data-quality problems, write one paragraph on each

Central Tendency: What Is Typical

Topics:

- Mean, median and mode, and what question each one answers
- How skew and outliers pull the mean away from the median
- Weighted means and when unweighted averages mislead
- Group comparisons: the danger of comparing averages of unequal groups
- Computing summaries in the spreadsheet with named functions
- Reading claims critically: what average salary headlines usually hide

Projects built:

- Income exercise: a salary dataset summarised with mean and median, with a written note on why they disagree and which to report

Practice: 12 central tendency problems on real data, each answer accompanied by one sentence on which measure fits and why

Spread: How Much Things Vary

Topics:

- Range and interquartile range
- Variance and standard deviation, built up from deviations rather than dropped as a formula
- The 1.5 IQR rule for flagging outliers
- z-scores: how unusual is this value
- Comparing variability across different units with the coefficient of variation
- Why spread is the half of the story that dashboards leave out

Projects built:

- Consistency study: two players or products with the same average compared on spread, with a recommendation written up

Practice: 14 spread problems including 4 z-score questions, standard deviation computed once by hand and thereafter by software

Charts That Tell the Truth

Topics:

- Histograms and the shapes of distributions: symmetric, skewed, bimodal
- Box plots and comparing groups at a glance
- Scatter plots and the first look at relationships
- Bar charts versus histograms, and why the difference matters
- Misleading charts: truncated axes, cherry-picked windows, area distortions
- Building clean charts in the spreadsheet, with labels that earn their space

Projects built:

- Chart audit: find one misleading chart in the wild, explain the distortion, and rebuild it honestly from the underlying data

Practice: Build five charts from a real dataset, each with a one-line takeaway written underneath

Assessment: Month 1 checkpoint: a full descriptive analysis of a provided dataset, summaries, charts and a short written brief

Month 2: Probability, the Grammar of Uncertainty (Weeks 5-8)

Probability from First Ideas

Topics:

- Experiments, outcomes, sample spaces and events
- Probability as long-run frequency and as degree of belief
- The complement rule and the addition rule
- Mutually exclusive events versus events that overlap
- Venn diagrams that actually resolve arguments
- Simulating simple probabilities in the spreadsheet to check intuition

Projects built:

- Simulation warm-up: estimate a dice probability by simulation, compare with the exact answer, watch the gap shrink with more trials

Practice: 16 probability problems from single events to unions, each hard one checked

by a quick simulation
Conditional Probability and Independence
Topics:
- Conditional probability: how information changes the odds
- The multiplication rule and tree diagrams
- Independence: what it means and how to test for it
- The base-rate fallacy introduced with a real screening example
- Sampling with and without replacement
- Why P(A given B) and P(B given A) are different, and the famous confusions
Projects built:
- Two-way table study: a real dataset cross-tabulated, with conditional probabilities read off and independence checked
Practice: 14 conditional probability problems, every tree diagram drawn before any multiplication
Bayes' Theorem: Updating Beliefs with Evidence
Topics:
- Bayes' theorem derived from the definition of conditional probability
- The medical test problem: why a positive result on a rare disease is usually a false alarm
- Prior, likelihood and posterior in plain language
- Natural frequencies: the counting method that makes Bayes intuitive
- Spam filters and diagnostic reasoning as everyday Bayes
- The prosecutor's fallacy and other courtroom probability errors
Projects built:
- Bayes explainer: work the medical test problem with natural frequencies and write an explanation a friend could follow
Practice: 10 Bayes problems solved twice each, once by formula and once by counting, until the two views agree in your head
Counting and Random Variables
Topics:
- Permutations and combinations, and which one the question wants
- Discrete random variables and probability distributions
- Expected value: what fair means, and why casinos always win
- Variance of a random variable
- Expected value in decisions: insurance, warranties and lotteries
- Building a probability distribution from real frequency data
Projects built:
- Game analysis: compute the expected value of a real lottery or carnival game and write the verdict in two sentences
Practice: 12 counting and random variable problems including 3 expected-value decisions argued in writing
Assessment: Month 2 checkpoint: a mixed probability paper from basic rules through Bayes and expected value

---
PHASE 2: DISTRIBUTIONS, SAMPLING AND ESTIMATION (MONTHS 3-4, WEEKS 9-16)

The named distributions that model real processes, then the machinery of inference: sampling distributions, the central limit theorem seen by simulation, and confidence intervals.

Month 3: The Distributions That Model the World (Weeks 9-12)
The Binomial Distribution
Topics:
- Bernoulli trials and the binomial conditions, checked in words
- The binomial formula and where it comes from
- Mean and variance of a binomial
- Computing binomial probabilities in the spreadsheet and, on the Python track, with scipy
- Cumulative probabilities and careful at-least versus at-most reading
- Real binomial settings: quality control, conversion rates, free-throw streaks
Projects built:
- Conversion study: model a marketing conversion scenario as a binomial and answer three business questions with it
Practice: 14 binomial problems, the conditions checked in writing before any computation
Poisson and Geometric: Counts and Waits
Topics:
- The Poisson distribution for counts per interval: arrivals, defects, calls
- When Poisson fits and when it does not
- Poisson as a limit of the binomial, seen numerically
- The geometric distribution: waiting for the first success
- Choosing the right model: binomial, Poisson or geometric from the story
- Fitting a Poisson to real count data and judging the fit by eye
Projects built:
- Arrivals study: fit a Poisson model to real count data, compare observed and expected counts in a table
Practice: 12 modelling problems where the first marked step is naming the right distribution and defending the choice
Continuous Distributions and the Normal
Topics:
- From histograms to density curves: probability as area

- The uniform distribution as the simplest continuous case
- The normal distribution: shape, parameters and the 68-95-99.7 rule
- Standardising and the standard normal
- Normal probabilities by software, with the table method understood once
- What is and is not normal in real data: heights yes, incomes no

Projects built:

- Normal check: test the 68-95-99.7 rule against a real dataset and report where it holds and where it breaks

Practice: 14 normal distribution problems, each one opened with a sketch of the shaded region

Working with the Normal

Topics:

- Inverse problems: finding cutoffs from probabilities
- Percentiles, reference ranges and grading on a curve
- Assessing normality with histograms and quantile plots
- The normal approximation to the binomial and when it is safe
- Combining independent normal quantities
- A distribution field guide: one page mapping situations to models

Projects built:

- Field guide build: your own one-page reference mapping real situations to binomial, Poisson, geometric, uniform or normal

Practice: 12 mixed distribution problems where the model is never announced in the question

Assessment: Month 3 checkpoint: a distributions paper mixing model choice, computation and interpretation

Month 4: Sampling, the CLT and Confidence Intervals (Weeks 13-16)

Sampling: How Data Gets Collected and Corrupted

Topics:

- Simple random, stratified, cluster and systematic sampling
- Convenience samples and why most internet polls mean little
- Sampling bias, nonresponse bias and question wording effects
- Famous polling failures and what went wrong
- Randomised experiments versus observational studies
- Designing a small survey that will not embarrass you

Projects built:

- Survey critique: take one published poll, identify its sampling method, and write three specific bias risks

Practice: 10 sampling design problems plus one short survey designed, with its weaknesses honestly listed

Sampling Distributions and the Central Limit Theorem

Topics:

- The sampling distribution of the mean: a distribution of statistics, not data
- Standard error and why bigger samples pin the mean down
- The central limit theorem stated carefully
- The CLT demonstrated by simulation: skewed populations, normal-looking sample means
- How sample size affects the shape, watched live
- Why the CLT is the load-bearing wall of everything that follows

Projects built:

- CLT simulation: run the resampling experiment yourself in the spreadsheet or Python, and keep the charts as evidence

Practice: 10 standard error and sampling distribution problems, plus the simulation repeated on a second population shape

Confidence Intervals for Means

Topics:

- The logic of interval estimation
- z-intervals and t-intervals, and when the t-distribution is the honest choice
- Degrees of freedom in plain language
- What 95 percent confidence actually means, and the misreadings to avoid
- Interval width: the three levers of confidence, spread and sample size
- Computing and reporting intervals from real data

Projects built:

- Interval report: a confidence interval for a real dataset's mean, reported in one correct sentence and one wrong-but-common sentence, labelled

Practice: 12 confidence interval problems, every interval followed by a correctly worded interpretation

Proportions and Planning Sample Sizes

Topics:

- Confidence intervals for proportions
- Margin of error and how polls report it
- Sample size calculations: how many people do we actually need
- Reading polling small print: confidence level, margin, population
- Intervals for differences, previewed
- Estimation recap: the full toolkit so far, organised

Projects built:

- Poll deconstruction: take a real published poll and verify its stated margin of error from its sample size

Practice: 12 proportion and sample size problems including two taken from real published surveys

Assessment: Month 4 checkpoint: an estimation paper from sampling design through confidence intervals, interpretation marked as strictly as computation

---

PHASE 3: HYPOTHESIS TESTING, REGRESSION AND THE CAPSTONE (MONTHS 5-6, WEEKS 17-24)

Testing claims with data, measuring and modelling relationships, and then a complete analysis of a real dataset of your own, written up like professional work.

Month 5: Hypothesis Testing Done Honestly (Weeks 17-20)

The Logic of a Hypothesis Test

Topics:

- Null and alternative hypotheses: the courtroom analogy done carefully
- Test statistics and the p-value defined precisely
- Significance levels and the arbitrary honesty of 0.05
- Type I and Type II errors, and which one your situation fears more
- What a p-value is not: not the probability the null is true
- One-tailed versus two-tailed tests and how to choose before seeing data

Projects built:

- Error analysis: for three real scenarios, name both error types and argue which is costlier in each

Practice: 10 test-logic problems with no computation at all: hypotheses, tails and error types only

One-Sample Tests

Topics:

- The one-sample t-test for a mean, start to finish
- The one-sample proportion test
- Checking conditions before trusting the test
- Computing tests in the spreadsheet and, on the Python track, with scipy
- Writing conclusions in context, with the p-value reported not worshipped
- Confidence intervals and tests as two views of the same evidence

Projects built:

- Claim check: test a real advertised claim against real or provided data, and write the verdict in three sentences

Practice: 12 one-sample tests, each conclusion written in context and checked against its confidence interval

Comparing Two Groups

Topics:

- Independent two-sample t-tests
- Paired tests and recognising paired designs
- Two-proportion tests: the A/B test in its natural form
- Effect size: statistically significant versus actually important
- How A/B testing works in product teams, and where it goes wrong
- Reading a published study's methods section without fear

Projects built:

- A/B analysis: a realistic conversion dataset analysed end to end, with a recommendation that weighs effect size, not just p

Practice: 12 two-group problems including one paired design hiding among independent ones

Chi-Square and the Ethics of Testing

Topics:

- Chi-square goodness of fit: does the data match the claimed distribution
- Chi-square test of independence on two-way tables
- Expected counts and when the test is valid
- Multiple comparisons: why testing twenty things finds one false discovery
- p-hacking and publication bias: how honest people fool themselves
- Pre-registering your question: deciding what to test before looking

Projects built:

- Independence study: a real two-way table tested for independence with the full expected-count working shown

Practice: 10 chi-square problems plus a written half-page on one real p-hacking case

Assessment: Month 5 checkpoint: a full inference paper across one-sample, two-sample and chi-square tests, conclusions marked for wording

Month 6: Correlation, Regression and the Capstone (Weeks 21-24)

Correlation

Topics:

- Covariance and the Pearson correlation coefficient
- What r measures and the linear-only caveat
- Anscombe's quartet: four datasets, one r, four different stories
- Correlation is not causation, with the confounder named each time
- Rank correlation for curved but monotonic relationships
- Computing and charting correlations across a real dataset

Projects built:

- Correlation hunt: find the three strongest relationships in a real dataset, chart them, and flag which might be confounded

Practice: 10 correlation problems, every claim of relationship backed by a scatter plot

Linear Regression

Topics:

- The least squares line: what is being minimised and why
- Interpreting slope and intercept in the units of the problem
- R-squared: how much variation the line explains
- Residual plots: the honesty check most analyses skip
- Prediction within range versus extrapolation beyond it
- Multiple regression previewed: several predictors at once, and its dangers

Projects built:
- Prediction model: fit a regression on real data, check residuals, make two predictions and state their limits

Practice: 10 regression problems with slope interpretations written in context every time

Assessment: Capstone gate: analysis plan approved, dataset, question, methods and chosen track (spreadsheet or Python)

Capstone Build

Topics:
- Framing an answerable question before touching the data
- Cleaning and documenting the dataset
- Choosing methods that fit the data, not the other way around
- Descriptive pass first: summaries and charts before any test
- Running the inference or regression the question calls for
- Instructor review checkpoint mid-build

Projects built:
- Capstone analysis in progress: the full pipeline from raw data to preliminary findings, on your own chosen dataset

Practice: Capstone work sessions with a written progress note after each: what was found, what is uncertain, what is next

Capstone Write-Up and Presentation

Topics:
- Structuring an analysis report: question, data, methods, findings, limitations
- Writing limitations honestly: what your analysis cannot claim
- Charts chosen for the reader, not the author
- Presenting analytical work in ten minutes
- Answering methods questions about your own work
- Where to go next: deeper statistics, Python for data, or machine learning

Projects built:
- Completed capstone: a written analysis report and a short presentation, delivered to the class and kept for your portfolio

Practice: Final report polish plus a dry-run presentation with peer questions

Assessment: Capstone review: report and presentation assessed on method, honesty of interpretation and clarity, plus certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- A full descriptive analysis brief on a real public dataset
- A Bayes explainer of the medical test problem in your own words
- An expected-value verdict on a real lottery or game
- A Poisson model fitted to real count data
- A CLT simulation run and charted by you
- A deconstruction of a real published poll's margin of error
- An A/B test analysed end to end with an effect-size-aware recommendation
- A regression prediction model with residual checks
- The capstone: a complete written analysis of a dataset you chose

Total projects: A portfolio of small real-data studies plus one complete capstone analysis with report and presentation

SKILLS THE STUDENT WILL HAVE AT THE END

- Describing data honestly with the right summaries and charts
- Probability, conditional probability and Bayes' theorem used correctly
- Choosing and applying binomial, Poisson and normal models
- Confidence intervals and hypothesis tests, computed and worded properly
- Correlation and regression with the honesty checks included
- Spreadsheet analysis throughout, plus optional Python with pandas and scipy on the parallel track

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, working on real data during class

Practice: 3-4 hours weekly of problem sets and dataset work between classes

Review: Homework reviewed with written feedback; interpretation wording corrected as strictly as calculations

Homework: A problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with worked solutions for stuck problems

Progress updates: Monthly progress notes covering test scores and capstone readiness

Career guidance: Honest guidance on next steps: which roles use these skills, what to learn next, and what this course alone does and does not qualify you for

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies mastery, not attendance

WHERE THIS CAN LEAD

- Data analyst and reporting analyst roles, where descriptive statistics and inference are the daily toolkit
- Business and product analytics: A/B testing, metrics and experiment literacy
- A prepared entry into data science and machine learning courses, including our Python and AI/ML tracks
- Research work in any field with a methods section: psychology, economics, public health
- Better decisions in your current role, which is the quiet majority use of statistics

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for roles where statistics is a core skill, in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 3-6 LPA (Data Analyst / Reporting Analyst / MIS Analyst)
Growing roles: Rs 6-12 LPA (Senior Analyst / Business Analyst / Product Analyst)
Experienced roles: Rs 12-25 LPA (Analytics Lead / Data Scientist, typically with added programming and ML skills)
International: $55k-110k USD (USA) for analyst roles, depending on role, industry and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Do I need to know programming for this course?
A: No, and this is a real no, not a marketing one. Every technique in the course, from descriptive statistics through regression, is taught and assessed in a spreadsheet, and the capstone can be completed entirely without code. A parallel Python track runs for those who want it: the same analyses redone with pandas and scipy, taught from zero. Roughly speaking, the spreadsheet track gives you the statistics; adding the Python track gives you the statistics plus a tool employers increasingly expect.

Q: How much maths do I need coming in?
A: School algebra: you should be able to rearrange a simple formula and be comfortable with fractions, percentages and squared terms. There is no calculus in the course. Where a result normally requires calculus to derive, we demonstrate it numerically or by simulation instead, which is often more convincing anyway. The week 1 diagnostic catches genuine gaps, and we patch them in the early weeks rather than turning you away.

Q: How is this different from a data science course?
A: A data science course is mostly tooling: Python, data wrangling, machine learning libraries, deployment. This course is the statistical reasoning underneath, which data science courses typically compress into two rushed weeks and which is exactly where most self-taught data scientists are weakest. If your goal is data science, this course first and a data science course second is the strong order. If your goal is to understand data, studies and experiments in your existing work, this course alone may be all you need.

Q: What is the capstone project, and do I choose the dataset?
A: In the final month you take one real dataset, yours from work, or one you choose from public sources with our help, frame a question, run the appropriate analysis and write it up with methods and limitations, then present it. The write-up format mirrors how analytical work is actually reported. Students on the spreadsheet track and the Python track complete the same capstone with different tools, and both versions are portfolio pieces you keep.

Q: I am a working professional. Can I manage this alongside a job?
A: The course is built for it: two live one-hour classes a week, scheduled in evening and weekend slots, plus three to four hours of practice you can place wherever your week allows. The pace is steady rather than punishing, and WhatsApp doubt support means a stuck Tuesday problem does not have to wait for Saturday. Professionals who miss a class can attend the same session with another batch where scheduling allows.

Q: Will this course make me a data analyst?
A: It gives you the statistical core of the job, which is genuinely the hard part, and a capstone analysis to show. It does not by itself supply everything analyst job listings ask for: SQL and dashboard tools are common requirements we do not cover here. Our honest advice, which we repeat in the career guidance sessions, is that this course plus SQL practice plus a portfolio of two or three analyses is a credible analyst preparation. The course alone is the

foundation, not the whole house.

Q: Is this course aligned to any university syllabus?
A: It covers the standard core of a first university statistics course: descriptive statistics, probability, distributions, estimation, hypothesis testing and regression, so students frequently use it alongside degree coursework in commerce, economics, psychology and engineering. It is not tied to any single university's paper pattern, and we do not teach to one exam. Students preparing for a specific university exam get targeted practice on their syllabus topics in the doubt sessions.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A problem set follows every class with written feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final exam covering the whole course. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real understanding, not attendance.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/statistics-probability-maths-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 128 OF 137: SWIFT AND IOS COURSE: SWIFTUI APPS FROM ZERO
==========

Course link: https://learn.modernagecoders.com/courses/complete-swift-programming-masterclass-college
Subject area: App Development
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Swift course for college students and professionals: modern Swift, then real iOS development with SwiftUI, data, networking and a capstone app in 10-12 months.

In one line: The Apple stack learned honestly: Swift to real depth, SwiftUI as the future it already is, and a capstone app built to App Store discipline.

ABOUT THE PROGRAMME

iOS development is one of the most rewarding specializations in software, and it deserves an honest course: Swift learned as a real language, optionals, value types, protocols and concurrency understood properly, then iOS built the modern way with SwiftUI, architecture and state done right, data persistence, networking, and the production habits that make apps feel like Apple apps. A finale month ships a capstone and sits the final exam. One honesty first: serious iOS development requires a Mac, Xcode runs only on macOS. This course says so upfront (see the FAQ for the realistic options) rather than discovering it for you in week three. With that settled, the pace is honest too: ten teaching months at two live classes a week plus four to six hours of practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

- The Mac requirement stated upfront, with realistic options, honesty most iOS courses skip
- Swift as a language first: optionals, value semantics, protocols and structured concurrency understood properly
- SwiftUI-first: the declarative future Apple is already building on
- Apple-feel as a standard: human interface habits taught alongside code
- An honest 10-month arc with 2 months in hand
- Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: Swift (Months 1-3): the language to real depth, optionals to protocols
Phase 2: SwiftUI (Months 4-6): views, state, navigation and interfaces that feel like Apple built them
Phase 3: Professional iOS (Months 7-9): architecture, data, networking, testing and release

discipline
Phase 4: The finale (Month 10): the capstone app, shipped and defended, and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

Coding experience: None. Swift is taught from the first line
Equipment: A Mac is required for iOS development (Xcode is macOS-only); see the FAQ for realistic options if you do not own one yet
Age: 18+; college students, graduates and working professionals
Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

Apple ecosystem fans: People who love the platform and want to build for it properly
College students: Students who want a premium, demonstrable specialization
Designers who code: Detail-oriented builders drawn to the craft end of apps
Career switchers: Professionals entering mobile on the Apple side
Indie dreamers: Builders who want one polished app of their own on the store someday

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & CORE SWIFT SKILLS (MONTHS 1-3, WEEKS 1-13)

Build rock-solid Swift fundamentals. Learn programming logic, master Swift syntax, protocol-oriented programming, and create your first iOS applications.

Months 1-2: Swift Fundamentals & Programming Basics (Weeks 1-7)
Introduction to Swift & Development Environment Setup
Topics:
- What is Swift? History, features, and evolution from Objective-C
- Swift ecosystem: Xcode, Swift Package Manager, Playgrounds
- Apple platforms overview: iOS, macOS, watchOS, tvOS, visionOS
- Swift applications: Mobile apps, desktop apps, server-side Swift
- Installing Xcode from Mac App Store
- Setting up development environment on macOS
- Understanding Xcode interface, panels, and shortcuts
- Your first Swift program: Hello World in Playground
- Swift program structure: imports, main entry point
- Running Swift code in Playgrounds vs Apps
- Understanding LLVM compiler and Swift runtime
- Comments: single-line (//), multi-line (/* */), documentation (///)
- Swift coding conventions and API Design Guidelines
- Introduction to Apple Human Interface Guidelines (HIG)
Projects built:
- Hello World variations with personalized messages in Playground
- Simple output programs using print() function
- Personal information display program
- ASCII art generator in Swift Playground
- Interactive Playground with markdown documentation
Practice: Daily: 30 min Swift syntax practice, write 5-10 simple programs in Playgrounds
Variables, Data Types & Operators
Topics:
- Variables (var) and Constants (let): declaration and initialization
- Type inference vs explicit type annotation
- Basic data types: Int, Double, Float, Bool, String, Character
- Understanding Int8, Int16, Int32, Int64, UInt variants
- Type safety and type inference in Swift
- Type conversion and casting with as, as?, as!
- Optionals: declaring with ?, unwrapping with !, optional binding
- Nil-coalescing operator ??
- Optional chaining with ?.
- Implicitly unwrapped optionals
- Operators: arithmetic (+, -, *, /, %)
- Compound assignment operators: +=, -=, *=, /=
- Comparison operators: ==, !=, >, <, >=, <=
- Logical operators: &&, ||, !
- Range operators: closed range (...), half-open range (..<)
- Ternary conditional operator: ? :
- Nil-coalescing operator: ??
- String basics: creation, interpolation, multiline strings
- String manipulation: concatenation, substring, characters
Projects built:
- Simple calculator (four basic operations) in Playground
- Temperature converter (Celsius, Fahrenheit, Kelvin)
- Age calculator with optional handling

    - BMI calculator with health categories
    - Currency converter with type safety
    - Area and perimeter calculator for shapes
    - Compound interest calculator
    - String formatter and validator
  Practice: Solve 30 problems on variables, data types, optionals, and operators
Control Flow Statements
  Topics:
    - Boolean expressions and conditions
    - If statement: single condition execution
    - If-else statement: two-way branching
    - If-else-if ladder: multiple conditions
    - Nested if statements
    - Switch statement: powerful pattern matching
    - Switch with value binding and where clauses
    - Switch with compound cases
    - Switch with ranges and tuples
    - No implicit fallthrough in Swift switch
    - While loop: pre-tested loop
    - Repeat-while loop: post-tested loop (equivalent to do-while)
    - For-in loop: iterating over sequences
    - For-in with ranges and stride
    - For-in with enumerated() for index access
    - Break statement: exit loop
    - Continue statement: skip iteration
    - Labeled statements for nested loops
    - Guard statement: early exit pattern
    - Defer statement: cleanup code
  Projects built:
    - Number guessing game with attempts limit
    - Grade calculator with letter grades using switch
    - Even/odd checker and list generator
    - Prime number checker and generator
    - Multiplication table generator
    - Factorial calculator (iterative)
    - Fibonacci series generator
    - Pattern printing (stars, numbers, pyramids, diamonds)
    - FizzBuzz challenge with switch
    - Menu-driven calculator with guard statements
    - Simple ATM simulation with validation
  Practice: Solve 40 control flow and loop problems
Functions & Collections
  Topics:
    - Functions: definition, purpose, and structure
    - Function declaration: parameters and return types
    - Function calling and argument labels
    - External and internal parameter names
    - Default parameter values
    - Variadic parameters (...)
    - In-out parameters for mutating arguments
    - Functions returning multiple values with tuples
    - Nested functions
    - Function types as parameters and return types
    - Closures: inline anonymous functions
    - Closure syntax and shorthand argument names ($0, $1)
    - Trailing closure syntax
    - Capturing values in closures
    - Escaping closures (@escaping)
    - Autoclosures (@autoclosure)
    - Arrays: declaration, initialization, manipulation
    - Array methods: append, insert, remove, contains, filter, map
    - Sets: unique unordered collections
    - Set operations: union, intersection, subtraction
    - Dictionaries: key-value pairs
    - Dictionary methods: updateValue, removeValue, keys, values
    - Iterating over collections
    - Higher-order functions: map, filter, reduce, flatMap, compactMap
  Projects built:
    - Function library for mathematical operations
    - Array statistics calculator (sum, average, min, max)
    - Linear search and binary search implementation
    - Sorting algorithms: bubble sort, selection sort
    - Contact list with dictionary
    - Todo list with array operations
    - Student grade management system
    - Factorial using recursion
    - Palindrome checker (string/number)
    - Prime number generator in range
    - Shopping cart with higher-order functions

- Word frequency counter with closures
Practice: Create 25 functions for various functionalities, solve 30 collection problems

Month 3: Object-Oriented & Protocol-Oriented Programming (Weeks 8-13)
Structures, Classes & Properties
Topics:
- Object-Oriented Programming paradigm in Swift
- Structures (struct): value types
- Classes (class): reference types
- Value types vs reference types comparison
- When to use struct vs class
- Properties: stored and computed properties
- Property observers: willSet and didSet
- Lazy stored properties
- Type properties (static)
- Instance methods
- Type methods (static and class)
- Mutating methods in structures
- Initializers: default, custom, memberwise
- Failable initializers (init?)
- Required initializers
- Deinitializers (deinit)
- Self keyword usage
- Access control: private, fileprivate, internal, public, open
- Encapsulation and data hiding
- Copy-on-write optimization
Projects built:
- Student struct with properties and methods
- Bank Account class with deposit/withdraw
- Book struct for library management
- Employee class with computed salary
- Rectangle/Circle structs with area/perimeter
- Car class with property observers
- Product struct for inventory system
- Person struct with multiple initializers
- Date struct implementation
- Timer class with proper memory management
Practice: Create 20 different structs and classes modeling real-world entities
Inheritance, Polymorphism & Memory Management
Topics:
- Inheritance: creating subclasses
- Base class and subclass relationship
- Overriding methods with 'override' keyword
- Overriding properties
- Preventing overrides with 'final'
- Calling superclass methods with 'super'
- Designated and convenience initializers
- Initializer inheritance and overriding
- Required initializers in inheritance
- Polymorphism in Swift
- Type casting: is, as, as?, as!
- Any and AnyObject types
- Automatic Reference Counting (ARC)
- Strong references and retain cycles
- Weak references (weak)
- Unowned references (unowned)
- Closure capture lists [weak self], [unowned self]
- Memory leaks detection and prevention
- Identity operators: === and !==
Projects built:
- Animal hierarchy (Animal -> Dog, Cat, Bird)
- Shape hierarchy with area calculation
- Vehicle system (Vehicle -> Car, Bike, Truck)
- Employee hierarchy (Employee -> Manager, Developer)
- Bank account types (Account -> Savings, Current)
- School management (Person -> Student, Teacher)
- Media player hierarchy (Media -> Audio, Video)
- Retain cycle demonstration and fix
- Memory-safe delegate implementation
Practice: Build 15 inheritance hierarchies, solve 25 ARC and memory problems
Protocols & Protocol-Oriented Programming
Topics:
- Protocols: defining blueprints
- Protocol syntax and requirements
- Property requirements in protocols
- Method requirements in protocols
- Mutating method requirements
- Initializer requirements in protocols
- Protocol adoption by structs, classes, enums

- Protocol inheritance
- Protocol composition with &
- Protocol extensions: default implementations
- Protocol-Oriented Programming (POP) philosophy
- POP vs OOP comparison
- Conditional conformance
- Associated types in protocols
- Self requirements in protocols
- Opaque types (some Protocol)
- Existential types (any Protocol)
- Common Swift protocols: Equatable, Hashable, Comparable, Codable
- Custom protocol creation
- Delegation pattern with protocols

Projects built:
- Shape calculator with protocol requirements
- Payment gateway with protocol abstraction
- Database abstraction with protocols
- Notification system (Email, SMS, Push protocols)
- Sortable collection with Comparable
- Custom Equatable and Hashable types
- Plugin architecture using protocols
- PHASE 1 MINI CAPSTONE: Library Management System (Console-based)
- Features: Books, members, issue/return, search, POP principles

Practice: Solve 30 protocol and POP problems
Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 2: INTERMEDIATE SWIFT & SWIFTUI MASTERY (MONTHS 4-6, WEEKS 14-26)

Master advanced Swift features, enumerations, generics, error handling, SwiftUI fundamentals, and basic iOS app development.

Month 4: Advanced Swift Features & SwiftUI Basics (Weeks 14-18)

Enumerations & Advanced Types

Topics:
- Enumerations (enum) in Swift
- Enum cases and raw values
- Enum with associated values
- Computed properties in enums
- Methods in enums
- Mutating methods in enums
- Recursive enumerations (indirect)
- CaseIterable protocol for enum iteration
- Pattern matching with enums in switch
- If-case and guard-case syntax
- Nested types: structs, classes, enums inside types
- Type aliases (typealias)
- Tuple types and destructuring
- Optional as enum (Optional<Wrapped>)
- Result type for error handling
- Never type for functions that don't return

Projects built:
- Compass direction enum with methods
- HTTP status code enum with raw values
- Network response enum with associated values
- Card game with suit and rank enums
- State machine with enum
- Currency converter with enum
- Error handling system with Result enum
- Menu system with nested enums
- Traffic light state machine
- JSON parsing result handling

Practice: Solve 35 enumeration and advanced type problems

Error Handling & Advanced Error Management

Topics:
- Error handling philosophy in Swift
- Error protocol and custom errors
- Creating custom error types with enum
- Throwing functions with 'throws' keyword
- Throwing initializers
- Do-catch blocks for error handling
- Multiple catch clauses
- Catching specific errors
- Converting errors to optionals with try?
- Forcing try with try! (use sparingly)
- Propagating errors with throws
- Rethrowing functions (rethrows)
- LocalizedError protocol for user messages

    - CustomNSError for bridging to Objective-C
    - Error handling best practices
    - Cleanup with defer statements
    - Assertions and preconditions
    - fatalError for unimplemented code
  Projects built:
    - Robust calculator with error handling
    - File reader with proper error handling
    - Custom banking error system
    - Network request error handling
    - User validation with custom errors
    - Age validator with localized errors
    - Division calculator with error handling
    - User registration with validation errors
    - JSON parser with comprehensive error handling
  Practice: Solve 25 error handling scenarios
Generics & Type System Deep Dive
  Topics:
    - Introduction to generics: type parameters
    - Generic functions
    - Generic types (struct, class, enum)
    - Type constraints with protocol conformance
    - Where clauses for complex constraints
    - Associated types in protocols
    - Generic subscripts
    - Generic extensions
    - Protocol extensions with constraints
    - Type erasure patterns
    - Opaque types (some Protocol) deep dive
    - Primary associated types (Swift 5.7+)
    - Existential types (any Protocol)
    - Conditional conformance
    - Specialized generic implementations
    - Standard library generic types: Array, Dictionary, Optional, Result
  Projects built:
    - Generic stack and queue implementation
    - Generic linked list
    - Type-safe data repository
    - Generic cache implementation
    - Custom Result type with generics
    - Generic sorting algorithms
    - Generic tree data structure
    - Type-safe networking layer
  Practice: Create 15 generic types, solve 25 generics problems
SwiftUI Fundamentals - Part 1
  Topics:
    - Introduction to SwiftUI: declarative UI
    - SwiftUI vs UIKit comparison
    - View protocol and basic views
    - Text view: styling, modifiers, formatting
    - Image view: SF Symbols, asset images, resizing
    - Button view: actions, styles, labels
    - TextField and SecureField for input
    - Toggle, Slider, Stepper, Picker
    - VStack, HStack, ZStack for layout
    - Spacer and Divider
    - Frame modifier: fixed and flexible sizing
    - Padding, background, overlay modifiers
    - ForEach for dynamic content
    - ScrollView for scrollable content
    - List view for table-like layouts
    - @State property wrapper for local state
    - Two-way binding with $
    - View modifiers and modifier order
  Projects built:
    - Hello World SwiftUI app
    - Profile card with image and text
    - Simple counter app with @State
    - Temperature converter UI
    - BMI calculator with input fields
    - Todo list with basic functionality
    - Color picker interface
    - Settings screen with toggles and pickers
  Practice: Build 15 SwiftUI views with various controls
SwiftUI Fundamentals - Part 2
  Topics:
    - @Binding for child view communication
    - @ObservableObject and @Published
    - @StateObject vs @ObservedObject

- @EnvironmentObject for app-wide state
- ObservableObject protocol implementation
- NavigationStack (iOS 16+) navigation
- NavigationLink for navigation
- NavigationPath for programmatic navigation
- TabView for tab-based navigation
- Sheet, fullScreenCover for modal presentation
- Alert and confirmationDialog
- Custom view extraction
- ViewBuilder for custom containers
- Creating reusable components
- SF Symbols integration
- Dark mode and color schemes

Projects built:
- Multi-screen navigation app
- Tab-based application
- Settings app with observable object
- Shopping list with CRUD operations
- Note-taking app with navigation
- Modal presentation examples
- Reusable component library
- Theme-aware application

Practice: Build 10 multi-view SwiftUI applications

Month 5: Advanced SwiftUI & UIKit Fundamentals (Weeks 19-22)
SwiftUI Lists, Forms & Data Display
Topics:
- List view deep dive
- List styles: plain, grouped, insetGrouped, sidebar
- SwipeActions for list items
- Contextual menus (contextMenu)
- List with sections and headers
- Dynamic lists with Identifiable
- List editing: delete, move, selection
- Searchable modifier for filtering
- Pull to refresh (refreshable)
- Form view for settings and input
- Section in forms
- LabeledContent for forms (iOS 16+)
- Grid layouts: LazyVGrid, LazyHGrid
- GridItem and grid configuration
- Lazy stacks for performance
- Table view (macOS and iPadOS)
- OutlineGroup for hierarchical data

Projects built:
- Contact list with search and sections
- Settings app with forms
- Photo gallery with grid layout
- Task manager with swipe actions
- Editable list with add/delete/reorder
- Recipe list with contextual menus
- Product catalog with grid view
- Hierarchical file browser

Practice: Build 10 list and grid-based applications
SwiftUI Animations & Custom Drawing
Topics:
- Animation fundamentals in SwiftUI
- Implicit animations with .animation modifier
- Explicit animations with withAnimation
- Animation types: linear, easeIn, easeOut, spring
- Custom spring animations
- Animatable protocol and animatableData
- Transitions: opacity, scale, slide, move
- Combined and asymmetric transitions
- MatchedGeometryEffect for hero animations
- TimelineView for continuous animations
- Canvas for high-performance drawing
- Shapes: Rectangle, Circle, Ellipse, Capsule
- Path for custom shapes
- GeometryReader for dynamic layouts
- Custom shapes with Shape protocol
- Gradients: Linear, Radial, Angular
- Blend modes and visual effects

Projects built:
- Animated button effects
- Loading spinner animations
- Card flip animation
- Hero transition between views
- Custom progress indicator

- Animated chart visualization
- Drawing app with Canvas
- Custom animated splash screen
- Particle effect animation
- Morphing shape animations

Practice: Create 15 animations and custom shapes

UIKit Fundamentals

Topics:

- UIKit overview and architecture
- UIViewController lifecycle
- Storyboards vs programmatic UI
- UIView and view hierarchy
- Auto Layout: constraints and anchors
- Programmatic Auto Layout with NSLayoutConstraint
- UIStackView for simplified layouts
- UILabel, UIButton, UIImageView
- UITextField and UITextView
- UITableView: cells, sections, data source
- UITableView delegate methods
- UICollectionView fundamentals
- UINavigationController navigation
- UITabBarController tabs
- Presenting view controllers modally
- UIAlertController for alerts and action sheets
- Integrating UIKit in SwiftUI (UIViewRepresentable)
- Integrating SwiftUI in UIKit (UIHostingController)

Projects built:

- UIKit todo list app
- Table view with custom cells
- Collection view photo gallery
- Navigation-based master-detail app
- Tab bar application
- Form with text fields and validation
- UIKit + SwiftUI hybrid app
- Custom table view cells

Practice: Build 12 UIKit-based views and controllers

Data Persistence - UserDefaults & File System

Topics:

- Data persistence options in iOS
- UserDefaults for simple preferences
- Storing primitives in UserDefaults
- Storing custom objects with Codable
- @AppStorage property wrapper
- FileManager for file operations
- iOS file system: Documents, Caches, tmp
- Reading and writing files
- JSON encoding and decoding
- Codable protocol: Encodable, Decodable
- CodingKeys for custom mapping
- Custom encoding/decoding logic
- PropertyListEncoder and Decoder
- Keychain for secure storage
- Keychain wrapper libraries
- iCloud key-value storage
- Data migration strategies

Projects built:

- User preferences with UserDefaults
- Settings app with @AppStorage
- Note-taking app with file storage
- JSON data persistence
- Secure login credentials with Keychain
- Configuration file manager
- Data export/import utility
- User profile with local storage

Practice: Implement persistence in all previous projects

Month 6: Concurrency, Core Data & Combine (Weeks 23-26)

Swift Concurrency - Async/Await

Topics:

- Concurrency in Swift: history and evolution
- Grand Central Dispatch (GCD) basics
- DispatchQueue: main, global, custom
- Async/await syntax (Swift 5.5+)
- Async functions and methods
- Calling async functions with await
- Task for spawning async work
- Task groups for parallel execution
- Async sequences (AsyncSequence)
- Actors for data race protection

- @MainActor for UI updates
- Global actors and custom actors
- Sendable protocol for thread safety
- Structured concurrency
- Task cancellation and priorities
- Migrating from GCD to async/await

Projects built:

- Async data fetching example
- Parallel image downloader
- Actor-based counter
- Async file operations
- Concurrent API calls
- Background task processing
- Thread-safe data manager
- Async search implementation

Practice: Convert 10 projects to async/await

Core Data Fundamentals

Topics:

- Core Data overview and architecture
- Core Data stack: NSPersistentContainer
- NSManagedObjectModel: data model
- NSManagedObjectContext: scratch pad
- NSPersistentStoreCoordinator
- Creating data model in Xcode
- Entities, attributes, and relationships
- NSManagedObject subclasses
- Creating and saving objects
- Fetching objects with NSFetchRequest
- Predicates for filtering (NSPredicate)
- Sorting with NSSortDescriptor
- Updating and deleting objects
- Core Data with SwiftUI: @FetchRequest
- @Environment(\.managedObjectContext)
- FetchedResults for displaying data
- Relationships: one-to-one, one-to-many, many-to-many

Projects built:

- Todo app with Core Data
- Contact manager with Core Data
- Expense tracker with categories
- Note-taking app with Core Data
- Book library with authors relationship
- Recipe app with ingredients relationship
- Core Data with SwiftUI integration
- Search and filter implementation

Practice: Build 8 Core Data applications

Combine Framework Fundamentals

Topics:

- Reactive programming concepts
- Publishers and Subscribers
- Built-in publishers: Just, Future, Empty, Fail
- @Published property wrapper
- Subjects: PassthroughSubject, CurrentValueSubject
- Operators: map, filter, flatMap, merge
- Combining publishers: combineLatest, zip, merge
- Error handling in Combine
- Scheduling and threads
- Cancellables and memory management
- Combine with SwiftUI
- Combine with URLSession
- Debounce and throttle for search
- Combine vs async/await comparison
- When to use Combine

Projects built:

- Form validation with Combine
- Search with debounce
- Real-time data binding
- API response handling with Combine
- Multi-field form validation
- Live search implementation
- Timer-based updates

Practice: Implement Combine in 8 projects

Swift Package Manager & Project Organization

Topics:

- Swift Package Manager (SPM) overview
- Creating Swift packages
- Package.swift manifest file
- Adding dependencies to projects
- Popular Swift packages and libraries
- Creating reusable libraries

- Local packages for modularization
- Publishing packages
- Semantic versioning
- Version control with Git (review/deep dive)
- Git branching strategies
- GitHub collaboration for iOS projects
- .gitignore for Xcode projects
- Code organization best practices
- MVVM architecture introduction

Projects built:
- Create custom Swift package
- Multi-module app architecture
- Networking library package
- UI component library package
- Git repository for all projects
- Modular app structure

Practice: Convert all projects to modular structure
Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 3: PROFESSIONAL IOS DEVELOPMENT (MONTHS 7-9, WEEKS 27-39)

Master professional iOS development with networking, advanced architecture, testing, accessibility, and real-world app development patterns.

Month 7: Networking & Architecture (Weeks 27-31)

Networking with URLSession

Topics:
- Networking fundamentals: HTTP protocol
- HTTP methods: GET, POST, PUT, DELETE, PATCH
- HTTP headers and status codes
- JSON format and structure
- URLSession overview and configuration
- URLSessionDataTask for data requests
- URLSessionDownloadTask for downloads
- URLSessionUploadTask for uploads
- URLRequest configuration
- Async/await with URLSession
- Error handling in networking
- Parsing JSON with Codable
- Handling nested JSON
- Custom CodingKeys and date handling
- Network error types and handling
- URL components and query parameters

Projects built:
- Weather app consuming REST API
- Movie database browser (TMDb API)
- News reader with API integration
- GitHub user search
- Currency exchange rate app
- Photo download and caching
- REST API client library
- API response error handling

Practice: Build 10 network-connected applications

Advanced Networking & Authentication

Topics:
- Building a generic networking layer
- Protocol-based API client
- Request/Response interceptors
- Authentication: API keys, tokens
- OAuth 2.0 flow implementation
- JWT token handling
- Refresh token management
- Keychain for token storage
- SSL pinning for security
- Background downloads and uploads
- Network reachability (NWPathMonitor)
- Caching strategies: URLCache
- Image caching solutions
- Request retry logic
- Rate limiting handling
- GraphQL basics with Swift

Projects built:
- Complete authenticated API client
- Social media login (OAuth)
- Image caching library
- Offline-first data sync
- Background upload manager

    - Network layer with unit tests
    - GraphQL client implementation
    - API wrapper for multiple endpoints
  Practice: Build production-ready networking layer
App Architecture - MVVM
  Topics:
    - Software architecture importance
    - MVC: Model-View-Controller (Apple's default)
    - Problems with Massive View Controller
    - MVVM: Model-View-ViewModel
    - ViewModel responsibilities
    - Data binding with Combine
    - Data binding with @Observable (iOS 17+)
    - Dependency Injection principles
    - Protocol-based dependencies
    - Coordinator pattern for navigation
    - Repository pattern for data access
    - Service layer architecture
    - Clean Architecture overview
    - VIPER architecture basics
    - Choosing the right architecture
    - Testing with architecture patterns
  Projects built:
    - Refactor app to MVVM
    - MVVM with Combine bindings
    - Coordinator-based navigation
    - Repository pattern implementation
    - Dependency injection container
    - Clean architecture example
    - Testable architecture demo
    - Multi-module MVVM app
  Practice: Apply MVVM to all major projects
Advanced Architecture & The Composable Architecture
  Topics:
    - State management challenges
    - Unidirectional data flow
    - The Composable Architecture (TCA) introduction
    - Reducers and State
    - Actions and Effects
    - Store and ViewStore
    - Scoping and composition
    - Testing in TCA
    - Alternatives: ReSwift, Redux-like patterns
    - Feature modules with TCA
    - Navigation in TCA
    - Dependency injection in TCA
    - Performance optimization
    - When to use TCA vs MVVM
    - Architecture decision factors
  Projects built:
    - Counter app with TCA
    - Todo list with TCA
    - Navigation with TCA
    - Feature composition example
    - TCA with network effects
    - Full app with TCA architecture
    - Testing TCA reducers
  Practice: Build 5 TCA-based features
Unit Testing & Test-Driven Development
  Topics:
    - Testing importance and types
    - XCTest framework overview
    - Writing test cases
    - XCTAssert functions
    - Test lifecycle: setUp, tearDown
    - Testing view models
    - Testing with protocols and mocks
    - Dependency injection for testing
    - Async testing with expectations
    - Testing async/await code
    - Testing Combine publishers
    - Test doubles: mocks, stubs, fakes, spies
    - Code coverage with Xcode
    - Test-Driven Development (TDD) methodology
    - Testing best practices
    - Continuous testing workflow
  Projects built:
    - Unit tests for calculator
    - ViewModel testing suite

```
      - Mock network layer tests
      - TDD implementation of stack
      - Core Data unit tests
      - Async code testing
      - Test coverage improvement
      - Complete test suite for capstone
    Practice: Write tests for all major projects, achieve 70%+ coverage

Month 8: UI Testing, Accessibility & Advanced Features (Weeks 32-36)
  UI Testing & Snapshot Testing
    Topics:
      - UI testing with XCUITest
      - XCUIApplication for app launching
      - XCUIElement for element identification
      - Accessibility identifiers for testing
      - Tapping, typing, swiping gestures
      - Assertions in UI tests
      - Waiting for elements
      - Testing navigation flows
      - Testing lists and tables
      - UI test recording
      - Snapshot testing introduction
      - swift-snapshot-testing library
      - Testing different device sizes
      - Testing dark mode
      - Testing accessibility features
      - Page Object pattern for UI tests
    Projects built:
      - UI test suite for login flow
      - Navigation UI tests
      - List interaction tests
      - Snapshot tests for components
      - Multi-device snapshot tests
      - Dark mode snapshot tests
      - Accessibility audit tests
      - End-to-end UI test suite
    Practice: Add UI tests to all major projects
  Accessibility & Localization
    Topics:
      - Accessibility importance and guidelines
      - VoiceOver support
      - Accessibility labels and hints
      - Accessibility traits
      - Accessibility value for dynamic content
      - SwiftUI accessibility modifiers
      - UIKit accessibility properties
      - Dynamic Type support
      - Color contrast requirements
      - Reduce Motion support
      - Accessibility testing with Xcode
      - Localization fundamentals
      - String localization: Localizable.strings
      - String catalogs (Xcode 15+)
      - Pluralization rules
      - Date and number formatting
      - Right-to-left (RTL) language support
      - Testing localization
    Projects built:
      - Accessibility audit and fixes
      - VoiceOver-optimized app
      - Dynamic Type implementation
      - Localized app in 3+ languages
      - RTL language support
      - Accessible custom controls
      - Localization testing automation
    Practice: Make all projects fully accessible
  Maps, Location & Background Tasks
    Topics:
      - MapKit framework overview
      - Displaying maps with Map (SwiftUI)
      - MKMapView in UIKit
      - Map annotations and overlays
      - Custom annotation views
      - Route calculation and directions
      - Core Location framework
      - Requesting location permissions
      - Getting user location
      - Significant location changes
      - Geofencing with CLCircularRegion
```

- Geocoding and reverse geocoding
- Background location updates
- Background tasks (BGTaskScheduler)
- Background app refresh
- Background processing tasks

Projects built:
- Map with user location
- Points of interest app
- Route planning app
- Location-based reminders (geofencing)
- Store finder with directions
- Background location tracker
- Background sync implementation
- Fitness tracker with background location

Practice: Build 8 location-aware applications

Push Notifications & App Extensions

Topics:
- Push notifications overview
- Local notifications with UNUserNotificationCenter
- Notification triggers: time, calendar, location
- Notification content and categories
- Custom notification actions
- Remote push notifications (APNs)
- Setting up APNs certificates
- Firebase Cloud Messaging (FCM) alternative
- Handling notifications in app
- Rich notifications with attachments
- App extensions overview
- Today widgets / Widget extension
- Share extension
- Action extension
- Notification content extension
- App groups for data sharing

Projects built:
- Local notification reminder app
- Scheduled notification system
- Remote push notification setup
- Rich notification with images
- Widget for app data
- Share extension for app
- Interactive notification actions
- Multi-widget app

Practice: Add notifications and widgets to 5 projects

Camera, Photos & Media

Topics:
- Camera access with UIImagePickerController
- PHPickerViewController for photo selection
- Camera with AVCaptureSession
- Custom camera UI
- Photo library access (PhotosUI)
- Saving photos to library
- Photo editing with Core Image
- CIFilter for image processing
- AVFoundation for video
- AVPlayer for media playback
- AVAudioPlayer for audio
- Recording audio with AVAudioRecorder
- Video capture and editing
- Media file formats and compression
- Metal basics for graphics

Projects built:
- Photo capture app
- Photo gallery with picker
- Image filter app
- Video player app
- Voice memo recorder
- Custom camera with filters
- Photo editor with Core Image
- Media-rich content app

Practice: Build 8 media-focused applications

Month 9: Performance, Purchases & Security (Weeks 37-39)

App Performance & Optimization

Topics:
- Performance measurement importance
- Instruments tool overview
- Time Profiler for CPU analysis
- Allocations for memory analysis
- Leaks instrument for memory leaks

- Network profiling
- Core Animation instrument
- App launch time optimization
- Memory management best practices
- Image optimization and caching
- Table/collection view performance
- Lazy loading patterns
- Background thread optimization
- Energy and battery impact
- MetricKit for production monitoring

Projects built:

- Performance profiling report
- Memory leak identification and fix
- App launch optimization
- Image caching implementation
- List performance optimization
- Battery efficiency improvements
- Production metrics setup

Practice: Profile and optimize all major projects

In-App Purchases & Subscriptions

Topics:

- StoreKit framework overview
- StoreKit 2 (modern API)
- Product types: consumable, non-consumable, subscription
- Configuring products in App Store Connect
- Fetching products
- Processing purchases
- Transaction verification
- Receipt validation
- Subscription management
- Subscription offers and trials
- Restoring purchases
- Server-side validation
- RevenueCat SDK integration
- Testing in-app purchases

Projects built:

- Consumable IAP implementation
- Premium feature unlock
- Subscription-based app
- Restore purchases feature
- Receipt validation implementation
- RevenueCat integration
- Premium app conversion

Practice: Add monetization to 3 projects

Security & Data Protection

Topics:

- iOS security architecture
- App Transport Security (ATS)
- Secure data storage best practices
- Keychain deep dive
- Keychain access groups
- Data encryption with CryptoKit
- Hashing and digital signatures
- Biometric authentication (Face ID, Touch ID)
- LocalAuthentication framework
- Secure text entry
- Jailbreak detection
- SSL certificate pinning
- Code obfuscation basics
- Privacy manifest (iOS 17+)
- Privacy nutrition labels

Projects built:

- Biometric login implementation
- Encrypted data storage
- Secure notes app
- Certificate pinning implementation
- Privacy-focused app
- Keychain wrapper library
- Security audit checklist

Practice: Security audit all projects

Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)

The capstone month: spec, build, ship and defend, then the final exam that decides the certificate.

Month 10: Capstone and the Final Exam (Weeks 40-44)
Capstone Spec and Setup
Topics:
- Choosing the capstone: a complete SwiftUI app with real data, networking and Apple-feel polish
- A one-page spec: scope, milestones, stack
- Repository and project skeleton ready on day one
- Instructor sign-off before the build begins
Projects built:
- Approved capstone spec plus the running skeleton
Practice: Pitch the spec to the batch and absorb one hard question
Capstone Build
Topics:
- Two focused weeks of building in vertical slices
- Instructor checkpoints and honest scope cuts
- Using the whole course: Swift depth, SwiftUI, architecture, persistence and testing
- Daily commits and an always-working build
Projects built:
- The capstone, feature-complete with version control
Practice: A working build at the end of every session
Ship and Polish
Topics:
- The finishing pass: edge cases, errors, documentation
- App Store readiness pass: assets, review rules and the honest release checklist
- A README worth reading and a demo worth watching
- A full-course spiral review before the exam
Projects built:
- The capstone shipped, documented and rehearsed
Practice: Two mock demos, each tighter than the last
Demo Day and the Final Exam
Topics:
- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: iOS roles, indie apps, and the wider Apple platforms
- Where our other courses continue the journey
Projects built:
- Demo day presentation delivered to the batch
Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: Swift programs from fundamentals through protocol-oriented tools
- Phase 2: SwiftUI interfaces with real state and navigation
- Phase 3: architected apps with persistence, networking and tests
- Phase 4: the capstone app, App-Store-disciplined and defended at demo day

Total projects: 20+ real builds, crowned by an App-Store-disciplined capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Swift to real depth: optionals, value types, protocols, concurrency
- SwiftUI: declarative interfaces with state done right
- Architecture: structure that survives features and time
- Data and networking: persistence and API clients done properly
- Apple craft: interface habits that make apps feel native

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

  - iOS development roles: SwiftUI-first is exactly where hiring is moving
  - Indie development: the App Store rewards craft like nowhere else
  - The wider Apple ecosystem: watchOS, macOS and visionOS share this foundation
  - Our full-stack course for the server half of app products

EARNING POTENTIAL (share only if asked, and never promise it)

  Market context: The ranges below are general market salary bands for these roles in India and
    abroad, drawn from public industry data. They are shown for career context only and are not a
    promise or guarantee of income. Actual pay depends on your skills, experience, location, and
    the job market.
  Early career roles: Rs 3-6 LPA (Junior iOS Developer / Fresher)
  Growing roles: Rs 6-12 LPA (iOS Developer); Rs 10-18 LPA (Senior iOS Developer)
  Experienced roles: Rs 15-30 LPA (Lead iOS Developer / Mobile Architect); Rs 22-50 LPA
    (Principal Engineer / Solutions Architect)
  Freelance: Rs 2,000-7,000/hour based on expertise
  International usa: $95k-210k USD (iOS Developer to Principal)
  International europe: €55k-140k EUR based on country and experience

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What does this Swift and iOS course cover?
  A: Swift as a real language first, optionals, value types, protocols and structured
  concurrency, then modern iOS with SwiftUI: views and state, navigation, architecture,
  persistence, networking, testing and App Store discipline, ending with a capstone app and the
  final exam. SwiftUI-first throughout, because that is where Apple and hiring are already
  going.

  Q: How long does it take to learn Swift and iOS properly?
  A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two
  more months in hand for learners who need them. The three teaching phases build the language
  and its real-world domain; the finale month ships a capstone and sits the final exam.

  Q: How am I assessed? Is the certificate just for attending?
  A: No. Problem sets follow every class with written feedback, monthly reviews deliberately
  mix new material with earlier topics, each phase ends with a cumulative exam, and the course
  ends with a final exam: a practical build plus written questions from the whole syllabus.
  Passing earns the certificate; falling short earns focused revision and a free retest.

  Q: Do I really need a Mac?
  A: Yes, honestly: Xcode runs only on macOS, and pretending otherwise wastes students' money.
  Realistic options: any Apple-silicon Mac including used M1 machines handles the whole course;
  some students share a family Mac for practice sessions. If a Mac is out of reach right now,
  our React Native-based app course builds cross-platform apps from any laptop, and this course
  will still be here.

  Q: Swift or React Native for mobile?
  A: Native Swift wins when you want the Apple platform mastered deeply, the best performance
  and feel, and access to every new framework the day it ships. React Native wins for
  one-codebase-both-platforms from any laptop. We teach both roads honestly; this is the
  native-depth one.

  Q: Can I try a class before enrolling?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or
  message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with
  no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at
  https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor reading your actual code, never
  pre-recorded videos.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/complete-swift-programming-masterclass-college
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
COURSE 129 OF 137: TRADING AUTOMATION COURSE: BOTS, BACKTESTING & SYSTEMS
==

Course link: https://learn.modernagecoders.com/courses/algorithmic-trading-automation-masterclass
Subject area: Data Science
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)

Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

  Trading automation course for adults and professionals: build, backtest and deploy automated trading systems in Python, with broker APIs, risk controls and production infrastructure, taught as engineering, in 10-12 months.

  In one line: The systems half of quant, done properly: build, backtest and deploy automated trading systems in Python, with broker APIs, risk controls and real production discipline.

ABOUT THE PROGRAMME

  Building a strategy is one skill; running it reliably, unattended, with real safeguards is another, and this course teaches the second one. It is an engineering course, not a money-making pitch: Python for finance, technical and fundamental analysis in code, strategy design and rigorous backtesting, then portfolio theory and risk-management systems, machine learning for trading, and the part most courses skip entirely, connecting to broker APIs, paper trading, and building live systems with monitoring, logging, compliance awareness and the safeguards that stop an automated system doing real damage. Throughout, the message is repeated and honest: markets carry real risk of loss, backtests do not predict the future, and this is education, not financial advice. A finale month builds and paper-deploys a full automated system and sits the final exam. The pace is honest: ten teaching months at two live classes a week plus four to six hours of practice, with two more in hand. Problem sets, monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free retest.

WHAT MAKES THIS COURSE DIFFERENT

  - The systems half done right: from backtest to reliable, monitored, unattended execution
  - Broker APIs and paper trading taught hands-on, the step most courses skip
  - Safeguards as a core topic: the controls that stop an automated system causing harm
  - Production engineering: deployment, monitoring, logging and compliance awareness
  - Honest about risk on every page: education, not financial advice, no profit claims
  - Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam with a free retest

THE LEARNING PATH, STEP BY STEP

  Phase 1: Foundations (Months 1-3): markets, Python for finance, analysis, strategy design and backtesting
  Phase 2: Risk, ML and going live (Months 4-6): portfolio theory, risk systems, ML, broker APIs and paper trading
  Phase 3: Production systems (Months 7-9): institutional concepts, infrastructure, deployment and compliance
  Phase 4: The finale (Month 10): a full paper-deployed automated system and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

  Coding experience: None. Everything is taught from the first line
  Equipment: Any computer with a stable internet connection; every tool used is free
  Age: 18+; college students, graduates and working professionals
  Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

  Developers: Programmers who want to build reliable automated trading systems
  College students: Students targeting quant developer and trading-systems roles
  Quant researchers: Strategy researchers who need the engineering to deploy their ideas
  Fintech bound: Engineers moving into fintech and trading infrastructure
  Disciplined learners: Adults who want real systems skill, not a signals subscription

FULL CURRICULUM, WEEK BY WEEK

  This is the complete teaching plan. Parents can ask about any week.

  PHASE 1: FINANCIAL MARKETS & PYTHON FOUNDATION (MONTHS 1-3, WEEKS 1-13)

  Build solid understanding of financial markets, trading fundamentals, and Python programming for finance.

Month 1: Financial Markets Fundamentals (Weeks 1-5)
  Introduction to Financial Markets
    Topics:
      - What is algorithmic trading? History and evolution
      - Financial markets overview: stocks, bonds, commodities, forex
      - Market participants: retail, institutional, market makers
      - Exchanges and trading venues: NYSE, NASDAQ, NSE, BSE
      - Order types: market, limit, stop-loss, stop-limit
      - Bid-ask spread and market microstructure
      - Trading hours and market sessions
      - Settlement and clearing processes
      - Regulatory environment: SEC, SEBI, regulations
      - Market indices: S&P 500, Nifty, Sensex
      - Economic indicators and their impact
      - Introduction to fundamental analysis
    Projects built:
      - Market research report on major indices
      - Trading account setup (paper trading)
      - Market terminology glossary
    Practice: Daily market observation and journaling
  Trading Basics & Market Mechanics
    Topics:
      - Long vs short positions
      - Margin trading and leverage
      - Market volatility and VIX
      - Trading psychology and behavioral finance
      - Common trading mistakes and biases
      - Position sizing basics
      - Risk-reward ratios
      - Trading costs: commissions, slippage, spreads
      - Dividends and corporate actions
      - Stock splits and bonus shares
      - Market cycles and trends
      - Introduction to derivatives: futures and options
    Projects built:
      - Paper trading diary with 10 trades
      - Risk-reward calculator
      - Trading cost analyzer
    Practice: Execute 20 paper trades with analysis
  Python Setup for Finance
    Topics:
      - Python environment setup for trading
      - Anaconda and Jupyter for finance
      - Essential libraries: NumPy, Pandas
      - Financial data structures in Python
      - Date and time handling for markets
      - Working with financial data types
      - Data cleaning for financial data
      - Basic statistics with Python
      - Visualization with Matplotlib
      - Plotly for interactive charts
      - Git for version control
      - Project structure for trading systems
    Projects built:
      - Python trading environment setup
      - Stock price data analyzer
      - Market statistics calculator
    Practice: Complete 30 Python finance exercises
  Financial Data Acquisition
    Topics:
      - Yahoo Finance API with yfinance
      - Alpha Vantage API integration
      - NSE/BSE data sources
      - Real-time vs historical data
      - WebSocket connections for live data
      - Data storage: CSV, databases, HDF5
      - Time series data management
      - Corporate actions data handling
      - Economic data from FRED
      - Alternative data sources
      - Data quality and validation
      - Building a data pipeline
    Projects built:
      - Multi-source data fetcher
      - Historical data downloader
      - Real-time price tracker
      - Financial database creator
    Practice: Collect data for 100 stocks

Month 2: Technical Analysis & Indicators (Weeks 6-9)
  Classical Technical Analysis
    Topics:
      - Price action and candlestick patterns
      - Support and resistance levels
      - Trend lines and channels
      - Chart patterns: head and shoulders, triangles, flags
      - Moving averages: SMA, EMA, WMA
      - Moving average crossovers
      - Volume analysis and interpretation
      - Market breadth indicators
      - Dow Theory principles
      - Elliott Wave basics
      - Fibonacci retracements and extensions
      - Japanese candlestick patterns
    Projects built:
      - Candlestick pattern detector
      - Support/resistance calculator
      - Moving average strategy implementation
      - Chart pattern recognition system
    Practice: Analyze 50 charts with technical analysis
  Technical Indicators Programming
    Topics:
      - Momentum indicators: RSI, Stochastic
      - MACD and signal lines
      - Bollinger Bands implementation
      - ATR (Average True Range)
      - ADX (Average Directional Index)
      - Volume indicators: OBV, VWAP
      - Ichimoku Cloud components
      - Custom indicator development
      - Indicator optimization
      - Multi-timeframe analysis
      - Indicator backtesting
      - TA-Lib library usage
    Projects built:
      - Technical indicator library
      - Custom indicator creator
      - Multi-indicator dashboard
      - Indicator signal generator
    Practice: Implement 20 technical indicators from scratch
  Fundamental Analysis Automation
    Topics:
      - Financial statements analysis
      - P/E, P/B, and valuation ratios
      - Revenue and earnings analysis
      - Cash flow analysis
      - Debt and leverage metrics
      - ROE, ROA, ROCE calculations
      - Sector comparison metrics
      - Earnings calendar tracking
      - Insider trading data
      - Analyst recommendations scraping
      - News sentiment analysis
      - Fundamental data APIs
    Projects built:
      - Fundamental analysis screener
      - Earnings announcement tracker
      - Financial ratio calculator
      - Company comparison tool
    Practice: Analyze 30 companies fundamentally
  Market Analysis & Screening
    Topics:
      - Stock screening strategies
      - Sector rotation analysis
      - Market regime detection
      - Correlation analysis
      - Beta calculation and usage
      - Relative strength analysis
      - Market sentiment indicators
      - Options flow analysis basics
      - Unusual activity detection
      - Pre-market and after-hours analysis
      - Global market correlations
      - Economic calendar integration
    Projects built:
      - Advanced stock screener
      - Market regime identifier

```
      - Correlation matrix builder
      - Sentiment analysis system
    Practice: Build 10 different screening strategies

Month 3: Strategy Development & Backtesting (Weeks 10-13)
  Trading Strategy Design
    Topics:
      - Types of trading strategies
      - Trend following strategies
      - Mean reversion strategies
      - Momentum strategies
      - Breakout strategies
      - Pairs trading and arbitrage
      - Market making basics
      - Strategy hypothesis formation
      - Entry and exit rules
      - Position sizing methods
      - Stop-loss and take-profit logic
      - Strategy documentation
    Projects built:
      - Moving average crossover strategy
      - RSI mean reversion strategy
      - Breakout trading system
      - Pairs trading strategy
    Practice: Design 10 trading strategies
  Backtesting Frameworks
    Topics:
      - Backtesting concepts and pitfalls
      - Look-ahead bias and survivorship bias
      - Vectorized backtesting with Pandas
      - Event-driven backtesting
      - Backtrader framework
      - Zipline backtesting engine
      - Custom backtesting framework
      - Transaction cost modeling
      - Slippage simulation
      - Market impact modeling
      - Walk-forward analysis
      - Out-of-sample testing
    Projects built:
      - Custom backtesting engine
      - Strategy comparison framework
      - Walk-forward optimizer
      - Monte Carlo simulator
    Practice: Backtest 20 strategies
  Performance Analytics
    Topics:
      - Returns calculation methods
      - Sharpe ratio and risk-adjusted returns
      - Maximum drawdown analysis
      - Calmar and Sortino ratios
      - Win rate and profit factor
      - Value at Risk (VaR)
      - Conditional VaR (CVaR)
      - Performance attribution
      - Benchmark comparison
      - Rolling performance metrics
      - Trade analysis and statistics
      - Performance reporting
    Projects built:
      - Performance analytics dashboard
      - Risk metrics calculator
      - Trade journal analyzer
      - Strategy report generator
    Practice: Analyze 15 strategy performances
  Strategy Optimization
    Topics:
      - Parameter optimization techniques
      - Grid search optimization
      - Random search methods
      - Bayesian optimization
      - Genetic algorithms for trading
      - Machine learning for optimization
      - Overfitting prevention
      - Robust parameter selection
      - Multi-objective optimization
      - Strategy combination methods
      - Ensemble strategies
      - Adaptive strategies
```

      Projects built:
        - Parameter optimizer tool
        - Genetic algorithm optimizer
        - Strategy ensemble system
        - Adaptive strategy framework
      Practice: Optimize 10 strategies
      Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

PHASE 2: ADVANCED STRATEGIES & PORTFOLIO MANAGEMENT (MONTHS 4-6, WEEKS 14-26)

Master portfolio theory, risk management, advanced strategies, and quantitative methods.

  Month 4: Portfolio Theory & Risk Management (Weeks 14-17)
    Modern Portfolio Theory
      Topics:
        - Portfolio theory foundations
        - Expected returns and variance
        - Covariance and correlation in portfolios
        - Efficient frontier construction
        - Capital Asset Pricing Model (CAPM)
        - Beta and systematic risk
        - Alpha generation concepts
        - Multi-factor models
        - Fama-French factors
        - Risk parity strategies
        - Black-Litterman model
        - Portfolio rebalancing strategies
      Projects built:
        - Efficient frontier calculator
        - CAPM implementation
        - Portfolio optimizer
        - Risk parity portfolio builder
      Practice: Build 10 optimal portfolios
    Risk Management Systems
      Topics:
        - Types of financial risk
        - Market risk measurement
        - Credit risk basics
        - Liquidity risk management
        - Operational risk in trading
        - Position sizing techniques
        - Kelly criterion implementation
        - Risk budgeting methods
        - Stress testing portfolios
        - Scenario analysis
        - Risk limits and controls
        - Real-time risk monitoring
      Projects built:
        - Risk management dashboard
        - Position sizing calculator
        - Risk limit monitor
        - Stress testing framework
      Practice: Implement 15 risk management techniques
    Advanced Trading Strategies
      Topics:
        - Statistical arbitrage strategies
        - Cointegration and pairs trading
        - Options strategies: covered calls, spreads
        - Delta hedging techniques
        - Volatility trading strategies
        - Calendar spread strategies
        - Sector rotation strategies
        - Factor-based strategies
        - Smart beta strategies
        - Alternative risk premia
        - Carry trade strategies
        - Global macro strategies
      Projects built:
        - Statistical arbitrage system
        - Options strategy builder
        - Volatility trading bot
        - Factor rotation strategy
      Practice: Implement 20 advanced strategies
    Market Microstructure & Execution
      Topics:
        - Order book dynamics
        - Market depth analysis

```
      - Liquidity provision strategies
      - Optimal execution algorithms
      - TWAP and VWAP execution
      - Implementation shortfall
      - Dark pools and alternative trading
      - High-frequency trading concepts
      - Latency optimization basics
      - Co-location advantages
      - Smart order routing
      - Transaction cost analysis
    Projects built:
      - Order book analyzer
      - VWAP execution algorithm
      - Market impact estimator
      - Smart order router
    Practice: Build 10 execution algorithms

Month 5: Machine Learning for Trading (Weeks 18-22)
  ML Fundamentals for Trading
    Topics:
      - Machine learning in finance overview
      - Supervised learning for trading
      - Feature engineering for markets
      - Technical indicators as features
      - Fundamental data features
      - Alternative data features
      - Train-test splitting for time series
      - Cross-validation for trading
      - Overfitting in financial ML
      - Feature selection methods
      - Data preprocessing for trading
      - Label engineering for trading signals
    Projects built:
      - ML feature engineering pipeline
      - Trading signal predictor
      - Feature importance analyzer
      - ML data preparation tool
    Practice: Engineer 100 trading features
  Classification & Regression for Trading
    Topics:
      - Logistic regression for signals
      - Support Vector Machines for trading
      - Random Forests for prediction
      - Gradient boosting: XGBoost, LightGBM
      - Neural networks for trading
      - Ensemble methods for signals
      - Price prediction models
      - Direction prediction models
      - Volatility forecasting
      - Multi-class classification for signals
      - Probability calibration
      - Model interpretation for trading
    Projects built:
      - ML signal generator
      - Price prediction system
      - Ensemble trading model
      - ML strategy builder
    Practice: Train 20 ML trading models
  Deep Learning for Markets
    Topics:
      - Deep learning in finance
      - LSTM for time series prediction
      - GRU networks for trading
      - CNN for pattern recognition
      - Attention mechanisms for trading
      - Transformer models for markets
      - Autoencoders for anomaly detection
      - GANs for synthetic data
      - Reinforcement learning basics
      - Deep Q-Learning for trading
      - Policy gradient methods
      - Multi-agent trading systems
    Projects built:
      - LSTM price predictor
      - Deep learning trading bot
      - RL trading agent
      - Transformer-based strategy
    Practice: Build 10 deep learning models
  Alternative Data & NLP
```

```
    Topics:
      - Alternative data sources for trading
      - News sentiment analysis
      - Social media sentiment: Twitter, Reddit
      - Earnings call transcripts analysis
      - SEC filings analysis
      - Web scraping for trading signals
      - Satellite data for trading
      - NLP preprocessing for finance
      - Named entity recognition for stocks
      - Topic modeling for market themes
      - Event detection from text
      - Real-time sentiment pipeline
    Projects built:
      - News sentiment trading system
      - Social media signal generator
      - Earnings call analyzer
      - Alternative data pipeline
    Practice: Process 10 alternative data sources
  ML Model Deployment
    Topics:
      - Model deployment strategies
      - Real-time inference for trading
      - Model versioning and management
      - A/B testing trading models
      - Model monitoring in production
      - Drift detection for financial models
      - Model retraining pipelines
      - Feature stores for trading
      - MLOps for trading systems
      - Cloud deployment for ML models
      - Edge deployment for low latency
      - Model governance and compliance
    Projects built:
      - ML model deployment pipeline
      - Model monitoring dashboard
      - Automated retraining system
      - Production ML trading system
    Practice: Deploy 5 ML models to production

Month 6: Live Trading & Production Systems (Weeks 23-26)
  Broker APIs & Connectivity
    Topics:
      - Interactive Brokers API
      - Alpaca Trading API
      - TD Ameritrade API
      - Zerodha Kite Connect
      - FIX protocol basics
      - WebSocket connections for real-time data
      - Order management systems
      - Order routing logic
      - API authentication and security
      - Rate limiting handling
      - Error handling and recovery
      - Connection redundancy
    Projects built:
      - Multi-broker connector
      - Order management system
      - Real-time data aggregator
      - API abstraction layer
    Practice: Connect to 5 different brokers
  Paper Trading Implementation
    Topics:
      - Paper trading environment setup
      - Simulated order execution
      - Paper trading engines
      - Real-time paper trading
      - Strategy deployment for paper trading
      - Performance tracking
      - Paper vs live trading differences
      - Transitioning to live trading
      - Risk controls for paper trading
      - Paper trading best practices
      - Multiple strategy management
      - Portfolio paper trading
    Projects built:
      - Paper trading platform
      - Strategy deployment system
      - Performance tracker
```

- Risk monitor for paper trading
Practice: Paper trade 10 strategies for 30 days
Live Trading Systems
Topics:
- Live trading infrastructure
- Production system architecture
- Order execution engines
- Risk management systems
- Position tracking and P&L
- Real-time monitoring dashboards
- Alerting and notifications
- Fail-safe mechanisms
- Circuit breakers implementation
- Disaster recovery planning
- System redundancy and backup
- Round-the-clock operation considerations
Projects built:
- Live trading engine
- Risk management system
- Monitoring dashboard
- Alert system implementation
Practice: Build complete live trading infrastructure
Cryptocurrency Trading
Topics:
- Cryptocurrency markets overview
- Crypto exchanges: Binance, Coinbase
- Crypto trading APIs
- Round-the-clock trading considerations
- Crypto-specific strategies
- Arbitrage in crypto markets
- DeFi and automated market makers
- Stablecoin strategies
- Cross-exchange arbitrage
- Crypto portfolio management
- Tax implications for crypto
- Regulatory considerations
Projects built:
- Crypto trading bot
- Arbitrage detection system
- DeFi yield optimizer
- Multi-exchange trading platform
Practice: Trade 10 crypto strategies
Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---
PHASE 3: PRODUCTION SYSTEMS AND ADVANCED APPLICATIONS (MONTHS 7-9, WEEKS 27-39)

From working strategies to production-grade systems: advanced instruments, trading infrastructure, cloud deployment, monitoring, compliance awareness and the safeguards that reliable automation demands.

Month 7: Advanced Instruments and Institutional Concepts (Weeks 27-31)
Options Trading Automation
Topics:
- Options fundamentals review
- Greeks: Delta, Gamma, Theta, Vega, Rho
- Options pricing models: Black-Scholes
- Binomial option pricing
- Implied volatility calculation
- Volatility surfaces and skew
- Options market making
- Delta-neutral strategies
- Options arbitrage strategies
- Automated options screening
- Risk management for options
- Options backtesting challenges
Projects built:
- Options pricing calculator
- Greeks dashboard
- Options strategy backtester
- Volatility trading system
Practice: Implement 15 options strategies
Futures & Derivatives Trading
Topics:
- Futures markets overview
- Futures pricing and basis
- Calendar spreads in futures
- Commodity trading strategies

```
      - Interest rate futures
      - Currency futures trading
      - Index futures strategies
      - Futures arbitrage
      - Roll strategies
      - Margin requirements management
      - Delivery and settlement
      - Futures data handling
    Projects built:
      - Futures trading system
      - Spread trading bot
      - Commodity trading strategy
      - Futures arbitrage detector
    Practice: Build 10 futures strategies
  High-Frequency Trading Concepts
    Topics:
      - HFT strategies overview
      - Market making algorithms
      - Statistical arbitrage HFT
      - Latency arbitrage
      - Order book imbalance strategies
      - Tick data processing
      - Ultra-low latency systems
      - FPGA and hardware acceleration
      - Co-location benefits
      - Network optimization
      - HFT risk management
      - Regulatory considerations for HFT
    Projects built:
      - HFT simulator
      - Market making bot
      - Latency measurement tool
      - Order book strategy
    Practice: Simulate 5 HFT strategies
  Fixed Income & Forex Trading
    Topics:
      - Bond markets and pricing
      - Yield curve strategies
      - Duration and convexity
      - Credit spread trading
      - Government bond strategies
      - Corporate bond trading
      - Forex market structure
      - Currency pair strategies
      - Carry trade implementation
      - Forex arbitrage
      - Central bank policy trading
      - Cross-asset strategies
    Projects built:
      - Bond trading system
      - Yield curve analyzer
      - Forex trading bot
      - Carry trade optimizer
    Practice: Develop 10 fixed income/forex strategies
  Quantitative Risk Analytics
    Topics:
      - Advanced risk metrics
      - Expected Shortfall (ES)
      - Stressed VaR
      - Backtesting VaR models
      - Extreme value theory
      - Copula models for risk
      - Credit risk modeling
      - Counterparty risk
      - Liquidity risk modeling
      - Model risk management
      - Risk aggregation methods
      - Regulatory capital calculations
    Projects built:
      - Advanced risk system
      - VaR backtesting framework
      - Credit risk model
      - Risk aggregation platform
    Practice: Implement 10 risk models

Month 8: Advanced Infrastructure & Deployment (Weeks 32-36)
  Trading Infrastructure Design
    Topics:
      - System architecture for trading
```

- Microservices for trading systems
- Message queuing: RabbitMQ, Kafka
- Event-driven architectures
- Database design for trading
- Time-series databases: InfluxDB
- In-memory databases: Redis
- Data lakes for trading
- Distributed systems design
- System scalability patterns
- High availability design
- Disaster recovery planning

Projects built:

- Microservices trading platform
- Event-driven trading system
- Scalable data architecture
- HA trading infrastructure

Practice: Design 5 trading architectures

Cloud Deployment for Trading

Topics:

- AWS for trading systems
- EC2 and auto-scaling
- S3 for data storage
- Lambda for serverless trading
- RDS and DynamoDB for trading data
- Google Cloud Platform for trading
- Azure for financial services
- Kubernetes for trading apps
- Docker containerization
- CI/CD for trading systems
- Infrastructure as Code
- Cloud cost optimization

Projects built:

- Cloud-native trading platform
- Serverless trading functions
- Kubernetes trading deployment
- Multi-cloud trading system

Practice: Deploy to 3 cloud providers

Monitoring & Observability

Topics:

- Monitoring trading systems
- Prometheus and Grafana
- ELK stack for logging
- Application performance monitoring
- Trading metrics and KPIs
- Real-time alerting systems
- Anomaly detection in monitoring
- Distributed tracing
- Performance profiling
- System health checks
- SLA monitoring
- Incident response procedures

Projects built:

- Monitoring dashboard
- Alert management system
- Performance analyzer
- Incident response toolkit

Practice: Monitor 10 trading systems

Compliance & Regulatory

Topics:

- Financial regulations overview
- MiFID II requirements
- Best execution obligations
- Market abuse regulations
- Trade reporting requirements
- Audit trail maintenance
- Data retention policies
- KYC and AML in trading
- Risk disclosure requirements
- Compliance monitoring systems
- Regulatory reporting automation
- License requirements

Projects built:

- Compliance monitoring system
- Audit trail generator
- Regulatory reporting tool
- Risk disclosure system

Practice: Implement compliance for 5 systems

Testing & Quality Assurance

Topics:

- Testing strategies for trading systems
- Unit testing trading logic
- Integration testing
- End-to-end testing
- Performance testing
- Stress testing systems
- Chaos engineering for trading
- Test data management
- Mock trading environments
- Continuous testing
- Test automation frameworks
- Quality metrics

Projects built:
- Testing framework for trading
- Stress testing suite
- Mock trading environment
- Quality assurance dashboard

Practice: Test 10 trading systems comprehensively

Month 9: Advanced Applications and Market Making (Weeks 37-39)

Advanced Machine Learning Applications

Topics:
- Deep reinforcement learning for trading
- Multi-agent reinforcement learning
- Evolutionary algorithms for strategy discovery
- Genetic programming for trading rules
- Neural architecture search for trading
- Transfer learning in finance
- Few-shot learning for markets
- Online learning for adaptation
- Federated learning for trading
- Explainable AI for trading
- Causal inference in markets
- Quantum computing for finance basics

Projects built:
- Advanced RL trading agent
- Strategy evolution system
- Adaptive learning trader
- Explainable AI dashboard

Practice: Implement 10 advanced ML techniques

Market Making & Liquidity Provision

Topics:
- Market making strategies
- Inventory management
- Quote generation algorithms
- Spread optimization
- Adverse selection mitigation
- Order book modeling
- Queue position strategies
- Cross-market making
- Options market making
- Risk management for market makers
- Regulatory requirements for market makers
- Market maker profitability

Projects built:
- Market making simulator
- Quote generation engine
- Inventory manager
- Market maker risk system

Practice: Build 5 market making strategies

Alternative Trading Venues

Topics:
- Dark pools and ATSs
- Decentralized exchanges (DEXs)
- Automated market makers (AMMs)
- Liquidity pools and yield farming
- Flash loans and arbitrage
- Cross-chain trading
- NFT trading automation
- Prediction markets trading
- Binary options automation
- Social trading platforms
- Copy trading systems
- Signal provider systems

Projects built:
- DEX trading bot
- AMM strategy implementation
- Cross-chain arbitrage bot
- Social trading platform

Practice: Trade on 5 alternative venues
Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)

The capstone month: spec, build, ship and defend, then the final exam that decides the certificate.

Month 10: Capstone and the Final Exam (Weeks 40-44)

Capstone Spec and Setup

Topics:

- Choosing the capstone: a full automated trading system, paper-deployed: strategy, risk controls, broker connectivity, monitoring and safeguards
- A one-page spec: scope, milestones, stack
- Repository and skeleton ready on day one
- Instructor sign-off before the build begins

Projects built:

- Approved capstone spec plus the running skeleton

Practice: Pitch the spec to the batch and absorb one hard question

Capstone Build

Topics:

- Two focused weeks in vertical slices
- Instructor checkpoints and honest scope cuts
- Using the whole course: Python engineering, backtesting, risk systems, broker APIs and production monitoring
- Daily commits and an always-working build

Projects built:

- The capstone, feature-complete with version control

Practice: A working build at the end of every session

Ship and Polish

Topics:

- The finishing pass: edge cases, errors, documentation
- Paper-deployed and monitored: a system that runs unattended with real safeguards, on paper trading
- A README worth reading and a demo worth watching
- A full-course spiral review before the exam

Projects built:

- The capstone shipped, documented and rehearsed

Practice: Two mock demos, each tighter than the last

Demo Day and the Final Exam

Topics:

- Presenting the capstone: live demo plus the hardest bug story
- The honest map of next roads: quant developer, trading-systems engineer or fintech roles; trading your own capital is a personal decision, not a placement
- Where our other courses continue the journey

Projects built:

- Demo day presentation delivered to the batch

Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: analysis tools, indicators and backtesting frameworks in Python
- Phase 2: risk systems, ML models and paper-trading connectivity
- Phase 3: production infrastructure with monitoring and compliance awareness
- Phase 4: the full paper-deployed automated-system capstone

Total projects: 15+ engineering projects, crowned by a paper-deployed automated system

SKILLS THE STUDENT WILL HAVE AT THE END

- Python engineering for finance: data, indicators and backtesting frameworks
- Risk-management systems and portfolio construction in code
- Broker API integration and paper trading
- Production systems: deployment, monitoring, logging and safeguards
- Compliance awareness and the discipline of unattended automation

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Quant developer and trading-systems engineer roles in funds and fintech
- Our algorithmic trading course for the research and strategy half
- General backend and data-engineering roles: the skills transfer directly
- Trading your own capital is a personal financial decision, made with full awareness of the risk

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 5-8 LPA (Junior Quant Analyst)
Growing roles: Rs 10-15 LPA (Algo Trader); Rs 15-25 LPA (Quant Developer)
Experienced roles: Rs 20-50+ LPA (Senior Quant Trader)
Freelance: Rs 2,000-10,000/hour consulting
International: $80k-300k+ USD based on location

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this trading automation course cover?
A: The systems half of quant trading, taught as engineering: Python for finance, technical and fundamental analysis in code, strategy design and rigorous backtesting, portfolio theory and risk-management systems, machine learning for trading, then connecting to broker APIs, paper trading, and building live systems with monitoring, logging, compliance awareness and safeguards. A finale month builds and paper-deploys a full automated system and sits the final exam. It is education, not financial advice.

Q: How long does it take to learn trading automation properly?
A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two more months in hand for learners who need them. The three teaching phases build the skill and its real-world application; the finale month ships a capstone and sits the final exam.

Q: How am I assessed? Is the certificate just for attending?
A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Is this financial advice? Will I make guaranteed profits?
A: No, and be wary of anyone who says otherwise. This is an engineering and quantitative course: it teaches you to research, build, backtest and automate trading systems, not what to buy or a way to get rich. Markets carry real risk of loss, backtest results do not guarantee future returns, and we make no profit claims of any kind. What you gain is genuine, in-demand skill; what you do with real money afterwards is your own decision and your own risk.

Q: How is this different from your algorithmic trading course?
A: This course is the systems-and-execution half: how to take a strategy and run it reliably and safely through broker APIs, with monitoring, safeguards and production discipline. The algorithmic trading course is the research half: what to trade, why, and how to backtest it honestly. Many serious learners take both; that one makes you the researcher, this one makes you the systems engineer.

Q: Do I need finance or trading experience to start?
A: No. The markets knowledge and the maths you need are taught from the ground up. This course leans slightly more toward software engineering, so comfort with programming logic helps, but everything is taught step by step from the first class.

Q: Can I try a class before enrolling?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real instructor reading your actual code, never
  pre-recorded videos.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link:
https://learn.modernagecoders.com/courses/algorithmic-trading-automation-masterclass
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
COURSE 130 OF 137: TYPESCRIPT COURSE: TYPED JAVASCRIPT, REACT AND NODE
==

Course link: https://learn.modernagecoders.com/courses/complete-typescript-programming-masterclass-college
Subject area: Web Development
Designed for: College students
Who can join: Complete beginner (18+); college students, graduates and working professionals
Course length: 10-12 months (44-52 weeks)
Time needed: 2 live classes/week + 4-6 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one
  live class a week, four a month; one to one outside India two live classes a week, eight a
  month. Each class is one hour.

WHAT THIS COURSE IS

  TypeScript course for college students and professionals: JavaScript foundations, the type
  system to depth, then typed React and Node in 10-12 months.

  In one line: The language modern teams actually write: JavaScript foundations, the type system as a superpower, then typed React
and Node end to end.

ABOUT THE PROGRAMME

  TypeScript won: new frontend and most serious Node work is written in it, and the type
  system, learned properly, is not bureaucracy but a superpower: bugs caught at compile time,
  refactors that stop being scary, editors that finish your thoughts. This live online
  masterclass builds it honestly: JavaScript foundations first (TypeScript is JavaScript
  underneath), then the type system to real depth, generics, narrowing, utility types, the
  honest limits, then typed application development: React with TypeScript the way product
  teams write it, and Node backends with types end to end. A finale month ships a full-stack
  typed capstone and sits the final exam. The pace is honest: ten teaching months at two live
  classes a week plus four to six hours of practice, with two more in hand. Problem sets,
  monthly mixed reviews, phase exams, and a final exam that decides the certificate with a free
  retest.

WHAT MAKES THIS COURSE DIFFERENT

  - JavaScript first, honestly: TypeScript without JS depth is decoration
  - The type system as a skill: generics and narrowing understood, not copy-pasted
  - Typed both ends: React and Node with types flowing end to end
  - Real-team habits: strictness levels, migration judgment, when any is a lie
  - An honest 10-month arc with 2 months in hand
  - Real assessment: weekly problem sets, monthly mixed reviews, phase exams and a final exam
    with a free retest

THE LEARNING PATH, STEP BY STEP

  Phase 1: Foundations (Months 1-3): JavaScript properly, then TypeScript fundamentals
  Phase 2: Typed frontend (Months 4-6): React with TypeScript as product teams write it
  Phase 3: Typed backend (Months 7-9): Node and Express with types end to end
  Phase 4: The finale (Month 10): a full-stack typed capstone and the final exam

WHAT A STUDENT NEEDS BEFORE STARTING

  Coding experience: None. Everything is taught from the first line
  Equipment: Any computer with a stable internet connection; every tool used is free
  Age: 18+; college students, graduates and working professionals
  Time commitment: 2 live classes plus 4-6 hours of practice a week

WHO THIS COURSE IS FOR

  Js developers: JavaScript developers making the move every team already made
  College students: Students who want the hiring default, not the legacy path

Self taught: Tutorial-experienced learners ready for the discipline layer
Backend bound: Developers who want typed Node without the framework fog
Career switchers: Professionals entering web development at the modern baseline

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION & TYPESCRIPT FUNDAMENTALS (MONTHS 1-3, WEEKS 1-13)

Build rock-solid JavaScript and TypeScript fundamentals. Learn programming logic, master TypeScript syntax, type systems, and create your first applications.

Months 1-2: JavaScript & TypeScript Fundamentals (Weeks 1-7)

Introduction to Programming & JavaScript Basics

Topics:

- What is programming? Introduction to software development
- JavaScript ecosystem: browsers, Node.js, runtime environments
- Why TypeScript? Benefits over plain JavaScript
- Installing Node.js and npm package manager
- Setting up development environment: VS Code with TypeScript extensions
- Understanding IDE features: IntelliSense, debugging, extensions
- Your first JavaScript program: Hello World
- JavaScript program structure: statements, expressions
- Running JavaScript in browser console and Node.js
- Comments: single-line (//), multi-line (/* */), documentation
- JavaScript coding conventions and naming standards
- Console methods: log, error, warn, table
- Introduction to ECMAScript versions (ES6+)

Projects built:

- Hello World variations with personalized messages
- Simple output programs using console.log
- Personal information display program
- ASCII art generator in JavaScript
- Interactive greeting program

Practice: Daily: 30 min JavaScript syntax practice, write 5-10 simple programs

Variables, Data Types & Operators

Topics:

- Variables: var, let, const - differences and best practices
- Primitive data types: string, number, boolean, null, undefined, symbol, bigint
- Type coercion: implicit and explicit conversion
- typeof operator for type checking
- Template literals and string interpolation
- Numbers: integers, floats, Infinity, NaN
- Operators: arithmetic (+, -, *, /, %, **, ++, --)
- Relational operators: ==, ===, !=, !==, >, <, >=, <=
- Logical operators: &&, ||, !, nullish coalescing (??)
- Assignment operators: =, +=, -=, *=, /=, %=
- Bitwise operators: &, |, ^, ~, <<, >>, >>>
- Ternary operator: ? :
- Operator precedence and associativity
- Optional chaining operator (?.) and its uses

Projects built:

- Simple calculator (four basic operations)
- Temperature converter (Celsius, Fahrenheit, Kelvin)
- Age calculator with detailed output
- BMI calculator with health categories
- Currency converter with exchange rates
- Area and perimeter calculator for shapes
- Compound interest calculator

Practice: Solve 30 problems on variables, data types, and operators

Control Flow & Functions

Topics:

- Boolean expressions and truthy/falsy values
- If statement: single condition execution
- If-else statement: two-way branching
- If-else-if ladder: multiple conditions
- Nested if statements
- Switch statement: multi-way branching
- Switch with fall-through and break
- While loop: pre-tested loop
- Do-while loop: post-tested loop
- For loop: counter-based iteration
- For-of loop: iterating arrays/strings
- For-in loop: iterating object properties
- Break and continue statements
- Labeled statements for nested loops
- Function declarations and expressions

- Arrow functions: concise syntax
- Parameters, arguments, default parameters
- Rest parameters and spread operator
- Return statement and return values
- Function scope and hoisting

Projects built:

- Number guessing game with attempts limit
- Grade calculator with letter grades
- Even/odd checker and list generator
- Prime number checker and generator
- Multiplication table generator
- Factorial calculator (iterative and recursive)
- Fibonacci series generator
- Pattern printing (stars, numbers, pyramids)
- FizzBuzz challenge
- Menu-driven calculator
- Simple ATM simulation

Practice: Solve 40 control flow and function problems

Arrays, Objects & ES6+ Features

Topics:

- Arrays: declaration, creation, initialization
- Array indexing and accessing elements
- Array methods: push, pop, shift, unshift, splice
- Array iteration: forEach, map, filter, reduce
- Array methods: find, findIndex, some, every, includes
- Array sorting and reversing
- Spread operator with arrays
- Destructuring arrays
- Objects: creation and property access
- Object methods and 'this' keyword
- Object shorthand properties and methods
- Computed property names
- Object destructuring
- Spread operator with objects
- Object.keys(), Object.values(), Object.entries()
- JSON: parse and stringify
- Classes: declaration, constructor, methods
- Class inheritance: extends and super
- Static methods and properties
- Getters and setters

Projects built:

- Array statistics calculator (sum, average, min, max)
- Todo list with array operations
- Linear search and binary search implementation
- Sorting algorithms: bubble sort, selection sort
- Student grade management system
- Contact book with objects
- Shopping cart implementation
- Product inventory system
- Simple class-based game (Rock Paper Scissors)

Practice: Create 25 programs using arrays and objects, solve 30 problems

Month 3: TypeScript Core & Type System (Weeks 8-13)

TypeScript Fundamentals & Setup

Topics:

- Introduction to TypeScript: static typing benefits
- TypeScript vs JavaScript: key differences
- Installing TypeScript globally and locally
- tsconfig.json configuration file
- Compiler options: target, module, strict mode
- Basic types: string, number, boolean
- Type annotations and type inference
- Arrays with type annotations
- Tuple types for fixed-length arrays
- Enum types: numeric and string enums
- Any type: when and when not to use
- Unknown type: safe alternative to any
- Void, null, and undefined types
- Never type for unreachable code
- Type assertions: as and angle-bracket syntax
- Literal types: string, number, boolean literals
- Union types: combining multiple types
- Type narrowing and type guards

Projects built:

- Convert JavaScript projects to TypeScript
- Type-safe calculator application
- Enum-based status system
- Student management with typed objects
- Configuration manager with TypeScript

```
        - Simple CLI tool with TypeScript
      Practice: Convert 10 JavaScript programs to TypeScript with proper types
    Interfaces, Type Aliases & Advanced Types
      Topics:
        - Interface declaration and usage
        - Optional properties with ?
        - Readonly properties
        - Index signatures for dynamic properties
        - Interface extending other interfaces
        - Implementing interfaces in classes
        - Type aliases with 'type' keyword
        - Interface vs type alias: when to use which
        - Intersection types: combining types with &
        - Discriminated unions for type safety
        - Type guards: typeof, instanceof, in operator
        - User-defined type guards with 'is' keyword
        - Nullable types and strict null checks
        - Optional chaining with types
        - Nullish coalescing with types
        - Function types and signatures
        - Call signatures in interfaces
        - Function overloading in TypeScript
      Projects built:
        - Type-safe user management system
        - Product catalog with interfaces
        - API response types modeling
        - Form validation with types
        - State management with discriminated unions
        - Plugin system with interface contracts
        - Event system with typed events
      Practice: Build 15 projects using interfaces and advanced types
    Generics & Advanced TypeScript
      Topics:
        - Introduction to generics: type parameters
        - Generic functions and type inference
        - Generic interfaces
        - Generic classes
        - Generic constraints with extends
        - Multiple type parameters
        - Default type parameters
        - Generic utility types: Partial, Required, Readonly
        - Pick, Omit, Record utility types
        - Extract, Exclude, NonNullable types
        - ReturnType and Parameters utility types
        - Conditional types with extends and infer
        - Mapped types for transforming types
        - Template literal types
        - Keyof operator for type keys
        - Typeof operator in type context
        - Module systems: ES modules in TypeScript
        - Declaration files (.d.ts) basics
      Projects built:
        - Generic data structure library (Stack, Queue, LinkedList)
        - Type-safe API client with generics
        - Generic form handler
        - State container with generic types
        - Generic validation library
        - PHASE 1 MINI CAPSTONE: CLI Task Manager
        - Features: Tasks, categories, priorities, file persistence, full TypeScript typing
      Practice: Create 15 generic utilities, solve 30 generics problems
      Assessment: Phase 1 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

----------------------------------------------------------------
PHASE 2: FRONTEND MASTERY WITH REACT & TYPESCRIPT (MONTHS 4-6, WEEKS 14-26)
----------------------------------------------------------------
Master modern frontend development with React, state management, testing, and build
production-ready user interfaces.

  Months 4-5: React Fundamentals & Advanced Patterns (Weeks 14-21)
    HTML, CSS & DOM Fundamentals
      Topics:
        - HTML5 semantic elements and structure
        - Forms, inputs, and validation attributes
        - CSS fundamentals: selectors, specificity, cascade
        - CSS Box Model and layout basics
        - Flexbox for one-dimensional layouts
        - CSS Grid for two-dimensional layouts
        - Responsive design with media queries
```

- CSS custom properties (variables)
- CSS animations and transitions
- DOM manipulation: selecting elements
- DOM events: click, submit, input, keyboard
- Event delegation and bubbling
- Creating and modifying DOM elements
- Local Storage and Session Storage
- Fetch API for HTTP requests

Projects built:

- Responsive portfolio website
- Interactive form with validation
- Todo app with DOM manipulation
- Image gallery with lightbox
- Weather app using Fetch API
- Quiz application
- Calculator with UI
- Responsive navigation menu

Practice: Build 10 interactive web pages with DOM manipulation

React Fundamentals & JSX

Topics:

- Introduction to React: component-based architecture
- TypeScript with Vite
- Vite for React TypeScript projects
- JSX syntax and expressions
- Components: functional components
- Props: passing data to components
- TypeScript with React: typing props
- Children prop and composition
- Conditional rendering techniques
- Lists and keys in React
- Event handling in React
- TypeScript event types
- Styling: CSS modules, styled-components
- Tailwind CSS with React
- Component organization and file structure

Projects built:

- Hello World React app with TypeScript
- Component library with props
- Card component with typed props
- Product listing page
- Navigation component
- Footer component
- Button component variants
- Simple landing page

Practice: Build 15 reusable React components with TypeScript

React Hooks & State Management

Topics:

- useState hook for local state
- Typing useState with TypeScript
- useEffect hook for side effects
- Cleanup functions in useEffect
- useRef hook for DOM references
- useMemo for expensive calculations
- useCallback for memoizing functions
- useReducer for complex state logic
- Typing useReducer actions and state
- Custom hooks creation
- Rules of hooks
- useContext for prop drilling solution
- Creating typed context providers
- React.memo for component memoization
- Performance optimization strategies

Projects built:

- Counter app with useState
- Todo list with useReducer
- Form handling with hooks
- Data fetching with useEffect
- Theme switcher with useContext
- Custom useFetch hook
- Custom useLocalStorage hook
- Debounced search with useMemo

Practice: Create 10 custom hooks with TypeScript

Advanced React Patterns

Topics:

- Compound components pattern
- Render props pattern
- Higher-Order Components (HOCs)
- Custom hooks vs HOCs vs render props
- Controlled vs uncontrolled components

- Form libraries: React Hook Form
- Typing forms with TypeScript
- Form validation with Zod
- Error boundaries for error handling
- Suspense for data fetching
- React.lazy for code splitting
- Portal for modals and tooltips
- Forwarding refs with forwardRef
- useImperativeHandle hook
- Accessibility (a11y) in React

Projects built:
- Modal component with portals
- Dropdown menu with compound components
- Form with React Hook Form and Zod
- Error boundary wrapper
- Lazy loaded routes
- Accessible form components
- Tooltip component
- Tabs component with compound pattern

Practice: Implement all advanced patterns in projects

React Router & Navigation

Topics:
- React Router v6 fundamentals
- BrowserRouter and Route components
- Link and NavLink for navigation
- Route parameters and useParams
- Query strings and useSearchParams
- Nested routes and Outlet
- Protected routes implementation
- useNavigate for programmatic navigation
- useLocation hook
- Route guards and authentication
- 404 and error pages
- Code splitting with routes
- Scroll restoration
- Route transitions and animations

Projects built:
- Multi-page application with routing
- Dashboard with nested routes
- Protected admin routes
- Blog with dynamic routes
- E-commerce product pages
- User profile with tabs

Practice: Build 5 multi-page applications with React Router

Month 6: State Management & Testing (Weeks 22-26)

Global State Management

Topics:
- State management overview and options
- Redux Toolkit fundamentals
- Creating slices with createSlice
- Redux store configuration
- useSelector and useDispatch hooks
- TypeScript with Redux Toolkit
- Async logic with createAsyncThunk
- RTK Query for data fetching
- Zustand: lightweight alternative
- Zustand with TypeScript
- Jotai: atomic state management
- React Query/TanStack Query
- Caching and background refetching
- Optimistic updates
- Comparing state management solutions

Projects built:
- Todo app with Redux Toolkit
- Shopping cart with Redux
- User authentication state
- Data fetching with RTK Query
- Blog with TanStack Query
- Real-time dashboard with Zustand
- Complex form with state management

Practice: Implement state management in all previous projects

Testing React Applications

Topics:
- Testing pyramid: unit, integration, e2e
- Jest testing framework setup
- React Testing Library fundamentals
- Rendering components for testing
- Querying elements: getBy, findBy, queryBy

    - User event simulation
    - Testing forms and inputs
    - Testing async operations
    - Mocking API calls with MSW
    - Testing hooks with renderHook
    - Testing context providers
    - Snapshot testing
    - Code coverage reports
    - Cypress for E2E testing
    - Playwright introduction
  Projects built:
    - Test suite for component library
    - Form testing with user events
    - API mocking with MSW
    - E2E tests with Cypress
    - Integration tests for features
    - Accessibility testing
  Practice: Write tests for all React components, aim for 80%+ coverage
Build Tools & Performance
  Topics:
    - Webpack fundamentals and configuration
    - Vite: modern build tool
    - Environment variables management
    - Bundle analysis and optimization
    - Tree shaking for dead code elimination
    - Code splitting strategies
    - Lazy loading components and routes
    - Image optimization
    - React DevTools profiler
    - Performance measurement with Web Vitals
    - React.memo and useMemo optimization
    - Virtualization for long lists
    - Service workers basics
    - PWA fundamentals
    - Lighthouse audits
  Projects built:
    - Optimized production build
    - Bundle size optimization
    - Performance-optimized dashboard
    - Virtualized list component
    - PWA conversion of existing app
    - Image gallery with lazy loading
  Practice: Optimize all projects for production performance
UI Libraries & Design Systems
  Topics:
    - Component libraries overview
    - Material-UI (MUI) with TypeScript
    - Chakra UI fundamentals
    - Ant Design components
    - Headless UI for unstyled components
    - Radix UI primitives
    - Shadcn/ui component system
    - Theme customization
    - Design tokens and theming
    - Dark mode implementation
    - Responsive design patterns
    - Animation libraries: Framer Motion
    - Building design systems
    - Storybook for component documentation
    - Component documentation best practices
  Projects built:
    - Dashboard with MUI components
    - Landing page with Chakra UI
    - Custom design system
    - Storybook documentation
    - Animated page transitions
    - Dark mode toggle implementation
  Practice: Build 5 projects with different UI libraries
Phase 2 Capstone Project
  Topics:
    - Full frontend application architecture
    - Component design and organization
    - State management implementation
    - Routing and navigation
    - Form handling and validation
    - API integration
    - Testing strategy
    - Performance optimization
  Projects built:

- PHASE 2 CAPSTONE: E-commerce Frontend
- Features: Product catalog, cart, wishlist, user auth UI, checkout flow, responsive design, dark mode, comprehensive testing
- Alternative: Social Media Dashboard (posts, feeds, profiles, messaging UI)
- Alternative: Project Management Tool (boards, tasks, drag-drop, real-time updates)
- Alternative: Learning Platform Frontend (courses, progress, quizzes, video player)

Assessment: Phase 2 exam: a practical build defended live, plus a written paper mixing this phase with everything before it

---

PHASE 3: BACKEND ENGINEERING WITH NODE.JS & TYPESCRIPT (MONTHS 7-9, WEEKS 27-39)

Master server-side development with Node.js, Express, databases, REST APIs, authentication, and production deployment.

Month 7: Node.js & Express Fundamentals (Weeks 27-31)

Node.js Fundamentals

Topics:

- Node.js architecture: event loop, non-blocking I/O
- Node.js vs browser JavaScript
- npm and package.json management
- CommonJS vs ES modules in Node
- TypeScript setup for Node.js
- Built-in modules: fs, path, os, url
- File system operations: read, write, delete
- Streams and buffers
- Event emitter pattern
- Process and environment variables
- Child processes and workers
- Nodemon for development
- ts-node for TypeScript execution
- Debugging Node.js applications
- Error handling in Node.js

Projects built:

- File system CLI tool
- Directory watcher application
- Log file parser
- Simple HTTP server from scratch
- Stream-based file processor
- Environment config manager
- Task runner CLI

Practice: Build 10 Node.js utility scripts

Express.js Framework

Topics:

- Introduction to Express.js
- Express with TypeScript setup
- Routing: GET, POST, PUT, DELETE, PATCH
- Route parameters and query strings
- Request and response objects
- Middleware concept and implementation
- Built-in middleware: json, urlencoded, static
- Custom middleware creation
- Error handling middleware
- Router for modular routes
- Request validation with express-validator
- File uploads with Multer
- CORS configuration
- Helmet for security headers
- Rate limiting with express-rate-limit

Projects built:

- Basic REST API server
- User CRUD API
- File upload service
- API with validation
- Middleware chain implementation
- Logging middleware
- Request timing middleware

Practice: Build 8 Express APIs with proper middleware

REST API Design & Best Practices

Topics:

- REST architecture principles
- Resource naming conventions
- HTTP status codes usage
- Request/response design
- API versioning strategies
- Pagination implementation
- Filtering and sorting
- HATEOAS principles
- API documentation with Swagger/OpenAPI

```
      - TypeScript types for API contracts
      - Zod for runtime validation
      - Error response standardization
      - API testing with Postman/Insomnia
      - API testing with supertest
      - Contract-first API design
    Projects built:
      - RESTful Blog API
      - E-commerce Product API
      - API with Swagger documentation
      - Paginated list API
      - Filter and search API
      - API test suite with supertest
    Practice: Design and document 5 production-ready APIs
  Database Fundamentals - SQL
    Topics:
      - Relational database concepts
      - SQL fundamentals: SELECT, INSERT, UPDATE, DELETE
      - WHERE, ORDER BY, LIMIT, OFFSET
      - JOINs: INNER, LEFT, RIGHT, FULL
      - Aggregations: COUNT, SUM, AVG, GROUP BY
      - Database design and normalization
      - Primary keys and foreign keys
      - Indexes for performance
      - PostgreSQL setup and usage
      - Node.js with pg driver
      - Connection pooling
      - Parameterized queries for security
      - Transactions in Node.js
      - Database migrations concept
      - SQL injection prevention
    Projects built:
      - User management with PostgreSQL
      - Blog with posts and comments
      - E-commerce products and orders
      - Library management system
      - Transaction-based banking API
      - Database migration scripts
    Practice: Build 8 database-connected applications
  ORMs - Prisma & Drizzle
    Topics:
      - ORM concepts and benefits
      - Prisma setup and configuration
      - Prisma schema definition
      - Prisma Client for queries
      - CRUD operations with Prisma
      - Relations: one-to-one, one-to-many, many-to-many
      - Prisma migrations
      - Prisma with TypeScript
      - Drizzle ORM introduction
      - Drizzle schema and queries
      - Type-safe queries with Drizzle
      - Raw SQL with ORMs
      - Seeding databases
      - ORM performance considerations
      - Choosing between ORMs
    Projects built:
      - Blog API with Prisma
      - User authentication with Prisma
      - E-commerce backend with Prisma
      - Multi-tenant application
      - Complex relations example
      - Database seeding scripts
    Practice: Convert all SQL projects to use Prisma

Month 8: Production Backend: Observability, Security & Deployment (Weeks 32-36)
  Authentication & Authorization
    Topics:
      - Authentication vs authorization
      - Password hashing with bcrypt
      - JWT (JSON Web Tokens) fundamentals
      - Access tokens and refresh tokens
      - JWT implementation in Express
      - Passport.js authentication strategies
      - Session-based authentication
      - Cookie security: httpOnly, secure, sameSite
      - Role-based access control (RBAC)
      - Permission-based authorization
      - OAuth 2.0 and OpenID Connect
```

- Social login: Google, GitHub, Facebook
- Two-factor authentication (2FA)
- Password reset flow
- Security best practices

Projects built:

- User registration and login API
- JWT authentication system
- Refresh token rotation
- Role-based API access
- OAuth social login
- Password reset with email
- 2FA implementation

Practice: Secure all APIs with authentication

NoSQL Databases - MongoDB

Topics:

- NoSQL database concepts
- MongoDB architecture and documents
- MongoDB Atlas cloud setup
- Mongoose ODM for Node.js
- Mongoose schemas and models
- TypeScript with Mongoose
- CRUD operations with Mongoose
- Mongoose middleware (hooks)
- Population for references
- Aggregation pipeline
- Indexing in MongoDB
- Schema validation
- Embedded vs referenced documents
- MongoDB transactions
- SQL vs NoSQL: when to use which

Projects built:

- Blog with MongoDB and Mongoose
- Social media posts API
- Real-time chat data model
- Analytics data storage
- Product catalog with nested data
- User activity tracking

Practice: Build 6 MongoDB-backed applications

Caching, Sessions & Real-time

Topics:

- Caching strategies overview
- Redis: in-memory data store
- Redis with Node.js (ioredis)
- Caching API responses
- Session storage with Redis
- Rate limiting with Redis
- Pub/Sub with Redis
- WebSocket fundamentals
- Socket.io for real-time communication
- TypeScript with Socket.io
- Real-time events and rooms
- Broadcasting messages
- Server-Sent Events (SSE)
- Long polling vs WebSocket
- Real-time architecture patterns

Projects built:

- Cached API responses
- Session management with Redis
- Real-time chat application
- Live notifications system
- Collaborative document editing
- Real-time dashboard updates
- Live auction system

Practice: Add real-time features to existing projects

Email, File Storage & External Services

Topics:

- Email sending with Nodemailer
- Email templates with EJS/Handlebars
- Transactional email services: SendGrid, Resend
- Queue-based email sending
- File storage strategies
- AWS S3 for object storage
- Cloudinary for image management
- File upload to cloud storage
- Image processing with Sharp
- PDF generation in Node.js
- Payment integration: Stripe
- Webhook handling
- Third-party API integration

- API rate limiting and retries
- Error handling for external services

Projects built:
- Email notification service
- Welcome email flow
- Password reset emails
- File upload to S3
- Image resize and optimization
- Payment checkout flow
- Invoice PDF generation

Practice: Integrate 5 external services

Testing Backend Applications

Topics:
- Backend testing strategies
- Unit testing with Jest
- Testing Express routes
- Supertest for HTTP testing
- Mocking database calls
- Integration testing setup
- Test database management
- Factory patterns for test data
- Testing authentication flows
- Testing async operations
- Code coverage for backend
- CI/CD testing pipeline
- Load testing with Artillery
- Performance testing basics
- End-to-end API testing

Projects built:
- Unit test suite for services
- Integration tests for APIs
- Authentication flow tests
- Database operation tests
- Load testing scripts
- CI/CD with automated tests

Practice: Write comprehensive tests for all APIs

Month 9: Production Backend: Observability, Security & Deployment (Weeks 37-39)

Logging, Monitoring & Observability

Topics:
- Logging best practices
- Winston logger configuration
- Pino for high-performance logging
- Log levels and formatting
- Structured logging (JSON)
- Request logging and tracing
- Application metrics with Prometheus
- Custom metrics collection
- Health check endpoints
- Error tracking with Sentry
- APM (Application Performance Monitoring)
- Distributed tracing basics
- OpenTelemetry introduction
- Monitoring dashboards
- Alerting strategies

Projects built:
- Centralized logging setup
- Metrics endpoint implementation
- Health check system
- Error tracking integration
- Performance monitoring
- Alerting rules configuration

Practice: Add observability to all backend services

Security & API Protection

Topics:
- OWASP Top 10 for Node.js
- SQL/NoSQL injection prevention
- XSS prevention strategies
- CSRF protection
- Security headers with Helmet
- Input sanitization
- Rate limiting implementation
- API key management
- Secrets management
- Environment security
- Dependency vulnerability scanning
- npm audit and fixes
- HTTPS and TLS
- Content Security Policy

```
        - Secure file uploads
      Projects built:
        - Security-hardened API
        - Rate limiting system
        - API key authentication
        - Vulnerability scanning setup
        - Security audit checklist
        - Penetration testing basics
      Practice: Security audit all backend applications
    Docker & Deployment
      Topics:
        - Docker fundamentals
        - Dockerfile for Node.js apps
        - Multi-stage builds
        - Docker Compose for development
        - Container orchestration basics
        - Environment configuration
        - Deploying to VPS (DigitalOcean, Linode)
        - Process managers: PM2
        - Nginx reverse proxy
        - SSL certificates with Let's Encrypt
        - CI/CD with GitHub Actions
        - Deployment to Railway, Render
        - Zero-downtime deployments
        - Database backups
        - Production checklist
      Projects built:
        - Dockerized Node.js app
        - Docker Compose setup
        - CI/CD pipeline creation
        - VPS deployment
        - Nginx configuration
        - Automated deployments
      Practice: Deploy all applications to production
      Assessment: Phase 3 exam: a practical build defended live, plus a written paper mixing
      this phase with everything before it

---------------------------------------------------------------
PHASE 4: THE FINALE (MONTH 10, WEEKS 40-44)
---------------------------------------------------------------
The capstone month: spec, build, ship and defend, then the final exam that decides the
certificate.

  Month 10: Capstone and the Final Exam (Weeks 40-44)
    Capstone Spec and Setup
      Topics:
        - Choosing the capstone: a full-stack typed application with shared types end to end
        - A one-page spec: scope, milestones, stack
        - Repository and skeleton ready on day one
        - Instructor sign-off before the build begins
      Projects built:
        - Approved capstone spec plus the running skeleton, deployed where applicable
      Practice: Pitch the spec to the batch and absorb one hard question
    Capstone Build
      Topics:
        - Two focused weeks in vertical slices
        - Instructor checkpoints and honest scope cuts
        - Using the whole course: the type system, typed React, typed Node and shared contracts
        - Daily commits and an always-working build
      Projects built:
        - The capstone, feature-complete with version control
      Practice: A working build at the end of every session
    Ship and Polish
      Topics:
        - The finishing pass: edge cases, errors, documentation
        - Strict-mode clean: the compiler as your first reviewer
        - A README worth reading and a demo worth watching
        - A full-course spiral review before the exam
      Projects built:
        - The capstone shipped, documented and rehearsed
      Practice: Two mock demos, each tighter than the last
    Demo Day and the Final Exam
      Topics:
        - Presenting the capstone: live demo plus the hardest bug story
        - The honest map of next roads: frontend depth, backend engineering, or full-stack product work
        - Where our other courses continue the journey
      Projects built:
        - Demo day presentation delivered to the batch
      Assessment: FINAL EXAM: a practical build plus a written paper with questions mixed
```

from every phase. Passing earns the certificate; falling short earns a focused revision plan and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1: JavaScript builds converted and hardened with types
- Phase 2: typed React applications with real state and data
- Phase 3: typed Node APIs with contracts the frontend shares
- Phase 4: the full-stack typed capstone, strict-mode clean

Total projects: 20+ real builds, crowned by a strict-mode-clean full-stack capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- JavaScript foundations solid enough to type
- The type system to depth: generics, narrowing, utility types and honest limits
- Typed React: props, state, hooks and data with the compiler on your side
- Typed Node: APIs with contracts shared across the stack
- Team habits: strictness judgment, migration strategy, review-quality types

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, coding along with the instructor
Homework: A problem set after every class, and a mixed-review assignment at the end of every month
Practice: 4-6 hours of building between classes
Review: Every submission reviewed with written feedback; recurring gaps reopened in class

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes
Progress updates: Honest monthly progress reviews: what is solid, what is wobbly, and the plan for the wobble

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final exam. Fall short and there is focused revision and a free retest: the certificate certifies knowledge, not attendance

WHERE THIS CAN LEAD

- Full-stack roles: TypeScript is the shared language of both ends now
- Our React and full-stack courses pair naturally either side of this one
- Backend depth: typed Node into production architectures
- DSA for interviews, with types making practice cleaner

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Early career roles: Rs 3-5 LPA (Junior Developer / Fresher)
Growing roles: Rs 5-10 LPA (Frontend Developer / React Developer); Rs 8-15 LPA (Full-Stack Developer)
Experienced roles: Rs 12-25 LPA (Senior Full-Stack Developer / Tech Lead); Rs 20-45 LPA (Lead Developer / Architect)
Freelance: Rs 1,500-6,000/hour based on expertise
International usa: $90k-200k USD (TypeScript Developer to Senior)
International europe: €55k-130k EUR based on country and experience

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What does this TypeScript course cover?
A: JavaScript foundations first (TypeScript is JavaScript underneath), the type system to real depth, generics, narrowing, utility types and their honest limits, then typed application development on both ends: React with TypeScript as product teams write it, and Node APIs with contracts the frontend shares. A finale month ships a strict-mode-clean full-stack capstone and sits the final exam.

Q: How long does it take to learn TypeScript properly?
A: Ten months of teaching at 2 live classes a week plus 4 to 6 hours of practice, with two more months in hand for learners who need them. The three teaching phases build the skill and its real-world application; the finale month ships a capstone and sits the final exam.

Q: How am I assessed? Is the certificate just for attending?

A: No. Problem sets follow every class with written feedback, monthly reviews deliberately mix new material with earlier topics, each phase ends with a cumulative exam, and the course ends with a final exam: a practical build plus written questions from the whole syllabus. Passing earns the certificate; falling short earns focused revision and a free retest.

Q: Should I learn JavaScript first, or start directly with TypeScript?
A: This course settles it: you learn JavaScript properly inside it, then add types. Starting with TypeScript syntax over weak JavaScript produces developers who fight the compiler; learning JS first then typing it produces developers the compiler helps. That order is the course.

Q: Can I try a class before enrolling?
A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Are classes live or recorded?
A: Live, always: small groups with a real instructor reading your actual code, never pre-recorded videos.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/complete-typescript-programming-masterclass-college
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 131 OF 137: VEDIC MATHS MASTERY: SPEED CALCULATION WITH ALL 16 SUTRAS

==========

Course link: https://learn.modernagecoders.com/courses/vedic-maths-course-speed-calculation-mastery
Subject area: Speed Mathematics
Designed for: Middle school
Who can join: Beginner friendly, ages 8 to adult
Course length: 6 months (24 weeks)
Time needed: 2 live classes/week + 20-30 minutes of drill practice most days
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Vedic maths course online with live classes: all 16 sutras for fast multiplication, squares, division and mental calculation. Ages 8 to adult, free demo class.

In one line: The same sums, done in a third of the time, with a method you can check.

ABOUT THE PROGRAMME

Vedic maths is a system of calculation shortcuts organised around 16 sutras, short working rules like Nikhilam (all from 9 and the last from 10) and Urdhva-Tiryagbhyam (vertically and crosswise). Used well, they turn sums like 97 x 96 or 998 squared into a few seconds of mental work. This 6-month live course teaches the system the way a maths teacher would: every shortcut is shown, practised against the clock, and then explained, because a student who knows why a trick works can trust it in an exam and adapt it when the numbers change. The course runs in one straight line: multiplication shortcuts first, because they pay off fastest, then squares and cubes, then division including the flag method, then fractions, decimals and percentages at speed, and finally the algebra applications of the same sutras. Week 1 starts with a timed diagnostic; week 24 ends with the same style of paper, so every student can see exactly how much faster and more accurate they have become. We are honest about what this course is: it makes arithmetic faster and frees up exam time. It does not replace school maths, and it will not turn anyone into a prodigy. It rewards steady drill, and we structure that drill for you.

WHAT MAKES THIS COURSE DIFFERENT

- Every shortcut comes with its reason: students see the algebra behind Nikhilam and the duplex method, so the rules stick instead of jumbling together
- Timed drills from week 1, with a personal speed log so progress is measured against your own diagnostic, not against a claim
- We teach judgement, not just tricks: which sutra fits which sum, and when the ordinary school method is actually the better choice
- Digit-sum checking is built in from the start, so speed never comes at the cost of accuracy

- Mixed-age friendly: the same system is drilled at school-exam pace for children and at competitive-exam pace for older students
- Live small-group classes where the teacher watches you calculate, catches bad habits early, and adjusts the drill sheets to your errors

THE LEARNING PATH, STEP BY STEP

Phase 1: The multiplication engine: complements, Nikhilam base multiplication, special patterns, then Urdhva-Tiryagbhyam for any numbers, all against the clock
Phase 2: Squares, cubes and roots with Yavadunam and the duplex method, then division three ways: Nikhilam, Paravartya and the flag method
Phase 3: Fractions, decimals and percentages at speed, the algebra applications of the sutras, and a timed capstone measured against the week 1 diagnostic

WHAT A STUDENT NEEDS BEFORE STARTING

Maths level: Comfortable addition and subtraction, and multiplication tables up to 10. That is genuinely all
Age: 8 to adult. Batches are grouped by age so drills run at the right pace
Equipment: A notebook, a pen, and a device with a camera and stable internet for live classes
Prior vedic maths: None expected. The course starts from the first sutra

WHO THIS COURSE IS FOR

School students: Students in classes 3 to 10 who lose exam time to slow arithmetic and want it back
Competitive exam aspirants: Older students preparing for arithmetic-heavy MCQ exams where a saved minute per section matters
Parents learning alongside: Adults who want to sharpen mental arithmetic, sometimes in the same batch as their child
Students who fear calculation: Children who understand concepts but freeze on big multiplications; the drills rebuild confidence with evidence
Maths enjoyers: Students who simply like numbers and want to know why the shortcuts work, not just that they do

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE MULTIPLICATION ENGINE (MONTHS 1-2, WEEKS 1-8)

Multiplication shortcuts pay off fastest, so they come first: complements and base multiplication, the special patterns, then Urdhva-Tiryagbhyam for any two numbers.

Month 1: Complements and Base Multiplication (Weeks 1-4)

How the System Works: Complements

Topics:
- Timed diagnostic paper: 30 mixed sums, your before picture for the whole course
- What the 16 sutras are and how this course walks through them
- Nikhilam Navatashcaramam Dashatah: all from 9 and the last from 10
- Instant subtraction from 100, 1000 and 10000 using complements
- Left-to-right working: why Vedic methods read numbers the way we say them
- Digit sums (Beejank) as a ten-second answer check

Projects built:
- Personal speed log opened: diagnostic scores recorded by sum type, to be updated all course

Practice: 40 complement subtractions from 100, 1000 and 10000, target under 5 minutes with every answer digit-sum checked

Nikhilam Multiplication Below the Base

Topics:
- Multiplying numbers just under 100: 97 x 96 in one line
- The two-part answer: cross subtraction on the left, deficiency product on the right
- Base 1000 and base 10: 988 x 997, 8 x 7 done the Nikhilam way
- Handling carries when the right-hand part overflows
- Why it works: the algebra of (base - a)(base - b), shown honestly
- Spotting when a sum is close enough to a base for Nikhilam to be worth it

Projects built:
- One-page proof poster: a student's own explanation of why Nikhilam works, with one worked example per base

Practice: 30 Nikhilam multiplications near 100 and 1000, target under 8 minutes, misses redone with the algebra written out

Above the Base and Working Bases

Topics:
- Numbers just above the base: 104 x 103, 1002 x 1015
- Mixed cases, one above and one below: 105 x 97 and the negative right-hand part
- Vinculum digits: writing 98 as 100 minus 2 inside a calculation
- Anurupyena, the working base idea: multiplying near 50, 200, 250 and 500
- Choosing the base that makes the sum smallest
- Speed drill technique: reading the sum, picking the method, then writing

Projects built:
- Worked set: 20 mixed base multiplications with the chosen base and method noted next to each

Practice: 35 mixed above/below/working-base multiplications, target under 10 minutes, with the base choice justified for any five

The Special Patterns

Topics:
- Ekadhikena Purvena: squares of numbers ending in 5, done in two seconds
- Antyayordashake'pi: same start, last digits adding to 10, such as 43 x 47
- Multiplying by 11 and by 12 with single-pass rules
- Multiplying by 5, 25 and 50 as fractions of powers of 10
- Pattern recognition drills: which of the five patterns fits, or none
- Consolidation of month 1: complements, Nikhilam, working bases, patterns

Projects built:
- Pattern wall chart: all month 1 shortcuts on one sheet in the student's own words and examples

Practice: 50-question pattern-spotting sheet mixing all month 1 techniques, target under 15 minutes

Assessment: Month 1 speed check: timed mixed paper, scores logged against the week 1 diagnostic

Month 2: Urdhva-Tiryagbhyam, Multiplication for Any Numbers (Weeks 5-8)

Vertically and Crosswise: Two Digits

Topics:
- Urdhva-Tiryagbhyam for 2-digit x 2-digit: the three-step cross pattern
- Where each part of the answer comes from, mapped to long multiplication
- Carry discipline: keeping the running answer clean
- One-line working: 34 x 72 without intermediate rows
- Checking with digit sums after every sum, as a habit not an afterthought
- When Nikhilam beats Urdhva-Tiryagbhyam and the reverse

Projects built:
- Cross-pattern diagram set: the student draws the crosswise structure for five sums before solving them

Practice: 40 two-digit by two-digit crosswise multiplications, target under 12 minutes at 90 percent accuracy

Three Digits and Mixed Sizes

Topics:
- The five-step cross pattern for 3-digit x 3-digit
- Mixed sizes: 3-digit x 2-digit by padding with a zero
- Managing bigger carries without losing the thread
- Writing less: moving from full working to compact working
- Common crosswise errors and how the digit-sum check catches them
- Speed strategy: which digit pairs to multiply first

Projects built:
- Worked set: 15 three-digit crosswise multiplications with compact working shown

Practice: 25 three-digit crosswise multiplications, target under 15 minutes, every answer digit-sum checked

Taking It Mental

Topics:
- Dropping the pen: holding the cross pattern in your head for 2-digit sums
- Chunking: multiplying 68 x 7 as 60 x 7 plus 8 x 7, said aloud
- Doubling and halving: 16 x 35 as 8 x 70
- Multiplying by 125, 75 and 15 through fraction thinking
- Estimation first: knowing roughly what the answer must be before starting
- Building a personal mental-maths warm-up routine

Projects built:
- Mental drill routine card: a 5-minute daily warm-up the student designs and follows

Practice: Daily 5-minute no-paper drills: 20 two-digit multiplications spoken or typed, streaks recorded in the speed log

Speed Lab One

Topics:
- Full multiplication mix under exam timing: patterns, bases, crosswise, mental
- Error triage: was it method choice, a carry slip, or rushing
- Reading your speed log: what actually got faster since week 1
- Accuracy floors: why we never trade below 90 percent accuracy for speed
- Personal weak-spot drill sheets, built from your own error log
- Setting the phase 2 targets with the teacher

Projects built:
- Speed lab paper one: a timed 60-question multiplication paper, marked and logged the same day

Practice: Redo every speed lab miss untimed with full working, then a second 20-question timed set on just those types

Assessment: Phase 1 milestone: timed multiplication paper compared line by line with the week 1 diagnostic

---

PHASE 2: SQUARES, CUBES AND DIVISION (MONTHS 3-4, WEEKS 9-16)

Powers and roots with Yavadunam and the duplex method, then the part most courses skip: division done three ways, including Tirthaji's general flag method.

Month 3: Squares, Cubes and Roots (Weeks 9-12)
  Squares Near a Base
    Topics:
      - Yavadunam: squaring by deficiency, 98 squared in one line
      - Squaring just above the base: 104 squared, 1012 squared
      - Working bases again: 48 squared using base 50
      - The algebra behind it: (base + d) squared, expanded and matched to the rule
      - Squares of numbers ending in 5 revisited and extended to 3 digits
      - Choosing between Yavadunam and the ending-in-5 rule
    Projects built:
      - Worked set: squares of every number from 91 to 109 by the fastest route, method noted for each
    Practice: 30 near-base squares across bases 50, 100, 500 and 1000, target under 10 minutes
  The Duplex Method: Any Square
    Topics:
      - The duplex (Dwandwa) of a number: definition and drill
      - Squaring any 2-digit number with duplexes
      - Squaring 3-digit numbers: the five duplex terms in order
      - Keeping carries straight in duplex work
      - Duplex squares against near-base squares: picking the faster tool
      - Digit-sum checking squares, including the common square-check traps
    Projects built:
      - Duplex table: the student builds a reference table of duplexes for 1 to 3 digit forms with examples
    Practice: 25 general squares of 2 and 3 digit numbers by duplex, target under 12 minutes at 90 percent accuracy
  Square Roots and Cube Patterns
    Topics:
      - Square roots of perfect squares: pairing digits and reading the first digit
      - Last-digit analysis: which digits can end a perfect square and what they tell you
      - Deciding between the two candidate roots quickly
      - Cubes near a base: 98 cubed and 102 cubed by the Nikhilam-style rule
      - The proportional (Anurupyena) cube rule for 2-digit numbers
      - Recognising perfect squares and cubes on sight
    Projects built:
      - Root-finder sheet: 20 perfect squares up to 5 digits, roots found and verified by squaring back
    Practice: 20 square roots of perfect squares and 10 near-base cubes, target under 15 minutes
  Cube Roots and the Powers Checkpoint
    Topics:
      - Cube roots of perfect cubes up to 6 digits from the two ends
      - Why cube last digits are honest: the 0 to 9 cube-ending map
      - Mixed powers drill: squares, cubes and roots on one sheet
      - Where powers shortcuts show up in school exams and MCQ papers
      - Month 3 consolidation: one method chart for all powers work
      - Speed log review: powers section opened and baselined
    Projects built:
      - Powers method chart: every squares, cubes and roots technique on one page with a worked example each
    Practice: Mixed 40-question powers paper covering all month 3 techniques, target under 18 minutes
    Assessment: Month 3 milestone: timed powers paper, results logged and weak types tagged for drilling

Month 4: Division Three Ways (Weeks 13-16)
  Nikhilam Division
    Topics:
      - Dividing by 9: the pattern that starts the whole method
      - Nikhilam division with divisors just below a base: 89, 98, 888
      - Quotient and remainder columns, and moving carries between them
      - When the remainder column overflows and how to fix it
      - Why it works: complements doing subtraction's job in division
      - Speed comparison against long division on the same sums
    Projects built:
      - Side-by-side set: 10 divisions worked both by long division and Nikhilam with times recorded for each
    Practice: 25 Nikhilam divisions with divisors near 10, 100 and 1000, target under 12 minutes
  Paravartya: Transpose and Apply
    Topics:
      - Paravartya Yojayet for divisors just above a base: 12, 112, 1023
      - Transposing digits: turning the divisor's excess into negative multipliers
      - Working with negative intermediate digits without panic
      - Nikhilam or Paravartya: reading the divisor and choosing in two seconds
      - Remainder checks with digit sums
      - The algebra behind Paravartya, shown for divisor 12
    Projects built:

- Choice drill sheet: 20 divisions labelled Nikhilam or Paravartya before solving, choices reviewed in class
Practice: 25 Paravartya divisions with divisors near 10, 100 and 1000, target under 14 minutes
Straight Division: The Flag Method
Topics:
- Dhvajanka, the flag digit: general division for any divisor
- Setting up the flag: splitting the divisor into main digit and flag
- The subtract-and-divide rhythm, worked slowly then at pace
- Two-digit flags for harder divisors
- Decimal answers: continuing the division past the point
- Why the flag method matters: one method that always works
Projects built:
- Worked set: 12 flag-method divisions including two carried to three decimal places
Practice: 20 flag-method divisions with mixed divisors from 13 to 87, target under 20 minutes
Divisibility and Speed Lab Two
Topics:
- Divisibility tests for 2 through 12, consolidated and drilled
- Osculation: Ekadhikena-based tests for 7, 13, 19 and other awkward primes
- Remainder thinking: Navashesh checks for division answers
- Full division mix under time: Nikhilam, Paravartya, flag, divisibility
- Error triage on division work
- Phase 3 targets set with the teacher
Projects built:
- Speed lab paper two: a timed 50-question powers and division paper, marked and logged the same day
Practice: Redo all speed lab misses untimed, then 15 divisibility questions on primes 7, 13 and 19 by osculation
Assessment: Phase 2 milestone: timed powers and division paper, logged against months 3 and 4 baselines

---
PHASE 3: FRACTIONS, ALGEBRA AND THE CAPSTONE (MONTHS 5-6, WEEKS 17-24)

The system applied where exams actually spend marks: fractions, decimals and percentages at speed, the algebra behind the sutras, and a timed capstone measured against week 1.

Month 5: Fractions, Decimals and Percentages at Speed (Weeks 17-20)
Recurring Decimals the Short Way
Topics:
- Ekadhikena Purvena on fractions: 1/19 written out in seconds
- The one-more-than-the-previous multiplier, forwards and backwards
- 1/7 and the cyclic pattern family
- Auxiliary fractions: converting awkward denominators before dividing
- Terminating or recurring: reading the denominator to know in advance
- Where recurring-decimal questions appear in school papers
Projects built:
- Decimal expansion chart: 1/7, 1/13, 1/17, 1/19 expanded by sutra and verified by flag division
Practice: 15 recurring-decimal expansions by the Ekadhikena method, each verified against straight division
Fraction Arithmetic Without the Grind
Topics:
- Adding and subtracting fractions crosswise, no LCM hunt for two terms
- When the LCM is still worth finding, honestly
- Comparing fractions instantly by cross multiplication
- Multiplying and dividing mixed numbers with vinculum thinking
- Simplifying early: spotting common factors before multiplying
- Fraction chains: working three-term expressions left to right
Projects built:
- Worked set: 20 fraction additions and comparisons done crosswise with times against the textbook method
Practice: 30 mixed fraction operations, target under 15 minutes, five of them written both ways to compare
Percentages and Ratio Shortcuts
Topics:
- Percentages as fractions: the 12.5 percent equals one-eighth family
- Flipping the sum: 8 percent of 50 as 50 percent of 8
- Percentage change chains done mentally
- Ratio splitting at speed for word problems
- Profit, loss and simple interest set up as one-line calculations
- Estimation guards: catching a wrong percentage before it costs marks
Projects built:
- Percent-fraction equivalence card: the conversion table every student memorises, self-tested twice
Practice: 40 mental percentage and ratio questions, target under 12 minutes, no written working allowed
Decimals and Speed Lab Three

Topics:
- Multiplying decimals with base methods: 9.8 x 9.7 via 98 x 97
- Placing the decimal point by estimation, not rules memorised blind
- Dividing decimals with the flag method
- Mixed arithmetic under time: the full toolkit on one paper
- Error triage across fraction, decimal and percentage work
- Month 5 consolidation chart

Projects built:
- Speed lab paper three: a timed 50-question fractions, decimals and percentages paper, marked and logged

Practice: Redo all speed lab misses untimed, then a 20-question timed set on your two weakest question types

Assessment: Month 5 milestone: timed mixed arithmetic paper, logged against the phase baselines

Month 6: Algebra Applications and the Speed-Test Capstone (Weeks 21-24)

The Sutras Meet Algebra

Topics:
- Urdhva-Tiryagbhyam on binomials: (x + 3)(x + 7) crosswise
- Multiplying larger polynomials by the same cross pattern
- Paravartya for dividing polynomials by (x - a)
- The connection made explicit: the arithmetic sutras were algebra all along
- Checking polynomial products with substitution
- Where this helps in school algebra chapters

Projects built:
- Bridge sheet: five arithmetic shortcuts and their algebraic twins worked side by side

Practice: 20 polynomial multiplications and 10 Paravartya polynomial divisions, checked by substitution at x equals 2

Equations at Speed

Topics:
- Sunyam Samyasamuccaye: spotting when a common factor makes the answer zero
- One-line solutions to equation types that usually take five lines
- Simultaneous equations by cross-multiplication patterns
- Anurupye Sunyamanyat: proportional coefficients and what they force
- Recognising equation types on sight before solving
- Honest limits: equations where the shortcuts do not apply

Projects built:
- Equation type-spotting sheet: 15 equations sorted by fastest method before any solving

Practice: 25 equations mixing shortcut types and standard types, with the method named before each solution

Choosing the Right Tool

Topics:
- Method triage: 40 mixed questions, decide the method in five seconds each
- Exam mapping: where each technique saves time in school and MCQ papers
- Where standard methods win and showing full working is required
- Building your personal shortcut handbook: what goes in, what gets dropped
- Mock speed test under full exam conditions
- Reviewing the mock: method choices, not just answers

Projects built:
- Personal shortcut handbook draft: every technique the student actually uses, in their own words and examples

Practice: Full 60-question mock speed test under timing, then a written half-page on your three best method choices

The Capstone Speed Test

Topics:
- Final timed paper: same structure as the week 1 diagnostic, full toolkit allowed
- Before and after: reading your speed log across all 24 weeks
- Accuracy audit: error rate then versus now
- Finishing the shortcut handbook: the take-away artifact of the course
- Keeping the speed: a 10-minute weekly maintenance routine
- Where to go next: exam-prep courses that use these skills daily

Projects built:
- Capstone speed test with a written before-and-after comparison against the week 1 diagnostic
- Finished personal shortcut handbook, reviewed by the teacher

Practice: Design your own maintenance drill sheet and run it once; bring the timing to the final class

Assessment: Course milestone: capstone speed test and handbook review, plus course-completion certificate review

PROJECTS THE STUDENT BUILDS AND KEEPS

- Personal speed log kept for all 24 weeks, from diagnostic to capstone
- One-page proof poster explaining why Nikhilam multiplication works
- Pattern wall chart of every month 1 shortcut in the student's own words
- Duplex reference table built by the student for general squares
- Side-by-side division sets timing Vedic methods against long division
- Decimal expansion chart for the awkward sevenths and nineteenths
- Percent-fraction equivalence card, memorised and self-tested
- Bridge sheet connecting five arithmetic sutras to their algebra twins

- Personal shortcut handbook: the student's complete method reference, finished in week 24
- Capstone speed test with a written before-and-after comparison

Total projects: 10+ artifacts, from a week 1 speed log to a complete self-written shortcut handbook

SKILLS THE STUDENT WILL HAVE AT THE END

- Base multiplication with Nikhilam, working bases and the special patterns
- General multiplication with Urdhva-Tiryagbhyam, on paper and mentally
- Squares, cubes and roots with Yavadunam, the duplex method and end-digit analysis
- Division by Nikhilam, Paravartya and the flag method, plus osculation divisibility tests
- Fractions, recurring decimals and percentages at exam speed
- Digit-sum checking as a permanent accuracy habit
- Method triage: reading a question and picking the fastest honest route

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, with timed drills done together in class
Practice: 20-30 minutes of drill sheets most days; short daily practice beats long weekend sessions for speed work
Review: Drill sheets reviewed with error triage: every miss classified as method, carry or rush, and drilled accordingly
Homework: A task or problem set after every class, and a monthly mixed-review assignment that deliberately folds in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, so a stuck technique never waits a week
Progress updates: Regular progress notes to parents with the speed log attached: what got faster, what needs drilling

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. Fall short and there is focused revision and a free retest: the certificate certifies real learning, not attendance

WHERE THIS CAN LEAD

- Faster arithmetic that pays off directly in school maths exams from class 4 upward
- A head start for competitive-exam arithmetic sections, where calculation speed buys thinking time
- Natural progression to our school maths and exam-prep courses, with calculation no longer the bottleneck
- A strong base for algebra, since Urdhva-Tiryagbhyam and Paravartya reappear there directly
- For adults: everyday numeracy, from splitting bills to sense-checking figures at work

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Vedic maths, honestly?
A: It is a system of 16 sutras, short calculation rules, compiled by Bharati Krishna Tirthaji and published in 1965. Historians debate how ancient the system really is, and we say so in class, because the techniques stand on their own: they are fast, they are checkable, and every one of them has ordinary algebra underneath. We teach the methods and the algebra, not the mythology.

Q: Will this replace my child's school maths?
A: No, and it should not. School maths teaches concepts and requires standard working in answers; this course makes the arithmetic inside those answers faster and more accurate. We explicitly teach when to use a shortcut, when to show the standard method, and how to use digit-sum checks on both. Think of it as removing the calculation bottleneck from maths a student is already learning.

Q: Does Vedic maths actually help in competitive exams?
A: It helps with one specific thing: calculation speed in arithmetic-heavy sections, which frees minutes for the questions that need real thought. It does not teach exam concepts, shortcuts for reasoning, or anything a syllabus tests directly. Students preparing for MCQ exams tend to feel the benefit most in percentages, fractions and multiplication-heavy questions, which is exactly where phase 3 of this course spends its time.

Q: Will this course make my child a genius or a human calculator?
A: No, and we would rather tell you that upfront. What steady practice on this course produces is documented in each student's speed log: the same kinds of sums done faster and with fewer errors than in week 1. That is a real, useful, measurable gain. Claims beyond that, about IQ or brain development, are not ones we make.

Q: What age is right for this course?
A: We take students from age 8, once multiplication tables are reasonably solid, and there is no upper limit; adults join to sharpen mental arithmetic, sometimes in the same house as an enrolled child. Batches are grouped by age so an 8-year-old drills at a different pace from a 16-year-old preparing for entrance exams.

Q: How is Vedic maths different from abacus classes?
A: Abacus training builds mental arithmetic through a physical tool and bead visualisation, and it works best when started young, roughly ages 5 to 10. Vedic maths is paper-and-head arithmetic built on algebraic patterns, so it suits any age that knows its tables and extends naturally into algebra. Younger children often do abacus first and Vedic maths later; we run both and will honestly tell you which fits your child.

Q: My child already struggles with maths. Is a speed course a bad idea?
A: Often it is the opposite, with one condition: the struggles must be with calculation, not with concepts. A child who understands what multiplication means but dreads big numbers usually gains confidence quickly here, because the methods are short and the progress is visible in their own log. If the gaps are conceptual, we will say so after the demo class and point you to our school maths courses instead.

Q: How am I assessed, and is the certificate just for showing up?
A: No. A task follows every class with feedback, monthly reviews deliberately mix new material with earlier topics so nothing is forgotten, and the course ends with a final exam covering the whole course. Passing earns the certificate; falling short earns focused revision and a free retest. We measure real learning, not attendance.

Q: Can we try a class before enrolling?
A: Yes, the first demo class is free and carries no obligation. Book it at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Parents are welcome to sit in, and the teacher will give you a straight opinion on whether this course or a school maths course is the better fit.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/vedic-maths-course-speed-calculation-mastery
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 132 OF 137: VIBE CODING FOR COLLEGE: FULL STACK, AI & DSA (AGES 18+)
==========

Course link: https://learn.modernagecoders.com/courses/vibe-coding-for-college-fullstack-ai-dsa-career-course
Subject area: College-Level Software Engineering
Designed for: College students
Who can join: Intermediate to Advanced (Ages 18+ / College Students)
Course length: 3 months (12 weeks)
Time needed: 10-15 hours/week
Certificate: Vibe Coder, College Level Full Stack & AI Engineer Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online vibe coding course for college students 18+: master the MERN stack, DSA, machine learning and system design, crack tech placements and internships.

In one line: From Student to Software Engineer, Ship Real Code, Ace Interviews, Land the Offer.

ABOUT THE PROGRAMME

Vibe Coding for College is the most comprehensive 3-month software engineering program for college students and fresh graduates aged 18+. 'Vibe Coding', coined by AI pioneer Andrej Karpathy in 2025, means building software by describing what you want in natural language and letting AI generate the code. In 2026, this has evolved into 'agentic engineering', orchestrating multiple AI agents to build production-grade software at startup speed. This course teaches you both: deep engineering fundamentals AND professional-grade AI development tools. You will master the MERN stack using Cursor IDE (the #1 AI code editor), accelerate development with Claude Code (Anthropic's terminal-based AI engineer), prototype at speed with Lovable and Bolt.new, and ship code with GitHub Copilot as your AI pair programmer. You'll also dive deep into DSA for placements, build ML projects, and learn System Design. But here's what separates great vibe coders from prompt-copy-pasters: understanding the code AI generates. That's why we teach you to be a developer FIRST and a vibe coder SECOND, so you can review, debug, secure, and scale what AI builds for you. Studies show 45% of AI-generated

code contains security vulnerabilities (OWASP Top 10). We teach you to catch them. At Modern Age Coders, we've placed students at top tech companies. Our curriculum is designed by engineers from FAANG-adjacent companies who know exactly what interviews look for and what internships demand of you from day one.

WHAT MAKES THIS COURSE DIFFERENT

- Industry-grade curriculum, same tools used at Google, Amazon, and top startups
- Professional vibe coding tools: Cursor IDE, Claude Code, Windsurf, GitHub Copilot
- AI app builders for rapid prototyping: Lovable, Bolt.new, v0.dev, Replit Agent
- Learn to BUILD with AI AND REVIEW what AI builds, security-aware development
- MERN Stack mastery, the most in-demand full-stack for 2026 hiring
- DSA preparation integrated throughout, not an afterthought
- Machine Learning with Python, scikit-learn, TensorFlow fundamentals
- System Design basics, asked in every senior/product company interview
- Agentic engineering workflows, orchestrating AI agents for multi-step development tasks
- Real project deployment on cloud (AWS EC2 / Render / Vercel)
- Code reviews by industry-experienced mentors (including AI-generated code review)
- Placement preparation: mock interviews, resume reviews, LinkedIn optimization
- Access to Modern Age Coders alumni network of placed students
- Open-source contribution guidance, building a credible GitHub

THE LEARNING PATH, STEP BY STEP

Phase 1: MERN Vibes (Weeks 1-4): MongoDB, Express.js, React.js, Node.js, build full-stack apps with Cursor IDE, GitHub Copilot & Claude Code
Phase 2: DSA & ML Vibes (Weeks 5-8): Data Structures, Algorithms, Python ML, model deployment + vibe coding rapid prototyping with Lovable & Bolt.new
Phase 3: Career Vibes (Weeks 9-12): System Design, agentic engineering, security for AI-generated code, Capstone, placement prep, professional launch

WHAT A STUDENT NEEDS BEFORE STARTING

Education: College student (any year) or recent graduate, any branch welcome
Coding experience: Familiarity with at least one programming language (Python, Java, C++, or JavaScript)
Web basics: Basic understanding of HTML/CSS (completing Vibe Coding for Teens is ideal but not required)
Equipment: Computer/laptop (Windows, Mac, or Linux) with stable internet
Time commitment: 10-15 hours per week, this is an intensive program
Motivation: Serious about building a career in software engineering

WHO THIS COURSE IS FOR

Cs students: CS/IT/MCA students preparing for campus placements and off-campus hiring
Non cs engineers: Engineering students from any branch (Mech, Civil, EE) switching to tech careers
Career switchers: Recent graduates wanting to break into software development
Dsa preparation: Students who need structured DSA preparation for product company interviews
Startup founders: Aspiring founders who want to build their own MVPs at startup speed with AI tools
Freelancers: Students who want to earn through freelance full-stack development while studying

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: MERN VIBES, FULL STACK WEB DEVELOPMENT (WEEKS 1-4)

Phase 1 builds you into a full-stack MERN developer. You'll master both the frontend (React) and backend (Node.js + Express + MongoDB), learn professional API design, authentication, and deploy your first production-level web application.

Week 1: JavaScript Mastery, Node.js & Your AI Development Toolkit

Topics:

- What is Vibe Coding?, Andrej Karpathy's 2025 concept 2026's 'agentic engineering' evolution
- The professional vibe coding workflow: Intent Spec Prompt Generate Review Iterate Ship
- Setting up Cursor IDE, your primary AI-powered development environment
- Cursor Composer & Agent mode, multi-file AI development across your entire codebase
- Setting up GitHub Copilot (free for students), inline AI completions and chat
- Introduction to Claude Code, terminal-based AI that understands your full project
- Windsurf IDE overview, Cascade agent and SWE-1.5 model for lightning-fast completions
- Advanced JavaScript: ES6+ features, destructuring, spread/rest, optional chaining
- Asynchronous JavaScript: callbacks, promises, async/await deep dive
- Event loop and JavaScript's single-threaded model explained
- Node.js architecture, non-blocking I/O, the event loop in Node
- Express.js setup and first REST API, hand-coded, then rebuilt with Cursor Composer

- RESTful API design principles, GET, POST, PUT, DELETE semantics
- Comparing hand-coded vs AI-generated Express APIs, reviewing what AI produces
- Environment variables, nodemon, and development tooling

Projects built:

- Products REST API, CRUD operations for a product catalog (in-memory, no DB yet)
- URL Shortener API, shorten URLs and redirect, stored in memory
- News Aggregation Proxy, fetch and forward data from a public news API

Practice: Build a small Node.js CLI utility in Cursor, then run one Composer multi file refactor and review the diff line by line before committing.

Week 2: MongoDB & Database Design, Persistent, Scalable Data

Topics:

- SQL vs. NoSQL, when to use each, MongoDB's document model
- MongoDB Atlas setup, cloud database in 5 minutes
- CRUD operations with MongoDB shell and Compass
- Mongoose ODM, schemas, models, and data validation
- Schema design: embedded vs. referenced documents
- Population, joining documents across collections
- Indexes for performance, understanding query plans
- Aggregation pipeline, complex data transformations
- Pagination and sorting on large datasets
- Data validation with Joi / Zod
- Seeding databases with realistic test data
- Connecting MongoDB Atlas to your Express app securely

Projects built:

- Blogging Platform API, full CRUD for posts, with categories, tags, and pagination
- Student Records System, schema design with references between students, courses, grades
- E-Commerce Product Catalog, categories, variants, stock tracking in MongoDB

Practice: Design a Mongoose schema for a blog with users, posts and comments, seed it on Atlas and run every CRUD operation from a script.

Week 3: React.js, Modern Frontend Development

Topics:

- React fundamentals: JSX, components, props, and the virtual DOM
- State management with useState, reactive UI basics
- useEffect, side effects, data fetching, and cleanup
- Component composition, building UIs from small, reusable pieces
- Conditional rendering and list rendering with keys
- Event handling in React, forms, clicks, input controlled components
- React Router v6, client-side navigation and nested routes
- Custom hooks, extracting and reusing complex logic
- Context API, sharing state without prop drilling
- React Query / TanStack Query, server state management
- Tailwind CSS with React, rapid, utility-first styling
- v0.dev, generating polished React/Next.js components from text descriptions
- Vibe coding React components: using Cursor Composer to scaffold entire features
- Reading and reviewing AI-generated React code, catching anti-patterns
- Performance: React.memo, useMemo, useCallback, when and why

Projects built:

- Task Management Dashboard, Kanban-style board with drag and React state
- Movie Browser, fetches TMDB API, search, filter, detailed movie pages
- Expense Tracker, add/categorize/delete expenses, charts with Chart.js

Practice: Build a searchable product list in React with useState, useEffect data fetching and correct list keys, then deploy it to a free host.

Week 4: Full MERN, Authentication, Authorization & Deployment

Topics:

- User authentication, registration, login, logout flows
- Password hashing with bcrypt, never store plain text passwords
- JWT (JSON Web Tokens), stateless authentication explained
- Protected routes, both API endpoints and React pages
- Role-based access control (RBAC), admin vs. user permissions
- Connecting React frontend to Express backend across environments
- CORS configuration, handling cross-origin requests
- HTTP-only cookies for secure token storage
- File uploads with Multer and Cloudinary / AWS S3
- Rapid prototyping with Lovable, build a full-stack MVP by chatting with AI
- Bolt.new for speed: idea deployed app in under 30 minutes
- Understanding what AI app builders generate vs. what you need to customize
- Deploying backend on Render (with MongoDB Atlas)
- Deploying React frontend on Vercel
- CI/CD basics with GitHub Actions, auto-deploy on push

Projects built:

- Full-Stack Social Notes App, register/login, create/edit/delete notes, private per user
- Recipe Sharing Platform, MERN app with auth, image uploads, saved recipes

Practice: Add JWT based register and login plus a protected dashboard route to your MERN app, with bcrypt hashed passwords and one admin only endpoint.

PHASE 2: DSA & ML VIBES, THINK DEEPLY, BUILD INTELLIGENTLY (WEEKS 5-8)

Phase 2 is the intellectual heart of the program. We go deep into Data Structures & Algorithms for placement preparation, and simultaneously build practical Machine Learning projects that solve real problems. This phase makes you stand out in every technical interview.

Week 5: Data Structures, The Foundation of Every Interview
Topics:
- Why DSA matters, the real purpose of data structures in engineering
- Time Complexity & Big O Notation, measuring efficiency
- Space Complexity, memory analysis
- Arrays: two-pointer technique, sliding window, prefix sums
- String problems: anagrams, palindromes, pattern matching
- Hash Maps & Hash Sets, O(1) lookup and counting patterns
- Linked Lists: singly, doubly, circular, implementation and operations
- Linked list problems: reverse, detect cycle, find middle, merge sorted lists
- Stacks: implementation, use cases (balanced brackets, undo operations)
- Queues: implementation, BFS patterns
- Deque and monotonic stack patterns
- LeetCode/HackerRank strategy, how to approach problems systematically

Projects built:
- LeetCode Top 50 Arrays & Strings, solve 15 curated problems with explanations
- Custom Hash Map Implementation, build a hash map from scratch in Python
- Text Editor Undo/Redo, stack-based implementation with UI

Practice: Solve eight array and string problems using two pointers, sliding window and prefix sums, recording time and space complexity for each.

Week 6: Trees, Graphs & Advanced Algorithms
Topics:
- Binary Trees, traversals: inorder, preorder, postorder, level order
- Binary Search Trees (BSTs), operations and balancing concepts
- Tree problems: height, diameter, LCA, path sum, zigzag traversal
- Heaps & Priority Queues, min-heap, max-heap, heapify
- Graphs: representations (adjacency list vs. matrix), directed vs. undirected
- BFS, shortest path in unweighted graphs, level-order problems
- DFS, cycle detection, topological sort, connected components
- Dijkstra's Algorithm, weighted shortest path
- Union-Find (Disjoint Set Union), efficient connectivity queries
- Dynamic Programming: memoization vs. tabulation, top-down vs. bottom-up
- Classic DP patterns: Fibonacci, coin change, 0/1 knapsack, LCS, LIS
- Greedy Algorithms, interval scheduling, activity selection

Projects built:
- Graph Visualizer, interactive BFS/DFS visualization built with React
- Pathfinding Visualizer, Dijkstra's and A* in a grid, visually animated
- Course Prerequisite Checker, topological sort applied to a real problem

Practice: Solve six tree and graph problems including one BFS, one DFS and one heap problem, and explain one solution aloud as if in an interview.

Week 7: Machine Learning, Teach Python to Predict the Future
Topics:
- Machine Learning overview, supervised, unsupervised, reinforcement learning
- The ML workflow: data features model evaluate deploy
- Data exploration with pandas, statistical analysis and visualizations
- Feature engineering, creating meaningful inputs for ML models
- Train/test/validation split, avoiding overfitting and data leakage
- Linear Regression, predicting continuous values (housing prices, salaries)
- Logistic Regression, binary classification problems
- Decision Trees and Random Forests, ensemble methods
- Support Vector Machines (SVM), classification with hyperplanes
- K-Means Clustering, unsupervised customer segmentation
- Model evaluation: accuracy, precision, recall, F1, ROC-AUC
- scikit-learn pipelines, cleanly combining preprocessing and models

Projects built:
- House Price Predictor, regression model trained on real estate dataset
- Email Spam Classifier, text-based binary classification with logistic regression
- Customer Segmentation, K-means on e-commerce purchase data

Practice: Train a scikit-learn model on a public dataset with a proper train test split, reporting accuracy and one feature engineering decision.

Week 8: Deep Learning & AI Project Deployment
Topics:
- Neural Networks, neurons, layers, activation functions explained intuitively
- How deep learning trains, gradient descent, backpropagation
- TensorFlow and Keras, building and training neural networks
- Convolutional Neural Networks (CNNs), image recognition
- Transfer Learning, using pre-trained models (MobileNet, ResNet) for new tasks
- Natural Language Processing basics, tokenization, embeddings, TF-IDF
- Hugging Face Transformers, using BERT and GPT models via Python

- Model serialization, saving and loading trained models
- Deploying ML models as REST APIs with Flask/FastAPI
- Streamlit, building beautiful ML demo apps with pure Python
- Using Claude Code to scaffold ML pipelines: describe your data AI writes the training code
- Vibe coding ML dashboards: using Lovable or Bolt.new to build a frontend for your ML API
- Monitoring ML models in production, drift, versioning, logging
- AI responsible use, bias auditing and model transparency

Projects built:
- Image Classifier API, custom CNN trained on a dataset of your choice, deployed
- Sentiment Analysis Web App, Hugging Face transformer, Flask API, Streamlit UI
- Real-Time Object Detector, webcam feed with YOLO model via OpenCV

Practice: Fine tune a pre-trained CNN such as MobileNet on a small image dataset with Keras and record a short working demo.

PHASE 3: CAREER VIBES, SYSTEM DESIGN, CAPSTONE & PROFESSIONAL LAUNCH (WEEKS 9-12)

The final phase transforms a skilled developer into a career-ready software engineer. System Design, advanced cloud deployment, the grand capstone project, and an intensive placement preparation program, this phase exists to get you hired.

Week 9: System Design & Agentic Engineering, Think Like a Senior Engineer

Topics:
- What is System Design? Why it's asked in every senior tech interview
- Agentic Engineering, Karpathy's evolution of vibe coding: orchestrating AI agents for complex tasks
- Using Claude Code for large codebase navigation, AI that understands your entire project
- Cursor Agent mode: giving AI a task, letting it plan and execute across dozens of files
- Multi-agent workflows: using Cursor (planning) + Claude Code (execution) + Copilot (inline) together
- Scalability: horizontal vs. vertical scaling
- Load Balancers, distributing traffic, round-robin, least connections
- Databases at scale: sharding, replication, read replicas
- SQL vs. NoSQL selection framework, when each is right
- Caching strategies: Redis basics, cache-aside, write-through, TTL
- Content Delivery Networks (CDN), how your assets go global
- Microservices vs. Monolith, honest trade-offs, not just hype
- Docker and containerization, packaging apps for consistent deployment
- Case studies: Design Twitter, Design YouTube, Design WhatsApp, methodically

Projects built:
- System Design Document, write a complete design document for a URL shortener at scale
- Redis Caching Layer, add Redis caching to your MERN capstone app
- Docker Compose Setup, containerize your full MERN stack with Docker Compose

Practice: Give Cursor Agent or Claude Code one scoped multi file task in your capstone repo, then audit the output and document what you kept, fixed or rejected.

Week 10: Cloud, DevOps & Security for AI-Generated Code

Topics:
- AWS core services: EC2, S3, RDS, CloudFront, Route 53
- Deploying a Node.js app on EC2 with Nginx reverse proxy
- AWS S3, storing and serving static assets and user uploads at scale
- SSL/TLS certificates, HTTPS with Let's Encrypt
- CI/CD pipelines with GitHub Actions, test, build, deploy automatically
- Environment management, dev, staging, production environments
- CRITICAL: Security risks of vibe-coded apps, 45% of AI-generated code contains OWASP vulnerabilities (Wiz Security study)
- Web Application Security: OWASP Top 10 for developers, with specific AI-generated code examples
- SQL injection, XSS, CSRF in AI-generated code, what AI commonly gets wrong
- Security audit workflow: using OWASP ZAP and Snyk to scan vibe-coded apps
- Rate limiting, IP blocking, and DDoS basics
- Secrets management, never commit credentials to GitHub (AI tools often leak .env files)
- Application monitoring with Datadog / Sentry (free tiers)
- The responsible vibe coder checklist: security review before every deployment

Projects built:
- Production Deployment: Deploy your MERN capstone with HTTPS on AWS EC2 via Nginx
- Automated CI/CD: Full GitHub Actions pipeline, lint, test, deploy on merge to main
- Security Audit: Run OWASP ZAP against your app and fix at least 3 vulnerabilities

Practice: Deploy your MERN app with HTTPS on EC2 or a comparable host, and add a GitHub Actions pipeline that runs tests before every deploy.

Week 11: Placement Preparation & Professional Identity

Topics:
- The placement roadmap, on-campus and off-campus strategy for 2026
- Resume that gets shortlisted, ATS optimization, STAR format, quantification
- GitHub portfolio for job seekers, README, contributions, and credibility
- LinkedIn optimization: headline, about section, skills endorsements
- Behavioural interview prep: STAR method for tech companies
- Technical interview process, online assessment, DSA round, system design, HR

- DSA rapid revision, top 20 most frequently asked interview questions
- Mock Interview Session 1: Full 45-minute technical mock with feedback
- Mock Interview Session 2: System design mock, 30 minutes
- Negotiating your offer, salary benchmarks for fresher SDE roles India
- Freelancing as a college developer, finding clients, pricing, contracts
- Open source strategy, how meaningful contributions open career doors

Projects built:
- Professional Portfolio Website v2, updated with all 12-week projects, live, SEO-optimized
- Optimized Resume, ATS-ready, reviewed by an industry mentor
- GitHub Portfolio Cleanup, all key repos have great READMEs, demo links, and screenshots

Practice: Finish an ATS friendly one page resume, refresh your GitHub profile README and rewrite your LinkedIn headline, then get feedback from one mentor or peer.

Week 12: Grand Capstone Launch, Showcase, Certify & Conquer

Topics:
- Grand Capstone final polish, code review, performance optimization
- Writing a technical blog post about your capstone project (Medium / Dev.to)
- SEO optimization for your portfolio, getting found by recruiters
- Open source contribution: submit a PR to a real open-source project
- The modern developer identity: traditional coding + vibe coding + agentic engineering
- Your vibe coding toolkit mastery recap: Cursor, Claude Code, Windsurf, Copilot, Lovable, Bolt.new, v0.dev
- The developer mindset, growth, continuous learning, imposter syndrome
- Building in public, Twitter/X, LinkedIn, community engagement
- Advanced paths beyond Vibe Coding: Competitive Programming, ML research, DevOps, AI Engineering
- Modern Age Coders Alumni Network, joining and giving back
- Capstone presentations, live demo to a panel of industry judges
- Project feedback from industry professionals
- Receiving the Vibe Coder, College Level Certification
- Next steps: internships, placements, freelancing, and startups

Projects built:
- GRAND CAPSTONE, 'Production-Grade Vibe Project': A complete, full-stack (MERN) OR ML-powered application built using a professional vibe coding workflow (Cursor IDE + Claude Code + AI app builders for prototyping). Must include: user authentication, real data, API integrations, responsive UI, full cloud deployment, CI/CD pipeline, security audit (AI-generated code reviewed for OWASP vulnerabilities), documented on GitHub, and presented live to a panel of industry mentors with a walkthrough of both the product AND the AI-assisted development process

Assessment: Grand Capstone: Live Demo Code Review System Design Q&A Panel Feedback

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Weeks 1-4): 8+ MERN stack projects, REST APIs, React apps, full-stack authenticated apps
- Phase 2 (Weeks 5-8): 6+ DSA visualizers + 3 ML models trained, evaluated & deployed
- Phase 3 (Weeks 9-12): System design documents, Docker/AWS deployments, Grand Capstone

Total projects: 15+ production-grade projects + 100+ DSA problems solved

SKILLS THE STUDENT WILL HAVE AT THE END

- Frontend: React 18, React Router, TanStack Query, Tailwind CSS, Vite, v0.dev
- Backend: Node.js, Express.js, REST API design, JWT auth, bcrypt, Multer
- Database: MongoDB, Mongoose, schema design, aggregation, indexing
- Vibe Coding: Cursor IDE (Composer + Agent), Claude Code, Windsurf, GitHub Copilot, Lovable, Bolt.new
- DSA: Arrays, Strings, Linked Lists, Trees, Graphs, DP, Greedy, 100+ LeetCode problems
- Machine Learning: scikit-learn, pandas, TensorFlow/Keras, Hugging Face, model deployment
- DevOps: Git, GitHub Actions, Docker, AWS EC2/S3, Nginx, CI/CD pipelines
- System Design: load balancing, caching, Redis, microservices, database scaling
- Security: OWASP Top 10, AI-generated code auditing, rate limiting, secrets management

HOW A TYPICAL WEEK IS SPLIT

Live sessions: 2 sessions (2 hours each)
Dsa practice: 3-4 hours (10+ problems/week)
Project work: 4-6 hours
Total per week: 10-15 hours

SUPPORT INCLUDED

Mentorship: Code reviews by industry-experienced mentors
Mock interviews: Technical mock interviews (DSA + System Design) with feedback
Placement support: Resume reviews, LinkedIn optimization
Community: Access to Modern Age Coders alumni network of placed students
Career guidance: Resume and interview preparation, plus portfolio and profile review.

CERTIFICATION DETAIL

Final certificate: Vibe Coder, College Level: Full Stack & AI Engineer (Modern Age Coders Certified)
Capstone required: Production-grade MERN or ML app with auth, cloud deployment, CI/CD, and security audit
Dsa milestone: 100+ LeetCode problems solved with documented approaches
Portfolio: Professional developer portfolio + optimized GitHub + LinkedIn

WHERE THIS CAN LEAD

- Full Stack Web Developer (Rs 6-18 LPA)
- Frontend Developer, React Specialist (Rs 7-20 LPA)
- Backend Developer, Node.js/Python (Rs 8-22 LPA)
- Junior Machine Learning Engineer (Rs 10-25 LPA)
- Software Development Engineer (SDE) at product companies
- Summer Internships at tech startups and MNCs
- Freelance Full Stack Developer (Rs 500-2,500/hour)
- AI/Vibe Coding Consultant, building MVPs for clients with AI tools

EARNING POTENTIAL (share only if asked, and never promise it)

Market context: The ranges below are general market salary bands for these roles in India and abroad, drawn from public industry data. They are shown for career context only and are not a promise or guarantee of income. Actual pay depends on your skills, experience, location, and the job market.
Experienced roles: Rs 15-25 LPA (with ML specialization or senior full-stack role)
Other roles: Rs 6-12 LPA (SDE-1 at product companies and MNCs); Rs 10-18 LPA (with strong portfolio and MERN skills); $40k, 100k USD (remote roles with international companies)
Freelance: Rs 500-2,500/hour based on project complexity and specialization

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Vibe Coding for College?
A: Vibe Coding for College is a 3-month, intensive software engineering program for college students and fresh graduates (18+). It uniquely combines deep engineering fundamentals with professional AI development tools: you master MERN stack using Cursor IDE and Claude Code, build rapid prototypes with Lovable and Bolt.new, learn DSA for placements, Machine Learning with Python, System Design, and security for AI-generated code, all taught live by industry-experienced mentors at Modern Age Coders.

Q: Who is this course for?
A: This course is for college students (any year) and recent graduates who know at least basic programming and want to break into software engineering. It's ideal for CS/IT/MCA students, but engineering students from any branch who are serious about tech can join. You should have basic knowledge of at least one programming language (Python, Java, C++, or JavaScript).

Q: What is the MERN stack and why learn it?
A: MERN stands for MongoDB (database), Express.js (backend framework), React.js (frontend library), and Node.js (JavaScript runtime). It is the most popular and in-demand full-stack technology combination for 2026, used by top startups and tech companies worldwide. MERN skills are specifically sought in Indian product companies, MNCs, and international remote roles.

Q: How does DSA preparation fit into this course?
A: DSA is integrated throughout all 12 weeks rather than crammed at the end. Weeks 5-6 are dedicated DSA weeks, but we recommend 10 problems per week from Week 1 onwards through our parallel preparation track. By Week 12, students have typically solved 100-150 problems across LeetCode, GeeksforGeeks, and Codeforces.

Q: Will studying 3 months really prepare me for placements?
A: Three months of focused, structured preparation, building real projects, solving 100+ DSA problems, and doing mock interviews, is absolutely transformative. Join as early in your college career as possible: students who take this course in their 2nd or 3rd year have maximally more time to practice and build. Our alumni have been placed at companies including Infosys, TCS, Wipro, Freshworks, Juspay, and international remote companies.

Q: What does 'Vibe Coding' mean at the college level?
A: Vibe Coding, coined by Andrej Karpathy in 2025, means building software by describing what you want and letting AI generate the code. At the college level, this evolves into 'agentic engineering': using professional AI tools (Cursor IDE, Claude Code, Windsurf, GitHub Copilot) alongside AI app builders (Lovable, Bolt.new, v0.dev) to build production-grade software at startup speed. But the key differentiator is that you also learn to REVIEW, SECURE, and SCALE what AI generates, because 45% of AI-generated code contains security vulnerabilities. You graduate as a developer who can build with AI AND without it.

Q: Do you help with interview and career preparation?

A: Week 11 is dedicated to career preparation: resume reviews, LinkedIn optimization, mock technical interviews, system design practice, and DSA rapid revision. You also get access to our alumni network and mentor panels. We prepare you thoroughly for interviews and applications, but we do not guarantee placement.

Q: What is the time commitment?
A: We recommend 10-15 hours per week: 2 live sessions (2 hours each), weekly DSA practice (3-4 hours), and project work (4-6 hours). This is an intensive program. The more you invest, the stronger your portfolio and placement readiness will be.

Q: Is Machine Learning part of this course?
A: Yes. Weeks 7 and 8 cover applied Machine Learning with Python, scikit-learn, regression, classification, clustering, neural networks with TensorFlow/Keras, and deploying ML models as APIs and web apps. This is applied/practical ML, not theory-heavy research ML.

Q: What certification will I receive?
A: This is a credential recognized by our industry partners and verifiable online.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/vibe-coding-for-college-fullstack-ai-dsa-career-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 133 OF 137: VIBE CODING FOR KIDS: AI, SCRATCH & GAME DEV (AGES 8-12)
==========

Course link: https://learn.modernagecoders.com/courses/vibe-coding-for-kids-beginners-ai-scratch-game-dev
Subject area: Kids Coding & Creative Technology
Designed for: Kids
Who can join: Complete Beginner (Ages 8-12)
Course length: 3 months (12 weeks)
Time needed: 4-6 hours/week
Certificate: Vibe Coder, Kids Level Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online vibe coding course for kids ages 8-12: learn Scratch, AI tools and game development in a 3-month beginner program, no prior experience needed.

In one line: Where Imagination Meets Code, Build Games, Stories & AI Projects!

ABOUT THE PROGRAMME

Vibe Coding for Kids is the ultimate 3-month beginner coding journey designed specifically for children aged 8 to 12. 'Vibe Coding', a term coined by AI pioneer Andrej Karpathy in 2025, means telling a computer what you want to build in plain English and letting AI create the code for you. It's the future of how software gets made, and your child can start learning it now. Through playful, visual, and project-driven learning, kids discover the magic of creating their own games, animated stories, music with code, and real AI-powered apps, no complex typing required to get started! Kids begin with visual block coding (Scratch) to learn how computers think, then graduate to actual vibe coding using tools like Replit, Bolt.new, and Google Gemini, the same AI tools professionals use today. At Modern Age Coders, we believe every child is a natural creator. Our expert instructors blend visual block programming, creative exploration, and real AI app-building tools to make coding feel like play. By the end of 12 weeks, your child will have built 15+ original projects, including apps they built by talking to AI, and gained real digital superpowers.

WHAT MAKES THIS COURSE DIFFERENT

- 100% project-based, kids build something real every single week
- Age-appropriate content designed for 8-12 year olds
- No prior experience or technical knowledge needed
- Visual block coding (Scratch, Code.org), no complex syntax to start
- Real vibe coding: kids build apps by talking to AI (Replit, Bolt.new)
- Includes AI tools, music coding, and creative art projects
- Live classes with expert instructors, never pre-recorded videos
- Weekly written progress notes to parents
- Portfolio of 15+ completed projects at course end
- Fun, encouraging, and bully-free classroom environment

THE LEARNING PATH, STEP BY STEP

Phase 1: Foundation Vibes (Weeks 1-4): Scratch basics, animations, storytelling with code
Phase 2: Game Dev Vibes (Weeks 5-8): Game mechanics, score systems, interactive adventures
Phase 3: AI & Vibe Coding Vibes (Weeks 9-12): Real vibe coding with AI app builders (Replit, Bolt.new), AI tools, music coding, capstone project showcase

WHAT A STUDENT NEEDS BEFORE STARTING

Age: 8-12 years old
Experience: Absolutely none, complete beginner friendly
Equipment: A computer or laptop with internet connection
Software: All tools are free and browser-based, nothing to install
Parental support: Parents encouraged to engage and review weekly projects

WHO THIS COURSE IS FOR

Curious kids: Children aged 8-12 who are curious about how games and apps are made
Creative kids: Kids who love drawing, storytelling, music, or building things
Screen time shift: Parents who want to turn screen time into productive learning time
Future ready: Families who want to give their child a head start in technology and AI
Struggling learners: Children with dyslexia, ADHD, or other learning differences, vibe coding removes syntax barriers

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FOUNDATION VIBES, SCRATCH & CREATIVE CODING (WEEKS 1-4)

Kids dive into the colourful world of Scratch and block-based programming. They create their first animations, interactive stories, and simple games while learning the core ideas of coding, loops, events, and conditionals, through hands-on fun.

Week 1: Welcome to Vibe Coding!, Your First Animations
Topics:
- What is Vibe Coding?, Telling computers what to build using your words (coined by AI scientist Andrej Karpathy in 2025)
- The two ways to code: Block Coding (Scratch) and Vibe Coding (talking to AI), we'll learn both!
- How computers think, understanding instructions step by step
- Introduction to Scratch 3.0, navigating the interface
- Sprites, stages, and backdrops, building your digital world
- Basic motion blocks: move, turn, glide
- Looks blocks: say, think, costumes, effects
- Sound blocks: play music, record your voice
- Your first animation: Make a character walk and talk
- Saving and sharing your Scratch project
- Online safety and being a kind digital citizen
- Setting up your Vibe Coder profile
Projects built:
- Animated Greeting Card, make a character say hello to the world
- Dancing Pet, a sprite that moves to your favourite music
- About Me Story, an interactive presentation about yourself
Practice: Build a mini dance party in Scratch where your sprite changes costume and moves each time you press an arrow key, then show someone at home.

Week 2: Storytelling with Code, Scenes, Dialogues & Adventures
Topics:
- Creating multi-scene stories with backdrop changes
- Broadcasting messages between sprites, making characters talk to each other
- Using the 'ask and answer' block for interactive dialogue
- Timing animations, wait blocks and synchronized stories
- Creating sprite costumes for character emotions
- Adding background music and sound effects
- Text-to-speech: making your characters speak in different voices
- Importing your own images and drawings as sprites
- Drawing custom sprites with the Scratch paint editor
- Story planning: beginning, middle, and end
- Sharing your story with family and friends
Projects built:
- A 3-Scene Adventure Story, hero, challenge, and victory
- Interactive Storybook, click to turn pages
- Talking Animal Show, two animals have a funny conversation
Practice: Create a two scene story where characters talk to each other using broadcast messages and ask the viewer one question with the ask block.

Week 3: Events & Interactivity, Making Things Happen!

Topics:
- Event blocks: when green flag clicked, when key pressed, when sprite clicked
- Keyboard controls, making characters move with arrow keys
- Mouse events, click-to-activate actions
- Creating clickable buttons and navigation menus
- Introduction to conditionals: If-Then logic blocks
- Touching and color detection, collision basics
- Creating interactive art, drawing with mouse movements
- Building a virtual musical keyboard
- Combining multiple events for complex interactions
- User input with 'ask' and storing answers
- Debugging basics: What to do when something goes wrong

Projects built:
- Virtual Piano, play notes by pressing keyboard keys
- Click-and-Colour Art Canvas, draw with code
- Interactive Quiz, 5 questions with right/wrong feedback

Practice: Make a clickable pet that reacts differently to a click, a key press and the green flag, using at least one if then block.

Week 4: Loops & Patterns, The Magic of Repetition

Topics:
- Forever loops, making things go on and on
- Repeat loops with a specific number of times
- Nested loops, loops inside loops for complex patterns
- Drawing geometric shapes with pen blocks
- Creating spirals, stars, and kaleidoscopes with loops
- The Stamp tool, making repeating patterns
- Color effects and graphic transformations
- Thinking like a computer: breaking big tasks into small loops
- Efficiency, doing more with less code
- Creating a looping background animation
- Phase 1 Review & Project Planning

Projects built:
- Geometric Art Machine, generates beautiful patterns automatically
- Spinning Galaxy, looping space animation
- Pattern Puzzle, create a design and challenge friends to recreate it

Practice: Draw a star, a spiral and one invented pattern with pen blocks inside repeat loops, and name your favorite creation.

PHASE 2: GAME DEV VIBES, BUILD YOUR OWN GAMES! (WEEKS 5-8)

Now the real fun begins! Kids learn the building blocks of game development: scoring, lives, enemies, levels, and game loops. By the end of Phase 2, every child will have built a complete, playable video game from scratch.

Week 5: Variables & Score Systems, 'You Win!' Moments

Topics:
- What are variables? Understanding data storage in simple terms
- Creating score, lives, and timer variables
- Changing variable values, adding points, losing lives
- Using random numbers, the element of surprise in games
- Countdown timers and time-based challenges
- Displaying variables on screen as a score HUD
- Win conditions: What happens when you reach a goal?
- Game Over conditions: Lives system and health bars
- High score saving basics
- Creating power-ups that boost your score
- Playtesting your games, the 'fun factor' check

Projects built:
- Catch the Falling Stars, catch sprites to earn points
- Number Guessing Game, with hints and score tracking
- Speed Challenge, collect 10 items before the timer runs out

Practice: Add a score, lives and a countdown timer to a simple catch the apple game and play test it three times.

Week 6: Collision Detection & Game Mechanics, Real Game Feels

Topics:
- Touching detection, sprites interacting with each other
- Color collision, detecting when you touch a wall or obstacle
- Gravity simulation using loops and variables
- Jumping mechanics, platformer physics basics
- Enemy movement, creating obstacles that move and follow
- Projectile mechanics, shooting and throwing
- Boundary detection, keeping sprites inside the screen
- Creating a scrolling background effect
- Power-up pickups and damage effects
- Sound effects for collisions, wins, and game over
- Making your game feel satisfying, game juice basics

Projects built:
- The Endless Runner, dodge obstacles as you run
- Space Invaders Jr., shoot incoming aliens
- Trampoline Jump, bounce as high as you can game

Practice: Give your game character a gravity powered jump and one moving enemy that ends the game on touch.

Week 7: Level Design & Multi-Screen Games, The Full Experience

Topics:
- What makes a great game level? Flow, challenge, and reward
- Designing level 1, level 2, and level 3 with increasing difficulty
- Using backdrops as levels, switching on game events
- Creating boss levels and final challenges
- Save and load game states using cloud variables
- Cutscenes and story sequences between levels
- Sound design, choosing music for different game moods
- UI design: Start screen, game over screen, victory screen
- Multiple character selection screens
- Accessibility in games, making your game easy to understand
- Beta testing with classmates and collecting feedback

Projects built:
- 3-Level Maze Adventure, escape each dungeon to advance
- Fruit Ninja Clone, 3 difficulty levels with progressive speed
- Boss Battle Game, survive the final boss to win

Practice: Design level 1 and a harder level 2 for your game using two backdrops that switch when the player wins.

Week 8: Complete Game Jam, Build Your Dream Game!

Topics:
- Game design document, planning your game on paper first
- Asset creation, custom sprites, backgrounds, and sounds
- Implementing full game loop: start, play, lose, win
- Bug hunting and debugging strategies
- Game polishing, making it look professional
- Writing game instructions for players
- Creating a game thumbnail and cover art
- Publishing your game to the Scratch community
- Presenting your game to classmates
- Getting featured on the Modern Age Coders showcase
- Reflecting on your journey from Week 1 to Week 8

Projects built:
- PHASE 2 CAPSTONE, My Dream Game: A complete, original game with: custom sprites, 2+ levels, score system, sound effects, start and end screens

Assessment: Phase 2 Game Development Assessment, completed game review with instructor feedback

---

PHASE 3: AI & REAL VIBE CODING, BUILD APPS BY TALKING TO AI! (WEEKS 9-12)

In the final phase, kids experience the real magic of vibe coding, they train their own machine learning models, build real apps by describing ideas to AI tools like Replit and Bolt.new, create music with code, design with AI, and build their capstone 'Vibe Showcase' project. This is where kids learn the skill that Andrej Karpathy says will define the future: telling AI what to build and watching it come to life.

Week 9: AI for Kids, Teaching Computers to See, Hear & Build!

Topics:
- What is Artificial Intelligence? A fun, simple explanation
- What is Vibe Coding?, The story of Andrej Karpathy and how AI writes code for you
- The vibe coding loop: Describe AI Builds You Test You Improve
- Machine Learning, teaching computers by showing examples
- Google Teachable Machine: train your first image model
- Training an AI to recognize your hand gestures
- Training an AI to recognize sounds
- Using your trained model in a Scratch project
- Introduction to Google Gemini, chatting with AI safely
- How to write good prompts, being clear and specific with AI
- What is bias in AI? Why AI needs diverse data
- Machine Learning for Kids (ML4Kids) platform
- AI ethics for kids, being responsible with technology
- The future of AI, jobs, creativity, and possibilities

Projects built:
- Rock Paper Scissors AI, a computer that plays against you using real ML
- Emotion Detector, AI that reads your facial expression
- AI Chat Experiment, ask Google Gemini to explain your favourite topic and fact-check its answers

Practice: Train a Teachable Machine model to tell two hand signs apart using ten photos of each, then test it and count how many it gets right.

Week 10: Vibe Coding Your First Apps, Build by Talking to AI!
Topics:
- Welcome to real vibe coding!, building apps by describing them in English
- Introduction to Replit, your browser-based coding playground
- Replit Agent, tell it what to build and watch it create your app live
- Writing great prompts: being specific about what you want (colours, buttons, features)
- Building your first vibe-coded app: a simple calculator
- Introduction to Bolt.new, type what you want, see it instantly
- Building a game with Bolt.new, describe the rules, AI codes it
- Testing your AI-built app, does it do what you asked?
- Fixing problems by telling AI what went wrong (iterating)
- Understanding the code AI wrote, reading what it made for you
- Comparing: Scratch (you drag blocks) vs. Vibe Coding (you describe in words)
- When AI makes mistakes, learning to spot and fix errors

Projects built:
- My Calculator App, tell Replit Agent to build a colourful calculator and customize it
- Quiz Game App, describe a quiz game to Bolt.new with topics you love
- Vibe-Coded Story App, describe an interactive story app and watch AI build it

Practice: Describe a tiny app to Replit Agent, like a compliment generator, then ask it for two improvements: a new color and an extra button.

Week 11: Music Coding, AI Art & Portfolio, Create Like a Pro
Topics:
- Introduction to Sonic Pi, coding music like a DJ
- Notes, beats, and rhythms in code, loops for musical patterns
- Live coding music, composing a 30-second original track
- Using AI to help compose: describe a mood, get a melody suggestion
- Introduction to AI image generation for kids (safe, educational tools)
- Design principles for kids: colour, space, balance, contrast
- Canva for Kids, templates, elements, and publishing
- Creating game covers and posters for your projects
- Vibe coding a portfolio: use Bolt.new to build your own showcase page
- Adding your Scratch games and vibe-coded apps to your portfolio
- Presentation design, making your showcase slides look amazing
- Building your Digital Portfolio, showcasing all 12 weeks of work

Projects built:
- My Vibe Track, compose an original beat in Sonic Pi
- Vibe-Coded Portfolio Website, tell Bolt.new to create a page showcasing your best projects
- Game Cover Design, a professional-looking cover for your dream game in Canva

Practice: Compose a 30 second Sonic Pi loop with two different beats and generate one AI artwork to be its album cover.

Week 12: Grand Vibe Showcase, Your Moment to Shine!
Topics:
- Finalizing your capstone Vibe Project, choosing your best idea
- Combining everything: block coding, vibe coding, AI, design, and music
- Project polishing and bug-free testing
- Writing a project description, SEO basics even for kids!
- Recording a demo video of your project
- Creating a 2-minute presentation pitch
- Peer review, giving kind and helpful feedback
- Receiving your Vibe Coder Kids Certification
- Sharing your work on the Modern Age Coders Hall of Fame
- What's next?, Preview of Vibe Coding for Teens
- Celebration and graduation ceremony

Projects built:
- GRAND CAPSTONE 'Vibe Project': An original multi-element creation combining Scratch game OR a vibe-coded app (built with Replit/Bolt.new), AI model, original music, and Canva design, presented live to parents and peers

Assessment: Final Capstone Assessment + Vibe Coder Kids Certification awarded

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Weeks 1-4): 9+ Scratch animations, stories & interactive projects
- Phase 2 (Weeks 5-8): 9+ games including Dream Game capstone
- Phase 3 (Weeks 9-12): 6+ AI, vibe coding & creative projects + Grand Capstone

Total projects: 15+ creative projects + Grand Vibe Showcase Capstone

SKILLS THE STUDENT WILL HAVE AT THE END

- Block Coding: Scratch 3.0, sprites, backdrops, motion, looks, sound blocks
- Programming Logic: loops, conditionals, events, variables, collision detection
- Game Development: score systems, lives, levels, enemy AI, game physics
- Vibe Coding: Replit Agent, Bolt.new, building apps by describing in English
- AI & Machine Learning: Google Teachable Machine, ML4Kids, Google Gemini
- Music Coding: Sonic Pi, beats, loops, live coding
- Digital Design: Canva, portfolio building, presentation skills
- Prompt Engineering: writing clear instructions for AI tools

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 sessions (1.5 hours each)
Independent projects: 1-3 hours
Vibe challenge: 1 fun weekly challenge
Total per week: 4-6 hours

SUPPORT INCLUDED

Live instruction: Expert-led live classes, never pre-recorded videos
Age appropriate: Content designed specifically for ages 8-12
Safe environment: Fun, encouraging, and bully-free classroom
Community: Access to Modern Age Coders kids community
Progress updates: Weekly written progress notes to parents

CERTIFICATION DETAIL

Final certificate: Vibe Coder, Kids Level (Modern Age Coders Certified)
Capstone required: Grand Vibe Showcase project combining coding, AI, music & design
Portfolio: 15+ completed projects showcased in digital portfolio

WHERE THIS CAN LEAD

- Digital Creator & Storyteller
- Junior Game Designer
- Creative Technologist
- AI Explorer
- Ready for Vibe Coding for Teens program

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Vibe Coding for Kids?
A: Vibe Coding for Kids is a 3-month, project-based program at Modern Age Coders designed for children aged 8-12. 'Vibe Coding' means building apps and software by describing what you want in plain English and letting AI create the code, a term coined by AI pioneer Andrej Karpathy in 2025 (Collins Dictionary Word of the Year!). Kids start with Scratch to learn coding fundamentals, then graduate to real vibe coding using AI tools like Replit, Bolt.new, and Google Gemini to build their own apps.

Q: What age is this course designed for?
A: This course is specifically designed for children aged 8 to 12. The content, pace, and teaching style are all optimized for this age group, with age-appropriate projects and language.

Q: Does my child need any prior coding experience?
A: Absolutely not! Vibe Coding for Kids is a complete beginner program. We start from zero, no prior coding or technical knowledge is required. All you need is curiosity and a computer!

Q: What tools and platforms will my child use?
A: Your child will use Scratch 3.0, Code.org, Replit (AI-powered browser coding), Bolt.new (AI app builder with instant preview), Google Gemini (AI chat), Google Teachable Machine, Sonic Pi (music coding), Canva, and ML4Kids. All of these are free, safe, and browser-based, nothing to install. These are the same AI tools professionals use for vibe coding in 2026.

Q: How many hours per week does the course require?
A: We recommend 4-6 hours per week, which includes 2 live classes (1.5 hours each) plus 1-3 hours of independent project time. Small batches make missed classes easy to recover: the mentor recaps in the next session, notes and the week's challenge are shared over WhatsApp, and longer absences get a short free catch-up slot.

Q: What will my child have built by the end of the course?
A: By the end of 12 weeks, your child will have built 15+ projects including animated stories, original video games, a trained AI model, apps built using real vibe coding (AI app builders), original music tracks, their own portfolio website, and a full digital portfolio, all showcased at a Grand Vibe Showcase event.

Q: Is there a certificate at the end?
A: Yes! Every student who completes the course and their capstone project receives the official 'Vibe Coder, Kids Level' certification from Modern Age Coders.

Q: What comes after this course?
A: Upon completing Vibe Coding for Kids, students are ready for the Vibe Coding for Teens program, where they learn real programming languages (Python, JavaScript), build apps, and explore advanced AI, building directly on the foundation from this course.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/vibe-

coding-for-kids-beginners-ai-scratch-game-dev
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========
COURSE 134 OF 137: VIBE CODING FOR TEENS: PYTHON, WEB DEV & AI (AGES 13-17)
==========

Course link: https://learn.modernagecoders.com/courses/vibe-coding-for-teens-python-web-ai-projects-course
Subject area: Teen Coding & Web Development
Designed for: Teens
Who can join: Beginner to Intermediate (Ages 13-17)
Course length: 3 months (12 weeks)
Time needed: 6-8 hours/week
Certificate: Vibe Coder, Teens Level Certification

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150
  Classes: group and mini batch two live classes a week, eight a month; one to one in India one
  live class a week, four a month; one to one outside India two live classes a week, eight a
  month. Each class is one hour.

WHAT THIS COURSE IS

  Live online vibe coding course for teens 13-17: learn Python, HTML, CSS, JavaScript and AI
  projects, build real apps and a portfolio in a 3-month program.

  In one line: Stop Scrolling. Start Building. Your Code. Your World.

ABOUT THE PROGRAMME

  Vibe Coding for Teens is the definitive 3-month coding program for ages 13-17 who want to go
  from beginner (or block-coding graduate) to building actual websites, Python apps, and
  AI-powered projects using the most cutting-edge tools in tech. 'Vibe Coding', coined by AI
  pioneer Andrej Karpathy in 2025 (Collins Dictionary Word of the Year!), means building
  software by describing what you want in natural language and letting AI generate the code.
  But here's the secret the pros know: the best vibe coders are also great traditional coders.
  That's why this course teaches you BOTH, you'll learn Python and JavaScript fundamentals AND
  master AI-powered development tools like Cursor IDE, GitHub Copilot, v0.dev, Bolt.new, and
  Lovable. At Modern Age Coders, we've designed this curriculum around what teens actually want
  to build: apps that solve real problems, games they can share with friends, websites that
  look professional, and AI tools that feel like the future. You'll use the same tools that
  professional developers use every day, not toy platforms. Graduate with a portfolio of 12+
  real projects that proves you can code, with AND without AI assistance.

WHAT MAKES THIS COURSE DIFFERENT

  - Real languages: Python and JavaScript, not toy platforms
  - Real vibe coding tools: Cursor IDE, GitHub Copilot, v0.dev, Bolt.new, Lovable
  - Learn BOTH ways: traditional coding fundamentals + AI-assisted vibe coding
  - Project-first approach, you build something real every single week
  - Portfolio focus, graduate with 12+ projects for college/job applications
  - Industry-relevant tools: Cursor, VS Code, Git, GitHub, APIs
  - AI-integrated projects, use and build AI tools
  - Prompt engineering, learn to communicate with AI effectively
  - Peer coding sessions, collaborate like real dev teams
  - 1-on-1 code reviews with expert instructors
  - Access to the Modern Age Coders teen developer community
  - Hackathon preparation and coding competitions support

THE LEARNING PATH, STEP BY STEP

  Phase 1: Python Vibes (Weeks 1-4): Python fundamentals with GitHub Copilot, functions, games,
  and automation scripts
  Phase 2: Web Dev Vibes (Weeks 5-8): HTML, CSS, JavaScript + vibe coding with v0.dev and
  Bolt.new, build responsive websites and web apps
  Phase 3: AI & Vibe Coding Mastery (Weeks 9-12): Cursor IDE, AI-powered development, Lovable
  for app building, final portfolio, deployment & showcase

WHAT A STUDENT NEEDS BEFORE STARTING

  Age: 13-17 years old
  Experience: Complete beginner friendly, no prior coding needed (Vibe Coding for Kids
    graduates will move faster)
  Equipment: Computer or laptop (Windows, Mac, or Linux) with stable internet
  Software: All tools are free, Cursor IDE, VS Code, GitHub Copilot (free for students),
    Python, Git

Motivation: Curiosity and willingness to build real projects every week

WHO THIS COURSE IS FOR

Complete beginners: Teens aged 13-17 with zero coding experience who want to start building real projects
Scratch graduates: Kids who completed block coding and want to learn real programming languages
Aspiring developers: Teens who dream of building their own apps, games, or websites
College prep: Students who want a strong portfolio for CS/tech college applications
Hackathon enthusiasts: Teens interested in coding competitions, hackathons, and tech clubs
Creative builders: Anyone who wants to bring ideas to life using code and AI

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: PYTHON VIBES, THINK LIKE A DEVELOPER (WEEKS 1-4)

Teens dive deep into Python, the world's most popular and versatile programming language. From variables and loops to building actual games and automation tools, Phase 1 transforms a complete beginner into a confident Python programmer.

Week 1: Python Unlocked, Your Developer Setup & First Programs
Topics:
- Why Python? The language of AI, data, and the web
- What is Vibe Coding?, Andrej Karpathy's 2025 concept that changed software forever
- The vibe coding workflow: Describe AI Generates You Review Iterate Ship
- Setting up VS Code & Cursor IDE, your professional AI-powered developer environment
- Setting up GitHub Copilot (free for students!), your AI pair programmer
- Python basics: print(), variables, strings, integers, floats
- String manipulation, slicing, formatting, f-strings
- Getting user input with input() function
- Basic arithmetic and math operations in Python
- Boolean logic: True, False, and comparisons
- Your first command-line programs
- Understanding error messages, the developer's best friend
- Running Python files from the terminal
- Introduction to Git, saving your code like a pro
- Creating your GitHub profile and first repository
Projects built:
- Personal Bio Generator, asks 5 questions and prints a formatted story about you
- Simple Calculator, performs all 4 operations with user input
- Mad Libs Creator, fill-in-the-blank story generator
Practice: Set up your editor, write a Python script that asks the user three questions and prints a profile card, then commit it to a new GitHub repo.

Week 2: Flow Control, Making Smart Programs
Topics:
- If, elif, else, decision-making in code
- Nested conditionals for complex logic
- While loops, repeating until a condition is met
- For loops, iterating over sequences
- Range() function, generating number sequences
- Loop control: break, continue, pass
- Lists, storing multiple values in one variable
- List methods: append, remove, sort, reverse, index
- Tuples and when to use them
- String loops, processing text character by character
- Nested loops for patterns and grids
- Debugging with print() statements and VS Code debugger
Projects built:
- Number Guessing Game, with hints, attempts counter, and high score
- Times Table Generator, print any multiplication table on demand
- Password Strength Checker, analyses a password and gives a strength rating
Practice: Build a number guessing game with a while loop and if elif else hints for higher or lower, counting the attempts taken.

Week 3: Functions & Modules, Writing Clean, Reusable Code
Topics:
- Defining functions with def keyword
- Parameters and arguments, passing data to functions
- Return values, getting data back from functions
- Default parameters and keyword arguments
- Scope: local vs. global variables
- Importing Python's built-in modules: math, random, datetime, os
- The random module, creating unpredictable programs
- Working with the datetime module for time-based features

- Writing your own modules and importing them
- Docstrings, documenting your functions properly
- Lambda functions, quick, anonymous functions
- Recursion basics, functions that call themselves

Projects built:
- Dice Rolling Simulator, roll any number of sides, track statistics
- Tip Calculator, restaurant bill splitter with formatted output
- Word Counter, analyses a paragraph for word count, unique words, and top 5 most used

Practice: Refactor your guessing game into functions with parameters and return values, then add a play again option.

Week 4: Data Structures & File Handling, Real Persistent Apps

Topics:
- Dictionaries, key-value pairs for structured data
- Dictionary methods: get, keys, values, items, update
- Nested dictionaries, representing complex real-world data
- Sets, unique collections and set operations
- List comprehensions, Pythonic one-line transformations
- Dictionary comprehensions
- Reading and writing text files with open()
- File modes: r, w, a, r+ explained
- Working with CSV files, reading datasets
- JSON files in Python, the language of the web
- Error handling with try, except, finally
- Building a simple contact book, CRUD operations

Projects built:
- Contact Manager, add, search, update, delete contacts saved to a file
- Student Grade Tracker, stores grades in JSON and calculates averages
- Vocabulary Flash Card App, add words, quiz yourself, track score

Practice: Create a contact book app that stores entries in a dictionary and saves them to a file so they survive a restart.

PHASE 2: WEB DEV VIBES, BUILD THE INTERNET (WEEKS 5-8)

Now teens build things the whole world can see. Phase 2 covers HTML, CSS, and JavaScript, the core technologies of every website on the internet. By Week 8, students will have built fully responsive, beautifully designed websites deployed live on the internet.

Week 5: HTML, Structure is Everything

Topics:
- How the web works, browsers, servers, HTTP basics
- HTML document structure: DOCTYPE, html, head, body
- Semantic HTML: header, nav, main, section, article, footer
- Text elements: h1-h6, p, strong, em, blockquote
- Links and navigation: anchor tags, href, relative vs. absolute URLs
- Images: img tag, alt text for accessibility and SEO
- Lists: ordered, unordered, nested lists
- Tables, displaying structured data
- HTML Forms: input types, labels, buttons, textareas
- HTML attributes: id, class, style
- Accessibility basics: ARIA roles, alt text, heading hierarchy
- Vibe Coding HTML: using v0.dev to generate page layouts from descriptions
- Comparing hand-written HTML vs AI-generated HTML, understanding what AI creates
- Validating your HTML with W3C Validator

Projects built:
- Personal Portfolio Page v1, bio, skills, and contact section in pure HTML
- Recipe Website, multi-page site with ingredients and instructions
- Event Landing Page, HTML structure for a fictional school event

Practice: Hand code a semantic personal profile page with a header, nav, three sections and a contact link, without any AI assistance.

Week 6: CSS, Make It Look Amazing

Topics:
- CSS syntax: selectors, properties, values
- The Box Model: margin, padding, border, content
- CSS colours: hex, RGB, HSL, and CSS variables
- Typography: fonts, Google Fonts integration, font-size, line-height
- CSS Flexbox, modern, powerful layout system
- CSS Grid, two-dimensional layout mastery
- Responsive design with Media Queries
- Mobile-first design principles
- CSS animations and transitions
- Pseudo-classes: hover, focus, active, nth-child
- CSS custom properties (variables) for design systems
- Dark mode implementation with CSS
- Vibe Coding CSS: using v0.dev and Bolt.new to generate beautiful UI components from text prompts
- Reading AI-generated Tailwind CSS, understanding utility-first styling

Projects built:
- Fully Styled Portfolio, transform Week 5's HTML into a beautiful, responsive portfolio
- Product Card Component, an Instagram-worthy card with hover effects
- Responsive Navigation Bar, a professional nav that collapses on mobile

Practice: Style your profile page with Flexbox, a Google Font and CSS variables, and make sure it still looks right at phone width.

Week 7: JavaScript, Bring Your Website to Life

Topics:
- What is JavaScript and why it makes the web interactive
- JS basics: variables (let, const), data types, operators
- Functions in JavaScript, declarations, expressions, arrow functions
- Arrays and array methods: map, filter, reduce, forEach
- Objects in JavaScript, structuring real-world data
- The DOM, Document Object Model explained simply
- Selecting elements: getElementById, querySelector
- Changing content and styles with JavaScript
- Event listeners: click, mouseover, input, submit
- Form validation with JavaScript
- Fetch basics, getting data from the internet
- Local Storage, persisting data in the browser

Projects built:
- Interactive To-Do App, add, complete, and delete tasks (persists with LocalStorage)
- Quiz App, 10-question quiz with timer, score, and results screen
- Real-time Character Counter, live word/character counts as you type

Practice: Add a dark mode toggle and a live character counter to your page using DOM events, plus one array method such as map or filter.

Week 8: APIs & Full Web Projects, Connect to the Real World

Topics:
- What is an API? REST APIs explained with real examples
- Fetch API, making HTTP GET requests in JavaScript
- Async/await, handling asynchronous operations cleanly
- JSON parsing, working with API response data
- Error handling in fetch requests
- Public APIs: Weather, Quotes, Space, Pokemon, SpaceX
- Displaying API data dynamically in your webpage
- API keys, working with authenticated APIs safely
- Introduction to the OpenWeather API
- Deploying your website with Netlify, live in 60 seconds
- Custom domain basics
- Vibe Coding a full web app: using Bolt.new to build and deploy a project in minutes
- Comparing your hand-coded projects vs vibe-coded projects, understanding the trade-offs
- Phase 2 portfolio review, making your projects job-ready

Projects built:
- Live Weather App, fetch real weather data for any city using OpenWeather API
- Random Quote Machine, fetches and displays random motivational quotes
- Space Explorer, displays real NASA photo of the day with Astronomy API

Practice: Fetch data from a free public API, such as weather or jokes, and render it on your page with async await and visible error handling.

PHASE 3: AI & VIBE CODING MASTERY, BUILD YOUR FUTURE (WEEKS 9-12)

The most exciting phase, teens master real vibe coding tools (Cursor IDE, Lovable, Claude/Gemini APIs), combine Python + Web skills to build AI-powered projects, set up their professional developer portfolio, and prepare for real-world opportunities: internships, hackathons, and competitive coding. This is where you become a true vibe coder, someone who can build anything by combining traditional coding skills with AI-powered development.

Week 9: AI Tools & Vibe Coding Mastery, Build with AI Like a Pro

Topics:
- Introduction to AI APIs, using AI as a superpower
- Working with the Google Gemini API (or Claude API), setup and authentication
- Chat Completion API, building a simple AI chatbot
- System prompts, guiding AI behaviour for your use case
- Deep dive into Cursor IDE, the #1 AI-powered code editor for vibe coding
- Cursor Composer: describe a feature in English Cursor writes the code across multiple files
- Cursor Tab: AI auto-completes your code as you type
- GitHub Copilot Chat, asking your AI pair programmer questions about your code
- Prompt engineering for code: writing clear, specific prompts that get better results
- AI for text: summarization, sentiment analysis, translation
- Image generation API basics with teen-legal tools (Firefly and Bing Image Creator APIs, concepts first)
- Comparing AI models: Claude Sonnet (best for code), GPT-4, Gemini, which to use when
- AI ethics for teen developers, bias, transparency, and responsibility

Projects built:
- AI Study Buddy, paste any topic, get an explanation in simple language (Python + Gemini API)
- Mood Journal Analyser, enter your journal entry, AI analyses the mood and gives advice

- Vibe-Coded AI App, use Cursor Composer to build an entire AI tool from a single description

Practice: Build a small chatbot on the Gemini or Claude API with a system prompt that gives it a personality, and note one prompt improvement you made.

Week 10: Vibe Coding Full Apps, Lovable, Bolt.new & Full Stack

Topics:

- Lovable, the AI app builder that generates clean React code from conversation
- Building a complete web app with Lovable: describe features AI codes you iterate
- Bolt.new deep dive, building and deploying apps entirely through AI prompts
- When to vibe code vs. hand code, understanding the trade-offs and limitations
- Introduction to Flask, Python for the web (understanding what AI builds for you)
- Setting up a Flask app, routes and views
- Connecting your AI Python code to a web frontend
- Building a simple AI web app, Python backend + HTML frontend
- Environment variables and .env files, keeping API keys safe
- Introduction to databases: SQLite with Python basics
- Deploying your apps: Replit, Vercel, Netlify, Render (free tier)
- Security basics: why vibe-coded apps need human review before going live
- Planning your capstone project, choosing between vibe-coded or hand-coded

Projects built:

- Lovable App, build a complete app by chatting with AI (e.g., habit tracker, expense tracker)
- AI Chatbot Web App, a proper web interface for your Week 9 AI chatbot using Flask
- Bolt.new Speed Build, go from idea to deployed app in under 30 minutes

Practice: Build the same to do app twice, once by hand and once with Lovable or Bolt.new, and write down three differences you noticed in the code.

Week 11: Developer Portfolio & Professional Presence

Topics:

- What is a developer portfolio and why every teen dev needs one
- Structuring your portfolio: projects, about, skills, contact
- Writing project descriptions that impress, STAR format
- README files on GitHub, making your repos presentable
- Vibe coding your portfolio website: use v0.dev or Lovable to build a stunning portfolio in minutes
- Picking your best 4 projects for the portfolio (mix of hand-coded and vibe-coded)
- GitHub profile optimization, pins, README profile, contributions graph
- LinkedIn basics for teen developers, setting up a professional profile
- Personal branding, your developer identity and niche
- Technical interview basics, what companies actually ask
- Writing a tech resume for internships and college applications
- Open source contribution basics, finding good first issues
- Hackathons: how to find, join, and win at hackathons

Projects built:

- Professional Developer Portfolio Website, multi-section, deployed, polished and live
- GitHub Profile README, a custom animated profile readme showcasing your skills
- Project README Collection, professional documentation for all 4 key portfolio projects

Practice: Draft your portfolio site with three project cards, STAR style descriptions and a proper README on your best repo.

Week 12: Grand Vibe Showcase, Launch Your Career!

Topics:

- Final capstone project polish, code review and bug fixes
- Code documentation best practices
- Recording a 3-minute project demo video
- The art of the technical demo, explaining code to non-coders
- Preparing for Q&A, answering questions about your code
- The vibe coding toolkit recap, Cursor, Copilot, v0.dev, Bolt.new, Lovable: when to use each
- Introduction to competitive coding: HackerRank, LeetCode, CodeChef
- What's next?, College applications with a portfolio, internships, freelancing
- Vibe Coding for College preview, MERN, DSA, Machine Learning, Claude Code, Windsurf, and more
- Networking with the Modern Age Coders alumni community
- Receiving the Vibe Coder Teens Certification
- Graduation ceremony and class showcase

Projects built:

- GRAND CAPSTONE, 'Vibe Project Launch': A complete, full-stack (or AI-powered) web application built using a combination of traditional coding and vibe coding tools (Cursor, Lovable, or Bolt.new). Must be deployed live, documented on GitHub, and presented at the showcase with a 3-minute live demo explaining both what you built AND how you used AI to build it

Assessment: Final Capstone Code Review + Live Demo + Panel Q&A

PROJECTS THE STUDENT BUILDS AND KEEPS

- Phase 1 (Weeks 1-4): 9+ Python CLI projects, games, calculators, automation tools
- Phase 2 (Weeks 5-8): 9+ web projects, portfolio, responsive sites, API-connected apps
- Phase 3 (Weeks 9-12): 6+ AI & vibe-coded projects + Grand Capstone deployed live

Total projects: 12+ real projects deployed live + professional portfolio

SKILLS THE STUDENT WILL HAVE AT THE END

- Python: variables, loops, functions, OOP basics, file handling, JSON, error handling
- Web Development: HTML5, CSS3, Flexbox, Grid, responsive design, JavaScript ES6+
- JavaScript: DOM manipulation, event listeners, Fetch API, LocalStorage, async/await
- Vibe Coding Tools: Cursor IDE, GitHub Copilot, v0.dev, Bolt.new, Lovable
- AI & APIs: Google Gemini API, Claude API, prompt engineering, chatbot development
- Backend Basics: Flask, Jinja2, SQLite, REST API concepts
- DevOps: Git, GitHub, deployment on Netlify, Vercel, Render
- Professional Skills: portfolio building, README writing, LinkedIn optimization

HOW A TYPICAL WEEK IS SPLIT

Live sessions: 2 sessions (1.5-2 hours each)
Independent projects: 3-4 hours
Vibe challenge: 1 weekly creative challenge
Total per week: 6-8 hours

SUPPORT INCLUDED

Live instruction: Expert-led live classes, never pre-recorded videos
Code reviews: 1-on-1 code reviews with instructors
Peer coding: Collaborative coding sessions with classmates
Community: Access to Modern Age Coders teen developer community
Hackathon prep: Guidance for coding competitions and hackathons

CERTIFICATION DETAIL

Final certificate: Vibe Coder, Teens Level (Modern Age Coders Certified)
Capstone required: Complete full-stack or AI-powered web app, deployed live, presented at showcase
Portfolio: 12+ completed projects + professional developer portfolio website

WHERE THIS CAN LEAD

- Junior Web Developer (freelance or internship)
- Python Developer Intern
- Tech Entrepreneur / App Creator
- Competitive Programmer (ICPC, CodeChef, HackerRank)
- Freelance Web Developer (Rs 500-1,500/hour)
- Tech Club Leader / School Tech Projects
- Ready for Vibe Coding for College, MERN, DSA, ML & more

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What is Vibe Coding for Teens?
A: Vibe Coding for Teens is a 3-month, project-first coding program for ages 13-17. 'Vibe Coding', coined by AI pioneer Andrej Karpathy in 2025 (Collins Dictionary Word of the Year!), means building software by describing what you want in natural language and letting AI generate the code. Teens learn both traditional coding (Python, HTML/CSS/JavaScript) AND modern vibe coding tools (Cursor IDE, GitHub Copilot, v0.dev, Bolt.new, Lovable). You graduate with 12+ real projects and the ability to build anything, with or without AI assistance.

Q: Do I need to have done Vibe Coding for Kids first?
A: No! Vibe Coding for Teens is designed to welcome complete beginners aged 13-17. If you completed Vibe Coding for Kids, you'll find Phase 1 comes very naturally to you and you'll move faster. But anyone with curiosity and a laptop can join from zero.

Q: What programming languages and AI tools will I learn?
A: You will learn Python (Phases 1 and 3) and the core web trio, HTML, CSS, and JavaScript (Phase 2). But what makes this course special is the vibe coding tools: Cursor IDE (the #1 AI code editor), GitHub Copilot (free for students!), v0.dev (AI UI generator by Vercel), Bolt.new (AI app builder), and Lovable (builds full-stack React apps from conversation). You'll also learn AI APIs (Gemini / Claude), Git, and GitHub. By the end, you'll know which tool to use for any project.

Q: Will I be able to build real websites and apps by the end?
A: Absolutely. By Week 12, you will have built and deployed multiple live websites and web applications that anyone can visit on the internet. You'll also have Python CLI apps, an AI-powered project, and a professional portfolio on GitHub.

Q: How much time should I set aside each week?
A: We recommend 6-8 hours per week: 2 live sessions (1.5-2 hours each) plus 3-4 hours of independent project work. The more time you put in, the more impressive your portfolio will be.

Q: What tools and software will I need?

A: You need a computer (Windows, Mac, or Linux) and a stable internet connection. All primary tools are free: Cursor IDE (free tier), VS Code, GitHub Copilot (free for students via GitHub Education), Python, Git, v0.dev (free tier), Bolt.new (free tier), Lovable (free tier), and Replit (free tier). We walk through every setup in Week 1. No paid subscriptions are required.

Q: Can this help with college applications or internship applications?
A: Yes! A strong developer portfolio with real, deployed projects is an incredible differentiator in college applications (especially for CS/tech programs) and internship applications. Our Week 11 module specifically teaches you how to present your portfolio professionally.

Q: What comes after Vibe Coding for Teens?
A: Graduates are ready for Vibe Coding for College, which dives into Data Structures & Algorithms, full MERN stack development, Machine Learning with Python, cloud deployment, professional AI tools (Claude Code, Windsurf), and building production-grade applications with advanced vibe coding and agentic engineering workflows.

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/vibe-coding-for-teens-python-web-ai-projects-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 135 OF 137: WATERLOO MATH CONTESTS: GAUSS TO EUCLID

==========

Course link: https://learn.modernagecoders.com/courses/waterloo-math-contests-gauss-to-euclid-course
Subject area: Competition Mathematics & Olympiad Training
Designed for: Middle school
Who can join: Students roughly aged 11 to 18 (Grades 7 to 12) who are comfortable with school math and want harder problems, in a Gauss track (Grades 7 and 8), a Grades 9 and 10 track and a Grades 11 and 12 track
Course length: 6 months (24 weeks) per season, joinable any month and timed to the CEMC contests each student will write
Time needed: 2 live classes/week + weekly problem sets, written solutions and timed practice rounds
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment (this is not a CEMC certificate or result)

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online prep for CEMC's Waterloo math contests, ages 11 to 18: Gauss, Pascal, Cayley, Fermat, Euclid, Fryer, Galois, Hypatia, CIMC and CSMC, with grade tracks and proof writing.

In one line: One problem-solving toolkit, three grade tracks: from the first Gauss counting ideas to the clear, complete written solutions the Euclid asks for.

ABOUT THE PROGRAMME

The Centre for Education in Mathematics and Computing (CEMC) at the University of Waterloo runs a family of math contests "Designed by teams of expert mathematicians and educators for Grades 5 to 12". Each contest is matched to a grade. The Gauss Contest is for "Students in Grades 7 and 8.", and CEMC adds that "Motivated students in lower grades are welcome to write the contest." The Pascal, Cayley and Fermat Contests are for Grade 9, Grade 10 and Grade 11. The Fryer, Galois and Hypatia Contests are for the same three grades, with "4 questions; a mix of final answer-only and full-solution." The Euclid Contest is for "Students in their final year of secondary school or CÉGEP students." and has "10 questions; a mix of final answer only and full-solution." The CIMC is for Grades 9 and 10 and the CSMC for Grades 11 and 12 and CÉGEP students, each with "9 questions; 6 are answer only and 3 are full-solution." For the 2026/27 contest year CEMC lists the CIMC and CSMC on Wednesday, November 18, 2026; the Pascal, Cayley and Fermat Contests on Tuesday, February 23, 2027 and Wednesday, February 24, 2027; the Euclid on Tuesday, April 6, 2027; the Fryer, Galois and Hypatia on Wednesday, April 7, 2027; and the Gauss from Monday, May 10, 2027 to Friday, May 21, 2027 (dates for North and South America). Students write through their school: CEMC says "It is not possible for students to individually register for a contest" and "all students must participate through their school." This course teaches the mathematics behind all of these contests in three grade tracks, and it teaches the two skills they test in different ways: fast, accurate work on multiple-choice and answer-only questions, and full solutions that explain every step. Practice comes from two places: our own original problems written for each topic, and CEMC's free past contests, which CEMC publishes with no registration or fee and which students work where CEMC publishes them. We never copy CEMC problems into our materials. We teach the ideas and the habits, and we make no promise of scores, certificates of distinction, prizes, scholarships or admissions.

WHAT MAKES THIS COURSE DIFFERENT

- Built around CEMC's contests by grade: Gauss for Grades 7 and 8, then Pascal, Cayley, Fermat, Fryer, Galois, Hypatia, CIMC, CSMC and Euclid
- Three grade tracks taught in separate batches, so every student works on problems at the right level
- Full-solution writing taught directly: stating the idea, showing every step and checking the answer, as the full-solution contests require
- Our own original problems for every topic, plus CEMC's free past contests used as practice where CEMC publishes them
- A timed practice round every month, in the format of the contest each student will write next
- Honest about entry and results: students write through their school, and we promise learning, not scores

THE LEARNING PATH, STEP BY STEP

Phase 1: The shared toolkit for every track: careful reading, number sense, algebra, counting, geometry, multiple-choice technique and the first full written solutions
Phase 2: The grade tracks: Gauss for Grades 7 and 8; Pascal, Cayley, Fryer, Galois and CIMC for Grades 9 and 10; Fermat, Hypatia, CSMC and Euclid for Grades 11 and 12
Phase 3: Contest season: timed rounds, full-solution polish, contest-day routine for each 2026/27 contest, then review and next steps

WHAT A STUDENT NEEDS BEFORE STARTING

Math level: Comfortable with school math for the student's grade, and happy to spend time on a hard problem
Age: Roughly 11 to 18 (Grades 7 to 12). The track is chosen by grade and level after a short diagnostic
Equipment: A computer with stable internet, paper and a pencil, and a way to share photos of written solutions
Contest entry: CEMC contests are written through a school; check that your school (or another local school) is taking part
Timing: Weekdays or weekends, flexible: choose your batch, or for a very personalised plan choose 1-on-1 at your convenience, weekend mornings or weekdays

WHO THIS COURSE IS FOR

Gauss students: Grade 7 and 8 students who enjoy puzzles and want to write the Gauss with real preparation
Grade 9 and 10 students: Students preparing for the Pascal, Cayley, Fryer, Galois or CIMC
Grade 11 and 12 students: Students preparing for the Fermat, Hypatia, CSMC or Euclid, including full-solution writing
Returning contest writers: Students who wrote a CEMC contest before and want to reach the harder questions
Students who want more: Students who find school math easy and want problems that make them think

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: THE SHARED TOOLKIT (MONTHS 1-2, WEEKS 1-8)

Everything all three tracks need: how to read a contest problem, the core ideas of number, algebra, counting and geometry, and the first full written solutions.

Month 1: Reading Problems, Number Sense and Algebra (Weeks 1-4)
Weeks 1-2: How the Waterloo Contests Work and First Problems
Topics:
- The CEMC contest family by grade, and which contests each student will write this year
- A short diagnostic to place each student on the Gauss, Grades 9 and 10, or Grades 11 and 12 track
- Multiple-choice, answer-only and full-solution questions, and what each one rewards
- Reading a problem twice: listing what is given, what is asked and what is hidden
- Trying small cases, drawing a diagram and making a table before calculating
- Starting an error log on day one

Projects built:
- A personal starting profile: track, target contests for the year and ten solved warm-up problems

Practice: Ten short original problems, each with the idea written in one sentence before the answer

Weeks 3-4: Number Sense and Algebra Habits
Topics:
- Fractions, ratios, percentages and averages handled exactly
- Factoring and expanding to make calculations short
- Setting up an equation from a word problem
- Systems of two equations solved by substitution and elimination

- Working backwards and guess-and-check used on purpose
- Checking an answer by substituting it back

Projects built:
- A word-problem pack of original problems, each solved with a clear equation and a check

Practice: A first CEMC past contest at the student's level, from CEMC's site, attempted untimed with a written review

Assessment: Month 1 timed round: a short practice round of original problems

Month 2: Counting, Geometry and Writing a Full Solution (Weeks 5-8)

Weeks 5-6: Counting and Geometry Foundations

Topics:
- Systematic lists, tree diagrams and the multiplication rule
- Counting by complements: count what you do not want and subtract
- Angles in triangles, parallel lines and polygons
- Area and perimeter of composite figures
- The Pythagorean theorem and when to look for a right triangle
- Patterns and sequences: finding the rule and testing it

Projects built:
- A counting and geometry problem set of original problems, with a labelled diagram for every geometry question

Practice: Mixed problems timed at a few minutes each, then reviewed for method, not only answers

Weeks 7-8: Full Solutions and Multiple-Choice Technique

Topics:
- What a full solution looks like: the idea, every step and a final statement
- Writing in complete sentences with clear algebra and labelled diagrams
- Explaining why, not only what: justifying each step
- Multiple-choice technique: estimation, checking options and eliminating wrong ones
- Pacing a 60-minute, 25-question multiple-choice paper
- Reviewing a paper: sorting mistakes into careless, method and knowledge

Projects built:
- Rewrite three of your own earlier answers as full solutions, then compare them with a model write-up

Practice: One full-solution question each class, marked with written comments on clarity and completeness

Assessment: Phase 1 assessment: a timed round with multiple-choice and full-solution questions, reviewed question by question

---

PHASE 2: THE GRADE TRACKS (MONTHS 3-4, WEEKS 9-16)

The tracks run in parallel in separate batches. Each track builds the mathematics and the question types of the contests its students will write.

Months 3-4, Gauss Track (Grades 7 and 8): Building the Gauss Toolkit (Weeks 9-16)

Weeks 9-10: Number Theory for Grades 7 and 8

Topics:
- Factors, multiples, primes and prime factorization
- Divisibility rules and why they work
- Greatest common factor and least common multiple in word problems
- Remainders and repeating patterns in long lists
- Digits and place value puzzles
- Choosing an organized list instead of guessing

Projects built:
- An original number puzzle pack, each solution explained in two or three sentences

Practice: Gauss-style multiple-choice sets of original problems, with a written check for every answer

Weeks 11-12: Ratios, Rates and Algebra in Words

Topics:
- Ratios and rates, including speed, distance and time
- Percent increase and decrease done in one step
- Averages: using the total instead of the list
- Writing and solving one-variable equations from words
- Simple patterns turned into algebraic rules
- Units and reasonableness checks

Projects built:
- A ratio and rate problem set solved two ways: once with a table, once with an equation

Practice: Timed sets of ten problems, with every miss logged and redone

Weeks 13-14: Counting and Probability

Topics:
- Counting arrangements and choices with the multiplication rule
- Counting without double counting
- Simple probability as favourable outcomes over equally likely outcomes
- Probability with two dice, coins and spinners
- Logic puzzles and careful case checking
- Spotting a counting question inside a story

Projects built:
- Design and solve three original counting problems for a classmate

Practice: Counting and probability problems, each with a list or tree drawn first

  Weeks 15-16: Geometry and Measurement
    Topics:
      - Angle facts in triangles, quadrilaterals and regular polygons
      - Area of triangles, trapezoids and circles, and shaded-region questions
      - Surface area and volume of prisms and cylinders
      - The Pythagorean theorem in two-step problems
      - Coordinates on a grid: distance and area
      - Drawing the diagram accurately before calculating
    Projects built:
      - A shaded-region problem pack of original problems, each with a labelled diagram
    Practice: A full 60-minute, 25-question practice paper of original problems
    Assessment: Gauss track assessment: a full timed paper reviewed question by question

Months 3-4, Grades 9 and 10 Track: Pascal, Cayley, Fryer, Galois and CIMC (Weeks 9-16)
  Weeks 9-10: Algebra for Contests
    Topics:
      - Linear equations, systems and inequalities set up from words
      - Factoring quadratics and the difference of squares
      - Exponent laws and simplifying powers
      - Substitution to make a hard expression simple
      - Linear relations: slope, intercepts and lines on a grid
      - Writing an algebra argument as a full solution
    Projects built:
      - An original algebra problem set with full written solutions for the last three questions
    Practice: Mixed multiple-choice and answer-only problems of original design, timed and reviewed
  Weeks 11-12: Number Theory and Counting
    Topics:
      - Prime factorization and counting divisors
      - Remainders and simple modular reasoning
      - The multiplication rule, arrangements and selections
      - Counting with restrictions, and counting by complements
      - Probability from counting
      - Organized casework and how to check every case is covered
    Projects built:
      - A casework problem pack where each solution lists its cases clearly
    Practice: Counting problems at several minutes each, with the counting method named before the answer
  Weeks 13-14: Geometry with Reasons
    Topics:
      - Similar triangles and ratios of lengths and areas
      - The Pythagorean theorem in two and three dimensions
      - Circles: radius, chords and angles, with reasons given
      - Coordinate geometry: distance, midpoint and area on the plane
      - Justifying each geometry step in words
      - Choosing between a coordinate approach and a pure geometry approach
    Projects built:
      - Three original geometry problems solved with full written reasons
    Practice: Geometry full-solution questions in the style of the Fryer and Galois, marked with written comments
  Weeks 15-16: Full Solutions and Mixed Rounds
    Topics:
      - The answer-only and full-solution mix of the Fryer, Galois and CIMC
      - Writing a full solution under time without losing clarity
      - Showing that an answer is the only one, not just that it works
      - Pacing a 60-minute, 25-question Pascal or Cayley style paper
      - Partial progress on a hard question: writing down what you can prove
      - Reviewing and rewriting a solution after feedback
    Projects built:
      - A personal model-solution file: five of your own best full solutions, rewritten after feedback
    Practice: A full timed practice paper of original problems in the style of the student's next contest
    Assessment: Grades 9 and 10 track assessment: a timed mixed paper plus one full solution marked in detail

Months 3-4, Grades 11 and 12 Track: Fermat, Hypatia, CSMC and Euclid (Weeks 9-16)
  Weeks 9-10: Functions, Equations and Inequalities
    Topics:
      - Quadratic functions: vertex form, roots and the discriminant
      - Systems of equations, including non-linear systems
      - Functions and their graphs, and reading information from a graph
      - Exponential and logarithmic equations
      - Inequalities and how to prove one holds
      - Writing a function argument as a full solution
    Projects built:
      - An original functions problem set with full written solutions for every question
    Practice: Answer-only and full-solution questions of original design, timed and reviewed

    Weeks 11-12: Sequences, Number Theory and Counting
      Topics:
        - Arithmetic and geometric sequences and their sums
        - Divisibility, remainders and integer solutions
        - Counting arrangements and selections, and the binomial coefficients
        - Probability built from counting
        - Recognizing a pattern and proving it holds in general
        - Casework that is complete and clearly organized
      Projects built:
        - A sequences and counting pack where each solution proves its pattern, not just finds it
      Practice: Mixed problems at the level of the later answer-only questions, with the idea
      written first
    Weeks 13-14: Trigonometry and Geometry
      Topics:
        - Trigonometric ratios, the sine law and the cosine law
        - Trigonometric equations and identities
        - Circle geometry with full reasons
        - Coordinate geometry: lines, circles and distances
        - Areas and lengths by more than one method
        - Labelling a diagram so a written solution is easy to follow
      Projects built:
        - Three original geometry problems, each solved by two different methods and written up in full
      Practice: Geometry and trigonometry full-solution questions, marked with written
      comments
    Weeks 15-16: Proof Writing for Full-Solution Contests
      Topics:
        - Direct proof, proof by cases and proof by contradiction
        - Proving that every case is covered, and that an answer is the only one
        - Writing for a reader: definitions, steps and a clear conclusion
        - The Euclid's mix of answer-only and full-solution questions over 2.5 hours
        - The CSMC and Hypatia full-solution questions and how to earn partial marks
        - Reviewing and rewriting a proof after feedback
      Projects built:
        - A personal proof file: five of your own full solutions, each rewritten after detailed
          feedback
      Practice: A full timed practice paper of original problems in the style of the
      student's next contest
      Assessment: Grades 11 and 12 track assessment: a timed mixed paper plus one proof
      marked in detail

PHASE 3: CONTEST SEASON (MONTHS 5-6, WEEKS 17-24)

Timed rounds in the format of each student's next contest, polish on full solutions, the
contest-day routine, then review and a plan for the next contest.

  Month 5: Timed Rounds and Contest Craft (Weeks 17-20)
    Weeks 17-18: Pacing Each Contest Format
      Topics:
        - Pacing a 60-minute, 25-question multiple-choice paper: Gauss, Pascal, Cayley and Fermat
        - Pacing a 75-minute, 4-question paper: Fryer, Galois and Hypatia
        - Pacing a 2-hour, 9-question paper: CIMC and CSMC
        - Pacing a 2.5-hour, 10-question paper: Euclid
        - Deciding when to move on from a stuck question
        - Using CEMC's past contests from the last 10 years as timed practice on CEMC's site
      Projects built:
        - Two full timed practice papers, each followed by a written pacing review
      Practice: One full timed paper a week at the student's level, then a careful review of
      every miss
    Weeks 19-20: Review and Weak-Spot Repair
      Topics:
        - Redoing every missed question unaided after a timed paper
        - Reading the error log to find the real weak spot
        - Targeted mini-lessons on each student's weakest topic
        - Polishing full solutions: shorter, clearer and complete
        - A short personal checklist for the last minutes of a paper
        - Making sure the school has the student on its contest order
      Projects built:
        - A personal weak-spot plan with three topics, each with a problem set completed
      Practice: Original problem sets on each student's weak topics
      Assessment: Month 5 assessment: a full timed paper reviewed one to one with the teacher

  Month 6: Final Practice, Contest Days and Next Steps (Weeks 21-24)
    Weeks 21-22: Final Practice Papers
      Topics:
        - Final timed papers in the exact format of the student's next contest
        - Calm starts: reading the whole paper and choosing an order
        - Checking answers and units in the last minutes
        - Full-solution presentation one last time: idea, steps, conclusion

- Rest and practice balance in the last two weeks
- Reading CEMC's current instructions for the contest together

Projects built:
- A final practice paper with a written contest-day plan

Practice: Light, steady practice: short problems that keep skills sharp

Weeks 23-24: Contest Week and After

Topics:
- Contest-day routine for the 2026/27 dates: CIMC and CSMC on November 18, 2026; Pascal, Cayley and Fermat on February 23 and 24, 2027; Euclid on April 6, 2027; Fryer, Galois and Hypatia on April 7, 2027; Gauss from May 10 to May 21, 2027
- After the contest: reviewing the paper once CEMC publishes it
- What the student learned from the contest, whatever the result
- Choosing the next contest in the CEMC calendar
- Moving up a track for the next season when ready
- Presenting a favourite solution of the season to the class

Projects built:
- A next-contest plan based on the error log and the practice-paper history

Practice: Write a full solution to your favourite problem of the season for a newer student

Assessment: Final assessment: a last timed paper plus a short conversation about the student's method choices

PROJECTS THE STUDENT BUILDS AND KEEPS

- Starting profile with track and target contests for the year
- Word-problem, counting and geometry packs of original problems
- Full solutions rewritten after feedback
- Number theory, ratio and shaded-region packs (Gauss track)
- A model-solution file of full solutions (Grades 9 and 10 track)
- A proof file of full solutions (Grades 11 and 12 track)
- Timed practice papers with written pacing reviews and a next-contest plan

Total projects: A season of solved original problems, reviewed CEMC past contests, a file of the student's own full solutions, an error log and a practice-paper history

SKILLS THE STUDENT WILL HAVE AT THE END

- Reading unfamiliar problems carefully and choosing an idea
- Number sense, number theory and algebra for contests
- Counting, probability and organized casework
- Geometry and trigonometry with written reasons
- Sequences, functions and inequalities (Grades 11 and 12 track)
- Writing clear full solutions and proofs
- Pacing multiple-choice, answer-only and full-solution papers

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, each grade track in its own batches

Practice: A problem set after every class, with core and stretch problems

Review: Written feedback on full solutions, a timed round every month and a monthly error log check

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with hints first and full solutions after a real attempt

Progress updates: A monthly progress note to parents with practice-paper results and next steps

Free past contests: CEMC's past contests from the last 10 years, free with no registration, used as practice where CEMC publishes them

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. It records real learning and is not a CEMC certificate or contest result. Fall short and there is focused revision and a free retest

WHERE THIS CAN LEAD

- A second season of this course, moving up a track or deeper into full-solution writing
- Senior school math such as Ontario's MHF4U and MCV4U, through our Grade 12 university math course
- Contest programming, such as the Canadian Computing Competition, through our CCC preparation course
- University mathematics, computer science or engineering, with strong problem-solving habits
- Further math contests and personal problem-solving projects

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: What are the Waterloo math contests?
A: They are the math contests run by CEMC (the Centre for Education in Mathematics and Computing) at the University of Waterloo, "Designed by teams of expert mathematicians and educators for Grades 5 to 12". This course covers the Gauss, Pascal, Cayley, Fermat and Euclid Contests, the Fryer, Galois and Hypatia Contests, and the CIMC and CSMC. CEMC also says: "Our contests are not tests or exams. There is no passing or failing score."

Q: Which contest should my child write?
A: CEMC matches each contest to a grade. Gauss is for "Students in Grades 7 and 8." Pascal is for Grade 9, Cayley for Grade 10 and Fermat for Grade 11. Fryer, Galois and Hypatia are for Grades 9, 10 and 11. The CIMC is for Grades 9 and 10, the CSMC for Grades 11 and 12 and CÉGEP students, and the Euclid for "Students in their final year of secondary school or CÉGEP students." In the first two weeks we place each student on a track and plan the contests for the year.

Q: When are the 2026/27 contests?
A: CEMC lists these dates for North and South America: CIMC and CSMC on Wednesday, November 18, 2026; Pascal, Cayley and Fermat on Tuesday, February 23, 2027 and Wednesday, February 24, 2027; Euclid on Tuesday, April 6, 2027; Fryer, Galois and Hypatia on Wednesday, April 7, 2027; and Gauss from Monday, May 10, 2027 to Friday, May 21, 2027. Schools order the contests before CEMC's deadlines, so check the current dates on CEMC's site with your school.

Q: How do we register for a contest?
A: Through a school, not through us. CEMC says "It is not possible for students to individually register for a contest" and "all students must participate through their school." If a student's school does not take part, CEMC suggests that "students can look for another local school that will allow them to join as a participant". We remind families to check the school's order early.

Q: What is the difference between multiple-choice and full-solution contests?
A: Gauss, Pascal, Cayley and Fermat each have 25 multiple-choice questions in 60 minutes, so speed and accuracy matter most. Fryer, Galois and Hypatia have "4 questions; a mix of final answer-only and full-solution." in 75 minutes, and the Euclid has "10 questions; a mix of final answer only and full-solution." in 2.5 hours. On a full-solution question the reasoning is marked, not just the answer, so we teach students to write clear, complete solutions.

Q: Do you use CEMC past contests?
A: Yes, the way CEMC intends. CEMC says "There is no registration or fee required to access these materials." and its past contests page says "Below you will find resources from the last 10 years." We set those past contests as practice, which students work on CEMC's site, and review them in class beside our own original problems. We never copy CEMC problems into our materials.

Q: Will this course get my child a certificate of distinction or a top score?
A: No course can promise that, and we do not. Results depend on the student, the paper and the day. What the course gives is steady practice, timed rounds, written feedback on full solutions and an error log, so progress is visible month by month.

Q: Do the Waterloo contests matter for university?
A: They are a good challenge, not a requirement. The University of Waterloo's Computer Science page says writing the CSMC or Euclid "is not required for admission, but writing one of these contests is required for some scholarships offered by the Faculty of Mathematics." Waterloo's Admission Information Form also says "This form asks about any Waterloo mathematics contests you've participated in". Always check each university's current pages. We make no admission or scholarship promises.

Q: Can a younger student write the Gauss?
A: CEMC says "Motivated students in lower grades are welcome to write the contest." This course starts at about age 11. A younger student who is strong in math can join the Gauss track after the free demo class, if the teacher agrees the level fits. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

Q: Does my child need to be top of the class in math?
A: No. A student should be comfortable with school math for their grade and willing to work on problems that take time. Contest problems feel unfamiliar at first for almost everyone; the course builds the habits that make them approachable, one idea at a time.

Q: How are the classes run?
A: Live online classes, twice a week, with a real teacher, in a group of 15 to 20 students, a mini batch of up to 6, or 1-on-1. Each grade track is taught in its own batches. Weekdays or weekends, flexible: choose your batch, or for a very personalised plan choose 1-on-1 at your convenience, weekend mornings or weekdays. Current fees for your region are shown in the pricing section of this page.

Q: Can we try a class first?

A: Yes. Book a free demo class. The teacher works through a short original problem with your child, suggests the right track and contests for the year, and recommends group, mini batch or 1-on-1 classes. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/waterloo-math-contests-gauss-to-euclid-course
For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 136 OF 137: WEB DEVELOPMENT FOR KIDS: BUILD REAL WEBSITES

==========

Course link: https://learn.modernagecoders.com/courses/kids-ai-web-development-course
Subject area: Kids Coding
Designed for: Kids
Who can join: Complete beginner, ages 9 to 12
Course length: 8-10 months (32-40 weeks)
Time needed: 2 live classes/week + 2-3 hours practice
Certificate: Certificate from Modern Age Coders, awarded on passing the final exam

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
Outside India, per month: group $100, one to one $150
Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Web development course for kids aged 9-12: HTML, CSS and first JavaScript in live online classes, with real published websites in 8-10 months.

In one line: The moment a child types real code and a real website appears is unforgettable. Eight months from that first tag to published sites they built themselves.

ABOUT THE PROGRAMME

There is a special kind of pride in a child saying "I made this website", and meaning typed, real, published code. This live online course gives that to children aged 9 to 12: HTML learned as the structure of every page, CSS as the art layer, layouts with flexbox and grid, responsive design tested on the family phone, and first JavaScript, the sparks that make pages respond. Every project is hand-typed by the child; this course builds real skill, not copy-paste. Publishing is the heartbeat: pages go live to real web addresses the family can visit, and the course ends with a capstone site, planned, built, polished and presented at demo day. Near the end, one honest week looks ahead at how professional developers use AI assistants, including why those tools have age rules (most are 13+, some 18+) and why the skills built in this course are exactly what will make such tools useful later. Nothing in the course requires any AI account, because nine-year-olds should be building their own skill, and the tools' own rules agree. The pace is honest and kid-sized: eight months at two live classes a week plus two to three hours of playful practice, with two more months in hand when needed. Homework after every class, a mixed challenge at the end of each month, and a final exam, a capstone demo plus a friendly viva, that decides the certificate, with a free retest after revision. Real code, real websites, real pride.

WHAT MAKES THIS COURSE DIFFERENT

- Real typed code from day one: no drag-and-drop site builders, no copy-paste dependence
- Publishing as a habit: children's pages go live at real web addresses their family can visit
- Kid-sized honest pace: 8 months at 2 classes a week, with 2 more in hand when needed
- Age-honest about AI: one look-ahead week explains how developers use AI assistants and why those tools are 13+ and 18+, no child accounts required or used
- Design taste built alongside code: spacing, color and layout judged out loud, kindly
- Real assessment, gently done: monthly mixed challenges and a final capstone exam with a free retest

THE LEARNING PATH, STEP BY STEP

Phase 1: HTML world (Months 1-2): structure, text, images, links, forms and multimedia, hand-typed from the first day
Phase 2: CSS magic (Months 3-4): color, typography, the box model, flexbox and grid
Phase 3: JavaScript sparks (Months 5-6): responsive design, then first real programming: logic, loops and little games
Phase 4: Real sites (Months 7-8): functions and DOM magic, publishing properly, the AI look-ahead, and the capstone exam

WHAT A STUDENT NEEDS BEFORE STARTING

  Coding experience: None. The course starts at the very first tag
  Reading typing: Comfortable reading; typing improves fast within the course
  Equipment: A computer or laptop with a stable internet connection; every tool used is free
  Age: 9 to 12. Younger children start better in our Kids Coding (Scratch-first) course
  Time commitment: 2 live classes plus 2-3 hours of playful practice a week

WHO THIS COURSE IS FOR

  Young builders: Children who want to make real things the whole family can visit online
  Scratch graduates: Kids who finished block coding and are hungry for real typed code
  Creative kids: Children who love drawing and design, CSS is where art meets code
  Future teens: Kids who will roll naturally into our teen web and full-stack courses
  Proud families: Parents who want screen time producing published, showable work

FULL CURRICULUM, WEEK BY WEEK

  This is the complete teaching plan. Parents can ask about any week.

  PHASE 1: HTML WORLD (MONTHS 1-2, WEEKS 1-8)

  The structure of every website, hand-typed: text, images, links, forms, tables and
  multimedia, each week ending with a page to show the family.

    Month 1: First Pages (Weeks 1-4)
      Welcome to Web Development
        Topics:
          - What is a website? How does the internet work (kid-friendly explanation)
          - Setting up your coding playground: VS Code with fun themes
          - Installing colorful extensions and emoji support
          - Your first HTML file: Hello World with emojis!
          - Understanding tags: like LEGO blocks for websites
          - Creating a personal 'About Me' page
          - HTML structure: <html>, <head>, <body> explained with stories
          - Saving and viewing your first webpage
          - Celebrating your first website with screenshots!
          - Typing practice for young coders: a little every day
        Projects built:
          - Build a digital pet profile page
          - Make a birthday invitation webpage
      HTML Elements Zoo - Text, Images & Links
        Topics:
          - Heading tags (h1-h6): Creating titles like newspaper headlines
          - Paragraph tag: Writing stories on your webpage
          - Making lists: Favorite games, friends, hobbies (ul, ol, li)
          - Bold and italic: Making text stand out (strong, em)
          - Adding images: Your photos, drawings, and GIFs
          - Image attributes: alt text for accessibility
          - Creating links: Connect your pages like a treasure map
          - Link to other websites: YouTube, games, educational sites
          - Making a photo album webpage
          - Building a collection of favorite things
        Projects built:
          - Digital photo album with captions
          - My favorite YouTube videos collection
          - Interactive storybook with multiple pages
          - School project presentation website
        Practice: Build 5 themed pages (animals, sports, hobbies, etc.)
        Assessment: Month 1 check: build a small page live from a sketch, plus a fun take-home
        challenge

    Month 2: HTML Powers (Weeks 5-8)
      Fun with Forms & Tables
        Topics:
          - Creating forms: Like making a quiz for friends
          - Input types: text, email, password, number
          - Fun input types: color picker, date picker, range slider
          - Checkboxes for multiple choices: Pizza toppings selector
          - Radio buttons for single choice: Favorite superhero poll
          - Dropdown menus: Choose your adventure options
          - Textarea: Writing stories and comments
          - Tables for organizing information: Sports scores, schedules
          - Making tables pretty with borders and colors
          - Building interactive surveys
        Projects built:
          - Friend information collector form
          - Class schedule table

- Restaurant menu with order form
Practice: Create 5 different interactive forms
Multimedia Magic - Audio, Video & Canvas
Topics:
- Adding videos to your webpage: YouTube embeds
- HTML5 video tag: Playing your own videos
- Audio elements: Adding music and sound effects
- Creating playlists on your webpage
- Canvas basics: Drawing on webpages
- Embedding games and interactive content
- iframe: Windows to other websites
- Making a multimedia gallery
- Creating a personal YouTube-style page
- Building an online portfolio of creations
Projects built:
- Personal video channel page
- Music playlist website
- Digital art gallery with canvas
- Multimedia learning resource page
Practice: Add multimedia to all previous projects
Assessment: Month 2 check: a multimedia page built live, with quiz questions reaching back to month 1

PHASE 2: CSS MAGIC (MONTHS 3-4, WEEKS 9-16)

The art layer: colors, fonts, the box model, then the two great layout tools, flexbox and grid, used to make pages genuinely beautiful.

Month 3: Style Arrives (Weeks 9-12)
CSS Basics - Making Websites Beautiful
Topics:
- What is CSS? The fashion designer for HTML
- Three ways to add CSS: inline, internal, external stylesheets
- CSS syntax: selector { property: value; }
- Selecting elements: tags, classes, and IDs
- Colors everywhere: background-color, color
- Color codes: named colors, hex codes, RGB
- Font families: Changing how text looks
- Google Fonts: Adding fun fonts to websites
- Text size, weight, and style
- Text alignment and decoration
Projects built:
- Rainbow text website
- Colorful mood board page
- Typography playground
Practice: Style 10 HTML pages with different themes
CSS Box Model & Spacing Magic
Topics:
- The Box Model: Every element is a box!
- Padding: Space inside the box
- Margin: Space outside the box
- Border: The box's frame (styles, colors, radius)
- Width and height: Sizing your boxes
- Making round corners with border-radius
- Creating shapes with CSS: circles, triangles
- Box shadows for 3D effects
- Background images and patterns
- Building card-based layouts
Projects built:
- Trading card collection webpage
- Social media profile page clone
- Product showcase with cards
- CSS art project with shapes
Practice: Create 10 different card designs
Assessment: Month 3 check: style a plain page live to match a design, plus mixed review

Month 4: Layout Superpowers (Weeks 13-16)
Flexbox - The Flexible Layout System
Topics:
- Introduction to Flexbox: Making layouts flexible
- Flex container and flex items
- Flex-direction: row, column, reverse
- Justify-content: Spacing items horizontally
- Align-items: Spacing items vertically
- Creating navigation bars with Flexbox
- Building photo galleries
- Flexbox games: Flexbox Froggy, Flexbox Defense
- Making responsive card layouts

- Creating hero sections
      Projects built:
- Responsive navigation menu
- Pinterest-style photo gallery
- Flexible dashboard layout
- Game character selection screen
      Practice: Complete all Flexbox Froggy levels
    CSS Grid - The Ultimate Layout Tool
      Topics:
- CSS Grid basics: Rows and columns
- Grid-template-columns and grid-template-rows
- Grid gaps: Adding space between items
- Placing items in specific grid cells
- Grid areas: Naming your layout sections
- Creating magazine-style layouts
- Building game boards with Grid
- Grid Garden game for practice
- Combining Flexbox and Grid
- Responsive grids with auto-fit
- Creating complex layouts easily
      Projects built:
- Magazine homepage layout
- Tic-tac-toe game board
- Calendar layout with Grid
- Minecraft-style inventory grid
      Practice: Build 10 different grid layouts
      Assessment: Month 4 check: build a card layout live with flexbox or grid, plus mixed review

---
PHASE 3: JAVASCRIPT SPARKS (MONTHS 5-6, WEEKS 17-24)

Pages that fit every screen, then the big step: first real programming, decisions, loops and little games, in the language of the web.

  Month 5: Every Screen, First Code (Weeks 17-20)
    Responsive Design - Mobile First!
      Topics:
- What is responsive design? One site for all devices
- Mobile-first approach: Start small, go big
- Media queries: Different styles for different screens
- Breakpoints: Phone, tablet, desktop
- Responsive units: %, vw, vh, rem, em
- Responsive images: srcset and picture element
- Responsive typography: Scaling text
- Testing on different devices
- Chrome DevTools device simulator
- Mobile-friendly navigation patterns
- Touch-friendly button sizes
      Projects built:
- Responsive portfolio website
- Mobile-first blog layout
- Adaptive gaming website
- Responsive school project site
      Practice: Make all previous projects responsive
    JavaScript Basics - Making Websites Interactive
      Topics:
- What is JavaScript? The brain of websites
- Adding JavaScript to HTML pages
- Console.log: Talking to the computer
- Variables: Storing information (let, const)
- Data types: Numbers, strings, booleans
- Basic math operations
- String concatenation: Joining words
- Template literals: Easy text formatting
- Comments in JavaScript
      Projects built:
- Interactive calculator
- Mad libs story generator
- Age calculator with messages
- Simple greeting program
      Practice: Write 20 simple JavaScript programs
      Assessment: Month 5 check: make a page respond to a click live, plus mixed review of earlier months

  Month 6: Logic, Loops and Little Games (Weeks 21-24)
    JavaScript Logic & Decisions
      Topics:
- If statements: Making decisions

- Else and else if: Multiple choices
- Comparison operators: >, <, ==, ===
- Logical operators: AND (&&), OR (||), NOT (!)
- Switch statements for multiple cases
- Ternary operator: Quick decisions
- Random numbers with Math.random()
- Building simple games with logic
- Creating quiz logic
- Score keeping systems

Projects built:
- Rock, Paper, Scissors game
- Number guessing game
- Quiz with scoring
- Adventure choice game

Practice: Build 10 decision-based programs

Loops & Arrays - Handling Multiple Items

Topics:
- Arrays: Lists of items
- Accessing array elements
- Array methods: push, pop, length
- For loops: Repeating actions
- While loops: Keep going until...
- Looping through arrays
- forEach: Modern array looping
- Creating patterns with loops
- Nested loops for complex patterns
- Building lists dynamically
- Array-based games

Projects built:
- Todo list manager
- Student grade calculator
- Pattern generator
- Memory card game logic

Practice: Solve 20 loop and array challenges

Assessment: Phase check: a little browser game built live, plus a mixed challenge spanning phases 1-3

PHASE 4: REAL SITES (MONTHS 7-8, WEEKS 25-32)

Functions and DOM magic, proper publishing, one honest look at how professionals use AI, and the capstone site presented at demo day.

Month 7: DOM Magic (Weeks 25-28)

Functions - Reusable Code Blocks

Topics:
- What are functions? Code recipes
- Creating simple functions
- Parameters: Giving functions information
- Return values: Getting answers back
- Function expressions vs declarations
- Arrow functions: Modern syntax
- Scope: Where variables live
- Built-in functions: Math, String methods
- Creating a function library
- Organizing code with functions

Projects built:
- Utility function library
- Calculator with functions
- Text analyzer tool
- Game with multiple functions

Practice: Create 30 useful functions

DOM Manipulation - Changing Websites with JavaScript

Topics:
- What is the DOM? The website's structure
- Selecting elements: getElementById, querySelector
- Changing text content dynamically
- Changing styles with JavaScript
- Adding and removing CSS classes
- Creating new elements
- Adding elements to the page
- Removing elements
- Show/hide elements
- Building dynamic interfaces
- Click, hover and keyboard events: pages that listen

Projects built:
- Dynamic content generator
- Interactive story with choices
- Color theme switcher

- DOM-based game
        Practice: Manipulate DOM in 20 different ways
        Assessment: Month 7 check: add an interactive feature to an old page live, plus mixed
        review

    Month 8: Publish, Look Ahead, and the Finale (Weeks 29-32)
      Publish and Shine
        Topics:
          - Publishing properly: your site at a real web address
          - The pre-launch checklist: links, images, spelling, phones
          - Accessibility for kind builders: alt text, contrast, readable fonts
          - THE HONEST AI LOOK-AHEAD: how professional developers use AI assistants today
          - Why the big AI chatbots are 13+ and some tools 18+, and why waiting is fine
          - Watching the teacher pair with an AI assistant on a class demo, then spotting its mistake
            together
          - The skills that will make AI useful later: exactly what you are building now
        Projects built:
          - Every good page from the course, live at real addresses
          - A class "spot the AI mistake" detective session
        Practice: Send your favorite live link to three family members
      Capstone and the Final Exam
        Topics:
          - Choosing the capstone: a site about something you love
          - Planning pages on paper before typing
          - Building with everything: HTML, CSS layouts, JavaScript sparks
          - Polish: spacing, colors, small screens
          - Rehearsing the demo: what it is, how it works, what was hard
        Projects built:
          - THE CAPSTONE: a complete multi-page website, published and presented at demo day
        Practice: A little building every day; keep it always working
        Assessment: FINAL EXAM: capstone demo plus a friendly viva and a mixed quiz from the
        whole course. Passing earns the certificate; a child who falls short gets focused
        revision and a free retest

PROJECTS THE STUDENT BUILDS AND KEEPS

  - Phase 1: hand-typed pages about hobbies, pets and heroes, each shown at home the same week
  - Phase 2: styled and laid-out pages, card galleries and a small design portfolio
  - Phase 3: responsive pages and first JavaScript games
  - Phase 4: interactive features, published sites, and the capstone presented at demo day

  Total projects: 25+ hand-built pages and mini-sites, crowned by a published capstone website

SKILLS THE STUDENT WILL HAVE AT THE END

  - HTML: real structure, typed by hand, validator-clean
  - CSS: color, typography, the box model, flexbox and grid
  - Responsive design: pages tested on the family phone
  - First JavaScript: logic, loops, functions and DOM magic
  - Publishing: real sites at real addresses, with a launch checklist
  - AI honesty: how developers use assistants, and why those tools wait until 13+

HOW A TYPICAL WEEK IS SPLIT

  Live classes: 2 live one-hour classes per week, typing along with the teacher
  Homework: A small build after every class, and a mixed challenge at the end of every month
  Practice: 2-3 hours of playful practice between classes
  Review: Every page reviewed with teacher feedback; wobbly skills reopened in class

SUPPORT INCLUDED

  Doubt support: WhatsApp doubt support between classes, for the page that suddenly went
    sideways
  Progress updates: Regular progress notes to parents, with live links to what your child
    published

CERTIFICATION DETAIL

  Completion: Certificate from Modern Age Coders, awarded on passing the final exam, a capstone
    demo plus a friendly viva. A child who falls short gets focused revision and a free retest:
    the certificate certifies skills, not attendance

WHERE THIS CAN LEAD

  - Full Stack for Teens: the direct continuation once they turn 13
  - Python for Kids: a second language while the web skills settle
  - JavaScript for Teens: deepening the sparks lit here
  - School projects that suddenly look professional
  - The family website: every club, trip and pet now has a webmaster

COMMON QUESTIONS ABOUT THIS COURSE

  For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

  Q: What will my child build in this web development course?
  A: More than 25 hand-typed pages and mini-sites across eight months: hobby pages, styled galleries, responsive layouts, first JavaScript games, and finally a capstone, a complete multi-page website about something they love, published to a real web address and presented at demo day. Everything is typed by the child; there are no drag-and-drop shortcuts and no copy-paste dependence.

  Q: What age is right for this course?
  A: Ages 9 to 12. Web development involves real typing, and this range handles it well and grows fast. Younger children are better served by our Kids Coding course, which starts with blocks and builds toward exactly this. Batches are grouped by age so the pace fits the room.

  Q: How long does it take a child to learn web development?
  A: Eight months at 2 live classes a week plus 2 to 3 hours of playful practice takes a child from first tag to a published multi-page capstone. The course keeps two more months in hand for children who need more repetition, no rushing and no penalty. The first live published page happens within the first month, and the pride is immediate.

  Q: How is my child assessed? Is the certificate just for attending?
  A: No, and it is all gentle: classwork in every session, a small build after every class, a monthly mixed challenge that revisits earlier skills, and a final exam at the end, a capstone demo plus a friendly viva and short quiz. Passing earns the certificate; a child who falls short gets focused revision and a free retest. Real skills, kindly tested.

  Q: Does this course use AI tools like ChatGPT?
  A: No, deliberately, and we are open about why: the major AI chatbots require users to be 13 or older (some tools 18+), and a nine-year-old building their own skill should be doing exactly that. The course includes one honest look-ahead week where children see how professional developers use AI assistants, watch the teacher demo one, and learn why the skills they are building are what will make such tools useful when they are old enough.

  Q: Will my child's websites really be on the internet?
  A: Yes: publishing is a habit here, not a finale. Pages go live to real addresses on free hosting through the course, and the capstone launches properly with a checklist. Grandparents can visit the URL; that moment is usually the one that hooks a child for good.

  Q: Is any paid software needed?
  A: No. VS Code, the browser and free hosting cover the entire course. Any reasonably recent computer or laptop is enough; web pages are the lightest thing a computer can build.

  Q: Is there a free trial class?
  A: Yes, a free demo class with no commitment: book at learn.modernagecoders.com/contact or message us on WhatsApp at +91 91233 66161. Your child will type and see a real page inside the demo itself. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

  Q: Are classes live or recorded?
  A: Live, always: small groups with a real teacher who sees your child's code as they type it, never pre-recorded videos. At this age the teacher's eyes matter most, catching the missing bracket before it becomes tears.

  To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/kids-ai-web-development-course
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==========

COURSE 137 OF 137: WESTERN CANADA GRADE 12 MATH: PRE-CALCULUS 12 AND MATH 30-1

==========

Course link: https://learn.modernagecoders.com/courses/western-canada-grade-12-math-pre-calculus-12-math-30-1-course
Subject area: Western Canada High School Mathematics
Designed for: High school
Who can join: Students in Grade 12 (roughly ages 16 to 18) taking Pre-calculus 12 in British Columbia or Math 30-1 in Alberta
Course length: 6 months (24 weeks) per plan, joinable any month and paced to your school course
Time needed: 2 live classes/week + short practice sets between classes
Certificate: Certificate from Modern Age Coders, awarded on passing the final assessment (this is not a high school credit in British Columbia, Alberta or any province)

PRICE FOR THIS COURSE (quote only the one that applies to the person asking)
  In India, per month: group Rs 1,499, mini batch Rs 2,999, one to one Rs 4,999
  Outside India, per month: group $100, one to one $150

Classes: group and mini batch two live classes a week, eight a month; one to one in India one live class a week, four a month; one to one outside India two live classes a week, eight a month. Each class is one hour.

WHAT THIS COURSE IS

Live online help for BC Pre-calculus 12 and Alberta Math 30-1, ages 16 to 18, taught alongside school: functions, logarithms, trigonometry and counting. We do not grant credits.

In one line: A second teacher for Grade 12 math in British Columbia and Alberta: functions, logarithms, trigonometry and counting understood properly, in step with your school course.

ABOUT THE PROGRAMME

Pre-calculus 12 is one of the Grade 12 courses on British Columbia's mathematics course list, beside Foundations of Mathematics 12 and Calculus 12. In Alberta, Mathematics 30-1 is one of the courses with a provincial diploma exam, and the diploma exam counts for a large share of the final mark. Alberta's diploma exams overview says "The weighting of diploma exams returned to 30% effective September 1, 2023." and "70% of the final mark comes from course work. The remaining 30% comes from the diploma exam." Alberta's provincial achievement tests page adds that "students write diploma exams and provincial achievement tests in a digital environment rather than pencil and paper". This course runs alongside the student's own Pre-calculus 12 or Math 30-1 class, and we follow the order of the student's school course outline so what is learned here is used in class the same week. What our course teaches: functions and their transformations, including inverses; polynomial, radical and rational functions; exponential and logarithmic functions; trigonometry in radians, with identities and equations; and permutations, combinations and the binomial theorem. Every topic starts from meaning, with graphs and small examples, before the algebra. For Alberta students we add exam technique for the Math 30-1 diploma exam, practised on screen; for BC students we prepare for the school's own unit tests and final exam. Tests and assignments stay the student's own work: we explain, question and review, and the student does the math. Please read this plainly: Modern Age Coders teaches alongside school and does not grant high school credits in British Columbia, Alberta or any other province. Families who need the credit take the course through their school or an approved provider. What we add is understanding, practice and confidence, in live classes with a real teacher. We make no promise of marks or diploma exam results.

WHAT MAKES THIS COURSE DIFFERENT

- Built for BC Pre-calculus 12 and Alberta Math 30-1 students, following the student's own school course outline week by week
- Meaning before algebra: graphs, tables and small examples come first, then the rules
- Exam technique for the Alberta Math 30-1 diploma exam, which counts for 30% of the final mark, practised on screen
- Honest about credits: we teach alongside school and do not grant high school credits
- Tests and assignments stay the student's own work: we explain and review, we never do school work for them
- A mistakes list and timed review papers, so revision targets what each student actually gets wrong

THE LEARNING PATH, STEP BY STEP

Phase 1: Functions and transformations, then polynomial, radical and rational functions
Phase 2: Exponential and logarithmic functions, then trigonometry in radians with identities and equations
Phase 3: Permutations, combinations and the binomial theorem, combining functions, and exam review for school finals and the Alberta diploma exam

WHAT A STUDENT NEEDS BEFORE STARTING

School course: Currently taking, or about to take, Pre-calculus 12 in British Columbia or Math 30-1 in Alberta
Math level: Comfortable with the Grade 11 math that came before, which we refresh in the first two weeks
Age: Roughly 16 to 18 (Grade 12)
Equipment: A computer with stable internet, a scientific or graphing calculator, and a way to share photos of written work
Timing: Weekdays or weekends, flexible: choose your batch, or for a very personalised plan choose 1-on-1 at your convenience, weekend mornings or weekdays

WHO THIS COURSE IS FOR

Bc pre calculus 12 students: BC students in Pre-calculus 12 who want functions, logarithms and trigonometry to make sense
Alberta math 30 1 students: Alberta students in Math 30-1 preparing for unit tests and the diploma exam
Students who feel behind: Students who missed a unit or lost confidence and want to catch up calmly
Students aiming higher: Students who are doing well and want cleaner, more complete written

solutions
Future stem students: Students heading toward calculus and science, engineering or computer science programs

FULL CURRICULUM, WEEK BY WEEK

This is the complete teaching plan. Parents can ask about any week.

PHASE 1: FUNCTIONS, TRANSFORMATIONS AND POLYNOMIAL, RADICAL AND RATIONAL FUNCTIONS (MONTHS 1-2, WEEKS 1-8)

The function toolkit every Grade 12 topic depends on, then the polynomial, radical and rational families.

Month 1: Functions and Transformations (Weeks 1-4)

Weeks 1-2: Starting Point and the Function Toolkit

Topics:

- A starting conversation and short diagnostic: the province, the school, the course outline, the next test and the Grade 11 ideas to refresh
- Function notation, domain and range, and reading a graph carefully
- Parent functions and their key points
- Factoring review: common factors, trinomials and the difference of squares
- Solving quadratic equations by factoring and by formula
- Setting up a clean notebook system for definitions, examples and mistakes

Projects built:

- A one-page function toolkit summary, with a sketch of each parent function and its key points

Practice: Short daily sets that mix new work with one or two Grade 11 refresher questions

Weeks 3-4: Transformations and Inverses

Topics:

- Translations, stretches and reflections, and the order to apply them
- Mapping notation for points on a transformed graph
- Writing the equation of a transformed function from its graph
- Inverse functions: finding them, graphing them and restricting the domain
- Reflections in the line y = x
- Common mistakes with horizontal stretches and shifts

Projects built:

- A transformation set: ten graphs transformed by hand from mapping rules, then checked with graphing technology

Practice: For every transformation, write the mapping rule before sketching

Assessment: Month 1 check: a short written test on functions, transformations and inverses, reviewed with written feedback

Month 2: Polynomial, Radical and Rational Functions (Weeks 5-8)

Weeks 5-6: Polynomial Functions

Topics:

- Degree, leading coefficient and end behaviour
- Long division and synthetic division
- The remainder theorem and the factor theorem
- Zeros and their multiplicity, and how the graph behaves at each one
- Sketching a polynomial from its factored form
- Writing an equation from a graph or from given zeros

Projects built:

- A polynomial sketch set with end behaviour, zeros and intercepts explained for each graph

Practice: Factor theorem and sketching sets, each answer checked by substitution or graph

Weeks 7-8: Radical and Rational Functions

Topics:

- Square root functions and their transformations
- Graphing y equal to the square root of a function, and its domain
- Rational functions: vertical asymptotes, horizontal asymptotes and holes
- Sketching rational functions from their factored form
- Solving radical and rational equations graphically and algebraically
- Checking for extraneous roots and restricted values

Projects built:

- A radical and rational function set with every restriction, asymptote and hole labelled

Practice: For every rational function, list restrictions and asymptotes first, then sketch

Assessment: Phase 1 assessment: a timed test on functions, transformations and polynomial, radical and rational functions, marked with comments

---

PHASE 2: EXPONENTIAL AND LOGARITHMIC FUNCTIONS, AND TRIGONOMETRY (MONTHS 3-4, WEEKS 9-16)

Growth, decay and logarithms, then trigonometry in radians with identities and equations.

Month 3: Exponential and Logarithmic Functions (Weeks 9-12)

Weeks 9-10: Exponential Functions

Topics:

- Exponential functions and their graphs
- Transformations of exponential functions
- Solving exponential equations with a common base
- Growth and decay: population, half-life and compound interest
- Choosing between a linear and an exponential model
- Reading an exponential graph to answer a question in context

Projects built:
- An applications pack: three real growth or decay situations modelled and solved in full

Practice: Mixed exponential equation sets, with answers checked on a graph

Weeks 11-12: Logarithms

Topics:
- The logarithm as the inverse of an exponential function
- Graphs of logarithmic functions and their transformations
- Laws of logarithms and when each one applies
- Solving exponential and logarithmic equations, and checking for extraneous roots
- Changing the base of a logarithm
- Logarithmic scales: sound, earthquakes and pH

Projects built:
- A logarithm laws file: ten equations solved in full, with each law named at the step where it is used

Practice: One log law proof and three equations every day

Assessment: Month 3 check: a written test on exponential and logarithmic functions, reviewed with written feedback

Month 4: Trigonometry in Radians, Identities and Equations (Weeks 13-16)

Weeks 13-14: Radians, the Unit Circle and Trigonometric Graphs

Topics:
- Radian measure, arc length and converting between radians and degrees
- The unit circle and exact values for special angles
- Graphs of sine, cosine and tangent
- Amplitude, period, phase shift and vertical displacement
- Writing a sinusoidal equation from a graph or a situation
- Modelling periodic situations such as tides and daylight hours

Projects built:
- A sinusoidal model of a real periodic situation, with the equation built from the data and explained

Practice: Unit circle and graph sets, each answer checked against a quick sketch

Weeks 15-16: Identities and Trigonometric Equations

Topics:
- Reciprocal, quotient and Pythagorean identities
- Sum, difference and double angle identities
- Proving identities: working one side and writing each step clearly
- Solving first and second degree trigonometric equations
- Finding all solutions in an interval, and the general solution
- Non-permissible values and common mistakes that lose solutions

Projects built:
- An identity proof file: ten identities proved in full, with the strategy for each written first

Practice: One identity proof and two equations every day, with every solution in the interval listed

Assessment: Phase 2 assessment: a mixed review paper on logarithms and trigonometry, marked with written comments

---

PHASE 3: PERMUTATIONS AND COMBINATIONS, COMBINING FUNCTIONS AND EXAM REVIEW (MONTHS 5-6, WEEKS 17-24)

Careful counting, combining functions, then exam review for school finals and, for Alberta students, the Math 30-1 diploma exam.

Month 5: Permutations, Combinations and Combining Functions (Weeks 17-20)

Weeks 17-18: Permutations, Combinations and the Binomial Theorem

Topics:
- The fundamental counting principle
- Permutations, including arrangements with repeated items and with restrictions
- Combinations, and how to tell a combination problem from a permutation problem
- Factorial notation and solving equations with factorials
- Pascal's triangle and the binomial theorem
- Finding a specific term in a binomial expansion

Projects built:
- A counting pack: twelve problems sorted by method, each solved with the reason for the method written first

Practice: Mixed counting sets, each starting with the question: does order matter?

Weeks 19-20: Combining Functions

Topics:
- Sums, differences, products and quotients of functions, and their graphs
- Composition of functions and its domain
- Sketching a combined function from the graphs of its parts
- Writing a function as a composition of simpler functions
- Domain restrictions that come from combining functions

- Links back to transformations, logarithms and trigonometry
Projects built:
- A combining functions set with graphs, equations and domains explained for each
Practice: Short daily sets that combine two functions from earlier units
Assessment: Month 5 check: a written test on counting and combining functions, reviewed with written feedback

Month 6: Exam Review Technique (Weeks 21-24)
Weeks 21-22: Mixed Review and Written Solutions
Topics:
- Making a one-page summary for each unit of the school course
- Mixed review sets that move between units without warning
- Full written solutions that show every step
- Using graphing technology to check, not to replace, the algebra
- Building and using a personal mistakes list
- Reading the teacher's exam outline so revision matches what will be assessed
Projects built:
- A personal review pack: unit summaries and a mistakes list with a corrected example for each mistake
Practice: A mixed review set after every class, with every mistake written up and fixed
Weeks 23-24: Timed Practice for School Finals and the Alberta Diploma Exam
Topics:
- Timed review papers in the style of school final exams, with careful review afterwards
- For Alberta students: pacing a long exam and checking answers before moving on
- For Alberta students: practising on screen, since diploma exams are written in a digital environment
- Reading Alberta's current diploma exam schedule and instructions together
- For BC students: matching revision to the school's own final exam outline
- A calm exam-week routine: sleep, short practice and one last review of the mistakes list
Projects built:
- Two reviewed timed papers and a written exam-week plan
Practice: A timed mixed review each week, reviewed in class
Assessment: Final assessment: a timed review paper plus a short live walk-through of two of the student's own solutions

PROJECTS THE STUDENT BUILDS AND KEEPS

- A function toolkit summary and a transformation set
- Polynomial, radical and rational function sketch sets
- Exponential applications pack and a logarithm laws file
- A sinusoidal model and an identity proof file
- A counting pack sorted by method
- A combining functions set
- A personal review pack and two reviewed timed papers

Total projects: A file of worked solutions, sketch sets, proofs and review papers, all the student's own work

SKILLS THE STUDENT WILL HAVE AT THE END

- Functions, transformations and inverses
- Polynomial, radical and rational functions
- Exponential and logarithmic functions and their applications
- Trigonometry in radians, identities and equations
- Permutations, combinations and the binomial theorem
- Combining functions
- Full written solutions and exam review technique

HOW A TYPICAL WEEK IS SPLIT

Live classes: 2 live one-hour classes per week, paced to the student's school course outline
Practice: Short practice sets after every class, separate from school assignments
Review: Practice work reviewed with written comments, and a monthly check that mixes in earlier topics

SUPPORT INCLUDED

Doubt support: WhatsApp doubt support between classes, with hints and questions first so the student does the thinking
Progress updates: A monthly progress note to parents with what was covered, what is strong and what to practise next

CERTIFICATION DETAIL

Completion: Certificate from Modern Age Coders, awarded on passing the final assessment. It records real learning and is not a high school credit. Fall short and there is focused revision and a free retest

WHERE THIS CAN LEAD

- Calculus at school or in first-year university, on a solid base
- University science, engineering, computer science or mathematics programs
- Math contests for senior students, through our Waterloo math contests course
- Further study in data science and programming, where functions and logarithms are used every day
- Our other high school math courses for the next step

COMMON QUESTIONS ABOUT THIS COURSE

For anything about cost, use PART 4 and RULE 1. Do not quote two regions.

Q: Does this course give my child a high school credit in BC or Alberta?
A: No. Modern Age Coders teaches alongside school and does not grant high school credits in British Columbia, Alberta or any other province. A family who needs the Pre-calculus 12 or Math 30-1 credit takes the course through the student's school or an approved provider. Our course helps the student understand and do well in the school course they are already taking.

Q: Which courses is this for?
A: Pre-calculus 12 in British Columbia, which is on BC's mathematics course list, and Mathematics 30-1 in Alberta, which is on Alberta's list of diploma exam courses. Both are Grade 12 courses heavy in functions, so we teach them together while following each student's own school course outline.

Q: What topics does the course teach?
A: Functions and transformations, including inverses; polynomial, radical and rational functions; exponential and logarithmic functions; trigonometry in radians, with identities and equations; and permutations, combinations and the binomial theorem, plus combining functions. Your school's course outline gives the exact list and order for your class, and we follow it.

Q: How much is the Math 30-1 diploma exam worth?
A: Alberta's diploma exams overview says "The weighting of diploma exams returned to 30% effective September 1, 2023." and "70% of the final mark comes from course work. The remaining 30% comes from the diploma exam." Mathematics 30-1 is one of the diploma exam courses on that page.

Q: Do you prepare Alberta students for the Math 30-1 diploma exam?
A: Yes, alongside their school course. We teach the mathematics, then exam technique: pacing, checking and timed practice papers with careful review. Alberta says "students write diploma exams and provincial achievement tests in a digital environment rather than pencil and paper", so some practice is done on screen. Exam schedules can change, so we read Alberta's current diploma exam information with each student. We make no promise of marks or results.

Q: What comes after Pre-calculus 12?
A: BC's mathematics course list also includes Calculus 12, and many university programs in science, engineering and computer science build on pre-calculus. The functions, logarithms and trigonometry in this course are exactly what calculus uses. Always check each university's current admission page for its own requirements.

Q: Does the course follow my school's order?
A: Yes. Schools order the units differently, so we use the student's own course outline and next test date to set the pace. The plan on this page shows our standard sequence, and each batch follows the order of its students' schools as closely as possible.

Q: Will you do my child's assignments or tests?
A: No. School work stays the student's own. We explain concepts, ask questions, review work the student has written and suggest how to improve it, and we follow the school's rules on outside help. That is also the only kind of help that lasts into the exam room.

Q: Can my child join partway through the school year?
A: Yes, the course is joinable any month. We start from the student's current unit and next test, fill gaps from earlier units as needed, and keep pace with the school course from then on.

Q: How are the classes run?
A: Live online classes, twice a week, with a real teacher, in a group of 15 to 20 students, a mini batch of up to 6, or 1-on-1. Weekdays or weekends, flexible: choose your batch, or for a very personalised plan choose 1-on-1 at your convenience, weekend mornings or weekdays. Current fees for your region are shown in the pricing section of this page.

Q: Can we try a class first?
A: Yes. Book a free demo class. The teacher looks at where your child is in Pre-calculus 12 or Math 30-1, works through a short problem with them, and suggests whether group, mini batch or 1-on-1 classes fit best. There is no obligation to enrol. (Agent note: the free demo is a waiting list with no fixed date. The Priority Live Demo is a guaranteed class today or tomorrow, booked at https://learn.modernagecoders.com/priority-demo . Offer both, as RULE 3 says.)

To enrol, message Shivam Sir on +91 9123366161 or send the parent this link: https://learn.modernagecoders.com/courses/western-

canada-grade-12-math-pre-calculus-12-math-30-1-course
  For a demo: the Priority Live Demo (guaranteed, today or tomorrow) is booked at
  https://learn.modernagecoders.com/priority-demo . The free demo is a waiting list with no fixed date.

==
PART 7. HOW SOMEONE JOINS
==

THE STEPS, IN ORDER

  1. First chat. Find out the age, the subject, the current level and the goal.
  2. Demo class. Priority Live Demo (paid, guaranteed, today or tomorrow) or the free
     waiting list. A real teacher either way. Parent may sit in.
  3. Pick a plan. Group, mini batch or one to one.
  4. Shivam Sir matches the student to a batch that fits their timetable.
  5. Pay the first month through a secure link.
  6. The student gets dashboard access, the class link and the recordings.
  7. Classes begin.
  8. Progress updates go to the parent every month.

WHAT TO COLLECT BEFORE HANDING A LEAD OVER

  - Student full name.
  - Age, and school class or grade.
  - School and board if it is a school subject (CBSE, ICSE, IGCSE, IB, state board).
  - Parent name.
  - Parent WhatsApp number, with the country code.
  - City and country, so we know the time zone.
  - Subject wanted: coding, maths, or both.
  - The goal: school marks, board exam, hobby, Olympiad, college prep, a job.
  - Roughly when they are free: weekdays or weekends, morning or evening.
  - Preferred language: English or Hindi.

WHAT THE PRIORITY LIVE DEMO ACTUALLY IS

  - A paid, guaranteed demo class. The family picks the day and time, usually today or tomorrow.
  - A mentor is reserved only for that family, and we confirm on WhatsApp within the hour.
  - A full live class of about 45 to 60 minutes with a real teacher. Not a sales call.
  - Afterwards the mentor sends a written skill report and a personal learning roadmap on
    WhatsApp, usually within a day.
  - If the family enrols, the demo fee is adjusted against the first month. They send the demo
    Order ID on WhatsApp and our team applies it. It does not show as a discount on the payment
    page.
  - One-time fee: 499 rupees in India, 10 US dollars outside India. Quote only the person's own
    region.
  - Booking it does not commit the family to any course.

WHAT THE FREE DEMO ACTUALLY IS

  - A waiting list. Mentors teach live all day, so a free demo happens only when one becomes
    free.
  - The wait can be several weeks and we cannot promise a date, or that one will open at all.
  - When it happens it is a live class with a real teacher, not a sales call and not a recording.
  - Free. No card details are ever asked for.
  - The parent is welcome to watch.
  - It doubles as a placement check, so the teacher recommends the right starting point.
  - One free demo per student.
  - If the child does not enjoy it, the family walks away owing nothing.

If a parent wants to judge us before booking, offer this: they can watch recordings of real,
unedited classes first.

==
PART 8. REFUND AND CANCELLATION. READ THIS OUT EXACTLY, DO NOT REWORD IT
==

REFUND POLICY

  Modern Age Coders offers a 7 day money back guarantee on all online courses, provided you
  have not completed more than 20 percent of the course content.

  To be eligible:
    - The request must come within 7 days of the original purchase date.
    - Refunds apply to the first purchase only, not to renewals or repeat subscriptions.

  Not refundable:
    - Courses where more than 20 percent of the content has been viewed or downloaded.
    - One to one mentorship sessions that have already been taught.
    - Customised bootcamps or special arrangements.

    - Payment gateway charges: 2.5 percent of the fee, added at checkout and shown on the bill
      before paying. A refund covers the fee itself, so someone who paid 100 plus 2.5 in gateway
      charges gets 100 back.

  Once a refund is approved it goes back to the original payment method. The credit appears
  within 7 to 10 business days, depending on the card issuer.

  To request one, email connect@modernagecoders.com with the order details and the reason.

CANCELLATION POLICY

  - Enrolment can be cancelled at any time. A cancellation after a live batch has started, or
    after materials have been accessed, falls under the refund policy above.
  - A recurring subscription can be cancelled any time from the account dashboard. It takes
    effect at the end of the current billing cycle and there is no further charge.
  - We may cancel or reschedule a course if too few students enrol or something unforeseen
    happens. Those students get a full refund or a transfer to another schedule.
  - After cancelling, access continues until the end of the billing period already paid for.
  - To cancel: use account settings, or email connect@modernagecoders.com.

IF SOMEONE ASKS "WHAT IF WE ARE NOT HAPPY?"

  Say: "There is a 7 day money back guarantee. If in the first week you have used less than 20
  percent of the course and it is not right for you, we refund the fee in full (the 2.5 percent
  payment gateway charges added at checkout are not refunded). And before you pay for a month
  you can see a full live class, either through the Priority Live Demo, whose fee is adjusted
  against your first month if you join, or through the free demo waiting list."

==
PART 9. OTHER PAGES YOU CAN SEND A PARENT
==

THE PAGES PARENTS ASK FOR MOST

  All courses   https://learn.modernagecoders.com/courses
  Prices   https://learn.modernagecoders.com/pricing
  Priority Live Demo, guaranteed today or tomorrow https://learn.modernagecoders.com/priority-demo
  Book a demo   https://learn.modernagecoders.com/book-demo
  Free demo or priority demo, compared   https://learn.modernagecoders.com/free-demo-vs-priority-demo
  What happens in a demo class   https://learn.modernagecoders.com/what-happens-in-a-demo-class
  Small batches and one to one classes   https://learn.modernagecoders.com/small-batches-and-1-on-1-classes
  Free trial class   https://learn.modernagecoders.com/free-trial
  Our zero risk promise   https://learn.modernagecoders.com/guarantee
  How we teach   https://learn.modernagecoders.com/how-we-teach
  About us   https://learn.modernagecoders.com/about
  Contact us   https://learn.modernagecoders.com/contact
  Reviews from families   https://learn.modernagecoders.com/love
  Student success stories   https://learn.modernagecoders.com/success-stories
  Real student projects   https://learn.modernagecoders.com/student-labs
  Course atlas, every course on one page   https://learn.modernagecoders.com/course-atlas
  What to learn and in what order   https://learn.modernagecoders.com/coding-roadmap
  Work out your monthly fee   https://learn.modernagecoders.com/coding-fee-calculator
  Free resources   https://learn.modernagecoders.com/free-resources
  Blog   https://learn.modernagecoders.com/blog
  Schools programme   https://learn.modernagecoders.com/schools
  For businesses   https://learn.modernagecoders.com/for-business
  Refund policy   https://learn.modernagecoders.com/refund
  Cancellation policy   https://learn.modernagecoders.com/cancellation
  Which AI tools are safe at which age   https://learn.modernagecoders.com/ai-tools-age-guide

FREE THINGS TO PLAY WITH. Good for a parent who is unsure.

  Beat the Babu, 90 second maths game   https://learn.modernagecoders.com/beat-the-babu
  Aura Type, typing game   https://learn.modernagecoders.com/aura
  Daily coding challenges   https://learn.modernagecoders.com/coding-challenges
  Maths challenges   https://learn.modernagecoders.com/maths-challenges
  Mistakes students make   https://learn.modernagecoders.com/mistakes-students-make
  Misti AI, our study assistant   https://misti.modernagecoders.com
  The Beautiful Mind, our blog   https://mind.modernagecoders.com

  Student dashboard   https://dashboard.modernagecoders.com
  Mark attendance   https://dashboard.modernagecoders.com/attendance
  Class recordings   https://dashboard.modernagecoders.com/recording

CITY PAGES IN INDIA (262 pages)

  https://learn.modernagecoders.com/best-coding-class-in-aberdeen
  https://learn.modernagecoders.com/best-coding-class-in-agra
  https://learn.modernagecoders.com/best-coding-class-in-ahmedabad

https://learn.modernagecoders.com/best-coding-class-in-ajmer
https://learn.modernagecoders.com/best-coding-class-in-akola
https://learn.modernagecoders.com/best-coding-class-in-aligarh
https://learn.modernagecoders.com/best-coding-class-in-amravati
https://learn.modernagecoders.com/best-coding-class-in-amritsar
https://learn.modernagecoders.com/best-coding-class-in-anand
https://learn.modernagecoders.com/best-coding-class-in-arklow
https://learn.modernagecoders.com/best-coding-class-in-armagh
https://learn.modernagecoders.com/best-coding-class-in-asansol
https://learn.modernagecoders.com/best-coding-class-in-ashbourne
https://learn.modernagecoders.com/best-coding-class-in-athlone
https://learn.modernagecoders.com/best-coding-class-in-athy
https://learn.modernagecoders.com/best-coding-class-in-aurangabad
https://learn.modernagecoders.com/best-coding-class-in-balasore
https://learn.modernagecoders.com/best-coding-class-in-balbriggan
https://learn.modernagecoders.com/best-coding-class-in-ballina
https://learn.modernagecoders.com/best-coding-class-in-bangor-northern-ireland
https://learn.modernagecoders.com/best-coding-class-in-bangor-wales
https://learn.modernagecoders.com/best-coding-class-in-bareilly
https://learn.modernagecoders.com/best-coding-class-in-bath
https://learn.modernagecoders.com/best-coding-class-in-belagavi
https://learn.modernagecoders.com/best-coding-class-in-belfast
https://learn.modernagecoders.com/best-coding-class-in-bengaluru
https://learn.modernagecoders.com/best-coding-class-in-berhampur
https://learn.modernagecoders.com/best-coding-class-in-bhagalpur
https://learn.modernagecoders.com/best-coding-class-in-bhavnagar
https://learn.modernagecoders.com/best-coding-class-in-bhilai
https://learn.modernagecoders.com/best-coding-class-in-bhopal
https://learn.modernagecoders.com/best-coding-class-in-bhubaneswar
https://learn.modernagecoders.com/best-coding-class-in-bikaner
https://learn.modernagecoders.com/best-coding-class-in-bokaro
https://learn.modernagecoders.com/best-coding-class-in-bradford
https://learn.modernagecoders.com/best-coding-class-in-bray
https://learn.modernagecoders.com/best-coding-class-in-brighton-and-hove
https://learn.modernagecoders.com/best-coding-class-in-bristol
https://learn.modernagecoders.com/best-coding-class-in-cambridge
https://learn.modernagecoders.com/best-coding-class-in-canterbury
https://learn.modernagecoders.com/best-coding-class-in-cardiff
https://learn.modernagecoders.com/best-coding-class-in-carlisle
https://learn.modernagecoders.com/best-coding-class-in-carlow
https://learn.modernagecoders.com/best-coding-class-in-carrigaline
https://learn.modernagecoders.com/best-coding-class-in-castlebar
https://learn.modernagecoders.com/best-coding-class-in-cavan
https://learn.modernagecoders.com/best-coding-class-in-celbridge
https://learn.modernagecoders.com/best-coding-class-in-chandigarh
https://learn.modernagecoders.com/best-coding-class-in-chelmsford
https://learn.modernagecoders.com/best-coding-class-in-chennai
https://learn.modernagecoders.com/best-coding-class-in-chester
https://learn.modernagecoders.com/best-coding-class-in-chichester
https://learn.modernagecoders.com/best-coding-class-in-clonmel
https://learn.modernagecoders.com/best-coding-class-in-cobh
https://learn.modernagecoders.com/best-coding-class-in-coimbatore
https://learn.modernagecoders.com/best-coding-class-in-colchester
https://learn.modernagecoders.com/best-coding-class-in-cork
https://learn.modernagecoders.com/best-coding-class-in-coventry
https://learn.modernagecoders.com/best-coding-class-in-cuttack
https://learn.modernagecoders.com/best-coding-class-in-darbhanga
https://learn.modernagecoders.com/best-coding-class-in-davanagere
https://learn.modernagecoders.com/best-coding-class-in-dehradun
https://learn.modernagecoders.com/best-coding-class-in-delhi
https://learn.modernagecoders.com/best-coding-class-in-derby
https://learn.modernagecoders.com/best-coding-class-in-derry-londonderry
https://learn.modernagecoders.com/best-coding-class-in-dhanbad
https://learn.modernagecoders.com/best-coding-class-in-doncaster
https://learn.modernagecoders.com/best-coding-class-in-drogheda
https://learn.modernagecoders.com/best-coding-class-in-dublin
https://learn.modernagecoders.com/best-coding-class-in-dundalk
https://learn.modernagecoders.com/best-coding-class-in-dundee
https://learn.modernagecoders.com/best-coding-class-in-dunfermline
https://learn.modernagecoders.com/best-coding-class-in-dungarvan
https://learn.modernagecoders.com/best-coding-class-in-durgapur
https://learn.modernagecoders.com/best-coding-class-in-durham
https://learn.modernagecoders.com/best-coding-class-in-edinburgh
https://learn.modernagecoders.com/best-coding-class-in-ely
https://learn.modernagecoders.com/best-coding-class-in-ennis
https://learn.modernagecoders.com/best-coding-class-in-enniscorthy
https://learn.modernagecoders.com/best-coding-class-in-erode
https://learn.modernagecoders.com/best-coding-class-in-exeter
https://learn.modernagecoders.com/best-coding-class-in-faridabad

https://learn.modernagecoders.com/best-coding-class-in-galway
https://learn.modernagecoders.com/best-coding-class-in-gandhinagar
https://learn.modernagecoders.com/best-coding-class-in-gangtok
https://learn.modernagecoders.com/best-coding-class-in-gaya
https://learn.modernagecoders.com/best-coding-class-in-ghaziabad
https://learn.modernagecoders.com/best-coding-class-in-glasgow
https://learn.modernagecoders.com/best-coding-class-in-gloucester
https://learn.modernagecoders.com/best-coding-class-in-goa
https://learn.modernagecoders.com/best-coding-class-in-gorakhpur
https://learn.modernagecoders.com/best-coding-class-in-gorey
https://learn.modernagecoders.com/best-coding-class-in-greater-noida
https://learn.modernagecoders.com/best-coding-class-in-greystones
https://learn.modernagecoders.com/best-coding-class-in-guntur
https://learn.modernagecoders.com/best-coding-class-in-gurgaon
https://learn.modernagecoders.com/best-coding-class-in-guwahati
https://learn.modernagecoders.com/best-coding-class-in-gwalior
https://learn.modernagecoders.com/best-coding-class-in-haldwani
https://learn.modernagecoders.com/best-coding-class-in-haridwar
https://learn.modernagecoders.com/best-coding-class-in-hereford
https://learn.modernagecoders.com/best-coding-class-in-hisar
https://learn.modernagecoders.com/best-coding-class-in-hubli
https://learn.modernagecoders.com/best-coding-class-in-hull
https://learn.modernagecoders.com/best-coding-class-in-hyderabad
https://learn.modernagecoders.com/best-coding-class-in-imphal
https://learn.modernagecoders.com/best-coding-class-in-indore
https://learn.modernagecoders.com/best-coding-class-in-inverness
https://learn.modernagecoders.com/best-coding-class-in-jabalpur
https://learn.modernagecoders.com/best-coding-class-in-jaipur
https://learn.modernagecoders.com/best-coding-class-in-jalandhar
https://learn.modernagecoders.com/best-coding-class-in-jammu
https://learn.modernagecoders.com/best-coding-class-in-jamnagar
https://learn.modernagecoders.com/best-coding-class-in-jamshedpur
https://learn.modernagecoders.com/best-coding-class-in-jhansi
https://learn.modernagecoders.com/best-coding-class-in-jodhpur
https://learn.modernagecoders.com/best-coding-class-in-kakinada
https://learn.modernagecoders.com/best-coding-class-in-kannur
https://learn.modernagecoders.com/best-coding-class-in-kanpur
https://learn.modernagecoders.com/best-coding-class-in-karimnagar
https://learn.modernagecoders.com/best-coding-class-in-karnal
https://learn.modernagecoders.com/best-coding-class-in-kharagpur
https://learn.modernagecoders.com/best-coding-class-in-kildare
https://learn.modernagecoders.com/best-coding-class-in-kilkenny
https://learn.modernagecoders.com/best-coding-class-in-killarney
https://learn.modernagecoders.com/best-coding-class-in-kochi
https://learn.modernagecoders.com/best-coding-class-in-kolhapur
https://learn.modernagecoders.com/best-coding-class-in-kolkata
https://learn.modernagecoders.com/best-coding-class-in-kollam
https://learn.modernagecoders.com/best-coding-class-in-kota
https://learn.modernagecoders.com/best-coding-class-in-kottayam
https://learn.modernagecoders.com/best-coding-class-in-kozhikode
https://learn.modernagecoders.com/best-coding-class-in-lancaster
https://learn.modernagecoders.com/best-coding-class-in-latur
https://learn.modernagecoders.com/best-coding-class-in-laytown-bettystown
https://learn.modernagecoders.com/best-coding-class-in-leeds
https://learn.modernagecoders.com/best-coding-class-in-leixlip
https://learn.modernagecoders.com/best-coding-class-in-letterkenny
https://learn.modernagecoders.com/best-coding-class-in-lichfield
https://learn.modernagecoders.com/best-coding-class-in-limerick
https://learn.modernagecoders.com/best-coding-class-in-lincoln
https://learn.modernagecoders.com/best-coding-class-in-lisburn
https://learn.modernagecoders.com/best-coding-class-in-liverpool
https://learn.modernagecoders.com/best-coding-class-in-london
https://learn.modernagecoders.com/best-coding-class-in-longford
https://learn.modernagecoders.com/best-coding-class-in-lucknow
https://learn.modernagecoders.com/best-coding-class-in-ludhiana
https://learn.modernagecoders.com/best-coding-class-in-luton
https://learn.modernagecoders.com/best-coding-class-in-madurai
https://learn.modernagecoders.com/best-coding-class-in-malahide
https://learn.modernagecoders.com/best-coding-class-in-mallow
https://learn.modernagecoders.com/best-coding-class-in-manchester
https://learn.modernagecoders.com/best-coding-class-in-mangalore
https://learn.modernagecoders.com/best-coding-class-in-maynooth
https://learn.modernagecoders.com/best-coding-class-in-meerut
https://learn.modernagecoders.com/best-coding-class-in-midleton
https://learn.modernagecoders.com/best-coding-class-in-milton-keynes
https://learn.modernagecoders.com/best-coding-class-in-mohali
https://learn.modernagecoders.com/best-coding-class-in-moradabad
https://learn.modernagecoders.com/best-coding-class-in-mullingar
https://learn.modernagecoders.com/best-coding-class-in-mumbai

https://learn.modernagecoders.com/best-coding-class-in-muzaffarpur
https://learn.modernagecoders.com/best-coding-class-in-mysore
https://learn.modernagecoders.com/best-coding-class-in-naas
https://learn.modernagecoders.com/best-coding-class-in-nagpur
https://learn.modernagecoders.com/best-coding-class-in-nashik
https://learn.modernagecoders.com/best-coding-class-in-navan
https://learn.modernagecoders.com/best-coding-class-in-navi-mumbai
https://learn.modernagecoders.com/best-coding-class-in-nellore
https://learn.modernagecoders.com/best-coding-class-in-newbridge
https://learn.modernagecoders.com/best-coding-class-in-newcastle-upon-tyne
https://learn.modernagecoders.com/best-coding-class-in-newport-wales
https://learn.modernagecoders.com/best-coding-class-in-newry
https://learn.modernagecoders.com/best-coding-class-in-nizamabad
https://learn.modernagecoders.com/best-coding-class-in-noida
https://learn.modernagecoders.com/best-coding-class-in-norwich
https://learn.modernagecoders.com/best-coding-class-in-nottingham
https://learn.modernagecoders.com/best-coding-class-in-oxford
https://learn.modernagecoders.com/best-coding-class-in-palakkad
https://learn.modernagecoders.com/best-coding-class-in-panipat
https://learn.modernagecoders.com/best-coding-class-in-patiala
https://learn.modernagecoders.com/best-coding-class-in-patna
https://learn.modernagecoders.com/best-coding-class-in-perth-scotland
https://learn.modernagecoders.com/best-coding-class-in-peterborough
https://learn.modernagecoders.com/best-coding-class-in-pimpri-chinchwad
https://learn.modernagecoders.com/best-coding-class-in-plymouth
https://learn.modernagecoders.com/best-coding-class-in-portlaoise
https://learn.modernagecoders.com/best-coding-class-in-portmarnock
https://learn.modernagecoders.com/best-coding-class-in-portsmouth
https://learn.modernagecoders.com/best-coding-class-in-prayagraj
https://learn.modernagecoders.com/best-coding-class-in-preston
https://learn.modernagecoders.com/best-coding-class-in-pune
https://learn.modernagecoders.com/best-coding-class-in-raipur
https://learn.modernagecoders.com/best-coding-class-in-rajahmundry
https://learn.modernagecoders.com/best-coding-class-in-rajkot
https://learn.modernagecoders.com/best-coding-class-in-ranchi
https://learn.modernagecoders.com/best-coding-class-in-ratoath
https://learn.modernagecoders.com/best-coding-class-in-reading
https://learn.modernagecoders.com/best-coding-class-in-ripon
https://learn.modernagecoders.com/best-coding-class-in-rishikesh
https://learn.modernagecoders.com/best-coding-class-in-rohtak
https://learn.modernagecoders.com/best-coding-class-in-roorkee
https://learn.modernagecoders.com/best-coding-class-in-rourkela
https://learn.modernagecoders.com/best-coding-class-in-rush
https://learn.modernagecoders.com/best-coding-class-in-salem
https://learn.modernagecoders.com/best-coding-class-in-salford
https://learn.modernagecoders.com/best-coding-class-in-salisbury
https://learn.modernagecoders.com/best-coding-class-in-sambalpur
https://learn.modernagecoders.com/best-coding-class-in-shannon
https://learn.modernagecoders.com/best-coding-class-in-sheffield
https://learn.modernagecoders.com/best-coding-class-in-shimla
https://learn.modernagecoders.com/best-coding-class-in-shimoga
https://learn.modernagecoders.com/best-coding-class-in-sikar
https://learn.modernagecoders.com/best-coding-class-in-siliguri
https://learn.modernagecoders.com/best-coding-class-in-skerries
https://learn.modernagecoders.com/best-coding-class-in-sligo
https://learn.modernagecoders.com/best-coding-class-in-slough
https://learn.modernagecoders.com/best-coding-class-in-solapur
https://learn.modernagecoders.com/best-coding-class-in-sonipat
https://learn.modernagecoders.com/best-coding-class-in-southampton
https://learn.modernagecoders.com/best-coding-class-in-southend-on-sea
https://learn.modernagecoders.com/best-coding-class-in-srinagar
https://learn.modernagecoders.com/best-coding-class-in-st-albans
https://learn.modernagecoders.com/best-coding-class-in-st-asaph
https://learn.modernagecoders.com/best-coding-class-in-st-davids
https://learn.modernagecoders.com/best-coding-class-in-stirling
https://learn.modernagecoders.com/best-coding-class-in-stoke-on-trent
https://learn.modernagecoders.com/best-coding-class-in-sunderland
https://learn.modernagecoders.com/best-coding-class-in-surat
https://learn.modernagecoders.com/best-coding-class-in-swansea
https://learn.modernagecoders.com/best-coding-class-in-swords
https://learn.modernagecoders.com/best-coding-class-in-thane
https://learn.modernagecoders.com/best-coding-class-in-thanjavur
https://learn.modernagecoders.com/best-coding-class-in-the-world
https://learn.modernagecoders.com/best-coding-class-in-thiruvananthapuram
https://learn.modernagecoders.com/best-coding-class-in-thrissur
https://learn.modernagecoders.com/best-coding-class-in-tirunelveli
https://learn.modernagecoders.com/best-coding-class-in-tirupati
https://learn.modernagecoders.com/best-coding-class-in-tralee
https://learn.modernagecoders.com/best-coding-class-in-tramore

https://learn.modernagecoders.com/best-coding-class-in-trichy
https://learn.modernagecoders.com/best-coding-class-in-truro
https://learn.modernagecoders.com/best-coding-class-in-tullamore
https://learn.modernagecoders.com/best-coding-class-in-udaipur
https://learn.modernagecoders.com/best-coding-class-in-udupi
https://learn.modernagecoders.com/best-coding-class-in-ujjain
https://learn.modernagecoders.com/best-coding-class-in-vadodara
https://learn.modernagecoders.com/best-coding-class-in-varanasi
https://learn.modernagecoders.com/best-coding-class-in-vellore
https://learn.modernagecoders.com/best-coding-class-in-vijayawada
https://learn.modernagecoders.com/best-coding-class-in-visakhapatnam
https://learn.modernagecoders.com/best-coding-class-in-wakefield
https://learn.modernagecoders.com/best-coding-class-in-warangal
https://learn.modernagecoders.com/best-coding-class-in-waterford
https://learn.modernagecoders.com/best-coding-class-in-wells
https://learn.modernagecoders.com/best-coding-class-in-wexford
https://learn.modernagecoders.com/best-coding-class-in-wicklow
https://learn.modernagecoders.com/best-coding-class-in-winchester
https://learn.modernagecoders.com/best-coding-class-in-wolverhampton
https://learn.modernagecoders.com/best-coding-class-in-worcester
https://learn.modernagecoders.com/best-coding-class-in-wrexham
https://learn.modernagecoders.com/best-coding-class-in-york

AREAS IN AND AROUND KOLKATA (48 pages)

https://learn.modernagecoders.com/coding-classes-near-ajc-bose-road-kolkata
https://learn.modernagecoders.com/coding-classes-near-alipore-kolkata
https://learn.modernagecoders.com/coding-classes-near-andul-howrah
https://learn.modernagecoders.com/coding-classes-near-baguiati-kolkata
https://learn.modernagecoders.com/coding-classes-near-bally-howrah
https://learn.modernagecoders.com/coding-classes-near-ballygunge-kolkata
https://learn.modernagecoders.com/coding-classes-near-baranagar-kolkata
https://learn.modernagecoders.com/coding-classes-near-bhowanipore-kolkata
https://learn.modernagecoders.com/coding-classes-near-camac-street-kolkata
https://learn.modernagecoders.com/coding-classes-near-deshapriya-park-kolkata
https://learn.modernagecoders.com/coding-classes-near-dhakuria-kolkata
https://learn.modernagecoders.com/coding-classes-near-domjur-howrah
https://learn.modernagecoders.com/coding-classes-near-dum-dum-kolkata
https://learn.modernagecoders.com/coding-classes-near-elgin-road-kolkata
https://learn.modernagecoders.com/coding-classes-near-gariahat-kolkata
https://learn.modernagecoders.com/coding-classes-near-golpark-kolkata
https://learn.modernagecoders.com/coding-classes-near-hazra-kolkata
https://learn.modernagecoders.com/coding-classes-near-howrah-city-howrah
https://learn.modernagecoders.com/coding-classes-near-jadavpur-kolkata
https://learn.modernagecoders.com/coding-classes-near-jodhpur-park-kolkata
https://learn.modernagecoders.com/coding-classes-near-kadamtala-howrah
https://learn.modernagecoders.com/coding-classes-near-kalighat-kolkata
https://learn.modernagecoders.com/coding-classes-near-kasba-kolkata
https://learn.modernagecoders.com/coding-classes-near-kestopur-kolkata
https://learn.modernagecoders.com/coding-classes-near-lake-gardens-kolkata
https://learn.modernagecoders.com/coding-classes-near-lake-town-kolkata
https://learn.modernagecoders.com/coding-classes-near-lansdowne-kolkata
https://learn.modernagecoders.com/coding-classes-near-liluah-howrah
https://learn.modernagecoders.com/coding-classes-near-loudon-street-kolkata
https://learn.modernagecoders.com/coding-classes-near-mandirtala-howrah
https://learn.modernagecoders.com/coding-classes-near-me-australia-online-or-local
https://learn.modernagecoders.com/coding-classes-near-nager-bazar-kolkata
https://learn.modernagecoders.com/coding-classes-near-new-alipore-kolkata
https://learn.modernagecoders.com/coding-classes-near-new-town-kolkata
https://learn.modernagecoders.com/coding-classes-near-park-circus-kolkata
https://learn.modernagecoders.com/coding-classes-near-park-street-kolkata
https://learn.modernagecoders.com/coding-classes-near-ramrajatala-howrah
https://learn.modernagecoders.com/coding-classes-near-rashbehari-kolkata
https://learn.modernagecoders.com/coding-classes-near-salt-lake-kolkata
https://learn.modernagecoders.com/coding-classes-near-santragachi-howrah
https://learn.modernagecoders.com/coding-classes-near-shakespeare-sarani-kolkata
https://learn.modernagecoders.com/coding-classes-near-shibpur-howrah
https://learn.modernagecoders.com/coding-classes-near-shyambazar-kolkata
https://learn.modernagecoders.com/coding-classes-near-southern-avenue-kolkata
https://learn.modernagecoders.com/coding-classes-near-sreebhumi-kolkata
https://learn.modernagecoders.com/coding-classes-near-tollygunge-kolkata
https://learn.modernagecoders.com/coding-classes-near-uluberia-howrah
https://learn.modernagecoders.com/coding-classes-near-vip-road-kolkata

COUNTRY AND REGION PAGES (461 pages)

https://learn.modernagecoders.com/coding-classes-in-aberdeenshire
https://learn.modernagecoders.com/coding-classes-in-abu-dhabi
https://learn.modernagecoders.com/coding-classes-in-ad-dakhiliyah

https://learn.modernagecoders.com/coding-classes-in-adam
https://learn.modernagecoders.com/coding-classes-in-adh-dhahirah
https://learn.modernagecoders.com/coding-classes-in-al-amarat
https://learn.modernagecoders.com/coding-classes-in-al-awabi
https://learn.modernagecoders.com/coding-classes-in-al-batinah-north
https://learn.modernagecoders.com/coding-classes-in-al-batinah-south
https://learn.modernagecoders.com/coding-classes-in-al-buraimi
https://learn.modernagecoders.com/coding-classes-in-al-ghubrah
https://learn.modernagecoders.com/coding-classes-in-al-hail
https://learn.modernagecoders.com/coding-classes-in-al-hamra
https://learn.modernagecoders.com/coding-classes-in-al-kamil-wal-wafi
https://learn.modernagecoders.com/coding-classes-in-al-khaburah
https://learn.modernagecoders.com/coding-classes-in-al-khoudh
https://learn.modernagecoders.com/coding-classes-in-al-khuwair
https://learn.modernagecoders.com/coding-classes-in-al-mazyunah
https://learn.modernagecoders.com/coding-classes-in-al-mouj
https://learn.modernagecoders.com/coding-classes-in-al-mudhaibi
https://learn.modernagecoders.com/coding-classes-in-al-musanaah
https://learn.modernagecoders.com/coding-classes-in-al-qabil
https://learn.modernagecoders.com/coding-classes-in-al-wusta
https://learn.modernagecoders.com/coding-classes-in-alberta
https://learn.modernagecoders.com/coding-classes-in-alkmaar
https://learn.modernagecoders.com/coding-classes-in-almelo
https://learn.modernagecoders.com/coding-classes-in-almere
https://learn.modernagecoders.com/coding-classes-in-alphen-aan-den-rijn
https://learn.modernagecoders.com/coding-classes-in-amersfoort
https://learn.modernagecoders.com/coding-classes-in-amstelveen
https://learn.modernagecoders.com/coding-classes-in-amsterdam
https://learn.modernagecoders.com/coding-classes-in-amsterdam-centrum
https://learn.modernagecoders.com/coding-classes-in-amsterdam-nieuw-west
https://learn.modernagecoders.com/coding-classes-in-amsterdam-noord
https://learn.modernagecoders.com/coding-classes-in-amsterdam-oost
https://learn.modernagecoders.com/coding-classes-in-amsterdam-west
https://learn.modernagecoders.com/coding-classes-in-amsterdam-zuid
https://learn.modernagecoders.com/coding-classes-in-amsterdam-zuidoost
https://learn.modernagecoders.com/coding-classes-in-andhra-pradesh
https://learn.modernagecoders.com/coding-classes-in-anglesey
https://learn.modernagecoders.com/coding-classes-in-angus
https://learn.modernagecoders.com/coding-classes-in-antrim-and-newtownabbey
https://learn.modernagecoders.com/coding-classes-in-apeldoorn
https://learn.modernagecoders.com/coding-classes-in-argyll-and-bute
https://learn.modernagecoders.com/coding-classes-in-arnhem
https://learn.modernagecoders.com/coding-classes-in-as-suwaiq
https://learn.modernagecoders.com/coding-classes-in-ash-sharqiyah-north
https://learn.modernagecoders.com/coding-classes-in-ash-sharqiyah-south
https://learn.modernagecoders.com/coding-classes-in-assam
https://learn.modernagecoders.com/coding-classes-in-assen
https://learn.modernagecoders.com/coding-classes-in-australia
https://learn.modernagecoders.com/coding-classes-in-avani-riverside-howrah
https://learn.modernagecoders.com/coding-classes-in-avidipta-mukundapur
https://learn.modernagecoders.com/coding-classes-in-azaiba
https://learn.modernagecoders.com/coding-classes-in-bahla
https://learn.modernagecoders.com/coding-classes-in-bahrain
https://learn.modernagecoders.com/coding-classes-in-ballincollig-cork
https://learn.modernagecoders.com/coding-classes-in-ballymun-dublin
https://learn.modernagecoders.com/coding-classes-in-bandar-seri-begawan
https://learn.modernagecoders.com/coding-classes-in-barka
https://learn.modernagecoders.com/coding-classes-in-barking-and-dagenham-london
https://learn.modernagecoders.com/coding-classes-in-barnet-london
https://learn.modernagecoders.com/coding-classes-in-bawshar
https://learn.modernagecoders.com/coding-classes-in-bedfordshire
https://learn.modernagecoders.com/coding-classes-in-bergen-op-zoom
https://learn.modernagecoders.com/coding-classes-in-berkshire
https://learn.modernagecoders.com/coding-classes-in-bexley-london
https://learn.modernagecoders.com/coding-classes-in-bidbid
https://learn.modernagecoders.com/coding-classes-in-bidiyah
https://learn.modernagecoders.com/coding-classes-in-bihar
https://learn.modernagecoders.com/coding-classes-in-birmingham
https://learn.modernagecoders.com/coding-classes-in-bishopstown-cork
https://learn.modernagecoders.com/coding-classes-in-blackrock-dublin
https://learn.modernagecoders.com/coding-classes-in-blaenau-gwent
https://learn.modernagecoders.com/coding-classes-in-blanchardstown-dublin
https://learn.modernagecoders.com/coding-classes-in-breda
https://learn.modernagecoders.com/coding-classes-in-brent-london
https://learn.modernagecoders.com/coding-classes-in-bridgend
https://learn.modernagecoders.com/coding-classes-in-british-columbia
https://learn.modernagecoders.com/coding-classes-in-bromley-london
https://learn.modernagecoders.com/coding-classes-in-brunei
https://learn.modernagecoders.com/coding-classes-in-buckinghamshire

https://learn.modernagecoders.com/coding-classes-in-caerphilly-county-borough
https://learn.modernagecoders.com/coding-classes-in-calcutta-riverside
https://learn.modernagecoders.com/coding-classes-in-california
https://learn.modernagecoders.com/coding-classes-in-cambridgeshire
https://learn.modernagecoders.com/coding-classes-in-camden-london
https://learn.modernagecoders.com/coding-classes-in-canada
https://learn.modernagecoders.com/coding-classes-in-capelle-aan-den-ijssel
https://learn.modernagecoders.com/coding-classes-in-carmarthenshire
https://learn.modernagecoders.com/coding-classes-in-castleknock-dublin
https://learn.modernagecoders.com/coding-classes-in-castletroy-limerick
https://learn.modernagecoders.com/coding-classes-in-causeway-coast-and-glens
https://learn.modernagecoders.com/coding-classes-in-ceredigion
https://learn.modernagecoders.com/coding-classes-in-charlois
https://learn.modernagecoders.com/coding-classes-in-cheshire
https://learn.modernagecoders.com/coding-classes-in-chhattisgarh
https://learn.modernagecoders.com/coding-classes-in-clackmannanshire
https://learn.modernagecoders.com/coding-classes-in-clondalkin-dublin
https://learn.modernagecoders.com/coding-classes-in-clontarf-dublin
https://learn.modernagecoders.com/coding-classes-in-conwy
https://learn.modernagecoders.com/coding-classes-in-cornwall
https://learn.modernagecoders.com/coding-classes-in-county-carlow
https://learn.modernagecoders.com/coding-classes-in-county-cavan
https://learn.modernagecoders.com/coding-classes-in-county-clare
https://learn.modernagecoders.com/coding-classes-in-county-cork
https://learn.modernagecoders.com/coding-classes-in-county-donegal
https://learn.modernagecoders.com/coding-classes-in-county-durham
https://learn.modernagecoders.com/coding-classes-in-county-galway
https://learn.modernagecoders.com/coding-classes-in-county-kerry
https://learn.modernagecoders.com/coding-classes-in-county-kildare
https://learn.modernagecoders.com/coding-classes-in-county-kilkenny
https://learn.modernagecoders.com/coding-classes-in-county-laois
https://learn.modernagecoders.com/coding-classes-in-county-leitrim
https://learn.modernagecoders.com/coding-classes-in-county-longford
https://learn.modernagecoders.com/coding-classes-in-county-louth
https://learn.modernagecoders.com/coding-classes-in-county-mayo
https://learn.modernagecoders.com/coding-classes-in-county-meath
https://learn.modernagecoders.com/coding-classes-in-county-monaghan
https://learn.modernagecoders.com/coding-classes-in-county-offaly
https://learn.modernagecoders.com/coding-classes-in-county-roscommon
https://learn.modernagecoders.com/coding-classes-in-county-sligo
https://learn.modernagecoders.com/coding-classes-in-county-tipperary
https://learn.modernagecoders.com/coding-classes-in-county-westmeath
https://learn.modernagecoders.com/coding-classes-in-county-wexford
https://learn.modernagecoders.com/coding-classes-in-county-wicklow
https://learn.modernagecoders.com/coding-classes-in-croydon-london
https://learn.modernagecoders.com/coding-classes-in-crumlin-dublin
https://learn.modernagecoders.com/coding-classes-in-cumbria
https://learn.modernagecoders.com/coding-classes-in-daba-al-bayah
https://learn.modernagecoders.com/coding-classes-in-dammam
https://learn.modernagecoders.com/coding-classes-in-delfshaven
https://learn.modernagecoders.com/coding-classes-in-delft
https://learn.modernagecoders.com/coding-classes-in-den-bosch
https://learn.modernagecoders.com/coding-classes-in-denbighshire
https://learn.modernagecoders.com/coding-classes-in-derbyshire
https://learn.modernagecoders.com/coding-classes-in-deventer
https://learn.modernagecoders.com/coding-classes-in-devon
https://learn.modernagecoders.com/coding-classes-in-dhank
https://learn.modernagecoders.com/coding-classes-in-dhofar
https://learn.modernagecoders.com/coding-classes-in-diamond-city-north
https://learn.modernagecoders.com/coding-classes-in-diamond-city-south
https://learn.modernagecoders.com/coding-classes-in-diemen
https://learn.modernagecoders.com/coding-classes-in-dima-wa-attayeen
https://learn.modernagecoders.com/coding-classes-in-dlf-new-town-heights
https://learn.modernagecoders.com/coding-classes-in-doha
https://learn.modernagecoders.com/coding-classes-in-dordrecht
https://learn.modernagecoders.com/coding-classes-in-dorset
https://learn.modernagecoders.com/coding-classes-in-douglas-cork
https://learn.modernagecoders.com/coding-classes-in-drenthe
https://learn.modernagecoders.com/coding-classes-in-drumcondra-dublin
https://learn.modernagecoders.com/coding-classes-in-dubai
https://learn.modernagecoders.com/coding-classes-in-dumfries-and-galloway
https://learn.modernagecoders.com/coding-classes-in-dun-laoghaire-dublin
https://learn.modernagecoders.com/coding-classes-in-dun-laoghaire-rathdown
https://learn.modernagecoders.com/coding-classes-in-dundrum-dublin
https://learn.modernagecoders.com/coding-classes-in-duqm
https://learn.modernagecoders.com/coding-classes-in-ealing-london
https://learn.modernagecoders.com/coding-classes-in-east-ayrshire
https://learn.modernagecoders.com/coding-classes-in-east-dunbartonshire
https://learn.modernagecoders.com/coding-classes-in-east-lothian

https://learn.modernagecoders.com/coding-classes-in-east-renfrewshire
https://learn.modernagecoders.com/coding-classes-in-east-riding-of-yorkshire
https://learn.modernagecoders.com/coding-classes-in-east-sussex
https://learn.modernagecoders.com/coding-classes-in-ede
https://learn.modernagecoders.com/coding-classes-in-eden-city-maheshtala
https://learn.modernagecoders.com/coding-classes-in-eindhoven
https://learn.modernagecoders.com/coding-classes-in-eindhoven-centrum
https://learn.modernagecoders.com/coding-classes-in-elita-garden-vista
https://learn.modernagecoders.com/coding-classes-in-emmen
https://learn.modernagecoders.com/coding-classes-in-enfield-london
https://learn.modernagecoders.com/coding-classes-in-enschede
https://learn.modernagecoders.com/coding-classes-in-escamp
https://learn.modernagecoders.com/coding-classes-in-essex
https://learn.modernagecoders.com/coding-classes-in-falkirk
https://learn.modernagecoders.com/coding-classes-in-feijenoord
https://learn.modernagecoders.com/coding-classes-in-fermanagh-and-omagh
https://learn.modernagecoders.com/coding-classes-in-fife
https://learn.modernagecoders.com/coding-classes-in-fingal
https://learn.modernagecoders.com/coding-classes-in-finglas-dublin
https://learn.modernagecoders.com/coding-classes-in-flevoland
https://learn.modernagecoders.com/coding-classes-in-flintshire
https://learn.modernagecoders.com/coding-classes-in-friesland
https://learn.modernagecoders.com/coding-classes-in-gadong
https://learn.modernagecoders.com/coding-classes-in-gelderland
https://learn.modernagecoders.com/coding-classes-in-genexx-valley-joka
https://learn.modernagecoders.com/coding-classes-in-georgia
https://learn.modernagecoders.com/coding-classes-in-germany
https://learn.modernagecoders.com/coding-classes-in-gestel
https://learn.modernagecoders.com/coding-classes-in-glanmire-cork
https://learn.modernagecoders.com/coding-classes-in-glasnevin-dublin
https://learn.modernagecoders.com/coding-classes-in-gloucestershire
https://learn.modernagecoders.com/coding-classes-in-goa
https://learn.modernagecoders.com/coding-classes-in-godrej-prakriti
https://learn.modernagecoders.com/coding-classes-in-gouda
https://learn.modernagecoders.com/coding-classes-in-greater-manchester
https://learn.modernagecoders.com/coding-classes-in-greenfield-city-behala
https://learn.modernagecoders.com/coding-classes-in-greenwich-london
https://learn.modernagecoders.com/coding-classes-in-groningen
https://learn.modernagecoders.com/coding-classes-in-groningen-province
https://learn.modernagecoders.com/coding-classes-in-gujarat
https://learn.modernagecoders.com/coding-classes-in-gwynedd
https://learn.modernagecoders.com/coding-classes-in-haagse-hout
https://learn.modernagecoders.com/coding-classes-in-haarlem
https://learn.modernagecoders.com/coding-classes-in-haarlemmermeer
https://learn.modernagecoders.com/coding-classes-in-hackney-london
https://learn.modernagecoders.com/coding-classes-in-haima
https://learn.modernagecoders.com/coding-classes-in-hammersmith-and-fulham-london
https://learn.modernagecoders.com/coding-classes-in-hampshire
https://learn.modernagecoders.com/coding-classes-in-haringey-london
https://learn.modernagecoders.com/coding-classes-in-harrow-london
https://learn.modernagecoders.com/coding-classes-in-haryana
https://learn.modernagecoders.com/coding-classes-in-havering-london
https://learn.modernagecoders.com/coding-classes-in-heerlen
https://learn.modernagecoders.com/coding-classes-in-helmond
https://learn.modernagecoders.com/coding-classes-in-hengelo
https://learn.modernagecoders.com/coding-classes-in-herefordshire
https://learn.modernagecoders.com/coding-classes-in-hertfordshire
https://learn.modernagecoders.com/coding-classes-in-highland
https://learn.modernagecoders.com/coding-classes-in-hiland-park
https://learn.modernagecoders.com/coding-classes-in-hiland-willows
https://learn.modernagecoders.com/coding-classes-in-hillegersberg-schiebroek
https://learn.modernagecoders.com/coding-classes-in-hillingdon-london
https://learn.modernagecoders.com/coding-classes-in-hilversum
https://learn.modernagecoders.com/coding-classes-in-himachal-pradesh
https://learn.modernagecoders.com/coding-classes-in-hong-kong
https://learn.modernagecoders.com/coding-classes-in-hoorn
https://learn.modernagecoders.com/coding-classes-in-hounslow-london
https://learn.modernagecoders.com/coding-classes-in-howth-dublin
https://learn.modernagecoders.com/coding-classes-in-ibra
https://learn.modernagecoders.com/coding-classes-in-ibri
https://learn.modernagecoders.com/coding-classes-in-ideal-niketan-tangra
https://learn.modernagecoders.com/coding-classes-in-ijsselmonde
https://learn.modernagecoders.com/coding-classes-in-illinois
https://learn.modernagecoders.com/coding-classes-in-india
https://learn.modernagecoders.com/coding-classes-in-inverclyde
https://learn.modernagecoders.com/coding-classes-in-ireland
https://learn.modernagecoders.com/coding-classes-in-isle-of-wight
https://learn.modernagecoders.com/coding-classes-in-islington-london
https://learn.modernagecoders.com/coding-classes-in-izki

https://learn.modernagecoders.com/coding-classes-in-jalan-bani-bu-ali
https://learn.modernagecoders.com/coding-classes-in-jalan-bani-bu-hassan
https://learn.modernagecoders.com/coding-classes-in-jeddah
https://learn.modernagecoders.com/coding-classes-in-jerudong
https://learn.modernagecoders.com/coding-classes-in-jharkhand
https://learn.modernagecoders.com/coding-classes-in-karnataka
https://learn.modernagecoders.com/coding-classes-in-kensington-and-chelsea-london
https://learn.modernagecoders.com/coding-classes-in-kent
https://learn.modernagecoders.com/coding-classes-in-kerala
https://learn.modernagecoders.com/coding-classes-in-khasab
https://learn.modernagecoders.com/coding-classes-in-kingston-upon-thames-london
https://learn.modernagecoders.com/coding-classes-in-knocknacarra-galway
https://learn.modernagecoders.com/coding-classes-in-kralingen-crooswijk
https://learn.modernagecoders.com/coding-classes-in-kuala-belait
https://learn.modernagecoders.com/coding-classes-in-kuwait
https://learn.modernagecoders.com/coding-classes-in-laak
https://learn.modernagecoders.com/coding-classes-in-lambeth-london
https://learn.modernagecoders.com/coding-classes-in-lancashire
https://learn.modernagecoders.com/coding-classes-in-leeuwarden
https://learn.modernagecoders.com/coding-classes-in-leicester
https://learn.modernagecoders.com/coding-classes-in-leicestershire
https://learn.modernagecoders.com/coding-classes-in-leiden
https://learn.modernagecoders.com/coding-classes-in-leidschendam-voorburg
https://learn.modernagecoders.com/coding-classes-in-leidschenveen-ypenburg
https://learn.modernagecoders.com/coding-classes-in-lelystad
https://learn.modernagecoders.com/coding-classes-in-lewisham-london
https://learn.modernagecoders.com/coding-classes-in-liang
https://learn.modernagecoders.com/coding-classes-in-limburg
https://learn.modernagecoders.com/coding-classes-in-lincolnshire
https://learn.modernagecoders.com/coding-classes-in-liwa
https://learn.modernagecoders.com/coding-classes-in-loosduinen
https://learn.modernagecoders.com/coding-classes-in-lucan-dublin
https://learn.modernagecoders.com/coding-classes-in-maastricht
https://learn.modernagecoders.com/coding-classes-in-madha
https://learn.modernagecoders.com/coding-classes-in-madhya-pradesh
https://learn.modernagecoders.com/coding-classes-in-madinat-al-irfan
https://learn.modernagecoders.com/coding-classes-in-madinat-al-sultan-qaboos
https://learn.modernagecoders.com/coding-classes-in-maharashtra
https://learn.modernagecoders.com/coding-classes-in-mahdah
https://learn.modernagecoders.com/coding-classes-in-mahout
https://learn.modernagecoders.com/coding-classes-in-manah
https://learn.modernagecoders.com/coding-classes-in-manitoba
https://learn.modernagecoders.com/coding-classes-in-maryland
https://learn.modernagecoders.com/coding-classes-in-masirah
https://learn.modernagecoders.com/coding-classes-in-mawaleh
https://learn.modernagecoders.com/coding-classes-in-meierijstad
https://learn.modernagecoders.com/coding-classes-in-merseyside
https://learn.modernagecoders.com/coding-classes-in-merthyr-tydfil-county-borough
https://learn.modernagecoders.com/coding-classes-in-merton-london
https://learn.modernagecoders.com/coding-classes-in-mid-and-east-antrim
https://learn.modernagecoders.com/coding-classes-in-mid-ulster
https://learn.modernagecoders.com/coding-classes-in-middelburg
https://learn.modernagecoders.com/coding-classes-in-midlothian
https://learn.modernagecoders.com/coding-classes-in-mirbat
https://learn.modernagecoders.com/coding-classes-in-monmouthshire
https://learn.modernagecoders.com/coding-classes-in-moray
https://learn.modernagecoders.com/coding-classes-in-musandam
https://learn.modernagecoders.com/coding-classes-in-muscat
https://learn.modernagecoders.com/coding-classes-in-muttrah
https://learn.modernagecoders.com/coding-classes-in-na-h-eileanan-siar
https://learn.modernagecoders.com/coding-classes-in-nakhal
https://learn.modernagecoders.com/coding-classes-in-natural-city-garia
https://learn.modernagecoders.com/coding-classes-in-neath-port-talbot
https://learn.modernagecoders.com/coding-classes-in-netherlands
https://learn.modernagecoders.com/coding-classes-in-new-brunswick
https://learn.modernagecoders.com/coding-classes-in-new-jersey
https://learn.modernagecoders.com/coding-classes-in-new-york
https://learn.modernagecoders.com/coding-classes-in-new-zealand
https://learn.modernagecoders.com/coding-classes-in-newfoundland-and-labrador
https://learn.modernagecoders.com/coding-classes-in-newham-london
https://learn.modernagecoders.com/coding-classes-in-nijmegen
https://learn.modernagecoders.com/coding-classes-in-nissewaard
https://learn.modernagecoders.com/coding-classes-in-nizwa
https://learn.modernagecoders.com/coding-classes-in-noord-brabant
https://learn.modernagecoders.com/coding-classes-in-noord-holland
https://learn.modernagecoders.com/coding-classes-in-norfolk
https://learn.modernagecoders.com/coding-classes-in-north-ayrshire
https://learn.modernagecoders.com/coding-classes-in-north-lanarkshire
https://learn.modernagecoders.com/coding-classes-in-north-yorkshire

https://learn.modernagecoders.com/coding-classes-in-northamptonshire
https://learn.modernagecoders.com/coding-classes-in-northumberland
https://learn.modernagecoders.com/coding-classes-in-northwest-territories
https://learn.modernagecoders.com/coding-classes-in-nottinghamshire
https://learn.modernagecoders.com/coding-classes-in-nova-scotia
https://learn.modernagecoders.com/coding-classes-in-odisha
https://learn.modernagecoders.com/coding-classes-in-oegstgeest
https://learn.modernagecoders.com/coding-classes-in-oman
https://learn.modernagecoders.com/coding-classes-in-ontario
https://learn.modernagecoders.com/coding-classes-in-orkney
https://learn.modernagecoders.com/coding-classes-in-oss
https://learn.modernagecoders.com/coding-classes-in-overijssel
https://learn.modernagecoders.com/coding-classes-in-oxfordshire
https://learn.modernagecoders.com/coding-classes-in-pembrokeshire
https://learn.modernagecoders.com/coding-classes-in-perth-and-kinross
https://learn.modernagecoders.com/coding-classes-in-powys
https://learn.modernagecoders.com/coding-classes-in-prins-alexander
https://learn.modernagecoders.com/coding-classes-in-punjab
https://learn.modernagecoders.com/coding-classes-in-purmerend
https://learn.modernagecoders.com/coding-classes-in-qatar
https://learn.modernagecoders.com/coding-classes-in-quebec
https://learn.modernagecoders.com/coding-classes-in-qurayyat
https://learn.modernagecoders.com/coding-classes-in-qurum
https://learn.modernagecoders.com/coding-classes-in-rajasthan
https://learn.modernagecoders.com/coding-classes-in-rathfarnham-dublin
https://learn.modernagecoders.com/coding-classes-in-rathmines-dublin
https://learn.modernagecoders.com/coding-classes-in-redbridge-london
https://learn.modernagecoders.com/coding-classes-in-renfrewshire
https://learn.modernagecoders.com/coding-classes-in-rhondda-cynon-taf
https://learn.modernagecoders.com/coding-classes-in-richmond-upon-thames-london
https://learn.modernagecoders.com/coding-classes-in-rijswijk
https://learn.modernagecoders.com/coding-classes-in-riyadh
https://learn.modernagecoders.com/coding-classes-in-roermond
https://learn.modernagecoders.com/coding-classes-in-roosendaal
https://learn.modernagecoders.com/coding-classes-in-rosedale-garden
https://learn.modernagecoders.com/coding-classes-in-rotterdam
https://learn.modernagecoders.com/coding-classes-in-rotterdam-centrum
https://learn.modernagecoders.com/coding-classes-in-rustaq
https://learn.modernagecoders.com/coding-classes-in-rutland
https://learn.modernagecoders.com/coding-classes-in-saham
https://learn.modernagecoders.com/coding-classes-in-salalah
https://learn.modernagecoders.com/coding-classes-in-samail
https://learn.modernagecoders.com/coding-classes-in-sandyford-dublin
https://learn.modernagecoders.com/coding-classes-in-saskatchewan
https://learn.modernagecoders.com/coding-classes-in-saudi-arabia
https://learn.modernagecoders.com/coding-classes-in-scheveningen
https://learn.modernagecoders.com/coding-classes-in-schiedam
https://learn.modernagecoders.com/coding-classes-in-scottish-borders
https://learn.modernagecoders.com/coding-classes-in-seeb
https://learn.modernagecoders.com/coding-classes-in-segbroek
https://learn.modernagecoders.com/coding-classes-in-seria
https://learn.modernagecoders.com/coding-classes-in-shalim
https://learn.modernagecoders.com/coding-classes-in-sharjah
https://learn.modernagecoders.com/coding-classes-in-sherwood-estate
https://learn.modernagecoders.com/coding-classes-in-shetland
https://learn.modernagecoders.com/coding-classes-in-shinas
https://learn.modernagecoders.com/coding-classes-in-shropshire
https://learn.modernagecoders.com/coding-classes-in-siddha-town-madhyamgram
https://learn.modernagecoders.com/coding-classes-in-singapore
https://learn.modernagecoders.com/coding-classes-in-sittard-geleen
https://learn.modernagecoders.com/coding-classes-in-sohar
https://learn.modernagecoders.com/coding-classes-in-somerset
https://learn.modernagecoders.com/coding-classes-in-south-ayrshire
https://learn.modernagecoders.com/coding-classes-in-south-city
https://learn.modernagecoders.com/coding-classes-in-south-dublin
https://learn.modernagecoders.com/coding-classes-in-south-lanarkshire
https://learn.modernagecoders.com/coding-classes-in-south-yorkshire
https://learn.modernagecoders.com/coding-classes-in-southwark-london
https://learn.modernagecoders.com/coding-classes-in-staffordshire
https://learn.modernagecoders.com/coding-classes-in-stillorgan-dublin
https://learn.modernagecoders.com/coding-classes-in-stratum
https://learn.modernagecoders.com/coding-classes-in-strijp
https://learn.modernagecoders.com/coding-classes-in-suffolk
https://learn.modernagecoders.com/coding-classes-in-sugam-habitat
https://learn.modernagecoders.com/coding-classes-in-sukhobrishti
https://learn.modernagecoders.com/coding-classes-in-sur
https://learn.modernagecoders.com/coding-classes-in-surrey
https://learn.modernagecoders.com/coding-classes-in-sutton-london
https://learn.modernagecoders.com/coding-classes-in-sweden

https://learn.modernagecoders.com/coding-classes-in-switzerland
https://learn.modernagecoders.com/coding-classes-in-tallaght-dublin
https://learn.modernagecoders.com/coding-classes-in-tamil-nadu
https://learn.modernagecoders.com/coding-classes-in-taqah
https://learn.modernagecoders.com/coding-classes-in-telangana
https://learn.modernagecoders.com/coding-classes-in-temburong
https://learn.modernagecoders.com/coding-classes-in-terenure-dublin
https://learn.modernagecoders.com/coding-classes-in-texas
https://learn.modernagecoders.com/coding-classes-in-the-city-of-london
https://learn.modernagecoders.com/coding-classes-in-the-hague
https://learn.modernagecoders.com/coding-classes-in-the-hague-centrum
https://learn.modernagecoders.com/coding-classes-in-the-west-midlands
https://learn.modernagecoders.com/coding-classes-in-thumrait
https://learn.modernagecoders.com/coding-classes-in-tilburg
https://learn.modernagecoders.com/coding-classes-in-tongelre
https://learn.modernagecoders.com/coding-classes-in-torfaen
https://learn.modernagecoders.com/coding-classes-in-tower-hamlets-london
https://learn.modernagecoders.com/coding-classes-in-tutong
https://learn.modernagecoders.com/coding-classes-in-tyne-and-wear
https://learn.modernagecoders.com/coding-classes-in-uae
https://learn.modernagecoders.com/coding-classes-in-udayan-condoville-em-bypass
https://learn.modernagecoders.com/coding-classes-in-uithoorn
https://learn.modernagecoders.com/coding-classes-in-united-kingdom
https://learn.modernagecoders.com/coding-classes-in-united-states
https://learn.modernagecoders.com/coding-classes-in-uniworld-city
https://learn.modernagecoders.com/coding-classes-in-upohar
https://learn.modernagecoders.com/coding-classes-in-urbana
https://learn.modernagecoders.com/coding-classes-in-utrecht
https://learn.modernagecoders.com/coding-classes-in-utrecht-province
https://learn.modernagecoders.com/coding-classes-in-uttar-pradesh
https://learn.modernagecoders.com/coding-classes-in-uttarakhand
https://learn.modernagecoders.com/coding-classes-in-vale-of-glamorgan
https://learn.modernagecoders.com/coding-classes-in-veenendaal
https://learn.modernagecoders.com/coding-classes-in-velsen
https://learn.modernagecoders.com/coding-classes-in-venlo
https://learn.modernagecoders.com/coding-classes-in-virginia
https://learn.modernagecoders.com/coding-classes-in-vlaardingen
https://learn.modernagecoders.com/coding-classes-in-voorschoten
https://learn.modernagecoders.com/coding-classes-in-wadi-al-maawil
https://learn.modernagecoders.com/coding-classes-in-wadi-bani-khalid
https://learn.modernagecoders.com/coding-classes-in-waltham-forest-london
https://learn.modernagecoders.com/coding-classes-in-wandsworth-london
https://learn.modernagecoders.com/coding-classes-in-warwickshire
https://learn.modernagecoders.com/coding-classes-in-washington
https://learn.modernagecoders.com/coding-classes-in-wassenaar
https://learn.modernagecoders.com/coding-classes-in-weesp
https://learn.modernagecoders.com/coding-classes-in-west-bengal
https://learn.modernagecoders.com/coding-classes-in-west-dunbartonshire
https://learn.modernagecoders.com/coding-classes-in-west-lothian
https://learn.modernagecoders.com/coding-classes-in-west-sussex
https://learn.modernagecoders.com/coding-classes-in-west-yorkshire
https://learn.modernagecoders.com/coding-classes-in-westland
https://learn.modernagecoders.com/coding-classes-in-westminster-london
https://learn.modernagecoders.com/coding-classes-in-wiltshire
https://learn.modernagecoders.com/coding-classes-in-woensel-noord
https://learn.modernagecoders.com/coding-classes-in-woensel-zuid
https://learn.modernagecoders.com/coding-classes-in-worcestershire
https://learn.modernagecoders.com/coding-classes-in-yanqul
https://learn.modernagecoders.com/coding-classes-in-zaanstad
https://learn.modernagecoders.com/coding-classes-in-zeeland
https://learn.modernagecoders.com/coding-classes-in-zoetermeer
https://learn.modernagecoders.com/coding-classes-in-zuid-holland
https://learn.modernagecoders.com/coding-classes-in-zwolle

SCHOOL PAGES (60 pages)

https://learn.modernagecoders.com/coding-and-maths-for-birla-high-school-kolkata
https://learn.modernagecoders.com/coding-and-maths-for-birla-high-school-mukundapur-kolkata
https://learn.modernagecoders.com/coding-and-maths-for-don-bosco-park-circus-kolkata
https://learn.modernagecoders.com/coding-and-maths-for-heritage-school-kolkata
https://learn.modernagecoders.com/coding-and-maths-for-mahadevi-birla-world-academy-kolkata
https://learn.modernagecoders.com/coding-and-maths-for-sushila-birla-girls-school-kolkata
https://learn.modernagecoders.com/coding-classes-for-11-and-12-year-olds-australia
https://learn.modernagecoders.com/coding-classes-for-5-and-6-year-olds-australia
https://learn.modernagecoders.com/coding-classes-for-7-and-8-year-olds-australia
https://learn.modernagecoders.com/coding-classes-for-9-and-10-year-olds-australia
https://learn.modernagecoders.com/coding-classes-for-adults
https://learn.modernagecoders.com/coding-classes-for-adults-australia
https://learn.modernagecoders.com/coding-classes-for-adults-in-canada

https://learn.modernagecoders.com/coding-classes-for-dps-bangalore
https://learn.modernagecoders.com/coding-classes-for-dps-dwarka-new-delhi
https://learn.modernagecoders.com/coding-classes-for-dps-greater-noida
https://learn.modernagecoders.com/coding-classes-for-dps-gurgaon
https://learn.modernagecoders.com/coding-classes-for-dps-hyderabad
https://learn.modernagecoders.com/coding-classes-for-dps-international-saket-delhi
https://learn.modernagecoders.com/coding-classes-for-dps-navi-mumbai
https://learn.modernagecoders.com/coding-classes-for-dps-new-town-kolkata
https://learn.modernagecoders.com/coding-classes-for-dps-noida
https://learn.modernagecoders.com/coding-classes-for-dps-rk-puram-new-delhi
https://learn.modernagecoders.com/coding-classes-for-dps-rohini-new-delhi
https://learn.modernagecoders.com/coding-classes-for-dps-ruby-park-kolkata
https://learn.modernagecoders.com/coding-classes-for-dps-vasant-kunj-new-delhi
https://learn.modernagecoders.com/coding-classes-for-elementary-students
https://learn.modernagecoders.com/coding-classes-for-girls
https://learn.modernagecoders.com/coding-classes-for-high-school-students
https://learn.modernagecoders.com/coding-classes-for-kids-adelaide
https://learn.modernagecoders.com/coding-classes-for-kids-brisbane
https://learn.modernagecoders.com/coding-classes-for-kids-canberra
https://learn.modernagecoders.com/coding-classes-for-kids-cost
https://learn.modernagecoders.com/coding-classes-for-kids-gold-coast
https://learn.modernagecoders.com/coding-classes-for-kids-in-abbotsford
https://learn.modernagecoders.com/coding-classes-for-kids-in-ajax-and-pickering
https://learn.modernagecoders.com/coding-classes-for-kids-in-cambridge-ontario
https://learn.modernagecoders.com/coding-classes-for-kids-in-canada
https://learn.modernagecoders.com/coding-classes-for-kids-in-doha
https://learn.modernagecoders.com/coding-classes-for-kids-in-dubai
https://learn.modernagecoders.com/coding-classes-for-kids-in-etobicoke-toronto
https://learn.modernagecoders.com/coding-classes-for-kids-in-langley-bc
https://learn.modernagecoders.com/coding-classes-for-kids-in-nassau
https://learn.modernagecoders.com/coding-classes-for-kids-in-north-york-toronto
https://learn.modernagecoders.com/coding-classes-for-kids-in-oshawa-and-whitby
https://learn.modernagecoders.com/coding-classes-for-kids-in-riyadh
https://learn.modernagecoders.com/coding-classes-for-kids-in-scarborough-toronto
https://learn.modernagecoders.com/coding-classes-for-kids-in-st-catharines-niagara
https://learn.modernagecoders.com/coding-classes-for-kids-melbourne
https://learn.modernagecoders.com/coding-classes-for-kids-netherlands
https://learn.modernagecoders.com/coding-classes-for-kids-newcastle
https://learn.modernagecoders.com/coding-classes-for-kids-perth
https://learn.modernagecoders.com/coding-classes-for-kids-sydney
https://learn.modernagecoders.com/coding-classes-for-la-martiniere-boys-kolkata
https://learn.modernagecoders.com/coding-classes-for-la-martiniere-girls-kolkata
https://learn.modernagecoders.com/coding-classes-for-middle-school-students
https://learn.modernagecoders.com/coding-classes-for-seniors-in-canada
https://learn.modernagecoders.com/coding-classes-for-teens-australia
https://learn.modernagecoders.com/coding-classes-for-teens-in-canada
https://learn.modernagecoders.com/coding-classes-for-teens-netherlands

PAGES BY THE CHILD'S AGE (19 pages)

https://learn.modernagecoders.com/coding-for-10-year-olds
https://learn.modernagecoders.com/coding-for-11-year-olds
https://learn.modernagecoders.com/coding-for-12-year-olds
https://learn.modernagecoders.com/coding-for-13-year-olds
https://learn.modernagecoders.com/coding-for-14-year-olds
https://learn.modernagecoders.com/coding-for-15-year-olds
https://learn.modernagecoders.com/coding-for-16-year-olds
https://learn.modernagecoders.com/coding-for-17-year-olds
https://learn.modernagecoders.com/coding-for-8-year-olds
https://learn.modernagecoders.com/coding-for-9-year-olds
https://learn.modernagecoders.com/python-for-10-year-olds
https://learn.modernagecoders.com/python-for-11-year-olds
https://learn.modernagecoders.com/python-for-12-year-olds
https://learn.modernagecoders.com/python-for-13-year-olds
https://learn.modernagecoders.com/python-for-14-year-olds
https://learn.modernagecoders.com/python-for-15-year-olds
https://learn.modernagecoders.com/python-for-16-year-olds
https://learn.modernagecoders.com/python-for-17-year-olds
https://learn.modernagecoders.com/python-for-9-year-olds

PAGES BY SCHOOL CLASS (17 pages)

https://learn.modernagecoders.com/coding-for-class-10
https://learn.modernagecoders.com/coding-for-class-11
https://learn.modernagecoders.com/coding-for-class-12
https://learn.modernagecoders.com/coding-for-class-3
https://learn.modernagecoders.com/coding-for-class-4
https://learn.modernagecoders.com/coding-for-class-5
https://learn.modernagecoders.com/coding-for-class-6

https://learn.modernagecoders.com/coding-for-class-7
https://learn.modernagecoders.com/coding-for-class-8
https://learn.modernagecoders.com/coding-for-class-9
https://learn.modernagecoders.com/python-for-class-10
https://learn.modernagecoders.com/python-for-class-11-cbse
https://learn.modernagecoders.com/python-for-class-12-cbse
https://learn.modernagecoders.com/python-for-class-6
https://learn.modernagecoders.com/python-for-class-7
https://learn.modernagecoders.com/python-for-class-8
https://learn.modernagecoders.com/python-for-class-9

BLOG ARTICLES (176)

https://learn.modernagecoders.com/blog/30-best-java-programs-for-class-8-students
https://learn.modernagecoders.com/blog/30-plus-scratch-project-ideas-kids-fun-coding-beginner-advanced
https://learn.modernagecoders.com/blog/50-java-programs-for-icse-class-10
https://learn.modernagecoders.com/blog/abacus-vs-vedic-vs-mental-maths
https://learn.modernagecoders.com/blog/advantages-functions-python-why-every-coder-should-use-them
https://learn.modernagecoders.com/blog/agentic-terminology-in-ai
https://learn.modernagecoders.com/blog/ai-automation-transforming-small-medium-businesses
https://learn.modernagecoders.com/blog/ap-computer-science-a-changes-2026
https://learn.modernagecoders.com/blog/ap-csp-vs-ap-csa-which-to-take-first
https://learn.modernagecoders.com/blog/app-idea-to-app-store-60-days-teen-developer-roadmap
https://learn.modernagecoders.com/blog/area-and-perimeter-difference
https://learn.modernagecoders.com/blog/armstrong-number-python-complete-guide-examples-code
https://learn.modernagecoders.com/blog/best-ai-coding-tools-for-students-2026
https://learn.modernagecoders.com/blog/best-byjus-alternative
https://learn.modernagecoders.com/blog/best-coding-languages-kids-10-18-beginner-advanced
https://learn.modernagecoders.com/blog/best-free-paid-coding-courses-students
https://learn.modernagecoders.com/blog/best-free-platforms-for-vibe-coding
https://learn.modernagecoders.com/blog/best-java-ides-for-beginners
https://learn.modernagecoders.com/blog/best-programming-languages-college-students-2026
https://learn.modernagecoders.com/blog/big-o-notation-explained-simply
https://learn.modernagecoders.com/blog/binary-numbers-explained
https://learn.modernagecoders.com/blog/bodmas-rule-explained
https://learn.modernagecoders.com/blog/cambridge-9618-paper-4-python-java-or-visual-basic
https://learn.modernagecoders.com/blog/canadian-computing-competition-junior-or-senior
https://learn.modernagecoders.com/blog/career-transition-guide-moving-non-tech-to-tech-roles
https://learn.modernagecoders.com/blog/cbse-icse-gap-what-schools-not-teaching-about-tech
https://learn.modernagecoders.com/blog/cbse-maths-tuition-online
https://learn.modernagecoders.com/blog/cloud-computing-explained-business-owners-without-tech-jargon
https://learn.modernagecoders.com/blog/coding-class-fees-india-2026
https://learn.modernagecoders.com/blog/coding-for-5-year-olds-young-kids-start-learning-programming
https://learn.modernagecoders.com/blog/coding-for-6-year-olds
https://learn.modernagecoders.com/blog/coding-for-7-year-olds-what-how-kids-should-learn-programming
https://learn.modernagecoders.com/blog/coding-for-kids-guide
https://learn.modernagecoders.com/blog/coding-for-kids-parents-complete-guide-2026
https://learn.modernagecoders.com/blog/coding-interview-preparation-for-freshers
https://learn.modernagecoders.com/blog/coding-is-the-new-english-why-2026-is-the-year-to-start
https://learn.modernagecoders.com/blog/coding-mathematics-engineering-science
https://learn.modernagecoders.com/blog/coding-vs-ai-what-should-my-child-learn
https://learn.modernagecoders.com/blog/common-myths-about-coding-for-school-students
https://learn.modernagecoders.com/blog/competitive-coding-vs-development-skills-what-students-focus
https://learn.modernagecoders.com/blog/conways-game-of-life-python
https://learn.modernagecoders.com/blog/custom-software-vs-ready-made-software
https://learn.modernagecoders.com/blog/data-analytics-math-help-businesses-better-decisions
https://learn.modernagecoders.com/blog/data-science-ai-math-skills-professionals
https://learn.modernagecoders.com/blog/data-science-course-india
https://learn.modernagecoders.com/blog/dijkstras-algorithm-fastest-route
https://learn.modernagecoders.com/blog/do-ai-detectors-work
https://learn.modernagecoders.com/blog/does-online-maths-tuition-work
https://learn.modernagecoders.com/blog/estimation-in-maths
https://learn.modernagecoders.com/blog/exception-handling-in-python
https://learn.modernagecoders.com/blog/fibonacci-series-in-python
https://learn.modernagecoders.com/blog/file-built-in-methods-python-guide
https://learn.modernagecoders.com/blog/file-organization-in-python
https://learn.modernagecoders.com/blog/file-types-in-python-complete-guide
https://learn.modernagecoders.com/blog/full-stack-vs-ai-engineer-salary-india
https://learn.modernagecoders.com/blog/git-vs-github-difference
https://learn.modernagecoders.com/blog/group-vs-1-on-1-coding-classes
https://learn.modernagecoders.com/blog/help-child-with-maths-anxiety
https://learn.modernagecoders.com/blog/help-child-with-maths-homework
https://learn.modernagecoders.com/blog/how-are-passwords-stored
https://learn.modernagecoders.com/blog/how-coding-education-changing-future-india
https://learn.modernagecoders.com/blog/how-coding-improves-mathematical-thinking-children
https://learn.modernagecoders.com/blog/how-college-students-build-real-world-projects-while-studying
https://learn.modernagecoders.com/blog/how-do-computers-store-images
https://learn.modernagecoders.com/blog/how-do-search-engines-work
https://learn.modernagecoders.com/blog/how-do-spam-filters-work

https://learn.modernagecoders.com/blog/how-does-encryption-work
https://learn.modernagecoders.com/blog/how-does-file-compression-work
https://learn.modernagecoders.com/blog/how-does-gps-work
https://learn.modernagecoders.com/blog/how-long-does-it-take-to-learn-python
https://learn.modernagecoders.com/blog/how-schools-parents-prepare-children-future-tech-careers
https://learn.modernagecoders.com/blog/how-to-build-menu-driven-program-in-python
https://learn.modernagecoders.com/blog/how-to-calculate-percentages
https://learn.modernagecoders.com/blog/how-to-choose-a-coding-class-for-your-child
https://learn.modernagecoders.com/blog/how-to-explain-division-to-a-child
https://learn.modernagecoders.com/blog/how-to-find-hcf-and-lcm-in-python
https://learn.modernagecoders.com/blog/how-to-get-a-5-in-ap-computer-science-a
https://learn.modernagecoders.com/blog/how-to-get-better-at-maths
https://learn.modernagecoders.com/blog/how-to-help-a-child-understand-fractions
https://learn.modernagecoders.com/blog/how-to-learn-times-tables-fast
https://learn.modernagecoders.com/blog/how-to-make-a-text-adventure-game-in-python
https://learn.modernagecoders.com/blog/how-to-make-maths-fun-for-kids
https://learn.modernagecoders.com/blog/how-to-read-and-write-files-in-python
https://learn.modernagecoders.com/blog/how-to-reverse-a-string-in-python
https://learn.modernagecoders.com/blog/how-to-revise-gcse-computer-science
https://learn.modernagecoders.com/blog/how-to-solve-maths-word-problems
https://learn.modernagecoders.com/blog/how-to-solve-quadratic-equations
https://learn.modernagecoders.com/blog/how-to-solve-simultaneous-equations
https://learn.modernagecoders.com/blog/how-to-start-learning-data-structures-and-algorithms
https://learn.modernagecoders.com/blog/how-to-teach-kids-ai-at-home
https://learn.modernagecoders.com/blog/how-to-use-chatgpt-to-study-without-cheating
https://learn.modernagecoders.com/blog/how-to-write-a-leap-year-program-in-python
https://learn.modernagecoders.com/blog/how-to-write-a-maths-proof
https://learn.modernagecoders.com/blog/html-css-javascript-project-ideas
https://learn.modernagecoders.com/blog/ib-computer-science-ia-client-rule-2027
https://learn.modernagecoders.com/blog/icse-class-10-computer-applications-revision
https://learn.modernagecoders.com/blog/igcse-computer-science-0478-pseudocode-guide
https://learn.modernagecoders.com/blog/igcse-computer-science-explained-for-parents
https://learn.modernagecoders.com/blog/ioi-syllabus-what-to-learn-and-what-to-skip
https://learn.modernagecoders.com/blog/is-coding-good-for-kids
https://learn.modernagecoders.com/blog/is-coding-worth-learning-2026
https://learn.modernagecoders.com/blog/is-vedic-maths-actually-useful
https://learn.modernagecoders.com/blog/java-array-programs-for-beginners
https://learn.modernagecoders.com/blog/java-constructors-explained
https://learn.modernagecoders.com/blog/java-for-kids-complete-guide-learning-programming-2026
https://learn.modernagecoders.com/blog/javascript-basics
https://learn.modernagecoders.com/blog/javascript-to-learn-before-react
https://learn.modernagecoders.com/blog/learn-python-step-by-step
https://learn.modernagecoders.com/blog/learn-to-code-career-change-2026
https://learn.modernagecoders.com/blog/logic-gates-for-beginners
https://learn.modernagecoders.com/blog/long-division-step-by-step
https://learn.modernagecoders.com/blog/mean-median-mode-explained
https://learn.modernagecoders.com/blog/mental-maths-tricks-for-kids
https://learn.modernagecoders.com/blog/minecraft-vs-roblox-which-teaches-kids-to-code
https://learn.modernagecoders.com/blog/mobile-apps-increase-customer-engagement-businesses
https://learn.modernagecoders.com/blog/modern-age-coders-certificate-earned-not-given
https://learn.modernagecoders.com/blog/modular-arithmetic-explained
https://learn.modernagecoders.com/blog/negative-numbers-explained
https://learn.modernagecoders.com/blog/outsourcing-software-development-vs-hiring-inhouse-developers
https://learn.modernagecoders.com/blog/parents-guide-starting-coding-education-home
https://learn.modernagecoders.com/blog/pcep-vs-pcap-which-python-certification
https://learn.modernagecoders.com/blog/pigeonhole-principle-explained
https://learn.modernagecoders.com/blog/place-value-explained
https://learn.modernagecoders.com/blog/prime-numbers-explained
https://learn.modernagecoders.com/blog/probability-for-beginners
https://learn.modernagecoders.com/blog/project-based-learning-why-theory-alone-not-enough-coding
https://learn.modernagecoders.com/blog/pythagoras-theorem-explained
https://learn.modernagecoders.com/blog/python-basic-programs-for-beginners
https://learn.modernagecoders.com/blog/python-dictionary-complete-guide
https://learn.modernagecoders.com/blog/python-for-beginners
https://learn.modernagecoders.com/blog/python-lambda-functions
https://learn.modernagecoders.com/blog/python-list-comprehension-explained
https://learn.modernagecoders.com/blog/python-oop-tutorial-for-beginners
https://learn.modernagecoders.com/blog/python-programs-for-cbse-class-12
https://learn.modernagecoders.com/blog/python-vs-java-which-to-learn-first
https://learn.modernagecoders.com/blog/python-vs-javascript-which-to-learn-first
https://learn.modernagecoders.com/blog/recursion-explained-for-beginners
https://learn.modernagecoders.com/blog/robotics-coding-stem-kits-build-logical-thinking
https://learn.modernagecoders.com/blog/role-of-mathematics-programming-logical-problem-solving
https://learn.modernagecoders.com/blog/safe-ai-tools-for-kids-homework-2025
https://learn.modernagecoders.com/blog/scratch-vs-python-which-better-young-learners
https://learn.modernagecoders.com/blog/signs-your-child-is-ready-to-learn-coding
https://learn.modernagecoders.com/blog/sohcahtoa-trigonometry-for-beginners
https://learn.modernagecoders.com/blog/sql-or-python-first
https://learn.modernagecoders.com/blog/star-pattern-programs-in-python

https://learn.modernagecoders.com/blog/stem-education-for-girls-india
https://learn.modernagecoders.com/blog/string-handling-in-java
https://learn.modernagecoders.com/blog/top-10-java-programs-every-college-student-must-know
https://learn.modernagecoders.com/blog/top-10-java-programs-for-school-students
https://learn.modernagecoders.com/blog/top-20-java-programs-for-icse-class-10
https://learn.modernagecoders.com/blog/top-coding-games-platforms-make-learning-fun-kids
https://learn.modernagecoders.com/blog/top-emerging-tech-skills-learn-2025-ai-low-code-cloud
https://learn.modernagecoders.com/blog/touch-typing-for-kids
https://learn.modernagecoders.com/blog/uae-ai-curriculum-schools-what-it-means
https://learn.modernagecoders.com/blog/usaco-bronze-to-silver-what-blocks-most-students
https://learn.modernagecoders.com/blog/vibe-coding-vs-learning-to-code-properly
https://learn.modernagecoders.com/blog/what-is-ai-complete-beginners-guide-how-to-start
https://learn.modernagecoders.com/blog/what-is-an-algorithm
https://learn.modernagecoders.com/blog/what-is-an-api-explained-simply
https://learn.modernagecoders.com/blog/what-is-blockly-for-kids
https://learn.modernagecoders.com/blog/what-is-floor-division-in-python
https://learn.modernagecoders.com/blog/what-is-the-responsibility-of-developers-using-generative-AI
https://learn.modernagecoders.com/blog/what-is-vibe-coding-future-of-software-development
https://learn.modernagecoders.com/blog/what-type-of-data-is-generative-ai-most-suitable-for
https://learn.modernagecoders.com/blog/whitehat-jr-alternative
https://learn.modernagecoders.com/blog/who-are-modern-age-coders-why-choose-us
https://learn.modernagecoders.com/blog/why-can-i-understand-solutions-but-not-solve-problems
https://learn.modernagecoders.com/blog/why-coding-is-new-literacy-kids-2025
https://learn.modernagecoders.com/blog/why-do-machine-learning-models-make-mistakes
https://learn.modernagecoders.com/blog/why-early-exposure-coding-gives-students-competitive-advantage
https://learn.modernagecoders.com/blog/why-every-business-needs-website-mobile-app-2025
https://learn.modernagecoders.com/blog/why-every-teen-should-learn-coding
https://learn.modernagecoders.com/blog/why-is-0-1-plus-0-2-not-0-3
https://learn.modernagecoders.com/blog/why-is-my-child-struggling-with-maths
https://learn.modernagecoders.com/blog/why-python-most-powerful-skill-professionals-today
https://learn.modernagecoders.com/blog/working-professionals-upskill-coding-without-quitting-job

==
PART 10. GENERAL QUESTIONS AND THE ANSWERS TO GIVE
==

Q: What is Modern Age Coders?
   A: We teach live online coding and mathematics to learners aged 6 to 67. Kids, teens,
   college students, working professionals and corporate teams. We also build software for
   businesses, which is how our advanced students get real project experience.

Q: Does my child need any experience?
   A: No. Our beginner courses start from the very beginning and most of our students start
   with zero experience.

Q: Are the classes live or recorded?
   A: Every class is live with a real teacher. Recordings are provided as well, so a missed
   class can be caught up.

Q: What does my child need at home?
   A: A computer, Windows, Mac or Chromebook, and steady internet. Most courses run in the
   browser so nothing needs installing. A webcam and microphone help, and for maths a notebook
   and pen.

Q: How many classes a week?
   A: Every class is one hour and live. Group class and mini batch: two live classes a week,
   eight a month. One to one: in India one live class a week, four a month; outside India two
   live classes a week, eight a month. Give the person only the line for their own plan and
   region.

Q: What do we get in one to one classes?
   A: A dedicated mentor teaching only your child, in private live one hour classes. In India
   that is one live class a week, four a month. The day and time are fixed around the student,
   and the pace and topics are planned for them alone. It is billed monthly and can be
   cancelled any month.

Q: How big are the batches?
   A: Group classes are capped at 20 students. A mini batch is up to 6. One to one is a single
   student with their own mentor.

Q: How long does a course take?
   A: Beginner courses usually run 3 to 6 months. Advanced ones run 6 to 9 months. Custom
   programmes are flexible.

Q: What ages do you teach?
   A: From 6 to 67. Kids are 6 to 12, teens are 13 to 18, and then college students and adults.

Q: Is there a free class?
   A: Yes, one free demo per student with a real teacher, and we never ask for card details. It

is a waiting list, because our mentors teach live all day, so we cannot promise a date. If you want a class today or tomorrow, the Priority Live Demo is a paid, guaranteed slot: a full class of about 45 to 60 minutes with a written skill report afterwards, and the fee is adjusted against your first month if you enrol. Parents are welcome to sit in on either.

Q: What if we miss a class?
A: Every class is recorded, so it can be watched later. For one to one classes we can often reschedule if you tell us in advance.

Q: Can we pause?
A: Yes, for exams or holidays. Tell us about a week ahead and we hold the spot.

Q: Can we switch from group to one to one later?
A: Yes, any time. Tell us a few days before the next billing date and we adjust it.

Q: Do you give a certificate?
A: Yes, when the course is completed. It lists the skills learned and the projects built.

Q: Can parents see progress?
A: Yes. Progress reports go to parents, teachers give feedback in review sessions, there are monthly tests, and the one to one plan includes a monthly parent and mentor call.

Q: What if my child finds a topic hard?
A: We add extra doubt clearing sessions and one to one help. No student is left behind.

Q: Which boards do you cover?
A: CBSE, ICSE, IGCSE, IB, A Level and GCSE, American Common Core and AP, Singapore MOE, and most major Indian state boards.

Q: Do you prepare for Olympiads and competitive exams?
A: Yes. Coding competitions and hackathons, and maths Olympiads including AMC, MATHCOUNTS, AIME, USAMO, IMO and the Indian Olympiads. Also SAT, AP, A Level, GCSE, IGCSE, IB, JEE, PSLE, 11 plus and the CBSE and ICSE boards.

Q: Are there girls only batches?
A: Yes. Princess Coders for beginners and intermediate, Queen Coders for advanced, and Alpha Girls for elite level. There is also a comeback programme for women returning to tech.

Q: Do you teach AI?
A: Yes. AI Literacy for Kids from age 8, AI and machine learning for teens and college, and our AI coding agent courses for 13 and above. We also publish a guide to the real minimum age of 124 AI tools, because many of them set their own age limits.

Q: Can college students and working adults join?
A: Yes. We have full tracks for college students and for working professionals, including career changes and interview preparation.

Q: Do you do corporate training?
A: Yes, for company teams. The curriculum is built around what the team actually needs. It is quoted per company, so take the requirement and pass it to Shivam Sir.

Q: Where is your office? Can we visit?
A: We are an online school, so every class happens live over video and there is no campus to visit. Our registered office is 1B, JK Ambika Tower, Kolkata, West Bengal 700002.

Q: Are your teachers qualified?
A: Yes. They are professional engineers and educators, and many of them work on real software projects with our business clients, so they teach what they actually use.

Q: Is my child too young at 5?
A: Our programmes start at 6. For a 5 year old, ask Shivam Sir on +91 9123366161. Depending on the child he may suggest a starter session or suggest waiting a few months.

Q: I am 50 or 60. Am I too old?
A: Not at all. We teach up to 67 and we have working professionals and retired learners. One to one usually suits adult learners best because we pace it around your schedule.

Q: Do you help with internships or real projects?
A: Yes. Advanced students get the chance to work on real client projects.

Q: Can I gift a course?
A: Yes. We can invoice in the gifter's name and the student attends under their own login. Ask Shivam Sir to set it up.

Q: Do you have a mobile app?
A: Classes are joined from a computer through the browser, so there is nothing to install. The student dashboard is at dashboard.modernagecoders.com.

Q: Can we join in the middle of the month?

A: Yes. We either place the student in a batch already running or start a new one.

==
PART 11. READY MADE REPLIES
==

FIRST MESSAGE

"Hello, welcome to Modern Age Coders. I can help you with our live coding and maths classes. Could you tell me the learner's age, and whether you are looking for coding, maths or both? I will point you to the right starting place."

WHEN THEY ASK THE PRICE. Give only their own region, never both.

If they are in India:
"Our classes are Rs 1,499 a month for a group class of up to 10, or Rs 2,999 for a mini batch of just up to 6 students, both with two live classes a week, eight a month. Fully one to one is Rs 4,999 a month, which is one live class a week, four a month with a mentor who teaches only your child. Every plan includes recordings, doubt support and a certificate. To see the teaching first, the Priority Live Demo is a full class today or tomorrow for 499 rupees, adjusted against your first month if you join, or you can join the free demo waiting list. Which would you prefer?"

If they are outside India:
"Our classes are $100 a month for a group class and $150 a month for fully one to one, both with two live one hour classes every week, and every plan includes recordings, doubt support and a certificate. To see the teaching first, the Priority Live Demo is a full class today or tomorrow for 10 US dollars, adjusted against your first month if you join, or you can join the free demo waiting list. Which would you prefer?"

Maths and the AI coding agent courses are priced and scheduled the same way. Build AI Agents with Google Gemini Enterprise has its own one to one fee, shown in its course block.

WHEN THEY ASK ABOUT TIMINGS

"We run batches on weekday evenings and on weekend mornings and afternoons, and we have slots for other time zones too. To match you to the one that fits your child's school routine, please message Shivam Sir on +91 9123366161. He arranges every batch himself."

WHEN THEY ASK "IS IT WORTH IT?"

"The honest answer is to see one real class. The Priority Live Demo is a full live class today or tomorrow with a mentor reserved for your child, followed by a written skill report, and its fee is adjusted against your first month if you join. If you would rather not pay anything yet, you can join the free demo waiting list, but we cannot promise a date for that. Which would suit you?"

WHEN THEY ASK FOR A DISCOUNT

"Let me check the current offers with Shivam Sir. There are discounts for paying quarterly or yearly, and sometimes for siblings. Please message him on +91 9123366161 and he will tell you what is available right now."

WHEN THE COST WORRIES THEM

"I understand. Two things may help. There is no lock in, no registration fee and no long package, so you pay one month at a time and can stop any month. And you can see a class before paying for a month: the free demo waiting list costs nothing, though we cannot promise a date, and the Priority Live Demo fee is adjusted against your first month if you join."

CLOSING THE CONVERSATION

"The fastest way to start is the Priority Live Demo, which you can book here: https://learn.modernagecoders.com/priority-demo . If you would rather join the free demo waiting list, I just need three things: the student's name and age, whether you want coding or maths, and your WhatsApp number with country code."

WHEN YOU DO NOT KNOW THE ANSWER

"That is a good question and I want to give you the right answer rather than guess. Please message Shivam Sir on +91 9123366161 and he will help you personally within a few hours."

==
END OF KNOWLEDGE BASE. WHEN IN DOUBT, SEND THEM TO SHIVAM SIR ON +91 9123366161.
==